Tong Xiao
Jingbo Zhu

Xử lý Ngôn ngữ Tự nhiên

Mạng nơ-ron và Mô hình ngôn ngữ lớn

Bản dịch tiếng Việt có chú giải

Natural Language Processing Lab, Northeastern University
&
NiuTrans Research

Nguyên tác: Natural Language Processing: Neural Networks and Large Language Models,
phiên bản ngày 19/10/2026

Nguyên tác: *Natural Language Processing: Neural Networks and Large Language Models*
Copyright © 2021–2026 Tong Xiao và Jingbo Zhu.
Natural Language Processing Lab, Northeastern University & NiuTrans Research.
https://github.com/NiuTrans/NLPBook https://niutrans.github.io/NLPBook

Nguyên tác được phát hành theo giấy phép Creative Commons Ghi công – Phi thương mại 4.0 (CC BY-NC 4.0), http://creativecommons.org/licenses/by-nc/4.0.

Về bản dịch này. Đây là bản dịch tiếng Việt *không chính thức*, phi thương mại, được thực hiện với sự hỗ trợ của mô hình ngôn ngữ lớn (Claude), dựa trên phiên bản PDF ngày 19/10/2026 của nguyên tác. Những thay đổi so với nguyên tác: (1) toàn bộ nội dung được dịch sang tiếng Việt; (2) bổ sung các khung *"Chú giải của người dịch"* và chú thích chân trang đánh dấu **[ND]** nhằm giúp người mới học xong trung học phổ thông theo dõi được; (3) chữ bên trong các hình vẽ được giữ nguyên tiếng Anh như bản gốc, phần chú thích hình đã được dịch; (4) thêm Bảng thuật ngữ Anh–Việt ở cuối sách. Danh mục tài liệu tham khảo và Chỉ mục được giữ nguyên bản gốc. Bản dịch này cũng được chia sẻ theo giấy phép CC BY-NC 4.0; không được dùng cho mục đích thương mại. Mọi sai sót trong bản dịch thuộc về người dịch, không thuộc về các tác giả.

Cuốn sách này xin dành tặng gia đình của chúng tôi.

Lời nói đầu

Xử lý ngôn ngữ tự nhiên (*natural language processing, NLP*) là một trong những lĩnh vực con cốt lõi của trí tuệ nhân tạo (*artificial intelligence, AI*). Trong một thời gian dài, nghiên cứu NLP chủ yếu tập trung giải quyết những bài toán cụ thể về hiểu và sinh ngôn ngữ, chẳng hạn như phân tích cú pháp (*parsing*) và dịch máy (*machine translation*). Cách tiếp cận nghiên cứu "hướng theo bài toán" này đã chi phối sự phát triển của NLP trong nhiều thập kỷ. Tuy nhiên, cùng với sự trỗi dậy của học sâu (*deep learning*), hệ hình nghiên cứu của NLP đã trải qua một cuộc chuyển đổi căn bản. Việc áp dụng mạng nơ-ron (*neural network*) sâu cho phép chúng ta giải quyết những bài toán ngày càng phức tạp. Quan trọng hơn, các nhà nghiên cứu đã phát hiện ra rằng: nếu tiền huấn luyện (*pre-training*) quy mô lớn một mô hình nền tảng trên lượng dữ liệu khổng lồ, rồi tinh chỉnh (*fine-tuning*) nó bằng một ít dữ liệu và tri thức dành riêng cho từng bài toán, thì có thể xây dựng những mô hình đa năng xử lý được nhiều bài toán cùng lúc. Hệ hình mới này đang làm thay đổi mạnh mẽ bức tranh nghiên cứu của NLP, và thậm chí của cả lĩnh vực AI nói chung.

Chú giải của người dịch: Một vài khái niệm mở đầu

- **Hệ hình** (*paradigm*): cách làm, cách suy nghĩ chung mà cả một cộng đồng nghiên cứu cùng theo đuổi trong một giai đoạn.
- **Mô hình** (*model*): trong học máy, "mô hình" là một chương trình máy tính có rất nhiều con số điều chỉnh được (gọi là *tham số*); máy "học" bằng cách tự điều chỉnh các con số này sao cho kết quả đầu ra ngày càng đúng.
- **Tiền huấn luyện rồi tinh chỉnh**: hãy hình dung một học sinh đọc thật nhiều sách đủ mọi lĩnh vực trong nhiều năm (tiền huấn luyện), sau đó chỉ cần học thêm vài tuần về một nghề cụ thể là làm được việc (tinh chỉnh).
- **Mô hình nền tảng** (*foundation model*): mô hình đã được tiền huấn luyện trên dữ liệu rất lớn, dùng làm "nền móng" cho nhiều ứng dụng khác nhau.

Cuốn sách này tập trung vào các phương pháp NLP hiện đại, lấy mạng nơ-ron và mô hình nền tảng (*foundation model*) làm trung tâm. Mục tiêu của sách là cung cấp một hướng dẫn thực tế để hiểu, xây dựng và ứng dụng những mô hình mạnh mẽ này. Khác với các giáo trình NLP truyền thống vốn sắp xếp chương mục theo từng bài toán cụ thể, cuốn sách này được tổ chức theo góc nhìn *xây dựng* các mô hình NLP dựa trên mạng nơ-ron, và được chia thành ba phần chính:

- **Nền tảng học máy và mạng nơ-ron** (Chương 1–2): Phần này giới thiệu các khái niệm và phương pháp cốt lõi của học máy (*machine learning*) và mạng nơ-ron, làm nền móng cho các chương sau. Phần này tương đối độc lập, có thể học riêng hoặc dùng làm tài liệu tham khảo khi cần.

- **Các mô hình nơ-ron cơ bản cho xử lý ngôn ngữ tự nhiên** (Chương 3–6): Phần này giải thích các mạng nơ-ron được dùng trong các bài toán NLP, bao gồm mô hình biểu diễn từ, mô hình chuỗi và mô hình chuỗi-sang-chuỗi. Ngoài ra, Transformer được giới thiệu trong một chương riêng. Những mô hình này không chỉ giới hạn ở từng bài toán NLP riêng lẻ, mà đóng vai trò là

công cụ đa năng cho rất nhiều ứng dụng.

- **Mô hình ngôn ngữ lớn** (Chương 7–12): Phần này tập trung vào các mô hình ngôn ngữ lớn (*large language model, LLM*), bao gồm các chủ đề như tiền huấn luyện, mô hình sinh, kỹ thuật prompt (*prompt engineering*), căn chỉnh (*alignment*), suy luận (*inference*) và lập luận (*reasoning*).

Cuốn sách hướng tới sinh viên đại học năm cuối, học viên sau đại học, các nhà nghiên cứu trong những lĩnh vực liên quan, và bất kỳ ai quan tâm đến NLP. Chúng tôi cố gắng viết rõ ràng, dễ tiếp cận, nhằm giới thiệu các khái niệm cốt lõi và phương pháp nền tảng thay vì đi sâu vào mọi kỹ thuật tiên tiến nhất. Vì vậy, cuốn sách vừa có thể dùng làm sách nhập môn cho người mới, vừa có thể dùng làm sổ tay tra cứu các khái niệm và phương pháp chủ chốt của NLP.

Nội dung cuốn sách dần thành hình qua nhiều năm giảng dạy và nghiên cứu của chúng tôi. Ban đầu, chúng tôi chỉ định viết hai phần đầu. Tuy nhiên, sự phát triển nhanh chóng và tầm quan trọng ngày càng lớn của LLM đã khiến chúng tôi đưa chủ đề này vào như một phần then chốt của sách. Đồng thời, chúng tôi rất vui mừng được chứng kiến sự phát triển nhanh chóng của NLP, và việc viết cuốn sách này cũng là cách chúng tôi hưởng ứng xu thế thú vị ấy.

Một số chương của sách đã từng được công bố trực tuyến, chẳng hạn *Introduction to Transformers: An NLP Perspective* (Chương 6) và *Foundations of Large Language Models* (Chương 7–12). Chúng tôi xin cảm ơn những góp ý quý báu của nhiều độc giả, đã giúp hoàn thiện đáng kể cuốn sách. Ngoài ra, trong quá trình viết, chúng tôi đã nhận được nhiều cảm hứng từ các tác phẩm kinh điển, bao gồm *Machine Learning* của Mitchell [1997], *Foundations of Statistical Natural Language Processing* của Manning và Schütze [1999], *Pattern Recognition and Machine Learning* của Bishop [2006], và *Speech and Language Processing* của Jurafsky và Martin [2008].

Cuối cùng, chúng tôi xin chân thành cảm ơn tất cả những người đã đóng góp ý kiến và chỉnh sửa nội dung cuốn sách. Họ là: Weiqiao Shan, Yongyu Mu, Chenglong Wang, Kaiyan Chang, Yuchun Fan, Hang Zhou, Chuanhao Lv, Xinyu Liu, Tao Zhou, Huiwen Bao, Tong Zheng, Junhao Ruan, Yingfeng Luo, Yuzhang Wu, Yifu Huo, Xiaoqian Liu, Yucheng Qiao, Junxiang Zhang, Miaohe Niu, Junxin Wang, Xin Liu, Xinyi Xu, Dan Chen, Xihan Yang và Jingqi Xiao.

Tong Xiao và Jingbo Zhu
Tháng 9 năm 2026

Hướng dẫn đọc bản dịch

Bản dịch này dành cho bạn đọc vừa học xong chương trình trung học phổ thông. Sách gốc viết cho sinh viên đại học năm cuối nên dùng khá nhiều kiến thức toán đại học. Để bạn không bị "mắc kẹt", người dịch đã bổ sung những yếu tố sau:

- **Khung "Chú giải của người dịch"** (nền vàng nhạt, như khung ở trang trước): giải thích thuật ngữ, ôn lại kiến thức toán cần dùng (vectơ, ma trận, đạo hàm riêng, xác suất có điều kiện, logarit…), đưa ví dụ bằng số cụ thể hoặc hình ảnh so sánh đời thường. Nếu đã hiểu, bạn có thể bỏ qua các khung này; chúng không thuộc nội dung gốc.
- **Chú thích chân trang có đánh dấu [ND]**: những giải thích ngắn của người dịch. Chú thích chân trang không có dấu [ND] là của tác giả.
- **Thuật ngữ**: lần đầu xuất hiện trong mỗi chương, thuật ngữ được ghi bằng tiếng Việt kèm tiếng Anh trong ngoặc, ví dụ "mạng nơ-ron (*neural network*)". Một số từ được giữ nguyên tiếng Anh vì giới nghiên cứu Việt Nam vẫn dùng như vậy (Transformer, token, prompt, gradient…). Bảng thuật ngữ Anh–Việt đầy đủ nằm ở cuối sách.
- **Hình vẽ**: được lấy nguyên từ sách gốc nên chữ trong hình vẫn là tiếng Anh; phần chú thích dưới hình đã được dịch, và khi cần, người dịch giải thích thêm các nhãn trong hình.
- **Đánh số**: số chương, mục, phương trình, hình và bảng giữ đúng như sách gốc để bạn tiện đối chiếu. Trích dẫn tài liệu dạng "Tên tác giả [năm]" tương ứng với Danh mục tài liệu tham khảo (giữ nguyên tiếng Anh) ở cuối sách.

Chú giải của người dịch: Lời khuyên khi đọc

Đừng cố hiểu hết mọi công thức ngay lần đọc đầu. Hãy đọc lời giải thích bằng chữ trước để nắm *ý tưởng*, sau đó mới quay lại với công thức và thử tính tay một ví dụ nhỏ. Hai chương đầu là nền móng; nếu thấy khó, hãy đọc chậm và làm lại các ví dụ bằng số.

Mục lục

Lời nói đầu **iv**

Hướng dẫn đọc bản dịch **vi**

I Kiến thức nền tảng **1**

1 Nền tảng của học máy **2**

1.1 Kiến thức toán cơ bản . . . 3
1.2 Thiết kế một bộ phân loại văn bản . . . 11
1.3 Các vấn đề chung . . . 24
1.4 Lựa chọn và đánh giá mô hình . . . 39
1.5 Các nhiệm vụ NLP dưới góc nhìn học máy . . . 50
1.6 Tóm tắt . . . 61

2 Nền tảng của mạng nơ-ron **62**

2.1 Mạng nơ-ron nhiều tầng . . . 62
2.2 Ví dụ: mô hình hóa ngôn ngữ bằng mạng nơ-ron . . . 71
2.3 Các kiến trúc mô hình cơ bản . . . 76
2.4 Huấn luyện mạng nơ-ron . . . 84
2.5 Các phương pháp chính quy hóa . . . 95
2.6 Các phương pháp không giám sát và bộ tự mã hóa . . . 104
2.7 Tóm tắt . . . 117

II Các mô hình cơ bản **118**

3 Từ và vectơ từ **119**

3.1 Tách token . . . 119
3.2 Biểu diễn vectơ cho từ . . . 133
3.3 Các mô hình dựa trên đếm . . . 138
3.4 Học nhúng từ từ các mô hình ngôn ngữ nơ-ron . . . 150
3.5 Các mô hình nhúng từ . . . 152
3.6 Đánh giá vectơ nhúng từ . . . 160
3.7 Tóm tắt . . . 165

4 Các mô hình chuỗi hồi quy và tích chập **168**
4.1 Phát biểu bài toán . . . 169
4.2 Các mô hình hồi quy . . . 170
4.3 Bộ nhớ . . . 180
4.4 Các mô hình tích chập . . . 185
4.5 Các ví dụ . . . 197
4.6 Tóm tắt . . . 209

5 Các mô hình chuỗi-sang-chuỗi **211**
5.1 Các bài toán chuỗi-sang-chuỗi . . . 212
5.2 Kiến trúc bộ mã hóa–bộ giải mã . . . 213
5.3 Cơ chế chú ý . . . 218
5.4 Tìm kiếm . . . 236
5.5 Tóm tắt . . . 265

6 Transformer **268**
6.1 Mô hình cơ bản . . . 268
6.2 Mô hình có nhận biết cú pháp . . . 284
6.3 Các kiến trúc cải tiến . . . 295
6.4 Các mô hình hiệu quả . . . 313
6.5 Ứng dụng . . . 350
6.6 Tóm tắt . . . 359

III Mô hình ngôn ngữ lớn **365**

7 Tiền huấn luyện **366**
7.1 Tiền huấn luyện trong NLP . . . 367
7.2 Các nhiệm vụ tiền huấn luyện tự giám sát . . . 374
7.3 Ví dụ: BERT . . . 391
7.4 Ứng dụng các mô hình BERT . . . 402
7.5 Tóm tắt . . . 408

8 Mô hình sinh **410**
8.1 Giới thiệu ngắn gọn về LLM . . . 412
8.2 Huấn luyện ở quy mô lớn . . . 432
8.3 Mô hình hóa chuỗi dài . . . 444
8.4 Tóm tắt . . . 474

9 Prompting **476**
9.1 Thiết kế prompt tổng quát . . . 477
9.2 Các phương pháp prompting nâng cao . . . 493

9.3 Học cách tạo prompt . . . 517
9.4 Tóm tắt . . . 534

10 Căn chỉnh **536**
10.1 Tổng quan về căn chỉnh LLM . . . 537
10.2 Căn chỉnh theo chỉ dẫn . . . 538
10.3 Căn chỉnh theo sở thích của con người: RLHF . . . 555
10.4 Cải tiến việc căn chỉnh theo sở thích của con người . . . 575
10.5 Tóm tắt . . . 591

11 Suy luận **593**
11.1 Điền trước và giải mã . . . 594
11.2 Các kỹ thuật suy luận hiệu quả . . . 614
11.3 Co giãn lúc suy luận . . . 628
11.4 Tóm tắt . . . 642

12 Lập luận **644**
12.1 Bối cảnh và các khái niệm . . . 644
12.2 Lập luận thông qua prompt . . . 648
12.3 Lập luận lúc suy luận . . . 649
12.4 Học cách lập luận . . . 660
12.5 Đánh giá các LLM lập luận . . . 670
12.6 Bàn thêm . . . 674
12.7 Tóm tắt . . . 678

Bảng thuật ngữ Anh – Việt **679**

Tài liệu tham khảo (nguyên bản) **723**

Chỉ mục (nguyên bản, số trang theo sách gốc) **795**

Phần I

Kiến thức nền tảng

Chương 1

Nền tảng của học máy

> *Mục tiêu của học máy là phát triển các phương pháp có thể tự động phát hiện các khuôn mẫu trong dữ liệu, rồi dùng những khuôn mẫu đã phát hiện được để dự đoán dữ liệu tương lai hoặc những kết quả khác mà ta quan tâm.*
>
> – Murphy [2012]

> *Học máy có thể được định nghĩa một cách khái quát là các phương pháp tính toán sử dụng kinh nghiệm để cải thiện hiệu năng hoặc để đưa ra dự đoán chính xác.*
>
> – Mohri và cộng sự [2018]

NLP dựa trên dữ liệu phù hợp với các định nghĩa trên[1]. Nó dạy máy tính học *kinh nghiệm* ngôn ngữ từ các kho ngữ liệu (*corpora*), rồi hiểu và sử dụng ngôn ngữ dựa trên *kinh nghiệm* đó. Việc kết nối học máy (*machine learning, ML*) với xử lý ngôn ngữ tự nhiên (*natural language processing, NLP*) không chỉ đơn thuần là một cách giúp máy tính bắt chước trí thông minh ngôn ngữ của con người từ dữ liệu. Nó đang dẫn dắt một cuộc cách mạng trong cả hai lĩnh vực: xử lý ngôn ngữ tự nhiên tiến bộ nhờ có được một công cụ mạnh mẽ để rút ra ý nghĩa từ kho ngữ liệu, còn học máy tiến bộ nhờ giải quyết các thách thức của NLP và được kiểm nghiệm trên dữ liệu thực tế.

Trong chương này, chúng tôi trình bày một số khái niệm và mô hình cơ bản của học máy. Không có phần nào thật sự khó, chỉ là một số kiến thức chuẩn bị cho các chương sau. Ở đây, chúng tôi tập trung vào cách áp dụng học máy cho các bài toán NLP, đặc biệt là cách phát biểu một bài toán NLP thành một bài toán học thống kê. Để làm điều này, chúng tôi bắt đầu với phân loại (*classification*) — một trong những ví dụ được dùng rộng rãi nhất trong hầu hết các sách nhập môn. Sau đó, chúng tôi trình bày một số vấn đề nền tảng của học máy. Tiếp theo là phần thảo luận về các bài toán NLP dưới góc nhìn học máy.

Chú giải của người dịch: Kho ngữ liệu là gì?

Kho ngữ liệu (*corpus*, số nhiều là *corpora*) là một tập hợp rất lớn các văn bản (hoặc bản ghi lời nói) được thu thập để máy tính "đọc" và học. Ví dụ: toàn bộ bài báo của một tờ báo điện tử trong 10 năm, hay hàng triệu trang Wikipedia. "Học kinh nghiệm ngôn ngữ" ở đây có nghĩa là máy tính tự rút ra các quy luật thống kê (từ nào hay đi với từ nào, câu nào thường mang nghĩa tích cực…) từ kho ngữ liệu đó, thay vì được con người viết

[1] Từ đây trở đi, chúng tôi bỏ cụm từ *dựa trên dữ liệu* và mặc định rằng mọi mô hình NLP trong phần còn lại của sách đều dựa trên dữ liệu.

sẵn từng quy tắc.

1.1 Kiến thức toán cơ bản

Trong phần còn lại của chương này và các chương sau, chúng ta sẽ bàn về các bài toán học máy bằng công cụ toán học ứng dụng. Dưới đây là phần kiến thức toán cơ bản. Nếu thấy các chi tiết này quá đơn giản, bạn có thể chuyển thẳng tới Mục 1.2.

1.1.1 Đại số tuyến tính

1. Vectơ và ma trận

Số vô hướng (*scalar*) có thể không phải là khái niệm đơn giản nhất trong đại số tuyến tính, nhưng chắc chắn là khái niệm phổ biến nhất mà ai cũng học ở trung học hay đại học. Một số vô hướng chính là một con số. Nó là một đại lượng có độ lớn nhưng không có hướng. Chẳng hạn, chiều cao, cân nặng, khoảng cách, nhiệt độ đều là những ví dụ về đại lượng vô hướng. Ở đây, ta dùng chữ in nghiêng để ký hiệu một số vô hướng, ví dụ a, b, x, A, v.v.

Vectơ (*vector*) và **ma trận** (*matrix*) được định nghĩa dựa trên số vô hướng. Một vectơ là một dãy các số vô hướng, hay đơn giản là một danh sách số. Một ma trận là một bảng hình chữ nhật gồm các số vô hướng. Trong sách này, chúng tôi theo quy ước dùng chữ in đậm để ký hiệu vectơ và ma trận. Ví dụ, một vectơ n chiều (*dimension*) có thể được viết là

$$\mathbf{a} = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix}, \tag{1.1}$$

trong đó $\{a_1, a_2, \cdots, a_n\}$ là các phần tử (*elements, entries*) của vectơ. Mỗi phần tử ứng với một chiều. Để tiện ký hiệu, đôi khi ta viết a_i thành $a(i)$. Một vectơ được gọi là vectơ thực chỉ khi mọi phần tử của nó đều là số thực (tức là $a_i \in \mathbb{R}$ với mọi i), ký hiệu là $\mathbf{a} \in \mathbb{R}^n$. Tương tự, ta có thể viết một ma trận $m \times n$ là

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1n} \\ A_{21} & A_{22} & \cdots & A_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ A_{m1} & A_{m2} & \cdots & A_{mn} \end{bmatrix}, \tag{1.2}$$

trong đó m là số hàng và n là số cột. A_{ij} là phần tử (i, j) của ma trận. Một ma trận thực được ký hiệu là $\mathbf{A} \in \mathbb{R}^{m \times n}$. Đôi khi, ta dùng $\mathbf{A}_{m \times n}$ để nhấn mạnh rằng ma trận có kích thước $m \times n$.

Có một vài ma trận đặc biệt. Chẳng hạn, ma trận có mọi phần tử đều bằng 0 là **ma trận không** (*zero matrix*), ký hiệu là $\mathbf{0}$. Một ví dụ khác là **ma trận đơn vị** (*identity matrix*), ký hiệu là $\mathbf{I}$. Đó là một ma trận vuông có các phần tử trên đường chéo chính đều bằng 1, còn các phần tử khác bằng 0. Vectơ cũng có thể được xem là một loại ma trận đặc biệt. Chẳng hạn, vectơ trong Phương trình (1.1) là một ma trận chỉ có một hàng.

Chú giải của người dịch: So sánh với vectơ ở lớp 10

Ở lớp 10, bạn đã gặp vectơ $\vec{u} = (3; 4)$ trong mặt phẳng tọa độ: một mũi tên có hướng và độ dài. Trong học máy, ta dùng cùng ý tưởng nhưng với *rất nhiều* tọa độ: vectơ $\mathbf{a} = [0.2 \ \ -1.5 \ \ \cdots \ \ 3.1]$ có thể có hàng trăm, hàng nghìn phần tử. Ta không "vẽ" được nó nữa, nhưng các phép toán (cộng, nhân với số, tích vô hướng) vẫn y hệt. *Ma trận* đơn giản là một bảng số hình chữ nhật, giống như bảng điểm của một lớp: mỗi hàng là một

học sinh, mỗi cột là một môn. Ví dụ, ma trận $\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}$ là ma trận đơn vị cỡ 2×2; với ma trận $\mathbf{A}$ ở Phương trình (1.3) dưới đây, $A_{12} = 0$ (hàng 1, cột 2) và $A_{23} = 7$ (hàng 2, cột 3). Ký hiệu $\mathbb{R}^{m\times n}$ đọc là "tập các ma trận thực có m hàng, n cột".

2. Chuyển vị ma trận

Chuyển vị (*transpose*) của một ma trận $A_{m\times n}$ là một ma trận $B_{n\times m}$ thỏa mãn $A_{ij} = B_{ji}$ với mọi cặp i và j. Chuyển vị của $\mathbf{A}$ thường được ký hiệu là $\mathbf{A}^\top$. Ví dụ, với ma trận

$$\mathbf{A} = \begin{bmatrix}8 & 0 & 0\\ 2 & 9 & 7\end{bmatrix}, \tag{1.3}$$

ma trận chuyển vị là

$$\mathbf{A}^\top = \begin{bmatrix}8 & 2\\ 0 & 9\\ 0 & 7\end{bmatrix}. \tag{1.4}$$

Ta cũng có thể chuyển vị một vectơ. Với vectơ

$$\mathbf{a} = \begin{bmatrix}1 & 9 & 7 & 3\end{bmatrix} \tag{1.5}$$

thì vectơ chuyển vị là

$$\mathbf{a}^\top = \begin{bmatrix}1\\ 9\\ 7\\ 3\end{bmatrix} \tag{1.6}$$

Nói chung, vectơ chỉ có một hàng được gọi là **vectơ hàng** (*row vector*) (như trong Phương trình (1.5)), còn vectơ chỉ có một cột được gọi là **vectơ cột** (*column vector*) (như trong Phương trình (1.6)). Trong sách này, mặc định mọi vectơ đều là vectơ hàng.

Chú giải của người dịch: Chuyển vị = "lật" ma trận qua đường chéo

Chuyển vị đơn giản là biến hàng thành cột: hàng thứ nhất $[8\ \ 0\ \ 0]$ của $\mathbf{A}$ trở thành cột thứ nhất của $\mathbf{A}^\top$, hàng thứ hai $[2\ \ 9\ \ 7]$ trở thành cột thứ hai. Vì thế một ma trận 2×3 sau khi chuyển vị sẽ có kích thước 3×2. Chuyển vị hai lần thì quay về ma trận ban đầu: $(\mathbf{A}^\top)^\top = \mathbf{A}$.

3. Các phép toán theo từng phần tử trên ma trận

Giả sử $\mathbf{A}$ và $\mathbf{B}$ là hai ma trận có cùng kích thước, chẳng hạn $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m\times n}$. **Phép cộng ma trận** (*matrix addition*) của $\mathbf{A}$ và $\mathbf{B}$ được viết là $\mathbf{A} + \mathbf{B}$. $\mathbf{A} + \mathbf{B}$ là một ma trận trong $\mathbb{R}^{m\times n}$ mà mỗi phần tử bằng tổng của các phần tử tương ứng của $\mathbf{A}$ và $\mathbf{B}$. Dưới đây là một ví dụ.

$$\begin{aligned}\mathbf{A} + \mathbf{B} &= \begin{bmatrix}8 & 0 & 0\\ 2 & 9 & 7\end{bmatrix} + \begin{bmatrix}1 & 1 & 1\\ 1 & 0 & 4\end{bmatrix}\\ &= \begin{bmatrix}9 & 1 & 1\\ 3 & 9 & 11\end{bmatrix}.\end{aligned} \tag{1.7}$$

Theo cách tương tự, ta có thể định nghĩa phép trừ ($\mathbf{A} - \mathbf{B}$), phép nhân ($\mathbf{A} \odot \mathbf{B}$), phép chia ($\mathbf{A} \oslash \mathbf{B}$) theo từng phần tử (*element-wise*) và các phép toán khác. Một trường hợp đặc biệt của phép toán theo từng phần tử là **phép nhân với số vô hướng** (*scalar multiplication*), trong đó ta nhân một ma trận $\mathbf{A}$ với một số vô hướng k, ký hiệu là $k \times \mathbf{A}$ hoặc $k\mathbf{A}$. Xem ví dụ dưới đây với $k = 2$ và $\mathbf{A} = \begin{bmatrix} 8 & 0 & 0 \\ 2 & 9 & 7 \end{bmatrix}$.

$$\begin{aligned} k\mathbf{A} &= 2 \begin{bmatrix} 8 & 0 & 0 \\ 2 & 9 & 7 \end{bmatrix} \\ &= \begin{bmatrix} 16 & 0 & 0 \\ 4 & 18 & 14 \end{bmatrix}. \end{aligned} \tag{1.8}$$

Chú giải của người dịch: Phép nhân theo từng phần tử

Ký hiệu $\odot$ (đôi khi gọi là tích Hadamard) nghĩa là nhân từng cặp phần tử ở cùng vị trí, giống hệt phép cộng ở trên. Ví dụ, với hai ma trận ở Phương trình (1.7): $\begin{bmatrix} 8 & 0 & 0 \\ 2 & 9 & 7 \end{bmatrix} \odot \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 4 \end{bmatrix} = \begin{bmatrix} 8 & 0 & 0 \\ 2 & 0 & 28 \end{bmatrix}$. Cần phân biệt phép nhân này với *phép nhân ma trận* (mục 5 dưới đây), vốn được tính theo một quy tắc hoàn toàn khác.

Cho $\mathbf{A}$, $\mathbf{B}$ và $\mathbf{C}$ là các ma trận trong $\mathbb{R}^{m \times n}$, k và l là các số vô hướng trong $\mathbb{R}$. Các phép toán ma trận có một số tính chất sau:

- Tính chất của ma trận không:

$$\begin{aligned} \mathbf{A} &= \mathbf{A} + \mathbf{0} \\ &= \mathbf{0} + \mathbf{A}. \end{aligned} \tag{1.9}$$

- **Tính giao hoán**:

$$\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A} \tag{1.10}$$

$$k l \mathbf{A} = l k \mathbf{A}. \tag{1.11}$$

- **Tính kết hợp**:

$$(\mathbf{A} + \mathbf{B}) + \mathbf{C} = \mathbf{A} + (\mathbf{B} + \mathbf{C}) \tag{1.12}$$

$$(kl)\mathbf{A} = l(k\mathbf{A}). \tag{1.13}$$

- **Tính phân phối**:

$$k(\mathbf{A} + \mathbf{B}) = k\mathbf{A} + k\mathbf{B} \tag{1.14}$$

$$(k + l)\mathbf{A} = k\mathbf{A} + l\mathbf{A}. \tag{1.15}$$

4. Tích vô hướng

Tích vô hướng (*dot product*) của hai vectơ cùng số chiều $\mathbf{a} = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix}$ và $\mathbf{b} = \begin{bmatrix} b_1 & b_2 & \cdots & b_n \end{bmatrix}$ được định nghĩa là:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_i b_i$$

$$= a_1 b_1 + a_2 b_2 + \cdots + a_n b_n. \tag{1.16}$$

Về mặt hình học, một vectơ thực **a** có thể được xem như một đối tượng hình học có cả độ lớn (ký hiệu là $|\mathbf{a}|$) và hướng. Tích vô hướng của **a** và **b** cũng có thể được định nghĩa là:

$$\mathbf{a} \cdot \mathbf{b} \;=\; |\mathbf{a}| \times |\mathbf{b}| \times \cos(\theta), \tag{1.17}$$

trong đó θ là góc giữa **a** và **b**.

Chú giải của người dịch: Ký hiệu tổng $\sum$ và ý nghĩa của tích vô hướng

Ký hiệu $\sum_{i=1}^{n} a_i b_i$ đọc là "tổng của $a_i b_i$ khi i chạy từ 1 đến n". Ví dụ, với $\mathbf{a} = [1 \;\; 2 \;\; 3]$ và $\mathbf{b} = [4 \;\; 0 \;\; -1]$: $\mathbf{a} \cdot \mathbf{b} = 1 \cdot 4 + 2 \cdot 0 + 3 \cdot (-1) = 1$. Đây chính là công thức tích vô hướng $\vec{u} \cdot \vec{v} = x_1 x_2 + y_1 y_2$ bạn đã học ở lớp 10, chỉ mở rộng ra n tọa độ; Phương trình (1.17) cũng là công thức quen thuộc $\vec{u} \cdot \vec{v} = |\vec{u}|\,|\vec{v}| \cos(\vec{u}, \vec{v})$. Trong NLP, tích vô hướng thường được dùng để đo mức độ "giống nhau" của hai vectơ: hai vectơ càng cùng hướng thì $\cos\theta$ càng gần 1 và tích vô hướng càng lớn.

5. Tích ma trận

Tích ma trận (*matrix product*) (hay **tích ma trận–ma trận** (*matrix-matrix product*)) tác động lên hai ma trận. Cho một ma trận $\mathbf{A} \in \mathbb{R}^{m \times p}$ và một ma trận $\mathbf{B} \in \mathbb{R}^{p \times n}$, tích ma trận của **A** và **B** tạo ra một ma trận $\mathbf{C} \in \mathbb{R}^{m \times n}$ có các phần tử được định nghĩa là:

$$\begin{aligned} C_{ij} &= \sum_{k=1}^{p} A_{ik} \times B_{kj} \\ &= A_{i1} \times B_{1j} + A_{i2} \times B_{2j} + \cdots + A_{ip} \times B_{pj}. \end{aligned} \tag{1.18}$$

Tích ma trận đòi hỏi số cột của **A** phải bằng đúng số hàng của **B**. Trong sách này, ta dùng **AB** để ký hiệu tích ma trận của **A** và **B**. Dưới đây là một vài tính chất của tích ma trận.

- **Tính phân phối**. Với $\mathbf{A} \in \mathbb{R}^{m \times p}$ và $\mathbf{B}, \mathbf{C} \in \mathbb{R}^{p \times n}$, tính phân phối bên trái được định nghĩa là:

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) \;=\; \mathbf{AB} + \mathbf{AC}. \tag{1.19}$$

  Với $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times p}$ và $\mathbf{C} \in \mathbb{R}^{p \times n}$, tính phân phối bên phải được định nghĩa là:

$$(\mathbf{A} + \mathbf{B})\mathbf{C} \;=\; \mathbf{AC} + \mathbf{BC}. \tag{1.20}$$

- **Tính kết hợp**. Với $\mathbf{A} \in \mathbb{R}^{m \times p}$, $\mathbf{B} \in \mathbb{R}^{p \times q}$ và $\mathbf{C} \in \mathbb{R}^{q \times n}$, tính kết hợp được phát biểu là

$$(\mathbf{AB})\mathbf{C} \;=\; \mathbf{A}(\mathbf{BC}). \tag{1.21}$$

- **Chuyển vị**. Với $\mathbf{A} \in \mathbb{R}^{m \times p}$ và $\mathbf{B} \in \mathbb{R}^{p \times n}$, ta có

$$(\mathbf{AB})^\top \;=\; \mathbf{B}^\top \mathbf{A}^\top. \tag{1.22}$$

Tích ma trận không có tính giao hoán, tức là ta không có $\mathbf{AB} = \mathbf{BA}$ với mọi **A** và **B**, ngay cả khi **A** và **B** là các ma trận vuông cùng kích thước. Dựa trên tích ma trận–ma trận, ta có thể định nghĩa tương ứng tích vectơ–ma trận và tích ma trận–vectơ. Việc này rất đơn giản, vì tất cả những gì ta cần làm là coi vectơ như một ma trận khi nhân.

Chú giải của người dịch: Nhân ma trận: "hàng nhân cột"

Phần tử C_{ij} của tích **AB** chính là *tích vô hướng* của hàng thứ i của **A** với cột thứ j của **B**. Ví dụ:

$$\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix}\begin{bmatrix}5 & 6\\7 & 8\end{bmatrix} = \begin{bmatrix}1\cdot 5+2\cdot 7 & 1\cdot 6+2\cdot 8\\3\cdot 5+4\cdot 7 & 3\cdot 6+4\cdot 8\end{bmatrix} = \begin{bmatrix}19 & 22\\43 & 50\end{bmatrix}.$$

Đổi thứ tự: $\begin{bmatrix}5 & 6\\7 & 8\end{bmatrix}\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix} = \begin{bmatrix}23 & 34\\31 & 46\end{bmatrix}$, khác kết quả trên — đó là lý do tích ma trận không giao hoán. Quy tắc kích thước dễ nhớ: $(m \times p)\cdot(p \times n) \to (m \times n)$; hai số "ở giữa" phải bằng nhau và "triệt tiêu" nhau. Chẳng hạn, vectơ hàng 1×3 nhân với ma trận 3×2 cho vectơ hàng 1×2. Phép nhân ma trận là phép toán cơ bản nhất trong mạng nơ-ron: phần lớn thời gian tính toán của một mô hình ngôn ngữ lớn là để nhân các ma trận khổng lồ.

6. Chuẩn

Trong một không gian vectơ, **chuẩn** (*norm*) là một hàm dùng để đo "độ dài" của vectơ. Với vectơ $\mathbf{a} \in \mathbb{R}^n$, chuẩn p (p-*norm*) (hay chuẩn l_p) được định nghĩa là:

$$\|\mathbf{a}\|_p = \Big(\sum_{i=1}^{n} |a_i|^p\Big)^{1/p}. \tag{1.23}$$

Các chuẩn p được dùng phổ biến nhất là với $p = 1$, 2 và ∞:

$$\|\mathbf{a}\|_1 = \sum_{i=1}^{n} |a_i| \tag{1.24}$$

$$\|\mathbf{a}\|_2 = \sqrt{\sum_{i=1}^{n} |a_i|^2} \tag{1.25}$$

$$\|\mathbf{a}\|_\infty = \max\{|a_1|, |a_2|, \cdots, |a_n|\}. \tag{1.26}$$

Chuẩn cũng có thể dùng để đo khoảng cách giữa hai điểm. Cho $\mathbf{b} \in \mathbb{R}^n$ là một vectơ khác. Khoảng cách theo chuẩn p giữa **a** và **b** là $\|\mathbf{a} - \mathbf{b}\|_p$.

Chú giải của người dịch: Tính thử các chuẩn

Với $\mathbf{a} = [3 \;\; -4]$: $\|\mathbf{a}\|_1 = |3| + |-4| = 7$; $\|\mathbf{a}\|_2 = \sqrt{3^2 + 4^2} = 5$ (chính là độ dài vectơ quen thuộc ở lớp 10, theo định lý Py-ta-go); $\|\mathbf{a}\|_\infty = \max\{3, 4\} = 4$. Hình dung trong thành phố có đường phố kẻ ô bàn cờ: chuẩn 1 là quãng đường phải đi dọc theo các phố ("khoảng cách Manhattan"), chuẩn 2 là đường chim bay, còn chuẩn ∞ chỉ quan tâm tới tọa độ chênh lệch lớn nhất. Với $\mathbf{b} = [0 \;\; 0]$, khoảng cách $\|\mathbf{a} - \mathbf{b}\|_2 = 5$.

1.1.2 Xác suất và thống kê

1. Xác suất là gì

Xác suất (*probability*) là một thước đo của sự không chắc chắn. Nó là một đại lượng mô tả khả năng một sự kiện xảy ra. Ví dụ, nếu sự kiện chắc chắn xảy ra, ta nói xác suất bằng 1; nếu sự kiện không bao giờ xảy ra, ta nói xác suất bằng 0.

Một sự kiện (*event*) là kết quả của một phép thử ngẫu nhiên. Ví dụ, trong phép thử tung đồng xu, một sự kiện có thể là "đồng xu ra mặt ngửa". Một tập hợp các sự kiện liên quan được mô tả bởi một **biến ngẫu nhiên** (*random variable*), gọi tắt là **biến** (*variable*). Ví dụ, ta có thể định nghĩa kết quả của việc tung một đồng xu là một biến x. Vì có hai kết quả (ngửa hoặc sấp), ta có hai lựa chọn cho giá trị của x. Ta có thể quy định $x = 1$ khi đồng xu ra mặt ngửa, và $x = 0$ trong trường hợp ngược lại. Do đó, x là một **biến nhị phân** (*binary variable*), hay chính xác hơn là một biến 0-1. Việc một biến ngẫu nhiên nhận một giá trị có nghĩa là một sự kiện xảy ra. Ví dụ, $x = 1$ nghĩa là sự kiện đồng xu ra mặt ngửa.

Trong toán học, xác suất được định nghĩa là một **độ đo xác suất** (*probability measure*) trên một không gian mẫu (*sample space*). Độ đo này phải thỏa mãn một số tiên đề nhất định, chẳng hạn tính cộng tính đếm được. Ở đây chúng tôi bỏ qua các chi tiết của lý thuyết độ đo, và coi xác suất là một hàm ánh xạ các sự kiện tới các số thực trong đoạn $[0, 1]$.

Thông thường, một độ đo xác suất được ký hiệu là một hàm $\Pr(\cdot)$, gọi là **hàm xác suất** (*probability function*). Khi đầu vào của $\Pr(\cdot)$ được định nghĩa trên một tập rời rạc các sự kiện, đầu ra của hàm là xác suất để một sự kiện xảy ra. Ví dụ, $\Pr(x = 1)$ là xác suất để x bằng 1. Lưu ý rằng $\Pr(x)$ là một hàm có đầu ra thay đổi khi ta chọn các giá trị khác nhau của x. Giả sử x_1 là một giá trị mà x có thể nhận. Khi ta viết $\Pr(x_1)$, điều đó có nghĩa là $\Pr(x = x_1)$. Vì xác suất trên tất cả các sự kiện phải bằng 1, mọi hàm xác suất đều phải thỏa mãn:

$$\sum_{x_i \in X} \Pr(x_i) \;=\; 1, \tag{1.27}$$

trong đó X là tập hợp mọi sự kiện. Một hàm xác suất có thể được định nghĩa trên hai biến trở lên. Dưới đây là một vài trường hợp được dùng rộng rãi.

- **Xác suất đồng thời** (*joint probability*). Đây là xác suất để hai sự kiện x và y cùng xảy ra, ký hiệu là $\Pr(x, y)$. Nếu x và y độc lập (*independent*), xác suất đồng thời của chúng bằng tích các xác suất riêng lẻ

$$\Pr(x, y) = \Pr(x) \Pr(y). \tag{1.28}$$

- **Xác suất có điều kiện** (*conditional probability*). Đây là xác suất để x xảy ra khi đã biết y xảy ra, ký hiệu là $\Pr(x|y)$. $\Pr(x|y)$ có thể được định nghĩa là:

$$\Pr(x|y) \;=\; \frac{\Pr(x, y)}{\Pr(y)}. \tag{1.29}$$

- **Xác suất biên** (*marginal probability*). Đây là một cách khác để định nghĩa xác suất của một biến đơn lẻ. Khi biết xác suất đồng thời trên hai biến, xác suất biên được định nghĩa là

$$\Pr(x) \;=\; \sum_{y_j \in Y} \Pr(x, y_j), \tag{1.30}$$

  trong đó Y là không gian sự kiện của y_j. Phương trình (1.30) nói rằng $\Pr(x)$ được lấy biên (*marginalized*) trên Y.

Thêm một lưu ý về xác suất đồng thời. Trong một số trường hợp, người ta muốn dùng xác suất có điều kiện để biểu diễn xác suất đồng thời. Để làm vậy, ta có thể viết lại xác suất đồng thời theo **quy tắc nhân** (*product rule*) hay **quy tắc chuỗi** (*chain rule*) như sau

$$\Pr(x, y) = \Pr(x|y) \Pr(y). \tag{1.31}$$

Chú giải của người dịch: Ví dụ tính tay: đồng thời, có điều kiện, biên

Một lớp có 40 học sinh. Gọi x là "học sinh thích Toán", y là "học sinh là nữ". Giả sử có 20 bạn nữ, trong đó 12 bạn thích Toán; và 20 bạn nam, trong đó 10 bạn thích Toán. Chọn ngẫu nhiên một học sinh:

- Xác suất đồng thời: $\Pr(x, y) = \Pr(\text{thích Toán và là nữ}) = 12/40 = 0.3$.
- Xác suất có điều kiện: $\Pr(x|y)$ là xác suất thích Toán *khi đã biết* đó là bạn nữ. Ta chỉ xét 20 bạn nữ: $\Pr(x|y) = 12/20 = 0.6$. Theo Phương trình (1.29): $\Pr(x, y)/\Pr(y) = 0.3/0.5 = 0.6$ — khớp.
- Xác suất biên: $\Pr(x) = \Pr(x, \text{nữ}) + \Pr(x, \text{nam}) = 12/40 + 10/40 = 0.55$ — ta "cộng dồn" theo mọi giá trị của y.

Ký hiệu $\Pr(x|y)$ đọc là "xác suất của x biết y". Lưu ý $\Pr(x|y)$ thường khác $\Pr(y|x)$: ở đây $\Pr(y|x) = \Pr(\text{nữ} \mid \text{thích Toán}) = 12/22 \approx 0.545$. Phương trình (1.31) chỉ là Phương trình (1.29) nhân chéo: $0.3 = 0.6 \times 0.5$. Vì $\Pr(x) = 0.55$, $\Pr(y) = 0.5$ mà $\Pr(x)\Pr(y) = 0.275 \neq 0.3$, nên hai sự kiện này *không* độc lập.

Với biến liên tục, xác suất để biến nhận chính xác một giá trị điểm nào đó bằng 0. Thay vào đó, ta có một mật độ tại điểm đó. Chặt chẽ hơn, cho một biến liên tục x, **mật độ xác suất** (*probability density*) của x được viết là $\Pr(x)$. Giả sử $x \in \mathbb{R}$. Xác suất để x nằm trong đoạn $[a, b]$ được định nghĩa qua một tích phân:

$$\Pr(x \in [a, b]) = \int_a^b \Pr(x)dx. \tag{1.32}$$

Hiển nhiên, ta có

$$\int_{-\infty}^{+\infty} \Pr(x)dx = 1. \tag{1.33}$$

Đối với các tính chất khác, như xác suất đồng thời và xác suất có điều kiện, dạng công thức cho biến liên tục gần như giống hệt với biến rời rạc. Ta chỉ cần thay các tổng trong công thức bằng tích phân.

Chú giải của người dịch: Vì sao xác suất tại một điểm bằng 0?

Hãy nghĩ tới chiều cao của một người: xác suất để ai đó cao *đúng* $1.700000\ldots$ m (vô hạn chữ số) gần như bằng 0, nhưng xác suất cao trong khoảng $[1.65; 1.75]$ m thì rõ ràng là dương. Mật độ $\Pr(x)$ giống như "mật độ dân cư": nó cho biết xác suất tập trung dày hay thưa quanh x, và muốn có xác suất thật ta phải lấy tích phân (tức "diện tích dưới đồ thị") trên một đoạn, như ở Phương trình (1.32). Phương trình (1.33) nói rằng tổng diện tích dưới toàn bộ đồ thị mật độ bằng 1.

2. Phân phối và kỳ vọng

Một **phân phối xác suất** (*probability distribution*) (gọi tắt là **phân phối**) mô tả cách các xác suất được gán cho các giá trị có thể có của một biến ngẫu nhiên. Nó được xác định bởi một **hàm khối xác suất** (*probability mass function, PMF*) đối với biến rời rạc, hoặc một **hàm mật độ xác suất** (*probability density function, PDF*) đối với biến liên tục. Ví dụ, một phân phối đều (*uniform distribution*) trên một biến rời rạc x nhận giá trị trong $\{x_1, x_2\}$ có thể được mô tả là $\Pr(x_i) = 1/2$, vì $\Pr(x_1) = \Pr(x_2)$ và $\Pr(x_1) + \Pr(x_2) = 1$; một phân phối đều trên một biến liên tục trong $[-2, 2]$ có thể được mô tả là $\Pr(x) = 1/4$, vì hàm mật độ xác suất $\Pr(x)$ là hằng số với mọi $x \in [-2, 2]$ và $\int_{-2}^{2} \Pr(x)dx = 1$. Các nhà thống kê đã xây dựng nhiều phân phối để mô tả thế giới chúng ta đang sống, chẳng hạn phân phối nhị thức (*binomial distribution*), phân phối Bernoulli (*Bernoulli distribution*) và phân phối Gauss/chuẩn (*Gaussian/normal distribution*). Có thể tìm thấy chi tiết về các phân phối này trong hầu hết các giáo trình thống kê [McClave và Sincich, 2006; Freedman và cộng sự, 2007].

Để mô tả các tính chất của một biến, một cách phổ biến là tính **giá trị kỳ vọng** (*expected value*) hay **kỳ vọng** (*expectation*) của biến đó. Cho x là một biến rời rạc nhận giá trị trong $\{x_1, \cdots, x_n\}$, và $\Pr(x)$ là một phân phối trên x. Giá trị kỳ vọng của x được định nghĩa là:

$$\mathbb{E}_{x\sim \Pr(x)}(x) \;=\; \sum_{i=1}^{n} x_i \cdot \Pr(x_i), \tag{1.34}$$

trong đó chỉ số dưới $x \sim \Pr(x)$ cho biết x tuân theo phân phối $\Pr(x)$. Trong nhiều trường hợp, ta có thể bỏ chỉ số dưới và viết gọn là $\mathbb{E}(x)$. Về bản chất, $\mathbb{E}(x)$ là giá trị trung bình có trọng số của x dưới phân phối $\Pr(x)$. Nó là một thước đo xu hướng trung tâm, và đôi khi được gọi là **giá trị trung bình** (*mean*) của một biến (ký hiệu là μ).

Tiếp theo, ta có thể định nghĩa **phương sai** (*variance*) của một biến là kỳ vọng của bình phương độ lệch so với giá trị trung bình

$$\mathrm{Var}(x) \;=\; \mathbb{E}[(x - \mathbb{E}(x))^2]. \tag{1.35}$$

Nói một cách không chính thức, nó mô tả các giá trị nằm cách giá trị trung bình xa đến mức nào. $\mathrm{Var}(x)$ thường được viết là σ^2, trong đó σ được gọi là **độ lệch chuẩn** (*standard deviation*).

Với một biến liên tục $x \in \mathbb{R}$, giá trị kỳ vọng được định nghĩa là:

$$\mathbb{E}(x) \;=\; \int_{-\infty}^{+\infty} x \cdot \Pr(x) dx, \tag{1.36}$$

trong đó $\Pr(x)$ là một hàm mật độ xác suất. Để tính phương sai của x, ta chỉ cần dùng lại Phương trình (1.35).

Chú giải của người dịch: Kỳ vọng và phương sai của một con xúc xắc

Gieo một con xúc xắc cân đối, $x \in \{1, 2, \ldots, 6\}$, mỗi mặt có xác suất $1/6$. Theo Phương trình (1.34): $\mathbb{E}(x) = 1\cdot\frac{1}{6} + 2\cdot\frac{1}{6} + \cdots + 6\cdot\frac{1}{6} = 3.5$. Không mặt nào ghi 3.5, nhưng nếu gieo rất nhiều lần thì trung bình cộng các kết quả sẽ tiến gần tới 3.5. Phương sai: $\mathrm{Var}(x) = \frac{1}{6}\left[(1-3.5)^2 + (2-3.5)^2 + \cdots + (6-3.5)^2\right] = \frac{17.5}{6} \approx 2.92$, nên $\sigma \approx 1.71$. Đây chính là số trung bình và phương sai trong thống kê lớp 10, chỉ khác là "tần suất" được thay bằng "xác suất". Ký hiệu $\mathbb{E}[\cdot]$ đọc là "kỳ vọng của…".

3. Entropy

Entropy (*entropy*) là một trong những công cụ quan trọng nhất để mô tả các biến ngẫu nhiên và các quá trình ngẫu nhiên [Shannon, 1948b]. Nó là thước đo của **độ bất ngờ kỳ vọng** (*expected surprisal*). Kết quả càng mang tính tất định thì càng ít bất ngờ và càng chứa ít thông tin. Để đơn giản, ở đây ta chỉ bàn về biến rời rạc[2]. Cho x là một biến và $\Pr(x)$ là một phân phối trên x. Entropy được viết là:

$$H(x) \;=\; -\sum_{i=1}^{n} \Pr(x_i) \cdot \log_b \Pr(x_i), \tag{1.37}$$

trong đó b là cơ số của hàm logarit. Giá trị của b thường được chọn là 2, 10 hoặc e.

[2] Với biến liên tục, ta có các phép tính tương tự.

Chú giải của người dịch: Entropy: đo mức "khó đoán"

Với $b = 2$, entropy đo bằng *bit*. Đồng xu cân đối (Pr $= 0.5$ cho mỗi mặt): $H = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = -(0.5 \cdot (-1) + 0.5 \cdot (-1)) = 1$ bit — khó đoán nhất. Đồng xu "méo" ra ngửa với xác suất 0.9: $H = -(0.9 \log_2 0.9 + 0.1 \log_2 0.1) \approx 0.469$ bit — dễ đoán hơn nên entropy nhỏ hơn. Đồng xu hai mặt đều là ngửa: $H = 0$ (quy ước $0 \cdot \log 0 = 0$), không có gì bất ngờ. Dấu trừ đứng trước vì log của một số trong $(0, 1]$ luôn ≤ 0, nhờ đó entropy luôn không âm.

Ngoài việc đo độ bất định của một phân phối, ta còn có thể định lượng mức độ khác biệt giữa hai phân phối. Giả sử $p(x)$ và $q(x)$ là các phân phối trên x. **Entropy tương đối** (*relative entropy*) của p so với q được định nghĩa là:

$$D_{\mathrm{KL}}(p \,\|\, q) \;=\; \sum_{i=1}^{n} p(x_i) \cdot \log_b \frac{p(x_i)}{q(x_i)}. \tag{1.38}$$

Ta có thể coi $p(x_i)$ là trọng số cho tỉ số hợp lý (*likelihood ratio*) dạng logarit $\log_b \frac{p(x_i)}{q(x_i)}$. Do đó, $D_{\mathrm{KL}}(p \,\|\, q)$ là tổng có trọng số của các tỉ số hợp lý trên mọi giá trị có thể có. Giá trị $D_{\mathrm{KL}}(p \,\|\, q)$ càng nhỏ cho thấy hai phân phối p và q càng gần nhau. Ví dụ, p và q sẽ trùng nhau nếu $D_{\mathrm{KL}}(p \,\|\, q) = 0$. Entropy tương đối còn được gọi là **độ phân kỳ Kullback-Leibler (KL)** (*Kullback-Leibler (KL) divergence*). Lưu ý rằng entropy tương đối không đối xứng, tức là ta không thể bảo đảm $D_{\mathrm{KL}}(p \,\|\, q) = D_{\mathrm{KL}}(q \,\|\, p)$.

Một khái niệm khác rất phổ biến trong học máy là **entropy chéo** (*cross-entropy*). Nó là thước đo lượng thông tin (tính bằng tổng số bit) cần thiết để truyền các sự kiện từ một nguồn tuân theo một phân phối khi dùng một phân phối khác. Chặt chẽ hơn, ta viết entropy chéo của phân phối p với phân phối q là $H_{\mathrm{cross}}(p, q)$. Nó được định nghĩa là:

$$H_{\mathrm{cross}}(p, q) \;=\; -\sum_{i=1}^{n} p(x_i) \cdot \log_2 q(x_i). \tag{1.39}$$

Giống như entropy tương đối, entropy chéo cũng không đối xứng. Trong bối cảnh tối ưu hóa, chẳng hạn khi huấn luyện các hệ thống NLP, cả hai đều thường được dùng làm hàm mục tiêu (*objective function*). Mối quan hệ giữa chúng được cho bởi $H(p, q) = H(p) + D_{\mathrm{KL}}(p \,\|\, q)$. Trong khi entropy tương đối đo số bit **tăng thêm** cần dùng do sử dụng q thay cho p, entropy chéo đo tổng số bit cần dùng.

Chú giải của người dịch: Ví dụ số về độ phân kỳ KL và entropy chéo

Lấy $p = (0.5;\ 0.5)$ (đồng xu cân đối, phân phối "thật") và $q = (0.9;\ 0.1)$ (phân phối mô hình "đoán"), dùng $\log_2$:

- $H_{\mathrm{cross}}(p, q) = -\big(0.5 \log_2 0.9 + 0.5 \log_2 0.1\big) \approx -(0.5 \cdot (-0.152) + 0.5 \cdot (-3.322)) \approx 1.737$ bit.
- $H(p) = 1$ bit, nên $D_{\mathrm{KL}}(p \,\|\, q) = H_{\mathrm{cross}}(p, q) - H(p) \approx 0.737$ bit (đúng hệ thức ở trên; ở đây $H(p, q)$ chính là $H_{\mathrm{cross}}(p, q)$).

Nếu mô hình đoán đúng $q = p$ thì $D_{\mathrm{KL}} = 0$ và entropy chéo đạt giá trị nhỏ nhất bằng $H(p)$. Vì vậy, khi huấn luyện mô hình, ta thường *cực tiểu hóa entropy chéo* giữa phân phối của dữ liệu thật p và phân phối dự đoán của mô hình q: làm vậy tương đương với kéo q lại gần p. Bạn sẽ gặp lại ý tưởng này ở hầu hết các chương sau.

1.2 Thiết kế một bộ phân loại văn bản

Phân loại là một trong những bài toán phổ biến nhất trong học máy. Nó nhằm tự động xếp các đối tượng vào một tập các lớp. Các lớp này được gọi là **nhãn** (*labels*), hay **thẻ** (*tags*), hay **hạng mục** (*categories*).

Nói chung, một chương trình thực hiện việc phân loại được gọi là một **bộ phân loại** (*classifier*) hay **hệ thống phân loại** (*classification system*). Bộ phân loại có vô số ứng dụng thực tế. Một ví dụ đơn giản là lọc thư rác, trong đó ta cần gán cho mỗi email nhãn "thư rác" hoặc "không phải thư rác". Những ví dụ khó hơn gồm: phân loại ảnh chụp cắt lớp vi tính (CT) các cơ quan nội tạng thành "bình thường" hoặc "bất thường", xác định một đoạn văn bản tiếng Trung có phải do người bản ngữ viết hay không, gán cho một đơn đăng ký sáng chế tập các mã IPC[3] mà nó thuộc về, v.v.

Nhiều lý thuyết và thuật toán học máy được mô hình hóa và kiểm nghiệm trên các bài toán phân loại. Theo thông lệ này, chúng tôi lấy phân loại văn bản (*text classification*) làm ví dụ mở đầu. Giả sử ta có một kho ngữ liệu như sau.

Văn bản	Nhãn
Trận đấu thật tuyệt vời.	Không phải đồ ăn
Tôi đã cố hết sức để làm lại món đó trong bếp nhà mình. Vị của nó tuyệt hảo.	Đồ ăn
Hàng thế kỷ nay, rong biển được coi là thức ăn của người bình dân.	Đồ ăn
Hôm nay bạn đã làm xong việc lập trình chưa?	Không phải đồ ăn
Tôi đang thắc mắc sao bạn lại có thể lỡ chuyến xe buýt.	Không phải đồ ăn
Tôi thích trái cây vì nó tốt cho sức khỏe.	Đồ ăn
Nghiên cứu xử lý ngôn ngữ tự nhiên thật thú vị.	Không phải đồ ăn
…	…

Mỗi dòng của kho ngữ liệu là một bộ gồm một đoạn văn bản (từ đây gọi là một tài liệu (*document*)) và một nhãn cho biết văn bản đó có nói về đồ ăn hay không. Ta gọi các bộ như vậy là **mẫu** (*samples*), hay chính xác hơn là **mẫu có nhãn** (*labeled samples*). Về bản chất, mẫu có nhãn là các cặp câu hỏi–câu trả lời, dù chúng không nhất thiết tuân theo dạng chuẩn của câu hỏi và câu trả lời. Chẳng hạn, trong các mẫu ở trên, ta có thể coi một tài liệu là câu hỏi và nhãn của nó là câu trả lời. Trong vài chương tới, chúng tôi sẽ chỉ ra rằng cách phát biểu này là tổng quát và phù hợp với hầu hết các bài toán trong NLP.

Tiếp theo, giả sử ta có một bộ phân loại học cách gán nhãn cho tài liệu từ các mẫu này. Sau đó, bộ phân loại được dùng để gán nhãn "Đồ ăn" hoặc "Không phải đồ ăn" cho những tài liệu mới, chưa từng gặp. Ví dụ, với văn bản

Trái cây không phải món tôi thích nhất, nhưng tôi vẫn có thể ăn ngon lành.

bộ phân loại sẽ xếp nó vào lớp "Đồ ăn".

Tuy nhiên, phân loại văn bản, dù bề ngoài có vẻ đơn giản, không chỉ đơn thuần là sắp xếp các mẫu chưa có nhãn vào các lớp. Nó đặt ra rất nhiều vấn đề, nhất là khi xét đến sự nhập nhằng và phong phú của ngôn ngữ. Các bộ phân loại hiện đại không phải là những hệ thống gồm một tập quy tắc do con người viết tay. Thay vào đó, chúng mô hình hóa bài toán phân loại theo cách xác suất, nhờ đó có thể học cách phân loại từ dữ liệu có nhãn quy mô lớn.

1.2.1 Phát biểu bài toán

Cho x là một tài liệu và c là một nhãn. Ở đây ta giả định có một bộ phân loại xác suất, ước lượng khả năng ta chọn c làm nhãn của x, ký hiệu là $\Pr(c|x)$. Nói chung, $\Pr(c|x)$ là một **mô hình phân loại** (*classification model*). Nó mô tả một phân phối trên tập mọi nhãn có thể có, thỏa mãn

$$\sum_{c \in \mathcal{C}} \Pr(c|x) \;=\; 1, \tag{1.40}$$

[3] **[ND]** IPC (*International Patent Classification*) là hệ thống Phân loại sáng chế quốc tế.

trong đó $\mathcal{C}$ là tập nhãn. Với mọi tài liệu, ta chọn nhãn có xác suất cao nhất làm đầu ra thông qua mô hình phân loại, như sau

$$\hat{c} = \arg\max_{c \in \mathcal{C}} \Pr(c|x), \tag{1.41}$$

trong đó $\hat{c}$ là nhãn "tốt nhất" do mô hình dự đoán. arg max là viết tắt của *arguments of the maxima* (các đối số làm hàm đạt cực đại). Nó trả về giá trị của đối số làm cho một hàm nào đó đạt giá trị lớn nhất.

Chú giải của người dịch: Phân biệt max **và** arg max

max trả về *giá trị lớn nhất*, còn arg max trả về *cái làm cho giá trị lớn nhất*. Ví dụ, giả sử với một tài liệu x, mô hình cho $\Pr(\text{Đồ ăn}|x) = 0.8$ và $\Pr(\text{Không phải đồ ăn}|x) = 0.2$ (tổng bằng 1, đúng Phương trình (1.40)). Khi đó $\max_c \Pr(c|x) = 0.8$, còn $\arg\max_c \Pr(c|x) = \text{Đồ ăn}$. Dấu mũ trong $\hat{c}$ (đọc là "c mũ") là quy ước thường dùng để chỉ một giá trị *dự đoán/ước lượng*, phân biệt với giá trị đúng c.

Phương trình (1.41) là phương trình nền tảng của phân loại. Nó kéo theo ba bài toán

- **Bài toán mô hình hóa**. $\Pr(c|x)$ là một thách thức về mặt tính toán vì không hiển nhiên là làm thế nào để có được giá trị của $\Pr(c|x)$ cho mỗi cặp x và c. Để xây dựng một mô hình phù hợp, ta có thể cần biểu diễn x và c theo một cách tính toán được và định nghĩa một quan hệ toán học kết nối chúng.

- **Bài toán học**. Từ góc nhìn học thống kê, dạng tổng quát của $\Pr(c|x)$ đại diện cho một họ các mô hình được cấu hình với các biến hay tham số (*parameters*) khác nhau. Các mô hình này về bản chất có cùng một dạng, nhưng sẽ hoạt động khác nhau nếu ta chọn các giá trị khác nhau cho các tham số đó. Vì vậy, ta cần chọn một mô hình "tốt" trong số chúng. Việc này thường được giải quyết bằng cách tối ưu các tham số trên dữ liệu có nhãn theo một tiêu chí nào đó.

- **Bài toán dự đoán**. Ở đây ta đang giải quyết một bài toán **phân loại nhị phân** (*binary classification*). Do đó, việc dự đoán lớp của tài liệu rất đơn giản, vì ta chỉ cần xác định lớp nào có xác suất cao hơn lớp còn lại. Tuy nhiên, khó mà hình dung được bài toán dự đoán trong thế giới thực khó đến mức nào, đặc biệt khi phải dự đoán các cấu trúc dạng cây hay đồ thị và các cấu trúc phi tuyến khác[4]. Với nhiều bài toán NLP, việc dự đoán cần đến các thuật toán tìm kiếm hiệu quả và nhanh.

Các bài toán này có tính tổng quát và bao trùm nhiều tác vụ học máy cũng như xử lý ngôn ngữ tự nhiên. Phân loại nhị phân, dù là một trong những trường hợp đơn giản nhất, cũng đủ để minh họa các khái niệm nền tảng của học máy. Phân loại còn có một số biến thể, chẳng hạn:

- **Phân loại đa lớp** (*multi-class classification*). Đây là dạng tổng quát hóa của phân loại nhị phân. Trong phân loại đa lớp, ta cần xếp mẫu vào một trong ba lớp trở lên.

- **Phân loại đa nhãn** (*multi-label classification*). Điều này có thể gây nhầm lẫn vì phân loại đa lớp và phân loại đa nhãn có vẻ là cùng một thứ. Theo cách dùng thuật ngữ thông thường, phân loại đa nhãn là việc gán nhiều nhãn cho một mẫu. Ngược lại, bài toán trình bày trong Phương trình (1.41) là một bài toán **phân loại đơn nhãn** (*single-label classification*).

[4]Dự đoán cây hay các cấu trúc khác không được coi là một bài toán con tiêu chuẩn của phân loại. Nó thường được gọi là **dự đoán cấu trúc** (*structure prediction*). Ở các mục sau, chúng tôi sẽ chỉ ra rằng cả phân loại và dự đoán cấu trúc đều có thể dùng chung một khuôn mẫu học máy tương tự nhau.

Chú giải của người dịch: Đa lớp hay đa nhãn?

Phân loại một bài báo vào *đúng một* trong các chuyên mục "Thể thao", "Kinh tế", "Giáo dục" là phân loại *đa lớp* (có từ ba lớp trở lên, nhưng mỗi mẫu chỉ có một nhãn). Gắn cho một bài báo *nhiều* thẻ cùng lúc, chẳng hạn vừa "Thể thao" vừa "Kinh tế" (bài viết về chuyển nhượng cầu thủ), là phân loại *đa nhãn*.

Phân loại sẽ thú vị và thách thức hơn nếu ta mở rộng nó sang trường hợp xử lý dữ liệu có cấu trúc phân cấp. Ví dụ, với dữ liệu sinh học và dữ liệu sáng chế, một số lớp có thể được gộp vào một lớp cha. Điều này tạo nên một hệ phân cấp các lớp và đòi hỏi một lược đồ phân loại phân cấp (*hierarchical classification*).

1.2.2 Biểu diễn tài liệu bằng vectơ đặc trưng

Vấn đề đầu tiên ta gặp phải khi thiết kế các mô hình phân loại văn bản là làm thế nào để biểu diễn một tài liệu. Coi x đơn thuần là một chuỗi ký tự không phải là giải pháp tốt. Người ta có thể muốn một cách biểu diễn (*representation*) mà qua đó con người hiểu được văn bản. Chẳng hạn, ta có thể phân tích mỗi câu trong tài liệu thành một cây cú pháp và dùng các cây làm biểu diễn văn bản. Tuy nhiên, cách này đòi hỏi công sức phát triển thêm các công cụ NLP khác (như bộ phân tích cú pháp).

Dĩ nhiên, biểu diễn là một vấn đề nền tảng trong NLP. Ở đây chúng tôi bỏ qua các mô hình phức tạp, tiên tiến nhất (*state-of-the-art*), để trình bày một cách tiếp cận đơn giản nhưng hiệu quả — **mô hình túi từ (BOW)** (*bag-of-words model*). Mô hình túi từ là một mô hình biểu diễn tài liệu dựa trên đặc trưng. Trong học máy, một **đặc trưng** (*feature*) là một thuộc tính của mẫu. Nó có thể được định nghĩa không chỉ là một thuộc tính phân loại (ví dụ tên hoặc giới tính) mà còn là một giá trị số, chẳng hạn một số thực.

Trong mô hình túi từ, một đặc trưng tương ứng với tần số (hay số lần xuất hiện) của một từ. Cho V là một bộ từ vựng (*vocabulary*). Một tài liệu có thể được biểu diễn bằng một vectơ đặc trưng (*feature vector*) $|V|$ chiều. Mỗi chiều tương ứng với một đặc trưng đếm từ, biểu diễn số lần từ thứ i trong V xuất hiện trong tài liệu. Chặt chẽ hơn, cho $\mathbf{x}$ là một vectơ đặc trưng. Phần tử thứ i của $\mathbf{x}$ được định nghĩa là:

$$x_i \;=\; \text{count}(V_i), \tag{1.42}$$

trong đó V_i là từ thứ i trong V và $\text{count}(\cdot)$ là một hàm đếm. Hãy xét ví dụ các dòng văn bản sau[5]. [6]

As I went to Bonner
I met a pig
Without a wig,
Upon my word and honor.

Khi đó ta có một bộ từ vựng được trích ra từ kho ngữ liệu[7], như sau

$$\begin{aligned} V \;=\; \{&\text{“a”, “and”, “as”, “Bonner”, “honor”,} \\ &\text{“I”, “met”, “word”, “my”, “pig”,} \\ &\text{“to”, “upon”, “went”, “wig”, “without”,} \end{aligned}$$

[5] Đoạn văn bản lấy từ tập đồng dao *Mother Goose rhymes*.

[6] **[ND]** Ví dụ này minh họa việc đếm từ tiếng Anh nên được giữ nguyên. Nghĩa đại ý: "Khi tôi đi đến Bonner / Tôi gặp một chú lợn / Không đội tóc giả, / Thề có lời danh dự của tôi."

[7] Chúng tôi chuyển mọi từ thành chữ thường ở đầu mỗi dòng.

“,”, “.”}.

Mỗi dòng của văn bản có thể được xem là một tài liệu và được biểu diễn bằng một vectơ đặc trưng. Dưới đây là các vectơ đặc trưng sinh ra bằng mô hình túi từ.

	a	and	as	Bonner	honor	I	met	word	my	pig	to	upon	went	wig	without	,	.
As I went to Bonner	[0	0	1	1	0	1	0	0	0	0	1	0	1	0	0	0	0]
I met a pig	[1	0	0	0	0	1	1	0	0	1	0	0	0	0	0	0	0]
Without a wig,	[1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0]
Upon my word and honor.	[0	1	0	0	1	0	0	1	1	0	0	1	0	0	0	0	1]

Chú giải của người dịch: Đọc bảng vectơ túi từ

Mỗi hàng là vectơ đặc trưng của một dòng thơ; mỗi cột ứng với một từ trong V (kể cả dấu phẩy và dấu chấm). Ví dụ, dòng “I met a pig” có các từ “I”, “met”, “a”, “pig”, mỗi từ xuất hiện 1 lần, nên vectơ có số 1 ở đúng 4 cột đó và số 0 ở 13 cột còn lại; $|V| = 17$ nên vectơ có 17 chiều. Tích vô hướng của vectơ dòng 2 và dòng 3 bằng 1 (chung đúng một từ “a”), còn của dòng 1 và dòng 3 bằng 0 (không chung từ nào). Chú ý rằng mô hình này bỏ qua hoàn toàn *thứ tự* từ: “I met a pig” và “a pig met I” cho cùng một vectơ — giống như đổ các từ vào một cái “túi” rồi chỉ đếm số lượng, đó là nguồn gốc của tên gọi.

Mô hình túi từ định nghĩa một không gian vectơ (*vector space*)[8]. Trong không gian này, độ tương đồng giữa hai vectơ được đo bằng các độ đo như tích vô hướng. Điều này giúp thiết lập quan hệ giữa các tài liệu: hai tài liệu có nhiều từ trùng nhau hơn được coi là giống nhau hơn. Khái niệm trực giác này là nền tảng của nhiều hệ thống phân loại văn bản.

Vẻ đẹp của mô hình túi từ đến từ dạng đơn giản của nó, ở chỗ mỗi từ độc lập với các từ khác. Giả định độc lập này cho phép mã hóa một tài liệu với vô số quan hệ giữa các từ thành một vectơ đếm được, có kích thước khả thi. Mặt khác, biểu diễn một tài liệu bằng vectơ đặc trưng gồm số lần xuất hiện của từ không phải là lựa chọn duy nhất. Để cải tiến, ta có thể tính thêm nhiều ngữ cảnh hơn, và/hoặc thiết kế các đặc trưng mạnh hơn. Điều này dẫn tới một hướng nghiên cứu sôi động về các phương pháp biểu diễn văn bản, từ các phương pháp dựa trên kinh nghiệm (heuristic) đến các phương pháp học biểu diễn (*representation learning*). Chúng ta sẽ thấy một vài phương pháp trong số đó ở các chương sau.

1.2.3 Bộ phân loại tuyến tính

Bộ phân loại tuyến tính (*linear classifier*) là một trong những mô hình nền tảng nhất của học máy. Cho $\mathbf{x} \in \mathbb{R}^n$ là một vectơ đặc trưng đầu vào, và $\mathbf{w} \in \mathbb{R}^n$, $b \in \mathbb{R}$ lần lượt là vectơ trọng số (*weight*) và tham số độ lệch (bias) (*bias*). Một hàm tính điểm tuyến tính được định nghĩa là:

$$\begin{aligned} s(\mathbf{x}, \mathbf{w}, b) &= \mathbf{w} \cdot \mathbf{x} + b \\ &= \sum_{i=1}^{n} w_i x_i + b. \end{aligned} \tag{1.43}$$

Để đơn giản hóa ký hiệu, ta thường dùng **mẹo gộp độ lệch** (*bias trick*), định nghĩa một vectơ đầu vào mở rộng $\mathbf{x}' = [\mathbf{x}; 1]$ và vectơ trọng số $\mathbf{w}' = [\mathbf{w}; b]$. Điều này cho phép viết Phương trình (1.43) gọn lại dưới dạng một tích vô hướng:

$$s(\mathbf{x}', \mathbf{w}') \;=\; \mathbf{w}' \cdot \mathbf{x}'. \tag{1.44}$$

[8] Một không gian vectơ phải đóng đối với phép cộng vectơ và phép nhân với số vô hướng.

Từ đây trở đi, chúng tôi sẽ bỏ số hạng độ lệch tường minh và gọi chung các tham số là $\mathbf{w}$.

Chú giải của người dịch: Hàm tính điểm tuyến tính và mẹo gộp độ lệch

Hàm $s = \mathbf{w}\cdot\mathbf{x}+b$ là phiên bản nhiều chiều của hàm bậc nhất $y = ax+b$ quen thuộc. Mỗi trọng số w_i cho biết đặc trưng x_i "đóng góp" bao nhiêu vào điểm số: w_i dương lớn thì từ thứ i càng ủng hộ lớp đang xét, w_i âm thì ngược lại. Ví dụ: $\mathbf{x} = [2\ \ 1]$, $\mathbf{w} = [0.5\ \ -1]$, $b = 3$ cho $s = 0.5\cdot 2 + (-1)\cdot 1 + 3 = 3$. Mẹo gộp độ lệch: nối thêm số 1 vào cuối $\mathbf{x}$ và b vào cuối $\mathbf{w}$: $\mathbf{x}' = [2\ \ 1\ \ 1]$, $\mathbf{w}' = [0.5\ \ -1\ \ 3]$, khi đó $\mathbf{w}'\cdot\mathbf{x}' = 1 - 1 + 3 = 3$ — cùng kết quả, nhưng công thức chỉ còn một tích vô hướng. Ký hiệu $[\mathbf{x}; 1]$ nghĩa là "ghép thêm phần tử 1 vào sau $\mathbf{x}$".

Trong bối cảnh phân loại, hàm tuyến tính định lượng mức độ "phù hợp" giữa một đầu vào và một lớp. Với phân loại nhị phân có các lớp $\mathcal{C} = \{c_a, c_b\}$, ta gắn với mỗi lớp một vectơ trọng số, ký hiệu là $\mathbf{w}_a$ và $\mathbf{w}_b$.

Để thực hiện dự đoán, ta ánh xạ các điểm số liên tục thành các nhãn lớp rời rạc bằng một hàm quyết định (*decision function*) (thường được gọi là **hàm kích hoạt** (*activation function*) hay **hàm ngưỡng** (*threshold function*)), $\psi(\cdot)$. Với phân loại nhị phân, quy tắc quyết định thường là:

$$\begin{aligned}\hat{c} &= \psi\big(s(\mathbf{x}, \mathbf{w}_a) - s(\mathbf{x}, \mathbf{w}_b)\big) \\ &= \begin{cases} c_a & s(\mathbf{x}, \mathbf{w}_a) > s(\mathbf{x}, \mathbf{w}_b) \\ c_b & \text{trường hợp ngược lại} \end{cases}\end{aligned} \tag{1.45}$$

Do tính tuyến tính, $s(\mathbf{x}, \mathbf{w}_a) - s(\mathbf{x}, \mathbf{w}_b) = s(\mathbf{x}, \mathbf{w}_a - \mathbf{w}_b)$. Biểu thức $f(\mathbf{x}) = s(\mathbf{x}, \mathbf{w}_a - \mathbf{w}_b)$ đóng vai trò là **hàm phân biệt tuyến tính** (*linear discriminant function*).

Dù các hàm phân biệt dự đoán trực tiếp nhãn lớp, bản thân chúng không cho ra các ước lượng xác suất. Để có đầu ra dạng xác suất (tức là $\Pr(c|x)$), ta phải ánh xạ các điểm số thành một phân phối xác suất hợp lệ bằng một hàm chuẩn hóa $\psi(\cdot)$, chẳng hạn hàm Softmax:

$$\big[\Pr(c_a|\mathbf{x}) \quad \Pr(c_b|\mathbf{x})\big] = \left[\frac{\exp(s(\mathbf{x}, \mathbf{w}_a))}{\exp(s(\mathbf{x}, \mathbf{w}_a)) + \exp(s(\mathbf{x}, \mathbf{w}_b))} \quad \frac{\exp(s(\mathbf{x}, \mathbf{w}_b))}{\exp(s(\mathbf{x}, \mathbf{w}_a)) + \exp(s(\mathbf{x}, \mathbf{w}_b))}\right] \tag{1.46}$$

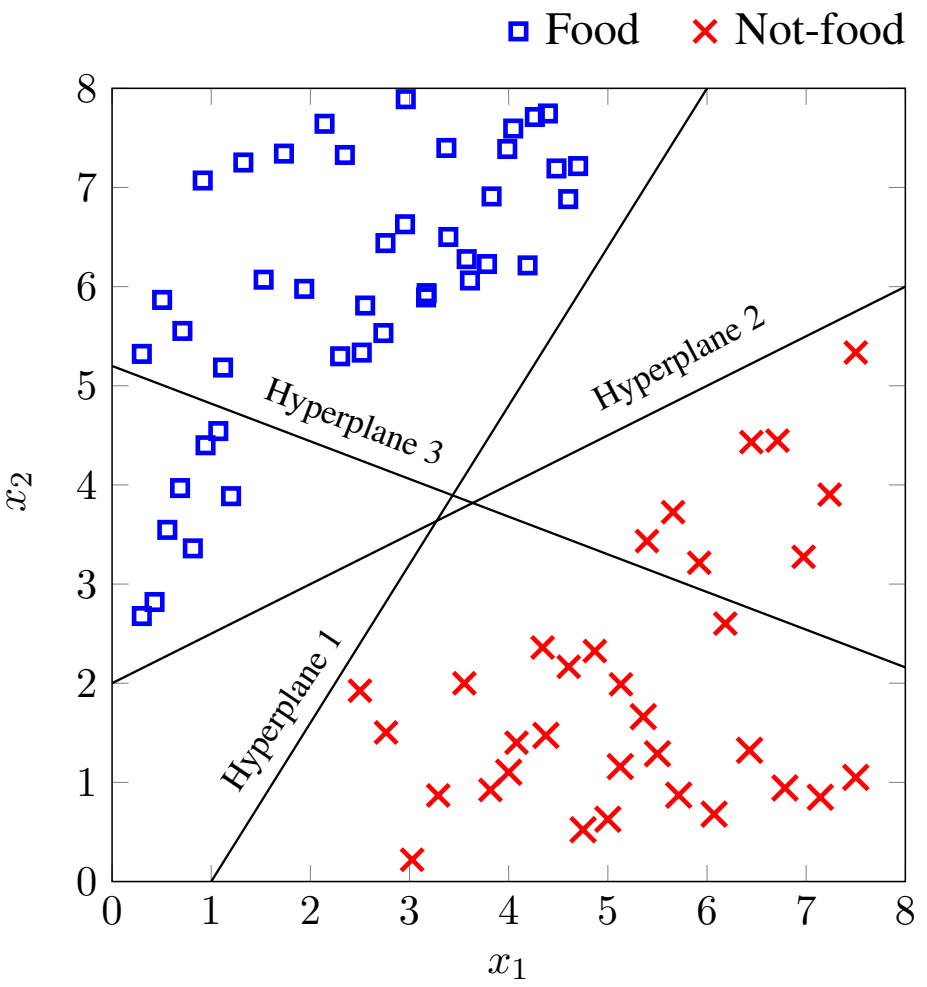
Theo cách phát biểu này, dự đoán lớp có xác suất cao nhất vẫn tương đương với dự đoán lớp có điểm số cao nhất, nhất quán với Phương trình (1.45).

Chú giải của người dịch: Hàm Softmax biến điểm số thành xác suất

$\exp(s)$ nghĩa là e^s với $e \approx 2.718$. Vì $e^s > 0$ với mọi s, và mỗi phần tử được chia cho tổng tất cả, nên các kết quả đều dương và cộng lại bằng 1 — tức là một phân phối xác suất hợp lệ. Ví dụ, điểm số $s_a = 2$, $s_b = 0$: $\Pr(c_a|\mathbf{x}) = \frac{e^2}{e^2+e^0} = \frac{7.389}{8.389} \approx 0.881$ và $\Pr(c_b|\mathbf{x}) \approx 0.119$. Vì e^s là hàm đồng biến, lớp có điểm cao hơn luôn có xác suất cao hơn, nên quyết định không thay đổi so với Phương trình (1.45).

Về mặt hình học, một bộ phân loại tuyến tính xác định một **siêu phẳng phân tách** (*separating hyperplane*) (hay **biên quyết định** (*decision boundary*)) trong không gian đặc trưng. Hình 1.1 minh họa điều này trong không gian hai chiều. Mục tiêu của việc học là tìm ra một siêu phẳng (như Siêu phẳng 1 và 2) tách đúng các điểm dữ liệu theo lớp, tránh các lỗi mà Siêu phẳng 3 mắc phải.

Cần lưu ý rằng tính tuyến tính là nền tảng của nhiều bộ phân loại, mặc dù phần lớn chúng không có cùng dạng với Phương trình (1.43–1.44). Mô hình tuyến tính là lý tưởng cho các bài toán **tách được tuyến tính** (*linearly separable*). Với dữ liệu không tách được tuyến tính, mô hình tuyến tính thường được kết hợp với các thành phần phi tuyến, chẳng hạn các phép biến đổi đặc trưng phi tuyến hoặc các hàm kích hoạt trong mạng nơ-ron (*neural network*), để nắm bắt những biên quyết định phức tạp hơn.



Hình 1.1: Các điểm dữ liệu và các siêu phẳng phân tách trong không gian hai chiều. Có hai lớp điểm dữ liệu (đồ ăn và không phải đồ ăn). Cả siêu phẳng 1 và 2 đều chia không gian thành hai không gian con, trong đó hai lớp điểm dữ liệu được tách riêng. Theo nghĩa này, bài toán ở đây là tách được tuyến tính. Ngược lại, siêu phẳng 3 không tách được hai lớp, tức là các điểm dữ liệu trong cùng một lớp bị phân vào hai lớp khác nhau. (Chú giải nhãn trong hình: *Food* = Đồ ăn, *Not-food* = Không phải đồ ăn, *Hyperplane* = Siêu phẳng.)

Chú giải của người dịch: Siêu phẳng là gì?

Trong mặt phẳng hai chiều, tập các điểm thỏa mãn $w_1x_1 + w_2x_2 + b = 0$ là một *đường thẳng* (bạn đã học phương trình tổng quát $ax + by + c = 0$ ở lớp 10). Trong không gian ba chiều, đó là một *mặt phẳng*. Trong không gian n chiều, ta gọi chung là *siêu phẳng*. Đường thẳng này chia mặt phẳng thành hai nửa: điểm nào làm $\mathbf{w} \cdot \mathbf{x} + b > 0$ thì thuộc một lớp, < 0 thì thuộc lớp kia. "Học" một bộ phân loại tuyến tính chính là tìm các hệ số $\mathbf{w}, b$ sao cho đường phân chia này đặt đúng chỗ, như Siêu phẳng 1 hay 2 trong Hình 1.1.

1.2.4 Mô hình sinh và mô hình phân biệt

Các cách tiếp cận phân loại thường được chia thành hai trường phái: **mô hình sinh** (*generative models*) và **mô hình phân biệt** (*discriminative models*). Dù cả hai cách tiếp cận thường dẫn tới các biên quyết định tuyến tính, chúng khác nhau căn bản ở cách mô hình hóa phân phối dữ liệu bên dưới.

1. Mô hình sinh

Một mục tiêu của phân loại là học $\Pr(c|\mathbf{x})$. Mô hình sinh không mô hình hóa trực tiếp xác suất có điều kiện này. Thay vào đó, chúng mô hình hóa xác suất đồng thời $\Pr(\mathbf{x}, c)$, rồi dùng quy tắc Bayes (*Bayes' rule*) để tính $\Pr(c|\mathbf{x})$. Điều này được cho bởi phương trình sau.

$$\begin{aligned}\Pr(c|\mathbf{x}) &= \frac{\Pr(\mathbf{x}, c)}{\Pr(\mathbf{x})} \\ &= \frac{\Pr(c)\Pr(\mathbf{x}|c)}{\Pr(\mathbf{x})},\end{aligned} \tag{1.47}$$

trong đó $\Pr(\mathbf{x}, c)$ được viết lại thành $\Pr(\mathbf{x}|c)\Pr(c)$.

Chú giải của người dịch: Định lý Bayes: "lật ngược" xác suất có điều kiện

Phương trình (1.47) có được bằng cách áp dụng Phương trình (1.31) cho tử số: $\Pr(\mathbf{x}, c) = \Pr(\mathbf{x}|c)\Pr(c)$. Ý nghĩa: ta muốn biết $\Pr(c|\mathbf{x})$ ("biết văn bản, đoán nhãn"), nhưng thường dễ ước lượng chiều ngược lại $\Pr(\mathbf{x}|c)$ ("biết nhãn, văn bản trông thế nào"). Ví dụ: 30% email là thư rác, $\Pr(\text{rác}) = 0.3$. Từ "khuyến mãi" xuất hiện trong 60% thư rác và 5% thư thường. Một email có chữ "khuyến mãi", xác suất nó là thư rác là $\frac{0.3 \times 0.6}{0.3 \times 0.6 + 0.7 \times 0.05} = \frac{0.18}{0.215} \approx 0.84$. Mẫu số $\Pr(\mathbf{x}) = 0.215$ được tính bằng xác suất biên, Phương trình (1.30). $\Pr(c)$ gọi là xác suất *tiên nghiệm* (biết trước khi nhìn dữ liệu), còn $\Pr(c|\mathbf{x})$ là xác suất *hậu nghiệm* (sau khi đã thấy $\mathbf{x}$).

Để tìm lớp tối ưu $\hat{c}$, ta chọn lớp c làm cực đại $\Pr(c|\mathbf{x})$ (xem Phương trình (1.41)). Vì mẫu số $\Pr(\mathbf{x})$ là hằng số đối với lớp c, ta chỉ cần cực đại hóa tử số. Khi đó, ta viết lại Phương trình (1.41) dưới dạng

$$\begin{aligned}\hat{c} &= \arg\max_{c \in \mathcal{C}} \Pr(c|\mathbf{x}) \\ &= \arg\max_{c \in \mathcal{C}} \frac{\Pr(c)\Pr(\mathbf{x}|c)}{\Pr(\mathbf{x})} \\ &= \arg\max_{c \in \mathcal{C}} \Pr(c)\Pr(\mathbf{x}|c),\end{aligned} \tag{1.48}$$

trong đó $\Pr(c)$ là xác suất tiên nghiệm (*prior*) của c, và $\Pr(\mathbf{x}|c)$ là xác suất có điều kiện của vectơ tài liệu đầu vào $\mathbf{x}$ khi biết c. Tính $\Pr(c)$ khá đơn giản. Ví dụ, **ước lượng hợp lý cực đại (MLE)** (*maximum likelihood estimation*) định nghĩa $\Pr(c)$ là một tần suất tương đối:

$$\Pr(c) = \frac{\text{count}(c)}{\sum_{c' \in \mathcal{C}} \text{count}(c')}, \tag{1.49}$$

trong đó count(c) đếm số lần xuất hiện của c trong một kho ngữ liệu.

Chú giải của người dịch: Ước lượng hợp lý cực đại (MLE) bằng "đếm"

Ý tưởng của MLE: chọn các giá trị tham số làm cho dữ liệu ta đã quan sát được có xác suất xảy ra *lớn nhất*. Với các phân phối rời rạc như ở đây, MLE cho ra đúng điều trực giác mách bảo: xác suất = tần suất. Ví dụ, nếu kho ngữ liệu có 100 tài liệu, trong đó 40 có nhãn "Đồ ăn" và 60 có nhãn "Không phải đồ ăn", thì $\Pr(\text{Đồ ăn}) = 40/(40+60) = 0.4$. (Có thể kiểm chứng: nếu tung đồng xu 10 lần được 7 ngửa, hàm $p^7(1-p)^3$ đạt cực đại tại $p = 0.7$ — tính bằng đạo hàm như ở lớp 12.)

Tuy nhiên, ước lượng $\Pr(\mathbf{x}|c)$ là việc khó khăn, vì tính thưa của dữ liệu (*data sparsity*) khiến ta không thể ước lượng chính xác với các vectơ nhiều chiều. Nhắc lại rằng mô hình túi từ định nghĩa x_i là tần số của từ V_i trong tài liệu. Ở đây ta giả định rằng vectơ đặc trưng $\mathbf{x} = [x_1, \cdots, x_n]$ được sinh ra bởi một phân phối đa thức (*multinomial distribution*) với các tham số $(p_1(c), \cdots, p_n(c))$, trong đó $p_i(c)$ là xác suất từ V_i xuất hiện trong lớp c. Dựa trên MLE, ta ước lượng $p_i(c)$ là:

$$p_i(c) \;=\; \frac{\text{count}(V_i, c)}{\sum_{1 \le i' \le n} \text{count}(V_{i'}, c)}, \tag{1.50}$$

trong đó count(V_i, c) là số lần V_i xuất hiện trong tất cả các tài liệu có nhãn c. Khi đó, $\Pr(\mathbf{x}|c)$ được cho bởi

$$\Pr(\mathbf{x}|c) \;=\; \frac{\left(\sum_{i=1}^n x_i\right)!}{x_1! \cdot x_2! \cdot \cdots \cdot x_n!} \cdot \prod_{i=1}^n p_i(c)^{x_i}. \tag{1.51}$$

Thay Phương trình (1.51) vào Phương trình (1.48), ta có

$$\hat{c} \;=\; \arg\max_{c \in \mathcal{C}} \; \Pr(c) \cdot \frac{\left(\sum_{i=1}^n x_i\right)!}{x_1! \cdot x_2! \cdot \cdots \cdot x_n!} \cdot \prod_{i=1}^n p_i(c)^{x_i}. \tag{1.52}$$

Chú giải của người dịch: Giải nghĩa Phương trình (1.51)

Ký hiệu $\prod_{i=1}^n$ là "tích" (giống $\sum$ nhưng nhân thay vì cộng). Hãy hình dung một tài liệu được "sinh ra" bằng cách rút lần lượt từng từ, mỗi lần rút từ V_i với xác suất $p_i(c)$ (như gieo một con xúc xắc n mặt không cân đối). Xác suất rút ra một *thứ tự* cụ thể gồm x_1 lần từ V_1, …, x_n lần từ V_n là $\prod_i p_i(c)^{x_i}$. Vì mô hình túi từ không quan tâm thứ tự, ta nhân thêm số cách sắp xếp, chính là số *hoán vị lặp* đã học ở THPT: $\frac{(x_1+\cdots+x_n)!}{x_1!\cdots x_n!}$. Ví dụ nhỏ: chỉ có 2 từ với $p_1 = 0.6, p_2 = 0.4$; tài liệu gồm 2 lần từ 1 và 1 lần từ 2 thì $\Pr = \frac{3!}{2!\,1!} \cdot 0.6^2 \cdot 0.4 = 3 \cdot 0.144 = 0.432$. Đây là mở rộng của công thức Bernoulli (nhị thức) ra nhiều kết quả.

Lưu ý rằng $\frac{(\sum_{i=1}^n x_i)!}{x_1! \cdot x_2! \cdot \cdots \cdot x_n!}$ là hằng số đối với biến lớp c. Ta bỏ nó đi trong arg max, và viết lại vế phải của phương trình theo thang logarit:

$$\hat{c} \;=\; \arg\max_{c \in \mathcal{C}} \; \log(\Pr(c)) + \sum_{i=1}^n x_i \cdot \log(p_i(c)). \tag{1.53}$$

Rõ ràng, đây là một mô hình tuyến tính. Nó định nghĩa vectơ đặc trưng và vectơ trọng số như sau

$$\mathbf{x} = \begin{bmatrix} 1 & x_1 & \cdots & x_n \end{bmatrix} \tag{1.54}$$

$$\mathbf{w} = \begin{bmatrix} \log(\Pr(c)) & \log(p_1(c)) & \cdots & \log(p_n(c)) \end{bmatrix}. \tag{1.55}$$

Dạng như vậy đôi khi được gọi là mô hình **log-tuyến tính** (*log-linear*), vì tính tuyến tính có được nhờ biến đổi bài toán ban đầu bằng một phép biến đổi logarit.

Chú giải của người dịch: Vì sao lấy logarit lại không đổi kết quả?

Hàm log đồng biến, nên số nào lớn hơn thì logarit của nó cũng lớn hơn: arg max của f và của $\log f$ là như nhau. Logarit biến tích thành tổng ($\log(ab) = \log a + \log b$) và lũy thừa thành phép nhân ($\log p^x = x \log p$), nên tích ở Phương trình (1.52) trở thành tổng ở Phương trình (1.53). Tổng này đúng bằng tích vô hướng $\mathbf{w} \cdot \mathbf{x}$ với $\mathbf{x}$, $\mathbf{w}$ ở Phương trình (1.54–1.55) — tức là một hàm tính điểm tuyến tính như Phương trình (1.44), với phần tử "1" và $\log \Pr(c)$ đóng vai trò độ lệch. Ngoài ra, trên máy tính, nhân hàng trăm xác suất nhỏ dễ cho kết quả quá gần 0 (tràn dưới), còn cộng các logarit thì an toàn.

Nói chung, Phương trình (1.53) được gọi là phương pháp **Bayes ngây thơ đa thức** (*multinomial naive Bayes*). Tất nhiên, cũng có các biến thể Bayes ngây thơ cho các loại vectơ đặc trưng khác. Ví dụ, với vectơ đặc trưng nhận giá trị nhị phân, ta có thể giả định một phân phối Bernoulli trên mỗi phần tử của vectơ và thiết kế một bộ phân loại **Bayes ngây thơ Bernoulli** (*Bernoulli naive Bayes*); với các vectơ có đặc trưng liên tục, ta có thể giả định một phân phối Gauss trên dữ liệu liên tục và thiết kế một bộ phân loại **Bayes ngây thơ Gauss** (*Gaussian naive Bayes*).

Chú giải của người dịch: Vì sao gọi là "ngây thơ"?

Mô hình giả định rằng, khi đã biết lớp c, mỗi từ xuất hiện *độc lập* với các từ khác: xác suất của cả tài liệu chỉ là tích các xác suất của từng từ. Thực tế điều này không đúng ("bánh" và "mì" hay đi cùng nhau), nên giả định này bị coi là "ngây thơ". Dù vậy, Bayes ngây thơ hoạt động khá tốt trong phân loại văn bản, rất nhanh và dễ cài đặt, chỉ cần đếm. Ví dụ minh họa: nếu trong các tài liệu "Đồ ăn" từ "ngon" chiếm 2% tổng số từ, còn trong tài liệu "Không phải đồ ăn" chỉ chiếm 0.1%, thì mỗi lần "ngon" xuất hiện, điểm $\log p_i(c)$ cộng thêm cho lớp "Đồ ăn" sẽ cao hơn hẳn, kéo quyết định về phía lớp này.

2. Mô hình phân biệt

Mô hình được định nghĩa bởi $\Pr(c|\mathbf{x}) = \frac{\Pr(\mathbf{x},c)}{\Pr(\mathbf{x})} = \frac{\Pr(c)\Pr(\mathbf{x}|c)}{\Pr(\mathbf{x})}$ (xem Phương trình (1.47)) được gọi là mô hình sinh vì nó giả định một quá trình cụ thể để sinh ra dữ liệu $\mathbf{x}$ khi biết nhãn c. Ý tưởng là dùng xác suất đồng thời $\Pr(\mathbf{x}, c)$ làm bước trung gian để tính $\Pr(c|\mathbf{x})$. Ngược lại, **mô hình phân biệt** không cố mô hình hóa phân phối của $\mathbf{x}$. Thay vào đó, chúng ước lượng trực tiếp $\Pr(c|\mathbf{x})$.

Một ví dụ là **hồi quy logistic** (*logistic regression*). Nhắc lại rằng với phân loại văn bản nhị phân, một bộ phân loại Bayes ngây thơ dự đoán nhãn c_a cho một tài liệu $\mathbf{x}$ chỉ khi hàm log-odds (*log-odds*) dương:

$$f_a(\mathbf{x}) \;=\; \log \frac{\Pr(c_a|\mathbf{x})}{\Pr(c_b|\mathbf{x})}. \tag{1.56}$$

Ta có thể giả định rằng đại lượng này tuân theo một mô hình tuyến tính:

$$\log \frac{\Pr(c_a|\mathbf{x})}{\Pr(c_b|\mathbf{x})} \;=\; \mathbf{w} \cdot \mathbf{x}. \tag{1.57}$$

Vì $\Pr(c_b|\mathbf{x}) = 1 - \Pr(c_a|\mathbf{x})$, giải theo $\Pr(c_a|\mathbf{x})$ ta được:

$$\Pr(c_a|\mathbf{x}) \;=\; \frac{1}{1 + \exp(-\mathbf{w} \cdot \mathbf{x})}. \tag{1.58}$$

Đây là một hàm sigmoid (*sigmoid function*), cụ thể là hàm logistic (*logistic function*). Theo mô hình này, ta dự đoán c_a chỉ khi $\mathbf{w} \cdot \mathbf{x} > 0$. Phương trình (1.58) định nghĩa bộ phân loại hồi quy logistic, đóng vai trò là phiên bản phân biệt tương ứng với bộ phân loại Bayes ngây thơ.

Chú giải của người dịch: Từ log-odds đến hàm sigmoid

Đặt $p = \Pr(c_a|\mathbf{x})$ và $z = \mathbf{w} \cdot \mathbf{x}$. Phương trình (1.57) là $\log \frac{p}{1-p} = z$, suy ra $\frac{p}{1-p} = e^z$, tức $p = e^z(1-p)$, nên $p(1+e^z) = e^z$ và $p = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}}$ — đúng Phương trình (1.58). Hàm $\sigma(z) = \frac{1}{1+e^{-z}}$ có đồ thị hình chữ S, "nén" mọi số thực vào khoảng $(0,1)$: $\sigma(0) = 0.5$, $\sigma(2) \approx 0.88$, $\sigma(-2) \approx 0.12$. Vì vậy $p > 0.5$ khi và chỉ khi $z > 0$. "Odds" (tỉ lệ cược) $\frac{p}{1-p}$ là cách nói quen thuộc trong cá cược: $p = 0.75$ ứng với odds 3 ("ba ăn một").

Một ưu điểm của mô hình phân biệt là sự linh hoạt khi phát biểu các bài toán phân loại. Trong khi mô hình sinh cố gắng ước lượng phân phối dữ liệu của $\mathbf{x}$, mô hình phân biệt tập trung vào việc thiết lập một ranh giới vững chắc giữa các lớp. Do đó, mô hình phân biệt quan tâm nhiều hơn tới việc lớp nào được ưu tiên hơn lớp nào khi biết $\mathbf{x}$, hay theo ngôn ngữ xác suất, lớp nào có khả năng xuất hiện cao hơn, thay vì đưa ra các giả định mạnh về từng điểm dữ liệu riêng lẻ. Điều này cho phép học mô hình bằng cách cực tiểu hóa một độ đo lỗi cụ thể, thay vì chỉ cực đại hóa hàm hợp lý (*likelihood*) của các điểm dữ liệu đầu vào. Cách tiếp cận này thuộc khuôn khổ **học dựa trên lỗi** (*error-driven learning*).

Có nhiều cách định nghĩa các lỗi này. Tương tự như mô hình sinh, ta có thể huấn luyện một mô hình phân biệt bằng cách tối ưu các tham số $\mathbf{w}$ để cực đại hóa hợp lý có điều kiện (*conditional likelihood*) của dữ liệu huấn luyện. Cho $\{(\mathbf{x}^{(1)}, c^{(1)}), \cdots, (\mathbf{x}^{(K)}, c^{(K)})\}$ là một tập các tài liệu có nhãn. Vectơ tham số tối ưu $\hat{\mathbf{w}}$ được cho bởi:

$$\hat{\mathbf{w}} \;=\; \arg\max_{\mathbf{w}} \sum_{k=1}^{K} \log \Pr(c^{(k)}|\mathbf{x}^{(k)}). \tag{1.59}$$

Áp dụng vào Phương trình (1.58), quá trình này tương ứng với việc cực đại hóa log-hợp lý (*log-likelihood*):

$$\begin{aligned}
\hat{\mathbf{w}} &= \arg\max_{\mathbf{w}} \sum_{k=1}^{K} \Big[\delta(c_a, c^{(k)}) \log \Pr(c_a|\mathbf{x}^{(k)}) + \delta(c_b, c^{(k)}) \log \Pr(c_b|\mathbf{x}^{(k)})\Big] \\
&= \arg\max_{\mathbf{w}} \sum_{k=1}^{K} \Bigg[\delta(c_a, c^{(k)}) \log \Big(\frac{1}{1+\exp(-\mathbf{w}\cdot\mathbf{x}^{(k)})}\Big) + \\
&\qquad\qquad \delta(c_b, c^{(k)}) \log \Big(1 - \frac{1}{1+\exp(-\mathbf{w}\cdot\mathbf{x}^{(k)})}\Big)\Bigg],
\end{aligned} \tag{1.60}$$

trong đó $\delta(\cdot,\cdot)$ là một **hàm chỉ thị** (*indicator function*), trả về 1 nếu hai đối số bằng nhau và 0 trong trường hợp ngược lại.

Chú giải của người dịch: Đọc Phương trình (1.59–1.60)

Chỉ số trên (k) trong $\mathbf{x}^{(k)}$ là *số thứ tự của mẫu* thứ k trong dữ liệu huấn luyện, không phải lũy thừa. Phương trình (1.59) nói: hãy chọn $\mathbf{w}$ sao cho mô hình gán xác suất càng cao càng tốt cho *nhãn đúng* của mọi mẫu. Lấy log rồi cộng lại tương đương với nhân các xác suất (xem chú giải ở Phương trình (1.53)). Trong Phương trình (1.60), hàm δ đóng vai trò "công tắc": với mỗi mẫu, đúng một trong hai số hạng được giữ lại — số hạng ứng với nhãn đúng của mẫu đó. Ví dụ, nếu mẫu k có nhãn c_a và mô hình cho $\Pr(c_a|\mathbf{x}^{(k)}) = 0.9$, mẫu đóng góp $\log 0.9 \approx -0.105$; nếu mô hình chỉ cho 0.1 thì đóng góp $\log 0.1 \approx -2.303$, bị "phạt" nặng hơn nhiều. Đổi dấu đại lượng này, ta được đúng *entropy chéo* đã gặp ở Mục 1.1.2.

Một cách khác là huấn luyện mô hình bằng cách cực tiểu hóa hàm mất mát 0-1 (*0-1 loss*), tức là ta

tính một lỗi mỗi khi nhãn dự đoán không khớp với nhãn đúng:

$$\hat{\mathbf{w}} \;=\; \arg\min_{\mathbf{w}} \sum_{k=1}^{K} \big(1 - \delta(\hat{c}^{(k)}, c^{(k)})\big), \tag{1.61}$$

trong đó $\hat{c}^{(k)}$ là nhãn dự đoán. Dù chặt chẽ về mặt toán học, việc tối ưu trực tiếp hàm mất mát 0-1 rất khó vì nó không khả vi, dẫn tới việc dùng các hàm mất mát thay thế (*surrogate losses*) như log-hợp lý nói trên.

Chú giải của người dịch: Vì sao "không khả vi" lại gây khó?

Hàm mất mát 0-1 chỉ đếm số mẫu bị đoán sai, nên nó nhận giá trị nguyên và "nhảy bậc": thay đổi $\mathbf{w}$ một chút thường không làm số lỗi thay đổi (đạo hàm bằng 0), còn khi thay đổi thì nhảy đột ngột (không có đạo hàm). Các phương pháp huấn luyện phổ biến (sẽ học ở các chương sau) dựa vào đạo hàm để biết nên chỉnh $\mathbf{w}$ theo hướng nào, nên cần một hàm "trơn" như log-hợp lý, vốn thay đổi liên tục theo $\mathbf{w}$.

Việc lựa chọn mục tiêu huấn luyện có vai trò then chốt trong các mô hình phân biệt. Chủ đề này rất rộng và sẽ được bàn kỹ hơn ở Mục 1.3.4. Hơn nữa, mô hình phân biệt không đòi hỏi các đặc trưng phải tuân thủ chặt chẽ một quá trình sinh xác suất. $\mathbf{x}$ có thể là bất kỳ vectơ đặc trưng nào được thiết kế cho bài toán. Ví dụ, cho $g(\mathbf{x})$ là điểm số đầu ra của một hệ thống khác. Theo Phương trình (1.58), ta có thể xây dựng một mô hình phân loại nhị phân mới:

$$\Pr(c_a|\mathbf{x}) \;=\; \frac{1}{1 + \exp(-\mathbf{w} \cdot g(\mathbf{x}))}. \tag{1.62}$$

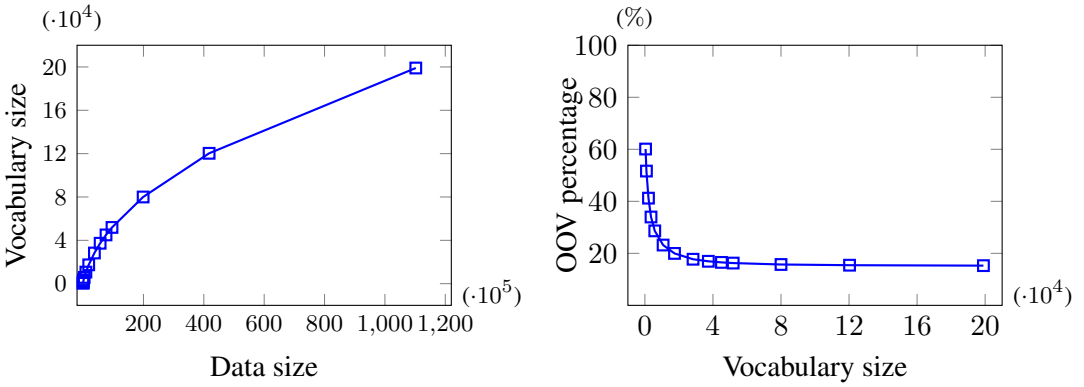
Để học $g(\mathbf{x})$, ta có thể tiền huấn luyện (*pre-train*) nó trên một số dữ liệu bổ sung, hoặc huấn luyện nó đồng thời với $\mathbf{w}$ (tức là các tham số của mô hình cấp trên $\Pr(c_a|\mathbf{x})$).

1.2.5 Từ ngoài bộ từ vựng và làm trơn

Vấn đề **từ ngoài bộ từ vựng** (*out-of-vocabulary*) (**OOV**) xảy ra khi một số từ của tài liệu không có trong bộ từ vựng. Từ OOV rất phổ biến trong NLP vì luôn có từ mới, bất kể ta đã thấy bao nhiêu văn bản. Hình 1.2 đưa ra hai đường cong minh họa vấn đề này. Như thể hiện trong Hình 1.2 (a), các từ mới liên tục xuất hiện khi có thêm dữ liệu. Khi ta cố định dữ liệu dùng để kiểm tra độ phủ của bộ từ vựng, từ OOV vẫn còn tồn tại ngay cả khi ta có một bộ từ vựng cực lớn (Hình 1.2 (b)).

Trong các hệ thống thực tế, từ OOV rất phổ biến vì bộ từ vựng thường bị giới hạn ở một số lượng "nhỏ" mục từ. Một phương pháp chuẩn là giữ lại n từ có tần số cao nhất và loại bỏ phần còn lại. Khi đó, các từ OOV được coi là từ *không xác định* (*unknown*). Ví dụ, một ký hiệu mới <unk> được đưa vào bộ từ vựng để mọi từ OOV đều được ký hiệu là <unk>. Một ý tưởng táo bạo hơn là xây dựng một hệ thống **từ vựng mở** (*open-vocabulary*) chấp nhận mọi từ có thể có, nhưng điều này đòi hỏi các thuật toán phức tạp hơn và có lẽ cả việc thiết kế cấu trúc dữ liệu riêng cho từng bài toán. Việc dùng <unk> hiện vẫn là chuẩn mực trên thực tế trong việc phát triển các hệ thống NLP.

Với những từ đã có trong bộ từ vựng, cũng tồn tại các **từ chưa gặp** (*unseen words*): những từ vắng mặt trong giai đoạn ước lượng tham số nhưng lại xuất hiện trong một tài liệu mới. Một cách cài đặt ngây thơ mô hình mô tả ở mục trước sẽ gặp khó khi xử lý các từ chưa gặp. Ví dụ, Phương trình (1.50) sẽ đơn giản cho xác suất bằng 0 nếu từ V_i không xuất hiện trong bất kỳ mẫu huấn luyện nào có nhãn c. Hệ quả là, với một mẫu mới chứa V_i, Phương trình (1.51) sẽ gán cho nó xác suất bằng 0. Kết quả này rõ ràng là bất hợp lý. Tuy nhiên, ta không nên đơn giản quy lỗi cho bản thân thiết kế mô hình. Thay vào đó, nguyên nhân chính là dữ liệu dùng để ước lượng tham số không đủ. Điều này cũng được giải thích



Hình 1.2: Kích thước dữ liệu (tính theo số từ), kích thước bộ từ vựng và tỉ lệ phần trăm OOV. Số liệu thống kê được thu thập trên dữ liệu tiếng Anh của bài toán dịch Trung–Anh (Zh-En) WMT21. Ta dùng càng nhiều dữ liệu thì bộ từ vựng càng lớn. Kích thước bộ từ vựng vẫn tiếp tục tăng ngay cả khi ta xây dựng bộ từ vựng trên dữ liệu hơn 100 triệu từ. Tuy nhiên, độ dốc của đường cong có xu hướng nhỏ dần khi dùng thêm nhiều dữ liệu. Điều thú vị là tỉ lệ OOV hội tụ về một mức nhất định khi kích thước bộ từ vựng tăng, cho thấy luôn có từ mới xuất hiện bất kể ta đã quan sát bao nhiêu từ. (Đồ thị trái là (a), đồ thị phải là (b). Nhãn trục: *Data size* = kích thước dữ liệu, *Vocabulary size* = kích thước bộ từ vựng, *OOV percentage* = tỉ lệ OOV; $(\cdot 10^4)$ nghĩa là giá trị trên trục cần nhân với 10^4.)

bởi **định luật Zipf** (*Zipf's Law*): một số ít từ xuất hiện rất thường xuyên, trong khi một số lượng lớn từ hiếm khi xuất hiện.

Chú giải của người dịch: Vì sao một số 0 làm hỏng cả tài liệu?

Phương trình (1.51) là một *tích* các $p_i(c)^{x_i}$. Chỉ cần một từ trong tài liệu có $p_i(c) = 0$ (với $x_i \geq 1$) thì cả tích bằng 0, bất kể các từ còn lại ủng hộ lớp c mạnh đến đâu. Ví dụ, một bài viết về ẩm thực dài 200 từ nhưng có một từ "phở cuốn" chưa từng xuất hiện trong dữ liệu huấn luyện lớp "Đồ ăn" sẽ có $\Pr(\mathbf{x}|\text{Đồ ăn}) = 0$ (và $\log 0 = -\infty$ trong Phương trình (1.53)) — rõ ràng là vô lý. Theo định luật Zipf, từ phổ biến thứ r có tần số xấp xỉ tỉ lệ với $1/r$: từ đứng đầu xuất hiện gấp đôi từ thứ hai, gấp ba từ thứ ba… nên luôn có một "cái đuôi" rất dài gồm các từ hiếm.

Tuy nhiên, ta không thể giả định rằng mình luôn có được dữ liệu trong đó mọi từ đều xuất hiện đủ nhiều. Thay vào đó, ta có thể dùng các kỹ thuật làm trơn (*smoothing*) để phân phối lại xác suất trên bộ từ vựng. Xét Phương trình (1.50) làm ví dụ. Ta có thể cộng thêm một số nhỏ cho mỗi V_i, và viết lại phương trình thành:

$$p_i(c) = \frac{\text{count}(V_i, c) + \alpha}{\sum_{1 \leq i' \leq n} \text{count}(V_{i'}, c) + n \cdot \alpha}, \tag{1.63}$$

trong đó α là số đếm mặc định ta gán cho mỗi từ. Bằng cách này, $p_i(c)$ cho xác suất khác 0 ngay cả khi $\text{count}(V_i, c) = 0$. Phương trình (1.63) chuyển một phần khối lượng xác suất từ các từ tần số cao sang các từ tần số thấp. Phương pháp này được gọi là **làm trơn cộng** (*additive smoothing*) hay **làm trơn cộng-α** (*add-α smoothing*). Đây là một trong những phương pháp làm trơn đơn giản nhất. Về các phương pháp khác, chúng tôi giới thiệu bạn đọc tới các bài báo về mô hình hóa ngôn ngữ, nơi làm trơn được dùng rất nhiều [Chen và Goodman, 1999].

Chú giải của người dịch: Ví dụ tính tay làm trơn cộng-α

Giả sử bộ từ vựng có $n = 4$ từ, và trong các tài liệu lớp "Đồ ăn" ta đếm được: "ngon" 5 lần, "bếp" 3 lần, "trái cây" 2 lần, "xe buýt" 0 lần (tổng 10). Không làm trơn (Phương trình (1.50)): $p = (0.5;\ 0.3;\ 0.2;\ 0)$. Với $\alpha = 1$ (còn gọi là *làm trơn Laplace*), mẫu số thành $10 + 4 \cdot 1 = 14$: $p = (6/14;\ 4/14;\ 3/14;\ 1/14) \approx (0.43;\ 0.29;\ 0.21;\ 0.07)$. Tổng vẫn bằng 1, từ chưa gặp "xe buýt" nhận xác suất nhỏ nhưng khác 0, còn các từ phổ biến bị "bớt" đi một chút. Hạng tử $n \cdot \alpha$ ở mẫu số chính là để bù cho việc mỗi trong n từ được cộng thêm α.

Cần lưu ý rằng trong NLP hiện đại, vấn đề OOV truyền thống ít được bàn đến hơn. Như chúng tôi sẽ giới thiệu ở các chương sau, các hệ thống NLP gần đây thường dùng các phương pháp tách token theo **từ con (subword)** (*subword*). Trong các hệ thống này, đơn vị xử lý không còn là các từ theo nghĩa ngôn ngữ học truyền thống nữa, mà là các "đơn vị" được học bằng thống kê. Vì tuyệt đại đa số các từ chưa gặp đều có thể được ghép lại từ các đơn vị từ con này (trong trường hợp cực đoan, từ có thể được coi đơn giản là chuỗi các ký tự), việc dùng một bộ từ vựng từ con tương đối gọn có thể giảm đáng kể vấn đề OOV.

Chú giải của người dịch: Từ con giúp gì cho vấn đề OOV?

Ví dụ, từ tiếng Anh *unhappiness* có thể chưa có trong bộ từ vựng, nhưng có thể được tách thành các từ con quen thuộc "un" + "happi" + "ness". Như vậy mô hình vẫn "đọc" được từ mới thông qua các mảnh đã biết, thay vì phải thay cả từ bằng `<unk>`. Cách tách cụ thể (ví dụ thuật toán BPE) sẽ được trình bày ở các chương sau.

1.3 Các vấn đề chung

Xây dựng một bộ phân loại văn bản đơn giản là một khởi đầu tốt nhưng chưa đủ để giải quyết các bài toán thực tế phức tạp và đa dạng. Để có một bức tranh tổng quát hơn về cách các hệ thống học máy hiện đại vận hành, giờ đây chúng ta sẽ bàn về một số vấn đề quan trọng khi thiết kế những hệ thống như vậy.

1.3.1 Mô hình có giám sát và không giám sát

Học có giám sát (*supervised learning*) làm việc với các mẫu có nhãn, nghĩa là mỗi đầu vào x được gắn với một đầu ra y. Cho một tập các cặp đầu vào–đầu ra $\{(x^{(1)}, y^{(1)}), \cdots, (x^{(K)}, y^{(K)})\}$, nhiệm vụ ở đây là học một hàm $f(\cdot)$ ánh xạ mỗi $x^{(k)}$ thành $y^{(k)}$:

$$y^{(k)} \ = \ f(x^{(k)}). \tag{1.64}$$

Quá trình này được gọi là *có giám sát* vì việc học $f(\cdot)$ được dẫn dắt bởi câu trả lời $y^{(k)}$ do con người gán nhãn thủ công cho $x^{(k)}$. Nói chung, $y^{(k)}$ được gọi là nhãn đúng thực tế (*ground-truth*) hay nhãn chuẩn vàng (*gold-standard*) của $x^{(k)}$. Sau đó, khi có một đầu vào mới x_{new}, ta dùng hàm $f(\cdot)$ đã học để dự đoán đầu ra. Ta nói rằng việc học có giám sát thành công nếu dự đoán $f(x_{\text{new}})$ khớp với nhãn đúng y_{new}.

Tuyệt đại đa số các bài toán NLP có thể được phát biểu thành bài toán học có giám sát. Gán một lớp cho một tài liệu chắc chắn là một trong những trường hợp đơn giản nhất. Các bài toán NLP khác bao gồm (nhưng không giới hạn ở) việc sinh ra một chuỗi nhãn, một đoạn văn bản, một cây cú pháp hay một đồ thị.

Trái với học có giám sát, **học không giám sát** (*unsupervised learning*) làm việc với các mẫu không có nhãn; nói cách khác, với mỗi mẫu, ta có đầu vào x nhưng không có đầu ra đúng y. Trong trường hợp này, ta cần một thuật toán học từ dữ liệu không nhãn $\{x^{(k)}\}$ một hàm ánh xạ từ $x^{(k)}$ sang một đầu ra nào đó. Vì không có sự can thiệp của con người vào đầu ra của hàm, các thuật toán loại này cần tự phát hiện các khuôn mẫu trong $\{x^{(k)}\}$ và tối ưu cách ta biểu diễn $\{x^{(k)}\}$ cùng đầu ra của hàm theo một tiêu chí nào đó. Các tiêu chí này thường xuất phát từ tri thức có trước của con người, để hàm thu được có thể cho ra thứ mà ta mong đợi.

Một ví dụ phổ biến trong học không giám sát là **phân cụm từ** (*word clustering*). Nó gom nhóm một tập các từ bằng cách xếp các từ tương tự vào cùng một cụm[9]. Dựa trên tiêu chí như vậy, ta có thể hướng $f(\cdot)$ tới việc cho ra cùng một cụm với các từ tương tự nhau. Một trường hợp khó hơn là **quy nạp từ điển song ngữ không giám sát** (*unsupervised bilingual dictionary induction*). Nó học một ánh xạ ở cấp độ từ giữa hai ngôn ngữ mà không cần dữ liệu song song (*parallel data*). Bài toán này thường được giải quyết (một phần) bằng cách tận dụng tính đẳng cấu (*isomorphism*) giữa các không gian biểu diễn từ của các ngôn ngữ khác nhau.

Chú giải của người dịch: Phân cụm từ và từ điển song ngữ không giám sát

Phân cụm giống như việc xếp sách vào các kệ mà không ai cho trước tên kệ: máy tự nhận ra rằng "táo, cam, chuối" nên ở cùng một nhóm, còn "ô tô, xe máy, xe đạp" ở một nhóm khác. *Dữ liệu song song* là các cặp câu cùng nghĩa ở hai ngôn ngữ (ví dụ một câu tiếng Việt và bản dịch tiếng Anh của nó). Ý tưởng "đẳng cấu" ở đây là: nếu biểu diễn mỗi từ bằng một điểm trong không gian, thì "hình dạng" của đám điểm tiếng Việt và đám điểm tiếng Anh khá giống nhau ("mèo" gần "chó" giống như *cat* gần *dog*). Vì vậy, ta có thể tìm cách xoay, khớp hai đám điểm vào nhau để suy ra từ nào dịch thành từ nào, mà không cần bất kỳ cặp câu dịch nào.

Nằm giữa học có giám sát và học không giám sát là **học bán giám sát** (*semi-supervised learning*). Nó xử lý những tình huống trong đó chỉ một phần nhỏ dữ liệu đầu vào có nhãn, còn phần lớn vẫn không có nhãn. Như vậy, nó tận dụng được ưu điểm của cả hai mô thức. Ví dụ, trong dịch máy (*machine translation*), ta có thể có một lượng dữ liệu song song (có nhãn) nhất định và lượng dữ liệu đơn ngữ (không nhãn) lớn hơn nhiều bậc độ lớn. Thông thường, một hệ thống cơ sở được huấn luyện trên dữ liệu song song theo cách có giám sát. Sau đó có thể cải thiện hệ thống bằng cách huấn luyện các thành phần của hệ thống cơ sở này trên dữ liệu đơn ngữ quy mô lớn. Điều này được thực hiện hoặc bằng cách kết hợp một mô hình dịch học từ dữ liệu song ngữ với một mô hình ngôn ngữ (*language model*) học từ dữ liệu đơn ngữ của ngôn ngữ đích, hoặc bằng cách tiền huấn luyện (*pre-training*) một số phần của mô hình trên dữ liệu đơn ngữ rồi tinh chỉnh (*fine-tuning*) toàn bộ mô hình trên dữ liệu song ngữ.

Nói rộng ra, mọi thuật toán học đều cần sự giám sát. Điều này nghe có vẻ trái trực giác vì học không giám sát dường như không nhận được tín hiệu nào từ dữ liệu nhãn đúng. Tuy nhiên, từ góc nhìn tổng quát về việc học, tín hiệu giám sát không nên bị giới hạn ở các nhãn do con người cung cấp. Ngay cả trong một bài toán học không giám sát, quá trình học vẫn được giám sát bởi tri thức có trước và bởi các khuôn mẫu phân phối ẩn chứa trong dữ liệu đầu vào $\{x^{(k)}\}$. Theo nghĩa này, học không giám sát không hẳn là "học mà không có giám sát".

Hiểu "giám sát" theo nghĩa rộng này, ta thấy xuất hiện thêm những mô thức khác cũng là các dạng của học máy, dù chúng không thuộc gọn vào các nhóm có giám sát hay không giám sát truyền thống. Một ví dụ là **học tăng cường** (*reinforcement learning*). Nó mô hình hóa cách một hệ thống đưa ra một chuỗi các quyết định bằng cách vận hành một tác tử (*agent*) trong một môi trường (*environment*). Tác tử nhận được phản hồi (hay một phần thưởng (*reward*)) từ môi trường khi thực hiện một hành động (*action*). Mục tiêu của học tăng cường là học một chính sách (policy) (*policy*) ra quyết định sao cho tối

[9] Việc xác định hai từ có tương tự nhau hay không có thể khó và mơ hồ. Chúng tôi để vấn đề này lại cho Chương 3.

đa hóa phần thưởng tích lũy qua các bước mà tác tử thực hiện. Thường thì phần thưởng thực sự chỉ có được khi tác tử đạt tới một trạng thái kết thúc (*terminal state*), chẳng hạn thắng hoặc thua một ván cờ. Do đó, học tăng cường rất phù hợp để mô tả các bài toán có đặc trưng là **phần thưởng trễ** (*delayed reward*) (hay phần thưởng thưa – *sparse reward*). Điều này phân biệt rõ rệt nó với học có giám sát thông thường, vốn dựa vào các tín hiệu giám sát tức thời, theo từng bước, được mã hóa sẵn từ trước. Đặc điểm phần thưởng trễ này phù hợp với nhiều bài toán NLP. Ví dụ, một hệ thống sinh văn bản đưa ra một chuỗi từ từ trái sang phải, nhưng thường khó xác định một từ đơn lẻ có được chọn đúng hay không cho đến khi toàn bộ văn bản đã được sinh ra và đánh giá.

Chú giải của người dịch: Học tăng cường qua ví dụ chơi cờ

Hãy hình dung bạn học chơi cờ tướng mà không có thầy chỉ từng nước "nước này đúng, nước kia sai". Bạn (tác tử) đi từng nước (hành động) trên bàn cờ (môi trường), và chỉ khi ván cờ kết thúc mới biết thắng (+1) hay thua (-1) — đó là phần thưởng trễ. Sau nhiều ván, bạn dần rút ra một "chiến lược" (chính sách): ở thế cờ nào thì nên đi nước nào để khả năng thắng cao nhất. Khó khăn là phải tự suy ra trong 40 nước đã đi, những nước nào thực sự góp phần vào chiến thắng. Với sinh văn bản cũng vậy: chỉ khi đọc xong cả câu trả lời ta mới đánh giá được nó tốt hay dở.

Một ví dụ khác là **học tự giám sát** (*self-supervised learning*). Nó giải quyết các bài toán học không giám sát theo cách của học có giám sát. Ý tưởng chung là phát biểu bài toán học không giám sát thành một bài toán tiền đề (*pretext task*) có thể được dùng để giải bài toán ban đầu. Trong bài toán tiền đề, nhãn đúng có thể được tạo ra từ chính dữ liệu đầu vào. Vì vậy, việc học nhận được sự giám sát từ các tín hiệu tự tạo thay vì từ các nhãn thủ công.

Học tự giám sát đã rất thành công trong NLP. **Tiền huấn luyện** có lẽ là ứng dụng nổi bật nhất thúc đẩy những tiến bộ gần đây. Trong tiền huấn luyện, một mô hình nền (chẳng hạn một mô hình ngôn ngữ) được huấn luyện bằng học tự giám sát trên các kho ngữ liệu khổng lồ, rồi các tham số đã học của nó được áp dụng cho một hệ thống hạ nguồn (*downstream*) (chẳng hạn một mô hình phân tích cảm xúc). Cách làm này có hai ưu điểm lớn. Thứ nhất, các mô hình tự giám sát có thể được mở rộng tới lượng văn bản gần như không giới hạn vì chúng không cần gán nhãn thủ công. Thứ hai, các biểu diễn học được trong quá trình tiền huấn luyện có khả năng tổng quát hóa cao và có thể được áp dụng cho nhiều loại bài toán hạ nguồn. Trong Chương 7, chúng ta sẽ tìm hiểu một số ví dụ về việc áp dụng học tự giám sát cho các mô hình NLP hiện đại.

Chú giải của người dịch: Nhãn "tự tạo" trong học tự giám sát

Lấy câu "Hôm nay trời rất đẹp". Ta che đi một từ: "Hôm nay trời rất ___" và yêu cầu mô hình đoán từ bị che. Đáp án đúng ("đẹp") có sẵn ngay trong câu gốc, không cần ai gán nhãn. Với hàng tỉ câu trên Internet, ta tạo được hàng tỉ "bài tập" như vậy miễn phí — đó là bài toán tiền đề. Để làm tốt bài tập này, mô hình buộc phải nắm được ngữ pháp và ý nghĩa của ngôn ngữ, và chính hiểu biết đó sau này được "chuyển giao" cho các bài toán hạ nguồn như phân loại cảm xúc bình luận.

1.3.2 Thiên kiến quy nạp

Một cách không hình thức, ta mô tả học máy (có giám sát) là một quá trình **suy luận quy nạp** (*inductive reasoning*) (hay **suy diễn quy nạp** (*inductive inference*)): ta dùng các quan sát cụ thể (chẳng hạn các tài liệu có nhãn) để xây dựng một mô hình tổng quát (chẳng hạn một bộ phân loại). Ví dụ, nếu ta quan sát thấy từ *cooking* (nấu ăn) thường xuyên xuất hiện trong các tài liệu về đồ ăn, suy luận quy nạp dẫn ta tới kết luận rằng *cooking* là một dấu hiệu mạnh cho mọi tài liệu liên quan đến đồ ăn. Phương pháp "từ

cụ thể đến tổng quát" này được gọi đơn giản là **quy nạp** (*induction*). Khi đã có một mô hình được quy nạp, việc áp dụng nó để dự đoán các quan sát mới, chưa gặp là một quá trình **diễn dịch** (*deduction*).

Quy nạp là nguyên lý được dùng rộng rãi nhất khi thiết kế bộ học của các hệ thống học máy hiện đại[10]. Hãy hình dung có một không gian giả thuyết (*hypothesis space*) (hay không gian mô hình, hay không gian các hàm có thể học) gồm mọi mô hình có thể có mà ta có thể tạo ra. Khi đó, học một mô hình chính là chọn một mô hình trong không gian giả thuyết bằng cách quy nạp từ các mẫu huấn luyện đã cho. Tuy nhiên, ta không thể đơn giản giả định có một mô hình "tiên tri" hoạt động tốt trên mọi mẫu chưa gặp. Một lý do là việc tìm mô hình "tốt nhất" trong một không gian giả thuyết khổng lồ là bất khả thi về mặt tính toán. Nếu không gian giả thuyết không bị ràng buộc, sẽ có vô số chiều mà theo đó ta có thể thiết kế mô hình. Điều này về bản chất liên quan đến **lời nguyền số chiều** (*the curse of dimensionality*)[11]. Một lý do khác là nhiều mô hình trong không gian giả thuyết có thể khớp tốt với các mẫu huấn luyện, nhưng chỉ một số trong đó khớp được với các mẫu chưa gặp. Có nguy cơ ta chọn phải một mô hình ghi nhớ hoàn hảo dữ liệu huấn luyện nhưng không tổng quát hóa (*generalize*) được cho dữ liệu chưa gặp. Điều này liên quan đến **quá khớp** (*overfitting*), một khái niệm mà chúng ta sẽ bàn ở phần sau.

Một giải pháp tự nhiên là định nghĩa các tri thức tiên nghiệm (*priors*) trên không gian giả thuyết sao cho một số mô hình được ưu tiên hơn các mô hình khác. Một ví dụ đơn giản là ta giới hạn các bộ phân loại ở các mô hình tuyến tính (xem Mục 1.2.3). Điều này tương đương với việc áp đặt một tiên nghiệm loại bỏ mọi mô hình phi tuyến khỏi không gian giả thuyết.

Một tiên nghiệm như vậy được gọi là **thiên kiến quy nạp** (*inductive bias*). Tóm lại, thiên kiến quy nạp là một tập các giả định về bài toán học[12]. Chẳng hạn, ta có thể giả định một dạng toán học cụ thể cho mô hình (thiên kiến mô hình (*model bias*)), chọn một thuật toán tối ưu cụ thể (thiên kiến thuật toán (*algorithmic bias*)), hoặc đưa ra giả định về cách dữ liệu được sinh ra (thiên kiến mẫu (*sample bias*))[13].

Các thiên kiến quy nạp cố gắng chỉ ra ta nên mô tả một bài toán theo cách nào. Nói chung, kết quả sẽ tốt hơn khi thiên kiến quy nạp phù hợp với phân phối thực sự ẩn sau dữ liệu. Điều này giải thích vì sao lời giải cho một số bài toán ưa chuộng những kiến trúc mô hình (hay thiên kiến mô hình) nhất định. Dĩ nhiên, càng nhiều và càng mạnh các thiên kiến quy nạp thì càng có thể giúp giải bài toán dễ dàng hơn. Tuy nhiên, thiên kiến quy nạp không phải lúc nào cũng có ích, đặc biệt khi chúng không gần với thực tế.

Hãy xét một trò chơi gieo xúc xắc để minh họa điều này. Giả sử bạn có một con xúc xắc 6 mặt. Trước mỗi lần gieo, bạn đoán một số từ 1 đến 6, với mục tiêu thắng càng nhiều lần càng tốt. Theo lẽ thường, đoán ngẫu nhiên là lựa chọn hợp lý duy nhất của bạn vì mọi mặt đều có cơ hội xuất hiện như nhau $(1/6)$. Điều này đúng với một con xúc xắc cân đối. Tuy nhiên, hãy tưởng tượng một ngày nọ bạn chơi với một con xúc xắc bị đổ chì, và việc thắng trở nên khó khăn. Bạn nhận thấy kết quả không còn tuân theo phân phối đều nữa. Khi đó, bạn giả định kết quả tuân theo một phân phối đa thức (vì bạn đã có kinh nghiệm xây dựng các bộ phân loại Bayes ngây thơ). Trước khi đặt cược, bạn quan sát 100 lần gieo thử, chọn mặt xuất hiện nhiều nhất cho các ván sau, và lại bắt đầu thắng.

Trong ví dụ này, giả định ban đầu của bạn (tức là cả sáu mặt có khả năng như nhau) là một thiên

[10] Không phải phương pháp học máy nào cũng tuân theo quy nạp một cách chặt chẽ. Các mô thức khác bao gồm suy luận diễn dịch (*deductive reasoning*), suy luận giả định (*abductive reasoning*) và suy luận loại suy (*analogical reasoning*, hay *transduction*) [Hurley, 2011].

[11] Lời nguyền số chiều chỉ những vấn đề thường xuất hiện khi số chiều của không gian giả thuyết tăng lên. Ví dụ, tính thưa của dữ liệu là một vấn đề phổ biến nảy sinh khi xử lý dữ liệu nhiều chiều, và do đó là một dạng của lời nguyền số chiều.

[12] Có thể tìm định nghĩa hình thức hơn trong các giáo trình học máy chuẩn [Mitchell, 1997].

[13] Cần lưu ý rằng thuật ngữ *bias* có nghĩa khác trong các ngữ cảnh khác (ví dụ, độ lệch (*bias*) trong mạng nơ-ron hay thiên kiến nhận thức (*cognitive bias*)). Chúng tôi sẽ nói rõ khi dùng với một nghĩa khác.

kiến quy nạp rất mạnh vì mô hình trong đầu bạn có *không* bậc tự do nào. Nó có vẻ hiển nhiên nhưng lại thất bại với con xúc xắc đổ chì. Thiên kiến quy nạp thứ hai của bạn (phân phối đa thức), dù phức tạp hơn về mặt toán học, thực ra lại là một giả định *yếu hơn*. Nó định nghĩa một họ mô hình rộng hơn, cho bạn sự linh hoạt để tìm ra các xác suất cụ thể khớp với con xúc xắc đổ chì.

Chú giải của người dịch: Thiên kiến mạnh và thiên kiến yếu

Với giả định "xúc xắc cân đối", mô hình đã cố định hoàn toàn: $p_1 = \cdots = p_6 = 1/6$, không có gì để học (0 bậc tự do). Với giả định "phân phối đa thức", mô hình có 6 xác suất $p_1, \ldots, p_6$ chỉ cần thỏa $p_i \geq 0$ và tổng bằng 1, tức có 5 bậc tự do (biết 5 số thì suy ra số thứ 6). Ví dụ, nếu trong 100 lần gieo thử mặt 6 xuất hiện 40 lần, còn mỗi mặt khác khoảng 12 lần, ta ước lượng $p_6 \approx 0.4$ và luôn đoán 6: xác suất thắng tăng từ $1/6 \approx 0.17$ lên khoảng 0.4. Giả định "cân đối" là trường hợp riêng của giả định đa thức, nên giả định đa thức yếu hơn (ít ràng buộc hơn) nhưng bao quát hơn.

Suy cho cùng, mọi mô hình học máy đều dựa vào các thiên kiến quy nạp. Nhiều giả định trong số đó là ngầm định; đôi khi ta thậm chí không nhận ra mình đang đưa ra chúng vì chúng có vẻ hoàn toàn "hợp lý". Thế nhưng, nếu một giả định sai từ gốc, nó sẽ gây hại cho toàn bộ quá trình học. Vì vậy, kinh nghiệm thực tế vẫn là yếu tố thiết yếu để nhận ra và tránh những cạm bẫy dễ bị bỏ qua này.

1.3.3 Tính phi tuyến

Hầu hết các bài toán thực tế về bản chất là phi tuyến, khiến chúng khó giải bằng các mô hình tuyến tính đơn giản. Hình 1.3 đưa ra các ví dụ với mức độ khó phân loại khác nhau. Trong Hình 1.3 (a), hai lớp có thể được tách hoàn toàn bằng một đường thẳng (một siêu phẳng). Trong trường hợp này, bài toán là **tách được tuyến tính** (*linearly separable*) vì biên quyết định có thể được biểu diễn bằng một hàm tuyến tính. Ngược lại, trong Hình 1.3 (b), ta không thể vẽ một siêu phẳng duy nhất để tách hoàn toàn các lớp. Thay vào đó, ta cần một biên quyết định phi tuyến, chẳng hạn một siêu cầu (*hypersphere*) (một đường tròn trong không gian 2 chiều). Một trường hợp còn khó hơn được minh họa trong Hình 1.3 (c), trong đó các lớp đan xen vào nhau và biên quyết định rất phức tạp.

Mặc dù việc mô hình hóa các hệ phi tuyến làm tăng độ phức tạp tính toán, đã có một số phương pháp phổ biến để đưa tính phi tuyến vào các hệ thống học máy.

- **Ánh xạ đặc trưng** (*feature mapping*) và **phương pháp hạt nhân** (*kernel methods*). Nhắc lại rằng một bộ phân loại tuyến tính có thể được phát biểu thành hàm $f(\mathbf{w} \cdot \mathbf{x})$, trong đó $\mathbf{w}$ là vectơ trọng số, $\mathbf{x}$ là vectơ đặc trưng, và $f(\cdot)$ là hàm kích hoạt (*activation function*) trả về một lớp (chẳng hạn c_a) nếu đối số của nó lớn hơn 0 và lớp còn lại (chẳng hạn c_b) trong trường hợp ngược lại. Ý tưởng của ánh xạ đặc trưng là ánh xạ vectơ đặc trưng $\mathbf{x}$ vào một không gian có số chiều cao hơn sao cho bài toán trở nên tách được tuyến tính trong không gian mới. Ví dụ, cho $\phi(\cdot) : \mathbb{R}^2 \to \mathbb{R}^4$ là một hàm ánh xạ và $\mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix}$ là một vectơ 2 chiều. Ta giả sử

$$\phi(\begin{bmatrix} x_1 & x_2 \end{bmatrix}) = \begin{bmatrix} x_1^2 + x_2^2 & x_1 & x_2 & 1 \end{bmatrix}. \tag{1.65}$$

Bằng cách chọn $\mathbf{w} = \begin{bmatrix} 1 & -8 & -8 & 28 \end{bmatrix}$, ta được một bộ phân loại mới:

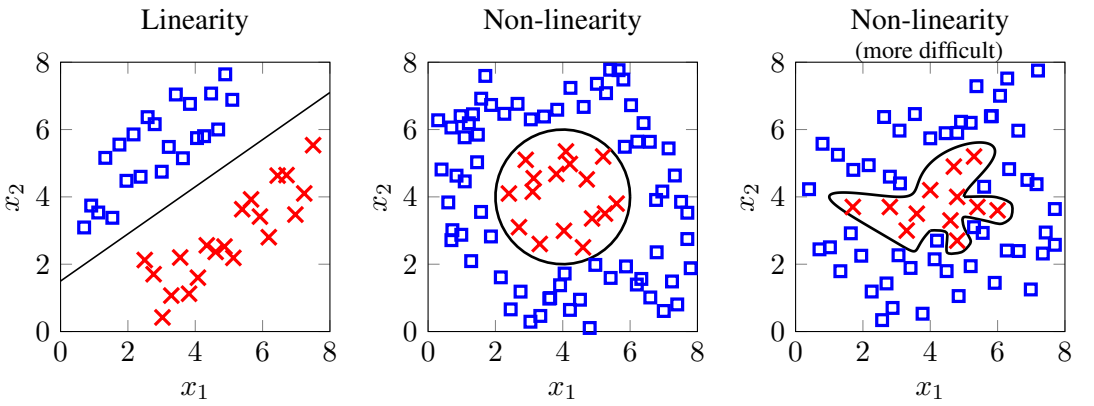
$$\begin{aligned} f(\mathbf{w} \cdot \phi(\mathbf{x})) &= f\big(\begin{bmatrix} 1 & -8 & -8 & 28 \end{bmatrix} \cdot \begin{bmatrix} x_1^2 + x_2^2 & x_1 & x_2 & 1 \end{bmatrix} \big) \\ &= f\big((x_1 - 4)^2 + (x_2 - 4)^2 - 4\big). \end{aligned} \tag{1.66}$$

Hàm này xác định một biên quyết định (tức là siêu cầu $(x_1 - 4)^2 + (x_2 - 4)^2 = 4$) phân loại hoàn hảo các mẫu trong Hình 1.3 (b). Nói cách khác, ta dùng một mô hình tuyến tính trên không

gian đặc trưng đã ánh xạ để tạo ra một mô hình phi tuyến. Tuy nhiên, việc tính hàm ánh xạ có thể kém hiệu quả. Vấn đề này thường được giải quyết bằng các phương pháp hạt nhân. Trong các phương pháp hạt nhân, việc tính tích vô hướng của các vectơ trong không gian mới được thực hiện hiệu quả bằng cách dùng một hàm hạt nhân (*kernel function*) trong không gian cũ. Phương pháp này được gọi là **thủ thuật hạt nhân** (*the kernel trick*). Nó đã được áp dụng thành công trong phân loại và các mô hình học máy khác, chẳng hạn **máy vectơ hỗ trợ** (*support vector machines*) [Cortes và Vapnik, 1995].

Chú giải của người dịch: Kiểm tra Phương trình (1.66) bằng tay

Tích vô hướng của hai vectơ 4 phần tử là tổng các tích từng cặp phần tử: $1 \cdot (x_1^2 + x_2^2) + (-8) \cdot x_1 + (-8) \cdot x_2 + 28 \cdot 1 = x_1^2 - 8x_1 + x_2^2 - 8x_2 + 28$. Vì $(x_1 - 4)^2 = x_1^2 - 8x_1 + 16$ và $(x_2 - 4)^2 = x_2^2 - 8x_2 + 16$, ta có biểu thức trên bằng $(x_1 - 4)^2 + (x_2 - 4)^2 - 4$. Đây chính là phương trình đường tròn tâm $(4; 4)$, bán kính 2 quen thuộc ở lớp 10, chuyển vế: điểm nằm *ngoài* đường tròn cho giá trị dương (một lớp), điểm nằm *trong* cho giá trị âm (lớp kia). Ví dụ điểm $(4; 4)$ cho $-4 < 0$, điểm $(0; 0)$ cho $16 + 16 - 4 = 28 > 0$. Điều kỳ diệu là: đối với các "đặc trưng mới" $(x_1^2 + x_2^2,\ x_1,\ x_2,\ 1)$, biểu thức này *tuyến tính*, dù đối với (x_1, x_2) ban đầu nó là một đường cong.



Hình 1.3: Minh họa tính tuyến tính và phi tuyến trong phân loại nhị phân. Bài toán thứ nhất (trái) là tách được tuyến tính vì tồn tại (ít nhất) một siêu phẳng tách hoàn toàn các điểm dữ liệu thuộc hai lớp. Tính tách được tuyến tính không còn đúng ở bài toán thứ hai (giữa). Thay vào đó, ta cần một biên quyết định có dạng giống đường tròn. Biên quyết định sẽ phức tạp hơn nữa nếu có những vùng mà hai lớp điểm dữ liệu trộn lẫn vào nhau và ít nhiều không phân biệt được (phải). (Tiêu đề các đồ thị: *Linearity* = tuyến tính, *Non-linearity* = phi tuyến, *more difficult* = khó hơn.)

- **Hàm kích hoạt phi tuyến** (*non-linear activation functions*). Một cách tiếp cận chuẩn khác để đưa vào tính phi tuyến là thông qua các hàm kích hoạt. Một mô thức phổ biến là áp dụng một hàm kích hoạt phi tuyến lên đầu ra của một phép biến đổi tuyến tính. Chẳng hạn, hàm $f(\cdot)$ dùng trong ví dụ trước đóng vai trò một hàm kích hoạt phi tuyến đơn giản. Tùy theo dạng đầu ra mong muốn, có thể chọn nhiều hàm kích hoạt khác nhau. Trong các mô hình phức tạp, nhiều hàm kích hoạt phi tuyến được lồng vào các bước tính toán trung gian để tăng mạnh năng lực biểu diễn (*expressive power*) của mô hình. Ví dụ, một **mạng nơ-ron sâu** (*deep neural network*) về bản chất là một chồng các mô hình con (thường được gọi là các tầng (*layers*)), trong đó mỗi tầng áp dụng một phép biến đổi tuyến tính rồi đến một hàm kích hoạt phi tuyến.

- **Phương pháp phi tham số** (*non-parametric methods*). Thuật ngữ *phi tham số* bắt nguồn từ thống

kê. Trong thống kê phi tham số, các suy luận thống kê được thực hiện mà không đưa ra bất kỳ giả định nào về phân phối ẩn sau dữ liệu. Trong học máy, các phương pháp phi tham số theo cùng ý tưởng đó. Chúng không giả định bất kỳ hàm ánh xạ nào từ đầu vào sang đầu ra như Phương trình (1.66). Điều này phân biệt chúng với các **phương pháp có tham số** (*parametric methods*), vốn học tường minh một dạng toán học của các biến (hay tham số) để mô tả bài toán. Một ví dụ về phương pháp phi tham số là k **láng giềng gần nhất** (k-*nearest neighbors*), phương pháp phân loại một mẫu mới dựa trên biểu quyết đa số của k mẫu huấn luyện gần nó nhất. Lưu ý rằng phi tham số không có nghĩa là không có tham số. Đúng hơn, nó có nghĩa là số lượng tham số không được cố định trước. Độ phức tạp của mô hình tăng lên một cách linh động khi có thêm dữ liệu huấn luyện. Bù lại, các phương pháp này có thể xử lý những bài toán phi tuyến cao khi có đủ mẫu huấn luyện.

Chú giải của người dịch: Phương pháp k láng giềng gần nhất

Muốn biết một học sinh mới chuyển đến thích đá bóng hay cầu lông, ta xem $k = 5$ bạn ở gần nhà em ấy nhất: nếu 3 bạn thích đá bóng, 2 bạn thích cầu lông, ta đoán em ấy thích đá bóng. Trong Hình 1.3, với một điểm mới, ta tìm k điểm huấn luyện gần nhất (theo khoảng cách trên mặt phẳng) và lấy nhãn chiếm đa số. Phương pháp này không cần công thức biên quyết định nào cả, nên vẫn xử lý được cả hình (c) phức tạp. "Tham số" của nó thực chất là toàn bộ dữ liệu huấn luyện được lưu lại, nên càng có nhiều dữ liệu, "mô hình" càng lớn.

Dù vậy, các phương pháp phi tuyến không hoạt động một mình. Tính tuyến tính là một thành phần quan trọng của hầu hết các hệ thống học máy thực tế. Điều này thể hiện ở hai khía cạnh. Thứ nhất, nhiều tính phi tuyến hơn không phải lúc nào cũng tốt hơn. Nếu một mô hình tuyến tính đã giải quyết bài toán đủ tốt thì việc thêm độ phức tạp phi tuyến là không cần thiết và có nguy cơ gây quá khớp. Phù hợp với nguyên lý **dao cạo Occam** (*Occam's Razor*) — *lời giải đơn giản nhất gần như luôn là lời giải tốt nhất* — hầu hết các mô hình tiên tiến nhất (*state-of-the-art*) đều duy trì sự cân bằng giữa các thành phần tuyến tính và phi tuyến. Thứ hai, các mô hình tuyến tính là những phép xấp xỉ tuyệt vời cho các hành vi phi tuyến. Nếu tính phi tuyến của một bài toán là nhẹ, việc dùng các hàm tuyến tính để xấp xỉ lời giải thường hiệu quả hơn đáng kể và dễ diễn giải hơn cho các mục đích kỹ thuật thực tế.

1.3.4 Huấn luyện và hàm mất mát

Hầu như mọi thuật toán học máy đều có một bước huấn luyện (*training*). Huấn luyện thường chỉ quá trình ước lượng hàm ánh xạ và các tham số đi kèm của nó từ dữ liệu. Ở đây, chúng ta theo một định nghĩa thông dụng của bài toán huấn luyện: cho trước một mô hình hay hàm ánh xạ, ta tối ưu một mục tiêu cụ thể bằng cách đánh giá mô hình qua một *kinh nghiệm huấn luyện* nào đó [Mitchell, 1997]. Ví dụ, huấn luyện một bộ phân loại văn bản Bayes ngây thơ đòi hỏi cực đại hóa một hàm hợp lý (mục tiêu) trên một tập các tài liệu có nhãn (kinh nghiệm huấn luyện).

Thường thì bài toán huấn luyện có thể được phát biểu thành một bài toán **tối ưu hóa** (*optimization*). Theo đó, ta tối ưu một **hàm mục tiêu** (*objective function*) thông qua một thuật toán huấn luyện. Mặc dù mục tiêu lý tưởng là thước đo hiệu năng thực tế trên các mẫu kiểm tra, ta không thể dùng trực tiếp nó để tối ưu, vì các mẫu kiểm tra và nhãn tương ứng của chúng không thể tiếp cận được trong giai đoạn huấn luyện. Thay vào đó, các hàm mục tiêu thực tế được định nghĩa như những đại lượng thay thế tính được cho thước đo dùng lúc kiểm tra. Các hàm mục tiêu này không nhất thiết là thước đo hiệu năng chính xác, mà là những đại lượng được giả định là có tương quan với hiệu năng trên dữ liệu kiểm tra.

Hãy xét một trường hợp tổng quát. Giả sử $\mathbf{y}_\theta = f_\theta(\mathbf{x})$ là một mô hình nhận vectơ đặc trưng $\mathbf{x}$ làm đầu vào và cho ra một vectơ đầu ra n chiều $\mathbf{y}_\theta$. Ví dụ, trong phân loại văn bản, $\mathbf{x}$ là biểu diễn túi từ của

một tài liệu, còn $\mathbf{y}_\theta$ là một phân phối xác suất trên một tập các lớp. Ở đây, θ biểu diễn các tham số của mô hình, và chỉ số dưới nhấn mạnh rằng hành vi của mô hình được quyết định bởi θ. Ta giả sử thêm $\mathbf{y}_{\text{gold}}$ là vectơ nhãn đúng (hay nhãn chuẩn vàng). Khi đó ta định nghĩa một **hàm mất mát** (*loss function*), ký hiệu là $\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})$, để đo mức độ sai khác hay sai số giữa dự đoán $\mathbf{y}_\theta$ và nhãn đúng $\mathbf{y}_{\text{gold}}$. Nó "phạt" mô hình khi dự đoán không chính xác. Với một kiến trúc mô hình cho trước, bài toán huấn luyện được mô tả là tìm các tham số "tốt nhất" $\hat{\theta}$ sao cho mất mát $\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})$ nhỏ nhất:

$$\hat{\theta} \;=\; \arg\min_{\theta} \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}). \tag{1.67}$$

Công thức này có thể dễ dàng mở rộng cho trường hợp có K mẫu huấn luyện:

$$\hat{\theta} \;=\; \arg\min_{\theta} \frac{1}{K} \sum_{k=1}^{K} \mathcal{L}(\mathbf{y}_\theta^{(k)}, \mathbf{y}_{\text{gold}}^{(k)}). \tag{1.68}$$

Khi đã có $\hat{\theta}$, ta có thể dùng $f_{\hat{\theta}}(\mathbf{x})$ như một mô hình cố định cho các dự đoán sau này.

Chú giải của người dịch: Đọc Phương trình (1.67–1.68)

$\arg\min_\theta g(\theta)$ nghĩa là "giá trị của θ làm cho $g(\theta)$ nhỏ nhất" (khác với $\min_\theta g(\theta)$ là chính giá trị nhỏ nhất đó). Ví dụ, với $g(\theta) = (\theta - 3)^2 + 1$ thì $\min g = 1$ còn $\arg\min g = 3$. Phương trình (1.68) lấy *trung bình* mất mát trên cả K mẫu huấn luyện, rồi tìm bộ tham số làm trung bình đó nhỏ nhất — giống như chọn cách học sao cho tổng số lỗi trên toàn bộ đề luyện tập là ít nhất, chứ không chỉ đúng một bài. Dấu mũ trong $\hat{\theta}$ là ký hiệu thông dụng cho "giá trị ước lượng được".

$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})$ thường được gọi là **hàm mất mát** (tính trên từng mẫu), còn giá trị trung bình của nó trên toàn bộ tập dữ liệu, $\frac{1}{K}\sum_{k=1}^{K} \mathcal{L}(\mathbf{y}_\theta^{(k)}, \mathbf{y}_{\text{gold}}^{(k)})$, thường được gọi là **hàm chi phí** (*cost function*) hay **rủi ro thực nghiệm** (*empirical risk*). Một hàm mất mát có thể được định nghĩa theo nhiều cách, tùy thuộc vào bài toán ta giải và những tiên nghiệm ta muốn áp đặt lên quá trình huấn luyện. Ở đây, trước hết ta xét trường hợp $\mathbf{y}_\theta$ là một phân phối xác suất. Trường hợp này khá phổ biến trong NLP: ví dụ, $\mathbf{y}_\theta$ có thể là một phân phối trên bộ từ vựng, trên một danh sách tài liệu, hay trên một tập nhãn cú pháp. Với loại đầu ra mô hình này, các hàm mất mát được dùng phổ biến nhất là các độ đo độ phân kỳ (*divergence*):

- **Mất mát dựa trên độ phân kỳ** (*divergence-based loss*). Các hàm mất mát dựa trên độ phân kỳ đo sự khác biệt giữa hai phân phối $\mathbf{y}_\theta$ và $\mathbf{y}_{\text{gold}}$. Ví dụ, entropy chéo (*cross-entropy*) (xem Mục 1.1.2) là một trong những hàm mất mát phổ biến nhất trong NLP. Ta cũng có thể chọn các độ đo dựa trên độ phân kỳ khác, chẳng hạn độ phân kỳ KL và độ phân kỳ Jensen–Shannon (JS), có thể tìm thấy trong hầu hết các giáo trình thống kê. Lưu ý rằng mục tiêu của **ước lượng hợp lý cực đại** (*maximum likelihood estimation*) (**MLE**) gắn liền về bản chất với các mục tiêu dựa trên độ phân kỳ. Cụ thể, cực tiểu hóa log-hợp lý âm (mục tiêu MLE chuẩn) tương đương về mặt toán học với cực tiểu hóa mất mát entropy chéo khi $\mathbf{y}_{\text{gold}}$ là một vectơ one-hot (*one-hot vector*) (vectơ trong đó phần tử ứng với nhãn đúng bằng 1 và mọi phần tử khác bằng 0).

Chú giải của người dịch: Vì sao entropy chéo với nhãn one-hot chính là log-hợp lý âm?

Giả sử có 3 lớp (Thể thao, Đồ ăn, Âm nhạc), nhãn đúng là "Đồ ăn" nên $\mathbf{y}_{\text{gold}} = [0 \;\; 1 \;\; 0]$, còn mô hình dự đoán $\mathbf{y}_\theta = [0.2 \;\; 0.7 \;\; 0.1]$. Entropy chéo là $-\sum_i y_{\text{gold}}(i) \log y_\theta(i) = -(0 \cdot \log 0.2 + 1 \cdot \log 0.7 + 0 \cdot \log 0.1) = -\log 0.7 \approx 0.36$ (logarit tự nhiên). Mọi số hạng có hệ số 0 đều biến mất, chỉ còn $-\log$ của xác suất mô hình gán cho nhãn đúng — đó đúng là log-hợp lý âm. Nếu mô hình tự tin hơn (0.9) thì mất mát giảm còn $-\log 0.9 \approx 0.11$; nếu chỉ gán 0.1 thì mất mát tăng tới ≈ 2.30.

Tuy nhiên, các hệ thống học máy không phải lúc nào cũng bị giới hạn ở đầu ra dạng phân phối. Thay vào đó, $\mathbf{y}_\theta$ có thể đơn giản là một vectơ trong $\mathbb{R}^n$. Một ví dụ là các hàm phân biệt dùng trong phân loại (xem Mục 1.2.3). Các hàm này gán một điểm số cho mỗi lớp, thể hiện mức độ mô hình tin rằng một lớp cụ thể là câu trả lời đúng. Một cách định nghĩa hàm mất mát cho các vectơ giá trị thực là biến đổi chúng về dạng giống phân phối[14] rồi dùng các mất mát dựa trên độ phân kỳ. Tuy nhiên, việc chuẩn hóa không phải lúc nào cũng cần thiết, đặc biệt khi ta cần các điểm số nằm ngoài khoảng $[0, 1]$. Phổ biến hơn là tính mất mát trực tiếp trên đầu ra thô của các mô hình này. Dưới đây là một số ví dụ.

- **Mất mát dựa trên khoảng cách** (*distance-based loss*). Một cách tự nhiên là phát biểu mất mát như một khoảng cách hình học. Một ví dụ tổng quát là khoảng cách theo chuẩn p (xem Mục 1.1.1):

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) \;=\; \Big(\sum_{i=1}^{n} |y_\theta(i) - y_{\text{gold}}(i)|^p\Big)^{1/p}. \tag{1.69}$$

  Ví dụ, đặt $p = 2$ cho ta khoảng cách Euclid thông thường. Về bản chất, các mất mát dựa trên khoảng cách mô tả một bài toán **khớp đường cong** (*curve fitting*): ta học một hàm $\mathbf{y}_\theta = f_\theta(\mathbf{x})$ để khớp với một tập điểm mục tiêu $\{(\mathbf{x}^{(k)}, \mathbf{y}^{(k)}_{\text{gold}})\}$. Cách tiếp cận này là điển hình trong các bài toán **hồi quy** (*regression*)[15]. Một ví dụ cụ thể là ước lượng chất lượng (*quality estimation*) của dịch máy[16]. Mô hình học cách dự đoán một điểm chất lượng bản dịch liên tục (y_θ) cho một cặp câu nguồn–đích ($\mathbf{x}$). Dự đoán được coi là chính xác nếu nó khớp sát với đánh giá của con người. Dựa trên Phương trình (1.69), **sai số bình phương trung bình** (*mean squared error*) (**MSE**) là một hàm mất mát hồi quy rất phổ biến. Nó bỏ đi phép khai căn bậc $1/p$ và chỉ đơn giản tính tổng bình phương khoảng cách Euclid:

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) \;=\; \sum_{i=1}^{n} |y_\theta(i) - y_{\text{gold}}(i)|^2. \tag{1.70}$$

  Một ví dụ khác là **sai số tuyệt đối trung bình** (*mean absolute error*) (**MAE**). Nó chính xác là dạng của Phương trình (1.69) khi $p = 1$.

  Chú giải của người dịch: Ví dụ số cho MSE và MAE

  Giả sử mô hình dự đoán điểm chất lượng của 3 bản dịch là $\mathbf{y}_\theta = [7 \;\; 5 \;\; 9]$, còn người chấm cho $\mathbf{y}_{\text{gold}} = [8 \;\; 5 \;\; 6]$. Hiệu từng phần tử: $-1,\ 0,\ 3$. Theo Phương trình (1.70): $1 + 0 + 9 = 10$. Theo dạng $p = 1$: $1 + 0 + 3 = 4$. Theo $p = 2$ (khoảng cách Euclid): $\sqrt{10} \approx 3.16$. Để ý rằng bình phương "phạt" sai số lớn (3) nặng hơn nhiều so với sai số nhỏ (1). (Trong thực hành, MSE và MAE thường còn được chia cho số phần tử n để lấy trung bình, như tên gọi "trung bình"; điều này không làm thay đổi tham số tối ưu.)

- **Mất mát 0-1** (*0-1 loss*). Mất mát 0-1 được dùng rộng rãi trong các bài toán phân loại. Nó gán mức phạt 1 cho một dự đoán sai và 0 cho một dự đoán đúng. Gọi $c_\theta = \arg\max_c y_\theta(c)$ là nhãn dự đoán, được chọn bằng cách tìm giá trị lớn nhất trong $\mathbf{y}_\theta$. Tương tự, gọi $c_{\text{gold}} = \arg\max_c y_{\text{gold}}(c)$ là nhãn đúng. Mất mát 0-1 được định nghĩa hình thức như sau:

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) = \mathcal{L}_{\text{0-1}}(c_\theta, c_{\text{gold}})$$

[14] Ví dụ, bằng cách chia các phần tử của vectơ cho tổng của tất cả các phần tử.

[15] Khi đầu ra của mô hình là một vectơ có từ hai chiều trở lên, bài toán được gọi là **hồi quy đa biến** (*multivariate regression*).

[16] Trong dịch máy, ước lượng chất lượng gồm nhiều bài toán con. Ở đây, chúng tôi dùng thuật ngữ này để chỉ việc dự đoán trực tiếp một điểm đánh giá.

$$= \begin{cases} 1 & c_\theta \neq c_{\text{gold}} \\ 0 & c_\theta = c_{\text{gold}} \end{cases}. \tag{1.71}$$

- **Mất mát dựa trên lề** (*margin-based loss*). Một **lề** (*margin*) biểu diễn hiệu số giữa điểm số dự đoán của nhãn đúng c_{gold} và của một nhãn sai c:

$$\text{margin}(c, c_{\text{gold}}) \;=\; y_\theta(c_{\text{gold}}) - y_\theta(c). \tag{1.72}$$

Nó thể hiện mức độ phân biệt giữa c_{gold} và c. Vì vậy, một ý tưởng tự nhiên là đảm bảo lề đủ lớn, hoặc ít nhất vượt quá một mức tối thiểu. Mô thức này được gọi là **huấn luyện lề lớn** (*large-margin training*). Gọi $\triangle(c, c_{\text{gold}})$ là chi phí phạt được định nghĩa trước khi phân loại nhầm c_{gold} thành c, thỏa mãn $\triangle(c, c_{\text{gold}}) \geq 0$, và $\triangle(c, c_{\text{gold}}) = 0$ khi và chỉ khi $c = c_{\text{gold}}$. Mục tiêu của ta là cực đại hóa biểu thức $\text{margin}(c, c_{\text{gold}}) - \triangle(c, c_{\text{gold}})$; tương đương, hàm mất mát cần phạt những trường hợp giá trị này nhỏ hoặc âm. Mất mát dựa trên lề được cho bởi:

$$\begin{aligned} \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) &= \max\Big(0, \max_c -\big(\text{margin}(c, c_{\text{gold}}) - \triangle(c, c_{\text{gold}})\big)\Big) \\ &= \max\Big(0, \max_c \big(y_\theta(c) - y_\theta(c_{\text{gold}}) + \triangle(c, c_{\text{gold}})\big)\Big) \\ &= \max_c \Big(0, y_\theta(c) - y_\theta(c_{\text{gold}}) + \triangle(c, c_{\text{gold}})\Big). \end{aligned} \tag{1.73}$$

Việc thiết kế $\triangle(c, c_{\text{gold}})$ phụ thuộc vào bài toán đích. Một lựa chọn đơn giản là $\triangle(c, c_{\text{gold}}) = 1$ với $c \neq c_{\text{gold}}$. Cách phát biểu này khiến Phương trình (1.73) trở thành một biến thể của **mất mát bản lề** (*hinge loss*). Một phương án khác là dùng tổng thay cho giá trị lớn nhất trên mọi lớp c:

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) \;=\; \sum_c \max\Big(0, y_\theta(c) - y_\theta(c_{\text{gold}}) + \triangle(c, c_{\text{gold}})\Big). \tag{1.74}$$

Chú giải của người dịch: Ví dụ tính mất mát bản lề

Ba lớp với điểm số mô hình $y_\theta = (\text{Thể thao} : 2,\ \text{Đồ ăn} : 3,\ \text{Âm nhạc} : 2.5)$, nhãn đúng là "Đồ ăn", và $\triangle = 1$ cho mọi lớp sai. Với "Thể thao": $2 - 3 + 1 = 0$; với "Âm nhạc": $2.5 - 3 + 1 = 0.5$; với chính "Đồ ăn": $3 - 3 + 0 = 0$. Theo Phương trình (1.73), mất mát $= \max(0;\ 0;\ 0.5) = 0.5$; theo Phương trình (1.74), mất mát $= 0 + 0.5 + 0 = 0.5$ (ở đây tình cờ bằng nhau). Dù mô hình đã đoán đúng ("Đồ ăn" có điểm cao nhất), mất mát vẫn dương vì "Âm nhạc" chỉ kém 0.5 điểm, chưa đạt lề yêu cầu là 1. Nếu điểm "Đồ ăn" tăng lên 3.5 thì mất mát về 0: mô hình không chỉ đúng mà còn đúng một cách "chắc chắn".

- **Mất mát dựa trên xếp hạng** (*ranking-based loss*). Mất mát dựa trên xếp hạng (hay mất mát xếp hạng (*ranking loss*)) được dùng trong nhiều lĩnh vực khác nhau, chẳng hạn truy xuất thông tin (*information retrieval*), phân loại và học độ đo (*metric learning*). Nó giải quyết bài toán trong đó ta muốn sắp thứ tự một tập các mục đã được chấm điểm. Giả sử đầu ra $\mathbf{y}_\theta$ của mô hình đánh giá n mục $\{c_i\}$. Ta định nghĩa $\{\psi_\theta(c_i)\}$ là thứ hạng dự đoán của $\{c_i\}$ khi sắp xếp theo điểm số $\{y_\theta(i)\}$ của chúng. Ví dụ, với điểm số $\{y_\theta(i)\} = \{0.3, -2, 1\}$, các thứ hạng sẽ là $\psi_\theta(c_1) = 2$, $\psi_\theta(c_2) = 3$ và $\psi_\theta(c_3) = 1$. Tương tự, ta định nghĩa $\{\psi_{\text{gold}}(c_i)\}$ là các thứ hạng chuẩn vàng. Lưu ý rằng nếu một bài toán chỉ đòi hỏi thứ tự tương đối thay vì điểm số tuyệt đối, $\{\psi_{\text{gold}}(c_i)\}$ có thể được định nghĩa mà không cần các giá trị đích $\mathbf{y}_{\text{gold}}$. Ý tưởng là phạt những chỗ đảo thứ hạng trong $\{\psi_\theta(c_i)\}$ so với $\{\psi_{\text{gold}}(c_i)\}$. Một phương pháp đơn giản là phạt các cặp mục bị sắp sai thứ

tự, cùng chung ý tưởng với phân loại nhị phân. Gọi Ω là tập các cặp được sắp đúng thứ tự theo nhãn đúng:

$$\Omega \;=\; \{(i,j) \mid \psi_{\text{gold}}(i) < \psi_{\text{gold}}(j)\}. \tag{1.75}$$

Hàm mất mát cộng gộp các mức phạt trên những cặp này:

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) \;=\; \sum_{(i,j)\in\Omega} \mathcal{L}_{\text{pair}}(y_\theta(i), y_\theta(j)), \tag{1.76}$$

trong đó $\mathcal{L}_{\text{pair}}$ là một hàm mất mát phân loại theo cặp, chẳng hạn mất mát bản lề được dùng trong [Collobert và Weston, 2008]:

$$\mathcal{L}_{\text{pair}}(y_\theta(i), y_\theta(j)) \;=\; \max\big(0, y_\theta(i) - y_\theta(j) + 1\big). \tag{1.77}$$

Cách tiếp cận này được gọi là **phương pháp theo cặp** (*pairwise method*). Mất mát xếp hạng cũng có thể được định nghĩa theo kiểu theo từng điểm (*pointwise*) hoặc theo danh sách (*listwise*), vốn được dùng rộng rãi trong các hệ thống học xếp hạng (*learning-to-rank*) hiện đại.

Chú giải của người dịch: Thứ hạng và cặp sắp đúng thứ tự

Thứ hạng 1 là mục có điểm cao nhất: với điểm $\{0.3, -2, 1\}$, mục c_3 (điểm 1) đứng đầu, c_1 (0.3) thứ hai, c_2 (-2) cuối. Giả sử thứ hạng đúng là c_1 nhất, c_3 nhì, c_2 ba. Khi đó Ω gồm 3 cặp: $(1,3)$, $(1,2)$, $(3,2)$ — cặp (i,j) nghĩa là "i phải xếp trên j". Mô hình đã xếp đúng cặp $(1,2)$ và $(3,2)$ nhưng đảo cặp $(1,3)$ (vì c_3 được xếp trên c_1). Hàm mất mát theo cặp chỉ cần nhìn từng cặp như một bài toán "ai đứng trên?" nhỏ, giống phân loại nhị phân, rồi cộng các mức phạt lại.

- **Mất mát tương phản** (*contrastive loss*). Mất mát tương phản thường được dùng trong **học tương phản** (*contrastive learning*). Nó giả định rằng, với một mẫu cho trước, có một mẫu tương tự được gán nhãn "dương" (*positive*), và có một số mẫu không tương tự được gán nhãn "âm" (*negative*). Một ý tưởng tự nhiên là cực tiểu hóa khoảng cách giữa các mẫu tương tự, đồng thời cực đại hóa khoảng cách giữa các mẫu không tương tự. Quay lại cách phát biểu của chúng ta, gọi $\mathbf{y}^+$ là đầu ra dương và $\mathbf{Y}^- = \{\mathbf{y}^-\}$ là tập các đầu ra âm. Đồng thời, ta dùng $\mathbf{y}_{\text{gold}}$ để ký hiệu bộ gồm $\mathbf{y}^+$ và $\mathbf{Y}^-$ thay vì một vectơ chuẩn vàng duy nhất. Một dạng của hàm mất mát tương phản được cho bởi phương trình:

$$\begin{aligned}
\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) &= \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}^+, \mathbf{Y}^-) \\
&= \log D(\mathbf{y}_\theta, \mathbf{y}^+) - \log \sum_{\mathbf{y}^-\in\mathbf{Y}^-} D(\mathbf{y}_\theta, \mathbf{y}^-) \\
&= \log \frac{D(\mathbf{y}_\theta, \mathbf{y}^+)}{\sum_{\mathbf{y}^-\in\mathbf{Y}^-} D(\mathbf{y}_\theta, \mathbf{y}^-)},
\end{aligned} \tag{1.78}$$

trong đó $D(\alpha, \beta)$ biểu diễn một độ đo khoảng cách (ví dụ khoảng cách Euclid) giữa α và β. Một thách thức trong thiết kế là tạo ra các đầu ra dương và âm này. Trong thiết lập có giám sát, vectơ nhãn đúng đương nhiên đóng vai trò đích dương, còn các mẫu âm có thể được lấy từ các lớp khác. Trong thiết lập không giám sát, $\mathbf{y}^+$ và $\mathbf{Y}^-$ thường được định nghĩa theo kinh nghiệm. Ví dụ, trong một khuôn khổ **tự mã hóa** (*auto-encoding*), $\mathbf{y}^+$ có thể đơn giản là chính đầu vào $\mathbf{x}$ (hoặc một phiên bản tăng cường (*augmented*) của nó), còn $\mathbf{Y}^-$ là một tập các mẫu được sinh ngẫu nhiên.

Chú giải của người dịch: Trực giác của mất mát tương phản

Hãy nghĩ tới việc học nhận mặt: ảnh chân dung của bạn Lan (đầu ra mô hình) cần ở "gần" một ảnh khác của chính Lan (mẫu dương) và ở "xa" ảnh của các bạn khác (mẫu âm). Trong Phương trình (1.78), tử số là khoảng cách tới mẫu dương (muốn nhỏ), mẫu số là tổng khoảng cách tới các mẫu âm (muốn lớn). Tỉ số càng nhỏ thì log của nó càng nhỏ (càng âm), nên cực tiểu hóa mất mát đồng thời kéo mẫu dương lại gần và đẩy các mẫu âm ra xa.

- **Mất mát dựa trên lỗi** (*error-based loss*). Các thước đo đánh giá, vốn thường dùng để định lượng lỗi của hệ thống, cũng có thể được dùng làm hàm mất mát. Ví dụ, trong dịch máy, BLEU là một thước đo thường dùng (càng cao càng tốt, nằm trong khoảng từ 0 đến 1). Ta có thể định nghĩa mục tiêu là cực tiểu hóa $1 - \text{BLEU}$. Gọi $\text{Error}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})$ là lỗi tính theo thước đo. Mất mát dựa trên lỗi đơn giản là giá trị này:

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) \;=\; \text{Error}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}). \tag{1.79}$$

Cho đến đây, chúng ta đã trình bày một số hàm mất mát cho nhiều loại bài toán khác nhau, như phân loại, hồi quy và xếp hạng. Như sẽ thấy trong cuốn sách này, các hàm mất mát khác nhau có ảnh hưởng khác nhau lên hành vi của mô hình. Tuy nhiên, thử nghiệm mọi hàm mất mát có thể có là không khả thi về mặt tính toán. Thay vào đó, người làm thực tế phải cẩn thận lựa chọn hoặc thiết kế hàm mục tiêu phù hợp nhất với bài toán cụ thể của mình. Quá trình này đòi hỏi thời gian và chuyên môn về lĩnh vực nhưng là thiết yếu.

Mặt khác, có những phương pháp chung để cải thiện việc thiết kế hàm mất mát. Ví dụ, thay vì coi đầu ra $\mathbf{y}_\theta$ của mô hình là một vectơ tất định, ta có thể mô hình hóa nó như một **biến ngẫu nhiên** (*random variable*) được rút ra từ một phân phối xác suất. Kết quả là, mất mát riêng lẻ $L(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})$ cũng trở thành một biến ngẫu nhiên. Trong khuôn khổ này, ta định nghĩa lại mục tiêu là **mất mát kỳ vọng** (*expected loss*) (hay rủi ro kỳ vọng) trên phân phối của $\mathbf{y}_\theta$:

$$\begin{aligned}\mathcal{L}(\mathbf{x}, \mathbf{y}_{\text{gold}}) &= \mathbb{E}_{\mathbf{y}_\theta \sim \Pr(\cdot|\mathbf{x})}\left[\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})\right] \\ &= \sum_{\mathbf{y}_\theta} \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) \cdot \Pr(\mathbf{y}_\theta|\mathbf{x}).\end{aligned} \tag{1.80}$$

Bằng cách lấy tổng (tích phân) trên toàn bộ không gian các dự đoán có thể có $\mathbf{y}_\theta$, phương pháp này cho một ước lượng mất mát vững hơn. Mô thức này bắt nguồn từ **cách tiếp cận Bayes** (*the Bayesian approach*), và $\mathcal{L}(\cdot)$ (cũng được ký hiệu là $\mathbb{L}(\cdot)$) thường được gọi là **rủi ro Bayes** (*Bayesian risk*) hay đơn giản là **rủi ro kỳ vọng** (*expected risk*).

Chú giải của người dịch: Ví dụ tính mất mát kỳ vọng

Giả sử với một đầu vào $\mathbf{x}$, mô hình có thể cho ra 3 câu trả lời khác nhau với xác suất 0.5, 0.3, 0.2, và mất mát tương ứng của từng câu là 0, 1, 4. Mất mát kỳ vọng là $0 \cdot 0.5 + 1 \cdot 0.3 + 4 \cdot 0.2 = 1.1$ — chính là công thức kỳ vọng "giá trị nhân xác suất rồi cộng lại" quen thuộc trong bài toán trò chơi may rủi ở THPT. Thay vì chỉ chấm một câu trả lời duy nhất, ta xét mọi câu trả lời mà mô hình có thể đưa ra, mỗi câu được tính theo mức độ mô hình "tin" vào nó.

Một cách khác để cải thiện việc huấn luyện là đưa tiên nghiệm vào mục tiêu. Một phương pháp điển hình là thêm một số hạng chính quy hóa (*regularization*) R vào mục tiêu, như sau

$$\hat{\theta} \;=\; \arg\min_\theta \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) + \alpha \cdot R \tag{1.81}$$

trong đó $R(\theta)$ là một hàm phạt một số khía cạnh của độ phức tạp mô hình, chẳng hạn độ lớn hoặc số lượng tham số khác 0. Hệ số α là một siêu tham số (*hyperparameter*) quy định độ mạnh của chính quy hóa, giúp cân bằng giữa việc khớp dữ liệu huấn luyện và việc giữ cho mô hình đơn giản. Bản thân việc thiết kế R là một lĩnh vực nghiên cứu quan trọng trong các hệ thống học máy thực tế. Chúng ta sẽ tìm hiểu một số kỹ thuật chính quy hóa cụ thể ở các phần sau của cuốn sách.

Chú giải của người dịch: Số hạng chính quy hóa hoạt động thế nào?

Một lựa chọn phổ biến là $R(\theta) = \sum_i \theta_i^2$ (tổng bình phương các tham số). Giả sử hai bộ tham số cho mất mát gần bằng nhau trên dữ liệu huấn luyện: $\theta_A = (1;\ 2)$ với mất mát 0.50 và $\theta_B = (10;\ -9)$ với mất mát 0.48. Với $\alpha = 0.01$: mục tiêu của A là $0.50 + 0.01 \cdot 5 = 0.55$, của B là $0.48 + 0.01 \cdot 181 = 2.29$. Vậy ta chọn A — mô hình "nhẹ nhàng" hơn, ít có nguy cơ học thuộc lòng dữ liệu. α càng lớn thì càng ưu tiên sự đơn giản; $\alpha = 0$ thì quay lại Phương trình (1.67). Gọi là *siêu* tham số vì α không được học cùng θ mà do con người chọn trước.

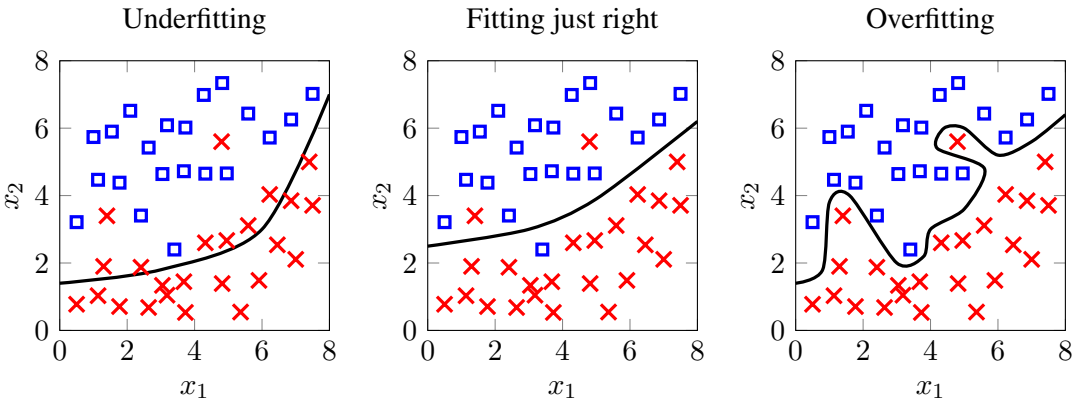
Khi đã định nghĩa được hàm mục tiêu, ta cần một thuật toán tối ưu để tìm các tham số tối ưu. Tối ưu hóa là một lĩnh vực rộng lớn trong học máy. Vì vậy, một thảo luận chi tiết về các thuật toán cụ thể nằm ngoài phạm vi của chương này. Rốt cuộc, không có một thuật toán vạn năng nào có thể giải hiệu quả mọi bài toán huấn luyện (một khái niệm liên quan đến định lý "Không có bữa trưa miễn phí" – *No Free Lunch*). Thay vào đó, các thuật toán cụ thể được lựa chọn dựa trên tính chất toán học của hàm mục tiêu. Ví dụ, ta có thể dùng *hạ gradient (gradient descent)* để huấn luyện một mô hình ngôn ngữ nơ-ron với mất mát entropy chéo [Bengio và cộng sự, 2003a], dùng *quy hoạch toàn phương (quadratic programming)* để huấn luyện một SVM với mất mát bản lề [Cortes và Vapnik, 1995], hoặc dùng **huấn luyện tỉ lệ lỗi tối thiểu** (*minimum error-rate training*) (**MERT**) để tối ưu một mô hình dịch máy thống kê với mất mát $1 -$ BLEU không khả vi [Och và Ney, 2002].

1.3.5 Quá khớp và chưa khớp

Quy trình chuẩn của học máy (có giám sát) gồm một bước huấn luyện và một bước kiểm tra. Trong khi ta có thể cố gắng cực tiểu hóa mất mát trên các mẫu huấn luyện, mô hình học được lại được dùng để xử lý những mẫu mới chưa từng gặp trước đó. Điều này tương tự với một trải nghiệm quen thuộc trong cuộc sống: một học sinh ôn thi cuối kỳ. Dù *học chăm chỉ = điểm cao* nói chung là đúng, việc học thuộc lòng câu hỏi và đáp án trong sách giáo khoa là một chiến lược tồi cho một kỳ thi có những câu hỏi mới. Câu hỏi thi chắc chắn sẽ khác với những ví dụ cụ thể đã ôn. Do đó, ta cần khả năng tổng quát hóa.

Trong học máy, *tổng quát hóa* được dùng để mô tả mức độ mà một mô hình học từ kinh nghiệm dự đoán tốt trên dữ liệu mới. Một hệ thống được coi là có hiệu năng tổng quát hóa xuất sắc nếu, với sự dẫn dắt của các thiên kiến quy nạp phù hợp, nó trích xuất thành công các khuôn mẫu ẩn sau dữ liệu huấn luyện mà không ghi nhớ nhiễu. Tuy nhiên, đạt được khả năng tổng quát hóa tốt không có nghĩa là huấn luyện ít đi. Trái lại, người làm thực tế muốn huấn luyện mô hình học máy trên nhiều dữ liệu huấn luyện hơn để ngăn nó ghi nhớ mọi thứ. Tổng quát hóa là một vấn đề rất phức tạp, được quyết định bởi nhiều yếu tố, bao gồm độ phức tạp của bài toán, kiến trúc mô hình, lượng dữ liệu huấn luyện, thuật toán huấn luyện, v.v. Dù không có quy tắc chuẩn nào bảo đảm khả năng tổng quát hóa tốt, các nhà nghiên cứu luôn cố gắng giải quyết nó bằng cách này hay cách khác.

Để mô tả mức độ một mô hình tổng quát hóa tốt cho dữ liệu mới, có hai khái niệm quan trọng: **chưa khớp** (*underfitting*) và quá khớp. Chưa khớp xảy ra khi một mô hình không học được đầy đủ các khuôn mẫu ẩn trong dữ liệu huấn luyện, dẫn đến hiệu năng kém trên cả tập huấn luyện lẫn tập kiểm tra. Điều này thường xảy ra nếu quá trình huấn luyện bị dừng quá sớm, hoặc đơn giản là năng lực (*capacity*) của



Hình 1.4: Biên quyết định của một bài toán phân loại nhị phân. Trái = chưa khớp, phải = quá khớp, giữa = khớp vừa đủ. Trong trường hợp chưa khớp, có một số lỗi hiển nhiên khi tách hai lớp điểm dữ liệu. Bằng cách dịch biên quyết định lên trên một chút (giữa), ta thu được kết quả tách thỏa đáng, trong đó hầu hết các điểm dữ liệu thuộc cùng một lớp nằm ở cùng một phía của biên quyết định. Ngược lại, để tách hoàn hảo thì cần một biên quyết định rất phức tạp (phải). (Tiêu đề các đồ thị: *Underfitting* = chưa khớp, *Fitting just right* = khớp vừa đủ, *Overfitting* = quá khớp.)

mô hình quá thấp so với bài toán. Nếu một mô hình bị chưa khớp, ta có thể cải thiện nó bằng cách tăng thời gian huấn luyện, thiết kế các đặc trưng có sức biểu diễn tốt hơn, hoặc dùng một mô hình có kiến trúc phức tạp hơn.

Ngược lại, quá khớp chỉ hiện tượng một mô hình khớp đặc biệt tốt với dữ liệu huấn luyện nhưng tổng quát hóa kém trên dữ liệu kiểm tra (xem Hình 1.4). Trong NLP, một biểu hiện cơ bản của quá khớp là khi một bộ phân loại văn bản ghi nhớ cứng nhắc chính xác bộ từ vựng đã thấy trong lúc huấn luyện, khiến nó thất bại hoàn toàn khi xuất hiện các từ OOV (xem Mục 1.2.5).

Nguyên nhân của quá khớp rất đa dạng. Một nguyên nhân chính là huấn luyện một mô hình quá phức tạp trên một tập dữ liệu nhỏ. Độ phức tạp của mô hình thường rất quan trọng khi ta thiết kế một mô hình học máy. Nếu mô hình phức tạp và có nhiều tham số, nó sẽ dễ quá khớp với một số ít mẫu hơn nhiều (xem Hình 1.4). Vấn đề sẽ khó hơn nữa nếu có dữ liệu nhiễu, do lỗi kiểu "rác vào, rác ra" (*garbage in, garbage out*) trong huấn luyện. Ngoài ra, huấn luyện quá mức là một nguyên nhân khác của quá khớp. Ví dụ, ta có thể tinh chỉnh mạnh một hệ thống để ép nó mô hình hóa dữ liệu không có lỗi nào. Hệ thống như vậy sẽ rất mong manh trước các mẫu mới, ngay cả khi đầu vào chỉ dao động nhỏ.

Chú giải của người dịch: Chưa khớp và quá khớp qua chuyện ôn thi

Hình dung ba học sinh ôn thi với cùng một đề cương 100 bài. Bạn A chỉ học qua loa vài công thức: làm sai nhiều cả bài trong đề cương lẫn bài trong đề thi — đó là *chưa khớp*. Bạn C học thuộc lòng đáp án của đúng 100 bài, kể cả vài đáp án in sai trong sách (nhiễu): làm đúng 100% đề cương nhưng gặp bài mới thì lúng túng — đó là *quá khớp*, ứng với đường biên ngoằn ngoèo ở Hình 1.4 (phải) cố "ôm" từng điểm. Bạn B hiểu phương pháp giải: có thể sai vài bài trong đề cương nhưng làm tốt đề thi — đó là "khớp vừa đủ". Dấu hiệu nhận biết thường dùng: quá khớp là khi lỗi trên dữ liệu huấn luyện rất nhỏ nhưng lỗi trên dữ liệu mới lớn hơn hẳn; chưa khớp là khi cả hai đều lớn.

Quá khớp có thể được giảm bớt theo nhiều cách. Dưới đây là một số kỹ thuật thường dùng.

- **Dùng nhiều dữ liệu huấn luyện (chất lượng cao) hơn**. Huấn luyện trên quy mô lớn giúp mô hình nắm bắt được các khuôn mẫu thực sự trong dữ liệu. Tuy nhiên, thêm dữ liệu nhiễu sẽ có tác

dụng tiêu cực.

- **Dùng dữ liệu kiểm định**. Dữ liệu kiểm định (*validation*) là một tập con rời (không giao) với phần còn lại của dữ liệu có sẵn, được dùng để đánh giá mô hình trong quá trình huấn luyện mà không trực tiếp cập nhật các tham số. Ví dụ, một tập dữ liệu có thể được chia thành một tập huấn luyện và một **tập kiểm định giữ lại** (*held-out validation set*). Người làm thực tế có thể theo dõi hiệu năng trên tập kiểm định và áp dụng **dừng sớm** (*early stopping*), chẳng hạn dừng quá trình huấn luyện ngay khi hiệu năng trên tập kiểm định bắt đầu giảm.
- **Dùng kiến trúc mô hình đơn giản hơn**. Như đã nói ở trên, dao cạo Occam là một nguyên lý ta có thể tuân theo khi thiết kế mô hình. Nếu một bài toán về bản chất là đơn giản, việc giảm năng lực của mô hình (ví dụ dùng ít tầng hoặc ít tham số hơn) buộc nó tập trung vào các khuôn mẫu chủ đạo, có tính tổng quát trong dữ liệu.
- **Chính quy hóa**. Chính quy hóa là một cách khác để kiểm soát độ phức tạp của mô hình. Thông thường, nó ràng buộc các tham số mô hình bằng các tiên nghiệm. Một ví dụ là làm trơn (xem Mục 1.2.5): nó ước lượng lại phân phối của các từ sau khi huấn luyện. Một phương pháp tổng quát hơn là huấn luyện có chính quy hóa (xem Phương trình (1.81)). Ví dụ, ta có thể định nghĩa thừa số chính quy hóa là chuẩn l_1 của các tham số, và hướng mô hình tới những bộ tham số không có giá trị tuyệt đối lớn.
- **Kết hợp nhiều mô hình**. Một dự đoán tốt hơn cũng có thể đạt được bằng cách kết hợp (*ensembling*) nhiều mô hình. Các mô hình này (gọi là **mô hình thành phần** (*component models*)) nói chung có tham số hoặc kiến trúc khác nhau, và/hoặc được huấn luyện trên các phần khác nhau của dữ liệu. Sự khác biệt giữa các mô hình có thể làm giảm nguy cơ tất cả chúng đều quá khớp dữ liệu theo đúng cùng một cách. Do đó, các mô hình này ít có khả năng mắc cùng những lỗi giống nhau khi dự đoán.

Chú giải của người dịch: Dừng sớm và kết hợp mô hình

Dừng sớm: giống như ôn thi bằng đề cương (tập huấn luyện) nhưng thỉnh thoảng tự làm một đề thử chưa từng xem (tập kiểm định). Khi điểm đề thử bắt đầu *giảm* dù điểm đề cương vẫn tăng, đó là lúc bạn đang "học vẹt" — nên dừng lại. *Kết hợp mô hình*: giống như hỏi ý kiến cả nhóm thay vì một người; ví dụ 5 mô hình bỏ phiếu, 4 nói "Đồ ăn", 1 nói "Không phải đồ ăn" thì chọn "Đồ ăn". Nếu mỗi mô hình mắc lỗi ở những chỗ khác nhau, các lỗi riêng lẻ có xu hướng bị "át" đi khi bỏ phiếu. Chuẩn l_1 của các tham số là tổng giá trị tuyệt đối $\sum_i |\theta_i|$ (xem Mục 1.1.1).

1.3.6 Dự đoán và suy luận

Mặc dù ở các mục trước chúng ta chỉ giới hạn thảo luận ở bài toán phân loại, học máy (có giám sát) không chỉ là bài toán dự đoán một nhãn cho một đối tượng đầu vào. Có nhiều loại bài toán học máy, tùy thuộc vào dạng của dự đoán được định nghĩa.

- **Phân loại** (*classification*). Phân loại có lẽ là một trong những bài toán học máy phổ biến nhất. Một hệ thống phân loại được yêu cầu gán một hoặc nhiều lớp cho một đối tượng đầu vào.
- **Hồi quy**. Trong thống kê, hồi quy nghiên cứu mối quan hệ giữa một **biến phụ thuộc** (*dependent variable*) (hay một kết quả) và một **biến độc lập** (*independent variable*). Dù hồi quy có nhiều ứng dụng, trong NLP nó thường được phát biểu dưới dạng dự đoán điểm số. Ví dụ, nhận một bài đánh giá phim làm đầu vào (tức là biến độc lập), mô hình hồi quy học cách dự đoán một điểm đề xuất (tức là biến phụ thuộc).

- **Xếp hạng** (*ranking*). Một mô hình xếp hạng dự đoán thứ tự của một tập các đối tượng đầu vào. Ví dụ, một mô hình xếp hạng một số bản dịch theo chất lượng dịch.

- **Dự đoán cấu trúc** (*structure prediction*). Nhiều mô hình học máy được yêu cầu cho ra không chỉ một giá trị thực hay một lớp mà còn một cây hay một chuỗi. Bài toán dự đoán các đầu ra có cấu trúc được gọi là dự đoán cấu trúc. Ví dụ, một bộ phân tích cú pháp (*syntactic parser*) là một hệ thống dự đoán cấu trúc, vì đầu ra của nó là một cấu trúc cây.

Mặc dù là một khía cạnh nền tảng của học máy, việc dự đoán theo thông lệ thường được coi là đơn giản, bởi nhiều mô hình và phương pháp được thử nghiệm trên các bài toán phân loại và hồi quy chuẩn. Mặt khác, dự đoán không hề đơn giản trong các bài toán dự đoán cấu trúc như phân tích cú pháp và dịch máy, vốn rất phổ biến trong NLP. Về bản chất, dự đoán một cây hay một chuỗi là một bài toán tìm kiếm (*search*). Ví dụ, về lý thuyết có vô số bản dịch hợp lệ cho một câu ở ngôn ngữ nguồn. Ngay cả khi có một mô hình hoàn hảo để đánh giá mọi bản dịch, việc tìm ra bản dịch tối ưu trong không gian tìm kiếm này vẫn là một thách thức tính toán nghiêm trọng.

Trong những trường hợp như vậy, ta phải dùng các cách hiệu quả để thực hiện tìm kiếm. Điều này được thực hiện hoặc bằng cách sử dụng các thuật toán tìm kiếm tổng quát từ lĩnh vực trí tuệ nhân tạo, hoặc bằng cách phát triển các heuristic (*heuristics*) mới cho từng bài toán cụ thể. Hơn nữa, việc phân tích bài toán tìm kiếm này cho ta một góc nhìn quan trọng về các lỗi mà một mô hình học máy mắc phải: lỗi có thể được phân thành lỗi mô hình (*model errors*) (do mô hình hóa xác suất không chính xác) hoặc lỗi tìm kiếm (*search errors*) (do thuật toán tìm kiếm không tìm được điểm tối ưu thực sự). Dù loại bỏ lỗi tìm kiếm là một mục tiêu chính, làm được điều đó thường đòi hỏi một lượng tính toán lớn đến mức không thể chấp nhận. Vì vậy, triển khai một mô hình học máy cho mục đích thực tế luôn đòi hỏi sự đánh đổi giữa hiệu quả và độ chính xác. Chúng ta sẽ thấy các ví dụ về sự đánh đổi này trong Chương 5.

Chú giải của người dịch: Vì sao dự đoán một câu lại là bài toán tìm kiếm?

Với phân loại 2 lớp, ta chỉ cần tính điểm 2 lựa chọn rồi chọn cái lớn hơn. Nhưng một bản dịch dài 20 từ, mỗi vị trí chọn từ một bộ từ vựng 10 000 từ, có tới $10\,000^{20} = 10^{80}$ khả năng — xấp xỉ số nguyên tử trong vũ trụ quan sát được! Không thể thử hết, nên ta dùng các chiến lược tìm kiếm "khôn" (như chọn từng từ tốt nhất một, hay giữ lại vài ứng viên tốt nhất ở mỗi bước) và chấp nhận có thể bỏ lỡ đáp án tối ưu. Khi kết quả sai, ta phân biệt: *lỗi mô hình* là khi mô hình chấm điểm bản dịch sai cao hơn bản dịch đúng; *lỗi tìm kiếm* là khi mô hình chấm đúng nhưng thuật toán tìm kiếm không tìm tới được bản dịch đúng.

1.4 Lựa chọn và đánh giá mô hình

Với hầu hết các bài toán học máy, mục tiêu là tìm được một mô hình hoạt động tốt nhất trên dữ liệu mới. Từ mục tiêu này có thể tách ra hai bài toán [Hastie và cộng sự, 2009]:

- **Lựa chọn mô hình** (*model selection*). Chọn mô hình tốt nhất trên dữ liệu huấn luyện theo một tiêu chí nào đó.

- **Đánh giá mô hình** (*model evaluation*). Ước lượng hiệu năng của một mô hình cho trước trên dữ liệu mới.

Như đã nói ở Mục 1.3.4, hàm mất mát (*loss function*) (hay hàm lỗi – *error function*) là cách thông dụng để đo lỗi của một dự đoán $\mathbf{y}_\theta = f_\theta(\mathbf{x})$ so với một đáp án chuẩn (*gold-standard*) $\mathbf{y}_{\text{gold}}$. Cho K mẫu

huấn luyện có nhãn $\{(\mathbf{x}^{(1)}, \mathbf{y}^{(1)}_{\text{gold}}), \cdots, (\mathbf{x}^{(K)}, \mathbf{y}^{(K)}_{\text{gold}})\}$, **lỗi huấn luyện** (*training error*) được cho bởi

$$\text{Err}_{\text{train}} \;=\; \mathcal{L}(\{\mathbf{y}^{(k)}_{\theta}\}, \{\mathbf{y}^{(k)}_{\text{gold}}\}), \tag{1.82}$$

trong đó $\{\mathbf{y}^{(k)}_{\theta}\}$ là các dự đoán trên tập dữ liệu huấn luyện, còn $\{\mathbf{y}^{(k)}_{\text{gold}}\}$ là các đáp án chuẩn tương ứng. Nói chung, $\mathcal{L}(\{\mathbf{y}^{(k)}_{\theta}\}, \{\mathbf{y}^{(k)}_{\text{gold}}\})$ được định nghĩa là mất mát trung bình trên toàn bộ các mẫu huấn luyện:

$$\mathcal{L}(\{\mathbf{y}^{(k)}_{\theta}\}, \{\mathbf{y}^{(k)}_{\text{gold}}\}) \;=\; \frac{1}{K}\sum_{k=1}^{K} \mathcal{L}(\mathbf{y}^{(k)}_{\theta}, \mathbf{y}^{(k)}_{\text{gold}}), \tag{1.83}$$

hoặc được định nghĩa là một độ đo duy nhất tính trên toàn bộ tập mẫu huấn luyện. Tương tự, ta có thể định nghĩa **lỗi kiểm tra** (*test error*) trên tập dữ liệu kiểm tra, ký hiệu là Err_{test}. Lỗi kiểm tra đóng vai trò là một ước lượng của **lỗi tổng quát hóa** (*generalization error*), cho biết mô hình được kỳ vọng sẽ hoạt động tốt đến đâu trên dữ liệu mới.

Chú giải của người dịch: Lỗi huấn luyện và lỗi kiểm tra

Hãy hình dung một học sinh ôn thi bằng một bộ đề có đáp án. *Lỗi huấn luyện* giống như số câu sai khi học sinh tự làm lại chính bộ đề đã ôn; *lỗi kiểm tra* là số câu sai trong kỳ thi thật với đề mới. Điều ta thực sự quan tâm là kỳ thi thật (dữ liệu mới), tức *lỗi tổng quát hóa*. Phương trình (1.83) chỉ là phép lấy trung bình cộng quen thuộc: nếu có $K = 4$ mẫu với mất mát lần lượt $0.2,\ 0.5,\ 0.1,\ 0.4$ thì $\text{Err}_{\text{train}} = \frac{1}{4}(0.2 + 0.5 + 0.1 + 0.4) = 0.3$.

Trong các mục trước, ta đã giả định rằng cực tiểu hóa $\text{Err}_{\text{train}}$ là mục tiêu chính của quá trình huấn luyện, tức là $\hat{\theta} = \arg\min_{\theta} \text{Err}_{\text{train}}$. Tuy nhiên, ta không thể mặc định rằng hàm thu được $f_{\hat{\theta}}(\cdot)$ đồng thời cũng đạt Err_{test} nhỏ nhất. Hình 1.5 minh họa các đường cong học (*learning curves*) của một hệ thống dịch máy. Rõ ràng Err_{test} không tương quan hoàn toàn với $\text{Err}_{\text{train}}$. Trong khi lỗi huấn luyện giảm liên tục theo tiến trình huấn luyện, lỗi kiểm tra ban đầu cũng giảm theo nhưng rồi bắt đầu tăng lên, cho thấy mô hình bị quá khớp (*overfitting*). Điều này khiến bài toán phức tạp hơn một chút, vì ta không thể lúc nào cũng tin vào $\text{Err}_{\text{train}}$, dù nó là (và nên là) thước đo mức độ tốt của việc huấn luyện. Chắc chắn ta cần một cách nào đó để chọn được mô hình tốt hơn, chứ không chỉ nhìn vào $\text{Err}_{\text{train}}$.

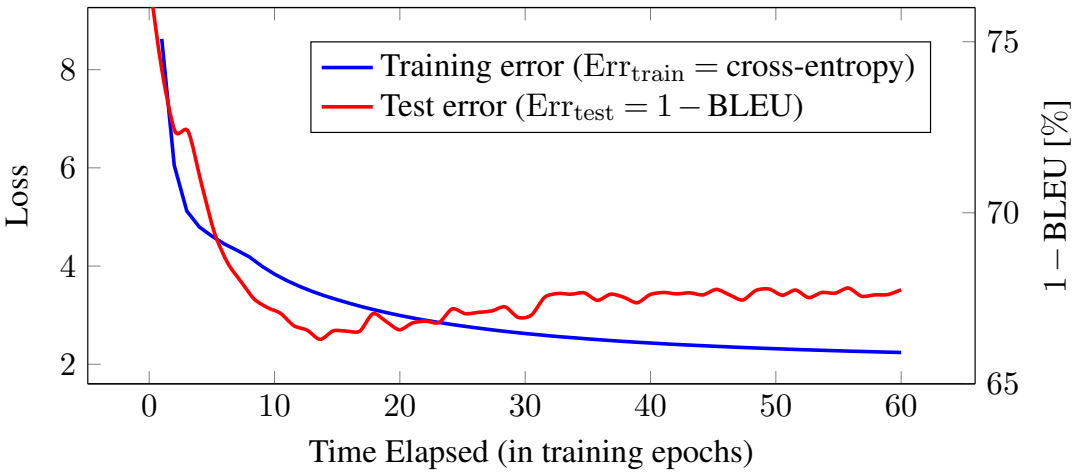
Chú giải của người dịch: BLEU và epoch

BLEU là một điểm số tự động (từ 0 đến 100%) đo mức độ giống nhau giữa bản dịch của máy và bản dịch tham khảo của con người; BLEU càng cao càng tốt, nên $1 - \text{BLEU}$ càng nhỏ càng tốt và có thể dùng làm "lỗi". Một **epoch** là một lượt mô hình duyệt qua toàn bộ dữ liệu huấn luyện. Trong Hình 1.5, sau khoảng 15 epoch, mô hình bắt đầu "học thuộc lòng" dữ liệu huấn luyện thay vì học quy luật chung: lỗi huấn luyện vẫn giảm nhưng lỗi kiểm tra lại tăng — đó chính là quá khớp.

1.4.1 Các chiến lược lựa chọn mô hình

Việc chọn mô hình tối ưu chỉ dựa trên dữ liệu huấn luyện về bản chất là một tham vọng lớn: ta kỳ vọng một mô hình học máy sẽ tổng quát hóa ra toàn bộ tổng thể (*population*) từ một tập mẫu hữu hạn, thường là nhỏ. Từ góc nhìn học thống kê, việc tổng quát hóa chỉ có thể thực hiện được nhờ cách ta phát biểu bài toán học. Một giả định nền tảng trong học máy tiêu chuẩn là giả định i.i.d. (*i.i.d. assumption*) — rằng cả dữ liệu huấn luyện lẫn dữ liệu kiểm tra đều *độc lập và có cùng phân phối* (*independent and identically distributed*), được sinh ra bởi cùng một phân phối sinh dữ liệu (*data-generating distribution*) nền tảng.

Nếu giả định này đúng và tập dữ liệu đủ lớn, một mô hình học từ dữ liệu huấn luyện nhiều khả năng sẽ hoạt động tốt trên dữ liệu kiểm tra. Ngược lại, nếu tập huấn luyện và tập kiểm tra được sinh ra một



Hình 1.5: Đường cong lỗi huấn luyện và lỗi kiểm tra của một hệ thống học máy. Lỗi huấn luyện được đo bằng mất mát entropy chéo, còn lỗi kiểm tra được đo bằng $1 - \mathrm{BLEU}$. Mọi số liệu được thu thập bằng cách chạy một hệ thống dịch máy nơ-ron trên bộ đánh giá chuẩn IWSLT Đức–Anh (De-En). Lỗi huấn luyện tiếp tục giảm khi số epoch huấn luyện tăng lên. Ngược lại, lỗi kiểm tra có xu hướng giảm trước rồi tăng sau. Khi lỗi kiểm tra bắt đầu tăng, nhiều khả năng mô hình đã quá khớp dữ liệu huấn luyện. (Nhãn trong hình: *Training error* = lỗi huấn luyện, *Test error* = lỗi kiểm tra, *Loss* = mất mát, *Time Elapsed (in training epochs)* = thời gian đã trôi qua, tính bằng số epoch huấn luyện.)

cách hoàn toàn tùy ý, không có ràng buộc gì (ví dụ, theo một phân phối đều trên toàn bộ không gian của mọi hàm có thể có), thì mô hình sẽ không thể tổng quát hóa. Khi đó, việc quan sát dữ liệu huấn luyện hoàn toàn không cho ta thông tin gì về dữ liệu kiểm tra.

Ý tưởng này được hình thức hóa bởi **Định lý Không có bữa trưa miễn phí** (*No Free Lunch Theorem*) [Wolpert, 1996; Wolpert và Macready, 1997], định lý chứng minh rằng nếu ta lấy trung bình hiệu năng trên mọi bài toán có thể có về mặt toán học, thì mọi thuật toán học đều tốt như nhau (tương đương với đoán ngẫu nhiên)[17]. Nói cách khác, một thuật toán học vạn năng là điều không thể: một thuật toán chỉ có thể thành công nếu nó chứa những thiên kiến quy nạp (*inductive bias*) phù hợp với chính bài toán đang xét. May mắn là, trong các ứng dụng thực tế, dữ liệu huấn luyện và dữ liệu kiểm tra nói chung được giả định là ít nhất cũng tuân theo một phân phối nào đó ở một mức độ nhất định. Vì vậy, thực sự có những cách để nắm bắt phân phối này và cải thiện khả năng tổng quát hóa của mô hình. Khi cải thiện các hệ thống học máy, người ta thường xét hai tình huống:

- Với một kiến trúc mô hình và thuật toán huấn luyện cố định, làm thế nào để xây dựng hoặc tuyển chọn dữ liệu huấn luyện nhằm giảm lỗi kiểm tra?
- Với một tập dữ liệu huấn luyện cố định, làm thế nào để thiết kế hoặc chọn kiến trúc mô hình nhằm giảm lỗi kiểm tra?

Tình huống thứ nhất phức tạp và liên quan đến các vấn đề kỹ thuật dữ liệu thực tiễn, như gán nhãn, làm sạch dữ liệu và ước lượng chất lượng dữ liệu. Ta sẽ để các chủ đề này cho các chương sau và ở đây tập trung vào bài toán lựa chọn mô hình trong tình huống thứ hai.

[17] Định lý không có bữa trưa miễn phí ban đầu được trình bày trong bối cảnh phân loại [Wolpert, 1996], và sau đó được mở rộng cho các bài toán tìm kiếm và tối ưu hóa [Wolpert và Macready, 1997].

Chú giải của người dịch: i.i.d. và "Không có bữa trưa miễn phí"

i.i.d. nghĩa là mỗi mẫu dữ liệu được "rút thăm" độc lập từ cùng một "chiếc hộp" (cùng phân phối). Ví dụ: nếu đề luyện thi và đề thi thật đều được rút ngẫu nhiên từ cùng một ngân hàng câu hỏi, thì luyện tốt sẽ thi tốt. Nếu đề thi thật lấy từ một môn hoàn toàn khác, việc ôn luyện chẳng giúp gì. Định lý "Không có bữa trưa miễn phí" nói rằng không có phương pháp học nào tốt nhất cho *mọi* bài toán: một phương pháp thắng ở loại bài toán này thì phải thua ở loại khác. Do đó, muốn học tốt, ta phải đưa vào mô hình những giả định (thiên kiến quy nạp) phù hợp với bài toán cụ thể.

1. Độ phức tạp của mô hình

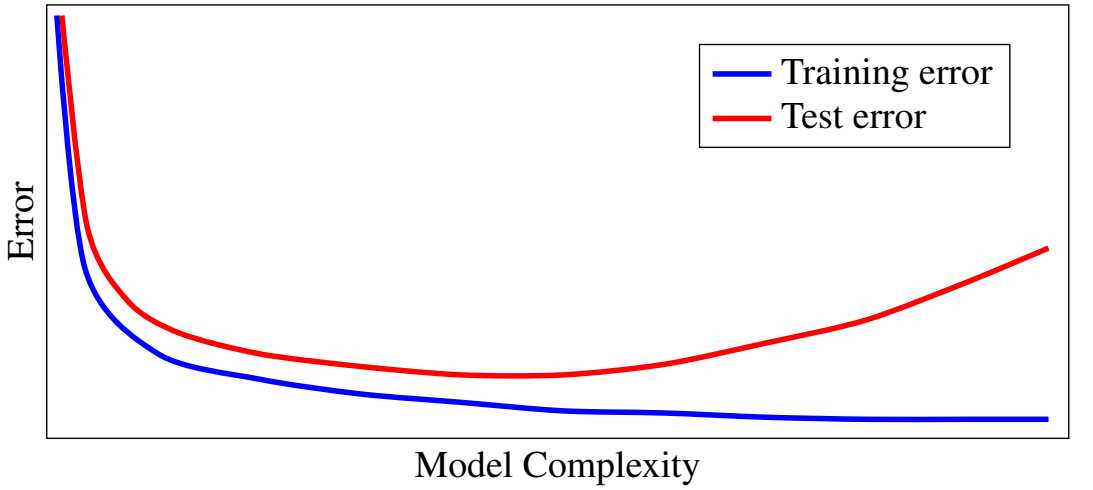
Phương pháp lựa chọn mô hình đơn giản nhất có lẽ là thử các mô hình trên dữ liệu kiểm định (*validation data*). Thông thường, dữ liệu này không trùng lặp với dữ liệu huấn luyện lẫn dữ liệu kiểm tra, nhưng được giả định là được sinh ra theo cùng cách với dữ liệu kiểm tra. Tuy nhiên, không phải lúc nào cũng có sẵn dữ liệu như vậy. Trong một số trường hợp, ta thậm chí không biết gì về dữ liệu kiểm tra. Vì thế, nhiều phương pháp lựa chọn mô hình không cần đến dữ liệu kiểm định.

Một lựa chọn thay thế phổ biến là dựa vào **độ phức tạp của mô hình** (*model complexity*) (hay **năng lực của mô hình** (*model capacity*)) làm chỉ báo định hướng. Trong học máy, độ phức tạp của mô hình có thể được hiểu theo nhiều cách khác nhau. Về mặt trực giác, một mô hình phi tuyến phức tạp hơn một mô hình tuyến tính, và một mô hình có hàng triệu tham số phức tạp hơn một mô hình chỉ có vài tham số. Những định nghĩa hình thức hơn có thể tìm thấy trong lý thuyết học tính toán, chẳng hạn **chiều Vapnik–Chervonenkis (VC)** (*Vapnik-Chervonenkis (VC) dimension*) [Vapnik và Chervonenkis, 1971]. Với mục đích của chúng ta, ta hiểu độ phức tạp của mô hình một cách rộng rãi là thước đo *năng lực biểu đạt* của mô hình: độ phức tạp càng cao thì không gian giả thuyết (*hypothesis space*) càng lớn.

Dù các mô hình có năng lực biểu đạt cao rất mạnh, độ phức tạp tăng lên không hẳn lúc nào cũng có lợi. Mô hình phức tạp rất dễ bị quá khớp, đặc biệt khi được huấn luyện trên tập dữ liệu nhỏ. Ngược lại, mô hình quá đơn giản dễ bị chưa khớp (*underfitting*). Mục tiêu của lựa chọn mô hình là xác định mức độ phức tạp "tối ưu" của mô hình. Hình 1.6 vẽ lỗi huấn luyện và lỗi kiểm tra theo năng lực của mô hình. Điều quan trọng là độ phức tạp tối ưu không nằm ở chỗ lỗi huấn luyện hội tụ về không, mà ở điểm lỗi kiểm định/kiểm tra đạt cực tiểu toàn cục trước khi bắt đầu tăng lên.

Khi không có lỗi kiểm định tường minh, người ta thường dùng dao cạo Occam (*Occam's Razor*): khi có nhiều mô hình hoạt động tốt tương đương nhau trên dữ liệu huấn luyện, nên ưu tiên mô hình đơn giản nhất. Có nhiều tiêu chí để định lượng độ phức tạp của mô hình. Ví dụ,

- **Số lượng tham số**. Tuy rất đơn giản, việc đếm số tham số tự do (*free parameters*) là một phương pháp kinh nghiệm trực quan và rất hiệu quả. Trong các mô hình làm trơn tuyến tính, điều này đôi khi được hình thức hóa thành **số bậc tự do hiệu dụng** (*effective degrees of freedom*), được định nghĩa là vết (*trace*) của ma trận chiếu (*projection matrix*) ánh xạ đáp án đúng $\mathbf{y}_{\text{gold}}$ thành các dự đoán $\mathbf{y}_\theta$.
- **Chuẩn p của tham số**. Chuẩn p của một ma trận trọng số là một chỉ báo mạnh về độ phức tạp (xem Mục 1.1.1). Ví dụ, dưới ràng buộc chuẩn L_1 hoặc L_2, một mô hình có giá trị tuyệt đối của tham số lớn hơn sẽ có biên quyết định cong hơn nhiều và do đó được coi là phức tạp hơn.
- **Độ dài mô tả**. Bắt nguồn từ lý thuyết thông tin và nén dữ liệu, tiêu chí này đánh giá số bit cần thiết để mã hóa mô hình cùng các lỗi của nó. Nguyên lý **Độ dài mô tả tối thiểu** (*Minimum Description Length*) (**MDL**) ưu tiên mô hình cho mức nén dữ liệu cao nhất.
- **Chiều VC**. Khái niệm này vốn bắt nguồn từ lý thuyết học tính toán. Chiều VC định lượng một



Hình 1.6: Đường cong lỗi huấn luyện và lỗi kiểm tra ứng với các mức độ phức tạp khác nhau của mô hình. Mô hình phức tạp giúp giảm lỗi huấn luyện vì chúng có thể tính được các hàm phức tạp để khớp với các điểm dữ liệu. Tuy nhiên, độ phức tạp lớn dễ dẫn đến quá khớp hơn và có hại cho khả năng tổng quát hóa của mô hình. Chẳng hạn, lỗi kiểm tra tăng lên khi độ phức tạp tiếp tục được thêm vào. (Nhãn trong hình: *Error* = lỗi, *Model Complexity* = độ phức tạp của mô hình, *Training error* = lỗi huấn luyện, *Test error* = lỗi kiểm tra.)

cách hình thức năng lực của mô hình là số điểm dữ liệu lớn nhất mà họ bộ phân loại có thể tách trọn vẹn (*shatter*) (tức là gán được mọi tổ hợp nhãn nhị phân có thể có cho các điểm đó).

Ngoài ra còn những lựa chọn khác để định nghĩa tiêu chí, bao gồm **tiêu chuẩn thông tin Akaike** (*Akaike information criterion*) (**AIC**), **tiêu chuẩn thông tin Bayes** (*Bayesian information criterion*) (**BIC**), **độ dài thông điệp tối thiểu** (*minimum message length*) (**MML**), v.v. Có thể tìm thấy chúng trong nhiều giáo trình về thống kê và/hoặc học thống kê [Burnham và Anderson, 2002; Konishi và Kitagawa, 2007; Hastie và cộng sự, 2009].

Chú giải của người dịch: Chiều VC qua một ví dụ

Xét các bộ phân loại là *đường thẳng* trong mặt phẳng (một bên đường là nhãn $+$, bên kia là nhãn $-$). Với 3 điểm không thẳng hàng, ta luôn kẻ được một đường thẳng tách chúng theo *bất kỳ* cách gán nhãn nào trong $2^3 = 8$ cách — ta nói đường thẳng "tách trọn vẹn" được 3 điểm. Nhưng với 4 điểm là 4 đỉnh của hình vuông, cách gán nhãn "hai đỉnh chéo nhau là $+$, hai đỉnh còn lại là $-$" thì không đường thẳng nào tách được. Người ta chứng minh được rằng không có 4 điểm nào được tách trọn vẹn, nên chiều VC của họ đường thẳng trong mặt phẳng là 3. Chiều VC càng lớn, họ mô hình càng "linh hoạt".

Dao cạo Occam là nguyên tắc "nếu hai lời giải thích tốt như nhau, hãy chọn cái đơn giản hơn". Về *vết* của một ma trận vuông: đó là tổng các phần tử trên đường chéo chính, ví dụ vết của $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ là $1 + 4 = 5$.

2. Sự đánh đổi giữa độ chệch và phương sai

Việc kiểm soát độ phức tạp của mô hình để tránh quá khớp và chưa khớp cũng gắn liền với sự đánh đổi giữa độ chệch và phương sai (*bias-variance tradeoff*). Độ chệch (*bias*) (hay độ chệch dự đoán – *prediction bias*) là mức chênh lệch giữa dự đoán của mô hình và giá trị thực. Trong thống kê, độ chệch là một **sai số hệ thống** (*systematic error*), không thể triệt tiêu dù ta thực hiện rất nhiều lần thí nghiệm lặp lại. Nói chung, sai số do độ chệch xuất phát từ những giả định sai về bài toán, chẳng hạn xấp xỉ một bài toán phi tuyến bằng một mô hình tuyến tính. Điều này rất thú vị! Ta có thể liên hệ sai số do độ chệch ở đây với thiên kiến quy nạp được dùng trong thiết kế mô hình (xem Mục 1.3.2). Ví dụ, với dữ liệu huấn luyện cho trước, một mô hình có độ chệch lớn thường là do có nhiều giả định hơn và mô hình chưa đủ phức tạp. Nói đơn giản, ta có thể nói rằng nhiều thiên kiến quy nạp hơn (hoặc mạnh hơn) có thể dẫn đến độ phức tạp mô hình thấp hơn và sai số do độ chệch trong dự đoán lớn hơn. Đôi khi, thuật ngữ *bias* được dùng như cách gọi tắt cho cả độ chệch trong dự đoán (theo góc nhìn thống kê) lẫn thiên kiến quy nạp (theo góc nhìn thiết kế mô hình), dù chúng được coi là có ý nghĩa khác nhau[18].

Mặt khác, phương sai (*variance*) mô tả mức độ phân tán của các dự đoán khi dữ liệu huấn luyện thay đổi. Sai số do phương sai cũng có tương quan với độ phức tạp của mô hình. Ví dụ, một mô hình phức tạp thường có phương sai cao hơn.

Cả độ chệch và phương sai đều góp phần vào lỗi tổng thể của một hệ thống. Một minh họa kinh điển là phân rã độ chệch–phương sai (*bias-variance decomposition*) của sai số bình phương trung bình (*mean squared error*). Ở đây ta dùng ký hiệu hơi khác so với các mục trước. Gọi $\mathcal{D}$ là một tập gồm K mẫu huấn luyện và $f_{\hat{\theta}(\mathcal{D})}(\cdot)$ là mô hình học được trên $\mathcal{D}$. Ngoài ra, với một mẫu mới $\mathbf{x}$, gọi $\mathbf{y}_{\hat{\theta}(\mathcal{D})} = f_{\hat{\theta}(\mathcal{D})}(\mathbf{x})$ là dự đoán của mô hình và $\mathbf{y}_{\text{gold}}$ là dự đoán "đúng". Độ chệch và phương sai được định nghĩa là:

$$\text{bias} = \mathbb{E}_{\mathcal{D}}[\mathbf{y}_{\hat{\theta}(\mathcal{D})}] - \mathbf{y}_{\text{gold}} \tag{1.84}$$

$$\text{variance} = \mathbb{E}_{\mathcal{D}}\left[\left\|\mathbb{E}_{\mathcal{D}}[\mathbf{y}_{\hat{\theta}(\mathcal{D})}] - \mathbf{y}_{\hat{\theta}(\mathcal{D})}\right\|^2\right], \tag{1.85}$$

trong đó $\mathbb{E}_{\mathcal{D}}[\mathbf{y}_{\hat{\theta}(\mathcal{D})}]$ là dự đoán kỳ vọng, lấy trung bình trên mọi tập dữ liệu huấn luyện có thể có với kích thước K. Như vậy, độ chệch biểu diễn độ sai lệch giữa dự đoán kỳ vọng và giá trị thực, còn phương sai biểu diễn bình phương độ lệch kỳ vọng của từng dự đoán riêng lẻ so với giá trị trung bình của chúng[19]. Lấy sai số bình phương trung bình làm thước đo lỗi, ta có thể viết lỗi kỳ vọng như sau:

$$\begin{aligned}\text{error} &= \mathbb{E}_{\mathcal{D}}\left[\left\|\mathbf{y}_{\hat{\theta}(\mathcal{D})} - \mathbf{y}_{\text{gold}}\right\|^2\right] \\ &= \|\text{bias}\|^2 + \text{variance}.\end{aligned} \tag{1.86}$$

Chú giải của người dịch: Độ chệch và phương sai: ví dụ bắn cung

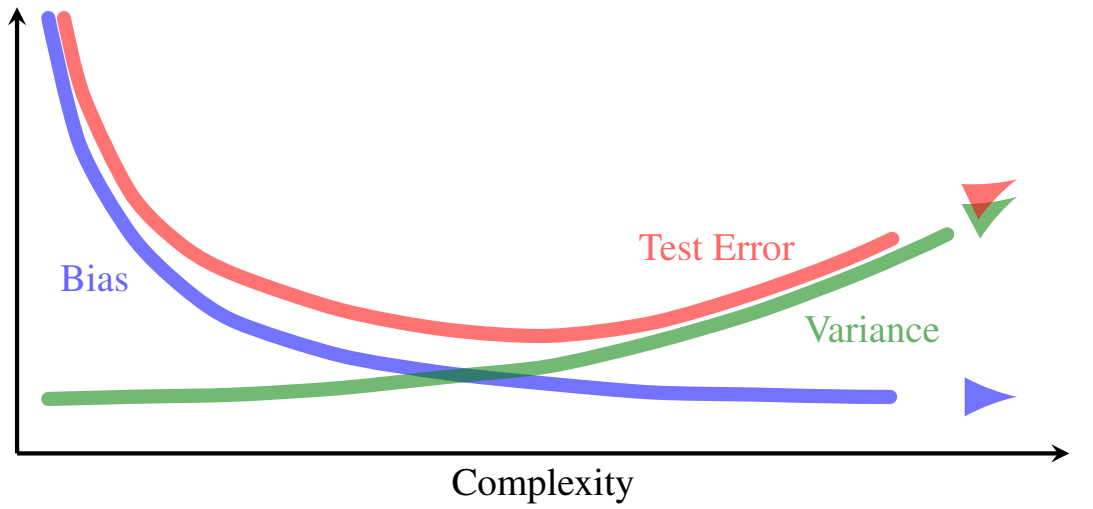
Hãy hình dung mỗi tập huấn luyện $\mathcal{D}$ khác nhau cho ra một "mũi tên" (một dự đoán), còn $\mathbf{y}_{\text{gold}}$ là hồng tâm. *Độ chệch* là khoảng cách từ *tâm của chùm tên* tới hồng tâm (bắn lệch có hệ thống); *phương sai* cho biết các mũi tên *tản mát* quanh tâm chùm đến mức nào. Ví dụ số (trường hợp một chiều): giả sử giá trị đúng là $y_{\text{gold}} = 5$ và trên 4 tập huấn luyện khác nhau mô hình dự đoán $4,\ 6,\ 7,\ 7$. Trung bình là 6, nên bias $= 6 - 5 = 1$; phương sai là $\frac{1}{4}[(6-4)^2 + (6-6)^2 + (6-7)^2 + (6-7)^2] = \frac{6}{4} = 1.5$. Lỗi bình phương trung bình là $\frac{1}{4}[(4-5)^2 + (6-5)^2 + (7-5)^2 + (7-5)^2] = \frac{10}{4} = 2.5 = 1^2 + 1.5$, đúng như Phương trình (1.86).

[18] Trong thống kê, *bias* (độ chệch) thường được dùng để mô tả một khía cạnh nào đó của một bộ ước lượng (*estimator*).

[19] Nói một cách chặt chẽ, bias như định nghĩa ở đây là một vectơ, còn variance là một số vô hướng. Hơn nữa, vì $\mathbf{y}$ biểu diễn một phân phối xác suất trong chương này, sự khác biệt giữa các phân phối thường được đo bằng các độ đo phân kỳ (như entropy chéo hoặc độ phân kỳ Kullback–Leibler) thay vì khoảng cách Euclid. Công thức độ chệch dựa trên khoảng cách Euclid trình bày ở đây chỉ đóng vai trò minh họa trực quan về mặt toán học. Về mặt khái niệm, có thể thay hiệu số Euclid bằng một độ đo phân kỳ khi đánh giá các phân phối.

Ký hiệu $\mathbb{E}_{\mathcal{D}}[\cdot]$ đọc là "kỳ vọng theo $\mathcal{D}$", hiểu nôm na là trung bình trên mọi tập huấn luyện có thể có; $\|\mathbf{v}\|^2$ là bình phương độ dài của vectơ $\mathbf{v}$.

Để sai số bình phương trung bình thấp hơn, giảm đồng thời cả độ chệch lẫn phương sai rõ ràng là mục tiêu lý tưởng. Tuy nhiên, giảm đại lượng này thường làm tăng đại lượng kia (xem Hình 1.7). Người làm thực tế phải tìm mức độ phức tạp tối ưu của mô hình để cân bằng sự đánh đổi này, nhờ đó tránh được quá khớp và chưa khớp. Sự cân bằng này phụ thuộc rất nhiều vào độ phức tạp vốn có của bài toán. Một mô hình tuyến tính đơn giản nhìn chung có phương sai thấp nhưng độ chệch cao, điều này dẫn đến chưa khớp nếu bài toán thực chất có tính phi tuyến cao.



Hình 1.7: Độ chệch và phương sai theo độ phức tạp của mô hình [Goodfellow và cộng sự, 2016]. Các đường cong cho thấy sự xung đột khi giảm đồng thời sai số do độ chệch và sai số do phương sai. Bằng cách thay đổi độ phức tạp của mô hình, ta có thể thu được một mô hình độ chệch thấp – phương sai cao, hoặc một mô hình độ chệch cao – phương sai thấp. Cả hai trường hợp đều có lỗi kiểm tra cao. Ví dụ, mô hình phương sai cao thường có độ phức tạp lớn hơn. Dù mô hình như vậy có thể xử lý các bài toán phức tạp, nó dễ quá khớp dữ liệu hơn. Mặt khác, mô hình độ chệch cao thường là mô hình đơn giản hơn nhưng lại có xu hướng chưa khớp dữ liệu. Để cải thiện khả năng tổng quát hóa trên dữ liệu kiểm tra, ta có thể tìm một điểm cân bằng giữa độ chệch và phương sai. Chẳng hạn, lỗi kiểm tra thấp khi ta chọn một mô hình "phức tạp vừa phải". (Nhãn trong hình: *Bias* = độ chệch, *Variance* = phương sai, *Test Error* = lỗi kiểm tra, *Complexity* = độ phức tạp.)

Quay lại bài toán lựa chọn mô hình, sự đánh đổi giữa độ chệch và phương sai không phải là một quy tắc để chọn mô hình, mà là một nguyên lý định hướng cho việc thiết kế mô hình. Thường thì ta phải chấp nhận những thỏa hiệp để tạo ra một mô hình tổng quát hóa tốt. Cũng cần lưu ý rằng, trong nhiều ứng dụng, mô hình phức tạp thường đi kèm với chi phí tính toán kém hiệu quả. Một cách làm hợp lý có thể là bắt đầu với một mô hình đơn giản và chỉ tăng thêm độ phức tạp khi cần.

3. Kết hợp mô hình

Chọn một mô hình tối ưu duy nhất không phải là chiến lược duy nhất để giảm lỗi tổng quát hóa. Một cách khác là kết hợp nhiều mô hình thành một bộ dự đoán duy nhất, vững chắc hơn. Mô thức này được gọi là **học kết hợp** (*ensemble learning*) [Seni và cộng sự, 2010; Zhou, 2012a]. Ý tưởng cốt lõi của học kết hợp là huấn luyện một tập các **mô hình thành phần** (*component models*) (thường gọi là bộ học cơ sở (*base learners*)) rồi tổng hợp đầu ra của chúng để tạo ra một dự đoán tốt hơn. Cách đơn giản nhất

là **lấy trung bình mô hình** (*model averaging*) (hay bỏ phiếu mềm (*soft voting*)), tức là tính trung bình các điểm số dự đoán từ từng mô hình riêng lẻ. Các phương pháp tinh vi hơn, như hỗn hợp chuyên gia (*mixtures of experts*), gán trọng số động hoặc kết hợp các biểu diễn bên trong của những mô hình con này.

Nói chung, các mô hình thành phần nên có *sự đa dạng trong dự đoán*. Sự đa dạng này thường đạt được bằng cách huấn luyện các mô hình trên những tập con khác nhau của dữ liệu, dùng các tập con đặc trưng khác nhau, hoặc dùng các cách khởi tạo trọng số khác nhau. Từ góc nhìn thống kê, những phương pháp này có thể giảm lỗi tổng quát hóa một cách có hệ thống. Ví dụ, bagging (*bagging*) giúp giảm phương sai [Breiman, 1996], còn boosting (*boosting*) giúp giảm độ chệch [Schapire, 1990]. Điều này gắn trở lại với những gì đã trình bày ở Mục 1.4.1: lỗi tổng quát hóa có thể được giảm bằng cách giảm sai số do độ chệch hoặc giảm sai số do phương sai.

Tuy nhiên, việc bàn về cách kết hợp mô hình nằm ngoài phạm vi chương này. Dù có thể không hoàn toàn chính xác khi xếp kết hợp mô hình vào chủ đề lựa chọn mô hình, nó vẫn có thể được xem là một cách cải thiện khả năng tổng quát hóa. Theo nghĩa này, cả kết hợp mô hình và lựa chọn mô hình đều giải quyết những vấn đề có chung một chủ đề. Thực tế, kết hợp mô hình hiệu quả đáng kể cho nhiều nhiệm vụ NLP. Chẳng hạn, phần lớn các hệ thống tiên tiến nhất trong NLP đều dựa trên sự kết hợp nhiều mô hình.

Chú giải của người dịch: Học kết hợp: "trí tuệ đám đông"

Giống như hỏi ý kiến nhiều giám khảo rồi lấy điểm trung bình thay vì tin một người, học kết hợp lấy trung bình dự đoán của nhiều mô hình. Ví dụ: ba mô hình cho xác suất lớp "Đồ ăn" của một văn bản là 0.9, 0.6 và 0.3; bỏ phiếu mềm cho $(0.9 + 0.6 + 0.3)/3 = 0.6$. Khi các mô hình mắc lỗi *khác nhau* (đa dạng), các lỗi ngẫu nhiên có xu hướng bù trừ nhau, nên phương sai giảm. *Bagging* huấn luyện mỗi mô hình trên một tập dữ liệu được rút ngẫu nhiên có hoàn lại từ tập gốc; *boosting* huấn luyện tuần tự, mô hình sau tập trung sửa những mẫu mà các mô hình trước làm sai.

1.4.2 Dữ liệu huấn luyện, kiểm định và kiểm tra

Bây giờ ta chuyển sang vấn đề dữ liệu. Như đã bàn ở các mục trước, trong giai đoạn huấn luyện, một tập dữ liệu huấn luyện (*training dataset*) được dùng để khớp các tham số của mô hình. Trong giai đoạn kiểm tra, một tập dữ liệu kiểm tra (*test dataset*) được dùng để đánh giá mô hình đã học. Liên quan chặt chẽ nhưng khác với dữ liệu kiểm tra là tập dữ liệu kiểm định (*validation dataset*). Dù tập kiểm định cũng được dùng để đánh giá, nó được sử dụng *trong* giai đoạn huấn luyện. Mục đích chính của nó là định hướng việc lựa chọn mô hình và tinh chỉnh siêu tham số (*hyperparameter*) (chẳng hạn, xác định khi nào áp dụng dừng sớm (*early stopping*)).

Trong tình huống lý tưởng, nhà nghiên cứu có các tập dữ liệu riêng biệt cho huấn luyện, kiểm định và kiểm tra. Tuy nhiên, điều này hiếm khi thực tế trong các ứng dụng thật. Ví dụ, nhà phát triển không thể tiếp cận dữ liệu thật của người dùng trong tương lai trước khi hệ thống được triển khai. Từ góc nhìn khoa học, không có dữ liệu mới "thật sự" để kiểm tra — khi bạn đã thấy dữ liệu mới, nó không còn mới nữa.

Một cách đơn giản, như trong nhiều bài báo nghiên cứu, là kiểm chứng các mô hình học máy trên các nhiệm vụ đánh giá chuẩn. Trong các nhiệm vụ này, toàn bộ dữ liệu đã được chuẩn bị sẵn, và tất cả những gì bạn cần làm là chạy mô hình của mình trên dữ liệu đó. Cách này giúp dễ dàng so sánh trực tiếp các hệ thống khác nhau, vì tất cả chúng được huấn luyện và kiểm tra trên cùng các tập dữ liệu. Đôi khi, ta chỉ được cung cấp một tập dữ liệu duy nhất chưa được chia sẵn. Khi đó, dữ liệu có thể được chia thành nhiều phần, mỗi phần dùng cho một mục đích. Ví dụ, có thể chia 60% để huấn luyện, 20% để

kiểm định và 20% để kiểm tra.

Dù việc chia dữ liệu cho ta một cách đánh giá hiệu năng của mô hình, kết quả đánh giá không phải lúc nào cũng ổn định do sai lệch lấy mẫu (*sampling bias*). Đôi khi hiệu năng thay đổi rất nhiều giữa các lần chia dữ liệu khác nhau. Vấn đề này càng rõ khi tập dữ liệu quá nhỏ để huấn luyện hay kiểm tra một cách đầy đủ.

Một cách thông dụng để giảm ảnh hưởng của sai lệch này là **kiểm định chéo** (*cross-validation*). Kiểm định chéo là một phương pháp lấy mẫu lại (*resampling*), trong đó dữ liệu được phân chia lặp đi lặp lại nhiều lần, và kết quả cuối cùng là trung bình các kết quả đánh giá qua mọi vòng. Một biến thể đơn giản là lấy mẫu con ngẫu nhiên (*random subsampling*), lặp lại việc phân chia dữ liệu một cách ngẫu nhiên và lấy trung bình hiệu năng qua các lần chạy. Một phương pháp có hệ thống hơn là kiểm định chéo k-phần (k-*fold cross-validation*). Phương pháp này chia dữ liệu thành k phần (hay *fold*) rời nhau. Trong mỗi vòng kiểm định chéo, $k-1$ phần được gộp lại làm dữ liệu huấn luyện, còn phần duy nhất còn lại được giữ ra làm dữ liệu kiểm tra (hoặc kiểm định). Ví dụ, trong kiểm định chéo 10-phần, mô hình được huấn luyện và đánh giá 10 lần, mỗi lần dùng một phần khác nhau làm tập kiểm tra.

Chú giải của người dịch: Kiểm định chéo 5-phần

Giả sử có 100 mẫu, chia thành 5 phần, mỗi phần 20 mẫu: P1,…,P5. Vòng 1: huấn luyện trên P2–P5, kiểm tra trên P1; vòng 2: huấn luyện trên P1, P3–P5, kiểm tra trên P2;… cho đến vòng 5. Nếu độ chính xác ở 5 vòng là $80\%, 84\%, 78\%, 82\%, 81\%$ thì kết quả báo cáo là trung bình 81%. Mỗi mẫu đều được dùng để kiểm tra đúng một lần, nên kết quả ít phụ thuộc vào "may rủi" của một lần chia.

Thêm một lưu ý về quy mô dữ liệu. Với người làm thực tế, một trong những câu hỏi thường gặp nhất là cần bao nhiêu mẫu thì đủ để học được một mô hình tốt. Hiếm khi có một sự đồng thuận chung. Dù lý thuyết học tính toán cung cấp các cận lỗi hình thức ứng với một kích thước mẫu cụ thể, kinh nghiệm thực tế phổ biến vẫn chỉ đơn giản là "càng nhiều càng tốt". Tuy nhiên, học máy hiện đại ngày càng chú trọng **hiệu quả mẫu** (*sample efficiency*). Một hệ thống hiệu quả về mẫu có thể đạt hiệu năng cao với số ví dụ huấn luyện ít hơn đáng kể hoặc cần ít epoch huấn luyện hơn. Ví dụ, trong các nghiên cứu NLP gần đây, tinh chỉnh (*fine-tuning*) một mô hình tiền huấn luyện (*pre-trained*) rất hiệu quả về mẫu vì nó tận dụng lượng tri thức có sẵn phong phú, chỉ cần cập nhật ở mức vừa phải thay vì học các biểu diễn từ đầu. Một biểu hiện cực đoan của điều này là **học few-shot** (*few-shot learning*), với mục tiêu tổng quát hóa một cách vững chắc chỉ sau khi quan sát một số ít mẫu của nhiệm vụ cụ thể.

1.4.3 Thước đo hiệu năng

Là một phần thiết yếu của mọi bài toán học máy, thước đo hiệu năng (*performance measure*) mô tả một hệ thống hoạt động tốt đến đâu trên một lượng dữ liệu nào đó. Thông thường, nó được dùng hoặc để thiết kế mục tiêu huấn luyện, hoặc để đánh giá kết quả của hệ thống cuối cùng. Ví dụ, tất cả các hàm mất mát mô tả ở Mục 1.3.4 đều có thể dùng làm thước đo hiệu năng.

Khi đánh giá hiệu năng trên dữ liệu kiểm tra, các độ đo thường được thiết kế để đếm số lỗi thực tế. Vì vậy, dùng lại các hàm mất mát trong huấn luyện có thể không phải là lựa chọn tốt để báo cáo điểm số cuối cùng. Chẳng hạn, các độ đo được dùng rộng rãi cho bài toán phân loại là độ chính xác dự đoán (*precision*), độ phủ (*recall*) và điểm F_1 (F_1 *score*). Chúng được đề xuất để định lượng năng lực của hệ thống phân loại ở một số khía cạnh nhất định: với một lớp c, độ chính xác dự đoán tính tỉ lệ dự đoán đúng trong số tất cả các mẫu được dự đoán là c, còn độ phủ tính tỉ lệ dự đoán đúng trên tất cả các mẫu có nhãn là c. Điểm F_1 là độ đo kết hợp độ chính xác dự đoán và độ phủ.

Chú giải của người dịch: Precision, recall và F_1 qua ví dụ

Có 10 văn bản thật sự thuộc lớp "Đồ ăn". Bộ phân loại gắn nhãn "Đồ ăn" cho 8 văn bản, trong đó 6 văn bản đúng là "Đồ ăn". Khi đó *precision* $= 6/8 = 0.75$ (trong những gì đã "bắt", bao nhiêu phần đúng), *recall* $= 6/10 = 0.6$ (trong những gì cần bắt, bắt được bao nhiêu phần). F_1 là *trung bình điều hòa* của hai số này: $F_1 = \frac{2 \cdot P \cdot R}{P+R} = \frac{2 \cdot 0.75 \cdot 0.6}{0.75+0.6} \approx 0.667$. Trung bình điều hòa "phạt" nặng khi một trong hai số quá thấp.

Lưu ý rằng các thước đo hiệu năng không nhất thiết được thiết kế để tối ưu hóa. Theo nghĩa đó, chúng có thể không đảm bảo một số tính chất toán học, chẳng hạn khả vi hay liên tục. Một ví dụ là độ đo **BLEU** được dùng rộng rãi trong dịch máy [Papineni và cộng sự, 2002]. BLEU tính một điểm độ chính xác n-gram (*n-gram precision*) có hiệu chỉnh, kết hợp với một hình phạt độ ngắn (*brevity penalty*). Những phép đếm rời rạc này khiến độ đo về bản chất không khả vi. NLP có rất nhiều độ đo đánh giá riêng cho từng nhiệm vụ như vậy. Điều này dẫn tới một sự lệch pha giữa huấn luyện và đánh giá (*training-evaluation discrepancy*) mang tính nền tảng: hàm mất mát được tối ưu trong huấn luyện khác với độ đo dùng để đánh giá mô hình cuối cùng. Đôi khi người làm thực tế cần tính đến khoảng cách này khi thiết kế hệ thống.

Một thách thức khác với các thước đo hiệu năng trong NLP là một đầu vào thường có nhiều đầu ra hợp lệ. Ví dụ, một câu ở ngôn ngữ nguồn thường có nhiều bản dịch tốt. Một giải pháp là tính đến nhiều đáp án chuẩn khi thiết kế thước đo hiệu năng. BLEU là một trường hợp như vậy: nó đếm số đoạn dịch đúng lớn nhất trên tất cả các bản dịch tham khảo (*reference translations*). Giải pháp thứ hai là đánh giá bởi con người (*human evaluation*). Cách đánh giá này chính xác hơn nhưng tất nhiên tốn kém hơn. Khi phát triển hệ thống thực tế, người làm thực tế thường huấn luyện và tinh chỉnh hệ thống bằng các thước đo tự động, và chỉ dùng đến đánh giá của con người cho lần kiểm tra cuối cùng.

Chú giải của người dịch: n-gram và tính khả vi

Một *n-gram* là một dãy n từ liên tiếp; ví dụ câu "tôi thích trái cây" có các 2-gram "tôi thích", "thích trái", "trái cây". BLEU đếm xem bao nhiêu n-gram của bản dịch máy xuất hiện trong bản dịch tham khảo. Vì kết quả đếm là số nguyên, thay đổi rất nhỏ tham số mô hình thường không làm điểm BLEU thay đổi chút nào (hoặc làm nó "nhảy bậc"), nên không thể lấy đạo hàm để biết nên chỉnh tham số theo hướng nào. Đó là lý do người ta huấn luyện bằng entropy chéo (trơn, lấy đạo hàm được) nhưng lại báo cáo kết quả bằng BLEU.

1.4.4 Kiểm định ý nghĩa thống kê

Giả sử bạn đang cải tiến một hệ thống theo cách nào đó, bạn có thể tự hỏi liệu cải tiến đó có đủ đáng kể hay không. Tất cả những gì bạn có là một thước đo hiệu năng. Vì vậy bạn có thể biết chênh lệch hiệu năng giữa hai thời điểm bất kỳ trong quá trình phát triển hệ thống, nhưng không thể biết chênh lệch đó là thật hay chỉ do ngẫu nhiên.

Trong ví dụ này, bạn đang ngầm cố gắng chấp nhận hay bác bỏ khẳng định rằng một hệ thống tốt hơn (hoặc không tốt hơn) một hệ thống khác. Trong thống kê, **kiểm định ý nghĩa thống kê** (*significance tests*) cung cấp một khuôn khổ hình thức để mô hình hóa bài toán này. Giả sử ta đánh giá hai hệ thống A và B trên nhiều tập dữ liệu hoặc nhiều cách chia dữ liệu kiểm tra, dùng cùng một thước đo hiệu năng. Ta phát biểu hai giả thuyết đối lập:

H_0: Hệ thống A không tốt hơn hệ thống B.
H_1: Hệ thống A tốt hơn hệ thống B.

trong đó H_0 là **giả thuyết không** (*null hypothesis*), còn H_1 là **giả thuyết đối** (*alternative hypothesis*),

mâu thuẫn với giả thuyết không. Bằng cách kiểm định các giả thuyết này dựa trên dữ liệu thực nghiệm, ta muốn xác định liệu có đủ bằng chứng để bác bỏ H_0 và chấp nhận H_1, hay không thể bác bỏ H_0. Kiểm định thống kê về bản chất có thể mắc lỗi: ta có thể bác bỏ nhầm giả thuyết không trong khi nó thực ra đúng (**sai lầm loại I** (*Type I error*), hay dương tính giả (*false positive*)), hoặc không bác bỏ giả thuyết không trong khi nó thực ra sai (**sai lầm loại II** (*Type II error*), hay âm tính giả (*false negative*)). Hai loại sai lầm này đánh đổi lẫn nhau. Một cách tiếp cận chuẩn trong thống kê tần suất (*frequentist statistics*) là giới hạn xác suất mắc sai lầm loại I dưới một ngưỡng định trước α, đồng thời cố gắng cực tiểu hóa sai lầm loại II. Ngưỡng α này được gọi là **mức ý nghĩa** (*significance level*) của kiểm định. Thông lệ trong học máy là chọn mức ý nghĩa 1% hoặc 5%.

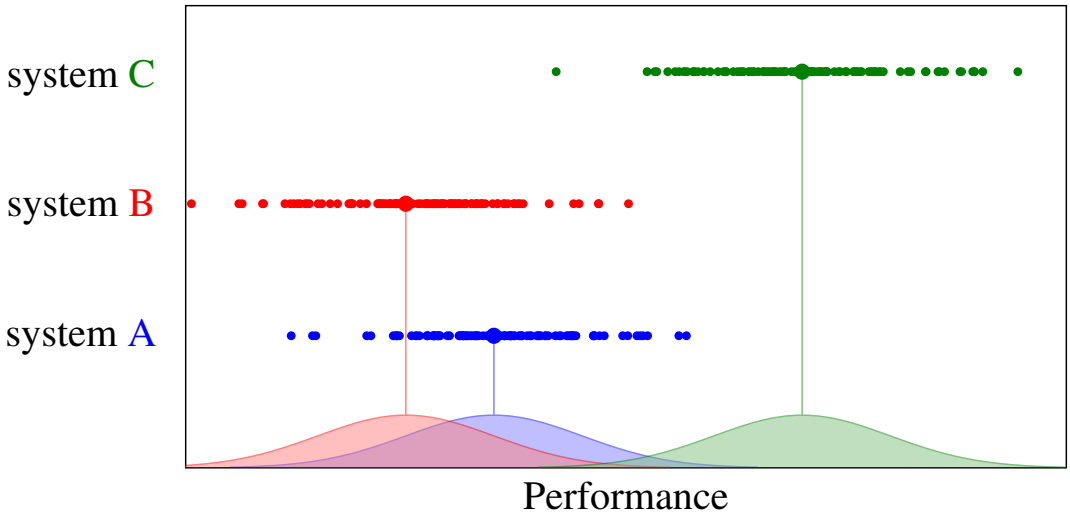
Khi tiến hành kiểm định, ta tính một **giá trị** p (*p-value*). Lưu ý rằng giá trị p *không phải* là xác suất mắc sai lầm loại I. Thay vào đó, nó là xác suất quan sát được một chênh lệch hiệu năng cực đoan bằng hoặc hơn chênh lệch thực sự quan sát được, *với giả định giả thuyết không là đúng*. Nếu giá trị p này nhỏ hơn mức ý nghĩa α, ta bác bỏ giả thuyết không. Ví dụ, với mức ý nghĩa 5%, một giá trị p quan sát được bằng 3% cho thấy cải thiện hiệu năng là có ý nghĩa thống kê. Để tìm hiểu đầy đủ hơn về kiểm định giả thuyết, bạn đọc có thể tham khảo các giáo trình thống kê thông dụng [McClave và Sincich, 2006; Freedman và cộng sự, 2007; Freedman, 2009].

Chú giải của người dịch: Giá trị p qua ví dụ đồng xu

Bạn nghi một đồng xu bị lệch về mặt ngửa. H_0: đồng xu cân đối. Tung 10 lần được 9 lần ngửa. Nếu H_0 đúng, xác suất được 9 hoặc 10 lần ngửa (tức kết quả "cực đoan bằng hoặc hơn") là $\frac{\binom{10}{9}+\binom{10}{10}}{2^{10}} = \frac{11}{1024} \approx 1.1\%$. Đó là giá trị p. Vì $1.1\% < 5\%$, ta bác bỏ H_0 ở mức ý nghĩa 5% (nhưng không bác bỏ được ở mức 1%). Ý tưởng: nếu kết quả quan sát được "quá khó xảy ra" khi H_0 đúng, ta nghi ngờ H_0. So sánh hai hệ thống NLP cũng vậy: "đồng xu cân đối" tương ứng với "hai hệ thống thực ra ngang nhau".

Lưu ý rằng kết luận của kiểm định ý nghĩa phụ thuộc vào nhiều yếu tố, như số lượng thí nghiệm và phương sai trong kết quả thí nghiệm. Một vấn đề khi áp dụng kiểm định ý nghĩa cho các nhiệm vụ NLP là thường có rất ít tập dữ liệu để chạy thí nghiệm [Dror và cộng sự, 2020]. Khi có thể yên tâm giả định rằng thống kê kiểm định (*test statistic*) tuân theo một phân phối xác suất đã biết (chẳng hạn phân phối chuẩn), ta có thể dùng kiểm định tham số (*parametric test*). Tuy nhiên, hiệu năng của các hệ thống NLP trong thực tế hiếm khi thỏa mãn những giả định chặt chẽ này. Do đó, các nhà nghiên cứu thường dựa vào các **kiểm định phi tham số** (*non-parametric tests*) cổ điển (không đặt giả định nào về phân phối nền tảng) hoặc các phương pháp lấy mẫu lại thực nghiệm (như bootstrap (*bootstrapping*) hoặc ngẫu nhiên hóa xấp xỉ (*approximate randomization*)). Những kỹ thuật lấy mẫu lại này xấp xỉ phân phối nền tảng bằng cách lấy mẫu lặp đi lặp lại từ tập dữ liệu sẵn có hoặc xáo trộn ngẫu nhiên đầu ra của các hệ thống.

Kiểm định ý nghĩa rất quan trọng để rút ra những kết luận thuyết phục khi phát triển hệ thống học máy, dù chúng thường vô tình bị bỏ qua. Hình 1.8 cho thấy kết quả đánh giá của ba mô hình. Mỗi mô hình được chạy nhiều lần với các tham số khởi tạo khác nhau. Dù hệ thống A vượt trội hệ thống B về hiệu năng trung bình, kết quả của chúng có phương sai lớn. Kiểm định ý nghĩa cho thấy chênh lệch này không có ý nghĩa thống kê. Ngược lại, chênh lệch giữa hệ thống A và hệ thống C là có ý nghĩa, vì hiệu năng của chúng khác nhau đủ lớn trong hầu hết các trường hợp. Mặt khác, các nhà nghiên cứu đã nhận thấy rằng thực sự có những ngưỡng tăng hiệu năng cho thấy ý nghĩa thống kê trong một số hoàn cảnh nhất định. Ví dụ, ta có thể nói rằng ý nghĩa thống kê có thể được chỉ ra một cách gần đúng bằng một mức tăng nhất định của độ đo nếu ta so sánh các hệ thống tương tự nhau [Berg-Kirkpatrick và cộng sự, 2012].



Hình 1.8: Hiệu năng của ba hệ thống học máy. Với mỗi hệ thống có nhiều kết quả khác nhau vì ta đưa vào một số yếu tố ngẫu nhiên trong huấn luyện (ví dụ xáo trộn dữ liệu, điểm khởi đầu ngẫu nhiên, v.v.). Dù có vẻ Hệ thống A vượt trội Hệ thống B, thực ra không có khác biệt thật sự giữa chúng, vì phân phối hiệu năng của chúng chồng lấn lên nhau rất nhiều (xem phần dưới cùng của hình). Khi so sánh Hệ thống C với Hệ thống A hoặc B, chênh lệch hiệu năng là có ý nghĩa, vì ta có thể chấp nhận giả thuyết H_1 (tức là Hệ thống C vượt trội Hệ thống A hoặc B) với một số lượng lớn thí nghiệm. (Nhãn trong hình: *system* = hệ thống, *Performance* = hiệu năng.)

1.5 Các nhiệm vụ NLP dưới góc nhìn học máy

Dù có rất nhiều loại nhiệm vụ NLP, phần lớn trong số chúng có thể được phát biểu trong một khuôn khổ học máy chung. Điều này cho phép một mô thức thống nhất để giải quyết những bài toán NLP đa dạng bằng các phương pháp học máy tổng quát. Thông thường, một nhiệm vụ NLP có thể được mô tả là học cách ánh xạ các đơn vị ngôn ngữ sang một đầu ra nào đó. Theo ký hiệu dùng trong chương này, ta dùng $\mathbf{x}$ để chỉ vectơ (hoặc ma trận) đặc trưng đầu vào của một nhiệm vụ NLP, và dùng $f(\mathbf{x})$ để chỉ hàm được học để xử lý $\mathbf{x}$. Dưới đây là một số nhiệm vụ thường gặp trong NLP.

1.5.1 Phân loại

Giả sử có một tập các lớp hay nhãn $\mathcal{C}$. Mỗi lớp được biểu diễn bằng một số nguyên riêng trong $\{1, \cdots, |\mathcal{C}|\}$. Một mô hình phân loại (*classification model*) là một hàm ánh xạ đầu vào $\mathbf{x}$ thành một vectơ $|\mathcal{C}|$ chiều $\mathbf{y}$, tức là $\mathbf{y} = f(\mathbf{x})$. Mỗi phần tử của $\mathbf{y}$ là một điểm số ứng với lớp i, ký hiệu là $y(i)$. Nhiệm vụ ở đây là gán $\mathbf{x}$ vào một hoặc nhiều lớp có điểm số cao nhất. Lấy phân loại đơn nhãn làm ví dụ. Dự đoán được cho bởi phương trình

$$\hat{c} = \underset{1 \le i \le |\mathcal{C}|}{\arg\max}\, y(i), \tag{1.87}$$

trong đó $\hat{c}$ là lớp "tốt nhất" được gán cho $\mathbf{x}$. Đôi khi ta cần một đầu ra mang dạng xác suất (xem Mục 1.2.1). Gọi $\psi(\cdot)$ là một hàm chuẩn hóa một vectơ thành một phân phối[20]. Khi đó ta thu được một bộ phân loại xác suất:

$$\mathbf{y} = \psi(f(\mathbf{x})). \tag{1.90}$$

Chú giải của người dịch: Chuẩn hóa thành phân phối: ví dụ số

Giả sử có 3 lớp và $f(\mathbf{x})$ cho điểm $\mathbf{a} = [2 \;\; 1 \;\; 1]$. Theo (1.87), $\hat{c} = 1$ vì $y(1) = 2$ lớn nhất. Chuẩn hóa theo (1.88): $[\frac{2}{4} \;\; \frac{1}{4} \;\; \frac{1}{4}] = [0.5 \;\; 0.25 \;\; 0.25]$. Chuẩn hóa bằng Softmax (1.89): $e^2 \approx 7.39$, $e^1 \approx 2.72$, tổng ≈ 12.83, nên được $\approx [0.576 \;\; 0.212 \;\; 0.212]$. Cả hai cách đều cho các số không âm có tổng bằng 1 (một phân phối xác suất) và giữ nguyên thứ tự lớn–nhỏ. Softmax có ưu điểm là dùng được cả khi điểm số âm (vì $e^a > 0$ với mọi a), còn cách (1.88) đòi hỏi các điểm số không âm.

Phân loại có lẽ là bài toán phổ biến nhất trong NLP. Ngoài việc xếp tài liệu vào các lớp định trước, còn có rất nhiều ứng dụng khác. Trong số đó có việc chọn nghĩa cho một từ [Yarowsky, 1994], xác định cực tính (*polarity*) (tích cực hay tiêu cực) của một câu [Pang và cộng sự, 2002], kiểm tra xem hai thực thể có nên được liên kết với nhau hay không [Krebs và cộng sự, 2018], phân loại cách một tham tố ngữ nghĩa (*semantic argument*) gắn với một động từ [Gildea và Jurafsky, 2002], v.v. Khi áp dụng một mô hình phân loại cho các nhiệm vụ này, tất cả những gì bạn cần làm là thiết kế dạng của $\mathbf{x}$ và tập các lớp.

1.5.2 Gán nhãn chuỗi

Một mở rộng của bài toán phân loại tiêu chuẩn là phân loại đồng thời một tập các mẫu. **Gán nhãn chuỗi** (*sequence labeling*) là một ví dụ về bài toán như vậy. Trong gán nhãn chuỗi, đầu vào là một chuỗi gồm n token, chẳng hạn một chuỗi n từ. Hệ thống gán nhãn chuỗi phải gán cho mỗi token đầu vào $\mathbf{x}(i)$ một nhãn $l(i)$. Ở đây chữ in đậm trong $\mathbf{x}(i)$ dùng để nhấn mạnh rằng token được biểu diễn dưới dạng một vectơ đặc trưng. Để tiện, ta viết $\mathbf{x}(i)$ là $\mathbf{x}_i$ và $l(i)$ là l_i. Hàm $f(\cdot)$ ánh xạ chuỗi $\mathbf{x}_1 \cdots \mathbf{x}_n$ thành một chuỗi khác $\mathbf{y}_1 \cdots \mathbf{y}_n$, trong đó $\mathbf{y}_i$ là vectơ đầu ra tương ứng với $\mathbf{x}_i$. Điều này có thể được phát biểu là:

$$\begin{bmatrix}\mathbf{y}_1 & \cdots & \mathbf{y}_n\end{bmatrix} = f\Big(\begin{bmatrix}\mathbf{x}_1 & \cdots & \mathbf{x}_n\end{bmatrix}\Big). \tag{1.91}$$

Với vectơ $\mathbf{y}_i$, mỗi phần tử $y_i(c)$ tương ứng với điểm dự đoán của một lớp $c \in \mathcal{C}$. Lưu ý rằng $f(\cdot)$ cho phép sử dụng một ngữ cảnh rộng hơn. Ví dụ, có thể đặt điều kiện dự đoán $\mathbf{y}_i$ trên toàn bộ chuỗi đầu vào [Lafferty và cộng sự, 2001]. Đầu ra cuối cùng của hệ thống có thể được định nghĩa là chuỗi nhãn "tối ưu" suy ra từ $\mathbf{y}_1 \cdots \mathbf{y}_n$. Một cách đơn giản là chọn chuỗi nhãn cực đại hóa tổng điểm số trên mọi vị trí

$$\begin{bmatrix}\hat{l}_1 & \cdots & \hat{l}_n\end{bmatrix} = \underset{l_1,\cdots,l_n \in \mathcal{C}}{\arg\max} \sum_{i=1}^{n} y_i(l_i). \tag{1.92}$$

Một ứng dụng trực tiếp của gán nhãn chuỗi trong NLP là gán thẻ cho từng token của chuỗi đầu vào, chẳng hạn **gán nhãn từ loại** (*part-of-speech tagging*) (hay **POS tagging**). Hơn nữa, gán nhãn chuỗi có

[20] Một ý tưởng tương tự có thể tìm thấy ở Phương trình (1.46). Cho một vectơ $\mathbf{a} = \begin{bmatrix}a(1) & \cdots & a(n)\end{bmatrix}$, hàm chuẩn hóa có dạng:

$$\psi(\mathbf{a}) = \begin{bmatrix}\frac{a(1)}{\sum_{i=1}^{n} a(i)} & \cdots & \frac{a(n)}{\sum_{i=1}^{n} a(i)}\end{bmatrix} \tag{1.88}$$

Một cách khác là dùng hàm Softmax:

$$\psi(\mathbf{a}) = \begin{bmatrix}\frac{\exp(a(1))}{\sum_{i=1}^{n} \exp(a(i))} & \cdots & \frac{\exp(a(n))}{\sum_{i=1}^{n} \exp(a(i))}\end{bmatrix} \tag{1.89}$$

Tokens :	Most	are	expected	to	fall	below	previous-month	levels	.
POS tags :	JJS	VBP	VBN	TO	VB	IN	JJ	NNS	·
Chunk tags :	B-NP	B-VP	I-VP	I-VP	I-VP	B-PP	B-NP	I-NP	O
	NP	VP				PP	NP		

Hình 1.9: Một ví dụ gán nhãn chuỗi cho bài toán gán nhãn từ loại và phân đoạn cụm. Ví dụ được lấy từ dữ liệu huấn luyện của nhiệm vụ chung CoNLL 2000. Mỗi token được gán một nhãn từ loại và một nhãn cụm. Nhãn cụm có ký tự đầu được chọn từ $\{B, I, O\}$, trong đó B = bắt đầu một cụm, I = bên trong một cụm, O = bên ngoài cụm. Như vậy, một cụm luôn bắt đầu bằng một nhãn "B", có thể theo sau là các nhãn "I". Ví dụ, cụm VP (cụm động từ) trong ví dụ trải trên chuỗi nhãn cụm "B-VP I-VP I-VP I-VP". (Câu ví dụ giữ nguyên tiếng Anh: "Most are expected to fall below previous-month levels." – "Phần lớn được dự kiến sẽ giảm xuống dưới mức của tháng trước." Nhãn trong hình: *Tokens* = các token, *POS tags* = nhãn từ loại, *Chunk tags* = nhãn cụm.)

thể xử lý những bài toán phức tạp hơn bằng cách dùng nhãn một cách khéo léo. Một ví dụ nổi tiếng là dùng định dạng nhãn "IOB" để nhận diện các cụm trải trên nhiều token (gọi là **phân đoạn cụm** (*chunking*)). Trong phương pháp này, "I", "O" và "B" lần lượt chỉ một token nằm trong cụm, một token nằm ngoài cụm, và token ngoài cùng bên trái của một cụm [Ramshaw và Marcus, 1995]. Như vậy, một cụm luôn bắt đầu bằng "B" và kết thúc ngay trước "B" tiếp theo hoặc một "O" mới. Xem Hình 1.9 để thấy kết quả gán nhãn từ loại và phân đoạn cụm trên một câu ví dụ. Vì gán nhãn chuỗi cho phép gán nhãn cả token lẫn các đoạn (span), nó đã được áp dụng với kết quả tốt cho nhiều nhiệm vụ, bao gồm gán nhãn từ loại [Bahl và Mercer, 1976], phân đoạn cụm [Tjong Kim Sang và Buchholz, 2000], **nhận dạng thực thể có tên** (*named entity recognition*) (**NER**) [Tjong Kim Sang, 2002], v.v.

Chú giải của người dịch: Nhãn từ loại và IOB

Các nhãn trong Hình 1.9 là ký hiệu từ loại tiếng Anh chuẩn: JJS = tính từ so sánh nhất, VBP/VBN/VB = các dạng động từ, TO = từ "to", IN = giới từ, JJ = tính từ, NNS = danh từ số nhiều; NP/VP/PP = cụm danh từ/cụm động từ/cụm giới từ. Ví dụ tiếng Việt với NER theo IOB: "Ông/O Nguyễn/B-PER Văn/I-PER An/I-PER đến/O Hà/B-LOC Nội/I-LOC" — ta đọc ra hai thực thể: tên người "Nguyễn Văn An" và địa danh "Hà Nội". Nhờ mẹo IOB, bài toán tìm *đoạn văn bản* được quy về bài toán gán *một nhãn cho mỗi token*. Phương trình (1.92) nói rằng nếu điểm của mỗi vị trí độc lập với nhau, chuỗi nhãn tốt nhất chính là chọn nhãn có điểm cao nhất ở từng vị trí.

1.5.3 Mô hình hóa ngôn ngữ/Dự đoán từ

Mô hình hóa ngôn ngữ thống kê (*statistical language modeling*) (gọi tắt là **mô hình hóa ngôn ngữ** (*language modeling*)) là nhiệm vụ gán một xác suất $\Pr(w_1, \cdots, w_n)$ cho một chuỗi từ $w_1 \cdots w_n$. Xác suất đồng thời này thường được phân tích thành tích của các xác suất có điều kiện bằng quy tắc chuỗi:

$$\begin{aligned}\Pr(w_1, \cdots, w_n) &= \Pr(w_1) \cdot \Pr(w_2|w_1) \cdots \Pr(w_n|w_1, \cdots, w_{n-1}) \\ &= \prod_{i=1}^{n} \Pr(w_i|w_1, \cdots, w_{i-1}).\end{aligned} \tag{1.93}$$

Phương trình (1.93) mô tả một quy trình sinh chuỗi từ từ trái sang phải (gọi là sinh **tự hồi quy** (*auto-regressive*)). Ước lượng $\Pr(w_i|w_1, \cdots, w_{i-1})$ về bản chất là bài toán dự đoán từ bị thiếu: ta che

đi từ cuối cùng của một chuỗi và hướng dẫn mô hình ngôn ngữ dự đoán đúng từ ở vị trí đó. Dưới đây là một chuỗi từ bị thiếu từ cuối cùng.

Pride and prejudice is one of the best known ___
("Kiêu hãnh và định kiến là một trong những ___ nổi tiếng nhất")

Ta có thể dùng lại ý tưởng của bài toán phân loại để mô hình hóa phân phối xác suất $\Pr(__ \mid \textit{Pride}, \textit{and}, \cdots, \textit{known})$. Gọi $\mathbf{x}_i$ là biểu diễn vectơ của w_i. Ta có thể định nghĩa một hàm đọc $\mathbf{x}_1 \cdots \mathbf{x}_{i-1}$ và tạo ra một vectơ $\mathbf{h}_i$:

$$\mathbf{h}_i \ = \ f(\mathbf{x}_1, \cdots, \mathbf{x}_{i-1}), \tag{1.94}$$

trong đó $\mathbf{h}_i$ biểu diễn trạng thái trung gian của mô hình tại vị trí i. Để thu được một phân phối xác suất hợp lệ, ta chuẩn hóa $\mathbf{h}_i$ bằng một hàm chuẩn hóa $\psi(\cdot)$ nào đó. Như vậy, phân phối tại vị trí i là

$$\begin{aligned} \mathbf{y}_i &= \psi(\mathbf{h}_i) \\ &= \psi(f(\mathbf{x}_1, \cdots, \mathbf{x}_{i-1})). \end{aligned} \tag{1.95}$$

Rõ ràng $y_i(w_i)$ là xác suất của w_i khi biết các từ đứng trước, tức là $y_i(w_i) = \Pr(w_i | w_1, \cdots, w_{i-1})$. Lưu ý rằng Phương trình (1.93) chỉ xét ngữ cảnh bên trái khi dự đoán một từ. Một mở rộng tự nhiên là đặt điều kiện dự đoán trên toàn bộ ngữ cảnh có sẵn. Chẳng hạn, xét một câu có một từ bị che ở giữa.

Pride and ___ is one of the best-known novels
("Kiêu hãnh và ___ là một trong những cuốn tiểu thuyết nổi tiếng nhất")

Trong ví dụ này, ta có thể dự đoán từ bị che bằng cách dùng cả ngữ cảnh bên trái (*Pride and*) lẫn ngữ cảnh bên phải (*is one of the best known novels*):

$$\mathbf{y}_i \ = \ \psi(f(\mathbf{x}_1, \cdots, \mathbf{x}_{i-1}, \mathbf{x}_{i+1}, \cdots, \mathbf{x}_n)). \tag{1.96}$$

Đây là một mô hình *hai chiều*, thường được dùng trong các phương pháp **tự mã hóa** (*auto-encoding*) để học các mô hình biểu diễn chuỗi [Devlin và cộng sự, 2019].

Chú giải của người dịch: Quy tắc chuỗi trong mô hình ngôn ngữ: ví dụ số

Với câu "tôi thích trái cây" (coi mỗi từ là một token), Phương trình (1.93) cho $\Pr(\text{tôi thích trái cây}) = \Pr(\text{tôi}) \cdot \Pr(\text{thích}|\text{tôi}) \cdot \Pr(\text{trái}|\text{tôi thích}) \cdot \Pr(\text{cây}|\text{tôi thích trái})$. Nếu bốn xác suất này lần lượt là $0.1,\ 0.2,\ 0.05,\ 0.9$ thì xác suất cả câu là $0.1 \cdot 0.2 \cdot 0.05 \cdot 0.9 = 0.0009$. Mỗi thừa số chính là một bài toán "đoán từ tiếp theo", tức một bài toán phân loại với số lớp bằng kích thước bộ từ vựng. Mô hình *tự hồi quy* chỉ nhìn sang trái (giống GPT); mô hình *hai chiều* như (1.96) nhìn cả hai phía (giống BERT).

Lưu ý rằng dù mô hình hóa ngôn ngữ trước đây được xem là một nhiệm vụ cơ bản trong NLP, ngày nay nó được công nhận là nền tảng tuyệt đối của các hệ thống NLP hiện đại. Đặc biệt, cùng với sự phát triển của các mô hình ngôn ngữ lớn (*large language models*), dự đoán từ đã trở thành cơ chế chính nhờ đó các mô hình thu nhận tri thức sâu về ngôn ngữ và về thế giới. Như ta sẽ thấy ở các Chương 7–11, mô thức mô hình hóa ngôn ngữ được sử dụng rộng rãi để giải quyết rất nhiều nhiệm vụ hạ nguồn (*downstream tasks*) khác nhau.

1.5.4 Sinh chuỗi

Sinh chuỗi (*sequence generation*) bao gồm nhiều bài toán NLP, như dịch máy, tóm tắt văn bản (*summarization*), hỏi đáp (*question answering*), hệ thống hội thoại (*dialogue systems*), v.v. Thông

thường, nó chỉ việc ánh xạ một dữ liệu nào đó thành một chuỗi. Ở đây ta tập trung vào bài toán **chuỗi-sang-chuỗi** (*sequence-to-sequence*), trong đó một chuỗi phía nguồn được biến đổi thành một chuỗi phía đích, dù sinh chuỗi không chỉ giới hạn ở việc làm việc với cấu trúc dạng chuỗi ở phía nguồn.

Để tiện ký hiệu, từ đây trở đi ta dùng biến in đậm để chỉ các chuỗi. Ví dụ, $\mathbf{a}$ là một chuỗi có độ dài n. Nó có thể được viết là $\begin{bmatrix} a_1 & \cdots & a_n \end{bmatrix}$ hoặc $a_1 \cdots a_n$. Gọi $\mathbf{s} = s_1 \cdots s_m$ và $\mathbf{t} = t_1 \cdots t_n$ lần lượt là chuỗi nguồn và chuỗi đích. Bài toán chuỗi-sang-chuỗi có thể được mô tả là tìm chuỗi phía đích cực đại hóa $\Pr(\mathbf{t}|\mathbf{s})$:

$$\hat{\mathbf{t}} \;=\; \arg\max_{\mathbf{t}} \Pr(\mathbf{t}|\mathbf{s}). \tag{1.97}$$

Giống như mô hình hóa ngôn ngữ, $\Pr(\mathbf{t}|\mathbf{s})$ có thể được hình thức hóa theo kiểu tự hồi quy:

$$\begin{aligned}
\Pr(\mathbf{t}|\mathbf{s}) &= \Pr(t_1, \cdots, t_n|\mathbf{s}) \\
&= \Pr(t_1|\mathbf{s}) \cdot \Pr(t_2|\mathbf{s}, t_1) \cdots \Pr(t_n|\mathbf{s}, t_1, \cdots, t_{n-1}) \\
&= \prod_{i=1}^{n} \Pr(t_i|\mathbf{s}, t_1, \cdots, t_{i-1}).
\end{aligned} \tag{1.98}$$

Phương trình (1.98) chỉ khác Phương trình (1.93) ở điều kiện bổ sung (tức $\mathbf{s}$) được đưa vào các xác suất. Theo nghĩa này, ta có thể dùng Phương trình (1.94–1.95) để mô hình hóa $\Pr(t_i|\mathbf{s}, t_1, \cdots, t_{i-1})$. Tuy nhiên, việc đặt điều kiện trên $\mathbf{s}$ làm bài toán phức tạp hơn, vì ta phải mô hình hóa mối quan hệ giữa hai chuỗi $\mathbf{s}$ và t_i. Một cách tiếp cận trực tiếp cho sinh chuỗi là phát biểu $\Pr(t_i|\mathbf{s}, t_1, \cdots, t_{i-1})$ theo mô thức **bộ mã hóa–bộ giải mã** (*encoder-decoder*). Có hai bước: trước tiên, một bộ mã hóa được dùng để biểu diễn $\mathbf{s}$ dưới một dạng trung gian nào đó (ví dụ một vectơ), sau đó một bộ giải mã được dùng để mô hình hóa cả các từ phía đích lẫn mối tương quan giữa đầu ra của bộ mã hóa và các từ phía đích. Gộp lại, đầu ra của mô hình bộ mã hóa–bộ giải mã có thể được định nghĩa là:

$$\mathbf{y}_i \;=\; \text{Decode}(\text{Encode}(\mathbf{s}), t_1, \cdots, t_{i-1}), \tag{1.99}$$

trong đó $\text{Encode}(\cdot)$ là bộ mã hóa và $\text{Decode}(\cdot)$ là bộ giải mã. $\mathbf{y}_i$ là một phân phối xác suất trên bộ từ vựng phía đích tại vị trí i, tức là $y_i(t_i) = \Pr(t_i|\mathbf{s}, t_1, \cdots, t_{i-1})$. Chương 5 sẽ mô tả chi tiết mô hình bộ mã hóa–bộ giải mã.

Chú giải của người dịch: Bộ mã hóa–bộ giải mã qua ví dụ dịch máy

Dịch câu nguồn $\mathbf{s}$ = "I like fruit" sang $\mathbf{t}$ = "tôi thích trái cây". Bộ mã hóa "đọc hiểu" toàn bộ câu tiếng Anh và tóm nó thành một biểu diễn trung gian. Bộ giải mã sinh từng từ tiếng Việt: lúc đầu đoán "tôi" dựa trên $\text{Encode}(\mathbf{s})$; sau đó đoán "thích" dựa trên $\text{Encode}(\mathbf{s})$ và "tôi"; rồi cứ thế tiếp tục. So với mô hình ngôn ngữ ở Phương trình (1.93), điểm khác duy nhất là mọi dự đoán đều được "nhìn" thêm câu nguồn $\mathbf{s}$.

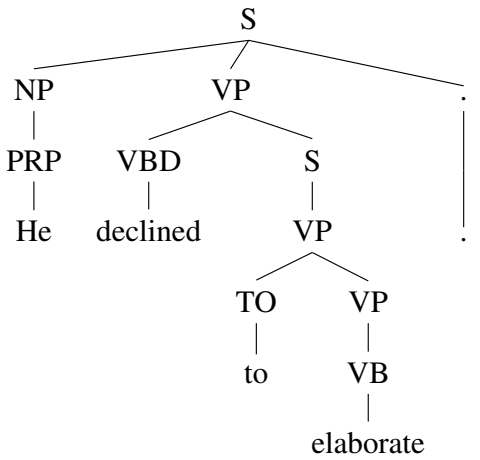
1.5.5 Sinh cây

Trong NLP, cây thường được dùng để biểu diễn cấu trúc hoặc ý nghĩa của dữ liệu tuần tự. Ví dụ, một **bộ phân tích cú pháp** (*syntactic parser*) phân tích một câu để tạo thành một **cây cú pháp** (*syntax tree*) hay **cây phân tích cú pháp** (*parse tree*). Một cách hình thức hơn, cho một chuỗi từ $\mathbf{s} = s_1 \cdots s_m$, bài toán phân tích cú pháp có thể được định nghĩa là:

$$\hat{d} \;=\; \arg\max_{d \in \mathcal{D}} \Pr(d|\mathbf{s}), \tag{1.100}$$

trong đó d là một cây phân tích cú pháp, và $\mathcal{D}$ là tập tất cả các cây phân tích cú pháp sinh ra $s_1 \cdots s_m$. Tính $\Pr(d|\mathbf{s})$ là một thách thức, vì độ phức tạp của việc mô hình hóa tăng theo hàm mũ khi chuyển từ chuỗi sang cây. Trong **phân tích cú pháp thống kê** (*statistical parsing*), một giải pháp là mô hình hóa d như một dẫn xuất (*derivation*) của các quy tắc cú pháp. Bằng cách này, $\Pr(d|\mathbf{s})$ có thể được phát biểu là tích các xác suất của các quy tắc. Hình 1.10 trình bày một ví dụ phân tích cú pháp với các quy tắc của văn phạm phi ngữ cảnh (*context-free grammar*) (CFG). Ngoài ra, $\Pr(d|\mathbf{s})$ cũng có thể được mô hình hóa theo kiểu đầu-cuối (*end-to-end*). Ví dụ, một số cách tiếp cận gần đây thực hiện phân tích cú pháp bằng cách định nghĩa một mạng nơ-ron trên cây phân tích cú pháp. Xác suất của một cây con có gốc tại một nút được tính bằng cách xét tương tác giữa nút này và các nút con của nó.

Parse Tree:



CFG Rules:

r_1 : PRP → He
r_2 : VBD → declined
r_3 : TO → to
r_4 : VB → elaborate
r_5 : . → .
r_6 : NP → PRP
r_7 : VP → VB
r_8 : VP → TO VP
r_9 : S → VP
r_{10} : VP → VBD S
r_{11} : S → NP VP .

$$\begin{aligned}\mathrm{P}(d|\mathbf{s}) &= \mathrm{P}(\mathrm{PRP} \rightarrow \mathrm{He}) \cdot \mathrm{P}(\mathrm{VBD} \rightarrow \mathrm{declined}) \cdot \mathrm{P}(\mathrm{TO} \rightarrow \mathrm{to}) \cdot \mathrm{P}(\mathrm{VB} \rightarrow \mathrm{elaborate}) \cdot \\ &\quad \mathrm{P}(. \rightarrow .) \cdot \mathrm{P}(\mathrm{NP} \rightarrow \mathrm{PRP}) \cdot \mathrm{P}(\mathrm{VP} \rightarrow \mathrm{VB}) \cdot \mathrm{P}(\mathrm{VP} \rightarrow \mathrm{TO\ VP}) \cdot \mathrm{P}(\mathrm{S} \rightarrow \mathrm{VP}) \cdot \\ &\quad \mathrm{P}(\mathrm{VP} \rightarrow \mathrm{VBDS}) \cdot \mathrm{P}(\mathrm{S} \rightarrow \mathrm{NP\ VP\ .}) \\ &= \textstyle\prod_{i=0}^{11} \mathrm{P}(r_i)\end{aligned}$$

Hình 1.10: Một ví dụ về cây phân tích cú pháp và các quy tắc CFG. Câu ví dụ được lấy từ dữ liệu huấn luyện của nhiệm vụ chung CoNLL 2000. Cây phân tích cú pháp được biểu diễn như một dẫn xuất từ các quy tắc CFG. Xác suất của cây phân tích cú pháp được định nghĩa là tích các xác suất của các quy tắc. (Câu ví dụ: "He declined to elaborate." – "Ông ấy từ chối nói rõ thêm." *Parse Tree* = cây phân tích cú pháp, *CFG Rules* = các quy tắc CFG; S = câu, NP = cụm danh từ, VP = cụm động từ, PRP = đại từ nhân xưng, VBD = động từ quá khứ, TO = từ "to", VB = động từ nguyên thể.)

Chú giải của người dịch: Đọc quy tắc văn phạm

Một quy tắc như "S → NP VP ." đọc là: "một câu (S) có thể được tạo thành từ một cụm danh từ, theo sau là một cụm động từ và dấu chấm". Quy tắc "PRP → He" nói rằng đại từ PRP có thể là từ "He". Bắt đầu từ S và lần lượt áp dụng 11 quy tắc $r_1, \ldots, r_{11}$, ta "mọc" ra đúng cây trong Hình 1.10. Nếu mỗi quy tắc có một xác suất (ví dụ $\mathrm{P}(\mathrm{S} \rightarrow \mathrm{NP\ VP\ .}) = 0.6$), xác suất của cả cây là tích các xác suất đó — tương tự cách mô hình ngôn ngữ nhân các xác suất từng từ ở Phương trình (1.93).

Một cách khác là coi phân tích cú pháp như bài toán sinh chuỗi. Ví dụ, ta có thể tuyến tính hóa

(*linearize*) một cây phân tích cú pháp và biểu diễn nó thành một chuỗi gồm các từ và nhãn cú pháp, hoặc biến quá trình sinh cây thành một chuỗi các hành động. Điều này cho phép dùng các kỹ thuật chuỗi-sang-chuỗi để giải quyết bài toán chuỗi-sang-cây.

Trong ngôn ngữ học và NLP, cấu trúc cây được dùng rất nhiều cho phân tích cú pháp. Ngoài phân tích câu, chúng còn được dùng để biểu diễn cấu trúc phân cấp bên trong của từ (hình thái học (*morphology*)), của cụm từ và của toàn bộ diễn ngôn (*discourse*). Tuy nhiên, cây không phải là cách duy nhất để biểu diễn các quan hệ phi tuyến. Một cấu trúc dữ liệu tổng quát hơn là đồ thị (*graph*). Dù cây có thể được xem là một lớp đặc biệt của đồ thị, có những hiện tượng ngôn ngữ mà cây không thể nắm bắt đầy đủ [Fellbaum, 2005; Singhal, 2005; Banarescu và cộng sự, 2013]. Ví dụ, trong biểu diễn ngữ nghĩa của một câu, ta thường cần một đồ thị để nối các vị từ (*predicates*) với các tham tố (*arguments*) của chúng, cho phép một nút có nhiều nút cha. Dù việc sinh đồ thị tổng quát đòi hỏi nhiều tính toán hơn so với phân tích một câu thành cây, ta vẫn có thể điều chỉnh thành công nhiều phương pháp tinh vi vốn được phát triển cho sinh chuỗi và sinh cây.

1.5.6 Mô hình hóa mức độ liên quan

Nói chung, mức độ liên quan (*relevance*) chỉ mức độ mà một thực thể có liên hệ với một thực thể khác. Khái niệm mức độ liên quan được dùng trong nhiều lĩnh vực con khác nhau của NLP và khoa học thông tin. Ví dụ, trong truy xuất thông tin (*information retrieval*), mức độ liên quan được dùng để mô tả một tài liệu được truy xuất đáp ứng truy vấn đến mức nào. Khái niệm này còn là nền tảng của hỏi đáp, hệ thống hội thoại, phân tích ngữ nghĩa và nhiều nhiệm vụ khác cần đến một cơ chế đối sánh hoặc truy xuất.

Hãy xét một mô tả tổng quát hơn. Giả sử ta có một query (truy vấn (*query*)) và một key (khóa (*key*)) biểu diễn thứ mà ta muốn đối sánh với truy vấn. Khi đó, ta định nghĩa vectơ đặc trưng của truy vấn và khóa là

$$\mathbf{q} = Q(\text{query}) \tag{1.101}$$

$$\mathbf{k} = K(\text{key}), \tag{1.102}$$

trong đó $Q(\cdot)$ và $K(\cdot)$ là các bộ trích xuất đặc trưng (*feature extractors*). Mức độ liên quan giữa truy vấn và khóa sau đó được tính bằng một hàm tính điểm (*scoring function*):

$$r \;=\; f(\mathbf{q}, \mathbf{k}). \tag{1.103}$$

Hàm $f(\cdot)$ có thể là một độ đo khoảng cách đơn giản (nếu $\mathbf{q}$ và $\mathbf{k}$ nằm trong cùng một không gian vectơ) cho đến một mô hình có tham số phức tạp thực hiện các phép biến đổi phi tuyến. Thực ra, cách định nghĩa mức độ liên quan này có thể được dùng trong nhiều tình huống khác nhau. Đôi khi mức độ liên quan còn được gọi là độ tương đồng (*similarity*) hay độ tương quan (*correlation*). Gọi x và y là hai mẫu (ví dụ hai từ). Độ tương đồng của x và y được cho bởi

$$r \;=\; f(g(x), g(y)), \tag{1.104}$$

trong đó $g(\cdot)$ là một bộ trích xuất đặc trưng, và $f(\cdot)$ là một **hàm tương đồng** (*similarity function*). Việc học cả $g(\cdot)$ lẫn $f(\cdot)$ được gọi là **học độ tương đồng** (*similarity learning*). Trong một cách thiết lập học độ tương đồng, ta cố định $f(\cdot)$ và học $g(\cdot)$ sao cho các mẫu tương đồng cho ra đầu ra $g(\cdot)$ tương đồng. Hơn nữa, việc huấn luyện bộ trích xuất đặc trưng không nhất thiết phải gắn liền với hàm tương đồng ở hạ nguồn. Ví dụ, để có được độ tương đồng giữa các từ, ta có thể tiền huấn luyện $g(\cdot)$ bằng một mục tiêu

mô hình hóa ngôn ngữ rồi sau đó dùng nó cùng với nhiều hàm tương đồng có sẵn khác nhau (như độ tương đồng cosin (*cosine similarity*)). Điều này phản ánh một triết lý rộng hơn trong học máy hiện đại: tách rời học biểu diễn (*representation learning*) khỏi nhiệm vụ hạ nguồn cụ thể. Ý tưởng này được áp dụng rộng rãi trong tiền huấn luyện, góp phần đẩy các kết quả tiên tiến nhất gần đây trên nhiều nhiệm vụ NLP.

Trong một cách thiết lập khác của học độ tương đồng, ta có thể học trực tiếp $f(\cdot)$. Việc này có thể được thực hiện bằng cách học đồng thời $f(\cdot)$ và $g(\cdot)$, hoặc học $f(\cdot)$ dựa trên một $g(\cdot)$ cố định. Bài toán này cũng liên quan đến **học độ đo** (*metric learning*). Thông thường, học độ đo được phát biểu như một bài toán có giám sát [Kulis, 2013]. Một hàm tương đồng mong muốn có thể được học với sự giám sát dựa trên một độ tương đồng chuẩn nào đó. Tuy nhiên, trong thực tế NLP thường không có thông tin giám sát như vậy. Khi đó, ta có thể dùng *khoảng cách tương đối* làm tín hiệu giám sát thay thế. Ví dụ, hàm tương đồng có thể được học bằng cách tối ưu một hàm mất mát tương phản (*contrastive loss*) (xem Mục 1.3.4).

Đo độ tương đồng giữa các đối tượng đóng vai trò quan trọng trong nhiều phương pháp học máy, như phân cụm (*clustering*) và phân loại láng giềng gần nhất (*nearest neighbor classification*). Trong NLP, nó hữu ích để khám phá quan hệ giữa các từ, cụm từ, câu và tài liệu; chẳng hạn, độ tương đồng là một cách để kiểm tra xem các vectơ từ tương ứng thế nào với hiểu biết của chúng ta về nghĩa của từ [Mikolov và cộng sự, 2013c; Pennington và cộng sự, 2014].

Chú giải của người dịch: Độ tương đồng cosin

Độ tương đồng cosin của hai vectơ chính là côsin của góc giữa chúng, như ở lớp 10: $\cos(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q} \cdot \mathbf{k}}{\|\mathbf{q}\| \, \|\mathbf{k}\|}$. Ví dụ $\mathbf{q} = (1; 2)$, $\mathbf{k} = (2; 4)$ cùng hướng nên cosin bằng $\frac{1 \cdot 2 + 2 \cdot 4}{\sqrt{5} \cdot \sqrt{20}} = \frac{10}{10} = 1$ (giống nhau nhất); hai vectơ vuông góc cho cosin bằng 0. Nếu $g(\cdot)$ biến các từ "mèo" và "chó" thành hai vectơ gần cùng hướng, còn "mèo" và "ô tô" thành hai vectơ gần vuông góc, ta nói $g(\cdot)$ đã nắm được nghĩa của từ. Ý tưởng truy vấn–khóa ở đây sẽ gặp lại trong cơ chế chú ý của Transformer.

1.5.7 Gióng hàng ngôn ngữ

Gióng hàng ngôn ngữ (*linguistic alignment*) là một nhóm bài toán trong đó ta thiết lập một sự tương ứng nào đó giữa hai tập đơn vị ngôn ngữ. Trong NLP, các bài toán chuỗi-sang-chuỗi và chuỗi-sang-cây về bản chất là các bài toán gióng hàng ngôn ngữ, vì cả hai đều kết nối hai đơn vị ngôn ngữ. Tuy nhiên, theo quy ước, thuật ngữ *gióng hàng* (*alignment*) dùng để chỉ việc gióng hàng nhiều đối tượng cùng một lúc[21].

Ví dụ, hãy xét nhiệm vụ nổi tiếng **gióng hàng từ** (*word alignment*): ta gióng các từ của một câu với các từ của một câu khác. Ta dùng lại ký hiệu ở Mục 1.5.4 vì cả nhiệm vụ chuỗi-sang-chuỗi lẫn gióng hàng từ đều làm việc trên một cặp chuỗi. Cho chuỗi từ phía nguồn $\mathbf{s} = s_1 \cdots s_m$ và chuỗi từ phía đích $\mathbf{t} = t_1 \cdots t_n$, gióng hàng từ giữa hai chuỗi được ký hiệu là một ma trận $\mathbf{A}$ kích thước $m \times n$. $A(i, j) = 1$ nếu có một **liên kết gióng hàng** (*alignment link*) giữa s_i và t_j, và $A(i, j) = 0$ trong trường hợp ngược lại. Phép gióng hàng tối ưu có thể được định nghĩa là:

$$\hat{\mathbf{A}} \;=\; \arg\max_{\mathbf{A}} \Pr(\mathbf{A}|\mathbf{s}, \mathbf{t}), \tag{1.105}$$

trong đó $\Pr(\mathbf{A}|\mathbf{s}, \mathbf{t})$ là xác suất gióng hàng từ. Giống như các bài toán học máy khác, ta có thể mô hình hóa $\Pr(\mathbf{A}|\mathbf{s}, \mathbf{t})$ theo kiểu sinh hoặc kiểu phân biệt (xem Mục 1.2.4). Chẳng hạn, trong công trình của

[21] Khái niệm *alignment* rất rộng. Ở đây chúng tôi dùng thuật ngữ *gióng hàng ngôn ngữ* (*linguistic alignment*) để phân biệt với *căn chỉnh* (*alignment*) của các mô hình ngôn ngữ lớn được bàn ở các chương sau.

Brown và cộng sự [1993], mô hình gióng hàng từ được phân tích thành nhiều bước sinh, mỗi bước ứng với một số giả định về bài toán[22].

Một ma trận gióng hàng 0-1 thể hiện một sơ đồ gióng hàng cứng (*hard alignment*). Vấn đề ở đây là mô hình cứng có thể không mô tả tốt những phép gióng hàng từ có tính nhập nhằng cao. Vì vậy, ta có thể biểu diễn $\mathbf{A}$ dưới dạng một ma trận số thực (gọi là **ma trận gióng hàng từ mềm** (*soft word alignment matrix*) hay **ma trận trọng số gióng hàng từ** (*word alignment weight matrix*)). Giả sử các từ phía nguồn được biểu diễn thành một chuỗi các vectơ đặc trưng $\mathbf{x} = \begin{bmatrix}\mathbf{x}_1 & \cdots & \mathbf{x}_m\end{bmatrix}$. Tương tự, các từ phía đích được biểu diễn là $\mathbf{y} = \begin{bmatrix}\mathbf{y}_1 & \cdots & \mathbf{y}_n\end{bmatrix}$. Một mô hình gióng hàng từ mềm được cho bởi:

$$\mathbf{A} \;=\; a(\mathbf{s}, \mathbf{t}), \tag{1.106}$$

trong đó $a(\cdot)$ là một hàm gióng hàng từ tính trọng số gióng hàng $A(i, j)$ cho mỗi cặp $\mathbf{x}_i$ và $\mathbf{y}_j$. Thực ra, mọi phương pháp bàn ở Mục 1.5.6 đều có thể áp dụng để thiết kế $a(\cdot)$. Điều này gắn việc mô hình hóa gióng hàng từ với việc mô hình hóa độ tương đồng, và cho phép giải quyết các bài toán NLP khác nhau bằng cùng một cách tiếp cận học máy.

Chú giải của người dịch: Ma trận gióng hàng: ví dụ

Gióng câu "I like fruit" ($m = 3$) với "tôi thích trái cây" ($n = 4$). Gióng hàng cứng là ma trận 3×4 gồm 0 và 1:

$$\mathbf{A} = \begin{bmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 1 & 1\end{bmatrix} \quad \text{(hàng: I, like, fruit; cột: tôi, thích, trái, cây)},$$

tức "fruit" ứng với cả "trái" và "cây". Ở gióng hàng mềm, các số 0/1 được thay bằng trọng số, chẳng hạn hàng "fruit" có thể là $[0.05 \;\; 0.05 \;\; 0.5 \;\; 0.4]$, diễn tả mức độ "chắc chắn" của từng liên kết — giống các ô vuông to nhỏ trong Hình 1.11(g).

Phương trình (1.106) cung cấp một cách rất tổng quát để khám phá mối liên hệ tiềm ẩn giữa các cặp biến. Ngoài việc gióng hàng các từ trong chuỗi, nó còn hữu ích để gióng hàng các đối tượng không có thứ tự. Ví dụ, trong quy nạp từ điển song ngữ (*bilingual dictionary induction*), ta có thể học một ma trận trọng số như vậy để ước lượng một từ trong ngôn ngữ này tương ứng mạnh đến đâu với một từ trong ngôn ngữ kia.

Lưu ý cuối cùng liên quan đến vai trò của các mô hình gióng hàng ngôn ngữ. Dù gióng hàng ngôn ngữ có thể được xem là một nhiệm vụ NLP độc lập, nó thường được dùng để thiết kế các mô hình con của một số hệ thống hạ nguồn. Nhiều hệ thống mô hình hóa quan hệ ở mức từ có chứa biểu diễn ngầm của gióng hàng ngôn ngữ. Do đó, gióng hàng ngôn ngữ được coi như một số trạng thái ẩn (*latent states*), và là sản phẩm phụ của các hệ thống này. Ví dụ, trong những ngày đầu của dịch máy thống kê (*statistical machine translation*), gióng hàng từ là một biến ẩn được dùng để mô hình hóa ánh xạ giữa các chuỗi. Kết quả gióng hàng từ có thể dễ dàng được suy ra từ một mô hình dịch máy. Gần đây hơn, các mô hình chuỗi-sang-chuỗi nơ-ron — tiêu biểu nhất là các mô hình có cơ chế chú ý (*attention*) [Bahdanau và cộng sự, 2014] — đã cố gắng làm điều tương tự như gióng hàng từ bằng cách tính các trọng số chú ý giữa các từ.

[22] Chính xác hơn, Brown và cộng sự [1993] mô hình hóa $\Pr(\mathbf{A}, \mathbf{s}|\mathbf{t})$, một đại lượng thay thế cho $\Pr(\mathbf{A}|\mathbf{s}, \mathbf{t})$, vì $\Pr(\mathbf{A}|\mathbf{s}, \mathbf{t}) = \frac{\Pr(\mathbf{A}, \mathbf{s}|\mathbf{t})}{\Pr(\mathbf{s}|\mathbf{t})} = \frac{\Pr(\mathbf{A}, \mathbf{s}|\mathbf{t})}{\sum_{\mathbf{A}'} \Pr(\mathbf{A}', \mathbf{s}|\mathbf{t})}$.

1.5.8 Trích xuất thông tin

Trong NLP, *trích xuất* không phải là một loại nhiệm vụ mà là một kiểu hành vi mà hệ thống thể hiện. Nói một cách không hình thức, nó chỉ quá trình thu thập và chắt lọc thông tin có cấu trúc từ một số nguồn thông tin. Vì vậy, thuật ngữ trích xuất thường xuất hiện cùng các thuật ngữ khác để tạo thành một nhiệm vụ cụ thể, như **trích xuất từ khóa** (*keyword extraction*), **trích xuất sự kiện** (*event extraction*) và **trích xuất quan hệ** (*relation extraction*). Nhiều nhiệm vụ trong số này có thể được xếp vào một lĩnh vực — **trích xuất thông tin** (*information extraction*). Trích xuất thông tin có lẽ là chủ đề rộng nhất trong NLP. Không có danh sách đầy đủ các nhiệm vụ trích xuất thông tin. Theo cuốn sách của Jurafsky và Martin [2008], nó bao gồm (nhưng không giới hạn ở) nhận dạng thực thể có tên, phân giải đồng tham chiếu (*coreference resolution*), trích xuất quan hệ, trích xuất sự kiện và điền mẫu (*template filling*).

Vì trích xuất thông tin là một tập hợp không đồng nhất của nhiều bài toán đa dạng, nó không thể được phát biểu trong một mô thức học máy duy nhất. May mắn là phần lớn các bài toán này có thể được đưa về các bài toán học máy tiêu chuẩn, như phân loại và gán nhãn chuỗi, và có thể được giải bằng các công cụ có sẵn. Trong một số trường hợp, có thể cần điều chỉnh nhẹ các mô hình hiện có để thích ứng với nhiệm vụ mới. Ví dụ, trích xuất một đoạn cụ thể từ văn bản có thể đòi hỏi hệ thống tạo ra một *đoạn* (span) cho biết vị trí bắt đầu và kết thúc của đoạn được trích xuất (gọi là **dự đoán đoạn** (*span prediction*)).

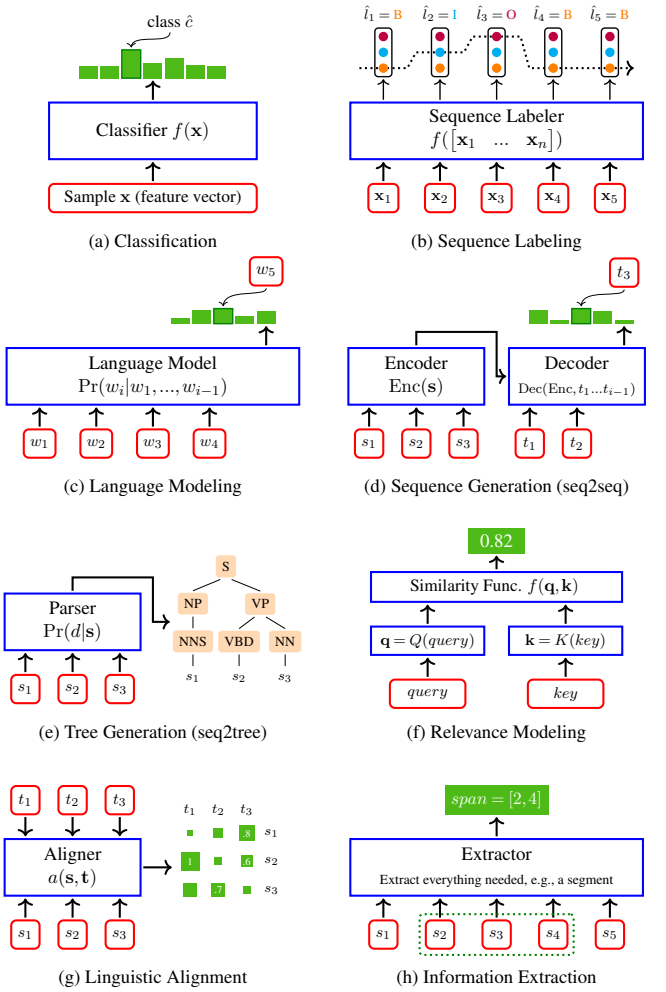
Về mặt thực tiễn, không phải lúc nào cũng cần học máy để trích xuất thông tin từ văn bản. Nhiều bài toán có thể giải bằng các quy tắc viết tay. Một ví dụ là dùng biểu thức chính quy (*regular expressions*) để nhận diện địa điểm và ngày tháng trong văn bản. Trong thực tế, các hệ thống thật thường là sự kết hợp giữa các phương pháp kinh nghiệm (heuristic) và các phương pháp học máy tự động.

Chú giải của người dịch: Dự đoán đoạn và biểu thức chính quy

Với câu "Hội nghị diễn ra tại Đà Nẵng vào ngày 15/3/2025" (đánh số token từ 1), nếu cần trích xuất địa điểm, hệ thống dự đoán đoạn chỉ cần đưa ra hai số: vị trí bắt đầu và kết thúc của "Đà Nẵng", tức $span = [6, 7]$ (tương tự $span = [2, 4]$ trong Hình 1.11(h)). Còn ngày tháng có thể bắt bằng một quy tắc viết tay dạng "một hoặc hai chữ số, dấu /, một hoặc hai chữ số, dấu /, bốn chữ số" — đó chính là ý tưởng của biểu thức chính quy, không cần học máy.

1.5.9 Nhận xét

Hình 1.11 minh họa các nhiệm vụ NLP nói trên. Lưu ý rằng ta mới chỉ tập trung vào các bài toán NLP trong mô thức học có giám sát (*supervised learning*). Nhiều nhiệm vụ không giám sát cũng quan trọng đối với nghiên cứu NLP. Ví dụ, việc phân cụm các từ hoặc tài liệu chưa gán nhãn để thuận tiện cho xử lý trong các hệ thống hạ nguồn là điều phổ biến. Có một số phương pháp có thể áp dụng trực tiếp cho nhiệm vụ này [Murphy, 2012]. Một xu hướng gần đây trong NLP là không cần đặt ranh giới chặt chẽ giữa việc dùng học có giám sát và học không giám sát (*unsupervised learning*). Trong nhiều trường hợp, các phương pháp không giám sát giúp ích cho các nhiệm vụ có giám sát, và ngược lại. Một ví dụ đáng chú ý là ta học một bộ trích xuất đặc trưng tiền huấn luyện trên dữ liệu chưa gán nhãn rồi xây dựng một bộ phân loại có giám sát bên trên nó. Cách tiếp cận này được biết đến rộng rãi với tên gọi **học tự giám sát** (*self-supervised learning*): nó vẫn tuân theo khuôn khổ học có giám sát, nhưng tín hiệu giám sát được rút ra từ chính dữ liệu chưa gán nhãn. Điều này đã dẫn tới một sự chuyển dịch mô thức hướng tới việc phát triển các mô hình biểu diễn (tức các bộ trích xuất đặc trưng) một cách độc lập, không phụ thuộc vào các nhiệm vụ có giám sát cụ thể ở hạ nguồn.



(a) Classification (b) Sequence Labeling

(c) Language Modeling (d) Sequence Generation (seq2seq)

(e) Tree Generation (seq2tree) (f) Relevance Modeling

(g) Linguistic Alignment (h) Information Extraction

Hình 1.11: Các nhiệm vụ xử lý ngôn ngữ tự nhiên dưới góc nhìn học máy. (Tên các hình con: (a) phân loại; (b) gán nhãn chuỗi; (c) mô hình hóa ngôn ngữ; (d) sinh chuỗi (chuỗi-sang-chuỗi); (e) sinh cây (chuỗi-sang-cây); (f) mô hình hóa mức độ liên quan; (g) gióng hàng ngôn ngữ; (h) trích xuất thông tin. Nhãn khác: *Classifier* = bộ phân loại, *Sample (feature vector)* = mẫu (vectơ đặc trưng), *Sequence Labeler* = bộ gán nhãn chuỗi, *Language Model* = mô hình ngôn ngữ, *Encoder/Decoder* = bộ mã hóa/bộ giải mã, *Parser* = bộ phân tích cú pháp, *Similarity Func.* = hàm tương đồng, *Aligner* = bộ gióng hàng, *Extractor* = bộ trích xuất, *Extract everything needed, e.g., a segment* = trích xuất mọi thứ cần thiết, chẳng hạn một đoạn văn bản.)

Chú giải của người dịch: Học tự giám sát

Ở mô hình ngôn ngữ (Mục 1.5.3), "nhãn" cần dự đoán chính là từ tiếp theo, vốn có sẵn trong văn bản; vì vậy ta có thể tạo ra hàng tỉ "bài tập có đáp án" từ văn bản thô mà không cần ai gán nhãn. Đây là ý nghĩa của "tín hiệu giám sát được rút ra từ chính dữ liệu". Mô hình học được theo cách này sau đó làm bộ trích xuất đặc trưng tốt cho nhiều nhiệm vụ khác — nền tảng của các mô hình như BERT và GPT ở các chương sau.

1.6 Tóm tắt

Chương này đã trình bày các ý tưởng cơ bản của học máy và ứng dụng của nó cho các bài toán NLP. Cụ thể, chúng tôi đã trình bày một bài toán phân loại văn bản đơn giản để bắt đầu làm quen với học máy. Chúng tôi cũng đã bàn về một số vấn đề chung của học máy, như các loại phương pháp học máy, thiên kiến quy nạp, hàm mất mát, quá khớp, v.v. Tiếp theo là phần thảo luận về lựa chọn và đánh giá mô hình. Ngoài ra, chúng tôi đã mô tả cách các bài toán NLP hiện đại được phát biểu thành các bài toán học máy.

Tuy nhiên, học máy là một lĩnh vực nghiên cứu khổng lồ. Một số chủ đề thú vị nằm ngoài phạm vi chương này. Một chủ đề mà chúng tôi hầu như chưa nói đến là học tăng cường (*reinforcement learning*). Nói chung, học tăng cường rất mạnh mẽ. Nó nên được, và đã được, xem là một cách tiếp cận để giải quyết các bài toán NLP, chẳng hạn huấn luyện một mô hình chuỗi-sang-chuỗi bằng một hàm mất mát dựa trên rủi ro (*risk*). Một giáo trình về học tăng cường sẽ cung cấp các ý tưởng chung của học tăng cường [Sutton và Barto, 2018]. Một chủ đề khác chúng tôi bỏ qua ở đây là học Bayes (*Bayesian learning*) [Gelman và cộng sự, 2020; McElreath, 2020; Downey, 2021]. Nó mở ra một hướng nghiên cứu đáng chú ý trong học thống kê và đã được dùng thành công trong các nhiệm vụ NLP. Hơn nữa, còn có nhiều chủ đề khác chuyên sâu về một số khía cạnh nhất định của học máy và được các nhà nghiên cứu, kỹ sư NLP quan tâm. Một vài trong số đó là học máy hiệu quả (*efficient machine learning*) [Tay và cộng sự, 2020b], học đa nhiệm (*multi-task learning*) [Zhang và Yang, 2021], học few-shot/zero-shot [Wang và cộng sự, 2019b; 2020c], v.v.

Một điểm cuối cùng để khép lại chương này. Chúng tôi bỏ qua phần thảo luận chi tiết về một số mô hình và thuật toán học máy, như các mô hình phân loại và hồi quy (*regression*), vì bạn đọc quan tâm có thể tìm thấy nhiều tài liệu nhập môn xuất sắc và đầy đủ [Bishop, 2006; Hastie và cộng sự, 2009; Murphy, 2012; Mohri và cộng sự, 2018]. Trong chương tiếp theo, chúng tôi sẽ bàn thêm một chút về mạng nơ-ron nhân tạo (*artificial neural networks*), nền tảng của học sâu và của các mô hình NLP tiên tiến nhất hiện nay.

Chương 2

Nền tảng của mạng nơ-ron

Mạng nơ-ron nhân tạo (*artificial neural networks*) (hay gọi tắt là **mạng nơ-ron** — *neural networks*, *neural nets*) là những công cụ học máy mạnh mẽ đã đẩy trình độ tiên tiến nhất (*state-of-the-art*) của NLP tiến lên trong những năm gần đây. Mặc dù lịch sử của mạng nơ-ron có thể truy ngược về thập niên 1940 [McCulloch và Pitts, 1943], trong một thời gian dài chúng không vượt trội một cách ổn định so với các phương pháp học máy khác. Điều này thay đổi vào khoảng năm 2006, khi những ý tưởng mới được phát triển để huấn luyện các **mạng nơ-ron sâu** (*deep neural networks*) [Hinton và cộng sự, 2006; Hinton, 2007]. Từ đó, các phương pháp này được biết đến với tên gọi **học sâu** (*deep learning*). Đến nay, học sâu chắc chắn đã trở thành một trong những lĩnh vực sôi động và có ảnh hưởng nhất của trí tuệ nhân tạo (*artificial intelligence, AI*); nó được hưởng lợi không chỉ từ các kiến trúc mô hình "sâu" mà còn từ vô số kỹ thuật giúp huấn luyện và triển khai các mô hình như vậy.

Trong chương này, chúng tôi trình bày các khái niệm nền tảng của mạng nơ-ron và học sâu. Dù không tập trung vào những nghiên cứu mới nhất, chương này bao quát nhiều khái niệm và kỹ thuật quan trọng được dùng rộng rãi khi xây dựng các hệ thống nơ-ron. Trong đó có các kiến trúc mô hình cơ bản, các phương pháp huấn luyện và chính quy hóa (*regularization*), các kỹ thuật học không giám sát (*unsupervised learning*) và bộ tự mã hóa (*auto-encoder*). Chúng tôi cũng trình bày một ví dụ áp dụng mạng nơ-ron vào bài toán mô hình hóa ngôn ngữ (*language modeling*).

Chú giải của người dịch: "Nơ-ron nhân tạo" và "sâu" nghĩa là gì?

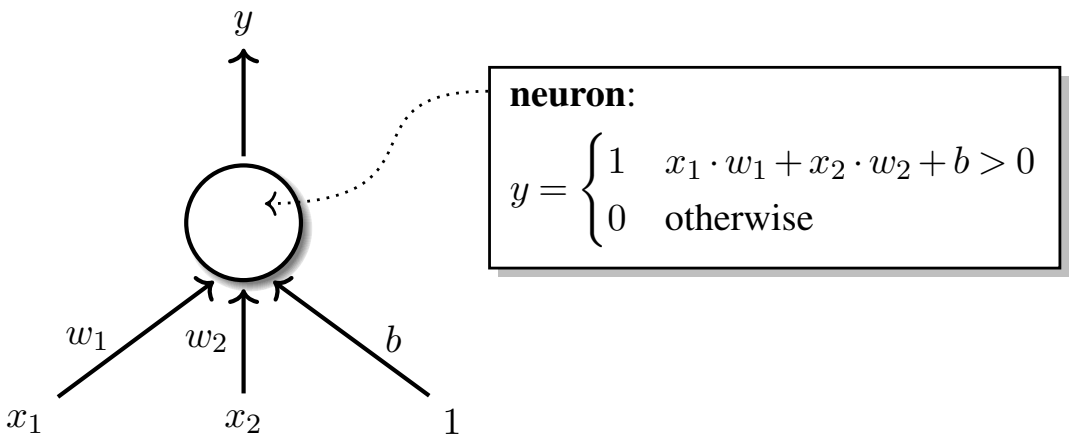
Tên gọi "mạng nơ-ron" bắt nguồn từ việc mô phỏng (rất thô sơ) các tế bào thần kinh trong não. Nhưng về bản chất, một mạng nơ-ron chỉ là một *hàm số* rất lớn: nhận vào một dãy số, thực hiện nhiều phép nhân, cộng và áp các hàm đơn giản, rồi trả ra một dãy số khác. Mạng được gọi là "sâu" khi nó gồm nhiều *tầng* xử lý chồng lên nhau (đầu ra của tầng này là đầu vào của tầng kế tiếp), giống như một dây chuyền sản xuất có nhiều công đoạn nối tiếp.

2.1 Mạng nơ-ron nhiều tầng

Để bắt đầu, chúng tôi giới thiệu ngắn gọn về **perceptron một tầng** (*single-layer perceptrons*), rồi mở rộng sang trường hợp tổng quát hơn, trong đó nhiều tầng được xếp chồng lên nhau để tạo thành một mạng phức tạp hơn.

2.1.1 Perceptron một tầng

Perceptron một tầng (gọi tắt là **perceptron**) có lẽ là những mạng nơ-ron đơn giản nhất đã được phát triển để sử dụng trong thực tế [Rosenblatt, 1957; Minsky và Papert, 1969]. Chúng thường được xem là các mô hình lấy cảm hứng từ sinh học, biến đổi một đầu vào nào đó thành một đầu ra nào đó. Một perceptron gồm một số **nơ-ron** (*neurons*) kết nối các biến đầu vào với các biến đầu ra. Hình 2.1 minh họa một perceptron chỉ có một nơ-ron. Trong ví dụ này, có hai biến nhận giá trị thực x_1 và x_2 làm đầu vào và một biến nhị phân (*binary variable*) y làm đầu ra. Nơ-ron đọc các biến đầu vào và quyết định chọn giá trị đầu ra nào. Quá trình này tương tự như ở một nơ-ron sinh học — nó nhận các tín hiệu điện hóa từ các nơ-ron khác và quyết định có truyền tiếp tín hiệu điện hóa đó hay không.



Hình 2.1: Minh họa một perceptron với hai biến đầu vào $\{x_1, x_2\}$ và một biến đầu ra y. Có hai trọng số $\{w_1, w_2\}$, mỗi trọng số tương ứng với một biến đầu vào. Đầu ra phụ thuộc vào tổng có trọng số của các biến đầu vào cộng với số hạng độ lệch b, chẳng hạn $y = 1$ nếu $x_1 \cdot w_1 + x_2 \cdot w_2 + b > 0$, và $y = 0$ trong trường hợp ngược lại.

(Trong hình: *neuron* = nơ-ron; *otherwise* = trường hợp còn lại.)

Về mặt toán học, một perceptron có thể được mô tả như một hàm ánh xạ. Gọi **x** là vectơ các biến đầu vào (tức là một **vectơ đặc trưng** (*feature vector*)). Một **phép biến đổi affine** (*affine transformation*) của **x** được cho bởi[1]:

$$\begin{aligned} f(\mathbf{x}) &= \mathbf{x} \cdot \mathbf{w} + b \\ &= \sum_i x_i \cdot w_i + b, \end{aligned} \tag{2.1}$$

trong đó **w** là vectơ trọng số (*weight*) và b là số hạng độ lệch (bias) (*bias*). Khi đó, một perceptron chuẩn có thể được định nghĩa là:

$$\begin{aligned} y &= \psi(f(\mathbf{x})) \\ &= \begin{cases} 1 & f(\mathbf{x}) > 0 \\ 0 & \text{trường hợp còn lại} \end{cases}, \end{aligned} \tag{2.2}$$

[1] Trong toán học, một **phép biến đổi tuyến tính** (*linear transformation*) ánh xạ mỗi vectơ **v** trong một không gian thành $f(\mathbf{v})$ trong một không gian khác, sao cho với mọi vectơ **x**, **y** và mọi số vô hướng α, β, ta có $f(\alpha\mathbf{x} + \beta\mathbf{y}) = \alpha f(\mathbf{x}) + \beta f(\mathbf{y})$. Phép biến đổi affine là một phép biến đổi tuyến tính theo sau bởi một phép tịnh tiến, thường được viết dưới dạng $f(\mathbf{x}) + \mathbf{b}$.

trong đó $\psi(\cdot)$ là một **hàm bậc thang** (*step function*) nhị phân. Hàm $\psi(\cdot)$ còn được gọi là hàm kích hoạt (*activation function*). Điều này liên hệ perceptron với các mô hình phân loại đã bàn ở Chương 1. Nói cách khác, bản thân Phương trình (2.2) đã là một bộ phân loại (*classifier*): $f(\mathbf{x})$ là một hàm phân biệt (*discriminant function*) xác định trên mỗi đầu vào $\mathbf{x}$, theo sau là một hàm kích hoạt $\psi(\cdot)$ dùng để tạo ra đầu ra mong muốn[2].

Chú giải của người dịch: Tính tay một perceptron

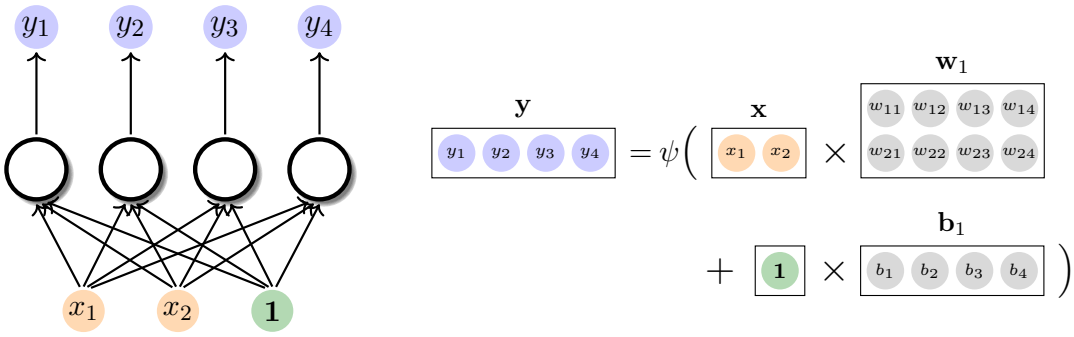
Giả sử $w_1 = 2$, $w_2 = -1$, $b = -1$. Với đầu vào $x_1 = 1$, $x_2 = 0{,}5$: $f(\mathbf{x}) = 1 \cdot 2 + 0{,}5 \cdot (-1) + (-1) = 0{,}5 > 0$, nên $y = 1$. Với $x_1 = 0$, $x_2 = 1$: $f(\mathbf{x}) = 0 - 1 - 1 = -2 \leq 0$, nên $y = 0$. Như vậy perceptron chỉ làm ba việc: nhân mỗi đầu vào với "mức độ quan trọng" của nó (trọng số), cộng lại kèm một hằng số (độ lệch), rồi so sánh với 0 để ra quyết định "có/không". Tập các điểm thỏa $2x_1 - x_2 - 1 = 0$ là một đường thẳng trong mặt phẳng (x_1, x_2); nó chia mặt phẳng thành hai nửa, mỗi nửa ứng với một quyết định — đó chính là "biên quyết định tuyến tính" nhắc tới trong chú thích chân trang. Ký hiệu $\sum_i x_i \cdot w_i$ nghĩa là "cộng tất cả các tích $x_i w_i$ với i chạy qua mọi chỉ số", và $\mathbf{x} \cdot \mathbf{w}$ chính là tích vô hướng quen thuộc ở lớp 10, chỉ là với nhiều tọa độ hơn.

Khi có từ hai nơ-ron trở lên, ta có thể nhóm các nơ-ron này thành một **tầng** (*layer*). Như minh họa trong Hình 2.2, mọi nơ-ron trong một tầng đều nhận tín hiệu từ cùng một vectơ đặc trưng đầu vào nhưng được đánh trọng số theo những cách khác nhau. Đầu ra của tầng là một vectơ đặc trưng mới, mỗi phần tử của nó tương ứng với một nơ-ron. Một cách hình thức hơn, khi xem $\psi(\cdot)$ và $f(\cdot)$ là các hàm vectơ, dạng toán học của perceptron một tầng được cho bởi các phương trình:

$$\mathbf{y} = \psi(f(\mathbf{x})), \tag{2.3}$$

$$f(\mathbf{x}) = \mathbf{x} \cdot \mathbf{w} + \mathbf{b}, \tag{2.4}$$

trong đó $\mathbf{x} \in \mathbb{R}^m$, $\mathbf{y} \in \mathbb{R}^n$, $\mathbf{w} \in \mathbb{R}^{m \times n}$ và $\mathbf{b} \in \mathbb{R}^n$.



Hình 2.2: Minh họa một perceptron một tầng gồm bốn nơ-ron. Tất cả các nơ-ron này đều nhận thông tin từ các biến đầu vào $\{x_1, x_2\}$. Perceptron mô tả một quá trình trong đó 1) trước hết ta biến đổi vectơ các biến đầu vào bằng một phép biến đổi affine $f(\mathbf{x}) = \mathbf{x} \cdot \mathbf{w} + \mathbf{b}$; và 2) tính đầu ra bằng cách đưa $f(\mathbf{x})$ vào hàm kích hoạt $\psi(\cdot)$.

Chú giải của người dịch: Phép nhân vectơ với ma trận trong Hình 2.2

Ở đây $\mathbf{x} = [x_1 \;\; x_2]$ là một hàng gồm $m = 2$ số, còn $\mathbf{w}$ là một bảng số (ma trận) gồm 2 hàng, 4 cột ($\mathbf{w} \in \mathbb{R}^{2 \times 4}$). Tích $\mathbf{x} \cdot \mathbf{w}$ cho ra một hàng gồm 4 số; số thứ j là tích vô hướng của $\mathbf{x}$ với *cột* thứ j của $\mathbf{w}$: $x_1 w_{1j} + x_2 w_{2j}$. Ví

[2] Vì biên quyết định là tuyến tính nên perceptron được xem là một bộ phân loại tuyến tính, mặc dù bản thân hàm bậc thang là phi tuyến.

dụ, với $\mathbf{x} = [1 \;\; 2]$ và $\mathbf{w} = \begin{bmatrix} 1 & 0 & 2 & -1 \\ 0 & 1 & 1 & 3 \end{bmatrix}$, ta có $\mathbf{x} \cdot \mathbf{w} = [1 \;\; 2 \;\; 4 \;\; 5]$. Như vậy, mỗi cột của $\mathbf{w}$ chính là bộ trọng số của *một* nơ-ron, và phép nhân ma trận cho phép tính đồng thời cả 4 nơ-ron. Ký hiệu $\mathbb{R}^{m \times n}$ chỉ tập hợp các ma trận số thực có m hàng, n cột; $\mathbb{R}^n$ là tập các vectơ có n số thực. Trong hình, số 1 màu xanh lá đi kèm $\mathbf{b}_1$ thể hiện "mẹo" coi độ lệch như trọng số của một đầu vào luôn bằng 1.

Một lưu ý nữa về hàm kích hoạt: hàm bậc thang, dù được dùng rộng rãi, không phải là dạng duy nhất của hàm kích hoạt. Có nhiều cách khác nhau để thực hiện phép kích hoạt. Ví dụ, ta có thể dùng hàm Softmax nếu muốn đầu ra có dạng giống một phân phối xác suất (*probability distribution*); dùng hàm Sigmoid nếu muốn đầu ra đơn điệu, liên tục và dễ tối ưu; dùng hàm ReLU nếu muốn đầu ra có dạng "đường dốc". Bảng 2.1 liệt kê một số hàm kích hoạt thường dùng. Lưu ý rằng, mặc dù một tầng nơ-ron được trang bị các hàm kích hoạt này có thể được gọi một cách không chặt chẽ là perceptron một tầng, nó có thể được xếp vào một khái niệm tổng quát hơn, gọi là **mạng nơ-ron một tầng** (*single-layer neural network*). Nếu không nói gì khác, chúng tôi sẽ dùng thuật ngữ *mạng nơ-ron một tầng* trong suốt cuốn sách này.

Tên	Công thức (cho phần tử thứ i của vectơ)
Đồng nhất (*Identity*)	$y_i = s_i$
Bậc thang nhị phân (*Binary Step*)	$y_i = \begin{cases} 1 & s_i > 0 \\ 0 & s_i \leq 0 \end{cases}$
Tang hyperbolic (*Hyperbolic Tangent*)	$y_i = \dfrac{\exp(s_i) - \exp(-s_i)}{\exp(s_i) + \exp(-s_i)}$
Tang cứng (*Hard Tangent*)	$y_i = \begin{cases} 1 & s_i > 1 \\ s_i & -1 \leq s_i \leq 1 \\ -1 & s_i < -1 \end{cases}$
Sigmoid (Logistic)	$y_i = \dfrac{1}{1 + \exp(-s_i)}$
ReLU (*Rectified Linear Unit*)	$y_i = \begin{cases} s_i & s_i > 0 \\ 0 & s_i \leq 0 \end{cases}$
Softplus	$y_i = \ln(1 + \exp(s_i))$
Gauss (*Gaussian*)	$y_i = \exp\left(-\dfrac{1}{2} \cdot \dfrac{(s_i - \mu_i)^2}{\sigma_i^2}\right)$
Softmax	$y_i = \dfrac{\exp(s_i)}{\sum_{i'=1}^{n} \exp(s_{i'})}$
Maxout	$y_i = \max(s_1, \ldots, s_n)$

Bảng 2.1: Các hàm kích hoạt ($\mathbf{y} = \psi(\mathbf{s})$, trong đó $\mathbf{s}, \mathbf{y} \in \mathbb{R}^n$). Tất cả các hàm này đều là hàm vectơ. Chúng tôi trình bày công thức cho phần tử thứ i của vectơ đầu vào và đầu ra. μ_i và σ_i^2 lần lượt là giá trị trung bình và phương sai.

Chú giải của người dịch: Đọc Bảng 2.1 bằng ví dụ số

$\exp(s)$ nghĩa là e^s với $e \approx 2{,}718$. Lấy $\mathbf{s} = [2 \;\; 0 \;\; -1]$:

- ReLU giữ số dương, biến số âm thành 0: $[2 \;\; 0 \;\; 0]$.

- Sigmoid "nén" mọi số vào khoảng $(0;1)$: $\frac{1}{1+e^{-2}} \approx 0{,}88$, $\frac{1}{1+e^{0}} = 0{,}5$, $\frac{1}{1+e^{1}} \approx 0{,}27$.
- Tang hyperbolic (tanh) giống Sigmoid nhưng nén vào khoảng $(-1;1)$: $\approx [0{,}96 \;\; 0 \;\; -0{,}76]$.
- Softmax biến cả vectơ thành các số dương có tổng bằng 1 (giống một phân phối xác suất): $e^{2} \approx 7{,}39$, $e^{0} = 1$, $e^{-1} \approx 0{,}37$, tổng $\approx 8{,}76$, nên kết quả $\approx [0{,}84 \;\; 0{,}11 \;\; 0{,}04]$. Số lớn nhất nhận "phần" lớn nhất.

Khác với các hàm còn lại (tính riêng từng phần tử), Softmax và Maxout cần nhìn *cả* vectơ để tính mỗi phần tử.

2.1.2 Xếp chồng nhiều tầng

Bước tiếp theo hiển nhiên là tạo ra một mạng nơ-ron gồm nhiều tầng. Để làm điều này, ta chỉ cần xếp chồng nhiều tầng để tạo thành một **mạng nơ-ron nhiều tầng** (*multi-layer neural network*). Hình 2.3 minh họa một ví dụ. Trong kiến trúc này, đầu ra của mỗi nơ-ron trong một tầng được nối tới tất cả các nơ-ron ở tầng kế tiếp. Vì vậy mạng này là **kết nối đầy đủ** (*fully connected*). Về bản chất, một mạng nơ-ron nhiều tầng mô tả một phép hợp các hàm. Ví dụ, ta có thể biểu diễn mạng nơ-ron trong Hình 2.3 như một hàm phức tạp được tạo thành bằng cách hợp nhiều hàm đơn giản hơn:

$$\mathbf{y} = \text{Softmax}(\text{Sigmoid}(\text{ReLU}(\mathbf{x} \cdot \mathbf{w}_1) \cdot \mathbf{w}_2) \cdot \mathbf{w}_3 + \mathbf{b}_3), \tag{2.5}$$

trong đó $\mathbf{w}_1 \in \mathbb{R}^{3\times4}$, $\mathbf{w}_2 \in \mathbb{R}^{4\times3}$, $\mathbf{w}_3 \in \mathbb{R}^{3\times3}$ và $\mathbf{b}_3 \in \mathbb{R}^{3}$ là các tham số (*parameters*).

Chú giải của người dịch: Hàm hợp — đọc Phương trình (2.5) từ trong ra ngoài

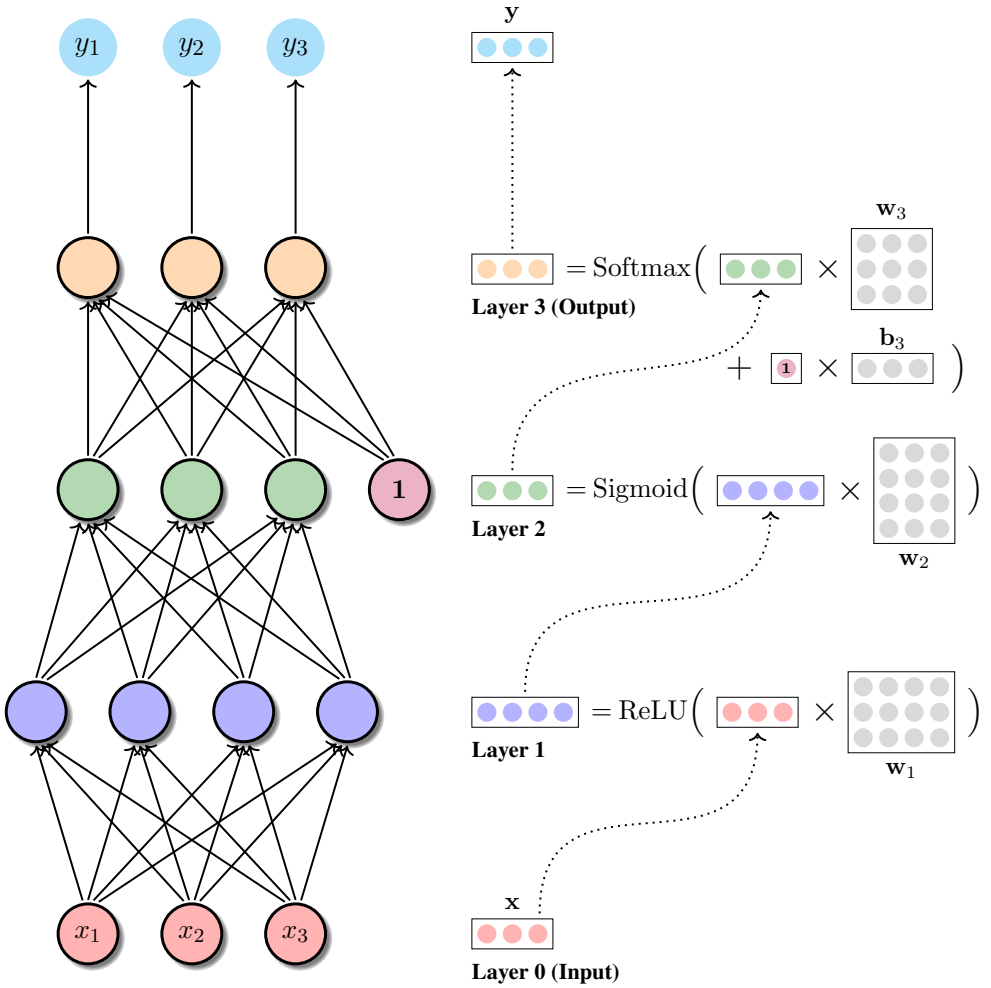
Ở THPT, ta đã gặp hàm hợp như $h(x) = \sin(x^2)$: tính x^2 trước, rồi lấy sin. Phương trình (2.5) cũng vậy, chỉ là nhiều lớp hơn: (1) nhân $\mathbf{x}$ (3 số) với $\mathbf{w}_1$ được 4 số, áp ReLU; (2) nhân kết quả với $\mathbf{w}_2$ được 3 số, áp Sigmoid; (3) nhân với $\mathbf{w}_3$, cộng $\mathbf{b}_3$, áp Softmax được 3 số cuối cùng. Kích thước các ma trận phải "khớp nhau": một hàng 3 số chỉ nhân được với ma trận có 3 hàng, và số cột của ma trận quyết định số phần tử của kết quả.

Thông thường, **độ sâu** (*depth*) của một mạng nơ-ron (đôi khi gọi là **độ sâu mô hình** (*model depth*)) được đo bằng số tầng của nó. Chẳng hạn, nếu coi vectơ đầu vào là tầng khởi đầu thì độ sâu của mạng ví dụ trong Hình 2.3 là 4. Một khái niệm liên quan là **độ rộng mô hình** (*model width*), được định nghĩa cho từng tầng chứ không phải cho toàn mạng. Một thước đo phổ biến cho độ rộng của một tầng là số nơ-ron mà tầng đó chứa. Ví dụ, độ rộng của tầng đầu ra trong Hình 2.3 là 3. Nếu mọi tầng của một mạng nơ-ron có cùng độ rộng n, ta có thể nói đơn giản rằng độ rộng của toàn mô hình là n. Cả độ sâu lẫn độ rộng mô hình đều có ảnh hưởng quan trọng tới năng lực biểu diễn (*representational capacity*) của mạng nơ-ron thu được. Ví dụ, người ta đã chứng minh bằng toán học rằng ngay cả một mạng nơ-ron chỉ có hai tầng nơ-ron với hàm kích hoạt Sigmoid cũng có thể xấp xỉ mọi hàm liên tục [Cybenko, 1989]. Tuy nhiên, với các bài toán phức tạp hơn, tăng độ sâu của mạng nhìn chung mang lại cải thiện đáng kể về hiệu năng.

Chú giải của người dịch: "Xấp xỉ mọi hàm liên tục" nghĩa là gì?

Kết quả của Cybenko (thường gọi là *định lý xấp xỉ phổ quát*) nói rằng: cho trước một hàm liên tục bất kỳ (trên một miền giới hạn) và một sai số cho phép nhỏ tùy ý, luôn tồn tại một mạng hai tầng với *đủ nhiều* nơ-ron mà đầu ra của nó sai khác hàm đó không quá sai số cho phép. Trực giác: mỗi nơ-ron Sigmoid tạo ra một "bậc thang mềm"; ghép đủ nhiều bậc thang nhỏ, ta có thể dựng lại hình dạng của bất kỳ đồ thị liên tục nào, giống như xếp các khối gạch nhỏ để mô phỏng một con dốc. Định lý chỉ khẳng định *tồn tại* mạng như vậy; nó không nói mạng cần bao nhiêu nơ-ron hay làm sao tìm được trọng số — vì thế trong thực tế mạng sâu thường hiệu quả hơn.

Việc xếp chồng các tầng tạo ra một loại mạng nơ-ron rất phổ biến — **mạng nơ-ron truyền thẳng** (*feed-forward neural networks, FFNNs*). Các mạng này được gọi là "truyền thẳng" vì không có chu trình trong các kết nối giữa các tầng và mọi dữ liệu đều di chuyển theo một chiều. Như ta sẽ thấy trong suốt cuốn sách, nhiều mạng nơ-ron hiện đại về bản chất là mạng truyền thẳng, dù một vài ngoại lệ đáng chú ý sẽ được thảo luận ở Mục 2.3.



Hình 2.3: Minh họa một mạng nơ-ron nhiều tầng. Tầng đầu vào gồm ba biến $\{x_1, x_2, x_3\}$. Các biến này được kết nối đầy đủ tới mọi nơ-ron của tầng 1. Đầu ra của tầng 1 là một vectơ mới $\mathbf{h}_1 = \text{ReLU}(\mathbf{x} \cdot \mathbf{w}_1)$. Vectơ này lại được kết nối đầy đủ tới tầng 2, thực hiện ánh xạ $\mathbf{h}_2 = \text{Sigmoid}(\mathbf{h}_1 \cdot \mathbf{w}_2)$. Đầu ra $\mathbf{h}_2$ của nó được đưa vào tầng 3, tầng này tạo ra đầu ra cuối cùng $\mathbf{y} = \text{Softmax}(\mathbf{h}_2 \cdot \mathbf{w}_3 + \mathbf{b}_3)$. Các tham số của mạng nơ-ron này là $\mathbf{w}_1$, $\mathbf{w}_2$, $\mathbf{w}_3$ và $\mathbf{b}_3$.

(Trong hình: *Layer 0 (Input)* = Tầng 0 (đầu vào); *Layer 1, 2* = Tầng 1, 2; *Layer 3 (Output)* = Tầng 3 (đầu ra).)

2.1.3 Đồ thị tính toán

Đồ thị tính toán (*computation graphs*) là một cách phổ biến để biểu diễn mạng nơ-ron. Giống như đồ thị trong toán học, một đồ thị tính toán được tạo thành từ các nút (*nodes*) và các cạnh (*edges*) nối giữa các nút. Mỗi nút biểu diễn hoặc một phép toán, hoặc một biến; mỗi cạnh biểu diễn luồng dữ liệu đi từ

nút này sang nút khác. Do đó, đồ thị tính toán là đồ thị có hướng (*directed*)[3]. Chẳng hạn, xét ba hàm sau:

$$\mathbf{y} = \mathbf{x} + \mathbf{w}, \tag{2.6}$$

$$\mathbf{y} = \text{Softmax}(\mathbf{x} \cdot \mathbf{w} + \mathbf{b}), \tag{2.7}$$

$$\mathbf{y} = \text{Sigmoid}(\mathbf{x} \cdot \mathbf{w}_1 + \mathbf{b}_1) - \text{ReLU}(\mathbf{x} \cdot \mathbf{w}_2). \tag{2.8}$$

Hình 2.4 cho thấy đồ thị tính toán của các hàm này. Từ góc độ phân tích cú pháp (*parsing*), mọi mạng nơ-ron đều có thể được xem là các biểu thức toán học. Vì vậy, đồ thị tính toán chính là biểu diễn của kết quả thu được khi phân tích cú pháp một biểu thức toán học. Theo cách này, mỗi nút của đồ thị cho ra một biểu thức con, còn nút gốc (*root node*) cho ra toàn bộ biểu thức.

Trong một đồ thị tính toán, một nút có thể được nối với nhiều nút ở bên dưới và/hoặc bên trên nó. Điều này cho phép tái sử dụng các đồ thị con khi biểu diễn những hàm phức tạp. Ví dụ, trong Phương trình (2.8), biến **x** được dùng hai lần và nút tương ứng có hai cạnh đi ra. Thực ra, việc tổ chức mạng nơ-ron thành đồ thị tính toán phản ánh bản chất *hợp thành* của mạng nơ-ron — thông thường, một mạng lớn được xây dựng bằng cách ghép các mạng nhỏ. Lấy Phương trình (2.8) làm ví dụ. Nó có thể được viết lại thành một hệ ba phương trình:

$$\mathbf{y} = \mathbf{h}_1 - \mathbf{h}_2, \tag{2.9}$$

$$\mathbf{h}_1 = \text{Sigmoid}(\mathbf{x} \cdot \mathbf{w}_1 + \mathbf{b}_1), \tag{2.10}$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{x} \cdot \mathbf{w}_2). \tag{2.11}$$

Trong phép hợp, các nút $\mathbf{h}_1$ và $\mathbf{h}_2$ trong Phương trình (2.9) được thay thế bởi đồ thị của các Phương trình (2.10–2.11).

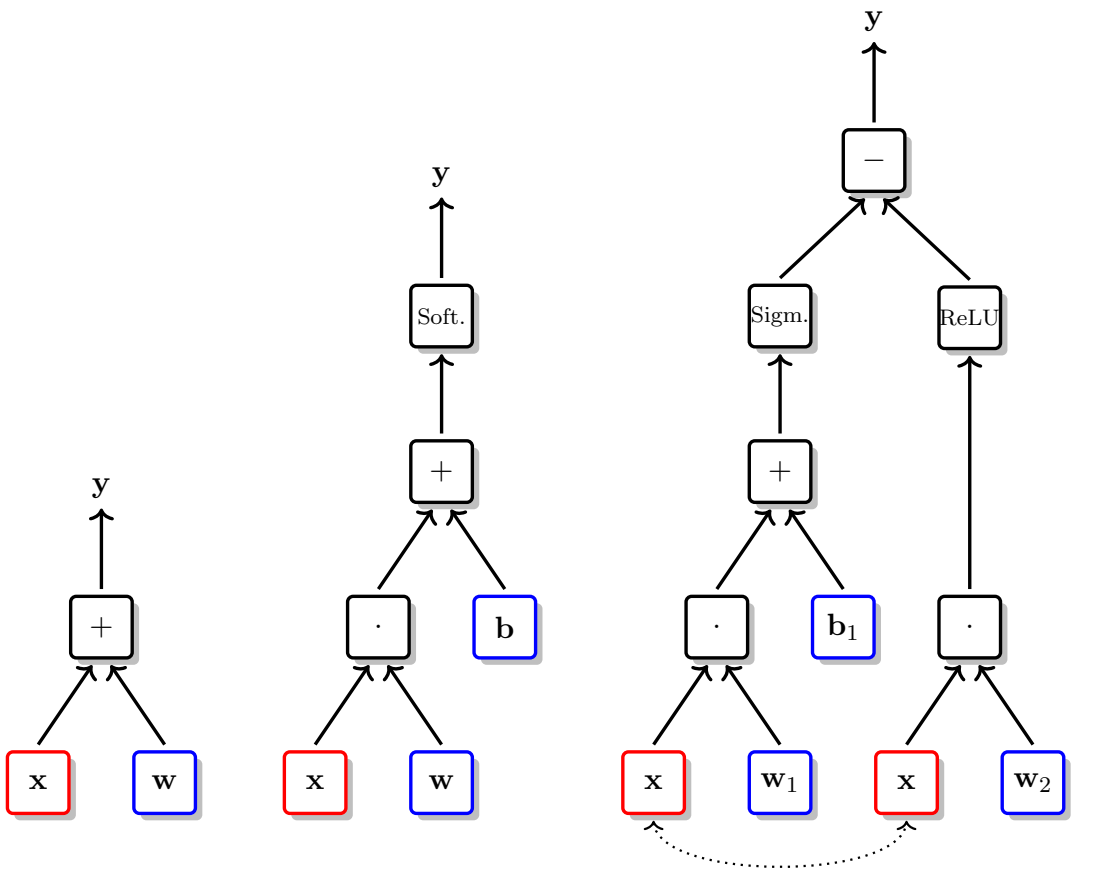
(Trong hình: *Soft.* = Softmax; *Sigm.* = Sigmoid; mũi tên chấm chấm ở (c) cho biết hai ô **x** thực chất là cùng một nút.)

Công dụng chính của đồ thị tính toán là để thực thi hàm số. Việc này hoàn toàn giống với việc dự đoán đầu ra của một mạng nơ-ron. Phương pháp khá đơn giản. Trước hết, các nút của đồ thị được sắp xếp tô-pô (*topologically sorted*), tức là được đặt theo một thứ tự nhất quán với luồng thông tin. Sau đó, với các giá trị được đưa vào các nút đầu vào, ta duyệt đồ thị sao cho tính được đầu ra của từng nút rồi chuyển tiếp nó tới các nút cha. Kết quả cuối cùng nhận được tại nút đầu ra. Thủ tục này thường được gọi là **lượt truyền xuôi** (*forward pass*). Lượt truyền xuôi có thể hiệu quả, vì mỗi nút chỉ cần được duyệt một lần và đầu ra của nó có thể được nhiều nút dùng lại mà không cần tính lại. Hơn nữa, lượt truyền xuôi có thể được tối ưu bằng cách tái cấu trúc đồ thị. Cách này có thể phát triển thêm ý tưởng tái sử dụng và tránh các phép tính cũng như mức tiêu thụ bộ nhớ không cần thiết.

Chú giải của người dịch: Lượt truyền xuôi — ví dụ bằng số

Xét Hình 2.4(a) với số thực đơn giản: $x = 3$, $w = 2$. Nút "+" đọc hai nút con và cho $y = 5$. Ở Hình 2.4(c), giả sử mọi thứ là số: $x = 1$, $w_1 = 2$, $b_1 = -2$, $w_2 = -1$. Đi từ dưới lên: $x \cdot w_1 = 2$; $2 + b_1 = 0$; $\text{Sigmoid}(0) = 0{,}5$. Nhánh phải: $x \cdot w_2 = -1$; $\text{ReLU}(-1) = 0$. Nút trên cùng: $y = 0{,}5 - 0 = 0{,}5$. "Sắp xếp tô-pô" chỉ đơn giản là sắp thứ tự tính sao cho *mọi nút được tính sau các nút mà nó cần*, giống như khi tính một biểu thức có ngoặc, ta luôn tính trong ngoặc trước. Nút x được dùng cho cả hai nhánh nhưng chỉ cần đọc một lần.

[3]Mặc dù một số mô hình học máy có thể được biểu diễn bằng đồ thị tính toán vô hướng, chúng không phải là trọng tâm của cuốn sách này.



(a) $\mathbf{y} = \mathbf{x} + \mathbf{w}$ (b) $\mathbf{y} = \text{Softmax}(\mathbf{x} \cdot \mathbf{w} + \mathbf{b})$ (c) $\mathbf{y} = \text{Sigmoid}(\mathbf{x} \cdot \mathbf{w}_1 + \mathbf{b}_1) - \text{ReLU}(\mathbf{x} \cdot \mathbf{w}_2)$

Hình 2.4: Đồ thị tính toán của ba mạng nơ-ron ví dụ. Các hộp màu đen biểu diễn các phép toán, còn các hộp có màu biểu diễn các biến. Một nút phép toán có các cạnh đi vào từ các nút khác, và mỗi nút đó có thể được coi là một đối số của phép toán. Ví dụ, trong hình con (a), nút phép cộng có hai nút con được gán nhãn lần lượt là $\mathbf{x}$ và $\mathbf{w}$. Nút này đọc đầu ra của các nút $\mathbf{x}$ và $\mathbf{w}$, rồi sinh ra đầu ra $\mathbf{y} = \mathbf{x} + \mathbf{w}$. Các đồ thị lớn hơn thể hiện những luồng dữ liệu phức tạp hơn. Trong đồ thị con (b), đầu ra của nút phép nhân chấm (tức $\mathbf{x} \cdot \mathbf{w}$) được truyền dọc theo cạnh tới nút phép cộng. Sau đó, nút phép cộng tính tổng của $\mathbf{x} \cdot \mathbf{w}$ và $\mathbf{b}$ làm đầu ra của nó. Ta có thể lặp lại cùng quá trình này trên tất cả các nút phép toán theo thứ tự từ dưới lên, và thu được kết quả cuối cùng của việc tính toàn bộ biểu thức tại nút trên cùng.

Một công dụng khác của đồ thị tính toán là tính gradient (*gradient*) một cách tự động. Khi huấn luyện mạng nơ-ron, nói chung ta cần tính các đạo hàm riêng (*partial derivatives*) của hàm mất mát (*loss function*) $\mathcal{L}$ theo mọi ma trận trọng số ($\mathbf{w}$) và mọi số hạng độ lệch ($\mathbf{b}$), chẳng hạn $\frac{\partial \mathcal{L}}{\partial \mathbf{w}}$ và $\frac{\partial \mathcal{L}}{\partial \mathbf{b}}$. Tuy nhiên, trước khi xem các đạo hàm riêng này được dùng như thế nào để cập nhật mô hình (xem Mục 2.4.1), chúng ta hãy tìm hiểu cách tính đạo hàm trong một đồ thị tính toán. Ví dụ, xét hàm dưới đây:

$$\mathbf{y} = \psi(\mathbf{x} \cdot \mathbf{w}_1 + \mathbf{b}_1) \cdot \mathbf{w}_2. \tag{2.12}$$

Chú giải của người dịch: Hàm mất mát, đạo hàm riêng và gradient

Hàm mất mát $\mathcal{L}$ là một con số đo xem đầu ra của mạng "sai" bao nhiêu so với đáp án đúng; huấn luyện là tìm cách chỉnh trọng số để con số này nhỏ đi. $\mathcal{L}$ phụ thuộc vào *rất nhiều* biến (mọi phần tử của $\mathbf{w}$, $\mathbf{b}$, …). **Đạo hàm riêng** $\frac{\partial \mathcal{L}}{\partial w}$ là đạo hàm của $\mathcal{L}$ theo *một* biến w, khi coi mọi biến khác là hằng số — cách tính y hệt đạo hàm một biến ở lớp 11–12. Ví dụ, với $\mathcal{L} = w^2 + 3wb$, ta có $\frac{\partial \mathcal{L}}{\partial w} = 2w + 3b$ và $\frac{\partial \mathcal{L}}{\partial b} = 3w$. Ký hiệu ∂ (đọc là "đạo hàm riêng") thay cho chữ d để nhắc rằng còn các biến khác. Khi $\mathbf{w}$ là một ma trận, $\frac{\partial \mathcal{L}}{\partial \mathbf{w}}$ là ma trận cùng kích thước gồm các đạo hàm riêng theo từng phần tử; tập hợp tất cả các đạo hàm riêng này gọi là **gradient**. Nó cho biết nếu nhích nhẹ từng trọng số thì mất mát tăng hay giảm, và nhanh tới mức nào.

Để tính $\frac{\partial \mathcal{L}}{\partial \mathbf{w}_1}$, $\frac{\partial \mathcal{L}}{\partial \mathbf{b}_1}$ và $\frac{\partial \mathcal{L}}{\partial \mathbf{w}_2}$, cách tự nhiên là dùng quy tắc chuỗi của phép lấy đạo hàm (*chain rule of differentiation*). Ví dụ, với một hàm hợp $y = p(q(x))$, công thức của quy tắc chuỗi là:

$$\frac{\partial y}{\partial x} = \frac{\partial p}{\partial q} \cdot \frac{\partial q}{\partial x}. \tag{2.13}$$

Tuy nhiên, công thức giải tích của đạo hàm dựa trên Phương trình (2.13) sẽ dẫn tới một biểu thức rất dài. Thay vào đó, ta có thể phân rã một hàm phức tạp thành nhiều hàm, mỗi hàm đại diện cho một phép toán. Khi đó, Phương trình (2.12) có thể được viết lại thành:

$$\mathbf{y} = \mathbf{h}_1 \cdot \mathbf{w}_2, \tag{2.14}$$

$$\mathbf{h}_1 = \psi(\mathbf{h}_2), \tag{2.15}$$

$$\mathbf{h}_2 = \mathbf{h}_3 + \mathbf{b}_1, \tag{2.16}$$

$$\mathbf{h}_3 = \mathbf{x} \cdot \mathbf{w}_1. \tag{2.17}$$

Tất cả các biến này có thể được hiểu một cách trực quan hơn qua đồ thị tính toán: mỗi biến là một nút của đồ thị, và các nút được nối với nhau bằng các phép toán đại số và phép hợp hàm. Dùng Phương trình (2.13) và một số kiến thức cơ bản về giải tích, ta tính đạo hàm của các biến như sau:

nút 1: $$\frac{\partial \mathcal{L}}{\partial \mathbf{y}} = \delta_{\mathbf{y}}, \tag{2.18}$$

nút 2: $$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \mathbf{w}_2^\top, \tag{2.19}$$

nút 3: $$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_2} = \mathbf{h}_1^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}}, \tag{2.20}$$

nút 4: $$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_2} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} \odot \psi'(\mathbf{h}_2), \tag{2.21}$$

nút 5: $$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_3} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_2}, \tag{2.22}$$

nút 6: $$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_2}, \tag{2.23}$$

nút 7: $$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_3} \cdot \mathbf{w}_1^\top, \tag{2.24}$$

nút 8: $$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_1} = \mathbf{x}^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{h}_3}, \tag{2.25}$$

trong đó $\delta_{\mathbf{y}}$ là đạo hàm của hàm mất mát theo đầu ra của mô hình. $\delta_{\mathbf{y}}$ phụ thuộc vào cách chọn hàm mất mát; ví dụ, nếu dùng hàm mất mát bình phương (*squared loss*) $\mathcal{L} = \frac{1}{2}(\mathbf{y} - \mathbf{y}_{\text{gold}})^2$, trong đó $\mathbf{y}_{\text{gold}}$ là đáp án đúng (*ground-truth*), thì $\delta_{\mathbf{y}} = \mathbf{y} - \mathbf{y}_{\text{gold}}$. Quá trình trên về bản chất là một **lượt truyền ngược** (*backward pass*), vì các gradient được truyền theo hướng từ trên xuống dưới. Một tên gọi khác của nó là **lan truyền ngược** (*backpropagation*). Đây đã trở thành tiêu chuẩn trên thực tế cho việc huấn luyện các mạng nơ-ron sâu. Để hiểu rõ hơn cách hoạt động của lượt truyền xuôi và lượt truyền ngược, Hình 2.5 trình bày hai ví dụ minh họa từng bước.

Chú giải của người dịch: Quy tắc chuỗi và lan truyền ngược với số vô hướng

Quy tắc chuỗi (2.13) chính là công thức đạo hàm hàm hợp ở lớp 12: $(p(q(x)))' = p'(q) \cdot q'(x)$. Để thấy các Phương trình (2.18–2.25) hoạt động, hãy coi mọi thứ là số (khi đó dấu chuyển vị $^\top$ không có tác dụng, còn $\odot$ là phép nhân thường). Chọn $\psi = \text{ReLU}$, $x = 1$, $w_1 = 2$, $b_1 = 1$, $w_2 = 3$, đáp án $y_{\text{gold}} = 5$. *Truyền xuôi:* $h_3 = 2$, $h_2 = 3$, $h_1 = \text{ReLU}(3) = 3$, $y = 9$. *Truyền ngược:* $\delta_y = y - y_{\text{gold}} = 4$; $\frac{\partial \mathcal{L}}{\partial h_1} = 4 \cdot 3 = 12$; $\frac{\partial \mathcal{L}}{\partial w_2} = h_1 \cdot 4 = 12$; vì $h_2 = 3 > 0$ nên $\psi'(h_2) = 1$, do đó $\frac{\partial \mathcal{L}}{\partial h_2} = 12$; $\frac{\partial \mathcal{L}}{\partial b_1} = \frac{\partial \mathcal{L}}{\partial h_3} = 12$; $\frac{\partial \mathcal{L}}{\partial w_1} = x \cdot 12 = 12$. Kiểm tra trực tiếp: $\mathcal{L} = \frac{1}{2}\big((xw_1 + b_1)w_2 - 5\big)^2$, đạo hàm theo w_2 là $\big((xw_1 + b_1)w_2 - 5\big)(xw_1 + b_1) = 4 \cdot 3 = 12$. Khớp! Ý tưởng then chốt: mỗi nút chỉ cần nhân "gradient nhận từ phía trên" với đạo hàm cục bộ của chính nó rồi chuyển xuống dưới; nhờ đó không phải viết ra một công thức đạo hàm khổng lồ. Ký hiệu $\odot$ là nhân theo từng phần tử (*element-wise product*): $[1\ \ 2] \odot [3\ \ 4] = [3\ \ 8]$; $\mathbf{w}^\top$ là chuyển vị (*transpose*) của ma trận (đổi hàng thành cột).

(Trong hình: *Forward Pass* = lượt truyền xuôi; *Backward Pass* = lượt truyền ngược; *Output* = đầu ra; *Loss* = mất mát. Các số trong chấm tròn là số thứ tự nút trong Phương trình (2.18–2.25).)

2.2 Ví dụ: mô hình hóa ngôn ngữ bằng mạng nơ-ron

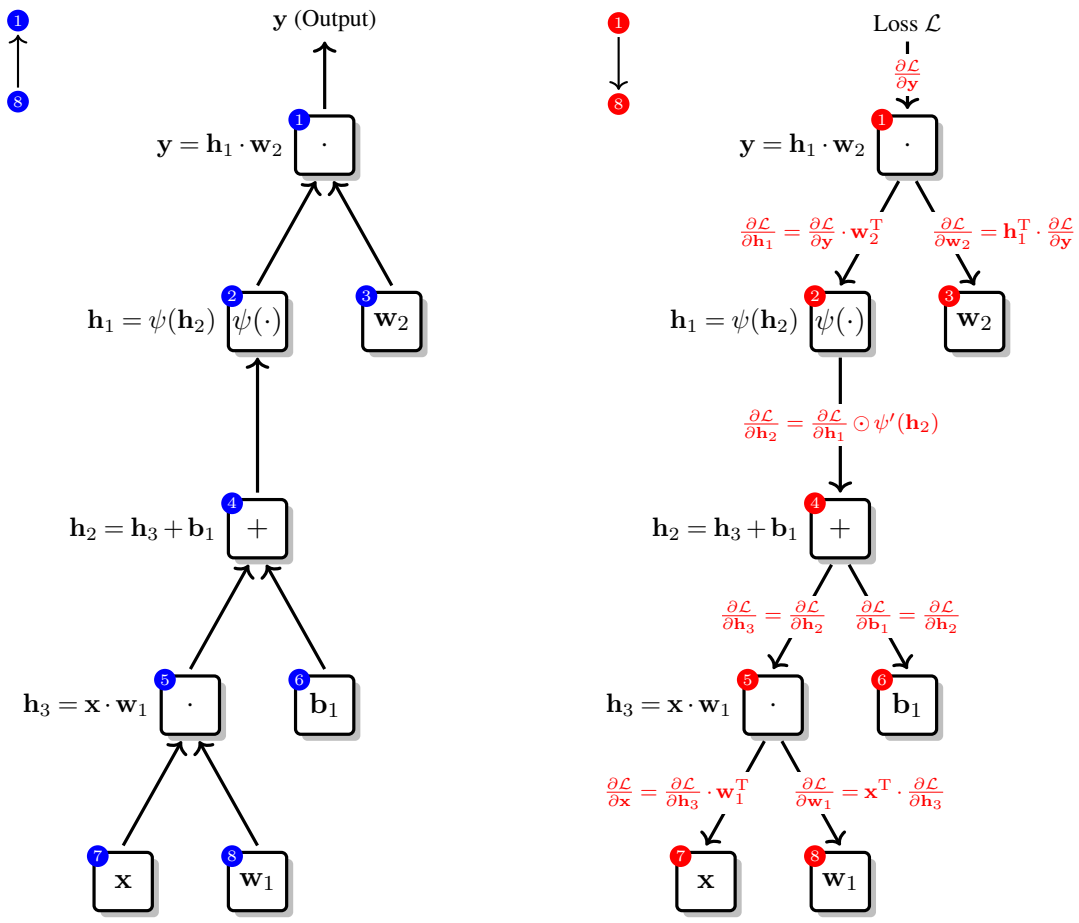
Mô hình hóa ngôn ngữ là một bài toán NLP nền tảng, nhằm ước lượng một phân phối xác suất trên các chuỗi từ. Cho một chuỗi gồm m từ $w_1 \cdots w_m$, xác suất đồng thời (*joint probability*) $\Pr(w_1, \ldots, w_m)$ được cho bởi phương trình:

$$\Pr(w_1, \ldots, w_m) = \prod_{i=1}^{m} \Pr(w_i | w_1, \ldots, w_{i-1}). \tag{2.26}$$

Như vậy, bài toán mô hình hóa ngôn ngữ được phát biểu thành bài toán dự đoán từ tiếp theo khi biết tất cả các từ ngữ cảnh đứng trước. Một cách đơn giản để mô hình hóa $\Pr(w_i | w_1, \ldots, w_{i-1})$ là cho dự đoán chỉ phụ thuộc vào một cửa sổ ngữ cảnh (*context window*) bao gồm một số cố định các từ đứng trước, như sau:

$$\Pr(w_i | w_1, \ldots, w_{i-1}) \approx \Pr(w_i | w_{i-n+1}, \ldots, w_{i-1}), \tag{2.27}$$

trong đó n là kích thước cửa sổ. Một cách để ước lượng xác suất này là phương pháp **mô hình ngôn ngữ n-gram** (n-*gram language modeling*): ta tính tần suất tương đối (*relative frequency*) của mỗi n-gram $w_{i-n+1} \cdots w_i$, tức là $\Pr(w_i | w_{i-n+1}, \ldots, w_{i-1}) = \frac{\text{count}(w_{i-n+1} \ldots w_i)}{\text{count}(w_{i-n+1} \ldots w_{i-1})}$. Mặc dù mô hình ngôn ngữ n-gram đã thống trị lĩnh vực NLP trong nhiều thập kỷ, chúng đòi hỏi những bảng khổng lồ để lưu mọi



(a) Forward Pass (b) Backward Pass

Hình 2.5: Minh họa lượt truyền xuôi và lượt truyền ngược trên một đồ thị tính toán ví dụ. Trong lượt truyền xuôi (bên trái), các nút được duyệt theo thứ tự từ đầu vào tới đầu ra, ví dụ từ nút 8 tới nút 1. Tại mỗi nút, ta thực hiện hàm tương ứng, chẳng hạn phép cộng, để sinh ra đầu ra; đầu ra này sau đó được các nút tiếp theo sử dụng. Ngược lại, trong lượt truyền ngược (bên phải), các nút được duyệt theo thứ tự đảo ngược, ví dụ từ nút 1 tới nút 8. Trong quá trình này, ta truyền gradient của hàm mất mát (hay lỗi) từ đầu ra về đầu vào; nghĩa là, với mỗi nút, ta tính gradient tại điểm đầu vào của nút bằng quy tắc chuỗi, khi đã biết gradient tại điểm đầu ra của nút.

xác suất n-gram. Hệ quả là, khi kho ngữ liệu huấn luyện lớn dần và ngữ cảnh được xét dài hơn, dữ liệu trở nên cực kỳ thưa. Đây là một ví dụ kinh điển của lời nguyền số chiều (*curse of dimensionality*).

Chú giải của người dịch: Mô hình ngôn ngữ và n-gram qua ví dụ

Phương trình (2.26) là *quy tắc nhân xác suất* áp dụng liên tiếp. Với câu "tôi đi học": $\Pr(\text{tôi}, \text{đi}, \text{học}) = \Pr(\text{tôi}) \cdot \Pr(\text{đi} \mid \text{tôi}) \cdot \Pr(\text{học} \mid \text{tôi}, \text{đi})$. Ký hiệu $\Pr(A|B)$ đọc là "xác suất của A khi biết B" (xác suất có điều kiện), và $\prod_{i=1}^{m}$ là "tích của m thừa số", tương tự $\sum$ nhưng với phép nhân. Mô hình n-gram "quên bớt" ngữ cảnh xa: với $n = 2$ (bigram), $\Pr(\text{học}|\text{tôi}, \text{đi}) \approx \Pr(\text{học}|\text{đi})$. Ước lượng bằng đếm: nếu trong kho ngữ liệu cụm "đi" xuất hiện 100 lần, trong đó "đi học" xuất hiện 20 lần, thì $\Pr(\text{học}|\text{đi}) \approx 20/100 = 0{,}2$. Vấn đề: với bộ từ vựng 10 000 từ và $n = 3$ đã có 10^{12} bộ ba có thể có; hầu hết chưa từng xuất hiện trong dữ liệu nên bị đếm bằng 0 — đó là tính "thưa" của dữ liệu.

Ở đây, chúng ta xét việc dùng mạng nơ-ron để giải quyết bài toán mô hình hóa ngôn ngữ [Bengio và cộng sự, 2000; 2003b]. Khác với mô hình n-gram, vốn hoạt động trong một không gian rời rạc đòi hỏi một số lượng lớn theo cấp số mũ các vectơ đặc trưng khác nhau, các **mô hình ngôn ngữ nơ-ron** (*neural language models*) khái quát hóa trong một không gian liên tục bằng cách mã hóa các từ thành những vectơ số thực dày đặc và có số chiều thấp. Cụ thể, ở đây ta dùng một mạng truyền thẳng để dự đoán khả năng từ w_i xuất hiện khi biết ngữ cảnh $w_{i-n+1} \cdots w_{i-1}$.

Hình 2.6 trình bày kiến trúc của **mô hình ngôn ngữ dựa trên mạng nơ-ron truyền thẳng** (*feed-forward neural network based language model, FFNNLM*). Đầu vào là các từ ngữ cảnh $w_{i-n+1} \cdots w_{i-1}$. Mỗi từ là một biến rời rạc nhận giá trị trong một bộ từ vựng (*vocabulary*) V. Vì mạng nơ-ron làm việc với các vectơ, mọi từ đều được vectơ hóa dưới dạng **biểu diễn one-hot** (*one-hot representations*). Khi đó, từ $w = V_k$ là một vectơ $|V|$ chiều, trong đó phần tử thứ k bằng 1 và mọi phần tử khác đều bằng 0. Ví dụ, xét bộ từ vựng $V = \{\text{"I"}, \text{"you"}, \text{"he"}, \text{"she"}, \text{"they"}\}$ ("tôi", "bạn", "anh ấy", "cô ấy", "họ"). Biểu diễn one-hot của "you" được viết là

$$\mathbf{1}_{\text{you}} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \end{bmatrix}. \tag{2.28}$$

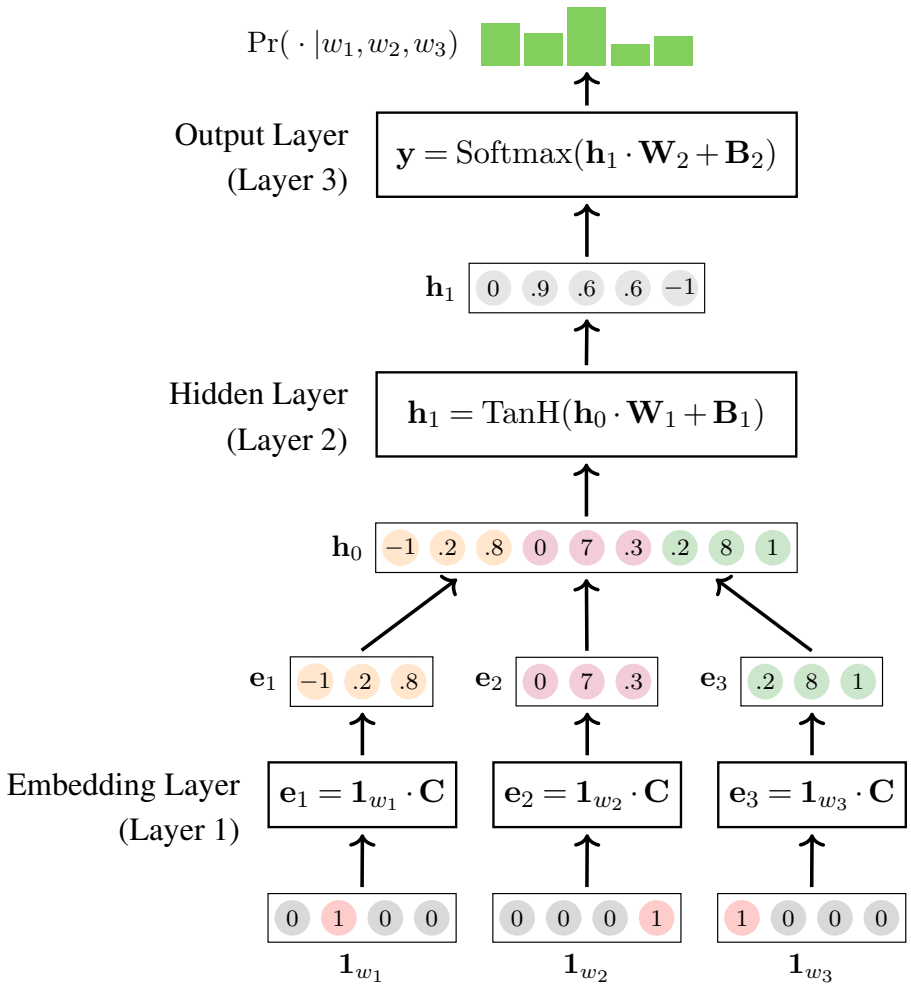
Mặc dù các vectơ one-hot xác định duy nhất từng từ, chúng không nắm bắt được các quan hệ ngữ nghĩa (tức là kỳ vọng rằng các từ tương tự nhau nên nằm gần nhau trong không gian vectơ). Bằng cách nới lỏng các biểu diễn dựa trên chỉ thị này thành các vectơ số thực liên tục, ta có thể nắm bắt quan hệ giữa các từ bằng cách tính độ tương đồng giữa các vectơ. Để làm được điều đó, một kỹ thuật hiệu quả là biến đổi biểu diễn one-hot thành **biểu diễn phân tán** (*distributed representations*). Một cách hình thức hơn, gọi $\mathbf{x}$ là vectơ one-hot của một từ w. Biểu diễn phân tán của nó là một vectơ số thực, được cho bởi:

$$\mathbf{e} = \mathbf{x} \cdot \mathbf{C}, \tag{2.29}$$

trong đó biểu diễn $\mathbf{e}$ là một vectơ thuộc $\mathbb{R}^{d_e}$, và d_e là số chiều của biểu diễn. Mỗi chiều của $\mathbf{e}$ có thể được xem như một khía cạnh đo đếm được nào đó của từ, dù không bắt buộc phải diễn giải được về mặt ngôn ngữ học. $\mathbf{C}$ là một ma trận $|V| \times d_e$, trong đó hàng thứ k tương ứng với vectơ của V_k. Do đó, phép $\mathbf{x} \cdot \mathbf{C}$ có tác dụng "chọn ra" một hàng của $\mathbf{C}$. Ví dụ, với $\mathbf{C} \in \mathbb{R}^{5\times 3}$, biểu diễn phân tán của "you" được cho bởi:

$$\begin{aligned} e(\text{"you"}) &= \mathbf{1}_{\text{you}} \cdot \mathbf{C} \\ &= \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 73 & 12 & 0.1 \\ 12 & 0.5 & 18 \\ 37 & 0.7 & 28 \\ 61 & 0.4 & 23 \\ 62 & 11 & 0.4 \end{bmatrix} \end{aligned}$$

$$= \begin{bmatrix} 12 & 0.5 & 18 \end{bmatrix}. \tag{2.30}$$



Hình 2.6: Minh họa một mô hình ngôn ngữ nơ-ron [Bengio và cộng sự, 2003b]. Các hộp biểu diễn các tầng của mạng nơ-ron. Đầu vào là ba từ ngữ cảnh ở dạng biểu diễn one-hot $\{\mathbf{1}_{w_1}, \mathbf{1}_{w_2}, \mathbf{1}_{w_3}\}$, và đầu ra là phân phối xác suất của từ tiếp theo $\Pr(w_4|w_1, w_2, w_3)$. Trước hết, một tầng nhúng được dùng để ánh xạ mỗi từ thành biểu diễn phân tán (tức là vectơ nhúng từ). Các vectơ nhúng của những từ này được ghép nối (*concatenated*) để tạo thành một vectơ lớn hơn $\mathbf{h}_0$, sao cho vectơ ghép nối này mã hóa toàn bộ thông tin đầu vào. Sau đó, $\mathbf{h}_0$ được lấy làm đầu vào cho một tầng thông thường thực hiện ánh xạ $\mathbf{h}_1 = \text{TanH}(\mathbf{h}_0 \cdot \mathbf{W}_1 + \mathbf{B}_1)$. Tầng cuối cùng đọc $\mathbf{h}_1$ và tạo ra một phân phối trên bộ từ vựng, tức là $\mathbf{y} = \text{Softmax}(\mathbf{h}_1 \cdot \mathbf{W}_2 + \mathbf{B}_2)$, trong đó $y_k = \Pr(V_k|w_1, w_2, w_3)$.

(Trong hình: *Embedding Layer (Layer 1)* = Tầng nhúng (tầng 1); *Hidden Layer (Layer 2)* = Tầng ẩn (tầng 2); *Output Layer (Layer 3)* = Tầng đầu ra (tầng 3).)

Chú giải của người dịch: One-hot và vectơ nhúng: vì sao cần?

Vectơ one-hot giống một "thẻ căn cước": mỗi từ có một vị trí riêng bật số 1. Nhưng mọi cặp vectơ one-hot khác nhau đều có tích vô hướng bằng 0 và khoảng cách như nhau, nên "anh ấy" và "cô ấy" cũng xa nhau y như "anh ấy" và "họ" — máy không thấy được từ nào giống từ nào. Phương trình (2.30) cho thấy nhân vectơ

one-hot với ma trận $\mathbf{C}$ thực chất chỉ là *tra bảng*: số 1 ở vị trí thứ 2 lấy ra đúng hàng thứ 2 (màu đỏ). Mỗi từ vì thế được gán một dãy số thực ngắn (ở đây 3 số) gọi là vectơ nhúng; các số này được *học* từ dữ liệu, và sau khi học, những từ có cách dùng giống nhau thường có vectơ gần nhau.

Phương trình (2.29) giới thiệu khái niệm nền tảng về việc học biểu diễn từ, vốn đã thúc đẩy những tiến bộ to lớn trong NLP. Thông thường, vectơ $\mathbf{e}$ được gọi là **vectơ nhúng từ** (*word embedding*), còn ma trận tham số $\mathbf{C}$ được gọi là **ma trận nhúng** (*embedding matrix*). Dù có nhiều phương pháp học vectơ nhúng từ, chúng tôi sẽ không tập trung vào chúng trong chương này. Bạn đọc có thể xem Chương 3 để biết thảo luận chi tiết về chủ đề này.

Để mã hóa các từ ngữ cảnh $\{w_{i-n+1}, \ldots, w_{i-1}\}$, một cách đơn giản là ghép nối các vectơ nhúng từ $\{\mathbf{e}_{i-n+1}, \ldots, \mathbf{e}_{i-1}\}$ thành một vectơ mới $\mathbf{h}_0$:

$$\mathbf{h}_0 = [\mathbf{e}_{i-n+1}, \ldots, \mathbf{e}_{i-1}].$$

Phần tiếp theo của mô hình là một mạng nơ-ron truyền thẳng 2 tầng. Tầng thứ nhất, gọi là **tầng ẩn** (*hidden layer*), là một tầng nơ-ron chuẩn, theo sau là hàm kích hoạt tang hyperbolic. Tầng này tạo ra một vectơ d_h chiều:

$$\mathbf{h}_1 = \text{TanH}(\mathbf{h}_0 \cdot \mathbf{W}_1 + \mathbf{B}_1). \tag{2.31}$$

Tầng thứ hai là tầng đầu ra. Nó tạo ra một phân phối trên V. Điều này có thể được biểu diễn là:

$$\Pr(\cdot\,|w_{i-n+1}, \ldots, w_{i-1}) = \text{Softmax}(\mathbf{h}_1 \cdot \mathbf{W}_2 + \mathbf{B}_2). \tag{2.32}$$

Các tham số của mô hình là $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$, $\mathbf{W}_1 \in \mathbb{R}^{(n-1)d_e \times d_h}$, $\mathbf{B}_1 \in \mathbb{R}^{d_h}$, $\mathbf{W}_2 \in \mathbb{R}^{d_h \times |V|}$ và $\mathbf{B}_2 \in \mathbb{R}^{|V|}$. Một cách phổ biến để tối ưu các tham số này là cực tiểu hóa hàm mất mát entropy chéo (*cross-entropy loss*) bằng hạ gradient (*gradient descent*). Ngoài ra, việc huấn luyện có thể được cải thiện nhờ chính quy hóa. Các phương pháp này sẽ được thảo luận ở Mục 2.4 và Mục 2.5.

Chú giải của người dịch: Đếm kích thước trong mô hình

Giả sử $n = 4$ (dùng 3 từ ngữ cảnh), $d_e = 3$ như Hình 2.6. Ghép ba vectơ nhúng 3 chiều được $\mathbf{h}_0$ có $(n-1)d_e = 9$ chiều — đúng 9 ô trong hình. Vì vậy $\mathbf{W}_1$ phải có 9 hàng; số cột d_h là độ rộng tầng ẩn (trong hình $d_h = 5$). Tầng ra nhân $\mathbf{h}_1$ (5 số) với $\mathbf{W}_2$ ($5 \times |V|$) được $|V|$ số — mỗi từ trong bộ từ vựng một điểm số — rồi Softmax biến chúng thành xác suất để từ đó là từ tiếp theo. Chữ "hạ gradient" ở đây nghĩa là liên tục nhích các tham số theo hướng làm hàm mất mát giảm, dùng gradient tính bằng lan truyền ngược như Mục 2.1.3.

Một vài nhận xét về mô hình ngôn ngữ nơ-ron: thứ nhất, nhờ dùng các vectơ đặc trưng phân tán, các từ khác nhau có thể chia sẻ một phần "nghĩa" với nhau. Điều này cho phép học được nghĩa của từ, qua đó độ tương đồng giữa các từ được xét đến một cách ngầm định. Một ưu điểm của mô hình như vậy là một thay đổi nhỏ trong vectơ từ sẽ không dẫn tới thay đổi lớn trong kết quả. Ví dụ, giả sử ta đã thấy câu "grapes are fruits" ("nho là trái cây") nhiều lần nhưng chưa bao giờ thấy "peaches are fruits" ("đào là trái cây"). Nếu vectơ nhúng của "grapes" và "peaches" gần nhau, mô hình có thể suy ra một cách hợp lý rằng cụm "peaches are fruits" có xác suất cao. Điều này làm mô hình ngôn ngữ nơ-ron khác biệt rất lớn so với mô hình ngôn ngữ n-gram, nơi các dạng bề mặt khác nhau được xem là những thực thể hoàn toàn độc lập.

Thứ hai, biểu diễn dày đặc của từ giúp mô hình nhỏ hơn. Ví dụ, một thiết lập phổ biến của d_e và d_h là nhỏ hơn 1000, giữ cho tổng số tham số ở mức kiểm soát được. Ngược lại, kích thước của một mô hình ngôn ngữ n-gram tăng lên gấp $|V|$ lần mỗi khi n tăng. Chẳng hạn, với một bộ từ vựng thông thường, sẽ có một bảng xác suất khổng lồ nếu n lớn hơn 3.

Thứ ba, mô hình ngôn ngữ nơ-ron tốn kém về tính toán vì sử dụng rất nhiều phép toán trên vectơ và ma trận, chẳng hạn phép nhân ma trận. Đây là vấn đề chung của hầu hết các hệ thống dựa trên mạng nơ-ron sâu. Một giải pháp phổ biến là chia bài toán tính toán thành các bài toán con độc lập để có thể xử lý song song các bài toán con này. Ở mức thấp, ta có thể dùng GPU hoặc các thiết bị tính toán song song khác để tăng tốc các phép toán đại số tuyến tính. Ở mức cao hơn, ta có thể phân phối các phần của mô hình hoặc các phần của dữ liệu lên nhiều thiết bị để tăng tốc ở mức mô hình hoặc mức tác vụ.

2.3 Các kiến trúc mô hình cơ bản

Bây giờ chúng ta mô tả chi tiết hơn một số khối xây dựng cơ bản của mạng nơ-ron. Chúng được dùng rộng rãi để phát triển các mô hình nơ-ron tiên tiến nhất (*state-of-the-art*) trong NLP.

2.3.1 Đơn vị hồi quy

Mạng nơ-ron hồi quy (*recurrent neural networks*) (**RNN**) là một lớp mạng nơ-ron đọc và/hoặc sinh ra dữ liệu dạng chuỗi hoặc dữ liệu chuỗi thời gian (*time-series*). Cũng như mạng nơ-ron truyền thẳng (*feed-forward neural network*), một RNN gồm các tầng nơ-ron và các kết nối giữa các nơ-ron [Hopfield, 1984; Rumelhart và cộng sự, 1986; Williams và Zipser, 1989; Elman, 1990]. Một số nơ-ron được dùng như một "bộ nhớ" lưu giữ trạng thái của bài toán khi quá trình xử lý tiến dần dọc theo một chuỗi tín hiệu. Nhờ vậy, việc dùng RNN để giải quyết các bài toán có độ dài thay đổi, chẳng hạn dịch máy (*machine translation*) và nhận dạng tiếng nói (*speech recognition*), là rất tự nhiên.

Ý tưởng chính đằng sau RNN là sử dụng lặp đi lặp lại một **đơn vị hồi quy** (*recurrent unit*) (hay **ô hồi quy** (*recurrent cell*)) để tính đầu ra tại mỗi vị trí của chuỗi đầu vào. Chính xác hơn, cho một chuỗi vectơ $\mathbf{x}_1 \cdots \mathbf{x}_m$, một **đơn vị hồi quy** chuẩn có thể được mô tả như một hàm $\text{RNN}(\cdot)$, tại mỗi bước thời gian (*time step*) nhận một đầu vào $\mathbf{x}_i$ và một trạng thái $\mathbf{s}_{i-1}$, rồi sinh ra một trạng thái mới $\mathbf{s}_i$:

$$\mathbf{s}_i \;=\; \text{RNN}(\mathbf{s}_{i-1}, \mathbf{x}_i). \tag{2.33}$$

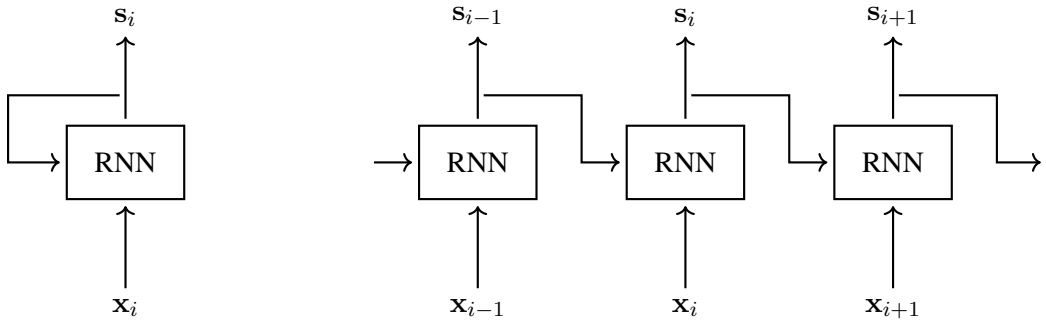
Trạng thái $\mathbf{s}_i$ có thể được xem như một "bộ nhớ" tóm tắt thông tin trong quá khứ và được cập nhật khi có dữ liệu mới đến. Xem Hình 2.7 (a) để thấy minh họa của Phương trình (2.33). Vòng lặp trong hình biểu thị việc tái sử dụng đơn vị hồi quy. Điều này có thể hiểu được bằng cách viết lại Phương trình (2.33) thành một dãy các lời gọi hàm $\text{RNN}(\cdot)$:

$$\begin{aligned}
\text{RNN}(\mathbf{s}_{i-1}, \mathbf{x}_i) &= \text{RNN}(\text{RNN}(\mathbf{s}_{i-2}, \mathbf{x}_{i-1}), \mathbf{x}_i) \\
&= \text{RNN}(\text{RNN}(\text{RNN}(\mathbf{s}_{i-3}, \mathbf{x}_{i-2}), \mathbf{x}_{i-1}), \mathbf{x}_i) \\
&= \dots \\
&= \text{RNN}(\text{RNN}(\cdots(\text{RNN}(\mathbf{s}_0, \mathbf{x}_1), \mathbf{x}_2)\cdots\mathbf{x}_{i-1}), \mathbf{x}_i).
\end{aligned} \tag{2.34}$$

Hình 2.7 (b) minh họa cấu trúc của mạng đã được khai triển này. Cách biểu diễn này thường được gọi là cấu trúc **trải phẳng** (*unrolled*) (hay **mở rộng** (*unfolded*)) của RNN. Về bản chất, Hình 2.7 (a) và (b) biểu diễn đúng cùng một tập phép toán. Trong khi RNN dạng cuộn (*rolled*) cho một dạng toán học gọn gàng và đẹp đẽ, thì RNN dạng trải phẳng giúp hình dung rõ hơn dòng dữ liệu chảy qua thời gian. Vì vậy, trong suốt cuốn sách này chúng tôi sẽ dùng cách biểu diễn trải phẳng của RNN.

Hơn nữa, đáng lưu ý rằng một RNN trải phẳng có thể được xem như một mạng nơ-ron truyền thẳng sâu, trong đó mỗi lần áp dụng đơn vị hồi quy đóng vai trò một "tầng" nhận thông tin từ "tầng" liền trước

(tức là bước thời gian trước đó). Theo nghĩa này, một RNN thực chất là một chồng các tầng trải dài theo chiều ngang qua thời gian. Lợi ích của góc nhìn này là ta có thể dùng những kỹ thuật tối ưu tương tự để huấn luyện cả hai kiến trúc. Chẳng hạn, cả RNN và mạng truyền thẳng đều dựa vào tối ưu hóa dựa trên gradient, chỉ có điều RNN dùng riêng một biến thể của lan truyền ngược gọi là **lan truyền ngược theo thời gian** (*back-propagation through time*) (**BPTT**) để lan truyền sai số ngược lại qua đồ thị tính toán (*computation graph*) đã trải phẳng.



(a) Rolled RNN Structure (b) Unrolled RNN Structure

Hình 2.7: Ví dụ về RNN (dạng cuộn và dạng trải phẳng). Một đơn vị RNN đọc đầu vào tại mỗi bước thời gian i và đầu ra tại bước thời gian trước $i-1$, rồi sinh ra một đầu ra mới s_i. Như vậy, ta có thể tái sử dụng cùng một đơn vị RNN để đưa ra dự đoán trên một chuỗi đầu vào (xem hình con (a)): với mỗi i, đầu vào hiện tại $\mathbf{x}_i$ và đầu ra trước đó $\mathbf{s}_{i-1}$ được tiếp nhận và ánh xạ thành đầu ra, đầu ra này lại được đưa vào chính đơn vị RNN đó để xử lý ở bước thời gian kế tiếp. Một cách tốt hơn để hình dung RNN là trải nó ra thành một mạng không có chu trình (xem hình con (b)). RNN trải phẳng có thể được xem như một mạng nơ-ron truyền thẳng sâu, trong đó tất cả các đơn vị RNN dùng chung một bộ tham số.

Nhãn trong hình: *Rolled RNN Structure* = cấu trúc RNN dạng cuộn; *Unrolled RNN Structure* = cấu trúc RNN dạng trải phẳng.

Chú giải của người dịch: RNN giống như đọc truyện từng trang

Hãy tưởng tượng bạn đọc một câu chuyện từng chữ một và giữ trong đầu một "bản tóm tắt" $\mathbf{s}$. Mỗi khi đọc thêm một chữ $\mathbf{x}_i$, bạn cập nhật bản tóm tắt: $\mathbf{s}_i = \text{RNN}(\mathbf{s}_{i-1}, \mathbf{x}_i)$. Điều quan trọng là bạn dùng *cùng một cách cập nhật* (cùng một hàm, cùng tham số) cho mọi chữ, nên câu dài 5 chữ hay 500 chữ đều xử lý được. Phương trình (2.34) chỉ là phép "thay thế liên tiếp" giống như với dãy số truy hồi ở lớp 11: với $u_n = 2u_{n-1} + 1$ ta có $u_n = 2(2u_{n-2} + 1) + 1 = \ldots$, cho đến khi gặp u_0. Ở đây, $\mathbf{s}_0$ đóng vai trò "số hạng đầu".

Có nhiều biến thể RNN, khác nhau ở cách định nghĩa hàm $\text{RNN}(\cdot)$. Đơn giản nhất là xây dựng $\text{RNN}(\cdot)$ như một mạng nơ-ron một tầng. Giả sử $\mathbf{s}_{i-1}$ và $\mathbf{x}_i$ thuộc $\mathbb{R}^{d_h}$. Dạng của $\text{RNN}(\cdot)$ là:

$$\text{RNN}(\mathbf{s}_{i-1}, \mathbf{x}_i) \;=\; \psi(\mathbf{s}_{i-1} \cdot \mathbf{U} + \mathbf{x}_i \cdot \mathbf{V}), \tag{2.35}$$

trong đó $\mathbf{U} \in \mathbb{R}^{d_h \times d_h}$ và $\mathbf{V} \in \mathbb{R}^{d_h \times d_h}$ là các tham số. Những lựa chọn phổ biến cho hàm kích hoạt (*activation function*) ψ là $\text{TanH}(\cdot)$, $\text{Sigmoid}(\cdot)$, $\text{ReLU}(\cdot)$, v.v. Phương trình (2.35) là một mạng nơ-ron một tầng vì nó có cùng dạng với Phương trình (2.3–2.4):

$$\psi(\mathbf{s}_{i-1} \cdot \mathbf{U} + \mathbf{x}_i \cdot \mathbf{V}) \;=\; \psi([\mathbf{s}_{i-1}, \mathbf{x}_i] \cdot \mathbf{W}), \tag{2.36}$$

trong đó $[\mathbf{s}_{i-1}, \mathbf{x}_i]$ là phép ghép nối (*concatenation*) $\mathbf{s}_{i-1}$ và $\mathbf{x}_i$, còn $\mathbf{W} \in \mathbb{R}^{2d_h \times d_h}$ là ma trận tham số

được tạo thành bởi $\begin{bmatrix}\mathbf{U}\\ \mathbf{V}\end{bmatrix}$.

Chú giải của người dịch: Vì sao Phương trình (2.36) đúng?

Ghép nối nghĩa là đặt hai vectơ cạnh nhau: nếu $\mathbf{s} = [1 \;\; 2]$ và $\mathbf{x} = [3 \;\; 4]$ thì $[\mathbf{s}, \mathbf{x}] = [1 \;\; 2 \;\; 3 \;\; 4]$. Khi nhân vectơ dài $2d_h$ này với ma trận $\mathbf{W}$ gồm $\mathbf{U}$ chồng lên trên $\mathbf{V}$, các phần tử của $\mathbf{s}$ chỉ "gặp" các hàng của $\mathbf{U}$, còn các phần tử của $\mathbf{x}$ chỉ "gặp" các hàng của $\mathbf{V}$, nên kết quả đúng bằng $\mathbf{s} \cdot \mathbf{U} + \mathbf{x} \cdot \mathbf{V}$. Vậy một bước RNN đơn giản thực chất chỉ là một tầng mạng nơ-ron bình thường có đầu vào là "trạng thái cũ + dữ liệu mới". Hàm $\text{TanH}(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ nhận giá trị trong khoảng $(-1; 1)$.

Trong thực tế, RNN thường được dùng như các thành phần con trong những kiến trúc lớn hơn. Chẳng hạn, đầu vào của một đơn vị hồi quy có thể là biểu diễn trực tiếp của dữ liệu thô (như vectơ nhúng từ (*word embeddings*)) hoặc đầu ra của một mô-đun mạng nơ-ron khác. Tương tự, các tầng mạng nơ-ron bổ sung thường được xếp chồng lên trên các đơn vị hồi quy. Trong nhiều hệ thống NLP thực tế, một tầng đầu ra (*output layer*) được đặt trực tiếp trên trạng thái ẩn (*hidden state*) $\mathbf{s}_i$ để chiếu nó vào không gian đầu ra mong muốn cho các tác vụ phân loại hoặc dự đoán.

2.3.2 Đơn vị tích chập

Mạng nơ-ron tích chập (*convolutional neural networks*) (**CNN**) là một lớp mạng nơ-ron nổi tiếng khác [Waibel và cộng sự, 1989; LeCun và cộng sự, 1989]. Về mặt sinh học, chúng được lấy cảm hứng từ hệ thị giác của con người: các nơ-ron phản ứng với kích thích trong một vùng giới hạn của thị trường (gọi là **trường tiếp nhận** (*receptive field*)) [Hubel và Wiesel, 1959]. Trong CNN, trường tiếp nhận mô tả vùng trong không gian đầu vào tham gia vào việc tạo ra đầu ra của một nơ-ron. Do đó, CNN là các mô hình "kết nối một phần", trong đó mỗi nơ-ron chỉ xét các đặc trưng đầu vào nằm trong một vùng giới hạn. Điều này phân biệt CNN với các mạng nơ-ron truyền thẳng kết nối đầy đủ (*fully connected*). Nhìn chung, CNN nắm bắt một cách tự nhiên tính phân cấp của các đặc trưng (*feature*) mô tả dữ liệu và co giãn hiệu quả hơn về mặt độ phức tạp.

Mặc dù CNN có nhiều ứng dụng trong xử lý dữ liệu 2 chiều, chẳng hạn phân loại ảnh, ở đây chúng tôi bàn về chúng trong bối cảnh xử lý dữ liệu chuỗi để giữ cách tiếp cận nhất quán trong suốt chương này. Thông thường, một CNN gồm một **tầng tích chập** (*convolutional layer*), một **tầng gộp** (*pooling layer*), và có thể thêm các loại tầng khác. Nó bắt đầu bằng tầng tích chập, nơi trường tiếp nhận được định nghĩa bởi một tập các **nhân tích chập** (*convolution kernels*) hay **bộ lọc** (*filters*). Bộ lọc là một hàm ánh xạ tuyến tính, thực hiện tích chập trên các đặc trưng đầu vào trong trường tiếp nhận để tạo thành một đặc trưng đầu ra duy nhất. Ví dụ, xét một dãy số $x_1 \cdots x_m$. Bộ lọc trải từ vị trí i đến vị trí $i + r - 1$ được định nghĩa là:

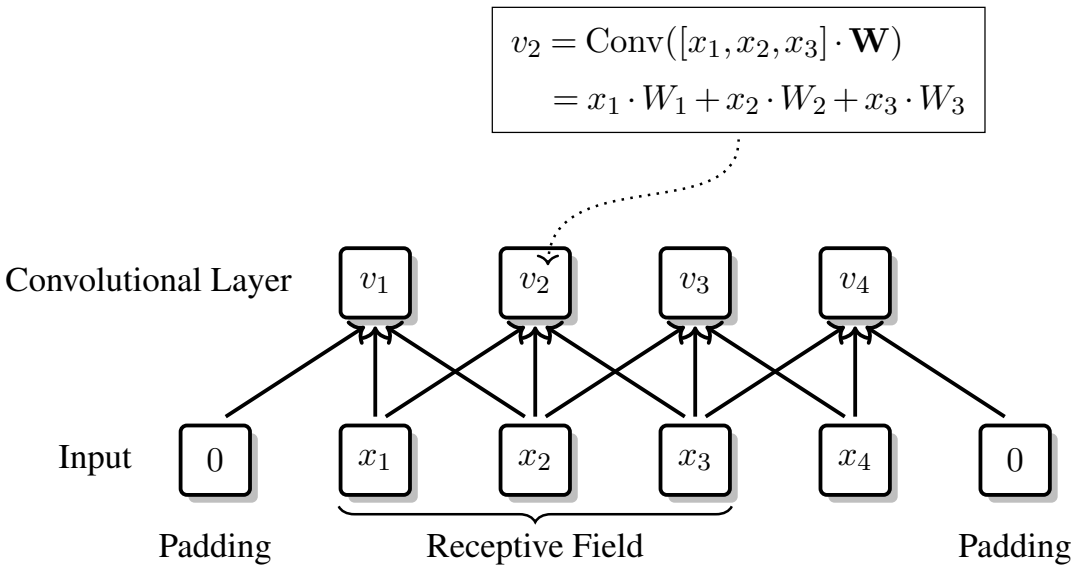
$$\begin{aligned} o &= \text{Conv}(\mathbf{x}_{[i,i+r-1]}, \mathbf{W}) \\ &= \mathbf{x}_{[i,i+r-1]} \cdot \mathbf{W} \\ &= \sum_{k=0}^{r-1} x_{i+k} \cdot W_k, \end{aligned} \tag{2.37}$$

trong đó r là kích thước của trường tiếp nhận, $\mathbf{x}_{[i,i+r-1]}$ là chuỗi con $x_i \cdots x_{i+r-1}$, và $\mathbf{W} \in \mathbb{R}^r$ là các tham số của bộ lọc. Sau đó, có thể sinh ra một chuỗi đặc trưng đầu ra bằng cách di chuyển bộ lọc dọc theo chuỗi đầu vào. Gọi stride (bước trượt (*stride*)) là khoảng cách giữa hai lần di chuyển liên tiếp. Đầu

ra của lần di chuyển thứ i khi đó được định nghĩa là:

$$v_i \;=\; \text{Conv}(\mathbf{x}_{[\text{stride}\times i,\ \text{stride}\times i+r-1]}, \mathbf{W}). \tag{2.38}$$

Bằng cách này, tầng tích chập biến chuỗi đầu vào $x_1 \cdots x_m$ thành chuỗi đầu ra $v_1 \cdots v_{\lfloor \frac{m}{\text{stride}} \rfloor}$[4]. Một lưu ý ở đây là các tham số $\mathbf{W}$ được dùng chung cho mọi vị trí của chuỗi đầu vào. Phương pháp này được gọi là **chia sẻ tham số** (*parameter sharing*) hay **chia sẻ trọng số** (*weight sharing*). Chia sẻ tham số làm CNN hiệu quả vì nó cần ít tham số hơn một mạng nơ-ron truyền thẳng có cùng số nơ-ron.



Hình 2.8: Minh họa phép tích chập trên một dãy số $\{x_1, x_2, x_3, x_4\}$ ($r = 3$ và stride $= 1$). Trường tiếp nhận xác định vùng của đầu vào được dùng để tính đầu ra. Ở đây trường tiếp nhận có kích thước 3, tức là phép tích chập bao phủ ba số liên tiếp trong chuỗi đầu vào. Bộ lọc (hay nhân tích chập) cho ra tổng có trọng số của các số này. Mỗi lần ta trượt trường tiếp nhận trên đầu vào, bộ lọc sinh ra một đầu ra mới. Như vậy, đầu ra của tầng tích chập là một vectơ các số. Ngoài ra, một số đệm (tức là 0) được thêm vào mỗi đầu của chuỗi để phép tích chập thực hiện được khi trường tiếp nhận không đầy đủ.

Nhãn trong hình: *Convolutional Layer* = tầng tích chập; *Input* = đầu vào; *Padding* = phần đệm; *Receptive Field* = trường tiếp nhận.

Một vấn đề với cách xây dựng trên là bộ lọc có thể không "lát" khít được chuỗi đầu vào. Chẳng hạn, khi stride $\times\, i + r - 1 > m$, đầu vào của bộ lọc bị thiếu. Một giải pháp thường dùng là **đệm** (*padding*). Nó đơn giản đặt các đặc trưng nằm ngoài chuỗi đầu vào bằng một hằng số. Ví dụ, ta có thể nối thêm các vectơ đặc trưng giả vào mỗi đầu của chuỗi để mọi phép tích chập đều thực hiện được. Xem Hình 2.8 để có ví dụ về một bộ lọc được tính trên một dãy số. Lưu ý rằng trường tiếp nhận của các lần áp dụng bộ lọc khác nhau có thể chồng lấn nhau. Điều này đôi khi có lợi vì nó giảm mất mát thông tin trong biểu diễn đặc trưng khi một đặc trưng cấp thấp được dùng để tạo nên nhiều đặc trưng cấp cao.

[4] $\lfloor \cdot \rfloor$ ký hiệu hàm phần nguyên (làm tròn xuống).

Chú giải của người dịch: Tính tay một phép tích chập

Lấy dãy $x = (1,\ 2,\ 3,\ 4)$, bộ lọc $\mathbf{W} = (W_0, W_1, W_2) = (1,\ 0,\ -1)$, $r = 3$, stride $= 1$, đệm một số 0 ở mỗi đầu: $(0,\ 1,\ 2,\ 3,\ 4,\ 0)$. Trượt "cửa sổ" 3 ô qua dãy đã đệm: $v_1 = 0 \cdot 1 + 1 \cdot 0 + 2 \cdot (-1) = -2$, $v_2 = 1 - 3 = -2$, $v_3 = 2 - 4 = -2$, $v_4 = 3 - 0 = 3$. Cùng ba số $(1, 0, -1)$ được dùng ở mọi vị trí — đó chính là *chia sẻ tham số*. Bộ lọc này đo "độ chênh" giữa hai bên cửa sổ; các bộ lọc khác có thể phát hiện các "mẫu" khác. Ký hiệu $\lfloor a \rfloor$ là số nguyên lớn nhất không vượt quá a, ví dụ $\lfloor 3{,}7 \rfloor = 3$.

Ngoài ra, một tầng tích chập có thể kèm theo một hàm kích hoạt $\psi(\cdot)$ để thực hiện ánh xạ phi tuyến trên đầu ra của bộ lọc. Gọi $m_k = \lfloor \frac{m}{\text{stride}} \rfloor$ là số lần áp dụng bộ lọc. Đầu ra của một tầng tích chập được cho bởi

$$\begin{bmatrix} h_1 & \cdots & h_{m_k} \end{bmatrix} = \psi\left(\begin{bmatrix} v_1 & \cdots & v_{m_k} \end{bmatrix}\right). \tag{2.39}$$

Nói chung, một tầng tích chập không nhất thiết bị giới hạn ở đầu vào là các số vô hướng hay chỉ một bộ lọc. Thường thì ta có thể lấy một vectơ làm biểu diễn của một token trong chuỗi đầu vào, và dùng một tập các bộ lọc làm bộ trích xuất đặc trưng. Để làm vậy, ta có thể áp dụng cùng cách xây dựng như trong Phương trình (2.37–2.39), nhưng thay x_i, v_i và h_i bằng các phiên bản dạng vectơ tương ứng.

Nhãn trong hình: *Filter* = bộ lọc; *Pooling* = gộp; *Features* = các đặc trưng; *Input Sequence* = chuỗi đầu vào; *padding* = phần đệm.

Một tầng tích chập thường được theo sau bởi một tầng gộp. Giống như tích chập, gộp là một hàm quét một bộ lọc trên một chuỗi. Nhưng phép gộp không có tham số nào. Thay vào đó, có thể xem nó như một hàm tổng hợp thực hiện giảm mẫu (*down-sampling*) trên chuỗi đầu vào. Có một số cách thiết kế hàm gộp. Một trong những phương pháp phổ biến nhất là **gộp cực đại** (*max pooling*), cho ra giá trị lớn nhất trong trường tiếp nhận. Một phương pháp khác là **gộp trung bình** (*average pooling*), cho ra giá trị trung bình trên trường tiếp nhận. Để có bức tranh đầy đủ về cách một CNN hoạt động, Hình 2.9 mô tả một ví dụ trong đó các phép tích chập và gộp được thực hiện trên một chuỗi vectơ đặc trưng thông qua hai bộ lọc.

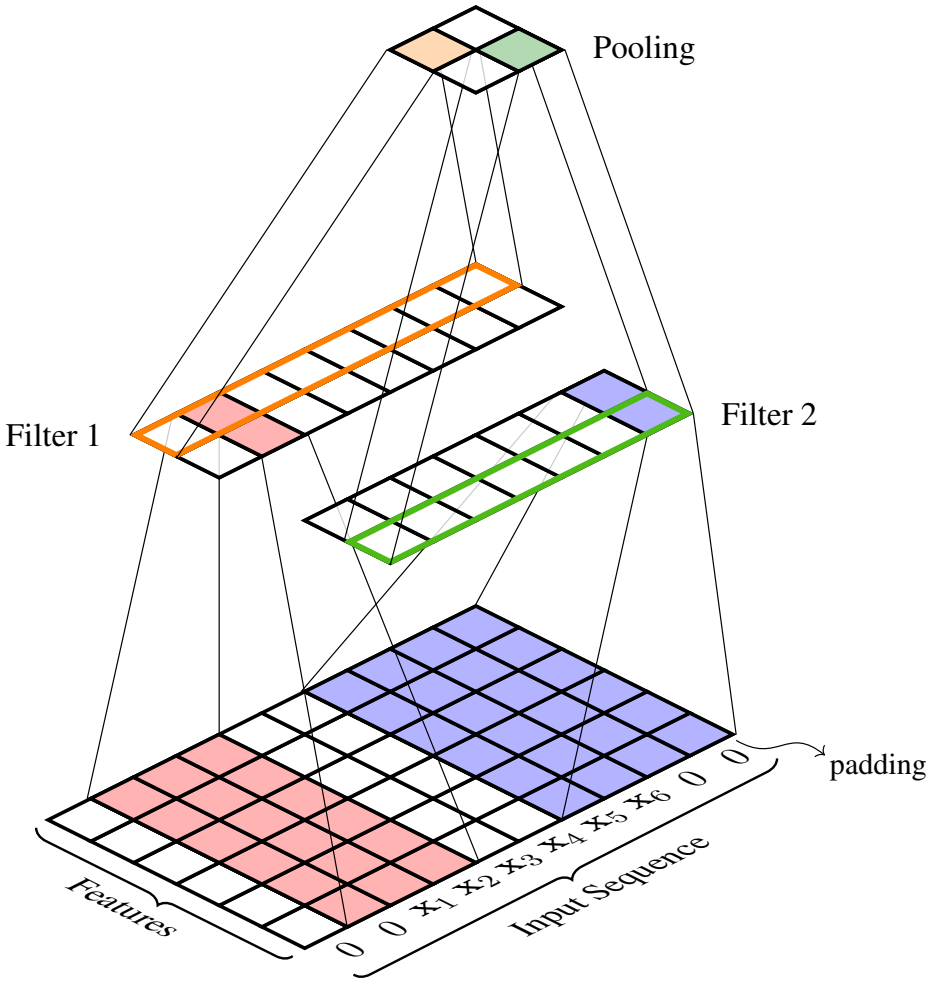
Chú giải của người dịch: Gộp cực đại và gộp trung bình

Giả sử đầu ra tầng tích chập là $(-2,\ -2,\ -2,\ 3)$. Gộp cực đại với cửa sổ 2, bước trượt 2: $\max(-2, -2) = -2$, $\max(-2, 3) = 3 \Rightarrow (-2,\ 3)$. Gộp trung bình: $(-2,\ 0{,}5)$. Chuỗi bị "rút gọn" một nửa mà vẫn giữ được thông tin nổi bật, và phép này không cần học tham số nào.

2.3.3 Đơn vị cổng

Trong mạng nơ-ron, một **cổng** (*gate*) được dùng để quyết định lượng thông tin được truyền đi [Hochreiter và Schmidhuber, 1997]. Lấy một RNN chuẩn làm ví dụ. Tại mỗi bước thời gian i, thay vì truyền trực tiếp trạng thái trước $\mathbf{s}_{i-1} \in \mathbb{R}^{d_h}$ vào đơn vị hồi quy, thường sẽ hiệu quả hơn nếu ta xác định phần nào của thông tin trong $\mathbf{s}_{i-1}$ là có liên quan cho quyết định tiếp theo. Trong bối cảnh này, ta muốn $\mathbf{s}_{i-1}$ hoạt động giống một bộ nhớ thực thụ hơn: theo thời gian, một số thông tin cần được giữ lại trong khi những thông tin khác bị loại bỏ.

Một cách phổ biến để làm được điều này là đưa vào một vectơ cổng (*gating vector*) để điều khiển độ lớn của dòng dữ liệu. Ở đây, chúng tôi dùng lại ký hiệu của RNN (xem Mục 2.3.1), nhưng mô tả của chúng tôi là tổng quát và áp dụng được cho bất kỳ kiến trúc nào cần một cơ chế như vậy. Gọi $\mathbf{z} \in [0, 1]^{d_h}$ là một vectơ cổng, trong đó $z_k = 0$ nghĩa là quên hoàn toàn ở chiều k, còn $z_k = 1$ nghĩa là giữ lại hoàn



Hình 2.9: Ví dụ về CNN. Tầng tích chập có hai bộ lọc. Đầu vào là một chuỗi 6 token được biểu diễn bằng các vectơ đặc trưng $\{\mathbf{x}_1, \ldots, \mathbf{x}_6\}$. Để các bộ lọc "lát" khít chuỗi đầu vào, hai vectơ đệm được gắn vào mỗi đầu của chuỗi. Khi áp dụng một bộ lọc, ta ánh xạ các vectơ đặc trưng trong trường tiếp nhận thành một vectơ đặc trưng mới. Ví dụ, với bộ lọc 1, trường tiếp nhận là một hình chữ nhật 6×3 trong đầu vào, và đầu ra là một vectơ đặc trưng 2 chiều. Bằng cách quét bộ lọc trên chuỗi, ta thu được một chuỗi các vectơ đặc trưng, chẳng hạn một chuỗi 8 vectơ đặc trưng, mỗi vectơ có 2 đặc trưng. Tầng gộp hợp nhất các đặc trưng dọc theo chuỗi. Ví dụ, thực hiện phép gộp trên đầu ra của bộ lọc 1 cho ra 2 đặc trưng đã hợp nhất. Đầu ra cuối cùng của CNN là hai vectơ đặc trưng 2 chiều.

toàn ở chiều k. Việc áp dụng $\mathbf{z}$ làm cổng cho $\mathbf{s}_{i-1}$ có thể được viết là:

$$\text{Gate}(\mathbf{z}, \mathbf{s}_{i-1}) \;=\; \mathbf{z} \odot \mathbf{s}_{i-1}, \tag{2.40}$$

trong đó $\odot$ ký hiệu phép nhân theo từng phần tử (*element-wise*) của hai vectơ. Phép toán $\text{Gate}(\mathbf{z}, \mathbf{s}_{i-1})$ biểu diễn một bản cập nhật đã được "lọc" của $\mathbf{s}_{i-1}$. Vectơ $\mathbf{z}$ có thể được gọi là cổng cập nhật (*update gate*), cổng quên (*forget gate*), hoặc một thành phần có chức năng tương tự. Một cách khác, hàm cổng có thể được định nghĩa là:

$$\text{Gate}(\mathbf{z}, \mathbf{s}_{i-1}) \;=\; (1 - \mathbf{z}) \odot \mathbf{s}_{i-1}. \tag{2.41}$$

Phương trình (2.40) và (2.41) mô tả cùng một cơ chế nền tảng nhưng mang những cách hiểu khác nhau về $\mathbf{z}$.

Câu hỏi trung tâm là tính $\mathbf{z}$ như thế nào. Một phương pháp chuẩn là định nghĩa $\mathbf{z}$ là đầu ra của một tầng mạng nơ-ron bên trong. Ví dụ, trong một đơn vị hồi quy, $\mathbf{z}$ có thể được định nghĩa là:

$$\mathbf{z} \;=\; \text{Sigmoid}(\mathbf{s}_{i-1} \cdot \mathbf{W}_1 + \mathbf{x}_i \cdot \mathbf{W}_2 + \mathbf{b}). \tag{2.42}$$

Việc dùng hàm kích hoạt Sigmoid bảo đảm các giá trị đầu ra nằm hẳn trong khoảng $[0, 1]$. Lưu ý rằng Phương trình (2.42) mô tả một cổng *học được*. Điều này đưa cơ chế cổng trực tiếp vào trong mô hình, cho phép nó được tối ưu bằng các quy trình huấn luyện chuẩn. Có nhiều kiến trúc cổng khác nhau, một số trong đó sẽ được tìm hiểu chi tiết ở Chương 4.

Chú giải của người dịch: Cổng như một "núm vặn âm lượng"

Nhân theo từng phần tử nghĩa là nhân các phần tử cùng vị trí: $[0{,}9 \;\; 0 \;\; 0{,}5] \odot [4 \;\; 7 \;\; 2] = [3{,}6 \;\; 0 \;\; 1]$. Mỗi z_k giống như một núm vặn âm lượng cho chiều k của bộ nhớ: $z_k = 1$ giữ nguyên, $z_k = 0$ tắt hẳn (quên), $z_k = 0{,}5$ giữ một nửa. Hàm Sigmoid $\text{Sigmoid}(a) = \frac{1}{1+e^{-a}}$ luôn cho giá trị trong khoảng $(0; 1)$, nên rất hợp để tạo ra các "núm vặn" này. Vì $\mathbf{z}$ được tính từ trạng thái cũ và dữ liệu mới bằng các tham số học được, mô hình tự học được *khi nào* nên nhớ và *khi nào* nên quên.

2.3.4 Đơn vị chuẩn hóa

Một mạng nơ-ron hoạt động bằng cách biến đổi các vectơ đặc trưng qua từng tầng. Mặc dù bản chất nhiều tầng, nhiều chiều của mạng nơ-ron cho phép mô hình tính được những hàm phức tạp, nó có thể dẫn tới những phân phối rất khác nhau của các giá trị kích hoạt đầu ra giữa các tầng hoặc giữa các đặc trưng. Đây là một vấn đề với các mạng nơ-ron sâu vì những mô hình như vậy phải thích nghi với các phân phối khác nhau ở các tầng khác nhau hoặc các đặc trưng khác nhau [Ioffe và Szegedy, 2015]. Đôi khi, do tham số mô hình được khởi tạo ngẫu nhiên ở mọi tầng và mọi chiều đặc trưng, một số đặc trưng có khả năng nhận những giá trị lớn bất thường. Trong trường hợp này, mô hình sẽ bị thiên lệch về phía những đặc trưng có độ lớn lớn hơn.

Một cách để giảm nhẹ vấn đề này là **chuẩn hóa** (*normalization*), tức là chuẩn hóa một vectơ đặc trưng n chiều $\mathbf{s}$ theo

$$\text{Normalize}(\mathbf{s}) \;=\; \boldsymbol{\alpha} \odot \frac{\mathbf{s} - \boldsymbol{\mu}}{\boldsymbol{\sigma} + \boldsymbol{\epsilon}} + \boldsymbol{\beta}, \tag{2.43}$$

trong đó $\boldsymbol{\mu} \in \mathbb{R}^n$ và $\boldsymbol{\sigma} \in \mathbb{R}^n$ lần lượt là trung bình và độ lệch chuẩn của $\mathbf{s}$. $\boldsymbol{\epsilon}$ là một số nhỏ dùng để bảo đảm ổn định số học [Chiang và Cholak, 2022]. $\boldsymbol{\alpha} \in \mathbb{R}^n$ và $\boldsymbol{\beta} \in \mathbb{R}^n$ là các tham số của đơn vị chuẩn hóa. Một lựa chọn đơn giản là $\alpha = \mathbf{1}$ và $\beta = \mathbf{0}$, còn một phương pháp tinh vi hơn là học α và β cùng với các tham số khác.

Ta có thể hiện thực Phương trình (2.43) theo nhiều cách, khác nhau ở cách định nghĩa $\boldsymbol{\mu}$ và $\boldsymbol{\sigma}$. Hãy xét điều này cho một chiều trong $\mathbf{s}$, gọi là s, trong một bối cảnh tổng quát. Giả sử s được lấy từ một tập các giá trị đặc trưng Ω_k. Trung bình và độ lệch chuẩn trên Ω_k khi đó được định nghĩa là:

$$\mu_k = \frac{1}{|\Omega_k|} \sum_{s \in \Omega_k} s, \tag{2.44}$$

$$\sigma_k = \sqrt{\frac{1}{|\Omega_k|} \sum_{s \in \Omega_k} (s - \mu_k)^2}. \tag{2.45}$$

Có một số phương pháp để định nghĩa Ω_k. Ví dụ, ta có thể định nghĩa Ω_k là các đặc trưng trong cùng một tầng [Ba và cộng sự, 2016], hoặc là các đặc trưng cùng một chiều trên các mẫu hoặc các vị trí đầu vào khác nhau [Ioffe và Szegedy, 2015; Ulyanov và cộng sự, 2016], hoặc là một cách nhóm lai giữa các chiều này [Wu và He, 2018].

Một ưu điểm của chuẩn hóa là đưa các đặc trưng về cùng một thang đo và làm cho chúng so sánh được với nhau. Điều này được thấy là rất hữu ích cho việc ổn định quá trình huấn luyện và cải thiện tính hội tụ. Như ta sẽ thấy ở các chương sau, chuẩn hóa đóng vai trò quan trọng trong nhiều hệ thống thành công.

Chú giải của người dịch: Tính tay một phép chuẩn hóa

Đây chính là "điểm chuẩn" (z-score) trong thống kê mô tả. Lấy $\mathbf{s} = (2,\ 4,\ 6)$ và chuẩn hóa trên chính các phần tử của nó (tức Ω_k là các đặc trưng trong cùng một tầng). Trung bình $\mu = \frac{2+4+6}{3} = 4$; phương sai $\frac{(2-4)^2+0^2+(6-4)^2}{3} = \frac{8}{3}$, nên $\sigma = \sqrt{8/3} \approx 1{,}63$. Với $\alpha = 1$, $\beta = 0$ và bỏ qua ϵ: Normalize$(\mathbf{s}) \approx (-1{,}22;\ 0;\ 1{,}22)$. Dù ban đầu là $(2, 4, 6)$ hay $(200, 400, 600)$, kết quả đều như nhau: các giá trị được "kéo" về quanh 0 với độ phân tán cỡ 1. Số ϵ rất nhỏ (ví dụ 10^{-6}) chỉ để tránh chia cho 0 khi mọi phần tử bằng nhau. Ký hiệu $|\Omega_k|$ là số phần tử của tập Ω_k.

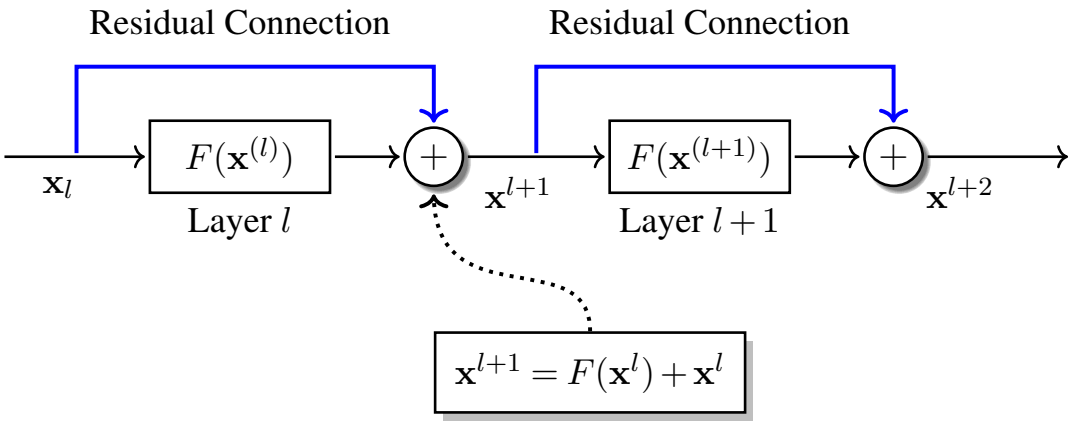
Nhân tiện nói thêm, mặc dù thuật ngữ *chuẩn hóa* (*normalization*) trong học sâu thường chỉ quá trình trừ đi trung bình và chia cho độ lệch chuẩn, thực ra đó là một quá trình **chuẩn tắc hóa** (*standardization*). Ngược lại, ở các lĩnh vực khác, chuẩn hóa thường được hiểu là một kỹ thuật co giãn mọi phần tử của một vectơ về khoảng $[0, 1]$. Chuẩn tắc hóa không có yêu cầu như vậy; thay vào đó nó có xu hướng làm cho đầu vào tập trung quanh giá trị 0. Theo nghĩa này, thuật ngữ *chuẩn hóa* trong học sâu phần nào là một cách gọi chưa chuẩn xác. Tuy vậy, trong cuốn sách này, chuẩn hóa và chuẩn tắc hóa được dùng thay thế cho nhau khi nói về các quá trình như Phương trình (2.43).

2.3.5 Đơn vị phần dư

Thành công của các mạng nơ-ron sâu phần lớn được cho là nhờ việc tăng số tầng dùng để tạo thành những hàm phức tạp hơn. Mặc dù xếp chồng thật nhiều tầng là cách đơn giản nhất để có một mô hình sâu, người ta đã chỉ ra rằng những mô hình như vậy khó huấn luyện. Có một số nguyên nhân, chẳng hạn vấn đề gradient biến mất (*vanishing gradient*) hoặc gradient bùng nổ (*exploding gradient*). Hơn nữa, việc đưa tuần tự một biểu diễn duy nhất qua mọi tầng có thể bất lợi khi các bộ trích xuất đặc trưng ở tầng trên thực ra có thể hưởng lợi từ việc truy cập trực tiếp các biểu diễn trung gian từ những tầng trước đó.

Nhãn trong hình: *Residual Connection* = kết nối tắt (residual); *Layer* = tầng.

Mạng nơ-ron phần dư (*residual neural networks*) là một trong những cách hiệu quả nhất để giải quyết các vấn đề này [He và cộng sự, 2016a]. Chúng là một loại mạng nơ-ron đặc biệt có thêm các **kết**



Hình 2.10: Minh họa một mạng nơ-ron phần dư 2 tầng. Với mỗi tầng (hoặc khối), có một kết nối bỏ qua hay kết nối tắt (màu xanh) cộng trực tiếp đầu vào vào đầu ra.

nối tắt (residual) (*residual connections*) (hay **kết nối bỏ qua** (*skip connections*), hoặc **kết nối đường tắt** (*shortcut connections*)) xuyên qua các tầng trong một chồng tầng. Gọi $F(\mathbf{x})$ là một tầng hoặc một dãy tầng ánh xạ $\mathbf{x}$ thành một đầu ra nào đó. Một khối phần dư (*residual block*) xây dựng trên $F(\mathbf{x})$ được định nghĩa bằng cách cộng ánh xạ $F(\mathbf{x})$ với ánh xạ đồng nhất (*identity mapping*) $\mathbf{x}$:

$$\mathbf{y} = F(\mathbf{x}) + \mathbf{x}. \tag{2.46}$$

Một cách dùng phổ biến hơn của kết nối tắt là trong mạng nơ-ron gồm nhiều tầng giống hệt nhau. Gọi $\mathbf{x}^l$ và $\mathbf{y}^l$ là đầu vào và đầu ra của tầng l trong một mạng nơ-ron phần dư nhiều tầng. Đầu ra của tầng l có thể được định nghĩa là:

$$\mathbf{x}^{l+1} = F(\mathbf{x}^l) + \mathbf{x}^l, \tag{2.47}$$

hoặc

$$\mathbf{y}^l = F(\mathbf{y}^{l-1}) + \mathbf{y}^{l-1}. \tag{2.48}$$

Hình 2.10 cho thấy kiến trúc của một mạng nơ-ron phần dư 2 tầng. Rõ ràng, các kết nối tắt cộng trực tiếp đầu vào của một tầng (hoặc khối) vào đầu ra của nó. Ánh xạ đồng nhất được cộng thêm này nhìn chung được xem là một trong những cách hiệu quả nhất để đơn giản hóa việc tối ưu và làm thông suốt dòng thông tin trong một mô hình sâu.

Chú giải của người dịch: Vì sao "cộng thêm $\mathbf{x}$" lại giúp ích?

Với $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$, tầng F chỉ cần học "phần chênh lệch" (*phần dư*) giữa đầu ra mong muốn và đầu vào, chứ không phải học lại toàn bộ. Nếu một tầng là thừa, mô hình chỉ cần đẩy $F(\mathbf{x})$ về gần 0 là đầu ra giữ nguyên đầu vào. Về mặt đạo hàm (một biến): nếu $y = F(x) + x$ thì $y' = F'(x) + 1$; số hạng "$+1$" giúp tín hiệu gradient đi ngược qua nhiều tầng mà không bị nhân với những số nhỏ liên tiếp rồi teo dần về 0 (hiện tượng gradient biến mất).

2.4 Huấn luyện mạng nơ-ron

Trong mục này, chúng ta chuyển sang bài toán huấn luyện, một vấn đề nền tảng trong việc phát triển các hệ thống dựa trên mạng nơ-ron. Phần lớn nội dung thảo luận ở đây tập trung vào các phương pháp

trong bối cảnh học có giám sát (*supervised learning*). Chúng ta sẽ bàn về các phương pháp không giám sát ở Mục 2.6.

2.4.1 Hạ gradient

Phương pháp gradient đã được chứng minh là một trong những phương pháp thành công nhất để huấn luyện mạng nơ-ron. Ý tưởng cơ bản là cập nhật tham số một cách lặp đi lặp lại sao cho ta có thể cực tiểu hóa một hàm mất mát (*loss function*) khả vi. Trong mỗi lần cập nhật, giá trị của các tham số được điều chỉnh theo cách làm cho mất mát giảm nhanh nhất. Về mặt toán học, điều này đòi hỏi bước cập nhật phải đi theo hướng ngược với gradient của mất mát. Cách làm này được gọi là **hạ gradient** (*gradient descent*) hay **hạ dốc nhất** (*steepest descent*). Gọi θ_t là các tham số tại bước t (gọi là **bước cập nhật** (*update step*) hay **bước huấn luyện** (*training step*)), và $\mathcal{L}(\theta_t)$ là mất mát tính bởi mô hình có tham số θ_t. **Quy tắc cập nhật** (*update rule*) (hay **quy tắc delta** (*delta rule*)) của hạ gradient được cho bởi phương trình:

$$\theta_{t+1} \;=\; \theta_t - \eta \cdot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}, \tag{2.49}$$

trong đó $\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$ là gradient của mất mát theo các tham số tại bước t. Nó có thể thu được bằng cách chạy thuật toán lan truyền ngược (*back-propagation*) đã trình bày ở Mục 2.1.3. Vì θ_t thường có nhiều chiều, $\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$ có thể là một vectơ hoặc ma trận có cùng hình dạng với θ_t. η là **tốc độ học** (*learning rate*), điều khiển độ dài bước ta đi theo hướng về điểm cực tiểu. Mặc dù có thể đặt η đơn giản là một hằng số trong suốt quá trình huấn luyện, người ta thường điều chỉnh giá trị của nó khi quá trình huấn luyện diễn ra (xem thảo luận ở Mục 2.4.4). Xem Hình 2.11 để có minh họa về hạ gradient.

Chú giải của người dịch: Hạ gradient như đi xuống thung lũng trong sương mù

Hãy tưởng tượng bạn đứng trên sườn đồi trong sương mù dày, muốn xuống đáy thung lũng. Bạn không nhìn thấy đáy, nhưng cảm nhận được độ dốc dưới chân. Cách hợp lý: bước một bước nhỏ theo hướng dốc xuống nhất, rồi lặp lại. Gradient chỉ hướng *dốc lên* nhanh nhất, nên ta đi ngược lại (dấu trừ). Ví dụ một biến: $\mathcal{L}(\theta) = \theta^2$, $\mathcal{L}'(\theta) = 2\theta$. Bắt đầu $\theta_0 = 3$, $\eta = 0{,}1$: $\theta_1 = 3 - 0{,}1 \cdot 6 = 2{,}4$; $\theta_2 = 2{,}4 - 0{,}1 \cdot 4{,}8 = 1{,}92$; … dần tiến về cực tiểu $\theta = 0$. Nếu η quá lớn (ví dụ $\eta = 1{,}1$) thì $\theta_1 = 3 - 6{,}6 = -3{,}6$, rồi $4{,}32$, … — nhảy qua lại và xa dần cực tiểu. Ký hiệu $\frac{\partial \mathcal{L}}{\partial \theta}$ (đạo hàm riêng) là đạo hàm của $\mathcal{L}$ theo θ khi coi các biến khác là hằng số; khi θ là vectơ, ta có một đạo hàm riêng cho mỗi phần tử, xếp thành vectơ gradient.

Phương trình (2.49) đưa ra một định nghĩa rất cơ bản của hạ gradient. Có nhiều cải tiến cho dạng của Phương trình (2.49). Một số trong đó là:

- **Hạ gradient với quán tính** (*Gradient Descent with Momentum*). Trong vật lý, **động lượng** (*momentum*) là một đại lượng vectơ mô tả khối lượng đang chuyển động. Nếu ta coi việc cập nhật tham số như di chuyển một vật trong không gian, thì ta cần xét động lượng của vật tại một vị trí để xác định hướng của bước di chuyển tiếp theo. Ý tưởng này có thể được hiện thực bằng cách định nghĩa lại quy tắc cập nhật như sau:

  $$\theta_{t+1} \;=\; \theta_t + v_t, \tag{2.50}$$

  trong đó v_t là vectơ vận tốc của động lượng. Trong phương pháp động lượng cổ điển [Polyak, 1964], v_t được định nghĩa là:

  $$v_t \;=\; \lambda \cdot v_{t-1} - \eta \cdot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}, \tag{2.51}$$

v_t giữ lại một phần động lượng trước đó (tức là v_{t-1}), sau đó là một hiệu chỉnh dựa trên gradient (tức là $\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$). λ là một số vô hướng dùng để đặt trọng số cho v_t trong một lần cập nhật. Một cải tiến nổi tiếng của Phương trình (2.51) là tính đến động lượng ngay trong gradient, và nhờ đó ngăn vận tốc trở nên quá lớn khi tiến gần điểm cực tiểu [Nesterov, 1983], như sau

$$v_t \;=\; \lambda \cdot v_{t-1} - \eta \cdot \frac{\partial \mathcal{L}(\theta_t + \lambda \cdot v_{t-1})}{\partial \theta_t}. \tag{2.52}$$

Có thể tìm thấy thảo luận chi tiết hơn về sự khác biệt giữa Phương trình (2.51) và Phương trình (2.52) trong bài báo của Sutskever và cộng sự [2013].

Chú giải của người dịch: Quán tính: hòn bi lăn xuống dốc

Hạ gradient thường giống một người đi từng bước, còn hạ gradient với quán tính giống một hòn bi lăn: nó "nhớ" vận tốc cũ v_{t-1} (nhân với λ, thường khoảng 0,9) và cộng thêm lực đẩy từ độ dốc hiện tại. Khi gradient liên tục chỉ cùng một hướng, vận tốc tích lũy và ta đi nhanh hơn; khi gradient đổi chiều liên tục (đường zig-zag như trong Hình 2.11), các thành phần ngược chiều triệt tiêu bớt nhau nên đường đi mượt hơn. Biến thể Nesterov (2.52) "nhìn trước": tính độ dốc tại điểm mà hòn bi *sắp* tới ($\theta_t + \lambda v_{t-1}$) chứ không phải điểm hiện tại.

Nhãn trong hình: *minimum* = điểm cực tiểu. Các đường cong khép kín là các đường mức (mọi điểm trên cùng một đường có cùng giá trị mất mát), giống như đường đồng mức trên bản đồ địa hình; các mũi tên là các bước cập nhật.

- **Hạ gradient thích nghi** (*Adaptive Gradient Descent*). Trong các phương pháp hạ gradient thích nghi, quy tắc cập nhật được điều chỉnh riêng cho từng tham số, thay vì cho toàn bộ mô hình. **AdaGrad** là một phương pháp thuộc loại này [Duchi và cộng sự, 2011]. Nó giảm tốc độ học cho các tham số được cập nhật thường xuyên và tăng tốc độ học cho các tham số ít được cập nhật. Giả sử θ_t và $\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$ đều là các vectơ d chiều. Ta có thể định nghĩa một biến mới $G \in \mathbb{R}^{d \times d}$ là tổng các tích ngoài (*outer product*) của gradient qua t bước vừa qua[5]:

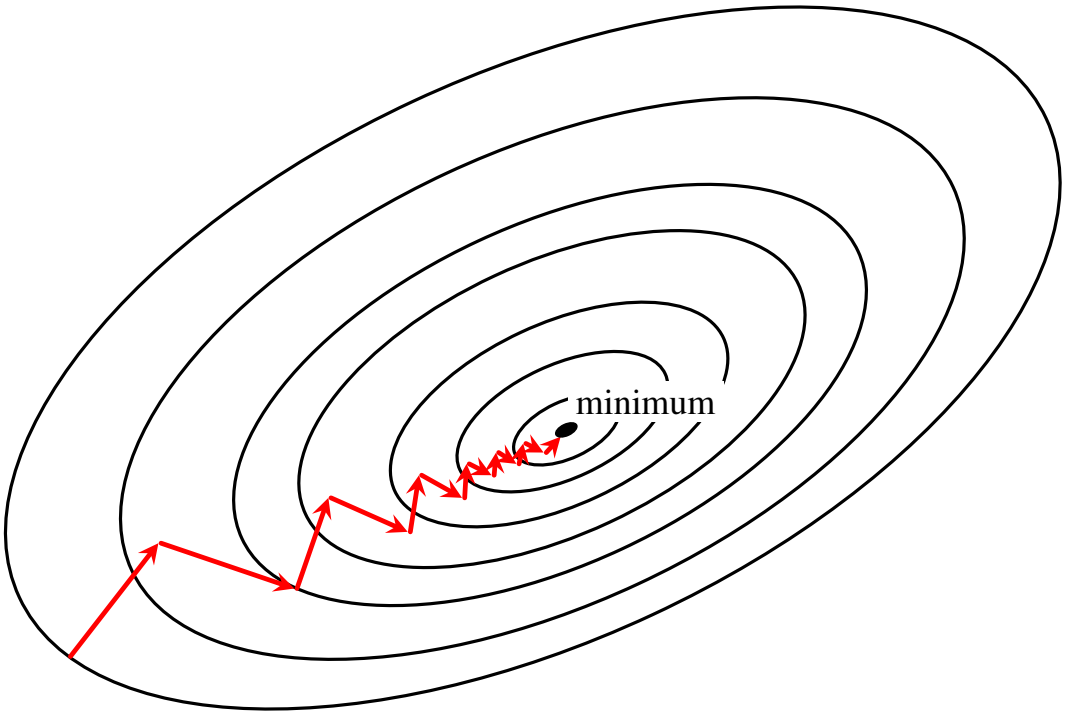
$$G_t \;=\; \sum_{i=1}^{t} \left[\frac{\partial \mathcal{L}(\theta_i)}{\partial \theta_i}\right]^\top \cdot \frac{\partial \mathcal{L}(\theta_i)}{\partial \theta_i}. \tag{2.54}$$

Nhìn chung, $(G_t)^{\frac{1}{2}} \in \mathbb{R}^{d \times d}$ có thể được xem như một chỉ báo mô tả mức độ một tham số đã được cập nhật cho đến thời điểm hiện tại. Tuy nhiên, việc tính $(G_t)^{\frac{1}{2}}$ cực kỳ tốn kém. Vì vậy, người ta thường dùng đường chéo của G_t thay thế. Khi đó, quy tắc cập nhật của AdaGrad được cho bởi:

$$\theta_{t+1} \;=\; \theta_t - \frac{\eta}{\sqrt{\text{diag}(G_t) + \epsilon}} \odot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}, \tag{2.55}$$

[5]Cho hai vectơ $\mathbf{a} = \begin{bmatrix} a_1 & \cdots & a_d \end{bmatrix}$ và $\mathbf{b} = \begin{bmatrix} b_1 & \cdots & b_d \end{bmatrix}$, tích ngoài của $\mathbf{a}$ và $\mathbf{b}$ là:

$$\begin{aligned} \mathbf{a} \otimes \mathbf{b} &= \mathbf{a}^\top \cdot \mathbf{b} \\ &= \begin{bmatrix} a_1 \\ \vdots \\ a_d \end{bmatrix} \cdot \begin{bmatrix} b_1 & \cdots & b_d \end{bmatrix} \\ &= \begin{bmatrix} a_1 b_1 & \cdots & a_1 b_d \\ \vdots & \ddots & \vdots \\ a_d b_1 & \cdots & a_d b_d \end{bmatrix}. \end{aligned} \tag{2.53}$$



Hình 2.11: Minh họa hạ gradient trong không gian 2 chiều (các đường màu đỏ biểu thị các tập mức). Mục tiêu là tìm các tham số (tức là các giá trị theo hai chiều) làm cực tiểu giá trị của một hàm mất mát cho trước. Hạ gradient làm điều này bằng cách bắt đầu từ một điểm ngẫu nhiên và bước dần tới điểm cực tiểu qua một số lần cập nhật tham số. Trong mỗi lần cập nhật, nó điều chỉnh các tham số θ_t theo hướng làm mất mát thấp hơn. Ý tưởng ở đây là bước cập nhật chọn hướng dốc nhất, tức là mô hình di chuyển một bước theo hướng gradient âm của mất mát (tức là $-\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$). Độ dài bước di chuyển được điều khiển bởi một siêu tham số η, gọi là tốc độ học. Như vậy, lượng thay đổi của các tham số là $-\eta \cdot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$. Cộng lượng này vào θ_t, ta thu được các tham số mới θ_{t+1}. Quá trình này lặp lại qua một số lần cập nhật cho đến khi giá trị của hàm mất mát gần với cực tiểu.

trong đó $\text{diag}(G_t)$ là đường chéo của G_t, tức là $\text{diag}(G_t)(k) = G_t(k,k)$. ϵ là một hệ số làm trơn để bảo đảm ổn định số học. Thay vì cộng các bình phương gradient đó một cách không có trọng số, một cách khác là giảm ảnh hưởng của các gradient "cũ" và làm cho các gradient "gần đây" quan trọng hơn. **AdaDelta** xét đến điều này bằng cách tích lũy bình phương gradient với một **hệ số suy giảm** (*decay factor*) [Zeiler, 2012]:

$$g_t^2 \;=\; \sigma \cdot g_{t-1}^2 + (1-\sigma) \cdot \left(\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t} \odot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t} \right), \tag{2.56}$$

trong đó σ là hệ số suy giảm có giá trị < 1. Giống Phương trình (2.55), quy tắc cập nhật của AdaDelta có thể thu được bằng cách thay $\text{diag}(G_t)$ bằng g_t^2:

$$\theta_{t+1} \;=\; \theta_t - \frac{\eta}{\sqrt{g_t^2 + \epsilon}} \odot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}. \tag{2.57}$$

Vì $\sqrt{g_t^2 + \epsilon}$ có thể được xem là căn quân phương (*root mean square*) (RMS) của gradient, Phương trình (2.56–2.57) còn được gọi là phương pháp **RMSProp** [Hinton, 2018].

Chú giải của người dịch: Tốc độ học "riêng" cho từng tham số

Ý tưởng của AdaGrad/RMSProp: tham số nào đã nhận gradient lớn (hoặc thường xuyên) thì chia cho một số lớn, nên bước đi nhỏ lại; tham số nào ít được cập nhật thì chia cho số nhỏ, nên bước đi lớn hơn. Ví dụ với hai tham số, nếu gradient qua hai bước là $(4,\ 0{,}1)$ rồi $(3,\ 0{,}2)$, thì tổng bình phương (đường chéo của G_2) là $(16{+}9,\ 0{,}01{+}0{,}04) = (25;\ 0{,}05)$, căn bậc hai $\approx (5;\ 0{,}22)$. Tham số thứ nhất có tốc độ học hiệu dụng $\eta/5$, tham số thứ hai $\approx \eta/0{,}22$ — lớn hơn khoảng 22 lần. Với AdaDelta/RMSProp, thay vì cộng dồn mãi, ta lấy "trung bình trượt": giá trị cũ nhân σ (ví dụ $0{,}9$), cộng 10% giá trị mới, nên các gradient quá xa trong quá khứ dần bị lãng quên.

- **Adam** (**Ước lượng mô-men thích nghi**, *Adaptive Moment Estimation*). Bộ tối ưu Adam kết hợp ưu điểm của cả phương pháp hạ gradient thích nghi và phương pháp động lượng [Kingma và Ba, 2014]. Nó định nghĩa một ước lượng cho trung bình của gradient (mô-men bậc nhất (*first moment*)) và một ước lượng cho phương sai không trừ tâm của gradient (mô-men gốc bậc hai (*second raw moment*)). Gọi m_t và v_t là hai ước lượng mô-men này. Chúng được cho bởi các phương trình:

$$m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}, \tag{2.58}$$

$$v_t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot \left(\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t} \odot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t} \right), \tag{2.59}$$

trong đó $\beta_1, \beta_2 \in [0,1]$ là các siêu tham số (*hyper-parameters*) dùng để cân bằng giữa ước lượng trước đó và gradient (hoặc bình phương gradient) tại bước hiện tại. β_1 và β_2 cũng được xem là hệ số suy giảm của các trung bình này. Ví dụ, các lựa chọn phổ biến cho β_1 và β_2 là 0.9 và 0.999. Vì các mô-men ban đầu được đặt bằng 0, các ước lượng này bị thiên lệch về phía vectơ không ở giai đoạn rất sớm của quá trình huấn luyện. Để giải quyết vấn đề này, Adam dùng hiệu chỉnh độ lệch (*bias correction*), dẫn tới các ước lượng đã hiệu chỉnh độ lệch:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \tag{2.60}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}. \tag{2.61}$$

Vì $\beta_1, \beta_2 < 1$, các hiệu chỉnh sẽ trở nên đủ nhỏ nếu thực hiện số lần cập nhật đủ lớn. Cuối cùng, quy tắc cập nhật được định nghĩa là:

$$\theta_{t+1} \;=\; \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \tag{2.62}$$

Phương trình (2.62) giống dạng tổng quát của hạ gradient, nhưng tận dụng cả phương pháp động lượng (tức là trung bình trượt (*moving average*) của các gradient trong quá khứ) và phương pháp thích nghi (tức là trung bình trượt của bình phương các gradient trong quá khứ). Trong thực tế, Adam đã trở thành một bộ tối ưu phổ biến để huấn luyện mạng nơ-ron.

Chú giải của người dịch: Vì sao cần hiệu chỉnh độ lệch trong Adam?

Với $\beta_1 = 0{,}9$ và $m_0 = 0$, giả sử gradient ở bước 1 là $g = 2$. Khi đó $m_1 = 0{,}9 \cdot 0 + 0{,}1 \cdot 2 = 0{,}2$ — nhỏ hơn nhiều so với gradient thật là 2, chỉ vì ta "khởi đầu từ 0". Chia cho $1 - \beta_1^1 = 0{,}1$ ta được $\hat{m}_1 = 2$, đúng như mong đợi. Khi t lớn, $\beta_1^t \to 0$ (ví dụ $0{,}9^{50} \approx 0{,}005$) nên phép chia gần như không còn tác dụng. Tóm lại, Adam = hướng đi lấy từ "quán tính" ($\hat{m}_t$) + độ dài bước được điều chỉnh riêng cho từng tham số ($\sqrt{\hat{v}_t}$). Lưu ý chữ v_t ở đây là mô-men bậc hai, khác với vận tốc v_t trong Phương trình (2.50–2.52).

Cải tiến hạ gradient là một lĩnh vực con sôi động của học sâu, nhưng thảo luận đầy đủ về tất cả các kỹ thuật đó nằm ngoài phạm vi của cuốn sách này. Một vài vấn đề liên quan sẽ được bàn trong phần còn lại của mục này.

Một lưu ý cuối cùng của tiểu mục này: một vấn đề thực tế cần cân nhắc khi dùng các phương pháp huấn luyện lặp là khi nào dừng huấn luyện. **Tiêu chí dừng** (*stopping criterion*) là một chủ đề chung trong tối ưu hóa. Với hạ gradient và các biến thể của nó, thực tiễn phổ biến là đặt một số bước huấn luyện tối đa hoặc một số **epoch huấn luyện** (*training epochs*)[6] tối đa, chẳng hạn 20,000 bước, hoặc 100 epoch. Một cách khác là ta có thể huấn luyện cho đến khi hội tụ. Ví dụ, ta có thể nói quá trình huấn luyện hội tụ nếu mất mát có xu hướng ổn định qua một số bước huấn luyện. Khi có sẵn một ít dữ liệu để kiểm định mô hình, một phương pháp tốt hơn có thể là kiểm tra trạng thái của mô hình trên dữ liệu kiểm định. Ví dụ, ta có thể dừng huấn luyện khi sai số dự đoán trên dữ liệu kiểm định tăng lên. Phương pháp này, được gọi là **dừng sớm** (*early stopping*), thường được dùng như một cách chính quy hóa (*regularization*). Ở Mục 2.5.3, ta sẽ thấy chi tiết hơn về cách áp dụng dừng sớm trong quá trình huấn luyện bằng một tập kiểm định (*validation set*). Về mặt thuật toán, đã có nhiều quan tâm và nghiên cứu về sự hội tụ và các cận sai số của các phương pháp học máy. Chúng tôi giới thiệu bạn đọc quan tâm một số sách giáo khoa để thảo luận thêm [Mohri và cộng sự, 2018; Kochenderfer và Wheeler, 2019].

2.4.2 Chia lô

Hàm mất mát là một khía cạnh thiết yếu trong việc huấn luyện mạng nơ-ron. Mặc dù có nhiều dạng toán học để định nghĩa hàm mất mát (xem Chương 1), ta vẫn cần quyết định quy mô các mẫu mà trên đó ta tính hàm mất mát. Có lẽ phương pháp đơn giản nhất là **hạ gradient ngẫu nhiên** (*stochastic gradient descent*) (**SGD**). Trong mỗi lần cập nhật tham số, SGD tính hàm mất mát trên một mẫu duy nhất được chọn ngẫu nhiên từ tập dữ liệu huấn luyện. Gọi $\mathcal{D}$ là một tập các mẫu huấn luyện, và $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}_{\text{gold}})$ là một mẫu được chọn ngẫu nhiên từ $\mathcal{D}$. Cho một mạng nơ-ron

$$\mathbf{y}^{(i)}_{\theta} \;=\; F_\theta\left(\mathbf{x}^{(i)}\right), \tag{2.63}$$

[6] Một epoch huấn luyện nghĩa là bộ huấn luyện duyệt qua toàn bộ tập dữ liệu huấn luyện đúng một lần.

mất mát của SGD được định nghĩa là:

$$\mathcal{L}(\theta) \;=\; \mathcal{L}\left(\mathbf{y}_\theta^{(i)}, \mathbf{y}_{\text{gold}}^{(i)}\right), \tag{2.64}$$

trong đó $\mathcal{L}(\mathbf{y}_\theta^{(i)}, \mathbf{y}_{\text{gold}}^{(i)})$ là một hàm mất mát ở cấp độ mẫu, định lượng sai số của đầu ra mô hình $\mathbf{y}_\theta^{(i)}$ so với đáp án chuẩn (*ground truth*) $\mathbf{y}_{\text{gold}}^{(i)}$.

SGD là một trong những phương pháp tối ưu quan trọng nhất trong học máy nhờ tính đơn giản của nó. Tuy nhiên, các lần cập nhật của SGD có phương sai (*variance*) cao vì một mẫu duy nhất chỉ cho một xấp xỉ nhiễu của gradient thật. Để ước lượng gradient chính xác hơn, ta có thể xét một tập các mẫu (gọi là một **lô** (*batch*)) khi tính mất mát. Phương pháp này được gọi là **chia lô** (*batching*). Gọi S là một tập các mẫu lấy từ $\mathcal{D}$. Hàm mất mát khi đó được định nghĩa trên S như sau

$$\begin{aligned}\mathcal{L}(\theta) &= \frac{1}{|S|} \sum_{(\mathbf{x}, \mathbf{y}_{\text{gold}}) \in S} \mathcal{L}(F_\theta(\mathbf{x}), \mathbf{y}_{\text{gold}}) \\ &= \frac{1}{|S|} \sum_{(\mathbf{x}, \mathbf{y}_{\text{gold}}) \in S} \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}).\end{aligned} \tag{2.65}$$

Nếu $S = \mathcal{D}$, ta có phương pháp **hạ gradient theo toàn lô** (*batch gradient descent*) (**BGD**), tức là gradient được ước lượng trên toàn bộ tập mẫu huấn luyện. Nói chung, hạ gradient theo toàn lô chính là cái mà ta thường gọi là hạ gradient. Tuy nhiên, tính mất mát trên tất cả các mẫu huấn luyện cùng một lúc rất tốn thời gian. Trong thực tế, người ta thường dùng một lô nhỏ hơn $\mathcal{D}$ rất nhiều. Cách này được gọi là **hạ gradient theo lô nhỏ** (*mini-batch gradient descent*). Nó được dùng để huấn luyện các hệ thống thực tế nhờ hiệu quả tính toán tốt và hiệu năng cao.

Chú giải của người dịch: Một mẫu, một lô nhỏ hay toàn bộ dữ liệu?

Hãy hình dung bạn muốn biết điểm trung bình môn Toán của cả trường 1000 học sinh để quyết định "điều chỉnh" phương pháp dạy. Hỏi *một* học sinh ngẫu nhiên (SGD) thì rất nhanh nhưng kết quả dao động mạnh; hỏi *cả* 1000 em (BGD) thì chính xác nhưng tốn công; hỏi một nhóm 32 em ngẫu nhiên (lô nhỏ) là cách dung hòa: khá chính xác mà vẫn nhanh. Gradient tính trên lô nhỏ chính là "trung bình mẫu" của gradient từng mẫu, như trong Phương trình (2.65).

Như một "phần thưởng" khác, chia lô thường được dùng như một cách để thực hiện tính toán dày đặc trên ma trận nhằm tăng tốc hệ thống. Giả sử S gồm m mẫu $\{(\mathbf{x}^{(i_1)}, \mathbf{y}_{\text{gold}}^{(i_1)}), \ldots, (\mathbf{x}^{(i_m)}, \mathbf{y}_{\text{gold}}^{(i_m)})\}$. Ta có thể gộp tất cả các vectơ đầu vào và các vectơ đáp án chuẩn thành các ma trận:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{(i_1)} \\ \vdots \\ \mathbf{x}^{(i_m)} \end{bmatrix}, \tag{2.66}$$

$$\mathbf{Y}_{\text{gold}} = \begin{bmatrix} \mathbf{y}_{\text{gold}}^{(i_1)} \\ \vdots \\ \mathbf{y}_{\text{gold}}^{(i_m)} \end{bmatrix}. \tag{2.67}$$

Sau đó, ta có thể chạy mạng nơ-ron trên đầu vào và đầu ra đã gộp lô, như sau:

$$\mathbf{Y}_\theta = F_\theta(\mathbf{X}). \tag{2.68}$$

Tương tự, ta có thể tính mất mát theo lô

$$\mathcal{L}(\theta) = \frac{1}{m} \cdot \mathcal{L}(\mathbf{Y}_\theta, \mathbf{Y}_{\text{gold}}), \tag{2.69}$$

trong đó $\mathcal{L}(\mathbf{Y}_\theta, \mathbf{Y}_{\text{gold}})$ vectơ hóa phép tính $\sum_{k=1}^{m} \mathcal{L}\left(\mathbf{y}_\theta^{(i_k)}, \mathbf{y}_{\text{gold}}^{(i_k)}\right)$. Phương trình (2.68–2.69) tránh việc gọi lặp lại lượt truyền xuôi (*forward pass*) và lượt truyền ngược (*backward pass*) trên từng mẫu riêng lẻ. Thay vào đó, chúng gói mọi thứ vào một lượt duy nhất qua mạng. Điều này tận dụng tốt hơn năng lực tính toán tối đa sẵn có trên các GPU hiện đại, vốn là loại thiết bị chủ yếu để chạy các hệ thống học sâu.

2.4.3 Khởi tạo tham số

Hạ gradient đòi hỏi quá trình huấn luyện bắt đầu từ một bộ tham số ban đầu nào đó. Vì hàm mục tiêu (*objective function*) huấn luyện trong một hệ thống thực tế thường là một hàm không lồi ở mức độ cao với nhiều cực tiểu địa phương (*local minima*), hiệu năng của mô hình thu được cực kỳ nhạy cảm với bước khởi tạo tham số. Ở đây, chúng tôi mô tả một số phương pháp khởi tạo tham số phổ biến nhất.

Chú giải của người dịch: Hàm lồi và cực tiểu địa phương

Hàm $y = x^2$ là hàm *lồi*: đồ thị hình "cái bát", chỉ có một đáy, nên đi xuống dốc từ đâu cũng tới cùng một điểm. Hàm như $y = x^4 - 3x^2 + x$ thì có hai "hõm" với độ sâu khác nhau: xuất phát ở bên nào sẽ rơi vào hõm bên đó. Hõm nông hơn gọi là cực tiểu địa phương. Hàm mất mát của mạng nơ-ron có vô số "hõm" như vậy trong không gian hàng triệu chiều, nên điểm xuất phát (khởi tạo) ảnh hưởng lớn đến kết quả.

- **Khởi tạo hằng số** (*Constant Initialization*). Phương pháp thứ nhất là gán cùng một giá trị cho mọi tham số (hoặc mọi phần tử của một ma trận tham số). Phương pháp này tuy đơn giản nhưng khiến mọi đơn vị ẩn được cập nhật giống hệt nhau trong quá trình huấn luyện. Kết quả là mạng không thể học được các khuôn mẫu phức tạp. Nó thường cho kết quả kém trong thực tế trừ khi có đưa vào một yếu tố ngẫu nhiên nào đó trong kiến trúc.

- **Khởi tạo theo các phân phối định trước** (*Initialization with Predefined Distributions*). Một cách tiếp cận thực tế hơn là khởi tạo ngẫu nhiên các tham số bằng các phân phối xác suất (*probability distributions*) được định trước. Phương pháp đơn giản nhất loại này là lấy giá trị tham số từ một phân phối đều hoặc phân phối Gauss, chẳng hạn gán một giá trị ngẫu nhiên trong khoảng $[-0.1, 0.1]$. Điều thú vị là cách làm theo kinh nghiệm này lại đủ tốt cho nhiều ứng dụng thực tế cơ bản.

- **Khởi tạo theo tầng** (*Layer-sensitive Initialization*). Một mở rộng của khởi tạo ngẫu nhiên cơ bản là dùng các phân phối được thiết kế riêng dựa trên số chiều của các tầng mạng nơ-ron khác nhau. Cho một tầng $\mathbf{y} = \psi(\mathbf{x} \cdot \mathbf{W} + \mathbf{B})$, gọi d_{in} và d_{out} lần lượt là số đầu vào (*fan-in*) và số đầu ra (*fan-out*) (tức là số hàng và số cột của $\mathbf{W}$). Phương pháp **khởi tạo LeCun** (*LeCun initialization*) [LeCun và cộng sự, 2012] gán một số ngẫu nhiên cho mọi tham số của $\mathbf{W}$ chỉ dựa trên số chiều đầu vào:

$$\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}} \sim U\left(-\frac{1}{\sqrt{d_{\text{in}}}}, \frac{1}{\sqrt{d_{\text{in}}}}\right), \tag{2.70}$$

  trong đó $U(-a, a)$ ký hiệu phân phối đều trên đoạn $[-a, a]$. Tương tự, số hạng độ lệch (bias) cũng có thể được khởi tạo theo cách tương tự. Dựa trên ý tưởng này, phương pháp **khởi tạo Xavier**

(*Xavier initialization*) chuẩn (còn gọi là khởi tạo Glorot) xét cả d_{in} và d_{out} để ổn định phương sai của các giá trị kích hoạt và gradient qua các tầng [Glorot và Bengio, 2010]:

$$\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}} \sim U\left(-\sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}, \sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}\right). \tag{2.71}$$

Lưu ý rằng các phân phối đều này có thể dễ dàng được thay bằng các phân phối chuẩn có trung bình 0 và phương sai $\frac{1}{d_{\text{in}}}$ (với khởi tạo LeCun) hoặc $\frac{2}{d_{\text{in}}+d_{\text{out}}}$ (với khởi tạo Xavier).

Chú giải của người dịch: Vì sao khoảng khởi tạo phụ thuộc vào d_{in}?

Mỗi đầu ra của tầng là tổng của d_{in} số hạng dạng $x_k \cdot W_{kj}$. Cộng càng nhiều số hạng ngẫu nhiên thì tổng càng "dao động" mạnh (phương sai của tổng các số ngẫu nhiên độc lập bằng tổng các phương sai). Nếu không thu nhỏ trọng số, sau vài chục tầng các giá trị có thể phình to hoặc teo nhỏ theo cấp số nhân. Chọn khoảng tỉ lệ với $1/\sqrt{d_{\text{in}}}$ giữ cho độ lớn đầu ra cỡ tương đương độ lớn đầu vào. Ví dụ, với $d_{\text{in}} = 100$ và $d_{\text{out}} = 200$: LeCun lấy trọng số trong $[-0{,}1;\ 0{,}1]$, còn Xavier lấy trong $[-\sqrt{6/300};\ \sqrt{6/300}] \approx [-0{,}14;\ 0{,}14]$. Ký hiệu "$\sim$" đọc là "được lấy mẫu từ phân phối".

Nhiều phương pháp khởi tạo tham số được thiết kế cho những loại mạng nơ-ron nhất định. Ví dụ, khởi tạo Xavier được suy ra với giả định dùng các hàm kích hoạt đối xứng, như Sigmoid và tang hyperbolic. Với ReLU, có thể tham khảo công trình của He và cộng sự [2015]. Một ví dụ khác là khởi tạo cho các mạng nơ-ron sâu. Người ta thấy rằng khởi tạo phù hợp là yếu tố then chốt cho thành công của những mô hình cực sâu trong NLP. Xét độ sâu của mô hình như một yếu tố bổ sung khi khởi tạo, ta có thể sửa Phương trình (2.71) thành:

$$\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}} \sim U\left(-\frac{\alpha_s}{l} \cdot \sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}, \frac{\alpha_s}{l} \cdot \sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}\right), \tag{2.72}$$

trong đó l là độ sâu của một tầng, và α_s là một siêu tham số. Ngoài ra, một số phương pháp đã được đề xuất để giải quyết việc khởi tạo cho mạng nơ-ron sâu, bao gồm khởi tạo Lipschitz [Xu và cộng sự, 2020], khởi tạo T-Fixup [Huang và cộng sự, 2020a], khởi tạo Admin [Liu và cộng sự, 2020c], v.v.

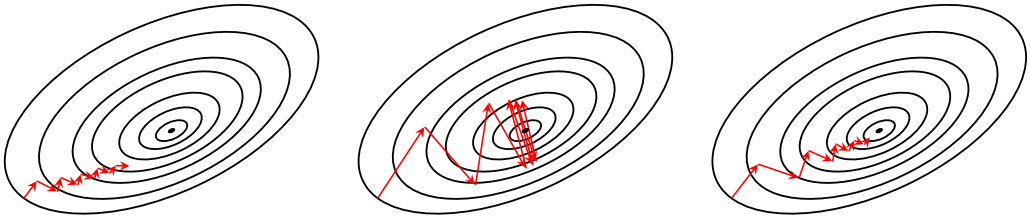
Lưu ý cuối cùng, trong thực tế, ta không nhất thiết giới hạn việc huấn luyện ở một điểm xuất phát duy nhất. Người ta thường thực hiện nhiều lần thử với các phương pháp khởi tạo hoặc hạt giống ngẫu nhiên (*random seeds*) khác nhau, rồi chọn mô hình tốt nhất trong các lần thử đó. Chiến lược này đặc biệt hữu ích khi "địa hình" tối ưu có rất nhiều cực tiểu địa phương.

2.4.4 Lập lịch tốc độ học

Để đạt được kết quả mong muốn, cần cấu hình cẩn thận tốc độ học trong suốt quá trình huấn luyện. Mặc dù một số quy tắc cập nhật nêu trên đã xét đến việc co giãn gradient cho các tham số khác nhau, **lập lịch tốc độ học** (*learning rate scheduling*) theo truyền thống tập trung nhiều hơn vào việc thiết kế các quy tắc kinh nghiệm để điều chỉnh η qua các bước huấn luyện. Trong thực tế, tốc độ học quá lớn thường dẫn tới việc "vượt quá" quanh điểm cực tiểu, trong khi tốc độ học quá nhỏ dẫn tới hội tụ chậm một cách khổ sở (xem Hình 2.12). Một chiến lược phổ biến là bước những bước lớn lúc đầu (tức là dùng tốc độ học lớn) và những bước nhỏ hơn khi mất mát tiến gần cực tiểu. Dưới đây, chúng tôi trình bày một số phương pháp lập lịch tốc độ học phổ biến.

Nhãn trong hình: (a) tốc độ học nhỏ; (b) tốc độ học lớn; (c) tốc độ học hợp lý.

- **Tốc độ học cố định** (*Fixed Learning Rates*). Cố định tốc độ học nhìn chung là một chiến lược



(a) A small learning rate (b) A large learning rate (c) A desirable learning rate

Hình 2.12: Minh họa việc học với các tốc độ học khác nhau. Tốc độ học nhỏ (trái) giúp ta bước về phía cực tiểu một cách chính xác, nhưng cần nhiều thời gian hơn đáng kể để hội tụ. Tốc độ học lớn (giữa) dẫn tới những cập nhật ban đầu nhanh, điều này rất có lợi khi còn ở xa cực tiểu. Tuy nhiên, khi tiến gần tới cực tiểu, tốc độ học quá lớn gây ra hiện tượng vượt quá. Một chiến lược đáng mong muốn hơn (phải) là cập nhật mô hình nhanh khi còn xa sự hội tụ, và làm chậm các cập nhật khi gần hội tụ.

chưa tối ưu, nhưng có thể được dùng khi làm nguyên mẫu hệ thống, chẳng hạn khi thử nhanh một phương pháp mới bằng cách chỉ huấn luyện vài epoch.

- **Suy giảm tốc độ học** (*Learning Rate Decay*). Suy giảm là một kỹ thuật thường dùng để lập lịch tốc độ học. Có nhiều cách hiện thực ý tưởng này. Ví dụ, ta có thể giảm một nửa tốc độ học sau mỗi epoch huấn luyện. Ở đây ta dùng n_t để biểu diễn bước huấn luyện hiện tại, và n_{decay} để biểu diễn tần suất thay đổi tốc độ học (ví dụ 100 bước). Bảng 2.2 trình bày một số hàm suy giảm dùng cho lập lịch tốc độ học.

- **Khởi động và suy giảm** (*Warmup and Decay*). Như đã nói ở trên, người ta thường khởi tạo tham số mô hình bằng các giá trị ngẫu nhiên trước khi huấn luyện một mạng nơ-ron. Tuy nhiên, bắt đầu từ con số không với một tốc độ học lớn thường không được khuyến khích, vì gradient ở các giai đoạn đầu của huấn luyện rất nhiễu và trạng thái của mô hình chưa ổn định. Vì vậy, hợp lý hơn là bắt đầu với một tốc độ học nhỏ và tăng dần nó lên. Sau đó, khi mô hình đã được huấn luyện một thời gian, tốc độ học bắt đầu suy giảm như thường lệ. Trực giác này là động lực cho phương pháp khởi động (*warmup*) và suy giảm. Một cách xây dựng phổ biến của phương pháp này được Vaswani và cộng sự [2017] đề xuất như sau:

$$\eta_{n_t} \;=\; \eta_0 \cdot \min\left(\left(\frac{n_t}{n_{\text{decay}}}\right)^{-0.5},\; \frac{n_t}{n_{\text{decay}}} \cdot (n_{\text{warmup}})^{-1.5}\right), \tag{2.73}$$

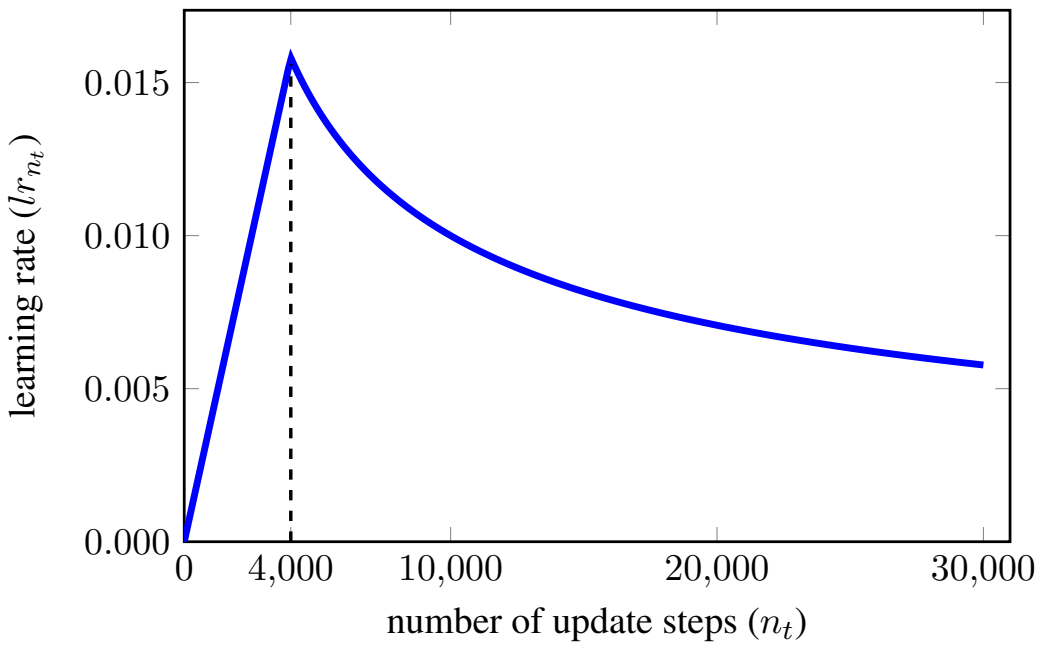
trong đó η_0 là tốc độ học ban đầu, và n_{warmup} là một siêu tham số chỉ định số bước của giai đoạn khởi động. Hình 2.13 vẽ đường cong của Phương trình (2.73) với n_{warmup}, n_{decay} và η_0 lần lượt được đặt là 4,000, 1 và 1. Ta thấy tốc độ học tăng tuyến tính trong n_{warmup} bước đầu tiên rồi suy giảm theo hàm nghịch đảo căn bậc hai.

Ghi chú: "(bậc thang)" dịch từ *(Drop)* trong bản gốc, chỉ các biến thể mà tốc độ học giảm theo từng nấc nhờ phép lấy phần nguyên $\lfloor \cdot \rfloor$.

Nhãn trong hình: trục tung *learning rate* = tốc độ học; trục hoành *number of update steps* = số bước cập nhật.

Bảng 2.2: Các hàm suy giảm. λ = tốc độ suy giảm, η_0 = tốc độ học ban đầu, và $\{\beta_i\}$, $\{\gamma_i\}$ và α = các siêu tham số khác.

Tên	Công thức	Siêu tham số
Suy giảm hằng số từng đoạn	$\eta_{n_t} = \beta_i$	các giá trị $\{\beta_1, \ldots, \beta_m\}$
	nếu $\gamma_i \leq \frac{n_t}{n_{\text{decay}}} < \gamma_{i+1}$	các ngưỡng $\{\gamma_1, \ldots, \gamma_m\}$
Suy giảm mũ	$\eta_{n_t} = \eta_0 \cdot \lambda^{\frac{n_t}{n_{\text{decay}}}}$	tốc độ suy giảm λ, tốc độ ban đầu η_0
Suy giảm mũ (bậc thang)	$\eta_{n_t} = \eta_0 \cdot \lambda^{\lfloor \frac{n_t}{n_{\text{decay}}} \rfloor}$	tốc độ suy giảm λ, tốc độ ban đầu η_0
Suy giảm mũ tự nhiên	$\eta_{n_t} = \eta_0 \cdot \exp(-\lambda \cdot \frac{n_t}{n_{\text{decay}}})$	tốc độ suy giảm λ, tốc độ ban đầu η_0
Suy giảm nghịch đảo thời gian	$\eta_{n_t} = \eta_0 \cdot \frac{1}{1+\lambda \cdot \frac{n_t}{n_{\text{decay}}}}$	tốc độ suy giảm λ, tốc độ ban đầu η_0
Suy giảm nghịch đảo thời gian (bậc thang)	$\eta_{n_t} = \eta_0 \cdot \frac{1}{1+\lambda \cdot \lfloor \frac{n_t}{n_{\text{decay}}} \rfloor}$	tốc độ suy giảm λ
Suy giảm cosin	$\eta_{n_t} = \eta_0 \cdot \big((1-\alpha) \cdot c_{\text{decay}} + \alpha\big)$	hệ số α
	$c_{\text{decay}} = \frac{1}{2} \cdot \big(1 + \cos(\pi \cdot \frac{n_t}{n_{\text{decay}}})\big)$	tốc độ ban đầu η_0



Hình 2.13: Lập lịch tốc độ học: khởi động rồi suy giảm ($n_{\text{warmup}} = 4,000$, $n_{\text{decay}} = 1$ và $\eta_0 = 1$). Tốc độ học tăng tuyến tính theo n_t trong 4,000 bước đầu tiên. Sau đó, tốc độ học đi theo một hàm nghịch đảo căn bậc hai và suy giảm khi quá trình huấn luyện tiếp diễn. Sự thay đổi của tốc độ học sẽ nhỏ nếu n_t đủ lớn, thể hiện việc tinh chỉnh nhẹ các tham số khi ta tiến gần cực tiểu của mất mát.

Chú giải của người dịch: Tính thử Phương trình (2.73)

Với $n_{\text{warmup}} = 4000$, $n_{\text{decay}} = 1$, $\eta_0 = 1$, ta có $\eta_{n_t} = \min\left(n_t^{-0,5},\ n_t \cdot 4000^{-1,5}\right)$.

- Tại $n_t = 1000$: $1000^{-0,5} \approx 0{,}0316$, còn $1000 \cdot 4000^{-1,5} \approx 0{,}0040 \Rightarrow$ lấy 0,0040 (đang ở pha tăng tuyến tính).
- Tại $n_t = 4000$: hai số bằng nhau, cùng bằng $4000^{-0,5} \approx 0{,}0158$ — đây là đỉnh trong Hình 2.13.
- Tại $n_t = 16000$: $16000^{-0,5} \approx 0{,}0079$, nhỏ hơn số thứ hai $\approx 0{,}0632 \Rightarrow$ lấy 0,0079 (pha suy giảm).

Hàm min tự động "chuyển pha" tại $n_t = n_{\text{warmup}}$. Lưu ý: $a^{-0,5} = \frac{1}{\sqrt{a}}$ và $a^{-1,5} = \frac{1}{a\sqrt{a}}$.

Chọn một chiến lược lập lịch tốc độ học phù hợp là một bài toán mang tính thực nghiệm cao, và không có lựa chọn nào tối ưu cho mọi trường hợp. Bài toán thậm chí còn phức tạp hơn khi xét đến mối tương quan giữa tốc độ học và các yếu tố động khác của quá trình huấn luyện, mặc dù lập lịch tốc độ học thường được coi là một nhiệm vụ riêng biệt. Ví dụ, khi dùng kích thước lô lớn hơn trong huấn luyện, thường cần một tốc độ học lớn hơn để đạt kết quả tốt [Ott và cộng sự, 2018b; Smith và cộng sự, 2018]. Do đó, chọn tốc độ học tối ưu vẫn là việc khó và tốn thời gian trong các ứng dụng mạng nơ-ron. Đôi khi, ta phải dựa vào rất nhiều lần chạy thử-và-sai để tìm được một thiết lập tốc độ học phù hợp cho bài toán cụ thể đang xét.

2.5 Các phương pháp chính quy hóa

Bây giờ chúng ta bàn về các phương pháp chính quy hóa (*regularization*) nhằm ngăn chặn hiện tượng quá khớp (*overfitting*). Chính quy hóa là một chủ đề rất rộng trong học máy; ở đây chúng tôi chỉ trình bày một số phương pháp thường được dùng khi huấn luyện mạng nơ-ron.

2.5.1 Phạt dựa trên chuẩn

Một trong những phương pháp phổ biến nhất là dùng một số hạng chính quy hóa dựa trên chuẩn l_p (l_p *norm*). Dạng tổng quát của hàm mục tiêu có chính quy hóa có thể được định nghĩa như sau:

$$\hat{\theta} \ = \ \arg\min_{\theta} \mathcal{L}(\theta) + \alpha \cdot R(\theta), \tag{2.74}$$

trong đó $R(\theta)$ là số hạng chính quy hóa (*regularization term*), được nhân với hệ số trọng số α. Nói chung, $R(\theta)$ đóng vai trò như một khoản mất mát bổ sung nhằm phạt các mô hình phức tạp. Động cơ của cách làm này là: các mô hình phức tạp dễ quá khớp dữ liệu hơn (xem Chương 1). Để phạt độ phức tạp của mô hình, một cách trực tiếp là định nghĩa $R(\theta)$ là chuẩn l_1 của vectơ tham số θ. Hàm phạt dựa trên chuẩn l_1 có dạng

$$R(\theta) \ = \ \sum_i |\theta_i|. \tag{2.75}$$

Phương trình (2.75) phạt những mô hình có giá trị tham số rất lớn. Có thể hiểu điều này một cách trực quan qua phép so sánh với hàm đa thức: hệ số của các biến trong một hàm đa thức càng lớn thì đường cong càng phức tạp. Thông thường, chính quy hóa với chuẩn l_1 được gọi là chính quy hóa l_1 hay **chính quy hóa Lasso** (*Lasso regularization*). Phương pháp này không đòi hỏi thay đổi căn bản nào trong thuật toán tối ưu và có thể cài đặt bằng hạ gradient tiêu chuẩn. Thú vị hơn, chính quy hóa l_1 thường cho ra các nghiệm thưa (*sparse*) đối với hàm mục tiêu huấn luyện ban đầu. Nó buộc nhiều tham số phải bằng đúng 0, chỉ để lại một tập nhỏ các đặc trưng quan trọng có trọng số khác 0. Điều này cũng

hàm ý một khả năng chọn lọc đặc trưng (*feature selection*) có sẵn, vì tham số của những đặc trưng kém quan trọng bị ép về 0.

Một lựa chọn thay thế cho chính quy hóa l_1 là **chính quy hóa** l_2 (l_2 *regularization*), còn được biết đến rộng rãi với tên **chính quy hóa Ridge** (*Ridge regularization*). Trong chính quy hóa l_2, số hạng chính quy hóa là bình phương chuẩn l_2 của các tham số

$$R(\theta) = \sum_i \theta_i^2. \tag{2.76}$$

Giống như chuẩn l_1, chuẩn l_2 phạt những vectơ tham số lệch xa gốc tọa độ. Tuy nhiên, nó hơi khác chuẩn l_1 ở chỗ chuẩn l_2 buộc mọi tham số đều có giá trị nhỏ (nhưng không nhất thiết bằng 0) và không có tham số nào có giá trị lớn. Theo nghĩa này, việc dùng chuẩn l_2 không đưa tính thưa chặt chẽ vào nghiệm, mà đúng hơn là tạo ra hiệu ứng "làm trơn" lên phân bố của các đặc trưng. Vì vậy, đôi khi nó được gọi là **suy giảm trọng số** (*weight decay*) để nhấn mạnh khả năng liên tục thu nhỏ giá trị tham số trong các bước cập nhật gradient.

Chú giải của người dịch: Chính quy hóa l_1 và l_2 qua ví dụ số

Phương trình (2.74) giống như một "bài thi có trừ điểm": ta muốn mất mát $\mathcal{L}(\theta)$ nhỏ, nhưng mỗi khi tham số "phình to" thì bị cộng thêm khoản phạt $\alpha \cdot R(\theta)$. Hệ số α là mức nghiêm khắc của khoản phạt: $\alpha = 0$ là không phạt, α càng lớn thì mô hình càng bị ép giữ tham số nhỏ.
Ví dụ với $\theta = (3, -0.5, 0)$: chuẩn l_1 cho $R = |3| + |-0.5| + |0| = 3.5$; bình phương chuẩn l_2 cho $R = 9 + 0.25 + 0 = 9.25$. Để ý rằng l_2 phạt tham số lớn (số 3) rất nặng (thành 9) nhưng phạt tham số nhỏ (0.5) rất nhẹ (thành 0.25). Vì thế l_2 thích "nhiều tham số nhỏ" hơn là "vài tham số lớn", nhưng ít có động lực đẩy một tham số vốn đã nhỏ về đúng 0. Ngược lại, với l_1, giảm một tham số từ 0.5 về 0 vẫn bớt được 0.5 tiền phạt, nên nhiều tham số bị đẩy hẳn về 0 — đó là nguồn gốc của tính "thưa".
Về tên gọi "suy giảm trọng số": đạo hàm của $\alpha\theta_i^2$ theo θ_i là $2\alpha\theta_i$, nên mỗi bước hạ gradient trừ bớt một phần tỉ lệ với chính θ_i, tức là nhân tham số với một số hơi nhỏ hơn 1 — tham số cứ thế "co lại" dần.

Theo một nghĩa rộng hơn trong học máy, Phương trình (2.74) cung cấp một khuôn khổ tổng quát để đưa tri thức tiên nghiệm (*prior knowledge*) vào quá trình huấn luyện mạng nơ-ron. Có rất nhiều cách thiết kế số hạng chính quy hóa, và giảm quá khớp chỉ là một trong những mục tiêu của các thiết kế đó. Chúng ta sẽ gặp nhiều ứng dụng khác nhau của cách tiếp cận này trong NLP, và sẽ tìm hiểu một vài ví dụ trong các chương còn lại của cuốn sách.

2.5.2 Dropout

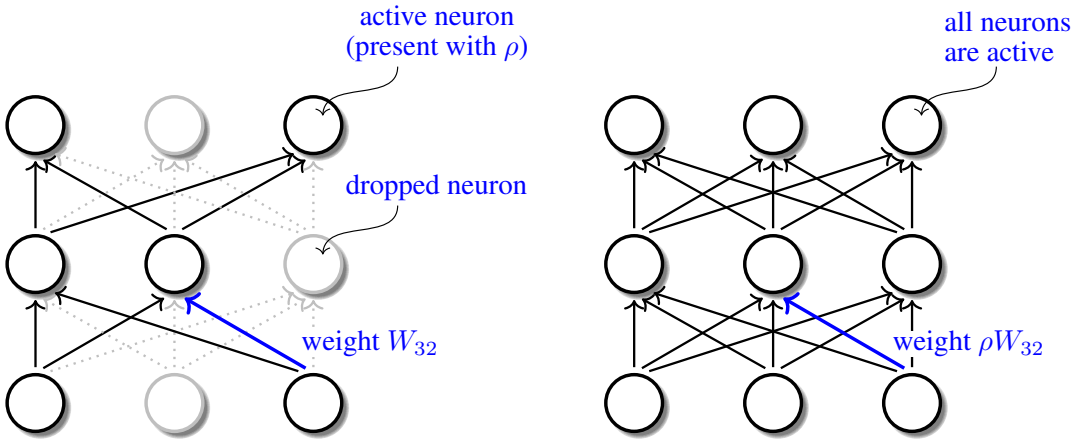
Trong một mạng nơ-ron thực tế, một tầng thường gồm hàng trăm hoặc hàng nghìn nơ-ron và tạo ra một vectơ đặc trưng tương ứng. Mặc dù mỗi đặc trưng được tính bởi một nơ-ron riêng, chúng cùng nhau tạo thành đầu vào cho từng nơ-ron của tầng tiếp theo. Do đó, một đặc trưng bị buộc phải hợp tác với các đặc trưng khác. Điều này giống như một nhóm người ngồi cùng nhau và đưa ra một quyết định tập thể. Dù mỗi thành viên có thể có ý kiến độc lập, đôi khi họ cố gắng sửa những lỗi do phần còn lại của nhóm gây ra. Trong trường hợp này, mỗi thành viên trong nhóm đều *đồng thích nghi* với những người khác [Hinton và cộng sự, 2012]. Từ góc độ thiết kế đặc trưng, sự **đồng thích nghi** (*co-adaptation*) giữa các nơ-ron có ích khi mô hình hóa các bài toán phức tạp, vì nó ngầm tạo ra một dạng đặc trưng bậc cao nào đó. Ngoài ra, thông tin giám sát mạnh (ví dụ, lan truyền sai số qua các tầng) có thể củng cố sự đồng thích nghi trong quá trình huấn luyện. Điều này phần nào giải thích vì sao một mạng nơ-ron với số lượng lớn nơ-ron có thể khớp những đường cong phức tạp. Tuy nhiên, tại thời điểm kiểm tra, sự đồng thích nghi lại cản trở khả năng tổng quát hóa. Vì mọi nơ-ron của một tầng đã học cách phối hợp thật ăn

ý trên dữ liệu huấn luyện, một thay đổi nhỏ ở đầu vào có thể ảnh hưởng tới tất cả các nơ-ron này và dẫn đến một thay đổi lớn trong hành vi của mạng nơ-ron.

Một cách để giảm bớt hoặc loại bỏ những sự đồng thích nghi phức tạp là cho mỗi nơ-ron học cách dự đoán khi vắng mặt các nơ-ron khác. Để làm vậy, ta chỉ cần bỏ đi một số nơ-ron trong lúc huấn luyện. Phương pháp này được gọi là **dropout** (*dropout*) [Srivastava và cộng sự, 2014]. Gọi n là số nơ-ron của một tầng. Cho trước một xác suất ρ (gọi $1-\rho$ là **tỉ lệ dropout** (*dropout rate*)), ta có thể sinh ra một vectơ mặt nạ (*mask vector*) n chiều $\mathbf{M}_{\text{drop}}$, trong đó mỗi phần tử được đặt bằng 1 với xác suất ρ và bằng 0 với xác suất $1-\rho$. Khi đó, một tầng dropout có thể được định nghĩa là

$$\mathbf{y} \;=\; \mathbf{M}_{\text{drop}} \odot \psi(\mathbf{x}\cdot\mathbf{W}+\mathbf{B}), \tag{2.77}$$

trong đó $\psi(\mathbf{x}\cdot\mathbf{W}+\mathbf{B})$ là một mạng nơ-ron một tầng thông thường. Phương trình (2.77) chỉ kích hoạt những nơ-ron có mặt nạ bằng 1. Với các nơ-ron bị bỏ, mọi kết nối đi tới hoặc đi ra từ chúng đều bị chặn (xem Hình 2.14 (a)). Trong quá trình huấn luyện, $\mathbf{M}_{\text{drop}}$ được sinh ngẫu nhiên ở mỗi lần gọi lượt truyền xuôi và lượt truyền ngược. Nhờ đó, mỗi lần một nơ-ron có thể học cách làm việc với những nơ-ron khác nhau và sẽ không thích nghi với cùng một nhóm "đồng nghiệp" cố định.



(a) Training the dropout network (b) Testing with the dropout network

Hình 2.14: Minh họa dropout cho một mạng nơ-ron nhiều tầng (huấn luyện so với kiểm tra). Khi huấn luyện, mỗi nơ-ron bị bỏ ngẫu nhiên với xác suất $1-\rho$, tạo ra một mạng "gầy" hơn. Theo nghĩa này, huấn luyện dropout về bản chất là quá trình học một số lượng lớn theo cấp số mũ các mạng con. Khi kiểm tra, mạng đầy đủ được dùng như bình thường, đó là kết quả của việc kết hợp tất cả các mạng con ấy để dự đoán. Vì mọi kết nối giữa các nơ-ron chỉ được kích hoạt với xác suất ρ trong lúc huấn luyện, các trọng số của mạng suy luận được thu nhỏ theo hệ số ρ.

Nhãn trong hình: (a) huấn luyện mạng dropout — *active neuron (present with ρ)*: nơ-ron hoạt động (có mặt với xác suất ρ); *dropped neuron*: nơ-ron bị bỏ; *weight* W_{32}: trọng số W_{32}. (b) kiểm tra với mạng dropout — *all neurons are active*: mọi nơ-ron đều hoạt động; *weight* ρW_{32}: trọng số ρW_{32}.

Một cách khác để hiểu dropout là xem nó như việc học các mô hình con của một mô hình "lớn". Việc dùng Phương trình (2.77) về bản chất là một quá trình lấy mẫu, trích ra một mạng con từ mạng ban đầu. Vì vậy, huấn luyện với dropout giống như huấn luyện một tổ hợp mô hình (*ensemble*) gồm số lượng lớn theo cấp số mũ các mạng con[7]. Mặt khác, việc huấn luyện vẫn hiệu quả vì các mạng con này dùng chung tham số cho cùng một nơ-ron, và việc cập nhật một tham số có thể mang lại lợi ích cho số lượng lớn theo cấp số mũ các mạng con.

[7] Với một mạng một tầng có n nơ-ron, có 2^n mạng con khả dĩ.

Chú giải của người dịch: Mặt nạ dropout hoạt động thế nào?

Ký hiệu $\odot$ là phép nhân *theo từng phần tử*: nhân hai vectơ cùng độ dài, vị trí nào nhân với vị trí đó. Giả sử một tầng có 4 nơ-ron cho đầu ra $(0.8,\ 1.2,\ 0.3,\ 2.0)$ và $\rho = 0.5$. Ở một bước huấn luyện, ta "tung đồng xu" cho từng nơ-ron và được mặt nạ $\mathbf{M}_{\text{drop}} = (1,\ 0,\ 1,\ 0)$; khi đó $\mathbf{y} = (0.8,\ 0,\ 0.3,\ 0)$: nơ-ron thứ 2 và thứ 4 coi như "nghỉ học" ở bước này. Bước sau, mặt nạ được tung lại, chẳng hạn $(0, 1, 1, 1)$, và một nhóm khác lại nghỉ. Vì không bao giờ chắc đồng đội nào có mặt, mỗi nơ-ron phải tự học những đặc trưng có ích, thay vì dựa dẫm vào vài đồng đội quen thuộc. Với 4 nơ-ron, có $2^4 = 16$ mặt nạ khả dĩ, tức 16 mạng con khác nhau dùng chung bộ trọng số.

Khi kiểm tra, tất cả các mạng con này được kết hợp lại để dự đoán. Trong trường hợp này, ta không cần bỏ nơ-ron nào mà dùng mạng ban đầu như bình thường. Điều này khiến dropout rất dễ cài đặt: trên dữ liệu huấn luyện, một nơ-ron có mặt với một xác suất nào đó, còn trên dữ liệu kiểm tra, tất cả các nơ-ron đều có mặt và cùng nhau làm việc. Vì các kết nối giữa các nơ-ron chỉ tham gia với xác suất ρ trong lúc huấn luyện, các trọng số đã học được thu nhỏ theo hệ số ρ trong mạng suy luận, tức là một tầng có dạng:

$$\mathbf{y} \ = \ \psi(\mathbf{x} \cdot \rho\mathbf{W} + \rho\mathbf{B}), \tag{2.78}$$

trong đó các trọng số $\mathbf{W}$ và $\mathbf{B}$ được nhân với hệ số ρ[8].

Xem Hình 2.14 để so sánh việc huấn luyện và áp dụng một mạng dropout. Phương trình (2.78) đòi hỏi phải cập nhật hệ thống suy luận. Một cách khác là chỉ xử lý vấn đề co giãn trong quá trình huấn luyện và giữ nguyên hệ thống suy luận. Ví dụ, ta có thể phóng to mọi tham số theo hệ số $\frac{1}{\rho}$ khi huấn luyện dropout, như sau

$$\mathbf{y} \ = \ \mathbf{M}_{\text{drop}} \odot \psi\Big(\mathbf{x} \cdot \frac{1}{\rho}\mathbf{W} + \frac{1}{\rho}\mathbf{B}\Big). \tag{2.79}$$

Vì nhân $\frac{1}{\rho}\mathbf{W}$ với ρ sẽ được $\mathbf{W}$ (điều này cũng đúng với số hạng độ lệch $\mathbf{B}$), ta có thể dùng $\mathbf{W}$ (và $\mathbf{B}$) làm tham số của hệ thống suy luận.

Chú giải của người dịch: Vì sao phải nhân với ρ?

Lấy $\rho = 0.5$: khi huấn luyện, trung bình chỉ một nửa số nơ-ron ở tầng dưới "gửi tín hiệu" lên, nên tổng đầu vào mà một nơ-ron ở tầng trên nhận được chỉ khoảng một nửa so với khi mọi nơ-ron đều có mặt. Mạng đã quen với mức tín hiệu "một nửa" đó. Khi kiểm tra, mọi nơ-ron đều bật, tín hiệu sẽ gấp đôi mức quen thuộc nếu không điều chỉnh; vì thế ta nhân trọng số với $\rho = 0.5$ để đưa về đúng mức (Phương trình (2.78)). Cách thứ hai (Phương trình (2.79), thường gọi là *dropout đảo ngược*) làm điều ngược lại: phóng to $\times\frac{1}{\rho} = \times 2$ ngay lúc huấn luyện, để khi kiểm tra dùng nguyên trọng số mà không phải sửa gì. Đây là cách các thư viện học sâu hiện nay thường cài đặt.

2.5.3 Dừng sớm

Ở Chương 1, chúng ta đã bàn một chút về cách dừng huấn luyện bằng việc theo dõi hiệu năng trên dữ liệu kiểm định. Có thể xem đây là một dạng lựa chọn mô hình (*model selection*) nhằm tìm điểm cân bằng tối ưu giữa chưa khớp (*underfitting*) và quá khớp. Lưu ý rằng dừng sớm (*early stopping*) không chỉ là một mẹo kinh nghiệm; nó còn có cơ sở vững chắc trong lý thuyết học thống kê. Ví dụ, các nhà

[8] Lưu ý rằng việc đặt hệ số co giãn bên trong hàm kích hoạt ψ (ví dụ dùng $\rho\mathbf{W}$) ngầm giả định rằng ψ là một hàm tuyến tính hoặc có tính thuần nhất dương (ví dụ hàm ReLU, với $\text{ReLU}(cx) = c\,\text{ReLU}(x)$ khi $c > 0$). Nếu dùng một hàm kích hoạt phi tuyến, chẳng hạn Sigmoid, thì không thể đưa hệ số co giãn vào bên trong. Trong những trường hợp đó, phép co giãn được áp dụng trực tiếp lên đầu ra của hàm kích hoạt để bảo toàn đúng giá trị kỳ vọng. Khi ấy, dạng toán học của phép co giãn tiêu chuẩn khi suy luận và của huấn luyện dropout đảo ngược lần lượt là $\mathbf{y} = \rho \cdot \psi(\mathbf{x} \cdot \mathbf{W} + \mathbf{B})$ và $\mathbf{y} = \frac{1}{\rho}\big(\mathbf{M}_{\text{drop}} \odot \psi(\mathbf{x} \cdot \mathbf{W} + \mathbf{B})\big)$.

nghiên cứu đã phát hiện rằng, dưới một số điều kiện nhất định, dừng sớm có tác dụng tương tự chính quy hóa l_2, giới hạn việc học trong một vùng có giá trị tham số nhỏ [Bishop, 1995a; Goodfellow và cộng sự, 2016]. Hơn nữa, các nghiên cứu cho thấy những quy tắc dừng sớm cụ thể có liên hệ chặt chẽ với sự đánh đổi độ chệch–phương sai (*bias-variance trade-off*) và có thể đảm bảo những tính chất hội tụ mong muốn [Yao và cộng sự, 2007].

Tuy nhiên, trong thực tế, dừng sớm cần đến một số mẹo kinh nghiệm để trở nên khả thi và hữu ích. Vấn đề chính nằm ở việc xác định chính xác tiêu chí dừng. Một cách lý tưởng, ta có thể hình dung rằng có một đường cong lỗi hình chữ U hoàn hảo trên dữ liệu kiểm định, và việc huấn luyện có thể dừng ngay khi lỗi bắt đầu tăng. Tuy nhiên, sự thật là đường cong lỗi không thể được mô tả đơn giản như một hàm lồi chặt theo thời gian huấn luyện. Sau khi lỗi giảm trong một số bước huấn luyện nhất định, hiệu năng của mô hình có xu hướng dao động, dẫn tới nhiều cực tiểu địa phương. Vấn đề còn thú vị hơn nếu ta muốn tiết kiệm thời gian và dừng huấn luyện càng sớm càng tốt. Nhưng ta không bao giờ biết lựa chọn hay quyết định hiện tại có phải là tốt nhất hay không, vì ta không biết điều gì sẽ xảy ra tiếp theo. Một phương pháp thường dùng là quyết định có nên dừng huấn luyện hay không bằng cách kiểm tra trạng thái của mô hình trong một số bước cập nhật (hoặc epoch) gần nhất [Prechelt, 1998]. Một số điều kiện dừng sớm là:

- Mức thay đổi của hiệu năng nhỏ hơn một ngưỡng trong một số bước (hoặc epoch) cho trước.
- Mức thay đổi của tham số mô hình nhỏ hơn một ngưỡng trong một số bước (hoặc epoch) cho trước.
- Hiệu năng trung bình trên một số bước (hoặc epoch) cho trước bắt đầu giảm.
- Hiệu năng cao nhất trên một số bước (hoặc epoch) cho trước bắt đầu suy giảm.

Chú giải của người dịch: Dừng sớm trong thực tế

Một quy tắc rất hay gặp là "kiên nhẫn" (*patience*): sau mỗi epoch ta đo lỗi trên tập kiểm định; nếu chẳng hạn suốt 5 epoch liền mà lỗi không thấp hơn mức tốt nhất từng đạt được, ta dừng và lấy lại mô hình ở thời điểm tốt nhất. Ví dụ, lỗi kiểm định qua các epoch là 0.50, 0.40, 0.35, 0.36, 0.34, 0.35, 0.36, 0.37, 0.38, 0.39: mức tốt nhất là 0.34 ở epoch 5; sau đó 5 epoch liền không cải thiện nên ta dừng ở epoch 10. Lưu ý rằng nếu dừng ngay khi lỗi tăng lần đầu (epoch 4), ta đã bỏ lỡ điểm tốt hơn ở epoch 5 — đó chính là khó khăn do đường cong lỗi "dao động" mà sách nhắc tới.

Tuy nhiên, dùng mô hình tại thời điểm dừng huấn luyện không phải lúc nào cũng là lựa chọn tốt. Trong thực tế, quanh thời điểm đó lỗi tổng quát hóa của mô hình thường có phương sai lớn, khiến việc lựa chọn mô hình khó khăn hơn. Thay vì "chọn" một mô hình, một cách khác là kết hợp nhiều mô hình. Ví dụ, ta có thể lưu lại mô hình sau mỗi một số bước huấn luyện cho trước (gọi mỗi bản sao của mô hình là một điểm lưu (*checkpoint*)). Mô hình cuối cùng thu được bằng cách lấy trung bình tham số của vài điểm lưu sau cùng. Để có kết quả tốt hơn, ta có thể dùng những phương pháp tổ hợp mô hình phức tạp hơn (xem Chương 1).

2.5.4 Làm trơn xác suất đầu ra

Trong thống kê, làm trơn (*smoothing*) là quá trình giảm ảnh hưởng của các điểm dữ liệu nhiễu (thường là các giá trị cực đoan) và phân bổ lại khối lượng xác suất cho các điểm dữ liệu bình thường hoặc cho các sự kiện chưa quan sát được. Nó được dùng rộng rãi khi một phân phối được ước lượng từ dữ liệu hạn chế, trong đó xác suất của các sự kiện hiếm được biểu diễn kém. Ví dụ, xét bài toán mô hình hóa ngôn ngữ đã mô tả ở Mục 2.2. Một mô hình ngôn ngữ được huấn luyện theo cách buộc mô hình phải

xuất ra một phân phối one-hot, tức là toàn bộ khối lượng xác suất bằng 1 được gán cho một từ duy nhất, còn các từ khác đều có xác suất bằng 0. Có lẽ sẽ tốt hơn nếu phân bổ xác suất cho tất cả các từ, dù nhiều từ trong số đó không được quan sát là câu trả lời khi biết các từ đứng trước. Bằng cách này, mô hình học cách đưa ra dự đoán "mềm" về xác suất của các từ, nhờ đó có thể tổng quát hóa tốt hơn trên dữ liệu chưa thấy.

Cho một phân phối $\mathbf{p} = \begin{bmatrix} p_1 & \cdots & p_n \end{bmatrix}$, mục đích của làm trơn là thu được một ước lượng mới nằm giữa $\mathbf{p}$ và phân phối đều $\frac{1}{n}$. Một cách tiếp cận phổ biến cho ý tưởng này là dùng một **bộ ước lượng co** (*shrinkage estimator*) để cải thiện $\mathbf{p}$ bằng cách kéo nó lại gần $\frac{1}{n}$. Ví dụ, làm trơn cộng đã nhắc tới ở Chương 1 là một dạng đơn giản của bộ ước lượng co. Ở đây, chẳng hạn, ta xét việc làm trơn một phân phối đa thức (*multinomial distribution*). Gọi p_k là xác suất của sự kiện k và s_k là một đại lượng mô tả "số đếm" quan sát được nào đó của sự kiện. Xác suất p_k được cho bởi

$$p_k \;=\; \frac{s_k}{\sum_{k=1}^{n} s_k}. \tag{2.80}$$

Khi đó, phiên bản đã làm trơn của p_k được định nghĩa là

$$\hat{p}_k \;=\; \frac{s_k + \alpha}{\sum_{k=1}^{n} (s_k + \alpha)}. \tag{2.81}$$

Nó đơn giản là cộng thêm một lượng α vào mỗi s_k. Giá trị của α điều khiển độ trơn của ước lượng thu được. Ví dụ, $\hat{p}_k = p_k$ nếu $\alpha = 0$, và $\hat{p}_k \approx \frac{1}{n}$ khi $\alpha \to \infty$.

Ngoài làm trơn cộng, ta có thể làm trơn một phân phối bằng hàm Softmax với co giãn nhiệt độ (*temperature scaling*), được định nghĩa như sau:

$$\hat{p}_k \;=\; \frac{\exp(s_k/\beta)}{\sum_{k=1}^{n} \exp(s_k/\beta)}. \tag{2.82}$$

Dạng này là một ví dụ kinh điển của **phân phối Boltzmann** (*Boltzmann distribution*) [Uffink, 2017], trong đó s_k có thể được xem là năng lượng âm của một trạng thái, còn β biểu diễn nhiệt độ (*temperature*) điều khiển mức độ làm trơn. Lưu ý rằng s_k có thể được hiểu theo nhiều cách. Chẳng hạn, trong một mạng nơ-ron, s_k thường là logit (*logit*) trước kích hoạt của một nơ-ron. Đôi khi s_k thậm chí có thể là một xác suất thô. Sự linh hoạt này hàm ý rằng ta có thể áp dụng trực tiếp Phương trình (2.81–2.82) cho bất kỳ phân phối $\mathbf{p}$ nào, ngay cả khi không biết trước $\mathbf{p}$ ban đầu được ước lượng ra sao. Ta có thể viết lại các phương trình này bằng cách thay s_k bởi p_k:

$$\hat{p}_k = \frac{p_k + \alpha}{\sum_{k=1}^{n} (p_k + \alpha)}, \tag{2.83}$$

$$\hat{p}_k = \frac{\exp(p_k/\beta)}{\sum_{k=1}^{n} \exp(p_k/\beta)}. \tag{2.84}$$

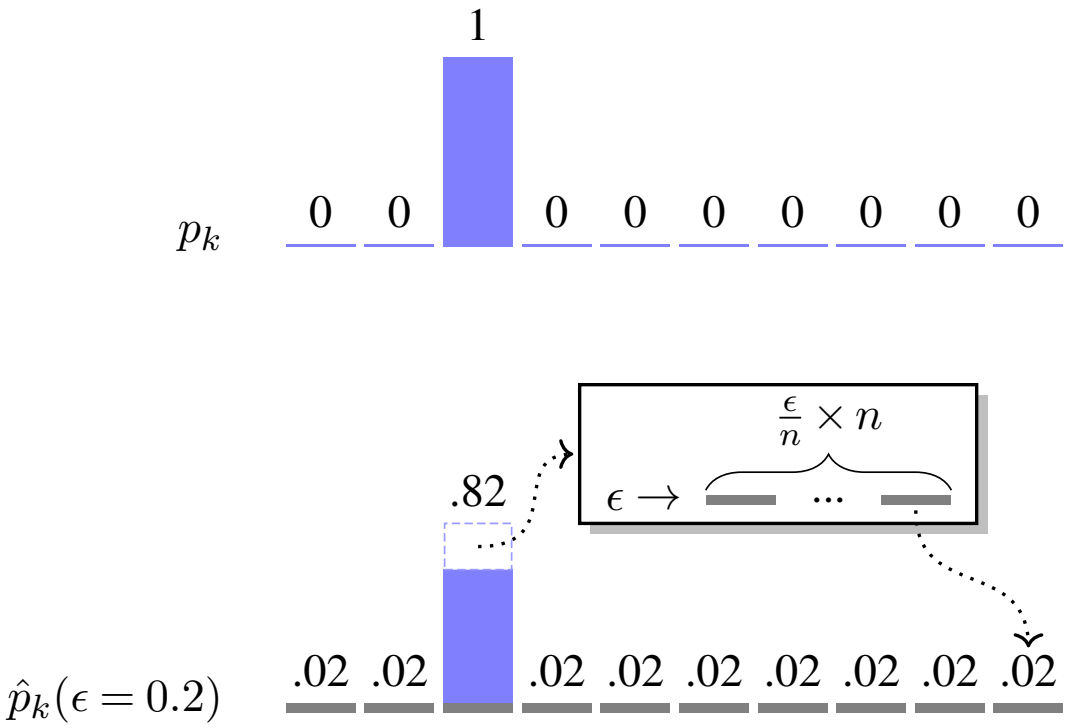
Chú giải của người dịch: Nhiệt độ β làm trơn phân phối như thế nào?

Lấy ba "điểm số" $s = (2,\ 1,\ 0)$. Với $\beta = 1$: $e^2 \approx 7.39$, $e^1 \approx 2.72$, $e^0 = 1$, tổng ≈ 11.11, nên $\hat{p} \approx (0.67,\ 0.24,\ 0.09)$. Với $\beta = 2$ (nhiệt độ cao hơn): $e^1 \approx 2.72$, $e^{0.5} \approx 1.65$, $e^0 = 1$, tổng ≈ 5.37, nên $\hat{p} \approx (0.51,\ 0.31,\ 0.19)$ — phân phối "phẳng" hơn, gần phân phối đều $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ hơn. Ngược lại, β nhỏ (ví dụ 0.5) làm phân phối "nhọn" hơn, dồn gần hết xác suất vào phần tử lớn nhất. Tên gọi "nhiệt độ" mượn từ vật lý: ở nhiệt độ cao, các phân tử ở nhiều trạng thái khác nhau với khả năng gần ngang nhau; ở nhiệt độ thấp, chúng dồn về trạng thái năng lượng thấp nhất.

Một phương pháp làm trơn đơn giản khác là nội suy tuyến tính (*linear interpolation*) **p** với một phân phối đều. Một dạng chuẩn của phép nội suy này là:

$$\hat{p}_k = (1 - \epsilon) \cdot p_k + \frac{\epsilon}{n}, \tag{2.85}$$

trong đó ϵ là một siêu tham số cho biết mức độ ta dựa vào phân phối đều. Để minh họa cách Phương trình (2.85) hoạt động, giả sử **p** là một vectơ one-hot (ví dụ $p_k = 1$ nếu $k = z$, và $p_k = 0$ trong các trường hợp còn lại). Khi áp dụng Phương trình (2.85), thực chất ta thu nhỏ phân phối ban đầu theo hệ số $(1 - \epsilon)$ và cộng một khối lượng xác suất đều bằng $\frac{\epsilon}{n}$ vào mọi chiều. Sự phân bổ lại này làm phân phối thu được phẳng hơn và trơn hơn. Xem Hình 2.15 để có minh họa trực quan.



Hình 2.15: Minh họa việc làm trơn một phân phối bằng cách nội suy nó với một phân phối đều: $\hat{p}_k = (1 - \epsilon) \cdot p_k + \frac{\epsilon}{n}$. Về mặt toán học, phép toán này giảm mỗi xác suất ban đầu p_k đi một lượng ϵp_k, gom lại tổng cộng một khối lượng xác suất bằng ϵ. Khối lượng này sau đó được chia đều cho tất cả n chiều, bảo đảm mỗi biến đều nhận được một lượng làm trơn cộng thêm bằng $\frac{\epsilon}{n}$.

Chú giải của người dịch: Kiểm tra các con số trong Hình 2.15

Trong hình có $n = 10$ chiều và $\epsilon = 0.2$. Mỗi chiều được cộng thêm $\frac{\epsilon}{n} = \frac{0.2}{10} = 0.02$. Chiều đúng (vốn có xác suất 1) trở thành $(1 - 0.2) \cdot 1 + 0.02 = 0.82$; chín chiều còn lại trở thành $(1 - 0.2) \cdot 0 + 0.02 = 0.02$. Kiểm tra tổng: $0.82 + 9 \times 0.02 = 1$, vẫn là một phân phối xác suất hợp lệ.

Trong NLP, vì nhiều hệ thống đưa ra dự đoán dưới dạng giống xác suất, một ứng dụng phổ biến của làm trơn là làm trơn đầu ra của hệ thống. Có hai cách. Thứ nhất, ta có thể làm trơn xác suất chuẩn đúng (*ground-truth*) để mô hình được dẫn dắt bởi sai số tổng quát hóa thay vì sai số do các quyết định

cứng gây ra. Ví dụ, kỹ thuật **làm trơn nhãn** (*label smoothing*) có cùng dạng với Phương trình (2.85) và cải thiện cách biểu diễn chuẩn đúng trên dữ liệu phân loại [Szegedy và cộng sự, 2016]. Thứ hai, ta có thể giảm độ "nhọn" và tăng độ "dày đuôi" của một phân phối dự đoán[9]. Phương pháp này thường được dùng khi cần đến xác suất hậu nghiệm (*posterior probability*) của dự đoán, chẳng hạn trong giải mã/huấn luyện rủi ro Bayes cực tiểu (*minimum Bayesian risk*) [Bickel và Doksum, 2015; Goodman, 1996a; Kumar và Byrne, 2004a].

2.5.5 Huấn luyện với nhiễu

Ở trên, ta đã thấy rằng cộng thêm một lượng nào đó vào mỗi số đếm quan sát được của các sự kiện khi dự đoán xác suất có thể cải thiện khả năng tổng quát hóa. Theo quan điểm **thống kê vững** (*robust statistics*) [Olive, 2022], điều này tương đương với việc tăng tính vững của một bộ ước lượng, trong tình huống mà một phân phối bị lệch thường dẫn tới một mô hình bị thiên lệch. Việc thêm một nhiễu loạn nhỏ vào ước lượng có thể ngăn những thiên lệch lớn gây ra bởi các điểm ngoại lai (*outliers*) và những lần quan sát bất ngờ của các sự kiện hiếm. Theo nghĩa này, làm trơn có thể được xem là một cách đưa nhiễu vào huấn luyện, tức là ta áp đặt một tiên nghiệm phân phối đều lên ước lượng, dù ước lượng đúng có thể không phải là phân phối đều.

Huấn luyện với nhiễu dựa trên ý tưởng rằng một mô hình nên học cách vận hành trong những điều kiện không lý tưởng, nhờ đó tránh quá khớp các điểm dữ liệu cực đoan. Ở đây, thuật ngữ *nhiễu* mang nghĩa rộng, và có nhiều cách khác nhau để chính quy hóa việc huấn luyện thông qua tiêm nhiễu (*noise injection*). Một trong những phương pháp đơn giản nhất là đưa nhiễu vào hàm mục tiêu huấn luyện. Ví dụ, làm trơn nhãn (như đã bàn ở trên) có thể được xem là tiêm nhiễu vào các nhãn chuẩn đúng. Ngoài ra, ta có thể tiêm nhiễu ngẫu nhiên trực tiếp vào đầu vào, các trạng thái ẩn trung gian, hoặc đầu ra của một mạng nơ-ron. Một lựa chọn rất phổ biến là **nhiễu Gauss** (*Gaussian noise*). Giả sử ta có một vectơ kích hoạt $\mathbf{x} \in \mathbb{R}^n$. Việc thêm nhiễu Gauss tạo ra một vectơ mới bị làm nhiễu như sau:

$$\mathbf{x}_{\text{noise}} \;=\; \mathbf{x} + \mathbf{g}, \tag{2.86}$$

trong đó $\mathbf{g} \in \mathbb{R}^n$ là vectơ nhiễu. Mỗi phần tử của $\mathbf{g}$ được rút độc lập từ một phân phối Gauss:

$$\mathbf{g} \;\sim\; \mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2)). \tag{2.87}$$

Với phần tử thứ k của $\mathbf{g}$, phân phối này định nghĩa xác suất $\Pr(g_k)$ là:

$$\Pr(g_k) \;=\; \frac{1}{\sigma_k\sqrt{2\pi}} \cdot \exp\Big(-\frac{(g_k-\mu_k)^2}{2\sigma_k^2}\Big), \tag{2.88}$$

trong đó μ_k là giá trị trung bình và σ_k là độ lệch chuẩn. Thông thường, μ_k được đặt bằng 0. Độ lệch chuẩn σ_k là một siêu tham số điều khiển độ lớn của nhiễu được tiêm vào. σ_k càng lớn thì nhiễu ngẫu nhiên càng trải rộng quanh giá trị trung bình, làm tăng khả năng sinh ra những nhiễu loạn mạnh.

Chú giải của người dịch: Phân phối Gauss (phân phối chuẩn)

Phương trình (2.88) là đường cong "hình chuông" quen thuộc: các giá trị gần μ_k có khả năng xuất hiện cao nhất, càng xa μ_k thì càng hiếm. Chính xác hơn, $\Pr(g_k)$ ở đây là *mật độ xác suất* (chiều cao của đường cong), không phải xác suất của một giá trị cụ thể. Một quy tắc dễ nhớ: khoảng 68% giá trị rơi vào đoạn

[9] Trong thống kê, điều này có liên hệ lỏng lẻo với việc biến một phân phối nhọn đỉnh (*leptokurtic*) thành một phân phối có độ nhọn trung bình (*mesokurtic*) hoặc bẹt đỉnh (*platykurtic*) hơn để tránh quá tự tin.

$[\mu_k - \sigma_k,\ \mu_k + \sigma_k]$ và khoảng 95% rơi vào $[\mu_k - 2\sigma_k,\ \mu_k + 2\sigma_k]$. Chẳng hạn với $\mu_k = 0$, $\sigma_k = 0.1$, hầu hết nhiễu nằm trong khoảng -0.2 đến 0.2; nếu $x_k = 1.5$ thì sau khi thêm nhiễu, x_k thường thành một số trong khoảng 1.3 đến 1.7. Ký hiệu $\text{diag}(\boldsymbol{\sigma}^2)$ trong Phương trình (2.87) chỉ đơn giản nói rằng mỗi phần tử có phương sai σ_k^2 riêng và các phần tử được rút độc lập với nhau.

Phương trình (2.86) rất tổng quát và có thể được áp dụng gần như ở bất cứ đâu trong một mạng nơ-ron. Cho một tầng $\mathbf{y} = \psi(\mathbf{x} \cdot \mathbf{W} + \mathbf{B})$, nhiễu (ký hiệu là $\mathbf{g}_{\text{input}}$) có thể được cộng trực tiếp vào đầu vào:

$$\mathbf{y} = \psi\big((\mathbf{x} + \mathbf{g}_{\text{input}}) \cdot \mathbf{W} + \mathbf{B}\big). \tag{2.89}$$

Tương tự, nhiễu (chẳng hạn $\mathbf{g}_{\text{output}}$) có thể được cộng vào giá trị kích hoạt (hay đầu ra):

$$\mathbf{y} = \psi(\mathbf{x} \cdot \mathbf{W} + \mathbf{B}) + \mathbf{g}_{output}. \tag{2.90}$$

Ví dụ, ta có thể đơn giản làm nhiễu các đầu vào thô (hoặc các giá trị đích) rồi chạy mạng như bình thường, hoặc tiêm nhiễu ngẫu nhiên vào giá trị kích hoạt của tất cả các tầng ẩn. Trong khi việc thêm nhiễu vào đầu vào và giá trị kích hoạt của các tầng là cách làm phổ biến [Plaut và cộng sự, 1986; Holmström và Koistinen, 1992; Bishop, 1995b], một cách tiếp cận khác biệt là thêm nhiễu ngẫu nhiên trực tiếp vào tham số mô hình hoặc gradient [Graves và cộng sự, 2013b; Neelakantan và cộng sự, 2015]. Ví dụ, làm nhiễu ma trận trọng số có dạng sau:

$$\mathbf{y} \ = \ \psi\big(\mathbf{x} \cdot (\mathbf{W} + \mathbf{g}_{\text{w}}) + \mathbf{B}\big), \tag{2.91}$$

trong đó $\mathbf{g}_{\text{w}}$ là một ma trận nhiễu có cùng kích thước với $\mathbf{W}$. Hơn nữa, ta có thể tiêm nhiễu (ký hiệu là $\mathbf{g}_{\text{gradient}}$) vào gradient của hàm mất mát theo $\mathbf{W}$. Gọi $\mathbf{s} = \mathbf{x} \cdot \mathbf{W} + \mathbf{B}$ là các logit trước kích hoạt. Gradient có nhiễu có thể được viết là:

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial \mathbf{W}} &= \mathbf{x}^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{s}} + \mathbf{g}_{\text{gradient}} \\
&= \mathbf{x}^\top \cdot \Big(\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \odot \frac{\partial \mathbf{y}}{\partial \mathbf{s}}\Big) + \mathbf{g}_{\text{gradient}} \\
&= \mathbf{x}^\top \cdot \Big(\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \odot \psi'(\mathbf{s})\Big) + \mathbf{g}_{\text{gradient}}.
\end{aligned} \tag{2.92}$$

Việc dùng gradient có nhiễu được nhận thấy không chỉ giúp huấn luyện vững hơn mà còn làm cho dòng gradient chảy qua mạng dễ dàng hơn [Gulcehre và cộng sự, 2016].

Cần lưu ý rằng nhiễu chỉ xuất hiện trong lúc huấn luyện, còn mô hình hoạt động mà không thêm nhiễu khi dự đoán trên dữ liệu mới. Theo nghĩa này, nhiều phương pháp chính quy hóa có thể được xếp vào khuôn khổ huấn luyện có nhiễu, vốn được dùng để ngăn việc khớp quá chính xác dữ liệu huấn luyện và giúp hệ thống suy luận tổng quát hóa tốt trên dữ liệu kiểm tra. Ví dụ, dropout che ngẫu nhiên các giá trị kích hoạt của tầng để mọi nơ-ron học cách làm việc trong một môi trường có nhiễu. Khi kiểm tra, mọi nơ-ron làm việc cùng nhau như trong một mạng nơ-ron thông thường.

Một ưu điểm nữa của huấn luyện có nhiễu là việc dùng nhiễu ngẫu nhiên tạo ra các mẫu huấn luyện "mới". Ngay cả với cùng một mẫu, các nhiễu khác nhau cũng có thể dẫn tới các kết quả huấn luyện khác nhau. Nói cách khác, về bản chất ta huấn luyện mô hình trên vô số mẫu. Ý tưởng này cũng gắn với một hướng nghiên cứu khác về huấn luyện bằng dữ liệu tổng hợp, gọi là **tăng cường dữ liệu** (*data augmentation*). Nói đơn giản, tăng cường dữ liệu bao gồm một loạt kỹ thuật được thiết kế để sinh ra các mẫu huấn luyện mới từ những mẫu sẵn có. Một ví dụ là **dịch ngược** (*back-translation*) [Sennrich và cộng sự, 2016a]. Khi phát triển một hệ thống dịch máy từ ngôn ngữ A sang ngôn ngữ B, trước tiên ta

có thể huấn luyện một hệ thống dịch theo chiều ngược lại (chẳng hạn hệ thống B→A) trên dữ liệu song ngữ. Sau đó, ta dùng hệ thống B→A để dịch một lượng dữ liệu bổ sung ở ngôn ngữ đích sang ngôn ngữ nguồn. Kết quả là ta có dữ liệu song ngữ mới, trong đó phần ngôn ngữ đích là thật còn phần ngôn ngữ nguồn là tổng hợp. Dữ liệu mới này có thể được dùng cùng với dữ liệu song ngữ khác để huấn luyện hệ thống A→B. Ngoài dịch ngược, có nhiều phương pháp tăng cường dữ liệu khác trong NLP, bao gồm thay từ bằng từ đồng nghĩa, hoán đổi hai từ, và xóa hoặc chèn từ. Hơn nữa, việc tăng cường có thể được thực hiện trong không gian ẩn (*latent space*) liên tục, chẳng hạn thay một vectơ từ bằng một vectơ ở gần nó về mặt hình học. Vì tăng cường dữ liệu bao trùm rất nhiều chủ đề, chúng tôi giới thiệu bạn đọc một vài bài tổng quan để tìm hiểu thêm [Feng và cộng sự, 2021; Shorten và Khoshgoftaar, 2019].

Chú giải của người dịch: Dịch ngược qua ví dụ Việt–Anh

Giả sử ta muốn xây dựng hệ thống dịch Việt→Anh (A = tiếng Việt, B = tiếng Anh) nhưng chỉ có ít cặp câu song ngữ, trong khi văn bản tiếng Anh đơn ngữ thì vô số. Ta huấn luyện trước một hệ thống Anh→Việt, rồi dùng nó dịch hàng triệu câu tiếng Anh sang tiếng Việt. Ta được các cặp (câu Việt do máy dịch, câu Anh thật). Phía tiếng Việt có thể hơi "vụng", nhưng phía tiếng Anh — chính là phía mà hệ thống Việt→Anh phải học cách *sinh ra* — là câu thật, chuẩn mực. Nhờ vậy, mô hình học được cách viết câu đích tự nhiên từ lượng dữ liệu lớn hơn nhiều.

Một lưu ý cuối cùng về tăng cường dữ liệu: dữ liệu tổng hợp cũng có thể được tạo ra cho những mục tiêu cụ thể. Một ý tưởng phổ biến là **học máy đối kháng** (*adversarial machine learning*). Nó sinh ra các **mẫu đối kháng** (*adversarial samples*) mà trên đó mô hình sẽ mắc lỗi (gọi những quá trình như vậy là các **cuộc tấn công** (*attacks*)) [Szegedy và cộng sự, 2014a; Goodfellow và cộng sự, 2015]. Mô hình được huấn luyện để đưa ra dự đoán đúng trên các mẫu này, tức là nó phòng thủ trước các cuộc tấn công. Ví dụ, trong một số trường hợp, một hệ thống dịch máy có thể cho ra bản dịch hoàn toàn sai chỉ vì giới tính của chủ ngữ trong câu đầu vào bị đảo ngược. Để có một hệ thống vững hơn, ta có thể huấn luyện mô hình dịch bằng dữ liệu cân bằng giới tính hơn, được thu thập thủ công hoặc tự động. Nhưng không dễ để tạo ra những mẫu trông giống câu bình thường mà lại đánh lừa được mô hình [Zhang và cộng sự, 2020b]. Điều này khiến việc sinh mẫu đối kháng trong NLP vừa thú vị vừa đầy thách thức, vì một thay đổi nhỏ trong câu (như thay một từ) có thể dẫn tới một câu mang nghĩa rất khác[10]. Thách thức này cũng thúc đẩy một hướng nghiên cứu về mẫu đối kháng trong NLP [Jia và Liang, 2017; Belinkov và Bisk, 2018; Ebrahimi và cộng sự, 2018; Alzantot và cộng sự, 2018].

2.6 Các phương pháp không giám sát và bộ tự mã hóa

Học không giám sát (*unsupervised learning*) quan tâm đến việc khám phá những khuôn mẫu ẩn bên dưới trong một tập điểm dữ liệu không có nhãn. Nhiều bài toán có thể được xem là bài toán học không giám sát kinh điển, dù chúng tôi sẽ không bàn chi tiết về chúng trong chương này. Ví dụ, phân cụm dữ liệu (*data clustering*) nhằm tìm ra các nhóm trong một tập đối tượng dữ liệu mà không có sự giám sát tường minh nào về cách nhóm đúng. Một ví dụ nổi tiếng khác là khai phá luật kết hợp (*association rule mining*), thường được phát biểu như một quá trình thiết lập mối quan hệ giữa các tập đối tượng dữ liệu. Dù những bài toán này đúng là thuộc phạm vi học không giám sát, chúng tôi sẽ tập trung vào các bài toán học biểu diễn (*representation learning*) không giám sát hay **học đặc trưng** (*feature learning*), trong đó mô hình học cách ánh xạ một đối tượng từ không gian đầu vào sang một không gian vectơ đặc

[10] Ngược lại, trong thị giác máy tính, việc tạo mẫu đối kháng dễ hơn nhiều bằng cách thay đổi chút ít đầu vào (ví dụ các điểm ảnh), vì không gian đầu vào là liên tục và một nhiễu loạn nhỏ ở đầu vào hầu như không ảnh hưởng tới toàn bộ bức ảnh.

trưng ít chiều[11].

Việc học các biểu diễn ít chiều đã được nghiên cứu sâu rộng trong bối cảnh tìm một phép biến đổi tuyến tính từ không gian ban đầu sang một không gian mới. Ví dụ, **phân tích thành phần chính** (*principal component analysis, PCA*) và các biến thể của nó nhằm tìm một hàm ánh xạ tuyến tính sao cho một đối tượng dữ liệu (nhiều chiều) có thể được biểu diễn bởi tọa độ của nó dọc theo những hướng có phương sai lớn nhất [Pearson, 1901; Wold và cộng sự, 1987]. Ở đây, chúng ta mở rộng hàm ánh xạ thành dạng tổng quát hóa phi tuyến tự nhiên của nó và dùng mạng nơ-ron làm lời giải cho bài toán ánh xạ.

Như các mô hình học máy khác, một mạng nơ-ron thường được huấn luyện bằng cách tối ưu tham số mô hình theo một hàm mất mát nào đó. Một thách thức đáng kể với học không giám sát là không có nhãn chuẩn đúng nào để dẫn dắt quá trình học. Một giải pháp cho vấn đề này là dùng đến các phương pháp phi tham số (*non-parametric*) hoặc các mẹo kinh nghiệm (xem Chương 1). Tuy nhiên, bản thân những phương pháp này không được thiết kế để giải quyết các vấn đề tối ưu của mạng nơ-ron quy mô lớn, đặc biệt khi mạng được xây dựng với số lượng tham số khổng lồ. Vì vậy, trong huấn luyện không giám sát mạng nơ-ron, cách phổ biến hơn là rút ra tín hiệu giám sát trực tiếp từ chính dữ liệu đầu vào. Có nhiều cách để làm điều này [Hopfield, 1982; Ackley và cộng sự, 1985; Dayan và cộng sự, 1995; Hinton và Salakhutdinov, 2006], nhưng trong mục này chúng ta sẽ tập trung vào **bộ tự mã hóa** (*auto-encoders*). Chúng tôi chọn bộ tự mã hóa để thảo luận vì chúng có dạng giống các mô hình có giám sát nói chung và có thể được huấn luyện liền mạch bằng lan truyền ngược.

Bộ tự mã hóa là một loại mạng nơ-ron cố gắng tái tạo dữ liệu đầu vào từ biểu diễn của nó. Nó được truyền cảm hứng từ ý tưởng giảm chiều (*dimensionality reduction*):

> *Dữ liệu nhiều chiều có thể được chuyển thành các mã ít chiều bằng cách huấn luyện một mạng nơ-ron nhiều tầng có một tầng trung tâm nhỏ để tái tạo các vectơ đầu vào nhiều chiều.*
>
> – Hinton và Salakhutdinov [2006]

Điều này cũng phát triển ý tưởng học biểu diễn, theo đó thông tin về một đối tượng có thể được biểu diễn đầy đủ bằng một vectơ số thực ít chiều. Thông thường, một bộ tự mã hóa gồm một hàm giảm chiều (có thể phi tuyến) (gọi là **bộ mã hóa** (*encoder*)) để ánh xạ đối tượng đầu vào thành biểu diễn vectơ đặc trưng ít chiều của nó (gọi là **mã** (*code*)). Ngoài ra, nó còn gồm một hàm ngược (gọi là **bộ giải mã** (*decoder*)) ánh xạ mã trở lại đối tượng. Vì vậy, mặc dù được gọi là "bộ mã hóa", bộ tự mã hóa không chỉ là một bộ mã hóa mà là sự kết hợp của một bộ mã hóa và một bộ giải mã. Một cách hình thức hơn, gọi $\mathbf{x} \in \mathbb{R}^{d_x}$ là vectơ đầu vào của mô hình, chẳng hạn một biểu diễn nhiều chiều của một từ. Bộ mã hóa xuất ra một vectơ mô tả mã hay biểu diễn ít chiều của $\mathbf{x}$, ký hiệu là $\mathbf{h} \in \mathbb{R}^{d_h}$. Vectơ này được định nghĩa là:

$$\mathbf{h} \;=\; \text{Encode}(\mathbf{x}), \tag{2.93}$$

trong đó $\text{Encode}(\cdot)$ là mạng mã hóa. $\text{Encode}(\cdot)$ thường là một mạng nơ-ron nhiều tầng và hoạt động như một mô-đun cắm thêm (*plug-in*) cho các hệ thống khác. Do đó, $\text{Encode}(\cdot)$ là một mô hình đa dụng. Trong các chương sau, chúng ta sẽ thấy nhiều ví dụ trong đó các bộ mã hóa được huấn luyện và áp dụng như những bộ phận của các hệ thống "lớn hơn".

Khi đã có mã, ta dùng bộ giải mã để ánh xạ nó trở lại không gian đầu vào:

$$\widetilde{\mathbf{x}} \;=\; \text{Decode}(\mathbf{h}), \tag{2.94}$$

[11] Ngoài việc học cách biểu diễn các đối tượng dữ liệu, mục này cũng đề cập một số chủ đề về việc sinh ra các đối tượng dữ liệu. Chúng ta sẽ tìm hiểu chúng ở Mục 2.6.3.

trong đó $\widetilde{\mathbf{x}} \in \mathbb{R}^{d_x}$ là bản tái tạo của đầu vào, và $\text{Decode}(\cdot)$ là mạng giải mã. Cho đầu vào ban đầu $\mathbf{x}$ và đầu vào được tái tạo $\widetilde{\mathbf{x}}$, mục tiêu của bộ tự mã hóa là cực tiểu hóa sự sai khác giữa $\mathbf{x}$ và $\widetilde{\mathbf{x}}$. Giả sử bộ mã hóa và bộ giải mã lần lượt được tham số hóa bởi θ và ω, ký hiệu là $\text{Encode}_\theta(\cdot)$ và $\text{Decode}_\omega(\cdot)$. Hàm mục tiêu huấn luyện trên một tập mẫu $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\}$ được định nghĩa là

$$\begin{aligned}(\hat{\theta}, \hat{\omega}) &= \arg\min_{(\theta,\omega)} \sum_{i=1}^{m} \mathcal{L}\Big(\mathbf{x}^{(i)}, \widetilde{\mathbf{x}}^{(i)}\Big) \\ &= \arg\min_{(\theta,\omega)} \sum_{i=1}^{m} \mathcal{L}\Big(\mathbf{x}^{(i)}, \text{Decode}_\omega(\mathbf{h}^{(i)})\Big) \\ &= \arg\min_{(\theta,\omega)} \sum_{i=1}^{m} \mathcal{L}\Big(\mathbf{x}^{(i)}, \text{Decode}_\omega(\text{Encode}_\theta(\mathbf{x}^{(i)}))\Big),\end{aligned} \tag{2.95}$$

trong đó $\mathcal{L}(\cdot)$ là hàm mất mát tính sự sai khác giữa $\mathbf{x}$ và $\widetilde{\mathbf{x}}$. Đôi khi nó được gọi là **mất mát tái tạo** (*reconstruction loss*). Các hàm mất mát tái tạo phổ biến gồm mất mát sai số bình phương trung bình (*mean squared error*), mất mát entropy chéo (*cross-entropy*), v.v.

Khi ghép bộ mã hóa và bộ giải mã lại với nhau, ta dễ nghĩ tới một mạng mà ta đưa thứ gì vào tầng đầu vào thì nhận lại đúng thứ đó ở tầng đầu ra. Thách thức ở đây là vectơ ít chiều $\mathbf{h}$ đóng vai trò một nút thắt cổ chai (*bottleneck*) trong dòng thông tin. Có nguy cơ mất thông tin trong phép biến đổi từ $\mathbf{x}$ sang $\mathbf{h}$ hoặc từ $\mathbf{h}$ sang $\widetilde{\mathbf{x}}$, khiến việc "sao chép" đầu vào sang đầu ra trở nên khó khăn. Thay vào đó, ta cần "ép" một đối tượng từ không gian nhiều chiều vào một không gian ít chiều, dày đặc, rồi "giải ép" nó từ không gian mới trở về không gian nhiều chiều ban đầu. Hệ quả của quá trình ép–giải ép này là bộ mã hóa bị buộc phải nén dữ liệu nhưng vẫn giữ được thông tin nhiều nhất có thể. Vì vậy, bộ tự mã hóa được bàn ở đây còn được gọi là **bộ tự mã hóa thiếu đủ** (*undercomplete auto-encoder*), vì $\mathbf{h}$ có kích thước nhỏ hơn $\mathbf{x}$ và $\widetilde{\mathbf{x}}$. Hình 2.16 minh họa cấu trúc của bộ tự mã hóa thiếu đủ.

Nhãn trong hình: *Training Objective*: hàm mục tiêu huấn luyện; *Input*: đầu vào; *Code (Bottleneck)*: mã (nút thắt cổ chai); *Reconstruction*: bản tái tạo; *Encoder/Decoder*: bộ mã hóa/bộ giải mã (Enc, Dec là viết tắt của Encode, Decode).

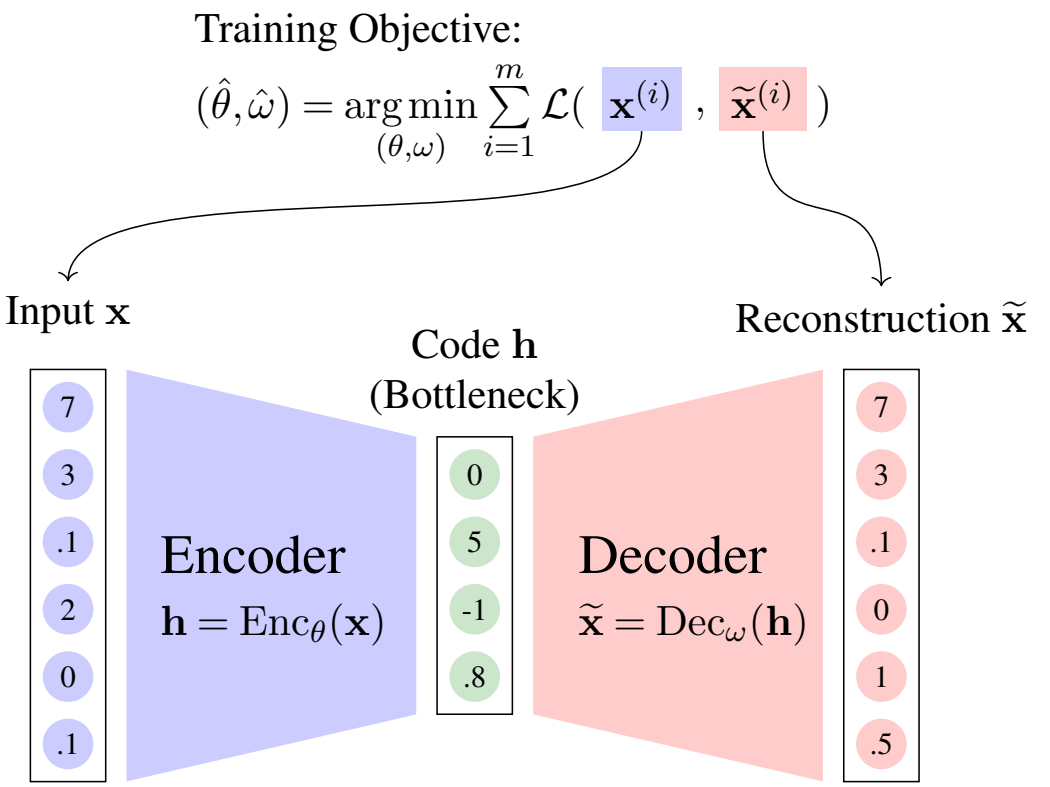
Cho hàm mất mát $\mathcal{L}(\cdot)$, bộ mã hóa $\text{Encode}_\theta(\cdot)$ và bộ giải mã $\text{Decode}_\omega(\cdot)$, các tham số $\hat{\theta}$ và $\hat{\omega}$ có thể được tối ưu bằng phương pháp hạ gradient như trong học có giám sát (xem Mục 2.4.1). Khi áp dụng bộ tự mã hóa, ta có thể đơn giản bỏ bộ giải mã đi và dùng bộ mã hóa như một bộ trích xuất đặc trưng (*feature extractor*), tức là, với một đầu vào mới $\mathbf{x}_{\text{new}}$, ta sinh ra một biểu diễn mới

$$\hat{\mathbf{h}}_{\text{new}} = \text{Encode}_{\hat{\theta}}(\mathbf{x}_{\text{new}}). \tag{2.96}$$

Chú giải của người dịch: Bộ tự mã hóa giống "tóm tắt rồi kể lại"

Hãy tưởng tượng bạn phải tóm tắt một bài văn dài 6 trang vào đúng 4 dòng (bộ mã hóa), rồi một người khác chỉ đọc 4 dòng đó và phải viết lại gần đúng bài văn gốc (bộ giải mã). Muốn người kia viết lại tốt, 4 dòng tóm tắt buộc phải chứa những ý *cốt lõi nhất*. Trong Hình 2.16, vectơ 6 số $\mathbf{x} = (7, 3, 0.1, 2, 0, 0.1)$ bị "nén" thành mã 4 số $\mathbf{h} = (0, 5, -1, 0.8)$, rồi được khôi phục thành $\widetilde{\mathbf{x}} = (7, 3, 0.1, 0, 1, 0.5)$: gần giống nhưng không hoàn hảo. Điều đáng chú ý là ta *không cần nhãn*: chính $\mathbf{x}$ vừa là đầu vào vừa là "đáp án". Sau khi huấn luyện xong, phần tóm tắt $\mathbf{h}$ thường là một biểu diễn rất có ích cho các bài toán khác.

Lưu ý rằng bộ mã hóa không phải là một hệ thống độc lập mà thường làm việc cùng các mô hình khác để tạo thành một hệ thống hoàn chỉnh. Ví dụ, ta có thể huấn luyện một bộ tự mã hóa trên một số câu, rồi đặt một tầng Softmax lên đầu ra của bộ mã hóa để xây dựng một bộ phân loại câu. Bộ phân loại này có thể được huấn luyện thêm trên dữ liệu riêng của một tác vụ để giải một bài toán mới, chẳng



Hình 2.16: Minh họa một bộ tự mã hóa thiếu đủ với một bộ mã hóa, một bộ giải mã và một tầng nút thắt cổ chai kẹp giữa chúng. Một đầu vào $\mathbf{x}$ (bên trái) được biến đổi thành mã $\mathbf{h}$ (ở giữa) rồi thành bản tái tạo $\widetilde{\mathbf{x}}$ (bên phải). Tham số của cả bộ mã hóa lẫn bộ giải mã được tối ưu bằng cách cực tiểu hóa sự sai khác giữa đầu vào $\mathbf{x}$ và bản tái tạo $\widetilde{\mathbf{x}}$ trên một số mẫu không có nhãn $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\}$. Với các mẫu mới, ta bỏ bộ giải mã đi và dùng bộ mã hóa để sinh ra các mã hay biểu diễn mới.

hạn gán cho câu nhãn cực tính cảm xúc (*sentiment polarity*) của nó. Điều này cũng khiến việc ứng dụng bộ tự mã hóa thuộc về mô thức chung của tiền huấn luyện (*pre-training*): một mô hình con (tức bộ mã hóa) trước tiên được huấn luyện trên dữ liệu quy mô lớn, không phụ thuộc tác vụ, rồi được dùng như một thành phần của một mô hình lớn hơn trong một tác vụ hạ nguồn (*downstream task*).

2.6.1 Bộ tự mã hóa với bộ chính quy hóa tường minh

Khi dùng các mạng nơ-ron phức tạp hơn, bộ tự mã hóa có xu hướng học một phép biến đổi đồng nhất (*identity transformation*), mặc dù nút thắt cổ chai khiến việc truyền qua mà không mất thông tin khó hơn đôi chút. Đây là điều ta vẫn thường mong đợi: ta có thể biến **h** thành một vật thay thế cho **x** và giải mã **h** thành thứ gì đó rất giống **x**. Mặt khác, việc học một phép biến đổi đồng nhất chính xác đòi hỏi một mô hình cực kỳ phức tạp và rất dễ quá khớp. May mắn là, như với các mô hình học máy khác, ta có thể chính quy hóa việc huấn luyện bằng các phương pháp đã trình bày ở Mục 2.5. Một trong những phương pháp phổ biến nhất là thêm một số hạng chính quy hóa tường minh vào hàm mất mát. Kết hợp Phương trình (2.74) và Phương trình (2.95), ta có thể định nghĩa hàm mục tiêu huấn luyện là:

$$(\hat{\theta}, \hat{\omega}) \;=\; \arg\min_{(\theta,\omega)} \sum_{i=1}^{m} \mathcal{L}\Big(\mathbf{x}^{(i)}, \text{Decode}_{\omega}(\text{Encode}_{\theta}(\mathbf{x}^{(i)}))\Big) + \alpha \cdot R, \tag{2.97}$$

trong đó R là số hạng chính quy hóa thể hiện một tri thức tiên nghiệm nào đó mà ta muốn áp đặt lên việc huấn luyện, và α là hệ số trọng số của nó. Một lựa chọn phổ biến cho R trong bộ tự mã hóa là **hàm phạt thưa** (*sparsity penalty*) (tạo ra cái được gọi là **bộ tự mã hóa thưa** (*sparse auto-encoder*)). Cách đơn giản nhất để cài đặt hàm phạt này là áp dụng chuẩn l_1 hoặc l_2 lên mã, như sau:

$$R_{l_1} = \sum_{i=1}^{m} \sum_{k} \left| h_k^{(i)} \right|, \tag{2.98}$$

$$R_{l_2} = \sum_{i=1}^{m} \sqrt{\sum_{k} \left(h_k^{(i)}\right)^2}. \tag{2.99}$$

Cần lưu ý rằng, khác với các hàm phạt lên tham số mô hình (xem Mục 2.5.1), hàm phạt thưa chính quy hóa để chính mã **h** (hay đầu ra của bộ mã hóa) trở nên thưa. Ý tưởng khuyến khích tính thưa trong biểu diễn bắt nguồn từ **mã hóa thưa** (*sparse coding*) [Olshausen và Field, 1997]. Ý tưởng này cho rằng thông tin của một đối tượng được ẩn trong những phụ thuộc phức tạp giữa các thuộc tính (hay đặc trưng) ban đầu của đối tượng. Một hệ thống học biểu diễn tốt nên trích xuất những phụ thuộc đó và biến đổi chúng thành một tập các đặc trưng độc lập. Và chỉ nên có một số ít trong các đặc trưng độc lập này được kích hoạt, trong khi các đặc trưng được kích hoạt sẽ thay đổi khi ta chuyển sang một đối tượng mới. Cũng lưu ý rằng, từ góc nhìn học Bayes, các hàm phạt khác trong huấn luyện có chính quy hóa có thể được hiểu là các tiên nghiệm trên mô hình. Tuy nhiên, hàm phạt thưa không phải là một tiên nghiệm, vì nó không phụ thuộc vào mô hình (hay tham số mô hình) mà phụ thuộc vào dữ liệu huấn luyện [Goodfellow và cộng sự, 2016]. Theo quan điểm này, hàm phạt thưa không nên được xem là một số hạng "chính quy hóa", mà chỉ đơn giản là một phân phối nào đó trên các trạng thái trung gian của mô hình. Mặt khác, tính thưa của mã, dù không được giải thích thỏa đáng theo cách dùng thông thường của số hạng chính quy hóa, thực sự hữu ích trong nhiều ứng dụng của bộ tự mã hóa, vì nó trực tiếp mô hình hóa cách biểu diễn đầu vào và áp đặt "tiên nghiệm" lên kết quả của bộ mã hóa. Khi xét từ góc độ thực nghiệm, hàm phạt thưa vẫn được coi là một bộ chính quy hóa, làm thiên lệch việc huấn luyện về phía một số mô hình nhất định.

Ngoài Phương trình (2.98–2.99), còn có những lựa chọn khác để định nghĩa số hạng chính quy hóa R. Ví dụ, một cách ép tính thưa là phạt những trường hợp mà giá trị trung bình của mỗi phần tử h_k cách xa một giá trị định trước [Nair và Hinton, 2009]. Khi h_k nhận giá trị trong $[0, 1]$, việc chính quy hóa có thể được cài đặt bằng cách định nghĩa R là độ phân kỳ KL (*KL divergence*) giữa mã trung bình trên một số mẫu và mã kỳ vọng (*expected code*)[12]. Gọi $\bar{\mathbf{h}}$ là mã trung bình trên $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\}$, trong đó giá trị $\bar{h}_k$ là trung bình của biến thứ k của mã:

$$\bar{h}_k = \frac{1}{m} \cdot \sum_{i=1}^{m} \text{Encode}(\mathbf{x}^{(i)})(k). \tag{2.100}$$

Đồng thời, gọi $\mathbf{q}$ là mã kỳ vọng, trong đó $q_k = \tau$ với mọi k. Nếu mỗi phần tử của mã trung bình được xem là một biến ngẫu nhiên Bernoulli có trung bình $\bar{h}_k$, và mỗi phần tử của mã kỳ vọng được xem là một biến ngẫu nhiên Bernoulli khác có trung bình τ, thì số hạng chính quy hóa có thể được định nghĩa là tổng độ phân kỳ KL giữa $\bar{h}_k$ và q_k trên tất cả các phần tử:

$$\begin{aligned} R &= \sum_k D_{\text{KL}}(q_k \,\|\, \bar{h}_k) \\ &= \sum_k \tau \cdot \log \frac{\tau}{\bar{h}_k} + (1-\tau) \cdot \log \frac{1-\tau}{1-\bar{h}_k}. \end{aligned} \tag{2.101}$$

Ở dạng này, R phạt mô hình khi $\bar{\mathbf{h}}$ lệch khỏi $\mathbf{q}$.

Chú giải của người dịch: Ý nghĩa của Phương trình (2.101) qua ví dụ số

Độ phân kỳ KL đo mức "khác nhau" giữa hai phân phối: bằng 0 khi chúng trùng nhau và dương khi chúng khác nhau. Ở đây mỗi phần tử k được coi như một "công tắc" bật với xác suất $\bar{h}_k$, và ta muốn nó bật với xác suất nhỏ τ. Lấy $\tau = 0.05$ (dùng logarit tự nhiên):

- Nếu $\bar{h}_k = 0.05$: hai số hạng đều là $\log 1 = 0$, nên không bị phạt.
- Nếu $\bar{h}_k = 0.5$ (nơ-ron bật quá thường xuyên): $0.05 \log \frac{0.05}{0.5} + 0.95 \log \frac{0.95}{0.5} \approx -0.115 + 0.610 = 0.495$, bị phạt khá nặng.

Như vậy, số hạng này "kéo" tần suất bật trung bình của mỗi nơ-ron trong mã về gần τ, khiến mã trở nên thưa.

Một biến thể khác của bộ tự mã hóa là **bộ tự mã hóa co** (*contractive auto-encoder, CAE*), cố gắng tăng tính vững của biểu diễn bằng cách đưa một số hạng chính quy hóa mới vào quá trình huấn luyện [Rifai và cộng sự, 2011]:

$$\begin{aligned} R &= \sum_{i=1}^{m} \left\| \frac{\partial \mathbf{h}^{(i)}}{\partial \mathbf{x}^{(i)}} \right\|_F^2 \\ &= \sum_{i=1}^{m} \left\| \frac{\partial \text{Encode}(\mathbf{x}^{(i)})}{\partial \mathbf{x}^{(i)}} \right\|_F^2, \end{aligned} \tag{2.102}$$

trong đó $\frac{\partial \text{Encode}(\mathbf{x}^{(i)})}{\partial \mathbf{x}^{(i)}}$ (hay $\frac{\partial \mathbf{h}^{(i)}}{\partial \mathbf{x}^{(i)}}$) là **ma trận Jacobi** (*Jacobian matrix*) của biểu diễn[13], và $\| \cdot \|_F$ là

[12] Nói chung, ta có thể đặt mọi phần tử của mã kỳ vọng bằng $\tau \in [0, 1]$. Các mã thưa sẽ được ưu tiên nếu τ gần 0, vì trong hầu hết các trường hợp các đặc trưng đều "không hoạt động". Ngược lại, các mã dày đặc sẽ được ưu tiên nếu τ gần 1.

[13] Giả sử bộ mã hóa là một hàm Encode$(\cdot)$: $\mathbf{x} \in \mathbb{R}^{d_x} \to \mathbf{h} \in \mathbb{R}^{d_h}$. Ma trận Jacobi của $\mathbf{h} = \text{Encode}(\mathbf{x})$ là một ma trận

chuẩn Frobenius (*Frobenius norm*) của một ma trận[14]. Hàm phạt co giúp chống lại ảnh hưởng của những nhiễu loạn nhỏ ở đầu vào. Theo nghĩa hình học, nó khuyến khích quan hệ lân cận được giữ nguyên ở các điểm dữ liệu đầu ra nếu các điểm dữ liệu đầu vào là những láng giềng gần nhau; nói cách khác, nó buộc Encode$(\cdot)$ hành xử giống một **ánh xạ co** (*contraction mapping*)[15] hơn, từ đó có tên bộ tự mã hóa co.

Chú giải của người dịch: Ma trận Jacobi và ý nghĩa của hàm phạt co

Mỗi phần tử $\frac{\partial h_j}{\partial x_i}$ của ma trận Jacobi cho biết: nếu ta nhích x_i một chút thì h_j thay đổi nhiều hay ít — giống đạo hàm $f'(x)$ ở THPT, nhưng giờ có nhiều đầu vào và nhiều đầu ra nên ta xếp tất cả các "đạo hàm" đó thành một bảng. Chuẩn Frobenius bình phương chỉ đơn giản là tổng bình phương mọi phần tử của bảng, ví dụ $\left\| \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix} \right\|_F^2 = 1 + 4 + 0 + 1 = 6$. Phạt đại lượng này nghĩa là ép mã $\mathbf{h}$ ít nhạy với những thay đổi nhỏ của $\mathbf{x}$. Ví dụ dễ hình dung về ánh xạ co: $f(x) = x/2$, vì $|f(x_1) - f(x_2)| = \frac{1}{2}|x_1 - x_2|$ — hai điểm gần nhau sau khi qua f lại càng gần nhau hơn.

2.6.2 Bộ tự mã hóa khử nhiễu

Một nguồn cảm hứng khác để tăng tính vững của mô hình đến từ ý tưởng **khử nhiễu** (*denoising*): ta thêm nhiễu vào đầu vào rồi loại bỏ nó để khôi phục dữ liệu sạch. **Bộ tự mã hóa khử nhiễu** (*denoising auto-encoders, DAEs*) là một lớp mạng nơ-ron kết hợp nguyên lý tự mã hóa và khử nhiễu. Đầu tiên, nhiễu được thêm vào vectơ đầu vào một cách ngẫu nhiên. Có thể mô tả điều này như một quá trình sinh ra đầu vào có nhiễu $\mathbf{x}_{\text{noise}} \in \mathbb{R}^{d_x}$ khi biết đầu vào ban đầu $\mathbf{x} \in \mathbb{R}^{d_x}$:

$$\mathbf{x}_{\text{noise}} \sim \Pr_{\text{noise}}(\mathbf{x}_{\text{noise}}|\mathbf{x}), \tag{2.106}$$

trong đó $\Pr_{\text{noise}}(\cdot|\mathbf{x})$ là một phân phối có điều kiện dùng để lấy mẫu $\mathbf{x}_{\text{noise}}$. Ví dụ, ta có thể làm theo phương pháp đã trình bày ở Mục 2.5.5 và rút đầu vào có nhiễu từ một phân phối Gauss nhiều chiều:

$$\mathbf{x}_{\text{noise}} \sim \mathcal{N}(\mathbf{x}, \sigma^2\mathbf{I}), \tag{2.107}$$

$d_h \times d_x$:

$$\begin{aligned} \text{Jacobian} &= \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \\ &= \begin{bmatrix} \frac{\partial \mathbf{h}}{\partial x_1} & \cdots & \frac{\partial \mathbf{h}}{\partial x_{d_x}} \end{bmatrix} \\ &= \begin{bmatrix} \frac{\partial h_1}{\partial x_1} & \cdots & \frac{\partial h_1}{\partial x_{d_x}} \\ \vdots & \ddots & \vdots \\ \frac{\partial h_{d_h}}{\partial x_1} & \cdots & \frac{\partial h_{d_h}}{\partial x_{d_x}} \end{bmatrix}. \end{aligned} \tag{2.103}$$

[14]Với một ma trận $\mathbf{A} \in \mathbb{R}^{d_x \times d_h}$, chuẩn Frobenius về bản chất là chuẩn l_2 của dạng vectơ được "duỗi phẳng" của ma trận, cho bởi phương trình:

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i,j} A_{i,j}^2}. \tag{2.104}$$

Lưu ý rằng trong đại số tuyến tính chặt chẽ, chuẩn l_2 của một ma trận là giá trị kỳ dị lớn nhất của nó (chuẩn phổ); đó là lý do ở đây thuật ngữ chuẩn Frobenius được dùng một cách tường minh để mô tả độ lớn tính theo từng phần tử.

[15]Cho X là một không gian metric với metric d. Cho một hàm $f(\cdot)$ từ X vào X, $f(\cdot)$ là một *ánh xạ co* nếu tồn tại một hằng số $\epsilon \in [0, 1)$ sao cho với mọi $x_1, x_2 \in X$:

$$d(f(x_1), f(x_2)) \leq \epsilon \cdot d(x_1, x_2). \tag{2.105}$$

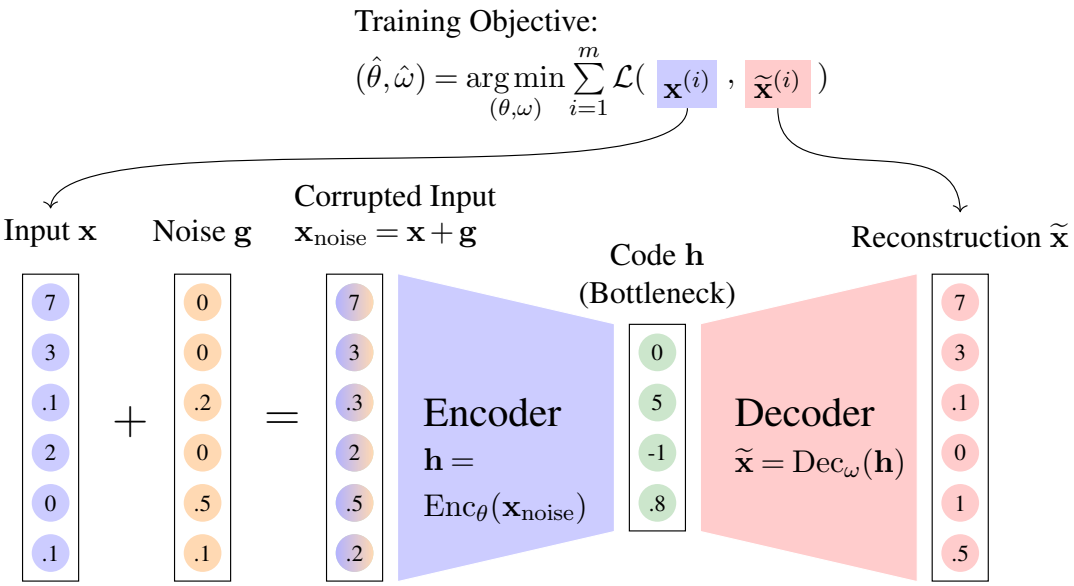
trong đó $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ ký hiệu một phân phối Gauss với vectơ trung bình $\boldsymbol{\mu} \in \mathbb{R}^{d_x}$ và ma trận hiệp phương sai (*covariance matrix*) $\boldsymbol{\Sigma} \in \mathbb{R}^{d_x \times d_x}$. Một cách tương đương, nhiễu cộng này có thể được viết tường minh bằng cách tách riêng số hạng nhiễu có trung bình bằng 0:

$$\mathbf{x}_{\text{noise}} = \mathbf{x} + \mathbf{g}, \tag{2.108}$$

$$\mathbf{g} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}), \tag{2.109}$$

trong đó $\sigma^2\mathbf{I}$ là một ma trận hiệp phương sai đường chéo, cho biết nhiễu Gauss độc lập với phương sai σ^2 được áp dụng vào từng chiều của đầu vào, và $\mathbf{I}$ là ma trận đơn vị $d_x \times d_x$.

Đôi khi quá trình này được gọi là sự **làm hỏng** (*corruption*) đầu vào, và $\mathbf{x}_{\text{noise}}$ được gọi là **đầu vào bị làm hỏng** (*corrupted input*). Ngoài nhiễu Gauss cộng, còn có một vài cách khác để làm hỏng đầu vào [Vincent và cộng sự, 2010]. Một trong những phương pháp phổ biến là đặt bằng 0 một số phần tử của $\mathbf{x}$. Ví dụ, ta có thể đặt mỗi phần tử bằng 0 với một xác suất định trước. Cách này còn được gọi là **nhiễu che** (*masking noise*). Một phương pháp khác là dùng **nhiễu muối tiêu** (*salt-and-pepper noise*) hay **nhiễu xung** (*impulse noise*) để làm hỏng. Nó chọn ngẫu nhiên một số phần tử và đặt mỗi phần tử đó bằng giá trị nhỏ nhất hoặc lớn nhất với một xác suất định trước. Các loại nhiễu khác nhau được dùng cho những ứng dụng khác nhau của bộ tự mã hóa. Ví dụ, nhiễu che rất phổ biến trong việc huấn luyện mô hình ngôn ngữ, còn nhiễu muối tiêu được dùng phổ biến hơn trong xử lý ảnh.



Hình 2.17: Minh họa một bộ tự mã hóa khử nhiễu. Một đầu vào $\mathbf{x}$ trước tiên bị làm hỏng thành đầu vào có nhiễu hay đầu vào bị làm hỏng $\mathbf{x}_{\text{noise}}$. Sau đó, nó được đưa qua một bộ mã hóa để tạo thành mã $\mathbf{h}$. Tiếp theo, mã được đưa qua một bộ giải mã để tạo thành đầu vào tái tạo $\widetilde{\mathbf{x}}$. Việc huấn luyện được thực hiện bằng cách cực tiểu hóa mất mát giữa $\mathbf{x}$ và $\widetilde{\mathbf{x}}$. Quá trình này được gọi là "khử nhiễu" vì nó cố gắng loại bỏ nhiễu khỏi $\mathbf{x}_{\text{noise}}$ và khôi phục đầu vào ban đầu $\mathbf{x}$.

Nhãn trong hình: *Noise*: nhiễu; *Corrupted Input*: đầu vào bị làm hỏng; các nhãn khác như ở Hình 2.16.

Sau đó, đầu vào bị làm hỏng $\mathbf{x}_{\text{noise}}$ được đưa vào một mạng mã hóa–giải mã, và mạng này tạo ra đầu vào tái tạo $\widetilde{\mathbf{x}} = \text{Decode}(\text{Encode}(\mathbf{x}_{\text{noise}}))$. Quá trình huấn luyện diễn ra như bình thường. Ta dùng lại Phương trình (2.95) để cực tiểu hóa mất mát khi thay $\mathbf{x}$ bằng $\widetilde{\mathbf{x}}$. Như vậy, ta có thể viết lại Phương

trình (2.95) để điều chỉnh hàm mục tiêu cho trường hợp khử nhiễu:

$$(\hat{\theta}, \hat{\omega}) = \arg\min_{(\theta,\omega)} \sum_{i=1}^{m} \mathcal{L}\Big(\mathbf{x}^{(i)}, \text{Decode}_{\omega}(\text{Encode}_{\theta}(\mathbf{x}^{(i)}_{\text{noise}}))\Big). \tag{2.110}$$

Phương trình (2.110) chỉ khác Phương trình (2.95) ở chỗ đầu vào của bộ tự mã hóa là $\mathbf{x}_{\text{noise}}$ thay vì $\mathbf{x}$. Nói cách khác, ta khử nhiễu đầu vào bị làm hỏng để khôi phục đầu vào ban đầu. Xem Hình 2.17 để biết cấu trúc của bộ tự mã hóa khử nhiễu.

Chú giải của người dịch: Khử nhiễu: "đoán chữ bị che"

Hãy nhớ lại trò chơi điền vào chỗ trống: "Hà Nội là ___ đô của Việt Nam". Muốn điền đúng chữ "thủ", ta phải hiểu cả câu. Bộ tự mã hóa khử nhiễu làm đúng như vậy: đầu vào bị làm hỏng (một số phần bị che hoặc bị nhiễu), nhưng đáp án mà mô hình phải tái tạo là bản *sạch* $\mathbf{x}$. Vì không thể chỉ "chép" đầu vào, mô hình buộc phải học quy luật bên trong dữ liệu. Đây chính là ý tưởng "nhiễu che" mà các mô hình ngôn ngữ như BERT (Chương 7) dùng khi che ngẫu nhiên một số từ rồi bắt mô hình đoán lại.

Lưu ý rằng cả bộ tự mã hóa co/thưa lẫn bộ tự mã hóa khử nhiễu đều có thể được xem là những cách tăng tính vững của bộ tự mã hóa. Điểm khác biệt nằm ở chỗ chúng chính quy hóa việc huấn luyện tại những vị trí khác nhau của mô hình. Bộ tự mã hóa co nhắm tới việc tăng tính vững của quá trình mã hóa, tức là biểu diễn được huấn luyện để không quá nhạy với những nhiễu loạn nhỏ ở đầu vào. Trong khi đó, bộ tự mã hóa khử nhiễu nhắm tới việc tăng tính vững của quá trình tái tạo. Chúng tác động đồng thời lên cả bộ mã hóa lẫn bộ giải mã. Theo một nghĩa nào đó, bộ tự mã hóa khử nhiễu là ứng dụng trực tiếp của huấn luyện có nhiễu vào bộ tự mã hóa (xem Mục 2.5.5). Tất nhiên, khó có thể nói mô hình nào tốt hơn. Ví dụ, bộ tự mã hóa co có sự dẫn dắt trực tiếp hơn đối với việc học biểu diễn, vốn thường là mục tiêu chính của chúng ta trong học đặc trưng. Việc huấn luyện bộ tự mã hóa khử nhiễu, dù chỉ có tác động gián tiếp lên quá trình mã hóa, lại nhận thêm tín hiệu khử nhiễu từ bộ giải mã. Điều này mang lại một góc nhìn mới về huấn luyện vững: một biểu diễn vững có thể được học ở cả điểm sinh ra nó (bộ mã hóa khử nhiễu) lẫn điểm sử dụng nó (bộ giải mã khử nhiễu).

2.6.3 Bộ tự mã hóa biến phân

Dù mang tên như vậy, **bộ tự mã hóa biến phân** (*variational auto-encoders, VAEs*) ban đầu không được đề xuất để giải các bài toán mã hóa biểu diễn thông thường. Về cơ bản, chúng được thiết kế để sinh ra dữ liệu mới tương tự dữ liệu quan sát được, do đó có cách phát biểu toán học rất khác so với các bộ tự mã hóa kinh điển đã bàn ở trên. Trong thống kê và học máy, bộ tự mã hóa biến phân thường được xem là những ví dụ của các phương pháp Bayes biến phân (*variational Bayesian methods*) và được dùng để thực hiện suy diễn thống kê (*statistical inference*) hiệu quả trên các biến ẩn (*latent variables*) khi xác suất hậu nghiệm của các biến này không tính được (*intractable*) [Kingma và Welling, 2014; 2019]. Mặt khác, bộ tự mã hóa biến phân, dù ngầm định hay tường minh, cũng giải quyết đúng việc mà ta làm khi suy ra biểu diễn ẩn bên dưới của một đối tượng quan sát được. Vì vậy, chúng tôi đưa chúng vào mục này để phần thảo luận được tương đối đầy đủ.

Ta bắt đầu với một "câu chuyện sinh" mô tả cách mỗi điểm dữ liệu được tạo ra. Giả sử rằng, với một mẫu quan sát được $\mathbf{x}$ trong tập dữ liệu, có một biến ẩn $\mathbf{h}$ không quan sát được mô tả $\mathbf{x}$. Bây giờ ta muốn xây dựng một mô hình xác suất mô tả quá trình sinh ra $\mathbf{x}$, cụ thể là ước lượng xác suất $\Pr(\mathbf{x})$. Xác suất này có thể được tính như phân phối biên (*marginal distribution*):

$$\Pr(\mathbf{x}) = \int \Pr(\mathbf{x}, \mathbf{h}) d\mathbf{h}, \tag{2.111}$$

trong đó ta đưa biến ẩn $\mathbf{h}$ một cách tường minh vào việc suy diễn $\mathbf{x}$. Để giải Phương trình (2.111), ta dùng một mô hình $p_\omega(\mathbf{x}, \mathbf{h})$ để xấp xỉ $\Pr(\mathbf{x}, \mathbf{h})$ (tức $p_\omega(\mathbf{x}, \mathbf{h}) \approx \Pr(\mathbf{x}, \mathbf{h})$), và ta có

$$p_\omega(\mathbf{x}) \;=\; \int p_\omega(\mathbf{x}, \mathbf{h}) d\mathbf{h}, \tag{2.112}$$

trong đó $p_\omega(\mathbf{x}, \mathbf{h})$ là một hàm mật độ xác suất (*probability density function*) được tham số hóa bởi ω. Ta thay vế trái của Phương trình (2.112) bằng $p_\omega(\mathbf{x})$ để nhấn mạnh rằng xác suất được xác định bởi mô hình $p_\omega(\cdot)$. Nói chung có nhiều cách định nghĩa $p_\omega(\mathbf{x}, \mathbf{h})$. Ở đây ta có thể đơn giản coi nó là một mạng nơ-ron.

Sau đó, ta có thể viết lại Phương trình (2.112) bằng quy tắc chuỗi (*chain rule*) (của xác suất):

$$p_\omega(\mathbf{x}) \;=\; \int p_\omega(\mathbf{h}) \cdot p_\omega(\mathbf{x}|\mathbf{h}) d\mathbf{h}, \tag{2.113}$$

trong đó $p_\omega(\mathbf{h})$ là phân phối tiên nghiệm (*prior*) trên $\mathbf{h}$, ví dụ một tiên nghiệm Gauss. Xác suất có điều kiện $p_\omega(\mathbf{x}|\mathbf{h})$ mô tả khả năng quan sát được $\mathbf{x}$ khi biết biến ẩn $\mathbf{h}$. Để mô hình hóa quá trình sinh này, $p_\omega(\mathbf{x}|\mathbf{h})$ thường được giả định là một phân phối Gauss được tham số hóa bởi trung bình $\boldsymbol{\mu}_p$ và phương sai σ_p của nó:

$$p_\omega(\mathbf{x}|\mathbf{h}) \;=\; \mathcal{N}(\boldsymbol{\mu}_p, \sigma_p^2 \mathbf{I}), \tag{2.114}$$

trong đó $\boldsymbol{\mu}_p \in \mathbb{R}^{d_x}$ và $\sigma_p \in \mathbb{R}$ được xác định bởi một mạng giải mã $\text{Decode}_\omega(\cdot)$ (chúng tôi sẽ giải thích sau vì sao nó được gọi là "giải mã"):

$$(\boldsymbol{\mu}_p, \sigma_p) \;=\; \text{Decode}_\omega(\mathbf{h}). \tag{2.115}$$

Chú giải của người dịch: Biến ẩn và "câu chuyện sinh"

Hãy hình dung việc vẽ một khuôn mặt: trước tiên ta (ngầm) chọn vài "yếu tố ẩn" như tuổi, kiểu tóc, nụ cười — đó là $\mathbf{h}$; rồi từ các yếu tố đó mới vẽ ra bức ảnh cụ thể — đó là $\mathbf{x}$. Ta chỉ nhìn thấy bức ảnh, không thấy $\mathbf{h}$. Phương trình (2.111) nói rằng xác suất của một bức ảnh bằng "tổng" (tích phân) xác suất của mọi cách chọn yếu tố ẩn có thể dẫn tới bức ảnh đó — giống công thức xác suất đầy đủ: $P(A) = \sum_i P(B_i)\, P(A|B_i)$, chỉ khác là $\mathbf{h}$ liên tục nên tổng thành tích phân. Ký hiệu $p(\mathbf{x}|\mathbf{h})$ đọc là "xác suất của $\mathbf{x}$ khi đã biết $\mathbf{h}$" (xác suất có điều kiện).

Tuy nhiên, hàm hợp lý biên (*marginal likelihood*) trong Phương trình (2.112) vẫn không tính được. Dù ta có thể dễ dàng tính tiên nghiệm $p_\omega(\mathbf{h})$ và xác suất có điều kiện $p_\omega(\mathbf{x}|\mathbf{h})$, việc thực hiện chính xác phép tích phân trên toàn bộ không gian ẩn liên tục của $\mathbf{h}$ là quá tốn kém về tính toán. Điều này cũng dẫn tới một phân phối hậu nghiệm (*posterior*) không tính được:

$$p_\omega(\mathbf{h}|\mathbf{x}) \;=\; \frac{p_\omega(\mathbf{h}) \cdot p_\omega(\mathbf{x}|\mathbf{h})}{p_\omega(\mathbf{x})}. \tag{2.116}$$

Có vẻ như ta bị mắc kẹt với $p_\omega(\mathbf{x})$ và $p_\omega(\mathbf{h}|\mathbf{x})$! Bộ tự mã hóa biến phân giải quyết vấn đề này bằng cách xấp xỉ $p_\omega(\mathbf{h}|\mathbf{x})$ bằng một phân phối hậu nghiệm tính được $q_\theta(\mathbf{h}|\mathbf{x})$:

$$q_\theta(\mathbf{h}|\mathbf{x}) \;\approx\; p_\omega(\mathbf{h}|\mathbf{x}), \tag{2.117}$$

trong đó θ là tham số của mô hình mới. Giống như Phương trình (2.114–2.115), $q_\theta(\mathbf{h}|\mathbf{x})$ được định nghĩa là một phân phối Gauss khác:

$$q_\theta(\mathbf{h}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_q, \sigma_q^2 \mathbf{I}), \tag{2.118}$$

$$(\boldsymbol{\mu}_q, \sigma_q) = \text{Encode}_\theta(\mathbf{x}), \tag{2.119}$$

trong đó $\text{Encode}_\theta(\cdot)$ là mạng mã hóa đọc $\mathbf{x}$ và sinh ra trung bình và phương sai của phân phối $q_\theta(\mathbf{h}|\mathbf{x})$. Điều này thật thú vị! Giờ đây ta có một con đường khả thi để tính $\Pr(\mathbf{x})$: ta có thể lấy mẫu một biến ẩn $\mathbf{h}$ từ phân phối hậu nghiệm biến phân (*variational posterior*) $q_\theta(\mathbf{h}|\mathbf{x})$, rồi tính xác suất đồng thời (*joint probability*) $p_\omega(\mathbf{x}, \mathbf{h})$. Về mặt toán học, ta có thể viết lại log-hợp lý (*log-likelihood*) của quan sát dưới dạng một kỳ vọng theo phân phối hậu nghiệm biến phân:

$$\begin{aligned}
&\log p_\omega(\mathbf{x}) \\
&= \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\Big[\log p_\omega(\mathbf{x})\Big] \\
&= \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\Big[\log \frac{p_\omega(\mathbf{h})\cdot p_\omega(\mathbf{x}|\mathbf{h})}{p_\omega(\mathbf{h}|\mathbf{x})}\Big] \\
&= \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\Big[\log \frac{p_\omega(\mathbf{h})\cdot p_\omega(\mathbf{x}|\mathbf{h})}{p_\omega(\mathbf{h}|\mathbf{x})}\cdot\frac{q_\theta(\mathbf{h}|\mathbf{x})}{q_\theta(\mathbf{h}|\mathbf{x})}\Big] \\
&= \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\Big[\log \frac{q_\theta(\mathbf{h}|\mathbf{x})}{p_\omega(\mathbf{h}|\mathbf{x})}\Big] + \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\Big[\log \frac{p_\omega(\mathbf{h})\cdot p_\omega(\mathbf{x}|\mathbf{h})}{q_\theta(\mathbf{h}|\mathbf{x})}\Big].
\end{aligned} \tag{2.120}$$

Chú giải của người dịch: Đọc từng bước của Phương trình (2.120)

- Bước 1: $\log p_\omega(\mathbf{x})$ không phụ thuộc vào $\mathbf{h}$, nên "lấy trung bình" nó theo $\mathbf{h}$ vẫn được chính nó (trung bình của một hằng số là hằng số đó).
- Bước 2: dùng công thức Bayes (2.116) đảo lại: $p_\omega(\mathbf{x}) = \frac{p_\omega(\mathbf{h})\,p_\omega(\mathbf{x}|\mathbf{h})}{p_\omega(\mathbf{h}|\mathbf{x})}$.
- Bước 3: nhân với $\frac{q}{q} = 1$, không làm thay đổi giá trị — một "mẹo" quen thuộc kiểu thêm bớt ở THPT.
- Bước 4: dùng $\log(ab) = \log a + \log b$ để tách thành hai phần, và kỳ vọng của một tổng bằng tổng các kỳ vọng.

Ký hiệu $\mathbb{E}_{\mathbf{h}\sim q}[\cdot]$ là *kỳ vọng* (giá trị trung bình) của biểu thức trong ngoặc khi $\mathbf{h}$ được rút ngẫu nhiên theo phân phối q.

Số hạng thứ nhất ở vế phải của Phương trình (2.120) chính là độ phân kỳ KL (hay entropy tương đối (*relative entropy*)) giữa phân phối hậu nghiệm xấp xỉ $q_\theta(\mathbf{h}|\mathbf{x})$ và phân phối hậu nghiệm thật $p_\omega(\mathbf{h}|\mathbf{x})$:

$$D_{\text{KL}}\big(q_\theta(\mathbf{h}|\mathbf{x}) \,\|\, p_\omega(\mathbf{h}|\mathbf{x})\big) \;=\; \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\Big[\log \frac{q_\theta(\mathbf{h}|\mathbf{x})}{p_\omega(\mathbf{h}|\mathbf{x})}\Big]. \tag{2.121}$$

Vì độ phân kỳ KL luôn không âm ($D_{\text{KL}}(q\,\|\,p) \geq 0$, và bằng 0 khi và chỉ khi $q = p$), ta có thể thiết lập một cận dưới chặt chẽ cho log-hợp lý:

$$\begin{aligned}
\log p_\omega(\mathbf{x}) &\geq \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\Big[\log \frac{p_\omega(\mathbf{h})\cdot p_\omega(\mathbf{x}|\mathbf{h})}{q_\theta(\mathbf{h}|\mathbf{x})}\Big] \\
&= \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\Big[\log p_\omega(\mathbf{x}|\mathbf{h}) + \log \frac{p_\omega(\mathbf{h})}{q_\theta(\mathbf{h}|\mathbf{x})}\Big] \\
&= \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\big[\log p_\omega(\mathbf{x}|\mathbf{h})\big] - D_{\text{KL}}\big(q_\theta(\mathbf{h}|\mathbf{x}) \,\|\, p_\omega(\mathbf{h})\big).
\end{aligned} \tag{2.122}$$

Vế phải của Phương trình (2.122) tạo thành **cận dưới bằng chứng** (*evidence lower bound, ELBO*) nền tảng của hàm hợp lý dữ liệu. Số hạng thứ nhất của ELBO (log-hợp lý tái tạo kỳ vọng) có thể được xấp xỉ bằng cách lấy mẫu Monte Carlo (*Monte Carlo sampling*) $\mathbf{h}$. Số hạng thứ hai dễ tính vì độ phân kỳ KL giữa hai phân phối Gauss có lời giải giải tích. Gọi $\mathcal{L}(\mathbf{x}, \theta, \omega)$ là ELBO lấy dấu âm. Khi đó, quá

trình huấn luyện một bộ tự mã hóa biến phân được phát biểu là cực tiểu hóa $\mathcal{L}(\cdot)$ trên một tập dữ liệu gồm các mẫu quan sát được $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\}$:

$$(\hat{\theta}, \hat{\omega}) \;=\; \arg\min_{(\theta,\omega)} \sum_{i=1}^{m} \mathcal{L}\Big(\mathbf{x}^{(i)}, \theta, \omega\Big). \tag{2.123}$$

Lưu ý rằng vì việc lấy mẫu $\mathbf{h}$ từ $q_\theta(\mathbf{h} \mid \mathbf{x})$ là một phép toán ngẫu nhiên, không khả vi, nên không thể tính trực tiếp gradient của nó. Để đưa việc huấn luyện bộ tự mã hóa vào khuôn khổ lan truyền ngược tiêu chuẩn, một cách thường dùng là sử dụng cái gọi là **mẹo tái tham số hóa** (*reparameterization trick*). Ở đây chúng tôi bỏ qua chi tiết và giới thiệu bạn đọc một vài bài báo để biết thêm thông tin [Kingma và Welling, 2014; Doersch, 2016].

Chú giải của người dịch: ELBO nói gì một cách trực quan?

Phương trình (2.122) cho thấy cực đại hóa ELBO là cân bằng hai mong muốn: (i) số hạng thứ nhất — từ mã $\mathbf{h}$ phải tái tạo lại $\mathbf{x}$ thật tốt (giống mất mát tái tạo của bộ tự mã hóa thông thường); (ii) số hạng thứ hai — phân phối mã $q_\theta(\mathbf{h}|\mathbf{x})$ không được lệch quá xa phân phối tiên nghiệm $p_\omega(\mathbf{h})$ (thường là phân phối Gauss chuẩn), giống như một số hạng chính quy hóa. Nhờ (ii), không gian mã trở nên "gọn gàng", nên sau này rút ngẫu nhiên một $\mathbf{h}$ từ tiên nghiệm rồi giải mã sẽ ra dữ liệu hợp lý.
Về mẹo tái tham số hóa (chỉ nêu ý chính): thay vì rút trực tiếp $\mathbf{h} \sim \mathcal{N}(\boldsymbol{\mu}_q, \sigma_q^2\mathbf{I})$, ta rút $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ rồi tính $\mathbf{h} = \boldsymbol{\mu}_q + \sigma_q \cdot \boldsymbol{\varepsilon}$. Tính ngẫu nhiên được "đẩy" sang $\boldsymbol{\varepsilon}$, còn $\mathbf{h}$ là một hàm bình thường của $\boldsymbol{\mu}_q, \sigma_q$ nên lấy đạo hàm và lan truyền ngược được.

Hình 2.18 minh họa cách một bộ tự mã hóa biến phân hoạt động. Nó gồm một quá trình sinh hai bước:

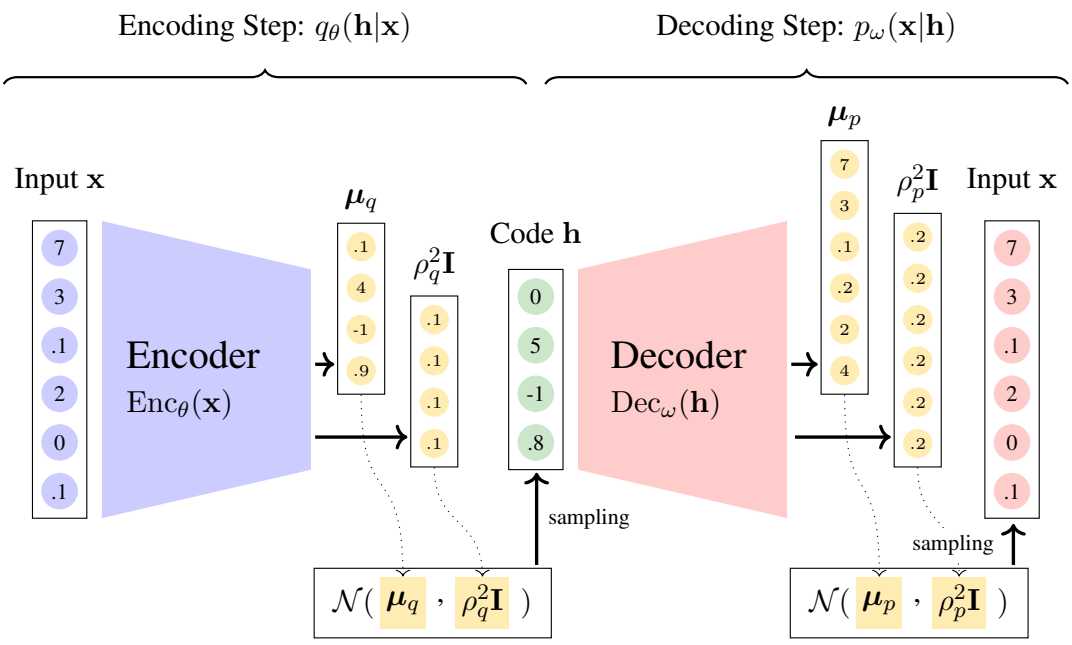
- **Mã hóa**. Với một mẫu đầu vào $\mathbf{x}$, ta lấy mẫu một biến ẩn $\mathbf{h}$ từ $q_\theta(\mathbf{h}|\mathbf{x})$. Bước này cần một mạng mã hóa $\text{Encode}_\theta(\mathbf{x})$ sinh ra trung bình $\boldsymbol{\mu}_q$ và phương sai σ_q^2 của phân phối Gauss $q_\theta(\mathbf{h}|\mathbf{x})$. Sau đó, biến ẩn được sinh ra bằng cách lấy mẫu từ $\mathcal{N}(\boldsymbol{\mu}_q, \sigma_q^2\mathbf{I})$.
- **Giải mã**. Với biến ẩn $\mathbf{h}$, ta lấy mẫu đầu vào ban đầu $\mathbf{x}$ từ $p_\omega(\mathbf{x}|\mathbf{h})$. Bước này lại theo một quá trình lấy mẫu Gauss: một mạng giải mã $\text{Decode}_\omega(\mathbf{h})$ được dùng để xác định trung bình $\boldsymbol{\mu}_p$ và phương sai σ_p^2 của phân phối. $\mathbf{x}$ được sinh ra theo $\mathcal{N}(\boldsymbol{\mu}_p, \sigma_p^2\mathbf{I})$.

Nhãn trong hình: *Encoding Step/Decoding Step*: bước mã hóa/bước giải mã; *sampling*: lấy mẫu; các nhãn khác như ở Hình 2.16. Ký hiệu $\rho_q^2\mathbf{I}$, $\rho_p^2\mathbf{I}$ trong hình ứng với $\sigma_q^2\mathbf{I}$, $\sigma_p^2\mathbf{I}$ trong văn bản.

Đôi khi $q_\theta(\mathbf{h}|\mathbf{x})$ và $p_\omega(\mathbf{x}|\mathbf{h})$ được gọi là "bộ mã hóa" và "bộ giải mã", vì chúng cố gắng "ánh xạ" một đầu vào thành một biểu diễn rồi "ánh xạ" nó trở lại đầu vào. Tuy nhiên, bản thân $q_\theta(\mathbf{h}|\mathbf{x})$ và $p_\omega(\mathbf{x}|\mathbf{h})$ hàm ý những mô hình không tất định, tức là chúng xuất ra hàm mật độ xác suất của các biến thay vì các ước lượng điểm (*point estimates*). Một hệ quả quan trọng của kết quả này là bộ tự mã hóa biến phân không có xu hướng tìm biểu diễn "tốt nhất" cho đầu vào. Thoạt nhìn, điều này có vẻ phản trực giác, vì các mô hình thông thường thường cho ra các biểu diễn có giá trị cố định. Tuy nhiên, đây chính là đặc điểm của suy diễn Bayes — ta chỉ học một phân phối trên các giá trị khả dĩ của một biến ẩn. Từ góc độ thực tiễn, nếu tác vụ hạ nguồn cần một biểu diễn tất định, ta có thể đơn giản dùng giá trị kỳ vọng (tức trung bình $\boldsymbol{\mu}_q$) của phân phối suy ra được, vốn nắm bắt một cách đáng tin cậy các đặc trưng thiết yếu của đầu vào [Knight, 2009].

Trong thực tế, công dụng chính của bộ tự mã hóa biến phân là để sinh dữ liệu chứ không phải để biểu diễn. Khi kiểm tra, với các tham số đã tối ưu $\hat{\theta}$ và $\hat{\omega}$, bộ mã hóa (tức $q_{\hat{\theta}}(\mathbf{h}|\mathbf{x})$) bị loại bỏ, và bộ giải mã (tức $p_{\hat{\omega}}(\mathbf{x}|\mathbf{h})$) làm việc với các $\mathbf{h}$ được sinh ngẫu nhiên. Chính xác hơn, ta lấy mẫu một biến ẩn $\mathbf{h}_{\text{new}}$ từ một phân phối Gauss, rồi suy ra một mẫu $\mathbf{x}_{\text{new}}$ bằng $p_{\hat{\omega}}(\mathbf{x}|\mathbf{h}_{\text{new}})$ như thường lệ. Trong các chương

$$\text{Training Objective: } (\hat{\theta}, \hat{\omega}) = \arg\min_{(\theta,\omega)} \sum_{i=1}^{m} \mathcal{L}(\mathbf{x}^{(i)}, \theta, \omega)$$



Hình 2.18: Minh họa một bộ tự mã hóa biến phân. Với một mẫu đầu vào $\mathbf{x}$, ta sinh ra một biến ẩn $\mathbf{h}$ bằng một bộ mã hóa $q_\theta(\mathbf{h}|\mathbf{x})$. Trong bước mã hóa, trước tiên một mạng nơ-ron $\text{Enc}_\theta(\cdot)$ được dùng để tạo ra trung bình và phương sai của một phân phối Gauss, tức $\boldsymbol{\mu}_q$ và σ_q^2. Sau đó, biến ẩn $\mathbf{h}$ được rút theo $\mathcal{N}(\boldsymbol{\mu}_q, \sigma_q^2\mathbf{I})$. Tiếp theo, ta tái sinh mẫu ban đầu $\mathbf{x}$ bằng một bộ giải mã $p_\omega(\mathbf{x}|\mathbf{h})$. Trong bước giải mã, tương tự quá trình sinh ở bộ mã hóa, một mạng nơ-ron $\text{Dec}_\omega(\cdot)$ được dùng để sinh ra trung bình $\boldsymbol{\mu}_p$ và phương sai σ_p^2 của $\mathcal{N}(\boldsymbol{\mu}_p, \sigma_p^2\mathbf{I})$. Cùng đầu vào $\mathbf{x}$ đó được sinh ra bằng cách lấy mẫu từ $\mathcal{N}(\boldsymbol{\mu}_p, \sigma_p^2\mathbf{I})$.

sau, chúng ta sẽ thấy nhiều bài toán NLP có thể được xếp vào loại bài toán sinh, trong đó các đối tượng dữ liệu dạng chuỗi hoặc dạng phân cấp được sinh ra với điều kiện là một số đối tượng dữ liệu hoặc biến ẩn cho trước.

2.7 Tóm tắt

Trong chương này, chúng ta đã bàn về mạng nơ-ron là gì, cũng như một vài kiến trúc cơ bản thường được dùng làm khối xây dựng để phát triển các hệ thống học sâu mạnh mẽ. Ngoài ra, chúng ta đã xem xét cách huấn luyện mạng nơ-ron, cách chính quy hóa quá trình huấn luyện, và cách áp dụng mạng nơ-ron vào việc học đặc trưng theo cách không giám sát.

Tuy nhiên, mạng nơ-ron và học sâu là những chủ đề rất rộng, và những gì chúng ta thảo luận mới chỉ chạm tới bề mặt. Để có một phần giới thiệu toàn diện hơn, cuốn sách của Goodfellow và cộng sự [2016] là một tài liệu tuyệt vời. Cuốn sách này cũng đề cập một số kỹ thuật nâng cao, như các mô hình có cấu trúc sâu và các phương pháp ngẫu nhiên hóa, để phát triển các hệ thống tiên tiến nhất. Như mọi khi, có một khoảng cách đáng kể giữa việc hiểu một kỹ thuật về mặt lý thuyết và việc thành thạo áp dụng nó để giải quyết các bài toán thực tế. Vì vậy, với những người làm thực tế muốn áp dụng mạng nơ-ron và học sâu trong nhiều tình huống khác nhau, có nhiều cuốn sách trình bày chi tiết việc cài đặt các hệ thống học sâu [Géron, 2019; Zhang và cộng sự, 2021a; Chollet, 2021], cùng với các dự án mã nguồn mở cung cấp mã nguồn tham khảo[16].

Trong các chương tiếp theo, chúng ta sẽ tập trung vào cách dùng các mô hình nơ-ron để giải quyết các bài toán NLP. Trên hành trình đó, chúng ta sẽ tìm hiểu cách học biểu diễn của từ và câu bằng các phương pháp đã bàn tới đây (Chương 3–4), và cách mô hình hóa nhiều bài toán NLP khác nhau bằng một số phương pháp nâng cao dựa trên mạng nơ-ron. Những phương pháp này bao gồm cơ chế chú ý và Transformer (Chương 5–6), tiền huấn luyện (Chương 7), và các mô hình ngôn ngữ lớn (Chương 8–11).

[16] Địa chỉ của một vài hướng dẫn trực tuyến phổ biến: `https://pytorch.org/tutorials`, `https://keras.io/examples/nlp`, và `https://www.tensorflow.org/tutorials`

Phần II

Các mô hình cơ bản

Chương 3

Từ và vectơ từ

Từ là đơn vị cơ bản của ngôn ngữ [Jackendoff, 1992]. Hầu hết các hệ thống ngôn ngữ tự nhiên dùng để diễn đạt và giao tiếp đều liên quan đến việc tạo ra, cấu thành và kết hợp các từ. Trước khi tìm hiểu cách các từ tạo nên những đơn vị ngôn ngữ lớn hơn, ta nên hiểu trước *từ* là gì. Điều này dẫn tới hai câu hỏi nền tảng:

- Hình thức bề mặt của một từ là gì?
- Nghĩa của một từ là gì?

Tuy nhiên, trả lời các câu hỏi này không dễ, vì không có quy tắc đơn giản nào mô tả đầy đủ cách các từ được cấu tạo, hay cách nghĩa của chúng được xác định và suy ra. Trong khi nhiều lý thuyết ngôn ngữ học khác nhau đã đề cập đến những câu hỏi này, các nhà nghiên cứu NLP lại quan tâm nhiều hơn đến hai vấn đề thực tiễn:

- **Tách token** (*tokenization*): Cho một chuỗi ký tự, làm thế nào để phân đoạn nó thành một chuỗi các từ (còn gọi là các *token*) sao cho các đơn vị này có thể dùng được trong các tác vụ NLP phía sau?
- **Học biểu diễn từ** (*word representation learning*): Cho một kho ngữ liệu (*corpus*), làm thế nào để biểu diễn mỗi từ dưới dạng có thể tính toán được, và các mô hình NLP thực hiện tính toán trên biểu diễn đó như thế nào?

Một mục tiêu của chương này là trình bày cách các câu được phân đoạn bằng phương pháp ngôn ngữ học hoặc phương pháp thống kê. Cụ thể, chúng tôi mô tả một số cách tách một chuỗi ký tự thành các từ hoặc từ con (*subword*), dùng các quy tắc kinh nghiệm hoặc các mô hình thống kê học từ dữ liệu. Mục tiêu còn lại là cho thấy cách biểu diễn các từ bằng các vectơ số thực. Chúng tôi trình bày các phương pháp hiện đại để học và đánh giá các vectơ từ (*word vector*) này. Trọng tâm của phần này không phải là việc suy diễn công thức và mô hình, mà là khái niệm cốt lõi về biểu diễn từ bằng vectơ — nền tảng của rất nhiều hệ thống NLP. Trong các chương sau, ta sẽ thấy ý tưởng này được tổng quát hóa để mô hình hóa những đơn vị ngôn ngữ lớn hơn, chẳng hạn như câu.

3.1 Tách token

Trong khoa học máy tính và các lĩnh vực liên quan, thuật ngữ *token* có thể được dùng theo nhiều cách khác nhau. Ở đây ta đơn giản coi một token là một *từ* theo nghĩa ngôn ngữ học, dù nó cũng có thể là thứ khác (xem Mục 3.1.4). Trong NLP, **tách token** (*tokenization*) hay **phân đoạn** (*segmentation*) là một tác vụ liên quan đến phân tích hình thái (*morphological analysis*) [Aronoff and Fudeman, 2011]. Trong khi

các bộ phân tích hình thái thường được dùng để nghiên cứu cấu trúc bên trong của từ, việc tách token quan tâm đến cách các câu được chia nhỏ thành các từ. Có vẻ như ta cần biết từ được cấu tạo ra sao nếu muốn biết câu được tạo thành từ các từ như thế nào. Sự phức tạp này còn tăng thêm bởi tính đa dạng của các ngôn ngữ, khiến việc xây dựng một hệ thống chung để mô tả hình thái học trở nên khó khăn. Ví dụ, các ngôn ngữ đơn lập (*analytic language*) (như tiếng Trung) có rất ít biến tố (*inflection*) và dựa vào trật tự từ để truyền đạt ý nghĩa. Ngược lại, các ngôn ngữ tổng hợp (*synthetic language*) (như tiếng Pháp) có thể có hệ thống biến tố phong phú, và nghĩa của một từ chịu ảnh hưởng lớn bởi hình thái.

Tiếng Trung	Đầu vào:	一直以来，完美的机器翻译是人类的梦想之一。
	Đầu ra:	一直/以来/，/完美/的/机器翻译/是/人类/的/梦想/之一/。
Tiếng Nhật	Đầu vào:	西日本や海はく晴れて、汗ばむ暑さとなる。
	Đầu ra:	西日本/や/海/は/く/晴れて/、/汗ばむ/暑さ/と/なる/。
Tiếng Anh	Đầu vào:	She said, “Deep learning is not the solution to all world’s problems”.
	Đầu ra:	She/said/,/“/Deep/learning/is/not/the/solution/to/all/world/’s/problems/”/.

Hình 3.1: Tách token cho các ngôn ngữ khác nhau (dấu gạch chéo = ranh giới từ). Với tiếng Trung và tiếng Nhật, nơi không có dấu phân cách giữa các từ, việc tách token thường được gọi là *phân đoạn từ* (*word segmentation*).

Chú giải của người dịch: Nghĩa của các câu ví dụ trong Hình 3.1

Câu tiếng Trung có nghĩa: “Từ trước đến nay, dịch máy hoàn hảo là một trong những ước mơ của nhân loại.” Câu tiếng Nhật đại ý: “Vùng Tây Nhật Bản và biển trời quang đãng, nắng nóng đến đổ mồ hôi.” Câu tiếng Anh: “Cô ấy nói: ‘Học sâu không phải là lời giải cho mọi vấn đề của thế giới’.” Chú ý trong tiếng Anh, *’s* (biểu thị sở hữu) được tách khỏi *world* thành một token riêng.

Mặt khác, việc chia câu thành các đơn vị ngôn ngữ nhỏ hơn là quan trọng trong nhiều tác vụ NLP, kể cả khi nhiều ngôn ngữ trên thế giới có rất ít hiện tượng hình thái. Chẳng hạn, tiếng Trung là một ngôn ngữ đơn giản về hình thái và không có ranh giới từ tường minh. Dù việc lấy các chữ (ký tự) làm đơn vị để hiểu nghĩa văn bản tiếng Trung cũng hợp lý, nhưng khi xử lý văn bản, xét các đơn vị lớn hơn lại là lựa chọn đáng mong muốn và hợp lý hơn. Lưu ý rằng, ngay cả với những ngôn ngữ có dấu phân cách giữa các từ như tiếng Anh, ta vẫn phải tách token các câu để chúng được chuẩn hóa khi làm đầu vào và/hoặc đầu ra của một hệ thống NLP.

Chú giải của người dịch: Còn tiếng Việt thì sao?

Tiếng Việt cũng là một ngôn ngữ đơn lập như tiếng Trung: từ không biến đổi hình thức (không có kiểu *learn/learns/learning*), ngữ pháp dựa chủ yếu vào trật tự từ và hư từ (*đã, đang, sẽ, của*…). Điểm đặc biệt là chữ viết tiếng Việt dùng dấu cách để phân tách *âm tiết*, không phải *từ*. Ví dụ câu “Học sinh học sinh học” có 5 âm tiết, nhưng nếu tách từ đúng ta được “Học_sinh / học / sinh_học” (3 từ: *học sinh*, *học*, *sinh học*). Vì vậy, chỉ tách theo dấu cách là chưa đủ; tách từ tiếng Việt cũng là một bài toán phân đoạn có nhập nhằng, gần với tiếng Trung hơn là tiếng Anh. Các công cụ tách từ tiếng Việt thường nối các âm tiết của một từ bằng dấu gạch dưới như trên.

Trong mục này, chúng tôi bỏ qua việc bàn chính xác *từ* là gì trong hình thái học và cú pháp học, mà đơn giản coi tách token là tác vụ thêm ranh giới từ hoặc ranh giới token vào một chuỗi cho trước (xem Hình 3.1). Chúng tôi sẽ cho thấy một câu có thể được chia thành các từ hoặc token theo cách kinh nghiệm hoặc theo cách thống kê. Lưu ý rằng quá trình này được thiết kế để tạo ra những đơn vị giúp việc xử lý ngôn ngữ trong các hệ thống NLP dễ dàng hơn, chứ không nhất thiết phải có ý nghĩa chặt

chẽ về mặt ngôn ngữ học.

3.1.1 Tách token bằng quy tắc và kinh nghiệm

Một cách tách token phổ biến và đơn giản là nhận diện mọi từ trong câu bằng cách áp dụng một tập quy tắc định sẵn. Nhìn chung, các quy tắc này xuất phát từ ngôn ngữ học và phản ánh hiểu biết có sẵn của chúng ta về hình thức của một từ. Ví dụ, xét câu tiếng Anh trong Hình 3.1. Ta có thể định nghĩa các quy tắc sau để tách token câu này:

- Từ không chứa dấu cách. Do đó, ta có thể chia câu thành các "ứng viên từ" dựa vào dấu cách.
- Mọi ứng viên từ chỉ gồm các chữ cái tiếng Anh (tức a–z và A–Z) là một từ.
- Mọi dấu câu (ví dụ dấu ngoặc kép, dấu phẩy, dấu chấm) phải được tách riêng thành một từ.
- *'s* được coi là một token riêng, biểu thị danh từ sở hữu.

Đây có lẽ là tập quy tắc tối thiểu ta có thể dùng để tách token tiếng Anh. Tất nhiên, có thể thêm nhiều quy tắc hơn để bao quát thêm các hiện tượng ngôn ngữ, ví dụ từ có dấu gạch nối, từ chứa chữ cái không thuộc tiếng Anh, v.v. Tuy nhiên, không có tiêu chuẩn nào để định nghĩa một tập quy tắc như vậy. Trong thực tế, đặc biệt là trong các ứng dụng NLP, ta muốn có một tập quy tắc tối thiểu xử lý được phần lớn các vấn đề, và việc tách token thường được cài đặt bằng một số biểu thức chính quy (*regular expression*). Ở đây chúng tôi không bàn chi tiết về các quy tắc và biểu thức chính quy này, mà giới thiệu bạn đọc một số giáo trình để đọc thêm [Lawson, 2003; Friedl, 2006; Jurafsky and Martin, 2008][1].

Chú giải của người dịch: Biểu thức chính quy

Biểu thức chính quy là một cách viết "mẫu" để máy tính tìm các đoạn văn bản khớp với mẫu đó. Ví dụ, mẫu `[a-zA-Z]+` nghĩa là "một hoặc nhiều chữ cái Latinh liên tiếp", còn mẫu `[,.!?]` khớp với một dấu câu bất kỳ trong bốn dấu đã liệt kê. Áp dụng lần lượt vài mẫu như vậy lên câu là ta có ngay một bộ tách token đơn giản.

Ngoài ra, người ta thường *chuẩn hóa* văn bản trước khi tách token để đầu vào của bộ tách token ở dạng chuẩn tắc. Ví dụ, với tiếng Anh và các ngôn ngữ dùng chữ cái khác, chuẩn hóa (*normalization*) hay chính tắc hóa (*canonicalization*) là quá trình chuyển các từ về chữ thường, chuẩn hóa cách biểu diễn ký tự (ví dụ các ký tự Unicode), v.v. Hơn nữa, ta có thể ánh xạ các dạng khác nhau của một từ về cùng một dạng để việc tách token khái quát hơn. Một cách đơn giản là gộp mọi dạng biến tố của một từ về dạng cơ bản của nó. Trong ngôn ngữ học, dạng cơ bản của một từ được gọi là **từ nguyên dạng** (*lemma*), và quá trình ánh xạ các từ về từ nguyên dạng được gọi là **đưa về nguyên dạng** (*lemmatization*). Dưới đây là một số ví dụ:

$$\begin{aligned} \textit{learn} &\rightarrow \textit{learn} \\ \textit{learning} &\rightarrow \textit{learn} \\ \textit{learns} &\rightarrow \textit{learn} \\ \textit{best} &\rightarrow \textit{good} \end{aligned}$$

Có những từ ứng với hai hay nhiều từ nguyên dạng khác nhau (thường có từ loại (*part-of-speech*) khác nhau). Khi đó, ta phải chọn từ nguyên dạng đúng tùy theo ngữ cảnh. Nói cách khác, việc đưa về nguyên dạng phụ thuộc vào ngữ cảnh.

[1] Các script tách token có thể tìm thấy trong nhiều dự án mã nguồn mở, chẳng hạn Moses [Koehn và cộng sự, 2007] (`https://github.com/moses-smt/mosesdecoder/blob/master/scripts/tokenizer/tokenizer.perl`) và các bộ tách token trong SacreBLEU (`https://github.com/mjpost/sacrebleu/tree/master/sacrebleu/tokenizers`).

Có liên hệ chặt chẽ với việc đưa về nguyên dạng là **lấy gốc từ** (*stemming*), tức biểu diễn một từ bằng *gốc từ* (*stem*) của nó. Giống từ nguyên dạng, gốc từ cũng là một dạng cơ bản nào đó của từ. Tuy nhiên, khác với từ nguyên dạng, gốc từ không nhất thiết phải là một từ hợp lệ, dù có nhiều từ có từ nguyên dạng và gốc từ trùng nhau. Một điểm khác nữa so với đưa về nguyên dạng là việc lấy gốc từ được thực hiện trên từng từ riêng lẻ, không cần ngữ cảnh để khử nhập nhằng. Vì vậy, lấy gốc từ không phụ thuộc ngữ cảnh. Có một số thuật toán hiệu quả để lấy gốc từ. Một thuật toán phổ biến là *cắt hậu tố* (*suffix stripping*) [Porter, 1980]. Nó đơn giản loại bỏ các hậu tố *ing*, *ed*, *ion*, v.v., như dưới đây:

$$\begin{aligned} \textit{remove} &\rightarrow \textit{remov} \\ \textit{removing} &\rightarrow \textit{remov} \\ \textit{removal} &\rightarrow \textit{remov} \\ \textit{best} &\rightarrow \textit{best} \end{aligned}$$

Để có thêm ví dụ, Hình 3.2 trình bày kết quả chuẩn hóa, đưa về nguyên dạng và lấy gốc từ cho một câu tiếng Anh.

Câu gốc	She said, “Deep learning is not the solution to all world’s problems”.
Chuẩn hóa	she said, “deep learning is not the solution to all world’s problems”.
Tách token	she/said/,/“/deep/learning/is/not/the/solution/to/all/world/’s/problems/”/.
Đưa về nguyên dạng	she/say/,/“/deep/learning/be/not/the/solution/to/all/world/’s/problem/”/.
Lấy gốc từ	she/said/,/“/deep/learn/is/not/the/solut/to/all/world/’s/problem/”/.

Hình 3.2: Chuẩn hóa, đưa về nguyên dạng và lấy gốc từ cho một câu tiếng Anh. Khi chuẩn hóa, toàn bộ câu được chuyển thành chữ thường. Khi đưa về nguyên dạng, mỗi từ được viết lại thành từ nguyên dạng của nó. Khi lấy gốc từ, hậu tố của một số từ bị loại bỏ.

Chú giải của người dịch: Nguyên dạng khác gốc từ thế nào?

Trong Hình 3.2, *said* (quá khứ của *say* — “nói”) được đưa về nguyên dạng *say*, *is* về *be* (“là”), vì bộ đưa về nguyên dạng “biết” từ điển. Còn bộ lấy gốc từ chỉ máy móc cắt đuôi: *solution* thành *solut* — không phải từ có thật — nhưng lại để nguyên *said*. Tiếng Việt hầu như không cần hai bước này vì từ không biến đổi hình thức: “học” luôn là “học”, dù ở thì nào hay ngôi nào.

Cần lưu ý rằng các phương pháp trên thường được cài đặt bằng biểu thức chính quy, tra từ điển và một số kinh nghiệm bổ sung. Dù phần tổng quan ngắn gọn này có thể khiến ta nghĩ rằng tách token là việc đơn giản, nhưng cần nhiều công sức hơn nữa để nó dùng được trong thực tế. Đặc biệt, khi xử lý các ngôn ngữ có hệ chữ viết không dùng bảng chữ cái, hoặc các ngôn ngữ không có khoảng trắng tường minh giữa các từ, việc tách token trở thành một bài toán phức tạp, và khi đó dùng các quy tắc đơn giản không phải là chiến lược tốt. Trong các tiểu mục tiếp theo, chúng tôi sẽ phát biểu lại việc tách token như một bài toán học máy (*machine learning*), trong đó cách tách token hay phân đoạn câu được học từ dữ liệu. Các phương pháp này không phụ thuộc vào ngôn ngữ và có thể áp dụng cho nhiều bài toán tách token hoặc các bài toán dạng phân đoạn.

3.1.2 Tách token như bài toán mô hình hóa ngôn ngữ

Giờ hãy chuyển sang mô hình hóa thống kê cho bài toán tách token. Để tiện trình bày, tiểu mục này tập trung vào các hệ chữ viết không có ranh giới từ tường minh, nhưng các phương pháp này có thể tổng quát hóa cho các ngôn ngữ có dấu phân cách. Gọi $\mathbf{x} = x_1 \cdots x_l$ là một chuỗi ký tự, và $\mathbf{y} = y_1 \cdots y_m$ là một chuỗi các từ hoặc token. Ta nói $\mathbf{y}$ là một kết quả tách token của $\mathbf{x}$ nếu $\mathbf{y}$ xác định một cách phân

đoạn trên **x**. Xét câu tiếng Trung sau:

$$\mathbf{x} \;=\; \text{机器翻译是人类的梦想之一。}$$

Ta có thể định nghĩa một cách phân đoạn cho câu này, chẳng hạn[2],

$$\begin{aligned}\mathbf{y} &= \text{机器翻译/是/人类/的/梦想/之一/。} \\ &= \left[\text{“机器翻译”}\ \text{“是”}\ \text{“人类”}\ \text{“的”}\ \text{“梦想”}\ \text{“之一”}\ \text{“。”}\right]\end{aligned} \tag{3.1}$$

(nghĩa: “Dịch máy / là / nhân loại / của / ước mơ / một trong / .”, tức “Dịch máy là một trong những ước mơ của nhân loại.”). Theo cách này, tách token có thể được phát biểu như bài toán ánh xạ **x** thành **y**. Cho một chuỗi đầu vào, đầu ra là cách phân đoạn có khả năng xảy ra cao nhất:

$$\begin{aligned}\hat{\mathbf{y}} &= \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x}) \\ &= \arg\max_{\mathbf{y}} \log \Pr(\mathbf{y}|\mathbf{x}).\end{aligned} \tag{3.2}$$

Phương trình (3.2) mô tả một mô hình dự đoán mà ta đã nhắc đến nhiều lần trong sách này. Tuy nhiên, bài toán ta đang xét dễ hơn, vì mọi cách phân đoạn hợp lệ **y** đều sinh ra đúng chuỗi **x** một cách tất định. Do đó, việc tìm cách phân đoạn có khả năng nhất được đơn giản hóa thành việc cực đại hóa xác suất tiên nghiệm (*prior probability*) $\Pr(\mathbf{y})$:

$$\begin{aligned}\hat{\mathbf{y}} &= \arg\max_{\mathbf{y}} \log \Pr(\mathbf{y}) \\ &= \arg\max_{\mathbf{y}} \log \Pr(y_1, \cdots, y_m).\end{aligned} \tag{3.3}$$

Chú giải của người dịch: Vì sao $\Pr(\mathbf{y}|\mathbf{x})$ chỉ còn $\Pr(\mathbf{y})$?

Ký hiệu $\arg\max_{\mathbf{y}} f(\mathbf{y})$ nghĩa là “giá trị **y** làm $f(\mathbf{y})$ lớn nhất” (khác với max là *giá trị lớn nhất* của f). Theo định nghĩa xác suất có điều kiện, $\Pr(\mathbf{y}|\mathbf{x}) = \Pr(\mathbf{x}, \mathbf{y}) / \Pr(\mathbf{x})$. Mẫu số $\Pr(\mathbf{x})$ không phụ thuộc **y** nên không ảnh hưởng đến việc chọn **y** tốt nhất. Còn khi **y** là một cách tách của **x** thì ghép các từ của **y** lại chắc chắn được **x**, nên $\Pr(\mathbf{x}, \mathbf{y}) = \Pr(\mathbf{y})$. Hàm log đồng biến nên lấy log không làm đổi vị trí cực đại; ta dùng log để biến tích các xác suất nhỏ thành tổng, dễ tính hơn.

Dễ kiểm tra rằng Phương trình (3.3) thực ra mô tả một bài toán mô hình hóa ngôn ngữ (*language modeling*). Có vài cách khác nhau để ước lượng xác suất đồng thời $\Pr(y_1, \cdots, y_m)$. Một cách đơn giản là viết lại $\log \Pr(y_1, \cdots, y_m)$ thành tổng của các logarit xác suất có điều kiện:

$$\begin{aligned}\log \Pr(y_1, \cdots, y_m) = \log \Pr(y_1) + \log \Pr(y_2|y_1) \\ + \cdots + \log \Pr(y_m|y_1, \cdots, y_{m-1}).\end{aligned} \tag{3.4}$$

Mỗi xác suất có điều kiện $\Pr(y_i|y_1, \cdots, y_{i-1})$ có thể được xấp xỉ bằng

$$\Pr(y_i|y_1, \cdots, y_{i-1}) \;\approx\; \Pr(y_i|y_{i-n+1}, \cdots, y_{i-1}), \tag{3.5}$$

tức là việc sinh ra y_i chỉ phụ thuộc vào $n-1$ từ ngữ cảnh đứng trước. Để tính $\Pr(y_i|y_{i-n+1}, \cdots, y_{i-1})$, ta có thể dùng phương pháp tần suất tương đối hoặc mạng nơ-ron (xem Chương 2).

[2]Theo ký hiệu đã dùng trước đây, chúng tôi dùng cả $\mathbf{y} = y_1 \cdots y_m$ và $\mathbf{y} = \begin{bmatrix} y_1 & \cdots & y_m \end{bmatrix}$ để ký hiệu một chuỗi biến.

Chú giải của người dịch: Ví dụ nhỏ với $n = 2$

Với $n = 2$ (mô hình *bigram*), mỗi từ chỉ phụ thuộc vào đúng một từ đứng trước. Khi đó, với câu ba từ, $\log \Pr(y_1, y_2, y_3) \approx \log \Pr(y_1) + \log \Pr(y_2|y_1) + \log \Pr(y_3|y_2)$. Giả sử $\Pr(y_1) = 0.1$, $\Pr(y_2|y_1) = 0.5$, $\Pr(y_3|y_2) = 0.2$ thì xác suất của câu là $0.1 \times 0.5 \times 0.2 = 0.01$. Ta tính điểm như vậy cho từng cách tách có thể có và chọn cách có điểm cao nhất.

Bây giờ ta có thể coi tách token là một bài toán học có giám sát (*supervised learning*). Quy trình được phác thảo như sau:

- Chuẩn bị một số câu đã được phân đoạn đúng.
- Học một mô hình ngôn ngữ (*language model*) $\Pr(\mathbf{y})$ trên các câu có nhãn này.
- Với một câu mới, tìm cách tách token "tốt nhất" $\hat{\mathbf{y}}$ làm cực đại $\Pr(\hat{\mathbf{y}})$, như trong Phương trình (3.3).

Dù quy trình này tuân theo các bước chuẩn của học có giám sát, vẫn có vài vấn đề thực tế ta phải giải quyết. Thứ nhất, mô hình ngôn ngữ cần một bộ từ vựng (*vocabulary*) định sẵn để tính xác suất cho mỗi token y_i, nhưng từ mới thì luôn xuất hiện. Để xử lý chúng, một cách là phân đoạn chuỗi con chưa biết thành các ký tự, tức là ta coi các ký tự là từ nếu chuỗi con sinh ra các ký tự đó không có trong bộ từ vựng. Một cách khác là xét mọi chuỗi con không được bộ từ vựng bao phủ, và thay chúng bằng thẻ <unk>. Mẹo <unk> được dùng rộng rãi trong các mô hình ngôn ngữ tiên tiến nhất (*state-of-the-art*) và thường có ích.

Thứ hai, mô hình ngôn ngữ mô tả ở trên có thiên hướng ưu tiên các chuỗi ngắn, vì $\Pr(y_1, \cdots, y_m)$ sẽ lớn nếu m nhỏ. Một cách chung để giảm thiên lệch này là đưa vào mô hình một *phần thưởng độ dài* (*length reward*, hay *length bonus*), chẳng hạn

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \log \Pr(\mathbf{y}) + \lambda \cdot m, \tag{3.6}$$

hoặc

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \frac{\log \Pr(\mathbf{y})}{m^\lambda}, \tag{3.7}$$

trong đó $\lambda \cdot m$ và m^λ thưởng cho các chuỗi dài, và $\lambda > 0$ là một siêu tham số (*hyperparameter*) điều khiển mức độ ta dựa vào phần thưởng khi đánh giá độ tốt của $\mathbf{y}$. Điều thú vị là trong thực tế, người ta nhận thấy thiên lệch độ dài không phải vấn đề lớn với tách token, vì độ dài của các kết quả tách token "tốt" ít dao động. Chẳng hạn, dùng một mô hình ngôn ngữ unigram (tức $n = 1$) không có phần thưởng độ dài nào vẫn cho kết quả tốt trong nhiều ứng dụng thực tế. Ta sẽ thấy vài ví dụ ở Mục 3.1.4.

Chú giải của người dịch: Vì sao mô hình ưa chuỗi ngắn?

Mỗi xác suất đều nhỏ hơn 1, nên mỗi lần nhân thêm một thừa số, tích lại nhỏ đi (tương ứng, mỗi số hạng log đều âm nên tổng càng nhiều số hạng càng âm). Ví dụ nếu mỗi từ có xác suất khoảng 0.1 thì cách tách thành 2 từ được điểm $0.1^2 = 0.01$, còn cách tách thành 4 từ chỉ được $0.1^4 = 0.0001$. Như vậy cách tách ít từ (từ dài) bị ưu ái "một cách không công bằng". Số hạng $\lambda \cdot m$ cộng thêm điểm cho mỗi từ để bù lại.

Thứ ba, việc thực hiện arg max là khó, vì số cách tách token ứng viên tăng theo hàm mũ. Tuy nhiên, việc dùng mô hình ngôn ngữ ở đây cho phép dùng các thuật toán tìm kiếm hiệu quả. Chẳng hạn, xét việc áp dụng mô hình ngôn ngữ unigram vào tách token. Với chuỗi đầu vào $\mathbf{x}$, tại mỗi vị trí j của $\mathbf{x}$, ta lưu một *trạng thái* mô tả xác suất của cách tách token tốt nhất trên $x_1 \cdots x_j$ (ký hiệu $p(j)$) cùng với từ cuối cùng của cách tách này. Tại vị trí $j + 1$, ta tạo một trạng thái mới và tính xác suất của cách tách

token tốt nhất trên $x_1 \cdots x_{j+1}$ bằng

$$\begin{aligned} p(j+1) &= \max_{1 \le i \le j} p(i) \cdot \Pr(x_{i+1} \cdots x_{j+1}) \\ &= \max_{1 \le i \le j} p(i) \cdot \Pr(w_{[i+1,j+1]}), \end{aligned} \tag{3.8}$$

trong đó $\Pr(w_{[i+1,j]})$ là xác suất của từ trải trên đoạn $x_{i+1} \cdots x_j$. Về mặt thuật toán, Phương trình (3.8) mô tả một phương pháp quy hoạch động (*dynamic programming*) có độ phức tạp thời gian $O(l^2)$ với đầu vào độ dài l. Để có đầu ra cuối cùng, ta có thể lần ngược từ trạng thái cuối và xuất ra chuỗi từ dọc theo đường đi của cách tách token tối ưu.

Chú giải của người dịch: Quy hoạch động qua ví dụ

Xét chuỗi 3 ký tự abc. Các cách tách có thể: abc; a/bc; ab/c; $a/b/c$ — tổng cộng $2^{l-1} = 4$ cách (mỗi khe giữa hai ký tự có thể cắt hoặc không), tăng theo hàm mũ khi l lớn. Quy hoạch động tránh liệt kê hết: ta tính lần lượt $p(1), p(2), p(3)$, trong đó $p(j)$ là điểm của cách tách tốt nhất cho j ký tự đầu. Muốn tính $p(3)$, ta chỉ cần thử "từ cuối cùng" là c, bc hay abc, rồi ghép với $p(2)$, $p(1)$ hay $p(0) = 1$ (chuỗi rỗng) đã tính sẵn. Mỗi vị trí thử tối đa l khả năng, nên tổng cộng cỡ l^2 phép tính — đó là ý nghĩa của ký hiệu $O(l^2)$.

Lưu ý rằng các phương pháp ở đây mang tính tổng quát và có thể áp dụng để tách token cho các ngôn ngữ khác. Chẳng hạn, khi áp dụng cho tiếng Anh, ta chỉ cần điều chỉnh nhẹ định dạng đầu vào: đầu vào không phải là chuỗi ký tự mà là chuỗi các mảnh nhỏ nhất có thể, được tách ra bởi dấu câu và dấu cách. Ví dụ, với câu *Is this Tom's laptop?* ("Đây có phải máy tính xách tay của Tom không?"), ta có

$$\mathbf{x} = \begin{bmatrix} \text{Is} & \text{this} & \text{Tom} & \text{'} & \text{s} & \text{laptop} & \text{?} \end{bmatrix}. \tag{3.9}$$

Sau đó, quá trình tách token có thể tiến hành như trong Phương trình (3.2–3.7).

3.1.3 Tách token như bài toán gán nhãn chuỗi

Một cách phổ biến để phân đoạn các ký tự thành từ là gán cho mỗi ký tự trong chuỗi một nhãn lấy từ một tập định sẵn, quá trình này được gọi là gán nhãn chuỗi (*sequence labeling*). Dù ở đây ta tập trung vào bài toán phân đoạn từ, khuôn khổ này cũng áp dụng được cho nhiều tác vụ NLP khác, chẳng hạn gán nhãn từ loại (*part-of-speech tagging*) và nhận dạng thực thể có tên (*named entity recognition*). Vì ý tưởng gán nhãn chuỗi đã được bàn ở Chương 1, ở đây chúng tôi trình bày cách điều chỉnh nó cho tác vụ tách token.

Các tập nhãn dùng trong tách token thường đơn giản. Một ví dụ là tập "IB". Nhãn "I" chỉ một mảnh ngôn ngữ nằm *bên trong* một từ (*inside*), còn nhãn "B" chỉ *đầu* một từ (*beginning*). Tập nhãn có thể được làm phong phú hơn bằng cách thêm nhãn "E" (tức *cuối* một từ, *ending*) và/hoặc tách nhãn "B" thành các nhãn con (ví dụ B_1 và B_2 chỉ mảnh ngôn ngữ thứ nhất và thứ hai của một từ) [Zhao và cộng sự, 2006]. Cho một chuỗi đầu vào $\mathbf{x}$ và một kết quả tách token $\mathbf{y}$, việc chuyển $\mathbf{y}$ thành chuỗi nhãn khá đơn giản. Xét lại ví dụ đã dùng ở tiểu mục trước. Ta có thể gán nhãn chuỗi theo các định dạng khác nhau:

x:	机	器	翻	译	是	人	类	的	梦	想	之	一	。
y:	机	器	翻	译	是	人	类	的	梦	想	之	一	。
{I,B}	B	I	I	I	B	B	I	B	B	I	B	I	B
{I,B,E}	B	I	I	E	B	B	E	B	B	E	B	E	B
$\{\text{I,B}_1\text{,B}_2\text{,E}\}$	B_1	B_2	I	E	B_1	B_1	B_2	B	B_1	B_2	B_1	B_2	B_1

Chú giải của người dịch: Gán nhãn B/I cho tiếng Việt

Với tiếng Việt, ta gán nhãn cho từng *âm tiết* thay vì từng chữ Hán. Chẳng hạn câu "Học sinh học sinh học" được tách thành "học_sinh / học / sinh_học" tương ứng chuỗi nhãn {I,B}: *Học*/B, *sinh*/I, *học*/B, *sinh*/B, *học*/I. Biết chuỗi nhãn là khôi phục được ngay cách tách: mỗi nhãn B mở đầu một từ mới. Vì vậy bài toán tách từ trở thành bài toán "đoán nhãn cho từng vị trí".

Vì chuỗi nhãn có thể được xem như một dạng khác của kết quả tách token, ta có thể phát biểu lại bài toán là tìm chuỗi nhãn tốt nhất cho một đầu vào:

$$\hat{\mathbf{c}} \;=\; \arg\max_{\mathbf{c}} \; \log \Pr(\mathbf{c}|\mathbf{x}), \tag{3.10}$$

trong đó $\mathbf{c} = c_1 \cdots c_l$ là một chuỗi nhãn độ dài l, và $\hat{\mathbf{c}}$ là chuỗi nhãn có xác suất cao nhất. Nhiều phương pháp đã được đề xuất để mô hình hóa $\Pr(\mathbf{c}|\mathbf{x})$. Một cách kinh điển là viết lại $\Pr(\mathbf{c}|\mathbf{x})$ bằng quy tắc Bayes:

$$\begin{aligned}\hat{\mathbf{c}} &= \arg\max_{\mathbf{c}} \; \log \frac{\Pr(\mathbf{x}|\mathbf{c})\Pr(\mathbf{c})}{\Pr(\mathbf{x})} \\ &= \arg\max_{\mathbf{c}} \; \log \Pr(\mathbf{x}|\mathbf{c}) + \log \Pr(\mathbf{c}).\end{aligned} \tag{3.11}$$

Vì $\Pr(\mathbf{x})$ là hằng số với mọi chuỗi ứng viên $\mathbf{c}$, ta có thể bỏ qua nó trong quá trình tìm kiếm. $\Pr(\mathbf{x}|\mathbf{c})$ là xác suất sinh ra đầu vào $\mathbf{x}$ (tức các *quan sát*) khi biết chuỗi nhãn $\mathbf{c}$ (tức các *biến ẩn*), còn $\Pr(\mathbf{c})$ là một mô hình ngôn ngữ định nghĩa trên chuỗi nhãn. Nhìn chung, cần đơn giản hóa để mô hình có thể tính toán được. Chẳng hạn, ta có thể đặt **giả thiết Markov** (*Markov assumption*) rằng việc chọn c_i chỉ phụ thuộc vào việc chọn c_{i-1}. Điều này dẫn tới **mô hình Markov ẩn** (*hidden Markov model, HMM*), được dùng rộng rãi trong mô hình hóa sinh cho các bài toán NLP.

Chú giải của người dịch: Quy tắc Bayes và giả thiết Markov

Quy tắc Bayes: $\Pr(\mathbf{c}|\mathbf{x}) = \dfrac{\Pr(\mathbf{x}|\mathbf{c})\Pr(\mathbf{c})}{\Pr(\mathbf{x})}$, cho phép "lật ngược" chiều điều kiện. Thay vì trực tiếp hỏi "với câu này thì nhãn nào hợp lý?", ta hỏi hai câu dễ ước lượng hơn: "chuỗi nhãn này tự nó có hợp lý không?" ($\Pr(\mathbf{c})$) và "nếu nhãn là thế thì sinh ra đúng câu này dễ đến đâu?" ($\Pr(\mathbf{x}|\mathbf{c})$). *Giả thiết Markov* giống như nói: để đoán nhãn của chữ hiện tại, chỉ cần nhìn nhãn của chữ ngay trước, không cần nhớ toàn bộ quá khứ. Ví dụ, sau nhãn E (cuối từ) thì nhãn tiếp theo phải là B, không thể là I hay E.

Một phương pháp khác là mô hình hóa phân biệt (*discriminative modeling*). Một ý tưởng phổ biến là coi gán nhãn chuỗi như một loạt các bài toán phân loại độc lập. Chẳng hạn, ta có thể xây dựng một bộ phân loại (*classifier*) cục bộ, dự đoán c_i dựa trên một tập đặc trưng (*feature*) xung quanh vị trí i. Trong các mô hình phức tạp hơn, như **trường ngẫu nhiên có điều kiện** (*conditional random fields, CRFs*), ngữ cảnh của toàn bộ chuỗi có thể được dùng để dự đoán. Bàn chi tiết về các mô hình gán nhãn chuỗi này nằm ngoài phạm vi của mục này. Bạn đọc có thể tham khảo [Kupiec, 1992; McCallum và cộng sự, 2000; Lafferty và cộng sự, 2001] để có thảo luận đầy đủ về cách các mô hình này được xây dựng và áp dụng. Ngoài ra, về so sánh giữa mô hình hóa sinh và mô hình hóa phân biệt, mời bạn đọc xem Chương 1.

3.1.4 Học từ con

Từ được xem rộng rãi là đơn vị cơ bản của ngôn ngữ. Tuy nhiên, chúng không phải là những thành phần ngôn ngữ nhỏ nhất. Thay vào đó, chúng có thể được phân tách thành các đơn vị có nghĩa nhỏ hơn,

chẳng hạn các hình vị (*morpheme*). Cấu trúc phân cấp này — trong đó hình vị tạo thành từ, còn từ tạo thành cụm từ và câu — làm nổi bật vai trò trung tâm của từ trong cú pháp. Dù vậy, trong NLP, việc coi câu một cách cứng nhắc là chuỗi các từ thường không phải lựa chọn tối ưu. Vấn đề chính là *dữ liệu thưa* gây ra bởi các từ hiếm, cản trở việc huấn luyện mô hình hiệu quả. Chẳng hạn, từ *uncopyrightable* ("không thể đăng ký bản quyền") hiếm khi xuất hiện trong kho ngữ liệu. Một hệ thống NLP có thể coi nó là một token ngoài bộ từ vựng (*out-of-vocabulary*), dù nghĩa của nó có thể suy ra được bằng cách tách nó thành *un* (không), *copy* (sao chép), *right* (quyền) và *able* (có thể). Một vấn đề khác là các tiêu chuẩn tách token dựa trên ngôn ngữ học hạn chế khả năng của mô hình trong việc tự động học các đơn vị phân đoạn tối ưu theo nghĩa học máy. Do đó, thường hiệu quả hơn nếu ta xác định các đơn vị "từ con" — tuy không hẳn mang tính ngôn ngữ học, nhưng phù hợp hơn với các hệ thống học máy.

1. Mã hóa cặp byte

Mã hóa cặp byte (*Byte Pair Encoding, BPE*) là một trong những phương pháp thành công nhất để học các đơn vị **từ con** từ một tập các chuỗi từ [Sennrich và cộng sự, 2016b]. Dù BPE bắt nguồn từ nén dữ liệu [Gage, 1994], trong NLP nó thường được dùng như một giải pháp cho bài toán *bộ từ vựng mở* (*open vocabulary*). Ý tưởng cơ bản của BPE là lặp đi lặp lại việc thay cặp byte xuất hiện nhiều nhất trong dữ liệu bằng một byte mới. Kết quả là các byte phổ biến thường tham gia vào việc gộp các chuỗi byte con, còn các byte hiếm thường đứng riêng lẻ và được coi là các đơn vị riêng. Đầu ra của BPE là một bộ từ vựng byte có thể dùng để mã hóa dữ liệu mới.

Trong NLP, một byte có thể tương ứng gần đúng với một ký tự. Mỗi mục trong bộ từ vựng là một chuỗi ký tự, được gọi là một *ký hiệu* (*symbol*) hay từ con. BPE bắt đầu bằng việc tách văn bản cho trước thành chuỗi ký tự, chẳng hạn ta có thể chèn một dấu cách sau mỗi chữ cái tiếng Anh hoặc dấu câu. Tuy nhiên, cách này thường tạo ra một chuỗi quá dài. Dù bản thân BPE không giới hạn độ dài đầu vào, cách phổ biến hơn là không cho phép các ký hiệu vượt qua ranh giới từ, để tăng hiệu quả. Vì vậy, ta có thể biểu diễn văn bản thành một danh sách các từ được phân cách bằng dấu cách, mỗi từ đi kèm tần suất của nó. Ví dụ, xét danh sách từ:

f l o w # : 2
b l o w # : 2
f l a t # : 1
f l a g # : 4

trong đó # là ký hiệu đặc biệt chỉ điểm kết thúc một từ[3]. Từ danh sách từ này, ta có thể thu được bộ từ vựng ban đầu:

f : 7	b : 2
l : 9	a : 5
o : 4	t : 1
w : 4	g : 4
# : 9	

[3]Thay vì coi # là một ký hiệu riêng, có thể gắn nó vào ký tự cuối cùng của mỗi từ

f l o w# : 2
b l o w# : 2
f l a t# : 1
f l a g# : 4

trong đó "w#", "t#" và "g#" biểu diễn các ký tự xuất hiện ở cuối một từ.

Sau đó, ta đếm số lần xuất hiện của mỗi *bigram ký hiệu* (cặp hai ký hiệu đứng liền nhau):

f l : 7	a g : 4
l a : 5	g # : 4
l o : 4	b l : 2
o w : 4	a t : 1
w # : 4	t # : 1

Ta gộp bigram ký hiệu xuất hiện nhiều nhất là "f l" thành một ký hiệu mới "fl" và thay mọi lần xuất hiện của "f l" trong danh sách từ bằng "fl":

fl o w # : 2
b l o w # : 2
fl a t # : 1
fl a g # : 4

Tương ứng, ký hiệu "fl" được thêm vào bộ từ vựng:

f : 7	b : 2
l : 9	a : 5
o : 4	t : 1
w : 4	g : 4
# : 9	fl : 7

Chú giải của người dịch: Các con số đếm được từ đâu?

Mỗi từ trong danh sách được "nhân" với tần suất của nó. Ví dụ ký hiệu "l" xuất hiện một lần trong mỗi từ, nên tổng là $2 + 2 + 1 + 4 = 9$; "f" có trong *flow*, *flat*, *flag* nên đếm được $2 + 1 + 4 = 7$. Cặp "f l" đứng liền nhau trong đúng ba từ đó nên cũng có tần suất 7 — cao nhất bằng, vì vậy được gộp đầu tiên. Có thể hình dung BPE như việc "đặt tên tắt" cho những cụm chữ hay đi cùng nhau nhất.

Quá trình này sau đó được lặp lại. Lần này, ta gộp bigram ký hiệu "fl a" và tạo ký hiệu mới "fla". Như vậy, ta có danh sách từ mới:

fl o w # : 2
b l o w # : 2
fla t # : 1
fla g # : 4

và bộ từ vựng mới:

f : 7	b : 2	fla : 5
l : 9	a : 5	
o : 4	t : 1	
w : 4	g : 4	
# : 9	fl : 7	

Quá trình gộp này có thể được lặp lại với một số lần định trước. Càng gộp nhiều lần, bộ từ vựng càng lớn. Các mục của bộ từ vựng cuối cùng được sắp xếp lại theo tần suất ký hiệu. Ví dụ, nếu ta đặt số phép gộp là 6, ta sẽ có bộ từ vựng như sau:

l : 9	fla : 5	ow# : 4
# : 9	o : 4	flag : 4
f : 7	w : 4	flag# : 4
fl : 7	g : 4	b : 2
a : 5	ow : 4	t : 1

Bộ từ vựng này ứng với danh sách từ:

fl ow# : 2
b l ow# : 2
fla t # : 1
flag# : 4

Khi đã có một bộ từ vựng như trên, ta có thể dùng nó để tách token cho các từ mới. Việc tách thành từ con tuân theo cùng thủ tục gộp các bigram ký hiệu như khi xây dựng bộ từ vựng. Cho một bộ từ vựng BPE, trước hết ta phân đoạn văn bản đầu vào thành các ký hiệu ký tự. Sau đó, ta xét từng bigram ký hiệu trong chuỗi và gộp bigram có tần suất cao nhất trong bộ từ vựng. Ta lặp lại thao tác này cho đến khi không còn phép gộp nào nữa. Ví dụ, xét văn bản sau:

tow a flag

("kéo một lá cờ"). Trước hết nó được chuyển thành chuỗi ký tự:

t o w # a # f l a g #

Dùng bộ từ vựng BPE đã thu được, ta có thể thực hiện các phép gộp BPE trên chuỗi này như sau

t o w # a # f l a g #
$\xrightarrow{\text{f l} \Rightarrow \text{fl}}$ t o w # a # fl a g #
$\xrightarrow{\text{fl a} \Rightarrow \text{fla}}$ t o w # a # fla g #
$\xrightarrow{\text{o w} \Rightarrow \text{ow}}$ t ow # a # fla g #
$\xrightarrow{\text{ow #} \Rightarrow \text{ow\#}}$ t ow# a # fla g #
. . . . . .
$\longrightarrow$ t ow# a # flag#

Chuỗi từ con này có thể làm đầu vào và/hoặc đầu ra cho một tác vụ NLP phía sau, chẳng hạn dịch máy (*machine translation*). Đôi khi ta muốn ánh xạ các từ con ngược lại thành từ. Việc này đơn giản: ta giữ dấu cách sau mỗi lần xuất hiện ký hiệu #, rồi xóa mọi dấu cách khác và mọi ký hiệu #. Cũng cần lưu ý rằng phương pháp BPE mô tả ở đây đòi hỏi đầu vào đã được tách từ, tức là ta cần một bộ *tách token sơ bộ* (*pre-tokenizer*) để chia sơ bộ chuỗi đầu vào thành một số đơn vị. Việc này có thể làm bằng các phương pháp trình bày ở Mục 3.1.1–3.1.3.

Chú giải của người dịch: Vì sao BPE giải quyết được từ hiếm?

Từ *tow* chưa hề có trong danh sách từ dùng để học, nhưng BPE vẫn biểu diễn được nó thành *t* + *ow#*. Trong trường hợp xấu nhất, một từ lạ vẫn luôn tách được thành các ký tự đơn lẻ, vốn chắc chắn có trong bộ từ vựng. Nhờ vậy, mô hình gần như không bao giờ gặp token "không biết" (<unk>). Còn các từ phổ biến (như *flag*) thì được giữ nguyên thành một token, giúp chuỗi ngắn gọn. Các mô hình ngôn ngữ lớn hiện nay (họ GPT,

LLaMA…) đều dùng một biến thể của BPE hoặc các phương pháp từ con dưới đây.

2. WordPiece

WordPiece rất giống BPE: trước hết nó chia văn bản đầu vào thành các ký hiệu nhỏ nhất, sau đó dần dần gộp các ký hiệu liên tiếp để tạo thành các ký hiệu lớn hơn [Schuster and Nakajima, 2012]. Khác biệt giữa hai phương pháp chỉ nằm ở cách chọn bigram ký hiệu để gộp. Ở mỗi bước, BPE gộp bigram ký hiệu có tần suất cao nhất. Gọi (x_i, x_{i+1}) là một bigram trong chuỗi $\mathbf{x}$. Quy tắc gộp của BPE có thể được mô tả là

$$(x_{\hat{i}}, x_{\hat{i}+1}) = \underset{i \in [1, |\mathbf{x}|-1]}{\arg\max} \text{count}(x_i, x_{i+1}), \tag{3.12}$$

trong đó hàm $\text{count}(x_i, x_{i+1})$ trả về tần suất của (x_i, x_{i+1}) trong kho ngữ liệu, và $(x_{\hat{i}}, x_{\hat{i}+1})$ là bigram có tần suất cao nhất.

Ngược lại, phương pháp WordPiece dùng tiêu chí hợp lý cực đại (*maximum likelihood*) để chọn bigram. Cụ thể, nó gộp bigram làm độ hợp lý (*likelihood*) của dữ liệu huấn luyện tăng nhiều nhất. Điều này có thể được hình thức hóa là:

$$\begin{aligned}(x_{\hat{i}}, x_{\hat{i}+1}) &= \underset{i \in [1, |\mathbf{x}|-1]}{\arg\max} \log \frac{\Pr(x_i, x_{i+1})}{\Pr(x_i)\Pr(x_{i+1})} \\ &= \underset{i \in [1, |\mathbf{x}|-1]}{\arg\max} \big[\log \Pr(x_i, x_{i+1}) - \log\big(\Pr(x_i)\Pr(x_{i+1})\big)\big].\end{aligned} \tag{3.13}$$

Số hạng $\log \Pr(x_i, x_{i+1}) - \log\big(\Pr(x_i)\Pr(x_{i+1})\big)$ biểu thị lượng tăng của log-độ hợp lý của kho ngữ liệu khi thay các ký hiệu liên tiếp (x_i, x_{i+1}) bằng ký hiệu đã gộp $x_i x_{i+1}$[4]. Do đó, áp dụng quy tắc gộp này sinh ra một dãy các thao tác làm tăng dần độ hợp lý tổng thể của dữ liệu. Kết quả của quá trình này là một *bảng mã* (tức một bộ từ vựng) mà nhờ nó ta có thể xác định chuỗi mã có khả năng nhất cho văn bản cho trước.

Chú giải của người dịch: BPE và WordPiece khác nhau ra sao?

BPE chỉ hỏi: "cặp nào đi cùng nhau *nhiều lần nhất*?". WordPiece hỏi: "cặp nào đi cùng nhau nhiều hơn *mức ngẫu nhiên* nhất?". Nếu a và b xuất hiện độc lập, xác suất gặp chúng đứng cạnh nhau xấp xỉ $\Pr(a)\Pr(b)$ (như tung hai đồng xu độc lập). Tỉ số $\frac{\Pr(a,b)}{\Pr(a)\Pr(b)}$ lớn hơn 1 nghĩa là a, b "hút nhau" thật sự. Ví dụ, hai chữ rất phổ biến như "e" và "t" có thể đứng cạnh nhau nhiều lần chỉ vì bản thân chúng phổ biến; WordPiece sẽ ưu tiên những cặp hiếm hơn nhưng gần như luôn đi cùng nhau, như "q" và "u" trong tiếng Anh.

3. SentencePiece

Cả BPE và WordPiece đều đòi hỏi văn bản đầu vào được tách sơ bộ thành từ. Sự phụ thuộc này làm quy trình tách token phức tạp thêm. Như một lựa chọn thay thế, SentencePiece cung cấp một khuôn khổ tổng quát hơn, xử lý trực tiếp văn bản thô bằng cách coi mọi ký tự, kể cả dấu cách, là các đơn vị nguyên tử cơ bản [Kudo and Richardson, 2018]. Dù SentencePiece có thể kết hợp với nhiều thuật toán từ con khác nhau, nó thường gắn với các mô hình ngôn ngữ unigram (hay gọi tắt là *phương pháp unigram*), vốn áp dụng một chiến lược xây dựng bộ từ vựng khác hẳn: thay vì xây bộ từ vựng từ dưới lên, nó bắt

[4] Trong thống kê, $\frac{\Pr(a,b)}{\Pr(a)\Pr(b)}$ được gọi là **thông tin tương hỗ theo điểm** (*pointwise mutual information*) của hai biến a và b. Xem thêm chi tiết ở Mục 3.3.1. Khái niệm này còn được gọi là **độ lợi thông tin** (*information gain*) và có thể được diễn giải bằng độ phân kỳ Kullback-Leibler hoặc các độ đo khác trong lý thuyết thông tin (xem Chương 1).

đầu với một bộ từ vựng lớn rồi dần dần cắt tỉa để cực đại hóa độ hợp lý tổng thể của dữ liệu [Kudo, 2018][5].

Phương pháp unigram phát biểu việc phân đoạn từ con như một bài toán mô hình hóa ngôn ngữ unigram, giống dạng tổng quát của Phương trình (3.3–3.4). Gọi $\mathbf{x}$ là một chuỗi ký tự và $\mathbf{y}$ là một chuỗi các ký hiệu hay từ con mà khi nối lại tạo thành $\mathbf{x}$. Xác suất của $\mathbf{y}$ là

$$\Pr(\mathbf{y}) = \prod_{i=1}^{|\mathbf{y}|} \Pr(y_i). \tag{3.14}$$

Khi đó, ta có thể viết độ hợp lý của $\mathbf{x}$ theo xác suất đồng thời của $\mathbf{x}$ và $\mathbf{y}$:

$$\Pr(\mathbf{x}) = \sum_{\mathbf{y} \in Y(\mathbf{x})} \Pr(\mathbf{x}, \mathbf{y}), \tag{3.15}$$

trong đó tổng được lấy trên mọi kết quả tách token có thể có $Y(\mathbf{x})$. Vì $\mathbf{x}$ được suy ra một cách tất định từ $\mathbf{y}$, xác suất đồng thời được đơn giản thành $\Pr(\mathbf{x}, \mathbf{y}) = \Pr(\mathbf{y})$. Do đó, ta có thể viết lại Phương trình (3.15) thành:

$$\begin{aligned} \Pr(\mathbf{x}) &= \sum_{\mathbf{y} \in Y(\mathbf{x})} \Pr(\mathbf{y}) \\ &= \sum_{\mathbf{y} \in Y(\mathbf{x})} \prod_{i=1}^{|\mathbf{y}|} \Pr(y_i). \end{aligned} \tag{3.16}$$

Chú giải của người dịch: Đọc Phương trình (3.16) bằng ví dụ

Ký hiệu $\prod$ là *tích* (giống $\sum$ là tổng): $\prod_{i=1}^{3} a_i = a_1 a_2 a_3$. Giả sử chuỗi $\mathbf{x}$ = "ab" và bộ từ vựng có {a, b, ab} với $\Pr(\text{a}) = 0.3$, $\Pr(\text{b}) = 0.2$, $\Pr(\text{ab}) = 0.1$. Có hai cách tách: "a/b" với xác suất $0.3 \times 0.2 = 0.06$, và "ab" với xác suất 0.1. Theo (3.16), $\Pr(\mathbf{x}) = 0.06 + 0.1 = 0.16$: ta *cộng* xác suất của mọi cách tách, vì đó là các khả năng loại trừ nhau dẫn đến cùng một chuỗi.

Từ phương trình này, log-độ hợp lý của một tập chuỗi X là

$$\begin{aligned} \Pr(X) &= \log \prod_{\mathbf{x} \in X} \sum_{\mathbf{y} \in Y(\mathbf{x})} \prod_{i=1}^{|\mathbf{y}|} \Pr(y_i) \\ &= \sum_{\mathbf{x} \in X} \log \left(\sum_{\mathbf{y} \in Y(\mathbf{x})} \prod_{i=1}^{|\mathbf{y}|} \Pr(y_i) \right). \end{aligned} \tag{3.17}$$

Nếu ta coi $-\Pr(X)$ là một hàm mất mát (*loss function*), thì nhiệm vụ ở đây có thể phát biểu là tìm ước lượng tốt nhất cho mỗi xác suất unigram $\Pr(y)$ sao cho $\Pr(X)$ lớn nhất có thể. Thoạt nhìn, bài toán tối ưu này có vẻ phức tạp. May mắn là có một số công cụ mạnh để giải nó. Một phương pháp phổ biến là dùng **thuật toán Kỳ vọng–Cực đại hóa** (*Expectation-Maximization*, **EM**) [Dempster và cộng sự, 1977], thường được dùng khi ta muốn tìm một mô hình thống kê làm cực đại độ hợp lý của dữ liệu. Lưu ý rằng lời giải dựa trên EM cho Phương trình (3.17) tương tự lời giải cho các bài toán NLP khác, chẳng hạn dịch máy thống kê, và đã được bàn kỹ trong những ngữ cảnh đó. Vì vậy, chúng tôi giới thiệu

[5] Thuật ngữ *kích thước bộ từ vựng* (*vocabulary size*) thường chỉ số mục trong bộ từ vựng. Tuy nhiên, trong một số ngữ cảnh tính toán, nó có thể chỉ tổng số byte cần để lưu bộ từ vựng.

bạn đọc tài liệu [Brown và cộng sự, 1993] để biết chi tiết về các phương pháp này. Trong chương này, ta dùng EM như một công cụ có sẵn để ước lượng $\Pr(y)$ dựa trên Phương trình (3.17).[6]

Chú giải của người dịch: Trực giác về thuật toán EM

Nếu biết sẵn mỗi câu được tách thế nào, ta chỉ việc *đếm* số lần mỗi từ con xuất hiện rồi chia cho tổng để có xác suất (như tính tần suất trong thống kê lớp 10). Khó khăn là ta không biết cách tách đúng. EM giải quyết như "con gà và quả trứng": đoán tạm các xác suất → dùng chúng để tính mỗi cách tách có khả năng bao nhiêu (bước E) → đếm "có trọng số": một cách tách có xác suất 0.7 thì mỗi từ con trong nó được cộng 0.7 lần, nên số đếm có thể lẻ, gọi là *số đếm phân số* → chuẩn hóa thành xác suất mới (bước M) → lặp lại. Mỗi vòng lặp không làm độ hợp lý giảm đi.

Về bản chất, SentencePiece là một phương pháp "cắt tỉa", loại bỏ các mục có xác suất thấp khỏi bộ từ vựng. Nó bắt đầu với một bộ từ vựng ban đầu lớn V. Chẳng hạn, ta có thể tạo bộ từ vựng ban đầu bằng cách liệt kê mọi chuỗi thỏa một ràng buộc về độ dài. Thông thường, các chuỗi vượt qua ranh giới từ bị loại để giảm kích thước bộ từ vựng. Sau đó, ta chạy các bước sau:

- Ước lượng xác suất cho mỗi mục y của V bằng cách tối ưu Phương trình (3.17).
- Tính mất mát cho mỗi mục y của V theo chiến lược *bỏ-một* (*remove-one*), tức là mất mát bằng lượng giảm của độ hợp lý (xem Phương trình (3.17)) khi y bị loại khỏi bộ từ vựng.
- Loại bỏ một tỉ lệ nhất định các mục của V có mất mát lớn. Chẳng hạn, ta giữ lại 80% số mục và bỏ phần còn lại.

Kết quả của quá trình này là một bộ từ vựng mới cùng với xác suất gán cho mỗi từ con. Ta có thể lặp lại quá trình này một số lần cho đến khi kích thước bộ từ vựng giảm đến mức mong muốn.

SentencePiece khác BPE và WordPiece ở chỗ nó xét mọi chuỗi từ con có thể có cho một chuỗi cho trước (xem tổng $\sum_{\mathbf{y}\in Y(\mathbf{x})}$ trong Phương trình (3.15)). Từ góc nhìn học máy, điều này có thể xem là một cách chính quy hóa (*regularization*), tức là ta có thể giảm nguy cơ ước lượng quá cao các tham số ứng với chuỗi từ con tốt nhất duy nhất, vốn có thể chứa lỗi. Một cách khác là chỉ xét một số chuỗi từ con trong $Y(\mathbf{x})$ để tăng hiệu quả. Chẳng hạn, ta có thể lấy mẫu k chuỗi từ con theo $\Pr(\mathbf{y})$ để tạo thành tập ứng viên $Y(\mathbf{x})$.

[6] Trong EM, ta có thể coi X là một *quan sát*, và $\Pr(X|\theta)$ là một mô hình thống kê mô tả khả năng quan sát đó xảy ra. Ở đây θ là các tham số mô hình mà ta cần xác định. EM dựa trên mục tiêu ước lượng hợp lý cực đại, tức là

$$\hat{\theta} \;=\; \arg\max_{\theta} \Pr(X|\theta). \tag{3.18}$$

Với mô hình ở đây, ta có thể coi $\{\Pr(y)\}$ là các tham số mô hình. Chúng tôi bỏ qua chi tiết suy diễn ước lượng EM của $\Pr(y)$ mà trình bày trực tiếp kết quả. Thuật toán EM gồm hai bước.

- **Bước Kỳ vọng** (hay *bước E*): Cho ước lượng hiện tại của $\Pr(y)$ (gọi là $\Pr_t(y)$), ta tính xác suất hậu nghiệm $\Pr_t(\mathbf{y})$ cho mỗi $\mathbf{y}$ theo Phương trình (3.14). Sau đó, ta tính **số đếm phân số** (*fractional count*) của mỗi từ con y trong bộ từ vựng V như sau

$$\text{fcount}(y) \;=\; \sum_{\mathbf{x}\in X}\sum_{\mathbf{y}\in Y(\mathbf{x})} \left(\Pr_t(\mathbf{y}) \sum_{i=1}^{|\mathbf{y}|} \delta(y, y_i) \right), \tag{3.19}$$

  trong đó $\delta(y, y_i)$ trả về 1 nếu $y = y_i$, và 0 nếu ngược lại. $\sum_{i=1}^{|\mathbf{y}|} \delta(y, y_i)$ đếm số lần y xuất hiện trong chuỗi từ con $\mathbf{y}$.
- **Bước Cực đại hóa** (hay *bước M*): Từ các số đếm phân số thu được ở bước E, ta ước lượng lại các xác suất unigram bằng phương trình:

$$\Pr_{t+1}(y) \;=\; \frac{\text{fcount}(y)}{\sum_{y'\in V} \text{fcount}(y')}. \tag{3.20}$$

Hai bước này được lặp lại qua nhiều vòng cho đến khi các tham số hội tụ về những giá trị nhất định.

chuỗi từ con	xác suất unigram ([từ con]:xác suất)
t/ow_/a/_flag	[t]:0.030 [ow_]:0.002 [a]:0.041 [_flag]:0.001
t/ow/_/a_/f/lag	[t]:0.030 [ow]:0.005 [_]:0.113 [a_]:0.093 [f]:0.041 [lag]:0.002
t/ow/_a_/fla/g	[t]:0.030 [ow]:0.005 [_a_]:0.084 [fla]:0.003 [g]:0.027
tow/_/a_/f/lag	[tow]:0.001 [_]:0.113 [a_]:0.093 [f]:0.041 [lag]:0.002
t/ow_/a_/flag	[t]:0.030 [ow_]:0.002 [a_]:0.093 [flag]:0.001

Hình 3.3: Các kết quả tách token khác nhau cho *tow a flag*. Mỗi từ con được gán một xác suất ước lượng bằng một mô hình ngôn ngữ unigram. Mỗi khoảng trắng được thay bằng "_" để trình bày rõ ràng.

Lưu ý rằng phương pháp SentencePiece không phụ thuộc vào chuỗi đầu vào đã được tách từ. Dù các phương pháp BPE và WordPiece cũng có thể xử lý văn bản thô nếu được điều chỉnh, phương pháp SentencePiece coi một cách tường minh dấu cách và các dấu phân cách khác là một phần của từ con. Xem Hình 3.3 để thấy một vài kết quả tách token cho *tow a flag*.

Chú giải của người dịch: Đọc Hình 3.3

Mỗi dòng là một cách tách; xác suất của cả cách tách bằng tích các xác suất trong dòng (Phương trình (3.14)). Ví dụ dòng cuối: $0.030 \times 0.002 \times 0.093 \times 0.001 \approx 5.6 \times 10^{-9}$. Chú ý dấu cách ("_") được coi như một ký tự bình thường, nên "a_" (chữ *a* kèm dấu cách theo sau) và "a" là hai từ con khác nhau. Nhờ vậy, từ chuỗi từ con có thể khôi phục chính xác văn bản gốc, kể cả vị trí các dấu cách — rất tiện cho các ngôn ngữ như tiếng Việt, nơi dấu cách nằm giữa các âm tiết.

Với một bộ từ vựng đã học và các xác suất unigram tương ứng, ta có thể dùng chúng để xử lý một văn bản mới. Đây thực chất là một bài toán tìm kiếm: ta tìm chuỗi từ con có khả năng nhất theo xác suất unigram. Vì mô hình hóa ngôn ngữ là một chủ đề đã được nghiên cứu kỹ trong NLP, nhiều thuật toán tìm kiếm có thể áp dụng trực tiếp cho trường hợp ở đây. Chẳng hạn, các phương pháp trình bày ở Mục 3.1.2 có thể áp dụng ngay.

3.2 Biểu diễn vectơ cho từ

Từ tự thân mang nghĩa[7]. Theo nghĩa rộng nhất, nghĩa của một từ đến từ cách nó được diễn giải. Đó là thứ nằm phía sau hình thức bề mặt của từ, nhưng người nói ngôn ngữ đó có thể hiểu được. Ví dụ, xét mấy dòng sau trong một bài thơ [Knight, 2018]:

There was a little sparrow
Who sat on a wheelbarrow,
And tweeted to all her friends around.
A cat with open jaws
And very pointed claws,
Spied her as he raced along the ground.

(Tạm dịch nghĩa: "Có một chú chim sẻ nhỏ / đậu trên chiếc xe cút kít, / ríu rít gọi bạn bè quanh đó. / Một con mèo há rộng hàm / với những móng vuốt thật nhọn, / phóng dọc mặt đất và rình thấy nó.")

[7]Dù đến giờ ta đã bàn về một số thành phần ngôn ngữ dùng trong NLP như từ con, ở đây ta vẫn dùng từ làm đơn vị cơ bản trong thảo luận. Các phương pháp trình bày trong phần còn lại của chương này nhìn chung có thể áp dụng cho các đơn vị ngôn ngữ khác dùng trong hệ thống NLP, bao gồm ký tự, từ con, v.v.

Những từ này không chỉ là các chuỗi ký tự tùy ý; chúng gợi ra những khái niệm cụ thể. Chẳng hạn, "little" nghĩa là nhỏ về kích thước, "sparrow" nghĩa là một loài chim (chim sẻ), còn "friends" nghĩa là những người bạn quý mến và tin tưởng. Tuy nhiên, từ góc nhìn NLP, nghĩa của từ (hay nghĩa từ (*word sense*)) không thể chỉ dừng lại là một khái niệm trừu tượng trong nhận thức; nó phải được hình thức hóa thành một dạng máy có thể đọc và tính toán được.

3.2.1 Biểu diễn one-hot

Một cách tự nhiên để biểu diễn nghĩa của từ là dùng ngôn ngữ để mô tả chúng. Chẳng hạn, ta có thể tra các từ vừa nhắc đến ở trên trong từ điển để tìm mã số (id) và định nghĩa của chúng[8]:

cat	511	*A small animal with fur, four legs, a tail, and claws, usually kept as a pet or for catching mice* (con mèo: động vật nhỏ có lông, bốn chân, đuôi và móng vuốt, thường nuôi làm thú cưng hoặc để bắt chuột)
her	5220	*Used, usually as the object of a verb or preposition, to refer to a woman, girl, or female animal that has just been mentioned or is just about to be mentioned* (cô ấy/nó: thường làm tân ngữ của động từ hoặc giới từ, chỉ một người phụ nữ, cô gái hay con vật cái vừa hoặc sắp được nhắc đến)
jaws	6186	*The mouth, including the teeth* (hàm: miệng, kể cả răng)
ground	6402	*The surface of the earth* (mặt đất)
sparrow	8331	*A common, small, gray-brown bird* (chim sẻ: loài chim nhỏ, phổ biến, màu nâu xám)
wheelbarrow	9954	*A large, open container for moving things in with a wheel at the front and two handles at the back, used especially in the garden* (xe cút kít: thùng lớn, hở, để chở đồ, có một bánh xe phía trước và hai tay cầm phía sau, dùng nhiều trong vườn)

Để biểu diễn một từ, ý tưởng đơn giản nhất là thay nó bằng id trong từ điển. Theo cách này, biểu diễn của mỗi từ là một con số duy nhất. Một dạng tương đương là **biểu diễn one-hot** (*one-hot representation*). Đó là một vectơ có số chiều bằng kích thước bộ từ vựng. Trong vectơ này, chỉ phần tử ứng với chỉ số của từ được đặt bằng 1, còn mọi phần tử khác đều bằng 0. Ví dụ, từ *sparrow* có thể được biểu diễn bằng vectơ one-hot $\mathbf{1}_{\text{sparrow}}$ dựa trên id của nó (8331):

$$\begin{array}{c} [\ 0\ 0\ \ldots\ 0\ \ 1\ \ 0\ \ldots\ 0\ 0\ ] \\ \qquad\quad\ \uparrow \\ \qquad \text{id} = 8331 \end{array}$$

Chú giải của người dịch: Vectơ one-hot

Giả sử bộ từ vựng chỉ có 4 từ: {*cat*, *dog*, *bird*, *tree*}. Khi đó $cat = [1\ 0\ 0\ 0]$, $dog = [0\ 1\ 0\ 0]$, $bird = [0\ 0\ 1\ 0]$, $tree = [0\ 0\ 0\ 1]$. Tên gọi "one-hot" ("một điểm nóng") chỉ việc đúng một vị trí "bật sáng". Với bộ từ vựng thật cỡ vài chục nghìn từ, mỗi vectơ dài vài chục nghìn chiều nhưng gần như toàn số 0.

3.2.2 Biểu diễn phân tán

Tuy nhiên, có vẻ như biểu diễn one-hot chỉ cung cấp "danh tính" của từ chứ không cung cấp "mô tả" xem từ đó là gì. Một vấn đề hiển nhiên là mọi từ đều *trực giao* với mọi từ khác. Do tính trực giao này,

[8] Tất cả các từ này và nghĩa của chúng được lấy từ `https://dictionary.cambridge.org/`.

không thể định lượng bằng toán học mối quan hệ ngữ nghĩa giữa các từ, kể cả những từ có liên hệ rất chặt trong ngôn ngữ tự nhiên. Ở đây, điều ta mong muốn là một mô hình trong đó các từ được mô tả bằng các thuộc tính định lượng được và sự gần gũi về ngữ nghĩa được nắm bắt một cách tự nhiên. Một cách làm điều đó là làm giàu biểu diễn bằng phần mô tả của từ. Xét lại từ *sparrow* làm ví dụ. Trong từ điển, nghĩa của nó là *a common, small, gray-brown bird*. Dùng các phương pháp tách token và chuẩn hóa nêu ở Mục 3.1.1, đoạn văn bản này có thể được chuyển thành một chuỗi từ

$$\left[\begin{array}{cccccccc} a & common & , & small & , & gray & - & brown & bird \end{array}\right]$$

Sau đó, ta vectơ hóa chuỗi này bằng mô hình túi từ (*bag-of-words*) (xem Chương 1), thu được một vectơ số mới

$$\begin{array}{c} [\,0\ 0\ \ldots\ 1\ \ldots\ 1\ \ldots\ 1\ \ldots\ 1\ \ldots\ 1\ \ldots\ 1\ \ldots\ 1\ \ldots\ 1\ \ldots\ 0\ 0\,] \\ \qquad\quad \uparrow \quad \uparrow \quad \uparrow \quad \uparrow \quad \uparrow \quad \uparrow \quad \uparrow \quad \uparrow \\ \qquad\quad , \quad - \quad a \quad bird \quad brown \quad common \quad gray \quad small \end{array}$$

trong đó giá trị của một phần tử bằng 1 nếu từ tương ứng có mặt, và bằng 0 nếu ngược lại. Cách này cho phép các từ chia sẻ nội dung với nhau. Ta nói hai từ là tương tự nhau nếu vectơ từ của chúng có phần trùng lặp. Xét một từ mới *cuckoo* (chim cu cu). Ta có thể tìm nghĩa của nó trong từ điển, ví dụ *a grey bird with a two-note call that sounds similar to its name* (một loài chim xám có tiếng kêu hai nốt nghe giống tên của nó). Dễ thấy *sparrow* và *cuckoo* là hai từ có điểm tương đồng, vì cả hai đều đánh dấu chiều "bird" bằng 1 và độ tương tự vectơ giữa hai vectơ từ này lớn hơn 0[9].

Chú giải của người dịch: Trực giao và độ tương tự cosin

Ở lớp 10, hai vectơ vuông góc khi tích vô hướng của chúng bằng 0, và $\cos(\vec{u}, \vec{v}) = \dfrac{\vec{u} \cdot \vec{v}}{|\vec{u}|\,|\vec{v}|}$. Với vectơ nhiều chiều, công thức y hệt: tích vô hướng là tổng các tích từng cặp tọa độ. Hai vectơ one-hot khác nhau luôn có tích vô hướng bằng 0 (không có vị trí nào cùng bằng 1), nên cosin bằng 0: *cat* "xa" *dog* đúng bằng *cat* "xa" *tree*. Còn với vectơ túi từ ở trên, *sparrow* và *cuckoo* cùng có 1 ở chiều "bird" (và chiều "a"), nên tích vô hướng dương và cosin lớn hơn 0.

Coi từ là vectơ số cung cấp một công cụ tổng quát để biểu diễn từ theo nhiều cách khác nhau. Ta thậm chí không cần giải thích vectơ từ dưới góc độ ngữ nghĩa. Chẳng hạn, ta có thể thêm một chiều mới vào vectơ để đánh dấu từ có thuộc một phạm trù cú pháp nào đó hay không. Theo nghĩa rộng, ta có thể định nghĩa một hàm tùy ý trên mỗi phần tử của vectơ và coi đầu ra của hàm là một đặc trưng (*feature*) mô tả từ. Ví dụ, một cải tiến đơn giản cho biểu diễn trên là dùng hàm đếm số lần xuất hiện của từ thay cho hàm nhị phân đánh dấu sự có mặt của từ. Có thể tìm thêm các hàm đặc trưng khác ở Mục 3.3.

Lưu ý rằng không nhất thiết phải ràng buộc các hàm đặc trưng ở những dạng có ý nghĩa ngôn ngữ học, dù các thiết kế hàm đặc trưng xuất phát từ ngôn ngữ học thường được các nhà nghiên cứu NLP quan tâm. Một dạng tổng quát hơn để biểu diễn từ đơn giản là một vectơ số thực nhiều chiều. Nó thường được gọi là **biểu diễn phân tán** (*distributed representation*) của từ, hay **nhúng từ** (*word embedding*). Chẳng hạn, từ *sparrow* có thể được biểu diễn bằng vectơ

$$\left[\begin{array}{cccccc} 1.9 & -7 & 3 & -1.2 & \ldots & 2.01 & -2.05 \end{array}\right]$$

Từ góc nhìn học máy, vectơ này có thể mô tả một số thuộc tính tiềm ẩn của từ. Những thuộc tính này có thể không giải thích được theo cách hiểu của con người nhưng có thể học được từ dữ liệu. Một trong

[9] Độ tương tự của hai vectơ có thể được đo bằng cosin của góc giữa chúng.

những thách thức khi học biểu diễn như vậy là khó đo được độ tốt của một vectơ. Nhìn chung, hỏi biểu diễn phân tán của một từ đơn lẻ là tốt hay không là vô nghĩa. Thay vào đó, ta muốn biết biểu diễn của một nhóm từ có "cư xử" hợp lý hay không. Chẳng hạn, người ta thường tin rằng các từ tương tự nhau nên có biểu diễn tương tự nhau. Vì vậy, quan hệ giữa các từ thường được hình dung như một loại "khoảng cách" giữa các biểu diễn từ trong một không gian vectơ. Điều này dẫn tới nhiều phương pháp trực quan hóa và đánh giá biểu diễn từ. Ở Mục 3.6, chúng tôi sẽ bàn chi tiết hơn về những vấn đề này.

Chú giải của người dịch: Vì sao gọi là "phân tán"?

Trong vectơ one-hot, toàn bộ thông tin về một từ "dồn" vào đúng một vị trí. Trong biểu diễn phân tán, thông tin được *rải* ra trên tất cả các chiều: mỗi chiều có thể phản ánh một phần ý nghĩa (một cách mơ hồ, chẳng hạn "mức độ liên quan đến động vật"), và mỗi từ là một tổ hợp riêng của các giá trị đó. Vectơ thường ngắn (vài trăm chiều) và dày đặc (hầu hết khác 0), trái ngược với vectơ one-hot dài và thưa.

Thông thường, biểu diễn từ không hoạt động độc lập trong các hệ thống NLP mà được dùng như những trạng thái trung gian của một mô hình. Một cách tiếp cận chuẩn trong NLP để học biểu diễn phân tán của từ là coi nó là *sản phẩm phụ* của việc huấn luyện một hệ thống "lớn". Tức là mô hình biểu diễn hoạt động như một thành phần của hệ thống, và được tối ưu cùng các thành phần khác khi hệ thống được huấn luyện theo một cách nào đó. Điều này gợi ra một mô hình học biểu diễn (*representation learning*) đầy hứa hẹn: mô hình biểu diễn được học như một mô hình con trong một hệ thống dễ huấn luyện, rồi có thể dùng như một mô-đun "cắm vào" cho một hệ thống hoàn toàn khác. Chẳng hạn, trong mô hình hóa ngôn ngữ bằng mạng nơ-ron, ta có thể buộc mô hình ánh xạ mỗi vectơ từ one-hot đầu vào thành một biểu diễn phân tán số thực, ít chiều. Các biểu diễn phân tán này được đưa vào một mạng nơ-ron (*neural network*) dự đoán một phân phối xác suất (*probability distribution*) trên bộ từ vựng. Hàm ánh xạ hay *hàm nhúng* được huấn luyện sao cho cực tiểu hóa mất mát của mô hình ngôn ngữ trên một tập dữ liệu nào đó (xem Mục 3.4). Khi áp dụng hàm nhúng đã học, ta bỏ mọi phần khác của mô hình ngôn ngữ và dùng hàm này để sinh biểu diễn phân tán cho mỗi từ trong các tác vụ phía sau. Một chiến lược khác là thiết kế mô hình chuyên biệt cho bài toán học biểu diễn từ. Các hệ thống loại này thường không được thiết kế để giải các bài toán NLP tiêu chuẩn, mà nhấn mạnh vào các vấn đề cụ thể trong học biểu diễn từ, chẳng hạn mô hình hóa tường minh quan hệ giữa các từ (xem Mục 3.5).

Chú giải của người dịch: Hàm nhúng thực chất là "tra bảng"

Nhân vectơ one-hot của từ thứ k với một bảng số (ma trận) có mỗi hàng là một vectơ nhúng, kết quả chính là hàng thứ k của bảng. Vì vậy, hàm nhúng có thể hình dung như một cuốn "từ điển số": tra id của từ, lấy ra vectơ của nó. Các con số trong bảng ban đầu được chọn ngẫu nhiên, rồi được điều chỉnh dần trong quá trình huấn luyện mô hình ngôn ngữ.

3.2.3 Tính kết hợp và tính phụ thuộc ngữ cảnh

Dù thảo luận của ta giới hạn ở việc học biểu diễn từ trong NLP, nghiên cứu nghĩa của từ là một phân ngành truyền thống của ngôn ngữ học. Chẳng hạn, trong ngữ nghĩa học từ vựng (*lexical semantics*), các nhà nghiên cứu tìm hiểu nghĩa của từ được định nghĩa thế nào và chúng kết hợp tạo nên nghĩa ở cấp câu ra sao. Về mặt lịch sử, biểu diễn từ trên máy tính không bám chặt vào ngữ nghĩa học ngôn ngữ, mà tập trung vào việc chuyển các đơn vị ngôn ngữ thành dạng tính toán được. Tuy nhiên, vì ngữ nghĩa là nền tảng của việc hiểu ngôn ngữ, việc đưa các nguyên lý ngữ nghĩa vào thiết kế mô hình biểu diễn vẫn rất quan trọng. Xét các tính chất sau:

- **Tính kết hợp** (*compositionality*). Tính kết hợp là một khái niệm phổ biến trong ngữ nghĩa học,

logic học và các lĩnh vực liên quan. Nó thường đi kèm với **nguyên lý kết hợp** (*principle of compositionality*):

> *Nghĩa của một biểu thức phức hợp được xác định bởi cấu trúc của nó và nghĩa của các thành phần cấu tạo nên nó.*
>
> – Szabó [2020]

Nguyên lý kết hợp là nền tảng và có mặt ở khắp nơi trong ngôn ngữ tự nhiên. Chẳng hạn, nghĩa của cụm *white cat* (mèo trắng) dễ dàng suy ra từ nghĩa riêng của *white* (trắng) và *cat* (mèo). Tương tự, câu ghép có được nghĩa tổng thể từ sự kết hợp của các mệnh đề độc lập và liên từ. Lưu ý rằng nguyên lý kết hợp không chỉ là một quy tắc đơn giản ta dùng để mô tả cách các đơn vị ngôn ngữ lớn được tạo nên từ các đơn vị nhỏ, dù các nhà nghiên cứu đã cố gắng định nghĩa nó một cách hình thức [Montague, 1974]. Vẫn đang có những tranh luận về việc nguyên lý này nên được diễn giải và mô hình hóa thỏa đáng bởi các lý thuyết ngữ nghĩa như thế nào. Dẫu vậy, nếu ta tập trung vào các ứng dụng NLP và gác lại các tranh luận lý thuyết ngôn ngữ học, tính kết hợp vẫn là một tính chất rất có giá trị cho việc thiết kế và đánh giá hệ thống. Khi một bài toán có tính kết hợp, điều đó hàm ý rằng các tác vụ phức tạp có thể được phân rã một cách có hệ thống thành các thành phần con dễ xử lý hơn. Với việc học biểu diễn từ, ta mong các vectơ thu được thể hiện tính kết hợp để phản ánh bản chất tổ hợp của ngôn ngữ. Ở Mục 3.6, ta sẽ thấy một số ví dụ về điều này. Chẳng hạn, các biểu diễn học được bởi mạng nơ-ron thường thể hiện những tính chất đại số có ý nghĩa dưới các phép toán tuyến tính, dù bản thân các mô hình là phi tuyến. Tuy nhiên, tính kết hợp không phải là quy tắc tuyệt đối. Nó thường bị phá vỡ trong các trường hợp như *cụm từ cố định* (*collocation*) và *thành ngữ* (*idiom*), nơi ngôn ngữ tự nhiên thể hiện tính phi kết hợp. Điều này giải thích vì sao các bài toán NLP lại khó đến vậy.

- **Tính phụ thuộc ngữ cảnh** (*contextuality*). Tính phụ thuộc ngữ cảnh thể hiện một dạng phi kết hợp. Nó nói rằng một từ có thể có nhiều nghĩa, và cách diễn giải cụ thể của nó được quyết định bởi ngữ cảnh (*context*) xung quanh. Ví dụ, xét các câu sau[10]

  They sat round the dinner table, arguing about politics.
  (Họ ngồi quanh bàn ăn tối, tranh luận về chính trị.)
  Come to the table everybody - supper's ready.
  (Mọi người lại bàn nào, bữa tối xong rồi.)
  He came in with four shopping bags and dumped them on the table.
  (Anh ấy bước vào với bốn túi đồ và quăng chúng lên bàn.)
  The table can help you evaluate the potential risks of investing in the Fund.
  (Bảng này có thể giúp bạn đánh giá rủi ro tiềm ẩn khi đầu tư vào Quỹ.)
  Building societies dominate the best-value tables for mortgages.
  (Các hiệp hội nhà ở chiếm lĩnh các bảng xếp hạng khoản vay thế chấp có lợi nhất.)
  This table represents export sales.
  (Bảng này thể hiện doanh số xuất khẩu.)

  Trong các câu ví dụ này, *table* là một từ đa nghĩa (*polysemous word*) với hai nghĩa:

  > *Sense 1: a flat surface used for putting things on.* (Nghĩa 1: một mặt phẳng dùng để đặt đồ vật lên — cái bàn.)

[10] Tất cả các câu này được lấy từ `https://dictionary.cambridge.org/dictionary/english/table`

> *Sense 2: an arrangement of items in rows, or columns, or blocks.* (Nghĩa 2: sự sắp xếp các mục theo hàng, cột hoặc khối — cái bảng.)

Nói cách khác, *table* là một từ nhập nhằng. Sự nhập nhằng này thường được giải quyết bằng cách xem xét các từ lân cận. Chẳng hạn, khi *table* đứng sau *dinner*, dễ nhận ra nó mang nghĩa 1. Sự nhập nhằng cũng tồn tại khi một từ bắt nguồn từ vài dạng hay *từ vị* (*lexeme*) khác nhau (hiện tượng được gọi là **đồng âm** (*homonymy*)). Ví dụ, *bear* có thể là động từ (chịu đựng) hoặc danh từ (con gấu). Khử nhập nhằng một từ đối với một tập nghĩa từ cho trước đã được nghiên cứu hàng chục năm trong NLP và thường được gọi là **khử nhập nhằng nghĩa từ** (*word sense disambiguation, WSD*) [Kelly and Stone, 1975]. Tuy nhiên, bài toán biểu diễn từ bàn ở đây khó hơn, vì ta thường không có sẵn một tập nghĩa từ định trước. Thay vào đó, ta muốn có một *mô hình biểu diễn theo ngữ cảnh* có thể sinh ra biểu diễn của một từ phụ thuộc vào ngữ cảnh của nó. Do đó, cần nhận thức rằng nghĩa của từ không cố định. Điều này khiến bài toán có phần khác với những gì ta bàn ở đầu mục, vì ta không còn một bảng tra cứu biểu diễn từ, mà là một mô hình sinh ra các biểu diễn khác nhau cho cùng một từ trong các ngữ cảnh khác nhau.

Chú giải của người dịch: Ví dụ trong tiếng Việt

Tính kết hợp: nghĩa của "mèo trắng" = nghĩa "mèo" + nghĩa "trắng". *Phi kết hợp*: thành ngữ "ăn cháo đá bát" không nói về cháo hay bát, mà chỉ người vô ơn; ghép nghĩa từng chữ sẽ hiểu sai. *Phụ thuộc ngữ cảnh*: từ "đường" trong "ăn *đường*" (chất ngọt) và "đi *đường*" (lối đi) là hai nghĩa khác hẳn; "bay" trong "chim bay" và "cái bay của thợ xây". Một vectơ cố định duy nhất cho "đường" khó biểu diễn đúng cả hai nghĩa — đây là động lực cho các mô hình biểu diễn theo ngữ cảnh (như BERT, GPT) ở các chương sau.

Trong các mục còn lại của chương này, chúng tôi tập trung vào việc học biểu diễn vectơ của từ từ sự phân bố của chúng trong việc sử dụng ngôn ngữ. Phần thảo luận về các mô hình theo ngữ cảnh để học biểu diễn từ dày đặc được để lại cho Chương 4–6.

3.3 Các mô hình dựa trên đếm

Chúng ta đã đặt việc quy nạp nghĩa của từ thành bài toán học các vectơ từ. Trong mục này, ta đi tiếp với giả định rằng nghĩa của một từ được quyết định bởi môi trường mà từ đó được sử dụng. Điều này thường được phát biểu thành **giả thuyết phân bố** (*distributional hypothesis*): các từ có nghĩa tương tự nhau nếu chúng xuất hiện trong những ngữ cảnh tương tự nhau [Harris, 1954; Firth, 1957]. Một biểu diễn từ được học theo giả thuyết này còn được gọi là **biểu diễn từ phân bố** (*distributional word representation*) hay **biểu diễn phân bố** (*distributional representation*)[11]. Tuy nhiên, để dễ đọc, trong suốt cuốn sách này chúng tôi vẫn sẽ dùng các thuật ngữ *vectơ từ* và *biểu diễn từ*. Tiếp theo, chúng tôi giới thiệu một vài phương pháp mô hình hóa sự phân bố của từ trong văn bản, rồi đưa ra một số cải tiến.

[11] Cần lưu ý rằng *biểu diễn phân bố* (*distributional representation*) và *biểu diễn phân tán* (*distributed representation*) là hai khái niệm khác nhau. Biểu diễn phân bố là biểu diễn mô tả sự phân bố của các đơn vị ngôn ngữ trong quá trình sử dụng ngôn ngữ. Một thuật ngữ liên quan là *biểu diễn phi phân bố* (*non-distributional representation*), chỉ những biểu diễn thu được từ các cơ sở dữ liệu từ vựng, chẳng hạn lời giải nghĩa của một từ trong từ điển. Mặt khác, biểu diễn phân tán là một vectơ gồm các biến tương ứng với một số thuộc tính tiềm ẩn của một đơn vị ngôn ngữ. Trái với biểu diễn phân tán, biểu diễn one-hot chỉ mô tả ký hiệu của từ.

Chú giải của người dịch: Giả thuyết phân bố nói gì?

Ý tưởng rất gần với câu tục ngữ "*Hãy cho tôi biết bạn chơi với ai, tôi sẽ nói bạn là người thế nào*". Nhà ngôn ngữ học Firth có câu nổi tiếng: "*Ta biết một từ qua những từ đi cùng nó*". Chẳng hạn, nếu bạn gặp một từ lạ "chôm chôm" trong các câu "chôm chôm chín đỏ", "ăn chôm chôm", "vườn chôm chôm", bạn có thể đoán đó là một loại quả, vì các từ xung quanh (*chín*, *ăn*, *vườn*) cũng hay đi cùng "xoài", "nhãn". Các phương pháp trong mục này biến ý tưởng ấy thành con số: *đếm* xem mỗi từ hay đi cùng những từ nào.

3.3.1 Ma trận đồng xuất hiện

Trong **ngữ nghĩa học phân bố** (*distributional semantics*), các từ được biểu diễn bằng những mô hình ngữ nghĩa có xét đến nhiều khía cạnh khác nhau của ngữ cảnh. Các mô hình này khác nhau ở cách mô hình hóa ngữ cảnh của một từ, chẳng hạn ngữ cảnh được xét rộng đến đâu, mỗi lần xuất hiện của một từ được đếm như thế nào, số chiều của một phân bố được định nghĩa ra sao, v.v. Trong mục này, giống như phần lớn các mô hình dùng trong NLP, ta giả định rằng biểu diễn từ được học từ một tập hợp các văn bản.

Một văn bản có thể được xem đơn giản là một tập hợp các từ không có thứ tự (tức là giả định túi từ (*bag-of-words*)). Khi đó, ta có thể coi mỗi lần xuất hiện của các từ này là một dấu hiệu ngữ cảnh độc lập. Bằng cách này, sự phân bố của một từ trong ngữ cảnh của nó có thể được mô tả bằng số lần từ đó đồng xuất hiện (*co-occur*) với các từ ngữ cảnh. Điều này được thực hiện bằng cách xây dựng một ma trận đồng xuất hiện (*co-occurrence matrix*), trong đó mỗi ô ghi lại tần suất đồng xuất hiện của một mục ở hàng và một mục ở cột. Hãy xét ví dụ các văn bản sau[12]:

Văn bản 1 *A berry is a small, pulpy, and often edible fruit.*
(Quả mọng là một loại quả nhỏ, nhiều thịt và thường ăn được.)
Văn bản 2 *In botanical terminology, a berry is a simple fruit with seeds and pulp produced from the ovary of a single flower.*
(Theo thuật ngữ thực vật học, quả mọng là một quả đơn có hạt và thịt quả, được tạo ra từ bầu nhụy của một bông hoa duy nhất.)
Văn bản 3 *The term "banana" is also used as the common name for the plants that produce the fruit.*
(Thuật ngữ "chuối" cũng được dùng làm tên gọi thông thường cho các loài cây sinh ra loại quả này.)
Văn bản 4 *Banana seeds are large and hard and spiky and liable to crack teeth.*
(Hạt chuối to, cứng, có gai và dễ làm mẻ răng.)
Văn bản 5 *A banana is an elongated, edible fruit - botanically a berry - produced by several kinds of large herbaceous flowering plants in the genus Musa.*
(Chuối là một loại quả thuôn dài, ăn được – về mặt thực vật học là một quả mọng – do một số loài cây thân thảo lớn có hoa thuộc chi Musa sinh ra.)

Với mỗi cặp từ, ta thu thập tổng số lần chúng đồng xuất hiện trong các văn bản này, dẫn tới một ma trận được gọi là **ma trận đồng xuất hiện từ–từ** (*word-word co-occurrence matrix*) hay **ma trận đồng xuất hiện thuật ngữ–thuật ngữ** (*term-term co-occurrence matrix*). Dưới đây là một phần của ma trận cho các văn bản trên (các từ tiếng Anh được giữ nguyên vì ma trận được đếm trên chính các văn bản tiếng Anh này).

[12]Các đoạn văn bản được lấy từ Wikipedia.

	flowering	*fruit*	*herbaceous*	…	*often*	*plants*	*seeds*
berry	1	3	1	…	1	1	1
terminology	0	1	0	…	0	0	1
common	0	1	0	…	0	1	0
teeth	0	0	0	…	0	0	1
banana	1	2	1	…	0	2	1
simple	0	1	0	…	0	0	1
and	0	2	0	…	1	0	2

Trong ma trận này, mỗi từ ở hàng gắn với một vectơ từ có $|V|$ phần tử. Các con số trong các phần tử mô tả mức độ thường xuyên mà từ ở hàng đồng xuất hiện với các từ ngữ cảnh khác nhau, tức là một từ cho trước phân bố như thế nào trong các "ngữ cảnh" khác nhau. Theo nghĩa hình học, nếu hai từ có phân bố tương tự nhau khi đồng xuất hiện với cùng một nhóm từ ngữ cảnh thì góc giữa hai vectơ từ của chúng sẽ nhỏ[13]. Ví dụ, nếu ta coi các từ này là các vectơ trong một không gian vectơ, thì *berry* (quả mọng) gần *banana* (chuối) hơn là *teeth* (răng) (xem Hình 3.4). Trực giác hình học này là nền tảng của nhiều mô hình biểu diễn. Các ví dụ khác sẽ được trình bày trong Chương 4 và Chương 5.

Chú giải của người dịch: Đọc ma trận đồng xuất hiện

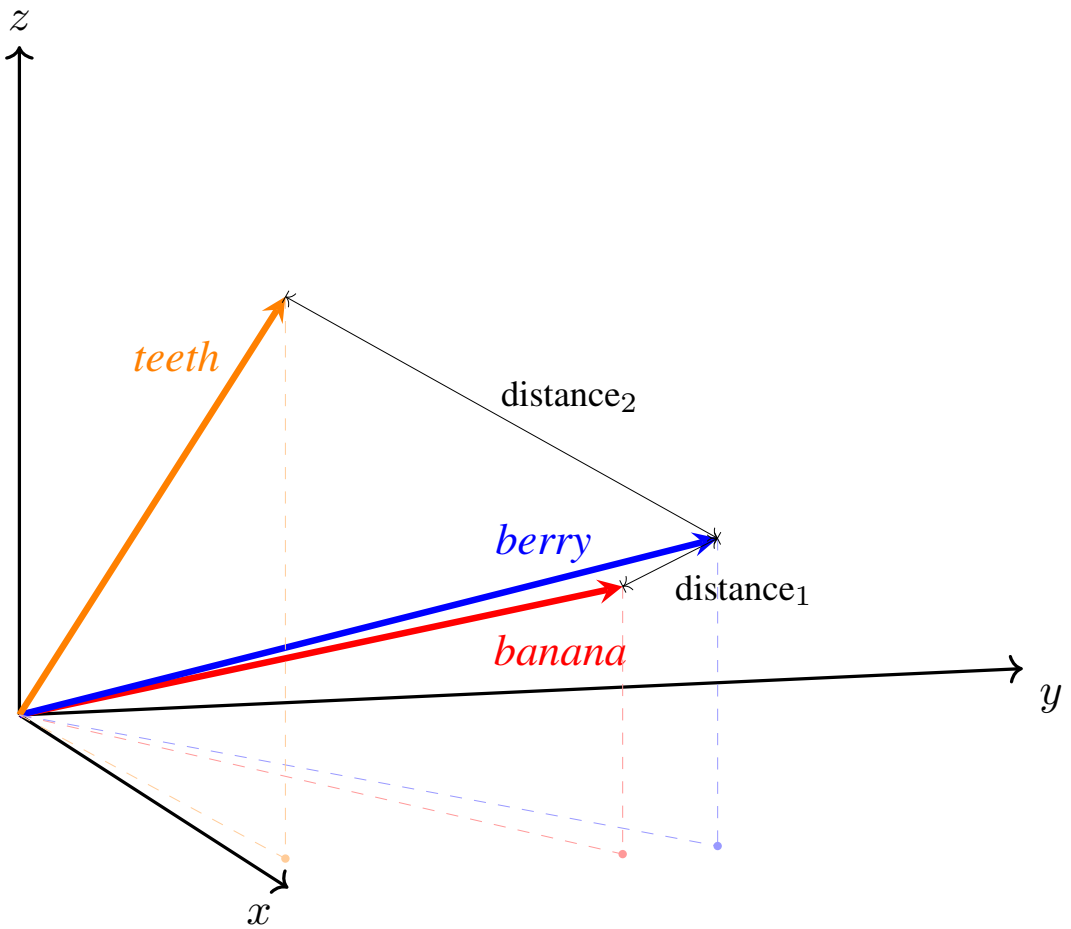
Hàng *berry* là $[1,\ 3,\ 1, \ldots, 1,\ 1,\ 1]$: từ *berry* đi cùng *fruit* 3 lần, cùng *flowering* 1 lần, v.v. Hàng *banana* là $[1,\ 2,\ 1, \ldots, 0,\ 2,\ 1]$ – khá giống hàng *berry*. Còn hàng *teeth* là $[0,\ 0,\ 0, \ldots, 0,\ 0,\ 1]$, hầu như không chung "bạn bè" với *berry*. Mỗi hàng chính là một vectơ nhiều chiều (mỗi cột là một tọa độ). Hai vectơ "cùng hướng" (góc nhỏ) nghĩa là hai từ hay xuất hiện với những từ giống nhau, nên có lẽ có nghĩa gần nhau. Đây chính là ý tưởng góc giữa hai vectơ mà bạn đã học ở lớp 10, chỉ là với nhiều tọa độ hơn.

Một vấn đề của phương pháp này là khoảng cách giữa các từ không được tính đến, mặc dù mối tương quan không còn mạnh lắm khi từ ngữ cảnh ở xa. Một giải pháp đơn giản là giới hạn các từ ngữ cảnh trong một cửa sổ, gọi là **cửa sổ ngữ cảnh** (*context window*), hay gọi tắt là cửa sổ [Lund và Burgess, 1996]. Ví dụ, với mỗi từ trong một văn bản, ta chỉ đếm các từ ở vị trí -2 đến $+2$ xung quanh nó (tức là một cửa sổ kích thước 5).

Lưu ý rằng các vectơ từ học được bằng mô hình túi từ ở Mục 3.2 là một trường hợp đặc biệt của ma trận đồng xuất hiện. Trong ví dụ đó, toàn bộ kho ngữ liệu chỉ gồm một văn bản duy nhất, và một hàm chỉ thị đánh dấu sự có mặt thay vì tần suất. Ngoài các phép đếm đơn giản hay các chỉ thị nhị phân, các phần tử của vectơ từ có thể được điền bằng nhiều thước đo mức độ liên kết khác nhau. Trong thực tế, giá trị của một phần tử trong vectơ từ có thể được xem là mức độ tương ứng giữa các từ. Nếu hai từ tương quan với nhau trong một ngữ cảnh nào đó, một hàm đặc trưng có thể gán cho chúng một điểm số theo bất kỳ cách nào. Điểm số này không nhất thiết phải là một số đếm, mà có thể là một số thực bất kỳ. Do đó, bài toán có thể được phát biểu thành việc đo độ mạnh liên kết giữa các từ. Thông thường người ta định nghĩa thước đo như vậy dựa trên các mô hình tương quan. Trong thống kê, tương quan mô tả mức độ hai biến liên hệ với nhau, được đo bằng các **hệ số tương quan** (*correlation coefficients*). Các hệ số tương quan phổ biến gồm hệ số tương quan Pearson (r của Pearson), hệ số tương quan hạng Spearman (ρ của Spearman), v.v.[14]. Trong NLP, một thước đo được dùng rộng rãi là **thông tin tương hỗ theo điểm** (*pointwise mutual information*) (**PMI**) [Church và Hanks, 1990]. Gọi a và b là hai từ.

[13] Góc giữa hai vectơ không phụ thuộc vào độ lớn của chúng. Nếu các vectơ được chuẩn hóa theo một cách nào đó (ví dụ theo chuẩn của vectơ), thì các vectơ tương tự nhau có nghĩa là phần lớn các phần tử của hai vectơ có giá trị gần nhau.

[14] Một số hệ số tương quan giả định dữ liệu tuân theo những phân phối nhất định. Ví dụ, hệ số tương quan Pearson được tính dựa trên hai biến tuân theo phân phối chuẩn.



Hình 3.4: Các vectơ từ trong một không gian vectơ được xây dựng từ thống kê đồng xuất hiện của từ trên dữ liệu tiếng Anh của WMT 2012. Tất cả các vectơ đều được chuẩn hóa và được biểu diễn bằng các mũi tên. Để trực quan hóa, ta chiếu các vectơ này từ không gian nhiều chiều xuống không gian 3 chiều bằng phép phân tích thành phần chính. Đúng như kỳ vọng, *berry* gần *banana* hơn là gần *teeth*. (Trên hình: *distance* = khoảng cách.)

Dạng toán học của PMI là

$$\text{PMI}(a, b) \;=\; \log \frac{\Pr(a, b)}{\Pr(a)\Pr(b)}, \tag{3.21}$$

trong đó $\Pr(a, b)$ là xác suất đồng thời để a và b cùng xuất hiện, và $\Pr(a)$ (hoặc $\Pr(b)$) là xác suất xuất hiện của a (hoặc b). Các xác suất này có thể được ước lượng đơn giản trên văn bản bằng phương pháp tần suất tương đối[15]. Cho một từ a và một bộ từ vựng gồm các từ ngữ cảnh $\{b_1, \cdots, b_{|V|}\}$, vectơ từ dựa trên PMI của a được viết là

$$e(a) \;=\; \Big[\text{PMI}(a, b_1) \;\; \cdots \;\; \text{PMI}(a, b_{|V|})\Big]. \tag{3.22}$$

Chú giải của người dịch: Ý nghĩa của PMI qua một ví dụ số

Nếu hai từ a, b *độc lập* (không liên quan gì nhau) thì, như ở xác suất THPT, $\Pr(a, b) = \Pr(a)\Pr(b)$, nên tỉ số bằng 1 và $\text{PMI} = \log 1 = 0$. PMI dương nghĩa là hai từ đi cùng nhau *thường hơn* mức ngẫu nhiên; PMI âm nghĩa là chúng "tránh" nhau. Ví dụ: giả sử trong kho ngữ liệu, $\Pr(\text{"trà"}) = 0.01$, $\Pr(\text{"đá"}) = 0.02$. Nếu độc lập, xác suất gặp cặp "trà đá" chỉ là $0.01 \times 0.02 = 0.0002$. Nhưng thực tế ta đếm được $\Pr(\text{trà, đá}) = 0.002$, gấp 10 lần, nên $\text{PMI} = \log_2 10 \approx 3.3$ (nếu dùng log cơ số 2): hai từ này liên kết rất chặt. Chú thích chân trang 15 cảnh báo: với từ rất hiếm, $\Pr(a)$ rất nhỏ ở mẫu số khiến PMI bị "thổi phồng".

Các hệ số tương quan thường được dùng để kiểm định xem hai biến có liên hệ (tuyến tính) với nhau hay không. Vì vậy, một phương pháp khác là định nghĩa mỗi phần tử của vectơ từ là kết quả của một phép kiểm định. Ví dụ, phần tử (a, b) nhận giá trị 1 nếu hệ số tương quan giữa hai từ a và b lớn hơn một ngưỡng, hoặc nếu mối tương quan giữa a và b được kiểm định giả thuyết ủng hộ một cách đủ mạnh.

Tuy nhiên, việc mô hình hóa từ bằng các vectơ gồm điểm tương quan giữa các từ phần nào giới hạn phạm vi thông tin ngữ cảnh mà ta có thể sử dụng. Một cách tiếp cận khác ở mức vĩ mô là phân tích xem các từ phân bố như thế nào trên toàn bộ các văn bản, tạo thành một **ma trận đồng xuất hiện từ–văn bản** (*word-document co-occurrence matrix*) hay **ma trận đồng xuất hiện thuật ngữ–văn bản** (*term-document co-occurrence matrix*). Ví dụ, với các văn bản đã nêu ở trên, ta có thể xây dựng ma trận

	Văn bản 1	Văn bản 2	Văn bản 3	Văn bản 4	Văn bản 5
berry	1	1	0	0	1
terminology	0	1	0	0	0
common	0	0	1	0	0
teeth	0	0	0	1	0
banana	0	0	1	1	1
simple	0	1	0	0	0
and	1	1	0	2	0

Trong ma trận này, giá trị của phần tử (a, d) được định nghĩa là số lần từ a xuất hiện trong văn bản d, thể hiện độ mạnh liên kết giữa a và d. Giá trị này thường được gọi là **tần suất từ** (*term frequency*) (**TF**) của a trong d (ký hiệu là $\text{tf}(a, d)$). Ngoài ra, ta cũng có thể dùng một hàm chỉ thị 0–1 để đánh dấu sự có mặt của từ (xem Mục 3.2). Xem Bảng 3.1 để biết một vài biến thể của hàm trọng số TF.

Cấu trúc này phục vụ hai mục đích cùng lúc: mỗi hàng cung cấp một biểu diễn vectơ của một từ cụ thể, còn mỗi cột đóng vai trò là biểu diễn vectơ của cả một văn bản. Ma trận đồng xuất hiện từ–văn bản

[15] Một vấn đề của PMI là thước đo này trở nên không ổn định khi các từ hiếm. Ví dụ, nếu một từ rất hiếm tình cờ xuất hiện trong một văn bản, thì giá trị PMI giữa từ này và bất kỳ từ nào khác trong văn bản đó sẽ lớn một cách vô lý.

Bảng 3.1: Các hàm của sơ đồ trọng số theo tần suất từ. $\text{count}(a; d)$ đếm số lần xuất hiện của từ a trong văn bản d.

Phần tử	Dạng toán học
Nhị phân	$\text{tf}(a, d) = \begin{cases} 1 & a \text{ xuất hiện trong } d \\ 0 & \text{ngược lại} \end{cases}$
Đếm	$\text{tf}(a, d) = \text{count}(a; d)$
Đếm lũy thừa	$\text{tf}(a, d) = \text{count}(a; d)^{\alpha}$
Đếm theo thang log	$\text{tf}(a, d) = \log(1 + \text{count}(a; d))$
Đếm chuẩn hóa (hay tần suất)	$\text{tf}(a, d) = \frac{\text{count}(a;d)}{\sum_{a'} \text{count}(a';d)}$

này về mặt khái niệm giống hệt mô hình túi từ đã giới thiệu cho bài toán phân loại văn bản ở Chương 1, trong đó thứ tự từ bị bỏ đi nhưng tần suất tương đối được giữ lại. Ở đây, ta thực hiện việc vectơ hóa văn bản bằng mô hình này trên một tập hợp văn bản.

3.3.2 TF-IDF

Mô hình hóa liên kết giữa từ và văn bản là việc quan trọng đối với nhiều bài toán NLP. Một cải tiến đáng kể so với việc dùng số đếm tần suất thô để xây dựng các vectơ này là sơ đồ trọng số **tần suất từ – nghịch đảo tần suất văn bản** (*term frequency-inverse document frequency*) (**TF-IDF**). Cho một tập văn bản D, TF-IDF gán một điểm số cho mỗi cặp từ–văn bản (a, d) theo phương trình sau:

$$\text{tfidf}(a, d, D) \;=\; \text{tf}(a, d) \cdot \text{idf}(a, D), \tag{3.23}$$

trong đó

- $\text{tf}(a, d)$ là tần suất từ (xem Bảng 3.1). Khi $\text{tf}(a, d)$ lớn, từ a là một dấu hiệu tốt cho văn bản d. Ngược lại, khi $\text{tf}(a, d)$ nhỏ, liên kết giữa từ và văn bản không mạnh lắm.
- $\text{idf}(a, D)$ là **nghịch đảo tần suất văn bản** (*inverse document frequency*) (**IDF**). Nó được xây dựng dựa trên thực tế là những từ phổ biến trên nhiều văn bản thì mang ít thông tin hơn. Ví dụ, với một tập văn bản về thể thao, rất có thể ta sẽ thấy *player* (cầu thủ) và *players* (các cầu thủ) trong hầu hết các văn bản. Trong trường hợp này, các từ *player* và *players* ít có ích trong việc phân biệt các văn bản hay ngữ cảnh khác nhau. Gọi $\text{df}(a, D)$ là số văn bản trong D có chứa từ a. Một dạng phổ biến của $\text{idf}(a, D)$ là

  $$\text{idf}(a, D) \;=\; \log \frac{|D|}{\text{df}(a, D)}. \tag{3.24}$$

  Phương trình này "phạt" những từ xuất hiện thường xuyên trên toàn kho ngữ liệu. Tương tự, ta có thể có một phiên bản làm trơn của $\text{idf}(a, D)$

  $$\text{idf}(a, D) \;=\; \log \frac{|D|}{\text{df}(a, D) + 1} + 1. \tag{3.25}$$

Sử dụng sơ đồ trọng số TF-IDF, ta có thể xây dựng một ma trận từ–văn bản trong đó giá trị của phần tử (a, d) là $\text{tfidf}(a, d, D)$. Như đã mô tả ở trên, mỗi hàng của ma trận này đóng vai trò là biểu diễn vectơ của từ tương ứng. Theo truyền thống, TF-IDF và ma trận thuật ngữ–văn bản được dùng rộng rãi để biểu diễn văn bản. Chẳng hạn, trong truy xuất thông tin (*information retrieval*), cả truy vấn lẫn văn

bản đều có thể được biểu diễn thành các vectơ cột TF-IDF, cho phép ta tính độ liên quan giữa truy vấn và văn bản thông qua độ tương tự vectơ. Mặc dù các mô hình không gian vectơ trong truy xuất thông tin nằm ngoài phạm vi của chương này, bạn đọc có thể tham khảo các giáo trình chuẩn để có cái nhìn đầy đủ [Manning và cộng sự, 2008; Buttcher và cộng sự, 2016].

Chú giải của người dịch: Tính TF-IDF bằng tay

Dùng 5 văn bản ở trên, $|D| = 5$. Từ *banana* xuất hiện trong 3 văn bản (Văn bản 3, 4, 5), nên theo Phương trình (3.24), $\text{idf} = \log(5/3) \approx 0.51$ (log tự nhiên). Từ *teeth* chỉ xuất hiện trong 1 văn bản, nên $\text{idf} = \log(5/1) \approx 1.61$. Trong Văn bản 4, cả hai từ đều xuất hiện 1 lần ($\text{tf} = 1$), nhưng $\text{tfidf}(\textit{teeth}) \approx 1.61$ lớn hơn $\text{tfidf}(\textit{banana}) \approx 0.51$: *teeth* "đặc trưng" cho Văn bản 4 hơn. Một từ có trong *mọi* văn bản (như "là", "của") sẽ có $\text{idf} = \log 1 = 0$, tức bị triệt tiêu hoàn toàn. Đó là lý do IDF được gọi là "phạt" các từ phổ biến.

3.3.3 Các mô hình ít chiều

Các ma trận đồng xuất hiện thường có số chiều rất lớn. Ví dụ, giả sử có một bộ từ vựng gồm $20{,}000$ từ khác nhau và một tập hợp $10{,}000{,}000$ văn bản. Khi đó, ma trận đồng xuất hiện từ–văn bản có $20{,}000 \times 10{,}000{,}000 = 2 \times 10^{11}$ phần tử. Với gánh nặng tính toán của một mô hình như vậy, việc biểu diễn mỗi từ bằng một vectơ $10{,}000{,}000$ chiều và mỗi văn bản bằng một vectơ $20{,}000$ chiều là không thực tế. Thay vào đó, ta mong muốn biểu diễn của một từ (hay một văn bản) chỉ cần một số lượng đặc trưng nhỏ vừa phải. Trong tiểu mục này, chúng ta bàn về các cách tiếp cận chuẩn để biến các ma trận đồng xuất hiện nhiều chiều thành các biểu diễn ít chiều hơn. Vì các kỹ thuật này đã được nghiên cứu sâu rộng và áp dụng thành công trong nhiều lĩnh vực [Barber, 2012; Wright và Ma, 2022], chúng tôi bỏ qua các chứng minh toán học bên dưới và tập trung vào việc áp dụng chúng trong thực tế để thu được các vectơ từ và vectơ văn bản.

1. Phân tích ngữ nghĩa tiềm ẩn

Trong NLP, **phân tích ngữ nghĩa tiềm ẩn** (*latent semantic analysis*) (**LSA**) là một phương pháp tìm kiếm cấu trúc ngữ nghĩa tiềm ẩn đằng sau các liên kết từ–văn bản [Deerwester và cộng sự, 1990; Landauer và cộng sự, 1998][16]. Nó giả định rằng cả từ lẫn văn bản đều có thể được biểu diễn bằng các vectơ ít chiều được chắt lọc từ ma trận đồng xuất hiện, trong khi vẫn giữ được tính chất của mô hình không gian vectơ ban đầu, chẳng hạn góc giữa các vectơ là nhỏ đối với các từ tương tự nhau.

Cụ thể hơn, LSA phân tích ma trận đồng xuất hiện thành tích của một ma trận biểu diễn từ, một ma trận biểu diễn văn bản, và một ma trận thứ ba nối hai ma trận đầu. Về mặt toán học, điều này có thể được đặt thành một quá trình **phân tích giá trị suy biến** (*singular value decomposition*) (**SVD**) [Stewart, 1993]. Gọi $\mathbf{M} \in \mathbb{R}^{|V| \times |D|}$ là một ma trận đồng xuất hiện trên bộ từ vựng V và tập văn bản D. SVD cho ta một phân tích của $\mathbf{M}$:

$$\mathbf{M} = \mathbf{P}\mathbf{\Sigma}\mathbf{Q}^\top, \tag{3.26}$$

trong đó $\mathbf{P} \in \mathbb{R}^{|V| \times r}$, $\mathbf{\Sigma} \in \mathbb{R}^{r \times r}$ và $\mathbf{Q}^\top \in \mathbb{R}^{r \times |D|}$. Trong phân tích này, các ma trận $\mathbf{P}$ và $\mathbf{Q}^\top$ là **nửa unita** (*semi-unitary*) (hay **nửa trực giao** (*semi-orthogonal*)), nghĩa là các cột của cả $\mathbf{P}$ và $\mathbf{Q}$ đều là các **vectơ trực chuẩn** (*orthonormal vectors*) với nhau (các vectơ đơn vị đôi một trực giao). Do đó, các cột của $\mathbf{P}$ và $\mathbf{Q}$ lần lượt tạo thành các cơ sở trực chuẩn cho không gian cột và không gian hàng của $\mathbf{M}$, trong đó r là hạng của $\mathbf{M}$. Điều này bảo đảm $\mathbf{M}$ được biểu diễn bằng số chiều tối thiểu cần thiết. Ma trận $\mathbf{\Sigma}$

[16] Phân tích ngữ nghĩa tiềm ẩn còn được gọi là **lập chỉ mục ngữ nghĩa tiềm ẩn** (*latent semantic indexing*) (**LSI**). Thuật ngữ này được dùng nhiều hơn trong truy xuất thông tin và các lĩnh vực liên quan.

là ma trận đường chéo:

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_r \end{bmatrix}. \tag{3.27}$$

Các phần tử trên đường chéo $\{\sigma_1, \cdots, \sigma_r\}$ đều là các số thực không âm, và được gọi là các **giá trị suy biến** (*singular values*) của **M**. Thông thường, $\{\sigma_1, \cdots, \sigma_r\}$ được sắp theo thứ tự giảm dần (tức là $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r$). Cách sắp xếp này giúp chuẩn hóa phép phân tích. Nếu ta viết **P** thành một dãy các vectơ cột (gọi là các **vectơ suy biến trái** (*left-singular vectors*))

$$\mathbf{P} = \begin{bmatrix} \mathbf{p}_1, \cdots, \mathbf{p}_r \end{bmatrix}, \tag{3.28}$$

và $\mathbf{Q}^\top$ thành một dãy các vectơ hàng (gọi là các **vectơ suy biến phải** (*right-singular vectors*))

$$\mathbf{Q}^\top = \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_r^\top \end{bmatrix}, \tag{3.29}$$

thì ta có thể viết **M** thành

$$\mathbf{M} = \sum_{i=1}^{r} \sigma_i \mathbf{p}_i \mathbf{q}_i^\top. \tag{3.30}$$

Chú giải của người dịch: SVD bằng hình ảnh đời thường

Một ma trận lớn có thể hình dung như một bức ảnh. Phương trình (3.30) nói rằng bức ảnh đó bằng tổng của r "lớp" đơn giản $\sigma_i \mathbf{p}_i \mathbf{q}_i^\top$; mỗi lớp là tích của một cột $\mathbf{p}_i$ với một hàng $\mathbf{q}_i^\top$ (một bảng "cửu chương" kiểu $p(a) \times q(b)$), được nhân với "độ quan trọng" σ_i. Ví dụ với $\mathbf{p} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$, $\mathbf{q}^\top = [3 \;\; 1]$ thì $\mathbf{pq}^\top = \begin{bmatrix} 3 & 1 \\ 6 & 2 \end{bmatrix}$. Vì các σ_i được xếp giảm dần, những lớp đầu chứa phần lớn "nội dung" của ma trận. *Hạng* r của ma trận có thể hiểu là số "lớp" độc lập tối thiểu cần để ghép lại đúng ma trận đó. Các vectơ *trực chuẩn* là các vectơ có độ dài 1 và đôi một vuông góc (tích vô hướng bằng 0), giống các trục Ox, Oy, Oz trong không gian.

Để thu được biểu diễn từ, ta diễn giải hàng thứ i của **P** là một vectơ đặc trưng r chiều $\mathbf{e}_i$ cho từ thứ i, ký hiệu a_i, trong bộ từ vựng. Nói cách khác, biểu diễn vectơ của a_i là

$$\mathbf{e}_i = \begin{bmatrix} p_1(i) & \cdots & p_r(i) \end{bmatrix}. \tag{3.31}$$

Tương tự, biểu diễn vectơ của một văn bản d_j có thể được viết là

$$\mathbf{h}_j = \begin{bmatrix} q_1(j) & \cdots & q_r(j) \end{bmatrix}. \tag{3.32}$$

Bằng cách này, ta có hai mô hình biểu diễn riêng biệt cho từ và văn bản: **P** đảm nhận biểu diễn từ, còn **Q** đảm nhận biểu diễn văn bản. Như vậy, ta có thể xem **M** là tích của các mô hình biểu diễn này:

$$\begin{aligned} \mathbf{M} &= \mathbf{P\Sigma Q}^\top \\ &= \underbrace{\begin{bmatrix} \mathbf{e}_1 \\ \vdots \\ \mathbf{e}_{|V|} \end{bmatrix}}_{\text{các từ}} \begin{bmatrix} \sigma_1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_r \end{bmatrix} \underbrace{\begin{bmatrix} \mathbf{h}_1^\top & \cdots & \mathbf{h}_{|D|}^\top \end{bmatrix}}_{\text{các văn bản}}. \end{aligned} \tag{3.33}$$

Trong thực tế, hạng r thường nhỏ hơn nhiều so với $|V|$ và $|D|$. Do đó, với mỗi từ (hoặc mỗi văn bản), ta có một biểu diễn mới với số chiều nhỏ hơn nhiều so với biểu diễn chứa trong ma trận đồng xuất hiện. Một cải tiến nữa là chỉ dùng r^* giá trị suy biến lớn nhất (tức là $\{\sigma_1, \cdots, \sigma_{r^*}\}$) và bỏ đi phần còn lại. Kết quả là ta chỉ giữ r^* vectơ suy biến trái đầu tiên trong $\mathbf{P}$ và r^* vectơ suy biến phải đầu tiên trong $\mathbf{Q}$. Ở đây $r^* < r$ là một siêu tham số xác định số vectơ trong $\mathbf{P}$ và $\mathbf{Q}$, tức là số đặc trưng dùng để mô tả một từ hay một văn bản. Bằng cách này, ta có một phân tích mới của $\mathbf{M}$:

$$\mathbf{M} \approx \sum_{i=1}^{r^*} \sigma_i \mathbf{p}_i \mathbf{q}_i^\top. \tag{3.34}$$

Vế phải của Phương trình (3.34) còn được gọi là một **xấp xỉ hạng thấp** (*low-rank approximation*) của $\mathbf{M}$. Bằng cách chọn r^*, ta có thể xấp xỉ $\mathbf{M}$ bằng một ma trận có hạng tùy ý nhỏ hơn r.

Có nhiều thuật toán để cài đặt SVD [Cline và Dhillon, 2014]. Thực tế, phần lớn các cài đặt SVD hiện đại đều hiệu quả và có khả năng mở rộng. Ta có thể dùng chúng như những bộ công cụ có sẵn trong các ứng dụng NLP.

Chú giải của người dịch: Vì sao giữ lại ít chiều vẫn tốt?

Giả sử $\sigma_1 = 50$, $\sigma_2 = 20$, $\sigma_3 = 0.5$, $\sigma_4 = 0.1$. Hai lớp đầu đóng góp gần như toàn bộ ma trận, nên chọn $r^* = 2$ thì sai số rất nhỏ, nhưng mỗi từ chỉ còn được mô tả bằng 2 số thay vì hàng triệu số. Hơn nữa, những lớp nhỏ thường là "nhiễu" (do việc dùng từ ngẫu nhiên), nên bỏ chúng đi đôi khi còn làm lộ rõ hơn quan hệ ngữ nghĩa "tiềm ẩn", ví dụ *xe hơi* và *ô tô* hiếm khi cùng nằm trong một văn bản nhưng lại có vectơ ít chiều rất gần nhau.

2. Phân tích thành phần chính

Trong phân tích dữ liệu, **phân tích thành phần chính** (*principal component analysis*) (**PCA**) là một kỹ thuật giảm chiều (*dimensionality reduction*) được dùng rộng rãi. Cho một tập điểm dữ liệu, PCA tìm một dãy các hướng trực giao với nhau trong không gian tọa độ sao cho phương sai của các điểm dữ liệu theo những hướng này là lớn nhất. Các hướng này thường được biểu diễn bằng các vectơ đơn vị, gọi là **tải thành phần chính** (*principal component loadings*) hay **hệ số thành phần chính** (*principal component coefficients*). Kết quả là chúng tạo thành một không gian tọa độ mới, và ta có thể ánh xạ các điểm dữ liệu đã cho vào đó bằng một phép biến đổi tuyến tính trực giao.

Xét một ma trận đồng xuất hiện từ–văn bản $\mathbf{M} \in \mathbb{R}^{|V| \times |D|}$, trong đó mỗi hàng là một vectơ từ (hay vectơ đặc trưng) $|D|$ chiều. PCA định nghĩa một ánh xạ tuyến tính từ $\mathbb{R}^{|D|}$ sang $\mathbb{R}^p$, nghĩa là ta biến mỗi vectơ từ $|D|$ chiều thành một vectơ từ p chiều. Ánh xạ này được cho bởi

$$\mathbf{N} = \mathbf{MC}, \tag{3.35}$$

trong đó $\mathbf{N} \in \mathbb{R}^{|V| \times p}$ biểu diễn ma trận các vectơ từ đã được ánh xạ trên bộ từ vựng V, và $\mathbf{C} \in \mathbb{R}^{|D| \times p}$ là ma trận của ánh xạ tuyến tính. Khi đó, ta có thể viết $\mathbf{C}$ thành một dãy các vectơ cột

$$\mathbf{C} = \begin{bmatrix} \mathbf{c}_1 & \cdots & \mathbf{c}_p \end{bmatrix}. \tag{3.36}$$

Mỗi vectơ cột $\mathbf{c}_i = \begin{bmatrix} c_i(1) \\ \vdots \\ c_i(|D|) \end{bmatrix}$ là một nhóm hệ số thành phần chính, biểu thị một hàm tuyến tính kết hợp các đặc trưng đầu vào thành một đặc trưng mới. Ví dụ, nếu ta xem $\mathbf{M}$ là giá trị của một loạt hàm đặc

trưng (chẳng hạn các vectơ cột $\{\mathbf{m}_1, \cdots, \mathbf{m}_{|D|}\}$), ta có thể ánh xạ $\mathbf{M}$ sang một không gian đặc trưng mới theo $\mathbf{c}_i$:

$$\begin{aligned}\mathbf{M}\mathbf{c}_i &= \begin{bmatrix}\mathbf{m}_1 & \cdots & \mathbf{m}_{|D|}\end{bmatrix} \begin{bmatrix} c_i(1) \\ \vdots \\ c_i(|D|)\end{bmatrix} \\ &= \sum_{k=1}^{|D|} c_i(k)\mathbf{m}_k. \end{aligned} \tag{3.37}$$

Tích $\mathbf{M}\mathbf{c}_i$ (tạo thành cột thứ i của $\mathbf{N}$) là một vectơ cột, trong đó mỗi phần tử biểu diễn giá trị đặc trưng mới của một từ trong V. Trong PCA, các vectơ $\{\mathbf{c}_1, \cdots, \mathbf{c}_p\}$ được tạo ra lần lượt để cực đại hóa phương sai của dữ liệu sau khi chiếu. Như vậy, với mỗi $i \in [1, p]$, các hệ số thành phần chính tối ưu được định nghĩa là:

$$\begin{aligned}\hat{\mathbf{c}}_i &= \arg\max_{\mathbf{c}_i} \text{Var}(\mathbf{M}\mathbf{c}_i) \\ &= \arg\max_{\mathbf{c}_i} \mathbf{c}_i^\top \mathbf{S}\mathbf{c}_i, \end{aligned} \tag{3.38}$$

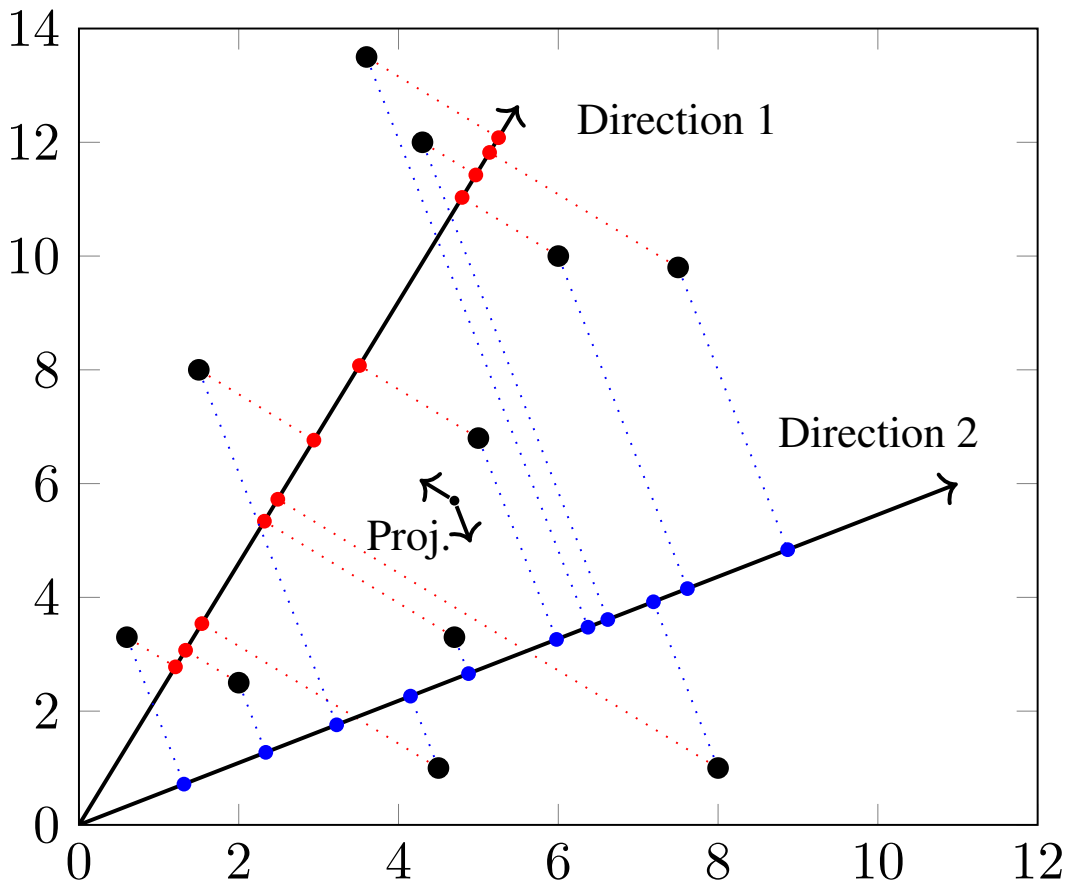
trong đó $\text{Var}(\mathbf{M}\mathbf{c}_i)$ là phương sai của dữ liệu đã chiếu, và $\mathbf{S}$ là ma trận hiệp phương sai (*covariance matrix*) kích thước $|D| \times |D|$ của $\mathbf{M}$. Để bảo đảm bài toán có nghiệm xác định, người ta thường ràng buộc $\mathbf{c}_i$ là một vectơ đơn vị (tức là $\mathbf{c}_i^\top \mathbf{c}_i = 1$). Dùng một nhân tử Lagrange (*Lagrange multiplier*) λ_i, bài toán tối ưu có ràng buộc này có thể được phát biểu bằng cách định nghĩa hàm Lagrange:

$$\mathcal{L}(\mathbf{c}_i, \lambda_i) \;=\; \mathbf{c}_i^\top \mathbf{S}\mathbf{c}_i - \lambda_i(\mathbf{c}_i^\top \mathbf{c}_i - 1). \tag{3.39}$$

Giải tìm các điểm dừng của hàm Lagrange này cho thấy $\hat{\mathbf{c}}_i$ phải là một vectơ riêng (*eigenvector*) của $\mathbf{S}$, với λ_i là giá trị riêng (*eigenvalue*) tương ứng [Jolliffe, 2002]. Vì $\mathbf{S}$ là một ma trận đối xứng kích thước $|D| \times |D|$, nó có đúng $|D|$ giá trị riêng cùng các vectơ riêng tương ứng đôi một trực giao. Bằng cách sắp xếp các vectơ riêng này theo giá trị riêng tương ứng giảm dần, ta chọn ra p vectơ riêng đầu tiên để tạo thành $\{\hat{\mathbf{c}}_1, \cdots, \hat{\mathbf{c}}_p\}$. Nói cách khác, $\hat{\mathbf{c}}_1$ là vectơ riêng ứng với giá trị riêng lớn nhất, $\hat{\mathbf{c}}_2$ ứng với giá trị riêng lớn thứ hai, và cứ thế tiếp tục. Phép chiếu $\mathbf{M}\hat{\mathbf{c}}_i$ được gọi là **thành phần chính** (*principal component*) thứ i của $\mathbf{M}$.

Một cách hình dung PCA về mặt hình học là xem nó như một phép đổi hệ tọa độ trong không gian Euclid. Với tập dữ liệu $\mathbf{M}$, mỗi hàng biểu diễn tọa độ của một điểm dữ liệu trong một không gian $|D|$ chiều A. PCA tìm cách biểu diễn các điểm dữ liệu này trong một hệ tọa độ mới p chiều B. Chiều thứ i của hệ mới này được xác định bởi hướng của vectơ đơn vị $\mathbf{c}_i$. Để tìm tọa độ thứ i của một điểm dữ liệu trong B, ta chiếu tọa độ ban đầu của nó lên đường thẳng sinh bởi $\mathbf{c}_i$. Hướng tối ưu $\mathbf{c}_i$ được chọn sao cho độ trải rộng (phương sai) của các điểm đã chiếu là lớn nhất. Về bản chất, ta đi tìm trục mà dọc theo nó các điểm dữ liệu phân tán nhiều nhất. Qua quá trình này, ta tạo ra một dãy các hệ số thành phần chính bằng cách lần lượt giải Phương trình (3.38). Hình 3.5 minh họa ý tưởng này bằng cách chiếu dữ liệu 2 chiều xuống một không gian con 1 chiều.

Trong các ứng dụng thực tế, p thường được chọn nhỏ hơn nhiều so với $|D|$, cho phép PCA giảm đáng kể số chiều của biểu diễn từ. Lưu ý rằng PCA là một phương pháp rất tổng quát và được thấy là hữu ích trong nhiều lĩnh vực. Rộng hơn, nếu $\mathbf{M}$ biểu diễn các quan sát trên một tập biến, PCA có thể cô đọng các quan sát này thành một tập nhỏ hơn các biến không tương quan với nhau (các thành phần chính).



Hình 3.5: Minh họa việc biến dữ liệu 2 chiều thành dữ liệu 1 chiều bằng PCA. Xét một tập điểm dữ liệu (biểu diễn bằng các chấm tròn đen) trên một mặt phẳng Euclid. Dùng PCA, ta tìm một hướng (biểu diễn bằng mũi tên) sao cho phương sai của dữ liệu đã chiếu (biểu diễn bằng các chấm tròn màu) theo hướng này là lớn nhất. Một hướng như vậy có thể được biểu diễn bằng một vectơ đơn vị gồm các hệ số thành phần chính. Trong ví dụ này, các hệ số thành phần chính mô tả một không gian tọa độ 1 chiều. Ta có thể ánh xạ dữ liệu từ không gian tọa độ 2 chiều sang không gian tọa độ 1 chiều bằng phép biến đổi tuyến tính. Dữ liệu sau ánh xạ được gọi là thành phần chính của các điểm dữ liệu ban đầu. (Trên hình: *Direction* = Hướng; *Proj.* = phép chiếu.)

Chú giải của người dịch: PCA bằng trực giác

Hãy nhìn Hình 3.5: các chấm đen nằm rải rác theo một dải nghiêng. Nếu buộc phải mô tả mỗi điểm chỉ bằng *một* con số, cách tốt nhất là chọn một trục (như "Direction 1") chạy dọc theo dải đó, rồi lấy hình chiếu vuông góc của mỗi điểm lên trục (giống hình chiếu vectơ ở lớp 10). Theo hướng đó, các hình chiếu trải ra rộng nhất nên giữ lại được nhiều thông tin phân biệt các điểm nhất; còn chiếu lên một hướng mà các hình chiếu dồn cục lại thì mất nhiều thông tin. *Phương sai* chính là thước đo "độ trải rộng" trong thống kê mô tả THPT. Về các ký hiệu: phương pháp *nhân tử Lagrange* là kỹ thuật tìm cực trị khi có ràng buộc (ở đây ràng buộc là $\mathbf{c}_i$ có độ dài 1, nếu không thì cứ kéo dài $\mathbf{c}_i$ là phương sai tăng vô hạn); còn *vectơ riêng* của ma trận $\mathbf{S}$ là vectơ $\mathbf{c}$ mà khi nhân với $\mathbf{S}$ chỉ bị co giãn chứ không đổi hướng: $\mathbf{Sc} = \lambda\mathbf{c}$, với hệ số co giãn λ là *giá trị riêng*.

3. Các phương pháp khác

Giảm chiều là một bài toán nền tảng trong học máy và đã được tổng quát hóa theo nhiều hướng. Chẳng hạn, các mô hình nhúng từ dùng mạng nơ-ron được mô tả trong Mục 3.4 và Mục 3.5 vốn đã học các vectơ từ ít chiều, giá trị thực từ văn bản. Dưới đây chúng tôi trình bày một số phương pháp giảm chiều mà bạn có thể gặp trong các tài liệu về NLP và học máy.

- **Mô hình chủ đề** (*topic models*). Về mặt kỹ thuật, mô hình chủ đề không phải là cách giảm chiều, mà là công cụ mô tả cách các văn bản và các từ được sinh ra dựa trên các phân phối theo chủ đề [Blei, 2012]. Ví dụ, **phân bổ Dirichlet tiềm ẩn** (*latent Dirichlet allocation*) (**LDA**) mô hình hóa quá trình sinh một văn bản bằng cách dùng các phân phối văn bản–chủ đề và chủ đề–từ [Blei và cộng sự, 2003]. Như một sản phẩm phụ, ta thu được một phân phối trên các từ cho mỗi chủ đề, cho biết khả năng một từ xuất hiện khi biết một chủ đề cụ thể. Nếu ta viết tất cả các phân phối chủ đề–từ này thành một ma trận $|V| \times K$, trong đó $|V|$ là số từ và K là số chủ đề, ta sẽ thu được các biểu diễn từ tương tự về mặt khái niệm với những biểu diễn đã mô tả ở các mục trước. Thông thường, K là một số tương đối nhỏ (ví dụ 200). Khi đó, ta có một mô hình ít chiều để biểu diễn từ. Mặc dù LDA không phổ biến lắm trong việc học biểu diễn từ cho các ứng dụng NLP, nó cung cấp một cách biểu diễn từ dưới dạng các phân phối trên những cấu trúc chủ đề tiềm ẩn.
- **Bộ tự mã hóa** (*auto-encoders*). Bộ tự mã hóa thiếu đủ (*undercomplete auto-encoder*) là một loại mô hình nơ-ron mã hóa các đặc trưng thành những mã ít chiều sao cho có thể khôi phục lại các đặc trưng đầu vào từ các mã đó. Một ưu điểm của bộ tự mã hóa là chúng không đặt giả định nào về cấu trúc ẩn của các đặc trưng. Vì vậy, bộ tự mã hóa học cách biến bất kỳ loại dữ liệu nào thành các biểu diễn ít chiều. Ví dụ, trong Chương 7 ta sẽ thấy các ví dụ áp dụng bộ tự mã hóa để học biểu diễn câu. Để biết thêm chi tiết về bộ tự mã hóa, bạn đọc có thể tham khảo Chương 2.
- **Giảm chiều có giám sát** (*supervised dimensionality reduction*). Các phương pháp giảm chiều truyền thống (như PCA) hoạt động theo cách không giám sát. Tuy nhiên, khi có sẵn nhãn của bài toán đích, việc tận dụng tín hiệu giám sát này sẽ có lợi. Một ví dụ kinh điển là giảm chiều có giám sát cho bài toán phân loại. Trong các phương pháp như **phân biệt tuyến tính Fisher** (*Fisher's linear discriminant*) và **phân tích phân biệt tuyến tính** (*linear discriminant analysis*), dữ liệu nhiều chiều được ánh xạ vào một không gian ít chiều hơn sao cho các lớp đã định trước được tách biệt nhau nhiều nhất. Ý tưởng này được tổng quát hóa cho các không gian đích nhiều chiều thông qua phân tích biến chính tắc (*Canonical Variates analysis*) [Barber, 2012].
- **Lựa chọn đặc trưng** (*feature selection*). Lựa chọn đặc trưng là quá trình xác định một tập con gồm những đặc trưng liên quan nhất để biểu diễn một đối tượng, qua đó giảm số chiều của biểu diễn một cách tường minh. Đây là một chủ đề rộng trong học máy với nhiều phương pháp [Guyon và Elisseeff, 2003; Liu và Motoda, 2012]. Cách đơn giản nhất là xem nó như một bài toán tìm

kiếm: ta tìm kiếm trong không gian các tập con đặc trưng sao cho các đặc trưng được chọn làm cực đại (hoặc cực tiểu) một mục tiêu nào đó. Nói chung, cách thiết kế mục tiêu phụ thuộc vào bài toán mà ta áp dụng các đặc trưng. Sự gắn kết chặt chẽ này khiến việc lựa chọn đặc trưng trở nên khó khăn, vì nó đòi hỏi cân bằng nhiều yếu tố, chẳng hạn thước đo hiệu năng của bài toán đích, hiệu quả tìm kiếm, và cách biểu diễn mỗi tập con đặc trưng. Lưu ý rằng lựa chọn đặc trưng thường được bàn đến trong học có giám sát, vốn cần dữ liệu có nhãn để tính hàm mất mát phục vụ tối ưu hóa. Bạn đọc có thể tham khảo bài tổng quan của Solorio-Fernández và cộng sự [2020] về các phương pháp lựa chọn đặc trưng không giám sát.

Trong thống kê, nhiều phương pháp có thể được xếp vào khuôn khổ giảm chiều và có liên hệ với những gì ta đã bàn trong mục này. Ví dụ, **phân tích nhân tố** (*factor analysis*) là một phương pháp tương tự PCA vì cả hai đều tìm một ánh xạ tuyến tính từ các biến đầu vào sang một số ít biến mới hơn. Điểm khác nhau là phân tích nhân tố tập trung vào mô hình hóa phần phương sai chung của các biến, còn PCA tập trung vào cực đại hóa phương sai của dữ liệu đã chiếu. Một ví dụ khác là **phân tích thành phần độc lập** (*independent component analysis*) (**ICA**). Khác với PCA, mục tiêu của ICA là tìm các thành phần độc lập có thể tách rời nhau theo phép cộng. Có thể tìm thêm các ví dụ khác trong các giáo trình học máy và thống kê [McClave và Sincich, 2006; Freedman và cộng sự, 2007; Barber, 2012].

3.4 Học nhúng từ từ các mô hình ngôn ngữ nơ-ron

Đếm số lần đồng xuất hiện từ–từ hay từ–văn bản là một phương pháp đơn giản, trực tiếp để biểu diễn từ dựa trên tính chất phân bố của chúng trong một kho ngữ liệu. Mặc dù phương pháp này hiệu quả trong nhiều ứng dụng, nó áp đặt một ràng buộc chặt lên biểu diễn từ: các phần tử của một vectơ từ phải diễn giải được như bằng chứng tường minh về việc từ đó phân bố như thế nào trong các ngữ cảnh khác nhau. Lý tưởng nhất, ta muốn biểu diễn từ ở dạng tổng quát hơn, cụ thể là các vectơ dày đặc, giá trị thực (thường được gọi là **vectơ nhúng từ** (*word embeddings*)), mà không áp đặt ràng buộc hay giả định định sẵn nào về ý nghĩa cụ thể của từng phần tử trong vectơ.

Học vectơ từ mà không có ràng buộc thì phải trả giá. Khác với các phương pháp dựa trên đếm trình bày ở Mục 3.3, ta không dựa vào các quy tắc kinh nghiệm hay tri thức thống kê có trước để ước lượng giá trị của vectơ. Thay vào đó, ta muốn quy nạp các biểu diễn có ý nghĩa trực tiếp từ dữ liệu. Một thách thức chính ở đây là không có "đáp án chuẩn" (*gold standard*), vì việc gán nhãn thủ công các biểu diễn liên tục, nhiều chiều cho cả một bộ từ vựng là không khả thi. Vì vậy, ta thường đặt việc quy nạp vectơ từ thành một mục tiêu phụ bên trong một bài toán được định nghĩa rõ ràng (thường gọi là **bài toán nền** (*background task*)). Các vectơ từ học được xuất hiện như một sản phẩm phụ giá trị của việc huấn luyện trên bài toán nền này.

Một ví dụ phổ biến là quy nạp vectơ từ từ các mô hình ngôn ngữ nơ-ron (*neural language models*) (NLM). Hãy nhớ lại NLM đã mô tả ở Chương 2. Mục tiêu của nó là dự đoán xác suất của một từ khi biết các từ đứng trước nó [Bengio và cộng sự, 2003a]. Một cách hình thức hơn, gọi w_i là từ ta muốn dự đoán, và $\{w_{i-n+1}, \cdots, w_{i-1}\}$ là các từ ngữ cảnh ta đã thấy. Đầu tiên, các từ ngữ cảnh $\{w_{i-n+1}, \cdots, w_{i-1}\}$ được chiếu thành các vectơ dày đặc d_e chiều $\{\mathbf{e}_{i-n+1}, \cdots, \mathbf{e}_{i-1}\}$ thông qua một tầng nhúng (*embedding layer*). Giả sử $\mathbf{1}_{w_j}$ là biểu diễn one-hot của từ j (một vectơ hàng kích thước $|V|$), vectơ từ dày đặc $\mathbf{e}_j$ được cho bởi

$$\mathbf{e}_j \;=\; \mathbf{1}_{w_j}\mathbf{C}, \tag{3.40}$$

trong đó $\mathbf{C} \in \mathbb{R}^{|V|\times d_e}$ là tham số của tầng nhúng. $\mathbf{C}$ thường được gọi là bảng nhúng từ (*word embedding table*), trong đó hàng thứ k là biểu diễn của từ thứ k trong V.

Chú giải của người dịch: Nhân vectơ one-hot với bảng nhúng = tra bảng

Giả sử bộ từ vựng có 3 từ {*mèo*, *chó*, *cá*} và $d_e = 2$, với $\mathbf{C} = \begin{bmatrix} 0.1 & 0.9 \\ 0.2 & 0.8 \\ 0.7 & -0.3 \end{bmatrix}$. Từ *chó* có one-hot $\mathbf{1}_{\text{chó}} = [0\ \ 1\ \ 0]$. Khi đó $\mathbf{1}_{\text{chó}}\mathbf{C} = 0 \cdot [0.1\ \ 0.9] + 1 \cdot [0.2\ \ 0.8] + 0 \cdot [0.7\ \ -0.3] = [0.2\ \ 0.8]$, đúng bằng hàng thứ 2 của $\mathbf{C}$. Vậy Phương trình (3.40) thực chất chỉ là "lấy ra hàng ứng với từ đó"; các con số trong $\mathbf{C}$ ban đầu được khởi tạo ngẫu nhiên rồi được điều chỉnh dần trong quá trình huấn luyện.

Sau đó, ta dùng một mạng nơ-ron truyền thẳng để tính phân phối xác suất của từ tại vị trí i. Phân phối này được cho bởi

$$\Pr(\cdot|w_{i-n+1}, \cdots, w_{i-1}) = F_\theta(\mathbf{e}_{i-n+1}, \cdots, \mathbf{e}_{i-1}), \tag{3.41}$$

trong đó $F_\theta(\cdot)$ là một mạng nơ-ron truyền thẳng có tham số θ. Thông thường, tầng nhúng có thể được xem là một thành phần của NLM. Ở đây, ta dùng ký hiệu hơi khác một chút để nhấn mạnh rằng NLM là một hàm của cả θ và $\mathbf{C}$

$$\Pr_{\theta,\mathbf{C}}(\cdot|w_{i-n+1}, \cdots, w_{i-1}) = F_{\theta,\mathbf{C}}(w_{i-n+1}, \cdots, w_{i-1}). \tag{3.42}$$

Khi huấn luyện, ta tối ưu cả θ lẫn $\mathbf{C}$ để cực tiểu hóa một hàm mất mát. Một phương pháp phổ biến là huấn luyện theo hợp lý cực đại, tức là cực đại hóa tổng log-hợp lý trên tất cả các n-gram trong dữ liệu. Cho một chuỗi từ $w_1 \cdots w_m$, mục tiêu huấn luyện được định nghĩa là:[17]

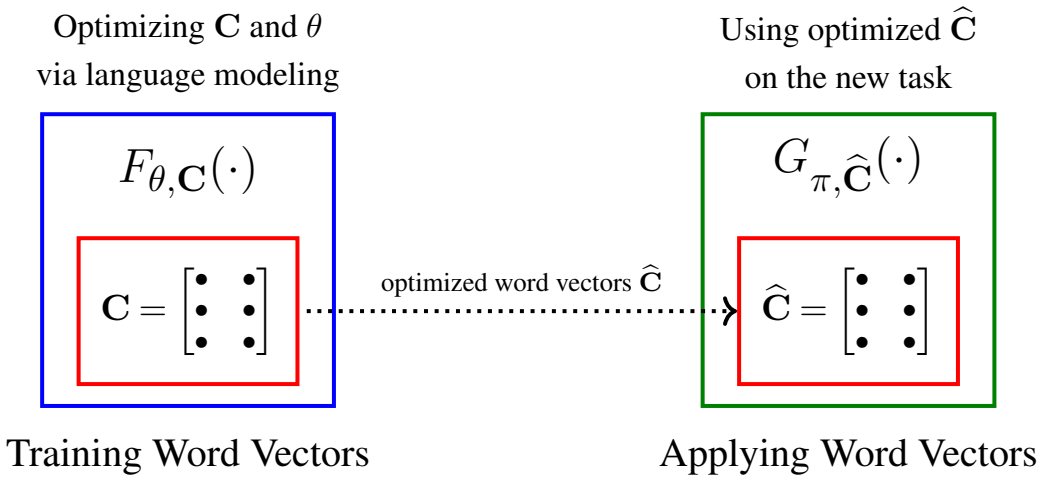
$$(\hat{\theta}, \widehat{\mathbf{C}}) = \arg\max_{\theta,\mathbf{C}} \sum_{i=n}^{m} \log \Pr_{\theta,\mathbf{C}}(w_i|w_{i-n+1}, \cdots, w_{i-1}). \tag{3.43}$$

Sau khi có các tham số tối ưu $\hat{\theta}$ và $\widehat{\mathbf{C}}$, ta có thể áp dụng $F_{\hat{\theta},\widehat{\mathbf{C}}}(\cdot)$ cho các n-gram chưa từng thấy. Quan trọng hơn, ta có được một số vectơ từ đã được huấn luyện tốt (tức là $\widehat{\mathbf{C}}$) có thể dùng trong các hệ thống khác ngoài NLM. Đây còn được gọi là tiền huấn luyện (*pre-training*) vectơ từ. Trong tiền huấn luyện, ta có thể định nghĩa $F_{\theta,\mathbf{C}}(\cdot)$ là bất kỳ hệ thống nào sử dụng các vectơ từ $\mathbf{C}$. Như vậy, bài toán học $\mathbf{C}$ được chuyển thành bài toán tối ưu $F_{\theta,\mathbf{C}}(\cdot)$ trên bài toán nền (xem minh họa ở Hình 3.6). Ưu điểm chính của phương pháp này là ta có thể tận dụng các bài toán NLP sẵn có để huấn luyện vectơ từ. Một rủi ro ở đây là các vectơ từ "tốt nhất" tìm được khi huấn luyện $F_{\theta,\mathbf{C}}(\cdot)$ có thể không thực sự phù hợp với hệ thống mà các vectơ từ được dùng trong thực tế. Thú vị là, trong nhiều trường hợp, các vectơ từ được tiền huấn luyện bằng NLM có chất lượng tốt cho các bài toán hạ nguồn (*downstream tasks*), hoặc ít nhất cũng là một điểm khởi đầu tốt để tiếp tục tinh chỉnh các vectơ từ này trong hệ thống đích.

Chú giải của người dịch: Ý tưởng "học ké"

Không ai cho máy biết vectơ "đúng" của từ *mèo* là gì. Nhưng ta có một bài toán có đáp án rõ ràng và dữ liệu gần như vô tận: đoán từ tiếp theo trong câu (mọi văn bản trên mạng đều là dữ liệu huấn luyện). Để đoán giỏi, mô hình buộc phải xếp các từ dùng giống nhau (*mèo*, *chó*) vào những vectơ gần nhau. Sau khi huấn luyện xong, ta "tháo" bảng $\widehat{\mathbf{C}}$ ra dùng cho bài toán khác, như học sinh mang vốn từ vựng từ việc đọc truyện sang làm bài văn. Đây là mầm mống của ý tưởng tiền huấn luyện, nền tảng của các mô hình ngôn ngữ lớn ở các chương sau.

[17] Điều này có thể được tổng quát hóa cho một tập dữ liệu gồm nhiều chuỗi.



Hình 3.6: Minh họa việc tiền huấn luyện vectơ từ trong một NLM. NLM có thể được ký hiệu là một hàm $F_{\theta,\mathbf{C}}(\cdot)$ của bảng nhúng từ (tức là $\mathbf{C}$) và các tham số khác của NLM (tức là θ). Việc tiền huấn luyện $\mathbf{C}$ về bản chất là quá trình huấn luyện $F_{\theta,\mathbf{C}}(\cdot)$ trên một bài toán nền. Kết quả là các vectơ từ tối ưu $\widehat{\mathbf{C}}$, sau đó được áp dụng vào một hệ thống mới $G_{\pi,\widehat{\mathbf{C}}}(\cdot)$ có thể khác với NLM. Trong hệ thống mới, $\widehat{\mathbf{C}}$ là bảng nhúng từ học được từ NLM và π biểu diễn các tham số dành riêng cho $G(\cdot)$. (Các nhãn trên hình: *Training Word Vectors* = Huấn luyện vectơ từ, tối ưu $\mathbf{C}$ và θ bằng mô hình hóa ngôn ngữ; *Applying Word Vectors* = Áp dụng vectơ từ, dùng $\widehat{\mathbf{C}}$ đã tối ưu cho bài toán mới; *optimized word vectors* = các vectơ từ đã tối ưu.)

3.5 Các mô hình nhúng từ

Về nguyên tắc, vectơ từ có thể được học theo bất kỳ cách nào. Coi vectơ từ là thành phần của các hệ thống NLP sẵn có là một lựa chọn, nhưng cách tiếp cận này thường thiếu những cân nhắc dành riêng cho bài toán. Một lựa chọn khác là phát triển các phương pháp được thiết kế riêng cho bài toán học biểu diễn. Vì việc huấn luyện các hệ thống như vậy không phải thỏa mãn những ràng buộc chặt chẽ của các bài toán NLP hạ nguồn tiêu chuẩn, việc quy nạp các vectơ từ chất lượng cao trở nên dễ xử lý hơn đáng kể.

3.5.1 Word2Vec

Word2Vec là tên gọi của một bộ các mô hình được đề xuất trong [Mikolov và cộng sự, 2013a;c]. Giống như các mô hình ngôn ngữ nơ-ron, các mô hình Word2Vec được xây dựng dựa trên mạng nơ-ron. Tuy nhiên, thay vì dùng cách mô hình hóa sinh tự hồi quy nặng nề trên các n-gram, Word2Vec đặt việc học vectơ từ theo dạng log-tuyến tính (*log-linear*) hiệu quả về mặt tính toán. Do đó, kiến trúc của các mô hình này khác biệt rõ rệt so với kiến trúc dùng trong mô hình hóa ngôn ngữ truyền thống. Họ Word2Vec có hai kiến trúc chính:

- **Mô hình túi từ liên tục** (*continuous bag-of-words model*) (hay **mô hình CBOW**). Mô hình CBOW là một mô hình dự đoán từ. Nó được dùng để dự đoán khả năng một từ xuất hiện tại vị trí i khi biết các cửa sổ từ $-n$ và $+n$ xung quanh nó. Cấu trúc của mô hình CBOW tương tự mô hình ngôn ngữ nơ-ron đã giới thiệu ở Chương 2 (xem Hình 3.7(a)). Đầu tiên, ta dùng một tầng nhúng để biến các từ ngữ cảnh $w_{i-n} \cdots w_{i-1}$ và $w_{i+1} \cdots w_{i+n}$ thành các vectơ từ tương ứng.

Việc này được thực hiện bằng cách nhân biểu diễn one-hot của mỗi từ đầu vào w_j với bảng nhúng $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$, như trong Phương trình (3.40). Các vectơ từ này sau đó được lấy trung bình để tạo ra một biểu diễn duy nhất cho các từ đầu vào, cho ta

$$\mathbf{h} \;=\; \frac{1}{2n}\left(\sum_{j=i-n}^{i-1} \mathbf{1}_{w_j}\mathbf{C} + \sum_{j=i+1}^{i+n} \mathbf{1}_{w_j}\mathbf{C}\right). \tag{3.44}$$

Lưu ý rằng công thức trên định nghĩa một mô hình hoàn toàn bỏ qua thứ tự của các từ đầu vào do dùng phép tổng. Điều này giải thích vì sao mô hình CBOW được gọi là *túi từ*. Tầng đầu ra của mô hình CBOW là một tầng Softmax tiêu chuẩn chiếu $\mathbf{h}$ thành một phân phối xác suất trên bộ từ vựng

$$\mathbf{y} \;=\; \text{Softmax}(\mathbf{h}\mathbf{U} + \mathbf{b}), \tag{3.45}$$

trong đó $\mathbf{U} \in \mathbb{R}^{d_e \times |V|}$ là ma trận tham số của ánh xạ tuyến tính và $\mathbf{b} \in \mathbb{R}^{|V|}$ là số hạng độ lệch. $\mathbf{y}$ là một phân phối trên bộ từ vựng, và $\Pr(w_i|w_{i-n}, \cdots, w_{i-1}, w_{i+1}, \cdots, w_{i+n}) = y(w_i)$. Phương trình (3.44–3.45) mô tả một mạng nơ-ron rất đơn giản. Một ưu điểm là mô hình thu được nhỏ gọn và hiệu quả so với các NLM. Việc huấn luyện mô hình CBOW rất đơn giản. Ta có thể đặt nó thành bài toán tìm ước lượng hợp lý cực đại cho các tham số của mô hình. Để đơn giản, gọi θ là các tham số khác ngoài $\mathbf{C}$ (tức là $\theta = \{\mathbf{U}, \mathbf{b}\}$). Ta có

$$(\hat{\theta}, \widehat{\mathbf{C}}) \;=\; \arg\max_{\theta,\mathbf{C}} \sum_{i=n+1}^{m-n} \log \Pr_{\theta,\mathbf{C}}(w_i|w_{i-n}, \cdots, w_{i-1}, w_{i+1}, \cdots, w_{i+n}), \tag{3.46}$$

trong đó m là độ dài của chuỗi từ. Sau khi huấn luyện, ta chỉ cần bỏ $\hat{\theta}$ đi và dùng $\widehat{\mathbf{C}}$ làm bảng tra cứu vectơ từ.

- **Mô hình skip-gram liên tục** (*continuous skip-gram model*) (hay **mô hình skip-gram**). Mô hình skip-gram là một mô hình dự đoán từ khác. Nó mô hình hóa bài toán ngược với bài toán mô tả trong Phương trình (3.44–3.45). Chính xác hơn, mục tiêu của ta là dự đoán từng từ trong $\pm n$ từ ngữ cảnh khi biết w_i. Điều này thường được đặt thành bài toán ước lượng xác suất w_j xuất hiện khi biết w_i ($i - n \leq j \leq i - 1$ hoặc $i + 1 \leq j \leq i + n$). Hình 3.7(b) cho thấy cấu trúc của mô hình skip-gram. Tầng nhúng xử lý w_i như thường lệ. Biểu diễn của w_i được cho bởi

$$\mathbf{h} \;=\; \mathbf{1}_{w_i}\mathbf{C}. \tag{3.47}$$

Sau đó nó được đưa vào một tầng Softmax để dự đoán xác suất của mỗi từ ngữ cảnh w_j (giả sử $j = i + k$)[18]

$$\mathbf{y}_k \;=\; \text{Softmax}(\mathbf{h}\mathbf{V}_k + \mathbf{b}_k), \tag{3.48}$$

trong đó $\mathbf{V}_k$ và $\mathbf{b}_k$ là các tham số của mô hình ($-n \leq k \leq -1$ và $1 \leq k \leq n$). Ta có

$$\begin{aligned}\Pr(w_j|w_i) &= \Pr(w_{i+k}|w_i) \\ &= y_k(w_{i+k}).\end{aligned} \tag{3.49}$$

Gọi θ là cách viết gọn của $\{\mathbf{V}_k\}$ và $\{\mathbf{b}_k\}$. Bài toán huấn luyện có thể được định nghĩa là:

$$(\hat{\theta}, \widehat{\mathbf{C}}) \;=\; \arg\max_{\theta,\mathbf{C}} \sum_{i=n+1}^{m-n} \sum_{\substack{-n\leq k\leq -1,\\ 1\leq k\leq n}} \log \Pr_{\theta,\mathbf{C}}(w_{i+k}|w_i). \tag{3.50}$$

[18] Khi $k > 0$, w_j là một từ trong cửa sổ ngữ cảnh bên phải của w_i; khi $k < 0$, w_j là một từ trong cửa sổ ngữ cảnh bên trái.

Chú giải của người dịch: CBOW và skip-gram: hai trò chơi điền từ

Xét câu "*con mèo đang ngủ trên ghế*" với $n = 2$ và từ trung tâm $w_i = ngủ$. **CBOW** chơi trò "điền vào chỗ trống": nhìn *mèo, đang, trên, ghế* rồi đoán từ ở giữa là *ngủ*; vectơ ngữ cảnh **h** là *trung bình* của 4 vectơ từ (cộng lại rồi chia cho $2n = 4$). **Skip-gram** chơi trò ngược lại: biết *ngủ*, đoán từng từ xung quanh *mèo*, *đang*, *trên*, *ghế*, nên mỗi vị trí $k \in \{-2, -1, 1, 2\}$ có một bộ tham số $\mathbf{V}_k, \mathbf{b}_k$ riêng. Hàm Softmax (xem Chương 2) biến một dãy điểm số bất kỳ thành các xác suất dương có tổng bằng 1. Tổng $\sum \log \Pr$ trong các Phương trình (3.46) và (3.50) là log của tích các xác suất, nên cực đại hóa nó nghĩa là làm cho mô hình "đoán trúng" dữ liệu thật nhiều nhất có thể.

Cả hai mô hình trên đều tương tự như bài kiểm tra điền khuyết (*cloze test*), khi chỉ xét quan hệ phụ thuộc theo từng cặp giữa các từ. Vì chúng bỏ qua thứ tự từ và các quan hệ cú pháp phức tạp, các phân phối dự đoán của chúng kém chính xác hơn so với các mô hình ngôn ngữ. Tuy nhiên, cần lưu ý rằng mục tiêu của các mô hình này không phải là dự đoán chính xác các từ bị thiếu khi biết ngữ cảnh, mà là học biểu diễn từ từ một bài toán nào đó nắm bắt được quan hệ giữa các từ. Vì vậy, hiệu năng dự đoán từ của mô hình học được không quá quan trọng.

Một ưu điểm khác của các mô hình này là chúng có kiến trúc rất đơn giản, dễ huấn luyện. Ví dụ, trong cả hai mô hình đều không có tầng ẩn, và tầng nhúng được nối trực tiếp với tầng đầu ra. Các cấu trúc mô hình này có thể được xem là những trường hợp của mô hình hóa log-tuyến tính trong học máy: các biến đầu vào được biến đổi tuyến tính thành một vectơ đặc trưng dày đặc (ví dụ Phương trình (3.44)), theo sau ngay là một hàm phân loại log-tuyến tính (ví dụ Phương trình (3.45)).

3.5.2 GloVe

Vectơ toàn cục (*Global vectors*), thường được gọi là **GloVe**, là các vectơ nhúng từ được học bằng cách kết hợp ưu điểm của phương pháp phân tích ma trận toàn cục và phương pháp cửa sổ ngữ cảnh cục bộ [Pennington và cộng sự, 2014]. Phương pháp GloVe bắt đầu bằng việc xây dựng một ma trận đồng xuất hiện từ–từ toàn cục (như đã bàn ở Mục 3.3), rồi sau đó xây dựng một mô hình log-song tuyến tính dựa trên một tập giả định lý thuyết cụ thể về nghĩa của từ.

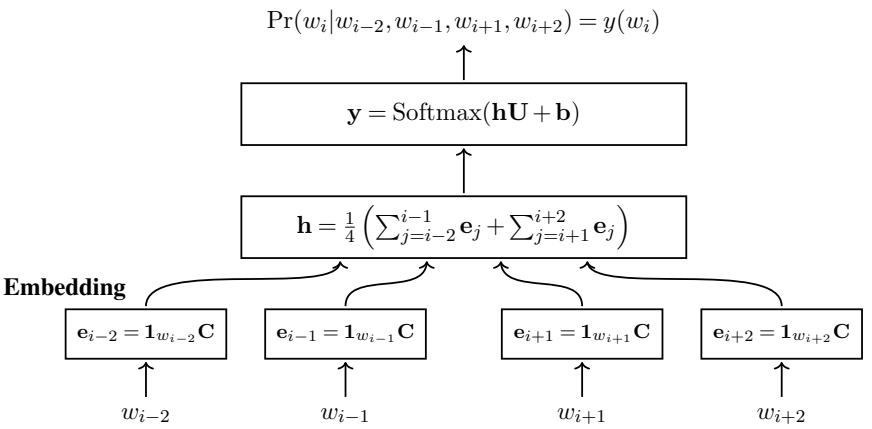
Cho một ma trận đồng xuất hiện từ–từ **M**, trong đó mỗi ô $M(a, b) = \text{count}(a, b)$ biểu diễn số lần từ $a \in V$ đồng xuất hiện với từ $b \in V$ trong một cửa sổ ngữ cảnh xác định, ta có thể tính xác suất có điều kiện $\Pr(b|a)$ như sau:

$$\begin{aligned} \Pr(b|a) &= \frac{\text{count}(a, b)}{\sum_{b'} \text{count}(a, b')} \\ &= \frac{\text{count}(a, b)}{\text{count}(a)}, \end{aligned} \tag{3.51}$$

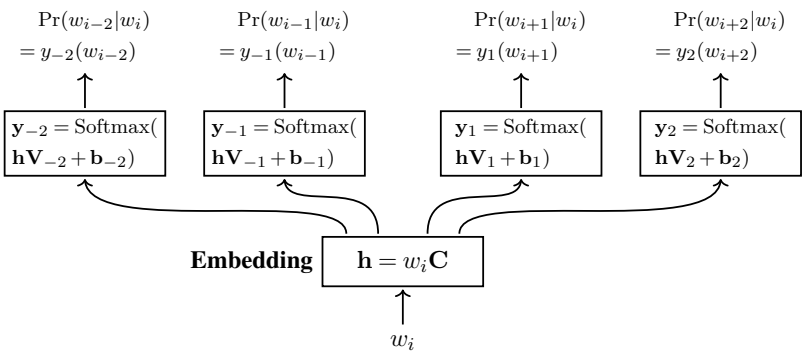
trong đó $\text{count}(a)$ biểu diễn tổng số lần từ a xuất hiện trong kho ngữ liệu.

Để hiểu động cơ đằng sau GloVe, hãy xét một ví dụ trong đó ta muốn phân biệt hai khái niệm nhiệt động học *air* (không khí) và *water* (nước). Dùng Phương trình (3.51), ta có thể tính xác suất các từ đích này xuất hiện khi biết các từ ngữ cảnh "thăm dò" khác nhau. Bảng sau hiển thị một phần nhỏ của ma trận $\Pr(b|a)$ được tính từ một kho ngữ liệu 3.8 tỷ từ trong WMT14 (*fly* = bay, *drink* = uống, *breath* = hơi thở, *live* = sống, *flow* = chảy):

Phần tử	$w = fly$	$w = drink$	$w = breath$	$w = live$	$w = flow$
$\Pr(air\|w)$	1.5×10^{-4}	6.2×10^{-5}	2.2×10^{-4}	1.6×10^{-4}	3.6×10^{-4}
$\Pr(water\|w)$	1.3×10^{-5}	4.1×10^{-4}	1.8×10^{-5}	1.4×10^{-4}	3.0×10^{-4}
$\Pr(air\|w)/\Pr(water\|w)$	11.54	0.15	12.2	1.14	1.2



(a) CBOW



(b) Skip-gram

Hình 3.7: Kiến trúc CBOW và skip-gram. Mô hình CBOW tính xác suất $\Pr(w_i|w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2})$, trong đó w_i là một từ trong chuỗi và $\{w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2}\}$ là các từ trong các cửa sổ ngữ cảnh ± 2. Biểu diễn ngữ cảnh **h** là trung bình của các vectơ từ được tạo ra qua một tầng nhúng. Sau đó **h** được đưa vào một tầng Softmax để cho ra một phân phối trên bộ từ vựng (tức là **y**). Xác suất dự đoán của w_i là $\Pr(w_i|w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2}) = y(w_i)$. Mô hình skip-gram cũng dựa trên cấu trúc nhúng + softmax. Nó mô hình hóa xác suất của mỗi từ ngữ cảnh w_j khi biết từ w_i. Điều này đạt được bằng cách đơn giản là tính đầu ra của một tầng softmax tiêu chuẩn nhận biểu diễn vectơ của w_i làm đầu vào. Cả hai mô hình CBOW và skip-gram đều được huấn luyện theo cách hợp lý cực đại. Bảng tra cứu thu được của tầng nhúng chính là các vectơ từ (hay vectơ nhúng) của các từ trong bộ từ vựng. (Trên hình: *Embedding* = tầng nhúng.)

Trong bảng này, $\Pr(\textit{air}|w)$ và $\Pr(\textit{water}|w)$ cho biết *air* và *water* tương quan mạnh đến mức nào với các từ thăm dò w khác nhau. Nhận xét then chốt xuất hiện khi ta xem xét tỉ số xác suất $\Pr(\textit{air}|w)/\Pr(\textit{water}|w)$ ở hàng cuối. Tỉ số này tách riêng một cách hiệu quả sự khác biệt ngữ nghĩa giữa hai từ đích. Khi w rất liên quan đến *air* nhưng không liên quan đến *water* (ví dụ $w =$ *fly* hay $w =$ *breath*), tỉ số này lớn. Ngược lại, khi w liên quan đến *water* nhưng không liên quan đến *air* (ví dụ $w =$ *drink*), tỉ số này nhỏ. Khi w liên quan đến cả hai (ví dụ $w =$ *live* hay $w =$ *flow*), hoặc không liên quan đến cả hai, tỉ số gần bằng 1.

Tiền đề nền tảng của GloVe là các vectơ từ nên mã hóa một cách tường minh tỉ số xác suất đồng xuất hiện này. Một điểm xuất phát tự nhiên là giả định có một hàm F nhận các vectơ từ làm đầu vào và xấp xỉ tỉ số này:

$$F(\mathbf{e}_a, \mathbf{e}_b, \tilde{\mathbf{e}}_w) \;=\; \frac{\Pr(a|w)}{\Pr(b|w)}, \tag{3.52}$$

trong đó $\mathbf{e}_a, \mathbf{e}_b \in \mathbb{R}^{d_e}$ là các biểu diễn vectơ đích của các từ a và b, còn $\tilde{\mathbf{e}}_w \in \mathbb{R}^{d_e}$ là biểu diễn vectơ ngữ cảnh của từ thăm dò w. Hãy chú ý sự phân biệt giữa $\mathbf{e}$ và $\tilde{\mathbf{e}}$: cái trước được lấy từ một bảng nhúng đích $\mathbf{C}$, còn cái sau được lấy từ một bảng nhúng ngữ cảnh riêng $\widetilde{\mathbf{C}}$. Việc dùng hai bảng nhúng khác nhau giúp giảm quá khớp trong quá trình huấn luyện. Cuối cùng, biểu diễn từ sau cùng thường được lấy là tổng (hoặc trung bình) của hai vectơ: $\mathbf{e}_{a,\text{final}} = \mathbf{1}_a(\mathbf{C} + \widetilde{\mathbf{C}})$.

Vì các không gian vectơ vốn có tính tuyến tính, cách tự nhiên nhất để mã hóa quan hệ giữa hai từ đích a và b là dùng phép trừ vectơ: $\mathbf{e}_a - \mathbf{e}_b$. Hơn nữa, để định lượng quan hệ giữa hiệu này và từ ngữ cảnh w, ta có thể dùng tích vô hướng của vectơ. Điều này giới hạn hàm F chỉ nhận một đầu vào vô hướng duy nhất:

$$F\Big((\mathbf{e}_a - \mathbf{e}_b)\tilde{\mathbf{e}}_w^\top\Big) \;=\; \frac{\Pr(a|w)}{\Pr(b|w)}. \tag{3.53}$$

Để áp đặt một cấu trúc đại số chặt chẽ, ta yêu cầu F là một đồng cấu (*homomorphism*) giữa nhóm cộng trong không gian vectơ và nhóm nhân trong không gian xác suất. Điều này bảo đảm phép trừ vectơ được ánh xạ trực tiếp thành phép chia xác suất:

$$F\Big((\mathbf{e}_a - \mathbf{e}_b)\tilde{\mathbf{e}}_w^\top\Big) \;=\; \frac{F(\mathbf{e}_a\tilde{\mathbf{e}}_w^\top)}{F(\mathbf{e}_b\tilde{\mathbf{e}}_w^\top)}. \tag{3.54}$$

Nghiệm liên tục duy nhất của Phương trình (3.54) là hàm mũ, $F(x) = \exp(x)$. Do đó, ta có:

$$\begin{aligned} F(\mathbf{e}_a\tilde{\mathbf{e}}_w^\top) &= \exp(\mathbf{e}_a\tilde{\mathbf{e}}_w^\top) \\ &= \Pr(a|w) \\ &= \frac{\text{count}(a,w)}{\text{count}(a)}. \end{aligned} \tag{3.55}$$

Lấy logarit hai vế, ta được:

$$\mathbf{e}_a\tilde{\mathbf{e}}_w^\top \;=\; \log\text{count}(a,w) - \log\text{count}(a). \tag{3.56}$$

Chú giải của người dịch: Vì sao lại là hàm mũ?

Phương trình (3.54) đòi hỏi $F(x-y) = F(x)/F(y)$: hiệu ở đầu vào phải thành thương ở đầu ra. Ở THPT bạn đã biết hàm mũ có đúng tính chất này: $e^{x-y} = e^x/e^y$. Ví dụ $e^{5-2} = e^3$ và $e^5/e^2 = e^3$. Lấy logarit là phép ngược của hàm mũ, nên từ $\exp(\mathbf{e}_a\tilde{\mathbf{e}}_w^\top) = \frac{\text{count}(a,w)}{\text{count}(a)}$ ta suy ra Phương trình (3.56) nhờ quy tắc $\log\frac{x}{y} = \log x - \log y$. Nói nôm na: GloVe muốn *tích vô hướng* của hai vectơ từ bằng *log của số lần chúng đi cùng*

nhau (sau khi trừ đi các phần "độ phổ biến" riêng của từng từ).

Một vấn đề của Phương trình (3.56) là số hạng $\log \text{count}(a)$ phá vỡ tính đối xứng; việc hoán đổi từ đích a và từ ngữ cảnh w (tương ứng với việc chuyển vị ma trận đồng xuất hiện) lẽ ra không được làm thay đổi cấu trúc hình học bên dưới. Để khôi phục tính đối xứng này, số hạng $\log \text{count}(a)$ được hấp thụ vào hai số hạng độ lệch được học, β_a và $\tilde{\beta}_w$:

$$\mathbf{e}_a \tilde{\mathbf{e}}_w^\top + \beta_a + \tilde{\beta}_w \;=\; \log \text{count}(a, w). \tag{3.57}$$

Phương trình (3.57) thiết lập quan hệ lý tưởng mà ta muốn các vectơ nhúng học được thỏa mãn. Để áp đặt điều này, ta xây dựng một bài toán hồi quy, nhằm cực tiểu hóa bình phương hiệu giữa log đồng xuất hiện dự đoán và log đồng xuất hiện thực tế. Hàm mất mát bình phương cho một cặp từ (a, w) được định nghĩa là:

$$\mathcal{L}_{a,w} \;=\; \Big(\mathbf{e}_a \tilde{\mathbf{e}}_w^\top + \beta_a + \tilde{\beta}_w - \log \text{count}(a, w)\Big)^2. \tag{3.58}$$

Hàm chi phí tổng trên toàn bộ bộ từ vựng là tổng có trọng số của các hàm mất mát riêng lẻ này:

$$L_{\text{GloVe}} \;=\; \sum_{a,w \in V} f(\text{count}(a, w)) \cdot L_{a,w}, \tag{3.59}$$

trong đó $f(\text{count}(a, w))$ là một hàm trọng số được thiết kế để ngăn các cặp từ hiếm chi phối hàm mất mát, đồng thời giới hạn ảnh hưởng của các cặp từ quá phổ biến. Pennington và cộng sự [2014] đề xuất hàm trọng số sau:

$$f(x) \;=\; \begin{cases} \left(\dfrac{x}{x_{\max}}\right)^{\alpha} & \text{nếu } x < x_{\max} \\ 1 & \text{ngược lại} \end{cases}, \tag{3.60}$$

trong đó $x_{\max}$ và α là các siêu tham số (thường $x_{\max} = 100$ và $\alpha = 3/4$). Như vậy, $f(x)$ "phạt" các cặp từ hiếm bằng cách gán cho chúng trọng số nhỏ hơn, đồng thời bảo đảm trọng số của các cặp phổ biến không tăng vô hạn.

Chú giải của người dịch: Tính thử hàm trọng số

Với $x_{\max} = 100$, $\alpha = 3/4$: một cặp từ chỉ đi cùng nhau $x = 1$ lần có trọng số $f(1) = (1/100)^{0.75} \approx 0.03$; cặp xuất hiện $x = 16$ lần có $f(16) = (0.16)^{0.75} \approx 0.25$; cặp xuất hiện từ 100 lần trở lên có trọng số 1, dù là 100 hay 1 triệu lần. Nhờ vậy, các cặp ngẫu nhiên hiếm hoi (dễ là nhiễu) ít ảnh hưởng, còn các cặp cực phổ biến (như "của" đi với "là") cũng không lấn át tất cả. Lưu ý: Phương trình (3.59) chỉ tính trên các cặp có $\text{count}(a, w) > 0$ trong thực tế, vì $f(0) = 0$ và $\log 0$ không xác định.

Các Phương trình (3.58–3.59) mô tả một khuôn khổ rất hiệu quả để học vectơ từ bằng các phép toán mạng nơ-ron tiêu chuẩn (ví dụ tích vô hướng và phép cộng vô hướng). Một đặc điểm quyết định của GloVe là nó dựa vào một hàm mục tiêu log-song tuyến tính đơn giản. Vì việc huấn luyện tránh được hàm mất mát entropy chéo và phép chuẩn hóa softmax tốn kém về tính toán mà các NLM đầy đủ và Word2Vec đòi hỏi, GloVe huấn luyện nhanh đáng kể. Trực giác cốt lõi là các quan hệ ngữ nghĩa có thể được mô hình hóa bằng phép hồi quy đơn giản trên các thống kê toàn cục của kho ngữ liệu, thay vì chỉ dựa vào độ phân kỳ dựa trên xác suất trên các cửa sổ ngữ cảnh cục bộ.

Một lưu ý nữa về việc sử dụng dữ liệu toàn cục đáng được nhắc lại. Ma trận đồng xuất hiện mã hóa thông tin được tổng hợp trên toàn bộ kho ngữ liệu. Bằng cách phân tích trực tiếp ma trận này, GloVe đặt việc học thành một bài toán tối ưu toàn cục một cách tường minh. Mặc dù Word2Vec cũng tìm một

tập tham số tối ưu toàn cục (ví dụ cực đại hóa hợp lý trên toàn bộ không gian đầu vào), hàm mục tiêu của nó phức tạp và thường được tối ưu bằng học trực tuyến (*online learning*) (hạ gradient ngẫu nhiên trên các lô cục bộ). Còn GloVe, nhờ tổng hợp trước dữ liệu thành một ma trận đồng xuất hiện, đã chắt lọc quá trình học thành một mô hình toàn cục trực tiếp hơn và hiệu quả hơn về tính toán.

3.5.3 Nhận xét

Ở các mục trước, ta đã thấy vectơ từ có thể được quy nạp bằng nhiều phương pháp khác nhau. Giờ đây ta bàn về một số cân nhắc quan trọng khi huấn luyện hay áp dụng các vectơ nhúng từ này trong thực tế.

1. Dựa trên đếm và dựa trên mạng nơ-ron

Sự đơn giản và tính diễn giải được của các phương pháp dựa trên đếm từ lâu đã được cộng đồng NLP đánh giá cao. Cả các mô hình dựa trên đếm lẫn các mô hình dựa trên mạng nơ-ron về cơ bản đều dựa trên giả thuyết phân bố, tức ý tưởng rằng các từ xuất hiện trong ngữ cảnh tương tự nhau thì có nghĩa tương tự nhau. Tuy nhiên, các phương pháp dựa trên đếm xây dựng một cách tường minh những ma trận nhiều chiều và thưa dựa trên giả định này, điều thường dẫn đến vấn đề dữ liệu thưa do lời nguyền số chiều (*curse of dimensionality*).

Ở đầu kia là các mô hình dựa trên mạng nơ-ron. Thay vì dựa vào việc đếm đồng xuất hiện tường minh và các sơ đồ trọng số thủ công, các phương pháp này xem vectơ từ là những trạng thái trung gian ít chiều, dày đặc bên trong một mạng nơ-ron được tối ưu cho một bài toán nền cụ thể. Bằng cách loại bỏ các ràng buộc ngôn ngữ học tường minh về ý nghĩa cụ thể của từng chiều trong vectơ, các mô hình dựa trên mạng nơ-ron học hiệu quả những đặc trưng ngữ nghĩa tiềm ẩn vốn khó thiết kế thủ công.

Sự đối lập giữa hai cách tiếp cận này giống với sự so sánh rộng hơn giữa hai mô thức học máy nổi tiếng: kỹ nghệ đặc trưng (*feature engineering*) và học biểu diễn đầu-cuối (*end-to-end representation learning*). Tuy nhiên, ở đây chúng tôi sẽ không đi sâu vào cuộc tranh luận mang tính triết lý này. Chỉ cần lưu ý rằng không có dạng nào tuyệt đối vượt trội so với dạng kia. Nói chung, việc chọn loại vectơ từ hoàn toàn phụ thuộc vào ứng dụng hạ nguồn đích và mức độ diễn giải được mong muốn. Ví dụ, nếu mục tiêu là thu được một dạng tính toán diễn giải được một cách tường minh, minh bạch về toán học trực tiếp từ một kho ngữ liệu, thì phân tích ma trận đồng xuất hiện vẫn là một lựa chọn vững chắc. Ngược lại, nếu mục tiêu là trích xuất các vectơ từ liên tục, dày đặc để tích hợp liền mạch vào một kiến trúc nơ-ron sâu, thì học biểu diễn bằng mạng nơ-ron thường được ưu tiên.

Cần lưu ý rằng việc học các vectơ nhúng từ liên tục, dày đặc đã trở thành mô thức chủ đạo trong những năm gần đây, cho thấy những tiến bộ vượt bậc của các mô hình nơ-ron cho NLP. Đồng thời, giới học thuật cũng rất quan tâm đến việc so sánh các phương pháp dựa trên đếm và dựa trên mạng nơ-ron, và làm sáng tỏ những quan hệ lý thuyết sâu sắc giữa chúng [Levy và Goldberg, 2014b; Baroni và cộng sự, 2014; Levy và Goldberg, 2014c; Schnabel và cộng sự, 2015a; Levy và cộng sự, 2015; Gladkova và cộng sự, 2016].

Chú giải của người dịch: “Thưa” và “dày đặc”

Một vectơ *thưa* là vectơ mà hầu hết phần tử bằng 0, như các hàng của ma trận đồng xuất hiện: với bộ từ vựng 20 nghìn từ, một từ thường chỉ đi cùng vài trăm từ khác, còn lại toàn số 0. Một vectơ *dày đặc* (như $[0.21 \quad -1.3 \quad 0.07 \quad \ldots]$ với vài trăm chiều) thì hầu hết phần tử khác 0 và mỗi chiều không có ý nghĩa riêng dễ gọi tên. “Lời nguyền số chiều” ám chỉ rằng khi số chiều quá lớn, dữ liệu trở nên quá thưa thớt so với không gian, khiến việc ước lượng và so sánh kém tin cậy.

2. Mô hình nông và mô hình sâu

Mặc dù mạng nơ-ron đã trở thành cách tiếp cận tiêu chuẩn để quy nạp vectơ từ, các cấu trúc mô hình giới thiệu từ đầu chương đến giờ đều đơn giản. Về mặt kỹ thuật, chúng chỉ gồm một hoặc hai tầng nơ-ron, và do đó được xếp vào loại mô hình nông (*shallow models*). Một ví dụ đáng chú ý là mô hình nhúng từ **vLBL** [Mnih và Kavukcuoglu, 2013], mô hình hóa tương tác giữa các từ bằng một mạng nơ-ron chỉ có đúng hai tầng. Kiến trúc này, thậm chí bỏ cả hàm chuẩn hóa softmax, theo hiểu biết của chúng tôi là một trong những mô hình nhúng từ đơn giản nhất. Dù đơn giản như vậy, các mô hình này vẫn rất hiệu quả trong nhiều tình huống. Lợi ích chính của mô hình nông là hiệu quả tính toán và khả năng mở rộng tới những tập dữ liệu khổng lồ. Sự hiệu quả này cho phép chúng được áp dụng cho các bài toán NLP phức tạp hơn. Một ví dụ nổi bật là hệ thống **fastText** cho phân loại văn bản [Joulin và cộng sự, 2017]. Nó có kiến trúc tương tự mô hình CBOW (xem Mục 3.5.1). Trong fastText, văn bản đầu vào được biểu diễn như một túi các vectơ từ, được lấy trung bình để tạo thành một biểu diễn ẩn toàn cục của văn bản. Theo sau ngay là một tầng đầu ra ánh xạ biểu diễn ẩn này thành một phân phối xác suất trên các lớp định trước. Bằng cơ chế này, mục tiêu phân loại và các vectơ từ bên dưới được huấn luyện đồng thời.

Mặc dù mô hình nông rất hiệu quả trong việc học các biểu diễn từ tĩnh, mạng nơ-ron sâu mang lại năng lực biểu diễn lớn hơn đáng kể. Như với hầu hết các kiến trúc nhiều tầng, học biểu diễn bằng mạng nơ-ron sâu có một số ưu điểm rõ rệt [Telgarsky, 2016]. Thứ nhất, mô hình sâu cho phép ta khám phá một không gian giả thuyết lớn hơn nhiều, có khả năng tìm ra những ánh xạ hàm tốt hơn. Thứ hai, việc đưa vào nhiều phép biến đổi phi tuyến làm tăng mạnh khả năng nắm bắt những khuôn mẫu rất phức tạp trong dữ liệu của mô hình. Có nhiều chiến lược tận dụng độ sâu trong học biểu diễn. Cách trực tiếp nhất là xếp chồng dần các tầng truyền thẳng, hồi quy hoặc tích chập lên trên một tầng nhúng cơ sở. Với việc các mô hình tiền huấn luyện quy mô lớn được dùng rộng rãi trong NLP, mô thức đã chuyển sang việc trích xuất các biểu diễn theo ngữ cảnh (*contextualized representations*) trực tiếp từ các kiến trúc rất sâu, đạt hiệu năng tiên tiến nhất trên rất nhiều bộ đánh giá chuẩn NLP [Radford và cộng sự, 2018; Devlin và cộng sự, 2019]. Tuy nhiên, năng lực mô hình hóa tăng lên này kéo theo những nút thắt tính toán nghiêm trọng và sự bất ổn định khi huấn luyện. Do đó, một khối lượng lớn nghiên cứu tập trung vào việc giảm nhẹ những thách thức tối ưu hóa này [Pascanu và cộng sự, 2013; Bapna và cộng sự, 2018; Wang và cộng sự, 2019a; Zhang và cộng sự, 2019a; Pham và cộng sự, 2019; Li và cộng sự, 2020b]. Trong Chương 4–6, ta sẽ tìm hiểu một số mô hình NLP thành công được xây dựng trên các kiến trúc nơ-ron sâu này.

3. Mục tiêu huấn luyện

Ý tưởng coi vectơ từ là tham số mô hình rất phù hợp với mô hình hóa biến tiềm ẩn (*latent-variable modeling*): một mô hình được tham số hóa bằng các vectơ từ có thể học, và giá trị tối ưu của chúng được suy ra bằng cách tối ưu một hàm mục tiêu toàn cục. Mặc dù mô thức học này là tiêu chuẩn, mục tiêu huấn luyện cụ thể có thể khác nhau rất nhiều. Một khó khăn chính là không có một mục tiêu tường minh chỉ dành riêng cho việc giám sát biểu diễn vectơ từ. Một giải pháp trực tiếp là tận dụng các bài toán NLP được định nghĩa rõ ràng. Chẳng hạn, ta có thể dùng vectơ từ làm biểu diễn đầu vào cho các hệ thống mô hình hóa ngôn ngữ hay phân loại văn bản. Khi đó, các vectơ từ đóng vai trò như những tham số mô hình thông thường và được tối ưu theo kiểu đầu-cuối.

Một cách tiếp cận khác là thiết kế những mục tiêu huấn luyện mới. Như thường thấy trong học máy, không gian thiết kế các mục tiêu như vậy là rất rộng. Vì vậy, một chiến lược phổ biến là điều chỉnh các bài toán sẵn có. Ví dụ, mục tiêu huấn luyện của mô hình CBOW về bản chất là một bài toán dự đoán từ tổng quát hóa và có dạng toán học tương tự mô hình hóa ngôn ngữ truyền thống.

Ta sẽ tìm hiểu một số mục tiêu mới bắt nguồn từ mô hình hóa ngôn ngữ trong Chương 7. Nói chặt chẽ, các bài toán huấn luyện này vốn không nhằm học vectơ từ. Đúng hơn, chúng cung cấp một mục tiêu thay thế được định nghĩa rõ ràng cho việc học biểu diễn. Hơn nữa, ta thường không thể giả định một kịch bản học có giám sát thuần túy: các vectơ nhúng học được cuối cùng sẽ được triển khai trong nhiều hệ thống hạ nguồn khác nhau mà ta chưa biết trong giai đoạn huấn luyện ban đầu. Điều này khiến quá trình quy nạp trở thành một dạng học tự giám sát, thiếu sự giám sát trực tiếp từ ứng dụng đích cuối cùng. Tuy nhiên, khi tiếp cận được ứng dụng đích và có sẵn dữ liệu có nhãn của bài toán, ta có thể tinh chỉnh thêm các vectơ nhúng từ đã tiền huấn luyện này để nắm bắt ngữ nghĩa đặc thù của lĩnh vực.

3.6 Đánh giá vectơ nhúng từ

Sau khi có được biểu diễn vectơ của các từ, ta cần đánh giá chất lượng của chúng. Lý tưởng nhất, ta sẽ đánh giá các vectơ từ này so với một đáp án chuẩn. Đáng tiếc là đáp án chuẩn như vậy hiếm khi tồn tại, vì người gán nhãn không thể trực tiếp gán các giá trị số liên tục, nhiều chiều để nắm bắt ngữ nghĩa của một từ. Một giải pháp thực tế là đánh giá các vectơ dựa trên hiệu năng của một hệ thống hạ nguồn đang hoạt động có sử dụng chúng. Nhìn chung, các cách đánh giá được chia thành hai loại [Schnabel và cộng sự, 2015b]:

- **Đánh giá ngoại tại** (*extrinsic evaluation*) (hay kiểm thử đầu-cuối). Ta đưa các vectơ từ vào một hệ thống NLP hạ nguồn và đo tác động của chúng lên hiệu năng tổng thể của hệ thống.
- **Đánh giá nội tại** (*intrinsic evaluation*). Ta kiểm tra khả năng của các vectơ từ trong việc nắm bắt những tính chất hình thái, cú pháp và ngữ nghĩa cụ thể của ngôn ngữ.

Dưới đây, chúng tôi mô tả ngắn gọn cách áp dụng các cách tiếp cận này vào việc đánh giá vectơ từ.

3.6.1 Đánh giá ngoại tại

Cách tiếp cận này được dùng rộng rãi trong thực tế, vì nó cho phép các nhà nghiên cứu và kỹ sư nhanh chóng kiểm tra một hệ thống thực tế hoạt động ra sao khi các biểu diễn bên dưới của nó bị thay đổi. Vì nhiều hệ thống NLP nhận đầu vào là các từ, việc thay biểu diễn từ dạng ký hiệu bằng các vectơ từ dày đặc là cách làm tiêu chuẩn. Ta đã thấy các hệ thống loại này, vốn thường dùng một tầng nhúng để biến biểu diễn one-hot của mỗi từ đầu vào thành một vectơ giá trị thực (ví dụ mô hình ngôn ngữ nơ-ron ở

Chương 2).

Cho một hệ thống như vậy và một tập vectơ từ đã tiền huấn luyện, ta có thể dùng hiệu năng hạ nguồn của hệ thống làm thước đo gián tiếp cho chất lượng của các vectơ nhúng. Tùy theo cách sử dụng các vectơ từ, có hai mô thức huấn luyện chính:

- **Vectơ từ là tham số cố định**. Ta "đóng băng" các vectơ từ và chỉ huấn luyện các tham số còn lại của hệ thống như thường lệ.
- **Vectơ từ là tham số khởi tạo** (tinh chỉnh). Ta khởi tạo tầng nhúng bằng các vectơ từ đã tiền huấn luyện và cập nhật chúng cùng với các tham số còn lại của mô hình trong quá trình huấn luyện.

Cả hai phương pháp đều tận dụng khái niệm tiền huấn luyện, và có thể được mở rộng cho nhiều tình huống mà một phần của mô hình được huấn luyện kỹ trước khi áp dụng vào một bài toán hạ nguồn. Đóng băng vectơ từ giúp đơn giản hóa quá trình huấn luyện và tách riêng việc đánh giá bản thân các biểu diễn. Ngược lại, coi vectơ từ là tham số có thể học làm tăng độ phức tạp của việc huấn luyện nhưng cho phép mô hình tinh chỉnh các biểu diễn, điều chỉnh chúng riêng cho hệ thống đích.

Lưu ý rằng mặc dù đánh giá ngoại tại rất thực tế, kết quả của nó phụ thuộc nhiều vào kiến trúc và bài toán hạ nguồn cụ thể. Vì việc phát triển các hệ thống NLP tiên tiến nhất bao gồm các kiến trúc phức tạp và việc tinh chỉnh siêu tham số rất công phu, sự thay đổi hiệu năng không phải lúc nào cũng quy được cho riêng chất lượng của vectơ nhúng từ. Sự phụ thuộc vào bài toán này là phổ biến trong NLP. Chẳng hạn, một phương pháp tách token rất tốt trong hệ thống dịch máy có thể hoạt động kém trong một hệ thống truy xuất thông tin. Vì vậy, để kiểm tra chặt chẽ khả năng tổng quát hóa của một tập vectơ từ, cách làm tiêu chuẩn là đánh giá chúng trên một loạt các bài toán NLP hạ nguồn đa dạng.

3.6.2 Đánh giá nội tại

Mặc dù phần lớn nghiên cứu về biểu diễn từ liên quan tới kiểm thử đầu-cuối trong các ứng dụng NLP, nó cũng bao gồm việc kiểm tra khả năng của biểu diễn trong việc giải quyết một số vấn đề nhất định, chẳng hạn diễn giải quan hệ giữa hai từ. Có nhiều cách thiết kế đánh giá nội tại, mỗi cách giải quyết một vấn đề cụ thể. Dưới đây chúng tôi mô tả một số phương pháp trong đó. Để có cái nhìn tổng quan đầy đủ hơn về các phương pháp đánh giá nội tại, mời bạn đọc tham khảo các tài liệu phong phú về chủ đề này [Baroni và cộng sự, 2014; Bakarov, 2018; Rogers và cộng sự, 2018].

1. Độ liên quan ngữ nghĩa

Mô hình hóa độ liên quan giữa các từ có lẽ là phương pháp phổ biến nhất để đánh giá chất lượng vectơ từ trong NLP [Reisinger và Mooney, 2010; Huang và cộng sự, 2012; Baroni và cộng sự, 2014]. Về cơ bản, việc này gồm tính một khoảng cách không gian giữa các từ (gọi là **khoảng cách ngữ nghĩa của từ** (*word semantic distance*) hay đơn giản là **khoảng cách từ** (*word distance*)). Động cơ ở đây là khoảng cách hình học trong không gian vectơ liên tục nên tương quan với phán đoán nhận thức của con người về độ liên quan ngữ nghĩa [Rubenstein và Goodenough, 1965]. Ví dụ, ta mong đợi vectơ của *dog* (chó) nằm gần *wolf* (sói) về mặt hình học, còn *peach* (quả đào) thì nên ở xa *television* (ti vi). Về mặt toán học, có nhiều độ đo có thể dùng để tính khoảng cách hay góc giữa hai vectơ. Một độ đo đơn giản và thường dùng là khoảng cách Euclid. Ngoài ra, tính độ tương tự cosin (*cosine similarity*) giữa hai vectơ sẽ cho một điểm số trong khoảng $[-1, 1]$[19].

[19]Trong thực tế, người ta thường chuyển nó thành *khoảng cách cosin* (*cosine distance*), định nghĩa là $1 - \cos(\theta)$, trong đó khoảng cách bằng 0 nghĩa là hai vectơ chỉ đúng cùng một hướng, còn bằng 1 nghĩa là hai vectơ vuông góc.

Chú giải của người dịch: Độ tương tự cosin

Với hai vectơ $\mathbf{u}$, $\mathbf{v}$, độ tương tự cosin là $\cos(\theta) = \dfrac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u}|\,|\mathbf{v}|}$, đúng công thức tính góc giữa hai vectơ ở lớp 10, chỉ khác là có nhiều tọa độ hơn. Ví dụ $\mathbf{u} = (1; 2)$, $\mathbf{v} = (2; 4)$ thì $\cos\theta = \frac{10}{\sqrt{5}\cdot\sqrt{20}} = 1$ (cùng hướng, rất giống nhau); $\mathbf{u} = (1; 0)$, $\mathbf{v} = (0; 1)$ thì $\cos\theta = 0$ (vuông góc, không liên quan). Điểm cosin không phụ thuộc độ dài vectơ, nên một từ phổ biến (vectơ đếm lớn) và một từ hiếm vẫn có thể được đánh giá là "giống nhau" nếu chúng cùng hướng.

Trong thiết lập đánh giá này, ta được cung cấp một tập dữ liệu gồm các cặp từ, mỗi cặp được người gán nhãn cho một điểm số độ liên quan kỳ vọng. Với một cặp từ, ta tính tương quan giữa điểm kỳ vọng của con người và khoảng cách hình học đo được trong không gian vectơ từ. Chất lượng của các vectơ từ được phản ánh qua độ mạnh của mối tương quan này. Tuy nhiên, một thách thức chính là không có một thước đo chuẩn tuyệt đối cho khoảng cách ngữ nghĩa. Ngay cả người gán nhãn cũng khó định lượng mức độ gần gũi ngữ nghĩa với độ chính xác số tuyệt đối. Một giải pháp thực tế là ánh xạ khoảng cách vào các hạng mục rời rạc hoặc các thang đánh giá kiểu Likert, chẳng hạn các số nguyên trong $[1, 5]$ [Reisinger và Mooney, 2010]. Điều này giảm đáng kể gánh nặng nhận thức trong quá trình gán nhãn dữ liệu. Một biến thể đánh giá khác yêu cầu mô hình xác định từ giống nhất với một từ đích cho trước trong một tập nhỏ các ứng viên (ví dụ bài kiểm tra từ đồng nghĩa). Cách này tránh việc phải dự đoán giá trị tuyệt đối. Thay vào đó, ta chỉ cần một cơ chế xếp hạng các độ tương tự tương đối [Baroni và cộng sự, 2014].

Tuy nhiên, việc đánh giá quan hệ giữa các từ có thể mơ hồ do bản chất linh hoạt của ngôn ngữ. Nói chung, nhiều yếu tố ngôn ngữ học ảnh hưởng tới phán đoán của con người về độ liên quan [Faruqui và cộng sự, 2016]. Ví dụ, *corn* (ngô) và *cornea* (giác mạc) giống nhau về hình thái do trùng một phần từ con, nhưng về ngữ nghĩa lại hoàn toàn không liên quan. Sự mơ hồ cũng bắt nguồn từ chính định nghĩa của các bài toán. **Độ liên quan ngữ nghĩa** (*semantic relatedness*) và **độ tương tự ngữ nghĩa** (*semantic similarity*) thường được dùng thay thế cho nhau, nhưng chúng biểu thị những khái niệm ngôn ngữ học khác nhau. Chẳng hạn, *car* (xe hơi) liên quan về chủ đề với *road* (con đường), nhưng theo nghĩa phân loại chặt chẽ thì *car* tương tự với *van* (xe tải nhỏ).

Hơn nữa, nghĩa của một từ nói chung phụ thuộc vào ngữ cảnh, và do đó gây nhiễu cho việc đánh giá những từ có nhiều nghĩa khác nhau (**đa nghĩa** (*polysemy*)). Nói rộng ra, đây là một hạn chế cố hữu của các mô hình vectơ từ tĩnh, trong đó mỗi loại từ được ánh xạ một cách tất định tới một vectơ cố định duy nhất. Để giải quyết vấn đề này, các chương sau sẽ giới thiệu các vectơ nhúng từ theo ngữ cảnh (*contextualized word embeddings*), vốn sinh ra một cách động một dạng tính toán riêng cho một từ dựa hoàn toàn vào văn bản xung quanh nó.

Chú giải của người dịch: Ví dụ tiếng Việt về đa nghĩa

Từ "*đường*" trong "*bỏ thêm đường vào cà phê*" và trong "*con đường làng*" có hai nghĩa hoàn toàn khác. Một mô hình vectơ từ tĩnh (như Word2Vec, GloVe) chỉ có *một* vectơ cho "đường", nên vectơ này buộc phải là một kiểu "trộn lẫn" của cả hai nghĩa, vừa hơi gần "muối", vừa hơi gần "phố". Vectơ nhúng theo ngữ cảnh (sẽ học ở các chương sau) tạo ra vectơ khác nhau cho "đường" tùy câu chứa nó.

2. Loại suy từ

Bài toán loại suy từ (*word analogy*) liên quan đến việc mô hình hóa quan hệ tương tự giữa các cặp từ. Bài toán này xuất phát từ tiền đề rằng các quan hệ ngữ nghĩa giữa các từ có thể được nắm bắt bằng những phép toán đại số đơn giản trên các biểu diễn liên tục tương ứng. Một ví dụ kinh điển do Mikolov và cộng sự [2013d] đưa ra cho thấy ngữ nghĩa quan hệ có thể được mô tả bằng phép trừ vectơ. Cụ thể, lấy vectơ của *king* (vua) trừ đi vectơ của *man* (đàn ông) rồi cộng vectơ của *woman* (phụ nữ) sẽ cho một vectơ rất gần với vectơ của *queen* (nữ hoàng):

$$\mathbf{e}_{\text{king}} - \mathbf{e}_{\text{man}} + \mathbf{e}_{\text{woman}} \approx \mathbf{e}_{\text{queen}}. \tag{3.61}$$

Một cách hình thức, loại suy từ là bài toán so sánh hai cặp từ (a, a^*) và (b, b^*). Một phép loại suy đúng nếu quan hệ ngữ nghĩa giữa a và a^* tương ứng với quan hệ giữa b và b^*. Điều này phản ánh những **quy luật ngôn ngữ** (*linguistic regularities*) tiềm ẩn trong không gian vectơ, có thể được phát biểu bằng đại số là:

$$\mathbf{e}_{a^*} - \mathbf{e}_a \approx \mathbf{e}_{b^*} - \mathbf{e}_b. \tag{3.62}$$

Như vậy, loại suy từ có thể được đặt thành một bài toán suy luận tương tự: mục tiêu là dự đoán $\mathbf{e}_{b^*}$ khi biết $\mathbf{e}_a$, $\mathbf{e}_{a^*}$ và $\mathbf{e}_b$. Cụ thể hơn, ta kỳ vọng vectơ $\mathbf{e}_{a^*} - \mathbf{e}_a + \mathbf{e}_b$ xấp xỉ sát $\mathbf{e}_{b^*}$ nếu (a, a^*) và (b, b^*) có quan hệ ngôn ngữ tương tự nhau. Có nhiều cách cải tiến cách phát biểu này. Chẳng hạn, thay vì chỉ tối ưu khoảng cách không gian để dự đoán $\mathbf{e}_{b^*}$, ta có thể đánh giá cosin của góc giữa các vectơ hiệu $\mathbf{e}_{a^*} - \mathbf{e}_a$ và $\mathbf{e}_{b^*} - \mathbf{e}_b$ để nắm bắt độ tương tự quan hệ một cách vững chắc hơn [Levy và Goldberg, 2014b].

Chú giải của người dịch: Phép loại suy như hình bình hành

Phương trình (3.62) nói rằng "mũi tên" đi từ a đến a^* gần bằng "mũi tên" đi từ b đến b^*; bốn điểm tạo thành gần như một hình bình hành, đúng như quy tắc hình bình hành của vectơ ở lớp 10. Ví dụ với $a = \textit{man}$, $a^* = \textit{woman}$, $b = \textit{king}$: vectơ $\mathbf{e}_{\text{woman}} - \mathbf{e}_{\text{man}}$ như mã hóa hướng "nam $\rightarrow$ nữ"; cộng hướng đó vào $\mathbf{e}_{\text{king}}$ ta đến gần $\mathbf{e}_{\text{queen}}$. Tương tự có thể thử "Hà Nội – Việt Nam + Nhật Bản $\approx$ Tokyo" (quan hệ thủ đô). Trong thực tế, người ta tìm từ có vectơ gần nhất (theo cosin) với $\mathbf{e}_{a^*} - \mathbf{e}_a + \mathbf{e}_b$, loại trừ chính các từ a, a^*, b.

Bài toán loại suy từ cung cấp một cơ chế đơn giản để kiểm tra các tính chất đại số tuyến tính của một mô hình vectơ từ. Những đặc điểm như vậy hiếm khi quan sát được ở các phương pháp dựa trên đếm truyền thống. Đáng chú ý là các mô hình nhúng từ hiện đại thể hiện hành vi tuyến tính rõ rệt dù các ràng buộc hình học như vậy không được áp đặt tường minh trong quá trình huấn luyện. Tính chất tự phát này cho các nhà nghiên cứu cái nhìn sâu sắc về những cấu trúc tiềm ẩn mà các mô hình này học được và các ứng dụng hạ nguồn tiềm năng của chúng [Levy và Goldberg, 2014b; Linzen, 2016; Allen và Hospedales, 2019]. Tuy nhiên, độ chính xác loại suy từ không phải là một thước đo có tính chỉ báo phổ quát. Trong nhiều tình huống, nó không tương quan mạnh với hiệu năng của các hệ thống NLP hạ nguồn. Vì vậy, nó thường được dùng như một công cụ phân tích để thăm dò các tính chất cấu trúc và ngữ nghĩa của biểu diễn từ.

3. Phân loại từ theo nhóm (hay phân cụm)

Một phương pháp khác để đánh giá xem vectơ từ có khớp với hiểu biết ngữ nghĩa của con người hay không là kiểm tra khả năng chúng được nhóm vào các hạng mục có ý nghĩa. Việc này thường được thực hiện bằng cách áp dụng các thuật toán phân cụm (*clustering*) tiêu chuẩn lên các vectơ từ. Lý tưởng nhất, các từ tương tự nhau về ngữ nghĩa nên được xếp vào cùng một cụm, còn các từ khác nhau được tách vào các cụm riêng. Ví dụ, *apple* (táo), *grape* (nho), *peach* (đào) và *orange* (cam) nên tự nhiên được gom vào cùng một cụm thuộc hạng mục ngữ nghĩa rộng hơn là trái cây. Một ưu điểm của cách đánh giá này

là có thể áp dụng ngay nhiều thuật toán phân cụm và các bộ đánh giá chuẩn về phân loại từ đã có sẵn. Tuy nhiên, như với hầu hết các bài toán phân cụm không giám sát, cần giải quyết một số thách thức thực tế, chẳng hạn xác định trước số cụm tối ưu.

Vì phần lớn các phương pháp phân cụm trong học máy dựa trên việc tính khoảng cách không gian giữa các điểm dữ liệu, phân cụm từ bắt nguồn từ khái niệm độ liên quan ngữ nghĩa. Tuy nhiên, khác với các bài toán đã bàn ở Mục 3.6.2, phân cụm đánh giá cấu trúc toàn cục thay vì phán xét tường minh từng điểm số khoảng cách theo cặp. Điều này cho thấy mối liên hệ nội tại giữa các quy trình đánh giá khác nhau, nhưng cũng có nghĩa là phân cụm từ mang những hạn chế cố hữu của các phương pháp liên quan đó. Đáng chú ý nhất, rất khó thiết lập một tiêu chí chuẩn tuyệt đối cho chất lượng phân cụm, vì các hệ phân loại ngữ nghĩa mang tính chủ quan, và các từ có thể được phân loại theo nhiều cách tùy vào góc nhìn ngữ nghĩa được chọn.

4. Đánh giá tiềm thức

Mục tiêu của đánh giá tiềm thức (*subconscious evaluation*) (hay đánh giá nhận thức) là xem xét mối tương quan giữa việc sử dụng vectơ từ và các hành vi tiềm thức hay hoạt động não bộ của con người trong khi đọc. Rất nhiều hiện tượng ngôn ngữ học tâm lý có thể dùng làm đại lượng đánh giá gián tiếp [Mitchell và Lapata, 2010]. Một mô thức nổi tiếng là **mồi** (*priming*), nghiên cứu cách một người phản ứng với các kích thích nối tiếp nhau [Schacter và Buckner, 1998; Tulving và Schacter, 1990; Wiggs và Martin, 1998]. Ví dụ, có thể thiết kế một thí nghiệm đo tốc độ một người tham gia đọc một từ cho trước (gọi là **từ đích** (*target word*)) ngay sau một từ khác (**từ mồi** (*prime word*)) [Meyer và Schvaneveldt, 1971; Lund, 1995; McNamara, 2005]. Nếu từ đích t được xử lý nhanh hơn khi đứng sau từ a so với khi đứng sau từ b, điều đó cho thấy liên kết nhận thức giữa t và a mạnh hơn. Độ trễ tâm lý này khi đó có thể dùng làm đại lượng gián tiếp để đánh giá độ tương tự ngữ nghĩa giữa các vectơ từ tương ứng. Để đo độ trễ khi đọc, một phương pháp phổ biến là bài toán **đọc tự điều chỉnh nhịp** (*self-paced reading*)[20]. Một cách khác là dùng thiết bị theo dõi mắt để tự động ghi lại thông tin chính xác về chuyển động và điểm dừng của mắt. Nhờ các kỹ thuật này, nhiều tập dữ liệu và phương pháp đã được phát triển để nghiên cứu nhiều vấn đề ngôn ngữ học tâm lý khác nhau [Mitchell và Lapata, 2010; Hutchison và cộng sự, 2013; Lapesa và Evert, 2014; Klerke và cộng sự, 2015; Søgaard, 2016; Auguste và cộng sự, 2017].

Ngoài việc theo dõi phản ứng hành vi trong khi đọc, ta có thể trực tiếp theo dõi hoạt động não bằng các kỹ thuật chụp ảnh thần kinh, chẳng hạn chụp cộng hưởng từ chức năng (fMRI) và điện não đồ (EEG) [Devereux và cộng sự, 2010; Søgaard, 2016; Bhattasali và cộng sự, 2020]. Ví dụ, việc hiểu ngôn ngữ gây ra những mẫu kích hoạt thần kinh cụ thể trong não. Do đó, ta có thể thử tìm tương quan trực tiếp giữa các mẫu kích hoạt sinh lý này với dạng tính toán của nghĩa từ do biểu diễn vectơ cung cấp. Tuy nhiên, một hạn chế là hiểu biết của chúng ta về các cơ chế nhận thức nằm bên dưới những quá trình này vẫn chưa đầy đủ, khiến việc liên hệ thỏa đáng kết quả của các nghiên cứu ngôn ngữ học tâm lý như vậy với hiệu năng của các hệ thống NLP thực tế trở nên khó khăn [Baroni và cộng sự, 2014; Bakarov, 2018].

Chú giải của người dịch: Hiệu ứng mồi trong đời thường

Nếu bạn vừa đọc chữ "*bác sĩ*", bạn sẽ nhận ra chữ "*y tá*" nhanh hơn một chút (vài chục mili giây) so với khi vừa đọc chữ "*bàn ghế*". Não đã được "mồi" sẵn cho các khái niệm liên quan. Ý tưởng đánh giá: nếu vectơ của "bác sĩ" gần "y tá" hơn "bàn ghế", thì vectơ từ phản ánh đúng cách con người liên kết các khái niệm.

[20] Trong đọc tự điều chỉnh nhịp, văn bản được chia thành các từ hoặc cụm từ, và người tham gia nhấn một nút để hiển thị lần lượt từng đoạn.

5. Đánh giá dựa trên ngôn ngữ học

Đánh giá dựa trên ngôn ngữ học xuất phát từ giả định rằng các vectơ từ được học từ dữ liệu nên phản ánh tri thức cấu trúc và ngữ nghĩa được mã hóa trong các tài nguyên ngôn ngữ học. Một cách thú vị để thực hiện việc đánh giá này là căn chỉnh các vectơ từ với biểu diễn của các mục từ trong từ điển [Tsvetkov và cộng sự, 2015; Acs và Kornai, 2016]. Chất lượng của vectơ nhúng từ khi đó được định lượng bằng mức tương quan giữa các vectơ liên tục này với các biểu diễn ngôn ngữ học rời rạc[21]. Ngoài các từ điển tiêu chuẩn, vectơ từ có thể được đánh giá theo cấu trúc tô-pô trong các mạng ngữ nghĩa, chẳng hạn WordNet. Bằng cách này, các thước đo đánh giá có thể được làm phong phú hơn nhờ dùng các thuật toán trên đồ thị để đo mức độ khớp về cấu trúc giữa không gian vectơ và các tài nguyên ngữ nghĩa này [Agirre và cộng sự, 2009].

3.6.3 Trực quan hóa

Bằng cách coi các vectơ từ là những điểm dữ liệu nhiều chiều, ta có thể dùng các kỹ thuật giảm chiều để chiếu chúng xuống không gian hai hoặc ba chiều nhằm trực quan hóa. Các phép chiếu như vậy cho phép ta phân tích trực quan những khuôn mẫu ngữ nghĩa được mã hóa trong các vectơ này và khám phá các quan hệ tiềm ẩn giữa các từ. Một phương pháp là PCA, vốn tìm một ánh xạ tuyến tính từ không gian nhiều chiều xuống không gian ít chiều (xem Mục 3.3.3). Một phương pháp nổi tiếng khác là **nhúng láng giềng ngẫu nhiên phân phối t** (*t-distributed stochastic neighbor embedding*) (**t-SNE**) [Hinton và Roweis, 2002; Van der Maaten và Hinton, 2008]. Đây là một phương pháp giảm chiều phi tuyến, và đã được dùng rộng rãi trong trực quan hóa dữ liệu nhiều chiều. Ngoài ra, ta có thể xem xét các phương pháp trình bày trong Mục 3.3.3 cũng như các phương pháp được thiết kế riêng cho việc trực quan hóa vectơ từ [Zhang và cộng sự, 2019b; Liu và cộng sự, 2017].

3.7 Tóm tắt

Trong chương này, chúng ta đã bàn về hai vấn đề thú vị trong NLP: tách token và biểu diễn từ (hay biểu diễn token). Thứ nhất, chúng tôi đã giới thiệu các mô hình chia một câu thành những đơn vị có nghĩa và/hoặc phù hợp với các bài toán hạ nguồn. Thứ hai, chúng tôi đã giới thiệu ý tưởng về các mô hình vectơ từ, đặc biệt chú trọng vào việc học cả các mô hình nhiều chiều dựa trên đếm lẫn các mô hình ít chiều giá trị thực. Mặc dù phần lớn các mô hình này đơn giản, chúng thường được dùng trong các hệ thống NLP phức tạp và là nền tảng của nhiều mô hình tiên tiến, như sẽ thấy ở các chương sau.

Tách token (hay phân đoạn) là một quá trình quan trọng trong NLP, thường là bước tiền xử lý cho nhiều ứng dụng [Webster và Kit, 1992]. Tuy nhiên, việc dùng thuật ngữ *tách token* (*tokenization*) hơi dễ gây hiểu lầm vì ban đầu nó chỉ quá trình cơ bản trong khoa học máy tính là chia một xâu thành các xâu con. Trong NLP hiện đại, tách token dựa trên các khái niệm từ nhiều lĩnh vực con. Về mặt ngôn ngữ học, nó gắn chặt với hai câu hỏi cốt lõi: các từ được cấu tạo về hình thái như thế nào và chúng kết hợp về cú pháp thành câu ra sao. Vì vậy, việc dùng các lý thuyết hình thái học và cú pháp học để định nghĩa các đơn vị cơ bản của một ngôn ngữ là điều tự nhiên, dẫn tới nhiều hệ thống tách token dựa trên quy tắc cho nhiều ngôn ngữ khác nhau. Từ góc nhìn học máy, tách token từ lâu đã được xem là bài toán học ranh giới token từ dữ liệu theo cách có giám sát hoặc không giám sát [Mielke và cộng sự, 2021]. Một cách tiếp cận phổ biến là trước hết gán nhãn một ít văn bản đã tách token bằng tri thức của con người về việc các đơn vị ngôn ngữ cơ bản nên là gì, rồi học cách tách token trên dữ liệu đã gán nhãn này (xem

[21] Trong ngữ cảnh này, biểu diễn ngôn ngữ học thường là một vectơ đặc trưng được xây dựng thủ công từ một tài nguyên ngôn ngữ học chính quy, chẳng hạn một từ điển từ vựng.

Mục 3.1.3). Gần đây hơn, việc học các bộ tách token không có ràng buộc ngôn ngữ học được thấy là đầy hứa hẹn (xem Mục 3.1.4). Vì bản thân ngôn ngữ tự nhiên là tập các chuỗi ký tự hay chuỗi byte, ta cũng có thể chia một câu thành các ký tự hoặc byte [Ling và cộng sự, 2015; Lee và cộng sự, 2017]. Các phương pháp không cần tách token nói chung hữu ích khi ta muốn một bộ tách token độc lập với ngôn ngữ và một quy trình xử lý văn bản đơn giản hơn.

Từ góc độ toán học, tách token có thể được xem là một ánh xạ từ dữ liệu đầu vào sang một chuỗi biến. Do đó, khái niệm tách token có thể được tổng quát hóa bằng cách nới lỏng giả định rằng cả biến đầu vào lẫn đầu ra đều bị giới hạn ở các giá trị rời rạc. Chẳng hạn, các hệ thống xử lý ảnh và tiếng nói gần đây biến dữ liệu cảm quan liên tục (như điểm ảnh và tín hiệu âm thanh) thành các chuỗi "token" dạng vectơ [Schneider và cộng sự, 2019; Dosovitskiy và cộng sự, 2021]. Một số mở rộng thú vị của những ý tưởng này thậm chí còn biến dữ liệu ảnh và tiếng nói thành một chuỗi các chỉ số rời rạc, qua đó đưa thị giác máy tính và xử lý tiếng nói đến gần hơn với các mô thức NLP truyền thống [Oord và cộng sự, 2017; Baevski và cộng sự, 2020; Hsu và cộng sự, 2021].

Sau khi chia văn bản đầu vào thành các đoạn nhỏ hơn, bước tự nhiên tiếp theo là biểu diễn những mảnh này theo cách nào đó nắm bắt được các đặc trưng bên dưới của chúng. Mặc dù biểu diễn các đơn vị ngôn ngữ bằng các vectơ số dày đặc đã trở thành tiêu chuẩn trên thực tế trong NLP hiện đại, các nghiên cứu về biểu diễn vectơ đã có từ những ngày đầu của ngôn ngữ học tính toán. Theo nhiều giáo trình và bài báo phổ biến [Manning và Schütze, 1999; Jurafsky và Martin, 2008], ý tưởng dùng một phân bố thống kê để biểu diễn nghĩa của từ, được gọi là **ngữ nghĩa học phân bố**, xuất hiện vào những năm 1950 cùng với sự trỗi dậy của chủ nghĩa kinh nghiệm. Trong thời kỳ đó, phần lớn các nghiên cứu chịu ảnh hưởng mạnh mẽ của thuyết phân bố của Harris [Harris, 1954] và các khuôn khổ triết học liên quan [Firth, 1957; Wittgenstein, 1953]. Cùng thời gian đó, Osgood [1952] đề xuất định nghĩa nghĩa của một khái niệm theo góc độ tâm lý học, như một điểm trong một không gian nhiều chiều. Những ý tưởng này đã định hình đáng kể cách các nhà ngôn ngữ học và các nhà nghiên cứu NLP quan niệm về nghĩa của từ trong những thập kỷ sau đó.

Các cách tiếp cận hiện đại cho ngữ nghĩa học phân bố xuất hiện vào những năm 1990, chủ yếu là kết quả của sự hồi sinh chủ nghĩa kinh nghiệm trong trí tuệ nhân tạo [Church, 2011]. Phần lớn chúng được dẫn dắt bởi giả thuyết phân bố: các từ có nghĩa tương tự nhau thì có nhiều khả năng xuất hiện trong những ngữ cảnh tương tự nhau. Theo đó, nhiều phương pháp đã được phát triển, khác nhau ở cách mô hình hóa ngữ cảnh. Ví dụ, ngữ cảnh có thể là các từ trong một cửa sổ ngữ cảnh, hoặc các từ có quan hệ với từ đã cho trong một cây cú pháp. Ngoài những phương pháp đã nêu ở Mục 3.3, các phương pháp không được đề cập trong chương này gồm có mô hình tương tự siêu không gian của ngôn ngữ (*hyperspace analogue of language*) (HAL) [Lund và Burgess, 1996], bộ nhớ phân bố (*distributional memory*) [Baroni và Lenci, 2010], các mô hình không gian ngữ nghĩa dựa trên quan hệ phụ thuộc [Padó và Lapata, 2007], v.v. Để có cái nhìn tổng quan đầy đủ về ngữ nghĩa học phân bố, bạn đọc có thể tham khảo các tài liệu chuyên sâu [Lenci, 2018; Mitchell và Lapata, 2010]. Lưu ý rằng phần lớn các công trình nêu trên có thể được xem là những trường hợp của mô hình không gian vectơ (*vector space model*), vốn có thể giải quyết các vấn đề vượt ra ngoài ngữ nghĩa từ vựng. Ví dụ, trong ngữ nghĩa học phân bố kết hợp (*compositional distributional semantics*), nghĩa của một cụm từ hay một câu được suy ra bằng cách thực hiện các phép toán đại số có hệ thống trên các vectơ từ thành phần của nó [Clark và cộng sự, 2008; Mitchell và Lapata, 2010; Blacoe và Lapata, 2012].

Mặc dù các mô hình phân bố đã thu hút sự chú ý của cộng đồng NLP trong nhiều năm, các mô hình nhúng từ học các vectơ từ ít chiều, giá trị thực trực tiếp từ văn bản gần đây đã trở thành cách tiếp cận chủ đạo. Như đã mô tả trong Mục 3.4–3.5, thay vì xây dựng tường minh các ma trận đồng xuất hiện thưa, các mô hình này tận dụng ngầm giả thuyết phân bố bằng cách tối ưu một vectơ các thuộc tính tiềm ẩn để dự đoán các ngữ cảnh xung quanh. Mô hình thu được là một mở rộng của mô hình ngữ nghĩa dựa

trên đặc trưng [Markman, 2013]. Một khác biệt được thừa nhận so với các phương pháp dựa trên đặc trưng truyền thống là ta không cần định nghĩa thủ công các đặc trưng. Thay vào đó, ta coi các đặc trưng này là tham số của mô hình và huấn luyện chúng theo kiểu đầu-cuối.

Việc phát biểu biểu diễn từ thành một bài toán học đầu-cuối mang lại một số lợi ích. Ưu điểm chính là có thể tìm ra những đặc trưng mới, vì không có ràng buộc nào đặt lên cách học và diễn giải các đặc trưng này. Hơn nữa, như đã trình bày ở Mục 3.6.2, các vectơ từ thu được theo cách này thực sự thể hiện một số tính chất ngôn ngữ học, dù các mô hình nhúng từ không được huấn luyện để đạt điều đó. Một lợi ích khác là các mô hình nhúng từ cũng thuộc mô thức rộng hơn của mô hình không gian vectơ, giúp dễ dàng sử dụng vectơ từ trong nhiều ứng dụng. Cũng có nhiều ví dụ về các phương pháp cố gắng cải tiến các hệ thống nhúng từ tiêu chuẩn. Các nhà nghiên cứu đã thành công trong việc bổ sung tri thức ngôn ngữ học bên ngoài vào các vectơ nhúng đã tiền huấn luyện [Levy và Goldberg, 2014a; Cotterell và Schütze, 2015; Tissier và cộng sự, 2017] và căn chỉnh các không gian vectơ để thu được những biểu diễn phổ quát cho nhiều ngôn ngữ [Klementiev và cộng sự, 2012; Mikolov và cộng sự, 2013b; Ammar và cộng sự, 2020; Smith và cộng sự, 2017; Artetxe và cộng sự, 2017].

Gắn liền với các mô hình nơ-ron trong NLP, ý tưởng biểu diễn phân tán (*distributed representation*) đã được tổng quát hóa thành công cho những đơn vị cấu trúc vượt xa các từ riêng lẻ. Điều này bao gồm việc học biểu diễn dày đặc cho câu [Le và Mikolov, 2014; Kalchbrenner và cộng sự, 2014; Kiros và cộng sự, 2015; Hill và cộng sự, 2016; Arora và cộng sự, 2017; Lin và cộng sự, 2017; Conneau và cộng sự, 2017b] cũng như cho các cấu trúc cây và đồ thị phức tạp [Socher và cộng sự, 2011; Perozzi và cộng sự, 2014; Tai và cộng sự, 2015; Grover và Leskovec, 2016]. Đặc biệt, các biểu diễn theo ngữ cảnh của từ, dù không được bàn trong chương này, nhìn chung được đánh giá cao trong việc mô hình hóa dữ liệu dạng chuỗi [McCann và cộng sự, 2017; Peters và cộng sự, 2018; Devlin và cộng sự, 2019].

Chương 4

Các mô hình chuỗi hồi quy và tích chập

Cái toàn thể lớn hơn tổng các bộ phận của nó.

– Aristotle, 384–322 TCN [Ross, 1924]

Có thể Aristotle đã nghĩ, cũng có thể đã không nghĩ, tới các hiện tượng ngôn ngữ khi phát biểu ý tưởng này, nhưng đó đúng là điều chúng tôi muốn truyền tải trong chương này: một câu hay một cụm từ có ý nghĩa vượt xa một tập hợp từ đơn thuần. Mỗi từ tự nó đã mang nghĩa, nhưng khi các từ kết hợp với nhau thành câu hay cụm từ, ý nghĩa của tổng thể có thể phức tạp và đa dạng hơn rất nhiều. Điều này dẫn tới khía cạnh đẹp đẽ nhất của ngôn ngữ: con người có thể diễn đạt vô số ý nghĩa khác nhau chỉ bằng một tập hữu hạn các thành phần (ví dụ: từ hoặc ký tự).

Bản chất vô hạn và phụ thuộc mạnh vào ngữ cảnh của ngôn ngữ khiến việc mô hình hóa một chuỗi từ khó hơn nhiều so với việc mô hình hóa từng từ riêng lẻ. Một thách thức là nghĩa của một từ có thể thay đổi hoàn toàn tùy theo ngữ cảnh (*context*) của nó. Dựa trên khái niệm nhúng từ (*word embedding*), trong đó mỗi từ được biểu diễn bằng một vectơ số thực có số chiều thấp, "ý nghĩa" của một đơn vị ngôn ngữ có thể được mô hình hóa trong một không gian liên tục. Do đó, ta có thể mở rộng các phương pháp biểu diễn phân tán từ từng từ đơn lẻ lên toàn bộ chuỗi. Điều này thúc đẩy việc tìm hiểu các mô hình xử lý chuỗi từ có độ dài thay đổi theo một cách liên tục về bản chất.

Ở đây, chúng ta tìm hiểu cách tiếp cận tổng quát để học các biểu diễn phân tán của chuỗi từ. Cụ thể, chúng ta xem xét mạng nơ-ron hồi quy (*recurrent neural network*) và mạng nơ-ron tích chập (*convolutional neural network*), vốn đã được dùng rộng rãi trong nhiều lĩnh vực, từ xử lý tiếng nói tới thị giác máy tính. Với đầu vào là ngôn ngữ tự nhiên, áp dụng các mô hình này cho ta một biểu diễn (*representation*) ở mức chuỗi. Biểu diễn này có thể là một vectơ số thực duy nhất, hoặc là một chuỗi các vectơ như vậy, trong đó mỗi vectơ ứng với một biểu diễn theo ngữ cảnh của một từ cụ thể trong chuỗi đầu vào. Các mô hình biểu diễn này, thường được gọi là bộ mã hóa (*encoder*), được dùng phổ biến trong nhiều hệ thống xử lý dữ liệu tuần tự. Chúng ta sẽ thấy một số ví dụ như vậy trong chương này.

Chú giải của người dịch: "Hồi quy" và "tích chập" ở đây nghĩa là gì?

Chữ *hồi quy* trong "mạng nơ-ron hồi quy" (*recurrent*) mang nghĩa "lặp đi lặp lại, quay vòng": mạng đọc từng từ một và mỗi bước lại dùng lại kết quả của bước trước — giống như khi đọc một câu, ta nhớ những gì đã đọc rồi cộng thêm từ mới. Nó *không* liên quan tới "hồi quy tuyến tính" (*regression*) trong thống kê. Còn *tích chập* (*convolution*) là phép "trượt" một cửa sổ nhỏ dọc theo chuỗi và tính toán trên từng nhóm vài từ liền nhau (sẽ trình bày ở Mục 4.4).

4.1 Phát biểu bài toán

Với nhiều ứng dụng NLP, mục tiêu của chúng ta là đưa ra dự đoán dựa trên một chuỗi đầu vào. Hãy xét lại bài toán phân loại (*classification*) văn bản đã nhắc tới ở Chương 1. Cho một văn bản có thể nói hoặc không nói về đồ ăn, ta muốn gán cho nó một trong hai lớp (ví dụ: Food (đồ ăn) hoặc Not-food (không phải đồ ăn)). Để làm điều này, một phương pháp phân loại phổ biến là biểu diễn văn bản dưới dạng một "túi" các đặc trưng (*feature*), ký hiệu là $\mathbf{H}$. Sau đó, mỗi lớp được gán một xác suất bằng một mô hình xác suất $\Pr(y|\mathbf{H})$. Lớp được dự đoán là lớp làm cực đại xác suất này: $\hat{y} = \arg\max_y \Pr(y|\mathbf{H})$.

Tuy đây là một quy trình chuẩn cho bài toán phân loại, ý tưởng đằng sau nó mô tả một bài toán rộng hơn nhiều. Một cách hình thức, gọi $\mathbf{w} = w_1 \cdots w_m$ là một chuỗi từ[1]. Một hệ thống NLP ở mức chuỗi có thể được phát biểu như một hàm ánh xạ chuỗi đầu vào $\mathbf{w}$ thành một đầu ra $\mathbf{y}$ nào đó. Quá trình này có thể chia thành hai bước riêng biệt: biểu diễn (hay mã hóa) và dự đoán.

- **Biểu diễn** (hay **Mã hóa**). Bước này biến đổi chuỗi đầu vào $\mathbf{w}$ thành một biểu diễn đặc trưng $\mathbf{H}$ bằng một bộ mã hóa $\text{Encode}(\cdot)$:

$$\mathbf{H} = \text{Encode}(\mathbf{w}). \tag{4.1}$$

- **Dự đoán**. Một bộ dự đoán $\text{Predict}(\cdot)$ nhận $\mathbf{H}$ và sinh ra đầu ra:

$$\mathbf{y} = \text{Predict}(\mathbf{H}). \tag{4.2}$$

Một dạng đơn giản của $\mathbf{H}$ là một vectơ đặc trưng (*feature vector*). Chẳng hạn, $\mathbf{H}$ có thể là một tập các đặc trưng chỉ báo do con người thiết kế, được trích từ $\mathbf{w}$ (tạo thành một biểu diễn thưa nhiều chiều), hoặc một tập các số thực biểu thị các đặc trưng ẩn (tạo thành một biểu diễn dày đặc ít chiều). Trong NLP, một dạng phổ biến khác của $\mathbf{H}$ là một chuỗi các vectơ, trong đó mỗi vectơ $\mathbf{h}_i$ tương ứng với một từ đầu vào w_i (xem Hình 4.1). Trong trường hợp này, $\mathbf{h}_i$ có thể được xem là một biểu diễn theo ngữ cảnh của cả w_i lẫn ngữ cảnh xung quanh nó trong $\mathbf{w}$[2]. Sự tương ứng giữa $\mathbf{h}_i$ và w_i cho phép biểu diễn phân biệt được các vị trí khác nhau trong chuỗi và, quan trọng hơn, tự điều chỉnh năng lực mô hình hóa của nó cho các đầu vào có độ dài thay đổi.

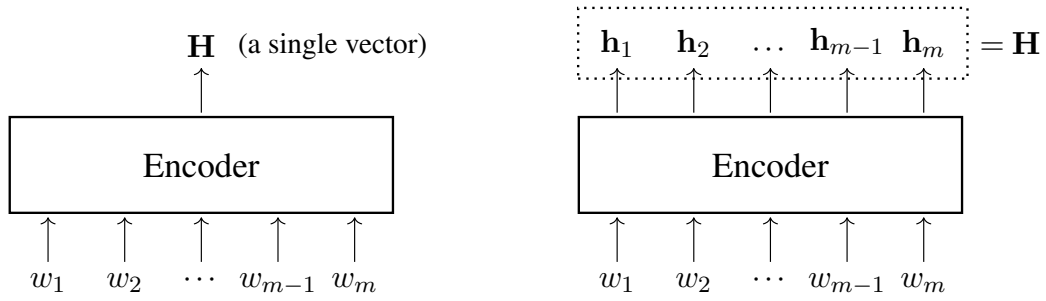
Dạng của $\mathbf{y}$ phụ thuộc vào bài toán ta muốn giải. Ví dụ, trong bài toán phân loại, $\mathbf{y}$ là chỉ số của một lớp (hoặc một phân phối trên các lớp); trong bài toán hồi quy (*regression*), $\mathbf{y}$ là một số thực; còn trong dịch máy (*machine translation*), $\mathbf{y}$ là một chuỗi từ trong ngôn ngữ đích. Lưu ý rằng trong mô hình này, biểu diễn và dự đoán có thể được xem là hai bài toán tách biệt. Một ưu điểm lớn của việc tách rời chúng là cùng một bộ mã hóa có thể được dùng lại cho nhiều ứng dụng với các bộ dự đoán khác nhau. Điều này cũng thúc đẩy một hướng nghiên cứu rất hứa hẹn, trong đó một bộ mã hóa đa dụng được tiền huấn luyện (*pre-training*) trên dữ liệu quy mô lớn rồi được dùng làm thành phần cốt lõi trong nhiều hệ thống ứng dụng phía sau [Peters và cộng sự, 2018; Devlin và cộng sự, 2019].

Chú giải của người dịch: Mã hóa rồi dự đoán: một ví dụ đời thường

Hãy hình dung việc chấm một bài văn. Bước *mã hóa* giống như giáo viên đọc bài và ghi tóm tắt các ý chính vào một phiếu nhận xét (đó là $\mathbf{H}$). Bước *dự đoán* là nhìn vào phiếu đó để cho điểm (đó là $\mathbf{y}$). Cùng một phiếu nhận xét có thể dùng cho nhiều mục đích khác nhau: cho điểm, xếp loại, hay chọn bài đăng báo trường — chỉ cần thay "người dự đoán". Đó chính là lợi ích của việc tách hai bước mà tác giả nói tới. Với Hình 4.1(b),

[1] Mặc dù chúng tôi giới hạn thảo luận ở chuỗi từ, các phương pháp này có thể xử lý chuỗi gồm bất kỳ đơn vị ngôn ngữ nào, chẳng hạn từ con (subword) hay ký tự.

[2] Kiến trúc này có thể mở rộng cho các bộ mã hóa có đầu vào và đầu ra khác độ dài. Ví dụ, đầu vào là $w_1 \cdots w_m$ còn đầu ra là $\mathbf{h}_1 \cdots \mathbf{h}_n$ ($m \neq n$).



(a) Encoding the sequence as a vector (b) Encoding the sequence as a vector sequence

Hình 4.1: Biểu diễn một chuỗi từ dưới dạng (a) một vectơ hoặc (b) một chuỗi các vectơ. (Nhãn trong hình: *Encoder* = bộ mã hóa; *a single vector* = một vectơ duy nhất; (a) mã hóa chuỗi thành một vectơ; (b) mã hóa chuỗi thành một chuỗi vectơ.)

"phiếu nhận xét" chi tiết hơn: mỗi từ w_i có riêng một vectơ $\mathbf{h}_i$ mô tả từ đó *trong bối cảnh cả câu*.
Ký hiệu $\arg\max_y \Pr(y|\mathbf{H})$ nghĩa là "giá trị y làm cho $\Pr(y|\mathbf{H})$ lớn nhất". Ví dụ, nếu $\Pr(\text{Food}|\mathbf{H}) = 0.8$ và $\Pr(\text{Not-food}|\mathbf{H}) = 0.2$ thì $\hat{y} = \text{Food}$.

Có nhiều cách hiện thực Encode$(\cdot)$ và Predict$(\cdot)$. Ví dụ, trong phân loại văn bản, một cách là định nghĩa Encode$(\cdot)$ là một hàm tính vectơ đặc trưng bằng các mẫu (*template*) do con người thiết kế, còn Predict$(\cdot)$ là một bộ phân loại (*classifier*) thống kê truyền thống (ví dụ: SVM hoặc mô hình entropy cực đại). Một cách khác là định nghĩa Encode$(\cdot)$ là một mạng nơ-ron nhiều tầng cho đầu ra là một vectơ số thực, và định nghĩa Predict$(\cdot)$ là một mạng nơ-ron đơn giản chỉ gồm một tầng Softmax. Trong chương này, chúng ta tập trung vào các bộ mã hóa dựa trên mạng nơ-ron. Mục 4.5 sẽ minh họa cách áp dụng các bộ mã hóa này cho nhiều tác vụ NLP khác nhau.

4.2 Các mô hình hồi quy

Nghiên cứu một cách toàn diện các mô hình chuỗi khác nhau không phải việc dễ dàng. Tuy nhiên, sẽ thuận tiện nếu trước tiên ta giới thiệu một trong những mô hình nơ-ron phổ biến và thực dụng nhất, gọi là mạng nơ-ron hồi quy (*recurrent neural networks, RNNs*). Về sau chúng ta sẽ thấy RNN được dùng rộng rãi trong mô hình hóa chuỗi, và các kỹ thuật trình bày ở đây mang tính tổng quát, áp dụng được cho nhiều hệ thống.

4.2.1 Mô hình ngôn ngữ dựa trên RNN

Có lẽ ứng dụng phổ biến nhất của các mô hình chuỗi trong NLP là ước lượng xác suất của một chuỗi từ, gọi là mô hình hóa ngôn ngữ (*language modeling*). Về mặt toán học, mô hình hóa ngôn ngữ là một trường hợp cụ thể của một bài toán cơ bản trong lĩnh vực **quá trình ngẫu nhiên** (*stochastic processes*, hay ***random processes***): mô hình hóa dữ liệu **chuỗi thời gian** (*time series*) [Hamilton, 1994; Chatfield, 2003; Fuller, 2009]. Khi được xem như một chuỗi thời gian, một chuỗi từ tạo thành các điểm dữ liệu rời rạc xuất hiện tại các bước thời gian liên tiếp. Theo nghĩa này, các phương pháp trình bày ở đây có thể áp dụng rộng rãi, mặc dù việc thảo luận đầy đủ về phân tích chuỗi thời gian tổng quát nằm ngoài

phạm vi cuốn sách này.

Cho một chuỗi từ $w_1 \cdots w_m$, mục tiêu của mô hình hóa ngôn ngữ là tính $\Pr(w_1, \ldots, w_m)$. Xác suất đồng thời (*joint probability*) này thường được viết thành tích của các xác suất có điều kiện (*conditional probability*) theo quy tắc chuỗi:

$$\Pr(w_1, \ldots, w_m) \;=\; \Pr(w_1) \cdot \Pr(w_2|w_1) \cdots \Pr(w_m|w_1, \ldots, w_{m-1}). \tag{4.3}$$

Chú giải của người dịch: Quy tắc chuỗi của xác suất

$\Pr(B|A)$ đọc là "xác suất của B *khi biết* A đã xảy ra". Quy tắc nhân quen thuộc $\Pr(A, B) = \Pr(A) \cdot \Pr(B|A)$ áp dụng lặp lại cho cả chuỗi sẽ cho Phương trình (4.3). Ví dụ với câu ba từ "tôi ăn cơm":

$$\Pr(\text{tôi, ăn, cơm}) = \Pr(\text{tôi}) \cdot \Pr(\text{ăn}|\text{tôi}) \cdot \Pr(\text{cơm}|\text{tôi, ăn}).$$

Nếu ba thừa số lần lượt là 0.1, 0.2 và 0.3 thì xác suất của cả câu là $0.1 \times 0.2 \times 0.3 = 0.006$. Như vậy, để tính xác suất của cả câu, ta chỉ cần biết cách *đoán từ tiếp theo khi đã biết các từ trước nó*.

Nói cách khác, bài toán sinh ra $w_1 \cdots w_m$ cũng chính là bài toán mỗi lần sinh ra một từ w_{i+1} dựa trên các từ trước đó $w_1 \cdots w_i$. Các mô hình ngôn ngữ dựa trên RNN biểu diễn $w_1 \cdots w_i$ bằng một đơn vị hồi quy $\text{RNN}(\cdot)$ [Mikolov và cộng sự, 2010]:

$$\mathbf{h}_i \;=\; \text{RNN}(\mathbf{h}_{i-1}, \mathbf{x}_i), \tag{4.4}$$

trong đó $\mathbf{x}_i \in \mathbb{R}^{d_e}$ là vectơ từ (hay vectơ nhúng từ) của w_i. Gọi V là bộ từ vựng (*vocabulary*) mà từ đó ta có thể chọn một từ. Nếu $\mathbf{1}_{w_i}$ là biểu diễn one-hot của w_i thì $\mathbf{x}_i$ được tính bằng cách nhân $\mathbf{1}_{w_i}$ với bảng nhúng từ $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$:

$$\begin{aligned} \mathbf{x}_i &= \text{Embed}(w_i) \\ &= \mathbf{1}_{w_i}\mathbf{C}. \end{aligned} \tag{4.5}$$

Như đã trình bày ở Chương 1, việc dùng $\mathbf{C}$ biến đổi một vectơ $|V|$ chiều (và có lẽ là rất nhiều chiều) thành một vectơ d_e chiều (và có lẽ là ít chiều). Lưu ý rằng $\mathbf{C}$ về bản chất là một bảng tra cứu, với mỗi từ trong V có một mục riêng (tức là một hàng) trong bảng. Vì vậy, vế phải của Phương trình (4.5) là một phép tra cứu, lấy ra hàng tương ứng của $\mathbf{C}$.

Chú giải của người dịch: Vì sao nhân với vectơ one-hot lại là "tra bảng"?

Vectơ one-hot $\mathbf{1}_{w_i}$ có $|V|$ phần tử, toàn số 0 trừ một số 1 ở vị trí của từ w_i. Giả sử bộ từ vựng chỉ có 3 từ và $d_e = 2$:

$$\mathbf{C} = \begin{bmatrix} 0.1 & 0.5 \\ 0.7 & -0.2 \\ 0.3 & 0.9 \end{bmatrix}, \qquad \mathbf{1}_w = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}.$$

Phép nhân (hàng nhân cột) cho $\mathbf{1}_w\mathbf{C} = [\,0 \cdot 0.1 + 1 \cdot 0.7 + 0 \cdot 0.3, \;\; 0 \cdot 0.5 + 1 \cdot (-0.2) + 0 \cdot 0.9\,] = [0.7 \;\; -0.2]$, đúng bằng hàng thứ 2 của $\mathbf{C}$. Vì vậy trong thực tế, máy tính không cần nhân thật mà chỉ việc "lấy hàng số 2 ra" — nhanh hơn nhiều. Ký hiệu $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$ nghĩa là $\mathbf{C}$ là một ma trận số thực có $|V|$ hàng và d_e cột.

Bây giờ ta quay lại Phương trình (4.4). Về mặt trực giác, phương trình này cho thấy trạng thái của phần ngữ cảnh đã thấy tới thời điểm hiện tại (tức $\mathbf{h}_i$) là một biểu diễn nào đó của sự kết hợp giữa đầu vào hiện tại (tức $\mathbf{x}_i$) và trạng thái của phần ngữ cảnh trước đó ($\mathbf{h}_{i-1}$). Nói cách khác, có thể xem đây là quá trình liên tục bổ sung thông tin của một từ mới vào một "bộ nhớ lịch sử". Một điểm tinh tế của quá

trình này là nó có thể được hiện thực dễ dàng bằng cách áp dụng lặp lại Phương trình (4.4) cho tới hết chuỗi.

RNN$(\cdot)$ có thể là bất kỳ hàm nào nhận $\mathbf{h}_{i-1}$ và $\mathbf{x}_i$ rồi sinh ra một vectơ mới $\mathbf{h}_i$. RNN cơ bản (*vanilla RNN*) có dạng

$$\text{RNN}(\mathbf{h}_{i-1}, \mathbf{x}_i) \;=\; \psi(\mathbf{h}_{i-1}\mathbf{U} + \mathbf{x}_i\mathbf{V}), \tag{4.6}$$

trong đó $\psi(\cdot)$ là một hàm kích hoạt (*activation function*), chẳng hạn TanH$(\cdot)$ và Sigmoid$(\cdot)$. Kết hợp với các Phương trình (4.4) và (4.5), ta có thể định nghĩa $\mathbf{h}_i$ như một hàm của $\mathbf{h}_{i-1}$ và w_i

$$\mathbf{h}_i \;=\; \psi(\mathbf{h}_{i-1}\mathbf{U} + \mathbf{1}_{w_i}\mathbf{CV}), \tag{4.7}$$

trong đó $\mathbf{U} \in \mathbb{R}^{d_h \times d_h}$, $\mathbf{V} \in \mathbb{R}^{d_e \times d_h}$ và $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$ là các tham số (*parameter*) học được của mô hình, còn d_h là một siêu tham số (*hyper-parameter*) cho biết số chiều của $\mathbf{h}_i$ và $\mathbf{h}_{i-1}$.

Chú giải của người dịch: RNN như một người đọc có trí nhớ

Hãy tưởng tượng bạn đọc câu "Hôm nay trời mưa to" từng từ một và sau mỗi từ ghi lại "ấn tượng hiện tại" vào một mẩu giấy. Mẩu giấy chính là $\mathbf{h}_i$. Khi đọc từ mới w_i, bạn kết hợp nội dung mẩu giấy cũ ($\mathbf{h}_{i-1}$, nhân với ma trận $\mathbf{U}$) với thông tin của từ mới ($\mathbf{x}_i$, nhân với $\mathbf{V}$), cộng lại, rồi "nén" qua hàm ψ (ví dụ tanh luôn cho giá trị trong khoảng $(-1; 1)$) để được mẩu giấy mới. Điểm then chốt: *cùng* các ma trận $\mathbf{U}, \mathbf{V}$ được dùng ở mọi bước, nên một mô hình có số tham số cố định có thể đọc câu dài bao nhiêu cũng được.
Ví dụ một chiều ($d_h = d_e = 1$), với $U = 0.5$, $V = 1$, $\psi = \tanh$, $h_0 = 0$ và các đầu vào $x_1 = 1$, $x_2 = -1$: $h_1 = \tanh(0.5 \cdot 0 + 1 \cdot 1) = \tanh(1) \approx 0.76$; $h_2 = \tanh(0.5 \cdot 0.76 + 1 \cdot (-1)) = \tanh(-0.62) \approx -0.55$. Giá trị h_2 vẫn "nhớ" một phần x_1 thông qua h_1.

Giờ đây ta đã có một bộ mã hóa biểu diễn chuỗi từ $w_1 \cdots w_m$ dưới dạng một chuỗi các đầu ra của RNN $\mathbf{H} = \{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$. Vì mỗi $\mathbf{h}_i$ mã hóa chuỗi con từ w_1 tới w_i, ta có thể đặt một tầng Softmax lên trên $\mathbf{h}_i$ để thu được một phân phối trên các từ:

$$\mathbf{y}_{i+1} \;=\; \text{Softmax}(\mathbf{h}_i\mathbf{O} + \mathbf{b}), \tag{4.8}$$

trong đó $\mathbf{O} \in \mathbb{R}^{d_h \times |V|}$ và $\mathbf{b} \in \mathbb{R}^{|V|}$. Lấy chỉ số của từ w_{i+1}, ta có

$$\Pr(w_{i+1}|w_1, \ldots, w_i) \;=\; y_{i+1}(w_{i+1}). \tag{4.9}$$

Như vậy, ta đã xây dựng được một mô hình ngôn ngữ sinh ra xác suất $\Pr(w_{i+1}|w_1, \ldots, w_i)$ tại mỗi bước. Hình 4.2 minh họa mô hình ngôn ngữ dựa trên RNN cho một chuỗi ví dụ. Để chạy mô hình này trên một chuỗi từ, tất nhiên ta muốn bắt đầu bằng việc dự đoán w_1, nhưng điều này đòi hỏi một từ đứng trước w_0 làm đầu vào. Một cách đơn giản và áp dụng rộng rãi để tạo trạng thái khởi đầu phù hợp cho RNN là thêm một ký hiệu bắt đầu $\langle\text{SOS}\rangle$ vào đầu chuỗi, để mọi chuỗi đều bắt đầu bằng cùng một "từ". Tương tự, ta có thể gắn một ký hiệu kết thúc $\langle\text{EOS}\rangle$ vào cuối chuỗi để mô hình hóa tính trọn vẹn của chuỗi. Điều này dẫn tới một dạng mới của xác suất của chuỗi

$$\begin{aligned}\Pr(\langle\text{SOS}\rangle, w_1, \ldots, w_m, \langle\text{EOS}\rangle) \;=\; &\Pr(\langle\text{SOS}\rangle)\cdot\\ &\Pr(w_1|\langle\text{SOS}\rangle)\cdot\\ &\Pr(w_2|\langle\text{SOS}\rangle, w_1)\cdot\\ &\ldots\\ &\Pr(w_m|\langle\text{SOS}\rangle, w_1, \ldots, w_{m-1})\cdot\\ &\Pr(\langle\text{EOS}\rangle|\langle\text{SOS}\rangle, w_1, \ldots, w_m).\end{aligned} \tag{4.10}$$

Ta có thể đơn giản giả sử $\Pr(\langle \text{SOS} \rangle) = 1$. Để tính $\Pr(\langle \text{SOS} \rangle, w_1, \ldots, w_m, \langle \text{EOS} \rangle)$, ta lấy $\langle \text{SOS} \rangle\, w_1 \cdots w_m$ làm chuỗi đầu vào và $w_1 \cdots w_m\, \langle \text{EOS} \rangle$ làm chuỗi đầu ra.

Chú giải của người dịch: Softmax và ký hiệu đầu/cuối câu

Tầng Softmax biến một vectơ điểm số bất kỳ thành một phân phối xác suất (các số dương, tổng bằng 1). Vectơ $\mathbf{y}_{i+1}$ có $|V|$ phần tử, mỗi phần tử là xác suất để *một từ* trong bộ từ vựng là từ tiếp theo; $y_{i+1}(w_{i+1})$ là phần tử ứng với đúng từ w_{i+1}. Ký hiệu $\langle \text{SOS} \rangle$ (*start of sequence*) và $\langle \text{EOS} \rangle$ (*end of sequence*) giống như "tiếng còi bắt đầu" và "dấu chấm hết": nhờ $\langle \text{EOS} \rangle$, mô hình học được cả việc *khi nào nên dừng* câu. Ví dụ, câu "tôi ăn cơm" cho đầu vào $\langle \text{SOS} \rangle$ tôi ăn cơm và đầu ra cần đoán là tôi ăn cơm $\langle \text{EOS} \rangle$ (lệch nhau một vị trí).

4.2.2 Huấn luyện

Là một kiến trúc nơ-ron khả vi, mô hình ngôn ngữ dựa trên RNN có thể được huấn luyện (*training*) bằng các kỹ thuật tối ưu hóa chuẩn. Những nguyên lý nền tảng của việc huấn luyện mạng nơ-ron đã được thảo luận kỹ ở Chương 2. Vì vậy, thay vì mô tả đầy đủ ở đây, chúng tôi chỉ phác họa các khái niệm cơ bản và thảo luận một số cải tiến dành cho các mô hình chuỗi.

Mô hình hóa ngôn ngữ dựa trên RNN có thể được phát biểu thành một bài toán dự đoán bước tiếp theo. Giả sử ta có một kho ngữ liệu (*corpus*) $\mathcal{D}$ gồm các chuỗi từ. Với mỗi chuỗi $\mathbf{w} = w_1, \ldots, w_{|\mathbf{w}|}$ trong $\mathcal{D}$, ta xây dựng một chuỗi các cặp đầu vào–đích như sau[3]:

$$\{(w_1, w_2), (w_2, w_3), \ldots, (w_{|\mathbf{w}|-1}, w_{|\mathbf{w}|})\}.$$

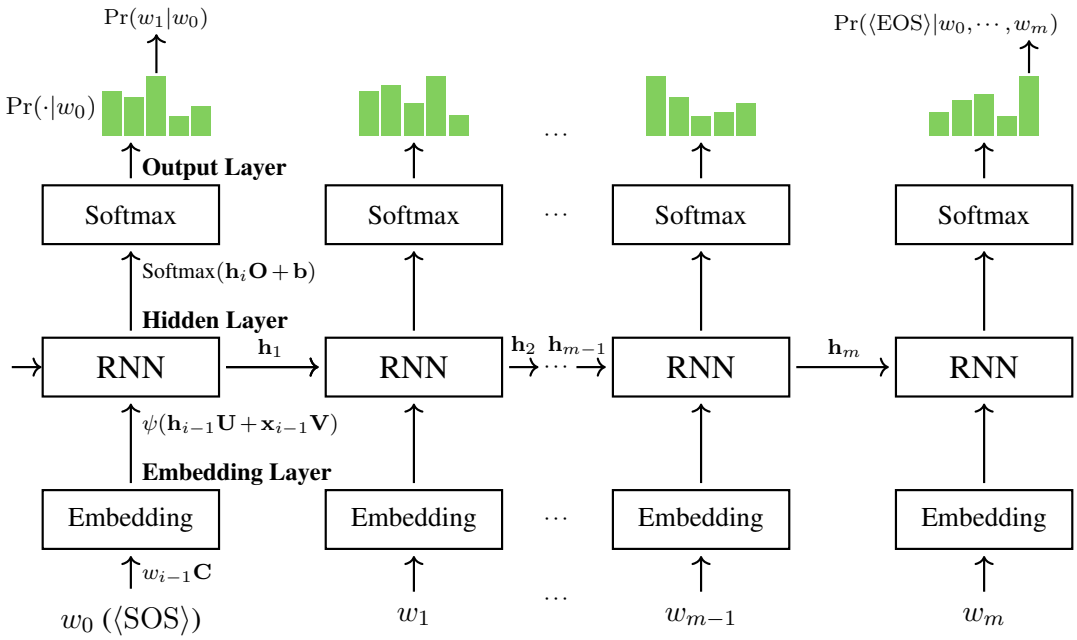
Mô hình ngôn ngữ xử lý chuỗi tiền tố đầu vào $w_1, \ldots, w_{|\mathbf{w}|-1}$ và đưa ra một chuỗi tương ứng các phân phối xác suất dự đoán $\hat{\mathbf{y}}_2, \ldots, \hat{\mathbf{y}}_{|\mathbf{w}|}$. Xem bảng sau để thấy minh họa đầu vào và đầu ra của mô hình.

Bước	Lịch sử	Đầu vào	Đầu ra (phân phối)	Đáp án chuẩn
1		w_1	$\hat{\mathbf{y}}_2$	w_2
2	w_1	w_2	$\hat{\mathbf{y}}_3$	w_3
3	w_1, w_2	w_3	$\hat{\mathbf{y}}_4$	w_4
...	...	...	...	...
$\lvert\mathbf{w}\rvert - 2$	$w_1, w_2, \ldots, w_{\lvert\mathbf{w}\rvert-3}$	$w_{\lvert\mathbf{w}\rvert-2}$	$\hat{\mathbf{y}}_{\lvert\mathbf{w}\rvert-1}$	$w_{\lvert\mathbf{w}\rvert-1}$
$\lvert\mathbf{w}\rvert - 1$	$w_1, w_2, \ldots, w_{\lvert\mathbf{w}\rvert-3}, w_{\lvert\mathbf{w}\rvert-2}$	$w_{\lvert\mathbf{w}\rvert-1}$	$\hat{\mathbf{y}}_{\lvert\mathbf{w}\rvert}$	$w_{\lvert\mathbf{w}\rvert}$

Một hàm mất mát (*loss function*) $\mathcal{L}(\hat{\mathbf{y}}_i, \mathbf{1}_{w_i})$ được định nghĩa để đo mức chênh lệch giữa phân phối dự đoán $\hat{\mathbf{y}}_i$ và biểu diễn one-hot của đích thực $\mathbf{1}_{w_i}$. Lựa chọn tiêu chuẩn cho tác vụ này là hàm mất mát entropy chéo (*cross-entropy*), đo độ sai khác giữa hai phân phối [Mitchell, 1997; Bishop, 2006]. Khi đó, mất mát trên toàn bộ tập dữ liệu được định nghĩa là:

$$\mathcal{L} = \sum_{\mathbf{w} \in \mathcal{D}} \sum_{i=2}^{|\mathbf{w}|} \mathcal{L}(\hat{\mathbf{y}}_i, \mathbf{1}_{w_i}). \tag{4.11}$$

[3] Mặc dù các token biên $\langle \text{SOS} \rangle$ và $\langle \text{EOS} \rangle$ là thiết yếu trong các hiện thực thực tế, ở đây chúng tôi lược bỏ chúng khỏi ký hiệu cho đơn giản.



Hình 4.2: Minh họa việc dùng mô hình ngôn ngữ dựa trên RNN để tính $\Pr(\langle\text{SOS}\rangle w_1 \cdots w_m \langle\text{EOS}\rangle)$. Đầu vào là $\langle\text{SOS}\rangle w_1 \cdots w_m$, còn đầu ra là xác suất $\Pr(w_1|\langle\text{SOS}\rangle)\Pr(w_2|\langle\text{SOS}\rangle\, w_1)\cdots\Pr(\langle\text{EOS}\rangle|\langle\text{SOS}\rangle\, w_1 \cdots w_m)$. Vì $\Pr(\langle\text{SOS}\rangle) = 1$, xác suất sinh ra chuỗi đơn giản là $\Pr(\langle\text{SOS}\rangle w_1 \cdots w_m \langle\text{EOS}\rangle) = \Pr(\langle\text{SOS}\rangle)\Pr(w_1|\langle\text{SOS}\rangle)\Pr(w_2|\langle\text{SOS}\rangle\, w_1)\cdots\Pr(\langle\text{EOS}\rangle|\langle\text{SOS}\rangle w_1 \cdots w_m)$. Với mỗi đầu vào w_i, trước tiên ta biểu diễn nó thành một vectơ từ $\mathbf{x}_i$ qua tầng nhúng, thu được một chuỗi vectơ từ $\mathbf{x}_0 \cdots \mathbf{x}_m$. Tầng RNN ánh xạ $\mathbf{x}_0 \cdots \mathbf{x}_m$ thành một chuỗi các trạng thái ẩn (*hidden state*) $\mathbf{h}_1 \cdots \mathbf{h}_{m+1}$. Trong quá trình này, phép toán được áp dụng một cách đệ quy: một đơn vị RNN nhận cả $\mathbf{h}_{i-1}$ và $\mathbf{x}_{i-1}$ rồi sinh ra trạng thái mới $\mathbf{h}_i$. Bên trên đó, ta dùng tầng đầu ra (Softmax) để thu được $\Pr(w_{i+1}|\langle\text{SOS}\rangle\, w_1 \cdots w_i)$. (Nhãn trong hình: *Embedding Layer* = tầng nhúng; *Hidden Layer* = tầng ẩn; *Output Layer* = tầng đầu ra.)

Chú giải của người dịch: Entropy chéo với vectơ one-hot

Khi đích là vectơ one-hot, entropy chéo rút gọn thành $\mathcal{L}(\hat{\mathbf{y}}_i, \mathbf{1}_{w_i}) = -\log \hat{y}_i(w_i)$, tức là "trừ logarit của xác suất mà mô hình gán cho từ đúng". Nếu mô hình gán xác suất 0.9 cho từ đúng thì mất mát là $-\ln 0.9 \approx 0.105$ (nhỏ, tốt); nếu chỉ gán 0.01 thì mất mát là $-\ln 0.01 \approx 4.6$ (lớn, tệ). Phương trình (4.11) chỉ đơn giản cộng các mất mát này qua mọi vị trí i của mọi câu $\mathbf{w}$ trong kho ngữ liệu. Tổng bắt đầu từ $i = 2$ vì từ đầu tiên w_1 không được dự đoán (xem bảng trên).

Khi đã biết mất mát, việc huấn luyện mô hình ngôn ngữ dựa trên RNN có thể thực hiện bằng hạ gradient (*gradient descent*). Một dạng đơn giản của phương pháp này là quy tắc delta (*delta rule*)

$$\theta_{\text{new}} \;=\; \theta_{\text{old}} - \eta \cdot \frac{\partial \mathcal{L}}{\partial \theta}, \tag{4.12}$$

trong đó θ ký hiệu các tham số. Với mô hình mô tả ở Mục 4.2.1, θ gồm $\mathbf{C}$, $\mathbf{U}$, $\mathbf{V}$, $\mathbf{O}$ và $\mathbf{b}$. $\frac{\partial \mathcal{L}}{\partial \theta}$ là đạo hàm của mất mát theo các tham số, gọi là **gradient lỗi** (*error gradient*).

Phương trình (4.12) có thể được hiểu là quá trình dịch chuyển các tham số hiện tại một bước nhỏ theo hướng dốc xuống nhanh nhất (tức là hướng của $-\frac{\partial \mathcal{L}}{\partial \theta}$). Ở đây η biểu thị ta đi xa bao nhiêu trong mỗi bước xuống dốc, còn được gọi là tốc độ học (*learning rate*). Để tính $\frac{\partial \mathcal{L}}{\partial \theta}$ thường cần một thuật toán lan truyền ngược (*back-propagation*) truyền lỗi ngược qua các bước thời gian đã được trải ra. Trong các hiện thực hiện đại của hệ thống học sâu (*deep learning*), nơi mạng nơ-ron được biểu diễn dưới dạng đồ thị tính toán (*computation graph*), lan truyền ngược trở nên đơn giản vì nó chỉ là sản phẩm phụ của việc duyệt đồ thị, và có nhiều bộ công cụ vi phân tự động làm việc này. Các thuật toán tương tự, gọi là **lan truyền ngược theo thời gian** (*back-propagation through time, BPTT*), cũng đã được dùng trong các hệ thống trước đây [Werbos, 1990]. Để tìm hiểu thêm về huấn luyện mạng nơ-ron, xem Chương 2 và/hoặc các giáo trình về chủ đề này [Goodfellow và cộng sự, 2016; Zhang và cộng sự, 2021a].

Chú giải của người dịch: Hạ gradient: xuống núi trong sương mù

Hãy hình dung $\mathcal{L}$ là độ cao của một ngọn đồi, còn θ là vị trí của bạn. Bạn muốn xuống thung lũng (mất mát nhỏ nhất) nhưng sương mù dày đặc, chỉ cảm nhận được độ dốc ngay dưới chân. Đạo hàm $\frac{\partial \mathcal{L}}{\partial \theta}$ cho biết độ dốc; đi ngược dấu đạo hàm là đi xuống. Ví dụ một tham số: $\mathcal{L}(\theta) = \theta^2$, $\theta_{\text{old}} = 3$, $\eta = 0.1$. Đạo hàm là $2\theta = 6$, nên $\theta_{\text{new}} = 3 - 0.1 \cdot 6 = 2.4$, gần cực tiểu $\theta = 0$ hơn. Khi có nhiều tham số, $\frac{\partial \mathcal{L}}{\partial \theta}$ là tập hợp các *đạo hàm riêng* theo từng tham số (giữ các tham số khác cố định), gọi chung là gradient. "Trải ra theo thời gian" nghĩa là vẽ RNN thành chuỗi các bản sao nối tiếp nhau như Hình 4.2, rồi lan truyền ngược qua chuỗi đó.

Nếu đầu vào là một chuỗi dài, việc áp dụng RNN sẽ tạo ra một mạng nơ-ron rất sâu. Khi đó, việc dùng quy tắc chuỗi cho đạo hàm có thứ tự khiến các đạo hàm của mất mát rất lớn hoặc rất nhỏ bị tích lũy lại, và do đó bước cập nhật tham số trong Phương trình (4.12) trở nên rất lớn hoặc rất nhỏ. Hiện tượng này thường được gọi là **bài toán bùng nổ gradient và tiêu biến gradient** (*exploding and vanishing gradient problems*). Có một số phương pháp để giảm nhẹ các vấn đề này cho RNN [Sutskever, 2013]. Trong đó có:

- **Chính quy hóa** (*Regularization*). Đưa các số hạng chính quy hóa (như chuẩn l_1 và l_2 trên các ma trận tham số) vào quá trình huấn luyện có thể tránh được các mô hình mà phần lớn tham số có giá trị lớn, và nhờ vậy giúp tránh bùng nổ gradient. Tương tự, ta có thể phạt các trường hợp chuẩn của gradient quá nhỏ [Pascanu và cộng sự, 2013].

- **Cắt gradient** (*Gradient Clipping*). Khi chuẩn của gradient quá lớn, một cách tự nhiên là trực tiếp thu nhỏ độ lớn của chúng. Một phương pháp đơn giản là cắt chuẩn gradient theo một ngưỡng τ.

Nếu chuẩn $||\frac{\partial \mathcal{L}}{\partial \theta}||$ lớn hơn τ, ta có thể co giãn lại $\frac{\partial \mathcal{L}}{\partial \theta}$ cho phù hợp, ví dụ $\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\tau}{||\frac{\partial \mathcal{L}}{\partial \theta}||} \cdot \frac{\partial \mathcal{L}}{\partial \theta}$[4].

- **Lan truyền ngược bị cắt ngắn** (*Truncated Back-propagation*). Một ý tưởng khác là chia một chuỗi dài các cặp đầu vào–đầu ra thành các đoạn ngắn hơn, rồi huấn luyện RNN trên các chuỗi con riêng biệt này [Williams và Peng, 1990; Elman, 1990]. Cách này giảm cả chi phí huấn luyện lẫn nguy cơ gradient lỗi tích lũy có giá trị quá lớn hoặc quá nhỏ.

- **Cải tiến kiến trúc** (*Improved Architectures*). Cũng có thể thiết kế lại mô hình để vượt qua các giới hạn của RNN tiêu chuẩn, thường bằng cơ chế bộ nhớ. Ở Mục 4.3, ta sẽ thấy một vài ví dụ thiết kế lại đơn vị hồi quy để giải quyết vấn đề tiêu biến gradient.

- **Khởi tạo và ràng buộc tham số** (*Initialization and Constraints of Parameters*). Khởi tạo các tham số của mô hình trong một vùng thích hợp nói chung có ích cho việc tối ưu hóa, và đôi khi giúp ngăn gradient quá nhỏ. Một phương pháp khác là đặt mô hình một cách ngẫu nhiên và chỉ học tham số của các tầng đầu ra [Jaeger và Haas, 2004].

- **Hàm kích hoạt không bão hòa** (*Non-saturating Activations*). Nhiều hàm kích hoạt phổ biến có miền giá trị đầu ra bị chặn, ví dụ hàm Sigmoid có miền giá trị $[0, 1]$. Chúng còn được gọi là **hàm kích hoạt bão hòa** (*saturating activation functions*)[5]. Việc dùng hàm kích hoạt bão hòa thường dẫn tới gradient suy giảm dần qua các tầng, tức là tiêu biến gradient. Do đó, dùng hàm kích hoạt không bão hòa thay thế, ví dụ hàm ReLU, là một hướng đầy hứa hẹn.

- **Chuẩn hóa giá trị kích hoạt** (*Normalization of Activations*). Hàm kích hoạt bão hòa còn có thể khiến mô hình bị mắc kẹt trong một vùng bão hòa của đầu ra, và ta cần tốc độ học lớn để thoát khỏi các cực trị địa phương [Ioffe và Szegedy, 2015]. Khi đó, quá trình huấn luyện sẽ không ổn định, và những thay đổi nhỏ ở đầu vào và/hoặc tham số mô hình có thể dẫn tới biến động lớn trong hành vi của mô hình. Một giải pháp khả dĩ là chuẩn hóa các giá trị kích hoạt để giảm phương sai, ví dụ trừ đi trung bình của các giá trị kích hoạt trong một nhóm mẫu (chẳng hạn các mẫu trong một lô nhỏ (*mini-batch*) huấn luyện) rồi chia cho độ lệch chuẩn của chúng.

Chú giải của người dịch: Vì sao gradient "bùng nổ" hay "tiêu biến"?

Theo quy tắc chuỗi, gradient truyền ngược qua n bước của RNN gần giống một *tích* của n thừa số (mỗi bước một thừa số). Tích của nhiều số lặp lại rất nhạy: $0.9^{50} \approx 0.005$ (gần như biến mất), còn $1.1^{50} \approx 117$ (phình to). Vì thế với câu dài, tín hiệu học từ các từ ở xa hoặc gần như mất hẳn (tiêu biến), hoặc lớn tới mức làm tham số nhảy loạn (bùng nổ).
Chuẩn $||\mathbf{g}||$ của một vectơ là độ dài của nó, như độ dài vectơ ở lớp 10: $||(3,4)|| = \sqrt{3^2+4^2} = 5$. Cắt gradient với $\tau = 1$ sẽ biến gradient $(3,4)$ thành $\frac{1}{5} \cdot (3,4) = (0.6, 0.8)$: giữ nguyên hướng nhưng độ dài chỉ còn 1. Phương trình (4.13) dùng max để gradient có chuẩn nhỏ hơn τ được giữ nguyên. Hàm bão hòa như Sigmoid "phẳng lì" ở hai đầu nên đạo hàm ở đó gần 0; còn $\text{ReLU}(x) = \max(0, x)$ có đạo hàm bằng 1 với mọi $x > 0$.

[4] Nó thường được phát biểu dưới dạng phương trình

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\tau}{\max(\tau, ||\frac{\partial \mathcal{L}}{\partial \theta}||)} \cdot \frac{\partial \mathcal{L}}{\partial \theta}. \tag{4.13}$$

[5] Một hàm kích hoạt $f(x)$ là không bão hòa khi và chỉ khi khi $x \to \infty$ (hoặc $-\infty$) thì $f(x) \to \infty$. Một hàm kích hoạt là bão hòa nếu nó không phải hàm kích hoạt không bão hòa.

4.2.3 Xếp chồng tầng

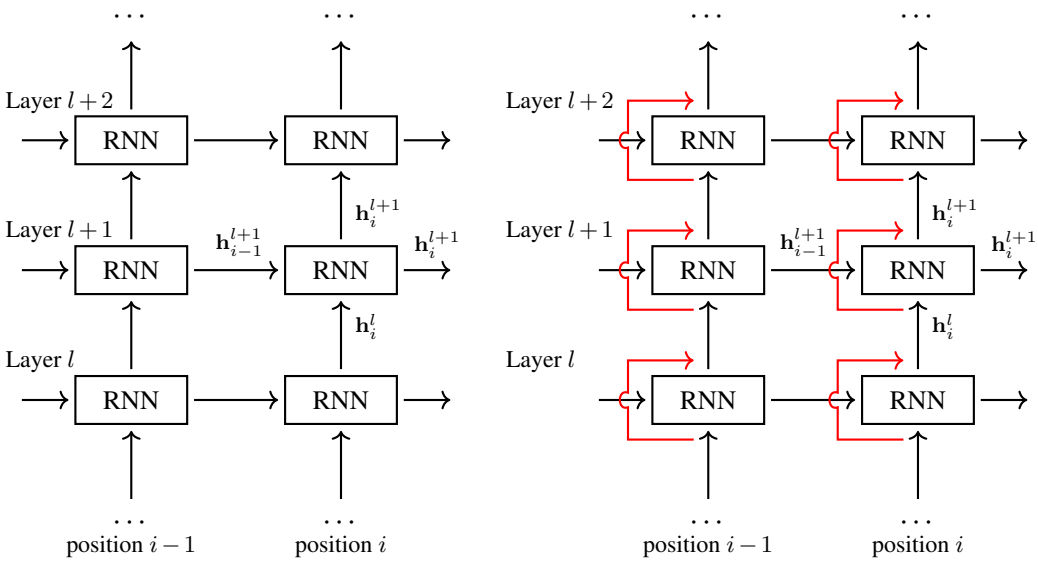
Nếu xem việc áp dụng một đơn vị hồi quy như một hàm ánh xạ một chuỗi biến thành một chuỗi biến mới có cùng độ dài, thì việc hợp hàm này với một hàm khác cùng loại, hoặc thậm chí với chính nó, là điều tự nhiên. Nhờ vậy, ta có thể dễ dàng mở rộng RNN thành mạng nơ-ron sâu chỉ bằng cách xếp chồng các tầng hồi quy.

Gọi $\mathbf{h}_i^l$ là đầu ra của đơn vị hồi quy thứ l trong chồng tại vị trí i. Ta có thể áp dụng một đơn vị hồi quy mới lên $\mathbf{h}_i^l$, thu được một đầu ra mới ở mức $l+1$

$$\mathbf{h}_i^{l+1} = \text{RNN}(\mathbf{h}_{i-1}^{l+1}, \mathbf{h}_i^l), \tag{4.14}$$

trong đó $\mathbf{h}_{i-1}^{l+1}$ là đầu ra của bước trước ở mức $l+1$. Để Phương trình (4.14) được xác định đầy đủ, ta thường định nghĩa $\mathbf{h}_i^0 = \mathbf{x}_i$. Nói cách khác, chồng tầng bắt đầu từ vectơ từ $\mathbf{x}_i$, sau đó là một loạt đầu ra RNN (tức là $\mathbf{h}_i^1$, $\mathbf{h}_i^2$, $\mathbf{h}_i^3$, v.v.).

Để minh họa, Hình 4.3(a) cho thấy một RNN xếp chồng dùng cho mô hình hóa ngôn ngữ. Ta thấy rằng áp dụng một chồng các đơn vị hồi quy tương đương với việc tạo đồng thời nhiều tầng RNN. Tuy nhiên, có thể gây nhầm lẫn nếu ta gọi một mạng hồi quy đã trải ra là một *tầng*, vì thuật ngữ *tầng* thường chỉ một tập các nơ-ron nhận cùng đầu vào trong một mạng nơ-ron truyền thẳng. Ở đây ta mở rộng thuật ngữ *tầng* cho một khái niệm tổng quát hơn: một nhóm nơ-ron được đặt ở cùng một mức về mặt cấu trúc liên kết. Vì vậy, ta nói mô hình ngôn ngữ trong Hình 4.3 có 3 tầng RNN.



(a) A 3-layer RNN. (b) A 3-layer RNN with residual connections.

Hình 4.3: RNN 3 tầng (có và không có kết nối tắt). Để xếp chồng các tầng RNN, ta đưa đầu ra của tầng l vào tầng $l+1$. Do đó đầu ra của tầng $l+1$ là $\mathbf{h}_i^{l+1} = \text{RNN}(\mathbf{h}_{i-1}^{l+1}, \mathbf{h}_i^l)$. Các đường màu đỏ biểu diễn kết nối tắt, cộng trực tiếp đầu vào của một tầng vào đầu ra của nó, cho ta $\mathbf{h}_i^{l+1} = \text{RNN}(\mathbf{h}_{i-1}^{l+1}, \mathbf{h}_i^l) + \mathbf{h}_i^l$. (Nhãn trong hình: *Layer* = tầng; *position* = vị trí; (a) RNN 3 tầng; (b) RNN 3 tầng có kết nối tắt.)

Khi xếp chồng nhiều tầng RNN, ta xây dựng được một mô hình sâu hơn nhưng khó huấn luyện hơn. Khó khăn này một phần đến từ trở ngại trong việc truyền thông tin qua RNN nhiều tầng. Để việc huấn luyện dễ dàng hơn, một cách tiếp cận phổ biến là đưa **kết nối nhảy** (*skip connections*) hay **kết nối tắt** (*residual connections*) vào mạng nơ-ron nhiều tầng [He và cộng sự, 2016a]. Các kết nối này nhằm tận

dụng một đường đi bổ sung để thông tin có thể bỏ qua một số tầng. Như đã mô tả ở Chương 2, dạng của một mạng nơ-ron residual được cho bởi:

$$\mathbf{y}^{l+1} = F(\mathbf{y}^l) + \mathbf{y}^l, \tag{4.15}$$

trong đó $\mathbf{y}^l$ là đầu ra của tầng l. Mở rộng công thức này cho Phương trình (4.14) ta được RNN nhiều tầng có kết nối tắt:

$$\mathbf{h}_i^{l+1} = \text{RNN}(\mathbf{h}_{i-1}^{l+1}, \mathbf{h}_i^l) + \mathbf{h}_i^l. \tag{4.16}$$

Điểm khác biệt duy nhất so với Phương trình (4.14) là ta thêm ánh xạ đồng nhất của $\mathbf{h}_i^l$ vào vế phải của Phương trình (4.16). Nhờ đó, đầu vào $\mathbf{h}_i^l$ có thể được tầng $l+1$ truy cập trực tiếp. Điều này đơn giản hóa rất nhiều cách thông tin lưu chuyển qua mạng nơ-ron, và cho phép hệ thống "bỏ qua" các tầng khi lan truyền lỗi. Hình 4.3(b) cho thấy một RNN 3 tầng có kết nối tắt.

Chú giải của người dịch: Chỉ số trên, chỉ số dưới và kết nối tắt

Trong $\mathbf{h}_i^l$, chỉ số dưới i là *vị trí* trong câu (từ thứ mấy), chỉ số trên l là *tầng* (tầng thứ mấy) — không phải số mũ. Ở Hình 4.3, i chạy theo chiều ngang, l chạy theo chiều dọc. Kết nối tắt giống như một "đường cao tốc" song song với con đường quanh co qua hàm RNN: nếu tầng $l+1$ chưa học được gì hữu ích, đầu ra vẫn ít nhất bằng $\mathbf{h}_i^l$, và gradient cũng có thể đi thẳng ngược xuống các tầng dưới qua phép cộng mà không bị thu nhỏ.

4.2.4 Các mô hình hai chiều

Việc dùng RNN cho phép ta phát biểu bài toán mã hóa một chuỗi từ thành bài toán sinh các từ từ trái sang phải. Một ưu điểm của cách tiếp cận này là việc mô hình hóa thông tin ngữ cảnh nảy sinh một cách tự nhiên: đầu ra của một đơn vị RNN theo một cách nào đó mô tả các từ ngữ cảnh tính tới thời điểm đó. Đặc điểm này khiến việc mô hình hóa phân phối xác suất $\Pr(w_{i+1}|w_1, \ldots, w_i)$ trở nên rất trực tiếp, vì ta có thể dùng $\mathbf{h}_i$ làm biểu diễn của ngữ cảnh $w_1 \cdots w_i$, tức là $\Pr(w_{i+1}|w_1, \ldots, w_i) = \Pr(w_{i+1}|\mathbf{h}_i)$.

Việc sinh từ trái sang phải được dùng rộng rãi trong sinh chuỗi, chẳng hạn dịch máy. Nó có thể được xem là một trường hợp của **quá trình tự hồi quy** (*autoregressive processes, AR processes*), trong đó trạng thái của một biến phụ thuộc vào trạng thái của các biến trước nó [Chatfield, 2003; Box và cộng sự, 2015][6]. Tuy nhiên, phương pháp này không phải lựa chọn duy nhất để mô hình hóa chuỗi. Thậm chí ta không nhất thiết phải giới hạn ở mô hình hóa ngôn ngữ khi huấn luyện một bộ mã hóa chuỗi. Điều này dẫn tới một câu hỏi thú vị: làm thế nào để xây dựng một bộ mã hóa chuỗi từ mà không cần giả định gì về bộ dự đoán? Trả lời câu hỏi này dẫn ta tới việc tách việc học bộ mã hóa văn bản khỏi một tác vụ

[6]Là một quá trình ngẫu nhiên, quá trình tự hồi quy biểu diễn một biến tại thời điểm t bằng cách liên hệ nó với các giá trị trong quá khứ của quá trình và giá trị hiện tại của một quá trình sai số [Chatfield, 2003]. Một cách hình thức, một chuỗi thời gian $\{z_1, \ldots, z_T\}$ mô tả một quá trình tự hồi quy bậc p nếu với mọi $t \in \{p+1, \ldots, T\}$

$$z_t = \sum_{i=1}^{p} \alpha_i z_{t-i} + \epsilon_t, \tag{4.17}$$

trong đó $\{\alpha_1, \ldots, \alpha_p\}$ là các tham số của quá trình, và ϵ_t là sai số tại thời điểm t. Quá trình này được gọi là *hồi quy* (*regressive*) vì nó có cùng dạng với **mô hình hồi quy tuyến tính bội** (*multiple linear regression model*). Tiền tố *tự* (*auto-*) xuất phát từ cách ta hồi quy z_t: z_t phụ thuộc vào các giá trị quá khứ của chính nó thay vì vào các biến độc lập bổ sung. Một cách diễn giải mô hình hóa ngôn ngữ dưới góc nhìn quá trình tự hồi quy là coi $\{z_1, \ldots, z_T\}$ như biểu diễn của một chuỗi từ $\{w_1, \ldots, w_T\}$. Nhờ vậy, ta có thể hình dung được phần nào việc dự đoán w_t từ các từ trước đó $\{w_1, \ldots, w_{t-1}\}$ bằng cách xét tính chất tự hồi quy của bài toán. Tuy nhiên, cần lưu ý rằng phần lớn các mô hình sinh chuỗi dùng trong NLP không tương đương về mặt toán học với Phương trình (4.17), mặc dù chúng thường được gọi là các mô hình hồi quy. Ví dụ, mô hình ngôn ngữ dựa trên RNN bàn ở đây không phải là mô hình tuyến tính. Thay vào đó, nó dùng nhiều tầng phi tuyến để mô tả các quan hệ phức tạp giữa các từ.

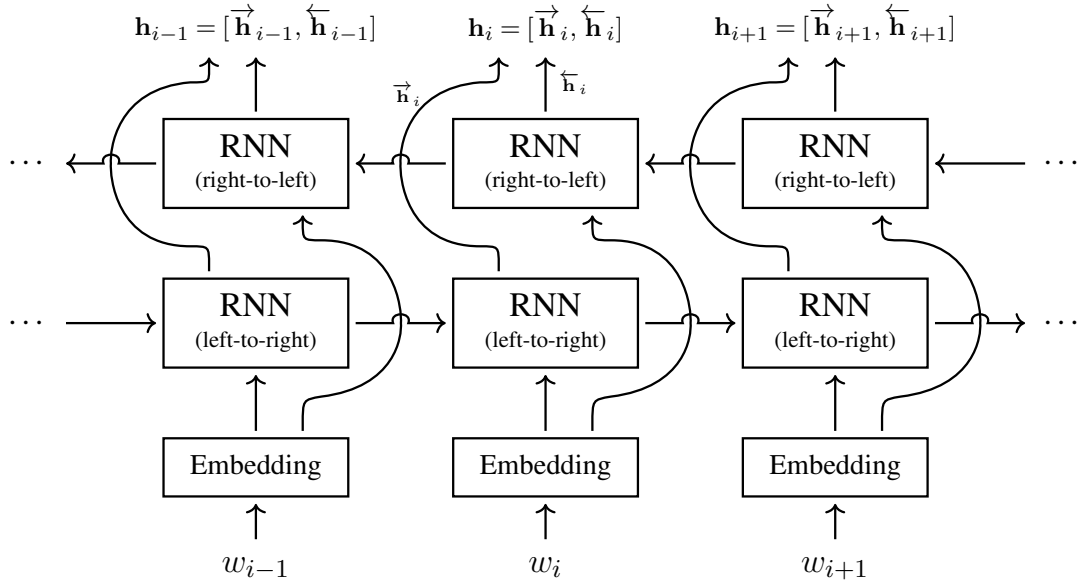
NLP cụ thể, và coi nó như một tác vụ riêng mà kết quả có thể áp dụng cho nhiều hệ thống khác. Thảo luận chi tiết hơn không phải trọng tâm ở đây và chúng tôi để dành cho các chương sau.

Chú giải của người dịch: Tự hồi quy: một ví dụ bằng số

Phương trình (4.17) với $p = 2, \alpha_1 = 0.5, \alpha_2 = 0.3$ cho $z_t = 0.5z_{t-1} + 0.3z_{t-2} + \epsilon_t$. Nếu $z_1 = 10, z_2 = 20$ và bỏ qua sai số thì $z_3 = 0.5 \cdot 20 + 0.3 \cdot 10 = 13$, $z_4 = 0.5 \cdot 13 + 0.3 \cdot 20 = 12.5$, … Mỗi giá trị mới được "đoán" từ chính các giá trị trước đó của chuỗi — giống như đoán nhiệt độ ngày mai từ nhiệt độ hai hôm trước. Mô hình ngôn ngữ cũng "đoán từ tiếp theo từ các từ đã có", nhưng bằng một hàm phi tuyến phức tạp hơn nhiều thay vì một tổng có trọng số đơn giản.

Bây giờ ta trình bày một mở rộng đơn giản của mô hình chuỗi từ trái sang phải bằng cách quay lại với RNN. Lưu ý rằng trong mô hình hóa chuỗi, điều ta mong muốn là một biểu diễn nào đó của toàn bộ chuỗi. Một vấn đề của RNN thông thường là chúng là **mô hình một chiều** (*uni-directional models*), trong đó các từ ngữ cảnh đứng sau w_i bị bỏ qua. Để xét cả ngữ cảnh bên trái lẫn bên phải của một từ, ta có thể dùng **mô hình hai chiều** (*bi-directional models*). Hình 4.4 cho thấy một ví dụ về RNN hai chiều. Có hai mô hình con: một RNN từ trái sang phải và một RNN từ phải sang trái. Chúng có cùng kiến trúc nhưng hoạt động theo hai hướng ngược nhau. Với mỗi từ đầu vào w_i, RNN từ trái sang phải cho ra một vectơ biểu diễn ngữ cảnh $\{w_1, \ldots, w_i\}$ (ký hiệu là $\overrightarrow{\mathbf{h}}_i$), còn RNN từ phải sang trái cho ra một vectơ biểu diễn ngữ cảnh $\{w_i, \ldots, w_m\}$ (ký hiệu là $\overleftarrow{\mathbf{h}}_i$). Ta có thể ghép nối $\overrightarrow{\mathbf{h}}_i$ và $\overleftarrow{\mathbf{h}}_i$ để thu được một biểu diễn hai chiều

$$\mathbf{h}_i = [\overrightarrow{\mathbf{h}}_i, \overleftarrow{\mathbf{h}}_i]. \tag{4.18}$$



Hình 4.4: Một mô hình RNN hai chiều. Cho một chuỗi từ, ta chạy một RNN từ trái sang phải và một RNN khác từ phải sang trái. Do đó, tại mỗi vị trí ta thu được một biểu diễn từ trái sang phải và một biểu diễn từ phải sang trái. Đầu ra là phép ghép nối hai biểu diễn này, nhờ vậy nó bao hàm cả ngữ cảnh bên trái lẫn bên phải. (Nhãn trong hình: *left-to-right* = từ trái sang phải; *right-to-left* = từ phải sang trái; *Embedding* = tầng nhúng.)

Như vậy, RNN hai chiều có đầu ra cùng dạng với RNN một chiều, tức là một chuỗi vectơ $\{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$. Khác với RNN một chiều, biểu diễn $\mathbf{h}_i$ ở đây mô tả ngữ cảnh ở cả hai phía.

Chú giải của người dịch: Ghép nối vectơ

Phép ghép nối $[\mathbf{a}, \mathbf{b}]$ đơn giản là đặt hai vectơ nối đuôi nhau: nếu $\overrightarrow{\mathbf{h}}_i = [0.2 \;\; 0.7]$ và $\overleftarrow{\mathbf{h}}_i = [-0.1 \;\; 0.4 \;\; 0.9]$ thì $\mathbf{h}_i = [0.2 \;\; 0.7 \;\; -0.1 \;\; 0.4 \;\; 0.9]$ (5 chiều). Lợi ích của hai chiều: trong câu "Con đường này *bàn* giao cho thành phố" và "Cái *bàn* này bằng gỗ", nghĩa của "bàn" chỉ rõ khi nhìn cả các từ phía sau.

Để tăng khả năng biểu đạt, RNN hai chiều có thể được mở rộng thành mạng nơ-ron gồm nhiều tầng RNN. Ví dụ, ta có thể chạy các RNN sâu theo hai hướng rồi kết hợp kết quả như trong Phương trình (4.18). Những kiến trúc mô hình như vậy đã được dùng rộng rãi trong các tác vụ xử lý ngôn ngữ và tiếng nói, bao gồm dịch máy [Wu và cộng sự, 2016], phân tích cảm xúc [Tang và cộng sự, 2015], gán nhãn từ loại (POS) [Huang và cộng sự, 2015], nhận dạng tiếng nói (*speech recognition*) [Graves và cộng sự, 2013a;b], v.v.

4.3 Bộ nhớ

RNN phù hợp cho việc học chuỗi vì chúng tóm tắt hiệu quả các đầu vào trong quá khứ tại mỗi bước và dựa trên bản tóm tắt lịch sử này cho các dự đoán tiếp theo. Một lợi ích then chốt của RNN là ta có thể biểu diễn một lịch sử có độ dài bất kỳ bằng một vectơ kích thước cố định và cập nhật nó một cách linh động khi có thông tin mới. Nói cách khác, chúng dùng một **bộ nhớ** (*memory*) ngầm định để lưu trữ thông tin ngữ cảnh. Tiếp theo, chúng tôi sẽ chỉ ra rằng cơ chế bộ nhớ này là một khái niệm tổng quát và có thể được dùng để cải thiện các mô hình chuỗi.

4.3.1 Bộ nhớ như một hệ thống

Trong tâm lý học, trí nhớ là năng lực nhận thức của tâm trí để mã hóa, lưu trữ và truy xuất thông tin. Trong số nhiều mô hình nhận thức của tâm lý học, một khung nền tảng là **mô hình đa kho** (*multi-store model*) [Atkinson và Shiffrin, 1968]. Mô hình này định nghĩa trí nhớ là một hệ thống gồm ba thành phần cấu trúc: **trí nhớ giác quan** (*sensory memory*) thoáng qua, **trí nhớ ngắn hạn** (*short-term memory*) và **trí nhớ dài hạn** (*long-term memory*). Trí nhớ giác quan lưu giữ các kích thích thô vốn phai đi rất nhanh, chẳng hạn dữ liệu thị giác và khứu giác tức thời. Trí nhớ ngắn hạn lưu thông tin lâu hơn một chút nhưng vẫn chỉ tạm thời. Ví dụ, khi cố ghi nhớ một số điện thoại, các chữ số chỉ được giữ tạm và có thể bị quên ngay sau đó. Ngược lại, trí nhớ dài hạn là bền vững, lưu giữ thông tin vô thời hạn. Chẳng hạn, người lớn có thể nhớ lại những sự kiện cụ thể từ thời thơ ấu.

Với cách phân loại này, dường như có những liên hệ thú vị giữa mô hình trí nhớ trên và các mạng nơ-ron ta đang bàn. Ví dụ, trạng thái ẩn của một đơn vị hồi quy có thể được xem là trí nhớ ngắn hạn: nó duy trì thông tin hoạt động trong suốt một chuỗi nhưng bị đặt lại hoàn toàn khi chuyển sang một chuỗi mới[7]. Mặt khác, toàn bộ mô hình ngôn ngữ cùng các tham số của nó hoạt động giống một trí nhớ dài hạn: mô hình mã hóa các khuôn mẫu xác suất tổng quát để dự đoán từ rút ra từ kho ngữ liệu huấn luyện, và lưu giữ vĩnh viễn tri thức này cho suy luận (*inference*) về sau. Hơn nữa, các thước đo nhận thức tiêu chuẩn khớp hoàn hảo với các khái niệm tính toán: **mã hóa** (*coding* hay *encoding*) chỉ cách thông tin được lưu trong bộ nhớ, **thời lượng** (*duration*) chỉ thông tin được lưu trong bộ nhớ bao lâu, và **dung lượng** (*capacity*) chỉ bao nhiêu thông tin được lưu trong bộ nhớ.

Trong học máy và NLP, ta có thể hiểu bộ nhớ bằng cách xét nó từ góc độ xử lý thông tin. Nói chung, bộ nhớ có thể được xem là một hệ thống ghi thông tin vào một "kho lưu trữ" và đọc nó ra khi được truy

[7] Hoặc, trạng thái ẩn tại bước i có thể đơn giản chỉ còn giữ rất ít thông tin về các bước đầu tiên do thông tin bị phai dần.

vấn. Nó có các chức năng sau.

- **Mã hóa** (*Encoding*). Đầu vào của hệ thống được mã hóa dưới một dạng dễ xử lý. Với đầu vào là văn bản, có thể đơn giản xem đây là cùng quá trình mã hóa mà ta bàn ở cả Chương 3 và chương này: một từ hoặc một chuỗi từ được biểu diễn thành một vectơ đặc trưng hoặc một chuỗi vectơ đặc trưng.
- **Cập nhật** (*Update*). Với thông tin đã mã hóa, ta lưu nó vào bộ nhớ. Thao tác này nói chung phụ thuộc vào cách tổ chức bộ nhớ. Ví dụ, có thể coi một nhóm các mục đã mã hóa như một kho dữ liệu có hệ thống chỉ mục. Khi đó, lưu một mục đòi hỏi tìm đúng chỗ để đặt nó. Một cách khác là biểu diễn bộ nhớ như một vectơ số duy nhất.
- **Truy xuất** (*Retrieval*). Thông tin đã lưu có thể được truy xuất. Việc này thường gồm đối sánh từng mục trong bộ nhớ với một truy vấn đầu vào. Nếu bộ nhớ được biểu diễn ở dạng đơn giản hơn, chẳng hạn một vectơ, thì có thể không cần truy xuất tường minh, và ta trả về toàn bộ bộ nhớ khi cần.

Các chức năng này có thể được thiết kế theo nhiều cách khác nhau, dẫn tới nhiều hệ thống NLP đa dạng. Một ví dụ đơn giản là truy xuất thông tin (*information retrieval, IR*) [Manning và cộng sự, 2008]. Một hệ thống IR điển hình lập chỉ mục cho một bộ sưu tập tài liệu khổng lồ và cho phép người dùng tìm kiếm thông tin liên quan. Để tìm kiếm hiệu quả, các tài liệu được mã hóa thành những biểu diễn thuận tiện. Ví dụ, một mô hình túi từ (*bag-of-words*) có thể tính điểm khớp giữa một tài liệu và một truy vấn, trong khi một chỉ mục ngược (*inverted index*) ánh xạ hiệu quả các từ trong truy vấn tới vị trí lưu trữ của chúng. Các hệ thống loại này bao phủ nhiều ứng dụng, gồm bộ nhớ dịch (*translation memory*), hội thoại, tóm tắt, phân loại tài liệu, v.v.

Một lựa chọn thiết kế khác cho hệ thống bộ nhớ là xét, một phần hoặc toàn bộ, dạng liên tục cho các thành phần trên. Một phương pháp là mã hóa mỗi mục đầu vào thành một vectơ số thực (ví dụ một vectơ nhúng từ) nhưng vẫn dùng các mô-đun cập nhật và truy xuất như trong các hệ thống kiểu truy xuất thông tin thông thường [Weston và cộng sự, 2015; Khandelwal và cộng sự, 2019]. Một phương pháp khác là dùng các hàm khả vi cho mọi bước xây dựng và truy cập bộ nhớ. Các mô hình này thường được hiện thực bằng mạng nơ-ron và huấn luyện bằng hạ gradient [Sukhbaatar và cộng sự, 2015; Graves và cộng sự, 2014; Kumar và cộng sự, 2016; Graves và cộng sự, 2016; Miller và cộng sự, 2016]. Ý tưởng này thúc đẩy các nghiên cứu về cách kết hợp mạng nơ-ron với bộ nhớ, chẳng hạn **mạng bộ nhớ đầu-cuối** (*end-to-end memory networks*) và **máy Turing nơ-ron** (*neural Turing machines*). Lưu ý rằng các mô hình trên đôi khi được gọi là **bộ nhớ ngoài** (*external memories*), vì chúng được dùng như các mô-đun riêng làm việc cùng các hệ thống khác.

Bộ nhớ cũng có thể đóng vai trò một thành phần bên trong hay ẩn của hệ thống. Trong trường hợp này, bộ nhớ thường được xây dựng lại cho mỗi mẫu đầu vào, nên có thể xem nó là một trường hợp của trí nhớ ngắn hạn. Có nhiều cách dùng loại bộ nhớ này để cải thiện các mô hình chuỗi. Trong phần còn lại của mục này, chúng ta tập trung vào việc dùng cơ chế bộ nhớ trong RNN. Ở Chương 5, ta sẽ thấy ý tưởng bộ nhớ được mở rộng thế nào để mô hình hóa sự tương ứng giữa các token của hai chuỗi.

Chú giải của người dịch: Bộ nhớ trong máy tính và trong mạng nơ-ron

Một thư viện là ví dụ dễ hình dung của bộ nhớ ngoài: *mã hóa* là viết thẻ mô tả cho mỗi cuốn sách, *cập nhật* là xếp sách lên đúng giá theo chỉ mục, *truy xuất* là tra thẻ để tìm sách khớp với yêu cầu. Còn trạng thái ẩn của RNN giống một trang nháp duy nhất: mọi thông tin bị "viết đè" lên cùng một chỗ, không có ngăn kéo hay chỉ mục, và được xóa sạch khi bắt đầu câu mới. Chữ "khả vi" nghĩa là tính được đạo hàm — điều kiện để có thể huấn luyện bằng hạ gradient.

4.3.2 Bộ nhớ dài-ngắn hạn

Trong RNN cơ bản trình bày ở Mục 4.2.1, phần tóm tắt các từ ngữ cảnh được cho bởi đầu ra của một đơn vị hồi quy. Nó ngầm định nghĩa một bộ nhớ, và do đó về nguyên tắc cho phép mô hình hóa các phụ thuộc có độ dài bất kỳ. Bộ nhớ này chỉ đơn giản kết hợp biểu diễn của lịch sử trước đó $w_1 \cdots w_{i-1}$ với đầu vào ở bước hiện tại w_i, mà không xét xem nên nén bao nhiêu thông tin từ các bước khác nhau vào một biểu diễn có độ dài cố định. Một vấn đề của mô hình này là khi dự đoán cần đến các phụ thuộc xa, bộ nhớ có thể không giữ đủ thông tin liên quan, và có thể khó học được các phụ thuộc này qua lan truyền ngược [Bengio và cộng sự, 1994; Pascanu và cộng sự, 2013]. Vì vậy, một cách tiếp cận mạnh hơn là tính toán xem nên giữ lại gì ở mỗi bước, và để mô hình tự học cách quyết định ghi nhớ hay quên đi.

Bộ nhớ dài-ngắn hạn (*long short-term memory, LSTM*) có lẽ là biến thể nổi tiếng nhất của RNN để đạt được mục tiêu trên [Hochreiter và Schmidhuber, 1997]. Ý tưởng cơ bản của LSTM là một đơn vị hồi quy có thể học cách ghi nhớ những điều hữu ích và quên những điều vô ích bằng cách duy trì một bộ nhớ tường minh [Gers và cộng sự, 2000]. Để làm vậy, đơn vị hồi quy cơ bản được thay bằng một đơn vị LSTM gồm một vectơ đầu ra (gọi là **ô hồi quy** (*recurrent cell*)), một vectơ bộ nhớ (gọi là **ô nhớ** (*memory cell*)), và ba cổng để điều khiển dòng thông tin bên trong đơn vị LSTM. Là một mở rộng của RNN, mạng LSTM xử lý chuỗi đầu vào như thường lệ: nó bắt đầu với một số trạng thái khởi tạo, rồi lặp lại việc nhận một đầu vào và cho ra một vectơ. Một khác biệt then chốt giữa mạng LSTM và RNN là đơn vị LSTM ở bước i nhận cả ô hồi quy lẫn ô nhớ của bước trước nó. Dạng của một đơn vị LSTM là:

$$(\mathbf{h}_i, \mathbf{c}_i) \;=\; \text{LSTM}(\mathbf{h}_{i-1}, \mathbf{c}_{i-1}, \mathbf{x}_i), \tag{4.19}$$

trong đó $\mathbf{h}_i \in \mathbb{R}^{d_h}$ là ô hồi quy ở bước i, $\mathbf{c}_i \in \mathbb{R}^{d_h}$ là ô nhớ ở bước i, và $\mathbf{x}_i \in \mathbb{R}^{d_e}$ là đầu vào ở bước i. Khi đã có LSTM$(\cdot)$, việc áp dụng mô hình LSTM rất đơn giản. Ta chỉ cần gọi lặp lại LSTM$(\cdot)$ cho các đầu vào $\{\mathbf{x}_1, \ldots, \mathbf{x}_m\}$ và thu được các đầu ra $\{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$. Điều này giống cách ta dùng RNN cơ bản, nên rất dễ mở rộng LSTM thành mô hình nhiều tầng (xem Mục 4.2.3) và mô hình hai chiều (xem Mục 4.2.4).

Ta có thể chia LSTM$(\cdot)$ thành ba bước.

- **Bước 1: Quên** (*Forget*). Giả sử $\mathbf{c}_{i-1}$ chứa thông tin mà mô hình ghi nhớ ở bước $i-1$, ta cần xác định bao nhiêu thông tin trong $\mathbf{c}_{i-1}$ bị loại bỏ khi xây dựng $\mathbf{c}_i$. Để làm điều này, một cổng được dùng để điều khiển mức độ quên ở từng chiều của $\mathbf{c}_{i-1}$. **Cổng quên** (*forget gate*) được định nghĩa là:

  $$\mathbf{f}_i \;=\; \text{Sigmoid}(\mathbf{h}_{i-1}\mathbf{U}_\text{f} + \mathbf{x}_i\mathbf{V}_\text{f} + \mathbf{b}_\text{f}), \tag{4.20}$$

  trong đó $\mathbf{f}_i \in [0,1]^{d_h}$ là một vectơ có cùng số chiều với $\mathbf{c}_{i-1}$. Hàm Sigmoid ánh xạ dữ liệu đầu vào vào khoảng $[0,1]$. Do đó, một phần tử của $\mathbf{f}_i$ cho biết phần tử tương ứng của $\mathbf{c}_{i-1}$ được giữ lại bao nhiêu. Xa hơn nữa, $\mathbf{f}_i \odot \mathbf{c}_{i-1}$ biểu diễn phần bộ nhớ được giữ lại sau khi đi qua cổng quên. Xem Hình 4.5(a) để thấy minh họa cổng quên trong đơn vị LSTM.

- **Bước 2: Cập nhật** (*Update*). Tiếp theo ta cập nhật bộ nhớ bằng cách xét cả trạng thái trước của bộ nhớ (tức $\mathbf{c}_{i-1}$) lẫn đầu vào của đơn vị LSTM (tức $\mathbf{x}_i$ và $\mathbf{h}_{i-1}$). Trước tiên ta kết hợp $\mathbf{x}_i$ và $\mathbf{h}_{i-1}$ bằng một mạng nơ-ron đơn giản:

  $$\hat{\mathbf{c}}_i \;=\; \text{TanH}(\mathbf{h}_{i-1}\mathbf{U}_\text{c} + \mathbf{x}_i\mathbf{V}_\text{c} + \mathbf{b}_\text{c}). \tag{4.21}$$

  $\hat{\mathbf{c}}_i$ có thể được xem là thông tin mới mà ta định thêm vào bộ nhớ ở bước i. Một lần nữa, ta cần một cách điều khiển lượng thông tin đi vào bộ nhớ. Vì vậy ta định nghĩa một **cổng vào** (*input gate*) là

  $$\mathbf{g}_i \;=\; \text{Sigmoid}(\mathbf{h}_{i-1}\mathbf{U}_\text{g} + \mathbf{x}_i\mathbf{V}_\text{g} + \mathbf{b}_\text{g}). \tag{4.22}$$

Phương trình này tương tự Phương trình (4.20) nhưng với các tham số khác. Sau đó ta định nghĩa $\mathbf{g}_i \odot \hat{\mathbf{c}}_i$ là thông tin mới thực sự mà ta quan tâm. Lấy cả $\mathbf{f}_i \odot \mathbf{c}_{i-1}$ và $\mathbf{g}_i \odot \hat{\mathbf{c}}_i$, ô nhớ ở bước i được cho bởi:

$$\mathbf{c}_i = \mathbf{f}_i \odot \mathbf{c}_{i-1} + \mathbf{g}_i \odot \hat{\mathbf{c}}_i. \tag{4.23}$$

Nói cách khác, ta quên đi một phần cái cũ trong $\mathbf{c}_{i-1}$ và ghi nhớ một phần cái mới trong $\hat{\mathbf{c}}_i$. Xem Hình 4.5(b) để thấy minh họa bước cập nhật.

- **Bước 3: Đầu ra** (*Output*). Ở bước cuối, ta sinh đầu ra $\mathbf{h}_i$ dựa trên bộ nhớ $\mathbf{c}_i$. Thay vì sao chép $\mathbf{c}_i$ sang $\mathbf{h}_i$, ta đưa $\mathbf{c}_i$ qua hàm tang hyperbolic rồi nhân kết quả với cổng ra. Giống các Phương trình (4.20) và (4.22), cổng ra được cho bởi:

$$\mathbf{o}_i = \text{Sigmoid}(\mathbf{h}_{i-1}\mathbf{U}_\text{o} + \mathbf{x}_i\mathbf{V}_\text{o} + \mathbf{b}_\text{o}). \tag{4.24}$$

Khi đó, đầu ra của đơn vị LSTM được định nghĩa là:

$$\mathbf{h}_i = \mathbf{o}_i \odot \text{TanH}(\mathbf{c}_i). \tag{4.25}$$

Xem Hình 4.5(c) để thấy minh họa bước đầu ra.

Mô hình LSTM được tham số hóa bởi $\mathbf{U}_\text{f}, \mathbf{U}_\text{c}, \mathbf{U}_\text{g}, \mathbf{U}_\text{o} \in \mathbb{R}^{d_h \times d_h}$, $\mathbf{V}_\text{f}, \mathbf{V}_\text{c}, \mathbf{V}_\text{g}, \mathbf{V}_\text{o} \in \mathbb{R}^{d_e \times d_h}$, và $\mathbf{b}_\text{f}, \mathbf{b}_\text{c}, \mathbf{b}_\text{g}, \mathbf{b}_\text{o} \in \mathbb{R}^{d_h}$. So với RNN cơ bản, các tham số bổ sung phát sinh trực tiếp từ kiến trúc dựa trên cổng này. Trong thực tế, có thể hiện thực đơn vị LSTM theo nhiều cách khác nhau, ví dụ dùng các hàm kích hoạt khác Sigmoid$(\cdot)$ và TanH$(\cdot)$, hay bỏ các số hạng độ lệch $\mathbf{b}_\text{f}$, $\mathbf{b}_\text{c}$, $\mathbf{b}_\text{g}$ và $\mathbf{b}_\text{o}$. Việc huấn luyện mô hình LSTM tuân theo khuôn mẫu chuẩn của việc huấn luyện các mô hình dựa trên RNN. Ví dụ, ta có thể xây dựng một mô hình ngôn ngữ dựa trên LSTM và huấn luyện nó bằng các phương pháp trình bày ở Mục 4.2.2.

Chú giải của người dịch: Các "cổng" của LSTM hoạt động thế nào?

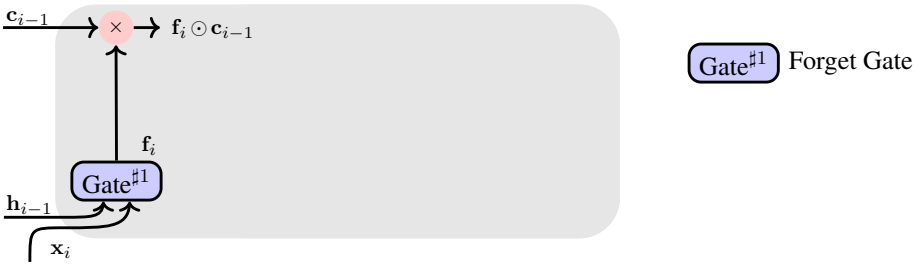
Ký hiệu $\odot$ là phép nhân *theo từng phần tử*: $[a_1\ a_2] \odot [b_1\ b_2] = [a_1 b_1\ \ a_2 b_2]$. Vì Sigmoid cho giá trị trong $[0, 1]$, mỗi phần tử của một cổng giống một "van nước": 0 là đóng hẳn, 1 là mở hết. Ví dụ với bộ nhớ 2 chiều: $\mathbf{c}_{i-1} = [5\ \ -2]$, cổng quên $\mathbf{f}_i = [1\ \ 0]$, thông tin mới $\hat{\mathbf{c}}_i = [0.5\ \ 0.8]$, cổng vào $\mathbf{g}_i = [0\ \ 1]$. Theo Phương trình (4.23): $\mathbf{c}_i = [1 \cdot 5\ \ 0 \cdot (-2)] + [0 \cdot 0.5\ \ 1 \cdot 0.8] = [5\ \ 0.8]$. Chiều thứ nhất giữ nguyên ký ức cũ, chiều thứ hai xóa ký ức cũ và nạp thông tin mới. Điểm mấu chốt: $\mathbf{c}_i$ được tính chủ yếu bằng *phép cộng* (thay vì đưa qua hàm kích hoạt mỗi bước như RNN cơ bản), nên khi cổng quên gần 1, thông tin — và gradient — có thể truyền qua nhiều bước mà không bị teo nhỏ. Đó là lý do LSTM giảm được vấn đề tiêu biến gradient.

4.3.3 Đơn vị hồi quy có cổng

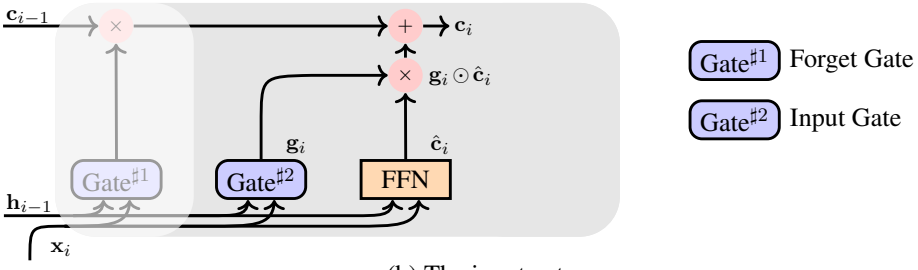
Ở trên, ta đã thấy vai trò quan trọng của các đơn vị cổng và ô nhớ. Nhìn chung, việc dùng các mạng nơ-ron này khiến mô hình tốn kém hơn về tính toán. Một lựa chọn thay thế hiệu quả hơn về tính toán cho LSTM là **đơn vị hồi quy có cổng** (*gated recurrent unit, GRU*), dùng một cấu trúc mô hình đơn giản hơn với ít hàm cổng hơn [Cho và cộng sự, 2014; Chung và cộng sự, 2014]. Khác với LSTM, GRU không có ô nhớ, nên giống như một đơn vị RNN, nó nhận vectơ trạng thái trước $\mathbf{h}_{i-1}$ và vectơ đầu vào hiện tại $\mathbf{x}_i$, rồi sinh ra vectơ trạng thái hiện tại $\mathbf{h}_i$.

Kiến trúc GRU gồm hai đơn vị cổng: **cổng đặt lại** (*reset gate*) và **cổng cập nhật** (*update gate*). Cổng đặt lại, đúng như tên gọi, được dùng để đặt lại (hoặc co giãn lại) trạng thái của GRU (tức $\mathbf{h}_{i-1}$). Tương tự các cổng trong LSTM, cổng đặt lại được định nghĩa là:

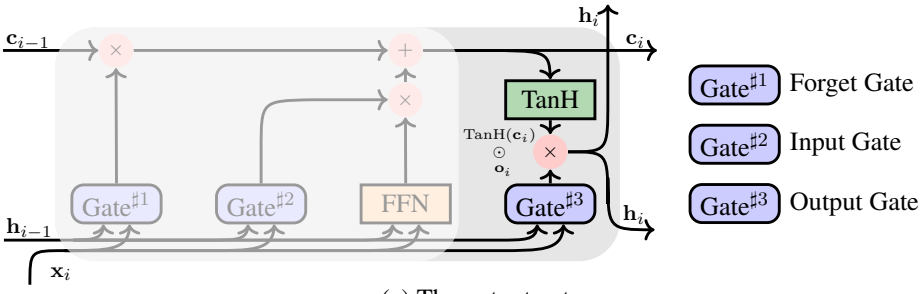
$$\mathbf{r}_i = \text{Sigmoid}(\mathbf{h}_{i-1}\mathbf{U}_\text{r} + \mathbf{x}_i\mathbf{V}_\text{r} + \mathbf{b}_\text{r}), \tag{4.26}$$



(a) The forget gate.



(b) The input gate.



(c) The output gate.

Hình 4.5: Kiến trúc của đơn vị LSTM. Tại bước i, nó nhận đầu vào $\mathbf{x}_i$, rồi cập nhật cả ô nhớ ($\mathbf{c}_{i-1} \rightarrow \mathbf{c}_i$) lẫn ô hồi quy ($\mathbf{h}_{i-1} \rightarrow \mathbf{h}_i$). Quá trình này gồm ba cổng: cổng quên điều khiển bao nhiêu thông tin trong $\mathbf{c}_{i-1}$ được giữ lại ở bước i, cổng vào điều khiển bao nhiêu thông tin trong $\mathbf{c}_{i-1}$ và $\mathbf{x}_i$ được giữ lại ở bước i, và cổng ra điều khiển bao nhiêu thông tin trong $\mathbf{c}_i$ được dùng để tạo thành $\mathbf{h}_i$. (Nhãn trong hình: *Forget Gate* = cổng quên; *Input Gate* = cổng vào; *Output Gate* = cổng ra; (a) cổng quên; (b) cổng vào; (c) cổng ra.)

trong đó $\mathbf{r}_i \in [0,1]^{d_h}$ là một vectơ các số vô hướng, mỗi chiều mô tả bao nhiêu thông tin ở chiều tương ứng của $\mathbf{h}_{i-1}$ được giữ lại. Như vậy, ta có một biểu diễn của thông tin được giữ lại

$$\upsilon_{i-1} = \mathbf{r}_i \odot \mathbf{h}_{i-1}. \tag{4.27}$$

Lấy cả thông tin được giữ lại υ_{i-1} và đầu vào hiện tại $\mathbf{x}_i$, một vectơ trạng thái mới được định nghĩa là:

$$\hat{\mathbf{h}}_i = \text{TanH}(\upsilon_{i-1}\mathbf{U}_\text{h} + \mathbf{x}_i\mathbf{V}_\text{h} + \mathbf{b}_\text{h}). \tag{4.28}$$

Cổng cập nhật khi đó được cho bởi:

$$\mathbf{u}_i = \text{Sigmoid}(\mathbf{h}_{i-1}\mathbf{U}_\text{u} + \mathbf{x}_i\mathbf{V}_\text{u} + \mathbf{b}_\text{u}), \tag{4.29}$$

$\mathbf{u}_i$ có thể được xem là một vectơ trọng số nội suy, cân bằng giữa việc giữ lại trạng thái cũ và việc tích hợp trạng thái ứng viên mới. Cuối cùng, đầu ra của GRU được định nghĩa là một phép nội suy tuyến tính giữa $\hat{\mathbf{h}}_i$ và $\mathbf{h}_{i-1}$

$$\mathbf{h}_i = \mathbf{u}_i \odot \hat{\mathbf{h}}_i + (1 - \mathbf{u}_i) \odot \mathbf{h}_{i-1}. \tag{4.30}$$

Hình 4.6 minh họa dòng thông tin trong một đơn vị GRU. Các tham số ở đây là $\mathbf{U}_\text{r}, \mathbf{U}_\text{h}, \mathbf{U}_\text{u} \in \mathbb{R}^{d_h \times d_h}$, $\mathbf{V}_\text{r}, \mathbf{V}_\text{h}, \mathbf{V}_\text{u} \in \mathbb{R}^{d_e \times d_h}$, và $\mathbf{b}_\text{r}, \mathbf{b}_\text{h}, \mathbf{b}_\text{u} \in \mathbb{R}^{d_h}$. Do đó, mô hình GRU nhỏ hơn mô hình LSTM vì dùng ít đơn vị cổng hơn. Lưu ý rằng việc bỏ ô nhớ khiến GRU hiệu quả hơn. Trong trường hợp này, vai trò của bộ nhớ được đảm nhận một cách ngầm định bởi trạng thái ẩn $\mathbf{h}_i$, vốn được cập nhật linh động để lưu giữ các thông tin quan trọng theo thời gian.

Chú giải của người dịch: Nội suy tuyến tính trong GRU

Phương trình (4.30) giống công thức "trung bình có trọng số" quen thuộc: với hai số a, b và $t \in [0,1]$, $t \cdot a + (1-t) \cdot b$ là một điểm nằm giữa a và b ($t = 1$ cho a, $t = 0$ cho b). Ví dụ một chiều: $\hat{h}_i = 0.8$, $h_{i-1} = -0.4$, $u_i = 0.25$ thì $h_i = 0.25 \cdot 0.8 + 0.75 \cdot (-0.4) = -0.1$, tức trạng thái mới vẫn nghiêng nhiều về trạng thái cũ. GRU áp dụng điều này cho từng chiều với trọng số riêng. So về số ma trận: LSTM có 4 bộ $(\mathbf{U}, \mathbf{V}, \mathbf{b})$, còn GRU chỉ có 3, nên GRU nhỏ hơn khoảng một phần tư.

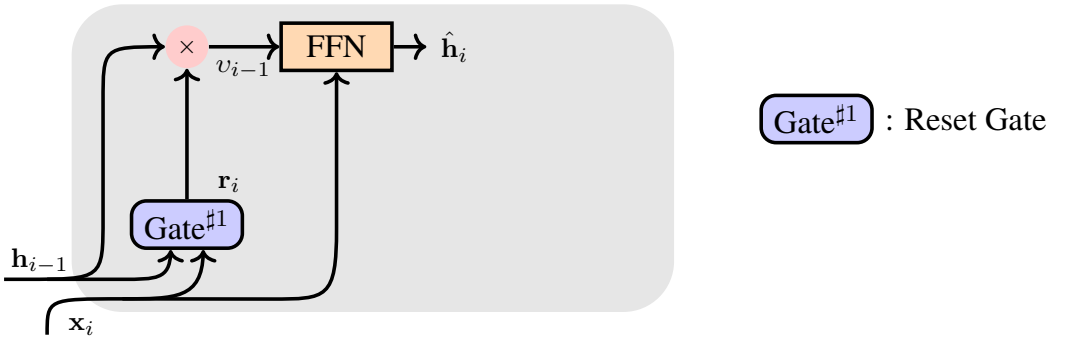
4.4 Các mô hình tích chập

Trong mục này, chúng ta mô tả một loại mô hình khác dùng cho mô hình hóa chuỗi, gọi là mạng nơ-ron tích chập (*convolutional neural networks, CNNs*). Phần tổng quan dưới đây trình bày các khái niệm cơ bản, nhưng không nhằm giới thiệu đầy đủ mọi biến thể CNN và các kỹ thuật tiên tiến nhất. Thay vào đó, chúng ta tập trung vào việc áp dụng CNN cho dữ liệu dạng chuỗi và trình bày một số cải tiến về kiến trúc.

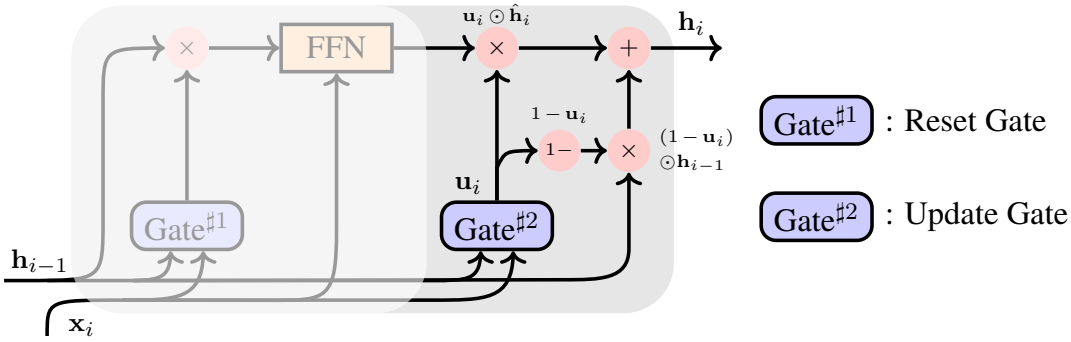
4.4.1 Tích chập

Một tính chất then chốt của CNN là *kiến trúc chia sẻ trọng số*: một nhân (*kernel*) hay bộ lọc (*filter*) trượt trên dữ liệu đầu vào và tạo ra một bản đồ đặc trưng (*feature map*). Ý tưởng là tại mỗi thời điểm, bộ lọc chỉ nhận tín hiệu từ một vùng giới hạn của dữ liệu (gọi là trường tiếp nhận (*receptive field*)), và tính tổng có trọng số của các tín hiệu đầu vào này. Để minh họa, ta xét trường hợp thường gặp là bộ lọc trong CNN được dùng để xử lý dữ liệu hai chiều. Xét ma trận dữ liệu 3×3:

$$\mathbf{A} = \begin{bmatrix} 1 & 9 & 7 \\ 3 & 1 & 2 \\ 0 & 1 & -1 \end{bmatrix}, \tag{4.31}$$



(a) The reset gate.



(b) The update gate.

Hình 4.6: Kiến trúc của GRU. Khác với đơn vị LSTM, GRU không có ô nhớ, do đó tuân theo cùng dạng đầu vào và đầu ra như một đơn vị RNN tiêu chuẩn. GRU có hai cổng. Cổng đặt lại điều khiển bao nhiêu thông tin trong $\mathbf{h}_{i-1}$ được giữ lại ở bước i. Thông tin được giữ lại sau đó được hợp nhất với đầu vào $\mathbf{x}_i$, sinh ra đầu ra ứng viên $\hat{\mathbf{h}}_i$. Cổng cập nhật tìm sự cân bằng giữa $\hat{\mathbf{h}}_i$ và $\mathbf{h}_{i-1}$ khi tính đầu ra cuối cùng của GRU. (Nhãn trong hình: *Reset Gate* = cổng đặt lại; *Update Gate* = cổng cập nhật; (a) cổng đặt lại; (b) cổng cập nhật.)

và một bộ lọc 2×2 có ma trận trọng số

$$\mathbf{W} = \begin{bmatrix} 2 & 0 \\ 2 & 2 \end{bmatrix}. \tag{4.32}$$

Ta áp dụng bộ lọc này lên từng ma trận con 2×2 của $\mathbf{A}$ (trong ví dụ này có bốn ma trận con như vậy) và tính tổng các phần tử có trọng số theo $\mathbf{W}$. Chẳng hạn, với ma trận con 2×2 ở góc trên bên trái của $\mathbf{A}$, đầu ra của bộ lọc được tính như sau:

$$\begin{aligned} \mathrm{Conv}\Big(\begin{bmatrix} 1 & 9 \\ 3 & 1 \end{bmatrix}, \mathbf{W}\Big) &= \mathrm{Conv}\Big(\begin{bmatrix} 1 & 9 \\ 3 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 2 & 2 \end{bmatrix}\Big) \\ &= 1 \times 2 + 9 \times 0 + 3 \times 2 + 1 \times 2 \\ &= 10. \end{aligned} \tag{4.33}$$

$\mathrm{Conv}(\cdot)$ định nghĩa một **phép tích chập** (*convolution operation*), tức là lấy tổng các phần tử của tích theo từng phần tử của hai ma trận. Phép tích chập có thể được mở rộng để phủ toàn bộ ma trận đầu vào bằng cách trượt bộ lọc trên nó, như sau:

$$\begin{aligned} \mathrm{Conv}(\mathbf{A}, \mathbf{W}) &= \mathrm{Conv}\Big(\begin{bmatrix} 1 & 9 & 7 \\ 3 & 1 & 2 \\ 0 & 1 & -1 \end{bmatrix}, \mathbf{W}\Big) \\ &= \begin{bmatrix} \mathrm{Conv}\Big(\begin{bmatrix} 1 & 9 \\ 3 & 1 \end{bmatrix}, \mathbf{W}\Big) & \mathrm{Conv}\Big(\begin{bmatrix} 9 & 7 \\ 1 & 2 \end{bmatrix}, \mathbf{W}\Big) \\ \mathrm{Conv}\Big(\begin{bmatrix} 3 & 1 \\ 0 & 1 \end{bmatrix}, \mathbf{W}\Big) & \mathrm{Conv}\Big(\begin{bmatrix} 1 & 2 \\ 1 & -1 \end{bmatrix}, \mathbf{W}\Big) \end{bmatrix} \\ &= \begin{bmatrix} 10 & 24 \\ 8 & 2 \end{bmatrix}. \end{aligned} \tag{4.34}$$

Đầu ra $\begin{bmatrix} 10 & 24 \\ 8 & 2 \end{bmatrix}$ còn được gọi là **bản đồ đặc trưng** (*feature map*) của bộ lọc $\mathbf{W}$ trên $\mathbf{A}$. Đôi khi, phép tích chập $\mathrm{Conv}(\mathbf{A}, \mathbf{W})$ được viết là $\mathbf{A} * \mathbf{W}$, trong đó ký hiệu $*$ biểu thị **tích chập** (*convolution product*)[8].

[8] Trong giải tích toán học, cho hai hàm khả tích $f(\cdot)$ và $g(\cdot)$, **tích chập** (*convolution*) định nghĩa một hàm khả tích mới $f * g(\cdot)$ mô tả tích phân của $f(\cdot)$ được đánh trọng số bởi hàm $g(\cdot)$ đã được lấy đối xứng và tịnh tiến. Chính xác hơn, tích chập của các hàm liên tục được định nghĩa là:

$$f * g(x) = \int_{\mathbb{R}} f(y)\, g(x - y)\, \mathrm{d}y, \tag{4.35}$$

trong đó $f(y)$ là hàm mà ta quan tâm, còn $g(x - y)$ là hàm trọng số, thu được bằng cách lấy đối xứng $g(y)$ qua trục y rồi tịnh tiến nó một đoạn x. Một trường hợp đặc biệt là khi x và y đều là số nguyên. Khi đó, ta có thể định nghĩa $f * g(\cdot)$ là

$$f * g(x) = \sum_{y} f(y)\, g(x - y), \tag{4.36}$$

đây chính là dạng cơ bản của Phương trình (4.33). Trong CNN, x, y và $x - y$ có thể được xem là chỉ số của các phần tử trong mảng dữ liệu. $f(y)$ là một phần tử dữ liệu trong mảng đầu vào, và $g(x - y)$ là trọng số tương ứng trong bộ lọc. Dùng Phương trình (4.36), ta tính được giá trị của phần tử có chỉ số x trong mảng đầu ra $f * g(x)$ (tức là bản đồ đặc trưng).

Chú giải của người dịch: Tích chập = "trượt và nhân–cộng"

Hãy hình dung bộ lọc $\mathbf{W}$ như một "tấm kính lúp" cỡ 2×2 đặt lên ma trận $\mathbf{A}$. Ở mỗi vị trí, ta nhân từng ô của kính với ô nằm ngay bên dưới rồi cộng tất cả lại để được *một số*. Sau đó dịch kính sang phải (hoặc xuống dưới) một ô và làm lại. Kiểm tra ô góc trên bên phải của Phương trình (4.34): ma trận con là $\begin{bmatrix} 9 & 7 \\ 1 & 2 \end{bmatrix}$, nên $9 \times 2 + 7 \times 0 + 1 \times 2 + 2 \times 2 = 18 + 0 + 2 + 4 = 24$. Vì bộ lọc 2×2 chỉ đặt được ở $2 \times 2 = 4$ vị trí trên ma trận 3×3, bản đồ đặc trưng có kích thước 2×2. Điểm quan trọng: *cùng một* bộ trọng số $\mathbf{W}$ được dùng ở mọi vị trí — đó là ý nghĩa của "chia sẻ trọng số".

Bây giờ ta xét một mô tả tổng quát hơn về tích chập trong CNN. Giả sử $\mathbf{A}$ là một mảng dữ liệu nhiều chiều. Một bộ lọc định nghĩa một cửa sổ (hay trường tiếp nhận) trên $\mathbf{A}$. Ta có thể di chuyển cửa sổ trên $\mathbf{A}$ theo các hướng khác nhau. Việc này tạo ra một tập các mảng dữ liệu, ký hiệu là Ω. Mỗi mảng dữ liệu $\mathbf{a}_p \in \Omega$ được tạo thành từ các phần tử thuộc vùng tương ứng của $\mathbf{A}$. Ví dụ, có bốn ma trận con trong Phương trình (4.34): $\mathbf{a}_1 = \begin{bmatrix} 1 & 9 \\ 3 & 1 \end{bmatrix}$, $\mathbf{a}_2 = \begin{bmatrix} 9 & 7 \\ 1 & 2 \end{bmatrix}$, $\mathbf{a}_3 = \begin{bmatrix} 3 & 1 \\ 0 & 1 \end{bmatrix}$ và $\mathbf{a}_4 = \begin{bmatrix} 1 & 2 \\ 1 & -1 \end{bmatrix}$. Ngoài ra, ta giả sử bộ lọc được tham số hóa bởi một mảng trọng số $\mathbf{W}$ có cùng kích thước với $\mathbf{a}_p$, tức là $|\mathbf{a}_p| = |\mathbf{W}|$. Kết quả của việc áp dụng bộ lọc lên $\mathbf{A}$ là một mảng các đặc trưng

$$\text{Conv}(\mathbf{A}, \mathbf{W}) = \begin{bmatrix} v_1 & \cdots & v_{|\Omega|} \end{bmatrix}. \tag{4.37}$$

Mỗi đặc trưng v_p được cho bởi:

$$\begin{aligned} v_p &= \text{Conv}(\mathbf{a}_p, \mathbf{W}) \\ &= \mathbf{a}_p \cdot \mathbf{W} \\ &= \sum_{k=1}^{|\mathbf{W}|} a_p(k) \cdot W(k), \end{aligned} \tag{4.38}$$

trong đó $a_p(k)$ và $W(k)$ lần lượt là phần tử thứ k của các mảng đã được duỗi phẳng $\mathbf{a}_p$ và $\mathbf{W}$. Lưu ý rằng mặc dù mảng $\begin{bmatrix} v_1 & \cdots & v_{|\Omega|} \end{bmatrix}$ có thể được định dạng lại thành nhiều dạng khác nhau (như ma trận hay tensor 3 chiều) tùy theo số chiều của đầu vào, các biểu diễn này về cơ bản là tương đương xét theo góc độ lưu trữ dữ liệu. Chẳng hạn, áp dụng một bộ lọc hai chiều lên dữ liệu đầu vào hai chiều sẽ cho một ma trận đặc trưng hai chiều, như minh họa trong Phương trình (4.34).

Chú giải của người dịch: "Duỗi phẳng" một mảng

Duỗi phẳng (*flatten*) nghĩa là đọc các phần tử của một ma trận lần lượt theo từng hàng và xếp thành một dãy số. Ví dụ $\mathbf{a}_1 = \begin{bmatrix} 1 & 9 \\ 3 & 1 \end{bmatrix}$ thành $(1, 9, 3, 1)$ và $\mathbf{W}$ thành $(2, 0, 2, 2)$. Khi đó Phương trình (4.38) chính là tích vô hướng quen thuộc của hai vectơ 4 chiều: $1 \cdot 2 + 9 \cdot 0 + 3 \cdot 2 + 1 \cdot 2 = 10$, giống như tích vô hướng $\vec{u} \cdot \vec{v} = x_1 x_2 + y_1 y_2$ ở lớp 10 nhưng có nhiều tọa độ hơn.

Hơn nữa, ta cần giải quyết hai vấn đề để mô hình dùng được trong thực tế. Thứ nhất, ta phải xác định **bước trượt** (*stride*), tức là độ dài mỗi bước dịch chuyển của bộ lọc khi nó trượt trên $\mathbf{A}$. Trong ví dụ trước, ta đã giả định bước trượt bằng 1 (tức là stride $= 1$). Bằng cách dùng bước trượt lớn hơn, ta thực chất đã giảm mẫu (*downsample*) $\mathbf{A}$, nén nó thành một số lượng đặc trưng ít hơn. Thứ hai, để kiểm soát kích thước không gian của bản đồ đặc trưng đầu ra, ta có thể thêm các phần tử giả (hay **phần đệm** (*padding*)) quanh biên của dữ liệu đầu vào. Đệm số 0 (*zero-padding*), tức là bao quanh vùng đầu vào

bằng các số 0, là cách làm phổ biến nhất. Ví dụ, xét ma trận dữ liệu 2×2:

$$\mathbf{A} = \begin{bmatrix} 1 & 9 \\ 7 & 3 \end{bmatrix}. \tag{4.39}$$

Ta có thể thêm các phần tử có giá trị 0 xung quanh nó để được một ma trận 4×4

$$\mathbf{A}_{\text{padding}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 9 & 0 \\ 0 & 7 & 3 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}. \tag{4.40}$$

Dùng cùng bộ lọc như trong Phương trình (4.33) với stride $= 1$, ta có bản đồ đặc trưng 3×3

$$\text{Conv}_{\text{stride}=1}(\mathbf{A}_{\text{padding}}, \mathbf{W}) = \begin{bmatrix} 2 & 20 & 18 \\ 14 & 22 & 24 \\ 0 & 14 & 6 \end{bmatrix}. \tag{4.41}$$

Nếu stride $= 2$ thì ta sẽ có bản đồ đặc trưng có cùng kích thước với dữ liệu đầu vào

$$\text{Conv}_{\text{stride}=2}(\mathbf{A}_{\text{padding}}, \mathbf{W}) = \begin{bmatrix} 2 & 18 \\ 0 & 6 \end{bmatrix}. \tag{4.42}$$

Chú giải của người dịch: Kích thước đầu ra

Với đầu vào (đã đệm) có cạnh dài n, bộ lọc cạnh f và bước trượt s, số vị trí đặt được bộ lọc trên mỗi cạnh là $\lfloor (n-f)/s \rfloor + 1$. Ở Phương trình (4.41): $n = 4$, $f = 2$, $s = 1$ cho $(4-2)/1 + 1 = 3$, nên đầu ra là 3×3. Ở Phương trình (4.42): $s = 2$ cho $(4-2)/2 + 1 = 2$, nên đầu ra là 2×2 — bộ lọc "nhảy cóc" 2 ô mỗi lần, chỉ dừng ở các góc trên-trái, trên-phải, dưới-trái, dưới-phải. Kiểm tra ô đầu tiên: ma trận con $\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$ cho $0 \cdot 2 + 0 \cdot 0 + 0 \cdot 2 + 1 \cdot 2 = 2$.

Lưu ý rằng, nói một cách chặt chẽ, phép toán được cài đặt trong học sâu về mặt kỹ thuật là **tương quan chéo** (*cross-correlation*) chứ không phải tích chập thực sự theo nghĩa toán học. Tích chập thực sự đòi hỏi bộ lọc phải được lấy đối xứng (lật) trước khi trượt trên đầu vào. Tuy nhiên, vì trọng số của bộ lọc trong CNN được học từ đầu, bước lật này là thừa về mặt toán học. Hầu hết các framework học sâu bỏ qua bước này để tính toán hiệu quả hơn, dù trong lĩnh vực này người ta vẫn quen giữ thuật ngữ "tích chập".

4.4.2 CNN cho mô hình hóa chuỗi

Theo cách trình bày ở các mục trước, ta giả sử đầu vào của một mô hình chuỗi là một chuỗi vectơ $\mathbf{x}_1 \cdots \mathbf{x}_m$ và đầu ra là một chuỗi vectơ khác $\mathbf{h}_1 \cdots \mathbf{h}_m$. Ví dụ, ta có thể xem $\mathbf{x}_1 \cdots \mathbf{x}_m$ như một ma trận $\mathbf{X} \in \mathbb{R}^{m \times d_e}$, trong đó vectơ hàng thứ i biểu diễn từ thứ i của chuỗi.

Việc thực hiện tích chập trên $\mathbf{X}$ khá đơn giản. Vì $\mathbf{x}_i$ không áp đặt thứ tự nào giữa các chiều đặc trưng, ta không cần thực hiện tích chập dọc theo các chiều đặc trưng. Do đó, ta có thể dùng một trường tiếp nhận kích thước $r \times d_e$ và xét đồng thời tất cả các chiều của $\mathbf{x}_i$ trong phép tích chập. Trong ứng dụng thực tế, người ta thường dùng nhiều bộ lọc để nắm bắt các khía cạnh khác nhau của đầu vào. Chẳng hạn, có thể dùng một bộ lọc có trường tiếp nhận lớn để đưa vào ngữ cảnh rộng hơn, đồng thời dùng một

bộ lọc khác có trường tiếp nhận nhỏ để tập trung vào các đặc trưng rất cục bộ. Hình 4.7 minh họa việc áp dụng hai bộ lọc khác nhau lên một chuỗi đầu vào.

Để phân phối các đặc trưng đã trích xuất vào $\{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$, ta gắn mỗi lần áp dụng bộ lọc với một vị trí cụ thể trong chuỗi. Để bảo đảm chuỗi đầu vào và chuỗi đầu ra có cùng độ dài, một vectơ đệm được thêm vào cả hai đầu của chuỗi đầu vào. Bảng dưới đây cho thấy đầu vào và đầu ra của một CNN trên một chuỗi ví dụ:

Vị trí	Đầu vào	Trường tiếp nhận	Đầu ra
0	$\mathbf{x}_0 (= 0)$	Không có	Không có
1	$\mathbf{x}_1$	$\{\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2\}$	$\mathbf{h}_1$
2	$\mathbf{x}_2$	$\{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3\}$	$\mathbf{h}_2$
3	$\mathbf{x}_3$	$\{\mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4\}$	$\mathbf{h}_3$
4	$\mathbf{x}_4$	$\{\mathbf{x}_3, \mathbf{x}_4, \mathbf{x}_5\}$	$\mathbf{h}_4$
5	$\mathbf{x}_5 (= 0)$	Không có	Không có

Thông thường, một hàm kích hoạt được áp dụng để đưa tính phi tuyến vào đầu ra cuối cùng. Bằng cách này, ta xây dựng được một tầng tích chập (*convolutional layer*) tiêu chuẩn, có thể xem như một chuỗi các mạng nơ-ron kết nối đầy đủ, mỗi mạng nhận đầu vào từ một cửa sổ có kích thước cố định. Tại vị trí thứ i, đầu ra của tầng tích chập được cho bởi:

$$v_i = \psi(\text{Conv}(\mathbf{a}_i, \mathbf{W})), \tag{4.43}$$

trong đó $\mathbf{a}_i$ biểu diễn các đầu vào trong trường tiếp nhận[9], và $\mathbf{W}$ biểu diễn các tham số của bộ lọc. Trong trường hợp có nhiều bộ lọc (ví dụ, d_h bộ lọc), ta có một tập tham số $\{\mathbf{W}^{(1)}, \ldots, \mathbf{W}^{(d_h)}\}$, một tập hàm kích hoạt $\{\psi^{(1)}, \ldots, \psi^{(d_h)}\}$, và một tập đầu vào $\{\mathbf{a}_i^{(1)}, \ldots, \mathbf{a}_i^{(d_h)}\}$. Mỗi bộ ba $(\mathbf{W}^{(k)}, \psi^{(k)}, \mathbf{a}_i^{(k)})$ cho một đầu ra

$$v_i^{(k)} = \psi^{(k)}(\text{Conv}(\mathbf{a}_i^{(k)}, \mathbf{W}^{(k)})). \tag{4.44}$$

Lưu ý rằng $v_i^{(k)}$ chỉ đơn giản là một phần tử của $\mathbf{h}_i$. Do đó, $\mathbf{h}_i$ có thể được viết là

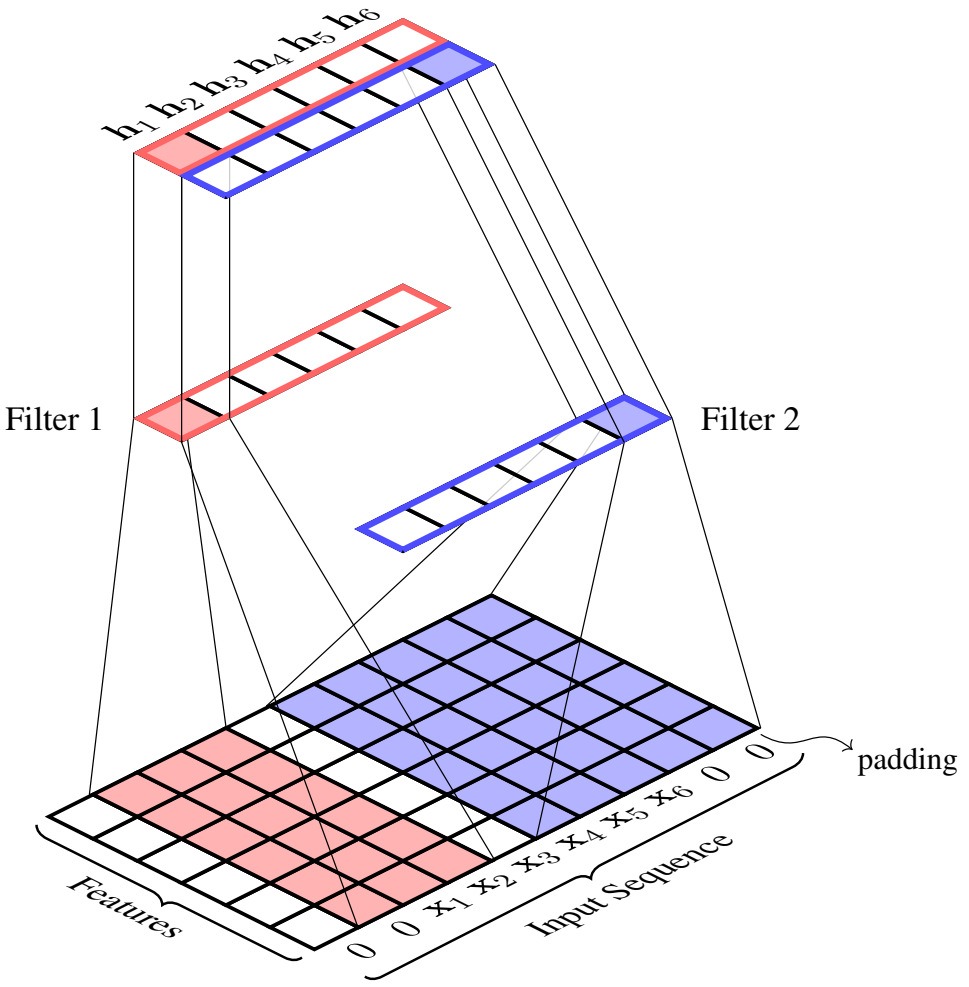
$$\mathbf{h}_i = \begin{bmatrix} v_i^{(1)} & \cdots & v_i^{(d_h)} \end{bmatrix}. \tag{4.45}$$

Chú giải của người dịch: Đọc Hình 4.7

Trong hình, *Features* là trục các chiều đặc trưng (6 chiều của mỗi vectơ từ), *Input Sequence* là trục vị trí trong chuỗi, *padding* là các từ đệm (toàn số 0). Mỗi bộ lọc "ôm" trọn cả 6 chiều đặc trưng và chỉ trượt dọc theo chuỗi. Mỗi bộ lọc cho ra *một số* tại mỗi vị trí; vì có 2 bộ lọc nên mỗi vị trí i nhận được 2 số, ghép lại thành vectơ $\mathbf{h}_i$ có $d_h = 2$ chiều (Phương trình (4.45)). Muốn $\mathbf{h}_i$ có 512 chiều thì dùng 512 bộ lọc.

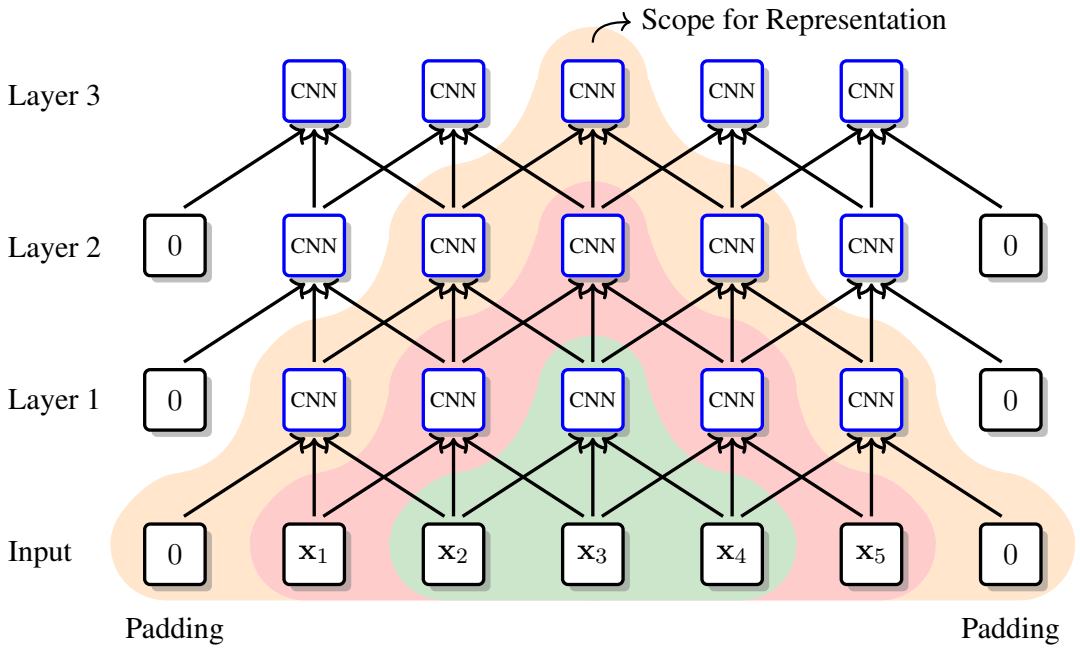
Nhiều hệ thống dựa trên CNN có ý nghĩa thực tiễn gồm hai tầng tích chập trở lên. Cách đơn giản nhất để làm điều này là xếp chồng các tầng, tương tự như RNN nhiều tầng (xem Mục 4.2.3). Cụ thể, chuỗi đầu ra của một tầng tích chập phía dưới được dùng trực tiếp làm đầu vào cho tầng tiếp theo. Hình 4.8 đưa ra một ví dụ về kiến trúc CNN gồm ba tầng tích chập xếp chồng. Một ưu điểm chính của CNN nhiều tầng là trường tiếp nhận của chúng được mở rộng theo cấp số nhân, dẫn đến phạm vi biểu diễn rộng hơn. Như minh họa trong Hình 4.8, trong khi một nơ-ron ở Tầng 1 chỉ tổng hợp thông tin từ ba vectơ đầu vào, một nơ-ron ở Tầng 3 tổng hợp gián tiếp thông tin trải trên bảy vectơ đầu vào. Vì các

[9] Với trường tiếp nhận kích thước $r \times d_e$, $\mathbf{a}_i$ được định nghĩa là $\{\mathbf{x}_{\lceil i - \frac{r}{2} \rceil}, \ldots, \mathbf{x}_{\lceil i + \frac{r}{2} - 1 \rceil}\}$ hoặc $\{\mathbf{x}_{\lfloor i - \frac{r}{2} \rfloor}, \ldots, \mathbf{x}_{\lfloor i + \frac{r}{2} - 1 \rfloor}\}$.



Hình 4.7: Hai bộ lọc được áp dụng lên một chuỗi vectơ từ. Đầu vào gồm mười vectơ từ (các từ $\mathbf{x}_1 \cdots \mathbf{x}_6$ và hai từ đệm ở mỗi đầu của chuỗi). Mỗi vectơ từ có 6 chiều, do đó đầu vào là một ma trận 10×6. Bộ lọc 1 (*Filter 1*) có trường tiếp nhận kích thước 3×6. Trượt nó trên ma trận đầu vào, ta thu được một chuỗi đầu ra, mỗi đầu ra ứng với một vị trí (tức là một chuỗi 8 đầu ra). Tương tự, ta áp dụng bộ lọc 2 (*Filter 2*) lên chuỗi đầu vào và thu được một chuỗi đầu ra khác. Hai chuỗi đầu ra sau đó được tổ chức thành một ma trận 2×6, trong đó vectơ hàng thứ i là $\mathbf{h}_i$.

nơ-ron ở tầng cao hơn xử lý tín hiệu từ một đoạn liên tiếp dài hơn nhiều của chuỗi gốc, chúng có khả năng trích xuất các biểu diễn ngữ nghĩa ở mức cao hơn và nắm bắt hiệu quả các phụ thuộc khoảng cách xa (*long-distance dependencies*).



Hình 4.8: Một CNN với 3 tầng tích chập (stride $= 1$ và $r = 3$). Mỗi tầng nhận một chuỗi vectơ và tạo ra một chuỗi vectơ khác. Trong quá trình này, một bộ lọc di chuyển trên đầu vào và thực hiện phép tích chập ở mỗi lần di chuyển. Ở tầng 1, trường tiếp nhận của bộ lọc là một vùng gồm ba phần tử đầu vào (xem vùng tô xanh lá). Tầng càng cao thì trường tiếp nhận của bộ lọc càng lớn. Ví dụ, ở tầng 3, một lần áp dụng bộ lọc có thể phủ tối đa toàn bộ chuỗi đầu vào (xem vùng tô màu cam). (*Layer*: tầng; *Input*: đầu vào; *Padding*: phần đệm; *Scope for Representation*: phạm vi biểu diễn.)

Chú giải của người dịch: Trường tiếp nhận lớn dần qua các tầng

Với bộ lọc rộng $r = 3$ và bước trượt 1, mỗi tầng mới "nhìn" thêm 1 phần tử ở mỗi bên. Tầng 1 thấy 3 phần tử, tầng 2 thấy $3 + 2 = 5$, tầng 3 thấy $5 + 2 = 7$; tổng quát, tầng L thấy $2L + 1$ phần tử đầu vào. Với các biến thể như tích chập giãn (*dilated convolution*), trong đó khoảng cách giữa các phần tử mà bộ lọc lấy vào được nhân đôi sau mỗi tầng, trường tiếp nhận còn có thể tăng gấp đôi qua mỗi tầng.

Trong một số ứng dụng, ta cần một biểu diễn có độ dài cố định và số chiều thấp cho toàn bộ chuỗi. Một cách phổ biến là thêm một tầng gộp (*pooling layer*) để hợp nhất $\{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$ thành một vectơ duy nhất. Ví dụ, ta có thể chọn giá trị lớn nhất (tức là gộp cực đại (*max-pooling*)) hoặc lấy trung bình các giá trị (tức là gộp trung bình (*average-pooling*)) theo từng chiều. Đây là một phương pháp chung trong học máy và áp dụng được cho hầu hết các mô hình chuỗi được bàn đến trong cuốn sách này.

Chú giải của người dịch: Ví dụ gộp

Giả sử chuỗi có 3 vị trí với $\mathbf{h}_1 = (1,\ 5)$, $\mathbf{h}_2 = (4,\ 2)$, $\mathbf{h}_3 = (3,\ -1)$. Gộp cực đại lấy số lớn nhất theo từng chiều: $(\max(1, 4, 3),\ \max(5, 2, -1)) = (4,\ 5)$. Gộp trung bình: $((1+4+3)/3,\ (5+2-1)/3) = (8/3,\ 2)$.

Dù chuỗi dài bao nhiêu, kết quả luôn là một vectơ có số chiều cố định (ở đây là 2).

4.4.3 Xử lý thông tin vị trí

Một tính chất hấp dẫn của CNN là khả năng cân bằng giữa độ phức tạp và hiệu quả bằng cách giới hạn kết nối đầy đủ trong một vùng cục bộ của dữ liệu đầu vào. Tính kết nối cục bộ này cho phép ta mô hình hóa một tầng tích chập về mặt toán học tương tự như một tầng kết nối đầy đủ: đầu ra của một nơ-ron là một biến đổi phi tuyến của tổng có trọng số các đầu vào của nó. Mặc dù kết nối cục bộ mang lại hiệu quả tính toán, một hạn chế của CNN tiêu chuẩn là tính bất biến đối với phép tịnh tiến (*translation invariance*). Dù nắm bắt được các cấu trúc cục bộ bên trong trường tiếp nhận, chúng không nhận biết được vị trí tuyệt đối mang tính toàn cục trong chuỗi.

Điều thú vị là, nếu chỉ giới hạn trong mô hình hóa chuỗi, điều này có vẻ không thành vấn đề vì đầu ra của mô hình bản thân nó cũng là một chuỗi. Có vẻ hợp lý khi giả định rằng chuỗi đầu ra giữ được thông tin thứ tự của chuỗi đầu vào. Tuy nhiên, việc áp dụng CNN cho dữ liệu chuỗi không bảo đảm một ánh xạ một-một chặt chẽ giữa các phần tử đầu vào và đầu ra. Về mặt kỹ thuật, $\mathbf{h}_i$ không đơn thuần là biểu diễn của $\mathbf{x}_i$. Thay vào đó, nó mã hóa một cửa sổ các đầu vào có tâm tại $\mathbf{x}_i$. Điều này làm vấn đề phức tạp lên đáng kể, vì rất khó suy ra từ $\mathbf{h}_i$ chính xác các đầu vào cục bộ đó ban đầu được sắp xếp theo thứ tự nào.

Chú giải của người dịch: Vì sao CNN "không biết" thứ tự?

Vì cùng một bộ lọc được dùng ở mọi vị trí, cụm "con mèo đen" cho ra cùng một đặc trưng dù nó xuất hiện ở đầu hay cuối câu — đó là tính bất biến đối với phép tịnh tiến. Hơn nữa, phép tích chập là tổng của các tích; nếu bộ lọc có các trọng số giống nhau thì đổi chỗ các từ trong cửa sổ không làm thay đổi kết quả (giống như $a + b + c = c + a + b$). Trong khi đó, với ngôn ngữ, "chó cắn người" và "người cắn chó" có nghĩa rất khác nhau, nên ta cần cách đưa thông tin vị trí vào mô hình một cách tường minh.

Mô hình hóa tường minh trật tự từ là rất quan trọng trong NLP, và đã được nghiên cứu rộng rãi trong các bài toán như dịch máy (*machine translation*) [Lopez, 2008; Koehn, 2010]. Với các mô hình dựa trên mạng nơ-ron, người ta có thể giải quyết vấn đề bằng cách dùng các kiến trúc mô hình nhạy với thứ tự như RNN. Một cách tiếp cận phổ biến hơn trong các hệ thống gần đây là phát triển một mô hình con **mã hóa vị trí** (*positional encoding*) và tích hợp nó vào các mô hình chuỗi sẵn có [Gehring và cộng sự, 2017b; Vaswani và cộng sự, 2017; Shaw và cộng sự, 2018; Dufter và cộng sự, 2022]. Một cách hình thức, ta nói rằng đầu vào tại vị trí i là sự kết hợp giữa đầu vào gốc $\mathbf{x}_i$ và mã hóa vị trí của i (ký hiệu là $\text{PE}(\cdot)$):

$$\mathbf{e}_i = \text{Merge}(\mathbf{x}_i, \text{PE}(i)), \tag{4.46}$$

trong đó $\text{Merge}(\cdot)$ định nghĩa một phép kết hợp. Việc dùng mã hóa vị trí rất đơn giản: ta chỉ cần thay $\{\mathbf{x}_1, \ldots, \mathbf{x}_m\}$ bằng $\{\mathbf{e}_1, \ldots, \mathbf{e}_m\}$ trong một mô hình chuỗi. Vì vậy, cách tiếp cận này *không phụ thuộc vào mô hình* (*model-agnostic*).

Trong tiểu mục này, chúng tôi trình bày một số phiên bản của $\text{PE}(\cdot)$ và các cách kết hợp chúng với $\mathbf{x}_i$. Lưu ý rằng thảo luận dưới đây không chỉ dành riêng cho CNN. Chúng tôi xét nó ở đây vì mã hóa vị trí hữu ích cho các mô hình không nhạy với thứ tự của đầu vào, và CNN là một ví dụ tốt để thấy nó được sử dụng như thế nào [Gehring và cộng sự, 2017a;b]. Trong Chương 6, chúng ta sẽ thấy một ứng dụng của mã hóa vị trí trong Transformer, một mô hình nơ-ron tiên tiến nhất trong nhiều lĩnh vực.

1. Mã hóa vị trí dựa trên độ lệch

Cách đơn giản nhất để biểu diễn một vị trí i là dùng chỉ số tuyệt đối của nó. Điều này có thể được biểu diễn dưới dạng "khoảng cách" tới một điểm tham chiếu

$$\text{PE}(i) = i - i_0, \tag{4.47}$$

trong đó i_0 là một số nguyên cho biết ta bắt đầu đếm từ đâu. Nếu $i_0 = 0$ thì $\text{PE}(i) = i$ cho ta vị trí tuyệt đối tiêu chuẩn. Lưu ý rằng $\text{PE}(i)$ có thể là số âm nếu $i_0 > i$. Dù không phải là khoảng cách vật lý theo nghĩa chặt chẽ, nó vẫn là một đặc trưng hoàn toàn hợp lệ đối với các thuật toán học máy. Để bắt buộc đại lượng này không âm, vế phải của Phương trình (4.47) có thể được lấy giá trị tuyệt đối:

$$\text{PE}(i) = |i - i_0|. \tag{4.48}$$

Coi vị trí là các số nguyên đơn thuần dẫn tới một mã hóa vị trí rời rạc và không bị chặn. Một phương pháp đáng mong muốn hơn có thể là dùng một biểu diễn liên tục trong một khoảng giá trị, vì nó cho phép hệ thống làm việc trong một không gian mẫu trơn và dễ tối ưu. Một cách đơn giản để làm điều này là chuẩn hóa. Ví dụ, chia $i - i_0$ cho một giá trị cực đại nào đó, ta được một phiên bản chuẩn hóa của mã hóa dựa trên độ lệch

$$\text{PE}(i) = \frac{i - i_0}{i_{\text{max}} - i_0}. \tag{4.49}$$

Ví dụ, ta có thể đặt $i_{\text{max}} =$ độ dài lớn nhất có thể có của chuỗi và $i_0 = 0$, để $\text{PE}(i)$ nhận giá trị trong $[0, 1]$. Một lựa chọn phổ biến khác là đặt $i_{\text{max}} = m$ (tức là độ dài của chuỗi đầu vào) và định nghĩa $\text{PE}(i)$ như một tỉ số có giá trị thay đổi khi m thay đổi.

Để tận dụng các vị trí dạng số vô hướng này, cách đơn giản là làm giàu các vectơ đầu vào gốc bằng cách thêm các chiều mới, miễn là chúng có thể được xem như các đặc trưng mới. Như vậy, $\mathbf{e}_i$ được cho bởi:

$$\mathbf{e}_i = [\mathbf{x}_i, \text{PE}(i)], \tag{4.50}$$

trong đó $[\cdot]$ biểu thị phép ghép nối (*concatenation*).

Chú giải của người dịch: Ví dụ số

Chuỗi có $m = 4$ từ, chọn $i_0 = 0$, $i_{\text{max}} = m = 4$. Theo Phương trình (4.49), các vị trí $1, 2, 3, 4$ được mã hóa thành 0.25, 0.5, 0.75, 1. Nếu $\mathbf{x}_2 = (0.3, -1.2)$ thì theo Phương trình (4.50), $\mathbf{e}_2 = (0.3, -1.2, 0.5)$: vectơ chỉ dài thêm một chiều chứa thông tin vị trí.

2. Mã hóa vị trí hình sin

Bước tiếp theo hiển nhiên là biểu diễn vị trí bằng vectơ thay vì số vô hướng. Mặc dù vectơ hóa biểu diễn vị trí nghe có vẻ phức tạp, một ý tưởng đơn giản là dùng một *hệ đếm có nhớ* (*carrying system*), mô tả cách một số tự nhiên được biểu diễn bằng một đa thức theo một cơ số [Kernes, 2021]. Ví dụ, i có thể được viết là

$$i = \sum_{k=0}^{k_{\text{max}}} a(i, k)\, b^k, \tag{4.51}$$

trong đó $a(i, k)$ là chữ số thứ k, $k_{\text{max}} + 1$ là số chữ số tối đa, và b là cơ số của hệ. Việc nhớ sang hàng tiếp theo xảy ra khi $a(i, k)$ đạt tới b: ta tăng $a(i, k + 1)$ thêm 1 và đưa $a(i, k)$ trở về 0. Theo cách này, ta có thể làm $a(i, k)$ thay đổi với chu kỳ b^k, tức là $a(i, 0)$ thay đổi với chu kỳ b^0, $a(i, 1)$ thay đổi với chu kỳ b^1, $a(i, 2)$ thay đổi với chu kỳ b^2, và cứ thế.

Dùng hệ này, i có thể được biểu diễn thành một vectơ

$$\text{PE}(i) = \begin{bmatrix} a(i,0) & a(i,1) & \cdots & a(i,k_{\text{max}}) \end{bmatrix}. \tag{4.52}$$

Ví dụ, khi $b = 2$ thì $\text{PE}(11) = \begin{bmatrix} 1 & 1 & 0 & 1 \end{bmatrix}$. Tuy nhiên, trong Phương trình (4.52), $\text{PE}(i)$ vẫn là một hàm rời rạc. Như đã bàn xuyên suốt cuốn sách này, ta có thể muốn có một biểu diễn vectơ liên tục, có khả năng mô tả các trạng thái trung gian giữa những sự kiện rời rạc. Coi $a(i,k)$ là một hàm tuần hoàn, một lựa chọn phổ biến là hàm **sin**. Như vậy, $a(i,k)$ có thể được định nghĩa lại như sau:

$$a(i,k) = \sin(i \cdot \omega_k). \tag{4.53}$$

Hàm này có biên độ bằng 1 và chu kỳ $\frac{2\pi}{\omega_k}$. Dùng một dạng chu kỳ tương tự như trong Phương trình (4.51), ta định nghĩa ω_k là

$$\omega_k = \frac{1}{(b_{\text{model}})^{k/d_{\text{model}}}}, \tag{4.54}$$

trong đó $b_{\text{model}} > 0$ và $d_{\text{model}} > 0$ là các siêu tham số của mô hình. Rõ ràng ta có $\frac{2\pi}{\omega_0} < \frac{2\pi}{\omega_1} < \cdots < \frac{2\pi}{\omega_{k_{\text{max}}}}$.

Chú giải của người dịch: Từ hệ nhị phân đến hàm sin

Viết số 11 trong hệ nhị phân: $11 = 1 \cdot 2^0 + 1 \cdot 2^1 + 0 \cdot 2^2 + 1 \cdot 2^3$, nên vectơ chữ số là $(1, 1, 0, 1)$ (đọc từ hàng đơn vị lên). Khi đếm $0, 1, 2, 3, \ldots$, chữ số hàng 2^0 đổi giá trị sau mỗi 1 bước, hàng 2^1 sau mỗi 2 bước, hàng 2^2 sau mỗi 4 bước… — hàng càng cao "dao động" càng chậm. Mã hóa hình sin giữ đúng ý tưởng này nhưng thay các chữ số nhảy bậc 0/1 bằng các sóng sin mượt, mỗi chiều k một sóng có chu kỳ $2\pi/\omega_k = 2\pi\,(b_{\text{model}})^{k/d_{\text{model}}}$ (nhớ lại ở lớp 11: hàm $\sin(\omega x)$ có chu kỳ $2\pi/\omega$). Vì $b_{\text{model}} > 1$ trong thực tế, k càng lớn thì chu kỳ càng dài.

Tương tự, ta có thể định nghĩa $a(i,k)$ thông qua hàm **cos**

$$a(i,k) = \cos(i \cdot \omega_k). \tag{4.55}$$

Kết hợp cả hai Phương trình (4.53) và (4.55), ta tạo ra một biểu diễn mới của i như sau:

$$\text{PE}(i) = \begin{bmatrix} \sin(i \cdot \omega_0) & \cos(i \cdot \omega_0) & \cdots & \sin(i \cdot \omega_{k_{\text{max}}}) & \cos(i \cdot \omega_{k_{\text{max}}}) \end{bmatrix}. \tag{4.56}$$

Vaswani và cộng sự [2017] đã cụ thể hóa dạng trên bằng cách đặt $b_{\text{model}} = 10{,}000$. Gọi $\text{PE}(i,k)$ là chiều thứ k của $\text{PE}(i)$. Phiên bản mã hóa vị trí của Vaswani và cộng sự [2017] được viết là

$$\text{PE}(i, 2k) = \sin\left(i \cdot \frac{1}{10000^{2k/d_{\text{model}}}}\right), \tag{4.57}$$

$$\text{PE}(i, 2k+1) = \cos\left(i \cdot \frac{1}{10000^{2k/d_{\text{model}}}}\right). \tag{4.58}$$

Việc chọn $b_{\text{model}} = 10{,}000$ là theo kinh nghiệm. Ta có thể điều chỉnh nó cho từng bài toán cụ thể. Hình 4.9 vẽ mã hóa vị trí cho các vị trí khác nhau. Ta thấy rằng khi k lớn dần, sự thay đổi màu sắc diễn ra theo chu kỳ dài hơn.

Lưu ý rằng Phương trình (6.14) và (6.15)[10] có một tính chất hữu ích là $\text{PE}(i+\mu)$ có thể dễ dàng được biểu diễn bằng một hàm tuyến tính của $\text{PE}(i)$ với một độ lệch μ cho trước[11]

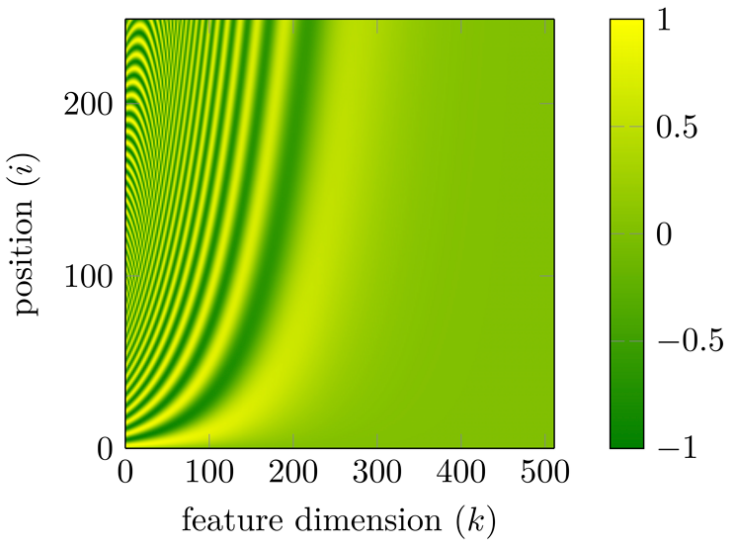
$$\text{PE}(i+\mu, 2k) = \text{PE}(i, 2k) \cdot \text{PE}(\mu, 2k+1) +$$

[10] **[ND]** Đây chính là hai công thức (4.57) và (4.58) ở trên; chúng được trình bày lại trong Chương 6 với số hiệu (6.14) và (6.15).

[11] Ta có thể suy ra các công thức này bằng cách dùng

$$\sin(\alpha+\beta) = \sin(\alpha) \cdot \cos(\beta) + \cos(\alpha) \cdot \sin(\beta), \tag{4.59}$$

$$\cos(\alpha+\beta) = \cos(\alpha) \cdot \cos(\beta) - \sin(\alpha) \cdot \sin(\beta). \tag{4.60}$$



Hình 4.9: Bản đồ nhiệt của mô hình nhúng vị trí theo Phương trình (6.14) và (6.15) ($b_{\text{model}} = 10{,}000$ và $d_{\text{model}} = 512$). Xét một vị trí i (tức là hàng thứ i), rồi di chuyển một vị trí khác j từ i lên trên hoặc xuống dưới. Một cách trực quan, khi i và j càng gần nhau thì các vectơ hàng tương ứng càng giống nhau. Ngược lại, khi j di chuyển ra xa i, sự giống nhau không còn rõ rệt như vậy. Tính chất này giúp giải thích ý tưởng đằng sau mô hình nhúng vị trí: "khoảng cách" giữa hai vị trí được mô hình hóa một cách ngầm định bằng cách so sánh các biểu diễn nhiều chiều của chúng. (Trục tung: vị trí i; trục hoành: chiều đặc trưng k; thang màu từ -1 đến 1.)

$$\text{PE}(i, 2k+1) \cdot \text{PE}(\mu, 2k), \tag{4.61}$$

$$\text{PE}(i+\mu, 2k+1) = \text{PE}(i, 2k+1) \cdot \text{PE}(\mu, 2k+1) - \text{PE}(i, 2k) \cdot \text{PE}(\mu, 2k). \tag{4.62}$$

Tính chất này có lợi vì nó cho phép mô hình chú ý tới các vị trí tương đối. Nghĩa là, trạng thái tại vị trí $i + \mu$ có thể được mô tả bằng cách bắt đầu từ i rồi cộng thêm độ lệch μ.

Chú giải của người dịch: Công thức cộng lượng giác ở đây

Đặt $\omega = 1/10000^{2k/d_{\text{model}}}$. Khi đó $\text{PE}(i, 2k) = \sin(i\omega)$, $\text{PE}(i, 2k+1) = \cos(i\omega)$. Áp dụng công thức cộng lớp 11 với $\alpha = i\omega$, $\beta = \mu\omega$: $\sin((i+\mu)\omega) = \sin(i\omega)\cos(\mu\omega) + \cos(i\omega)\sin(\mu\omega)$ — đó chính là Phương trình (4.61). Điểm mấu chốt: với μ cố định, $\cos(\mu\omega)$ và $\sin(\mu\omega)$ chỉ là các hằng số, nên mã hóa của vị trí $i + \mu$ là một tổ hợp tuyến tính (nhân với hằng số rồi cộng) của mã hóa của vị trí i. Nhờ vậy mô hình dễ "học" được quan hệ kiểu "cách nhau μ vị trí".

Khi áp dụng mã hóa vị trí hình sin, một cách là dùng Phương trình (4.50) để ghép nối $\mathbf{x}_i$ và $\text{PE}(i)$. Trong công trình của Vaswani và cộng sự [2017], thay vào đó họ giả định $\text{PE}(i)$ là một vectơ có cùng kích thước với $\mathbf{x}_i$ (tức là $|\text{PE}(i)| = |\mathbf{x}_i| = d_e$), và cộng $\text{PE}(i)$ vào $\mathbf{x}_i$

$$\mathbf{e}_i = \mathbf{x}_i + \text{PE}(i). \tag{4.63}$$

Mô hình cộng dạng hình sin này đã là nền tảng của nhiều cách tiếp cận mã hóa vị trí [Dehghani và cộng sự, 2018; Likhomanenko và cộng sự, 2021; Su và cộng sự, 2021].

3. Mã hóa vị trí học được

Việc áp dụng mã hóa vị trí hình sin cho ta một bảng tra (*lookup table*) $\mathbf{C}_{\text{PE}} \in \mathbb{R}^{m_{\text{max}} \times d_e}$ (trong đó m_{max} là độ dài chuỗi lớn nhất)

$$\mathbf{C}_{\text{PE}} = \begin{bmatrix} \text{PE}(1) \\ \vdots \\ \text{PE}(m_{\text{max}}) \end{bmatrix}. \tag{4.64}$$

Trong bảng này, mỗi vectơ hàng $\text{PE}(i)$ tương ứng với vectơ nhúng của một vị trí i. Như đã mô tả ở trên, các vectơ hình sin này là tất định và được suy ra từ các hàm toán học cố định. Một cách tiếp cận khác là coi các vectơ vị trí là tham số của mô hình và học chúng như thường lệ. Trong trường hợp này, cả vectơ nhúng từ và vectơ nhúng vị trí đều có thể được huấn luyện theo cùng một cách. Xem Chương 2 và 3 để biết thêm về việc học vectơ nhúng từ trong các mô hình ngôn ngữ nơ-ron.

Một lưu ý cuối cùng về mã hóa vị trí. Những gì chúng tôi trình bày trong mục này nhìn chung có thể được xếp vào mô thức **mã hóa vị trí tuyệt đối** (*absolute positional encoding*): một vị trí được mô tả bằng chỗ của nó trong một hệ tọa độ. Một khái niệm khác đáng tìm hiểu là **mã hóa vị trí tương đối** (*relative positional encoding*) [Shaw và cộng sự, 2018]. Ví dụ, ta có thể mở rộng Phương trình (4.48) để định nghĩa khoảng cách giữa hai vị trí i và j

$$\text{PE}(i, j) = |i - j|. \tag{4.65}$$

Trong trường hợp này, mã hóa vị trí không còn là một thuộc tính của đầu vào thứ i, mà là một biểu diễn nào đó của khoảng cách so với một vị trí tham chiếu j. Thực ra, hầu hết các phương pháp mã hóa vị trí tương đối đều là các biến thể xoay quanh một chủ đề chung, trong đó các vị trí được mô tả bằng quan hệ theo từng cặp của chúng. Thiên kiến quy nạp mang tính quan hệ này tạo nên nền tảng của một số kiến trúc tiên tiến, sẽ được trình bày chi tiết trong Chương 6 và 8.

Chú giải của người dịch: Tuyệt đối và tương đối

Ví dụ đời thường: "nhà tôi ở số 25 phố Huế" là cách mô tả *tuyệt đối*; "nhà tôi cách nhà bạn 3 căn" là cách mô tả *tương đối*. Với ngôn ngữ, điều quan trọng thường là quan hệ giữa hai từ (tính từ đứng ngay sau danh từ, động từ cách chủ ngữ 2 từ…) hơn là việc một từ đứng ở vị trí thứ 37 trong văn bản; đó là lý do mã hóa vị trí tương đối được quan tâm nhiều.

4.5 Các ví dụ

Cả mô hình hồi quy và mô hình tích chập đều đã được dùng thành công trong rất nhiều ứng dụng. Ở đây chúng ta bàn về một vài ví dụ thú vị. Dù các mô hình này phần lớn là cơ bản, chúng tạo nền móng cho nhiều hệ thống tiên tiến nhất.

4.5.1 Phân loại văn bản

Để minh họa cách dùng các mô hình chuỗi, trước hết ta xét bài toán phân loại văn bản (*text classification*), trong đó ta gán một lớp định sẵn cho một chuỗi văn bản cho trước. Khung bài toán này mở rộng một cách tự nhiên sang rất nhiều bài toán NLP, bao gồm phân loại bài báo, phân tích cảm xúc, nhận diện thư rác, phát hiện bình luận lừa đảo, v.v.

Trong phân loại văn bản, ta quan tâm tới việc dự đoán nhãn lớp từ một tập C, khi cho trước một chuỗi từ $w_1 \cdots w_m$:

$$\hat{c} = \arg\max_{c \in C} \text{Score}(c, w_1 \cdots w_m). \tag{4.66}$$

Ở đây $\text{Score}(c, w_1 \cdots w_m)$ đo mức độ phù hợp khi dự đoán lớp c cho chuỗi đầu vào $w_1 \cdots w_m$. Ta ánh xạ chuỗi từ thành chuỗi vectơ từ (hay vectơ nhúng từ), tức là $w_1 \cdots w_m \to \mathbf{x}_1 \cdots \mathbf{x}_m$. Giả sử $\text{Score}(\cdot)$ là một hàm xác suất mô tả phân phối của các lớp, ta có thể phát biểu lại bài toán thành

$$\begin{aligned} \hat{c} &= \arg\max_{c \in C} \Pr(c | w_1 \cdots w_m) \\ &= \arg\max_{c \in C} \Pr(c | \mathbf{x}_1 \cdots \mathbf{x}_m). \end{aligned} \tag{4.67}$$

Vấn đề trung tâm ở đây là mô hình hóa $\Pr(c|\mathbf{x}_1 \cdots \mathbf{x}_m)$. Ta định nghĩa $\Pr(c|\mathbf{x}_1 \cdots \mathbf{x}_m)$ theo khung *bộ mã hóa – bộ dự đoán* tổng quát như sau

- Đầu vào $\mathbf{x}_1 \cdots \mathbf{x}_m$ được biểu diễn thành một vectơ đặc trưng $\mathbf{H} \in \mathbb{R}^{d_h}$ nhờ một bộ mã hóa (*encoder*)

$$\mathbf{H} = \text{Encode}(\mathbf{x}_1 \cdots \mathbf{x}_m). \tag{4.68}$$

- $\mathbf{H}$ được đưa vào một tầng Softmax tiêu chuẩn để dự đoán phân phối các lớp

$$\Pr(\cdot|\mathbf{H}) = \text{Softmax}(\mathbf{H} \cdot \mathbf{U} + \mathbf{b}), \tag{4.69}$$

  trong đó $\mathbf{U} \in \mathbb{R}^{d_h \times |C|}$ và $\mathbf{b} \in \mathbb{R}^{|C|}$ là các tham số của mô hình.

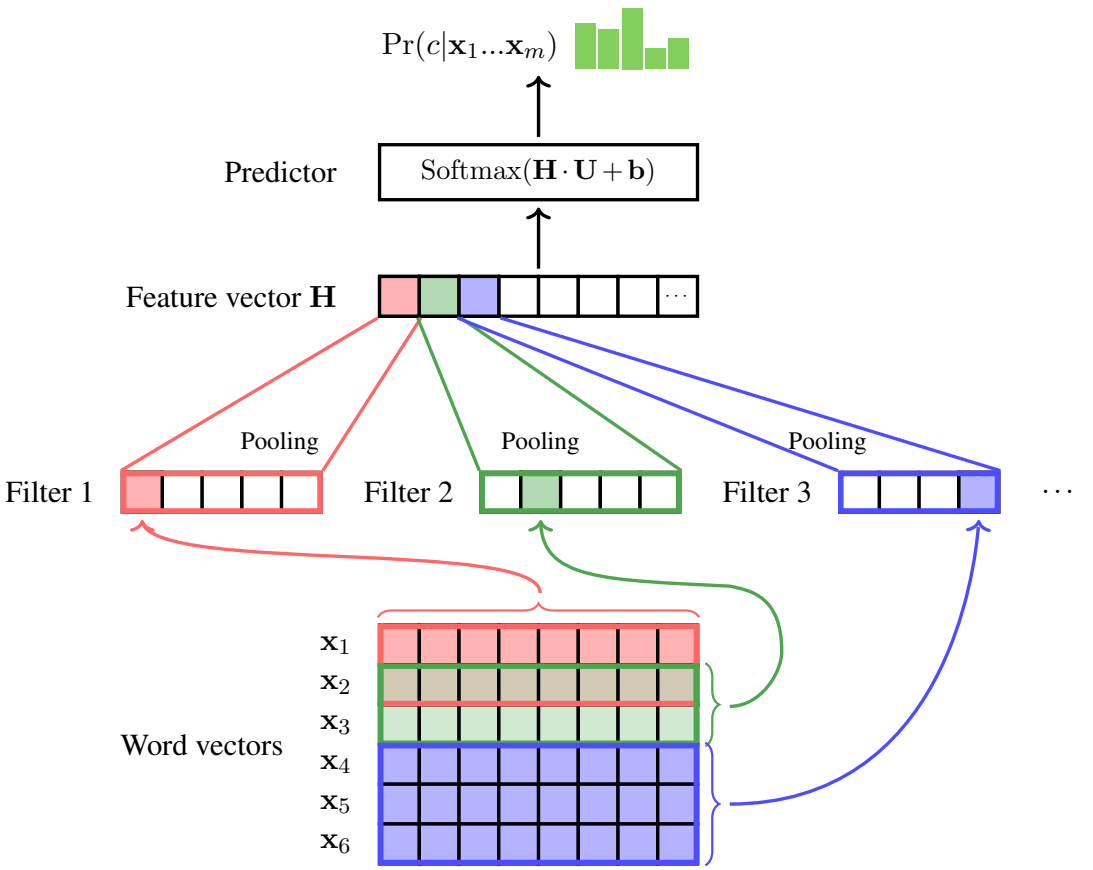
Chú giải của người dịch: Ví dụ phân loại cảm xúc

Giả sử $C = \{\text{tích cực}, \text{ tiêu cực}\}$, nên $|C| = 2$. Bộ mã hóa nén cả câu "Phim này hay quá" thành một vectơ $\mathbf{H}$ (chẳng hạn $d_h = 100$ số). Nhân với $\mathbf{U}$ (kích thước 100×2) rồi cộng $\mathbf{b}$ cho ra 2 "điểm số", ví dụ $(2.0,\ 0.0)$. Softmax biến chúng thành xác suất: $e^2/(e^2 + e^0) \approx 0.88$ cho "tích cực" và ≈ 0.12 cho "tiêu cực". Phép arg max trong Phương trình (4.67) đơn giản là chọn lớp có xác suất lớn nhất: "tích cực".

Thành phần $\text{Encode}(\cdot)$ giống như các kiến trúc đã bàn ở những mục trước. Do đó, rất nhiều mô hình mã hóa chuỗi có thể được áp dụng trực tiếp ở đây. Ví dụ, xét bộ mã hóa dựa trên CNN được trình bày trong công trình của Kim [2014]. Mô hình của Kim [2014] dựa trên một tầng tích chập duy nhất gồm d_h bộ lọc. Việc áp dụng một bộ lọc tạo ra một tập đặc trưng, mỗi đặc trưng gắn với một vị trí của chuỗi. Vì ta muốn có một vectơ duy nhất để biểu diễn toàn bộ chuỗi, một tầng gộp được thêm vào để số đặc trưng của mỗi bộ lọc giảm xuống còn một. Sau đó, với mọi c, xác suất $\Pr(c|\mathbf{H})$ có thể được tính trực tiếp theo Phương trình (4.69). Xem Hình 4.10 để thấy minh họa của bộ phân loại này.

Huấn luyện một mô hình như vậy bao gồm việc tối ưu một hàm mất mát định sẵn, và như đã nhắc nhiều lần trong sách, một trong những phương pháp phổ biến nhất là cực tiểu hóa mất mát entropy chéo bằng hạ gradient. Ngoài ra, ta có thể dùng chính quy hóa để cải thiện việc huấn luyện CNN. Chi tiết về các kỹ thuật này có thể xem ở Chương 2.

Đáng lưu ý là dù mô hình cơ sở này đơn giản, nó vẫn là một trong những kiến trúc hiệu quả nhất cho phân loại văn bản. Có nhiều cải tiến kiến trúc. Các mở rộng nổi bật của khung này gồm CNN sâu [Conneau và cộng sự, 2017c], CNN mức ký tự [Santos và Gatti, 2014; Zhang và cộng sự, 2015], và các mô hình lai hồi quy–tích chập [Lai và cộng sự, 2015].



Hình 4.10: Một bộ phân loại văn bản dựa trên CNN [Kim, 2014]. Đầu vào là một chuỗi vectơ từ (*Word vectors*). Một tầng tích chập gồm nhiều bộ lọc (*Filter*) được dùng để trích xuất đặc trưng theo các chiều khác nhau. Một tầng gộp (*Pooling*) được dùng để giảm số lượng đặc trưng biểu diễn văn bản đầu vào, dẫn tới một vectơ đặc trưng (*Feature vector*) **H** có số chiều thấp. Việc dự đoán được thực hiện có điều kiện trên **H** bằng một tầng Softmax tiêu chuẩn (*Predictor*: bộ dự đoán).

4.5.2 Nhận dạng tiếng nói đầu-cuối

Nhận dạng tiếng nói (*speech recognition*) là bài toán ánh xạ một chuỗi tín hiệu âm thanh thành một chuỗi từ hoặc ký tự (gọi là **bản phiên âm** (*transcription*)) [Reddy, 1976; Rabiner và Juang, 1993]. Vì đầu vào gốc là một dạng sóng âm theo miền thời gian, cách làm tiêu chuẩn là biến đổi nó thành một chuỗi các đoạn sóng (gọi là **khung** (*frames*)). Thông thường, một khung được biểu diễn bằng một vectơ đặc trưng, ký hiệu là $\mathbf{x}_i$. Điều này được thực hiện bằng cách dùng hoặc các hàm đặc trưng được thiết kế thủ công từ lĩnh vực xử lý tín hiệu [Davis và Mermelstein, 1980; Picone, 1993; Campbell, 1997], hoặc các vectơ nhúng nơ-ron học được [Chorowski và cộng sự, 2019; Schneider và cộng sự, 2019]. Về phía đầu ra, các hệ thống nhận dạng tiếng nói nói chung không trực tiếp sinh ra các từ hoàn chỉnh. Thay vào đó, chúng xuất ra chuỗi các **đơn vị phiên âm** (*transcription units*) (hay **nhãn phiên âm** (*transcription labels*)), chẳng hạn âm vị, ký tự hoặc từ con. Trong tiểu mục này, ta giả định đầu ra của hệ thống là một chuỗi ký tự, ký hiệu là $y_1, \ldots, y_n \in V_y^n$. Bộ từ vựng V_y gồm các chữ cái tiếng Anh tiêu chuẩn (a − z), chữ số (0 − 9), dấu cách (⟨sp⟩), dấu chấm (⟨pe⟩) và các dấu câu khác. Giống như hầu hết các hệ thống nhận dạng tiếng nói hiện đại, ta thêm một ký hiệu trống ϵ vào bảng chữ cái để biểu thị đầu ra rỗng.

Mục tiêu ở đây là tìm một chuỗi ký tự $\hat{y}_1 \cdots \hat{y}_n$ cho một chuỗi đầu vào $\mathbf{x}_1 \cdots \mathbf{x}_m$ cho trước, sao cho

$$\hat{y}_1 \cdots \hat{y}_n = \underset{y_1 \cdots y_n}{\arg\max} \Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m). \tag{4.70}$$

Bài toán này khó hơn bài toán phân loại ở tiểu mục trước, vì đầu ra có thể là một chuỗi ký tự bất kỳ chứ không phải một lớp trong một tập lớp định sẵn. Bài toán sinh chuỗi dẫn tới hai khó khăn. Thứ nhất, ta cần một cơ chế nào đó để mô hình hóa $\Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m)$ cho một số lượng cặp chuỗi đầu vào–đầu ra lớn theo cấp số mũ. Thứ hai, trong thực tế $y_1 \cdots y_n$ thường ngắn hơn nhiều so với $\mathbf{x}_1 \cdots \mathbf{x}_m$ (tức là $n < m$), nên ta cần một cơ chế để gióng một chuỗi dài với một chuỗi ngắn. Tuy nhiên, ta không cần xét các khó khăn này ở giai đoạn biểu diễn chuỗi đầu vào, và vẫn có thể mã hóa chuỗi đầu vào $\mathbf{x}_1 \cdots \mathbf{x}_m$ theo cùng cách như các mô hình chuỗi khác. Cụ thể, ta biểu diễn $\mathbf{x}_1 \cdots \mathbf{x}_m$ dưới dạng

$$\mathbf{h}_1 \cdots \mathbf{h}_m = \text{Encode}(\mathbf{x}_1 \cdots \mathbf{x}_m). \tag{4.71}$$

Bộ mã hóa có thể là RNN, CNN, hoặc các mô hình tiên tiến hơn (như Transformer). Ở đây ta xét kiến trúc bộ mã hóa được dùng trong công trình của Graves và cộng sự [2013b] và Graves và Jaitly [2014]. Đó là một mô hình LSTM hai chiều nhiều tầng. Chúng tôi bỏ qua chi tiết của mô hình này mà không làm mất mạch trình bày, vì bạn đọc có lẽ đã quen với nó qua các Mục 4.2.3, 4.2.4 và 4.3.2.

Sau khi có biểu diễn chuỗi $\mathbf{H} = \mathbf{h}_1 \cdots \mathbf{h}_m$, một tầng Softmax được dùng để ánh xạ mỗi $\mathbf{h}_i \in \mathbb{R}^{d_h}$ thành một phân phối trên các nhãn phiên âm, cho bởi:

$$\Pr(\cdot | \mathbf{h}_i) = \text{Softmax}(\mathbf{h}_i \cdot \mathbf{U} + \mathbf{b}), \tag{4.72}$$

trong đó $\mathbf{U} \in \mathbb{R}^{d_h \times |V_y|}$ và $\mathbf{b} \in \mathbb{R}^{|V_y|}$ là các tham số mô hình. $\Pr(\cdot | \mathbf{h}_i) \in \mathbb{R}^{|V_y|}$ là một phân phối xác suất trên V_y, và xác suất của nhãn phiên âm l_i tại vị trí i đơn giản là $\Pr(l_i | \mathbf{h}_i)$. Khi đó ta có thể viết xác suất của một chuỗi nhãn dưới dạng

$$\Pr(l_1 \cdots l_m | \mathbf{H}) = \prod_{i=1}^{m} \Pr(l_i | \mathbf{h}_i). \tag{4.73}$$

Ta có thể áp dụng phép arg max để tìm chuỗi nhãn có xác suất cao nhất như thường lệ. Tuy nhiên, $l_1 \cdots l_m$ không thể dùng trực tiếp làm đầu ra của hệ thống, vì nó thường chứa nhiều ký hiệu trùng lặp

và ký hiệu trống. Để hậu xử lý $l_1 \cdots l_m$, trước hết ta gộp các nhãn giống nhau liên tiếp thành một nhãn duy nhất, rồi xóa các ký hiệu trống. Ví dụ, xét chuỗi nhãn

$$\text{s s } \epsilon \text{ e e } \epsilon \text{ } \epsilon \text{ e } \langle\text{sp}\rangle \text{ y } \epsilon \text{ o o u.}$$

Bằng cách gộp "s s", "e e" và "o o", ta có

$$\text{s } \epsilon \text{ e } \epsilon \text{ } \epsilon \text{ e } \langle\text{sp}\rangle \text{ y } \epsilon \text{ o u.}$$

Sau đó, ta xóa mọi ϵ và thu được

$$\text{s e e } \langle\text{sp}\rangle \text{ y o u.}$$

Chuỗi trên chính là cái mà ta gọi là một bản phiên âm ("see you" — "hẹn gặp lại").

Chú giải của người dịch: Vì sao cần ký hiệu trống ϵ?

Âm thanh được cắt thành rất nhiều khung (ví dụ 100 khung mỗi giây), nên một chữ thường kéo dài qua nhiều khung liên tiếp: "s s" chỉ là một chữ s được nói kéo dài. Vì vậy ta gộp các nhãn lặp liền nhau. Nhưng từ "see" có hai chữ e đứng cạnh nhau thật sự; nếu chỉ gộp thì "e e e" sẽ thành một chữ e duy nhất. Ký hiệu trống ϵ đóng vai trò "vách ngăn": "e ϵ e" sau khi gộp và xóa ϵ vẫn giữ được hai chữ e. Ngoài ra, ϵ còn dùng cho những khung im lặng hoặc không ứng với chữ nào.

Rõ ràng, các chuỗi nhãn khác nhau có thể ứng với cùng một bản phiên âm. Gọi $B(y_1 \cdots y_n)$ là tập các chuỗi nhãn ứng với $y_1 \cdots y_n$[12]. Bây giờ ta chuyển sang dạng sau của xác suất phiên âm (xem minh họa ở Hình 4.11)

$$\Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m) = \sum_{l_1 \cdots l_m \in B(y_1 \cdots y_n)} \Pr(l_1 \cdots l_m | \mathbf{H}). \tag{4.74}$$

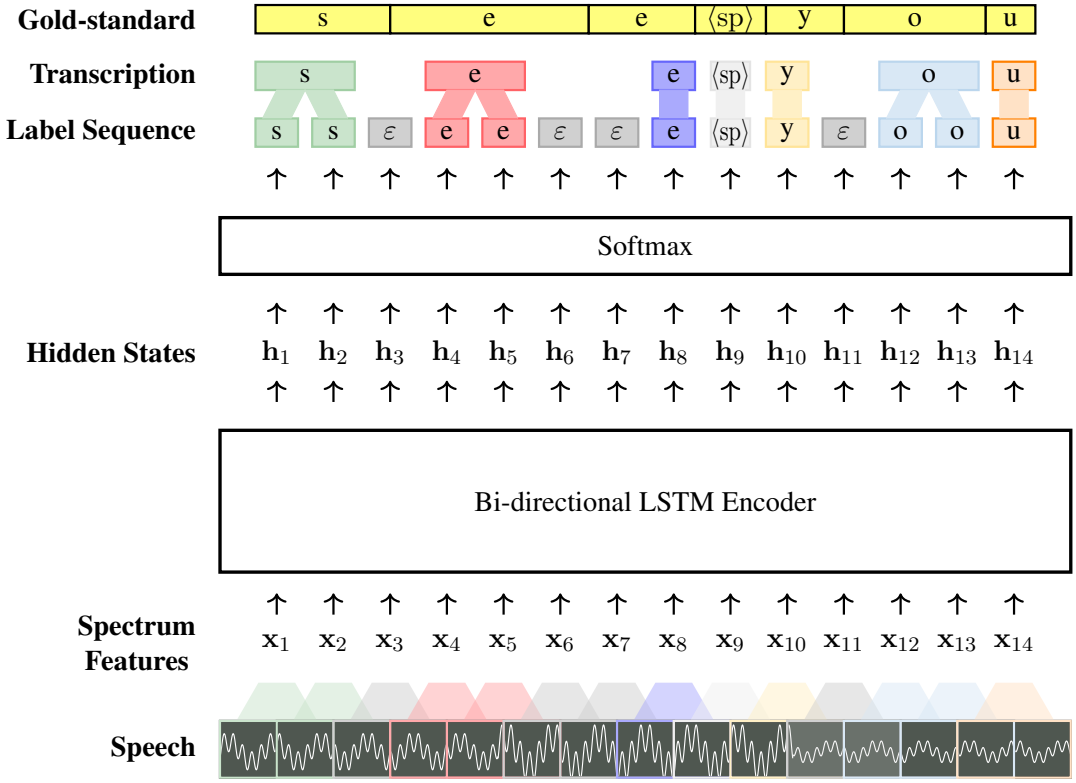
Một vấn đề của mô hình này là số chuỗi trong $B(y_1 \cdots y_n)$ tăng theo cấp số mũ với n (và m). May mắn là có những phương pháp rất hiệu quả để tính $\sum_{l_1 \cdots l_m \in B(y_1 \cdots y_n)} \Pr(l_1 \cdots l_m | \mathbf{H})$. Xem [Graves và cộng sự, 2006] để biết một thuật toán quy hoạch động (*dynamic programming*) giải bài toán này.

Lưu ý rằng Phương trình (4.74) còn được biết đến như một dạng của **phân loại thời gian kết nối** (*connectionist temporal classification*) (**CTC**) [Graves và cộng sự, 2006]. Đây là một trong những phương pháp được dùng rộng rãi nhất để huấn luyện các hệ thống nhận dạng tiếng nói và dịch tiếng nói đầu-cuối. Một ưu điểm của CTC là nó cho phép gióng bất kỳ chuỗi nhãn nào với một bản phiên âm theo cách rất đơn giản và hiệu quả. Việc dùng CTC để huấn luyện một hệ thống nhận dạng tiếng nói khá dễ dàng. Giả sử có một tập các cặp gồm chuỗi đầu vào và bản phiên âm, ký hiệu là $\mathcal{D}$. Một mục tiêu huấn luyện phổ biến là cực đại hóa hợp lý của các bản phiên âm này khi biết các đầu vào tương ứng, được viết là

$$\hat{\theta} = \arg\max_{\theta} \sum_{(y_1 \cdots y_n, \mathbf{x}_1 \cdots \mathbf{x}_m) \in \mathcal{D}} \log \Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m, \theta), \tag{4.75}$$

trong đó $\Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m, \theta)$ là xác suất được tính qua Phương trình (4.74), và θ là các tham số mô hình.

[12] $B(y_1 \cdots y_n)$ có thể chứa các chuỗi nhãn có độ dài bất kỳ. Tuy nhiên, nếu ta giới hạn đầu vào là chuỗi $\mathbf{x}_1 \cdots \mathbf{x}_m$ thì mỗi chuỗi trong $B(y_1 \cdots y_n)$ có độ dài m.



Hình 4.11: Một kiến trúc nhận dạng tiếng nói đầu-cuối. Đầu vào của hệ thống là một chuỗi tín hiệu âm thanh (*Speech*) được biểu diễn thành một chuỗi vectơ đặc trưng (*Spectrum Features*: đặc trưng phổ, tức là $\mathbf{x}_1 \cdots \mathbf{x}_{14}$). Các vectơ đặc trưng này được đưa vào một bộ mã hóa LSTM hai chiều (*Bi-directional LSTM Encoder*). Đầu ra của bộ mã hóa là một chuỗi các biểu diễn theo ngữ cảnh (*Hidden States*: trạng thái ẩn, tức là $\mathbf{h}_1 \cdots \mathbf{h}_{14}$), sau đó được đưa vào một tầng Softmax để sinh một phân phối nhãn tại mỗi vị trí. Từ các phân phối này, ta có thể sinh ra một chuỗi nhãn (*Label Sequence*). Ta ánh xạ mỗi chuỗi nhãn thành một dạng đầu ra cuối cùng (*Transcription*: bản phiên âm) bằng cách loại bỏ các ký hiệu trùng lặp và ký hiệu trống. Một đầu ra của hệ thống ứng với nhiều chuỗi nhãn, và xác suất của đầu ra là tổng xác suất của các chuỗi nhãn này. (*Gold-standard*: bản phiên âm chuẩn.)

Chú giải của người dịch: Một ví dụ nhỏ về CTC

Giả sử chỉ có $m = 3$ khung và bản phiên âm là "a". Các chuỗi nhãn dài 3 thu gọn được thành "a" gồm: a a a, a a ϵ, a ϵ ϵ, ϵ a a, ϵ a ϵ, ϵ ϵ a — tổng cộng 6 chuỗi (lưu ý a ϵ a thu gọn thành "aa" nên không tính). Theo Phương trình (4.74), Pr("a") là *tổng* xác suất của 6 chuỗi này, mỗi chuỗi có xác suất là tích các xác suất theo từng khung (Phương trình (4.73)). Ta không cần biết trước chữ "a" được nói ở khung nào — mô hình tự "gióng" bằng cách cộng mọi khả năng. Khi m lớn, số khả năng tăng rất nhanh nên cần quy hoạch động để tính tổng mà không phải liệt kê từng chuỗi.

Khi kiểm tra trên dữ liệu mới, ta tìm một bản phiên âm tối ưu như trong Phương trình (4.70). Quá trình này, còn gọi là **giải mã** (*decoding*), nói chung cần các thuật toán tìm kiếm được tối ưu hóa và các kỹ thuật cắt tỉa. Ví dụ, ta có thể dùng chuỗi nhãn tốt nhất (1-best) thay vì mọi chuỗi nhãn có thể để xấp xỉ Phương trình (4.74), tức là $\Pr(y_1 \cdots y_n|\mathbf{x}_1 \cdots \mathbf{x}_m) = \max \Pr(l_1 \cdots l_m|\mathbf{H})$. Điều này dẫn tới một phương pháp giải mã hiệu quả, gọi là **giải mã Viterbi** (*Viterbi decoding*), đã được dùng rộng rãi trong nhận dạng tiếng nói và dịch máy [Lopez, 2008]. Để biết thêm chi tiết về giải mã trong sinh chuỗi, mời bạn đọc xem Chương 5.

4.5.3 Gán nhãn chuỗi với LSTM và mô hình đồ thị

Gán nhãn chuỗi (*sequence labeling*) là một cách tiếp cận đơn giản về mặt khái niệm để gán nhãn cho dữ liệu dạng chuỗi. Trong NLP, nó đã được dùng rộng rãi trong nhiều lĩnh vực con, như tách từ, gán nhãn từ loại và phân đoạn cụm (*chunking*). Việc học trong các mô hình này gồm việc dự đoán một nhãn từ một tập nhãn định sẵn V_y tại mỗi vị trí của chuỗi. Lý tưởng nhất, ta muốn dự đoán chuỗi nhãn tối ưu khi biết toàn bộ đầu vào, được phát biểu là:

$$\hat{y}_1 \cdots \hat{y}_m = \underset{y_1 \cdots y_m}{\arg\max} \Pr(y_1 \cdots y_m|\mathbf{x}_1 \cdots \mathbf{x}_m), \tag{4.76}$$

trong đó $\mathbf{x}_1 \cdots \mathbf{x}_m$ là một chuỗi đầu vào (chẳng hạn một chuỗi vectơ từ), và $y_1 \cdots y_m$ là một chuỗi nhãn, trong đó mỗi nhãn y_i ứng với một phần tử đầu vào $\mathbf{x}_i$.

Chú giải của người dịch: Ví dụ gán nhãn từ loại

Với câu "Tôi đọc sách", đầu vào là 3 vectơ từ và đầu ra mong muốn là chuỗi nhãn "đại từ – động từ – danh từ": mỗi từ đúng một nhãn, nên chuỗi nhãn dài bằng chuỗi đầu vào (khác với nhận dạng tiếng nói ở tiểu mục trước, nơi $n < m$).

Như ta đã thấy trong chương này, Phương trình (4.76) rất khớp với cách phát biểu bài toán mô hình hóa chuỗi. Bước đầu tiên, ta dùng một bộ mã hóa để ánh xạ $\mathbf{x}_1 \cdots \mathbf{x}_m$ thành một chuỗi biểu diễn theo ngữ cảnh như sau:

$$\mathbf{h}_1 \cdots \mathbf{h}_m = \text{Encode}(\mathbf{x}_1 \cdots \mathbf{x}_m). \tag{4.77}$$

Ta định nghĩa Encode$(\cdot)$ là một mô hình LSTM hai chiều vì nó có cơ chế nhớ để mô hình hóa các phụ thuộc tầm xa ở cả ngữ cảnh bên trái lẫn bên phải. Do đó, kiến trúc của bộ mã hóa giống như kiến trúc dùng ở tiểu mục trước.

$\mathbf{h}_1 \cdots \mathbf{h}_m$ sau đó có thể làm đầu vào cho một hệ thống gán nhãn chuỗi tiêu chuẩn (xem Hình 4.12). Bài toán gán nhãn chuỗi đã được bàn ở Chương 1, và nhiều mô hình có thể được áp dụng để giải nó. Đơn giản nhất là một bộ phân loại (như mô hình entropy cực đại hay các mô hình dựa trên SVM) dự đoán một phân phối nhãn cho mỗi $\mathbf{h}_i$. Hạn chế của các mô hình này là các dự đoán được đưa ra độc lập với nhau. Một cách tiếp cận mạnh hơn là dùng **mô hình đồ thị** (*graphical models*) để tính đến sự phụ

thuộc giữa các nhãn được dự đoán. Ví dụ, **mô hình Markov ẩn** (*hidden Markov models*) (**HMM**) mô tả cách một chuỗi quan sát (tức là $\mathbf{x}_1 \cdots \mathbf{x}_m$) được sinh ra khi cho trước một chuỗi biến (tức là $y_1 \cdots y_m$). Ý tưởng then chốt là viết lại $\Pr(y_1 \cdots y_m | \mathbf{x}_1 \cdots \mathbf{x}_m)$ bằng cách đưa ra các giả định độc lập để phân tích phân phối thành nhân tử, và xấp xỉ $\Pr(y_1 \cdots y_m | \mathbf{x}_1 \cdots \mathbf{x}_m)$ bằng một tích của các nhân tử đơn giản. Tuy nhiên, các mô hình này đòi hỏi hàm mật độ xác suất của các biến liên tục (ví dụ $\Pr(\mathbf{x}_i | y_i)$), vốn khó ước lượng. Điều này làm việc dùng HMM trong các mô hình nơ-ron khác biệt đáng kể so với trong các mô hình truyền thống, nơi mọi trạng thái và quan sát đều là biến rời rạc[13].

Chú giải của người dịch: Ý tưởng của HMM qua một ví dụ

Phương trình (4.78) dùng *định lý Bayes*: $\Pr(A|B) = \Pr(A)\Pr(B|A)/\Pr(B)$, cho phép "lật" chiều của xác suất có điều kiện ($\Pr(A|B)$ là xác suất xảy ra A khi đã biết B xảy ra). HMM thêm hai giả định đơn giản hóa: (1) nhãn hiện tại chỉ phụ thuộc vào nhãn ngay trước nó, $\Pr(y_i|y_{i-1})$ — ví dụ sau "động từ" thì khả năng gặp "danh từ" khá cao; (2) mỗi từ chỉ phụ thuộc vào nhãn của chính nó, $\Pr(\mathbf{x}_i|y_i)$ — ví dụ nhãn "động từ" hay "phát ra" từ "đọc". Nhờ đó, xác suất cả chuỗi trở thành tích của nhiều thừa số nhỏ, dễ ước lượng.

HMM và các mô hình hậu duệ của nó có thể được xem là các trường hợp của mô hình sinh. Một lớp mô hình khác thường được dùng cho gán nhãn chuỗi là mô hình phân biệt. Một mô hình như vậy là **trường ngẫu nhiên có điều kiện** (*conditional random fields*) (**CRF**) [Lafferty và cộng sự, 2001]. Đặc trưng của mô hình CRF là khả năng mô hình hóa trực tiếp xác suất có điều kiện của một chuỗi nhãn khi biết chuỗi đầu vào, điều này giúp dễ dàng tích hợp các đặc trưng tùy ý. Lấy ví dụ **CRF chuỗi tuyến tính** (*linear-chain CRF*) [Sutton và McCallum, 2012]. Nó định nghĩa $\Pr(y_1 \cdots y_m | \mathbf{h}_1 \cdots \mathbf{h}_m)$ dưới dạng sau

$$\begin{aligned}\Pr(y_1 \cdots y_m | \mathbf{h}_1 \cdots \mathbf{h}_m) &= \frac{\Pr(y_1 \cdots y_m, \mathbf{h}_1 \cdots \mathbf{h}_m)}{\Pr(\mathbf{h}_1 \cdots \mathbf{h}_m)} \\ &= \frac{\exp(\text{Score}(y_1 \cdots y_m, \mathbf{h}_1 \cdots \mathbf{h}_m))}{Z(\mathbf{h}_1 \cdots \mathbf{h}_m)},\end{aligned} \tag{4.80}$$

trong đó $Z(\mathbf{h}_1 \cdots \mathbf{h}_m)$ là một hệ số chuẩn hóa, có dạng

$$Z(\mathbf{h}_1 \cdots \mathbf{h}_m) = \sum_{y'_1 \cdots y'_m} \exp(\text{Score}(y'_1 \cdots y'_m, \mathbf{h}_1 \cdots \mathbf{h}_m)). \tag{4.81}$$

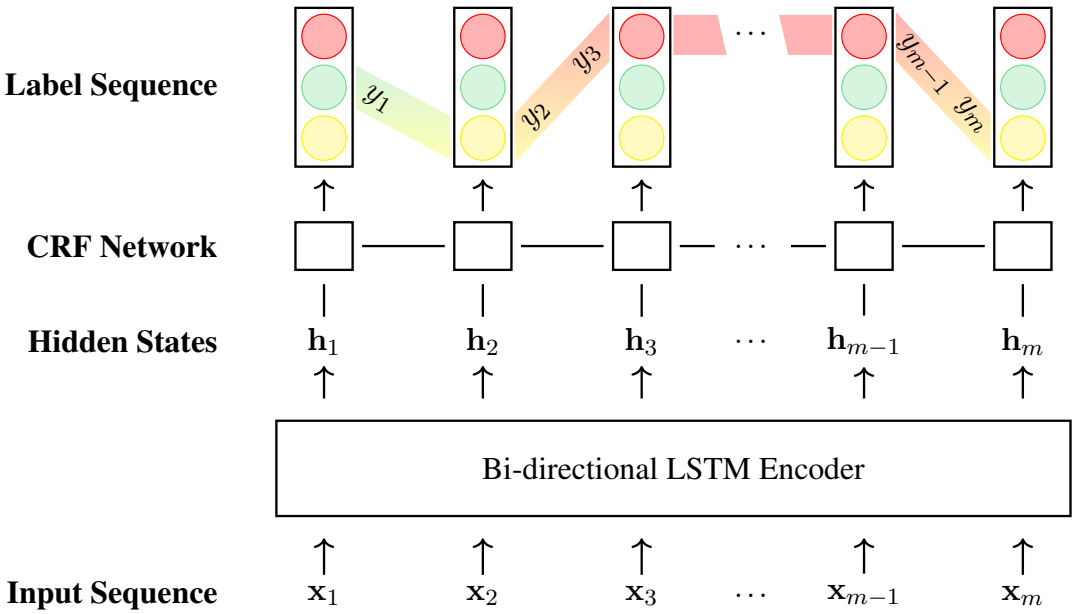
[13] Trong HMM, một chuỗi biến có thể được xem như một đường đi chuyển qua một số trạng thái, với giá trị được chọn từ một tập định sẵn. Trong mỗi lần chuyển từ trạng thái này sang trạng thái khác, ta quan sát được một thứ gì đó (gọi là một quan sát). Bằng cách đưa ra một số giả định, ta có thể xấp xỉ $\Pr(y_1 \cdots y_m | \mathbf{x}_1 \cdots \mathbf{x}_m)$ theo cách sau

$$\begin{aligned}\Pr(y_1 \cdots y_m | \mathbf{x}_1 \cdots \mathbf{x}_m) &= \frac{\Pr(y_1 \cdots y_m) \cdot \Pr(\mathbf{x}_1 \cdots \mathbf{x}_m | y_1 \cdots y_m)}{\Pr(\mathbf{x}_1 \cdots \mathbf{x}_m)} \\ &\approx \frac{\prod_{i=1}^m \Pr(y_i|y_{i-1}) \cdot \prod_{i=1}^m \Pr(\mathbf{x}_i|y_i)}{\Pr(\mathbf{x}_1 \cdots \mathbf{x}_m)} \\ &= \frac{\prod_{i=1}^m \Pr(y_i|y_{i-1}) \Pr(\mathbf{x}_i|y_i)}{\Pr(\mathbf{x}_1 \cdots \mathbf{x}_m)},\end{aligned} \tag{4.78}$$

trong đó $\Pr(y_i|y_{i-1})$ là **xác suất chuyển** (*transition probability*) từ y_{i-1} sang y_i (khi $i = 1$, ta định nghĩa $\Pr(y_i|y_{i-1}) = \Pr(y_1|y_0) = \Pr(y_1)$), và $\Pr(\mathbf{x}_i|y_i)$ là **xác suất phát xạ** (*emission probability*) của việc quan sát được $\mathbf{x}_i$ khi biết y_i. Vì mẫu số $\Pr(\mathbf{x}_1 \cdots \mathbf{x}_m)$ là hằng số đối với các $y_1 \cdots y_m$ khác nhau, nó có thể được bỏ đi trong phép argmax của Phương trình (4.76), như sau:

$$\hat{y}_1 \cdots \hat{y}_m = \arg\max_{y_1 \cdots y_m} \prod_{i=1}^m \Pr(y_i|y_{i-1}) \Pr(\mathbf{x}_i|y_i). \tag{4.79}$$

Để ước lượng $\Pr(\mathbf{x}_i|y_i)$, một giải pháp khả dĩ là lấy $\Pr(\mathbf{x}_i|y_i) = \frac{\Pr(y_i|\mathbf{x}_i)\Pr(\mathbf{x}_i)}{\Pr(y_i)}$, và dùng một mạng nơ-ron để tính $\Pr(y_i|\mathbf{x}_i)$.



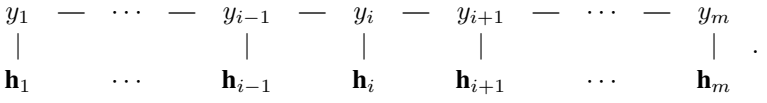
Hình 4.12: Kiến trúc BiLSTM + mô hình đồ thị cho gán nhãn chuỗi. Bộ mã hóa là một mô hình LSTM hai chiều tiêu chuẩn (*Bi-directional LSTM Encoder*). Cho một chuỗi vectơ đặc trưng đầu vào (*Input Sequence*, tức là $\mathbf{x}_1 \cdots \mathbf{x}_m$), nó tạo ra một chuỗi vectơ đặc trưng mới để ánh xạ đầu vào thành các biểu diễn theo ngữ cảnh (*Hidden States*, tức là $\mathbf{h}_1 \cdots \mathbf{h}_m$). Một mạng CRF (*CRF Network*) được đặt trên $\mathbf{h}_1 \cdots \mathbf{h}_m$ để dự đoán một phân phối trên các chuỗi nhãn (*Label Sequence*). Chuỗi nhãn tối ưu là chuỗi có xác suất lớn nhất.

Score$(\cdot)$ là một điểm số dùng để đánh trọng số cho cặp chuỗi $(y_1 \cdots y_m, \mathbf{h}_1 \cdots \mathbf{h}_m)$. Nó được tính bằng cách lấy tổng giá trị của một tập hàm đặc trưng (*feature functions*) $\{f_1(\cdot), \ldots, f_J(\cdot)\}$

$$\text{Score}(y_1 \cdots y_m, \mathbf{h}_1 \cdots \mathbf{h}_m) = \sum_{i=1}^{m} \sum_{j=1}^{J} f_j(y_i, y_{i-1}, \mathbf{h}_i). \tag{4.82}$$

Vòng lặp ngoài của tổng ứng với việc duyệt qua từng vị trí i. Với mỗi i, mỗi hàm $f_j(\cdot)$ nhận nhãn hiện tại y_i, nhãn trước đó y_{i-1} và vectơ đầu vào hiện tại $\mathbf{h}_i$, rồi trả về giá trị của một đặc trưng.

Mô hình này được gọi là *chuỗi tuyến tính* vì nó giả định tính chất Markov bậc nhất: mỗi nhãn y_i chỉ phụ thuộc vào các nhãn láng giềng trực tiếp y_{i-1} và y_{i+1}, với điều kiện là chuỗi quan sát $\mathbf{H}$[14], như sau:



Trong CRF, ta giả định rằng các biến trong đồ thị chỉ phụ thuộc vào các biến láng giềng của chúng. Do đó, $f_j(y_i, y_{i-1}, \mathbf{h}_i)$ có thể được định nghĩa tùy theo cách y_i được kết nối. Nhìn chung có hai loại đặc trưng.

[14]CRF nhìn chung có thể được xếp vào loại **mô hình đồ thị vô hướng** (*undirected graphical models*). Chúng định nghĩa một đồ thị trên một tập biến quan sát được và một tập biến không quan sát được. Các biến này được nối với nhau theo một cách nào đó để tạo thành một đồ thị.

- **Đặc trưng kiểu chuyển** (*Transition-like Features*). Các đặc trưng này mô hình hóa mối liên hệ giữa các nhãn liên tiếp (y_{i-1}, y_i), cho bởi:

$$f_1(y_i, y_{i-1}, \mathbf{h}_i) = u(y_{i-1}, y_i), \tag{4.83}$$

  trong đó $u(y_{i-1}, y_i)$ là một phần tử của ma trận trọng số $\mathbf{u}$, được đánh chỉ số bởi (y_{i-1}, y_i).

- **Đặc trưng kiểu phát xạ** (*Emission-like Features*). Loại đặc trưng thứ hai mô hình hóa mối liên hệ giữa một nhãn y_i và đầu vào tương ứng $\mathbf{x}_i$, cho bởi:

$$f_2(y_i, y_{i-1}, \mathbf{h}_i) = g_i(y_i), \tag{4.84}$$

  trong đó $g_i(y_i)$ là phần tử thứ y_i của một vectơ $\mathbf{g}_i \in \mathbb{R}^{|V_y|}$. Vectơ $\mathbf{g}_i$ biểu diễn các trọng số gắn $\mathbf{h}_i$ với từng nhãn, dưới dạng

$$\mathbf{g}_i = \mathbf{h}_i \cdot \mathbf{v}, \tag{4.85}$$

  trong đó $\mathbf{v} \in \mathbb{R}^{d_h \times |V_y|}$ là một ma trận trọng số.

Để đơn giản hóa ký hiệu, ta dùng y_i (hoặc y_{i-1}) để chỉ biểu diễn one-hot của một nhãn[15]. Khi đó, thay các hàm đặc trưng ở trên vào Phương trình (4.82), hàm tính điểm có thể được viết dưới dạng

$$\begin{aligned}
\text{Score}(y_1 \cdots y_m, \mathbf{h}_1 \cdots \mathbf{h}_m) &= \sum_{i=1}^{m} u(y_{i-1}, y_i) + g_i(y_i) \\
&= \sum_{i=1}^{m} y_{i-1} \cdot \mathbf{u} \cdot y_i^\top + \mathbf{h}_i \cdot \mathbf{v} \cdot y_i^\top \\
&= \sum_{i=1}^{m} (y_{i-1} \cdot \mathbf{u} + \mathbf{h}_i \cdot \mathbf{v}) \cdot y_i^\top .
\end{aligned} \tag{4.86}$$

Chú giải của người dịch: Vectơ one-hot "chọn" phần tử

Giả sử có 3 nhãn {danh từ, động từ, tính từ}. Vectơ one-hot của "động từ" là $(0, 1, 0)$: chỉ có một số 1 tại vị trí của nhãn đó. Nhân một vectơ one-hot với ma trận có tác dụng "rút" ra đúng một hàng/cột: nếu y_{i-1} là one-hot của nhãn a và y_i là one-hot của nhãn b thì $y_{i-1} \cdot \mathbf{u} \cdot y_i^\top$ đúng bằng phần tử $u(a, b)$ ở hàng a, cột b của $\mathbf{u}$. Tương tự, $\mathbf{g}_i \cdot y_i^\top$ chính là $g_i(y_i)$. Vì vậy dòng thứ hai của Phương trình (4.86) chỉ là cách viết lại dòng thứ nhất bằng phép nhân ma trận — loại phép toán mà mạng nơ-ron tính rất nhanh. Ma trận $\mathbf{u}$ có thể hiểu là "bảng điểm chuyển": $u(a, b)$ lớn nghĩa là nhãn b hay đứng sau nhãn a.

Vế phải của phương trình chỉ gồm các phép toán đại số đơn giản trên vectơ và ma trận, cho phép xem mô hình này như một mạng nơ-ron tiêu chuẩn. Nhờ vậy, việc cài đặt hệ thống gán nhãn chuỗi bằng các bộ công cụ mạng nơ-ron khác nhau trở nên thuận tiện. Ta chỉ cần xếp một mạng CRF lên trên một mạng mã hóa và học toàn bộ mạng như thường lệ. Ví dụ, có thể huấn luyện hệ thống này bằng phương pháp hợp lý cực đại, và tối ưu hàm mất mát bằng hạ gradient. Lưu ý rằng, giống như các mô hình dạng chuỗi hay dạng lưới khác, mạng CRF có thể chạy hiệu quả vì có các thuật toán quy hoạch động, gọi là các phương pháp tiến–lùi (*forward-backward*), để tính cả $\text{Score}(y_1 \cdots y_m, \mathbf{h}_1 \cdots \mathbf{h}_m)$ lẫn $Z(\mathbf{h}_1 \cdots \mathbf{h}_m)$. Bạn đọc quan tâm có thể tham khảo các bài báo liên quan để xem thảo luận chi tiết hơn [Lafferty và cộng sự, 2001; Sutton và McCallum, 2012].

Một ưu điểm chính của việc tích hợp các biểu diễn phân tán vào gán nhãn chuỗi là nó loại bỏ nhu cầu thiết kế đặc trưng thủ công. Thay vào đó, mô hình tự học từ đầu vào thô những đặc trưng hiệu quả

[15] Trong trường hợp này, $y_i \in \mathbb{R}^{|V_y|}$, dù ban đầu nó được dùng như một số vô hướng.

nhất cho việc tối ưu mục tiêu gán nhãn chuỗi. Kiến trúc như vậy đã được dùng làm mô hình xương sống cho một số hệ thống tiên tiến nhất [Huang và cộng sự, 2015; Lample và cộng sự, 2016; Ma và Hovy, 2016; Li và cộng sự, 2020c].

4.5.4 Mô hình lai cho mô hình hóa ngôn ngữ

Như đã nói, nhiều bài toán mô hình hóa chuỗi có thể được giải quyết bằng mô hình dựa trên RNN hoặc dựa trên CNN. Mỗi kiến trúc có đặc điểm riêng: RNN ban đầu được thiết kế để xử lý dữ liệu thời gian có độ dài thay đổi, trong khi CNN hiệu quả hơn trong việc nắm bắt thông tin cục bộ trong các vùng giới hạn của đầu vào. Ở đây, ta xét một cách tiếp cận lai cho mô hình hóa ngôn ngữ để tận dụng lợi thế của cả hai.

Nhắc lại từ Mục 4.2.1 rằng một mô hình ngôn ngữ nơ-ron được huấn luyện để dự đoán một phân phối xác suất trên bộ từ vựng khi biết một biểu diễn nào đó của các từ đứng trước. Gọi $w_1 \cdots w_m$ là một chuỗi từ mà ta muốn tính xác suất. Trước hết, ta biểu diễn mỗi từ w_i bằng một vectơ từ $\mathbf{x}_i$. Sau đó, một mô hình RNN xử lý lần lượt từng vectơ từ và xuất ra xác suất:

$$\begin{aligned}\Pr(w_{i+1}|w_1 \cdots w_i) &= \Pr(w_{i+1}|\mathbf{x}_1 \cdots \mathbf{x}_i) \\ &= \Pr(w_{i+1}|\mathbf{h}_i),\end{aligned} \tag{4.87}$$

trong đó $\mathbf{h}_i$ là trạng thái của đơn vị hồi quy tại bước i.

Quá trình chuyển các từ từ dạng ký hiệu sang biểu diễn liên tục đóng vai trò quan trọng trong mô hình này. Dù người làm thực tế thường lấy $\mathbf{x}_i$ từ một bảng nhúng từ, cách làm này coi mỗi từ là một khối nguyên vẹn và bỏ qua cấu trúc bên trong của nó. Kết quả là có thể khó học được các vectơ phân biệt cho các từ hiếm trong những ngôn ngữ có bộ từ vựng lớn [Bojanowski và cộng sự, 2017].

Ở đây ta xét một cách khác để biểu diễn từ. Ý tưởng rất đơn giản: dùng thêm một mạng nơ-ron để nhúng từ [Ling và cộng sự, 2015; Kim và cộng sự, 2016]. Giả sử mỗi từ w_i có thể được biểu diễn thành một chuỗi ký tự. Ta biểu diễn các ký tự này thành các vectơ số thực $\mathbf{z}_{i,1} \cdots \mathbf{z}_{i,\text{len}_i}$ thông qua một bảng nhúng ký tự. Theo công trình của Kim và cộng sự [2016], ta có thể dùng một CNN để biểu diễn $\mathbf{z}_{i,1} \cdots \mathbf{z}_{i,\text{len}_i}$ thành một vectơ từ dưới dạng sau

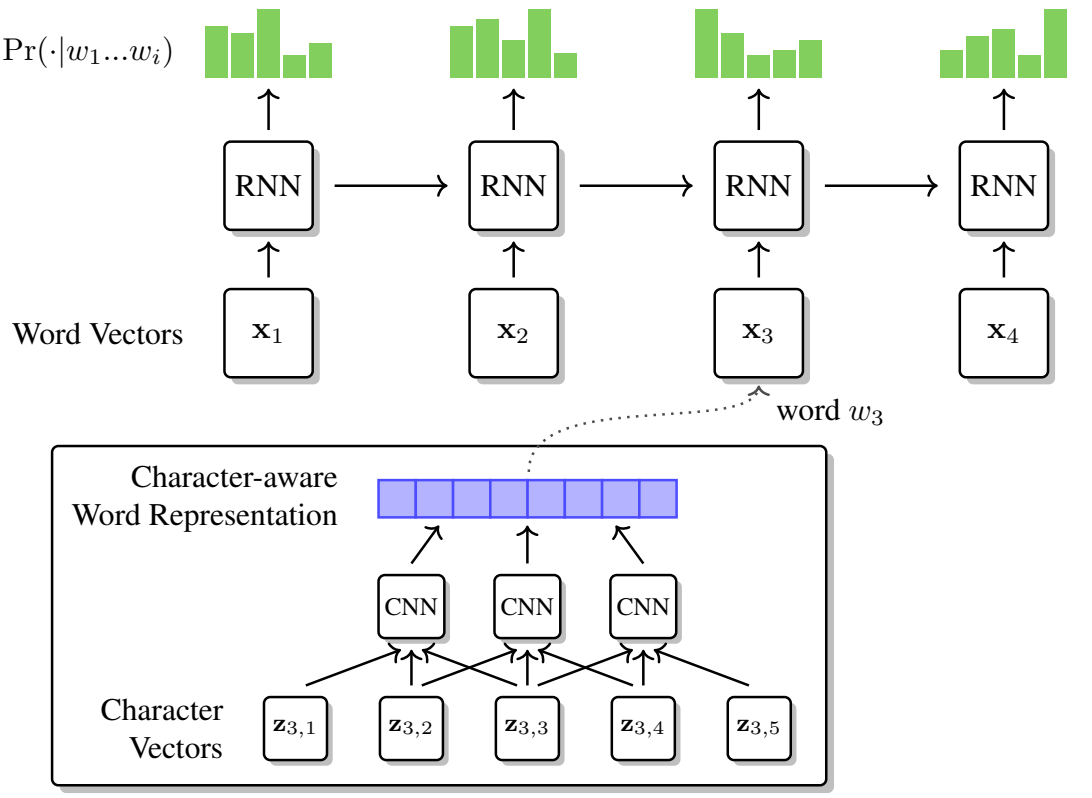
$$\begin{aligned}\mathbf{x}_i &= \text{CNN}(\mathbf{z}_{i,1} \cdots \mathbf{z}_{i,\text{len}_i}, \mathbf{W}) \\ &= \text{Pooling}(\text{TanH}(\text{Conv}(\mathbf{z}_{i,1} \cdots \mathbf{z}_{i,\text{len}_i}, \mathbf{W}))),\end{aligned} \tag{4.88}$$

trong đó $\text{Conv}(\cdot, \mathbf{W})$ là một tầng tích chập với tham số $\mathbf{W}$, $\text{TanH}(\cdot)$ là hàm tang hyperbol, và $\text{Pooling}(\cdot)$ là một tầng gộp.

Hình 4.13 trình bày kiến trúc của mô hình. Mô hình này có cấu trúc phân cấp, trong đó các ký tự tạo thành từ, và các từ tạo thành câu hay cụm từ. Xét từ góc độ thực tiễn, trong nhiều bài toán NLP, việc xét đến bản chất phân cấp của ngôn ngữ là khá tự nhiên. Chúng ta sẽ thấy một vài ví dụ về việc tận dụng mối quan hệ giữa các cấp độ biểu diễn ngôn ngữ khác nhau ở các chương sau.

Chú giải của người dịch: Lợi ích của biểu diễn mức ký tự

Giả sử mô hình đã gặp "teach" (dạy) và "teacher" (giáo viên) nhiều lần nhưng chỉ gặp "teachers" một lần. Với bảng nhúng từ thông thường, "teachers" là một mục riêng, học được rất ít. Với CNN trên ký tự, các bộ lọc nhận ra những cụm ký tự quen thuộc như "teach", "-er", "-s", nên vectơ của "teachers" tự nhiên gần với "teacher". Tiếng Việt cũng có hiện tượng tương tự ở mức âm tiết: "giáo viên", "giáo sư", "giáo trình" chia sẻ âm tiết "giáo".



Hình 4.13: Một mô hình ngôn ngữ với biểu diễn từ có xét đến ký tự (*Character-aware Word Representation*) [Kim và cộng sự, 2016]. Là một mô hình ngôn ngữ, mục tiêu của mô hình này là tính xác suất $\Pr(w_{i+1}|w_1 \cdots w_i)$ với mỗi i. Ta biểu diễn mỗi từ w_i bằng một vectơ số thực $\mathbf{x}_i$ (*Word Vectors*). Vectơ này là đầu ra của một CNN nhận vào chuỗi ký tự (*Character Vectors*) tương ứng với từ đó. Sau đó, chuỗi vectơ từ $\mathbf{x}_1 \cdots \mathbf{x}_m$ được dùng làm đầu vào cho một mô hình RNN + Softmax. Tại mỗi vị trí i, mô hình xuất ra một phân phối trên các từ, trong đó phần tử ứng với w_{i+1} cho ta $\Pr(w_{i+1}|w_1 \cdots w_i)$. Cấu trúc phân cấp này tạo ra một cách tiếp cận đa tỉ lệ cho mô hình hóa ngôn ngữ: một câu được mô hình hóa bằng cách xét các từ, và một từ được mô hình hóa bằng cách xét các ký tự.

4.6 Tóm tắt

Chương này đã giới thiệu các cách tiếp cận mạng nơ-ron hồi quy và mạng nơ-ron tích chập để mô hình hóa chuỗi từ. Một mặt, mạng nơ-ron hồi quy được thiết kế để xử lý dữ liệu dạng chuỗi và có khả năng ứng dụng rộng rãi trong NLP. Để tăng sức mạnh mô hình hóa của các mô hình này, người ta thường dùng các cơ chế nhớ. Đặc biệt, chúng ta đã giới thiệu LSTM và GRU, hai mô hình phổ biến cho các chuỗi dài. Mặt khác, dù mạng nơ-ron tích chập thường được dùng để xử lý dữ liệu hình ảnh, chúng có thể được áp dụng trực tiếp cho mô hình hóa chuỗi. Ta đã thấy rằng tất cả các mô hình này đều có thể dùng trong nhiều bài toán xử lý ngôn ngữ và tiếng nói, bao gồm phân loại văn bản, nhận dạng tiếng nói, gán nhãn chuỗi và mô hình hóa ngôn ngữ.

Nguồn gốc của việc mô hình hóa chuỗi các đơn vị ngôn ngữ có thể truy về những công trình ban đầu trong nhiều lĩnh vực khác nhau. Ví dụ, quá trình sinh một chuỗi từ có thể được mô tả như một xích Markov (*Markov chain*), trong đó việc dự đoán một từ chỉ phụ thuộc vào một số lượng hữu hạn các từ đứng trước [Markov, 1913]. Ý tưởng này thúc đẩy các phương pháp n-gram cho mô hình hóa chuỗi [Shannon, 1948a], cũng như các mô hình Markov ẩn, vốn xuất hiện sau đó và trở nên phổ biến trong việc mô hình hóa chuỗi các cặp biến quan sát được và không quan sát được [Baum và Petrie, 1966; Baum và cộng sự, 1970]. Các mô hình này và các biến thể của chúng đã đặt nền móng cho nhiều hệ thống NLP thành công trong những thập kỷ qua [Manning và Schütze, 1999; Jurafsky và Martin, 2008].

Ý tưởng dùng mạng nơ-ron trong mô hình hóa chuỗi cũng đã được nghiên cứu từ khá lâu. Một ví dụ minh họa cách mạng nơ-ron được phát triển và áp dụng cho mô hình hóa chuỗi là nhận dạng tiếng nói [Lippmann, 1989]. Hầu hết các nghiên cứu thời kỳ đầu của lĩnh vực này hoặc cố gắng kết hợp mạng nơ-ron với các mô hình sẵn có [Bourlard và Wellekens, 1990; Bourlard và Morgan, 1993; Trentin và Gori, 2001], hoặc giải quyết các bài toán con của nhận dạng tiếng nói [Tank và Hopfield, 1987; Waibel và cộng sự, 1989; Lang và cộng sự, 1990; Bengio, 1991]. Tuy nhiên, việc mở rộng quy mô mạng nơ-ron rất khó khăn vì huấn luyện các mạng nơ-ron sâu đòi hỏi nhiều tài nguyên tính toán và dữ liệu. Lĩnh vực này từ lâu đã bị chi phối bởi các cách tiếp cận dựa trên mô hình Markov ẩn và mô hình hỗn hợp Gauss (*Gaussian mixture models*) (GMM), với một quy trình gồm nhiều mô-đun cần tinh chỉnh cẩn thận. Mặt khác, dù các cách tiếp cận hoàn toàn bằng mạng nơ-ron chưa phải là tiên tiến nhất vào thời điểm đó, các nhà nghiên cứu đã nhận thấy tiềm năng của chúng trong việc học biểu diễn của đầu vào âm thanh và giải phóng chúng khỏi các đặc trưng thiết kế thủ công [LeCun và Bengio, 1995].

Một sự chuyển dịch mạnh mẽ từ các cách tiếp cận dạng quy trình nhiều khâu truyền thống sang các cách tiếp cận đầu-cuối (*end-to-end*) đã đến cùng với sự hồi sinh của mạng nơ-ron trong những năm 2000 [Hannun và cộng sự, 2014; Graves và cộng sự, 2013b]. Sự chuyển dịch này có ảnh hưởng lớn đến mức nó đưa nhiều lĩnh vực lại gần nhau hơn bao giờ hết. Chẳng hạn, trong cả thị giác máy tính lẫn xử lý tiếng nói, thập kỷ vừa qua đã chứng kiến những bước tiến vượt bậc về hiệu năng nhờ các mạng nơ-ron rất sâu và học đầu-cuối [Hinton và cộng sự, 2006; Graves và cộng sự, 2013b; He và cộng sự, 2016a; Krizhevsky và cộng sự, 2017]. Trong NLP, sự chuyển đổi mô thức bắt đầu với các công trình về nhúng từ [Mikolov và cộng sự, 2013a; Pennington và cộng sự, 2014], và tiếp diễn khi các mô hình biểu diễn chuỗi mạnh hơn được phát triển [Vaswani và cộng sự, 2017]. Những cách tiếp cận đơn giản cho mô hình hóa chuỗi, dù không được bàn sâu trong chương này, là các mô hình kết hợp (*compositional models*) [Janssen, 2012]. Ví dụ, ta có thể dùng mô hình túi từ (*bag-of-words*) để cộng hoặc lấy trung bình các vectơ từ của một chuỗi. Dù có kiến trúc đơn giản, các cách tiếp cận này đạt kết quả khả quan trong nhiều bài toán, cung cấp các mô hình cơ sở mạnh cho các nghiên cứu tiếp theo về những mô hình tiên tiến hơn [Conneau và cộng sự, 2018]. Bước tiếp theo, việc áp dụng các mô hình nơ-ron hồi quy và tích chập cho mô hình hóa chuỗi là khá tự nhiên. Điều này không có gì ngạc nhiên vì các mô hình này

đã được nghiên cứu khá kỹ trong các lĩnh vực khác [Lipton và cộng sự, 2015; Li và cộng sự, 2021d; Khan và cộng sự, 2020]. Đặc biệt, mô hình LSTM rất phù hợp để xử lý các chuỗi dài và do đó được các nhà nghiên cứu NLP rất quan tâm [Sundermeyer và cộng sự, 2012; Huang và cộng sự, 2015; Wu và cộng sự, 2016]. Tuy nhiên, nghiên cứu trong lĩnh vực này vẫn đang tiếp diễn. Mô hình hóa chuỗi là một trong những lĩnh vực nghiên cứu sôi động nhất và chưa thấy điểm dừng. Có nhiều mô hình dựa trên các kiến trúc mới và cho hiệu năng mạnh hơn trong nhiều bài toán. Thảo luận thêm về một số mô hình này có thể xem ở Chương 6, 7 và 8.

Lưu ý rằng thuật ngữ *mô hình hóa chuỗi* hiện được dùng theo nhiều cách khác nhau, chỉ các bài toán khác nhau. Trong nhiều trường hợp, người ta hay dùng các thuật ngữ *mã hóa* và *bộ mã hóa* hơn để nhấn mạnh quá trình ánh xạ một chuỗi ký hiệu thành một biểu diễn liên tục. Như đã bàn ở các mục trước, một lợi ích của việc xem mã hóa như một bài toán riêng là ta có thể học một mô hình biểu diễn tổng quát, không phụ thuộc vào nơi ta áp dụng nó. Điều này mở ra cánh cửa cho rất nhiều bộ mã hóa được tiền huấn luyện để học cách biểu diễn nhiều loại dữ liệu khác nhau, như văn bản [Peters và cộng sự, 2018; Devlin và cộng sự, 2019], tiếng nói [Oord và cộng sự, 2018; Hsu và cộng sự, 2021; Chen và cộng sự, 2022], hình ảnh [Chen và He, 2021; Bao và cộng sự, 2021; He và cộng sự, 2022], và tổ hợp của các phương thức này [Chuang và cộng sự, 2020; Li và cộng sự, 2021c; Kim và cộng sự, 2021]. Một khái niệm liên quan chặt chẽ với mã hóa văn bản là nhúng văn bản (*text embedding*) hay nhúng câu (*sentence embedding*) [Conneau và cộng sự, 2017a; Cer và cộng sự, 2018]. Có thể xem chúng nhìn chung là tương đương. Nói chung, một mô hình nhúng trong NLP là một quá trình biến đổi văn bản đầu vào thành một vectơ duy nhất có số chiều thấp, thay vì tạo ra các chuỗi vectơ theo ngữ cảnh [Kiros và cộng sự, 2015; Hill và cộng sự, 2016].

Trong nhiều bài toán NLP, các hệ thống không nhất thiết có đầu vào và/hoặc đầu ra dạng chuỗi. Ví dụ, trong phân loại văn bản, một hệ thống có thể nhận đầu vào có cấu trúc cây và sinh ra một nhãn [Tai và cộng sự, 2015; Yang và cộng sự, 2016]. Trong trường hợp này, ta cần một cơ chế nào đó để mã hóa các cấu trúc phân cấp. Một cách tiếp cận khác là chuyển cây thành chuỗi (hay *cây tuyến tính hóa*) để có thể dùng trực tiếp các mô hình chuỗi xử lý dữ liệu không có dạng chuỗi [Vinyals và cộng sự, 2015]. Đây là một ý tưởng hấp dẫn vì nó mở ra khả năng phát triển một bộ mã hóa áp dụng được cho mọi loại dữ liệu, nếu đầu vào của bộ mã hóa có thể được tuyến tính hóa theo cách nào đó. Ví dụ, bằng cách biểu diễn một hình ảnh thành một chuỗi các mảnh ảnh, các mô hình chuỗi có thể được áp dụng trực tiếp cho phân loại ảnh, đạt kết quả tiên tiến nhất trên một số bài toán [Chen và cộng sự, 2020a; Dosovitskiy và cộng sự, 2021].

Chương 5

Các mô hình chuỗi-sang-chuỗi

> 天下万物之理，无独必有对。
>
> *Theo lẽ của Trời Đất và muôn vật, không có gì tồn tại đơn độc mà ắt phải có cái đối với nó.*
>
> –《近思录》 *Cận tư lục* (Suy ngẫm về những điều gần gũi), Chu Hy (1130–1200) và Lã Tổ Khiêm (1137–1181), bản dịch tiếng Anh của Chang [1967]

Trong thế giới ngôn ngữ, mọi thứ thường đi theo cặp. Có câu hỏi thì có câu trả lời; có một văn bản tiếng Trung thì có bản dịch tiếng Anh; có một câu thì có cây phân tích cú pháp của nó. Nhiều hệ thống NLP được thiết kế để mô hình hóa sự tương ứng giữa những cặp như vậy, tức là một phần tử được lấy làm đầu vào và phần tử kia làm đầu ra. Các bài toán này có thể được biểu diễn dưới một dạng mà ta đã gặp nhiều lần như sau:

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x}). \tag{5.1}$$

Chương này tập trung vào một họ bài toán đặc biệt, trong đó cả $\mathbf{x}$ và $\mathbf{y}$ đều là chuỗi từ, gọi là các bài toán **chuỗi-sang-chuỗi** (*sequence-to-sequence*) (hay *seq2seq*). Khác với các bài toán phân loại, nơi đầu ra $\hat{\mathbf{y}}$ được chọn từ một tập cố định các lớp, bài toán chuỗi-sang-chuỗi đòi hỏi mô hình sinh ra đầu ra từ một không gian đầu ra lớn theo cấp số mũ. Việc tìm $\hat{\mathbf{y}}$ trong trường hợp này phức tạp hơn đáng kể so với phân loại, vì nó cần những mô hình mạnh hơn để biểu diễn $\Pr(\mathbf{y}|\mathbf{x})$ và những thuật toán tìm kiếm hiệu quả hơn để giải Phương trình (5.1).

Chương này trước hết giới thiệu kiến trúc bộ mã hóa (*encoder*)–bộ giải mã (*decoder*) nổi tiếng cho việc mô hình hóa chuỗi-sang-chuỗi, rồi trình bày cơ chế chú ý (*attention mechanism*) như một mở rộng quan trọng của kiến trúc này. Các mô hình này đặt nền móng cho nhiều cách tiếp cận tiên tiến nhất được bàn đến trong các chương sau. Ngoài ra, chương này sẽ thảo luận bài toán tìm kiếm (giải mã (*decoding*)), vốn đóng vai trò quan trọng trong sinh chuỗi và các nhiệm vụ liên quan.

Chú giải của người dịch: Không gian đầu ra "lớn theo cấp số mũ" là thế nào?

Giả sử bộ từ vựng chỉ có $|V| = 10\,000$ từ và câu đích dài $n = 10$ từ. Mỗi vị trí có $10\,000$ lựa chọn, nên số câu có thể có là $10\,000^{10} = 10^{40}$ — nhiều hơn rất xa số giây kể từ Vụ Nổ Lớn (khoảng $4 \cdot 10^{17}$). Ta không thể liệt kê hết để chọn câu tốt nhất, vì vậy cần các thuật toán tìm kiếm thông minh (Mục 5.4). So với bài toán phân loại cảm xúc chỉ có 2–3 lớp, đây là một khác biệt khổng lồ.

5.1 Các bài toán chuỗi-sang-chuỗi

Trong suốt chương này, chúng ta dùng dịch máy (*machine translation*) làm ví dụ minh họa, vì hiện nay đây là một trong những nhiệm vụ chuỗi-sang-chuỗi phổ biến nhất. Ta dùng $\mathbf{x} = x_1 \dots x_m$ để ký hiệu một chuỗi từ trong một ngôn ngữ (gọi là chuỗi phía nguồn (*source-side sequence*) hay *chuỗi nguồn*), và dùng $\mathbf{y} = y_1 \dots y_n$ để ký hiệu một chuỗi từ trong một ngôn ngữ khác (gọi là chuỗi phía đích (*target-side sequence*) hay *chuỗi đích*). Với ký hiệu này, Phương trình (5.1) có thể được viết lại thành:

$$\begin{aligned}\hat{\mathbf{y}} &= \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x}) \\ &= \arg\max_{y_1 \dots y_n} \Pr(y_1 \dots y_n | x_1 \dots x_m). \end{aligned} \tag{5.2}$$

Như đã bàn ở Chương 1 và trong [Brown và cộng sự, 1993], cách phát biểu này kéo theo ba vấn đề cơ bản.

- **Mô hình hóa.** Trước hết, ta cần xác định dạng của $\Pr(\mathbf{y}|\mathbf{x})$. Trong chương này, chúng ta sẽ thấy $\Pr(\mathbf{y}|\mathbf{x})$ có thể được mô hình hóa bằng một mạng nơ-ron duy nhất dựa trên kiến trúc bộ mã hóa–bộ giải mã và cơ chế chú ý. Lưu ý rằng đôi khi ta chỉ cần một mô hình để phân biệt chuỗi đích "tốt" với chuỗi đích "tồi". Khi đó, không nhất thiết đòi hỏi mô hình phải mang tính xác suất, và ta có thể dùng một hàm phân biệt (*discriminant function*) thay thế.

- **Huấn luyện.** Tiếp theo, ta cần học các tham số của mô hình $\Pr(\mathbf{y}|\mathbf{x})$ từ một tập dữ liệu huấn luyện. Vì $\Pr(\mathbf{y}|\mathbf{x})$ được hiện thực bằng một mạng nơ-ron, ta có thể huấn luyện nó theo cách tiêu chuẩn: tối ưu một hàm mất mát (*loss function*) bằng hạ gradient (*gradient descent*). Xem Chương 3 về các cách tiếp cận phổ biến để huấn luyện mạng nơ-ron. Trong chương này và các chương sau, chúng ta cũng sẽ bàn về các kỹ thuật được thiết kế riêng cho từng nhiệm vụ cụ thể.

- **Tìm kiếm (hay Giải mã).** Khi đã học được mô hình, ta sẽ tìm $\hat{\mathbf{y}}$ bằng cách tìm kiếm chuỗi đích làm cực đại $\Pr(\mathbf{y}|\mathbf{x})$. Điều này đặt ra một thách thức về tính toán, vì số chuỗi ứng viên tăng theo độ dài chuỗi tối đa và kích thước bộ từ vựng (*vocabulary*). Ở Mục 5.4, chúng ta sẽ bàn về các phương pháp tìm kiếm hiệu quả cho bài toán chuỗi-sang-chuỗi, đặc biệt là cho dịch máy.

Nhiều bài toán NLP có dạng như Phương trình (5.2) có thể được phát biểu thành bài toán chuỗi-sang-chuỗi. Bảng 5.1 liệt kê các ví dụ phổ biến về bài toán chuỗi-sang-chuỗi lấy từ các tài liệu nghiên cứu. Khi phía đích là văn bản, các bài toán như vậy có thể được xếp chung vào nhóm bài toán sinh văn bản (*text generation*), dù một hệ thống sinh văn bản tổng quát không đòi hỏi phía nguồn phải có dạng chuỗi. Ngoài xử lý ngôn ngữ và tiếng nói, bài toán chuỗi-sang-chuỗi có thể được tổng quát hóa cho những trường hợp mà đầu vào và/hoặc đầu ra của hệ thống vốn không có dạng chuỗi. Ví dụ, các hệ thống sinh văn bản từ ảnh (hay chú thích ảnh (*image captioning*)) và sinh ảnh từ văn bản đều phải xử lý ảnh, vốn thường được biểu diễn dưới dạng dữ liệu 2 chiều. Bằng cách biểu diễn ảnh thành chuỗi theo một cách nào đó (chẳng hạn chuỗi các *mảnh ảnh — patches*), các mô hình chuỗi-sang-chuỗi có thể được áp dụng trực tiếp cho các nhiệm vụ này.

Trong lịch sử, hầu hết các hệ thống cho những nhiệm vụ này được phát triển khá độc lập với nhau, dẫn đến các kiến trúc, đặc trưng và phương pháp huấn luyện khác nhau cho từng nhiệm vụ. Tuy nhiên, như sẽ thấy trong chương này, khi ta biểu diễn các mô hình này bằng mạng nơ-ron và huấn luyện chúng theo kiểu đầu-cuối (*end-to-end*), dường như xuất hiện một khuôn mẫu "phổ quát" cho tất cả các bài toán này. Đây là một thay đổi lớn đối với cộng đồng AI, vì nhiều lĩnh vực nghiên cứu hội tụ lại với nhau và các hệ thống có thể được dùng chung giữa các lĩnh vực. Ta có thể hiểu sâu hơn về bản chất chung của

Bảng 5.1: Các ví dụ về bài toán chuỗi-sang-chuỗi.

Nhiệm vụ	Nguồn	Đích
Dịch máy	Văn bản ở một ngôn ngữ	Văn bản ở ngôn ngữ khác
Hỏi đáp	Câu hỏi	Câu trả lời
Hệ thống hội thoại	Ngữ cảnh hội thoại	Câu đáp
Tóm tắt	Văn bản	Bản tóm tắt
Đơn giản hóa văn bản	Văn bản	Văn bản đã đơn giản hóa
Chuyển đổi phong cách văn bản	Văn bản theo một phong cách	Văn bản theo phong cách khác
Sửa lỗi ngữ pháp	Văn bản có lỗi	Văn bản đã sửa
Nhận dạng tiếng nói	Tiếng nói	Văn bản
Tổng hợp tiếng nói	Văn bản	Tiếng nói
Dịch tiếng nói	Tiếng nói ở một ngôn ngữ	Tiếng nói hoặc văn bản ở ngôn ngữ khác

rất nhiều bài toán khác nhau, dù trong thực tế vẫn có nhiều cân nhắc riêng cho từng nhiệm vụ. Trong các mục tiếp theo, chúng ta sẽ bàn về một số điểm chung giữa các mô hình chuỗi-sang-chuỗi.

Chú giải của người dịch: Huấn luyện "đầu-cuối"

Trước đây, một hệ thống dịch máy thường gồm nhiều mô-đun được xây dựng và tinh chỉnh riêng rẽ (tách từ, gióng hàng từ, bảng cụm từ, mô hình sắp xếp lại…). Huấn luyện *đầu-cuối* nghĩa là cả hệ thống là một mạng nơ-ron duy nhất, nhận đầu vào thô và trả ra đầu ra cuối cùng; mọi tham số được điều chỉnh đồng thời theo cùng một hàm mất mát. Nhờ vậy, cùng một "công thức" có thể dùng cho dịch máy, tóm tắt hay nhận dạng tiếng nói — chỉ cần thay dữ liệu huấn luyện.

5.2 Kiến trúc bộ mã hóa–bộ giải mã

Trong mục này, chúng ta bàn về kiến trúc bộ mã hóa–bộ giải mã và một mô hình dịch máy nơ-ron đơn giản xây dựng trên kiến trúc này.

5.2.1 Mã hóa và giải mã

Từ góc nhìn học có giám sát (*supervised learning*), lý tưởng nhất là ta muốn học một mô hình từ một số cặp chuỗi sao cho bất kỳ chuỗi phía nguồn nào cũng có thể được ánh xạ sang chuỗi phía đích tương ứng. Tuy nhiên, việc học các ánh xạ giữa những chuỗi biến rời rạc thường đòi hỏi xử lý dữ liệu nhiều chiều, và không tránh khỏi lời nguyền số chiều (*curse of dimensionality*).

Một cách tiếp cận để học ánh xạ như vậy là chia bài toán thành các bài toán con đơn giản hơn. Ta giả sử có một biểu diễn (*representation*) ít chiều được dùng chung bởi $\mathbf{x}$ và $\mathbf{y}$, ký hiệu là $\mathbf{H}$. Khi đó, ánh xạ $\mathbf{x} \rightarrow \mathbf{y}$ có thể được thực hiện bằng cách ánh xạ $\mathbf{x}$ sang $\mathbf{H}$ rồi từ đó sang $\mathbf{y}$. Một cách hình thức, cho một chuỗi phía nguồn $\mathbf{x}$, ta ánh xạ nó sang biểu diễn $\mathbf{H}$ bằng một hệ thống mã hóa (gọi là *bộ mã hóa*)

$$\mathbf{H} = \text{Encode}(\mathbf{x}). \tag{5.3}$$

Sau đó, ta ánh xạ $\mathbf{H}$ sang chuỗi phía đích $\mathbf{y}$ bằng một hệ thống giải mã (gọi là *bộ giải mã*)[1]

$$\mathbf{y} = \text{Decode}(\mathbf{H}). \tag{5.4}$$

Kiến trúc này, còn gọi là kiến trúc bộ mã hóa–bộ giải mã (*encoder-decoder architecture*), được dùng rộng rãi trong các hệ thống chuỗi-sang-chuỗi hiện đại (xem minh họa ở Hình 5.1). Dễ thấy dạng của Phương trình (5.3) giống với các mô hình chuỗi đã nhắc đến ở Chương 4, nên có nhiều mô hình mã hóa để lựa chọn, chẳng hạn LSTM hai chiều. Mục tiêu của bộ giải mã là sinh ra chuỗi phía đích "tốt nhất" khi biết biểu diễn của chuỗi phía nguồn. Giống như các mô hình phân loại, việc dự đoán được thực hiện bằng cách trước hết tạo ra một phân phối trên mọi chuỗi có thể, rồi chọn chuỗi có xác suất lớn nhất. Như vậy, ta có thể định nghĩa lại $\text{Decode}(\cdot)$ như một hàm xác suất

$$\begin{aligned}\Pr(\cdot|\mathbf{H}) &= \text{Decode}(\mathbf{H}) \\ &= \text{Decode}(\text{Encode}(\mathbf{x})).\end{aligned} \tag{5.5}$$

Nói cách khác, cho một chuỗi phía đích $\mathbf{y}$ và một chuỗi phía nguồn $\mathbf{x}$, bộ giải mã gán cho $\mathbf{y}$ một xác suất

$$\Pr(\mathbf{y}|\mathbf{x}) = \Pr(\mathbf{y}|\mathbf{H}). \tag{5.6}$$

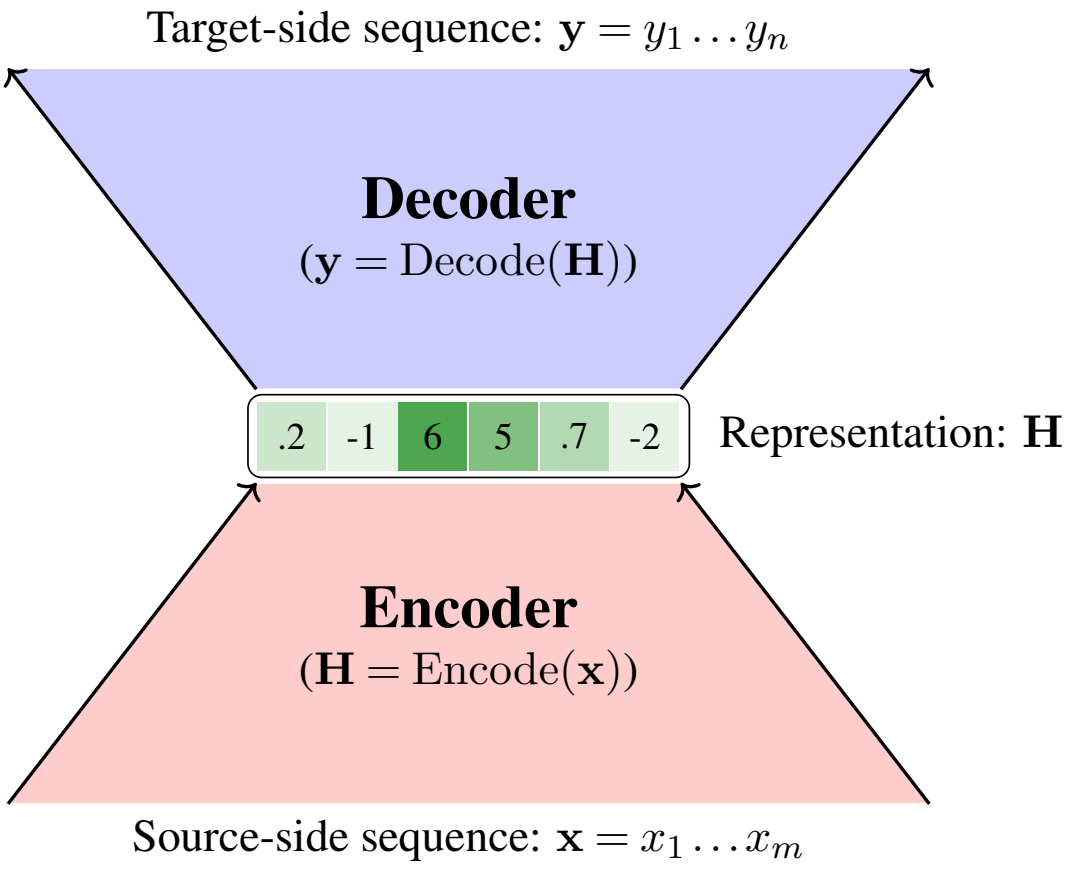
Sau đó, chuỗi tối ưu $\hat{\mathbf{y}}$ thu được bằng cách thực hiện $\arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x})$ như trong Phương trình (5.2). Trong nhiều hệ thống dựa trên kiến trúc bộ mã hóa–bộ giải mã, cả $\text{Encode}(\cdot)$ và $\text{Decode}(\cdot)$ đều là các mô hình được xây dựng từ mạng nơ-ron. Vì vậy, ta có thể coi mô hình chuỗi-sang-chuỗi là một mạng nơ-ron duy nhất và huấn luyện nó như thường lệ, miễn là toàn bộ mô hình là một tổ hợp nào đó của $\text{Encode}(\cdot)$ và $\text{Decode}(\cdot)$.

Chú giải của người dịch: Hình dung bộ mã hóa–bộ giải mã

Hãy tưởng tượng một phiên dịch viên: trước hết họ nghe hết câu tiếng Anh và "hiểu ý" trong đầu (đó là bước *mã hóa*, ý hiểu chính là $\mathbf{H}$), rồi diễn đạt ý đó thành câu tiếng Việt (bước *giải mã*). Trong máy, "ý hiểu" $\mathbf{H}$ là một dãy số thực, chẳng hạn vectơ $[0.2, -1, 6, 5, 0.7, -2]$ trong Hình 5.1. Ký hiệu $\Pr(\mathbf{y}|\mathbf{x})$ đọc là "xác suất của $\mathbf{y}$ khi đã biết $\mathbf{x}$" (xác suất có điều kiện): mô hình cho biết câu dịch $\mathbf{y}$ "hợp lý" đến mức nào đối với câu nguồn $\mathbf{x}$.

Để áp dụng kiến trúc bộ mã hóa–bộ giải mã vào một nhiệm vụ thực tế, ta cần đưa ra một số lựa chọn thiết kế, chẳng hạn dạng của $\mathbf{H}$, $\text{Encode}(\cdot)$ và $\text{Decode}(\cdot)$. Lấy một ví dụ rất đơn giản: xét nhiệm vụ tái tạo lại một từ đầu vào. Ta có thể định nghĩa $\text{Encode}(\cdot)$ là một mạng nơ-ron truyền thẳng nhận vào một từ (ở dạng biểu diễn one-hot (*one-hot*)) và trả ra một vectơ từ. Theo cách này, $\mathbf{H}$ là một biểu diễn phân tán (*distributed representation*) của từ đó. Sau đó, ta định nghĩa $\text{Decode}(\cdot)$ là một mạng nơ-ron truyền thẳng khác, nhận vectơ từ và sinh ra một phân phối trên bộ từ vựng. Khi huấn luyện, ta mong muốn học được một hệ thống gán xác suất lớn nhất cho chính từ đầu vào. Như đã bàn ở Chương 2, ta có thể gọi hệ thống này là một bộ tự mã hóa (*auto-encoder*), vốn là một trường hợp đặc biệt của kiến trúc bộ mã hóa–bộ giải mã.

[1] Điều quan trọng là phải phân biệt khái niệm giải mã (hay bộ giải mã) dùng trong các hệ thống chuỗi-sang-chuỗi truyền thống với khái niệm dùng trong kiến trúc bộ mã hóa–bộ giải mã. Hai khái niệm này thường bị nhầm lẫn, dù bản chất của chúng khác nhau. Trong nhiều hệ thống dịch máy hay nhận dạng tiếng nói, giải mã có cùng nghĩa với dịch hay phiên âm, tức là ta khôi phục $\mathbf{y}$ tối ưu từ $\mathbf{x}$. Như đã chỉ ra ở Phương trình (5.2), quá trình này bao gồm việc tìm kiếm trên mọi ứng viên $\mathbf{y}$. Do đó, cách dùng truyền thống của từ giải mã trong các hệ thống này là để chỉ một quá trình tìm kiếm (tức phép toán arg max trong Phương trình (5.2)) [Koehn, 2010]. Ngược lại, trong kiến trúc bộ mã hóa–bộ giải mã, giải mã có nghĩa là quá trình khôi phục chuỗi phía đích $\mathbf{y}$ từ biểu diễn trung gian $\mathbf{H}$. Nó hoàn toàn thuộc về mô hình hóa chứ không phải tìm kiếm. Cũng cần lưu ý rằng, dù thuật ngữ giải mã (hay bộ giải mã) được dùng theo những cách khác nhau, nó đều có thể được hiểu là quá trình ánh xạ một thông điệp đã mã hóa trở lại thông điệp gốc trong một hệ thống truyền thông như định nghĩa trong lý thuyết thông tin [Shannon, 1948b]. Theo nghĩa này, các quá trình giải mã trong những hệ thống đó đều làm đúng như tên gọi của nó: chuyển một thứ gì đó về dạng ban đầu.



Hình 5.1: Kiến trúc bộ mã hóa–bộ giải mã. Trong trường hợp bài toán chuỗi-sang-chuỗi, nó biến đổi một chuỗi phía nguồn $\mathbf{x} = x_1 \dots x_m$ thành một chuỗi phía đích $\mathbf{y} = y_1 \dots y_n$. Quy trình này gồm hai bước: trước hết $\mathbf{x}$ được mã hóa thành một biểu diễn $\mathbf{H}$, rồi biểu diễn này được giải mã thành $\mathbf{y}$. (Nhãn trong hình: *Source-side sequence* = chuỗi phía nguồn, *Target-side sequence* = chuỗi phía đích, *Representation* = biểu diễn.)

5.2.2 Ví dụ: Dịch máy nơ-ron

Tiếp theo, chúng ta minh họa việc áp dụng kiến trúc bộ mã hóa–bộ giải mã qua một ví dụ cụ thể: **dịch máy nơ-ron** (*neural machine translation*) (**NMT**). Ta xét một mô hình NMT nổi tiếng dùng RNN hoặc các biến thể của nó cho cả bộ mã hóa lẫn bộ giải mã [Cho và cộng sự, 2014; Sutskever và cộng sự, 2014]. Bộ mã hóa của mô hình NMT này là một mô hình chuỗi tiêu chuẩn dựa trên RNN. Vì mô hình chuỗi dựa trên RNN đã được bàn chi tiết ở Chương 4, ở đây chúng ta chỉ điểm lại ngắn gọn. Giả sử bộ từ vựng phía nguồn là V_x và mỗi từ phía nguồn x_j được biểu diễn bằng một vectơ one-hot trong $\mathbb{R}^{|V_x|}$. Khi đó, x_j được biến đổi thành một vectơ h_s chiều (hay nhúng từ (*word embedding*))

$$\mathbf{x}_j^e = \text{Embed}_\text{s}(x_j), \tag{5.7}$$

trong đó $\text{Embed}_\text{s}(\cdot)$ là hàm nhúng từ. Chi tiết hơn về các mô hình nhúng từ có thể xem ở Chương 3.

Mô hình RNN nhận chuỗi các vectơ nhúng từ $\mathbf{x}_1^e \dots \mathbf{x}_m^e$ và sinh ra một chuỗi các vectơ trạng thái RNN $\mathbf{h}_1 \dots \mathbf{h}_m$. Một vectơ trạng thái RNN $\mathbf{h}_j \in \mathbb{R}^{d_h}$ được định nghĩa là

$$\mathbf{h}_j = \text{RNN}(\mathbf{h}_{j-1}, \mathbf{x}_j^e). \tag{5.8}$$

Ở đây, $\text{RNN}(\cdot)$ là một đơn vị RNN tóm tắt thông tin tính đến vị trí j bằng cách kết hợp trạng thái trước đó $\mathbf{h}_{j-1}$ và đầu vào hiện tại $\mathbf{x}_j^e$. Khi đó, trạng thái cuối cùng $\mathbf{h}_m$ có thể được xem là một biểu diễn nén của chuỗi đầu vào $x_1 \dots x_m$, và ta dùng $\mathbf{h}_m$ làm đầu ra của bộ mã hóa, viết là

$$\mathbf{h}_m = \text{Encode}(x_1 \dots x_m). \tag{5.9}$$

Hình 5.2 (a–b) minh họa quá trình mã hóa. Lưu ý rằng mô hình mô tả ở trên chỉ gồm một RNN một tầng. Trong các hệ thống thực tế, khung này có thể dễ dàng được mở rộng để có nhiều tầng và các đơn vị hồi quy mạnh hơn (chẳng hạn các đơn vị LSTM).

Chú giải của người dịch: RNN "đọc" câu nguồn như thế nào?

Hãy hình dung RNN như một người đọc từng từ một và ghi chú vào một cuốn sổ nhỏ có kích thước cố định (d_h ô số). Ở mỗi bước j, người đó nhìn ghi chú cũ $\mathbf{h}_{j-1}$ và từ mới $\mathbf{x}_j^e$, rồi viết lại ghi chú mới $\mathbf{h}_j$. Ví dụ với câu "I love cats" ($m = 3$): $\mathbf{h}_1$ tóm tắt "I", $\mathbf{h}_2$ tóm tắt "I love", $\mathbf{h}_3$ tóm tắt cả câu — và $\mathbf{h}_3$ chính là $\mathbf{H}$ được chuyển sang bộ giải mã. Ký hiệu $\mathbf{h}_j \in \mathbb{R}^{d_h}$ nghĩa là $\mathbf{h}_j$ là một vectơ gồm d_h số thực.

Bộ giải mã của mô hình NMT là một mô hình ngôn ngữ (*language model*) tiêu chuẩn dựa trên RNN, tức là nó dự đoán từ tiếp theo y_{i+1} khi biết mọi từ trước đó $y_1 \dots y_i$. Để đưa thông tin phía nguồn vào bản dịch, một cách đơn giản là coi $\mathbf{h}_m$ là trạng thái khởi đầu của RNN phía đích. Gọi $\mathbf{y}_0^e \in \mathbb{R}^{d_s}$ là vectơ từ của ký hiệu bắt đầu $\langle\text{SOS}\rangle$ (ký hiệu là y_0). Trạng thái RNN khởi đầu tương ứng được cho bởi

$$\mathbf{s}_0 = \text{RNN}(\mathbf{h}_m, \mathbf{y}_0^e). \tag{5.10}$$

Ở đây, $\text{RNN}(\cdot)$ có cùng dạng với đơn vị hồi quy dùng trong bộ mã hóa, nhưng với các tham số khác.

Với $i > 0$, vectơ trạng thái $\mathbf{s}_i \in \mathbb{R}^{d_s}$ được định nghĩa là

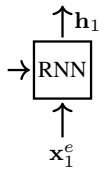
$$\mathbf{s}_i = \text{RNN}(\mathbf{s}_{i-1}, \mathbf{y}_i^e). \tag{5.11}$$

Sau đó, $\mathbf{s}_i$ được đưa vào một tầng Softmax để tạo ra một phân phối trên bộ từ vựng phía đích V_y. Đầu ra của tầng Softmax được cho bởi

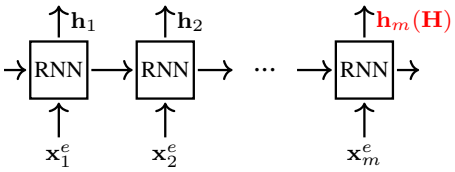
$$\Pr(\cdot|y_1 \dots y_i, x_1 \dots x_m) = \Pr(\cdot|\mathbf{s}_i)$$

$$= \text{Softmax}(\mathbf{s}_i \mathbf{U}_y + \mathbf{b}_y), \tag{5.12}$$

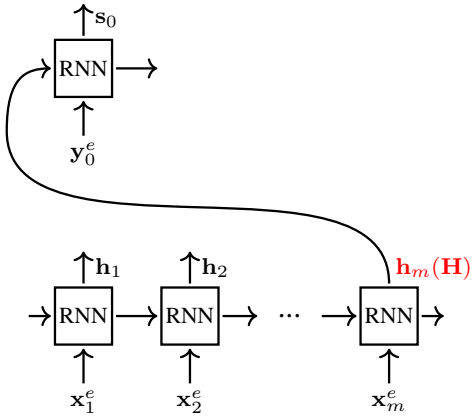
trong đó $\mathbf{U}_y \in \mathbb{R}^{d_s \times |V_y|}$ và $\mathbf{b}_y \in \mathbb{R}^{|V_y|}$ là các tham số của tầng Softmax. $\Pr(y_{i+1}|y_1 \dots y_i, x_1 \dots x_m)$ có thể được xem là xác suất dự đoán từ y_{i+1} với điều kiện đã biết cả các từ đã dịch $y_1 \dots y_i$ và chuỗi phía nguồn $x_1 \dots x_m$. Xem Hình 5.2 (c–d) để thấy minh họa về việc dự đoán từ của một bộ giải mã.



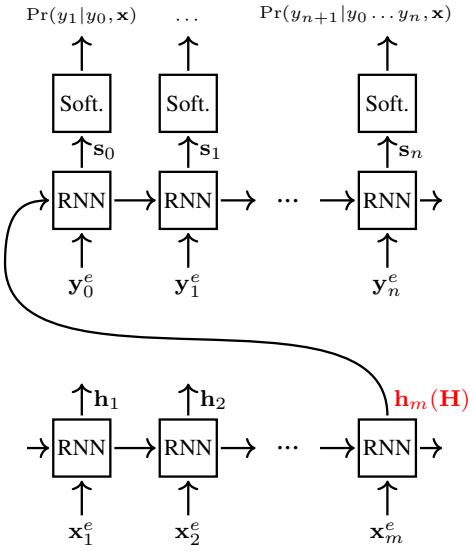
(a) The encoding process starts



(b) The encoder represents $\mathbf{x}$ as $\mathbf{H}$



(c) The decoder takes the representation of $\mathbf{x}$



(d) The decoder predicts the target-side words

Hình 5.2: Các bước mã hóa và giải mã của một hệ thống NMT dựa trên RNN. Bộ mã hóa là một RNN tiêu chuẩn. Quá trình mã hóa bắt đầu từ từ đầu tiên của phía nguồn và kết thúc ở từ cuối cùng của phía nguồn. Trạng thái cuối của RNN được lấy làm biểu diễn của toàn bộ chuỗi phía nguồn (tức $\mathbf{H} = \mathbf{h}_m$). Bộ giải mã là một RNN khác. Ở bước đầu tiên, nó nhận $\mathbf{H}$ từ bộ mã hóa. Sau khi biểu diễn $(\mathbf{y}_0^e \dots \mathbf{y}_i^e, \mathbf{H})$ thành $\mathbf{s}_i$ tại vị trí i, một tầng Softmax được áp dụng để dự đoán từ tiếp theo y_{i+1}. (Các tiểu hình: (a) quá trình mã hóa bắt đầu; (b) bộ mã hóa biểu diễn $\mathbf{x}$ thành $\mathbf{H}$; (c) bộ giải mã nhận biểu diễn của $\mathbf{x}$; (d) bộ giải mã dự đoán các từ phía đích. "Soft." là tầng Softmax.)

Dựa trên mô hình dự đoán từ này, xác suất đồng thời thường dùng trong các tài liệu về NMT được phát biểu như sau:

$$\begin{aligned}
\Pr(\mathbf{y}|\mathbf{x}) &= \Pr(y_0 y_1 \dots y_n | x_1 \dots x_m) \\
&= \Pr(y_0|x_1 \dots x_m) \Pr(y_1 \dots y_n | y_0, x_1 \dots x_m) \\
&= \prod_{i=0}^{n-1} \Pr(y_{i+1}|y_0 \dots y_i, x_1 \dots x_m).
\end{aligned} \tag{5.13}$$

Đôi khi, phương trình này cũng được viết dưới một dạng tương đương:

$$\Pr(\mathbf{y}|\mathbf{x}) = \prod_{i=1}^{n} \Pr(y_i|y_0 \dots y_{i-1}, x_1 \dots x_m). \tag{5.14}$$

Ở đây, ta giả sử $\mathbf{y}$ luôn bắt đầu bằng y_0 (tức $\langle\text{SOS}\rangle$), sao cho $\Pr(y_0|x_1 \dots x_m) = 1$. Trong nhiều hệ thống thực tế, người ta cũng thường giả sử $\mathbf{y}$ kết thúc bằng một ký hiệu đặc biệt $\langle\text{EOS}\rangle$. Do đó, ta có thể sửa phương trình này để bao gồm $\langle\text{SOS}\rangle$ và $\langle\text{EOS}\rangle$ ở cả phía nguồn lẫn phía đích như sau:

$$\begin{aligned}
\Pr(y_0\mathbf{y}y_{n+1}|x_0\mathbf{x}x_{m+1}) &= \Pr(y_0y_1 \dots y_ny_{n+1}|x_0x_1 \dots x_mx_{m+1}) \\
&= \Pr(y_0|x_0 \dots x_{m+1}) \cdot \\
&\quad \Pr(y_1 \dots y_ny_{n+1}|y_0, x_0 \dots x_{m+1}) \\
&= \prod_{i=0}^{n} \Pr(y_{i+1}|y_0 \dots y_i, x_0 \dots x_{m+1}),
\end{aligned} \tag{5.15}$$

trong đó $x_0 = y_0 = \langle\text{SOS}\rangle$, $x_{m+1} = y_{n+1} = \langle\text{EOS}\rangle$, và $\Pr(y_0|x_0x_1 \dots x_mx_{m+1}) = 1$.

Chú giải của người dịch: Tách xác suất của cả câu thành tích các xác suất từng từ

Phương trình (5.13) dùng *quy tắc nhân xác suất*: $\Pr(A, B) = \Pr(A) \cdot \Pr(B|A)$, áp dụng lặp lại. Ví dụ dịch "I love cats" thành "tôi yêu mèo" ($n = 3$), mô hình tính $\Pr(\text{tôi}|\langle\text{SOS}\rangle, \mathbf{x}) \cdot \Pr(\text{yêu}|\langle\text{SOS}\rangle \text{ tôi}, \mathbf{x}) \cdot \Pr(\text{mèo}|\langle\text{SOS}\rangle \text{ tôi yêu}, \mathbf{x})$. Nếu ba xác suất này lần lượt là 0.6, 0.5, 0.8 thì $\Pr(\mathbf{y}|\mathbf{x}) = 0.6 \cdot 0.5 \cdot 0.8 = 0.24$. Ký hiệu $\prod$ là tích (giống $\sum$ là tổng). Ký hiệu $\langle\text{EOS}\rangle$ (*end of sequence*) giúp mô hình "biết" khi nào dừng sinh từ, còn $\langle\text{SOS}\rangle$ (*start of sequence*) là tín hiệu bắt đầu.

Vì $\Pr(\mathbf{y}|\mathbf{x})$ có thể được hiện thực bằng một mạng nơ-ron, việc huấn luyện mô hình này khá đơn giản. Như đã mô tả ở Chương 4, các mô hình ngôn ngữ dựa trên RNN được huấn luyện bằng mất mát entropy chéo (*cross-entropy*) và hạ gradient. NMT dùng chính phương pháp này để huấn luyện các tham số mô hình. Khi có được mô hình đã tối ưu, ta có thể dùng nó để dịch các câu mới. Việc tìm bản dịch tốt nhất cho một câu phía nguồn bất kỳ là một bài toán tìm kiếm tiêu chuẩn, mà chúng ta sẽ bàn ở Mục 5.4.

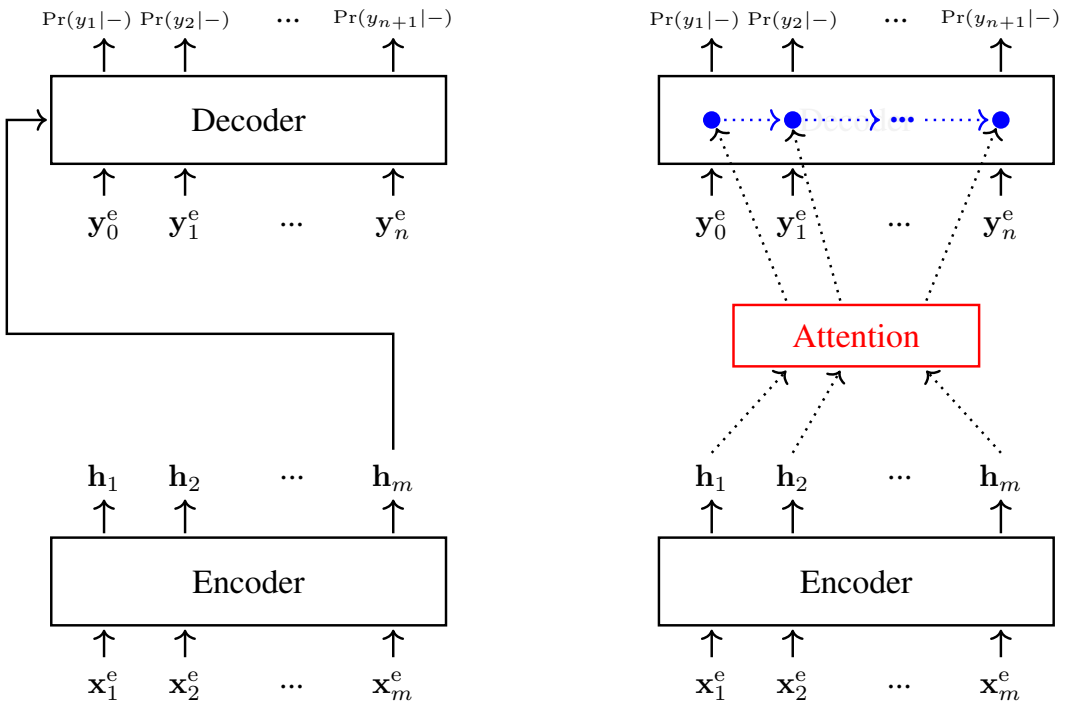
5.3 Cơ chế chú ý

Mô hình NMT bàn ở mục trước dựa trên một biểu diễn có độ dài cố định của chuỗi phía nguồn. Mặc dù mô hình này dễ hiện thực, trong nhiều ứng dụng thực tế nó thường không đủ tốt, vì một vectơ có độ dài cố định không đủ để biểu diễn một chuỗi có độ dài thay đổi, đặc biệt khi chuỗi dài. Do đó, ta cần một cơ chế nào đó để gắn kết bộ mã hóa và bộ giải mã một cách chi tiết hơn. Trong mục này, chúng ta bàn về cơ chế chú ý, nhờ đó mô hình có thể học, cho mỗi từ trong chuỗi phía đích, một biểu diễn thích ứng tập trung nhiều hơn vào những phần quan trọng của chuỗi phía nguồn.

Ý tưởng này liên hệ chặt chẽ với các mô hình về sự chú ý trong tâm lý học, vì bản thân việc dịch là một quá trình nhận thức [Sternberg, 1996; Neisser, 2014]. Ý tưởng then chốt đằng sau loại mô hình này rất trực quan: sự chú ý thường tập trung vào những phần cụ thể của đầu vào. Trực giác này tạo nền tảng cho nhiều mô hình chuỗi-sang-chuỗi tiên tiến nhất, và cơ chế chú ý đã trở thành tiêu chuẩn trên thực tế trong mô hình hóa chuỗi-sang-chuỗi.

5.3.1 Một mô hình cơ bản

Nhớ lại rằng trong mô hình NMT ở mục trước, bộ mã hóa biểu diễn một chuỗi từ phía nguồn thành $\mathbf{h}_1, \ldots, \mathbf{h}_m$, còn bộ giải mã biểu diễn một chuỗi từ phía đích thành $\mathbf{s}_1, \ldots, \mathbf{s}_n$. Cơ chế chú ý giải quyết câu hỏi: làm thế nào học được một biểu diễn từ $\mathbf{h}_1, \ldots, \mathbf{h}_m$ sao cho biểu diễn này mã hóa tốt chuỗi phía nguồn đối với một trạng thái truy vấn $\mathbf{s}_i$ cho trước[2]. Từ góc độ xử lý thông tin, chừng nào ta bỏ qua ý nghĩa của $\mathbf{h}_1, \ldots, \mathbf{h}_m$ và $\mathbf{s}_i$ trong NMT, cơ chế chú ý có thể được coi là một cơ chế tổng quát để xử lý thông tin đầu vào $\mathbf{h}_1, \ldots, \mathbf{h}_m$ bằng cách xem xét mỗi $\mathbf{h}_j$ liên quan đến trạng thái truy vấn $\mathbf{s}_i$ như thế nào. Hình 5.3 so sánh các kiến trúc NMT có và không có cơ chế chú ý.



(a) An NMT system without attention (b) An NMT system with attention

Hình 5.3: Các kiến trúc NMT không có (trái) và có (phải) mô hình chú ý. Khi không dùng mô hình chú ý, một biểu diễn có độ dài cố định được dùng để sinh toàn bộ chuỗi phía đích. Ngược lại, khi dùng mô hình chú ý, một biểu diễn mới được tính riêng cho từng trạng thái phía đích, nhờ đó bộ giải mã có thể học cách tập trung vào các phần khác nhau của chuỗi phía nguồn để dự đoán một từ phía đích. ((a) Hệ thống NMT không có cơ chế chú ý; (b) hệ thống NMT có cơ chế chú ý.)

Một cách hình thức hơn, một mô hình chú ý tạo ra một tổ hợp có trọng số của $\{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$ có dạng

$$\mathbf{c}_i = \sum_{j=1}^{m} \alpha_{i,j} \cdot \mathbf{h}_j, \tag{5.16}$$

trong đó $\alpha_{i,j}$ là trọng số chú ý (*attention weight*), mô tả mô hình nên dựa vào $\mathbf{h}_j$ đến mức nào khi tính $\mathbf{c}_i$ cho $\mathbf{s}_i$. Đôi khi $\mathbf{c}_i$ còn được gọi là vectơ ngữ cảnh (*context vector*).

[2]Theo quy ước trong dịch máy [Brown và cộng sự, 1993], ta dùng j để chỉ một vị trí trong chuỗi phía nguồn, và dùng i để chỉ một vị trí trong chuỗi phía đích.

Một cách phổ biến để tính trọng số chú ý là chuẩn hóa các điểm gióng hàng (*alignment score*) theo dạng sau

$$\alpha_{i,j} = \frac{\exp(a(\mathbf{s}_i, \mathbf{h}_j))}{\sum_{j'=1}^{m} \exp(a(\mathbf{s}_i, \mathbf{h}_{j'}))}. \tag{5.17}$$

Ở đây, điểm gióng hàng $a(\mathbf{s}_i, \mathbf{h}_j)$ đo mức độ liên quan mạnh đến đâu giữa $\mathbf{h}_j$ và $\mathbf{s}_i$.

Chú giải của người dịch: Trọng số chú ý: một phép "bình quân có trọng số"

Phương trình (5.17) chính là hàm Softmax áp dụng lên các điểm $a(\mathbf{s}_i, \mathbf{h}_1), \ldots, a(\mathbf{s}_i, \mathbf{h}_m)$: mọi $\alpha_{i,j}$ đều dương và có tổng bằng 1. Ví dụ, với $m = 3$ và các điểm lần lượt là 2, 0, 0: $e^2 \approx 7.39$, $e^0 = 1$, nên $\alpha_{i,\cdot} \approx (7.39/9.39,\ 1/9.39,\ 1/9.39) \approx (0.79,\ 0.11,\ 0.11)$. Phương trình (5.16) khi đó giống cách tính *điểm trung bình môn có hệ số*: $\mathbf{c}_i \approx 0.79\,\mathbf{h}_1 + 0.11\,\mathbf{h}_2 + 0.11\,\mathbf{h}_3$ — vectơ ngữ cảnh chủ yếu "lấy" thông tin từ từ nguồn thứ nhất. Khi dịch từ "mèo", mô hình sẽ dồn trọng số vào vị trí của "cats".

Nói chung, $a(\mathbf{s}_i, \mathbf{h}_j)$ có thể được định nghĩa theo nhiều cách khác nhau [Graves và cộng sự, 2014; Bahdanau và cộng sự, 2014; Luong và cộng sự, 2015]. Một danh sách đầy đủ các hàm này có thể tìm thấy trong các bài tổng quan về chủ đề này [Chaudhari và cộng sự, 2021]. Dưới đây chúng ta giới thiệu một số hàm phổ biến.

- **Chú ý tích vô hướng** (*Dot-product Attention*). Một trong những phương pháp đơn giản nhất là đo độ tương đồng giữa $\mathbf{h}_j$ và $\mathbf{s}_i$. Như vậy, ta có thể tính tích vô hướng (*dot-product*) của hai vectơ như sau

$$\begin{aligned} a(\mathbf{s}_i, \mathbf{h}_j) &= \mathbf{s}_i \mathbf{h}_j^\top \\ &= \sum_{k=1}^{d_h} s_i(k) \cdot h_j(k). \end{aligned} \tag{5.18}$$

  Một biến thể của mô hình này, gọi là **chú ý tích vô hướng có co giãn** (*scaled dot-product attention*), nhân vế phải của Phương trình (5.18) với hệ số co giãn $\frac{1}{\beta}$ như sau

$$a(\mathbf{s}_i, \mathbf{h}_j) = \frac{\mathbf{s}_i \mathbf{h}_j^\top}{\beta}, \tag{5.19}$$

  trong đó β là một siêu tham số. Chúng ta sẽ thấy một ví dụ về mô hình này ở phần sau của mục này.

- **Chú ý cosin** (*Cosine Attention*). Một độ đo tương đồng phổ biến khác trong đại số vectơ là cosin của góc giữa hai vectơ, cho bởi

$$\begin{aligned} a(\mathbf{s}_i, \mathbf{h}_j) &= \cos(\mathbf{s}_i, \mathbf{h}_j) \\ &= \frac{\mathbf{s}_i \mathbf{h}_j^\top}{\|\mathbf{s}_i\|_2 \cdot \|\mathbf{h}_j\|_2}, \end{aligned} \tag{5.20}$$

  trong đó $\|\mathbf{a}\|_2 = (\mathbf{a} \cdot \mathbf{a})^{\frac{1}{2}}$ là chuẩn Euclid (*Euclidean norm*) của vectơ $\mathbf{a}$.

- **Chú ý tích vô hướng có trọng số** (*Weighted Dot-product Attention*). Mô hình chú ý này thực hiện một ánh xạ tuyến tính lên các vectơ đầu vào trước khi thực hiện phép tích vô hướng, cho bởi

$$a(\mathbf{s}_i, \mathbf{h}_j) = \mathbf{s}_i \mathbf{W}_\text{a} \mathbf{h}_j^\top, \tag{5.21}$$

  trong đó $\mathbf{W}_\text{a} \in \mathbb{R}^{d_h \times d_h}$ là ma trận tham số của ánh xạ tuyến tính. Cả cách tiếp cận này và cách chú ý tích vô hướng đều còn được gọi là **chú ý nhân** (*multiplicative attention*) [Ruder, 2017].

- **Chú ý cộng** (*Additive Attention*). Trong chú ý cộng, các phần tử của hai vectơ được cộng lại theo một cách nào đó. Một dạng được dùng rộng rãi được đưa ra bởi Bahdanau và cộng sự [2014]

$$a(\mathbf{s}_i, \mathbf{h}_j) = \mathbf{v}_\mathrm{a}^\top \mathrm{TanH}(\mathbf{s}_i \mathbf{W}_\mathrm{s} + \mathbf{h}_j \mathbf{W}_\mathrm{h}), \tag{5.22}$$

trong đó $\mathbf{W}_\mathrm{h}, \mathbf{W}_\mathrm{s} \in \mathbb{R}^{d_h \times d_a}$ và $\mathbf{v}_\mathrm{a} \in \mathbb{R}^{d_a}$ là các tham số. $\mathrm{TanH}(\mathbf{s}_i \mathbf{W}_\mathrm{s} + \mathbf{h}_j \mathbf{W}_\mathrm{h})$ tạo ra một vectơ d_a chiều, trong đó mỗi phần tử là một tổng có trọng số đã được biến đổi của các phần tử của $\mathbf{h}_j$ và $\mathbf{s}_i$. Tiếp theo là một phép tích vô hướng với một vectơ trọng số khác $\mathbf{v}_\mathrm{a}$.

Chú giải của người dịch: Tích vô hướng và cosin — liên hệ với hình học lớp 10

Ở lớp 10 ta có $\vec{u} \cdot \vec{v} = |\vec{u}|\,|\vec{v}| \cos(\vec{u}, \vec{v})$, nên $\cos(\vec{u}, \vec{v}) = \dfrac{\vec{u} \cdot \vec{v}}{|\vec{u}|\,|\vec{v}|}$ — đó chính là Phương trình (5.20), chỉ khác là vectơ có d_h tọa độ thay vì 2. Ví dụ $\mathbf{s}_i = [1, 2]$, $\mathbf{h}_j = [3, 1]$: tích vô hướng $= 1 \cdot 3 + 2 \cdot 1 = 5$, $\|\mathbf{s}_i\|_2 = \sqrt{5}$, $\|\mathbf{h}_j\|_2 = \sqrt{10}$, nên cosin $= 5/\sqrt{50} \approx 0.71$. Hai vectơ càng "cùng hướng" thì điểm càng lớn, tức $\mathbf{h}_j$ càng liên quan đến $\mathbf{s}_i$. Cách viết $\mathbf{s}_i \mathbf{h}_j^\top$ là tích của vectơ hàng $\mathbf{s}_i$ với vectơ cột $\mathbf{h}_j^\top$ (chuyển vị của $\mathbf{h}_j$), cho ra đúng tổng $\sum_k s_i(k) h_j(k)$. Hàm TanH là hàm tang hyperbolic, "nén" mọi số thực về khoảng $(-1, 1)$.

Giờ hãy quay lại Phương trình (5.16–5.17) và suy nghĩ lại về vai trò của các trọng số chú ý. Phương trình (5.17) định nghĩa một cách không chính thức một "phân phối" trên $\mathbf{h}_1 \dots \mathbf{h}_m$, viết là

$$\Pr(\mathbf{h}_j | \mathbf{s}_i) = \alpha_{i,j}. \tag{5.23}$$

Nếu ta coi $\mathbf{h}$ là một biến ngẫu nhiên nhận giá trị trong $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$, thì $\alpha_{i,j}$ có thể được xem là xác suất để $\mathbf{h} = \mathbf{h}_j$ với điều kiện $\mathbf{s}_i$, và Phương trình (5.16) có thể được viết lại thành

$$\begin{aligned} \mathbf{c}_i &= \sum_{j=1}^{m} \Pr(\mathbf{h}_j | \mathbf{s}_i) \cdot \mathbf{h}_j \\ &= \mathbb{E}_{\mathbf{h} \sim \Pr(\mathbf{h}|\mathbf{s}_i)}(\mathbf{h}). \end{aligned} \tag{5.24}$$

Nói cách khác, $\mathbf{c}_i$ có thể được xem là một *biểu diễn kỳ vọng* của chuỗi phía nguồn khi biết trạng thái phía đích $\mathbf{s}_i$, tức là kỳ vọng (*expectation*) của $\mathbf{h}$ trên tập $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ theo phân phối $\Pr(\mathbf{h}_j | \mathbf{s}_i)$. Điều này cho ta một khung tổng quát để mô tả cách bộ giải mã nhận thông tin từ bộ mã hóa: bộ giải mã là một *bên nhận*, quyết định nhận bao nhiêu thông tin từ mỗi *bên gửi*. Chẳng hạn, trong mô hình NMT ở mục trước, chỉ có một bên gửi là $\mathbf{h}_m$, nên bên nhận nhận toàn bộ thông tin mà bên gửi gửi đến. Ngược lại, trong mô hình NMT được trang bị cơ chế chú ý, có m bên gửi $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ và bên nhận nhận thông tin theo một phân phối mức ưu tiên trên các bên gửi.

Chú giải của người dịch: Kỳ vọng — "giá trị trung bình theo xác suất"

Ký hiệu $\mathbb{E}_{\mathbf{h} \sim \Pr(\mathbf{h}|\mathbf{s}_i)}(\mathbf{h})$ đọc là "kỳ vọng của $\mathbf{h}$ khi $\mathbf{h}$ được rút ngẫu nhiên theo phân phối $\Pr(\mathbf{h}|\mathbf{s}_i)$". Với biến ngẫu nhiên rời rạc, kỳ vọng là tổng "giá trị × xác suất". Ví dụ quen thuộc: tung một con xúc xắc cân đối, kỳ vọng số chấm là $1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + \dots + 6 \cdot \frac{1}{6} = 3.5$. Ở đây "giá trị" là các vectơ $\mathbf{h}_j$ và "xác suất" là $\alpha_{i,j}$, nên kỳ vọng chính là tổng có trọng số ở Phương trình (5.16).

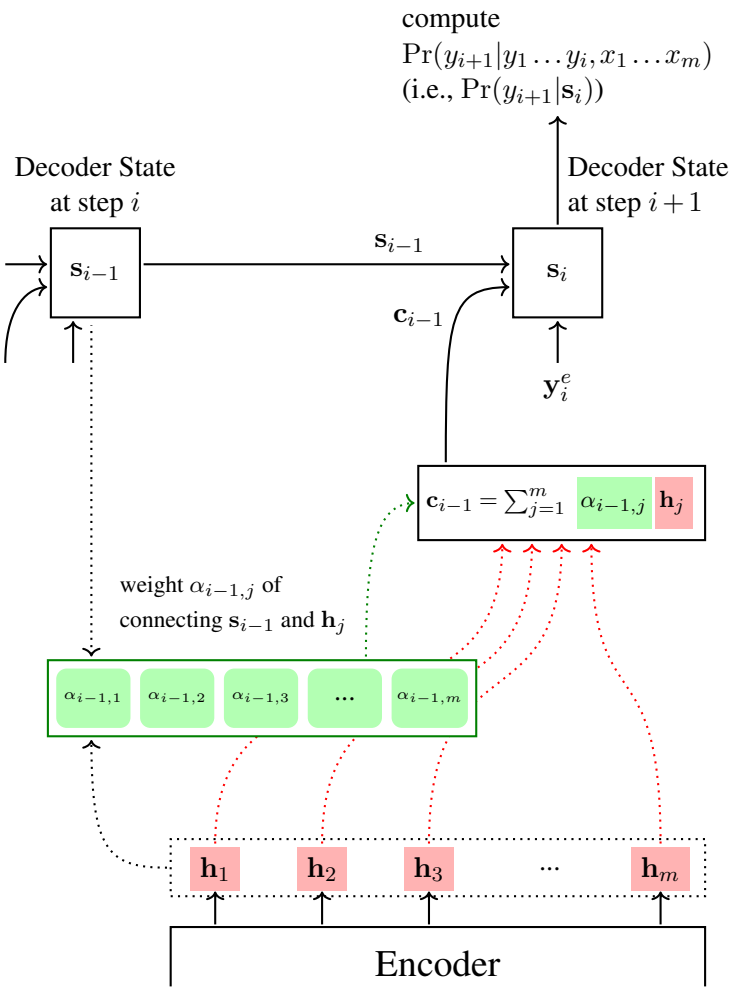
Việc đưa mô hình chú ý vào quá trình dự đoán từ khá đơn giản. Để tận dụng cả thông tin phía nguồn và phía đích ở mỗi bước giải mã, ta sửa nhẹ định nghĩa của $\mathbf{s}_i$ để bao gồm vectơ ngữ cảnh tương ứng với trạng thái trước đó $\mathbf{s}_{i-1}$, như sau:

$$\mathbf{s}_i = \mathrm{RNN}(\mathbf{s}_{i-1}, \mathbf{c}_{i-1}, \mathbf{y}_i^e). \tag{5.25}$$

So với mô hình ở Phương trình (5.11), mô hình ở Phương trình (5.25) nhận thêm $\mathbf{c}_{i-1}$ làm đầu vào. Do đó, mô hình này xét cả biểu diễn của các từ phía đích $y_1 \dots y_{i-1}$ (được mã hóa trong $\mathbf{s}_{i-1}$ và $\mathbf{y}_i^e$) lẫn biểu diễn của toàn bộ chuỗi phía nguồn $x_1 \dots x_m$ (được mã hóa trong $\mathbf{c}_{i-1}$). Khi đó, phân phối của các từ đích tại vị trí i có thể được lấy điều kiện theo $\mathbf{s}_i$ như thường lệ:

$$\Pr(\cdot|y_1, \dots, y_i, x_1, \dots, x_m) = \Pr(\cdot|\mathbf{s}_i), \tag{5.26}$$

trong đó $\Pr(\cdot|\mathbf{s}_i)$ thường là một tầng Softmax. Quá trình này được minh họa ở Hình 5.4.



Hình 5.4: Một mô hình chú ý cho NMT. Giả sử đến thời điểm này ta đã có các biểu diễn $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ và trạng thái bộ giải mã $\mathbf{s}_{i-1}$. Ta muốn tính trạng thái bộ giải mã ở bước tiếp theo. Để làm vậy, trước hết ta tính các trọng số chú ý bằng cách chuẩn hóa các điểm chú ý giữa $\mathbf{s}_{i-1}$ và $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$, rồi tính vectơ ngữ cảnh $\mathbf{c}_{i-1}$ bằng cách lấy tổng trên $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ với các trọng số chú ý đó. Một trạng thái bộ giải mã mới $\mathbf{s}_i$ được tạo ra từ vectơ ngữ cảnh $\mathbf{c}_{i-1}$, trạng thái trước $\mathbf{s}_{i-1}$ và biểu diễn từ $\mathbf{y}_i^e$. $\mathbf{s}_i$ sẽ được dùng làm điều kiện để dự đoán phân phối các từ ở bước $i+1$. (Nhãn trong hình: *Decoder State at step* i = trạng thái bộ giải mã ở bước i; *weight* $\alpha_{i-1,j}$ *of connecting* $\mathbf{s}_{i-1}$ *and* $\mathbf{h}_j$ = trọng số $\alpha_{i-1,j}$ nối $\mathbf{s}_{i-1}$ với $\mathbf{h}_j$; *compute* = tính.)

Như vậy, ta đã có một mô hình để tính $\Pr(y_{i+1}|y_1 \dots y_i, x_1 \dots x_m)$. Các bước chính của mô hình này được tóm tắt như sau

1. Mã hóa chuỗi phía nguồn thành $\mathbf{h}_1 \dots \mathbf{h}_m$, trong đó $\mathbf{h}_j = \text{RNN}(\mathbf{h}_{j-1}, \mathbf{x}_j^e)$.
2. Lặp lại quy trình sau từ $i = 1$ đến $n - 1$.
   a. Tính điểm gióng hàng $a(\mathbf{s}_{i-1}, \mathbf{h}_j)$ cho mỗi j.
   b. Tính các trọng số chú ý $\{\alpha_{i-1,1}, \dots, \alpha_{i-1,m}\}$, trong đó $\alpha_{i-1,j} = \frac{\exp(a(\mathbf{s}_{i-1}, \mathbf{h}_j))}{\sum_{j'=1}^{m} \exp(a(\mathbf{s}_{i-1}, \mathbf{h}_{j'}))}$.
   c. Tính vectơ ngữ cảnh $\mathbf{c}_{i-1} = \sum_{j=1}^{m} \alpha_{i-1,j} \cdot \mathbf{h}_j$.
   d. Tính trạng thái phía đích $\mathbf{s}_i = \text{RNN}(\mathbf{s}_{i-1}, \mathbf{c}_{i-1}, \mathbf{y}_i^e)$.
   e. Tính phân phối của các từ phía đích $\Pr(\cdot|\mathbf{s}_i)$.
   f. Tính $\Pr(y_{i+1}|y_1 \dots y_i, x_1 \dots x_m) = \Pr(y_{i+1}|\mathbf{s}_i)$ cho một từ y_{i+1} cho trước (như khi huấn luyện), hoặc chọn từ có khả năng nhất $\hat{y}_{i+1} = \arg\max_{y_{i+1}} \Pr(y_{i+1}|y_1 \dots y_i, x_1 \dots x_m)$ (như khi kiểm tra).

Trong các hệ thống thực tế, mô hình cơ bản này có thể được sửa đổi để dự đoán các từ phía đích tốt hơn. Chẳng hạn, ta có thể thêm các tầng hợp nhất (*fusion layer*) để kết hợp $\mathbf{s}_i$, $\mathbf{c}_{i-1}$ và $\mathbf{y}_i^e$ trước tầng Softmax, nhờ đó có một mô hình sâu hơn để dự đoán [Bahdanau và cộng sự, 2014]. Một cách tiếp cận thường dùng khác là xếp chồng nhiều tầng RNN ở phía đích. Khi đó, ta có thể thực hiện chú ý ở mỗi tầng của chồng [Wu và cộng sự, 2016] hoặc chỉ ở tầng trên cùng [Luong và cộng sự, 2015]. Xem Mục 5.3.4 để biết thêm về các cách tiếp cận chú ý nhiều tầng.

5.3.2 Chú ý QKV

Vì cơ chế chú ý là một cách tiếp cận mạnh mẽ đến vậy, nhiều biến thể của nó đã được phát triển. Có lẽ cách tiếp cận được dùng rộng rãi nhất là phát biểu lại bài toán chú ý thành bài toán khớp một truy vấn (*query*) với một tập các cặp khóa (*key*)–giá trị (*value*). Nó đặt nền móng cho mô hình chuỗi nổi tiếng — Transformer [Vaswani và cộng sự, 2017].

Ở đây ta giả sử có một số cặp khóa–giá trị $\{(\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_m, \mathbf{v}_m)\}$ và một truy vấn $\mathbf{q}$. Mục tiêu của mô hình chú ý truy vấn–khóa–giá trị (*query-key-value attention*) (hay *chú ý QKV*) là tổng hợp các giá trị bằng cách xét sự tương ứng giữa truy vấn và các khóa. Đây là một bài toán tìm kiếm tiêu chuẩn trong các hệ cơ sở dữ liệu, trong đó thông tin được trả về ở dạng gốc hoặc một dạng mới khi nó khớp với truy vấn. Trong chú ý QKV, kết quả tìm kiếm không phải là một giá trị đơn lẻ trong $\{\mathbf{v}_1, \dots, \mathbf{v}_m\}$ mà là một tổ hợp có trọng số của các giá trị này. Đây là khác biệt then chốt giữa mô hình chú ý này và các mô hình tìm kiếm thông thường.

Một cách hình thức, kết quả của chú ý QKV được định nghĩa là

$$\mathbf{c} = \sum_{j=1}^{m} \alpha_j \mathbf{v}_j, \tag{5.27}$$

trong đó

$$\alpha_j = \frac{\exp(\mathbf{q}\mathbf{k}_j^\top / \beta)}{\sum_{j'=1}^{m} \exp(\mathbf{q}\mathbf{k}_{j'}^\top / \beta)} \tag{5.28}$$

là trọng số chú ý. Hóa ra mô hình trên có đúng cùng dạng tổng quát với mô hình mô tả ở tiểu mục trước, và $\mathbf{c}$ đơn giản có thể được xem là một vectơ ngữ cảnh.

Chú giải của người dịch: Truy vấn, khóa, giá trị — ví dụ từ thư viện

Hãy tưởng tượng bạn đến thư viện với một *câu hỏi* (truy vấn $\mathbf{q}$). Mỗi cuốn sách có một *nhãn gáy* mô tả nội dung (khóa $\mathbf{k}_j$) và *nội dung bên trong* (giá trị $\mathbf{v}_j$). Một hệ cơ sở dữ liệu thông thường chỉ trả về đúng cuốn sách có nhãn khớp nhất. Chú ý QKV thì "mềm" hơn: nó so câu hỏi với mọi nhãn (bằng tích vô hướng $\mathbf{q}\mathbf{k}_j^\top$), đổi điểm thành tỉ lệ α_j bằng Softmax, rồi trộn nội dung các cuốn sách theo tỉ lệ đó. Ví dụ nếu $\alpha = (0.7, 0.2, 0.1)$ thì $\mathbf{c} = 0.7\mathbf{v}_1 + 0.2\mathbf{v}_2 + 0.1\mathbf{v}_3$.

Mặc dù dạng cơ bản của chú ý QKV không có gì mới, nó có thể xử lý nhiều bài toán khác nhau bằng cách gán cho $\mathbf{q}$, $\mathbf{k}_j$ và $\mathbf{v}_j$ những ý nghĩa thích hợp. Ở đây ta xét một trường hợp tổng quát hơn, trong đó có n truy vấn $\{\mathbf{q}_1, \ldots, \mathbf{q}_n\}$ và n vectơ đầu ra $\{\mathbf{c}_1, \ldots, \mathbf{c}_n\}$. Để đơn giản hóa ký hiệu, ta dùng $\mathbf{Q}$ để ký hiệu ma trận có hàng thứ i là $\mathbf{q}_i$:

$$\mathbf{Q} = \begin{bmatrix} \mathbf{q}_1 \\ \vdots \\ \mathbf{q}_n \end{bmatrix}. \tag{5.29}$$

Tương tự, ta có thể định nghĩa $\mathbf{K} = \begin{bmatrix} \mathbf{k}_1 \\ \vdots \\ \mathbf{k}_m \end{bmatrix}$, $\mathbf{V} = \begin{bmatrix} \mathbf{v}_1 \\ \vdots \\ \mathbf{v}_m \end{bmatrix}$ và $\mathbf{C} = \begin{bmatrix} \mathbf{c}_1 \\ \vdots \\ \mathbf{c}_n \end{bmatrix}$. Khi đó, mô hình chú ý có thể được phát biểu thành

$$\mathbf{C} = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\beta}\right)\mathbf{V}. \tag{5.30}$$

Hình 5.5 minh họa phương trình này. Lưu ý rằng Softmax$(\frac{\mathbf{Q}\mathbf{K}^\top}{\beta})$ là hàm Softmax theo từng hàng, tạo ra một ma trận các trọng số chú ý

$$\text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\beta}\right) = \begin{bmatrix} \alpha_{1,1} & \ldots & \alpha_{1,m} \\ \vdots & & \vdots \\ \alpha_{n,1} & \ldots & \alpha_{n,m} \end{bmatrix}, \tag{5.31}$$

trong đó một vectơ hàng $\begin{bmatrix} \alpha_{i,1} & \ldots & \alpha_{i,m} \end{bmatrix}$ biểu diễn một phân phối trên chỉ số của các giá trị. Ta có thể khai triển Phương trình (5.30) để dễ hiểu mô hình hơn

$$\begin{aligned} \mathbf{C} &= \begin{bmatrix} \mathbf{c}_1 \\ \vdots \\ \mathbf{c}_n \end{bmatrix} \\ &= \begin{bmatrix} \sum_{j=1}^m \alpha_{1,j}\mathbf{v}_j \\ \vdots \\ \sum_{j=1}^m \alpha_{n,j}\mathbf{v}_j \end{bmatrix} \\ &= \begin{bmatrix} \alpha_{1,1} & \ldots & \alpha_{1,m} \\ \vdots & & \vdots \\ \alpha_{n,1} & \ldots & \alpha_{n,m} \end{bmatrix} \begin{bmatrix} \mathbf{v}_1 \\ \vdots \\ \mathbf{v}_m \end{bmatrix}. \end{aligned} \tag{5.32}$$

Chú giải của người dịch: Phép nhân ma trận trong Phương trình (5.30–5.32)

Tích hai ma trận **AB**: phần tử ở hàng i, cột j của kết quả bằng tích vô hướng của hàng i của **A** với cột j của **B**. Vì vậy phần tử (i, j) của $\mathbf{Q}\mathbf{K}^\top$ đúng bằng $\mathbf{q}_i\mathbf{k}_j^\top$ — điểm của truy vấn i với khóa j. Ví dụ nhỏ với vectơ 2 chiều: $\mathbf{Q} = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}$, $\mathbf{K} = \begin{bmatrix}2 & 0\\ 0 & 1\end{bmatrix}$ thì $\mathbf{Q}\mathbf{K}^\top = \begin{bmatrix}2 & 0\\ 0 & 1\end{bmatrix}$. Sau khi áp Softmax cho *từng hàng* (mỗi hàng có tổng 1), nhân với **V** cho ra hàng i là $\sum_j \alpha_{i,j}\mathbf{v}_j$ — tức là tính n vectơ ngữ cảnh *cùng một lúc* chỉ bằng vài phép nhân ma trận, rất hợp với GPU.

Trong mô hình hóa chuỗi-sang-chuỗi, **Q**, **K** và **V** có thể được định nghĩa theo nhiều cách khác nhau. Để mô tả sự tương ứng giữa chuỗi phía nguồn và chuỗi phía đích, một cách tiếp cận, gọi là **chú ý bộ mã hóa–bộ giải mã** (*encoder-decoder attention*), đơn giản là giả sử

$$\mathbf{Q} = \begin{bmatrix}\mathbf{s}_1\\ \vdots\\ \mathbf{s}_n\end{bmatrix}, \tag{5.33}$$

và

$$\mathbf{K} = \mathbf{V} = \begin{bmatrix}\mathbf{h}_1\\ \vdots\\ \mathbf{h}_m\end{bmatrix}. \tag{5.34}$$

Trong trường hợp này, **C** là một chuỗi các biểu diễn mới của chuỗi phía nguồn khi biết các biểu diễn của chuỗi phía đích. Giống như mô hình mô tả ở tiểu mục trước, mỗi $\mathbf{c}_i \in \mathbf{C}$ có thể được dùng để dự đoán từ y_{i+1}.

Ngoài việc áp dụng mô hình này cho các bài toán chuỗi-sang-chuỗi, một loại cách tiếp cận khác là coi nó như một mô hình chuỗi, tức là ta dùng chú ý QKV để biểu diễn một chuỗi trong một ngôn ngữ. Trong trường hợp này, chú ý QKV còn được gọi là **tự chú ý** (*self-attention*), vốn là nền tảng của mô hình Transformer nổi tiếng [Vaswani và cộng sự, 2017]. Chẳng hạn, xét chuỗi trạng thái $\mathbf{h}_1 \ldots \mathbf{h}_m$. Mô hình tự chú ý giả sử rằng

$$\mathbf{Q} = \mathbf{K} = \mathbf{V} = \begin{bmatrix}\mathbf{h}_1\\ \vdots\\ \mathbf{h}_m\end{bmatrix}. \tag{5.35}$$

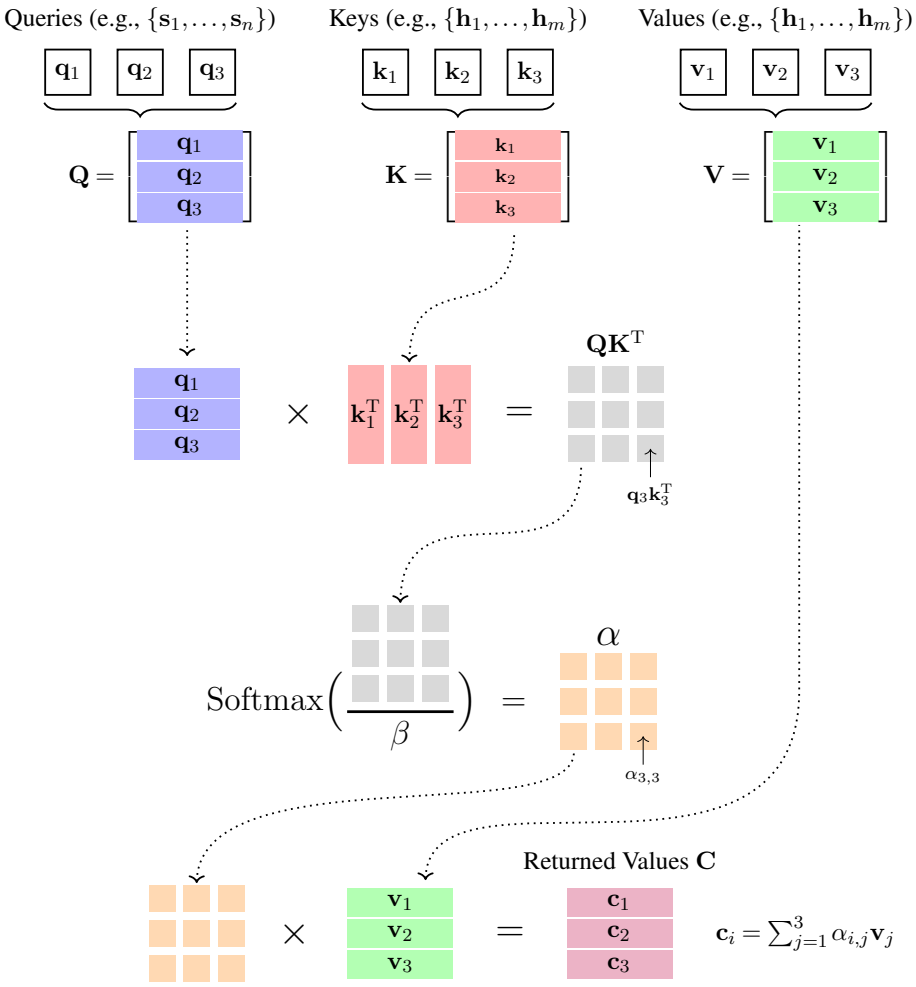
Khi đó, đầu ra của mô hình là một chuỗi biểu diễn $\mathbf{c}_1 \ldots \mathbf{c}_m$. $\mathbf{c}_j$ là một biểu diễn nắm bắt mối tương quan giữa $\mathbf{h}_j$ và mọi phần tử của chuỗi đầu vào. Chúng ta sẽ thấy thảo luận chi tiết hơn về mô hình này ở Chương 6.

Chú giải của người dịch: Chú ý bộ mã hóa–bộ giải mã và tự chú ý khác nhau ở đâu?

Cùng một công thức $\text{Softmax}(\mathbf{Q}\mathbf{K}^\top/\beta)\mathbf{V}$, chỉ khác ở chỗ "ai hỏi ai". Trong *chú ý bộ mã hóa–bộ giải mã*, các từ của câu *đích* đặt câu hỏi (**Q** là các $\mathbf{s}_i$) và tìm câu trả lời trong câu *nguồn* ($\mathbf{K} = \mathbf{V}$ là các $\mathbf{h}_j$). Trong *tự chú ý*, các từ của *cùng một câu* hỏi lẫn nhau. Ví dụ trong câu "Con mèo đuổi con chuột vì *nó* đói", tự chú ý có thể giúp biểu diễn của "nó" lấy nhiều thông tin từ "con mèo".

5.3.3 Chú ý đa đầu

Chú ý đa đầu (*multi-head attention*) là một mở rộng tự nhiên của các mô hình trên. Ý tưởng then chốt là thực hiện chú ý đồng thời trong các không gian con khác nhau của không gian biểu diễn, thay vì trong



Hình 5.5: Mô hình chú ý QKV cho các lô truy vấn (**Q**), khóa (**K**) và giá trị (**V**). Hình cho thấy một cách hiện thực trực tiếp công thức $\mathbf{C} = \text{Softmax}(\frac{\mathbf{QK}^\top}{\beta})\mathbf{V}$. $\text{Softmax}(\frac{\mathbf{QK}^\top}{\beta})$ tính các trọng số chú ý bằng cách chuẩn hóa tích vô hướng có co giãn của **Q** và $\mathbf{K}^\top$. Kết quả là một ma trận α, trong đó mỗi vectơ hàng biểu diễn các trọng số trên các giá trị khác nhau. Nhân α với **V**, ta thu được một chuỗi giá trị mới, mỗi giá trị là một tổng có trọng số của các giá trị ban đầu. (Nhãn trong hình: *Queries/Keys/Values* = truy vấn/khóa/giá trị; *Returned Values* = các giá trị trả về.)

một không gian biểu diễn duy nhất. Để minh họa, xét một mô hình chú ý tiêu chuẩn nhận các chuỗi trạng thái phía nguồn và phía đích rồi trả ra một chuỗi trạng thái mới, viết là

$$\mathbf{c}_1 \dots \mathbf{c}_n = \text{Att}(\mathbf{h}_1 \dots \mathbf{h}_m, \mathbf{s}_1 \dots \mathbf{s}_n), \tag{5.36}$$

trong đó $\mathbf{h}_j, \mathbf{s}_i, \mathbf{c}_i \in \mathbb{R}^{d_h}$, và $\text{Att}(\cdot)$ là hàm chú ý. Ta có thể ánh xạ $\mathbf{h}_j$ thành τ vectơ $\{\mathbf{h}_j^{[1]}, \dots, \mathbf{h}_j^{[\tau]}\}$ bằng các phép biến đổi tuyến tính sau

$$\mathbf{h}_j^{[1]} = \mathbf{h}_j \mathbf{W}_h^{[1]}, \tag{5.37}$$

$$\vdots$$

$$\mathbf{h}_j^{[\tau]} = \mathbf{h}_j \mathbf{W}_h^{[\tau]}, \tag{5.38}$$

trong đó $\mathbf{h}_j^{[1]}, \dots, \mathbf{h}_j^{[\tau]} \in \mathbb{R}^{\frac{d_h}{\tau}}$, và $\mathbf{W}_h^{[1]}, \dots, \mathbf{W}_h^{[\tau]} \in \mathbb{R}^{d_h \times \frac{d_h}{\tau}}$.

Tương tự, ta có thể ánh xạ $\mathbf{s}_i$ thành τ vectơ $\{\mathbf{s}_i^{[1]}, \dots, \mathbf{s}_i^{[\tau]}\}$. Sau đó ta định nghĩa τ **không gian con đặc trưng** (*feature subspace*), trong đó hàm chú ý được thực hiện độc lập. Với không gian con đặc trưng thứ k, ta có

$$\mathbf{c}_1^{[k]} \dots \mathbf{c}_n^{[k]} = \text{Att}(\mathbf{h}_1^{[k]} \dots \mathbf{h}_m^{[k]}, \mathbf{s}_1^{[k]} \dots \mathbf{s}_n^{[k]}). \tag{5.39}$$

Đầu ra của mô hình là một chuỗi các vectơ d_h chiều, mỗi vectơ có được bằng cách ghép nối các vectơ sinh ra trong tất cả các không gian con đặc trưng này, rồi thực hiện một phép biến đổi tuyến tính. Quy trình này được cho bởi

$$\mathbf{c}_1 = [\mathbf{c}_1^{[1]}, \dots, \mathbf{c}_1^{[\tau]}]\mathbf{W}_c, \tag{5.40}$$

$$\dots$$

$$\mathbf{c}_n = [\mathbf{c}_n^{[1]}, \dots, \mathbf{c}_n^{[\tau]}]\mathbf{W}_c, \tag{5.41}$$

trong đó $\mathbf{W}_c \in \mathbb{R}^{d_h \times d_h}$.

Chú giải của người dịch: Ví dụ kích thước trong chú ý đa đầu

Giả sử $d_h = 6$ và $\tau = 3$ đầu. Mỗi $\mathbf{h}_j$ (6 số) được chiếu thành 3 vectơ nhỏ, mỗi vectơ $6/3 = 2$ số (nhân với ma trận $\mathbf{W}_h^{[k]}$ cỡ 6×2). Mỗi "đầu" thực hiện chú ý riêng trên các vectơ 2 chiều của mình và cho ra $\mathbf{c}_i^{[k]}$ (2 số). Ghép 3 kết quả lại được vectơ $2 + 2 + 2 = 6$ số, rồi nhân với $\mathbf{W}_c$ cỡ 6×6 để ra $\mathbf{c}_i$. Giống như một bài văn được 3 giám khảo chấm, mỗi người chú trọng một tiêu chí (ngữ pháp, nội dung, bố cục), rồi tổng hợp lại.

Theo ký hiệu dùng ở tiểu mục trước, ta có thể biểu diễn một chuỗi vectơ thành một ma trận, chẳng hạn $\mathbf{H} = \begin{bmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_m \end{bmatrix} \in \mathbb{R}^{m \times d_h}$, $\mathbf{S} = \begin{bmatrix} \mathbf{s}_1 \\ \vdots \\ \mathbf{s}_n \end{bmatrix} \in \mathbb{R}^{n \times d_h}$ và $\mathbf{C} = \begin{bmatrix} \mathbf{c}_1 \\ \vdots \\ \mathbf{c}_n \end{bmatrix} \in \mathbb{R}^{n \times d_h}$. Với ký hiệu này, ta viết lại Phương trình (5.36) thành

$$\mathbf{C} = \text{Att}(\mathbf{H}, \mathbf{S}). \tag{5.42}$$

Để đưa ra định nghĩa hình thức của chú ý đa đầu, trước hết ta giới thiệu các hàm tách (*split*) và gộp (*merge*). Hàm tách chia mỗi vectơ hàng của một ma trận thành một số vectơ con, tạo ra một tensor 3 chiều. Ví dụ, tách một ma trận $\mathbf{A}$ cỡ $m \times d_h$ với τ sẽ tạo ra một tensor cỡ $\tau \times m \times \frac{d_h}{\tau}$[3]

$$\mathbf{A}_{\text{heads}} = \text{Split}(\mathbf{A}, \tau). \tag{5.43}$$

[3]Một tensor cỡ $a \times b \times c$ có thể được xem là một mảng gồm a ma trận, mỗi ma trận có kích thước $b \times c$.

Hàm gộp thực hiện phép toán ngược của hàm tách. Cho một tensor cỡ $\tau \times n \times \frac{d_h}{\tau}$ (chẳng hạn $\mathbf{A}_{\text{heads}}$), nó gộp mỗi nhóm gồm τ mảng con $\frac{d_h}{\tau}$ chiều theo dạng

$$\mathbf{A}_{\text{merge}} = \text{Merge}(\mathbf{A}_{\text{heads}}, \tau). \tag{5.44}$$

Như vậy, dạng của chú ý đa đầu được cho bởi

$$\begin{aligned}
\mathbf{C} &= \mathbf{C}_{\text{merge}}\mathbf{W}_c \\
&= \text{Merge}(\mathbf{C}_{\text{heads}}, \tau)\mathbf{W}_c \\
&= \text{Merge}(\text{Att}(\mathbf{H}_{\text{heads}}, \mathbf{S}_{\text{heads}}), \tau)\mathbf{W}_c && (5.45)\\
\mathbf{H}_{\text{heads}} &= \text{Split}(\mathbf{H}\mathbf{W}_h, \tau) && (5.46)\\
\mathbf{S}_{\text{heads}} &= \text{Split}(\mathbf{S}\mathbf{W}_s, \tau), && (5.47)
\end{aligned}$$

trong đó $\mathbf{W}_h, \mathbf{W}_s \in \mathbb{R}^{d_h \times d_h}$ là các tham số. $\text{Split}(\mathbf{H}\mathbf{W}_h, \tau)$ hiện thực các phép chiếu ở Phương trình (5.37–5.38) cho mọi $\mathbf{h}_j$. Tương tự, $\text{Split}(\mathbf{S}\mathbf{W}_s, \tau)$ cũng có cách hiểu như vậy. Lưu ý rằng ở đây $\text{Att}(\cdot)$ được mở rộng để xử lý đầu vào đa đầu. Xem minh họa mô hình này ở Hình 5.6.

Chú ý đa đầu là một cách tiếp cận rất tổng quát, có thể mở rộng cho nhiều mô hình. Lấy một ví dụ đơn giản về sự mở rộng này, xét mô hình chú ý QKV đã bàn ở tiểu mục trước. Gọi $\text{Att}_{\text{QKV}}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ là hàm chú ý, và $\mathbf{Q} \in \mathbb{R}^{d_k}, \mathbf{K} \in \mathbb{R}^{d_k}, \mathbf{V} \in \mathbb{R}^{d_v}$ là các đầu vào. Mô hình chú ý QKV đa đầu được cho bởi

$$\begin{aligned}
\mathbf{C} &= \text{Merge}(\text{Att}_{\text{QKV}}(\mathbf{Q}_{\text{heads}}, \mathbf{K}_{\text{heads}}, \mathbf{V}_{\text{heads}}))\mathbf{W}_c, && (5.48)\\
\mathbf{Q}_{\text{heads}} &= \text{Split}(\mathbf{Q}\mathbf{W}_q, \tau), && (5.49)\\
\mathbf{K}_{\text{heads}} &= \text{Split}(\mathbf{K}\mathbf{W}_k, \tau), && (5.50)\\
\mathbf{V}_{\text{heads}} &= \text{Split}(\mathbf{V}\mathbf{W}_v, \tau), && (5.51)
\end{aligned}$$

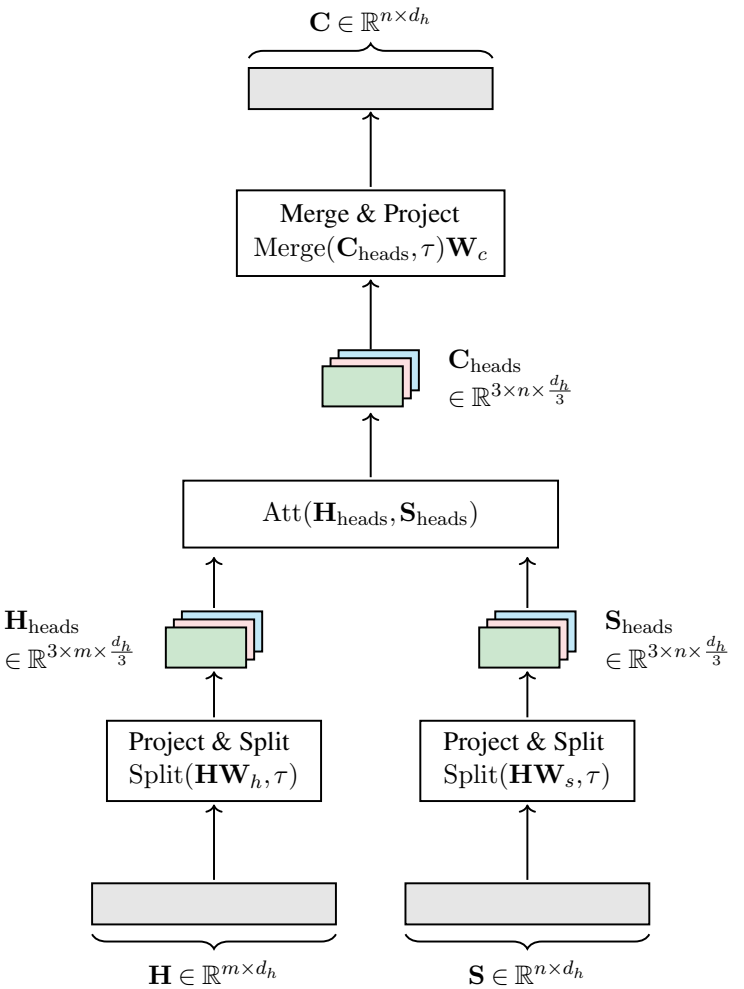
trong đó $\mathbf{W}_q \in \mathbb{R}^{d_k \times d_k}, \mathbf{W}_k \in \mathbb{R}^{d_k \times d_k}, \mathbf{W}_v \in \mathbb{R}^{d_v \times d_v}, \mathbf{W}_c \in \mathbb{R}^{d_v \times d_v}$ là các tham số mô hình.

Một ưu điểm của chú ý đa đầu là mỗi không gian con đặc trưng mô tả một góc nhìn khác nhau của sự chú ý (gọi là một **đầu chú ý** (*attention head*), hay gọi tắt là **đầu** — *head*). Do đó, việc ghép nối đầu ra của các đầu này biểu diễn một *tập hợp* (*ensemble*) các mô hình chú ý xử lý các phần khác nhau của dữ liệu. Điều này tương tự như học độc lập một nhóm mô hình rồi kết hợp chúng để tạo thành một mô hình mạnh hơn. Những cách tiếp cận như vậy đã được chứng minh là hữu ích trong nhiều bài toán [Opitz và Maclin, 1999; Zhou, 2012b]. Lưu ý rằng các mô hình chú ý đa đầu bàn ở đây được tham số hóa bởi các phép chiếu tuyến tính trên không gian đầu vào và đầu ra. Việc dùng các phép chiếu tuyến tính này nói chung là có ích khi mô hình trở nên sâu hơn và có thể mô tả những bài toán phức tạp hơn.

Từ góc độ thiết kế kiến trúc, chú ý đa đầu thuộc về một lớp rộng các mạng nơ-ron — những mạng gồm nhiều nhánh chồng tầng cùng xử lý một đầu vào (gọi là **mạng nơ-ron đa nhánh** (*multi-branch neural network*)). Tuy nhiên, khác với các cách tiếp cận thông thường đòi hỏi các kiến trúc mô hình khác nhau cho các nhánh khác nhau, cách tiếp cận chú ý đa đầu dựa trên một mô hình duy nhất cho tất cả các đầu. Nhờ vậy, các hệ thống này rất hiệu quả trong thực tế vì quá trình chú ý có thể chạy song song trên các đầu.

5.3.4 Chú ý nhiều tầng

Cho đến giờ, ta đã xét trường hợp **chú ý một tầng** (*single-layer attention*) — đầu ra của các mô hình chú ý có thể được viết thành một tổ hợp tuyến tính của các biểu diễn phía nguồn. Bây giờ ta mở rộng



Hình 5.6: Một mô hình chú ý với $\tau = 3$ đầu. Trước hết, ta biến đổi các ma trận đầu vào thành biểu diễn đa đầu, tức là các tensor 3 chiều $\mathbf{H}_{\text{heads}} \in \mathbb{R}^{3 \times m \times \frac{d_h}{3}}$ và $\mathbf{S}_{\text{heads}} \in \mathbb{R}^{3 \times n \times \frac{d_h}{3}}$. Các tensor này sau đó được đưa vào một mô hình chú ý. Đầu ra của mô hình này là một tensor $\mathbf{C}_{\text{heads}} \in \mathbb{R}^{3 \times n \times \frac{d_h}{3}}$. Sau đó ta gộp các đầu của $\mathbf{C}_{\text{heads}}$, rồi thực hiện một phép biến đổi tuyến tính. Cuối cùng, ta thu được n vectơ kích thước d_h, biểu diễn bằng một ma trận $n \times d_h$. (Nhãn trong hình: *Project & Split* = chiếu và tách; *Merge & Project* = gộp và chiếu.)

nó một cách tự nhiên thành **chú ý nhiều tầng** (*multi-layer attention*), trong đó quy trình chú ý một tầng được chạy nhiều lần để tạo thành một mô hình chú ý "sâu hơn".

Để làm điều này, một mạng nơ-ron nhiều tầng được tạo ra ở phía đích. Kiến trúc mô hình khá đều đặn. Ta xếp chồng một số tầng chú ý, mỗi tầng tương tác với chuỗi phía nguồn và đưa đầu ra của nó cho tầng tiếp theo. Trong một tầng chú ý, ta thực hiện chú ý như thường lệ. Với tầng thứ l trong chồng, bước này nhận chuỗi phía nguồn (ký hiệu là $\mathbf{h}_1 \dots \mathbf{h}_m$) cùng đầu ra của tầng trước (ký hiệu là $\mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}$), và sinh ra một chuỗi vectơ bằng

$$\mathbf{c}_1^l \dots \mathbf{c}_n^l = \text{Att}(\mathbf{h}_1 \dots \mathbf{h}_m, \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}), \tag{5.52}$$

trong đó $\text{Att}(\cdot)$ có thể là bất kỳ hàm chú ý nào được mô tả trong chương này.

Sau đó, ta tạo thêm một mạng nơ-ron khác $f(\cdot)$ để tăng khả năng mô hình hóa cho mô hình. Do đó, đầu ra của tầng chú ý được định nghĩa là

$$\mathbf{s}_1^l \dots \mathbf{s}_n^l = f(\mathbf{c}_1^l \dots \mathbf{c}_n^l, \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}). \tag{5.53}$$

$f(\cdot)$ có thể được thiết kế theo nhiều cách [Sukhbaatar và cộng sự, 2015; Wu và cộng sự, 2016; Vaswani và cộng sự, 2017]. Một lựa chọn phổ biến là định nghĩa $f(\cdot)$ là một mạng truyền thẳng (*feed-forward network*) với một kết nối tắt (*residual connection*), cho bởi

$$f(\mathbf{c}_1^l \dots \mathbf{c}_n^l, \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}) = \text{FFN}(\mathbf{c}_1^l \dots \mathbf{c}_n^l) + \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}. \tag{5.54}$$

Thay các vectơ $\mathbf{c}_1^l \dots \mathbf{c}_n^l$ bằng Phương trình (5.52), đầu ra của tầng l được viết dưới dạng

$$\mathbf{s}_1^l \dots \mathbf{s}_n^l = \text{FFN}(\text{Att}(\mathbf{h}_1 \dots \mathbf{h}_m, \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1})) + \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}. \tag{5.55}$$

Giống như các mô hình ở các tiểu mục trước, sẽ thuận tiện hơn nếu dùng ký hiệu gọn bằng cách biểu diễn một chuỗi vectơ thành một ma trận. Như vậy, mô hình này có thể được viết dưới một dạng khác

$$\mathbf{S}^l = \text{FFN}(\text{Att}(\mathbf{H}, \mathbf{S}^{l-1})) + \mathbf{S}^{l-1}. \tag{5.56}$$

Ở đây, $\text{FFN}(\cdot)$ thường là một mạng nơ-ron nhiều tầng với các hàm kích hoạt (*activation function*) phi tuyến. Ánh xạ đồng nhất (*identity mapping*) (tức $+\mathbf{S}^{l-1}$) tạo ra một đường đi trực tiếp từ đầu vào tới đầu ra của tầng, giúp việc huấn luyện một mạng nơ-ron sâu trở nên dễ dàng hơn.

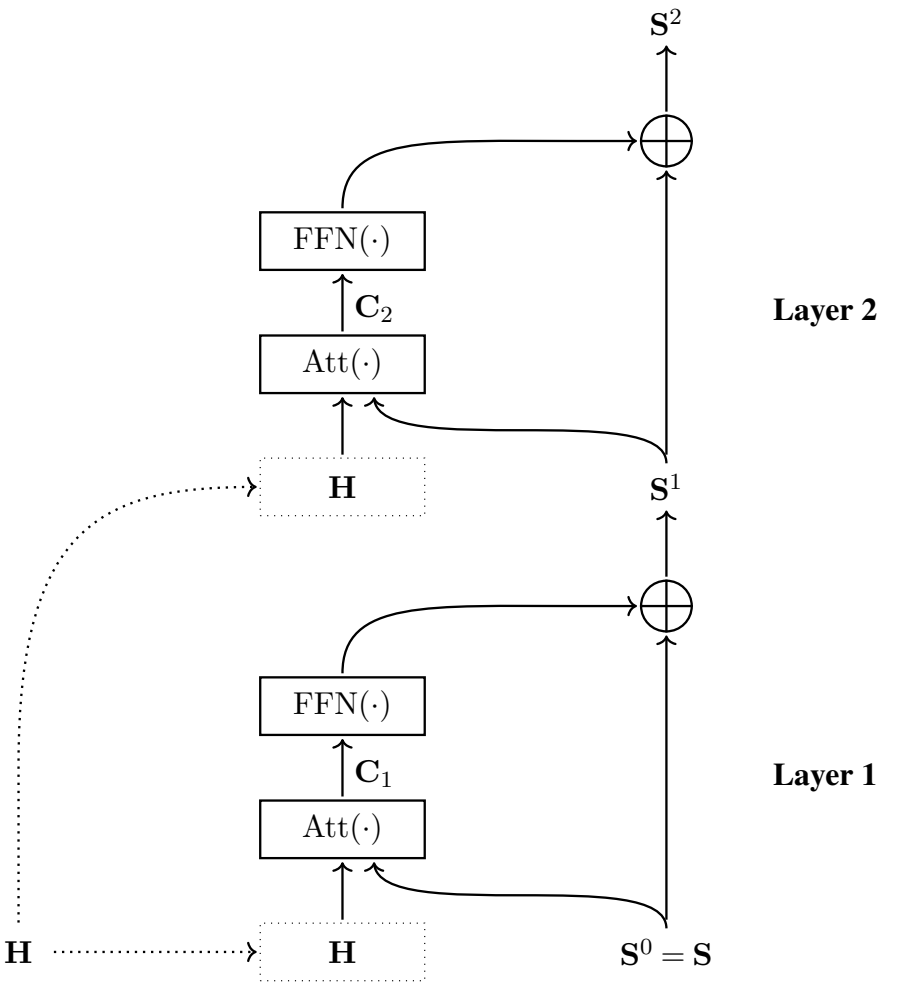
Chú giải của người dịch: Vì sao kết nối tắt giúp huấn luyện mạng sâu?

Trong Phương trình (5.56), đầu ra = đầu vào + một "phần chỉnh sửa" $\text{FFN}(\text{Att}(\cdot))$. Mỗi tầng chỉ cần học *phần thay đổi* so với đầu vào, chứ không phải học lại toàn bộ. Nếu một tầng chưa học được gì hữu ích, phần chỉnh sửa gần bằng 0 và thông tin vẫn đi thẳng qua nhờ $+\mathbf{S}^{l-1}$. Khi lan truyền ngược, gradient cũng có "đường cao tốc" này để đi từ tầng trên cùng xuống các tầng dưới mà không bị suy giảm quá nhiều qua hàng chục tầng.

Hình 5.7 cho thấy kiến trúc của mô hình này. Mô hình chú ý bắt đầu với biểu diễn khởi đầu của chuỗi phía đích, tức là $\mathbf{S}^0 = \mathbf{S} = \begin{bmatrix} \mathbf{s}_1 \\ \vdots \\ \mathbf{s}_n \end{bmatrix}$. Nếu có L tầng chú ý thì đầu ra cuối cùng sẽ là $\mathbf{S}^L$.

5.3.5 Nhận xét

Ở trên, chúng ta đã xét một mô hình chú ý cơ bản và một loạt cải tiến. Các tài liệu về chú ý và các chủ đề liên quan chứa rất nhiều phương pháp mô hình hóa cách một hệ thống tập trung vào các phần khác



Hình 5.7: Một mô hình chú ý 2 tầng. Các tầng này nhận cùng các cặp "khóa–giá trị" (tức $\mathbf{H}$) nhưng mỗi tầng nhận một "truy vấn" khác nhau (tức $\mathbf{S}^l$). Mô hình chú ý là tiêu chuẩn: các vectơ ngữ cảnh $\mathbf{C}^l$ được sinh ra bằng cách nhận cả $\mathbf{H}$ và $\mathbf{S}^l$. Một mạng nơ-ron truyền thẳng được xây dựng để biến đổi $\mathbf{C}^l$, sau đó cộng thêm $\mathbf{S}^l$. Như vậy, mô hình này có thể được phát biểu thành $\mathbf{S}^l = \text{FFN}(\text{Att}(\mathbf{H}, \mathbf{S}^{l-1})) + \mathbf{S}^{l-1}$. $\mathbf{S}^l$ sau đó được dùng làm truy vấn ở tầng tiếp theo, tức là tầng $l+1$ nhận $\mathbf{H}$ và $\mathbf{S}^l$ rồi lặp lại quá trình trên. Đầu ra của tầng cuối cùng có thể được xem là một biểu diễn sâu hơn của $\mathbf{H}$ dành cho $\mathbf{S}$. (*Layer* = tầng.)

nhau của đầu vào, cũng như nhiều ứng dụng của các mô hình như vậy. Tiểu mục này bàn về một số vấn đề thú vị.

1. Gióng hàng từ và chú ý

Chú ý có liên quan tới một dòng nghiên cứu lâu đời về gióng hàng (*alignment*) (từ), vốn mô hình hóa sự tương ứng giữa hai nhóm đơn vị ngôn ngữ. Trong NLP, gióng hàng là một khái niệm rất tổng quát, được dùng để chỉ nhiều bài toán khác nhau. Chẳng hạn, hầu hết các hệ thống dịch máy thống kê (*statistical machine translation*) được huấn luyện trên các văn bản song ngữ có gióng hàng từ-với-từ [Koehn và cộng sự, 2003; Chiang, 2005]. Do đó, các mô hình **gióng hàng từ** (*word alignment*) được phát triển để tạo ra các liên kết giữa các từ trong hai câu [Vogel và cộng sự, 1996; Och và Ney, 2003; Taskar và cộng sự, 2005; Dyer và cộng sự, 2013]. Mặc dù đầu ra của các hệ thống này là các biến rời rạc, các mô hình bên dưới thường mang tính xác suất và có thể dùng các biểu diễn liên tục. Vì vậy, sự tương ứng giữa gióng hàng từ và các mô hình chú ý bàn ở đây là rõ ràng, vì cả hai đều gán trọng số cho các cặp từ.

Lưu ý rằng dù gióng hàng từ và chú ý có điểm tương đồng, mục tiêu của chúng khác nhau. Trong hầu hết trường hợp, các mô hình gióng hàng từ được dùng như những hệ thống riêng lẻ để tạo ra kết quả gióng hàng cho các hệ thống phía sau, trong khi các mô hình chú ý thường được coi là thành phần của những hệ thống lớn hơn và không hoạt động độc lập (xem Hình 5.8 để so sánh các mô hình này). Kết quả là, trong thực tế chúng phù hợp với những loại hệ thống chuỗi-sang-chuỗi khác nhau: gióng hàng từ là một bước trong các hệ thống nối tầng (*cascaded system*), còn chú ý biểu diễn các trạng thái trung gian của một mạng nơ-ron.

Tuy vậy, gióng hàng từ và chú ý là hai bài toán liên quan, và có thể hỗ trợ nhau trong một số trường hợp. Ví dụ, một cách để xem mô hình chú ý hoạt động ra sao là suy ra gióng hàng từ từ nó rồi đo chất lượng của các gióng hàng này [Tu và cộng sự, 2016; Li và cộng sự, 2019; Garg và cộng sự, 2019]. Ngoài ra, các hệ thống có cơ chế chú ý cũng có thể được định hướng bởi các nguồn gióng hàng từ bên ngoài [Mi và cộng sự, 2016b; Liu và cộng sự, 2016b].

Chú giải của người dịch: Đọc bản đồ nhiệt ở Hình 5.8

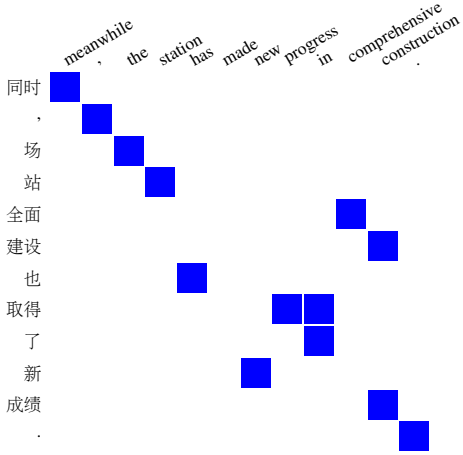
Mỗi hàng là một từ tiếng Trung, mỗi cột là một từ tiếng Anh. Ở hình (a), ô xanh đậm nghĩa là "hai từ này là bản dịch của nhau" (ví dụ 同时 — *meanwhile*, 新 — *new*); mỗi ô chỉ có thể là 0 hoặc 1. Ở hình (b), mỗi ô là một trọng số chú ý $\alpha_{i,j}$ trong khoảng $[0, 1]$: màu càng đậm thì mô hình càng "nhìn" vào cặp từ đó. Ta thấy các ô đậm ở (b) phần lớn trùng với các liên kết ở (a), nhưng chú ý "mờ" hơn và trải ra nhiều ô — đúng như bản chất mềm (phân phối xác suất) của nó.

2. Đưa vào tri thức tiên nghiệm

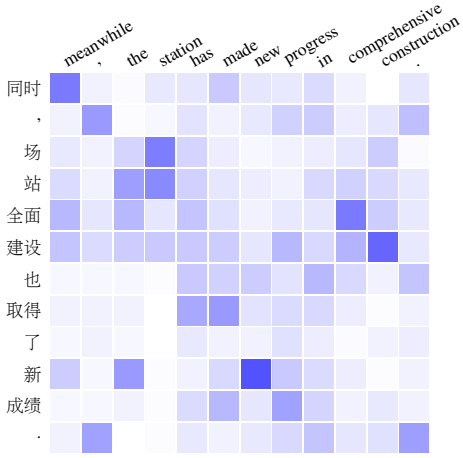
Như đã bàn ở Mục 5.3.1, các mô hình chú ý ngầm định nghĩa một phân phối chú ý (*attention distribution*) trên $\{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$, được dùng để tính tổng có trọng số của các biểu diễn này. Phân phối này được biểu diễn qua các trọng số gióng hàng và được học như một phần của mô hình. Ngoài việc học phân phối chú ý theo kiểu đầu-cuối, ta cũng có thể định nghĩa nó dựa trên tri thức tiên nghiệm về cách sự chú ý được phân bố trong chuỗi.

Một cách tiếp cận là áp đặt trực tiếp một cấu trúc nào đó lên phân phối. Một ví dụ đơn giản là giới hạn sự chú ý trong một đoạn cục bộ của chuỗi và bỏ qua phần còn lại. Gọi $[\rho_i - D, \rho_i + D]$ là một cửa sổ $(2D + 1)$ từ có tâm tại vị trí ρ_i của chuỗi phía nguồn. Ta có thể giới hạn sự chú ý từ $\mathbf{s}_i$ tới $\mathbf{h}_{\rho_i - D} \ldots \mathbf{h}_{\rho_i + D}$, thu được một vectơ ngữ cảnh cục bộ được phát biểu như sau:

$$\mathbf{c}_i = \text{Att}(\mathbf{h}_{\rho_i - D} \ldots \mathbf{h}_{\rho_i + D}, \mathbf{s}_i). \tag{5.57}$$



(a) Heat map of word alignments



(b) Heat map of attention weights

Hình 5.8: Bản đồ nhiệt của một mô hình gióng hàng từ và một mô hình chú ý cho một cặp câu tiếng Trung và tiếng Anh. Các bản đồ nhiệt được biểu diễn dưới dạng lưới tô màu, trong đó mỗi ô mô tả sự tương ứng của một cặp từ tiếng Trung và tiếng Anh. Sự tương ứng này được thể hiện trên thang màu từ trắng (trọng số 0) đến xanh lam đậm (trọng số 1). ((a) Bản đồ nhiệt của gióng hàng từ; (b) bản đồ nhiệt của trọng số chú ý. Câu tiếng Anh: "Meanwhile, the station has made new progress in comprehensive construction." — "Trong khi đó, nhà ga đã đạt được tiến bộ mới trong việc xây dựng toàn diện.")

Cách tiếp cận này còn được gọi là **chú ý cục bộ** (*local attention*). Việc xác định ρ_i tương tự như **bài toán sắp xếp lại trật tự** (*reordering problem*) trong dịch máy. Khi dịch giữa các ngôn ngữ có trật tự từ tương tự nhau, người ta thường giả sử bản dịch là *đơn điệu* và ρ_i tuyến tính theo i [Koehn, 2004], chẳng hạn $\rho_i = \lfloor m\frac{i}{n} \rfloor$ hoặc $\lceil m\frac{i}{n} \rceil$. Một phương pháp khác là dùng một mạng nơ-ron để dự đoán một vị trí tương đối trong chuỗi phía nguồn (ký hiệu là r_i) [Luong và cộng sự, 2015]. Khi đó ρ_i có thể được định nghĩa là $\lfloor mr_i \rfloor$ hoặc $\lceil mr_i \rceil$.

Chú giải của người dịch: Ví dụ về chú ý cục bộ

$\lfloor x \rfloor$ là phần nguyên dưới (làm tròn xuống), $\lceil x \rceil$ là làm tròn lên. Giả sử câu nguồn có $m = 10$ từ, câu đích có $n = 5$ từ. Khi sinh từ đích thứ $i = 2$, ta đoán vị trí tương ứng ở câu nguồn là $\rho_2 = \lfloor 10 \cdot \frac{2}{5} \rfloor = 4$. Với $D = 1$, mô hình chỉ chú ý tới cửa sổ $[3, 5]$, tức $\mathbf{h}_3, \mathbf{h}_4, \mathbf{h}_5$, thay vì cả 10 vectơ. Giả thiết "đơn điệu" hợp lý với các cặp ngôn ngữ có trật tự từ gần nhau (như tiếng Anh–tiếng Pháp).

Trong một hướng nghiên cứu khác, một phân phối mới được xây dựng bằng cách kết hợp phân phối chú ý ban đầu với một phân phối tiên nghiệm (*prior distribution*) nào đó. Phân phối đơn giản nhất loại này có dạng nội suy tuyến tính

$$\widetilde{\Pr}(\mathbf{h}_j|\mathbf{s}_i) = \eta \cdot \Pr(\mathbf{h}_j|\mathbf{s}_i) + (1 - \eta) \cdot \text{Prior}, \tag{5.58}$$

trong đó Prior ký hiệu phân phối tiên nghiệm, và η ký hiệu hệ số nội suy. Khi $\eta = 1$, đây là một mô hình chú ý tiêu chuẩn. Ngược lại, khi $\eta = 0$, sự chú ý hoàn toàn phụ thuộc vào phân phối tiên nghiệm [You và cộng sự, 2020].

Phân phối tiên nghiệm có thể được chọn theo nhiều cách. Một lựa chọn đơn giản là giả sử Prior là một phân phối Gauss (*Gaussian distribution*) $\mathcal{N}(\mu, \sigma^2)$. Điều này cho một cách hiểu trực quan rõ ràng: sự chú ý tập trung tại một điểm tiêu điểm trong chuỗi và giảm dần khi khoảng cách tới điểm này tăng lên. Để xác định giá trị trung bình (*mean*) và phương sai (*variance*) của phân phối Gauss, ta có thể dùng cùng kỹ thuật mô tả ở trên, ví dụ xây dựng hai mạng nơ-ron để lần lượt tính giá trị trung bình và phương sai.

Chú giải của người dịch: Phân phối Gauss và phép nội suy

Phân phối Gauss (phân phối chuẩn) $\mathcal{N}(\mu, \sigma^2)$ có đồ thị hình "quả chuông": cao nhất tại μ (tâm) và thấp dần đối xứng hai bên; σ càng nhỏ thì chuông càng hẹp. Dùng nó làm tiên nghiệm nghĩa là "ta tin rằng các từ nguồn gần vị trí μ quan trọng hơn". Phương trình (5.58) trộn hai ý kiến: ví dụ với $\eta = 0.8$, nếu mô hình chú ý cho $\Pr = 0.5$ còn tiên nghiệm cho 0.2 thì kết quả là $0.8 \cdot 0.5 + 0.2 \cdot 0.2 = 0.44$. Vì là trung bình có trọng số của hai phân phối nên kết quả vẫn có tổng bằng 1.

Phép nội suy cũng có thể được coi là một bước trung gian trong quá trình tính phân phối chú ý. Ví dụ, xét chú ý QKV đã bàn ở Mục 5.3.2. Phép nội suy có thể được áp dụng lên tích vô hướng truy vấn–khóa [Yang và cộng sự, 2018a; Guo và cộng sự, 2019]. Để làm vậy, ta có thể sửa Phương trình (5.28) thành dạng sau

$$\begin{aligned} \alpha_j &= \text{Softmax}\left(\frac{\mathbf{q}\mathbf{k}_j^\top}{\beta} + \eta \,\text{Prior}\right) \\ &= \text{Softmax}\left(\frac{\mathbf{s}_i\mathbf{h}_j^\top}{\beta} + \eta \,\text{Prior}\right). \end{aligned} \tag{5.59}$$

Vì $\frac{\mathbf{q}\mathbf{k}_j^\top}{\beta}$ (hay $\frac{\mathbf{s}_i\mathbf{h}_j^\top}{\beta}$) không bị giới hạn trong $[0, 1]$, Prior được co giãn lại bởi một siêu tham số η.

Đôi khi, tiên nghiệm xuất hiện khi tri thức về sự chú ý đến từ các nguồn bên ngoài. Như đã bàn ở trên, đưa gióng hàng từ vào mô hình chú ý là một trong những cách đơn giản nhất để làm điều này. Ý tưởng này có thể được mở rộng để tận dụng nhiều thông tin cấu trúc hơn, như độ sinh sản (*fertility*) và độ bao phủ (*coverage*) [Cohn và cộng sự, 2016; Feng và cộng sự, 2016; Tu và cộng sự, 2016], hoặc các ràng buộc đặc thù cho nhiệm vụ hơn, như gióng hàng đơn điệu giữa chuỗi đầu vào và chuỗi đầu ra [Raffel và cộng sự, 2017; Chiu và Raffel, 2018]. Ngoài ra, giống như các hệ thống dịch máy dựa trên cú pháp, các cây phân tích cú pháp có thể được dùng làm đầu vào phụ trợ để định hướng quá trình chú ý. Chẳng hạn, cây phụ thuộc (*dependency tree*) là một nguồn thông tin được dùng rộng rãi trong mô hình hóa sự tương ứng giữa các từ, cho cả bài toán chuỗi-sang-chuỗi [Chen và cộng sự, 2018a] lẫn bài toán mô hình hóa chuỗi [Zhang và cộng sự, 2020c; Nguyen và cộng sự, 2020; Xu và cộng sự, 2021b].

Vì các cơ chế chú ý tiêu chuẩn có thể tốn kém về tính toán trong các ứng dụng quy mô lớn, các nhà nghiên cứu đã phát triển những phương án hiệu quả hơn bằng cách đưa các thiên kiến quy nạp (*inductive bias*) mạnh hơn vào thiết kế mô hình [Tay và cộng sự, 2020b]. Hướng nghiên cứu này thuộc về nhóm rộng hơn các phương pháp NLP hiệu quả. Chương 6 bàn chi tiết về các phương pháp này.

3. Chú ý trong mạng bộ nhớ

Không chỉ được quan tâm nhiều trong các hệ thống chuỗi-sang-chuỗi, cơ chế chú ý còn được dùng rộng rãi trong các mô hình nơ-ron dựa trên bộ nhớ [Sukhbaatar và cộng sự, 2015; Graves và cộng sự, 2014; Kumar và cộng sự, 2016]. Như đã bàn ở Chương 4, một hệ thống bộ nhớ duy trì một tập các mục dữ liệu, cho phép mô hình lưu giữ và truy xuất thông tin. Cho một truy vấn, hệ thống tính một điểm khớp giữa truy vấn và khóa của mỗi mục dữ liệu. Quy trình này còn được gọi là **định địa chỉ** (*addressing*) [Graves và cộng sự, 2014]. Việc định địa chỉ như vậy thường được hiện thực bằng cách trước hết biểu diễn truy vấn và các mục dữ liệu thành vectơ số thực, rồi tính một trọng số dựa trên một độ đo tương đồng giữa chúng. Kết quả truy xuất là một tổng có trọng số của các mục dữ liệu. Cách hình thức hóa này khớp hoàn toàn với mô hình chú ý QKV đã bàn ở Mục 5.3.2.

Với mối liên hệ về khái niệm giữa chú ý và cơ chế bộ nhớ, ít nhất là từ góc độ tính toán, nếu không phải từ góc độ tâm lý học, ta có thể xem chú ý như một quá trình truy xuất thông tin từ một ngân hàng bộ nhớ (tức $\{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$) bằng một truy vấn cho trước (tức $\mathbf{s}_i$). Như vậy, ta có thể diễn giải một hệ thống chuỗi-sang-chuỗi có cơ chế chú ý như sau. Ở phía nguồn, ta lưu thông tin vào một bộ nhớ được biểu diễn bằng một chuỗi vectơ $\mathbf{h}_1 \ldots \mathbf{h}_m$. Sau đó, ở phía đích, ta truy xuất từ bộ nhớ này để khôi phục từng bước thông tin phía nguồn.

4. Vượt ra ngoài bài toán chuỗi-sang-chuỗi

Dù trong mục này chúng ta giới hạn thảo luận ở bài toán biến đổi một chuỗi thành một chuỗi khác, cơ chế chú ý tổng quát có thể được áp dụng cho một phạm vi nhiệm vụ rộng hơn. Như đã nhắc ở Mục 5.3.2 và sẽ bàn ở Chương 6, một biến thể nổi tiếng của cách tiếp cận này là tự chú ý. Trong tự chú ý, mô hình chú ý QKV được dùng như một mô hình chuỗi. Nó cung cấp một khung tổng quát để mô hình hóa các tương tác và phụ thuộc giữa các biến đầu vào, nhờ đó áp dụng được cho nhiều bài toán. Chẳng hạn, ta có thể ghép $\mathbf{h}_1 \dots \mathbf{h}_m$ và $\mathbf{s}_1 \dots \mathbf{s}_n$ thành một chuỗi kết hợp duy nhất $\mathbf{h}_1 \dots \mathbf{h}_m \mathbf{s}_1 \dots \mathbf{s}_n$, rồi áp dụng mô hình cho chuỗi mới này. Bằng cách đó, tự chú ý có thể được mở rộng một cách tự nhiên thành một mô hình chuỗi-sang-chuỗi [Lample và Conneau, 2019; Raffel và cộng sự, 2020]. Cách tiếp cận này hiệu quả ngay cả khi $\mathbf{h}_1 \dots \mathbf{h}_m$ và $\mathbf{s}_1 \dots \mathbf{s}_n$ biểu diễn các phương thức (*modality*) khác nhau. Ví dụ, nếu ta dùng $\mathbf{h}_1 \dots \mathbf{h}_m$ để biểu diễn văn bản và $\mathbf{s}_1 \dots \mathbf{s}_n$ để biểu diễn một ảnh, ta sẽ có một mô hình đa phương thức (*multi-modal model*) hợp nhất biểu diễn văn bản và hình ảnh bằng cách thực hiện tự chú ý trên cả hai [Chen và cộng sự, 2020c].

Một cách tiếp cận khác để học biểu diễn chung của các chuỗi là xây dựng nhiều mô hình chú ý, cho phép mỗi chuỗi chú ý tới chuỗi kia. Một ví dụ của cách tiếp cận này là **đồng chú ý** (*co-attention*), được dùng rộng rãi trong xử lý ngôn ngữ đa phương thức [Lu và cộng sự, 2016]. Trong các nhiệm vụ như hỏi đáp trực quan (*visual question answering*) (VQA), ta muốn xác định những phần nào của ảnh tương ứng với một từ cụ thể trong câu hỏi, và ngược lại, những từ nào trong câu hỏi khớp với một vùng hình ảnh cho trước. Để làm được điều này, ta xây dựng hai mô hình chú ý: một cho chú ý từ ảnh sang văn bản, và một cho chú ý từ văn bản sang ảnh. Đầu ra của cả hai mô hình đóng vai trò biểu diễn chung của ảnh và văn bản, và do đó có thể được dùng làm đặc trưng để dự đoán câu trả lời.

Các mô hình chú ý bàn trong mục này *bất biến* đối với thứ tự của đầu vào. Dù tính bất biến hoán vị (*permutation invariance*) này gây khó khăn cho việc mô hình hóa dữ liệu tuần tự, vấn đề có thể được giải quyết bằng cách mã hóa tường minh thông tin vị trí vào chính đầu vào (xem Chương 4 và Chương 6). Mặt khác, sự đơn giản của cách phát biểu này khiến các mô hình này áp dụng được rộng rãi: dữ liệu đầu vào không cần có dạng chuỗi. Nhờ đó, cơ chế chú ý có thể được áp dụng trực tiếp cho các cấu trúc dữ liệu phức tạp hơn, chẳng hạn đồ thị [Veličković và cộng sự, 2018; Lee và cộng sự, 2019].

Chú giải của người dịch: Vì sao chú ý "không biết thứ tự"?

Trong Phương trình (5.16), $\mathbf{c}_i = \sum_j \alpha_{i,j} \mathbf{h}_j$ là một *tổng*, mà phép cộng thì giao hoán: nếu ta tráo thứ tự các $\mathbf{h}_j$ (cùng với trọng số của chúng), kết quả không đổi. Như vậy, nếu không có thông tin vị trí, chú ý xem "mèo đuổi chuột" và "chuột đuổi mèo" như cùng một "túi từ". Đó là lý do Transformer phải thêm *mã hóa vị trí* vào đầu vào (Chương 6). Ngược lại, chính tính chất này giúp chú ý dùng được cho dữ liệu không có thứ tự tự nhiên như đồ thị.

5.4 Tìm kiếm

Tìm kiếm (*search*) là một bài toán nền tảng của trí tuệ nhân tạo và đóng vai trò then chốt trong nhiều hệ thống NLP. Ở đây, bài toán tìm kiếm là một thách thức về mặt tính toán, bởi số lượng giả thuyết (*hypothesis*) trong không gian tìm kiếm (*search space*) tăng theo hàm mũ cùng với độ dài của chuỗi và kích thước của bộ từ vựng phía đích. Trong bối cảnh này, tìm kiếm vét cạn (*exhaustive search*) tốn kém đến mức không thể thực hiện được. Vì vậy, các hệ thống thực tế thường dùng các thuật toán tìm kiếm hoặc các phương pháp heuristic (*heuristic*) để bảo đảm có thể tìm được lời giải tối ưu hoặc gần tối ưu trong một khoảng thời gian tính toán chấp nhận được.

Với nhiều ứng dụng chuỗi-sang-chuỗi trong thực tế, bài toán tìm kiếm — thường được gọi là suy luận hay giải mã (*decoding*) — có thể được phát biểu như sau:

$$\hat{\mathbf{y}} \;=\; \arg\max_{\mathbf{y}\in\Omega} \text{Score}(\mathbf{x}, \mathbf{y}), \tag{5.60}$$

trong đó $\text{Score}(\mathbf{x}, \mathbf{y})$ là một hàm tính điểm (*scoring function*) đo chất lượng của $\mathbf{y}$ khi biết $\mathbf{x}$.

Cách phát biểu này hơi khác so với Phương trình (5.2). Thứ nhất, ta dùng $\text{Score}(\mathbf{x}, \mathbf{y})$ thay cho $\Pr(\mathbf{y}|\mathbf{x})$ làm hàm mục tiêu. Mặc dù cách tiêu chuẩn để huấn luyện các mô hình chuỗi-sang-chuỗi là cực đại hóa $\Pr(\mathbf{y}|\mathbf{x})$ (hoặc $\Pr(\mathbf{x}, \mathbf{y})$), ở thời điểm suy luận ta thường phải tính đến những vấn đề riêng của từng tác vụ, chẳng hạn thiên lệch độ dài (*length bias*). Do đó, người ta thường đưa thêm các số hạng phạt và thưởng bên cạnh $\Pr(\mathbf{y}|\mathbf{x})$ để định nghĩa mục tiêu tìm kiếm cuối cùng (xem Mục 5.4.1). Khác biệt thứ hai nằm ở chính không gian tìm kiếm: nó bị giới hạn trong một tập con Ω. Trên thực tế, Ω là một không gian tìm kiếm đã được cắt tỉa, chỉ chứa một số tương đối nhỏ các giả thuyết có xác suất cao. Điều này thường đạt được nhờ các kỹ thuật cắt tỉa (*pruning*) và các thuật toán tìm kiếm tiên tiến (xem Mục 5.4.2). Trong mục này, chúng ta sẽ tìm hiểu các giải pháp cho những thách thức trên và bàn về một số cải tiến. Dù các phương pháp này phần lớn được thúc đẩy bởi những phát triển trong dịch máy, các khái niệm là tổng quát và áp dụng rộng rãi cho hầu hết các tác vụ sinh văn bản.

Chú giải của người dịch: Vì sao không thể thử hết mọi câu?

Giả sử bộ từ vựng chỉ có 10 từ và câu đầu ra dài đúng 5 từ. Theo quy tắc nhân trong tổ hợp, số câu có thể có là $10^5 = 100\,000$. Nhưng bộ từ vựng thực tế thường có vài chục nghìn từ, câu dài vài chục từ, nên số câu có thể có lớn hơn cả số nguyên tử trong vũ trụ quan sát được. Vì thế, "tìm kiếm" ở đây là tìm một cách khôn ngoan một câu có điểm cao, chứ không thể liệt kê hết rồi chọn. Ký hiệu $\arg\max_{\mathbf{y}\in\Omega}$ đọc là "giá trị $\mathbf{y}$ trong tập Ω làm cho biểu thức đạt lớn nhất".

5.4.1 Vấn đề độ dài

Nhắc lại từ Mục 5.2.2 rằng xác suất của chuỗi phía đích $\mathbf{y}$ khi biết chuỗi phía nguồn $\mathbf{x}$ có thể viết thành tích các xác suất của từng từ y_i khi biết cả các từ đã sinh trước đó $y_0 \dots y_{i-1}$ và $\mathbf{x}$. Ở đây, ta viết lại Phương trình (5.14) bằng một ký hiệu gọn hơn:

$$\Pr(\mathbf{y}|\mathbf{x}) \;=\; \prod_{i=1}^{n} \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}), \tag{5.61}$$

trong đó $\mathbf{y}_{<i}$ ký hiệu chuỗi $y_0 \dots y_{i-1}$. Một cách tương đương, ta có thể biểu diễn bằng log xác suất

$$\log \Pr(\mathbf{y}|\mathbf{x}) \;=\; \sum_{i=1}^{n} \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}). \tag{5.62}$$

Dù cách mô hình hóa đơn giản này mang lại những lợi thế rõ ràng cho các ứng dụng NLP thực tế, đáng tiếc là nó gây ra một thiên lệch mạnh về phía các chuỗi đích ngắn hơn. Vì vậy, dùng trực tiếp $\text{Score}(\mathbf{x}, \mathbf{y}) = \Pr(\mathbf{y}|\mathbf{x})$ hoặc $\log \Pr(\mathbf{y}|\mathbf{x})$ làm mục tiêu tìm kiếm là có vấn đề, vì đầu ra rất có thể sẽ là một chuỗi ngắn quá mức (ví dụ, một chuỗi chỉ gồm một hoặc hai từ). Vấn đề này là hệ quả trực tiếp của thiên kiến quy nạp (*inductive bias*) của mô hình. Nhìn từ góc độ học có giám sát, vấn đề này nảy sinh vì kỹ thuật teacher forcing (*teacher forcing*) được dùng trong huấn luyện: mô hình được tiếp xúc với các chuỗi đích chuẩn (*ground-truth*) và bị buộc phải dự đoán đúng chúng. Ngược lại, khi suy luận trên dữ liệu kiểm tra, ta phải khám phá một không gian giả thuyết rất lớn gồm các $\mathbf{y}$ với độ dài khác nhau, và so sánh chúng bằng một hàm tính điểm mà trên thực tế khác với hàm được tối ưu trong huấn luyện.

Chú giải của người dịch: Vì sao xác suất "thích" câu ngắn?

Mỗi xác suất $\Pr(y_i|\mathbf{y}_{<i}, \mathbf{x})$ là một số trong khoảng $(0, 1]$, nên logarit của nó luôn ≤ 0. Mỗi khi thêm một từ, ta lại cộng thêm một số âm (hay nhân thêm một số nhỏ hơn 1). Ví dụ, một câu 2 từ với xác suất mỗi từ là 0.5 có xác suất $0.5 \cdot 0.5 = 0.25$; một câu 6 từ với mỗi từ có xác suất 0.8 (tốt hơn hẳn!) lại chỉ có $0.8^6 \approx 0.26$, gần như bằng câu ngắn. Câu càng dài thì tích càng nhỏ, nên nếu chỉ so sánh xác suất, mô hình có xu hướng chọn câu ngắn cụt lủn dù chất lượng kém hơn.

Vấn đề này có thể được giải quyết bằng một kỹ thuật gọi là **thưởng độ dài** (*length reward*), tức là cộng điểm thưởng cho các chuỗi dài hơn bằng cách thêm một số hạng vào $\text{Score}(\mathbf{x}, \mathbf{y})$ [He và cộng sự, 2016c]. Như đã bàn ở Chương 3, một dạng thưởng độ dài thường dùng là

$$\text{Score}(\mathbf{x}, \mathbf{y}) \;=\; \log \Pr(\mathbf{y}|\mathbf{x}) + \lambda \cdot n. \tag{5.63}$$

Ở đây, số hạng thưởng độ dài là độ dài của $\mathbf{y}$ (tức $n = |\mathbf{y}|$), được nhân với trọng số là tham số $\lambda > 0$. Các cải tiến của cách làm này thay n bằng một độ dài chuỗi được ước lượng nhờ một mô hình dự đoán độ dài. Chẳng hạn, ta có thể chặn trên phần thưởng theo dạng sau [Huang và cộng sự, 2017b; Yang và cộng sự, 2018b]

$$\text{Score}(\mathbf{x}, \mathbf{y}) \;=\; \log \Pr(\mathbf{y}|\mathbf{x}) + \lambda \cdot \max(l_p, n), \tag{5.64}$$

trong đó l_p là một độ dài dự đoán, thường được định nghĩa là độ dài của $\mathbf{x}$ nhân với một hệ số, tức là $l_p = \text{scalar}_p \cdot m$.

Nếu thay log xác suất $\log \Pr(\mathbf{y}|\mathbf{x})$ cho bởi Phương trình (5.62) vào Phương trình (5.63), ta thu được

$$\begin{aligned} \text{Score}(\mathbf{x}, \mathbf{y}) &= \sum_{i=1}^{n} \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) + \lambda \cdot n \\ &= \sum_{i=1}^{n} \big[\log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) + \lambda \big]. \end{aligned} \tag{5.65}$$

Như vậy, ta có thể hiểu số hạng thưởng độ dài là một phần thưởng dành cho mỗi từ y_i. Phương pháp này đã được dùng rộng rãi trong các hệ thống **dịch máy thống kê** (*statistical machine translation, SMT*) (**SMT**), trong đó các phần thưởng được xem là đặc trưng của một mô hình log-tuyến tính (*log-linear model*) [Koehn và cộng sự, 2003; Chiang, 2007]. Để tìm giá trị thích hợp cho λ, ta có thể dùng huấn luyện cực tiểu tỉ lệ lỗi (*minimum error rate training*) [Och, 2003] theo truyền thống của SMT, hoặc dùng các phương pháp dựa trên gradient như trong các hệ thống dựa trên mạng nơ-ron [Murray và Chiang, 2018].

Chú giải của người dịch: Thưởng độ dài bằng số cụ thể

Lấy lại ví dụ ở chú giải trên, với log tự nhiên: câu 2 từ có $\log 0.25 \approx -1.39$; câu 6 từ có $\log 0.26 \approx -1.34$. Nếu chọn $\lambda = 0.3$, điểm theo Phương trình (5.63) là $-1.39 + 0.3 \cdot 2 = -0.79$ cho câu ngắn và $-1.34 + 0.3 \cdot 6 = 0.46$ cho câu dài. Nhờ phần thưởng, câu dài và tốt hơn giờ thắng rõ rệt. Phương trình (5.65) cho thấy điều này tương đương với việc cộng thêm λ cho *mỗi* từ được sinh ra.

Cách thứ hai để khiến việc tìm kiếm nghiêng về các chuỗi dài hơn, gọi là **chuẩn hóa độ dài** (*length normalization*), là chia $\log \Pr(\mathbf{y}|\mathbf{x})$ cho một số hạng hiệu chỉnh độ dài, viết dưới dạng

$$\text{Score}(\mathbf{x}, \mathbf{y}) \;=\; \frac{\log \Pr(\mathbf{y}|\mathbf{x})}{n_{\text{correct}}}. \tag{5.66}$$

Một ví dụ đơn giản của mô hình này là định nghĩa số hạng hiệu chỉnh độ dài chính là độ dài chuỗi [Jean và cộng sự, 2015], như sau

$$\begin{aligned} n_{\text{correct}} &= n \\ &= |\mathbf{y}|. \end{aligned} \tag{5.67}$$

Trong trường hợp này, $\frac{\log \Pr(\mathbf{y}|\mathbf{x})}{n} = \frac{\sum_{i=1}^{n} \log \Pr(y_i|\mathbf{y}_{<i},\mathbf{x})}{n}$ có thể được xem là trung bình nhân (*geometric mean*) trên thang log của các xác suất $\{\Pr(y_i|\mathbf{y}_{<i},\mathbf{x})\}$[4].

Bảng 5.2: Các hàm tính điểm cho thưởng độ dài và chuẩn hóa độ dài. $m = |\mathbf{x}|$, $n = |\mathbf{y}|$ và $l_p = \text{scalar}_p \cdot m$. λ và α là các tham số.

Phương pháp	Dạng của $\text{Score}(\mathbf{x}, \mathbf{y})$
Không thưởng/chuẩn hóa	$\log \Pr(\mathbf{y}\|\mathbf{x})$
Thưởng độ dài	$\log \Pr(\mathbf{y}\|\mathbf{x}) + \lambda \cdot n$
Thưởng độ dài có chặn	$\log \Pr(\mathbf{y}\|\mathbf{x}) + \lambda \cdot \max(l_p, n)$
Chuẩn hóa độ dài (cơ bản)	$\frac{\log \Pr(\mathbf{y}\|\mathbf{x})}{n}$
Chuẩn hóa độ dài (GNMT)	$\frac{\log \Pr(\mathbf{y}\|\mathbf{x})}{(5+n)^\alpha/(5+1)^\alpha}$

Một ví dụ nổi bật khác là chiến lược chuẩn hóa được dùng trong hệ thống GNMT [Wu và cộng sự, 2016]. Nó dùng một hệ số phạt độ dài được co giãn theo lũy thừa và dịch chuyển:

$$n_{\text{correct}} = \frac{(5+n)^\alpha}{(5+1)^\alpha}, \tag{5.70}$$

trong đó số mũ α là một siêu tham số và có thể được xác định bằng thực nghiệm trên một tập tinh chỉnh. Để so sánh các phương pháp khác nhau, Bảng 5.2 liệt kê các hàm tính điểm cho thưởng độ dài và chuẩn hóa độ dài.

Chú giải của người dịch: Chuẩn hóa độ dài: lấy "điểm trung bình mỗi từ"

Chia cho n giống như tính điểm trung bình mỗi môn thay vì tổng điểm: một học sinh học nhiều môn hơn không bị thiệt chỉ vì tổng điểm âm cộng dồn nhiều hơn. Với ví dụ trên: câu 2 từ có $-1.39/2 \approx -0.69$, câu 6 từ có $-1.34/6 \approx -0.22$, nên câu dài thắng. Chú thích chân trang kèm Phương trình (5.68–5.69) nhắc lại tính chất logarit $\log a_1 + \cdots + \log a_n = \log(a_1 \cdots a_n)$: trung bình cộng của các log bằng log của trung bình nhân $\sqrt[n]{a_1 \cdots a_n}$. Trong công thức GNMT (5.70), khi $\alpha = 0$ thì $n_{\text{correct}} = 1$ (không chuẩn hóa); khi $\alpha = 1$ thì $n_{\text{correct}} = (5+n)/6$, gần với chia cho độ dài. Vậy α là "núm vặn" điều chỉnh mức độ ưu ái câu dài.

Trong dịch máy, vấn đề độ dài gắn liền với vấn đề **độ phủ** (*coverage*), một chủ đề được nghiên cứu sâu rộng trong SMT. Khi dịch một chuỗi nguồn, điều quan trọng là theo dõi mỗi từ đã được dịch bao nhiêu lần. Ta nói rằng xảy ra **dịch thừa** (*over-translation*) (tạo ra một bản dịch dài sai) nếu các từ nguồn bị dịch nhiều lần, và xảy ra **dịch thiếu** (*under-translation*) (tạo ra một bản dịch bị cụt) nếu các từ nguồn

[4] Giả sử $\{a_1, \ldots, a_n\}$ là n biến. Vì

$$\exp\Big(\frac{\sum_{i=1}^{n} \log a_i}{n}\Big) = \Big(\prod_{i=1}^{n} a_i\Big)^{\frac{1}{n}}, \tag{5.68}$$

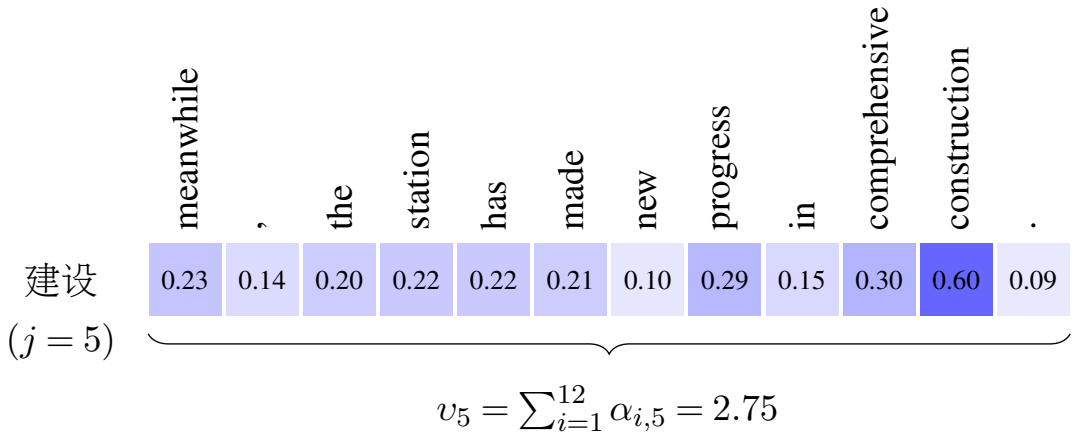
nên ta có

$$\frac{\sum_{i=1}^{n} \log a_i}{n} = \log\Big(\prod_{i=1}^{n} a_i\Big)^{\frac{1}{n}}. \tag{5.69}$$

không được dịch đầy đủ. Theo truyền thống, độ phủ của một chuỗi nguồn được mô hình hóa bằng một **vectơ độ phủ** (*coverage vector*) m chiều $\begin{bmatrix}\upsilon_1 & \dots & \upsilon_m\end{bmatrix}$. Mỗi phần tử υ_j cho biết mức độ mà từ nguồn x_j đã được dịch. Trong các hệ thống SMT, υ_j thường là một biến nhị phân: 0 nghĩa là chưa dịch, và 1 nghĩa là đã dịch. Tuy nhiên, các hệ thống NMT tiêu chuẩn không có một cơ chế độ phủ tường minh dạng ký hiệu như vậy. Thay vào đó, chúng dựa vào các trọng số chú ý phân bố trên các từ đích. Vì thế, một cách định lượng độ phủ của từ nguồn x_j là đo xem nó được các từ đích đã sinh ra chú ý tới mạnh đến mức nào. Để làm vậy, ta định nghĩa lại υ_j là một biến liên tục:

$$\upsilon_j \ = \ \sum_{i=1}^{n} \alpha_{i,j}. \tag{5.71}$$

Do đó, υ_j có thể được xem là "số lần" x_j được dịch; ví dụ, $\upsilon_j = 0$ nghĩa là x_j hoàn toàn không được dịch, và $\upsilon_j = 1$ nghĩa là x_j chỉ được tính một lần trong bản dịch. Hãy xét ví dụ trong Hình 5.8. Với từ phía nguồn 建设[5], các trọng số chú ý tương ứng được cho dưới đây.



Ta sẽ nói rằng 建设 được dịch 2.75 lần. Có thể dùng $\{\upsilon_1, \dots, \upsilon_m\}$ để định nghĩa mức độ chuỗi phía nguồn được phủ trong bản dịch. Một cách đơn giản là xây dựng một điểm độ phủ (*coverage score*) $\text{cp}(\mathbf{x}, \mathbf{y})$ bằng cách kết hợp $\{\upsilon_1, \dots, \upsilon_m\}$. Chẳng hạn, hệ thống GNMT định nghĩa $\text{cp}(\mathbf{x}, \mathbf{y})$ theo dạng sau

$$\text{cp}(\mathbf{x}, \mathbf{y}) \ = \ \beta \sum_{j=1}^{m} \log(\min(\upsilon_j, 1)), \tag{5.72}$$

trong đó β quyết định trọng số của mô hình độ phủ. Ý tưởng trực giác là: nếu $\upsilon_j \geq 1$, từ x_j được xem là đã được dịch đầy đủ; nếu $\upsilon_j < 1$, x_j chưa được dịch đủ. Kết quả là $\text{cp}(\mathbf{x}, \mathbf{y})$ phạt những giả thuyết có các từ nguồn bị dịch thiếu. Một phiên bản cải tiến của hình phạt này được Li và cộng sự [2018] đề xuất:

$$\text{cp}(\mathbf{x}, \mathbf{y}) \ = \ \beta \sum_{j=1}^{m} \log(\max(\upsilon_j, \gamma)), \tag{5.73}$$

trong đó γ là siêu tham số cắt ngưỡng, cho phép một mức dung sai đối với việc dịch thiếu. Một dạng tương tự được đề xuất trong [Chorowski và Jaitly, 2017]

$$\text{cp}(\mathbf{x}, \mathbf{y}) \ = \ \beta \sum_{j=1}^{m} \mathbb{1}(\upsilon_j > \gamma). \tag{5.74}$$

[5] **[ND]** Tiếng Trung, đọc là *jiànshè*, nghĩa là "xây dựng".

Nó đếm số lần $\upsilon_j > \gamma$.

Chú giải của người dịch: Đọc hiểu điểm độ phủ

Trong hình trên, mỗi ô là trọng số chú ý $\alpha_{i,5}$ mà từ đích thứ i dành cho từ nguồn 建设; cộng cả 12 ô được $\upsilon_5 = 2.75$ (từ "construction" — "xây dựng" — chú ý mạnh nhất, 0.60). Ở Phương trình (5.72): nếu $\upsilon_j \geq 1$ thì $\min(\upsilon_j, 1) = 1$ và $\log 1 = 0$, tức không bị phạt; nếu, chẳng hạn, $\upsilon_j = 0.1$ thì $\log 0.1 \approx -2.3$, một mức phạt âm khá lớn. Vì vậy, từ nào "bị bỏ quên" sẽ kéo điểm xuống. Ở Phương trình (5.74), $\mathbb{1}(\cdot)$ là *hàm chỉ thị*: bằng 1 nếu điều kiện trong ngoặc đúng, bằng 0 nếu sai; tổng của nó đơn giản là đếm số từ nguồn có $\upsilon_j > \gamma$.

$\text{cp}(\mathbf{x}, \mathbf{y})$ có thể dễ dàng được đưa vào quá trình tìm kiếm bằng cách cộng nó vào $\text{Score}(\mathbf{x}, \mathbf{y})$. Ví dụ, hàm tính điểm kiểu GNMT được định nghĩa là

$$\text{Score}(\mathbf{x}, \mathbf{y}) \;=\; \frac{\log \Pr(\mathbf{y}|\mathbf{x})}{(5+n)^\alpha/(5+1)^\alpha} + \text{cp}(\mathbf{x}, \mathbf{y}). \tag{5.75}$$

Trong thực tế, sửa đổi $\text{Score}(\mathbf{x}, \mathbf{y})$ không phải là cách duy nhất để giảm nhẹ các vấn đề độ dài và độ phủ trong quá trình tìm kiếm. Một chiến lược khác là thay đổi kiến trúc để mô hình hóa tường minh các hiện tượng này [Tu và cộng sự, 2016; Mi và cộng sự, 2016a; Sankaran và cộng sự, 2016; See và cộng sự, 2017; Malaviya và cộng sự, 2018]. Lưu ý rằng trong một số trường hợp, độ dài của chuỗi đích được chỉ định hoặc dự đoán trước. Khi đó, ta có thể dùng các mô hình phi tự hồi quy (*non-autoregressive model*) [Gu và cộng sự, 2018] hoặc thiết kế các hệ thống sinh văn bản có kiểm soát độ dài cho những ứng dụng cụ thể [Rush và cộng sự, 2015; Kikuchi và cộng sự, 2016].

5.4.2 Cắt tỉa và tìm kiếm chùm

Có nhiều cách để định nghĩa một không gian tìm kiếm. Là một khái niệm chung trong khoa học máy tính, không gian tìm kiếm thường được định nghĩa là tập hợp mọi lời giải có thể có của một bài toán. Với các bài toán chuỗi-sang-chuỗi, ta có thể xem một giả thuyết là một ánh xạ từ một chuỗi phía nguồn $\mathbf{x}$ sang một chuỗi phía đích $\mathbf{y}$, và không gian tìm kiếm là tập hợp tất cả những giả thuyết như vậy[6].

Ta có thể hiện thực một thuật toán tìm kiếm bằng cách tổ chức các giả thuyết. Nhắc lại rằng trong Phương trình (5.61) và (5.62), ta tính xác suất của $\mathbf{y}$ khi biết $\mathbf{x}$ bằng một phép phân tích từ trái sang phải. Một hệ thống tìm kiếm điển hình duy trì một tập các giả thuyết (hoặc giả thuyết bộ phận (*partial hypothesis*)) và mở rộng chúng dần dần[7]. Thủ tục tìm kiếm bắt đầu với một tập giả thuyết ban đầu Z_0 chứa một giả thuyết duy nhất z_0, có phía đích là $y_0 = \langle\text{SOS}\rangle$, ký hiệu bắt đầu của mọi chuỗi đích. Sau đó ta mở rộng tập giả thuyết này qua một loạt bước tìm kiếm. Gọi $Z_0 \ldots Z_{n_{\max}}$ là dãy các tập giả thuyết, trong đó $n_{\max}$ là số bước tìm kiếm tối đa. Ở bước i, ta mở rộng mỗi giả thuyết bằng cách nối thêm một từ mới v_k lấy từ bộ từ vựng V_y. Gọi $z.src$ và $z.tgt$ lần lượt là phía nguồn và phía đích của giả thuyết z. Hiển nhiên $z.src = \mathbf{x}$ với mọi z. Cho một giả thuyết $z_{\text{cur}} \in Z_{i-1}$, ta có thể mở rộng nó để sinh ra $|V_y|$ giả thuyết mới, ký hiệu là $\{z^1_{\text{next}}, \ldots, z^{|V_y|}_{\text{next}}\}$:

$$\{z^1_{\text{next}}, \ldots, z^{|V_y|}_{\text{next}}\} = \text{Extend}(z_{\text{cur}}, V_y)$$

[6] Ở đây ta dùng $(\mathbf{x}, \mathbf{y})$ để ký hiệu một giả thuyết. Khi có nhiều ánh xạ từ $\mathbf{x}$ sang $\mathbf{y}$, một giả thuyết có thể được biểu diễn là $(\mathbf{x}, \mathbf{y}, d)$, trong đó d ký hiệu ánh xạ cụ thể. Ví dụ, nếu ta biến đổi $\mathbf{x}$ thành $\mathbf{y}$ bằng một văn phạm đồng bộ, có thể có nhiều cách dẫn xuất bằng các luật văn phạm để đạt được điều đó.

[7] Một giả thuyết được xem là bộ phận khi chuỗi phía đích tương ứng không kết thúc bằng $\langle\text{EOS}\rangle$, tức là một chuỗi chưa hoàn chỉnh. Trong mục này, chúng tôi dùng các thuật ngữ *giả thuyết* và *giả thuyết bộ phận* thay thế cho nhau vì dạng của chúng giống hệt nhau.

$$= \bigcup_{v_k \in V_y} \text{Extend}(z_{\text{cur}}, v_k). \tag{5.76}$$

Ở đây, $\text{Extend}(z_{\text{cur}}, v_k)$ là hàm mở rộng giả thuyết đầu vào z_{cur} với một từ $v_k \in V_y$. Phía đích của giả thuyết thu được là phép nối $z_{\text{cur}}.tgt$ với v_k, viết là[8]:

$$z^k_{\text{next}}.tgt \;=\; z_{\text{cur}}.tgt \circ v_k. \tag{5.77}$$

Các giả thuyết mới $\{z^1_{\text{next}}, \ldots, z^{|V_y|}_{\text{next}}\}$ sau đó được thêm vào Z_i. Hình 5.9 minh họa vài bước đầu của quá trình mở rộng này. Mọi giả thuyết có thể được tổ chức thành một cấu trúc cây, trong đó Z_i tương ứng với tập các nút ở mức i của cây tìm kiếm. Do đó ta có:

$$|Z_i| \;=\; |V_y| \cdot |Z_{i-1}|. \tag{5.78}$$

Nói cách khác, kích thước của Z_i tăng theo hàm mũ với số bước, cho ra $|Z_i| = |V_y|^i$.

Mỗi giả thuyết z gắn với một log xác suất $\log \Pr(z.tgt|z.src)$. Theo dạng của Phương trình (5.62), xác suất này có thể được tính theo kiểu đệ quy:

$$\begin{aligned}\log \Pr(z^k_{\text{next}}.tgt|z^k_{\text{next}}.src) = \log \Pr(z_{\text{cur}}.tgt|z_{\text{cur}}.src) + \\ \log \Pr(v_k|z_{\text{cur}}.tgt, z_{\text{cur}}.src).\end{aligned} \tag{5.79}$$

Ví dụ, giả sử $z^k_{\text{next}}.tgt = y_0 \ldots y_{i+1}$. Dạng của Phương trình (5.79) trở nên rõ ràng từ biểu thức sau

$$\begin{aligned}\underbrace{\log \Pr(y_0 \ldots y_{i+1}|\mathbf{x})}_{\log \Pr(z^k_{\text{next}}.tgt|z^k_{\text{next}}.src)} &= \underbrace{\log \Pr(y_0 \ldots y_i|\mathbf{x})}_{\log \Pr(z_{\text{cur}}.tgt|z_{\text{cur}}.src)} + \underbrace{\log \Pr(y_{i+1}|y_0 \ldots y_i, \mathbf{x})}_{\log \Pr(v_k|z_{\text{cur}}.tgt, z_{\text{cur}}.src)} \\ &= \sum_{k=1}^{i} \log \Pr(y_k|\mathbf{y}_{<k}, \mathbf{x}) + \log \Pr(y_{i+1}|y_0 \ldots y_i, \mathbf{x}) \\ &= \sum_{k=1}^{i+1} \log \Pr(y_k|\mathbf{y}_{<k}, \mathbf{x}).\end{aligned} \tag{5.80}$$

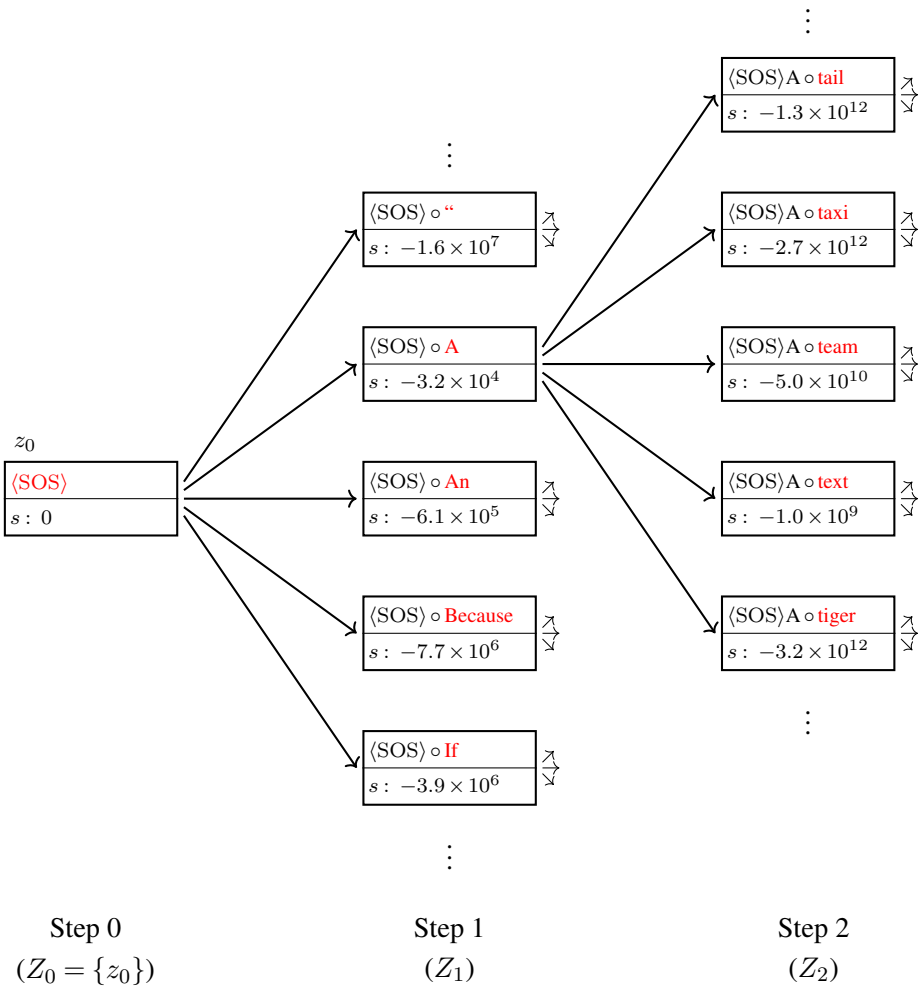
Có xác suất này, ta có thể tính $z.score = \text{Score}(z.src, z.tgt)$ như ở Mục 5.4.1. Điểm này cho phép ta xếp hạng và so sánh các giả thuyết khác nhau. Nếu một giả thuyết kết thúc bằng ký hiệu $\langle\text{EOS}\rangle$, nó được xem là hoàn chỉnh và không được mở rộng nữa. Khi đã hoàn chỉnh, nó được đẩy vào một đống cực đại (*max-heap*)[9]. Sau đó ta có thể lấy ra các giả thuyết có điểm cao nhất từ đống này. Thủ tục tìm kiếm thường dừng khi đã tìm được một số lượng giả thuyết hoàn chỉnh định trước, chẳng hạn khi đống đã đầy. Thuật toán tìm kiếm thu được được trình bày dưới đây.

Chú giải của người dịch: Đọc thuật toán và cấu trúc đống

Một *thuật toán* là dãy các bước rõ ràng mà máy tính thực hiện. Ở đây, ba vòng lặp **for** lồng nhau có nghĩa là: với mỗi bước, với mỗi giả thuyết hiện có, thử nối thêm *từng* từ trong bộ từ vựng. Ký hiệu $z.tgt$, $z.score$ đọc là "trường tgt (hay $score$) của đối tượng z", giống như "họ tên" và "điểm" là các trường trong hồ sơ của một học sinh. *Đống cực đại* (max-heap) là một cấu trúc dữ liệu luôn giữ phần tử lớn nhất ở "đỉnh", nhờ đó lấy ra phần tử có điểm cao nhất rất nhanh — giống như một bảng xếp hạng luôn được cập nhật. Phương trình (5.79)

[8] Ta dùng $a \circ b$ để ký hiệu phép nối hai xâu a và b.

[9] Cho một đống cực đại a, ta dùng a.Pop() để ký hiệu hàm lấy ra và trả về phần tử ở đỉnh của a, và a.PopAll() cho hàm lấy ra tất cả các phần tử.



Hình 5.9: Minh họa việc mở rộng giả thuyết trong 3 bước đầu tiên. Mỗi giả thuyết (bộ phận) được biểu diễn bằng một hộp, trong đó ta ghi chuỗi phía đích tương ứng và điểm của mô hình. Mỗi bước tìm kiếm gắn với một tập giả thuyết Z_i. Ta bắt đầu với giả thuyết $z_0 \in Z_0$ biểu thị ký hiệu bắt đầu ⟨SOS⟩. Ở bước i, ta mở rộng mọi giả thuyết trong Z_{i-1} bằng cách thử nối thêm mọi từ trong bộ từ vựng V_y (xem các từ màu đỏ). Thao tác này cho ra $|V_y| \cdot |Z_{i-1}|$ giả thuyết, tạo thành tập giả thuyết Z_i. Thủ tục mở rộng giả thuyết này thể hiện một thuật toán tìm kiếm theo chiều rộng: ta tạo tất cả các nút (hay trạng thái tìm kiếm) ở độ sâu $i-1$ trước khi chuyển sang độ sâu i. Một cấu trúc cây được tạo ra cùng với thủ tục này, và từ một nút lá của cây có thể lần ngược đường tìm kiếm về tới nút gốc.

Thuật toán: Một thuật toán tìm kiếm theo chiều rộng đơn giản

SIMPLESEARCH($\mathbf{x}$)

// Tìm giả thuyết tốt nhất khi biết chuỗi phía nguồn $\mathbf{x}$

Tạo một Heap (đống) có $size_{\text{heap}}$ phần tử
$Z_0 = \{z_0\}$ với $z_0.src = \mathbf{x}$ và $z_0.tgt = y_0$
for mỗi bước $i = 1$ đến n_{max} **do**
    for mỗi giả thuyết $z_{\text{cur}} \in Z_{i-1}$ **do**
        for mỗi từ $v_k \in V_y$ **do**
            $z_{\text{next}} = \text{Extend}(z_{\text{cur}}, v_k, \mathbf{x})$
            if $z_{\text{next}}.tgt$ kết thúc bằng $\langle\text{EOS}\rangle$ **then**
                Thêm z_{next} vào Heap
            else
                Thêm z_{next} vào Z_i
            end if
            if Heap đã đầy và/hoặc thỏa các tiêu chí dừng khác **then**
                Thoát khỏi tất cả các vòng lặp
            end if
        end for
    end for
end for
return Heap.Pop()

Extend(z_{cur}, v_k, src)

// Tạo một giả thuyết mới bằng cách nối một từ mới v_k *vào phía đích của* z_{cur}

Tạo một giả thuyết mới z_{next}
$z_{\text{next}}.src = src$
$z_{\text{next}}.tgt = z_{\text{cur}}.tgt \circ v_k$
$z_{\text{next}}.prob = z_{\text{cur}}.prob + \log \Pr(v_k | z_{\text{cur}}.tgt, z_{\text{cur}}.src)$ ▷ xem Phương trình (5.79)
$z_{\text{next}}.score = \text{Score}(z_{\text{next}}.src, z_{\text{next}}.tgt)$ ▷ xem Mục 5.4.1
return z_{next}

cho thấy điểm log của câu mới bằng điểm log của câu cũ cộng thêm log xác suất của từ mới, nên không cần tính lại từ đầu.

Nếu đống giả thuyết có sức chứa vô hạn ($size_{\text{heap}} = \infty$), thuật toán này thực hiện một phép tìm kiếm vét cạn trên không gian của mọi giả thuyết có độ dài tối đa n_{max}, tạo ra nhiều nhất $1 + |V_y| + |V_y|^2 + \cdots + |V_y|^{n_{\text{max}}} = \frac{|V_y|^{n_{\text{max}}+1}-1}{|V_y|-1}$ giả thuyết hoàn chỉnh. Không gian tìm kiếm này cực kỳ lớn và trên thực tế không thể tính toán nổi[10]. Vì vậy, các hệ thống thực tế dựa rất nhiều vào các kỹ thuật cắt tỉa để làm cho quá trình tìm kiếm khả thi. Trong phần còn lại của mục này, chúng tôi sẽ giới thiệu hai thuật toán tìm kiếm phổ biến sử dụng cắt tỉa để giải mã hiệu quả.

Chú giải của người dịch: Tổng cấp số nhân

Biểu thức $1+|V_y|+|V_y|^2+\cdots+|V_y|^{n_{\text{max}}}$ chính là tổng của một cấp số nhân có công bội $q = |V_y|$ (kiến thức lớp 11): $1+q+\cdots+q^N = \frac{q^{N+1}-1}{q-1}$. Với $q = 20{,}000$ và $N = 20$, số hạng trội là $q^{20} = (2\times10^4)^{20} \approx 1.05\times10^{86}$, lớn hơn cả ước tính số nguyên tử trong vũ trụ quan sát được (khoảng 10^{80}).

1. Tìm kiếm tham lam

Chiến lược tham lam (*greedy strategy*) là một khái niệm nền tảng trong thiết kế thuật toán. Nó dựa trên heuristic rằng có thể thu được lời giải tối ưu toàn cục bằng cách đưa ra các quyết định tối ưu cục bộ. Dù chiến lược này chỉ bảo đảm tính tối ưu cục bộ, nó vẫn đủ dùng cho nhiều ứng dụng thực tế, và độ phức tạp tính toán thấp là một lợi thế rõ ràng của nó.

Áp dụng chiến lược tham lam cho bài toán tìm kiếm của chúng ta rất đơn giản. Ở mỗi bước mở rộng i, ta chỉ xét một giả thuyết tốt nhất cho đến bước này. Chính xác hơn, với mọi tập Z_i, ta chỉ giữ lại giả thuyết có điểm mô hình cao nhất và loại bỏ phần còn lại. Giả thuyết tốt nhất ở mỗi bước của tìm kiếm tham lam được cho bởi

$$z_{\text{best}} = \underset{z_{\text{next}} \in \text{Extend}(Z_{i-1}, V_y)}{\arg\max} z_{\text{next}}.score. \tag{5.81}$$

Ở đây, hàm $\text{Extend}(Z_{i-1}, V_y)$ có cùng ý nghĩa như trong Phương trình (5.76), nhưng tác động trên một tập giả thuyết, tức là

$$\text{Extend}(Z_{i-1}, V_y) = \bigcup_{z \in Z_{i-1}} \text{Extend}(z, V_y). \tag{5.82}$$

Khi đó, Z_i được định nghĩa là

$$Z_i = \{z_{\text{best}}\}. \tag{5.83}$$

Dưới đây là một thuật toán tìm kiếm tham lam cho các bài toán chuỗi-sang-chuỗi.

Ở mỗi bước tìm kiếm, ta chỉ có một giả thuyết đang hoạt động cần mở rộng (tức $|Z_{i-1}| = 1$). Do đó, ta sinh ra $|V_y|$ phần mở rộng, rồi chọn phần mở rộng tốt nhất cho bước tiếp theo. Tổng số lần gọi $\text{Extend}(z_{\text{cur}}, v_k)$ là $|V_y| \cdot n_{\text{max}}$. Giả sử $\text{Extend}(z_{\text{cur}}, v_k)$ chạy trong thời gian hằng số, thì độ phức tạp thời gian (*time complexity*) tổng thể của thuật toán tăng tuyến tính theo $|V_y|$ và n_{max}.

[10] Ví dụ, xét kích thước bộ từ vựng là 20,000 ($|V_y| = 20{,}000$) và giới hạn độ dài là 20 ($n_{\text{max}} = 20$). Tổng số giả thuyết $\frac{|V_y|^{n_{\text{max}}+1}-1}{|V_y|-1}$ sẽ xấp xỉ 1.05×10^{86}.

Thuật toán: Một thuật toán tìm kiếm tham lam

GreedySearch($\mathbf{x}$)
// Tìm giả thuyết "tốt nhất" theo cách tham lam

Tạo một giả thuyết z_{best}
$Z_0 = \{z_0\}$ với $z_0.src = \mathbf{x}$ và $z_0.tgt = y_0$
for mỗi bước $i = 1$ đến $n_{\max}$ **do**
  $z_{\text{best}}.score = -\infty$
  for mỗi giả thuyết $z_{\text{cur}} \in Z_{i-1}$ **do**
    for mỗi từ $v_k \in V_y$ **do**
      $z_{\text{next}} = \text{Extend}(z_{\text{cur}}, v_k, \mathbf{x})$
      if $z_{\text{best}}.score < z_{\text{next}}.score$ **then**
        $z_{\text{best}} = z_{\text{next}}$
      end if
    end for
  end for
  if $z_{\text{best}}.tgt$ kết thúc bằng ⟨EOS⟩ và/hoặc thỏa các tiêu chí dừng khác **then**
    Thoát khỏi vòng lặp
  end if
  $Z_i = \{z_{\text{best}}\}$
end for
return z_{best}

Chú giải của người dịch: Tham lam: chọn tốt nhất trước mắt

Hãy hình dung leo núi trong sương mù: ở mỗi bước, bạn chỉ nhìn thấy xung quanh và luôn bước theo hướng dốc lên nhất. Cách này nhanh, nhưng có thể đưa bạn lên một đỉnh đồi nhỏ thay vì đỉnh núi cao nhất. Ví dụ: bước 1 chọn "A" (xác suất 0.5) thay vì "The" (0.4); nhưng nếu sau "A" từ tốt nhất chỉ có xác suất 0.3 còn sau "The" có từ với xác suất 0.9, thì cặp "The …" có xác suất $0.4 \cdot 0.9 = 0.36$, lớn hơn $0.5 \cdot 0.3 = 0.15$. Tìm kiếm tham lam đã bỏ lỡ lời giải tốt hơn. Bù lại, với $|V_y| = 20{,}000$ và $n_{\max} = 20$, nó chỉ cần 400,000 lần gọi Extend thay vì khoảng 10^{86}.

2. Tìm kiếm chùm

Tìm kiếm chùm (*beam search*) là một sự mở rộng tự nhiên của thuật toán tìm kiếm tham lam ở trên. Nó cũng dựa trên heuristic tham lam, và vì thế là một loại thuật toán tham lam. Ý tưởng cốt lõi của tìm kiếm chùm là giữ lại một số lượng định trước các giả thuyết hứa hẹn nhất ở mỗi bước, thay vì chỉ một giả thuyết tốt nhất. Một **chùm** (*beam*) là một cấu trúc dữ liệu lưu các giả thuyết tốt nhất mà ta đã sinh ra cho đến lúc đó. Số giả thuyết trong một chùm là một tham số định trước, gọi là **độ rộng chùm** (*beam width*) hay **kích thước chùm** (*beam size*). Ở đây ta có thể đơn giản xem Z_i là một chùm, viết là

$$Z_i \;=\; \{z_{\text{best}}^1, \ldots, z_{\text{best}}^{size_{\text{beam}}}\}, \tag{5.84}$$

trong đó $size_{\text{beam}}$ là kích thước chùm. z_{best}^1 là giả thuyết tốt nhất trong tập $\text{Extend}(Z_{i-1}, V_y)$ (xem Phương trình (5.81)), z_{best}^2 là giả thuyết tốt thứ hai trong $\text{Extend}(Z_{i-1}, V_y)$, và cứ thế tiếp tục.

Mã giả sau đây mô tả một thuật toán tìm kiếm chùm cho các bài toán chuỗi-sang-chuỗi.

Hàm UpdateBeam(Beam, z_{next}) có thể được xem là một trường hợp của cắt tỉa theo biểu đồ tần suất (*histogram pruning*). Lưu ý rằng khung tổng quát này cho ta một cách đơn giản để hiện thực các phương pháp cắt tỉa khác, và ta có thể sửa UpdateBeam(Beam, z_{next}) khi cần. Ví dụ, một phương pháp khác

Thuật toán: Một thuật toán tìm kiếm chùm

BeamSearch($\mathbf{x}$)
// Tìm giả thuyết "tốt nhất" bằng cách xét một số ứng viên tốt nhất ở mỗi bước

Tạo một Heap (đống) có $size_{\text{heap}}$ phần tử
$Z_0 = \{z_0\}$ với $z_0.src = \mathbf{x}$ và $z_0.tgt = y_0$
for mỗi bước $i = 1$ đến n_{max} **do**
    Tạo một đống Beam (chùm) có $size_{\text{beam}}$ phần tử
    for mỗi giả thuyết $z_{\text{cur}} \in Z_{i-1}$ **do**
        for mỗi từ $v_k \in V_y$ **do**
            $z_{\text{next}} = \text{Extend}(z_{\text{cur}}, v_k, \mathbf{x})$
            if $z_{\text{next}}.tgt$ kết thúc bằng $\langle\text{EOS}\rangle$ **then**
                Thêm z_{next} vào Heap
            else
                UpdateBeam(Beam, z_{next})
            end if
            if Heap đã đầy và/hoặc thỏa các tiêu chí dừng khác **then**
                Thoát khỏi tất cả các vòng lặp
            end if
        end for
    end for
    $Z_i = \text{Beam.PopAll}()$
end for
return Heap.Pop()

UpdateBeam(Beam, z_{next})
// Cập nhật Beam *với một giả thuyết mới sinh* z_{next}

Thêm z_{next} vào Beam [a]

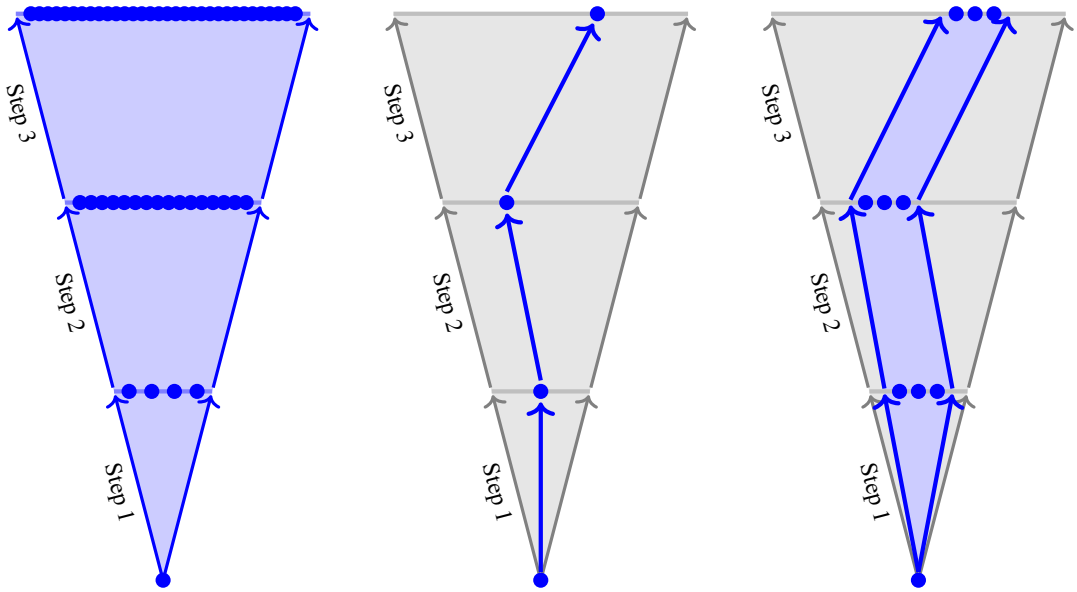
[a] Beam là một đống cực đại có giới hạn, duy trì $size_{\text{beam}}$ giả thuyết đứng đầu. Vì vậy, nếu $z_{\text{next}}.score$ thấp hơn điểm thấp nhất trong đống thì nó bị loại bỏ. Nói cách khác, Beam chỉ lưu $size_{\text{beam}}$ giả thuyết tốt nhất và bỏ qua phần còn lại.

gọi là **cắt tỉa theo ngưỡng** (*threshold pruning*) giữ lại những giả thuyết có điểm mô hình chênh lệch so với giả thuyết tốt nhất trong Beam ít hơn một ngưỡng định trước; chẳng hạn, ta loại bỏ z_{next} trong UpdateBeam(Beam, z_{next}) nếu

$$z_{\text{next}}.score \ < \ z_{\text{best}}.score - \theta_{\text{beam}}, \tag{5.85}$$

trong đó z_{best} là giả thuyết tốt nhất trong Beam. Ngoài ra, ta có thể dùng phương pháp ngưỡng tương đối [Freitag và Al-Onaizan, 2017], cho bởi

$$z_{\text{next}}.score \ < \ z_{\text{best}}.score \cdot \theta_{\text{beam}}. \tag{5.86}$$



(a) Exhausted Search (b) (1-best) Greedy Search (c) Beam Search

Hình 5.10: So sánh tìm kiếm vét cạn, tìm kiếm tham lam (1-best) và tìm kiếm chùm. Các quả cầu biểu diễn trạng thái tìm kiếm hay giả thuyết bộ phận. Tìm kiếm vét cạn khám phá mọi trạng thái trong không gian tìm kiếm. Ngược lại, tìm kiếm tham lam chỉ giữ đường đi tốt nhất (1-best) gồm các trạng thái tìm kiếm và cắt bỏ phần còn lại. Tìm kiếm chùm là sự dung hòa giữa hai cách trên và giữ các trạng thái tìm kiếm hứa hẹn nhất trong một chùm ở mỗi bước. (Nhãn trong hình: *Step* = bước; (a) tìm kiếm vét cạn; (b) tìm kiếm tham lam (1-best); (c) tìm kiếm chùm.)

Hình 5.10 so sánh tìm kiếm vét cạn, tìm kiếm tham lam (1-best) và tìm kiếm chùm. Ở một thái cực, lời giải tối ưu được bảo đảm nhưng phải thăm một số trạng thái tìm kiếm lớn theo hàm mũ. Ở thái cực kia, chỉ một số tối thiểu trạng thái được thăm, nhưng lời giải là dưới tối ưu (*sub-optimal*). Còn tìm kiếm chùm dung hòa giữa hai phương pháp. Kích thước chùm lớn hơn nghĩa là tốn công tìm kiếm hơn và có nhiều cơ hội tìm được giả thuyết tốt hơn, trong khi kích thước chùm nhỏ hơn nghĩa là tìm kiếm nhanh hơn và rủi ro bỏ lỡ lời giải tối ưu cao hơn. Cũng có thể dùng kích thước chùm thay đổi để dung hòa tốt hơn trong quá trình tìm kiếm [Buckman và cộng sự, 2016; Post và Vilar, 2018; Kulikov và cộng sự, 2019].

Chú giải của người dịch: Tìm kiếm chùm qua một ví dụ nhỏ

Giả sử kích thước chùm là 2. Bước 1, trong mọi từ đầu câu, ta giữ 2 từ có điểm cao nhất, ví dụ "A" và "The". Bước 2, nối thêm mọi từ vào *cả hai* (tạo $2 \cdot |V_y|$ ứng viên), rồi lại chỉ giữ 2 ứng viên tốt nhất, chẳng hạn "The cat" và "A cat". Cứ thế tiếp tục. Nhờ giữ nhiều hơn một ứng viên, ví dụ ở chú giải về tìm kiếm tham lam sẽ không bỏ lỡ cặp "The …". Mỗi bước chỉ tốn khoảng $size_{\text{beam}} \cdot |V_y|$ phép mở rộng, nên tổng chi phí vẫn tăng tuyến tính theo độ dài câu. Cắt tỉa theo ngưỡng (5.85) thì khác: thay vì giữ cố định k ứng viên, ta giữ mọi ứng viên có điểm không kém ứng viên tốt nhất quá θ_{beam}.

Một vấn đề quan trọng liên quan đến các thuật toán tìm kiếm này là vấn đề lỗi tìm kiếm (*search error*). Nói chung, lỗi tìm kiếm có thể được định nghĩa theo nhiều cách khác nhau. Ở đây ta nói rằng một lỗi tìm kiếm xảy ra nếu kết quả tìm kiếm không giống với kết quả của tìm kiếm vét cạn. Theo trực giác, giảm thiểu lỗi tìm kiếm sẽ cho kết quả chất lượng cao hơn. Vì vậy, ta có thể kỳ vọng rằng tăng kích thước chùm sẽ luôn cải thiện các chuỗi được sinh ra. Tuy nhiên, giả định này không phải lúc nào cũng đúng. Chẳng hạn, trong các hệ thống dịch máy nơ-ron, giải mã với kích thước chùm quá lớn có thể làm giảm chất lượng bản dịch [Koehn và Knowles, 2017]. Hiện tượng phản trực giác này đã thúc đẩy các nghiên cứu về sự suy giảm hiệu năng gắn với kích thước chùm lớn [Ott và cộng sự, 2018b; Yang và cộng sự, 2018b; Stahlberg và Byrne, 2019].

3. Tiêu chí dừng

Mặc dù độ phức tạp thời gian của các thuật toán đã bàn ở trên bị chặn bởi số bước tìm kiếm tối đa (tức $n_{\max}$), điều quan trọng là thiết kế các tiêu chí dừng (*stopping criteria*) hiệu quả hơn để kết thúc việc tìm kiếm càng sớm càng tốt, đặc biệt với các ứng dụng nhạy cảm với độ trễ (*latency*). Điều này thường đòi hỏi các heuristic để thiết kế thêm những tiêu chí giúp dừng thủ tục tìm kiếm ở một điểm tối ưu. Các tiêu chí dừng thường gặp gồm:

- Nếu đã tạo ra một số lượng giả thuyết hoàn chỉnh cho trước thì ta dừng tìm kiếm. Ví dụ, trong thuật toán tìm kiếm chùm mô tả ở tiểu mục này, chương trình tìm kiếm kết thúc khi ta có $size_{\text{heap}}$ giả thuyết hoàn chỉnh. Một cách khác để hiện thực ý tưởng này là thu hẹp chùm khi số giả thuyết hoàn chỉnh tăng lên. Trong hệ thống của Bahdanau và cộng sự [2014], mỗi khi một giả thuyết hoàn chỉnh mới được tạo ra, kích thước chùm giảm đi 1. Do đó, chương trình tìm kiếm sẽ kết thúc nếu kích thước chùm giảm về 0.

- Nếu mọi giả thuyết ở bước i đều có điểm thấp hơn điểm của giả thuyết hoàn chỉnh tốt nhất trong Heap một khoảng nào đó thì ta dừng tìm kiếm. Giả sử $z_{\text{bestinall}}$ là giả thuyết tốt nhất mà ta đã sinh ra cho đến lúc này (tức $z_{\text{bestinall}} = \text{Heap.Pop}()$). Nếu mọi giả thuyết z_{next} ở bước i thỏa mãn

$$z_{\text{bestinall}}.score - z_{\text{next}}.score \geq \theta_{\text{all}}, \tag{5.87}$$

  thì quá trình tìm kiếm kết thúc. Ở đây, θ_{all} là một siêu tham số thường được xác định bằng thực nghiệm. Một lựa chọn đơn giản là $\theta_{\text{all}} = 0$, được dùng trong một số framework chuỗi-sang-chuỗi phổ biến [Ott và cộng sự, 2019]. Trong một số điều kiện nhất định, chiến lược dừng sớm (*early stopping*) như vậy thậm chí có thể bảo đảm tính tối ưu của việc tìm kiếm [Huang và cộng sự, 2017b; Yang và cộng sự, 2018b].

- Nếu mọi giả thuyết ở bước i đều có điểm thấp hơn điểm của giả thuyết hoàn chỉnh cuối cùng trong Heap một khoảng nào đó thì ta dừng tìm kiếm. Đây là một điều kiện yếu cho việc dừng sớm.

- Nếu các giả thuyết xếp hạng đầu ở bước i đều là giả thuyết hoàn chỉnh thì ta dừng tìm kiếm. Đây

là một phiên bản dừng sớm quyết liệt hơn. Ví dụ, trong hệ thống của Klein và cộng sự [2017], chương trình tìm kiếm kết thúc ở bước i nếu giả thuyết đứng đầu (top-1) là một giả thuyết hoàn chỉnh.

- Nếu ngân sách tính toán (ví dụ, số phép toán dấu phẩy động tối đa hoặc một giới hạn chặt về thời gian thực) đã cạn thì việc tìm kiếm kết thúc. Trong các ứng dụng mà tài nguyên tính toán rất hạn chế và độ trễ là mối quan tâm hàng đầu, tiêu chí này trở nên thiết yếu.

Đôi khi thuật toán tìm kiếm có thể không tìm được giả thuyết hoàn chỉnh nào trước khi chạm giới hạn độ dài $n_{\max}$. Trên thực tế, một chiến lược dự phòng phổ biến trong những trường hợp này là hoàn chỉnh giả thuyết bộ phận có điểm cao nhất bằng cách nối token $\langle\text{EOS}\rangle$ vào cuối của nó.

Lưu ý rằng việc chọn tiêu chí dừng phù hợp phản ánh sự thỏa hiệp giữa hiệu quả tính toán và độ chính xác dự đoán khi suy luận (thường gọi là đánh đổi tốc độ–độ chính xác (*speed-accuracy trade-off*)). Dù dành thêm thời gian tính toán không bảo đảm đầu ra tốt hơn cho một hệ thống chuỗi-sang-chuỗi, việc hiểu kết quả heuristic của ta xấp xỉ lời giải tối ưu sát đến mức nào, bằng cách khám phá một vùng rộng hơn của không gian tìm kiếm, vẫn là điều quan trọng. Các thuật toán tìm kiếm chính xác (*exact search*) bảo đảm được tính tối ưu này sẽ được bàn thêm ở Mục 5.4.4.

Chú giải của người dịch: Vì sao $\theta_{\text{all}} = 0$ có thể đủ an toàn?

Nếu điểm chỉ là log xác suất (không có thưởng độ dài), thì nối thêm một từ luôn cộng thêm một số ≤ 0, nên điểm của một giả thuyết chỉ có thể *giảm* khi nó dài thêm. Do đó, nếu mọi giả thuyết đang dở dang đều đã có điểm không cao hơn giả thuyết hoàn chỉnh tốt nhất, thì không giả thuyết nào trong số chúng có thể vượt lên được nữa — dừng lại là không mất gì. Đây là ý tưởng đằng sau nhận xét rằng dừng sớm có thể bảo đảm tính tối ưu trong một số điều kiện.

5.4.3 Tìm kiếm trực tuyến

Cho đến giờ, trong phần thảo luận chung về các bài toán chuỗi-sang-chuỗi, ta đã giả định rằng mọi từ phía nguồn được quan sát dưới dạng một chuỗi hoàn chỉnh. Tuy nhiên, trong một số ứng dụng thực tế, đầu vào được nhận lần lượt, và ta muốn đưa ra dự đoán dựa trên các quan sát bộ phận. Một ví dụ là nhận dạng tiếng nói tự động trực tuyến (*online*), trong đó hệ thống liên tục nhận các tín hiệu âm thanh mới và đồng thời xuất ra các đơn vị phiên âm tương ứng.

Theo trực giác, ta có thể xem việc sinh từ đích thứ i là bài toán ánh xạ một tiền tố (*prefix*) của chuỗi phía nguồn sang một từ đích. Ta có thể phát biểu điều này bằng cách đưa vào hàm $g(i)$, biểu thị độ dài tối đa của tiền tố của $\mathbf{x}$ mà ta dùng khi sinh y_i. Như vậy, xác suất của y_i khi biết toàn bộ chuỗi $\mathbf{x}$ và các từ đã sinh trước đó $\mathbf{y}_{<i}$ có thể được xấp xỉ bởi

$$\Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) \;\approx\; \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}_{\leq g(i)}), \tag{5.88}$$

trong đó $\mathbf{x}_{\leq g(i)}$ ký hiệu chuỗi con $x_1 \dots x_{g(i)}$. Khi đó, log xác suất của chuỗi phía đích $\mathbf{y}$ khi biết chuỗi phía nguồn $\mathbf{x}$ được viết là

$$\begin{aligned}\log \Pr(\mathbf{y}|\mathbf{x}) &= \sum_{i=1}^{n} \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) \\ &\approx \sum_{i=1}^{n} \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}_{\leq g(i)}).\end{aligned} \tag{5.89}$$

Phương trình này phát biểu một bài toán chuỗi-sang-chuỗi thành một bài toán tiền tố-sang-tiền tố (*prefix-to-prefix*), tức là tiền tố $\mathbf{y}_{\leq i}$ chỉ phụ thuộc vào tiền tố $\mathbf{x}_{\leq g(i)}$. Suy luận cho mô hình này rất đơn giản. Với mỗi i, hệ thống tìm kiếm đợi cho đến khi nhận được $g(i)$ từ phía nguồn đầu tiên, rồi mở rộng các giả thuyết như thường lệ. Điều này có thể thực hiện bằng cách dùng lại các thuật toán mô tả ở tiểu mục trước. Ví dụ, ta có thể sửa thuật toán tìm kiếm chùm và thu được thuật toán tìm kiếm trực tuyến sau.

Thuật toán: Một thuật toán tìm kiếm chùm trực tuyến

OnlineBeamSearch($\mathbf{x}, g(\cdot)$)
// Tìm kiếm trực tuyến, được thực hiện mỗi khi đã nhận đủ số từ đầu vào.
// Ở mỗi bước tìm kiếm, xét một số lượng định trước các ứng viên hứa hẹn.

Tạo một Heap có sức chứa $size_{\text{heap}}$
$Z_0 = \{z_0\}$ với $z_0.tgt = y_0$
$j = 1$ ▷ chỉ số từ nguồn
$i = 1$ ▷ bước sinh phía đích
$input = \emptyset$
while $i \leq n_{\text{max}}$ **do**
  if $j \leq g(i)$ **then** ▷ đọc một từ từ luồng đầu vào
    $input = input \circ x_j$
    j++
  else ▷ dự đoán ở bước i khi đã quan sát được $g(i)$ từ đầu vào (lưu trong $input$)
    Tạo một đống có giới hạn Beam kích thước $size_{\text{beam}}$
    for mỗi giả thuyết $z_{\text{cur}} \in Z_{i-1}$ **do**
      for mỗi từ $v_k \in V_y$ **do**
        $z_{\text{next}} = \text{Extend}(z_{\text{cur}}, v_k, input)$
        if $input$ bằng $\mathbf{x}$ và $z_{\text{next}}.tgt$ kết thúc bằng $\langle\text{EOS}\rangle$ **then**
          Thêm z_{next} vào Heap
        else
          UpdateBeam(Beam, z_{next})
        end if
        if Heap đã đầy và/hoặc thỏa các tiêu chí dừng khác **then**
          Thoát khỏi tất cả các vòng lặp
        end if
      end for
    end for
    OutputPartial(Beam)
    $Z_i = \text{Beam.PopAll}()$
    i++
  end if
end while
return Heap.Pop()

OutputPartial(Beam)
// Xuất ra một kết quả bộ phận

Hiển thị giả thuyết tốt nhất trong Beam

Một ưu điểm của hệ thống này là đầu ra ở bước i có thể được tạo ra ngay khi ta đã thấy $\mathbf{x}_{\leq g(i)}$. Điều này cho ta một hệ thống chuỗi-sang-chuỗi trực tuyến, trong đó các từ đầu vào đến thành một luồng liên tục và dự đoán được sinh ra ngay khi đã quan sát đủ số từ đầu vào.

Dù bài toán tìm kiếm ở đây có vẻ đơn giản, vẫn còn nhiều việc phải làm để định nghĩa $g(i)$. Hiển nhiên, $g(i)$ là một hàm đơn điệu không giảm. Một ví dụ đơn giản là định nghĩa $g(i) = m$ với mọi i. Khi đó, thuật toán trên trở về đúng thuật toán tìm kiếm chùm tiêu chuẩn làm việc với một chuỗi đầu vào hoàn chỉnh. Ngược lại, trong các tác vụ chuỗi-sang-chuỗi trực tuyến, ta muốn $g(i)$ càng nhỏ càng tốt, để có thể bắt đầu tính toán càng sớm càng tốt khi suy luận. Trường hợp đơn giản nhất là khi chuỗi đầu vào và chuỗi đầu ra đồng bộ theo một cách nào đó. Ví dụ, một hệ thống nhận dạng tiếng nói tự động gán cho mỗi khung phổ (*spectral frame*) một đơn vị phiên âm. Trong trường hợp này, ta có một sự tương ứng đơn giản giữa đầu vào và đầu ra: $m = n$ (tức $|\mathbf{x}| = |\mathbf{y}|$), và x_i tương ứng với y_i. Khi đó, ta chỉ cần định nghĩa $g(i) = i$; nói cách khác, mỗi khi một đầu vào mới đến, ta đưa ra một dự đoán.

Một trường hợp phức tạp hơn là các bài toán chuỗi-sang-chuỗi trực tuyến có đảo trật tự (*reordering*), chẳng hạn dịch song song (*simultaneous translation*) (dịch đồng thời), trong đó một từ phía đích có thể phụ thuộc vào các từ phía nguồn ở xa. Một cách đơn giản để giải quyết điều này là trì hoãn các dự đoán thêm một số bước. Ví dụ, phương pháp wait-k (*wait-k*) buộc mỗi dự đoán phải đi chậm hơn đầu vào k từ [Ma và cộng sự, 2019]. Một cách hình thức hơn, phiên bản wait-k của hàm $g(i)$ được định nghĩa là

$$g(i) \;=\; \min(m, k + i - 1). \tag{5.90}$$

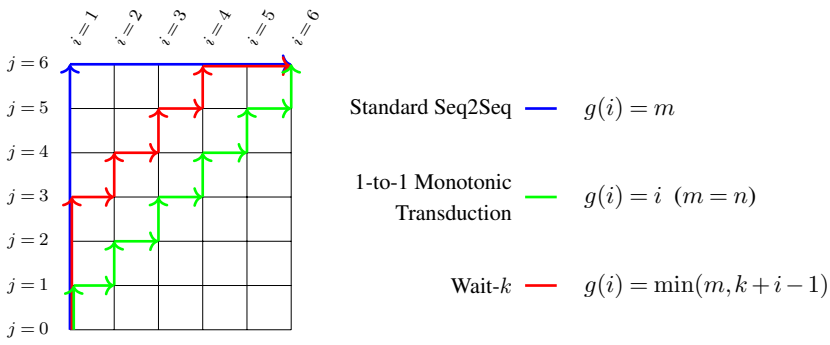
Ở đây k là một siêu tham số điều khiển độ lớn của ngữ cảnh phía nguồn được xét khi dự đoán các từ phía đích. Khi $k = \infty$, nó giống với các phương pháp tìm kiếm tiêu chuẩn cho suy luận chuỗi-sang-chuỗi. Trong dịch song song và các tác vụ liên quan, kết quả nhìn chung là thỏa đáng khi dùng một giá trị k nhỏ. Hình 5.11 so sánh các hàm $g(i)$ khác nhau.

Chú giải của người dịch: Wait-k bằng số cụ thể

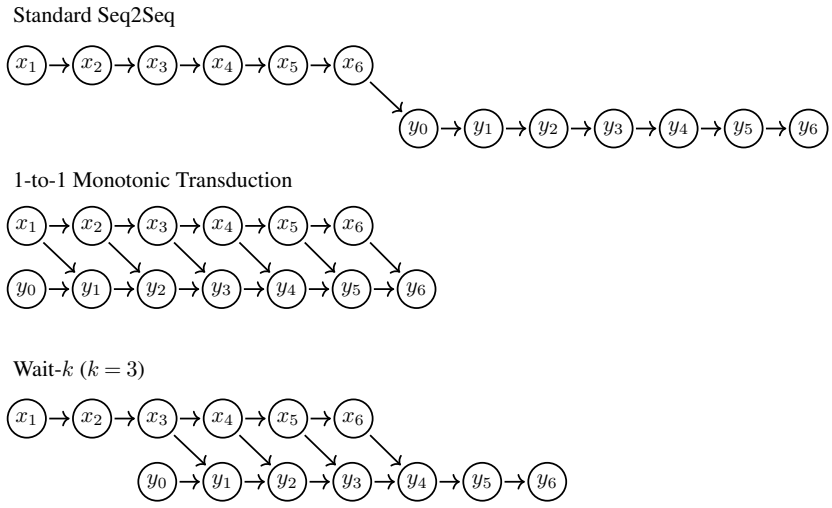
Với $m = 6$ từ nguồn và $k = 3$: $g(1) = \min(6, 3) = 3$, $g(2) = 4$, $g(3) = 5$, $g(4) = \min(6, 6) = 6$, $g(5) = \min(6, 7) = 6$, … Nghĩa là hệ thống đọc 3 từ nguồn rồi mới dịch từ đích đầu tiên, sau đó cứ đọc thêm một từ lại dịch thêm một từ; khi đã đọc hết câu nguồn thì dịch nốt phần còn lại. Đây chính là cách một phiên dịch viên "dịch đuổi" làm việc: không đợi diễn giả nói hết câu, nhưng cũng chờ vài từ để nắm ý trước khi nói. k nhỏ thì trễ ít nhưng dễ dịch sai (chưa đủ ngữ cảnh); k lớn thì ngược lại.

Trong một số ứng dụng của bài toán chuỗi-sang-chuỗi trực tuyến, ta có thể biết khi nào cần thực hiện tìm kiếm và khi nào cần đọc đầu vào. Ví dụ, trong dịch máy tương tác (*interactive machine translation*) [Casacuberta và cộng sự, 2009], việc dịch một chuỗi đầu vào bộ phận được kích hoạt trực tiếp bởi hành động của người dùng (ví dụ, nhấn phím). Trong trường hợp này, ta không cần định nghĩa $g(i)$ mà xem nó như một biến đầu vào của mô hình.

Lưu ý rằng mặc dù có thể dùng trực tiếp các mô hình mức chuỗi đã được tiền huấn luyện để suy luận trực tuyến, việc phát triển các hệ thống như vậy thường đòi hỏi thêm công sức huấn luyện. Một cách tiếp cận có cơ sở hơn cho mô hình hóa chuỗi-sang-chuỗi trực tuyến là mô hình hóa phép biến đổi từ $\mathbf{x}$ sang $\mathbf{y}$ như một chuỗi các hành động (*action*) [Grissom II và cộng sự, 2014; Cho và Esipova, 2016; Gu và cộng sự, 2017; Zheng và cộng sự, 2019]. Ví dụ, một hành động có thể là thao tác *dự đoán* (*predict*), thực hiện tìm kiếm ở bước hiện tại, hoặc thao tác *đọc* (*read*), nhận một từ đầu vào mới. Khi đó, ta có thể phát biểu nhiệm vụ thiết kế hàm $g(i)$ thành việc học một chính sách (policy) để quyết định hành động nào được thực hiện khi biết một tiền tố phía nguồn $\mathbf{x}_{\leq j}$ và một tiền tố phía đích $\mathbf{y}_{<i}$. Như vậy, các mô hình chuỗi-sang-chuỗi có thể được huấn luyện trên các trạng thái của những chuỗi hành động này để đưa ra dự đoán tốt hơn khi chỉ dựa trên một phần của đầu vào. Tuy nhiên, việc bàn về huấn luyện các mô hình chuỗi-sang-chuỗi trực tuyến nằm ngoài phạm vi của mục này. Chúng tôi giới thiệu bạn đọc các bài báo trên để biết thêm chi tiết về những phương pháp này.



(a) Visualization of $g(i)$.



(b) Action sequences.

Hình 5.11: Biểu diễn trực quan (trên) và các chuỗi hành động (dưới) của các hàm $g(i)$ khác nhau cho một cặp chuỗi ($\mathbf{x} = x_1 \dots x_6$, $\mathbf{y} = y_1 \dots y_6$). Trong một chuỗi hành động, x_j được khoanh tròn biểu thị hành động đọc một từ phía nguồn (x_j), và y_i được khoanh tròn biểu thị hành động dự đoán xác suất của y_i khi biết $\mathbf{x}_{\leq g(i)}$ và $\mathbf{y}_{<i}$. Các mũi tên ở đây biểu thị sự phụ thuộc giữa các từ. Vì y_0 ký hiệu ký hiệu bắt đầu $\langle$SOS$\rangle$, nó có thể được sinh ra mà không phụ thuộc vào từ nào. (Nhãn trong hình: (a) biểu diễn trực quan của $g(i)$; (b) các chuỗi hành động; *Standard Seq2Seq* = chuỗi-sang-chuỗi tiêu chuẩn; *1-to-1 Monotonic Transduction* = chuyển đổi đơn điệu một-một.)

5.4.4 Tìm kiếm chính xác

Từ góc độ hình thức, lý tưởng nhất là ta muốn xây dựng một hệ thống không có lỗi tìm kiếm. Mặc dù các thuật toán tìm kiếm xấp xỉ (*approximate search*) đã được dùng thành công trong nhiều ứng dụng, điều quan trọng là nghiên cứu lỗi mô hình (*model error*) của các hệ thống này, và vì thế tập trung vào bài toán về mặt nguyên lý chứ không chỉ về mặt thực tiễn. Do đó, phát triển các thuật toán tìm kiếm chính xác cho các mô hình chuỗi-sang-chuỗi từ lâu đã là một chủ đề thú vị trong nghiên cứu NLP. Tuy nhiên, bài toán tìm kiếm cho một hệ thống dịch máy dựa trên từ đơn giản với các mô hình ngôn ngữ n-gram đã được chứng minh là một bài toán NP-khó (*NP-hard*) [Knight, 1999]. Phần lớn các nghiên cứu trước đây đã phát biểu bài toán tìm kiếm thành các bài toán tối ưu tổ hợp (*combinatorial optimization*) kinh điển, như bài toán quy hoạch tuyến tính (*linear programming*) và bài toán người du lịch (*traveling salesman problem*), rồi dùng các bộ giải tương ứng [Germann và cộng sự, 2004; Zaslavskiy và cộng sự, 2009]. Các nỗ lực nghiên cứu khác đã khám phá các thuật toán tìm kiếm chính xác cho các hệ thống dịch máy thống kê bằng kỹ thuật nới lỏng Lagrange (*Lagrangian relaxation*) [Chang và Collins, 2011; Rush và Collins, 2012] và ôtômat hữu hạn trạng thái (*finite-state automata*) [de Gispert và cộng sự, 2010; Allauzen và cộng sự, 2014].

Chú giải của người dịch: NP-khó nghĩa là gì?

Nói nôm na, một bài toán là *NP-khó* nếu cho tới nay người ta chưa biết thuật toán nào giải *chính xác* nó trong thời gian tăng "vừa phải" (đa thức, như n^2 hay n^3) theo kích thước đầu vào, và nhiều người tin rằng không tồn tại thuật toán như vậy. Bài toán người du lịch (tìm đường ngắn nhất đi qua tất cả các thành phố, mỗi thành phố một lần) là ví dụ nổi tiếng: với 20 thành phố đã có khoảng 10^{17} lộ trình. Vì vậy, người ta thường chấp nhận lời giải xấp xỉ, hoặc tìm cách khai thác cấu trúc đặc biệt của bài toán như ở phần dưới đây.

Khác với các phương pháp này — vốn ít nhiều phụ thuộc vào việc tích hợp mô hình ngôn ngữ n-gram vào mô hình chuỗi-sang-chuỗi — các mô hình được mô tả trong chương này có dạng đơn giản hơn. Ta bắt đầu với một mô hình cơ bản, trong đó hàm tính điểm $\text{Score}(\mathbf{x}, \mathbf{y})$ là log xác suất $\log \Pr(\mathbf{y}|\mathbf{x})$. Phương trình (5.62) cho ta biết $\log \Pr(\mathbf{y}|\mathbf{x})$ có thể viết thành tổng các log xác suất mức từ, và $\log \Pr(\mathbf{y}|\mathbf{x})$ trở nên nhỏ hơn khi sinh thêm nhiều từ phía đích (tức n lớn hơn)[11]. Nói cách khác, $\log \Pr(\mathbf{y}|\mathbf{x})$ là một hàm đơn điệu giảm theo độ dài phía đích n: với mọi i, ta có

$$\begin{aligned} \log \Pr(\mathbf{y}_{\leq i}|\mathbf{x}) &= \log \Pr(\mathbf{y}_{<i}|\mathbf{x}) + \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) \\ &\leq \log \Pr(\mathbf{y}_{<i}|\mathbf{x}). \end{aligned} \tag{5.91}$$

Tính chất này còn được gọi là tính đơn điệu (*monotonicity*) của hàm tính điểm.

Khi đó, bằng cách tận dụng tính đơn điệu của điểm mô hình, ta có thể xây dựng một heuristic để loại bỏ những giả thuyết không bao giờ có thể là tốt nhất. Gọi $z_{\text{bestinall}}$ là giả thuyết hoàn chỉnh tốt nhất toàn cục tìm được cho đến lúc này. Nếu một giả thuyết bộ phận mới có điểm mô hình thấp hơn hẳn $z_{\text{bestinall}}.score$, nó không thể được mở rộng để vượt qua $z_{\text{bestinall}}$, nên ta không cần mở rộng nó. Điều này cho phép ta khám phá một vùng nhỏ hơn đáng kể so với không gian tìm kiếm ban đầu mà không hề mất tính tối ưu. Lưu ý rằng khi việc tìm kiếm tiến triển, $z_{\text{bestinall}}.score$ liên tục tăng. Việc tìm được một giả thuyết tốt hơn ngày càng khó, dẫn đến cắt tỉa ngày càng mạnh tay hơn khi việc tìm kiếm tiếp diễn. Mã giả dưới đây phác thảo một thuật toán tìm kiếm chính xác cho mô hình chuỗi-sang-chuỗi được định nghĩa bởi Phương trình (5.62).

Đây là một thuật toán tổng quát cho tìm kiếm chính xác, và hiệu quả tìm kiếm của nó chịu ảnh

[11] Xét $\log \Pr(\mathbf{y}|\mathbf{x}) = \sum_{i=1}^{n} \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x})$. Vì $\log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x})$ có giá trị không dương, $\log \Pr(\mathbf{y}|\mathbf{x})$ sẽ nhỏ đi hoặc không đổi khi n tăng.

Thuật toán: Một thuật toán tìm kiếm chính xác

ExactSearch($\mathbf{x}$)

*// Tìm giả thuyết "tốt nhất" bằng cách tận dụng tính đơn điệu của hàm tính điểm (*Score$(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}|\mathbf{x})$*).*

Tạo một hàng đợi ưu tiên (đống cực đại) Queue
Tạo một giả thuyết z_{best} với $z_{\text{best}}.score = -\infty$
while Queue chưa rỗng **do**
    $z_{\text{cur}} =$ Queue.Pop()
    if $|z_{\text{cur}}.tgt| > n_{\max}$ **then**
        bỏ qua z_{cur} và tiếp tục vòng lặp
    end if
    for mỗi từ $v_k \in V_y$ **do**
        $z_{\text{next}} =$ Extend$(z_{\text{cur}}, v_k, \mathbf{x})$
        $bound = z_{\text{best}}.score$ ▷ một cận dưới của điểm mô hình
        if $bound < z_{\text{next}}.score$ **then** ▷ cắt tỉa chấp nhận được
            if $z_{\text{next}}.tgt$ kết thúc bằng ⟨EOS⟩ **then**
                $z_{\text{best}} = z_{\text{next}}$
                $bound = z_{\text{next}}.score$
            else
                Thêm z_{next} vào Queue
            end if
        end if
    end for
end while
return z_{best}

hưởng lớn bởi thiết kế của hàng đợi ưu tiên (*priority queue*) [Meister và cộng sự, 2020]. Ví dụ, ta có thể xem Score$(\mathbf{x}, \mathbf{y})$ là độ ưu tiên của mỗi giả thuyết trong hàng đợi ưu tiên, như trong một đống cực đại[12]. Khi đó, thuật toán thu được thực hiện một thủ tục tìm kiếm gần giống tìm kiếm theo chiều rộng, vì một giả thuyết có chuỗi phía đích ngắn hơn thường có điểm mô hình cao hơn và dễ đứng đầu hàng đợi ưu tiên hơn. Tuy nhiên, để tìm kiếm hiệu quả, ta muốn tìm ra các giả thuyết hoàn chỉnh càng sớm càng tốt, sao cho nhiều giả thuyết kém hứa hẹn có thể bị loại bỏ ngay ở giai đoạn đầu của việc tìm kiếm. Để làm vậy, ta có thể thiên vị độ ưu tiên của một giả thuyết về phía chuỗi đích dài hơn. Điều này cho ta một thuật toán tìm kiếm theo chiều sâu (*depth-first search*), có nhiều khả năng tìm được giả thuyết hoàn chỉnh trong thời gian ngắn hơn [Stahlberg và Byrne, 2019].

Chú giải của người dịch: Ý tưởng "nhánh và cận"

Thuật toán trên giống trò chơi tìm đường rẻ nhất: nếu bạn đã biết một lộ trình hoàn chỉnh tốn 100 nghìn đồng, thì một lộ trình đang đi dở mà đã tốn 120 nghìn đồng chắc chắn không thể tốt hơn (vì đi tiếp chỉ tốn thêm), nên bỏ nó đi ngay. Ở đây, "chi phí" là $-\log \Pr$, luôn tăng khi thêm từ (Phương trình (5.91)). Biến $bound$ đóng vai trò "kỷ lục hiện tại"; kỷ lục càng tốt thì càng nhiều nhánh bị cắt. Vì vậy, tìm sớm một giả thuyết hoàn chỉnh tốt (nhờ tìm kiếm theo chiều sâu) giúp thuật toán chạy nhanh hơn rất nhiều mà vẫn cho kết quả chính xác.

Mặc dù thuật toán tìm kiếm chính xác trở nên rõ ràng khi xét tính đơn điệu của $\Pr(\mathbf{y}|\mathbf{x})$, trong các hệ thống thực tế, như đã bàn ở Mục 5.4.1, Score$(\mathbf{x}, \mathbf{y})$ thường có dạng phức tạp hơn, bao gồm thưởng độ dài hoặc chuẩn hóa độ dài, nên tính đơn điệu không còn đúng. May mắn là có thể bỏ giả định đơn

[12] Ta có thể hiện thực một hàng đợi ưu tiên bằng một đống cực đại.

điệu với cái giá là nới lỏng một chút cận dưới của điểm mô hình dùng để cắt tỉa. Ở đây ta định nghĩa $bound$ là điểm mô hình thấp nhất mà một giả thuyết cần có để trong trường hợp tốt nhất nó có thể được mở rộng thành một giả thuyết tốt ngang z_{best}. Ví dụ, xét mô hình thưởng từ đơn giản mô tả ở Phương trình (5.63): $\text{Score}(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}|\mathbf{x}) + \lambda \cdot n$. Với một giả thuyết z_{next}, có tối đa $n_{\max} - |z_{\text{next}}.tgt|$ từ mà ta có thể dự đoán để thu được một giả thuyết hoàn chỉnh. Giả sử tất cả $n_{\max} - |z_{\text{next}}.tgt|$ từ này đều được dự đoán với xác suất 1. Khi đó, điểm mô hình của giả thuyết thu được (ký hiệu là z_{new}) sẽ là

$$\begin{aligned} z_{\text{new}}.score &= z_{\text{next}}.score + \sum_{i=|z_{\text{next}}.tgt|+1}^{n_{\max}} (\log 1 + \lambda) \\ &= z_{\text{next}}.score + \lambda \cdot (n_{\max} - |z_{\text{next}}.tgt|). \end{aligned} \tag{5.92}$$

Dùng kết quả này, ta có thể định nghĩa $bound$ là

$$bound \;=\; z_{\text{best}}.score - \lambda \cdot (n_{\max} - |z_{\text{next}}.tgt|). \tag{5.93}$$

Một cách khác để suy ra cận dưới là chỉ cần trừ đi phần thưởng từ lớn nhất có thể, cho bởi

$$bound \;=\; z_{\text{best}}.score - \lambda \cdot n_{\max}. \tag{5.94}$$

Đây là một cận dưới lỏng và dẫn tới ít cắt tỉa hơn.

Trong trường hợp chuẩn hóa độ dài, ta có thể làm tương tự. Ví dụ, xét mô hình chuẩn hóa độ dài $\text{Score}(\mathbf{x}, \mathbf{y}) = \frac{\log \Pr(\mathbf{y}|\mathbf{x})}{n}$, như trong Phương trình (5.66–5.67). Một cận dưới cho các điểm mô hình chấp nhận được được cho bởi

$$bound \;=\; \frac{\Pr(z_{\text{next}}.tgt|\mathbf{x})}{n_{\max}}. \tag{5.95}$$

Trong thực tế, một cận dưới như vậy có thể được định nghĩa theo nhiều cách khác nhau để bảo đảm tính tối ưu của việc tìm kiếm, tùy thuộc vào mô hình và chiến lược tìm kiếm được dùng trong hệ thống chuỗi-sang-chuỗi [Huang và cộng sự, 2017b; Stahlberg và Byrne, 2019].

Ta có thể dễ dàng áp dụng các cận dưới này cho thuật toán tìm kiếm chính xác ở trên bằng cách thay dòng 9 bằng Phương trình (5.93) hoặc (5.95). Như một tác dụng phụ, việc tìm kiếm sẽ khám phá nhiều giả thuyết hơn và do đó chậm hơn nhiều.

Chú giải của người dịch: Vì sao phải "nới" cận khi có thưởng độ dài?

Khi có thưởng λ cho mỗi từ, một giả thuyết dở dang hiện đang thua kỷ lục vẫn có thể "lội ngược dòng" nhờ các từ thưởng sau này. Phương trình (5.92) tính kịch bản lạc quan nhất: mọi từ còn lại đều có xác suất 1 (nên $\log 1 = 0$, không mất điểm) và mỗi từ được cộng thêm λ. Ví dụ: $\lambda = 0.5$, $n_{\max} = 10$, giả thuyết đã có 4 từ; nó còn có thể được cộng tối đa $0.5 \cdot 6 = 3$ điểm. Vì vậy, chỉ khi nó thua kỷ lục *hơn* 3 điểm thì mới chắc chắn bỏ được — đó là Phương trình (5.93).

5.4.5 Tìm kiếm khả vi

Ta đã giải quyết bài toán tìm kiếm bằng cách đưa ra các thuật toán tìm kiếm heuristic, trong đó ta cố gắng cực đại hóa hàm tính điểm trên một tập các chuỗi biến rời rạc. Một cách khác là nới lỏng các biến rời rạc này thành biến liên tục và phát biểu bài toán thành một bài toán **tối ưu liên tục** (*continuous optimization*) [Hoang và cộng sự, 2017; Kumar và cộng sự, 2021]. Mặc dù chúng tôi cố gắng dùng ký hiệu nhất quán trong toàn bộ cuốn sách, ở đây sẽ tiện hơn nếu đưa vào một ký hiệu hơi khác so với các chương trước. Ta sẽ dùng vectơ $\mathbf{y}_i^w \in \{0, 1\}^{|V_y|}$ để ký hiệu biểu diễn one-hot (*one-hot representation*)

của y_i. Giả sử đầu ra ở bước i là một phân phối trên V_y, ký hiệu là $\Pr(\cdot|\mathbf{y}_{<i}, \mathbf{x})$. Khi đó, ta có thể biểu diễn log xác suất của y_i bằng tích vô hướng của hai vectơ:

$$\begin{aligned}\log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) &= \mathbf{y}_i^w \cdot \log \Pr(\cdot|\mathbf{y}_{<i}, \mathbf{x}) \\ &= \mathbf{y}_i^w \cdot \log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x}),\end{aligned} \tag{5.96}$$

trong đó $\mathbf{y}_{<i} = y_0 \dots y_{i-1}$ được biểu diễn thành một chuỗi các vectơ one-hot $\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w$. Như đã bàn ở Chương 3, vế phải của phương trình trên tương ứng với việc chọn phần tử y_i của vectơ $\log \Pr(\cdot|\mathbf{y}_{<i}, \mathbf{x})$ (hoặc $\log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x})$).

Với ký hiệu này, ta có thể viết $\log \Pr(\mathbf{y}|\mathbf{x})$ là

$$\begin{aligned}\log \Pr(\mathbf{y}|\mathbf{x}) &= \sum_{i=1}^{n} \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) \\ &= \sum_{i=1}^{n} \mathbf{y}_i^w \cdot \log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x}).\end{aligned} \tag{5.97}$$

Chú giải của người dịch: Vectơ one-hot "chọn" một phần tử

Giả sử bộ từ vựng chỉ có 3 từ {tôi, đi, học} và $\log \Pr(\cdot|\dots) = [-2.3, -0.1, -1.6]$. Nếu $y_i =$ "đi" thì vectơ one-hot là $\mathbf{y}_i^w = [0, 1, 0]$, và tích vô hướng $[0, 1, 0] \cdot [-2.3, -0.1, -1.6] = 0 \cdot (-2.3) + 1 \cdot (-0.1) + 0 \cdot (-1.6) = -0.1$ đúng bằng log xác suất của "đi". Nếu thay $[0, 1, 0]$ bằng một vectơ "mềm" như $[0.2, 0.7, 0.1]$ (các số không âm, tổng bằng 1), tích vô hướng cho $0.2 \cdot (-2.3) + 0.7 \cdot (-0.1) + 0.1 \cdot (-1.6) = -0.69$: đó là *giá trị trung bình có trọng số* (kỳ vọng) của log xác suất. Đây chính là ý tưởng "nới lỏng" ở phần dưới.

Nếu ta dùng $\log \Pr(\mathbf{y}|\mathbf{x})$ làm hàm mục tiêu (tức $\text{Score}(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}|\mathbf{x})$), bài toán tìm kiếm có thể được phát biểu là

$$\hat{\mathbf{y}}_0^w \dots \hat{\mathbf{y}}_n^w \;=\; \underset{\mathbf{y}_1^w \dots \mathbf{y}_n^w}{\arg\max} \sum_{i=1}^{n} \mathbf{y}_i^w \cdot \log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x}). \tag{5.98}$$

Điều này tương đương với dạng tiêu chuẩn của suy luận trong các mô hình chuỗi-sang-chuỗi, cho bởi

$$\begin{aligned}\hat{\mathbf{y}} &= \hat{y}_0 \dots \hat{y}_n \\ &= \underset{y_0 \dots y_n}{\arg\max} \Pr(y_0 \dots y_n|\mathbf{x}).\end{aligned} \tag{5.99}$$

Từ Phương trình (5.98), giờ ta có thể nới lỏng mỗi vectơ one-hot thành một vectơ giá trị thực với ràng buộc rằng tổng mọi phần tử của nó bằng 1, tức là

$$\mathbf{y}_i^w \in \mathbb{R}_+^{|V_y|} \tag{5.100}$$

$$\text{với điều kiện} \quad \|\mathbf{y}_i^w\|_1 = 1. \tag{5.101}$$

Bằng cách này, $\mathbf{y}_i^w$ có thể được xem một cách không chính thức là một biểu diễn $|V_y|$ chiều của y_i, dù nó có nhiều chiều hơn hẳn các vectơ nhúng dày đặc thường dùng trong NLP. Thay vì tương ứng với một từ cụ thể trong bộ từ vựng, $\mathbf{y}_i^w$ giờ mô tả một phân phối liên tục trên bộ từ vựng. Kết quả là số hạng $\mathbf{y}_i^w \cdot \log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x})$ tính **log xác suất kỳ vọng** (*expected log-probability*) của dự đoán ở bước i. Để làm cho phân phối có điều kiện $\Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x})$ khả vi theo tiền tố liên tục, Hoang và cộng sự [2017] thay thao tác tra bảng nhúng từ rời rạc tiêu chuẩn ở mỗi bước của bộ giải mã bằng một **vectơ nhúng kỳ vọng** (*expected embedding*), tức là một trung bình có trọng số của ma trận nhúng từ theo

vectơ xác suất $\mathbf{y}_{i-1}^w$. Điều thú vị là cách phát biểu này ngụ ý rằng Phương trình (5.98) định nghĩa một tác vụ "mới", trong đó ta cố gắng cực đại hóa một hàm mục tiêu liên tục (tức là tổng của n log xác suất kỳ vọng).

Ta có thể giải Phương trình (5.98) bằng các bộ công cụ tối ưu có sẵn. Vì $\mathbf{y}_i^w$ bị ràng buộc trên một **đơn hình** (*simplex*)[13], việc áp dụng các thuật toán **tối ưu có ràng buộc** (*constrained optimization*) tổng quát là khá trực tiếp. Ngoài ra, có thể dùng các thuật toán được thiết kế riêng để giải bài toán tối ưu với ràng buộc đơn hình.

Chú giải của người dịch: Đơn hình qua hình ảnh quen thuộc

Với $k = 1$, đơn hình là một đoạn thẳng nối $\mathbf{p}_0$ và $\mathbf{p}_1$; với $k = 2$ là một tam giác (kể cả miền trong); với $k = 3$ là một tứ diện. Các điểm $a_0\mathbf{p}_0 + a_1\mathbf{p}_1 + a_2\mathbf{p}_2$ với $a_i \geq 0$, $a_0 + a_1 + a_2 = 1$ chính là mọi điểm trong tam giác — giống công thức trọng tâm khi $a_0 = a_1 = a_2 = \frac{1}{3}$. Một vectơ xác suất trên bộ từ vựng (các số không âm, tổng bằng 1, như ràng buộc (5.100–5.101)) chính là một điểm trên đơn hình có $|V_y|$ đỉnh; mỗi vectơ one-hot là một *đỉnh* của nó. "Nới lỏng" nghĩa là cho phép đi vào bên trong đơn hình thay vì chỉ nhảy giữa các đỉnh, nhờ đó có thể dùng đạo hàm để tối ưu.

Cách tiếp cận thứ ba để giải Phương trình (5.98) là đưa tường minh các ràng buộc vào hàm mục tiêu và dùng các phương pháp hạ gradient để tối ưu hàm này. Ví dụ, Hoang và cộng sự [2017] sửa Phương trình (5.98) và thu được một dạng mới để tối ưu

$$\hat{\mathbf{y}}_0^w \dots \hat{\mathbf{y}}_n^w = \underset{\mathbf{y}_1^w \cdots \mathbf{y}_n^w}{\arg\max} \sum_{i=1}^{n} \text{Softmax}(\mathbf{y}_i^w) \cdot \log \Pr(\cdot | \mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x}). \tag{5.105}$$

Ở đây ta bỏ ràng buộc đơn hình trên $\mathbf{y}_i^w$ và thay vào đó áp đặt nó thông qua đầu ra của Softmax.

Khi đã thu được chuỗi tối ưu $\hat{\mathbf{y}}_0^w \dots \hat{\mathbf{y}}_n^w$, ta cần ánh xạ mỗi $\mathbf{y}_i^w$ về một từ. Một cách đơn giản là chọn từ tương ứng với phần tử có giá trị lớn nhất của $\mathbf{y}_i^w$. Tuy nhiên, điều này có thể phá vỡ tính tối ưu của lời giải, vì điều kiện $\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w$ bị thay đổi khi các biến này được rời rạc hóa. Một cách thực tế hơn là thực hiện tối ưu để dự đoán từ tiếp theo khi biết một tiền tố; ví dụ, ta cố định $\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w$ bằng các biểu diễn one-hot của tiền tố tối ưu, và cực đại hóa $\sum_{k=i}^{n} \mathbf{y}_k^w \cdot \log \Pr(\cdot | \mathbf{y}_0^w \dots \mathbf{y}_{k-1}^w, \mathbf{x})$. Sau đó, ta chọn từ tốt nhất ở vị trí i và chuyển sang vị trí tiếp theo.

Cho đến giờ, ta đã giả định rằng mục tiêu tìm kiếm được suy ra từ log xác suất $\log \Pr(\mathbf{y}|\mathbf{x})$ và độ dài đầu ra được xác định trước. Để tìm kiếm trên các chuỗi có độ dài khác nhau, ta có thể lặp lại thủ tục tối ưu trên cho mọi $n \in [1, n_{\text{max}}]$ và chọn chuỗi có điểm cao nhất. Cách làm này cũng giúp dễ dàng tích hợp chuẩn hóa độ dài và thưởng độ dài vào việc tìm kiếm. Cụ thể, ta có thể bỏ qua thiên lệch độ dài trong lúc tối ưu với n cố định và áp dụng các mô hình độ dài sau đó (*post-hoc*). Tức là, ta giữ nguyên mục tiêu tìm kiếm ban đầu và, ở bước cuối cùng, chọn ứng viên tốt nhất trên mọi độ dài dựa trên một $\text{Score}(\mathbf{x}, \mathbf{y})$ đã được chuẩn hóa độ dài.

[13] Đơn hình là một khái niệm hình học. Trong không gian Euclid, một k-đơn hình là một đa diện k chiều được mô tả bởi một tập $k + 1$ điểm độc lập $\{\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_k\}$. Đa diện này được định nghĩa là tập các điểm

$$P_{k\text{-simplex}} = \{a_0 \cdot \mathbf{p}_0 + a_1 \cdot \mathbf{p}_1 + \dots + a_k \cdot \mathbf{p}_k\}, \tag{5.102}$$

trong đó

$$\sum_{i=0}^{k} a_i = 1, \tag{5.103}$$

$$a_i \geq 0 \quad \text{với mọi } i \in [0, k]. \tag{5.104}$$

5.4.6 Tính đa dạng của giả thuyết

Việc sinh ra nhiều đầu ra thường cần thiết khi ta muốn chấm điểm lại (*rescore*) các đầu ra này và/hoặc tương tác với hệ thống. Một trong những phương pháp được dùng rộng rãi nhất là dùng tìm kiếm chùm để sinh ra các giả thuyết xếp hạng đầu. Ví dụ, ta có thể đơn giản xem các phần tử của Heap là k giả thuyết tốt nhất (*k-best*) trong tìm kiếm chùm (xem Mục 5.4.2). Tuy nhiên, cách này thường tạo ra các giả thuyết rất giống nhau, và khó xác định cái nào tốt hơn ngay cả khi có nhiều lựa chọn hơn. Bảng 5.3 cho thấy 4 đầu ra tốt nhất của một hệ thống tóm tắt văn bản (*text summarization*). Ta thấy các văn bản này khá giống nhau. Một lý do của hiện tượng này là các giả thuyết đa dạng, kể cả những giả thuyết cuối cùng có thể đạt điểm mô hình cao khi hoàn chỉnh, sẽ bị cắt tỉa nếu chúng rơi ra khỏi chùm ở bất kỳ giai đoạn trung gian nào của việc tìm kiếm. Từ góc độ mô hình hóa, ta có thể hiểu điều này là một hạn chế cố hữu của các mô hình chuẩn hóa cục bộ (*locally normalized*) được dùng ở đây: mọi dự đoán đều được đưa ra ở một bước trung gian của việc tìm kiếm, và không có cơ chế nào để sửa các quyết định trong quá khứ một khi một tiền tố đã được chốt [Murray và Chiang, 2018].

Bảng 5.3: 4 đầu ra tốt nhất của một hệ thống tóm tắt văn bản trên một mẫu trong bộ dữ liệu CNN/Daily Mail (kích thước chùm $= 4$). Ta thấy các văn bản này chỉ khác nhau vài từ. (Người dịch giữ nguyên văn tiếng Anh và in nghiêng phần khác biệt; bản dịch nghĩa ở dưới.)

Hạng	Đầu ra
1	Manuela Arbelaez accidentally revealed the correct answer to a guessing game for a new Hyundai Sonata. Host Drew Carey couldn't stop laughing. It's been a busy week for "The Price Is Right" when Bob Barker*, 91,* showed up to run *his old* show.
2	Manuela Arbelaez accidentally revealed the correct answer to a guessing game for a new Hyundai Sonata. Host Drew Carey couldn't stop laughing. It's been a busy week for "The Price Is Right" when Bob Barker showed up to run *his old* show.
3	Manuela Arbelaez accidentally revealed the correct answer to a guessing game for a new Hyundai Sonata. Host Drew Carey couldn't stop laughing. It's been a busy week for "The Price Is Right" when Bob Barker*, 91,* showed up to run *the* show.
4	Manuela Arbelaez accidentally revealed the correct answer to a guessing game for a new Hyundai Sonata. Host Drew Carey couldn't stop laughing. It's been a busy week for "The Price Is Right" when Bob Barker*, 91,* showed up to run *his* show.

Dịch nghĩa (hạng 1): Manuela Arbelaez vô tình để lộ đáp án đúng của một trò chơi đoán giá để giành chiếc Hyundai Sonata mới. Người dẫn chương trình Drew Carey không nhịn được cười. Đó là một tuần bận rộn của chương trình "The Price Is Right" khi Bob Barker, 91 tuổi, xuất hiện để dẫn lại chương trình cũ của mình. **Khác biệt:** hạng 2 bỏ "91 tuổi"; hạng 3 viết "dẫn chương trình" thay vì "dẫn chương trình cũ của mình"; hạng 4 viết "dẫn chương trình của mình".

Một cách để cải thiện tính đa dạng của giả thuyết là đưa vào các hình phạt khi các giả thuyết trong chùm thiếu sự khác biệt [Li và Jurafsky, 2016; Vijayakumar và cộng sự, 2018]. Một hàm mục tiêu đơn giản được cho bởi

$$\text{Score}_d(\mathbf{x}, \mathbf{y}) \;=\; \text{Score}(\mathbf{x}, \mathbf{y}) - \lambda \cdot dp. \tag{5.106}$$

Nó kết hợp điểm mô hình gốc $\text{Score}(\mathbf{x}, \mathbf{y})$ với một hình phạt đa dạng (*diversity penalty*) dp. dp có thể được định nghĩa theo nhiều cách. Một cách là phạt các giả thuyết nằm gần nhau trên cây tìm kiếm. Ví dụ, có thể định nghĩa dp là thứ hạng của một giả thuyết trong tập các "anh em" của nó, tức các giả thuyết được mở rộng từ cùng một giả thuyết cha, nhờ đó chùm có thể trải các thành viên của nó trên một vùng rộng hơn của không gian giả thuyết [Li và Jurafsky, 2016]. Một cách khác để đưa vào các thước đo đa dạng là xét sự khác biệt giữa các chuỗi phía đích của các giả thuyết trong chùm. Ví dụ, ta có thể định nghĩa dp là độ tương đồng xâu trung bình giữa một giả thuyết cho trước và các giả thuyết khác trong chùm [Xiao và cộng sự, 2013].

Ý tưởng trên cũng có thể được biểu diễn thành các ràng buộc áp đặt lên thủ tục tìm kiếm. Ví dụ, ta có thể ràng buộc chùm chỉ chứa những giả thuyết bắt nguồn từ các giả thuyết cha khác nhau ở bước trước [Boulanger-Lewandowski và cộng sự, 2013]. Chính xác hơn, với mỗi giả thuyết $z_{\text{cur}} \in Z_{i-1}$, ta chọn giả thuyết tốt nhất cho bước tiếp theo như sau

$$\hat{z}_{\text{next}} = \underset{z_{\text{next}} \in \text{Extend}(z_{\text{cur}}, V_y)}{\arg\max} \Pr(z_{\text{next}}.tgt|\mathbf{x}). \tag{5.107}$$

Giả thuyết $\hat{z}_{\text{next}}$ sau đó được thêm vào Z_i. Lưu ý rằng về bản chất đây là một phương pháp không gian con (*subspace method*): chia không gian giả thuyết thành các không gian con và thu thập kết quả trên các không gian con này. Ý tưởng trực giác đằng sau phương pháp là các không gian con khác nhau có thể mô tả các khía cạnh khác nhau của bài toán, và nhờ đó ta có được các lời giải đa dạng.

Một cách tiếp cận khác để giải quyết vấn đề đa dạng là nhiễu loạn (*perturb*) tìm kiếm chùm bằng cách đưa các giả thuyết được sinh ngẫu nhiên vào chùm [Holtzman và cộng sự, 2020a; Wiher và cộng sự, 2022]. Một cách phổ biến để làm điều này là chọn một số từ ngẫu nhiên để mở rộng một giả thuyết, rồi thêm các giả thuyết đã mở rộng vào chùm. Nói chung, các từ này có thể được lấy mẫu từ phân phối $\Pr(\cdot|\mathbf{y}_{<i}, \mathbf{x})$ trên toàn bộ bộ từ vựng hoặc một tập con của nó. Tính ngẫu nhiên cũng có thể được thêm vào đầu vào của hệ thống lúc kiểm tra. Ví dụ, một từ đầu vào có thể được biểu diễn thành một tổ hợp tuyến tính của vectơ nhúng gốc của nó và vectơ nhúng của một từ được lấy mẫu từ một chuỗi huấn luyện chọn ngẫu nhiên [Li và cộng sự, 2021b]. Trong các bài toán có nhiều cực tiểu địa phương (*local minima*), thêm nhiễu ngẫu nhiên vào quá trình tìm kiếm nhìn chung là có ích, vì ta có thể khám phá các giả thuyết đa dạng hơn và tránh để hệ thống bị mắc kẹt ở một số vùng nhất định của không gian tìm kiếm.

Thay vì thực hiện tìm kiếm bằng một hệ thống duy nhất, ta có thể dùng nhiều hệ thống để thu được các giả thuyết đa dạng. Các hệ thống này có thể được xây dựng trên các kiến trúc khác nhau hoặc các cấu trúc/cấu hình ẩn khác nhau [He và cộng sự, 2018; Shen và cộng sự, 2019; Wu và cộng sự, 2020a; Sun và cộng sự, 2020a]. Mặc dù những phương pháp này không thuộc khung tìm kiếm mà ta đang bàn, việc kết hợp kết quả từ nhiều hệ thống nhìn chung là có ích. Tiểu mục sau sẽ bàn về vấn đề này.

Chú giải của người dịch: Vì sao cần đa dạng?

Hãy hình dung bạn nhờ 4 người bạn gợi ý quà sinh nhật, nhưng cả 4 cùng gợi ý "một chiếc áo phông xanh", chỉ khác cỡ áo — có 4 gợi ý mà như chỉ có 1. Bảng 5.3 cho thấy tình huống tương tự: 4 bản tóm tắt chỉ khác nhau vài từ. Hình phạt *dp* trong Phương trình (5.106) trừ điểm những ứng viên "na ná" các ứng viên khác, còn Phương trình (5.107) buộc mỗi giả thuyết cha chỉ được "gửi" một con tốt nhất vào chùm, nên các thành viên của chùm đến từ các nhánh khác nhau của cây tìm kiếm.

5.4.7 Kết hợp nhiều mô hình

Trong học máy, kết hợp mô hình (*ensembling*) (ensemble) chỉ các phương pháp nhằm giải quyết những hạn chế về mô hình hóa chứ không phải các vấn đề về tìm kiếm. Trong tiểu mục này, chúng tôi bàn về các phương pháp này vì việc hiện thực chúng thường đòi hỏi sửa đổi các mô-đun tìm kiếm, và ta có thể hiểu sâu hơn về hệ thống thu được khi nhìn nó từ góc độ tìm kiếm.

Các phương pháp kết hợp mô hình nhằm cải thiện hiệu năng dự đoán bằng cách kết hợp đầu ra của nhiều hệ thống thành phần (*constituent/component system*). Bài toán kết hợp nhiều mô hình đã được bàn luận rộng rãi kể từ khi các mô hình thống kê xuất hiện trong NLP, nơi các kỹ thuật này thường được gọi là các phương pháp **kết hợp hệ thống** (*system combination*) để nhấn mạnh tính hữu dụng thực tiễn của chúng. Với các mô hình chuỗi-sang-chuỗi được bàn ở đây, một dạng kết hợp hệ thống được dùng

rộng rãi là **lấy trung bình dự đoán** (*prediction averaging*) [Sutskever và cộng sự, 2014]. Giả sử ta có một tập hợp K mô hình chuỗi-sang-chuỗi đã huấn luyện. Log xác suất sinh từ phía đích y_i, khi biết ngữ cảnh bên trái $\mathbf{y}_{<i}$ và chuỗi phía nguồn $\mathbf{x}$, có thể được định nghĩa bằng trung bình nhân:

$$\log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) \;=\; \frac{1}{K}\sum_{k=1}^{K} \log \Pr_k(y_i|\mathbf{y}_{<i}, \mathbf{x}), \tag{5.108}$$

hoặc bằng trung bình cộng

$$\log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) \;=\; \log \frac{1}{K}\sum_{k=1}^{K} \Pr_k(y_i|\mathbf{y}_{<i}, \mathbf{x}), \tag{5.109}$$

trong đó $\Pr_k(y_i|\mathbf{y}_{<i}, \mathbf{x})$ là đầu ra của hệ thống thành phần thứ k. Các dạng này đơn giản đến mức có thể hiện thực cho bất kỳ mô hình chuỗi-sang-chuỗi nào mà không cần sửa đổi đáng kể các hệ thống hiện có, và chúng đã được dùng làm nền tảng cho nhiều hệ thống thành công trong các cuộc thi đánh giá khác nhau [Barrault và cộng sự, 2020; Akhbardeh và cộng sự, 2021].

Chú giải của người dịch: Trung bình nhân và trung bình cộng

Giả sử có $K = 2$ mô hình, cho từ y_i các xác suất 0.9 và 0.1. Trung bình cộng: $\frac{0.9+0.1}{2} = 0.5$. Trung bình nhân: $\sqrt{0.9 \cdot 0.1} = 0.3$ (Phương trình (5.108) chính là log của trung bình nhân, xem Phương trình (5.68–5.69)). Trung bình nhân "khắt khe" hơn: chỉ cần một mô hình cho xác suất rất thấp là kết quả bị kéo xuống mạnh — giống như cần mọi giám khảo đồng thuận. Trung bình cộng "dễ dãi" hơn: một mô hình rất tự tin có thể bù cho mô hình kia.

Một hạn chế của việc lấy trung bình dự đoán là mọi hệ thống thành phần phải tuân theo cùng một cách phát biểu tự hồi quy cơ bản (xem Phương trình (5.62)), và ta cần truy cập trực tiếp vào các xác suất đầu ra theo từng bước $\{\Pr_k(y_i|\mathbf{y}_{<i}, \mathbf{x})\}$ của chúng. Khi làm việc với các hệ thống hộp đen (*black-box*), ta buộc phải thực hiện kết hợp mức chuỗi (*sequence-level ensembling*). Một chiến lược phổ biến là tổng hợp các chuỗi hoàn chỉnh do các hệ thống thành phần tạo ra. Cách trực tiếp nhất là chọn giả thuyết (*hypothesis selection*) [Hildebrand và Vogel, 2008], trong đó ta đơn giản chọn ra một chuỗi "tốt nhất" từ tập các chuỗi đã sinh dựa trên một tiêu chí định trước. Một cách khác là sinh lại một chuỗi mới, có thể khác với mọi đầu ra riêng lẻ trong tập [Matusov và cộng sự, 2006; Rosti và cộng sự, 2007]. Điều này thường đòi hỏi nén các chuỗi đã sinh thành một cấu trúc thống nhất gọn nhẹ (như một lưới từ (*word lattice*)), tiếp theo là một lượt tìm kiếm bổ sung (ví dụ, giải mã trên lưới và chấm điểm lại) để trích ra chuỗi đồng thuận tối ưu [Deoras và cộng sự, 2011; Stahlberg và cộng sự, 2016; Khayrallah và cộng sự, 2017].

Lưu ý rằng việc kết hợp các mô hình chuỗi-sang-chuỗi có liên quan tới vấn đề đa dạng đã bàn ở tiểu mục trước. Người ta thừa nhận rộng rãi rằng các hệ thống thành phần phải đủ đa dạng thì việc kết hợp mới mang lại lợi ích đáng kể. Vì vậy, các mô hình nên được huấn luyện hoặc xây dựng sao cho nắm bắt các kiểu hành vi khác nhau [Sutskever và cộng sự, 2014; Zhou và cộng sự, 2017]. Một trong những phương pháp phổ biến nhất là kết hợp các điểm lưu (*checkpoint ensembling*). Nó dùng nhiều "ảnh chụp" của cùng một mô hình tại các điểm lưu (*checkpoint*) khác nhau trong quá trình huấn luyện và kết hợp các dự đoán của chúng bằng cách lấy trung bình. Phương pháp này có thể hữu ích để giảm nhẹ vấn đề quá khớp trong thực tế. Ngoài ra, có thể tạo ra các mô hình khác nhau từ một mô hình cơ sở với các thiết lập khác nhau. Ví dụ, ta có thể xây dựng các mô hình có số tham số khác nhau dựa trên một mô hình xương sống (*backbone*) chung. Một cách tổng quát hơn là dùng các mô hình dựa trên những kiến trúc khác nhau, dù điều này phải trả giá bằng nhiều công sức phát triển hơn.

Một cách nhìn khác về kết hợp mức chuỗi là nó tạo ra một sơ đồ tìm kiếm hai lượt. Ở lượt thứ nhất, nhiều hệ thống độc lập thực hiện suy luận. Vì mỗi hệ thống có những thiên kiến quy nạp riêng về mô hình hóa và giải mã, chúng tự nhiên khám phá những vùng khác nhau của không gian tìm kiếm. Một giả thuyết chất lượng cao được một hệ thống phát hiện có thể bị hệ thống khác bỏ lỡ hoàn toàn. Kết quả của lượt thứ nhất là một tập giả thuyết đa dạng, mỗi giả thuyết được xem là "tối ưu" theo một góc nhìn cụ thể. Ở lượt thứ hai, tập đa dạng này xác định một không gian tìm kiếm có ràng buộc và chất lượng cao, trên đó một mô hình chấm điểm lại tinh vi có thể "điều hướng" để xác định kết quả cuối cùng, tối ưu toàn cục.

5.4.8 Thêm các mục tiêu tìm kiếm khác

Trong tiểu mục này, ta xét thêm các hàm mục tiêu khác có thể áp dụng cho bài toán tìm kiếm.

1. Tìm kiếm với điểm tương lai

Phần lớn các thuật toán mô tả ở trên có thể được xem là các biến thể của thuật toán tìm kiếm tốt nhất trước (*best-first search*) [Meister và cộng sự, 2020]. Tìm kiếm A* (*A* search*) là một ví dụ khác của thuật toán tìm kiếm tốt nhất trước và được coi rộng rãi là một giải pháp tốt cho bài toán tìm kiếm tổng quát. Tìm kiếm A* nguyên bản đòi hỏi sắp xếp mọi trạng thái tìm kiếm ở mỗi bước, điều không khả thi về tính toán trong nhiều bài toán NLP. Do đó, ta tiếp tục tập trung vào tìm kiếm chùm và tìm kiếm tham lam, nhưng dùng một hàm mục tiêu kiểu A*. Cụ thể, cho một trạng thái tìm kiếm $(\mathbf{x}, \mathbf{y}_{\leq i})$, hàm mục tiêu kiểu A* có thể được định nghĩa là:

$$\text{Score}_{\text{A}^*}(\mathbf{x}, \mathbf{y}_{\leq i}) \;=\; g(\mathbf{x}, \mathbf{y}_{\leq i}) + h(\mathbf{x}, \mathbf{y}_{\leq i}). \tag{5.110}$$

Ở đây $g(\mathbf{x}, \mathbf{y}_{\leq i})$ là phần thưởng tích lũy từ trạng thái bắt đầu đến $(\mathbf{x}, \mathbf{y}_{\leq i})$, còn $h(\mathbf{x}, \mathbf{y}_{\leq i})$ là phần thưởng ước lượng của đường đi "tối ưu" từ $(\mathbf{x}, \mathbf{y}_{\leq i})$ đến đích cuối cùng. Vì $g(\mathbf{x}, \mathbf{y}_{\leq i})$ và $h(\mathbf{x}, \mathbf{y}_{\leq i})$ có thể có dạng tùy ý, khung này rất tổng quát. Ví dụ, nếu ta định nghĩa

$$g(\mathbf{x}, \mathbf{y}_{\leq i}) = \text{Score}(\mathbf{x}, \mathbf{y}_{\leq i}), \tag{5.111}$$

$$h(\mathbf{x}, \mathbf{y}_{\leq i}) = 0, \tag{5.112}$$

thì $\text{Score}_{\text{A}^*}(\mathbf{x}, \mathbf{y})$ trở về các hàm mục tiêu đã bàn trước đó.

Để tận dụng đầy đủ cách phát biểu này, việc tìm một hàm ước lượng phần thưởng hay chi phí tương lai là điều tự nhiên. Lý tưởng nhất, ta muốn $h(\mathbf{x}, \mathbf{y}_{\leq i})$ tính chính xác lượng phần thưởng bổ sung có thể thu được khi mở rộng $(\mathbf{x}, \mathbf{y}_{\leq i})$ thành giả thuyết hoàn chỉnh tối ưu. Tuy nhiên, điều này không khả thi vì nó đòi hỏi khám phá mọi giả thuyết có thể mở rộng từ $(\mathbf{x}, \mathbf{y}_{\leq i})$ để tìm ra cái tốt nhất. Trong thực tế, người ta thường dùng một mô hình rẻ hơn về tính toán để xấp xỉ phần thưởng tương lai thực sự. Các cách tiếp cận truyền thống dựa vào heuristic để định nghĩa $h(\mathbf{x}, \mathbf{y}_{\leq i})$ [Koehn và cộng sự, 2007], chẳng hạn ước lượng đóng góp của các từ có thể được sinh ra. Các heuristic này có thể được điều chỉnh dựa trên thiết kế kiến trúc của mô hình chuỗi-sang-chuỗi [He và cộng sự, 2017; Zheng và cộng sự, 2018]. Một cách tiếp cận có cơ sở hơn là dùng cách phát biểu dựa trên giá trị (*value-based*) [Ren và cộng sự, 2017; Li và cộng sự, 2017a; Leblond và cộng sự, 2021]. Ta có thể xây dựng một chính sách (policy) dự đoán phân phối của y_i khi biết $\mathbf{x}$ và $\mathbf{y}_{<i}$, cùng với một hàm giá trị (*value function*) ước lượng các phần thưởng tương lai. Do đó, Phương trình (5.110) có thể được hiểu là một tổ hợp tuyến tính của điểm chính sách cho $(\mathbf{x}, \mathbf{y}_{\leq i})$ và giá trị tương ứng của nó. Cách xử lý mục tiêu tìm kiếm như vậy thuộc khung tìm kiếm dựa trên giá trị (*value-based search*) và đã được dùng thành công trong học tăng cường [Silver và cộng sự, 2017].

Chú giải của người dịch: Ý tưởng của A*

Hãy nghĩ đến việc tìm đường bằng ứng dụng bản đồ. g là quãng đường bạn đã đi được (đã biết chắc), còn h là ước lượng quãng đường còn lại (ví dụ, khoảng cách đường chim bay đến đích). Chọn bước đi theo $g + h$ thay vì chỉ theo g giúp tránh những ngả đường "khởi đầu đẹp" nhưng dẫn ra xa đích. Trong sinh văn bản, h ước lượng "câu này nếu viết tiếp thì sẽ được thêm/bớt bao nhiêu điểm", giúp không loại bỏ quá sớm những giả thuyết đang có điểm thấp nhưng có triển vọng.

2. Tìm kiếm với mô hình ngôn ngữ

Từ lâu, các mô hình ngôn ngữ đã đóng vai trò trung tâm trong các tác vụ sinh văn bản. Ví dụ, các hệ thống dịch máy thống kê và nhận dạng tiếng nói tự động thường dựa vào các mô hình ngôn ngữ n-gram lớn để tạo ra văn bản trôi chảy. Mặc dù các mô hình chuỗi-sang-chuỗi hiện đại không cần mô hình ngôn ngữ riêng, việc tích hợp chúng trong quá trình tìm kiếm chuỗi-sang-chuỗi vẫn hấp dẫn về mặt trực giác đối với dịch máy và các bài toán liên quan.

Theo truyền thống coi mô hình ngôn ngữ như một đặc trưng của một mô hình log-tuyến tính (hoặc tuyến tính) [Och và Ney, 2002], mục tiêu có bổ sung mô hình ngôn ngữ có thể được định nghĩa là

$$\text{Score}_{\text{lm}}(\mathbf{x}, \mathbf{y}) \;=\; \log \Pr(\mathbf{y}|\mathbf{x}) + \lambda \cdot \log \Pr(\mathbf{y}). \tag{5.113}$$

Cách phát biểu này không bao gồm các số hạng thưởng độ dài và chuẩn hóa độ dài, nhưng cả hai đều có thể dễ dàng được đưa vào như các đặc trưng bổ sung. Nói chung, mô hình ngôn ngữ $\Pr(\mathbf{y})$ chỉ được huấn luyện trên các chuỗi phía đích, cho phép dùng dữ liệu đơn ngữ (*monolingual*) quy mô lớn trong các mô hình chuỗi-sang-chuỗi [Gulcehre và cộng sự, 2017]. Điều thú vị là người ta nhận thấy các mô hình chuỗi-sang-chuỗi hiện nay tự bản thân chúng đã là những mô hình ngôn ngữ mạnh nếu được huấn luyện đủ, và một chiến lược hiệu quả hơn để tận dụng dữ liệu phía đích là tạo ra các kho ngữ liệu tổng hợp, một kỹ thuật được gọi là **tăng cường dữ liệu** (*data augmentation*). Một ví dụ là **dịch ngược** (*back-translation*), trong đó một hệ thống dịch từ phía đích sang phía nguồn dịch các câu đơn ngữ phía đích để sinh ra các câu nguồn tổng hợp. Dữ liệu song ngữ tổng hợp này sau đó được dùng để bổ sung cho tập huấn luyện [Sennrich và cộng sự, 2016a; Edunov và cộng sự, 2018]. Trong nhiều tác vụ, phương pháp đơn giản này có thể cải thiện đáng kể chất lượng bản dịch, nhưng những kết quả này đặt ra câu hỏi về sự cần thiết của việc dùng thêm mô hình ngôn ngữ trong dịch máy nơ-ron.

Lưu ý rằng mô hình ở Phương trình (5.113) phụ thuộc vào việc chọn hệ số λ. Với dịch máy, ta thường quan tâm tới giá trị dương của λ để hệ thống có thể tạo ra đầu ra trôi chảy hơn. Ngược lại, giá trị âm của λ nghĩa là ta muốn các đầu ra ít phổ biến hơn. Ví dụ, nếu $\lambda = -1$, Phương trình (5.113) có thể được viết thành thông tin tương hỗ theo điểm (*point-wise mutual information*) của $\mathbf{x}$ và $\mathbf{y}$

$$\begin{aligned} \text{Score}_{\text{lm}}(\mathbf{x}, \mathbf{y}) &= \log \Pr(\mathbf{y}|\mathbf{x}) - \log \Pr(\mathbf{y}) \\ &= \log \frac{\Pr(\mathbf{x}, \mathbf{y})}{\Pr(\mathbf{x}) \cdot \Pr(\mathbf{y})}. \end{aligned} \tag{5.114}$$

Hàm tính điểm này đã được chứng tỏ là hữu ích để sinh ra các đầu ra đa dạng hơn cho các hệ thống hội thoại nơ-ron [Li và cộng sự, 2016].

Chú giải của người dịch: Vì sao $\lambda = -1$ giúp tránh câu trả lời nhàm chán?

Chatbot thường hay trả lời "Tôi không biết" vì câu này có xác suất cao trong *mọi* ngữ cảnh. Trừ đi $\log \Pr(\mathbf{y})$ nghĩa là phạt những câu vốn phổ biến bất kể câu hỏi là gì. Ví dụ: nếu $\Pr(\mathbf{y}|\mathbf{x}) = 0.2$ và $\Pr(\mathbf{y}) = 0.1$ (câu chung chung), tỉ số là 2; còn một câu cụ thể có $\Pr(\mathbf{y}|\mathbf{x}) = 0.05$ nhưng $\Pr(\mathbf{y}) = 0.001$ thì tỉ số là 50, nên câu

cụ thể được ưu tiên. Dòng thứ hai của (5.114) dùng công thức xác suất có điều kiện $\Pr(\mathbf{y}|\mathbf{x}) = \frac{\Pr(\mathbf{x},\mathbf{y})}{\Pr(\mathbf{x})}$; tỉ số $\frac{\Pr(\mathbf{x},\mathbf{y})}{\Pr(\mathbf{x})\Pr(\mathbf{y})}$ bằng 1 khi $\mathbf{x}$ và $\mathbf{y}$ độc lập, và càng lớn khi chúng càng "gắn bó" với nhau.

3. Tìm kiếm rủi ro Bayes cực tiểu

Cho đến giờ, phần bàn về mục tiêu tìm kiếm của ta tập trung vào việc chọn giả thuyết có điểm cao nhất, một quá trình được gọi là tìm kiếm hậu nghiệm cực đại (*maximum a posteriori, MAP*) (MAP)[14]. Một giả định nền tảng của phương pháp này là xác suất hậu nghiệm $\Pr(\mathbf{y}|\mathbf{x})$ (hay điểm mô hình $\text{Score}(\mathbf{x},\mathbf{y})$) tương quan với chất lượng của đầu ra. Trong thực tế, giả định này mang lại một số tính chất hữu ích. Chẳng hạn, hệ thống tìm kiếm dễ hiện thực, và mục tiêu tìm kiếm nhất quán với mục tiêu huấn luyện. Tuy nhiên, tìm kiếm MAP có một số nhược điểm, khiến các nhà nghiên cứu tìm kiếm những phương án mạnh hơn. Một vấn đề của tìm kiếm MAP là mục tiêu của nó không khớp với các độ đo đánh giá (*evaluation metric*) thực tế của hệ thống. Các độ đo dùng trong đánh giá đầu-cuối của một hệ thống có thể có dạng khác hẳn $\Pr(\mathbf{y}|\mathbf{x})$. Vấn đề thứ hai là MAP, với tư cách là một ước lượng điểm (*point estimate*) cụ thể trong lý thuyết quyết định Bayes (*Bayesian decision theory*), không cung cấp thước đo nào về độ bất định và thường quá tự tin. Trong một số ứng dụng, các mô hình chuỗi-sang-chuỗi trải khối lượng xác suất quá mỏng trên rất nhiều giả thuyết khác nhau [Ott và cộng sự, 2018a], nghĩa là ước lượng MAP có thể không nắm được phần lớn của phân phối thật.

Chú giải của người dịch: MAP, MLE và quy tắc Bayes

Xác suất có điều kiện $\Pr(A|B)$ là xác suất xảy ra A khi biết B đã xảy ra. *Quy tắc Bayes* cho phép "đảo chiều": $\Pr(\theta|x) = \frac{\Pr(x|\theta)\Pr(\theta)}{\Pr(x)}$. Mẫu số $\Pr(x)$ không phụ thuộc θ nên có thể bỏ khi tìm arg max, đó là dòng thứ hai của (5.116). Ví dụ: một đồng xu tung 3 lần đều ra mặt ngửa. MLE kết luận xác suất ra ngửa là 1 (vì 1^3 là lớn nhất); còn MAP, nếu ta có niềm tin ban đầu (tiên nghiệm) rằng đồng xu thường cân đối, sẽ cho một ước lượng "dè dặt" hơn, nằm giữa 0.5 và 1. Khi tiên nghiệm là đều (mọi giá trị như nhau), $\Pr(\theta)$ là hằng số và MAP trùng với MLE.

Ở đây, ta xét tìm kiếm **rủi ro Bayes cực tiểu** (*minimum Bayes risk, MBR*) (**MBR**), một cách có cơ sở để đưa các độ đo đánh giá vào quá trình giải mã và tận dụng toàn bộ phân phối trên các giả thuyết. Phương pháp MBR định nghĩa một hàm rủi ro (*risk function*) trên một cặp chuỗi, ký hiệu là $R(\mathbf{y},\mathbf{y}_r)$. Hàm này tính chi phí của việc dự đoán $\mathbf{y}$ khi tham chiếu (*reference*) đúng là $\mathbf{y}_r$, được đánh giá theo một

[14] Trong thống kê, MAP là một phương pháp suy ra các tham số của một mô hình thống kê. Giả sử ta có một mô hình mô tả phân phối của một biến x, được tham số hóa bởi θ. MAP tìm giá trị tối ưu của θ bằng cách cực đại hóa xác suất của θ khi biết x, viết là:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} \Pr(\theta|x). \tag{5.115}$$

$\hat{\theta}_{\text{MAP}}$ còn được gọi là *mốt* (giá trị có xác suất lớn nhất) của phân phối hậu nghiệm của θ. Với bài toán tìm kiếm MAP ở đây, ta chỉ cần ký hiệu θ là $\mathbf{y}$ và tìm mốt của $\Pr(\mathbf{y}|\mathbf{x})$. Là một phương pháp Bayes, ta có thể viết lại phương trình trên bằng quy tắc Bayes

$$\begin{aligned}\hat{\theta}_{\text{MAP}} &= \arg\max_{\theta} \frac{\Pr(x|\theta)\cdot\Pr(\theta)}{\Pr(x)} \\ &= \arg\max_{\theta} \Pr(x|\theta)\cdot\Pr(\theta),\end{aligned} \tag{5.116}$$

trong đó θ được xem là một biến ngẫu nhiên với phân phối tiên nghiệm $\Pr(\theta)$. Ngược lại, MLE trực tiếp cực đại hóa hàm hợp lý $\Pr(x|\theta)$

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} \Pr(x|\theta). \tag{5.117}$$

Do đó, kết quả MAP có thể được xem là một ước lượng của θ xét đến cả khả năng (likelihood) của x khi biết θ lẫn phân phối tiên nghiệm của θ. Lưu ý rằng MAP và MLE là tương đương nếu $\Pr(\theta)$ là phân phối đều.

độ đo cụ thể. Ví dụ, trong dịch máy, ta có thể định nghĩa điểm rủi ro là $1 - \text{BLEU}$. Tổng rủi ro của một ứng viên $\mathbf{y}$ trên một tập Ω sau đó được định nghĩa là kỳ vọng của $R(\mathbf{y}, \mathbf{y}_r)$ theo phân phối hậu nghiệm $\Pr(\mathbf{y}_r|\mathbf{x})$:

$$\begin{aligned}\text{Risk}(\mathbf{y}) &= \mathbb{E}_{\mathbf{y}_r \sim \Pr(\mathbf{y}_r|\mathbf{x})} R(\mathbf{y}, \mathbf{y}_r) \\ &= \sum_{\mathbf{y}_r \in \Omega} R(\mathbf{y}, \mathbf{y}_r) \cdot \Pr(\mathbf{y}_r|\mathbf{x}).\end{aligned} \tag{5.118}$$

Tuy nhiên, phép lấy tổng trên mọi chuỗi phía đích có thể có là không khả thi về tính toán. Do đó, ta giới hạn Ω trong k đầu ra tốt nhất hoặc các đầu ra được lấy mẫu của một hệ thống [Eikema và Aziz, 2020], ký hiệu là Ω_{system}. Sau đó, ta lấy $\text{Score}(\mathbf{x}, \mathbf{y}) = -\text{Risk}(\mathbf{y})$ và thu được mục tiêu sau cho tìm kiếm MBR

$$\begin{aligned}\hat{\mathbf{y}} &= \arg\max_{\mathbf{y}} \; -\text{Risk}(\mathbf{y}) \\ &= \arg\min_{\mathbf{y}} \sum_{\mathbf{y}_r \in \Omega_{\text{system}}} R(\mathbf{y}, \mathbf{y}_r) \cdot \Pr(\mathbf{y}_r|\mathbf{x}).\end{aligned} \tag{5.119}$$

Chú giải của người dịch: MBR: chọn ứng viên "được đồng thuận" nhất

Kỳ vọng $\mathbb{E}$ ở (5.118) đơn giản là trung bình có trọng số theo xác suất, giống như điểm trung bình có hệ số. Giả sử hệ thống sinh ra 3 bản dịch A, B, C với xác suất 0.4, 0.35, 0.25, trong đó B và C rất giống nhau còn A khác hẳn. MAP chọn A vì xác suất riêng lẻ cao nhất. MBR thì hỏi: "nếu chọn A, trung bình nó lệch bao nhiêu so với các bản dịch khả dĩ khác?". Vì B và C cộng lại chiếm 0.6 xác suất và giống nhau, chọn B (hoặc C) sẽ có rủi ro trung bình thấp hơn. Như vậy, MBR ưa ứng viên nằm ở "trung tâm" của phân phối, giống như chọn ý kiến được nhiều người đồng tình nhất.

Mô hình này rất tổng quát và áp dụng được cho nhiều bài toán NLP, trong đó cần tìm một giả thuyết tối ưu trong một tập lớn các ứng viên [Goodman, 1996b; Goel và Byrne, 2000; Kumar và Byrne, 2004b]. Nó cho phép các dạng hàm rủi ro linh hoạt, chẳng hạn kết hợp nhiều yếu tố khác nhau khi đánh giá các giả thuyết. Gần đây, tìm kiếm MBR được các nhà nghiên cứu NLP quan tâm vì người ta thấy nó hiệu quả trong việc loại bỏ các thiên lệch do tìm kiếm MAP gây ra [Müller và Sennrich, 2021; Freitag và cộng sự, 2022]. Bên cạnh việc cung cấp một cách phát biểu mục tiêu tìm kiếm, các phương pháp MBR còn có thể được dùng để huấn luyện các mô hình chuỗi-sang-chuỗi, và được xem là một giải pháp hứa hẹn cho sự chênh lệch giữa mục tiêu huấn luyện và mục tiêu đánh giá [Shen và cộng sự, 2016].

5.5 Tóm tắt

Trong chương này, chúng tôi đã trình bày tổng quan về mô hình hóa chuỗi-sang-chuỗi, vốn là nền tảng của nhiều hệ thống NLP hiện đại. Mô hình hóa chuỗi-sang-chuỗi là một lĩnh vực nghiên cứu rất phong phú, và đã được bàn luận rộng rãi trong nhiều ngành khác nhau, thậm chí vượt ra ngoài NLP. Chương này không phải là một bài tổng quan toàn bộ tài liệu về chủ đề này, mà tập trung vào một số phương pháp và ý tưởng cốt lõi. Chúng tôi bắt đầu bằng việc giới thiệu các bài toán chuỗi-sang-chuỗi, cũng như kiến trúc bộ mã hóa–bộ giải mã, vốn đặt nền móng cho hầu hết các hệ thống chuỗi-sang-chuỗi tiên tiến nhất. Để minh họa ứng dụng của kiến trúc này, chúng tôi đã xét bài toán dịch máy nơ-ron và xây dựng một mô hình dịch máy nơ-ron đơn giản bằng những kiến thức cơ bản đã học cho đến lúc đó.

Chúng tôi cũng đã trình bày cơ chế chú ý và một loạt cải tiến của nó. Nhìn lại vài năm qua, ta thấy rằng khám phá các mô hình chú ý là bước tiếp theo tự nhiên trong việc phát triển các mô hình chuỗi-sang-chuỗi. Mặc dù các mô hình chú ý ban đầu trở nên phổ biến nhờ thành công trong dịch máy, từ đó chúng đã chiếm ưu thế trong cộng đồng NLP. Các mô hình chú ý cũng rất được quan tâm trong một số lĩnh vực con khác của AI, như thị giác máy tính (*computer vision*) [Borji và Itti, 2012; Xu và cộng sự, 2015; Jaderberg và cộng sự, 2015] và xử lý tiếng nói (*speech processing*) [Chorowski và cộng sự, 2015; Chan và cộng sự, 2016; Bahdanau và cộng sự, 2016]. Nhờ vậy, vài năm qua là quãng thời gian đầy hứng khởi đối với những người làm việc trong các lĩnh vực này.

Các mô hình chuỗi-sang-chuỗi thành công đến mức có một xu hướng ngày càng mạnh là thống nhất các bài toán đa dạng vào một khung duy nhất. Không chỉ là ta đã phát triển được các mô hình chuỗi-sang-chuỗi mạnh để giải quyết các bài toán rất tổng quát, mà các xu hướng nghiên cứu hiện nay ngày càng ưa chuộng các cách tiếp cận thống nhất. Một ví dụ là Transformer, một mô hình chuỗi-sang-chuỗi dựa trên tự chú ý, đã trở thành xương sống nền tảng cho các tác vụ trải rộng trên nhiều phương thức dữ liệu (*data modality*) khác nhau, từ văn bản đến hình ảnh và tiếng nói. Nó thậm chí có thể được mở rộng để xử lý các bài toán đa phương thức (*multimodal*), vốn đôi khi khó khăn hơn. Điều này làm mọi thứ trở nên thú vị và hấp dẫn hơn: một cải tiến cho một mô hình có thể được dùng để cải thiện các hệ thống trong nhiều tác vụ khác nhau. Và ta đang chứng kiến một thay đổi đáng kể trong mô thức nghiên cứu (*research paradigm*), trong đó các lĩnh vực NLP và học máy đang hội tụ, và các kết quả nghiên cứu NLP ngày càng có ảnh hưởng hơn. Tuy nhiên, mặt trái của đồng xu là ta đang dành nhiều sự chú ý hơn cho một số bài toán trong khi lơ là những bài toán khác. Trong các hội nghị NLP gần đây, có rất nhiều bài báo bàn về cách huấn luyện các mô hình chuỗi-sang-chuỗi lớn và áp dụng chúng vào các tác vụ sinh văn bản khác nhau, nhưng lại có tương đối ít bài báo về phân tích cú pháp (*parsing*). Đã luôn có những tranh luận về điều này trong vài thập kỷ qua, chẳng hạn: ta cần những tri thức tiên nghiệm nào, và cần bao nhiêu, để xây dựng một hệ thống NLP? [Church, 2011; See, 2018] Một thảo luận chi tiết về các tranh luận này nằm ngoài phạm vi của chương. May mắn là nghiên cứu NLP hứa hẹn sẽ tiếp tục đa dạng và sôi động, mang lại những hiểu biết giá trị từ cả hai góc nhìn. Ví dụ, có những phát hiện thú vị rằng các mô hình chuỗi nơ-ron có thể học được một số thuộc tính ngôn ngữ học từ dữ liệu, và các cấu trúc ngôn ngữ học có thể giúp ích cho việc thiết kế hệ thống. Ở Chương 6, ta sẽ thấy một vài ví dụ.

Sự "thiên lệch" về trọng tâm nghiên cứu cũng tồn tại ở phía học máy của việc giải quyết vấn đề. Ví dụ, với các bài toán chuỗi-sang-chuỗi bàn ở đây, những năm gần đây đã chứng kiến sự quan tâm tăng vọt đối với thiết kế mô hình và phương pháp huấn luyện, nhưng chỉ một nhóm tương đối nhỏ người bàn về bài toán tìm kiếm. Mặc dù tìm kiếm là một bài toán kinh điển trong AI và đóng vai trò quan trọng trong các hệ thống thực tế [Russell và Norvig, 2010], nó thậm chí không được bàn đến trong các bài hướng dẫn và bài tổng quan gần đây về NLP. Điều này thôi thúc chúng tôi viết một mục về chủ đề này để có được bức tranh đầy đủ hơn về bài toán. Tuy nhiên, phần bàn luận chung của chúng tôi không bao quát mọi khía cạnh của bài toán tìm kiếm. Một chủ đề quan trọng mà chúng tôi bỏ qua là hiệu quả [Birch và cộng sự, 2018; Heafield và cộng sự, 2021]. Dù chương này có một số thảo luận về vấn đề hiệu quả, như các tiêu chí dừng của thuật toán tìm kiếm, các phương pháp hiệu quả là một chủ đề rộng và nhìn chung phụ thuộc vào kiến trúc mô hình. Có thể tìm thấy thảo luận chi tiết hơn về chúng ở Chương 6. Một chủ đề khác mà bạn đọc có thể quan tâm là tìm kiếm có ràng buộc (*constrained search*), trong đó các ràng buộc được áp đặt lên quá trình tìm kiếm [Hokamp và Liu, 2017; Anderson và cộng sự, 2017]. Nói chung, các ràng buộc này đến từ tri thức tiên nghiệm của ta hoặc từ tương tác với người dùng. Ví dụ, tìm kiếm có ràng buộc đã được dùng để áp đặt các ràng buộc dịch thuật ngữ trong dịch máy [Hasler và cộng sự, 2018; Post và Vilar, 2018].

Một lưu ý cuối cùng về các hạn chế của chương này. Cách phát biểu bài toán chuỗi-sang-chuỗi tổng quát mô tả ở đây dựa trên phép phân tích từ trái sang phải của $\Pr(\mathbf{y}|\mathbf{x})$, dẫn đến một mô hình tự hồi quy

(*autoregressive model*). Một hạn chế của cách phát biểu này là mỗi dự đoán ở một bước nào đó chỉ phụ thuộc vào các từ đứng trước, nên mô hình không thể truy cập ngữ cảnh bên phải. Để tận dụng ngữ cảnh bên phải của một từ, một cách đơn giản là xây dựng thêm một mô hình sinh từ phải sang trái. Các mô hình trái-sang-phải và phải-sang-trái sau đó có thể được kết hợp để sinh ra chuỗi đầu ra tốt hơn [Liu và cộng sự, 2016a; Hoang và cộng sự, 2017; Zhang và cộng sự, 2018b; 2020a]. Một cách tiếp cận khác là **sinh phi tự hồi quy** (*non-autoregressive generation*) hay **giải mã phi tự hồi quy** (*non-autoregressive decoding*), trong đó ràng buộc của việc sinh tự hồi quy được loại bỏ và mỗi dự đoán từ được đặt điều kiện trên ngữ cảnh toàn cục [Gu và cộng sự, 2018; Ghazvininejad và cộng sự, 2019; Lee và cộng sự, 2020]. Một ưu điểm then chốt của sinh phi tự hồi quy là khả năng tăng tốc hệ thống, vì mọi từ trong một chuỗi có thể được sinh song song và ta có thể làm điều này hiệu quả bằng GPU.

Chú giải của người dịch: Tự hồi quy và phi tự hồi quy

Tự hồi quy giống như viết câu từng chữ một từ trái sang phải: muốn viết chữ thứ 5 phải đợi viết xong 4 chữ đầu. *Phi tự hồi quy* giống như điền đồng thời mọi ô của một ô chữ khi đã biết trước số ô: các vị trí được dự đoán cùng lúc, nên nhanh hơn nhiều trên GPU (vốn giỏi làm nhiều phép tính song song), nhưng khó bảo đảm các từ ăn khớp với nhau vì chúng không "nhìn thấy" lựa chọn của nhau trong lúc sinh.

Chương 6

Transformer

Cho đến nay, chúng ta đã bàn về một số mô hình cơ bản để giải các bài toán chuỗi-sang-chuỗi (*sequence-to-sequence*). Bây giờ ta sẽ tìm hiểu một lớp mô hình mới dựa trên một kiến trúc rất mạnh, gọi là **Transformer**. Transformer khác với các mô hình đã trình bày ở Chương 4 và 5 ở nhiều điểm. Thứ nhất, chúng không dựa vào mạng nơ-ron hồi quy (*recurrent neural network*) hay mạng nơ-ron tích chập (*convolutional neural network*) để mô hình hóa chuỗi từ, mà chỉ dùng cơ chế chú ý (*attention mechanism*) và mạng nơ-ron truyền thẳng (*feed-forward neural network*). Thứ hai, việc dùng tự chú ý (*self-attention*) trong Transformer giúp xử lý dễ dàng hơn các ngữ cảnh toàn cục và các quan hệ phụ thuộc giữa các từ. Thứ ba, Transformer là những kiến trúc rất linh hoạt, có thể dễ dàng sửa đổi để phù hợp với nhiều tác vụ khác nhau. Thập kỷ vừa qua đã chứng kiến sự trỗi dậy của Transformer không chỉ trong NLP mà còn trong nhiều lĩnh vực khác. Khi Transformer và các biến thể của nó ngày càng hoàn thiện, những mô hình này đang đóng vai trò ngày càng quan trọng trong nghiên cứu và ứng dụng trí tuệ nhân tạo (*artificial intelligence*).

Trong chương này, chúng ta sẽ bàn về những ý tưởng cốt lõi của Transformer. Ta bắt đầu bằng việc xem xét kiến trúc Transformer chuẩn. Sau đó ta sẽ xem một số bước phát triển đáng chú ý, chẳng hạn các cải tiến cho kiến trúc cơ bản và các phương pháp hiệu quả (về tính toán). Chúng tôi cũng sẽ giới thiệu một số ứng dụng mà ở đó mô hình Transformer đã được dùng rộng rãi. Tuy vậy, Transformer là một chủ đề rất rộng và đã có rất nhiều bài báo liên quan. Chương này không nhằm cung cấp một bản tổng quan đầy đủ về các công trình đã có, mà là trình bày một tập hợp các chủ đề được chọn lọc, có thể được cộng đồng NLP quan tâm.

6.1 Mô hình cơ bản

Ở đây ta xem xét mô hình được trình bày trong công trình của Vaswani và cộng sự [2017]. Ta bắt đầu với kiến trúc Transformer rồi bàn chi tiết về từng thành phần của nó.

6.1.1 Kiến trúc Transformer

Hình 6.1 cho thấy mô hình Transformer chuẩn, đi theo khung bộ mã hóa (*encoder*)–bộ giải mã (*decoder*) chuẩn. Một bộ mã hóa Transformer gồm nhiều **tầng mã hóa** (*encoding layers*) (hay **khối mã hóa** (*encoding blocks*)) xếp chồng lên nhau. Mỗi tầng mã hóa có hai tầng con (*sub-layers*) (hay khối con) khác nhau, gọi là tầng con tự chú ý và tầng con mạng truyền thẳng (*feed-forward neural network, FFN*) (FFN). Giả sử ta có một chuỗi phía nguồn $\mathbf{x} = x_1 \cdots x_m$ và một chuỗi phía đích $\mathbf{y} = y_1 \cdots y_n$. Đầu vào của một tầng mã hóa là một chuỗi gồm m vectơ $\mathbf{h}_1 \cdots \mathbf{h}_m$, mỗi vectơ có d_{model} chiều (hay d chiều

cho gọn). Theo ký hiệu đã dùng ở các chương trước, ta dùng $\mathbf{H} \in \mathbb{R}^{m \times d}$ để ký hiệu các vectơ đầu vào này[1]. Tầng con tự chú ý trước hết thực hiện một phép tự chú ý $\text{Att}_{\text{self}}(\cdot)$ trên $\mathbf{H}$ để sinh ra đầu ra $\mathbf{C}$:

$$\mathbf{C} = \text{Att}_{\text{self}}(\mathbf{H}). \tag{6.1}$$

Ở đây $\mathbf{C}$ có cùng kích thước với $\mathbf{H}$, và do đó có thể được xem là một biểu diễn mới của các đầu vào. Sau đó, một kết nối tắt (residual) (*residual connection*) và một khối chuẩn hóa tầng (*layer normalization*) được thêm vào đầu ra để mô hình thu được dễ tối ưu hơn.

Chú giải của người dịch: Ma trận H: mỗi hàng là một từ

Hãy hình dung câu nguồn có $m = 3$ từ, và mỗi từ được biểu diễn bằng một vectơ $d = 4$ chiều. Xếp ba vectơ này thành ba hàng, ta được một bảng số 3×4, chính là ma trận $\mathbf{H}$:

$$\mathbf{H} = \begin{bmatrix} \mathbf{h}_1 \\ \mathbf{h}_2 \\ \mathbf{h}_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 0 \\ 3 & 1 & 0 & 1 \end{bmatrix} \quad \text{(hàng 1 là từ thứ nhất, hàng 2 là từ thứ hai, …).}$$

Mỗi tầng của Transformer nhận vào một bảng như vậy và trả ra một bảng *cùng kích thước* $m \times d$: số hàng vẫn là số từ, nhưng nội dung từng hàng đã được "làm giàu" thêm thông tin từ các từ khác trong câu. Vì kích thước không đổi nên ta có thể xếp chồng nhiều tầng giống hệt nhau lên nhau.

Mô hình Transformer gốc dùng cấu trúc **post-norm** (*chuẩn hóa sau*), trong đó kết nối tắt được tạo ra trước khi thực hiện chuẩn hóa tầng, như sau:

$$\mathbf{H}_{\text{self}} = \text{LNorm}(\mathbf{C} + \mathbf{H}), \tag{6.2}$$

trong đó phép cộng thêm $\mathbf{H}$ biểu thị kết nối tắt, còn $\text{LNorm}(\cdot)$ là hàm chuẩn hóa tầng. Thay Phương trình (6.1) vào Phương trình (6.2), ta thu được dạng của tầng con tự chú ý

$$\begin{aligned} \text{Layer}_{\text{self}}(\mathbf{H}) &= \mathbf{H}_{\text{self}} \\ &= \text{LNorm}(\text{Att}_{\text{self}}(\mathbf{H}) + \mathbf{H}). \end{aligned} \tag{6.3}$$

Định nghĩa của $\text{LNorm}(\cdot)$ và $\text{Att}_{\text{self}}(\cdot)$ đã được trình bày ở Chương 2 và 5, và chúng ta cũng sẽ bàn lại chúng ở phần sau của mục này.

Tầng con FFN nhận $\mathbf{H}_{\text{self}}$ và cho ra một biểu diễn mới $\mathbf{H}_{\text{ffn}} \in \mathbb{R}^{m \times d}$. Nó có cùng dạng với tầng con tự chú ý, chỉ khác là hàm chú ý được thay bằng hàm FFN:

$$\begin{aligned} \text{Layer}_{\text{ffn}}(\mathbf{H}_{\text{self}}) &= \mathbf{H}_{\text{ffn}} \\ &= \text{LNorm}(\text{FFN}(\mathbf{H}_{\text{self}}) + \mathbf{H}_{\text{self}}). \end{aligned} \tag{6.4}$$

Ở đây $\text{FFN}(\cdot)$ có thể là bất kỳ mạng nơ-ron truyền thẳng nào có hàm kích hoạt (*activation function*) phi tuyến. Cấu trúc phổ biến nhất của $\text{FFN}(\cdot)$ là một mạng hai tầng gồm hai phép biến đổi tuyến tính và một hàm kích hoạt ReLU ở giữa.

[1] Với $\mathbf{h}_j \in \mathbb{R}^d$ là một vectơ hàng, ta có $\mathbf{H} = \begin{bmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_m \end{bmatrix}$.

Chú giải của người dịch: Kết nối tắt: "cộng lại cái cũ"

Phép cộng $\mathbf{C} + \mathbf{H}$ nghĩa là cộng hai ma trận cùng kích thước theo từng phần tử (phần tử ở hàng i, cột j của ma trận này cộng với phần tử ở đúng vị trí đó của ma trận kia). Ý tưởng: tầng con chỉ cần học "phần cần sửa thêm" $\mathbf{C}$ chứ không phải tạo lại toàn bộ biểu diễn; thông tin gốc $\mathbf{H}$ luôn được chuyển thẳng lên trên. Giống như khi sửa bài văn: thay vì viết lại từ đầu, ta giữ bản cũ và chỉ ghi thêm phần chỉnh sửa. Nhờ "đường tắt" này, khi huấn luyện, tín hiệu gradient cũng đi xuyên qua nhiều tầng dễ dàng hơn, nên mô hình sâu dễ tối ưu hơn.

Với các mô hình sâu, ta có thể xếp chồng các mạng nơ-ron trên. Gọi $\mathbf{H}^l$ là đầu ra của tầng l. Khi đó ta có thể biểu diễn $\mathbf{H}^l$ như một hàm của $\mathbf{H}^{l-1}$. Ta viết nó dưới dạng hợp của hai tầng con

$$\mathbf{H}^l = \text{Layer}_{\text{ffn}}(\mathbf{H}^l_{\text{self}}), \tag{6.5}$$

$$\mathbf{H}^l_{\text{self}} = \text{Layer}_{\text{self}}(\mathbf{H}^{l-1}). \tag{6.6}$$

Các nhãn trong Hình 6.1: *Encoder*/*Decoder* = bộ mã hóa/bộ giải mã; *Self-Attention* = tự chú ý; *Feed-Forward Network* = mạng truyền thẳng; *Encoder-Decoder Attention* = chú ý bộ mã hóa–bộ giải mã; *Add & LayerNorm* = cộng (kết nối tắt) rồi chuẩn hóa tầng; *Word*/*Position* = (nhúng) từ/(nhúng) vị trí.

Nếu có L tầng mã hóa thì $\mathbf{H}^L$ là đầu ra cuối cùng của bộ mã hóa. Khi đó, $\mathbf{H}^L$ có thể được xem là một biểu diễn (*representation*) theo ngữ cảnh đã được học của chuỗi đầu vào. $\mathbf{H}^0$ ký hiệu đầu vào ban đầu của bộ mã hóa. Trong các mô hình hồi quy và tích chập, $\mathbf{H}^0$ thường chỉ gồm các vectơ nhúng từ (*word embeddings*) của chuỗi đầu vào. Transformer dùng một cách tiếp cận khác để biểu diễn các từ đầu vào và mã hóa thông tin vị trí một cách tường minh. Ở Mục 6.1.2, ta sẽ bàn về mô hình nhúng được dùng trong Transformer.

Bộ giải mã Transformer có cấu trúc tương tự bộ mã hóa Transformer, gồm L **tầng giải mã** (*decoding layers*) (hay **khối giải mã**, *decoding blocks*) xếp chồng. Gọi $\mathbf{S}^l$ là đầu ra của tầng giải mã thứ l. Ta có thể viết một tầng giải mã bằng các phương trình sau:

$$\mathbf{S}^l = \text{Layer}_{\text{ffn}}(\mathbf{S}^l_{\text{cross}}), \tag{6.7}$$

$$\mathbf{S}^l_{\text{cross}} = \text{Layer}_{\text{cross}}(\mathbf{H}^L, \mathbf{S}^{l-1}_{\text{self}}), \tag{6.8}$$

$$\mathbf{S}^l_{\text{self}} = \text{Layer}_{\text{self}}(\mathbf{S}^{l-1}). \tag{6.9}$$

Ở đây có ba tầng con của bộ giải mã. Tầng con tự chú ý và tầng con FFN giống với các tầng con dùng trong bộ mã hóa. $\text{Layer}_{\text{cross}}(\cdot)$ ký hiệu một tầng con chú ý chéo (*cross attention*) (hay tầng con bộ mã hóa–bộ giải mã), mô hình hóa phép biến đổi từ phía nguồn sang phía đích. Ở Mục 6.1.6, ta sẽ thấy rằng $\text{Layer}_{\text{cross}}(\cdot)$ có thể được cài đặt bằng chính hàm dùng cho $\text{Layer}_{\text{self}}(\cdot)$.

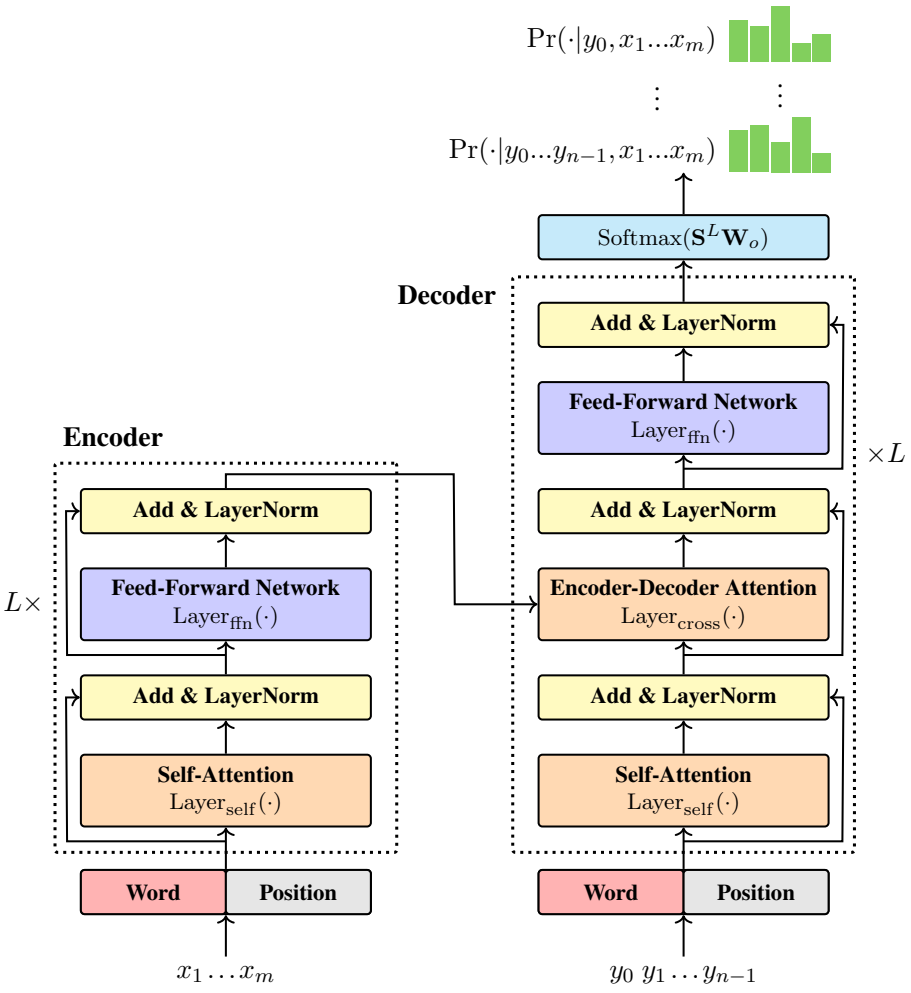
Bộ giải mã Transformer cho ra một phân phối trên bộ từ vựng (*vocabulary*) V_{y} tại mỗi vị trí phía đích. Điều này được thực hiện bằng một tầng Softmax, chuẩn hóa một phép biến đổi tuyến tính của $\mathbf{S}^L$ thành các phân phối trên các từ phía đích. Để làm vậy, ta ánh xạ $\mathbf{S}^L$ thành một ma trận $\mathbf{O}$ kích thước $n \times |V_{\text{y}}|$ bằng

$$\mathbf{O} = \mathbf{S}^L \cdot \mathbf{W}_{\text{o}}, \tag{6.10}$$

trong đó $\mathbf{W}_{\text{o}} \in \mathbb{R}^{d \times |V_{\text{y}}|}$ là ma trận tham số của phép biến đổi tuyến tính.

Khi đó, đầu ra của bộ giải mã Transformer có dạng

$$\begin{bmatrix} \Pr(\cdot|y_0, \mathbf{x}) \\ \vdots \\ \Pr(\cdot|y_0 \cdots y_{n-1}, \mathbf{x}) \end{bmatrix} = \text{Softmax}(\mathbf{O})$$



Hình 6.1: Kiến trúc Transformer [Vaswani và cộng sự, 2017]. Mỗi phía bộ mã hóa và bộ giải mã có L tầng xếp chồng. Một tầng mã hóa gồm một tầng con tự chú ý và một tầng con FFN. Cả hai tầng con này có chung một cấu trúc, gồm một hàm lõi ($\text{Layer}_{\text{self}}(\cdot)$ hoặc $\text{Layer}_{\text{ffn}}(\cdot)$), theo sau là một kết nối tắt và một khối chuẩn hóa tầng. Mỗi tầng giải mã có kiến trúc tương tự các tầng mã hóa, nhưng có thêm một tầng con chú ý bộ mã hóa–bộ giải mã kẹp giữa tầng con tự chú ý và tầng con FFN. Giống như hầu hết các mô hình chuỗi-sang-chuỗi, Transformer nhận $x_1 \cdots x_m$ và $y_0 \cdots y_{i-1}$ để dự đoán y_i. Biểu diễn của một từ đầu vào là tổng của một vectơ nhúng từ và một vectơ nhúng vị trí. Các phân phối $\{\Pr(\cdot|y_0 \cdots y_{i-1}, x_1 \cdots x_m)\}$ được sinh lần lượt bởi một tầng Softmax, tầng này tác động lên một phép biến đổi tuyến tính của đầu ra từ tầng giải mã cuối cùng.

$$= \begin{bmatrix} \text{Softmax}(\mathbf{o}_1) \\ \vdots \\ \text{Softmax}(\mathbf{o}_n) \end{bmatrix}, \tag{6.11}$$

trong đó $\mathbf{o}_i$ là vectơ hàng thứ i của $\mathbf{O}$, và y_0 là ký hiệu bắt đầu $\langle\text{SOS}\rangle$. Theo mô hình này, xác suất của $\mathbf{y}$ khi biết $\mathbf{x}$ được định nghĩa như thường lệ,

$$\log \Pr(\mathbf{y}|\mathbf{x}) \;=\; \sum_{i=1}^{n} \log \Pr(y_i|y_0 \cdots y_{i-1}, \mathbf{x}). \tag{6.12}$$

Chú giải của người dịch: Từ ma trận $\mathbf{O}$ đến xác suất từng từ

Giả sử bộ từ vựng đích chỉ có 3 từ {*tôi*, *đi*, *học*} và $d = 2$. Nếu hàng thứ nhất của $\mathbf{S}^L$ là $[1 \quad 2]$ và $\mathbf{W}_o = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & -1 \end{bmatrix}$ (2×3) thì hàng thứ nhất của $\mathbf{O}$ là

$$\mathbf{o}_1 = [1 \cdot 1 + 2 \cdot 0 \quad 1 \cdot 0 + 2 \cdot 1 \quad 1 \cdot 1 + 2 \cdot (-1)] = [1 \;\; 2 \;\; -1],$$

tức một "điểm số" cho mỗi từ trong bộ từ vựng. Softmax biến các điểm thành xác suất: $\text{Softmax}(\mathbf{o}_1) = \left[\frac{e^1}{S} \quad \frac{e^2}{S} \quad \frac{e^{-1}}{S}\right] \approx [0.259 \quad 0.705 \quad 0.035]$ với $S = e^1 + e^2 + e^{-1} \approx 10.47$. Các số này không âm và có tổng bằng 1. Softmax được áp dụng *riêng cho từng hàng* (mỗi hàng ứng với một vị trí đích), như Phương trình (6.11) viết. Phương trình (6.12) thì nói: xác suất cả câu đích bằng tích xác suất của từng từ (lấy log thì tích thành tổng).

Phương trình này giống với dạng tổng quát của mô hình hóa ngôn ngữ (*language modeling*): ta dự đoán từ ở vị trí i khi biết mọi từ cho đến vị trí $i-1$. Vì vậy, chuỗi đầu ra đích được dịch đi một bước về phía trước theo thời gian so với chuỗi đầu vào, nghĩa là bộ giải mã nhận $y_0 \cdots y_{n-1}$ làm đầu vào để dự đoán chuỗi đầu ra $y_1 \cdots y_n$.

Kiến trúc Transformer vừa trình bày có một số biến thể đã được dùng thành công trong nhiều lĩnh vực khác nhau của NLP. Ví dụ, ta có thể dùng một bộ mã hóa Transformer để biểu diễn văn bản (gọi là **kiến trúc chỉ có bộ mã hóa** (*encoder-only architecture*)), một bộ giải mã Transformer để sinh văn bản (gọi là **kiến trúc chỉ có bộ giải mã** (*decoder-only architecture*)), hoặc một mô hình Transformer bộ mã hóa–bộ giải mã chuẩn để biến đổi một chuỗi đầu vào thành một chuỗi đầu ra. Trong phần còn lại của chương này, phần lớn nội dung không phụ thuộc vào ứng dụng cụ thể, và sẽ chủ yếu tập trung vào kiến trúc bộ mã hóa–bộ giải mã. Ở Mục 6.5, ta sẽ thấy các ứng dụng của kiến trúc chỉ có bộ mã hóa và chỉ có bộ giải mã.

6.1.2 Mã hóa vị trí

Ở dạng ban đầu, cả FFN lẫn các mô hình chú ý dùng trong Transformer đều bỏ qua một khía cạnh quan trọng của việc mô hình hóa chuỗi, đó là thứ tự của các từ đóng vai trò then chốt trong việc biểu đạt ý nghĩa của chuỗi. Điều này có nghĩa là bộ mã hóa và bộ giải mã không nhạy với thông tin vị trí của các từ đầu vào. Một cách đơn giản để khắc phục vấn đề này là cộng thêm các mã hóa vị trí (*positional encodings*) vào biểu diễn của mỗi từ trong chuỗi. Chính xác hơn, một từ x_j có thể được biểu diễn bằng một vectơ d chiều

$$\mathbf{e}_j \;=\; \mathbf{x}_j + \text{PE}(j). \tag{6.13}$$

Ở đây $\mathbf{x}_j \in \mathbb{R}^d$ là vectơ nhúng từ, có thể thu được bằng các mô hình nhúng từ tiêu chuẩn như đã mô tả ở Chương 3. $\text{PE}(j) \in \mathbb{R}^d$ là biểu diễn của vị trí j. Transformer nguyên bản dùng một mô hình mã hóa

vị trí dạng sin, được viết dưới dạng

$$\mathrm{PE}(j, 2k) = \sin\left(j \cdot \frac{1}{10000^{2k/d}}\right), \tag{6.14}$$

$$\mathrm{PE}(j, 2k+1) = \cos\left(j \cdot \frac{1}{10000^{2k/d}}\right), \tag{6.15}$$

trong đó $\mathrm{PE}(j, k)$ là phần tử thứ k của $\mathrm{PE}(j)$. Ý tưởng của mã hóa vị trí là phân biệt các vị trí khác nhau bằng các hàm liên tục. Ở đây ta dùng các hàm sin và cos với những tần số khác nhau. Bạn đọc quan tâm có thể xem Chương 4 để thấy rằng phương pháp này có thể được hiểu như một hệ đếm theo vị trí. Vì việc mã hóa dựa trên từng vị trí riêng lẻ, nó còn được gọi là **mã hóa vị trí tuyệt đối** (*absolute positional encoding*). Ở Mục 6.3.1, ta sẽ thấy một cải tiến cho phương pháp này.

Chú giải của người dịch: Tính tay mã hóa vị trí sin/cos

Lấy $d = 4$, khi đó $k \in \{0, 1\}$ và có hai "tần số": $\omega_0 = 1/10000^{0/4} = 1$ và $\omega_1 = 1/10000^{2/4} = 1/100 = 0.01$. Vectơ vị trí 4 chiều là $\mathrm{PE}(j) = [\sin(j\omega_0) \;\; \cos(j\omega_0) \;\; \sin(j\omega_1) \;\; \cos(j\omega_1)]$ (góc tính bằng radian):

$$\mathrm{PE}(0) = [0 \;\; 1 \;\; 0 \;\; 1], \quad \mathrm{PE}(1) \approx [0.841 \;\; 0.540 \;\; 0.010 \;\; 1.000], \quad \mathrm{PE}(2) \approx [0.909 \;\; -0.416 \;\; 0.020 \;\; 1.000].$$

Hai chiều đầu "quay nhanh" (như kim giây), hai chiều sau "quay chậm" (như kim giờ). Giống như đọc giờ trên đồng hồ, tổ hợp của các kim quay với tốc độ khác nhau cho phép phân biệt rất nhiều vị trí. Mọi giá trị đều nằm trong $[-1, 1]$, nên vectơ vị trí không "lấn át" vectơ nhúng từ khi cộng vào theo Phương trình (6.13).

Khi đã có các vectơ nhúng này, chuỗi $\mathbf{e}_1 \cdots \mathbf{e}_m$ được dùng làm đầu vào cho bộ mã hóa Transformer, tức là

$$\mathbf{H}^0 = \begin{bmatrix} \mathbf{e}_1 \\ \vdots \\ \mathbf{e}_m \end{bmatrix}. \tag{6.16}$$

Tương tự, ta có thể định nghĩa đầu vào ở phía bộ giải mã.



(a) RNN (b) CNN ($r = 3$) (c) Self-attention

Hình 6.2: Các luồng thông tin trong mô hình hồi quy (a, RNN), tích chập (b, CNN với độ rộng cửa sổ $r = 3$) và tự chú ý (c), được biểu diễn bằng các mũi tên nối giữa các vị trí.

6.1.3 Tự chú ý đa đầu

Việc dùng tự chú ý có lẽ là một trong những tiến bộ quan trọng nhất của các mô hình chuỗi-sang-chuỗi. Nó cố gắng học và tận dụng các tương tác trực tiếp giữa mọi cặp đầu vào. Từ góc nhìn học biểu diễn, các mô hình tự chú ý giả định rằng biểu diễn học được tại vị trí i (ký hiệu là $\mathbf{c}_i$) là một tổng có trọng số của các đầu vào trên toàn chuỗi. Do đó đầu ra $\mathbf{c}_i$ được cho bởi

$$\mathbf{c}_i = \sum_{j=1}^{m} \alpha_{i,j} \mathbf{h}_j, \tag{6.17}$$

trong đó $\alpha_{i,j}$ là trọng số chú ý (*attention weight*) gán cho $\mathbf{h}_j$ khi tính biểu diễn tại vị trí i. Như vậy, ta có thể xem $\mathbf{c}_i$ là một biểu diễn của ngữ cảnh toàn cục tại vị trí i. $\alpha_{i,j}$ có thể được định nghĩa theo nhiều cách khác nhau tùy vào mô hình chú ý cụ thể. Ở đây ta dùng hàm chú ý tích vô hướng có co giãn (*scaled dot-product attention*) để tính $\alpha_{i,j}$ như sau

$$\begin{aligned}\alpha_{i,j} &= \text{Softmax}\left(\mathbf{h}_i \mathbf{h}_j^\top / \beta\right) \\ &= \frac{\exp(\mathbf{h}_i \mathbf{h}_j^\top / \beta)}{\sum_{k=1}^{m} \exp(\mathbf{h}_i \mathbf{h}_k^\top / \beta)},\end{aligned} \tag{6.18}$$

trong đó hệ số co giãn β thường được đặt bằng $\sqrt{d}$.

Chú giải của người dịch: Tự chú ý bằng số: một câu 3 từ

$\mathbf{h}_i \mathbf{h}_j^\top$ là *tích vô hướng* của hai vectơ hàng (nhân từng cặp phần tử rồi cộng lại), giống $\vec{u} \cdot \vec{v} = u_1 v_1 + u_2 v_2$ ở lớp 10: hai vectơ càng "cùng hướng" thì tích càng lớn, tức hai từ càng "liên quan". Lấy $d = 2$ (nên $\beta = \sqrt{2} \approx 1.41$) và ba từ $\mathbf{h}_1 = [1 \;\; 0]$, $\mathbf{h}_2 = [0 \;\; 1]$, $\mathbf{h}_3 = [1 \;\; 1]$. Tính cho vị trí $i = 1$: các tích vô hướng là $\mathbf{h}_1 \mathbf{h}_1^\top = 1$, $\mathbf{h}_1 \mathbf{h}_2^\top = 0$, $\mathbf{h}_1 \mathbf{h}_3^\top = 1$; chia cho 1.41 được $[0.71 \;\; 0 \;\; 0.71]$. Softmax: $e^{0.71} \approx 2.03$, $e^0 = 1$, tổng ≈ 5.06, nên $[\alpha_{1,1} \; \alpha_{1,2} \; \alpha_{1,3}] \approx [0.40 \;\; 0.20 \;\; 0.40]$. Theo Phương trình (6.17): $\mathbf{c}_1 \approx 0.40[1 \;\; 0] + 0.20[0 \;\; 1] + 0.40[1 \;\; 1] = [0.80 \;\; 0.60]$. Biểu diễn mới của từ 1 là một "hỗn hợp" của cả ba từ, trong đó các từ giống nó nhiều hơn (từ 1 và từ 3) đóng góp nhiều hơn. Chia cho β để các tích vô hướng không quá lớn khi d lớn (nếu không, Softmax sẽ dồn gần hết trọng số vào một từ).

So với các mô hình hồi quy và tích chập thông thường, một ưu điểm của mô hình tự chú ý là chúng rút ngắn "khoảng cách" tính toán giữa hai đầu vào. Hình 6.2 minh họa luồng thông tin trong các mô hình này. Ta thấy rằng, với đầu vào tại vị trí i, mô hình tự chú ý có thể truy cập trực tiếp bất kỳ đầu vào nào khác. Ngược lại, mô hình hồi quy và tích chập có thể cần hai bước nhảy trở lên mới "nhìn thấy" được toàn bộ chuỗi.

Ta có thể nhìn tự chú ý từ một góc độ tổng quát hơn bằng cách dùng mô hình chú ý QKV. Giả sử ta có một chuỗi gồm κ truy vấn (*queries*) $\mathbf{Q} = \begin{bmatrix} \mathbf{q}_1 \\ \vdots \\ \mathbf{q}_\kappa \end{bmatrix}$, và một chuỗi gồm ψ cặp khóa (*key*)–giá trị (*value*) ($\mathbf{K} = \begin{bmatrix} \mathbf{k}_1 \\ \vdots \\ \mathbf{k}_\psi \end{bmatrix}$, $\mathbf{V} = \begin{bmatrix} \mathbf{v}_1 \\ \vdots \\ \mathbf{v}_\psi \end{bmatrix}$). Đầu ra của mô hình là một chuỗi vectơ, mỗi vectơ ứng với một truy vấn. Dạng của chú ý QKV được cho bởi

$$\text{Att}_{\text{qkv}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \;=\; \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}. \tag{6.19}$$

Ta có thể viết đầu ra của mô hình chú ý QKV dưới dạng một chuỗi các vectơ hàng

$$\begin{aligned}\mathbf{C} &= \begin{bmatrix} \mathbf{c}_1 \\ \vdots \\ \mathbf{c}_\kappa \end{bmatrix} \\ &= \text{Att}_{\text{qkv}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}).\end{aligned} \tag{6.20}$$

Để áp dụng phương trình này cho tự chú ý, ta chỉ cần đặt

$$\mathbf{H}^q = \mathbf{H}\mathbf{W}^q, \tag{6.21}$$

$$\mathbf{H}^k = \mathbf{H}\mathbf{W}^k, \tag{6.22}$$
$$\mathbf{H}^v = \mathbf{H}\mathbf{W}^v, \tag{6.23}$$

trong đó $\mathbf{W}^q, \mathbf{W}^k, \mathbf{W}^v \in \mathbb{R}^{d\times d}$ biểu diễn các phép biến đổi tuyến tính của $\mathbf{H}$.

Xét Phương trình (6.1), ta thu được

$$\begin{aligned}\mathbf{C} &= \text{Att}_{\text{self}}(\mathbf{H}) \\ &= \text{Att}_{\text{qkv}}(\mathbf{H}^q, \mathbf{H}^k, \mathbf{H}^v) \\ &= \text{Softmax}\left(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}}\right)\mathbf{H}^v.\end{aligned} \tag{6.24}$$

Ở đây $\text{Softmax}(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}})$ là một ma trận $m \times m$, trong đó mỗi hàng biểu diễn một phân phối trên tất cả các vị trí đầu vào, tức là

$$\text{hàng } i \;=\; \begin{bmatrix}\alpha_{i,1} & \cdots & \alpha_{i,m}\end{bmatrix}. \tag{6.25}$$

Chú giải của người dịch: Đọc Phương trình (6.24) theo kích thước ma trận

Nhân ma trận. Nếu $\mathbf{A}$ cỡ $m \times n$ và $\mathbf{B}$ cỡ $n \times p$ thì $\mathbf{AB}$ cỡ $m \times p$, và phần tử hàng i, cột j của nó là tích vô hướng của *hàng* i của $\mathbf{A}$ với *cột* j của $\mathbf{B}$. Ví dụ $\begin{bmatrix}1 & 2\\ 0 & 1\end{bmatrix}\begin{bmatrix}3\\ 1\end{bmatrix} = \begin{bmatrix}1\cdot 3 + 2\cdot 1\\ 0\cdot 3 + 1\cdot 1\end{bmatrix} = \begin{bmatrix}5\\ 1\end{bmatrix}$. **Chuyển vị** $\mathbf{A}^\top$ là đổi hàng thành cột.

Với m từ và d chiều: $\mathbf{H}^q, \mathbf{H}^k, \mathbf{H}^v$ đều cỡ $m \times d$ (mỗi hàng là truy vấn/khóa/giá trị của một từ). $[\mathbf{H}^k]^\top$ cỡ $d \times m$, nên $\mathbf{H}^q[\mathbf{H}^k]^\top$ cỡ $m \times m$: ô (i, j) là tích vô hướng giữa truy vấn của từ i và khóa của từ j — tức "từ i quan tâm đến từ j bao nhiêu". Softmax được lấy *theo từng hàng*, nên mỗi hàng thành một phân phối xác suất (Phương trình (6.25)). Cuối cùng nhân với $\mathbf{H}^v$ ($m \times m$ nhân $m \times d$) ra $\mathbf{C}$ cỡ $m \times d$: hàng i của $\mathbf{C}$ chính là $\sum_j \alpha_{i,j}\mathbf{v}_j$, đúng như Phương trình (6.17) nhưng tính cho mọi vị trí cùng một lúc. Hình ảnh đời thường: truy vấn là "câu hỏi" của một từ, khóa là "nhãn" trên mỗi ngăn tủ, giá trị là "đồ vật" trong ngăn; từ i so câu hỏi với từng nhãn rồi lấy về một hỗn hợp đồ vật theo mức độ khớp.

Ta có thể cải tiến mô hình tự chú ý trên bằng một kỹ thuật gọi là **chú ý đa đầu** (*multi-head attention*). Phương pháp này có thể được lý giải từ góc độ học từ nhiều không gian con (*subspaces*) đặc trưng có số chiều thấp hơn. Nó chiếu đầu vào vào nhiều không gian con và học các biểu diễn riêng trong từng không gian. Cụ thể, ta chiếu toàn bộ không gian đầu vào vào τ không gian con (gọi là các đầu (*heads*)). Ví dụ, ta biến đổi $\mathbf{H} \in \mathbb{R}^{m\times d}$ thành τ ma trận cỡ $m \times \frac{d}{\tau}$, ký hiệu là $\{\mathbf{H}_1^{\text{head}}, \ldots, \mathbf{H}_\tau^{\text{head}}\}$. Mô hình chú ý sau đó được áp dụng τ lần, mỗi lần cho một đầu. Cuối cùng, các đầu ra của những lần chạy này được ghép nối lại và biến đổi bằng một phép chiếu tuyến tính. Quy trình này có thể được biểu diễn bằng

$$\mathbf{C} \;=\; \text{Merge}(\mathbf{C}_1^{\text{head}}, \cdots, \mathbf{C}_\tau^{\text{head}})\mathbf{W}_c. \tag{6.26}$$

Với mỗi đầu h,

$$\mathbf{C}_h^{\text{head}} = \text{Softmax}\left(\frac{\mathbf{H}_h^q[\mathbf{H}_h^k]^\top}{\sqrt{d}}\right)\mathbf{H}_h^v, \tag{6.27}$$
$$\mathbf{H}_h^q = \mathbf{H}\mathbf{W}_h^q, \tag{6.28}$$
$$\mathbf{H}_h^k = \mathbf{H}\mathbf{W}_h^k, \tag{6.29}$$
$$\mathbf{H}_h^v = \mathbf{H}\mathbf{W}_h^v. \tag{6.30}$$

Ở đây $\text{Merge}(\cdot)$ là hàm ghép nối, và $\text{Att}_{\text{qkv}}(\cdot)$ là hàm chú ý đã mô tả ở Phương trình (6.20). $\mathbf{W}_h^q, \mathbf{W}_h^k, \mathbf{W}_h^v \in \mathbb{R}^{d\times\frac{d}{\tau}}$ là các tham số của các phép chiếu từ không gian d chiều sang không

gian $\frac{d}{\tau}$ chiều cho truy vấn, khóa và giá trị. Do đó, $\mathbf{H}_h^q$, $\mathbf{H}_h^k$, $\mathbf{H}_h^v$ và $\mathbf{C}_h^{\text{head}}$ đều là các ma trận $m \times \frac{d}{\tau}$. $\text{Merge}(\mathbf{C}_1^{\text{head}}, \cdots, \mathbf{C}_\tau^{\text{head}})$ tạo ra một ma trận $m \times d$. Ma trận này sau đó được biến đổi bằng một ánh xạ tuyến tính $\mathbf{W}_c \in \mathbb{R}^{d \times d}$, cho ra kết quả cuối cùng $\mathbf{C} \in \mathbb{R}^{m \times d}$.

Dù ký hiệu có vẻ rườm rà, việc cài đặt các mô hình đa đầu lại rất đơn giản với các bộ công cụ học sâu hiện đại. Một cách làm phổ biến trong các hệ thống dựa trên Transformer là lưu đầu vào của tất cả các đầu trong những cấu trúc dữ liệu gọi là tensor (*tensors*), nhờ đó tận dụng được tài nguyên tính toán song song để tăng hiệu quả. Một thảo luận tổng quát hơn về chú ý QKV và chú ý đa đầu có thể xem ở Chương 5.

Chú giải của người dịch: Đa đầu: chia nhỏ rồi ghép lại

Ví dụ $d = 512$, $\tau = 8$: mỗi đầu làm việc trong không gian $512/8 = 64$ chiều. Đầu thứ h tạo ra $\mathbf{C}_h^{\text{head}}$ cỡ $m \times 64$; Merge đặt 8 ma trận này cạnh nhau theo chiều ngang thành $m \times (8 \cdot 64) = m \times 512$, rồi nhân với $\mathbf{W}_c$ (512×512). Ý tưởng: mỗi đầu có bộ $\mathbf{W}_h^q, \mathbf{W}_h^k, \mathbf{W}_h^v$ riêng nên có thể "chú ý" theo một kiểu khác nhau (đầu này để ý quan hệ ngữ pháp, đầu kia để ý từ đứng gần…), giống một nhóm 8 người cùng đọc một câu, mỗi người tìm một loại manh mối, rồi gộp ý kiến lại. Vì mỗi đầu nhỏ hơn, tổng khối lượng tính toán xấp xỉ một đầu đầy đủ d chiều.

6.1.4 Chuẩn hóa tầng

Chuẩn hóa tầng là một cách đơn giản và hiệu quả để làm cho việc huấn luyện mạng nơ-ron ổn định hơn, bằng cách chuẩn hóa các giá trị kích hoạt của các tầng ẩn (*hidden layers*) theo từng tầng. Như Ba và cộng sự [2016] đề xuất, với đầu ra của một tầng $\mathbf{h} \in \mathbb{R}^d$, phương pháp chuẩn hóa tầng tính một đầu ra đã chuẩn hóa $\text{LNorm}(\mathbf{h}) \in \mathbb{R}^d$ bằng

$$\text{LNorm}(\mathbf{h}) = \boldsymbol{\alpha} \odot \frac{\mathbf{h} - \mu}{\sigma + \epsilon} + \boldsymbol{\beta}. \tag{6.31}$$

Ở đây $\mu \in \mathbb{R}$ và $\sigma \in \mathbb{R}$ lần lượt là trung bình và độ lệch chuẩn (là các số vô hướng) của các giá trị kích hoạt. Gọi h_k là chiều thứ k của $\mathbf{h}$. Các số vô hướng μ và σ được định nghĩa là

$$\mu = \frac{1}{d} \sum_{k=1}^{d} h_k, \tag{6.32}$$

$$\sigma = \sqrt{\frac{1}{d} \sum_{k=1}^{d} (h_k - \mu)^2}. \tag{6.33}$$

Ở đây $\boldsymbol{\alpha} \in \mathbb{R}^d$ và $\boldsymbol{\beta} \in \mathbb{R}^d$ là các tham số co giãn và dịch chuyển. Chúng có thể được coi là tham số của chuẩn hóa tầng, với giá trị được học cùng với các tham số khác của mô hình Transformer. Việc cộng ϵ vào σ nhằm đảm bảo ổn định số học. Nói chung, ϵ được chọn là một số rất nhỏ.

Ta minh họa phương pháp chuẩn hóa tầng cho các trạng thái ẩn của một bộ mã hóa bằng ví dụ sau (giả sử $m = 4$, $d = 3$, $\boldsymbol{\alpha} = \mathbf{1}$, $\boldsymbol{\beta} = \mathbf{0}$ và $\epsilon = 0.1$).

$$\begin{array}{cc} \\ \mathbf{h}_1 \\ \mathbf{h}_2 \\ \mathbf{h}_3 \\ \mathbf{h}_4 \end{array} \begin{bmatrix} 1 & 1 & 2 \\ 0.9 & 0.9 & 0 \\ 0.7 & 0.8 & 0 \\ 3 & 1 & 7 \end{bmatrix} \begin{array}{l} \\ \mu = 1.3,\ \sigma = 0.5 \\ \mu = 0.6,\ \sigma = 0.4 \\ \mu = 0.5,\ \sigma = 0.4 \\ \mu = 3.7,\ \sigma = 2.5 \end{array} \Longrightarrow \begin{bmatrix} \frac{1-1.3}{0.5+0.1} & \frac{1-1.3}{0.5+0.1} & \frac{2-1.3}{0.5+0.1} \\ \frac{0.9-0.6}{0.4+0.1} & \frac{0.9-0.6}{0.4+0.1} & \frac{0-0.6}{0.4+0.1} \\ \frac{0.7-0.5}{0.4+0.1} & \frac{0.8-0.5}{0.4+0.1} & \frac{0-0.5}{0.4+0.1} \\ \frac{3-3.7}{2.5+0.1} & \frac{1-3.7}{2.5+0.1} & \frac{7-3.7}{2.5+0.1} \end{bmatrix}$$

Chú giải của người dịch: Chuẩn hóa tầng giống "quy về điểm chuẩn"

Đây chính là phép chuẩn hóa quen thuộc trong thống kê mô tả: trừ đi trung bình rồi chia cho độ lệch chuẩn, áp dụng cho *từng hàng* (từng từ) một cách riêng rẽ. Tính hàng $\mathbf{h}_1 = [1 \quad 1 \quad 2]$: $\mu = 4/3 \approx 1.33$; $\sigma = \sqrt{\frac{1}{3}\left((-0.33)^2 + (-0.33)^2 + 0.67^2\right)} \approx \sqrt{0.22} \approx 0.47$ (sách làm tròn thành 0.5). Kết quả hàng 1 $\approx [-0.5 \quad -0.5 \quad 1.17]$. Sau chuẩn hóa, các số của mỗi hàng có trung bình xấp xỉ 0 và độ phân tán xấp xỉ 1, dù ban đầu hàng $\mathbf{h}_4 = [3 \quad 1 \quad 7]$ có các số lớn hơn hẳn các hàng khác — nhờ vậy giá trị trong mạng không "phình to" hay "co lại" quá mức qua nhiều tầng. Ký hiệu $\odot$ là nhân theo từng phần tử; $\boldsymbol{\alpha}$ và $\boldsymbol{\beta}$ cho phép mô hình tự co giãn, dịch chuyển lại nếu cần.

Như đã bàn ở Mục 6.1.1, khối chuẩn hóa tầng trong mỗi tầng con được dùng để chuẩn hóa đầu ra của một khối residual. Ở đây ta mô tả một cách viết tổng quát hơn cho cấu trúc này. Giả sử $F(\cdot)$ là một tầng con của mạng nơ-ron. Khi đó, cấu trúc post-norm của $F(\cdot)$ được cho bởi:

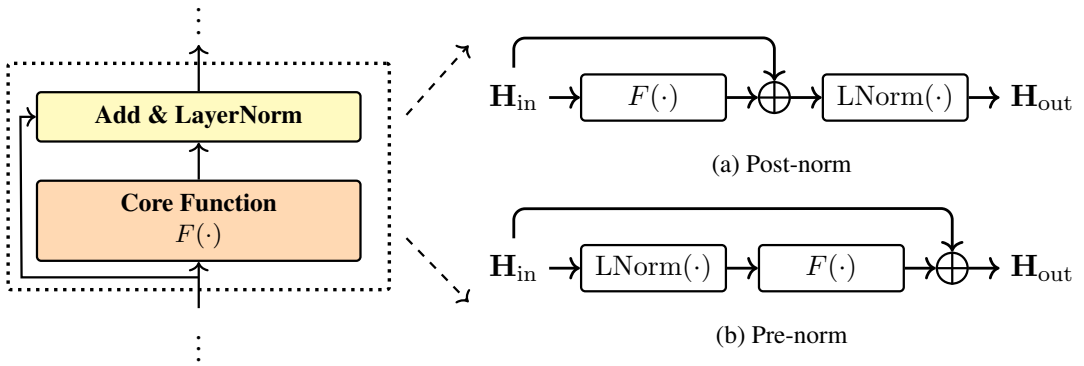
$$\mathbf{H}_{\text{out}} = \text{LNorm}(F(\mathbf{H}_{\text{in}}) + \mathbf{H}_{\text{in}}), \tag{6.34}$$

trong đó $\mathbf{H}_{\text{in}}$ và $\mathbf{H}_{\text{out}}$ là đầu vào và đầu ra của tầng con này. Rõ ràng, Phương trình (6.3) là một trường hợp cụ thể của phương trình này.

Một cách khác để đưa chuẩn hóa tầng và kết nối tắt vào mô hình là thực hiện hàm $\text{LNorm}(\cdot)$ ngay trước hàm $F(\cdot)$, và thiết lập một ánh xạ đồng nhất (*identity mapping*) từ đầu vào tới đầu ra của toàn bộ tầng con. Cấu trúc này, gọi là cấu trúc **pre-norm** (*chuẩn hóa trước*), có thể được viết dưới dạng

$$\mathbf{H}_{\text{out}} = F(\text{LNorm}(\mathbf{H}_{\text{in}})) + \mathbf{H}_{\text{in}}. \tag{6.35}$$

Cả mô hình Transformer post-norm lẫn pre-norm đều được dùng rộng rãi trong các hệ thống NLP. Xem Hình 6.3 để so sánh hai cấu trúc này. Nói chung, kết nối tắt được xem là một cách hiệu quả để việc huấn luyện các mạng nơ-ron nhiều tầng trở nên dễ dàng hơn. Theo nghĩa này, Transformer pre-norm có vẻ hứa hẹn vì nó tuân theo quy ước rằng một kết nối tắt được tạo ra để đi vòng qua toàn bộ mạng, và ánh xạ đồng nhất từ đầu vào tới đầu ra giúp tối ưu các mô hình sâu dễ hơn. Tuy nhiên, khi xét tới khả năng biểu đạt của mô hình, việc dùng Transformer post-norm có thể có lợi thế về mặt mô hình hóa, vì nó không phụ thuộc quá nhiều vào kết nối tắt và buộc việc học biểu diễn phải được mô hình hóa tinh vi hơn. Ở Mục 6.3.2, ta sẽ bàn thêm về vấn đề này.



Hình 6.3: Cấu trúc post-norm (a) và pre-norm (b). $F(\cdot)$ = hàm lõi (*Core Function*), $\text{LNorm}(\cdot)$ = chuẩn hóa tầng, và $\oplus$ = kết nối tắt.

Chú giải của người dịch: Post-norm và pre-norm khác nhau ở đâu?

Hãy so hai dòng: post-norm $\text{LNorm}(F(\mathbf{x}) + \mathbf{x})$ và pre-norm $F(\text{LNorm}(\mathbf{x})) + \mathbf{x}$. Ở pre-norm, đầu vào $\mathbf{x}$ đi "thẳng" lên đầu ra mà không bị biến đổi gì (ánh xạ đồng nhất); xếp chồng nhiều tầng thì có một "đường cao tốc" từ đáy lên đỉnh mạng, nên mạng rất sâu vẫn dễ huấn luyện. Ở post-norm, mọi thứ (kể cả phần $\mathbf{x}$) đều phải đi qua LNorm ở mỗi tầng, nên không còn đường đi thẳng như vậy — khó huấn luyện hơn khi mạng sâu, nhưng có thể biểu đạt tốt hơn.

6.1.5 Mạng nơ-ron truyền thẳng

Việc dùng FFN trong Transformer một phần xuất phát từ thực tế là các đầu ra phức tạp có thể được tạo ra bằng cách biến đổi đầu vào qua các phép phi tuyến. Dù bản thân mô hình tự chú ý đã có một chút tính phi tuyến (trong $\text{Softmax}(\cdot)$), cách phổ biến hơn để đưa tính phi tuyến vào là thêm các tầng có hàm kích hoạt phi tuyến và các phép biến đổi tuyến tính. Với đầu vào $\mathbf{H}_{\text{in}} \in \mathbb{R}^{m \times d}$ và đầu ra $\mathbf{H}_{\text{out}} \in \mathbb{R}^{m \times d}$, hàm $\mathbf{H}_{\text{out}} = \text{FFN}(\mathbf{H}_{\text{in}})$ trong Transformer có dạng

$$\mathbf{H}_{\text{out}} = \mathbf{H}_{\text{hidden}} \mathbf{W}_f + \mathbf{b}_f, \tag{6.36}$$

$$\mathbf{H}_{\text{hidden}} = \text{ReLU}(\mathbf{H}_{\text{in}} \mathbf{W}_h + \mathbf{b}_h), \tag{6.37}$$

trong đó $\mathbf{H}_{\text{hidden}} \in \mathbb{R}^{m \times d_{\text{ffn}}}$ biểu diễn các trạng thái ẩn (*hidden states*), còn $\mathbf{W}_h \in \mathbb{R}^{d \times d_{\text{ffn}}}$, $\mathbf{b}_h \in \mathbb{R}^{d_{\text{ffn}}}$, $\mathbf{W}_f \in \mathbb{R}^{d_{\text{ffn}} \times d}$ và $\mathbf{b}_f \in \mathbb{R}^d$ là các tham số. Đây là một FFN hai tầng, trong đó tầng thứ nhất (hay tầng ẩn) đưa vào tính phi tuyến thông qua $\text{ReLU}(\cdot)$[2] và tầng thứ hai chỉ là một phép biến đổi tuyến tính. Trong Transformer, người ta thường dùng kích thước tầng ẩn lớn hơn. Ví dụ, một lựa chọn phổ biến là $d_{\text{ffn}} = 4d$, tức là kích thước của mỗi biểu diễn ẩn gấp 4 lần kích thước của đầu vào.

Chú giải của người dịch: FFN: "mở rộng rồi thu hẹp"

Với $d = 512$, $d_{\text{ffn}} = 2048$: mỗi hàng (mỗi từ) 1×512 được nhân với $\mathbf{W}_h$ (512×2048) để "mở rộng" thành 2048 số, cho qua ReLU (giữ số dương, biến số âm thành 0, ví dụ $\text{ReLU}([-2 \quad 3]) = [0 \quad 3]$), rồi nhân với $\mathbf{W}_f$ (2048×512) để "thu hẹp" về 512 số. Khác với tự chú ý, FFN xử lý từng từ *riêng rẽ* (các hàng không trộn lẫn với nhau), nhưng cùng một bộ tham số dùng chung cho mọi vị trí. Số tham số của FFN: $512 \cdot 2048 + 2048 + 2048 \cdot 512 + 512 \approx 2{,}1$ triệu, nhiều gấp đôi khoảng $4 \cdot 512^2 \approx 1{,}05$ triệu của tầng chú ý.

Lưu ý rằng việc dùng một tầng con FFN rộng đã chứng tỏ giá trị thực tiễn lớn trong nhiều hệ thống tiên tiến nhất (*state-of-the-art*). Tuy nhiên, hệ quả là số lượng tham số của mô hình bị chi phối bởi FFN. Bảng 6.1 cho thấy số tham số và độ phức tạp thời gian của các mô-đun khác nhau trong một hệ thống Transformer chuẩn. Ta thấy FFN chiếm phần lớn kích thước mô hình khi d_{ffn} lớn, dù chúng không phải là thành phần tốn thời gian nhất. Do đó, với các mô hình Transformer rất lớn, ta mong muốn giải quyết vấn đề này để xây dựng những hệ thống hiệu quả hơn.

6.1.6 Chú ý phía bộ giải mã

Một tầng giải mã gồm hai tầng con chú ý: tầng thứ nhất là tầng con tự chú ý, tầng thứ hai là tầng con chú ý chéo. Các tầng con này dựa trên cấu trúc post-norm hoặc pre-norm, nhưng khác nhau ở cách định nghĩa hàm chú ý. Chẳng hạn, xét cấu trúc post-norm mô tả ở Phương trình (6.34). Ta có thể định nghĩa

[2] $\text{ReLU}(x) = \max\{0, x\}$.

tầng con chú ý chéo và tầng con tự chú ý của một tầng giải mã là

$$\begin{aligned}\mathbf{S}_{\text{cross}} &= \text{Layer}_{\text{cross}}(\mathbf{H}_{\text{enc}}, \mathbf{S}_{\text{self}}) \\ &= \text{LNorm}(\text{Att}_{\text{cross}}(\mathbf{H}_{\text{enc}}, \mathbf{S}_{\text{self}}) + \mathbf{S}_{\text{self}}), \qquad (6.38)\\ \mathbf{S}_{\text{self}} &= \text{Layer}_{\text{self}}(\mathbf{S}) \\ &= \text{LNorm}(\text{Att}_{\text{self}}(\mathbf{S}) + \mathbf{S}), \qquad (6.39)\end{aligned}$$

trong đó $\mathbf{S} \in \mathbb{R}^{n\times d}$ là đầu vào của tầng con tự chú ý, $\mathbf{S}_{\text{cross}} \in \mathbb{R}^{n\times d}$ và $\mathbf{S}_{\text{self}} \in \mathbb{R}^{n\times d}$ là đầu ra của các tầng con, còn $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{m\times d}$ là đầu ra của bộ mã hóa[3].

Bảng 6.1: Số tham số và độ phức tạp thời gian của các mô-đun Transformer khác nhau trong các thiết lập khác nhau. m = độ dài chuỗi nguồn, n = độ dài chuỗi đích, d = số chiều mặc định của một tầng ẩn, d_{ffn} = số chiều của tầng ẩn FFN, τ = số đầu trong các mô hình chú ý, và L = số tầng mã hóa hoặc giải mã. Cột $\times$ cho biết số lần một mô hình con được áp dụng ở phía bộ mã hóa hoặc bộ giải mã. Độ phức tạp thời gian được ước lượng bằng cách đếm số phép nhân dấu phẩy động.

	Mô hình con	Số tham số	Độ phức tạp thời gian	$\times$
Bộ mã hóa	Tự chú ý đa đầu	$4d^2$	$O(m^2 \cdot d)$	L
	Mạng truyền thẳng	$2d \cdot d_{\text{ffn}} + d + d_{\text{ffn}}$	$O(m \cdot d \cdot d_{\text{ffn}})$	L
	Chuẩn hóa tầng	$2d$	$O(m \cdot d)$	$2L$
Bộ giải mã	Tự chú ý đa đầu	$4d^2$	$O(n^2 \cdot d)$	L
	Chú ý chéo đa đầu	$4d^2$	$O(m \cdot n \cdot d)$	L
	Mạng truyền thẳng	$2d \cdot d_{\text{ffn}} + d + d_{\text{ffn}}$	$O(n \cdot d \cdot d_{\text{ffn}})$	L
	Chuẩn hóa tầng	$2d$	$O(n \cdot d)$	$3L$

Chú giải của người dịch: Đọc Bảng 6.1

Ký hiệu $O(\cdot)$ ("bậc của") cho biết khối lượng tính toán tăng theo các kích thước như thế nào, bỏ qua hằng số. Ví dụ, tự chú ý tốn $O(m^2 \cdot d)$ vì ma trận $\mathbf{H}^q[\mathbf{H}^k]^\top$ có $m \times m$ ô, mỗi ô là một tích vô hướng d chiều (cần d phép nhân): câu dài gấp đôi thì phần này tốn gấp bốn. Số tham số $4d^2$ của tự chú ý đến từ bốn ma trận $d \times d$: $\mathbf{W}^q, \mathbf{W}^k, \mathbf{W}^v$ và $\mathbf{W}_c$. Chuẩn hóa tầng có $2d$ tham số là $\boldsymbol{\alpha}$ và $\boldsymbol{\beta}$; mỗi tầng mã hóa có 2 khối chuẩn hóa, mỗi tầng giải mã có 3.

Giống như các mô hình chú ý thông thường, chú ý chéo chủ yếu được dùng để mô hình hóa sự tương ứng giữa chuỗi phía nguồn và chuỗi phía đích. Hàm $\text{Att}_{\text{cross}}(\cdot)$ dựa trên mô hình chú ý QKV, mô hình này sinh đầu ra bằng cách truy vấn một tập các cặp khóa–giá trị. Cụ thể hơn, ta định nghĩa truy vấn, khóa và giá trị là các ánh xạ tuyến tính của $\mathbf{S}_{\text{self}}$ và $\mathbf{H}_{\text{enc}}$ như sau

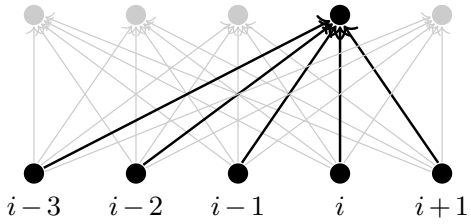
$$\mathbf{S}^q_{\text{self}} = \mathbf{S}_{\text{self}}\mathbf{W}^q_{\text{cross}}, \qquad (6.40)$$
$$\mathbf{H}^k_{\text{enc}} = \mathbf{H}_{\text{enc}}\mathbf{W}^k_{\text{enc}}, \qquad (6.41)$$
$$\mathbf{H}^v_{\text{enc}} = \mathbf{H}_{\text{enc}}\mathbf{W}^v_{\text{enc}}, \qquad (6.42)$$

trong đó $\mathbf{W}^q_{\text{cross}}, \mathbf{W}^k_{\text{enc}}, \mathbf{W}^v_{\text{enc}} \in \mathbb{R}^{d\times d}$ là tham số của các ánh xạ. Nói cách khác, truy vấn được định nghĩa dựa trên $\mathbf{S}_{\text{self}}$, còn khóa và giá trị được định nghĩa dựa trên $\mathbf{H}_{\text{enc}}$.

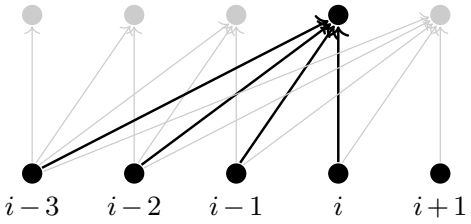
Khi đó, $\text{Att}_{\text{cross}}(\cdot)$ được định nghĩa là

$$\text{Att}_{\text{cross}}(\mathbf{H}_{\text{enc}}, \mathbf{S}_{\text{self}}) = \text{Att}_{\text{qkv}}(\mathbf{S}^q_{\text{self}}, \mathbf{H}^k_{\text{enc}}, \mathbf{H}^v_{\text{enc}})$$

[3] Với một bộ mã hóa có L tầng mã hóa, $\mathbf{H}_{\text{enc}} = \mathbf{H}^L$.



(a) Encoder-side Self-attention



(b) Decoder-side Self-attention

Hình 6.4: Tự chú ý ở phía bộ mã hóa (a) và phía bộ giải mã (b). Mỗi đường nối một đầu vào với một đầu ra của mô hình tự chú ý, cho biết trạng thái đầu ra phụ thuộc vào trạng thái đầu vào đó. Với tự chú ý của bộ mã hóa, đầu ra tại bất kỳ vị trí nào cũng được tính khi có quyền truy cập toàn bộ chuỗi. Ngược lại, với tự chú ý của bộ giải mã, đầu ra tại vị trí i được tính khi chỉ nhìn thấy các đầu vào ở các vị trí từ i trở về trước.

$$= \text{Softmax}\left(\frac{\mathbf{S}^q_{\text{self}}[\mathbf{H}^k_{\text{enc}}]^\top}{\sqrt{d}}\right)\mathbf{H}^v_{\text{enc}}. \tag{6.43}$$

Chú giải của người dịch: Chú ý chéo: ma trận $n \times m$

Ở đây truy vấn đến từ phía đích (n hàng) còn khóa và giá trị đến từ phía nguồn (m hàng), nên $\mathbf{S}^q_{\text{self}}[\mathbf{H}^k_{\text{enc}}]^\top$ là ma trận $n \times m$: hàng i cho biết từ đích thứ i "nhìn" vào từng từ nguồn nhiều hay ít. Ví dụ khi dịch "I love cats" thành "Tôi yêu mèo", lúc sinh từ *mèo*, hàng tương ứng sẽ dồn phần lớn trọng số vào *cats*. Kết quả nhân với $\mathbf{H}^v_{\text{enc}}$ ($m \times d$) cho ma trận $n \times d$, cùng kích thước với $\mathbf{S}_{\text{self}}$ nên cộng kết nối tắt được.

Hàm $\text{Att}_{\text{self}}(\cdot)$ có dạng tương tự $\text{Att}_{\text{cross}}(\cdot)$, với các ánh xạ tuyến tính của $\mathbf{S}$ được lấy làm truy vấn, khóa và giá trị, như sau

$$\begin{aligned}\text{Att}_{\text{self}}(\mathbf{S}) &= \text{Att}_{\text{qkv}}(\mathbf{S}^q, \mathbf{S}^k, \mathbf{S}^v)\\ &= \text{Softmax}\Big(\frac{\mathbf{S}^q[\mathbf{S}^k]^\top}{\sqrt{d}} + \mathbf{M}\Big)\mathbf{S}^v,\end{aligned} \tag{6.44}$$

trong đó $\mathbf{S}^q = \mathbf{S}\mathbf{W}^q_{\text{dec}}$, $\mathbf{S}^k = \mathbf{S}\mathbf{W}^k_{\text{dec}}$ và $\mathbf{S}^v = \mathbf{S}\mathbf{W}^v_{\text{dec}}$ là các ánh xạ tuyến tính của $\mathbf{S}$ với tham số $\mathbf{W}^q_{\text{dec}}, \mathbf{W}^k_{\text{dec}}, \mathbf{W}^v_{\text{dec}} \in \mathbb{R}^{d\times d}$.

Dạng này tương tự Phương trình (6.20). Tuy nhiên, một điểm khác biệt then chốt so với tự chú ý của bộ mã hóa là mô hình ở đây phải tuân theo quy tắc sinh từ trái sang phải (xem Hình 6.4). Tức là, với một từ phía đích ở vị trí i, mô hình chỉ được nhìn thấy các từ phía đích trong ngữ cảnh bên trái $y_1, \ldots, y_{i-1}$. Để đảm bảo điều này, ta cộng một biến mặt nạ (*masking variable*) $\mathbf{M}$ vào ma trận trọng số chưa chuẩn hóa $\frac{\mathbf{S}^q[\mathbf{S}^k]^\top}{\sqrt{d}}$. Cả $\mathbf{M}$ và ma trận trọng số đều có kích thước $n \times n$, nên các giá trị âm lớn trong $\mathbf{M}$ sẽ triệt tiêu các điểm chú ý tương ứng. Để ngăn việc chú ý tới ngữ cảnh bên phải (các từ tương lai) tại bước i, $\mathbf{M}$ được định nghĩa là:

$$M(i,j) = \begin{cases}0 & i \geq j\\ -\infty & i < j\end{cases}, \tag{6.45}$$

trong đó $M(i,j)$ là một số hạng độ lệch cho điểm căn chỉnh giữa vị trí i và j. Dưới đây là một ví dụ về cách áp dụng biến mặt nạ (giả sử $n = 4$).

$$\text{Softmax}\Big(\frac{\mathbf{S}^q[\mathbf{S}^k]^\top}{\sqrt{d}} + \mathbf{M}\Big)$$

$$
\begin{aligned}
&= \text{Softmax}\left(\begin{bmatrix} 2 & 0.1 & 1 & 1 \\ 0 & 0.9 & 0.9 & 0.9 \\ 0.2 & 0.8 & 0.7 & 2 \\ 0.3 & 1 & 0.3 & 3 \end{bmatrix} + \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}\right) \\
&= \text{Softmax}\left(\begin{bmatrix} 2 & -\infty & -\infty & -\infty \\ 0 & 0.9 & -\infty & -\infty \\ 0.2 & 0.8 & 0.7 & -\infty \\ 0.3 & 1 & 0.3 & 3 \end{bmatrix}\right) \\
&= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.3 & 0.7 & 0 & 0 \\ 0.2 & 0.4 & 0.4 & 0 \\ 0.05 & 0.1 & 0.05 & 0.8 \end{bmatrix}.
\end{aligned} \tag{6.46}
$$

Chú giải của người dịch: Vì sao cộng $-\infty$ lại "che" được tương lai?

Softmax lấy $e^{(\cdot)}$ của từng số, mà $e^{-\infty} = 0$ (thực tế người ta dùng một số âm rất lớn như -10^9, khi đó $e^{-10^9} \approx 0$). Vì vậy mọi ô bị cộng $-\infty$ đều nhận trọng số 0 sau Softmax, và phần còn lại của hàng tự chia lại cho có tổng bằng 1. Kiểm tra hàng 2: chỉ còn $[0 \quad 0.9]$, $e^0 = 1$, $e^{0.9} \approx 2.46$, nên trọng số là $1/3.46 \approx 0.29 \approx 0.3$ và $2.46/3.46 \approx 0.71 \approx 0.7$, khớp với Phương trình (6.46). Mặt nạ có dạng "tam giác dưới": hàng i chỉ được mở tới cột i, đúng như học sinh làm bài thi không được xem đáp án các câu sau. Nhờ mặt nạ, khi huấn luyện ta vẫn tính song song cho cả n vị trí cùng lúc mà không vị trí nào "nhìn trộm" từ tương lai.

Như đã nói ở Mục 6.1.3, có thể dễ dàng cải tiến các mô hình này bằng cơ chế chú ý đa đầu. Ngoài ra, vì bộ giải mã thường là thành phần tốn thời gian nhất trong các ứng dụng thực tế, hiệu quả tính toán tổng thể của các hệ thống này phụ thuộc nhiều vào việc tối ưu chú ý phía bộ giải mã.

6.1.7 Huấn luyện và suy luận

Transformer có thể được huấn luyện và sử dụng theo cách tiêu chuẩn. Ví dụ, ta có thể huấn luyện một mô hình Transformer bằng hạ gradient (*gradient descent*) để cực tiểu hóa một hàm mất mát (*loss function*) trên dữ liệu huấn luyện (xem Chương 2), và kiểm tra mô hình đã huấn luyện bằng tìm kiếm chùm (*beam search*) trên dữ liệu chưa thấy (xem Chương 5). Dưới đây chúng tôi trình bày một số kỹ thuật thường được dùng trong huấn luyện và suy luận (*inference*) với mô hình Transformer.

- **Lập lịch tốc độ học** (*Learning Rate Scheduling*). Giống như các mạng nơ-ron tiêu chuẩn, Transformer có thể được huấn luyện trực tiếp bằng lan truyền ngược (*back-propagation*). Quá trình huấn luyện thường được lặp lại rất nhiều lần để mô hình khớp tốt với dữ liệu huấn luyện. Ở mỗi bước huấn luyện, ta cập nhật trọng số của mạng nơ-ron bằng cách dịch chúng một bước nhỏ theo hướng ngược với gradient của hàm mất mát. Có nhiều cách thiết kế quy tắc cập nhật cho huấn luyện. Một lựa chọn phổ biến là phương pháp tối ưu Adam [Kingma và Ba, 2014]. Để điều chỉnh tốc độ học (*learning rate*) trong quá trình huấn luyện, Vaswani và cộng sự [2017] đưa ra một chiến lược lập lịch tốc độ học: tăng tốc độ học tuyến tính trong một số bước rồi giảm dần. Họ thiết kế tốc độ học có dạng

$$
\eta = \eta_0 \cdot \min\left\{n_{\text{step}}^{-0.5},\ n_{\text{step}} \cdot (n_{\text{warmup}})^{-1.5}\right\}, \tag{6.47}
$$

  trong đó η_0 là tốc độ học ban đầu, n_{step} là số bước huấn luyện đã thực hiện, và n_{warmup} là số bước khởi động (*warmup*). Trong n_{warmup} bước đầu tiên, tốc độ học η tăng dần khi quá trình huấn luyện

tiến triển. Nó đạt giá trị cao nhất tại thời điểm $n_{\text{step}} = n_{\text{warmup}}$, rồi giảm theo hàm nghịch đảo căn bậc hai (tức là $\eta_0 \cdot n_{\text{step}}^{-0.5}$).

Chú giải của người dịch: Thử số với Phương trình (6.47)

Lấy $n_{\text{warmup}} = 4000$. Hai biểu thức trong min là $1/\sqrt{n_{\text{step}}}$ (giảm dần) và $n_{\text{step}}/4000^{1.5}$ (tăng tuyến tính). Tại $n_{\text{step}} = 1000$: $1/\sqrt{1000} \approx 0.0316$ và $1000/252982 \approx 0.0040$, lấy số nhỏ hơn là 0.0040 (đang ở pha tăng). Tại $n_{\text{step}} = 4000$: cả hai đều ≈ 0.0158 (đỉnh). Tại $n_{\text{step}} = 16000$: $1/\sqrt{16000} \approx 0.0079$, chỉ còn một nửa đỉnh. Lúc đầu tham số còn ngẫu nhiên, bước đi quá lớn dễ làm huấn luyện "bung"; vì thế ta "khởi động" chậm như khởi động trước khi chạy bộ.

- **Chia lô và đệm** (*Batching and Padding*). Để cân bằng giữa tối ưu toàn cục và sự hội tụ của huấn luyện, người ta thường cập nhật trọng số bằng một tập mẫu tương đối nhỏ, gọi là một **lô nhỏ** (*minibatch*). Vì vậy, ta có thể xét phiên bản theo lô của quá trình tính xuôi và tính ngược, trong đó toàn bộ lô nhỏ được dùng cùng lúc để thu thông tin gradient. Một ưu điểm của chia lô là nó cho phép hệ thống tận dụng các phép toán tensor hiệu quả để xử lý nhiều chuỗi trong một lần chạy. Điều này đòi hỏi mọi chuỗi đầu vào trong một lô nhỏ được lưu trong một khối bộ nhớ duy nhất để có thể được đọc và xử lý đồng thời. Để minh họa ý tưởng này, xét một lô nhỏ gồm bốn mẫu có phía nguồn là:

A	B	C	D	E	F
M	N				
R	S	T			
W	X	Y	Z		

Ta có thể lưu các chuỗi này trong một khối liên tục 4×6, trong đó mỗi "hàng" biểu diễn một chuỗi, như sau

A	B	C	D	E	F
M	N	□	□	□	□
R	S	T	□	□	□
W	X	Y	Z	□	□

Ở đây, các từ đệm (*padding words*) □ được nối vào cuối các chuỗi ngắn hơn để mọi chuỗi được căn thẳng hàng trong bộ nhớ. Thông thường, ta không muốn phần đệm ảnh hưởng tới hoạt động của hệ thống, nên có thể đơn giản định nghĩa □ là một vectơ không (gọi là **đệm không** (*zero padding*)). Mặt khác, trong một số trường hợp ta lại muốn dùng phần đệm để mô tả điều gì đó mà các chuỗi đầu vào không bao quát. Ví dụ, ta có thể thay các từ đệm bằng các từ ngữ cảnh ở bên trái (hoặc bên phải) của một chuỗi, dù điều này có thể đòi hỏi sửa đổi hệ thống để bảo đảm các từ ngữ cảnh mới thêm vào không làm xuất hiện nội dung ngoài ý muốn ở đầu ra.

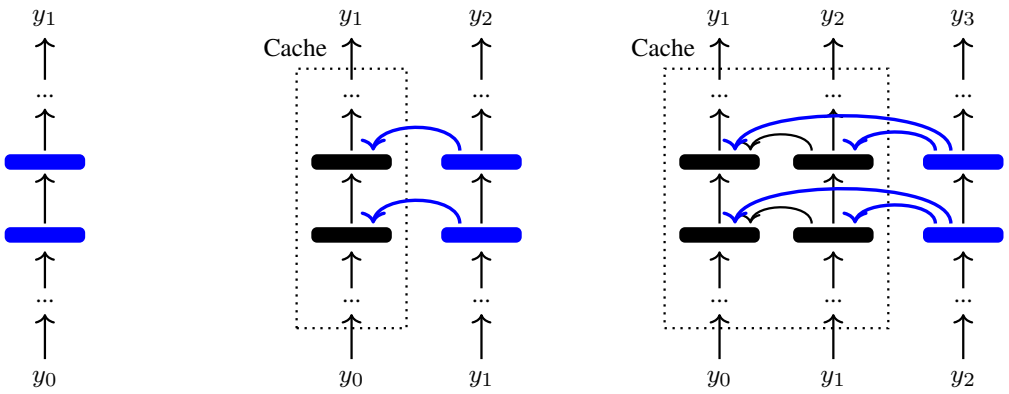
Chú giải của người dịch: Đệm và mặt nạ

Máy tính xử lý nhanh nhất khi dữ liệu là một "bảng" chữ nhật đều đặn, giống như xếp học sinh thành các hàng ghế bằng nhau: hàng nào thiếu người thì đặt ghế trống. Trong thực tế, ngoài việc đặt vectơ đệm bằng 0, người ta còn dùng một mặt nạ (giống $\mathbf{M}$ ở Phương trình (6.45)) với giá trị $-\infty$ tại các cột ứng với □, để không từ thật nào "chú ý" vào ghế trống.

- **Tìm kiếm và bộ nhớ đệm** (*Search and Caching*). Khi kiểm tra, ta cần tìm kiếm trong không gian các giả thuyết ứng viên (hay các chuỗi phía đích ứng viên) để xác định giả thuyết có điểm cao nhất

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \text{score}(\mathbf{x}, \mathbf{y}), \tag{6.48}$$

trong đó score$(\mathbf{x}, \mathbf{y})$ là điểm của mô hình cho chuỗi phía đích $\mathbf{y}$ khi biết chuỗi phía nguồn $\mathbf{x}$. Dù có nhiều thuật toán tìm kiếm được thiết kế cho mục đích này, phần lớn có cấu trúc tương tự nhau: chương trình tìm kiếm hoạt động bằng cách mở rộng một tập các chuỗi phía đích ứng viên, mỗi lần thêm một token. Theo cách này, thuật toán thu được có thể được xem là một thủ tục sinh từ trái sang phải. Để thảo luận chi tiết hơn về các thuật toán tìm kiếm và điểm mô hình của các mô hình chuỗi-sang-chuỗi tổng quát, xem Chương 5. Lưu ý rằng mọi thiết kế của score$(\mathbf{x}, \mathbf{y})$, dù phức tạp đến đâu, đều dựa trên việc tính $\Pr(\mathbf{y}|\mathbf{x})$. Vì các mô hình chú ý dùng trong Transformer đòi hỏi tính tích vô hướng của mọi cặp vectơ đầu vào trong một tầng, độ phức tạp thời gian của thuật toán tìm kiếm là một hàm bậc hai theo độ dài của $\mathbf{y}$. Do đó, việc tính đi tính lại đầu ra của các mô hình chú ý cho những vị trí đã xử lý là không hiệu quả. Vấn đề này có thể được giải quyết bằng cách lưu vào bộ nhớ đệm (*cache*) các trạng thái của mỗi tầng cho những từ đã thấy. Hình 6.5 minh họa việc dùng cơ chế bộ nhớ đệm trong một bước tìm kiếm. Mọi trạng thái của các vị trí $< i$ được lưu giữ và dễ dàng truy cập trong bộ nhớ đệm. Tại vị trí i, ta chỉ cần tính các trạng thái cho từ mới được thêm vào rồi cập nhật bộ nhớ đệm.



(a) decoding step 1 (b) decoding step 2 (c) decoding step 3

Hình 6.5: Minh họa cơ chế bộ nhớ đệm (*Cache*) trong bộ giải mã Transformer, ở các bước giải mã 1, 2, 3 (a–c). Các hình chữ nhật biểu thị trạng thái của các tầng giải mã hoặc tầng con. Tại bước i, mọi trạng thái từ các bước trước được lưu trong bộ nhớ đệm (xem các khung nét chấm), và ta chỉ cần tính các trạng thái cho bước hiện tại (xem các hình chữ nhật và mũi tên màu xanh). Sau đó ta thêm các trạng thái mới sinh vào bộ nhớ đệm và chuyển sang bước $i + 1$.

Chú giải của người dịch: Vì sao bộ nhớ đệm giúp tiết kiệm?

Khi sinh từ thứ i, nhờ mặt nạ, trạng thái của các vị trí $1, \ldots, i-1$ không phụ thuộc vào từ mới, nên chúng y hệt như ở bước trước. Không có bộ nhớ đệm, để sinh câu 100 từ ta phải tính lại $1 + 2 + \cdots + 100 = 5050$ "lượt vị trí"; có bộ nhớ đệm, mỗi bước chỉ tính 1 vị trí mới, tổng cộng 100 lượt (phép chú ý của vị trí mới vẫn phải đọc lại các khóa, giá trị đã lưu). Đây chính là ý tưởng của *bộ nhớ đệm KV* sẽ gặp lại nhiều lần trong các chương sau.

6.2 Mô hình có nhận biết cú pháp

Mặc dù Transformer là các mô hình học sâu không sử dụng một cách tường minh các cấu trúc hay giả định ngôn ngữ học, việc đưa tri thức có sẵn của chúng ta vào những hệ thống như vậy có thể vẫn cần thiết. Điều này một phần là vì các nhà nghiên cứu NLP từ lâu đã tin rằng cần một mức trừu tượng cao hơn trong biểu diễn dữ liệu để phát triển các hệ thống NLP lý tưởng, và đã có rất nhiều hệ thống sử dụng tiên nghiệm cấu trúc (*structural priors*). Tuy nhiên, cấu trúc là một chủ đề rộng. Để thảo luận về các loại cấu trúc khác nhau mà ta có thể xét tới, xem công trình của See [2018]. Chẳng hạn, các thiên kiến quy nạp (*inductive biases*) dùng trong thiết kế mô hình có thể được xem như một dạng tiên nghiệm cấu trúc, và các mô hình NLP cũng có thể tự học cấu trúc tiềm ẩn của bài toán. Trong tiểu mục này, chúng ta sẽ bàn về một số vấn đề như vậy. Ta sẽ tập trung vào các phương pháp đưa cấu trúc ngôn ngữ học vào mô hình Transformer. Vì Transformer có thể được áp dụng cho nhiều tác vụ NLP với định dạng đầu vào và đầu ra rất khác nhau, chúng tôi sẽ chủ yếu bàn về các sửa đổi cho bộ mã hóa Transformer (gọi là **bộ mã hóa Transformer có nhận biết cú pháp** (*syntax-aware Transformer encoders*)). Tuy vậy, thảo luận của chúng tôi mang tính tổng quát, và các phương pháp này có thể dễ dàng mở rộng cho bộ giải mã Transformer.

Chú giải của người dịch: Cú pháp là gì?

Cú pháp là cách các từ kết hợp với nhau thành cụm và câu theo quy tắc ngữ pháp. Ví dụ trong câu "Con mèo đen ngủ", "con mèo đen" là một *cụm danh từ* (làm chủ ngữ), "ngủ" là *động từ* (làm vị ngữ). Các nhãn như vậy (từ loại, quan hệ chủ–vị…) chính là "tri thức ngôn ngữ học" mà mục này muốn đưa thêm vào Transformer.

6.2.1 Đầu vào và đầu ra có nhận biết cú pháp

Một trong những phương pháp đơn giản nhất để đưa cấu trúc vào hệ thống NLP là sửa đổi chuỗi đầu vào, còn hệ thống giữ nguyên. Lấy một ví dụ đơn giản, xét một câu trong đó mỗi từ x_j được gán một tập gồm κ nhãn cú pháp $\{\text{tag}_j^1, \cdots, \text{tag}_j^\kappa\}$ (chẳng hạn nhãn từ loại (*part-of-speech, POS*) và nhãn quan hệ phụ thuộc (*dependency*)). Ta có thể ghép nối các ký hiệu này để định nghĩa một "từ" mới

$$x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa$$

Khi đó, vectơ nhúng của từ này được cho bởi

$$\mathbf{e}_j \;=\; e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa) + \text{PE}(j), \tag{6.49}$$

trong đó $e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa) \in \mathbb{R}^d$ là vectơ nhúng của $x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa$. Vì $x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa$ là một ký hiệu phức hợp, ta phân rã bài toán học $e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa)$ thành các bài toán dễ hơn. Ví dụ, ta có thể xây dựng κ mô hình nhúng, mỗi mô hình sinh vectơ nhúng cho một loại nhãn. Khi đó ta viết $e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa)$ dưới dạng tổng của vectơ nhúng từ và các vectơ nhúng nhãn

$$e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa) \;=\; \mathbf{x}_j + e(\text{tag}_j^1) + \cdots + e(\text{tag}_j^\kappa), \tag{6.50}$$

trong đó $\{e(\text{tag}_j^1), \cdots, e(\text{tag}_j^\kappa)\}$ là các vectơ nhúng của các nhãn. Một cách khác là kết hợp các vectơ nhúng này qua một mạng nơ-ron có dạng

$$e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa) \;=\; \text{FFN}_{\text{embed}}(\mathbf{x}_j, e(\text{tag}_j^1), \cdots, e(\text{tag}_j^\kappa)), \tag{6.51}$$

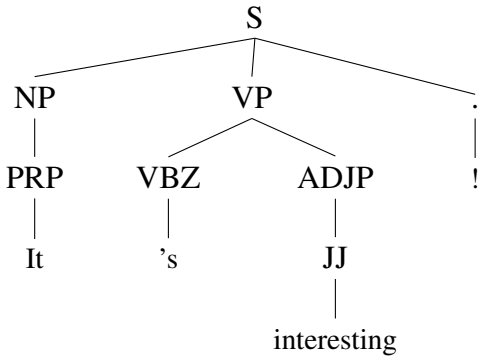
trong đó $\text{FFN}_{\text{embed}}(\cdot)$ là một mạng nơ-ron truyền thẳng có một hoặc hai tầng.

Chú giải của người dịch: Ví dụ về "từ" gắn nhãn

Với từ "cats" có nhãn từ loại NNS (danh từ số nhiều) và nhãn phụ thuộc *dobj* (tân ngữ trực tiếp), "từ" mới là `cats/NNS/dobj`. Nếu học riêng một vectơ cho mỗi tổ hợp như vậy thì số tổ hợp sẽ bùng nổ (mỗi từ $\times$ mỗi nhãn từ loại $\times$ mỗi nhãn phụ thuộc). Phương trình (6.50) tránh điều này: chỉ cần một vectơ cho "cats", một cho "NNS", một cho "dobj" rồi cộng lại, giống cách vectơ vị trí được cộng vào vectơ từ ở Phương trình (6.13).

Ta có thể áp dụng kỹ thuật tương tự ở phía bộ giải mã, coi $y_i/\text{tag}_i^1/\cdots/\text{tag}_i^\kappa$ là một từ đích được bổ sung cú pháp. Tuy nhiên, làm như vậy có thể làm tăng mạnh kích thước bộ từ vựng phía đích, gây ra thách thức tính toán cho cả huấn luyện lẫn suy luận.

Một cách phổ biến khác để biểu diễn một câu là thông qua cây cú pháp (*syntax tree*). Trong ngôn ngữ học, cú pháp của một câu có thể được diễn giải theo nhiều cách khác nhau, dẫn tới nhiều loại ngữ pháp và các biểu diễn dựa trên cây (hoặc đồ thị) tương ứng. Dù các biểu diễn này khác nhau ở dạng cú pháp cụ thể, một cách tiếp cận chung để sử dụng chúng trong mô hình hóa chuỗi là **tuyến tính hóa cây** (*tree linearization*). Xét câu sau đây được chú giải bằng một cây phân tích thành phần (*constituency-based parse tree*):



Ta có thể viết cấu trúc cây này thành một chuỗi gồm các từ, nhãn cú pháp và dấu ngoặc bằng một thuật toán duyệt cây, như sau

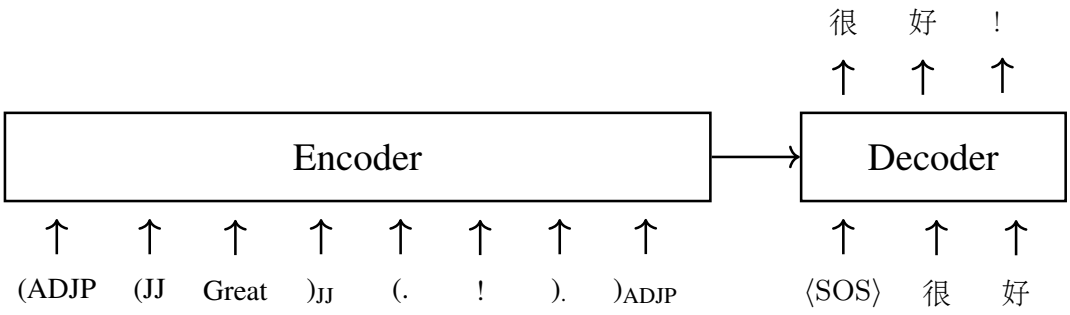
(S	(NP	(PRP	It	)$_{\text{PRP}}$	)$_{\text{NP}}$	(VP	(VBZ	's	)$_{\text{VBZ}}$
(ADJP	(JJ	interesting	)$_{\text{JJ}}$	)$_{\text{ADJP}}$	)$_{\text{VP}}$	(.	!	).	)$_{\text{S}}$

Chú giải của người dịch: Đọc cây cú pháp "It's interesting!"

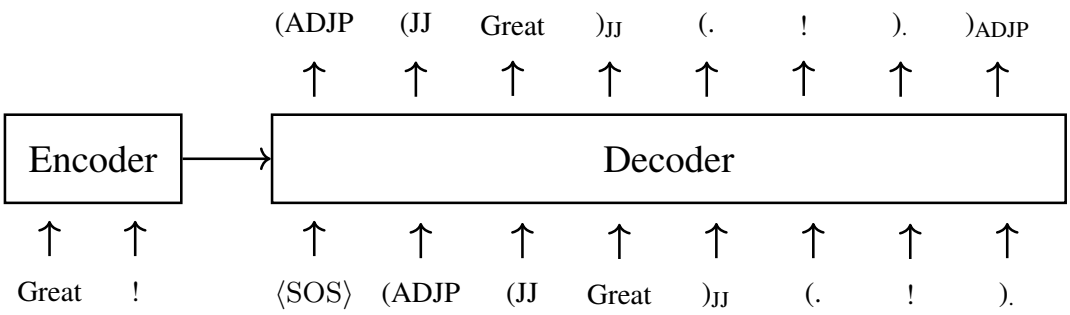
Câu tiếng Anh "It's interesting!" (Thật thú vị!) được giữ nguyên vì ví dụ minh họa chính cấu trúc tiếng Anh. Các nhãn: S = câu, NP = cụm danh từ, VP = cụm động từ, ADJP = cụm tính từ, PRP = đại từ ("It"), VBZ = động từ ngôi thứ ba số ít ("'s" = "is"), JJ = tính từ, "." = dấu câu. Duyệt cây từ trên xuống, từ trái sang phải: gặp một nút thì viết "(nhãn", đi hết các con của nó thì viết ")nhãn". Giống như viết biểu thức toán có ngoặc lồng nhau, chuỗi ngoặc này chứa đủ thông tin để dựng lại cây.

Chuỗi các token cú pháp này có thể được dùng trực tiếp làm đầu vào cho bộ mã hóa, tức là mỗi token được biểu diễn bằng tổng của vectơ nhúng từ và vectơ nhúng vị trí của nó, và vectơ nhúng kết hợp này được coi như một đầu vào tiêu chuẩn của bộ mã hóa. Một ứng dụng của cây tuyến tính hóa là dịch máy (*machine translation*) cây-sang-chuỗi (*tree-to-string*), trong đó một cây cú pháp ở một ngôn ngữ được dịch thành một chuỗi ở ngôn ngữ khác [Li và cộng sự, 2017b; Currey và Heafield, 2018]. Cây tuyến tính hóa cũng có thể được dùng để sinh cây. Ví dụ, các tác vụ phân tích cú pháp (*parsing*) có thể

được phát biểu thành các bài toán chuỗi-sang-chuỗi, ánh xạ một văn bản đầu vào thành biểu diễn tuần tự của cây cú pháp tương ứng [Vinyals và cộng sự, 2015; Choe và Charniak, 2016]. Hình 6.6 minh họa các mô hình này. Cần lưu ý rằng các phương pháp mô tả ở đây không dành riêng cho Transformer và có thể được áp dụng cho nhiều kiến trúc dựa trên chuỗi, chẳng hạn các mô hình dựa trên RNN.



(a) Tree-to-String Machine Translation



(b) Constituency Parsing

Hình 6.6: Minh họa tuyến tính hóa cây ở phía bộ mã hóa hoặc phía bộ giải mã. (a) Với dịch máy cây-sang-chuỗi (*Tree-to-String Machine Translation*), bộ mã hóa nhận một biểu diễn tuần tự của cây phân tích đầu vào, và bộ giải mã cho ra bản dịch tương ứng (ở đây là câu tiếng Trung *hěn hǎo!*, nghĩa là "Rất tốt!", bản dịch của "Great!"). (b) Với phân tích cú pháp (*Constituency Parsing*), bộ mã hóa nhận một câu, và bộ giải mã cho ra cây cú pháp tương ứng.

6.2.2 Mô hình chú ý có nhận biết cú pháp

Với mô hình Transformer, việc dùng cây cú pháp để định hướng quá trình học biểu diễn chuỗi cũng là điều tự nhiên. Ở mục trước, ta đã thấy biểu diễn của một chuỗi có thể được tính bằng cách liên hệ các vị trí khác nhau trong chuỗi đó. Điều này cho phép ta áp đặt cấu trúc lên các phân phối chú ý trên các vị trí. Để làm vậy, ta dùng tự chú ý của bộ mã hóa với một mặt nạ cộng (*additive mask*)

$$\text{AttSyn}_{\text{self}}(\mathbf{H}) \;=\; \text{Softmax}\Big(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}} + \mathbf{M}\Big)\mathbf{H}^v \tag{6.52}$$

hoặc với một mặt nạ nhân (*multiplicative mask*)

$$\text{AttSyn}_{\text{self}}(\mathbf{H}) \;=\; \text{Softmax}\Big(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}} \odot \mathbf{M}\Big)\mathbf{H}^v, \tag{6.53}$$

trong đó $\mathbf{M} \in \mathbb{R}^{m\times m}$ là một ma trận các biến mặt nạ, trong đó giá trị $M(i,j)$ càng lớn thể hiện tương quan cú pháp giữa vị trí i và j càng mạnh. Trong phần mô tả dưới đây, ta chọn Phương trình (6.53) làm dạng cơ bản.

Một cách phổ biến để thiết kế $\mathbf{M}$ là chiếu các quan hệ cú pháp của cấu trúc cây đầu vào thành các ràng buộc trên chuỗi. Ở đây, ta xét cây phân tích thành phần và cây phân tích phụ thuộc (*dependency parse tree*) để minh họa. Nói chung, có hai loại phương pháp mặt nạ được sử dụng.

- **Mặt nạ 0-1** (*0-1 Masking*). Phương pháp này gán $M(i,j) = 1$ nếu các từ ở vị trí i và j được coi là có tương quan cú pháp, và $M(i,j) = 0$ trong trường hợp ngược lại [Zhang và cộng sự, 2020c; Bai và cộng sự, 2021]. Để mô hình hóa quan hệ giữa hai từ trong cây cú pháp, ta có thể xét khoảng cách giữa các nút tương ứng của chúng. Một trong những dạng đơn giản nhất là

$$M(i,j) \;=\; \begin{cases} 1 & \omega(i,j) \le \omega_{\text{max}} \\ 0 & \text{trường hợp khác} \end{cases}, \tag{6.54}$$

  trong đó $\omega(i,j)$ là độ dài đường đi ngắn nhất giữa các nút ứng với các từ ở vị trí i và j. Ví dụ, với một cây phân tích phụ thuộc, $\omega(i,j)$ là số cạnh phụ thuộc trên đường đi giữa hai từ. Với cây phân tích thành phần, mọi từ đều là nút lá. Do đó, $\omega(i,j)$ biểu thị khoảng cách trên cây giữa hai lá trong cùng một nhánh của cây. ω_{max} là một tham số dùng để kiểm soát khoảng cách tối đa giữa hai nút mà vẫn được coi là có tương quan cú pháp. Ví dụ, giả sử có một cây phân tích phụ thuộc với $\omega_{\text{max}} = 1$, Phương trình (6.54) áp đặt ràng buộc rằng điểm chú ý giữa vị trí i và j chỉ được tính nếu chúng có quan hệ cha–con (từ chi phối–từ phụ thuộc)[4].

- **Mặt nạ mềm** (*Soft Masking*). Thay vì coi $\mathbf{M}$ là một ràng buộc cứng, ta có thể dùng nó như một ràng buộc mềm, co giãn trọng số chú ý giữa vị trí i và j dựa trên mức độ tương quan giữa các từ tương ứng. Một cách tiếp cận là giảm trọng số chú ý khi $\omega(i,j)$ tăng lên. Một phương pháp đơn giản là biến đổi $\omega(i,j)$ sao cho $M(i,j)$ tương quan nghịch với $\omega(i,j)$ và nằm trong đoạn $[0,1]$:

$$M(i,j) \;=\; \text{DNorm}(\omega(i,j)). \tag{6.55}$$

  Có một số thiết kế khác nhau cho $\text{DNorm}(\cdot)$. Ví dụ, ta có thể tính điểm chuẩn hóa của $-\omega(i,j)$ bằng cách trừ đi trung bình và chia cho độ lệch chuẩn [Chen và cộng sự, 2018a], hoặc chuẩn hóa $1/\omega(i,j)$ trên mọi j có thể có trong chuỗi [Xu và cộng sự, 2021b]. Trong trường hợp bộ phân tích cú pháp có thể cho ra một điểm số giữa vị trí i và j, điểm số này cũng có thể được dùng để tính $M(i,j)$. Ví dụ, một bộ phân tích phụ thuộc có thể cho ra xác suất để từ ở vị trí i là cha của từ ở vị trí j [Strubell và cộng sự, 2018]. Khi đó ta có thể viết $M(i,j)$ là

$$M(i,j) \;=\; \text{Pr}_{\text{parent}}(i|j) \tag{6.56}$$

  hoặc

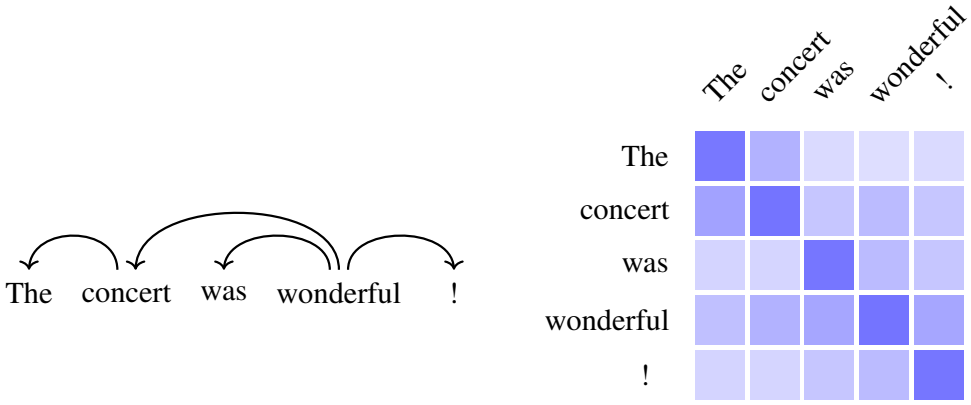
$$M(i,j) \;=\; \max\{\text{Pr}_{\text{parent}}(i|j), \text{Pr}_{\text{parent}}(j|i)\}, \tag{6.57}$$

  trong đó $\text{Pr}_{\text{parent}}(i|j)$ và $\text{Pr}_{\text{parent}}(j|i)$ là các xác suất do bộ phân tích cú pháp đưa ra. Xem Hình 6.7 để có ví dụ về cách suy ra một biến mặt nạ mềm từ một cây phân tích phụ thuộc.

[4] Với mặt nạ nhân, $M(i,j) = 0$ không có nghĩa là trọng số chú ý cuối cùng giữa j và i bằng 0, vì đầu vào bằng 0 của hàm Softmax không cho ra xác suất bằng 0. Một cách để "che" hẳn một phần tử của Softmax $\Big(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}}\Big)$ là dùng mặt nạ cộng và đặt $M(i,j) = -\infty$ nếu $\omega(i,j) > \omega_{\text{max}}$.

Chú giải của người dịch: Mặt nạ cộng và mặt nạ nhân, bằng số

Giả sử điểm chú ý (trước Softmax) của một hàng là $[2 \;\; 1 \;\; 3]$ và mặt nạ 0-1 là $[1 \;\; 1 \;\; 0]$ (từ thứ ba ở "quá xa" trên cây). Mặt nạ nhân: $[2 \;\; 1 \;\; 3] \odot [1 \;\; 1 \;\; 0] = [2 \;\; 1 \;\; 0]$; Softmax cho $\approx [0.67 \;\; 0.24 \;\; 0.09]$ — từ thứ ba vẫn nhận trọng số 0.09, không phải 0, đúng như chú thích chân trang 4 nói. Mặt nạ cộng với $[0 \;\; 0 \;\; -\infty]$: $[2 \;\; 1 \;\; -\infty]$; Softmax cho $\approx [0.73 \;\; 0.27 \;\; 0]$ — che hẳn. Còn với mặt nạ mềm, các số trong $\mathbf{M}$ là số thực trong $[0, 1]$, chẳng hạn $[1 \;\; 0.5 \;\; 0.2]$: từ càng xa trên cây thì điểm bị "thu nhỏ" càng nhiều.



(a) Dependency Parse Tree (b) Mask $\mathbf{M}$ (darker color means larger value)

Hình 6.7: Các tiên nghiệm suy ra từ một cây phân tích phụ thuộc: (a) cây phân tích phụ thuộc của câu "The concert was wonderful!" (Buổi hòa nhạc thật tuyệt vời!); (b) mặt nạ $\mathbf{M}$ (màu càng đậm nghĩa là giá trị càng lớn). Hàng i của ma trận $\mathbf{M}$ biểu diễn một phân phối mô tả ta có thể gán bao nhiêu trọng số cho $M(i, j)$ theo khoảng cách cú pháp giữa i và j.

6.2.3 Mô hình nhiều nhánh

Việc tích hợp cú pháp vào các hệ thống NLP đặt ra một số thách thức. Điều này một phần là vì các cây phân tích tự động có thể chứa lỗi, và một phần là vì việc dùng cú pháp có thể dẫn tới những giả định mạnh về cấu trúc tiềm ẩn của câu. Thay vì kết hợp thông tin cú pháp và thông tin từ vào một mô hình nguyên khối duy nhất, có thể linh hoạt và hiệu quả hơn nếu xây dựng một mô hình để mã hóa thông tin cú pháp và một mô hình khác để mã hóa chuỗi từ. Một cách để đạt được điều này là dùng nhiều mạng nơ-ron (gọi là các **nhánh** (*branches*) hay **đường** (*paths*)), mỗi mạng xử lý một loại đầu vào. Đầu ra của các nhánh này sau đó được kết hợp để tạo ra đầu ra cuối cùng [Xie và cộng sự, 2017; Fan và cộng sự, 2020; Lin và cộng sự, 2022b]. Vì vậy, nhiều phương pháp khác nhau đã được dùng để kết hợp các loại đầu vào khác nhau cho các mô hình nơ-ron như Transformer.

Một cách tiếp cận thường dùng là xây dựng hai bộ mã hóa riêng biệt, trong đó một mô hình được huấn luyện để mã hóa đầu vào cú pháp (ký hiệu là $\mathbf{t}$), và mô hình kia được huấn luyện để mã hóa đầu vào văn bản tiêu chuẩn (ký hiệu là $\mathbf{x}$). Hình 6.8(a) minh họa kiến trúc nhiều bộ mã hóa này. Bộ mã hóa cú pháp $\text{Encode}_{\text{syn}}(\mathbf{t})$ dựa trên các mô hình trình bày ở Mục 6.2.1 và 6.2.2, còn bộ mã hóa văn bản $\text{Encode}_{\text{text}}(\mathbf{x})$ là một bộ mã hóa Transformer tiêu chuẩn. Các biểu diễn do những bộ mã hóa này sinh ra sau đó được đưa vào mô hình kết hợp làm đầu vào và được kết hợp thành một biểu diễn lai:

$$\mathbf{H}_{\text{hybrid}} = \text{Combine}(\mathbf{H}_{\text{syn}}, \mathbf{H}_{\text{text}})$$

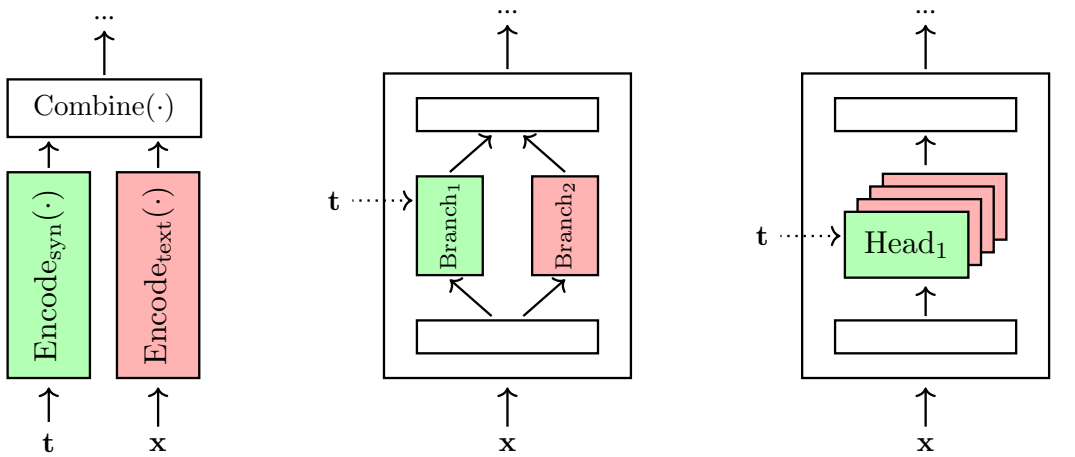
$$= \text{Combine}(\text{Encode}_{\text{syn}}(\mathbf{t}), \text{Encode}_{\text{text}}(\mathbf{x})). \tag{6.58}$$

Có một số thiết kế cho Combine$(\cdot)$, tùy theo tác vụ mà các bộ mã hóa được áp dụng. Ví dụ, nếu ta muốn xây dựng một bộ phân loại (*classifier*) văn bản, Combine$(\cdot)$ có thể là một mạng gộp (*pooling*) đơn giản. Với các tác vụ phức tạp hơn, như dịch máy, Combine$(\cdot)$ cũng có thể là một bộ mã hóa Transformer, và ta có thể hợp nhất thông tin từ các nguồn khác nhau bằng cách thực hiện tự chú ý trên $[\mathbf{H}_{\text{syn}}, \mathbf{H}_{\text{text}}]$.

Chú giải của người dịch: Ghép rồi tự chú ý

Nếu $\mathbf{H}_{\text{syn}}$ có 12 hàng (12 token cú pháp) và $\mathbf{H}_{\text{text}}$ có 5 hàng (5 từ), cùng d cột, thì $[\mathbf{H}_{\text{syn}}, \mathbf{H}_{\text{text}}]$ ở đây được hiểu là xếp chồng chúng thành một ma trận $17 \times d$. Tự chú ý trên ma trận này cho phép mỗi từ "nhìn" vào cả các token cú pháp và ngược lại, nhờ vậy hai nguồn thông tin được trộn lẫn. Còn "gộp" (*pooling*) đơn giản là, chẳng hạn, lấy trung bình các hàng để được một vectơ duy nhất đại diện cho cả câu.

Mặc dù trong mục này chúng ta tập trung chủ yếu vào các mô hình cú pháp, kiến trúc nhiều bộ mã hóa tổng quát có thể được dùng trong nhiều tác vụ cần đầu vào từ các nguồn bổ sung. Ví dụ, ta có thể dùng một bộ mã hóa để biểu diễn một câu và một bộ mã hóa khác để biểu diễn câu đứng trước nó trong cùng văn bản. Kết hợp hai bộ mã hóa này sẽ cho một mô hình có nhận biết ngữ cảnh [Voita và cộng sự, 2018; Li và cộng sự, 2020a]. Hơn nữa, kiến trúc của các bộ mã hóa không nhất thiết bị giới hạn ở Transformer. Ta có thể chọn các mô hình khác nhau cho các nhánh khác nhau. Chẳng hạn, trong một kiến trúc mã hóa hai nhánh được dùng rộng rãi, ta có thể dùng một bộ mã hóa dựa trên CNN để mô hình hóa ngữ cảnh cục bộ và một bộ mã hóa Transformer để mô hình hóa ngữ cảnh toàn cục [Wu và cộng sự, 2020b].



(a) Multi-encoder (b) Multi-branch as a Sub-model (c) Multi-head Attention

Hình 6.8: Các kiến trúc nhiều nhánh: (a) nhiều bộ mã hóa (*Multi-encoder*); (b) nhiều nhánh trong vai trò một mô hình con (*Multi-branch as a Sub-model*); (c) chú ý đa đầu (*Multi-head Attention*). Có hai đầu vào: một câu (ký hiệu là **x**) và cây cú pháp của câu (ký hiệu là **t**). Trong kiến trúc nhiều bộ mã hóa (xem hình con (a)), hai bộ mã hóa được xây dựng để lần lượt mã hóa **x** và **t**. Sau đó một mô hình kết hợp nhận đầu ra của các bộ mã hóa và tạo ra một biểu diễn kết hợp của **x** và **t**. Ý tưởng mạng nhiều nhánh có thể được dùng để thiết kế các mô hình con của bộ mã hóa. Một ví dụ đơn giản là ta tạo nhiều đường song song cho một số tầng của bộ mã hóa (xem hình con (b)). Một ví dụ khác là chú ý đa đầu (xem hình con (c)), trong đó ta dùng các đầu khác nhau để học các biểu diễn khác nhau.

Các mô hình con của Transformer cũng có thể là mạng nơ-ron nhiều nhánh. Xem Hình 6.8(b) để

có ví dụ gồm hai nhánh tự chú ý. Một nhánh là mạng tự chú ý tiêu chuẩn $\text{Att}_{\text{self}}(\mathbf{H})$, nhánh kia là mạng tự chú ý có nhận biết cú pháp $\text{AttSyn}_{\text{self}}(\mathbf{H})$. Đầu ra của mô hình tự chú ý là một tổ hợp tuyến tính của đầu ra hai nhánh này [Xu và cộng sự, 2021b]:

$$\mathbf{H}_{\text{self}} = \alpha \cdot \text{Att}_{\text{self}}(\mathbf{H}) + (1-\alpha) \cdot \text{AttSyn}_{\text{self}}(\mathbf{H}), \tag{6.59}$$

trong đó α là hệ số kết hợp. $\mathbf{H}_{\text{self}}$ có thể được dùng như thường lệ bằng cách áp dụng hàm chuẩn hóa tầng và thêm kết nối tắt, nhờ đó giữ cho kiến trúc tổng thể giống hệt các mô hình Transformer tiêu chuẩn.

Mạng chú ý đa đầu cũng có thể được xem là một dạng mô hình nhiều nhánh. Do đó, ta có thể đưa hướng dẫn từ cú pháp vào chỉ một số đầu, còn các đầu còn lại giữ nguyên [Strubell và cộng sự, 2018]. Cách tiếp cận này được minh họa ở Hình 6.8(c), trong đó chỉ một đầu của tầng con tự chú ý dùng cây cú pháp để tính trọng số chú ý.

6.2.4 Mô hình đa tỉ lệ

Trong ngôn ngữ học, cú pháp nghiên cứu cách câu được xây dựng từ các thành phần nhỏ hơn. Các cấp độ khác nhau của những thành phần này thường được tổ chức thành một cấu trúc phân cấp, gọi là **phân cấp cú pháp** (*syntactic hierarchy*). Vì vậy, có thể dùng nhiều cấp độ thành phần cú pháp để giải thích cùng một câu. Ví dụ, các từ cho thấy câu được xây dựng từ những đơn vị nhỏ có nghĩa như thế nào, còn các cụm từ cho thấy câu được xây dựng từ những đơn vị ngôn ngữ lớn hơn như thế nào.

Transformer đa tỉ lệ (*multi-scale*) tận dụng các mức trừu tượng khác nhau trong dữ liệu để biểu diễn một câu bằng các đặc trưng ở nhiều tỉ lệ. Một cách tiếp cận phổ biến là viết một câu dưới nhiều dạng khác nhau rồi kết hợp chúng bằng một mạng nhiều nhánh [Hao và cộng sự, 2019]. Ví dụ, xét câu sau ("Cơ sở nấu bia lâu đời nhất được phát hiện ở Trung Quốc"; giữ nguyên tiếng Anh vì ví dụ minh họa việc tách từ và cụm từ):

The oldest beer-making facility was discovered in China.

Ta có thể tách nó thành một chuỗi các từ, ký hiệu là

$$\mathbf{x}_{\text{words}} = \text{The oldest beer-making facility was discovered in China .}$$

Hoặc, ta có thể viết nó thành một chuỗi các cụm từ nhờ một bộ phân tích cú pháp, ký hiệu là

$$\mathbf{x}_{\text{phrases}} = [\text{The oldest beer-making facility}]_{\text{NP}}\ [\text{was discovered in China}]_{\text{VP}}\ [.].$$

Cách đơn giản nhất để xây dựng một mô hình đa tỉ lệ là mã hóa $\mathbf{x}_{\text{words}}$ và $\mathbf{x}_{\text{phrases}}$ bằng hai bộ mã hóa Transformer riêng biệt. Đầu ra của các bộ mã hóa này sau đó được kết hợp theo một cách nào đó. Điều này dẫn tới cùng dạng với Phương trình (6.58), và ta có thể xem mô hình này là một trường hợp của kiến trúc nhiều bộ mã hóa tổng quát.

Cả $\mathbf{x}_{\text{words}}$ và $\mathbf{x}_{\text{phrases}}$ đều có thể được xem là các chuỗi token. Ví dụ, $\mathbf{x}_{\text{words}}$ có chín token dựa trên từ, còn $\mathbf{x}_{\text{phrases}}$ có ba token dựa trên cụm từ[5]. Tuy nhiên, liệt kê mọi cụm từ có thể có sẽ dẫn tới một bộ từ vựng khổng lồ. Do đó, ta cần một phương pháp để biểu diễn mỗi cụm từ thành một vectơ nhúng theo cách hiệu quả về tính toán. Bằng cách coi việc nhúng cụm từ là một bài toán mô hình hóa chuỗi, ta có thể dễ dàng học biểu diễn của các chuỗi con bằng các mô hình chuỗi đã mô tả ở các chương trước. Khi đó ta có một quá trình học hai giai đoạn. Ở giai đoạn thứ nhất, ta học vectơ nhúng của các đơn vị đầu

[5] $\mathbf{x}_{\text{phrases}}$ gồm ba token: *The oldest beer-making facility*, *was discovered in China* và .

vào ở các tỉ lệ khác nhau bằng các mô hình riêng biệt. Ở giai đoạn thứ hai, ta học cách mã hóa các chuỗi ở các tỉ lệ khác nhau bằng một mô hình nhiều nhánh.

Tổng quát hơn, trong học biểu diễn đa tỉ lệ, ta không cần tự giới hạn ở các đơn vị có ý nghĩa ngôn ngữ học. Ví dụ, ta có thể học cách tách từ con (subword) (*sub-word*) từ dữ liệu và biểu diễn một câu đầu vào thành một chuỗi các từ con. Điều này cho ta một biểu diễn phân cấp của câu (ví dụ: từ con $\rightarrow$ từ $\rightarrow$ cụm từ). Dù các từ con học được có thể không có ý nghĩa ngôn ngữ học, chúng mang lại những góc nhìn mới cho việc mô hình hóa từ và cụm từ, cũng như một tỉ lệ đặc trưng mới. Ngoài ra, ta không nhất thiết phải xây dựng nhiều bộ mã hóa cho mô hình hóa đa tỉ lệ. Một cách tiếp cận khác là đưa các biểu diễn ở các tỉ lệ khác nhau trực tiếp vào các mô-đun tự chú ý đa đầu, giúp dễ mô hình hóa tương tác giữa các tỉ lệ khác nhau hơn [Guo và cộng sự, 2020; Li và cộng sự, 2022b].

Tuy nhiên, một vấn đề của các cách tiếp cận trên là các biểu diễn (hoặc ma trận trọng số chú ý) học được ở các tỉ lệ khác nhau có kích thước khác nhau. Ví dụ, trong các tình huống trên, biểu diễn học từ $\mathbf{x}_{\text{words}}$ là một ma trận $9 \times d$, trong khi biểu diễn học từ $\mathbf{x}_{\text{phrases}}$ là một ma trận $3 \times d$. Một giải pháp đơn giản cho vấn đề này là thực hiện tăng mẫu (*upsampling*) biểu diễn dựa trên cụm từ để mở rộng nó thành một ma trận $9 \times d$. Tương tự, ta có thể thực hiện giảm mẫu (*downsampling*) biểu diễn dựa trên từ để thu nhỏ nó thành một ma trận $3 \times d$. Khi đó mô hình kết hợp Combine$(\cdot)$ có thể được cài đặt như mô tả ở Mục 6.2.3.

Chú giải của người dịch: Tăng mẫu và giảm mẫu, cụ thể

Cụm thứ nhất "The oldest beer-making facility" gồm 4 từ, cụm thứ hai "was discovered in China" gồm 4 từ, cụm thứ ba "." gồm 1 từ: tổng 9. Một cách tăng mẫu đơn giản là *chép* hàng của mỗi cụm cho từng từ thuộc cụm đó: hàng 1 chép 4 lần, hàng 2 chép 4 lần, hàng 3 giữ 1 lần, được ma trận $9 \times d$. Ngược lại, một cách giảm mẫu là lấy *trung bình* các hàng của những từ cùng cụm, đưa ma trận $9 \times d$ về $3 \times d$. Sau đó hai biểu diễn có cùng kích thước nên có thể cộng hoặc ghép với nhau.

Đáng lưu ý là mô hình hóa đa tỉ lệ được bàn đến rộng rãi trong nhiều lĩnh vực. Chẳng hạn, trong thị giác máy tính (*computer vision*), mô hình hóa đa tỉ lệ thường được hiểu là quá trình học một chuỗi các bản đồ đặc trưng (*feature maps*) từ ảnh đầu vào [Fan và cộng sự, 2021; Li và cộng sự, 2022f]. Khác với các mô hình nhiều nhánh trình bày ở đây, các mô hình Transformer thị giác đa tỉ lệ tận dụng bản chất phân cấp của đặc trưng khi biểu diễn ảnh. Các hệ thống loại này thường dựa trên một chồng các tầng, trong đó mỗi tầng học đặc trưng ở một tỉ lệ lớn hơn (ví dụ, dung lượng kênh cao hơn) từ các đặc trưng do tầng trước sinh ra.

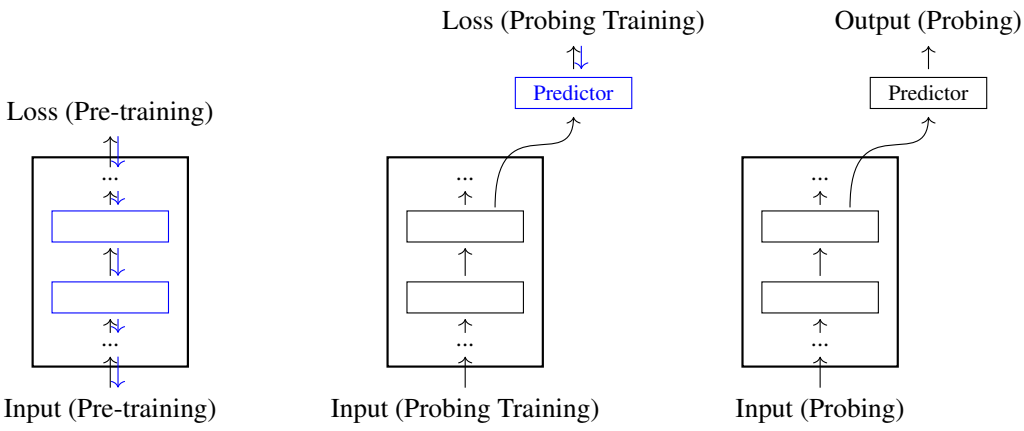
6.2.5 Transformer như một bộ học cú pháp

Cho đến nay, chúng ta đã bàn về cây cú pháp như những ràng buộc hay tiên nghiệm cho quá trình mã hóa, để có thể tận dụng các biểu diễn ngôn ngữ học khi học mạng nơ-ron. Một câu hỏi tự nhiên là liệu các mô hình nơ-ron này có thể học trực tiếp từ dữ liệu một số tri thức về cấu trúc ngôn ngữ, mà không cần các chú giải ngôn ngữ học do con người thiết kế hay không. Điều này phản ánh một mục tiêu nền tảng của NLP: học tri thức ngôn ngữ từ dữ liệu và mã hóa nó một cách ngầm định bên trong mô hình.

Để khám phá các tính chất ngôn ngữ mà hệ thống NLP đã học được, một phương pháp đơn giản là xem xét hành vi cú pháp trong đầu ra của hệ thống. Ví dụ, ta có thể kiểm tra xem đầu ra của các hệ thống sinh ngôn ngữ có chứa lỗi ngữ pháp hay không. Một cách tiếp cận khác là yêu cầu các hệ thống này thực hiện những tác vụ có ý nghĩa về mặt ngôn ngữ học, dù chúng không được huấn luyện tường minh cho việc đó [Brown và cộng sự, 2020]. Tuy nhiên, chỉ xem xét cách các dự đoán của mô hình thể hiện năng lực cú pháp là chưa đủ để trả lời trọn vẹn câu hỏi. Cũng có thể mạng nơ-ron học được tri thức

ngôn ngữ nhưng không sử dụng nó khi dự đoán [Clark và cộng sự, 2019a]. Do đó, ta cần nghiên cứu những gì được mô hình hóa và học được bên trong chính mạng nơ-ron.

Một cách tiếp cận để xem xét cấu trúc ngôn ngữ tiềm ẩn trong mô hình Transformer là xây dựng các phép thăm dò (*probes*) nhằm xác định xem các mô hình này có nắm bắt các khái niệm ngôn ngữ học như quan hệ phụ thuộc và từ loại hay không, và ở mức độ nào. Một cách thăm dò tổng quát là trích xuất các biểu diễn bên trong của mô hình rồi thăm dò chúng để tìm các hiện tượng ngôn ngữ. Với Transformer, việc này thường được thực hiện bằng cách xem xét các bản đồ chú ý (*attention maps*) và/hoặc đầu ra của các tầng chú ý. Sau đó ta xây dựng một bộ dự đoán thăm dò (*probing predictor*) (hay bộ phân loại thăm dò (*probing classifier*)) nhận các biểu diễn bên trong này làm đầu vào và cho ra các khái niệm ngôn ngữ học tương ứng [Belinkov, 2022]. Bộ dự đoán thăm dò có thể dựa trên các quy tắc kinh nghiệm đơn giản hoặc trên các mô hình có tham số được tối ưu riêng cho tác vụ thăm dò. Các công trình gần đây cho thấy các mô hình ngôn ngữ quy mô lớn dựa trên Transformer thể hiện những năng lực mạnh, gọi là **năng lực đột sinh** (*emergent abilities*), trên nhiều tác vụ thăm dò khác nhau. Chúng tôi sẽ không bàn chi tiết về các hệ thống mô hình hóa ngôn ngữ này trong chương này mà để dành cho các chương sau. Dù vậy, ở đây ta giả sử có một bộ mã hóa Transformer đã được huấn luyện đầy đủ trên dữ liệu không gán nhãn và sẵn sàng để thăm dò. Hình 6.9 minh họa quá trình thăm dò này.



(a) Training the Transformer Model (b) Training the Probing Predictor (c) Probing on New Data

Hình 6.9: Tổng quan về thăm dò cho các mô hình dựa trên Transformer. Cho một mô hình Transformer (ví dụ, một mô hình ngôn ngữ dựa trên Transformer), trước hết ta tối ưu tham số mô hình trên một lượng dữ liệu không gán nhãn (xem hình con (a): huấn luyện mô hình Transformer, với *Loss/Input (Pre-training)* = hàm mất mát/đầu vào của tiền huấn luyện). Sau đó, ta xây dựng một bộ dự đoán (*Predictor*) nhận trạng thái của một tầng ẩn của mô hình Transformer và sinh đầu ra cho một tác vụ thăm dò. Bộ dự đoán có thể được huấn luyện như thường lệ, trong đó chỉ tham số của bộ dự đoán được tối ưu còn tham số của mô hình Transformer được giữ cố định (xem hình con (b): huấn luyện bộ dự đoán thăm dò). Mô hình Transformer và bộ dự đoán được dùng cùng nhau để đưa ra dự đoán trên dữ liệu mới nhằm thăm dò (xem hình con (c): thăm dò trên dữ liệu mới).

Nhiều phương pháp thăm dò đã được dùng trong các công trình gần đây nhằm phân tích và hiểu những gì được học trong các bộ mã hóa nơ-ron. Dưới đây chúng tôi mô tả một số phương pháp phổ biến.

- **Cây** (*Trees*). Với một bộ mã hóa Transformer đã huấn luyện, có thể dễ dàng xác định khả năng hai từ trong một câu có quan hệ ngôn ngữ với nhau bằng cách tính trọng số chú ý giữa chúng. Ta

có thể dùng đại lượng này để định nghĩa một độ đo khoảng cách cú pháp giữa hai từ ở vị trí i và j

$$d_s(i,j) \;=\; 1 - \alpha(i,j). \tag{6.60}$$

Với độ đo này, có thể dễ dàng xây dựng **cây khung nhỏ nhất** (*minimum-spanning tree*) cho câu. Tức là, ta nối tất cả các từ thành một cấu trúc cây có tổng khoảng cách nhỏ nhất. Cấu trúc cây này có thể được xem là một biểu diễn cây tiềm ẩn của câu được suy ra từ mạng nơ-ron. Dù cấu trúc giống cây phụ thuộc này có thể là một nguồn thông tin cú pháp đã học cho các tác vụ hạ nguồn, nó lại cho biết rất ít về tri thức cú pháp thực sự của mô hình. Một cách để căn chỉnh các biểu diễn trong bộ mã hóa với cấu trúc ngôn ngữ là học cách sinh ra các cây cú pháp nhất quán với chú giải của con người. Để làm được điều này, ta cần xây dựng một bộ dự đoán thăm dò được huấn luyện trên dữ liệu có chú giải cây. Giả sử ta có một cây phụ thuộc do con người chú giải cho một câu. Với mỗi cặp từ, ta có thể thu được một khoảng cách $\omega(i,j)$ bằng cách đếm số cạnh giữa chúng. Sau đó, ta có thể học một độ đo khoảng cách dựa trên các biểu diễn bên trong của bộ mã hóa để xấp xỉ $\omega(i,j)$. Một dạng đơn giản của độ đo như vậy dựa trên khoảng cách Euclid [Manning và cộng sự, 2020]. Gọi $\mathbf{A} \in \mathbb{R}^{d \times k_s}$ là một ma trận tham số. Bình phương khoảng cách Euclid được cho bởi

$$d_s^2(i,j) \;=\; \|(\mathbf{h}_i - \mathbf{h}_j)\mathbf{A}\|_2^2, \tag{6.61}$$

trong đó $\mathbf{h}_i$ và $\mathbf{h}_j$ là các biểu diễn do một tầng mã hóa sinh ra tại vị trí i và j[6]. Với một tập S các câu có chú giải cây, ta có thể tối ưu mô hình bằng

$$\hat{\mathbf{A}} \;=\; \arg\min_{\mathbf{A}} \sum_{s \in S} \frac{1}{|s|^2} \sum_{i \in s, j \in s} |\omega(i,j) - d_s^2(i,j)|, \tag{6.62}$$

trong đó $|s|$ là độ dài của câu s, và (i,j) chỉ một cặp từ trong s. Mô hình đã tối ưu sau đó được dùng để phân tích cú pháp các câu kiểm tra bằng thuật toán cây khung nhỏ nhất, và ta có thể so sánh các cây phân tích này với các cây do con người chú giải. Để thu được cây có hướng, vốn là dạng chuẩn của cú pháp phụ thuộc, có thể cập nhật mô hình trên bằng cách xét khoảng cách tương đối của một từ tới gốc. Chi tiết hơn có thể xem trong công trình của Manning và cộng sự [2020]. Ở đây, bộ dự đoán thăm dò hoạt động tương tự một bộ phân tích cú pháp nơ-ron, được huấn luyện để dự đoán cây cú pháp dựa trên một biểu diễn của câu đầu vào. Ý tưởng này có thể được mở rộng cho các dạng cấu trúc cú pháp khác, chẳng hạn cây cấu trúc cụm từ [Shi và cộng sự, 2016].

Chú giải của người dịch: Thăm dò cây bằng khoảng cách

Ý tưởng: nếu bộ mã hóa thật sự "hiểu" cú pháp, thì sau một phép biến đổi tuyến tính $\mathbf{A}$ phù hợp, khoảng cách giữa hai vectơ từ sẽ gần bằng số cạnh giữa hai từ trên cây. Ví dụ, trong "The concert was wonderful" với cây ở Hình 6.7, "The" nối với "concert" ($\omega = 1$), "concert" nối với "wonderful" ($\omega = 1$), nên $\omega(\text{The}, \text{wonderful}) = 2$. Phương trình (6.62) tìm $\mathbf{A}$ sao cho d_s^2 khớp với các số $1, 2, \ldots$ này nhất có thể (lấy trung bình sai số tuyệt đối trên mọi cặp từ, chia cho $|s|^2$ là số cặp). **Cây khung nhỏ nhất**: trong số mọi cách nối n từ bằng $n-1$ cạnh thành một cây, chọn cách có tổng "chiều dài" cạnh nhỏ nhất — giống bài toán nối điện cho các làng với tổng chiều dài dây ngắn nhất. Vì $\mathbf{A}$ là phép biến đổi tuyến tính đơn giản, nếu thăm dò thành công thì thông tin cú pháp hẳn đã có sẵn trong các vectơ $\mathbf{h}$, chứ không phải do bộ thăm dò tự "học" ra.

[6] Nói chung, $\mathbf{h}_i$ và $\mathbf{h}_j$ là đầu ra của tầng cuối cùng của bộ mã hóa. Ngoài ra, chúng cũng có thể là tổng có trọng số của đầu ra tất cả các tầng.

- **Nhãn cú pháp và ngữ nghĩa** (*Syntactic and Semantic Labels*). Nhiều tác vụ phân tích cú pháp và ngữ nghĩa có thể được phát biểu thành bài toán dự đoán nhãn ngôn ngữ học khi biết một câu hoặc các đoạn của nó. Một ví dụ đơn giản là gán nhãn từ loại (*part-of-speech tagging*), trong đó mỗi từ của câu được gán một lớp từ. Một bộ thăm dò cho gán nhãn từ loại có thể là một bộ phân loại, mỗi lần nhận một biểu diễn $\mathbf{h}_j$ và cho ra lớp từ tương ứng. Một cách thăm dò tổng quát cho các bài toán này là **thăm dò cạnh** (*edge probing*) [Tenney và cộng sự, 2019b;a]. Cho một câu, một cạnh có nhãn được định nghĩa là một bộ

$$(\text{span}_1, \text{span}_2, \text{label}),$$

trong đó span_1 là một đoạn $[i_1, j_1]$, span_2 là một đoạn thứ hai tùy chọn $[i_2, j_2]$, và label là nhãn tương ứng. Mục tiêu của ta là học một bộ thăm dò để dự đoán label khi biết span_1 và span_2. Ví dụ, với gán nhãn từ loại, span_1 là đoạn đơn vị $[j, j]$ cho mỗi vị trí j, span_2 là đoạn rỗng, và label là nhãn từ loại ứng với từ thứ j của câu. Với phân tích phụ thuộc và phân giải đồng tham chiếu (*coreference resolution*), span_1 và span_2 là hai từ hoặc hai thực thể, và label là quan hệ giữa chúng. Với phân tích thành phần, span_1 là một đoạn từ, span_2 là đoạn rỗng, và label là phạm trù cú pháp của nút cây sinh ra span_1. Trong các trường hợp đơn giản, mô hình thăm dò có thể là một mạng nơ-ron truyền thẳng nhiều tầng với tầng đầu ra Softmax. Như thường lệ, mô hình này được huấn luyện trên dữ liệu có nhãn rồi kiểm tra trên dữ liệu mới.

Chú giải của người dịch: Ví dụ về thăm dò cạnh

Câu "Lan nói cô ấy sẽ đến": với đồng tham chiếu, span_1 = "Lan" (đoạn $[1, 1]$), span_2 = "cô ấy" (đoạn $[3, 4]$), label = *cùng chỉ một người*. Với gán nhãn từ loại, $\text{span}_1 = [2, 2]$ ("nói"), không có span_2, label = *động từ*. Cùng một khuôn dạng "(đoạn, đoạn, nhãn)" nên một bộ thăm dò chung có thể kiểm tra nhiều loại tri thức ngôn ngữ khác nhau.

- **Hình thức bề mặt của từ và câu** (*Surface Forms of Words and Sentences*). Các tác vụ thăm dò cũng có thể được thiết kế để kiểm tra xem các biểu diễn có mã hóa thông tin bề mặt của câu hay của từ hay không [Adi và cộng sự, 2016; Conneau và cộng sự, 2018]. Một tác vụ thăm dò mức câu đơn giản là **dự đoán độ dài câu** (*sentence length prediction*). Để làm điều này, trước hết ta biểu diễn câu bằng một vectơ duy nhất $\mathbf{h}$[7], rồi xây dựng một bộ phân loại để xếp $\mathbf{h}$ vào khoảng độ dài tương ứng. Tương tự, có thể xây dựng các bộ thăm dò để dự đoán xem hai từ ở vị trí i và j có bị đảo thứ tự trong câu hay không khi biết $\mathbf{h}_i$ và $\mathbf{h}_j$. Ta cũng có thể xây dựng các bộ thăm dò cho những bài toán truyền thống của hình thái học (*morphology*). Ví dụ, ta khôi phục lại từ ở vị trí j hoặc dự đoán nghĩa của nó từ biểu diễn $\mathbf{h}_j$. Ngoài ra, các tác vụ thăm dò có thể tập trung vào những vấn đề ngôn ngữ cụ thể, chẳng hạn khả năng xử lý số [Wallace và cộng sự, 2019] và hư từ (*function words*) [Kim và cộng sự, 2019].

- **Điền vào chỗ trống** (*Cloze*). Tất nhiên, ta có thể thăm dò các mô hình nơ-ron cho những bài toán vượt ra ngoài cú pháp và hình thái học. Một góc nhìn về các mô hình Transformer tiền huấn luyện (*pre-trained*) quy mô lớn là xem chúng như những cơ sở tri thức chứa các sự kiện về thế giới. Vì vậy, thật hấp dẫn khi thử xem liệu ta có thể dùng chúng để kiểm tra tri thức sự kiện hay không. Một phương pháp đơn giản là yêu cầu bộ thăm dò khôi phục thành phần bị thiếu của một câu [Petroni và cộng sự, 2019]. Ví dụ, nếu ta có một câu điền vào chỗ trống

Shiji was written by ____. (*Sử ký* được viết bởi ____.)

[7] $\mathbf{h}$ có thể được tính bằng cách thực hiện một phép gộp trên $\{\mathbf{h}_1, \cdots, \mathbf{h}_m\}$.

ta mong bộ thăm dò đưa ra câu trả lời *Sima Qian* (Tư Mã Thiên) vì có một sự kiện dạng chủ thể–quan hệ–đối tượng (Shiji, written-by, Sima Qian). Bộ thăm dò này đơn giản có thể là một **mô hình ngôn ngữ có mặt nạ** (*masked language model*), loại mô hình được dùng rộng rãi trong học tự giám sát (*self-supervised learning*) của bộ mã hóa Transformer.

Trong NLP, thăm dò có liên hệ chặt chẽ với việc tiền huấn luyện các mô hình ngôn ngữ lớn (*large language models*) (xem Chương 7 và 8). Nói chung, ta có thể xem các tác vụ thăm dò như những ứng dụng hạ nguồn của các mô hình tiền huấn luyện này, dù thăm dò thường được dùng như một công cụ phân tích để nhanh chóng kiểm tra năng lực của mô hình. Lý tưởng nhất, ta muốn xây dựng một bộ thăm dò tận dụng tốt nhất các biểu diễn bên trong để giải một tác vụ cho trước. Tuy nhiên, khi bộ thăm dò phức tạp và được huấn luyện đủ tốt, sẽ khó xác định bài toán được giải nhờ thông tin phong phú trong các biểu diễn hay nhờ năng lực mô hình hóa của chính bộ thăm dò. Một cách phổ biến để tách riêng đóng góp của các biểu diễn là so sánh các bộ thăm dò với những mô hình cơ sở (baseline) (*baselines*) hợp lý, hoặc tiến hành so sánh bằng các tác vụ đối chứng (*control tasks*) [Hewitt và Liang, 2019; Belinkov, 2022].

Chú giải của người dịch: Vì sao bộ thăm dò nên đơn giản?

Hãy hình dung ta muốn biết một học sinh có thật sự hiểu bài không. Nếu cho phép em tra cứu sách và hỏi gia sư trong lúc làm bài (bộ thăm dò quá mạnh), điểm cao chưa chắc phản ánh hiểu biết của chính em. *Tác vụ đối chứng* giống như một bài kiểm tra với "đáp án ngẫu nhiên" vô nghĩa: nếu bộ thăm dò vẫn đạt điểm cao trên đó, nghĩa là nó chỉ đang "học thuộc", và kết quả trên tác vụ thật cần được đánh giá thận trọng.

6.3 Các kiến trúc cải tiến

Trong mục này, chúng ta trình bày một số cải tiến cho mô hình Transformer nguyên bản (*vanilla*). Khác với mục trước, phần lớn các cải tiến ở đây xuất phát từ góc nhìn của học máy chứ không phải của ngôn ngữ học.

6.3.1 Các mô hình chú ý cục bộ

Các phương pháp tự chú ý (*self-attention*) đã bàn trong Mục 6.1.3 có thể được xem là học các biểu diễn (*representation*) trên toàn bộ chuỗi đầu vào. Mặc dù cơ chế chú ý toàn cục (*global attention*) này có khả năng mạnh trong việc nắm bắt các phụ thuộc xa (*long-distance dependency*), nó lại không mô hình hóa thông tin cục bộ một cách tường minh. Ở đây, ta xét một vài kỹ thuật cố gắng nắm bắt tường minh tính cục bộ trong các biểu diễn.

1. Tri thức tiên nghiệm cho mô hình hóa cục bộ

Một trong những cách đơn giản nhất để đưa việc mô hình hóa ngữ cảnh cục bộ vào Transformer là thêm một số hạng phạt (*penalty term*) vào hàm chú ý, nhờ đó hạn chế các trọng số chú ý lớn giữa những vị trí cách xa nhau. Ở phía bộ mã hóa (*encoder*), điều này dẫn đến một dạng mà ta đã gặp vài lần trong chương này:

$$\text{AttLocal}_{\text{self}}(\mathbf{H}) \;=\; \text{Softmax}\left(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}} - \gamma \cdot \mathbf{G}\right)\mathbf{H}^v, \tag{6.63}$$

trong đó γ là trọng số (hay "nhiệt độ") của số hạng phạt, và $\mathbf{G} \in \mathbb{R}^{m \times m}$ là ma trận phạt. Mỗi phần tử $G(i, j)$ cho biết ta phạt mô hình nặng đến mức nào khi nó chú ý từ vị trí i tới vị trí j. Một lựa chọn đơn

giản cho $G(i,j)$ là khoảng cách tuyệt đối giữa i và j, chẳng hạn

$$G(i,j) \;=\; |i-j|. \tag{6.64}$$

Ngoài ra, $G(i,j)$ có thể được định nghĩa bằng một hàm phạt Gauss [Yang và cộng sự, 2018a]

$$G(i,j) \;=\; \frac{(i-j)^2}{2\sigma_i^2}, \tag{6.65}$$

trong đó σ_i là độ lệch chuẩn của phân phối Gauss. Với một i cho trước, cả hai số hạng phạt đều tăng (theo kiểu tuyến tính hoặc theo hàm mũ) khi khoảng cách $|i-j|$ tăng lên.

Chú giải của người dịch: Phạt khoảng cách hoạt động thế nào?

Hàm Softmax biến một dãy điểm số thành các xác suất: điểm càng cao thì trọng số chú ý càng lớn. Trừ đi $\gamma \cdot G(i,j)$ nghĩa là "trừ điểm" những vị trí ở xa. Ví dụ với $\gamma = 1$, $G(i,j) = |i-j|$ và vị trí $i = 3$: điểm chú ý tới các vị trí $j = 1,2,3,4,5$ bị trừ lần lượt $2,1,0,1,2$. Nếu ban đầu mọi điểm đều bằng 0 thì sau khi trừ, Softmax của $(-2,-1,0,-1,-2)$ xấp xỉ $(0.07;\ 0.18;\ 0.49;\ 0.18;\ 0.07)$: mô hình tập trung vào chính vị trí 3 và các láng giềng gần. Với dạng Gauss $(i-j)^2/(2\sigma_i^2)$, mức phạt tăng theo bình phương khoảng cách, nên các vị trí xa bị "dập" mạnh hơn nhiều; σ_i càng lớn thì "cửa sổ" chú ý càng rộng.

Phương pháp này có thể mở rộng cho mô hình chú ý chéo (*cross-attention*) như sau

$$\text{AttLocal}_{\text{cross}}(\mathbf{H},\mathbf{S}) \;=\; \text{Softmax}\Big(\frac{\mathbf{S}^q[\mathbf{H}^k]^\top}{\sqrt{d}} - \gamma\cdot\mathbf{G}\Big)\mathbf{H}^v, \tag{6.66}$$

trong đó $\mathbf{G}$ là một ma trận $n \times m$. Mỗi phần tử của $\mathbf{G}$ có thể được định nghĩa là

$$G(i,j) \;=\; \frac{(\mu_i - j)^2}{2\sigma_i^2}, \tag{6.67}$$

trong đó μ_i là kỳ vọng (giá trị trung bình) của phân phối Gauss trên các vị trí phía nguồn. Cả μ_i và σ_i đều có thể được xác định bằng các quy tắc kinh nghiệm (*heuristics*). Một cách khác là ta xây dựng thêm các mạng nơ-ron để mô hình hóa chúng, học các tham số của chúng đồng thời với phần còn lại của mô hình. Ví dụ, ta có thể dùng một mạng nơ-ron truyền thẳng (*feed-forward neural network*) để dự đoán μ_i khi biết $\mathbf{s}_i$.

Một phương án thay thế cho Phương trình (6.63) (hoặc Phương trình (6.66)) là coi số hạng phạt như một phân phối riêng và kết hợp tuyến tính nó với mô hình chú ý ban đầu. Chẳng hạn, ta có thể định nghĩa mô hình tự chú ý là

$$\text{AttLocal}_{\text{self}}(\mathbf{H}) \;=\; \Big((1-\beta)\cdot\text{Softmax}\Big(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}}\Big) + \beta\cdot\text{Softmax}(-\gamma\cdot\mathbf{G})\Big)\mathbf{H}^v, \tag{6.68}$$

trong đó $\beta \in [0,1]$ là hệ số kết hợp tuyến tính. Lưu ý rằng, để tránh phải tinh chỉnh thủ công siêu tham số (*hyperparameter*) β, ta có thể dùng một mạng cổng (*gating network*) để dự đoán β một cách động và huấn luyện nó theo kiểu đầu-cuối (*end-to-end*).

Một phương án khác nữa là dùng một mặt nạ nhân (*multiplicative mask*) để đưa tri thức tiên nghiệm vào mô hình, tương tự Phương trình (6.53). Công thức như sau

$$\text{AttLocal}_{\text{self}}(\mathbf{H}) \;=\; \text{Softmax}\Big(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}} \odot \mathbf{G}'\Big)\mathbf{H}^v. \tag{6.69}$$

Ở đây, $\mathbf{G}' \in [0,1]^{m \times m}$ là một ma trận có các phần tử nằm trong $[0,1]$. Đại lượng vô hướng $G'(i,j)$ đạt giá trị lớn nhất bằng 1 khi $i = j$ và giảm dần khi j càng xa i. $G'(i,j)$ có thể thu được bằng cách chuẩn hóa $-G(i,j)$ trên mọi j, hoặc bằng cách dùng các hàm suy giảm khác.

Chú giải của người dịch: Ba cách đưa "tính cục bộ" vào cơ chế chú ý

- Phương trình (6.63): *trừ* điểm phạt trước khi qua Softmax (phạt cộng).
- Phương trình (6.68): *trộn* hai phân phối — phân phối chú ý thông thường và một phân phối chỉ phụ thuộc khoảng cách — theo tỉ lệ $(1-\beta) : \beta$. Ví dụ $\beta = 0.3$ nghĩa là 70% "nghe theo" nội dung, 30% "nghe theo" khoảng cách.
- Phương trình (6.69): *nhân* từng phần tử ($\odot$) điểm chú ý với một hệ số trong $[0,1]$, bằng 1 ở đường chéo và nhỏ dần khi xa đường chéo.

Cả ba đều là ràng buộc "mềm": vị trí xa không bị cấm hẳn, chỉ bị giảm ưu tiên.

2. Chú ý cục bộ

Thuật ngữ *chú ý cục bộ (local attention)* được dùng rộng rãi để mô tả nhiều loại bài toán khác nhau và để chỉ nhiều mô hình khác nhau trong các tài liệu NLP. Các phương pháp nêu ở trên là những phương pháp áp đặt ràng buộc mềm (*soft constraint*) lên mô hình chú ý. Thật ra, chú ý cục bộ bắt nguồn từ những nỗ lực giới hạn phạm vi của mô hình chú ý vì các lý do về mô hình hóa và tính toán [Luong và cộng sự, 2015]. Các nghiên cứu trong hướng này thường tập trung vào việc đưa vào các ràng buộc cứng (*hard constraint*), để mô hình thu được có thể tập trung vào một phần của đầu vào và bỏ qua phần còn lại. Ví dụ, ta có thể dự đoán một khoảng các vị trí phía nguồn cần chú ý tới, khi biết một vị trí phía đích [Sperber và cộng sự, 2018; Yang và cộng sự, 2018a; Sukhbaatar và cộng sự, 2019]. Ngoài ra, các khoảng chú ý cũng có thể được suy ra từ cây cú pháp; chẳng hạn, cấu trúc cây con cú pháp có thể giúp thu hẹp phạm vi chú ý. Do đó, nhiều mô hình có ràng buộc cú pháp có thể được xem là các trường hợp cụ thể của chú ý cục bộ (xem Mục 6.2.4). Hơn nữa, khái niệm chú ý cục bộ có thể được khái quát hóa cho một tập hợp phong phú các mô hình, chẳng hạn các **mô hình chú ý thưa** (*sparse attention models*), mặc dù những mô hình này thường được bàn tới trong bối cảnh các phương pháp học máy hiệu quả. Ta sẽ thấy một vài ví dụ ở Mục 6.4.

Trong học sâu (*deep learning*), một trong những kiến trúc được dùng rộng rãi nhất để học đặc trưng từ một vùng giới hạn của đầu vào là CNN. Vì vậy, sẽ rất thú vị nếu xét các phương pháp kết hợp CNN với mô hình Transformer để có được ưu điểm của cả hai cách tiếp cận, chẳng hạn CNN xử lý các phụ thuộc ngắn hạn, còn mô hình tự chú ý xử lý các phụ thuộc dài hạn. Một cách tiếp cận là xây dựng một mô hình chuỗi hai nhánh, trong đó một nhánh dựa trên CNN và nhánh kia dựa trên mô hình tự chú ý [Wu và cộng sự, 2020b]. Một cách tiếp cận khác là tích hợp các tầng CNN vào các khối Transformer theo cách cho phép mô hình học được cả biểu diễn cục bộ lẫn toàn cục [Wu và cộng sự, 2019; Gulati và cộng sự, 2020].

Chú giải của người dịch: Ràng buộc mềm và ràng buộc cứng

Ràng buộc mềm giống như "khuyến khích" học sinh hỏi bạn ngồi cạnh: vẫn được hỏi bạn ở xa nhưng ít khi làm vậy. *Ràng buộc cứng* giống như quy định "chỉ được trao đổi với bạn cùng bàn": những vị trí ngoài cửa sổ bị loại hẳn (trọng số chú ý bằng 0). Ràng buộc cứng còn giúp tiết kiệm tính toán, vì ta không cần tính điểm chú ý cho các cặp vị trí bị loại.

3. Nhúng vị trí tương đối

Nhúng vị trí tương đối (*relative positional embedding*), còn được gọi là **biểu diễn vị trí tương đối** (*relative positional representation*) (**RPR**), là một cải tiến của phương pháp nhúng vị trí tuyệt đối (*absolute positional embedding*) được dùng trong Transformer chuẩn [Shaw và cộng sự, 2018; Huang và cộng sự, 2018]. Ý tưởng của RPR là mô hình hóa khoảng cách giữa hai vị trí trong một chuỗi, thay vì gán cho mỗi vị trí một biểu diễn cố định. Kết quả là ta có một biểu diễn theo cặp $\text{PE}(i, j)$ cho hai vị trí i và j. Một cách đơn giản để định nghĩa $\text{PE}(i, j)$ là coi nó như một bảng tra cứu (*lookup table*) cho mọi cặp i và j. Cụ thể hơn, gọi $\mathbf{u}_\pi$ là một biểu diễn d chiều ứng với một khoảng cách π cho trước. Dạng của $\text{PE}(i, j)$ trong phương pháp RPR nguyên bản là

$$\text{PE}(i, j) \;=\; \mathbf{u}_{\text{clip}(j-i, k_{\text{rpr}})}, \tag{6.70}$$

trong đó $\text{clip}(x, k_{\text{rpr}})$ là hàm "cắt" x vào trong đoạn $[-k_{\text{rpr}}, k_{\text{rpr}}]$

$$\text{clip}(x, k_{\text{rpr}}) \;=\; \max\{-k_{\text{rpr}}, \min\{x, k_{\text{rpr}}\}\}. \tag{6.71}$$

Như vậy, ta có một mô hình với các tham số

$$\mathbf{U}_{\text{rpr}} \;=\; \begin{bmatrix} \mathbf{u}_{-k_{\text{rpr}}} \\ \vdots \\ \mathbf{u}_0 \\ \vdots \\ \mathbf{u}_{k_{\text{rpr}}} \end{bmatrix}. \tag{6.72}$$

Mặc dù cách viết này hơi không chính thức, ta có thể xem $\mathbf{U}_{\text{rpr}}$ là một ma trận $\in \mathbb{R}^{(2k_{\text{rpr}}+1)\times d}$, và chọn hàng ứng với $\text{clip}(j - i, k_{\text{rpr}})$ khi tính RPR cho i và j cho trước.

Chú giải của người dịch: Ví dụ tính hàm clip

Lấy $k_{\text{rpr}} = 3$. Khi đó chỉ có $2 \cdot 3 + 1 = 7$ vectơ $\mathbf{u}_{-3}, \ldots, \mathbf{u}_3$. Với $i = 1, j = 2$: $j - i = 1$, $\text{clip}(1, 3) = 1$, nên $\text{PE}(1, 2) = \mathbf{u}_1$. Với $i = 1, j = 5$: $j - i = 4$, nhưng $\text{clip}(4, 3) = \max\{-3, \min\{4, 3\}\} = 3$, nên $\text{PE}(1, 5) = \mathbf{u}_3$. Với $i = 5, j = 1$: $j - i = -4$, $\text{clip}(-4, 3) = -3$, nên $\text{PE}(5, 1) = \mathbf{u}_{-3}$. Nghĩa là mọi khoảng cách từ 3 trở lên được "gộp chung" một vectơ. Bạn có thể đối chiếu với bảng nhỏ ở bên phải Hình 6.10(b). Điểm mấu chốt: biểu diễn chỉ phụ thuộc vào *hiệu* $j - i$ (cách nhau bao xa, về phía nào), không phụ thuộc vào việc i, j đứng ở đâu trong câu.

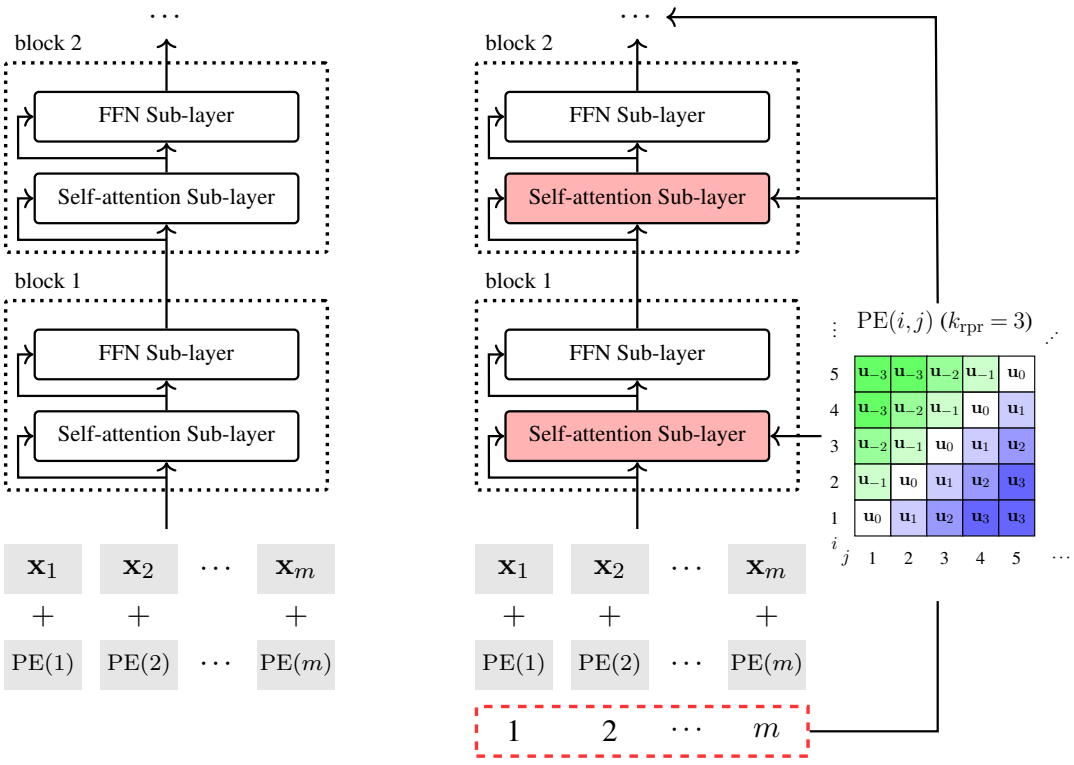
Dùng phương pháp trên, ta có thể định nghĩa ba mô hình RPR $\text{PE}^q(i, j)$, $\text{PE}^k(i, j)$ và $\text{PE}^v(i, j)$ lần lượt cho truy vấn (*query*), khóa (*key*) và giá trị (*value*). Khi đó, theo dạng của Phương trình (6.17), đầu ra của mô hình tự chú ý tại vị trí i có thể được viết là

$$\begin{aligned} \mathbf{c}_i &= \sum_{j=1}^{m} \alpha_{i,j} \left[\mathbf{h}_j^v + \text{PE}^v(i, j)\right] \\ &= \sum_{j=1}^{m} \alpha_{i,j} \mathbf{h}_j^v + \sum_{j=1}^{m} \alpha_{i,j} \text{PE}^v(i, j), \end{aligned} \tag{6.73}$$

trong đó $\mathbf{h}_j^v$ là vectơ hàng thứ j của $\mathbf{H}^v$. Biểu diễn này gồm hai thành phần: $\sum_{j=1}^{m} \alpha_{i,j} \mathbf{h}_j^v$ là biểu diễn cơ bản, còn $\sum_{j=1}^{m} \alpha_{i,j} \text{PE}^v(i, j)$ là biểu diễn vị trí.

Trọng số chú ý $\alpha_{i,j}$ được tính theo cách thông thường, nhưng có thêm các số hạng $\text{PE}^q(i,j)$ và $\text{PE}^k(i,j)$ được cộng vào mỗi truy vấn và khóa:

$$\alpha_{i,j} = \text{Softmax}\Big(\frac{[\mathbf{h}_i^q + \text{PE}^q(i,j)][\mathbf{h}_j^k + \text{PE}^k(i,j)]^\top}{\sqrt{d}}\Big). \tag{6.74}$$



(a) Transformer without RPR (b) Transformer with RPR

Hình 6.10: Minh họa bộ mã hóa Transformer không có và có biểu diễn vị trí tương đối (RPR). Trong RPR, mỗi cặp vị trí được biểu diễn bằng một vectơ $\text{PE}(i,j)$ nhờ một mô hình được tham số hóa bởi $\mathbf{U}_{\text{rpr}}$. $\text{PE}(i,j)$ được đưa vào từng tầng con tự chú ý, nhờ đó ta có thể tận dụng thông tin vị trí ở các bước trung gian của quá trình học biểu diễn. (Nhãn trong hình: *block* = khối; *FFN Sub-layer* = tầng con FFN; *Self-attention Sub-layer* = tầng con tự chú ý; (a) Transformer không có RPR; (b) Transformer có RPR.)

Hình 6.10 cho thấy kiến trúc bộ mã hóa Transformer có và không có RPR. Khi dùng RPR, $\text{PE}^q(i,j)$, $\text{PE}^k(i,j)$, $\text{PE}^v(i,j)$ được đưa trực tiếp vào từng tầng con tự chú ý, nhờ vậy ta có thể tận dụng tốt hơn thông tin vị trí cho việc mô hình hóa chuỗi. Lưu ý rằng việc dùng hàm cắt (xem Phương trình (6.71)) làm cho việc mô hình hóa trở nên đơn giản, vì ta không cần phân biệt các khoảng cách tương đối trong các trường hợp $|j-i| \geq k_{\text{rpr}}$. Mô hình dựa trên khoảng cách đã bị cắt này có thể giúp mô hình hóa tốt hơn trong các cửa sổ ngữ cảnh cục bộ.

Các Phương trình (6.73) và (6.74) đưa ra một cách tiếp cận tổng quát cho việc mô hình hóa chuỗi có nhạy với vị trí. Mô hình này có nhiều biến thể. Trong công trình đầu tiên về RPR của Shaw và cộng sự [2018], các biểu diễn vị trí cho truy vấn bị loại bỏ, và mô hình chỉ làm việc với $\text{PE}^k(i,j)$ và $\text{PE}^v(i,j)$,

như sau

$$\alpha_{i,j} = \text{Softmax}\left(\frac{\mathbf{h}_i^q[\mathbf{h}_j^k + \text{PE}^k(i,j)]^\top}{\sqrt{d}}\right). \tag{6.75}$$

Ngược lại, một số biến thể cố gắng cải tiến mô hình RPR khi tính trọng số chú ý nhưng bỏ qua $\text{PE}^v(i,j)$ khi tính biểu diễn giá trị [Dai và cộng sự, 2019; He và cộng sự, 2021]. Thay vì coi RPR là một số hạng cộng thêm vào từng biểu diễn, các nhà nghiên cứu cũng đã khám phá những cách khác để đưa RPR vào Transformer [Huang và cộng sự, 2020b; Raffel và cộng sự, 2020]. Chúng tôi giới thiệu bạn đọc quan tâm tới các bài báo này để biết thêm chi tiết.

6.3.2 Các mô hình sâu

Nhiều hệ thống NLP tiên tiến nhất (*state-of-the-art*) dựa trên các mô hình Transformer sâu. Ví dụ, các mô hình ngôn ngữ lớn (*large language model*) gần đây thường gồm hàng chục tầng Transformer (hay chính xác hơn là hàng trăm tầng), và cho thấy hiệu năng mạnh trên nhiều tác vụ [Ouyang và cộng sự, 2022; Touvron và cộng sự, 2023a]. Bằng cách xếp chồng các tầng Transformer, ta dễ dàng xây dựng được một mô hình sâu. Tuy nhiên, như thường lệ, huấn luyện các mạng nơ-ron rất sâu là việc khó. Một khó khăn xuất phát từ thực tế là mặt sai số (*error surface*) của mạng nơ-ron sâu có tính không lồi (*non-convex*) rất cao và chứa nhiều cực trị địa phương (*local optima*), khiến quá trình huấn luyện dễ bị mắc kẹt. Mặc dù có những thuật toán tối ưu giúp giảm nhẹ vấn đề này, hầu hết các cách tiếp cận thực tế đều dựa vào các phương pháp dựa trên gradient để tối ưu mạng nơ-ron sâu. Kết quả là, việc huấn luyện một mô hình có nhiều tầng Transformer trở nên khó khăn do hiện tượng tiêu biến gradient (*vanishing gradient*) và bùng nổ gradient (*exploding gradient*) trong quá trình lan truyền ngược (*back-propagation*). Ở đây, ta xét một số kỹ thuật để huấn luyện các mô hình Transformer sâu.

Chú giải của người dịch: Tiêu biến và bùng nổ gradient

Khi lan truyền ngược qua L tầng, gradient gần như bị *nhân* liên tiếp với L hệ số (mỗi tầng một hệ số). Nếu mỗi hệ số khoảng 0.9 thì sau 100 tầng còn $0.9^{100} \approx 0.00003$: gradient gần như biến mất, các tầng đầu hầu như không học được (*tiêu biến*). Nếu mỗi hệ số khoảng 1.1 thì $1.1^{100} \approx 13\,781$: gradient lớn vọt, việc cập nhật tham số trở nên hỗn loạn (*bùng nổ*). Đây cũng giống như hàm mũ a^n ở THPT: với $a < 1$ nó tiến về 0, với $a > 1$ nó tăng rất nhanh.

1. Nhìn lại các kiến trúc Pre-Norm và Post-Norm

Như đã giới thiệu trước đây, một tầng con Transformer là một mạng phần dư (*residual network*), trong đó một "đường tắt" được tạo ra để cộng trực tiếp đầu vào của mạng vào đầu ra của tầng con. Điều này cho phép gradient chảy trực tiếp hơn từ đầu ra về đầu vào, làm giảm nhẹ vấn đề tiêu biến gradient. Nói chung, một kết nối tắt (*residual connection*) trong Transformer được dùng cùng với một đơn vị chuẩn hóa tầng (*layer normalization*) để tạo thành một tầng con. Điều này dẫn tới hai loại kiến trúc, gọi là chuẩn hóa sau (*post-norm*) và chuẩn hóa trước (*pre-norm*). Cụ thể, nhắc lại từ Mục 6.1.4 rằng kiến trúc post-norm có thể được biểu diễn là:

$$\mathbf{z}^l = \text{LNorm}(F^l(\mathbf{z}^{l-1}) + \mathbf{z}^{l-1}), \tag{6.76}$$

trong đó $\mathbf{z}^l$ và $\mathbf{z}^{l-1}$ lần lượt là đầu ra và đầu vào của tầng con l, và $F^l(\cdot)$ là hàm lõi của tầng con này. Kiến trúc pre-norm đưa ánh xạ đồng nhất (*identity mapping*) $\mathbf{z}^{l-1}$ ra ngoài hàm chuẩn hóa tầng, có dạng

$$\mathbf{z}^l = F^l(\text{LNorm}(\mathbf{z}^{l-1})) + \mathbf{z}^{l-1}. \tag{6.77}$$

Hãy xét sự khác biệt giữa dòng thông tin trong hai kiến trúc này:

- Kiến trúc post-norm ngăn không cho ánh xạ đồng nhất của đầu vào được cộng trực tiếp vào đầu ra cuối cùng của tầng con. Đây không phải một mạng phần dư thực thụ, vì toàn bộ thông tin đều phải đi qua một hàm phi tuyến (tức là đơn vị chuẩn hóa tầng). Do đó, kiến trúc post-norm kém hiệu quả hơn cho lan truyền ngược. Wang và cộng sự [2019a] chỉ ra rằng gradient của hàm mất mát của một mạng Transformer gồm L tầng con theo $\mathbf{z}^l$ được cho bởi

$$\frac{\partial E}{\partial \mathbf{z}^l} = \frac{\partial E}{\partial \mathbf{z}^L} \cdot \prod_{k=l}^{L-1} \frac{\partial \text{LNorm}(\mathbf{v}^k)}{\partial \mathbf{v}^k} \cdot \prod_{k=l}^{L-1} \left(1 + \frac{\partial F^k(\mathbf{z}^k)}{\partial \mathbf{z}^k}\right), \tag{6.78}$$

  trong đó $\mathbf{z}^L$ là đầu ra của tầng cuối cùng, $\mathbf{v}^k$ là cách viết tắt của $F^k(\mathbf{z}^{k-1})$, và E là sai số được đo bằng một hàm mất mát nào đó. $\frac{\partial \text{LNorm}(\mathbf{v}^k)}{\partial \mathbf{v}^k}$ và $\frac{\partial F^k(\mathbf{z}^k)}{\partial \mathbf{z}^k}$ lần lượt là gradient của hàm chuẩn hóa tầng và của hàm lõi. Mặc dù phương trình ở đây trông phức tạp, ta thấy rằng $\prod_{k=l}^{L-1} \frac{\partial \text{LNorm}(\mathbf{v}^k)}{\partial \mathbf{v}^k}$ chỉ đơn giản là tích của $L - l$ thừa số. Điều này có nghĩa là gradient sai số sẽ bị co giãn lặp đi lặp lại khi L lớn lên, dẫn đến nguy cơ tiêu biến hoặc bùng nổ gradient cao hơn ở các mô hình sâu hơn.

- Kiến trúc pre-norm mô tả một mạng nơ-ron phần dư chuẩn, trong đó đầu vào của toàn bộ mạng được cộng trực tiếp vào đầu ra của nó. Ta có thể viết gradient của sai số tại $\mathbf{z}^l$ là:

$$\begin{aligned}\frac{\partial E}{\partial \mathbf{z}^l} &= \frac{\partial E}{\partial \mathbf{z}^L} \cdot \left(1 + \sum_{k=l}^{L-1} \frac{\partial F^k(\text{LNorm}(\mathbf{z}^k))}{\partial \mathbf{z}^k}\right) \\ &= \frac{\partial E}{\partial \mathbf{z}^L} + \frac{\partial E}{\partial \mathbf{z}^L} \cdot \sum_{k=l}^{L-1} \frac{\partial F^k(\text{LNorm}(\mathbf{z}^k))}{\partial \mathbf{z}^k}.\end{aligned} \tag{6.79}$$

  Dễ thấy rằng $\frac{\partial E}{\partial \mathbf{z}^l}$ nhận được phản hồi trực tiếp về sai số, vì số hạng đầu tiên ở vế phải (tức là $\frac{\partial E}{\partial \mathbf{z}^L}$) là gradient của đầu ra mô hình, vốn không phụ thuộc vào độ sâu của mạng.

Chú giải của người dịch: Vì sao pre-norm dễ huấn luyện hơn?

Hãy hình dung gradient như một "tín hiệu sửa lỗi" truyền từ tầng trên cùng xuống tầng dưới. Trong Phương trình (6.78) (post-norm), tín hiệu phải đi qua một *tích* gồm $L - l$ thừa số: nếu mỗi thừa số là 0.8 và có 50 thừa số thì tín hiệu chỉ còn $0.8^{50} \approx 0.000014$ lần ban đầu. Trong Phương trình (6.79) (pre-norm), gradient là một *tổng*: số hạng thứ nhất $\frac{\partial E}{\partial \mathbf{z}^L}$ đi thẳng từ đầu ra xuống tầng l mà không bị nhân với gì cả, giống như có một "đường cao tốc" riêng. Vì vậy dù mạng sâu đến đâu, tầng dưới vẫn luôn nhận được một phần tín hiệu nguyên vẹn. (Ký hiệu $\frac{\partial E}{\partial \mathbf{z}}$ là *đạo hàm riêng*: đạo hàm của E theo từng thành phần của vectơ $\mathbf{z}$, khi giữ các thành phần khác cố định.)

Việc dùng kiến trúc pre-norm cũng giúp ích cho quá trình tối ưu trong những bước hạ gradient đầu tiên. Ví dụ, người ta nhận thấy rằng mô hình Transformer pre-norm có thể được huấn luyện với tốc độ học (*learning rate*) lớn hơn ở giai đoạn đầu của huấn luyện, thay vì cần khởi động tốc độ học (*learning rate warmup*) tăng dần từ một giá trị nhỏ [Xiong và cộng sự, 2020].

Mặc dù kiến trúc pre-norm giúp việc tối ưu các mô hình Transformer sâu dễ dàng hơn, ta không thể đơn giản kết luận rằng nó là lựa chọn tốt hơn trong mọi trường hợp so với kiến trúc post-norm. Thực tế, cả hai loại mô hình Transformer post-norm và pre-norm đều đã được dùng thành công trong nhiều ứng dụng. Ví dụ, kiến trúc post-norm được dùng rộng rãi trong các mô hình kiểu BERT, trong khi kiến trúc pre-norm là lựa chọn phổ biến hơn cho các mô hình ngôn ngữ lớn dạng sinh gần đây. Nhìn rộng ra, hai

kiến trúc này cung cấp các khuôn mẫu khác nhau để thiết kế mô hình Transformer sâu, mỗi loại có ưu và nhược điểm riêng. Kiến trúc post-norm buộc biểu diễn phải được học qua nhiều hàm phi tuyến hơn, dẫn đến một mô hình có khả năng biểu đạt cao nhưng phức tạp và tương đối khó huấn luyện. Ngược lại, kiến trúc pre-norm giúp việc huấn luyện mô hình Transformer dễ hơn, nhưng có thể kém biểu đạt hơn so với kiến trúc post-norm tương ứng nếu mô hình học được trở nên quá phụ thuộc vào các đường tắt.

Một cải tiến cho các kiến trúc này là kiểm soát mức độ mà ta muốn "bỏ qua" một tầng con. Một cách đơn giản để làm việc này là gán trọng số cho các đường khác nhau thay vì coi chúng như nhau. Ví dụ, có thể đưa vào một hệ số vô hướng cho kết nối tắt để quyết định ta coi trọng kết nối tắt này đến mức nào so với đường đi qua hàm lõi [He và cộng sự, 2016b; Liu và cộng sự, 2020a;b]. Một dạng tổng quát hơn của mô hình này là

$$\mathbf{z}^l \;=\; \text{LNorm}(F^l(\mathbf{z}^{l-1}) + \beta \cdot \mathbf{z}^{l-1}) + \gamma \cdot \mathbf{z}^{l-1}, \tag{6.80}$$

trong đó β là trọng số của ánh xạ đồng nhất bên trong hàm chuẩn hóa tầng, và γ là trọng số của ánh xạ đồng nhất bên ngoài hàm chuẩn hóa tầng. Rõ ràng, cả hai kiến trúc post-norm và pre-norm đều có thể xem là trường hợp đặc biệt của phương trình này. Nghĩa là, nếu $\beta = 1$ và $\gamma = 0$ thì nó trở thành Phương trình (6.76); nếu $\beta = 0$ và $\gamma = 1$ thì nó trở thành Phương trình (6.77)[8]. Mô hình này đưa ra một góc nhìn nhiều nhánh khi xây dựng các khối phần dư. Đầu vào của khối này có thể được tính qua nhiều đường với độ phức tạp mô hình hóa khác nhau. Khi β và γ nhỏ, biểu diễn bị buộc phải học thông qua một mô hình "sâu" với nhiều tầng đơn vị phi tuyến nối tiếp nhau. Ngược lại, khi β và γ lớn, biểu diễn có nhiều khả năng được học bằng một mô hình "nông" với ít tầng hơn. Để xác định các giá trị tối ưu của β và γ, ta có thể gán cho chúng các giá trị cố định dựa trên một số tính chất lý thuyết hoặc hiệu năng của hệ thống trên tập kiểm định (*validation set*), hoặc tính các giá trị này bằng những hàm bổ sung có thể huấn luyện được để làm việc đó [Srivastava và cộng sự, 2015]. Cần nhấn mạnh rằng có thể cân nhắc nhiều loại kiến trúc khác khi thiết kế một tầng con Transformer. Chẳng hạn, có thể đưa thêm nhiều đơn vị chuẩn hóa tầng vào một tầng con [Ding và cộng sự, 2021; Wang và cộng sự, 2022b], hoặc ngược lại, đơn giản là loại bỏ chúng khỏi tầng con [Bachlechner và cộng sự, 2021].

2. Khởi tạo tham số

Cũng như với các mạng nơ-ron sâu khác, có nhiều sự quan tâm tới việc phát triển các phương pháp khởi tạo tham số (*parameter initialization*) cho mô hình Transformer sâu, nhằm giúp quá trình tối ưu diễn ra trong một vùng thuận lợi hơn của không gian tham số. Tuy nhiên, khởi tạo là một chủ đề rộng trong tối ưu các mô hình học máy, và việc bàn tổng quát về chủ đề này nằm ngoài phạm vi của mục này. Ở đây, ta sẽ tập trung vào các phương pháp khởi tạo tham số cụ thể được dùng trong các hệ thống dựa trên Transformer, thay vì các bài toán tối ưu tổng quát.

Mặc dù tham số của một mạng nơ-ron có thể được khởi tạo theo nhiều cách, hầu hết các hệ thống thực tế đều dùng những kỹ thuật đơn giản để đưa ra giá trị ban đầu thích hợp. Chẳng hạn, xét khởi tạo Xavier (*Xavier initialization*) cho một ma trận tham số $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ [Glorot và Bengio, 2010]. Ta định nghĩa một biến η như sau

$$\eta \;=\; \text{gain} \cdot \sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}, \tag{6.81}$$

trong đó gain là một siêu tham số, mặc định bằng 1. Khi đó, mỗi phần tử của $\mathbf{W}$ có thể được khởi tạo bằng một phân phối đều (*uniform distribution*)

$$W \;\sim\; U(-\eta, \eta) \tag{6.82}$$

[8] **[ND]** Chính xác hơn, với $\beta = 0, \gamma = 1$ ta được $\mathbf{z}^l = \text{LNorm}(F^l(\mathbf{z}^{l-1})) + \mathbf{z}^{l-1}$, tức là chuẩn hóa tầng đặt sau hàm lõi chứ không phải trước như Phương trình (6.77); tuy vậy, điểm cốt yếu giống pre-norm là ánh xạ đồng nhất $\mathbf{z}^{l-1}$ nằm ngoài hàm chuẩn hóa tầng và được cộng thẳng vào đầu ra.

hoặc dùng một phân phối Gauss (*Gaussian distribution*)

$$W \sim \mathcal{N}\left(0, \eta^2\right). \tag{6.83}$$

Chú giải của người dịch: Ví dụ khởi tạo Xavier

Giả sử $d_{\text{in}} = d_{\text{out}} = 512$ và gain $= 1$. Khi đó $\eta = \sqrt{6/1024} \approx 0.077$. Với Phương trình (6.82), mỗi phần tử của $\mathbf{W}$ được chọn ngẫu nhiên, "đều tay" trong khoảng $(-0.077;\ 0.077)$ (ký hiệu $\sim$ đọc là "được lấy mẫu theo"). Với Phương trình (6.83), mỗi phần tử được lấy theo phân phối chuẩn (hình chuông) có trung bình 0 và độ lệch chuẩn η. Ý tưởng: ma trận càng "to" ($d_{\text{in}} + d_{\text{out}}$ lớn) thì các số khởi tạo càng nhỏ, để tổng của rất nhiều tích không làm giá trị đầu ra phình to hoặc teo nhỏ khi đi qua mỗi tầng.

Phương pháp này có thể dễ dàng điều chỉnh để khởi tạo các mô hình Transformer có số tầng lớn. Một cách tiếp cận phổ biến là tìm một giá trị gain phù hợp hơn, khi tính đến thực tế rằng điều kiện ban đầu tối ưu cho việc tối ưu hóa có thể khác nhau giữa các mạng nơ-ron có độ sâu khác nhau. Ví dụ, ta có thể tăng giá trị gain khi độ sâu của mô hình tăng. Như vậy, gain có thể được định nghĩa là một hàm của độ sâu mạng theo dạng sau

$$\text{gain} = a \cdot L^b, \tag{6.84}$$

trong đó a là một số vô hướng, và L^b là độ sâu của mạng lũy thừa b. Thông thường, cả a và b đều là số dương, cho thấy xu hướng ưu tiên giá trị tham số ban đầu lớn hơn ở các mô hình sâu hơn. Ví dụ, Wang và cộng sự [2022a] chứng minh rằng bằng cách chọn giá trị phù hợp cho a và b, có thể huấn luyện thành công một mô hình Transformer rất sâu.

Phương trình (6.84) gán cho gain cùng một giá trị cho mọi tầng con. Tuy nhiên, người ta nhận thấy rằng chuẩn (*norm*) của gradient trở nên nhỏ hơn khi tầng con nằm càng xa tầng đầu ra. Việc áp dụng một giá trị gain thống nhất cho toàn bộ mô hình có thể khiến các tầng thấp bị huấn luyện chưa đủ do vấn đề tiêu biến gradient. Vì lý do này, ta có thể phát triển các phương pháp nhạy với vị trí của tầng con trong mạng nơ-ron. Dạng tổng quát của các phương pháp như vậy là

$$\text{gain} = \frac{a}{l^b}. \tag{6.85}$$

Ở đây l ký hiệu độ sâu của một tầng con. Nếu l lớn hơn (tức là tầng con gần đầu ra hơn), gain sẽ nhỏ hơn và các tham số tương ứng sẽ được đặt giá trị nhỏ hơn. Một ví dụ của phương pháp này có thể tìm thấy trong công trình của Zhang và cộng sự [2019a].

Chú giải của người dịch: So sánh hai cách chọn gain

Lấy $a = 1$, $b = 0.5$. Theo Phương trình (6.84), mạng có $L = 16$ tầng con dùng chung gain $= 16^{0.5} = 4$ cho *mọi* tầng. Theo Phương trình (6.85), mỗi tầng con có gain riêng: tầng $l = 1$ có gain $= 1$, tầng $l = 4$ có gain $= 1/2$, tầng $l = 16$ có gain $= 1/4$. Như vậy tầng càng gần đầu ra thì tham số khởi tạo càng nhỏ.

Tất nhiên, việc áp dụng các phương pháp tổng quát để khởi tạo mạng nơ-ron nhiều tầng sâu cho mô hình Transformer cũng rất đơn giản. Một ví dụ là xét **hằng số Lipschitz** (*Lipschitz constant*) khi khởi tạo tham số; điều này đã được chứng minh giúp cải thiện độ ổn định khi huấn luyện các mô hình sâu [Szegedy và cộng sự, 2014b; Xu và cộng sự, 2020]. Một cách tiếp cận khác là dùng các phương pháp bậc hai (*second-order methods*) để ước lượng giá trị phù hợp của các tham số. Ví dụ, ta có thể tính ma trận Hessian (*Hessian*) của mỗi ma trận tham số để mô hình hóa độ cong của nó [Skorski và cộng sự, 2021].

Chú giải của người dịch: Hằng số Lipschitz và Hessian

Một hàm f có *hằng số Lipschitz* K nếu $|f(x) - f(y)| \leq K\,|x - y|$ với mọi x, y: đầu vào thay đổi một chút thì đầu ra không thể thay đổi quá K lần lượng đó. Ví dụ $f(x) = 3x$ có $K = 3$. Giữ K của mỗi tầng không quá lớn giúp tín hiệu (và gradient) không bị phóng đại dần khi đi qua nhiều tầng. *Ma trận Hessian* là bảng các đạo hàm bậc hai; với hàm một biến nó chính là $f''(x)$ mà bạn đã biết cho biết đồ thị "cong" lên hay xuống và cong nhiều hay ít.

Với các mô hình có số tầng lớn, cũng có thể tiền huấn luyện một số tầng bằng các mô hình nhỏ hơn rồi dùng tham số đã huấn luyện của chúng để khởi tạo mô hình lớn hơn [Chen và cộng sự, 2015]. Nghĩa là, trước tiên ta thu được một ước lượng thô của các tham số theo cách ít tốn kém, rồi tiếp tục quá trình huấn luyện trên toàn bộ mô hình như thường lệ. Các phương pháp này thuộc về một nhóm phương pháp huấn luyện gọi là **tăng trưởng mô hình** (*model growth*) hoặc **tăng trưởng độ sâu** (*depth growth*).

Lấy một ví dụ đơn giản, xét một mô hình Transformer (chẳng hạn bộ mã hóa Transformer) có $2L$ tầng con. Ta có thể huấn luyện mô hình này bằng phương pháp **huấn luyện từ nông đến sâu** (*shallow-to-deep training*) [Li và cộng sự, 2020b]. Trước tiên, ta huấn luyện một mô hình L tầng con (gọi là mô hình nông) theo cách thông thường. Sau đó, ta tạo một mô hình $2L$ tầng con (gọi là mô hình sâu) bằng cách xếp chồng mô hình nông hai lần, rồi tiếp tục huấn luyện mô hình sâu này. Để xây dựng các mô hình sâu hơn nữa, quy trình này có thể được lặp lại nhiều lần; chẳng hạn, ta bắt đầu với một mô hình L tầng con và thu được một mô hình $L \cdot 2^I$ tầng con sau I lần lặp. Lưu ý rằng nhiều mô hình tiền huấn luyện cũng được dùng theo cách tương tự. Ví dụ, với các phương pháp kiểu BERT, một bộ mã hóa Transformer được huấn luyện trên dữ liệu quy mô lớn, rồi các tham số đã tối ưu được dùng để khởi tạo các hệ thống hạ nguồn (*downstream*).

3. Hợp nhất tầng

Một vấn đề khác khi huấn luyện mô hình Transformer sâu là dự đoán cuối cùng chỉ phụ thuộc vào tầng cuối cùng của mạng nơ-ron. Mặc dù việc dùng kết nối tắt cho phép một tầng cao truy cập trực tiếp vào các tầng thấp hơn, vẫn còn một đường "dài" để truyền thông tin từ dưới lên trên. Một cách đơn giản để giải quyết điều này là tạo ra các kết nối tắt bỏ qua nhiều tầng hơn. Ví dụ, xét một nhóm L tầng con Transformer. Với tầng con ở độ sâu l, ta có thể đưa vào $l - 1$ kết nối tắt, mỗi kết nối nối tầng con này với một tầng con phía trước. Bằng cách này, ta xây dựng một mạng kết nối dày đặc (*densely connected network*), trong đó mỗi tầng con nhận đầu vào là đầu ra của tất cả các tầng con trước nó [Huang và cộng sự, 2017a]. Đầu ra của tầng con cuối cùng có thể xem là sự kết hợp của các biểu diễn ở các cấp độ khác nhau.

Theo ký hiệu dùng ở các tiểu mục trước, ta ký hiệu đầu ra của tầng con ở độ sâu l là $\mathbf{z}^l$, và ký hiệu hàm của tầng con là $\text{Layer}^l(\cdot)$. Khi đó, $\mathbf{z}^l$ có thể được biểu diễn là

$$\mathbf{z}^l \;=\; \text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}). \tag{6.86}$$

Ta có thể đơn giản coi $\text{Layer}^l(\cdot)$ là một hàm hợp nhất thông tin từ $\{\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}\}$. Có nhiều lựa chọn khả dĩ cho $\text{Layer}^l(\cdot)$. Ví dụ, một dạng đơn giản của $\text{Layer}^l(\cdot)$ là

$$\text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \text{LNorm}(F^l(\mathbf{Z}^l)), \tag{6.87}$$

$$\mathbf{Z}^l = \phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}). \tag{6.88}$$

Ở đây $\phi(\cdot)$ nhận các đầu ra của tầng $\{\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}\}$ và hợp nhất chúng thành một biểu diễn duy nhất $\mathbf{Z}^l$. Một ví dụ đơn giản của $\phi(\cdot)$ là gộp trung bình (*average pooling*), tính tổng của $\{\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}\}$ rồi chia cho $l - 1$. Xem Bảng 6.2 để biết thêm các ví dụ về $\phi(\cdot)$.

Áp dụng một kiến trúc tương tự cho tầng con Transformer, ta cũng có thể xét dạng post-norm

$$\text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \text{LNorm}(\mathbf{Z}^l), \tag{6.89}$$

$$\mathbf{Z}^l = \phi(F^l(\mathbf{z}^{l-1}), \mathbf{z}^1, \cdots, \mathbf{z}^{l-1}), \tag{6.90}$$

hoặc dạng pre-norm

$$\text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \mathbf{Z}^l, \tag{6.91}$$

$$\mathbf{Z}^l = \phi(\text{LNorm}(F^l(\mathbf{z}^{l-1})), \mathbf{z}^1, \cdots, \mathbf{z}^{l-1}). \tag{6.92}$$

Bảng 6.2: Các hàm hợp nhất. FFN$(\cdot)$ = mạng nơ-ron truyền thẳng, $[\cdot]$ = ghép nối các vectơ đầu vào, và $\text{Att}_{\text{self}}(\cdot)$ = hàm tự chú ý. Mọi hàm hợp nhất đều có thể được theo sau bởi một hàm chuẩn hóa tầng; ví dụ, ta có thể viết tổng có trọng số của $\{\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}\}$ là $\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \text{LNorm}(\sum_{k=1}^{l-1} \text{weight}_k \cdot \mathbf{z}^k)$.

Mục	Hàm
Gộp trung bình	$\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \frac{1}{l-1}\sum_{k=1}^{l-1} \mathbf{z}^k$
Tổng có trọng số	$\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \sum_{k=1}^{l-1} \text{weight}_k \cdot \mathbf{z}^k$
Mạng truyền thẳng	$\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \text{FFN}([\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}])$
Tự chú ý	$\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \text{FFN}([\text{Att}_{\text{self}}(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1})])$

Các mô hình này khá tổng quát. Ví dụ, một tầng con bộ mã hóa post-norm chuẩn có thể thu lại được như một trường hợp đặc biệt của các Phương trình (6.89–6.90), nếu ta bỏ sự phụ thuộc vào các tầng con từ 1 đến $l-2$, và định nghĩa $\phi(\cdot)$ là

$$\phi(F^l(\mathbf{z}^{l-1}), \mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) \;=\; F^l(\mathbf{z}^{l-1}) + \mathbf{z}^{l-1}. \tag{6.93}$$

Chú giải của người dịch: Hợp nhất tầng bằng ví dụ số

Giả sử ta đang ở tầng con $l = 4$ và (cho gọn) mỗi đầu ra chỉ là một số: $z^1 = 2$, $z^2 = 4$, $z^3 = 9$. Gộp trung bình cho $\phi = (2 + 4 + 9)/3 = 5$. Tổng có trọng số với weight $= (0.1;\ 0.2;\ 0.7)$ cho $\phi = 0.2 + 0.8 + 6.3 = 7.3$ — tầng gần nhất được coi trọng nhất. Các trọng số này có thể được học trong quá trình huấn luyện. Nhờ vậy, tầng 4 "nhìn thấy" trực tiếp cả tầng 1 chứ không chỉ tầng 3 như Transformer thông thường.

Các mạng kết nối dày đặc giúp thông tin dễ chảy hơn qua các kết nối trực tiếp giữa các tầng con, nhưng mô hình thu được phức tạp hơn một chút, nhất là khi ta dùng các hàm hợp nhất có tham số. Trong thực tế, ta thường chỉ thêm kết nối dày đặc cho một tập con các tầng con, để toàn bộ mạng không quá dày đặc. Ví dụ, ta có thể chỉ thêm kết nối từ các tầng con thấp tới vài tầng con cuối cùng. Nhờ đó, việc dự đoán có thể được thực hiện với khả năng truy cập trực tiếp vào các cấp độ biểu diễn khác nhau [Wang và cộng sự, 2018a].

4. Chính quy hóa

Trong học máy, chính quy hóa (*regularization*) được dùng để giảm nhẹ hiện tượng quá khớp (*overfitting*) khi huấn luyện các mạng nơ-ron sâu. Vì vậy, việc áp dụng các kỹ thuật chính quy hóa cho mô hình Transformer là điều đương nhiên. Do chủ đề chính quy hóa đã được bàn ở Chương 2, ở đây ta xét một số phương pháp chưa được đề cập trong sách này nhưng đặc biệt hữu ích cho việc huấn luyện mô hình Transformer sâu.

Một cách tiếp cận để chính quy hóa mô hình Transformer sâu là bỏ qua ngẫu nhiên các tầng con hoặc các tầng trong quá trình huấn luyện [Huang và cộng sự, 2016; Pham và cộng sự, 2019]. Trong mỗi

lần chạy mô hình, ta giữ lại mỗi tầng con với xác suất ρ và xếp chồng các tầng con được chọn để tạo thành một mô hình mới. Như vậy, về bản chất ta huấn luyện một tập hợp mô hình (*ensemble*) gồm các mạng nơ-ron khác nhau có chung kiến trúc và tham số trên cùng một tập dữ liệu. Bằng cách này, mỗi tầng con học cách hoạt động tương đối độc lập, và hiện tượng quá khớp được giảm bớt nhờ ngăn chặn sự đồng thích nghi (*co-adaptation*) giữa các tầng con. Thực ra, dropout các tầng con (hoặc các tầng) và dropout các nơ-ron về cơ bản là những biến thể của cùng một ý tưởng. Đôi khi, phương pháp mô tả ở đây được gọi là **dropout tầng con** (*sub-layer dropout*) hoặc **dropout tầng** (*layer dropout*).

Tại thời điểm kiểm tra, ta cần kết hợp tất cả các mạng khả dĩ để đưa ra dự đoán. Một cách đơn giản để làm điều này là co giãn lại đầu ra của các thành phần ngẫu nhiên trong mô hình [Li và cộng sự, 2021a]. Ví dụ, giả sử mỗi tầng con có kiến trúc pre-norm. Khi đó, đầu ra của tầng con ở độ sâu l là

$$\mathbf{z}^l \;=\; \rho \cdot \text{LNorm}(F^l(\mathbf{z}^{l-1})) + \mathbf{z}^{l-1}. \tag{6.94}$$

Chú giải của người dịch: Vì sao nhân với ρ khi kiểm tra?

Khi huấn luyện, nhánh $\text{LNorm}(F^l(\cdot))$ chỉ "có mặt" với xác suất ρ; nếu vắng mặt, tầng con chỉ chuyển nguyên $\mathbf{z}^{l-1}$ lên trên. Vậy *trung bình* (kỳ vọng) đóng góp của nhánh này là $\rho \cdot \text{LNorm}(F^l(\mathbf{z}^{l-1})) + (1-\rho)\cdot 0$. Khi kiểm tra, ta giữ mọi tầng con nhưng nhân nhánh đó với ρ để đầu ra có cùng "độ lớn trung bình" như lúc huấn luyện. Ví dụ $\rho = 0.8$: nhánh lõi chỉ được tính 80%.

Một ý tưởng khác là chia sẻ tham số giữa các tầng con. Một trong những phương pháp đơn giản nhất là dùng đúng cùng một bộ tham số cho mọi tầng con tương ứng [Dehghani và cộng sự, 2018]. Ví dụ, mọi tầng con FFN có thể dựa trên cùng một mạng truyền thẳng. Cách tiếp cận này có tác dụng chính quy hóa tương tự các phương pháp cộng chuẩn của các ma trận tham số vào hàm mất mát để phạt độ phức tạp của mô hình. Với các hệ thống thực tế, việc dùng kiến trúc chia sẻ có thể mang lại lợi ích đáng kể, vì nó cho phép ta dùng lại cùng một mô hình con để xây dựng một mạng nơ-ron sâu, nhờ đó giảm đáng kể tổng lượng bộ nhớ sử dụng. Ta sẽ bàn thêm về các vấn đề hiệu quả ở Mục 6.4.4.

6.3.3 Các mô hình lấy cảm hứng từ phương pháp số

Một mạng phần dư tính đầu ra của nó bằng tổng của ánh xạ đồng nhất và một phép biến đổi của đầu vào. Mô hình như vậy có thể được hiểu là một rời rạc hóa Euler (*Euler discretization*) của các **phương trình vi phân thường** (*ordinary differential equations*) (**ODE**) [Ee, 2017; Haber và Ruthotto, 2017]. Để minh họa ý tưởng này, ta xét một dạng tổng quát của mạng phần dư

$$\mathbf{z}^l \;=\; f^l\big(\mathbf{z}^{l-1}\big) + \mathbf{z}^{l-1}, \tag{6.95}$$

trong đó $f^l(\mathbf{z}^{l-1})$ ký hiệu một hàm nhận biến đầu vào $\mathbf{z}^{l-1}$ và tạo ra một biến đầu ra trong cùng không gian. Rõ ràng, một tầng con Transformer là trường hợp đặc biệt của phương trình này. Ví dụ, với Transformer pre-norm, ta có $f^l(\cdot) = \text{LNorm}(F^l(\cdot))$.

Để đơn giản ký hiệu, ta viết lại phương trình trên dưới một dạng tương đương

$$\mathbf{z}(l) \;=\; f\big(\mathbf{z}(l-1), l\big) + \mathbf{z}(l-1). \tag{6.96}$$

Ta dùng các ký hiệu $\mathbf{z}(l)$ và $f(\mathbf{z}(\cdot, l))$ để nhấn mạnh rằng $\mathbf{z}(\cdot)$ và $f(\cdot)$ là các hàm của l. Ở đây ta giả sử l là một biến rời rạc. Nếu ta nới lỏng l thành một biến liên tục và $\mathbf{z}(l)$ thành một hàm liên tục của l, thì có thể biểu diễn Phương trình (6.96) là

$$\mathbf{z}(l) \;=\; \triangle l \cdot f\big(\mathbf{z}(l - \triangle l), l\big) + \mathbf{z}(l - \triangle l). \tag{6.97}$$

Phương trình này có thể viết tiếp thành

$$\frac{\mathbf{z}(l) - \mathbf{z}(l - \triangle l)}{\triangle l} \;=\; f\big(\mathbf{z}(l - \triangle l), l\big). \tag{6.98}$$

Lấy giới hạn $\triangle l \to 0$, ta được một ODE

$$\frac{\mathrm{d}\mathbf{z}(l)}{\mathrm{d}l} \;=\; f\big(\mathbf{z}(l), l\big). \tag{6.99}$$

Do đó, ta có thể nói rằng một tầng con Transformer pre-norm (tức các Phương trình (6.96) và (6.95)) là một rời rạc hóa Euler của ODE trên. Đây là một kết quả thú vị! Về bản chất, một tầng con đóng vai trò như một bộ giải ODE (*ODE solver*).

Chú giải của người dịch: Phương trình vi phân và phương pháp Euler

Vế trái của Phương trình (6.98) chính là "tỉ số gia số" $\frac{\triangle y}{\triangle x}$ mà bạn gặp khi học định nghĩa đạo hàm ở lớp 11; cho $\triangle l \to 0$ ta được đạo hàm, đó là Phương trình (6.99). Một *phương trình vi phân* cho biết "tốc độ thay đổi" của một đại lượng thay vì cho chính đại lượng đó. *Phương pháp Euler* giải gần đúng bằng cách đi từng bước nhỏ: giá trị mới = giá trị cũ + bước × tốc độ. Ví dụ với $\frac{\mathrm{d}z}{\mathrm{d}l} = z$, $z(0) = 1$ (nghiệm đúng $z = e^l$) và bước 1: $z(1) = 1 + 1 \cdot 1 = 2$, $z(2) = 2 + 1 \cdot 2 = 4$ (nghiệm đúng $e^2 \approx 7.39$; bước càng nhỏ thì càng chính xác). Một tầng con có kết nối tắt làm đúng việc "giá trị cũ $\mathbf{z}^{l-1}$ + thay đổi $f^l(\mathbf{z}^{l-1})$", nên mỗi tầng tương ứng với một bước Euler có độ dài bước bằng 1.

Các Phương trình (6.96) và (6.95) biểu diễn các dạng chuẩn của **phương pháp Euler** (*Euler method*). Phương pháp này tính một ước lượng mới của nghiệm bằng cách tiến thêm một bước dọc theo l từ ước lượng trước đó. Nói chung, có hai khía cạnh có thể được cân nhắc khi thiết kế các phương pháp số cho ODE.

- **Phương pháp đa bước tuyến tính** (*Linear Multi-step Methods*). Một phương pháp đa bước tuyến tính tính ước lượng hiện tại của nghiệm bằng cách lấy các ước lượng và thông tin đạo hàm từ nhiều bước trước đó. Công thức tổng quát của phương pháp p bước có thể được viết là

$$\mathbf{z}(l) \;=\; \sum_{i=1}^{p} a_i \cdot \mathbf{z}(l-i) + h \sum_{i=1}^{p+1} b_i \cdot f\big(\mathbf{z}(l-i),\, l-i+1\big), \tag{6.100}$$

  trong đó h là độ dài bước ta tiến mỗi lần[9], tức là $\triangle l$ trong các Phương trình (6.97) và (6.98). $\{a_i\}$ và $\{b_i\}$ là các hệ số của các điểm nghiệm và các đạo hàm trong tổ hợp tuyến tính. Với định nghĩa này, ta có thể coi phương pháp Euler là một phương pháp giải ODE một bước, bậc thấp[10].

- **Phương pháp Runge–Kutta (bậc cao)** (*(Higher-order) Runge-Kutta Methods*). Các phương pháp Runge–Kutta (RK) và các biến thể của chúng cho phép tính nghiệm ở bước tiếp theo bằng cách sử dụng các kết quả trung gian khi giải ODE. Kết quả là ta thu được các phương pháp bậc cao hơn nhưng vẫn theo dạng phương pháp một bước, nghĩa là nghiệm ước lượng chỉ phụ thuộc vào $\mathbf{z}(l-1)$ chứ không phụ thuộc vào đầu ra ở nhiều bước trước đó.

[9] Gọi $\{t_0, \cdots, t_i\}$ là các giá trị của biến l tại các bước $\{0, \cdots, i\}$. Trong các phương pháp đa bước tuyến tính, ta giả sử $t_i = t_0 + ih$.

[10] Trong giải tích số, **sai số chặt cụt địa phương** (*local truncation error*) của một phương pháp giải ODE tại một bước được định nghĩa là hiệu giữa nghiệm xấp xỉ tính bằng phương pháp đó và nghiệm đúng. Phương pháp được gọi là có bậc p nếu nó có sai số chặt cụt địa phương là $O(h^{p+1})$.

Chú giải của người dịch: Ký hiệu $O(h^{p+1})$

$O(h^{p+1})$ đọc là "bậc h^{p+1}": sai số nhỏ cỡ h^{p+1} khi bước h nhỏ. Ví dụ với $h = 0.1$: phương pháp bậc 1 (như Euler) có sai số mỗi bước cỡ $0.1^2 = 0.01$, còn phương pháp bậc 4 có sai số mỗi bước cỡ $0.1^5 = 0.00001$. Bậc càng cao thì giảm bước đi một chút sẽ giảm sai số đi rất nhiều.

Thực ra, các phương pháp đa bước tuyến tính, dù không được nhắc tới một cách tường minh, đã được dùng trong các phương pháp hợp nhất tầng bàn ở Mục 6.3.2. Ví dụ, lấy các Phương trình (6.91) và (6.92) cùng một hàm hợp nhất tuyến tính, một tầng con pre-norm có kết nối dày đặc tới mọi tầng con trước đó có thể được biểu diễn là

$$\text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) \;=\; a_1 \cdot \mathbf{z}^{l-1} + \cdots + a_{l-1} \cdot \mathbf{z}^1 + b_1 \cdot \text{LNorm}(F^l(\mathbf{z}^{l-1})). \tag{6.101}$$

Phương trình này là một trường hợp cụ thể của Phương trình (6.100), trong đó ta đặt $h = 1$ và bỏ đi một số số hạng ở vế phải.

Việc áp dụng các phương pháp Runge–Kutta cho mô hình Transformer cũng rất đơn giản [Li và cộng sự, 2022a]. Cho một ODE như mô tả trong Phương trình (6.99), nghiệm Runge–Kutta hiển bậc p được cho bởi

$$\mathbf{z}(l) = \mathbf{z}(l-1) + \sum_{i=1}^{p} \gamma_i \cdot \mathbf{g}_i, \tag{6.102}$$

$$\mathbf{g}_i = h \cdot f\Big(\mathbf{z}(l-1) + \sum_{j=1}^{i-1} \beta_{i,j} \cdot \mathbf{g}_j,\; l - 1 + \lambda_i \cdot h\Big). \tag{6.103}$$

Ở đây $\mathbf{g}_i$ biểu diễn một bước trung gian, chỉ xuất hiện trong quá trình trên. $\{\gamma_i\}$, $\{\beta_{i,j}\}$ và $\{\lambda_i\}$ là các hệ số được xác định bằng cách dùng chuỗi Taylor (*Taylor series*) của $\mathbf{z}(l)$. Để đơn giản hóa mô hình, ta giả sử cùng một hàm f được dùng cho mọi $\{\mathbf{g}_i\}$. Khi đó, ta bỏ sự phụ thuộc vào số hạng $l - 1 + \lambda_i \cdot h$ trong f, và viết lại Phương trình (6.103) thành

$$\mathbf{g}_i \;=\; h \cdot f\Big(\mathbf{z}(l-1) + \sum_{j=1}^{i-1} \beta_{i,j} \cdot \mathbf{g}_j\Big), \tag{6.104}$$

trong đó $f(\cdot)$ là một hàm không phụ thuộc vào i.

Lấy ví dụ, xét nghiệm Runge–Kutta bậc 4 (RK4)

$$\mathbf{z}(l) = \mathbf{z}(l-1) + \frac{1}{6}(\mathbf{g}_1 + 2\mathbf{g}_2 + 2\mathbf{g}_3 + \mathbf{g}_4), \tag{6.105}$$

$$\mathbf{g}_1 = h \cdot f(\mathbf{z}(l-1)), \tag{6.106}$$

$$\mathbf{g}_2 = h \cdot f(\mathbf{z}(l-1) + \tfrac{1}{2}\mathbf{g}_1), \tag{6.107}$$

$$\mathbf{g}_3 = h \cdot f(\mathbf{z}(l-1) + \tfrac{1}{2}\mathbf{g}_2), \tag{6.108}$$

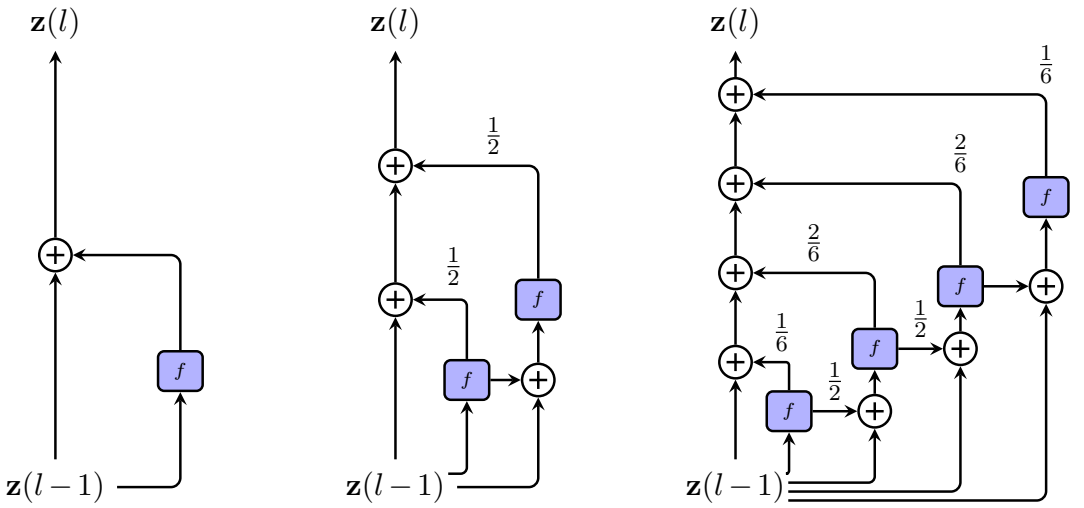
$$\mathbf{g}_4 = h \cdot f(\mathbf{z}(l-1) + \mathbf{g}_3). \tag{6.109}$$

Các phương trình này định nghĩa một kiến trúc tầng con mới. Ví dụ, bằng cách đặt $h = 1$ và $f(\cdot) = \text{LNorm}(F^l(\cdot))$, ta thu được một tầng con Transformer RK4, như minh họa trong Hình 6.11. Phương pháp này dẫn tới một mô hình sâu, vì mỗi tầng con bao gồm bốn lần tính $f(\cdot)$ nối tiếp nhau. Mặt khác, mô hình thu được lại tiết kiệm tham số, vì ta dùng lại cùng một hàm $f(\cdot)$ bên trong tầng con mà không đưa thêm tham số mới.

Chú giải của người dịch: RK4 bằng ví dụ số

Lấy lại ví dụ $\frac{dz}{dl} = z$, tức $f(z) = z$, với $z(0) = 1$, $h = 1$: $g_1 = 1$; $g_2 = 1 + 0.5 \cdot 1 = 1.5$; $g_3 = 1 + 0.5 \cdot 1.5 = 1.75$; $g_4 = 1 + 1.75 = 2.75$. Suy ra $z(1) = 1 + \frac{1}{6}(1 + 3 + 3.5 + 2.75) \approx 2.708$, rất gần nghiệm đúng $e \approx 2.718$, trong khi một bước Euler chỉ cho 2. Đổi lại, RK4 phải tính f bốn lần cho mỗi bước. Trong Transformer RK4, f là cùng một mạng con được gọi bốn lần, nên số tham số không đổi nhưng lượng tính toán tăng.

Cho đến đây trong tiểu mục này, phần thảo luận của ta tập trung vào thiết kế các kiến trúc Transformer lấy cảm hứng từ các hệ động lực (*dynamical systems*). Mặc dù mô hình ODE cơ bản là liên tục theo độ sâu l, các phương pháp này vẫn theo khuôn khổ chung của mạng nơ-ron, trong đó l được coi là biến rời rạc, và năng lực biểu diễn của mô hình phần lớn được quyết định bởi siêu tham số này. Một cách tiếp cận khác là dùng ODE nơ-ron (*neural ODE*) để nới lỏng "độ sâu" thành một biến liên tục. Bằng cách này, ta có thể thu được một mô hình có độ sâu liên tục để tính nghiệm của ODE. Tuy nhiên, vì thảo luận chi tiết về ODE nơ-ron nằm ngoài phạm vi chương này, chúng tôi giới thiệu bạn đọc quan tâm tới các bài báo liên quan để biết thêm chi tiết [Chen và cộng sự, 2018c; Kidger, 2022; Xiao và cộng sự, 2026].



(a) Pre-norm (b) RK2 (c) RK4

Hình 6.11: Minh họa kiến trúc tầng con pre-norm (a) và Runge–Kutta (b và c). $\mathbf{z}(l-1)$ ký hiệu đầu vào của tầng con ở độ sâu l, $\mathbf{z}(l)$ ký hiệu đầu ra của tầng con, và f (trong các hộp màu xanh) ký hiệu hàm $f(\cdot) = \text{LNorm}(F^l(\cdot))$.

6.3.4 Các mô hình rộng

Phần lớn các phương pháp ta đã nghiên cứu trong mục này là những ví dụ về việc học và sử dụng mô hình sâu. Một lựa chọn thiết kế khác mà ta thường gặp là xác định độ rộng của mạng nơ-ron. Thông thường, **độ rộng** (*width*) của một mô hình Transformer được định nghĩa là số chiều của biểu diễn tại một vị trí nào đó trong chuỗi đầu vào, tức là tham số d. Tăng độ rộng này là một cách phổ biến để có được một mô hình phức tạp và mạnh hơn. Ví dụ, trong công trình của Vaswani và cộng sự [2017], một mô hình rộng (gọi là Transformer-Big) mang lại cải thiện đáng kể về chất lượng dịch cho các hệ thống

dịch máy (*machine translation*). Những mô hình rộng như vậy đã được dùng rộng rãi để nâng cao hiệu năng hệ thống trên các tác vụ quy mô lớn [Lepikhin và cộng sự, 2021; Fedus và cộng sự, 2022b].

Tuy nhiên, việc phát triển các mô hình Transformer rất rộng là khó khăn. Một thách thức lớn là huấn luyện những hệ thống như vậy rất tốn kém về tính toán. Trong khi số tham số của mô hình (kích thước mô hình) tăng tuyến tính theo d, thì độ phức tạp thời gian của mô hình tăng theo bình phương của d (xem Bảng 6.1). Trong một số tác vụ NLP, người ta nhận thấy bằng thực nghiệm rằng công sức huấn luyện cần thiết để đạt hiệu năng thỏa đáng tuân theo một định luật co giãn (*scaling law*) [Kaplan và cộng sự, 2020]. Những kết quả này nhấn mạnh sự cần thiết của các cách cải thiện hiệu quả huấn luyện khi tăng d.

Một phương pháp đơn giản là tăng dần mô hình theo chiều d, thay vì huấn luyện mô hình từ đầu. Giả sử ta có một mô hình ban đầu với ma trận tham số $\mathbf{W}_1$ kích thước $d_1 \times d_1$, chẳng hạn phép biến đổi tuyến tính của mỗi truy vấn hoặc khóa trong một tầng nào đó. Ta có thể huấn luyện mô hình này theo cách thông thường để thu được $\mathbf{W}_1$ đã tối ưu. Sau đó, ta muốn mở rộng mô hình này thành một mô hình rộng hơn, trong đó $\mathbf{W}_1$ được thay bằng ma trận tham số $\mathbf{W}_2$ kích thước $d_2 \times d_2$. Để đơn giản, giả sử $d_2 = kd_1$. Có một vài cách để mở rộng một ma trận $d_1 \times d_1$ thành một ma trận $kd_1 \times kd_1$. Đơn giản nhất có lẽ là dùng $\mathbf{W}_1$ để lấp đầy $\mathbf{W}_2$. Ta có thể viết $\mathbf{W}_2$ dưới dạng

$$\mathbf{W}_2 = \overbrace{\begin{bmatrix} \frac{\mathbf{W}_1}{\rho} & \cdots & \frac{\mathbf{W}_1}{\rho} \\ \vdots & & \vdots \\ \frac{\mathbf{W}_1}{\rho} & \cdots & \frac{\mathbf{W}_1}{\rho} \end{bmatrix}}^{k \text{ lần}} \Bigg\} k \text{ lần}, \tag{6.110}$$

trong đó ρ là một siêu tham số dùng để kiểm soát chuẩn của $\mathbf{W}_2$. Ví dụ, nếu $\rho = k$ thì $\mathbf{W}_2$ sẽ có cùng chuẩn l_1 với $\mathbf{W}_1$. Phương trình trên cung cấp một điểm xuất phát tốt cho việc huấn luyện mô hình rộng, và ta có thể huấn luyện $\mathbf{W}_2$ như thường lệ sau khi khởi tạo. Quy trình này có thể được lặp lại nhiều lần để xây dựng mô hình có độ rộng tùy ý. Cả phương pháp này lẫn phương pháp tăng trưởng độ sâu mô tả ở Mục 6.3.2 đều là những trường hợp cụ thể của phương pháp tổng quát tăng trưởng mô hình. Nói cách khác, ta có thể thu được một mô hình lớn hơn bằng cách mở rộng một mô hình nhỏ theo chiều dọc hoặc chiều ngang, hoặc cả hai. Các phương pháp khác để biến đổi $\mathbf{W}_1$ thành $\mathbf{W}_2$ bao gồm những phương pháp xét tới các tính chất toán học khác của phép biến đổi [Chen và cộng sự, 2015]. Các mô hình này có thể xếp vào nhóm mạng nơ-ron tái sử dụng (*reusable neural networks*), nơi ta quan tâm tới các mô hình và thuật toán để chuyển tham số từ mô hình nhỏ sang mô hình lớn hơn (đáng kể) [Wang và cộng sự, 2023b].

Chú giải của người dịch: Ví dụ mở rộng ma trận

Lấy $k = 2$, $\mathbf{W}_1 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ ($d_1 = 2$) và $\rho = 2$. Khi đó $\mathbf{W}_2$ là ma trận 4×4 gồm 2×2 khối, mỗi khối là $\mathbf{W}_1/2 = \begin{bmatrix} 0.5 & 1 \\ 1.5 & 2 \end{bmatrix}$. Nếu đầu vào mới cũng là đầu vào cũ được "lặp đôi", ví dụ $[x_1, x_2, x_1, x_2]$, thì mỗi nửa đầu ra của $\mathbf{W}_2$ đúng bằng đầu ra cũ của $\mathbf{W}_1$ (vì hai khối, mỗi khối chia 2, cộng lại). Như vậy mô hình rộng "kế thừa" được những gì mô hình hẹp đã học. (Chuẩn l_1 của ma trận ở đây có thể hiểu là tổng giá trị tuyệt đối lớn nhất theo một cột; mỗi cột của $\mathbf{W}_2$ gồm k khối chia cho $\rho = k$, nên tổng này không đổi.)

Khó khăn thứ hai khi xây dựng một mô hình Transformer rộng là yêu cầu bộ nhớ lớn. Do mạng truyền thẳng thường có tầng ẩn lớn hơn các phần khác của mô hình, nó cần tương đối nhiều bộ nhớ hơn

khi mô hình trở nên rộng hơn. Xét mạng truyền thẳng mô tả ở Mục 6.1.5

$$\begin{aligned}\mathbf{H}_{\text{out}} &= \text{FFN}(\mathbf{H}_{\text{in}}) \\ &= \text{ReLU}(\mathbf{H}_{\text{in}} \cdot \mathbf{W}_h + \mathbf{b}_h) \cdot \mathbf{W}_f + \mathbf{b}_f,\end{aligned} \tag{6.111}$$

trong đó $\mathbf{W}_h \in \mathbb{R}^{d \times d_{\text{ffn}}}$ và $\mathbf{W}_f \in \mathbb{R}^{d_{\text{ffn}} \times d}$ là các tham số của các phép biến đổi tuyến tính. d_{ffn} thường lớn hơn d vài lần. Do đó, $\mathbf{W}_h$ và $\mathbf{W}_f$ sẽ chiếm phần lớn kích thước mô hình nếu d và d_{ffn} có giá trị rất lớn.

Trong một số trường hợp, kích thước của mạng truyền thẳng có thể vượt quá dung lượng bộ nhớ của một thiết bị đơn lẻ. Vấn đề này có thể được giải quyết bằng các mô hình **hỗn hợp chuyên gia** (*mixture-of-experts*) (**MoE**) [Shazeer và cộng sự, 2017]. Một mô hình MoE gồm M mô hình chuyên gia $\{e_1(\cdot), \cdots, e_M(\cdot)\}$. Cho một đầu vào $\mathbf{h}_{\text{in}} \in \mathbb{R}^d$, mỗi mô hình chuyên gia tạo ra một đầu ra $e_k(\mathbf{h}_{\text{in}})$. Đầu ra của mô hình MoE là một tổ hợp tuyến tính của $\{e_1(\mathbf{h}_{\text{in}}), \cdots, e_M(\mathbf{h}_{\text{in}})\}$, cho bởi

$$\mathbf{h}_{\text{out}} = \sum_{i=1}^{M} g_i(\mathbf{h}_{\text{in}}) \cdot e_i(\mathbf{h}_{\text{in}}), \tag{6.112}$$

trong đó $g(\cdot)$ là một mô hình cổng (*gating model*) (còn gọi là **mô hình định tuyến** (*routing model*)). Đầu ra của nó là một vectơ $g(\mathbf{h}_{\text{in}}) = \begin{bmatrix} g_1(\mathbf{h}_{\text{in}}) & \cdots & g_M(\mathbf{h}_{\text{in}}) \end{bmatrix}$, trong đó mỗi phần tử $g_i(\mathbf{h}_{\text{in}})$ cho biết trọng số của mô hình chuyên gia tương ứng. Trong nhiều ứng dụng, người ta giả sử $g(\mathbf{h}_{\text{in}})$ là một vectơ thưa (*sparse vector*). Điều này có nghĩa là chỉ một số ít mô hình chuyên gia tham gia vào việc tính đầu ra. Một dạng $g(\mathbf{h}_{\text{in}})$ được dùng rộng rãi là dùng tầng Softmax

$$g(\mathbf{h}_{\text{in}}) = \text{Softmax}(\mathbf{h}_{\text{in}} \cdot \mathbf{W}_g), \tag{6.113}$$

trong đó $\mathbf{W}_g \in \mathbb{R}^{d \times M}$ là ma trận tham số của tầng này. Để buộc $g(\mathbf{h}_{\text{in}})$ thưa, ta có thể chọn k phần tử lớn nhất (top-k) của $g(\mathbf{h}_{\text{in}})$, tức là đặt các phần tử không thuộc top-k bằng 0. Một phương pháp khác là trước tiên chọn top-k trên $\mathbf{h}_{\text{in}} \cdot \mathbf{W}_g$, rồi chuẩn hóa k phần tử đó bằng hàm Softmax.

Gọi π là tập chỉ số của k mô hình chuyên gia được chọn. Mô hình MoE với định tuyến top-k có dạng sau

$$\mathbf{h}_{\text{out}} = \sum_{i \in \pi} g_i(\mathbf{h}_{\text{in}}) \cdot e_i(\mathbf{h}_{\text{in}}). \tag{6.114}$$

Chú giải của người dịch: MoE bằng ví dụ

Hãy hình dung một bệnh viện có $M = 4$ bác sĩ chuyên khoa (các "chuyên gia") và một quầy tiếp đón (mô hình cổng). Mỗi bệnh nhân (đầu vào) chỉ được chuyển tới $k = 2$ bác sĩ phù hợp nhất, chứ không phải gặp cả bốn. Bằng số: giả sử $\mathbf{h}_{\text{in}} \cdot \mathbf{W}_g = (2;\ 0;\ 1;\ -1)$. Chọn top-2 là chuyên gia 1 và 3 (điểm 2 và 1), nên $\pi = \{1, 3\}$. Chuẩn hóa bằng Softmax trên hai điểm này: $g_1 = \frac{e^2}{e^2+e^1} \approx 0.73$, $g_3 \approx 0.27$. Đầu ra là $0.73 \cdot e_1(\mathbf{h}_{\text{in}}) + 0.27 \cdot e_3(\mathbf{h}_{\text{in}})$; chuyên gia 2 và 4 hoàn toàn không phải chạy, nhờ đó tiết kiệm tính toán dù tổng số tham số rất lớn.

Một ưu điểm của cách tiếp cận này là ta có thể phân bổ các mô hình chuyên gia khác nhau lên các bộ xử lý khác nhau, giúp có thể thực thi các mô hình này trên các máy tính toán song song. Trong mỗi lần chạy mô hình MoE, dù trong huấn luyện hay suy luận (*inference*), ta chỉ cần kích hoạt và sử dụng k mô hình chuyên gia thay vì tất cả. Bằng cách này, cách tiếp cận MoE tự động học một mô hình thưa bằng cách giới hạn số mô hình chuyên gia hoạt động mỗi lần trong huấn luyện và suy luận. Độ thưa được quyết định bởi siêu tham số k; chẳng hạn, giá trị k nhỏ dẫn tới một mô hình thưa, còn giá trị k lớn dẫn tới một mô hình dày đặc.

Hãy quay lại thảo luận về Phương trình (6.111). Việc áp dụng cách tiếp cận MoE cho mạng nơ-ron truyền thẳng là đơn giản. Để đơn giản hóa thảo luận, xét phép biến đổi tuyến tính của tầng thứ nhất như trong Phương trình (6.111), tức là $\mathbf{H}_{\text{in}} \cdot \mathbf{W}_h$. Ta có thể xấp xỉ $\mathbf{H}_{\text{in}} \cdot \mathbf{W}_h$ dưới dạng MoE

$$\begin{aligned}\mathbf{H}_{\text{in}} \cdot \mathbf{W}_h &\approx \sum_{i \in \pi} g_i(\mathbf{H}_{\text{in}}) \cdot e_i(\mathbf{H}_{\text{in}}) \\ &= \sum_{i \in \pi} g_i(\mathbf{H}_{\text{in}}) \cdot [\mathbf{H}_{\text{in}} \cdot \mathbf{W}_h^i].\end{aligned} \tag{6.115}$$

Ở đây $\mathbf{W}_h$ được chia thành M lát (hay ma trận con) $\{\mathbf{W}_h^1, \cdots, \mathbf{W}_h^M\}$, viết là

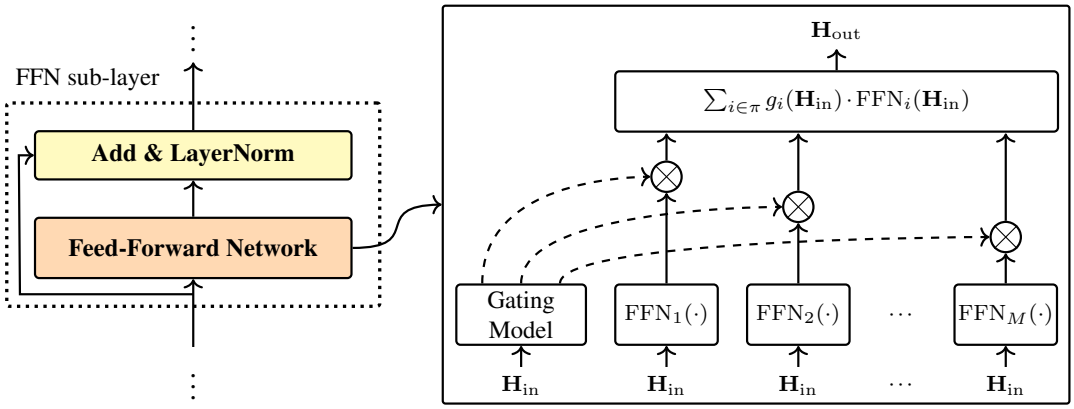
$$\mathbf{W}_h \;=\; \begin{bmatrix}\mathbf{W}_h^1 & \cdots & \mathbf{W}_h^M\end{bmatrix}. \tag{6.116}$$

Vì vậy, mỗi mô hình chuyên gia $e_i(\mathbf{H}_{\text{in}}) = \mathbf{H}_{\text{in}} \cdot \mathbf{W}_h^i$ giải một bài toán con của phép ánh xạ tuyến tính ban đầu, và Phương trình (6.115) có thể được xem là một lời giải theo kiểu chia để trị (*divide-and-conquer*) cho bài toán nhân ma trận.

Tất nhiên, ta có thể coi bất kỳ mạng nơ-ron truyền thẳng nào là một mô hình chuyên gia, dẫn tới mô hình sau

$$\mathbf{H}_{\text{out}} \;=\; \sum_{i \in \pi} g_i(\mathbf{H}_{\text{in}}) \cdot \text{FFN}_i(\mathbf{H}_{\text{in}}), \tag{6.117}$$

trong đó $\text{FFN}_i(\cdot)$ là một mạng nơ-ron truyền thẳng "nhỏ" có cùng dạng với Phương trình (6.111). Mô hình này được minh họa bằng một ví dụ trong Hình 6.12. Trong các cài đặt thực tế, mọi mô hình chuyên gia này có thể chạy song song trên các thiết bị khác nhau, nên hệ thống thu được rất hiệu quả.



Hình 6.12: Minh họa mô hình MoE áp dụng cho một tầng con FFN. Có M mạng FFN (gọi là các mô hình chuyên gia) và một mô hình cổng. Mỗi FFN được gán trọng số bởi mô hình cổng. Đầu ra của mô hình là tổng các đầu ra có trọng số của top-k FFN (ký hiệu bởi π). Vì các FFN này hoạt động độc lập và có thể đặt trên các thiết bị tính toán khác nhau, mô hình có thể dễ dàng mở rộng quy mô khi M lớn hơn. (Nhãn trong hình: *FFN sub-layer* = tầng con FFN; *Add & LayerNorm* = cộng và chuẩn hóa tầng; *Feed-Forward Network* = mạng truyền thẳng; *Gating Model* = mô hình cổng.)

Lưu ý rằng, từ góc nhìn học máy, MoE là một cách tiếp cận tổng quát để kết hợp các mạng nơ-ron khác nhau, mỗi mạng được phát triển để giải quyết một khía cạnh khác nhau của bài toán [Yuksel và cộng sự, 2012; Masoudnia và Ebrahimpour, 2014]. Ứng dụng ở đây chỉ là một trường hợp đặc biệt của khuôn khổ tổng quát của MoE. Cách tiếp cận này cũng thường được dùng để cải thiện hiệu năng tổng

thể của các bộ dự đoán, điều có thể được bàn trong lĩnh vực học tập hợp (*ensemble learning*) [Zhou, 2012a].

Một khó khăn khác khi phát triển các mô hình Transformer khổng lồ là sự bất ổn định khi huấn luyện (*training instability*). Cũng như nhiều mạng nơ-ron đặc biệt lớn khác, việc tối ưu trực tiếp một mô hình Transformer có hàng tỉ tham số có thể khiến mô hình bị mắc kẹt ở các cực tiểu địa phương (*local minima*), và đôi khi gây ra những đột biến (*spike*) lớn của hàm mất mát trong quá trình huấn luyện [Lepikhin và cộng sự, 2021; Fedus và cộng sự, 2022b; Chowdhery và cộng sự, 2022]. Ngay cả khi tinh chỉnh cẩn thận siêu tham số, chiến lược huấn luyện và giá trị tham số ban đầu, vẫn thường gặp những tình huống phải khởi động lại việc huấn luyện từ một điểm lưu (*checkpoint*) trước đó để thoát khỏi các vùng tối ưu bất ổn định. Một trong những lý do của khó khăn huấn luyện này là các cài đặt chuẩn của các phép toán đại số tuyến tính, như nhân ma trận, có thể trở nên bất ổn định số (*numerically unstable*) khi thao tác trên các vectơ và ma trận cực lớn. Do đó, điều cốt yếu là cải thiện độ ổn định huấn luyện bằng cách dùng các phương pháp vững về mặt số học và các kỹ thuật độ chính xác hỗn hợp (*mixed precision*).

Chú giải của người dịch: Bất ổn định số và độ chính xác hỗn hợp

Máy tính chỉ lưu số thực với một số chữ số hữu hạn. Ví dụ, với định dạng 16 bit (FP16), số lớn nhất biểu diễn được chỉ khoảng 65 504; cộng dồn hàng triệu tích trong một phép nhân ma trận lớn có thể vượt ngưỡng này (tràn số) hoặc làm mất các chữ số nhỏ (sai số làm tròn). *Độ chính xác hỗn hợp* nghĩa là phần lớn phép tính dùng định dạng ít bit (nhanh, tiết kiệm bộ nhớ), còn những chỗ nhạy cảm (như cộng dồn, cập nhật tham số) dùng định dạng nhiều bit hơn (FP32) để giữ độ chính xác.

6.4 Các mô hình hiệu quả

Hiệu quả là một yếu tố quan trọng cần cân nhắc trong nhiều ứng dụng thực tế của mô hình Transformer. Ví dụ, ta có thể muốn chạy và/hoặc huấn luyện một mô hình Transformer trong giới hạn về bộ nhớ và thời gian. Hiệu quả không phải là một vấn đề đơn lẻ mà bao gồm cả một loạt bài toán. Dù các bài toán này có thể được phân loại theo nhiều cách khác nhau, có hai khía cạnh cơ bản mà ta có thể xem xét trong một bài toán về hiệu quả.

- **Hiệu quả về thời gian và không gian.** Với một bài toán cho trước, ta muốn mô hình nhỏ và nhanh, trong khi vẫn giải bài toán chính xác nhất có thể. Ví dụ, trong một số ứng dụng dịch máy, ta có thể xây dựng một mô hình với số lượng tham số nhỏ để vừa với bộ nhớ hạn chế, và có thể phát triển một thuật toán tìm kiếm nhanh để đạt được việc dịch với độ trễ (*latency*) thấp. Một khó khăn thực tế ở đây là việc cải thiện hiệu quả thường dẫn đến dự đoán kém hơn. Trong nhiều trường hợp, ta cần tìm một sự đánh đổi giữa hiệu quả và độ chính xác.
- **Khả năng mở rộng (*scalability*).** Khi quy mô bài toán tăng lên, ta mong muốn chi phí tính toán phát sinh thêm là nhỏ nhất. Ví dụ, việc huấn luyện một mạng nơ-ron được gọi là hiệu quả nếu thời gian cần để tối ưu nó vẫn ngắn ở mức hợp lý khi khối lượng dữ liệu huấn luyện tăng lên. Một thước đo hiệu quả khác là lượng tài nguyên tiêu tốn khi xử lý những đầu vào lớn hơn. Chẳng hạn, một hệ thống dịch máy là kém hiệu quả trong việc dịch các câu dài nếu lượng bộ nhớ chiếm dụng (*memory footprint*) và độ trễ tăng theo hàm mũ với số từ đầu vào.

Trong mục này, chúng ta sẽ không bàn về mọi vấn đề liên quan đến hiệu quả, vốn là một chủ đề rất rộng. Thay vào đó, ta xem xét các phương pháp hiệu quả được dùng rộng rãi cho việc mô hình hóa và sinh chuỗi dựa trên Transformer; một số trong đó là những cải tiến về kiến trúc mô hình, còn một số là

các phương pháp không phụ thuộc kiến trúc và cũng có thể dùng trong các hệ thống khác. Phần lớn nội dung ở đây tập trung vào việc phát triển các mô hình Transformer gọn nhẹ và nhanh, tương đối vững vàng trước các chuỗi đầu vào và đầu ra dài.

Nói chung, cùng một phương pháp tối ưu có thể được áp dụng cho các mô-đun khác nhau của một hệ thống dựa trên Transformer. Để đơn giản hóa thảo luận, trong mục này chúng ta chủ yếu xét các tầng con tự chú ý và các tầng con FFN. Tuy vậy, thảo luận của chúng ta mang tính tổng quát và các phương pháp được trình bày ở đây có thể áp dụng cho các phần khác của một mô hình Transformer, chẳng hạn các tầng con chú ý chéo.

Chú giải của người dịch: Vì sao "hiệu quả" lại quan trọng đến vậy?

Một mô hình ngôn ngữ lớn có thể có hàng chục tỷ tham số. Nếu mỗi tham số lưu bằng 2 byte, thì riêng việc chứa 10 tỷ tham số đã cần khoảng 20 GB bộ nhớ, chưa kể bộ nhớ cho các phép tính trung gian. Vì vậy, các kỹ thuật làm mô hình "nhẹ hơn" (tốn ít bộ nhớ) và "nhanh hơn" (ít phép tính hơn) quyết định việc mô hình có chạy được trên một máy cụ thể hay không, và người dùng phải chờ bao lâu để nhận câu trả lời.

6.4.1 Chú ý thưa

Trên thực tế, các cơ chế chú ý dùng trong Transformer rất tốn thời gian, đặc biệt khi chuỗi đầu vào dài. Để minh họa, xét một bộ giải mã Transformer dự đoán một phân phối trên bộ từ vựng tại mỗi bước thời gian khi đã biết các từ đứng trước. Giả sử chuỗi do bộ giải mã sinh ra có độ dài n và đầu vào của một tầng con tự chú ý là một ma trận $n \times d$ là $\mathbf{S}$. Trước hết, $\mathbf{S}$ được biến đổi tuyến tính để thu được các truy vấn $\mathbf{S}^q \in \mathbb{R}^{n\times d}$, các khóa $\mathbf{S}^k \in \mathbb{R}^{n\times d}$ và các giá trị $\mathbf{S}^v \in \mathbb{R}^{n\times d}$. Để đơn giản ký hiệu trong tiểu mục này, ta dùng $\mathbf{Q}$, $\mathbf{K}$ và $\mathbf{V}$ để lần lượt biểu diễn $\mathbf{S}^q$, $\mathbf{S}^k$ và $\mathbf{S}^v$.

Khi đó, đầu ra của tầng con tự chú ý có thể được tính bằng

$$\text{Att}_{\text{self}}(\mathbf{S}) \;=\; \mathbf{AV}, \tag{6.118}$$

trong đó $\mathbf{A}$ là một ma trận chú ý (*attention matrix*) kích thước $n \times n$, hay bản đồ chú ý (*attention map*)

$$\mathbf{A} \;=\; \text{Softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d}} + \mathbf{M}\right). \tag{6.119}$$

Ở đây, $\mathbf{M}$ là một ma trận mặt nạ dùng để ngăn mô hình chú ý tới ngữ cảnh tương lai tại mỗi vị trí. Cụ thể, với vị trí i, $M(i,j) = 0$ nếu $j \leq i$, và $M(i,j) = -\infty$ trong trường hợp ngược lại. Cả độ phức tạp thời gian lẫn độ phức tạp không gian của tầng con tự chú ý đều tăng theo bình phương của n[11]. Do đó, cơ chế tự chú ý chuẩn trở nên quá tốn kém về tính toán khi n lớn.

Chú giải của người dịch: Vì sao chi phí tăng theo n^2?

Ma trận $\mathbf{A}$ có n hàng và n cột: mỗi phần tử $A(i,j)$ cho biết vị trí i "chú ý" tới vị trí j bao nhiêu. Với chuỗi 1 000 token, ta phải tính và lưu $1\,000 \times 1\,000 = 10^6$ trọng số; với chuỗi 100 000 token thì con số này là 10^{10} — gấp 10 000 lần, dù chuỗi chỉ dài gấp 100 lần. Đây chính là ý nghĩa của "tăng theo bình phương". Ma trận mặt nạ $\mathbf{M}$ cộng $-\infty$ vào các ô $j > i$; sau Softmax, $e^{-\infty} = 0$, nên các vị trí tương lai nhận trọng số bằng 0.

Cách cài đặt chuẩn của mô hình trên dựa vào các phép tính trên ma trận dày đặc, chẳng hạn các phép nhân trong Phương trình (6.118) và (6.119). Một cách tiếp cận phổ biến để giảm cả lượng bộ nhớ chiếm dụng lẫn số phép toán dấu phẩy động là làm thưa (*sparsification*). Bằng cách giả sử $\mathbf{A}$ là một ma

[11] Chính xác hơn, lượng bộ nhớ chiếm dụng là $n^2 + n \cdot d$, trong đó số hạng n^2 chiếm ưu thế khi $n \gg d$.

trận thưa (*sparse matrix*), chỉ một phần $\varrho \cdot n^2$ trong số các phần tử của nó còn khác 0, trong đó ϱ là **tỷ lệ thưa** (*sparsity ratio*). Việc dùng các biểu diễn ma trận thưa giảm mạnh yêu cầu về bộ nhớ. Hơn nữa, việc tính $\frac{\mathbf{QK}^\top}{\sqrt{d}}$ và $\mathbf{AV}$ trở nên hiệu quả hơn, vì mô hình chỉ xử lý một tập con nhỏ các vị trí liên quan.

Cho một vị trí i, ta định nghĩa **trường chú ý** (*attention field*) π_i là tập các vị trí được xem xét khi tính biểu diễn tại vị trí này. Do đó, ta chỉ cần tính chú ý tích vô hướng giữa vị trí i đã cho và mỗi vị trí $j \in \pi_i$. Điều này dẫn đến một ma trận chú ý thưa $\mathbf{A}'$, trong đó

$$A'(i,j) \;=\; \begin{cases} a_{i,j} & j \in \pi_i \text{ và } j \leq i \\ 0 & \text{trường hợp khác} \end{cases}, \tag{6.120}$$

với $a_{i,j}$ là một trọng số khác 0. Một cách cài đặt đơn giản cho mô hình này chỉ cần sửa đổi nhẹ $\mathbf{M}$, dẫn đến một biến mặt nạ mới $\mathbf{M}'$

$$M'(i,j) \;=\; \begin{cases} 0 & j \in \pi_i \text{ và } j \leq i \\ -\infty & \text{trường hợp khác} \end{cases}. \tag{6.121}$$

Trong các cài đặt thực tế, một cách hiệu quả hơn là dùng các phép toán thưa cho $\mathbf{QK}^\top$ và $\mathbf{A}'\mathbf{V}$ bằng cách lần lượt xét $\mathbf{M}'$ và $\mathbf{A}'$. Nghĩa là, ta chỉ tính cho những cặp có trọng số chú ý khác 0 và bỏ qua phần còn lại.

Chú giải của người dịch: Ví dụ nhỏ về trường chú ý

Giả sử $n = 5$ và ta cho mỗi vị trí chỉ được nhìn chính nó và một vị trí liền trước: $\pi_i = \{i-1, i\}$. Khi đó, ở hàng $i = 4$ của $\mathbf{M}'$, chỉ hai ô $j = 3$ và $j = 4$ bằng 0, ba ô còn lại bằng $-\infty$. Mỗi hàng của $\mathbf{A}'$ chỉ có tối đa 2 số khác 0 thay vì tối đa 5 — tổng cộng 9 ô khác 0 thay vì 15 ô của ma trận chú ý "tam giác dưới" đầy đủ. Với n rất lớn, nếu mỗi hàng chỉ giữ một số cố định w ô, tổng số ô cần tính chỉ khoảng $w \cdot n$ thay vì cỡ n^2.

Có thể xét một số cách tiếp cận cho tự chú ý thưa.

- **Chú ý dựa trên khoảng (*span-based attention*)/Chú ý cục bộ (*local attention*).** Như đã bàn ở Mục 6.3.1, trong nhiều trường hợp việc sử dụng ngữ cảnh trong mô hình hóa chuỗi mang tính cục bộ. Ý tưởng cơ bản của chú ý cục bộ là giới hạn sự chú ý trong một vùng cục bộ của chuỗi đầu vào. Khi đó ta có thể viết π_i là

  $$\pi_i \;=\; [a_i^l, a_i^r], \tag{6.122}$$

  trong đó a_i^l và a_i^r là đầu trái và đầu phải của π_i. Đại lượng $a_i^r - a_i^l + 1$ xác định kích thước của vùng, nên ta có thể dùng nó để điều khiển độ thưa của mô hình chú ý; chẳng hạn, nếu $a_i^r - a_i^l + 1 \ll n$, mô hình sẽ rất thưa. a_i^l và a_i^r có thể thu được bằng các phương pháp heuristic hoặc các phương pháp học máy. Bạn đọc có thể tham khảo các bài báo liên quan để biết thêm chi tiết [Luong và cộng sự, 2015; Sperber và cộng sự, 2018; Yang và cộng sự, 2018a; Sukhbaatar và cộng sự, 2019]. Xem Hình 6.13 (b) để thấy minh họa về chú ý cục bộ.

- **Chú ý theo khối (*chunked attention*).** Trong phương pháp này, ta chia một chuỗi thành các khối và chạy mô hình chú ý trên từng khối [Parmar và cộng sự, 2018; Qiu và cộng sự, 2020a]. Cho một chuỗi $\{1, \cdots, n\}$, ta định nghĩa $\{\text{chunk}_1, \cdots, \text{chunk}_q\}$ là một cách phân đoạn chuỗi. Một khối có thể được biểu diễn dưới dạng một khoảng

  $$\text{chunk}_k \;=\; [c_k^l, c_k^r]. \tag{6.123}$$

  Ở bước chú ý, ta coi mỗi khối là một chuỗi và thực hiện tự chú ý trên nó như thường lệ. Nói cách khác, biểu diễn tại vị trí i được tính chỉ bằng ngữ cảnh trong khối mà i thuộc về. Theo nghĩa này,

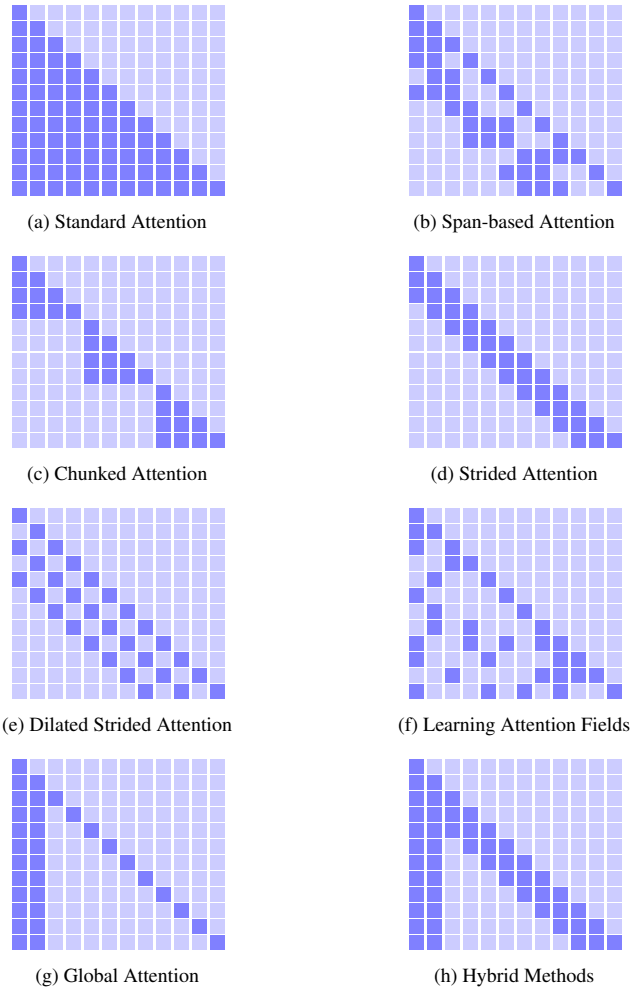
mô hình này có thể được xem như một biến thể của chú ý cục bộ. Hình 6.13 (c) minh họa mô hình này. Một vấn đề còn lại là phân đoạn chuỗi như thế nào. Có một vài cách để làm điều này. Ví dụ, như đã bàn ở Mục 6.2.4, ta có thể phân đoạn theo góc nhìn ngôn ngữ học, chia chuỗi thành các đơn vị có cơ sở ngôn ngữ học. Trong các hệ thống thực tế, đôi khi thuận tiện hơn nếu chia chuỗi thành các khối có độ dài bằng nhau. Như vậy, độ thưa của mô hình được điều khiển bởi kích thước các khối; chẳng hạn, dùng các khối nhỏ hơn sẽ cho một mô hình chú ý thưa hơn.

- **Chú ý theo bước trượt (*strided attention*).** Vì phương pháp chú ý theo khối áp đặt một cách phân đoạn cứng lên chuỗi đầu vào, nó có thể mất khả năng học biểu diễn từ các đầu vào ở những khối khác nhau. Một cách khác để đạt được chú ý theo khối là cho phép các khối chồng lấn nhau [Child và cộng sự, 2019; Beltagy và cộng sự, 2020; Ainslie và cộng sự, 2020]. Cách tiếp cận này tương tự họ các phương pháp thường được dùng để áp dụng một mô hình cục bộ lên dữ liệu 1D hoặc 2D nhằm sinh ra đầu ra có cùng hình dạng. Giống như CNN, ta dùng một cửa sổ ngữ cảnh để biểu diễn trường đầu vào của mô hình chú ý. Cửa sổ ngữ cảnh trượt dọc theo chuỗi, mỗi lần tiến lên một bước có kích thước $stride$ (*bước trượt*). Trong trường hợp đặc biệt, nếu $stride$ bằng kích thước cửa sổ ngữ cảnh, mô hình này quy về mô hình chú ý theo khối đã nói ở trên. Nếu $stride$ nhỏ hơn kích thước cửa sổ ngữ cảnh, mô hình chú ý sẽ trở nên dày đặc hơn. Hình 6.13 (d) cho thấy trường hợp $stride = 1$, khi đó sự chồng lấn giữa các khối là lớn nhất. Một cách để đạt được chú ý tương đối thưa hơn là dùng một cửa sổ ngữ cảnh giãn (*dilated context window*). Hình 6.13 (e) cho thấy một ví dụ về mô hình chú ý theo bước trượt có giãn, trong đó cửa sổ ngữ cảnh không liên tục, với các khoảng trống có kích thước 1.

- **Học trường chú ý.** Vì trường chú ý π_i có thể là bất kỳ tập con nào của $\{1, \cdots, n\}$, ta có thể phát triển các mô hình chú ý thưa tổng quát hơn bằng cách xét các bản đồ chú ý vượt ra ngoài các mẫu dựa trên khối. Câu hỏi duy nhất là làm sao xác định mô hình sẽ chú ý tới những vị trí nào đối với một vị trí cho trước. Một cách đơn giản là dùng một mô hình rẻ hơn về tính toán để ước lượng "mức độ quan trọng" của mỗi vị trí. Sau đó, trọng số chú ý chỉ được tính cho một số vị trí được cho là quan trọng nhất [Zhou và cộng sự, 2021]. Cách thứ hai là gom nhóm: các vị trí được chia thành nhóm, rồi trọng số chú ý chỉ được tính cho các vị trí trong cùng một nhóm. Điều này có thể thực hiện bằng cách phân cụm các khóa và truy vấn. Chẳng hạn, ta có thể phân cụm khóa và truy vấn bằng phân cụm k-means (k-*means clustering*). Các tâm cụm (*centroid*) có thể được coi là các tham số bổ sung của mô hình chú ý, và do đó có thể được học trong quá trình tối ưu [Roy và cộng sự, 2021]. Một lợi ích của việc học trường chú ý là mô hình có thể trải rộng sự chú ý của nó trên chuỗi. Đây là một tính chất hữu ích cho nhiều bài toán NLP vì sự phụ thuộc giữa các từ đôi khi có tầm xa, không bị giới hạn trong một cửa sổ ngữ cảnh cục bộ. Xem Hình 6.13 (f) để thấy ví dụ về bản đồ chú ý học được qua mô hình này. Các cách khác để học cách chú ý là dùng các hàm sắp xếp hoặc hàm băm để gom các vectơ khóa và truy vấn tương tự nhau [Kitaev và cộng sự, 2020; Tay và cộng sự, 2020a]. Các hàm này có thể là những hàm được thiết kế theo heuristic hoặc các mạng nơ-ron với tham số học được. Bằng cách dùng các hàm này, ta có thể sắp xếp lại chuỗi sao cho các đầu vào trong cùng một nhóm nằm cạnh nhau trong chuỗi đã sắp xếp lại. Theo cách này, bản đồ chú ý thu được có dạng theo khối, và mô hình hiệu quả về mặt tính toán nhờ sử dụng cách tiếp cận chú ý theo khối.

- **Các phương pháp lai.** Ở trên, chúng ta đã bàn về một loạt mô hình chú ý thưa khác nhau. Việc tìm hiểu các phương pháp kết hợp nhiều mô hình với nhau để tận dụng ưu điểm của chúng theo cách nào đó là điều tự nhiên. Một cách đơn giản để làm điều này là kết hợp các trường chú ý của các mô hình khác nhau. Ví dụ, trong hệ thống của Zaheer và cộng sự [2020], bản đồ chú ý được sinh ra bằng cách xét ba mô hình thưa khác nhau, gồm chú ý cục bộ (chú ý theo khối), chú ý toàn

cục (*global attention*) và chú ý ngẫu nhiên (*random attention*)[12]. Mô hình thu được vẫn là một mô hình thưa, nhưng có phần vững vàng hơn vì nó bao gồm nhiều mẫu từ các góc nhìn khác nhau của việc mô hình hóa chú ý. Một cách khác để kết hợp nhiều mô hình chú ý là dùng các mô hình khác nhau cho các đầu khác nhau trong chú ý đa đầu [Child và cộng sự, 2019; Beltagy và cộng sự, 2020]. Ví dụ, có thể dùng một đầu làm mô hình chú ý cục bộ, và dùng một đầu khác làm mô hình chú ý toàn cục (xem Hình 6.13 (g–h)).



(a) Standard Attention (b) Span-based Attention

(c) Chunked Attention (d) Strided Attention

(e) Dilated Strided Attention (f) Learning Attention Fields

(g) Global Attention (h) Hybrid Methods

Hình 6.13: Minh họa bản đồ chú ý của các mô hình khác nhau (tự chú ý ở phía bộ giải mã). Ô sẫm màu nghĩa là $A'(i,j) \neq 0$ (tức là i chú ý tới j), và ô sáng màu nghĩa là $A'(i,j) = 0$ (tức là i không chú ý tới j). Trong mọi bản đồ chú ý này, ta giả sử mặc định mỗi vị trí đều chú ý tới chính nó (xem các đường chéo). Các hình con: (a) chú ý chuẩn; (b) chú ý dựa trên khoảng; (c) chú ý theo khối; (d) chú ý theo bước trượt; (e) chú ý theo bước trượt có giãn; (f) học trường chú ý; (g) chú ý toàn cục; (h) các phương pháp lai.

[12] Ở đây, mô hình chú ý toàn cục cho mỗi từ chỉ chú ý tới một từ đặc biệt đại diện cho toàn bộ chuỗi, thường được đặt ở đầu chuỗi. Mô hình chú ý ngẫu nhiên cho mỗi từ chú ý tới một tập ngẫu nhiên các từ trong chuỗi.

Chú giải của người dịch: Đọc Hình 6.13 thế nào?

Mỗi bản đồ là một bảng ô vuông: hàng thứ i ứng với vị trí đang được tính biểu diễn, cột thứ j ứng với vị trí được "nhìn tới". Ô sẫm ở hàng i, cột j nghĩa là vị trí i có dùng thông tin của vị trí j. Ở (a), cả tam giác dưới đều sẫm: mỗi vị trí nhìn mọi vị trí đứng trước. Ở (c), các ô sẫm tạo thành những khối vuông nhỏ dọc đường chéo: mỗi vị trí chỉ nhìn trong khối của mình. Ở (d), một "dải" hẹp chạy dọc đường chéo: mỗi vị trí nhìn vài vị trí ngay trước nó. Ở (g), cột đầu tiên sẫm: mọi vị trí đều nhìn tới từ đặc biệt ở đầu chuỗi. Càng ít ô sẫm, mô hình càng thưa và càng tiết kiệm.

Một nhược điểm của các mô hình thưa so với các mô hình dày đặc là chúng thường kém hiệu quả hơn trên GPU/CPU do các ràng buộc phần cứng. Dù về lý tưởng các mô hình thưa có thể giảm cả yêu cầu bộ nhớ lẫn yêu cầu tính toán, thông lượng (*throughput*) tính toán của các mô hình thưa chậm hơn nhiều so với các mô hình dày đặc. Trên thực tế, các mô hình thưa khó tiến gần tới FLOPS đỉnh của một GPU hay CPU[13]. Do đó, chúng chủ yếu được dùng để cải thiện hiệu quả bộ nhớ hơn là để tăng tốc độ tính toán thuần túy trên phần cứng hiện nay. Mặt khác, các mô hình thưa vẫn rất hữu ích cho những người làm NLP trong bối cảnh Transformer tiết kiệm bộ nhớ, đặc biệt khi Transformer được dùng để xử lý các chuỗi cực dài.

Chú giải của người dịch: Vì sao "thưa" chưa chắc đã nhanh?

GPU được thiết kế để làm cực nhanh các phép nhân trên những khối số liền mạch, đều đặn (như cả một ma trận đầy). Với ma trận thưa, máy phải "nhảy cóc" tới các ô khác 0 nằm rải rác, đọc thêm thông tin về vị trí của chúng, nên phần cứng không chạy hết công suất. Hình dung như việc chở hàng: chở một xe tải đầy một lần thường nhanh hơn đi giao lẻ từng gói nhỏ tới nhiều địa chỉ, dù tổng số hàng ít hơn.

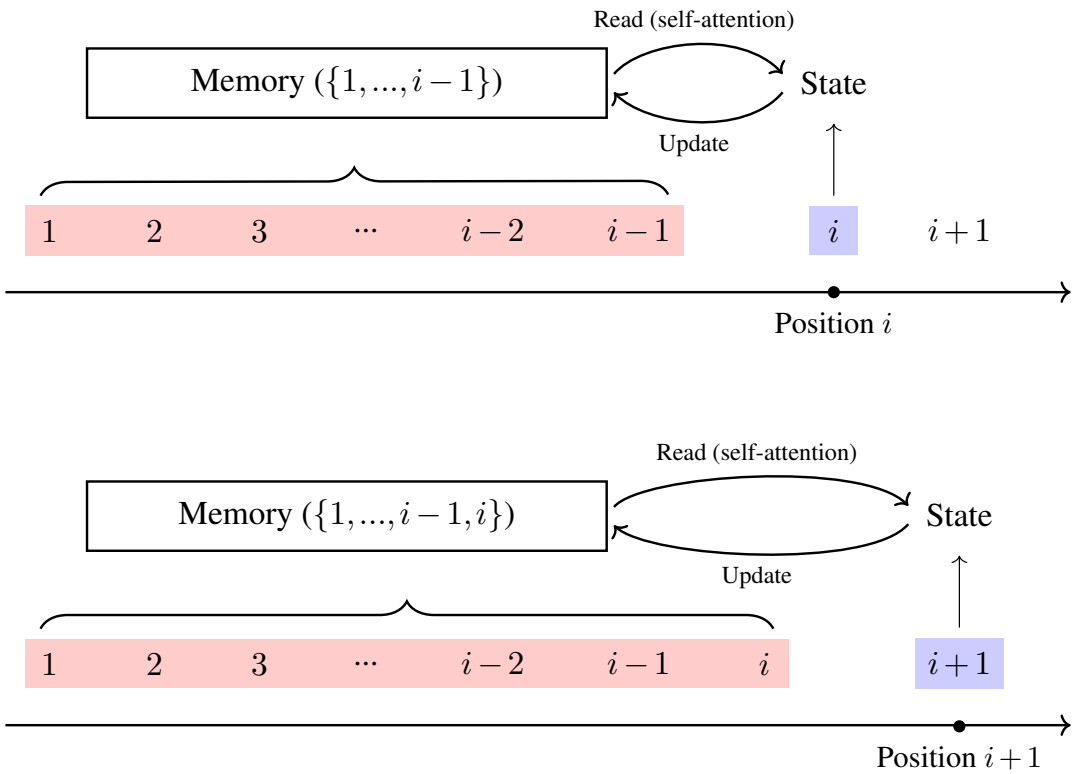
6.4.2 Mô hình hồi quy và mô hình bộ nhớ

Đối với các bài toán sinh chuỗi, Transformer cũng có thể được xem như các hệ thống bộ nhớ. Xét lại thiết lập tự hồi quy tổng quát, trong đó ta được cho các trạng thái của $i-1$ vị trí trước đó và muốn dự đoán trạng thái tiếp theo. Trong tự chú ý, điều này được thực hiện bằng cách dùng truy vấn tại vị trí i (tức là $\mathbf{q}_i$) để truy cập các cặp khóa–giá trị của các vị trí trước đó (tức là $\{(\mathbf{k}_1, \mathbf{v}_1), \cdots, (\mathbf{k}_{i-1}, \mathbf{v}_{i-1})\}$). Sau đó, ta chuyển sang vị trí $i+1$ và nối thêm $(\mathbf{k}_i, \mathbf{v}_i)$ vào tập các cặp khóa–giá trị. Quy trình này có thể được diễn giải một cách trực quan bằng các cơ chế bộ nhớ (xem Chương 4). Một mô hình Transformer duy trì một kho bộ nhớ (*memory bank*) lưu giữ thông tin trong quá khứ. Khi mô hình xử lý chuỗi, nó lặp lại cùng một thao tác: sinh đầu ra bằng cách đọc từ bộ nhớ, rồi cập nhật bộ nhớ để đưa thông tin mới vào. Điều này được minh họa trong Hình 6.14.

Chú giải của người dịch: Bộ nhớ ở đây chính là KV cache

Khi sinh từng token, mô hình cần khóa và giá trị của mọi token đã sinh. Thay vì tính lại chúng ở mỗi bước, ta lưu sẵn trong một "kho" — chính là bộ nhớ đệm KV (KV cache). Mỗi bước: (1) *đọc*: truy vấn mới so khớp với mọi khóa trong kho để lấy thông tin; (2) *ghi*: thêm cặp khóa–giá trị mới vào kho. Vấn đề là kho này cứ lớn dần theo độ dài chuỗi, nên các phương pháp dưới đây tìm cách giữ nó nhỏ hoặc truy cập nó nhanh hơn.

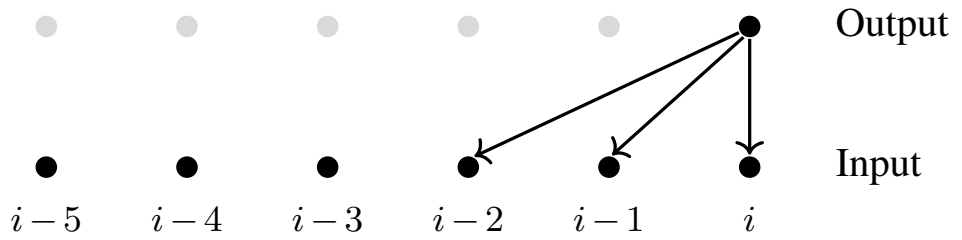
[13] FLOPS = số phép toán dấu phẩy động mỗi giây (*floating point operations per second*).



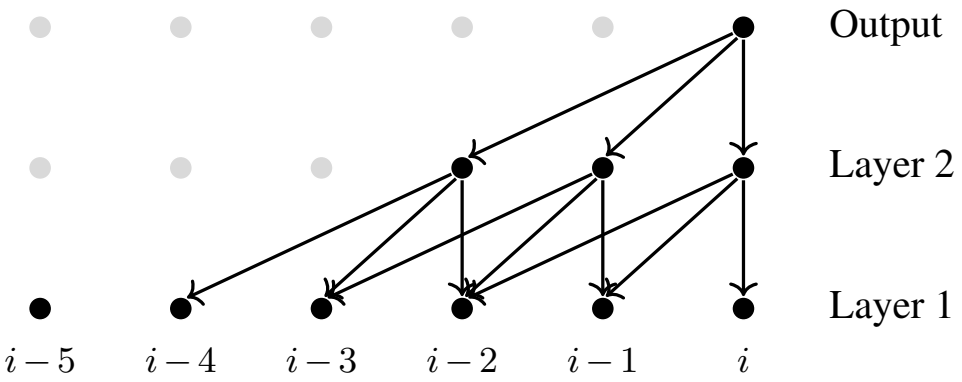
Hình 6.14: Transformer như một hệ thống bộ nhớ. Tại vị trí i, tập các cặp khóa–giá trị từ các vị trí $\{1, \cdots, i-1\}$ đóng vai trò bộ nhớ về ngữ cảnh quá khứ. Mô hình truy cập bộ nhớ này để sinh ra một đầu ra, rồi sau đó thêm cặp khóa–giá trị của vị trí i vào bộ nhớ. Chuyển sang vị trí tiếp theo, chu trình truy cập và cập nhật bộ nhớ này được lặp lại. (Các nhãn trong hình: *Memory* = bộ nhớ; *State* = trạng thái; *Read (self-attention)* = đọc (bằng tự chú ý); *Update* = cập nhật; *Position* = vị trí.)

1. Bộ nhớ dựa trên bộ đệm

Bộ nhớ ở đây có thể được xem như một kho lưu trữ các vectơ. Từ góc nhìn học máy, đây là một mô hình phi tham số (*non-parametric model*), và chi phí truy cập mô hình tăng lên khi chuỗi dài ra. Rõ ràng, một bộ nhớ có độ dài thay đổi như vậy nói chung sẽ không khả thi nếu mô hình phải xử lý các chuỗi cực dài. Để mô hình hóa các chuỗi có độ dài tùy ý, người ta thường dùng một bộ nhớ có độ dài cố định. Như trong nhiều bài toán NLP, một trong những cách đơn giản nhất để làm điều này là duy trì một bộ đệm (*cache*) chứa thông tin gần đây, tức là ta giới hạn việc mô hình hóa trong một cửa sổ ngữ cảnh. Gọi n_c là kích thước cửa sổ ngữ cảnh. Mô hình theo dõi $n_c - 1$ trạng thái gần nhất đứng trước vị trí hiện tại, để có thể xét các vị trí đứng trước gần nhất ở mỗi bước. Điều này có nghĩa là, với mỗi vị trí, một tầng con tự chú ý chú ý tới $n_c - 1$ vị trí trước đó, như minh họa trong Hình 6.15 (a).



(a) Cache-based memory with a single self-attention layer.



(b) Expanded context window with multiple self-attention layers.

Hình 6.15: Minh họa các cơ chế bộ nhớ dựa trên bộ đệm trong mô hình Transformer. (a) Bộ nhớ dựa trên bộ đệm với một tầng tự chú ý duy nhất. (b) Cửa sổ ngữ cảnh được mở rộng nhờ nhiều tầng tự chú ý. (*Input* = đầu vào; *Output* = đầu ra; *Layer* = tầng.)

Nếu xếp chồng nhiều tầng con tự chú ý, một cửa sổ ngữ cảnh lớn hơn sẽ được xét đến. Chẳng hạn, một mô hình gồm hai tầng con tự chú ý có cửa sổ ngữ cảnh kích thước $2n_c - 1$, như minh họa trong Hình 6.15 (b).

Vì vậy, ta có thể nắm bắt một ngữ cảnh đủ lớn bằng cách dùng một mô hình Transformer nhiều tầng. Lưu ý rằng mô hình cửa sổ ngữ cảnh ở đây về bản chất giống mô hình chú ý theo bước trượt đã trình bày ở tiểu mục trước. Các hệ thống loại này thường dễ cài đặt: ta trượt một cửa sổ dọc theo chuỗi,

và ở mỗi lần di chuyển, ta đưa ra dự đoán tại vị trí cuối cùng của cửa sổ (khi suy luận), hoặc lan truyền ngược sai số (khi huấn luyện).

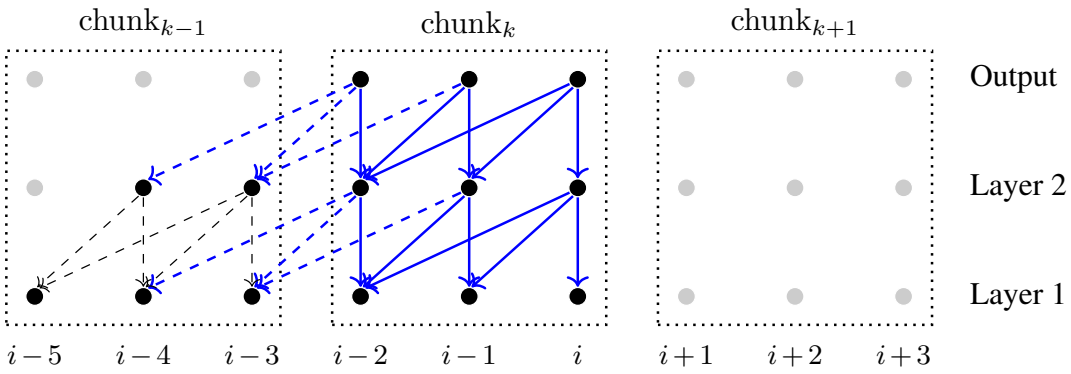
Chú giải của người dịch: Vì sao xếp chồng tầng lại mở rộng ngữ cảnh?

Lấy $n_c = 3$: ở tầng 1, vị trí i chỉ nhìn được $i-2, i-1$ (và chính nó). Nhưng biểu diễn tầng 1 của $i-2$ lại đã "chứa" thông tin của $i-4, i-3$. Vì vậy ở tầng 2, khi i nhìn vào $i-2$ của tầng 1, nó gián tiếp nhận được thông tin từ tận $i-4$. Tổng cộng tầm nhìn là $i-4, \ldots, i$, tức $5 = 2 \cdot 3 - 1$ vị trí. Giống như "tin đồn" lan qua nhiều người: mỗi người chỉ nói với hàng xóm, nhưng qua vài lượt, tin đi rất xa. Với L tầng, tầm nhìn tăng lên khoảng $L(n_c - 1) + 1$ vị trí.

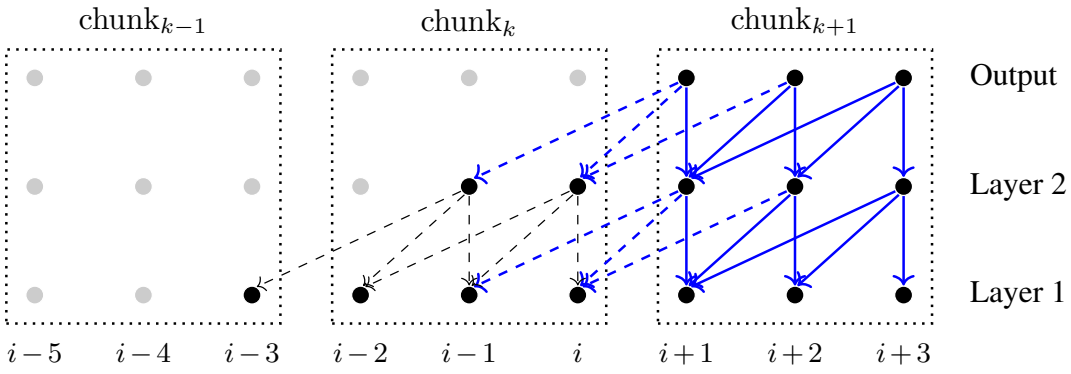
Một chiến lược khác để huấn luyện mô hình cửa sổ ngữ cảnh là chú ý theo khối. Chuỗi được chia một cách tường minh thành các chuỗi con (hay các khối) có độ dài cố định n_c. Các khối này sau đó được coi là những mẫu huấn luyện độc lập. Tuy nhiên, cách tiếp cận này hoàn toàn bỏ qua mối quan hệ giữa các đầu vào ở những khối khác nhau. Một cách để giải quyết vấn đề này là đưa vào sự phụ thuộc giữa các khối. Chẳng hạn, mô hình **Transformer-XL** cho phép mỗi khối truy cập một hoặc nhiều khối đứng trước nó [Dai và cộng sự, 2019]. Trong trường hợp đơn giản nhất, xét một ví dụ trong đó chunk_k có thể thấy khối liền trước nó là chunk_{k-1}. Mỗi vị trí trong chunk_k có thể chú ý tới mọi vị trí đứng trước nó trong cả chunk_k lẫn chunk_{k-1}.

Trong Transformer-XL, cách tiếp cận này được cài đặt dưới một dạng đơn giản hóa. Thứ nhất, mỗi vị trí bị ràng buộc chỉ được chú ý tới $n_c - 1$ vị trí trước đó, sao cho kích thước trường chú ý của một vị trí là như nhau trong giai đoạn huấn luyện và suy luận. Cách làm này đưa bài toán trở về chú ý theo bước trượt, khiến việc cài đặt mô hình chú ý trở nên đơn giản. Tuy nhiên, khác với chú ý theo bước trượt chuẩn, Transformer-XL thực hiện huấn luyện theo từng khối. Khi đã huấn luyện xong trên một khối, ta chuyển thẳng sang khối kế tiếp, thay vì trượt cửa sổ ngữ cảnh tiến lên một bước nhỏ. Thứ hai, dù cách tiếp cận này cho phép có kết nối giữa các khối, các tham số của mạng con ứng với chunk_{k-1} được giữ cố định, và ta chỉ cập nhật tham số của mạng con trên chunk_k ở bước thứ k. Xem Hình 6.16 để thấy minh họa.

Mô hình trên có tinh thần tương tự các mô hình hồi quy, vì cả hai đều yêu cầu phép tính ở một bước phụ thuộc vào trạng thái của các bước trước. Tuy nhiên, nó không ở dạng chuẩn của một mô hình hồi quy, trong đó đầu ra của một đơn vị hồi quy ở một bước là đầu vào ở bước tiếp theo. Thay vào đó, tính "hồi quy" được thể hiện thông qua các kết nối xuyên tầng và xuyên khối, tức là đầu ra của một tầng trong chunk_{k-1} được dùng làm đầu vào của một tầng cao hơn trong chunk_k.



(a) Step k of chunk-wise training



(b) Step $k+1$ of chunk-wise training

Hình 6.16: Minh họa việc huấn luyện theo khối trong Transformer-XL [Dai và cộng sự, 2019]. Chuỗi đầu vào được chia thành các khối không chồng lấn có độ dài n_c. Việc huấn luyện tiến hành lần lượt từng khối. Bên trong chunk$_k$, tầm chú ý của mọi token bị giới hạn trong một cửa sổ ngữ cảnh bên trái có kích thước n_c. Cấu trúc này cho phép chú ý xuyên khối. Chẳng hạn, vị trí $i-2$ trong chunk$_k$ có thể chú ý tới các vị trí $i-3$ và $i-4$ nằm trong chunk$_{k-1}$ (xem hình con (a)). Trong quá trình lan truyền ngược, gradient được tính cho mạng con hiện tại (chunk$_k$), còn các giá trị kích hoạt đã lưu đệm từ các khối trước được coi là các hằng số cố định. Các đường nét đứt biểu thị các luồng thông tin của lượt truyền xuôi không nhận gradient. Sau khi hoàn tất bước huấn luyện cho chunk$_k$, quá trình chuyển sang khối tiếp theo và lặp lại. ((a) Bước k và (b) bước $k+1$ của huấn luyện theo khối.)

2. Mã hóa bộ nhớ dài hạn

Một ý tưởng khác để biểu diễn các trạng thái của một chuỗi là đặt nhiệm vụ này dưới dạng một bài toán mã hóa. Thay vì lưu tất cả các vectơ khóa–giá trị trong quá trình sinh từ trái sang phải, ta nén bộ nhớ của toàn bộ "lịch sử" thành một số lượng cố định các vectơ khóa–giá trị đã mã hóa. Các vectơ đã mã hóa này có thể là một tập con nhỏ của chuỗi gốc $\{(\mathbf{k}_1, \mathbf{v}_1), \cdots, (\mathbf{k}_{i-1}, \mathbf{v}_{i-1})\}$, hoặc một tập nhỏ các vectơ mới được sinh ra để mã hóa toàn bộ lịch sử này.

Một cách để thực hiện việc mã hóa này là áp dụng một phép gộp (*pooling*) lên tập $\{(\mathbf{k}_1, \mathbf{v}_1), \cdots, (\mathbf{k}_{i-1}, \mathbf{v}_{i-1})\}$ [Rae và cộng sự, 2019a]. Ví dụ, dùng gộp trung bình (*average pooling*), bộ nhớ được rút gọn thành một cặp khóa–giá trị duy nhất $(\bar{\mathbf{k}}, \bar{\mathbf{v}})$:

$$\bar{\mathbf{k}} = \frac{1}{i-1} \sum_{j=1}^{i-1} \mathbf{k}_j, \tag{6.124}$$

$$\bar{\mathbf{v}} = \frac{1}{i-1} \sum_{j=1}^{i-1} \mathbf{v}_j. \tag{6.125}$$

Điều này dẫn tới một mô hình rất hiệu quả, vì ta chỉ cần cập nhật các vectơ tổng hợp $(\bar{\mathbf{k}}, \bar{\mathbf{v}})$ một cách tăng dần [Zhang và cộng sự, 2018a]. Gọi $(\bar{\mathbf{k}}[i], \bar{\mathbf{v}}[i])$ là trạng thái của bộ nhớ tại vị trí i. Một công thức tổng quát hơn được cho dưới dạng đệ quy:

$$\bar{\mathbf{k}}[i] = \text{KMem}(\bar{\mathbf{k}}[i-1], \mathbf{k}_{i-1}), \tag{6.126}$$

$$\bar{\mathbf{v}}[i] = \text{VMem}(\bar{\mathbf{v}}[i-1], \mathbf{v}_{i-1}), \tag{6.127}$$

trong đó $\text{KMem}(\cdot)$ và $\text{VMem}(\cdot)$ là các hàm cập nhật bộ nhớ bằng cách nhận cả trạng thái của bộ nhớ ở vị trí trước (tức $\bar{\mathbf{k}}[i-1]$ và $\bar{\mathbf{v}}[i-1]$) lẫn các trạng thái mới (tức $\mathbf{k}_{i-1}$ và $\mathbf{v}_{i-1}$). Có nhiều lựa chọn khả dĩ cho các hàm $\text{KMem}(\cdot)$ và $\text{VMem}(\cdot)$. Chẳng hạn, nếu $\text{KMem}(\cdot)$ và $\text{VMem}(\cdot)$ là các hàm tổng có trọng số, ta có thể suy ra cùng dạng như Phương trình (6.124) và (6.125). Nếu $\text{KMem}(\cdot)$ và $\text{VMem}(\cdot)$ là các ô hồi quy (ví dụ các ô RNN hoặc LSTM chuẩn), ta thu được một mô hình bộ nhớ hồi quy.

Chú giải của người dịch: Cập nhật trung bình "tăng dần"

Giống như tính điểm trung bình môn học: nếu đã biết trung bình của m bài kiểm tra là $\bar{x}_m$ và có thêm điểm mới x, ta không cần cộng lại từ đầu mà chỉ cần $\bar{x}_{m+1} = \frac{m\,\bar{x}_m + x}{m+1}$. Ví dụ: trung bình 3 bài là 7, bài thứ tư được 9, thì trung bình mới là $(3 \cdot 7 + 9)/4 = 7{,}5$. Phương trình (6.126) chính là ý tưởng này viết dưới dạng tổng quát: trạng thái mới chỉ phụ thuộc vào trạng thái cũ và dữ liệu mới. Nhờ vậy bộ nhớ luôn có kích thước cố định, dù chuỗi dài bao nhiêu.

Mở rộng khái niệm trên cho các bộ nhớ có nhiều hơn một cặp khóa–giá trị là điều đơn giản. Một cách là biểu diễn mức khối (*chunk-level representation*). Gọi $\{(\bar{\mathbf{k}}_1, \bar{\mathbf{v}}_1), \cdots, (\bar{\mathbf{k}}_\kappa, \bar{\mathbf{v}}_\kappa)\}$ là một bộ nhớ kích thước κ, trong đó mỗi $(\bar{\mathbf{k}}_j, \bar{\mathbf{v}}_j)$ là biểu diễn của một khối có độ dài n_c. Như vậy, bộ nhớ này có thể mã hóa một chuỗi có độ dài tối đa $\kappa \cdot n_c$, bằng cách tính mỗi $(\bar{\mathbf{k}}_j, \bar{\mathbf{v}}_j)$ theo Phương trình (6.126) và (6.127) trên khối tương ứng của nó. Cách thứ hai là tổ chức bộ nhớ thành một hàng đợi ưu tiên (*priority queue*). Một hàm chấm điểm được học, thường là một mạng nơ-ron phụ trợ, đánh giá và gán một điểm ưu tiên cho mỗi cặp khóa–giá trị được sinh ra. Các cặp có điểm cao được đưa vào hàng đợi thông qua một thao tác đẩy (*push*). Theo cách này, bộ nhớ chủ động duy trì một tập động chỉ gồm những cặp khóa–giá trị có giá trị nhất trên toàn chuỗi đầu vào.

Mặc dù biểu diễn bộ nhớ như một tập các vectơ là một lựa chọn thiết kế trực quan cho Transformer, dung lượng của nó bị giới hạn bởi số lượng vectơ. Một mô hình thay thế là **bộ nhớ liên tục** (*continuous*

memory). Cách tiếp cận này tận dụng xấp xỉ hàm (*function approximation*), coi các tập khóa $\{\mathbf{k}_1, \cdots, \mathbf{k}_{i-1}\}$ hoặc tập giá trị $\{\mathbf{v}_1, \cdots, \mathbf{v}_{i-1}\}$ như một dãy các điểm dữ liệu cần được khớp bởi một hàm liên tục. Thay vì lưu trữ tường minh các vectơ này, bộ nhớ được tham số hóa bởi chính các hàm đó. Một phương pháp phổ biến là dùng tổ hợp tuyến tính của các hàm cơ sở (*basis function*) đơn giản để xấp xỉ hàm phức tạp được xác định bởi các vectơ khóa hoặc giá trị rời rạc [Martins và cộng sự, 2022].

Chú giải của người dịch: Bộ nhớ liên tục: "vẽ đường cong thay vì lưu từng điểm"

Hãy tưởng tượng bạn có 1 000 điểm đo nhiệt độ trong ngày. Thay vì nhớ cả 1 000 con số, bạn tìm một hàm, chẳng hạn $f(t) = c_1 + c_2 \sin t + c_3 \cos t$, đi gần các điểm đó, rồi chỉ cần nhớ 3 hệ số c_1, c_2, c_3. Các hàm $1, \sin t, \cos t$ gọi là *hàm cơ sở*. Bộ nhớ liên tục làm điều tương tự với các vectơ khóa/giá trị: số hệ số cần lưu cố định, không phụ thuộc chuỗi dài bao nhiêu (đổi lại, thông tin bị xấp xỉ chứ không còn chính xác tuyệt đối).

Cũng dễ dàng dùng đồng thời một bộ nhớ ngắn hạn và một bộ nhớ dài hạn để kết hợp ưu điểm của cả hai. Chẳng hạn, ta dùng một bộ nhớ dựa trên bộ đệm để nắm bắt ngữ cảnh cục bộ, và dùng một bộ nhớ dài hạn hiệu quả mã hóa toàn bộ lịch sử để mô hình hóa các phụ thuộc tầm xa. Ý tưởng này cũng tương tự ý tưởng dùng để kết hợp các mô hình chú ý thưa khác nhau như đã bàn ở tiểu mục trước.

3. Các phương pháp dựa trên truy xuất

Cho tới giờ trong tiểu mục này, chúng ta đã bàn về các cách tiếp cận dựa trên mô hình có độ dài cố định. Cũng có thể phát triển các mô hình bộ nhớ hiệu quả bằng cách cải thiện hiệu quả truy cập bộ nhớ, thay vì giảm cứng dung lượng bộ nhớ. Một cách để làm điều này là lưu các cặp khóa–giá trị trong quá khứ vào một cơ sở dữ liệu (gọi là **cơ sở dữ liệu vectơ** (*vector database*)), và tìm các cặp khóa–giá trị giống nhất khi truy vấn cơ sở dữ liệu. Chính xác hơn, cho một truy vấn $\mathbf{q}$, ta dùng cơ sở dữ liệu để tìm một tập gồm p cặp khóa–giá trị liên quan nhất (ký hiệu Ω_p) bằng cách thực hiện tìm kiếm tương tự (*similarity search*) dựa trên tích vô hướng giữa truy vấn và các vectơ khóa. Sau đó truy vấn $\mathbf{q}$ chú ý tới Ω_p như trong các mô hình tự chú ý chuẩn. Ý tưởng đằng sau phương pháp này là chỉ xét một số ít phần tử đóng góp nhiều nhất vào kết quả chú ý. Do đó, về bản chất, mô hình là một mô hình chú ý thưa hiệu quả về tính toán. Một ưu điểm khác của phương pháp này là nó cho phép tìm kiếm tương tự nhanh trên một tập vectơ rất lớn nhờ các cài đặt được tối ưu hóa cao của cơ sở dữ liệu vectơ. Việc đặt bộ nhớ dưới dạng một hệ thống truy xuất thuộc về khuôn khổ rộng hơn của **sinh văn bản tăng cường truy xuất** (*retrieval-augmented generation*) (**RAG**) hay các cách tiếp cận tăng cường truy xuất. Nó cung cấp một khuôn khổ có khả năng mở rộng để tích hợp bộ nhớ ngoài vào các kiến trúc nơ-ron như Transformer [Guu và cộng sự, 2020; Lewis và cộng sự, 2020b; Wu và cộng sự, 2021].

Chú giải của người dịch: Truy xuất giống như tra mục lục

Thay vì đọc lại cả cuốn sách mỗi khi cần một thông tin, ta tra mục lục để mở đúng vài trang liên quan nhất. Ở đây, "mục lục" là cơ sở dữ liệu vectơ: nó nhanh chóng tìm ra p khóa có tích vô hướng lớn nhất với truy vấn (tức là "giống" truy vấn nhất), và mô hình chỉ chú ý tới p cặp đó. Vì sau Softmax, các khóa có tích vô hướng nhỏ chỉ nhận trọng số rất gần 0, bỏ qua chúng thường làm kết quả thay đổi không đáng kể.

6.4.3 Các mô hình số chiều thấp

Trong nhiều ứng dụng thực tế, Transformer là các mô hình "nhiều chiều". Điều này không chỉ vì dữ liệu đầu vào và/hoặc đầu ra nằm trong các không gian nhiều chiều, mà còn vì một số biểu diễn trung gian của dữ liệu trong mô hình có số chiều cao. Như đã bàn ở Mục 6.4.1, số chiều cao này một phần đến từ

việc tính tổng có trọng số của các vectơ giá trị, như trong Phương trình (6.128) (nhắc lại ở đây cho tiện theo dõi)

$$\text{Att}_{\text{self}}(\mathbf{S}) \;=\; \mathbf{AV} \tag{6.128}$$

và từ việc tính chính ma trận chú ý, như trong Phương trình (6.129)

$$\mathbf{A} \;=\; \text{Softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d}} + \mathbf{M}\right), \tag{6.129}$$

vốn bao gồm các phép nhân ma trận lớn $\mathbf{QK}^\top$ và $\mathbf{AV}$ khi độ dài n và số chiều ẩn d lớn.

Các phép toán $\mathbf{AV}$ và $\mathbf{QK}^\top$ có độ phức tạp thời gian $O(n^2 \cdot d)$ và độ phức tạp không gian $O(n^2 + n \cdot d)$. Trong khi các cách tiếp cận trước đã giảm bớt độ phức tạp này bằng các mô hình thưa, tiểu mục này tập trung vào các phương pháp xấp xỉ những phép toán này bằng tính toán dày đặc. Một ý tưởng đơn giản là chiếu $\mathbf{Q}$, $\mathbf{K}$ và $\mathbf{V}$ thành các ma trận nhỏ hơn, nhờ đó giảm gánh nặng tính toán của phép nhân ma trận. Vì $\mathbf{Q}$, $\mathbf{K}$ và $\mathbf{V}$ đều thuộc $\mathbb{R}^{n \times d}$, điều này có thể đạt được bằng cách giảm chiều độ dài n, giảm chiều ẩn d, hoặc giảm cả hai.

Chú giải của người dịch: Ký hiệu $O(\cdot)$ và đếm phép tính

$O(n^2 \cdot d)$ đọc là "bậc $n^2 d$": số phép tính tăng tỷ lệ với $n^2 d$ khi các đại lượng lớn dần (bỏ qua hằng số). Thật vậy, nhân ma trận $n \times d$ với ma trận $d \times n$ cho ra $n \cdot n$ phần tử, mỗi phần tử là một tích vô hướng của hai vectơ d chiều (khoảng d phép nhân), tổng cộng cỡ $n \cdot n \cdot d$ phép nhân. Với $n = 4\,096$, $d = 1\,024$ thì đó là khoảng $1{,}7 \times 10^{10}$ phép nhân chỉ cho một tầng, một đầu chú ý.

1. Giảm n

Lưu ý rằng đầu ra $\text{Att}_{\text{self}}(\mathbf{S})$ bắt buộc phải là ma trận $n \times d$, nên ta không thể giảm số lượng truy vấn. Thay vào đó, ta xét việc giảm số lượng khóa và giá trị. Giả sử n' là một số nguyên với $n' < n$, và $\mathbf{K}$, $\mathbf{V}$ có thể được biến đổi theo cách nào đó thành các ma trận $n' \times d$ là $\mathbf{K}'$ và $\mathbf{V}'$. Ta có thể thu được một mô hình "nhỏ hơn" đơn giản bằng cách thay $\mathbf{K}$ và $\mathbf{V}$ bởi $\mathbf{K}'$ và $\mathbf{V}'$, cho ta

$$\text{Att}_{\text{self}}(\mathbf{S}) = \mathbf{AV}' \tag{6.130}$$

$$\mathbf{A} = \text{Softmax}\left(\frac{\mathbf{Q}[\mathbf{K}']^\top}{\sqrt{d}} + \mathbf{M}\right). \tag{6.131}$$

Mô hình này có dạng chuẩn của tự chú ý, nhưng có độ phức tạp thời gian và không gian thấp hơn, tức là $O(n' \cdot n \cdot d) < O(n^2 \cdot d)$ và $O(n' \cdot n + n' \cdot d) < O(n^2 + n \cdot d)$. Nếu $n' \ll n$, mô hình thu được sẽ tuyến tính theo n.

Chú giải của người dịch: Kích thước các ma trận sau khi giảm n

$\mathbf{Q}$ vẫn là $n \times d$, còn $[\mathbf{K}']^\top$ là $d \times n'$, nên $\mathbf{A}$ giờ là ma trận $n \times n'$ (mỗi truy vấn chỉ có n' trọng số). Nhân $\mathbf{A}$ $(n \times n')$ với $\mathbf{V}'$ $(n' \times d)$ lại cho ra đúng $n \times d$ như yêu cầu. Nếu giữ $n' = 256$ cố định, khi n tăng gấp đôi thì chi phí chỉ tăng gấp đôi (tuyến tính), thay vì gấp bốn như với n^2.

Vấn đề then chốt ở đây là làm sao thu được $\mathbf{K}'$ và $\mathbf{V}'$ theo cách giữ lại phần lớn thông tin trong $\mathbf{K}$ và $\mathbf{V}$. Có một số cách để làm việc này. Một phương pháp đơn giản là chọn những khóa và giá trị được cho là quan trọng. Mức độ quan trọng của một khóa (hoặc giá trị) có thể được tính theo một thước đo rẻ về tính toán nào đó. Ví dụ, ta có thể lấy mẫu một số ít tích vô hướng truy vấn–khóa và ước lượng mức độ quan trọng của một khóa bằng cách tổng hợp các kết quả tích vô hướng này.

Dù đơn giản, phương pháp trên vẫn dựa vào các phép toán thưa như lấy mẫu và tổng hợp. Một cách khác là dùng tính toán dày đặc để chiếu $\mathbf{K}$ và $\mathbf{V}$ xuống $\mathbf{K}'$ và $\mathbf{V}'$. Một cách tiếp cận phổ biến là dùng CNN [Liu và cộng sự, 2018]. Gọi $\text{Conv}(\cdot)$ là một phép tích chập 1D trượt dọc theo chiều n. Khi đó $\mathbf{K}'$ được tính như sau:

$$\mathbf{K}' = \text{Conv}(\mathbf{K}, \mathbf{W}_c, \text{size}_r, \text{stride}), \tag{6.132}$$

trong đó $\mathbf{W}_c$ là ma trận tham số của các bộ lọc, size_r là kích thước của trường tiếp nhận (*receptive field*), và stride là kích thước bước của phép tích chập. Nói chung, ta có thể đạt tỷ lệ nén cao bằng cách chọn các giá trị lớn cho size_r và stride. Tương tự, ta có thể tính $\mathbf{V}'$ bằng một hàm tích chập khác. Đáng chú ý là, nếu tham số n' cố định cho mọi mẫu, việc nén $\mathbf{K}$ và $\mathbf{V}$ dọc theo chiều độ dài về bản chất giống mô hình bộ nhớ có độ dài cố định đã mô tả ở tiểu mục trước. Các phương pháp trình bày ở đây tổng quát hơn và có thể được áp dụng cho các bộ nhớ có độ dài thay đổi.

Chú giải của người dịch: Tích chập 1D để nén chuỗi

Hình dung một "cửa sổ" rộng size_r trượt dọc theo n hàng của $\mathbf{K}$, mỗi lần nhảy stride hàng; tại mỗi vị trí, nó gộp các hàng trong cửa sổ thành một hàng mới (bằng tổ hợp tuyến tính với trọng số học được $\mathbf{W}_c$). Ví dụ $n = 12$, $\text{size}_r = \text{stride} = 3$: cửa sổ đặt ở các hàng 1–3, 4–6, 7–9, 10–12, cho ra $n' = 4$ hàng, tức là nén chuỗi khóa xuống 3 lần.

Ta cũng có thể muốn mô hình hóa hàm chú ý bằng cách coi ma trận chú ý $\mathbf{A}$ là một biểu diễn nhiều chiều của dữ liệu, rồi áp dụng các phương pháp giảm chiều (*dimensionality reduction*) truyền thống. Theo thực nghiệm, trong nhiều bài toán, $\mathbf{A}$ (hay chính xác hơn là $\mathbf{Q}\mathbf{K}^\top$) được thấy là một ma trận hạng thấp (*low-rank matrix*). Trong trường hợp này, ta có thể nén $\mathbf{A}$ trong khi vẫn giữ lại được nhiều thông tin nhất có thể. Có nhiều cách để làm điều này. Ví dụ, ta có thể dùng tích của các ma trận nhỏ hơn làm xấp xỉ cho $\mathbf{A}$ thông qua kỹ thuật SVD (xem Chương 3). Tuy nhiên, việc dùng SVD gây thêm chi phí tính toán so với mô hình chú ý chuẩn. Một ý tưởng đơn giản hơn là biến đổi trực tiếp $\mathbf{K}$ và $\mathbf{V}$ thành các ma trận kích thước nhỏ hơn bằng các ánh xạ tuyến tính, cho bởi

$$\mathbf{K}' = \mathbf{U}^k\mathbf{K}, \tag{6.133}$$

$$\mathbf{V}' = \mathbf{U}^v\mathbf{V}, \tag{6.134}$$

trong đó $\mathbf{U}^k \in \mathbb{R}^{n' \times n}$ và $\mathbf{U}^v \in \mathbb{R}^{n' \times n}$ là các ma trận tham số. Rõ ràng, điều này dẫn tới một mô hình tương đương với mô hình được mô tả trong Phương trình (6.130) và (6.131). Dù rất trực quan, cách tiếp cận này đã được chứng minh là cho sai số xấp xỉ ϵ đủ nhỏ, miễn là n' tăng tuyến tính theo d/ϵ^2 [Wang và cộng sự, 2020b].

Chú giải của người dịch: Ma trận "hạng thấp" là gì?

Một ma trận có *hạng thấp* khi các hàng của nó "lặp lại" nhau theo nghĩa có thể tạo ra từ một vài hàng gốc bằng cách nhân với số rồi cộng lại. Ví dụ, ma trận $\begin{bmatrix}1 & 2 & 3\\ 2 & 4 & 6\\ 3 & 6 & 9\end{bmatrix}$ có 9 số nhưng mọi hàng đều là bội của $[1\ 2\ 3]$; nó bằng tích của cột $[1; 2; 3]$ với hàng $[1\ 2\ 3]$, nên chỉ cần lưu 6 số (hạng bằng 1). Nếu ma trận chú ý "gần" hạng thấp như vậy, ta có thể nén nó mà mất rất ít thông tin. SVD là một công cụ đại số tuyến tính tìm cách phân tích như thế một cách tối ưu.

2. Giảm d

Một cách tiếp cận khác để giảm độ phức tạp tính toán là giảm số chiều d. Một trong những phương pháp đơn giản nhất là chiếu tất cả truy vấn và khóa lên một không gian d' chiều ($d' < d$), và tính tích vô hướng giữa các cặp truy vấn–khóa trong không gian mới. Về mặt mô hình hóa, ta chỉ cần thay $\mathbf{Q} \in \mathbb{R}^{n\times d}$ và $\mathbf{K} \in \mathbb{R}^{n\times d}$ bằng các biểu diễn mới $\mathbf{Q}' \in \mathbb{R}^{n\times d'}$ và $\mathbf{K}' \in \mathbb{R}^{n\times d'}$. Ta có thể dễ dàng sửa Phương trình (6.129) để dùng $\mathbf{Q}'$ và $\mathbf{K}'$ khi tính ma trận chú ý

$$\mathbf{A} \;=\; \text{Softmax}\Big(\frac{\mathbf{Q}'[\mathbf{K}']^\top}{\sqrt{d'}} + \mathbf{M}\Big). \tag{6.135}$$

$\mathbf{Q}'$ và $\mathbf{K}'$ được cho bởi

$$\mathbf{Q}' = \mathbf{Q}\mathbf{U}^q, \tag{6.136}$$

$$\mathbf{K}' = \mathbf{K}\mathbf{U}^k, \tag{6.137}$$

trong đó $\mathbf{U}^q \in \mathbb{R}^{d\times d'}$ và $\mathbf{U}^k \in \mathbb{R}^{d\times d'}$ là các ma trận tham số của các phép biến đổi tuyến tính.

Cũng có thể khai thác các **phương pháp hạt nhân** (*kernel methods*) để thu được một mô hình chú ý tích vô hướng hiệu quả. Ý tưởng cơ bản là ánh xạ các điểm dữ liệu (biểu diễn bằng vectơ) từ một không gian sang không gian khác, biến một bài toán khó trong không gian gốc thành một bài toán dễ xử lý hơn trong không gian mới. "Thủ thuật hạt nhân (*kernel trick*)" cho phép ta tính các tích trong như vậy một cách ngầm định thông qua một hàm hạt nhân (*kernel function*), mà không cần xây dựng tường minh ánh xạ $\phi(\cdot)$[14]. Các tích trong như vậy trong không gian đặc trưng thường được biểu diễn bằng các hàm hạt nhân, ký hiệu là $K(\cdot,\cdot)$.

Một điều thú vị là xấp xỉ $\mathbf{A}$ theo cách tương tự $K(\cdot,\cdot)$ trong các phương pháp hạt nhân. Để minh họa, lưu ý rằng trong Phương trình (6.129), $\mathbf{A}$ biểu diễn các trọng số chú ý đã được chuẩn hóa. Tử số có thể được viết dưới dạng

$$\widetilde{\mathbf{A}} \;=\; \text{Mask}\Big(\exp\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\Big)\Big). \tag{6.139}$$

Ở đây $\text{Mask}(\cdot)$ là một hàm có tác dụng giống như việc dùng biến mặt nạ cộng $\mathbf{M}$. Khi đó, $\mathbf{A}$ có thể được biểu diễn là

$$\mathbf{A} \;=\; \mathbf{D}^{-1}\widetilde{\mathbf{A}}, \tag{6.140}$$

trong đó $\mathbf{D}$ là một ma trận đường chéo (*diagonal matrix*) kích thước $n \times n$. Mỗi phần tử trên đường chéo chính bằng tổng các phần tử của hàng tương ứng trong $\widetilde{\mathbf{A}}$, biểu thị hệ số chuẩn hóa của Softmax. Thay phương trình này vào Phương trình (6.129), ta có

$$\text{Att}_{\text{self}}(\mathbf{S}) \;=\; \mathbf{D}^{-1}\widetilde{\mathbf{A}}\mathbf{V}. \tag{6.141}$$

[14]Trong giải tích toán học, tích trong (*inner product*) là một khái niệm tổng quát hóa của tích vô hướng, thường được ký hiệu là $\langle\cdot,\cdot\rangle$. Một định nghĩa hình thức đòi hỏi $\langle\cdot,\cdot\rangle$ thỏa mãn một số tính chất trong một không gian vectơ. Mặc dù tích trong có những dạng khác nhau trong các bối cảnh khác nhau, trong không gian Euclid $\mathbb{R}^d$ nó tương đương với tích vô hướng. Tức là, cho hai vectơ $\mathbf{a} \in \mathbb{R}^d$ và $\mathbf{b} \in \mathbb{R}^d$, ta có

$$\begin{aligned}\langle \mathbf{a}, \mathbf{b}\rangle &= \mathbf{a}\cdot\mathbf{b}\\ &= \sum_{i=1}^{d} a_i \cdot b_i\end{aligned} \tag{6.138}$$

Chú giải của người dịch: Tách Softmax thành "tử số" và "mẫu số"

Nhớ lại Softmax của một hàng $(z_1, \ldots, z_n)$ cho trọng số $e^{z_j} / \sum_{j'} e^{z_{j'}}$. $\widetilde{\mathbf{A}}$ chứa các tử số e^{z_j} (ô bị che thì bằng 0), còn $\mathbf{D}$ là ma trận chỉ có số trên đường chéo, mỗi số là mẫu số $\sum_{j'} e^{z_{j'}}$ của một hàng. Nhân với $\mathbf{D}^{-1}$ (ma trận đường chéo chứa các số nghịch đảo) đơn giản là chia mỗi hàng của $\widetilde{\mathbf{A}}$ cho tổng của hàng đó. Ví dụ hàng $(e^0, e^{\ln 3}) = (1, 3)$ có tổng 4, sau chuẩn hóa thành $(0{,}25;\ 0{,}75)$.

Trong mô hình này, $\widetilde{A}(i, j)$ có thể được xem là một hàm tương tự trên mọi cặp truy vấn–khóa trong một không gian d chiều. Ở đây ta giả sử rằng hàm này, vốn có dạng tích vô hướng của các vectơ, có thể được xấp xỉ bằng một hàm hạt nhân

$$\begin{aligned}\widetilde{A}(i, j) &= K(\mathbf{q}_i, \mathbf{k}_j) \\ &= \langle \phi(\mathbf{q}_i), \phi(\mathbf{k}_j) \rangle.\end{aligned}$$

$\phi(\cdot)$ là một ánh xạ từ $\mathbb{R}^d$ sang $\mathbb{R}^{d'}$. Ta có thể biểu diễn các truy vấn và khóa theo dạng sau

$$\begin{aligned}\mathbf{Q}' &= \phi(\mathbf{Q}) \\ &= \begin{bmatrix} \phi(\mathbf{q}_1) \\ \vdots \\ \phi(\mathbf{q}_n) \end{bmatrix}, \end{aligned} \tag{6.142}$$

$$\begin{aligned}\mathbf{K}' &= \phi(\mathbf{K}) \\ &= \begin{bmatrix} \phi(\mathbf{k}_1) \\ \vdots \\ \phi(\mathbf{k}_n) \end{bmatrix}. \end{aligned} \tag{6.143}$$

Sau đó, ta xây dựng một mô hình chú ý hạt nhân hóa (*kernelized attention model*) bằng cách xấp xỉ trọng số chú ý $\alpha_{i,j}$ dưới dạng

$$\alpha_{i,j} \approx \frac{\phi(\mathbf{q}_i)\phi(\mathbf{k}_j)^\top}{\sum_{j'=1}^{n} \phi(\mathbf{q}_i)\phi(\mathbf{k}_{j'})^\top}. \tag{6.144}$$

Ý tưởng then chốt đằng sau mô hình chú ý hạt nhân hóa này là ta có thể bỏ hàm Softmax nếu các truy vấn và khóa được ánh xạ sang một không gian mới. Dùng phép xấp xỉ này, vectơ đầu ra thứ i của mô hình chú ý (tức là vectơ hàng thứ i của $\text{Att}_{\text{self}}(\mathbf{S})$) được cho bởi

$$\begin{aligned}\mathbf{c}_i &= \sum_{j=1}^{n} \alpha_{i,j} \cdot \mathbf{v}_j \\ &\approx \sum_{j=1}^{n} \Big(\frac{\phi(\mathbf{q}_i)\phi(\mathbf{k}_j)^\top}{\sum_{j'=1}^{n} \phi(\mathbf{q}_i)\phi(\mathbf{k}_{j'})^\top} \cdot \mathbf{v}_j \Big) \\ &= \frac{\sum_{j=1}^{n} \phi(\mathbf{q}_i)\phi(\mathbf{k}_j)^\top \mathbf{v}_j}{\sum_{j'=1}^{n} \phi(\mathbf{q}_i)\phi(\mathbf{k}_{j'})^\top} \\ &= \frac{\phi(\mathbf{q}_i)\big(\sum_{j=1}^{n} \phi(\mathbf{k}_j)^\top \mathbf{v}_j\big)}{\phi(\mathbf{q}_i)\big(\sum_{j'=1}^{n} \phi(\mathbf{k}_{j'})^\top\big)}. \end{aligned} \tag{6.145}$$

Ở đây $\phi(\mathbf{q}_i) \in \mathbb{R}^{1 \times d'}$, còn $\phi(\mathbf{k}_j)^\top \mathbf{v}_j \in \mathbb{R}^{d' \times d}$. Do đó, số hạng bên trong $\sum_{j=1}^{n} \phi(\mathbf{k}_j)^\top \mathbf{v}_j$ là một ma trận $d' \times d$, và tích cuối cùng cho ra một vectơ trong $\mathbb{R}^{1 \times d}$.

Dù phương trình trông có vẻ hơi phức tạp, ý tưởng lại đơn giản: thay vì cho truy vấn chú ý tới mọi khóa để thu được trọng số chú ý $\alpha_{i,j}$, ta có thể tính tổng các tích $\sum_{j=1}^{n} \phi(\mathbf{k}_j)^\top \mathbf{v}_j \in \mathbb{R}^{d' \times d}$ rồi nhân nó với truy vấn đã hạt nhân hóa $\phi(\mathbf{q}_i)$. Quay lại ký hiệu dùng trong Phương trình (6.141), ta định nghĩa phần tử đường chéo thứ i của $\mathbf{D}$ là $\phi(\mathbf{q}_i) \sum_{j'=1}^{n} \phi(\mathbf{k}_{j'})^\top$. Khi đó, mô hình chú ý có thể được viết lại dưới dạng

$$\begin{aligned} \mathrm{Att}_{\text{self}}(\mathbf{S}) &= \mathbf{D}^{-1}\phi(\mathbf{Q})\phi(\mathbf{K})^\top \mathbf{V} \\ &= \mathbf{D}^{-1}\mathbf{Q}'\mathbf{K}'^\top \mathbf{V} \\ &= \mathbf{D}^{-1}\big(\mathbf{Q}'(\mathbf{K}'^\top \mathbf{V})\big). \end{aligned} \tag{6.146}$$

Ở đây ta đổi thứ tự tính toán từ trái-sang-phải thành phải-sang-trái bằng cách dùng dấu ngoặc. Với $\mathbf{Q}' \in \mathbb{R}^{n \times d'}$ và $\mathbf{K}' \in \mathbb{R}^{n \times d'}$, mô hình này có độ phức tạp thời gian và không gian lần lượt là $O(n \cdot d \cdot d')$ và $O(n \cdot d + n \cdot d' + d \cdot d')$. Do đó, mô hình tuyến tính theo độ dài chuỗi n, và đôi khi được gọi là **mô hình chú ý tuyến tính** (*linear attention model*). Một ưu điểm tính toán của mô hình này là ta chỉ cần tính phép nhân $\mathbf{K}'^\top \mathbf{V}$ (tức là $\sum_{j=1}^{n} \phi(\mathbf{k}_j)^\top \mathbf{v}_j$) và hệ số chuẩn hóa tương ứng (tức là $\sum_{j'=1}^{n} \phi(\mathbf{k}_{j'})^\top$) một lần. Các kết quả này sau đó có thể được dùng cho bất kỳ truy vấn nào [Katharopoulos và cộng sự, 2020]. Mô hình cần duy trì $\sum_{j=1}^{n} \phi(\mathbf{k}_j)^\top \mathbf{v}_j$ và $\sum_{j'=1}^{n} \phi(\mathbf{k}_{j'})^\top$, và cập nhật chúng khi có các vectơ khóa và giá trị mới đến.

Chú giải của người dịch: Vì sao đổi thứ tự nhân lại tiết kiệm đến vậy?

Phép nhân ma trận có tính kết hợp: $(\mathbf{XY})\mathbf{Z} = \mathbf{X}(\mathbf{YZ})$, kết quả như nhau nhưng chi phí có thể rất khác. Tính $(\mathbf{Q}'\mathbf{K}'^\top)\mathbf{V}$ trước tiên tạo ra ma trận $n \times n$ (cỡ $n^2 d'$ phép nhân). Còn tính $\mathbf{K}'^\top \mathbf{V}$ trước chỉ tạo ma trận nhỏ $d' \times d$ (cỡ $n\,d'd$ phép nhân), rồi nhân $\mathbf{Q}'$ với nó (thêm cỡ $n\,d'd$). Với $n = 100\,000$, $d = d' = 64$: cách thứ nhất cần khoảng $6{,}4 \times 10^{11}$ phép nhân, cách thứ hai chỉ khoảng 8×10^8 — ít hơn gần 800 lần. Mẹo này chỉ dùng được khi đã bỏ Softmax, vì Softmax phải áp dụng lên từng hàng của ma trận $n \times n$ nên buộc ta tính nó trước.

Dù vậy, mô hình hạt nhân hóa này còn một số hạn chế, chẳng hạn làm thế nào để xây dựng ánh xạ đặc trưng (*feature map*) $\phi(\cdot)$ sao cho xấp xỉ tốt mô hình chú ý chuẩn. Bạn đọc quan tâm có thể tham khảo công trình của Choromanski và cộng sự [2020] để biết thêm chi tiết.

Ý tưởng thứ hai để giảm d là dùng các mô hình không gian con (*sub-space model*), trong đó một bài toán trong không gian d chiều được biến đổi thành các bài toán con trong những không gian có số chiều thấp hơn, và lời giải của bài toán gốc được xấp xỉ bằng một tổ hợp nào đó của các lời giải của những bài toán con này. Trong một mô hình không gian con tổng quát, một vectơ khóa d chiều $\mathbf{k}$ có thể được ánh xạ thành một tập các vectơ d' chiều $\{\mathbf{k}'_1, \cdots, \mathbf{k}'_\eta\}$. Để đơn giản hóa việc mô hình hóa, ta có thể làm điều này bằng cách phân đoạn vectơ, tức là chia $\mathbf{k}$ thành η vectơ con, mỗi vectơ có $d' = \frac{d}{\eta}$ chiều. Ta có thể biến đổi mọi vectơ truy vấn và giá trị theo cùng cách đó. Sau đó, mô hình chú ý được áp dụng trong từng không gian con này.

Tuy nhiên, phương pháp này không làm giảm tổng lượng tính toán. Như được trình bày trong công trình của Lample và cộng sự [2019], thay vào đó ta có thể xấp xỉ chú ý tích vô hướng trên một tập các cặp khóa–giá trị bằng cách xét p ứng viên hàng đầu trong mỗi không gian con. Chính xác hơn, ta tìm p cặp khóa–giá trị tốt nhất trong mỗi không gian con, việc này rẻ hơn về tính toán. Tích Descartes (*Cartesian product*) của các tập p khóa tốt nhất này gồm p^η khóa tích (*product key*). Tương tự, ta thu được p^η giá trị tích. Phần việc còn lại rất đơn giản: các truy vấn d chiều chú ý tới những khóa tích và giá trị tích d chiều này. Một khác biệt thú vị giữa mô hình không gian con này và mô hình trong không gian d chiều là các khóa tích và giá trị tích được sinh ra có thể khác với bất kỳ cặp khóa–giá trị gốc nào trong $\{(\mathbf{k}_1, \mathbf{v}_1), \cdots, (\mathbf{k}_{i-1}, \mathbf{v}_{i-1})\}$. Điều này mang lại một cách để học các biểu diễn mới của thông

tin trong quá khứ.

Chú giải của người dịch: Khóa tích: ghép các "nửa" tốt nhất

Lấy $\eta = 2$: mỗi khóa được cắt làm hai nửa. Ở nửa thứ nhất, ta tìm $p = 3$ nửa-khóa khớp nhất với nửa đầu của truy vấn, ví dụ $\{a_1, a_2, a_3\}$; ở nửa thứ hai tìm được $\{b_1, b_2, b_3\}$. Ghép mọi cặp (tích Descartes, giống việc đếm số cách chọn một bộ "áo–quần" theo quy tắc nhân) cho $3 \times 3 = 3^2 = 9$ khóa đầy đủ $[a_s; b_t]$. Lưu ý một khóa ghép như $[a_1; b_3]$ có thể chưa từng xuất hiện nguyên vẹn trong dữ liệu — đó là lý do sách nói mô hình có thể tạo ra "biểu diễn mới".

Cho tới giờ, chúng ta đã bàn về các cách tiếp cận giảm chiều theo chiều n hoặc chiều d. Có thể dễ dàng kết hợp chúng để xây dựng một mô hình "ít chiều hơn". Ví dụ, giả sử ta có phép giảm $n \to n'$ cho khóa và giá trị, và phép giảm $d \to d'$ cho truy vấn và khóa. Mô hình có dạng

$$\begin{aligned} \mathrm{Att}_{\mathrm{self}}(\mathbf{S}) &= \mathbf{A}\mathbf{V}', \\ \mathbf{A} &= \mathrm{Softmax}\left(\frac{\mathbf{Q}'\mathbf{K}'^{\top}}{\sqrt{d'}} + \mathbf{M}\right), \end{aligned} \tag{6.147}$$

trong đó $\mathbf{Q}' \in \mathbb{R}^{n \times d'}$, $\mathbf{K}' \in \mathbb{R}^{n' \times d'}$ và $\mathbf{V}' \in \mathbb{R}^{n' \times d}$ là các biểu diễn ít chiều của truy vấn, khóa và giá trị. Như thường lệ, ta có thể dễ dàng thu được các biểu diễn này thông qua các ánh xạ tuyến tính của $\mathbf{Q}$, $\mathbf{K}$ và $\mathbf{V}$. Độ phức tạp thời gian và không gian của mô hình này là $O(n' \cdot n \cdot d')$ và $O(n' \cdot n + n' \cdot d')$.

6.4.4 Chia sẻ tham số và giá trị kích hoạt

Sự dư thừa (*redundancy*) là hiện tượng phổ biến ở hầu hết các mạng nơ-ron quy mô lớn. Kết quả là nhiều mô hình trong số này bị thừa tham số (*over-parameterized*), khiến việc huấn luyện và suy luận kém hiệu quả hơn. Một cách tiếp cận phổ biến để giảm dư thừa là đơn giản hóa mô hình bằng cách loại bỏ các thành phần không cần thiết. Chẳng hạn, ta có thể tỉa (*prune*) một mô hình phức tạp hoặc chia sẻ các mô-đun con giữa các phần khác nhau của mô hình để thu được một mô hình gọn hơn. Trong tiểu mục này, chúng ta bàn về các phương pháp chia sẻ tham số và trạng thái trung gian trong các mô hình Transformer. Việc thảo luận về chuyển giao mô hình và tỉa mô hình được để lại cho Mục 6.4.7.

Các kiến trúc chia sẻ tham số được dùng rộng rãi trong các hệ thống dựa trên mạng nơ-ron. Các ví dụ nổi tiếng gồm CNN và RNN, trong đó cùng một bộ tham số (hoặc các tầng) được áp dụng trên những vùng khác nhau của đầu vào. Điều này tạo ra một mạng nơ-ron lớn mà nhiều phần của nó có cùng kiến trúc và cùng các tham số được chia sẻ. Đối với Transformer cũng như các mô hình chuỗi khác, cơ chế chia sẻ có thể được áp dụng ở các mức mô hình hóa khác nhau. Một ví dụ đơn giản là chia sẻ vectơ nhúng (*embedding sharing*). Trong dịch máy, một chiến lược điển hình để xử lý các từ thuộc hai ngôn ngữ là xây dựng hai mô hình nhúng riêng biệt. Một cách khác là dùng một mô hình nhúng duy nhất cho cả hai ngôn ngữ. Khi đó, các tham số của mô hình được học trong quá trình huấn luyện cả mạng phía nguồn lẫn mạng phía đích. Chiến lược như vậy cũng thường được áp dụng trong các mô hình chuỗi đa ngôn ngữ, chẳng hạn các mô hình ngôn ngữ có khả năng xử lý văn bản bằng nhiều ngôn ngữ khác nhau.

Đối với các mạng nơ-ron nhiều tầng, một phương pháp phổ biến là chia sẻ theo tầng (*layer-wise sharing*). Giả sử có một chồng các tầng, tất cả đều có cùng dạng

$$\mathbf{S}^l = \mathrm{Layer}(\mathbf{S}^{l-1}; \theta^l). \tag{6.148}$$

Ta có thể ràng buộc tham số cho một số hoặc tất cả các tầng này. Chẳng hạn, cho một tập các tầng $\{l_1, l_2, \cdots, l_n\}$, ta áp đặt ràng buộc $\theta^{l_1} = \theta^{l_2} = \cdots = \theta^{l_n}$, để có thể thu được một mô hình nhỏ hơn và

việc tối ưu mô hình có thể dễ dàng hơn. Trên thực tế, mô hình chia sẻ tầng này rất có lợi nếu có nhiều tầng tham gia, vì ta có thể lặp lại cùng một quá trình nhiều lần để xây dựng một mạng nơ-ron rất sâu [Dehghani và cộng sự, 2018]. Ví dụ, việc chia sẻ một tầng con FFN duy nhất cho tất cả các tầng của bộ mã hóa được thấy là hiệu quả trong việc giảm dư thừa ở các hệ thống dịch máy [Pires và cộng sự, 2023].

Chú giải của người dịch: Chia sẻ tầng tiết kiệm bao nhiêu?

Giả sử mỗi tầng có 10 triệu tham số và mô hình có 12 tầng: không chia sẻ thì cần $12 \times 10 = 120$ triệu tham số. Nếu cả 12 tầng dùng chung một bộ θ, mô hình chỉ còn 10 triệu tham số, nhưng vẫn "chạy" qua 12 lượt biến đổi — giống như dùng cùng một chiếc máy ép 12 lần thay vì mua 12 chiếc máy. Lưu ý: số tham số (bộ nhớ) giảm mạnh, nhưng số phép tính khi chạy thì không giảm.

Đối với Transformer, việc chia sẻ cũng có thể được thực hiện trong chú ý đa đầu. Một ví dụ là **chú ý đa truy vấn** (*multi-query attention*) [Shazeer, 2019]. Nhớ lại từ Mục 6.1.3 rằng đầu ra của một đầu h trong tự chú ý đa đầu chuẩn có thể được viết là

$$\begin{aligned}\mathbf{C}_h^{\text{head}} &= \text{Att}_{\text{qkv}}(\mathbf{S}_h^q, \mathbf{S}_h^k, \mathbf{S}_h^v) \\ &= \text{Att}_{\text{qkv}}(\mathbf{S}\mathbf{W}_h^q, \mathbf{S}\mathbf{W}_h^k, \mathbf{S}\mathbf{W}_h^v).\end{aligned} \tag{6.149}$$

Ở đây $\mathbf{S}_h^q = \mathbf{S}\mathbf{W}_h^q$, $\mathbf{S}_h^k = \mathbf{S}\mathbf{W}_h^k$ và $\mathbf{S}_h^v = \mathbf{S}\mathbf{W}_h^v$ là các truy vấn, khóa và giá trị, thu được bằng cách biến đổi tuyến tính đầu vào $\mathbf{S}$ với các ma trận tham số riêng biệt $\mathbf{W}_h^q$, $\mathbf{W}_h^k$ và $\mathbf{W}_h^v$. Trong chú ý đa truy vấn, ta chia sẻ cùng các khóa và giá trị cho tất cả các đầu, nhưng dùng các truy vấn khác nhau cho các đầu khác nhau. Dạng của mô hình này được cho bởi

$$\mathbf{C}_h^{\text{head}} = \text{Att}_{\text{qkv}}(\mathbf{S}\mathbf{W}_h^q, \mathbf{S}\mathbf{W}_0^k, \mathbf{S}\mathbf{W}_0^v). \tag{6.150}$$

Vì các khóa $\mathbf{S}\mathbf{W}_0^k$ và giá trị $\mathbf{S}\mathbf{W}_0^v$ không phụ thuộc vào h, chúng chỉ được tính và lưu một lần, thay vì lặp lại một cách dư thừa cho mỗi đầu. Điều này giúp tiết kiệm đáng kể về tính toán và băng thông bộ nhớ (*memory bandwidth*) trong quá trình suy luận, đặc biệt khi số đầu chú ý lớn. Nhờ tính hiệu quả của nó, chú ý đa truy vấn (và biến thể của nó, chú ý truy vấn theo nhóm (*grouped-query attention*)) đã được tích hợp thành công vào các mô hình ngôn ngữ lớn hiện đại, bao gồm Llama 2 [Touvron và cộng sự, 2023b] và Falcon[15].

Chú giải của người dịch: Đa truy vấn và theo nhóm

Với 32 đầu chú ý, chú ý đa đầu chuẩn phải lưu 32 bộ khóa–giá trị cho mỗi token trong bộ nhớ đệm KV; chú ý đa truy vấn chỉ lưu 1 bộ dùng chung, tức là nhỏ hơn 32 lần. Chú ý truy vấn theo nhóm là phương án ở giữa: chia 32 đầu thành, chẳng hạn, 8 nhóm, mỗi nhóm 4 đầu dùng chung một bộ khóa–giá trị, nên chỉ cần lưu 8 bộ. Cách này cân bằng giữa tiết kiệm bộ nhớ và giữ chất lượng mô hình.

Mở rộng ý tưởng chia sẻ sang các tình huống tổng quát hơn, về nguyên tắc, các trạng thái trung gian có thể được chia sẻ trong khắp một mạng nơ-ron. Chẳng hạn, việc tái sử dụng giá trị kích hoạt của nơ-ron cho phép các thành phần con được tính nhiều lần một cách hiệu quả. Trong Transformer, phương pháp này có thể được áp dụng trực tiếp cho các ma trận tự chú ý. Các nghiên cứu đã cho thấy bằng thực nghiệm rằng các bản đồ chú ý ở những tầng kề nhau thường có tương quan cao trong một số tác vụ NLP [Xiao và cộng sự, 2019]. Do đó, một chiến lược hiệu quả về tính toán là tính bản đồ chú ý dày đặc một lần và tái sử dụng nó ở các tầng tiếp theo.

[15] `https://falconllm.tii.ae/index.html`

Nói rộng hơn, cơ chế chia sẻ có thể được xem như một quá trình trong đó ta tái sử dụng các kết quả đã tính trước đó thay vì tính lại chúng ngay tại chỗ. Do đó, có thể tái sử dụng thông tin giữa các lần chạy khác nhau của một mạng nơ-ron. Một ví dụ liên quan là mạng phần dư khả nghịch (*reversible residual network*), trong đó giá trị kích hoạt của một tầng có thể được khôi phục từ giá trị kích hoạt của tầng ngay sau nó [Gomez và cộng sự, 2017]. Vì vậy, ta chỉ giữ đầu ra của tầng mới nhất trong lượt truyền xuôi. Sau đó, trong lượt truyền ngược của quá trình huấn luyện, ta tái tạo đầu ra của mỗi tầng từ tầng kế sau nó. Một ưu điểm của thiết kế khả nghịch này là thông tin sinh ra trong lượt truyền xuôi được chia sẻ một cách ngầm định, và mô hình tiết kiệm bộ nhớ [Kitaev và cộng sự, 2020].

Chú giải của người dịch: "Khả nghịch" nghĩa là gì?

Một phép biến đổi là khả nghịch nếu từ đầu ra ta suy ngược được đầu vào, như hàm $y = 2x + 1$ có hàm ngược $x = (y - 1)/2$. Bình thường, khi huấn luyện, ta phải lưu đầu ra của *mọi* tầng để dùng lúc lan truyền ngược, rất tốn bộ nhớ khi mạng sâu. Với các tầng khả nghịch, ta chỉ cần lưu đầu ra tầng cuối, rồi lần lượt "tính ngược" ra đầu ra của các tầng trước khi cần — đổi một ít phép tính lấy rất nhiều bộ nhớ.

6.4.5 Các phương án thay thế cho tự chú ý

Ta đã thấy rằng việc dùng tự chú ý (*self-attention*) là một trong những nguồn chính gây ra chi phí tính toán và bộ nhớ cao trong Transformer. Một câu hỏi tự nhiên là liệu có những phương án thay thế hiệu quả cho các mô hình tự chú ý hay không. Ở đây, chúng tôi trình bày ngắn gọn một số biến thể của Transformer, trong đó các tầng con tự chú ý được thay bằng những loại kiến trúc mạng nơ-ron khác.

1. CNN thay thế cho tự chú ý

Mạng nơ-ron tích chập (*convolutional neural network*) (CNN) là loại mạng nơ-ron đơn giản và được dùng rộng rãi, và được coi là phương án tiềm năng để thay thế các mô hình tự chú ý. Để áp dụng CNN vào Transformer, tất cả những gì ta cần là xây dựng một tầng con tích chập để thay cho tầng con tự chú ý trong một khối Transformer. Mặc dù một bộ lọc CNN có trường tiếp nhận (*receptive field*) hạn chế và do đó chỉ nhận đầu vào từ một cửa sổ ngữ cảnh "cục bộ", ta vẫn có thể dễ dàng mô hình hóa ngữ cảnh rộng bằng cách xếp chồng nhiều tầng con tích chập. Một ưu điểm chính của CNN là độ phức tạp tính toán của nó tăng tuyến tính theo độ dài chuỗi n, so với mức tăng bậc hai của các mô hình tự chú ý. Hơn nữa, việc có sẵn các cài đặt CNN đã được tối ưu hóa rất tốt khiến chúng dễ áp dụng cho mô hình hóa chuỗi trong các hệ thống thực tế. Để cải thiện thêm hiệu quả bộ nhớ, ta có thể dùng các biến thể CNN gọn nhẹ, chẳng hạn CNN theo chiều sâu (*depthwise CNN*) [Wu và cộng sự, 2018a][16].

[16] Nhắc lại từ Chương 2 rằng trong CNN, một bộ lọc (hoặc một tập bộ lọc) ánh xạ các biến đầu vào nằm trong trường tiếp nhận của nó thành một biến đầu ra (hoặc một tập biến đầu ra) thông qua một ánh xạ tuyến tính. Giả sử đầu vào và đầu ra của một bài toán được biểu diễn dưới dạng chuỗi các vectơ đặc trưng. Cho một bộ lọc chuẩn có trường tiếp nhận kích thước $k \times d$, ta trượt nó dọc theo chuỗi. Ở mỗi bước, bộ lọc nhận $k \times d$ đặc trưng đầu vào và tạo ra một đặc trưng đầu ra duy nhất. Quá trình này thường được biểu diễn như sau:

$$y = \text{ReduceSum}(\mathbf{x} \odot \mathbf{W}), \tag{6.151}$$

trong đó $\mathbf{x} \in \mathbb{R}^{k \times d}$ là biểu diễn ma trận của đầu vào, $y \in \mathbb{R}$ là đặc trưng đầu ra, và $\mathbf{W} \in \mathbb{R}^{k \times d}$ là ma trận trọng số. Hàm $\text{ReduceSum}(\cdot)$ tính tổng của tất cả các tích theo từng phần tử giữa $\mathbf{x}$ và $\mathbf{W}$. Nếu muốn đầu vào và đầu ra có cùng số đặc trưng d, ta phải áp dụng d bộ lọc khác nhau, dẫn đến tổng cộng $d \cdot (k \cdot d) = d^2 \cdot k$ tham số. Trong CNN theo chiều sâu, ta tách phép tích chập theo không gian khỏi các chiều đặc trưng. Thay vì áp dụng các bộ lọc đầy đủ $k \times d$, mỗi kênh trong d kênh đặc trưng đầu vào được tích chập độc lập với bộ lọc không gian 1 chiều riêng có kích thước k. Nhờ đó, số tham số riêng biệt giảm mạnh xuống còn $d \cdot k$ (mỗi kênh trong d kênh ứng với một bộ lọc riêng chứa k tham số riêng biệt).

Chú giải của người dịch: Vì sao CNN theo chiều sâu tiết kiệm tham số?

Lấy ví dụ nhỏ: cửa sổ $k = 3$ từ, mỗi từ có $d = 4$ đặc trưng. Một bộ lọc chuẩn nhìn cả khối $3 \times 4 = 12$ số, nhân từng số với một trọng số rồi cộng lại thành *một* số đầu ra. Muốn đầu ra cũng có 4 đặc trưng thì cần 4 bộ lọc như vậy: $4 \times 12 = 48$ tham số ($= d^2 \cdot k$). Với CNN theo chiều sâu, mỗi "kênh" (mỗi cột đặc trưng) chỉ được trộn *với chính nó* dọc theo 3 vị trí, nên mỗi kênh chỉ cần 3 trọng số: tổng cộng $4 \times 3 = 12$ tham số ($= d \cdot k$). Khi d lớn (ví dụ $d = 1024$), mức tiết kiệm là khoảng d lần.

2. Chú ý tuyến tính

Cũng như nhiều cách tiếp cận thực tế cho mô hình hóa chuỗi, có sự quan tâm đáng kể tới việc phát triển các mô hình tuyến tính nhằm tăng tốc xử lý chuỗi dài. Tuy có nhiều cách định nghĩa một mô hình tuyến tính, một dạng tổng quát thường được dùng trong các mô hình chuỗi là

$$\mathbf{z}_i \ = \ f(a \cdot \mathbf{z}_{i-1} + b \cdot \mathbf{s}_i). \tag{6.152}$$

Ở đây $\mathbf{s}_i$ biểu diễn một số trạng thái trung gian của mô hình tại bước i, và $\mathbf{z}_i$ biểu diễn bản tóm tắt các trạng thái quá khứ tính đến bước i. Dễ thấy đây là một mô hình hồi quy: đầu ra tại bước i chỉ phụ thuộc vào đầu vào ở bước hiện tại và đầu ra ở bước trước đó. Giống như các lựa chọn thiết kế phổ biến trong các hệ thống dựa trên mạng nơ-ron, phần tuyến tính được nối tiếp bởi một phép biến đổi $f(\cdot)$, có thể là một hàm kích hoạt hoặc một mạng nơ-ron truyền thẳng. Lưu ý rằng Phương trình (6.152) chỉ định nghĩa một mô hình tuyến tính chuẩn khi $f(\cdot)$ là hàm tuyến tính. Việc dùng $f(\cdot)$ mang lại sự linh hoạt lớn hơn trong mô hình hóa bài toán, mặc dù thuật ngữ *mô hình tuyến tính* có thể không còn áp dụng được nếu $f(\cdot)$ có dạng phi tuyến.

Công thức trên mô tả một mô hình có cấu trúc tuyến tính, có thể được xem như một trường hợp của một họ mô hình toán học tổng quát. Thông thường, nó có thể được biểu diễn như một cấu trúc chuỗi, hay một tập các nút có thứ tự. Mô hình lặp lại cùng một quá trình tính toán từ nút đầu tiên đến nút cuối cùng, mỗi lần lấy thông tin từ bước hiện tại và bước trước, rồi tạo ra một vectơ đầu ra được dùng ở các bước thời gian tiếp theo. Do đó, chi phí bộ nhớ và thời gian của mô hình tăng tuyến tính theo độ dài của chuỗi.

Ta có thể mở rộng Phương trình (6.152) thành một mô hình RNN chuẩn chỉ bằng cách áp dụng một phép biến đổi tuyến tính cho đầu vào hiện tại và trạng thái trước đó, tức là $\mathbf{z}_i = f(\mathbf{z}_{i-1} \cdot \mathbf{W}_z + \mathbf{s}_i \cdot \mathbf{W}_s)$. Do vậy, việc áp dụng RNN và các biến thể của nó vào Transformer để thu được một mô hình lai là khá đơn giản. Ví dụ, ta có thể dùng LSTM và GRU để xây dựng một số tầng Transformer, nhằm kết hợp ưu điểm của cả mô hình hồi quy lẫn mô hình tự chú ý [Chen và cộng sự, 2018b]. Vì các mô hình hồi quy truyền thống đã được bàn kỹ ở Chương 2, chúng tôi bỏ qua phần thảo luận về chúng ở đây.

Chú giải của người dịch: Mô hình "hồi quy" tuyến tính hiểu thế nào?

Phương trình (6.152) giống một *dãy số truy hồi* quen thuộc ở lớp 11: số hạng sau được tính từ số hạng trước. Chẳng hạn với $f(x) = x$, $a = 0.5$, $b = 1$ và các đầu vào vô hướng $s_1 = 2, s_2 = 4, s_3 = 6$, bắt đầu từ $z_0 = 0$: $z_1 = 2$, $z_2 = 0.5 \cdot 2 + 4 = 5$, $z_3 = 0.5 \cdot 5 + 6 = 8.5$. Mỗi bước chỉ cần nhớ *một* giá trị z_{i-1}, nên đi hết chuỗi dài n chỉ tốn n bước — chi phí tăng tuyến tính. Ngược lại, tự chú ý cho mỗi vị trí "nhìn" lại tất cả vị trí khác, nên cần khoảng $n \times n$ phép so sánh.

Thực ra, ta có thể quan tâm nhiều hơn tới việc phát triển các mô hình chú ý tuyến tính (*linear attention*), để có được một hệ thống hiệu quả mà vẫn giữ được lợi ích của việc mô hình hóa chuỗi bằng chú ý toàn cục. Một phần khó khăn khi làm điều này là dạng của tự chú ý không tuyến tính. Hãy dành một chút thời gian để xem khó khăn này nảy sinh như thế nào. Nhắc lại rằng kết quả của tự chú ý có thể

được viết dưới dạng sau

$$\begin{aligned}\text{Att}_{\text{self}} &= \mathbf{A}\cdot\mathbf{V}\\ &= \psi(\mathbf{Q}\cdot\mathbf{K}^\top)\cdot\mathbf{V}.\end{aligned} \tag{6.153}$$

Ở đây $\psi(\cdot)$ là một hàm được ghép từ các phép co giãn, lấy mũ, che (masking) và chuẩn hóa (tức là $\psi(\mathbf{a}) = \text{Normalize}(\text{Mask}(\exp(\frac{\mathbf{a}}{\sqrt{d}})))$). Vì $\psi(\cdot)$ là một hàm phi tuyến phức tạp, không có cách trực tiếp nào để đơn giản hóa phép tính, và ta thường phải tính riêng hai phép nhân ma trận (một bên trong $\psi(\cdot)$ và một bên ngoài $\psi(\cdot)$). Hệ quả là ta cần lưu trữ tường minh tất cả các cặp khóa–giá trị, và duyệt qua từng cặp với mỗi truy vấn. Không có gì ngạc nhiên khi điều này dẫn tới một mô hình có chi phí tính toán tăng theo bình phương độ dài chuỗi n.

Mặc dù trong tự chú ý, khóa và giá trị gắn liền với nhau, chúng được dùng ở các bước riêng biệt. Một giải pháp tinh tế là tách truy vấn khỏi tương tác khóa–giá trị, sao cho thông tin ngữ cảnh có thể được mã hóa độc lập với bất kỳ truy vấn cụ thể nào. Ý tưởng then chốt là bỏ qua hàm Softmax phi tuyến bằng cách áp dụng một ánh xạ đặc trưng (*feature map*) $\phi(\cdot)$ cho các truy vấn và khóa. Như đã giới thiệu ở Mục 6.4.3, ta biến đổi $\mathbf{Q}$ và $\mathbf{K}$ thành $\mathbf{Q}' = \phi(\mathbf{Q}) \in \mathbb{R}^{n\times d'}$ và $\mathbf{K}' = \phi(\mathbf{K}) \in \mathbb{R}^{n\times d'}$. Khi đó cơ chế chú ý có thể được viết lại thành:

$$\begin{aligned}\text{Att}_{\text{self}} &\equiv \psi'(\mathbf{Q}'\cdot\mathbf{K}'^\top)\cdot\mathbf{V}\\ &= \mathbf{D}^{-1}(\mathbf{Q}'\cdot\mathbf{K}'^\top)\cdot\mathbf{V}\\ &= \mathbf{D}^{-1}\Big(\mathbf{Q}'\cdot(\mathbf{K}'^\top\cdot\mathbf{V})\Big),\end{aligned} \tag{6.154}$$

trong đó $\mathbf{D}$ là ma trận chuẩn hóa dạng đường chéo, và $\psi'(\mathbf{Z}) = \mathbf{D}^{-1}\mathbf{Z}$[17]. Trong không gian đã biến đổi này, tích vô hướng truy vấn–khóa không cần đến chuẩn hóa Softmax dạng mũ. Nó tương ứng với việc chuẩn hóa bằng cách nhân với $\mathbf{D}^{-1}$. Vì phép toán cốt lõi giờ chỉ còn là phép nhân ma trận thuần túy, ta có thể khai thác tính kết hợp của phép nhân ma trận để thay đổi thứ tự tính toán.

Chú giải của người dịch: Đổi thứ tự nhân để tiết kiệm

Phép nhân ma trận có tính kết hợp: $(\mathbf{XY})\mathbf{Z} = \mathbf{X}(\mathbf{YZ})$, giống $(2\cdot 3)\cdot 4 = 2\cdot(3\cdot 4)$, nhưng *khối lượng tính toán* của hai cách có thể rất khác nhau. Với $\mathbf{Q}', \mathbf{K}'$ cỡ $n\times d'$ và $\mathbf{V}$ cỡ $n\times d$: tính $\mathbf{Q}'\mathbf{K}'^\top$ trước tạo ra ma trận $n\times n$ (khoảng $n^2 d'$ phép nhân), còn tính $\mathbf{K}'^\top\mathbf{V}$ trước chỉ tạo ra ma trận nhỏ $d'\times d$ (khoảng $nd'd$ phép nhân). Ví dụ $n = 10\,000$, $d = d' = 64$: cách đầu cần cỡ 6.4 tỷ phép nhân cho bước này, cách sau chỉ cỡ 41 triệu. Điều này chỉ làm được khi bỏ Softmax, vì Softmax "kẹp" giữa $\mathbf{Q}$ và $\mathbf{K}$ không cho ta đổi thứ tự.

Điều này dẫn tới một cách biểu diễn tự hồi quy mạnh mẽ: các khóa và giá trị trước hết được gộp lại qua $\mathbf{K}'^\top\cdot\mathbf{V}$, rồi các truy vấn chú ý tới biểu diễn toàn cục này. Vì $\mathbf{K}'^\top\cdot\mathbf{V} = \sum_{j=1}^{n}\mathbf{k}_j'^\top\cdot\mathbf{v}_j$, ta có thể viết $\mathbf{K}'^\top\cdot\mathbf{V}$ dưới dạng của Phương trình (6.152) như sau

$$\boldsymbol{\mu}_j \;=\; \boldsymbol{\mu}_{j-1} + \mathbf{k}_j'^\top\cdot\mathbf{v}_j. \tag{6.155}$$

Ở đây $\boldsymbol{\mu}_j \in \mathbb{R}^{d'\times d}$ là một biến được cộng dồn thêm $\mathbf{k}_j'^\top\cdot\mathbf{v}_j$ mỗi lần. Tương tự, ta có thể định nghĩa một biến khác $\boldsymbol{\nu}_j \in \mathbb{R}^{d'}$

$$\boldsymbol{\nu}_j \;=\; \boldsymbol{\nu}_{j-1} + \mathbf{k}_j'^\top. \tag{6.156}$$

Khi đó, đầu ra của tự chú ý cho truy vấn thứ j có thể được viết là (xem thêm Phương trình (6.145))

$$\text{Att}_{\text{self},j} \;=\; \frac{\mathbf{q}_j'\cdot\boldsymbol{\mu}_n}{\mathbf{q}_j'\cdot\boldsymbol{\nu}_n}. \tag{6.157}$$

[17] Nếu $\mathbf{D}$ là ma trận đường chéo, đôi khi chúng tôi ký hiệu $\mathbf{D}^{-1}\mathbf{A}$ là $\frac{\mathbf{A}}{\mathbf{D}}$.

Rõ ràng, cách này tạo ra một mô hình có thời gian tuyến tính, vì $\boldsymbol{\mu}_n$ và $\boldsymbol{\nu}_n$ là tuyến tính theo n. Trong các cài đặt đơn giản của mô hình này, chỉ $\boldsymbol{\mu}_j$ và $\boldsymbol{\nu}_j$ được lưu giữ. Mỗi khi gặp một truy vấn mới, ta cập nhật $\boldsymbol{\mu}_j$ và $\boldsymbol{\nu}_j$ bằng Phương trình (6.155) và (6.156), rồi tính $\text{Att}_{\text{self},j} = \frac{\mathbf{q}'_j \cdot \boldsymbol{\mu}_j}{\mathbf{q}'_j \cdot \boldsymbol{\nu}_j}$[18].

Chú giải của người dịch: Ý nghĩa của $\boldsymbol{\mu}_j$ và $\boldsymbol{\nu}_j$

Hãy coi $\boldsymbol{\mu}_j$ là một "cuốn sổ" kích thước cố định ($d' \times d$ số) ghi tổng dồn mọi cặp khóa–giá trị đã gặp, còn $\boldsymbol{\nu}_j$ ghi tổng dồn các khóa (dùng để chuẩn hóa, giống như chia cho tổng trọng số khi tính trung bình có trọng số). Khi một từ mới đến, ta chỉ *cộng thêm* một mục vào sổ, không phải đọc lại toàn bộ lịch sử. Tử số $\mathbf{q}'_j \cdot \boldsymbol{\mu}_j$ là "tổng có trọng số của các giá trị", mẫu số $\mathbf{q}'_j \cdot \boldsymbol{\nu}_j$ là "tổng các trọng số"; thương của chúng là một trung bình có trọng số — đúng tinh thần của cơ chế chú ý, nhưng với chi phí mỗi bước không đổi.

Một mở rộng đơn giản của mô hình chú ý tuyến tính là cho phép Phương trình (6.155) và (6.156) kết hợp các số hạng khác nhau với trọng số khác nhau. Ví dụ, ta có thể định nghĩa lại $\boldsymbol{\mu}_j$ và $\boldsymbol{\nu}_j$ là

$$\boldsymbol{\mu}_j = a \cdot \boldsymbol{\mu}_{j-1} + (1-a) \cdot \mathbf{k}'^{\top}_j \cdot \mathbf{v}_j, \tag{6.158}$$

$$\boldsymbol{\nu}_j = a \cdot \boldsymbol{\nu}_{j-1} + (1-a) \cdot \mathbf{k}'^{\top}_j \tag{6.159}$$

và huấn luyện tham số a như thường lệ. Ngoài ra, ta có thể coi a như một cổng và dùng một mạng nơ-ron khác để tính a [Peng và cộng sự, 2021]. Một thiết kế mô hình khác là thêm nhiều số hạng hơn vào Phương trình (6.155) và (6.156) để có một công thức biểu đạt mạnh hơn cho cách tiếp cận chú ý tuyến tính [Bello, 2020; Schlag và cộng sự, 2021].

Ta đã thấy ý tưởng chung của việc thiết kế các mô hình tuyến tính cho cơ chế chú ý. Lựa chọn thiết kế then chốt của những mô hình như vậy là loại bỏ phép chuẩn hóa dựa trên Softmax, nhờ đó thu được các dạng biểu diễn tuyến tính dựa trên những trạng thái trung gian khác nhau của mô hình. Điều này thúc đẩy một số phương án thay thế tự chú ý được phát triển gần đây, trong đó các hệ thống suy luận hiệu quả được xây dựng trên nền các mô hình hồi quy cho mô hình hóa chuỗi [Peng và cộng sự, 2023; Sun và cộng sự, 2023]. Mặc dù các hệ thống này có kiến trúc khác nhau, các mô hình bên dưới có dạng tương tự nhau, như mô tả trong Phương trình (6.152). Lưu ý rằng khi dùng công thức tổng quát của các mô hình hồi quy, ta không bị giới hạn trong cơ chế chú ý QKV chuẩn. Thay vào đó, ta có thể gán ý nghĩa và hình thức mới cho các truy vấn, khóa và giá trị.

Phần thảo luận ở đây cũng liên quan đến các mô hình bộ nhớ đã bàn ở Mục 6.4.2. Từ góc độ bộ nhớ, các khóa và giá trị có thể được xem như các mã hóa của ngữ cảnh. Vì vậy, trong mô hình chú ý tuyến tính ở trên, ta có một hệ thống bộ nhớ trong đó hai biến đơn giản $\boldsymbol{\mu}_j$ và $\boldsymbol{\nu}_j$ được dùng để biểu diễn toàn bộ thông tin ngữ cảnh tính đến vị trí j. Điều này tạo ra một bộ nhớ có độ dài cố định, rất hữu ích trong thực tế. Cũng có những cách tiếp cận tuyến tính khác để mã hóa chuỗi dài. Chẳng hạn, ta có thể xem mô hình **trung bình trượt** (*moving average*) như một trường hợp của Phương trình (6.152), và lấy trung bình (có trọng số hoặc không) một dãy các vectơ trạng thái của mô hình Transformer.

3. Mô hình không gian trạng thái

Trong các hệ thống điều khiển, **mô hình không gian trạng thái** (*state-space model*) (**SSM**) là cách biểu diễn một hệ thống mà đầu vào và đầu ra liên hệ với nhau qua một số **biến trạng thái** (*state variable*) (gọi tắt là *trạng thái*), và động lực học của hệ được mô tả bằng các phương trình vi phân bậc nhất của các trạng thái này. Để lấy ví dụ đơn giản, ta xét một hệ tuyến tính bất biến theo thời gian, liên tục theo thời gian, được cho dưới dạng biểu diễn không gian trạng thái. Trong suốt mục này, ta dùng quy ước

[18] Trong sinh văn bản tự hồi quy, ta sinh chuỗi từ trái sang phải. Trong trường hợp này, ta không cần xét các khóa và giá trị ở các vị trí $> j$.

vectơ hàng: mọi biến đều được biểu diễn dưới dạng vectơ hàng và các phép biến đổi tuyến tính được áp dụng ở bên phải:

$$\frac{d\mathbf{z}(t)}{dt} = \mathbf{z}(t) \cdot \mathbf{A} + \mathbf{s}(t) \cdot \mathbf{B}, \tag{6.160}$$

$$\mathbf{o}(t) = \mathbf{z}(t) \cdot \mathbf{C} + \mathbf{s}(t) \cdot \mathbf{D}. \tag{6.161}$$

Ở đây, $\mathbf{s}(t)$, $\mathbf{o}(t)$ và $\mathbf{z}(t)$ lần lượt biểu diễn giá trị của biến đầu vào, biến đầu ra và biến trạng thái tại thời điểm t[19]. Trong trường hợp tổng quát, các biến này có thể có số chiều khác nhau. Để đơn giản, ta giả sử $\mathbf{s}(t), \mathbf{o}(t) \in \mathbb{R}^d$ và $\mathbf{z}(t) \in \mathbb{R}^{d_z}$[20]. Phương trình (6.160) được gọi là **phương trình trạng thái** (*state equation*), trong đó $\mathbf{A} \in \mathbb{R}^{d_z \times d_z}$ là ma trận trạng thái và $\mathbf{B} \in \mathbb{R}^{d \times d_z}$ là ma trận đầu vào. Phương trình (6.161) được gọi là **phương trình đầu ra** (*output equation*), trong đó $\mathbf{C} \in \mathbb{R}^{d_z \times d}$ là ma trận đầu ra và $\mathbf{D} \in \mathbb{R}^{d \times d}$ là ma trận truyền thẳng.

Chú giải của người dịch: Phương trình vi phân $\frac{d\mathbf{z}}{dt}$ nói gì?

$\frac{d\mathbf{z}(t)}{dt}$ là đạo hàm của trạng thái theo thời gian, tức *tốc độ thay đổi* của trạng thái. Phương trình (6.160) nói rằng trạng thái thay đổi nhanh hay chậm tùy vào trạng thái hiện tại (qua $\mathbf{A}$) và đầu vào hiện tại (qua $\mathbf{B}$). Ví dụ đời thường, một chiều: lượng nước z trong bể có lỗ rò, vòi bơm vào với lưu lượng $s(t)$: $\frac{dz}{dt} = -0.1\, z + s(t)$ (nước càng nhiều thì rò càng nhanh). "Trạng thái" z chính là *trí nhớ* của hệ về toàn bộ lịch sử đầu vào. Đầu ra (Phương trình (6.161)) là thứ ta quan sát được, tính từ trạng thái và đầu vào.

Các phương trình này mô tả một ánh xạ liên tục từ đầu vào $\mathbf{s}(t)$ đến đầu ra $\mathbf{o}(t)$ theo thời gian. Vì vậy chúng thường được dùng để xử lý dữ liệu chuỗi thời gian liên tục. Để áp dụng mô hình này cho mô hình hóa chuỗi, ta cần sửa các phương trình trên để có dạng rời rạc của biểu diễn không gian trạng thái. Giả sử $\{\mathbf{s}_0, \mathbf{s}_1, \cdots, \mathbf{s}_n\}$ là một chuỗi các điểm dữ liệu đầu vào được lấy mẫu từ $\mathbf{s}(t)$ với bước thời gian Δt. Tương tự, ta định nghĩa $\{\mathbf{z}_0, \mathbf{z}_1, \cdots, \mathbf{z}_n\}$ và $\{\mathbf{o}_0, \mathbf{o}_1, \cdots, \mathbf{o}_n\}$ là các chuỗi vectơ trạng thái và vectơ đầu ra. Với ký hiệu này, ta có một phiên bản rời rạc hóa của SSM, được viết là

$$\mathbf{z}_t = \mathbf{z}_{t-1} \cdot \overline{\mathbf{A}} + \mathbf{s}_t \cdot \overline{\mathbf{B}}, \tag{6.162}$$

$$\mathbf{o}_t = \mathbf{z}_t \cdot \overline{\mathbf{C}} + \mathbf{s}_t \cdot \overline{\mathbf{D}}. \tag{6.163}$$

Công thức SSM này định nghĩa một RNN có kết nối tắt (residual). Nói chính xác hơn, Phương trình (6.162) mô tả một đơn vị hồi quy đọc đầu vào tại bước t và trạng thái tại bước $t-1$, mà không dùng bất kỳ hàm kích hoạt nào. Phương trình (6.163) mô tả một tầng đầu ra cộng các phép biến đổi tuyến tính của trạng thái $\mathbf{z}_t$ và ánh xạ tắt $\mathbf{s}_t$.

Các tham số $\overline{\mathbf{A}}$, $\overline{\mathbf{B}}$, $\overline{\mathbf{C}}$ và $\overline{\mathbf{D}}$ có thể được suy ra từ $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$ và $\mathbf{D}$ theo nhiều cách khác nhau, tùy vào việc Phương trình (6.160) được xấp xỉ bằng Phương trình (6.162) như thế nào[21]. Một cách rời rạc hóa theo thời gian là **biến đổi song tuyến tính** (*bilinear transform*) (còn gọi là phương pháp Tustin). Một

[19] Chúng tôi dùng chữ in đậm để nhấn mạnh rằng các biến này là vectơ.

[20] Trong lý thuyết không gian trạng thái tổng quát, các biến và ma trận này được định nghĩa trên trường số phức. Vì các mô hình định nghĩa trên trường số phức bao hàm cả không gian trạng thái giá trị thực, để đơn giản chúng tôi giới hạn thảo luận trong trường số thực nhiều chiều.

[21] Quá trình rời rạc hóa có thể được hiểu là giải số phương trình vi phân bên dưới. Lưu ý rằng Phương trình (6.160) là một phương trình vi phân thường (ODE):

$$\frac{d\mathbf{z}(t)}{dt} = g(\mathbf{z}(t), t), \tag{6.164}$$

trong đó

$$g(\mathbf{z}(t), t) = \mathbf{z}(t) \cdot \mathbf{A} + \mathbf{s}(t) \cdot \mathbf{B}. \tag{6.165}$$

Có nhiều phương pháp số để xấp xỉ nghiệm của một ODE. Ví dụ, phương pháp Euler để giải ODE có thể được biểu diễn là (xem

cách khác là dùng phép rời rạc hóa **giữ bậc không** (*Zero-Order-Hold*) (**ZOH**). Thảo luận chi tiết về các cách tiếp cận này nằm ngoài phạm vi của cuốn sách, và chúng tôi giới thiệu bạn đọc quan tâm tới các giáo trình chuẩn về lý thuyết điều khiển để biết thêm chi tiết [Åström và Wittenmark, 2013].

Chú giải của người dịch: Rời rạc hóa bằng phương pháp Euler

Ý tưởng giống hệt định nghĩa đạo hàm ở lớp 11: với Δt nhỏ, $\frac{dz}{dt} \approx \frac{z_t - z_{t-1}}{\Delta t}$, nên $z_t \approx z_{t-1} + \Delta t \cdot \frac{dz}{dt}$. Với ví dụ bể nước một chiều $\frac{dz}{dt} = -0.1z + s$ và $\Delta t = 1$: $z_t = z_{t-1}(1 - 0.1) + 1 \cdot s_t$, tức $\overline{A} = 0.9$, $\overline{B} = 1$. Vậy hệ liên tục đã biến thành một công thức truy hồi từng bước — đúng dạng RNN tuyến tính ở Phương trình (6.162). Ma trận $\mathbf{I}$ trong Phương trình (6.168) là *ma trận đơn vị*, đóng vai trò như số 1 trong phép nhân ma trận.

Dạng hồi quy của Phương trình (6.162) giúp dễ dàng tính các trạng thái và đầu ra trên một chuỗi các bước thời gian rời rạc. Ta có thể trải $\mathbf{z}_t$ và $\mathbf{o}_t$ ra theo kiểu truyền thẳng

$$
\begin{array}{l|l}
\mathbf{z}_0 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} & \mathbf{o}_0 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} + \mathbf{s}_0 \cdot \overline{\mathbf{D}} \\
\mathbf{z}_1 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}} + \mathbf{s}_1 \cdot \overline{\mathbf{B}} & \mathbf{o}_1 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}} \cdot \overline{\mathbf{C}} + \mathbf{s}_1 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} + \mathbf{s}_1 \cdot \overline{\mathbf{D}} \\
\mathbf{z}_2 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^2 + \mathbf{s}_1 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}} + \mathbf{s}_2 \cdot \overline{\mathbf{B}} & \mathbf{o}_2 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^2 \cdot \overline{\mathbf{C}} + \mathbf{s}_1 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}} \cdot \overline{\mathbf{C}} + \\
 & \quad \mathbf{s}_2 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} + \mathbf{s}_2 \cdot \overline{\mathbf{D}} \\
\ldots\ldots & \ldots\ldots
\end{array}
$$

Dễ dàng viết được

$$\mathbf{z}_t = \sum_{i=0}^{t} \mathbf{s}_i \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{t-i}, \tag{6.170}$$

$$\mathbf{o}_t = \sum_{i=0}^{t} \mathbf{s}_i \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{t-i} \cdot \overline{\mathbf{C}} + \mathbf{s}_t \cdot \overline{\mathbf{D}}. \tag{6.171}$$

Rõ ràng, vế phải của Phương trình (6.171) có thể được hiểu là tổng của một số hạng tích chập và một số hạng tuyến tính. Vì

$$\sum_{i=0}^{t} \mathbf{s}_i \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{t-i} \cdot \overline{\mathbf{C}} = \begin{bmatrix} \mathbf{s}_0 & \mathbf{s}_1 & \cdots & \mathbf{s}_t \end{bmatrix} \cdot \begin{bmatrix} \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^t \cdot \overline{\mathbf{C}} & \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{t-1} \cdot \overline{\mathbf{C}} & \cdots & \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} \end{bmatrix}^\top, \tag{6.172}$$

ta định nghĩa một bộ lọc tích chập với các tham số

$$\mathbf{W}_{\text{ssm}} = \begin{bmatrix} \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{n_{\max}} \cdot \overline{\mathbf{C}} & \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{n_{\max}-1} \cdot \overline{\mathbf{C}} & \cdots & \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} \end{bmatrix}, \tag{6.173}$$

Mục 6.3.3)

$$\mathbf{z}_t = \mathbf{z}_{t-1} + \Delta t \cdot g(\mathbf{z}_{t-1}, t). \tag{6.166}$$

Thay Phương trình (6.165) vào Phương trình (6.166), ta được

$$\begin{aligned} \mathbf{z}_t &= \mathbf{z}_{t-1} + \Delta t(\mathbf{z}_{t-1} \cdot \mathbf{A} + \mathbf{s}_t \cdot \mathbf{B}) \\ &= \mathbf{z}_{t-1} \cdot (\mathbf{I} + \Delta t \cdot \mathbf{A}) + \mathbf{s}_t \cdot (\Delta t \cdot \mathbf{B}). \end{aligned} \tag{6.167}$$

Điều này cho ta một trong những dạng đơn giản nhất của phương trình trạng thái rời rạc hóa [Gu và cộng sự, 2022b], tức là

$$\overline{\mathbf{A}} = \mathbf{I} + \Delta t \cdot \mathbf{A}, \tag{6.168}$$

$$\overline{\mathbf{B}} = \Delta t \cdot \mathbf{B} \tag{6.169}$$

trong đó n_{max} là độ dài tối đa của chuỗi[22]. Khi đó, đầu ra của mô hình không gian trạng thái cho một chuỗi $\mathbf{S} = \begin{bmatrix} \mathbf{s}_0 \\ \vdots \\ \mathbf{s}_n \end{bmatrix}$ có thể được biểu diễn là

$$\mathbf{O} \;=\; \text{Conv}(\mathbf{S}, \mathbf{W}_{\text{ssm}}) + \text{Linear}(\mathbf{S}, \overline{\mathbf{D}}), \tag{6.174}$$

trong đó $\text{Conv}(\cdot)$ là phép tích chập, và $\text{Linear}(\cdot)$ là phép biến đổi tuyến tính. Cách xử lý mô hình không gian trạng thái như vậy cho phép cài đặt hệ thống một cách hiệu quả bằng các thuật toán tích chập song song nhanh.

Chú giải của người dịch: Một mô hình, hai cách tính

Xét trường hợp một chiều với $\overline{A} = 0.5$, $\overline{B} = \overline{C} = 1$, $\overline{D} = 0$ và đầu vào $s_0 = 4, s_1 = 2, s_2 = 6$. *Cách hồi quy* (từng bước): $z_0 = 4$, $z_1 = 0.5 \cdot 4 + 2 = 4$, $z_2 = 0.5 \cdot 4 + 6 = 8$. *Cách tích chập* (Phương trình (6.170)): $z_2 = s_0 \cdot 0.5^2 + s_1 \cdot 0.5 + s_2 = 1 + 1 + 6 = 8$ — cùng kết quả. Cách thứ hai cho thấy mỗi đầu ra là tổng có trọng số của các đầu vào với "bộ lọc" $[0.25,\ 0.5,\ 1]$ cố định, nên mọi vị trí có thể tính *đồng thời* (song song) khi huấn luyện; còn cách thứ nhất chỉ cần nhớ một trạng thái, rất tiện khi sinh từng token lúc suy luận.

Tuy nhiên, các cài đặt ngây thơ của mô hình này thường hoạt động kém. Như với nhiều kiến trúc mạng nơ-ron sâu khác, việc khởi tạo tham số cẩn thận là rất quan trọng. Cụ thể, việc giới hạn ma trận trạng thái $\mathbf{A}$ vào những cấu trúc đặc biệt (ví dụ ma trận HiPPO) đã được chứng minh là thiết yếu để ghi nhớ và tổng quát hóa trên các chuỗi dài [Gu và cộng sự, 2022a].

Một nút thắt tính toán khác là phép nhân ma trận lặp lại cần thiết để tính $\overline{\mathbf{A}}^n$. Khi n lớn, phép toán này tốn kém về tính toán và không ổn định về mặt số học. Một cách tiếp cận phổ biến trong các SSM hiện đại là **chéo hóa** (*diagonalization*). Ý tưởng là áp dụng một phép đổi cơ sở cho không gian trạng thái sao cho $\mathbf{A}$ (hoặc $\overline{\mathbf{A}}$) trở thành ma trận đường chéo. Cho một SSM được tham số hóa bởi $(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})$, ta có thể định nghĩa một mô hình tương đương $(\mathbf{UAU}^{-1}, \mathbf{BU}^{-1}, \mathbf{UC}, \mathbf{D})$ thông qua một ma trận biến đổi khả nghịch $\mathbf{U}$. Có thể chứng minh rằng hai cách biểu diễn này tương đương về mặt toán học dưới một phép đổi cơ sở[23].

Bằng cách áp dụng phép biến đổi này, và giả sử $\overline{\mathbf{A}}$ có thể được chéo hóa về dạng chính tắc $\mathbf{P}^{-1}\Lambda\mathbf{P}$ (trong đó Λ là ma trận đường chéo), ta áp đặt ràng buộc rằng ma trận trạng thái là đường chéo, từ đó có các **mô hình không gian trạng thái đường chéo** (*diagonal state-space model*). Để minh họa, hãy xét bộ lọc được dùng trong biểu diễn tích chập (Phương trình (6.172)). Thay $\overline{\mathbf{A}} = \mathbf{P}^{-1}\Lambda\mathbf{P}$, ta có thể viết lại số hạng $\overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^t \cdot \overline{\mathbf{C}}$ thành:

$$\begin{aligned} \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^t \cdot \overline{\mathbf{C}} &= \overline{\mathbf{B}} \cdot (\mathbf{P}^{-1}\Lambda\mathbf{P})^t \cdot \overline{\mathbf{C}} \\ &= \overline{\mathbf{B}} \cdot (\mathbf{P}^{-1}\Lambda\mathbf{P}) \cdot (\mathbf{P}^{-1}\Lambda\mathbf{P}) \cdots (\mathbf{P}^{-1}\Lambda\mathbf{P}) \cdot \overline{\mathbf{C}} \\ &= (\overline{\mathbf{B}} \cdot \mathbf{P}^{-1}) \cdot \Lambda^t \cdot (\mathbf{P} \cdot \overline{\mathbf{C}}). \end{aligned} \tag{6.175}$$

Vì Λ là ma trận đường chéo, ta có thể tính Λ^t một cách hiệu quả bằng cách đơn giản nâng tất cả các phần tử của Λ lên lũy thừa t. Khi đó ta có một mô hình rẻ hơn về mặt tính toán, trong đó

$$\overline{\mathbf{A}}' = \Lambda, \tag{6.176}$$

$$\overline{\mathbf{B}}' = \overline{\mathbf{B}} \cdot \mathbf{P}^{-1}, \tag{6.177}$$

[22] Ở đây $\mathbf{W}_{\text{ssm}}$ có thể được biểu diễn dưới dạng một tensor kích thước $n_{\text{max}} \times d \times d$.

[23] Một phép biến đổi không gian trạng thái ánh xạ các biến trạng thái vào một hệ tọa độ mới: $\mathbf{z}'(t) = \mathbf{z}(t) \cdot \mathbf{U}^{-1}$.

$$\overline{\mathbf{C}}' = \mathbf{P} \cdot \overline{\mathbf{C}}, \tag{6.178}$$

$$\overline{\mathbf{D}}' = \overline{\mathbf{D}}. \tag{6.179}$$

Các thảo luận chi tiết hơn về mô hình không gian trạng thái đường chéo trong mô hình hóa chuỗi có thể được tìm thấy trong công trình của Gu và cộng sự [2021].

Chú giải của người dịch: Vì sao ma trận đường chéo dễ lũy thừa?

Ma trận đường chéo chỉ có số khác 0 trên đường chéo chính. Khi nhân hai ma trận đường chéo, ta chỉ việc nhân các phần tử tương ứng trên đường chéo, nên $\begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}^t = \begin{bmatrix} 2^t & 0 \\ 0 & 3^t \end{bmatrix}$ — chỉ cần d_z phép lũy thừa số thông thường, thay vì nhân ma trận đầy đủ t lần. Ở dòng thứ hai của Phương trình (6.175), các cặp $\mathbf{P}\mathbf{P}^{-1}$ đứng cạnh nhau triệt tiêu (bằng ma trận đơn vị), chỉ còn $\mathbf{P}^{-1}\Lambda^t\mathbf{P}$.

Việc áp dụng mô hình không gian trạng thái vào Transformer khá đơn giản. Trong trường hợp này, mỗi tầng con tự chú ý được thay bằng một tầng con SSM như mô tả trong Phương trình (6.162) và (6.163). Như ta đã thấy, có mối liên hệ chặt chẽ giữa mô hình không gian trạng thái với cả CNN và RNN. Đối với mô hình hóa chuỗi, ta có thể xử lý một chuỗi token hoặc tuần tự như trong RNN, hoặc song song như trong CNN. Điều này dẫn tới một mô thức mới kết hợp cả góc nhìn tuần tự lẫn góc nhìn song song đối với bài toán mô hình hóa chuỗi — khi huấn luyện, hệ thống hoạt động như CNN để tận dụng các thuật toán huấn luyện song song nhanh; khi dự đoán, bài toán được chuyển thành bài toán cập nhật tuần tự, có thể giải hiệu quả bằng các mô hình kiểu RNN.

Mặc dù hình thức luận của mô hình không gian trạng thái khác với những gì ta đã thảo luận trong chương này, nó cung cấp một khuôn khổ thống nhất cho mô hình hóa chuỗi. Bằng cách nhìn bài toán qua các góc nhìn bổ trợ nhau (song song và tuần tự), ta có thể tối ưu cho các ràng buộc khác nhau về phần cứng và độ trễ. Một số kiến trúc gần đây được thúc đẩy bởi tính đối ngẫu này, tạo ra các hệ thống kết hợp hiệu quả khả năng song song hóa của Transformer với hiệu quả suy luận của RNN [Orvieto và cộng sự, 2023; Sun và cộng sự, 2023].

6.4.6 Tính toán có điều kiện

Cho đến giờ, trong thảo luận về các mô hình Transformer hiệu quả, ta đã giả định rằng kiến trúc mô hình được xác định trước khi huấn luyện và giữ cố định trong suốt quá trình. Bây giờ ta chuyển sang trường hợp học các kiến trúc mô hình hiệu quả. Ta có thể viết một mô hình dưới dạng

$$\mathbf{y} \;=\; \text{Model}(\mathbf{x}, g(\mathbf{x})), \tag{6.180}$$

trong đó $\mathbf{x}$ và $\mathbf{y}$ là đầu vào và đầu ra của mô hình. $g(\mathbf{x})$ là một hàm trả về kiến trúc mô hình và các tham số tương ứng với đầu vào $\mathbf{x}$ cho trước. Nói chung, ta theo quy ước phổ biến trong các bài toán học là dùng một kiến trúc mô hình cố định và chỉ học các tham số, chẳng hạn $g(\mathbf{x}) = \theta$. Trong trường hợp này, mục tiêu là học các giá trị tham số tối ưu khi đã biết kiến trúc và dữ liệu huấn luyện. Trên dữ liệu kiểm tra, ta đưa ra dự đoán bằng cùng kiến trúc mô hình đó cùng với các tham số đã được tối ưu.

Một mở rộng tự nhiên của cách tiếp cận này là xét việc học cả kiến trúc mô hình lẫn tham số. Trong học kiến trúc, ta muốn tìm một hàm $\hat{g}(\mathbf{x})$ tạo ra kiến trúc mô hình và giá trị tham số tối ưu cho đầu vào $\mathbf{x}$. Tuy nhiên, việc tìm kiếm trên toàn bộ không gian giả thuyết gồm mọi tổ hợp kiến trúc và tham số có thể có là bất khả thi, nên cần đến các phương pháp thực tế. Có hai lớp phương pháp có thể áp dụng:

- **Tìm kiếm kiến trúc mạng nơ-ron** (*Neural Architecture Search*) (**NAS**). Trong **học máy tự động** (*automated machine learning*) (**AutoML**), tìm kiếm kiến trúc mạng nơ-ron là quá trình khám phá

một không gian các mạng nơ-ron để tìm ra mạng phù hợp nhất với một số tiêu chí [Zoph và Le, 2016; Elsken và cộng sự, 2019b]. Khi đã xác định được mạng nơ-ron tối ưu, các tham số của nó được huấn luyện như thường lệ, rồi áp dụng cho dữ liệu mới. Để việc tìm kiếm khả thi, người ta thường dùng thêm một số kỹ thuật, chẳng hạn cắt tỉa không gian tìm kiếm và các thuật toán tìm kiếm nhanh. Việc áp dụng tìm kiếm kiến trúc mạng nơ-ron vào phát triển các mạng nơ-ron hiệu quả khá đơn giản [Howard và cộng sự, 2019; Tan và Le, 2019]. Ta chỉ cần đưa các thước đo hiệu quả vào việc ước lượng hiệu năng của mạng nơ-ron; ví dụ, việc tìm kiếm có thể được dẫn dắt bởi một tiêu chí phạt những mạng nơ-ron có độ trễ cao hoặc đòi hỏi bộ nhớ quá lớn.

- **Mạng nơ-ron động** (*Dynamic Neural Networks*). Ý tưởng chính của mạng nơ-ron động là điều chỉnh mạng nơ-ron một cách linh hoạt theo các đầu vào khác nhau [Gupta và cộng sự, 2004; Han và cộng sự, 2021b]. Lý tưởng nhất, ta muốn học $\hat{g}(\cdot)$, rồi với mọi đầu vào $\mathbf{x}_{\text{new}}$, ta áp dụng mô hình $\text{Model}(\mathbf{x}_{\text{new}}, \hat{g}(\mathbf{x}_{\text{new}}))$. Kết quả là, ở thời điểm kiểm tra, ta có thể có cấu trúc mô hình và/hoặc tham số khác nhau cho các đầu vào khác nhau. Tuy nhiên, không khả thi để xây dựng một hàm $\hat{g}(\cdot)$ có thể mô hình hóa các mạng nơ-ron tùy ý. Trong thực tế, $\hat{g}(\cdot)$ thường được xem là biểu diễn một họ các mạng con của một siêu mạng. Do đó, bài toán được đặt lại thành một bài toán đơn giản hơn: học cách chọn mạng con nào được dùng cho một đầu vào cho trước.

Từ góc độ học máy, các phương pháp tìm kiếm kiến trúc mạng nơ-ron mang tính tổng quát và có thể áp dụng cho bất kỳ mạng nơ-ron nào. Mặt khác, từ góc độ thực tiễn, vẫn khó tìm được một mạng nơ-ron hiệu quả mà đủ mạnh và tổng quát hóa tốt. Mặc dù tìm kiếm kiến trúc mạng nơ-ron mang lại những ý tưởng thú vị cho việc phát triển các mô hình Transformer hiệu quả, chúng tôi không bàn sâu về nó ở đây. Thay vào đó, bạn đọc có thể tham khảo các bài báo trên để có ý niệm chung, và tham khảo các công trình của So và cộng sự [2019], Wang và cộng sự [2020a], và Hu và cộng sự [2021] về ứng dụng của nó cho Transformer.

Trong tiểu mục này, chúng tôi tập trung vào một họ phương pháp cụ thể của mạng nơ-ron động, gọi là **tính toán có điều kiện** (*conditional computation*). Khái niệm này ban đầu xuất phát từ việc lựa chọn động các nơ-ron của một mạng nơ-ron [Bengio và cộng sự, 2013; 2015]. Gần đây hơn, nó thường được dùng để chỉ quá trình lựa chọn động các phần của một mạng nơ-ron. Một cách nhìn hẹp về tính toán có điều kiện là coi $g(\cdot)$ như một mạng nơ-ron thích nghi, có thể giảm hoặc tăng số đơn vị tính toán (như nơ-ron và tầng) một cách linh hoạt. Nhờ đó, việc tính toán có thể thích ứng với các điều kiện thay đổi, và ta có thể tìm một sự đánh đổi tốt giữa độ chính xác và độ trễ thông qua cơ chế thích nghi này.

Một cách phổ biến để đạt được điều này là học cách bỏ qua một số bước tính toán, để ta chỉ làm việc với một tập con của mạng [Xu và Mcauley, 2023]. Một trong những phương pháp đơn giản nhất, đôi khi được gọi là **thoát sớm** (*early exiting*), là dừng tính toán tại một thời điểm nào đó trong khi mã hóa hoặc giải mã một chuỗi. Kỹ thuật này thường được dùng trong các ứng dụng sinh chuỗi thực tế đòi hỏi độ trễ thấp. Giả sử $y_1 \cdots y_{n_{\max}}$ là chuỗi dài nhất mà hệ thống có thể sinh ra, và $\mathbf{s}_1 \cdots \mathbf{s}_{n_{\max}}$ là chuỗi trạng thái ẩn tương ứng của tầng Transformer trên cùng. Khi đó ta xây dựng một mô hình $f_{\text{stop}}(\cdot)$ nhận lần lượt từng trạng thái ẩn $\mathbf{s}_i$ và tạo ra phân phối của một biến nhị phân $c \in \{\text{stop}, \text{nonstop}\}$ (dừng, không dừng):

$$\Pr(c|\mathbf{s}_i) \;=\; f_{\text{stop}}(\mathbf{s}_i). \tag{6.181}$$

Quá trình sinh kết thúc nếu $\Pr(\text{stop}|\mathbf{s}_i)$ đủ lớn, chẳng hạn

$$\Pr(\text{stop}|\mathbf{s}_i) \;\geq\; \Pr(\text{nonstop}|\mathbf{s}_i) + \theta_{\text{stop}}, \tag{6.182}$$

trong đó θ_{stop} là biên tối thiểu để phân biệt hai hành động[24]. Công thức này cũng liên quan đến bài toán

[24] Một dạng tương đương của Phương trình (6.182) là $\Pr(\text{stop}|\mathbf{s}_i) \geq \frac{1+\theta_{\text{stop}}}{2}$.

tiêu chí dừng thường được bàn đến trong các thuật toán tìm kiếm cho sinh chuỗi (xem Chương 5). Hàm $f_{\text{stop}}(\cdot)$ có thể được thiết kế theo nhiều cách khác nhau. Ví dụ, trong nhiều ứng dụng thực tế, tiêu chí dừng dựa trên các quy tắc kinh nghiệm đơn giản. Ngoài ra, ta có thể định nghĩa hàm $f_{\text{stop}}(\cdot)$ là một mạng nơ-ron và huấn luyện nó bằng dữ liệu có nhãn.

Chú giải của người dịch: Biên θ_{stop}

Vì chỉ có hai khả năng nên $\Pr(\text{stop}) + \Pr(\text{nonstop}) = 1$. Thay $\Pr(\text{nonstop}) = 1 - \Pr(\text{stop})$ vào Phương trình (6.182) ta được $2\Pr(\text{stop}) \geq 1 + \theta_{\text{stop}}$, chính là dạng trong chú thích chân trang. Ví dụ $\theta_{\text{stop}} = 0.4$ thì mô hình chỉ dừng khi $\Pr(\text{stop}) \geq 0.7$: biên càng lớn, mô hình càng "thận trọng", chỉ dừng khi thật sự chắc chắn.

Cách tiếp cận trên có thể dễ dàng mở rộng cho những tình huống trong đó một số token bị bỏ qua. Cách tiếp cận *học cách bỏ qua* này thường được dùng ở giai đoạn mã hóa, khi mọi token đầu vào đều đã có sẵn từ trước. Gọi $\mathbf{h}_1 \cdots \mathbf{h}_m$ là các biểu diễn mức thấp của chuỗi $x_1 \cdots x_m$. Tương tự Phương trình (6.181), ta có thể xây dựng một mô hình $\Pr(c|\mathbf{s}_i)$ ($c \in \{\text{skip}, \text{nonskip}\}$, tức bỏ qua/không bỏ qua) để xác định xem token x_i có thể bị bỏ qua hay không. Hình 6.17 (a) và (b) minh họa việc thoát sớm và bỏ qua token. Lưu ý rằng phương pháp học cách bỏ qua có phần trùng lặp với các hướng nghiên cứu khác về huấn luyện mạng nơ-ron. Ví dụ, việc xóa bớt một số token đầu vào khi huấn luyện được thấy là hữu ích để giúp mô hình Transformer tổng quát hóa tốt hơn [Shen và cộng sự, 2020a; Kim và Cho, 2021]. Phương pháp này cũng liên quan đến các phương pháp giảm mẫu sẽ được thảo luận ở Mục 6.4.8.

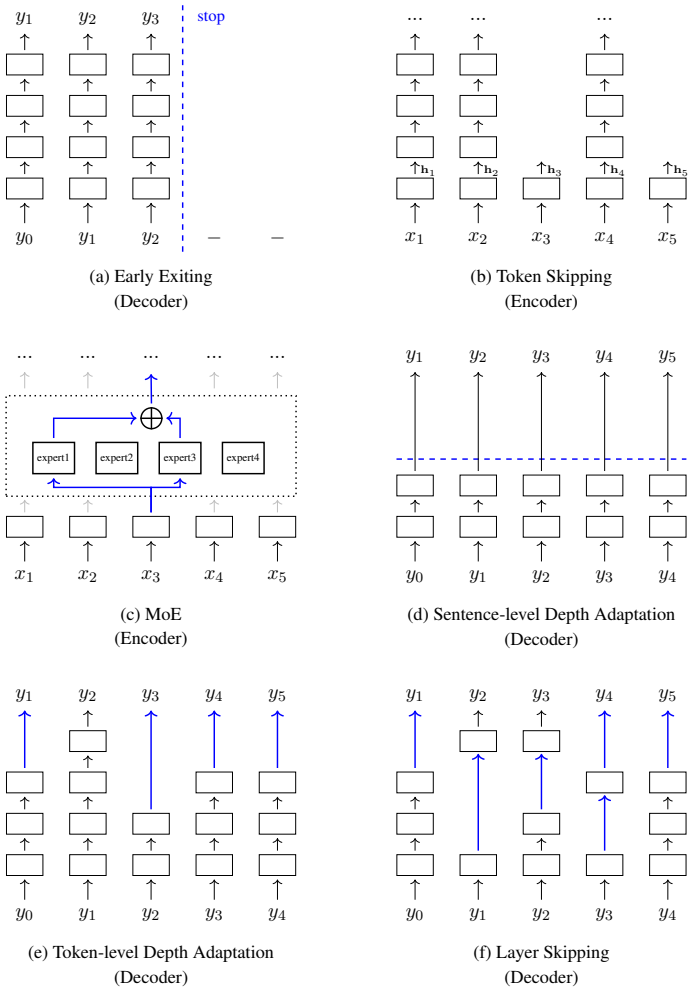
Cách tiếp cận thứ hai cho tính toán có điều kiện là dùng các **mô hình chuyên gia thưa** (*sparse expert model*), hoặc trường hợp phổ biến của nó — MoE [Yuksel và cộng sự, 2012]. Trong học sâu, một mô hình loại này thường được xây dựng từ một số chuyên gia, là các mạng nơ-ron có cùng cấu trúc nhưng khác tham số. Bằng cách này, ta có thể tạo ra một mô hình lớn chỉ bằng cách tăng số lượng chuyên gia. Khi chạy mô hình, dù là lúc huấn luyện hay dự đoán, ta chỉ kích hoạt một số ít chuyên gia nhờ các thuật toán định tuyến (xem Hình 6.17 (c)). Mô hình MoE là một mạng thích nghi, vì mỗi khi có đầu vào mới, mô hình định tuyến nó tới các chuyên gia khác nhau. Ở Mục 6.3.4, chúng tôi đã trình bày dạng cơ bản của MoE và cho thấy cách mở rộng quy mô các mô hình Transformer bằng phương pháp thưa này. Để có cái nhìn tổng quan về các tiến bộ gần đây của MoE, chúng tôi giới thiệu bạn đọc quan tâm tới công trình của Fedus và cộng sự [2022a].

Cách tiếp cận thứ ba có thể dùng để điều chỉnh mô hình Transformer theo đầu vào thay đổi là thu gọn số tầng một cách động. Nhiều phương pháp đã được đề xuất để làm việc này nhằm cải thiện hiệu quả suy luận. Đơn giản nhất trong số đó là thoát ra tại một số tầng ẩn mà ở đó ta vẫn có thể đưa ra dự đoán chính xác cho đầu vào (xem Hình 6.17 (d) và (e)). Để làm điều này, ta có thể hoặc xác định độ sâu phù hợp cho toàn bộ chuỗi (gọi là **mô hình thích nghi độ sâu mức câu** (*sentence-level depth-adaptive model*)), hoặc dùng độ sâu thích nghi cho từng token (gọi là **mô hình thích nghi độ sâu mức token** (*token-level depth-adaptive model*)). Ở đây ta xét các mô hình thích nghi độ sâu mức token, nhưng các phương pháp này có thể dễ dàng mở rộng cho mô hình thích nghi độ sâu mức chuỗi.

Ghi chú nhãn hình (người dịch): (a) *Early Exiting (Decoder)*: thoát sớm (bộ giải mã), *stop*: dừng; (b) *Token Skipping (Encoder)*: bỏ qua token (bộ mã hóa); (c) MoE (bộ mã hóa), *expert*: chuyên gia; (d) *Sentence-level Depth Adaptation*: thích nghi độ sâu mức câu; (e) *Token-level Depth Adaptation*: thích nghi độ sâu mức token; (f) *Layer Skipping*: bỏ qua tầng.

Giả sử có L tầng xếp chồng tại vị trí i[25]. Lý tưởng nhất, ta muốn tìm một tầng trong chồng tầng đó, có thể dùng làm tầng ẩn cuối cùng để đưa ra dự đoán, và có độ sâu thấp nhất có thể. Tuy nhiên, ta không thể đơn giản dùng tầng thứ L của chồng làm "đáp án chuẩn" (oracle) cho bài toán này, vì ta không bao

[25] Một tầng ở đây là một khối Transformer chuẩn gồm vài tầng con.



(a) Early Exiting (Decoder)

(b) Token Skipping (Encoder)

(c) MoE (Encoder)

(d) Sentence-level Depth Adaptation (Decoder)

(e) Token-level Depth Adaptation (Decoder)

(f) Layer Skipping (Decoder)

Hình 6.17: Các phương pháp tính toán có điều kiện, gồm thoát sớm, bỏ qua token, MoE, thích nghi độ sâu mức câu, thích nghi độ sâu mức token, và bỏ qua tầng. Mặc dù các phương pháp này được minh họa bằng quá trình mã hóa hoặc giải mã, phần lớn chúng có thể áp dụng cho cả bộ mã hóa lẫn bộ giải mã Transformer.

giờ biết trước tầng cuối cùng sẽ sinh ra gì trong lúc suy luận. Thay vào đó, ta cần xác định xem mạng có nên ngừng tăng thêm tầng ở độ sâu i hay không, dựa trên các tầng đã sinh ra cho đến lúc đó.

Bây giờ giả sử ta có một bộ giải mã Transformer tạo ra một phân phối trên bộ từ vựng V ở mỗi bước. Như thường lệ, ta ký hiệu đầu ra của tầng thứ l tại bước i là $\mathbf{s}_i^l$. Với mỗi $\mathbf{s}_i^l$, ta tạo một tầng đầu ra sinh ra phân phối $\mathbf{p}_i^l$ trên bộ từ vựng (gọi là **bộ phân loại thoát sớm** (*early exit classifier*)), cho bởi

$$\mathbf{p}_i^l \;=\; \text{Softmax}(\mathbf{s}_i^l \cdot \mathbf{W}_{\text{o}}^l), \tag{6.183}$$

trong đó $\mathbf{W}_{\text{o}}^l \in \mathbb{R}^{d\times|V|}$ là ma trận tham số. Như vậy ta có thêm $L-1$ tầng đầu ra, mỗi tầng ứng với một tầng ẩn từ độ sâu 1 đến $L-1$. Khi huấn luyện, ta xét các hàm mất mát entropy chéo của $\{\mathbf{p}_i^1, \cdots, \mathbf{p}_i^{L-1}\}$, và huấn luyện các tầng này cùng với mô hình Transformer. Khi kiểm tra, độ sâu của mạng tăng dần như thường lệ, và ta dùng $\{\mathbf{p}_i^1, \cdots, \mathbf{p}_i^l\}$ và/hoặc $\{\mathbf{s}_i^1, \cdots, \mathbf{s}_i^l\}$ để xác định xem có nên thoát ở tầng thứ l hay không. Có một số tiêu chí thoát, chẳng hạn:

- Các tiêu chí phổ biến dựa trên thước đo độ tin cậy của dự đoán. Một phương pháp đơn giản là tính entropy của $\mathbf{p}_i^l$, và thoát nếu entropy này vượt quá một giá trị định trước.
- Ngoài ra, có thể xem xác suất lớn nhất trong các phần tử của $\mathbf{p}_i^l$ là độ tin cậy của dự đoán.
- Thay vì xét đầu ra của một tầng duy nhất, ta cũng có thể xem xét sự thay đổi của đầu ra hoặc trạng thái ẩn qua một số tầng. Ví dụ, ta có thể đo độ tương đồng giữa $\mathbf{p}_i^{l-1}$ và $\mathbf{p}_i^l$ hoặc giữa $\mathbf{s}_i^{l-1}$ và $\mathbf{s}_i^l$. Nếu độ tương đồng vượt quá một ngưỡng cho trước, ta nói rằng đầu ra của mạng nơ-ron có xu hướng hội tụ và số tầng có thể ngừng tăng.
- Các phương pháp trên có thể được mở rộng để xem xét sự thay đổi trong các dự đoán của những bộ phân loại gắn với các tầng. Ví dụ, mô hình có thể chọn thoát nếu dự đoán của các bộ phân loại không thay đổi qua một số tầng.

Chú giải của người dịch: Entropy như thước đo "độ do dự"

Entropy của phân phối $\mathbf{p} = (p_1, \ldots, p_{|V|})$ là $H = -\sum_k p_k \log p_k$. Nó *nhỏ* khi mô hình chắc chắn và *lớn* khi mô hình phân vân. Ví dụ với 2 từ: $\mathbf{p} = (0.99,\ 0.01)$ cho $H \approx 0.056$ (dùng ln), còn $\mathbf{p} = (0.5,\ 0.5)$ cho $H = \ln 2 \approx 0.69$. Do đó trong thực tế, khi dùng entropy làm thước đo độ tin cậy, người ta thường cho mô hình thoát khi entropy đủ *thấp* (tức độ tin cậy đủ cao); tương đương, khi "độ tin cậy" vượt một ngưỡng định trước.

Thảo luận về các tiêu chí này có thể tìm thấy trong các bài báo liên quan [Xin và cộng sự, 2020; Zhou và cộng sự, 2020; Schuster và cộng sự, 2022]. Có nhiều cách để cải thiện các phương pháp thoát sớm này. Một cách là khám phá các dạng dự đoán khác cho mỗi tầng. Ví dụ, ta có thể xây dựng một mô hình dự đoán trực tiếp cần bao nhiêu tầng để mô hình hóa đầu vào [Elbayad và cộng sự, 2020]. Một hướng nghiên cứu khác về thoát sớm tập trung vào việc huấn luyện các mô hình này tốt hơn, chẳng hạn ta có thể xét các hàm mất mát khác nhau để huấn luyện các bộ phân loại [Schwartz và cộng sự, 2020; Schuster và cộng sự, 2022]. Ngoài ra, cũng có sự quan tâm tới việc học cách kết hợp đầu ra của nhiều tầng, để có thể đưa ra dự đoán bằng cách sử dụng nhiều mức biểu diễn [Zhou và cộng sự, 2020; Liao và cộng sự, 2021].

Một vấn đề của các mô hình thích nghi độ sâu mức token là biểu diễn ở một số độ sâu nhất định có thể không tồn tại ở các bước trước. Khi đó, tự chú ý chuẩn không áp dụng trực tiếp được, vì ta có thể không chú ý được tới các token trước ở cùng mức biểu diễn. Khi huấn luyện, vấn đề này có thể giải quyết bằng cách dùng đủ L tầng của mô hình đầy đủ. Khi suy luận, ta có thể hoặc nhân bản tầng mà ta thoát ra để lấp đầy chồng tầng, hoặc sửa mô hình tự chú ý để nó có thể chú ý tới biểu diễn của các token trước ở các độ sâu khác nhau.

Cũng có thể chọn một tập con bất kỳ của các tầng để xây dựng một mạng nông. Do đó, các mô hình thích nghi có thể được tổng quát hóa thành các mô hình bỏ qua (xem Hình 6.17 (f)). Giống như bài toán thoát sớm, bài toán bỏ qua có thể được đặt thành một bài toán học, trong đó một bộ phân loại được huấn luyện để quyết định xem một tầng có nên bị bỏ đi hay không. Bài toán học cách bỏ qua đã được nghiên cứu trong lĩnh vực thị giác máy tính [Wang và cộng sự, 2018b; Wu và cộng sự, 2018b]. Tuy nhiên, việc học một mô hình bỏ qua cho các mạng nơ-ron sâu, quy mô lớn là khó. Với các hệ thống thực tế, có vẻ vẫn hợp lý khi dùng các quy tắc kinh nghiệm hoặc các mô hình rẻ để có được một mạng nơ-ron với các tầng bị bỏ qua, điều này đã được bàn đến trong các mô hình NLP tiền huấn luyện gần đây [Wang và cộng sự, 2022c; Del Corro và cộng sự, 2023].

6.4.7 Chuyển giao và cắt tỉa mô hình

Nhiều mô hình Transformer lớn đã được phát triển thành công để giải quyết các bài toán NLP. Một câu hỏi thường gặp là: liệu ta có thể biến đổi một mô hình lớn, đã được huấn luyện tốt thành một mô hình nhỏ hơn để suy luận hiệu quả hơn không? Ở mức khái quát, điều này có thể được coi là một bài toán **học chuyển giao** (*transfer learning*), trong đó tri thức được chuyển từ mô hình này sang mô hình khác. Tuy nhiên, chúng tôi sẽ không bàn về chủ đề tổng quát này, vốn trải rộng trên nhiều vấn đề và mô hình, phần lớn nằm ngoài phạm vi chương này. Thay vào đó, chúng tôi thu hẹp thảo luận vào hai cách tiếp cận được dùng rộng rãi để huấn luyện các mạng nơ-ron nhỏ hơn từ các mạng lớn hơn.

1. Chưng cất tri thức

Chưng cất tri thức (*knowledge distillation*) là quá trình nén tri thức trong một mạng nơ-ron lớn (hoặc một tổ hợp các mạng nơ-ron) vào một mạng nơ-ron nhỏ hơn [Hinton và cộng sự, 2015]. Trong học có giám sát các mạng nơ-ron, hàm mục tiêu thường được thiết kế để định lượng sai số giữa câu trả lời dự đoán và câu trả lời đúng. Do đó, bằng cách cực tiểu hóa hàm mất mát này, mô hình được huấn luyện để đưa ra câu trả lời đúng. Mặc dù mô hình thường được tối ưu trên dữ liệu huấn luyện theo cách này, điều ta thực sự muốn là tổng quát hóa tốt trên dữ liệu chưa thấy. Tuy nhiên, điều này khó, vì chỉ huấn luyện với các nhãn đúng không cung cấp tín hiệu giám sát trực tiếp nào về khả năng tổng quát hóa. Trong chưng cất tri thức, thay vì ép mô hình khớp với đầu ra đúng, ta huấn luyện nó bắt chước hành vi (hay phân phối đầu ra) của một mô hình lớn hơn. Để làm được điều này, ta chuyển trực tiếp tri thức của một mô hình đã tiền huấn luyện sang mô hình mà ta muốn huấn luyện.

Một cách tiếp cận thường dùng cho chưng cất tri thức là **huấn luyện thầy–trò** (*teacher-student training*). Mô hình *thầy* thường là một mạng nơ-ron tương đối lớn, đã được huấn luyện và có khả năng tổng quát hóa tốt. Mô hình *trò* là một mạng nơ-ron nhỏ hơn, chẳng hạn có ít tầng hơn, mà ta chuyển tri thức sang. Một cách đơn giản để chưng cất tri thức từ mô hình thầy vào mô hình trò là dùng đầu ra của mô hình thầy làm câu trả lời "đúng" để huấn luyện mô hình trò. Giả sử ta có một mô hình Transformer thầy có thể sinh ra một chuỗi các phân phối $\{\Pr(\cdot|y_0, \mathbf{x}), \cdots, \Pr(\cdot|y_0 \cdots y_{n-1}, \mathbf{x})\}$ cho đầu vào $\mathbf{x}$. Để ký hiệu đơn giản, ta ký hiệu phân phối $\Pr(\cdot|y_0 \cdots y_{i-1}, \mathbf{x})$ là $\widetilde{\mathbf{p}}_i$. Tương tự, ta ký hiệu đầu ra của mô hình Transformer trò với cùng đầu vào là $\mathbf{p}_i$. Như thường lệ, ta xét một hàm mất mát $\text{Loss}(\widetilde{\mathbf{p}}_i, \mathbf{p}_i)$ (chẳng hạn hàm entropy chéo) để tính khoảng cách giữa $\widetilde{\mathbf{p}}_i$ và $\mathbf{p}_i$. Khi đó, ta có thể định nghĩa hàm mất mát trên toàn bộ chuỗi là

$$\mathcal{L}(\mathbf{x}, \theta) \ = \ \frac{1}{n} \sum_{i=1}^{n} \text{Loss}(\widetilde{\mathbf{p}}_i, \mathbf{p}_i), \tag{6.184}$$

trong đó θ là tham số của mô hình trò[26]. Dùng hàm mất mát này, ta có thể tối ưu θ cho một tập chuỗi

[26] Chúng tôi bỏ qua tham số của mô hình thầy vì chúng được giữ cố định trong suốt quá trình huấn luyện.

nguồn bất kỳ $\{\mathbf{x}_1, \cdots, \mathbf{x}_K\}$ bằng cách cực tiểu hóa hàm mục tiêu $\sum_{k=1}^{K} \mathcal{L}(\mathbf{x}_k, \theta)$.

Chú giải của người dịch: Vì sao học từ "thầy" lại tốt hơn học từ nhãn?

Nhãn đúng chỉ cho biết *một* đáp án, ví dụ từ tiếp theo là "mèo" (phân phối $(1, 0, 0)$ trên ba từ "mèo", "chó", "xe"). Mô hình thầy lại cho một phân phối "mềm" như $(0.7,\ 0.25,\ 0.05)$: nó ngầm nói rằng "chó" cũng khá hợp lý, còn "xe" thì không. Thông tin về *mức độ gần đúng* của các phương án sai giúp mô hình trò học được nhiều hơn từ mỗi ví dụ — giống như học sinh học từ lời giảng của thầy chứ không chỉ từ đáp án cuối sách.

Một số mở rộng của phương pháp cơ bản này đã được phát triển để mô hình hóa bài toán chuyển giao tri thức giữa hai mô hình. Một cách đơn giản là dùng các trạng thái ẩn thay vì xác suất đầu ra làm mục tiêu huấn luyện [Romero và cộng sự, 2014]. Trong trường hợp này, mục tiêu là cực tiểu hóa sự khác biệt giữa một số trạng thái ẩn của mô hình thầy và các trạng thái tương ứng của mô hình trò. Thay vì dùng đầu ra của các tầng khác nhau làm mục tiêu huấn luyện mô hình trò, một kỹ thuật khác là mô hình hóa quan hệ giữa các mẫu và huấn luyện mô hình trò bằng cách cực tiểu hóa sự khác biệt giữa các mã hóa quan hệ của mô hình thầy và mô hình trò [Park và cộng sự, 2019; Peng và cộng sự, 2019]. Ví dụ, ta có thể xây dựng một mô hình mã hóa quan hệ dựa trên kiến trúc Transformer. Mục tiêu khi đó là tối ưu mô hình trò sao cho mã hóa quan hệ tương ứng của nó trên một nhóm mẫu càng gần với mã hóa của mô hình thầy càng tốt.

Đối với các bài toán sinh chuỗi, một trường hợp đặc biệt của chưng cất tri thức, có thể được xem như một cách **tăng cường dữ liệu** (*data augmentation*), thường được dùng để phát triển các mô hình gọn nhẹ [Kim và Rush, 2016]. Ví dụ, xét bài toán chuyển khả năng dịch của một mô hình dịch máy đã phát triển tốt (tức mô hình thầy) sang một mô hình mới (tức mô hình trò). Cho một tập câu phía nguồn $\{\mathbf{x}_1, \cdots, \mathbf{x}_K\}$, ta có thể dùng mô hình thầy để dịch mỗi $\mathbf{x}_k$ thành một câu phía đích $\widetilde{\mathbf{y}}_k$. Sau đó, bằng cách coi $\mathbf{x}_k$ và $\widetilde{\mathbf{y}}_k$ là các cặp câu, ta thu được một tập dữ liệu song ngữ gồm $\{(\mathbf{x}_1, \widetilde{\mathbf{y}}_1), \cdots, (\mathbf{x}_K, \widetilde{\mathbf{y}}_K)\}$. Ta có thể dùng tập dữ liệu song ngữ này làm tập dữ liệu có nhãn để huấn luyện mô hình trò như thường lệ. Một ưu điểm của phương pháp tăng cường dữ liệu này là nó không phụ thuộc vào kiến trúc. Nó không đòi hỏi truy cập vào kiến trúc bên trong của mô hình thầy hay mô hình trò. Do đó, nó có thể được áp dụng ngay cả khi mô hình thầy được dùng như một hộp đen. Các thảo luận chi tiết hơn về chưng cất tri thức có thể tìm thấy trong các bài tổng quan của Gou và cộng sự [2021] và Wang và Yoon [2021].

2. Cắt tỉa có cấu trúc

Cắt tỉa là một trong những phương pháp nén mô hình phổ biến nhất và đã được áp dụng cho rất nhiều hệ thống. Một cách tiếp cận thông dụng là **cắt tỉa phi cấu trúc** (*unstructured pruning*), trong đó ta chỉ giữ lại một tập con thưa các kết nối giữa các nơ-ron. Tuy nhiên, như với hầu hết các mô hình thưa, mạng được cắt tỉa theo cách này thường đòi hỏi các cài đặt chuyên biệt và sự hỗ trợ của phần cứng, điều này lại hạn chế mức tăng hiệu quả trong một số ứng dụng. Một phương án đơn giản hơn nhưng mạnh tay hơn là **cắt tỉa có cấu trúc** (*structured pruning*). Trong học sâu, cắt tỉa có cấu trúc là kỹ thuật loại bỏ đồng thời cả một nhóm nơ-ron hoặc kết nối. Ví dụ, ta có thể loại bỏ toàn bộ một tầng nơ-ron khỏi mạng để thu được một mô hình nông hơn. Với kiến trúc nhiều tầng và nhiều đầu, Transformer rất phù hợp với cắt tỉa có cấu trúc, và ta có thể cắt tỉa một mạng Transformer theo nhiều cách khác nhau. Chẳng hạn, ta có thể cắt bỏ một số đầu cụ thể trong cơ chế chú ý đa đầu [Voita và cộng sự, 2019; Michel và cộng sự, 2019], hoặc toàn bộ một số tầng trong chồng tầng [Hou và cộng sự, 2020; Kim và Awadalla, 2020].

Một cách hình thức, ta có thể biểu diễn một mạng nơ-ron như một tập các nhóm tham số $\{\theta_1, \cdots, \theta_R\}$, mỗi nhóm tương ứng với một thành phần hay mô-đun con cụ thể của mạng. Mục tiêu của ta là tìm một tập con của $\{\theta_1, \cdots, \theta_R\}$ cho phép xây dựng một mô hình có hiệu năng tốt trong khi vẫn giữ độ phức tạp tính toán thấp hơn đáng kể. Tuy nhiên, tìm kiếm vét cạn một tập con tối ưu như

vậy là bất khả thi, vì số ứng viên có thể có tăng theo cấp số tổ hợp, và việc đánh giá tất cả chúng tốn kém quá mức về mặt tính toán.

Chú giải của người dịch: Bao nhiêu cách cắt tỉa?

Với R nhóm tham số, mỗi nhóm có thể giữ hoặc bỏ, nên có 2^R tập con (kiến thức tổ hợp lớp 11). Một Transformer 24 tầng, mỗi tầng 16 đầu chú ý, có $R = 384$ đầu; 2^{384} là con số có hơn 115 chữ số — không thể thử hết, dù mỗi lần thử chỉ mất một phần tỷ giây.

Một cách tiếp cận cắt tỉa có cấu trúc là loại bỏ ngẫu nhiên các thành phần của mô hình. Ta có thể lặp lại quá trình cắt tỉa ngẫu nhiên này nhiều lần để tạo ra một tập các mô hình ứng viên, rồi chọn mô hình có hiệu năng tốt nhất. Một cách tiếp cận khác dựa vào các quy tắc kinh nghiệm để xác định thành phần nào là dư thừa và có thể loại bỏ an toàn. Các thước đo thông dụng để đánh giá tầm quan trọng của một nhóm tham số θ_r bao gồm nhiều chỉ số dựa trên chuẩn (norm) của trọng số hoặc gradient của nó [Santacroce và cộng sự, 2023]. Khi đó ta có thể cắt bỏ θ_r nếu các chỉ số này thấp hơn hoặc vượt quá các ngưỡng định trước. Cách tiếp cận thứ ba đặt bài toán cắt tỉa thành một bài toán tối ưu liên tục bằng cách đưa vào các cổng có thể huấn luyện, cho biết sự có mặt hay vắng mặt của các thành phần khác nhau [McCarley và cộng sự, 2019; Wang và cộng sự, 2020d; Lagunas và cộng sự, 2021]. Mô hình cắt tỉa cuối cùng có thể được suy ra trực tiếp từ các cổng đã huấn luyện này. Đáng chú ý là, trong nhiều trường hợp, cắt tỉa không còn chỉ là một bước hậu xử lý cho một mô hình đã huấn luyện xong, mà là một phần không thể tách rời của chính quá trình huấn luyện.

6.4.8 Nén chuỗi

Trong các bài toán mô hình hóa và sinh chuỗi, độ phức tạp thời gian và không gian chịu ảnh hưởng mạnh bởi độ dài của chuỗi đầu vào hoặc đầu ra. Vì vậy, chuỗi ngắn hơn thường được ưa chuộng hơn. Điều này đặc biệt quan trọng với các mô hình Transformer, vì độ phức tạp thời gian và không gian của chúng tăng theo bình phương độ dài chuỗi, khiến lượng bộ nhớ chiếm dụng và độ trễ trở thành những nút thắt tính toán đáng kể với các chuỗi rất dài. Trong khi các tiểu mục trước bàn về những thay đổi trong kiến trúc Transformer để xử lý chuỗi dài, ở mục này chúng tôi tìm hiểu các phương pháp nén chuỗi về những độ dài dễ xử lý hơn.

Một cách tiếp cận đơn giản là ánh xạ chuỗi đầu vào thành một biểu diễn có kích thước cố định. Ví dụ, dùng các mô hình hồi quy đã bàn ở Mục 6.4.2, ta có thể mã hóa một chuỗi vectơ thành một vectơ duy nhất. Phương pháp này có thể dễ dàng được mở rộng để tạo ra một biểu diễn lớn hơn, nhờ đó biểu diễn này có thể giữ lại nhiều thông tin hơn từ đầu vào ban đầu. Ví dụ, ta có thể chọn một số lượng cố định các trạng thái ẩn trên chuỗi để tạo thành một chuỗi mới có độ dài cố định. Một cách khác để biểu diễn một chuỗi có độ dài thay đổi thành một chuỗi có độ dài cố định là thực hiện chú ý chéo (*cross-attention*) giữa các vectơ đầu vào và một tập các biểu diễn ẩn có thể học được. Trong công trình của Jaegle và cộng sự [2021], điều này được thực hiện bằng cách đưa vào r biểu diễn ẩn $\{\mathbf{u}_1, \cdots, \mathbf{u}_r\}$, rồi áp dụng chú ý chéo giữa các vectơ đầu vào $\mathbf{x}_1, \cdots, \mathbf{x}_m$ và các biểu diễn ẩn này. Mô hình chú ý có thể là một mô hình chú ý QKV chuẩn, trong đó ta coi $\{\mathbf{u}_1, \cdots, \mathbf{u}_r\}$ là các truy vấn và $\{\mathbf{x}_1, \cdots, \mathbf{x}_m\}$ là các khóa và giá trị. Đầu ra của mô hình này là một chuỗi gồm r vectơ, có thể dùng làm đầu vào độ dài cố định cho các hệ thống phía sau.

Chú giải của người dịch: Vì sao đầu ra luôn có r vectơ?

Trong chú ý, mỗi *truy vấn* tạo ra đúng một vectơ đầu ra (một trung bình có trọng số của các giá trị). Ở đây truy vấn là r vectơ $\mathbf{u}$ cố định, còn đầu vào chỉ đóng vai khóa và giá trị. Vì thế dù câu dài $m = 10$ hay $m = 10\,000$ token, đầu ra vẫn luôn là r vectơ (ví dụ $r = 64$) — giống như 64 "phóng viên" mỗi người đọc toàn bộ văn bản rồi viết một bản tóm tắt theo góc nhìn riêng.

Cách tiếp cận thứ hai là dùng giảm mẫu (*downsampling*) để nén chuỗi thành một chuỗi ngắn hơn. Một phương pháp giảm mẫu điển hình là tích chập có bước nhảy (*strided convolution*), đã được dùng rộng rãi trong thị giác máy tính và xử lý tiếng nói. Ví dụ, giả sử có một chuỗi m vectơ trong $\mathbb{R}^d$. Ta có thể định nghĩa một bộ lọc có độ rộng 2 và bước nhảy 2. Khi nhận chuỗi làm đầu vào, bộ lọc tạo ra một chuỗi gồm $\frac{m}{2}$ vectơ mới trong $\mathbb{R}^d$, nên ta có hệ số rút gọn là 2. Ngoài ra, ta có thể xếp chồng nhiều tầng tích chập hoặc tầng gộp (pooling) để đạt được mức rút gọn độ dài mong muốn, gọi là **giảm mẫu lũy tiến** (*progressive downsampling*). Tuy nhiên, có vẻ không thể tránh khỏi việc giảm mẫu sẽ dẫn đến mất mát thông tin [Han và cộng sự, 2020; Burchi và Vielzeuf, 2021]. Ta cần cân nhắc sự đánh đổi giữa độ dài chuỗi đã nén và hiệu năng của các hệ thống phía sau [Xu và cộng sự, 2023b].

Trong NLP, bài toán nén chuỗi cũng liên quan chặt chẽ đến bài toán tách token cho chuỗi đầu vào. Do đó, tách token là một cách tiếp cận thực tế có thể dùng để giải quyết vấn đề độ dài. Chia một chuỗi ký tự thành các token nhỏ (chẳng hạn ký tự) nói chung làm giảm độ thưa của dữ liệu, giúp việc học vectơ nhúng của các token này dễ hơn, nhưng các cách làm như vậy thường dẫn đến chuỗi dài. Ngược lại, ta sẽ có chuỗi ngắn hơn nếu chia chuỗi đầu vào thành các đơn vị lớn hơn, nhưng khi đó có thể gặp vấn đề dữ liệu thưa. Trong các phương pháp tách token tất định, vốn tạo ra kết quả tách token dựa trên thống kê thu thập từ toàn bộ tập dữ liệu, độ dài chuỗi có thể được kiểm soát ở một mức độ nào đó bằng cách điều chỉnh các siêu tham số cụ thể; ví dụ, trong mã hóa cặp byte (byte pair encoding) [Sennrich và cộng sự, 2016b], tăng kích thước bộ từ vựng nói chung làm giảm số token thu được. Một cách khác để có được một chuỗi token phù hợp là dùng một mô hình để lựa chọn giữa các ứng viên tách token [Kudo, 2018; Provilkov và cộng sự, 2020]. Giống như nhiều mô hình xác suất cho sinh văn bản, trong trường hợp này ta có thể đưa các tri thức tiên nghiệm (prior) vào tiêu chí chọn cách tách token, để thể hiện sự ưu tiên cho chuỗi ngắn hơn so với chuỗi dài hơn.

Cách tiếp cận thứ tư[27] cho nén chuỗi là loại bỏ một số token trong chuỗi. Chẳng hạn, trong nhiều ứng dụng thực tế, chuỗi đơn giản bị cắt cụt khi độ dài vượt quá một ngưỡng định trước. Ta có thể liên hệ điều này với các cách tiếp cận thoát sớm và bỏ qua trong tính toán có điều kiện. Vì vậy, các phương pháp đã bàn ở Mục 6.4.6 có thể áp dụng trực tiếp. Các phương pháp loại bỏ token cũng có thể được xem là các phương pháp cắt tỉa, gọi là **cắt tỉa token** (*token pruning*). Bằng cách loại bỏ những token ít quan trọng đối với việc biểu diễn toàn bộ chuỗi, cắt tỉa token có thể giảm đáng kể độ dài chuỗi mà vẫn duy trì hiệu năng của các hệ thống NLP trên các tác vụ phía sau [Kim và cộng sự, 2023].

6.4.9 Các phương pháp tính toán hiệu năng cao

Cho đến giờ trong mục này, chúng ta đã bàn về các mô hình Transformer hiệu quả từ góc độ học sâu. Tuy nhiên, ta chưa xét đến hiệu quả phần cứng của chúng. Vì phần cứng hiện đại cung cấp nhiều mô thức thực thi khác nhau, mức tiết kiệm thời gian và bộ nhớ trên thực tế thường phụ thuộc vào thông số kỹ thuật của hệ thống phần cứng. Một hướng nghiên cứu về sử dụng hiệu quả tài nguyên tính toán là khám phá các phương pháp tính toán song song. Đã có nhiều nỗ lực phát triển các mô hình Transformer

[27] **[ND]** Sách gốc viết "thứ tư" (*fourth*) dù chỉ nêu tường minh ba cách tiếp cận trước đó; có thể hiểu cách dùng tách token là cách thứ ba.

quy mô lớn bằng cách dùng một cụm máy. Thông thường, việc mở rộng Transformer lên các mô hình có hàng tỷ hoặc thậm chí hàng chục tỷ tham số đòi hỏi thiết kế cẩn thận các chiến lược song song để phân mảnh các mạng lớn. Cài đặt hiệu quả hơn cho những hệ thống như vậy còn đòi hỏi xem xét chi phí truyền thông trong cụm, cũng như tận dụng các mô hình thưa chỉ kích hoạt một tập con nhỏ tham số cho mỗi mẫu, nhờ đó có thể dùng những mô hình rất lớn. Phần lớn các phương pháp này đã được nghiên cứu sâu rộng trong các tài liệu [Lepikhin và cộng sự, 2021; Barham và cộng sự, 2022; Fedus và cộng sự, 2022b]. Kết quả của các nghiên cứu này là nền tảng cho nhiều công trình tiếp theo khảo sát các **định luật co giãn** (*scaling law*) của mô hình ngôn ngữ lớn [Brown và cộng sự, 2020; Chowdhery và cộng sự, 2022]. Vì các mô hình phân tán quy mô lớn mang tính tổng quát và không chuyên biệt cho Transformer, chúng tôi bỏ qua phần thảo luận về chúng ở đây. Bạn đọc quan tâm có thể tham khảo các bài báo trên để biết thêm chi tiết.

Trong tiểu mục này, chúng tôi xét các phương pháp có tính đến phần cứng nhằm đạt hiệu quả tính toán cao hơn cho mô hình Transformer. Trước hết, ta xét một phương pháp đơn giản nhưng được dùng rộng rãi, nhằm biểu diễn và thực thi mạng nơ-ron bằng các định dạng độ chính xác thấp hơn hoặc độ chính xác hỗn hợp [Gholami và cộng sự, 2022]. Các mạng nơ-ron thông thường thường dựa trên biểu diễn dấu phẩy động độ chính xác đơn và/hoặc độ chính xác kép cho dữ liệu. Mặc dù kiểu dữ liệu dấu phẩy động độ chính xác đơn cung cấp một cách đủ chính xác để biểu diễn tham số và trạng thái trung gian trong hầu hết trường hợp, ở một số ứng dụng chúng là không cần thiết. Thay vào đó, có thể dùng định dạng độ chính xác một nửa (hoặc thậm chí thấp hơn) để biểu diễn số thực dấu phẩy động trong mạng nơ-ron. Kích thước của mô hình thu được vì thế chỉ bằng một nửa mô hình ban đầu. Một ưu điểm của việc dùng biểu diễn dấu phẩy động độ chính xác một nửa là, mặc dù xử lý các kiểu dữ liệu này đòi hỏi các API mới cho các phép toán đại số tuyến tính và sự hỗ trợ của phần cứng, nó không làm thay đổi kiến trúc mô hình, chỉ cần sửa đổi nhỏ trong hệ thống. Ví dụ, biểu diễn dấu phẩy động độ chính xác một nửa có thể được áp dụng cho huấn luyện hoặc suy luận của Transformer, hoặc cả hai.

Chú giải của người dịch: Độ chính xác đơn, kép, một nửa

Máy tính lưu số thực theo dạng "dấu phẩy động", gần giống cách viết khoa học 6.02×10^{23}: một phần chữ số có nghĩa và một phần số mũ. Độ chính xác *kép* dùng 64 bit (khoảng 15–16 chữ số có nghĩa), *đơn* dùng 32 bit (khoảng 7 chữ số), *một nửa* dùng 16 bit (khoảng 3–4 chữ số). Một mô hình 7 tỷ tham số ở dạng 32 bit chiếm khoảng 28 GB; chuyển sang 16 bit chỉ còn khoảng 14 GB. Mạng nơ-ron thường chịu được sai số nhỏ như vậy mà vẫn cho kết quả tốt.

Gần đây, việc triển khai các mô hình Transformer lớn được cải thiện thêm nhờ lượng tử hóa (*quantization*) các mô hình này. Trong xử lý tín hiệu, lượng tử hóa là quá trình ánh xạ các giá trị liên tục (tức biểu diễn dấu phẩy động) thành một tập giá trị rời rạc (tức biểu diễn dấu phẩy tĩnh). Quá trình này thường được thực hiện bằng một hệ thống gọi là *bộ lượng tử hóa*. Trong ngữ cảnh mạng nơ-ron, một bộ lượng tử hóa gồm hai hàm — hàm lượng tử hóa và hàm giải lượng tử hóa. Hàm lượng tử hóa ánh xạ một số dấu phẩy động thành một số nguyên (với ít bit hơn). Một hàm lượng tử hóa đơn giản được cho bởi

$$Q(x) \ = \ \left\lfloor \frac{x}{s} \right\rceil, \tag{6.185}$$

trong đó $\lfloor \cdot \rceil$ là hàm làm tròn[28], x là đầu vào giá trị thực, và s là kích thước bước lượng tử hóa, điều khiển mức độ lượng tử hóa. Hàm lượng tử hóa đi kèm với một hàm giải lượng tử hóa

$$D(r) \ = \ s \cdot r. \tag{6.186}$$

[28] $\lfloor a \rceil$ trả về số nguyên gần a nhất.

Với ký hiệu này, bộ lượng tử hóa có thể được biểu diễn là $D(Q(x)) = s \cdot \lfloor \frac{x}{s} \rceil$. Hiệu giữa $D(Q(x))$ và x được gọi là *sai số lượng tử hóa*. Giá trị s nhỏ hơn thường làm giảm sai số lượng tử hóa. Tuy nhiên, trong thực tế, ta muốn chọn một giá trị s phù hợp để trải đều các giá trị có thể có của $Q(x)$ trên toàn bộ miền biểu diễn số nguyên. Ví dụ, $s = \frac{\max\{D(r)\}}{2^p - 1}$, trong đó p là số bit dùng để biểu diễn một số nguyên và $\max\{D(r)\}$ là giá trị lớn nhất có thể biểu diễn sau khi giải lượng tử hóa. Các phương trình trên cho thấy một trong những trường hợp đơn giản nhất của lượng tử hóa. Các thảo luận tổng quát hơn về lượng tử hóa có thể tìm thấy trong các sách về xử lý tín hiệu số và các bài tổng quan liên quan [Oppenheim và Schafer, 1975; Rabiner và Gold, 1975; Gray, 1998].

Chú giải của người dịch: Ví dụ lượng tử hóa bằng số

Giả sử các trọng số nằm trong khoảng $[0; 2.55]$ và ta dùng $p = 8$ bit (số nguyên từ 0 đến 255). Chọn $s = \frac{2.55}{2^8 - 1} = 0.01$. Với $x = 1.2345$: $Q(x) = \lfloor 123.45 \rceil = 123$ (lưu bằng 1 byte thay vì 4 byte), và $D(123) = 1.23$. Sai số lượng tử hóa là $|1.23 - 1.2345| = 0.0045$, không vượt quá $s/2$. Nếu chọn s nhỏ hơn, sai số giảm nhưng khi đó các số nguyên cần thiết có thể vượt quá 255, tức cần nhiều bit hơn.

Áp dụng lượng tử hóa cho Transformer tương đối đơn giản. Ý tưởng là ta lượng tử hóa đầu vào và tham số mô hình bằng $Q(x)$ rồi đưa chúng vào một mô hình Transformer đã lượng tử hóa, trong đó mọi tầng đều hoạt động trên các tensor giá trị nguyên. Nói cách khác, ta cài đặt mô hình chỉ bằng số học số nguyên. Tuy nhiên, cái giá phải trả cho mô hình nén này, như với nhiều cách tiếp cận xấp xỉ khác trong học sâu, là các dự đoán của nó kém chính xác hơn so với Transformer chuẩn. Dùng phép toán số nguyên để xấp xỉ phép toán giá trị liên tục nói chung gây ra sai số xấp xỉ. Các sai số này sẽ tích lũy nếu mạng nơ-ron lượng tử hóa sâu. Hơn nữa, mô hình Transformer có những thành phần (như các tầng con tự chú ý) đòi hỏi các phép toán đại số tuyến tính tương đối phức tạp. Chỉ đơn giản áp dụng lượng tử hóa cho các mô-đun này sẽ làm giảm độ chính xác đáng kể. Một giải pháp là đơn giản hóa kiến trúc mô hình và phát triển các mô hình con mới phù hợp hơn với lượng tử hóa. Ngoài ra, một mô thức phổ biến hơn trong các mạng nơ-ron lượng tử hóa là thêm các hàm giải lượng tử hóa vào mạng, sao cho đầu ra của một tầng là các tensor dấu phẩy động và có thể dùng như thường lệ ở các bước tiếp theo. Xét một ví dụ đơn giản, trong đó ta nhân một ma trận đầu vào giá trị thực $\mathbf{a}$ với một ma trận tham số giá trị thực $\mathbf{A}$. Trước tiên ta lượng tử hóa $\mathbf{a}$ và $\mathbf{A}$, rồi nhân chúng bằng phép nhân ma trận số nguyên. Kết quả sau đó được giải lượng tử hóa thành một ma trận giá trị thực. Bằng cách này, ta thu được một xấp xỉ $D(Q(\mathbf{a}) \cdot Q(\mathbf{A}))$ cho $\mathbf{a} \cdot \mathbf{A}$ với chi phí tính toán rất thấp.

Tuy nhiên, việc kẹp mỗi tầng giữa $Q(\cdot)$ và $D(\cdot)$ sẽ dẫn đến chi phí phụ thêm cho việc chạy $Q(\cdot)$ và $D(\cdot)$. Trong một số ứng dụng thực tế, chi phí tính toán phát sinh do $Q(\cdot)$ và $D(\cdot)$ thậm chí còn lớn hơn thời gian tiết kiệm được từ việc thực hiện các phép toán số nguyên. Nói chung, lợi ích của việc lượng tử hóa mạng nơ-ron chỉ lớn hơn chi phí khi mạng nơ-ron đủ lớn. Vì vậy, trong thực tế, người ta thường chỉ thực hiện tính toán lượng tử hóa cho những phép toán có chi phí tính toán cao. Ví dụ, trong các mô hình ngôn ngữ lớn gần đây, lượng tử hóa chủ yếu được áp dụng cho phép nhân các ma trận lớn, mang lại mức tiết kiệm đáng kể về thời gian và bộ nhớ.

Mặc dù các cách tiếp cận lượng tử hóa có thể được dùng cả khi huấn luyện lẫn suy luận, một cách tiếp cận được dùng rộng rãi là lượng tử hóa mô hình Transformer sau khi huấn luyện (gọi là **lượng tử hóa sau huấn luyện** (*post-training quantization*)). Trong cách tiếp cận này, lượng tử hóa được thực hiện trên các mạng nơ-ron dấu phẩy động đã được huấn luyện tốt, nhờ đó ít sai số liên quan đến lượng tử hóa hơn. Tuy nhiên, các sai số này không dễ được bù đắp vì chúng xuất hiện sau khi quá trình huấn luyện đã hoàn tất. Một cách tiếp cận hứa hẹn hơn là đưa lượng tử hóa vào trong quá trình huấn luyện, để mô hình có thể học cách bù đắp các sai số liên quan đến lượng tử hóa [Jacob và cộng sự, 2018; Nagel và cộng sự, 2021]. Đã có một số nỗ lực áp dụng huấn luyện có tính đến lượng tử hóa cho Transformer

[Bondarenko và cộng sự, 2021; Stock và cộng sự, 2021; Yang và cộng sự, 2023b]. Ngoài hiệu quả tính toán, một yếu tố quan trọng khác đối với các hệ thống hiệu năng cao là các ràng buộc của hệ phân cấp bộ nhớ. Nói chung, thiết kế hệ thống tốt hơn đòi hỏi phải xem xét tốc độ và dung lượng của các cấp bộ nhớ khác nhau. Vấn đề còn phức tạp hơn khi huấn luyện các mô hình Transformer lớn trên phần cứng hiện đại, nơi cả GPU và CPU đều được sử dụng. Một nguyên tắc chung của thiết kế hệ thống là cần giảm thiểu việc truyền dữ liệu giữa các cấp bộ nhớ khác nhau. Mặc dù lý tưởng nhất là có một bộ nhớ cấp cao, dung lượng lớn để lưu mọi dữ liệu cần xử lý, trong nhiều tình huống thực tế, kích thước của bộ nhớ nhanh trên chip nhỏ hơn tổng kích thước dữ liệu nhiều bậc độ lớn. Trong trường hợp này, ta có thể sắp xếp lại thứ tự truy cập bộ nhớ trong các thuật toán sao cho dữ liệu dùng ở các bước tính toán liền kề có thể được nạp vào bộ nhớ tốc độ cao cùng một lúc. Ý tưởng này thúc đẩy sự phát triển của nhiều thư viện đại số tuyến tính nhanh. Ví dụ, có những thuật toán nhân ma trận được tối ưu hóa cao cho các hình dạng ma trận đầu vào khác nhau.

Việc dùng các thuật toán đại số tuyến tính đã tối ưu này để xây dựng một hệ thống dựa trên Transformer là tương đối đơn giản, nhưng các mô-đun của hệ thống này không được tối ưu hóa đồng thời về hiệu quả. Ví dụ, một tầng con tự chú ý gồm một loạt các phép co giãn, chuẩn hóa và nhân ma trận. Mặc dù mỗi phép toán này đều được cài đặt trong các thư viện đại số tuyến tính hiệu quả, việc gọi chúng liên tiếp vẫn đòi hỏi nhiều lần truyền dữ liệu bộ nhớ khi chuyển từ phép toán này sang phép toán khác. Trong thực tế, một cách tốt hơn là giữ một số trạng thái trung gian trong bộ nhớ trên chip và dùng lại chúng ở các bước tính toán tiếp theo thay vì nạp lại từ bộ nhớ chậm hơn. Ví dụ, trên GPU hiện đại, một cách đơn giản để làm điều này là gộp nhiều phép toán thành một phép toán duy nhất, một kỹ thuật được gọi là **hợp nhất nhân** (*kernel fusion*). Với mô hình Transformer, ý tưởng chung là thiết kế các chiến lược phân chia và bố trí dữ liệu sao cho tối đa hóa lượng tính toán trên mỗi khối dữ liệu được nạp vào bộ nhớ hiệu năng cao, đồng thời giảm thiểu việc truyền dữ liệu bộ nhớ. Đã có một số nỗ lực dùng các chiến lược này để cải thiện các mô hình chú ý trong Transformer [Ivanov và cộng sự, 2021; Pope và cộng sự, 2023]. Một số phương pháp trong đó, như FlashAttention và PagedAttention, đã được tích hợp thành công vào các mô hình ngôn ngữ lớn gần đây [Dao và cộng sự, 2022; Kwon và cộng sự, 2023].

Chú giải của người dịch: Hệ phân cấp bộ nhớ, ví von đời thường

Hãy hình dung bạn nấu ăn: mặt bàn bếp (bộ nhớ trên chip) nhỏ nhưng lấy đồ cực nhanh; tủ lạnh (bộ nhớ GPU chính) to hơn nhưng mỗi lần mở tủ tốn thời gian; siêu thị (bộ nhớ CPU/ổ đĩa) còn xa hơn nữa. Nấu hiệu quả là lấy đủ nguyên liệu cho vài bước liên tiếp ra bàn một lần, làm xong hết rồi mới cất, thay vì mỗi bước lại mở tủ. "Hợp nhất nhân" chính là gộp các bước co giãn, Softmax, nhân ma trận thành một lượt làm việc trên mặt bàn; FlashAttention áp dụng ý tưởng này cho cơ chế chú ý.

6.5 Ứng dụng

Transformer có rất nhiều ứng dụng trong các tác vụ NLP. Mặc dù mô hình Transformer do Vaswani và cộng sự [2017] giới thiệu dựa trên một kiến trúc bộ mã hóa (*encoder*)–bộ giải mã (*decoder*) chuẩn, nó chủ yếu được dùng theo ba cấu hình khác nhau:

- **Mô hình chỉ có bộ giải mã** (*Decoder-only Models*). Nếu bỏ các tầng con chú ý chéo, bộ giải mã trở thành một mô hình ngôn ngữ (*language model*) chuẩn. Do đó, mô hình chỉ có bộ giải mã này có thể được áp dụng cho các bài toán sinh văn bản. Ví dụ, cho một chuỗi token ngữ cảnh bên trái, ta dùng mô hình để dự đoán các token tiếp theo.

- **Mô hình chỉ có bộ mã hóa** (*Encoder-only Models*). Bộ mã hóa Transformer có thể được xem là mô hình chuỗi nhận một chuỗi token làm đầu vào và tạo ra một chuỗi biểu diễn, mỗi biểu diễn ứng với một token đầu vào. Các biểu diễn này có thể được xem như một dạng mã hóa của chuỗi đầu vào, và thường được dùng làm đầu vào cho một mô hình dự đoán. Kiến trúc bộ mã hóa–bộ dự đoán này là nền tảng của nhiều hệ thống NLP, chẳng hạn các hệ thống phân loại câu và gán nhãn chuỗi (*sequence labeling*). Bộ mã hóa Transformer đã tiền huấn luyện cũng có thể được dùng để ánh xạ các văn bản vào cùng một không gian vectơ, để ta có thể tính khoảng cách hay độ tương đồng giữa hai văn bản bất kỳ.
- **Mô hình bộ mã hóa–bộ giải mã** (*Encoder-Decoder Models*). Mô hình bộ mã hóa–bộ giải mã thường được dùng để mô hình hóa các bài toán chuỗi-sang-chuỗi (*sequence-to-sequence*). Ứng dụng bao gồm nhiều tác vụ trong NLP và các lĩnh vực liên quan, như dịch máy (*machine translation*) và sinh chú thích ảnh.

Lưu ý rằng mặc dù phần lớn các hệ thống dựa trên Transformer có thể được xếp vào ba loại trên, cùng một bài toán NLP nói chung có thể được giải quyết bằng các loại mô hình khác nhau. Ví dụ, các mô hình chỉ có bộ giải mã gần đây đã thể hiện hiệu năng mạnh trên rất nhiều bài toán bằng cách đặt chúng thành các tác vụ sinh văn bản, dù trước đây nhiều bài toán trong số đó được giải quyết bằng mô hình bộ mã hóa–bộ giải mã hoặc mô hình chỉ có bộ mã hóa. Để minh họa cách áp dụng các mô hình trên, mục này xét một vài ứng dụng trong đó Transformer được chọn làm mô hình xương sống.

6.5.1 Mô hình hóa ngôn ngữ

Mô hình hóa ngôn ngữ là một tác vụ NLP, trong đó ta dự đoán token tiếp theo khi biết các token đứng trước nó. Bài toán này thường được phát biểu là ước lượng phân phối của các token tại vị trí $i+1$ khi biết các token tại các vị trí $0 \sim i$. Ở đây ta ký hiệu phân phối này là $\Pr(\cdot \mid x_0, \ldots, x_i)$, trong đó $x_0, \ldots, x_i$ là các token quan sát được. Token dự đoán thu được bằng cách cực đại hóa xác suất:

$$\hat{x}_{i+1} = \arg\max_{x_{i+1} \in V} \Pr(x_{i+1} | x_0, \cdots, x_i), \tag{6.187}$$

trong đó V là bộ từ vựng. Việc dự đoán có thể được mở rộng cho các token theo sau $\hat{x}_{i+1}$

$$\hat{x}_{k+1} = \arg\max_{x_{k+1} \in V} \Pr(x_{k+1} | x_0, \cdots, x_i, \hat{x}_{i+1}, \cdots, \hat{x}_k). \tag{6.188}$$

Mô hình này là nền tảng của nhiều hệ thống sinh văn bản: cho các token ngữ cảnh $x_0 \cdots x_i$, ta sinh các token còn lại $\hat{x}_{i+1} \cdots \hat{x}_{k+1}$ để chuỗi trở nên hoàn chỉnh và mạch lạc.

Chú giải của người dịch: Sinh văn bản từng token một

Phương trình (6.187–6.188) mô tả cách "viết tiếp": với ngữ cảnh "Hôm nay trời", mô hình cho xác suất các từ tiếp theo, ví dụ "đẹp" 0.4, "mưa" 0.3, "nắng" 0.2…; arg max chọn từ có xác suất lớn nhất ("đẹp"). Rồi "Hôm nay trời đẹp" trở thành ngữ cảnh mới để đoán từ kế tiếp, cứ thế cho tới khi đủ câu. Dấu mũ $\hat{}$ nhắc rằng đây là token do mô hình *tự đoán*, không phải token cho sẵn.

Như đã bàn ở Mục 6.1.1, bộ giải mã Transformer về bản chất là mô hình ngôn ngữ. Một khác biệt then chốt giữa bài toán giải mã trong Transformer bộ mã hóa–bộ giải mã và bài toán mô hình hóa ngôn ngữ là bộ giải mã Transformer đưa ra dự đoán có điều kiện trên các token ngữ cảnh ở cả phía bộ mã hóa lẫn phía bộ giải mã, thay vì chỉ có điều kiện trên các token đứng trước. Để sửa bộ giải mã

Transformer thành một mô hình ngôn ngữ chuẩn, ta chỉ cần bỏ các tầng con chú ý chéo, và một khối giải mã Transformer có thể được biểu diễn là

$$\mathbf{S}^l = \text{Layer}_{\text{ffn}}(\mathbf{S}^l_{\text{self}}), \tag{6.189}$$

$$\mathbf{S}^l_{\text{self}} = \text{Layer}_{\text{self}}(\mathbf{S}^{l-1}). \tag{6.190}$$

Ở đây, $\mathbf{S}^l$ ký hiệu đầu ra của khối ở độ sâu l. $\text{Layer}_{\text{self}}(\cdot)$ ký hiệu tầng con tự chú ý, và $\text{Layer}_{\text{ffn}}(\cdot)$ ký hiệu tầng con FFN. Ta thấy khối giải mã này có cùng dạng với một khối mã hóa. Khác biệt giữa khối giải mã và khối mã hóa nằm ở chiến lược che được dùng khi huấn luyện: khối giải mã che sự chú ý từ một vị trí i tới mọi vị trí ngữ cảnh bên phải $k > i$, còn khối mã hóa không có ràng buộc này. Một tầng Softmax được đặt lên trên khối cuối cùng và được dùng để tạo ra phân phối trên bộ từ vựng tại mỗi vị trí. Khi suy luận, bộ giải mã Transformer hoạt động theo kiểu tự hồi quy, như mô tả trong Phương trình (6.188).

Việc huấn luyện các mô hình này theo một mô thức chuẩn. Ta học mô hình bằng cách cập nhật tham số nhiều lần, dựa trên gradient của hàm mất mát trên các mẫu huấn luyện. Mô thức này có thể được mở rộng cho việc huấn luyện các mô hình ngôn ngữ lớn dựa trên Transformer, vốn đã được áp dụng rộng rãi trong AI tạo sinh. Tuy nhiên, huấn luyện mô hình Transformer ở quy mô lớn, bao gồm mô hình chỉ có bộ giải mã, chỉ có bộ mã hóa và bộ mã hóa–bộ giải mã, có thể dẫn đến những khó khăn mới, chẳng hạn sự bất ổn định khi huấn luyện. Chúng tôi sẽ bàn thêm về những vấn đề này trong các chương sau, nơi tiền huấn luyện (*pre-training*) quy mô lớn là trọng tâm chính.

6.5.2 Mã hóa văn bản

Với nhiều bài toán NLP, một mô thức được dùng rộng rãi là trước hết học một biểu diễn của chuỗi đầu vào, rồi đưa ra dự đoán cho các tác vụ phía sau dựa trên biểu diễn này. Nhờ đó, ta tách việc biểu diễn chuỗi khỏi các tác vụ NLP cụ thể. Một ưu điểm của mô thức này là ta có thể huấn luyện một mô hình chuỗi không chuyên biệt cho bài toán cụ thể nào, nhờ đó nó có thể tổng quát hóa tốt.

Rõ ràng, bộ mã hóa Transformer là một loại mô hình chuỗi và có thể được dùng làm bộ mã hóa văn bản. Xét một bộ mã hóa Transformer với L tầng mã hóa. Đầu ra của tầng mã hóa cuối cùng có thể được xem là biểu diễn của chuỗi. Ở đây, ta thêm một token đặc biệt x_0 vào đầu mỗi chuỗi, cho biết điểm bắt đầu của chuỗi (thường được viết là $\langle\text{SOS}\rangle$ hoặc [CLS]). Cho một chuỗi $m+1$ token đầu vào $x_0, x_1, \cdots, x_m$, đầu ra của bộ mã hóa sẽ là một chuỗi $m+1$ vectơ $\mathbf{h}^L_0, \mathbf{h}^L_1, \cdots, \mathbf{h}^L_m$. Vì x_0 không ứng với một từ thực sự nào và có vectơ nhúng vị trí cố định, nó đóng vai trò như một token biểu diễn toàn cục, thu thập thông tin từ các vị trí khác thông qua cơ chế tự chú ý. Do đó, $\mathbf{h}^L_0$ đóng vai trò biểu diễn của toàn bộ chuỗi, mà không thiên lệch về bất kỳ token hay vị trí cụ thể nào. Trong nhiều trường hợp, ta cần một biểu diễn duy nhất của chuỗi để làm đầu vào cho các thành phần phía sau của hệ thống. Ví dụ, ta có thể xây dựng một hệ thống phân loại câu dựa trên một vectơ duy nhất được sinh ra từ $\{\mathbf{h}^L_0, \ldots, \mathbf{h}^L_m\}$. Trong trường hợp này, ta có thể đơn giản dùng $\mathbf{h}^L_0$ làm biểu diễn của chuỗi. Một cách tổng quát hơn là thêm một tầng gộp (pooling) vào bộ mã hóa, cho phép ta thử các phương pháp gộp khác nhau để sinh vectơ nhúng của chuỗi từ $\{\mathbf{h}^L_0, \cdots, \mathbf{h}^L_m\}$.

Trong mã hóa văn bản, các chuỗi token được biểu diễn bằng các vectơ giá trị thực, thường được gọi là biểu diễn câu (*sentence representation*) hay vectơ nhúng câu (*sentence embedding*), có thể được hiểu như các điểm trong một không gian nhiều chiều [Hill và cộng sự, 2016]. Do đó, một ứng dụng khác của mã hóa văn bản là tính độ tương đồng ngữ nghĩa hoặc cú pháp giữa các chuỗi token dựa trên mức độ gần nhau tương đối của chúng trong không gian này. Một phương pháp trực tiếp cho việc này là tính khoảng cách Euclid hoặc độ tương đồng cosin giữa các vectơ nhúng của chuỗi. Khoảng cách giữa hai

chuỗi càng ngắn thì chúng được coi là càng giống nhau. Có nhiều độ đo khoảng cách khác nhau, và có thể kết hợp chúng để có một thước đo độ tương đồng chuỗi vững chắc hơn. Những phép tính độ tương đồng như vậy được áp dụng trong các lĩnh vực như suy diễn văn bản (textual entailment), truy xuất thông tin và đánh giá bản dịch [Cer và cộng sự, 2018; Reimers và Gurevych, 2019]. Ngoài ra, chúng cũng thường được dùng để đánh giá chất lượng nội tại của các mô hình mã hóa văn bản.

Chú giải của người dịch: Độ tương đồng cosin

Ở lớp 10 ta đã biết $\cos(\vec{u}, \vec{v}) = \frac{\vec{u}\cdot\vec{v}}{|\vec{u}|\,|\vec{v}|}$. Công thức y hệt áp dụng cho vectơ nhiều chiều. Hai câu có nghĩa gần nhau ("Never give up." — "Không bao giờ bỏ cuộc." và "Never say die!" — "Đừng bao giờ nản chí!") có vectơ nhúng gần cùng hướng, cosin gần 1; hai câu chẳng liên quan cho cosin gần 0. Ví dụ với $\vec{u} = (3; 4)$, $\vec{v} = (4; 3)$: $\cos = \frac{24}{5\cdot 5} = 0.96$, rất giống nhau.

Mã hóa văn bản cũng là một thành phần then chốt của các mô hình chuỗi-sang-chuỗi. Trong các tác vụ chuỗi-sang-chuỗi, ta có thể phát triển một bộ mã hóa Transformer riêng để mô hình hóa chuỗi phía nguồn trong một hệ thống bộ mã hóa–bộ giải mã (xem Hình 6.18). Chẳng hạn, ta có thể tiền huấn luyện một bộ mã hóa Transformer trên một kho ngữ liệu lớn gồm văn bản phía nguồn, rồi dùng nó làm bộ mã hóa trong một mô hình bộ mã hóa–bộ giải mã ở phía sau. Đáng chú ý là, mặc dù bộ mã hóa dựa trên kiến trúc Transformer, bộ giải mã không bị giới hạn trong kiến trúc này. Sự linh hoạt này cho phép ta đưa các bộ mã hóa Transformer đã tiền huấn luyện vào các kiến trúc chuỗi-sang-chuỗi lai, chẳng hạn các hệ thống kết hợp một bộ mã hóa Transformer với một bộ giải mã LSTM.

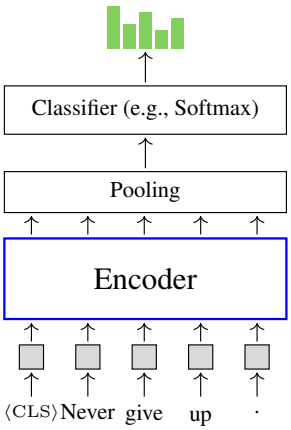
Ghi chú nhãn hình (người dịch): (a) *Classification*: phân loại; *Classifier (e.g., Softmax)*: bộ phân loại (ví dụ Softmax); *Pooling*: gộp; (b) *Similarity Computation (e.g., FFN)*: tính độ tương đồng (ví dụ FFN), *Similarity* $= 0.7$: độ tương đồng bằng 0.7; (c) *Sequence-to-Sequence Modeling*: mô hình hóa chuỗi-sang-chuỗi. Câu ví dụ "Never give up." (Không bao giờ bỏ cuộc.), "Never say die!" (Đừng bao giờ nản chí!), và bản dịch tiếng Trung 永不放弃。("không bao giờ từ bỏ").

Trong các tình huống học có giám sát, việc huấn luyện một bộ mã hóa Transformer khá đơn giản. Ta có thể coi nó là một thành phần thông thường của mô hình đích và huấn luyện mô hình này trên dữ liệu có nhãn của tác vụ cụ thể. Tuy nhiên, cách này đòi hỏi bộ mã hóa phải được tối ưu trên từng tác vụ, và bộ mã hóa thu được không phải lúc nào cũng tổng quát hóa tốt sang các tác vụ khác, nhất là khi dữ liệu có nhãn khan hiếm trong hầu hết các trường hợp. Một cách tiếp cận phổ biến hơn là đặt việc huấn luyện bộ mã hóa văn bản thành một tác vụ độc lập, trong đó tín hiệu giám sát được lấy hoàn toàn từ văn bản thô. Điều này dẫn các nhà nghiên cứu tới việc phát triển các bộ mã hóa Transformer tự giám sát (*self-supervised*), như BERT, vốn tận dụng lượng lớn văn bản không nhãn, và các bộ mã hóa này được thấy là tổng quát hóa tốt trên nhiều tác vụ phía sau. Thảo luận thêm về các bộ mã hóa Transformer tiền huấn luyện có thể tìm thấy ở Chương 7.

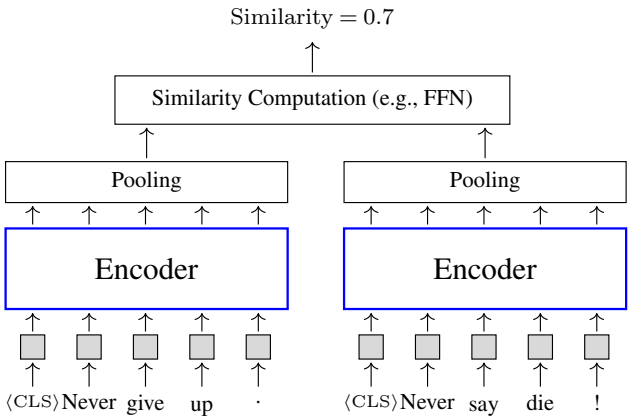
6.5.3 Dịch tiếng nói

Như đã minh họa ở Mục 6.1, mô hình Transformer bộ mã hóa–bộ giải mã chuẩn được đề xuất để mô hình hóa các bài toán chuỗi-sang-chuỗi. Ở đây ta xét bài toán dịch tiếng nói trong một ngôn ngữ thành văn bản trong một ngôn ngữ khác — một bài toán theo truyền thống được giải quyết bằng cả kỹ thuật nhận dạng tiếng nói (*speech recognition*) tự động (ASR) lẫn dịch máy. Thay vì nối tiếp một hệ thống nhận dạng tiếng nói tự động với một hệ thống dịch máy, ta có thể dùng mô hình Transformer để xây dựng một hệ thống dịch tiếng nói sang văn bản (S2T) đầu-cuối (end-to-end), dịch trực tiếp tiếng nói đầu vào thành văn bản đầu ra.

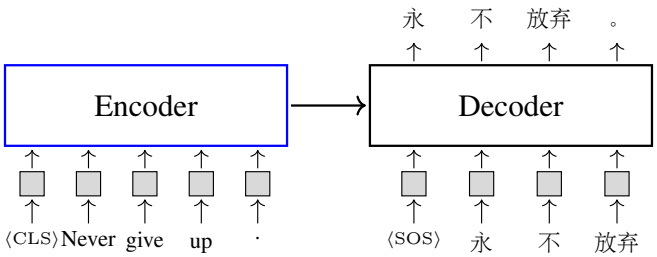
Để đơn giản hóa thảo luận, ta giả sử đầu vào của một hệ thống dịch S2T là một chuỗi các vectơ đặc



(a) Classification



(b) Similarity Computation



(c) Sequence-to-Sequence Modeling

Hình 6.18: Minh họa việc tích hợp bộ mã hóa Transformer làm thành phần của các hệ thống khác nhau. Một cách phổ biến là đưa đầu ra của bộ mã hóa (sau khi gộp) vào một bộ phân loại để có một hệ thống phân loại chuỗi. Một cách khác để sử dụng bộ mã hóa Transformer là tính độ tương đồng giữa hai chuỗi. Ta dùng cùng một bộ mã hóa để biểu diễn hai chuỗi, rồi xây dựng một mạng nơ-ron trên hai biểu diễn đó để tạo ra điểm tương đồng giữa chúng. Ngoài ra, bộ mã hóa Transformer cũng có thể được dùng trong các hệ thống bộ mã hóa–bộ giải mã để mô hình hóa các bài toán chuỗi-sang-chuỗi.

trưng âm học phía nguồn, ký hiệu là $\mathbf{a}_1 \cdots \mathbf{a}_m$, và đầu ra của hệ thống là một chuỗi token phía đích, ký hiệu là $y_1 \cdots y_n$[29]. Ánh xạ $\mathbf{a}_1 \cdots \mathbf{a}_m$ thành $y_1 \cdots y_n$ là một bài toán chuỗi-sang-chuỗi. Vì vậy, có thể dễ dàng mô hình hóa bài toán bằng một mô hình Transformer bộ mã hóa–bộ giải mã, và việc huấn luyện cũng như suy luận của mô hình này là chuẩn, như trong dịch máy nơ-ron.

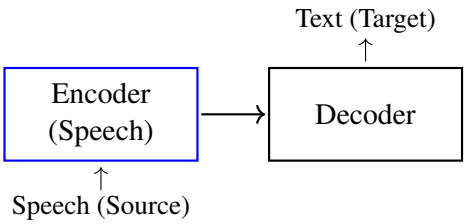
Tuy nhiên, trong dịch S2T, ta phải xử lý các ánh xạ chuỗi vượt qua cả ranh giới phương thức (âm thanh – văn bản) lẫn ngôn ngữ. Điều này đặt ra những thách thức mới so với các bài toán dịch máy thông thường và ảnh hưởng đến thiết kế của các mô hình dịch S2T. Đã có một số cải tiến cho Transformer để nó phù hợp hơn với các tác vụ dịch S2T. Một số cải tiến tập trung vào thiết kế các tầng Transformer [Di Gangi và cộng sự, 2019]. Ví dụ, trong hệ thống của Gulati và cộng sự [2020], một tầng con CNN và vectơ nhúng vị trí tương đối được tích hợp vào mỗi tầng Transformer, giúp mô hình nắm bắt hiệu quả cả đặc trưng cục bộ lẫn toàn cục.

Một hướng nghiên cứu khác về dịch S2T tập trung vào cải tiến kiến trúc bộ mã hóa–bộ giải mã. Việc này bao gồm sửa đổi bộ mã hóa, bộ giải mã, hoặc cả hai. Để minh họa, Hình 6.19 trình bày kiến trúc của ba mô hình dịch S2T. Tất cả đều dựa trên Transformer nhưng có kiến trúc bộ mã hóa khác nhau. Như thấy trong hình, kiến trúc bộ mã hóa–bộ giải mã chuẩn có một bộ mã hóa Transformer để đọc đầu vào phía nguồn $\mathbf{a}_1, \cdots, \mathbf{a}_m$ và một bộ giải mã Transformer để tạo ra đầu ra phía đích $y_1, \cdots, y_n$. Ngược lại, mô hình bộ mã hóa tách rời phân rã bộ mã hóa thành hai bộ mã hóa xếp chồng — một cho mô hình hóa âm học (gọi là **bộ mã hóa tiếng nói** (*speech encoder*)) và một cho mô hình hóa văn bản (gọi là **bộ mã hóa văn bản** (*text encoder*)) [Liu và cộng sự, 2020d; Xu và cộng sự, 2021a]. Thiết kế này phản ánh một cấu trúc phân cấp trong mô hình hóa, trong đó các biểu diễn ở những mức khác nhau của mạng phụ trách những khía cạnh khác nhau của bài toán. Ví dụ, bộ mã hóa tiếng nói mô hình hóa các đặc trưng mức thấp khi ánh xạ các vectơ nhúng âm học thành các đơn vị ngôn ngữ lớn hơn, còn bộ mã hóa văn bản mô hình hóa các đặc trưng ngữ nghĩa hoặc cú pháp khi biểu diễn toàn bộ chuỗi đầu vào. Một ưu điểm của việc tách riêng bộ mã hóa văn bản là quá trình mã hóa tuân theo tri thức tiên nghiệm của ta: trước hết cần phiên âm tiếng nói đầu vào thành văn bản, rồi dịch bản phiên âm sang ngôn ngữ đích. Do đó, ta có thể huấn luyện bộ mã hóa tiếng nói tương tự như cách huấn luyện một hệ thống ASR. Điều này cho phép ta tiền huấn luyện bộ mã hóa tiếng nói và bộ mã hóa văn bản trên dữ liệu không nhãn, rồi sau đó đưa các bộ mã hóa đã tiền huấn luyện vào hệ thống dịch S2T.

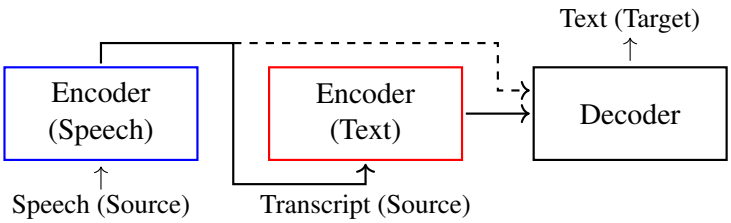
Một kiến trúc mã hóa khác là kiến trúc hai luồng, như trong Hình 6.19(c). Giống kiến trúc bộ mã hóa tách rời, kiến trúc này có một bộ mã hóa tiếng nói và một bộ mã hóa văn bản, nhưng hai bộ mã hóa hoạt động song song thay vì nối tiếp [Ye và cộng sự, 2021]. Bộ mã hóa tiếng nói nhận đặc trưng âm học làm đầu vào, còn bộ mã hóa văn bản nhận các token (hoặc vectơ nhúng của chúng) làm đầu vào. Một bộ mã hóa thứ ba, gọi là **bộ mã hóa dùng chung** (*shared encoder*), tích hợp đầu ra của cả bộ mã hóa tiếng nói lẫn bộ mã hóa văn bản, hợp nhất các biểu diễn từ hai phương thức. Kiến trúc hai luồng này linh hoạt vì nó cung cấp nhiều cách để huấn luyện mô hình dịch S2T. Một cách phổ biến là huấn luyện riêng từng nhánh. Ví dụ, nếu ta che bộ mã hóa tiếng nói, mô hình sẽ trở thành một mô hình dịch máy có thể huấn luyện bằng văn bản song ngữ. Ngược lại, nếu che bộ mã hóa văn bản, ta có thể huấn luyện mô hình như một mô hình dịch S2T chuẩn. Khi suy luận, có thể bỏ bộ mã hóa văn bản, và đầu vào tiếng nói được mô hình hóa bằng bộ mã hóa tiếng nói và bộ mã hóa dùng chung.

Ghi chú nhãn hình (người dịch): Encoder (Speech)/(Text): bộ mã hóa (tiếng nói)/(văn bản); *Shared Encoder*: bộ mã hóa dùng chung; *Decoder*: bộ giải mã; *Speech (Source)*: tiếng nói (nguồn); *Transcript (Source)*: bản phiên âm (nguồn); *Text (Target)*: văn bản (đích). (a) Một bộ mã hóa + một bộ giải mã; (b)

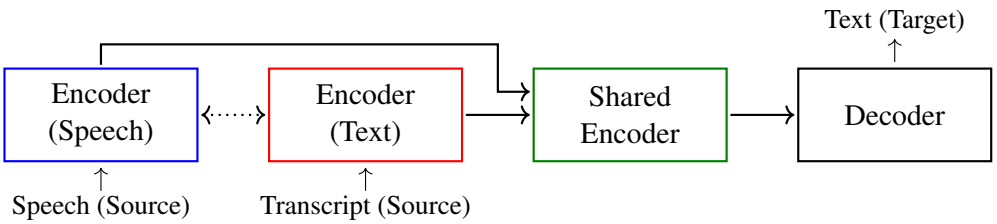
[29]Để có chuỗi đầu vào cho hệ thống, ta cần rời rạc hóa tiếng nói liên tục thành các tín hiệu được biểu diễn bằng vectơ đặc trưng. Quá trình này thường không đơn giản, đòi hỏi hoặc một bộ trích xuất đặc trưng dựa trên nhiều phép xử lý tín hiệu, hoặc một mạng nơ-ron học ánh xạ đặc trưng theo cách đầu-cuối. Nhưng chúng tôi sẽ không đi sâu vào chi tiết các phương pháp này mà đơn giản coi bộ trích xuất đặc trưng đầu vào là một hệ thống phía trước.



(a) Single Encoder + Single Decoder



(b) Decoupled Encoder + Single Decoder



(c) Two-stream Encoder + Single Decoder

Hình 6.19: Sơ đồ minh họa các mô hình dịch tiếng nói sang văn bản dựa trên Transformer. Ngoài kiến trúc bộ mã hóa–bộ giải mã chuẩn, ta có thể mô hình hóa tường minh thông tin âm học và thông tin văn bản (ngữ nghĩa) bằng hai bộ mã hóa riêng, gọi là bộ mã hóa tiếng nói và bộ mã hóa văn bản. Trong kiến trúc bộ mã hóa tách rời, hai bộ mã hóa được xếp chồng, tức là việc mã hóa văn bản là quá trình diễn ra sau mã hóa tiếng nói. Trong kiến trúc bộ mã hóa hai luồng, hai bộ mã hóa hoạt động song song, và đầu ra của chúng được hợp nhất bằng một bộ mã hóa bổ sung, gọi là bộ mã hóa dùng chung. Đường chấm thể hiện khả năng tương tác giữa hai bộ mã hóa. Ví dụ, ta có thể định nghĩa một hàm mất mát để cực tiểu hóa sự khác biệt giữa đầu ra của chúng, từ đó hướng mô hình tới các biểu diễn được căn chỉnh tốt hơn.

Bộ mã hóa tách rời + một bộ giải mã; (c) Bộ mã hóa hai luồng + một bộ giải mã.

Trong học sâu, huấn luyện thường gắn liền với thiết kế kiến trúc. Vì ta xử lý dữ liệu ở hai phương thức và hai ngôn ngữ, ta có thể xây dựng nhiều tín hiệu giám sát cho việc học đa nhiệm của các mô hình dịch S2T. Một phương pháp được dùng rộng rãi là đưa một hàm mất mát liên quan đến ASR vào việc huấn luyện bộ mã hóa tiếng nói. Ví dụ, trong mô hình bộ mã hóa tách rời, có thể xây dựng một bộ phân loại dựa trên đầu ra của bộ mã hóa tiếng nói. Bằng cách cực tiểu hóa hàm mất mát phân loại thời gian kết nối (*connectionist temporal classification*) (CTC) cho bộ phân loại này, bộ mã hóa tiếng nói có thể được tối ưu theo cách tương tự ASR. Nói chung, huấn luyện mô hình dịch S2T là khó vì dữ liệu tiếng nói–văn bản được căn chỉnh rất khan hiếm. Các giải pháp điển hình cho thách thức này gồm tăng cường dữ liệu, tiền huấn luyện, chưng cất tri thức với dịch máy, v.v. Tuy nhiên, thảo luận sâu về các phương pháp này vượt ra ngoài phạm vi của mục về Transformer này. Bạn đọc quan tâm có thể tham khảo một bài tổng quan gần đây về dịch tiếng nói để biết thêm chi tiết [Xu và cộng sự, 2023a].

6.5.4 Mô hình thị giác

Mặc dù Transformer lần đầu được dùng trong NLP, việc ứng dụng nó sang các lĩnh vực khác đã trở thành một chủ đề nghiên cứu nổi bật. Ví dụ, trong thị giác máy tính, có một xu hướng đáng chú ý là chuyển từ CNN sang Transformer làm mô hình xương sống. Trong tiểu mục này, ta xét **Vision Transformer** (*Vision Transformer*) (**ViT**) — một ứng dụng thú vị của Transformer vào phân loại ảnh [Dosovitskiy và cộng sự, 2021]. Vision Transformer là một mô hình mang tính cột mốc, mở đường cho các mô hình thị giác thuần Transformer. Ở đây, ta xét cấu trúc cơ bản của Vision Transformer để giữ mục này tập trung và mạch lạc, dù đã có rất nhiều tài liệu về Vision Transformer và các biến thể của nó. Các thảo luận chi tiết hơn về mô hình thị giác có thể tìm thấy trong các bài tổng quan gần đây [Han và cộng sự, 2022; Liu và cộng sự, 2023e].

Ý tưởng cốt lõi của Vision Transformer là biến một bức ảnh thành một chuỗi các token thị giác, rồi đưa chuỗi này vào một bộ mã hóa Transformer để sinh ra biểu diễn của ảnh. Bộ mã hóa Transformer là chuẩn, nên chúng tôi không bàn lại ở đây, vì đã giới thiệu Transformer trong chương này. Ánh xạ một ảnh 2 chiều thành một chuỗi token đòi hỏi thêm một số xử lý. Giả sử ta có một ảnh được biểu diễn dưới dạng một bản đồ đặc trưng $H \times W \times C$, trong đó H là chiều cao, W là chiều rộng, và C là số kênh. Bước đầu tiên là chia ảnh này thành một tập các **mảnh** (*patch*). Giả sử mọi mảnh đều là hình vuông cạnh P. Khi đó, các mảnh thu được có thể được biểu diễn bằng các bản đồ đặc trưng có hình dạng $P \times P \times C$. Bằng cách sắp xếp các mảnh này theo một thứ tự cụ thể, ta thu được một chuỗi gồm $\frac{HW}{P^2}$ mảnh, mỗi mảnh được coi như một "token".

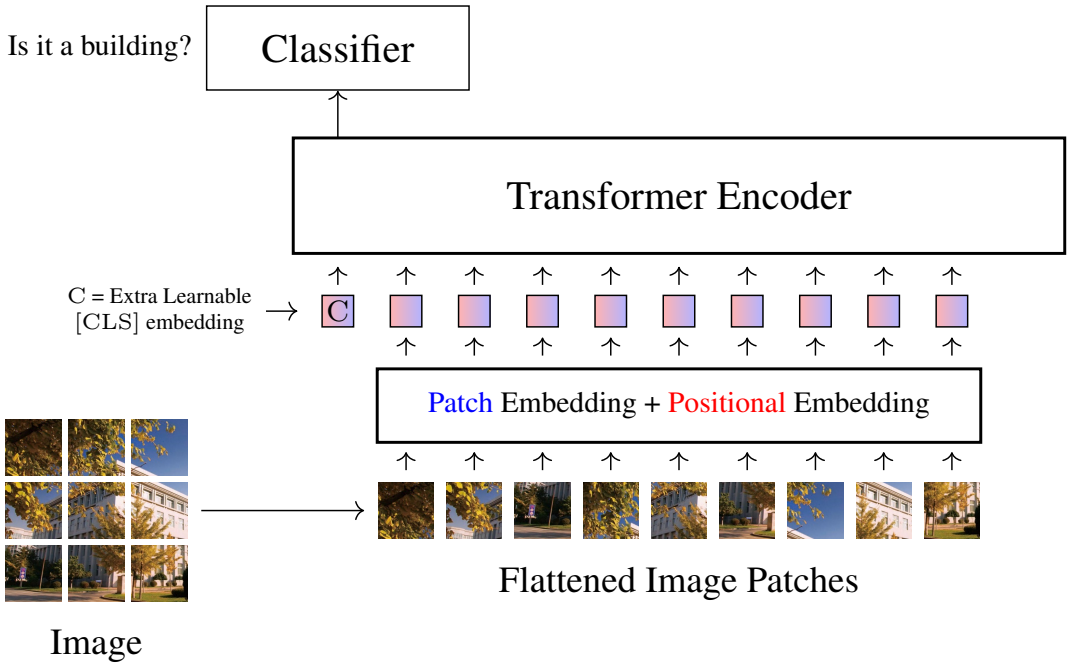
Chú giải của người dịch: Ví dụ đếm số mảnh

Một ảnh màu 224×224 điểm ảnh có $C = 3$ kênh (đỏ, lục, lam). Chia thành các mảnh 16×16 thì mỗi chiều có $224/16 = 14$ mảnh, tổng cộng $14 \times 14 = 196$ mảnh, đúng bằng $\frac{HW}{P^2} = \frac{224 \cdot 224}{256}$. "Làm phẳng" một mảnh nghĩa là xếp $16 \times 16 \times 3 = 768$ con số của nó thành một hàng dài, rồi nhân với một ma trận để được vectơ nhúng d chiều. Như vậy bức ảnh trở thành một "câu" gồm 196 "từ".

Với chuỗi mảnh này, các bước tiếp theo khá đơn giản. Với mảnh ở mỗi vị trí, ta thu được một vectơ nhúng d chiều thông qua một phép biến đổi tuyến tính của mảnh đã được làm phẳng. Đầu vào của bộ mã hóa Transformer là một chuỗi vectơ d chiều, mỗi vectơ là tổng của vectơ nhúng mảnh tương ứng và vectơ nhúng vị trí. Hình 6.20 minh họa các bước chia mảnh và nhúng trong Vision Transformer.

Khi đã có một chuỗi vectơ biểu diễn ảnh, ta có thể dùng bộ mã hóa Transformer để mã hóa chuỗi. Quá trình mã hóa hoàn toàn giống như trong mã hóa văn bản, như đã bàn ở Mục 6.5.2. Với các bài toán

phân loại, ta chỉ cần một biểu diễn duy nhất của đầu vào. Cách tiện lợi là lấy đầu ra của bộ mã hóa tại vị trí 0 (ký hiệu $\mathbf{h}_0^L$) và đưa nó vào một bộ phân loại. Vì token đầu tiên ([CLS]) đóng vai trò một token toàn cục đặc biệt được mọi token khác chú ý tới, $\mathbf{h}_0^L$ có thể được xem là một biểu diễn tổng hợp của toàn bộ chuỗi.



Hình 6.20: Minh họa Vision Transformer cho phân loại ảnh [Dosovitskiy và cộng sự, 2021]. Có ba bước. Ở bước thứ nhất, ảnh đầu vào được chia thành các mảnh, sau đó các mảnh được làm phẳng và ánh xạ thành vectơ nhúng. Ở bước thứ hai, một bộ mã hóa Transformer được dùng để xử lý chuỗi vectơ nhúng, biểu diễn ảnh dưới dạng một vectơ giá trị thực (ví dụ đầu ra của bộ mã hóa tại vị trí đầu tiên). Ở bước cuối, một bộ phân loại được xây dựng trên biểu diễn ảnh này.

Ghi chú nhãn hình (người dịch): Image: ảnh; *Flattened Image Patches*: các mảnh ảnh đã làm phẳng; *Patch Embedding + Positional Embedding*: nhúng mảnh + nhúng vị trí; *C = Extra Learnable [CLS] embedding*: C là vectơ nhúng [CLS] bổ sung, học được; *Classifier*: bộ phân loại; *Is it a building?*: Đây có phải tòa nhà không?

Một cách chuẩn để huấn luyện Vision Transformer là cực tiểu hóa một hàm mất mát theo tác vụ trên dữ liệu có nhãn, chẳng hạn ImageNet. Gần đây hơn, lấy cảm hứng từ học tự giám sát trong các mô hình kiểu BERT, đã có những nỗ lực thành công trong việc huấn luyện các bộ mã hóa ảnh dựa trên Transformer trên dữ liệu không nhãn quy mô lớn [Caron và cộng sự, 2021; Bao và cộng sự, 2021; He và cộng sự, 2022]. Lưu ý rằng một trong những đóng góp quan trọng nhất của Vision Transformer là nó thống nhất các mô hình biểu diễn cho các phương thức khác nhau. Điều này gợi ý rằng nếu một đối tượng, dù là ảnh hay văn bản, được biểu diễn dưới dạng một chuỗi vectơ nhúng, thì nó có thể được mô hình hóa dễ dàng bằng kiến trúc Transformer.

6.5.5 Mô hình đa phương thức

Thảo luận ở trên về Vision Transformer mở ra khả năng thống nhất các biểu diễn từ nhiều phương thức bằng cùng một kiến trúc Transformer. Thực tế, nhiều hệ thống đa phương thức gần đây lấy cảm hứng rất lớn từ Transformer [Xu và cộng sự, 2023c]. Các hệ thống như vậy chuyển các đối tượng thuộc những phương thức khác nhau thành các chuỗi vectơ và đưa các vectơ này vào một mô hình Transformer duy nhất. Đầu ra là một biểu diễn hợp nhất của mọi đầu vào, có thể được dùng trong các hệ thống phía sau.

Để lấy ví dụ đơn giản, xét tác vụ mã hóa một cặp gồm một văn bản và bức ảnh tương ứng. Trước hết, ta biểu diễn cả văn bản và ảnh dưới dạng các chuỗi vectơ nhúng có cùng số chiều. Đây là một bước thông thường trong mô hình hóa chuỗi mà ta đã gặp nhiều lần. Ta có thể làm điều này bằng một mô hình nhúng đơn giản (ví dụ mô hình nhúng từ hoặc nhúng mảnh) hoặc một mô hình đã tiền huấn luyện (ví dụ một mô hình thị giác). Sau đó, hai chuỗi này được nối thành một chuỗi dài gồm cả vectơ nhúng văn bản lẫn vectơ nhúng thị giác. Bước tiếp theo là chuẩn: một bộ mã hóa Transformer nhận chuỗi vectơ nhúng đã nối làm đầu vào và tạo ra biểu diễn của văn bản và ảnh ở đầu ra. Lưu ý rằng nối chuỗi văn bản và chuỗi thị giác là một trong những phương pháp đơn giản nhất cho mô hình hóa thị giác–văn bản. Có một số cách khác để hợp nhất thông tin từ các phương thức khác nhau. Ví dụ, ta có thể đưa các biểu diễn thị giác vào các tầng chú ý của một bộ mã hóa hoặc bộ giải mã văn bản [Li và cộng sự, 2022e; Alayrac và cộng sự, 2022].

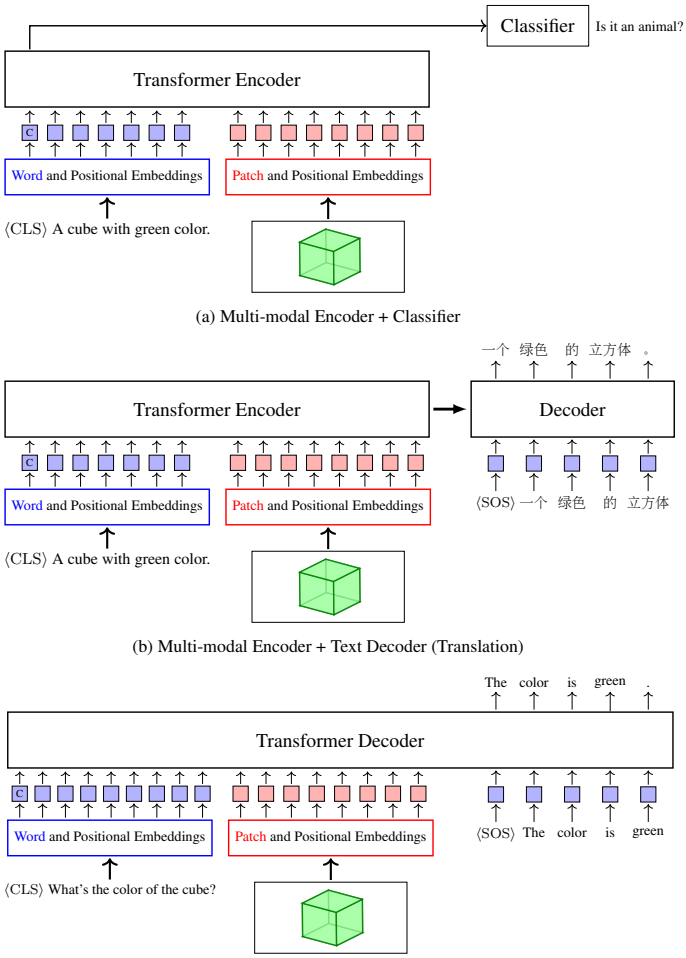
Bộ mã hóa đa phương thức ở trên có thể được dùng trong cả hệ thống chỉ có bộ mã hóa lẫn hệ thống bộ mã hóa–bộ giải mã. Với hệ thống chỉ có bộ mã hóa, xét ví dụ: cho một bức ảnh và mô tả của nó, ta dự đoán lớp của ảnh bằng một bộ phân loại xây dựng trên bộ mã hóa [Kim và cộng sự, 2021]. Với hệ thống bộ mã hóa–bộ giải mã, ta ghép bộ mã hóa với một bộ giải mã, như trong mô hình hóa chuỗi-sang-chuỗi [Cho và cộng sự, 2021]. Ví dụ, ta có thể dùng một bộ giải mã Transformer để sinh văn bản dựa trên đầu ra của bộ mã hóa. Một ứng dụng phổ biến của kiến trúc này là **hỏi đáp thị giác** (*visual question answering*) (**VQA**), trong đó hệ thống được cho một bức ảnh và một câu hỏi về bức ảnh, và có nhiệm vụ sinh ra câu trả lời [Antol và cộng sự, 2015]. Kiến trúc của các mô hình này được minh họa trong Hình 6.21 (a–b).

Các mô hình ngôn ngữ lớn cũng đã được mở rộng để xử lý cả văn bản và các phương thức khác như ảnh, video và âm thanh, dẫn tới những tiến bộ mới trong xử lý đa phương thức [Liu và cộng sự, 2023a; Yin và cộng sự, 2023]. Bằng cách biểu diễn mọi đầu vào dưới dạng một chuỗi vectơ nhúng token, bài toán trở nên đơn giản: ta dự đoán token tiếp theo khi biết ngữ cảnh của nó. Điều này có thể thực hiện bằng các hệ thống chỉ có bộ giải mã, như trong Hình 6.21 (c).

Ghi chú nhãn hình (người dịch): Word and Positional Embeddings: nhúng từ và nhúng vị trí; *Patch and Positional Embeddings*: nhúng mảnh và nhúng vị trí. (a) *Multi-modal Encoder + Classifier*: bộ mã hóa đa phương thức + bộ phân loại; văn bản "A cube with green color." (Một khối lập phương màu xanh lục.), câu hỏi "Is it an animal?" (Đây có phải con vật không?). (b) *Multi-modal Encoder + Text Decoder (Translation)*: bộ mã hóa đa phương thức + bộ giải mã văn bản (dịch), đầu ra tiếng Trung 一个绿色的立方体。("một khối lập phương màu xanh lục"). (c) *Multi-modal Decoder (Language Modeling)*: bộ giải mã đa phương thức (mô hình hóa ngôn ngữ); câu hỏi "What's the color of the cube?" (Khối lập phương màu gì?), câu trả lời "The color is green." (Màu xanh lục.).

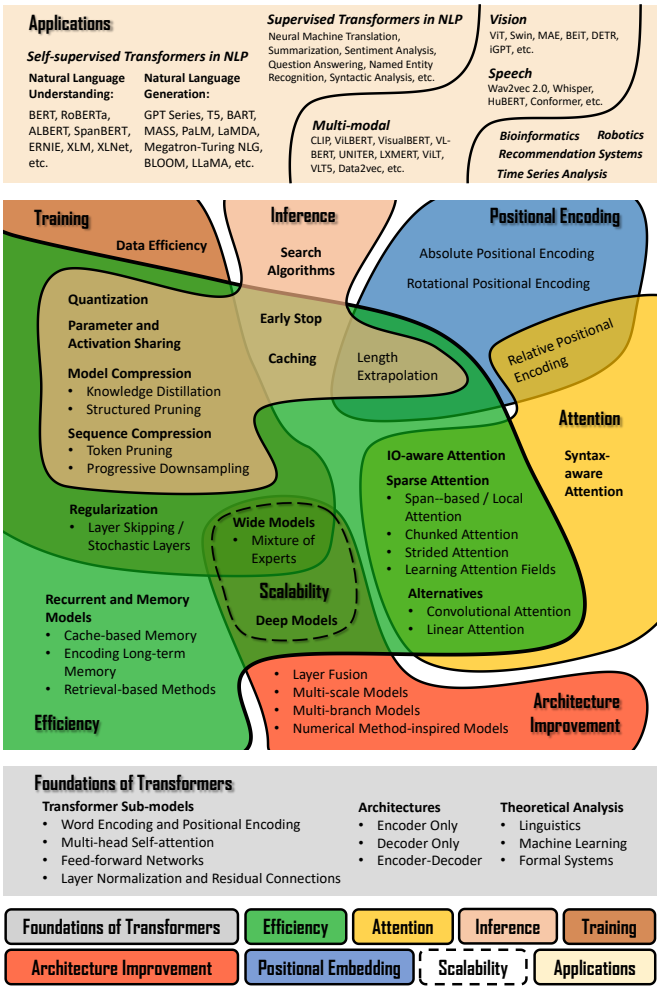
6.6 Tóm tắt

Mô hình Transformer đã được áp dụng rộng rãi kể từ khi kiến trúc này lần đầu được Vaswani và cộng sự [2017] đề xuất. Sự phổ biến này đã thúc đẩy sự phát triển của rất nhiều thuật toán, hệ thống và khái



Hình 6.21: Minh họa các mô hình thị giác–văn bản. Các ô màu xanh lam biểu diễn vectơ nhúng từ + vị trí, còn các ô màu đỏ biểu diễn vectơ nhúng mảnh ảnh + vị trí.

niệm. Một thảo luận thấu đáo về Transformer đòi hỏi bao quát rất rộng, nên không thể đề cập mọi chủ đề hay đưa ra một danh sách tài liệu tham khảo đầy đủ. Hình 6.22 đưa ra một cái nhìn tổng quan ở mức cao về bức tranh Transformer. Lưu ý rằng các mô hình và kỹ thuật liên quan này có thể được phân loại theo nhiều cách, và chúng tôi trình bày một góc nhìn như vậy. Tóm lại, chúng tôi nhấn mạnh các điểm sau:



Hình 6.22: Tổng quan về Transformer.

Ghi chú nhãn hình (người dịch): Các vùng màu ứng với các nhóm chủ đề: *Foundations of Transformers* (nền tảng của Transformer), *Efficiency* (hiệu quả), *Attention* (cơ chế chú ý), *Inference* (suy luận), *Training* (huấn luyện), *Architecture Improvement* (cải tiến kiến trúc), *Positional Embedding* (nhúng vị trí), *Scalability* (khả năng mở rộng quy mô), *Applications* (ứng dụng). Bên trong là các kỹ thuật đã bàn trong chương, ví dụ *Quantization* (lượng tử hóa), *Knowledge Distillation* (chưng cất tri thức), *Structured Pruning* (cắt tỉa có cấu trúc), *Sparse Attention* (chú ý thưa), *Linear Attention* (chú ý tuyến tính), *Mixture of Experts* (hỗn hợp chuyên gia), *Early Stop* (dừng sớm), *Caching* (bộ nhớ đệm), *Length Extrapolation* (ngoại suy độ dài), cùng tên các mô hình tiêu biểu.

- **Nền tảng của Transformer**. Mặc dù tác động của Transformer mang tính cách mạng, chúng

không phải là mô hình hoàn toàn "mới". Từ góc độ học sâu, Transformer được ghép từ những khối xây dựng thông dụng, bao gồm nhúng từ (*word embedding*) và nhúng vị trí [Bengio và cộng sự, 2003a; Mikolov và cộng sự, 2013c; Gehring và cộng sự, 2017b], cơ chế chú ý [Bahdanau và cộng sự, 2014; Luong và cộng sự, 2015], kết nối tắt (*residual connection*) [He và cộng sự, 2016b], chuẩn hóa tầng (*layer normalization*) [Ba và cộng sự, 2016], v.v. Nhiều thành phần trong số này đã xuất hiện ở các hệ thống trước đó; ví dụ, những ý tưởng tương tự chú ý QKV có thể thấy trong mạng bộ nhớ [Sukhbaatar và cộng sự, 2015] và mạng chú ý phân cấp [Yang và cộng sự, 2016]. Transformer mang đến một cách tiếp cận mới để tích hợp các thành phần này, tạo ra một kiến trúc độc đáo. Ví dụ, trong Transformer, sự kết hợp giữa chú ý đa đầu (*multi-head attention*) và chú ý QKV dạng tích vô hướng, cùng với việc đưa vào chuẩn hóa tầng và kết nối tắt, đã tạo ra một khối mạng nơ-ron đặc trưng, cụ thể là tầng con tự chú ý. Thiết kế này từ đó đã trở thành tiêu chuẩn trên thực tế trong nhiều hệ thống mô hình hóa chuỗi về sau.

- **Mô hình chú ý**. Thành công của Transformer trong các tác vụ NLP phần lớn được quy cho việc sử dụng tự chú ý đa đầu cho mô hình hóa chuỗi. Điều này dẫn tới sự quan tâm bùng nổ đối với việc cải tiến cơ chế chú ý trong Transformer. Dù không thể trình bày chi tiết mọi mô hình chú ý, có một số hướng nghiên cứu đáng chú ý. Một hướng nổi bật là sửa đổi dạng của chú ý QKV và chú ý đa đầu để cải thiện hiệu năng. Phạm vi của hướng này rất rộng, vì có vô số khía cạnh cần xem xét khi cải tiến Transformer [Lin và cộng sự, 2022a]. Ví dụ, ta có thể thêm các thành phần mới vào các tầng con tự chú ý để thích ứng chúng với các tác vụ cụ thể, tạo ra nhiều biến thể Transformer khác nhau. Hướng thứ hai là đưa tri thức tiên nghiệm vào thiết kế mô hình chú ý. Điều này hợp lý, vì phần lớn trọng tâm của NLP truyền thống là dùng hiểu biết ngôn ngữ học để định hướng thiết kế hệ thống, và nói chung ta muốn các hệ thống NLP có thể giải thích được về mặt ngôn ngữ học. Ví dụ, nhiều hệ thống dựa trên Transformer nhận cây phân tích cú pháp làm đầu vào dưới nhiều dạng và tận dụng cú pháp trong mô hình hóa chuỗi. Hướng thứ ba là phát triển các mô hình chú ý hiệu quả [Tay và cộng sự, 2020b]. Tự chú ý từ lâu đã bị chỉ trích vì độ phức tạp thời gian bậc hai và việc mỗi token mới phụ thuộc vào mọi token trước đó. Để khắc phục, nhiều nhà nghiên cứu tập trung đơn giản hóa cấu trúc của tự chú ý, hoặc xấp xỉ nó bằng các mô hình thưa hay mô hình hồi quy. Mối quan tâm về hiệu quả này cũng thúc đẩy việc phát triển các phương án thay thế cho tự chú ý, chẳng hạn các mô hình chú ý có độ phức tạp thời gian tuyến tính. Bên cạnh việc khám phá các mô hình chú ý mạnh hơn và hiệu quả hơn, một điều tự nhiên là xem xét những tri thức nào được các mô hình đó học được. Thú vị là, các nhà nghiên cứu đã phát hiện rằng cấu trúc nền tảng của ngôn ngữ có thể được học bởi các mô hình tự chú ý đa đầu, dù các mô hình này không được huấn luyện để biểu diễn tri thức đó [Manning và cộng sự, 2020].

- **Nhúng từ và nhúng vị trí**. Transformer biểu diễn mỗi từ (hay token) đầu vào bằng sự kết hợp của vectơ nhúng từ và vectơ nhúng vị trí. Việc học các vectơ nhúng từ này không phải là điều riêng có của Transformer; ta có thể dùng các vectơ nhúng đã tiền huấn luyện, như Word2Vec hay GloVe, hoặc coi chúng là các tham số học được trong mô hình. Một vấn đề liên quan mật thiết là tách token cho chuỗi đầu vào. Nói chung, cách tách token quyết định số token thu được và ảnh hưởng đến độ khó của việc học các vectơ nhúng tương ứng. Do đó, chọn một phương pháp tách token phù hợp là rất quan trọng với nhiều ứng dụng. Hơn nữa, nhúng vị trí đóng vai trò thiết yếu trong Transformer, vì cơ chế chú ý bên dưới về thiết kế là không nhạy với thứ tự [Dufter và cộng sự, 2022]. Mặc dù mã hóa vị trí là một chủ đề nghiên cứu chung, nhiều nỗ lực đã được dành để điều chỉnh nó cho Transformer, dẫn tới nhiều sửa đổi kiến trúc khác nhau [Shaw và cộng sự, 2018; Huang và cộng sự, 2018]. Ngoài ra, các nghiên cứu cho thấy với những chuỗi dài hơn đáng kể so với chuỗi gặp khi huấn luyện, có thể đạt khả năng ngoại suy tốt hơn bằng cách thay vectơ nhúng

hình sin bằng vectơ nhúng vị trí xoay (RoPE) hoặc bằng cách co giãn trọng số chú ý bằng các hệ số vô hướng theo vị trí [Raffel và cộng sự, 2020; Su và cộng sự, 2021; Press và cộng sự, 2021].

- **Huấn luyện và mở rộng quy mô mô hình**. Trong kỷ nguyên học sâu, các hệ thống hiệu năng cao thường được xây dựng bằng mạng nơ-ron quy mô lớn. Một cách trực tiếp để tăng năng lực của Transformer là xếp chồng thêm tầng hoặc mở rộng kích thước biểu diễn ẩn. Thực vậy, các mô hình Transformer sâu và rộng luôn vượt trội các phiên bản nhỏ hơn. Tuy nhiên, huấn luyện các mô hình cực lớn đặt ra những thách thức đáng kể, nhất là khi áp dụng hạ gradient (*gradient descent*) trên các tập dữ liệu khổng lồ, đòi hỏi tài nguyên tính toán rất lớn. Các giải pháp kỹ thuật bao gồm phân tán việc huấn luyện trên các cụm máy tính [Lepikhin và cộng sự, 2021; Chowdhery và cộng sự, 2022]. Mặc dù huấn luyện phân tán là một mô thức chung không giới hạn ở Transformer, nó ảnh hưởng mạnh tới thiết kế kiến trúc; chẳng hạn, các mô hình hỗn hợp chuyên gia (MoE) thưa có thể tạo điều kiện huấn luyện với lượng tham số khổng lồ và là nền tảng cho nhiều hệ thống quy mô lớn. Việc mở rộng quy mô cũng cho phép nghiên cứu các định luật co giãn — định lượng cách hiệu năng thay đổi theo kích thước mô hình, khối lượng dữ liệu và ngân sách tính toán [Hestness và cộng sự, 2017; Kaplan và cộng sự, 2020]. Việc mở rộng quy mô này đôi khi đi kèm hiện tượng "trồi sinh (*emergence*)" [Wei và cộng sự, 2022b], trong đó mô hình bộc lộ những năng lực mới không lường trước được. Trong các nghiên cứu gần đây, việc có được những năng lực trồi sinh như vậy được xem là dấu hiệu đặc trưng của các mô hình ngôn ngữ lớn mạnh.
- **Mô hình hiệu quả**. Có những mục tiêu hiệu quả khác nhau. Một hệ thống có thể ưu tiên hiệu quả bộ nhớ với các bài toán bị giới hạn bởi bộ nhớ, hoặc độ trễ thấp khi tốc độ suy luận là then chốt. Cân bằng các yêu cầu này dẫn đến nhiều chiến lược tối ưu khác nhau. Trong Transformer, nhiều tối ưu hóa liên quan đến việc sửa đổi cơ chế chú ý; chẳng hạn, một số biến thể giảm lượng bộ nhớ chiếm dụng khi xử lý chuỗi dài [Tay và cộng sự, 2020b], trong khi một số khác giảm thiểu tính toán để giảm độ trễ. Hơn nữa, Transformer có thể hưởng lợi từ các tối ưu hóa không phụ thuộc kiến trúc vốn điển hình cho mạng nơ-ron, như tính toán có điều kiện, chưng cất tri thức, cắt tỉa có cấu trúc và nén chuỗi. Vấn đề hiệu quả cũng có thể được giải quyết từ góc độ kiến trúc máy tính [Kim và cộng sự, 2023]. Ví dụ, trong các tác vụ chuỗi-sang-chuỗi, quá trình mã hóa thường nặng về tính toán, còn quá trình giải mã tự hồi quy thường nặng về vào/ra (IO). Do đó, ta có thể dùng các phương pháp tối ưu khác nhau cho các thành phần khác nhau của Transformer.
- **Suy luận**. Bài toán suy luận thường được bàn đến trong sinh chuỗi. Trong NLP, ta thường cần tìm giả thuyết "tốt nhất" trong một không gian gồm các chuỗi hàng trăm, thậm chí hàng nghìn token trên một bộ từ vựng. Là một bài toán tìm kiếm kinh điển trong trí tuệ nhân tạo, có thể áp dụng nhiều thuật toán, bao gồm tìm kiếm theo chiều rộng, theo chiều sâu và tìm kiếm A*. Trong nhiều ứng dụng NLP thực tế, hiệu quả của hệ thống tìm kiếm là một yếu tố quan trọng. Vì thế, cần đến các thuật toán tìm kiếm được tối ưu hóa. Phần lớn các thuật toán này đã được khám phá trong dịch máy và ASR, và có thể áp dụng trực tiếp cho các mô hình sinh văn bản nơ-ron như Transformer. Ngoài tìm kiếm truyền thống, các tối ưu hóa gần đây như giải mã suy đoán (*speculative decoding*) được thiết kế riêng cho kiến trúc Transformer [Leviathan và cộng sự, 2023]. Hơn nữa, các mô hình chú ý hiệu quả đã nhắc tới trước đây hiện được triển khai rộng rãi để tăng tốc các mô hình ngôn ngữ lớn và hệ thống dịch máy nơ-ron [Heafield và cộng sự, 2021; Dao và cộng sự, 2023].
- **Ứng dụng**. Các ứng dụng của Transformer bao trùm nhiều bài toán NLP. Trong quá trình phát triển, Transformer thoạt đầu được dùng để xây dựng các mô hình có giám sát thực hiện những tác vụ cụ thể. Về sau, thành công lớn hơn đạt được nhờ dùng chúng làm mạng xương sống cho việc học tự giám sát quy mô lớn các mô hình nền tảng (*foundation model*) [Bommasani và cộng sự, 2021]. Sự chuyển dịch này đã thay đổi rõ rệt mô thức NLP: một mô hình trước hết được tiền

huấn luyện trên lượng văn bản khổng lồ để thu được tri thức ngôn ngữ tổng quát, rồi được điều chỉnh cho các tác vụ phía sau bằng các phương pháp hiệu quả như tinh chỉnh (*fine-tuning*) hoặc dùng prompt. Ngoài NLP, ta đã chứng kiến sự bùng nổ ứng dụng Transformer trong các lĩnh vực như thị giác máy tính, xử lý tiếng nói và tin sinh học. Ý tưởng cốt lõi là biểu diễn các loại dữ liệu đa dạng dưới dạng chuỗi token, mở rộng Transformer thành các mô hình biểu diễn đa năng có khả năng xử lý dữ liệu đa phương thức.

- **Mô hình ngôn ngữ lớn như các mô hình nền tảng**. Transformer là nền tảng của các mô hình ngôn ngữ lớn (*large language model*) gần đây, như dòng GPT, vốn cho thấy những đột phá đáng ngạc nhiên trong NLP [Bubeck và cộng sự, 2023; Yang và cộng sự, 2023a]. Phần lớn nghiên cứu về mô hình ngôn ngữ lớn ít nhiều liên quan đến Transformer. Như đã bàn ở Mục 6.5.1, huấn luyện các mô hình ngôn ngữ này thường đồng nghĩa với huấn luyện các bộ giải mã Transformer quy mô lớn, và những cải tiến kiến trúc cho bộ giải mã trực tiếp mang lại lợi ích cho mô hình ngôn ngữ. Mặt khác, sự phát triển nhanh chóng của mô hình ngôn ngữ lớn đã thúc đẩy thêm đổi mới trong các kỹ thuật liên quan đến Transformer, đặc biệt là phát triển các phương pháp thích ứng hiệu quả và chi phí thấp cho các mô hình khổng lồ.

- **Phân tích lý thuyết**. Trong khi Transformer thống trị về mặt thực nghiệm, nền tảng lý thuyết của chúng trước đây ít được chú trọng hơn so với các cải tiến kỹ thuật — một xu hướng "thực nghiệm trước" phổ biến trong cộng đồng học máy nói chung. Để khắc phục, các nhà nghiên cứu đã bắt đầu phân tích Transformer sâu hơn. Một cách tiếp cận là coi Transformer là mạng nơ-ron sâu và diễn giải chúng bằng các công cụ toán học. Chẳng hạn, các kết nối tắt trong Transformer có thể được xem như phép rời rạc hóa Euler của các ODE. Sự tương đương này gợi ý rằng ta có thể tận dụng hiểu biết từ các phương pháp số giải ODE để định hướng thiết kế mô hình. Một hướng hứa hẹn khác nhằm phát triển hiểu biết lý thuyết về cơ chế tự chú ý, thứ phân biệt Transformer với các mô hình học sâu trước đó. Các nghiên cứu đã diễn giải tự chú ý qua nhiều lăng kính lý thuyết, bao gồm nén dữ liệu [Yu và cộng sự, 2023a], tối ưu hóa [Li và cộng sự, 2022c] và xấp xỉ hàm [Yun và cộng sự, 2019]. Hơn nữa, Transformer có thể được ánh xạ sang các hệ hình thức, như máy Turing [Pérez và cộng sự, 2018], máy đếm [Bhattamishra và cộng sự, 2020], ngôn ngữ chính quy và ngôn ngữ phi ngữ cảnh [Hahn, 2020], mạch Boole [Hao và cộng sự, 2022; Merrill và cộng sự, 2022], ngôn ngữ lập trình [Weiss và cộng sự, 2021], và logic bậc nhất [Chiang và cộng sự, 2023a]. Những liên hệ này cung cấp công cụ lý thuyết để nghiên cứu chặt chẽ năng lực biểu đạt của Transformer. Tuy nhiên, đáng lưu ý là dù ta có thể phân tích Transformer qua những góc nhìn đa dạng này, vẫn chưa có một lý thuyết thống nhất nào giải thích bản chất cơ bản của các mô hình này. Lấp đầy khoảng trống lý thuyết này vẫn là một thách thức trung tâm của lĩnh vực, nhưng lại thiết yếu để phát triển các hệ thống nơ-ron phức tạp có hành vi giải thích được và dự đoán được.

Chú giải của người dịch: Kết nối tắt và phương pháp Euler

Kết nối tắt có dạng $\mathbf{x}_{l+1} = \mathbf{x}_l + F(\mathbf{x}_l)$, còn phương pháp Euler (xem chú thích ở Phương trình (6.166)) có dạng $\mathbf{z}_t = \mathbf{z}_{t-1} + \Delta t \cdot g(\mathbf{z}_{t-1})$. Hai công thức giống hệt nhau nếu coi mỗi tầng là một "bước thời gian" và F đóng vai $\Delta t \cdot g$. Nhờ vậy, một mạng nhiều tầng có thể được hiểu như việc giải gần đúng một phương trình vi phân, và những phương pháp giải số chính xác hơn Euler gợi ý những kiểu kết nối giữa các tầng tốt hơn.

Phần III

Mô hình ngôn ngữ lớn

Chương 7

Tiền huấn luyện

Sự phát triển của các mô hình chuỗi dựa trên mạng nơ-ron, chẳng hạn Transformer, cùng với những tiến bộ trong học tự giám sát (*self-supervised learning*) quy mô lớn, đã mở ra cánh cửa cho việc hiểu và sinh ngôn ngữ một cách phổ quát. Thành tựu này phần lớn có được nhờ tiền huấn luyện (*pre-training*): ta tách ra những thành phần chung từ nhiều hệ thống dựa trên mạng nơ-ron rồi huấn luyện chúng trên một lượng khổng lồ dữ liệu chưa gán nhãn bằng cơ chế tự giám sát. Các mô hình đã được tiền huấn luyện này đóng vai trò là mô hình nền tảng (*foundation model*), có thể dễ dàng được thích nghi cho nhiều nhiệm vụ khác nhau thông qua tinh chỉnh (*fine-tuning*) hoặc kỹ thuật prompt (*prompting*). Nhờ đó, mô thức của NLP đã thay đổi một cách căn bản. Trong nhiều trường hợp, ta không còn cần đến học có giám sát (*supervised learning*) quy mô lớn cho từng nhiệm vụ cụ thể nữa, mà chỉ cần thích nghi các mô hình nền tảng đã được tiền huấn luyện.

Mặc dù tiền huấn luyện mới trở nên phổ biến trong các nghiên cứu NLP gần đây, khái niệm này đã có từ hàng chục năm trước, từ những ngày đầu của học sâu (*deep learning*). Ví dụ, những nỗ lực ban đầu nhằm tiền huấn luyện các hệ thống học sâu bao gồm học không giám sát cho RNN, cho các mạng truyền thẳng sâu, cho bộ tự mã hóa (*autoencoder*), v.v. [Schmidhuber, 2015]. Trong kỷ nguyên học sâu hiện đại, tiền huấn luyện đã hồi sinh, một phần nhờ việc huấn luyện không giám sát quy mô lớn các mô hình nhúng từ (*word embedding*) khác nhau [Mikolov và cộng sự, 2013c; Pennington và cộng sự, 2014]. Cũng trong giai đoạn đó, tiền huấn luyện thu hút sự quan tâm lớn trong lĩnh vực thị giác máy tính, nơi các mô hình xương sống (*backbone model*) được huấn luyện trên những bộ dữ liệu có nhãn tương đối lớn như ImageNet, rồi được áp dụng cho các nhiệm vụ hạ nguồn (*downstream task*) khác nhau [He và cộng sự, 2019; Zoph và cộng sự, 2020]. Nghiên cứu quy mô lớn về tiền huấn luyện trong NLP bắt đầu cùng với sự phát triển của các mô hình ngôn ngữ (*language model*) sử dụng học tự giám sát. Họ mô hình này gồm nhiều ví dụ nổi tiếng như BERT [Devlin và cộng sự, 2019] và GPT [Brown và cộng sự, 2020], tất cả đều có chung một ý tưởng: khả năng hiểu và sinh ngôn ngữ tổng quát có thể đạt được bằng cách huấn luyện mô hình dự đoán các từ bị che hoặc các từ tiếp theo trong một lượng văn bản khổng lồ. Bất chấp sự đơn giản của cách tiếp cận này, các mô hình thu được cho thấy khả năng đáng kinh ngạc trong việc nắm bắt cấu trúc ngôn ngữ, ngay cả khi không được huấn luyện tường minh để làm điều đó. Tính tổng quát của các nhiệm vụ tiền huấn luyện dẫn đến những hệ thống có hiệu năng mạnh trên rất nhiều bài toán NLP, thậm chí vượt qua các hệ thống có giám sát tiên tiến nhất (*state-of-the-art*) trước đó. Gần đây hơn, các mô hình ngôn ngữ lớn (*large language model*) được tiền huấn luyện đã đạt được thành công còn lớn hơn, cho thấy triển vọng đầy hứa hẹn hướng tới trí tuệ nhân tạo tổng quát hơn [Bubeck và cộng sự, 2023].

Chú giải của người dịch: Tiền huấn luyện – hình dung đơn giản

Hãy nghĩ tới việc học của một học sinh. Trước khi học chuyên sâu một nghề (ví dụ thi vào ngành y), em đã trải qua 12 năm học phổ thông: đọc rất nhiều, học ngôn ngữ, toán, kiến thức chung về thế giới. Giai đoạn "học nền" này giống với *tiền huấn luyện*: mô hình đọc hàng tỉ câu văn để học cách ngôn ngữ vận hành. Sau đó, việc học một nhiệm vụ cụ thể (phân loại cảm xúc, dịch máy…) chỉ cần một ít dữ liệu và công sức thêm — giống như học chuyên ngành trên nền kiến thức phổ thông đã có. Đó là *tinh chỉnh* (hoặc dùng *prompt*). Một mô hình được tiền huấn luyện tốt, dùng làm "nền móng" cho nhiều ứng dụng, được gọi là *mô hình nền tảng*. "Dữ liệu chưa gán nhãn" là văn bản thô (bài báo, sách, trang web) không có ai ghi chú đáp án; còn "dữ liệu có nhãn" là dữ liệu kèm đáp án do con người ghi, ví dụ mỗi câu bình luận kèm nhãn "tích cực" hay "tiêu cực". Dữ liệu có nhãn tốn công và đắt, còn văn bản thô thì có sẵn rất nhiều trên Internet.

Chương này bàn về khái niệm tiền huấn luyện trong bối cảnh NLP. Chương bắt đầu bằng phần giới thiệu chung về các phương pháp tiền huấn luyện và ứng dụng của chúng. Sau đó, BERT được dùng làm ví dụ để minh họa cách một mô hình chuỗi được huấn luyện thông qua một nhiệm vụ tự giám sát, gọi là **mô hình hóa ngôn ngữ có che** (*masked language modeling*). Tiếp theo là phần thảo luận về các phương pháp thích nghi các mô hình chuỗi đã tiền huấn luyện cho nhiều nhiệm vụ NLP khác nhau. Vì chương này tập trung chủ yếu vào mô thức tiền huấn luyện, chúng tôi sẽ để phần thảo luận chi tiết về các mô hình ngôn ngữ lớn dạng sinh sang các chương sau.

7.1 Tiền huấn luyện trong NLP

Tiền huấn luyện trong NLP thường liên quan đến hai loại nhiệm vụ: mô hình hóa chuỗi (*sequence modeling*) (hay mã hóa chuỗi (*sequence encoding*)) và sinh chuỗi (*sequence generation*). Mặc dù hai loại nhiệm vụ này có hình thức khác nhau, để đơn giản, ta mô tả chúng bằng một mô hình hợp nhất duy nhất được định nghĩa như sau:

$$\begin{aligned}\mathbf{o} &= g(x_0, x_1, \cdots, x_m; \theta) \\ &= g_\theta(x_0, x_1, \cdots, x_m),\end{aligned} \tag{7.1}$$

trong đó $\{x_0, x_1, \ldots, x_m\}$ ký hiệu một chuỗi token đầu vào[1], x_0 là một ký hiệu đặc biệt ($\langle s \rangle$ hoặc [CLS]) được gắn vào đầu chuỗi, $g(\cdot; \theta)$ (cũng được viết là $g_\theta(\cdot)$) ký hiệu một mạng nơ-ron (*neural network*) với tham số (*parameter*) θ, và $\mathbf{o}$ ký hiệu đầu ra của mạng nơ-ron. Dạng của đầu ra $\mathbf{o}$ thay đổi tùy theo bài toán cụ thể. Ví dụ, trong các nhiệm vụ dự đoán token (như mô hình hóa ngôn ngữ), $\mathbf{o}$ là một phân phối xác suất (*probability distribution*) trên một bộ từ vựng (*vocabulary*); trong các nhiệm vụ mã hóa chuỗi, $\mathbf{o}$ là một biểu diễn (*representation*) của chuỗi đầu vào, thường được biểu diễn dưới dạng một chuỗi các vectơ số thực.

Chú giải của người dịch: Đọc Phương trình (7.1)

Hãy xem g như một "cỗ máy" (một hàm số rất phức tạp): đầu vào là dãy token $x_0, x_1, \ldots, x_m$ (ví dụ các từ của một câu), đầu ra là $\mathbf{o}$. Hình dạng cụ thể của cỗ máy được quyết định bởi các "núm vặn" θ — đó là tham số, thường gồm hàng triệu đến hàng tỉ con số. Giống như hàm $y = ax + b$ ở THPT: a, b là tham số, đổi a, b thì đổi hàm. Cách viết $g(\ldots; \theta)$ hay $g_\theta(\ldots)$ chỉ để nhấn mạnh rằng hàm g phụ thuộc vào θ. "Huấn luyện" chính là đi tìm bộ θ tốt. Token x_0 là một "ký hiệu mở đầu" quy ước (ví dụ [CLS]), không phải một từ thật.

[1] Ở đây ta giả định rằng token là đơn vị cơ bản của văn bản được tạo ra bởi quá trình tách token (*tokenization*). Đôi khi, chúng tôi dùng các thuật ngữ *token* và *từ* (*word*) thay thế cho nhau, mặc dù trong NLP chúng có nghĩa gần nhau nhưng hơi khác nhau.

Có hai khía cạnh cơ bản cần xem xét ở đây:

- *Tối ưu hóa θ cho một nhiệm vụ tiền huấn luyện.* Khác với các bài toán học tiêu chuẩn trong NLP, tiền huấn luyện không giả định trước các nhiệm vụ hạ nguồn cụ thể mà mô hình sẽ được áp dụng. Thay vào đó, mục tiêu là huấn luyện một mô hình có thể tổng quát hóa (*generalize*) trên nhiều nhiệm vụ khác nhau.

- *Áp dụng mô hình đã tiền huấn luyện $g_{\hat{\theta}}(\cdot)$ cho các nhiệm vụ hạ nguồn.* Để thích nghi mô hình với các nhiệm vụ cụ thể này, ta cần tinh chỉnh các tham số $\hat{\theta}$ bằng dữ liệu có nhãn, hoặc prompt mô hình bằng các mô tả nhiệm vụ.

Trong mục này, chúng ta bàn về các khái niệm cơ bản để giải quyết hai khía cạnh trên.

7.1.1 Tiền huấn luyện không giám sát, có giám sát và tự giám sát

Trong học sâu, tiền huấn luyện là quá trình tối ưu hóa một mạng nơ-ron trước khi nó được tinh chỉnh thêm và áp dụng cho các nhiệm vụ hạ nguồn. Cách tiếp cận này dựa trên giả định rằng một mô hình được tiền huấn luyện cho một nhiệm vụ có thể được thích nghi hiệu quả để thực hiện một nhiệm vụ khác. Nhờ vậy, ta không cần huấn luyện một mạng nơ-ron sâu và phức tạp từ đầu cho các nhiệm vụ có ít dữ liệu gán nhãn. Thay vào đó, ta có thể tận dụng những nhiệm vụ mà tín hiệu giám sát dễ thu thập hơn. Điều này làm giảm sự phụ thuộc vào việc gán nhãn thủ công của con người cho từng nhiệm vụ, cho phép phát triển những mô hình tổng quát hơn, không bị bó hẹp trong các bài toán cụ thể.

Trong thời kỳ hồi sinh của mạng nơ-ron nhờ học sâu, nhiều nỗ lực tiền huấn luyện ban đầu tập trung vào **học không giám sát** (*unsupervised learning*). Trong các phương pháp này, tham số của mạng nơ-ron được tối ưu hóa theo một tiêu chí không liên quan trực tiếp đến các nhiệm vụ đích cụ thể. Ví dụ, một mô hình có thể được huấn luyện để cực tiểu hóa entropy chéo (*cross-entropy*) tái tạo của vectơ đầu vào ở mỗi tầng (*layer*) [Bengio và cộng sự, 2006]. Tiền huấn luyện không giám sát thường được dùng như một bước sơ bộ trước học có giám sát, mang lại nhiều lợi ích, chẳng hạn giúp tìm ra các cực tiểu địa phương (*local minima*) tốt hơn và đóng vai trò như một bộ chính quy hóa (*regularizer*) trong quá trình huấn luyện [Erhan và cộng sự, 2010]. Những lợi ích này khiến giai đoạn học có giám sát sau đó dễ dàng và ổn định hơn.

Cách tiếp cận thứ hai là tiền huấn luyện mạng nơ-ron trên các nhiệm vụ **học có giám sát**. Ví dụ, xét một mô hình chuỗi được thiết kế để mã hóa các chuỗi đầu vào thành các biểu diễn vectơ. Trong quá trình tiền huấn luyện, mô hình này được kết hợp với một tầng phân loại và được huấn luyện trên một nhiệm vụ có giám sát ban đầu, chẳng hạn phân tích cảm xúc (*sentiment analysis*) (ví dụ, phân loại một câu là tích cực hay tiêu cực). Để thích nghi mô hình chuỗi đã tiền huấn luyện cho một nhiệm vụ hạ nguồn (ví dụ, xác định một chuỗi mang tính chủ quan hay khách quan), ta thay tầng phân loại ban đầu bằng một tầng mới cho nhiệm vụ đích. Thông thường, ta cần tinh chỉnh các tham số của mô hình kết hợp này bằng dữ liệu có nhãn riêng của nhiệm vụ để tối ưu hiệu năng. Mô hình đã tinh chỉnh sau đó được dùng để phân loại các chuỗi mới. Một ưu điểm lớn của tiền huấn luyện có giám sát là nó khớp một cách trực tiếp với các mô thức học có giám sát tiêu chuẩn. Tuy nhiên, khi độ phức tạp của mô hình tăng lên, nhu cầu về dữ liệu có nhãn cũng tăng theo, khiến cách tiền huấn luyện này gặp khó khăn khi không có sẵn các bộ dữ liệu có nhãn quy mô lớn.

Cách tiếp cận thứ ba là **học tự giám sát** (*self-supervised learning*). Trong mô thức này, mạng nơ-ron được huấn luyện bằng các tín hiệu giám sát rút ra từ chính dữ liệu, chứ không phải từ nhãn do con người gán. Điều này thường được thực hiện bằng cách xây dựng các nhiệm vụ huấn luyện trực tiếp từ dữ liệu chưa gán nhãn, chẳng hạn dự đoán những phần bị thiếu của đầu vào. Mặc dù học tự giám sát gần đây

đã trở thành phương pháp chủ đạo trong NLP, triết lý nền tảng của nó có nhiều điểm tương đồng với các khái niệm học máy cũ hơn như **tự huấn luyện** (*self-training*). Trong tự huấn luyện, mô hình cần một lượng nhỏ dữ liệu hạt giống (*seed data*) để xây dựng một bộ phân loại (*classifier*) ban đầu, rồi bộ phân loại này sinh ra các nhãn giả (*pseudo-label*) cho dữ liệu chưa gán nhãn. Các nhãn giả này sau đó được dùng để tinh chỉnh dần và tự nâng cấp (*bootstrap*) mô hình qua nhiều vòng lặp. Phương pháp này đã được áp dụng thành công cho các lĩnh vực NLP như khử nhập nhằng nghĩa của từ (*word sense disambiguation*) [Yarowsky, 1995] và phân loại văn bản [Blum và Mitchell, 1998]. Tuy nhiên, khác với tự huấn luyện tiêu chuẩn, tiền huấn luyện tự giám sát trong NLP không dựa vào một mô hình ban đầu để gán các nhãn giả rời rạc. Thay vào đó, tín hiệu giám sát được rút ra trực tiếp từ văn bản, và toàn bộ mô hình được huấn luyện từ đầu. Một ví dụ nổi tiếng là huấn luyện các mô hình chuỗi bằng cách che một số token trong văn bản và yêu cầu mô hình dự đoán chúng dựa trên ngữ cảnh xung quanh. Điều này cho phép học tự giám sát quy mô lớn cho các mạng nơ-ron sâu, dẫn tới thành công của tiền huấn luyện trong nhiều nhiệm vụ hiểu, sinh và suy luận (*reasoning*).

Chú giải của người dịch: Ba kiểu học: so sánh nhanh

- *Có giám sát*: mỗi ví dụ có "đáp án" do người ghi. Ví dụ: câu "Món ăn ngon tuyệt" kèm nhãn *tích cực*.
- *Không giám sát*: chỉ có dữ liệu, không có đáp án; mô hình tự tìm cấu trúc (ví dụ học cách nén rồi tái tạo lại đầu vào).
- *Tự giám sát*: cũng không cần người ghi nhãn, nhưng ta *tự tạo ra đáp án từ chính dữ liệu*. Ví dụ, từ câu "Hôm nay trời rất đẹp", ta che đi một từ: "Hôm nay trời rất ___" và bắt mô hình đoán; đáp án "đẹp" có sẵn trong câu gốc. Vì văn bản trên mạng là vô tận, ta có thể tạo ra hàng tỉ "bài tập" như vậy mà không tốn công gán nhãn.

Tự huấn luyện thì khác: một bộ phân loại nhỏ (học từ ít dữ liệu có nhãn) tự "chấm" nhãn cho dữ liệu chưa có nhãn, rồi dùng chính các nhãn tự chấm này (nhãn giả) để học tiếp. "Cực tiểu địa phương" là điểm thấp của hàm mất mát chỉ trong một vùng lân cận (như đáy một thung lũng nhỏ, chưa chắc là đáy sâu nhất của cả dãy núi).

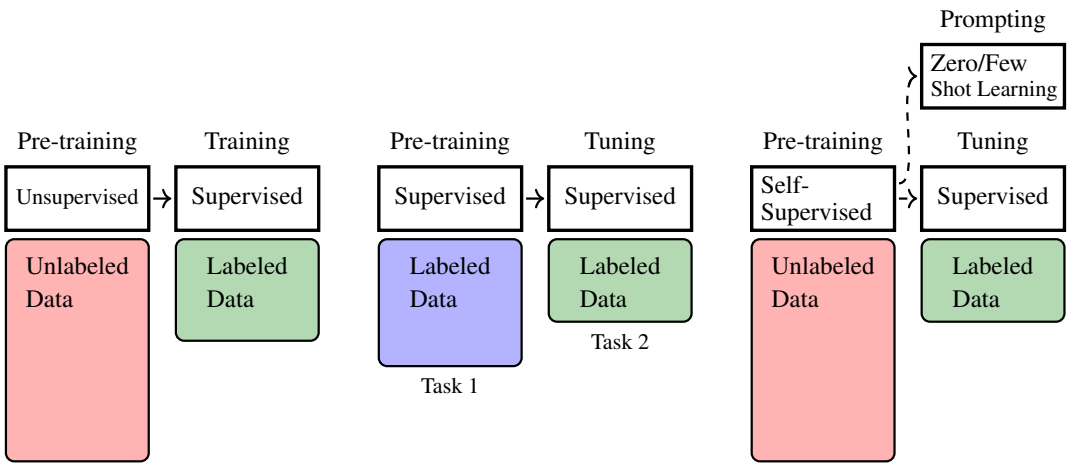
Hình 7.1 so sánh ba cách tiếp cận tiền huấn luyện nói trên. Tiền huấn luyện tự giám sát thành công đến mức hầu hết các mô hình NLP tiên tiến nhất hiện nay đều dựa trên mô thức này. Vì vậy, trong suốt chương này và phần còn lại của sách, chúng ta sẽ tập trung vào tiền huấn luyện tự giám sát. Ta sẽ thấy các mô hình chuỗi được tiền huấn luyện thông qua tự giám sát như thế nào và các mô hình đã tiền huấn luyện được áp dụng ra sao.

Giải nghĩa nhãn trong hình: Pre-training = tiền huấn luyện; Training = huấn luyện; Tuning = tinh chỉnh; Unsupervised/Supervised/Self-Supervised = không giám sát/có giám sát/tự giám sát; Unlabeled/Labeled Data = dữ liệu chưa gán nhãn/có nhãn; Task 1/2 = nhiệm vụ 1/2; Prompting – Zero/Few Shot Learning = dùng prompt – học zero-shot/few-shot.

7.1.2 Thích nghi các mô hình đã tiền huấn luyện

Như đã nói ở trên, có hai loại mô hình chính được sử dụng rộng rãi trong tiền huấn luyện NLP.

- **Mô hình mã hóa chuỗi** (*Sequence Encoding Models*). Cho một chuỗi từ hoặc token, mô hình mã hóa chuỗi biểu diễn chuỗi này dưới dạng một vectơ số thực hoặc một chuỗi các vectơ, và thu được một biểu diễn của chuỗi. Biểu diễn này thường được dùng làm đầu vào cho một mô hình khác, chẳng hạn một hệ thống phân loại câu.
- **Mô hình sinh chuỗi** (*Sequence Generation Models*). Trong NLP, sinh chuỗi thường chỉ bài toán



(a) Unsupervised Pre-training (b) Supervised Pre-training (c) Self-supervised Pre-training

Hình 7.1: Minh họa tiền huấn luyện không giám sát, có giám sát và tự giám sát. Trong tiền huấn luyện không giám sát, việc tiền huấn luyện được thực hiện trên dữ liệu chưa gán nhãn quy mô lớn. Nó có thể được xem như một bước sơ bộ để có một điểm xuất phát tốt cho quá trình tối ưu hóa tiếp theo, dù vẫn cần nhiều công sức để huấn luyện thêm mô hình bằng dữ liệu có nhãn sau khi tiền huấn luyện. Trong tiền huấn luyện có giám sát, giả định nền tảng là các nhiệm vụ học (có giám sát) khác nhau có liên quan với nhau. Vì vậy, ta có thể huấn luyện mô hình trên một nhiệm vụ trước, rồi chuyển mô hình thu được sang một nhiệm vụ khác với một chút công sức huấn luyện hoặc tinh chỉnh. Trong tiền huấn luyện tự giám sát, mô hình được tiền huấn luyện trên dữ liệu chưa gán nhãn quy mô lớn thông qua tự giám sát. Mô hình có thể được huấn luyện hiệu quả theo cách này, và ta có thể thích nghi nó một cách hiệu quả cho các nhiệm vụ mới thông qua tinh chỉnh hoặc prompting.

sinh ra một chuỗi token dựa trên một ngữ cảnh (*context*) cho trước. Thuật ngữ *ngữ cảnh* mang nghĩa khác nhau tùy ứng dụng. Ví dụ, trong mô hình hóa ngôn ngữ, nó là các token đứng trước; còn trong dịch máy (*machine translation*), nó là chuỗi ở ngôn ngữ nguồn[2].

Ta cần những kỹ thuật khác nhau để áp dụng các mô hình này cho các nhiệm vụ hạ nguồn sau khi tiền huấn luyện. Ở đây ta quan tâm đến hai phương pháp sau.

1. Tinh chỉnh các mô hình đã tiền huấn luyện

Với tiền huấn luyện mã hóa chuỗi, một phương pháp phổ biến để thích nghi các mô hình đã tiền huấn luyện là tinh chỉnh. Gọi $\text{Encode}_\theta(\cdot)$ là một bộ mã hóa (*encoder*) với tham số θ; ví dụ, $\text{Encode}_\theta(\cdot)$ có thể là một bộ mã hóa Transformer tiêu chuẩn. Giả sử ta đã tiền huấn luyện mô hình này theo cách nào đó và thu được các tham số tối ưu $\hat{\theta}$, ta có thể dùng nó để mã hóa một chuỗi đầu vào như sau

$$\mathbf{H} = \text{Encode}_{\hat{\theta}}(\mathbf{x}), \tag{7.2}$$

trong đó $\mathbf{x}$ là chuỗi đầu vào $\{x_0, \cdots, x_m\}$, và $\mathbf{H}$ là biểu diễn đầu ra, là một chuỗi các vectơ số thực $\{\mathbf{h}_0, \mathbf{h}_1, \cdots, \mathbf{h}_m\}$. Vì bộ mã hóa không hoạt động như một hệ thống NLP độc lập, nó thường được tích hợp như một thành phần trong một hệ thống lớn hơn. Ví dụ, xét bài toán phân loại văn bản, trong đó ta xác định cực tính (*polarity*) (chẳng hạn tích cực, tiêu cực hay trung tính) của một văn bản cho trước. Ta có thể xây dựng một hệ thống phân loại văn bản bằng cách đặt một bộ phân loại lên trên bộ mã hóa. Gọi $\text{Classify}_\omega(\cdot)$ là một mạng nơ-ron với tham số ω. Khi đó, mô hình phân loại văn bản có thể được biểu diễn dưới dạng

$$\begin{aligned}\text{Pr}_{\omega,\hat{\theta}}(\cdot|\mathbf{x}) &= \text{Classify}_\omega(\mathbf{H}) \\ &= \text{Classify}_\omega(\text{Encode}_{\hat{\theta}}(\mathbf{x})).\end{aligned} \tag{7.3}$$

Ở đây $\text{Pr}_{\omega,\hat{\theta}}(\cdot|\mathbf{x})$ là một phân phối xác suất trên tập nhãn $\{\text{positive}, \text{negative}, \text{neutral}\}$ (tích cực, tiêu cực, trung tính), và nhãn có xác suất cao nhất trong phân phối này được chọn làm đầu ra. Để ký hiệu gọn gàng, ta sẽ dùng $F_{\omega,\hat{\theta}}(\cdot)$ để ký hiệu $\text{Classify}_\omega(\text{Encode}_{\hat{\theta}}(\cdot))$.

Vì các tham số mô hình ω và $\hat{\theta}$ không được tối ưu hóa tường minh cho nhiệm vụ phân loại đích, ta không thể dùng trực tiếp mô hình kết hợp này. Thay vào đó, ta phải dùng một phiên bản đã được sửa đổi của mô hình, được thích nghi cho nhiệm vụ. Một cách tiếp cận điển hình là tinh chỉnh mô hình bằng cách cung cấp các nhãn tường minh từ nhiệm vụ hạ nguồn. Ta có thể huấn luyện $F_{\omega,\hat{\theta}}(\cdot)$ trên một bộ dữ liệu có nhãn, coi đó như một nhiệm vụ học có giám sát tiêu chuẩn. Quá trình tinh chỉnh này cho ra các tham số được tối ưu thêm, $\tilde{\omega}$ và $\tilde{\theta}$. Một cách khác là ta có thể đóng băng (*freeze*) các tham số bộ mã hóa $\hat{\theta}$ để giữ nguyên trạng thái đã tiền huấn luyện và chỉ tập trung tối ưu hóa ω. Điều này cho phép bộ phân loại được thích nghi một cách hiệu quả để làm việc tốt với bộ mã hóa đã tiền huấn luyện.

Chú giải của người dịch: Các dấu mũ ˆ và dấu ngã ˜

Sách dùng các ký hiệu để phân biệt các "phiên bản" của tham số: θ là tham số nói chung (chưa biết giá trị); $\hat{\theta}$ (đọc "theta mũ") là giá trị *thu được sau tiền huấn luyện*; $\tilde{\theta}$ ("theta ngã") là giá trị *sau khi tinh chỉnh thêm*. Tương tự với ω của bộ phân loại. Hình dung hệ thống gồm hai khối nối tiếp: khối Encode (đã học sẵn, "đọc hiểu" văn bản thành dãy vectơ $\mathbf{H}$) và khối Classify (mới, nhỏ, "ra quyết định" nhãn). Tinh chỉnh có hai lựa chọn: sửa cả hai khối (ω và θ đều thay đổi), hoặc "đóng băng" khối Encode và chỉ huấn luyện khối Classify — rẻ hơn nhiều vì số tham số cần học ít.

[2]Chính xác hơn, trong giải mã tự hồi quy (*autoregressive decoding*) của dịch máy, mỗi token ở ngôn ngữ đích được sinh ra dựa trên cả các token đứng trước nó lẫn chuỗi ngôn ngữ nguồn.

Khi đã có mô hình được tinh chỉnh, ta có thể dùng nó để phân loại một văn bản mới. Ví dụ, giả sử có một bình luận được đăng trên một trang web du lịch:

I love the food here. It's amazing!
(Tôi thích đồ ăn ở đây. Thật tuyệt vời!)

Đầu tiên, ta tách văn bản này thành các token[3], rồi đưa chuỗi token $\mathbf{x}_{\text{new}}$ vào mô hình đã tinh chỉnh $F_{\tilde{\omega},\tilde{\theta}}(\cdot)$. Mô hình sinh ra một phân phối trên các lớp như sau

$$F_{\tilde{\omega},\tilde{\theta}}(\mathbf{x}_{\text{new}}) = \begin{bmatrix} \Pr(\text{positive}|\mathbf{x}_{\text{new}}) & \Pr(\text{negative}|\mathbf{x}_{\text{new}}) & \Pr(\text{neutral}|\mathbf{x}_{\text{new}}) \end{bmatrix}. \tag{7.4}$$

Sau đó ta chọn nhãn có xác suất cao nhất làm đầu ra. Trong ví dụ này, nhãn đó là positive (tích cực).

Chú giải của người dịch: Ví dụ bằng số

Giả sử mô hình trả về vectơ $[0.92 \;\; 0.03 \;\; 0.05]$. Ba số này là xác suất của ba nhãn tích cực, tiêu cực, trung tính, và có tổng bằng 1. Số lớn nhất là 0.92 ứng với *tích cực*, nên mô hình kết luận bình luận mang cảm xúc tích cực. Ký hiệu $\Pr(\text{positive}|\mathbf{x}_{\text{new}})$ đọc là "xác suất nhãn tích cực khi biết văn bản $\mathbf{x}_{\text{new}}$" (xác suất có điều kiện).

Nhìn chung, lượng dữ liệu có nhãn dùng trong tinh chỉnh nhỏ so với lượng dữ liệu tiền huấn luyện, nên tinh chỉnh tốn ít chi phí tính toán hơn. Điều này khiến việc thích nghi các mô hình đã tiền huấn luyện trở nên rất hiệu quả trong thực tế: với một mô hình đã tiền huấn luyện và một nhiệm vụ hạ nguồn, ta chỉ cần thu thập một ít dữ liệu có nhãn và điều chỉnh nhẹ các tham số mô hình trên dữ liệu này. Phần thảo luận chi tiết hơn về tinh chỉnh có thể xem ở Mục 7.4.

2. Prompting các mô hình đã tiền huấn luyện

Khác với các mô hình mã hóa chuỗi, các mô hình sinh chuỗi thường được dùng độc lập để giải quyết các bài toán sinh ngôn ngữ, chẳng hạn hỏi đáp (*question answering*) và dịch máy, mà không cần thêm mô-đun nào. Do đó, việc tinh chỉnh các mô hình này như những hệ thống hoàn chỉnh trên các nhiệm vụ hạ nguồn là điều đơn giản. Ví dụ, ta có thể tinh chỉnh một mô hình đa ngôn ngữ dạng bộ mã hóa–bộ giải mã (*encoder-decoder*) đã tiền huấn luyện trên một ít dữ liệu song ngữ để cải thiện hiệu năng của nó trên một nhiệm vụ dịch cụ thể.

Trong số các mô hình sinh chuỗi, một ví dụ đáng chú ý là các mô hình ngôn ngữ lớn được huấn luyện trên lượng dữ liệu rất lớn. Các mô hình này được huấn luyện đơn giản chỉ để dự đoán token tiếp theo khi biết các token đứng trước. Mặc dù dự đoán token là một nhiệm vụ đơn giản vốn được dùng từ lâu cho mô hình hóa ngôn ngữ, việc thực hiện nó ở quy mô lớn giúp mô hình thu nhận được tri thức rộng về ngôn ngữ và về thế giới. Kết quả là các LLM đã tiền huấn luyện thể hiện khả năng dự đoán token đáng kinh ngạc, khiến ta có thể đưa nhiều nhiệm vụ NLP về dạng những bài toán sinh văn bản đơn giản thông qua prompting. Ví dụ, ta có thể đưa bài toán phân loại văn bản ở trên về dạng một nhiệm vụ sinh văn bản.

I love the food here. It's amazing! I'm ___
(Tôi thích đồ ăn ở đây. Thật tuyệt vời! Tôi thấy ___)

Ở đây __ chỉ từ hoặc cụm từ mà ta muốn dự đoán (gọi là **phần hoàn thành** (*completion*)). Nếu từ được dự đoán là *happy* (vui), *glad* (vui mừng), *satisfied* (hài lòng), hoặc một từ tích cực liên quan, ta có

[3] Văn bản có thể được tách token theo nhiều cách khác nhau. Một trong những cách đơn giản nhất là tách văn bản thành các từ tiếng Anh và dấu câu {I, love, the, food, here, ., It, 's, amazing, !}.

thể phân loại văn bản là positive. Ví dụ này cho thấy một phương pháp prompting đơn giản, trong đó ta nối văn bản đầu vào với *I'm* ("Tôi thấy") để tạo thành một prompt (*prompt*). Sau đó, phần hoàn thành giúp quyết định nhãn nào được gán cho văn bản gốc.

Với khả năng hiểu và sinh ngôn ngữ mạnh mẽ của các mô hình ngôn ngữ lớn, một prompt có thể chỉ dẫn các mô hình này thực hiện những nhiệm vụ phức tạp hơn. Ví dụ sau minh họa cách chỉ dẫn một LLM thực hiện phân loại cực tính:

Giả sử cực tính của một văn bản là một nhãn được chọn từ {tích cực, tiêu cực, trung tính}. Hãy xác định cực tính của đầu vào.
Đầu vào: Tôi thích đồ ăn ở đây. Thật tuyệt vời!
Cực tính: ___

Hai câu đầu tiên là phần mô tả nhiệm vụ. Đầu vào (*Input*) và Cực tính (*Polarity*) lần lượt là các dấu hiệu chỉ đầu vào và đầu ra. Ta kỳ vọng mô hình sẽ hoàn thành văn bản và đồng thời đưa ra nhãn cực tính đúng. Bằng cách dùng các prompt dựa trên chỉ dẫn, ta có thể thích nghi các mô hình ngôn ngữ lớn để giải các bài toán NLP mà không cần huấn luyện thêm.

Ví dụ này cũng cho thấy khả năng học zero-shot (*zero-shot learning*) của các mô hình ngôn ngữ lớn: chúng có thể thực hiện những nhiệm vụ chưa từng gặp trong quá trình huấn luyện. Một phương pháp khác để khơi gợi những năng lực mới từ mạng nơ-ron là học few-shot (*few-shot learning*). Điều này thường được thực hiện thông qua **học trong ngữ cảnh** (*in-context learning*) (**ICL**). Cụ thể hơn, ta thêm vào một số mẫu minh họa cách một đầu vào tương ứng với một đầu ra. Các mẫu này, được gọi là **ví dụ minh họa** (*demonstrations*), được dùng để dạy các mô hình ngôn ngữ lớn cách thực hiện nhiệm vụ. Dưới đây là một ví dụ có kèm các ví dụ minh họa

Giả sử cực tính của một văn bản là một nhãn được chọn từ {tích cực, tiêu cực, trung tính}. Hãy xác định cực tính của đầu vào.
Đầu vào: Giao thông vào giờ cao điểm thật tồi tệ, khiến việc đến sân bay đúng giờ trở nên khó khăn.
Cực tính: Tiêu cực
Đầu vào: Thời tiết ở đây thật tuyệt.
Cực tính: Tích cực
Đầu vào: Tôi thích đồ ăn ở đây. Thật tuyệt vời!
Cực tính: ___

Chú giải của người dịch: Zero-shot và few-shot

"Shot" ở đây có thể hiểu là "ví dụ mẫu". *Zero-shot*: không cho mô hình ví dụ mẫu nào, chỉ mô tả nhiệm vụ (prompt thứ nhất). *Few-shot*: cho vài ví dụ mẫu đã có đáp án (prompt thứ hai có 2 ví dụ, gọi là 2-shot). Điều đáng chú ý là trong cả hai trường hợp, tham số của mô hình *không hề thay đổi*: mô hình chỉ đọc prompt và viết tiếp. Các ví dụ minh họa giúp mô hình hiểu rõ hơn định dạng và ý nghĩa của nhãn cần đưa ra.

Prompting và học trong ngữ cảnh đóng vai trò quan trọng trong sự trỗi dậy gần đây của các mô hình ngôn ngữ lớn. Chúng ta sẽ bàn chi tiết hơn về các kỹ thuật này ở Chương 9. Tuy nhiên, cần lưu ý rằng mặc dù prompting là một cách mạnh mẽ để thích nghi các mô hình ngôn ngữ lớn, vẫn cần một số công sức tinh chỉnh để bảo đảm mô hình có thể làm theo chỉ dẫn một cách chính xác. Ngoài ra, quá trình tinh chỉnh rất quan trọng để căn chỉnh (*alignment*) các giá trị của những mô hình này với giá trị của con người. Phần thảo luận chi tiết hơn về tinh chỉnh có thể xem ở Chương 10.

7.2 Các nhiệm vụ tiền huấn luyện tự giám sát

Trong mục này, ta xem xét các cách tiếp cận tiền huấn luyện tự giám sát cho các kiến trúc nơ-ron khác nhau, bao gồm kiến trúc chỉ có bộ giải mã (*decoder-only*), chỉ có bộ mã hóa (*encoder-only*) và kiến trúc bộ mã hóa–bộ giải mã. Ta giới hạn thảo luận ở Transformer vì chúng là nền tảng của hầu hết các mô hình đã tiền huấn luyện trong NLP. Tuy nhiên, tiền huấn luyện là một khái niệm rộng, nên ở đây chúng tôi chỉ giới thiệu ngắn gọn các cách tiếp cận cơ bản để mục này được súc tích.

7.2.1 Tiền huấn luyện chỉ với bộ giải mã

Kiến trúc chỉ có bộ giải mã đã được dùng rộng rãi để phát triển các mô hình ngôn ngữ [Radford và cộng sự, 2018]. Ví dụ, ta có thể dùng một bộ giải mã (*decoder*) Transformer làm mô hình ngôn ngữ bằng cách đơn giản là bỏ đi các tầng con chú ý chéo (*cross-attention*) của nó. Một mô hình như vậy dự đoán phân phối xác suất trên bộ từ vựng cho token tiếp theo dựa trên các token đứng trước, và đầu ra dự đoán là token có xác suất cao nhất. Cách tiêu chuẩn để huấn luyện mô hình này, giống như trong mô hình hóa ngôn ngữ truyền thống, là cực tiểu hóa một hàm mất mát (*loss function*) trên một tập các chuỗi token.

Gọi $\text{Decoder}_\theta(\cdot)$ là một bộ giải mã với tham số θ. Tại mỗi vị trí i, bộ giải mã sinh ra một phân phối cho token tiếp theo dựa trên ngữ cảnh đứng trước $\{x_0, \ldots, x_i\}$, ký hiệu là $\Pr_\theta(\cdot|x_0, \ldots, x_i)$ (hay viết tắt là $\mathbf{p}^\theta_{i+1}$). Giả sử ta có phân phối đích chuẩn (*gold-standard*) tại cùng vị trí, ký hiệu là $\mathbf{p}^{\text{gold}}_{i+1}$. Với mô hình hóa ngôn ngữ, ta có thể xem $\mathbf{p}^{\text{gold}}_{i+1}$ là biểu diễn one-hot (*one-hot representation*) của từ tiếp theo đúng. Sau đó ta định nghĩa một hàm mất mát $\mathcal{L}(\mathbf{p}^\theta_{i+1}, \mathbf{p}^{\text{gold}}_{i+1})$ để đo độ sai khác giữa dự đoán của mô hình và phân phối đích. Trong NLP, hàm mất mát entropy chéo thường được sử dụng.

Cho một chuỗi token $\mathbf{x} = \{x_0, \cdots, x_m\}$, mất mát trên chuỗi này là tổng mất mát tại các vị trí $\{0, \cdots, m-1\}$, cho bởi

$$\begin{aligned}\text{Loss}_\theta(x_0, \cdots, x_m) &= \sum_{i=0}^{m-1} \mathcal{L}(\mathbf{p}^\theta_{i+1}, \mathbf{p}^{\text{gold}}_{i+1}) \\ &= \sum_{i=0}^{m-1} \text{CrossEntropy}(\mathbf{p}^\theta_{i+1}, \mathbf{p}^{\text{gold}}_{i+1}),\end{aligned} \tag{7.5}$$

trong đó $\text{CrossEntropy}(\cdot)$ ký hiệu hàm mất mát entropy chéo tiêu chuẩn, và $\mathbf{p}^{\text{gold}}_{i+1}$ là biểu diễn one-hot của x_{i+1}.

Chú giải của người dịch: One-hot và entropy chéo – ví dụ nhỏ

Giả sử bộ từ vựng chỉ có 4 từ: {tôi, thích, mèo, chó} và từ đúng tiếp theo là "mèo". Biểu diễn one-hot của "mèo" là $[0\ 0\ 1\ 0]$: chỉ có một số 1 ở đúng vị trí của từ đó. Nếu mô hình dự đoán $\mathbf{p}^\theta = [0.1\ \ 0.1\ \ 0.6\ \ 0.2]$, thì entropy chéo giữa hai phân phối bằng $-\log(\text{xác suất mô hình gán cho từ đúng}) = -\log 0.6 \approx 0.51$ (logarit tự nhiên). Nếu mô hình tự tin hơn, gán 0.9 cho "mèo", mất mát giảm còn $-\log 0.9 \approx 0.11$; nếu gán chỉ 0.01, mất mát tăng vọt tới ≈ 4.6. Vì vậy, cực tiểu hóa entropy chéo nghĩa là ép mô hình gán xác suất cao cho từ đúng. Phương trình (7.5) chỉ đơn giản cộng các mất mát này qua mọi vị trí trong câu.

Hàm mất mát này có thể được mở rộng cho một tập các chuỗi $\mathcal{D}$. Khi đó, mục tiêu của tiền huấn luyện là tìm các tham số tối ưu cực tiểu hóa mất mát trên $\mathcal{D}$

$$\hat{\theta} = \arg\min_\theta \sum_{\mathbf{x} \in \mathcal{D}} \text{Loss}_\theta(\mathbf{x}). \tag{7.6}$$

Lưu ý rằng mục tiêu này tương đương về mặt toán học với ước lượng hợp lý cực đại (*maximum likelihood estimation*), và có thể được viết lại như sau:

$$\begin{aligned}\hat{\theta} &= \arg\max_{\theta} \sum_{\mathbf{x}\in\mathcal{D}} \log \Pr_\theta(\mathbf{x}) \\ &= \arg\max_{\theta} \sum_{\mathbf{x}\in\mathcal{D}} \sum_{i=0}^{|\mathbf{x}|-2} \log \Pr_\theta(x_{i+1}|x_0,\ldots,x_i).\end{aligned} \tag{7.7}$$

Với các tham số đã tối ưu $\hat{\theta}$ này, ta có thể dùng mô hình ngôn ngữ đã tiền huấn luyện $\text{Decoder}_{\hat{\theta}}(\cdot)$ để tính xác suất $\Pr_{\hat{\theta}}(x_{i+1}|x_0,\cdots,x_i)$ tại mỗi vị trí của một chuỗi cho trước.

Chú giải của người dịch: Đọc arg min**,** arg max **và vì sao (7.6) tương đương (7.7)**

$\arg\min_\theta f(\theta)$ là "giá trị của θ làm f nhỏ nhất" (khác với $\min f$ là bản thân giá trị nhỏ nhất). Ví dụ với $f(\theta) = (\theta-3)^2+1$ thì $\min f = 1$ còn $\arg\min f = 3$. Tương tự, arg max là giá trị làm hàm lớn nhất.
Vì mất mát entropy chéo với nhãn one-hot đúng bằng $-\log$ của xác suất gán cho token đúng, nên tổng mất mát bằng $-\sum \log \Pr_\theta(x_{i+1}|x_0,\ldots,x_i)$. Làm cực tiểu một số âm của một đại lượng cũng chính là làm cực đại đại lượng đó, nên arg min trong (7.6) trở thành arg max trong (7.7). Thêm nữa, theo quy tắc nhân xác suất, xác suất cả câu bằng tích các xác suất có điều kiện của từng token (khi biết các token trước), và logarit biến tích thành tổng: $\log(ab) = \log a + \log b$ — đó là lý do có dấu $\sum_i$ bên trong. Chỉ số chạy tới $|\mathbf{x}|-2$ vì chuỗi có $|\mathbf{x}| = m+1$ token (x_0 đến x_m) và ta dự đoán từ x_1 đến x_m.

7.2.2 Tiền huấn luyện chỉ với bộ mã hóa

Như đã định nghĩa ở Mục 7.1.2, bộ mã hóa $\text{Encode}_\theta(\cdot)$ là một hàm đọc một chuỗi token $\mathbf{x} = \{x_0,\ldots,x_m\}$ và tạo ra một chuỗi vectơ $\mathbf{H} = \{\mathbf{h}_0,\ldots,\mathbf{h}_m\}$[4]. Việc huấn luyện mô hình này không đơn giản, vì ta không có dữ liệu chuẩn để đo chất lượng của các biểu diễn liên tục mà hàm này tạo ra. Một cách tiếp cận điển hình cho tiền huấn luyện bộ mã hóa là kết hợp bộ mã hóa với một tầng đầu ra dành riêng cho nhiệm vụ để có được những tín hiệu giám sát sẵn có. Hình 7.2 cho thấy một kiến trúc phổ biến để tiền huấn luyện bộ mã hóa Transformer, trong đó ta thêm một tầng Softmax lên trên bộ mã hóa. Kiến trúc này về cơ bản giống với kiến trúc của mô hình ngôn ngữ dựa trên bộ giải mã, và đầu ra là một chuỗi các phân phối xác suất:

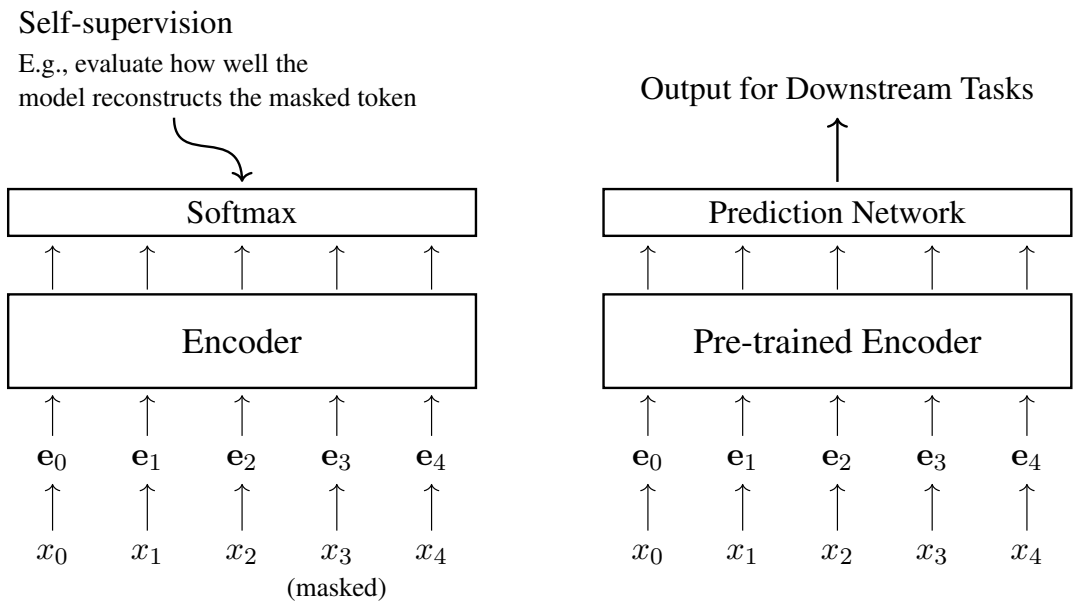
$$\begin{bmatrix}\mathbf{p}_1^{\mathbf{W},\theta} \\ \vdots \\ \mathbf{p}_m^{\mathbf{W},\theta}\end{bmatrix} = \text{Softmax}_{\mathbf{W}}(\text{Encode}_\theta(\mathbf{x})). \tag{7.8}$$

Ở đây $\mathbf{p}_i^{\mathbf{W},\theta}$ là phân phối đầu ra $\Pr(\cdot|\mathbf{x})$ tại vị trí i. Ta dùng $\text{Softmax}_{\mathbf{W}}(\cdot)$ để chỉ rằng tầng Softmax được tham số hóa bởi $\mathbf{W}$, tức là $\text{Softmax}_{\mathbf{W}}(\mathbf{H}) = \text{Softmax}(\mathbf{H}\cdot\mathbf{W})$. Để ký hiệu đơn giản, đôi khi ta sẽ bỏ các chỉ số trên $\mathbf{W}$ và θ gắn với mỗi phân phối xác suất.

Giải nghĩa nhãn trong hình: Self-supervision = tự giám sát (ví dụ: đánh giá mô hình tái tạo token bị che tốt đến đâu); Encoder / Pre-trained Encoder = bộ mã hóa / bộ mã hóa đã tiền huấn luyện; (masked)

[4] Nếu ta xem $\mathbf{h}_i$ là một vectơ hàng, $\mathbf{H}$ có thể được viết là

$$\mathbf{H} = \begin{bmatrix}\mathbf{h}_0 \\ \vdots \\ \mathbf{h}_m\end{bmatrix}.$$



(a) Pre-training (b) Applying the Pre-trained Encoder

Hình 7.2: Tiền huấn luyện một bộ mã hóa Transformer (trái) rồi áp dụng bộ mã hóa đã tiền huấn luyện (phải). Trong giai đoạn tiền huấn luyện, bộ mã hóa cùng với một tầng Softmax được huấn luyện thông qua tự giám sát. Trong giai đoạn áp dụng, tầng Softmax bị bỏ đi, và bộ mã hóa đã tiền huấn luyện được kết hợp với một mạng dự đoán để giải quyết các bài toán cụ thể. Nhìn chung, để thích nghi tốt hơn với các nhiệm vụ này, hệ thống được tinh chỉnh bằng dữ liệu có nhãn.

= bị che; Prediction Network = mạng dự đoán; Output for Downstream Tasks = đầu ra cho các nhiệm vụ hạ nguồn; $\mathbf{e}_i$ là vectơ nhúng của token x_i.

Điểm khác biệt giữa mô hình này và các mô hình ngôn ngữ tiêu chuẩn là đầu ra $\mathbf{p}_i$ mang ý nghĩa khác nhau trong tiền huấn luyện bộ mã hóa và trong mô hình hóa ngôn ngữ. Trong mô hình hóa ngôn ngữ, $\mathbf{p}_i$ là phân phối xác suất để dự đoán token tiếp theo. Điều này tuân theo quá trình giải mã tự hồi quy: mô hình ngôn ngữ chỉ quan sát các từ cho đến vị trí i và dự đoán từ tiếp theo. Ngược lại, trong tiền huấn luyện bộ mã hóa, toàn bộ chuỗi được quan sát cùng lúc, và do đó việc dự đoán một token là quá dễ khi mô hình đã nhìn thấy token đó.

Chú giải của người dịch: Softmax và "nhìn trộm đáp án"

Hàm Softmax biến một dãy số thực bất kỳ thành các xác suất dương có tổng bằng 1: $\text{Softmax}(z_1, \ldots, z_n)_k = e^{z_k} / \sum_j e^{z_j}$. Ví dụ $(2, 1, 0) \mapsto (0.67,\ 0.24,\ 0.09)$. Ma trận $\mathbf{W}$ chuyển mỗi vectơ $\mathbf{h}_i$ thành một điểm số cho từng từ trong bộ từ vựng trước khi đưa qua Softmax.
Vấn đề nêu ở đoạn trên: bộ mã hóa "nhìn" được toàn bộ câu, kể cả từ mà nó cần đoán. Nếu bắt nó đoán từ ở vị trí i mà từ đó vẫn nằm sẵn trong đầu vào thì nó chỉ việc "chép" lại — giống như làm bài khi đáp án in ngay trong đề. Vì thế cần một cách đặt bài toán khác, chẳng hạn *che* từ cần đoán đi, như mục tiếp theo trình bày.

1. Mô hình hóa ngôn ngữ có che

Một trong những phương pháp tiền huấn luyện bộ mã hóa phổ biến nhất là **mô hình hóa ngôn ngữ có che** (*masked language modeling*), là nền tảng của mô hình BERT nổi tiếng [Devlin và cộng sự, 2019]. Ý tưởng của mô hình hóa ngôn ngữ có che là tạo ra các thử thách dự đoán bằng cách che đi một số token trong chuỗi đầu vào và huấn luyện mô hình dự đoán các token bị che. Theo nghĩa này, bài toán mô hình hóa ngôn ngữ thông thường, đôi khi được gọi là **mô hình hóa ngôn ngữ nhân quả** (*causal language modeling*), là một trường hợp đặc biệt của mô hình hóa ngôn ngữ có che: tại mỗi vị trí, ta che các token trong ngữ cảnh bên phải, và dự đoán token ở vị trí này bằng ngữ cảnh bên trái của nó. Tuy nhiên, trong mô hình hóa ngôn ngữ nhân quả, chỉ ngữ cảnh bên trái được dùng để dự đoán token, và mô hình không được truy cập các token trong ngữ cảnh bên phải. Ngược lại, trong mô hình hóa ngôn ngữ có che, tất cả các token không bị che đều được dùng để dự đoán từ, dẫn tới một mô hình hai chiều (*bidirectional*) đưa ra dự đoán dựa trên cả ngữ cảnh bên trái lẫn bên phải.

Một cách hình thức hơn, với chuỗi đầu vào $\mathbf{x} = x_0 \cdots x_m$, giả sử ta che các token tại các vị trí $\mathcal{A}(\mathbf{x}) = \{i_1, \cdots, i_u\}$. Khi đó ta thu được một chuỗi token bị che $\bar{\mathbf{x}}$, trong đó token tại mỗi vị trí thuộc $\mathcal{A}(\mathbf{x})$ được thay bằng một ký hiệu đặc biệt [MASK]. Ví dụ, với chuỗi sau

The early bird catches the worm
(nghĩa đen: Con chim dậy sớm bắt được sâu)

ta có thể có một chuỗi token bị che như thế này

The [MASK] bird catches the [MASK]

trong đó ta che các token *early* và *worm* (tức là $i_1 = 2$ và $i_2 = 6$).

Bây giờ ta có hai chuỗi $\mathbf{x}$ và $\bar{\mathbf{x}}$. Mô hình sau đó được tối ưu hóa sao cho ta có thể dự đoán đúng $\mathbf{x}$ dựa trên $\bar{\mathbf{x}}$. Điều này có thể được xem như một quá trình giống tự mã hóa (*autoencoding*), và mục tiêu huấn luyện là cực đại hóa xác suất tái tạo $\Pr(\mathbf{x}|\bar{\mathbf{x}})$. Lưu ý rằng có một sự tương ứng đơn giản theo từng vị trí giữa $\mathbf{x}$ và $\bar{\mathbf{x}}$. Vì một token không bị che trong $\bar{\mathbf{x}}$ giống hệt token trong $\mathbf{x}$ ở cùng vị trí, ta không cần xét việc dự đoán cho token không bị che này. Điều này dẫn tới một mục tiêu huấn luyện đơn giản hơn, chỉ cực đại hóa xác suất cho các token bị che. Ta có thể biểu diễn mục tiêu này theo kiểu ước lượng hợp

lý cực đại

$$(\widehat{\mathbf{W}}, \hat{\theta}) = \arg\max_{\mathbf{W},\theta} \sum_{\mathbf{x}\in\mathcal{D}} \sum_{i\in\mathcal{A}(\mathbf{x})} \log \Pr_i^{\mathbf{W},\theta}(x_i|\bar{\mathbf{x}}), \tag{7.9}$$

hoặc biểu diễn nó bằng hàm mất mát entropy chéo

$$(\widehat{\mathbf{W}}, \hat{\theta}) = \arg\min_{\mathbf{W},\theta} \sum_{\mathbf{x}\in\mathcal{D}} \sum_{i\in\mathcal{A}(\mathbf{x})} \text{CrossEntropy}(\mathbf{p}_i^{\mathbf{W},\theta}, \mathbf{p}_i^{\text{gold}}), \tag{7.10}$$

trong đó $\Pr_k^{\mathbf{W},\theta}(x_k|\bar{\mathbf{x}})$ là xác suất của token đúng x_k tại vị trí k khi biết đầu vào đã bị làm hỏng $\bar{\mathbf{x}}$, và $\mathbf{p}_k^{\mathbf{W},\theta}$ là phân phối xác suất tại vị trí k khi biết đầu vào bị làm hỏng $\bar{\mathbf{x}}$. Để minh họa, xét ví dụ trên, trong đó hai token của chuỗi "*the early bird catches the worm*" bị che. Với ví dụ này, mục tiêu là cực đại hóa tổng các log-xác suất

$$\begin{aligned}\text{Loss} = \log \Pr(x_2 = \textit{early}|\bar{\mathbf{x}} = [\text{CLS}]\ \textit{The}\ \underbrace{[\text{MASK}]}_{\bar{x}_2}\ \textit{bird catches the}\ \underbrace{[\text{MASK}]}_{\bar{x}_6}) + \\ \log \Pr(x_6 = \textit{worm}|\bar{\mathbf{x}} = [\text{CLS}]\ \textit{The}\ \underbrace{[\text{MASK}]}_{\bar{x}_2}\ \textit{bird catches the}\ \underbrace{[\text{MASK}]}_{\bar{x}_6}).\end{aligned} \tag{7.11}$$

Khi đã thu được các tham số tối ưu $\widehat{\mathbf{W}}$ và $\hat{\theta}$, ta có thể bỏ $\widehat{\mathbf{W}}$. Sau đó, ta có thể tinh chỉnh thêm bộ mã hóa đã tiền huấn luyện $\text{Encode}_{\hat{\theta}}(\cdot)$ hoặc áp dụng trực tiếp nó cho các nhiệm vụ hạ nguồn.

Chú giải của người dịch: Che từ để học – giống bài tập điền từ

Mô hình hóa ngôn ngữ có che giống hệt bài tập "điền vào chỗ trống" trong môn Ngoại ngữ. Với câu "[CLS] The [MASK] bird catches the [MASK]", mô hình phải đoán ô 1 là *early*, ô 2 là *worm*, và được "chấm điểm" bằng log xác suất nó gán cho các đáp án đúng — Phương trình (7.11) chính là tổng hai điểm số đó (hai số hạng ứng với hai ô trống). Lưu ý vị trí đếm từ 0: [CLS] là x_0, *The* là x_1, *early* là x_2, …, *worm* là x_6. Khác với mô hình ngôn ngữ nhân quả (chỉ nhìn bên trái), ở đây mô hình được nhìn cả hai phía: từ "bird" bên phải giúp đoán "early" bên trái.

2. Mô hình hóa ngôn ngữ hoán vị

Mặc dù mô hình hóa ngôn ngữ có che đơn giản và được áp dụng rộng rãi, nó lại phát sinh những vấn đề mới. Một nhược điểm là việc dùng token đặc biệt [MASK], vốn chỉ được dùng trong lúc huấn luyện mà không có lúc kiểm tra. Điều này dẫn tới sự sai lệch giữa huấn luyện và suy luận (*inference*). Hơn nữa, quá trình tự mã hóa bỏ qua sự phụ thuộc giữa các token bị che. Ví dụ, trong ví dụ đã bàn ở trên, việc dự đoán x_2 (tức token bị che thứ nhất) được thực hiện độc lập với x_6 (tức token bị che thứ hai), mặc dù x_6 và x_2 phụ thuộc lẫn nhau trong ngữ cảnh của câu.

Những vấn đề này có thể được giải quyết bằng cách tiếp cận tiền huấn luyện **mô hình hóa ngôn ngữ hoán vị** (*permuted language modeling*) [Yang và cộng sự, 2019]. Tương tự như mô hình hóa ngôn ngữ nhân quả, mô hình hóa ngôn ngữ hoán vị đưa ra các dự đoán token một cách tuần tự. Tuy nhiên, khác với mô hình hóa nhân quả trong đó các dự đoán tuân theo trình tự tự nhiên của văn bản (như từ trái sang phải hoặc từ phải sang trái), mô hình hóa ngôn ngữ hoán vị cho phép dự đoán theo thứ tự bất kỳ. Cách tiếp cận này đơn giản: ta xác định một thứ tự dự đoán token rồi huấn luyện mô hình theo cách mô hình hóa ngôn ngữ tiêu chuẩn, như đã mô tả ở Mục 7.2.1. Lưu ý rằng trong cách tiếp cận này, thứ tự thực của các token trong văn bản vẫn giữ nguyên, và chỉ thứ tự mà ta dự đoán các token này khác với mô hình hóa ngôn ngữ tiêu chuẩn. Ví dụ, xét một chuỗi gồm 5 token $x_0x_1x_2x_3x_4$. Gọi $\mathbf{e}_i$ là vectơ nhúng của x_i (tức là tổ hợp của nhúng token và nhúng vị trí (*positional embedding*)). Trong mô hình

hóa ngôn ngữ tiêu chuẩn, ta sẽ sinh chuỗi này theo thứ tự $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4$. Xác suất của chuỗi có thể được mô hình hóa thông qua một quá trình sinh.

$$\begin{aligned}\Pr(\mathbf{x}) = \; & \Pr(x_0) \cdot \Pr(x_1|x_0) \cdot \Pr(x_2|x_0, x_1) \cdot \Pr(x_3|x_0, x_1, x_2) \cdot \\ & \Pr(x_4|x_0, x_1, x_2, x_3).\end{aligned} \tag{7.12}$$

Trong thực tế, mô hình lấy điều kiện trên các vectơ nhúng token chứ không phải trên chính các token. Do đó, ta có thể viết lại biểu thức trên thành

$$\begin{aligned}\Pr(\mathbf{x}) = \; & \Pr(x_0) \cdot \Pr(x_1|\mathbf{e}_0) \cdot \Pr(x_2|\mathbf{e}_0, \mathbf{e}_1) \cdot \Pr(x_3|\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_2) \cdot \\ & \Pr(x_4|\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3).\end{aligned} \tag{7.13}$$

Bây giờ, hãy xét một thứ tự dự đoán token khác: $x_0 \rightarrow x_4 \rightarrow x_2 \rightarrow x_1 \rightarrow x_3$. Quá trình sinh chuỗi khi đó có thể được biểu diễn như sau:

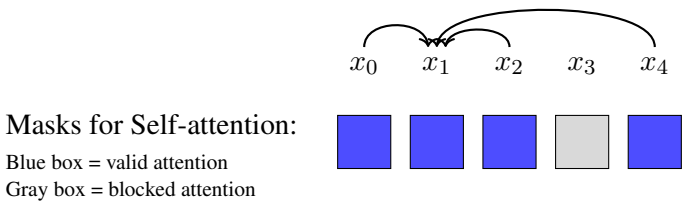
$$\begin{aligned}\Pr(\mathbf{x}) = \; & \Pr(x_0) \cdot \Pr(x_4|\mathbf{e}_0) \cdot \Pr(x_2|\mathbf{e}_0, \mathbf{e}_4) \cdot \Pr(x_1|\mathbf{e}_0, \mathbf{e}_4, \mathbf{e}_2) \cdot \\ & \Pr(x_3|\mathbf{e}_0, \mathbf{e}_4, \mathbf{e}_2, \mathbf{e}_1).\end{aligned} \tag{7.14}$$

Thứ tự dự đoán mới này cho phép một số token được sinh ra dựa trên một ngữ cảnh rộng hơn, thay vì chỉ giới hạn ở các token đứng trước như trong các mô hình ngôn ngữ tiêu chuẩn. Ví dụ, khi sinh x_3, mô hình xét cả ngữ cảnh bên trái (tức $\{\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_2\}$) lẫn ngữ cảnh bên phải (tức $\{\mathbf{e}_4\}$). Các vectơ nhúng $\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_4$ có chứa thông tin vị trí của x_0, x_1, x_2, x_4, nhờ đó giữ được thứ tự ban đầu của các token. Kết quả là cách tiếp cận này phần nào giống với mô hình hóa ngôn ngữ có che: ta che x_3 và dùng các token xung quanh nó là x_0, x_1, x_2, x_4 để dự đoán token này.

Chú giải của người dịch: Quy tắc nhân xác suất và "đổi thứ tự đoán"

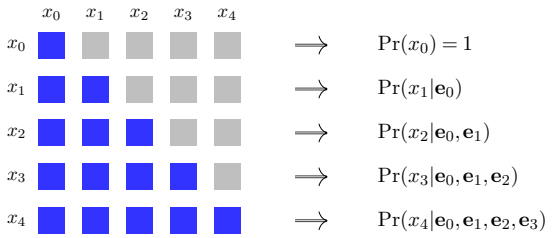
Phương trình (7.12) dựa trên quy tắc nhân: $\Pr(A, B) = \Pr(A) \cdot \Pr(B|A)$. Áp dụng liên tiếp, xác suất cả chuỗi bằng tích xác suất của từng token khi biết những token đã "đoán" trước nó. Quy tắc này đúng với *mọi* thứ tự, nên ta có thể đoán theo thứ tự x_0, x_4, x_2, x_1, x_3 như (7.14) mà xác suất cả chuỗi về lý thuyết không đổi. Điểm lợi: khi đoán x_3 ở bước cuối, mô hình đã biết cả x_4 (bên phải) — điều mà mô hình trái-sang-phải không làm được. Vì mỗi vectơ $\mathbf{e}_i$ có "gắn" thông tin vị trí, mô hình vẫn biết x_4 đứng cuối câu dù nó được đoán thứ hai.

Việc cài đặt các mô hình ngôn ngữ hoán vị tương đối dễ dàng với Transformer. Vì bản thân cơ chế tự chú ý (*self-attention*) là bất biến với hoán vị (*permutation-invariant*), và thông tin vị trí được mã hóa riêng bằng các vectơ nhúng vị trí, ta không cần sắp xếp lại một cách tường minh chuỗi đầu vào để có một phân tích thành tích như Phương trình (7.14). Thay vào đó, việc hoán vị có thể được thực hiện bằng cách đặt các mặt nạ (*mask*) thích hợp cho tự chú ý. Ví dụ, xét bước tính $\Pr(x_1|\mathbf{e}_0, \mathbf{e}_4, \mathbf{e}_2)$. Ta có thể giữ x_0, x_1, x_2, x_3, x_4 ở các vị trí ban đầu của chúng và chỉ cần ngăn x_1 chú ý tới x_3 trong quá trình tính tự chú ý, như minh họa dưới đây

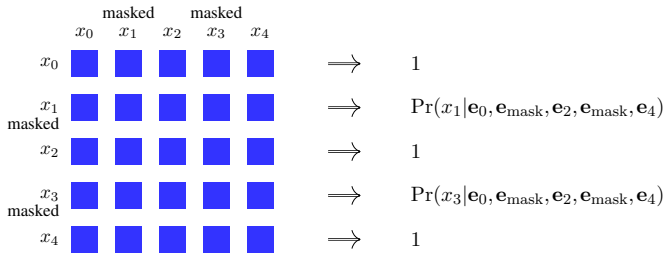


(Masks for Self-attention = mặt nạ cho tự chú ý; Blue box = ô xanh: chú ý hợp lệ; Gray box = ô xám: chú ý bị chặn)

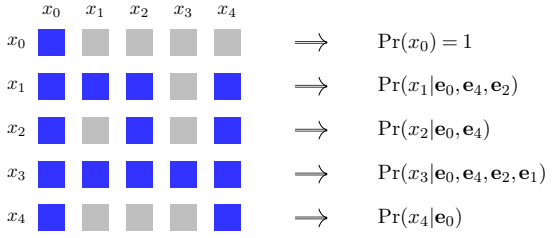
Để minh họa rõ hơn, ta so sánh kết quả đặt mặt nạ tự chú ý của mô hình hóa ngôn ngữ nhân quả, mô hình hóa ngôn ngữ có che và mô hình hóa ngôn ngữ hoán vị trong Hình 7.3.



(a) Causal Language Modeling (order: $x_0 \to x_1 \to x_2 \to x_3 \to x_4$)



(b) Masked Language Modeling (order: $x_0, [\text{MASK}], x_2, [\text{MASK}], x_4 \to x_1, x_3$)



(c) Permuted Language Modeling (order: $x_0 \to x_4 \to x_2 \to x_1 \to x_3$)

Hình 7.3: So sánh kết quả đặt mặt nạ tự chú ý của mô hình hóa ngôn ngữ nhân quả, mô hình hóa ngôn ngữ có che và mô hình hóa ngôn ngữ hoán vị. Ô màu xám biểu thị token ở vị trí j không chú ý tới token ở vị trí i. Ô màu xanh (i, j) biểu thị token ở vị trí j chú ý tới token ở vị trí i. $\mathbf{e}_{\text{mask}}$ biểu diễn vectơ nhúng của ký hiệu [MASK], là tổ hợp của nhúng token và nhúng vị trí.

Giải nghĩa nhãn trong hình: (a) Causal Language Modeling = mô hình hóa ngôn ngữ nhân quả; (b) Masked Language Modeling = mô hình hóa ngôn ngữ có che (masked = bị che); (c) Permuted Language Modeling = mô hình hóa ngôn ngữ hoán vị; order = thứ tự dự đoán.

Chú giải của người dịch: Đọc bảng mặt nạ trong Hình 7.3

Mỗi hàng ứng với một token đang được dự đoán, mỗi cột là một token mà nó có thể "nhìn" tới. Ô xanh = được nhìn, ô xám = bị che mắt. Ở (a), hàng x_3 chỉ xanh ở các cột x_0, x_1, x_2, x_3: nó chỉ thấy quá khứ (tam giác dưới). Ở (c), hàng x_1 xanh ở cột x_0, x_2, x_4 (cùng chính nó) nhưng xám ở x_3 — đúng như $\Pr(x_1|\mathbf{e}_0, \mathbf{e}_4, \mathbf{e}_2)$, vì theo thứ tự $x_0 \to x_4 \to x_2 \to x_1 \to x_3$ thì x_3 chưa được "đoán" khi đến lượt x_1. Như vậy chỉ cần thay

đổi mặt nạ, không cần xáo trộn câu.

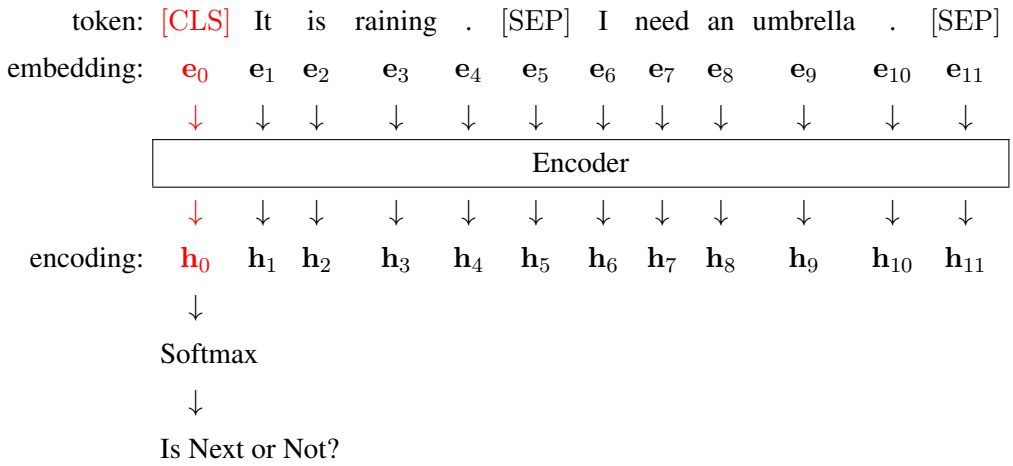
3. Tiền huấn luyện bộ mã hóa như các bộ phân loại

Một cách tiếp cận phổ biến khác để huấn luyện bộ mã hóa là dùng các nhiệm vụ phân loại. Trong học tự giám sát, điều này thường được thực hiện bằng cách tạo ra những thử thách phân loại mới từ văn bản chưa gán nhãn. Có nhiều cách khác nhau để thiết kế các nhiệm vụ phân loại. Ở đây chúng tôi trình bày hai nhiệm vụ phổ biến.

Một phương pháp đơn giản, gọi là **dự đoán câu tiếp theo** (*next sentence prediction*) (**NSP**), được trình bày trong bài báo gốc của BERT [Devlin và cộng sự, 2019]. NSP dựa trên giả định rằng một bộ mã hóa văn bản tốt cần nắm bắt được mối quan hệ giữa hai câu. Để mô hình hóa mối quan hệ này, NSP dùng đầu ra của bộ mã hóa cho hai câu liên tiếp Sent_A và Sent_B để xác định xem Sent_B có phải là câu tiếp theo ngay sau Sent_A hay không. Ví dụ, giả sử $\text{Sent}_A =$ '*It is raining* .' (Trời đang mưa.) và $\text{Sent}_B =$ '*I need an umbrella* .' (Tôi cần một cái ô.). Chuỗi đầu vào của bộ mã hóa có thể là

[CLS] It is raining . [SEP] I need an umbrella . [SEP]

trong đó [CLS] là ký hiệu bắt đầu (tức x_0) thường được dùng trong tiền huấn luyện bộ mã hóa, và [SEP] là một ký hiệu phân tách (*separator*) ngăn cách hai câu. Việc xử lý chuỗi này tuân theo quy trình mã hóa Transformer tiêu chuẩn: đầu tiên ta biểu diễn mỗi token x_i bằng vectơ nhúng tương ứng $\mathbf{e}_i$, rồi đưa chuỗi vectơ nhúng $\{\mathbf{e}_0, \cdots, \mathbf{e}_m\}$ vào bộ mã hóa để thu được chuỗi đầu ra $\{\mathbf{h}_0, \cdots, \mathbf{h}_m\}$. Vì $\mathbf{h}_0$ thường được xem là biểu diễn của toàn bộ chuỗi, ta thêm một tầng Softmax lên trên nó để xây dựng một hệ thống phân loại nhị phân (*binary classification*). Quá trình này được minh họa như sau:



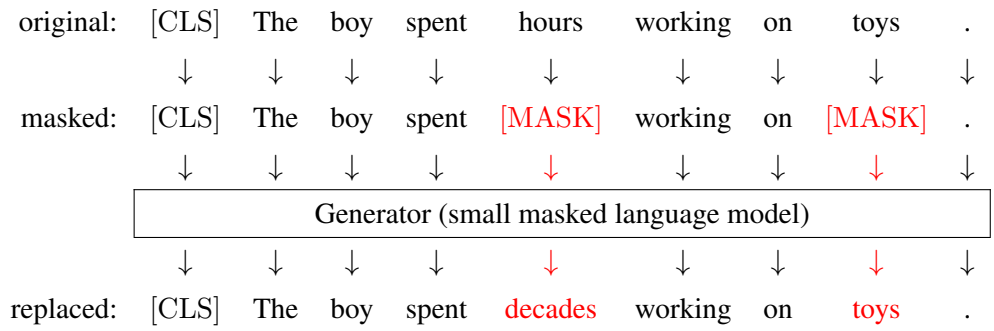
(token / embedding / encoding = token / vectơ nhúng / mã hóa; Encoder = bộ mã hóa; Is Next or Not? = có phải câu tiếp theo hay không?)

Để tạo các mẫu huấn luyện, ta trích các cặp câu từ một kho ngữ liệu văn bản. Một mẫu dương (*positive sample*) được tạo bằng cách chọn hai câu thực sự liên tiếp nhau, còn một mẫu âm (*negative sample*) được tạo bằng cách ghép một câu với một câu được chọn ngẫu nhiên từ kho ngữ liệu. Nhờ vậy, việc huấn luyện mô hình này giống hệt như huấn luyện một bộ phân loại. Thông thường, NSP được dùng như một hàm mất mát huấn luyện bổ sung cho tiền huấn luyện dựa trên mô hình hóa ngôn ngữ có che.

Chú giải của người dịch: Ví dụ mẫu dương và mẫu âm cho NSP

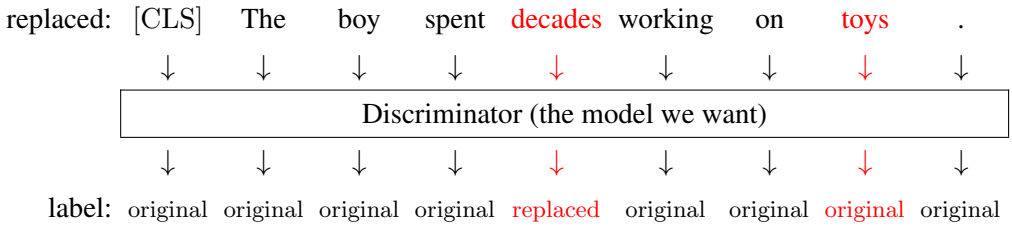
Mẫu dương: "Trời đang mưa." + "Tôi cần một cái ô." → nhãn *Đúng là câu tiếp theo*. Mẫu âm: "Trời đang mưa." + "Giá vàng hôm nay giảm nhẹ." (câu lấy ngẫu nhiên ở chỗ khác) → nhãn *Không phải*. Nhãn được tạo tự động từ chính văn bản, không cần ai gán — đó là lý do đây vẫn là học *tự giám sát*. "Nhị phân" nghĩa là chỉ có hai nhãn.

Cách tiếp cận thứ hai để huấn luyện bộ mã hóa Transformer như bộ phân loại là áp dụng các tín hiệu giám sát dựa trên phân loại ở mức token. Ví dụ, Clark và cộng sự [2019b] đề xuất, trong mô hình ELECTRA của họ, huấn luyện một bộ mã hóa Transformer để nhận biết mỗi token đầu vào có giống token gốc hay đã bị thay đổi theo cách nào đó. Bước đầu tiên của phương pháp này là sinh ra một chuỗi mới từ một chuỗi token cho trước, trong đó một số token bị thay đổi. Để làm điều này, ta dùng một mô hình ngôn ngữ có che cỡ nhỏ (gọi là bộ sinh (*generator*)): ta che ngẫu nhiên một số token và huấn luyện mô hình này dự đoán các token bị che. Với mỗi mẫu huấn luyện, mô hình ngôn ngữ có che này đưa ra một token tại mỗi vị trí bị che, token này có thể khác token gốc. Đồng thời, ta huấn luyện một bộ mã hóa Transformer khác (gọi là bộ phân biệt (*discriminator*)) để xác định mỗi token được dự đoán giống token gốc hay đã bị thay đổi. Cụ thể hơn, ta dùng bộ sinh để sinh ra một chuỗi trong đó một số token bị thay thế. Dưới đây là một minh họa.



(original = gốc; masked = bị che; replaced = sau thay thế; Generator (small masked language model) = bộ sinh (mô hình ngôn ngữ có che cỡ nhỏ). Câu gốc: "Cậu bé đã dành hàng giờ làm đồ chơi."; sau thay thế: "Cậu bé đã dành hàng chục năm làm đồ chơi.")

Sau đó, ta dùng bộ phân biệt để gán nhãn cho mỗi token là original (gốc) hay replaced (bị thay thế), như sau



(Discriminator (the model we want) = bộ phân biệt (mô hình ta cần); label = nhãn)

Chú giải của người dịch: Vì sao "toys" vẫn có nhãn original?

Ở vị trí bị che thứ hai, bộ sinh đoán ra đúng từ gốc *toys*, nên dù vị trí này từng bị che, token cuối cùng vẫn trùng với bản gốc và được gán nhãn *original*. Chỉ *decades* (thay cho *hours*) mới là *replaced*. Bộ phân biệt

giống một "thám tử" phải tìm ra những từ bị đánh tráo trong câu; để làm tốt, nó phải hiểu ngữ cảnh rất kỹ — và chính khả năng hiểu này là thứ ta muốn có ở một bộ mã hóa tốt. Khác với BERT chỉ học từ khoảng 15% token bị che, ELECTRA nhận tín hiệu học ở *mọi* token trong câu.

Khi huấn luyện, bộ sinh được tối ưu hóa như một mô hình ngôn ngữ có che bằng ước lượng hợp lý cực đại, còn bộ phân biệt được tối ưu hóa như một bộ phân loại bằng hàm mất mát dựa trên phân loại. Trong ELECTRA, hàm mất mát dựa trên hợp lý cực đại và hàm mất mát dựa trên phân loại được kết hợp để huấn luyện đồng thời cả bộ sinh lẫn bộ phân biệt. Một cách tiếp cận khác là dùng mạng đối sinh (*generative adversarial networks*) (GAN), tức là bộ sinh được huấn luyện để đánh lừa bộ phân biệt, còn bộ phân biệt được huấn luyện để phân biệt đầu ra của bộ sinh với phân phối thật. Tuy nhiên, kiểu huấn luyện GAN làm nhiệm vụ huấn luyện phức tạp hơn và khó mở rộng quy mô hơn. Dù sao, một khi việc huấn luyện hoàn tất, bộ sinh sẽ bị loại bỏ, và phần mã hóa của bộ phân biệt được dùng làm mô hình đã tiền huấn luyện cho các nhiệm vụ hạ nguồn.

7.2.3 Tiền huấn luyện bộ mã hóa–bộ giải mã

Trong NLP, kiến trúc bộ mã hóa–bộ giải mã thường được dùng để mô hình hóa các bài toán chuỗi-sang-chuỗi (*sequence-to-sequence*), chẳng hạn dịch máy và hỏi đáp. Ngoài những bài toán chuỗi-sang-chuỗi điển hình này trong NLP, các mô hình bộ mã hóa–bộ giải mã có thể được mở rộng để xử lý nhiều bài toán khác. Một ý tưởng đơn giản là coi văn bản vừa là đầu vào vừa là đầu ra của một bài toán, nhờ đó ta có thể áp dụng trực tiếp các mô hình bộ mã hóa–bộ giải mã. Ví dụ, cho một văn bản đầu vào, ta có thể chỉ dẫn mô hình sinh ra văn bản mô tả cảm xúc của nó (chẳng hạn *positive* – tích cực, *negative* – tiêu cực, hoặc *neutral* – trung tính).

Ý tưởng như vậy cho phép ta phát triển một hệ thống văn bản-sang-văn bản (*text-to-text*) duy nhất để giải quyết bất kỳ bài toán NLP nào. Ta có thể đưa các bài toán khác nhau về cùng một định dạng văn bản-sang-văn bản. Đầu tiên, ta huấn luyện một mô hình bộ mã hóa–bộ giải mã để thu nhận tri thức đa dụng về ngôn ngữ thông qua tự giám sát. Mô hình này sau đó được tinh chỉnh cho các nhiệm vụ hạ nguồn cụ thể bằng dữ liệu văn bản-sang-văn bản tương ứng.

1. Tiền huấn luyện bộ mã hóa–bộ giải mã có che

Trong mô hình **T5** của Raffel và cộng sự [2020], nhiều nhiệm vụ khác nhau được đưa về cùng một nhiệm vụ văn bản-sang-văn bản. Mỗi mẫu trong T5 tuân theo định dạng

$$\text{Văn bản nguồn} \quad \rightarrow \quad \text{Văn bản đích}$$

Ở đây $\rightarrow$ ngăn cách văn bản nguồn, gồm một mô tả nhiệm vụ hay chỉ dẫn cùng với đầu vào đưa cho hệ thống, với văn bản đích, là câu trả lời cho nhiệm vụ đầu vào. Ví dụ, xét nhiệm vụ dịch từ tiếng Trung sang tiếng Anh. Một mẫu huấn luyện có thể được biểu diễn như sau:

$$\text{[CLS] Dịch từ tiếng Trung sang tiếng Anh: 你好!} \quad \rightarrow \quad \langle s \rangle \text{ Hello!}$$

trong đó [CLS] và $\langle s \rangle$ lần lượt là các ký hiệu bắt đầu ở phía nguồn và phía đích[5].

Tương tự, ta có thể biểu diễn các nhiệm vụ khác theo cùng cách này. Ví dụ

[CLS] Trả lời: Albert Einstein sinh năm nào?
$\rightarrow$ $\langle s \rangle$ Ông sinh ngày 14 tháng 3 năm 1879.

[5]Ta có thể dùng cùng một ký hiệu bắt đầu cho các chuỗi khác nhau. Ở đây chúng tôi dùng các ký hiệu khác nhau để phân biệt các chuỗi ở phía bộ mã hóa và phía bộ giải mã.

[CLS] Rút gọn câu: vị giáo sư, người đã công bố rất nhiều bài báo trong lĩnh vực của mình, sẽ có một bài giảng về chủ đề này vào tuần tới.
→ ⟨s⟩ Vị giáo sư giàu kinh nghiệm sẽ giảng bài vào tuần tới.

[CLS] Văn bản: John đã mua một chiếc xe mới. Giả thuyết: John có một chiếc xe.
→ ⟨s⟩ Kéo theo (*Entailment*)

[CLS] Chấm điểm bản dịch từ tiếng Anh sang tiếng Trung. Tiếng Anh: when in Rome, do as the Romans do. Tiếng Trung: 人 在 罗马 就 像 罗马 人 一样 做事 。
→ ⟨s⟩ 0.81

trong đó các chỉ dẫn được tô xám. Điều thú vị là trong ví dụ cuối cùng, ta đưa một bài toán chấm điểm liên tục về dạng một nhiệm vụ sinh văn bản. Mục tiêu của ta là sinh ra chuỗi văn bản biểu diễn con số 0.81, thay vì xuất ra một giá trị số liên tục.

Chú giải của người dịch: Giải nghĩa các ví dụ

Ví dụ thứ ba là nhiệm vụ *suy diễn ngôn ngữ tự nhiên*: cho một văn bản và một giả thuyết, mô hình phải cho biết văn bản có "kéo theo" (*entailment*) giả thuyết hay không — ở đây có, vì đã mua xe thì có xe. Ví dụ thứ tư chấm chất lượng bản dịch tục ngữ "when in Rome, do as the Romans do" (nhập gia tùy tục) trên thang từ 0 đến 1. Điểm mấu chốt: mọi nhiệm vụ, dù là trả lời câu hỏi, rút gọn câu, phân loại hay chấm điểm, đều được viết thành "văn bản vào → văn bản ra", nên chỉ cần *một* mô hình cho tất cả.

Cách tiếp cận mô tả ở trên mang lại một khuôn khổ mới cho việc hiểu và sinh ngôn ngữ phổ quát. Cả chỉ dẫn nhiệm vụ lẫn đầu vào của bài toán đều được cung cấp cho hệ thống dưới dạng văn bản. Hệ thống sau đó làm theo chỉ dẫn để hoàn thành nhiệm vụ. Phương pháp này gộp các bài toán khác nhau lại với nhau, với lợi ích là huấn luyện một mô hình duy nhất có thể thực hiện nhiều nhiệm vụ cùng lúc.

Nhìn chung, việc tinh chỉnh là cần thiết để thích nghi mô hình đã tiền huấn luyện cho một nhiệm vụ hạ nguồn cụ thể. Trong quá trình này, có thể dùng những cách khác nhau để chỉ dẫn mô hình về nhiệm vụ, chẳng hạn dùng tên ngắn của nhiệm vụ làm tiền tố cho chuỗi đầu vào thực tế, hoặc cung cấp một mô tả chi tiết về nhiệm vụ. Vì chỉ dẫn nhiệm vụ được cung cấp dưới dạng văn bản và được xử lý như một phần của đầu vào, năng lực làm theo chỉ dẫn tổng quát có thể được thu nhận ngay trong giai đoạn tiền huấn luyện. Điều này hỗ trợ rất nhiều cho học zero-shot. Chẳng hạn, các mô hình đã tiền huấn luyện thường có thể tổng quát hóa để giải những bài toán mới mà chỉ dẫn riêng cho nhiệm vụ đó chưa từng xuất hiện trong quá trình huấn luyện.

Đã có một số phương pháp học tự giám sát mạnh mẽ cho bộ mã hóa hoặc bộ giải mã Transformer. Áp dụng các phương pháp này để tiền huấn luyện các mô hình bộ mã hóa–bộ giải mã là tương đối đơn giản. Một lựa chọn phổ biến là huấn luyện các mô hình bộ mã hóa–bộ giải mã như các mô hình ngôn ngữ. Ví dụ, bộ mã hóa nhận một tiền tố (*prefix*) của chuỗi, còn bộ giải mã sinh ra phần còn lại của chuỗi. Tuy nhiên, điều này khác với mô hình hóa ngôn ngữ nhân quả tiêu chuẩn, trong đó toàn bộ chuỗi được sinh theo kiểu tự hồi quy bắt đầu từ token đầu tiên. Trong trường hợp của ta, bộ mã hóa xử lý tiền tố cùng một lúc, sau đó bộ giải mã dự đoán các token tiếp theo theo cách của mô hình hóa ngôn ngữ nhân quả. Nói chính xác hơn, đây là một bài toán **mô hình hóa ngôn ngữ tiền tố** (*prefix language modeling*): mô hình ngôn ngữ dự đoán phần chuỗi tiếp theo khi biết một tiền tố, đóng vai trò ngữ cảnh cho việc dự đoán.

Xét ví dụ sau

$$\underbrace{\text{[CLS] The puppies are frolicking}}_{\text{Tiền tố}} \rightarrow \underbrace{\langle s\rangle \text{ outside the house .}}_{\text{Phần chuỗi tiếp theo}}$$

(câu đầy đủ "The puppies are frolicking outside the house ." nghĩa là "Những chú cún con đang nô đùa bên ngoài ngôi nhà."). Ta có thể huấn luyện trực tiếp một mô hình bộ mã hóa–bộ giải mã bằng những ví dụ như thế này. Khi đó, bộ mã hóa học cách hiểu tiền tố, còn bộ giải mã học cách viết tiếp dựa trên sự hiểu đó. Đối với tiền huấn luyện quy mô lớn, rất dễ tạo ra một số lượng lớn ví dụ huấn luyện từ văn bản chưa gán nhãn.

Cần lưu ý rằng để các mô hình bộ mã hóa–bộ giải mã đã tiền huấn luyện hoạt động hiệu quả trong các nhiệm vụ đa ngôn ngữ (*multilingual*) và xuyên ngôn ngữ (*cross-lingual*) (ví dụ dịch máy), chúng cần được huấn luyện trên dữ liệu đa ngôn ngữ. Điều này thường đòi hỏi bộ từ vựng phải chứa các token của nhiều ngôn ngữ. Bằng cách đó, mô hình có thể học được các biểu diễn dùng chung giữa các ngôn ngữ khác nhau, từ đó có năng lực cả hiểu lẫn sinh ngôn ngữ trong bối cảnh đa ngôn ngữ và xuyên ngôn ngữ.

Cách tiếp cận thứ hai để tiền huấn luyện các mô hình bộ mã hóa–bộ giải mã là mô hình hóa ngôn ngữ có che. Trong cách tiếp cận này, như đã bàn ở Mục 7.2.2, các token trong một chuỗi được thay ngẫu nhiên bằng một ký hiệu che, rồi mô hình được huấn luyện để dự đoán các token bị che này dựa trên chuỗi bị làm hỏng.

Để minh họa, xét nhiệm vụ che và tái tạo câu

The puppies are frolicking outside the house .

Bằng cách che hai token (chẳng hạn *frolicking* và *the*), ta có đầu vào và đầu ra của mô hình theo kiểu BERT như sau

[CLS] The puppies are [MASK] outside [MASK] house .
$\rightarrow$ $\langle s\rangle$ ___ ___ ___ frolicking ___ the ___ ___

Ở đây __ biểu thị vị trí bị che mà tại đó ta không đưa ra dự đoán token. Bằng cách thay đổi tỉ lệ phần trăm token bị che trong văn bản, cách tiếp cận này có thể được tổng quát hóa về phía huấn luyện kiểu BERT hoặc huấn luyện kiểu mô hình hóa ngôn ngữ [Song và cộng sự, 2019]. Ví dụ, nếu ta che tất cả các token, thì mô hình được huấn luyện để sinh ra toàn bộ chuỗi

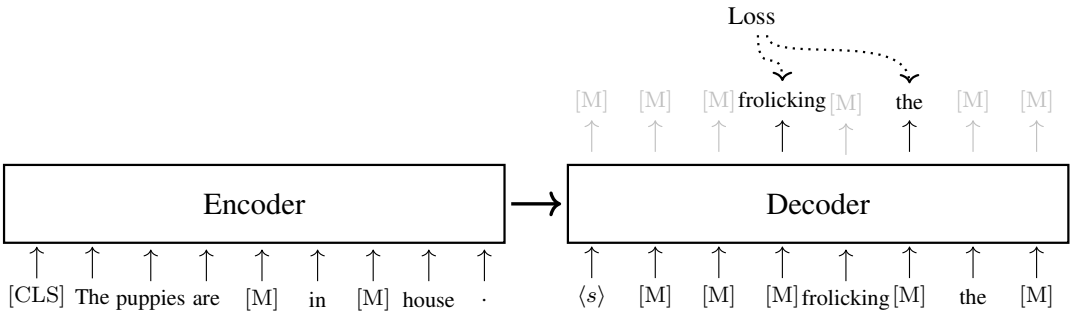
[CLS] [MASK] [MASK] [MASK] [MASK] [MASK] [MASK] [MASK] [MASK]
$\rightarrow$ $\langle s\rangle$ The puppies are frolicking outside the house .

Trong trường hợp này, ta huấn luyện bộ giải mã như một mô hình ngôn ngữ.

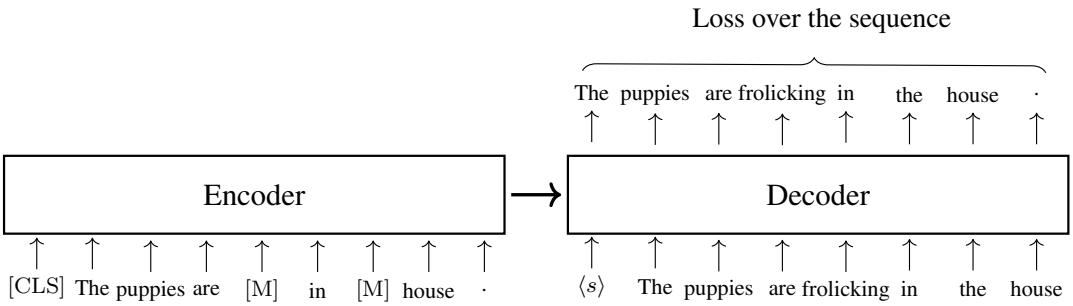
Lưu ý rằng, trong bối cảnh kiến trúc bộ mã hóa–bộ giải mã, ta có thể dùng bộ mã hóa để đọc chuỗi bị che, và dùng bộ giải mã để dự đoán chuỗi gốc. Với mục tiêu này, về bản chất ta có một bộ tự mã hóa khử nhiễu (*denoising autoencoder*): bộ mã hóa biến đổi đầu vào bị làm hỏng thành một biểu diễn ẩn nào đó, và bộ giải mã tái tạo đầu vào chưa bị làm hỏng từ biểu diễn ẩn này. Đây là một ví dụ về đầu vào và đầu ra cho huấn luyện khử nhiễu.

[CLS] The puppies are [MASK] outside [MASK] house .
$\rightarrow$ $\langle s\rangle$ The puppies are frolicking outside the house .

Bằng cách học ánh xạ từ chuỗi bị làm hỏng này sang bản gốc chưa bị làm hỏng của nó, mô hình có được khả năng hiểu ở phía bộ mã hóa và khả năng sinh ở phía bộ giải mã. Xem Hình 7.4 để thấy minh họa cách một mô hình bộ mã hóa–bộ giải mã được huấn luyện với mục tiêu kiểu BERT và mục tiêu tự mã hóa khử nhiễu.



(a) Training an encoder-decoder model with BERT-style masked language modeling



(b) Training an encoder-decoder model with denoising autoencoding

Hình 7.4: Minh họa việc huấn luyện một mô hình bộ mã hóa–bộ giải mã bằng phương pháp kiểu BERT và phương pháp tự mã hóa khử nhiễu. Trong cả hai phương pháp, đầu vào của bộ mã hóa là một chuỗi token bị làm hỏng, trong đó một số token bị che và thay bằng [MASK] (viết tắt là [M]). Bộ giải mã dự đoán các token bị che này, nhưng theo những cách khác nhau. Trong huấn luyện kiểu BERT, bộ giải mã chỉ cần tính mất mát cho các token bị che, còn các token còn lại trong chuỗi có thể đơn giản được coi là token [MASK]. Trong tự mã hóa khử nhiễu, bộ giải mã dự đoán chuỗi gồm tất cả các token theo kiểu tự hồi quy. Kết quả là mất mát được tính bằng cách cộng dồn mất mát của tất cả các token này, như trong mô hình hóa ngôn ngữ tiêu chuẩn.

Giải nghĩa nhãn trong hình: Encoder/Decoder = bộ mã hóa/bộ giải mã; Loss = mất mát; Loss over the sequence = mất mát trên toàn chuỗi; (a) huấn luyện mô hình bộ mã hóa–bộ giải mã với mô hình hóa ngôn ngữ có che kiểu BERT; (b) huấn luyện với tự mã hóa khử nhiễu.

Chú giải của người dịch: Khử nhiễu là gì?

"Nhiễu" ở đây là những hư hỏng ta cố ý gây ra cho câu (che từ, xóa từ, đảo thứ tự…). Bộ tự mã hóa khử nhiễu giống như người phục chế một bức ảnh cũ bị rách: nhìn bản hỏng, phải vẽ lại bản nguyên vẹn. Muốn phục chế đúng, người đó phải hiểu bức ảnh vẽ gì. Tương tự, để tái tạo câu gốc, mô hình buộc phải học ngữ pháp và ý nghĩa. Khác biệt giữa (a) và (b) trong Hình 7.4: ở (a) chỉ chấm điểm tại 2 ô bị che; ở (b) bộ giải mã viết lại cả câu, và mọi token đều góp vào mất mát.

Khi chọn ngẫu nhiên các token để che, ta có thể che các token liên tiếp nhau [Joshi và cộng sự, 2020]. Đây là một ví dụ.

[CLS] The puppies are [MASK] outside [MASK] [MASK] .
→ ⟨s⟩ The puppies are frolicking outside the house .

Một cách khác để xử lý các token bị che liên tiếp là biểu diễn chúng dưới dạng các đoạn (*span*). Ở đây ta theo công trình của Raffel và cộng sự [2020], dùng [X], [Y] và [Z] để ký hiệu các token canh gác (*sentinel token*) bao trùm một hoặc nhiều token bị che liên tiếp. Với ký hiệu này, ta có thể viết lại ví dụ huấn luyện ở trên thành:

[CLS] The puppies are [X] outside [Y] .
→ ⟨s⟩ [X] frolicking [Y] the house [Z]

Ý tưởng là ta biểu diễn chuỗi bị làm hỏng như một chuỗi chứa các ô giữ chỗ. Nhiệm vụ huấn luyện là điền vào các ô này những token đúng bằng cách dùng ngữ cảnh xung quanh. Một ưu điểm của cách tiếp cận này là các chuỗi dùng trong huấn luyện sẽ ngắn hơn, giúp việc huấn luyện hiệu quả hơn. Lưu ý rằng mô hình hóa ngôn ngữ có che cung cấp một khuôn khổ rất tổng quát để huấn luyện các mô hình bộ mã hóa–bộ giải mã. Có thể điều chỉnh nhiều thiết lập khác nhau để có những phiên bản huấn luyện khác nhau, chẳng hạn thay đổi tỉ lệ token bị che và độ dài tối đa của các đoạn bị che.

Chú giải của người dịch: Đọc ví dụ với token canh gác

Ở đầu vào, [X] thay cho "frolicking" và [Y] thay cho "the house". Đầu ra chỉ cần liệt kê: "[X] là frolicking, [Y] là the house", và [Z] đánh dấu kết thúc. Bộ giải mã không phải chép lại các từ không bị che (The, puppies, are, outside), nên chuỗi đích ngắn hơn hẳn — đó là lý do huấn luyện nhanh hơn.

2. Huấn luyện khử nhiễu

Nếu ta xem tiền huấn luyện bộ mã hóa–bộ giải mã là một bài toán tự mã hóa khử nhiễu, thì có rất nhiều cách làm hỏng đầu vào. Chẳng hạn, ngoài việc che token ngẫu nhiên, ta còn có thể sửa đổi token hoặc sắp xếp lại thứ tự của chúng.

Giả sử ta có một mô hình bộ mã hóa–bộ giải mã có thể ánh xạ một chuỗi đầu vào $\mathbf{x}$ thành một chuỗi đầu ra $\mathbf{y}$

$$\begin{aligned}\mathbf{y} &= \text{Decode}_\omega(\text{Encode}_\theta(\mathbf{x})) \\ &= \text{Model}_{\theta,\omega}(\mathbf{x}),\end{aligned} \tag{7.15}$$

trong đó θ và ω lần lượt là tham số của bộ mã hóa và bộ giải mã. Trong tự mã hóa khử nhiễu, ta thêm một ít nhiễu vào $\mathbf{x}$ để thu được một đầu vào bị nhiễu, bị làm hỏng $\mathbf{x}_{\text{noise}}$. Bằng cách đưa $\mathbf{x}_{\text{noise}}$ vào bộ mã

hóa, bộ giải mã được huấn luyện để tái tạo đầu vào gốc. Mục tiêu huấn luyện có thể được định nghĩa như sau:

$$(\hat{\theta}, \hat{\omega}) = \arg\min_{\theta,\omega} \text{Loss}(\text{Model}_{\theta,\omega}(\mathbf{x}_{\text{noise}}), \mathbf{x}), \tag{7.16}$$

Ở đây hàm mất mát $\text{Loss}(\text{Model}_{\theta,\omega}(\mathbf{x}_{\text{noise}}), \mathbf{x})$ đánh giá mức độ mô hình $\text{Model}_{\theta,\omega}(\mathbf{x}_{\text{noise}})$ tái tạo tốt đầu vào gốc $\mathbf{x}$ đến đâu. Như thường lệ, ta có thể chọn hàm mất mát entropy chéo.

Với kiến trúc mô hình và khuôn khổ huấn luyện đã có, vấn đề còn lại là làm hỏng đầu vào như thế nào. Lewis và cộng sự [2020a], trong mô hình **BART** của họ, đề xuất một số cách làm hỏng chuỗi đầu vào.

- **Che token** (*Token Masking*). Đây chính là phương pháp che mà ta đã dùng trong mô hình hóa ngôn ngữ có che. Các token được chọn ngẫu nhiên và thay bằng [MASK].

- **Xóa token** (*Token Deletion*). Phương pháp này tương tự che token. Tuy nhiên, thay vì thay các token được chọn bằng ký hiệu đặc biệt [MASK], ta xóa hẳn các token này khỏi chuỗi. Xem ví dụ sau để so sánh phương pháp che token và xóa token.

Gốc ($\mathbf{x}$):	The puppies are frolicking outside the house .
Che token ($\mathbf{x}_{\text{noise}}$):	The puppies are [MASK] outside [MASK] house .
Xóa token ($\mathbf{x}_{\text{noise}}$):	The puppies are ~~frolicking~~ outside ~~the~~ house .

  trong đó các token được gạch dưới trong chuỗi gốc là các token bị che hoặc bị xóa.

- **Che đoạn** (*Span Masking*). Các đoạn không chồng lấn nhau được lấy mẫu ngẫu nhiên trên chuỗi. Mỗi đoạn được che bằng [MASK]. Ta cũng cho phép các đoạn có độ dài 0, và trong những trường hợp đó, [MASK] đơn giản được chèn vào một vị trí trong chuỗi. Ví dụ, ta có thể dùng che đoạn để làm hỏng chuỗi trên như sau

Gốc ($\mathbf{x}$):	The 0 puppies are frolicking outside the house .
Che đoạn ($\mathbf{x}_{\text{noise}}$):	The [MASK] puppies are [MASK] house .

  Ở đây đoạn *frolicking outside the* được thay bằng một [MASK] duy nhất. 0 biểu thị một đoạn độ dài 0, vì vậy ta chèn một [MASK] vào giữa *The* và *puppies*. Che đoạn đặt ra những thử thách dự đoán mới, trong đó mô hình cần biết có bao nhiêu token được sinh ra từ một đoạn. Bài toán này rất giống với mô hình hóa độ sinh sản (*fertility modeling*) trong dịch máy [Brown và cộng sự, 1993].

Nếu ta xét một chuỗi gồm nhiều câu, có thể áp dụng thêm các phương pháp làm hỏng khác. Trong mô hình BART có hai phương pháp như vậy.

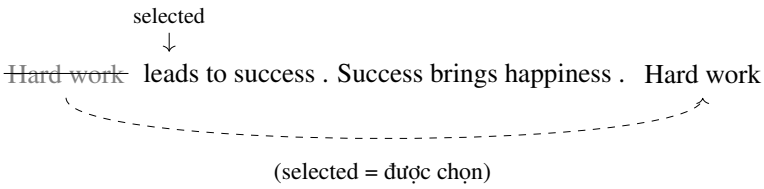
- **Sắp xếp lại câu** (*Sentence Reordering*). Phương pháp này hoán vị ngẫu nhiên các câu để mô hình có thể học cách sắp xếp lại các câu trong một văn bản. Ví dụ, xét hai câu liên tiếp

  Hard work leads to success . Success brings happiness .
  (Chăm chỉ dẫn tới thành công. Thành công mang lại hạnh phúc.)

  Ta có thể đảo thứ tự hai câu để có một chuỗi đầu vào bị làm hỏng

  Success brings happiness . Hard work leads to success .

- **Xoay văn bản** (*Document Rotation*). Mục tiêu của nhiệm vụ này là xác định token bắt đầu của chuỗi. Đầu tiên, một token được chọn ngẫu nhiên từ chuỗi. Sau đó, chuỗi được xoay vòng sao cho token được chọn trở thành token đầu tiên. Ví dụ, giả sử ta chọn token *leads* từ chuỗi ở trên. Chuỗi sau khi xoay là



  trong đó dãy con *Hard work* đứng trước *leads* được nối vào cuối chuỗi.

Khi tiền huấn luyện, ta có thể áp dụng nhiều phương pháp làm hỏng để học được những mô hình bền vững (*robust*), chẳng hạn chọn ngẫu nhiên một trong số chúng cho mỗi mẫu huấn luyện. Trong thực tế, kết quả của tiền huấn luyện bộ mã hóa–bộ giải mã phụ thuộc rất nhiều vào các phương pháp làm hỏng đầu vào được sử dụng, vì vậy ta thường cần chọn các mục tiêu huấn luyện thích hợp thông qua thực nghiệm cẩn thận.

Chú giải của người dịch: Vì sao làm hỏng câu theo nhiều cách?

Mỗi kiểu làm hỏng "ép" mô hình học một kỹ năng khác: che/xóa token buộc mô hình đoán từ còn thiếu (với xóa token, mô hình còn phải tự tìm ra *chỗ* bị xóa); che đoạn buộc nó đoán cả *số lượng* từ bị mất; sắp xếp lại câu buộc nó hiểu mạch lạc giữa các câu (câu nào là nguyên nhân, câu nào là kết quả); xoay văn bản buộc nó nhận ra đâu là điểm bắt đầu tự nhiên của một đoạn văn. Giống như ôn thi bằng nhiều dạng bài tập khác nhau thì hiểu bài sâu hơn.

7.2.4 So sánh các nhiệm vụ tiền huấn luyện

Đến đây, chúng ta đã bàn về nhiều nhiệm vụ tiền huấn luyện khác nhau. Vì cùng một mục tiêu huấn luyện thường có thể được áp dụng cho các kiến trúc khác nhau (ví dụ, dùng mô hình hóa ngôn ngữ có che cho cả tiền huấn luyện chỉ với bộ mã hóa lẫn tiền huấn luyện bộ mã hóa–bộ giải mã), việc phân loại các nhiệm vụ này chỉ dựa trên kiến trúc mô hình là chưa đủ. Thay vào đó, ta tóm tắt chúng dựa trên mục tiêu huấn luyện.

- **Mô hình hóa ngôn ngữ** (*Language Modeling*). Thông thường, cách tiếp cận này chỉ một quá trình sinh chuỗi tự hồi quy. Tại mỗi bước, mô hình dự đoán token tiếp theo chỉ dựa trên ngữ cảnh đứng trước nó.

- **Mô hình hóa ngôn ngữ có che** (*Masked Language Modeling*). Mô hình hóa ngôn ngữ có che tuân theo một khuôn khổ tổng quát "che rồi dự đoán". Nó che ngẫu nhiên các token trong một chuỗi và dự đoán các token này bằng cách dùng toàn bộ chuỗi đã bị che.

- **Mô hình hóa ngôn ngữ hoán vị** (*Permuted Language Modeling*). Mô hình hóa ngôn ngữ hoán vị có nhiều điểm giống mô hình hóa ngôn ngữ có che, nhưng loại bỏ ràng buộc cố định từ trái sang phải bằng cách hoán vị thứ tự phân tích thành tích. Nó giữ nguyên chuỗi đầu vào nhưng sắp xếp lại thứ tự phân tích (dự đoán). Mỗi dự đoán dựa trên một số token ngữ cảnh được chọn ngẫu nhiên.

- **Huấn luyện phân biệt** (*Discriminative Training*). Trong huấn luyện phân biệt, tín hiệu giám sát được tạo ra từ các nhiệm vụ phân loại. Các mô hình đã tiền huấn luyện được tích hợp vào các bộ phân loại và được huấn luyện cùng với các phần còn lại của bộ phân loại để nâng cao hiệu năng phân loại.

- **Tự mã hóa khử nhiễu** (*Denoising Autoencoding*). Cách tiếp cận này được áp dụng cho tiền huấn luyện các mô hình bộ mã hóa–bộ giải mã. Đầu vào là một chuỗi bị làm hỏng (với các loại nhiễu như che, xóa hoặc hoán vị), và mô hình được huấn luyện để tái tạo chuỗi gốc.

Bảng 7.1 minh họa các phương pháp này và các biến thể của chúng bằng ví dụ. Mặc dù định dạng đầu vào/đầu ra trong các ví dụ nhìn chung không phụ thuộc vào mô hình, chúng tôi chỉ rõ các kiến trúc mô hình tương thích với mỗi nhiệm vụ tiền huấn luyện. Trong mỗi ví dụ, đầu vào là một chuỗi token, còn đầu ra là một chuỗi token hoặc một số xác suất. Với các nhiệm vụ sinh, chẳng hạn mô hình hóa ngôn ngữ, các chỉ số trên được dùng để chỉ thứ tự sinh ở phía đích. Nếu các chỉ số trên bị lược bỏ, điều đó có nghĩa là các token đầu ra có thể được dự đoán hoặc theo kiểu tự hồi quy hoặc song song (không tự hồi quy). Ở phía nguồn, ta giả định rằng chuỗi trải qua quá trình mã hóa Transformer tiêu chuẩn, nghĩa là mỗi token có thể nhìn thấy toàn bộ chuỗi trong cơ chế tự chú ý. Ngoại lệ duy nhất là mô hình hóa ngôn ngữ hoán vị, trong đó quá trình sinh tự hồi quy được thực hiện bằng cách đặt các mặt nạ chú ý ở phía bộ mã hóa. Để đơn giản hóa thảo luận, ta bỏ token $\langle s \rangle$ khỏi phía đích của mỗi ví dụ.

Phương pháp	Enc	Dec	E-D	Đầu vào	Đầu ra
LM nhân quả		•	•		The^1 kitten^2 is^3 chasing^4 the^5 ball^6 .^{7}
LM tiền tố		•	•	[C] The kitten is	chasing^1 the^2 ball^3 .^{4}
LM có che	•		•	[C] The kitten [M] chasing the [M] .	_ _ is _ _ ball _
Kiểu MASS	•		•	[C] The kitten [M] [M] [M] ball .	_ _ is chasing the _ _
Kiểu BERT	•		•	[C] The kitten [M] playing the [M] .	_ kitten is chasing _ ball _
LM hoán vị	•			[C] The kitten is chasing the ball .	The^5 kitten^7 is^6 chasing^1 the^4 ball^2 .^{3}
Dự đoán câu tiếp theo	•			[C] The kitten is chasing the ball . Birds eat worms .	Pr(IsNext | representation-of-[C])
So sánh câu	•			Sent$_a$: The kitten is chasing the ball . Sent$_b$: A cat is running after a toy .	Score(Sent$_a$, Sent$_b$)
Phân loại token	•			[C] The kitten is chasing the ball .	Pr(·|The) Pr(·|kitten) ... Pr(·|.)
Sắp xếp lại token			•	[C] . kitten the chasing The is ball	The^1 kitten^2 is^3 chasing^4 the^5 ball^6 .^{7}
Xóa token			•	[C] The kitten is ~~chasing~~ the ball .	The^1 kitten^2 is^3 chasing^4 the^5 ball^6 .^{7}
Che đoạn			•	[C] The kitten [M] is [M] .	The^1 kitten^2 is^3 chasing^4 the^5 ball^6 .^{7}
Che bằng token canh gác			•	[C] The kitten [X] the [Y]	[X]^1 is^2 chasing^3 [Y]^4 ball^5 .^{6}
Sắp xếp lại câu			•	[C] The ball rolls away swiftly . The kitten is chasing the ball .	The^1 kitten^2 is^3 chasing^4 the^5 ball^6 .^{7} The^8 ball^9 rolls^{10} away^{11} swiftly^{12} .^{13}
Xoay văn bản			•	[C] chasing the ball . The ball rolls away swiftly . The kitten is	The^1 kitten^2 is^3 chasing^4 the^5 ball^6 .^{7} The^8 ball^9 rolls^{10} away^{11} swiftly^{12} .^{13}

Bảng 7.1: So sánh các nhiệm vụ tiền huấn luyện, gồm mô hình hóa ngôn ngữ, mô hình hóa ngôn ngữ có che, mô hình hóa ngôn ngữ hoán vị, huấn luyện phân biệt và tự mã hóa khử nhiễu. [C] = [CLS], [M] = [MASK], [X], [Y] = token canh gác. Enc, Dec và E-D cho biết cách tiếp cận có thể được áp dụng lần lượt cho mô hình chỉ có bộ mã hóa, chỉ có bộ giải mã, và bộ mã hóa–bộ giải mã hay không. Với các nhiệm vụ sinh, chỉ số trên được dùng để biểu diễn thứ tự của các token. (LM = mô hình ngôn ngữ.)

Chú giải của người dịch: Đọc Bảng 7.1

Các ví dụ dùng câu "The kitten is chasing the ball ." (Chú mèo con đang đuổi theo quả bóng.) và "The ball rolls away swiftly ." (Quả bóng lăn đi thật nhanh.); "Birds eat worms ." (Chim ăn sâu.) là câu ngẫu nhiên không liên quan, dùng làm mẫu âm cho dự đoán câu tiếp theo; "A cat is running after a toy ." (Một con mèo đang chạy theo món đồ chơi.) dùng để chấm độ tương đồng với câu thứ nhất. Chỉ số trên cho biết thứ tự sinh: ở dòng LM hoán vị, "chasing" được đoán đầu tiên (1), rồi "ball" (2), "." (3)…. Dấu _ là vị trí không cần dự đoán. Ở dòng kiểu BERT, từ "playing" tô xám là một token đã bị thay bằng từ ngẫu nhiên (thay cho "chasing" gốc), một kỹ thuật sẽ được trình bày ở Mục 7.3.

Mặc dù các nhiệm vụ tiền huấn luyện này khác nhau, ta vẫn có thể so sánh chúng trong cùng một khuôn khổ và thiết lập thực nghiệm [Dong và cộng sự, 2019; Raffel và cộng sự, 2020; Lewis và cộng sự, 2020a]. Lưu ý rằng chúng tôi không thể liệt kê tất cả các nhiệm vụ tiền huấn luyện ở đây vì có rất nhiều nhiệm vụ như vậy. Để tìm hiểu thêm về các nhiệm vụ tiền huấn luyện, bạn đọc quan tâm có thể tham khảo một số bài tổng quan về chủ đề này [Qiu và cộng sự, 2020b; Han và cộng sự, 2021a].

7.3 Ví dụ: BERT

Trong mục này, chúng ta giới thiệu BERT, một trong những mô hình mã hóa chuỗi (*sequence encoding*) được tiền huấn luyện (*pre-trained*) sử dụng rộng rãi nhất trong NLP.

7.3.1 Mô hình chuẩn

Mô hình BERT chuẩn, do Devlin và cộng sự [2019] đề xuất, là một bộ mã hóa (*encoder*) Transformer hai chiều (*bidirectional*) được huấn luyện bằng hai nhiệm vụ tự giám sát (*self-supervised*): mô hình hóa ngôn ngữ có che (*masked language modeling*) và dự đoán câu tiếp theo (*next sentence prediction*). Hàm mất mát huấn luyện tổng thể đơn giản là tổng các hàm mất mát của hai nhiệm vụ này:

$$\text{Loss}_{\text{BERT}} = \text{Loss}_{\text{mlm}} + \text{Loss}_{\text{nsp}}. \tag{7.17}$$

Như thường lệ trong học sâu (*deep learning*), ta tối ưu các tham số của mô hình bằng cách cực tiểu hóa hàm mất mát này. Trong quá trình huấn luyện, ta lấy mẫu các lô nhỏ (*mini-batch*) từ một kho ngữ liệu (*corpus*) văn bản, tính $\text{Loss}_{\text{BERT}}$ trên mỗi lô, rồi cập nhật lặp đi lặp lại các tham số của mô hình bằng hạ gradient (*gradient descent*) hoặc các biến thể của nó. Quá trình này lặp lại cho đến khi thỏa mãn một tiêu chí dừng, chẳng hạn hàm mất mát huấn luyện hội tụ hoặc đạt tới số bước tối đa đã định trước.

Chú giải của người dịch: Vì sao chỉ cần cộng hai hàm mất mát?

Hàm mất mát là một con số đo mức độ "sai" của mô hình: càng nhỏ càng tốt. Ở đây mô hình phải làm tốt cùng lúc hai bài tập (đoán từ bị che và đoán hai câu có nối tiếp nhau không). Cộng hai mức sai lại thành một con số duy nhất giống như cộng điểm phạt của hai môn thi: muốn tổng điểm phạt nhỏ thì phải làm tốt cả hai môn. Mỗi bước huấn luyện, thuật toán hạ gradient điều chỉnh các tham số một chút theo hướng làm tổng này giảm xuống.

1. Các hàm mất mát

Nói chung, mô hình BERT được dùng để biểu diễn một câu đơn hoặc một cặp câu, nhờ vậy có thể xử lý nhiều bài toán hiểu ngôn ngữ (*language understanding*) hạ nguồn (*downstream*) khác nhau. Trong mục này, ta giả sử biểu diễn đầu vào là một chuỗi chứa hai câu Sent_A và Sent_B, có dạng

$$[\text{CLS}] \quad \text{Sent}_A \quad [\text{SEP}] \quad \text{Sent}_B \quad [\text{SEP}]$$

Ở đây ta theo cách ký hiệu của [Devlin và cộng sự, 2019] và dùng [SEP] để chỉ ký hiệu phân tách (*separator*).

Với chuỗi này, ta có thể tính riêng Loss_{mlm} và Loss_{nsp}. Với mô hình hóa ngôn ngữ có che, một tập con các token (thường là 15%) được chọn ngẫu nhiên để dự đoán. Sau đó chuỗi được sửa đổi theo ba cách:

- **Che token** (*Token Masking*). 80% số token được chọn bị che và thay bằng ký hiệu [MASK]. Ví dụ

  Gốc: [CLS] It is raining . [SEP] I need an umbrella . [SEP]
  Đã che: [CLS] It is [MASK] . [SEP] I need [MASK] umbrella . [SEP]

  trong đó các token được chọn được gạch dưới. Việc dự đoán các token bị che khuyến khích mô hình học các biểu diễn từ ngữ cảnh xung quanh.

- **Thay thế ngẫu nhiên** (*Random Replacement*). 10% số token được chọn bị đổi thành một token ngẫu nhiên. Ví dụ

  Gốc: [CLS] It is raining . [SEP] I need an umbrella . [SEP]
  Token ngẫu nhiên: [CLS] It is raining . [SEP] I need a hat . [SEP]

  Điều này giúp mô hình học cách khôi phục một token từ đầu vào đã bị làm hỏng.

- **Giữ nguyên** (*Unchanged*). 10% số token được chọn được giữ nguyên. Ví dụ,

  Gốc: [CLS] It is raining . [SEP] I need an umbrella . [SEP]
  Token giữ nguyên: [CLS] It is raining . [SEP] I need an umbrella . [SEP]

  Mặc dù dự đoán một token không bị thay đổi là việc dễ dàng, chiến lược này lại rất quan trọng. Nó khuyến khích mô hình duy trì một biểu diễn có ý nghĩa cho chính từ thực sự quan sát được, qua đó giúp giảm bớt sự chênh lệch giữa tiền huấn luyện (có dùng [MASK]) và tinh chỉnh (*fine-tuning*) (không dùng).

Chú giải của người dịch: Câu ví dụ và các ký hiệu đặc biệt

Câu ví dụ tiếng Anh "It is raining. I need an umbrella." nghĩa là "Trời đang mưa. Tôi cần một chiếc ô."; "a hat" là "một cái mũ". [CLS] (viết tắt của *classification*) là token đặc biệt luôn đặt ở đầu chuỗi, [SEP] đánh dấu chỗ kết thúc mỗi câu. Trò chơi ở đây giống bài tập "điền từ vào chỗ trống" trong sách tiếng Anh: mô hình nhìn các từ xung quanh ("trời mưa", "tôi cần … ô") để đoán từ bị che. Ví dụ về con số: với một chuỗi 100 token, khoảng 15 token được chọn; trong đó khoảng 12 token bị thay bằng [MASK], khoảng 1–2 token bị thay bằng từ ngẫu nhiên, và khoảng 1–2 token giữ nguyên. Mô hình không biết token nào thuộc loại nào, nên phải "nghi ngờ" mọi vị trí và luôn xây dựng biểu diễn tốt cho từng từ.

Gọi $\mathcal{A}(\mathbf{x})$ là tập các vị trí được chọn trong chuỗi token $\mathbf{x}$, và $\bar{\mathbf{x}}$ là chuỗi đã sửa đổi của $\mathbf{x}$. Hàm mất mát của mô hình hóa ngôn ngữ có che có thể được định nghĩa là:

$$\text{Loss}_{\text{mlm}} = -\sum_{i \in \mathcal{A}(\mathbf{x})} \log \Pr_i(x_i|\bar{\mathbf{x}}), \tag{7.18}$$

trong đó $\Pr_i(x_i|\bar{\mathbf{x}})$ là xác suất dự đoán đúng x_i tại vị trí i khi biết $\bar{\mathbf{x}}$. Hình 7.5 trình bày một ví dụ minh họa từng bước cách tính Loss_{mlm}.

Chú giải của người dịch: Đọc Phương trình (7.18)

$\Pr_i(x_i|\bar{\mathbf{x}})$ là xác suất mà mô hình gán cho *từ đúng* x_i tại vị trí i sau khi nhìn chuỗi đã bị sửa $\bar{\mathbf{x}}$. Nếu mô hình đoán tốt, xác suất này gần 1 nên log gần 0; nếu đoán kém, xác suất gần 0 nên log là số âm rất lớn. Dấu trừ đổi nó thành "điểm phạt" dương. Ví dụ: tại 4 vị trí được chọn, mô hình gán xác suất cho từ đúng lần lượt là 0.5, 0.8, 0.25, 0.9; khi đó $\text{Loss}_{\text{mlm}} = -(\ln 0.5 + \ln 0.8 + \ln 0.25 + \ln 0.9) \approx 0.69 + 0.22 + 1.39 + 0.11 = 2.41$. Lưu ý chỉ các vị trí được chọn mới được tính, các vị trí khác không đóng góp vào tổng.

Đối với dự đoán câu tiếp theo, ta làm theo phương pháp đã mô tả trong Mục 7.2.2. Mỗi mẫu huấn luyện được gán một nhãn từ tập $\{\text{IsNext}, \text{NotNext}\}$, ví dụ,

Chuỗi: [CLS] It is raining . [SEP] I need an umbrella . [SEP]
Nhãn: IsNext

Chuỗi: [CLS] The cat sleeps on the windowsill . [SEP] Apples grow on trees . [SEP]
Nhãn: NotNext

[6]Vectơ đầu ra của bộ mã hóa cho token đầu tiên [CLS] được xem là biểu diễn của cả chuỗi, ký hiệu là $\mathbf{h}_{\text{cls}}$ (hoặc $\mathbf{h}_0$). Một bộ phân loại (*classifier*) được xây dựng bên trên $\mathbf{h}_{\text{cls}}$. Khi đó, ta có thể tính xác suất của nhãn c khi biết $\mathbf{h}_{\text{cls}}$, tức là $\Pr(c|\mathbf{h}_{\text{cls}})$. Có nhiều hàm mất mát có thể chọn cho các bài toán phân loại. Ví dụ, trong huấn luyện theo ước lượng hợp lý cực đại (*maximum likelihood*), ta có thể định nghĩa Loss_{nsp} là

$$\text{Loss}_{\text{nsp}} = -\log \Pr(c_{\text{gold}}|\mathbf{h}_{\text{cls}}), \tag{7.19}$$

trong đó c_{gold} là nhãn đúng của mẫu này.

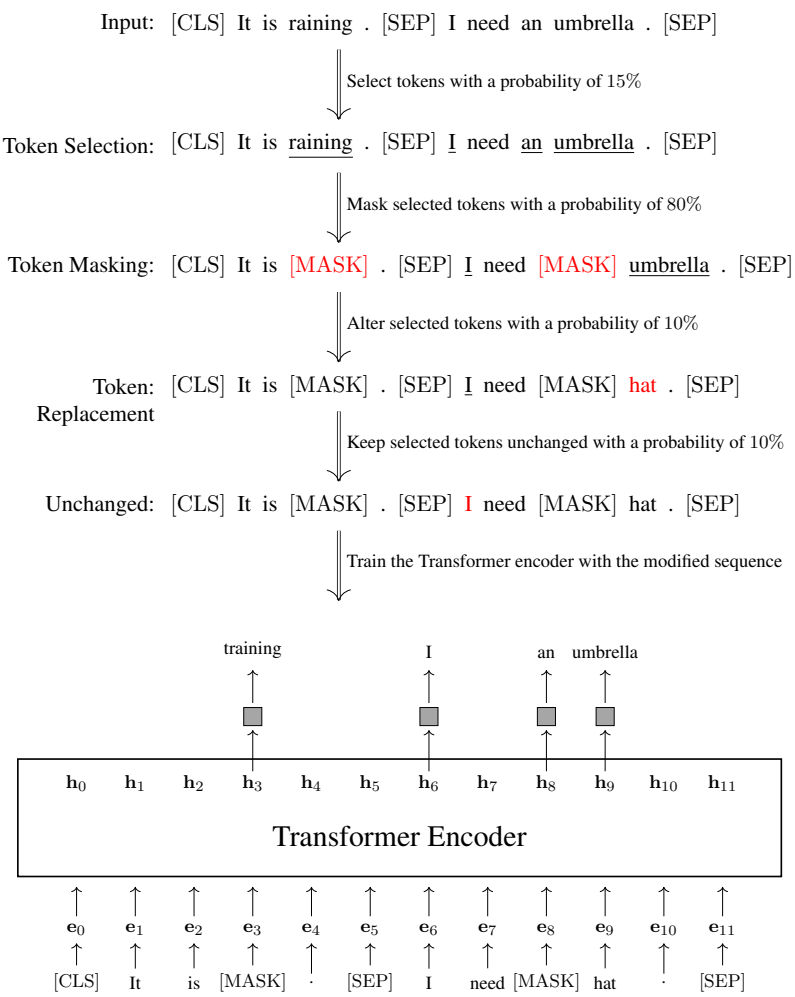
2. Thiết lập mô hình

Như được minh họa trong Hình 7.6, các mô hình BERT dựa trên kiến trúc bộ mã hóa Transformer chuẩn. Đầu vào là một chuỗi các vectơ nhúng (*embedding*), mỗi vectơ là tổng của nhúng token (*token embedding*), nhúng vị trí (*positional embedding*) và nhúng đoạn (*segment embedding*).

Giải nghĩa các nhãn trong Hình 7.5: Input – đầu vào; *Token Selection* – chọn token ("chọn token với xác suất 15%"); *Token Masking* – che token ("che các token được chọn với xác suất 80%"); *Token Replacement* – thay token ("thay đổi các token được chọn với xác suất 10%"); *Unchanged* – giữ nguyên ("giữ nguyên các token được chọn với xác suất 10%"); dòng cuối: "huấn luyện bộ mã hóa Transformer bằng chuỗi đã sửa đổi". Phía trên, mô hình phải đoán lại các từ gốc *raining*, *I*, *an*, *umbrella* tại các vị trí được chọn.

$$\mathbf{e} = \mathbf{x} + \mathbf{e}_{\text{pos}} + \mathbf{e}_{\text{seg}}. \tag{7.20}$$

[6]**[ND]** Câu thứ hai: "Con mèo ngủ trên bậu cửa sổ. Táo mọc trên cây." – hai câu không liên quan nên có nhãn NotNext (không phải câu tiếp theo); còn "Trời đang mưa. Tôi cần một chiếc ô." là hai câu nối tiếp hợp lý nên có nhãn IsNext (là câu tiếp theo).



Hình 7.5: Một ví dụ minh họa từng bước cho mô hình hóa ngôn ngữ có che kiểu BERT. Trước hết, 15% số token được chọn ngẫu nhiên. Các token được chọn này sau đó được xử lý theo một trong ba cách: thay bằng token [MASK] (80% số lần), thay bằng một token ngẫu nhiên (10% số lần), hoặc giữ nguyên (10% số lần). Mô hình được huấn luyện để dự đoán các token được chọn này dựa trên chuỗi đã sửa đổi. $\mathbf{e}_i$ biểu diễn vectơ nhúng của token tại vị trí i. Các ô màu xám biểu diễn các tầng Softmax.

Cả nhúng token ($\mathbf{x}$) và nhúng vị trí ($\mathbf{e}_{\text{pos}}$) đều giống hệt với những gì dùng trong Transformer nguyên bản. Nhúng đoạn ($\mathbf{e}_{\text{seg}}$) là thành phần mới được bổ sung, cho biết một token thuộc về Sent_A hay Sent_B. Điều này được minh họa qua ví dụ sau:

Token	[CLS]	It	is	raining	.	[SEP]	I	need	an	umbrella	.	[SEP]
$\mathbf{x}$	$\mathbf{x}_0$	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$	$\mathbf{x}_6$	$\mathbf{x}_7$	$\mathbf{x}_8$	$\mathbf{x}_9$	$\mathbf{x}_{10}$	$\mathbf{x}_{11}$
$\mathbf{e}_{\text{pos}}$	PE(0)	PE(1)	PE(2)	PE(3)	PE(4)	PE(5)	PE(6)	PE(7)	PE(8)	PE(9)	PE(10)	PE(11)
$\mathbf{e}_{\text{seg}}$	$\mathbf{e}_A$	$\mathbf{e}_A$	$\mathbf{e}_A$	$\mathbf{e}_A$	$\mathbf{e}_A$	$\mathbf{e}_A$	$\mathbf{e}_B$	$\mathbf{e}_B$	$\mathbf{e}_B$	$\mathbf{e}_B$	$\mathbf{e}_B$	$\mathbf{e}_B$

Chú giải của người dịch: Cộng ba vectơ nhúng

Mỗi token được mô tả bằng ba "thẻ thông tin", mỗi thẻ là một vectơ cùng số chiều: *nó là từ gì* ($\mathbf{x}$), *nó đứng ở vị trí thứ mấy* ($\mathbf{e}_{\text{pos}}$, ở đây $\text{PE}(i)$ là mã hóa vị trí của vị trí i), và *nó thuộc câu A hay câu B* ($\mathbf{e}_{\text{seg}}$, chỉ có hai giá trị $\mathbf{e}_A$, $\mathbf{e}_B$). Ba vectơ được cộng theo từng tọa độ như cộng vectơ ở lớp 10. Ví dụ thu nhỏ với 3 chiều: $\mathbf{x} = (0.2;\ 1.0;\ -0.5)$, $\mathbf{e}_{\text{pos}} = (0.1;\ 0;\ 0.3)$, $\mathbf{e}_{\text{seg}} = (0;\ -0.2;\ 0.1)$ cho $\mathbf{e} = (0.3;\ 0.8;\ -0.1)$. Cùng một từ "I" đặt ở câu A hay câu B, ở vị trí khác nhau, sẽ cho vectơ $\mathbf{e}$ khác nhau.

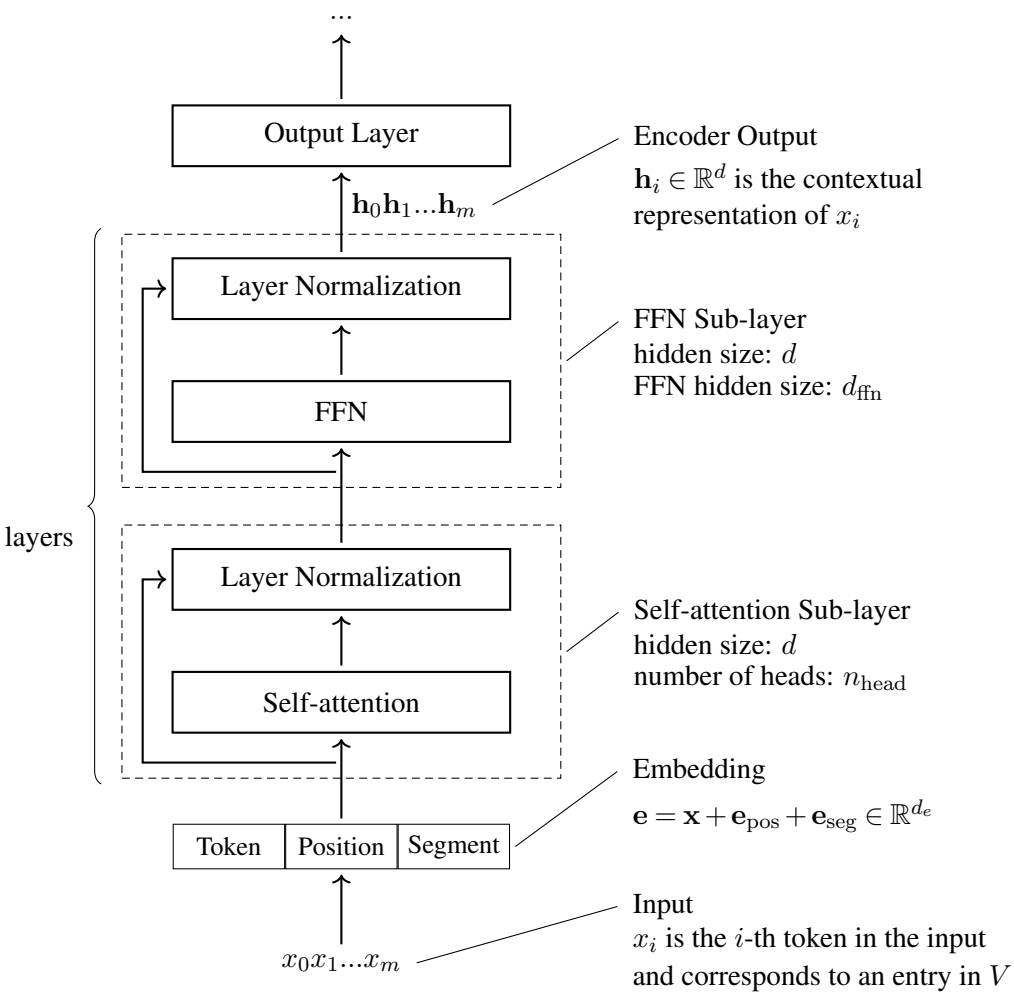
Phần chính của các mô hình BERT là một mạng Transformer nhiều tầng. Một tầng Transformer gồm một tầng con (*sub-layer*) tự chú ý và một tầng con FFN. Cả hai tầng con đều theo kiến trúc chuẩn hóa sau (*post-norm*): output $= \text{LNorm}(F(\text{input}) + \text{input})$, trong đó $F(\cdot)$ là hàm lõi của tầng con (hoặc là một mô hình tự chú ý (*self-attention*), hoặc là một FFN), và $\text{LNorm}(\cdot)$ là đơn vị chuẩn hóa tầng (*layer normalization*). Thông thường, nhiều tầng Transformer được xếp chồng lên nhau để tạo thành một mạng sâu. Với mỗi vị trí trong chuỗi, biểu diễn đầu ra cuối cùng là vectơ số thực do tầng trên cùng sinh ra.

Có một số khía cạnh mà ta có thể cân nhắc khi phát triển các mô hình BERT.

- **Kích thước bộ từ vựng** ($|V|$). Trong Transformer, mỗi token tương ứng với một mục trong bộ từ vựng (*vocabulary*) V. Bộ từ vựng lớn có thể bao phủ nhiều biến thể hình thức bề mặt của từ hơn, nhưng có thể dẫn tới yêu cầu lưu trữ tăng lên.
- **Kích thước nhúng** (d_e). Mỗi token được biểu diễn bằng một vectơ số thực d_e chiều. Như đã trình bày ở trên, vectơ này là tổng của nhúng token, nhúng vị trí và nhúng đoạn, tất cả đều cũng là các vectơ số thực d_e chiều.
- **Kích thước ẩn** (d). Đầu vào và đầu ra của một tầng con có d chiều. Ngoài ra, phần lớn các trạng thái ẩn (*hidden state*) của một tầng con là các vectơ d chiều. Nói chung, d có thể được xem gần đúng là độ rộng của mạng.
- **Số đầu chú ý** (n_{head}). Trong các tầng con tự chú ý, ta cần chỉ định số đầu dùng trong tự chú ý đa đầu (*multi-head self-attention*). Số đầu lớn hơn cho phép mô hình đồng thời chú ý tới thông tin từ nhiều không gian con biểu diễn (*representation subspaces*) hơn. Trong các hệ thống thực tế, ta thường đặt $n_{\text{head}} \geq 4$.
- **Kích thước ẩn của FFN** (d_{ffn}). Kích thước tầng ẩn của các FFN dùng trong Transformer thường lớn hơn d. Ví dụ, một thiết lập điển hình là $d_{\text{ffn}} = 4d$. Với các Transformer lớn hơn, chẳng hạn các mô hình lớn gần đây, d_{ffn} có thể được đặt ở một giá trị rất lớn.
- **Độ sâu mô hình** (L). Dùng mạng sâu là một cách hiệu quả để cải thiện năng lực biểu đạt (*expressive power*) của Transformer. Với các mô hình BERT, L thường được đặt là 12 hoặc 24. Tuy nhiên, các mạng còn sâu hơn nữa cũng khả thi và có thể được áp dụng để cải thiện thêm.

Các thiết lập khác nhau của những siêu tham số (*hyper-parameter*) này dẫn tới các kích thước mô hình khác nhau. Hai cấu hình được dùng rộng rãi là $\text{BERT}_{\text{base}}$ và $\text{BERT}_{\text{large}}$:

- BERT$_{\text{base}}$: $d = 768$, $L = 12$, $n_{\text{head}} = 12$, tổng số tham số $= 110$M.
- BERT$_{\text{large}}$: $d = 1{,}024$, $L = 24$, $n_{\text{head}} = 16$, tổng số tham số $= 340$M.



Hình 7.6: Kiến trúc mô hình của BERT (bộ mã hóa Transformer). Các token đầu vào trước hết được biểu diễn thành các vectơ nhúng, mỗi vectơ là tổng của nhúng token, nhúng vị trí và nhúng đoạn tương ứng. Sau đó, chuỗi vectơ nhúng được xử lý bởi một chồng các tầng Transformer. Mỗi tầng trong chồng này gồm một tầng con tự chú ý và một tầng con FFN. Đầu ra của mô hình BERT là một chuỗi vectơ do tầng Transformer cuối cùng sinh ra.

Giải nghĩa các nhãn trong Hình 7.6: Input – đầu vào (x_i là token thứ i của đầu vào, ứng với một mục trong V); *Embedding* – vectơ nhúng (*Token/Position/Segment*: nhúng token/vị trí/đoạn); *Self-attention Sub-layer* – tầng con tự chú ý (*hidden size*: kích thước ẩn, *number of heads*: số đầu); *FFN Sub-layer* – tầng con FFN (*FFN hidden size*: kích thước ẩn của FFN); *Layer Normalization* – chuẩn hóa tầng; *layers* – các tầng (khối được lặp lại L lần); *Output Layer* – tầng đầu ra; *Encoder Output* – đầu ra bộ mã hóa ($\mathbf{h}_i \in \mathbb{R}^d$ là biểu diễn theo ngữ cảnh của x_i).

Chú giải của người dịch: Hình dung các con số 110M, 340M

"M" là triệu (*million*): $\text{BERT}_{\text{base}}$ có khoảng 110 triệu tham số, tức 110 triệu con số thực cần học. Phần lớn tham số nằm trong các ma trận trọng số của các tầng. Ước lượng thô: mỗi tầng Transformer có khoảng $12d^2$ tham số (khoảng $4d^2$ cho tự chú ý và $8d^2$ cho FFN khi $d_{\text{ffn}} = 4d$). Với $d = 768$, $L = 12$: $12 \times 12 \times 768^2 \approx 85$ triệu; cộng thêm bảng nhúng token (khoảng 30 000 từ $\times 768 \approx 23$ triệu) là xấp xỉ 110 triệu. Làm tương tự với $d = 1024$, $L = 24$ cho khoảng 300 triệu, cộng bảng nhúng ra gần 340 triệu.

Việc huấn luyện các mô hình BERT tuân theo quy trình chuẩn cho Transformer. Huấn luyện các mô hình lớn hơn như $\text{BERT}_{\text{large}}$ đòi hỏi nhiều công sức và thời gian huấn luyện hơn. Đây là vấn đề phổ biến của tiền huấn luyện, đặc biệt khi mô hình được huấn luyện trên một lượng dữ liệu rất lớn. Trong thực tế, hiệu quả huấn luyện thường là mối quan tâm lớn. Một cách làm phổ biến là trước hết huấn luyện mô hình BERT trên các chuỗi tương đối ngắn trong một số lượng lớn bước huấn luyện, rồi tiếp tục huấn luyện nó trên các chuỗi có độ dài đầy đủ trong các bước huấn luyện còn lại.

7.3.2 Huấn luyện nhiều hơn và mô hình lớn hơn

BERT là một cột mốc trong NLP và đã khơi nguồn cho rất nhiều công trình tiếp theo. Một hướng là mở rộng quy mô tiền huấn luyện, thường gồm hai cách: tăng lượng dữ liệu huấn luyện và tăng số tham số của mô hình.

RoBERTa, một mở rộng của mô hình BERT chuẩn, là một ví dụ cho những nỗ lực như vậy [Liu và cộng sự, 2019]. Nó đưa ra hai cải tiến chính. Thứ nhất, chỉ cần dùng nhiều dữ liệu huấn luyện hơn và nhiều tài nguyên tính toán hơn là có thể cải thiện mô hình BERT mà không cần thay đổi kiến trúc mô hình. Thứ hai, việc bỏ hàm mất mát NSP không làm giảm hiệu năng trên các nhiệm vụ hạ nguồn nếu quy mô huấn luyện được mở rộng. Những phát hiện này gợi ý một hướng khám phá chung cho tiền huấn luyện: ta có thể tiếp tục cải thiện tiền huấn luyện bằng cách mở rộng quy mô trên những nhiệm vụ tiền huấn luyện đơn giản.

Cách thứ hai để cải thiện mô hình BERT là tăng số tham số của mô hình. Ví dụ, trong công trình của He và cộng sự [2021], một mô hình tương tự BERT với 1,5 tỉ tham số được xây dựng bằng cách tăng cả độ sâu mô hình lẫn kích thước ẩn. Tuy nhiên, việc mở rộng quy mô BERT và nhiều mô hình tiền huấn luyện khác đặt ra những thách thức mới trong huấn luyện, chẳng hạn huấn luyện các mô hình rất lớn thường trở nên không ổn định và khó hội tụ. Điều này làm bài toán phức tạp hơn và đòi hỏi cân nhắc cẩn thận nhiều khía cạnh, như kiến trúc mô hình, tính toán song song (*parallel computation*) và khởi tạo tham số (*parameter initialization*). Trong một ví dụ khác, Shoeybi và cộng sự [2019] đã huấn luyện thành công một mô hình tương tự BERT với 3,9 tỉ tham số, trong đó hàng trăm GPU được dùng để đáp ứng nhu cầu tính toán tăng lên.

Chú giải của người dịch: GPU và "khó hội tụ"

GPU (*graphics processing unit*, bộ xử lý đồ họa) vốn là chip dùng để vẽ hình trong trò chơi điện tử, nhưng rất giỏi làm hàng nghìn phép nhân cùng lúc nên trở thành "động cơ" chính để huấn luyện mạng nơ-ron. "Hội tụ" nghĩa là hàm mất mát giảm dần rồi ổn định ở một giá trị thấp, giống dãy số có giới hạn mà bạn học ở lớp 11. Mô hình quá lớn có thể "không ổn định": hàm mất mát đột ngột tăng vọt hoặc dao động mạnh thay vì giảm đều.

7.3.3 Các mô hình hiệu quả hơn

So với các mô hình đi trước, BERT là một mô hình tương đối lớn vào thời điểm nó được đề xuất. Kích thước mô hình tăng lên dẫn đến yêu cầu bộ nhớ lớn hơn và suy luận (*inference*) chậm hơn. Việc phát triển các mô hình BERT nhỏ hơn và nhanh hơn là một phần của thách thức rộng hơn về xây dựng các Transformer hiệu quả, vốn đã được thảo luận kỹ trong Chương 6. Tuy nhiên, việc đi sâu vào chủ đề này nằm ngoài phạm vi của mục này. Ở đây, ta chỉ tập trung vào một vài biến thể hiệu quả của BERT.

Có một số hướng nghiên cứu đặc biệt đáng quan tâm để phát triển các mô hình BERT hiệu quả. Thứ nhất, các công trình về chưng cất tri thức (*knowledge distillation*), chẳng hạn huấn luyện một mô hình học trò (*student model*) bằng đầu ra của một mô hình thầy (*teacher model*) đã được huấn luyện tốt, cho thấy có thể thu được những mô hình gọn hơn bằng cách chuyển giao tri thức từ các mô hình lớn hơn. Vì mô hình BERT là mạng nhiều tầng với một số loại tầng khác nhau, chưng cất tri thức có thể được áp dụng ở nhiều cấp độ biểu diễn khác nhau. Ví dụ, ngoài việc chưng cất tri thức từ các tầng đầu ra, ta cũng có thể đưa vào một hàm mất mát huấn luyện đo sự khác biệt giữa đầu ra của các tầng ẩn của mô hình thầy và mô hình học trò [Sun và cộng sự, 2020b; Jiao và cộng sự, 2020]. Thực tế, chưng cất tri thức đã là một trong những kỹ thuật được dùng rộng rãi nhất để học các mô hình tiền huấn luyện nhỏ.

Chú giải của người dịch: Chưng cất tri thức

Hãy hình dung một thầy giáo giỏi (mô hình lớn) và một học trò (mô hình nhỏ). Thay vì chỉ cho học trò xem đáp án đúng ("nhãn cứng"), thầy cho xem cả mức độ tự tin của mình với từng phương án, ví dụ "70% là A, 25% là B, 5% là C". Thông tin chi tiết này giúp học trò học nhanh và tốt hơn. Học trò được huấn luyện để đầu ra của mình gần với đầu ra của thầy; nhờ đó mô hình nhỏ đạt được phần lớn năng lực của mô hình lớn nhưng chạy nhanh hơn nhiều.

Thứ hai, các phương pháp nén mô hình (*model compression*) truyền thống có thể được áp dụng trực tiếp cho BERT. Một cách phổ biến là áp dụng các phương pháp tỉa (*pruning*) đa dụng cho các bộ mã hóa Transformer [Gale và cộng sự, 2019]. Việc này thường bao gồm loại bỏ toàn bộ một số tầng [Fan và cộng sự, 2019] hoặc một tỉ lệ phần trăm nhất định các tham số trong mạng [Sanh và cộng sự, 2020; Chen và cộng sự, 2020b]. Tỉa cũng áp dụng được cho các mô hình chú ý đa đầu (*multi-head attention*). Ví dụ, Michel và cộng sự [2019] chỉ ra rằng việc loại bỏ một số đầu chú ý không làm giảm đáng kể hiệu năng của mô hình BERT, mà lại tăng tốc suy luận của các mô hình này. Một cách khác để nén mô hình BERT là lượng tử hóa (*quantization*) [Shen và cộng sự, 2020b]. Bằng cách biểu diễn các tham số mô hình dưới dạng số có độ chính xác thấp (*low-precision*), ta có thể nén mô hình đáng kể. Mặc dù phương pháp này không dành riêng cho mô hình BERT, nó tỏ ra hiệu quả với các kiến trúc lớn dựa trên Transformer.

Chú giải của người dịch: Lượng tử hóa

Mỗi tham số thường được lưu bằng một số thực 32 bit (4 byte). Lượng tử hóa giống việc làm tròn số: thay vì lưu $0.73215\ldots$ với độ chính xác cao, ta chỉ lưu một số nguyên 8 bit (từ -128 đến 127) cùng một hệ số tỉ lệ chung. Ví dụ, nếu hệ số tỉ lệ là 0.01 thì 0.73215 được lưu thành số nguyên 73 (giá trị khôi phục là 0.73). Bộ nhớ giảm khoảng 4 lần (110 triệu tham số: từ khoảng 440 MB xuống khoảng 110 MB), đổi lại sai số làm tròn nhỏ.

Thứ ba, vì mô hình BERT là những mạng tương đối sâu và lớn, một hướng nghiên cứu khác sử dụng mạng động (*dynamic network*) để điều chỉnh các mô hình này cho suy luận hiệu quả. Một ý tưởng trong khuôn khổ này là chọn động các tầng dùng để xử lý một token; ví dụ, trong các mô hình thích ứng độ sâu (*depth-adaptive*), ta thoát ra ở một độ sâu tối ưu nào đó và do đó bỏ qua các tầng còn lại trong chồng

tầng [Xin và cộng sự, 2020; Zhou và cộng sự, 2020]. Tương tự, ta có thể phát triển các mô hình thích ứng độ dài (*length-adaptive*), trong đó độ dài của chuỗi đầu vào được điều chỉnh động. Ví dụ, ta có thể bỏ qua một số token trong chuỗi đầu vào để mô hình giảm khối lượng tính toán cho những token ít quan trọng, qua đó nâng cao hiệu quả tổng thể.

Thứ tư, ta cũng có thể chia sẻ tham số (*parameter sharing*) giữa các tầng để giảm kích thước mô hình BERT. Một cách đơn giản để làm điều này là chia sẻ tham số của cả một tầng Transformer cho toàn bộ chồng tầng [Dehghani và cộng sự, 2018; Lan và cộng sự, 2020]. Ngoài việc giảm số tham số, điều này cho phép tái sử dụng cùng một tầng trong mạng Transformer nhiều tầng, giúp tiết kiệm bộ nhớ khi chạy kiểm tra.

7.3.4 Các mô hình đa ngôn ngữ

Mô hình BERT ban đầu chủ yếu tập trung vào tiếng Anh. Ngay sau khi mô hình này được đề xuất, nó đã được mở rộng sang nhiều ngôn ngữ. Một cách đơn giản là phát triển một mô hình riêng cho mỗi ngôn ngữ. Một cách khác, đã trở nên phổ biến hơn trong các công trình gần đây về mô hình ngôn ngữ lớn (*large language model*), là huấn luyện trực tiếp các mô hình đa ngôn ngữ (*multilingual*) trên dữ liệu của tất cả các ngôn ngữ. Theo đó, các mô hình BERT đa ngôn ngữ (mBERT) đã được phát triển bằng cách huấn luyện trên văn bản của 104 ngôn ngữ[7]. Khác biệt chính so với các mô hình BERT đơn ngữ (*monolingual*) là mô hình mBERT dùng bộ từ vựng lớn hơn để bao phủ token của nhiều ngôn ngữ. Nhờ vậy, biểu diễn của các token thuộc các ngôn ngữ khác nhau được ánh xạ vào cùng một không gian, cho phép chia sẻ tri thức giữa các ngôn ngữ.

Một trong những ứng dụng quan trọng nhất của các mô hình tiền huấn luyện đa ngôn ngữ là học chuyển giao xuyên ngôn ngữ (*cross-lingual transfer learning*). Trong thiết lập xuyên ngôn ngữ zero-shot (*zero-shot cross-lingual*), một mô hình được tinh chỉnh cho một nhiệm vụ bằng dữ liệu của một ngôn ngữ nguồn (*source language*) rồi được áp dụng cho cùng nhiệm vụ đó ở một ngôn ngữ đích (*target language*) mà không cần bất kỳ dữ liệu huấn luyện nào của ngôn ngữ đích. Chẳng hạn, trong phân loại văn bản (*text classification*) xuyên ngôn ngữ, ta có thể tinh chỉnh một mô hình tiền huấn luyện đa ngôn ngữ trên các tài liệu tiếng Anh đã gán nhãn, rồi dùng trực tiếp mô hình đã tinh chỉnh để phân loại các tài liệu tiếng Trung.

Chú giải của người dịch: Chuyển giao xuyên ngôn ngữ và tiếng Việt

Điều này đặc biệt có ích cho tiếng Việt: dữ liệu gán nhãn tiếng Việt cho nhiều nhiệm vụ còn ít, trong khi tiếng Anh có rất nhiều. Ta có thể tinh chỉnh một mô hình đa ngôn ngữ (ví dụ mBERT, vốn có tiếng Việt trong số các ngôn ngữ huấn luyện) để phân loại cảm xúc bình luận tiếng Anh, rồi đem dùng ngay cho bình luận tiếng Việt. Điều này khả thi vì trong không gian biểu diễn chung, câu "I love this movie" và "Tôi rất thích bộ phim này" có xu hướng được biểu diễn bằng những vectơ gần nhau.

Một cải tiến cho các mô hình tiền huấn luyện đa ngôn ngữ như mBERT là đưa dữ liệu song ngữ (*bilingual data*) vào tiền huấn luyện. Thay vì chỉ huấn luyện trên dữ liệu đơn ngữ của nhiều ngôn ngữ, huấn luyện song ngữ mô hình hóa một cách tường minh mối quan hệ giữa các token trong hai ngôn ngữ. Mô hình thu được sẽ có khả năng chuyển giao xuyên ngôn ngữ, và do đó có thể dễ dàng thích ứng với các ngôn ngữ khác nhau. Lample và Conneau [2019] đề xuất một cách tiếp cận để tiền huấn luyện **mô hình ngôn ngữ xuyên ngôn ngữ** (*cross-lingual language models*) (**XLM**). Trong công trình của họ, một mô hình ngôn ngữ xuyên ngôn ngữ có thể được huấn luyện bằng mô hình hóa ngôn ngữ nhân quả (*causal language modeling*) hoặc mô hình hóa ngôn ngữ có che. Với tiền huấn luyện bằng mô hình

[7] https://github.com/google-research/bert/

hóa ngôn ngữ có che, mô hình được xem là một bộ mã hóa. Mục tiêu huấn luyện giống như BERT: ta cực đại hóa xác suất của một số token được chọn ngẫu nhiên, vốn hoặc bị che, hoặc bị thay bằng token ngẫu nhiên, hoặc được giữ nguyên trong đầu vào. Nếu xét dữ liệu song ngữ trong tiền huấn luyện, mỗi lần ta lấy mẫu một cặp câu đã được căn chỉnh (*aligned*) với nhau. Sau đó, hai câu được ghép lại thành một chuỗi duy nhất dùng để huấn luyện. Ví dụ, xét một cặp câu Anh–Trung

$$鲸鱼\ 是\ 哺乳\ 动物\ 。 \quad \leftrightarrow \quad \text{Whales are mammals .}$$

Ta có thể ghép chúng lại để được một chuỗi như sau

$$\text{[CLS]}\ 鲸鱼\ 是\ 哺乳\ 动物\ 。\ \text{[SEP] Whales are mammals . [SEP]}$$

Sau đó ta chọn một tỉ lệ phần trăm nhất định các token và thay chúng bằng [MASK].

$$\text{[CLS] [MASK]}\ 是\ \text{[MASK]}\ 动物\ 。\ \text{[SEP] Whales [MASK] [MASK] . [SEP]}$$

[8]Mục tiêu của tiền huấn luyện là cực đại hóa tích các xác suất của những token bị che khi biết chuỗi trên. Bằng cách huấn luyện như vậy, mô hình có thể học cách biểu diễn cả chuỗi tiếng Anh lẫn chuỗi tiếng Trung, đồng thời nắm bắt được sự tương ứng giữa các token trong hai ngôn ngữ. Ví dụ, dự đoán token tiếng Trung 鲸鱼 có thể cần thông tin từ token tiếng Anh *Whales*. Việc căn chỉnh biểu diễn của hai ngôn ngữ về bản chất biến mô hình thành một mô hình "dịch". Vì vậy, mục tiêu huấn luyện này còn được gọi là **mô hình hóa ngôn ngữ dịch** (*translation language modeling*). Hình 7.7 minh họa cách tiếp cận này.

Ghi chú Hình 7.7: (zh) là tiếng Trung, (en) là tiếng Anh; mô hình phải đoán lại 鲸鱼 (cá voi), 哺乳 (cho bú), *are* (là), *mammals* (động vật có vú).

Một lợi ích của các mô hình tiền huấn luyện đa ngôn ngữ là khả năng vốn có trong việc xử lý chuyển mã (*code-switching*). Trong NLP và ngôn ngữ học, chuyển mã là việc chuyển đổi qua lại giữa các ngôn ngữ trong một văn bản. Ví dụ, sau đây là một văn bản hỗn hợp chứa cả tiếng Trung và tiếng Anh:

$$周末\ 我们\ 打算\ 去\ 做\ \text{hiking},\ 你\ 想\ 一起\ 来\ 吗\ ?$$

(Cuối tuần chúng tôi định đi *hiking* (đi bộ đường dài), bạn có muốn đi cùng không?)

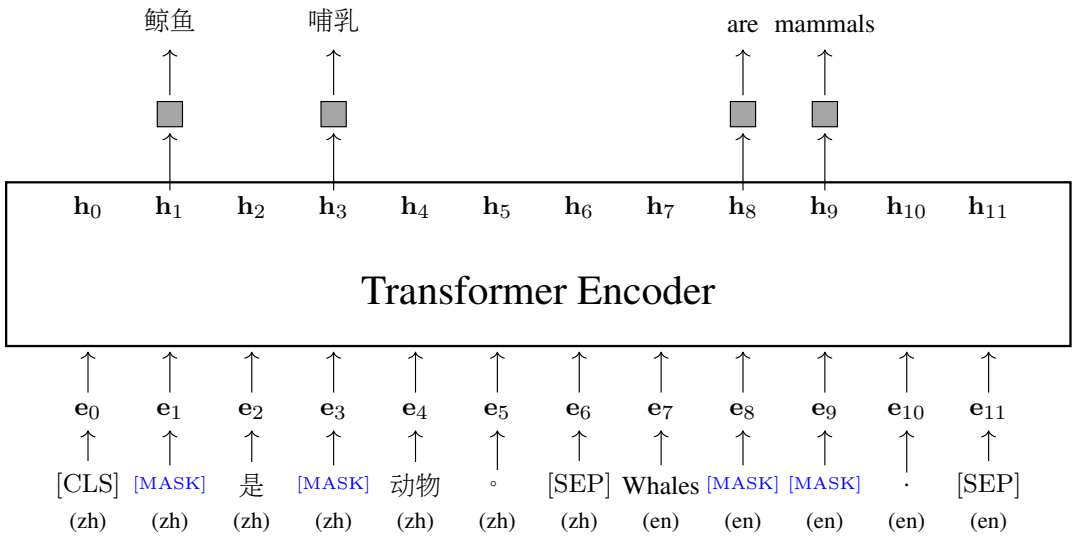
Với các mô hình tiền huấn luyện đa ngôn ngữ, ta không cần xác định một token là tiếng Trung hay tiếng Anh. Thay vào đó, mỗi token chỉ là một mục trong bộ từ vựng chung. Có thể hình dung điều này như việc tạo ra một ngôn ngữ "mới" bao gồm tất cả các ngôn ngữ mà ta muốn xử lý.

Chú giải của người dịch: Chuyển mã quanh ta

Chuyển mã rất quen thuộc với người Việt trẻ, chẳng hạn "Deadline tuần này căng quá, mình phải check lại slide trước khi meeting". Một mô hình đa ngôn ngữ coi "deadline", "check" hay "căng" đều chỉ là các mục trong cùng một bộ từ vựng, nên đọc được những câu như vậy mà không cần biết trước từ nào thuộc ngôn ngữ nào.

Kết quả của tiền huấn luyện đa ngôn ngữ chịu ảnh hưởng của nhiều yếu tố. Khi kiến trúc mô hình đã cố định, ta cần chỉ định nhiều cấu hình, chẳng hạn kích thước bộ từ vựng chung, số lượng (hoặc tỉ lệ) mẫu của mỗi ngôn ngữ và kích thước mô hình. Conneau và cộng sự [2020] chỉ ra một số vấn đề thú vị liên quan đến tiền huấn luyện đa ngôn ngữ quy mô lớn cho các mô hình kiểu XLM. Thứ nhất, khi số

[8]**[ND]** Câu ví dụ nghĩa là "Cá voi là động vật có vú." Trong câu tiếng Trung, 鲸鱼 = cá voi, 是 = là, 哺乳 = (cho) bú, 动物 = động vật.



Hình 7.7: Minh họa mô hình hóa ngôn ngữ dịch (TLM). Để dễ hiểu, chúng tôi trình bày một ví dụ đơn giản hóa trong đó mọi token được chọn đều bị che. Mô hình được huấn luyện để dự đoán các token bị che này. Vì chuỗi đầu vào chứa các token song song của hai ngôn ngữ, việc dự đoán một token bị che ở ngôn ngữ này cho phép mô hình chú ý tới ngữ cảnh ở ngôn ngữ kia, qua đó buộc mô hình phải căn chỉnh xuyên ngôn ngữ một cách tường minh. Trong cách xây dựng XLM nguyên bản của Lample và Conneau [2019], vectơ nhúng đầu vào (tức $\mathbf{e}_i$) là tổng của nhúng token, nhúng vị trí và một nhúng ngôn ngữ (*language embedding*) riêng. Điều này đòi hỏi mỗi token phải được gán một nhãn ngôn ngữ, cho phép mô hình phân biệt giữa các ngôn ngữ. Tuy nhiên, trong các kiến trúc tiền huấn luyện đa ngôn ngữ hiện đại (như XLM-R), việc chỉ rõ ngôn ngữ là không cần thiết, vì chỉ cần một bộ từ vựng chung là đủ. Hơn nữa, việc dựa vào nhúng ngôn ngữ khiến khó xử lý hiện tượng chuyển mã. Do đó, ở đây ta giả định rằng mọi biểu diễn token đều không phụ thuộc ngôn ngữ (*language-agnostic*).

ngôn ngữ được hỗ trợ tăng lên, cần một mô hình lớn hơn để xử lý các ngôn ngữ này. Thứ hai, một bộ từ vựng chung lớn hơn giúp mô hình hóa sự đa dạng tăng lên của các ngôn ngữ. Thứ ba, các ngôn ngữ ít tài nguyên (*low-resource languages*) được hưởng lợi đáng kể từ chuyển giao xuyên ngôn ngữ, đặc biệt khi kho ngữ liệu tiền huấn luyện có các ngôn ngữ nhiều tài nguyên (*high-resource languages*) tương đồng về mặt ngôn ngữ học. Tuy nhiên, một hiện tượng được gọi là **lời nguyền đa ngôn ngữ** (*the curse of multilinguality*) (hay giao thoa năng lực (*capacity interference*)) xuất hiện khi một mô hình có năng lực cố định được huấn luyện trên quá nhiều ngôn ngữ: việc thêm nhiều ngôn ngữ hơn cuối cùng sẽ buộc các ngôn ngữ phải cạnh tranh tham số, làm giảm hiệu năng trên các ngôn ngữ nhiều tài nguyên. Vì vậy, trong các hệ thống thực tế, có một sự đánh đổi giữa số ngôn ngữ được hỗ trợ và năng lực của mô hình.

Chú giải của người dịch: Lời nguyền đa ngôn ngữ

Hãy hình dung mô hình như một chiếc ba lô có dung tích cố định (số tham số cố định). Mỗi ngôn ngữ cần một chỗ để "cất" kiến thức. Khi chỉ có vài ngôn ngữ, chúng còn giúp nhau (ví dụ tiếng Lào học được nhiều từ tiếng Thái gần gũi). Nhưng khi nhồi quá nhiều ngôn ngữ vào cùng một ba lô, mỗi ngôn ngữ chỉ còn ít chỗ, và ngay cả những ngôn ngữ vốn mạnh như tiếng Anh cũng bị "chật" và làm kém đi.

7.4 Ứng dụng các mô hình BERT

Sau khi được tiền huấn luyện, một mô hình BERT có thể được dùng để giải các bài toán NLP. Nhưng các mô hình BERT chưa sẵn sàng ngay cho các nhiệm vụ hạ nguồn. Nói chung, cần tinh chỉnh thêm để thích ứng chúng. Bước đầu tiên, ta cần một bộ dự đoán (*predictor*) để khớp đầu ra của mô hình với nhiệm vụ cần giải. Gọi $\text{BERT}_{\hat{\theta}}(\cdot)$ là một mô hình BERT với các tham số tiền huấn luyện $\hat{\theta}$, và $\text{Predict}_{\omega}(\cdot)$ là một mạng dự đoán (*prediction network*) với tham số ω. Bằng cách gắn mạng dự đoán vào đầu ra của mô hình BERT, ta xây dựng được một mô hình để giải các nhiệm vụ hạ nguồn. Mô hình này có thể được biểu diễn là:

$$\mathbf{y} = \text{Predict}_{\omega}(\text{BERT}_{\hat{\theta}}(\mathbf{x})), \tag{7.21}$$

trong đó $\mathbf{x}$ là đầu vào và $\mathbf{y}$ là đầu ra phù hợp với bài toán. Ví dụ, trong các bài toán phân loại, mô hình đưa ra một phân phối xác suất (*probability distribution*) trên các nhãn.

Sau đó, ta thu thập một tập các mẫu có nhãn $\mathcal{D}$ và tinh chỉnh mô hình bằng

$$(\tilde{\omega}, \tilde{\theta}) = \arg\min_{\omega, \hat{\theta}^+} \sum_{(\mathbf{x}, \mathbf{y}_{\text{gold}}) \in \mathcal{D}} \text{Loss}(\mathbf{y}_{\omega, \hat{\theta}^+}, \mathbf{y}_{\text{gold}}), \tag{7.22}$$

trong đó $(\mathbf{x}, \mathbf{y}_{\text{gold}})$ biểu diễn một bộ gồm một đầu vào và đầu ra tương ứng của nó. Ký hiệu của phương trình này có vẻ hơi phức tạp, nhưng quá trình huấn luyện/tinh chỉnh là chuẩn. Ta tối ưu mô hình bằng cách cực tiểu hóa hàm mất mát trên dữ liệu huấn luyện. Kết quả là các tham số đã tối ưu $\tilde{\omega}$ và $\tilde{\theta}$. Quá trình tối ưu bắt đầu từ các tham số tiền huấn luyện $\hat{\theta}$. Ở đây ta dùng $\hat{\theta}^+$ để chỉ rằng các tham số được khởi tạo bằng $\hat{\theta}$, và dùng $\mathbf{y}_{\omega, \hat{\theta}^+}$ để ký hiệu đầu ra của mô hình được tính bằng các tham số ω và $\hat{\theta}^+$.

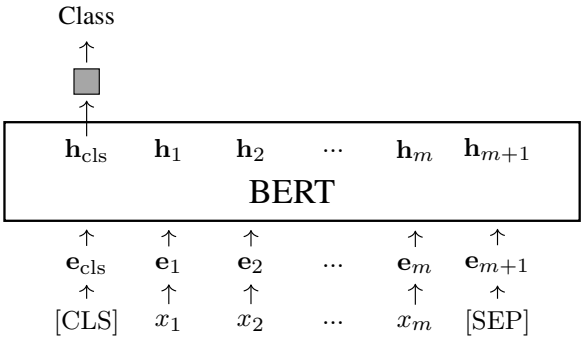
Chú giải của người dịch: Đọc các ký hiệu $\hat{\theta}$, $\hat{\theta}^+$, $\tilde{\theta}$

Có thể hiểu theo ba "thời điểm" của bộ tham số BERT: $\hat{\theta}$ ("mũ") là tham số *sau tiền huấn luyện*; $\hat{\theta}^+$ là tham số *đang được tinh chỉnh*, xuất phát từ $\hat{\theta}$; còn $\tilde{\theta}$ ("ngã") là tham số *sau tinh chỉnh*. Tương tự ω là tham số của mạng dự đoán nhỏ gắn thêm, $\tilde{\omega}$ là giá trị sau khi học xong. arg min đọc là "giá trị của biến làm cho biểu thức nhỏ nhất"; ví dụ $\arg\min_x (x-3)^2 = 3$. Như vậy, Phương trình (7.22) nói: tìm bộ tham số làm tổng điểm phạt trên toàn bộ dữ liệu có nhãn $\mathcal{D}$ là nhỏ nhất. Điểm khác so với huấn luyện từ đầu chỉ là ta *xuất phát* từ một mô

hình đã "biết" ngôn ngữ, nên chỉ cần ít dữ liệu có nhãn.

Với các tham số đã tinh chỉnh $\tilde{\omega}$ và $\tilde{\theta}$, ta có thể áp dụng mô hình $\text{Predict}_{\tilde{\omega}}(\text{BERT}_{\tilde{\theta}}(\cdot))$ cho dữ liệu mới của chính những nhiệm vụ mà mô hình đã được tinh chỉnh. Dạng của nhiệm vụ hạ nguồn quyết định định dạng đầu vào và đầu ra của mô hình, cũng như kiến trúc của mạng dự đoán. Dưới đây, ta liệt kê một số nhiệm vụ mà mô hình BERT rất phù hợp.

- **Phân loại** (văn bản đơn). Một trong những ứng dụng phổ biến nhất của mô hình BERT là phân loại văn bản. Trong nhiệm vụ này, mô hình BERT nhận một chuỗi token và mã hóa nó thành một chuỗi vectơ. Vectơ đầu ra đầu tiên $\mathbf{h}_{\text{cls}}$ (hoặc $\mathbf{h}_0$) thường được dùng làm biểu diễn của toàn bộ văn bản. Mạng dự đoán nhận $\mathbf{h}_{\text{cls}}$ làm đầu vào để sinh ra một phân phối trên các nhãn. Gọi $[\text{CLS}]\, x_1 x_2 \cdots x_m$ là chuỗi đầu vào. Xem Hình 7.8 để thấy minh họa phân loại văn bản dựa trên BERT. Nhiều bài toán NLP có thể được quy về các nhiệm vụ phân loại văn bản, và đã có một số



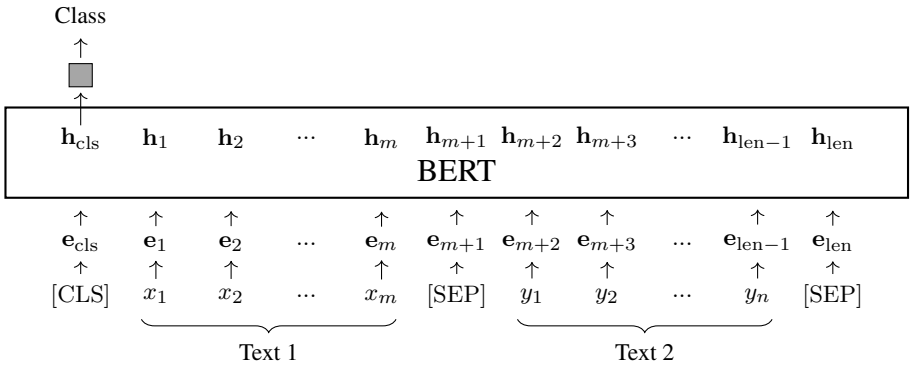
Hình 7.8: Phân loại dựa trên BERT cho một văn bản đơn. Ở đây ô màu xám biểu thị mạng dự đoán. (*Class*: lớp/nhãn dự đoán.)

bộ đánh giá chuẩn (*benchmark*) phân loại văn bản để đánh giá các mô hình tiền huấn luyện. Ví dụ, ta có thể phân loại văn bản theo tính đúng ngữ pháp (*grammaticality*) hoặc sắc thái cảm xúc (*sentiment*) [Socher và cộng sự, 2013; Warstadt và cộng sự, 2019]. Lưu ý rằng mạng dự đoán có thể là bất kỳ mô hình phân loại nào, chẳng hạn một mạng nơ-ron sâu hoặc một mô hình phân loại truyền thống hơn. Toàn bộ mô hình sau đó có thể được huấn luyện hoặc tinh chỉnh theo cách của một mô hình phân loại chuẩn. Ví dụ, mạng dự đoán có thể chỉ đơn giản là một tầng Softmax, và các tham số mô hình có thể được tối ưu bằng cách cực đại hóa xác suất của các nhãn đúng.

Chú giải của người dịch: Mạng dự đoán đơn giản nhất

Giả sử bài toán phân loại bình luận thành 3 nhãn {tích cực, trung tính, tiêu cực}. Mạng dự đoán có thể chỉ là: nhân $\mathbf{h}_{\text{cls}}$ (768 số) với một ma trận trọng số để được 3 điểm số, ví dụ $(2.0;\ 0.5;\ -1.0)$, rồi đưa qua Softmax để thành xác suất: $e^2 \approx 7.39$, $e^{0.5} \approx 1.65$, $e^{-1} \approx 0.37$, tổng ≈ 9.41, nên xác suất là khoảng $(0.79;\ 0.18;\ 0.04)$. Mô hình chọn "tích cực". Lớp gắn thêm này chỉ có $768 \times 3 + 3 = 2\,307$ tham số mới, rất nhỏ so với 110 triệu tham số của BERT.

- **Phân loại** (cặp văn bản). Phân loại cũng có thể được thực hiện trên một cặp văn bản. Giả sử ta có hai văn bản $x_1 \cdots x_m$ và $y_1 \cdots y_n$. Ta có thể nối hai văn bản này để tạo thành một chuỗi duy nhất có độ dài len. Sau đó, ta dự đoán một nhãn cho chuỗi văn bản kết hợp này dựa trên vectơ $\mathbf{h}_{\text{cls}}$, như trong Hình 7.9. Phân loại cặp văn bản bao gồm một số bài toán, như đánh giá tương đương ngữ nghĩa (*semantic equivalence*) (xác định hai văn bản có tương đương về ngữ nghĩa hay



Hình 7.9: Phân loại dựa trên BERT cho một cặp văn bản. Ở đây, len $= n + m + 2$. (*Text 1/Text 2*: văn bản 1/văn bản 2; *Class*: lớp.)

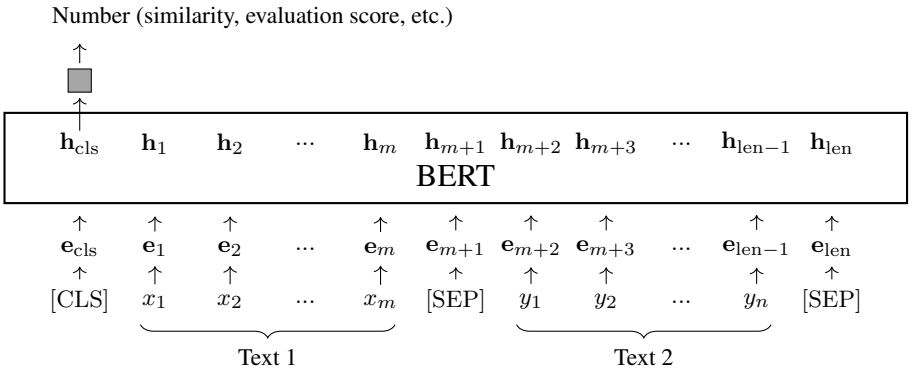
không) [Dolan và Brockett, 2005], đánh giá kéo theo văn bản (*text entailment*) (xác định một giả thuyết có thể được suy ra hoặc kéo theo một cách logic từ một tiền đề hay không) [Bentivogli và Giampiccolo, 2011; Williams và cộng sự, 2018], lập luận thường thức có căn cứ (*grounded commonsense inference*) (xác định một sự kiện có khả năng xảy ra hay không khi biết ngữ cảnh của nó) [Zellers và cộng sự, 2018], và lập luận hỏi–đáp (xác định một câu trả lời có tương ứng với một câu hỏi cho trước hay không).

Chú giải của người dịch: Ví dụ kéo theo văn bản

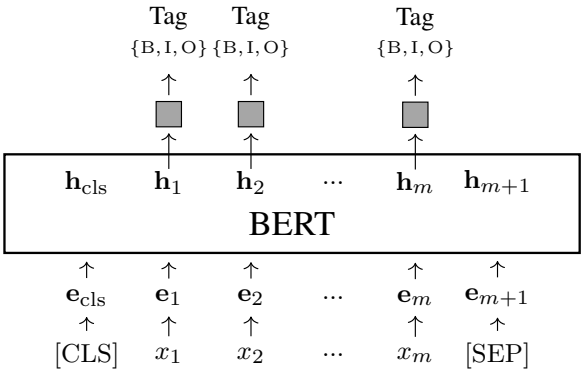
Tiền đề: "Lan đã đỗ đại học Bách khoa." Giả thuyết: "Lan đã thi đại học." ⇒ nhãn *kéo theo* (giả thuyết chắc chắn đúng nếu tiền đề đúng). Nếu giả thuyết là "Lan học ngành Công nghệ thông tin" thì nhãn là *trung lập* (không suy ra được), còn "Lan chưa từng thi đại học" thì nhãn là *mâu thuẫn*. Giống mệnh đề kéo theo $P \Rightarrow Q$ ở lớp 10, nhưng ở đây máy phải dựa vào hiểu biết ngôn ngữ và thường thức chứ không phải ký hiệu logic.

- **Hồi quy** (*Regression*). Thay vì sinh ra một phân phối nhãn, ta có thể cho mạng dự đoán xuất ra một điểm số thực. Ví dụ, bằng cách thêm một tầng Sigmoid vào mạng dự đoán, hệ thống có thể được dùng để tính độ tương đồng giữa hai câu cho trước. Kiến trúc giống như của các hệ thống phân loại dựa trên BERT, chỉ có tầng đầu ra là thay đổi, như trong Hình 7.10. Khi huấn luyện hoặc tinh chỉnh, ta có thể cực tiểu hóa hàm mất mát hồi quy của đầu ra mô hình như thường lệ.

- **Gán nhãn chuỗi** (*Sequence Labeling*). Gán nhãn chuỗi (*sequence labeling*) là một cách tiếp cận học máy áp dụng được cho nhiều bài toán NLP. Cách tiếp cận này gán một nhãn cho mỗi token trong chuỗi đầu vào, rồi từ chuỗi nhãn này có thể suy ra một số chú thích ngôn ngữ học. Một ví dụ về gán nhãn chuỗi trong NLP là gán nhãn từ loại (*part-of-speech (POS) tagging*): ta gán cho mỗi từ trong câu nhãn từ loại tương ứng. Một ví dụ khác là nhận dạng thực thể có tên (*named entity recognition*) (NER), trong đó ta gán cho mỗi từ một nhãn NER, và các thực thể có tên được xác định nhờ những nhãn này. Xem Hình 7.11 để thấy minh họa kiến trúc mô hình cho NER. Trong mô hình này, ta dùng lược đồ gán nhãn BIO (*BIO tagging scheme*) để biểu diễn các thực thể có tên. Ví dụ, B-ORG nghĩa là phần bắt đầu của một tổ chức, I-ORG nghĩa là từ nằm bên trong một tổ chức, và O nghĩa là từ không thuộc thực thể có tên nào. Mô hình NER này có thể xuất ra một phân phối trên tập nhãn tại mỗi vị trí, ký hiệu là $\mathbf{p}_i$. Việc huấn luyện hoặc tinh chỉnh mô hình có thể được thực hiện trên các phân phối $\{\mathbf{p}_1, \cdots, \mathbf{p}_m\}$ này. Ví dụ, giả sử $p_i(\text{tag}_i)$ là xác suất của



Hình 7.10: Hồi quy dựa trên BERT cho một cặp văn bản. (*Number (similarity, evaluation score, etc.)*: một con số (độ tương đồng, điểm đánh giá, v.v.).)



Hình 7.11: Gán nhãn chuỗi dựa trên BERT cho nhận dạng thực thể có tên. Ở đây $\{\text{B},\text{I},\text{O}\}$ là tập nhãn của NER. (*Tag*: nhãn.)

nhãn đúng tại vị trí i. Hàm mất mát huấn luyện có thể được định nghĩa là log-hợp lý âm (*negative log-likelihood*)

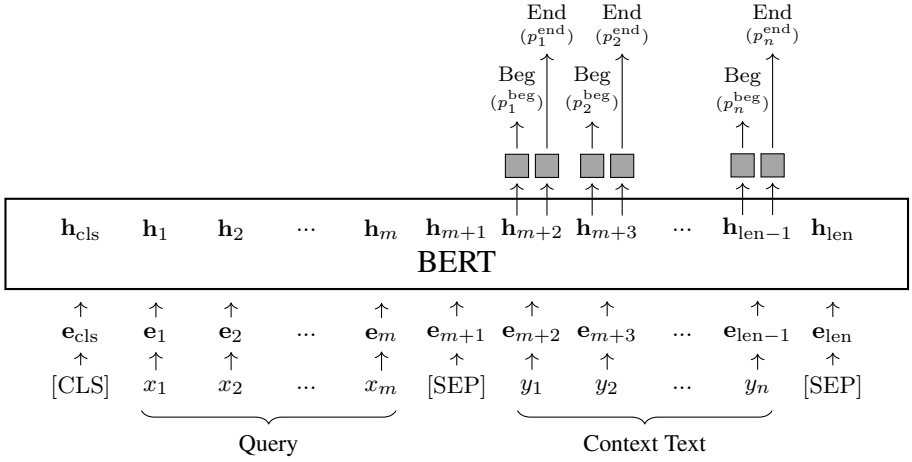
$$\text{Loss} = -\frac{1}{m}\sum_{i=1}^{m}\log p_i(\text{tag}_i). \tag{7.23}$$

Việc tìm chuỗi nhãn tốt nhất khi đã có một mô hình NER được huấn luyện là một bài toán được nghiên cứu kỹ trong NLP. Điều này thường được thực hiện bằng quy hoạch động (*dynamic programming*), phương pháp mà, trong bối cảnh tìm đường trên một lưới (*lattice*), có độ phức tạp tuyến tính [Huang, 2009].

Chú giải của người dịch: Ví dụ gán nhãn BIO

Câu "Nguyễn Văn An làm việc tại Đại học Bách khoa Hà Nội" (tách theo âm tiết) được gán nhãn: Nguyễn/B-PER Văn/I-PER An/I-PER làm/O việc/O tại/O Đại/B-ORG học/I-ORG Bách/I-ORG khoa/I-ORG Hà/I-ORG Nội/I-ORG (PER = người, ORG = tổ chức). Nhãn B đánh dấu chữ *bắt đầu* một thực thể, I là chữ *bên trong* thực thể đó, O là *ngoài* mọi thực thể. Đọc chuỗi nhãn, ta lấy lại được hai thực thể: một tên người và một tên tổ chức. Phương trình (7.23) đơn giản là trung bình "điểm phạt" $-\log p$ trên m vị trí; chia cho m để câu dài hay ngắn đều có thang đo như nhau.

- **Dự đoán đoạn** (*Span Prediction*). Một số nhiệm vụ NLP đòi hỏi dự đoán một đoạn (*span*) trong văn bản. Một ví dụ phổ biến là đọc hiểu (*reading comprehension*). Trong nhiệm vụ này, ta được cho một truy vấn $x_1 \cdots x_m$ và một văn bản ngữ cảnh $y_1 \cdots y_n$. Mục tiêu là xác định một đoạn liên tục trong $y_1 \cdots y_n$ trả lời tốt nhất truy vấn đó. Bài toán này có thể được đặt dưới dạng một nhiệm vụ tương tự gán nhãn chuỗi, trong đó ta dự đoán một nhãn cho mỗi y_j để chỉ ra điểm bắt đầu hoặc kết thúc của đoạn. Theo Seo và cộng sự [2017], ta thêm hai mạng bên trên đầu ra BERT cho y_j: một mạng sinh xác suất để y_j là điểm bắt đầu của đoạn (ký hiệu là p_j^{beg}), và một mạng sinh xác suất để y_j là điểm kết thúc của đoạn (ký hiệu là p_j^{end}). Kiến trúc mô hình thu được được trình bày trong Hình 7.12. Ta ghép truy vấn và văn bản ngữ cảnh lại để được chuỗi đầu vào. Các



Hình 7.12: Dự đoán đoạn dựa trên BERT cho đọc hiểu. (*Query*: truy vấn; *Context Text*: văn bản ngữ cảnh; *Beg/End*: bắt đầu/kết thúc.)

mạng dự đoán chỉ được áp dụng cho các đầu ra ứng với văn bản ngữ cảnh, sinh ra các xác suất p_j^{beg} và p_j^{end} tại mỗi vị trí. Hàm mất mát có thể được tính bằng cách cộng log-hợp lý âm của hai

mô hình trên toàn bộ văn bản ngữ cảnh.

$$\text{Loss} = -\frac{1}{n}\sum_{j=1}^{n}\left(\log p_j^{\text{beg}} + \log p_j^{\text{end}}\right). \tag{7.24}$$

Khi kiểm tra, ta tìm đoạn tốt nhất bằng

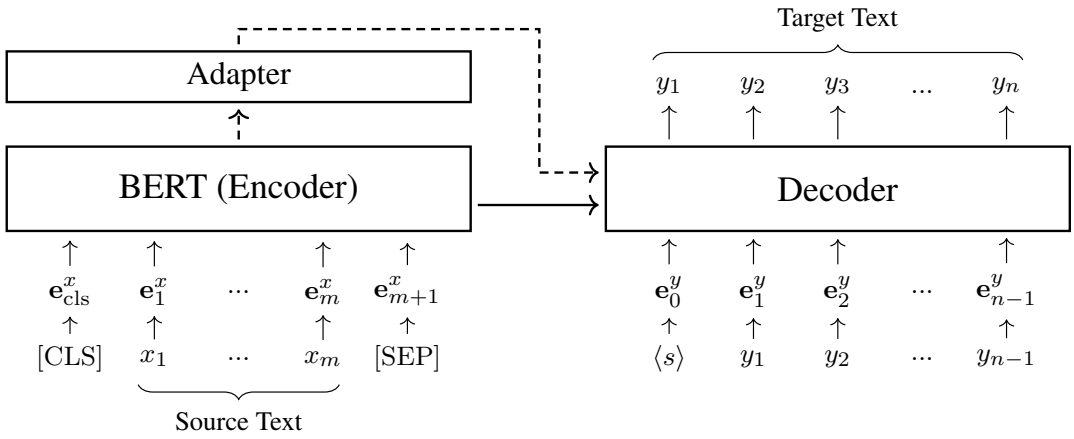
$$(\hat{j}_1, \hat{j}_2) = \underset{1\leq j_1\leq j_2\leq n}{\arg\max}\left(\log p_{j_1}^{\text{beg}} + \log p_{j_2}^{\text{end}}\right). \tag{7.25}$$

Chú giải của người dịch: Tìm đoạn trả lời

Ví dụ: truy vấn "Thủ đô của Việt Nam là gì?", văn bản ngữ cảnh "Hà Nội là thủ đô của Việt Nam .". Mô hình cho mỗi chữ trong ngữ cảnh hai xác suất: làm điểm *bắt đầu* và làm điểm *kết thúc* của câu trả lời. Nếu p^{beg} lớn nhất ở "Hà" và p^{end} lớn nhất ở "Nội" thì câu trả lời là đoạn "Hà Nội". Phương trình (7.25) thử mọi cặp (bắt đầu j_1, kết thúc j_2) với điều kiện $j_1 \leq j_2$ (điểm bắt đầu không thể nằm sau điểm kết thúc) và chọn cặp có tổng log-xác suất lớn nhất; vì log của tích bằng tổng các log, đó cũng là cặp có tích $p_{j_1}^{\text{beg}} \cdot p_{j_2}^{\text{end}}$ lớn nhất.

- **Mã hóa cho các mô hình mã hóa–giải mã** (*Encoding for Encoder-Decoder Models*). Mặc dù trọng tâm của mục này chủ yếu là các bài toán hiểu ngôn ngữ, cần lưu ý rằng mô hình BERT có thể được áp dụng cho nhiều loại nhiệm vụ NLP hơn. Thực tế, mô hình BERT có thể được dùng trong mọi tình huống mà ta cần mã hóa một đoạn văn bản. Một ứng dụng mà ta chưa đề cập là sinh văn bản (*text generation*), bao gồm nhiều nhiệm vụ như dịch máy (*machine translation*), tóm tắt (*summarization*), hỏi đáp (*question answering*) và sinh hội thoại (*dialogue generation*). Những nhiệm vụ này có thể được phát biểu dưới dạng các bài toán chuỗi-sang-chuỗi (*sequence-to-sequence*): ta dùng một bộ mã hóa để biểu diễn văn bản nguồn và một bộ giải mã (*decoder*) để sinh văn bản đích tương ứng. Một cách đơn giản để áp dụng mô hình BERT là xem chúng như các bộ mã hóa. Trước khi tinh chỉnh, ta có thể khởi tạo tham số của bộ mã hóa bằng tham số của một mô hình BERT đã tiền huấn luyện. Sau đó, mô hình mã hóa–giải mã có thể được tinh chỉnh trên các cặp văn bản như thường lệ. Hình 7.13 trình bày kiến trúc của một hệ thống dịch máy nơ-ron (*neural machine translation*) trong đó một mô hình BERT được áp dụng ở phía nguồn.

Tinh chỉnh mô hình BERT là một bài toán kỹ thuật phức tạp, chịu ảnh hưởng của nhiều yếu tố, như lượng dữ liệu tinh chỉnh, kích thước mô hình và bộ tối ưu (*optimizer*) dùng khi tinh chỉnh. Nói chung, ta mong muốn tinh chỉnh các mô hình này đủ để chúng hoạt động tốt trên các nhiệm vụ hạ nguồn. Tuy nhiên, việc tinh chỉnh mô hình BERT cho các nhiệm vụ cụ thể có thể dẫn đến quá khớp (*overfitting*), từ đó làm giảm khả năng tổng quát hóa (*generalize*) sang các nhiệm vụ khác. Ví dụ, giả sử ta có một mô hình BERT hoạt động tốt trên một nhiệm vụ cụ thể. Nếu sau đó ta tinh chỉnh nó cho các nhiệm vụ mới, hiệu năng của nó trên nhiệm vụ ban đầu có thể giảm xuống. Vấn đề này liên quan tới bài toán **quên thảm khốc** (*catastrophic forgetting*) trong huấn luyện liên tục (*continual training*), trong đó một mạng nơ-ron quên đi thông tin đã học trước đó khi được cập nhật trên các mẫu mới. Trong các ứng dụng thực tế, một cách phổ biến để giảm nhẹ hiện tượng quên thảm khốc là thêm một số dữ liệu cũ vào quá trình tinh chỉnh và huấn luyện mô hình với dữ liệu đa dạng hơn. Ngoài ra, ta có thể dùng các phương pháp chuyên biệt cho quên thảm khốc, như phát lại kinh nghiệm (*experience replay*) [Rolnick và cộng sự, 2019] và củng cố trọng số đàn hồi (*elastic weight consolidation*) [Kirkpatrick và cộng sự, 2017]. Bạn đọc quan tâm có thể tham khảo một số bài tổng quan để có thảo luận chi tiết hơn về vấn đề này trong học liên tục (*continual learning*) [Parisi và cộng sự, 2019; Wang và cộng sự, 2023a;f].



Hình 7.13: Dùng BERT làm bộ mã hóa trong một mô hình mã hóa–giải mã. Ở đây $x_1 \cdots x_m$ ký hiệu chuỗi nguồn, $y_1 \cdots y_n$ ký hiệu chuỗi đích, $\mathbf{e}_1^x \cdots \mathbf{e}_m^x$ ký hiệu chuỗi vectơ nhúng của $x_1 \cdots x_m$, và $\mathbf{e}_1^y \cdots \mathbf{e}_n^y$ ký hiệu chuỗi vectơ nhúng của $y_1 \cdots y_n$. Bộ tiếp hợp (*adapter*), vốn là tùy chọn, ánh xạ đầu ra của mô hình BERT sang dạng phù hợp hơn với bộ giải mã. (*Source Text*: văn bản nguồn; *Target Text*: văn bản đích; $\langle s \rangle$: token bắt đầu chuỗi.)

Chú giải của người dịch: Quên thảm khốc

Hãy tưởng tượng một học sinh ôn thi khối A rất kỹ, rồi trong ba tháng chỉ học tiếng Pháp; khi quay lại làm đề Toán, bạn ấy có thể quên nhiều. Mạng nơ-ron còn "quên" nặng hơn: khi các tham số bị điều chỉnh để phục vụ nhiệm vụ mới, chính những tham số đó vốn đang lưu kiến thức cho nhiệm vụ cũ. Cách chữa "trộn lại dữ liệu cũ" giống việc vừa học tiếng Pháp vừa thỉnh thoảng làm lại vài đề Toán.

7.5 Tóm tắt

Trong chương này, chúng ta đã giới thiệu khái niệm chung về tiền huấn luyện trong NLP. Cụ thể, chúng ta đã thảo luận về tiền huấn luyện tự giám sát và ứng dụng của nó cho các kiến trúc chỉ có bộ mã hóa, chỉ có bộ giải mã, và mã hóa–giải mã. Hơn nữa, chúng ta đã trình bày và so sánh nhiều nhiệm vụ tiền huấn luyện cho các kiến trúc này. Như một ví dụ, chúng ta đã dùng BERT để minh họa cách các mô hình chuỗi được tiền huấn luyện thông qua mô hình hóa ngôn ngữ có che và được áp dụng cho các nhiệm vụ hạ nguồn khác nhau.

Những năm gần đây đã chứng kiến những tiến bộ đáng kể trong NLP, dẫn dắt bởi việc sử dụng tiền huấn luyện tự giám sát ở quy mô lớn. Những bước tiến quan trọng đã đạt được trên nhiều nhiệm vụ, không chỉ trong NLP mà cả trong thị giác máy tính (*computer vision*) và các lĩnh vực khác của trí tuệ nhân tạo (*artificial intelligence*) (AI). Một ý tưởng đằng sau những tiến bộ này là một lượng lớn tri thức về thế giới có thể được học chỉ bằng cách huấn luyện các hệ thống AI này trên những lượng khổng lồ dữ liệu không gán nhãn (*unlabeled data*). Ví dụ, một mô hình ngôn ngữ có thể học được một số tri thức chung về một ngôn ngữ bằng cách lặp đi lặp lại việc dự đoán các từ bị che trong văn bản quy mô lớn. Kết quả là, mô hình ngôn ngữ đã tiền huấn luyện này có thể đóng vai trò một mô hình nền tảng (*foundation model*), có thể dễ dàng được thích ứng để giải các nhiệm vụ NLP hạ nguồn cụ thể. Sự chuyển dịch mô thức (*paradigm shift*) này trong NLP đã cho phép phát triển những hệ thống mạnh mẽ cho hiểu ngôn ngữ, sinh ngôn ngữ và lập luận (*reasoning*) [Manning, 2022]. Tuy nhiên, điều quan trọng là phải nhận

thức rằng chúng ta vẫn đang ở giai đoạn đầu của việc tạo ra những hệ thống thực sự thông minh, và còn một chặng đường dài phía trước. Dù vậy, tiền huấn luyện quy mô lớn đã mở ra cánh cửa tới những hệ thống thông minh mà các nhà nghiên cứu từ lâu đã mong muốn phát triển, mặc dù vẫn còn một số hướng nghiên cứu then chốt đang bỏ ngỏ để khám phá, chẳng hạn học hiệu quả từ dữ liệu hạn chế, và có được năng lực lập luận và lập kế hoạch (*planning*) phức tạp.

Lưu ý rằng chương này chủ yếu mang tính nhập môn và không thể bao quát mọi khía cạnh của tiền huấn luyện. Ví dụ, có nhiều phương pháp để tinh chỉnh một mô hình tiền huấn luyện, mang lại những cách khác nhau để thích ứng mô hình với các tình huống đa dạng. Hơn nữa, các mô hình ngôn ngữ lớn, vốn được xem là một trong những thành tựu quan trọng nhất của AI trong những năm gần đây, chưa được đề cập trong chương này. Chúng tôi để dành việc thảo luận các chủ đề này cho các chương tiếp theo.

Chương 8

Mô hình sinh

Một trong những tiến bộ quan trọng nhất của NLP trong những năm gần đây là sự phát triển của các mô hình ngôn ngữ lớn (*large language models, LLMs*). Bước đột phá này đã giúp tạo ra những hệ thống có thể hiểu và sinh ngôn ngữ tự nhiên ở mức gần với con người. Các hệ thống này thậm chí còn thể hiện được năng lực lập luận (*reasoning*) — một thách thức tồn tại từ lâu trong AI. Với những thành tựu đó, NLP đã đạt được tiến bộ đáng kể và bước vào một kỷ nguyên nghiên cứu mới, trong đó những bài toán ngày càng phức tạp đang được giải quyết, chẳng hạn như xây dựng các tác tử hội thoại (*conversational agents*) có thể tương tác trôi chảy với con người.

Khái niệm mô hình hóa ngôn ngữ (*language modeling*) bắt nguồn từ những thí nghiệm ban đầu của Shannon [1951]. Trong công trình của mình, Shannon đề xuất một mô hình ngôn ngữ để ước lượng mức độ có thể đoán trước của văn bản tiếng Anh — tức là chữ cái tiếp theo của một văn bản có thể được dự đoán tốt đến đâu khi đã biết N chữ cái đứng trước nó. Dù các thí nghiệm của Shannon mới chỉ là bước sơ khởi, mục tiêu và phương pháp cơ bản của mô hình hóa ngôn ngữ hầu như không thay đổi qua nhiều thập kỷ. Trong nhiều năm, đặc biệt là trước năm 2010, cách tiếp cận thống trị trong mô hình hóa ngôn ngữ là cách tiếp cận *n*-gram [Jurafsky và Martin, 2008]. Trong mô hình hóa ngôn ngữ *n*-gram, ta ước lượng xác suất của một từ khi biết $n-1$ từ đứng trước nó, và nhờ vậy xác suất của một chuỗi có thể được xấp xỉ bằng tích các xác suất *n*-gram. Các xác suất này thường được ước lượng từ tần suất tương đối (đã được làm trơn (*smoothed*)) của các *n*-gram trong văn bản. Dù đơn giản, cách tiếp cận này đã được dùng rộng rãi trong NLP. Chẳng hạn, thành công của các hệ thống nhận dạng tiếng nói (*speech recognition*) và dịch máy (*machine translation*) thống kê hiện đại phần lớn dựa vào việc sử dụng mô hình ngôn ngữ *n*-gram [Jelinek, 1998; Koehn, 2010].

Chú giải của người dịch: Mô hình *n*-gram qua một ví dụ

Một *n*-gram là một cụm gồm n từ liên tiếp. Với mô hình 2-gram (bigram), ta chỉ nhìn *một* từ đứng trước để đoán từ tiếp theo. Giả sử trong kho văn bản cụm "ăn cơm" xuất hiện 30 lần, còn từ "ăn" xuất hiện 100 lần; khi đó ước lượng $\Pr(\text{cơm} \mid \text{ăn}) \approx 30/100 = 0.3$. "Làm trơn" là kỹ thuật chia bớt một chút xác suất cho những cụm chưa từng gặp, để chúng không bị gán xác suất bằng 0 (vì một cụm chưa thấy trong dữ liệu chưa chắc là không thể xảy ra).

Việc áp dụng mạng nơ-ron vào mô hình hóa ngôn ngữ từ lâu đã là một hướng nghiên cứu sôi động, nhưng bước đột phá lớn chỉ đến cùng với sự tiến bộ của các kỹ thuật học sâu (*deep learning*). Một nghiên cứu được trích dẫn rộng rãi là công trình của Bengio và cộng sự [2003a], trong đó xác suất *n*-gram được mô hình hóa bằng một mạng truyền thẳng (*feed-forward network*) được huấn luyện đầu-cuối. Một thành quả quan trọng của mô hình này là các biểu diễn phân tán (*distributed representations*) của từ, được gọi là nhúng từ (*word embeddings*). Thay vì biểu diễn từ như các biến rời rạc, nhúng từ ánh xạ các từ thành

những vectơ số thực có số chiều thấp, nhờ đó có thể nắm bắt các quan hệ ngữ nghĩa giữa các từ và giữa các n-gram từ trong một không gian biểu diễn liên tục. Kết quả là các mô hình ngôn ngữ không còn bị đè nặng bởi lời nguyền số chiều (*curse of dimensionality*), mà có thể biểu diễn một số lượng n-gram tăng theo hàm mũ bằng một mô hình nơ-ron gọn nhẹ.

Chú giải của người dịch: Lời nguyền số chiều

Nếu bộ từ vựng có $|V| = 10\,000$ từ thì số 3-gram có thể có là $10\,000^3 = 10^{12}$ — quá nhiều để đếm và lưu trữ, và tuyệt đại đa số chúng chẳng bao giờ xuất hiện trong dữ liệu. Đó là "lời nguyền số chiều": số trường hợp tăng theo hàm mũ khi n tăng. Nhúng từ giải quyết chuyện này bằng cách cho các từ gần nghĩa (như "mèo" và "chó") có vectơ gần nhau, nên những gì mô hình học được về "con mèo chạy" có thể dùng lại cho "con chó chạy" dù cụm sau chưa từng gặp.

Ý tưởng học biểu diễn từ thông qua mô hình ngôn ngữ nơ-ron đã truyền cảm hứng cho các nghiên cứu tiếp theo về học biểu diễn (*representation learning*) trong NLP. Tuy nhiên, cách tiếp cận này không thu hút được nhiều sự quan tâm của cộng đồng NLP trong vài năm đầu sau khi được đề xuất. Dù vậy, vào khoảng năm 2012, đã có những tiến bộ đáng kể trong việc học nhúng từ từ văn bản quy mô lớn thông qua các nhiệm vụ dự đoán từ đơn giản. Nhiều phương pháp, trong đó có Word2Vec, đã được đề xuất để học hiệu quả các vectơ nhúng như vậy, và chúng sau đó được ứng dụng thành công trong nhiều hệ thống NLP [Mikolov và cộng sự, 2013a;c]. Nhờ những tiến bộ này, các nhà nghiên cứu bắt đầu khám phá việc học biểu diễn chuỗi bằng những mô hình ngôn ngữ mạnh hơn, chẳng hạn các mô hình dựa trên LSTM [Sutskever và cộng sự, 2014; Peters và cộng sự, 2018]. Tiếp đó, các tiến bộ và sự quan tâm đến biểu diễn chuỗi tăng lên nhanh chóng sau khi kiến trúc Transformer được đề xuất. Cùng với sự trỗi dậy của kiến trúc Transformer, khái niệm mô hình hóa ngôn ngữ được tổng quát hóa để bao hàm cả những mô hình học cách dự đoán từ theo những mô thức khác nhau. Nhiều mô hình mạnh dựa trên Transformer đã được tiền huấn luyện (*pre-trained*) bằng các nhiệm vụ dự đoán từ này và được áp dụng thành công cho nhiều nhiệm vụ hạ nguồn (*downstream tasks*) [Devlin và cộng sự, 2019].

Quả thật, việc huấn luyện mô hình ngôn ngữ trên dữ liệu quy mô lớn đã đưa nghiên cứu NLP đến một thời kỳ đầy hứng khởi. Trong khi mô hình hóa ngôn ngữ từ lâu được xem là một kỹ thuật nền tảng, không có liên hệ trực tiếp với những mục tiêu rộng lớn hơn của trí tuệ nhân tạo, thì nó lại dẫn đến sự xuất hiện của những hệ thống thông minh có thể thu nhận tri thức tổng quát thông qua việc dự đoán từ lặp đi lặp lại. Các nghiên cứu gần đây cho thấy một LLM duy nhất, được huấn luyện tốt, có thể thực hiện rất nhiều loại nhiệm vụ và tổng quát hóa sang nhiệm vụ mới chỉ với mức điều chỉnh tối thiểu [Bubeck và cộng sự, 2023]. Đây là một bước tiến tới những dạng trí tuệ nhân tạo cao cấp hơn, và nó thúc đẩy việc khám phá sâu hơn nhằm phát triển những mô hình ngôn ngữ mạnh hơn như một lớp mô hình nền tảng (*foundation models*).

Trong chương này, chúng ta bàn về các khái niệm cơ bản của LLM sinh. Để đơn giản, ta dùng từ LLM để chỉ các mô hình sinh (*generative models*) như GPT, mặc dù theo nghĩa rộng thuật ngữ này cũng có thể bao gồm các mô hình không sinh như BERT. Ta bắt đầu bằng phần giới thiệu chung về LLM, bao gồm các bước then chốt để xây dựng những mô hình như vậy. Sau đó, ta bàn về hai thách thức then chốt khi *mở rộng quy mô* LLM: LLM được huấn luyện ở quy mô lớn như thế nào, và chúng có thể được cải tiến ra sao để xử lý các đầu vào có ngữ cảnh dài. Cuối cùng, chương khép lại bằng một phần tóm tắt ngắn.

8.1 Giới thiệu ngắn gọn về LLM

Trong mục này, chúng tôi giới thiệu các khái niệm cơ bản về LLM cần thiết cho phần còn lại của chương này và các chương sau. Chúng tôi sẽ dùng các thuật ngữ *từ* và *token* thay thế cho nhau. Cả hai đều chỉ các đơn vị cơ bản dùng trong mô hình hóa ngôn ngữ, dù nghĩa gốc của chúng hơi khác nhau.

Trước khi trình bày chi tiết kỹ thuật, ta hãy xem các mô hình ngôn ngữ hoạt động như thế nào. Mục tiêu cốt lõi của mô hình hóa ngôn ngữ là ước lượng xác suất của một chuỗi token. Gọi $\{x_0, x_1, \cdots, x_m\}$ là một chuỗi token, trong đó x_0 là ký hiệu bắt đầu $\langle s \rangle$ (hoặc $\langle \text{SOS} \rangle$)[1]. Xác suất của chuỗi này có thể được phân tích bằng quy tắc chuỗi (*chain rule*)

$$\begin{aligned}\Pr(x_0, \cdots, x_m) &= \Pr(x_0) \cdot \Pr(x_1|x_0) \cdot \Pr(x_2|x_0, x_1) \cdots \Pr(x_m|x_0, \cdots, x_{m-1}) \\ &= \prod_{i=0}^{m} \Pr(x_i|x_0, \cdots, x_{i-1}),\end{aligned} \tag{8.1}$$

hoặc dưới dạng logarit

$$\log \Pr(x_0, \cdots, x_m) = \sum_{i=0}^{m} \log \Pr(x_i|x_0, \cdots, x_{i-1}). \tag{8.2}$$

Ở đây, $\Pr(x_i|x_0, \cdots, x_{i-1})$ là xác suất của token x_i khi biết mọi token đứng trước nó $\{x_0, \cdots, x_{i-1}\}$[2]. Trong kỷ nguyên học sâu, một cách tiếp cận điển hình để mô hình hóa ngôn ngữ là ước lượng các xác suất này bằng mạng nơ-ron sâu. Các mạng nơ-ron được huấn luyện cho nhiệm vụ này nhận vào một chuỗi ngữ cảnh $x_0, \cdots, x_{i-1}$ và xuất ra một phân phối xác suất (*probability distribution*) trên bộ từ vựng (*vocabulary*) V (ký hiệu là $\Pr(\cdot|x_0, \cdots, x_{i-1})$). Xác suất $\Pr(x_i|x_0, \cdots, x_{i-1})$ chính là phần tử ứng với token x_i trong phân phối này.

Chú giải của người dịch: Quy tắc chuỗi và xác suất có điều kiện

Ký hiệu $\Pr(b|a)$ đọc là "xác suất của *b khi biết a*" (xác suất có điều kiện (*conditional probability*)). Quy tắc chuỗi nói rằng xác suất để cả chuỗi xảy ra bằng tích các xác suất "từng bước một". Ví dụ với câu "tôi đi học":

$$\Pr(\text{tôi đi học}) = \Pr(\text{tôi}) \cdot \Pr(\text{đi} \mid \text{tôi}) \cdot \Pr(\text{học} \mid \text{tôi đi}).$$

Nếu ba xác suất lần lượt là 0.1, 0.2, 0.5 thì xác suất cả câu là $0.1 \cdot 0.2 \cdot 0.5 = 0.01$. Lấy logarit biến tích thành tổng (vì $\log(ab) = \log a + \log b$, như đã học ở lớp 11): $\log 0.01 = \log 0.1 + \log 0.2 + \log 0.5$. Dùng tổng các logarit giúp tránh việc nhân rất nhiều số nhỏ với nhau đến mức máy tính làm tròn thành 0.

Khi áp dụng một mô hình ngôn ngữ đã huấn luyện, một nhiệm vụ thường gặp là tìm token tiếp theo có khả năng cao nhất khi biết ngữ cảnh của nó. Điều này có thể được viết thành

$$\hat{x}_i = \arg\max_{x_i \in V} \Pr(x_i|x_0, \cdots, x_{i-1}). \tag{8.3}$$

Ta có thể thực hiện việc dự đoán token này lặp đi lặp lại để sinh văn bản một cách tuần tự: ở mỗi bước, ta dự đoán token có khả năng cao nhất $\hat{x}_i$, rồi nối nó vào ngữ cảnh để dự đoán token tiếp theo

[1] Trong một số mô hình như BERT, một token đặc biệt như [CLS] được thêm vào đầu chuỗi, dù nó phục vụ một mục đích khác với ký hiệu bắt đầu trong các mô hình tự hồi quy.

[2] Ta quy ước rằng khi $i = 0$, $\Pr(x_i|x_0, \cdots, x_{i-1}) = \Pr(x_0) = 1$. Do đó, $\Pr(x_0, \cdots, x_m) = \Pr(x_0) \Pr(x_1, \cdots, x_m|x_0) = \Pr(x_1, \cdots, x_m|x_0)$.

$\hat{x}_{i+1}$. Điều này tạo thành một quá trình sinh tự hồi quy (*autoregressive*), từ trái sang phải, hiện thực hóa Phương trình (8.1) và (8.2). Để minh họa, hãy xét việc sinh ba token khi biết tiền tố "$\langle s\rangle\ a$", như trong Bảng 8.1. Tiếp theo, chúng ta sẽ bàn về cách LLM được xây dựng, huấn luyện và áp dụng trong thực tế.

Chú giải của người dịch: Ký hiệu arg max

arg $\max_{x\in V} f(x)$ nghĩa là "giá trị x trong tập V làm cho $f(x)$ lớn nhất" (khác với max, là chính giá trị lớn nhất của f). Ví dụ, nếu mô hình cho $\Pr(\text{học}|\ldots) = 0.5$, $\Pr(\text{chơi}|\ldots) = 0.3$, $\Pr(\text{ngủ}|\ldots) = 0.2$ thì arg max là từ "học", còn max là số 0.5. Chọn token theo cách này ở mỗi bước gọi là tìm kiếm tham lam (*greedy search*).

Ngữ cảnh	Dự đoán	Quy tắc quyết định	Xác suất chuỗi
$\langle s\rangle\ a$	b	$\arg\max_{x_2\in V} \Pr(x_2\|\langle s\rangle\ a)$	$\Pr(\langle s\rangle)\cdot\Pr(a\|\langle s\rangle)\cdot \Pr(b\|\langle s\rangle\ a)$
$\langle s\rangle\ a\ b$	c	$\arg\max_{x_3\in V} \Pr(x_3\|\langle s\rangle\ a\ b)$	$\Pr(\langle s\rangle)\cdot\Pr(a\|\langle s\rangle)\cdot\Pr(b\|\langle s\rangle\ a)\cdot$ $\Pr(c\|\langle s\rangle\ a\ b)$
$\langle s\rangle\ a\ b\ c$	d	$\arg\max_{x_4\in V} \Pr(x_4\|\langle s\rangle\ a\ b\ c)$	$\Pr(\langle s\rangle)\cdot\Pr(a\|\langle s\rangle)\cdot\Pr(b\|\langle s\rangle\ a)\cdot$ $\Pr(c\|\langle s\rangle\ a\ b)\cdot \Pr(d\|\langle s\rangle\ a\ b\ c)$

Bảng 8.1: Minh họa việc sinh ba token $b\ c\ d$ khi biết tiền tố $\langle s\rangle\ a$ bằng một mô hình ngôn ngữ. Ở mỗi bước, mô hình chọn một token x_i từ V sao cho $\Pr(x_i|x_0,\cdots,x_{i-1})$ đạt cực đại. Token này sau đó được nối vào cuối chuỗi ngữ cảnh. Ở bước tiếp theo, ta lặp lại quá trình tương tự, nhưng dựa trên ngữ cảnh mới.

8.1.1 Transformer chỉ có bộ giải mã

Theo cách làm tiêu chuẩn, đầu vào của một mô hình ngôn ngữ là một chuỗi token (ký hiệu là $\{x_0,\cdots,x_{m-1}\}$). Ở mỗi bước, một token mới được sinh ra và nối vào chuỗi để phục vụ lần dự đoán tiếp theo. Để làm vậy, mô hình ngôn ngữ xuất ra một phân phối xác suất $\Pr(\cdot|x_0,\cdots,x_{i-1})$ tại mỗi vị trí i, và token tiếp theo được sinh ra theo phân phối này (ví dụ bằng lấy mẫu (*sampling*) hoặc giải mã tham lam (*greedy decoding*)). Mô hình được huấn luyện bằng cách cực đại hóa log-hợp lý (*log-likelihood*) $\sum_{i=1}^{m}\log\Pr(x_i|x_0,\cdots,x_{i-1})$[3].

Ở đây, ta tập trung vào kiến trúc Transformer chỉ có bộ giải mã (*decoder-only*), vì đây là một trong những kiến trúc được dùng rộng rãi nhất cho LLM. Chuỗi token đầu vào được biểu diễn bằng một chuỗi các vectơ d_e chiều $\{\mathbf{e}_0,\cdots,\mathbf{e}_{m-1}\}$. Mỗi vectơ $\mathbf{e}_i$ là tổng của vectơ nhúng token (*token embedding*) của x_i và vectơ nhúng vị trí (*positional embedding*) của vị trí i. Phần lõi của mô hình gồm một chồng các khối (hay tầng) Transformer. Mỗi khối Transformer gồm hai tầng con: một tầng con tự chú ý (*self-attention*) và một tầng con FFN. Các tầng con này có thể được định nghĩa theo kiến trúc *post-norm*

$$\text{output} \;=\; \text{LNorm}(F(\text{input}) + \text{input}), \tag{8.4}$$

hoặc kiến trúc *pre-norm*

$$\text{output} \;=\; F(\text{LNorm}(\text{input})) + \text{input}, \tag{8.5}$$

trong đó input và output ký hiệu đầu vào và đầu ra, cả hai đều là ma trận $m\times d$. Hàng thứ i của input và output có thể được hiểu là biểu diễn theo ngữ cảnh của token thứ i trong chuỗi.

$F(\cdot)$ là hàm lõi của một tầng con. Với tầng con FFN, $F(\cdot)$ là một FFN hai tầng. Với tầng con tự chú ý, $F(\cdot)$ là một hàm tự chú ý đa đầu (*multi-head self-attention*). Nói chung, tự chú ý được biểu diễn

[3]Lưu ý rằng $\sum_{i=1}^{m}\log\Pr(x_i|x_0,\cdots,x_{i-1}) = \sum_{i=0}^{m}\log\Pr(x_i|x_0,\cdots,x_{i-1})$ vì $\log\Pr(x_0) = 0$.

dưới dạng chú ý QKV

$$\text{Att}_{\text{qkv}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \;=\; \text{Softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{Mask}\Big)\mathbf{V}, \tag{8.6}$$

trong đó $\mathbf{Q}$, $\mathbf{K}$ và $\mathbf{V} \in \mathbb{R}^{m\times d}$ lần lượt là các truy vấn (*queries*), khóa (*keys*) và giá trị (*values*). Điều quan trọng cần lưu ý là khi dự đoán một token, chỉ những token đứng trước mới được xét đến. Vì vậy, một biến che (mặt nạ) $\mathbf{Mask} \in \mathbb{R}^{m\times m}$ được đưa vào cơ chế tự chú ý để đạt được điều này. Phần tử (i, k) của $\mathbf{Mask}$ có giá trị 0 nếu $i \geq k$, và giá trị $-\infty$ trong trường hợp ngược lại.

Chú giải của người dịch: Mặt nạ trong tự chú ý

Hàm Softmax biến một dãy số $s_1, \ldots, s_n$ thành các trọng số $e^{s_k} / \sum_j e^{s_j}$, đều dương và có tổng bằng 1. Vì $e^{-\infty} = 0$, cộng $-\infty$ vào một vị trí sẽ khiến trọng số chú ý ở vị trí đó bằng đúng 0. Với $m = 3$, ma trận mặt nạ là

$$\mathbf{Mask} = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}:$$

hàng 1 (token đầu tiên) chỉ nhìn được chính nó, hàng 2 nhìn được token 1 và 2, hàng 3 nhìn được cả ba. Nhờ vậy mô hình không thể "nhìn trộm" các token phía sau — những token mà lúc sinh văn bản thật ta chưa hề có.

Cho một biểu diễn $\mathbf{H} \in \mathbb{R}^{m\times d}$, hàm tự chú ý đa đầu có thể được định nghĩa là

$$F(\mathbf{H}) \;=\; \text{Merge}(\text{head}_1, \cdots, \text{head}_\tau)\mathbf{W}^{\text{head}}, \tag{8.7}$$

trong đó $\text{Merge}(\cdot)$ biểu thị phép ghép nối các đầu vào của nó, và $\mathbf{W}^{\text{head}} \in \mathbb{R}^{d\times d}$ là một ma trận tham số. head_j là đầu ra của chú ý QKV trên một không gian con của biểu diễn:

$$\text{head}_j \;=\; \text{Att}_{\text{qkv}}(\mathbf{Q}^{[j]}, \mathbf{K}^{[j]}, \mathbf{V}^{[j]}). \tag{8.8}$$

Ở đây, $\mathbf{Q}^{[j]}$, $\mathbf{K}^{[j]}$ và $\mathbf{V}^{[j]}$ là các truy vấn, khóa và giá trị được chiếu lên không gian con thứ j thông qua các phép biến đổi tuyến tính

$$\mathbf{Q}^{[j]} = \mathbf{H}\mathbf{W}_j^q \tag{8.9}$$

$$\mathbf{K}^{[j]} = \mathbf{H}\mathbf{W}_j^k \tag{8.10}$$

$$\mathbf{V}^{[j]} = \mathbf{H}\mathbf{W}_j^v, \tag{8.11}$$

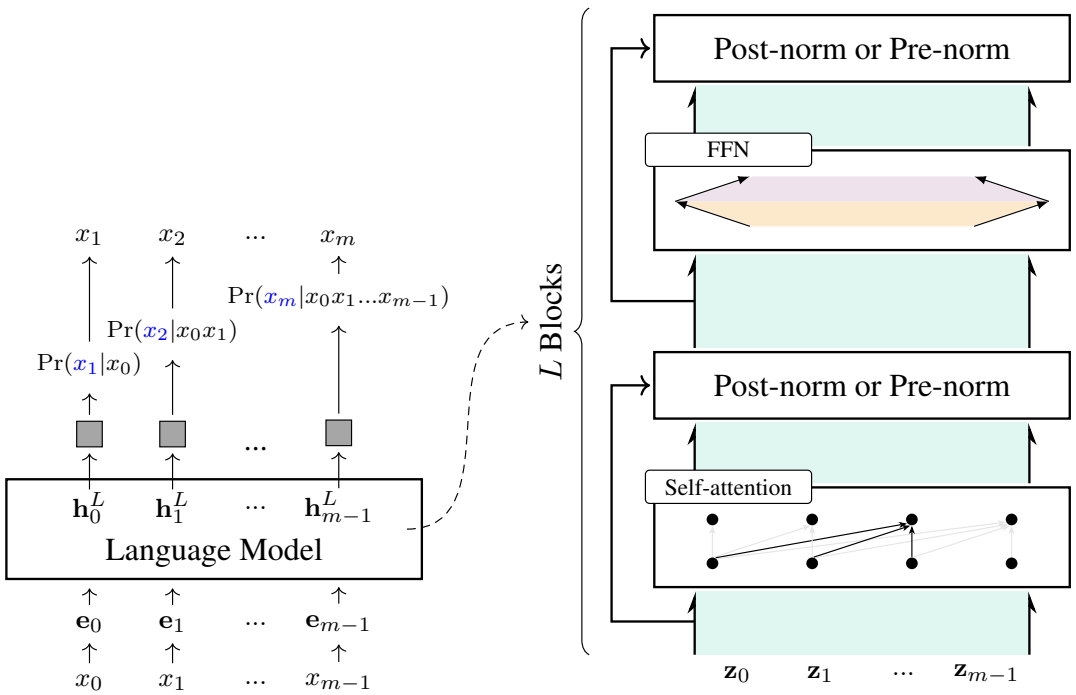
trong đó $\mathbf{W}_j^q$, $\mathbf{W}_j^k$ và $\mathbf{W}_j^v \in \mathbb{R}^{d\times \frac{d}{\tau}}$ là các ma trận tham số của các phép biến đổi.

(Nhãn trong hình: *Language Model* = mô hình ngôn ngữ; *L Blocks* = L khối; *Post-norm or Pre-norm* = chuẩn hóa sau hoặc chuẩn hóa trước; *Self-attention* = tự chú ý.)

Giả sử ta có L khối Transformer. Một tầng Softmax được xây dựng phía trên đầu ra của khối cuối cùng. Tầng Softmax xuất ra một chuỗi gồm m phân phối trên bộ từ vựng, như sau

$$\begin{bmatrix} \Pr(\cdot|x_0) \\ \Pr(\cdot|x_0, x_1) \\ \vdots \\ \Pr(\cdot|x_0, \cdots, x_{m-1}) \end{bmatrix} \;=\; \text{Softmax}(\mathbf{H}^L\mathbf{W}^o), \tag{8.12}$$

trong đó $\mathbf{H}^L$ là đầu ra của khối Transformer cuối cùng, và $\mathbf{W}^o \in \mathbb{R}^{d\times|V|}$ là ma trận tham số.



Hình 8.1: Kiến trúc bộ giải mã Transformer dùng cho mô hình hóa ngôn ngữ. Thành phần trung tâm là L khối Transformer xếp chồng, mỗi khối gồm một tầng con tự chú ý và một tầng con FFN. Để ngăn mô hình truy cập ngữ cảnh bên phải, một biến che được đưa vào cơ chế tự chú ý. Tầng đầu ra dùng hàm Softmax để sinh một phân phối xác suất cho token tiếp theo, khi biết chuỗi các token đứng trước. Trong quá trình lập luận, mô hình lấy token vừa dự đoán để dự đoán token tiếp theo, lặp lại quá trình này cho đến khi đạt tới cuối chuỗi. $\{\mathbf{z}_0, \cdots, \mathbf{z}_{m-1}\}$ ký hiệu các đầu vào của một khối Transformer, và $\{\mathbf{h}_0^L, \cdots, \mathbf{h}_{m-1}^L\}$ ký hiệu các đầu ra của khối Transformer cuối cùng.

Chú giải của người dịch: Kích thước các ma trận trong Phương trình (8.12)

$\mathbf{H}^L$ có m hàng (mỗi hàng ứng với một vị trí) và d cột; $\mathbf{W}^o$ có d hàng và $|V|$ cột ($|V|$ là số token trong bộ từ vựng). Tích $\mathbf{H}^L\mathbf{W}^o$ vì thế có kích thước $m \times |V|$: mỗi hàng là một dãy $|V|$ "điểm số", mỗi điểm cho một token có thể xuất hiện tiếp theo. Softmax áp dụng cho từng hàng biến các điểm số này thành xác suất có tổng bằng 1. Chẳng hạn với $|V| = 50\,000$, mỗi vị trí cho ra một bảng 50 000 xác suất, và token tiếp theo được chọn dựa trên bảng này.

Hình 8.1 cho thấy kiến trúc Transformer dùng cho mô hình hóa ngôn ngữ. Suy luận với mô hình này tuân theo một quá trình tự hồi quy. Mỗi lần, mô hình ngôn ngữ nhận tiền tố $x_0, \cdots, x_{i-1}$ làm đầu vào và dự đoán một token x_i làm cực đại xác suất $\Pr(x_i|x_0, \cdots, x_{i-1})$. Điều quan trọng cần lưu ý là, dù khác nhau ở các chi tiết hiện thực, nhiều LLM có chung kiến trúc được mô tả ở trên. Các mô hình này được gọi là *lớn* vì cả độ sâu lẫn độ rộng của chúng đều đáng kể. Bảng 8.2 cho thấy kích thước mô hình của một vài LLM, cùng với cấu hình mô hình của chúng.

LLM	Số tham số	Độ sâu L	Độ rộng d	Số đầu (Q/KV)
GPT-1 [Radford và cộng sự, 2018]	0.117B	12	768	12/12
GPT-2 [Radford và cộng sự, 2019]	1.5B	48	1,600	25/25
GPT-3 [Brown và cộng sự, 2020]	175B	96	12,288	96/96
Llama2 [Touvron và cộng sự, 2023b]	7B	32	4,096	32/32
	13B	40	5,120	40/40
	70B	80	8,192	64/64
Llama3/3.1 [Dubey và cộng sự, 2024]	8B	32	4,096	32/8
	70B	80	8,192	64/8
	405B	126	16,384	128/8
Gemma2 [Team và cộng sự, 2024]	2B	26	2,304	8/4
	9B	42	3,584	16/8
	37B	46	4,608	32/16
Qwen2.5 [Yang và cộng sự, 2024]	0.5B	24	896	14/2
	7B	28	3,584	28/4
	72B	80	8,192	64/8
DeepSeek-V3 [Liu và cộng sự, 2024a]	671B	61	7,168	128/128
Falcon [Penedo và cộng sự, 2023]	7B	32	4,544	71/71
	40B	60	8,192	128/128
	180B	80	14,848	232/232
Mistral [Jiang và cộng sự, 2023a]	7B	32	4,096	32/32

Bảng 8.2: So sánh một số LLM về kích thước mô hình, độ sâu mô hình, độ rộng mô hình và số đầu chú ý (a/b nghĩa là có a đầu cho truy vấn và b đầu dùng chung cho cả khóa và giá trị).

Chú giải của người dịch: Đọc Bảng 8.2

Chữ "B" là viết tắt của *billion* (tỷ): 175B nghĩa là 175 tỷ tham số. Dấu phẩy trong các số như 12,288 là dấu phân cách hàng nghìn theo kiểu tiếng Anh (tức 12 288). "Độ sâu" L là số khối Transformer xếp chồng, "độ rộng" d là số chiều của vectơ biểu diễn mỗi token. Khi số đầu khóa/giá trị (KV) nhỏ hơn số đầu truy vấn (như

64/8), nhiều đầu truy vấn dùng chung một bộ khóa–giá trị để tiết kiệm bộ nhớ; kỹ thuật này sẽ được bàn ở các phần sau.

8.1.2 Huấn luyện LLM

Bây giờ giả sử ta có một tập huấn luyện $\mathcal{D}$ gồm K chuỗi. Log-hợp lý của mỗi chuỗi $\mathbf{x} = x_0 \cdots x_m$ trong $\mathcal{D}$ có thể được tính bằng một mô hình ngôn ngữ

$$\mathcal{J}_\theta(\mathbf{x}) \;=\; \sum_{i=1}^{m} \log \Pr_\theta(x_i | x_0, \cdots, x_{i-1}), \tag{8.13}$$

trong đó chỉ số dưới θ trong $\mathcal{L}_\theta(\cdot)$ và $\Pr_\theta(\cdot)$ ký hiệu các tham số của mô hình ngôn ngữ. Khi đó, mục tiêu của huấn luyện hợp lý cực đại (*maximum likelihood*) được định nghĩa là

$$\hat{\theta} \;=\; \arg\max_\theta \sum_{\mathbf{x} \in \mathcal{D}} \mathcal{J}_\theta(\mathbf{x}). \tag{8.14}$$

Chú giải của người dịch: Huấn luyện hợp lý cực đại nói nôm na

θ là tập hợp tất cả các tham số (hàng tỷ con số) của mô hình. $\mathcal{J}_\theta(\mathbf{x})$ đo xem mô hình với bộ tham số θ "tin" vào câu thật $\mathbf{x}$ đến mức nào: mô hình càng gán xác suất cao cho đúng các token thật sự xuất hiện thì giá trị này càng lớn (càng gần 0, vì logarit của một số trong $(0, 1]$ là số không dương). Huấn luyện là đi tìm $\hat{\theta}$ sao cho tổng này trên toàn bộ dữ liệu là lớn nhất, tức là mô hình "đoán đúng từ tiếp theo" giỏi nhất có thể trên dữ liệu huấn luyện.

Việc huấn luyện các mô hình ngôn ngữ dựa trên Transformer với mục tiêu trên thường được xem là một quá trình tối ưu hóa chuẩn cho mạng nơ-ron. Điều này có thể thực hiện bằng các thuật toán hạ gradient (*gradient descent*), vốn được hỗ trợ rộng rãi bởi các bộ công cụ học sâu có sẵn. Có phần đáng ngạc nhiên là hiệu năng mô hình vẫn tiếp tục được cải thiện khi các mô hình ngôn ngữ tiến hóa thành những kiến trúc đòi hỏi nhiều tính toán hơn và được huấn luyện trên những tập dữ liệu lớn hơn [Kaplan và cộng sự, 2020]. Những thành công này đã thúc đẩy các nhà nghiên cứu NLP liên tục mở rộng cả dữ liệu huấn luyện lẫn kích thước mô hình để xây dựng những mô hình ngôn ngữ ngày càng mạnh.

Tuy nhiên, khi mô hình ngôn ngữ trở nên lớn hơn, ta gặp phải những thách thức huấn luyện mới, làm thay đổi đáng kể bài toán so với việc huấn luyện các mô hình tương đối nhỏ. Một trong những thách thức đó xuất phát từ nhu cầu cần có các hệ thống phân tán quy mô lớn để quản lý dữ liệu, tham số mô hình và các quy trình huấn luyện. Việc phát triển và duy trì những hệ thống như vậy đòi hỏi nỗ lực đáng kể về cả kỹ thuật phần mềm lẫn phần cứng, bên cạnh chuyên môn về học sâu. Một vấn đề liên quan là việc mở rộng quy mô huấn luyện đòi hỏi tài nguyên tính toán khổng lồ để hoàn thành quá trình trong một khoảng thời gian hợp lý. Chẳng hạn, huấn luyện từ đầu một LLM có hàng chục tỷ tham số thường cần hàng trăm hoặc hàng nghìn GPU. Yêu cầu này làm tăng mạnh chi phí huấn luyện, nhất là khi tính đến rất nhiều vòng thử nghiệm cần thiết trong quá trình phát triển mô hình. Hơn nữa, xét từ góc độ tối ưu hóa, quá trình huấn luyện có thể trở nên rất bất ổn định khi tăng độ sâu mạng và số lượng tham số. Do đó, thường cần sửa đổi kiến trúc để LLM thích ứng với việc huấn luyện quy mô lớn. Trong Mục 8.2, chúng ta sẽ bàn chi tiết hơn về những vấn đề này.

8.1.3 Tinh chỉnh và ứng dụng LLM

Sau khi đã tiền huấn luyện một LLM, ta có thể áp dụng nó để thực hiện nhiều nhiệm vụ NLP khác nhau. Theo truyền thống, mô hình ngôn ngữ được dùng như thành phần của những hệ thống khác. Chẳng hạn, chúng từng được dùng rộng rãi để chấm điểm các bản dịch trong các hệ thống dịch máy thống kê. Ngược lại, trong AI tạo sinh, LLM được xem là những hệ thống hoàn chỉnh và được dùng để giải quyết các bài toán NLP bằng cách tận dụng bản chất sinh của chúng. Một cách tiếp cận phổ biến là mô tả nhiệm vụ cần làm bằng văn bản rồi *prompt* LLM sinh văn bản dựa trên mô tả này. Đây là một nhiệm vụ sinh văn bản tiêu chuẩn, trong đó mô hình viết tiếp hoặc hoàn thành văn bản bắt đầu từ một ngữ cảnh cho trước.

Một cách hình thức hơn, gọi $\mathbf{x} = x_0 \cdots x_m$ là một chuỗi token biểu diễn ngữ cảnh do người dùng cung cấp, và $\mathbf{y} = y_1 \cdots y_n$ là một chuỗi token nối tiếp ngữ cảnh đó. Quá trình suy luận của một LLM có thể được định nghĩa là tìm chuỗi $\mathbf{y}$ có khả năng cao nhất khi biết $\mathbf{x}$:

$$\begin{aligned}\hat{\mathbf{y}} &= \arg\max_{\mathbf{y}} \log \Pr(\mathbf{y}|\mathbf{x}) \\ &= \arg\max_{\mathbf{y}} \sum_{i=1}^{n} \log \Pr(y_i|x_0, \cdots, x_m, y_1, \cdots, y_{i-1}).\end{aligned} \tag{8.15}$$

Ở đây, $\sum_{i=1}^{n} \log \Pr(y_i|x_0, \cdots, x_m, y_1, \cdots, y_{i-1})$ về mặt toán học tương đương với vế phải của Phương trình (8.2). Nó mô hình hóa log-xác suất của việc dự đoán các token bắt đầu từ vị trí $m+1$, thay vì từ vị trí 0. Trong suốt chương này và các chương sau, chúng ta sẽ dùng các biến riêng $\mathbf{x}$ và $\mathbf{y}$ để phân biệt đầu vào và đầu ra của một LLM, mặc dù chúng có thể được xem là các chuỗi con thuộc cùng một chuỗi liên tục. Khi dùng ký hiệu này, ta có thể thấy phương trình trên rất giống với các công thức dùng trong những mô hình sinh văn bản khác của NLP, chẳng hạn các kiến trúc dịch máy nơ-ron.

Để minh họa cách áp dụng LLM, hãy xét bài toán xác định một câu cho trước có đúng ngữ pháp hay không. Ta có thể định nghĩa một khuôn mẫu (*template*) như sau

```
{∗sentence∗}
Câu hỏi: Câu này có đúng ngữ pháp không?
Trả lời: ___
```

Ở đây ___ biểu diễn phần văn bản ta muốn sinh ra. {∗sentence∗} là một biến chỗ trống (*placeholder*) sẽ được thay bằng câu thực tế do người dùng cung cấp. Ví dụ, giả sử ta có câu "*John seems happy today.*" (John hôm nay có vẻ vui). Ta có thể thay {∗sentence∗} trong khuôn mẫu bằng câu này để tạo đầu vào cho mô hình ngôn ngữ

```
John seems happy today.
Câu hỏi: Câu này có đúng ngữ pháp không?
Trả lời: ___
```

Để thực hiện nhiệm vụ, mô hình ngôn ngữ được cho ngữ cảnh $\mathbf{x}$ = "John seems happy today .\n Câu hỏi : Câu này có đúng ngữ pháp không?\n Trả lời :"[4]. Sau đó, dựa trên ngữ cảnh, nó sinh ra phần văn bản tiếp theo làm câu trả lời. Chẳng hạn, mô hình ngôn ngữ có thể xuất ra "Có" (tức là $\mathbf{y}$ = "Có") nếu đây là văn bản có xác suất cao nhất khi biết ngữ cảnh này.

[4] \n là một ký tự đặc biệt dùng để xuống dòng.

Chú giải của người dịch: Vì sao một mô hình "đoán từ tiếp theo" lại trả lời được câu hỏi?

Mô hình không có "nút" nào dành riêng cho việc kiểm tra ngữ pháp. Nó chỉ làm đúng một việc: viết tiếp văn bản sao cho hợp lý nhất. Nhưng khi văn bản kết thúc bằng "Câu hỏi: … Trả lời:", cách viết tiếp hợp lý nhất chính là một câu trả lời đúng cho câu hỏi đó — vì trong hàng tỷ trang văn bản mô hình đã đọc, sau những câu hỏi như vậy thường là câu trả lời đúng. Như thế, bằng cách khéo léo "đóng khung" nhiệm vụ thành một đoạn văn cần viết tiếp, ta biến mọi bài toán thành bài toán sinh văn bản.

Tương tự, ta có thể định nghĩa thêm các khuôn mẫu để giải quyết những nhiệm vụ khác. Ví dụ, ta có thể dịch một câu tiếng Anh sang tiếng Trung bằng khuôn mẫu sau

```
{∗sentence∗}
Câu hỏi: Bản dịch tiếng Trung của câu tiếng Anh này là gì?
Trả lời: ___
```

hoặc dùng một khuôn mẫu dạng chỉ dẫn

```
{∗sentence∗}
Hãy dịch câu này từ tiếng Anh sang tiếng Trung.

___
```

hoặc dùng một khuôn mẫu dạng mã.

```
[src-lang] = English [tgt-lang] = Chinese [input] = {∗sentence∗}
[output] = ___
```

(Trong khuôn mẫu dạng mã, `src-lang` là ngôn ngữ nguồn, `tgt-lang` là ngôn ngữ đích, `input/output` là đầu vào/đầu ra.)

Các khuôn mẫu trên cung cấp một phương pháp đơn giản mà hiệu quả để "prompt" một LLM duy nhất thực hiện nhiều nhiệm vụ khác nhau mà không cần thay đổi cấu trúc của mô hình. Tuy nhiên, cách làm này đòi hỏi LLM phải nhận biết được và làm theo các chỉ dẫn hay câu hỏi. Một cách để đạt được điều đó là đưa các mẫu huấn luyện gồm chỉ dẫn và câu trả lời tương ứng vào tập dữ liệu tiền huấn luyện. Dù phương pháp này đơn giản, việc xây dựng và huấn luyện LLM từ đầu rất tốn kém về tính toán. Hơn nữa, để dữ liệu làm theo chỉ dẫn phát huy hiệu quả trong tiền huấn luyện, cần một lượng lớn dữ liệu như vậy, nhưng việc thu thập dữ liệu có nhãn quy mô lớn cho mọi nhiệm vụ quan tâm là rất khó.

Phương pháp thứ hai, vốn đã trở thành chuẩn mực trên thực tế trong các nghiên cứu gần đây, là điều chỉnh LLM thông qua tinh chỉnh (*fine-tuning*). Nhờ đó, năng lực dự đoán token học được trong giai đoạn tiền huấn luyện có thể được tổng quát hóa để hoàn thành các nhiệm vụ mới. Ý tưởng đằng sau tinh chỉnh là một phần tri thức tổng quát về ngôn ngữ đã được thu nhận trong quá trình tiền huấn luyện, nhưng ta cần một cơ chế để *kích hoạt* tri thức này nhằm áp dụng nó cho nhiệm vụ mới. Để làm được điều đó, ta có thể tinh chỉnh nhẹ các tham số mô hình bằng dữ liệu làm theo chỉ dẫn. Cách tiếp cận này được gọi là **tinh chỉnh theo chỉ dẫn** (*instruction fine-tuning*).

Một mẫu tinh chỉnh theo chỉ dẫn, được biểu diễn bằng một chuỗi token, có thể được xem là một bộ gồm một đầu vào và đầu ra mong muốn. Ở đây, đầu vào bao gồm các chỉ dẫn, thông tin hệ thống (hay tiền tố hệ thống), và bất kỳ thông tin nào khác do người dùng cung cấp[5]. Để minh họa, hãy xét các ví dụ sau (chữ màu xanh = đầu vào và chữ gạch dưới = đầu ra).

[5]Thông tin hệ thống là một chuỗi token được thêm vào đầu một đầu vào nhằm định hướng hành vi của LLM, chẳng hạn: *bạn là một trợ lý hữu ích và không được xuất ra nội dung độc hại*.

Đọc câu sau và trả lời xem nó có đúng ngữ pháp không.
LLMs are powerful models but are expensive to build.

Có

LLMs are powerful models but are expensive to build.
Câu này có hợp lý về mặt ngữ pháp không?
Phương án trả lời
- Có
- Không

Có

Có hay Không? Có = câu sau đúng ngữ pháp. Không = câu sau có lỗi ngữ pháp.
LLMs are powerful models but are expensive to build.

Có

(Câu tiếng Anh được giữ nguyên vì đó là đối tượng cần kiểm tra ngữ pháp; nghĩa: "LLM là những mô hình mạnh nhưng tốn kém để xây dựng.")

Tất cả các mẫu này mô tả cùng một nhiệm vụ phân loại nhị phân (*binary classification*), nhưng với những chỉ dẫn khác nhau. Để tăng tính đa dạng của dữ liệu tinh chỉnh và giúp LLM tổng quát hóa, ta có thể định nghĩa thêm nhiều nhiệm vụ bằng chỉ dẫn.

Hãy viết một truyện ngắn về ba nhân vật.
Nhân vật 1: Một cậu bé tên là Andy.
Nhân vật 2: Chú chó của Andy.
Nhân vật 3: Bạn của Andy, Jane.

Giữa lòng một thành phố nhộn nhịp, Andy, chú chó trung thành Rocket và cô bạn Jane đã khám phá ra một khu vườn bí mật dưới ánh hoàng hôn. Được bao quanh bởi những hàng cây cổ thụ và tiếng gió thì thầm, họ chia sẻ cho nhau những ước mơ và bí mật, gắn kết một tình bạn không gì phá vỡ được. Khi màn đêm buông xuống, họ cùng hứa dưới bầu trời sao rằng sẽ luôn tìm thấy điều kỳ diệu trong những điều bình dị. Cùng nhau, họ nhận ra rằng cuộc phiêu lưu thực sự nằm ở những khoảnh khắc bên bạn bè, và mỗi ngày đều là một kho báu đang chờ được khám phá.

Hãy chỉ cho tôi công thức làm kem.

Nguyên liệu:
2 cốc kem tươi béo; 1 cốc sữa; 3/4 cốc đường; 2 thìa cà phê tinh chất vani;
Một nhúm muối.
Các bước:
1. Đánh đều tất cả nguyên liệu cho đến khi đường tan.
2. Để hỗn hợp trong tủ lạnh 2 giờ cho lạnh.
3. Cấp đông trong một khay nông, khuấy 30 phút một lần, 3–4 lần.
4. Cấp đông thêm 2 giờ cho đông cứng.
5. Thưởng thức món kem tự làm của bạn.

Nếu bạn mua 5 quả táo và mỗi quả giá $1.20, thì tổng cộng bạn tiêu bao nhiêu tiền?

$6.00

Hãy viết một chương trình Python để tính tổng bình phương của các số sau.
1 , 2 , 10 , -9 , 78

```
numbers = [1,2,10,-9,78]
sum_of_squares = sum(x**2 for x in numbers)
print(sum_of_squares)
```

Để có được năng lực làm theo chỉ dẫn, cần một lượng dữ liệu tinh chỉnh nhất định. Dữ liệu này có thể gồm những chỉ dẫn đa dạng và các câu trả lời khả dĩ. Người ta nhận thấy rằng tăng số lượng nhiệm vụ tinh chỉnh có lợi cho việc cải thiện hiệu năng của LLM [Chung và cộng sự, 2022]. Lưu ý rằng mặc dù càng nhiều dữ liệu tinh chỉnh càng tốt, lượng dữ liệu này thường nhỏ hơn dữ liệu tiền huấn luyện nhiều bậc độ lớn. Chẳng hạn, LLM có thể được tinh chỉnh với hàng chục hoặc hàng trăm nghìn mẫu, thậm chí ít hơn nếu các mẫu có chất lượng cao [Zhou và cộng sự, 2023a; Chen và cộng sự, 2023b], trong khi tiền huấn luyện những mô hình như vậy có thể cần hàng tỷ hoặc hàng nghìn tỷ token, dẫn đến nhu cầu tính toán lớn hơn đáng kể và thời gian huấn luyện dài hơn [Touvron và cộng sự, 2023a].

Cũng cần lưu ý rằng ta không nên kỳ vọng dữ liệu tinh chỉnh bao quát mọi nhiệm vụ hạ nguồn mà ta định áp dụng LLM. Một cách hiểu phổ biến về cơ chế hoạt động của cách tiếp cận *tiền huấn luyện + tinh chỉnh* là: LLM đã thu nhận được tri thức để hiểu chỉ dẫn và sinh câu trả lời trong giai đoạn tiền huấn luyện. Tuy nhiên, những năng lực này chưa được kích hoạt hoàn toàn cho đến khi ta đưa vào một hình thức giám sát nào đó. Hành vi làm theo chỉ dẫn tổng quát xuất hiện khi ta tinh chỉnh mô hình với một lượng tương đối nhỏ dữ liệu có nhãn. Nhờ vậy, ta có thể đạt được một mức độ **học zero-shot** (*zero-shot learning*): các mô hình đã tinh chỉnh có thể xử lý những nhiệm vụ mới mà chúng chưa từng được huấn luyện hay tinh chỉnh một cách tường minh [Sanh và cộng sự, 2022; Wei và cộng sự, 2022a]. Năng lực học zero-shot này phân biệt LLM sinh với các mô hình tiền huấn luyện trước đây như BERT, vốn chủ yếu được tinh chỉnh cho các nhiệm vụ cụ thể.

Chú giải của người dịch: Tiền huấn luyện và tinh chỉnh: một phép so sánh

Có thể hình dung tiền huấn luyện giống như nhiều năm đọc sách: mô hình đọc hàng nghìn tỷ token và tích lũy hiểu biết rộng về ngôn ngữ và thế giới, nhưng chưa biết cách "làm bài theo đề". Tinh chỉnh theo chỉ dẫn giống như một khóa luyện ngắn với vài chục nghìn bài mẫu "đề bài — lời giải": không dạy thêm nhiều kiến thức mới, mà dạy cách dùng kiến thức sẵn có để trả lời đúng yêu cầu. "Zero-shot" nghĩa là sau khóa luyện đó, mô hình làm được cả những dạng bài chưa từng xuất hiện trong bài mẫu.

Khi đã tuyển chọn xong một tập dữ liệu chỉ dẫn, bản thân quá trình tinh chỉnh tương đối đơn giản. Nó có thể được xem là việc huấn luyện mạng nơ-ron tiêu chuẩn, tương tự tiền huấn luyện, nhưng được thực hiện trên một tập huấn luyện nhỏ hơn nhiều. Gọi $\mathcal{D}_{\text{tune}}$ là tập dữ liệu tinh chỉnh và $\hat{\theta}$ là các tham số mô hình đã được tối ưu qua tiền huấn luyện. Ta có thể sửa Phương trình (8.14) để có được mục tiêu của tinh chỉnh

$$\tilde{\theta} \;=\; \arg\max_{\hat{\theta}^+} \sum_{\text{sample}\in\mathcal{D}_{\text{tune}}} \mathcal{J}_{\hat{\theta}^+}(\text{sample}). \tag{8.16}$$

Ở đây $\tilde{\theta}$ ký hiệu các tham số tối ưu. Ký hiệu $\hat{\theta}^+$ có nghĩa là quá trình tinh chỉnh bắt đầu từ các tham số đã tiền huấn luyện $\hat{\theta}$.

Với mỗi sample $\in \mathcal{D}_{\text{tune}}$ (mỗi *mẫu*), ta chia nó thành một đoạn đầu vào $\mathbf{x}_{\text{sample}}$ và một đoạn đầu ra $\mathbf{y}_{\text{sample}}$, tức là

$$\text{sample} \;=\; [\mathbf{x}_{\text{sample}}, \mathbf{y}_{\text{sample}}]. \tag{8.17}$$

Sau đó ta định nghĩa hàm log-hợp lý là

$$\mathcal{J}_{\hat{\theta}+}(\text{sample}) = \log \Pr_{\hat{\theta}+}(\mathbf{y}_{\text{sample}}|\mathbf{x}_{\text{sample}}). \tag{8.18}$$

Nói cách khác, ta tính hàm mất mát trên chuỗi con $\mathbf{y}_{\text{sample}}$ thay vì trên toàn bộ chuỗi. Khi hiện thực lan truyền ngược (*back-propagation*) cho phương trình này trong thực tế, chuỗi $[\mathbf{x}_{\text{sample}}, \mathbf{y}_{\text{sample}}]$ được xây dựng trong lượt truyền xuôi (*forward pass*) như thường lệ. Tuy nhiên, trong lượt truyền ngược (*backward pass*), gradient lỗi chỉ được lan truyền ngược qua những phần của mạng tương ứng với $\mathbf{y}_{\text{sample}}$, còn phần còn lại của mạng giữ nguyên. Ví dụ, xét chuỗi

$$\underbrace{\langle s\rangle \text{ Bình phương số này . 2 .}}_{\text{Ngữ cảnh (Đầu vào)}} \underbrace{\text{Kết quả là 4 .}}_{\text{Dự đoán (Đầu ra)}}$$

Hàm mất mát chỉ được tính và lan truyền ngược cho Kết quả là 4 ..

Chú giải của người dịch: Vì sao chỉ tính mất mát trên phần đầu ra?

Phần đầu vào (câu hỏi, chỉ dẫn) là do người dùng viết; ta không cần mô hình học cách *viết ra câu hỏi*, mà chỉ cần nó học cách *trả lời*. Vì thế, trong ví dụ trên, mô hình vẫn "đọc" cả chuỗi, nhưng chỉ bị "chấm điểm" ở các token *Kết quả là 4 .*: nếu nó gán xác suất thấp cho các token này thì bị phạt và tham số được điều chỉnh; còn các token của câu hỏi không đóng góp vào hàm mất mát.

Tinh chỉnh theo chỉ dẫn cũng đòi hỏi khối lượng công việc kỹ thuật đáng kể. Để đạt kết quả thỏa đáng, người ta có thể phải thử nghiệm với các thiết lập khác nhau của tốc độ học (*learning rate*), kích thước lô (*batch*), số bước tinh chỉnh, v.v. Điều này thường đòi hỏi nhiều lượt tinh chỉnh và đánh giá. Chi phí và công sức thử nghiệm của tinh chỉnh vẫn rất đáng kể và không nên bị xem nhẹ, dù chúng thấp hơn nhiều so với giai đoạn tiền huấn luyện.

Mặc dù ở đây ta tập trung vào tinh chỉnh theo chỉ dẫn như một ví dụ minh họa, các kỹ thuật tinh chỉnh đóng vai trò quan trọng trong việc phát triển nhiều loại LLM và được dùng rộng rãi hơn thế. Ví dụ bao gồm tinh chỉnh LLM thành chatbot (*chatbot*) bằng dữ liệu hội thoại, và điều chỉnh các mô hình này để xử lý những chuỗi rất dài. Việc tinh chỉnh được ứng dụng rộng rãi đã khiến các nhà nghiên cứu cải tiến những kỹ thuật này, chẳng hạn thiết kế các thuật toán tinh chỉnh hiệu quả hơn. Dù nghiên cứu về tinh chỉnh rất phong phú, trong mục này chúng tôi chỉ trình bày khái quát các bước then chốt. Ta sẽ thấy những thảo luận chi tiết hơn về chủ đề này trong các chương sau.

8.1.4 Căn chỉnh LLM với thế giới

Tinh chỉnh theo chỉ dẫn cung cấp một cách đơn giản để điều chỉnh LLM cho những nhiệm vụ có thể được định nghĩa rõ ràng. Nhìn rộng ra, điều này có thể được xem là một bài toán **căn chỉnh** (*alignment*). Ở đây, căn chỉnh là quá trình dẫn dắt LLM hành xử theo những cách phù hợp với các giá trị và mục tiêu của con người. Sự dẫn dắt này có thể đến từ dữ liệu có nhãn, phản hồi của con người (*human feedback*), hoặc bất kỳ hình thức nào khác của sở thích (ưu tiên) của con người (*human preferences*). Chẳng hạn, ta muốn LLM không chỉ làm theo chỉ dẫn một cách chính xác, mà còn không thiên lệch, trung thực và vô hại. Vì vậy, ta cần giám sát mô hình hướng tới các giá trị và kỳ vọng của con người. Một ví dụ điển hình là khi ta hỏi một LLM cách chế tạo vũ khí, nó có thể đưa ra danh sách các bước then chốt để làm việc đó nếu không được căn chỉnh cẩn thận. Tuy nhiên, một mô hình có trách nhiệm cần nhận ra và tránh đáp ứng những yêu cầu về thông tin có hại hoặc bất hợp pháp. Trong trường hợp này, căn chỉnh là yếu tố then chốt để bảo đảm LLM hành động có trách nhiệm và tuân thủ các nguyên tắc đạo đức.

Một khái niệm liên quan chặt chẽ đến căn chỉnh là an toàn AI (*AI safety*). Một mục tiêu tối thượng của nghiên cứu AI là xây dựng những hệ thống thông minh an toàn và có ích cho xã hội. Để đạt được điều này, ta phải bảo đảm các hệ thống đó luôn vững vàng, bảo mật và kiểm soát được trong nhiều điều kiện thực tế khác nhau, kể cả những trường hợp bị lạm dụng hay bị tấn công đối kháng (*adversarial attacks*). Với LLM, mức độ an toàn có thể được nâng cao đáng kể nhờ căn chỉnh đúng đắn theo sự hướng dẫn của con người, thông qua dữ liệu do con người gán nhãn và các tương tác động với người dùng trong quá trình triển khai.

Căn chỉnh là việc khó vì các giá trị và kỳ vọng của con người rất đa dạng và luôn thay đổi. Đôi khi, rất khó mô tả chính xác con người muốn gì, trừ khi ta thấy được câu trả lời của LLM cho các yêu cầu của người dùng. Điều này khiến căn chỉnh không còn là bài toán điều chỉnh LLM trên những nhiệm vụ định sẵn, mà là một bài toán lớn hơn: huấn luyện chúng thông qua tương tác với thế giới thực.

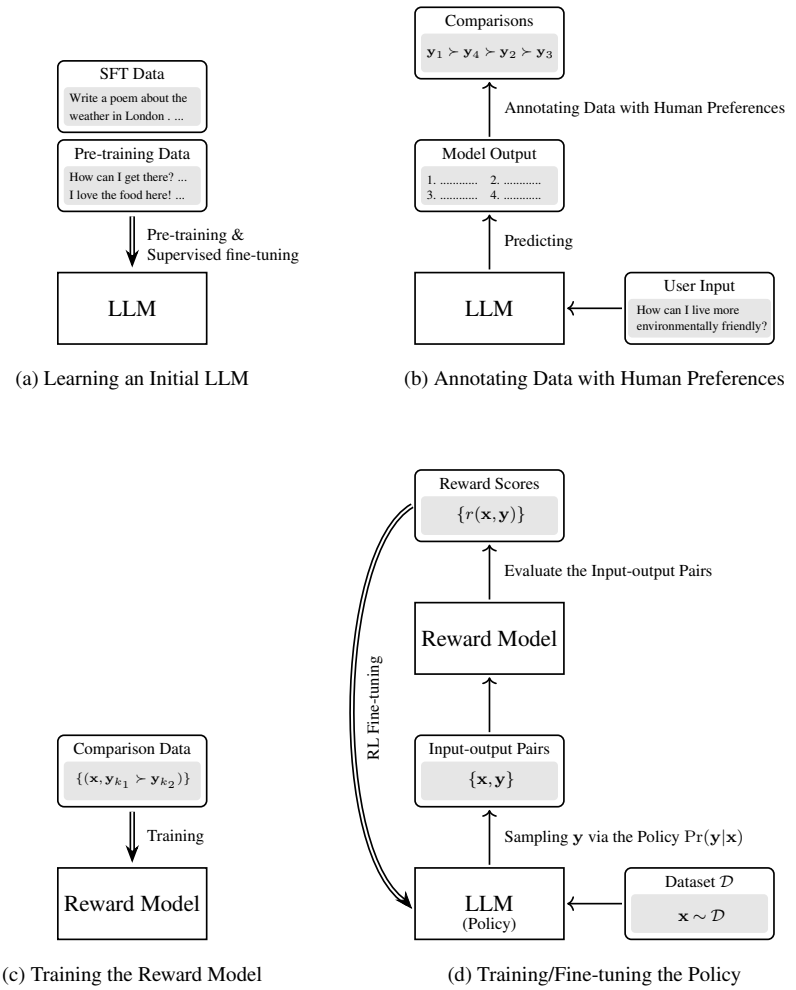
Do những lo ngại về việc kiểm soát các hệ thống AI, nghiên cứu về vấn đề căn chỉnh LLM đã bùng nổ. Thông thường, hai bước căn chỉnh được áp dụng sau khi LLM được tiền huấn luyện trên dữ liệu không nhãn quy mô lớn.

- **Tinh chỉnh có giám sát** (**SFT**, *Supervised Fine-tuning*). Bước này tiếp tục huấn luyện LLM đã tiền huấn luyện trên dữ liệu có nhãn mới, hướng tới nhiệm vụ. Một kỹ thuật SFT thường dùng là tinh chỉnh theo chỉ dẫn. Như đã mô tả ở tiểu mục trước, bằng cách học từ dữ liệu được chú thích theo cặp chỉ dẫn–câu trả lời, LLM có thể được căn chỉnh với các hành vi mong muốn trong việc làm theo chỉ dẫn, nhờ đó có khả năng thực hiện nhiều nhiệm vụ được mô tả bằng chỉ dẫn. Tinh chỉnh có giám sát có thể được xem là đi theo mô thức *tiền huấn luyện + tinh chỉnh*, và là một phương pháp tương đối đơn giản để điều chỉnh LLM.

- **Học từ phản hồi của con người** (*Learning from Human Feedback*). Sau tiền huấn luyện và tinh chỉnh có giám sát, LLM có thể đáp ứng yêu cầu của người dùng khi được prompt phù hợp. Tuy nhiên, mô hình vẫn có thể sinh ra nội dung sai sự thật, thiên lệch hoặc có hại. Để căn chỉnh LLM tốt hơn với các giá trị của con người, một cách hiệu quả là học trực tiếp từ phản hồi của con người. Chẳng hạn, cho một chỉ dẫn và câu trả lời do mô hình sinh ra, các chuyên gia được yêu cầu đánh giá xem câu trả lời phù hợp đến mức nào với sở thích của họ và với các nguyên tắc an toàn. Phản hồi này sau đó được dùng để tiếp tục huấn luyện LLM nhằm đạt được sự căn chỉnh tốt hơn.

(Nhãn trong hình: (a) *Learning an Initial LLM* = học một LLM ban đầu, *Pre-training Data / SFT Data* = dữ liệu tiền huấn luyện / dữ liệu SFT; (b) *Annotating Data with Human Preferences* = gán nhãn dữ liệu theo sở thích của con người, *User Input* = đầu vào của người dùng (“Làm thế nào để tôi sống thân thiện với môi trường hơn?”), *Model Output* = đầu ra mô hình, *Predicting* = dự đoán, *Comparisons* = các so sánh; (c) *Training the Reward Model* = huấn luyện mô hình phần thưởng, *Comparison Data* = dữ liệu so sánh; (d) *Training/Fine-tuning the Policy* = huấn luyện/tinh chỉnh chính sách, *Sampling* **y** *via the Policy* = lấy mẫu **y** theo chính sách, *Input-output Pairs* = các cặp đầu vào–đầu ra, *Evaluate the Input-output Pairs* = đánh giá các cặp đầu vào–đầu ra, *Reward Scores* = điểm phần thưởng, *RL Fine-tuning* = tinh chỉnh bằng học tăng cường.)

Một phương pháp điển hình để học từ phản hồi của con người là xem nó như một bài toán học tăng cường (*reinforcement learning, RL*), được gọi là **học tăng cường từ phản hồi của con người** (*reinforcement learning from human feedback*) (**RLHF**) [Ouyang và cộng sự, 2022]. Phương pháp RLHF ban đầu được đề xuất để giải quyết các bài toán ra quyết định tuần tự tổng quát [Christiano và cộng sự, 2017], và sau đó được sử dụng thành công trong việc phát triển các mô hình thuộc dòng GPT [Stiennon và cộng sự, 2020]. Là một cách tiếp cận học tăng cường, mục tiêu của RLHF là học một chính sách (*policy*) bằng cách cực đại hóa một phần thưởng nào đó nhận được từ môi trường. Cụ thể,



(a) Learning an Initial LLM

(b) Annotating Data with Human Preferences

(c) Training the Reward Model

(d) Training/Fine-tuning the Policy

Hình 8.2: Tổng quan về RLHF. Có 4 bước then chốt: a) huấn luyện một LLM ban đầu (tức chính sách) bằng tiền huấn luyện và tinh chỉnh có giám sát; b) thu thập dữ liệu sở thích của con người bằng cách xếp hạng các đầu ra của LLM; c) huấn luyện một mô hình phần thưởng bằng kết quả xếp hạng; d) tinh chỉnh chính sách bằng học tăng cường dựa trên mô hình phần thưởng. Các mũi tên nét đôi biểu thị huấn luyện hoặc tinh chỉnh.

RLHF gồm hai thành phần chính:

- **Tác tử** (*Agent*). Tác tử, còn được gọi là tác tử LM, chính là LLM mà ta muốn tối ưu. Tác tử này hoạt động bằng cách tương tác với môi trường của nó: nó nhận văn bản từ môi trường và tạo ra các câu trả lời bằng văn bản. Chính sách của tác tử này là phân phối xác suất có điều kiện do LLM định nghĩa, ký hiệu là $\Pr(\mathbf{y}|\mathbf{x})$.
- **Mô hình phần thưởng** (*Reward Model*). Mô hình phần thưởng đóng vai trò đại diện thay cho môi trường con người. Mỗi khi tác tử tạo ra một chuỗi đầu ra, mô hình phần thưởng đánh giá nó và gán cho nó một điểm số (tức là phần thưởng (*reward*)). Điểm số này cho tác tử biết một cách tường minh về chất lượng và mức độ căn chỉnh của đầu ra.

Chú giải của người dịch: Học tăng cường qua hình ảnh huấn luyện thú cưng

Học tăng cường giống cách ta dạy một chú chó: ta không chỉ cho nó từng động tác đúng, mà để nó tự thử, rồi thưởng khi nó làm tốt. Dần dần nó làm nhiều hơn những hành vi được thưởng. Ở đây, "chú chó" là LLM (tác tử), "hành vi" là câu trả lời nó viết ra, còn "người thưởng" là mô hình phần thưởng — một mô hình được huấn luyện để chấm điểm giống như con người sẽ chấm. *Chính sách* chỉ đơn giản là "quy tắc hành xử" của tác tử: với LLM, đó là phân phối xác suất $\Pr(\mathbf{y}|\mathbf{x})$ quyết định nó trả lời $\mathbf{y}$ thế nào cho mỗi câu hỏi $\mathbf{x}$.

Trong RLHF, có hai giai đoạn học riêng biệt được thực hiện: 1) học mô hình phần thưởng, tức là huấn luyện mô hình phần thưởng bằng phản hồi của con người về các đầu ra của tác tử, và 2) học chính sách, tức là dùng các thuật toán học tăng cường để tối ưu chính sách dưới sự dẫn dắt của mô hình phần thưởng đã huấn luyện. Các bước then chốt của RLHF có thể được tóm tắt như sau:

- Xây dựng một chính sách ban đầu bằng tiền huấn luyện và tinh chỉnh theo chỉ dẫn.
- Dùng chính sách để sinh nhiều đầu ra cho mỗi đầu vào, rồi thu thập phản hồi của con người về các đầu ra này (ví dụ: so sánh giữa các đầu ra).
- Học một mô hình phần thưởng từ phản hồi của con người.
- Tinh chỉnh chính sách dưới sự giám sát của mô hình phần thưởng.

Hình 8.2 trình bày tổng quan về RLHF. Vì mục này chỉ là phần giới thiệu ngắn về các khái niệm của LLM, chúng tôi sẽ không bàn chi tiết các kỹ thuật RLHF. Thay vào đó, chúng tôi minh họa các ý tưởng cơ bản đằng sau RLHF bằng một ví dụ đơn giản.

Giả sử ta đã huấn luyện một LLM bằng tiền huấn luyện và tinh chỉnh theo chỉ dẫn. LLM này được triển khai để đáp ứng yêu cầu của người dùng. Chẳng hạn, một người dùng có thể nhập

Làm thế nào để tôi có một lối sống thân thiện với môi trường hơn?

Ta dùng LLM để sinh 4 đầu ra khác nhau (ký hiệu là $\{\mathbf{y}_1, \cdots, \mathbf{y}_4\}$) bằng cách lấy mẫu từ phân phối đầu ra

Đầu ra 1 ($\mathbf{y}_1$): Hãy cân nhắc chuyển sang dùng xe điện hoặc xe đạp thay cho ô tô truyền thống để giảm phát thải carbon và bảo vệ hành tinh của chúng ta.

Đầu ra 2 ($\mathbf{y}_2$): Hãy theo đuổi lối sống tối giản. Sở hữu ít đồ đạc hơn để giảm tiêu dùng và giảm tác động môi trường của việc sản xuất và thải bỏ.

Đầu ra 3 ($\mathbf{y}_3$): Hãy sống tách khỏi lưới điện. Tự sản xuất năng lượng tái tạo và hứng nước mưa để hoàn toàn tự cung tự cấp và giảm phụ thuộc vào các nguồn tài nguyên không tái tạo.

Đầu ra 4 ($\mathbf{y}_4$): Hãy ủng hộ nông sản địa phương để giảm lượng khí thải carbon do vận chuyển thực phẩm, đồng thời được thưởng thức đồ ăn tươi ngon, lành mạnh.

Sau đó ta yêu cầu những người gán nhãn đánh giá các đầu ra này. Một cách đơn giản là gán một điểm đánh giá cho từng đầu ra. Khi đó, bài toán học mô hình phần thưởng có thể được phát biểu như một nhiệm vụ huấn luyện mô hình hồi quy (*regression*). Nhưng việc cho điểm bằng số đối với các đầu ra của LLM không phải là nhiệm vụ dễ dàng cho người gán nhãn. Thường rất khó thiết kế một tiêu chuẩn gán nhãn mà mọi người gán nhãn đều đồng ý và dễ dàng tuân theo. Một phương pháp thay thế, phổ biến hơn trong việc phát triển LLM, là *xếp hạng* các đầu ra này. Ví dụ, một cách xếp hạng khả dĩ cho các đầu ra trên là

$$\mathbf{y}_1 \succ \mathbf{y}_4 \succ \mathbf{y}_2 \succ \mathbf{y}_3$$

Chú giải của người dịch: Ký hiệu $\succ$

$\mathbf{y}_a \succ \mathbf{y}_b$ đọc là "$\mathbf{y}_a$ được ưa thích hơn $\mathbf{y}_b$". Dãy trên nghĩa là người gán nhãn thấy đầu ra 1 tốt nhất, rồi đến 4, rồi 2, kém nhất là 3 (lời khuyên "sống tách khỏi lưới điện" khó thực hiện với đa số người). So sánh "cái nào hơn" dễ và nhất quán hơn nhiều so với việc cho điểm tuyệt đối (7 hay 8 điểm?), giống như giám khảo dễ chọn bài hay hơn trong hai bài văn hơn là chấm chính xác từng bài. Từ một xếp hạng 4 đầu ra, ta có thể rút ra $\binom{4}{2} = 6$ cặp so sánh để huấn luyện.

Sau đó, một mô hình phần thưởng được huấn luyện bằng kết quả xếp hạng này. Nói chung, mô hình phần thưởng trong RLHF là một mô hình ngôn ngữ có cùng kiến trúc với LLM đích, nhưng có kích thước nhỏ hơn. Cho đầu vào $\mathbf{x}$ và đầu ra $\mathbf{y}_k$, ta nối chúng lại để tạo thành chuỗi $\text{seq}_k = [\mathbf{x}, \mathbf{y}_k]$. Chuỗi này được xử lý từ trái sang phải bằng giải mã cưỡng bức (*forced decoding*). Vì trong mô hình hóa ngôn ngữ mỗi vị trí chỉ truy cập được ngữ cảnh bên trái của nó, đầu ra của tầng Transformer trên cùng tại vị trí đầu tiên không thể dùng làm biểu diễn của chuỗi. Thay vào đó, một ký hiệu đặc biệt (ví dụ $\langle /s \rangle$) được thêm vào cuối chuỗi, và đầu ra tương ứng của chồng các tầng Transformer được xem là biểu diễn của toàn bộ chuỗi. Một tầng đầu ra, chẳng hạn một tầng biến đổi tuyến tính, được xây dựng phía trên biểu diễn này để sinh ra phần thưởng, ký hiệu là $R(\text{seq}_k)$ hoặc $R(\mathbf{x}, \mathbf{y}_k)$.

Ta huấn luyện mô hình phần thưởng này bằng hàm mất mát xếp hạng (*ranking loss*). Chẳng hạn, một hàm mất mát xếp hạng theo cặp có thể được viết dưới dạng

$$\text{Loss}_\omega(\mathcal{D}_r) \;=\; -\mathbb{E}_{(\mathbf{x},\mathbf{y}_{k_1},\mathbf{y}_{k_2})\sim\mathcal{D}_r} \log\big(\text{Sigmoid}(R_\omega(\mathbf{x},\mathbf{y}_{k_1}) - R_\omega(\mathbf{x},\mathbf{y}_{k_2}))\big), \tag{8.19}$$

trong đó ω biểu diễn các tham số của mô hình phần thưởng, và $\mathcal{D}_r$ biểu diễn một tập các bộ gồm một đầu vào và một cặp đầu ra. $(\mathbf{x}, \mathbf{y}_{k_1}, \mathbf{y}_{k_2}) \sim \mathcal{D}_r$ là một phép lấy mẫu, rút một mẫu $(\mathbf{x}, \mathbf{y}_{k_1}, \mathbf{y}_{k_2})$ từ $\mathcal{D}_r$ với một xác suất nào đó. Ví dụ, giả sử trước tiên ta rút một đầu vào mô hình $\mathbf{x}$ theo phân phối đều (*uniform distribution*), rồi rút một cặp đầu ra mô hình với xác suất $\mathbf{y}_{k_1} \succ \mathbf{y}_{k_2}$ khi biết $\mathbf{x}$ (ký hiệu là $\Pr(\mathbf{y}_{k_1} \succ \mathbf{y}_{k_2}|\mathbf{x})$). Hàm mất mát tương ứng là

$$\begin{aligned}
&\text{Loss}_\omega(\mathcal{D}_r)\\
&= -\sum \Pr(\mathbf{x}) \cdot \Pr(\mathbf{y}_{k_1} \succ \mathbf{y}_{k_2}|\mathbf{x}) \cdot \log\big(\text{Sigmoid}(R_\omega(\mathbf{x},\mathbf{y}_{k_1}) - R_\omega(\mathbf{x},\mathbf{y}_{k_2}))\big)\\
&= -\frac{1}{K}\sum \Pr(\mathbf{y}_{k_1} \succ \mathbf{y}_{k_2}|\mathbf{x}) \cdot \log\big(\text{Sigmoid}(R_\omega(\mathbf{x},\mathbf{y}_{k_1}) - R_\omega(\mathbf{x},\mathbf{y}_{k_2}))\big),
\end{aligned} \tag{8.20}$$

trong đó K biểu diễn số lượng đầu vào mô hình tham gia vào việc lấy mẫu. Dù dạng của các hàm này trông có vẻ phức tạp, ý tưởng của chúng rất đơn giản: ta phạt mô hình nếu thứ hạng dự đoán của hai

đầu ra khác với thứ hạng do con người gán nhãn. Ngược lại, mô hình được "thưởng" nếu thứ hạng dự đoán khớp với thứ hạng do con người gán nhãn.

Chú giải của người dịch: Hiểu Phương trình (8.19) bằng số cụ thể

Ký hiệu $\mathbb{E}$ là *kỳ vọng*: hiểu nôm na là giá trị trung bình khi lấy mẫu nhiều lần. Hàm $\text{Sigmoid}(z) = \frac{1}{1+e^{-z}}$ biến mọi số thực thành một số trong $(0, 1)$, có thể đọc là "xác suất mô hình tin rằng $\mathbf{y}_{k_1}$ tốt hơn $\mathbf{y}_{k_2}$". Giả sử con người nói $\mathbf{y}_{k_1}$ tốt hơn.

- Nếu mô hình chấm $R(\mathbf{x}, \mathbf{y}_{k_1}) = 3, R(\mathbf{x}, \mathbf{y}_{k_2}) = 1$: hiệu là 2, $\text{Sigmoid}(2) \approx 0.88$, mất mát $-\ln 0.88 \approx 0.13$ (nhỏ: mô hình đúng).
- Nếu chấm ngược lại, 1 và 3: hiệu là -2, $\text{Sigmoid}(-2) \approx 0.12$, mất mát $-\ln 0.12 \approx 2.13$ (lớn: mô hình bị phạt nặng).

Lưu ý rằng chỉ *hiệu* của hai điểm số là quan trọng, nên mô hình học cách cho điểm sao cho câu trả lời được ưa thích luôn có điểm cao hơn.

Ta có thể huấn luyện mô hình phần thưởng bằng cách cực tiểu hóa hàm mất mát xếp hạng ở trên

$$\hat{\omega} = \arg\min_{\omega} \text{Loss}_{\omega}(\mathcal{D}_r). \tag{8.21}$$

Mô hình thu được $R_{\hat{\omega}}(\cdot)$ có thể được dùng để đánh giá bất kỳ cặp đầu vào và đầu ra nào. Lưu ý rằng mặc dù mô hình phần thưởng được huấn luyện bằng một mục tiêu dựa trên xếp hạng, nó lại được dùng để *chấm điểm*. Điều này cho phép nó cung cấp các tín hiệu giám sát liên tục, rất có lợi cho việc huấn luyện những mô hình khác.

Bây giờ ta chuyển sang bài toán học chính sách. Một mục tiêu thường được dùng là cực đại hóa phần thưởng trên một tập các cặp đầu vào–đầu ra. Theo một dạng tương tự Phương trình (8.16), ta có được một mục tiêu huấn luyện đơn giản cho việc tinh chỉnh bằng học tăng cường

$$\tilde{\theta} = \arg\max_{\hat{\theta}^+} \mathbb{E}_{(\mathbf{x}, \mathbf{y}_{\hat{\theta}^+}) \sim \mathcal{D}_{\text{rlft}}} R_{\hat{\omega}}(\mathbf{x}, \mathbf{y}_{\hat{\theta}^+}), \tag{8.22}$$

trong đó các tham số tối ưu $\tilde{\theta}$ có được bằng cách tinh chỉnh các tham số đã tiền huấn luyện $\hat{\theta}$. $\mathcal{D}_{\text{rlft}}$ là tập dữ liệu tinh chỉnh bằng học tăng cường. Với mỗi mẫu $(\mathbf{x}, \mathbf{y}_{\hat{\theta}^+})$, $\mathbf{x}$ được lấy mẫu từ một tập dữ liệu chuỗi đầu vào đã chuẩn bị sẵn, còn $\mathbf{y}_{\hat{\theta}^+}$ được lấy mẫu từ phân phối $\Pr_{\hat{\theta}^+}(\mathbf{y}|\mathbf{x})$ do chính sách đưa ra.

Trong thực tế, các thuật toán học tăng cường tiên tiến hơn, chẳng hạn **tối ưu chính sách cận kề** (*proximal policy optimization*) (**PPO**), thường được dùng để huấn luyện ổn định hơn cũng như đạt hiệu năng tốt hơn. Chúng tôi để dành phần thảo luận chi tiết về các thuật toán học tăng cường cho những phần sau của cuốn sách, nơi RLHF được dùng rộng rãi cho việc căn chỉnh.

Một câu hỏi thú vị nảy sinh ở đây: tại sao không coi việc học từ sở thích của con người là một bài toán học có giám sát (*supervised learning*) tiêu chuẩn? Câu hỏi này liên quan mật thiết đến thảo luận ở trên về độ khó của việc gán nhãn dữ liệu. Thường thì việc mô tả các giá trị và mục tiêu của con người đã là thách thức, và việc con người tự viết ra những đầu ra được căn chỉnh tốt lại càng khó hơn. Thay vào đó, việc gán nhãn sở thích cho một danh sách đầu ra mô hình cho trước là một nhiệm vụ đơn giản hơn. Bằng cách đó, ta có thể tạo ra một mô hình hiểu được sở thích của con người, rồi dùng nó làm mô hình phần thưởng để huấn luyện các chính sách. Từ góc độ học máy, RLHF đặc biệt hữu ích trong những tình huống mà hành vi mong muốn của tác tử khó *minh họa* nhưng lại dễ được con người *nhận ra*. Một ưu điểm khác của RLHF là khả năng khám phá không gian mẫu. Nhờ sử dụng các kỹ thuật lấy mẫu, các mô hình được huấn luyện bằng học tăng cường có thể vượt ra ngoài tập dữ liệu đã gán nhãn để khám phá thêm những mẫu khác. Khả năng khám phá này cho phép RLHF tìm ra những chính sách có thể có lợi mà không hiển nhiên nếu chỉ dựa vào dữ liệu có nhãn.

8.1.5 Prompt LLM

Cho đến giờ, ta đã thấy rằng LLM có thể được dùng để thực hiện nhiều nhiệm vụ khác nhau bằng cách cung cấp cho chúng các prompt phù hợp. Không có giới hạn nào đối với các prompt này; chúng có thể chứa bất kỳ thông tin nào ta muốn cung cấp cho LLM, chẳng hạn chỉ dẫn bằng ngôn ngữ tự nhiên và ngữ cảnh hội thoại. Vì cách tiếp cận này không đòi hỏi thêm việc huấn luyện hay tinh chỉnh nào, việc điều chỉnh LLM trở nên rất hiệu quả một khi chúng đã được phát triển. Điều này làm thay đổi căn bản mô thức của NLP: ta không còn cần phát triển các hệ thống riêng cho từng nhiệm vụ mà có thể dùng một LLM duy nhất, được huấn luyện tốt, để thực hiện các nhiệm vụ khác nhau chỉ bằng cách prompt nó. Từ đó, một khía cạnh hấp dẫn của LLM xuất hiện: người dùng có thể dễ dàng tạo ra các hệ thống tùy biến bằng cách tự thiết kế prompt cho LLM. Do vai trò quan trọng của prompting đối với LLM, **thiết kế prompt** (*prompt engineering*) đã trở thành một lĩnh vực nghiên cứu rất sôi động trong NLP.

Thuật ngữ *prompt* được dùng theo nhiều cách khác nhau trong các tài liệu. Trong chương này, thuật ngữ này chỉ *toàn bộ* đầu vào của LLM, và vì vậy ta dùng các thuật ngữ *prompt* và *đầu vào mô hình* thay thế cho nhau. Trước khi bàn thêm về prompting, hãy cùng xem một vài ví dụ trong đó prompt phức tạp hơn những prompt đã trình bày ở các tiểu mục trước. Lưu ý rằng tiểu mục này không nhằm hướng dẫn viết prompt chất lượng cao, mà nhằm làm nổi bật một số vấn đề thú vị khi prompt LLM.

Một trong những cách dùng LLM phổ biến là giao cho chúng một "vai" khi sinh câu trả lời. Chẳng hạn, LLM có thể đóng vai một nhà tâm lý học khi trả lời câu hỏi.

Hãy giải thích trì hoãn sự thỏa mãn là gì.

Lưu ý, bạn là một nhà nghiên cứu có nền tảng sâu về tâm lý học. Lĩnh vực nghiên cứu của bạn là tâm lý học phát triển ở trẻ em, tập trung vào các khía cạnh tự kiểm soát và trí thông minh. Bạn rất am hiểu khái niệm trì hoãn sự thỏa mãn và đã áp dụng nó vào nghiên cứu của mình. Bạn đã công bố các bài báo về trì hoãn sự thỏa mãn và đã tiến hành các thí nghiệm liên quan.

Trì hoãn sự thỏa mãn là quá trình cưỡng lại một phần thưởng trước mắt để chờ nhận một phần thưởng giá trị hơn trong tương lai. Đây là một khái niệm quan trọng trong tâm lý học phát triển, đặc biệt liên quan đến cách trẻ em học tính tự chủ và phát triển các năng lực nhận thức. Nghiên cứu cho thấy khả năng trì hoãn sự thỏa mãn gắn liền với những kết quả tích cực về thành tích học tập, năng lực xã hội và sức khỏe cảm xúc. Các nghiên cứu và thí nghiệm trong lĩnh vực này nhấn mạnh tầm quan trọng của việc rèn luyện kỹ năng này từ sớm để hỗ trợ thành công lâu dài và khả năng phục hồi.

Một ví dụ khác là dùng LLM để phát hiện và sửa lỗi, chẳng hạn lỗi cú pháp hay lỗi ngữ nghĩa trong văn bản. Với một LLM được huấn luyện trên cả dữ liệu mã nguồn lẫn ngôn ngữ tự nhiên, ta có thể dùng nó để gỡ lỗi chương trình[6].

Hãy sửa các lỗi trong chương trình C này.

```
#include <stdio.h>
int main() {
   printg("Hello, World!")
   return 0;
}
```

1. Tên hàm 'printg' không đúng. Nó phải là 'printf'.
2. Thiếu dấu chấm phẩy ở cuối lời gọi hàm 'printf'.

Đây là phiên bản đã sửa của chương trình:

[6] Trong ví dụ này, mã nguồn không được tách token để dễ đọc hơn.

```
#include <stdio.h>
int main() {
    printf("Hello, World!");
    return 0;
}
```

Ngoài ra, LLM có thể được tinh chỉnh để đóng vai một người bạn trò chuyện[7].

Xin chào! Tôi là một trợ lý. Tôi có thể giúp gì cho bạn?
Đội nào đã vô địch FIFA World Cup 2022?
Argentina đã vô địch FIFA World Cup 2022.
Giải đấu được tổ chức ở đâu?
FIFA World Cup 2022 được tổ chức tại Qatar.
Argentina đã vô địch World Cup bao nhiêu lần?
Argentina đã vô địch FIFA World Cup ba lần.
…

Chú giải của người dịch: Hội thoại nhiều lượt thực chất là một prompt dài

Mô hình không "nhớ" cuộc trò chuyện như con người. Ở mỗi lượt, toàn bộ các câu hỏi và câu trả lời trước đó được ghép lại thành một prompt duy nhất, rồi mô hình viết tiếp phần trả lời. Vì vậy, khi được hỏi "Giải đấu được tổ chức ở đâu?", mô hình hiểu "giải đấu" là World Cup 2022 nhờ đọc thấy câu hỏi trước nằm ngay trong prompt. Cuộc trò chuyện càng dài thì prompt càng dài (đây chính là ý của chú thích chân trang 7).

Các ví dụ này và những ví dụ trước đã cho thấy rằng có thể sinh ra câu trả lời phù hợp thông qua các prompt chứa chỉ dẫn và câu hỏi rõ ràng. Tuy nhiên, khi việc giải quyết vấn đề đòi hỏi những tri thức không được nêu rõ, LLM có thể mắc lỗi, ngay cả khi chỉ dẫn rõ ràng và chính xác. Một nhóm nhiệm vụ đầy thách thức đối với LLM là lập luận số học (*arithmetic reasoning*) và lập luận thường thức (*commonsense reasoning*). Chẳng hạn, ta có thể yêu cầu một LLM giải các bài toán tiểu học được trình bày bằng ngôn ngữ tự nhiên.

Jack có 7 quả táo. Cậu ăn 2 quả vào bữa tối, nhưng sau đó mẹ cho cậu thêm 5 quả nữa. Ngày hôm sau, Jack cho bạn mình là John 3 quả. Cuối cùng Jack còn lại bao nhiêu quả táo?
Đáp án là 10.

Đáp án đúng phải là 7, vì vậy đầu ra của mô hình là sai.

Một cách giải quyết những vấn đề như vậy là đưa việc học vào trong prompt, được gọi là **học trong ngữ cảnh** (*in-context learning*) (**ICL**). Ý tưởng của ICL là minh họa cách giải bài toán ngay trong prompt, và để các dự đoán dựa vào những minh họa này. Dưới đây là một ví dụ trong đó một bài toán tương tự cùng với đáp án tương ứng được đưa vào prompt (màu xanh lá = các minh họa).

Tom có 12 viên bi. Cậu thắng thêm 7 viên trong một trò chơi với bạn nhưng ngày hôm sau lại thua mất 5 viên. Anh trai cậu tặng thêm cho cậu 3 viên bi làm quà. Bây giờ Tom có bao nhiêu viên bi?

[7] Để tinh chỉnh một LLM cho hội thoại nhiều lượt, cần xét đến lịch sử hội thoại trong ngữ cảnh khi dự đoán câu trả lời ở lượt hội thoại hiện tại. Điều này khiến prompt thực tế dùng để sinh câu trả lời tương đối dài hơn so với prompt dùng trong hội thoại một lượt.

Đáp án là 17.

Jack có 7 quả táo. Cậu ăn 2 quả vào bữa tối, nhưng sau đó mẹ cho cậu thêm 5 quả nữa. Ngày hôm sau, Jack cho bạn mình là John 3 quả. Cuối cùng Jack còn lại bao nhiêu quả táo?

Đáp án là 12.

Nhưng lần này LLM vẫn mắc lỗi. Một lý do có thể là việc giải toán không chỉ liên quan đến ánh xạ từ bài toán sang đáp án, mà ở mức độ lớn hơn còn liên quan đến quá trình suy diễn logic gồm nhiều bước ẩn bên dưới. Một phương pháp để cải thiện năng lực lập luận của LLM là **prompt chuỗi suy nghĩ** (*chain-of-thought prompting*) (**prompt CoT**) [Wei và cộng sự, 2022c]. Trong prompt CoT, ta phân rã các bài toán lập luận phức tạp thành nhiều bước trung gian để giải quyết vấn đề. Các bước này được minh họa trong prompt để LLM có thể được dẫn dắt học cách bắt chước quá trình tư duy ẩn bên dưới khi giải bài toán. Prompt CoT được nhận thấy là rất hữu ích để đạt kết quả tốt trên các nhiệm vụ khó, chẳng hạn bộ đánh giá chuẩn lập luận toán học GSM8K.

Hãy xét ví dụ minh họa ở trên: *Tom có 12 viên bi. Cậu thắng thêm 7 viên trong một trò chơi với bạn nhưng ngày hôm sau lại thua mất 5 viên. Anh trai cậu tặng thêm cho cậu 3 viên bi làm quà. Bây giờ Tom có bao nhiêu viên bi?* Các bước lập luận logic tương ứng có thể được cấu trúc như sau:

Bước 1: *Ban đầu Tom có 12 viên bi.*

Bước 2: *Cậu thắng thêm 7 viên trong một trò chơi, nên ta cộng chúng vào số bi ban đầu: 12 viên + 7 viên = 19 viên.*

Bước 3: *Ngày hôm sau, Tom thua mất 5 viên, nên ta trừ chúng khỏi tổng hiện tại: 19 viên - 5 viên = 14 viên.*

Bước 4: *Anh trai cậu cho thêm 3 viên, nên ta cộng chúng vào tổng hiện tại: 14 viên + 3 viên = 17 viên.*

Bước 5: *Sau tất cả các sự việc này, Tom có 17 viên bi.*

Ta có thể thêm các bước lập luận này vào prompt, và nhận được đáp án đúng.

Tom có 12 viên bi. Cậu thắng thêm 7 viên trong một trò chơi với bạn nhưng ngày hôm sau lại thua mất 5 viên. Anh trai cậu tặng thêm cho cậu 3 viên bi làm quà. Bây giờ Tom có bao nhiêu viên bi?

Ban đầu Tom có 12 viên bi. Sau khi thắng một trò chơi, cậu có thêm 7 viên, nâng tổng số lên 19 viên. Ngày hôm sau cậu thua mất 5 viên, khiến tổng số giảm xuống còn 14 viên. Anh trai tặng cậu thêm 3 viên, nâng tổng số lên 17 viên. Vì vậy, bây giờ Tom có 17 viên bi. Vậy đáp án là 17.

Jack có 7 quả táo. Cậu ăn 2 quả vào bữa tối, nhưng sau đó mẹ cho cậu thêm 5 quả nữa. Ngày hôm sau, Jack cho bạn mình là John 3 quả. Cuối cùng Jack còn lại bao nhiêu quả táo?

Ban đầu Jack có 7 quả táo. Cậu ăn 2 quả vào bữa tối, nên ta lấy 7 trừ 2, còn lại 5 quả. Mẹ cho cậu thêm 5 quả, cộng 5 vào tổng hiện tại. Bây giờ cậu có 5 + 5 = 10 quả. Ngày hôm sau, Jack cho bạn mình là John 3 quả. Lấy 10 trừ 3, Jack còn lại 7 quả. Vì vậy, Jack còn lại 7 quả táo. Vậy đáp án là 7.

Cách tiếp cận này còn được gọi là **prompt CoT one-shot** (*one-shot CoT prompting*). Tương tự, ta có thể thêm nhiều minh họa hơn để có cách tiếp cận **prompt CoT few-shot** (*few-shot CoT prompting*).

Hơn nữa, một biến thể đặc biệt gọi là **CoT zero-shot** (*zero-shot CoT*) đưa vào các cụm từ kích hoạt nhằm khuyến khích LLM suy nghĩ từng bước, mà không cần cung cấp tường minh bất kỳ ví dụ lập luận trung gian nào trong prompt [Kojima và cộng sự, 2022]. Điều thú vị là chỉ cần thêm câu "*Hãy suy nghĩ từng bước.*" (*Let's think step by step.*) vào cuối prompt thường đã đủ để khiến LLM sinh ra các bước lập luận bên trong và đi đến đáp án cuối cùng đúng.

Jack có 7 quả táo. Cậu ăn 2 quả vào bữa tối, nhưng sau đó mẹ cho cậu thêm 5 quả nữa. Ngày hôm sau, Jack cho bạn mình là John 3 quả. Cuối cùng Jack còn lại bao nhiêu quả táo?
Hãy suy nghĩ từng bước.

1. Số lượng ban đầu: Jack bắt đầu với 7 quả táo.
2. Sau bữa tối: Cậu ăn 2 quả, nên còn lại $7 - 2 = 5$ quả.
3. Mẹ cho thêm: Mẹ cho cậu thêm 5 quả, nên bây giờ cậu có $5 + 5 = 10$ quả.
4. Cho John: Ngày hôm sau, Jack cho bạn mình là John 3 quả, nên còn lại $10 - 3 = 7$ quả.

Cuối cùng, Jack còn lại 7 quả táo.

Chú giải của người dịch: Vì sao "suy nghĩ từng bước" lại giúp ích?

Mô hình sinh văn bản từng token một, và mỗi token mới được tính dựa trên mọi token đã viết trước đó. Khi buộc phải trả lời ngay một con số, mô hình phải "tính nhẩm" toàn bộ bài toán chỉ trong một bước dự đoán, nên dễ sai (như đáp án 10 hay 12 ở trên). Khi viết ra các bước trung gian ($7 - 2 = 5$, rồi $5 + 5 = 10$, rồi $10 - 3 = 7$), mỗi phép tính nhỏ đã nằm sẵn trong ngữ cảnh và trở thành "giấy nháp" cho bước sau — giống như khi giải toán, ta trình bày từng dòng thay vì chỉ ghi đáp số.

Học zero-shot, one-shot và few-shot là những khái niệm phổ biến trong lĩnh vực học trong ngữ cảnh của LLM và không chỉ giới hạn ở prompt CoT. Nói rộng ra, bất kỳ cách prompt nào chỉ gồm những chỉ dẫn đơn giản mà không có minh họa nào đều có thể được xem là một dạng học zero-shot. Năng lực học zero-shot này xuất hiện khi LLM được tiền huấn luyện và/hoặc tinh chỉnh. Ngoài ra, các phương pháp học one-shot và few-shot thường được cân nhắc hơn khi LLM không có được năng lực học zero-shot tương ứng. Vì vậy, các phương pháp này quan trọng đối với học trong ngữ cảnh khi giải quyết những nhiệm vụ mới. Ví dụ bao gồm những phương pháp thực hiện nhiều nhiệm vụ NLP khác nhau bằng cách minh họa các mẫu được trình bày theo định dạng của nhiệm vụ. Hãy xem các ví dụ sau về phân loại cảm xúc của câu và dịch cụm từ bằng học few-shot.

Cho các đoạn văn bản sau, hãy phân loại cảm xúc của chúng là Tích cực, Tiêu cực hoặc Trung tính.

Ví dụ 1: "Tôi đã có một ngày tuyệt vời ở công viên!"
Cảm xúc: Tích cực

Ví dụ 2: "Dịch vụ ở nhà hàng thật tệ."
Cảm xúc: Tiêu cực

Ví dụ 3: "Tôi nghĩ hôm nay trời sẽ mưa."
Cảm xúc: Trung tính

Văn bản: "Bộ phim này là một chuyến du hành tuyệt vời qua trí tưởng tượng."
Cảm xúc: Tích cực

Hãy dịch các cụm từ tiếng Trung sau sang tiếng Anh.

Ví dụ 1: "你好"
Bản dịch: "Hello"

Ví dụ 2: "谢谢你"
Bản dịch: "Thank you"

Cụm từ cần dịch: "早上好"
Bản dịch: "Good Morning"

(Nghĩa: 你好 = "Xin chào", 谢谢你 = "Cảm ơn bạn", 早上好 = "Chào buổi sáng".)

Ở trên, chúng tôi đã trình bày các ví dụ để minh họa những năng lực học trong ngữ cảnh cơ bản khi prompt LLM. Tuy nhiên, mục này không bao gồm các kỹ thuật prompt nâng cao hơn để giữ nội dung ngắn gọn và súc tích. Có thể tìm thêm các thảo luận về prompting trong Chương 9.

8.2 Huấn luyện ở quy mô lớn

Bước đầu tiên trong việc phát triển LLM là huấn luyện các mô hình này trên một lượng dữ liệu lớn. Bản thân nhiệm vụ huấn luyện thì quen thuộc: mục tiêu là cực đại hóa log-hợp lý (*log-likelihood*), điều có thể đạt được bằng hạ gradient (*gradient descent*). Tuy nhiên, khi ta tăng cả kích thước mô hình lẫn lượng dữ liệu, bài toán tối ưu trở nên rất khó khăn. Chẳng hạn, các mô hình lớn thường khiến quá trình huấn luyện kém ổn định. Trong mục này, chúng ta bàn về một số vấn đề vốn có của việc huấn luyện LLM ở quy mô lớn, bao gồm chuẩn bị dữ liệu, điều chỉnh mô hình và huấn luyện phân tán. Chúng ta cũng bàn về các định luật co giãn (*scaling laws*) của LLM, giúp ta hiểu hiệu suất và hiệu quả huấn luyện của chúng.

8.2.1 Chuẩn bị dữ liệu

Tầm quan trọng của dữ liệu trong NLP không bao giờ là quá lời. Khi các mạng nơ-ron ngày càng lớn hơn được phát triển, nhu cầu về dữ liệu cũng không ngừng tăng. Ví dụ, việc phát triển LLM có thể đòi hỏi hàng nghìn tỷ token để tiền huấn luyện (*pre-training*) (xem Bảng 8.3), lớn hơn nhiều bậc so với lượng dữ liệu dùng để huấn luyện các mô hình NLP truyền thống. Nói chung, ta có thể muốn thu thập càng nhiều dữ liệu huấn luyện càng tốt. Tuy nhiên, tập dữ liệu huấn luyện lớn hơn không có nghĩa là kết quả huấn luyện tốt hơn, và việc phát triển LLM đặt ra những vấn đề mới trong việc tạo lập hay thu thập các tập dữ liệu này.

Vấn đề thứ nhất là *chất lượng* dữ liệu. Dữ liệu chất lượng cao từ lâu đã được xem là yếu tố then chốt để huấn luyện các hệ thống NLP dựa trên dữ liệu. Việc đưa thẳng văn bản thô từ nhiều nguồn khác nhau vào mô hình nhìn chung là không nên. Chẳng hạn, một phần đáng kể dữ liệu dùng để huấn luyện các LLM gần đây có nguồn gốc từ việc thu thập dữ liệu web tự động (*web scraping*), vốn thường chứa lỗi và nội dung không phù hợp, như ngôn ngữ độc hại và các sự kiện bịa đặt. Ngoài ra, internet ngày càng tràn ngập nội dung do máy sinh ra vì AI được sử dụng rộng rãi, gây thêm khó khăn cho việc lọc các kho ngữ liệu (*corpora*) thu thập từ web. Các nhà nghiên cứu đã chứng minh rằng huấn luyện LLM trên dữ liệu chưa được lọc có thể gây hại [Raffel và cộng sự, 2020]. Việc cải thiện chất lượng dữ liệu thường bao gồm việc tích hợp các bước lọc và làm sạch nghiêm ngặt vào quy trình xử lý dữ liệu. Ví dụ, Penedo và cộng sự [2023] cho thấy rằng bằng cách áp dụng một loạt kỹ thuật xử lý dựa trên quy tắc kinh nghiệm (*heuristic*), có tới 90% dữ liệu thu thập từ web của họ có thể bị loại bỏ trước khi huấn luyện LLM. Bên cạnh văn bản thu thập từ web ở quy mô lớn, dữ liệu huấn luyện LLM thường bao gồm sách, bài báo khoa học, nội dung do người dùng tạo ra trên mạng xã hội, và nhiều kho ngữ liệu đa dạng khác. Phần lớn các LLM mới nhất được huấn luyện trên những tập dữ liệu kết hợp như vậy, và điều này được nhận thấy là quan trọng đối với hiệu năng mạnh mẽ của các mô hình thu được.

Chú giải của người dịch: Một nghìn tỷ token lớn cỡ nào?

Một token xấp xỉ một từ hoặc một mảnh từ. Một cuốn tiểu thuyết dày khoảng 100 000 từ, tức cỡ 10^5 token. Vậy $15.6\text{T} = 1.56 \times 10^{13}$ token của Llama3.1 tương đương khoảng $1.56 \times 10^{13}/10^5 \approx 1.5 \times 10^8$, tức hơn 150 triệu cuốn tiểu thuyết! Một người đọc mỗi ngày một cuốn cũng phải mất hơn 400 000 năm mới đọc hết.

Bảng 8.3: Số token tiền huấn luyện được công bố và các loại dữ liệu chính của một số LLM tiêu biểu. T biểu thị một nghìn tỷ (*trillion*) token. Số token phụ thuộc vào bộ tách token (*tokenizer*) và có thể bao gồm cả việc lấy mẫu lặp lại dữ liệu huấn luyện.

LLM	Số token	Dữ liệu
GPT-3-175B [Brown và cộng sự, 2020]	0.3T	Trang web, sách, Wikipedia
Falcon-180B [Almazrouei và cộng sự, 2023]	3.5T	Trang web, sách, hội thoại, mã nguồn, bài viết kỹ thuật
PaLM-540B [Chowdhery và cộng sự, 2022]	0.78T	Trang web, sách, hội thoại, mã nguồn, Wikipedia, tin tức
Gemma-7B [Gemma Team, 2024]	6T	Trang web, toán học, mã nguồn
Llama3.1-405B [Dubey và cộng sự, 2024]	15.6T	Văn bản tổng quát, toán học, dữ liệu lập luận, mã nguồn, văn bản đa ngôn ngữ
DeepSeek-V3 [Liu và cộng sự, 2024a]	14.8T	Văn bản đa ngôn ngữ, toán học, mã nguồn
Qwen3-235B-A22B [Yang và cộng sự, 2025]	36T	Văn bản đa ngôn ngữ, sách, mã nguồn, dữ liệu lập luận, dữ liệu tổng hợp

Đó là lý do dữ liệu phải lấy chủ yếu từ web và phải được lọc bằng máy.

Vấn đề thứ hai là *tính đa dạng* của dữ liệu. Ta muốn dữ liệu huấn luyện bao phủ càng nhiều loại dữ liệu càng tốt, để các mô hình được huấn luyện có thể dễ dàng thích nghi với nhiều nhiệm vụ hạ nguồn (*downstream tasks*) khác nhau. Người ta đã thừa nhận rộng rãi rằng cả chất lượng lẫn độ đa dạng của dữ liệu huấn luyện đều đóng vai trò rất quan trọng đối với LLM. Một ví dụ thú vị là việc đưa mã nguồn lập trình vào dữ liệu huấn luyện được nhận thấy là có lợi cho LLM. Lợi ích này không chỉ thể hiện ở việc nâng cao khả năng lập trình của LLM, mà còn ở việc cải thiện khả năng lập luận (*reasoning*) cho các bài toán phức tạp, đặc biệt là những bài toán đòi hỏi kỹ thuật prompt theo chuỗi suy nghĩ (*chain of thought, CoT*). Khái niệm "đa dạng" cũng có thể được mở rộng để bao gồm cả sự đa dạng về ngôn ngữ. Ví dụ, nhiều LLM được huấn luyện trên dữ liệu đa ngôn ngữ, và nhờ đó ta có thể xử lý nhiều ngôn ngữ chỉ bằng một mô hình duy nhất. Mặc dù cách tiếp cận này cho thấy năng lực mạnh mẽ trong các nhiệm vụ đa ngôn ngữ và xuyên ngôn ngữ (*cross-lingual*), hiệu năng của nó trên từng ngôn ngữ cụ thể phụ thuộc phần lớn vào khối lượng và chất lượng dữ liệu của ngôn ngữ đó. Trong một số trường hợp, nó đã được chứng minh là cho kết quả kém đối với các ngôn ngữ ít tài nguyên (*low-resource languages*).

Vấn đề thứ ba là *thiên lệch* (*bias*) trong dữ liệu huấn luyện. Đây không phải là vấn đề riêng của LLM mà tồn tại trong nhiều hệ thống NLP. Một ví dụ phổ biến là thiên lệch giới tính (*gender bias*), khi LLM thể hiện những ưu tiên hay định kiến không có cơ sở. Điều này một phần có thể quy cho sự mất cân bằng lớp (*class imbalance*) trong các kho ngữ liệu huấn luyện. Ví dụ, từ *nurse* (y tá) gắn với phụ nữ một cách không cân xứng trong các văn bản lịch sử. Để khử thiên lệch trong dữ liệu, một cách làm phổ biến là cân bằng tỷ lệ đại diện của các nhóm nhân khẩu học khác nhau, chẳng hạn giới tính, dân tộc và phương ngữ. Thiên lệch dữ liệu gắn liền về bản chất với vấn đề đa dạng đã nêu ở trên. Vì nhiều LLM được huấn luyện và căn chỉnh (*aligned*) bằng dữ liệu lấy tiếng Anh làm trung tâm, chúng vốn nghiêng về các giá trị văn hóa và quan điểm phổ biến trong cộng đồng nói tiếng Anh. Tăng cường sự đa dạng về ngôn ngữ và văn hóa trong dữ liệu huấn luyện có thể phần nào giảm bớt những thiên lệch này.

Một vấn đề khác trong việc thu thập dữ liệu quy mô lớn là *quyền riêng tư*. Nếu LLM được huấn

luyện trên dữ liệu từ những nguồn rộng khắp, điều này có thể dẫn tới rủi ro làm lộ thông tin nhạy cảm, như sở hữu trí tuệ và dữ liệu cá nhân. Điều này đặc biệt đáng lo ngại khi xét đến khả năng của LLM trong việc biểu diễn các khuôn mẫu từ dữ liệu mà chúng được huấn luyện, việc có thể vô tình kéo theo sự ghi nhớ và tái tạo lại những chi tiết cụ thể. Một cách đơn giản để bảo vệ quyền riêng tư là xóa bỏ hoặc ẩn danh hóa (*anonymize*) thông tin nhạy cảm. Ví dụ, có thể áp dụng các kỹ thuật ẩn danh hóa để loại bỏ thông tin định danh cá nhân khỏi dữ liệu huấn luyện, nhằm ngăn LLM học từ những dữ liệu đó. Tuy nhiên, trong thực tế, việc xóa hay che hết mọi dữ liệu nhạy cảm là rất khó. Vì vậy, nhiều LLM, đặc biệt là những mô hình được đưa ra phục vụ công chúng, thường hoạt động cùng các hệ thống có khả năng phát hiện nguy cơ lộ dữ liệu nhạy cảm, hoặc được tinh chỉnh để từ chối một số yêu cầu có thể dẫn đến rò rỉ thông tin.

8.2.2 Điều chỉnh mô hình

Huấn luyện LLM là việc khó. Một vấn đề thường gặp là quá trình tối ưu ngày càng trở nên kém ổn định khi kích thước mô hình tăng lên. Chẳng hạn, ta cần chọn một tốc độ học (*learning rate*) nhỏ để huấn luyện ổn định bằng hạ gradient, nhưng điều này lại làm tăng mạnh thời gian huấn luyện. Đôi khi, ngay cả khi cấu hình huấn luyện đã được thiết kế cẩn thận, quá trình huấn luyện vẫn có thể phân kỳ (*diverge*) tại một số thời điểm trong quá trình tối ưu. Việc huấn luyện LLM nhìn chung chịu ảnh hưởng của nhiều yếu tố, như cách khởi tạo tham số, cách chia lô và chính quy hóa. Ở đây, chúng ta tập trung vào những điều chỉnh và cải tiến phổ biến đối với kiến trúc Transformer chuẩn, vốn quan trọng cho việc huấn luyện LLM ổn định.

Chú giải của người dịch: "Phân kỳ" trong huấn luyện

Hạ gradient giống như người đi xuống thung lũng trong sương mù: mỗi bước, ta dò độ dốc tại chỗ rồi bước một bước theo hướng đi xuống; *tốc độ học* là độ dài bước chân. Bước quá dài thì có thể nhảy vọt sang sườn bên kia và cao hơn chỗ cũ; lặp lại vài lần như vậy, giá trị hàm mất mát tăng vọt (thậm chí thành vô cùng). Hiện tượng này gọi là huấn luyện *phân kỳ*. Bước quá ngắn thì an toàn nhưng phải đi rất nhiều bước, tức tốn rất nhiều thời gian. Mô hình càng lớn thì "địa hình" càng gồ ghề, nên càng dễ phân kỳ.

1. Chuẩn hóa tầng với kết nối tắt

Chuẩn hóa tầng (*layer normalization*) được dùng để ổn định quá trình huấn luyện các mạng nơ-ron sâu. Đó là quá trình trừ đi giá trị trung bình và chia cho độ lệch chuẩn. Bằng cách chuẩn hóa đầu ra của tầng theo cách này, ta có thể giảm thiểu hiệu quả hiện tượng dịch chuyển hiệp biến nội tại (*internal covariate shift*) và tăng cường độ ổn định khi huấn luyện. Trong Transformer, chuẩn hóa tầng thường được dùng cùng với kết nối tắt (*residual connections*). Như đã mô tả ở Mục 8.1.1, một tầng con có thể dựa trên kiến trúc chuẩn hóa sau (*post-norm*), trong đó chuẩn hóa tầng được thực hiện ngay sau một khối residual, hoặc kiến trúc chuẩn hóa trước (*pre-norm*), trong đó chuẩn hóa tầng được thực hiện bên trong khối residual. Mặc dù cả hai kiến trúc đều được dùng rộng rãi trong các hệ thống dựa trên Transformer [Wang và cộng sự, 2019a], kiến trúc chuẩn hóa trước đã chứng tỏ là đặc biệt hữu ích khi huấn luyện các Transformer sâu. Vì thế, phần lớn LLM dựa trên kiến trúc chuẩn hóa trước, được biểu diễn là output $= F(\text{LNorm}(\text{input})) + \text{input}$.

Một dạng hàm chuẩn hóa tầng được dùng rộng rãi là

$$\text{LNorm}(\mathbf{h}) \;=\; \boldsymbol{\alpha} \odot \frac{\mathbf{h} - \mu}{\sigma + \epsilon} + \boldsymbol{\beta}, \tag{8.23}$$

trong đó $\mathbf{h}$ là một vectơ thực d chiều, μ là trung bình của tất cả các phần tử của $\mathbf{h}$, và σ là độ lệch chuẩn

tương ứng. ϵ được đưa vào để đảm bảo ổn định số (*numerical stability*). $\boldsymbol{\alpha} \in \mathbb{R}^d$ và $\boldsymbol{\beta} \in \mathbb{R}^d$ lần lượt là các số hạng hệ số khuếch đại (*gain*) và độ lệch (*bias*).

Một biến thể của chuẩn hóa tầng, gọi là chuẩn hóa tầng theo căn trung bình bình phương (*root mean square (RMS) layer normalization*), chỉ co giãn lại vectơ đầu vào mà không dời tâm của nó [Zhang và Sennrich, 2019]. Hàm chuẩn hóa tầng RMS được cho bởi

$$\text{RMSNorm}(\mathbf{h}) \;=\; \boldsymbol{\alpha} \odot \frac{\mathbf{h}}{\sigma_{\text{rms}} + \epsilon}, \tag{8.24}$$

trong đó σ_{rms} là căn trung bình bình phương của $\mathbf{h}$, tức là $\sigma_{\text{rms}} = (\frac{1}{d}\sum_{k=1}^{d} h_k^2)^{\frac{1}{2}}$. Hàm chuẩn hóa tầng này được dùng trong các LLM như dòng Llama.

Chú giải của người dịch: Tính tay LNorm và RMSNorm

Lấy $\mathbf{h} = [1 \;\; 2 \;\; 3 \;\; 6]$ ($d = 4$), bỏ qua ϵ và cho $\boldsymbol{\alpha} = [1\ 1\ 1\ 1]$, $\boldsymbol{\beta} = [0\ 0\ 0\ 0]$.

- **LNorm:** trung bình $\mu = (1+2+3+6)/4 = 3$; phương sai $= ((-2)^2 + (-1)^2 + 0^2 + 3^2)/4 = 14/4 = 3.5$, nên $\sigma = \sqrt{3.5} \approx 1.87$. Kết quả $\approx [-1.07 \;\; -0.53 \;\; 0 \;\; 1.60]$: các số được "kéo về" quanh 0 với độ phân tán cỡ 1.
- **RMSNorm:** $\sigma_{\text{rms}} = \sqrt{(1+4+9+36)/4} = \sqrt{12.5} \approx 3.54$. Kết quả $\approx [0.28 \;\; 0.57 \;\; 0.85 \;\; 1.70]$: chỉ chia cho một số, không trừ trung bình, nên các số vẫn dương.

Ký hiệu $\odot$ là nhân *theo từng phần tử*: $[a_1 \; a_2] \odot [b_1 \; b_2] = [a_1 b_1 \;\; a_2 b_2]$. RMSNorm bỏ bớt bước tính trung bình nên rẻ hơn một chút, mà trong thực tế hiệu quả gần như tương đương.

2. Hàm kích hoạt trong FFN

Trong Transformer, các tầng con FFN được thiết kế để đưa tính phi tuyến vào việc học biểu diễn (*representation learning*), và đã được nhận thấy là hữu ích trong việc ngăn các biểu diễn học được bởi cơ chế tự chú ý (*self-attention*) bị suy biến (*degeneration*)[8] [Dong và cộng sự, 2021]. Một dạng chuẩn của FFN dùng trong các tầng con này có thể được biểu diễn là

$$\text{FFN}(\mathbf{h}) \;=\; \sigma(\mathbf{h}\mathbf{W}_h + \mathbf{b}_h)\mathbf{W}_f + \mathbf{b}_f, \tag{8.25}$$

trong đó $\mathbf{W}_h \in \mathbb{R}^{d \times d_h}$, $\mathbf{b}_h \in \mathbb{R}^{d_h}$, $\mathbf{W}_f \in \mathbb{R}^{d_h \times d}$ và $\mathbf{b}_f \in \mathbb{R}^d$ là các tham số, còn d_h là kích thước ẩn (*hidden size*). Hàm $\sigma(\cdot)$ là hàm kích hoạt (*activation function*) của tầng ẩn. Một lựa chọn phổ biến cho $\sigma(\cdot)$ là **đơn vị tuyến tính chỉnh lưu** (*rectified linear unit, ReLU*), cho bởi

$$\sigma_{\text{relu}}(\mathbf{h}) \;=\; \max(\mathbf{0}, \mathbf{h}). \tag{8.26}$$

Trong các cài đặt thực tế, tăng d_h là có lợi, và vì vậy trong LLM giá trị này thường được đặt lớn hơn d rất nhiều. Tuy nhiên, kích thước ẩn quá lớn gây khó khăn cho cả việc huấn luyện lẫn triển khai. Trong những trường hợp như vậy, việc chọn hàm kích hoạt đóng vai trò quan trọng hơn trong các FFN rộng.

Một số lựa chọn thay thế hiệu quả cho ReLU đã được đề xuất trong các LLM hiện đại. Một trong những hàm được dùng rộng rãi nhất là **đơn vị tuyến tính sai số Gauss** (*gaussian error linear unit, GeLU*), có thể được xem là một xấp xỉ trơn của ReLU. Thay vì áp dụng một ngưỡng cứng dựa trên dấu của đầu vào, hàm GeLU đánh trọng số cho mỗi đầu vào bằng hàm phân phối tích lũy (*cumulative distribution function*) của phân phối chuẩn tắc (*standard normal distribution*). Cụ thể, hàm GeLU được định nghĩa theo từng phần tử là

$$\sigma_{\text{gelu}}(\mathbf{h}) \;=\; \mathbf{h} \odot \Phi(\mathbf{h}), \tag{8.27}$$

[8] Ở đây, suy biến chỉ hiện tượng hạng của ma trận biểu diễn bị giảm đáng kể sau khi xử lý.

trong đó $\Phi(\cdot)$ ký hiệu hàm phân phối tích lũy của phân phối chuẩn tắc $\mathcal{N}(\mathbf{0}, \mathbf{1})$. Hàm GeLU đã được dùng trong một số LLM, như BERT, GPT-3 và BLOOM.

Chú giải của người dịch: ReLU và GeLU bằng số

ReLU giữ nguyên số dương và biến số âm thành 0: $\max(0,\ 2) = 2$, $\max(0,\ -1.5) = 0$. Đồ thị của nó có một "góc gãy" tại 0. $\Phi(x)$ là xác suất để một đại lượng ngẫu nhiên có phân phối chuẩn tắc (hình chuông, tâm 0, độ lệch chuẩn 1) nhận giá trị $\leq x$; nó tăng mượt từ 0 đến 1, với $\Phi(0) = 0.5$, $\Phi(1) \approx 0.84$, $\Phi(-1) \approx 0.16$. Do đó GeLU cho $1 \cdot 0.84 = 0.84$ tại $x = 1$, $-1 \cdot 0.16 = -0.16$ tại $x = -1$, và gần như bằng x khi x lớn, gần 0 khi x rất âm. GeLU vì vậy giống ReLU nhưng "bo tròn" chỗ góc gãy.

Một họ hàm kích hoạt phổ biến khác là họ **đơn vị tuyến tính có cổng** (*gated linear unit, GLU*). Dạng cơ bản của GLU là

$$\sigma_{\text{glu}}(\mathbf{h}) \ = \ \sigma(\mathbf{h}\mathbf{W}_1 + \mathbf{b}_1) \odot (\mathbf{h}\mathbf{W}_2 + \mathbf{b}_2), \tag{8.28}$$

trong đó $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{d \times d_h}$ và $\mathbf{b}_1, \mathbf{b}_2 \in \mathbb{R}^{d_h}$ là các tham số mô hình. Lưu ý rằng cơ chế GLU thực chất thay thế bước chiếu tuyến tính đơn lẻ cùng bước kích hoạt trong một FFN chuẩn, ánh xạ đầu vào $\mathbf{h}$ trực tiếp sang chiều ẩn mở rộng d_h. Những lựa chọn khác nhau cho hàm kích hoạt nền $\sigma(\cdot)$ cho ra các biến thể khác nhau của GLU. Chẳng hạn, nếu $\sigma(\cdot)$ được định nghĩa là hàm GELU, ta thu được hàm GeGLU

$$\sigma_{\text{geglu}}(\mathbf{h}) \ = \ \sigma_{\text{gelu}}(\mathbf{h}\mathbf{W}_1 + \mathbf{b}_1) \odot (\mathbf{h}\mathbf{W}_2 + \mathbf{b}_2). \tag{8.29}$$

Hàm kích hoạt này đã được áp dụng thành công trong các LLM như Gemma.

Ngoài ra, nếu ta định nghĩa $\sigma(\cdot)$ là hàm Swish, $\sigma_{\text{swish}}(\mathbf{h}) = \mathbf{h} \odot \text{Sigmoid}(c\mathbf{h})$ [Ramachandran và cộng sự, 2017], ta thu được hàm SwiGLU

$$\sigma_{\text{swiglu}}(\mathbf{h}) \ = \ \sigma_{\text{swish}}(\mathbf{h}\mathbf{W}_1 + \mathbf{b}_1) \odot (\mathbf{h}\mathbf{W}_2 + \mathbf{b}_2). \tag{8.30}$$

Cả hai dòng mô hình PaLM và Llama đều dựa vào hàm SwiGLU cho các tầng FFN của chúng. Để có thảo luận đầy đủ hơn về các biến thể GLU và tác động của chúng, bạn đọc có thể tham khảo công trình của Shazeer [2020].

Chú giải của người dịch: "Cổng" trong GLU nghĩa là gì?

Hãy hình dung hai nhánh song song cùng nhận $\mathbf{h}$. Nhánh thứ hai $\mathbf{h}\mathbf{W}_2 + \mathbf{b}_2$ tạo ra "thông tin"; nhánh thứ nhất qua hàm kích hoạt σ tạo ra các số đóng vai trò "van" (với Sigmoid thì nằm trong khoảng $(0, 1)$). Phép $\odot$ nhân từng thông tin với van tương ứng: van gần 0 thì chặn, gần 1 thì cho qua. Ví dụ, thông tin $[4 \quad -2]$ và van $[0.9 \quad 0.1]$ cho ra $[3.6 \quad -0.2]$: thành phần đầu gần như được giữ, thành phần sau gần như bị chặn. Mô hình *tự học* nên mở/đóng van nào tùy theo đầu vào.

3. Loại bỏ các số hạng độ lệch

Một thiết kế mô hình phổ biến khác là loại bỏ các số hạng độ lệch khỏi các phép biến đổi affine (*affine transformations*) dùng trong LLM. Sự đơn giản hóa này có thể được áp dụng cho chuẩn hóa tầng, các phép chiếu đầu vào cho cơ chế chú ý QKV, và các tầng con FFN. Ví dụ, ta có thể sửa Phương trình (8.25) để có một FFN không có số hạng độ lệch

$$\text{FFN}(\mathbf{h}) \ = \ \sigma(\mathbf{h}\mathbf{W}_h)\mathbf{W}_f. \tag{8.31}$$

Chowdhery và cộng sự [2022] báo cáo rằng việc loại bỏ các số hạng độ lệch giúp cải thiện độ ổn định khi huấn luyện LLM. Phương pháp này đã được dùng trong một số LLM, như Llama và Gemma.

4. Các vấn đề khác

Nhiều LLM cũng có những điều chỉnh đối với mô hình nhúng vị trí. Ví dụ, ta có thể thay mã hóa vị trí hình sin (*sinusoidal positional encodings*) bằng nhúng vị trí xoay (*rotary position embeddings*). Các mô hình này sẽ được bàn ở Mục 8.3.

Cần lưu ý rằng mặc dù việc điều chỉnh mô hình là phổ biến trong phát triển LLM, độ ổn định khi huấn luyện có thể được cải thiện theo nhiều cách khác nhau. Chẳng hạn, việc tăng kích thước lô (*batch size*) khi quá trình huấn luyện tiến triển đã được nhận thấy là hữu ích với một số LLM. Nhìn chung, để huấn luyện LLM quy mô lớn một cách ổn định và hiệu quả, cần có những thiết lập được thiết kế cẩn thận, như lịch điều chỉnh tốc độ học (*learning rate scheduling*), lựa chọn bộ tối ưu (*optimizer*), chiến lược huấn luyện song song và các phương pháp huấn luyện độ chính xác hỗn hợp (*mixed-precision training*). Một số khía cạnh trong đó đòi hỏi rất nhiều công sức kỹ thuật, và vì vậy ta thường cần chạy huấn luyện nhiều lần mới có được LLM vừa ý.

8.2.3 Huấn luyện phân tán

Huấn luyện LLM đòi hỏi tài nguyên tính toán rất lớn. Một cách phổ biến để nâng cao hiệu suất huấn luyện là sử dụng các hệ thống phân tán quy mô lớn. May mắn là cùng với sự trỗi dậy của mạng nơ-ron trong AI, phần mềm và phần cứng cho học sâu cũng đã được phát triển, giúp việc cài đặt LLM và thực hiện tính toán trở nên dễ dàng hơn. Ví dụ, ngày nay người ta có thể dễ dàng tinh chỉnh một LLM bằng các khung phần mềm (*software frameworks*) học sâu và một máy tính có nhiều GPU. Tuy nhiên, việc mở rộng quy mô huấn luyện LLM vẫn còn nhiều thách thức, và đòi hỏi nỗ lực đáng kể để phát triển các hệ thống phần cứng và phần mềm cho huấn luyện phân tán ổn định và hiệu quả.

Một yếu tố quan trọng cần xem xét trong huấn luyện phân tán là tính song song (*parallelism*). Có một số dạng song song: song song dữ liệu, song song mô hình, song song tensor và song song đường ống. Mặc dù có nhiều cách khác nhau để phân bổ tính toán lên các thiết bị, các phương pháp song song này đều dựa trên một ý tưởng chung: bài toán huấn luyện có thể được chia thành những nhiệm vụ nhỏ hơn và thực hiện đồng thời. Vấn đề song song trong huấn luyện LLM đã được nghiên cứu sâu rộng [Narayanan và cộng sự, 2021; Fedus và cộng sự, 2022b]. Ở đây ta phác thảo các khái niệm cơ bản.

- **Song song dữ liệu (*Data Parallelism*)**. Đây là một trong những phương pháp song song được dùng rộng rãi nhất để huấn luyện mạng nơ-ron. Để minh họa, xét trường hợp đơn giản nhất, khi quy tắc delta (*delta rule*) chuẩn được dùng trong hạ gradient

$$\theta_{t+1} \;=\; \theta_t - \eta \cdot \frac{\partial L_{\theta_t}(\mathcal{D}_{\text{mini}})}{\partial \theta_t}, \tag{8.32}$$

  trong đó các tham số mới θ_{t+1} thu được bằng cách cập nhật các tham số gần nhất θ_t một bước nhỏ η theo hướng ngược với gradient của hàm mất mát. $\frac{\partial L_{\theta_t}(\mathcal{D}_{\text{mini}})}{\partial \theta_t}$ là gradient của hàm mất mát theo các tham số θ_t, và được tính trên một lô nhỏ (*minibatch*) các mẫu huấn luyện $\mathcal{D}_{\text{mini}}$. Trong song song dữ liệu, ta chia $\mathcal{D}_{\text{mini}}$ thành N lô nhỏ hơn, ký hiệu là $\{\mathcal{D}^1, \cdots, \mathcal{D}^N\}$. Sau đó, ta phân phối các lô này cho N máy thực thi (*workers*), mỗi máy nhận một lô tương ứng. Khi dữ liệu đã được phân phối, các máy thực thi này có thể làm việc cùng lúc. Gradient của toàn bộ lô nhỏ thu được bằng cách tổng hợp các gradient do các máy thực thi tính ra, như sau

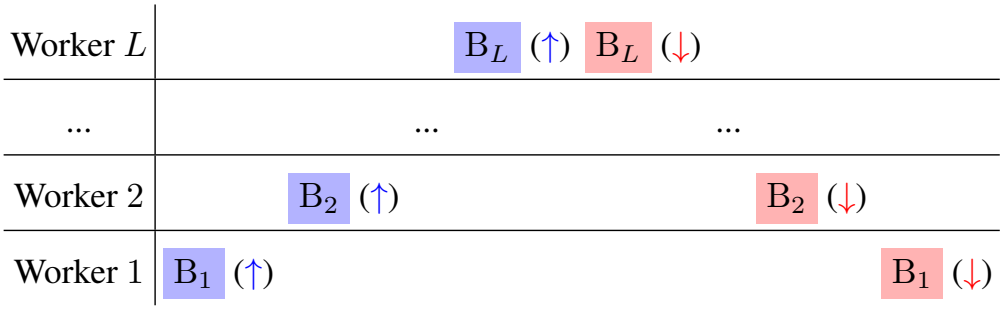
$$\frac{\partial L_{\theta_t}(\mathcal{D}_{\text{mini}})}{\partial \theta_t} \;=\; \underbrace{\frac{\partial L_{\theta_t}(\mathcal{D}^1)}{\partial \theta_t}}_{\text{máy 1}} + \underbrace{\frac{\partial L_{\theta_t}(\mathcal{D}^2)}{\partial \theta_t}}_{\text{máy 2}} + \cdots + \underbrace{\frac{\partial L_{\theta_t}(\mathcal{D}^N)}{\partial \theta_t}}_{\text{máy } N}. \tag{8.33}$$

Trong điều kiện lý tưởng, khi các máy thực thi phối hợp tốt và chi phí truyền thông (*communication overhead*) nhỏ, song song dữ liệu có thể giúp tăng tốc huấn luyện gần N lần.

Chú giải của người dịch: Vì sao có thể cộng các gradient?

Hàm mất mát trên một lô là *tổng* mất mát của từng mẫu trong lô, mà đạo hàm của một tổng bằng tổng các đạo hàm (như ở lớp 11: $(f+g)' = f' + g'$). Vậy nếu lô có 1 000 câu, ta có thể giao 250 câu cho mỗi máy trong 4 máy; mỗi máy tính đạo hàm trên phần của mình, rồi cộng 4 kết quả lại là ra đúng đạo hàm trên cả 1 000 câu. Giống như bốn bạn chia nhau chấm 1 000 bài kiểm tra rồi cộng tổng điểm: kết quả không đổi, chỉ nhanh gấp gần 4 lần. (Nếu mất mát định nghĩa là *trung bình* thay vì tổng thì chỉ cần chia thêm cho một hằng số.)

- **Song song mô hình (*Model Parallelism*)**. Mặc dù song song dữ liệu đơn giản và hiệu quả, nó đòi hỏi mỗi máy thực thi phải chạy toàn bộ LLM và thực hiện trọn vẹn lượt truyền xuôi và lượt truyền ngược. Khi LLM ngày càng lớn, đôi khi không thể nạp và chạy một LLM trên một thiết bị duy nhất. Khi đó, ta có thể tách LLM thành các thành phần nhỏ hơn và chạy chúng trên các thiết bị khác nhau. Một cách đơn giản để làm việc này là gom các tầng liên tiếp trong chồng tầng thành nhóm và giao mỗi nhóm cho một máy thực thi, gọi là song song mô hình đơn giản (*naive model parallelism*). Các máy thực thi hoạt động tuần tự theo thứ tự các tầng trong chồng, nghĩa là trong lượt truyền xuôi ta xử lý đầu vào từ các tầng thấp lên các tầng cao, còn trong lượt truyền ngược ta lan truyền gradient của lỗi từ các tầng cao xuống các tầng thấp. Chẳng hạn, xét một bộ giải mã Transformer gồm L khối xếp chồng. Để phân bổ khối lượng tính toán, mỗi khối được giao cho một máy thực thi. Hình minh họa dưới đây cho thấy một lần chạy lượt truyền xuôi và lượt truyền ngược của mô hình này.



Ở đây B_l ký hiệu phép tính của khối l, còn các ký hiệu ↑ và ↓ lần lượt chỉ lượt truyền xuôi và lượt truyền ngược (*Worker* l là máy thực thi thứ l). Lưu ý rằng phương pháp song song này buộc các máy thực thi chạy tuần tự, nên một máy phải đợi máy trước nó hoàn thành công việc. Điều này khiến các thiết bị phải nằm chờ phần lớn thời gian. Trong các hệ thống thực tế, song song mô hình thường được dùng kết hợp với các cơ chế song song khác để tận dụng tối đa các thiết bị.

- **Song song tensor (*Tensor Parallelism*)**. Song song hóa cũng có thể được thực hiện bên trong một bước tính toán duy nhất. Một cách phổ biến là chia một ma trận tham số lớn thành các khối, nhân tensor đầu vào với từng khối một cách riêng rẽ, rồi ghép nối (*concatenate*) kết quả của các phép nhân này để tạo thành đầu ra. Ví dụ, xét phép nhân biểu diễn $\mathbf{h} \in \mathbb{R}^d$ với ma trận tham số $\mathbf{W}_h \in \mathbb{R}^{d \times d_h}$ trong một tầng con FFN (xem Phương trình (8.25)). Ta có thể cắt ma trận

$\mathbf{W}_h \in \mathbb{R}^{d \times d_h}$ theo chiều dọc thành M ma trận con

$$\mathbf{W}_h = \begin{bmatrix} \mathbf{W}_h^1 & \mathbf{W}_h^2 & \cdots & \mathbf{W}_h^M \end{bmatrix}, \tag{8.34}$$

trong đó mỗi ma trận con $\mathbf{W}_h^k$ có kích thước $d \times \frac{d_h}{M}$. Phép nhân $\mathbf{h}$ với $\mathbf{W}_h$ có thể được viết là

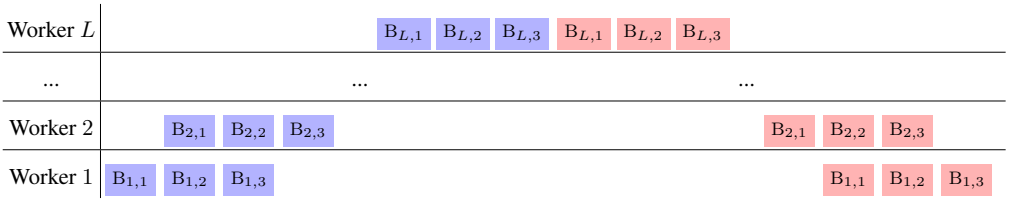
$$\begin{aligned} \mathbf{h}\mathbf{W}_h &= \mathbf{h} \begin{bmatrix} \mathbf{W}_h^1 & \mathbf{W}_h^2 & \cdots & \mathbf{W}_h^M \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{h}\mathbf{W}_h^1 & \mathbf{h}\mathbf{W}_h^2 & \cdots & \mathbf{h}\mathbf{W}_h^M \end{bmatrix}. \end{aligned} \tag{8.35}$$

Ta có thể thực hiện các phép nhân ma trận $\{\mathbf{h}\mathbf{W}_h^1, \mathbf{h}\mathbf{W}_h^2, \cdots, \mathbf{h}\mathbf{W}_h^M\}$ riêng rẽ trên M thiết bị. Nhờ đó, ta phân bổ một phép nhân ma trận lớn lên nhiều thiết bị, mỗi thiết bị có thể chỉ có bộ nhớ tương đối nhỏ. Từ góc độ thiết kế GPU hiện đại, song song tensor trên các GPU mang lại một cách tiếp cận hai cấp, dựa trên chia ô (*tile-based*), cho tính toán song song. Trước hết, ở cấp cao hơn, ta phân rã một phép nhân ma trận thành các phép nhân ma trận con vừa khít bộ nhớ của GPU. Sau đó, ở cấp thấp hơn, ta thực hiện các phép nhân ma trận con này trên GPU bằng các thuật toán song song chia ô được tối ưu riêng cho GPU.

Chú giải của người dịch: Ví dụ nhỏ về cắt ma trận theo cột

Nhân vectơ hàng $\mathbf{h} = [1 \quad 2]$ với ma trận 2×4: mỗi phần tử kết quả chỉ phụ thuộc vào *một cột* của ma trận (phần tử thứ j = h_1·(số hàng 1, cột j) + h_2·(số hàng 2, cột j)). Vì vậy, nếu cắt ma trận thành hai nửa trái (cột 1–2) và phải (cột 3–4), máy A nhân $\mathbf{h}$ với nửa trái được 2 số đầu, máy B nhân với nửa phải được 2 số cuối; đặt cạnh nhau là ra đúng kết quả đầy đủ. Mỗi máy chỉ cần giữ một nửa ma trận trong bộ nhớ.

- **Song song đường ống (*Pipeline Parallelism*)**. Ở phần song song mô hình bên trên, chúng ta đã mô tả một cách đơn giản để phân bổ các nhóm thành phần của mô hình lên nhiều thiết bị. Nhưng phương pháp này kém hiệu quả vì trong quá trình xử lý, tại mỗi thời điểm chỉ có một thiết bị hoạt động. Song song đường ống giải quyết vấn đề này bằng cách cho các phép tính trên các thiết bị khác nhau chồng lấp lên nhau [Harlap và cộng sự, 2018; Huang và cộng sự, 2019]. Để làm vậy, một lô mẫu được chia thành nhiều lô vi mô (*micro-batches*), rồi các lô vi mô này được mỗi máy thực thi xử lý như bình thường. Ngay khi một lô vi mô được một máy xử lý xong và chuyển sang máy kế tiếp, lô vi mô tiếp theo lập tức thế chỗ trên chính máy đó. Nói cách khác, ta tạo ra một đường ống, trong đó các bước tính toán khác nhau có thể chồng lấp nếu nhiều công việc được đưa vào đường ống. Hình dưới đây minh họa song song đường ống khi xử lý 3 lô vi mô.



Ở đây $B_{l,k}$ biểu diễn việc máy thực thi thứ l xử lý lô vi mô thứ k. Lý tưởng nhất, ta muốn tăng tối đa số lô vi mô, và nhờ đó giảm tối thiểu thời gian nằm chờ của các máy thực thi. Tuy nhiên, trong thực tế, dùng các lô vi mô nhỏ thường làm giảm mức độ tận dụng GPU và tăng chi phí chuyển đổi tác vụ. Điều này đến lượt nó có thể làm giảm thông lượng (*throughput*) tổng thể của hệ thống.

Chú giải của người dịch: Đường ống giống dây chuyền lắp ráp

Hình dung một xưởng làm bánh có 3 công đoạn: nhào bột, nướng, đóng gói, mỗi công đoạn mất 1 giờ. Nếu làm từng mẻ một cách tuần tự (song song mô hình đơn giản), 3 mẻ mất $3 \times 3 = 9$ giờ và mỗi lúc chỉ có một người làm việc. Theo dây chuyền: khi mẻ 1 chuyển sang lò nướng thì mẻ 2 bắt đầu nhào bột ngay, … Tổng thời gian chỉ còn $3 + 2 = 5$ giờ. Càng chia nhiều mẻ nhỏ thì thời gian chờ ở đầu và cuối dây chuyền càng ít đáng kể, nhưng mẻ quá nhỏ thì mỗi lần chuyển mẻ lại tốn công, đúng như sách nói về lô vi mô.

Mục tiêu tối thượng của xử lý song song là đạt được mức tăng hiệu suất tuyến tính, nghĩa là số mẫu có thể được xử lý trong một đơn vị thời gian tăng tuyến tính theo số thiết bị. Tuy nhiên, huấn luyện phân tán rất phức tạp và chịu ảnh hưởng của nhiều yếu tố ngoài phương pháp song song mà ta chọn. Một vấn đề thường gắn với các hệ thống phân tán là chi phí truyền thông. Ta có thể xem một hệ thống phân tán như một nhóm các nút (*nodes*) được kết nối mạng với nhau. Mỗi nút có thể thực hiện tính toán cục bộ hoặc truyền dữ liệu cho các nút khác. Nếu số nút như vậy rất lớn, việc phân phát và thu gom dữ liệu giữa chúng sẽ rất tốn kém. Đôi khi, thời gian tiết kiệm được nhờ song song hóa lại bị triệt tiêu bởi chi phí truyền thông của một mạng lớn. Một vấn đề khác với các hệ thống phân tán quy mô lớn là việc đồng bộ hóa (*synchronization*) các nút gây thêm chi phí. Như thường lệ, một số nút có thể mất nhiều thời gian hơn để hoàn thành nhiệm vụ, khiến các nút khác phải đợi những nút chậm nhất. Mặc dù có thể dùng huấn luyện bất đồng bộ (*asynchronous training*) để xử lý sự không đồng đều về tài nguyên tính toán, điều này có thể dẫn tới gradient lỗi thời (*stale gradients*) và không đảm bảo hội tụ. Hơn nữa, khi càng có thêm nhiều nút trong mạng, khả năng có nút bị sập trong quá trình huấn luyện càng cao. Khi đó, ta cần bảo đảm toàn bộ hệ thống có khả năng chịu lỗi (*fault-tolerant*). Trong nhiều tình huống thực tế, để tăng khả năng mở rộng (*scalability*), người ta cần tính đến thêm nhiều vấn đề, bao gồm thiết kế kiến trúc, việc cho truyền dữ liệu chồng lấp với tính toán, cân bằng tải (*load balancing*), băng thông bộ nhớ (*memory bandwidth*), v.v.

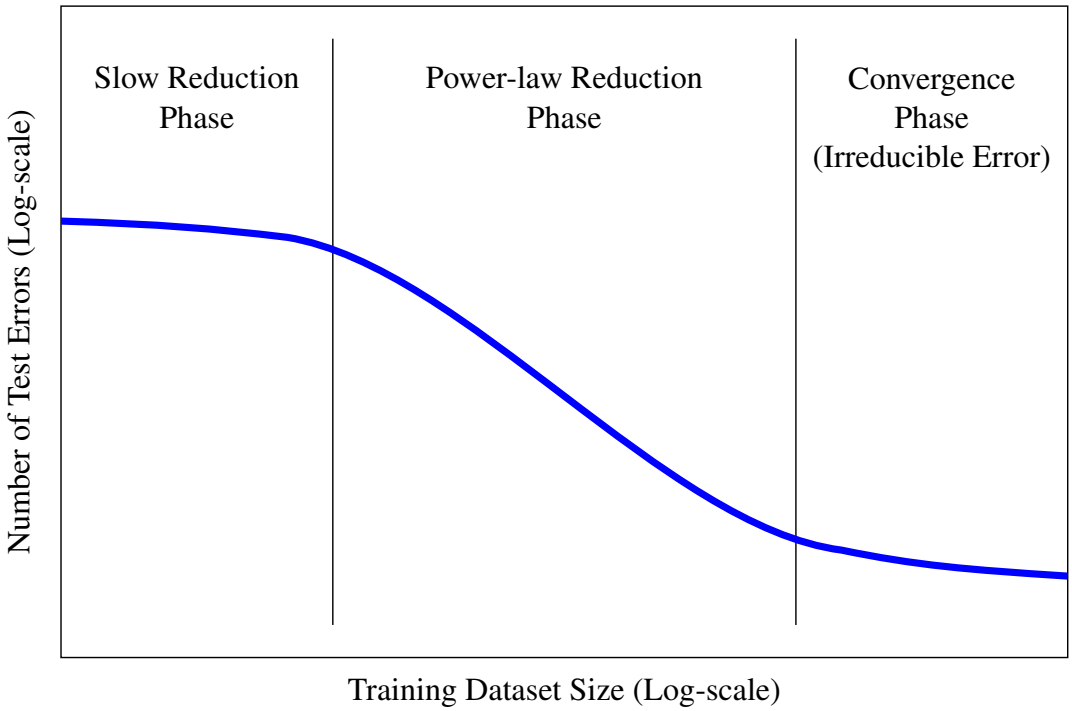
Huấn luyện LLM tốn kém về tính toán đến mức, dù đã dùng huấn luyện phân tán, các nhà nghiên cứu và kỹ sư vẫn thường áp dụng nhiều phương pháp nén mô hình (*model compression*) và tăng tốc khác nhau để nâng cao hiệu suất huấn luyện [Weng, 2021]. Một ví dụ là huấn luyện độ chính xác hỗn hợp, trong đó dữ liệu độ chính xác thấp (như dữ liệu FP16 và FP8) được dùng để tính gradient trên mỗi nút, còn dữ liệu độ chính xác cao hơn (ví dụ FP32/FP64) được dùng để cập nhật mô hình [Micikevicius và cộng sự, 2018]. Một thao tác then chốt trong cách tiếp cận này là tích lũy gradient (*gradient accumulation*), trong đó các gradient cần được cộng dồn và đồng bộ giữa các nút. Tuy nhiên, do phép cộng số dấu phẩy động không có tính kết hợp, điều này có thể dẫn tới những sai khác số học nhỏ trong các gradient tích lũy trên các nút khác nhau, có thể ảnh hưởng tới sự hội tụ của mô hình và hiệu năng cuối cùng. Vấn đề này càng rõ rệt khi có nhiều nút tham gia huấn luyện phân tán, đặc biệt khi các phép tính số học độ chính xác thấp có thể gặp các vấn đề tràn trên (*overflow*) và tràn dưới (*underflow*), cũng như sự không nhất quán giữa các thiết bị phần cứng khác nhau. Do đó, việc thiết kế hệ thống phân tán cần xét đến các vấn đề tính toán số này để bảo đảm kết quả và sự hội tụ thỏa đáng.

Chú giải của người dịch: Phép cộng máy tính không có tính kết hợp

Máy tính lưu số thực với một số lượng chữ số có nghĩa giới hạn (FP16 chỉ khoảng 3–4 chữ số thập phân, FP32 khoảng 7). Giả sử máy chỉ giữ 3 chữ số có nghĩa. Khi đó $(1000 + 0.4) + 0.4$: bước đầu 1000.4 bị làm tròn thành 1000, rồi $1000 + 0.4$ lại thành 1000. Nhưng $1000 + (0.4 + 0.4) = 1000 + 0.8 = 1000.8 \to 1001$. Hai cách nhóm cho hai kết quả khác nhau! Vì vậy, khi các nút cộng gradient theo những thứ tự khác nhau, kết quả có thể lệch nhau chút ít. "Tràn trên" là khi số quá lớn vượt khả năng biểu diễn (FP16 tối đa khoảng 65 504), "tràn dưới" là khi số quá nhỏ bị làm tròn thành 0.

8.2.4 Định luật co giãn

Thành công của LLM cho thấy việc huấn luyện các mô hình ngôn ngữ lớn hơn với nhiều tài nguyên hơn có thể giúp cải thiện hiệu năng mô hình. Các nhà nghiên cứu đã mô tả hiện tượng này là **định luật co giãn** (*scaling laws*) của LLM. Cụ thể hơn, định luật co giãn mô tả mối quan hệ giữa hiệu năng của LLM và các yếu tố tham gia vào quá trình huấn luyện, như kích thước mô hình, lượng tính toán dùng để huấn luyện và lượng dữ liệu huấn luyện. Chẳng hạn, Hestness và cộng sự [2017] chỉ ra rằng hiệu năng của các mạng nơ-ron sâu là một hàm lũy thừa (*power-law function*) của kích thước dữ liệu huấn luyện. Ban đầu, khi lượng dữ liệu huấn luyện còn tương đối ít, hiệu năng của mô hình được cải thiện chậm. Sau đó, khi dùng thêm nhiều dữ liệu huấn luyện, mô hình bước vào giai đoạn hiệu năng tăng nhanh, và đường cong hiệu năng đi theo xu hướng hàm lũy thừa. Cuối cùng, mức cải thiện hiệu năng lại trở nên chậm, và thêm dữ liệu không đem lại lợi ích đáng kể. Hình 8.3 cho thấy một ví dụ về những đường cong như vậy.



Hình 8.3: Minh họa định luật co giãn của lỗi kiểm tra theo một biến được quan tâm (ví dụ: kích thước tập dữ liệu huấn luyện) [Hestness và cộng sự, 2017]. Đường cong của định luật co giãn có thể được chia thành ba giai đoạn. Ban đầu, lỗi kiểm tra giảm chậm khi dùng thêm dữ liệu huấn luyện, nhưng giai đoạn này chỉ kéo dài trong thời gian ngắn. Ở giai đoạn thứ hai, lỗi kiểm tra giảm mạnh, và đường cong trở thành một đường cong lũy thừa. Sau đó, mức giảm lỗi lại chậm dần ở giai đoạn thứ ba. Lưu ý rằng có những lỗi không thể khử được, bất kể lượng dữ liệu huấn luyện là bao nhiêu. (Nhãn trong hình: *Slow Reduction Phase* – giai đoạn giảm chậm; *Power-law Reduction Phase* – giai đoạn giảm theo hàm lũy thừa; *Convergence Phase (Irreducible Error)* – giai đoạn hội tụ (lỗi không thể khử); trục tung: số lỗi trên tập kiểm tra (thang log); trục hoành: kích thước tập dữ liệu huấn luyện (thang log).)

Trong NLP, một quan điểm truyền thống cho rằng lợi ích về hiệu năng sẽ biến mất tại một thời điểm nào đó khi quy mô huấn luyện tăng lên. Tuy nhiên, các kết quả gần đây cho thấy rằng nếu xét vấn đề ở quy mô lớn hơn, việc mở rộng quy mô huấn luyện vẫn là một phương pháp rất hiệu quả để có được

những LLM mạnh hơn. Ví dụ, cả LLM mã nguồn đóng lẫn mã nguồn mở đều có thể hưởng lợi từ việc có thêm dữ liệu, dù đã dùng tới hàng nghìn tỷ token để huấn luyện.

Khi quy mô huấn luyện mô hình tăng lên, LLM thể hiện những năng lực mới, được gọi là **năng lực đột sinh** (*emergent abilities*) của LLM. Chẳng hạn, Wei và cộng sự [2022b] đã nghiên cứu các tính chất co giãn của LLM với những kích thước mô hình và lượng tài nguyên tính toán khác nhau. Công trình của họ cho thấy một số năng lực xuất hiện khi ta tăng kích thước mô hình tới một mức nhất định. Việc quan sát thấy các năng lực đột sinh đã chứng minh vai trò của huấn luyện quy mô lớn trong việc nâng cao hiệu năng của LLM, và ở một mức độ nào đó, cũng thôi thúc các nhà nghiên cứu không ngừng thử huấn luyện các mô hình lớn hơn. Khi các LM lớn hơn và mạnh hơn tiếp tục xuất hiện, hiểu biết của chúng ta về định luật co giãn ngày càng chín muồi. Hiểu biết này giúp các nhà nghiên cứu dự đoán hiệu năng của LLM trong quá trình huấn luyện và ước tính lượng tài nguyên tính toán cần thiết để đạt được một mức hiệu năng cho trước.

Để hiểu hiệu năng mô hình thay đổi thế nào theo các yếu tố khác nhau trong huấn luyện, người ta thường biểu diễn hiệu năng mô hình như một hàm của các yếu tố đó. Ví dụ, trong trường hợp đơn giản nhất, ta có thể biểu diễn mất mát hoặc lỗi của một LLM như một hàm của một biến được quan tâm duy nhất. Tuy nhiên, không có định luật co giãn phổ quát nào mô tả được mối quan hệ này. Thay vào đó, người ta đề xuất những hàm khác nhau để khớp với đường cong học (*learning curves*) của LLM.

Gọi x là biến được quan tâm (chẳng hạn số tham số của mô hình) và $\mathcal{L}(x)$ là mất mát của mô hình ứng với x (chẳng hạn mất mát entropy chéo trên dữ liệu kiểm tra). Dạng đơn giản nhất của $\mathcal{L}(x)$ là một luật lũy thừa (*power law*)

$$\mathcal{L}(x) \;=\; ax^b, \tag{8.36}$$

trong đó a và b là các tham số được ước lượng bằng thực nghiệm. Dù đơn giản, hàm này đã giải thích thành công hành vi co giãn của các mô hình ngôn ngữ và các hệ thống dịch máy theo kích thước mô hình (ký hiệu là N) và kích thước tập dữ liệu huấn luyện (ký hiệu là D) [Gordon và cộng sự, 2021; Hestness và cộng sự, 2017]. Ví dụ, Kaplan và cộng sự [2020] nhận thấy rằng, sau một giai đoạn chuyển tiếp ban đầu, hiệu năng mô hình ngôn ngữ của họ được cải thiện theo luật lũy thừa của N hoặc của D, và biểu diễn các mối quan hệ này bằng $\mathcal{L}(N) = \left(\frac{N}{8.8\times 10^{13}}\right)^{-0.076}$ và $\mathcal{L}(D) = \left(\frac{D}{5.4\times 10^{13}}\right)^{-0.095}$ (xem Hình 8.4).

Chú giải của người dịch: Luật lũy thừa và thang logarit

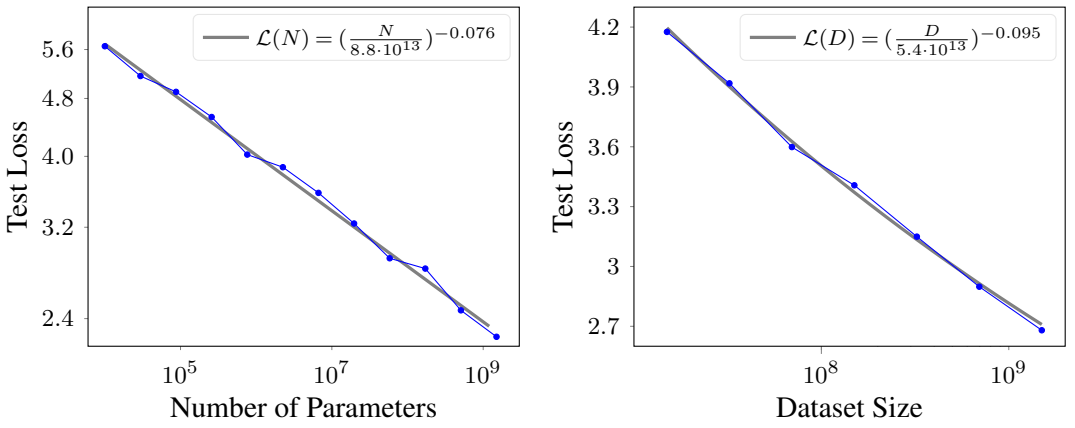
Lấy logarit hai vế của $\mathcal{L}(x) = ax^b$: $\log \mathcal{L} = \log a + b \log x$. Đây là một hàm *bậc nhất* theo $\log x$, nên khi vẽ trên hai trục thang log (như Hình 8.4) ta được một *đường thẳng* có hệ số góc b (âm, vì mất mát giảm). Với $\mathcal{L}(N) = (N/(8.8 \times 10^{13}))^{-0.076}$: tăng số tham số lên 10 lần thì mất mát nhân với $10^{-0.076} \approx 0.84$, tức giảm khoảng 16%. Muốn giảm thêm 16% nữa lại phải tăng mô hình thêm 10 lần – lợi ích có thật nhưng ngày càng đắt.

Một cải tiến cho định luật co giãn này là thêm một số hạng **lỗi không thể khử** (*irreducible error*) vào luật lũy thừa. Khi đó dạng của $\mathcal{L}(x)$ là

$$\mathcal{L}(x) \;=\; ax^b + \epsilon_\infty, \tag{8.37}$$

trong đó ϵ_∞ là lỗi không thể khử, đại diện cho phần lỗi do các biến chưa biết gây ra, vẫn tồn tại ngay cả khi $x \to \infty$. Phương trình (8.37) là một trong những dạng được dùng rộng rãi nhất để mô hình hóa định luật co giãn của LLM. Ví dụ, Rosenfeld và cộng sự [2020] đã xây dựng một định luật co giãn bao gồm cả việc co giãn mô hình lẫn co giãn tập dữ liệu, như sau

$$\mathcal{L}(N, D) \;=\; aN^b + cD^d + \epsilon_\infty. \tag{8.38}$$



Hình 8.4: Mất mát trên tập kiểm tra (*Test Loss*) theo kích thước mô hình (N, *Number of Parameters* – số tham số) và kích thước tập dữ liệu huấn luyện (D, *Dataset Size*) (các điểm dữ liệu được vẽ chỉ nhằm minh họa). Ta vẽ mất mát kiểm tra như một hàm của N, được định nghĩa là $\mathcal{L}(N) = \left(\frac{N}{8.8 \times 10^{13}}\right)^{-0.076}$, và như một hàm của D, được định nghĩa là $\mathcal{L}(D) = \left(\frac{D}{5.4 \times 10^{13}}\right)^{-0.095}$ [Kaplan và cộng sự, 2020].

Một ví dụ cho dạng công thức này là định luật co giãn Chinchilla (*Chinchilla scaling law*). Định luật này phát biểu rằng mất mát kiểm tra trên mỗi token bằng tổng của các hàm lũy thừa nghịch của N và D, cộng thêm một số hạng lỗi không thể khử. Hoffmann và cộng sự [2022] biểu diễn định luật co giãn này là

$$\mathcal{L}(N, D) = \underbrace{\frac{406.4}{N^{0.34}}}_{\text{co giãn mô hình}} + \underbrace{\frac{410.7}{D^{0.28}}}_{\text{co giãn dữ liệu}} + \underbrace{1.69}_{\text{lỗi không thể khử}} . \tag{8.39}$$

Chú giải của người dịch: Thử số với định luật Chinchilla

Lấy mô hình $N = 10^{10}$ tham số (10 tỷ) huấn luyện trên $D = 2 \times 10^{11}$ token (200 tỷ). Ta có $N^{0.34} = 10^{3.4} \approx 2512$ nên số hạng đầu $\approx 406.4/2512 \approx 0.16$; $D^{0.28} = 10^{0.28 \times 11.3} \approx 10^{3.16} \approx 1445$ nên số hạng thứ hai $\approx 410.7/1445 \approx 0.28$. Tổng mất mát $\approx 0.16 + 0.28 + 1.69 = 2.13$. Dù N và D lớn đến đâu, mất mát cũng không thể xuống dưới 1.69: đó là phần "không thể đoán trước" vốn có của ngôn ngữ (cùng một ngữ cảnh, người viết thật vẫn có thể viết tiếp theo nhiều cách khác nhau).

Tất cả các định luật co giãn kể trên đều dựa trên các hàm đơn điệu. Vì vậy chúng không thể bao quát những hàm có điểm uốn (*inflection points*), chẳng hạn các đường cong suy giảm kép (*double descent*). Để giải quyết điều này, các nhà nghiên cứu đã khám phá những hàm phức tạp hơn để khớp với đường cong học. Có thể tìm thấy ví dụ về những hàm như vậy trong công trình của Alabdulmohsin và cộng sự [2022] và Caballero và cộng sự [2023].

Ý nghĩa của định luật co giãn nằm ở chỗ nó đưa ra định hướng cho nghiên cứu LLM: nếu ta vẫn còn ở vùng đường cong lũy thừa, việc dùng nhiều tài nguyên hơn để huấn luyện các mô hình lớn hơn là một hướng đi rất hứa hẹn. Dù kết quả này buộc các nhóm nghiên cứu và công ty lớn đầu tư mạnh vào tài nguyên tính toán để huấn luyện các mô hình lớn hơn – điều rất tốn kém – định luật co giãn vẫn liên tục đẩy xa giới hạn của AI. Mặt khác, hiểu biết về định luật co giãn giúp các nhà nghiên cứu đưa ra quyết định có cơ sở khi huấn luyện LLM. Ví dụ, với lượng tài nguyên tính toán sẵn có, ta có thể ước tính được hiệu năng cuối cùng của một LLM.

Một lưu ý cuối cùng về định luật co giãn trong mục này: đối với LLM, mất mát kiểm tra thấp hơn

không phải lúc nào cũng có nghĩa là hiệu năng tốt hơn trên mọi nhiệm vụ hạ nguồn. Để thích nghi LLM, có một số bước như tinh chỉnh và kỹ thuật prompt có thể ảnh hưởng đến kết quả cuối cùng. Do đó, trong thực tế, định luật co giãn cho các nhiệm vụ hạ nguồn khác nhau có thể khác nhau.

8.3 Mô hình hóa chuỗi dài

Chúng ta đã thấy rằng trong huấn luyện quy mô lớn, có thể phát triển các mô hình ngôn ngữ mạnh hơn bằng cách tăng quy mô cả dữ liệu lẫn tài nguyên tính toán. Tuy nhiên, việc co giãn cũng có thể diễn ra theo những chiều khác. Chẳng hạn, trong nhiều ứng dụng hiện đại, LLM thường cần được điều chỉnh để xử lý các chuỗi rất dài. Một tình huống phổ biến là tiền huấn luyện LLM trên các văn bản có độ dài thông thường, rồi sau đó áp dụng nó cho những chuỗi dài hơn rất nhiều so với những gì đã thấy khi tiền huấn luyện. Ở đây, ta dùng $\Pr(\mathbf{y}|\mathbf{x})$ để ký hiệu xác suất có điều kiện của việc sinh văn bản, trong đó $\mathbf{x}$ là ngữ cảnh và $\mathbf{y}$ là văn bản được sinh ra. Nói chung, các bài toán mô hình hóa chuỗi dài có thể được chia thành ba loại:

- **Sinh văn bản với ngữ cảnh dài** (*Long-context text generation*) (tức $\mathbf{x}$ là một chuỗi dài). Ví dụ, sinh một bản tóm tắt ngắn cho một tài liệu dài.
- **Sinh văn bản dài** (*Long-text generation*) (tức $\mathbf{y}$ là một chuỗi dài). Ví dụ, sinh một câu chuyện hoàn chỉnh từ vài từ khóa.
- **Sinh văn bản dài với ngữ cảnh dài** (*Long-context and long-text generation*) (tức cả $\mathbf{x}$ và $\mathbf{y}$ đều là chuỗi dài). Ví dụ, dịch một tài liệu dài từ tiếng Trung sang tiếng Anh.

Gần đây, các nhà nghiên cứu NLP ngày càng quan tâm đến việc áp dụng và đánh giá LLM trên các nhiệm vụ có văn bản đầu vào cực kỳ dài. Hãy tưởng tượng một LLM đọc một tệp mã nguồn C++ gồm hàng chục nghìn dòng, rồi phác thảo chức năng của chương trình ứng với tệp đó. Những mô hình có khả năng xử lý ngữ cảnh văn bản lớn như vậy thường được gọi là **LLM ngữ cảnh dài** (*long-context LLMs*). Trong mục này, chúng ta sẽ giới hạn ở LLM ngữ cảnh dài, nhưng các phương pháp được bàn ở đây cũng có thể áp dụng cho các bài toán khác.

Với Transformer, việc xử lý chuỗi dài rất tốn kém về tính toán, vì chi phí tính toán của cơ chế tự chú ý tăng theo bình phương độ dài chuỗi. Điều này khiến việc huấn luyện và triển khai các mô hình như vậy cho đầu vào rất dài là không khả thi. Có hai hướng nghiên cứu đã cố gắng điều chỉnh Transformer cho mô hình hóa ngôn ngữ ngữ cảnh dài.

- Hướng thứ nhất tìm tòi các phương pháp huấn luyện và kiến trúc mô hình hiệu quả để học các mô hình tự chú ý từ dữ liệu chuỗi dài.
- Hướng thứ hai điều chỉnh các LLM đã được tiền huấn luyện để xử lý chuỗi dài mà chỉ cần tinh chỉnh ít hoặc không cần tinh chỉnh.

Ở đây, chúng ta sẽ bàn ngắn gọn về hướng thứ nhất vì Chương 6 đã trình bày kỹ nhiều phương pháp thuộc hướng này. Chúng ta sẽ tập trung vào hướng thứ hai, nhấn mạnh các phương pháp phổ biến trong các LLM gần đây. Chúng ta cũng sẽ bàn về điểm mạnh và hạn chế của các mô hình chuỗi dài này.

Chú giải của người dịch: "Tăng theo bình phương" nghĩa là gì?

Trong tự chú ý, mỗi token phải "nhìn" vào mọi token khác, nên với chuỗi m token ta cần cỡ $m \times m = m^2$ điểm chú ý. Chuỗi 1 000 token cần 10^6 điểm; chuỗi 100 000 token (dài gấp 100 lần) cần 10^{10} điểm – gấp

$100^2 = 10\,000$ lần. Giống như số cái bắt tay khi mọi người trong phòng đều bắt tay nhau: số người tăng gấp đôi thì số cái bắt tay tăng gần gấp bốn.

8.3.1 Tối ưu hóa phần cứng và tính toán song song

Chúng ta bắt đầu bằng việc xem xét các cải tiến cho mô hình Transformer chuẩn từ góc độ tính toán hiệu năng cao (*high-performance computing*). Phần lớn các cải tiến này, dù không được thiết kế riêng cho LLM, đã được áp dụng rộng rãi cho nhiều mô hình học sâu khác nhau [Kim và cộng sự, 2023]. Một cách tiếp cận thường dùng là sử dụng các định dạng dữ liệu độ chính xác giảm. Ví dụ, thay vì dùng phép tính dấu phẩy động (*floating-point*) 32 bit chuẩn (FP32), các mô hình có thể dùng số dấu phẩy động 16 bit (FP16/BF16), hay thậm chí lượng tử hóa (*quantization*) thành số nguyên 8 bit (INT8). Dùng các kiểu dữ liệu độ chính xác thấp này có thể cải thiện hiệu suất tính toán và thông lượng bộ nhớ, để có thể xử lý các chuỗi dài hơn dễ dàng hơn. Một cách tiếp cận khác là cải tiến Transformer bằng các kỹ thuật nhận biết phần cứng (*hardware-aware*). Ví dụ, trên các GPU hiện đại, hiệu suất của Transformer có thể được cải thiện bằng các cài đặt hàm tự chú ý có nhận biết vào/ra (*IO-aware*) [Dao và cộng sự, 2022; Kwon và cộng sự, 2023].

Một cách khác để xử lý chuỗi dài là thông qua song song chuỗi (*sequence parallelism*) [Li và cộng sự, 2023b; Korthikanti và cộng sự, 2023]. Cụ thể, xét bài toán tổng quát: cho truy vấn $\mathbf{q}_i$ tại vị trí i chú ý tới các khóa $\mathbf{K}$ và các giá trị $\mathbf{V}$. Ta có thể chia $\mathbf{K}$ theo chiều chuỗi (tức theo hàng) và thu được một tập ma trận con $\{\mathbf{K}^{[1]}, \cdots, \mathbf{K}^{[n_u]}\}$, mỗi ma trận ứng với một đoạn của chuỗi. Tương tự, ta có thể thu được các ma trận con của $\mathbf{V}$, ký hiệu là $\{\mathbf{V}^{[1]}, \cdots, \mathbf{V}^{[n_u]}\}$. Sau đó, ta giao mỗi cặp $\mathbf{K}^{[u]}$ và $\mathbf{V}^{[u]}$ cho một nút tính toán (*compute node*) (ví dụ một GPU trong một cụm GPU). Các nút được giao việc có thể chạy song song, nhờ đó song song hóa phép toán chú ý.

Nhắc lại rằng đầu ra của mô hình tự chú ý có thể được viết là

$$\mathrm{Att}_{\mathrm{qkv}}(\mathbf{q}_i, \mathbf{K}, \mathbf{V}) \;=\; \sum_{j=0}^{m-1} \alpha_{i,j}\mathbf{v}_j, \tag{8.40}$$

trong đó $\alpha_{i,j}$ là trọng số chú ý giữa vị trí i và j. Trong Transformer, $\alpha_{i,j}$ thu được bằng cách chuẩn hóa tích vô hướng đã được co giãn giữa $\mathbf{q}_i$ và $\mathbf{k}_j$. Gọi $\beta_{i,j}$ là điểm chú ý giữa $\mathbf{q}_i$ và $\mathbf{k}_j$. Ta có

$$\beta_{i,j} \;=\; \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}} + \mathrm{Mask}(i,j), \tag{8.41}$$

trong đó $\mathrm{Mask}(i,j)$ là giá trị mặt nạ nhân quả (*causal mask*) cho cặp vị trí (i,j). Khi đó, ta định nghĩa trọng số chú ý $\alpha_{i,j}$ là

$$\begin{aligned}\alpha_{i,j} &= \mathrm{Softmax}(\beta_{i,j}) \\ &= \frac{\exp(\beta_{i,j})}{\sum_{j'}\exp(\beta_{i,j'})}.\end{aligned} \tag{8.42}$$

Trên mỗi nút tính toán, ta cần thực hiện các phương trình này. Với các khóa và giá trị được giao cho nút này, việc tính tử số ở vế phải của Phương trình (8.42) (tức $\exp(\beta_{i,j})$) là đơn giản, vì mọi thông tin cần thiết đều nằm trên nút. Tuy nhiên, việc tính mẫu số ở vế phải của Phương trình (8.42) đòi hỏi một tổng của $\exp(\beta_{i,j'})$ trên mọi j', điều này cần truyền dữ liệu qua lại với các nút khác. Để minh họa, giả sử $\mathbf{v}_j$ và $\mathbf{k}_j$ được đặt trên nút u. Ta có thể viết lại Phương trình (8.42) thành

$\alpha_{i,j}$

$$= \frac{\overbrace{\exp(\beta_{i,j})}^{\text{nút } u}}{\underbrace{\sum_{\mathbf{k}_{j'} \in \mathbf{K}^{[1]}} \exp(\beta_{i,j'})}_{\text{nút 1}} + \cdots + \underbrace{\sum_{\mathbf{k}_{j'} \in \mathbf{K}^{[u]}} \exp(\beta_{i,j'})}_{\text{nút } u} + \cdots + \underbrace{\sum_{\mathbf{k}_{j'} \in \mathbf{K}^{[n_u]}} \exp(\beta_{i,j'})}_{\text{nút } n_u}}, \tag{8.43}$$

trong đó ký hiệu $\mathbf{k}_{j'} \in \mathbf{K}^{[u]}$ cho biết $\mathbf{k}_{j'}$ là một vectơ hàng của $\mathbf{K}^{[u]}$. Trong một cài đặt trực tiếp, trước hết ta thực hiện các phép tính tổng $\{\sum_{\mathbf{k}_{j'} \in \mathbf{K}^{[u]}} \exp(\beta_{i,j'})\}$ một cách riêng rẽ trên các nút tương ứng. Sau đó, ta thu thập các kết quả tổng này từ các nút khác nhau để kết hợp thành một kết quả cuối cùng. Điều này tương ứng với một phép toán tập thể (*collective operation*) trong bối cảnh xử lý song song. Có nhiều cách cài đặt hiệu quả cho những phép toán như vậy, chẳng hạn các phép toán *all-reduce*. Do đó, tổng của mọi giá trị $\exp(\beta_{i,j})$ có thể được tính bằng các thủ tục đã được tối ưu trong các bộ công cụ truyền thông tập thể (*collective communication*).

Với các trọng số chú ý $\{\alpha_{i,j}\}$ đã có, ta tính kết quả chú ý bằng Phương trình (8.40). Bài toán có thể được biểu diễn lại là

$$\begin{aligned}&\text{Att}_{\text{qkv}}(\mathbf{q}_i, \mathbf{K}, \mathbf{V}) \\ &= \underbrace{\sum_{\mathbf{v}_{j'} \in \mathbf{V}^{[1]}} \alpha_{i,j'} \mathbf{v}_{j'}}_{\text{nút 1}} + \cdots + \underbrace{\sum_{\mathbf{v}_{j'} \in \mathbf{V}^{[u]}} \alpha_{i,j'} \mathbf{v}_{j'}}_{\text{nút } u} + \cdots + \underbrace{\sum_{\mathbf{v}_{j'} \in \mathbf{V}^{[n_u]}} \alpha_{i,j'} \mathbf{v}_{j'}}_{\text{nút } n_u}.\end{aligned} \tag{8.44}$$

Giống như Phương trình (8.43), Phương trình (8.44) có thể được cài đặt như một phép rút gọn (*reduction operation*) trong xử lý song song. Trước hết, ta thực hiện đồng thời các phép tính tổng có trọng số của các giá trị trên các nút khác nhau. Sau đó, ta thu thập kết quả từ các nút này bằng các phép toán tập thể.

Chú giải của người dịch: Song song chuỗi qua một ví dụ nhỏ

Giả sử chuỗi có 6 token, chia cho 2 nút: nút 1 giữ khóa/giá trị của token 0–2, nút 2 giữ token 3–5. Để tính trọng số chú ý, mẫu số của Softmax là $e^{\beta_0} + \cdots + e^{\beta_5}$. Nút 1 tự tính $S_1 = e^{\beta_0} + e^{\beta_1} + e^{\beta_2}$, nút 2 tự tính $S_2 = e^{\beta_3} + e^{\beta_4} + e^{\beta_5}$, rồi chỉ cần trao đổi *một con số* S_1, S_2 để mỗi nút có tổng $S = S_1 + S_2$. Sau đó mỗi nút tính phần tổng có trọng số $\sum \alpha_{i,j}\mathbf{v}_j$ trên các token của mình, và cuối cùng cộng hai vectơ lại. "All-reduce" chính là thao tác "mọi nút góp phần của mình, rồi mọi nút đều nhận được tổng chung".

Lưu ý rằng mặc dù mục này chủ yếu tập trung vào mô hình hóa chuỗi dài, phần lớn động lực của song song chuỗi đến từ các phương pháp huấn luyện phân tán cho mạng sâu, như đã bàn ở Mục 8.2.3. Do đó, việc cài đặt các phương pháp này có thể được xây dựng trên cùng một thư viện xử lý song song.

8.3.2 Các kiến trúc hiệu quả

Một khó khăn khi áp dụng Transformer cho chuỗi dài là cơ chế tự chú ý chuẩn có độ phức tạp thời gian và bộ nhớ bậc hai theo độ dài chuỗi. Hơn nữa, việc sinh văn bản tự hồi quy (*autoregressive*) đòi hỏi duy trì một **bộ nhớ đệm khóa-giá trị** (*key-value cache*) (hay **bộ nhớ đệm KV** (*KV cache*)), có kích thước tăng dần khi càng nhiều token được xử lý. Mặc dù dung lượng bộ nhớ của bộ nhớ đệm KV chỉ tăng tuyến tính theo độ dài chuỗi, với những ngữ cảnh cực kỳ dài, yêu cầu bộ nhớ này trở nên quá lớn, khiến việc triển khai LLM chuẩn không còn thực tế. Kết quả là kiến trúc mô hình của các LLM ngữ cảnh dài nhìn chung rời xa Transformer chuẩn, chuyển sang phát triển các biến thể và phương án thay thế hiệu quả hơn.

Một cách tiếp cận là thay cơ chế tự chú ý chuẩn bằng chú ý thưa (*sparse attention*). Họ mô hình này xuất phát từ nhận xét rằng các phân phối chú ý trên chuỗi dài thường rất lệch. Do đó, chỉ một tập con nhỏ các token có ảnh hưởng đáng kể tới ngữ cảnh của một token bất kỳ. Vì vậy, ta có thể cắt tỉa phần lớn các trọng số chú ý, biểu diễn cơ chế này dưới dạng được nén mạnh. Để minh họa, xét công thức tự chú ý chuẩn:

$$\text{Att}_{\text{qkv}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \;=\; \alpha(\mathbf{Q}, \mathbf{K})\mathbf{V}, \tag{8.45}$$

trong đó ma trận trọng số chú ý $\alpha(\mathbf{Q}, \mathbf{K}) \in \mathbb{R}^{m \times m}$ thu được bởi

$$\begin{aligned}\alpha(\mathbf{Q}, \mathbf{K}) &= \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{Mask}\right) \\ &= \begin{bmatrix} \alpha_{0,0} & 0 & 0 & \cdots & 0 \\ \alpha_{1,0} & \alpha_{1,1} & 0 & \cdots & 0 \\ \alpha_{2,0} & \alpha_{2,1} & \alpha_{2,2} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \alpha_{m-1,0} & \alpha_{m-1,1} & \alpha_{m-1,2} & \cdots & \alpha_{m-1,m-1} \end{bmatrix}.\end{aligned} \tag{8.46}$$

Mỗi vectơ hàng $\begin{bmatrix} \alpha_{i,0} & \cdots & \alpha_{i,i} & 0 & \cdots & 0 \end{bmatrix}$ tương ứng với một phân phối trên các token, cho biết token thứ i chú ý tới các token khác như thế nào. Vì mô hình ngôn ngữ dự đoán token tiếp theo chỉ dựa trên ngữ cảnh bên trái, ta thường viết đầu ra của mô hình chú ý tại vị trí i là

$$\begin{aligned}\text{Att}_{\text{qkv}}(\mathbf{q}_i, \mathbf{K}_{\leq i}, \mathbf{V}_{\leq i}) &= \begin{bmatrix} \alpha_{i,0} & \cdots & \alpha_{i,i} \end{bmatrix} \begin{bmatrix} \mathbf{v}_0 \\ \vdots \\ \mathbf{v}_i \end{bmatrix} \\ &= \sum_{j=0}^{i} \alpha_{i,j}\mathbf{v}_j,\end{aligned} \tag{8.47}$$

trong đó $\mathbf{K}_{\leq i} = \begin{bmatrix} \mathbf{k}_0 \\ \vdots \\ \mathbf{k}_i \end{bmatrix}$ và $\mathbf{V}_{\leq i} = \begin{bmatrix} \mathbf{v}_0 \\ \vdots \\ \mathbf{v}_i \end{bmatrix}$ là các khóa và giá trị tính đến vị trí i.

Trong tự chú ý chuẩn, vectơ $\begin{bmatrix} \alpha_{i,0} & \cdots & \alpha_{i,i} \end{bmatrix}$ được giả định là dày đặc (*dense*), nghĩa là gần như mọi phần tử hợp lệ đều khác 0. Trong chú ý thưa, chỉ một tập con định trước của các phần tử này được tính, còn phần còn lại bị bỏ qua một cách tường minh (ép bằng 0). Gọi $G \subseteq \{0, \cdots, i\}$ là tập chỉ số của các phần tử hoạt động (khác 0). Khi đó, đầu ra của mô hình chú ý thưa tại vị trí i trở thành:

$$\text{Att}_{\text{sparse}}(\mathbf{q}_i, \mathbf{K}_{\leq i}, \mathbf{V}_{\leq i}) \;=\; \sum_{j \in G} \alpha'_{i,j}\mathbf{v}_j. \tag{8.48}$$

Ở đây $\{\alpha'_{i,j}\}$ được chuẩn hóa trên G. Vì thế giá trị của chúng khác với các trọng số chú ý ban đầu (thực tế ta có $\alpha'_{i,j} > \alpha_{i,j}$). Độ thưa của mô hình được quyết định bởi kích thước của G. Các mô hình chú ý thưa khác nhau ở cách định nghĩa G. Một cách đơn giản là định nghĩa G dựa trên các mẫu được thiết kế theo kinh nghiệm. Ví dụ, một mẫu được dùng rộng rãi là cho G bao phủ một cửa sổ các token nằm gần vị trí i [Parmar và cộng sự, 2018].

Chú giải của người dịch: Vì sao $\alpha'_{i,j} > \alpha_{i,j}$?

Giả sử token i chú ý tới 4 token với trọng số $[0.1 \quad 0.2 \quad 0.3 \quad 0.4]$ (tổng bằng 1). Nếu chú ý thưa chỉ giữ hai token cuối, $G = \{2, 3\}$, thì ta chuẩn hóa lại cho tổng bằng 1: $0.3/(0.3 + 0.4) \approx 0.43$ và $0.4/0.7 \approx 0.57$. Cả hai đều lớn hơn giá trị cũ vì "phần bánh" của các token bị bỏ đi được chia lại cho các token còn giữ. Đổi lại, mỗi token chỉ phải tính $|G|$ trọng số thay vì $i + 1$ trọng số.

Dù chú ý thưa giảm khối lượng tính toán nhờ dùng các phép toán thưa, những mô hình này vẫn có hạn chế đáng kể vì ta cần giữ toàn bộ bộ nhớ đệm KV (tức $\mathbf{K}_{\leq i}$ và $\mathbf{V}_{\leq i}$) trong quá trình suy luận. Nếu chuỗi rất dài, việc lưu bộ nhớ đệm này sẽ tốn rất nhiều bộ nhớ. Để giải quyết, ta có thể xét một lớp mô hình chú ý khác, trong đó bộ nhớ đệm KV không được lưu giữ một cách tường minh. Chú ý tuyến tính (*Linear attention*) là một cách tiếp cận như vậy [Katharopoulos và cộng sự, 2020]. Nó dùng một hàm nhân (*kernel function*) $\phi(\cdot)$ để ánh xạ mỗi truy vấn và khóa vào một không gian đặc trưng, lần lượt cho ra $\mathbf{q}'_i = \phi(\mathbf{q}_i)$ và $\mathbf{k}'_i = \phi(\mathbf{k}_i)$. Bằng cách thay phép chuẩn hóa Softmax bằng một công thức tuyến tính[9], dạng của mô hình chú ý thu được là

$$\begin{aligned}\text{Att}_{\text{qkv}}(\mathbf{q}_i, \mathbf{K}_{\leq i}, \mathbf{V}_{\leq i}) &\approx \text{Att}_{\text{linear}}(\mathbf{q}'_i, \mathbf{K}'_{\leq i}, \mathbf{V}_{\leq i}) \\ &= \frac{\mathbf{q}'_i \mu_i}{\mathbf{q}'_i \nu_i},\end{aligned} \tag{8.49}$$

trong đó μ_i và ν_i là các biến được tính theo cách hồi quy

$$\mu_i = \mu_{i-1} + \mathbf{k}_i'^{\top} \mathbf{v}_i \tag{8.50}$$

$$\nu_i = \nu_{i-1} + \mathbf{k}_i'^{\top}. \tag{8.51}$$

μ_i và ν_i có thể được xem như các bản tóm tắt lịch sử tính đến vị trí i. Một lợi ích của mô hình này là ta không cần lưu tường minh mọi khóa và giá trị trong quá khứ. Thay vào đó, chỉ các biểu diễn mới nhất μ_i và ν_i được sử dụng. Vì vậy chi phí tính toán ở mỗi bước là một hằng số, và mô hình có thể dễ dàng mở rộng để xử lý chuỗi dài.

Chú giải của người dịch: Ý tưởng "tổng cộng dồn" của chú ý tuyến tính

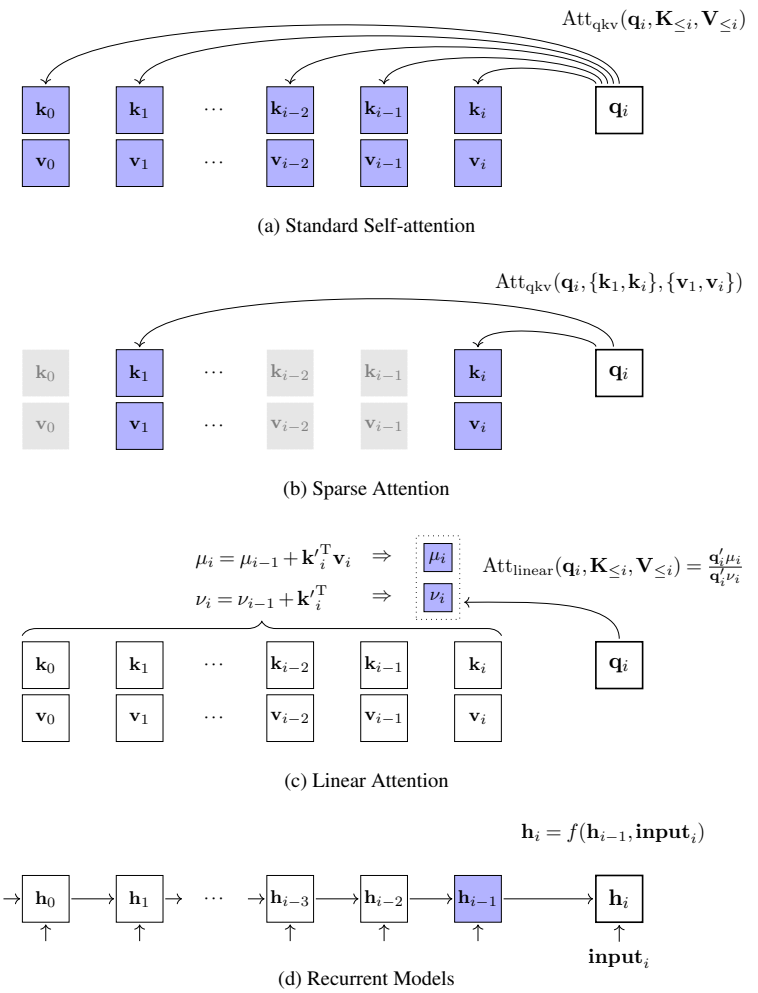
Phương trình (8.50–8.51) giống cách tính tổng tiền tiết kiệm: thay vì ghi lại từng khoản gửi rồi mỗi lần lại cộng hết từ đầu, ta chỉ giữ *số dư* và cộng thêm khoản mới: số dư$_i$ = số dư$_{i-1}$ + khoản gửi$_i$. Ở đây "số dư" là μ_i (một ma trận cố định kích thước) và ν_i (một vectơ). Mỗi token mới chỉ tốn một phép cộng vào hai đại lượng này, nên bộ nhớ và thời gian cho mỗi bước không phụ thuộc vào việc chuỗi đã dài bao nhiêu. Điều này làm được là vì đã bỏ hàm mũ của Softmax: tổng $\sum_j (\mathbf{q}'_i \mathbf{k}'^{\top}_j) \mathbf{v}_j$ có thể viết thành $\mathbf{q}'_i \big(\sum_j \mathbf{k}'^{\top}_j \mathbf{v}_j \big)$, tức đưa $\mathbf{q}'_i$ ra ngoài tổng.

Thực ra, cách tiếp cận tuần tự này đối với mô hình hóa chuỗi dài xuất hiện một cách tự nhiên khi ta nhìn từ góc độ các mô hình hồi quy. Những mô hình như vậy đọc một token (hoặc một số ít token) mỗi lần, cập nhật trạng thái hồi quy (*recurrent state*) bằng các đầu vào này, rồi loại bỏ chúng trước khi token tiếp theo đến. Đầu ra ở mỗi bước được sinh ra chỉ dựa trên trạng thái hồi quy, chứ không dựa trên tất cả các trạng thái trước đó. Dung lượng bộ nhớ được quyết định bởi trạng thái hồi quy, vốn có kích thước cố định. Các mô hình hồi quy có thể được dùng trong các tình huống học thời gian thực, khi dữ liệu đến theo dòng và dự đoán có thể được đưa ra tại bất kỳ bước thời gian nào. Trong NLP, áp dụng mô hình hồi quy cho mô hình hóa ngôn ngữ là một trong những nỗ lực thành công sớm nhất để học biểu

[9] Trong không gian mới sau phép biến đổi này, phép chuẩn hóa Softmax có thể được chuyển thành phép chuẩn hóa co giãn đơn giản.

diễn của chuỗi. Mặc dù Transformer đã được dùng làm kiến trúc nền tảng của LLM, các mô hình hồi quy vẫn là những mô hình mạnh, đặc biệt cho việc phát triển các LLM hiệu quả. Chẳng hạn, các mô hình hồi quy, nhất là các mô hình không gian trạng thái (*state space models*) hiện đại như Mamba [Gu và Dao, 2023], đã hồi sinh trong mô hình hóa ngôn ngữ và được xem xét lại như một phương án thay thế đầy hứa hẹn cho Transformer.

Hình 8.5 so sánh các mô hình được bàn trong tiểu mục này. Vì các mô hình này, cùng những mô hình khác không được nhắc tới ở đây, đã được bàn kỹ trong Chương 6 và trong các bài tổng quan liên quan [Tay và cộng sự, 2020b], chúng tôi bỏ qua phần thảo luận chi tiết hơn ở đây.



(a) Standard Self-attention

(b) Sparse Attention

(c) Linear Attention

(d) Recurrent Models

Hình 8.5: Minh họa cơ chế tự chú ý, chú ý thưa, chú ý tuyến tính và các mô hình hồi quy. Các hộp màu xanh lam là những trạng thái được lưu đệm để tạo ra đầu ra tại vị trí i. $f(\cdot)$ là một ô hồi quy (*recurrent cell*). (Các nhãn con: (a) tự chú ý chuẩn; (b) chú ý thưa; (c) chú ý tuyến tính; (d) các mô hình hồi quy.)

8.3.3 Bộ nhớ đệm và bộ nhớ

Các LLM dựa trên kiến trúc Transformer chuẩn sử dụng cơ chế chú ý toàn cục (dày đặc). Trong quá trình lập luận tự hồi quy, các mô hình này cần lưu trữ toàn bộ ngữ cảnh bên trái để dự đoán các token

tương lai. Điều này đòi hỏi một bộ nhớ đệm KV (*KV cache*), trong đó các biểu diễn (tức là khóa và giá trị) của mọi token đã được sinh ra trước đó đều được giữ lại. Kết quả là dung lượng bộ nhớ mà bộ nhớ đệm này chiếm dụng tăng tuyến tính theo tiến trình suy luận. Trong các mục trước, chúng ta đã bàn về những phương pháp giảm bớt nút thắt ngữ cảnh này thông qua các thuật toán chú ý hiệu quả, như chú ý thưa và chú ý tuyến tính. Một hướng tiếp cận thay thế có liên quan là mã hóa và nén ngữ cảnh một cách tường minh thông qua một mô-đun bộ nhớ phụ trợ (*auxiliary memory module*).

Chú giải của người dịch: Vì sao bộ nhớ đệm KV "phình" ra?

Khi sinh văn bản, mô hình tạo từng token một. Để tạo token thứ $i+1$, truy vấn $\mathbf{q}_i$ phải "nhìn" lại khóa và giá trị của *tất cả* token trước đó. Thay vì tính lại chúng mỗi bước, ta lưu chúng lại (đó là bộ nhớ đệm KV). Nếu mỗi token cần lưu, chẳng hạn, 1 MB khóa–giá trị, thì một văn bản 1000 token cần 1 GB, còn 100 000 token cần tới 100 GB. Dung lượng tăng tỉ lệ thuận với độ dài, nên với ngữ cảnh rất dài, bộ nhớ trở thành "nút cổ chai". Ý tưởng của tiểu mục này: giữ cho bộ nhớ có kích thước *cố định*, bất kể văn bản dài bao nhiêu.

1. Bộ nhớ đệm KV kích thước cố định

Một cách tiếp cận trực tiếp là biểu diễn các khóa và giá trị bằng một mô hình bộ nhớ có kích thước cố định. Giả sử ta có một bộ nhớ Mem lưu giữ thông tin ngữ cảnh. Ta có thể viết phép toán chú ý tại vị trí i dưới dạng tổng quát

$$\mathrm{Att}(\mathbf{q}_i, \mathrm{Mem}) = \mathrm{Att}_{\mathrm{qkv}}(\mathbf{q}_i, \mathbf{K}_{\leq i}, \mathbf{V}_{\leq i}). \tag{8.52}$$

Trong mô hình này, Mem đơn giản chính là bộ nhớ đệm KV, tức là $\mathrm{Mem} = (\mathbf{K}_{\leq i}, \mathbf{V}_{\leq i})$. Do đó kích thước của Mem được quyết định bởi i. Nếu Mem được định nghĩa có kích thước cố định thì chi phí thực hiện $\mathrm{Att}(\mathbf{q}_i, \mathrm{Mem})$ cũng sẽ cố định. Có một số cách khác nhau để thiết kế Mem.

- Một trong những phương pháp đơn giản nhất là xét một cửa sổ có kích thước cố định gồm các khóa và giá trị trước đó. Khi đó Mem được cho bởi

  $$\mathrm{Mem} = (\mathbf{K}_{[i-n_c+1,i]}, \mathbf{V}_{[i-n_c+1,i]}), \tag{8.53}$$

  trong đó n_c là kích thước của cửa sổ. Các ký hiệu $\mathbf{K}_{[i-n_c+1,i]}$ và $\mathbf{V}_{[i-n_c+1,i]}$ chỉ các khóa và giá trị trên các vị trí từ $i-n_c+1$ đến i.[10] Mô hình này có thể được xem là một dạng mô hình chú ý cục bộ (*local attention*).

- Ta cũng có thể định nghĩa Mem là một cặp vectơ tóm tắt, dẫn tới một biểu diễn lịch sử được nén nhiều hơn. Một cách đơn giản để tóm tắt các khóa và giá trị trước đó là dùng trung bình trượt (*moving average*) của các vectơ này. Ví dụ, Mem có thể được định nghĩa là trung bình trượt không trọng số của n_c khóa và giá trị gần nhất

  $$\mathrm{Mem} = \left(\frac{\sum_{j=i-n_c+1}^{i} \mathbf{k}_j}{n_c}, \frac{\sum_{j=i-n_c+1}^{i} \mathbf{v}_j}{n_c} \right). \tag{8.54}$$

  Ngoài ra, ta có thể dùng trung bình trượt có trọng số

  $$\mathrm{Mem} = \left(\frac{\sum_{j=i-n_c+1}^{i} \beta_{j-i+n_c} \mathbf{k}_j}{\sum_{j=1}^{n_c} \beta_j}, \frac{\sum_{j=i-n_c+1}^{i} \beta_{j-i+n_c} \mathbf{v}_j}{\sum_{j=1}^{n_c} \beta_j} \right). \tag{8.55}$$

[10] Một cách hình thức hơn, ta viết $\mathbf{K}_{[i-n_c+1,i]} = \begin{bmatrix} \mathbf{k}_{i-n_c+1} \\ \vdots \\ \mathbf{k}_i \end{bmatrix}$ và $\mathbf{V}_{[i-n_c+1,i]} = \begin{bmatrix} \mathbf{v}_{i-n_c+1} \\ \vdots \\ \mathbf{v}_i \end{bmatrix}$. Đôi khi, để ký hiệu gọn hơn, ta viết $\mathbf{K}_{[i-n_c+1,i]}$ thành $\{\mathbf{k}_{i-n_c+1}, \cdots, \mathbf{k}_i\}$ và $\mathbf{V}_{[i-n_c+1,i]}$ thành $\{\mathbf{v}_{i-n_c+1}, \cdots, \mathbf{v}_i\}$.

Ở đây $\{\beta_1, \cdots, \beta_{n_c}\}$ là các hệ số, có thể được học như tham số mô hình hoặc được xác định bằng các quy tắc kinh nghiệm (*heuristics*). Ví dụ, ta có thể đặt chúng là các hệ số tăng dần (tức là $\beta_1 < \beta_2 < \cdots < \beta_{n_c-1} < \beta_{n_c}$) để gán trọng số lớn hơn cho những vị trí gần i hơn. Ta có thể mở rộng trung bình trượt để bao gồm tất cả các vị trí cho tới i. Điều này dẫn tới trung bình tích lũy (*cumulative average*) của các khóa và giá trị, có dạng

$$\text{Mem} = \left(\frac{\sum_{j=0}^{i} \mathbf{k}_j}{i+1}, \frac{\sum_{j=0}^{i} \mathbf{v}_j}{i+1} \right). \tag{8.56}$$

Nói chung, trung bình tích lũy có thể được viết dưới dạng đệ quy

$$\text{Mem}_i = \frac{(\mathbf{k}_i, \mathbf{v}_i) + i \cdot \text{Mem}_{i-1}}{i+1}, \tag{8.57}$$

trong đó Mem_i và Mem_{i-1} lần lượt là trung bình tích lũy tại vị trí hiện tại và vị trí trước đó. Ưu điểm của mô hình này là trong khi suy luận ta chỉ cần lưu một cặp khóa–giá trị duy nhất, thay vì lưu tất cả các cặp khóa–giá trị. Lưu ý rằng các mô hình bộ nhớ ở trên có liên hệ với các mô hình hồi quy, và nhiều kỹ thuật tiên tiến hơn đã được dùng để phát triển các phương án thay thế cho cơ chế tự chú ý trong Transformer [Ma và cộng sự, 2023].

Chú giải của người dịch: Kiểm tra công thức trung bình tích lũy bằng số

Lấy các "khóa" là các số đơn giản: $k_0 = 2$, $k_1 = 4$, $k_2 = 9$. Theo Phương trình (8.56), trung bình tích lũy tại $i = 2$ là $(2+4+9)/3 = 5$. Dùng dạng đệ quy (8.57): $\text{Mem}_0 = 2$; $\text{Mem}_1 = (4+1\cdot 2)/2 = 3$; $\text{Mem}_2 = (9 + 2 \cdot 3)/3 = 5$. Kết quả trùng khớp! Điểm mấu chốt: ở mỗi bước ta chỉ cần nhớ *một* giá trị trung bình cũ (nhân với i để "khôi phục" tổng cũ), chứ không cần nhớ toàn bộ dãy $k_0, k_1, \ldots$. Đây cũng là cách bạn tính lại điểm trung bình môn khi có thêm một bài kiểm tra mà không cần xem lại mọi điểm cũ.

Trong trung bình trượt có trọng số (8.55), với $n_c = 3$ và $\beta_1 = 1, \beta_2 = 2, \beta_3 = 3$, token gần nhất ($j = i$) có trọng số $\beta_3 = 3$, token xa nhất trong cửa sổ có trọng số $\beta_1 = 1$; tổng được chia cho $1 + 2 + 3 = 6$ để các "tỉ lệ đóng góp" cộng lại bằng 1.

- Bộ nhớ Mem cũng có thể là một mạng nơ-ron. Tại mỗi bước, nó nhận cả đầu ra trước đó của bộ nhớ lẫn các trạng thái hiện tại của mô hình làm đầu vào, rồi tạo ra đầu ra mới của bộ nhớ. Mạng nơ-ron này có thể được biểu diễn dưới dạng hàm

$$\text{Mem} = \text{Update}(S_{\text{kv}}, \text{Mem}_{\text{pre}}). \tag{8.58}$$

Ở đây Mem và Mem_{pre} lần lượt là đầu ra của bộ nhớ tại bước hiện tại và bước trước đó. S_{kv} là một tập các cặp khóa–giá trị, biểu diễn các trạng thái gần đây của mô hình. Cách biểu diễn này mang tính tổng quát và cho phép ta phát triển nhiều mô hình bộ nhớ khác nhau bằng cách chọn các cấu hình $\text{Update}(\cdot)$ và S_{kv} khác nhau. Ví dụ, nếu S_{kv} chỉ chứa cặp khóa–giá trị mới nhất $(\mathbf{k}_i, \mathbf{v}_i)$ và $\text{Update}(\cdot)$ được định nghĩa là một ô hồi quy, thì Phương trình (8.58) có thể được biểu diễn thành một mô hình giống RNN

$$\text{Mem} = f((\mathbf{k}_i, \mathbf{v}_i), \text{Mem}_{\text{pre}}), \tag{8.59}$$

trong đó $f(\cdot)$ là một ô hồi quy (*recurrent cell*). Phép hồi quy cũng có thể được áp dụng cho việc mô hình hóa ở mức đoạn để tăng hiệu quả. Một cách đơn giản là chia chuỗi thành các đoạn (*segment*) và coi S_{kv} là một đoạn. Áp dụng các mô hình hồi quy cho $\text{Update}(\cdot)$ sẽ cho ra các mô hình bộ nhớ hoạt động trên các đoạn. Một trường hợp đặc biệt là định nghĩa $\text{Update}(\cdot)$ là một

hàm FIFO, thêm S_{kv} vào bộ nhớ và loại bỏ đoạn khóa–giá trị cũ nhất khỏi bộ nhớ, cho bởi

$$\text{Mem} = \text{FIFO}(S_{\text{kv}}, \text{Mem}_{\text{pre}}). \tag{8.60}$$

Xét một bộ nhớ gồm hai đoạn: một cho đoạn hiện tại và một cho đoạn trước đó. Trong phép toán chú ý, mỗi vị trí có thể truy cập các cặp khóa–giá trị lịch sử trong hai đoạn liên tiếp gần nhất. Tuy đơn giản về mặt khái niệm, bộ nhớ cục bộ dạng khối trượt (*sliding-chunk local memory*) này cùng các biến thể của nó đã được dùng rộng rãi trong các kiến trúc hồi quy mức đoạn [Dai và cộng sự, 2019; Hutchins và cộng sự, 2022; Bulatov và cộng sự, 2022].

Chú giải của người dịch: FIFO là gì?

FIFO (*First In, First Out* – vào trước, ra trước) là cách hoạt động của một *hàng đợi*, giống hàng người xếp hàng mua vé: ai đến trước được phục vụ (và rời hàng) trước. Với bộ nhớ có sức chứa cố định là 2 đoạn: khi đoạn mới S_3 đến, bộ nhớ đang chứa $\{S_1, S_2\}$ sẽ "đẩy" S_1 (cũ nhất) ra và trở thành $\{S_2, S_3\}$. Nhờ vậy kích thước bộ nhớ luôn không đổi.

- Các mô hình bộ nhớ trên có thể được mở rộng để dùng nhiều bộ nhớ. Một ví dụ cho cách tiếp cận này là Transformer nén (*compressive Transformer*) [Rae và cộng sự, 2019b]. Mô hình này dùng hai bộ nhớ riêng biệt có kích thước cố định: một để mô hình hóa ngữ cảnh cục bộ (ký hiệu Mem), và một để mô hình hóa và nén lịch sử dài hạn (ký hiệu CMem). Bộ nhớ đệm KV trong mô hình này là sự kết hợp của Mem và CMem. Hàm chú ý có thể được viết là

$$\text{Att}_{\text{com}}(\mathbf{q}_i, \text{Mem}, \text{CMem}) = \text{Att}_{\text{qkv}}(\mathbf{q}_i, [\text{Mem}, \text{CMem}]), \tag{8.61}$$

trong đó $[\text{Mem}, \text{CMem}]$ là bộ nhớ gộp của Mem và CMem. Tương tự các mô hình mức đoạn chuẩn, Transformer nén xử lý chuỗi theo từng khối. Mỗi đoạn gồm n_s token liên tiếp, và S^k_{kv} ký hiệu các cặp khóa–giá trị thô sinh ra từ đoạn thứ k. Khi một đoạn mới đến, Mem được cập nhật qua một hàng đợi FIFO: nó nối thêm n_s cặp khóa–giá trị mới từ S^k_{kv} và đồng thời loại bỏ n_s cặp cũ nhất:

$$\text{Mem} = \text{FIFO}(S^k_{\text{kv}}, \text{Mem}_{\text{pre}}). \tag{8.62}$$

Thay vì vứt bỏ các cặp bị đẩy ra này, chúng được đưa qua một mạng nén (*compression network*). n_s cặp thô được nén thành một tập nhỏ hơn gồm $\frac{n_s}{c}$ cặp khóa–giá trị cô đọng. Sau đó CMem đóng vai trò hàng đợi FIFO thứ cấp, nối thêm $\frac{n_s}{c}$ cặp đã nén này và loại bỏ $\frac{n_s}{c}$ mục cũ nhất của nó để giữ kích thước cố định. Điều này được cho bởi

$$\text{CMem} = \text{FIFO}(C^k_{\text{kv}}, \text{CMem}_{\text{pre}}), \tag{8.63}$$

trong đó C^k_{kv} biểu diễn tập các cặp khóa–giá trị đã nén. Transformer nén ngầm giả định rằng ngữ cảnh cục bộ nên được biểu diễn tường minh với mức mất mát thông tin tối thiểu, còn ngữ cảnh xa có thể được nén nhiều hơn.

Chú giải của người dịch: Ví dụ số cho Transformer nén

Giả sử mỗi đoạn có $n_s = 4$ token và tỉ lệ nén $c = 2$. Khi đoạn mới đến, 4 cặp khóa–giá trị mới được đưa vào Mem, đồng thời 4 cặp cũ nhất bị đẩy ra. Thay vì xóa, 4 cặp này được "nén" (chẳng hạn lấy trung bình từng đôi một) thành $4/2 = 2$ cặp, rồi đưa vào CMem; CMem lại đẩy ra 2 cặp cũ nhất của nó. Giống như cách ta ghi nhớ: chuyện vừa xảy ra thì nhớ chi tiết, chuyện lâu rồi chỉ nhớ ý chính.

- Chúng ta đã thấy rằng cả ngữ cảnh toàn cục lẫn cục bộ đều hữu ích và có thể được mô hình hóa bằng các mô hình chú ý. Quan điểm này thúc đẩy việc mở rộng các mô hình chú ý để kết hợp

cả bộ nhớ cục bộ lẫn bộ nhớ dài hạn [Ainslie và cộng sự, 2020; Zaheer và cộng sự, 2020; Gupta và Berant, 2020]. Một cách đơn giản nhưng được dùng rộng rãi là đưa vài token đầu tiên của chuỗi vào phép chú ý, đóng vai trò các token toàn cục (*global tokens*). Cách này thường được áp dụng cùng với các mô hình chú ý thưa khác. Một ưu điểm của việc đưa vào các token toàn cục là chúng đóng vai trò một bể hút chú ý (*attention sink*), hấp thụ các điểm chú ý dư thừa, nhờ đó ổn định hiệu năng mô hình khi kích thước ngữ cảnh rất lớn [Xiao và cộng sự, 2024]. Tuy nhiên, một nhược điểm là việc dùng bộ nhớ toàn cục có kích thước cố định có thể gây mất mát thông tin. Khi xử lý các chuỗi dài, ta cần mở rộng bộ nhớ đệm KV để có đủ biểu diễn cho ngữ cảnh, nhưng điều này lại làm tăng chi phí tính toán.

Chú giải của người dịch: Bể hút chú ý

Trọng số chú ý được tính bằng Softmax nên luôn có tổng bằng 1: dù truy vấn không thực sự "cần" token nào, nó vẫn phải phân bổ hết 1 đơn vị chú ý. Thực nghiệm cho thấy mô hình thường "đổ" phần chú ý thừa vào vài token đầu chuỗi. Nếu ta cắt bỏ các token đầu này khỏi cửa sổ (như trong bộ nhớ đệm cửa sổ trượt thuần túy), phần chú ý thừa không còn chỗ "đổ" và chất lượng có thể giảm mạnh. Giữ lại vài token đầu như các token toàn cục giúp tránh hiện tượng đó.

Hình 8.6 minh họa các cách tiếp cận trên. Lưu ý rằng mặc dù ở đây ta tập trung vào việc tối ưu bộ nhớ đệm KV, vấn đề này có liên hệ chặt chẽ với những vấn đề đã bàn trong mục trước. Mọi phương pháp ta đã nêu đến giờ nhìn chung đều có thể xếp vào nhóm các cách tiếp cận chú ý hiệu quả, vốn được dùng rộng rãi trong nhiều biến thể Transformer.

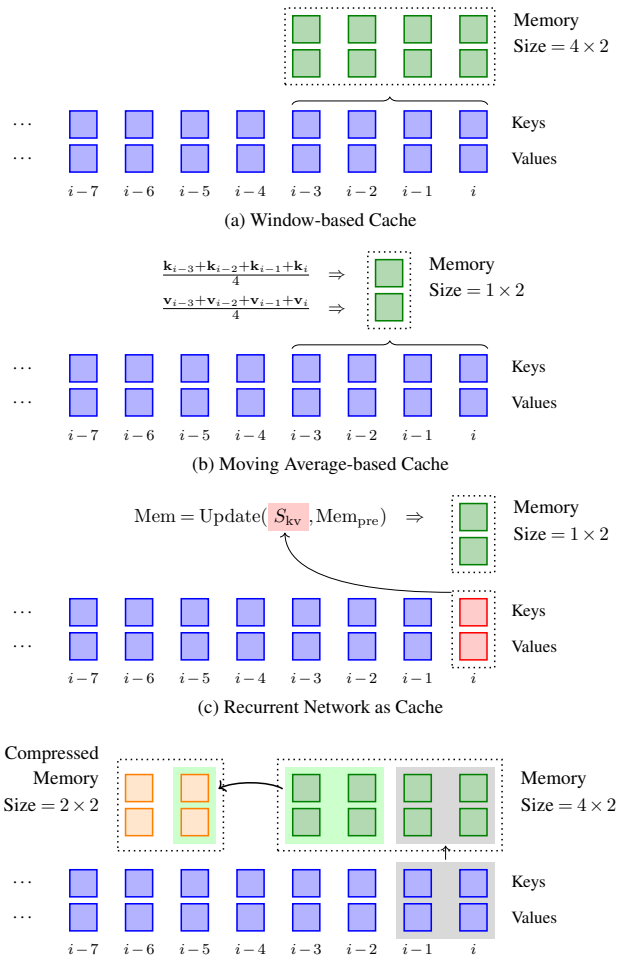
2. Các mô hình dựa trên bộ nhớ

Việc mô hình hóa bộ nhớ đã bàn ở trên dựa trên các cập nhật đối với bộ nhớ đệm KV, và các mô hình thu được thường được gọi là **bộ nhớ trong** (*internal memories*). Bây giờ ta xét một họ mô hình khác, gọi là **bộ nhớ ngoài** (*external memories*), hoạt động độc lập để truy cập các ngữ cảnh quy mô lớn cho LLM. Nhiều cách tiếp cận trong số này bắt nguồn từ các **phương pháp dựa trên bộ nhớ** (*memory-based methods*), một khái niệm đã được nghiên cứu sâu rộng trong học máy cổ điển [Bishop, 2006]. Một ví dụ phổ biến là các thuật toán láng giềng gần nhất: ta lưu các biểu diễn ngữ cảnh trong một kho dữ liệu (*datastore*) và truy vấn nó để tìm những biểu diễn đã lưu giống nhất. Các biểu diễn ngữ cảnh được truy xuất này sau đó được đưa vào để tăng cường cơ chế chú ý cho truy vấn hiện tại.

Ở đây, ta xét phương pháp k **láng giềng gần nhất** (k-*nearest neighbors,* k-*NN*), một trong những phương pháp dựa trên bộ nhớ phổ biến nhất. Vì trọng tâm của mục này là mô hình hóa ngôn ngữ, ta định nghĩa một mẫu trong kho dữ liệu là một cặp khóa–giá trị ứng với một trạng thái ngữ cảnh nào đó. Lưu ý rằng "ngữ cảnh" ở đây là một khái niệm rộng, không chỉ là tiền tố của chuỗi trong sinh văn bản. Chẳng hạn, ta có thể xem toàn bộ tập dữ liệu là ngữ cảnh để dự đoán token. Điều này cho phép ta truy xuất ngữ cảnh giống nhất trong một tập các chuỗi, thay vì trong một tiền tố chuỗi cho trước. Dù ta sẽ giới hạn ở việc mô hình hóa ngữ cảnh cho một chuỗi duy nhất, trong tiểu mục này ta bàn về một trường hợp tổng quát hơn.

Giả sử ta có một tập các khóa $\{\mathbf{k}_j\}$ cùng các giá trị tương ứng $\{\mathbf{v}_j\}$, và giả sử ta lưu các cặp khóa–giá trị này trong một cơ sở dữ liệu vectơ (*vector database*)[11]. Với mỗi truy vấn $\mathbf{q}_i$, ta tìm k láng giềng gần nhất của nó bằng cách tăng dần bán kính của mặt cầu tâm $\mathbf{q}_i$ cho tới khi mặt cầu chứa k điểm dữ liệu trong $\{\mathbf{k}_j\}$. Kết quả là một tập gồm k khóa cùng các giá trị tương ứng, ký hiệu là Mem_{knn}. Như trước, ta ký hiệu Mem là bộ nhớ cục bộ của truy vấn, chẳng hạn bộ nhớ đệm KV của các token lân cận. Mục

[11] Cơ sở dữ liệu vectơ (hay kho vectơ) là một cơ sở dữ liệu cung cấp các giao diện truy xuất được tối ưu hóa cao để tìm những vectơ đã lưu khớp gần nhất với một vectơ truy vấn.



Hình 8.6: Minh họa các bộ nhớ đệm KV kích thước cố định trong LLM. Các hộp màu xanh lam biểu diễn các khóa và giá trị được sinh ra trong quá trình suy luận của LLM, các hộp màu xanh lục biểu diễn các khóa và giá trị được lưu trữ hoặc mã hóa trong bộ nhớ chính, còn các hộp màu cam biểu diễn các khóa và giá trị được lưu trữ hoặc mã hóa trong bộ nhớ nén. (Các nhãn con: (a) bộ nhớ đệm dựa trên cửa sổ; (b) bộ nhớ đệm dựa trên trung bình trượt; (c) mạng hồi quy làm bộ nhớ đệm; (d) bộ nhớ đệm lai (bộ nhớ nén + bộ nhớ cục bộ). "Keys/Values" = khóa/giá trị; "Size = 4 × 2" = kích thước bộ nhớ.)

tiêu của ta là cho phép truy vấn $\mathbf{q}_i$ chú ý tới cả bộ nhớ cục bộ Mem lẫn bộ nhớ dài hạn Mem_{knn}. Dĩ nhiên có nhiều cách để đưa Mem và Mem_{knn} vào mô hình chú ý. Ví dụ, ta có thể đơn giản gộp chúng lại để tạo thành một bộ nhớ đệm KV duy nhất $[\text{Mem}, \text{Mem}_{\text{knn}}]$, rồi cho $\mathbf{q}_i$ chú ý tới $[\text{Mem}, \text{Mem}_{\text{knn}}]$ bằng phép chú ý QKV chuẩn. Hoặc ta có thể dùng Mem và Mem_{knn} trong các bước chú ý riêng rẽ. Một ví dụ cho cách tiếp cận này là mô hình do Wu và cộng sự [2021] phát triển. Mô hình này kết hợp tuyến tính hai loại chú ý, cho bởi

$$\text{Att}(\mathbf{q}_i, \text{Mem}, \text{Mem}_{\text{knn}}) = \mathbf{g} \odot \text{Att}_{\text{local}} + (1 - \mathbf{g}) \odot \text{Att}_{\text{knn}} \tag{8.64}$$

$$\text{Att}_{\text{local}} = \text{Att}(\mathbf{q}_i, \text{Mem}) \tag{8.65}$$

$$\text{Att}_{\text{knn}} = \text{Att}(\mathbf{q}_i, \text{Mem}_{\text{knn}}). \tag{8.66}$$

Ở đây $\mathbf{g} \in \mathbb{R}^d$ là vectơ hệ số, có thể là đầu ra của một cổng (*gate*) được học.

Chú giải của người dịch: Ý nghĩa của Phương trình (8.64)

Ký hiệu $\odot$ là phép nhân theo từng phần tử. Mỗi thành phần g của vectơ $\mathbf{g}$ nằm trong khoảng $[0, 1]$ và đóng vai trò "núm vặn" trộn hai nguồn thông tin. Ví dụ với một thành phần: nếu $g = 0.8$, kết quả chú ý cục bộ cho 5 và kết quả từ k-NN cho 10, thì đầu ra là $0.8 \cdot 5 + 0.2 \cdot 10 = 6$, nghiêng về thông tin cục bộ. Vì $\mathbf{g}$ là vectơ, mỗi chiều có thể có tỉ lệ trộn riêng.

Với mô hình bộ nhớ dựa trên k-NN mô tả ở trên, việc còn lại là xác định những cặp khóa–giá trị nào được giữ trong kho dữ liệu. Với các tác vụ mô hình hóa ngôn ngữ chuẩn, ta coi các token đã thấy trước đó trong chuỗi là ngữ cảnh, nên có thể thêm khóa và giá trị của tất cả các token này vào kho dữ liệu. Trong trường hợp đó, mô hình chú ý dựa trên k-NN thu được về bản chất tương đương với một mô hình chú ý thưa [Gupta và cộng sự, 2021].

Mặt khác, ta có thể mở rộng ngữ cảnh từ một chuỗi thành một tập các chuỗi. Ví dụ, ta có thể thu thập mọi cặp khóa–giá trị trên các chuỗi trong một tập dữ liệu huấn luyện và thêm chúng vào kho dữ liệu để mô hình hóa một ngữ cảnh lớn hơn. Nhờ vậy, LLM có thể dự đoán token dựa trên một ngữ cảnh tổng quát hóa. Một vấn đề của cách này là chi phí tính toán có thể lớn nếu có nhiều chuỗi tham gia. Vì các chuỗi này là một phần dữ liệu huấn luyện của ta, ta có thể xây dựng và tối ưu một chỉ mục (*index*) cho các vectơ trong kho dữ liệu trước khi chạy LLM. Kết quả là việc truy xuất các vectơ tương tự có thể rất hiệu quả, như thường được cài đặt trong các cơ sở dữ liệu vectơ hiện đại.

Thực ra, tất cả các phương pháp kể trên đều là những trường hợp cụ thể của mô thức tăng cường truy xuất rộng hơn. Ngoài việc đưa trực tiếp các vectơ được truy xuất vào cơ chế chú ý, việc truy xuất còn có thể được áp dụng ở mức phân phối đầu ra. Một ứng dụng điển hình là **mô hình hóa ngôn ngữ k-NN** (*k-NN language modeling, k-NN LM*) [Khandelwal và cộng sự, 2020]. Trực giác là: các trạng thái ẩn giống nhau trong một mạng Transformer có khả năng dự báo rất tốt rằng token tiếp theo cũng giống nhau, dù ở các ngữ cảnh khác nhau. Trong một k-NN LM, mỗi mục trong kho dữ liệu được lưu dưới dạng một bộ $(\mathbf{z}, w)$, trong đó $\mathbf{z}$ là trạng thái ẩn cuối cùng của LLM tại một vị trí cụ thể, còn w là token tiếp theo thực sự quan sát được tại vị trí đó. Thông thường, kho dữ liệu này được xây dựng *ngoại tuyến* (trước khi sử dụng) bằng cách cho toàn bộ tập dữ liệu huấn luyện đi qua LLM.

Trong khi suy luận, với một tiền tố chuỗi cho trước, LLM sinh ra trạng thái ẩn hiện tại $\mathbf{h}_i$. Ta truy vấn kho dữ liệu bằng $\mathbf{h}_i$ để lấy ra k trạng thái khớp gần nhất, thu được một tập các bộ $\{(\mathbf{z}_1, w_1), \cdots, (\mathbf{z}_k, w_k)\}$. Các token đích $\{w_1, \cdots, w_k\}$ đóng vai trò các đầu ra tham chiếu. Để tận dụng các tham chiếu này, ta chuyển các khoảng cách truy xuất thành một phân phối xác suất trên bộ từ vựng V. Gọi $d(\mathbf{h}_i, \mathbf{z}_j)$ là độ đo khoảng cách. Khối lượng xác suất mà phép truy xuất k-NN gán cho một token $v \in V$ được tính bằng cách cộng gộp các độ tương tự (đã chuẩn hóa bằng Softmax) của những

láng giềng được truy xuất:

$$\Pr_{\text{knn}}(v|\mathbf{h}_i) \propto \sum_{\substack{(\mathbf{z}_j, w_j)\in \text{top-}k, \\ w_j=v}} \exp\left(-d(\mathbf{h}_i, \mathbf{z}_j)\right). \tag{8.67}$$

Nếu một token v không xuất hiện trong số k láng giềng được truy xuất thì xác suất $\Pr_{\text{knn}}(v|\mathbf{h}_i)$ của nó bằng 0. Cuối cùng, ta dùng một siêu tham số điều chỉnh được $\lambda \in [0, 1]$ để nội suy tuyến tính giữa phân phối dựa trên truy xuất này và phân phối tham số chuẩn $\Pr_{\text{lm}}(\cdot|\mathbf{h}_i)$ do đầu phân loại (*classification head*) của LLM sinh ra:

$$\Pr(\cdot|\mathbf{h}_i) = \lambda \cdot \Pr_{\text{knn}}(\cdot|\mathbf{h}_i) + (1-\lambda) \cdot \Pr_{\text{lm}}(\cdot|\mathbf{h}_i). \tag{8.68}$$

Sau đó, như thường lệ, ta có thể chọn token tiếp theo y bằng cách cực đại hóa xác suất $\Pr(y|\mathbf{h}_i)$.

Chú giải của người dịch: Ví dụ tính tay cho k-NN LM

Giả sử $k = 3$ láng giềng được truy xuất là: ($\mathbf{z}_1$, "mèo") với khoảng cách $d = 1$, ($\mathbf{z}_2$, "mèo") với $d = 2$, và ($\mathbf{z}_3$, "chó") với $d = 1$. Theo (8.67), điểm của "mèo" là $e^{-1} + e^{-2} \approx 0.368 + 0.135 = 0.503$, điểm của "chó" là $e^{-1} \approx 0.368$. Ký hiệu $\propto$ ("tỉ lệ với") nghĩa là ta còn phải chia cho tổng $0.503 + 0.368 = 0.871$ để được xác suất: $\Pr_{\text{knn}}(\text{mèo}) \approx 0.58$, $\Pr_{\text{knn}}(\text{chó}) \approx 0.42$, mọi token khác có xác suất 0. Khoảng cách càng nhỏ thì e^{-d} càng lớn, tức láng giềng càng gần càng "có tiếng nói". Nếu LLM thông thường cho $\Pr_{\text{lm}}(\text{mèo}) = 0.3$ và chọn $\lambda = 0.5$ thì theo (8.68): $\Pr(\text{mèo}) = 0.5 \cdot 0.58 + 0.5 \cdot 0.3 = 0.44$.

Giống như các hệ thống truy xuất thông tin (*information retrieval, IR*), kho dữ liệu có thể lưu và truy xuất dữ liệu dạng văn bản. Ví dụ, ta có thể lưu một tập văn bản trong một máy tìm kiếm có chỉ mục toàn văn (*full-text indexing*), rồi tìm trong đó những văn bản khớp với một truy vấn dạng văn bản cho trước. Áp dụng các kỹ thuật IR cho LLM dẫn tới một khung tổng quát gọi là **sinh văn bản tăng cường truy xuất** (*retrieval-augmented generation, RAG*). Khung RAG hoạt động như sau. Ta dùng ngữ cảnh $\mathbf{x}$ làm truy vấn và tìm k đoạn văn bản (*document chunks*) liên quan nhất $\{\mathbf{c}_1, \cdots, \mathbf{c}_k\}$ từ kho dữ liệu bằng các kỹ thuật IR hiệu quả[12]. Các kết quả tìm kiếm này được kết hợp với ngữ cảnh gốc thông qua một khuôn mẫu prompt $g(\cdot)$[13], tạo thành một đầu vào được tăng cường cho LLM

$$\mathbf{x}' = g(\mathbf{c}_1, \cdots, \mathbf{c}_k, \mathbf{x}). \tag{8.69}$$

Sau đó, ta dùng $\mathbf{x}'$ làm ngữ cảnh và dự đoán phần văn bản tiếp theo bằng mô hình $\Pr(\mathbf{y}|\mathbf{x}')$. Một ưu điểm của RAG là ta không cần sửa đổi kiến trúc của LLM, mà chỉ tăng cường đầu vào cho LLM thông qua một hệ thống IR bổ sung. Hình 8.7 so sánh việc sử dụng các loại bộ nhớ ngoài khác nhau trong LLM.

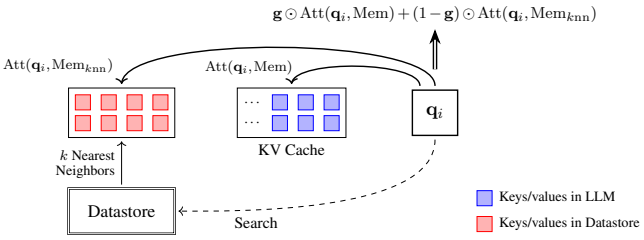
3. Dung lượng bộ nhớ

Một mô hình bộ nhớ trong LLM, dù ở dạng một bộ nhớ đệm khóa–giá trị đơn giản hay một kho dữ liệu ngoài, nhìn chung đều có thể được xem là một bộ mã hóa thông tin ngữ cảnh. Lý tưởng nhất, trước khi nói rằng một mô hình bộ nhớ đại diện được cho toàn bộ ngữ cảnh trong việc dự đoán token, ta phải bảo đảm rằng mô hình có thể tái tạo chính xác bất kỳ phần nào của ngữ cảnh đó. Bộ nhớ đệm KV chuẩn là

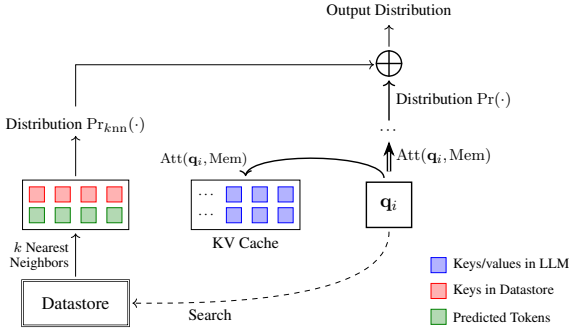
[12] Trong các ứng dụng thực tế, truy vấn thường được tạo bởi một hệ thống sinh truy vấn, hệ thống này có thể mở rộng truy vấn bằng các biến thể của token và ý định truy vấn.

[13] Ví dụ, khuôn mẫu có thể là:

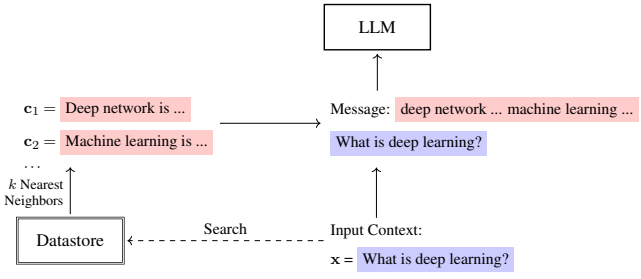
message (thông điệp) = {*$\mathbf{c}_1$*} $\cdots$ {*$\mathbf{c}_k$*}
input (đầu vào): {*$\mathbf{x}$*}
output (đầu ra): ___



(a) k-NN Search Augmented Attention



(b) k-NN Language Modeling



(c) Retrieval-augmented Generation

Hình 8.7: Minh họa các bộ nhớ ngoài (hay kho dữ liệu) cho mô hình hóa ngôn ngữ. (Các nhãn con: (a) chú ý được tăng cường bằng tìm kiếm k-NN; (b) mô hình hóa ngôn ngữ k-NN; (c) sinh văn bản tăng cường truy xuất. Các nhãn khác: "Datastore" = kho dữ liệu, "Search" = tìm kiếm, "k Nearest Neighbors" = k láng giềng gần nhất, "Keys/values in LLM / in Datastore" = khóa/giá trị trong LLM / trong kho dữ liệu, "Predicted Tokens" = các token dự đoán, "Output Distribution" = phân phối đầu ra. Trong (c), $\mathbf{c}_1$ = "Mạng sâu là …", $\mathbf{c}_2$ = "Học máy là …", ngữ cảnh đầu vào $\mathbf{x}$ = "Học sâu là gì?".)

một mô hình như vậy, vì nó giữ lại toàn bộ lịch sử của chuỗi. Trong trường hợp này, mô hình được gọi là có dung lượng (*capacity*) đủ (hay không giới hạn) để ghi nhớ ngữ cảnh. Tuy nhiên, trong nhiều ứng dụng thực tế, việc ghi nhớ trọn vẹn là không cần thiết. Thay vào đó, mục tiêu chính là giúp LLM truy cập được những thông tin ngữ cảnh nổi bật. Do đó, các mô hình bộ nhớ hiệu quả và được nén đã được phát triển, như đã mô tả trong mục này. Lưu ý rằng chuỗi càng dài thì một mô hình bộ nhớ dung lượng thấp càng khó nắm bắt những thông tin ngữ cảnh then chốt. Vì vậy, một thực hành phổ biến là đơn giản tăng dung lượng bộ nhớ khi xử lý ngữ cảnh dài.

Dù các mô hình bộ nhớ dung lượng cao nhìn chung là đáng mong muốn, chúng lại tốn kém về tính toán khi huấn luyện và triển khai. Một tình huống khó khăn xảy ra khi các token đến thành một luồng (*stream*) liên tục, khiến ngữ cảnh tăng lên vô hạn. Phát triển LLM cho các tác vụ dạng luồng như vậy là khó, vì nó đòi hỏi huấn luyện Transformer trên những chuỗi cực dài. Một giải pháp khả dĩ là dùng các cách tiếp cận phi tham số (*non-parametric*), chẳng hạn các phương pháp dựa trên truy xuất. Ví dụ, như đã bàn ở trên, ta có thể dùng một cơ sở dữ liệu vectơ để lưu mọi cặp khóa–giá trị được sinh ra, nhờ đó biểu diễn hiệu quả ngữ cảnh thông qua một mô hình bộ nhớ ngoài. Mặc dù cách này né được giới hạn độ dài ngữ cảnh vốn có của Transformer, việc xây dựng và duy trì các mô hình bộ nhớ ngoài lại gây ra chi phí tính toán đáng kể. Hơn nữa, các mô hình này thường được áp dụng cho những bài toán mà ngữ cảnh được cung cấp trước và giữ nguyên trong khi suy luận, do đó không phù hợp với việc mô hình hóa ngữ cảnh động, dạng luồng.

Trong những trường hợp kích thước ngữ cảnh tăng liên tục, áp dụng các mô hình bộ nhớ kích thước cố định là một cách làm phổ biến. Ví dụ, trong các mô hình hồi quy, một chuỗi có độ dài tùy ý có thể được tóm tắt thành một tập các trạng thái ẩn, dẫn tới chi phí tính toán cố định cho mỗi bước. Mặc dù trong những ứng dụng ban đầu của học sâu vào NLP, các mô hình hồi quy được thấy là không giỏi xử lý các phụ thuộc xa (*long-distance dependencies*) trong mô hình hóa chuỗi, những tiến bộ gần đây đã cho thấy các biến thể của chúng nay đã hiệu quả trong việc mô hình hóa các chuỗi cực dài [Bulatov và cộng sự, 2022; Hutchins và cộng sự, 2022; Munkhdalai và cộng sự, 2024; Ma và cộng sự, 2024].

Không có một định nghĩa chung nào về dung lượng bộ nhớ trong LLM. Một cách đơn giản có thể là xem xét lượng bộ nhớ lưu trữ được dùng để giữ thông tin ngữ cảnh. Ví dụ, dung lượng bộ nhớ có thể được định nghĩa bằng kích thước của bộ nhớ đệm KV trong Transformer hoặc của cơ sở dữ liệu vectơ dùng trong các phương pháp dựa trên truy xuất. Một khái niệm liên quan là độ phức tạp mô hình (*model complexity*). Trong học máy, có nhiều cách định nghĩa độ phức tạp mô hình. Một trong những cách đơn giản nhất là đếm số tham số. Tuy nhiên, cần nhấn mạnh rằng các mô hình bộ nhớ bàn ở đây chủ yếu dùng để lưu trữ thông tin, chứ không phải để thêm các tham số huấn luyện được. Do đó, một mô hình có dung lượng bộ nhớ lớn không nhất thiết phức tạp hơn. Dù vậy, trên thực tế, việc xác định dung lượng của một mô hình bộ nhớ không hề đơn giản. Nói chung, ta cần cân nhắc sự đánh đổi giữa việc tối đa hóa hiệu năng và kiểm soát lượng bộ nhớ chiếm dụng.

Chú giải của người dịch: Bộ nhớ trong và bộ nhớ ngoài – tóm tắt

Bộ nhớ trong giống trí nhớ ngắn hạn trong đầu: nhỏ gọn, nằm ngay trong mô hình (bộ nhớ đệm KV và các biến thể nén của nó). *Bộ nhớ ngoài* giống một thư viện hay cuốn sổ tay: có thể rất lớn, nằm ngoài mô hình, và mô hình "tra cứu" khi cần (k-NN, RAG). Bộ nhớ ngoài không làm tăng số tham số của mô hình, nhưng việc xây dựng chỉ mục và tra cứu lại tốn chi phí riêng.

8.3.4 Chia sẻ giữa các đầu và các tầng

Trong Transformer, bộ nhớ đệm KV là một cấu trúc dữ liệu có thể được điều chỉnh linh hoạt theo nhiều chiều, như số đầu, số tầng và độ dài chuỗi. Ví dụ, xét một LLM có L tầng. Mỗi tầng có τ đầu chú ý, và mỗi đầu tạo ra một đầu ra d_h chiều. Trong khi suy luận, ta lưu khóa và giá trị cho tối đa m token. Độ phức tạp không gian (*space complexity*) của cơ chế lưu đệm này là $O(L \cdot \tau \cdot d_h \cdot m)$. Như đã thấy ở trên, độ phức tạp này có thể được giảm bằng cách lưu đệm khóa và giá trị cho ít token hơn. Ví dụ, trong chú ý cửa sổ trượt (*sliding window attention*), một cửa sổ có kích thước cố định được dùng để lưu đệm khóa và giá trị trong ngữ cảnh cục bộ. Mô hình cửa sổ trượt có độ phức tạp không gian $O(L \cdot \tau \cdot d_h \cdot m_w)$, với m_w là kích thước cửa sổ.

Chú giải của người dịch: Ký hiệu $O(\cdot)$ và một ví dụ số

Ký hiệu $O(L \cdot \tau \cdot d_h \cdot m)$ (đọc là "bậc của …") cho biết lượng bộ nhớ tăng *tỉ lệ* với tích các đại lượng đó, bỏ qua hằng số. Ví dụ, một mô hình có $L = 32$ tầng, $\tau = 32$ đầu, $d_h = 128$, ngữ cảnh $m = 4096$ token cần lưu $32 \cdot 32 \cdot 128 \cdot 4096 \approx 5.4 \times 10^8$ số cho khóa, và chừng đó cho giá trị; với mỗi số chiếm 2 byte, tổng cộng khoảng 2 GB cho *một* chuỗi. Giảm bất kỳ thừa số nào (số token, số đầu, số tầng) đều giảm bộ nhớ theo cùng tỉ lệ.

Ngoài việc giảm m, ta cũng có thể giảm kích thước bộ nhớ đệm KV theo các chiều khác. Một cách làm được dùng rộng rãi là cho phép chia sẻ giữa các đầu trong tự chú ý đa đầu. Nhắc lại từ Mục 8.1.1 rằng tự chú ý đa đầu dùng nhiều bộ truy vấn, khóa và giá trị (mỗi bộ gọi là một *đầu*), mỗi bộ thực hiện cơ chế chú ý QKV như thường lệ. Điều này có thể được biểu diễn là

$$\text{Output} \;=\; \text{Merge}(\text{head}_1, \cdots, \text{head}_\tau)\mathbf{W}^{\text{head}}, \tag{8.70}$$

trong đó $\text{head}_j \in \mathbb{R}^{d_h}$ là đầu ra của đầu thứ j, được tính bằng hàm chú ý QKV chuẩn

$$\text{head}_j = \text{Att}_{\text{qkv}}(\mathbf{q}_i^{[j]}, \mathbf{K}_{\leq i}^{[j]}, \mathbf{V}_{\leq i}^{[j]}). \tag{8.71}$$

Ở đây $\mathbf{q}_i^{[j]}$ là vectơ truy vấn tại vị trí i, còn $\mathbf{K}_{\leq i}^{[j]}$ và $\mathbf{V}_{\leq i}^{[j]}$ là các ma trận khóa và giá trị tương ứng tính tới vị trí i, với mỗi vectơ được chiếu vào không gian con đặc trưng (*feature subspace*) thứ j. Vì vậy mô hình chú ý đa đầu này có thể được hiểu là thực hiện phép chú ý song song trên một nhóm các không gian con đặc trưng (xem Hình 8.8 (b)). Bộ nhớ đệm KV cần giữ khóa và giá trị cho tất cả các đầu này, tức là $\{(\mathbf{K}_{\leq i}^{[1]}, \mathbf{V}_{\leq i}^{[1]}), \cdots, (\mathbf{K}_{\leq i}^{[\tau]}, \mathbf{V}_{\leq i}^{[\tau]})\}$.

Một cải tiến của mô hình chú ý đa đầu, gọi là **chú ý đa truy vấn** (*multi-query attention, MQA*), là chia sẻ khóa và giá trị giữa các đầu, trong khi vẫn giữ truy vấn riêng cho từng đầu [Shazeer, 2019]. Trong MQA, chỉ có một bộ khóa và giá trị duy nhất $(\mathbf{K}_{\leq i}, \mathbf{V}_{\leq i})$. Ngoài ra, có τ truy vấn $\{\mathbf{q}_i^{[1]}, \cdots, \mathbf{q}_i^{[\tau]}\}$, mỗi truy vấn ứng với một đầu khác nhau. Với mỗi đầu, ta có

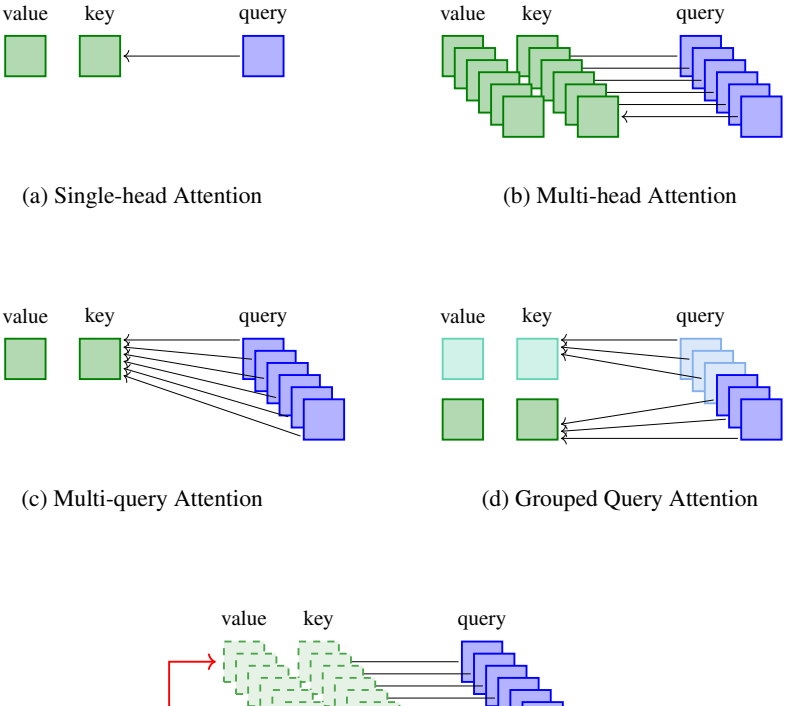
$$\text{head}_j = \text{Att}_{\text{qkv}}(\mathbf{q}_i^{[j]}, \mathbf{K}_{\leq i}, \mathbf{V}_{\leq i}). \tag{8.72}$$

Hình 8.8 (c) minh họa mô hình này. Nhờ chia sẻ khóa và giá trị, kích thước bộ nhớ đệm KV sẽ là $O(L \cdot d_h \cdot m)$.

Chú ý truy vấn theo nhóm (*grouped query attention, GQA*) là một mở rộng tự nhiên của chú ý đa đầu và MQA [Ainslie và cộng sự, 2023]. Trong GQA, các đầu được chia thành n_g nhóm, mỗi nhóm ứng với một bộ khóa và giá trị dùng chung. Do đó ta có n_g bộ khóa và giá trị $\{(\mathbf{K}_{\leq i}^{[1]}, \mathbf{V}_{\leq i}^{[1]}), \cdots, (\mathbf{K}_{\leq i}^{[n_g]}, \mathbf{V}_{\leq i}^{[n_g]})\}$. Xem Hình 8.8 (d) để có minh họa. Gọi $g(j)$ là mã số nhóm của đầu thứ j. Mô hình GQA có thể được biểu diễn là

$$\text{head}_j = \text{Att}_{\text{qkv}}(\mathbf{q}_i^{[j]}, \mathbf{K}_{\leq i}^{[g(j)]}, \mathbf{V}_{\leq i}^{[g(j)]}). \tag{8.73}$$

Kích thước bộ nhớ đệm KV của GQA là $O(L \cdot n_g \cdot d_h \cdot m)$. Một lợi ích của GQA là ta có thể đánh đổi giữa hiệu quả tính toán và khả năng biểu đạt (*expressiveness*) của mô hình bằng cách điều chỉnh n_g. Khi $n_g = \tau$, mô hình trở thành mô hình chú ý đa đầu chuẩn. Ngược lại, khi $n_g = 1$, nó trở thành mô hình MQA.



(a) Single-head Attention

(b) Multi-head Attention

(c) Multi-query Attention

(d) Grouped Query Attention

(e) Cross-layer Multi-head Attention

Hình 8.8: Minh họa phép chú ý QKV dựa trên các cơ chế đa đầu và chia sẻ khác nhau. (a) = chú ý một đầu, và (b–e) = chú ý với nhiều đầu. (Các nhãn con: (a) chú ý một đầu; (b) chú ý đa đầu; (c) chú ý đa truy vấn; (d) chú ý truy vấn theo nhóm; (e) chú ý đa đầu xuyên tầng. "value/key/query" = giá trị/khóa/truy vấn; "Sharing" = chia sẻ; "Layer l" = tầng l.)

Chú giải của người dịch: So sánh MHA, MQA, GQA bằng số

Giả sử mỗi tầng có $\tau = 8$ đầu. Chú ý đa đầu chuẩn lưu 8 bộ khóa–giá trị. MQA chỉ lưu 1 bộ (giảm 8 lần bộ nhớ đệm), nhưng 8 đầu buộc phải "nhìn" cùng một khóa–giá trị nên mô hình kém linh hoạt hơn. GQA với $n_g = 2$: đầu 1–4 dùng chung bộ 1, đầu 5–8 dùng chung bộ 2 (tức $g(1) = \cdots = g(4) = 1$, $g(5) = \cdots = g(8) = 2$), lưu 2 bộ – giảm 4 lần mà vẫn giữ được một phần sự đa dạng. Truy vấn thì vẫn riêng cho từng đầu vì truy vấn không cần lưu đệm (chỉ dùng cho token hiện tại).

Việc chia sẻ cũng có thể được thực hiện giữa các tầng. Phương pháp như vậy thuộc họ các phương pháp chia sẻ trọng số (*shared weight*) và chia sẻ giá trị kích hoạt (*shared activation*), vốn đã được dùng rộng rãi trong Transformer [Dehghani và cộng sự, 2018; Lan và cộng sự, 2020]. Ví dụ, ta có thể chia sẻ các giá trị kích hoạt KV hoặc các trọng số chú ý giữa các tầng để giảm cả lượng tính toán lẫn bộ nhớ chiếm dụng [Xiao và cộng sự, 2019; Brandon và cộng sự, 2024]. Hình 8.8 (e) minh họa phương pháp này, trong đó một truy vấn ở một tầng truy cập trực tiếp bộ nhớ đệm KV của một tầng thấp hơn.

8.3.5 Ngoại suy và nội suy vị trí

Vì các tầng Transformer bất biến theo thứ tự (*order-invariant*) đối với đầu vào, ta cần một cách nào đó để mã hóa thông tin vị trí vào các token đầu vào. Để làm điều này, cách phổ biến là cộng các vectơ nhúng vị trí vào các vectơ nhúng token, rồi đưa các vectơ nhúng kết hợp này vào chồng các tầng Transformer. Khi đó, vectơ nhúng tại vị trí i có thể được biểu diễn là

$$\mathbf{e}_i \;=\; \mathbf{x}_i + \mathrm{PE}(i), \tag{8.74}$$

trong đó $\mathbf{x}_i \in \mathbb{R}^d$ là vectơ nhúng token, và $\mathrm{PE}(i) \in \mathbb{R}^d$ là vectơ nhúng vị trí tại vị trí i. Vì vectơ nhúng token $\mathbf{x}_i$ không phụ thuộc vị trí, vectơ nhúng vị trí $\mathrm{PE}(i)$ mã hóa ngữ cảnh tuyệt đối hoặc tương đối của token. Một cách cài đặt trực tiếp là coi $\mathrm{PE}(i)$ là một tập các biến học được, được huấn luyện cùng với các tham số còn lại của mô hình. Điều này cho phép mô hình học một biểu diễn vectơ riêng cho mỗi vị trí rời rạc, nhờ đó phân biệt các token dựa trên chỗ chúng xuất hiện trong chuỗi.

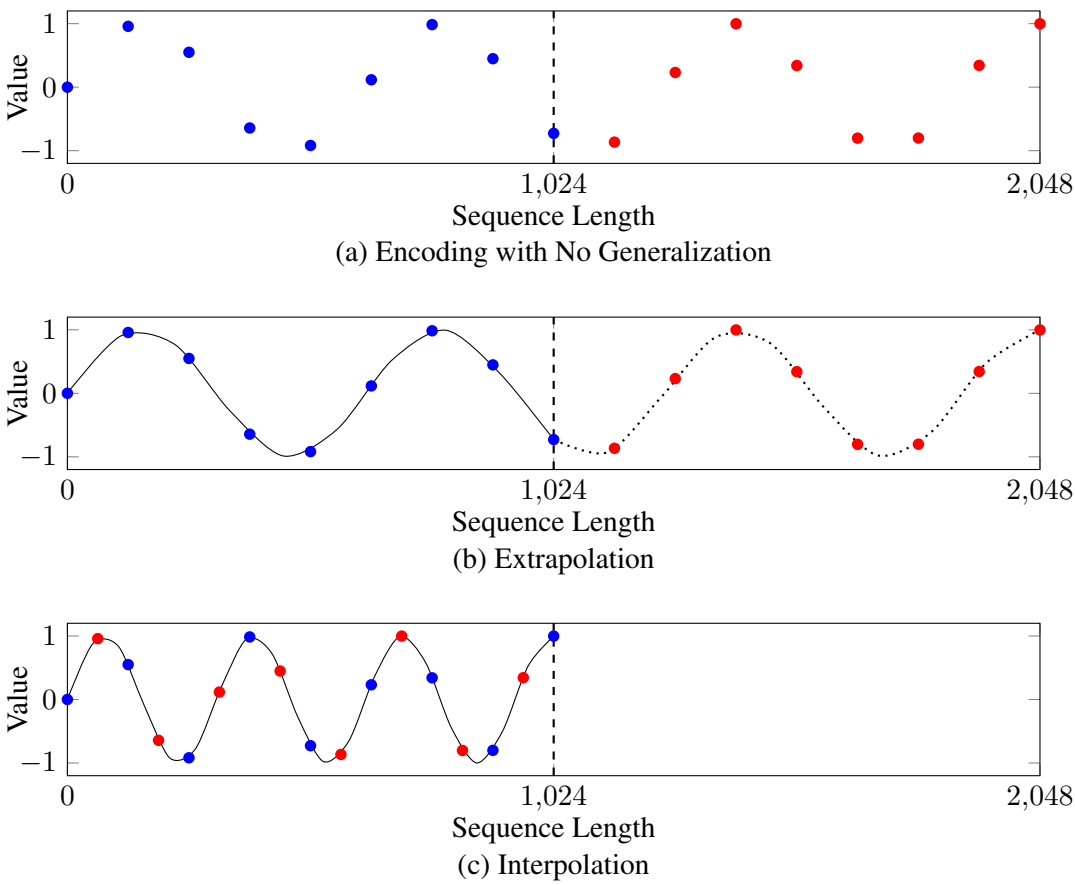
Dùng các vectơ học được để biểu diễn vị trí tuyệt đối hoạt động khá tốt khi độ dài chuỗi gặp phải lúc suy luận gần khớp với độ dài đã thấy khi huấn luyện. Tuy nhiên, trong tiền huấn luyện LLM thực tế, ta phải áp đặt các giới hạn độ dài tối đa nghiêm ngặt (chẳng hạn m_l) lên các chuỗi huấn luyện để giữ cho việc tính toán khả thi, nhưng ta lại thường muốn áp dụng mô hình đã huấn luyện cho những chuỗi dài hơn đáng kể khi triển khai ở các tác vụ hạ nguồn. Trong những tình huống như vậy, dùng các vectơ nhúng vị trí học được có nhược điểm rõ ràng, vì không có vectơ nhúng nào đã được huấn luyện cho những vị trí không xuất hiện trong giai đoạn huấn luyện.

Một cách tiếp cận khác để mô hình hóa thông tin vị trí là phát triển các vectơ nhúng vị trí có khả năng tổng quát hóa (*generalize*): một khi đã huấn luyện, mô hình nhúng có thể được dùng để xử lý các chuỗi dài hơn. Giả sử ta huấn luyện một mô hình nhúng vị trí trên các chuỗi có độ dài tối đa m_l, và ta muốn áp dụng mô hình đã huấn luyện cho một chuỗi có độ dài m ($m \gg m_l$). Nếu mô hình chỉ được huấn luyện trên một khoảng vị trí giới hạn thì mô hình này sẽ đơn giản là thất bại khi xử lý dữ liệu mới nằm ngoài khoảng đó. Xem Hình 8.9 (a) để thấy một minh họa, trong đó mô hình nhúng đã học không thể mô hình hóa các điểm dữ liệu nằm ngoài miền huấn luyện nếu nó thiếu khả năng ngoại suy.

Có một số cách tiếp cận giúp các mô hình nhúng vị trí tổng quát hóa. Chúng có thể được chia thành hai nhóm.

- **Ngoại suy** (*Extrapolation*). Mô hình học được trên các điểm dữ liệu đã quan sát (tức là các vị trí) có thể được dùng trực tiếp để gán giá trị có ý nghĩa cho các điểm dữ liệu nằm ngoài khoảng ban đầu. Chẳng hạn, nếu một mô hình học được một quan hệ hàm liên tục cho các vị trí $\{1, \cdots, 10\}$ thì về lý thuyết nó có thể tính chính hàm đó tại vị trí 15, mà vẫn giữ đúng khoảng cách tương đối và các tính chất hình học so với các token phía trước. Hình 8.9 (b) minh họa khái niệm này, trong đó một hàm được khớp trên miền huấn luyện được kéo dài một cách trơn tru để ước lượng giá trị cho các tọa độ ngoài phân phối (*out-of-distribution*).
- **Nội suy** (*Interpolation*). Cách tiếp cận này ánh xạ một khoảng điểm dữ liệu lớn hơn vào khoảng quan sát ban đầu. Ví dụ, giả sử ta có một mô hình được thiết kế cho các số trong khoảng $[1, 10]$.

Khi gặp một khoảng mới $[1, 20]$, ta có thể thu nhỏ nó bằng cách chia mọi số cho 2, nhờ đó đưa tất cả các số vào $[1, 10]$. Phép co giãn này cho phép ta dùng mô hình đã huấn luyện trên khoảng $[1, 10]$ để mô tả các điểm dữ liệu trong khoảng mở rộng $[1, 20]$. Xem Hình 8.9 (c) để có minh họa cho cách tiếp cận này.



Hình 8.9: Minh họa các hành vi khác nhau của vectơ nhúng vị trí trên một dải độ dài chuỗi liên tục. Các điểm màu xanh lam là những vị trí được quan sát tường minh trong khi huấn luyện, còn các điểm màu đỏ là những vị trí chưa thấy gặp phải lúc suy luận. Trong hình con (a), mô hình nhúng ghi nhớ miền huấn luyện và không thể tổng quát hóa. Trong các hình con (b) và (c), mô hình tổng quát hóa thành công tới các độ dài chưa thấy, lần lượt bằng ngoại suy và nội suy. (Nhãn: "Value" = giá trị, "Sequence Length" = độ dài chuỗi; (a) mã hóa không tổng quát hóa; (b) ngoại suy; (c) nội suy.)

Chú giải của người dịch: Ngoại suy và nội suy – liên hệ THPT

Hãy hình dung bạn biết giá trị của hàm số $y = \sin x$ tại các điểm $x = 1, 2, \ldots, 10$. *Ngoại suy* là dùng chính công thức đó để tính y tại $x = 15$ – nằm ngoài vùng đã biết. *Nội suy* thì làm ngược lại: khi cần xử lý x từ 1 đến 20, ta "nén" trục bằng cách thay x bởi $x/2$, nên mọi điểm mới rơi vào vùng $[0{,}5;\ 10]$ quen thuộc (nằm *giữa* các điểm đã biết). Cái giá phải trả của nội suy là các vị trí kề nhau trở nên "sát" nhau hơn, nên mô hình phải phân biệt những khác biệt nhỏ hơn.

Thực ra, các vectơ nhúng vị trí trong nhiều hệ thống đã đạt được một mức tổng quát hóa nhất định.

Ví dụ, mã hóa hình sin (*sinusoidal encoding*), phương pháp nhúng vị trí phổ biến nhất, dùng các hàm sin và cos vốn có thể tự nhiên mở rộng cho chuỗi có độ dài bất kỳ. Mặc dù cách tiếp cận này có vẻ trực tiếp và đơn giản, nó không hoạt động tốt khi ta kéo dài đáng kể các chuỗi cần xử lý. Trong tiểu mục này, ta sẽ bàn về một số phương pháp thay thế dựa trên ngoại suy hoặc nội suy.

1. Chú ý với các độ lệch học được

Một vấn đề với Phương trình (8.74) là mô hình nhúng xử lý mỗi token một cách độc lập và do đó bỏ qua khoảng cách giữa các token khác nhau. Một cải tiến phổ biến cho mô hình này, gọi là nhúng vị trí tương đối (*relative positional embedding*), là xét quan hệ theo cặp giữa các token [Shaw và cộng sự, 2018]. Ý tưởng chung là tính độ lệch giữa hai vị trí bất kỳ và đưa nó vào mô hình tự chú ý. Một trong những dạng đơn giản nhất của tự chú ý với nhúng vị trí tương đối được cho bởi

$$\text{Att}_{\text{qkv}}(\mathbf{q}_i, \mathbf{K}_{\leq i}, \mathbf{V}_{\leq i}) = \sum_{j=0}^{i} \alpha(i,j)\mathbf{v}_j \tag{8.75}$$

$$\alpha(i,j) = \text{Softmax}\left(\frac{\mathbf{q}_i \mathbf{k}_j^\top + \text{PE}(i,j)}{\sqrt{d}} + \text{Mask}(i,j)\right). \tag{8.76}$$

Điểm khác biệt duy nhất giữa mô hình này và mô hình tự chú ý gốc là trong mô hình mới, một số hạng độ lệch $\text{PE}(i,j)$ được cộng vào tích truy vấn–khóa. Về mặt trực giác, $\text{PE}(i,j)$ có thể được hiểu là một mức phạt khoảng cách (*distance penalty*) cho cặp vị trí i và j. Khi khoảng cách giữa i và j tăng, $\text{PE}(i,j)$ giảm.

$\text{PE}(i,j)$ có thể được định nghĩa theo nhiều cách khác nhau. Ở đây, ta xét phiên bản nhúng vị trí tương đối của T5, gọi là độ lệch T5 (*T5 bias*) [Raffel và cộng sự, 2020]. Với mỗi cặp truy vấn $\mathbf{q}_i$ và khóa $\mathbf{k}_j$, độ lệch vị trí giữa chúng được định nghĩa là[14]

$$d(i,j) \;=\; i - j. \tag{8.77}$$

Một cách xây dựng ngây thơ cho độ lệch $\text{PE}(i,j)$ là gán một số vô hướng học được riêng cho mỗi độ lệch rời rạc có thể có, tức là $\text{PE}(i,j) = u_{i-j}$, trong đó u_{i-j} là tham số gắn với đúng khoảng cách $i-j$. Tuy nhiên, chỉ đơn giản gán một giá trị riêng cho mỗi độ lệch sẽ giới hạn mô hình vào những độ lệch đã quan sát trong khi huấn luyện. Khi $i-j$ lớn hơn độ lệch lớn nhất đã được huấn luyện, mô hình không thể tổng quát hóa.

Độ lệch T5 đưa ra một phiên bản tổng quát hóa của cách tiếp cận này. Thay vì cấp một tham số riêng cho mọi độ lệch nguyên có thể có, nó gom các độ lệch vị trí này vào các "thùng" (*buckets*) riêng biệt, trong đó mọi độ lệch được ánh xạ vào cùng một thùng sẽ dùng chung một tham số học được. Cụ thể, với n_b thùng, chiến lược phân bổ được định nghĩa như sau:

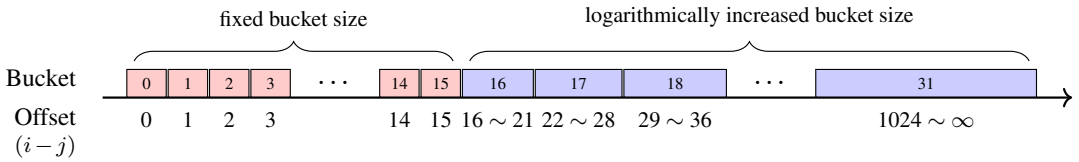
- Với nửa đầu số thùng (chỉ số từ 0 đến $\lfloor \frac{n_b}{2} \rfloor - 1$), phép ánh xạ là một–một. Mỗi độ lệch được ánh xạ vào một thùng riêng: thùng $0 \leftrightarrow$ độ lệch 0, thùng $1 \leftrightarrow$ độ lệch 1, và cứ thế. Ta có thể biểu diễn phép ánh xạ tuyến tính này là $b(i-j) = i-j$.
- Với nửa còn lại của số thùng (chỉ số từ $\lfloor \frac{n_b}{2} \rfloor$ đến $n_b - 1$), kích thước thùng tăng theo hàm logarit. Kết quả là các độ lệch lớn hơn được gom vào những ngăn ngày càng rộng. Với một độ lệch $i - j \geq \lfloor \frac{n_b}{2} \rfloor$, chỉ số thùng được tính là:

$$b(i-j) \;=\; \lfloor \frac{n_b}{2} \rfloor + \left\lfloor \frac{\log(i-j) - \log(\lfloor \frac{n_b}{2} \rfloor)}{\log(\text{dist}_{\max}) - \log(\lfloor \frac{n_b}{2} \rfloor)} \cdot \left(n_b - \lfloor \frac{n_b}{2} \rfloor - 1\right) \right\rfloor, \tag{8.78}$$

[14] Với mô hình hóa ngôn ngữ, một truy vấn chỉ được phép chú ý tới ngữ cảnh bên trái của nó, nên ta có $i - j \geq 0$. Trong trường hợp tổng quát hơn của tự chú ý, khi một token có thể chú ý tới mọi token trong chuỗi, ta có thể có độ lệch âm khi $i < j$.

trong đó $\text{dist}_{\max}$ là một siêu tham số cho biết ngưỡng độ lệch tối đa dự kiến.

- Khi $i - j \geq \text{dist}_{\max}$, độ lệch bị cắt và được đặt vào thùng cuối cùng $n_b - 1$. Nói cách khác, thùng này đóng vai trò "thu gom tất cả" cho mọi khoảng cách lớn tùy ý vượt quá độ dài chuỗi dự kiến.



Hình 8.10: Minh họa việc phân phối các độ lệch truy vấn–khóa vào các thùng trong mô hình T5 ($n_b = 32$ và $\text{dist}_{\max} = 1024$). Các hộp biểu diễn các thùng. Ở nửa đầu số thùng, ta dùng kích thước thùng cố định. Ở nửa sau, ta tăng kích thước thùng theo hàm logarit. Thùng cuối cùng chứa mọi độ lệch truy vấn–khóa chưa được các thùng trước bao phủ. (Nhãn: "Bucket" = thùng, "Offset" = độ lệch, "fixed bucket size" = kích thước thùng cố định, "logarithmically increased bucket size" = kích thước thùng tăng theo logarit.)

Gộp lại, hàm chia thùng được định nghĩa là

$$b(i-j) = \begin{cases} i-j & \text{nếu } 0 \leq i-j < \lfloor \frac{n_b}{2} \rfloor, \\ \min\left(n_b - 1,\ \lfloor \frac{n_b}{2} \rfloor + \left\lfloor \frac{\log(i-j) - \log(\lfloor \frac{n_b}{2} \rfloor)}{\log(\text{dist}_{\max}) - \log(\lfloor \frac{n_b}{2} \rfloor)} \cdot \left(n_b - \lfloor \frac{n_b}{2} \rfloor - 1\right) \right\rfloor \right) & \text{nếu } i-j \geq \lfloor \frac{n_b}{2} \rfloor. \end{cases} \tag{8.79}$$

Hình 8.10 minh họa các thùng này. Ta thấy ở nửa đầu số thùng, mỗi thùng chỉ gắn với một giá trị của $i - j$, còn ở nửa sau, kích thước thùng tăng lên khi $i - j$ tăng. Thùng cuối cùng được thiết kế để xử lý các chuỗi có độ dài tùy ý.

Chú giải của người dịch: Tính thử chỉ số thùng

Ký hiệu $\lfloor x \rfloor$ là phần nguyên (làm tròn xuống). Lấy $n_b = 32$, $\text{dist}_{\max} = 1024$ như Hình 8.10, khi đó $\lfloor n_b/2 \rfloor = 16$ và $n_b - 16 - 1 = 15$. Với độ lệch $i - j = 5 < 16$: thùng 5. Với $i - j = 64$: $\frac{\log 64 - \log 16}{\log 1024 - \log 16} = \frac{\log 4}{\log 64} = \frac{2}{6} \approx 0.333$ (logarit cơ số nào cũng được vì là tỉ số), nhân 15 được 5, nên $b = 16 + 5 = 21$. Với $i - j = 5000 > 1024$: công thức cho số lớn hơn 31, nên min cắt về thùng 31. Như vậy các khoảng cách gần được phân biệt rất "tỉ mỉ", còn các khoảng cách xa được gộp thô – giống việc ta nói "hôm qua", "hôm kia" nhưng chỉ nói "mấy năm trước" cho chuyện xa.

Mọi khoảng cách tương đối được ánh xạ vào cùng một thùng $b(i - j)$ dùng chung một số vô hướng độ lệch học được $u_{b(i-j)}$. Thay $\text{PE}(i, j) = u_{b(i-j)}$ trở lại Phương trình (8.76), việc tính trọng số chú ý cuối cùng trở thành[15]

$$\alpha(i, j) = \text{Softmax}\left(\frac{\mathbf{q}_i \mathbf{k}_j^\top + u_{b(i-j)}}{\sqrt{d}} + \text{Mask}(i, j)\right), \tag{8.81}$$

Các tham số $\{u_0, \cdots, u_{n_b-1}\}$ được học cùng với các tham số khác của mô hình. Cần nhấn mạnh rằng mô hình này có thể tổng quát hóa tới các chuỗi dài. Lý do là các $\text{PE}(i, j)$ có độ lệch truy vấn–khóa

[15] Lưu ý rằng trong mô hình T5 của Raffel và cộng sự [2020], phép co giãn lại tích truy vấn–khóa bị loại bỏ. Trọng số chú ý $\alpha(i, j)$ khi đó được cho bởi

$$\alpha(i, j) = \text{Softmax}(\mathbf{q}_i \mathbf{k}_j^\top + u_{b(i-j)} + \text{Mask}(i, j)). \tag{8.80}$$

tương tự nhau dùng chung một tham số, và chiến lược chia sẻ này đặc biệt quan trọng để đạt được khả năng tổng quát hóa tốt, bởi các độ lệch truy vấn–khóa lớn hiếm khi xuất hiện trong huấn luyện. Trên thực tế, ta thường đặt n_b ở mức vừa phải, nhờ đó giúp kiểm soát hiện tượng quá khớp của các mô hình nhúng vị trí.

2. Chú ý với các độ lệch cố định

Các mô hình nhúng vị trí tương đối dựa trên một tập các độ lệch học được, cộng vào tích truy vấn–khóa trong tự chú ý. Một cách tiếp cận khác là gán các giá trị cố định cho những độ lệch này thông qua các quy tắc kinh nghiệm định sẵn, thay vì học chúng từ dữ liệu. Một lợi ích của cách tiếp cận dựa trên quy tắc kinh nghiệm này là nó không phụ thuộc vào huấn luyện, nên có thể áp dụng trực tiếp cho bất kỳ chuỗi nào một khi các độ lệch đã được đặt.

Một ví dụ cho cách tiếp cận như vậy là phương pháp của Press và cộng sự [2022], gọi là **chú ý với độ lệch tuyến tính** (*attention with linear biases*), viết tắt là **ALiBi**. Trong cách tiếp cận ALiBi, số hạng độ lệch được định nghĩa là độ lệch truy vấn–khóa được co giãn và lấy dấu âm

$$\begin{aligned}\text{PE}(i, j) &= -\beta \cdot (i - j) \\ &= \beta \cdot (j - i),\end{aligned} \tag{8.82}$$

trong đó β là hệ số co giãn. Cộng số hạng này vào tích truy vấn–khóa, ta thu được một dạng mới của trọng số chú ý

$$\alpha(i, j) \;=\; \text{Softmax}\left(\frac{\mathbf{q}_i \mathbf{k}_j^\top + \beta \cdot (j - i)}{\sqrt{d}} + \text{Mask}(i, j)\right). \tag{8.83}$$

Mô hình này có thể được hiểu là áp đặt một mức phạt tuyến tính lên điểm chú ý $\mathbf{q}_i \mathbf{k}_j^\top$ cho mỗi bước khóa j dịch xa khỏi truy vấn i. Xem Hình 8.11 để so sánh độ lệch T5 và độ lệch ALiBi.

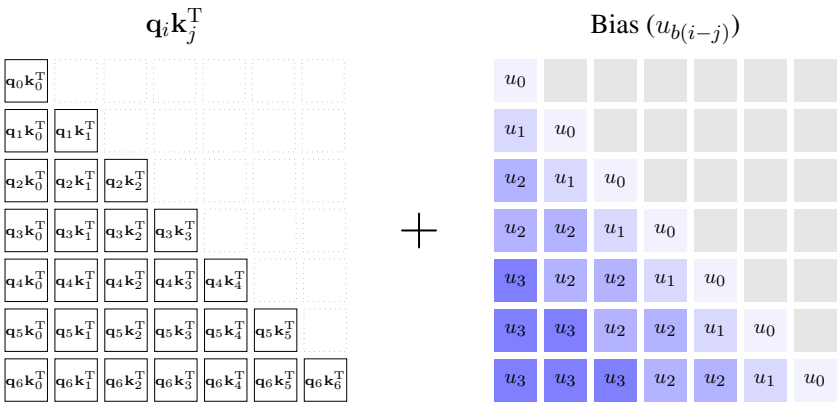
Về nguyên tắc, siêu tham số vô hướng β có thể được điều chỉnh bằng tìm kiếm lưới (*grid search*) trên một tập dữ liệu kiểm định. Tuy nhiên, Press và cộng sự [2022] đã chứng minh rằng việc đặt β một cách có hệ thống cho các đầu chú ý mang lại hiệu năng mạnh trên nhiều tác vụ khác nhau mà không cần điều chỉnh gì. Cụ thể, với một tầng tự chú ý đa đầu gồm n_{head} đầu, các giá trị β giảm theo cấp số nhân. Số vô hướng cho đầu thứ k được định nghĩa là

$$\beta_k \;=\; 2^{-\frac{8k}{n_{\text{head}}}}. \tag{8.84}$$

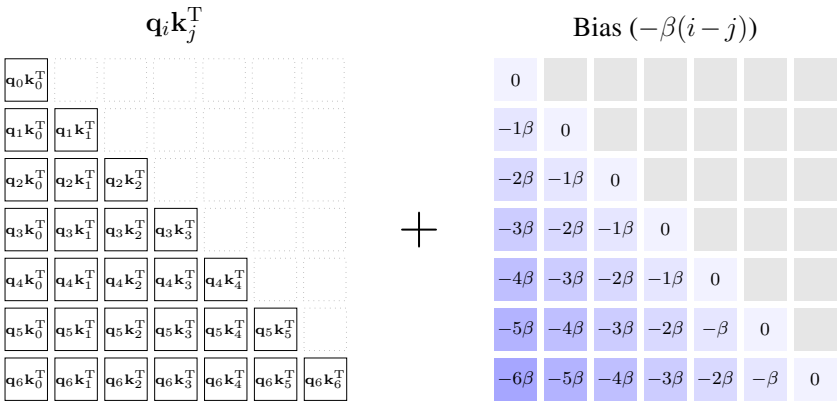
Chú giải của người dịch: ALiBi bằng số

Với $n_{\text{head}} = 8$, theo (8.84) ta có $\beta_k = 2^{-k}$: đầu 1 có $\beta = 1/2$, đầu 2 có $1/4$, …, đầu 8 có $1/256$ – đúng là một cấp số nhân công bội $1/2$ mà bạn đã học. Với đầu 1, khóa cách truy vấn 4 bước bị trừ $4 \cdot 0.5 = 2$ điểm trước khi qua Softmax; vì Softmax dùng hàm mũ, trừ 2 điểm nghĩa là trọng số bị nhân với $e^{-2} \approx 0.135$. Đầu có β lớn vì vậy chủ yếu "nhìn gần", còn đầu có β nhỏ vẫn "nhìn xa" được. Vì mức phạt chỉ phụ thuộc vào $i - j$ theo một công thức cố định, ta tính được nó cho mọi độ dài, kể cả độ dài chưa từng gặp khi huấn luyện.

Cách tiếp cận ALiBi cung cấp một dạng đơn giản của nhúng vị trí tương đối. Có những phương pháp tương tự khác để thiết kế độ lệch truy vấn–khóa dựa trên độ lệch $i - j$. Bảng 8.4 so sánh các độ lệch như vậy. Nhân tiện, đáng lưu ý rằng dạng của vế phải Phương trình (8.82) tương tự các đặc trưng dựa trên khoảng cách được dùng trong các hệ thống dựa trên đặc trưng truyền thống. Ví dụ, trong các hệ thống dịch máy thống kê (*statistical machine translation*), những đặc trưng như vậy được dùng rộng rãi để mô hình hóa bài toán sắp xếp lại trật tự từ (*word reordering*), cho ra những mô hình tổng quát hóa tốt trên nhiều tác vụ dịch khác nhau [Koehn, 2010].



(a) The T5 bias ($n_b = 3$ and $\text{dist}_{\max} = 5$)



(b) The ALiBi bias

Hình 8.11: Các tích truy vấn–khóa cùng với độ lệch (trên = độ lệch T5 và dưới = độ lệch ALiBi). Thang màu của các độ lệch đi từ xanh nhạt, biểu thị giá trị tuyệt đối nhỏ, tới xanh đậm, biểu thị giá trị tuyệt đối lớn. (Nhãn con: (a) độ lệch T5 ($n_b = 3$ và $\text{dist}_{\max} = 5$); (b) độ lệch ALiBi. "Bias" = độ lệch.)

Bảng 8.4: Các độ lệch truy vấn–khóa dùng làm nhúng vị trí tương đối. β, β_1, β_2, $\bar{d}$ và m_{len} là các siêu tham số. Trong mô hình T5, $b(i-j)$ là thùng được gán cho $i-j$. Trong mô hình FIRE, $\psi(\cdot)$ là một hàm chặn đơn điệu tăng, chẳng hạn $\psi(x) = \log(cx+1)$, còn $f(\cdot)$ là một FFN.

Mô hình	Độ lệch truy vấn–khóa ($\text{PE}(i,j)$)
T5 [Raffel và cộng sự, 2020]	$u_{b(i-j)}$
ALiBi [Press và cộng sự, 2022]	$-\beta \cdot (i-j)$
Kerple [Chi và cộng sự, 2022]	$-\beta_1(i-j)^{\beta_2}$ (dạng lũy thừa) $-\beta_1 \log(1+\beta_2(i-j))$ (dạng logarit)
Sandwich [Chi và cộng sự, 2023]	$\sum_{k=1}^{\bar{d}/2} \cos\left((i-j)/10000^{2k/\bar{d}}\right)$
FIRE [Li và cộng sự, 2024b]	$f\left(\psi(i-j)/\psi(\max(m_{\text{len}}, i))\right)$

3. Nhúng vị trí xoay

Tương tự các vectơ nhúng hình sin, nhúng vị trí xoay (*rotary positional embeddings*) dùng các thang tần số định sẵn [Su và cộng sự, 2024]. Nhắc lại rằng trong mô hình nhúng hình sin, các vị trí tuyệt đối được biểu diễn bằng các vectơ cố định tạo thành từ các hàm sin và cos với tần số khác nhau, rồi được cộng vào các vectơ nhúng token. Ngược lại, nhúng vị trí xoay mô hình hóa ngữ cảnh vị trí bằng cách áp dụng các phép quay có ý nghĩa hình học lên vectơ nhúng token trong miền số phức. Điều này dẫn tới một cách biểu diễn dựa trên các tương tác dạng nhân:

$$\mathbf{e}_i \;=\; \mathbf{x}_i R(i), \tag{8.85}$$

trong đó $R(i) \in \mathbb{R}^{d\times d}$ là ma trận quay (*rotation matrix*) biểu diễn các phép quay thực hiện trên vectơ nhúng token $\mathbf{x}_i \in \mathbb{R}^d$.

Để đơn giản, trước hết ta xét các vectơ nhúng chỉ có hai chiều, rồi sẽ quay lại bàn về cách biểu diễn tổng quát hơn sau. Giả sử ta có một vectơ nhúng token hai chiều được biểu diễn dưới dạng vectơ hàng $\mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix}$. Ta có thể biểu diễn nó như một vectơ trong mặt phẳng, có gốc tại gốc tọa độ $(0, 0)$ và ngọn tại (x_1, x_2). Một phép quay ngược chiều kim đồng hồ của vectơ này là thao tác di chuyển vectơ quanh gốc tọa độ trong khi giữ nguyên độ dài của nó, như minh họa trong Hình 8.12 (a). Mức độ quay thường được xác định bởi một góc cụ thể, ký hiệu là θ. Phép quay có thể được biểu diễn bằng toán học dưới dạng

$$\begin{aligned}\mathrm{Ro}(\mathbf{x}, \theta) &= \mathbf{x}R_\theta \\ &= \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \\ &= \begin{bmatrix} \cos\theta \cdot x_1 - \sin\theta \cdot x_2 & \sin\theta \cdot x_1 + \cos\theta \cdot x_2 \end{bmatrix}, \end{aligned} \tag{8.86}$$

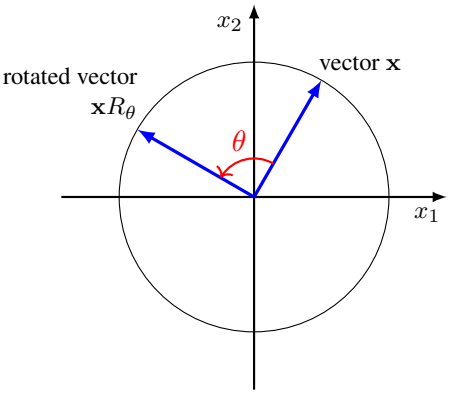
trong đó $R_\theta = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$ là ma trận quay. Nếu thực hiện hai hay nhiều phép quay trên cùng một vectơ, ta có thể quay vectơ đi xa hơn nữa. Điều này suy ra từ việc hợp của các phép quay liên tiếp bản thân nó cũng là một phép quay. Một cách hình thức hơn, việc quay một vectơ một góc θ, lặp lại t lần, có thể được biểu diễn là

$$\begin{aligned}\mathrm{Ro}(\mathbf{x}, t\theta) &= \mathbf{x}R_{t\theta} \\ &= \begin{bmatrix} \cos t\theta \cdot x_1 - \sin t\theta \cdot x_2 & \sin t\theta \cdot x_1 + \cos t\theta \cdot x_2 \end{bmatrix}. \end{aligned} \tag{8.87}$$

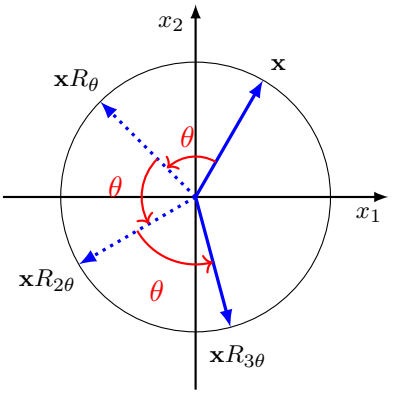
Chú giải của người dịch: Phép quay – nối với lượng giác lớp 10–11

Ở THPT, quay điểm (x_1, x_2) quanh gốc tọa độ một góc θ ngược chiều kim đồng hồ cho điểm $(x_1\cos\theta - x_2\sin\theta,\ x_1\sin\theta + x_2\cos\theta)$ – đúng là kết quả của (8.86); ma trận R_θ chỉ là cách viết gọn công thức này. Ví dụ, $\mathbf{x} = (1, 0)$, $\theta = 90°$: $\cos\theta = 0, \sin\theta = 1$, ta được $(0, 1)$ – vectơ nằm ngang quay thành thẳng đứng, độ dài vẫn bằng 1. Quay t lần mỗi lần θ thì bằng quay một lần góc $t\theta$ (công thức cộng góc lượng giác). Ý tưởng RoPE: token ở vị trí t được quay góc $t\theta$. Hai token cách nhau 7 vị trí thì luôn lệch nhau góc 7θ, dù ở đầu hay cuối câu (Hình 8.12 (c)).

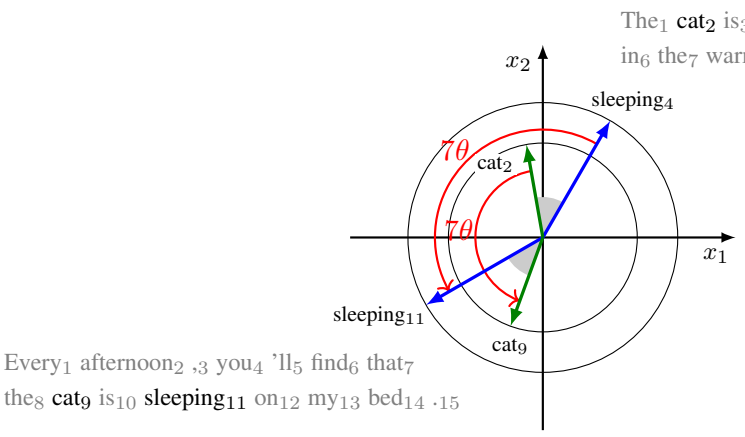
Nếu ta hiểu t là vị trí của một token (được biểu diễn bởi $\mathbf{x}$) trong một chuỗi, ta sẽ thấy phương trình trên định nghĩa một mô hình nhúng vị trí đơn giản. Như trong Hình 8.12 (b), ta bắt đầu dịch chuyển token từ vị trí 0. Mỗi khi tiến thêm một bước, vectơ được quay thêm một góc θ. Khi tới vị trí t, biểu diễn của token kèm ngữ cảnh vị trí được cho bởi $\mathrm{Ro}(\mathbf{x}, t\theta)$. Vì các phép quay không thay đổi độ lớn của



(a) Single-step Rotation



(b) Multi-step Rotation



(c) Angles between embeddings of two tokens at different positions

Hình 8.12: Minh họa các phép quay vectơ trong mặt phẳng. Các hình con (a) và (b) lần lượt cho thấy phép quay một vectơ trong một bước và trong nhiều bước. Hình con (c) cho thấy các vectơ nhúng của token *cat* (con mèo) và *sleeping* (đang ngủ) trong hai câu khác nhau. Ta viết các câu này với một chỉ số dưới gắn vào mỗi token để cho biết vị trí của nó. Nếu biểu diễn token bằng vectơ, ta có thể thêm thông tin vị trí bằng cách quay các vectơ này. Phép quay này bảo toàn "khoảng cách" giữa các vectơ. Ví dụ, vì khoảng cách giữa *cat* và *sleeping* là như nhau trong cả hai câu, góc giữa các vectơ nhúng của chúng cũng giữ nguyên qua phép quay. (Nhãn con: (a) quay một bước; (b) quay nhiều bước; (c) góc giữa vectơ nhúng của hai token ở các vị trí khác nhau. Hai câu ví dụ: "The cat is sleeping peacefully in the warm sunlight." – *Con mèo đang ngủ yên bình dưới nắng ấm.*; "Every afternoon, you'll find that the cat is sleeping on my bed." – *Chiều nào bạn cũng sẽ thấy con mèo đang ngủ trên giường tôi.*)

vectơ nhúng, "ý nghĩa" ban đầu của token được giữ lại. Thông tin vị trí được đưa vào vectơ nhúng khi nó bị quay.

Một cách phổ biến để hiểu phép quay vectơ là thông qua số phức. Dễ dàng biến đổi mỗi vectơ $\mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix}$ trong không gian Euclid hai chiều $\mathbb{R}^2$ thành một số phức $\mathbf{x}' = x_1 + \mathrm{i}x_2$ trong không gian phức $\mathbb{C}$ thông qua một ánh xạ tuyến tính song ánh (*bijective linear map*). Khi đó, phép quay $\mathbf{x}$ một góc $t\theta$ tương ứng với phép nhân với $e^{\mathrm{i}t\theta}$. Vì $e^{\mathrm{i}t\theta} = \cos t\theta + \mathrm{i}\sin t\theta$, phép quay có thể được viết lại dưới dạng

$$\begin{aligned}\mathbf{x}R_{t\theta} &\mapsto \mathbf{x}'e^{\mathrm{i}t\theta} \\ &= (x_1 + \mathrm{i}x_2)(\cos t\theta + \mathrm{i}\sin t\theta) \\ &= \cos t\theta \cdot x_1 - \sin t\theta \cdot x_2 + \mathrm{i}(\sin t\theta \cdot x_1 + \cos t\theta \cdot x_2).\end{aligned} \tag{8.88}$$

Ở đây ta ký hiệu biểu diễn token $\mathbf{x}'e^{\mathrm{i}t\theta}$ là $C(\mathbf{x}, t\theta)$. Tích trong của các biểu diễn của các token tại vị trí t và s có thể được viết là

$$\langle C(\mathbf{x}, t\theta), C(\mathbf{y}, s\theta)\rangle \;=\; (\mathbf{x}'\overline{\mathbf{y}'})e^{\mathrm{i}(t-s)\theta}, \tag{8.89}$$

trong đó $\overline{\mathbf{y}'}$ là số phức liên hợp (*complex conjugate*) của $\mathbf{y}'$. Kết quả của tích trong này chứa số hạng $t - s$, nên nó có thể mô hình hóa độ lệch giữa hai token.

Chú giải của người dịch: Số phức và công thức Euler

Số phức $x_1 + \mathrm{i}x_2$ (với $\mathrm{i}^2 = -1$) có thể xem như điểm (x_1, x_2) trên mặt phẳng (chương trình Toán 12 có giới thiệu). Công thức Euler $e^{\mathrm{i}\varphi} = \cos\varphi + \mathrm{i}\sin\varphi$ là một số phức có độ dài 1 và "góc" φ; nhân một số phức với nó nghĩa là quay số phức đó một góc φ. Số phức liên hợp của $a + b\mathrm{i}$ là $a - b\mathrm{i}$ ("quay ngược"). Vì vậy trong (8.89), góc $t\theta$ của $\mathbf{x}$ và góc $-s\theta$ (do liên hợp) của $\mathbf{y}$ cộng lại thành $(t - s)\theta$: chỉ còn *hiệu* hai vị trí – đúng là thông tin vị trí tương đối.

Bây giờ ta quay lại các biểu diễn trong không gian Euclid hai chiều. Tích vô hướng của $\mathrm{Ro}(\mathbf{x}, t\theta)$ và $\mathrm{Ro}(\mathbf{y}, s\theta)$ có thể được viết thành một hàm của $(t - s)\theta$

$$\begin{aligned}\mathrm{Ro}(\mathbf{x}, t\theta)[\mathrm{Ro}(\mathbf{y}, s\theta)]^\top &= \mathbf{x}R_{t\theta}[\mathbf{y}R_{s\theta}]^\top \\ &= \mathbf{x}R_{t\theta}[R_{s\theta}]^\top\mathbf{y}^\top \\ &= \mathbf{x}R_{(t-s)\theta}\mathbf{y}^\top.\end{aligned} \tag{8.90}$$

Với kết quả này, nếu ta coi $\mathrm{Ro}(\mathbf{x}, t\theta)$ và $\mathrm{Ro}(\mathbf{y}, s\theta)$ lần lượt là truy vấn và khóa thì phép tự chú ý sẽ ngầm bao hàm việc mô hình hóa ngữ cảnh vị trí tương đối.

Nhúng vị trí xoay này có thể được mở rộng cho các vectơ nhúng nhiều chiều. Với một vectơ nhúng token d chiều $\mathbf{x} = \begin{bmatrix} x_1 & x_2 & \cdots & x_d \end{bmatrix}$, ta có thể coi nó là một vectơ phức $\frac{d}{2}$ chiều $\mathbf{x}' = \begin{bmatrix} x'_1 & x'_2 & \cdots & x'_{d/2} \end{bmatrix} = \begin{bmatrix} x_1 + \mathrm{i}x_2 & x_3 + \mathrm{i}x_4 & \cdots & x_{d-1} + \mathrm{i}x_d \end{bmatrix}$, trong đó mỗi cặp phần tử liên tiếp tạo thành một số phức. Khi đó, nhúng vị trí xoay trong không gian phức được cho bởi

$$C(\mathbf{x}, t\theta) \;=\; \sum_{k=1}^{d/2} x'_k e^{\mathrm{i}t\theta_k}\vec{e}_k, \tag{8.91}$$

trong đó $\vec{e}_k$ là vectơ cơ sở chuẩn, chỉ có một giá trị khác 0 tại tọa độ thứ k và bằng 0 ở mọi tọa độ khác [Biderman và cộng sự, 2021].

Mặc dù công thức này có biểu thức phức tạp, dạng tương đương của nó trong không gian Euclid d chiều lại khá dễ hiểu. Ta có thể viết

$$\text{Ro}(\mathbf{x}, t\theta) = \begin{bmatrix} x_1 & x_2 & \cdots & x_d \end{bmatrix} \begin{bmatrix} R_{t\theta_1} & & & \\ & R_{t\theta_2} & & \\ & & \ddots & \\ & & & R_{t\theta_{d/2}} \end{bmatrix}, \tag{8.92}$$

trong đó $R_{t\theta_k} = \begin{bmatrix} \cos t\theta_k & \sin t\theta_k \\ -\sin t\theta_k & \cos t\theta_k \end{bmatrix}$. $\theta = \begin{bmatrix} \theta_1, \cdots, \theta_{d/2} \end{bmatrix}$ là các tham số điều khiển góc quay ở các chiều khác nhau. Thông thường, θ_k được đặt bằng $10000^{-\frac{2(k-1)}{d}}$, tương tự cách thiết lập trong nhúng hình sin.

Trong cài đặt thực tế, Phương trình (8.92) có thể được viết lại thành một dạng chỉ dựa vào phép nhân theo từng phần tử và phép cộng vectơ:

$$\text{Ro}(\mathbf{x}, t\theta) = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{d-1} \\ x_d \end{bmatrix}^\top \odot \begin{bmatrix} \cos t\theta_1 \\ \cos t\theta_1 \\ \vdots \\ \cos t\theta_{d/2} \\ \cos t\theta_{d/2} \end{bmatrix}^\top + \begin{bmatrix} -x_2 \\ x_1 \\ \vdots \\ -x_d \\ x_{d-1} \end{bmatrix}^\top \odot \begin{bmatrix} \sin t\theta_1 \\ \sin t\theta_1 \\ \vdots \\ \sin t\theta_{d/2} \\ \sin t\theta_{d/2} \end{bmatrix}^\top. \tag{8.93}$$

Cuối cùng, ta viết lại Phương trình (8.85) để có dạng vectơ nhúng tại vị trí i

$$\mathbf{e}_i = \text{Ro}(\mathbf{x}_i, i\theta). \tag{8.94}$$

Chú giải của người dịch: RoPE nhiều chiều: quay từng cặp

Phương trình (8.92)–(8.93) nói rằng: chia vectơ d chiều thành $d/2$ cặp $(x_1, x_2), (x_3, x_4), \ldots$ và quay *mỗi cặp* như một vectơ 2 chiều, nhưng với tốc độ quay khác nhau θ_k. Với $d = 4$: $\theta_1 = 10000^0 = 1$ và $\theta_2 = 10000^{-2/4} = 0.01$. Cặp thứ nhất quay nhanh (1 radian mỗi vị trí, giống kim giây), cặp thứ hai quay rất chậm (0.01 radian mỗi vị trí, giống kim giờ). Phối hợp nhiều "kim" với tốc độ khác nhau giúp phân biệt cả khoảng cách gần lẫn xa, như đồng hồ dùng kim giây, kim phút, kim giờ.

4. Nội suy vị trí

Trong nội suy vị trí (*position interpolation*), mục tiêu của ta là ánh xạ các vị trí trong chuỗi mới sao cho khớp với khoảng đã quan sát khi huấn luyện. Giả sử mô hình được huấn luyện trên các chỉ số vị trí trong $[0, m_l]$. Khi suy luận cần xử lý một độ dài $m > m_l$, ta nén các vị trí trong khoảng $[0, m]$ sao cho biểu diễn của chúng vừa với miền ban đầu $[0, m_l]$.

Để minh họa, xét mô hình nhúng vị trí xoay mô tả ở trên. Vectơ nhúng của mỗi token được cho bởi phép biến đổi $\text{Ro}(\mathbf{x}_i, i\theta)$, trong đó vectơ tham số là $\theta = \begin{bmatrix} \theta_1, \cdots, \theta_{d/2} \end{bmatrix}$. Như trong Phương trình (8.93), $\text{Ro}(\mathbf{x}_i, i\theta)$ dựa vào sự kết hợp các hàm tuần hoàn trên các chiều đặc trưng khác nhau

$$\cos i\theta = \begin{bmatrix} \cos i\theta_1 & \cdots & \cos i\theta_{d/2} \end{bmatrix} \tag{8.95}$$

$$\sin i\theta = \begin{bmatrix} \sin i\theta_1 & \cdots & \sin i\theta_{d/2} \end{bmatrix}, \tag{8.96}$$

θ_k có dạng hàm mũ

$$\theta_k = b^{-\frac{2(k-1)}{d}}, \tag{8.97}$$

trong đó b là cơ số (*base*). Chu kỳ của $\cos i\theta_k$ và $\sin i\theta_k$ là

$$T_k = 2\pi \cdot b^{\frac{2(k-1)}{d}}. \tag{8.98}$$

Trực giác then chốt đằng sau nội suy vị trí tuyến tính là điều chỉnh các chu kỳ này sao cho các vị trí mở rộng mới có thể được mã hóa trong phạm vi đã tiền huấn luyện $[0, m_l]$. Cách trực tiếp nhất để làm điều này là tăng tuyến tính T_k theo tỉ lệ ngoại suy $\frac{m}{m_l}$, thu được:

$$T'_k = \frac{m}{m_l} \cdot 2\pi \cdot b^{\frac{2(k-1)}{d}}. \tag{8.99}$$

Làm như vậy, mọi chỉ số vị trí trong $[0, m]$ được nén đều vào $[0, m_l]$. Phép co giãn tuyến tính này có thể được cài đặt bằng cách chia chỉ số vị trí trong mô hình nhúng cho hệ số co giãn [Chen và cộng sự, 2023c]. Mô hình được sửa đổi với nội suy vị trí tuyến tính trở thành

$$\text{Ro}'(\mathbf{x}_i, i\theta) = \text{Ro}\left(\mathbf{x}_i, \frac{m_l}{m} i\theta\right). \tag{8.100}$$

Chú giải của người dịch: Chu kỳ và nội suy tuyến tính bằng số

Ở lớp 11, hàm $y = \cos(\omega x)$ có chu kỳ $T = 2\pi/\omega$; vì $\theta_k = b^{-2(k-1)/d}$ nên $T_k = 2\pi/\theta_k = 2\pi b^{2(k-1)/d}$, đúng như (8.98). Giả sử mô hình được huấn luyện với $m_l = 2048$ và ta muốn dùng $m = 8192$, tỉ lệ $m/m_l = 4$. Theo (8.100), vị trí $i = 8000$ được "giả vờ" là vị trí $8000/4 = 2000$ – nằm trong vùng quen thuộc $[0, 2048]$. Mọi chu kỳ dài ra 4 lần, nên các vị trí mới nằm *xen giữa* các vị trí cũ (vị trí 1 được coi như 0.25), đúng tinh thần nội suy ở Hình 8.9 (c).

Một phương pháp nội suy vị trí khác là co giãn cơ số b, một kỹ thuật được gọi là **RoPE co giãn có nhận biết NTK** (*NTK-aware scaled RoPE*)[16]. Giả sử cơ số b được co giãn theo hệ số λ. Mục tiêu là buộc chu kỳ của mô hình mới này ở chiều tần số cao nhất (tức là chiều cuối cùng, nơi $k = \frac{d}{2}$) khớp với chu kỳ của mô hình nội suy vị trí tuyến tính. Thay $k = \frac{d}{2}$ vào số mũ ta được $\frac{2(d/2-1)}{d} = \frac{d-2}{d}$. Cho hai chu kỳ bằng nhau ta có:

$$2\pi \cdot (\lambda b)^{\frac{d-2}{d}} = \frac{m}{m_l} \cdot 2\pi \cdot b^{\frac{d-2}{d}}. \tag{8.101}$$

Giải phương trình này, ta có thể tách riêng λ:

$$\lambda^{\frac{d-2}{d}} = \frac{m}{m_l}. \tag{8.102}$$

Khi đó, ta có

$$\lambda = \left(\frac{m}{m_l}\right)^{\frac{d}{d-2}}. \tag{8.103}$$

Cách tiếp cận này cho ra một mô hình nhúng được sửa đổi:

$$\text{Ro}'(\mathbf{x}_i, i\theta) = \text{Ro}(\mathbf{x}_i, i\theta'), \tag{8.104}$$

trong đó

$$\theta' = \left[(\lambda b)^{-\frac{0}{d}}, (\lambda b)^{-\frac{2}{d}}, \cdots, (\lambda b)^{-\frac{d-2}{d}}\right]. \tag{8.105}$$

Lưu ý rằng việc co giãn cơ số mang lại một cách co giãn *không đều* các chu kỳ trên các chiều khác nhau của θ. Phương pháp này đã được thấy là hữu ích trong việc mở rộng LLM cho các chuỗi dài hơn, và một số cải tiến đã được phát triển [Peng và cộng sự, 2024; Ding và cộng sự, 2024].

[16] Phương pháp này lần đầu được đề xuất trên một diễn đàn trực tuyến: `https://www.reddit.com/r/LocalLLaMA/comments/14lz7j5/ntkaware_scaled_rope_allows_llama_models_to_have/`

Chú giải của người dịch: Vì sao co giãn cơ số là "không đều"?

Lấy $d = 4$, $m/m_l = 4$. Theo (8.103): $\lambda = 4^{4/2} = 16$. Với $k = 1$, số mũ $\frac{0}{d} = 0$ nên $\theta_1' = (\lambda b)^0 = 1 = \theta_1$: chiều quay nhanh nhất *không đổi* (vẫn phân biệt tốt các token kề nhau). Với $k = 2$ (chiều cuối), $\theta_2' = (16b)^{-1/2} = \theta_2/4$: chu kỳ dài ra đúng 4 lần như nội suy tuyến tính. Như vậy, khác với (8.100) vốn kéo dài *mọi* chu kỳ 4 lần, cách này giữ nguyên các "kim giây" và chỉ kéo dài các "kim giờ". (Phép giải (8.102)–(8.103): chia hai vế (8.101) cho $2\pi b^{(d-2)/d}$, rồi nâng hai vế lên lũy thừa $\frac{d}{d-2}$.)

8.3.6 Nhận xét

Trong mục này, chúng ta đã trình bày nhiều phương pháp cho mô hình hóa ngôn ngữ với ngữ cảnh dài. Ta khép lại mục này bằng việc thảo luận một số vấn đề thú vị liên quan tới các phương pháp này.

1. Nhu cầu về ngữ cảnh dài

Một trong những mục tiêu tối thượng của LLM ngữ cảnh dài là giúp các mô hình này mã hóa chính xác ngữ cảnh vô hạn (*infinite context*). Ở đây, thuật ngữ *ngữ cảnh vô hạn* chỉ khả năng của một LLM liên tục xử lý các token đang đến. Điều này thúc đẩy việc phát triển các LLM có thể xử lý ngữ cảnh rất dài hoặc dữ liệu dạng luồng. Như đã bàn ở Mục 8.3.3, việc dùng các mô hình bộ nhớ kích thước cố định để xử lý ngữ cảnh mở rộng liên tục là phổ biến. Nhiều hệ thống như vậy dựa trên các kiến trúc hồi quy hoặc biến thể của chúng, vì bản chất chúng phù hợp để mô hình hóa dữ liệu tuần tự, nơi ảnh hưởng của các đầu vào trong quá khứ kéo dài mãi mãi. Một cách khác để đạt được bộ nhớ vô hạn là phát triển các phương án thay thế cho mô hình tự chú ý, chẳng hạn ta có thể dùng các mô hình chú ý trong không gian liên tục (*continuous-space attention*) để mã hóa ngữ cảnh, giúp loại bỏ sự phụ thuộc vào độ dài ngữ cảnh [Martins và cộng sự, 2022].

Khi nghiên cứu các LLM ngữ cảnh dài, ta tự nhiên sẽ thắc mắc những cơ chế nào có thể giải thích việc sử dụng ngữ cảnh dài trong mô hình hóa ngôn ngữ. Liệu ta có thể nén biểu diễn của ngữ cảnh vô hạn vào một mô hình tương đối nhỏ không? Có phải mọi token ngữ cảnh đều hữu ích cho việc dự đoán token tiếp theo? Liệu ta có thể biết trước thông tin ngữ cảnh nào là then chốt cho dự đoán? Câu trả lời tổng quát cho tất cả các câu hỏi này không hiển nhiên, nhưng chúng gợi mở những nghiên cứu tiếp theo về các mô hình có thể giải thích (*explainable models*), và một số kết quả thú vị đã được tìm ra. Ví dụ, Deletang và cộng sự [2024] đã tiến hành nhiều thí nghiệm để cho thấy LLM là những bộ nén mạnh mẽ trong ngữ cảnh. Mặc dù việc xem các mô hình dự đoán như các mô hình nén đã được nghiên cứu từ lâu trong học máy, nó cũng mang lại những hiểu biết sâu sắc cho việc ta hiểu các định luật co giãn (scaling law) của LLM. Hơn nữa, Pal và cộng sự [2023] và Wu và cộng sự [2024] đã nghiên cứu xem liệu các đặc trưng nội tại học được tính tới bước tính toán hiện tại, dù không được tối ưu tường minh cho mục đích đó, đã đủ để dự đoán chính xác các token ở nhiều bước trong tương lai hay chưa. Lưu ý rằng nhu cầu về ngữ cảnh dài trong mô hình hóa ngôn ngữ phụ thuộc nhiều vào bài toán mà ta giải quyết. Một vấn đề liên quan là áp dụng LLM ở đâu và đánh giá chúng như thế nào. Ví dụ, trong các tác vụ tóm tắt, ta có thể chỉ cần chắt lọc và tập trung vào vài khía cạnh chính của văn bản, trong khi ở các tác vụ giống truy xuất, ta cần "ghi nhớ" toàn bộ ngữ cảnh để có thể truy cập thông tin liên quan. Ta sẽ bàn về vấn đề đánh giá ở phần sau của tiểu mục này.

Chú giải của người dịch: Dự đoán và nén có liên quan gì?

Nếu một mô hình đoán rất chuẩn chữ tiếp theo, ta chỉ cần ghi lại những chỗ nó đoán sai là đủ để khôi phục cả văn bản – tức là văn bản được "nén" lại. Ví dụ, với câu "Hà Nội là thủ đô của Việt __", một mô hình tốt chắc

chẩn đoán "Nam", nên chữ này gần như không tốn chỗ lưu. Vì vậy, dự đoán giỏi đồng nghĩa với nén giỏi; đây là lý do các nhà nghiên cứu xem LLM như những bộ nén.

2. Tiền huấn luyện hay thích nghi LLM?

Việc huấn luyện LLM đòi hỏi tài nguyên tính toán rất lớn. Mặc dù huấn luyện LLM trên các chuỗi dài là việc làm trực tiếp, chi phí huấn luyện sẽ trở nên quá đắt đỏ đối với các tập dữ liệu lớn. Thực hành phổ biến là tiền huấn luyện LLM trên các tập dữ liệu tổng quát, rồi thích nghi (*adapt*) chúng bằng một lượng tinh chỉnh vừa phải. Ví dụ, các LLM có nhúng vị trí tương đối hoặc nhúng vị trí xoay có thể được huấn luyện trực tiếp trên dữ liệu quy mô lớn trong giai đoạn tiền huấn luyện. Mặc dù các mô hình thu được có thể thể hiện một số khả năng ngoại suy độ dài (*length extrapolation*) trong giai đoạn suy luận, việc tinh chỉnh chúng trên các chuỗi dài hơn có thể hiệu quả hơn.

Lý tưởng nhất, ta muốn tiền huấn luyện LLM với các kiến trúc Transformer chuẩn rồi thích nghi chúng cho các tác vụ mới. Điều này cho phép ta dùng nhiều LLM có sẵn và thích nghi chúng một cách hiệu quả để xử lý các chuỗi dài. Tuy nhiên, khi áp dụng các kiến trúc mới, dường như không tránh khỏi việc phải huấn luyện các mô hình này từ đầu. Điều này gây khó khăn thực tế cho việc phát triển LLM ngữ cảnh dài, vì ta không thể tận dụng các mô hình tiền huấn luyện đã được phát triển tốt mà phải tự huấn luyện chúng. Mặt khác, tinh chỉnh vẫn là một cách hiệu quả để thích nghi các LLM có một số kiến trúc khác với kiến trúc dùng trong tiền huấn luyện. Một ví dụ là các mô hình được tăng cường bằng bộ nhớ ngoài. Trong các mô hình này, LLM đã tiền huấn luyện được giữ cố định, và trọng tâm là cách LLM tương tác với các mô-đun bộ nhớ ngoài. Chẳng hạn trong RAG, người ta thường tinh chỉnh LLM để cải thiện cách chúng sử dụng các đầu vào được tăng cường truy xuất. Một ví dụ khác là huấn luyện một LLM với chú ý đầy đủ, rồi thay thế nó bằng chú ý thưa trong giai đoạn tinh chỉnh. LLM đã tiền huấn luyện cung cấp giá trị khởi tạo cho các tham số của mô hình mới, và mô hình này sau đó được tinh chỉnh như thường lệ.

3. Đánh giá LLM ngữ cảnh dài

Đánh giá LLM ngữ cảnh dài là quan trọng, nhưng đây là một bài toán tương đối mới trong NLP. Ý tưởng chung là: nếu ta đưa một ngữ cảnh dài vào LLM, ta có thể xem xét đầu ra của LLM để xác định liệu nó có hiểu toàn bộ ngữ cảnh và tận dụng ngữ cảnh đó khi dự đoán các token tiếp theo hay không. Trong nghiên cứu NLP truyền thống, những đánh giá như vậy thường nhằm kiểm tra khả năng của mô hình NLP trong việc xử lý các phụ thuộc xa. Tuy nhiên, độ dài ngữ cảnh dùng trong các LLM gần đây lớn hơn nhiều so với độ dài dùng trong các hệ thống NLP cách đây vài năm. Điều này thúc đẩy các nhà nghiên cứu phát triển các bộ đánh giá chuẩn (benchmark) và độ đo mới cho LLM ngữ cảnh dài.

Một cách tiếp cận là dùng độ đo độ hỗn loạn (*perplexity*). Tuy nhiên, dù đơn giản, phương pháp này có xu hướng phản ánh khả năng của LLM trong việc tận dụng ngữ cảnh cục bộ hơn là ngữ cảnh toàn cục. Vì vậy, người ta muốn phát triển các phương pháp đánh giá dành riêng cho LLM ngữ cảnh dài. Các phương pháp phổ biến bao gồm nhiều tác vụ tổng hợp (*synthetic tasks*), trong đó dữ liệu được sinh nhân tạo hoặc được sửa đổi được dùng để đánh giá những năng lực cụ thể của LLM ngữ cảnh dài. Chẳng hạn, trong các tác vụ mò kim đáy bể (*needle-in-a-haystack*)[17] và truy xuất mật mã (*passkey retrieval*) [Mohtashami và Jaggi, 2024; Chen và cộng sự, 2023c], LLM được yêu cầu xác định và trích ra một mẩu thông tin nhỏ, có liên quan từ một khối lượng lớn văn bản cho trước. Giả định ở đây là một LLM có đủ bộ nhớ sẽ nhớ được các phần trước đó của văn bản khi xử lý thông tin mới. Nhờ vậy LLM có thể nhặt ra từ văn bản những chi tiết liên quan, vốn có thể thưa thớt và ẩn giữa rất nhiều thông tin không

[17] https://github.com/gkamradt/LLMTest_NeedleInAHaystack

liên quan. Ngoài ra, trong các tác vụ sao chép bộ nhớ (*copy memory tasks*) (gọi tắt là tác vụ sao chép), LLM được yêu cầu lặp lại văn bản đầu vào hoặc một đoạn cụ thể nhiều lần. Các tác vụ này ban đầu được đề xuất để kiểm tra mức độ mà các mô hình hồi quy có thể lưu giữ và nhớ lại các token đã thấy trước đó [Hochreiter và Schmidhuber, 1997; Arjovsky và cộng sự, 2016], và đã được dùng để đánh giá các LLM gần đây [Bulatov và cộng sự, 2022; Gu và Dao, 2023].

Chú giải của người dịch: Bài kiểm tra "mò kim đáy bể"

Người ta chèn một câu "kim" (ví dụ: "Mật khẩu bí mật là 7429.") vào một vị trí ngẫu nhiên giữa hàng chục nghìn token văn bản "rơm" không liên quan, rồi hỏi mô hình: "Mật khẩu bí mật là gì?". Thay đổi độ dài văn bản và vị trí chèn kim, ta vẽ được bản đồ cho biết mô hình "nhớ" tốt tới đâu. Như sách nói ở đoạn sau, làm tốt bài này chưa chắc mô hình đã *hiểu* toàn bộ ngữ cảnh.

Một cách tiếp cận khác để đánh giá LLM ngữ cảnh dài là kiểm tra chúng trên các tác vụ NLP có chuỗi đầu vào rất dài. Ví dụ bao gồm tóm tắt văn bản dài hoặc tóm tắt nhiều văn bản, hỏi đáp trên văn bản dài, hoàn thiện mã nguồn, v.v. Một lợi ích của cách tiếp cận này là nó có thể làm cho việc đánh giá phù hợp với kỳ vọng của người dùng.

Mặc dù nhiều phương pháp đã được phát triển, vẫn chưa có một cách tổng quát để đánh giá LLM ngữ cảnh dài [Liu và cộng sự, 2024c]. Một vấn đề là hầu hết các phương pháp này tập trung vào những khía cạnh cụ thể của LLM, thay vì năng lực nền tảng trong việc mô hình hóa ngữ cảnh rất dài. Ngay cả khi một LLM có thể nhặt ra đúng đoạn văn bản thích hợp từ đầu vào, ta cũng không thể nói rằng nó thực sự hiểu toàn bộ ngữ cảnh. Thay vào đó, nó có thể chỉ nhớ một số phần quan trọng của ngữ cảnh, hoặc thậm chí đơn giản là nhớ lại câu trả lời thông qua mô hình đã học trong tiền huấn luyện. Hơn nữa, dữ liệu dùng trong nhiều tác vụ có quy mô nhỏ và còn tương đối sơ khai, dẫn tới sự chênh lệch giữa kết quả đánh giá và hiệu năng ứng dụng thực tế. Một vấn đề đáng chú ý hơn là kết quả của LLM chịu ảnh hưởng bởi nhiều yếu tố khác và các thiết lập thí nghiệm, chẳng hạn dùng các prompt khác nhau có thể dẫn tới kết quả rất khác nhau. Điều này khiến việc đánh giá càng khó khăn hơn, vì các cải thiện có thể không hoàn toàn đến từ việc mô hình hóa ngữ cảnh dài tốt hơn, và có nguy cơ khẳng định quá mức mức tăng hiệu năng. Dù vậy, vẫn còn nhiều câu hỏi mở trong việc phát triển và đánh giá LLM ngữ cảnh dài. Ví dụ, các mô hình này vẫn gặp những hạn chế như độ dài ngữ cảnh bị giới hạn và độ trễ (*latency*) cao. Nghiên cứu các vấn đề này nhiều khả năng sẽ là một hướng đi giá trị trong tương lai.

8.4 Tóm tắt

Trong chương này, chúng ta đã bàn về khái niệm LLM và các kỹ thuật liên quan. Đây có thể được xem là một phần giới thiệu tổng quát, dù chưa toàn diện, về LLM, đặt nền móng cho những thảo luận sâu hơn về các chủ đề nâng cao trong các chương sau. Hơn nữa, chúng ta đã tìm hiểu hai hướng để mở rộng quy mô LLM. Hướng thứ nhất tập trung vào tiền huấn luyện LLM quy mô lớn, điều then chốt để phát triển các mô hình tiên tiến nhất. Hướng thứ hai tập trung vào các phương pháp thích nghi LLM cho đầu vào dài, bao gồm tối ưu các mô hình chú ý, thiết kế các bộ nhớ đệm KV hiệu quả và được nén hơn, đưa vào các mô hình bộ nhớ, và khám phá các vectơ nhúng vị trí tốt hơn.

Sức mạnh của LLM nằm ở khả năng phá vỡ giới hạn của việc huấn luyện các mô hình NLP cho một số ít tác vụ cụ thể. Thay vào đó, LLM học từ lượng văn bản khổng lồ thông qua một tác vụ đơn giản là dự đoán token – dự đoán token tiếp theo trong chuỗi khi biết ngữ cảnh phía trước nó. Một quan điểm chung là: bằng cách lặp lại tác vụ dự đoán token này rất nhiều lần, LLM có thể thu nhận một số tri thức về thế giới và ngôn ngữ, rồi áp dụng chúng cho các tác vụ mới. Kết quả là LLM có thể được prompt

để thực hiện bất kỳ tác vụ nào bằng cách diễn đạt tác vụ đó thành bài toán dự đoán các token tiếp theo khi cho trước prompt. Năng lực đột sinh (*emergent ability*) này của các mô hình ngôn ngữ đến từ nhiều chiều, chẳng hạn mở rộng quy mô huấn luyện, kích thước mô hình và kích thước ngữ cảnh. Không thể phủ nhận rằng các định luật co giãn hiện là nguyên lý nền tảng được áp dụng trong phát triển mô hình ngôn ngữ lớn, mặc dù việc chỉ đơn thuần tăng kích thước mô hình vẫn chưa được chứng minh là đủ để đạt tới trí tuệ nhân tạo tổng quát (*AGI*). Các LLM được mở rộng liên tục này đã được thấy là có năng lực hiểu ngôn ngữ, sinh ngôn ngữ và lập luận (reasoning) đa mục đích. Gần đây hơn, người ta thấy rằng việc tăng lượng tính toán lúc suy luận (tức tính toán lúc kiểm tra (*test-time compute*)) cũng có thể dẫn tới những cải thiện đáng kể trong các tác vụ suy luận phức tạp [OpenAI, 2024].

Với sức mạnh đáng kinh ngạc của mình, LLM đã thu hút sự quan tâm đáng kể, cả về kỹ thuật lẫn ứng dụng. Kết quả là sự bùng nổ quan tâm nghiên cứu về LLM cũng dẫn tới một số lượng khổng lồ các kỹ thuật và mô hình mới. Tuy nhiên, chúng tôi không cố gắng đưa ra một tổng quan tài liệu toàn diện về mọi khía cạnh của LLM, do lĩnh vực này phát triển rất nhanh. Dù vậy, bạn đọc vẫn có thể tìm hiểu về LLM từ các bài tổng quan chung [Zhao và cộng sự, 2023; Minaee và cộng sự, 2024] hoặc các thảo luận tập trung hơn vào những chủ đề cụ thể [Ruan và cộng sự, 2024].

Chương 9

Prompting

Prompting (*prompting*) (kỹ thuật prompt) là quá trình cung cấp cho một LLM một đầu vào hoặc một gợi ý cụ thể để nó sinh ra đầu ra mong muốn hoặc thực hiện một nhiệm vụ. Ví dụ, nếu muốn LLM dịch một câu từ tiếng Anh sang tiếng Trung, ta có thể "prompt" nó như sau

> Dịch văn bản sau từ tiếng Anh sang tiếng Trung.
> Văn bản: The early bird catches the worm.
> Bản dịch: ___

Chú giải của người dịch: Prompt là gì?

Prompt (giữ nguyên tiếng Anh, có thể hiểu là "lời nhắc", "câu lệnh đầu vào") là đoạn văn bản ta gõ vào để "bảo" mô hình làm việc gì đó — giống như đề bài ta đưa cho một học sinh. Mô hình ngôn ngữ lớn (LLM) như ChatGPT đọc đoạn văn bản này rồi viết tiếp phần trả lời. Trong ví dụ trên, câu tiếng Anh *The early bird catches the worm* nghĩa đen là "Con chim dậy sớm bắt được sâu" (tục ngữ: ai chịu dậy sớm, bắt tay vào việc sớm thì sẽ được lợi). Dòng "Bản dịch:" để trống chính là chỗ ta chờ LLM viết tiếp. Trong prompt gốc, câu tiếng Anh được giữ nguyên vì đó là văn bản cần dịch.

Prompting có vai trò then chốt đối với LLM vì nó ảnh hưởng trực tiếp đến mức độ hiệu quả mà các mô hình này hiểu và đáp lại các truy vấn của người dùng. Một prompt được soạn kỹ có thể dẫn dắt LLM sinh ra những câu trả lời chính xác hơn, liên quan hơn và phù hợp với ngữ cảnh hơn. Hơn nữa, quá trình này có thể được tinh chỉnh lặp đi lặp lại. Bằng cách phân tích các câu trả lời của LLM, người dùng có thể điều chỉnh prompt để sát hơn với nhu cầu cụ thể của mình. Do tầm quan trọng của prompting trong việc ứng dụng LLM, việc thiết kế prompt đã trở thành một kỹ năng thiết yếu đối với người dùng và nhà phát triển làm việc với LLM. Tầm quan trọng ngày càng tăng của việc thiết kế prompt đã làm nảy sinh một lĩnh vực nghiên cứu sôi động gọi là thiết kế prompt (*prompt engineering*), trong đó chúng ta thiết kế các prompt hiệu quả để tận dụng LLM tốt hơn và nâng cao tính hữu dụng của chúng trong các ứng dụng thực tế.

Một khái niệm quan trọng liên quan đến prompting là học trong ngữ cảnh (*in-context learning*). Khi prompt một LLM, ta có thể thêm thông tin mới vào ngữ cảnh, chẳng hạn các ví dụ minh họa cách giải quyết vấn đề. Điều này cho phép LLM học từ ngữ cảnh đó cách giải bài toán. Dưới đây là một ví dụ prompt LLM bằng vài ví dụ minh họa về cách phân loại văn bản theo cực cảm xúc (*sentiment polarity*).

> Dưới đây là một số ví dụ về phân loại văn bản.
> Ví dụ 1: Chúng tôi đã có một bữa tối thật vui vẻ cùng nhau. → Nhãn: Tích cực
> Ví dụ 2: Tôi bực mình vì những lần trì hoãn. → Nhãn: Tiêu cực

Nhãn của câu "Lời nhận xét đó khá là tổn thương." là gì?
Nhãn: ___

Học trong ngữ cảnh thường được xem là một năng lực đột sinh (*emergent ability*) của LLM, xuất hiện sau quá trình tiền huấn luyện (*pre-training*). Mặc dù LLM có thể được huấn luyện hoặc tinh chỉnh (*fine-tuning*) để thực hiện nhiệm vụ mới, học trong ngữ cảnh cung cấp một cách rất hiệu quả để thích nghi các mô hình này mà không cần bất kỳ công sức huấn luyện hay tinh chỉnh nào. Có lẽ đây là một trong những đặc điểm nổi bật nhất của LLM: chúng thực sự học được tri thức tổng quát về thế giới và về ngôn ngữ trong quá trình tiền huấn luyện, và ta có thể dễ dàng áp dụng tri thức đó vào những thử thách mới. Hơn nữa, học trong ngữ cảnh phản ánh xu hướng rộng hơn là làm cho các hệ thống AI tổng quát hơn và thân thiện hơn với người dùng. Thay vì cần các kỹ sư chuyên môn tinh chỉnh mô hình cho từng nhiệm vụ riêng, người dùng có thể tương tác với LLM theo cách trực quan hơn, chỉ cần đưa ví dụ hoặc điều chỉnh ngữ cảnh khi cần.

Chú giải của người dịch: Học trong ngữ cảnh khác gì với "huấn luyện"?

Khi *huấn luyện* hay *tinh chỉnh*, ta thay đổi các tham số (hàng tỉ con số bên trong mô hình) để mô hình "nhớ" cách làm một việc mới — tốn nhiều máy tính và thời gian. Còn *học trong ngữ cảnh* thì không đụng đến tham số nào: ta chỉ đặt vài ví dụ mẫu ngay trong prompt, mô hình đọc chúng rồi bắt chước theo khi trả lời câu hỏi mới. Giống như khi làm bài kiểm tra, thầy cô cho sẵn hai bài mẫu đã giải ở đầu đề, ta nhìn cách giải mẫu để làm bài tương tự — ta không cần "học lại" cả chương, và khi sang đề khác thì các bài mẫu ấy cũng không còn. Ở ví dụ phân loại phía trên, LLM nhìn hai ví dụ có nhãn và sẽ đoán câu "Lời nhận xét đó khá là tổn thương" có nhãn *Tiêu cực*.
"Năng lực đột sinh" là năng lực hầu như không thấy ở các mô hình nhỏ nhưng "bỗng nhiên" xuất hiện khi mô hình đủ lớn và được tiền huấn luyện trên đủ nhiều dữ liệu.

Trong chương này, chúng ta tập trung vào các kỹ thuật prompting cho LLM. Ta bắt đầu bằng việc xem xét một số thiết kế prompt thú vị thường được dùng trong thiết kế prompt. Sau đó, ta bàn về một loạt cải tiến cho các phương pháp này. Cuối cùng, ta khám phá các cách tiếp cận nhằm tự động hóa việc thiết kế prompt.

9.1 Thiết kế prompt tổng quát

Mục này trình bày các khái niệm cơ bản trong thiết kế prompt, cùng với các ví dụ về cách prompt LLM cho nhiều nhiệm vụ NLP khác nhau. Vì hiệu quả của prompting phụ thuộc rất nhiều vào LLM được sử dụng, prompt thường khác nhau giữa các LLM, khiến ta khó đưa ra một danh sách đầy đủ các prompt cho mọi LLM và mọi nhiệm vụ hạ nguồn (*downstream task*). Vì vậy, phần thảo luận này không tập trung vào một LLM cụ thể nào. Thay vào đó, mục tiêu là đưa ra các nguyên tắc định hướng cho việc thiết kế prompt.

9.1.1 Những điều cơ bản

Thuật ngữ *prompt* được dùng theo nhiều cách khác nhau. Trong chương này, ta định nghĩa prompt là văn bản đầu vào của một LLM, ký hiệu là $\mathbf{x}$. LLM sinh ra đầu ra $\mathbf{y}$ theo xác suất có điều kiện (*conditional probability*) $\Pr(\mathbf{y}|\mathbf{x})$. Trong quá trình sinh này, prompt đóng vai trò là điều kiện để đưa ra dự đoán, và nó có thể chứa bất kỳ thông tin nào giúp mô tả và giải quyết vấn đề.

Chú giải của người dịch: Đọc ký hiệu $\Pr(\mathbf{y}|\mathbf{x})$

$\Pr(\mathbf{y}|\mathbf{x})$ đọc là "xác suất của **y** khi biết **x**". Ở đây **x** là prompt ta đưa vào, **y** là một câu trả lời có thể có. Với cùng một câu hỏi, mô hình "thấy" có rất nhiều câu trả lời khả dĩ, mỗi câu có một xác suất; mô hình sẽ ưu tiên sinh ra những câu trả lời có xác suất cao. Thay đổi prompt **x** tức là thay đổi điều kiện, nên phân phối xác suất trên các câu trả lời cũng thay đổi theo — đó là lý do một prompt tốt có thể dẫn tới câu trả lời tốt hơn.

Một prompt có thể được tạo ra bằng một **khuôn mẫu prompt** (*prompt template*) (gọi tắt là **khuôn mẫu** (*template*)) [Liu và cộng sự, 2023b]. Khuôn mẫu là một đoạn văn bản chứa các chỗ trống (*placeholder*) hay biến, trong đó mỗi chỗ trống có thể được điền bằng một thông tin cụ thể. Sau đây là hai khuôn mẫu để hỏi LLM về gợi ý cho cuối tuần.

```
Hãy cho tôi vài gợi ý để có một cuối tuần vui vẻ.
```

```
Nếu {∗premise∗}, bạn có gợi ý gì cho một cuối tuần vui vẻ.
```

Ở khuôn mẫu thứ nhất, ta chỉ đơn giản yêu cầu LLM đưa ra vài gợi ý. Vì vậy khuôn mẫu này chỉ là một đoạn văn bản không có biến nào. Ở khuôn mẫu thứ hai, biến {∗premise∗} (*tiền đề*) cần được người dùng chỉ định để cung cấp một tiền đề cho việc đưa ra gợi ý. Ví dụ, nếu ta nhập

$$\text{premise} = \text{thời tiết cuối tuần này đẹp}, \tag{9.1}$$

thì ta có thể sinh ra prompt

```
Nếu thời tiết cuối tuần này đẹp,
bạn sẽ gợi ý gì cho một cuối tuần vui vẻ?
```

Ta cũng có thể thiết kế một khuôn mẫu có nhiều biến. Dưới đây là một ví dụ trong đó ta so sánh độ tương đồng ngữ nghĩa giữa hai câu.

```
Đây là một câu
{∗sentence1∗}
Đây là một câu khác
{∗sentence2∗}
Hãy tính độ tương đồng ngữ nghĩa giữa hai câu
```

Chú giải của người dịch: Khuôn mẫu giống như "mẫu đơn" điền thông tin

Khuôn mẫu prompt giống một tờ đơn in sẵn: phần chữ cố định đã viết, còn các ô trống như {∗sentence1∗}, {∗sentence2∗} (câu 1, câu 2) thì mỗi lần dùng ta điền nội dung khác nhau. Nhờ đó, một khuôn mẫu có thể tạo ra hàng nghìn prompt cho hàng nghìn cặp câu khác nhau mà không phải viết lại từ đầu. Ký hiệu {∗…∗} trong sách chỉ đánh dấu chỗ trống, không phải là chữ thực sự đưa vào mô hình.

Một cách phổ biến để định dạng prompt là viết mỗi đầu vào hoặc đầu ra theo kiểu "tên: nội dung". Ví dụ, ta có thể mô tả một cuộc hội thoại giữa hai người tên John và David, rồi dùng LLM để viết tiếp cuộc hội thoại. Sau đây là một khuôn mẫu cho những prompt như vậy.

```
John: {∗utterance1∗}
David: {∗utterance2∗}
```

John: {∗utterance3∗}
David: {∗utterance4∗}
John: {∗utterance5∗}
David: {∗utterance6∗}
John: {∗utterance7∗}
David: ___

(*utterance* nghĩa là "lượt lời".)

Định dạng "tên: nội dung" có thể được dùng để định nghĩa nhiệm vụ mà ta muốn LLM thực hiện. Ví dụ, vì "Q" và "A" là các chữ viết tắt thường dùng của "Question" (câu hỏi) và "Answer" (câu trả lời), ta có thể dùng khuôn mẫu sau để thực hiện nhiệm vụ hỏi–đáp.

Q: {∗question∗}
A: ___

Định dạng này có thể được dùng để mô tả những nhiệm vụ phức tạp hơn. Chẳng hạn, sau đây là một ví dụ cung cấp đặc tả cho một nhiệm vụ dịch:

Nhiệm vụ: Dịch
Ngôn ngữ nguồn: Tiếng Anh
Ngôn ngữ đích: Tiếng Trung
Văn phong: Văn bản trang trọng
Khuôn mẫu: Dịch câu sau: {∗sentence∗}

Trong các hệ thống thực tế, người ta thường biểu diễn và lưu trữ những dữ liệu như vậy dưới dạng các cặp khóa–giá trị, chẳng hạn JSON[1].

Chú giải của người dịch: Cặp khóa–giá trị và JSON

Một *cặp khóa–giá trị* gồm một tên trường (*khóa*) và nội dung của trường đó (*giá trị*), ví dụ khóa "Ngôn ngữ đích" có giá trị "Tiếng Trung" — giống như các dòng "Họ và tên: …", "Ngày sinh: …" trong sơ yếu lý lịch. JSON là một định dạng văn bản rất phổ biến trong lập trình để lưu những cặp như vậy, theo kiểu `"khóa": "giá trị"` đặt trong cặp ngoặc nhọn.

Khi vấn đề khó mô tả theo kiểu liệt kê thuộc tính như trên, người ta thường hướng dẫn LLM bằng một mô tả rõ ràng và chi tiết. Có nhiều cách để làm điều này. Một ví dụ là gán cho LLM một vai trò và cung cấp đủ ngữ cảnh. Sau đây là một khuôn mẫu hướng dẫn LLM đóng vai một chuyên gia và trả lời câu hỏi của trẻ em.

[1] Biểu diễn JSON là

```
{
   "Task": "Translation",
   "Source language": "English",
   "Target language": "Chinese",
   "Style": "Formal text",
   "Template": "Translate the following sentence: {∗sentence∗}",
}
```

(Các trường lần lượt là: nhiệm vụ, ngôn ngữ nguồn, ngôn ngữ đích, văn phong, khuôn mẫu.)

Bạn là một nhà khoa học máy tính có kiến thức sâu rộng trong lĩnh vực học sâu.
Hãy giải thích khái niệm liên quan đến máy tính sau đây cho một em bé khoảng 10 tuổi, dùng các ví dụ đơn giản bất cứ khi nào có thể.
{∗concept∗}
___

Đoạn văn bản "Bạn là một nhà khoa học máy tính … học sâu." đôi khi được gọi là prompt hệ thống (*system prompt*) (hay *thông tin hệ thống*), và được cung cấp để giúp LLM hiểu ngữ cảnh hoặc các ràng buộc của nhiệm vụ mà nó được yêu cầu thực hiện.

Chú giải của người dịch: Prompt hệ thống

Trong các ứng dụng trò chuyện như ChatGPT, prompt hệ thống thường là phần "lời dặn" đặt trước, do nhà phát triển viết sẵn (người dùng thường không nhìn thấy), ví dụ: "Bạn là trợ lý học tập, luôn trả lời ngắn gọn bằng tiếng Việt". Phần câu hỏi người dùng gõ vào sau đó mới là *prompt người dùng*. Prompt hệ thống giống như việc giáo viên dặn trước "Hôm nay em đóng vai hướng dẫn viên du lịch" rồi mới đặt câu hỏi.

9.1.2 Học trong ngữ cảnh

Việc học có thể diễn ra ngay trong lúc suy luận (*inference*) ở các mô hình ngôn ngữ lớn. Học trong ngữ cảnh là một phương pháp như vậy, trong đó prompt chứa các ví dụ minh họa cách giải quyết vấn đề, và LLM có thể học cách giải các bài toán mới từ những minh họa này. Vì ta không cập nhật tham số của mô hình trong quá trình này, học trong ngữ cảnh có thể được xem là một cách kích hoạt hiệu quả tri thức đã học trong giai đoạn tiền huấn luyện mà không cần huấn luyện hay tinh chỉnh thêm. Điều này cho phép LLM nhanh chóng thích nghi với các bài toán mới và mở rộng năng lực của các mô hình tiền huấn luyện mà không cần điều chỉnh riêng cho từng nhiệm vụ.

Học trong ngữ cảnh có thể được minh họa bằng cách so sánh ba phương pháp: học zero-shot (*zero-shot learning*), học one-shot (*one-shot learning*) và học few-shot (*few-shot learning*). Học zero-shot, đúng như tên gọi, không bao gồm một quá trình "học" theo nghĩa truyền thống. Thay vào đó, nó dùng trực tiếp LLM để giải quyết các bài toán mới chưa từng được quan sát trong quá trình huấn luyện. Trong thực tế, ta có thể điều chỉnh prompt lặp đi lặp lại để dẫn dắt LLM sinh ra câu trả lời tốt hơn, mà không cần minh họa các bước giải hay đưa ra ví dụ. Hãy xem ví dụ sau. Giả sử ta muốn dùng LLM làm trợ lý giúp sửa các câu tiếng Anh. Một prompt học zero-shot như sau

SYSTEM Bạn là một trợ lý hữu ích, và rất giỏi sửa lỗi ngữ pháp.
USER Bạn sẽ được cung cấp một câu tiếng Anh. Nhiệm vụ là xuất ra câu đúng.
Input: She don't like going to the park.
Output: ___

Ở đây các chữ màu xám được dùng để chỉ các trường khác nhau của prompt.

Chú giải của người dịch: Zero-shot, one-shot, few-shot

"Shot" ở đây có thể hiểu là "ví dụ mẫu". *Zero-shot*: không cho ví dụ nào, chỉ mô tả nhiệm vụ. *One-shot*: cho đúng một ví dụ mẫu. *Few-shot*: cho một vài ví dụ mẫu. Các nhãn xám trong hộp: SYSTEM (prompt hệ thống), USER (phần của người dùng), DEMO (*demonstration* — ví dụ minh họa). Các câu tiếng Anh được giữ nguyên vì nhiệm vụ chính là sửa ngữ pháp tiếng Anh; "Input" là câu vào, "Output" là câu ra. Câu *She*

don't like going to the park ("Cô ấy không thích đi công viên") sai vì chủ ngữ ngôi thứ ba số ít *she* phải đi với *doesn't*; câu đúng là *She doesn't like going to the park*.

Trong học one-shot, ta mở rộng prompt này bằng cách thêm một ví dụ minh họa cách sửa câu, nhờ đó cho phép LLM học từ kinh nghiệm mới được bổ sung này.

SYSTEM	Bạn là một trợ lý hữu ích, và rất giỏi sửa lỗi ngữ pháp.
DEMO	Bạn sẽ được cung cấp một câu tiếng Anh. Nhiệm vụ là xuất ra câu đúng. Input: There is many reasons to celebrate. Output: There are many reasons to celebrate.
USER	Bạn sẽ được cung cấp một câu tiếng Anh. Nhiệm vụ là xuất ra câu đúng. Input: She don't like going to the park. Output: ___

Hơn nữa, ta có thể thêm nhiều ví dụ minh họa hơn để thực hiện học few-shot.

SYSTEM	Bạn là một trợ lý hữu ích, và rất giỏi sửa lỗi ngữ pháp.
DEMO1	Bạn sẽ được cung cấp một câu tiếng Anh. Nhiệm vụ là xuất ra câu đúng. Input: There is many reasons to celebrate. Output: There are many reasons to celebrate.
DEMO2	Bạn sẽ được cung cấp một câu tiếng Anh. Nhiệm vụ là xuất ra câu đúng. Input: Me and my friend goes to the gym every day. Output: My friend and I go to the gym every day.
USER	Bạn sẽ được cung cấp một câu tiếng Anh. Nhiệm vụ là xuất ra câu đúng. Input: She don't like going to the park. Output: ___

(Nghĩa các câu ví dụ: "Có nhiều lý do để ăn mừng"; "Tôi và bạn tôi đến phòng tập thể dục mỗi ngày".)

Trong học few-shot, về bản chất ta cung cấp một khuôn mẫu ánh xạ một số đầu vào sang các đầu ra tương ứng. LLM cố gắng làm theo khuôn mẫu này khi đưa ra dự đoán, miễn là prompt chứa đủ số lượng ví dụ minh họa (dù số lượng này thường nhỏ). Ta cũng có thể dùng những khuôn mẫu đơn giản hơn để đạt được điều này. Ví dụ, có thể dùng prompt học few-shot sau để dịch các từ từ tiếng Trung sang tiếng Anh.

DEMO	现在	→	now
	来	→	come
	去	→	go
	男孩	→	boy
USER	女孩	→	___

(Nghĩa tiếng Việt: 现在 – now – "bây giờ"; 来 – come – "đến"; 去 – go – "đi"; 男孩 – boy – "bé trai"; 女孩 – "bé gái", đáp án mong đợi là *girl*.)

Nếu LLM đủ mạnh, học few-shot có thể giúp nó giải quyết các bài toán phức tạp, chẳng hạn lập luận toán học. Ví dụ, hãy xét nhiệm vụ sau: cộng hai số rồi chia tổng cho tích của chúng.

DEMO	12 5	$\rightarrow$	$(12+5)/(12\times 5)=0.283$
	3 1	$\rightarrow$	$(3+1)/(3\times 1)=1.33$
	-9 4	$\rightarrow$	$(-9+4)/(-9\times 4)=0.138$
	15 15	$\rightarrow$	$(15+15)/(15\times 15)=0.133$
USER	19 73	$\rightarrow$	___

Chú giải của người dịch: Mô hình phải tự "đoán ra" quy tắc

Trong prompt trên, ta không hề nói "hãy cộng rồi chia cho tích"; mô hình phải tự nhận ra quy tắc từ bốn ví dụ. Chẳng hạn $(-9+4)/(-9\times 4)=(-5)/(-36)\approx 0.138$. Câu trả lời mong đợi cho cặp 19 và 73 là $(19+73)/(19\times 73)=92/1387\approx 0.0663$. Ở đây dấu chấm là dấu thập phân (viết theo kiểu quốc tế như sách gốc), tức 0.283 chính là $0{,}283$ theo cách viết ở Việt Nam.

Trong nhiều ứng dụng thực tế, hiệu quả của học trong ngữ cảnh phụ thuộc nhiều vào chất lượng của prompt và năng lực nền tảng của LLM đã tiền huấn luyện. Một mặt, cần nhiều công sức thiết kế prompt để xây dựng được những prompt phù hợp giúp LLM học hiệu quả hơn từ các ví dụ minh họa. Mặt khác, những LLM mạnh hơn có thể tận dụng học trong ngữ cảnh tốt hơn để thực hiện nhiệm vụ mới. Ví dụ, giả sử ta muốn dùng một LLM để dịch các từ từ tiếng Inuktitut sang tiếng Anh. Nếu LLM không được tiền huấn luyện trên dữ liệu tiếng Inuktitut, hiểu biết của nó về tiếng Inuktitut sẽ yếu, và mô hình sẽ khó dịch tốt cho dù ta prompt nó như thế nào. Trong trường hợp này, ta cần tiếp tục huấn luyện LLM với nhiều dữ liệu tiếng Inuktitut hơn, thay vì cố tìm những prompt tốt hơn. [2]

Một câu hỏi thú vị nữa là học trong ngữ cảnh hình thành như thế nào trong quá trình tiền huấn luyện và vì sao nó hoạt động được lúc suy luận. Một cách giải thích trực quan là: dù LLM đã thu được tri thức giải quyết vấn đề trong quá trình tiền huấn luyện, không gian đầu ra quá lớn khiến mô hình khó chọn được dự đoán đúng cho các nhiệm vụ mới lạ. Việc cung cấp ví dụ minh họa sẽ dẫn LLM đi theo những con đường "đúng". Hơn nữa, một số nhà nghiên cứu đã cố gắng diễn giải học trong ngữ cảnh từ nhiều góc nhìn khác nhau, bao gồm suy luận Bayes (*Bayesian inference*) [Xie và cộng sự, 2022], hạ gradient (*gradient descent*) [Dai và cộng sự, 2023; Von Oswald và cộng sự, 2023], hồi quy tuyến tính (*linear regression*) [Akyürek và cộng sự, 2023], học cách học (*meta-learning*) [Garg và cộng sự, 2022], v.v.

9.1.3 Các chiến lược thiết kế prompt

Thiết kế prompt mang tính thực nghiệm rất cao. Nói chung, có nhiều cách để prompt một LLM thực hiện cùng một nhiệm vụ, và ta thường phải thử–sai nhiều lần mới tìm được một prompt vừa ý. Để viết prompt tốt hiệu quả hơn, ta có thể tuân theo một số chiến lược. Ví dụ về các nguyên tắc prompting phổ biến gồm:

- **Mô tả nhiệm vụ càng rõ ràng càng tốt.** Khi áp dụng LLM để giải quyết một vấn đề, ta cần mô tả vấn đề một cách chính xác, cụ thể, rõ ràng và hướng dẫn LLM hành xử như ta mong muốn. Điều này đặc biệt quan trọng khi ta muốn đầu ra của LLM đáp ứng những kỳ vọng nhất định. Ví dụ, giả sử ta tò mò về biến đổi khí hậu. Một prompt đơn giản để yêu cầu LLM cung cấp thông tin là

  > Hãy kể cho tôi về biến đổi khí hậu.

[2] **[ND]** Inuktitut là ngôn ngữ của người Inuit ở vùng Bắc Cực Canada; đây là một ngôn ngữ có rất ít dữ liệu văn bản trên Internet.

Vì chỉ dẫn này quá chung chung, LLM có thể sinh ra câu trả lời đề cập đến bất kỳ khía cạnh nào của biến đổi khí hậu, có thể không khớp với mối quan tâm cụ thể của ta. Trong trường hợp này, ta có thể dùng những prompt cụ thể và chi tiết hơn. Một ví dụ như vậy là

> Hãy giải thích chi tiết các nguyên nhân và hệ quả của biến đổi khí hậu, bao gồm tác động lên nhiệt độ toàn cầu, các kiểu thời tiết và mực nước biển. Đồng thời, hãy thảo luận các giải pháp khả thi và những hành động đang được thực hiện để giảm nhẹ các tác động này.

Bây giờ giả sử ta định giải thích về biến đổi khí hậu cho một em bé 10 tuổi. Ta có thể điều chỉnh thêm prompt ở trên.

> Hãy giải thích các nguyên nhân và hệ quả của biến đổi khí hậu cho một em bé 10 tuổi. Nói về việc nó ảnh hưởng thế nào đến thời tiết, mực nước biển và nhiệt độ. Đồng thời, hãy nhắc đến một số việc mọi người đang làm để giúp đỡ. Cố gắng giải thích bằng ngôn ngữ đơn giản và không vượt quá 500 từ.

- **Dẫn dắt LLM suy nghĩ.** LLM đã thể hiện một khả năng "suy nghĩ" mạnh đến bất ngờ. Một ví dụ thường gặp là các LLM được phát triển tốt đã đạt hiệu năng ấn tượng trong các nhiệm vụ lập luận toán học, vốn được xem là khó. Trong thiết kế prompt, khả năng suy nghĩ của LLM cần được kích hoạt bằng cách prompt phù hợp, đặc biệt với các bài toán đòi hỏi nhiều công sức suy luận. Trong nhiều trường hợp, một LLM được yêu cầu "suy nghĩ" có thể cho ra kết quả hoàn toàn khác so với khi yêu cầu chính mô hình đó thực hiện nhiệm vụ trực tiếp. Ví dụ, Kojima và cộng sự [2022] phát hiện rằng chỉ cần thêm câu "Let's think step by step" ("Hãy suy nghĩ từng bước một") vào cuối mỗi prompt là có thể cải thiện hiệu năng của LLM trên nhiều nhiệm vụ lập luận. Có nhiều cách để prompt LLM "suy nghĩ". Một phương pháp là yêu cầu LLM sinh ra các bước lập luận về bài toán trước khi đi đến câu trả lời cuối cùng. Ví dụ, hãy xét nhiệm vụ giải toán. Sau đây là một prompt đơn giản cho nhiệm vụ này.

> Bạn là một nhà toán học. Bạn sẽ được cung cấp một bài toán. Hãy giải bài toán đó.

Vì giải toán đòi hỏi một quá trình lập luận chi tiết, LLM có lẽ sẽ mắc lỗi nếu cố đưa ra đáp án ngay. Vì vậy, ta có thể yêu cầu tường minh LLM tuân theo một quy trình suy luận cho trước trước khi đi đến kết luận.

> Bạn là một nhà toán học. Bạn sẽ tuân theo các bước lập luận chi tiết sau khi giải toán.
>
> **Bước 1: Diễn giải bài toán.**
> Nhà toán học lắng nghe kỹ câu hỏi của bạn và hiểu các chi tiết phức tạp của thử thách toán học mà bạn đưa ra.
>
> **Bước 2: Xây dựng chiến lược.**
> Dựa trên vốn kiến thức sâu rộng của mình, nhà toán học chọn chiến lược hiệu quả nhất phù hợp với loại bài toán, dù đó là đại số, giải tích hay hình học.
>
> **Bước 3: Tính toán chi tiết.**
> Với sự chính xác và chuyên môn, nhà toán học thực hiện các phép tính cần thiết từng bước một, tuân thủ mọi nguyên tắc toán học.
>
> **Bước 4: Rà soát lời giải.**
> Trước khi đưa ra đáp án cuối cùng, nhà toán học kiểm tra tỉ mỉ các phép tính để đảm bảo độ chính xác và đưa ra lời giải thích hoặc lập luận ngắn gọn cho lời giải.
>
> Bạn sẽ được cung cấp một bài toán. Hãy giải bài toán đó.

{∗problem∗}

Một phương pháp khác để dẫn dắt LLM "suy nghĩ" là thông qua nhiều lượt tương tác với LLM. Ví dụ, ở bước đầu tiên, ta có thể yêu cầu LLM giải bài toán trực tiếp

Bạn sẽ được cung cấp một bài toán. Hãy giải bài toán đó.
{∗problem∗}

Bây giờ ta đã có một đáp án ban đầu cho bài toán. Ở bước thứ hai, ta prompt LLM đánh giá tính đúng đắn của đáp án và, nếu cần, làm lại để tìm ra lời giải tốt hơn.

Bạn sẽ được cung cấp một bài toán, cùng với một lời giải. Hãy đánh giá tính đúng đắn của lời giải này, và chỉ ra các lỗi nếu có. Sau đó, hãy tự giải bài toán theo cách của bạn.
Bài toán: {∗problem∗}
Lời giải: {∗solution∗}

Các prompt được trình bày ở đây liên quan chặt chẽ đến một dòng nghiên cứu dài về các bài toán lập luận (*reasoning*) trong LLM. Không thể thảo luận đầy đủ mọi vấn đề liên quan vì chủ đề này bao trùm một họ phương pháp rất lớn. Nhưng ta sẽ thấy một thảo luận tương đối chi tiết hơn về cách cải thiện prompting thông qua lập luận nhiều hơn ở Mục 9.2.

Chú giải của người dịch: Vì sao "suy nghĩ từng bước" lại giúp ích?

LLM sinh văn bản lần lượt từng token, token sau dựa vào các token đã viết trước đó. Nếu bắt mô hình viết ngay đáp số, nó phải "nhẩm" toàn bộ lời giải trong một bước — giống như học sinh làm bài toán nhiều bước mà chỉ được ghi đáp số. Khi được yêu cầu viết ra các bước trung gian, mỗi bước viết ra lại trở thành ngữ cảnh giúp bước sau dễ đúng hơn, tương tự như ta làm nháp. Cách "hai lượt" (giải trước, rồi tự kiểm tra) thì giống thói quen dò lại bài trước khi nộp.

- **Cung cấp thông tin tham khảo.** Như đã thảo luận ở mục trước, ta có thể đưa các ví dụ minh họa vào prompt và cho phép LLM học trong ngữ cảnh từ những minh họa này cách thực hiện nhiệm vụ. Thực ra, với khả năng hiểu ngôn ngữ đáng kể của LLM, ta có thể thêm bất kỳ loại văn bản nào vào prompt để các mô hình này dự đoán dựa trên ngữ cảnh phong phú hơn. Trong nhiều ứng dụng, ta có nhiều loại thông tin liên quan đến truy vấn của người dùng. Thay vì dùng LLM để dự đoán không ràng buộc, ta thường muốn LLM tạo ra đầu ra gói gọn trong phạm vi văn bản liên quan. Một ví dụ như vậy là RAG[3], trong đó văn bản liên quan đến truy vấn của người dùng được truy xuất bằng một hệ thống truy xuất thông tin (*information retrieval, IR*), rồi ta prompt LLM sinh ra câu trả lời dựa trên văn bản liên quan được cung cấp này. Prompt sau đây cho thấy một ví dụ.

Bạn là một chuyên gia có thể sinh câu trả lời cho các truy vấn đầu vào. Bây giờ bạn được cung cấp một truy vấn và thông tin ngữ cảnh tương ứng. Hãy sinh câu trả lời dựa trên thông tin ngữ cảnh này. Lưu ý rằng bạn cần đưa ra câu trả lời bằng lời lẽ của chính mình, không chỉ sao chép từ ngữ cảnh được cung cấp.
Thông tin ngữ cảnh: {∗IR-result∗}
Truy vấn: {∗query∗}

[3] **[ND]** RAG: sinh văn bản tăng cường truy xuất (*retrieval-augmented generation*).

Nếu thông tin ngữ cảnh rất đáng tin cậy, ta thậm chí có thể giới hạn LLM chỉ được trả lời bằng văn bản được cung cấp. Một prompt ví dụ như sau

> Bạn là một chuyên gia có nhiệm vụ sinh câu trả lời từ các truy vấn đầu vào. Bạn được cung cấp một truy vấn và thông tin ngữ cảnh tương ứng, được sắp xếp thành một bảng trong đó mỗi hàng là một bản ghi hữu ích. Hãy sinh câu trả lời chỉ dùng thông tin ngữ cảnh này. Hãy đảm bảo bạn đưa ra câu trả lời bằng lời lẽ của chính mình.
> Thông tin ngữ cảnh: {∗table∗}
> Truy vấn: {∗query∗}

Chú giải của người dịch: RAG — "làm bài có tài liệu"

RAG giống như một bài kiểm tra được mở sách: trước khi trả lời, hệ thống tìm kiếm (giống một máy tìm kiếm như Google) lấy ra vài đoạn văn bản liên quan đến câu hỏi ({∗IR-result∗}), rồi nhét chúng vào prompt. LLM đọc tài liệu đó và trả lời dựa trên nó, thay vì chỉ dựa vào "trí nhớ" từ lúc tiền huấn luyện — nhờ vậy câu trả lời cập nhật hơn và ít bịa đặt hơn.

Khi xử lý các bài toán thực tế, ta thường có tri thức tiên nghiệm và thông tin bổ sung về bài toán giúp tạo ra câu trả lời tốt hơn. Việc đưa những thông tin như vậy vào prompt nói chung có ích cho việc cải thiện kết quả.

- **Chú ý đến định dạng của prompt.** Nói chung, hiệu năng của LLM rất nhạy cảm với prompt mà ta đưa vào. Đôi khi một thay đổi nhỏ trong prompt có thể dẫn đến thay đổi lớn ở đầu ra của mô hình. Một ví dụ thú vị là việc thay đổi thứ tự các câu trong prompt có thể khiến LLM sinh ra kết quả khác. Để prompt dễ đọc và giảm sự mơ hồ, người ta thường định dạng chúng sao cho rõ ràng. Một ví dụ là ta định nghĩa vài trường cho prompt và điền thông tin khác nhau vào mỗi trường. Một ví dụ khác là ta có thể dùng prompt theo kiểu mã lệnh cho những LLM có thể hiểu và sinh ra cả ngôn ngữ tự nhiên lẫn mã. Xem prompt kiểu mã lệnh sau đây thực hiện việc dịch, trong đó có một ví dụ minh họa.

  > [English] = [I have an apple.]
  > [German] = [Ich habe einen Apfel.]
  > [English] = [I have an orange.]
  > [German] = ___

  (English: tiếng Anh; German: tiếng Đức. Hai câu tiếng Anh nghĩa là "Tôi có một quả táo" và "Tôi có một quả cam".)

  LLM có thể nhận văn bản ở nhiều định dạng khác nhau. Điều này cho phép ta dùng các ký tự điều khiển, thẻ XML, và các định dạng đặc thù để biểu diễn dữ liệu phức tạp. Hơn nữa, sẽ hữu ích nếu ta chỉ rõ đầu vào và đầu ra nên được định dạng hay cấu trúc như thế nào. Ví dụ, ta có thể phân tách các phần văn bản bằng dấu ngoặc kép và prompt LLM cho phù hợp (chẳng hạn thêm vào prompt một câu như "văn bản đầu vào được đặt giữa hai dấu ngoặc kép").

Ở trên, ta mới bàn về một vài chiến lược để viết prompt tốt. Tất nhiên còn rất nhiều phương pháp như vậy, và mỗi người cần tự phát triển phương pháp của mình qua thực hành. Bạn đọc quan tâm có thể tham khảo nhiều tài liệu trực tuyến để biết thêm thông tin, chẳng hạn hướng dẫn của OpenAI về các mô hình dòng GPT[4].

[4] Xem `https://platform.openai.com/docs/guides/prompt-engineering/six-strategies-for-getting-better-results`.

9.1.4 Thêm các ví dụ

Trong tiểu mục này, ta xét thêm các ví dụ về prompt LLM để thực hiện nhiều nhiệm vụ NLP khác nhau. Mục đích ở đây không phải là đưa ra các prompt chuẩn cho các nhiệm vụ này, mà là dùng những ví dụ đơn giản để minh họa cách prompt LLM nhằm giải quyết các bài toán NLP.

1. Phân loại văn bản

Phân loại văn bản (*text classification*) có lẽ là một trong những bài toán phổ biến nhất trong NLP. Nhiều nhiệm vụ có thể được xếp chung vào loại gán các nhãn định sẵn cho một văn bản cho trước. Ở đây, ta xét bài toán phân loại cực cảm xúc trong phân tích cảm xúc (*sentiment analysis*). Ta chọn phân loại cực cảm xúc để minh họa vì đây là một trong những nhiệm vụ phân loại văn bản phổ biến nhất và được định nghĩa rõ ràng nhất. Trong một thiết lập tổng quát cho phân loại cực cảm xúc, mục tiêu là xếp một văn bản cho trước vào một trong ba loại: tiêu cực, tích cực hoặc trung tính. Dưới đây là một prompt đơn giản cho nhiệm vụ này (để dễ đọc, chúng tôi tô sáng phần mô tả nhiệm vụ trong prompt).

Hãy phân tích cực cảm xúc của văn bản sau và phân loại nó là tích cực, tiêu cực hoặc trung tính.

Văn bản:
Dịch vụ ở nhà hàng chậm hơn dự kiến, điều này khiến tôi hơi bực mình.

Cực cảm xúc của văn bản có thể được phân loại là tiêu cực.

Để đầy đủ, chúng tôi đưa ra cả câu trả lời do LLM sinh ra (phần chữ được gạch dưới).

Mặc dù câu trả lời là đúng, LLM lại tạo ra một câu mô tả thay vì một nhãn rời rạc. Khó khăn này nảy sinh vì về bản chất LLM được thiết kế để sinh văn bản chứ không phải để gán nhãn rời rạc. Do đó, phân loại được xem như một bài toán sinh văn bản. Kết quả là ta cần một hệ thống khác để ánh xạ đầu ra của LLM vào không gian nhãn (một quá trình gọi là **ánh xạ nhãn** (*label mapping*)). Chẳng hạn, ta trích ra "tiêu cực" từ câu "Cực cảm xúc của văn bản có thể được phân loại là tiêu cực". Việc này đơn giản trong hầu hết các trường hợp vì ta có thể nhận ra các từ nhãn bằng những quy tắc kinh nghiệm (*heuristics*) đơn giản. Nhưng thỉnh thoảng LLM có thể không diễn đạt kết quả phân loại bằng các từ nhãn này. Khi đó bài toán trở nên phức tạp hơn, vì ta cần một ánh xạ từ văn bản được sinh ra sang các nhãn định sẵn.

Một phương pháp để thu được nhãn đầu ra từ LLM là phát biểu lại bài toán thành một nhiệm vụ điền vào chỗ trống (*cloze task*). Ví dụ, sau đây là một prompt kiểu điền vào chỗ trống cho phân loại cực cảm xúc.

Hãy phân tích cực cảm xúc của văn bản sau và phân loại nó là tích cực, tiêu cực hoặc trung tính.

Văn bản:
Dịch vụ ở nhà hàng chậm hơn dự kiến, điều này khiến tôi hơi bực mình.

Cực cảm xúc của văn bản là tiêu cực

Ta có thể dùng LLM để hoàn thành văn bản và điền vào chỗ trống từ phù hợp nhất. Lý tưởng nhất, từ được điền phải là positive (tích cực), negative (tiêu cực) hoặc neutral (trung tính). Tuy nhiên, không có gì bảo đảm LLM sẽ sinh ra đúng các từ nhãn này. Một phương pháp để giải quyết vấn đề này là giới hạn không gian đầu ra vào tập các từ nhãn và chọn từ có xác suất cao nhất. Khi đó, nhãn đầu ra được

cho bởi

$$\text{label} = \arg\max_{y \in Y} \Pr(y|\mathbf{x}), \tag{9.2}$$

trong đó y ký hiệu token (hoặc từ) được điền vào chỗ trống, và Y ký hiệu tập các từ nhãn $\{\text{positive}, \text{negative}, \text{neutral}\}$.

Chú giải của người dịch: Đọc Phương trình (9.2)

$\arg\max_{y \in Y}$ nghĩa là "chọn phần tử y trong tập Y làm cho biểu thức phía sau lớn nhất". Ví dụ, sau khi đọc prompt $\mathbf{x}$, giả sử mô hình gán xác suất cho từ tiếp theo là: "tiêu cực" 0.62, "trung tính" 0.20, "tích cực" 0.03, còn lại 0.15 dành cho mọi từ khác (như "khá", "hơi",...). Ta bỏ qua các từ khác, chỉ so sánh ba từ nhãn, và chọn "tiêu cực" vì nó có xác suất lớn nhất. Nhờ vậy đầu ra luôn là một trong ba nhãn hợp lệ.

Một cách tiếp cận khác để sinh nhãn bằng LLM là ràng buộc đầu ra bằng prompt. Ví dụ, ta có thể prompt LLM dự đoán từ một tập nhãn định sẵn. Đây là một ví dụ.

Hãy phân tích cực cảm xúc của văn bản sau và phân loại nó là tích cực, tiêu cực hoặc trung tính.

Văn bản:
Dịch vụ ở nhà hàng chậm hơn dự kiến, điều này khiến tôi hơi bực mình.

Cực cảm xúc của văn bản là gì?
Chỉ trả lời: tích cực, tiêu cực, hoặc trung tính.

Tiêu cực

Phân tích cảm xúc là một bài toán NLP phổ biến mà LLM nhìn chung đã "hiểu" khá tốt thông qua tiền huấn luyện hoặc tinh chỉnh. Do đó, ta có thể prompt LLM bằng những chỉ dẫn đơn giản để thực hiện nhiệm vụ. Tuy nhiên, với những bài toán phân loại mới, có thể cần cung cấp thêm chi tiết về nhiệm vụ, chẳng hạn các tiêu chuẩn phân loại, để LLM có thể làm đúng. Để làm điều này, ta có thể thêm một mô tả nhiệm vụ chi tiết hơn và/hoặc minh họa các ví dụ phân loại trong prompt. Để minh họa, hãy xét ví dụ sau.

Hãy phân tích cực cảm xúc của văn bản sau và phân loại nó là tích cực, tiêu cực hoặc trung tính. Dưới đây là ý nghĩa của từng loại:
Tích cực: Văn bản truyền tải một cảm xúc hoặc thái độ tích cực. Ví dụ, các văn bản thể hiện niềm vui, sự hài lòng, sự hào hứng hoặc sự ngưỡng mộ được coi là tích cực.
Tiêu cực: Văn bản thể hiện một cảm xúc hoặc thái độ tiêu cực. Nó bao gồm các cảm xúc buồn bã, tức giận, bực bội hoặc chỉ trích.
Trung tính: Cảm xúc trung tính được dùng để mô tả những văn bản không thể hiện rõ cảm xúc tích cực hay tiêu cực mà mang giọng điệu cung cấp thông tin, nêu sự thật, hoặc dửng dưng.

Văn bản:
Dịch vụ ở nhà hàng chậm hơn dự kiến, điều này khiến tôi hơi bực mình.

Cực cảm xúc của văn bản là gì?

Tiêu cực

Mặc dù có vẻ dùng LLM cho các bài toán phân loại là đơn giản, vẫn còn những vấn đề chưa được giải quyết tốt. Ví dụ, khi phải xử lý một số lượng lớn các lớp, việc prompt LLM một cách hiệu quả vẫn còn là thách thức. Lưu ý rằng nếu ta gặp một bài toán phân loại rất khó và có một lượng dữ liệu được

gán nhãn nhất định, thì việc tinh chỉnh LLM hoặc dùng các kiến trúc kiểu "BERT + bộ phân loại" cũng là lựa chọn đáng cân nhắc.

2. Trích xuất thông tin

Nhiều nhiệm vụ NLP có thể được phát biểu thành các bài toán trích xuất thông tin (*information extraction*), tức là xác định và trích ra những mẩu thông tin cụ thể từ văn bản phi cấu trúc. Những thông tin này bao gồm các thực thể có tên (*named entity*), các quan hệ, các sự kiện và các điểm dữ liệu liên quan khác. Mục tiêu của trích xuất thông tin là chuyển dữ liệu thô thành một định dạng có cấu trúc để có thể dễ dàng phân tích và sử dụng trong nhiều ứng dụng hạ nguồn.

Vì trích xuất thông tin bao trùm rất nhiều bài toán, ta không thể bàn hết ở đây. Thay vào đó, ta bắt đầu với nhiệm vụ nhận dạng thực thể có tên (*named entity recognition*) — một nhiệm vụ từ lâu đã là trọng tâm trong NLP. Nhận dạng thực thể có tên là nhiệm vụ xác định và phân loại các thực thể có tên trong văn bản. Những thực thể này thường bao gồm danh từ riêng và các biểu thức khác như ngày tháng, giá trị tiền tệ và tỉ lệ phần trăm. Trước tiên, hãy xét một ví dụ đơn giản về trích xuất tên người từ một văn bản cho trước[5].

Hãy xác định tất cả tên người trong văn bản được cung cấp.

Văn bản:
Liệu nước Anh có thực sự đang làm ăn tệ đến vậy, hay xu hướng du lịch đã thay đổi sau đại dịch? Theo Tom Jenkins, CEO của European Tourism Organisation, thì là điều thứ hai. "Tôi nghĩ UK vẫn đang làm rất tốt nhưng chúng ta sẽ thấy nhiều người đi đến châu Âu (Europe) hơn," ông nói về năm 2024, đồng thời cho biết "nghe nói nhu cầu của người Mỹ đối với UK đang hơi chững lại."

Tên người được xác định trong văn bản là:
Tom Jenkins

Chú giải của người dịch: Thực thể có tên

Thực thể có tên là những "đối tượng cụ thể có tên gọi riêng": người (Tom Jenkins), địa điểm (UK — Vương quốc Anh, Europe — châu Âu), tổ chức (European Tourism Organisation — Tổ chức Du lịch châu Âu), thời gian (2024)… Nhận dạng thực thể có tên giống như việc dùng bút dạ quang tô các tên riêng trong một bài báo rồi ghi bên cạnh đó là người, nơi chốn hay tổ chức. Trong bản dịch các ví dụ, chúng tôi giữ nguyên tên riêng tiếng Anh để khớp với kết quả mà LLM trả về trong sách gốc.

Tiếp theo, ta có thể mở rộng phạm vi để trích xuất mọi thực thể có tên.

Hãy xác định và phân loại tất cả các thực thể có tên trong văn bản được cung cấp vào các loại như tên người, địa điểm, ngày tháng và tổ chức. Liệt kê mỗi thực thể cùng với loại của nó trên một dòng.

Văn bản:
Liệu nước Anh có thực sự đang làm ăn tệ đến vậy, hay xu hướng du lịch đã thay đổi sau đại dịch? Theo Tom Jenkins, CEO của European Tourism Organisation, thì là điều thứ hai. "Tôi nghĩ UK vẫn đang làm rất tốt nhưng chúng ta sẽ thấy nhiều người đi đến châu Âu (Europe) hơn," ông nói về năm 2024, đồng thời cho biết "nghe nói nhu cầu của người Mỹ đối với UK đang hơi chững lại."

[5] Văn bản lấy từ `https://edition.cnn.com/travel`.

1. Tom Jenkins – Tên người
2. European Tourism Organisation – Tổ chức
3. UK – Địa điểm
4. Europe – Địa điểm
5. 2024 – Ngày tháng

Với các thực thể có tên đã xác định được, ta có thể áp dụng các nhiệm vụ trích xuất thông tin khác lên văn bản. Ví dụ, ta có thể xác định quan hệ giữa các thực thể có tên này (gọi là trích xuất quan hệ (*relation extraction*)). Prompt tương ứng được cho dưới đây.

Cho một văn bản và danh sách các thực thể có tên đã được xác định trong đó, hãy phân tích và mô tả các quan hệ giữa những thực thể này. Giải thích mỗi thực thể liên quan đến các thực thể khác như thế nào trong ngữ cảnh.

Văn bản:
Liệu nước Anh có thực sự đang làm ăn tệ đến vậy, hay xu hướng du lịch đã thay đổi sau đại dịch? Theo Tom Jenkins, CEO của European Tourism Organisation, thì là điều thứ hai. "Tôi nghĩ UK vẫn đang làm rất tốt nhưng chúng ta sẽ thấy nhiều người đi đến châu Âu (Europe) hơn," ông nói về năm 2024, đồng thời cho biết "nghe nói nhu cầu của người Mỹ đối với UK đang hơi chững lại."

Thực thể có tên:
Tom Jenkins, European Tourism Organisation, UK, Europe, 2024

Tom Jenkins – European Tourism Organisation: Tom Jenkins là CEO của European Tourism Organisation. Điều này cho thấy một mối liên hệ nghề nghiệp, trong đó Tom Jenkins có lẽ giữ vai trò quan trọng trong việc điều hành các hoạt động của tổ chức, đặc biệt là những hoạt động liên quan đến du lịch ở châu Âu.
…

Nếu một LLM đã được tinh chỉnh để làm theo chỉ dẫn, việc điều chỉnh nó cho các nhiệm vụ trích xuất thông tin khác nhau là tương đối đơn giản. Ví dụ, sau đây là một khuôn mẫu prompt tổng quát cho trích xuất thông tin.

Bạn sẽ được cung cấp một văn bản. Nhiệm vụ của bạn là {∗task-description∗}

Văn bản: {∗text∗}

Ở đây task-description (mô tả nhiệm vụ) là chỉ dẫn cho nhiệm vụ. Ví dụ về các chỉ dẫn như vậy gồm

trích xuất các từ khóa từ văn bản.
trích xuất các sự kiện chính từ văn bản.
phát hiện các liên kết đồng tham chiếu và hồi chỉ giữa các thực thể trong văn bản.

Thông thường, cung cấp một mô tả nhiệm vụ chi tiết hơn sẽ giúp LLM đưa ra câu trả lời chính xác hơn.

Chú giải của người dịch: Đồng tham chiếu và hồi chỉ

Hai cách diễn đạt *đồng tham chiếu* (*coreference*) khi chúng cùng chỉ một đối tượng. Trong văn bản ví dụ ở trên, "Tom Jenkins" và "ông" (trong "ông nói về năm 2024") cùng chỉ một người. *Hồi chỉ* (*anaphora*) là trường hợp một từ (thường là đại từ như "ông", "nó", "họ") quay lại chỉ một đối tượng đã được nhắc đến trước đó.

3. Sinh văn bản

Trong NLP, thuật ngữ *sinh văn bản* (*text generation*) được dùng theo nghĩa rộng, bao trùm nhiều loại bài toán, nhiều trong số đó tập trung vào việc sinh nội dung mới dựa trên một đầu vào. Có thể tạm chia các bài toán này thành hai nhóm: 1) viết tiếp từ một văn bản đầu vào ban đầu (gọi là **hoàn thành văn bản** (*text completion*)), và 2) chuyển văn bản đầu vào thành một văn bản khác (gọi là **biến đổi văn bản** (*text transformation*)). Ở đây, ta trình bày các ví dụ prompt cho nhóm thứ nhất và sẽ bàn về nhóm thứ hai ở phần sau.

Một ví dụ đơn giản của hoàn thành văn bản là viết tiếp một câu chuyện bắt đầu từ vài câu mở đầu.

Bạn là một nhà văn rất giỏi viết truyện. Bạn được cung cấp vài câu mở đầu. Hãy viết tiếp câu chuyện từ những câu đã cho.

Mở đầu câu chuyện:
Một buổi chiều nắng đẹp, Lily phát hiện một chiếc chìa khóa nhỏ xíu bằng vàng giấu dưới một hòn đá trong sân sau nhà mình. Tò mò, cô bé nhặt nó lên và tự hỏi nó có thể mở được thứ gì. Đúng lúc đó, chú mèo Whiskers của cô bắt đầu kêu meo meo đầy phấn khích, dẫn cô về phía cây sồi già ở rìa khu vườn.

Lily đi theo Whiskers, tim đập rộn ràng vì phấn khích xen chút hồi hộp. Cây sồi già lúc nào cũng có vẻ bí ẩn đối với cô, những cành cây xương xẩu của nó đổ những chiếc bóng dài ngay cả vào những ngày nắng nhất. Khi họ đến bên gốc cây, Whiskers lao lên trước và bắt đầu cào vào một thứ gì đó bị vùi dưới đống lá mùa thu …

Tương tự, ta có thể hoàn thành một cuộc hội thoại khi biết ngữ cảnh hội thoại.

Bạn được cung cấp một cuộc hội thoại giữa một du khách và một tài xế taxi. Hãy hoàn thành cuộc hội thoại.

Du khách: Anh có thể chở tôi đến bảo tàng ở trung tâm thành phố được không?
Tài xế taxi: Tất nhiên rồi! Lần đầu anh đến thành phố này à?
Du khách: Vâng, quanh đây có chỗ nào nhất định phải ghé không?
Tài xế taxi: Nhất định anh nên ghé vườn bách thảo sau khi đi bảo tàng. Nó ở gần đó và khá đẹp.

Du khách: Tuyệt quá, cảm ơn anh đã mách nhé!

Tài xế taxi: Không có gì. Chúc anh tham quan vui vẻ!

Ngoài việc viết tiếp đơn thuần, mô hình (khuôn khổ) này có thể được mở rộng cho các nhiệm vụ sinh văn bản theo chỉ dẫn, trong đó mô hình "hoàn thành" một câu trả lời dựa trên yêu cầu của người dùng. Những yêu cầu này có thể quy định văn phong, giọng điệu, độ dài và nội dung mong muốn. Chẳng hạn, người dùng có thể yêu cầu một bản báo cáo trang trọng, một câu chuyện sáng tạo, hoặc một đoạn mã chương trình. Dưới đây là một ví dụ prompt để sáng tác một bài thơ dựa trên các ràng buộc cho trước.

Bạn là một nhà thơ. Hãy viết một bài thơ theo các yêu cầu sau.

Chủ đề: Sự hồi sinh của mùa xuân
Cấu trúc: Thơ bốn câu với nhịp điệu đều đặn
Cảm xúc: Truyền tải cảm giác hạnh phúc và đổi mới qua những hình ảnh mùa xuân sống động.

Xuân thức dậy dưới nắng vàng rực rỡ
Hoa tươi non nhảy múa giữa ban mai
Đồng xanh thì thầm khúc ca vui khẽ
Hy vọng mới bừng lên theo gió xuân

(Bài thơ trong gốc là thơ tiếng Anh do LLM sáng tác; ở đây chúng tôi dịch nghĩa sang tiếng Việt.)

Hơn nữa, khi một LLM được tiền huấn luyện hoặc tinh chỉnh trên một kho ngữ liệu bao gồm cả ngôn ngữ tự nhiên lẫn ngôn ngữ lập trình, ta có thể prompt nó thực hiện các nhiệm vụ sinh mã từ văn bản. Trong trường hợp này, mô hình đơn giản là "dịch" các chỉ dẫn bằng ngôn ngữ tự nhiên thành mã chạy được. Ví dụ sau cho thấy cách prompt LLM viết một hàm Python.

Hãy viết một hàm Python để tính trung bình cộng của một danh sách các số.

```
def calculate_average(numbers):
    if numbers:  # Kiểm tra danh sách có rỗng không
        return sum(numbers) / len(numbers)
    else:
        return 0  # Trả về 0 nếu danh sách rỗng
```

Chú giải của người dịch: Đọc đoạn mã Python

Hàm `calculate_average` ("tính trung bình") nhận vào một danh sách số `numbers`. Nếu danh sách không rỗng, nó trả về `sum(numbers) / len(numbers)`, tức tổng các số chia cho số lượng số — đúng công thức trung bình cộng ở THPT; ví dụ với $[2, 4, 9]$ kết quả là $15/3 = 5$. Nếu danh sách rỗng thì trả về 0 để tránh phép chia cho 0. Phần sau dấu `#` là lời chú thích cho người đọc, máy tính bỏ qua (trong gốc chú thích viết bằng tiếng Anh).

4. Biến đổi văn bản

Các nhiệm vụ biến đổi văn bản điển hình gồm dịch máy (*machine translation*), tóm tắt (*summarization*), chuyển đổi văn phong (*text style transfer*), v.v. Các prompt dùng để thực hiện những nhiệm vụ này không khác biệt đáng kể so với các prompt đã trình bày trước đó. Ở đây, ta đưa ra vài ví dụ để minh họa các prompt này.

Hãy dịch văn bản sau từ tiếng Anh sang tiếng Tây Ban Nha.

Văn bản:
The quick brown fox jumps over the lazy dog.

El rápido zorro marrón salta sobre el perro perezoso.

(Nghĩa: "Con cáo nâu nhanh nhẹn nhảy qua con chó lười biếng." Câu tiếng Anh và câu tiếng Tây Ban Nha được giữ nguyên vì đây là ví dụ dịch giữa hai ngôn ngữ này.)

Hãy tóm tắt bài viết sau trong không quá 50 từ:

Bài viết:
Trong những năm gần đây, các khu vực đô thị đang phải đối mặt với những thách thức đáng kể liên quan đến tính bền vững và quản lý môi trường. Các thành phố đang chịu mức ô nhiễm cao hơn, tình trạng tắc nghẽn giao thông gia tăng và nhu cầu lớn hơn về cơ sở hạ tầng. Sự tăng trưởng này đã dẫn đến nhiều vấn đề môi trường, bao gồm mức độ ô nhiễm không khí và nước tăng cao, lượng rác thải gia tăng, và các dịch vụ công bị quá tải …

Các khu vực đô thị đang vật lộn với những thách thức về tính bền vững, như ô nhiễm gia tăng, tắc nghẽn giao thông và nhu cầu về cơ sở hạ tầng …

Hãy viết lại văn bản này với giọng điệu trang trọng.

Văn bản:
Ê, dạo này sao rồi? Lâu quá không gặp!

Xin chào, dạo này bạn thế nào? Đã khá lâu rồi kể từ lần cuối chúng ta gặp nhau!

5. Hỏi đáp

Trong hỏi đáp (*question answering*), một hệ thống nhận một câu hỏi và được kỳ vọng đưa ra một câu trả lời phù hợp. Các khuôn mẫu prompt cho hỏi đáp đa mục đích có thể là:

{∗question∗}

___

Câu hỏi: {∗question∗}
Trả lời: ___

Q: {∗question∗}
A: ___

Hỏi đáp quan trọng trong NLP vì nhiều bài toán có thể được phát biểu dưới dạng nhiệm vụ hỏi đáp. Đặc biệt, nhiều nhiệm vụ lập luận gần đây được định nghĩa dưới dạng hỏi đáp. Ví dụ, trong bộ đánh giá chuẩn MMLU [Hendrycks và cộng sự, 2021a], mỗi mẫu gồm một câu hỏi trắc nghiệm, và LLM được yêu cầu chọn đáp án đúng. Sau đây là một prompt ví dụ để trả lời một câu hỏi trong tập dữ liệu này.

$(\mathbb{Z}, *)$ là một nhóm với $a * b = a + b + 1$ với mọi a, b thuộc $\mathbb{Z}$. Phần tử nghịch đảo của a là
(A) 0
(B) -2
(C) $a - 2$
(D) $(2 + a) * -1$

D

Chú giải của người dịch: Câu hỏi MMLU ở trên nói gì?

Đây là một câu hỏi đại số trình độ đại học. $\mathbb{Z}$ là tập số nguyên, và ta định nghĩa một "phép toán mới" $a * b = a + b + 1$ (ví dụ $2 * 3 = 6$). "Phần tử đơn vị" e là số thỏa $a * e = a$, tức $a + e + 1 = a$, nên $e = -1$. "Phần tử nghịch đảo" của a là số b thỏa $a * b = e = -1$, tức $a + b + 1 = -1$, suy ra $b = -a - 2$. Đáp án (D): $(2 + a) * (-1) = (2 + a) + (-1) + 1 = 2 + a$; nếu hiểu biểu thức là $(2 + a) \cdot (-1)$ (nhân thông thường) thì được $-a - 2$, khớp với kết quả trên — đó là cách hiểu cho đáp án D mà đề này dùng. Bạn không cần nắm phần toán này để theo dõi chương; điều quan trọng là LLM chỉ cần trả lời bằng một chữ cái.

Một bộ đánh giá chuẩn khác được dùng rộng rãi là tập dữ liệu GSM8K [Cobbe và cộng sự, 2021]. Nó gồm hàng nghìn bài toán đố (có lời văn) trình độ tiểu học. Với mỗi bài toán, ta dùng LLM để sinh ra lời giải bằng ngôn ngữ tự nhiên. Ví dụ, sau đây là một prompt dùng để giải các bài toán trong GSM8K.

DEMO Q: Hai cô con gái của Jerry chơi bóng mềm ở hai đội khác nhau. Mùa giải này mỗi đội có 8 trận đấu. Mỗi đội tập luyện 4 giờ cho mỗi trận đấu. Nếu mỗi trận kéo dài 2 giờ, thì tổng cộng Jerry sẽ dành bao nhiêu giờ ở sân để xem các con gái thi đấu và tập luyện?

A: Jerry sẽ dành 8 trận * 2 giờ mỗi trận $=\ll 8*2=16 \gg$ 16 giờ để xem một cô con gái thi đấu. Ông sẽ dành $16*2 =\ll 16*2=32 \gg$ 32 giờ để xem cả hai con gái thi đấu. Ông sẽ dành 8 trận * 4 giờ tập $=\ll 8*4=32 \gg$ 32 giờ để xem một cô con gái tập luyện. Ông sẽ dành $32*2 =\ll 32*2=64 \gg$ 64 giờ để xem cả hai con gái tập luyện. Ông sẽ dành tổng cộng 32 giờ xem thi đấu + 64 giờ xem tập luyện $=\ll 32+64=96 \gg$ 96 giờ. #### 96

Q: Mary mua sáu quả táo ở cửa hàng. Từ số táo đã mua, cứ mỗi quả Mary ăn, cô lại trồng hai cây từ những quả còn lại. Mary đã ăn bao nhiêu quả táo?

A: Cô đã trồng tám cây. Điều này nghĩa là cô đã dùng một nửa số đó là táo, tức là 8 cây $/2$ cây/quả táo $=\ll 8/2=4 \gg$ 4 quả táo. Nghĩa là cô đã trồng bốn trong sáu quả táo đã mua, chỉ còn lại 6 quả táo $-$ 4 quả táo $=\ll 6-4=2 \gg$ 2 quả táo để ăn. #### 2

USER Q: Boris có 100 quả táo. Beck có ít hơn Boris 23 quả táo. Nếu Boris cho Beck 10 quả táo, thì bây giờ Beck có ít hơn Boris bao nhiêu quả táo?

A: Boris bắt đầu với 100 quả táo và cho Beck 10 quả táo, còn lại 100 quả táo -10 quả táo $=\ll 100-10=90 \gg$ 90 quả táo. Ban đầu, Beck có ít hơn Boris 23 quả táo, nên số táo ban đầu của Beck là $100-23 =\ll 100-23=77 \gg$ 77 quả táo. Sau khi nhận 10 quả táo từ Boris, tổng số táo của Beck là 77 quả táo $+10$ quả táo $=\ll 77+10=87 \gg$ 87 quả táo. Do đó, bây giờ Beck có ít hơn Boris $90-87 =\ll 90-87=3 \gg$ 3 quả táo. #### 3

Ở đây, một prompt few-shot được sử dụng. Từ các ví dụ minh họa gồm những cặp bài toán–lời giải này, LLM không chỉ học cách giải bài toán mà còn học cả cách định dạng đầu ra. Ví dụ, kết quả cuối cùng của phép tính xuất hiện sau token ####, và $\ll \cdots \gg$ chú thích các bước tính toán chi tiết (gọi là chú thích tính toán (*calculation annotation*))[6].

Chú giải của người dịch: Ví dụ few-shot vừa dạy cách giải, vừa dạy cách trình bày

Hai bài mẫu (DEMO) cho mô hình thấy một "khuôn trình bày" cố định: viết lời giải từng bước, mỗi phép tính được bọc trong $\ll \gg$, và đáp số cuối cùng đặt sau ####. Nhờ khuôn cố định này, một chương trình đơn giản có thể tự động lấy đáp số (chỉ cần đọc phần sau ####) để chấm điểm. Còn các phép tính trong $\ll \gg$ có thể được giao cho máy tính tính hộ thay vì để LLM "tính nhẩm" — vì LLM vốn dự đoán chữ chứ không thực sự làm phép tính, nên đôi khi tính sai những phép tính đơn giản. Ở câu hỏi của USER, phần gạch dưới là lời giải do LLM sinh ra, theo đúng khuôn của hai bài mẫu.

9.2 Các phương pháp prompting nâng cao

Cho đến đây trong chương này, chúng ta đã giới thiệu các khái niệm cơ bản liên quan đến việc prompt LLM và trình bày một số prompt cho các nhiệm vụ NLP. Giờ ta xem xét một vài kỹ thuật giúp nâng cao hiệu quả của prompting.

[6]Trong quá trình dự đoán, một máy tính bỏ túi (chương trình tính toán) được dùng khi ta gặp $\ll \cdots \gg$. Cụ thể hơn, mỗi khi LLM gặp dấu "=" bên trong một $\ll \cdots \gg$, máy tính sẽ tính biểu thức ở vế trái của "=". Phương pháp này giúp giảm các lỗi tính toán do LLM gây ra.

9.2.1 Chuỗi suy nghĩ

Chúng ta đã nhiều lần gặp khái niệm **chuỗi suy nghĩ** (*chain of thought*) (**CoT**) trong chương này và các chương trước [Wei và cộng sự, 2022c; Chowdhery và cộng sự, 2022]. Các phương pháp CoT cung cấp một cách đơn giản để prompt LLM sinh ra quá trình lập luận từng bước cho các bài toán phức tạp, nhờ đó tiếp cận nhiệm vụ theo cách giống con người hơn. Thay vì đi thẳng tới kết luận, các phương pháp CoT hướng dẫn LLM sinh ra các bước lập luận, hoặc học từ các ví dụ minh họa quá trình lập luận chi tiết. Để minh họa CoT, ta xét bài toán tính toán đại số, như thường được mô tả trong các tài liệu. Giả sử ta có một bài toán đại số

Tính trung bình cộng của các số 2, 4 và 9.

Ta có thể đặt nó thành một câu hỏi để prompt LLM.

Q: Hãy tính trung bình cộng của các số 2, 4 và 9.
A: Đáp án là 6.

Với những nhiệm vụ lập luận không tầm thường, việc prompt trực tiếp một LLM không phải lúc nào cũng cho đáp án đúng. Một cải tiến đơn giản là thêm vào prompt các ví dụ minh họa của những bài toán tương tự, nhờ đó LLM có thể học từ các ví dụ này.

Q: Hãy tính trung bình cộng của các số 1, 3, 5 và 7.
A: Đáp án là 4.

Q: Hãy tính trung bình cộng của các số 2, 4 và 9.
A: Đáp án là 7.

Vấn đề ở đây là mặc dù ta đã cho thấy một cặp câu hỏi–đáp án tương tự, LLM vẫn khó lập luận ra đáp án đúng. Trong CoT, LLM không chỉ học từ sự tương ứng giữa câu hỏi và đáp án mà còn được lợi từ các bước giải chi tiết dùng để đi tới đáp án. Để làm điều này, ta có thể đưa một số bước lập luận vào prompt để có được một *prompt CoT*.

Q: Hãy tính trung bình bình phương của các số 1, 3, 5 và 7.
A: Tính bình phương của từng số: $1^2 = 1$, $3^2 = 9$, $5^2 = 25$ và $7^2 = 49$. Cộng các bình phương lại, $1 + 9 + 25 + 49 = 84$. Có tất cả 4 số. Chia tổng cho số phần tử, $84/4 = 21$. Đáp án là 21.

Q: Hãy tính trung bình cộng của các số 2, 4 và 9.
A: Tính $2 + 4 + 9$, kết quả bằng 15. Có ba số. Chia tổng cho số lượng các số, được $15/3 = 5$. Đáp án là 5.

Ở đây các bước lập luận được tô màu xanh lá. Nhờ được cung cấp các ví dụ minh họa lập luận chi tiết, LLM được khuyến khích sinh ra các bước trung gian dẫn tới đáp án đúng.

Chú giải của người dịch: Vì sao thêm các bước giải lại giúp LLM?

Ở prompt thứ hai, LLM chỉ thấy "câu hỏi → đáp số" nên nó phải "đoán" đáp số trong một bước (và đoán sai: 7). Trung bình cộng đúng là $(2 + 4 + 9)/3 = 15/3 = 5$. Khi bài mẫu trình bày cả cách làm (cộng lại, đếm số phần tử, chia), LLM bắt chước viết ra từng bước, và mỗi bước nhỏ thì dễ làm đúng hơn nhiều. Vì LLM sinh văn bản lần lượt từng token, các bước đã viết ra trở thành một phần ngữ cảnh giúp nó dự đoán đáp số

cuối cùng — giống như học sinh làm nháp ra giấy thay vì tính nhẩm. Lưu ý rằng bài mẫu (trung bình bình phương) không giống hệt bài cần giải; mô hình học *kiểu lập luận* chứ không chép đáp số.

Việc dùng prompting CoT có một số lợi ích. Thứ nhất, CoT cho phép LLM phân rã các bài toán phức tạp thành những bước lập luận nhỏ hơn, nối tiếp nhau. Điều này giống cách con người giải quyết bài toán phức tạp từng bước một, nên nó đặc biệt hiệu quả với những nhiệm vụ đòi hỏi suy luận chi tiết, nhiều bước. Thứ hai, CoT làm cho quá trình lập luận minh bạch và dễ diễn giải hơn. Vì các bước lập luận trung gian đều hiện ra, ta có thể hiểu rõ hơn kết luận đã được rút ra như thế nào. Thứ ba, nếu người dùng theo dõi được logic đằng sau quá trình lập luận, họ sẽ có xu hướng tin tưởng các dự đoán của LLM hơn. Điều này đặc biệt quan trọng khi áp dụng LLM trong các lĩnh vực như y tế, giáo dục và tài chính. Thứ tư, CoT là một cách tiếp cận học trong ngữ cảnh, vì vậy nó áp dụng được cho hầu hết các LLM có sẵn đã được huấn luyện tốt. Hơn nữa, CoT cung cấp những cách hiệu quả để thích nghi LLM với các loại bài toán khác nhau. Nó thậm chí có thể gợi mở những lời giải sáng tạo hơn bằng cách khám phá nhiều đường lập luận thay thế khác nhau, những đường mà có thể không hiển nhiên khi đi thẳng tới kết luận.

Phương pháp mô tả ở trên đòi hỏi cung cấp một hoặc nhiều ví dụ lập luận CoT, thường được gọi là phương pháp *CoT few-shot*. Ngược lại, phương pháp *CoT zero-shot* không cần các ví dụ như vậy. Thay vào đó, nó prompt LLM lập luận từng bước bằng cách đưa các chỉ dẫn cụ thể vào prompt. Ví dụ, dưới đây là một prompt CoT zero-shot.

Q: Hãy tính trung bình cộng của các số 2, 4 và 9.
A: Hãy suy nghĩ từng bước một.
Ta có ba số: 2, 4 và 9. Cộng các số này lại, $2 + 4 + 9 = 15$. Xác định có bao nhiêu số, trong trường hợp này là ba. Trung bình cộng được tính bằng cách chia tổng cho số phần tử. Thực hiện phép chia được $15/3 = 5$. Vậy đáp án là 5.

Theo chỉ dẫn "Hãy suy nghĩ từng bước một" (*Let's think step by step*), LLM được prompt để sinh ra các bước lập luận chi tiết. Như đã bàn trong công trình của Kojima và cộng sự [2022], prompt bằng những chỉ dẫn như vậy có thể khiến LLM chỉ sinh ra các bước lập luận mà không có kết luận rõ ràng. Trong trường hợp này, có thể dùng một vòng prompting thứ hai để trích xuất đáp án từ các bước lập luận đó. Chẳng hạn, Kojima và cộng sự [2022] tạo một prompt thứ hai kết hợp cả đầu vào lẫn đầu ra của vòng prompting thứ nhất. Dùng đầu vào kết hợp này, LLM có thể tiếp tục quá trình lập luận rồi sinh ra đáp án đúng. Hơn nữa, có thể prompt LLM lập luận bằng những chỉ dẫn khác ngoài "Hãy suy nghĩ từng bước một", chẳng hạn "Hãy suy nghĩ một cách logic" và "Hãy cho tôi xem các bước suy nghĩ của bạn trước".

Mặc dù ta đã minh họa các phương pháp CoT bằng một bài toán lập luận đại số, các phương pháp này có thể áp dụng cho nhiều loại bài toán. Các ứng dụng điển hình của CoT gồm lập luận toán học, lập luận logic, lập luận thường thức, lập luận ký hiệu, sinh mã, v.v. Xem Hình 9.1 để có thêm các ví dụ về việc áp dụng CoT trong nhiều nhiệm vụ khác nhau.

CoT là một trong những chủ đề nghiên cứu sôi động nhất của thiết kế prompt. Nó không chỉ cải thiện hiệu năng của việc prompt LLM mà còn thúc đẩy rất nhiều nghiên cứu về đánh giá và kiểm chứng năng lực lập luận của LLM. Mặc dù trong mục này ta tập trung vào ý tưởng cơ bản của CoT, nó có thể được cải tiến theo nhiều cách. Chẳng hạn, ta có thể xem quá trình lập luận là bài toán tìm kiếm qua nhiều đường đi khả dĩ, mỗi đường có thể gồm nhiều trạng thái trung gian (tức các bước lập luận). Nói chung, ta muốn không gian tìm kiếm được định nghĩa rõ ràng và đủ lớn, để có nhiều khả năng tìm được kết quả tối ưu hơn. Vì lý do này, một hướng nghiên cứu LLM hiện nay nhằm thiết kế các cấu trúc tốt

hơn để biểu diễn quá trình lập luận, cho phép LLM giải quyết những thử thách lập luận phức tạp hơn. Các cấu trúc này gồm cấu trúc dạng cây [Yao và cộng sự, 2024], cấu trúc dạng đồ thị [Besta và cộng sự, 2024], v.v. Nhờ dùng các biểu diễn gọn này cho các đường lập luận, LLM có thể khám phá một phạm vi rộng hơn các đường ra quyết định, tương tự như tư duy Hệ thống 2[7]. Một hướng nghiên cứu khác tập trung vào việc prompt LLM bằng các tương tác nhiều vòng. Điều này bao gồm phân rã bài toán phức tạp thành các bài toán con, kiểm chứng và tinh chỉnh đầu ra của mô hình, sử dụng kết hợp nhiều mô hình (*model ensembling*), v.v. Lưu ý rằng các phương pháp này và những vấn đề liên quan không chỉ giới hạn ở CoT. Thực tế, chúng thường được dùng như những cách tiếp cận tổng quát hơn để cải thiện LLM, còn CoT có thể được xem là một cách để kiểm tra năng lực của LLM. Ta sẽ thấy phần thảo luận về một số vấn đề này trong các tiểu mục tiếp theo.

Chú giải của người dịch: Suy luận như một bài toán tìm kiếm

Hãy hình dung việc giải một bài toán như đi trong mê cung: mỗi bước lập luận là một ngã rẽ, và một lời giải là một đường đi từ cửa vào (đề bài) tới cửa ra (đáp án). CoT thông thường chỉ đi *một* đường thẳng từ đầu đến cuối. Cấu trúc dạng *cây* cho phép tại mỗi bước thử vài khả năng khác nhau (rẽ nhánh) rồi bỏ các nhánh kém; cấu trúc dạng *đồ thị* còn cho phép gộp các nhánh lại với nhau. Cách làm này giống "Hệ thống 2" trong tâm lý học: khi giải một bài hình khó, ta không viết ngay đáp số theo cảm giác (Hệ thống 1) mà thử vài hướng, kiểm tra, quay lại khi đi vào ngõ cụt.

Trước khi kết thúc phần thảo luận về CoT, ta nên xét đến những hạn chế thực tế của nó. Một hạn chế lớn là trong các tình huống CoT few-shot cần có các ví dụ minh họa lập luận chi tiết, nhiều bước, vốn có thể khó thu thập, dù là tự động hay thủ công. Ngoài ra, không có phương pháp chuẩn nào để chia nhỏ bài toán phức tạp thành các bước giải đơn giản hơn. Việc này thường phụ thuộc nhiều vào kinh nghiệm của người dùng. Thêm nữa, lỗi ở các bước trung gian cũng có thể ảnh hưởng đến độ chính xác của kết luận cuối cùng. Để tìm hiểu thêm về ưu và nhược điểm của CoT, bạn đọc quan tâm có thể tham khảo các bài tổng quan gần đây về chủ đề này [Chu và cộng sự, 2023; Yu và cộng sự, 2023b; Zhang và cộng sự, 2023a].

Chú giải của người dịch: Bốn nhiệm vụ trong Hình 9.1

CSQA (*CommonsenseQA*) và StrategyQA là các bộ câu hỏi về *thường thức*: câu trả lời không nằm sẵn trong đề mà cần hiểu biết đời thường (tivi cần cáp, hamster là con mồi…). *Ngôn ngữ Dyck* là các chuỗi dấu ngoặc đóng mở đúng cách, như ([ ]); lời giải dùng một *ngăn xếp* (*stack*) — giống chồng đĩa: ngoặc mở nào vào sau thì phải được đóng trước. Với đầu vào "< [[", ngăn xếp là < [[, nên phải đóng lần lượt]] >. *Ghép chữ cái cuối* là nhiệm vụ lập luận ký hiệu: lấy chữ cái cuối của mỗi từ rồi nối lại; các tên riêng được giữ nguyên vì nhiệm vụ phụ thuộc vào chính chữ cái của chúng.

9.2.2 Phân rã bài toán

Ta đã thấy LLM có thể được lợi khi giải một bài toán phức tạp bằng cách chia nhỏ nó thành những bài toán con đơn giản hơn. Cách tiếp cận như vậy có thể xem là một ví dụ của một mô thức rộng hơn gọi là **phân rã bài toán** (*problem decomposition*), vốn đã được khám phá và bàn luận rộng rãi trong tâm lý học và khoa học máy tính. Dưới góc độ tâm lý học, giải quyết vấn đề phức tạp là quá trình xử lý một

[7] Tư duy Hệ thống 1 và Hệ thống 2, như Kahneman [2011] mô tả, đại diện cho hai chế độ xử lý nhận thức khác nhau. Hệ thống 1 nhanh, tự động, trực giác và cảm tính. Chế độ tư duy này vận hành không tốn sức và nhanh chóng, thường là thứ dẫn dắt các quyết định, phán đoán và ấn tượng hằng ngày của chúng ta. Hệ thống 2 thì chậm, có chủ ý và mang tính phân tích. Nó được kích hoạt khi ta cần thực hiện những tính toán phức tạp.

Q: Thiết bị giải trí gia đình nào cần dùng cáp?
Các lựa chọn: (a) cửa hàng radio (b) trạm biến áp (c) tivi (d) tủ

A: Đáp án phải là thứ cần dùng cáp. Trong các lựa chọn trên, chỉ có tivi cần cáp. Vậy đáp án là (c).

Q: Kiểu người nào thường mắc bệnh?
Các lựa chọn: (a) bệnh viện (b) cái đầu (c) người ốm (d) người cao tuổi (e) phòng khám bác sĩ

A: Đáp án phải là một người dễ mắc bệnh. Trong các lựa chọn, "người cao tuổi" là đúng vì họ thường có hệ miễn dịch yếu hơn. Vậy đáp án là (d).

(a) CSQA

Q: Chuột hamster có là thức ăn cho loài động vật nào không?

A: Hamster là động vật bị săn mồi. Con mồi là thức ăn cho thú săn mồi. Do đó, hamster là thức ăn cho một số loài động vật. Vậy đáp án là có.

Q: Có thể dùng Microsoft Office mà không có internet không?

A: Có, bạn có thể dùng các ứng dụng Microsoft Office, mặc dù những tính năng như lưu trữ đám mây và cộng tác trực tuyến cần internet. Vậy đáp án là có.

(b) StrategyQA

Q: Hãy hoàn thành phần còn lại của chuỗi, bảo đảm các dấu ngoặc được đóng đúng cách. Đầu vào: [{

A: Hãy suy nghĩ từng bước một.
0: ngăn xếp rỗng
1: [; ngăn xếp: [
2: { ; ngăn xếp: [{
Vậy đáp án là }].

Q: Hãy hoàn thành phần còn lại của chuỗi, bảo đảm các dấu ngoặc được đóng đúng cách. Đầu vào: < [[

A: Hãy suy nghĩ từng bước một.
0: ngăn xếp rỗng
1: < ; ngăn xếp: <
2: [; ngăn xếp: < [
3: [; ngăn xếp: < [[
Vậy đáp án là]] >.

(c) Ngôn ngữ Dyck

Q: Lấy các chữ cái cuối của các từ trong "Elon Musk" rồi ghép chúng lại.

A: Chữ cái cuối của "Elon" là "n". Chữ cái cuối của "Musk" là "k". Ghép chúng lại được "nk". Đáp án là nk.

Q: Lấy các chữ cái cuối của từng từ trong "Renata Mariela Mona Kristin" rồi ghép chúng lại.

A: Chữ cái cuối của "Renata" là "a". Chữ cái cuối của "Mariela" là "a". Chữ cái cuối của "Mona" là "a". Chữ cái cuối của "Kristin" là "n". Ghép chúng lại được "aaan". Đáp án là aaan.

(d) Ghép chữ cái cuối

Hình 9.1: CoT trong bốn nhiệm vụ lập luận khác nhau, gồm CSQA, StrategyQA, ngôn ngữ Dyck (*Dyck languages*) và ghép chữ cái cuối (*Last Letter Concatenation*). Các phần CoT được tô màu xanh lá.

vấn đề bằng tri thức giúp vượt qua các rào cản của vấn đề đó[8]. Thường không có con đường rõ ràng nào để giải một vấn đề phức tạp. Tuy nhiên, việc dùng các chiến lược phân rã vấn đề thường có lợi, vì chúng giúp xử lý các bài toán con tương ứng dễ dàng hơn và ít tốn công hơn. Chẳng hạn, hãy xét việc viết một bài blog về các rủi ro của AI. Nếu ta chỉ đơn giản prompt LLM bằng chỉ dẫn "Hãy viết một bài blog về các rủi ro của AI", LLM có thể sinh ra một bài blog với cấu trúc và văn phong tùy ý. Một cách tốt hơn là lập dàn ý cho bài blog và cung cấp thông tin chi tiết hơn về từng phần. Hãy xét prompt sau

> Bạn là một người viết blog. Hãy làm theo dàn ý được cung cấp dưới đây để viết một bài blog về các rủi ro của AI.
>
> - Mở đầu
> Giới thiệu về AI, tầm quan trọng của nó, và tầm quan trọng của việc hiểu các rủi ro của nó đối với giới trẻ.
> - Lo ngại về quyền riêng tư
> Bàn về việc AI có thể xâm phạm quyền riêng tư cá nhân như thế nào thông qua các tương tác trực tuyến.
> - Thông tin sai lệch
> Tìm hiểu vai trò của AI trong việc lan truyền thông tin sai lệch và tác động đến các quyết định của người trẻ.
> - Bắt nạt trên mạng
> Làm nổi bật cách các công cụ AI có thể bị lợi dụng để bắt nạt trên mạng và tác động của việc này đến sức khỏe tinh thần.
> - Lời khuyên để dùng AI an toàn
> Đưa ra các hướng dẫn sử dụng AI có trách nhiệm và khuyến khích tư duy phản biện.
> - Kết luận
> Tóm tắt các ý chính và khuyến khích chủ động quan tâm đến đạo đức AI.
>
> ___

Ở đây ta đưa ra tiêu đề và các ý chính cho từng phần. Sau đó, LLM có thể dùng cấu trúc này để chia nhỏ nhiệm vụ viết bằng cách điền nội dung cho các phần ấy. Lưu ý rằng cách cấu trúc bài blog có thể do con người cung cấp, hoặc thậm chí được sinh tự động. Ví dụ, ta có thể dùng LLM để sinh dàn ý trước, rồi yêu cầu nó làm theo dàn ý đó để hoàn thành bài viết.

Trong khoa học máy tính, phân rã bài toán phức tạp là một chiến lược thường dùng trong thiết kế hệ thống phần mềm và phần cứng. Một ví dụ nổi tiếng là mô thức chia để trị (*divide-and-conquer*), thường được dùng để thiết kế thuật toán cho những bài toán tính toán có thể quy về các bài toán đơn giản hơn, dễ xử lý hơn. Ví dụ, xét bài toán xác định xem một tài liệu có bàn về các rủi ro của AI hay không. Ta có thể chỉ dẫn LLM bằng prompt sau.

> Bạn được cung cấp một văn bản. Hãy xác định xem văn bản đó có bàn về các rủi ro của AI hay không.
> {∗document∗}
>
> ___

Nếu tài liệu dài, việc suy luận có thể trở nên tốn kém về tính toán. Thay vào đó, ta có thể chia tài liệu thành các đoạn tương đối ngắn và thực hiện cùng một nhiệm vụ trên từng đoạn. Các đoạn này có thể được xử lý song song để giảm thêm chi phí tính toán. Tiếp theo, ta xác định mức độ liên quan của từng đoạn tới chủ đề rủi ro của AI. Đầu ra cuối cùng sau đó được sinh ra bằng một prompt khác.

[8] Một định nghĩa tương đối hình thức có thể tìm thấy trong sách của Frensch và Funke [2014]: *giải quyết vấn đề phức tạp diễn ra nhằm vượt qua các rào cản giữa một trạng thái cho trước và một trạng thái mục tiêu mong muốn thông qua các hoạt động hành vi và/hoặc nhận thức gồm nhiều bước*.

Nhiệm vụ của bạn là xác định xem một văn bản có bàn về các rủi ro của AI hay không. Văn bản này đã được chia thành các đoạn, và bạn đã có mức độ liên quan của từng đoạn tới chủ đề rủi ro của AI. Dựa vào đó, hãy đưa ra kết quả cuối cùng của bạn.

Đoạn 1: {∗relevance-to-the-topic1∗}
Đoạn 2: {∗relevance-to-the-topic2∗}
Đoạn 3: {∗relevance-to-the-topic3∗}
…

___

Chú giải của người dịch: Chia để trị

"Chia để trị" là một ý tưởng quen thuộc: muốn làm một việc lớn, ta chia nó thành nhiều việc nhỏ cùng loại, làm từng việc nhỏ rồi ghép kết quả lại. Ví dụ, muốn biết một cuốn sách 300 trang có nói về rủi ro của AI hay không, ta có thể nhờ 10 bạn mỗi người đọc 30 trang (làm *song song*, cùng lúc), mỗi bạn báo lại "phần của mình có liên quan hay không"; cuối cùng chỉ cần đọc 10 bản báo cáo ngắn để kết luận. Với LLM, chi phí xử lý tăng nhanh theo độ dài văn bản đầu vào (với Transformer, chi phí tự chú ý tăng theo bình phương độ dài), nên chia nhỏ văn bản thường rẻ hơn nhiều. Trong prompt thứ hai, {∗relevance-to-the-topic1∗} (*mức độ liên quan tới chủ đề* của đoạn 1) là chỗ trống sẽ được điền bằng kết quả của bước trước.

Giờ hãy quay lại phần thảo luận tổng quát hơn về phân rã bài toán trong prompting. Mặc dù phân rã bài toán có thể áp dụng cho nhiều bài toán NLP khác nhau, gần đây nó được thảo luận và thử nghiệm rộng rãi hơn cả trong các nhiệm vụ lập luận. Với các nhiệm vụ lập luận phức tạp, ta thường cần một đường lập luận nhiều bước để đi tới kết luận đúng. Ta có thể dùng LLM để thực hiện lập luận như vậy theo ba cách khác nhau. Thứ nhất, LLM có thể đi thẳng tới kết luận. Nói cách khác, chúng có thể dự đoán mà không có quá trình lập luận tường minh, và quá trình lập luận bên dưới vẫn ẩn và không diễn giải được. Thứ hai, LLM được prompt để sinh ra một đường lập luận nhiều bước dẫn tới kết luận, giống như CoT. Tuy nhiên, ta chỉ chạy LLM một lần, và mọi bước trung gian của quá trình lập luận được sinh ra trong một lần dự đoán duy nhất. Thứ ba, ta chia bài toán ban đầu thành một số bài toán con, mỗi bài toán con được xử lý trong các lần chạy LLM riêng biệt hoặc được giải bằng các hệ thống khác. Ở đây ta tập trung vào cách thứ ba, vốn liên quan chặt chẽ đến phân rã bài toán. Tuy nhiên cần lưu ý rằng một thảo luận đầy đủ hơn có thể bao quát cả ba cách này, còn hai cách đầu đã được bàn tới ở mức độ nào đó trong chương này.

Một khung tổng quát cho phân rã bài toán gồm hai thành phần.

- **Sinh bài toán con** (*Sub-problem Generation*). Thành phần này phân rã bài toán đầu vào thành một số bài toán con.

- **Giải bài toán con** (*Sub-problem Solving*). Thành phần này giải từng bài toán con và rút ra các kết luận trung gian cũng như kết luận cuối cùng thông qua suy luận.

Hai vấn đề này có thể được mô hình hóa theo những cách khác nhau, dẫn tới nhiều phương pháp phân rã bài toán khác nhau. Một cách tiếp cận là coi chúng là hai bước riêng biệt trong một quy trình hai bước. Chẳng hạn, xét nhiệm vụ viết blog đã mô tả ở đầu tiểu mục này. Ở bước thứ nhất, ta phân rã toàn bộ bài toán thành các bài toán con cùng một lúc (tức là lập dàn ý cho bài blog). Ở bước thứ hai, ta giải các bài toán con theo thứ tự lần lượt hoặc theo một thứ tự khác (tức là điền nội dung cho từng phần khi cần). Đầu ra cuối cùng của quy trình này kết hợp các kết quả giải từng bài toán con. Mặc dù phương pháp này đơn giản và dễ hiểu, nó giả định rằng bài toán có tính *kết hợp* (*compositional*) — tức lời giải của cả bài được ghép từ lời giải của các phần — nên nó phù hợp hơn với những nhiệm vụ như

viết văn và sinh mã.

Tuy nhiên, nhiều bài toán trong thực tế đòi hỏi lập luận phức tạp. Một đặc điểm then chốt của những bài toán này là đường lập luận của chúng không được cố định trước và có thể phụ thuộc vào các kết quả trung gian thu được trong quá trình lập luận. Trong những trường hợp đó, việc sinh trước các bài toán con một cách cố định là không mong muốn. Thay vào đó, các bài toán con nên được sinh ra một cách động dựa trên bài toán đầu vào, và nếu có thể, được sinh ngay trong lúc suy luận. Điều này khiến phân rã bài toán khó khăn hơn so với thiết kế các thuật toán chia để trị. Lý tưởng nhất, ta muốn thiết kế đồng thời cả hai hệ thống sinh bài toán con và giải bài toán con. Nhưng một cách tiếp cận thực tế hơn và được dùng rộng rãi là dùng các mô hình riêng cho các nhiệm vụ này. Một cách đơn giản để làm được điều đó là thích nghi một LLM cho các nhiệm vụ này bằng cách prompt hoặc tinh chỉnh mô hình.

Ở đây ta xét một phương pháp dựa trên ý tưởng trên, gọi là **prompting từ dễ đến khó** (*least-to-most prompting*) [Zhou và cộng sự, 2023b]. Động lực của phương pháp này xuất phát từ khó khăn khi giải các bài toán lập luận khó — những bài không thể giải quyết chỉ bằng cách khái quát hóa từ một vài ví dụ. Với những bài toán này, một chiến lược giải hiệu quả hơn là đi theo một chuỗi các bài toán con tăng dần, dẫn tới kết luận một cách có hệ thống. Cụ thể hơn, trong phương pháp prompting từ dễ đến khó, việc sinh bài toán con được thực hiện bằng cách prompt LLM với các chỉ dẫn và/hoặc ví dụ minh họa. Chẳng hạn, dưới đây là một prompt 2-shot dùng để sinh bài toán con trong prompting từ dễ đến khó.

TASK Nhiệm vụ của bạn là phân rã một bài toán thành vài bài toán con. Bạn sẽ được cho vài ví dụ để minh họa cách làm điều này.

DEMO Q: Trong một cộng đồng, 5% dân số là trẻ sơ sinh, 15% là trẻ em, 40% là người trưởng thành và 40% là người cao tuổi. Nhóm nào chiếm tỉ lệ dân số lớn nhất?

A: Để trả lời câu hỏi "Nhóm nào chiếm tỉ lệ dân số lớn nhất?", ta cần biết: "Trẻ sơ sinh chiếm bao nhiêu phần trăm?", "Trẻ em chiếm bao nhiêu phần trăm?", "Người trưởng thành chiếm bao nhiêu phần trăm?", "Người cao tuổi chiếm bao nhiêu phần trăm?".

Q: Alice, Bob và Charlie mang hạt cườm đến cho dự án nhóm trong giờ thủ công. Alice có số hạt cườm gấp đôi Bob, và Bob có số hạt cườm gấp năm lần Charlie. Nếu Charlie có 6 hạt cườm, thì họ có thể dùng bao nhiêu hạt cườm cho dự án thủ công của mình?

A: Để trả lời câu hỏi "Họ có thể dùng bao nhiêu hạt cườm cho dự án thủ công của mình?", ta cần biết: "Bob có bao nhiêu hạt cườm?", "Alice có bao nhiêu hạt cườm?".

USER Q: Nghiên cứu môi trường được tiến hành từ năm 2015 đến năm 2020 cho thấy nhiệt độ trung bình trong khu vực đã tăng 2.3 độ C. Nghiên cứu môi trường này kéo dài bao lâu?

A: Để trả lời câu hỏi "Nghiên cứu môi trường này kéo dài bao lâu?", ta cần biết: "Nghiên cứu môi trường bắt đầu khi nào?", "Nghiên cứu môi trường kết thúc khi nào?".

Bằng cách học từ các ví dụ, LLM có thể sinh ra hai bài toán con để trả lời bài toán mới "Nghiên cứu môi trường này kéo dài bao lâu?" (được tô màu xanh dương và cam). Có các bài toán con này rồi, ta giải chúng lần lượt. Với mỗi bài toán con, ta lấy tất cả các cặp hỏi–đáp đã sinh trước đó làm ngữ cảnh, rồi đưa ra câu trả lời. Với ví dụ trên, ta cần trả lời bài toán con thứ nhất bằng cách prompt LLM như sau

Nghiên cứu môi trường được tiến hành từ năm 2015 đến năm 2020 cho thấy nhiệt độ trung bình trong khu vực đã tăng 2.3 độ C.

SUB-PROB1 Q: Nghiên cứu môi trường bắt đầu khi nào?
A: Nghiên cứu môi trường bắt đầu vào năm 2015.

Khi đã có câu trả lời cho bài toán con thứ nhất, ta chuyển sang bài toán con thứ hai. Lần này, ta đưa cả bài toán con thứ nhất và câu trả lời tương ứng của nó vào đầu vào.

Nghiên cứu môi trường được tiến hành từ năm 2015 đến năm 2020 cho thấy nhiệt độ trung bình trong khu vực đã tăng 2.3 độ C.

SUB-PROB1 Q: Nghiên cứu môi trường bắt đầu khi nào?
A: Nghiên cứu môi trường bắt đầu vào năm 2015.

SUB-PROB2 Q: Nghiên cứu môi trường kết thúc khi nào?
A: Nghiên cứu môi trường kết thúc vào năm 2020.

Cuối cùng, ta dùng LLM để giải bài toán ban đầu khi đã có câu trả lời cho tất cả các bài toán con.

Nghiên cứu môi trường được tiến hành từ năm 2015 đến năm 2020 cho thấy nhiệt độ trung bình trong khu vực đã tăng 2.3 độ C.

SUB-PROB1 Q: Nghiên cứu môi trường bắt đầu khi nào?
A: Nghiên cứu môi trường bắt đầu vào năm 2015.

SUB-PROB2 Q: Nghiên cứu môi trường kết thúc khi nào?
A: Nghiên cứu môi trường kết thúc vào năm 2020.

FINAL Q: Nghiên cứu môi trường này kéo dài bao lâu?
A: Nghiên cứu môi trường kéo dài 5 năm.

Chú giải của người dịch: Prompting "từ dễ đến khó" hoạt động thế nào?

Tên gọi *least-to-most* nghĩa là "từ (bài) ít khó nhất đến (bài) khó nhất". Quy trình gồm hai giai đoạn: (1) nhờ LLM *liệt kê* các câu hỏi nhỏ cần trả lời trước (bắt đầu khi nào? kết thúc khi nào?); (2) lần lượt nhờ LLM trả lời từng câu hỏi nhỏ, mỗi lần đều "mang theo" các câu hỏi và câu trả lời trước đó trong prompt, rồi cuối cùng trả lời câu hỏi gốc: $2020 - 2015 = 5$ năm. Mỗi lần gọi LLM chỉ phải làm một việc nhỏ và dễ, nên ít sai hơn so với đòi nó làm tất cả trong một lần. Các màu (xanh dương, cam, đỏ) trong các hộp trên giúp ta theo dõi mỗi câu hỏi con và câu trả lời của nó đi qua các prompt như thế nào.

Phương pháp từ dễ đến khó cung cấp một cách tiếp cận cơ bản để prompt LLM sinh ra và giải các bài toán con một cách riêng rẽ. Ta có thể cải tiến nó theo nhiều cách. Một cải tiến đơn giản là áp dụng các kỹ thuật prompting nâng cao khác nhau, vốn không đòi hỏi thay đổi khung phân rã bài toán. Ví dụ, ta có thể đưa CoT vào prompt để nâng cao hiệu năng lập luận khi sinh và giải các bài toán con.

Một cải tiến khác là tìm hiểu các phương pháp phân rã bài toán và tổ chức các đường giải tốt hơn. Để mô tả những cách tiếp cận này, ta sẽ dùng ký hiệu p_0 để chỉ bài toán đầu vào, và dùng các ký hiệu $\{p_1, \cdots, p_n\}$ để chỉ các bài toán con tương ứng với p_0. Với prompting từ dễ đến khó, ta phân rã p_0 thành $\{p_1, \cdots, p_n\}$, cho bởi

$$\{p_1, \cdots, p_n\} = G(p_0), \tag{9.3}$$

trong đó $G(\cdot)$ là hàm sinh bài toán con. Sau đó, ta giải lần lượt các bài toán con $\{p_1, \cdots, p_n\}$, thu được một dãy câu trả lời $\{a_1, \cdots, a_n\}$. Để trả lời bài toán con thứ i là p_i, ta đưa cả bài toán ban đầu p_0 lẫn tất cả các cặp bài toán–câu trả lời đã sinh trước đó vào ngữ cảnh để dự đoán. Câu trả lời a_i được cho bởi

$$a_i \ = \ S_i(p_i, \{p_0, p_{<i}, a_{<i}\}), \tag{9.4}$$

trong đó $p_{<i} = \{p_1, \cdots, p_{i-1}\}$ và $a_{<i} = \{a_1, \cdots, a_{i-1}\}$. $S_i(\cdot)$ là hàm giải bài toán con p_i khi biết ngữ cảnh $\{p_0, p_{<i}, a_{<i}\}$. Bước cuối cùng là sinh câu trả lời cho bài toán ban đầu p_0, có thể được biểu diễn tương tự như Phương trình (9.4).

$$a_0 \ = \ S_0(p_0, \{p_{\leq n}, a_{\leq n}\}). \tag{9.5}$$

Chú giải của người dịch: Đọc các ký hiệu trong Phương trình (9.3–9.5)

Áp vào ví dụ nghiên cứu môi trường: p_0 = "Nghiên cứu kéo dài bao lâu?"; $G(p_0)$ trả về p_1 = "Bắt đầu khi nào?" và p_2 = "Kết thúc khi nào?" (ở đây $n = 2$). Khi giải p_2, ngữ cảnh gồm p_0 và các cặp đã có trước, tức $p_{<2} = \{p_1\}$, $a_{<2} = \{a_1\}$ với a_1 = "2015"; kết quả a_2 = "2020". Cuối cùng S_0 nhận p_0 cùng tất cả các cặp $p_{\leq 2}, a_{\leq 2}$ và trả lời a_0 = "5 năm". Ký hiệu "$< i$" nghĩa là "mọi chỉ số nhỏ hơn i", còn "$\leq n$" là "mọi chỉ số từ 1 đến n". Các "hàm" G và S_i ở đây thực chất là những lần gọi LLM với prompt phù hợp.

Một cách để cải tiến mô hình này là sửa đổi hàm $G(\cdot)$ để mô hình có thể sinh câu trả lời một cách động. Thay vì sinh tất cả các bài toán con cùng một lúc, ta có thể sinh từng bài toán con trong quá trình giải [Dua và cộng sự, 2022]. Để làm điều này, ta có thể thay Phương trình (9.3) bằng

$$p_i \ = \ G_i(p_0, \{p_{<i}, a_{<i}\}). \tag{9.6}$$

Nhờ vậy ta có một mô hình sinh bài toán con hoạt động theo kiểu từng bước. Ở bước i, đầu tiên ta prompt LLM bằng bài toán ban đầu p_0 cùng với lịch sử suy luận trước đó $\{p_{<i}, a_{<i}\}$ để sinh ra một bài toán con mới p_i. Sau đó ta giải bài toán con này bằng chính LLM đó hoặc một LLM khác, với điều kiện là cùng ngữ cảnh ấy (xem Phương trình (9.4)). Phương pháp này mở rộng hiệu quả năng lực lập luận của LLM bằng cách cho phép chúng sinh và giải các bài toán con một cách động trong các bước lập luận trung gian. Kết quả là các đường lập luận không bị cố định trước, và mô hình có thể lựa chọn và điều chỉnh chiến lược lập luận của mình trong quá trình giải.

Một cách khác để cải tiến mô hình trên là tập trung phát triển các bộ giải bài toán con tốt hơn. Trong phần thảo luận trước, ta giới hạn $S_i(\cdot)$ là các LLM được prompt để giải bài toán con p_i. Thực ra, ta có thể mở rộng hàm này thành bất kỳ hệ thống nào có khả năng xử lý bài toán con. Chẳng hạn, $S_i(\cdot)$ có thể gọi tới các hệ thống truy xuất thông tin (*information retrieval, IR*), nhờ đó cho phép ta truy cập một phạm vi dữ liệu rộng hơn để giải bài toán. Một ví dụ khác là dùng $S_i(\cdot)$ như một máy tính bỏ túi để tính chính xác kết quả khi giải toán. Nếu bài toán con p_i phức tạp và cần nhiều bước giải trung gian, ta cũng có thể phân rã tiếp p_i thành các bài toán con nhỏ hơn. Ví dụ, $S_i(\cdot)$ có thể được định nghĩa là một chương trình đệ quy sinh ra và giải các bài toán con. Điều này đưa đệ quy vào việc giải bài toán và cho phép ta xử lý bài toán bằng cách phân rã lặp đi lặp lại. Kết quả là ta có thể định nghĩa một cấu trúc phân cấp cho việc giải bài toán [Khot và cộng sự, 2023].

Chú giải của người dịch: Đệ quy

Một chương trình *đệ quy* là chương trình tự gọi lại chính nó trên một phiên bản nhỏ hơn của bài toán, cho tới khi bài toán đủ nhỏ để giải trực tiếp. Ví dụ quen thuộc: $n! = n \cdot (n-1)!$, với $1! = 1$. Ở đây, nếu bài toán con p_i vẫn còn khó, bộ giải S_i lại phân rã nó thành các bài toán con nhỏ hơn nữa, tạo thành một "cây" bài toán nhiều tầng (cấu trúc phân cấp).

Một bước tổng quát hóa xa hơn của cách phát biểu này là xem nó như một bài toán học tăng cường (*reinforcement learning*). Một phương pháp điển hình là mô hình hóa quá trình giải bài toán như một quá trình ra quyết định. Ở mỗi bước của quá trình này, một hành động được thực hiện dựa trên trạng thái hiện tại. Các hành động này có thể bao gồm tất cả các hàm sinh và giải bài toán con (tức $G_i(\cdot)$ và $S_i(\cdot)$). Như vậy, dãy hành động tương ứng với một đường giải bài toán. Vì việc thảo luận các bài toán học tăng cường nằm ngoài phạm vi chương này, ta bỏ qua mô tả chính xác của nhiệm vụ học này. Tuy vậy, phát triển một tác tử (*agent*) hay bộ điều khiển để quyết định khi nào và làm thế nào sinh và giải một bài toán con cũng là một lựa chọn tự nhiên.

Trong NLP, phân rã bài toán có liên quan tới một dòng nghiên cứu lâu đời về hỏi đáp đa bước (*multi-hop question answering*) [Mavi và cộng sự, 2024]. Nhiệm vụ này đòi hỏi hệ thống thu thập và kết hợp thông tin từ nhiều đoạn văn bản để đưa ra câu trả lời chính xác cho một câu hỏi phức tạp. Ví dụ, để trả lời câu hỏi "Thủ đô của quốc gia nơi Albert Einstein sinh ra là gì?", ta cần biết "Albert Einstein sinh ra ở đâu?" và "Thủ đô của Đức là gì?". Các công trình trước đây trong lĩnh vực này và các lĩnh vực liên quan đã nghiên cứu vấn đề phân rã bài toán, dù các phương pháp có thể không dựa trên LLM. Ví dụ, một phương pháp phổ biến là phát triển thêm một mô hình nơ-ron để sinh ra các câu hỏi đơn giản hơn, mỗi câu hỏi đề cập một khía cạnh khác nhau của câu hỏi ban đầu [Andreas và cộng sự, 2016; Talmor và Berant, 2018; Min và cộng sự, 2019]. Bộ sinh câu hỏi này có thể tạo các câu hỏi theo lô (cùng một lúc) hoặc tuần tự.

Nói rộng hơn, phân rã bài toán cũng liên quan tới vấn đề *tính kết hợp* (*compositionality*) trong NLP [Drozdov và cộng sự, 2022; Press và cộng sự, 2023]. Chẳng hạn, trong phân tích ngữ nghĩa (*semantic parsing*), ta ánh xạ các câu ngôn ngữ tự nhiên thành các biểu diễn ý nghĩa có cấu trúc bằng cách chia nhỏ chúng thành các thành phần cấu tạo và hiểu câu dựa trên ý nghĩa của các thành phần đó cùng các quy tắc dùng để kết hợp chúng. Trong những nghiên cứu ban đầu của lĩnh vực này, các câu có tính kết hợp cao được coi là những trường hợp kiểm tra dễ hơn để đánh giá hệ thống, vì việc phân rã những câu như vậy và kết hợp ý nghĩa các phần của chúng là tương đối đơn giản. Tuy nhiên, nhiệm vụ trở nên khó hơn nhiều khi cần khái quát hóa nhiều hơn để mô hình hóa tính kết hợp trên dữ liệu mới. Trong trường hợp này, ta muốn hệ thống có năng lực **khái quát hóa kết hợp** (*compositional generalization*) tốt hơn. Trong các nghiên cứu gần đây hơn về LLM, vấn đề này thường được bàn tới trong các nhiệm vụ lập luận kết hợp, như SCAN[9], vì nó được coi là một khía cạnh quan trọng để kiểm tra năng lực hiểu ngôn ngữ và lập luận của LLM. Điều này cũng đặt ra những nhiệm vụ mới để phát triển và kiểm nghiệm các phương pháp phân rã bài toán.

Chú giải của người dịch: Tính kết hợp và ví dụ SCAN

Tính kết hợp nghĩa là ý nghĩa của cả câu được ghép từ ý nghĩa của các phần theo những quy tắc nhất định. Trong SCAN, nếu hệ thống hiểu "jump" = JUMP (nhảy), "walk" = WALK (đi), "thrice" = lặp ba lần, "opposite left" = quay trái hai lần (LTURN LTURN, tức quay ngược lại), thì nó có thể tự ghép ra "LTURN LTURN JUMP WALK WALK WALK" cho một mệnh lệnh chưa từng thấy. *Khái quát hóa kết hợp* chính là khả năng xử lý đúng những tổ hợp mới như vậy — giống như khi đã biết nghĩa của "ba" và của "quả táo", ta hiểu ngay "ba quả táo" dù chưa gặp cụm này.

Trong LLM, một ứng dụng thú vị của phân rã bài toán là *sử dụng công cụ*. Trong một số trường hợp, cần tích hợp các công cụ bên ngoài vào LLM để truy cập dữ liệu chính xác không có sẵn trong lúc

[9]Các nhiệm vụ SCAN (phiên bản đơn giản hóa của các nhiệm vụ CommAI Navigation) được thiết kế để đánh giá khả năng khái quát hóa kết hợp của LLM [Lake và Baroni, 2018]. Chúng yêu cầu dịch các mệnh lệnh ngôn ngữ tự nhiên thành một dãy hành động. Ví dụ, mệnh lệnh "jump opposite left and walk thrice" (*nhảy về phía đối diện bên trái rồi đi bộ ba lần*) có thể được dịch thành dãy hành động "LTURN LTURN JUMP WALK WALK WALK".

huấn luyện hay tinh chỉnh. Ví dụ, LLM có thể tích hợp với các API để lấy dữ liệu thời gian thực như cập nhật thời tiết, giá chứng khoán hay tin tức, nhờ đó đưa ra các câu trả lời cập nhật cho truy vấn của người dùng. Khi dùng công cụ, dự đoán của LLM có thể chứa các dấu hiệu chỉ ra chỗ nào và cách nào để gọi API bên ngoài. Điều này đòi hỏi phân rã bài toán thành các bài toán con, một số do LLM xử lý và số khác do các công cụ bên ngoài xử lý. Thảo luận chi tiết hơn về chủ đề này sẽ được trình bày ở Mục 9.2.5.

9.2.3 Tự tinh chỉnh

Trong nhiều trường hợp, dự đoán của LLM có thể không chính xác hoặc có sai sót. Vì các LLM hiện nay có khả năng thực hiện những nhiệm vụ như chỉnh sửa và sửa lỗi, việc tìm hiểu các phương pháp cho phép các mô hình này *tự tinh chỉnh* (*self-refine*) đầu ra của chính chúng là rất có giá trị. Tự tinh chỉnh là một hiện tượng phổ biến trong quá trình nhận thức và hành vi hằng ngày của con người. Chẳng hạn, khi thiết kế một sản phẩm, nhà thiết kế có thể tạo một nguyên mẫu cơ bản trước, rồi tinh chỉnh thiết kế sau khi đánh giá và thử nghiệm để nâng cao trải nghiệm người dùng và tính năng. Quá trình tinh chỉnh này có thể lặp lại nhiều lần cho tới khi thiết kế đạt yêu cầu. Mô thức *dự đoán rồi tinh chỉnh* có lịch sử lâu dài trong NLP. Một ví dụ sớm là bộ gán nhãn của Brill [Brill, 1992], trong đó kết quả gán nhãn từ loại (*part-of-speech tagging, POS tagging*) ban đầu của một câu có thể được tinh chỉnh lặp đi lặp lại bằng một hệ thống dựa trên luật. Trong kỷ nguyên học sâu, rất nhiều công trình về các bài toán chuỗi-sang-chuỗi, như sửa lỗi ngữ pháp và viết lại văn bản, cũng có thể được xem là ví dụ của mô thức này.

Ta có thể prompt LLM để tự tinh chỉnh. Hãy xét một ví dụ đơn giản, trong đó đầu tiên ta dùng một LLM để dịch một văn bản từ tiếng Trung sang tiếng Anh, rồi dùng chính mô hình đó để tinh chỉnh bản dịch. Đây là cách ta có thể prompt mô hình để dịch (văn bản tiếng Trung và tiếng Anh được giữ nguyên vì đây là ví dụ dịch máy; nghĩa tiếng Việt được ghi ở chú giải phía dưới):

> Hãy dịch văn bản sau từ tiếng Trung sang tiếng Anh:
>
> Tiếng Trung:
> 一系列考古发现奠定红山文化在中华文明起源研究中的重要地位。最新公布的研究成果认为，大约从距今5800年开始，中华大地上各个区域相继出现较为明显的社会分化，进入文明起源的加速阶段。
>
> Tiếng Anh:

Sau đó, ta có thể prompt nó tinh chỉnh bản dịch:

> Bạn được cung cấp một văn bản tiếng Trung cùng với bản dịch tiếng Anh của nó. Hãy tinh chỉnh bản dịch để cải thiện độ chính xác và độ trôi chảy nếu cần.
>
> Tiếng Trung:
> 一系列考古发现奠定红山文化在中华文明起源研究中的重要地位。最新公布的研究成果认为，大约从距今5800年开始，中华大地上各个区域相继出现较为明显的社会分化，进入文明起源的加速阶段。
>
> Tiếng Anh:

A series of discoveries have cemented the significant role of the Hongshan culture in studies on the origins of Chinese civilization. The latest research findings suggest that, starting from around 5800 years ago, many regions across China began to experience noticeable social differentiations, entering an accelerated phase in the origin of civilization.

Bản tinh chỉnh:

A series of archaeological discoveries have cemented the significant role of the Hongshan culture in studies on the origins of Chinese civilization. The latest research findings suggest that, starting from around 5800 years ago, various regions across China began to experience noticeable social differentiations, entering an accelerated phase in the origin of civilization.

Ở đây những chỗ sửa đổi so với bản dịch đầu vào được tô màu xanh lá. Quá trình này có thể lặp lại: ở mỗi bước, ta dùng bản dịch của bước trước làm đầu vào và prompt LLM tạo ra một bản dịch có thể tốt hơn.

Chú giải của người dịch: Nghĩa của đoạn văn trong ví dụ

Đoạn văn tiếng Trung (và bản dịch tiếng Anh) có nghĩa: "Một loạt phát hiện khảo cổ đã khẳng định vị trí quan trọng của văn hóa Hồng Sơn trong nghiên cứu về nguồn gốc văn minh Trung Hoa. Các kết quả nghiên cứu mới công bố cho rằng, bắt đầu từ khoảng 5800 năm trước, nhiều khu vực trên đất Trung Hoa lần lượt xuất hiện sự phân hóa xã hội tương đối rõ rệt, bước vào giai đoạn tăng tốc của quá trình hình thành văn minh." Bản dịch đầu tiên bỏ sót chữ 考古 ("khảo cổ"); bản tinh chỉnh bổ sung *archaeological* (khảo cổ), và đổi *many* (nhiều) thành *various* (các, khác nhau) cho sát với 各个 ("từng/các") hơn.

Trong ví dụ trên, việc tinh chỉnh đầu ra ban đầu của mô hình được thực hiện bằng cách prompt LLM với một chỉ dẫn đơn giản như "Hãy tinh chỉnh nó!". Tuy nhiên, việc tinh chỉnh chỉ dựa vào khả năng làm theo chỉ dẫn của LLM, và không có hướng dẫn hay giám sát nào về việc cải thiện đầu ra của mô hình như thế nào và ở đâu. Một cách tiếp cận hiệu quả hơn là xét đến *phản hồi* về những khía cạnh cụ thể cần tinh chỉnh. Chẳng hạn, ta có thể prompt LLM bằng "Hãy sửa tất cả các lỗi ngữ pháp trong bản dịch", để mô hình tập trung hơn vào việc sửa lỗi ngữ pháp trong lúc tinh chỉnh.

Một khung tổng quát cho việc tự tinh chỉnh bằng LLM gồm ba bước [Madaan và cộng sự, 2024].

- **Dự đoán** (*Prediction*). Ta dùng một LLM để tạo ra đầu ra ban đầu của mô hình.
- **Thu thập phản hồi** (*Feedback Collection*). Ta thu được phản hồi về đầu ra của mô hình.
- **Tinh chỉnh** (*Refinement*). Ta dùng LLM để tinh chỉnh đầu ra của mô hình dựa trên phản hồi.

Hai bước cuối có thể được lặp lại nhiều lần, dẫn tới một quá trình tự tinh chỉnh lặp. Trong khung này, việc nhận được phản hồi chính xác và chi tiết là then chốt vì nó định hướng cho bước tinh chỉnh tiếp theo. Nói chung, phản hồi có thể thu được theo nhiều cách [Fernandes và cộng sự, 2023; Pan và cộng sự, 2024]. Ta có thể tự kiểm tra thủ công các dự đoán của mô hình để phát hiện vấn đề. Ta cũng có thể dùng một *mô hình phản hồi* để tự động sinh phản hồi. Chẳng hạn, một mô hình phần thưởng (*reward model*) được huấn luyện trên một ít dữ liệu có gán nhãn có thể giúp đánh giá đầu ra của mô hình và trả về một điểm số cho biết chất lượng của nó.

Vì trọng tâm của chương này là prompting LLM, ta xét việc dùng LLM để đưa ra phản hồi. Để minh họa, hãy xét một ví dụ sinh câu trả lời tốt cho các câu hỏi của người dùng:

- Đầu tiên, ta prompt một LLM trả lời câu hỏi đầu vào, như thường lệ.

> Hãy viết câu trả lời cho câu hỏi: "Hiện nay có những vấn đề môi trường lớn nào?"
>
> Câu trả lời:
> Những vấn đề môi trường lớn hiện nay gồm biến đổi khí hậu, ô nhiễm không khí và nước, phá rừng, suy giảm đa dạng sinh học và suy thoái đại dương.

- Sau đó, bằng prompt dưới đây, ta lại prompt chính LLM đó để nhận phản hồi về câu trả lời đã sinh ra.

> Bạn sẽ được cung cấp một câu hỏi và câu trả lời tương ứng. Hãy đánh giá độ chính xác và độ sâu của câu trả lời này, đồng thời chỉ ra mọi chỗ sai về sự kiện hoặc những chỗ mà việc bổ sung chi tiết có thể giúp hiểu rõ hơn.
>
> Câu hỏi:
> Hiện nay có những vấn đề môi trường lớn nào?
>
> Câu trả lời:
> Những vấn đề môi trường lớn hiện nay gồm biến đổi khí hậu, ô nhiễm không khí và nước, phá rừng, suy giảm đa dạng sinh học và suy thoái đại dương.
>
> Phản hồi:
> Câu trả lời được đưa ra là chính xác và nêu bật các vấn đề môi trường then chốt. Tuy nhiên, nó sẽ tốt hơn nếu có mô tả cụ thể và chi tiết hơn về những vấn đề này. Dưới đây là các gợi ý cải thiện:
>
> 1. Nêu các ví dụ cụ thể về các vấn đề môi trường.
> 2. Bổ sung thông tin về các chất gây ô nhiễm chính.
> 3. Chỉ rõ những khu vực bị ảnh hưởng nặng nhất.

- Với phản hồi mà LLM tạo ra, ta prompt nó tinh chỉnh câu trả lời.

> Bạn sẽ được cung cấp một câu hỏi, câu trả lời tương ứng và một số phản hồi về câu trả lời này.
>
> Câu hỏi:
> Hiện nay có những vấn đề môi trường lớn nào?
>
> Câu trả lời:
> Những vấn đề môi trường lớn hiện nay gồm biến đổi khí hậu, ô nhiễm không khí và nước, phá rừng, suy giảm đa dạng sinh học và suy thoái đại dương.
>
> Phản hồi:
> Câu trả lời được đưa ra là chính xác và nêu bật các vấn đề môi trường then chốt. Tuy nhiên, nó sẽ tốt hơn nếu có mô tả cụ thể và chi tiết hơn về những vấn đề này. Dưới đây là các gợi ý cải thiện:
>
> 1. Nêu các ví dụ cụ thể về các vấn đề môi trường.
> 2. Bổ sung thông tin về các chất gây ô nhiễm chính.
> 3. Chỉ rõ những khu vực bị ảnh hưởng nặng nhất.
>
> Bây giờ hãy tinh chỉnh câu trả lời dựa trên phản hồi đã cho.
>
> Câu trả lời đã tinh chỉnh:
> Các vấn đề môi trường then chốt gồm biến đổi khí hậu, thể hiện qua nhiệt độ toàn cầu tăng và các kiểu thời tiết khắc nghiệt; ô nhiễm không khí và nước nghiêm trọng ở các khu đô thị và công nghiệp; nạn phá rừng tràn lan ở những vùng như Amazon, gây suy giảm đa dạng sinh học; và suy thoái đại dương, nổi bật là hiện tượng san hô bị tẩy trắng và đánh bắt cá quá mức trên diện rộng.

Lý tưởng nhất, nếu dùng một LLM mạnh, ta muốn nó thực hiện cả ba bước mà không cần huấn luyện thêm. Mặt khác, nếu có đủ dữ liệu gán nhãn cho nhiệm vụ quan tâm, ta có thể nâng cao hiệu năng của LLM bằng học có giám sát. Ví dụ, ta có thể tinh chỉnh LLM để nó thích nghi tốt hơn với các nhiệm vụ tinh chỉnh đầu ra, hoặc dùng các mô hình chuyên cho nhiệm vụ, vốn không nhất thiết phải dựa trên

LLM [Welleck và cộng sự, 2023; Schick và cộng sự, 2023]. Theo nghĩa rộng hơn, việc cải thiện LLM cho các nhiệm vụ tự tinh chỉnh có thể được xem là một vấn đề căn chỉnh (*alignment*). Chẳng hạn, người ta đã thấy rằng một số năng lực tự sửa lỗi có thể được kích hoạt thông qua RLHF [Ganguli và cộng sự, 2023]. Tuy nhiên, việc thảo luận những vấn đề này nằm ngoài phạm vi chương này. Thảo luận thêm có thể tìm thấy ở Chương 10.

Chú giải của người dịch: Tự tinh chỉnh giống việc tự chấm và sửa bài

Ba bước *dự đoán – thu thập phản hồi – tinh chỉnh* giống cách ta làm bài tập làm văn: viết bản nháp, nhờ thầy cô (hoặc tự mình) đọc lại và ghi nhận xét "thiếu dẫn chứng, cần ví dụ cụ thể", rồi viết lại theo nhận xét. Điểm đặc biệt ở đây là cả ba vai (người viết, người nhận xét, người sửa) đều do *cùng một* LLM đảm nhận, chỉ khác nhau ở prompt. Lời nhận xét cụ thể ("thêm ví dụ, nêu chất gây ô nhiễm, chỉ rõ khu vực") hữu ích hơn nhiều so với một yêu cầu chung chung "hãy viết hay hơn". (RLHF — học tăng cường từ phản hồi của con người — là một kỹ thuật căn chỉnh sẽ được trình bày ở Chương 10.)

Trong bối cảnh LLM, tự tinh chỉnh liên quan tới một số khái niệm mô phỏng năng lực lập luận nâng cao, như khả năng *tự phản tư* (*self-reflection*). Một quan điểm cho rằng việc trang bị khả năng tự phản tư cho LLM giúp chúng sinh ra các dự đoán chính xác hơn và thể hiện khả năng tự sửa lỗi. Năng lực phản tư này có thể được kích hoạt theo nhiều cách, chẳng hạn bằng cách prompt LLM tham gia vào quá trình lập luận có cân nhắc, từng bước một, hoặc bằng cách cung cấp các ví dụ trong ngữ cảnh để mô hình học hỏi và điều chỉnh. Để minh họa, ta xem xét phương pháp **cân nhắc rồi sinh** (*deliberate-then-generate*) (**DTG**) do Li và cộng sự [2023a] đề xuất, phương pháp này prompt LLM một cách tường minh để cân nhắc. Trong khung DTG, cho một đầu ra ban đầu của mô hình, có thể có sai sót, LLM được prompt để xác định các loại lỗi cụ thể trước khi đưa ra một bản sửa tốt hơn. Dưới đây là một khuôn mẫu prompt DTG cho nhiệm vụ dịch từ tiếng Trung sang tiếng Anh.

Cho câu tiếng Trung: {∗source∗}
Bản dịch tiếng Anh là: {∗target∗}
Hãy phát hiện loại lỗi trước, rồi tinh chỉnh bản dịch.
Loại lỗi: _____

_____

Ta muốn mô hình dự đoán loại lỗi trước (ô màu đỏ), rồi tạo ra một bản dịch đã tinh chỉnh (ô màu xanh dương). Quá trình cân nhắc này được dẫn dắt bởi chỉ dẫn "Hãy phát hiện loại lỗi trước, rồi tinh chỉnh bản dịch". Nó khuyến khích LLM ban đầu phân tích thấu đáo rồi mới đưa ra kết quả tốt hơn. Vì việc dự đoán loại lỗi và việc tinh chỉnh được thực hiện trong một lần chạy LLM, phương pháp này gộp cả hai bước phản hồi và tinh chỉnh vào một quá trình.

Trong các prompt trên, ta giả định rằng LLM được dùng có khả năng xem xét bản dịch đầu vào và xác định đúng các loại lỗi của nó. Tuy nhiên, điều này đặt ra những khó khăn mới vì mô hình có thể không giỏi tìm lỗi trong bản dịch. Điều đó lại dẫn tới công sức tinh chỉnh hoặc thiết kế prompt bổ sung. Vì vậy, một phương pháp đơn giản hơn là giảm gánh nặng xác định lỗi và chỉ dùng LLM để cân nhắc. Để làm điều này, ta có thể thay bản dịch đầu vào bằng một bản dịch ngẫu nhiên và gán một loại lỗi mặc định. Một ví dụ về prompt như vậy được trình bày dưới đây (câu tiếng Trung và bản dịch tiếng Anh được giữ nguyên).

Cho câu tiếng Trung:
一系列考古发现奠定红山文化在中华文明起源研究中的重要地位。
Bản dịch tiếng Anh là:
A variety of innovative techniques have redefined the importance of modern art in contemporary cultural studies.
Hãy phát hiện loại lỗi trước, rồi tinh chỉnh bản dịch.
Loại lỗi: Dịch sai

Chú giải của người dịch: Vì sao lại cố ý đưa vào một bản dịch sai?

Câu tiếng Trung nghĩa là "Một loạt phát hiện khảo cổ đã khẳng định vị trí quan trọng của văn hóa Hồng Sơn trong nghiên cứu nguồn gốc văn minh Trung Hoa", còn câu tiếng Anh tô xanh lại nghĩa là "Nhiều kỹ thuật đổi mới đã định nghĩa lại tầm quan trọng của nghệ thuật hiện đại trong nghiên cứu văn hóa đương đại" — hoàn toàn không liên quan. Vì ta *biết chắc* bản dịch này sai, ta có thể điền sẵn loại lỗi là "Dịch sai" mà không cần mô hình tự tìm lỗi. Mô hình chỉ cần làm việc còn lại: nhìn câu gốc, nhận ra một ví dụ "sai", và viết một bản dịch đúng vào ô xanh dương.

Trong ví dụ này, bản dịch đầu vào không phải do LLM sinh ra mà được lấy mẫu ngẫu nhiên từ tập dữ liệu. Vì vậy, nó đơn giản chỉ là một bản dịch sai của câu nguồn, và ta có thể đặt loại lỗi tương ứng. LLM sau đó sinh ra một bản dịch mới bằng cách lấy cả câu nguồn lẫn bản dịch sai làm đầu vào. Thiết kế prompt này cũng có thể được xem là kích hoạt năng lực học của LLM thông qua "bằng chứng phủ định" (*negative evidence*) [Marcus, 1993], nhờ đó cho phép chúng phản tư và tạo ra kết quả tốt hơn thông qua phân tích đối chiếu. Tuy nhiên, phương pháp này không dựa vào bất kỳ phản hồi nào và có thể nâng cao hiệu năng của một lần dự đoán duy nhất của LLM chỉ bằng prompting đơn giản.

Lưu ý rằng mặc dù DTG không lặp, việc học và tinh chỉnh lặp lại được dùng phổ biến trong NLP. Một ưu điểm của các cách tiếp cận lặp là chúng mô phỏng cách con người học và giải quyết vấn đề, trong đó phản hồi và điều chỉnh liên tục dẫn tới kết quả ngày càng tốt hơn. Các phương pháp lặp có thể áp dụng cho nhiều bài toán prompting LLM. Ví dụ, trong phân rã bài toán, ta có thể đưa các bài toán con mới và lời giải của chúng vào ngữ cảnh ở mỗi bước, nhờ đó LLM có thể dần dần tiến tới lời giải của bài toán ban đầu. Mặt khác, các phương pháp lặp nảy sinh một số vấn đề không có ở các phương pháp không lặp. Chẳng hạn, lỗi ở các bước trước có thể ảnh hưởng xấu tới việc giải ở các bước sau, và việc xác định khi nào dừng lặp thường đòi hỏi thêm công sức kỹ thuật.

9.2.4 Kết hợp nhiều mô hình

Việc kết hợp mô hình (*model ensembling*) cho sinh văn bản đã được bàn luận rộng rãi trong các tài liệu NLP. Ý tưởng là kết hợp các dự đoán của hai hay nhiều mô hình để sinh ra một đầu ra cuối cùng tốt hơn. Kỹ thuật này có thể áp dụng trực tiếp cho LLM. Ví dụ, ta có thể tập hợp một nhóm LLM và chạy từng mô hình trên cùng một đầu vào. Đầu ra cuối cùng thu được bằng cách kết hợp các dự đoán của những mô hình này.

Với prompting LLM, cũng có thể cải thiện hiệu năng bằng cách kết hợp các dự đoán dựa trên những prompt khác nhau. Giả sử ta có một LLM và một tập các prompt cùng giải quyết một nhiệm vụ. Ta có thể chạy LLM này với từng prompt rồi kết hợp các dự đoán. Ví dụ, dưới đây là ba khuôn mẫu prompt khác nhau cho nhiệm vụ đơn giản hóa văn bản (*text simplification*).

Hãy làm cho văn bản này đơn giản hơn.
{∗text∗}

___

Hãy rút gọn và đơn giản hóa văn bản này.
{∗text∗}

___

Hãy viết lại cho dễ đọc.
{∗text∗}

___

Mỗi prompt này sẽ dẫn tới một dự đoán khác nhau, và ta có thể kết hợp ba dự đoán này để tạo ra đầu ra cuối cùng.

Một cách hình thức, gọi $\{\mathbf{x}_1, \cdots, \mathbf{x}_K\}$ là K prompt dùng để thực hiện cùng một nhiệm vụ. Cho một LLM $\Pr(\cdot|\cdot)$, ta có thể tìm dự đoán có khả năng cao nhất cho mỗi $\mathbf{x}_i$ bằng $\hat{\mathbf{y}}_i = \arg\max_{\mathbf{y}_i} \Pr(\mathbf{y}_i|\mathbf{x}_i)$. Các dự đoán này có thể được kết hợp để tạo thành một dự đoán "mới":

$$\hat{\mathbf{y}} \;=\; \text{Combine}(\hat{\mathbf{y}}_1, \cdots, \hat{\mathbf{y}}_K). \tag{9.7}$$

Ở đây $\text{Combine}(\cdot)$ là mô hình kết hợp, có thể được thiết kế theo nhiều cách khác nhau. Chẳng hạn, ta có thể chọn dự đoán tốt nhất bằng bỏ phiếu, hoặc bằng cách tìm dự đoán trùng lặp nhiều nhất với các dự đoán khác. Một cách tiếp cận khác là kết hợp điểm xác suất ở mức token trong quá trình giải mã. Gọi $\hat{y}_j$ là token được dự đoán ở bước thứ j khi kết hợp mô hình. Token dự đoán $\hat{y}_j$ được cho bởi

$$\hat{y}_j \;=\; \arg\max_{y_j} \sum_{k=1}^{K} \log \Pr(y_j|\mathbf{x}_k, \hat{y}_1, \cdots, \hat{y}_{j-1}). \tag{9.8}$$

Bạn đọc quan tâm có thể tham khảo Chương 5 để biết thêm chi tiết về các phương pháp này.

Chú giải của người dịch: Hai cách kết hợp trong Phương trình (9.7) và (9.8)

Phương trình (9.7) kết hợp ở mức *toàn bộ câu trả lời*: mỗi prompt cho ra một bản đơn giản hóa hoàn chỉnh, rồi ta chọn bản "được nhiều phiếu nhất" hoặc giống các bản khác nhất. Phương trình (9.8) kết hợp ở mức *từng token*: khi sinh token thứ j, ta hỏi cả K prompt xem mỗi token ứng viên có xác suất bao nhiêu, cộng các log xác suất lại và chọn token có tổng lớn nhất. Ví dụ với $K = 2$: token A có xác suất 0.6 và 0.5, token B có 0.9 và 0.1. Tổng log của A là $\log 0.6 + \log 0.5 = \log 0.30$, của B là $\log 0.9 + \log 0.1 = \log 0.09$, nên chọn A — token mà *cả hai* prompt đều thấy khá hợp lý, thay vì token chỉ một prompt rất thích. (Cộng các log tương đương với nhân các xác suất.)

Khi kết hợp trong prompting LLM, nói chung nên dùng các prompt đa dạng để sự kết hợp có thể bao quát một phạm vi rộng hơn các câu trả lời tiềm năng. Cách làm này phổ biến trong học kết hợp (*ensemble learning*), vì sự đa dạng giúp "trung bình hóa" những thiên lệch và sai sót có thể chỉ riêng một mô hình hay một cấu hình mắc phải. Từ góc nhìn Bayes, ta có thể coi prompt $\mathbf{x}$ là một biến ẩn (*latent variable*), khi biết bài toán quan tâm p. Điều này cho phép viết phân phối dự đoán của $\mathbf{y}$ khi biết p dưới dạng phân phối $\Pr(\mathbf{y}|\mathbf{x})$ được lấy biên (*marginalized*) trên mọi prompt khả dĩ

$$\Pr(\mathbf{y}|p) \;=\; \int \Pr(\mathbf{y}|\mathbf{x}) \Pr(\mathbf{x}|p)\, d\mathbf{x}. \tag{9.9}$$

Tích phân này tính xác suất toàn phần của $\mathbf{y}$ bằng cách xét mọi giá trị khả dĩ của $\mathbf{x}$, có trọng số là xác suất của chúng khi biết p. Ở đây $\Pr(\mathbf{y}|\mathbf{x})$ do LLM cho, còn $\Pr(\mathbf{x}|p)$ là phân phối tiên nghiệm (*prior distribution*) của các prompt cho bài toán. Cách phát biểu này có lợi vì tích phân tính đến sự bất định trong việc chọn $\mathbf{x}$. Kết quả là phân phối dự đoán cuối cùng $\Pr(\mathbf{y}|p)$ trở nên vững hơn trước những biến động và thiên lệch của từng prompt riêng lẻ. Tuy nhiên, tính trực tiếp tích phân này có thể bất khả thi về mặt tính toán do không gian của $\mathbf{x}$ có thể vô hạn. Một cách giải quyết vấn đề này là dùng các phương pháp như lấy mẫu Monte Carlo (*Monte Carlo sampling*), xấp xỉ tích phân bằng một số hữu hạn, vừa phải các prompt.

Chú giải của người dịch: Ý nghĩa của Phương trình (9.9)

Đây là "công thức xác suất toàn phần" cho trường hợp có vô số khả năng. Hãy hình dung phiên bản rời rạc: giả sử chỉ có 3 prompt có thể dùng, với xác suất được chọn $\Pr(\mathbf{x}_k|p)$ là 0.5, 0.3, 0.2, và xác suất mỗi prompt dẫn tới đáp án đúng $\mathbf{y}$ là 0.9, 0.6, 0.5. Khi đó $\Pr(\mathbf{y}|p) = 0.5 \cdot 0.9 + 0.3 \cdot 0.6 + 0.2 \cdot 0.5 = 0.73$ — một "trung bình có trọng số" qua các prompt. Dấu tích phân $\int$ chỉ là phiên bản "tổng" khi có vô số prompt. Vì không thể cộng vô số số hạng, *Monte Carlo* chọn ngẫu nhiên một số prompt (theo $\Pr(\mathbf{x}|p)$) rồi lấy trung bình của $\Pr(\mathbf{y}|\mathbf{x})$ trên các prompt đó để ước lượng. Phân phối *tiên nghiệm* là niềm tin "trước khi thấy dữ liệu" về việc prompt nào phù hợp.

Mặc dù cách xử lý theo Bayes được định nghĩa chặt chẽ về mặt toán học, trong NLP người ta thường giả định một tiên nghiệm không mang thông tin hoặc tiên nghiệm đều, và thay vào đó tập trung xây dựng một tập prompt đa dạng. Do đó, đầu ra có thể được tính bằng một mô hình kết hợp đơn giản, như mô tả ở Phương trình (9.7). Vấn đề tạo ra các prompt đa dạng, chất lượng cao đã được nghiên cứu trong CoT và các lĩnh vực học trong ngữ cảnh khác. Phần lớn nghiên cứu tập trung vào việc đưa nhiều ví dụ minh họa khác nhau vào các prompt khác nhau. Dưới đây ta liệt kê một số phương pháp như vậy.

- Với một bài toán, ta tự tạo thủ công một số ví dụ minh họa và dùng các ví dụ khác nhau cho các prompt khác nhau.
- Với một bài toán, ta dùng LLM để tự động sinh ví dụ minh họa và prompt.
- Với một prompt, ta tạo các prompt khác nhau bằng cách thay đổi thứ tự các ví dụ minh họa trong prompt.
- Với một prompt, ta dùng LLM để sinh ra một số prompt tương tự.
- Với một prompt, ta biến đổi nó sang các dạng khác, chẳng hạn dịch nó sang các ngôn ngữ khác.

Tất nhiên, trong thực tế, ta có thể kết hợp các phương pháp này để đạt được sự đa dạng lớn hơn. Một giả định ngầm ở đây là các prompt đa dạng có thể dẫn tới các đầu ra đa dạng của mô hình. Điều này đặc biệt đúng khi xử lý những bài toán tương đối mới lạ và phức tạp. Với những LLM mạnh và vững hơn, mức độ biến thiên của đầu ra với các prompt tương tự có thể không lớn. Trong trường hợp này, lợi ích của việc dùng nhiều prompt có thể khiêm tốn.

Ngoài việc cung cấp các prompt đa dạng cho LLM, một cách tiếp cận khác là tận dụng tính biến thiên vốn có trong đầu ra của LLM. Một cách đơn giản là lấy mẫu nhiều đầu ra từ không gian giả thuyết. Điều này dễ làm với LLM vì các thuật toán tìm kiếm của chúng vốn phù hợp với việc tạo ra nhiều kết quả. Chẳng hạn, nếu dùng tìm kiếm chùm (beam search), ta có thể dễ dàng thu thập tất cả các giả thuyết đã hoàn chỉnh trong chùm để tạo ra nhiều ứng viên đầu ra. Ngoài ra, ta có thể sửa đổi thuật toán tìm kiếm để cài đặt những phương pháp lấy mẫu mạnh hơn và khám phá phạm vi rộng hơn các giả thuyết chất lượng cao.

Câu hỏi tiếp theo là làm thế nào để kết hợp các dự đoán đa dạng của mô hình. Điều này đưa ta trở

lại vấn đề kết hợp mô hình như mô tả ở Phương trình (9.7). Ở đây ta xét phương pháp **tự nhất quán** (*self-consistency*), phương pháp này không nhất thiết đưa ra dự đoán có xác suất cao nhất, mà đưa ra dự đoán nhất quán nhất với các dự đoán khác [Wang và cộng sự, 2022d; 2023c]. Đầu tiên, LLM được prompt bằng CoT như thường lệ và sinh ra nhiều đường lập luận bằng cách lấy mẫu. Sau đó, tần suất của mỗi đáp án được đếm trên các đường lập luận này. Đầu ra cuối cùng là đáp án xuất hiện nhiều nhất. Dưới đây là ba dự đoán khác nhau được sinh ra bằng cách prompt một LLM với CoT.

Dự đoán 1 (Đúng):

> Q: Ba người bạn chơi một trò chơi tung một đồng xu cân đối. Mỗi người tung đồng xu một lần. Xác suất để có đúng một người tung được mặt sấp là bao nhiêu?
>
> A: Hãy suy nghĩ từng bước một.
> Mỗi lần tung đồng xu là một biến cố độc lập với 50% khả năng ra mặt sấp (H) và 50% ra mặt ngửa (T). Xét các kết quả có thể có của ba lần tung: HHH, HHT, HTH, HTT, THH, THT, TTH, TTT. Trong số này, chỉ HHH, HHT, HTH và THH là liên quan. Ta chỉ quan tâm đến các trường hợp có đúng một mặt sấp: HTT, THT và TTH. Có 3 kết quả như vậy trong tổng số 8, cho xác suất là $3/8$.

Dự đoán 2 (Đúng):

> Q: Ba người bạn chơi một trò chơi tung một đồng xu cân đối. Mỗi người tung đồng xu một lần. Xác suất để có đúng một người tung được mặt sấp là bao nhiêu?
>
> A: Hãy suy nghĩ từng bước một.
> Dùng công thức xác suất nhị thức, trong đó p (thành công, tức mặt sấp) là 0.5, và mỗi lần thử trong 3 lần thử (lần tung) là độc lập, ta tìm trường hợp có đúng một thành công (mặt sấp). Công thức cho đúng k thành công trong n lần thử được cho bởi công thức hệ số nhị thức $\binom{n}{k}p^k(1-p)^{n-k}$. Thay các giá trị vào, ta được $\binom{3}{1}(0.5)^1(0.5)^{3-1} = 3 \cdot 0.5 \cdot 0.25 = 0.375$.

Dự đoán 3 (Sai):

> Q: Ba người bạn chơi một trò chơi tung một đồng xu cân đối. Mỗi người tung đồng xu một lần. Xác suất để có đúng một người tung được mặt sấp là bao nhiêu?
>
> A: Hãy suy nghĩ từng bước một.
> Vì mỗi lần tung đồng xu có 50% khả năng ra mặt sấp và 50% khả năng ra mặt ngửa, và ta đang tìm trường hợp chỉ một trong ba lần tung ra mặt sấp, thoạt đầu ta có thể nghĩ đây đơn giản là việc chọn lần tung nào ra mặt sấp. Có ba lần tung, nên người ta có thể giả định sai rằng khả năng chọn được một kết quả cụ thể như vậy là 1 trên 3. Do đó, họ có thể kết luận rằng xác suất có đúng một mặt sấp là $1/3 = 33.3\%$.

Dự đoán 1 và 2 xác định đúng ba trường hợp có đúng một mặt sấp, cả hai đều thu được xác suất 37.5%. Lập luận trong Dự đoán 3 không tính đến tổng số kết quả có thể có khi tung đồng xu ba lần, nên cho đáp án sai là 33.3%. Vì vậy, ta chọn 37.5% làm đáp án cuối cùng vì đó là đáp án được đồng thuận.

Chú giải của người dịch: Kiểm tra lại bài toán đồng xu

Ba lần tung có $2^3 = 8$ kết quả đồng khả năng. Kết quả có đúng một mặt sấp (H) là HTT, THT, TTH: 3 kết quả, nên xác suất là $3/8 = 0.375 = 37.5\%$. Cách 2 dùng công thức nhị thức quen thuộc ở lớp 11 (*công thức Bernoulli*): $C_3^1 \cdot 0.5^1 \cdot 0.5^2 = 3 \cdot 0.125 = 0.375$, với $\binom{n}{k} = C_n^k$. Dự đoán 3 sai vì nhầm "chọn 1 trong 3 lần tung" thành xác suất $1/3$, quên rằng mỗi lần tung còn có thể ra sấp hoặc ngửa. (Lưu ý: Dự đoán 1 có một câu lập luận thừa, hơi lủng củng — "chỉ HHH, HHT, HTH và THH là liên quan" — nhưng vẫn đi đến đáp án đúng; LLM thật cũng thường mắc những lỗi nhỏ như vậy.) Tự nhất quán giống như hỏi ý kiến nhiều người rồi

chọn đáp án được nhiều người đưa ra nhất: 2 trong 3 lời giải cho 0.375, nên chọn 0.375.

Tự nhất quán cung cấp một tiêu chí để xác định dự đoán tốt nhất trong một tập ứng viên. Vì prompt và mô hình được giữ cố định trong phương pháp này, nó không hẳn là một phương pháp kết hợp prompt. Thay vào đó, nó có thể được xem là một trường hợp của các phương pháp kết hợp đầu ra, còn gọi là các phương pháp *lựa chọn giả thuyết* (*hypothesis selection*), vốn đã được khám phá từ lâu trong NLP, đặc biệt cho các bài toán sinh văn bản [Xiao và cộng sự, 2013]. Trong những phương pháp này, nhiều đầu ra được sinh ra bằng cách thay đổi kiến trúc hoặc tham số của mô hình. Mỗi đầu ra sau đó được gán một điểm số theo một tiêu chí nào đó, và các đầu ra được xếp hạng lại dựa trên các điểm số này. Có nhiều cách định nghĩa hàm tính điểm, chẳng hạn đo mức độ đồng thuận giữa một đầu ra với các đầu ra khác, hay dùng một mô hình mạnh hơn để chấm lại điểm cho từng đầu ra[10]. Hình 9.2 so sánh các phương pháp kết hợp khác nhau cho LLM.

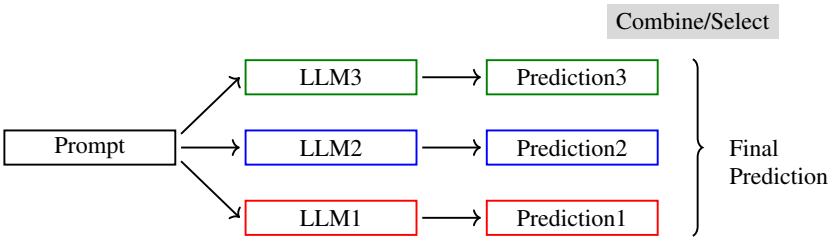
Chú giải của người dịch: Đọc Hình 9.2

Nhãn trong hình: *Prompt* = prompt, *Prediction* = dự đoán, *Combine/Select* = kết hợp/lựa chọn, *Final Prediction* = dự đoán cuối cùng, *Sample* = lấy mẫu; (a) *Model Ensembling* = kết hợp mô hình, (b) *Prompt Ensembling* = kết hợp prompt, (c) *Output Ensembling* = kết hợp đầu ra. Ba sơ đồ khác nhau ở chỗ "cái gì được nhân lên": nhiều mô hình (a), nhiều prompt (b), hay nhiều lần lấy mẫu từ cùng một mô hình với cùng một prompt (c) — tự nhất quán thuộc loại (c).

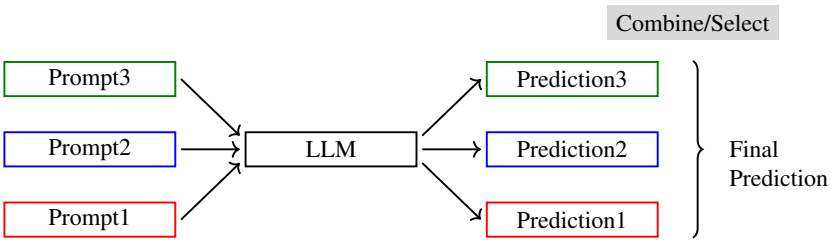
Bây giờ, hãy điểm lại ngắn gọn các phương pháp đã bàn trong mục này, như phân rã bài toán và tự tinh chỉnh. Rõ ràng các phương pháp này cải thiện việc ra quyết định bằng cách đưa thêm nhiều "lựa chọn" vào quá trình lập luận. Ở một mức độ nào đó, tất cả chúng đều liên quan tới việc đánh giá và đưa ra phản hồi về kết quả của LLM. Chẳng hạn, trong tự tinh chỉnh, ta cần đưa ra gợi ý để cải thiện dự đoán của LLM, còn trong kết hợp đầu ra, ta chọn đầu ra tối ưu từ một tập ứng viên. Theo nghĩa này, các phương pháp này thuộc nhóm rộng hơn gồm các cách tiếp cận *dự đoán rồi kiểm chứng*, trong đó dự đoán được đưa ra trước, sau đó được kiểm chứng và tinh chỉnh. Vấn đề cốt lõi ở đây là kiểm chứng và đánh giá kết quả suy luận hoặc các bước trung gian. Vấn đề này phần nào liên quan tới bài toán huấn luyện mô hình phần thưởng trong RLHF, mặc dù RLHF giải quyết một khía cạnh khác. Trên thực tế, việc phát triển các *bộ kiểm chứng* (*verifier*) đã được khám phá và triển khai trong lập luận bằng LLM. Thay vì phát triển các thuật toán dựa trên quy tắc kinh nghiệm chạy lúc suy luận, phần lớn công trình tập trung vào việc học các bộ kiểm chứng theo cách có giám sát. Một phương pháp đơn giản là huấn luyện bộ kiểm chứng như các bộ phân loại nhị phân, chẳng hạn phân loại một câu trả lời là đúng hay sai, mặc dù các bộ kiểm chứng này thường được dùng như mô hình chấm điểm. Cho một đường lập luận của một bài toán, bộ kiểm chứng có thể được dùng để chấm điểm hoặc toàn bộ đường đi (gọi là các cách tiếp cận *dựa trên kết quả*, *outcome-based*) [Cobbe và cộng sự, 2021], hoặc từng bước lập luận riêng lẻ (gọi là các cách tiếp cận *dựa trên quá trình*, *process-based*) [Uesato và cộng sự, 2022; Lightman và cộng sự, 2024].

[10] Một cách diễn giải tự nhất quán là xem nó như một quá trình tìm kiếm với *rủi ro Bayes cực tiểu* (*minimum Bayes risk*). Nó tìm đầu ra tốt nhất bằng cách cực tiểu hóa rủi ro Bayes. Cụ thể hơn, một hàm rủi ro $R(\mathbf{y}, \mathbf{y}_r)$ được định nghĩa trên mỗi cặp đầu ra (ký hiệu $(\mathbf{y}, \mathbf{y}_r)$), biểu thị chi phí khi thay $\mathbf{y}$ bằng $\mathbf{y}_r$. Cho một tập đầu ra $\mathcal{Y}$, rủi ro của một đầu ra $\mathbf{y} \in \mathcal{Y}$ được cho bởi

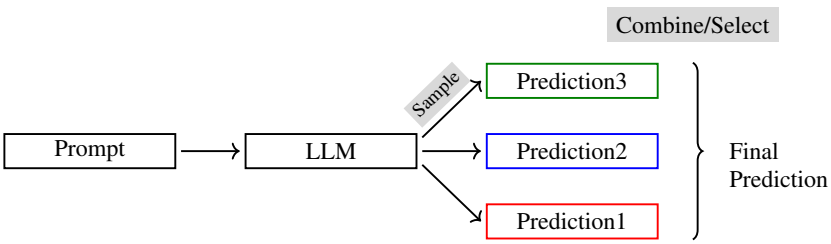
$$\begin{aligned} \text{Risk}(\mathbf{y}) &= \mathbb{E}_{\mathbf{y}_r \sim \Pr(\mathbf{y}_r|\mathbf{x})} R(\mathbf{y}, \mathbf{y}_r) \\ &= \sum_{\mathbf{y}_r \in \mathcal{Y}} R(\mathbf{y}, \mathbf{y}_r) \cdot \Pr(\mathbf{y}_r|\mathbf{x}). \end{aligned} \tag{9.10}$$



(a) Model Ensembling



(b) Prompt Ensembling



(c) Output Ensembling

Hình 9.2: Minh họa các phương pháp kết hợp cho LLM. Trong kết hợp mô hình tiêu chuẩn (a), nhiều LLM khác nhau về kiến trúc hoặc tham số được sử dụng. Mỗi LLM nhận cùng một prompt và tạo ra một dự đoán. Các dự đoán này được kết hợp để sinh ra dự đoán cuối cùng. Trong kết hợp prompt (b), ta có một LLM và nhiều prompt. LLM tạo ra một dự đoán cho mỗi prompt, và các dự đoán này được kết hợp như thường lệ. Trong kết hợp đầu ra (c), LLM lấy mẫu nhiều dự đoán trên không gian dự đoán với một prompt cho trước. Nó có thể được xem là một phương pháp tăng cường hiệu năng của chính LLM. Lưu ý rằng các phương pháp kết hợp này có thể được phối hợp với nhau để tăng tính đa dạng của dự đoán. Ví dụ, ta có thể dùng cả kết hợp prompt lẫn kết hợp đầu ra để thu được các dự đoán đa dạng hơn.

9.2.5 RAG và sử dụng công cụ

RAG thường được dùng khi các LLM tiêu chuẩn thiếu độ chính xác về sự kiện hoặc không truy cập được tri thức cập nhật. Bằng cách lấy thông tin từ các cơ sở dữ liệu và tài liệu bên ngoài, RAG có thể cải thiện đáng kể mức độ phù hợp với ngữ cảnh và tính có căn cứ về sự kiện của câu trả lời. Cách tiếp cận này đặc biệt hữu ích trong các tình huống đòi hỏi độ chính xác cao về sự kiện và thông tin cập nhật, như hỏi đáp phức tạp.

Mặc dù RAG đã được giới thiệu ở các chương trước, ở đây ta tóm tắt ngắn gọn các bước cốt lõi của nó cho đầy đủ.

- Ta chuẩn bị một tập văn bản đóng vai trò nguồn tri thức bên ngoài.
- Ta truy xuất các văn bản liên quan cho một truy vấn cho trước.
- Ta đưa cả các văn bản truy xuất được lẫn truy vấn vào LLM, rồi prompt nó tạo ra dự đoán cuối cùng.

Bước 1 và 2 có thể được thực hiện bằng một hệ thống truy xuất thông tin bên ngoài. Chẳng hạn, ta có thể lưu tập văn bản trong một cơ sở dữ liệu vectơ (*vector database*) rồi truy xuất các văn bản liên quan bằng các kỹ thuật tìm kiếm dựa trên vectơ. Vì truy xuất thông tin không phải trọng tâm của chương này, ta sẽ giả định rằng các hệ thống như vậy có sẵn và dùng trực tiếp chúng.

Ở đây ta trình bày cách prompt LLM để tận dụng các văn bản truy xuất được. Để minh họa, hãy xét một ví dụ dùng LLM để trả lời câu hỏi sau.

Thế vận hội 2028 sẽ được tổ chức ở đâu?

Ta có thể đơn giản nhập câu hỏi này vào một công cụ tìm kiếm trực tuyến. Nó sẽ trả về các đoạn văn bản liên quan tìm thấy trên internet, chẳng hạn,

> (Wikipedia)
> *Thế vận hội Mùa hè 2028, tên chính thức là Đại hội Thể thao Olympic lần thứ XXXIV và thường được gọi là Los Angeles 2028 hay LA28, là một sự kiện thể thao đa môn quốc tế sắp diễn ra, dự kiến tổ chức từ ngày 14 đến 30 tháng 7 năm 2028 tại Hoa Kỳ. …*
>
> (The Sporting News)
> *Năm 2028, Los Angeles sẽ trở thành thành phố thứ ba, sau London và Paris, đăng cai ba kỳ Thế vận hội, sau khi đã tổ chức Thế vận hội Mùa hè vào các năm 1932 và 1984. Đây cũng sẽ là lần đầu tiên Hoa Kỳ đăng cai một kỳ Thế vận hội kể từ Thế vận hội Mùa đông 2002 ở Salt Lake City. …*
>
> …

Ta có thể dùng các văn bản truy xuất được này làm ngữ cảnh bổ sung, và prompt LLM sinh câu trả lời dựa trên các văn bản đó. Dưới đây là một ví dụ prompt RAG.

> Nhiệm vụ của bạn là trả lời câu hỏi sau. Để giúp bạn, các văn bản liên quan được cung cấp kèm theo. Hãy dựa vào các văn bản này để trả lời.
>
> Câu hỏi:
> Thế vận hội 2028 sẽ được tổ chức ở đâu?
>
> Văn bản liên quan 1:
> Thế vận hội Mùa hè 2028, tên chính thức là Đại hội Thể thao Olympic lần thứ XXXIV và thường được gọi là

Los Angeles 2028 hay LA28 …
Văn bản liên quan 2:
Năm 2028, Los Angeles sẽ trở thành thành phố thứ ba, sau London và Paris, đăng cai ba kỳ Thế vận hội sau khi …
…
Thế vận hội 2028 sẽ được tổ chức tại Los Angeles.

Prompt này giả định rằng các văn bản được cung cấp có liên quan tới câu hỏi và mong LLM sinh ra một câu trả lời trung thực dựa trên các văn bản đó. Tuy nhiên, hệ thống truy xuất thông tin đôi khi có thể cung cấp các văn bản không liên quan hoặc không chính xác, điều này có thể khiến LLM đưa ra câu trả lời sai. Một cách đơn giản để giải quyết vấn đề này là cải thiện độ chính xác của hệ thống truy xuất thông tin. Tuy vậy, như với hầu hết các hệ thống AI, lỗi vẫn có thể xảy ra. Vì vậy, cũng cần tăng cường tính vững của LLM, để nó có thể đưa ra dự đoán hợp lý ngay cả khi đầu vào không chính xác. Dưới đây là một prompt mới giúp LLM trung thành hơn với sự thật, và cho phép nó chọn không trả lời khi thông tin được cung cấp không chính xác.

Nhiệm vụ của bạn là trả lời câu hỏi sau. Để giúp bạn, các văn bản liên quan được cung cấp kèm theo. Hãy dựa vào các văn bản này để trả lời.
Xin lưu ý rằng câu trả lời của bạn cần chính xác nhất có thể và trung thành với sự thật. Nếu thông tin được cung cấp không đủ để trả lời chính xác, bạn có thể chỉ cần xuất ra ”Không có câu trả lời!”.
Câu hỏi:
Thế vận hội 2028 sẽ được tổ chức ở đâu?
Văn bản liên quan 1:
Thế vận hội Mùa hè 2024, tên chính thức là Đại hội Thể thao Olympic lần thứ XXXIII và mang thương hiệu Paris 2024, là một sự kiện thể thao đa môn quốc tế …
…
Không có câu trả lời!

Trong ví dụ này, LLM từ chối trả lời vì thông tin được cung cấp không đủ và không liên quan tới câu hỏi.

Chú giải của người dịch: RAG giống làm bài “được mở tài liệu”

RAG giống như một bài kiểm tra cho phép mở tài liệu: trước khi trả lời, ta tìm vài trang sách liên quan rồi đặt chúng ngay trong đề bài để mô hình đọc và trả lời dựa vào đó, thay vì chỉ dựa vào “trí nhớ” từ lúc huấn luyện (vốn có thể đã cũ hoặc nhớ sai). Nhưng nếu “tài liệu” tìm được lại nói về chuyện khác — như trong ví dụ thứ hai, đoạn văn nói về Thế vận hội 2024 ở Paris chứ không phải 2028 — thì một học sinh trung thực nên trả lời “không đủ thông tin” thay vì đoán bừa. Dòng chỉ dẫn được tô xanh dạy cho mô hình đúng thái độ đó, giúp giảm hiện tượng ảo giác (*hallucination*) — tức mô hình “bịa” ra câu trả lời nghe có vẻ hợp lý nhưng sai.

Cả RAG và tinh chỉnh đều là những phương pháp phổ biến để thích nghi LLM bằng dữ liệu riêng của nhiệm vụ. RAG tiêu chuẩn không cần huấn luyện và có thể áp dụng trực tiếp cho LLM. Để cải thiện RAG hơn nữa, cũng có thể tinh chỉnh LLM, dù việc này đòi hỏi một ít công sức huấn luyện. Chẳng hạn, ta có thể tinh chỉnh LLM bằng dữ liệu do con người gán nhãn để giám sát chúng học cách từ chối trả lời. Lưu ý rằng mặc dù các ví dụ ở trên có vẻ đơn giản, RAG không hề tầm thường. Từ góc độ thiết kế prompt, các trường hợp sử dụng khác nhau có thể cần các prompt khác nhau, mặc dù mục tiêu rộng hơn của ta là phát triển các chiến lược prompting có thể khái quát hóa qua nhiều nhiệm vụ. Trong nhiều

trường hợp, ta cần kiểm soát mức độ phụ thuộc vào ngữ cảnh truy xuất được khi đưa ra dự đoán. Đôi khi, LLM phải sinh câu trả lời hoàn toàn dựa trên các văn bản được cung cấp, trong khi những lúc khác, chúng có thể cần sinh câu trả lời bằng tri thức sẵn có của mình nếu các văn bản được cung cấp không đủ. Có nhiều khía cạnh của RAG, như cải tiến hệ thống truy xuất, không thể đề cập trong chương này. Bạn đọc quan tâm có thể tham khảo các bài tổng quan về kỹ thuật RAG để biết thêm thông tin [Li và cộng sự, 2022d; Gao và cộng sự, 2023c].

Một lý do ta bàn về RAG ở đây là nó có thể được xem, theo nghĩa rộng, như một trường hợp của khung phân rã bài toán tổng quát (xem Mục 9.2.2). RAG chia việc giải bài toán thành hai bước. Ở bước thứ nhất, ta thu thập thông tin liên quan và hỗ trợ cho một truy vấn cho trước từ nhiều nguồn tri thức khác nhau. Ở bước thứ hai, ta dùng LLM để sinh câu trả lời dựa trên thông tin đã thu thập. Nếu mở rộng khái niệm phân rã bài toán hơn nữa, ta sẽ thấy rằng nhiều nhiệm vụ đòi hỏi dùng các hệ thống hay công cụ bên ngoài có thể được xử lý như những bài toán tương tự. Một ví dụ như vậy là *sử dụng công cụ* (*tool use*) trong LLM. Trong nhiều ứng dụng, LLM cần dùng các cơ sở dữ liệu bên ngoài, các API, và thậm chí cả các công cụ mô phỏng để sinh ra câu trả lời chính xác. Ví dụ, LLM có thể truy cập dữ liệu thời gian thực từ thị trường tài chính để cung cấp thông tin tài chính, hoặc truy cập các cơ sở tri thức y khoa để đưa ra thông tin y tế được cá nhân hóa. Sự tích hợp này mở rộng năng lực của LLM bằng cách cho phép chúng tương tác với các hệ thống bên ngoài, và trong một số bối cảnh, tác động tới hoặc điều khiển các hệ thống đó. Nhờ vậy, LLM có thể hoạt động như những *tác tử tự hành* (*autonomous agent*) đơn giản chứ không chỉ là những bộ sinh văn bản [Franklin và Graesser, 1996].

Sử dụng công cụ là một lĩnh vực nghiên cứu rộng. Ở đây ta thu hẹp thảo luận vào những nhiệm vụ có thể được hỗ trợ bằng cách gọi các API bên ngoài để giải một số bài toán con [Parisi và cộng sự, 2022; Gao và cộng sự, 2023b]. Hãy xét lại ví dụ yêu cầu LLM trả lời câu hỏi "Thế vận hội 2028 sẽ được tổ chức ở đâu?". Giả sử LLM có thể truy cập một công cụ tìm kiếm web. Khi đó ta có thể prompt LLM trả lời câu hỏi bằng tìm kiếm web như sau:

> Nhiệm vụ của bạn là trả lời câu hỏi sau. Bạn có thể dùng các công cụ bên ngoài, như tìm kiếm web, để hỗ trợ.
> Câu hỏi:
> Thế vận hội 2028 sẽ được tổ chức ở đâu?
> Thông tin liên quan đến câu hỏi này như sau:
> `{tool: web-search, query: "2028 Olympics"}`
> Vậy câu trả lời là: Los Angeles

Ở đây `tool: web-search, query: "2028 Olympics"` biểu thị một yêu cầu gửi tới hệ thống tìm kiếm web với truy vấn "2028 Olympics" (Thế vận hội 2028). Lệnh gọi công cụ được sinh ra sẽ bị một hệ thống bên ngoài chặn lại; hệ thống này thực thi công cụ tương ứng và nối kết quả trả về vào ngữ cảnh. Ngữ cảnh đã cập nhật sau đó được đưa trở lại LLM cho các bước dự đoán tiếp theo, và nhờ vậy mô hình có thể sinh ra câu trả lời đúng bằng thông tin truy xuất được.

Hãy xét một ví dụ khác, trong đó ta yêu cầu LLM giải một bài toán.

> Bài toán:
> Cần bơm đầy nước vào một bể bơi. Bể dài 10 mét, rộng 4 mét và sâu 2 mét. Hãy tính thể tích của bể theo mét khối, rồi xác định cần bao nhiêu lít nước để làm đầy bể (biết 1 mét khối bằng 1000 lít).
> Lời giải:
> Để giải bài toán này, LLM cần tính thể tích của bể trước bằng công thức thể tích hình hộp chữ nhật: Dài $\times$ Rộng $\times$ Sâu. Do đó, thể tích là $10\,\text{m} \times 4\,\text{m} \times 2\,\text{m} =$ `{tool: calculator, expression: 10 * 4 * 2}`

m^3. Tiếp theo, để biết cần bao nhiêu lít nước, LLM nhân thể tích tính bằng mét khối với 1000 (vì 1 mét khối bằng 1000 lít). Như vậy, 80 $\times 1000$ = `{tool: calculator, expression: 80*1000}` lít.

Ở đây chuỗi `{tool: calculator, expression: 10 * 4 * 2}` kích hoạt việc gọi một trình thông dịch toán học để tính kết quả của biểu thức. Lưu ý rằng kết quả (tức 80) sẽ thay thế `{tool: calculator, expression: 10 * 4 * 2}` và có thể được tham chiếu trong các bước dự đoán token tiếp theo. Chẳng hạn, ở bước cuối của lời giải, 80 được dùng thay cho `{tool: calculator, expression: 10 * 4 * 2}`.

Chú giải của người dịch: LLM "gọi công cụ" như thế nào?

LLM không tự bấm máy tính hay tự lên mạng. Nó chỉ *viết ra* một đoạn văn bản theo khuôn dạng quy ước, như `{tool: calculator, expression: 10 * 4 * 2}`. Một chương trình bên ngoài luôn "theo dõi" văn bản mô hình đang sinh; hễ thấy khuôn dạng này, nó tạm dừng mô hình, tự tính $10 \cdot 4 \cdot 2 = 80$, thay đoạn lệnh bằng số 80, rồi cho mô hình viết tiếp. Vì thế mô hình "thấy" như thể chính nó đã viết ra số 80, và có thể dùng tiếp ở bước sau ($80 \times 1000 = 80\,000$ lít). Cách làm này giống như học sinh được phép dùng máy tính bỏ túi: học sinh lo phần lập luận (dùng công thức nào), còn máy lo phần tính toán cho chính xác.

Một điểm khác biệt chính giữa các ví dụ sử dụng công cụ ở đây và các ví dụ RAG đã bàn trước đó là trong sử dụng công cụ, các hàm bên ngoài có thể được gọi *trong lúc* suy luận. Ngược lại, trong RAG, các văn bản truy xuất được cung cấp *trước khi* quá trình dự đoán bắt đầu. Tuy nhiên, từ góc độ mô hình hóa ngôn ngữ, chúng tương tự nhau: trước khi sinh kết quả cuối cùng, ta dùng các công cụ bên ngoài, một cách thủ công hoặc tự động, để có được ngữ cảnh đầy đủ và liên quan. Một cách diễn giải ở mức cao về các cách tiếp cận này là cả hai đều dựa vào một "tác tử" có thể xác định chỗ nào và cách nào để gọi các hàm bên ngoài nhằm tạo ra ngữ cảnh cần thiết cho việc dự đoán.

Một vấn đề với việc sử dụng công cụ là LLM ban đầu không được huấn luyện để sinh ra các dấu hiệu cần thiết cho việc dùng công cụ. Vì vậy, ta cần tinh chỉnh LLM để thích nghi nó với các nhiệm vụ này [Schick và cộng sự, 2024]. Vì chương này tập trung vào prompting, ta sẽ không trình bày chi tiết quá trình tinh chỉnh này. Nói đơn giản, trước tiên ta cần gán nhãn dữ liệu. Với mỗi ví dụ tinh chỉnh, ta thay những phần của đầu ra cần dùng công cụ bên ngoài bằng các lệnh hoặc dấu hiệu được định nghĩa trước. Sau đó, ta dùng dữ liệu đã gán nhãn này để tinh chỉnh các tham số của LLM như thường lệ. Kết quả là LLM có thể có được khả năng sinh ra các lệnh gọi công cụ bên ngoài. Trong lúc suy luận, ta có thể thực thi các lệnh dùng công cụ này trong đầu ra của mô hình để nhận trợ giúp từ các công cụ bên ngoài.

9.3 Học cách tạo prompt

Cho đến giờ trong chương này, chúng ta đã xem xét một số chiến lược prompting cơ bản và nhiều cải tiến khác nhau cho chúng. Tuy nhiên, tất cả các prompt mà ta đã bàn đều được thiết kế thủ công. Điều này dẫn đến một số vấn đề. Thứ nhất, thiết kế prompt chất lượng cao là việc khó và thường đòi hỏi thử nghiệm rất nhiều để tìm ra các prompt hiệu quả. Vì các LLM khác nhau có thể nhạy cảm ở mức khác nhau với các phong cách prompt, việc xây dựng những prompt tổng quát hóa tốt trên nhiều mô hình có thể tốn nhiều tài nguyên. Thứ hai, thiết kế prompt thủ công phụ thuộc nặng nề vào chuyên môn của con người, điều này có thể hạn chế sự đa dạng của các cách tiếp cận và bỏ sót những prompt có tiềm năng hiệu quả nhưng con người không nhận ra ngay. Thứ ba, các prompt do con người tạo ra có thể phức tạp và dài dòng, dẫn đến đầu vào dài hơn cho LLM và chi phí tính toán cao hơn.

Trong mục này, chúng ta bàn về các kỹ thuật prompting tự động (*automated prompting*). Các phương pháp này nhằm tự động tạo ra, tối ưu hóa và biểu diễn prompt sao cho các nhiệm vụ hạ nguồn (*downstream task*) có thể được giải quyết hiệu quả hơn (cả về chất lượng lẫn chi phí). Cụ thể, ở đây ta xem xét ba vấn đề.

- Làm thế nào để tự động hóa quá trình thiết kế và tối ưu hóa prompt cho LLM?
- Có những cách biểu diễn prompt nào khác ngoài chuỗi văn bản thuần túy hay không, và làm thế nào để học được những biểu diễn đó?
- Làm thế nào để prompt ngắn gọn và cô đọng hơn, qua đó giảm độ phức tạp và độ dài của chúng?

Lưu ý rằng có nhiều bối cảnh trong đó ta có thể nghiên cứu các vấn đề này. Ví dụ, prompt có thể được tối ưu hóa cho một LLM cụ thể, hoặc được thiết kế để tổng quát hóa trên nhiều LLM khác nhau. Những bối cảnh này có thể dẫn đến các phương pháp và kịch bản ứng dụng khác nhau, nhưng các phương pháp ấy có thể trùng lặp nhau ở một số khía cạnh. Trong phần thảo luận dưới đây, ta sẽ đề cập đến một vài kịch bản khác nhau và bàn về mối liên hệ giữa các phương pháp.

9.3.1 Tối ưu hóa prompt

Vì thiết kế prompt tốn nhiều công sức, ta mong muốn dùng các mô hình học máy để tìm ra prompt tối ưu cho một nhiệm vụ cụ thể (gọi là **thiết kế prompt tự động** (*automatic prompt design*) hay **tối ưu hóa prompt** (*prompt optimization*)). Cách tiếp cận này, nói rộng ra, có thể được xem là một trường hợp của **học máy tự động** (*automated machine learning*) (**AutoML**), lĩnh vực nhằm giảm bớt hoặc loại bỏ nhu cầu thiết kế mô hình học máy thủ công bởi chuyên gia. Mặc dù trọng tâm của ta ở đây là thiết kế prompt, bản thân prompt là những cấu trúc rời rạc. Do đó, tối ưu hóa prompt tương tự như việc tìm kiếm trên các kiến trúc mô hình rời rạc trong học máy. Một lĩnh vực có liên quan chặt chẽ là **tìm kiếm kiến trúc nơ-ron** (*neural architecture search*) (**NAS**), trong đó các mạng nơ-ron tối ưu được xác định bằng cách khám phá một không gian các mạng nơ-ron khả dĩ [Zoph và Le, 2016; Elsken và cộng sự, 2019a]. Nếu xem tối ưu hóa prompt là một quá trình tìm kiếm, ta có thể mô tả một khung tối ưu hóa prompt tổng quát gồm các thành phần sau:

- **Không gian tìm kiếm prompt** (*Prompt Search Space*). Thành phần này xác định mọi prompt khả dĩ mà thuật toán có thể khám phá. Ví dụ, ta có thể chỉnh sửa một số prompt hạt giống (*seed prompt*) để sinh ra một tập các prompt ứng viên đa dạng.
- **Ước lượng hiệu năng** (*Performance Estimation*). Khi một prompt đã được chọn, nó cần được đánh giá. Ví dụ, một cách đơn giản là đưa nó vào một LLM và đo hiệu năng trên một tập kiểm định (*validation set*).
- **Chiến lược tìm kiếm** (*Search Strategy*). Quá trình tìm kiếm nhìn chung giống với quá trình được dùng trong nhiều hệ thống AI. Ở mỗi bước, hệ thống khám phá một tập các prompt triển vọng trong không gian tìm kiếm và đánh giá chúng. Quá trình này tiếp diễn khi ngày càng nhiều prompt được khám phá. Kết quả của việc tìm kiếm là prompt có hiệu năng tốt nhất quan sát được cho đến khi việc tìm kiếm dừng lại.

Chú giải của người dịch: Tối ưu hóa prompt giống việc gì quen thuộc?

Hãy tưởng tượng bạn cần viết một "câu đề bài" sao cho một bạn học sinh (ở đây là LLM) làm bài đúng nhiều nhất. Bạn thử viết vài phiên bản đề (*không gian tìm kiếm*), cho bạn ấy làm thử 50 câu đã biết đáp án rồi đếm

số câu đúng (*ước lượng hiệu năng*), giữ lại vài phiên bản tốt nhất, sửa chúng thành phiên bản mới rồi thử tiếp (*chiến lược tìm kiếm*). Tối ưu hóa prompt là tự động hóa vòng lặp "thử – chấm điểm – sửa" này bằng máy tính. Gọi prompt là "rời rạc" vì nó là một dãy từ: ta không thể "nhích" một từ đi một lượng nhỏ như nhích một số thực, mà chỉ có thể thay từ này bằng từ khác.

Đây là một khung rất tổng quát, và các hệ thống tối ưu hóa prompt khác nhau có thể khác nhau ở cách thiết kế từng thành phần. Một cách tiếp cận được dùng rộng rãi là lấy LLM làm nền tảng để xây dựng các thành phần này. Ban đầu, một vài prompt được cung cấp. Sau đó, quá trình sau được lặp lại cho đến khi thỏa mãn một tiêu chí dừng: 1) các prompt được đánh giá trên một tập kiểm định; 2) một tập ứng viên (*candidate pool*) được duy trì bằng cách chỉ giữ lại những prompt triển vọng nhất; và 3) các prompt mới được tạo ra bằng cách dùng LLM suy ra những prompt tương tự từ tập ứng viên này. Một lợi ích của cách tiếp cận này là nó cho phép ta dùng các LLM có sẵn để thực hiện các nhiệm vụ nói trên mà không cần phát triển hệ thống phức tạp. Để làm được điều này, ta có thể prompt hoặc tinh chỉnh LLM để thích nghi chúng với các nhiệm vụ này. Ở đây ta xét phương pháp của Zhou và cộng sự [2023c] để minh họa việc tối ưu hóa prompt dựa trên LLM. Phương pháp này gồm các bước sau.

- **Khởi tạo** (*Initialization*). Gọi C là tập ứng viên các prompt cần khám phá. Bước đầu tiên là thêm các prompt ban đầu vào C. Ta có thể làm việc này theo vài cách. Một cách đơn giản là tự tay viết các prompt như vậy cho một nhiệm vụ cho trước. Tuy nhiên, trong nhiều trường hợp con người có hiểu biết hạn chế về cách viết prompt hiệu quả cho nhiệm vụ, việc xây dựng prompt trở nên khó khăn. Trong những trường hợp đó, ta nên dùng LLM để sinh prompt. Ví dụ, ta có thể trực tiếp yêu cầu LLM tạo ra prompt, kèm theo mô tả về nhiệm vụ.

  > Bạn được giao một nhiệm vụ cần hoàn thành bằng LLM. Hãy viết một prompt để hướng dẫn LLM.
  > {∗mô-tả-nhiệm-vụ∗}
  >
  > ___

  Phương pháp này đơn giản, nhưng nó vẫn cần một mô tả nhiệm vụ do con người cung cấp. Một phương pháp khác là dùng LLM để sinh prompt khi được cho các ví dụ về đầu vào và đầu ra của nhiệm vụ. Dưới đây là một khuôn mẫu prompt (*prompt template*).

  > Bạn được cung cấp một vài cặp đầu vào – đầu ra của một nhiệm vụ. Hãy viết một chỉ dẫn để thực hiện nhiệm vụ này.
  > Đầu vào: {∗đầu-vào-1∗} Đầu ra: {∗đầu-ra-1∗}
  > Đầu vào: {∗đầu-vào-2∗} Đầu ra: {∗đầu-ra-2∗}
  > ...
  >
  > ___

  Bằng cách này, LLM có thể suy ra chỉ dẫn tương ứng cho nhiệm vụ từ các đầu vào và đầu ra được cung cấp.

- **Đánh giá** (*Evaluation*). Khi đã có tập ứng viên C, ta cần đánh giá các prompt trong C. Một cách là đưa từng prompt vào một LLM và đánh giá kết quả trên nhiệm vụ hạ nguồn. Ví dụ, ta có thể đánh giá đầu ra của LLM với một đầu vào cho trước bằng một độ đo (*metric*) định sẵn, hoặc dùng log-hợp lý (*log-likelihood*) của đầu ra làm thước đo chất lượng của prompt.

- **Cắt tỉa** (*Pruning*). Nếu C chứa một số lượng lớn prompt, việc cắt bỏ những prompt kém triển vọng trong đó là hợp lý, nhờ vậy giảm gánh nặng tính toán ở các bước sau. Đây là một bài toán

cắt tỉa tiêu chuẩn. Với điểm đánh giá của mỗi prompt, một cách đơn giản là chỉ giữ lại một tỷ lệ phần trăm nhất định các prompt và loại bỏ phần còn lại.

- **Mở rộng** (*Expansion*). Mở rộng là một thao tác then chốt trong các thuật toán tìm kiếm, dùng để khám phá các trạng thái khác nhau trong không gian tìm kiếm. Thao tác mở rộng ở đây có thể được định nghĩa là một hàm:

$$C' = \text{Expand}(C, f), \tag{9.11}$$

trong đó C' là tập các prompt mới được sinh ra từ C bằng mô hình f. Nếu xem f là một LLM, ta có thể thực hiện thao tác mở rộng bằng cách yêu cầu f sinh ra các prompt mới và có liên quan dựa trên C. Dưới đây là một ví dụ.

> Dưới đây là một prompt dành cho LLM. Hãy đưa ra một vài prompt mới để thực hiện cùng nhiệm vụ đó.
> Đầu vào: {∗prompt∗}
>
> ___

Sau đó, ta thay C bằng C'. Các bước đánh giá, cắt tỉa và mở rộng có thể được lặp lại, nhờ đó ta có thể dần dần khám phá một phạm vi prompt rộng hơn.

Chú giải của người dịch: Một vòng lặp tìm kiếm cụ thể

Giả sử nhiệm vụ là phân loại cảm xúc của câu nhận xét. Khởi tạo: C gồm 10 prompt do LLM viết. Đánh giá: mỗi prompt được thử trên 100 câu đã có nhãn, ví dụ prompt A đúng 81 câu, prompt B đúng 74 câu, … Cắt tỉa: giữ 3 prompt điểm cao nhất (giữ 30%). Mở rộng: nhờ LLM viết lại mỗi prompt còn lại thành 3–4 biến thể, được tập C' khoảng 10 prompt mới. Lặp lại vài vòng, cuối cùng chọn prompt có điểm cao nhất. Trong Phương trình (9.11), Expand chỉ là tên của thao tác "sinh prompt mới"; f là mô hình dùng để sinh. *Log-hợp lý* của đầu ra là logarit của xác suất mà LLM gán cho đáp án đúng: xác suất càng gần 1 thì log càng gần 0, tức prompt càng "dẫn" mô hình tới đáp án đúng một cách tự tin.

Trong tối ưu hóa prompt, bước mở rộng đóng vai trò then chốt, vì nó xác định cách ta khám phá không gian tìm kiếm, và mục tiêu của ta là tìm ra kết quả tối ưu với ít công sức nhất. Một cải tiến cho bước này là coi bài toán như một nhiệm vụ diễn đạt lại (*paraphrasing*). Một cách đơn giản là áp dụng các hệ thống diễn đạt lại có sẵn, dựa trên LLM hoặc các mô hình khác, để biến đổi prompt đầu vào thành các dạng tương đương về ngữ nghĩa [Jiang và cộng sự, 2020]. Ngoài ra, ta có thể định nghĩa các thao tác chỉnh sửa cụ thể, chẳng hạn chèn và sửa đổi, cho mỗi token. Một prompt cho trước có thể được chỉnh sửa thành các prompt mới bằng cách áp dụng các thao tác này [Prasad và cộng sự, 2023]. Đồng thời, có thể áp dụng thêm đánh giá và cắt tỉa để lọc bỏ các prompt chất lượng thấp. Ngoài việc coi việc sinh prompt là một bài toán diễn đạt lại, ta có thể cải thiện chất lượng prompt trong bước mở rộng bằng cách học từ phản hồi (*feedback*) [Pryzant và cộng sự, 2023]. Cách tiếp cận này có phần liên quan đến vấn đề tự tinh chỉnh (*self-refinement*) đã bàn ở Mục 9.2.3. Một LLM có thể được dùng để sinh phản hồi về một prompt đầu vào, và prompt này sau đó được sửa lại dựa trên phản hồi đó. Chu trình phản hồi – sửa đổi này có thể được lặp lại nhiều lần cho đến khi kết quả hội tụ hoặc đạt được kết quả mong muốn.

Một cách tiếp cận khác cho tối ưu hóa prompt là áp dụng các kỹ thuật tối ưu hóa kinh điển. Ví dụ, bài toán có thể được phát biểu thành một bài toán tính toán tiến hóa (*evolutionary computation*), trong đó các prompt được coi là các ứng viên tiến hóa qua từng thế hệ khi quá trình tối ưu hóa diễn ra [Guo và cộng sự, 2024]. Vì nhiều thuật toán tối ưu hóa mạnh đã được phát triển trong các lĩnh vực liên quan, chúng có thể được áp dụng trực tiếp cho bài toán này.

Chú giải của người dịch: Tính toán tiến hóa

Ý tưởng mượn từ thuyết tiến hóa trong sinh học (lớp 12): trong một "quần thể" prompt, các prompt có điểm cao ("thích nghi tốt") được giữ lại làm "bố mẹ"; con cháu được tạo ra bằng cách "lai" (ghép một nửa prompt này với một nửa prompt kia) và "đột biến" (thay vài từ). Qua nhiều thế hệ, chất lượng trung bình của quần thể tăng dần, giống chọn lọc tự nhiên.

Trong thực tế, ta có thể muốn dùng các API LLM sẵn có để hiện thực các bước nói trên. Tuy nhiên, cách làm như vậy phụ thuộc nhiều vào khả năng lập luận và học trong ngữ cảnh của LLM. Nếu các LLM này không đủ mạnh và chưa được thích nghi với các nhiệm vụ, chúng có thể đưa lỗi vào quá trình tìm kiếm, chẳng hạn sinh ra các prompt sai trong bước mở rộng. Trong những trường hợp đó, tốt hơn là huấn luyện các mô hình phù hợp hơn với nhiệm vụ. Một cách tiếp cận theo hướng nghiên cứu này sử dụng học tăng cường (*reinforcement learning*), vốn đã được dùng rộng rãi để giải các bài toán ra quyết định và tối ưu hóa rời rạc. Ví dụ, Deng và cộng sự [2022] đã phát triển một bộ sinh prompt (*prompt generator*) bằng cách tích hợp một bộ thích nghi (*adaptor*) dựa trên FFN vào một LLM. Bộ sinh prompt được huấn luyện như một mạng chính sách (*policy network*) điển hình, nhưng chỉ các tham số của bộ thích nghi được cập nhật, còn các tham số còn lại của mô hình được giữ nguyên. Trong quá trình huấn luyện, phần thưởng (*reward*) có được bằng cách kiểm tra các prompt sinh ra bằng một LLM khác, tương tự phương pháp đánh giá đã bàn ở trên. Khi huấn luyện xong, bộ sinh prompt được dùng để sinh ra các prompt mới.

Chú giải của người dịch: Học tăng cường ở đây hoạt động thế nào?

Có thể hình dung bộ sinh prompt như một người chơi: mỗi lượt nó "viết" ra một prompt (một chuỗi quyết định chọn từng token), rồi nhận về một điểm thưởng — chẳng hạn tỷ lệ câu trả lời đúng khi đưa prompt đó cho một LLM khác. Prompt nào được điểm cao thì lần sau nó có xu hướng viết giống thế hơn. "Mạng chính sách" là mạng nơ-ron quyết định "nên chọn token nào tiếp theo". Chỉ một phần nhỏ (bộ thích nghi) được học, phần LLM khổng lồ còn lại để nguyên, nên việc huấn luyện rẻ hơn nhiều.

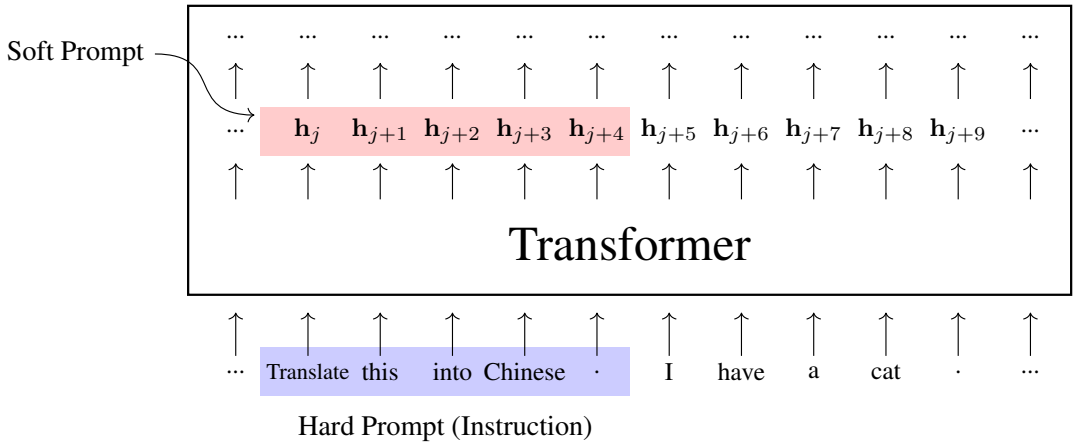
Lưu ý rằng trong thảo luận ở đây, prompt đơn giản được xem như các chuỗi token, và đầu ra của tối ưu hóa prompt là một chuỗi như vậy. Tuy nhiên, nói một cách chặt chẽ, prompt thường gồm nhiều thành phần, chẳng hạn chỉ dẫn, ví dụ minh họa và đầu vào của người dùng. Mặc dù các cách tiếp cận đã bàn phần lớn mang tính tổng quát, nhiều công trình về tối ưu hóa prompt tập trung vào việc học các chỉ dẫn tốt hơn cho prompting. Cụ thể, mục tiêu là sinh ra các chỉ dẫn dẫn dắt LLM một cách hiệu quả dựa trên một nhiệm vụ cho trước. Dĩ nhiên, khái niệm tối ưu hóa prompt cũng có thể được mở rộng sang việc học các phần khác của prompt. Ví dụ, đã có rất nhiều quan tâm nghiên cứu về việc học cách chọn hoặc sinh các ví dụ minh họa trong CoT [Liu và cộng sự, 2022; Rubin và cộng sự, 2022; Zhang và cộng sự, 2023b]. Một trong những khác biệt giữa học chỉ dẫn và học ví dụ minh họa là việc sinh ví dụ minh họa chất lượng cao bằng LLM tương đối dễ, và trọng tâm của học ví dụ minh họa thường là cách lấy mẫu những ví dụ minh họa phù hợp từ một tập ứng viên. Ngược lại, khó khăn trong việc học chỉ dẫn một phần là do các LLM đã tiền huấn luyện không phù hợp để dự đoán chất lượng của chỉ dẫn, và việc kiểm tra các chỉ dẫn này trên nhiệm vụ hạ nguồn tốn kém về tính toán. Điều này khiến các phương pháp tối ưu hóa tốn kém khi áp dụng, và việc khám phá nhiều loại chỉ dẫn khác nhau đặt ra những thách thức đáng kể.

9.3.2 Prompt mềm

Mặc dù xây dựng prompt bằng ngôn ngữ tự nhiên, dù thủ công hay tự động, là một cách tiếp cận đơn giản và được áp dụng rộng rãi, nó vẫn có một số vấn đề. Một vấn đề là prompt bằng ngôn ngữ tự nhiên có thể phức tạp và dài, gây ra gánh nặng tính toán đáng kể khi được LLM xử lý. Trong nhiều ứng dụng, người dùng có thể cần thực hiện một nhiệm vụ lặp đi lặp lại, và việc nhập cùng một prompt dài vào LLM rất nhiều lần rõ ràng là kém hiệu quả. Một vấn đề khác là mặc dù prompt thường được biểu diễn dưới dạng các chuỗi token rời rạc (gọi là **prompt cứng** (*hard prompt*)) trong đầu vào LLM thông thường, LLM lại mã hóa chúng thành các vectơ liên tục. Điều này đặt ra câu hỏi liệu có cách nào gọn hơn và hiệu quả hơn để biểu diễn prompt hay không.

Trong tiểu mục này, chúng ta giới thiệu khái niệm **prompt mềm** (*soft prompt*), có thể được xem là các biểu diễn phân tán (*distributed representation*) ẩn của prompt. Khi prompt một LLM, ta quan tâm đến việc truyền đạt nhiệm vụ hoặc câu hỏi để gợi ra những câu trả lời mong muốn. Ta có thể định nghĩa prompt cứng là các chuỗi văn bản tường minh, định sẵn mà người dùng nhập trực tiếp vào LLM để dẫn dắt câu trả lời. Ngược lại, ta có thể nghĩ về prompt mềm như những khuôn mẫu prompting ngầm, có thể thích nghi, được biểu diễn trong không gian ẩn (*hidden space*) liên tục của LLM. Khác với prompt cứng, vốn được diễn đạt bằng ngôn ngữ tự nhiên và cần dễ hiểu đối với con người, prompt mềm được mã hóa ở một định dạng dễ hiểu đối với mô hình hơn là đối với con người. Để minh họa, hãy xét một prompt đơn giản

> Translate the sentence into Chinese. (Dịch câu này sang tiếng Trung.)
>
> Cứ coi như xong rồi!
>
> ——



Hình 9.3: Minh họa prompt cứng và prompt mềm. Ở đây prompt cứng là chỉ dẫn mà ta đưa vào LLM để thực hiện nhiệm vụ. LLM mã hóa chỉ dẫn này như thường lệ, và các biểu diễn trung gian tương ứng với chỉ dẫn có thể được xem như một dạng prompt mềm.

Ở đây, chỉ dẫn "Translate the sentence into Chinese" có thể được xem là một prompt cứng, ký hiệu bằng chuỗi token $c_1 \cdots c_5$. Khi đưa các token này vào một LLM, chúng được biến đổi thành một chuỗi các vectơ giá trị thực $\mathbf{h}_1 \cdots \mathbf{h}_5$, mỗi vectơ tương ứng với một token. Ta có thể xem $\mathbf{h}_1 \cdots \mathbf{h}_5$ như một minh họa trực quan cho prompt mềm, như trong Hình 9.3.

Chú giải của người dịch: Đọc Hình 9.3

Hàng dưới cùng là các token đầu vào: phần tô màu "Translate this into Chinese ." (*Dịch câu này sang tiếng Trung.*) là *prompt cứng* (*Hard Prompt (Instruction)* — prompt cứng, dạng chỉ dẫn), tiếp theo là câu cần dịch "I have a cat." (*Tôi có một con mèo.*). Bên trong Transformer, mỗi token được biến thành một vectơ $\mathbf{h}$ — một dãy hàng trăm, hàng nghìn con số. Các vectơ $\mathbf{h}_j, \ldots, \mathbf{h}_{j+4}$ ứng với phần chỉ dẫn (được tô màu, nhãn *Soft Prompt*) chính là một "prompt mềm": cùng mang ý nghĩa "hãy dịch sang tiếng Trung" nhưng ở dạng số mà chỉ máy "đọc" được. Chữ "cứng" (rời rạc, chọn từ một bộ từ vựng hữu hạn) và "mềm" (liên tục, mỗi tọa độ có thể nhận bất kỳ số thực nào) gợi lại sự khác nhau giữa tập số nguyên và tập số thực.

Mặc dù ví dụ trên cho thấy prompt mềm có thể được sinh ra bằng cách biến đổi prompt cứng, giữa chúng không nhất thiết có sự tương ứng trực tiếp. Thực ra, ta thậm chí không cần diễn giải prompt mềm bằng văn bản có nghĩa. Thay vào đó, prompt mềm được biểu diễn dưới dạng các vectơ liên tục trong không gian ẩn của LLM và có thể được tối ưu hóa trực tiếp như những tham số học được. Cách xử lý như vậy cho phép ta khám phá các phương pháp prompting vượt ra ngoài văn bản. Một lợi ích nữa là prompt mềm cung cấp các biểu diễn dày đặc, học được, để mã hóa cách ta dẫn dắt LLM sinh ra những đầu ra cụ thể. Việc huấn luyện và áp dụng các biểu diễn này đòi hỏi chi phí tính toán thấp hơn đáng kể so với chi phí cần thiết để xử lý các prompt cứng dài. Cách tiếp cận này sẽ có giá trị thực tiễn lớn trong các ứng dụng suy luận LLM mà cùng một prompt được dùng lặp đi lặp lại.

1. Thích nghi LLM với prompt đơn giản hóa

Một cách hiển nhiên để thích nghi một LLM cho một nhiệm vụ cụ thể là đơn giản tinh chỉnh mô hình bằng dữ liệu có nhãn. Điều này dẫn đến nhiều phương pháp căn chỉnh (*alignment*) LLM, chẳng hạn tinh chỉnh có giám sát (*supervised fine-tuning*), vốn cập nhật tham số mô hình bằng cách căn chỉnh câu trả lời cho các prompt cho trước với các tín hiệu giám sát. Các LLM đã tinh chỉnh nhúng thông tin liên quan đến nhiệm vụ vào tham số mô hình, và nhờ vậy các mô hình này có thể trả lời đúng khi gặp những prompt tương tự với các prompt trong quá trình tinh chỉnh.

Nếu đẩy ý tưởng này đi xa hơn, ta có thể kỳ vọng LLM hấp thụ càng nhiều càng tốt tri thức về cách prompt một nhiệm vụ trong quá trình tinh chỉnh. Khi đó, thông tin prompting được nắm bắt một phần trong tham số mô hình, và LLM đã tinh chỉnh có thể thực hiện nhiệm vụ với ít prompting hơn. Ở đây ta xét một dạng prompt đơn giản, chỉ gồm một chỉ dẫn (ký hiệu là $\mathbf{c}$) và một đầu vào của người dùng (ký hiệu là $\mathbf{z}$). Một prompt có thể được biểu diễn bằng bộ sau:

$$\mathbf{x} \;=\; (\mathbf{c}, \mathbf{z}). \tag{9.12}$$

Cho một tập các cặp prompt – câu trả lời $\mathcal{D} = \{(\mathbf{x}, \mathbf{y})\}$, mục tiêu của tinh chỉnh là cực tiểu hóa tổng mất mát trên tập này. Một phương pháp phổ biến là cực tiểu hóa log-hợp lý âm (*negative log-likelihood*) (tức là cực đại hóa log-hợp lý) theo các tham số mô hình θ:

$$\begin{aligned}\hat{\theta} &= \arg\max_{\theta} \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \log \Pr_\theta(\mathbf{y}|\mathbf{x}) \\ &= \arg\max_{\theta} \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \log \Pr_\theta(\mathbf{y}|\mathbf{c},\mathbf{z}),\end{aligned} \tag{9.13}$$

trong đó $\Pr_\theta(\cdot|\cdot)$ là xác suất do một LLM với tham số θ dự đoán[11].

[11] Trong thực tế, ta khởi tạo θ bằng các tham số thu được từ tiền huấn luyện, rồi điều chỉnh θ một cách vừa phải để đảm bảo kết quả sau tinh chỉnh không lệch quá xa so với kết quả tiền huấn luyện.

Chú giải của người dịch: Đọc Phương trình (9.13)

$\Pr_\theta(\mathbf{y}|\mathbf{c},\mathbf{z})$ là xác suất mô hình (với bộ tham số θ) sinh ra đúng câu trả lời $\mathbf{y}$ khi nhận chỉ dẫn $\mathbf{c}$ và đầu vào $\mathbf{z}$ — một *xác suất có điều kiện*: "xác suất của $\mathbf{y}$ khi đã biết $\mathbf{c}$ và $\mathbf{z}$". Lấy log rồi cộng qua mọi cặp trong tập dữ liệu $\mathcal{D}$; $\arg\max_\theta$ nghĩa là "chọn bộ tham số θ làm cho tổng này lớn nhất". Vì log là hàm đồng biến, làm tổng log lớn nhất cũng là làm cho mô hình gán xác suất cao cho các câu trả lời đúng. Ví dụ với hai mẫu có xác suất 0.5 và 0.8: tổng log $= \log 0.5 + \log 0.8 \approx -0.92$ (log tự nhiên); nếu tham số mới cho 0.9 và 0.9 thì tổng ≈ -0.21, lớn hơn, tức tốt hơn. Dấu mũ trong $\hat{\theta}$ chỉ "giá trị tìm được" của θ.

Nói chung, chỉ dẫn trong mỗi ví dụ tinh chỉnh nên tuân theo các nguyên tắc thiết kế prompt; ví dụ, một chỉ dẫn tốt nên rõ ràng nhất có thể và mô tả chi tiết nhiệm vụ. Tuy nhiên, phương pháp mô tả trong phương trình trên không ràng buộc chỉ dẫn phải có dạng cụ thể nào. Sự linh hoạt này cho phép ta ra chỉ dẫn cho LLM theo bất kỳ cách nào ta muốn. Xét một ví dụ trong đó ta muốn chỉ dẫn LLM dịch một câu tiếng Anh sang tiếng Trung. Dĩ nhiên, như đã nói ở phần trước của chương này, ta có thể prompt LLM bằng chỉ dẫn

Dịch câu sau từ tiếng Anh sang tiếng Trung.

Nếu muốn chỉ dẫn đơn giản hơn, ta có thể diễn đạt lại nó ở dạng đơn giản hơn

Dịch câu này sang tiếng Trung.

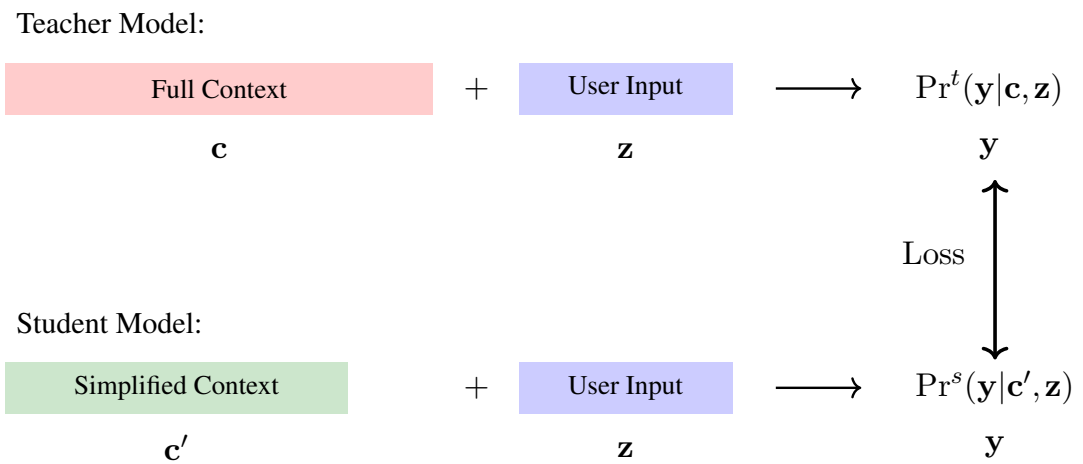
Ta thậm chí có thể định nghĩa chỉ dẫn chỉ là một cụm từ duy nhất

Dịch!

Với tinh chỉnh, ta có thể thích nghi LLM để làm theo bất kỳ chỉ dẫn nào trong số này. Từ góc độ prompting hiệu quả, việc đơn giản hóa chỉ dẫn trong prompting mang lại lợi thế về tính toán. Ví dụ, ta có thể dùng những chỉ dẫn đơn giản như "Dịch!" để thực hiện những nhiệm vụ mà thông thường đòi hỏi chỉ dẫn phức tạp và chi tiết hơn. Điều này có thể khiến việc prompting sau đó trong lúc suy luận dễ dàng hơn nhiều. Mặt khác, tinh chỉnh LLM với chỉ dẫn bị đơn giản hóa quá mức có thể gây hại cho khả năng tổng quát hóa của mô hình. Vì chỉ dẫn đơn giản hóa có thể dẫn đến mất mát thông tin, nhiều khả năng LLM sẽ quá khớp (*overfit*) dữ liệu tinh chỉnh và không tổng quát hóa được ra ngoài các chỉ dẫn đó. Vấn đề này càng rõ rệt khi cả chỉ dẫn phức tạp lẫn chỉ dẫn đơn giản hóa cùng được đưa vào quá trình tinh chỉnh, vì dữ liệu có nhãn sẵn có thường không đủ để bao phủ một phạm vi rộng các biến thể của chỉ dẫn.

Một cách khác để thích nghi LLM với chỉ dẫn đơn giản hóa là thông qua chưng cất tri thức (*knowledge distillation*). Làm ví dụ, ta xét phương pháp chưng cất ngữ cảnh [Snell và cộng sự, 2022]. Mục tiêu của phương pháp này là học một mô hình trò (*student model*) có thể sử dụng chỉ dẫn đơn giản hóa, từ một mô hình thầy (*teacher model*) đã được huấn luyện tốt để làm theo chỉ dẫn. Hình 9.4 minh họa cách tiếp cận này. Mô hình thầy được xây dựng bằng một quá trình tinh chỉnh tiêu chuẩn: trước hết ta thu thập một lượng dữ liệu nhất định gồm chỉ dẫn, đầu vào của người dùng và câu trả lời đúng, rồi tiếp tục huấn luyện một mô hình đã tiền huấn luyện với tập dữ liệu này. Để xây dựng mô hình trò, ta cần xây dựng một tập dữ liệu mới $\mathcal{D}'$ trong đó mỗi mẫu là một bộ gồm một chỉ dẫn, một chỉ dẫn đơn giản hóa tương ứng và một đầu vào của người dùng, ký hiệu là $\mathbf{x}' = (\mathbf{c}, \mathbf{c}', \mathbf{z})$. Chưng cất tri thức được thực hiện bằng cách cực tiểu hóa một hàm mất mát định nghĩa trên đầu ra của mô hình thầy và mô hình trò:

$$\hat{\theta} \;=\; \arg\min_{\theta} \sum_{\mathbf{x}' \in \mathcal{D}'} \mathrm{Loss}\big(\Pr{}^{t}(\cdot|\cdot),\, \Pr{}^{s}_{\theta}(\cdot|\cdot),\, \mathbf{x}'\big), \tag{9.14}$$



Hình 9.4: Minh họa chưng cất ngữ cảnh (*context distillation*) [Snell và cộng sự, 2022]. Mô hình thầy là một LLM tiêu chuẩn, nhận cả ngữ cảnh lẫn đầu vào của người dùng làm đầu vào mô hình và đưa ra một dự đoán làm đầu ra. Sau đó, ta đơn giản hóa ngữ cảnh (ví dụ, đơn giản hóa chỉ dẫn trong prompt) và dùng mô hình trò để đưa ra dự đoán dựa trên ngữ cảnh đã đơn giản hóa và đầu vào của người dùng. Mô hình trò được huấn luyện bằng cách cực tiểu hóa mất mát giữa các dự đoán do hai mô hình tạo ra.

trong đó $\Pr^t(\cdot|\cdot)$ ký hiệu mô hình thầy đã huấn luyện sẵn, và $\Pr^s_\theta(\cdot|\cdot)$ ký hiệu mô hình trò với tham số θ. Để ký hiệu đơn giản, ta sẽ viết gọn $\text{Loss}(\Pr^t(\cdot|\cdot), \Pr^s_\theta(\cdot|\cdot), \mathbf{x}')$ thành Loss. Một hàm mất mát thường dùng là mất mát mức chuỗi (*sequence-level loss*), có dạng cơ bản:

$$\text{Loss} \;=\; -\sum_{\mathbf{y}} \Pr^t(\mathbf{y}|\mathbf{c},\mathbf{z}) \log \Pr^s_\theta(\mathbf{y}|\mathbf{c}',\mathbf{z}). \tag{9.15}$$

Nhưng hàm này không khả thi về mặt tính toán vì nó đòi hỏi lấy tổng trên một số lượng lớn theo cấp số mũ các chuỗi đầu ra khả dĩ. Một biến thể của phương pháp này là huấn luyện mô hình trò bằng các đầu ra do mô hình thầy sinh ra. Với mỗi mẫu, ta dùng mô hình thầy để tạo ra một đầu ra $\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \log \Pr^t(\mathbf{y}|\mathbf{c},\mathbf{z})$. Sau đó ta coi $\hat{\mathbf{y}}$ là đích để học, và hàm mất mát được cho bởi

$$\text{Loss} \;=\; -\log \Pr^s_\theta(\hat{\mathbf{y}}|\mathbf{c}',\mathbf{z}). \tag{9.16}$$

Ngoài ra, ta có thể cực tiểu hóa khoảng cách giữa các phân phối xác suất do hai mô hình sinh ra [Askell và cộng sự, 2021]. Ví dụ, hàm mất mát có thể được định nghĩa là độ phân kỳ KL (*KL divergence*) giữa hai phân phối đầu ra:

$$\text{Loss} \;=\; \mathrm{D}_{\mathrm{KL}}(\mathrm{P}^t \,\|\, \mathrm{P}^s_\theta), \tag{9.17}$$

trong đó

$$\mathrm{P}^t = \Pr^t(\cdot|\mathbf{c},\mathbf{z}), \tag{9.18}$$
$$\mathrm{P}^s_\theta = \Pr^s_\theta(\cdot|\mathbf{c}',\mathbf{z}). \tag{9.19}$$

Chú giải của người dịch: Thầy dạy trò bằng "chỉ dẫn rút gọn"

Mô hình thầy được đọc chỉ dẫn đầy đủ $\mathbf{c}$ (ví dụ "Dịch câu sau từ tiếng Anh sang tiếng Trung"), còn mô hình trò chỉ được đọc chỉ dẫn rút gọn $\mathbf{c}'$ ("Dịch!"), nhưng phải học cách trả lời *giống thầy*. Phương trình (9.16): thầy

viết ra bản dịch tốt nhất $\hat{\mathbf{y}}$, trò được huấn luyện để gán xác suất cao cho đúng bản dịch đó (mất mát $-\log$ xác suất càng nhỏ khi xác suất càng gần 1). Phương trình (9.17): thay vì chỉ bắt chước một đáp án, trò bắt chước cả "mức độ tin tưởng" của thầy vào mọi đáp án có thể. Độ phân kỳ KL đo hai phân phối xác suất khác nhau bao nhiêu: bằng 0 khi chúng trùng nhau và càng lớn khi chúng càng khác nhau. Ví dụ, thầy cho hai đáp án xác suất $(0.7; 0.3)$, trò cho $(0.7; 0.3)$ thì KL $= 0$; trò cho $(0.5; 0.5)$ thì KL $= 0.7 \ln \frac{0.7}{0.5} + 0.3 \ln \frac{0.3}{0.5} \approx 0.08 > 0$. Phương trình (9.15) không tính được vì số câu trả lời $\mathbf{y}$ khả dĩ là khổng lồ: với bộ từ vựng 50 000 token và câu dài 20 token, có tới $50\,000^{20}$ chuỗi.

Việc đơn giản hóa prompt thông qua tinh chỉnh hoặc chưng cất không nhất thiết tạo ra prompt mềm tường minh theo nghĩa thông thường. Tuy nhiên, các cách tiếp cận này có chung một trực giác: hành vi prompting đặc thù cho nhiệm vụ có thể dần dần được hấp thụ vào các biểu diễn liên tục gọn nhẹ bên trong mô hình.

Mặc dù ta mới chỉ giới hạn ở việc chưng cất tri thức cho chỉ dẫn, các cách tiếp cận bàn ở đây mang tính tổng quát. Bằng cách học từ đầu ra của mô hình thầy, tri thức trong prompting có thể được chưng cất vào tham số của mô hình trò. Do đó, mô hình đã chưng cất có thể được xem như mã hóa ngầm hành vi prompting đặc thù cho nhiệm vụ trong tham số của nó. Phương pháp này có thể được áp dụng cho nhiều bài toán khác trong học prompt, chẳng hạn nén ngữ cảnh dài và học prompt mềm như những thành phần cụ thể của LLM.

2. Học prompt mềm cho tinh chỉnh hiệu quả tham số

Tinh chỉnh toàn bộ (*full fine-tuning*), tức cập nhật mọi tham số của mô hình, là một phương pháp phổ biến để thích nghi LLM với các nhiệm vụ hạ nguồn. Mặc dù tinh chỉnh ít tốn kém về tính toán hơn nhiều so với tiền huấn luyện, áp dụng nó trong thực tế vẫn tốn kém. Vấn đề này thúc đẩy sự phát triển của các phương pháp tinh chỉnh hiệu quả tham số (*parameter-efficient fine-tuning*), nhằm giảm thiểu số tham số cần được cập nhật.

Một cách tiếp cận, gọi là **tinh chỉnh tiền tố** (*prefix fine-tuning*), gắn thêm một chuỗi các vectơ huấn luyện được, hay tiền tố (*prefix*), vào trước đầu vào của mỗi tầng Transformer [Li và Liang, 2021]. Các tiền tố này có thể được coi là các prompt mềm đóng vai trò ngữ cảnh bổ sung để dẫn dắt hành vi của mô hình trong các nhiệm vụ cụ thể. Trong quá trình tinh chỉnh, ta chỉ cần học các tiền tố để nhúng tri thức đặc thù cho nhiệm vụ. Vì vậy, phương pháp này hiệu quả vì nó chỉ thay đổi một phần nhỏ của mô hình thay vì điều chỉnh toàn bộ tham số mô hình.

Cụ thể, gọi đầu vào của một tầng ở độ sâu l là $\mathbf{H}^l = \mathbf{h}_0^l \mathbf{h}_1^l \cdots \mathbf{h}_m^l$. Đầu ra của tầng có thể được biểu diễn là:

$$\mathbf{H}^{l+1} = \text{Layer}(\mathbf{H}^l). \tag{9.20}$$

Trong tinh chỉnh tiền tố, ta mở rộng chuỗi $\mathbf{h}_0^l \mathbf{h}_1^l \cdots \mathbf{h}_m^l$ bằng cách thêm vài vectơ vào đầu, ký hiệu là $\mathbf{p}_0^l \mathbf{p}_1^l \cdots \mathbf{p}_n^l$. Do đó $\mathbf{H}^l$ có thể được viết dưới dạng:

$$\mathbf{H}^l = \underbrace{\mathbf{p}_0^l\, \mathbf{p}_1^l\, \cdots\, \mathbf{p}_n^l}_{\text{huấn luyện được}}\ \underbrace{\mathbf{h}_0^l\, \mathbf{h}_1^l\, \cdots\, \mathbf{h}_m^l}_{\text{đầu ra của tầng trước}}. \tag{9.21}$$

Ta chỉ giữ lại $m+1$ biểu diễn cuối cùng làm các trạng thái ẩn truyền sang tầng tiếp theo:

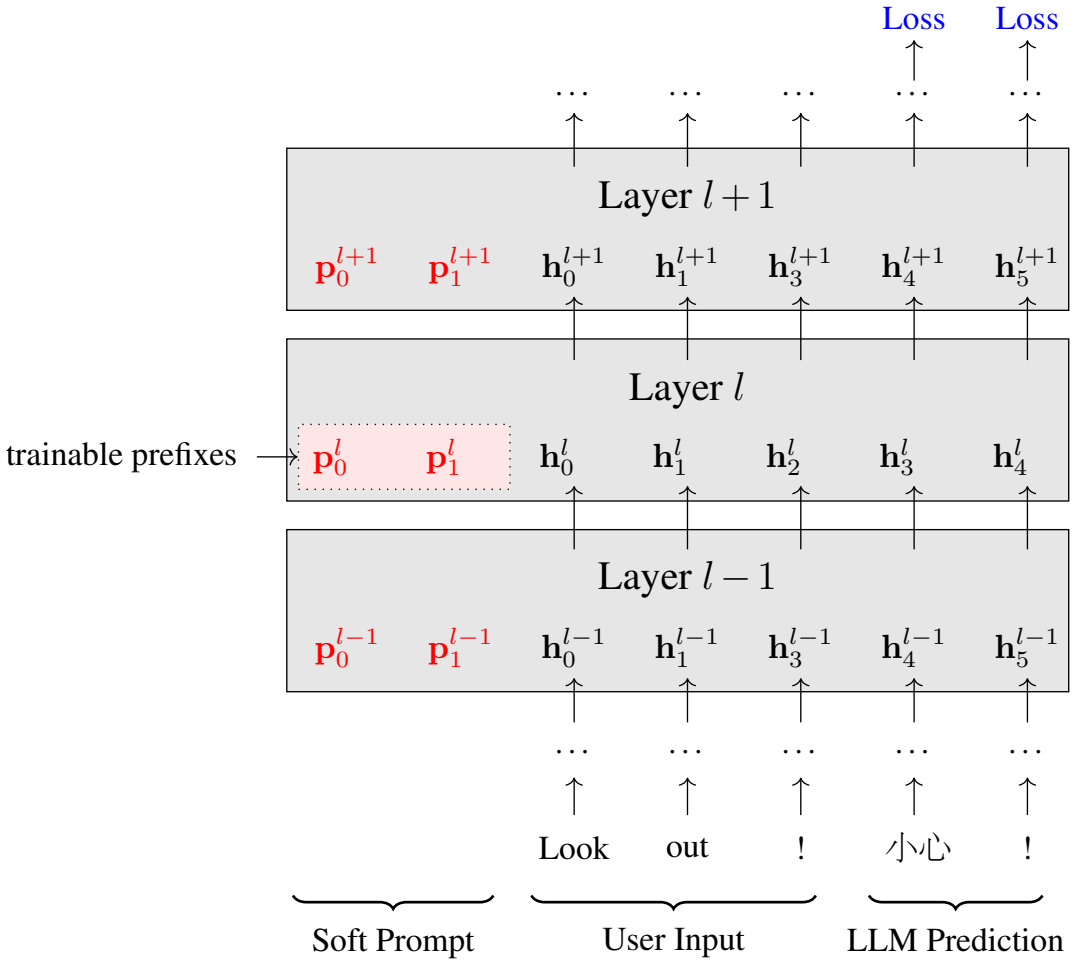
$$\begin{aligned} \overline{\mathbf{H}}^{l+1} &= \text{Layer}(\mathbf{H}^l)[-m-1:] \\ &= \mathbf{h}_0^{l+1} \mathbf{h}_1^{l+1} \cdots \mathbf{h}_m^{l+1}, \end{aligned} \tag{9.22}$$

trong đó $[-m-1:]$ ký hiệu phép cắt lát lấy ra $m+1$ phần tử cuối của một chuỗi. Cho $\overline{\mathbf{H}}^{l+1}$, đầu vào

của tầng tiếp theo có thể được biểu diễn cùng dạng với Phương trình (9.21):

$$\begin{aligned}\mathbf{H}^{l+1} &= \mathbf{p}_0^{l+1}\, \mathbf{p}_1^{l+1}\, \cdots\, \mathbf{p}_n^{l+1}\, \overline{\mathbf{H}}^{l+1} \\ &= \mathbf{p}_0^{l+1}\, \mathbf{p}_1^{l+1}\, \cdots\, \mathbf{p}_n^{l+1} \mathbf{h}_0^{l+1} \mathbf{h}_1^{l+1} \cdots \mathbf{h}_m^{l+1}.\end{aligned} \tag{9.23}$$

Ở đây mỗi $\mathbf{p}_i \in \mathbb{R}^d$ có thể được xem là một tham số học được. Trong quá trình huấn luyện, $\mathbf{p}_0^l \mathbf{p}_1^l \cdots \mathbf{p}_n^l$ được huấn luyện như thường lệ, còn các tham số của mô hình Transformer ban đầu được giữ cố định.



Hình 9.5: Minh họa tinh chỉnh tiền tố cho một nhiệm vụ dịch (Look out! → 小心!). Ở mỗi tầng, ta thêm hai tiền tố $\mathbf{p}_0^l$ và $\mathbf{p}_1^l$ vào đầu. LLM được huấn luyện để cực tiểu hóa mất mát trên các dự đoán khi cho trước đầu vào. Trong quá trình này, chỉ các tiền tố được tối ưu hóa còn các tham số còn lại được giữ cố định. Vì vậy, mô hình có thể thích nghi với nhiệm vụ chỉ với một số ít tham số huấn luyện được. Khi suy luận, LLM làm việc với các tiền tố đã tối ưu hóa và có thể thực hiện nhiệm vụ mà không cần prompt cứng tường minh.

Hình 9.5 minh họa tinh chỉnh tiền tố cho một nhiệm vụ dịch. Ở đây, chỉ các vectơ tiền tố $\mathbf{p}_0^l$ và $\mathbf{p}_1^l$ được cập nhật thông qua gradient lan truyền từ tầng đầu ra (tức bản dịch tiếng Trung). Bằng cách điều chỉnh các vectơ này cho nhiệm vụ dịch, mô hình thích nghi tương ứng. Điều này khiến $\mathbf{p}_0^l$ và $\mathbf{p}_1^l$ đóng vai trò như các prompt "kích hoạt" LLM thực hiện nhiệm vụ mà không cần prompt đầu vào tường minh như "Dịch câu sau từ tiếng Anh sang tiếng Trung". Khi kiểm tra, ta gắn $\mathbf{p}_0^l$ và $\mathbf{p}_1^l$ đã tối ưu hóa vào trước

tầng, và LLM sẽ dịch câu đầu vào. Lưu ý rằng tinh chỉnh tiền tố đưa vào thêm $L \times n \times d$ tham số, trong đó L là số tầng, n là số tiền tố, và d là số chiều của mỗi tiền tố. Tuy nhiên, con số này nhỏ hơn rất nhiều so với tổng số tham số của LLM, khiến quá trình tinh chỉnh rất hiệu quả.

Chú giải của người dịch: Đọc Hình 9.5 và ước lượng số tham số

Hình vẽ ba tầng liên tiếp ($l-1, l, l+1$) của Transformer. Ở mỗi tầng, bên trái có hai ô màu đỏ $\mathbf{p}_0, \mathbf{p}_1$ là *tiền tố huấn luyện được* (*trainable prefixes*) — phần *Soft Prompt*; tiếp theo là các vectơ $\mathbf{h}$ của câu người dùng "Look out!" (*Coi chừng!*) — phần *User Input*; cuối cùng là bản dịch "小心!" do LLM dự đoán (*LLM Prediction*), nơi tính mất mát (*Loss*). Chỉ số trên l là "thuộc tầng thứ l", không phải số mũ. Phép cắt $[-m-1:]$ trong Phương trình (9.22) nghĩa là bỏ đi các vị trí của tiền tố ở đầu ra, chỉ giữ $m+1$ vị trí ứng với token thật, rồi tầng sau lại gắn bộ tiền tố riêng của nó. Ví dụ số: một LLM có $L = 32$ tầng, mỗi vectơ $d = 4096$ chiều, dùng $n = 10$ tiền tố mỗi tầng thì chỉ thêm $32 \times 10 \times 4096 \approx 1{,}3$ triệu tham số — rất nhỏ so với khoảng 7 tỉ tham số của cả mô hình (chưa tới 0,02%).

Mặc dù tinh chỉnh tiền tố đơn giản, nó vẫn đòi hỏi phải sửa đổi LLM. Thay vào đó, tách prompt mềm khỏi LLM cho phép ta giữ nguyên kiến trúc mô hình ban đầu, giúp việc triển khai trên nhiều nhiệm vụ khác nhau hiệu quả hơn mà không cần điều chỉnh mô hình lõi. Một phương pháp như vậy là tinh chỉnh prompt (*prompt tuning*) [Lester và cộng sự, 2021]. Giống như tinh chỉnh tiền tố, tinh chỉnh prompt đưa vào các vectơ huấn luyện được để LLM có thể thích nghi với nhiệm vụ cho trước bằng cách điều chỉnh các vectơ này. Tuy nhiên, tinh chỉnh prompt khác ở chỗ nó chỉ thay đổi tầng nhúng.

Nhớ lại rằng trong LLM, mỗi token đầu vào z_i được biểu diễn bằng một vectơ nhúng $\mathbf{e}_i$. Các vectơ nhúng này thường được học thông qua một mô hình nhúng token, rồi được dùng làm đầu vào thực sự cho LLM, thay cho các token được biểu diễn bằng ký hiệu. Trong tinh chỉnh prompt, một số vectơ nhúng giả (*pseudo embedding*) $\mathbf{p}_0 \cdots \mathbf{p}_n$ được thêm vào đầu chuỗi vectơ nhúng token. Vì vậy đầu vào thực sự của LLM có thể được biểu diễn là:

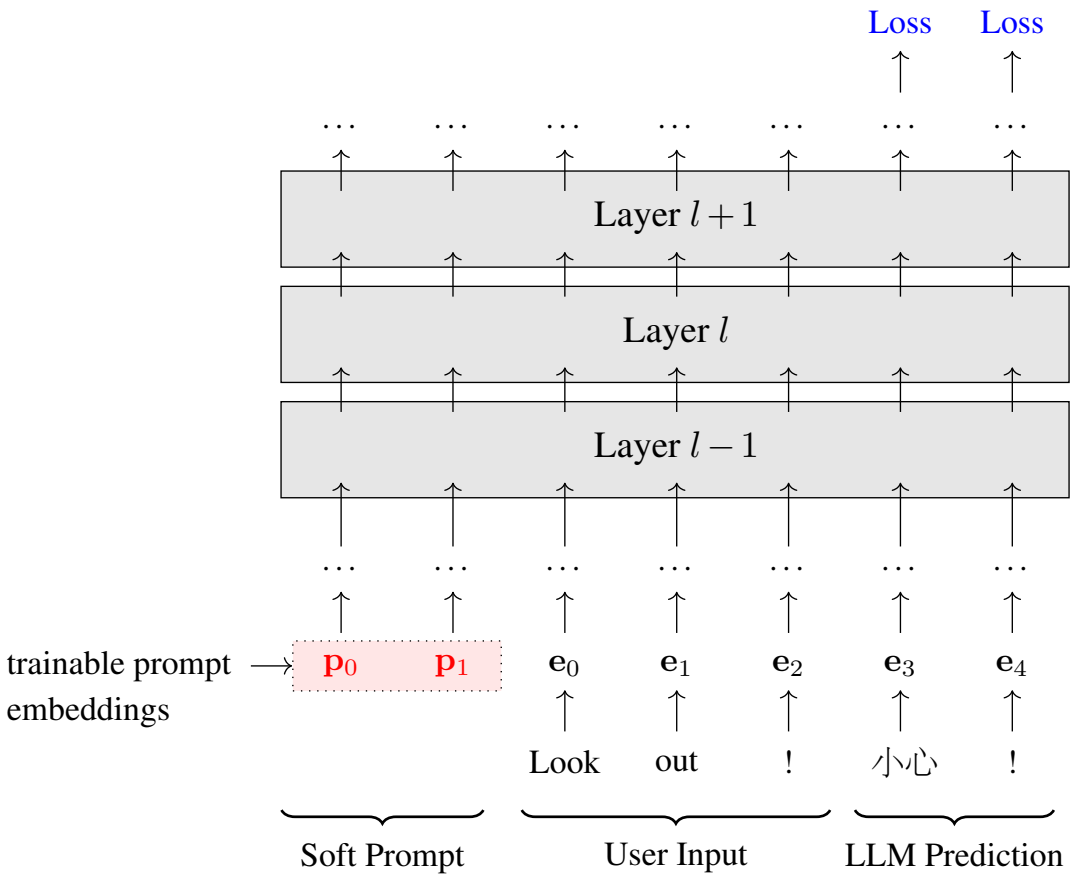
$$\underbrace{\mathbf{p}_0\, \mathbf{p}_1\, \cdots\, \mathbf{p}_n}_{\text{huấn luyện được}}\ \underbrace{\mathbf{e}_0\, \mathbf{e}_1\, \cdots\, \mathbf{e}_m}_{\text{vectơ nhúng token}}. \tag{9.24}$$

Lưu ý rằng một vectơ nhúng giả không cần tương ứng với bất kỳ token nào trong ngôn ngữ tự nhiên. Thay vào đó, các vectơ nhúng này có thể được xem là "vectơ nhúng prompt mềm" dùng để điều kiện hóa LLM. Bằng cách huấn luyện các vectơ nhúng prompt mềm trên dữ liệu đặc thù cho nhiệm vụ, chúng học cách tương tác một cách thích nghi với các vectơ nhúng token $\mathbf{e}_0 \cdots \mathbf{e}_m$ và dẫn dắt hành vi của LLM. Vì tinh chỉnh prompt không thay đổi các tham số nền tảng của LLM đã tiền huấn luyện, nó được coi là một phương pháp tinh chỉnh nhẹ và hiệu quả, giúp cải thiện hiệu năng trên nhiệm vụ cụ thể trong khi vẫn duy trì khả năng tổng quát hóa của mô hình. Xem Hình 9.6 để thấy minh họa về tinh chỉnh prompt.

Chú giải của người dịch: Tinh chỉnh tiền tố và tinh chỉnh prompt khác nhau ở đâu?

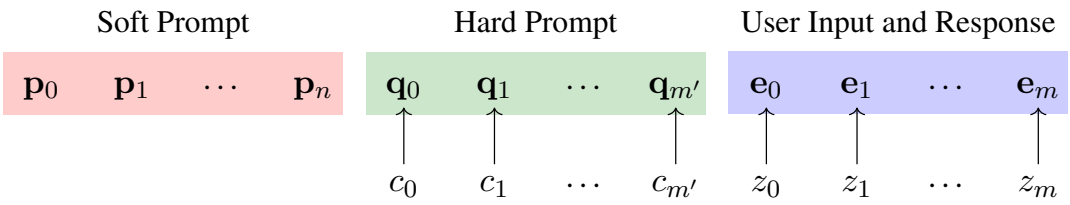
Cả hai đều "đóng băng" LLM và chỉ học vài vectơ prompt mềm. Tinh chỉnh tiền tố (Hình 9.5) chèn vectơ tiền tố riêng vào *mọi* tầng. Tinh chỉnh prompt (Hình 9.6) chỉ chèn vài vectơ $\mathbf{p}_0, \mathbf{p}_1$ (nhãn *trainable prompt embeddings* — vectơ nhúng prompt huấn luyện được) ở *tầng nhúng đầu vào*, ngay cạnh vectơ nhúng của các token thật; các tầng phía trên giữ nguyên. Vì thế với tinh chỉnh prompt, ta có thể dùng chung một LLM cho nhiều nhiệm vụ, chỉ cần đổi vài vectơ $\mathbf{p}$ — giống như dùng chung một chiếc máy ảnh nhưng thay "bộ lọc" cho từng kiểu chụp.

Vì $\mathbf{p}_0\, \mathbf{p}_1\, \cdots\, \mathbf{p}_n$ tự nó là một chuỗi, ta có thể dùng các mô hình chuỗi để biểu diễn nó tốt hơn. Ví dụ, một mô hình Transformer có thể mã hóa chuỗi này, và biểu diễn thu được sau đó có thể được dùng làm



Hình 9.6: Minh họa tinh chỉnh prompt cho một nhiệm vụ dịch (Look out! → 小心!). Thay vì dùng các prompt văn bản cố định, prompt mềm là các vectơ nhúng học được, được thêm vào đầu chuỗi vectơ nhúng. Trong quá trình tinh chỉnh, chỉ các vectơ nhúng prompt này được tối ưu hóa để thích nghi LLM với nhiệm vụ cho trước một cách hiệu quả. Sau khi đã tối ưu hóa, các vectơ nhúng prompt được dùng để chỉ dẫn LLM thực hiện nhiệm vụ khi có dữ liệu mới.

đầu vào cho LLM. Nói cách khác, ta có thể xây dựng một mô hình bổ sung để mã hóa prompt mềm. Một cách khác để cải thiện prompting là kết hợp prompt mềm và prompt cứng, qua đó tận dụng ưu điểm của cả hai loại [Liu và cộng sự, 2023c]. Trong chuỗi vectơ nhúng, ta có thể sắp xếp hoặc xen kẽ các prompt này. Điều này dẫn đến các mẫu prompt khác nhau. Ví dụ, xem Hình 9.7 để thấy một mẫu đơn giản kết hợp cả hai loại prompt.



Hình 9.7: Một mẫu prompt kết hợp prompt mềm (*Soft Prompt*), prompt cứng (*Hard Prompt*) và đầu vào của người dùng (*User Input and Response* — đầu vào của người dùng và câu trả lời). Ở đây, $c_0 \cdots c_{m'}$ ký hiệu prompt cứng và $\mathbf{q}_0 \cdots \mathbf{q}_{m'}$ ký hiệu chuỗi vectơ nhúng tương ứng.

Ở đây ta đã xem xét các phương pháp chèn prompt mềm vào LLM. Nhưng ta bỏ qua chi tiết huấn luyện các prompt mềm này và giả định rằng bạn đọc đã quen với quá trình học có giám sát tiêu chuẩn, tức là cực đại hóa hàm hợp lý (*likelihood*) của đầu ra đúng khi cho trước đầu vào mô hình. Thực ra, việc học prompt mềm có thể liên hệ với nhiều vấn đề trong tinh chỉnh LLM. Ví dụ, nếu coi nó là một bài toán nén ngữ cảnh, ta có thể áp dụng các phương pháp chưng cất tri thức đã mô tả ở trên. Trong công trình của Mu và cộng sự [2024], prompt được nén và biểu diễn thành một vài token giả (*pseudo token*), được gắn vào mỗi chuỗi đầu vào. Các vectơ nhúng của những token giả này được tối ưu hóa để bắt chước dự đoán của một mô hình được prompt theo cách tiêu chuẩn. Nói cách khác, tri thức prompting được chưng cất từ một mô hình thầy vào các token giả.

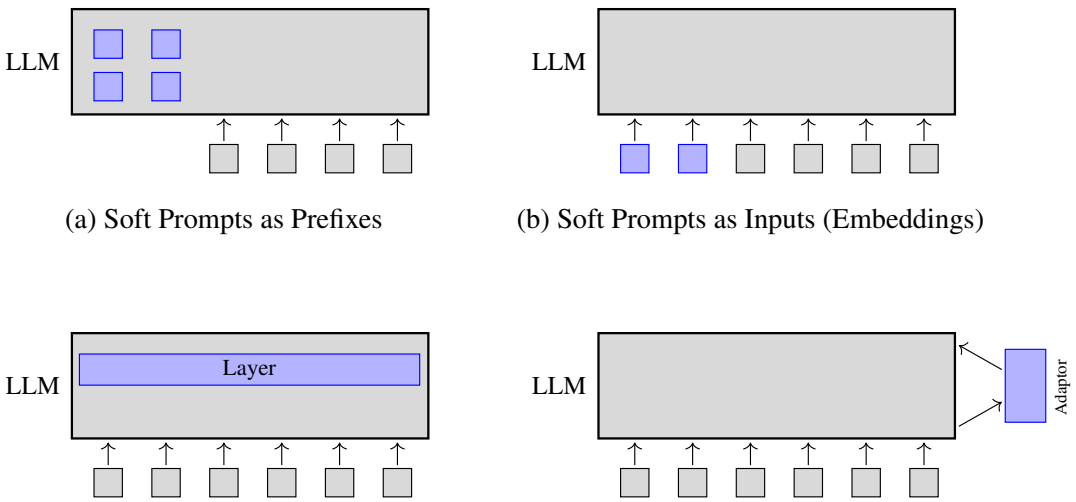
Nói rộng ra, nhiều phương pháp tinh chỉnh hiệu quả tham số có thể được coi là học một dạng prompt mềm nào đó [Lialin và cộng sự, 2023]. Khi ta tinh chỉnh một phần của LLM cho một nhiệm vụ, về bản chất quá trình này có thể được xem như việc "tiêm" thông tin prompting liên quan đến nhiệm vụ vào phần cụ thể đó của mô hình. Một cách tiếp cận được dùng rộng rãi khác cho tinh chỉnh hiệu quả tham số là thêm một tầng thích nghi (*adaptor layer*) vào giữa các tầng sẵn có của mô hình. Cách tiếp cận này cho phép ta chỉ tinh chỉnh tầng thích nghi trên các nhiệm vụ cụ thể mà không thay đổi kiến trúc nền tảng hay huấn luyện lại toàn bộ mô hình. Theo nghĩa này, các tầng thích nghi có thể được xem như prompt mềm mã hóa thông tin prompting và thông tin liên quan đến nhiệm vụ, tương tác với LLM ban đầu để giúp nó thích nghi. Tóm lại, Hình 9.8 so sánh các phương pháp khác nhau để sử dụng prompt mềm trong LLM.

3. Học prompt mềm bằng nén

Một cách tiếp cận khác để học prompt mềm là phát biểu nó thành một bài toán nén. Làm ví dụ đơn giản, xét bài toán xấp xỉ một ngữ cảnh dài bằng một biểu diễn liên tục [Wingate và cộng sự, 2022]. Giả sử ta có một đầu vào của người dùng $\mathbf{z}$ và ngữ cảnh của nó là $\mathbf{c}$ (chẳng hạn các chỉ dẫn và ví dụ minh họa dài). Giờ ta muốn học một biểu diễn nén của ngữ cảnh, ký hiệu là σ, sao cho dự đoán có điều kiện trên $\mathbf{z}$ và σ càng gần càng tốt với dự đoán có điều kiện trên $\mathbf{z}$ và $\mathbf{c}$. Mục tiêu này có thể được phát biểu một cách không chặt chẽ là

$$\hat{\sigma} \;=\; \arg\min_{\sigma} s(\hat{\mathbf{y}}, \hat{\mathbf{y}}_\sigma), \tag{9.25}$$

trong đó $\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{c}, \mathbf{z})$ và $\hat{\mathbf{y}}_\sigma = \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\sigma, \mathbf{z})$ lần lượt là dự đoán của LLM khi cho ngữ



(a) Soft Prompts as Prefixes

(b) Soft Prompts as Inputs (Embeddings)

(c) Fine-tuning Parts of the Model

(d) Fine-tuning the Adaptor

Hình 9.8: Minh họa việc sử dụng prompt mềm trong LLM. Ở đây các prompt mềm điều chỉnh được được vẽ màu xanh, còn các thành phần có tham số cố định trong quá trình tinh chỉnh được vẽ màu xám. Trong hình con (a) (*Soft Prompts as Prefixes* — prompt mềm làm tiền tố), prompt mềm là các tiền tố gắn vào mỗi tầng của LLM. Trong hình con (b) (*Soft Prompts as Inputs (Embeddings)* — prompt mềm làm đầu vào, dạng vectơ nhúng), prompt mềm được dùng làm vectơ nhúng đầu vào cho LLM. Trong các hình con (c) (*Fine-tuning Parts of the Model* — tinh chỉnh một phần mô hình, ở đây là một tầng, *Layer*) và (d) (*Fine-tuning the Adaptor* — tinh chỉnh bộ thích nghi, *Adaptor*), prompt mềm được hiểu theo nghĩa rộng là các thành phần của mô hình được tinh chỉnh để thích nghi với nhiệm vụ.

cảnh đầy đủ và ngữ cảnh đã nén. Hàm $s(\cdot,\cdot)$ đo mức độ khác biệt giữa hai dự đoán.

Một khung tổng quát để đạt được điều này là chưng cất tri thức, trong đó $\hat{\mathbf{y}}$ và $\hat{\mathbf{y}}_\sigma$ lần lượt có thể được xem là dự đoán của mô hình thầy và mô hình trò. Cách hình thức hóa này nối thảo luận của ta với bài toán chưng cất ngữ cảnh đã bàn ở trên. Mục tiêu huấn luyện có thể được định nghĩa tương tự như Phương trình (9.16) và (9.17). Chẳng hạn, một mục tiêu hợp lý cực đại (*maximum likelihood*) đơn giản được cho bởi

$$\hat{\sigma} \;=\; \arg\max_{\sigma} \log \Pr(\hat{\mathbf{y}}|\sigma,\mathbf{z}). \tag{9.26}$$

Ngoài ra, ta có thể cực tiểu hóa độ phân kỳ KL giữa các phân phối đầu ra, cho ta

$$\hat{\sigma} \;=\; \arg\min_{\sigma} \mathrm{D}_{\mathrm{KL}}\big(\Pr(\cdot|\mathbf{c},\mathbf{z}) \,\|\, \Pr(\cdot|\sigma,\mathbf{z})\big). \tag{9.27}$$

Khác biệt so với các mô hình trong Phương trình (9.16) và (9.17) là ở đây ngữ cảnh đã nén được biểu diễn dưới dạng các vectơ giá trị thực (gọi là **vectơ nhúng prompt** (*prompt embedding*)), chứ không phải các token thông thường. Bằng cách áp dụng các phương pháp trên, ta chưng cất ngữ cảnh từ chuỗi token $\mathbf{c}$ vào các vectơ nhúng σ. Lưu ý rằng mô hình thầy $\Pr(\cdot|\mathbf{c},\mathbf{z})$ và mô hình trò $\Pr(\cdot|\sigma,\mathbf{z})$ có thể không có cùng kiến trúc hay cùng thiết lập mô hình. Trong thực tế, ta thường mong mô hình thầy mạnh hơn, còn mô hình trò nhỏ hơn và hiệu quả hơn.

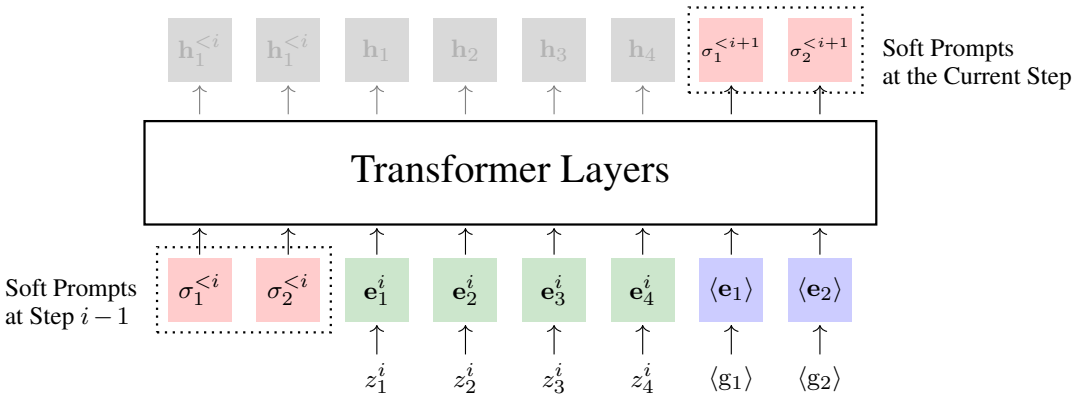
Chú giải của người dịch: Nén ngữ cảnh giống "tóm tắt bài học"

Ngữ cảnh $\mathbf{c}$ có thể dài hàng nghìn token (chỉ dẫn + nhiều ví dụ). Ta muốn thay nó bằng σ — chỉ vài vectơ số — mà mô hình vẫn trả lời như khi đọc đủ $\mathbf{c}$. Giống như thay vì mang cả cuốn sách vào phòng thi, ta mang một "tờ tóm tắt" nhỏ nhưng vẫn làm bài tốt như nhau. Điểm khác với Phương trình (9.16)–(9.17) là ở đó ta học *tham số θ của mô hình trò*, còn ở đây ta học chính *các vectơ σ* (mô hình giữ nguyên).

Mặc dù nén toàn bộ ngữ cảnh thành các biểu diễn liên tục là một cách tiếp cận đơn giản để học prompt mềm, nó đòi hỏi một mô hình thầy có thể xử lý các chuỗi đầu vào dài. Tuy nhiên, trong nhiều trường hợp, ngữ cảnh dài đến mức việc áp dụng một LLM quá tốn kém hoặc không khả thi. Mô hình hóa các chuỗi đầu vào dài có thể được xếp vào họ rộng các phương pháp hiệu quả cho LLM ngữ cảnh dài. Nhiều kỹ thuật đã được phát triển để giải quyết vấn đề này. Ví dụ, có thể dùng một bộ nhớ đệm KV (*KV cache*) kích thước cố định để lưu thông tin quá khứ ở mỗi bước trong quá trình lập luận. Các kiến trúc Transformer hiệu quả và LLM ngữ cảnh dài đã được bàn rộng rãi trong cuốn sách này. Để có thảo luận chi tiết hơn về các chủ đề này, bạn đọc quan tâm có thể tham khảo Chương 6 và 8.

Cũng có những phương pháp được thiết kế riêng để nén các chuỗi dài thành prompt mềm. Ở đây ta xét phương pháp của Chevalier và cộng sự [2023] làm ví dụ. Ý tưởng cơ bản là học dần prompt mềm bằng cách nén chuỗi đầu vào thành một biểu diễn có kích thước cố định. Để đơn giản, ta dùng $\mathbf{z}$ để ký hiệu toàn bộ chuỗi đầu vào cần nén. Cho một chuỗi đầu vào dài, trước hết ta chia nó thành một số đoạn $\{\mathbf{z}^1,\cdots,\mathbf{z}^K\}$. Sau đó ta xử lý lần lượt các đoạn này, mỗi lần sinh ra một biểu diễn của phần đầu vào đã xử lý tới lúc đó, ký hiệu là $\sigma^{<i+1}$. Để làm việc này, ta đưa vào một vài token tóm tắt (*summary token*) $\langle \mathrm{g}_1\rangle,\cdots,\langle \mathrm{g}_\kappa\rangle$. Ở mỗi bước, ta lấy một đoạn $\mathbf{z}^i = z_1^i\cdots z_{m_i}^i$, cùng với biểu diễn nén trước đó $\sigma^{<i}$ và các token tóm tắt $\langle \mathrm{g}_1\rangle,\cdots,\langle \mathrm{g}_\kappa\rangle$ làm đầu vào, và dùng một LLM để tạo ra chuỗi biểu diễn ẩn tương ứng ở tầng Transformer cuối cùng. Một ví dụ của quá trình này được minh họa trong Hình 9.9.

Ở đây $\sigma^{<i}$ về bản chất là một bộ nhớ. Mô hình hoạt động theo kiểu RNN. Mỗi lần ta lấy một đoạn và cập nhật bộ nhớ này bằng cách mã hóa cả trạng thái bộ nhớ trước đó lẫn đoạn đó. Vì vậy, $\sigma^{<i}$ được tạo ra ở đoạn cuối cùng là một biểu diễn của toàn bộ chuỗi ngữ cảnh. Mô hình Transformer dùng để học các biểu diễn này có thể là một LLM tiêu chuẩn, nhưng ta cần tinh chỉnh nó để thích nghi với nhiệm vụ biểu diễn ngữ cảnh này.



Hình 9.9: Minh họa việc nén một đoạn ngữ cảnh thành prompt mềm ($\kappa = 2$ và $m_i = 4$). Đầu vào của LLM gồm các prompt mềm từ bước trước ($\sigma_1^{<i}$ và $\sigma_2^{<i}$), các token của đoạn (z_1, z_2, z_3 và z_4), và các token tóm tắt ($\langle g_1 \rangle$ và $\langle g_2 \rangle$). Với các đầu vào này, LLM hoạt động như thường lệ. Sau đó ta trích các đầu ra ở tầng Transformer cuối cùng tương ứng với các token tóm tắt. Các đầu ra này có thể được xem là các prompt mềm tích lũy được cho tới đoạn này.

Chú giải của người dịch: Đọc Hình 9.9: "cuốn chiếu" từng đoạn

Giả sử văn bản dài 4000 token, chia thành $K = 1000$ đoạn mỗi đoạn $m_i = 4$ token. Ở bước i, LLM đọc: 2 vectơ "ghi nhớ" từ bước trước ($\sigma_1^{<i}, \sigma_2^{<i}$, ô đỏ bên trái, nhãn *Soft Prompts at Step* $i-1$), 4 token của đoạn thứ i (ô xanh lá), và 2 token tóm tắt đặc biệt $\langle g_1 \rangle, \langle g_2 \rangle$ (ô tím). Hai đầu ra ở vị trí token tóm tắt (ô đỏ phía trên, nhãn *Soft Prompts at the Current Step*) trở thành "ghi nhớ" mới $\sigma^{<i+1}$. Như vậy dù văn bản dài bao nhiêu, "bộ nhớ" luôn chỉ gồm $\kappa = 2$ vectơ — giống một học sinh đọc từng trang sách và sau mỗi trang chỉ cập nhật lại hai dòng ghi chú tóm tắt. Đây chính là cách mạng RNN mang trạng thái ẩn từ bước này sang bước sau.

Lưu ý rằng ở đây ta đơn giản coi *prompt* và *ngữ cảnh* là những thuật ngữ tương tự, mặc dù chúng không giống nhau. Dù ta dùng thuật ngữ *prompt* theo nghĩa rộng để bao gồm cả thông tin ngữ cảnh cung cấp cho mô hình, ta thường có thể xem nó như một loại ngữ cảnh. Từ góc nhìn này, các phương pháp bàn ở đây có thể được áp dụng cho các bài toán nén văn bản nói chung.

9.3.3 Rút gọn độ dài prompt

Mặc dù prompt mềm cung cấp các biểu diễn ẩn dày đặc, chúng nhìn chung không diễn giải trực tiếp được. Việc thiếu khả năng diễn giải này khiến người dùng khó hiểu prompt ảnh hưởng thế nào tới đầu ra của LLM. Hơn nữa, việc sửa đổi prompt mềm thường đòi hỏi tinh chỉnh bổ sung, điều này hạn chế tính hữu dụng của chúng trong các môi trường biến động, nơi prompt thay đổi thường xuyên.

Một cách tiếp cận khác để cải thiện hiệu quả của prompt là đơn giản hóa chính văn bản prompt. Khác với prompt mềm, prompt văn bản đã đơn giản hóa vẫn diễn giải được và có thể được người dùng sửa đổi trực tiếp. Ví dụ, dưới đây là một prompt để trả lời các câu hỏi về y tế và tài chính.

Nhiệm vụ là xây dựng một mô hình ngôn ngữ có khả năng hiểu và trả lời các câu hỏi của người dùng trong nhiều lĩnh vực khác nhau, đặc biệt chú trọng vào y tế và tài chính. Xét đến phạm vi rộng của các câu hỏi có thể có, từ những chi tiết cụ thể của chẩn đoán y khoa đến những sắc thái của các quy định tài chính, mô hình phải đảm bảo hiểu một cách toàn diện và trả lời chính xác.

Câu hỏi:
Đâu là những thực hành tốt nhất khi sử dụng trí tuệ nhân tạo trong chẩn đoán các bệnh tim mạch?

___

Ta có thể đơn giản hóa phần mô tả nhiệm vụ bằng cách xóa các phần không quan trọng.

> *Nhiệm vụ là xây dựng một mô hình ngôn ngữ có khả năng hiểu và trả lời các câu hỏi của người dùng ~~trong nhiều lĩnh vực khác nhau, đặc biệt chú trọng vào~~ về y tế và tài chính. ~~Xét đến phạm vi rộng của các câu hỏi có thể có, từ những chi tiết cụ thể của chẩn đoán y khoa đến những sắc thái của các quy định tài chính.~~ Mô hình phải đảm bảo hiểu một cách toàn diện và trả lời chính xác.*

Ta cũng có thể diễn đạt lại nó thành một văn bản ngắn hơn.

> *Nhiệm vụ là xây dựng một mô hình ngôn ngữ tập trung vào y tế và tài chính, có khả năng hiểu và trả lời chính xác nhiều loại câu hỏi của người dùng.*

Bài toán này có thể được xem là một nhiệm vụ NLP tiêu chuẩn — đơn giản hóa văn bản (*text simplification*). Vì vậy, các phương pháp dùng cho đơn giản hóa văn bản nhìn chung áp dụng được và không chỉ giới hạn ở việc đơn giản hóa prompt. Có nhiều cách để đạt được điều này. Một cách đơn giản là định nghĩa một số quy tắc kinh nghiệm (*heuristics*) và xác định các từ thừa có thể loại bỏ mà không làm mất thông tin thiết yếu. Ví dụ, ta có thể xem xét từng token trong một chuỗi về mức độ đóng góp của nó vào nghĩa tổng thể và loại bỏ những token đóng góp rất ít [Li và cộng sự, 2023c; Jiang và cộng sự, 2023b]. Một phương pháp khác là phát biểu bài toán thành một nhiệm vụ chuỗi-sang-chuỗi (*sequence-to-sequence*). Cho dữ liệu có nhãn về đơn giản hóa văn bản, ta có thể huấn luyện một mô hình bộ mã hóa (*encoder*)–bộ giải mã (*decoder*) để biến đổi mỗi văn bản đầu vào thành dạng đơn giản hóa của nó. Ngoài ra, vì nhiều LLM đã được tinh chỉnh và căn chỉnh để thực hiện các nhiệm vụ đơn giản hóa văn bản, dùng các mô hình này để đơn giản hóa prompt là việc đơn giản. Ví dụ, ta có thể prompt một LLM đơn giản hóa một prompt trong khi thỏa mãn các ràng buộc như giới hạn độ dài.

Chú giải của người dịch: Vì sao prompt ngắn lại có lợi?

Chi phí tính toán (và cả tiền phải trả khi dùng LLM qua API) tăng theo số token đầu vào; với cơ chế tự chú ý, khối lượng tính toán còn tăng nhanh hơn tỷ lệ thuận với độ dài. Nếu một prompt 60 token được rút còn 25 token mà câu trả lời vẫn tốt như cũ, và prompt được dùng một triệu lần mỗi ngày, ta tiết kiệm được 35 triệu token xử lý mỗi ngày. Khác với prompt mềm, prompt rút gọn vẫn là chữ nên con người đọc và sửa được.

9.4 Tóm tắt

Trong chương này, chúng ta đã bàn về nhiều chủ đề liên quan đến prompting LLM. Thảo luận của chúng ta tập trung chủ yếu vào hai khía cạnh:

- Làm thế nào để thiết kế các prompt cơ bản nhằm dẫn dắt dự đoán của LLM, và tinh chỉnh các prompt này để giải quyết vấn đề hiệu quả hơn (cả về chất lượng lẫn chi phí)?
- Làm thế nào để tự động hóa việc thiết kế và biểu diễn prompt?

Lời giải cho các vấn đề này bao gồm cả những thiết kế prompt tổng quát lẫn các kỹ thuật nâng cao hơn, như CoT và học prompt, vốn đã được khám phá rộng rãi trong các nghiên cứu gần đây.

Trong NLP, prompting có thể được xem là một công nghệ phát triển song hành cùng LLM, và theo một nghĩa nào đó, nó đã mở cánh cửa cho việc ứng dụng thực tế các mô hình này trong một phạm vi lĩnh vực bài toán rộng đến ấn tượng. Thực ra, nếu mở rộng khái niệm prompt ở một mức độ nào đó, ta có thể truy nguyên nó về những ngày đầu của học máy và NLP. Ví dụ, nhiều hệ thống NLP dùng các đặc trưng và khuôn mẫu thủ công để "prompt" các nhiệm vụ cụ thể. Hãy tưởng tượng việc xây dựng một đặc trưng cho biết một văn bản là trang trọng hay thân mật. Ta có thể đưa đặc trưng này vào một hệ thống dịch máy (*machine translation*) để bản dịch được điều kiện hóa theo loại của văn bản đầu vào.

Việc sử dụng rộng rãi khái niệm prompt hiện đại bắt đầu cùng với sự trỗi dậy của các mô hình tiền huấn luyện lớn trong lĩnh vực NLP. Ban đầu, các mô hình này, như BERT, được thích nghi với các nhiệm vụ hạ nguồn cụ thể chủ yếu thông qua tinh chỉnh. Tuy nhiên, các nhà nghiên cứu sớm phát hiện ra rằng bằng cách thiết kế các "prompt" cụ thể — thêm một số từ hoặc câu nhất định vào đầu vào — các mô hình có thể được kích hoạt để đáp ứng các nhiệm vụ cụ thể mà không cần tinh chỉnh nhiều. Điều này thúc đẩy cộng đồng NLP phát triển và áp dụng các mô hình nền tảng (*foundation model*) phổ quát có thể được prompt để giải quyết nhiều nhiệm vụ khác nhau mà không thay đổi kiến trúc nền tảng và quy trình tiền huấn luyện.

Các cách tiếp cận prompting lúc đầu được thử nghiệm với các mô hình nhỏ hơn và sau đó đã thể hiện những năng lực ấn tượng với các mô hình lớn như GPT-3, vốn có thể sinh văn bản chất lượng cao để đáp lại các prompt đơn giản trong nhiều nhiệm vụ. Khi công nghệ prompting phát triển, thiết kế prompt nổi lên như một lĩnh vực nghiên cứu then chốt. Như đã bàn trong chương này, nói rộng ra nó liên quan đến việc thiết kế các prompt hiệu quả để tối đa hóa hiệu năng mô hình, bao gồm cả prompt thủ công lẫn prompt sinh tự động. Các nghiên cứu gần đây hơn đã khám phá cách nâng cao hiệu quả của prompting thông qua các kỹ thuật như học few-shot, học zero-shot và suy luận CoT, giúp LLM hoạt động hiệu quả trong nhiều tình huống khác nhau. Một thảo luận tổng quát về prompting có thể rất rộng, và chúng tôi không thể đề cập hết mọi chi tiết trong chương này. Về các kỹ thuật prompting nâng cao hơn, bạn đọc có thể tham khảo các bài tổng quan gần đây. Các chủ đề bao gồm học trong ngữ cảnh [Li, 2023; Dong và cộng sự, 2022], CoT [Chu và cộng sự, 2023; Yu và cộng sự, 2023b; Zhang và cộng sự, 2023a], prompting hiệu quả [Chang và cộng sự, 2024a], và thiết kế prompt nói chung [Liu và cộng sự, 2023d; Chen và cộng sự, 2023a].

Lưu ý rằng mặc dù lý tưởng nhất là ta muốn xây dựng các phương pháp prompting tổng quát mà không cần điều chỉnh kiến trúc và tham số mô hình, kết quả của prompting nhìn chung phụ thuộc nhiều vào chất lượng và kích thước của LLM cho trước. Với những mô hình mạnh hơn, như các LLM trực tuyến thương mại, các prompt đơn giản có thể đủ để chỉ dẫn mô hình thực hiện nhiệm vụ đúng. Trong trường hợp này, thiết kế prompt tương đối dễ, dù việc tinh chỉnh prompt cẩn thận vẫn cần thiết. Ngược lại, với những mô hình kém năng lực hơn hoặc có quy mô nhỏ hơn, ta có thể cần thiết kế prompt cẩn thận để đạt được kết quả mong muốn. Trong nhiều trường hợp, tinh chỉnh vẫn cần thiết để thích nghi mô hình với các chiến lược prompting phức tạp.

Chương 10

Căn chỉnh

Căn chỉnh (*alignment*) không phải là một khái niệm mới trong NLP, nhưng ý nghĩa của nó thay đổi theo từng lĩnh vực và theo thời gian. Trong NLP truyền thống, thuật ngữ *căn chỉnh* thường chỉ nhiệm vụ liên kết các phần tử tương ứng giữa hai tập hợp, chẳng hạn căn chỉnh các từ giữa một câu tiếng Trung và một câu tiếng Anh. Khi LLM ngày càng trở nên quan trọng trong nghiên cứu NLP, thuật ngữ này hiện được dùng rộng hơn để mô tả quá trình làm cho đầu ra của mô hình phù hợp với kỳ vọng của con người. Vấn đề mà căn chỉnh giải quyết là đầu ra của mô hình có thể không khớp với những mục tiêu hay bối cảnh cụ thể mà người dùng mong muốn. Ví dụ, các LLM đã được tiền huấn luyện có thể không làm theo được chỉ dẫn của người dùng, vì chúng không được huấn luyện để làm việc đó. Một ví dụ khác là LLM có thể sinh ra nội dung độc hại hoặc duy trì những thiên kiến vốn có trong dữ liệu huấn luyện của chúng. Điều này đặt ra những thách thức mới: phải bảo đảm đầu ra của LLM không chỉ chính xác và liên quan, mà còn đúng đắn về mặt đạo đức và không mang tính phân biệt đối xử.

Chỉ tiền huấn luyện LLM thôi có thể dẫn tới nhiều loại vấn đề căn chỉnh khác nhau. Mục tiêu cuối cùng của căn chỉnh là giải quyết hoặc giảm nhẹ các vấn đề này để bảo đảm LLM vừa hữu ích vừa an toàn. Vì LLM được huấn luyện trên một lượng dữ liệu khổng lồ, người ta có thể cho rằng với đủ dữ liệu chất lượng cao, tiền huấn luyện có thể làm LLM đủ chính xác và an toàn, và có lẽ không cần một giai đoạn căn chỉnh riêng nữa. Tuy nhiên, thực tế là gần như không thể thu thập được dữ liệu bao quát mọi nhiệm vụ hoặc thể hiện đầy đủ sở thích của con người. Điều này khiến việc đạt được căn chỉnh chỉ bằng tiền huấn luyện là rất khó. Hiện nay, căn chỉnh vẫn là một bước then chốt trong quá trình phát triển LLM.

Trong chương này, chúng ta tập trung vào các phương pháp căn chỉnh cho LLM. Ta bắt đầu bằng việc bàn về các khái niệm chung của căn chỉnh. Sau đó, ta xem xét hai cách tiếp cận được dùng rộng rãi: **căn chỉnh theo chỉ dẫn** (*instruction alignment*) và **căn chỉnh theo sở thích của con người** (*human preference alignment*). Cách thứ nhất dùng các kỹ thuật tinh chỉnh có giám sát (*supervised fine-tuning*) để hướng dẫn LLM sinh ra đầu ra bám sát chỉ dẫn của người dùng. Cách thứ hai thường dựa vào các kỹ thuật học tăng cường (*reinforcement learning*), trong đó mô hình được huấn luyện dựa trên phản hồi của con người. Dù hai phương pháp này xuất phát từ những mục tiêu khác nhau, chúng thường được dùng kết hợp để phát triển các LLM được căn chỉnh tốt.

Chú giải của người dịch: "Căn chỉnh" ở đây nghĩa là gì?

Hãy hình dung một LLM vừa tiền huấn luyện xong như một người đã đọc gần hết sách báo trên đời nhưng chưa từng đi làm: biết rất nhiều, nhưng chưa biết *làm theo yêu cầu* của người khác, và có thể nói ra cả những điều thiếu lịch sự hay sai lệch mà nó đã đọc được. "Căn chỉnh" là quá trình "uốn nắn" mô hình sao cho hành vi của nó *khớp* với điều con người muốn: hiểu và làm đúng yêu cầu, trả lời hữu ích, trung thực và không gây

hại. Chữ *alignment* vốn có nghĩa là "thẳng hàng" — ở đây là làm cho mục tiêu của mô hình "thẳng hàng" với mục tiêu của con người.

10.1 Tổng quan về căn chỉnh LLM

Căn chỉnh có thể được thực hiện theo nhiều cách khác nhau. Ta cần nhiều phương pháp khác nhau cho việc căn chỉnh LLM vì bản thân bài toán này phức tạp và nhiều mặt, đòi hỏi kết hợp nhiều chiến lược kỹ thuật. Ở đây ta xét ba cách tiếp cận được dùng rộng rãi để căn chỉnh LLM.

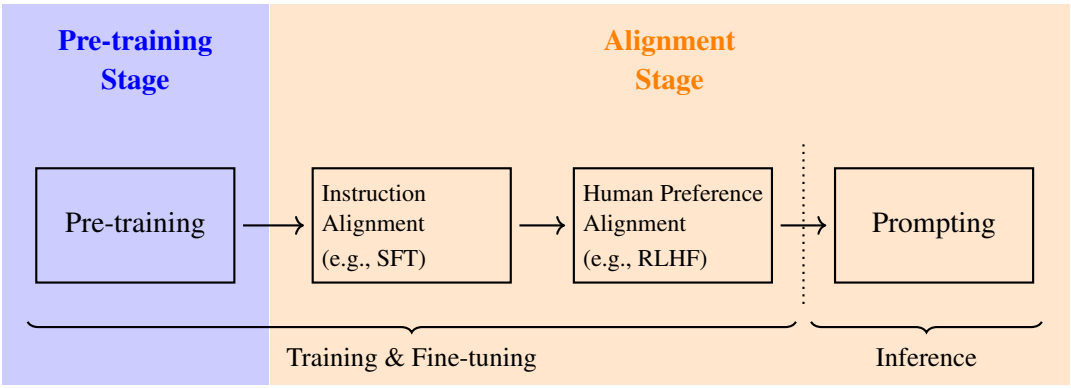
Cách tiếp cận thứ nhất là tinh chỉnh LLM bằng dữ liệu có gán nhãn. Cách này đơn giản vì nó chỉ mở rộng quá trình tiền huấn luyện để thích nghi LLM với các nhiệm vụ cụ thể. Một ví dụ là **tinh chỉnh có giám sát (SFT)** (*supervised fine-tuning*), trong đó LLM được huấn luyện thêm trên một tập dữ liệu gồm các chỉ dẫn dành riêng cho từng nhiệm vụ, ghép cặp với đầu ra mong đợi của chúng. Tập dữ liệu SFT thường nhỏ hơn nhiều so với tập huấn luyện ban đầu, nhưng dữ liệu này có tính chuyên biệt cao. Kết quả của SFT là LLM có thể học cách thực hiện nhiệm vụ dựa trên chỉ dẫn của người dùng. Đó có thể là những nhiệm vụ đã gặp trong SFT, hoặc những nhiệm vụ mới nhưng tương tự. Ví dụ, tinh chỉnh LLM trên một tập các cặp câu hỏi–câu trả lời giúp mô hình trả lời được các câu hỏi cụ thể, kể cả những câu hỏi không có trực tiếp trong tập dữ liệu SFT. Phương pháp này đặc biệt hữu ích khi quan hệ giữa đầu vào và đầu ra tương đối dễ mô tả và dữ liệu dễ gán nhãn.

Cách tiếp cận thứ hai là tinh chỉnh LLM bằng các mô hình phần thưởng (*reward model*). Một khó khăn trong căn chỉnh là các giá trị và kỳ vọng của con người rất phức tạp và khó mô tả. Trong nhiều trường hợp, chính con người cũng khó diễn đạt chính xác điều gì là đúng đắn về mặt đạo đức hay phù hợp về mặt văn hóa. Do đó, việc thu thập và gán nhãn dữ liệu tinh chỉnh không đơn giản như với SFT. Hơn nữa, căn chỉnh không chỉ đơn thuần là một nhiệm vụ khớp dữ liệu. Một số ít mẫu do con người gán nhãn thường không đủ để định nghĩa đầy đủ những hành vi phức tạp này. Thay vào đó, mô hình phải được dạy cách đánh giá xem đầu ra nào phản ánh tốt nhất sở thích của con người. Ví dụ, ta không chỉ muốn đầu ra chính xác về mặt kỹ thuật mà còn muốn nó phù hợp với kỳ vọng và giá trị của con người. Một ý tưởng là xây dựng một mô hình phần thưởng mô phỏng sự đánh giá của con người. Mô hình phần thưởng này sẽ hoạt động bằng cách thưởng cho LLM mỗi khi nó sinh ra câu trả lời sát hơn với sở thích của con người, giống như cách thầy giáo nhận xét bài làm của học sinh. Để có được mô hình phần thưởng như vậy, ta có thể huấn luyện một hàm chấm điểm từ dữ liệu sở thích của con người. Mô hình phần thưởng đã huấn luyện sau đó được dùng làm "người dẫn đường" để điều chỉnh và cải thiện LLM. Cách nhìn này biến nhiệm vụ căn chỉnh LLM thành một nhiệm vụ học tăng cường. Các phương pháp thu được, chẳng hạn **học tăng cường từ phản hồi của con người (RLHF)** (*reinforcement learning from human feedback*), đã được chứng minh là đặc biệt thành công trong việc thích nghi LLM để tuân theo những điều tinh tế trong hành vi và chuẩn mực xã hội của con người.

Cách tiếp cận thứ ba là thực hiện căn chỉnh trong lúc suy luận (*inference*) thay vì trong lúc huấn luyện hay tinh chỉnh. Từ góc nhìn này, prompting đối với LLM cũng có thể được xem là một hình thức căn chỉnh không cần huấn luyện. Nhờ đó, ta có thể thích nghi LLM một cách linh hoạt với nhiều nhiệm vụ khác nhau với chi phí tối thiểu. Một cách khác để căn chỉnh lúc suy luận là chấm điểm lại các đầu ra của LLM. Ví dụ, ta có thể xây dựng một hệ thống chấm điểm mô phỏng phản hồi của con người đối với các đầu ra của LLM (giống như một mô hình phần thưởng) và ưu tiên những đầu ra nhận được phản hồi tích cực hơn.

Ba phương pháp nêu trên thường được dùng nối tiếp nhau sau khi tiền huấn luyện hoàn tất: trước tiên ta thực hiện SFT, sau đó là RLHF, rồi đến prompting LLM trong lúc suy luận. Điều này chia quá

trình phát triển LLM thành hai giai đoạn một cách đại thể — giai đoạn tiền huấn luyện và giai đoạn căn chỉnh. Hình 10.1 minh họa điều này. Vì các kỹ thuật prompting đã được bàn kỹ trong chương trước, phần còn lại của chương này sẽ chủ yếu tập trung vào các phương pháp căn chỉnh dựa trên tinh chỉnh.



Hình 10.1: Sơ đồ minh họa quy trình "tiền huấn luyện rồi căn chỉnh" để phát triển LLM. Trong giai đoạn tiền huấn luyện, ta huấn luyện một LLM trên lượng dữ liệu khổng lồ bằng cách dự đoán token tiếp theo. Sau đó, trong giai đoạn căn chỉnh, ta căn chỉnh LLM theo chỉ dẫn, ý định và sở thích của người dùng. Giai đoạn này bao gồm căn chỉnh theo chỉ dẫn, căn chỉnh theo sở thích của con người, và prompting.

Chú giải của người dịch: Đọc Hình 10.1

Các nhãn trong hình: *Pre-training Stage* = giai đoạn tiền huấn luyện; *Alignment Stage* = giai đoạn căn chỉnh; *Instruction Alignment (e.g., SFT)* = căn chỉnh theo chỉ dẫn (ví dụ SFT); *Human Preference Alignment (e.g., RLHF)* = căn chỉnh theo sở thích của con người (ví dụ RLHF); *Training & Fine-tuning* = huấn luyện và tinh chỉnh (làm thay đổi tham số mô hình); *Inference* = suy luận (chỉ dùng mô hình, không đổi tham số). Có thể so sánh với việc đào tạo một nhân viên: đi học đại học (tiền huấn luyện) → được hướng dẫn làm đúng các loại việc ở công ty (SFT) → được sếp và khách hàng nhận xét để làm vừa ý hơn (RLHF) → mỗi lần giao việc thì dặn dò cụ thể (prompting).

10.2 Căn chỉnh theo chỉ dẫn

Một đặc điểm của LLM là chúng có thể làm theo các prompt do người dùng cung cấp để thực hiện nhiều nhiệm vụ khác nhau. Trong nhiều ứng dụng, một prompt gồm một chỉ dẫn đơn giản và đầu vào của người dùng, và mục tiêu là để LLM thực hiện nhiệm vụ một cách chính xác theo chỉ dẫn đó. Năng lực này thường được gọi là khả năng làm theo chỉ dẫn (*instruction following*). Ví dụ, dưới đây là một prompt trong đó LLM được yêu cầu rút ra các ý chính và viết một bản tóm tắt ngắn gọn cho một bài báo dài.

Instruction Tóm tắt văn bản này trong ba câu.

Input Giờ mùa hè (Daylight Savings Time – DST) — việc vặn đồng hồ nhanh thêm một giờ vào mùa hè — được bắt đầu ở Đức vào năm 1916. Trong Chiến tranh thế giới thứ nhất, đó là một cách để tiết kiệm …

Output ____

Nhiệm vụ này đòi hỏi LLM hiểu chỉ dẫn "Tóm tắt văn bản này trong ba câu" và tóm tắt văn bản theo đúng như vậy. Tuy nhiên, các LLM nền đã tiền huấn luyện thường được huấn luyện để dự đoán token tiếp theo chứ không phải để sinh ra đầu ra làm theo chỉ dẫn. Áp dụng một LLM đã tiền huấn luyện vào ví dụ trên nhiều khả năng sẽ khiến mô hình viết tiếp bài báo thay vì tóm tắt nó. Mục tiêu của căn chỉnh theo chỉ dẫn (hay **tinh chỉnh theo chỉ dẫn** (*instruction fine-tuning*)) là điều chỉnh LLM để nó đáp ứng chính xác chỉ dẫn và ý định của người dùng. Phần còn lại của mục này sẽ bàn về một số chủ đề liên quan đến căn chỉnh theo chỉ dẫn, bao gồm tinh chỉnh LLM để làm theo chỉ dẫn, sinh ra hoặc thu thập dữ liệu chỉ dẫn, và tổng quát hóa căn chỉnh theo chỉ dẫn.

Chú giải của người dịch: Vì sao LLM nền lại "viết tiếp" thay vì tóm tắt?

Trong tiền huấn luyện, mô hình chỉ học một việc: đọc một đoạn văn và đoán token (mẩu từ) kế tiếp. Trên Internet, sau câu "…đó là một cách để tiết kiệm" thường là phần tiếp theo của chính bài báo ("…than đá và nhiên liệu"), chứ hiếm khi là một bản tóm tắt. Vì vậy, mô hình nền có xu hướng viết tiếp cho "trơn tru" thay vì hiểu rằng dòng đầu là một *yêu cầu*. Tinh chỉnh theo chỉ dẫn dạy mô hình nhận ra cấu trúc "chỉ dẫn + đầu vào → đầu ra".

10.2.1 Tinh chỉnh có giám sát

Một cách tiếp cận đơn giản để thích nghi LLM cho việc làm theo chỉ dẫn là tinh chỉnh các mô hình này bằng các cặp đầu vào–đầu ra đã được gán nhãn [Ouyang và cộng sự, 2022; Wei và cộng sự, 2022a]. Trong bối cảnh căn chỉnh LLM, SFT thường được triển khai dưới dạng tinh chỉnh theo chỉ dẫn. Khác với huấn luyện mô hình ngôn ngữ thông thường, trong đó hàm mất mát được tính trên tất cả các token của chuỗi, SFT chỉ tính hàm mất mát trên các token đầu ra, với điều kiện là đầu vào. Gọi $\mathbf{x} = x_0 \dots x_m$ là một chuỗi đầu vào (ví dụ: chỉ dẫn + đầu vào của người dùng) và $\mathbf{y} = y_1 \dots y_n$ là chuỗi đầu ra tương ứng. Dữ liệu SFT gồm các cặp đầu vào–đầu ra như vậy (ký hiệu là S), trong đó mỗi đầu ra là một câu trả lời mong muốn ứng với đầu vào. Ví dụ, bảng sau đây cho thấy một số mẫu trong một tập dữ liệu SFT.

x (chỉ dẫn + đầu vào của người dùng)	**y** (đầu ra)
Tóm tắt bài báo sau. Bài báo: Trong những năm gần đây, năng lượng mặt trời đã tăng trưởng chưa từng có, trở thành nguồn năng lượng phát triển nhanh nhất …	{∗bản tóm tắt∗}
Phân tích cảm xúc của bài nhận xét sau. Nhận xét: Tôi cực kỳ thích trải nghiệm ăn uống mới này. Đồ ăn tuyệt hảo và phục vụ thì không chê vào đâu được.	Tích cực
Dịch câu sau sang tiếng Pháp. Câu: practice indeed helps. ("luyện tập quả thật có ích")	La pratique aide effectivement.
Trích xuất các số liệu tài chính chính từ báo cáo kết quả kinh doanh sau. Báo cáo: Công ty báo cáo doanh thu 10 triệu đô la trong quý đầu tiên với biên lợi nhuận 15% …	Doanh thu: 10 triệu đô la, Biên lợi nhuận: 15%
Phân loại email sau là thư rác hay không phải thư rác. Văn bản: Chúc mừng! Bạn đã trúng một thẻ quà tặng trị giá 500 đô la. Nhấn vào đây để nhận ngay.	Thư rác

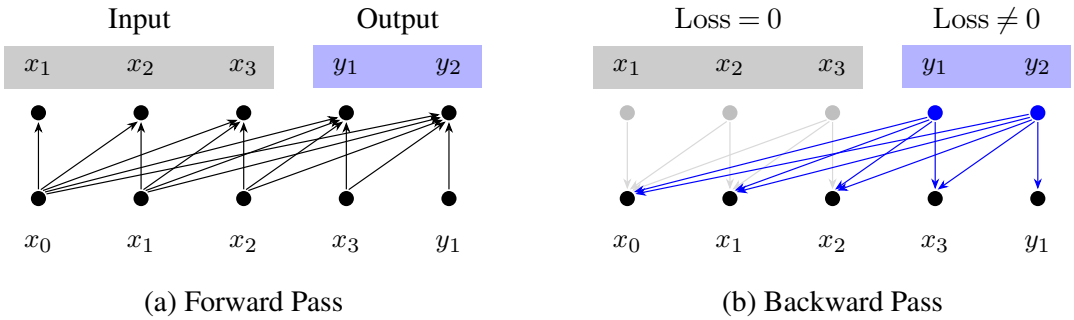
x (chỉ dẫn + đầu vào của người dùng)	**y** (đầu ra)
Đưa ra giải pháp cho sự cố kỹ thuật sau. Sự cố: máy tính của tôi chạy chậm và thường xuyên bị treo.	Trước tiên, hãy kiểm tra …

trong đó các chỉ dẫn được tô sáng. Tập dữ liệu này chứa các chỉ dẫn và đầu ra tương ứng cho nhiều bài toán NLP khác nhau, vì vậy ta có thể tinh chỉnh một LLM để xử lý đồng thời nhiều nhiệm vụ.

Trong SFT, ta muốn cực đại hóa xác suất của đầu ra đúng khi biết đầu vào. Xét một LLM với các tham số đã tiền huấn luyện $\hat{\theta}$. Khi đó mục tiêu tinh chỉnh có thể được viết như sau:

$$\tilde{\theta} \;=\; \arg\max_{\hat{\theta}^{+}} \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \log \Pr_{\hat{\theta}^{+}}(\mathbf{y}|\mathbf{x}), \tag{10.1}$$

trong đó $\tilde{\theta}$ ký hiệu các tham số được tối ưu qua tinh chỉnh, và $\hat{\theta}^{+}$ biểu diễn một sự điều chỉnh của $\hat{\theta}$. Từ đây ta sẽ bỏ chỉ số trên $+$ và dùng θ để biểu diễn $\hat{\theta}^{+}$ cho ký hiệu gọn gàng. Nhưng bạn đọc nên nhớ rằng quá trình tinh chỉnh bắt đầu từ các tham số đã tiền huấn luyện chứ không phải từ các tham số khởi tạo ngẫu nhiên.



(a) Forward Pass (b) Backward Pass

Hình 10.2: Minh họa tinh chỉnh có giám sát cho LLM. Ta nối đầu vào và đầu ra thành một chuỗi duy nhất. Trong lượt truyền xuôi, ta chạy LLM như bình thường. Trong lượt truyền ngược, ta chỉ tính hàm mất mát cho phần đầu ra và đơn giản đặt hàm mất mát cho phần đầu vào bằng 0. (Nhãn trong hình: *Input* = đầu vào, *Output* = đầu ra, *Loss* = mất mát; (a) lượt truyền xuôi, (b) lượt truyền ngược.)

Hàm mục tiêu $\log \Pr_\theta(\mathbf{y}|\mathbf{x})$ được tính bằng cách cộng log-xác suất của các token trong $\mathbf{y}$, với điều kiện là đầu vào $\mathbf{x}$ và tất cả các token đứng trước $\mathbf{y}_{<i}$:

$$\log \Pr_\theta(\mathbf{y}|\mathbf{x}) \;=\; \sum_{i=1}^{n} \log \Pr_\theta(y_i|\mathbf{x}, \mathbf{y}_{<i}). \tag{10.2}$$

Cách viết này tương đương với việc cực tiểu hóa hàm mất mát entropy chéo (*cross-entropy*).

Chú giải của người dịch: Ý nghĩa của Phương trình (10.1) và (10.2)

$\Pr_\theta(\mathbf{y}|\mathbf{x})$ là xác suất mô hình (với bộ tham số θ) gán cho câu trả lời đúng $\mathbf{y}$ khi đọc đầu vào $\mathbf{x}$. Ta muốn xác suất này càng lớn càng tốt cho mọi cặp mẫu trong tập dữ liệu, nên lấy tổng log của chúng rồi tìm θ làm tổng này lớn nhất (arg max). Mô hình sinh câu trả lời từng token một, nên theo quy tắc nhân xác suất, xác suất cả câu bằng *tích* xác suất từng token (mỗi token tính khi đã biết đầu vào và các token trước nó). Lấy logarit biến tích thành tổng: $\log(ab) = \log a + \log b$ — đó chính là Phương trình (10.2). Ví dụ: nếu $\mathbf{y}$ có 2 token với xác suất lần lượt 0.5 và 0.8 thì $\Pr(\mathbf{y}|\mathbf{x}) = 0.5 \times 0.8 = 0.4$ và $\log 0.4 = \log 0.5 + \log 0.8$. Cực đại hóa tổng

log-xác suất cũng chính là cực tiểu hóa *số đối* của nó, tức là hàm mất mát entropy chéo.

Nếu ta nối $\mathbf{x}$ và $\mathbf{y}$ thành một chuỗi duy nhất, một dạng tổng quát hơn của mô hình hóa ngôn ngữ dựa trên log-xác suất đồng thời $\log \Pr_\theta(\mathbf{x}, \mathbf{y})$, tức là ta cực tiểu hóa hàm mất mát (hay cực đại hóa log-hợp lý) trên tất cả các token của chuỗi $\text{seq}_{\mathbf{x},\mathbf{y}} = [\mathbf{x}, \mathbf{y}]$. Ta có thể viết xác suất của chuỗi này bằng quy tắc chuỗi

$$\begin{aligned} \log \Pr_\theta(\text{seq}_{\mathbf{x},\mathbf{y}}) &= \log \Pr_\theta(\mathbf{x}, \mathbf{y}) \\ &= \underbrace{\log \Pr_\theta(\mathbf{x})}_{\text{đặt bằng 0}} + \underbrace{\log \Pr_\theta(\mathbf{y}|\mathbf{x})}_{\text{tính mất mát}}. \end{aligned} \tag{10.3}$$

Vế phải của phương trình có hai số hạng. Ta có thể đơn giản bỏ qua số hạng thứ nhất $\log \Pr_\theta(\mathbf{x})$ và chỉ tập trung vào số hạng thứ hai $\log \Pr_\theta(\mathbf{y}|\mathbf{x})$ để tính mất mát. Nhờ vậy, việc huấn luyện có thể được thực hiện bằng các LLM tiêu chuẩn. Với chuỗi $\text{seq}_{\mathbf{x},\mathbf{y}}$, trước tiên ta chạy lượt truyền xuôi như bình thường. Sau đó, trong lượt truyền ngược, ta buộc phần mất mát tương ứng với $\mathbf{x}$ bằng không. Hình 10.2 minh họa quá trình này.

Chú giải của người dịch: "Che" phần đầu vào khi tính mất mát

Phương trình (10.3) dựa trên công thức xác suất quen thuộc $P(A \cap B) = P(A) \cdot P(B|A)$, lấy log hai vế thành tổng. Trong thực tế, ta đưa cả chuỗi "chỉ dẫn + đầu vào + câu trả lời" vào mô hình, mô hình đoán từng token như lúc tiền huấn luyện, nhưng chỉ "chấm điểm" (tính mất mát) ở những vị trí thuộc câu trả lời. Ta không muốn mô hình học cách *viết ra câu hỏi*, mà chỉ muốn nó học cách *trả lời*. Giống như chấm bài: giáo viên chỉ chấm phần bài làm, không chấm phần đề bài được chép lại.

Lấy $\log \Pr_\theta(\text{seq}_{\mathbf{x},\mathbf{y}})$ làm hàm mục tiêu, ta có thể mô tả SFT bằng một dạng huấn luyện mô hình ngôn ngữ thông thường:

$$\tilde{\theta} = \arg\max_\theta \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \log \Pr_\theta(\text{seq}_{\mathbf{x},\mathbf{y}}). \tag{10.4}$$

Bài toán ta xét ở trên về bản chất là một bài toán **dự đoán một lượt** (*single-round prediction*), trong đó LLM sinh ra câu trả lời dựa trên một đầu vào duy nhất mà không có thêm tương tác hay phản hồi nào từ người dùng. Đầu vào được xử lý, và đầu ra được sinh ra trong một lần. Đây là tình huống điển hình khi chỉ có một câu hỏi được đặt ra và một câu trả lời được đưa ra, không có câu hỏi tiếp theo hay yêu cầu làm rõ nào. Tuy nhiên, trong thực tế, đôi khi ta phải xử lý các bài toán dự đoán nhiều lượt (*multi-round prediction*), chẳng hạn khi một LLM tham gia một cuộc hội thoại qua nhiều lượt. Trong bối cảnh này, LLM không chỉ sinh câu trả lời dựa trên đầu vào ban đầu mà còn phải tiếp nhận các đầu vào tiếp theo, vốn có thể làm rõ hoặc mở rộng những trao đổi trước đó. Ví dụ, ta có thể dùng LLM làm một chatbot trợ lý chăm sóc sức khỏe và trò chuyện với người dùng như sau

User	Dạo này tôi thấy rất mệt mỏi.
Chatbot	Tôi rất tiếc khi nghe điều đó. Ngoài cảm giác mệt mỏi, bạn có nhận thấy triệu chứng nào khác không?
User	Có, tôi cũng hay bị đau đầu.
Chatbot	Các triệu chứng này đã kéo dài bao lâu rồi?
User	Khoảng một tuần nay.
Chatbot	Có lẽ bạn nên đi khám với nhân viên y tế. Bạn có muốn tôi giúp đặt lịch hẹn không?

User Vâng, làm ơn. Có thể hẹn sau giờ làm việc được không?

Chatbot Chắc chắn rồi, tôi có thể sắp xếp. Còn chỗ trống vào thứ Tư và thứ Năm tuần sau, sau 5 giờ chiều. Ngày nào thuận tiện hơn cho bạn?

…

Trong nhiệm vụ này có nhiều lượt hội thoại, mỗi lượt đòi hỏi sinh ra một câu trả lời dựa trên yêu cầu hoặc câu hỏi của người dùng và lịch sử hội thoại. Giả sử ta có K lượt hội thoại, ký hiệu là $\{\mathbf{x}^1, \mathbf{y}^1, \mathbf{x}^2, \mathbf{y}^2, \ldots, \mathbf{x}^K, \mathbf{y}^K\}$. Ở đây $\mathbf{x}^k$ và $\mathbf{y}^k$ lần lượt ký hiệu yêu cầu của người dùng và câu trả lời ở lượt thứ k. Log-xác suất sinh ra câu trả lời có thể được viết là $\log \Pr_\theta(\mathbf{y}^k|\mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{x}^k)$. Khi đó, mục tiêu của ta là cực đại hóa tổng các log-xác suất này

$$\tilde{\theta} \;=\; \arg\max_\theta \sum_{k=1}^{K} \log \Pr_\theta(\mathbf{y}^k|\mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{x}^k). \tag{10.5}$$

Một cách cài đặt đơn giản là tính xác suất có điều kiện cho từng k. Tuy nhiên, cách này đòi hỏi chạy LLM K lần, mỗi lần với lịch sử hội thoại dài thêm để đưa ra dự đoán. Một phương pháp hiệu quả hơn là tính mất mát cho tất cả các câu trả lời chỉ trong một lần chạy LLM. Để làm vậy, ta biểu diễn cuộc hội thoại dưới dạng một chuỗi $\text{seq}_{\mathbf{x}^1,\mathbf{y}^1,\ldots,\mathbf{x}^K,\mathbf{y}^K} = [\mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{x}^K, \mathbf{y}^K]$ (gọi tắt là seq). Log-xác suất của chuỗi này là

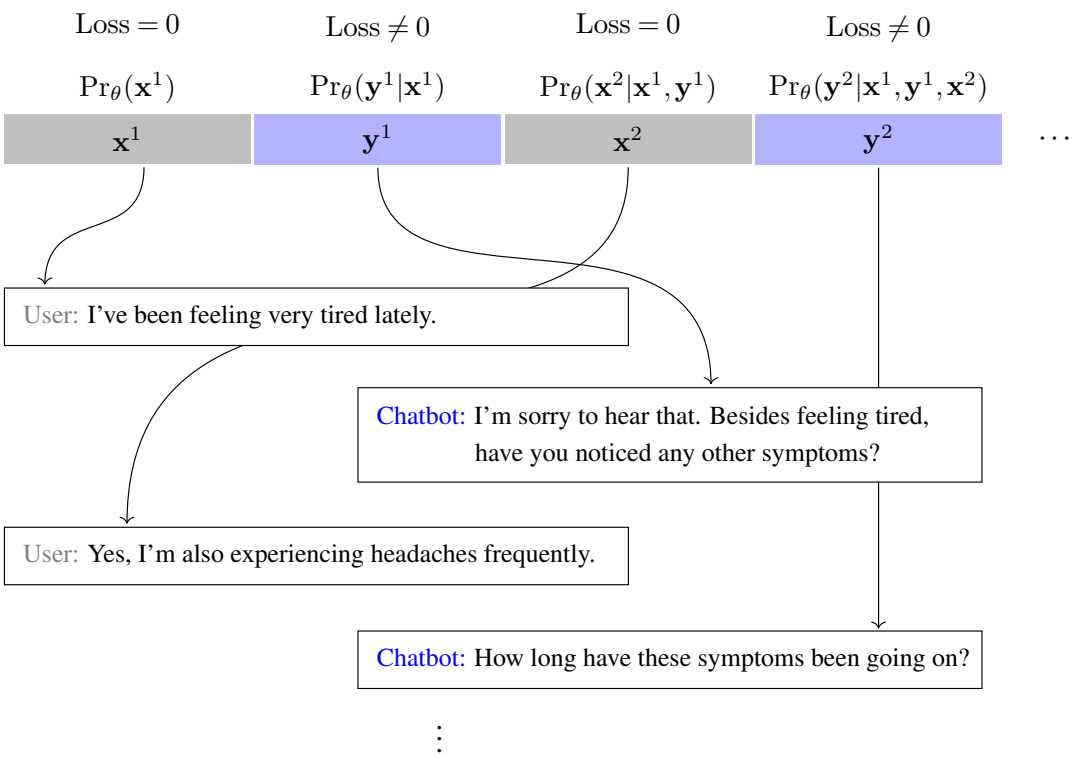
$$\begin{aligned}
\log \Pr_\theta(\text{seq}) &= \log \Pr_\theta(\mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{x}^K, \mathbf{y}^K) \\
&= \underbrace{\log \Pr_\theta(\mathbf{x}^1)}_{\text{đặt bằng 0}} + \underbrace{\log \Pr_\theta(\mathbf{y}^1|\mathbf{x}^1)}_{\text{tính mất mát}} + \cdots + \\
&\quad \underbrace{\log \Pr_\theta(\mathbf{x}^K|\mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{y}^{K-1})}_{\text{đặt bằng 0}} + \\
&\quad \underbrace{\log \Pr_\theta(\mathbf{y}^K|\mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{x}^K)}_{\text{tính mất mát}}.
\end{aligned} \tag{10.6}$$

Mẹo ở đây là ta bỏ qua mất mát của việc sinh ra các đầu vào của người dùng (tức là $\log \Pr_\theta(\mathbf{x}^1)$, …, $\log \Pr_\theta(\mathbf{x}^K|\mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{y}^{K-1})$), như minh họa ở Hình 10.3. Do đó ta chỉ tính xác suất sinh ra các câu trả lời khi biết lịch sử hội thoại của chúng; nói cách khác, Phương trình (10.6) tương đương với Phương trình (10.5). Giống như với Phương trình (10.4), việc huấn luyện mô hình dự đoán nhiều lượt này có thể được thực hiện bằng cách cực đại hóa log-hợp lý trên một tập dữ liệu huấn luyện $\mathcal{D}$:

$$\tilde{\theta} \;=\; \arg\max_\theta \sum_{\text{seq}\in\mathcal{D}} \log \Pr_\theta(\text{seq}). \tag{10.7}$$

Chú giải của người dịch: Hội thoại nhiều lượt: một lần chạy cho cả cuộc trò chuyện

Phương trình (10.6) là quy tắc nhân xác suất áp dụng liên tiếp: xác suất của cả cuộc hội thoại bằng tích xác suất của từng "lời nói", mỗi lời tính khi đã biết mọi lời trước nó; lấy log thì tích thành tổng. Trong tổng đó, các số hạng ứng với lời của *người dùng* bị bỏ đi (đặt mất mát bằng 0), chỉ giữ các số hạng ứng với lời của *chatbot* — chính là các số hạng trong Phương trình (10.5). Lợi ích: với một hội thoại $K = 4$ lượt, thay vì chạy mô hình 4 lần (mỗi lần với lịch sử dài hơn), ta chỉ cần đưa cả cuộc hội thoại vào một lần và "che" các phần của người dùng khi tính mất mát.



Hình 10.3: Minh họa tinh chỉnh có giám sát cho các mô hình hội thoại. Ở đây LLM đóng vai một chatbot trả lời từng yêu cầu dựa trên lịch sử hội thoại. Cuộc hội thoại diễn ra bằng cách luân phiên giữa người dùng (*User*) và chatbot. Trong SFT, ta coi toàn bộ cuộc hội thoại là một chuỗi, giống như trong các LLM tiêu chuẩn, nhưng chỉ tính mất mát cho các câu trả lời của LLM. (Nội dung các ô hội thoại là đoạn đầu của ví dụ hội thoại ở trên.)

Mặc dù việc cài đặt các phương pháp SFT nêu trên có vẻ đơn giản vì về cơ bản chúng giống huấn luyện mô hình ngôn ngữ thông thường, vẫn còn những vấn đề cần cân nhắc trong thực tế. Chẳng hạn,

- SFT đòi hỏi dữ liệu có gán nhãn. Điều này khiến SFT khá khác với tiền huấn luyện, nơi văn bản thô được dùng làm dữ liệu huấn luyện và luôn sẵn có. Cũng như trong các bài toán học máy có giám sát khác, việc gán nhãn và chọn lọc dữ liệu trong SFT không phải là việc đơn giản. Nói chung, ta muốn xây dựng dữ liệu SFT vừa nhiều về số lượng vừa cao về chất lượng, và dữ liệu này phải liên quan chặt chẽ đến những nhiệm vụ mà LLM sẽ thực hiện. Mặt khác, lại cần tinh chỉnh LLM với ít dữ liệu hơn để giảm thiểu chi phí tính toán và chi phí xây dựng dữ liệu. Chất lượng của LLM thường phụ thuộc rất nhiều vào dữ liệu dùng trong SFT. Vì vậy, dữ liệu này phải được xây dựng và kiểm tra cẩn thận. Như ta sẽ thấy ở các tiểu mục sau, SFT có thể hiệu quả và hiệu nghiệm hơn nhờ các kỹ thuật xây dựng dữ liệu tiên tiến hơn.

- SFT vẫn tốn kém về mặt tính toán đối với LLM do kích thước lớn của chúng. Kết quả là việc duy trì và cập nhật các mô hình như vậy tiêu tốn nhiều tài nguyên. Ví dụ, việc cập nhật gradient cho hàng tỉ tham số trong một LLM đòi hỏi năng lực tính toán và bộ nhớ đáng kể. Điều này thường cần các môi trường tính toán hiệu năng cao vốn tốn kém để vận hành. Để giải quyết những thách thức này, nhiều chiến lược tối ưu đã được khám phá, chẳng hạn cắt tỉa (*pruning*), lượng tử hóa (*quantization*), và sử dụng các thuật toán huấn luyện hiệu quả hơn. Đặc biệt, đã có nhiều quan tâm đến các phương pháp tinh chỉnh hiệu quả tham số (*parameter-efficient fine-tuning*), được thiết kế để duy trì hiệu năng tiên tiến nhất mà không cần nhiều tài nguyên tính toán. Ta đã thấy ở Chương 9 rằng áp dụng các kỹ thuật như prompt mềm (*soft prompts*) có thể làm quá trình tinh chỉnh hiệu quả hơn. Để tìm hiểu thêm về các phương pháp hiệu quả tham số, bạn đọc có thể tham khảo các bài báo liên quan [Houlsby và cộng sự, 2019; Hu và cộng sự, 2022; Han và cộng sự, 2024].

- SFT có thể được xem là một bước hậu huấn luyện (*post-training*) tiếp sau tiền huấn luyện. Đó là một giai đoạn huấn luyện riêng, được thiết kế để giữ lại những ưu điểm của quá trình tiền huấn luyện ban đầu đồng thời bổ sung những điều chỉnh mới. Điều này có vẻ nghịch lý, vì cập nhật một LLM đã tiền huấn luyện bằng thêm dữ liệu có thể khiến mô hình quên đi một phần tri thức trước đó. Hãy hình dung tình huống ta có một lượng lớn dữ liệu SFT và tinh chỉnh LLM rất nhiều. Khi đó, LLM có thể quá khớp (*overfit*) dữ liệu, điều này lại có thể làm giảm khả năng tổng quát hóa hoặc gây ra hiện tượng quên thảm khốc (*catastrophic forgetting*). Một chiến lược phổ biến để giảm nhẹ vấn đề này là dùng các kỹ thuật chính quy hóa (*regularization*) và dừng sớm (*early stopping*). Một cách tiếp cận thực tế khác là dùng tốc độ học (*learning rate*) nhỏ hơn để điều chỉnh nhẹ nhàng các trọng số của LLM. Ngoài ra, tinh chỉnh bằng dữ liệu từ nhiều nguồn và nhiều lĩnh vực bài toán khác nhau cũng có thể có lợi. Tuy vậy, trong thực tế, bước SFT thường được xem xét cẩn thận và đòi hỏi nhiều công sức kỹ thuật cũng như thực nghiệm để tối ưu.

Chú giải của người dịch: Quên thảm khốc và cách phòng tránh

Quên thảm khốc là hiện tượng mô hình học việc mới thì "quên" mất nhiều điều đã học trước đó, vì cùng một bộ tham số bị kéo sang phục vụ mục tiêu mới. Hình dung một học sinh ôn thi cấp tốc chỉ một dạng bài suốt nhiều tuần, đến lúc gặp dạng khác lại lúng túng. Các biện pháp nêu trên đều nhằm "thay đổi vừa phải": *tốc độ học nhỏ* nghĩa là mỗi bước cập nhật chỉ xê dịch tham số một chút; *dừng sớm* là ngừng huấn luyện khi kết quả trên tập kiểm định bắt đầu xấu đi; *chính quy hóa* là thêm ràng buộc để tham số không đi quá xa giá trị ban đầu. *Lượng tử hóa* là lưu mỗi tham số bằng ít bit hơn (ví dụ 8 bit thay vì 32 bit) để tiết kiệm bộ nhớ; *cắt tỉa* là bỏ bớt những tham số ít quan trọng.

10.2.2 Xây dựng dữ liệu tinh chỉnh

Dữ liệu tinh chỉnh quan trọng đến mức rất nhiều nghiên cứu gần đây về LLM đã tập trung vào việc xây dựng các tập dữ liệu cho tinh chỉnh theo chỉ dẫn. Giống như với hầu hết các nhiệm vụ học máy, nhìn chung có hai cách tiếp cận để xây dựng tập dữ liệu tinh chỉnh: dữ liệu do con người tạo ra và dữ liệu được sinh tự động.

1. Dữ liệu do con người tạo ra

Một phương pháp đơn giản là thuê người gán nhãn tạo ra các cặp đầu vào–đầu ra cho những nhiệm vụ ta quan tâm. Khác với việc gán nhãn dữ liệu trong NLP truyền thống, chẳng hạn phân loại văn bản, nơi người gán nhãn chỉ việc gán nhãn cho các văn bản đã thu thập theo hướng dẫn, việc tạo dữ liệu tinh chỉnh cho LLM đòi hỏi nhiều bước và công sức hơn, do đó khó khăn hơn. Giả sử ta muốn có dữ liệu tinh chỉnh cho nhiệm vụ dịch máy từ tiếng Anh sang tiếng Trung. Bước đầu tiên là viết một khuôn mẫu prompt để mô tả nhiệm vụ và định dạng bài toán một cách rõ ràng. Ví dụ,

Instruction	Dịch văn bản từ tiếng Anh sang tiếng Trung.
User Input	{∗text∗}
Output	{∗translation∗}

Sau đó, ta thu thập các cặp văn bản nguồn và đích (tức là văn bản tiếng Anh và bản dịch tiếng Trung tương ứng), rồi thay các biến {∗text∗} (văn bản) và {∗translation∗} (bản dịch) để sinh ra các mẫu tinh chỉnh. Ví dụ, với một cặp câu tiếng Anh và tiếng Trung

$$\underset{\{*\text{text}*\}}{\text{How's the weather today?}} \quad \rightarrow \quad \underset{\{*\text{translation}*\}}{\text{今天天气怎么样?}}$$

(cả hai câu đều có nghĩa là "Hôm nay thời tiết thế nào?"), ta có thể sinh ra một mẫu tinh chỉnh bằng khuôn mẫu prompt như sau

Instruction	Dịch văn bản từ tiếng Anh sang tiếng Trung.
User Input	How's the weather today?
Output	今天天气怎么样?

Tức là,

x: Dịch văn bản từ tiếng Anh sang tiếng Trung.\n How's the weather today?
y: 今天天气怎么样?

[1] Ta có thể dùng cặp $(\mathbf{x}, \mathbf{y})$ này để tinh chỉnh LLM như đã mô tả ở tiểu mục trước.

Một khó khăn ở đây là có rất nhiều cách khác nhau để viết khuôn mẫu prompt cho cùng một nhiệm vụ, và những người khác nhau có thể tạo ra các khuôn mẫu với chất lượng và độ phức tạp khác nhau. Đôi khi ta viết khuôn mẫu prompt với các chỉ dẫn quá phức tạp hoặc dài dòng. Đôi khi ta thậm chí không biết chính xác nhiệm vụ đích là gì và nên mô tả nó thế nào. Một chiến lược được áp dụng rộng rãi là tạo khuôn mẫu prompt cho các nhiệm vụ NLP sẵn có, vì đã có rất nhiều bài toán và bộ đánh giá chuẩn NLP được xây dựng hoàn chỉnh [Bach và cộng sự, 2022; Wang và cộng sự, 2022e; Mishra và

[1] **[ND]** \n là ký hiệu xuống dòng: chỉ dẫn và đầu vào của người dùng được nối với nhau thành một chuỗi văn bản, ngăn cách bởi một dấu xuống dòng.

cộng sự, 2022]. Trong trường hợp này, người gán nhãn có thể được cung cấp mô tả nhiệm vụ gốc cùng nhiều ví dụ. Sau đó, họ có thể dùng cách riêng của mình để diễn đạt việc prompt LLM thực hiện các nhiệm vụ đó. Lưu ý rằng, mặc dù phương pháp này có thể giúp việc tạo prompt dễ dàng hơn, ta vẫn cần các khung gán nhãn và hệ thống huy động cộng đồng (*crowdsourcing*) để quản lý công việc và kiểm soát chất lượng. Ví dụ, nhìn chung ta cần thiết kế hướng dẫn gán nhãn và một định dạng thống nhất để viết khuôn mẫu prompt, đặc biệt khi có nhiều người gán nhãn cùng đóng góp cho một nhiệm vụ. Một ưu điểm của việc xây dựng prompt từ các nhiệm vụ NLP sẵn có là, một khi các khuôn mẫu prompt đã được xây dựng, ta dễ dàng sinh ra các ví dụ tinh chỉnh bằng các mẫu đã được gán nhãn trong các tập dữ liệu gốc. Ví dụ, với một tập dữ liệu song ngữ cho dịch Anh–Trung, ta có thể dễ dàng tạo ra nhiều ví dụ tinh chỉnh bằng cách điền các cặp câu trong tập dữ liệu này vào các chỗ trống của khuôn mẫu trên.

Một cách tiếp cận khác là trực tiếp dùng dữ liệu có sẵn tự nhiên trên Internet. Một ví dụ phổ biến là thu thập các cặp hỏi–đáp từ các trang web hỏi đáp để tinh chỉnh LLM cho các nhiệm vụ hỏi đáp miền mở (*open-domain QA*) [Joshi và cộng sự, 2017]. Nhiều bộ đánh giá chuẩn về hỏi đáp được xây dựng theo cách này, vì có quá nhiều loại câu hỏi đến mức một nhóm nhỏ người gán nhãn khó lường trước được mọi loại câu hỏi có thể có. Thay vào đó, dùng dữ liệu từ các trang web ấy có thể bảo đảm dữ liệu tinh chỉnh LLM đủ lớn và đủ đa dạng.

Ngoài việc sử dụng các nguồn tài nguyên sẵn có, một cách đơn giản khác để xây dựng tập dữ liệu tinh chỉnh là huy động cộng đồng tạo dữ liệu. Một cách đơn giản là cho phép người dùng nhập bất kỳ câu hỏi nào, sau đó câu trả lời hoặc được viết thủ công, hoặc được LLM sinh tự động rồi được người gán nhãn xem xét và sửa chữa. Nhờ vậy, ta có thể nắm bắt hành vi thực của người dùng và từ đó thu thập đầu vào và đầu ra cho rất nhiều bài toán mới mà các nhiệm vụ NLP truyền thống không bao quát.

Một vấn đề liên quan đến việc xây dựng tập dữ liệu tinh chỉnh là ta thường muốn dữ liệu càng đa dạng càng tốt. Nhiều nghiên cứu đã phát hiện rằng tăng tính đa dạng của dữ liệu tinh chỉnh có thể cải thiện độ bền vững (*robustness*) và khả năng tổng quát hóa của LLM. Vì lý do này, đã có nhiều quan tâm đến việc đưa thêm các prompt và nhiệm vụ đa dạng hơn vào các tập dữ liệu tinh chỉnh LLM. Ta sẽ bàn thêm về tổng quát hóa của tinh chỉnh ở Mục 10.2.4.

2. Dữ liệu được sinh tự động

Một hạn chế của việc tạo dữ liệu thủ công là chất lượng và tính đa dạng phụ thuộc rất nhiều vào kinh nghiệm và sự sáng tạo của con người. Do đó, nếu ta muốn LLM tổng quát hóa được cho các chỉ dẫn đa dạng của người dùng, việc dựa vào dữ liệu do con người gán nhãn để tinh chỉnh LLM thường kém hiệu quả. Hơn nữa, độ bao phủ của dữ liệu như vậy có thể hạn chế, và dữ liệu thậm chí có thể chứa những thiên kiến do chính người gán nhãn đưa vào. Một cách tiếp cận thay thế là sinh dữ liệu tự động. Ví dụ, ta có thể thu thập một số câu hỏi bằng cách huy động cộng đồng, rồi dùng một LLM đã được tinh chỉnh tốt để sinh câu trả lời cho các câu hỏi đó. Các cặp câu hỏi–câu trả lời này sau đó được dùng làm mẫu tinh chỉnh như thường lệ. Phương pháp này, dù rất đơn giản, đã được áp dụng rộng rãi để sinh dữ liệu tinh chỉnh quy mô lớn cho LLM.

Cách tạo dữ liệu tinh chỉnh tổng hợp nêu trên tương tự các phương pháp dùng trong tăng cường dữ liệu (*data augmentation*) của NLP. Nếu có một LLM, ta có thể tạo ra một đầu ra đáp lại bất kỳ đầu vào nào. Lặp lại quá trình này cho nhiều đầu vào khác nhau cho phép ta tạo ra lượng lớn dữ liệu tinh chỉnh. Phương pháp như vậy đặc biệt hữu ích khi tinh chỉnh các LLM mới bằng một LLM đã được tinh chỉnh tốt. Tuy nhiên, một nhược điểm của cách này là nó dựa vào các đầu vào do con người soạn hoặc thu thập để sinh dữ liệu, và những đầu vào đó có thể hóa ra không phù hợp cho việc tổng quát hóa LLM. Trong nhiều ứng dụng LLM, một thách thức lớn nảy sinh từ phạm vi rất rộng của các câu hỏi và yêu cầu của người dùng, nhiều trong số đó không được bao quát trong các nhiệm vụ và tập dữ liệu NLP hiện

có. Trong những trường hợp này, cần phải sinh ra không chỉ các dự đoán mà cả chính các đầu vào.

Ở đây ta xét phương pháp **tự tạo chỉ dẫn** (*self-instruct*) làm ví dụ để minh họa cách sinh mẫu tinh chỉnh LLM [Wang và cộng sự, 2023e; Honovich và cộng sự, 2023]. Ý tưởng là ta có thể prompt một LLM để tạo ra một chỉ dẫn mới bằng cách học từ các chỉ dẫn khác. Với chỉ dẫn này, LLM sau đó có thể điền các trường khác (chẳng hạn đầu vào của người dùng) và sinh ra đầu ra. Hình 10.4 là sơ đồ minh họa phương pháp tự tạo chỉ dẫn. Dưới đây ta phác thảo ngắn gọn các bước chính.

- Thuật toán tự tạo chỉ dẫn duy trì một *kho nhiệm vụ*. Ban đầu, kho chứa một số nhiệm vụ hạt giống do con người soạn, mỗi nhiệm vụ gồm một chỉ dẫn và một mẫu đầu vào–đầu ra. Khi thuật toán chạy, các chỉ dẫn và mẫu do LLM sinh ra sẽ được thêm vào kho này.

- Ở mỗi bước, một số ít chỉ dẫn được rút ra từ kho chỉ dẫn. Ví dụ, ta có thể chọn ngẫu nhiên vài chỉ dẫn do con người viết và vài chỉ dẫn do LLM sinh ra để bảo đảm tính đa dạng.

- Các chỉ dẫn được chọn được dùng làm ví dụ minh họa trong ngữ cảnh để sinh ra một chỉ dẫn mới. Dưới đây là một khuôn mẫu ví dụ để prompt LLM.

> Bạn được cung cấp một vài chỉ dẫn khác nhau để thực hiện một số nhiệm vụ. Hãy sinh ra một chỉ dẫn dựa trên những chỉ dẫn này.
> Nhiệm vụ 1: $\{*\text{instruction}_1*\}$
> Nhiệm vụ 2: $\{*\text{instruction}_2*\}$
> Nhiệm vụ 3: $\{*\text{instruction}_3*\}$
> Nhiệm vụ 4: $\{*\text{instruction}_4*\}$
> Nhiệm vụ mới: ___

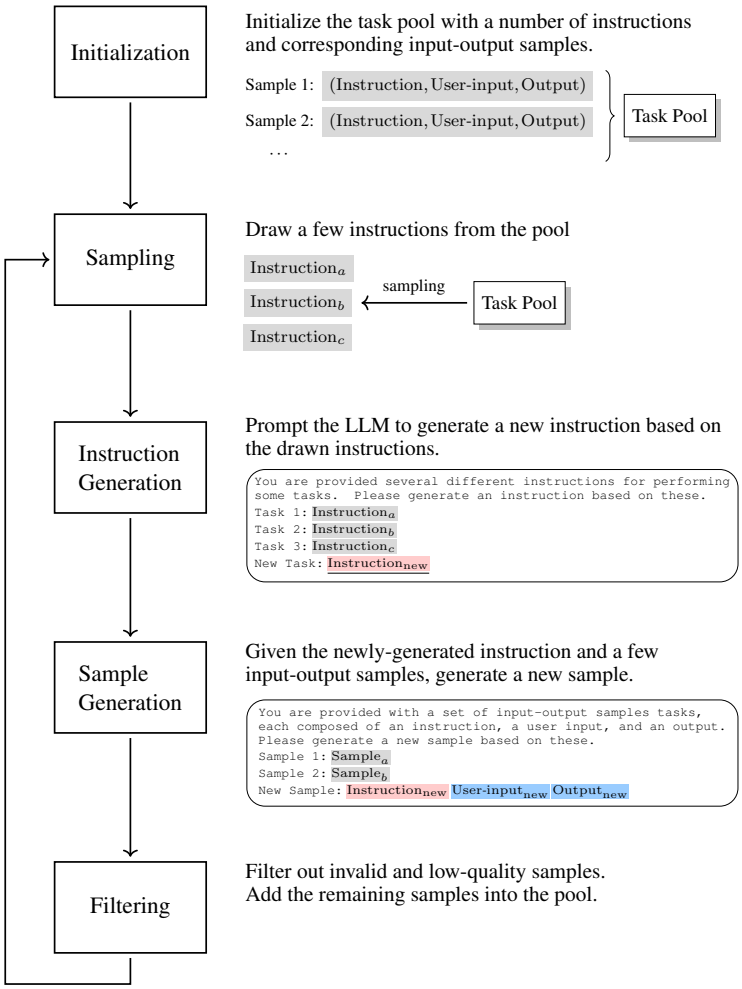
- Với chỉ dẫn vừa sinh ra, LLM tiếp tục được prompt để hoàn thiện mẫu bằng cách điền các trường đầu vào còn lại và sinh ra đầu ra tương ứng. Dưới đây là một khuôn mẫu prompt.

> Bạn được cung cấp một tập các mẫu đầu vào–đầu ra, mỗi mẫu gồm một chỉ dẫn, một đầu vào của người dùng và một đầu ra. Hãy sinh ra một mẫu mới dựa trên các mẫu này.
> Mẫu 1: $\{*\text{instruction}_1*\}$
> Đầu vào: $\{*\text{user-input}_1*\}$
> Đầu ra: $\{*\text{output}_1*\}$
> Mẫu 2: $\{*\text{instruction}_2*\}$
> Đầu vào: $\{*\text{user-input}_2*\}$
> Đầu ra: $\{*\text{output}_2*\}$
> Mẫu mới: $\{*\text{new-instruction}*\}$
> ___

- Mẫu mới sinh ra này được đánh giá hoặc lọc bằng các quy tắc kinh nghiệm (chẳng hạn loại bỏ các mẫu hoặc chỉ dẫn giống với những mẫu đã có trong kho). Nếu vượt qua, mẫu và chỉ dẫn được thêm vào kho.

2

[2] **[ND]** Trong các khuôn mẫu trên, *instruction* = chỉ dẫn, *user-input* = đầu vào của người dùng, *output* = đầu ra, *new-instruction* = chỉ dẫn mới; mỗi $\{*\ldots*\}$ là một biến sẽ được thay bằng nội dung cụ thể.



Hình 10.4: Minh họa phương pháp tự tạo chỉ dẫn [Wang và cộng sự, 2023c]. Phương pháp này duy trì một kho chỉ dẫn và các mẫu đầu vào–đầu ra tương ứng. Ban đầu, kho chứa một số chỉ dẫn và mẫu do con người soạn. Mỗi lần, ta rút một vài chỉ dẫn từ kho. Sau đó, một LLM được prompt để sinh ra các chỉ dẫn và mẫu mới dựa trên những chỉ dẫn đã rút. Cuối cùng, các chỉ dẫn và mẫu mới sinh được lọc rồi thêm vào kho.

Chú giải của người dịch: Đọc Hình 10.4 và ý tưởng "tự tạo chỉ dẫn"

Năm bước trong hình (từ trên xuống): *Initialization* — khởi tạo kho nhiệm vụ (*Task Pool*) bằng một số chỉ dẫn và mẫu (*Instruction, User-input, Output*); *Sampling* — rút vài chỉ dẫn từ kho; *Instruction Generation* — prompt LLM sinh chỉ dẫn mới từ các chỉ dẫn đã rút; *Sample Generation* — với chỉ dẫn mới và vài mẫu, sinh ra một mẫu mới; *Filtering* — loại bỏ mẫu không hợp lệ hoặc kém chất lượng, thêm phần còn lại vào kho. Các hộp chữ nhỏ trong hình chính là hai khuôn mẫu prompt đã dịch ở trên.
Hình dung một giáo viên có sẵn 10 đề bài mẫu, nhờ một trợ giảng giỏi "nhìn mấy đề này rồi ra thêm một đề tương tự nhưng khác", sau đó tự giải luôn đề mới. Đề nào trùng lặp hay sai thì bỏ, đề tốt thì đưa vào ngân hàng đề. Lặp lại nhiều lần, ngân hàng đề lớn dần từ vài chục lên hàng chục nghìn đề — đó là cách LLM "tự làm" ra dữ liệu để huấn luyện.

Quá trình sinh này có thể được lặp lại nhiều lần để thu được đủ số lượng mẫu tinh chỉnh. Lưu ý rằng ở trên ta chỉ trình bày các khuôn mẫu prompt đơn giản để sinh chỉ dẫn và mẫu tinh chỉnh. Tất nhiên, ta có thể xây dựng các khuôn mẫu tốt hơn để sinh ra chỉ dẫn và mẫu tinh chỉnh đa dạng và chính xác hơn. Ví dụ, với một số nhiệm vụ như phân loại văn bản, LLM có thể có xu hướng đưa ra các dự đoán thiên lệch, chẳng hạn hầu hết các mẫu sinh ra đều thuộc về một lớp duy nhất. Trong những trường hợp như vậy, ta có thể điều chỉnh thứ tự sinh các trường khác nhau. Cụ thể hơn, ta có thể chỉ định trước đầu ra (tức là lớp) theo một phân phối tiên nghiệm nào đó, rồi prompt LLM sinh ra đầu vào của người dùng khi biết cả chỉ dẫn lẫn đầu ra. Phương pháp này giống với đảo ngược đầu vào (*input inversion*), trong đó LLM sinh ra đầu vào dựa trên đầu ra đã được chỉ định [Longpre và cộng sự, 2023].

Dùng các chỉ dẫn và mẫu tinh chỉnh do LLM sinh ra đã trở thành một phương pháp phổ biến để phát triển LLM, nhất là khi việc xây dựng dữ liệu như vậy bằng tay tốn kém đến mức hầu hết các nhóm nghiên cứu không kham nổi. Ở một số LLM được tinh chỉnh tốt, tập dữ liệu tinh chỉnh của chúng bao gồm một lượng dữ liệu tổng hợp nhất định, và điều này đã được chứng tỏ là hiệu quả [Ouyang và cộng sự, 2022; Taori và cộng sự, 2023; Chiang và cộng sự, 2023b]. Đã có thêm nhiều nghiên cứu về sinh dữ liệu tổng hợp cho tinh chỉnh LLM. Ví dụ, ta có thể sinh ra các chỉ dẫn đa dạng hơn bằng cách đưa vào các thuật toán tiến hóa (*evolutionary algorithms*) [Xu và cộng sự, 2024], hoặc dùng dữ liệu tổng hợp làm tín hiệu giám sát trong một quá trình tinh chỉnh tiên tiến hơn [Chen và cộng sự, 2024b]. Gần đây hơn, cũng có nhiều quan tâm đến việc dùng dữ liệu tổng hợp ngay trong giai đoạn tiền huấn luyện [Gunasekar và cộng sự, 2023; Allal và cộng sự, 2024].

Trong nhiều ứng dụng, một tình huống thực tế là: với một nhiệm vụ cho trước, ta có thể thu thập hoặc gán nhãn một lượng tương đối nhỏ dữ liệu tinh chỉnh, chẳng hạn thuê chuyên gia soạn câu hỏi cho các nhiệm vụ hỏi đáp trong một lĩnh vực cụ thể. Nhưng số lượng và độ đa dạng của dữ liệu này nhìn chung là không đủ. Trong trường hợp này, ta có thể dùng các kỹ thuật tự tạo chỉ dẫn để sinh thêm các cặp câu hỏi–câu trả lời đa dạng hơn, nhờ đó tăng cường dữ liệu tinh chỉnh. Điều này cung cấp một cách "tự nâng" (*bootstrapping*) LLM bắt đầu từ một tập mẫu tinh chỉnh hạt giống. Lưu ý rằng dùng dữ liệu tự sinh là một thực hành phổ biến và đã được áp dụng từ lâu trong NLP. Ví dụ, cách tiếp cận này đã được dùng thành công trong phân tích cú pháp và dịch máy [Charniak, 1997; Sennrich và cộng sự, 2016a].

10.2.3 Tinh chỉnh với ít dữ liệu hơn

Với vai trò ngày càng nổi bật của tinh chỉnh theo chỉ dẫn, nhu cầu về dữ liệu tinh chỉnh quy mô lớn, chất lượng cao đã tăng vọt. Ví dụ, tập dữ liệu tinh chỉnh FLAN, được tổng hợp từ 1,836 nhiệm vụ, chứa 15 triệu mẫu [Longpre và cộng sự, 2023].[3] Tinh chỉnh LLM với các tập dữ liệu lớn như vậy thường là một nhiệm vụ tốn kém về tính toán, đặc biệt vì việc cập nhật số lượng lớn tham số của LLM tiêu tốn

[3] **[ND]** Giữ cách viết số như bản gốc: 1,836 = 1 836 nhiệm vụ.

nhiều tài nguyên. Một cách để giảm nhẹ vấn đề này là tìm các phương pháp huấn luyện mô hình hiệu quả, chẳng hạn dùng các phương pháp hiệu quả tham số để chỉ cập nhật một phần nhỏ của mô hình.

Một cách tiếp cận khác là tinh chỉnh LLM chỉ bằng những mẫu huấn luyện giàu thông tin nhất. Nhờ đó ta có thể giảm lượng dữ liệu cần xử lý mà vẫn duy trì hiệu năng tinh chỉnh. Có một số phương pháp để làm điều này. Ví dụ, Zhou và cộng sự [2023a] đã xây dựng một tập dữ liệu làm theo chỉ dẫn chỉ gồm 1,000 mẫu bằng cách soạn prompt cẩn thận và thu thập mẫu từ nhiều nhiệm vụ NLP khác nhau. Họ cho thấy mô hình LLaMa 65B được tinh chỉnh trên tập dữ liệu này có thể cạnh tranh ngang ngửa, thậm chí tốt hơn các mô hình được tinh chỉnh công phu hơn nhiều. Điều này gợi ý rằng LLM có thể được thích nghi để đáp ứng các nhiệm vụ đa dạng mà không nhất thiết phải tinh chỉnh trên mọi loại dữ liệu làm theo chỉ dẫn. Chen và cộng sự [2024a] đã phát triển một hệ thống dựa trên mô hình GPT-3.5 để đánh giá chất lượng của từng mẫu làm theo chỉ dẫn. Nhờ vậy, họ có thể chọn ra các mẫu chất lượng cao từ các tập dữ liệu sẵn có, cho hiệu năng tinh chỉnh tốt hơn với ít mẫu tinh chỉnh hơn. Các nhà nghiên cứu cũng đã phát triển những phương pháp chọn lọc hoặc loại bỏ dữ liệu bằng các quy tắc kinh nghiệm [Zhao và cộng sự, 2024; Ge và cộng sự, 2024], hoặc ưu tiên những dữ liệu có ảnh hưởng lớn hơn đến quá trình tinh chỉnh [Xia và cộng sự, 2024]. Thực ra, hầu hết các phương pháp này có thể được xem là trường hợp riêng của những họ phương pháp lựa chọn và lọc dữ liệu rộng hơn. Và thường thì dùng dữ liệu chất lượng cao hơn (dù có thể ít hơn) sẽ có lợi cho việc huấn luyện các mô hình NLP.

Những phát hiện trong tinh chỉnh theo chỉ dẫn có phần khác với quan điểm truyền thống trong NLP: khả năng xử lý các bài toán phức tạp của mô hình có thể được "kích hoạt" bằng một lượng nhỏ dữ liệu có gán nhãn, thay vì cần lượng dữ liệu có giám sát khổng lồ để huấn luyện kỹ lưỡng. Một cách giải thích khả dĩ là khả năng sinh câu trả lời đúng theo chỉ dẫn đã được học trong quá trình tiền huấn luyện, nhưng LLM có thể không gán xác suất cao cho những câu trả lời như vậy lúc suy luận. Tinh chỉnh có thể điều chỉnh nhẹ mô hình để nó làm theo chỉ dẫn, và do đó cần ít công sức huấn luyện hơn đáng kể so với tiền huấn luyện. Điều này liên quan chặt chẽ đến cái gọi là **giả thuyết căn chỉnh bề mặt** (*superficial alignment hypothesis*), cho rằng việc học chủ yếu diễn ra trong tiền huấn luyện, còn giai đoạn tinh chỉnh hay căn chỉnh sau đó không đóng góp đáng kể vào nền tảng tri thức của một LLM [Zhou và cộng sự, 2023a]. Vì năng lực cốt lõi và tri thức của mô hình đã được hình thành từ tiền huấn luyện, việc tinh chỉnh hiệu quả để căn chỉnh với nhu cầu người dùng có thể đạt được với công sức tinh chỉnh tương đối nhỏ. Điều này hàm ý khả năng tinh chỉnh LLM với rất ít dữ liệu. Theo một hướng khác, có thể không cần giới hạn việc tinh chỉnh vào dữ liệu theo cặp chỉ dẫn–câu trả lời. Ví dụ, Hewitt và cộng sự [2024] phát hiện rằng khả năng làm theo chỉ dẫn có thể đạt được một cách ngầm định bằng cách tinh chỉnh LLM chỉ trên các câu trả lời, không cần các chỉ dẫn tương ứng.

Chú giải của người dịch: Giả thuyết căn chỉnh bề mặt

Theo giả thuyết này, hầu như mọi "kiến thức" và "kỹ năng" của LLM đến từ tiền huấn luyện; tinh chỉnh chỉ dạy mô hình *cách trình bày* — ví dụ: khi thấy một câu hỏi thì trả lời thẳng vào câu hỏi, theo văn phong của một trợ lý. Giống như một học sinh đã học giỏi kiến thức, chỉ cần được hướng dẫn vài buổi về cách trình bày bài thi là làm bài tốt, không cần học lại từ đầu. Vì vậy, chỉ khoảng 1 000 mẫu được chọn kỹ đã có thể đủ.

Một khái niệm liên quan đến thảo luận ở đây là **hiệu quả mẫu** (*sample efficiency*). Một phương pháp học máy được gọi là hiệu quả mẫu nếu nó có thể học tốt từ một số ít mẫu huấn luyện. Theo nghĩa này, tinh chỉnh theo chỉ dẫn là hiệu quả mẫu so với tiền huấn luyện. Từ góc nhìn học máy, các phương pháp hiệu quả mẫu có thể được xem là những cách hiệu quả để lấy mẫu không gian dữ liệu, và có lợi thế vì chúng tận dụng tối ưu dữ liệu khan hiếm. Những ý tưởng này cũng liên quan đến các bối cảnh học tăng cường, nơi hiệu quả mẫu đặc biệt quan trọng. Do đó, các kỹ thuật học dựa trên lấy mẫu, chẳng hạn nhiều thuật toán học tăng cường, có thể được hưởng lợi từ những cách tiếp cận hiệu quả mẫu này.

Ví dụ, trong căn chỉnh theo sở thích của con người, ta có thể lấy mẫu dữ liệu sở thích một cách hiệu quả thông qua mô hình phần thưởng [Liu và cộng sự, 2024b] hoặc cải thiện hiệu quả lấy mẫu trong việc học chính sách (*policy*) [Wang và cộng sự, 2024a].

10.2.4 Tổng quát hóa chỉ dẫn

Trong nhiều bài toán học máy và NLP, huấn luyện một mô hình có khả năng tổng quát hóa (*generalization*) là một mục tiêu cơ bản. Ví dụ, trong phân loại văn bản, ta muốn mô hình phân loại đúng những văn bản mới chưa từng thấy trong huấn luyện. Tuy nhiên, tổng quát hóa đặt ra thêm những thách thức trong tinh chỉnh theo chỉ dẫn. Ta muốn các LLM đã tinh chỉnh theo chỉ dẫn không chỉ sinh ra câu trả lời phù hợp cho các đầu vào khác nhau trong cùng một nhiệm vụ, mà còn thực hiện chính xác nhiều nhiệm vụ khác nhau được mô tả bằng những chỉ dẫn khác nhau. Để minh họa vấn đề này, xét một LLM $\Pr(\mathbf{y}|\mathbf{c},\mathbf{z})$, trong đó $\mathbf{c}$ là một chỉ dẫn, $\mathbf{z}$ là một đầu vào của người dùng, và $\mathbf{y}$ là đầu ra tương ứng của mô hình (tức là câu trả lời). Giả sử hiệu năng của mô hình này được đánh giá theo một độ đo, viết là $\text{Performance}(\Pr(\mathbf{y}|\mathbf{c},\mathbf{z}))$, gọi tắt là $\mathrm{P}(\mathbf{c},\mathbf{z},\mathbf{y})$. Một cách không hình thức, khi ta nói mô hình này có thể tổng quát hóa trong một nhiệm vụ cho trước (được chỉ định bởi chỉ dẫn $\mathbf{c}^*$), tức là tồn tại một ngưỡng ϵ nào đó sao cho hiệu năng trung bình trên các đầu vào mới lớn hơn giá trị này:

$$\frac{1}{|\mathcal{Z}|}\sum_{\mathbf{z}'\in\mathcal{Z}}\mathrm{P}(\mathbf{c}^*,\mathbf{z}',\mathbf{y}')>\epsilon, \tag{10.8}$$

trong đó $\mathcal{Z}$ là tập các đầu vào mới, $\mathbf{z}'$ là một đầu vào, và $\mathbf{y}'$ là đầu ra của mô hình tương ứng với đầu vào $\mathbf{z}'$.

Tương tự, ta có thể nói mô hình này tổng quát hóa được giữa các nhiệm vụ nếu hiệu năng trung bình trên tất cả các cặp chỉ dẫn–đầu vào lớn hơn một ngưỡng ϵ nào đó:

$$\frac{1}{|\mathcal{D}|}\sum_{(\mathbf{c}',\mathbf{z}')\in\mathcal{D}}\mathrm{P}(\mathbf{c}',\mathbf{z}',\mathbf{y}')>\epsilon, \tag{10.9}$$

trong đó $\mathcal{D}$ là tập các cặp chỉ dẫn–đầu vào mới.

Chú giải của người dịch: Đọc Phương trình (10.8) và (10.9)

Cả hai công thức đều chỉ là "điểm trung bình phải vượt ngưỡng". $|\mathcal{Z}|$ là số phần tử của tập $\mathcal{Z}$; tổng chia cho số phần tử chính là trung bình cộng quen thuộc trong thống kê. Ở (10.8), chỉ dẫn cố định ($\mathbf{c}^*$, ví dụ "Dịch sang tiếng Pháp"), chỉ thay đổi đầu vào: mô hình dịch tốt cả những câu chưa gặp. Ở (10.9), cả chỉ dẫn lẫn đầu vào đều mới: mô hình làm tốt cả những *loại việc* chưa gặp. Ví dụ: kiểm tra trên 4 cặp mới, điểm lần lượt 0.9, 0.7, 0.8, 0.6 thì trung bình là 0.75; nếu chọn ngưỡng $\epsilon = 0.7$ thì ta nói mô hình tổng quát hóa được.

Ở đây, ta phải xử lý sự biến thiên theo hai chiều: chỉ dẫn và đầu vào của người dùng. Điều này làm bài toán tổng quát hóa trở nên rất phức tạp, vì theo trực giác, một mô hình cần học từ rất nhiều nhiệm vụ và nhiều cặp đầu vào–đầu ra khác nhau ứng với mỗi nhiệm vụ để đạt được khả năng tổng quát hóa tốt. Như ta đã bàn nhiều lần trong sách này, đạt được khả năng tổng quát hóa như vậy tốn ít chi phí hơn nhiều so với tiền huấn luyện. Nhìn chung, tinh chỉnh LLM bằng dữ liệu chỉ dẫn–câu trả lời ở một mức độ nào đó có thể giúp mô hình bộc lộ khả năng làm theo chỉ dẫn trên các nhiệm vụ mới. Tuy vậy, người ta thường cho rằng vẫn cần một số nỗ lực nhất định để thích nghi LLM sao cho chúng hiểu và thực hiện được chỉ dẫn một cách rộng rãi.

Một cách để tổng quát hóa tinh chỉnh theo chỉ dẫn là tăng tính đa dạng của dữ liệu tinh chỉnh. Trong các nghiên cứu ban đầu về tinh chỉnh theo chỉ dẫn, các nhà nghiên cứu đã xây dựng nhiều tập dữ liệu

bao quát rất nhiều nhiệm vụ NLP và các chỉ dẫn khác nhau cho mỗi nhiệm vụ [Wang và cộng sự, 2022e; Sanh và cộng sự, 2022; Longpre và cộng sự, 2023]. Bằng cách chuyển các nhiệm vụ này về một định dạng thống nhất, ta có thể tinh chỉnh LLM với một số lượng mẫu đủ lớn. Ví dụ, một số tập dữ liệu tinh chỉnh theo chỉ dẫn chứa hơn 100 nhiệm vụ NLP và hơn một triệu mẫu. Tuy nhiên, các tập dữ liệu ban đầu này chủ yếu tập trung vào các bài toán học thuật sẵn có, chứ không phải những nhiệm vụ gặp trong ứng dụng thực tế. Nhiều nghiên cứu gần đây đã chuyển trọng tâm sang giải quyết các bài toán mới và thực tế hơn. Ví dụ, đã có nhiều quan tâm đến việc xây dựng các tập dữ liệu chứa những ví dụ minh họa lớn, phức tạp và các câu trả lời của những mô hình tiên tiến nhất (SOTA) cho truy vấn thực của người dùng [Wang và cộng sự, 2023d; Teknium, 2023].

Có lẽ việc dùng các tập dữ liệu tinh chỉnh lớn và đa dạng bắt nguồn từ những nỗ lực mở rộng quy mô LLM theo nhiều chiều khác nhau. Thật vậy, các định luật co giãn (*scaling laws*) đã được dùng rộng rãi làm động lực để phát triển rất nhiều LLM tinh chỉnh theo chỉ dẫn khác nhau. Và việc mở rộng quy mô tinh chỉnh theo chỉ dẫn để LLM làm theo được nhiều loại chỉ dẫn là hợp lý. Tuy nhiên, từ góc nhìn căn chỉnh LLM, mở rộng quy mô tinh chỉnh theo chỉ dẫn có thể không phải là cách hiệu quả nhất để đạt được khả năng tổng quát hóa.

Một vấn đề là tinh chỉnh theo chỉ dẫn dựa vào học có giám sát, vốn mô hình hóa sự tổng quát hóa giữa các nhiệm vụ dựa trên các ánh xạ chỉ dẫn–câu trả lời tường minh. Tuy nhiên, cách tiếp cận như vậy không nắm bắt được những sở thích tinh tế hay phức tạp của con người (ví dụ: giọng điệu, văn phong, hay chất lượng mang tính chủ quan), vì những điều này khó mã hóa thành dữ liệu chỉ dẫn–câu trả lời tường minh. Hơn nữa, hiệu năng tổng quát hóa bị giới hạn bởi tính đa dạng và chất lượng của tập dữ liệu chỉ dẫn–câu trả lời. Với những hạn chế này, ta muốn dùng thêm các mô hình sở thích như một bước tinh chỉnh bổ sung sau tinh chỉnh theo chỉ dẫn, để LLM có thể tổng quát hóa xa hơn (xem Mục 10.3).

Một quan điểm khác cho rằng một số ánh xạ chỉ dẫn–câu trả lời có thể đã được học trong tiền huấn luyện. Tuy nhiên, vì ta thường không biết chính xác dữ liệu nào được dùng trong tiền huấn luyện, khó mà đánh giá liệu các ánh xạ đó có còn cần được học trong tinh chỉnh hay không. Một câu hỏi liên quan là: khả năng tổng quát hóa ngoài phân phối (*out-of-distribution generalization*) chủ yếu đạt được trong tiền huấn luyện hay trong tinh chỉnh? Trả lời trực tiếp câu hỏi này nằm ngoài phạm vi chương này, nhưng người ta đã chỉ ra rằng tiền huấn luyện trên các tập dữ liệu lớn và đa dạng giúp cải thiện hiệu năng ngoài phân phối một cách hiệu quả [Hendrycks và cộng sự, 2020; Radford và cộng sự, 2021; Gunasekar và cộng sự, 2023]. Điều này đặt ra một vấn đề thú vị: nếu một LLM đã được tiền huấn luyện tốt ở quy mô lớn, tinh chỉnh có thể không còn thiết yếu đến thế cho việc tổng quát hóa ngoài phân phối, vì mô hình có thể đã gặp đủ sự biến thiên về phân phối. Điều này thúc đẩy các nhà nghiên cứu tinh chỉnh LLM với công sức vừa phải, hoặc tìm các phương pháp mới để đạt được khả năng làm theo chỉ dẫn. Ví dụ, như đã bàn ở tiểu mục trước, khả năng làm theo chỉ dẫn có thể đạt được bằng cách tinh chỉnh trên một số ít cặp chỉ dẫn–câu trả lời được chọn lọc kỹ [Zhou và cộng sự, 2023a], hoặc thậm chí bằng những phương pháp vốn không được thiết kế tường minh cho mục đích đó [Kung và Peng, 2023].

Chú giải của người dịch: "Ngoài phân phối" là gì?

Dữ liệu *trong phân phối* là dữ liệu "giống" với dữ liệu huấn luyện (cùng loại, cùng phong cách); dữ liệu *ngoài phân phối* là dữ liệu khác hẳn. Ví dụ: một mô hình chỉ được luyện trên các đề toán lớp 10, nay gặp một bài toán đố dạng truyện tranh — đó là tình huống ngoài phân phối. Mô hình tổng quát hóa ngoài phân phối tốt là mô hình vẫn làm tốt trong những tình huống "lạ" như vậy.

Thảo luận trên đưa ra hai chiến lược khác nhau: một là mở rộng quy mô tập dữ liệu tinh chỉnh để tăng tính đa dạng, hai là dùng các tập dữ liệu tinh chỉnh nhỏ nhưng thiết yếu để thích nghi LLM một cách hiệu quả. Tuy nhiên, trong thực tế, đưa vào các chỉ dẫn đa dạng thường có ích. Trong nhiều trường

hợp, ta cần thích nghi LLM cho những mục đích cụ thể. Nhưng LLM, vốn có thể đã mã hóa các ánh xạ làm theo chỉ dẫn rộng rãi trong tiền huấn luyện, có thể có xu hướng hành xử như một "người thực thi chỉ dẫn" đa năng ngay cả khi chỉ được tinh chỉnh vừa phải. Một hiện tượng thú vị là khi tinh chỉnh trên dữ liệu toán học, LLM thu được có thể không chuyên biệt hóa vào các đầu ra toán học. Thay vào đó, mô hình này vẫn có thể đáp lại bình thường các chỉ dẫn tổng quát, chẳng hạn nó có thể làm thơ nếu được yêu cầu [Hewitt, 2024]. Đây không phải là điều xấu, nhưng nó cho thấy LLM có thể không dễ thay đổi bản chất làm theo các chỉ dẫn tổng quát của mình. Trong trường hợp này, các bước thích nghi bổ sung với dữ liệu đa dạng hơn có thể giúp điều chỉnh cách LLM làm theo chỉ dẫn, đặc biệt là cho những nhiệm vụ mà ta nhắm tới.

10.2.5 Tổng quát hóa từ yếu đến mạnh

Cho đến giờ, ta đã khám phá nhiều phương pháp tinh chỉnh theo chỉ dẫn dựa trên dữ liệu có gán nhãn. Một hạn chế của nhiều phương pháp như vậy là chúng đòi hỏi dữ liệu được con người gán nhãn hoặc được sinh ra bởi các LLM mạnh, cả hai đều có thể cung cấp tín hiệu giám sát chính xác. Tuy nhiên, trong nhiều trường hợp, LLM ta đang có đã mạnh sẵn (hoặc ít nhất là xuất sắc ở một số khía cạnh giải quyết vấn đề), nên không dễ tìm được một mô hình giỏi hơn để giám sát nó. Ngay cả với chuyên gia con người, khi bài toán trở nên phức tạp, việc đưa ra câu trả lời đúng và chi tiết có thể rất khó, đôi khi bất khả thi. Ví dụ, khi đối mặt với một văn bản cực kỳ dài, các chuyên gia sẽ thấy khó phát hiện bất kỳ điểm thiếu nhất quán, thiên kiến tinh vi hay ý chính bị bỏ sót nào nếu không tiến hành một đợt rà soát toàn diện và tốn thời gian.

Đến đây có thể đặt câu hỏi: liệu ta có thể dùng các LLM yếu để giám sát các LLM mạnh? Đây có vẻ là một thách thức lớn, nhưng nó có thể phản ánh một viễn cảnh tương lai, khi ta cần giám sát những hệ thống AI thông minh hơn con người hay các hệ thống AI hiện có [Burns và cộng sự, 2023b]. Bài toán dùng các mô hình nhỏ hơn, kém phức tạp hơn để cải thiện việc huấn luyện các mô hình lớn hơn, phức tạp hơn còn được gọi là bài toán **tổng quát hóa từ yếu đến mạnh** (*weak-to-strong generalization*). Dù hiện chưa có các cách tiếp cận hoàn thiện cho tổng quát hóa từ yếu đến mạnh, việc dùng các mô hình nhỏ hơn để hỗ trợ các mô hình mạnh hơn quả thực đã tỏ ra hữu ích trong một số lĩnh vực của LLM.

Đối với tinh chỉnh theo chỉ dẫn, một trong những cách đơn giản nhất để tận dụng LLM yếu là dùng các mô hình này để sinh dữ liệu tinh chỉnh tổng hợp. Giả sử ta có một tập các đầu vào X, trong đó mỗi đầu vào gồm một chỉ dẫn và, nếu cần, một đầu vào của người dùng. Với mỗi $\mathbf{x} \in X$, ta dùng một LLM yếu $\Pr^w(\cdot)$ để sinh ra một dự đoán $\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \Pr^w(\mathbf{y}|\mathbf{x})$. Sau đó, LLM mạnh $\Pr^s_\theta(\cdot)$ có thể được huấn luyện trên các dự đoán này (xem Phương trình (10.1)):

$$\tilde{\theta} \;=\; \arg\max_{\theta} \sum_{\mathbf{x}\in X} \log \Pr^s_\theta(\hat{\mathbf{y}}|\mathbf{x}), \tag{10.10}$$

trong đó θ biểu diễn các tham số của mô hình.

Dạng trên biến bài toán tinh chỉnh thành một bài toán chưng cất tri thức (*knowledge distillation*); nói cách khác, ta chưng cất tri thức từ mô hình yếu sang mô hình mạnh. Do đó, ta có thể dùng nhiều phương pháp chưng cất tri thức khác nhau để đạt mục tiêu này. Tuy nhiên, giải thích tinh chỉnh từ yếu đến mạnh từ góc nhìn chưng cất tri thức không hề đơn giản. Một mối lo ngại lớn là mô hình mạnh có thể chỉ bắt chước hoặc quá khớp các lỗi của mô hình yếu và không tổng quát hóa được. Ví dụ, mô hình mạnh sau tinh chỉnh vẫn không giải được những bài toán khó mà mô hình yếu không dự đoán chính xác được. May mắn là các thí nghiệm bước đầu theo hướng nghiên cứu này đã cho kết quả tích cực và đầy hứa hẹn. Ví dụ, Burns và cộng sự [2023a] phát hiện rằng tinh chỉnh các mô hình cỡ GPT-4 với sự giám

sát từ mô hình cỡ GPT-2 có thể cải thiện khả năng tổng quát hóa trên một số nhiệm vụ NLP. Để đo mức độ mô hình yếu cải thiện khả năng tổng quát hóa của mô hình mạnh, ta định nghĩa các đại lượng sau:

- **Hiệu năng yếu** (*Weak Performance*, P_{weak}). Đây là hiệu năng trên tập kiểm tra của mô hình yếu, có thể xem là hiệu năng cơ sở.
- **Hiệu năng từ yếu đến mạnh** (*Weak-to-strong Performance*, $\mathrm{P}_{\text{weak}\to\text{strong}}$). Đây là hiệu năng trên tập kiểm tra của mô hình mạnh được tinh chỉnh bằng mô hình yếu.
- **Hiệu năng trần của mô hình mạnh** (*Strong Ceiling Performance*, $\mathrm{P}_{\text{ceiling}}$). Đây là hiệu năng trên tập kiểm tra của mô hình mạnh được tinh chỉnh bằng dữ liệu nhãn đúng (*ground truth*). Ví dụ, ta tinh chỉnh mô hình mạnh bằng dữ liệu do con người gán nhãn và lấy mô hình thu được làm mức trần.

Khi đó, **tỉ lệ khoảng cách hiệu năng được thu hồi (PGR)** (*performance gap recovered*) có thể được định nghĩa là

$$\text{PGR} \;=\; \max\left\{0,\; \frac{\mathrm{P}_{\text{weak}\to\text{strong}} - \mathrm{P}_{\text{weak}}}{\mathrm{P}_{\text{ceiling}} - \mathrm{P}_{\text{weak}}}\right\}. \tag{10.11}$$

Độ đo này cho biết mô hình từ yếu đến mạnh có thể thu hồi được bao nhiêu phần của khoảng cách giữa hiệu năng trần và hiệu năng của mô hình yếu. PGR bằng 1 cho thấy tinh chỉnh từ yếu đến mạnh có thể lấp hoàn toàn khoảng cách hiệu năng, còn PGR bằng 0 cho thấy không có cải thiện nào. Trong nghiên cứu của Burns và cộng sự [2023a], PGR được chỉ ra là có thể đạt khoảng 0.8 trên 22 nhiệm vụ phân loại NLP. Cần lưu ý rằng, dù tiềm năng của tinh chỉnh từ yếu đến mạnh là đầy hứa hẹn, việc đạt được khả năng tổng quát hóa từ yếu đến mạnh đáng kể vẫn là một mục tiêu đầy thách thức cần được nghiên cứu thêm [Aschenbrenner, 2024].

Chú giải của người dịch: Tính thử PGR

Giả sử trên một bài kiểm tra: mô hình yếu đúng 60% ($\mathrm{P}_{\text{weak}} = 0.6$); mô hình mạnh học từ nhãn đúng của con người đạt 90% ($\mathrm{P}_{\text{ceiling}} = 0.9$); mô hình mạnh học từ "bài giải" của mô hình yếu đạt 84% ($\mathrm{P}_{\text{weak}\to\text{strong}} = 0.84$). Khoảng cách cần lấp là $0.9 - 0.6 = 0.3$; phần đã lấp được là $0.84 - 0.6 = 0.24$. Vậy $\text{PGR} = 0.24/0.3 = 0.8$: học trò giỏi học từ thầy kém hơn mình mà vẫn lấy lại được 80% khoảng cách so với khi học từ đáp án chuẩn. Phép $\max\{0, \cdot\}$ chỉ để PGR không âm khi mô hình mạnh còn tệ hơn cả mô hình yếu.

Tinh chỉnh LLM bằng giám sát yếu chỉ là một lựa chọn trong việc dùng mô hình nhỏ để cải thiện mô hình lớn. Mặc dù mục này chủ yếu tập trung vào tinh chỉnh LLM, ta cũng đề cập thêm các phương pháp khác ở đây để phần thảo luận đầy đủ hơn (xem minh họa các phương pháp này ở Hình 10.5).

- Thay vì dùng mô hình nhỏ để sinh dữ liệu tổng hợp, ta cũng có thể đơn giản đưa vào một hàm mất mát chưng cất tri thức dựa trên các mô hình này. Ví dụ, một hàm mất mát đơn giản đo sự khác biệt giữa mô hình nhỏ và mô hình lớn có thể được định nghĩa là:

$$\mathcal{L}_{\text{kd}} \;=\; \mathrm{D}_{\text{KL}}\big(\Pr^{w}(\cdot|\mathbf{x}) \,\|\, \Pr^{s}_{\theta}(\cdot|\mathbf{x})\big), \tag{10.12}$$

  hàm này khuyến khích phân phối của mô hình mạnh khớp với phân phối của mô hình yếu. Sau đó, ta có thể cộng hàm mất mát này vào hàm mất mát mô hình hóa ngôn ngữ ban đầu và thu được mục tiêu huấn luyện sau

$$\tilde{\theta} \;=\; \arg\max_{\theta} \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \log \Pr^{s}_{\theta}(\mathbf{y}|\mathbf{x}) - \lambda \cdot \mathcal{L}_{\text{kd}}, \tag{10.13}$$

  trong đó $\mathcal{D}$ là tập các cặp đầu vào và đầu ra, và λ là hệ số nội suy (*interpolation coefficient*). Phương pháp này có thể được dùng trong giai đoạn tiền huấn luyện hoặc tinh chỉnh. Ta có thể

điều chỉnh λ để kiểm soát mức độ ảnh hưởng của mô hình nhỏ lên quá trình huấn luyện. Ví dụ, ta có thể giảm dần λ để việc huấn luyện dựa nhiều hơn vào hàm mất mát mô hình hóa ngôn ngữ ban đầu khi mô hình lớn trở nên giỏi hơn.

- Một cách khác để đưa mô hình nhỏ vào tiền huấn luyện và tinh chỉnh LLM là dùng chúng để chọn lọc hoặc lọc dữ liệu. Với một chuỗi cho trước, ta có thể tính độ hợp lý (likelihood) hoặc entropy chéo bằng một mô hình nhỏ. Các đại lượng này sau đó có thể được dùng làm tiêu chí để chọn lọc hoặc lọc dữ liệu. Ví dụ, các chuỗi có độ hợp lý thấp hoặc entropy chéo cao có thể bị loại khỏi tập huấn luyện, vì chúng ít khớp với phân phối mà mô hình nhỏ đã học. Ngược lại, các chuỗi có độ hợp lý cao hoặc entropy chéo thấp có thể được ưu tiên, bảo đảm việc huấn luyện tập trung vào dữ liệu liên quan hơn hoặc chất lượng cao hơn.

- Học kết hợp (*ensemble learning*) là một cách đơn giản và hiệu quả để xây dựng một mô hình mạnh bằng cách kết hợp nhiều mô hình yếu. Áp dụng kỹ thuật này cho LLM rất đơn giản. Ta có thể gộp các phân phối được dự đoán bởi nhiều mô hình nhỏ hoặc các mô hình con chuyên biệt, rồi rút ra dự đoán cuối cùng từ kết quả đã gộp. Việc gộp này có thể được thực hiện bằng các phương pháp như bỏ phiếu đa số (*majority voting*), trung bình có trọng số (*weighted averaging*), hoặc xếp chồng (*stacking*).

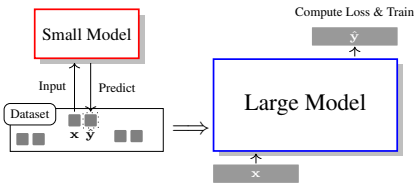
- Mô hình nhỏ cũng có thể được dùng lúc suy luận để cải thiện hiệu quả tổng thể. Giả sử ta có một mô hình lớn chậm nhưng chính xác hơn, và một mô hình nhỏ nhanh nhưng kém chính xác hơn. Trong phân tầng mô hình (*model cascading*), mô hình nhỏ xử lý dữ liệu đầu vào trước, nhanh chóng sinh ra kết quả sơ bộ. Nếu các kết quả này thỏa mãn một số tiêu chí định trước, chúng có thể được dùng trực tiếp. Tuy nhiên, nếu kết quả ban đầu chưa đủ tốt, đầu vào sẽ được chuyển cho mô hình lớn hơn, chính xác hơn để tạo ra kết quả tốt hơn. Cách tiếp cận này giảm đáng kể chi phí tính toán và độ trễ, vì mô hình nhỏ có thể tự xử lý hiệu quả nhiều đầu vào mà không cần đến mô hình lớn.

Chú giải của người dịch: Nhãn trong Hình 10.5 và ý nghĩa của Phương trình (10.12)

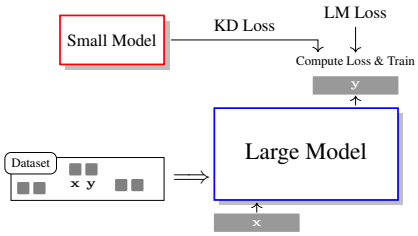
Các nhãn: *Small/Large Model* = mô hình nhỏ/lớn; *Dataset* = tập dữ liệu; *Predict* = dự đoán; *Compute Loss & Train* = tính mất mát và huấn luyện; *KD Loss* = mất mát chưng cất tri thức; *LM Loss* = mất mát mô hình hóa ngôn ngữ; *Data Selection* = chọn lọc dữ liệu; *Combination Model* = mô hình kết hợp; *Step 1 (cheap)/Step 2 (expensive)* = bước 1 (rẻ)/bước 2 (đắt); dòng chữ trên cùng của (e): "Nếu bước 1 chưa đạt yêu cầu, chuyển sang bước 2"; *Cascading (at inference time)* = phân tầng (lúc suy luận).
$\mathrm{D}_{\mathrm{KL}}(P \,\|\, Q)$ là *độ phân kỳ Kullback–Leibler*, một con số không âm đo hai phân phối xác suất P và Q "khác nhau" bao nhiêu; nó bằng 0 khi hai phân phối trùng nhau. Trong (10.13), ta vừa muốn mô hình mạnh đoán đúng dữ liệu (số hạng log-xác suất), vừa muốn nó không lệch quá xa "lời khuyên" của mô hình nhỏ (trừ đi $\lambda \cdot \mathcal{L}_{\mathrm{kd}}$). λ lớn thì nghe theo mô hình nhỏ nhiều hơn; $\lambda = 0$ thì bỏ qua mô hình nhỏ hoàn toàn.

10.3 Căn chỉnh theo sở thích của con người: RLHF

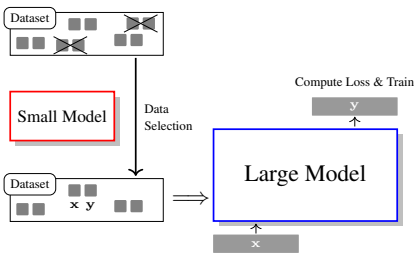
Cho đến giờ trong chương này, chúng ta tập trung vào việc tinh chỉnh LLM bằng các cặp đầu vào–đầu ra có gán nhãn. Cách tiếp cận này cho phép ta điều chỉnh LLM để làm theo chỉ dẫn thông qua học có giám sát. Tuy nhiên, trong nhiều ứng dụng, LLM không chỉ cần làm theo chỉ dẫn mà còn cần hành xử theo những cách phù hợp hơn với các giá trị và sở thích của con người. Hãy xét tình huống một người dùng hỏi LLM cách xâm nhập trái phép vào một hệ thống máy tính. Nếu LLM không được căn chỉnh thích đáng, nó có thể trả lời bằng cách cung cấp chi tiết về cách thực hiện hành vi phạm pháp này. Thay



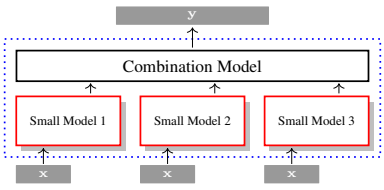
(a) Fine-tuning on data generated by a small model (weak-to-strong generalization)



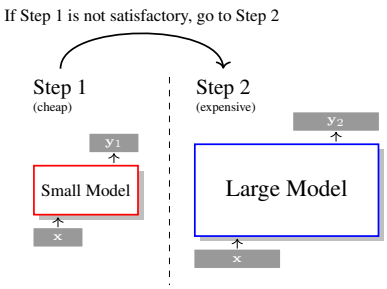
(b) Fine-tuning with KD Loss from a small model (weak-to-strong generalization)



(c) Data selection with a small model



(d) Ensemble of multiple small models



(e) Cascading (at inference time)

Hình 10.5: Minh họa các cách dùng mô hình nhỏ để cải thiện mô hình lớn trong LLM. Một cách tiếp cận là dùng các mô hình nhỏ hơn cho việc tinh chỉnh hoặc tiền huấn luyện các mô hình lớn hơn. Cách này bao gồm sinh dữ liệu tổng hợp (a), đưa vào hàm mất mát phụ trợ (b), và chọn lọc dữ liệu phù hợp (c). Một cách tiếp cận khác là kết hợp mô hình nhỏ và mô hình lớn. Cách này bao gồm học một mô hình mạnh bằng cách gộp nhiều mô hình nhỏ (d), và phân tầng mô hình nhỏ với mô hình lớn (e).

vào đó, một câu trả lời đáng mong muốn hơn có thể là khuyên người dùng không tham gia vào các hoạt động phạm pháp và đưa ra cái nhìn tổng quát về hậu quả của những hành vi như vậy. Khó khăn trong việc đạt được điều này là những sắc thái đạo đức và các cân nhắc theo ngữ cảnh mà LLM cần có để phản hồi phù hợp trong các tình huống như thế thường rất khó mã hóa một cách tường minh vào dữ liệu tinh chỉnh. Khó khăn hơn nữa là chính con người nhiều khi cũng không diễn đạt được chính xác sở thích của bản thân.

Trong mục này, chúng ta bàn về một phương pháp tinh chỉnh LLM khác, gọi là học tăng cường từ phản hồi của con người (*reinforcement learning from human feedback*), gọi tắt là RLHF [Christiano và cộng sự, 2017; Stiennon và cộng sự, 2020]. Ý tưởng cơ bản của RLHF là LLM có thể học từ việc so sánh các đầu ra của mô hình nhờ mô hình phần thưởng (*reward model*) (xem Hình 10.6). Để làm điều này, ta có thể thuê các chuyên gia là con người chỉ ra họ thích đầu ra nào hơn trong một cặp đầu ra của mô hình. Dữ liệu sở thích này được dùng để huấn luyện một mô hình phần thưởng có khả năng dự đoán chất lượng (theo cảm nhận của con người) của các đầu ra của LLM. Sau khi được huấn luyện, mô hình phần thưởng cung cấp phản hồi bằng cách chấm điểm cho các đầu ra mới mà LLM sinh ra đáp lại các đầu vào. LLM dùng các điểm số này để cập nhật tham số của mình thông qua các thuật toán học tăng cường (*reinforcement learning*). Trong phần còn lại của mục này, trước tiên chúng tôi giới thiệu các khái niệm cơ bản của học tăng cường để thuận tiện cho việc thảo luận, sau đó bàn về các phương pháp huấn luyện mô hình phần thưởng và căn chỉnh LLM bằng các mô hình này.

Chú giải của người dịch: Vì sao không chỉ dùng tinh chỉnh có giám sát?

Trong tinh chỉnh có giám sát (SFT), ta phải *viết sẵn* câu trả lời "chuẩn" cho từng câu hỏi. Nhưng với nhiều câu hỏi, rất khó viết ra một đáp án duy nhất đúng — trong khi việc *so sánh* hai câu trả lời và nói "cái này tốt hơn" lại dễ hơn nhiều. Giống như một giám khảo cuộc thi nấu ăn: ông ấy có thể không tự nấu được món hoàn hảo, nhưng nếm hai món thì dễ dàng nói món nào ngon hơn. RLHF tận dụng đúng khả năng "chấm so sánh" này của con người: (1) thu thập các phán đoán "A tốt hơn B"; (2) huấn luyện một mô hình phần thưởng bắt chước giám khảo; (3) cho LLM "luyện tập" để đạt điểm cao từ vị giám khảo máy đó.

10.3.1 Cơ bản về học tăng cường

Chúng ta bắt đầu bằng việc xem xét một số khái niệm cơ bản của học tăng cường. Lưu ý rằng ký hiệu dùng ở đây hơi khác so với ký hiệu ở các mục và chương trước, vì chúng tôi muốn cách trình bày nhất quán hơn với tài liệu về học tăng cường. Dù vậy, chúng tôi sẽ chỉ ra các ký hiệu này tương ứng với ký hiệu của mô hình hóa ngôn ngữ như thế nào. Bạn đọc đã quen với các kỹ thuật học tăng cường có thể bỏ qua hoặc đọc lướt tiểu mục này.

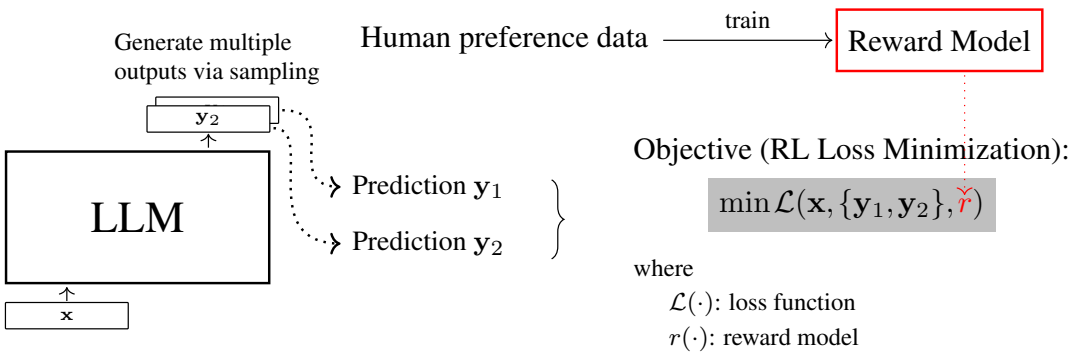
Chú giải của người dịch: Đọc các nhãn trong Hình 10.6

(a) *Supervised fine-tuning*: tinh chỉnh có giám sát; *Gold-standard Predictions (n one-hot distributions)*: dự đoán chuẩn (n phân phối one-hot); *Predicted Token Distributions*: các phân phối token mà mô hình dự đoán; *Objective (MLE):* $\max \Pr(\mathbf{y}|\mathbf{x})$: mục tiêu ước lượng hợp lý cực đại, với $\mathbf{x}$ là đầu vào (*input*), $\mathbf{y}$ là đầu ra chuẩn (*gold-standard output*). (b) *Generate multiple outputs via sampling*: sinh nhiều đầu ra bằng lấy mẫu; *Prediction* $\mathbf{y}_1$, $\mathbf{y}_2$: hai dự đoán; *Human preference data* $\xrightarrow{\text{train}}$ *Reward Model*: dữ liệu sở thích của con người dùng để huấn luyện mô hình phần thưởng; *Objective (RL Loss Minimization)*: mục tiêu là cực tiểu hóa mất mát học tăng cường $\mathcal{L}(\cdot)$, trong đó $r(\cdot)$ là mô hình phần thưởng.

Một khung học tăng cường tổng quát mô tả cách một tác tử (*agent*) tương tác với một môi trường (*environment*) động. Sự tương tác này được mô hình hóa thành một chuỗi các hành động (*action*) mà



(a) Supervised fine-tuning (maximizing the prediction probability given the input)



(b) Reinforcement Learning from Human Feedback

Hình 10.6: Tinh chỉnh có giám sát so với học tăng cường từ phản hồi của con người. Trong tinh chỉnh có giám sát, ta tối ưu LLM bằng cách cực đại hóa xác suất của dự đoán khi biết đầu vào. Trong học tăng cường từ phản hồi của con người, trước tiên ta huấn luyện một mô hình phần thưởng trên dữ liệu sở thích của con người (với mỗi cặp dự đoán, người đánh giá được yêu cầu chọn cái mà họ thích hơn). Sau đó, ta dùng mô hình phần thưởng này để giám sát LLM trong quá trình tinh chỉnh.

tác tử thực hiện để đáp lại trạng thái (*state*) của môi trường. Tại mỗi bước thời gian, tác tử quan sát trạng thái hiện tại, chọn một hành động dựa trên chính sách của nó, thực hiện hành động đó, rồi nhận phản hồi từ môi trường dưới dạng một phần thưởng và một trạng thái mới. Quá trình quan sát–hành động–phần thưởng này được lặp lại cho đến khi tác tử đạt được mục tiêu của mình.

Một hệ thống học tăng cường gồm một số thành phần:

- **Tác tử**. Đây là người học hay người ra quyết định trong học tăng cường. Trong bối cảnh LLM, có thể xem tác tử chính là LLM.
- **Môi trường**. Môi trường bao gồm mọi thứ bên ngoài tác tử mà tác tử tương tác cùng. Tuy nhiên, môi trường trong LLM không hẳn là một không gian vật lý hay ảo, mà đúng hơn là cái khung trong đó tác tử (ví dụ một LLM) nhận phản hồi và học.
- **Trạng thái** (s). Một trạng thái biểu diễn tình huống hiện tại của môi trường. Với một chuỗi token trong mô hình hóa ngôn ngữ, trạng thái tại một bước thời gian có thể được xem là các token ta đã quan sát được cho đến lúc đó, tức là các token ngữ cảnh mà ta dùng để dự đoán token tiếp theo. Ví dụ, ta có thể định nghĩa $(\mathbf{x}, \mathbf{y}_{<t})$ là trạng thái khi dự đoán token tiếp theo ở bước thời gian t.
- **Hành động** (a). Hành động biểu diễn các quyết định khả dĩ mà tác tử có thể đưa ra. Trong bối cảnh LLM, các hành động tương ứng với các token ứng viên trong bộ từ vựng.
- **Phần thưởng** (R). Phần thưởng là phản hồi từ môi trường, đánh giá mức độ thành công của một hành động. Ví dụ, $r(s, a, s')$ ký hiệu phần thưởng mà tác tử nhận được khi thực hiện hành động a ở trạng thái s và chuyển sang trạng thái tiếp theo s'. Nếu đã cho chuỗi trạng thái–hành động, ta có thể ký hiệu phần thưởng tại bước thời gian t là $r_t = r(s_t, a_t, s_{t+1})$. Cũng lưu ý rằng nếu quá trình ra quyết định là tất định, ta có thể bỏ s_{t+1} vì nó được xác định bởi s_t và a_t. Trong những trường hợp như vậy, ta có thể dùng $r(s_t, a_t)$ làm cách viết tắt cho $r(s_t, a_t, s_{t+1})$.
- **Chính sách** (π). Với một LLM, chính sách được định nghĩa là phân phối xác suất trên các token mà LLM dự đoán, khi biết các token ngữ cảnh đứng trước. Một cách hình thức, điều này có thể được biểu diễn là

$$\pi(a|s) \;=\; \Pr(y_t|\mathbf{x}, \mathbf{y}_{<t}), \tag{10.14}$$

  trong đó a tương ứng với token y_t, và s tương ứng với ngữ cảnh $(\mathbf{x}, \mathbf{y}_{<t})$. Hình 10.7 minh họa cách một LLM có thể được xem như một chính sách trong khung học tăng cường.
- **Hàm giá trị** (V và Q). Một **hàm giá trị trạng thái** (*state-value function*) (gọi tắt là hàm giá trị (*value function*)) đánh giá lợi tức chiết khấu (*discounted return*) kỳ vọng (tức tổng phần thưởng tích lũy) của một tác tử bắt đầu từ một trạng thái s cụ thể và đi theo một chính sách π cụ thể. Nó được định nghĩa là:

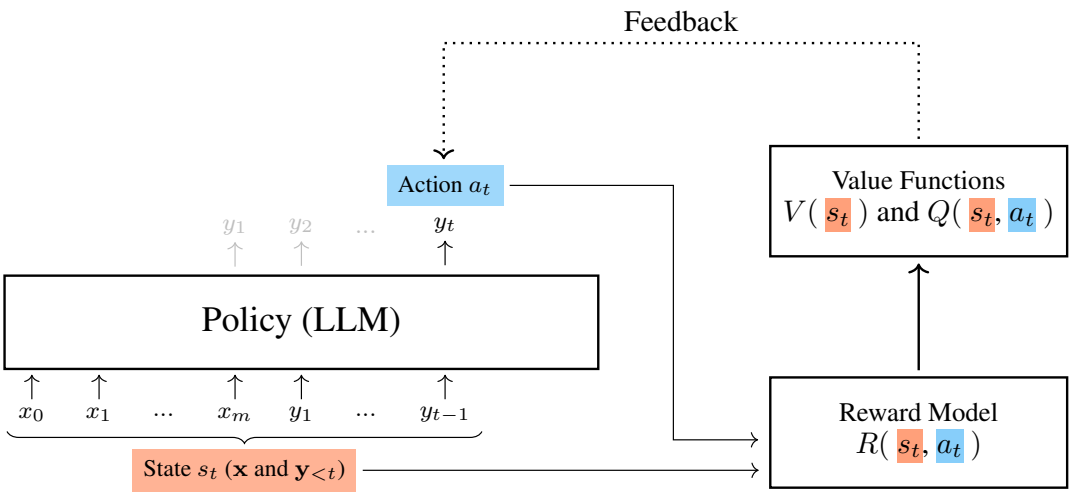
$$\begin{aligned} V(s) &= \mathbb{E}\Big[r(s_0, a_0, s_1) + \gamma r(s_1, a_1, s_2) + \gamma^2 r(s_2, a_2, s_3) + \cdots \;\Big|\; s_0 = s, \pi\Big] \\ &= \mathbb{E}\Big[r_0 + \gamma r_1 + \gamma^2 r_2 + \cdots \;\Big|\; s_0 = s, \pi\Big] \\ &= \mathbb{E}\Big[\sum_{t=0}^{\infty} \gamma^t r_t \;\Big|\; s_0 = s, \pi\Big], \end{aligned} \tag{10.15}$$

  trong đó $\gamma \in [0, 1]$ là hệ số chiết khấu (*discount factor*), điều chỉnh tầm quan trọng của các phần thưởng tương lai; $s_0 = s$ chỉ ra rằng tác tử bắt đầu từ trạng thái s; và kỳ vọng $\mathbb{E}$ được lấy trên mọi quỹ đạo (*trajectory*) khả dĩ (tức các chuỗi trạng thái–hành động). Tương tự, một **hàm giá trị**

hành động (*action-value function*) (hay **hàm giá trị Q** (*Q-value function*)) đo lợi tức kỳ vọng khi bắt đầu từ trạng thái s, thực hiện hành động a, và sau đó đi theo chính sách π, được cho bởi

$$Q(s,a) \;=\; \mathbb{E}\Big[\sum_{t=0}^{\infty}\gamma^t r_t \;\Big|\; s_0 = s, a_0 = a, \pi\Big], \tag{10.16}$$

trong đó $a_0 = a$ chỉ ra rằng hành động được thực hiện ở trạng thái ban đầu là a.



Hình 10.7: LLM đóng vai trò chính sách trong học tăng cường. Tại mỗi bước t, LLM dự đoán một token y_t khi biết đầu vào mô hình $\mathbf{x}$ và các token đã sinh trước đó $\mathbf{y}_{<t}$. Quá trình này có thể được phát biểu thành một bài toán học tăng cường, trong đó y_t đóng vai trò hành động, $(\mathbf{x}, \mathbf{y}_{<t})$ là trạng thái, và phân phối dự đoán $\Pr(y_t|\mathbf{x}, \mathbf{y}_{<t})$ là chính sách. Mô hình phần thưởng đánh giá câu trả lời được sinh ra và gán điểm phần thưởng, rồi các điểm này được dùng để ước lượng các hàm giá trị như $V(s_t)$ và $Q(s_t, a_t)$ phục vụ việc tối ưu chính sách. Trong thực tế, mô hình phần thưởng trong RLHF thường được định nghĩa trên toàn bộ câu trả lời, dù mục tiêu tối ưu vẫn có thể được phân rã theo từng token.

Chú giải của người dịch: Học tăng cường qua ví dụ chú chó học lệnh

Hãy hình dung bạn dạy một chú chó ngồi theo lệnh.

- **Tác tử** là chú chó (người học); **môi trường** là mọi thứ xung quanh, kể cả bạn.
- **Trạng thái** là tình huống hiện tại: "chủ vừa hô *Ngồi!*, tay đang cầm bánh".
- **Hành động** là việc chó chọn làm: ngồi, sủa, chạy đi…
- **Phần thưởng** là phản hồi: được bánh $(+1)$ hoặc không được gì (0).
- **Chính sách** là "thói quen" của chó: ở mỗi trạng thái, nó có xác suất bao nhiêu để chọn từng hành động, ví dụ $\pi(\text{ngồi}|\text{nghe lệnh}) = 0.7$, $\pi(\text{sủa}|\text{nghe lệnh}) = 0.3$. Học tăng cường là dần điều chỉnh các xác suất này để được nhiều bánh hơn.

Với LLM: *trạng thái* là câu hỏi cùng phần câu trả lời đã viết; *hành động* là chọn token tiếp theo; *chính sách* chính là phân phối xác suất trên bộ từ vựng mà LLM đưa ra; *phần thưởng* là điểm do mô hình phần thưởng chấm.

Chú giải của người dịch: Hàm giá trị, hệ số chiết khấu và kỳ vọng

Kỳ vọng $\mathbb{E}[\cdot]$ là "giá trị trung bình theo xác suất": nếu một trò chơi cho 10 điểm với xác suất 0.3 và 0 điểm với xác suất 0.7, thì kỳ vọng là $10 \cdot 0.3 + 0 \cdot 0.7 = 3$ điểm. Vì chính sách chọn hành động ngẫu nhiên, ta không biết trước tổng phần thưởng sẽ là bao nhiêu, nên lấy trung bình trên mọi khả năng.
Hệ số chiết khấu γ giống "lãi suất ngược": phần thưởng nhận ngay có giá trị hơn phần thưởng ở xa. Với $\gamma = 0.9$ và chuỗi phần thưởng $r_0 = 1, r_1 = 1, r_2 = 1$, lợi tức chiết khấu là $1 + 0.9 \cdot 1 + 0.81 \cdot 1 = 2.71$ (thay vì 3). Với $\gamma = 1$, mọi bước được coi trọng như nhau.
$V(s)$ trả lời câu hỏi: "Đang ở tình huống s, nếu cứ hành xử theo thói quen π thì trung bình về sau ta thu được bao nhiêu điểm?". $Q(s, a)$ hỏi chi tiết hơn: "Đang ở s, nếu *bước này* ta chọn a, rồi sau đó mới theo π, thì trung bình được bao nhiêu điểm?". Ví dụ khi chơi cờ: V là "thế cờ này tốt đến mức nào", còn Q là "nước đi này từ thế cờ này tốt đến mức nào".

Mục tiêu của học tăng cường là học một chính sách cực đại hóa **phần thưởng tích lũy** (*cumulative reward*) (hay **lợi tức** (*return*)) mà tác tử nhận được về lâu dài. Để đơn giản, trong các suy dẫn sau đây chúng tôi bỏ hệ số chiết khấu (tức $\gamma = 1$), trừ khi có nói khác. Cho một chuỗi trạng thái–hành động $\tau = \{(s_1, a_1), \ldots, (s_T, a_T)\}$[4], phần thưởng tích lũy trên chuỗi này có thể được viết là

$$R(\tau) \; = \; \sum_{t=1}^{T} r_t. \tag{10.17}$$

Kỳ vọng của phần thưởng tích lũy này trên một không gian các chuỗi trạng thái–hành động có dạng

$$\begin{aligned} J(\theta) &= \mathbb{E}_{\tau \sim \mathcal{T}} \Big[R(\tau) \,\Big|\, \pi_\theta \Big] \\ &= \sum_{\tau \in \mathcal{T}} \Pr_\theta(\tau) R(\tau) \\ &= \sum_{\tau \in \mathcal{T}} \Pr_\theta(\tau) \sum_{t=1}^{T} r_t, \end{aligned} \tag{10.18}$$

trong đó $\tau \sim \mathcal{T}$ chỉ ra rằng τ được lấy ra từ không gian các chuỗi trạng thái–hành động $\mathcal{T}$, và chỉ số dưới θ chỉ các tham số của chính sách. $J(\theta)$ còn được gọi là **hàm hiệu năng** (*performance function*). Khi đó, mục tiêu huấn luyện là cực đại hóa $J(\theta)$:

$$\tilde{\theta} \; = \; \arg\max_{\theta} J(\theta). \tag{10.19}$$

Chú giải của người dịch: Hàm hiệu năng $J(\theta)$ bằng một ví dụ số

Giả sử chính sách chỉ có thể sinh ra hai quỹ đạo: τ_1 với xác suất $\Pr_\theta(\tau_1) = 0.6$ và tổng phần thưởng $R(\tau_1) = 5$; τ_2 với xác suất 0.4 và $R(\tau_2) = -2$. Khi đó $J(\theta) = 0.6 \cdot 5 + 0.4 \cdot (-2) = 2.2$. Nếu thay đổi θ để xác suất của τ_1 tăng lên 0.9 thì $J = 0.9 \cdot 5 + 0.1 \cdot (-2) = 4.3$. Học tăng cường chính là tìm cách "dịch chuyển xác suất" về phía các quỹ đạo được thưởng cao.

Đến đây, ta có một cách tiếp cận học tăng cường đơn giản: 1) lấy mẫu một số chuỗi trạng thái–hành động; rồi 2) đánh giá từng chuỗi bằng hàm hiệu năng; rồi 3) cập nhật mô hình để cực đại hóa hàm hiệu

[4]Chúng tôi giả sử chuỗi trạng thái–hành động bắt đầu bằng s_1 và a_1 thay vì s_0 và a_0, để thống nhất với ký hiệu thường dùng trong chương này, nơi dự đoán **y** thường bắt đầu từ y_1. Dĩ nhiên, trong tài liệu cũng thường ký hiệu chuỗi trạng thái–hành động là $\{(s_0, a_0), \ldots, (s_T, a_T)\}$ hoặc $\{(s_0, a_0), \ldots, (s_{T-1}, a_{T-1})\}$. Nhưng sự khác biệt về ký hiệu này không ảnh hưởng đến việc thảo luận các mô hình được trình bày ở đây.

năng này. Nếu ta lấy Phương trình (10.18) và dùng hạ gradient để tối ưu chính sách, điều này tạo thành một phương pháp gradient chính sách (*policy gradient*) [Williams, 1992].

Lưu ý rằng trong nhiều bài toán NLP, chẳng hạn dịch máy, phần thưởng thường thưa (*sparse*). Ví dụ, phần thưởng chỉ được nhận ở cuối một câu hoàn chỉnh. Điều này có nghĩa là $r_t = 0$ với mọi $t < T$, và r_t chỉ khác không khi $t = T$. Lý tưởng nhất, người ta có thể muốn phản hồi đến ngay lập tức và thường xuyên (dày đặc), nhờ đó việc huấn luyện chính sách có thể dễ dàng và hiệu quả hơn. Dù đã có một số phương pháp được đề xuất để xử lý phần thưởng thưa, chẳng hạn định hình phần thưởng (*reward shaping*), trong phần thảo luận tiếp theo chúng tôi vẫn giả định thiết lập phần thưởng thưa, trong đó phần thưởng chỉ có được khi hoàn tất dự đoán.

Mô hình mô tả trong các Phương trình (10.17–10.19) thiết lập một dạng học tăng cường cơ bản, và nhiều biến thể cũng như cải tiến của mô hình này đã được phát triển. Trước khi trình bày những mô hình tinh vi hơn đó, hãy dành chút thời gian diễn giải hàm mục tiêu $J(\theta)$ từ góc nhìn gradient chính sách. Trong hạ gradient, ta cần tính gradient của $J(\theta)$ theo θ:

$$\begin{aligned}\frac{\partial J(\theta)}{\partial \theta} &= \frac{\partial \sum_{\tau \in \mathcal{T}} \Pr_\theta(\tau) R(\tau)}{\partial \theta} \\ &= \sum_{\tau \in \mathcal{T}} \frac{\partial \Pr_\theta(\tau)}{\partial \theta} R(\tau) \\ &= \sum_{\tau \in \mathcal{T}} \Pr_\theta(\tau) \frac{\partial \Pr_\theta(\tau)/\partial \theta}{\Pr_\theta(\tau)} R(\tau) \\ &= \sum_{\tau \in \mathcal{T}} \Pr_\theta(\tau) \frac{\partial \log \Pr_\theta(\tau)}{\partial \theta} R(\tau). \end{aligned} \tag{10.20}$$

Chú giải của người dịch: Mẹo đạo hàm logarit

Dòng cuối của Phương trình (10.20) dùng một quy tắc quen thuộc ở THPT: $(\ln u)' = \frac{u'}{u}$, nên $u' = u \cdot (\ln u)'$. Ở đây $u = \Pr_\theta(\tau)$ và đạo hàm lấy theo tham số θ (ký hiệu ∂ chỉ "đạo hàm riêng": đạo hàm theo một biến trong khi coi các biến khác là hằng số). Lợi ích của mẹo này: tổng $\sum_\tau \Pr_\theta(\tau)[\ldots]$ lại có dạng một *kỳ vọng*, nên ta có thể ước lượng nó bằng cách lấy mẫu vài quỹ đạo rồi tính trung bình, thay vì phải duyệt qua mọi quỹ đạo (là con số khổng lồ).

Trong thực tế, kỳ vọng thường được xấp xỉ bằng các quỹ đạo được lấy mẫu đều từ một lô (tức $\Pr_\theta(\tau) = 1/|\mathcal{T}|$). Khi đó ta có thể đơn giản hóa Phương trình (10.20) và chỉ cần xét các số hạng $\frac{\partial \log \Pr_\theta(\tau)}{\partial \theta}$ và $R(\tau)$:

$$\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{m} \sum_{\tau \in \mathcal{T}} \frac{\partial \log \Pr_\theta(\tau)}{\partial \theta} R(\tau). \tag{10.21}$$

Một ưu điểm của kết quả này là $R(\tau)$ không cần khả vi, nghĩa là ta có thể dùng bất kỳ loại hàm phần thưởng nào trong học tăng cường.

Chú giải của người dịch: Ý nghĩa của Phương trình (10.21)

Ở đây $\mathcal{T}$ là tập các quỹ đạo lấy mẫu trong lô và m là số quỹ đạo đó (tức $m = |\mathcal{T}|$). Công thức nói: với mỗi quỹ đạo, hãy đẩy tham số theo hướng làm tăng log-xác suất của quỹ đạo đó, với "độ mạnh" tỷ lệ với phần thưởng $R(\tau)$. Quỹ đạo được thưởng lớn được "khuyến khích" mạnh; quỹ đạo có phần thưởng âm bị "kìm hãm" (đẩy theo hướng ngược lại). Vì $R(\tau)$ chỉ đóng vai trò một con số nhân vào, nó có thể đến từ bất cứ đâu — một người chấm điểm, một chương trình kiểm tra đáp án, hay một mô hình phần thưởng — mà không cần

lấy đạo hàm của nó.

Bằng cách xem việc sinh ra chuỗi τ như một quá trình quyết định Markov (*Markov decision process*), ta có thể suy dẫn tiếp $\frac{\partial \log \Pr_\theta(\tau)}{\partial \theta}$ và thu được:

$$\begin{aligned}\frac{\partial \log \Pr_\theta(\tau)}{\partial \theta} &= \frac{\partial}{\partial \theta} \log \prod_{t=1}^{T} \pi_\theta(a_t|s_t) \Pr(s_{t+1}|s_t, a_t) \\ &= \frac{\partial}{\partial \theta} \sum_{t=1}^{T} \underbrace{\log \pi_\theta(a_t|s_t)}_{\text{chính sách}} + \frac{\partial}{\partial \theta} \sum_{t=1}^{T} \underbrace{\log \Pr(s_{t+1}|s_t, a_t)}_{\text{động lực}},\end{aligned} \tag{10.22}$$

trong đó gradient được phân rã thành hai phần: gradient chính sách và gradient động lực (*dynamics*). Thành phần chính sách, $\log \pi_\theta(a_t|s_t)$, xác định log-xác suất của việc thực hiện hành động a_t khi ở trạng thái s_t, và được tham số hóa bởi θ. Thành phần động lực, $\log \Pr(s_{t+1}|s_t, a_t)$, biểu diễn log-xác suất chuyển sang trạng thái s_{t+1} từ trạng thái s_t sau khi thực hiện hành động a_t. Trong các thiết lập học tăng cường điển hình, động lực không bị ảnh hưởng trực tiếp bởi tham số chính sách θ, do đó đạo hàm của chúng thường bằng không. Vì vậy, trong trường hợp này, Phương trình (10.22) có thể được đơn giản thành:

$$\frac{\partial \log \Pr_\theta(\tau)}{\partial \theta} = \frac{\partial}{\partial \theta} \sum_{t=1}^{T} \log \pi_\theta(a_t|s_t). \tag{10.23}$$

Nói cách khác, ta chỉ tập trung tối ưu chính sách mà không cần bận tâm đến động lực bên dưới.

Chú giải của người dịch: Quá trình quyết định Markov và "động lực"

"Markov" nghĩa là bước tiếp theo chỉ phụ thuộc vào hiện tại (s_t và a_t), không cần nhớ cả quá khứ. Xác suất của cả quỹ đạo là tích xác suất từng bước (như quy tắc nhân trong xác suất), và log của tích bằng tổng các log. Mỗi bước gồm hai việc: *tác tử chọn* hành động (phần chính sách π_θ, phụ thuộc θ) và *thế giới phản ứng* (phần động lực, không phụ thuộc θ). Như chơi cờ: bạn chỉ điều khiển nước đi của mình, còn đối thủ đi thế nào không thay đổi khi bạn "chỉnh" cách nghĩ của mình. Với LLM thì còn đơn giản hơn: trạng thái mới chỉ là trạng thái cũ nối thêm token vừa chọn, nên động lực là tất định.

Thay Phương trình (10.23) vào Phương trình (10.21), và khai triển $R(\tau)$, ta thu được

$$\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \frac{\partial}{\partial \theta} \Big(\sum_{t=1}^{T} \log \pi_\theta(a_t|s_t) \sum_{t=1}^{T} r_t \Big). \tag{10.24}$$

Dù cách tiếp cận gradient chính sách này đơn giản, nó gặp vấn đề là phương sai (*variance*) của các gradient ước lượng có thể rất cao, khiến quá trình học bị nhiễu và kém hiệu quả. Một lý do của vấn đề phương sai cao này là phần thưởng có thể biến động rất lớn giữa các bước hoặc các tình huống khác nhau. Hãy hình dung trong một chuỗi các quyết định hành động, mô hình phần thưởng có xu hướng gán phần thưởng nhỏ cho các hành động tốt (ví dụ $R_t = 2$) và mức phạt lớn cho các hành động kém (ví dụ $R_t = -50$). Những thang phần thưởng chênh lệch như vậy cho hành động tốt và kém có thể dẫn đến tổng phần thưởng rất thấp cho cả chuỗi, ngay cả khi chuỗi đó có chứa các hành động tốt.

Một phương pháp đơn giản để giảm phương sai của gradient là đặt một đường cơ sở (*baseline*) b và trừ nó khỏi $\sum_{t=1}^{T} r_t$, được $\sum_{t=1}^{T} r_t - b$.[5] Ở đây, đường cơ sở có thể được hiểu như một điểm tham

[5] Thực ra, việc dùng đường cơ sở b không làm thay đổi phương sai của tổng phần thưởng $\sum_{t=1}^{T} r_t$. Tuy nhiên, cần lưu ý rằng

chiếu. Bằng cách đưa phần thưởng về quanh đường cơ sở này, ta loại bỏ các thiên lệch có hệ thống trong tín hiệu phần thưởng, khiến các bước cập nhật ổn định hơn và ít nhạy cảm hơn với những dao động cực đoan của từng phần thưởng riêng lẻ.

Chú giải của người dịch: Vì sao trừ đi một "đường cơ sở"?

Giả sử mọi bài làm của bạn đều được chấm từ 80 đến 90 điểm. Nếu ta dùng điểm thô làm tín hiệu, mọi bài đều "được khen" (vì điểm dương), chỉ khác nhau mức độ — rất khó phân biệt bài nào thực sự tốt. Nếu trừ đi điểm trung bình $b = 85$, bài 90 điểm thành $+5$ (khuyến khích), bài 80 điểm thành -5 (kìm hãm): tín hiệu rõ ràng hơn nhiều. *Phương sai* (đã gặp trong thống kê mô tả) đo mức độ phân tán; khi tín hiệu dao động ít quanh 0, các bước cập nhật tham số đỡ "giật cục".

Mô hình gradient chính sách có đường cơ sở này có thể được cho bởi

$$\begin{aligned}\frac{\partial J(\theta)}{\partial \theta} &= \frac{1}{|\mathcal{T}|}\sum_{\tau\in\mathcal{T}}\frac{\partial}{\partial\theta}\Big(\sum_{t=1}^{T}\log\pi_\theta(a_t|s_t)\Big)\Big(\sum_{t=1}^{T}r_t - b\Big)\\ &= \frac{1}{|\mathcal{T}|}\sum_{\tau\in\mathcal{T}}\frac{\partial}{\partial\theta}\Big[\sum_{t=1}^{T}\log\pi_\theta(a_t|s_t)\Big(\sum_{k=1}^{T}r_k - b\Big)\Big]\\ &= \frac{1}{|\mathcal{T}|}\sum_{\tau\in\mathcal{T}}\frac{\partial}{\partial\theta}\Big[\sum_{t=1}^{T}\log\pi_\theta(a_t|s_t)\Big(\sum_{k=1}^{t-1}r_k + \sum_{k=t}^{T}r_k - b\Big)\Big].\end{aligned} \tag{10.25}$$

Ở đây ta viết $\sum_{k=1}^{T} r_k$ thành tổng của hai số hạng $\sum_{k=1}^{t-1} r_k$ và $\sum_{k=t}^{T} r_k$ để phân biệt giữa các phần thưởng tích lũy trước và sau hành động tại bước thời gian t. Lưu ý rằng các phần thưởng nhận được trước bước thời gian t không phụ thuộc vào hành động thực hiện tại bước t. Do đó, số hạng $\sum_{k=1}^{t-1} r_k$ không đóng góp vào gradient và có thể bỏ đi. Điều này dẫn tới một phiên bản đơn giản hơn của Phương trình (10.25)

$$\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{|\mathcal{T}|}\sum_{\tau\in\mathcal{T}}\frac{\partial}{\partial\theta}\Big[\sum_{t=1}^{T}\log\pi_\theta(a_t|s_t)\Big(\sum_{k=t}^{T}r_k - b\Big)\Big]. \tag{10.26}$$

Cũng lưu ý rằng việc bỏ $\sum_{k=1}^{t-1} r_k$ còn có thể giảm thêm phương sai của gradient.

Chú giải của người dịch: "Quá khứ không thể thay đổi"

Hành động ở bước t chỉ có thể ảnh hưởng đến những gì xảy ra *từ bước t trở đi*. Ví dụ, khi đánh giá một quyết định đổi chiến thuật ở phút 60 của trận bóng, ta chỉ nên tính các bàn thắng/thua từ phút 60 về sau; các bàn đã ghi trước đó không do quyết định này gây ra. Tổng $\sum_{k=t}^{T} r_k$ vì thế thường được gọi là "phần thưởng còn lại tính từ bước t" (*reward-to-go*).

Có nhiều cách để định nghĩa đường cơ sở b. Ở đây ta xét hàm giá trị của trạng thái s_t, tức là giá trị ước lượng của việc ở trạng thái s_t: $V(s_t) = \mathbb{E}(r_t + r_{t+1} + \cdots + r_T)$. Do đó ta có

$$\begin{aligned}A(s_t, a_t) &= \sum_{k=t}^{T} r_k - b\\ &= \sum_{k=t}^{T} r_k - V(s_t),\end{aligned} \tag{10.27}$$

mặc dù việc đưa vào đường cơ sở không làm thay đổi phương sai tổng thể của phần thưởng, nó giúp giảm phương sai của các gradient ước lượng. Nói chung, phép toán $\sum_{t=1}^{T} r_t - b$ đưa phần thưởng về quanh giá trị không (ví dụ b được định nghĩa là giá trị kỳ vọng của $\sum_{t=1}^{T} r_t$), điều này có thể làm giảm phương sai của tích $\sum_{t=1}^{T}\log\pi_\theta(a_t|s_t)(\sum_{t=1}^{T} r_t - b)$.

trong đó $\sum_{k=t}^{T} r_k$ biểu diễn lợi tức thực tế nhận được, còn $V(s_t)$ biểu diễn lợi tức kỳ vọng. $A(s_t, a_t)$ (viết tắt là A_t) được gọi là **lợi thế** (*advantage*) tại bước thời gian t; nó định lượng lợi ích tương đối của hành động a_t so với giá trị kỳ vọng của việc đi theo chính sách từ trạng thái s_t trở đi.

Bằng cách dùng hàm lợi thế $A(s_t, a_t)$, gradient của $J(\theta)$ có thể được viết dưới dạng

$$\frac{\partial J(\theta)}{\partial \theta} \;=\; \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \frac{\partial}{\partial \theta} \Big(\sum_{t=1}^{T} \log \pi_\theta(a_t|s_t) A(s_t, a_t) \Big). \tag{10.28}$$

Chú giải của người dịch: Lợi thế: "tốt hơn mong đợi bao nhiêu?"

Giả sử một học sinh thường đạt trung bình 7 điểm môn Toán (đây là $V(s_t)$ — mức kỳ vọng). Lần này, sau khi áp dụng một cách ôn mới (hành động a_t), bạn ấy được 9 điểm (lợi tức thực tế). Lợi thế là $A = 9 - 7 = +2$: cách ôn mới tốt hơn mong đợi, nên "tăng xác suất" dùng lại nó. Nếu chỉ được 6 điểm thì $A = -1$: nên giảm. Lưu ý: điều quan trọng không phải là điểm cao hay thấp tuyệt đối, mà là *cao hơn hay thấp hơn mức thường lệ*.

Mục tiêu này tạo nền tảng cho các phương pháp **tác nhân–phê bình có lợi thế** (*advantage actor-critic*) (**A2C**) trong học tăng cường [Mnih và cộng sự, 2016]. Trong phương pháp này, tác nhân (*actor*) có nhiệm vụ học một chính sách. Nó cập nhật các tham số chính sách bằng Phương trình (10.28) để tập trung nhiều hơn vào các hành động có khả năng cải thiện hiệu năng. Mặt khác, bộ phê bình (*critic*) cập nhật ước lượng của nó về hàm giá trị, thứ được dùng để tính hàm lợi thế $A(s_t, a_t)$, nhờ đó đóng vai trò người đánh giá chính sách mà tác nhân đang học.

Trong phương pháp A2C, $A(s_t, a_t)$ thường được biểu diễn là hiệu của hàm giá trị hành động $Q(s_t, a_t)$ và hàm giá trị trạng thái $V(s_t)$:

$$A(s_t, a_t) \;=\; Q(s_t, a_t) - V(s_t). \tag{10.29}$$

Thoạt nhìn, mô hình này có vẻ khó xây dựng vì nó đòi hỏi hai mô hình con riêng biệt để lần lượt tính $Q(s_t, a_t)$ và $V(s_t)$. May mắn thay, xét rằng $Q(s_t, a_t)$ có thể được định nghĩa là lợi tức $r_t + V(s_{t+1})$, ta có thể viết lại Phương trình (10.29) thành

$$A(s_t, a_t) \;=\; r_t + V(s_{t+1}) - V(s_t), \tag{10.30}$$

hoặc đưa thêm hệ số chiết khấu γ vào để có dạng tổng quát hơn

$$A(s_t, a_t) \;=\; r_t + \gamma V(s_{t+1}) - V(s_t). \tag{10.31}$$

$A(s_t, a_t) = r_t + \gamma V(s_{t+1}) - V(s_t)$ còn được gọi là sai số **chênh lệch thời gian** (*temporal difference*) (**TD**). Điều ta cần làm là huấn luyện một mạng phê bình cho hàm giá trị $V(s_t)$, rồi dùng nó để tính hàm lợi thế[6].

[6]Hàm mất mát huấn luyện cho mạng giá trị (hay mạng phê bình) trong A2C thường được phát biểu là sai số bình phương trung bình giữa lợi tức tính được $r_t + \gamma V(s_{t+1})$ và giá trị trạng thái dự đoán $V(s_t)$. Giả sử mạng giá trị được tham số hóa bởi ω. Hàm mất mát được cho bởi

$$\mathcal{L}_v(\omega) \;=\; \frac{1}{M} \sum \big(r_t + \gamma V_\omega(s_{t+1}) - V_\omega(s_t) \big)^2, \tag{10.32}$$

trong đó M là số mẫu huấn luyện; ví dụ, với một chuỗi gồm T token, ta có thể đặt $M = T$.

Chú giải của người dịch: Tác nhân–phê bình và sai số chênh lệch thời gian

Hãy nghĩ tới một cầu thủ (*tác nhân*) và huấn luyện viên (*bộ phê bình*). Cầu thủ quyết định đá thế nào; huấn luyện viên không đá, nhưng đánh giá "tình thế này đáng giá bao nhiêu" (V). Sau mỗi pha bóng, ta so sánh: *dự đoán trước đó* $V(s_t)$ với *dự đoán mới sau khi đã thấy một bước thực tế* $r_t + \gamma V(s_{t+1})$. Hiệu số giữa hai giá trị đó chính là sai số TD. Ví dụ: huấn luyện viên dự đoán tình thế hiện tại đáng $V(s_t) = 3$; cầu thủ chuyền bóng, nhận ngay $r_t = 0$, nhưng tình thế mới đáng $V(s_{t+1}) = 5$; với $\gamma = 1$, $A = 0 + 5 - 3 = +2 > 0$: đường chuyền tốt hơn mong đợi. Hay ở chỗ: ta không cần đợi đến cuối trận mới biết hành động có tốt không, chỉ cần nhìn một bước rồi "tin" vào ước lượng của bộ phê bình. Đồng thời, bộ phê bình tự cải thiện bằng cách làm cho sai số này nhỏ dần (Phương trình (10.32)).

Cho đến đây, chúng ta đã dành khá nhiều dung lượng để bàn về những điều cơ bản của học tăng cường, đặc biệt là cách suy dẫn mục tiêu tối ưu cho phương pháp A2C. Tuy nhiên, học tăng cường là một lĩnh vực rộng lớn, và nhiều chi tiết kỹ thuật không thể đề cập ở đây. Bạn đọc quan tâm có thể tham khảo các sách về học tăng cường để biết thêm chi tiết [Sutton và Barto, 2018; Szepesvári, 2010]. Dù vậy, giờ đây ta đã có đủ kiến thức cần thiết để thảo luận tiếp về RLHF. Trong các tiểu mục tiếp theo, chúng ta sẽ quay lại thảo luận về căn chỉnh LLM, cho thấy cách dùng phương pháp A2C để căn chỉnh theo sở thích của con người.

10.3.2 Huấn luyện mô hình phần thưởng

Chúng ta đã thấy rằng mô hình phần thưởng đóng vai trò rất quan trọng trong khung học tăng cường tổng quát và là cơ sở để tính các hàm giá trị. Giờ ta xét bài toán huấn luyện các mô hình phần thưởng này.

Trong RLHF, mô hình phần thưởng là một mạng nơ-ron ánh xạ một cặp chuỗi token đầu vào và đầu ra thành một điểm phần thưởng vô hướng. Cho một đầu vào $\mathbf{x}$ và một đầu ra $\mathbf{y}$, phần thưởng có thể được biểu diễn là

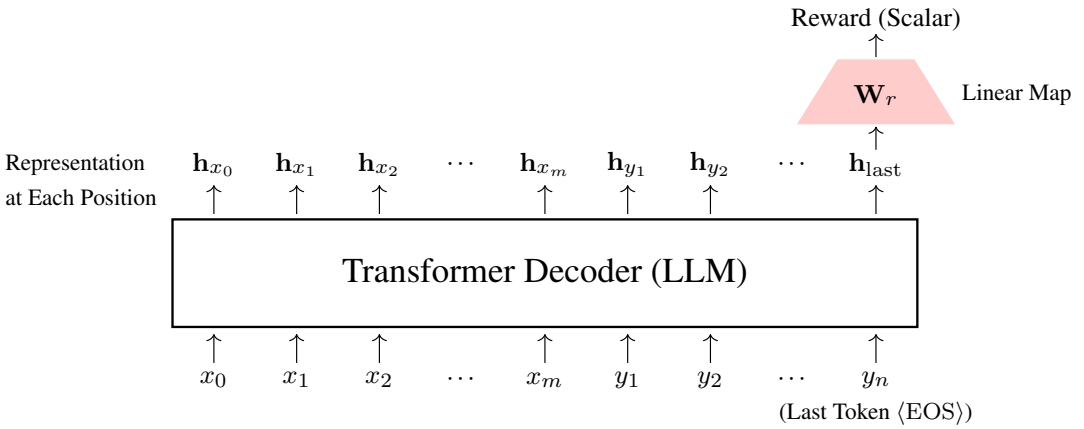
$$r \;=\; \text{Reward}(\mathbf{x}, \mathbf{y}), \tag{10.33}$$

trong đó $\text{Reward}(\cdot)$ là mô hình phần thưởng. r có thể được hiểu là thước đo mức độ đầu ra $\mathbf{y}$ phù hợp với hành vi mong muốn khi cho trước đầu vào $\mathbf{x}$. Như đã bàn ở tiểu mục trước, cả $\mathbf{x}$ và $\mathbf{y}$ đều được giả định là các chuỗi token hoàn chỉnh. Do đó, mô hình phần thưởng đánh giá chất lượng của một đầu ra đã được sinh trọn vẹn, có điều kiện theo đầu vào, chứ không gán phần thưởng cho các bước sinh trung gian. Ví dụ, trong quá trình sinh chuỗi, phần thưởng thường chỉ được cung cấp tại vị trí cuối cùng của chuỗi đầu ra $\mathbf{y} = y_1 \dots y_n$. Để ký hiệu đỡ rườm rà, từ đây trở đi ta sẽ dùng $r(\mathbf{x}, \mathbf{y})$ để ký hiệu mô hình phần thưởng $\text{Reward}(\mathbf{x}, \mathbf{y})$.

Có nhiều cách để hiện thực mô hình phần thưởng. Một cách đơn giản là xây dựng mô hình phần thưởng dựa trên một LLM đã tiền huấn luyện. Cụ thể hơn, ta có thể nối $\mathbf{x}$ và $\mathbf{y}$ để tạo thành một chuỗi token duy nhất $\text{seq}_{\mathbf{x},\mathbf{y}} = [\mathbf{x}, \mathbf{y}]$. Ta chạy một LLM đã tiền huấn luyện trên chuỗi này như thường lệ, và tại mỗi vị trí, ta thu được một biểu diễn từ tầng Transformer cuối cùng. Sau đó, ta lấy biểu diễn tại vị trí cuối cùng (ký hiệu là $\mathbf{h}_{\text{last}}$) và ánh xạ nó thành một số vô hướng thông qua một phép biến đổi tuyến tính:

$$r(\mathbf{x}, \mathbf{y}) \;=\; \mathbf{h}_{\text{last}} \mathbf{W}_r, \tag{10.34}$$

trong đó $\mathbf{h}_{\text{last}}$ là một vectơ d chiều, và $\mathbf{W}_r$ là một ma trận ánh xạ tuyến tính kích thước $d \times 1$. Kiến trúc này của mô hình phần thưởng được minh họa trong Hình 10.8.



Hình 10.8: Kiến trúc của một mô hình phần thưởng dựa trên Transformer. Thành phần chính của mô hình này vẫn là một LLM. Ta dùng bộ giải mã Transformer làm mô hình biểu diễn chuỗi. Ta lấy biểu diễn tại vị trí cuối cùng của bộ giải mã làm biểu diễn của cả chuỗi $[\mathbf{x}, \mathbf{y}]$. Sau đó ta ánh xạ biểu diễn này thành một số vô hướng thông qua một phép biến đổi tuyến tính; số này đóng vai trò điểm phần thưởng của $[\mathbf{x}, \mathbf{y}]$.

Chú giải của người dịch: Đọc Hình 10.8 và Phương trình (10.34)

Các nhãn trong hình: *Representation at Each Position* — biểu diễn tại mỗi vị trí; *Transformer Decoder (LLM)* — bộ giải mã Transformer (chính là LLM); *Last Token* $\langle EOS \rangle$ — token cuối cùng (ký hiệu kết thúc chuỗi); *Linear Map* — ánh xạ tuyến tính; *Reward (Scalar)* — phần thưởng (một số vô hướng). Nhân vectơ hàng $\mathbf{h}_{\text{last}}$ (có d số) với ma trận cột $\mathbf{W}_r$ (d số) chính là *tích vô hướng*: $r = h_1 w_1 + h_2 w_2 + \cdots + h_d w_d$. Ví dụ $d = 3$: $\mathbf{h}_{\text{last}} = [1,\ 2,\ -1]$, $\mathbf{W}_r = [0.5,\ 0.1,\ 0.3]^\top$ thì $r = 0.5 + 0.2 - 0.3 = 0.4$. Vì LLM đọc từ trái sang phải, token cuối "đã nhìn thấy" toàn bộ câu hỏi và câu trả lời, nên biểu diễn của nó là bản tóm tắt tốt của cả chuỗi.

Để huấn luyện mô hình phần thưởng, bước đầu tiên là thu thập phản hồi của con người trên một tập các đầu ra đã sinh. Cho một đầu vào $\mathbf{x}$, ta dùng LLM để tạo ra nhiều đầu ra ứng viên $\{\mathbf{y}_1, \ldots, \mathbf{y}_N\}$. Phản hồi của con người có thể được thu thập theo một số cách:

- **So sánh theo cặp (xếp hạng theo cặp**, *Pairwise Comparison / Pairwise Ranking*). Cho hai đầu ra khác nhau, các chuyên gia là con người chọn ra cái nào tốt hơn.
- **Chấm điểm** (*Rating*). Các chuyên gia là con người cho mỗi đầu ra một điểm số hay mức đánh giá. Điểm này thường là một giá trị số liên tục hoặc rời rạc, chẳng hạn điểm trên một thang (ví dụ 1–5 sao, hoặc 1–10 điểm). Trong một số trường hợp, đánh giá có thể là nhị phân, thể hiện sở thích kiểu "có/không" hoặc "tích cực/tiêu cực".
- **Xếp hạng theo danh sách** (*Listwise Ranking*). Các chuyên gia là con người được yêu cầu xếp hạng hoặc sắp thứ tự tập các đầu ra khả dĩ đã cho.

Ở đây ta xét phản hồi dạng so sánh theo cặp vì đó là một trong những dạng phản hồi của con người đơn giản và phổ biến nhất được dùng trong RLHF. Trong thiết lập này, với mỗi lần so sánh, hai đầu ra $(\mathbf{y}_a, \mathbf{y}_b)$ được lấy ngẫu nhiên từ tập ứng viên $\{\mathbf{y}_1, \ldots, \mathbf{y}_N\}$. Sau đó các cặp này được đưa cho các chuyên gia là con người, và họ được yêu cầu quyết định mình thích đầu ra nào hơn dựa trên các tiêu chí như độ rõ ràng, mức độ liên quan và độ chính xác. Phản hồi của con người có thể được mã hóa thành một nhãn nhị phân. Ví dụ, $\mathbf{y}_a \succ \mathbf{y}_b$ biểu thị sự ưa thích dành cho $\mathbf{y}_a$, và $\mathbf{y}_b \succ \mathbf{y}_a$ biểu thị sự ưa thích dành cho $\mathbf{y}_b$.

Một mô hình đơn giản và được dùng rộng rãi để mô tả các so sánh theo cặp như vậy là **mô hình Bradley–Terry** (*Bradley-Terry model*) [Bradley và Terry, 1952]. Đây là một mô hình xác suất ước lượng xác suất một đối tượng được ưa thích hơn một đối tượng khác. Điều chỉnh mô hình này theo ký hiệu dùng ở đây, ta có thể viết xác suất $\mathbf{y}_a$ được ưa thích hơn $\mathbf{y}_b$ dưới dạng

$$\begin{aligned}\Pr(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x}) &= \frac{e^{r(\mathbf{x},\mathbf{y}_a)}}{e^{r(\mathbf{x},\mathbf{y}_a)} + e^{r(\mathbf{x},\mathbf{y}_b)}} \\ &= \frac{e^{r(\mathbf{x},\mathbf{y}_a)-r(\mathbf{x},\mathbf{y}_b)}}{e^{r(\mathbf{x},\mathbf{y}_a)-r(\mathbf{x},\mathbf{y}_b)} + 1} \\ &= \text{Sigmoid}\big(r(\mathbf{x},\mathbf{y}_a) - r(\mathbf{x},\mathbf{y}_b)\big). \end{aligned} \tag{10.35}$$

Chú giải của người dịch: Mô hình Bradley–Terry qua ví dụ thi đấu

Mô hình này ban đầu dùng để dự đoán kết quả đối đầu: mỗi đội có một "chỉ số sức mạnh", và xác suất đội A thắng đội B chỉ phụ thuộc vào *hiệu* hai chỉ số. Ở đây "chỉ số sức mạnh" của một câu trả lời là điểm phần thưởng r. Hàm $\text{Sigmoid}(z) = \dfrac{1}{1+e^{-z}}$ biến một số thực bất kỳ thành xác suất trong $(0,1)$. Ví dụ: $r(\mathbf{x},\mathbf{y}_a) = 2$, $r(\mathbf{x},\mathbf{y}_b) = 1$ thì $\Pr(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x}) = \text{Sigmoid}(1) = \dfrac{1}{1+e^{-1}} \approx 0.73$. Nếu hai điểm bằng nhau thì xác suất là 0.5 ("ngang tài ngang sức"). Để ý rằng cộng cùng một hằng số vào mọi điểm thì xác suất không đổi — chỉ hiệu số là quan trọng.

Khi huấn luyện mô hình phần thưởng, mục tiêu của ta là cực đại hóa xác suất ưa thích này. Một hàm mất mát dựa trên mô hình Bradley–Terry được cho bởi

$$\mathcal{L}_r(\phi) \;=\; -\mathbb{E}_{(\mathbf{x},\mathbf{y}_a,\mathbf{y}_b)\sim\mathcal{D}_r}\big[\log \Pr_\phi(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x})\big], \tag{10.36}$$

trong đó $(\mathbf{x},\mathbf{y}_a,\mathbf{y}_b)$ được lấy từ một tập dữ liệu do con người gán nhãn $\mathcal{D}_r$, gồm các cặp đầu ra có thứ tự ưa thích cùng với đầu vào tương ứng của chúng. ϕ biểu diễn các tham số của mô hình phần thưởng, bao gồm cả tham số của bộ giải mã Transformer lẫn ma trận ánh xạ tuyến tính $\mathbf{W}_r$. Trong thực tế, giả sử $(\mathbf{x},\mathbf{y}_a,\mathbf{y}_b)$ được lấy mẫu đều từ $\mathcal{D}_r$, kỳ vọng có thể được xấp xỉ bằng trung bình thực nghiệm

$$\mathcal{L}_r(\phi) \;=\; -\frac{1}{|\mathcal{D}_r|}\sum_{(\mathbf{x},\mathbf{y}_a,\mathbf{y}_b)\in\mathcal{D}_r} \log \Pr_\phi(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x}). \tag{10.37}$$

Mục tiêu của việc huấn luyện mô hình phần thưởng là tìm các tham số tối ưu $\hat{\phi}$ cực tiểu hóa hàm mất mát này, được cho bởi

$$\hat{\phi} \;=\; \arg\min_\phi \mathcal{L}_r(\phi). \tag{10.38}$$

Vì bản thân mô hình phần thưởng cũng là một LLM, ta có thể đơn giản dùng quy trình huấn luyện Transformer để tối ưu mô hình phần thưởng. Điểm khác biệt so với việc huấn luyện một LLM thông thường là ta chỉ cần thay mất mát entropy chéo bằng mất mát so sánh theo cặp như mô tả trong Phương trình (10.37). Sau khi huấn luyện mô hình phần thưởng, ta có thể áp dụng mô hình phần thưởng đã huấn luyện $r_{\hat{\phi}}(\cdot)$ để giám sát LLM đích nhằm căn chỉnh nó.

Chú giải của người dịch: Hàm mất mát của mô hình phần thưởng bằng số

Với mỗi cặp mà con người nói $\mathbf{y}_a$ tốt hơn $\mathbf{y}_b$, mất mát là $-\log \text{Sigmoid}(r_a - r_b)$, với r_a, r_b là điểm mô hình chấm. Nếu mô hình chấm đúng chiều và cách xa, ví dụ $r_a - r_b = 3$, thì $\text{Sigmoid}(3) \approx 0.95$ và mất mát $\approx -\ln 0.95 \approx 0.05$ (rất nhỏ). Nếu chấm ngược, ví dụ $r_a - r_b = -2$, thì $\text{Sigmoid}(-2) \approx 0.12$ và mất mát

$\approx -\ln 0.12 \approx 2.1$ (lớn). Cực tiểu hóa trung bình các mất mát này buộc mô hình chấm câu được ưa thích cao hơn câu bị chê. Lưu ý mô hình không cần biết "điểm tuyệt đối" đúng là bao nhiêu — nó chỉ học *thứ tự*.

Đáng lưu ý là mặc dù ta huấn luyện mô hình phần thưởng để thực hiện xếp hạng theo cặp, trong quá trình căn chỉnh ta lại dùng nó để chấm điểm độc lập từng cặp đầu vào–đầu ra. Mục tiêu xếp hạng theo cặp đảm bảo mô hình phần thưởng nhạy với những khác biệt tinh tế giữa các đầu ra, nhưng ta dựa vào các điểm số liên tục do mô hình phần thưởng tạo ra để định hướng việc tối ưu LLM. Một ưu điểm của cách tiếp cận này là ta có thể chọn hoặc kết hợp nhiều hàm mất mát xếp hạng (*ranking loss*) khác nhau, mà vẫn áp dụng các mô hình phần thưởng thu được theo cùng cách như ta đã làm trong tiểu mục này. Sự linh hoạt này khiến khung mô hình hóa phần thưởng áp dụng được rộng rãi cho nhiều dạng giám sát bằng sở thích của con người.

10.3.3 Huấn luyện LLM

Khi đã có mô hình phần thưởng, ta tiếp tục huấn luyện chính sách (tức LLM) bằng phương pháp A2C. Nhớ lại từ Mục 10.3.1 rằng một chuỗi trạng thái–hành động hay quỹ đạo τ có thể được đánh giá bằng hàm tiện ích (*utility function*)

$$U(\tau;\theta) \;=\; \sum_{t=1}^{T} \log \pi_\theta(a_t|s_t) A(s_t, a_t), \tag{10.39}$$

trong đó $A(s_t, a_t)$ là lợi thế của việc thực hiện hành động a_t khi ở trạng thái s_t. Một ước lượng của $A(s_t, a_t)$ được định nghĩa là sai số TD $r_t + \gamma V(s_{t+1}) - V(s_t)$, trong đó hàm giá trị $V(s_t)$ được huấn luyện với mô hình phần thưởng.

Với hàm tiện ích này, hàm mất mát dựa trên A2C có thể được viết dưới dạng

$$\begin{aligned}\mathcal{L}(\theta) &= -\mathbb{E}_{\tau\sim\mathcal{T}}\big[U(\tau;\theta)\big] \\ &= -\mathbb{E}_{\tau\sim\mathcal{T}}\Big[\sum_{t=1}^{T} \log \pi_\theta(a_t|s_t) A(s_t, a_t)\Big],\end{aligned} \tag{10.40}$$

trong đó $\mathcal{T}$ là một không gian các chuỗi trạng thái–hành động. Như thường lệ, mục tiêu của việc huấn luyện chính sách là cực tiểu hóa hàm mất mát này

$$\tilde{\theta} \;=\; \arg\min_\theta \mathcal{L}(\theta). \tag{10.41}$$

Nếu ánh xạ bài toán trở lại bài toán mô hình hóa ngôn ngữ và dùng ký hiệu của LLM, hàm mất mát có thể được viết là:

$$\mathcal{L}(\theta) \;=\; -\mathbb{E}_{(\mathbf{x},\mathbf{y})\sim\mathcal{D}}\big[U(\mathbf{x},\mathbf{y};\theta)\big], \tag{10.42}$$

trong đó

$$U(\mathbf{x},\mathbf{y};\theta) \;=\; \sum_{t=1}^{T} \log \pi_\theta(y_t|\mathbf{x},\mathbf{y}_{<t}) A(\mathbf{x},\mathbf{y}_{<t}, y_t). \tag{10.43}$$

Ở đây $\mathcal{D}$ là một tập các cặp đầu vào–đầu ra, và $\pi_\theta(y_t|\mathbf{x},\mathbf{y}_{<t}) = \Pr_\theta(y_t|\mathbf{x},\mathbf{y}_{<t})$ là LLM được tham số hóa bởi θ.

Nói chung, trong RLHF ta không có một tập dữ liệu đầu vào–đầu ra $\mathcal{D}$ do con người gán nhãn, mà chỉ có một tập dữ liệu chứa các đầu vào. Trong trường hợp này, các đầu ra thường là các dự đoán do LLM tạo ra. Khi đó hàm mất mát được định nghĩa là

$$\mathcal{L}(\theta) \;=\; -\mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\big[U(\mathbf{x},\mathbf{y};\theta)\big], \tag{10.44}$$

trong đó $\mathcal{D}$ ký hiệu tập dữ liệu chỉ có đầu vào, và $\mathbf{y} \sim \pi_\theta(\cdot|\mathbf{x})$ chỉ ra rằng đầu ra $\mathbf{y}$ được lấy mẫu bởi chính sách $\pi_\theta(\cdot|\mathbf{x})$.

Chú giải của người dịch: So sánh với tinh chỉnh có giám sát

Trong SFT, mất mát là $-\sum_t \log \Pr_\theta(y_t|\mathbf{x}, \mathbf{y}_{<t})$ với $\mathbf{y}$ là đáp án chuẩn: mọi token của đáp án đều được "khuyến khích" như nhau. Phương trình (10.43)–(10.44) khác ở hai điểm: (1) $\mathbf{y}$ do chính LLM tự viết ra (lấy mẫu), không phải do người viết sẵn; (2) mỗi số hạng log-xác suất được nhân với lợi thế A, nên token nào giúp câu trả lời tốt hơn mong đợi ($A > 0$) thì được tăng xác suất, token nào làm câu trả lời tệ đi ($A < 0$) thì bị giảm xác suất. Có thể nói SFT là "chép bài mẫu", còn RLHF là "tự làm bài rồi được chấm điểm".

Công thức trên cung cấp một dạng cơ bản của phương pháp A2C cho LLM. Các phiên bản cải tiến của mô hình này được dùng phổ biến hơn trong RLHF. Trong phần thảo luận tiếp theo, chúng tôi vẫn dùng ký hiệu học tăng cường để đơn giản hóa cách trình bày và sẽ quay lại ký hiệu mô hình hóa ngôn ngữ sau.

Một cải tiến phổ biến của các phương pháp gradient chính sách là xây dựng một mục tiêu thay thế bằng **lấy mẫu theo trọng số quan trọng** (*importance sampling*). Mục tiêu thu được có thể viết là

$$U(\tau;\theta) \;=\; \sum_{t=1}^{T} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)} A(s_t, a_t). \tag{10.45}$$

Ở đây ta thay log-xác suất $\log \pi_\theta(a_t|s_t)$ bằng tỉ số $\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}$. θ_{ref} ký hiệu các tham số của chính sách trước đó (chẳng hạn mô hình ban đầu mà từ đó ta bắt đầu huấn luyện). Do vậy, $\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}$, còn được gọi là **hàm tỉ số** (*ratio function*), có thể được hiểu là tỉ số xác suất giữa chính sách hiện tại π_θ và chính sách tham chiếu $\pi_{\theta_{\text{ref}}}$ (gọi nó là chính sách tham chiếu (*reference policy*)). Bằng cách dùng hàm tỉ số, ta đánh trọng số lại các phần thưởng quan sát được dựa trên khả năng xảy ra của các hành động dưới chính sách hiện tại so với chính sách tham chiếu. Khi $\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)} > 1$, hành động a_t được chính sách hiện tại ưa chuộng hơn so với chính sách tham chiếu. Ngược lại, khi $\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)} < 1$, hành động a_t ít được chính sách hiện tại ưa chuộng hơn[7].

[7] Xét một trường hợp tổng quát hơn, trong đó ta muốn đánh giá chính sách bằng phần thưởng kỳ vọng của nó (xem thêm Phương trình (10.18))

$$J(\theta) \;=\; \mathbb{E}_{\tau\sim\pi_\theta}\Big[R(\tau)\Big]. \tag{10.46}$$

Ở đây $\tau \sim \pi_\theta$ nghĩa là chuỗi τ được sinh ra bởi chính sách π_θ. Ngoài ra, ta có thể viết $J(\theta)$ dưới một dạng khác

$$J(\theta) \;=\; \mathbb{E}_{\tau\sim\pi_{\theta_{\text{ref}}}}\Big[\frac{\Pr_\theta(\tau)}{\Pr_{\theta_{\text{ref}}}(\tau)} R(\tau)\Big]. \tag{10.47}$$

Dễ thấy vế phải của hai phương trình này về bản chất là như nhau vì $\mathbb{E}_{\tau\sim\pi_{\theta_{\text{ref}}}}\Big[\frac{\Pr_\theta(\tau)}{\Pr_{\theta_{\text{ref}}}(\tau)} R(\tau)\Big] = \sum_\tau \Pr_{\theta_{\text{ref}}}(\tau)\frac{\Pr_\theta(\tau)}{\Pr_{\theta_{\text{ref}}}(\tau)} R(\tau) = \sum_\tau \Pr_\theta(\tau) R(\tau) = \mathbb{E}_{\tau\sim\pi_\theta}\Big[R(\tau)\Big]$.

Lưu ý rằng sự tương đương này chỉ đúng khi kỳ vọng được lấy trên toàn bộ không gian chuỗi. Tuy nhiên, trong thực tế, khi học chính sách ta thường chỉ lấy mẫu một số tương đối nhỏ các chuỗi bằng một chính sách. Do đó, bản thân phương pháp lấy mẫu là quan trọng. Phương trình (10.47) đưa ra một cách thú vị để tách quá trình lấy mẫu và quá trình tính phần thưởng: trước tiên ta dùng một chính sách cơ sở (với θ_{ref}) để lấy mẫu một số chuỗi, rồi dùng chính sách đích (với θ) để tính phần thưởng kỳ vọng.

Chú giải của người dịch: Lấy mẫu theo trọng số quan trọng

Ý tưởng: dữ liệu được thu bằng chính sách *cũ* $\pi_{\theta_{\text{ref}}}$, nhưng ta muốn đánh giá chính sách *mới* π_θ mà không phải sinh lại dữ liệu. Cách làm: đánh trọng số mỗi mẫu bằng tỉ số "chính sách mới thích nó gấp mấy lần chính sách cũ". Ví dụ: chính sách cũ chọn token "Xin" với xác suất 0.2, chính sách mới với xác suất 0.3; tỉ số $= 1.5$, nên phần thưởng/lợi thế của mẫu này được tính "nặng" gấp 1.5 lần. Tương tự như khi điều tra xã hội học: nếu nhóm nào bị lấy mẫu ít hơn tỉ lệ thực tế trong dân số, ta nhân trọng số cho nhóm đó lớn hơn để kết quả trung bình vẫn đúng.

Một vấn đề với mô hình trình bày trong Phương trình (10.47) (cũng như trong Phương trình (10.39)) là phương sai của các ước lượng gradient thường cao, khiến quá trình học không ổn định. Để giảm nhẹ vấn đề này, các kỹ thuật như cắt ngưỡng (*clipping*) thường được dùng để chặn các trọng số quan trọng và ngăn những bước cập nhật lớn. Một phiên bản cắt ngưỡng của hàm tiện ích (còn gọi là hàm mục tiêu thay thế có cắt ngưỡng) được cho bởi

$$U_{\text{clip}}(\tau;\theta) \;=\; \sum_{t=1}^{T} \text{Clip}\Big(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}, A(s_t,a_t)\Big), \tag{10.49}$$

trong đó

$$\begin{aligned}&\text{Clip}\Big(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}, A(s_t,a_t)\Big)\\ &\quad= \min\Big(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}A(s_t,a_t),\ \text{bound}\Big(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}, 1-\epsilon, 1+\epsilon\Big)A(s_t,a_t)\Big).\end{aligned} \tag{10.50}$$

Ở đây hàm $\text{bound}(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}, 1-\epsilon, 1+\epsilon)$ giới hạn hàm tỉ số trong khoảng $[1-\epsilon, 1+\epsilon]$.

Chú giải của người dịch: Cắt ngưỡng hoạt động thế nào?

Hàm $\text{bound}(z, 1-\epsilon, 1+\epsilon)$ "kẹp" z vào đoạn $[1-\epsilon, 1+\epsilon]$: nếu z nhỏ hơn $1-\epsilon$ thì lấy $1-\epsilon$, lớn hơn $1+\epsilon$ thì lấy $1+\epsilon$, còn lại giữ nguyên. Ví dụ $\epsilon = 0.2$, lợi thế $A = +2$. Nếu tỉ số đã là 1.5 (chính sách mới đã tăng xác suất hành động tốt này lên nhiều), thì $\min(1.5 \cdot 2,\ 1.2 \cdot 2) = 2.4$: phần vượt quá 1.2 không được tính thêm, nên mô hình không còn động lực đẩy xác suất lên nữa. Giống như quy định "mỗi lần điều chỉnh giá không quá 20%": cho phép tiến bộ, nhưng từng bước nhỏ, an toàn. Phép min đảm bảo ta luôn chọn ước lượng "bi quan" hơn trong hai giá trị.

Một cải tiến tiếp theo của mô hình trên là xét các **vùng tin cậy** (*trust region*) trong tối ưu hóa [Schulman và cộng sự, 2015]. Trong học tăng cường, một bước cập nhật lớn cho chính sách có thể dẫn đến mất ổn định, khi tác tử có thể bắt đầu hoạt động tệ hơn sau một lần cập nhật. Một ý tưởng hợp lý là

Bằng cách này, ta tách riêng chính sách dùng để thu thập dữ liệu và chính sách dùng để tính gradient. Cách tiếp cận này tránh được việc phải lấy mẫu trực tiếp từ chính sách đang được đánh giá, điều có thể có lợi trong những trường hợp việc sinh chuỗi từ chính sách đích tốn kém hoặc khó khăn. Trong học tăng cường, $\mathbb{E}_{\tau\sim\pi_{\theta_{\text{ref}}}}\Big[\frac{\Pr_\theta(\tau)}{\Pr_{\theta_{\text{ref}}}(\tau)}R(\tau)\Big]$ thường được gọi là **mục tiêu thay thế** (*surrogate objective*).

Phương trình (10.47) cũng có thể được diễn giải từ góc nhìn gradient chính sách. Với $\mathbb{E}_{\tau\sim\pi_{\theta_{\text{ref}}}}\Big[\frac{\Pr_\theta(\tau)}{\Pr_{\theta_{\text{ref}}}(\tau)}R(\tau)\Big]$, gradient tại $\theta = \theta_{\text{ref}}$ được cho bởi

$$\frac{\partial}{\partial\theta}\mathbb{E}_{\tau\sim\pi_{\theta_{\text{ref}}}}\Big[\frac{\Pr_\theta(\tau)}{\Pr_{\theta_{\text{ref}}}(\tau)}R(\tau)\Big]\Big|_{\theta=\theta_{\text{ref}}} \;=\; \mathbb{E}_{\tau\sim\pi_{\theta_{\text{ref}}}}\Big[\frac{\partial \Pr_\theta(\tau)|_{\theta=\theta_{\text{ref}}}}{\partial\theta}R(\tau)\Big]. \tag{10.48}$$

Vế phải là một dạng chuẩn dùng trong các phương pháp gradient chính sách, nghĩa là ta tính hướng cập nhật tham số tại điểm $\theta = \theta_{\text{ref}}$ trên bề mặt tối ưu.

tối ưu mô hình trong vùng tin cậy, tức là một vùng quanh ước lượng tham số hiện tại, nơi mô hình hoạt động tốt. Một cách để đưa vùng tin cậy vào là áp đặt một ràng buộc lên độ lớn của bước cập nhật chính sách, bảo đảm chính sách hiện tại không lệch đáng kể khỏi chính sách tham chiếu. Trong RLHF, PPO thường được kết hợp với một khoản phạt bổ sung kiểu KL, ràng buộc chính sách luôn ở gần chính sách tham chiếu. Điều này có thể đạt được bằng cách thêm một khoản phạt liên quan đến sự khác biệt giữa chính sách hiện tại và chính sách tham chiếu. Một dạng đơn giản của khoản phạt như vậy được cho bởi hiệu giữa log-xác suất của chuỗi τ dưới chính sách hiện tại và dưới chính sách tham chiếu:

$$\text{Penalty} \;=\; \log \pi_\theta(\tau) - \log \pi_{\theta_{\text{ref}}}(\tau). \tag{10.51}$$

Trong thực tế, khoản phạt này có thể được xấp xỉ bằng cách chỉ xét các xác suất của chính sách và bỏ qua động lực. Điều này cho

$$\text{Penalty} \;=\; \sum_{t=1}^{T} \log \pi_\theta(a_t|s_t) - \sum_{t=1}^{T} \log \pi_{\theta_{\text{ref}}}(a_t|s_t). \tag{10.52}$$

Bằng cách đưa khoản phạt này vào mục tiêu tối ưu, ta khuyến khích chính sách hiện tại ở gần chính sách tham chiếu, hạn chế những bước cập nhật quá lớn có thể làm quá trình học mất ổn định.

Ta có thể đưa khoản phạt này vào hàm mục tiêu thay thế có cắt ngưỡng, và thu được

$$U_{\text{ppo-clip}}(\tau;\theta) \;=\; U_{\text{clip}}(\tau;\theta) - \beta\,\text{Penalty}, \tag{10.53}$$

trong đó β là trọng số của khoản phạt. Phương pháp huấn luyện này được gọi là **tối ưu chính sách cận kề** (*proximal policy optimization*) (**PPO**), một trong những phương pháp học tăng cường phổ biến nhất được dùng trong LLM và nhiều lĩnh vực khác [Schulman và cộng sự, 2017].

Chú giải của người dịch: Khoản phạt KL: "đừng đi quá xa nguồn gốc"

Khoản phạt (Phương trình (10.51)–(10.52)) đo xem chính sách mới gán xác suất khác chính sách tham chiếu đến mức nào (lấy trung bình trên các mẫu, đại lượng này xấp xỉ *độ phân kỳ KL* — một "khoảng cách" giữa hai phân phối xác suất). Vì sao cần? Mô hình phần thưởng chỉ là giám khảo *máy*, có thể có "điểm mù". Nếu để LLM tự do tối ưu, nó có thể tìm ra những câu trả lời kỳ quặc nhưng được chấm điểm cao (giống học sinh "học tủ" theo sở thích giám khảo thay vì học thật). Khoản phạt với trọng số β giữ LLM gần mô hình ban đầu — vốn đã nói năng tự nhiên — trong khi vẫn cho phép cải thiện. *Cận kề* (*proximal*) trong tên PPO chính là ý "chỉ đi những bước gần".

Giờ ta có thể viết mục tiêu huấn luyện LLM dưới dạng PPO

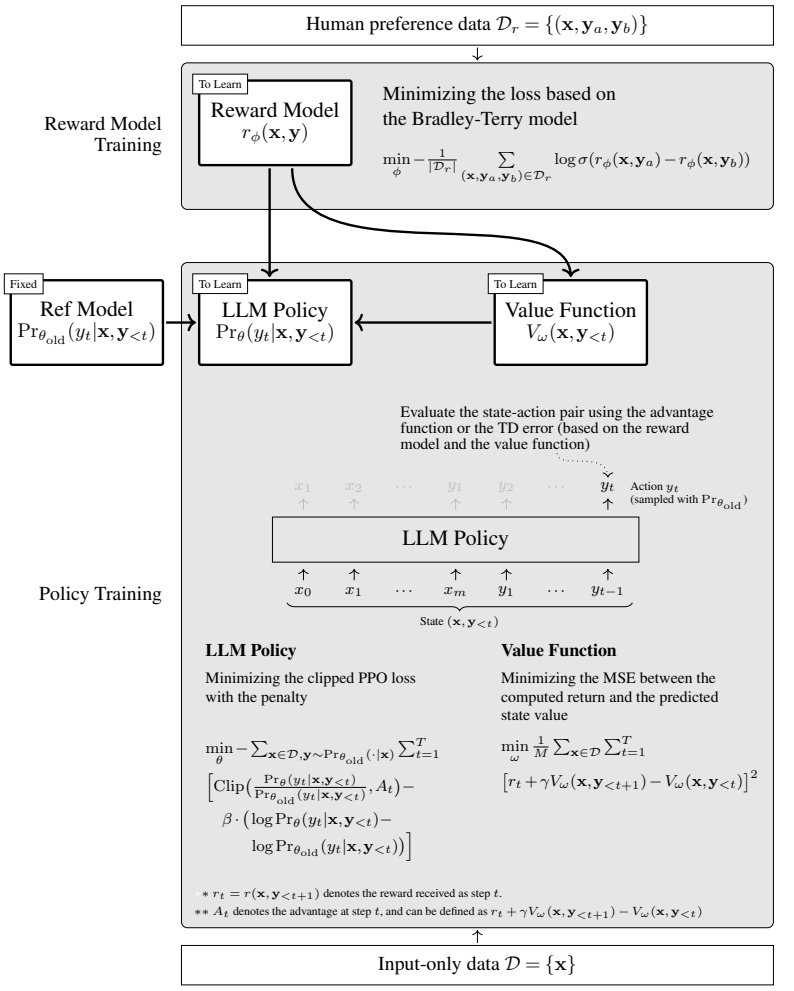
$$U_{\text{ppo-clip}}(\mathbf{x},\mathbf{y};\theta) \;=\; U_{\text{clip}}(\mathbf{x},\mathbf{y};\theta) - \beta\,\text{Penalty}, \tag{10.54}$$

trong đó

$$U_{\text{clip}}(\mathbf{x},\mathbf{y};\theta) = \sum_{t=1}^{T} \text{Clip}\Big(\frac{\pi_\theta(y_t|\mathbf{x},\mathbf{y}_{<t})}{\pi_{\theta_{\text{ref}}}(y_t|\mathbf{x},\mathbf{y}_{<t})}, A(\mathbf{x},\mathbf{y}_{<t},y_t)\Big), \tag{10.55}$$

$$\begin{aligned}\text{Penalty} &= \log \Pr_\theta(\mathbf{y}|\mathbf{x}) - \log \Pr_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \\ &= \sum_{t=1}^{T} \log \Pr_\theta(y_t|\mathbf{x},\mathbf{y}_{<t}) - \sum_{t=1}^{T} \log \Pr_{\theta_{\text{ref}}}(y_t|\mathbf{x},\mathbf{y}_{<t}).\end{aligned} \tag{10.56}$$

Mặc dù ký hiệu ở đây trông hơi rườm rà, ý tưởng của PPO rất đơn giản: ta xây dựng một mục tiêu bằng cách kết hợp tỉ số khả năng (đã cắt ngưỡng) giữa chính sách đích và chính sách tham chiếu với một



Hình 10.9: Minh họa RLHF. Bước đầu tiên là thu thập dữ liệu sở thích của con người và huấn luyện mô hình phần thưởng bằng dữ liệu này. Khi mô hình phần thưởng đã được tối ưu, cùng với mô hình tham chiếu, ta tiến hành huấn luyện cả chính sách lẫn hàm giá trị. Tại mỗi bước dự đoán, ta tính tổng mất mát dựa trên PPO và cập nhật tham số của chính sách. Việc này đòi hỏi phải có sẵn mô hình phần thưởng, mô hình tham chiếu và hàm giá trị. Đồng thời, ta cập nhật tham số của hàm giá trị bằng cách cực tiểu hóa mất mát MSE.

hàm lợi thế, rồi áp thêm một khoản phạt bảo đảm các bước cập nhật chính sách không quá lớn. RLHF dựa trên PPO được minh họa trong Hình 10.9.

Chú giải của người dịch: Đọc các nhãn trong Hình 10.9

Human preference data — dữ liệu sở thích của con người; *Reward Model Training* — huấn luyện mô hình phần thưởng, bằng cách *cực tiểu hóa mất mát dựa trên mô hình Bradley–Terry*; *To Learn* — cần học (tham số được cập nhật); *Fixed* — cố định; *Ref Model* — mô hình tham chiếu; *LLM Policy* — chính sách LLM; *Value Function* — hàm giá trị; *Policy Training* — huấn luyện chính sách. Dòng chữ giữa hình: *đánh giá cặp trạng thái–hành động bằng hàm lợi thế hoặc sai số TD (dựa trên mô hình phần thưởng và hàm giá trị)*; *Action* y_t *(sampled with* $\Pr_{\theta_{\text{old}}}$*)* — hành động y_t được lấy mẫu bằng mô hình cũ. Phía dưới: chính sách LLM được huấn luyện bằng *cực tiểu hóa mất mát PPO có cắt ngưỡng kèm khoản phạt*; hàm giá trị được huấn luyện bằng *cực tiểu hóa MSE (sai số bình phương trung bình) giữa lợi tức tính được và giá trị trạng thái dự đoán*. Hai ghi chú cuối: $r_t = r(\mathbf{x}, \mathbf{y}_{<t+1})$ là phần thưởng nhận được tại bước t; A_t là lợi thế tại bước t, có thể định nghĩa là $r_t + \gamma V_\omega(\mathbf{x}, \mathbf{y}_{<t+1}) - V_\omega(\mathbf{x}, \mathbf{y}_{<t})$. *Input-only data* — dữ liệu chỉ có đầu vào. (Trong hình, θ_{old} đóng vai trò của θ_{ref} trong văn bản.)

Tóm lại, hiện thực RLHF đòi hỏi xây dựng bốn mô hình, tất cả đều dựa trên kiến trúc bộ giải mã Transformer.

- **Mô hình phần thưởng** ($r_\phi(\cdot)$, trong đó ϕ ký hiệu các tham số). Mô hình phần thưởng học từ dữ liệu sở thích của con người để dự đoán phần thưởng cho mỗi cặp chuỗi token đầu vào và đầu ra. Nó là một bộ giải mã Transformer theo sau bởi một tầng tuyến tính, ánh xạ một chuỗi (phép nối giữa đầu vào và đầu ra) thành một điểm phần thưởng là số thực.

- **Mô hình giá trị** hay **hàm giá trị** ($V_\omega(\cdot)$, trong đó ω ký hiệu các tham số). Hàm giá trị nhận điểm phần thưởng từ mô hình phần thưởng và được huấn luyện để dự đoán tổng phần thưởng kỳ vọng có thể thu được khi bắt đầu từ một trạng thái. Nó thường dựa trên cùng kiến trúc với mô hình phần thưởng.

- **Mô hình tham chiếu** ($\pi_{\theta_{\text{ref}}}(\cdot) = \Pr_{\theta_{\text{ref}}}(\cdot)$, trong đó θ_{ref} ký hiệu các tham số). Mô hình tham chiếu là LLM cơ sở, đóng vai trò điểm xuất phát cho việc huấn luyện chính sách. Trong RLHF, nó biểu diễn phiên bản trước đó của mô hình, hoặc một mô hình được huấn luyện mà không có phản hồi của con người. Nó được dùng để lấy mẫu các chuỗi đầu ra và tham gia vào việc tính mất mát khi huấn luyện chính sách.

- **Mô hình đích** hay **chính sách** ($\pi_\theta(\cdot) = \Pr_\theta(\cdot)$, trong đó θ ký hiệu các tham số). Chính sách này chi phối cách LLM quyết định token tiếp theo thích hợp nhất khi biết ngữ cảnh của nó. Nó được huấn luyện dưới sự giám sát của cả mô hình phần thưởng lẫn mô hình giá trị.

Trong thực tế, các mô hình này cần được huấn luyện theo một thứ tự nhất định. Trước tiên, ta cần khởi tạo chúng bằng các mô hình sẵn có. Ví dụ, mô hình phần thưởng và mô hình giá trị có thể được khởi tạo bằng một LLM đã tiền huấn luyện, còn mô hình tham chiếu và mô hình đích có thể được khởi tạo bằng một mô hình đã được tinh chỉnh theo chỉ dẫn. Lưu ý rằng, tại thời điểm này, mô hình tham chiếu đã sẵn sàng để dùng và sẽ không được cập nhật thêm. Thứ hai, ta cần thu thập dữ liệu sở thích của con người và huấn luyện mô hình phần thưởng trên dữ liệu này. Thứ ba, cả mô hình giá trị lẫn chính sách được huấn luyện đồng thời bằng mô hình phần thưởng. Tại mỗi vị trí trong một chuỗi token đầu ra, ta cập nhật mô hình giá trị bằng cách cực tiểu hóa MSE của dự đoán giá trị, và chính sách được cập nhật bằng cách cực tiểu hóa mất mát PPO.

Chú giải của người dịch: Toàn cảnh RLHF trong một vòng lặp

Một vòng huấn luyện PPO có thể hình dung như sau: (1) lấy một lô câu hỏi $\mathbf{x}$ (chỉ có câu hỏi, không có đáp án); (2) LLM tự viết câu trả lời $\mathbf{y}$; (3) mô hình phần thưởng chấm điểm câu trả lời; (4) mô hình giá trị ước lượng "mức kỳ vọng" tại từng vị trí token, từ đó tính lợi thế A_t cho từng token; (5) cập nhật LLM bằng mất mát PPO (khuyến khích token có $A_t > 0$, có cắt ngưỡng và phạt nếu đi xa mô hình tham chiếu); (6) cập nhật mô hình giá trị để dự đoán chính xác hơn. Lặp lại nhiều lần. Bốn mô hình cùng lúc nghĩa là RLHF tốn rất nhiều bộ nhớ và tính toán — đây là động lực cho các phương pháp đơn giản hơn ở Mục 10.4.

10.4 Cải tiến việc căn chỉnh theo sở thích của con người

Ở mục trước, chúng ta đã điểm lại các khái niệm cơ bản của học tăng cường (*reinforcement learning*) và khung tổng quát của RLHF. Trong mục này, chúng ta sẽ bàn về một số cải tiến của RLHF và các phương pháp thay thế để đạt được việc căn chỉnh (*alignment*) theo sở thích của con người (*human preference*).

10.4.1 Mô hình hóa phần thưởng tốt hơn

Ở Mục 10.3.2, chúng ta đã nhấn mạnh nhiệm vụ học từ sở thích của con người cũng như việc dùng hàm mất mát xếp hạng theo cặp (*pairwise ranking loss*) để huấn luyện mô hình phần thưởng (*reward model*). Ở đây ta xem xét thêm các phương pháp mô hình hóa phần thưởng (*reward modeling*). Thảo luận của chúng ta sẽ tương đối tổng quát, và vì mô hình phần thưởng được dùng rộng rãi trong nhiều bài toán học tăng cường, ta sẽ dễ dàng áp dụng các phương pháp bàn ở đây cho RLHF và các ứng dụng liên quan.

1. Tín hiệu giám sát

Việc huấn luyện mô hình phần thưởng nói chung có thể được xem như một bài toán xếp hạng (*ranking*), trong đó mô hình học cách gán điểm cho các đầu ra sao cho thứ hạng thu được phản ánh sở thích mà con người đã chỉ ra. Có một số cách tiếp cận để huấn luyện mô hình phần thưởng từ góc độ xếp hạng.

Một cách tiếp cận là mở rộng xếp hạng theo cặp thành xếp hạng theo danh sách (*listwise ranking*). Với mỗi mẫu trong một tập dữ liệu, ta có thể dùng LLM để sinh ra nhiều đầu ra, rồi nhờ các chuyên gia con người sắp thứ tự các đầu ra này. Chẳng hạn, cho một tập gồm bốn đầu ra $\{\mathbf{y}_1, \mathbf{y}_2, \mathbf{y}_3, \mathbf{y}_4\}$, một thứ tự có thể có của chúng là $\mathbf{y}_2 \succ \mathbf{y}_3 \succ \mathbf{y}_1 \succ \mathbf{y}_4$. Một cách đơn giản để mô hình hóa những thứ hạng như vậy là cộng các hàm mất mát so sánh theo cặp. Ví dụ, ta có thể định nghĩa hàm mất mát theo danh sách bằng cách cộng dồn mất mát trên mọi cặp đầu ra:

$$\mathcal{L}_{\text{list}} = -\mathbb{E}_{(\mathbf{x},Y)\sim\mathcal{D}_r}\Big[\frac{2}{N(N-1)}\sum_{\substack{\mathbf{y}_a\in Y,\mathbf{y}_b\in Y\\ \mathbf{y}_a\succ\mathbf{y}_b}}\log\Pr(\mathbf{y}_a\succ\mathbf{y}_b|\mathbf{x})\Big], \tag{10.57}$$

trong đó Y là một danh sách các đầu ra, và N là số đầu ra trong danh sách. $\Pr(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x})$ có thể được định nghĩa bằng mô hình Bradley-Terry, tức là $\Pr(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x}) = \text{Sigmoid}(r(\mathbf{x}, \mathbf{y}_a) - r(\mathbf{x}, \mathbf{y}_b))$. Ở đây ta bỏ chỉ số ϕ trên $\Pr(\cdot)$ để ký hiệu đỡ rườm rà.

Chú giải của người dịch: Vì sao có hệ số $\frac{2}{N(N-1)}$?

Với N đầu ra đã sắp thứ tự, số cặp $(\mathbf{y}_a, \mathbf{y}_b)$ mà $\mathbf{y}_a$ đứng trước $\mathbf{y}_b$ chính là số tổ hợp chập 2 của N: $C_N^2 = \frac{N(N-1)}{2}$. Nhân với $\frac{2}{N(N-1)}$ nghĩa là lấy *trung bình* mất mát trên tất cả các cặp. Ví dụ với 4 đầu ra $\mathbf{y}_2 \succ \mathbf{y}_3 \succ$

$\mathbf{y}_1 \succ \mathbf{y}_4$, có $C_4^2 = 6$ cặp: $(\mathbf{y}_2, \mathbf{y}_3)$, $(\mathbf{y}_2, \mathbf{y}_1)$, $(\mathbf{y}_2, \mathbf{y}_4)$, $(\mathbf{y}_3, \mathbf{y}_1)$, $(\mathbf{y}_3, \mathbf{y}_4)$, $(\mathbf{y}_1, \mathbf{y}_4)$. Ký hiệu $\succ$ đọc là "được ưa thích hơn".

Một mở rộng của mô hình Bradley-Terry cho xếp hạng theo danh sách có thể dùng một cơ chế xếp hạng xét đến toàn bộ danh sách đầu ra thay vì chỉ so sánh từng cặp. Một mô hình như vậy là **mô hình Plackett-Luce** (*Plackett-Luce model*), mô hình này tổng quát hóa mô hình Bradley-Terry để xử lý nhiều phần tử trong một thứ hạng [Plackett, 1975]. Trong mô hình Plackett-Luce, với mỗi phần tử trong danh sách, ta định nghĩa một "giá trị" (*worth*) cho phần tử đó, phản ánh sức mạnh tương đối của việc nó được chọn so với các phần tử khác. Với bài toán mô hình hóa phần thưởng ở đây, giá trị của $\mathbf{y}$ trong danh sách Y có thể được định nghĩa là

$$\alpha(\mathbf{y}) \;=\; \exp(r(\mathbf{x}, \mathbf{y})). \tag{10.58}$$

Khi đó, xác suất để $\mathbf{y}$ được chọn làm phần tử xếp hạng cao nhất từ Y là

$$\begin{aligned}\Pr(\mathbf{y} \text{ được chọn} | \mathbf{x}, Y) &= \frac{\alpha(\mathbf{y})}{\sum_{\mathbf{y}' \in Y} \alpha(\mathbf{y}')} \\ &= \frac{\exp(r(\mathbf{x}, \mathbf{y}))}{\sum_{\mathbf{y}' \in Y} \exp(r(\mathbf{x}, \mathbf{y}'))}.\end{aligned} \tag{10.59}$$

Giả sử $\mathring{Y}$ là một danh sách có thứ tự $\mathbf{y}_{j_1} \succ \mathbf{y}_{j_2} \succ \cdots \succ \mathbf{y}_{j_N}$. Log-xác suất tổng thể của danh sách có thứ tự này có thể được định nghĩa là tổng các log-xác suất có điều kiện ở mỗi bước lựa chọn:

$$\begin{aligned}\log \Pr(\mathring{Y}|\mathbf{x}) &= \log \Pr(\mathbf{y}_{j_1} \succ \mathbf{y}_{j_2} \succ \cdots \succ \mathbf{y}_{j_N}|\mathbf{x}) \\ &= \log \Pr(\mathbf{y}_{j_1}|\mathbf{x}, \{\mathbf{y}_{j_1}, \mathbf{y}_{j_2}, ..., \mathbf{y}_{j_N}\}) + \\ &\quad \log \Pr(\mathbf{y}_{j_2}|\mathbf{x}, \{\mathbf{y}_{j_2}, ..., \mathbf{y}_{j_N}\}) + \\ &\quad \cdots + \\ &\quad \log \Pr(\mathbf{y}_{j_N}|\mathbf{x}, \{\mathbf{y}_{j_N}\}) \\ &= \sum_{k=1}^{N} \log \Pr(\mathbf{y}_{j_k}|\mathbf{x}, \mathring{Y}_{\geq k}),\end{aligned} \tag{10.60}$$

trong đó $\mathring{Y}_{\geq k}$ biểu diễn tập con các đầu ra trong danh sách còn chưa được chọn ở bước thứ k, tức là $\mathring{Y}_{\geq k} = \{\mathbf{y}_{j_k}, ..., \mathbf{y}_{j_N}\}$. Với log-xác suất $\log \Pr(\mathring{Y}|\mathbf{x})$, ta có thể định nghĩa hàm mất mát dựa trên mô hình Plackett-Luce như sau

$$\mathcal{L}_{\text{pl}} \;=\; -\mathbb{E}_{(\mathbf{x}, \mathring{Y}) \sim \mathcal{D}_r} \big[\log \Pr(\mathring{Y}|\mathbf{x}) \big]. \tag{10.61}$$

Chú giải của người dịch: Mô hình Plackett-Luce: "bốc thăm" lần lượt

Hãy hình dung việc xếp hạng như bốc lần lượt từng phần tử ra khỏi một chiếc hộp: phần tử có điểm thưởng cao thì "nặng ký" hơn và dễ được bốc trước. Giả sử có 3 đầu ra với điểm thưởng $r = \ln 3,\ \ln 2,\ \ln 1$, khi đó các "giá trị" $\alpha = \exp(r)$ lần lượt là $3, 2, 1$. Xác suất của thứ tự $\mathbf{y}_1 \succ \mathbf{y}_2 \succ \mathbf{y}_3$ là: bước 1 chọn $\mathbf{y}_1$ trong cả 3 phần tử với xác suất $\frac{3}{3+2+1} = \frac{1}{2}$; bước 2 chọn $\mathbf{y}_2$ trong 2 phần tử còn lại với xác suất $\frac{2}{2+1} = \frac{2}{3}$; bước 3 chỉ còn $\mathbf{y}_3$ nên xác suất là 1. Vậy xác suất cả thứ tự là $\frac{1}{2} \cdot \frac{2}{3} \cdot 1 = \frac{1}{3}$; lấy logarit thì tích biến thành tổng, đúng như Phương trình (10.60). Khi danh sách chỉ có 2 phần tử, công thức này trùng với mô hình Bradley-Terry: $\frac{e^{r_a}}{e^{r_a}+e^{r_b}} = \text{Sigmoid}(r_a - r_b)$.

Cũng có nhiều phương pháp theo cặp và theo danh sách khác để mô hình hóa thứ hạng, chẳng hạn RankNet [Burges và cộng sự, 2005] và ListNet [Cao và cộng sự, 2007]. Tất cả các phương pháp này có

thể được xếp vào một họ lớn các phương pháp học để xếp hạng (*learning-to-rank*), và phần lớn trong số chúng đều áp dụng được cho bài toán mô hình hóa sở thích của con người. Tuy nhiên, việc thảo luận các phương pháp này nằm ngoài phạm vi của chương. Bạn đọc quan tâm có thể tham khảo các sách về chủ đề này để biết thêm chi tiết [Liu, 2009; Li, 2011].

Bên cạnh xếp hạng theo cặp và theo danh sách, các phương pháp theo điểm (*pointwise methods*) cung cấp một cách khác để huấn luyện mô hình phần thưởng và nắm bắt sở thích của con người. Khác với các phương pháp tập trung vào thứ hạng tương đối giữa các đầu ra khác nhau, phương pháp theo điểm xử lý từng đầu ra một cách độc lập. Chẳng hạn, các chuyên gia con người có thể gán điểm cho từng đầu ra riêng lẻ, ví dụ chấm theo thang năm bậc. Mục tiêu là điều chỉnh mô hình phần thưởng sao cho đầu ra của nó khớp với các điểm số này. Một cách đơn giản để huấn luyện theo điểm là dùng các kỹ thuật hồi quy (*regression*), trong đó phần thưởng của mỗi đầu ra được xem là một biến mục tiêu. Gọi $\varphi(\mathbf{x}, \mathbf{y})$ là điểm con người gán cho $\mathbf{y}$ khi cho trước $\mathbf{x}$. Mô hình phần thưởng theo điểm có thể được huấn luyện bằng cách cực tiểu hóa một hàm mất mát, thường dựa trên sai số bình phương trung bình (*mean squared error*) hoặc các hàm mất mát hồi quy khác, giữa phần thưởng dự đoán $r(\mathbf{x}, \mathbf{y})$ và phản hồi thực tế của con người $\varphi(\mathbf{x}, \mathbf{y})$. Chẳng hạn, hàm mất mát có thể là

$$\mathcal{L}_{\text{point}} = \mathbb{E}_{(\mathbf{x},\mathbf{y})\sim\mathcal{D}_r}\left[(\varphi(\mathbf{x}, \mathbf{y}) - r(\mathbf{x}, \mathbf{y}))^2\right]. \tag{10.62}$$

Tuy phương pháp theo điểm đơn giản hơn về mặt khái niệm và có thể trực tiếp hướng dẫn mô hình phần thưởng dự đoán điểm số, chúng không phải lúc nào cũng là lựa chọn tốt nhất trong RLHF. Một vấn đề là các phương pháp này có thể gặp khó khăn với phương sai cao trong phản hồi của con người, đặc biệt khi các chuyên gia khác nhau cho những điểm số không nhất quán đối với các đầu ra tương tự nhau. Vì chúng tập trung khớp với điểm số tuyệt đối thay vì chênh lệch tương đối, sự không nhất quán trong chấm điểm có thể khiến mô hình hoạt động kém. Hơn nữa, việc khớp với các đầu ra được chấm điểm cụ thể có thể làm hại khả năng tổng quát hóa (*generalization*), nhất là khi dữ liệu huấn luyện trong RLHF thường rất hạn chế. Ngược lại, các phương pháp xét sở thích tương đối có thể thúc đẩy việc học những khuôn mẫu sở thích tổng quát hơn. Dù vậy, vẫn có những tình huống mà phương pháp theo điểm có thể phù hợp. Ví dụ, trong các nhiệm vụ có dữ liệu huấn luyện dồi dào và chi phí thu được các nhãn chính xác, nhất quán là thấp, phương pháp theo điểm có thể tỏ ra hiệu quả.

Chú giải của người dịch: Theo điểm hay theo cặp?

Thử nghĩ đến việc chấm bài văn. Hai giám khảo có thể "khó tính" khác nhau: một người hay cho 6–7 điểm, người kia hay cho 8–9 điểm cho cùng chất lượng bài. Nếu huấn luyện theo điểm (Phương trình 10.62), mô hình bị buộc phải khớp với cả hai thang "lệch" này. Nhưng nếu chỉ hỏi "bài A hay hơn bài B không?", hai giám khảo thường đồng ý với nhau hơn nhiều. Đó là lý do so sánh tương đối (theo cặp, theo danh sách) thường bền vững hơn với dữ liệu sở thích của con người.

Thực ra, để tín hiệu giám sát dùng huấn luyện mô hình phần thưởng bền vững hơn, ta cũng có thể đưa thêm các số hạng chính quy hóa (*regularization*) vào quá trình huấn luyện. Ví dụ, nếu ta xem số hạng thứ nhất $U_{\text{ppo-clip}}(\mathbf{x}, \mathbf{y}; \theta)$ trong Phương trình (10.54) như một loại phần thưởng tổng quát hóa, thì số hạng thứ hai (tức số hạng phạt) có thể được xem là một dạng chính quy hóa cho mô hình phần thưởng, chỉ khác là ở đây mục tiêu là huấn luyện chính sách (*policy*) chứ không phải mô hình phần thưởng. Một ví dụ khác là Eisenstein và cộng sự [2023] xây dựng một số hạng chính quy hóa dựa trên bình phương của tổng các phần thưởng, rồi cộng nó vào hàm mất mát so sánh theo cặp trong RLHF:

$$\begin{aligned}\mathcal{L}_{\text{reg}} &= \mathcal{L}_{\text{pair}} - \lambda\mathbb{E}_{(\mathbf{x},\mathbf{y}_a,\mathbf{y}_b)\sim\mathcal{D}_r}\left[(r(\mathbf{x}, \mathbf{y}_a) + r(\mathbf{x}, \mathbf{y}_b))^2\right] \\ &= -\mathbb{E}_{(\mathbf{x},\mathbf{y}_a,\mathbf{y}_b)\sim\mathcal{D}_r}\left[\log \Pr_{\phi}(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x})\right]\end{aligned}$$

$$- \lambda \mathbb{E}_{(\mathbf{x},\mathbf{y}_a,\mathbf{y}_b)\sim\mathcal{D}_r}\left[(r(\mathbf{x},\mathbf{y}_a) + r(\mathbf{x},\mathbf{y}_b))^2\right], \tag{10.63}$$

trong đó λ là trọng số của số hạng chính quy hóa. Tối ưu hóa với số hạng chính quy hóa này có thể giúp giảm nhẹ tính bất định (*underdetermination*) của mô hình phần thưởng[8].

Chú giải của người dịch: Vì sao mô hình phần thưởng "bất định"?

Mô hình Bradley-Terry chỉ quan tâm đến *hiệu* $r(\mathbf{x},\mathbf{y}_a) - r(\mathbf{x},\mathbf{y}_b)$. Nếu cộng cùng một hằng số vào mọi phần thưởng (ví dụ đổi $r_a = 3, r_b = 1$ thành $r_a = 103, r_b = 101$), hiệu vẫn là 2 và mất mát không đổi. Như vậy có vô số mô hình "tốt như nhau" với mức phần thưởng trôi nổi tùy ý. Số hạng $(r_a + r_b)^2$ nhằm kéo tổng các phần thưởng về gần 0, giúp "neo" thang đo lại: trong ví dụ trên, cặp $r_a = 1, r_b = -1$ sẽ được ưa chuộng hơn.

2. Phần thưởng thưa và phần thưởng dày

Như đã bàn ở Mục 10.3, phần thưởng do phản hồi của con người cung cấp thường *thưa*: chúng chỉ được quan sát ở cuối chuỗi, chứ không liên tục trong suốt quá trình sinh. Xử lý phần thưởng thưa (*sparse rewards*) từ lâu đã là một mối quan tâm trong học tăng cường, và là một trong những thách thức của nhiều ứng dụng thực tế. Ví dụ, trong robot học, hàm phần thưởng thường cần được "nắn" lại để dễ tối ưu hơn thay vì chỉ dựa vào phần thưởng ở cuối chuỗi. Nhiều phương pháp đã được phát triển để giải quyết vấn đề này. Một cách phổ biến là định hình phần thưởng (*reward shaping*), trong đó hàm ban đầu được sửa đổi để bao gồm các phần thưởng trung gian, nhờ đó cung cấp phản hồi tức thời hơn. Ngoài ra, ta có thể áp dụng học theo giáo trình (*curriculum learning*) để sắp xếp các nhiệm vụ sao cho độ phức tạp của chúng tăng dần. Điều này giúp mô hình thành thạo các nhiệm vụ đơn giản trước và dần chuẩn bị cho những thử thách phức tạp hơn. Có nhiều phương pháp như vậy có thể làm giảm tác động của phần thưởng thưa, chẳng hạn động lực nội tại (*intrinsic motivation*) và phát lại kinh nghiệm nhìn lại (*hindsight experience replay*). Phần lớn các phương pháp này mang tính tổng quát và được thảo luận rộng rãi trong tài liệu về học tăng cường nói chung, như cuốn sách giáo khoa kinh điển của Sutton và Barto [2018].

Dù ở đây chúng ta không bàn chi tiết các phương pháp giảm nhẹ phần thưởng thưa, một câu hỏi thú vị nảy sinh: tại sao phần thưởng thưa lại thành công đến vậy trong RLHF? Nhớ lại từ Mục 10.3.1 rằng tín hiệu giám sát nhận được tại mỗi bước thời gian t không phải là phần thưởng của hành động hiện tại, mà là một dạng phần thưởng tích lũy từ t cho đến bước thời gian cuối cùng. Những tín hiệu giám sát như vậy là *dày* trên chuỗi, vì phần thưởng nhận được ở cuối chuỗi có thể được truyền ngược về bước thời gian đó, bất kể đó là bước nào. Nói cách khác, phần thưởng thưa đã được biến đổi thành tín hiệu giám sát dày. Hơn nữa, từ góc độ định hình phần thưởng, Ng và cộng sự [1999] chỉ ra rằng phần thưởng tại t được tính bằng

$$r'(s_t, a_t, s_{t+1}) \;=\; r(s_t, a_t, s_{t+1}) + f(s_t, a_t, s_{t+1}), \tag{10.64}$$

trong đó $r'(\cdot)$ là hàm phần thưởng đã biến đổi, $r(\cdot)$ là hàm phần thưởng ban đầu, và $f(\cdot)$ là hàm phần thưởng định hình (*shaping reward function*). Để đảm bảo tính tối ưu của chính sách dưới hàm phần thưởng đã biến đổi, hàm phần thưởng định hình có thể được cho ở dạng

$$f(s_t, a_t, s_{t+1}) \;=\; \gamma\Phi(s_{t+1}) - \Phi(s_t), \tag{10.65}$$

trong đó $\Phi(s)$ được gọi là giá trị thế năng (*potential value*) của trạng thái s. Nếu ta định nghĩa $\Phi(s)$ là hàm giá trị như trong Phương trình (10.15) và thế Phương trình (10.65) vào Phương trình (10.64), ta

[8] Một mô hình được gọi là bất định (*underdetermined*) nếu có nhiều bộ tham số khác nhau cùng có thể đạt được một mục tiêu như nhau.

thu được

$$r'(s_t, a_t, s_{t+1}) \;=\; r(s_t, a_t, s_{t+1}) + \gamma V(s_{t+1}) - V(s_t). \tag{10.66}$$

Thật thú vị khi thấy biểu thức này tương đương với sai số chênh lệch thời gian (*temporal-difference error*) một bước được dùng trong PPO. Điều này liên hệ các phương pháp dựa trên lợi thế (*advantage*) với định hình phần thưởng: lợi thế về bản chất là một phần thưởng đã được định hình.

Chú giải của người dịch: Định hình phần thưởng dựa trên thế năng

Tên gọi "thế năng" gợi nhớ đến thế năng trong Vật lý: công của trọng lực khi đi từ A đến B chỉ phụ thuộc vào hiệu thế năng ở hai đầu, không phụ thuộc đường đi. Tương tự, nếu cộng $f = \gamma\Phi(s_{t+1}) - \Phi(s_t)$ ở mọi bước rồi tính tổng dọc theo cả chuỗi (tạm lấy $\gamma = 1$), các số hạng ở giữa triệt tiêu nhau: $(\Phi(s_2) - \Phi(s_1)) + (\Phi(s_3) - \Phi(s_2)) + \cdots = \Phi(s_{\text{cuối}}) - \Phi(s_1)$. Vì vậy phần thưởng bổ sung không làm thay đổi việc chuỗi hành động nào là tốt nhất, nhưng lại cho tác tử những "gợi ý" ở từng bước, giúp học nhanh hơn.

Mặt khác, một trong những lý do dùng phần thưởng ở cuối chuỗi nằm ở bản chất của các nhiệm vụ RLHF. Khác với các môi trường học tăng cường truyền thống, nơi tác tử (*agent*) tương tác với một môi trường động, các nhiệm vụ RLHF thường liên quan đến việc ra quyết định phức tạp dựa trên các quá trình ngôn ngữ hoặc các quá trình nhận thức cấp cao khác. Những quá trình này không dễ tạo ra các phần thưởng trung gian thường xuyên và có ý nghĩa, vì chất lượng và mức độ phù hợp của các hành động chỉ có thể được đánh giá đầy đủ sau khi quan sát tác động của chúng trong bối cảnh lớn hơn của toàn bộ chuỗi hoặc nhiệm vụ. Trong trường hợp này, tín hiệu phần thưởng dựa trên phản hồi của con người, tuy rất thưa, thường lại rất giàu thông tin và chính xác. Do đó, tính thưa này, cùng với lượng thông tin lớn và độ chính xác cao của phản hồi từ con người, có thể khiến việc học vừa bền vững vừa hiệu quả.

3. Phần thưởng chi tiết

Với nhiều ứng dụng, mục tiêu của chúng ta sẽ phức tạp hơn việc chỉ đánh giá toàn bộ một văn bản. Chẳng hạn, trong phân tích cảm xúc (*sentiment analysis*), ta thường không chỉ xác định cảm xúc của một văn bản, mà cần phân tích cảm xúc chi tiết hơn bằng cách gắn nó với các khía cạnh cụ thể của chủ đề được bàn đến trong văn bản. Xét câu "The camera of the phone is excellent, but the battery life is disappointing" (Camera của điện thoại thì tuyệt vời, nhưng thời lượng pin thì đáng thất vọng). Trong ví dụ này, ta cần phân tích riêng cảm xúc được bày tỏ về camera và về pin. Kiểu phân tích này, gọi là phân tích cảm xúc theo khía cạnh (*aspect-based sentiment analysis*), giúp hiểu nhận xét của khách hàng ở mức chi tiết hơn so với phân tích cảm xúc thông thường.

Với bài toán mô hình hóa phần thưởng, ta thường cần mô hình hóa các phần khác nhau của một chuỗi ở mức chi tiết tương tự. Một cách đơn giản để làm điều này là chia chuỗi thành các đoạn riêng biệt và tính phần thưởng cho từng đoạn một cách độc lập [Wu và cộng sự, 2023b]. Giả sử một chuỗi token đầu ra $\mathbf{y}$ có thể được chia thành n_s đoạn $\{\bar{\mathbf{y}}_1, ..., \bar{\mathbf{y}}_{n_s}\}$ dựa trên một tiêu chí nào đó. Khi đó ta có thể dùng mô hình phần thưởng để đánh giá từng đoạn. Bằng cách xét đầu vào $\mathbf{x}$, toàn bộ đầu ra $\mathbf{y}$ và đoạn cụ thể $\bar{\mathbf{y}}_k$, điểm thưởng cho đoạn thứ k là

$$r^k \;=\; r(\mathbf{x}, \mathbf{y}, \bar{\mathbf{y}}_k). \tag{10.67}$$

Tổng điểm thưởng cho toàn bộ chuỗi đầu ra khi đó chỉ đơn giản là tổng của các phần thưởng cấp đoạn này

$$r(\mathbf{x}, \mathbf{y}) \;=\; \sum_{k=1}^{n_s} r(\mathbf{x}, \mathbf{y}, \bar{\mathbf{y}}_k). \tag{10.68}$$

Phần thưởng $r(\mathbf{x}, \mathbf{y})$ này sau đó có thể được dùng để huấn luyện chính sách bằng các thuật toán học tăng cường tiêu chuẩn như PPO.

Một vấn đề của cách tiếp cận này là huấn luyện mô hình phần thưởng ở cấp đoạn không đơn giản như học từ sở thích của con người trên toàn bộ văn bản, vì rất khó thu thập dữ liệu sở thích của con người ở cấp đoạn. Với các bài toán dạng chấm điểm (ví dụ, ta chấm một đoạn theo mức độ chứa thông tin sai lệch), một cách đơn giản là gán điểm cho từng đoạn và huấn luyện mô hình phần thưởng bằng phương pháp theo điểm. Chẳng hạn, ta có thể dùng một LLM mạnh để chấm các chuỗi $\bar{\mathbf{y}}_1 ... \bar{\mathbf{y}}_{k-1}$ và $\bar{\mathbf{y}}_1 ... \bar{\mathbf{y}}_k$, thu được các điểm $s(\bar{\mathbf{y}}_1 ... \bar{\mathbf{y}}_{k-1})$ và $s(\bar{\mathbf{y}}_1 ... \bar{\mathbf{y}}_k)$. Khi đó, ta có thể định nghĩa điểm của đoạn $\bar{\mathbf{y}}_k$ là hiệu giữa $s(\bar{\mathbf{y}}_1 ... \bar{\mathbf{y}}_k)$ và $s(\bar{\mathbf{y}}_1 ... \bar{\mathbf{y}}_{k-1})$

$$s(\bar{\mathbf{y}}_k) \;=\; s(\bar{\mathbf{y}}_1 ... \bar{\mathbf{y}}_k) - s(\bar{\mathbf{y}}_1 ... \bar{\mathbf{y}}_{k-1}). \tag{10.69}$$

Với các điểm cấp đoạn này, ta có thể huấn luyện mô hình phần thưởng bằng một hàm mất mát hồi quy

$$\mathcal{L}_{\text{rating}} \;=\; \mathbb{E}_{(\mathbf{x}, \mathbf{y}, \bar{\mathbf{y}}_k) \sim \mathcal{D}_r} \big[(s(\bar{\mathbf{y}}_k) - r(\mathbf{x}, \mathbf{y}, \bar{\mathbf{y}}_k))^2 \big]. \tag{10.70}$$

Chú giải của người dịch: Điểm của một đoạn bằng "phần đóng góp thêm"

Giống như tính điểm đóng góp của từng cầu thủ bằng cách xem tỉ số thay đổi thế nào khi họ vào sân. Ví dụ, một câu trả lời gồm 3 đoạn; LLM chấm phần $\bar{\mathbf{y}}_1$ được 6 điểm, phần $\bar{\mathbf{y}}_1 \bar{\mathbf{y}}_2$ được 8 điểm, phần $\bar{\mathbf{y}}_1 \bar{\mathbf{y}}_2 \bar{\mathbf{y}}_3$ được 5 điểm. Theo Phương trình (10.69), đoạn 2 có điểm $8 - 6 = +2$ (làm câu trả lời tốt hơn), còn đoạn 3 có điểm $5 - 8 = -3$ (có lẽ chứa thông tin sai, làm câu trả lời tệ đi).

Đôi khi, việc căn chỉnh có thể được xem như một bài toán phân loại (*classification*), chẳng hạn ta đánh giá xem một đoạn có vấn đề về đạo đức hay không. Trong trường hợp này, đoạn văn có thể được gán nhãn là có đạo đức hoặc phi đạo đức, bởi con người hoặc bằng các bộ phân loại bổ sung. Với các nhãn rời rạc này, ta có thể huấn luyện mô hình phần thưởng bằng các hàm mất mát phân loại. Giả sử nhãn đúng là $\hat{r} \in \{1, -1\}$, trong đó -1 chỉ một đoạn phi đạo đức và 1 chỉ một đoạn có đạo đức, và gọi $r(\mathbf{x}, \mathbf{y}, \bar{\mathbf{y}}_k)$ là đầu ra vô hướng do mô hình phần thưởng tạo ra[9]. Khi đó mô hình phần thưởng có thể được huấn luyện bằng hàm mất mát bản lề (*hinge loss*):

$$\mathcal{L}_{\text{hinge}} \;=\; \mathbb{E}_{(\mathbf{x}, \mathbf{y}, \bar{\mathbf{y}}_k) \sim \mathcal{D}_r} \big[\max(0, 1 - r(\mathbf{x}, \mathbf{y}, \bar{\mathbf{y}}_k) \cdot \hat{r}) \big], \tag{10.71}$$

trong đó $\hat{r} \in \{1, -1\}$ ký hiệu nhãn đúng.

Chú giải của người dịch: Hàm mất mát bản lề

Đặt $m = r \cdot \hat{r}$. Nếu mô hình đoán đúng dấu và "tự tin" ($m \geq 1$) thì $\max(0, 1 - m) = 0$: không bị phạt. Ví dụ đoạn có đạo đức ($\hat{r} = 1$) và mô hình cho $r = 2$: mất mát $\max(0, 1 - 2) = 0$. Nếu mô hình cho $r = 0.3$ (đúng dấu nhưng chưa đủ chắc): mất mát 0.7. Nếu cho $r = -1$ (sai dấu): mất mát 2. Đồ thị của hàm này gãy khúc tại $m = 1$, trông giống một chiếc bản lề – vì vậy mà có tên gọi.

Vấn đề còn lại ở đây là cách chia $\mathbf{y}$ thành các đoạn. Một cách là định nghĩa phép chia có độ dài cố định, trong đó $\mathbf{y}$ được chia thành các khúc dài bằng nhau. Tuy nhiên, cách này không phải lúc nào cũng lý tưởng, vì nội dung của chuỗi có thể không khớp tốt với các ranh giới cố định. Một cách khác là chia $\mathbf{y}$ dựa trên các dấu hiệu ngôn ngữ hoặc ngữ nghĩa cụ thể, như ranh giới câu, sự chuyển chủ đề, hoặc các cấu trúc có ý nghĩa khác trong văn bản. Phép chia như vậy có thể thực hiện bằng các hệ thống phân

[9] Để mô hình phần thưởng có thể đưa ra xác suất phân loại hoặc logit, ta có thể điều chỉnh tầng tuyến tính mô tả ở Mục 10.3.2 cho phù hợp.

đoạn ngôn ngữ hoặc bằng cách dùng prompt để LLM xác định các điểm ngắt tự nhiên trong chuỗi. Một cách tiếp cận khác nữa là dùng các phương pháp phân đoạn động dựa trên độ phức tạp của chuỗi. Ví dụ, ranh giới đoạn có thể được đặt ở những chỗ điểm thưởng thay đổi đáng kể, điều có thể tương ứng với sự chuyển đổi trong nhiệm vụ đang được mô hình hóa.

4. Kết hợp các mô hình phần thưởng

Một mô hình phần thưởng có thể được xem là một đại diện thay thế (*proxy*) cho môi trường. Vì môi trường thật thường quá phức tạp hoặc không biết trước, nói chung không thể xây dựng một đại diện hoàn hảo cho môi trường. Kết quả là, việc căn chỉnh LLM quá mức theo đại diện không hoàn hảo này có thể làm giảm hiệu năng, hiện tượng được gọi là **vấn đề tối ưu hóa quá mức** (*overoptimization problem*) [Stiennon và cộng sự, 2020; Gao và cộng sự, 2023a][10]. Ta cũng có thể giải thích điều này qua định luật Goodhart (*Goodhart's law*), phát biểu rằng: *khi một thước đo trở thành mục tiêu, nó không còn là một thước đo tốt nữa* [Goodhart, 1984].

Giải quyết vấn đề tối ưu hóa quá mức không hề dễ, và hiện chưa có giải pháp nào hoàn toàn thỏa đáng. Cách lý tưởng có lẽ là xây dựng một mô hình phần thưởng "tiên tri" (*oracle*) có thể nắm bắt hoàn hảo mục tiêu thật của nhiệm vụ và ngăn tác tử "gian lận". Tuy nhiên, tạo ra một mô hình như vậy là cực kỳ khó do độ phức tạp của môi trường thực tế, cũng như thách thức trong việc định nghĩa mọi yếu tố liên quan góp phần tạo nên kết quả mong muốn. Thay vào đó, một cách thực tế hơn là kết hợp nhiều mô hình phần thưởng, nhờ đó giảm sự lệch giữa mục tiêu huấn luyện và mục tiêu thật vốn nảy sinh khi chỉ dùng một mô hình phần thưởng duy nhất [Coste và cộng sự, 2024].

Kết hợp một tập các mô hình phần thưởng là việc đơn giản về mặt khái niệm và thường có thể được phát biểu như một bài toán học kết hợp (*ensemble learning*). Một cách đơn giản nhưng phổ biến là kết hợp đầu ra của các mô hình này để có được ước lượng phần thưởng đáng tin cậy hơn:

$$r_{\text{combine}} = \sum_{k=1}^{K} w_k \cdot r_k(\mathbf{x}, \mathbf{y}), \tag{10.72}$$

trong đó $r_k(\cdot)$ là mô hình phần thưởng thứ k trong tập kết hợp, w_k là trọng số của $r_k(\cdot)$, và K là số mô hình phần thưởng. Phần thưởng kết hợp này sau đó có thể được dùng để giám sát việc huấn luyện một chính sách. Thực ra, có nhiều cách để kết hợp các mô hình khác nhau, chẳng hạn có thể dự đoán bằng lấy trung bình mô hình Bayes (*Bayesian model averaging*) hoặc xây dựng một mạng hợp nhất để học cách kết hợp dự đoán từ các mô hình khác nhau. Một lựa chọn khác là phát biểu nhiệm vụ này như một bài toán tối ưu hóa đa mục tiêu (*multi-objective optimization*) và dùng nhiều mô hình phần thưởng để huấn luyện chính sách đồng thời. Các phương pháp này đã được thảo luận rộng rãi trong tài liệu về tối ưu hóa và học máy [Miettinen, 1999; Bishop, 2006].

Chú giải của người dịch: Vì sao kết hợp nhiều "giám khảo"?

Phương trình (10.72) giống cách tính điểm trung bình môn có hệ số: mỗi mô hình phần thưởng là một "giám khảo", w_k là hệ số của giám khảo đó. Ví dụ, hai mô hình cho điểm $r_1 = 4$, $r_2 = 2$ với $w_1 = 0.7$, $w_2 = 0.3$ thì $r_{\text{combine}} = 0.7 \cdot 4 + 0.3 \cdot 2 = 3.4$. Mỗi giám khảo đều có những "điểm mù" riêng mà chính sách có thể lợi dụng; nhưng khó tìm được một mẹo lừa được *tất cả* giám khảo cùng lúc, nên lách phần thưởng trở nên khó hơn.

[10] Vấn đề này còn được gọi là **lách phần thưởng** (*reward hacking*) hay **lợi dụng phần thưởng** (*reward gaming*) [Krakovna và cộng sự, 2020; Skalse và cộng sự, 2022; Pan và cộng sự, 2022], chỉ hiện tượng tác tử cố "đánh lừa" mô hình phần thưởng nhưng không làm cho hành động của mình khớp với mục tiêu thực sự của nhiệm vụ. Hãy tưởng tượng một học sinh được giao bài tập về nhà và được thưởng điểm hoặc lời khen khi hoàn thành. Học sinh đó có thể tìm cách hoàn thành bài tập với ít công sức nhất để tối đa hóa phần thưởng, chẳng hạn sao chép lời giải từ internet hoặc từ các bài tập trước, thay vì tự giải các bài toán.

Ngoài các phương pháp kết hợp mô hình, một vấn đề quan trọng khác là làm sao thu thập hoặc xây dựng nhiều mô hình phần thưởng khác nhau. Một trong những cách đơn giản nhất là dùng các kỹ thuật học kết hợp, chẳng hạn xây dựng các mô hình phần thưởng đa dạng từ các tập con khác nhau của một tập dữ liệu cho trước hoặc từ nhiều nguồn dữ liệu khác nhau. Với RLHF, cũng có thể xây dựng các mô hình phần thưởng dựa trên việc xem xét các khía cạnh khác nhau của căn chỉnh. Ví dụ, ta có thể xây dựng một mô hình phần thưởng để đánh giá độ chính xác về mặt sự kiện của đầu ra và một mô hình phần thưởng khác để đánh giá tính đầy đủ của đầu ra. Hai mô hình này bổ sung cho nhau và có thể được kết hợp để cải thiện việc đánh giá tổng thể đầu ra. Một cách tiếp cận khác là dùng các LLM có sẵn khác nhau làm mô hình phần thưởng. Cách này đơn giản và thực tế, vì có nhiều LLM đã được tiền huấn luyện mạnh sẵn có và ta chỉ cần dùng chúng với ít hoặc không cần sửa đổi. Điều này đặt ra một câu hỏi thú vị: liệu một LLM được căn chỉnh theo các LLM khác có thể vượt qua chính các LLM đó không? Thoạt nhìn có lẽ là không. Theo trực giác, người ta có thể cho rằng một LLM đích chỉ bắt chước các LLM khác dựa trên sự giám sát hạn chế sẽ khó nắm bắt đầy đủ những hành vi tinh tế của các "người giám sát" mình. Tuy nhiên, với khả năng tổng quát hóa mạnh của LLM, cách tiếp cận này thực ra có thể khá hữu ích. Ví dụ, dùng các LLM mã nguồn mở hoặc thương mại làm mô hình phần thưởng đã cho thấy hiệu năng mạnh trong việc căn chỉnh LLM, thậm chí đạt kết quả tiên tiến nhất trên một số nhiệm vụ phổ biến [Lambert và cộng sự, 2024].

10.4.2 Tối ưu hóa sở thích trực tiếp

Mặc dù học mô hình phần thưởng là một bước tiêu chuẩn trong học tăng cường, nó khiến toàn bộ quá trình huấn luyện phức tạp hơn nhiều so với huấn luyện có giám sát thông thường. Bản thân việc huấn luyện một mô hình phần thưởng đáng tin cậy đã không dễ, và một mô hình phần thưởng được huấn luyện kém có thể ảnh hưởng lớn đến kết quả học chính sách. Giờ ta xét một phương pháp căn chỉnh thay thế, gọi là **tối ưu hóa sở thích trực tiếp** (*direct preference optimization*) (**DPO**), phương pháp này đơn giản hóa khung huấn luyện bằng cách loại bỏ nhu cầu mô hình hóa phần thưởng một cách tường minh [Rafailov và cộng sự, 2024]. Phương pháp này tối ưu hóa trực tiếp chính sách dựa trên sở thích của người dùng, thay vì huấn luyện một mô hình phần thưởng riêng. Nhờ đó, ta có thể đạt được việc căn chỉnh theo sở thích của con người theo cách giống như học có giám sát. Hình 10.10 so sánh phương pháp RLHF tiêu chuẩn và phương pháp DPO.

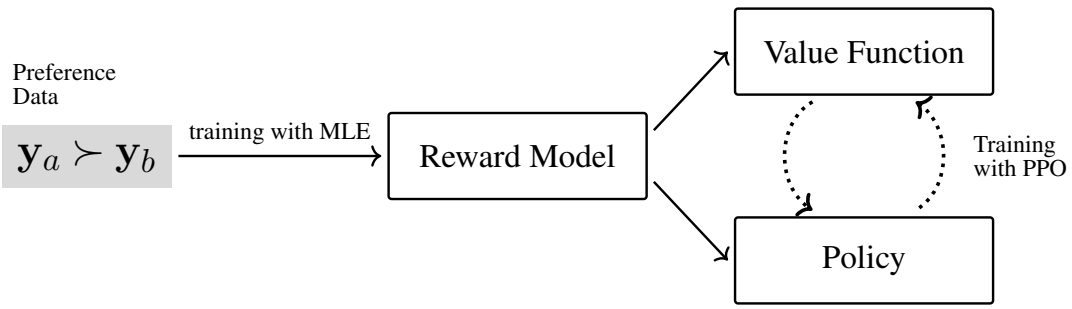
Trước khi suy ra mục tiêu của DPO, hãy cùng điểm lại mục tiêu huấn luyện chính sách được dùng trong RLHF. Như đã bàn ở Mục 10.3.3, chính sách thường được huấn luyện bằng cách tối ưu một hàm mất mát có số hạng phạt. Phương pháp DPO giả định một hàm mất mát đơn giản, trong đó chất lượng của đầu ra $\mathbf{y}$ khi cho đầu vào $\mathbf{x}$ được đánh giá bởi mô hình phần thưởng $r(\mathbf{x}, \mathbf{y})$. Mục tiêu huấn luyện do đó là

$$\tilde{\theta} \;=\; \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}} \mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})} \Big[\underbrace{-r(\mathbf{x}, \mathbf{y})}_{\text{mất mát}} + \beta \underbrace{\big(\log \pi_\theta(\mathbf{y}|\mathbf{x}) - \log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \big)}_{\text{hình phạt}} \Big]. \tag{10.73}$$

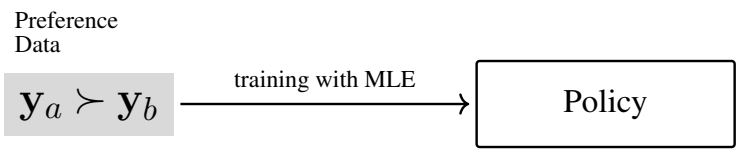
Lưu ý rằng trong bài toán tối ưu này, chỉ có số hạng $\pi_\theta(\mathbf{y}|\mathbf{x})$ phụ thuộc vào chính sách đích $\pi_\theta(\cdot)$. Cả mô hình phần thưởng $r(\mathbf{x}, \mathbf{y})$ lẫn mô hình tham chiếu (*reference model*) $\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x})$ đều được giả định là cố định khi cho $\mathbf{x}$ và $\mathbf{y}$. Đây là một giả định mạnh so với PPO, nhưng như sẽ thấy ở phần sau, nó đơn giản hóa bài toán và rất quan trọng để suy ra mục tiêu của DPO.

Vì θ là biến ta muốn tối ưu, ta sắp xếp lại vế phải của Phương trình (10.73) để làm rõ sự phụ thuộc vào θ:

$$\tilde{\theta}$$



(a) RLHF (PPO)



(b) DPO

Hình 10.10: RLHF tiêu chuẩn (PPO) so với DPO. Trong RLHF, dữ liệu sở thích của con người được dùng để huấn luyện một mô hình phần thưởng (*Reward Model*), mô hình này sau đó được dùng trong việc huấn luyện chính sách (*Policy*) cũng như hàm giá trị (*Value Function*). Trong DPO, việc dùng dữ liệu sở thích của con người trực tiếp hơn, và chính sách được huấn luyện trên dữ liệu này mà không cần huấn luyện mô hình phần thưởng. (Nhãn trong hình: *Preference Data* – dữ liệu sở thích; *training with MLE* – huấn luyện bằng ước lượng hợp lý cực đại; *Training with PPO* – huấn luyện bằng PPO.)

$$
\begin{aligned}
&= \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\left[\beta \log \pi_\theta(\mathbf{y}|\mathbf{x}) - \beta \log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) - r(\mathbf{x},\mathbf{y})\right] \\
&= \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\left[\log \pi_\theta(\mathbf{y}|\mathbf{x}) - \left(\log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) + \frac{1}{\beta} r(\mathbf{x},\mathbf{y})\right)\right] \\
&= \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\Big[\underbrace{\log \pi_\theta(\mathbf{y}|\mathbf{x})}_{\text{phụ thuộc } \theta} - \underbrace{\log\left(\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp\left(\tfrac{1}{\beta} r(\mathbf{x},\mathbf{y})\right)\right)}_{\text{không phụ thuộc } \theta}\Big].
\end{aligned} \tag{10.74}
$$

Phương trình này định nghĩa hàm mục tiêu là hiệu giữa log-xác suất của chính sách và một hàm khác của $\mathbf{y}$. Dạng hàm mục tiêu này chưa lý tưởng, vì ta thường muốn thấy hiệu giữa hai phân phối, để có thể diễn giải hiệu này như một loại độ phân kỳ giữa các phân phối. Một ý tưởng đơn giản là biến số hạng thứ hai (tức $\log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp(\frac{1}{\beta} r(\mathbf{x},\mathbf{y}))$) thành một phân phối log-xác suất trên miền của $\mathbf{y}$. Nếu ta xem $\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp(\frac{1}{\beta} r(\mathbf{x},\mathbf{y}))$ là một xác suất chưa chuẩn hóa của y, ta có thể biến nó thành xác suất đã chuẩn hóa bằng cách chia cho một hệ số chuẩn hóa (*normalization factor*):

$$
Z(\mathbf{x}) \;=\; \sum_{\mathbf{y}} \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp\left(\tfrac{1}{\beta} r(\mathbf{x},\mathbf{y})\right). \tag{10.75}
$$

Do đó, ta có thể định nghĩa một phân phối xác suất bởi

$$
\pi^*(\mathbf{y}|\mathbf{x}) \;=\; \frac{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp\left(\tfrac{1}{\beta} r(\mathbf{x},\mathbf{y})\right)}{Z(\mathbf{x})}. \tag{10.76}
$$

Sau đó ta viết lại Phương trình (10.74) thành

$$
\begin{aligned}
\tilde{\theta} &= \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\left[\log \pi_\theta(\mathbf{y}|\mathbf{x}) - \log \frac{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp\left(\tfrac{1}{\beta} r(\mathbf{x},\mathbf{y})\right)}{Z(\mathbf{x})}\right. \\
&\qquad\qquad \left. - \log Z(\mathbf{x})\right] \\
&= \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\left[\log \pi_\theta(\mathbf{y}|\mathbf{x}) - \log \pi^*(\mathbf{y}|\mathbf{x}) - \log Z(\mathbf{x})\right] \\
&= \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\left[\mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\left[\log \pi_\theta(\mathbf{y}|\mathbf{x}) - \log \pi^*(\mathbf{y}|\mathbf{x})\right]\right. \\
&\qquad\qquad \left. - \mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\left[\log Z(\mathbf{x})\right]\right] \\
&= \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\Big[\underbrace{D_{\text{KL}}\big(\pi_\theta(\cdot|\mathbf{x}) \,\|\, \pi^*(\cdot|\mathbf{x})\big)}_{\text{độ phân kỳ KL}} - \underbrace{\log Z(\mathbf{x})}_{\text{hằng số đối với } \theta}\Big].
\end{aligned} \tag{10.77}
$$

Vì $\log Z(\mathbf{x})$ không phụ thuộc vào θ, nó không ảnh hưởng đến kết quả của phép $\arg\min_\theta$, và có thể bỏ khỏi hàm mục tiêu. Giờ ta thu được một mục tiêu huấn luyện mới, tìm chính sách tối ưu π_θ bằng cách cực tiểu hóa độ phân kỳ KL (*KL divergence*) giữa $\pi_\theta(\cdot|\mathbf{x})$ và $\pi^*(\cdot|\mathbf{x})$

$$
\tilde{\theta} \;=\; \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\Big[D_{\text{KL}}\big(\pi_\theta(\cdot|\mathbf{x}) \,\|\, \pi^*(\cdot|\mathbf{x})\big)\Big]. \tag{10.78}
$$

Rõ ràng, nghiệm của bài toán tối ưu này là

$$
\begin{aligned}
\pi_\theta(\mathbf{y}|\mathbf{x}) &= \pi^*(\mathbf{y}|\mathbf{x}) \\
&= \frac{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp\left(\tfrac{1}{\beta} r(\mathbf{x},\mathbf{y})\right)}{Z(\mathbf{x})}.
\end{aligned} \tag{10.79}
$$

Chú giải của người dịch: Vì sao nghiệm "rõ ràng" là $\pi_\theta = \pi^*$?

Độ phân kỳ KL $D_{\text{KL}}(p \,\|\, q)$ đo mức "khác nhau" giữa hai phân phối xác suất p và q. Nó có hai tính chất quan trọng: luôn ≥ 0, và bằng 0 khi và chỉ khi $p = q$. Vậy giá trị nhỏ nhất có thể của Phương trình (10.78) là 0, đạt được đúng khi chính sách trùng với π^*. Về trực giác, π^* là mô hình tham chiếu được "nghiêng" về phía các câu trả lời có phần thưởng cao: mỗi câu trả lời được nhân thêm hệ số $\exp(r/\beta)$. Ví dụ, với $\beta = 1$, hai câu trả lời có $\pi_{\theta_{\text{ref}}} = 0.5$ và 0.5, phần thưởng $r = \ln 3$ và $r = \ln 1 = 0$: trọng số chưa chuẩn hóa là 1.5 và 0.5, $Z = 2$, nên $\pi^* = 0.75$ và 0.25. Hệ số β càng lớn thì sự nghiêng càng ít, tức chính sách càng bám sát mô hình tham chiếu.

Từ phương trình này, ta có thể biểu diễn phần thưởng $r(\mathbf{x}, \mathbf{y})$ bằng mô hình đích $\pi_\theta(\mathbf{y}|\mathbf{x})$, mô hình tham chiếu $\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x})$ và hệ số chuẩn hóa $Z(\mathbf{x})$:

$$r(\mathbf{x}, \mathbf{y}) \;=\; \beta\Big(\log \frac{\pi_\theta(\mathbf{y}|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x})} + \log Z(\mathbf{x})\Big). \tag{10.80}$$

Điều này thú vị ở chỗ ban đầu ta muốn học chính sách $\pi_\theta(\cdot)$ bằng mô hình phần thưởng $r(\mathbf{x}, \mathbf{y})$, nhưng cuối cùng lại thu được một biểu diễn của mô hình phần thưởng dựa trên chính sách. Với mô hình phần thưởng định nghĩa ở Phương trình (10.80), ta có thể áp dụng nó vào mô hình Bradley-Terry để tính xác suất sở thích (xem thêm Mục 10.3.2):

$$\begin{aligned}
\Pr_\theta(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x}) &= \text{Sigmoid}(r(\mathbf{x}, \mathbf{y}_a) - r(\mathbf{x}, \mathbf{y}_b)) \\
&= \text{Sigmoid}\Big(\beta\Big(\log \frac{\pi_\theta(\mathbf{y}_a|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}_a|\mathbf{x})} + \log Z(\mathbf{x})\Big) - \\
&\qquad \beta\Big(\log \frac{\pi_\theta(\mathbf{y}_b|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}_b|\mathbf{x})} + \log Z(\mathbf{x})\Big)\Big) \\
&= \text{Sigmoid}\Big(\beta \log \frac{\pi_\theta(\mathbf{y}_a|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}_a|\mathbf{x})} - \beta \log \frac{\pi_\theta(\mathbf{y}_b|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}_b|\mathbf{x})}\Big).
\end{aligned} \tag{10.81}$$

Công thức này rất đẹp vì nó biến hiệu các phần thưởng thành hiệu các hàm tỉ số, và ta không cần tính giá trị của $Z(\mathbf{x})$. Nhờ đó, ta không cần mô hình phần thưởng nữa, mà chỉ cần chính sách đích và mô hình tham chiếu để tính xác suất sở thích. Cuối cùng, ta có thể huấn luyện chính sách đích bằng cách cực tiểu hóa hàm mất mát DPO sau

$$\mathcal{L}_{\text{dpo}}(\theta) \;=\; -\mathbb{E}_{(\mathbf{x}, \mathbf{y}_a, \mathbf{y}_b) \sim \mathcal{D}_r}\big[\log \Pr_\theta(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x})\big]. \tag{10.82}$$

Dạng của hàm mất mát này rất giống với hàm mất mát dùng để huấn luyện mô hình phần thưởng trong RLHF (xem Phương trình (10.36)). Nhưng cần lưu ý rằng hàm mất mát ở đây phụ thuộc vào tham số của chính sách (tức θ) chứ không phải tham số của mô hình phần thưởng (tức ϕ).

Chú giải của người dịch: DPO hoạt động thế nào, nói một cách giản dị

Đại lượng $\log \frac{\pi_\theta(\mathbf{y}|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x})}$ cho biết mô hình đang huấn luyện đã *tăng* (nếu dương) hay *giảm* (nếu âm) xác suất sinh ra câu trả lời $\mathbf{y}$ bao nhiêu so với mô hình ban đầu. Cực tiểu hóa Phương trình (10.82) nghĩa là đẩy cho lượng tăng của câu trả lời được ưa thích $\mathbf{y}_a$ lớn hơn lượng tăng của câu trả lời bị chê $\mathbf{y}_b$. Ví dụ, nếu sau huấn luyện $\pi_\theta(\mathbf{y}_a|\mathbf{x})$ gấp đôi so với ban đầu còn $\pi_\theta(\mathbf{y}_b|\mathbf{x})$ chỉ còn một nửa, thì với $\beta = 1$ đối số của Sigmoid là $\ln 2 - \ln \frac{1}{2} = \ln 4 \approx 1.39$, cho $\Pr_\theta(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x}) = \frac{4}{4+1} = 0.8$. Toàn bộ quá trình chỉ cần dữ liệu cặp và hai mô hình ngôn ngữ, giống như huấn luyện có giám sát thông thường.

Ưu điểm chính của DPO nằm ở sự đơn giản và hiệu quả. Mục tiêu của DPO rất trực tiếp — nó tối ưu trực tiếp theo phản hồi dựa trên sở thích, thay vì dựa vào các mô hình phần thưởng được xây dựng riêng. Hơn nữa, DPO nhìn chung hiệu quả hơn về mẫu, vì nó học từ một tập dữ liệu cố định mà không cần quá trình lấy mẫu tốn kém về tính toán như trong PPO. Điều này khiến DPO trở thành một phương pháp phổ biến để căn chỉnh theo sở thích của con người, đặc biệt khi việc xây dựng và áp dụng mô hình phần thưởng thông qua học tăng cường gặp nhiều khó khăn.

Nhìn chung, DPO có thể được xem là một phương pháp **học tăng cường ngoại tuyến** (*offline reinforcement learning*), trong đó dữ liệu huấn luyện được thu thập trước và cố định, và không có sự khám phá (*exploration*). Ngược lại, các phương pháp học tăng cường trực tuyến (*online reinforcement learning*) như PPO, vốn đòi hỏi khám phá các trạng thái mới thông qua tương tác với môi trường (dùng mô hình phần thưởng làm đại diện), cũng có những ưu điểm riêng. Một trong những lợi ích của học tăng cường trực tuyến là nó cho phép tác tử liên tục thích nghi với những thay đổi của môi trường bằng cách học từ phản hồi theo thời gian thực. Điều này có nghĩa là, khác với các phương pháp ngoại tuyến, phương pháp trực tuyến không bị ràng buộc bởi bản chất tĩnh của dữ liệu thu thập sẵn và có thể khám phá ra các chiến lược giải quyết vấn đề mới. Ngoài ra, việc khám phá có thể giúp tác tử bao quát được một phạm vi rộng hơn các cặp trạng thái–hành động, nhờ đó cải thiện khả năng tổng quát hóa. Đây có thể là một ưu điểm quan trọng đối với LLM, vì khả năng tổng quát hóa được xem là một khía cạnh then chốt khi ứng dụng những mô hình lớn như vậy.

10.4.3 Tự động sinh dữ liệu sở thích

Mặc dù học từ sở thích của con người là một phương pháp hiệu quả và phổ biến để căn chỉnh LLM, việc gán nhãn dữ liệu sở thích rất tốn kém. Học từ phản hồi của con người không chỉ hạn chế về khả năng mở rộng quy mô, mà còn có thể đưa vào thiên lệch, vì phản hồi của con người vốn mang tính chủ quan. Do đó, ta có thể chuyển sang các phương pháp phản hồi từ AI (*AI feedback*) để giải quyết các vấn đề về khả năng mở rộng và tính nhất quán này mà không gặp những hạn chế gắn với người gán nhãn.

Cũng như việc sinh dữ liệu cho tinh chỉnh theo chỉ dẫn, sinh dữ liệu sở thích bằng LLM là việc đơn giản. Cho một tập đầu vào, trước tiên ta dùng một LLM để sinh ra các cặp đầu ra. Sau đó, ta dùng prompt để yêu cầu LLM xác định đầu ra được ưa thích hơn trong mỗi cặp, khi biết đầu vào tương ứng. Dưới đây là một ví dụ dùng prompt để LLM sinh nhãn sở thích cho một cặp phản hồi chăm sóc khách hàng.

Xét một tình huống chăm sóc khách hàng, trong đó một khách hàng đưa ra yêu cầu. Bạn sẽ xem xét hai phản hồi cho yêu cầu này. Hãy cho biết phản hồi nào được ưa thích hơn. Lưu ý rằng một phản hồi tốt cần lịch sự, rõ ràng và ngắn gọn. Nó nên giải quyết trực tiếp mối quan tâm của khách hàng, cung cấp thông tin hữu ích hoặc giải pháp, và giữ giọng điệu tích cực.

Yêu cầu:
Xin chào, tôi để ý rằng đơn hàng của tôi vẫn chưa đến, dù theo lịch đáng lẽ nó đã đến từ vài ngày trước. Bạn có thể cập nhật cho tôi tình trạng đơn hàng không? Cảm ơn!
Phản hồi A:
Tôi rất xin lỗi vì sự chậm trễ và hiểu rằng điều này có thể gây thất vọng thế nào. Chúng tôi đang cố gắng hết sức để nhanh chóng giải quyết việc này cho bạn.
Phản hồi B:
Ê, chuyện thường mà! Gói hàng của bạn tới lúc nào thì tới, khỏi phải căng thẳng.

Phản hồi A được ưa thích hơn.

Khi đã thu thập được các nhãn sở thích như vậy, ta có thể dùng chúng, cùng với đầu vào và hai phản hồi, để huấn luyện mô hình phần thưởng. Tất nhiên, ta có thể cân nhắc cung cấp một vài ví dụ minh họa hoặc dùng các kỹ thuật prompt tinh vi như CoT để cải thiện chất lượng gán nhãn. Chẳng hạn, ta có thể đưa vào prompt một ví dụ cho thấy cách thức và lý do một trong hai phản hồi được ưa thích hơn, dựa trên một lập luận theo kiểu chuỗi suy nghĩ (*chain of thought*).

Bên cạnh nhãn sở thích, ta cũng có thể lấy xác suất gắn với mỗi nhãn [Lee và cộng sự, 2023]. Một cách đơn giản là trích xác suất của các token nhãn, chẳng hạn "A" và "B", từ các xác suất do LLM đưa ra. Sau đó, ta có thể dùng hàm Softmax hoặc các kỹ thuật chuẩn hóa khác để chuẩn hóa lại các xác suất này thành một phân phối trên các nhãn. Các xác suất của nhãn được ưa thích này có thể đóng vai trò tín hiệu giám sát theo điểm để huấn luyện mô hình phần thưởng, như đã bàn ở Mục 10.4.1.

Chú giải của người dịch: Chuẩn hóa lại xác suất nhãn

Giả sử khi đọc prompt trên, LLM gán xác suất 0.6 cho token "A", 0.15 cho token "B", và phần còn lại cho các token khác (như "Phản", "Cả", …). Chỉ giữ hai nhãn và chia cho tổng $0.6+0.15 = 0.75$, ta được $\Pr(\mathrm{A}) = 0.8$, $\Pr(\mathrm{B}) = 0.2$. Con số 0.8 cho biết LLM khá (nhưng không tuyệt đối) chắc chắn rằng A tốt hơn – thông tin mềm này giàu hơn một nhãn cứng "A".

Mặc dù mở rộng quy mô sinh dữ liệu bằng AI là dễ dàng, việc đảm bảo độ chính xác và tính đa dạng của dữ liệu vẫn rất quan trọng. Ở đây, vấn đề chất lượng và đa dạng của dữ liệu không chỉ liên quan đến việc gán nhãn sở thích mà cả đầu vào và đầu ra của mô hình. Vì vậy, ta thường cần dùng nhiều kỹ thuật khác nhau để có được dữ liệu quy mô lớn, chất lượng cao. Ví dụ, có thể sinh ra các đầu ra và chú thích đa dạng của mô hình bằng cách dùng các LLM, prompt, ví dụ minh họa trong ngữ cảnh khác nhau, v.v. [Cui và cộng sự, 2024]. Dubois và cộng sự [2024] cho biết tính đa dạng trong dữ liệu sở thích theo cặp là quan trọng đối với việc huấn luyện LLM từ phản hồi của con người hay của AI.

Tuy học từ phản hồi của AI có khả năng mở rộng rất cao và nhìn chung nhất quán hơn phản hồi của con người, phương pháp này phù hợp hơn với các nhiệm vụ được định nghĩa rõ ràng, có sẵn các thước đo hiệu năng khách quan. Ngược lại, học từ phản hồi của con người có lợi thế hơn khi căn chỉnh các hệ thống AI với giá trị, sở thích của con người, và với các nhiệm vụ thực tế phức tạp đòi hỏi hiểu những bối cảnh tinh tế hoặc chủ quan. Các phương pháp này có thể được kết hợp để huấn luyện những LLM tận dụng được cả hiểu biết của con người lẫn khả năng mở rộng của phản hồi từ AI.

10.4.4 Căn chỉnh từng bước

Cho đến giờ, thảo luận của chúng ta về căn chỉnh chủ yếu tập trung vào việc dùng mô hình phần thưởng để đánh giá toàn bộ các cặp chuỗi đầu vào–đầu ra. Những phương pháp này có thể dễ dàng thích ứng với các tình huống mà tính đúng đắn của đầu ra có thể được kiểm chứng dựa trên đáp án cuối cùng. Chẳng hạn, trong nhiệm vụ tính giá trị một biểu thức toán học, mô hình phần thưởng có thể đưa ra phản hồi tích cực nếu đáp án đúng và phản hồi tiêu cực nếu đáp án sai. Tuy nhiên, trong nhiều bài toán đòi hỏi lập luận (*reasoning*) phức tạp, chỉ xem xét tính đúng đắn của kết quả cuối cùng là chưa đủ để học. Hãy tưởng tượng một học sinh chỉ được cho đáp số cuối cùng của một bài toán khó. Biết đáp số cuối cùng đúng hay sai không giúp học sinh nhận ra mình đã sai ở đâu và làm thế nào để tính ra đáp án đúng. Một cách tốt hơn là hướng dẫn học sinh bằng cách phân tích từng bước của quá trình giải bài, và giúp học sinh hiểu các khái niệm và logic nền tảng đằng sau các bước đó.

Ở Chương 9, chúng ta đã học các phương pháp CoT, dùng prompt để LLM viết ra một cách tường minh các bước trung gian hoặc quá trình lập luận cần thiết để đi đến một kết luận hay giải một bài toán. Ta đã thấy rằng chia nhỏ bài toán thành các phần nhỏ hơn có thể giúp dễ hiểu đường đi đến lời giải hơn

và làm tăng độ chính xác của đầu ra. Những phương pháp này có thể được mở rộng một cách tự nhiên sang việc căn chỉnh LLM, tức là ta giám sát mô hình trong các bước lập luận trung gian. Xét một nhiệm vụ lập luận trong đó LLM tạo ra một chuỗi các bước lập luận $\mathbf{y} = \{\bar{\mathbf{y}}_1, ..., \bar{\mathbf{y}}_{n_s}\}$ cho đầu vào đã cho. Kết quả của quá trình lập luận được giả định là nằm trong bước cuối $\bar{\mathbf{y}}_{n_s}$ và có thể dễ dàng kiểm chứng. Với bài toán lập luận này, Uesato và cộng sự [2022] chia các cách tinh chỉnh LLM thành hai loại:

- **Các phương pháp dựa trên kết quả** (*Outcome-based Approaches*). Việc giám sát chỉ diễn ra khi kết quả cuối cùng được kiểm chứng. Đây là phương pháp tiêu chuẩn để học từ phản hồi của con người mà chúng ta đã bàn trong chương này. Ví dụ, LLM được tối ưu để cực đại hóa một dạng phần thưởng $r(\mathbf{x}, \mathbf{y})$ nào đó.
- **Các phương pháp dựa trên quá trình** (*Process-based Approaches*). Việc giám sát được áp dụng cho mọi bước trung gian bên cạnh bước cuối cùng. Để làm được điều này, ta cần xây dựng một mô hình phần thưởng đưa ra tín hiệu giám sát ở mỗi bước, và thiết kế các hàm mất mát có thể tận dụng những tín hiệu giám sát như vậy.

Câu hỏi (trích từ Olympic Toán học Thiếu niên Quốc tế):
Jessica có rất nhiều ứng dụng trên điện thoại. Số ứng dụng cô có chia hết cho 5. Nếu cô tải thêm 10 ứng dụng nữa, cô vẫn sẽ có ít hơn 50 ứng dụng. Hỏi số ứng dụng ban đầu lớn nhất mà Jessica có thể có là bao nhiêu?

Phản hồi 1:

$\bar{\mathbf{y}}_1$: x phải là bội của 5, tức là $x \equiv 0 \pmod 5$.

$\bar{\mathbf{y}}_2$: Sau khi tải 10 ứng dụng, tổng tổng số ứng dụng vẫn phải nhỏ hơn 50: $x + 10 < 50$. Giải bất phương trình này: $x < 40$.

$\bar{\mathbf{y}}_3$: Vì x phải là bội của 5 và nhỏ hơn 40, giá trị lớn nhất của x là 35.

$\bar{\mathbf{y}}_4$: Kiểm tra kết quả. Nếu $x = 35$ thì sau khi tải 10 ứng dụng, Jessica sẽ có $35 + 10 = 45$, thỏa mãn $45 < 50$.

$\bar{\mathbf{y}}_5$: Số ứng dụng ban đầu lớn nhất là $\underline{35}$ (*đúng*).

Phản hồi 2:

$\bar{\mathbf{y}}_1$: Số ứng dụng chia hết cho 5, nên x phải là bội của 5.

$\bar{\mathbf{y}}_2$: Nếu cô tải thêm 10 ứng dụng, tổng mới sẽ là $x + 10$, và số này phải nhỏ hơn 50. Do đó $x + 10 < 50$. Rút gọn ta được $x < 40$.

$\bar{\mathbf{y}}_3$: x phải nhỏ hơn hoặc bằng bội lớn nhất của 5, tức là 40. có vấn đề

$\bar{\mathbf{y}}_4$: Nhưng $x + 10$ không được lớn hơn hoặc bằng 50. Nên ta cần trừ 5 từ 40. có vấn đề

$\bar{\mathbf{y}}_5$: Do đó, kết quả cuối cùng là $\underline{35}$ (*đúng*).

Hình 10.11: Hai phản hồi của LLM cho một bài toán. Ở phản hồi 1, cả kết quả cuối cùng lẫn mọi bước lập luận đều đúng. Ở phản hồi 2, kết quả cuối cùng đúng, nhưng có những lỗi trong quá trình lập luận (tô màu đỏ). Với các phương pháp dựa trên kết quả, cả hai phản hồi đều được coi là đúng. Với các phương pháp dựa trên quá trình, các lỗi trong phản hồi 2 có thể được tính đến trong mô hình hóa phần thưởng.

Hình 10.11 cho thấy hai đầu ra của LLM cho một bài toán ví dụ. Dù LLM đưa ra đáp án cuối cùng đúng trong cả hai trường hợp, nó mắc lỗi trong quá trình giải ở đầu ra thứ hai. Các phương pháp dựa trên kết quả bỏ qua những lỗi này và đưa ra phản hồi tích cực cho toàn bộ lời giải. Ngược lại, các phương pháp dựa trên quá trình có thể tính đến những lỗi này và cung cấp thêm hướng dẫn về các bước lập luận chi tiết.

Chú giải của người dịch: Lỗi ở phản hồi 2 nằm ở đâu?

Bước $\bar{\mathbf{y}}_3$ của phản hồi 2 nói $x \leq 40$, mâu thuẫn với bước trước ($x < 40$, tức x không thể bằng 40). Bước $\bar{\mathbf{y}}_4$ "trừ 5 từ 40" là một cách sửa chữa mang tính mò mẫm, không có lập luận chặt chẽ, chỉ tình cờ cho đáp số đúng 35. Giống như bài kiểm tra ở trường: giáo viên chấm cả lời giải chứ không chỉ đáp số, vì đáp số đúng từ lập luận sai thì lần sau rất dễ sai.

Một thách thức then chốt của các phương pháp dựa trên quá trình là ta cần có phản hồi ở cấp từng

bước trong một đường lập luận (có thể rất) dài. Ta có thể thu thập hoặc sinh các đường lập luận ứng với các bài toán từ những tập dữ liệu có sẵn. Sau đó, các chuyên gia con người gán cho mỗi bước một nhãn về tính đúng đắn. Các chú thích này có thể được dùng để huấn luyện trực tiếp LLM hoặc làm phần thưởng trong mô hình hóa phần thưởng. Tuy nhiên, trong thực tế, người ta thường dùng các chú thích phong phú hơn [Lightman và cộng sự, 2024]. Ngoài các nhãn *đúng* và *sai*, một bước còn có thể được gán nhãn *trung tính* để chỉ rằng tuy bước đó có thể đúng về mặt kỹ thuật, nó vẫn có thể có vấn đề trong toàn bộ quá trình lập luận. Hơn nữa, để nâng cao hiệu quả gán nhãn dữ liệu, có thể áp dụng các kỹ thuật như học chủ động (*active learning*). Việc chỉ ra các lỗi hiển nhiên thường không đóng góp đáng kể vào việc học từ các lỗi suy luận. Thay vào đó, gán nhãn những bước mà mô hình tự tin cho là đúng nhưng thực ra có vấn đề thường hiệu quả hơn.

Với một tập các đường lập luận đã được gán nhãn ở cấp bước cùng các đầu vào tương ứng, ta có thể huấn luyện một mô hình phần thưởng để cung cấp phản hồi giám sát việc học chính sách. Mô hình phần thưởng có thể được xem là một mô hình phân loại, nên kiến trúc của nó có thể là một bộ giải mã Transformer với một tầng Softmax đặt ở trên cùng. Tại bước k, mô hình phần thưởng nhận cả mô tả bài toán (ký hiệu $\mathbf{x}$) lẫn các bước lập luận đã sinh ra cho đến lúc đó (ký hiệu $\bar{\mathbf{y}}_{\leq k}$) làm đầu vào, và đưa ra một phân phối xác suất trên tập nhãn $\{\text{đúng}, \text{sai}\}$ hoặc $\{\text{đúng}, \text{sai}, \text{trung tính}\}$. Sau đó, mô hình phần thưởng đã học được dùng để đánh giá các đường lập luận bằng cách đánh giá tính đúng đắn của từng bước. Một cách đơn giản để mô hình hóa tính đúng đắn là đếm số bước được phân loại là đúng:

$$r(\mathbf{x}, \mathbf{y}) \;=\; \sum_{k=1}^{n_s} \delta(\text{đúng}, C(\mathbf{x}, \bar{\mathbf{y}}_{\leq k})), \tag{10.83}$$

trong đó $C(\mathbf{x}, \bar{\mathbf{y}}_{\leq k})$ ký hiệu nhãn có xác suất lớn nhất. Ta cũng có thể dùng log-xác suất phân loại để định nghĩa phần thưởng của toàn bộ đường lập luận

$$r(\mathbf{x}, \mathbf{y}) \;=\; \sum_{k=1}^{n_s} \log \Pr(\text{đúng}|\mathbf{x}, \bar{\mathbf{y}}_{\leq k}), \tag{10.84}$$

trong đó $\Pr(\text{đúng}|\mathbf{x}, \bar{\mathbf{y}}_{\leq k})$ ký hiệu xác suất của nhãn "đúng" do mô hình phần thưởng đưa ra. Điểm thưởng $r(\mathbf{x}, \mathbf{y})$ sau đó có thể được dùng để huấn luyện chính sách trong RLHF như thường lệ.

Chú giải của người dịch: Tính thử hai cách cho điểm

Hàm $\delta(a, b)$ bằng 1 nếu $a = b$ và bằng 0 nếu ngược lại. Với phản hồi 2 ở Hình 10.11 (5 bước), giả sử mô hình phần thưởng cho xác suất "đúng" ở các bước lần lượt là $0.9,\ 0.9,\ 0.2,\ 0.3,\ 0.8$. Theo Phương trình (10.83), có 3 bước được phân loại là đúng (xác suất > 0.5), nên $r = 3$; còn phản hồi 1 với mọi bước đúng sẽ được $r = 5$. Theo Phương trình (10.84), $r = \ln 0.9 + \ln 0.9 + \ln 0.2 + \ln 0.3 + \ln 0.8 \approx -3.33$: một bước rất đáng ngờ (xác suất nhỏ) sẽ kéo tổng xuống mạnh, vì ln của số gần 0 rất âm.

Tuy chúng ta giới hạn thảo luận ở các bài toán toán học, các phương pháp mô tả ở đây mang tính tổng quát và có thể áp dụng cho rất nhiều nhiệm vụ liên quan đến suy luận và ra quyết định nhiều bước. Hơn nữa, ta có thể xét nhiều khía cạnh khi đánh giá chất lượng một bước, chứ không chỉ tính đúng đắn của nó. Ví dụ, trong các hệ thống hội thoại, phản hồi không chỉ phải chính xác mà còn phải phù hợp với ngữ cảnh qua nhiều lượt hội thoại. Nếu một mô hình đưa ra phản hồi đúng nhưng không duy trì được sự mạch lạc trong ngữ cảnh của cuộc hội thoại đang diễn ra, phản hồi ở cấp bước có thể giúp mô hình nhận ra và sửa những sự lệch lạc như vậy. Cũng cần lưu ý rằng các phương pháp dựa trên quá trình có liên hệ với các phương pháp mô hình hóa phần thưởng chi tiết đã bàn ở Mục 10.4.1. Tất cả các phương pháp này về bản chất đều nhằm cung cấp sự giám sát chi tiết hơn cho LLM bằng cách chia đầu ra của

chúng thành các bước nhỏ hơn, dễ xử lý hơn. Tuy nhiên, phản hồi dựa trên quá trình tập trung hơn vào việc đánh giá tính đúng đắn của một bước dựa trên các bước đứng trước nó, còn các phương pháp ở Mục 10.4.1 nhấn mạnh việc đánh giá từng bước một cách độc lập.

Ý tưởng căn chỉnh LLM từng bước có tiềm năng ứng dụng to lớn, đặc biệt khi xét đến xu hướng gần đây chuyển sang các nhiệm vụ lập luận phức tạp hơn trong việc sử dụng LLM. Ví dụ, cả hai mô hình GPT o1 và o3 đều được thiết kế với các kỹ thuật suy luận tiên tiến hơn (như CoT nội tại dài) để giải các bài toán thách thức như suy luận khoa học và toán học [OpenAI, 2024]. Những nhiệm vụ này thường dựa trên các đường lập luận dài và phức tạp, do đó việc đưa vào các tín hiệu giám sát chi tiết trong quá trình lập luận có vẻ là thiết yếu. Hơn nữa, từ góc độ thực tiễn, giám sát hiệu quả trên các đường lập luận dài không chỉ cải thiện hiệu năng lập luận mà còn giúp mô hình loại bỏ những bước lập luận không cần thiết, nhờ đó giảm độ phức tạp của suy luận và tăng hiệu quả.

10.4.5 Căn chỉnh tại thời điểm suy luận

Từ đầu mục này đến giờ, chúng ta đã tìm hiểu nhiều phương pháp căn chỉnh mô hình theo sở thích và chú thích của con người. Tuy nhiên, một trong những hạn chế đáng kể của nhiều phương pháp như vậy là LLM phải được tinh chỉnh. Với RLHF và các biến thể của nó, huấn luyện LLM bằng mô hình phần thưởng có thể tốn kém về tính toán và không ổn định, làm tăng độ phức tạp và chi phí khi áp dụng các phương pháp này. Trong trường hợp đó, ta có thể cân nhắc thực hiện căn chỉnh tại thời điểm suy luận (*inference*), nhờ đó tránh được độ phức tạp và công sức phát sinh thêm.

Một cách đơn giản để căn chỉnh tại thời điểm suy luận là dùng mô hình phần thưởng để chọn đầu ra tốt nhất trong N đầu ra thay thế do LLM sinh ra, phương pháp được gọi là **lấy mẫu tốt nhất trong N** (*Best-of-N sampling*) (**lấy mẫu BoN**). Ta có thể xem lấy mẫu BoN như một dạng xếp hạng lại (*reranking*). Thực ra, các phương pháp xếp hạng lại đã được dùng rộng rãi trong các nhiệm vụ NLP, chẳng hạn dịch máy (*machine translation*), từ lâu. Chúng thường được áp dụng trong những tình huống mà việc huấn luyện các mô hình phức tạp tốn kém. Trong những trường hợp như vậy, việc xếp hạng lại trực tiếp các đầu ra cho phép tích hợp các mô hình phức tạp này với chi phí rất thấp[11].

Trong quá trình lấy mẫu BoN, LLM nhận chuỗi đầu vào $\mathbf{x}$ và sinh ra N chuỗi đầu ra khác nhau $\{\hat{\mathbf{y}}_1, ..., \hat{\mathbf{y}}_N\}$:

$$\{\hat{\mathbf{y}}_1, ..., \hat{\mathbf{y}}_N\} \;=\; \underset{\mathbf{y}}{\text{argTopN}} \left[\Pr(\mathbf{y}|\mathbf{x}) \right], \tag{10.85}$$

trong đó phép toán argTopN trả về N đầu ra đứng đầu làm cực đại hàm $\Pr(\mathbf{y}|\mathbf{x})$. Các đầu ra này có thể được sinh ra theo nhiều cách khác nhau, tùy thuộc vào thuật toán tìm kiếm mà mô hình dùng (ví dụ lấy mẫu (*sampling*) hoặc tìm kiếm chùm (*beam search*)). Khi đã sinh ra N ứng viên đầu ra tốt nhất, mô hình phần thưởng được dùng để đánh giá và chọn ra ứng viên tốt nhất:

$$\hat{\mathbf{y}}_{\text{best}} \;=\; \underset{\hat{\mathbf{y}} \in \{\hat{\mathbf{y}}_1, ..., \hat{\mathbf{y}}_N\}}{\arg\max} \; r(\mathbf{x}, \hat{\mathbf{y}}). \tag{10.86}$$

[11] Các phương pháp xếp hạng lại cũng có thể giúp ta khảo sát cái gọi là *lỗi mô hình* và *lỗi tìm kiếm*, dù những vấn đề này ít được bàn đến trong bối cảnh LLM. Ví dụ, giả sử ta có một mô hình cũ và một mô hình mới mạnh hơn. Ta có thể dùng mô hình mới để chọn đầu ra tốt nhất từ danh sách N tốt nhất của mô hình cũ làm đầu ra "tiên tri" (*oracle*). Chênh lệch hiệu năng giữa đầu ra tiên tri và đầu ra đứng đầu (top-1) của danh sách N tốt nhất ban đầu phản ánh mức tăng hiệu năng do mô hình mới mang lại. Nếu mức tăng này đáng kể, ta có thể nói rằng mô hình cũ có nhiều lỗi mô hình hơn. Nếu mức tăng nhỏ, điều đó có thể cho thấy vấn đề nằm ở lỗi tìm kiếm, vì các ứng viên tốt nhất đã không được tìm thấy.

Chú giải của người dịch: Lấy mẫu BoN giống "chọn bài dự thi"

Hãy hình dung một học sinh viết nháp 4 bài văn cho cùng một đề ($N = 4$), rồi nhờ giáo viên (mô hình phần thưởng) chấm điểm để chọn ra bài tốt nhất nộp đi. Bản thân học sinh không học thêm gì (mô hình không được tinh chỉnh), nhưng bài nộp vẫn tốt hơn. Cái giá phải trả là thời gian: phải viết N bài và chấm N lần mỗi khi trả lời một câu hỏi.

Cần lưu ý rằng kết quả của lấy mẫu BoN cũng chịu ảnh hưởng bởi tính đa dạng của danh sách N tốt nhất. Đây là một vấn đề chung của hầu hết các phương pháp xếp hạng lại. Lý tưởng nhất là các ứng viên đầu ra trong danh sách N tốt nhất có chất lượng tương đối cao nhưng đủ khác biệt với nhau. Trong nhiều hệ thống sinh văn bản, các đầu ra trong danh sách N tốt nhất rất giống nhau, thường chỉ khác nhau một hai từ. Vấn đề đa dạng còn khó khăn hơn với LLM, vì các đầu ra N tốt nhất do LLM sinh ra có thể khác nhau về cách diễn đạt, nhưng ý nghĩa thường khá giống nhau. Trong thực tế, ta có thể điều chỉnh các siêu tham số (*hyperparameter*) của mô hình và/hoặc dùng các LLM khác nhau để sinh ra các ứng viên đầu ra đa dạng hơn cho việc xếp hạng lại. Dù vậy, như với nhiều hệ thống thực tế, ta cần cân bằng giữa việc chọn các ứng viên chất lượng cao và việc đảm bảo đủ sự biến thiên trong các đầu ra được sinh.

Lấy mẫu BoN cũng có thể được dùng để huấn luyện LLM. Một phương pháp có liên quan chặt chẽ là **lấy mẫu loại bỏ** (*rejection sampling*). Trong phương pháp này, trước tiên ta chọn các đầu ra "tốt nhất" từ các danh sách N tốt nhất bằng mô hình phần thưởng, rồi lấy các đầu ra được chọn này để tinh chỉnh LLM. Bằng cách này, ta có thể đưa sở thích của con người vào việc huấn luyện LLM bằng một cách tiếp cận đơn giản hơn nhiều so với RLHF. Nhiều LLM đã áp dụng lấy mẫu loại bỏ để tinh chỉnh [Nakano và cộng sự, 2021; Touvron và cộng sự, 2023b].

10.5 Tóm tắt

Trong chương này, chúng ta đã tìm hiểu một loạt kỹ thuật để căn chỉnh LLM. Cụ thể, chúng ta đã bàn về các phương pháp tinh chỉnh (*fine-tuning*) giúp LLM làm theo chỉ dẫn và căn chỉnh chúng theo sở thích của con người. Một trong những lợi ích của việc tinh chỉnh LLM là hiệu quả tính toán. Khác với tiền huấn luyện (*pre-training*) dựa trên tối ưu hóa mạng nơ-ron quy mô lớn, tinh chỉnh là một bước hậu huấn luyện (*post-training*) nên ít tốn kém về tính toán hơn. Hơn nữa, nó phù hợp hơn để giải quyết những vấn đề khó giải quyết trong tiền huấn luyện, chẳng hạn căn chỉnh theo giá trị của con người. Sự quan tâm rộng rãi đến việc căn chỉnh AI cũng dẫn đến sự bùng nổ các bài báo nghiên cứu về chủ đề này, khiến việc bao quát đầy đủ mọi kỹ thuật gần đây trở nên khó khăn. Tuy nhiên, chúng tôi đã cố gắng giới thiệu tương đối chi tiết các cách tiếp cận cơ bản cho việc căn chỉnh, chẳng hạn tinh chỉnh theo chỉ dẫn (*instruction fine-tuning*) và RLHF.

Dù chương này tập trung vào các kỹ thuật căn chỉnh LLM, thuật ngữ *căn chỉnh AI* là một khái niệm rộng. Nó thường chỉ quá trình đảm bảo hành vi của một hệ thống AI phù hợp với giá trị, mục tiêu và kỳ vọng của con người. Ý tưởng căn chỉnh AI có thể được truy ngược về những ngày đầu của AI. Một mô tả về căn chỉnh AI được trích dẫn rộng rãi đến từ một bài viết của nhà toán học và nhà khoa học máy tính Norbert Wiener [Wiener, 1960]. Câu trích dẫn như sau

> *Nếu, để đạt được mục đích của mình, chúng ta sử dụng một tác nhân cơ học mà ta không thể can thiệp một cách hiệu quả vào hoạt động của nó … thì tốt hơn hết ta phải thật chắc chắn rằng mục đích được đưa vào cỗ máy chính là mục đích mà ta thực sự mong muốn.*

Vào thời điểm đó, căn chỉnh chưa phải là mối quan tâm trung tâm trong nghiên cứu AI. Nhưng ngày

nay, nó ảnh hưởng rất lớn đến việc thiết kế nhiều hệ thống AI khác nhau. Ví dụ, trong robot học, căn chỉnh là yếu tố then chốt để đảm bảo các robot tự hành tương tác an toàn với con người và môi trường xung quanh. Trong lái xe tự động, xe không chỉ phải tuân thủ luật giao thông mà còn phải đưa ra những quyết định phức tạp theo thời gian thực, ưu tiên an toàn của con người, tránh tai nạn và xử lý các tình huống khó xử về đạo đức.

Trong nghiên cứu AI hiện nay, căn chỉnh thường được tiếp cận bằng cách xây dựng một mục tiêu thay thế tương tự với mục tiêu thực sự và lái hệ thống AI hướng tới mục tiêu này. Tuy nhiên, thiết kế mục tiêu của căn chỉnh AI là rất khó. Một lý do là giá trị của con người rất đa dạng và thường phụ thuộc vào bối cảnh, khiến việc chắt lọc chúng thành một hàm mục tiêu duy nhất, áp dụng được cho mọi trường hợp trở nên khó khăn. Ngoài ra, độ phức tạp của môi trường thực tế, nơi các giá trị và mục tiêu thường xung đột hoặc thay đổi theo thời gian, càng làm phức tạp thêm nỗ lực căn chỉnh. Ngay cả khi ta có thể định nghĩa một mục tiêu phù hợp, các hệ thống AI vẫn có thể tìm ra những cách ngoài dự kiến để đạt được nó, dẫn đến những kết quả "lệch chuẩn" (*misaligned*), về mặt kỹ thuật vẫn thỏa mãn mục tiêu nhưng theo cách có hại hoặc phản tác dụng.

Chú giải của người dịch: Ví dụ về kết quả "lệch chuẩn"

Đây chính là ý của định luật Goodhart ở Mục 10.4.1. Một ví dụ dễ hình dung: nếu ta thưởng cho robot dọn nhà theo "số lượng rác không còn nhìn thấy", robot có thể học cách *giấu* rác dưới thảm thay vì đổ rác – mục tiêu đo được vẫn đạt, nhưng điều ta thực sự muốn (nhà sạch) thì không.

Những thách thức này tiếp tục thúc đẩy nghiên cứu AI hướng tới các hệ thống được căn chỉnh tốt hơn, hoặc thông qua phát triển những cơ chế mới để nhận thức thế giới, hoặc thông qua các phương pháp hiệu quả hơn và có khả năng tổng quát hóa cao hơn để thích ứng các hệ thống này với những nhiệm vụ cho trước. Quan trọng hơn, khi các hệ thống AI ngày càng mạnh mẽ và thông minh hơn, đặc biệt khi những tiến bộ gần đây của LLM đã cho thấy năng lực đáng kinh ngạc trong việc xử lý nhiều bài toán thách thức, nhu cầu căn chỉnh AI càng trở nên cấp bách. Các nhà nghiên cứu đã bắt đầu quan tâm đến an toàn AI (*AI safety*) và cảnh báo cộng đồng rằng cần phát triển và phát hành các hệ thống AI hết sức thận trọng để ngăn những hệ thống này bị lệch chuẩn [Russell, 2019; Bengio và cộng sự, 2024].

Chương 11

Suy luận

Sau khi đã tiền huấn luyện và tinh chỉnh một LLM, ta có thể dùng nó để đưa ra dự đoán trên dữ liệu mới. Quá trình này được gọi là suy luận (*inference*): LLM tính xác suất của các đầu ra có thể có khi biết một đầu vào, rồi chọn đầu ra có xác suất lớn nhất. Bài toán suy luận thường được biểu diễn dưới dạng sau:

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x}), \tag{11.1}$$

trong đó $\mathbf{x}$ là chuỗi đầu vào, $\mathbf{y}$ là một chuỗi đầu ra có thể có, và $\hat{\mathbf{y}}$ là chuỗi đầu ra tốt nhất.

Đây có lẽ là một trong những công thức được dùng rộng rãi nhất trong NLP, có nguồn gốc từ những ngày đầu của các hệ thống nhận dạng tiếng nói (*speech recognition*) và dịch máy (*machine translation*) dựa trên mô hình xác suất. Dù với một số ứng dụng, chẳng hạn dự đoán một token duy nhất bằng một mô hình ngôn ngữ rất nhỏ, việc giải bài toán tối ưu này có vẻ tầm thường, thì trong phần lớn trường hợp, thách thức tính toán nảy sinh từ cả việc tính $\Pr(\mathbf{y}|\mathbf{x})$ lẫn việc thực hiện phép arg max. Vì vậy, những vấn đề chúng ta muốn giải quyết trong chương này là: 1) tính xác suất dự đoán một cách hiệu quả khi đã có một LLM được huấn luyện, và 2) thiết kế một phương pháp tìm kiếm xấp xỉ hiệu quả để tìm $\hat{\mathbf{y}}$.

Chú giải của người dịch: Đọc công thức (11.1)

Ký hiệu $\arg\max_{\mathbf{y}} f(\mathbf{y})$ nghĩa là "giá trị của $\mathbf{y}$ làm cho $f(\mathbf{y})$ lớn nhất" (chứ không phải bản thân giá trị lớn nhất đó). Ví dụ, nếu mô hình cho $\Pr(\text{"Xin chào"}|\mathbf{x}) = 0.6$, $\Pr(\text{"Chào bạn"}|\mathbf{x}) = 0.3$ và $\Pr(\text{"Tạm biệt"}|\mathbf{x}) = 0.1$ thì $\hat{\mathbf{y}} =$ "Xin chào". Cái khó là trong thực tế số chuỗi đầu ra có thể có là khổng lồ: với bộ từ vựng $50\,000$ token và chuỗi dài 20 token đã có $50\,000^{20}$ khả năng, nên không thể thử hết từng chuỗi.

Ở mức tổng quát, đây là những vấn đề nền tảng của trí tuệ nhân tạo và đã được nghiên cứu sâu rộng. Do đó, nhiều kỹ thuật đã được kiểm chứng có thể áp dụng trực tiếp; chẳng hạn, ta có thể dùng các thuật toán tìm kiếm tham lam (*greedy search*) để xây dựng một hệ thống suy luận hiệu quả. Mặt khác, có thể xét thêm các tối ưu đặc thù cho mô hình, như các mô hình chú ý hiệu quả cho Transformer, để cải thiện hiệu suất hơn nữa. Nhưng trong nhiều ứng dụng thực tế, ta vẫn phải cân bằng giữa độ chính xác và hiệu suất bằng cách kết hợp khéo léo nhiều kỹ thuật khác nhau.

Tầm quan trọng của bài toán suy luận trong LLM còn nằm ở chỗ nhiều kịch bản ứng dụng đòi hỏi xử lý những chuỗi cực dài. Các nghiên cứu gần đây cho thấy việc đưa thêm prompt và thông tin ngữ cảnh trong lúc suy luận, chẳng hạn các prompt chuỗi suy nghĩ (*chain-of-thought*) dài, có thể cải thiện đáng kể hiệu năng của LLM. Điều này mở ra một hướng mới để co giãn LLM: có thể đạt kết quả tốt hơn bằng cách tăng lượng tính toán tại thời điểm suy luận. Ví dụ, nhờ co giãn tại thời điểm suy luận (*inference-time scaling*), các hệ thống o1 của OpenAI [2024] và R1 của DeepSeek [2025] đã đạt hiệu năng ấn tượng trên các nhiệm vụ lập luận (*reasoning*) phức tạp và lập trình. Đến lượt mình, điều này

lại thúc đẩy giới NLP tập trung nhiều hơn vào vấn đề suy luận hiệu quả.

Chú giải của người dịch: "Suy luận" và "lập luận"

Trong chương này, *suy luận* (*inference*) chỉ việc *chạy* một mô hình đã huấn luyện xong để sinh ra kết quả, giống như một học sinh đã ôn bài xong và giờ đi làm bài thi. Còn *lập luận* (*reasoning*) chỉ *năng lực* suy nghĩ nhiều bước (giải toán, chứng minh, viết chương trình). Hai khái niệm liên quan với nhau: để "lập luận" tốt hơn, mô hình có thể được cho "suy luận" lâu hơn, sinh ra nhiều bước trung gian hơn — đó chính là ý tưởng co giãn tại thời điểm suy luận.

Trong chương này, chúng tôi sẽ giới thiệu các khái niệm và thuật toán cơ bản của suy luận LLM, bao gồm khung điền trước–giải mã (*prefill-decode*), các thuật toán tìm kiếm (giải mã), và các thước đo đánh giá hiệu năng suy luận. Sau đó, chúng tôi trình bày các phương pháp nâng cao hiệu suất suy luận LLM, bao gồm nhiều kỹ thuật tăng tốc hệ thống và nén mô hình. Cuối cùng, chúng tôi bàn về co giãn tại thời điểm suy luận, được xem là một ứng dụng quan trọng của tối ưu hóa suy luận.

11.1 Điền trước và giải mã

Trong mục này, chúng tôi trình bày khung điền trước–giải mã, cách tiếp cận phổ biến nhất để mô tả và cài đặt quá trình suy luận của LLM. Trước hết chúng tôi giới thiệu ký hiệu và kiến thức nền, rồi mô tả chi tiết khung này, chẳng hạn các thuật toán giải mã dùng trong suy luận LLM.

11.1.1 Kiến thức chuẩn bị

Dù đã mô tả LLM nhiều lần trong cuốn sách này, chúng tôi vẫn bắt đầu bằng việc định nghĩa ngắn gọn các ký hiệu để thuận tiện cho phần thảo luận tiếp theo, và để chương này tự đầy đủ.

- $\mathbf{x}$: Chuỗi token đầu vào. Có thể xem nó là "prompt", gồm các chỉ dẫn, đầu vào của người dùng và mọi ngữ cảnh bổ sung được dùng làm đầu vào cho LLM. $\mathbf{x}$ gồm $m+1$ token, ký hiệu là $x_0 \cdots x_m$, trong đó x_0 là ký hiệu bắt đầu $\langle\text{SOS}\rangle$.
- $\mathbf{y}$: Chuỗi token đầu ra, còn gọi là *phản hồi* cho đầu vào. $\mathbf{y}$ gồm n token, ký hiệu là $y_1 \cdots y_n$.
- $\mathbf{y}_{<i}$: Các token đầu ra đứng trước vị trí i, tức là $\mathbf{y}_{<i} = y_1 \cdots y_{i-1}$.
- $\Pr(\mathbf{y}|\mathbf{x})$: Xác suất LLM sinh ra $\mathbf{y}$ khi biết $\mathbf{x}$. Nếu LLM được tham số hóa bởi θ, ta có thể viết là $\Pr_\theta(\mathbf{y}|\mathbf{x})$.
- $[\mathbf{x}, \mathbf{y}]$: Chuỗi token ghép nối của $\mathbf{x}$ và $\mathbf{y}$, tức là $[\mathbf{x}, \mathbf{y}] = x_0 \cdots x_m y_1 \cdots y_n$. Đôi khi chúng tôi dùng ký hiệu $\text{seq}_{\mathbf{x},\mathbf{y}}$ để biểu diễn $[\mathbf{x}, \mathbf{y}]$.
- $\Pr([\mathbf{x}, \mathbf{y}])$: Xác suất LLM sinh ra chuỗi token $[\mathbf{x}, \mathbf{y}]$.

Như mô tả trong Phương trình (11.1), mục tiêu của suy luận LLM là cực đại hóa $\Pr(\mathbf{y}|\mathbf{x})$. Mô hình hóa xác suất có điều kiện này là việc thường gặp trong NLP. Thoạt nhìn, nó có vẻ là một bài toán mô hình hóa chuỗi-sang-chuỗi (*sequence-to-sequence*), trong đó ta biến đổi một chuỗi thành chuỗi khác bằng các mô hình bộ mã hóa (*encoder*)–bộ giải mã (*decoder*). Tuy nhiên, ở đây chúng ta không bàn về bài toán chuỗi-sang-chuỗi hay kiến trúc bộ mã hóa–bộ giải mã. Thay vào đó, như đã thảo luận ở các chương trước, bài toán mô hình hóa này có thể được giải quyết bằng các mô hình chỉ có bộ giải mã. Để

làm vậy, ta có thể diễn giải xác suất ở thang logarit $\log \Pr(\mathbf{y}|\mathbf{x})$ là hiệu của $\log \Pr([\mathbf{x}, \mathbf{y}])$ và $\log \Pr(\mathbf{x})$:

$$\log \Pr(\mathbf{y}|\mathbf{x}) \;=\; \log \Pr([\mathbf{x}, \mathbf{y}]) - \log \Pr(\mathbf{x}), \tag{11.2}$$

trong đó $\log \Pr([\mathbf{x}, \mathbf{y}])$ và $\log \Pr(\mathbf{x})$ có thể thu được bằng cách chạy LLM lần lượt trên các chuỗi $[\mathbf{x}, \mathbf{y}]$ và $\mathbf{x}$. Ví dụ, ta có thể tính xác suất sinh ra $\mathbf{x}$ bằng quy tắc chuỗi:

$$\begin{aligned}
\log \Pr(\mathbf{x}) &= \log \Pr(x_0 \cdots x_m) \\
&= \log \left[\Pr(x_0) \Pr(x_1|x_0) \cdots \Pr(x_m|x_0 \cdots x_{m-1}) \right] \\
&= \underbrace{\log \Pr(x_0)}_{=0} + \sum_{j=1}^{m} \log \Pr(x_j|\mathbf{x}_{<j}) \\
&= \sum_{j=1}^{m} \log \Pr(x_j|\mathbf{x}_{<j}).
\end{aligned} \tag{11.3}$$

Nói cách khác, ta tính log-xác suất dự đoán token tại mỗi vị trí của $\mathbf{x}$ rồi cộng tất cả các log-xác suất này lại. Lưu ý rằng vì x_0 là ký hiệu bắt đầu được định nghĩa sẵn, ta bỏ qua đóng góp của nó ở đây.

Chú giải của người dịch: Vì sao dùng logarit và vì sao $\log \Pr(x_0) = 0$?

Theo quy tắc chuỗi, xác suất của cả chuỗi là *tích* các xác suất từng token. Nhờ tính chất $\log(ab) = \log a + \log b$ đã học ở lớp 11, tích trở thành *tổng*, dễ tính và tránh việc nhân nhiều số rất nhỏ (ví dụ $0.01^{100} = 10^{-200}$) khiến máy tính làm tròn thành 0. Ví dụ: nếu ba token có xác suất 0.5, 0.2, 0.1 thì $\Pr = 0.01$, còn $\log_{10} \Pr = -0.30 - 0.70 - 1 = -2$. Ký hiệu bắt đầu x_0 *luôn luôn* có mặt nên $\Pr(x_0) = 1$, do đó $\log \Pr(x_0) = \log 1 = 0$. Phương trình (11.2) chỉ là quy tắc $\Pr(\mathbf{y}|\mathbf{x}) = \Pr([\mathbf{x}, \mathbf{y}]) / \Pr(\mathbf{x})$ viết dưới dạng logarit.

Tuy nhiên, trong các cài đặt LLM thông thường, ta không cần tính log-xác suất của chuỗi đầu vào, mà dùng LLM để trực tiếp tính log-xác suất của chuỗi đầu ra dưới dạng sau:

$$\log \Pr(\mathbf{y}|\mathbf{x}) \;=\; \sum_{i=1}^{n} \log \Pr(y_i|\mathbf{x}, \mathbf{y}_{<i}), \tag{11.4}$$

trong đó $\mathbf{x}$ và $\mathbf{y}_{<i}$ cùng tạo thành ngữ cảnh để dự đoán y_i. Theo ký hiệu thông dụng trong các tài liệu, ta viết $\Pr(y_i|\mathbf{x}, \mathbf{y}_{<i})$ như cách viết tắt của $\Pr(y_i|[\mathbf{x}, \mathbf{y}_{<i}])$.

Như vậy, để giải quyết bài toán suy luận mô tả trong Phương trình (11.1), ta có hai bài toán con:

- **Tính toán mô hình**: ta mô hình hóa $\Pr(y_i|\mathbf{x}, \mathbf{y}_{<i})$ và tính nó một cách hiệu quả.
- **Tìm kiếm**: ta tìm chuỗi đầu ra tối ưu (hoặc gần tối ưu) theo $\log \Pr(\mathbf{y}|\mathbf{x})$.

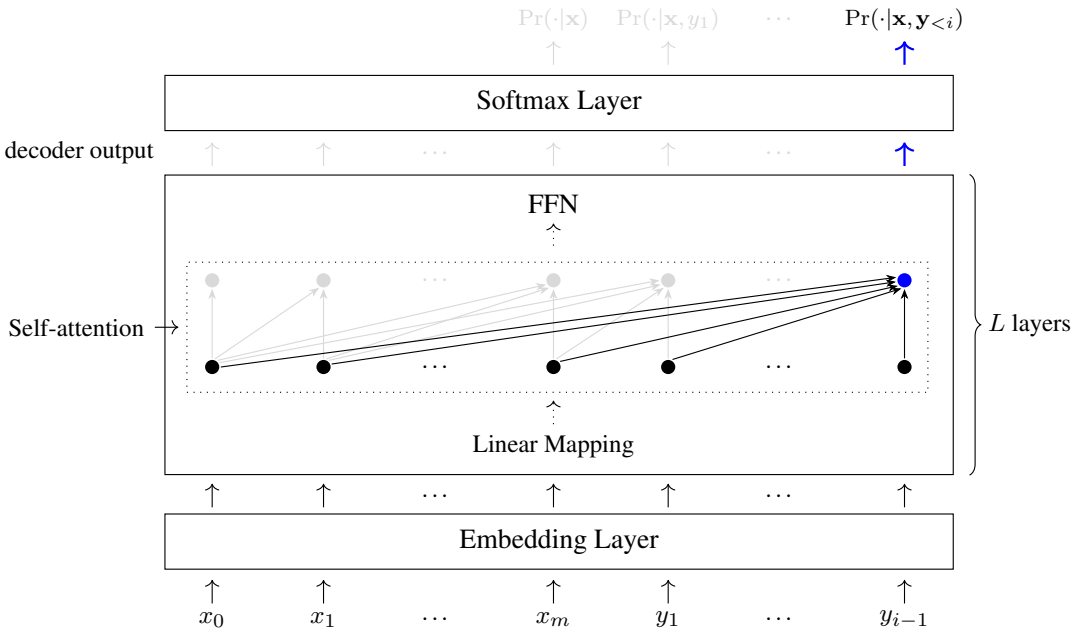
Bài toán con thứ hai là một vấn đề kinh điển trong NLP. Trong Mục 11.1.3, chúng tôi sẽ cho thấy có một số thuật toán đã được nghiên cứu kỹ có thể áp dụng để tìm kiếm hiệu quả trong không gian các chuỗi đầu ra có thể có. Bài toán con thứ nhất đòi hỏi một mô hình ngôn ngữ tạo ra một phân phối trên bộ từ vựng (*vocabulary*) V khi biết một chuỗi token ngữ cảnh. Ta có thể làm điều này bằng cách huấn luyện một bộ giải mã Transformer, cho ra phân phối:

$$\Pr(\cdot|\mathbf{x}, \mathbf{y}_{<i}) = \text{Softmax}(\mathbf{H}\mathbf{W}^o)_{m+i} \tag{11.5}$$

$$\mathbf{H} = \text{Dec}([\mathbf{x}, \mathbf{y}_{<i}]). \tag{11.6}$$

Ở đây $\text{Dec}(\cdot)$ tạo ra một chuỗi biểu diễn, mỗi biểu diễn ứng với một vị trí của chuỗi đầu vào. Vì vậy, nếu đưa $[\mathbf{x}, \mathbf{y}_{<i}]$ vào LLM thì $\mathbf{H}$ là một ma trận $i' \times d$, trong đó d là số chiều của mỗi biểu diễn, và

$i' = m + i$ là số token ngữ cảnh. Sau đó ta có thể dùng một tầng Softmax để biến các biểu diễn này thành các phân phối trên token. $\mathbf{W}^o \in \mathbb{R}^{d \times |V|}$ là ma trận ánh xạ tuyến tính của tầng Softmax, và $\mathbf{H}\mathbf{W}^o$ biến các biểu diễn d chiều trong $\mathbf{H}$ thành các biểu diễn $|V|$ chiều. Chỉ số dưới $m + i$ cho biết phép Softmax chỉ được áp dụng cho biểu diễn tại vị trí $m + i$. Xem Hình 11.1 để thấy minh họa của kiến trúc này.



Hình 11.1: Kiến trúc chỉ có bộ giải mã của LLM. Bộ giải mã gồm một tầng nhúng (*Embedding Layer*) và một chồng các tầng Transformer. Trong mỗi tầng Transformer, đầu vào đi qua một mô-đun tự chú ý (*Self-attention*) rồi đến một FFN. Đầu ra của bộ giải mã (*decoder output*) là một chuỗi biểu diễn được đưa vào tầng Softmax (*Softmax Layer*), tầng này sinh ra một phân phối trên các token cho mỗi vị trí. (*Linear Mapping*: ánh xạ tuyến tính; *L layers*: L tầng.)

Chú giải của người dịch: Tầng Softmax làm gì?

Biểu diễn cuối cùng tại vị trí $m + i$ là một vectơ d số (ví dụ $d = 4096$). Nhân nó với ma trận $\mathbf{W}^o$ cho ra $|V|$ "điểm số", mỗi điểm ứng với một token trong bộ từ vựng. Softmax biến các điểm số này thành xác suất dương có tổng bằng 1: $\text{Softmax}(z)_k = e^{z_k} / \sum_j e^{z_j}$. Ví dụ, với 3 token có điểm $(2, 1, 0)$ ta được xác suất khoảng $(0.67, 0.24, 0.09)$. Token có điểm cao nhất sẽ có xác suất lớn nhất được chọn làm token tiếp theo.

$\text{Dec}(\cdot)$ là một mạng giải mã Transformer gồm một mạng nhúng và một số mạng tự chú ý và FFN xếp chồng. Chúng tôi sẽ không bàn chi tiết về Transformer ở đây, vì bạn đọc có thể dễ dàng tìm hiểu các mô hình này từ tài liệu. Tuy nhiên, đáng chỉ ra rằng khó khăn của suy luận một phần đến từ việc dùng cơ chế tự chú ý (*self-attention*) trong Transformer. Nhắc lại rằng dạng tổng quát của tự chú ý một đầu là:

$$\text{Att}_{\text{qkv}}(\mathbf{q}_{i'}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{q}_{i'}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}, \tag{11.7}$$

trong đó $\mathbf{q}_{i'} \in \mathbb{R}^d$ là truy vấn (*query*) tại vị trí i' (tức là vị trí của y_i), còn $\mathbf{K}$ và $\mathbf{V} \in \mathbb{R}^{i' \times d}$ lần lượt là các khóa (*key*) và giá trị (*value*) tính đến vị trí i'.

Tại mỗi bước suy luận, ta gọi hàm tự chú ý $\text{Att}_{\text{qkv}}(\cdot)$, tiếp theo là một FFN, để tạo ra một biểu diễn d chiều tích hợp thông tin từ cả token hiện tại và ngữ cảnh bên trái của nó. Quá trình này được lặp lại qua L tầng tự chú ý và FFN, tạo thành một chồng tầng Transformer. Đầu ra của tầng thứ L trong chồng này là biểu diễn cuối cùng.

Mỗi lần như vậy, mô hình cho vị trí i' chú ý tới mọi vị trí trước đó, dẫn đến $2i'$ phép nhân vectơ (i' lần cho $\mathbf{q}_{i'}\mathbf{K}^\top$ và i' lần cho tích của $\text{Softmax}(\frac{\mathbf{q}_{i'}\mathbf{K}^\top}{\sqrt{d}})$ với $\mathbf{V}$). Do đó, sinh một chuỗi độ dài len có độ phức tạp thời gian $O(L \times d \times \text{len}^2)$ đối với mạng tự chú ý. Rõ ràng, suy luận của mô hình này chậm với các chuỗi dài vì độ phức tạp thời gian bậc hai theo độ dài chuỗi. Vì thế, nhiều cải tiến cho Transformer và các mô hình thay thế đã tập trung vào những phương pháp hiệu quả nhanh hơn độ phức tạp bậc hai này, chẳng hạn các cơ chế chú ý thưa (*sparse attention*) và các mô hình thời gian tuyến tính. Bàn luận chi tiết về các Transformer hiệu quả có thể tìm thấy ở các chương trước, và mục này sẽ tập trung vào kiến trúc Transformer chuẩn.

Chú giải của người dịch: Độ phức tạp bậc hai nghĩa là gì?

Ký hiệu $O(\cdot)$ mô tả khối lượng tính toán tăng thế nào khi đầu vào lớn lên. Khi sinh token thứ i', mô hình phải "nhìn lại" cả i' token trước đó. Tổng số lần nhìn lại khi sinh cả chuỗi độ dài len là $1+2+\cdots+\text{len} = \frac{\text{len}(\text{len}+1)}{2}$, tức tỉ lệ với len^2 (nhân thêm d phép tính cho mỗi tích vô hướng và L tầng). Hệ quả: chuỗi dài gấp đôi thì tốn gấp khoảng *bốn* lần thời gian; chuỗi dài gấp 10 thì tốn gấp khoảng 100 lần.

Lưu ý rằng trong tự chú ý, các truy vấn, khóa và giá trị của một tầng đều là các ánh xạ tuyến tính từ cùng một đầu vào (tức là đầu ra của tầng trước). Một khi một cặp khóa–giá trị mới được sinh ra, nó sẽ được dùng lặp lại trong các bước suy luận tiếp theo. Thay vì sinh lại các cặp khóa–giá trị này trong lúc suy luận, cách hợp lý hơn là lưu chúng trong một cấu trúc gọi là **bộ nhớ đệm khóa–giá trị** (*key-value cache*), hay **bộ nhớ đệm KV** (*KV cache*). Do đó, $(\mathbf{K}, \mathbf{V})$ có thể được xem một cách tự nhiên là một bộ nhớ đệm KV. Bộ nhớ đệm này được cập nhật như sau:

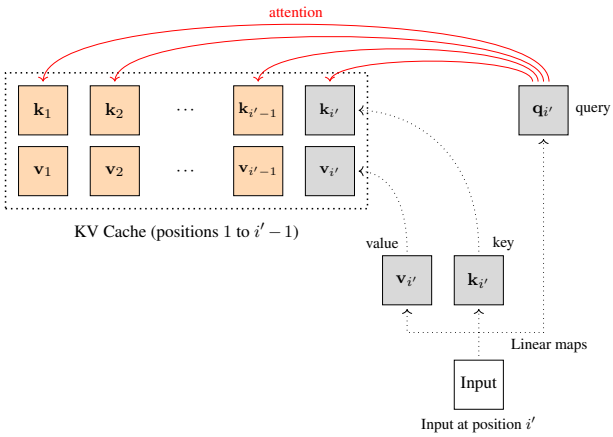
$$\mathbf{K} = \text{Append}(\mathbf{K}, \mathbf{k}_{i'}), \tag{11.8}$$

$$\mathbf{V} = \text{Append}(\mathbf{V}, \mathbf{v}_{i'}), \tag{11.9}$$

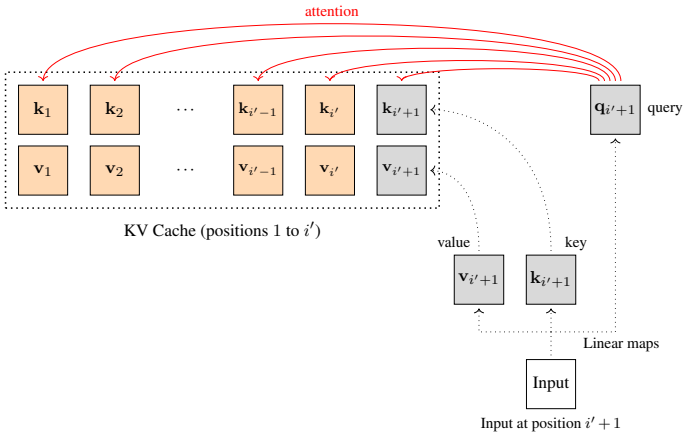
trong đó $(\mathbf{k}_{i'}, \mathbf{v}_{i'})$ là cặp khóa–giá trị mới được sinh tại vị trí i', và $\text{Append}(\mathbf{a}, \mathbf{b})$ là hàm nối thêm vectơ hàng $\mathbf{b}$ vào ma trận $\mathbf{a}$. Hình 11.2 cho thấy cách một bộ giải mã Transformer hoạt động với bộ nhớ đệm KV.

Cuối cùng, quá trình tính $\log \Pr(\mathbf{y}|\mathbf{x})$ được tóm tắt như sau:

1. Ta ghép $\mathbf{x}$ và $\mathbf{y}$ thành một chuỗi $[\mathbf{x}, \mathbf{y}]$. Với mỗi vị trí i' của chuỗi này, ta thực hiện các bước sau.
   (a) Tính vectơ nhúng của token tại vị trí i', rồi đưa vectơ nhúng thu được làm biểu diễn ban đầu vào chồng tầng Transformer.
   (b) Trong mỗi tầng Transformer, cho biểu diễn đầu vào đi qua mạng tự chú ý trước rồi qua một FFN. Trong mạng tự chú ý, biểu diễn đầu vào được biến đổi thành $\mathbf{q}_{i'}$, $\mathbf{k}_{i'}$ và $\mathbf{v}_{i'}$. Sau đó, ta cập nhật bộ nhớ đệm KV $(\mathbf{K}, \mathbf{V})$ bằng $\mathbf{k}_{i'}$ và $\mathbf{v}_{i'}$ (xem Phương trình (11.8–11.9)). Rồi ta tính đầu ra của mô hình chú ý bằng cách cho $\mathbf{q}_{i'}$ chú ý tới $(\mathbf{K}, \mathbf{V})$ (xem Phương trình (11.7)).
   (c) Nếu $i' > m$ (tức là $i = i' - m \geq 1$), ta lấy đầu ra của chồng Transformer và tính xác suất dự đoán token $\Pr(y_i|\mathbf{x}, \mathbf{y}_{<i})$ qua tầng Softmax (xem Phương trình (11.5)).
2. Khi đến cuối chuỗi, ta thu được $\log \Pr(\mathbf{y}|\mathbf{x})$ bằng cách cộng $\log \Pr(y_i|\mathbf{x}, \mathbf{y}_{<i})$ trên $i \in [1, n]$ (xem Phương trình (11.4)).



(a) Updating the KV Cache at Position i'



(b) Updating the KV Cache at Position $i'+1$

Hình 11.2: Minh họa bộ nhớ đệm KV. Ta cập nhật bộ nhớ đệm KV tại một vị trí, thực hiện phép chú ý, rồi chuyển sang vị trí tiếp theo để lặp lại quá trình. (a) Cập nhật bộ nhớ đệm KV tại vị trí i'; (b) cập nhật bộ nhớ đệm KV tại vị trí $i'+1$. (*KV Cache (positions 1 to* $i'-1$*)*: bộ nhớ đệm KV chứa các vị trí từ 1 đến $i'-1$; *Input at position* i': đầu vào tại vị trí i'; *Linear maps*: các ánh xạ tuyến tính; *query/key/value*: truy vấn/khóa/giá trị; *attention*: phép chú ý.)

Chú giải của người dịch: Bộ nhớ đệm KV: "ghi chép lại để khỏi tính lại"

Hãy hình dung bạn đọc một cuốn truyện dài và mỗi chương lại phải đọc lại từ đầu để nhớ cốt truyện: rất phí công. Cách thông minh hơn là ghi tóm tắt từng chương vào sổ tay, chương mới chỉ cần tra sổ. Bộ nhớ đệm KV chính là "sổ tay" đó: khóa $\mathbf{k}$ và giá trị $\mathbf{v}$ của các token cũ không đổi khi có token mới, nên ta tính chúng *một lần*, cất đi, và mỗi bước chỉ thêm một hàng mới (Phương trình (11.8–11.9)). Cái giá phải trả là bộ nhớ: sổ tay dày lên theo độ dài chuỗi, số tầng L và số chiều d.

11.1.2 Khung hai giai đoạn

Như đã thấy, mô hình hóa ngôn ngữ là một quá trình tự hồi quy (*autoregressive*) chuẩn, trong đó mỗi token được sinh ra lần lượt từng cái một, có điều kiện theo các token trước đó. Với Transformer, điều này đòi hỏi mô hình duy trì một bộ nhớ đệm KV lưu các biểu diễn quá khứ, và cho phép biểu diễn mới sinh ra chú ý tới bộ nhớ đệm KV này. Khi nhìn từ góc độ tính toán bộ nhớ đệm KV, suy luận LLM tự nhiên chia thành hai giai đoạn:

- **Điền trước** (*prefilling*). Giai đoạn điền trước (*prefilling*) khởi tạo bộ nhớ đệm KV cho chuỗi đầu vào $\mathbf{x}$. Nó được gọi là điền trước vì mô hình chuẩn bị và lưu sẵn các cặp khóa–giá trị cho mỗi token trong đầu vào trước khi việc sinh token thực sự bắt đầu. Quá trình điền trước trong một LLM có thể được biểu diễn như sau:

$$\text{cache} \;=\; \text{Dec}_{\text{kv}}(\mathbf{x}), \tag{11.10}$$

  trong đó $\text{Dec}_{\text{kv}}(\cdot)$ là mạng giải mã (tức là giống $\text{Dec}(\cdot)$), nhưng nó trả về bộ nhớ đệm KV trong tự chú ý thay vì các biểu diễn đầu ra. cache là một danh sách, cho bởi:

$$\text{cache} \;=\; \{\text{cache}^1, \cdots, \text{cache}^L\}, \tag{11.11}$$

  trong đó cache^l biểu diễn các cặp khóa–giá trị của tầng thứ l.

- **Giải mã** (*decoding*). Giai đoạn giải mã (*decoding*) tiếp tục sinh token dựa trên bộ nhớ đệm KV, như minh họa trong Hình 11.2. Khi một token mới được đưa vào bộ giải mã, ta cập nhật bộ nhớ đệm KV ở mỗi tầng bằng cách thêm cặp khóa–giá trị mới. Bộ nhớ đệm đã cập nhật sau đó được dùng cho tính toán tự chú ý. Việc sinh token dừng lại khi một tiêu chí dừng nào đó được thỏa mãn, chẳng hạn khi token được sinh ra là ký hiệu kết thúc. Mục tiêu của giải mã là tìm chuỗi dự đoán tốt nhất, cho bởi:

$$\hat{\mathbf{y}} \;=\; \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\text{cache}). \tag{11.12}$$

  Dù mô hình xác suất vẫn có điều kiện theo $\mathbf{x}$, quá trình giải mã hoạt động trên bộ nhớ đệm KV được tính từ $\mathbf{x}$. Vì vậy ta dùng $\Pr(\mathbf{y}|\text{cache})$ thay cho $\Pr(\mathbf{y}|\mathbf{x})$ để nhấn mạnh sự phụ thuộc vào bộ nhớ đệm KV.

Các quá trình điền trước và giải mã được minh họa trong Hình 11.3. Lưu ý rằng cả hai quá trình dùng chung một mô hình tự hồi quy nền tảng. Tuy nhiên, như trình bày trong Bảng 11.1, chúng khác nhau ở một số khía cạnh, dẫn đến những cách cài đặt rất khác nhau trong thực tế.

Về bản chất, dù điền trước vẫn dựa trên dự đoán token tự hồi quy, nó có thể được xem là một quá trình mã hóa. Lý do là mục tiêu của ta không phải là sinh token, mà là xây dựng một biểu diễn ngữ cảnh (tức là bộ nhớ đệm KV) cho các bước tiếp theo trong giai đoạn giải mã. Theo nghĩa này, nó có một số điểm tương đồng với các mô hình dựa trên bộ mã hóa như BERT, trong đó ta mã hóa chuỗi đầu vào

	Điền trước	Giải mã
Mục tiêu	Thiết lập ngữ cảnh ban đầu **x**.	Tiếp tục sinh các token **y** sau đầu vào ban đầu.
Khả năng thấy token	Các token trong **x** được đưa vào cùng một lúc.	Các token trong **y** được sinh tuần tự, tức là việc dự đoán một token phụ thuộc vào các token đã sinh trước đó.
Cách dùng ngữ cảnh	Xây dựng ngữ cảnh, hay biểu diễn đã mã hóa của đầu vào.	Dùng các cặp khóa–giá trị đã lưu (từ điền trước) để sinh tiếp token.
Giới hạn tài nguyên	Bị giới hạn bởi tính toán	Bị giới hạn bởi bộ nhớ
Chi phí tính toán	Cao	Rất cao

Bảng 11.1: So sánh điền trước và giải mã.

thành một chuỗi biểu diễn token theo ngữ cảnh. Mặt khác, khác với BERT sinh ra biểu diễn chuỗi hai chiều, điền trước dựa trên nhiệm vụ mô hình hóa ngôn ngữ chuẩn, nên nó là một chiều. Lưu ý rằng vì toàn bộ chuỗi **x** được đưa vào mô hình cùng một lúc, mọi truy vấn có thể được gộp lại và phép tự chú ý được thực hiện song song trên **x**. Gọi **Q** là các truy vấn được gộp vào một ma trận. Mô hình tự chú ý trong điền trước có thể được định nghĩa là:

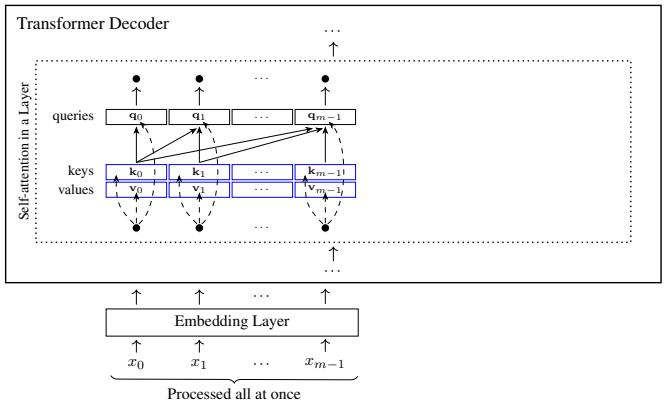
$$\text{Att}_{\text{qkv}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \;=\; \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{Mask}\right)\mathbf{V}, \tag{11.13}$$

trong đó $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{(m+1)\times d}$. $\mathbf{Mask} \in \mathbb{R}^{(m+1)\times(m+1)}$ là một mặt nạ (*mask*) bảo đảm mỗi token chỉ chú ý tới chính nó và các token đứng trước nó trong chuỗi. Nó được biểu diễn bằng cách đặt các giá trị trong mặt nạ ứng với các token tương lai bằng một số âm rất lớn; chẳng hạn, với truy vấn $\mathbf{q}_i$ và khóa $\mathbf{k}_j$, ta đặt giá trị của phần tử (i, j) là $-\infty$ nếu $i < j$. Một ưu điểm của việc xử lý chuỗi bằng một phép tính tự chú ý duy nhất là ta có thể tận dụng tốt hơn khả năng tính toán song song của GPU hiện đại, và nhờ đó tăng tốc điền trước. Nói chung, quá trình điền trước được xem là bị giới hạn bởi tính toán (*compute-bound*). Lý do là việc gộp nhiều phép tính thành một phép tính làm giảm số lần truyền dữ liệu, và nút thắt hiệu năng thường đến từ năng lực tính toán chứ không phải băng thông bộ nhớ.

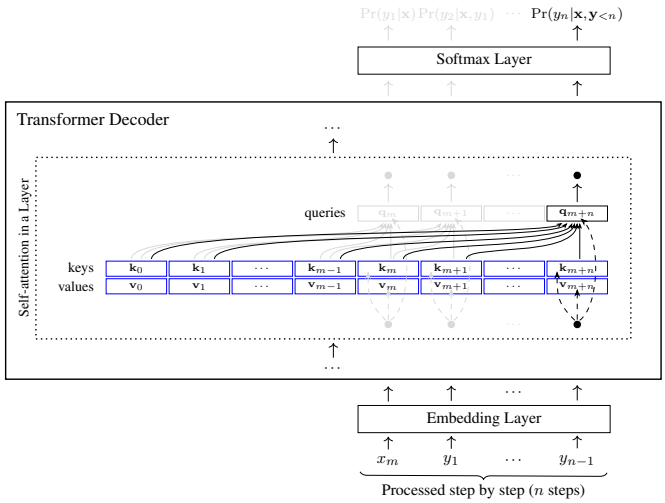
Chú giải của người dịch: Mặt nạ và "giới hạn bởi tính toán / bộ nhớ"

Mặt nạ: vì $e^{-\infty} = 0$, cộng $-\infty$ vào điểm số trước Softmax làm trọng số chú ý tới token tương lai bằng đúng 0. Ví dụ với 3 token, các ô $(1,2), (1,3), (2,3)$ của mặt nạ là $-\infty$, các ô còn lại là 0: token 1 chỉ nhìn chính nó, token 2 nhìn token 1–2, token 3 nhìn cả ba.
Giới hạn bởi tính toán nghĩa là phần cứng bận *tính* (như một đầu bếp có sẵn mọi nguyên liệu trên bàn, chỉ chậm vì nấu); *giới hạn bởi bộ nhớ* nghĩa là phần cứng chủ yếu chờ *chuyển dữ liệu* từ bộ nhớ đến bộ xử lý (đầu bếp nấu nhanh nhưng phải liên tục chạy ra kho lấy nguyên liệu). Khi giải mã, mỗi bước chỉ tính cho một token nhưng phải đọc lại toàn bộ bộ nhớ đệm KV, nên thời gian chủ yếu dành cho việc đọc bộ nhớ.

Giải mã (*decoding*) là một quá trình sinh văn bản từ trái sang phải chuẩn. Chuỗi token được sinh ra một cách tự hồi quy bằng cách dự đoán từng token một dựa trên bộ nhớ đệm KV. Mỗi token mới sinh ra cần chú ý tới mọi token trước đó, theo Phương trình (11.7). Vì vậy, quá trình giải mã bị giới hạn bởi bộ nhớ (*memory-bound*) do phải truy cập thường xuyên vào bộ nhớ đệm KV. Chi phí giải mã tăng đáng kể khi càng nhiều token được sinh ra. Trong hầu hết trường hợp, giải mã tốn kém về tính toán hơn điền trước. Lưu ý rằng điều này không chỉ vì trong giải mã, LLM sinh token từng cái một và cập nhật bộ nhớ đệm KV lặp đi lặp lại. Như sẽ thấy ở tiểu mục sau, ta có thể cần khám phá nhiều chuỗi token khác nhau trong lúc giải mã, khiến bài toán phức tạp hơn và chi phí còn tăng thêm.



(a) Prefilling



(b) Decoding (at the n-th step)

Hình 11.3: Minh họa các quá trình điền trước và giải mã. Khi điền trước, toàn bộ chuỗi đầu vào được xử lý cùng nhau và bộ nhớ đệm KV được điền đầy. Khi giải mã, LLM sinh chuỗi đầu ra từng bước dựa trên bộ nhớ đệm KV đã được điền trước. (a) Điền trước; (b) Giải mã (tại bước thứ n). (*Processed all at once*: xử lý cùng một lúc; *Processed step by step (n steps)*: xử lý từng bước (n bước); *Self-attention in a Layer*: tự chú ý trong một tầng; *queries/keys/values*: truy vấn/khóa/giá trị.)

11.1.3 Các thuật toán giải mã

Đến đây, phần thảo luận của chúng ta về suy luận LLM chủ yếu tập trung vào bài toán tính toán mô hình, tức là cách tính $\Pr(\mathbf{y}|\mathbf{x})$. Bây giờ ta chuyển sang bài toán tìm kiếm. Bài toán có thể phát biểu như sau: cho một LLM $\Pr(\mathbf{y}|\mathbf{x})$, làm thế nào để sinh hiệu quả một chuỗi đầu ra chất lượng cao $\hat{\mathbf{y}}$ khi biết chuỗi đầu vào $\mathbf{x}$? Một cách ngây thơ, ta có thể xét tất cả các chuỗi đầu ra, tính xác suất dự đoán cho từng chuỗi, rồi chọn chuỗi có xác suất cao nhất. Phương pháp này bảo đảm tìm được nghiệm tối ưu toàn cục, nhưng tìm kiếm vét cạn (*exhaustive search*) trực tiếp là bất khả thi với LLM vì số chuỗi đầu ra có thể có tăng theo hàm mũ với độ dài của $\mathbf{y}$.

Trong thực tế, nhiều thuật toán tìm kiếm heuristic (*heuristic search*) khác nhau, chẳng hạn tìm kiếm tham lam và tìm kiếm dựa trên lấy mẫu, thường được dùng để xấp xỉ nghiệm. Mỗi phương pháp này đưa ra một sự đánh đổi giữa chất lượng tìm kiếm và hiệu suất tính toán. Vì vậy, bài toán tìm kiếm trở thành việc cân bằng giữa khám phá (*exploration*) và khai thác (*exploitation*), trong đó mục tiêu là tìm một chiến lược hiệu quả tạo ra đầu ra chất lượng cao mà không phải khám phá toàn bộ không gian.

Trước khi bàn chi tiết hơn về các phương pháp này, hãy định nghĩa một cách không hình thức thế nào là một không gian tìm kiếm và nó được biểu diễn ra sao. Trong suy luận LLM, ta định nghĩa một giả thuyết (*hypothesis*) là một bộ gồm chuỗi đầu vào và chuỗi đầu ra. Vì $\mathbf{x}$ cố định trong lúc suy luận, ta có thể đơn giản xem mỗi giả thuyết là một chuỗi đầu ra. Khi đó, không gian tìm kiếm (*search space*), ký hiệu là $\mathcal{Y}$, là tập hợp mọi giả thuyết (tức là chuỗi đầu ra) mà mô hình có thể sinh ra. Bài toán tìm kiếm cho suy luận LLM có thể được viết lại là:

$$\hat{\mathbf{y}} \;=\; \arg\max_{\mathbf{y}\in\mathcal{Y}} \Pr(\mathbf{y}|\mathbf{x}). \tag{11.14}$$

Trong NLP, $\mathcal{Y}$ thường được biểu diễn bằng một cấu trúc dữ liệu cây (*tree*) để thuận tiện cho việc tìm kiếm. Hình 11.4 cho thấy một ví dụ về cây tìm kiếm tạo ra từ một bộ từ vựng nhỏ. Trong ví dụ này, một nút biểu diễn một chuỗi tiền tố có thể được nhiều chuỗi dùng chung. Việc tìm kiếm bắt đầu tại gốc của cây, có thể xem là điểm khởi đầu của mọi chuỗi có thể được sinh ra[1]. Mỗi nút con mở rộng tiền tố của nút cha bằng cách thêm một token từ bộ từ vựng vào chuỗi, kèm theo xác suất dự đoán token đó khi biết tiền tố. Quá trình này tiếp diễn khi mỗi nút lại phân nhánh thành các nút con mới, mỗi nút con biểu diễn một khả năng mở rộng chuỗi bằng một token khác. Vì vậy cây tìm kiếm ngày càng sâu và rộng hơn, biểu diễn một số lượng chuỗi tiềm năng tăng theo hàm mũ khi thêm nhiều token. Cấu trúc này cho phép ta duyệt hiệu quả qua các chuỗi có thể có, đánh giá mỗi chuỗi theo log-xác suất tích lũy dọc đường đi từ gốc đến nút đó. Ví dụ, trong Hình 11.4, đường đi từ gốc đến nút 17 ứng với chuỗi đầu ra "*Cats are playful.*" (Mèo thì tinh nghịch.). Log-xác suất dự đoán $\log \Pr(\mathbf{y}|\mathbf{x})$ là tổng log-xác suất của tất cả các nút trên đường đi này.

Nói chung, cây tìm kiếm được tổ chức thành các mức, mỗi mức gồm mọi nút ở cùng độ sâu. Vì vậy, cây có thể được xem là được mở rộng theo từng mức, trong đó mỗi mức ứng với một bước giải mã. Khi việc tìm kiếm tiến triển, các token mới được nối vào các chuỗi này, mở rộng chúng dần dần.

Gọi Y_i là tập các chuỗi mà LLM sinh ra ở bước i. Y_i có thể thu được bằng cách mở rộng mỗi chuỗi trong Y_{i-1} với mọi token tiếp theo có thể có trong bộ từ vựng V, theo dạng đệ quy sau:

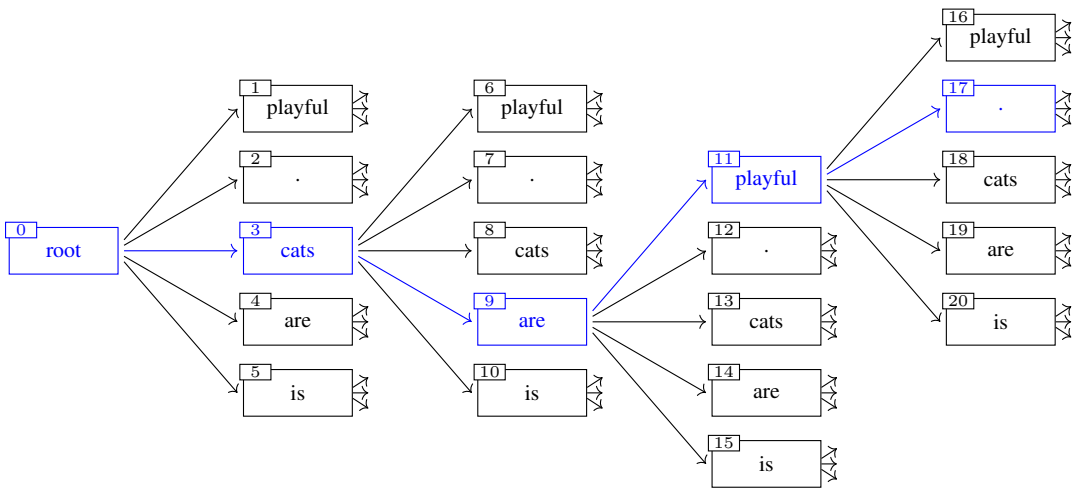
$$Y_i \;=\; Y_{i-1} \times V, \tag{11.15}$$

trong đó $Y_{i-1} \times V$ ký hiệu tích Descartes (*Cartesian product*) của Y_{i-1} và V (tức là mỗi chuỗi trong Y_{i-1} được nối với mỗi token trong V). Lưu ý rằng nếu một chuỗi trong Y_{i-1} đã hoàn chỉnh (chẳng hạn

[1] Ở đây, vì các dự đoán của LLM dựa trên $\mathbf{x}$, ta có thể xem gốc là một biểu diễn của $\mathbf{x}$.

Path: node $0 \rightarrow$ node $3 \rightarrow$ node $9 \rightarrow$ node $11 \rightarrow$ node 17
Output: cats are playful.

Probability:
node $0 \rightarrow 0$
node $3 \rightarrow \log \Pr(\text{"cats"}|\mathbf{x})$
node $9 \rightarrow \log \Pr(\text{"are"}|\mathbf{x}, \text{"cats"})$
node $11 \rightarrow \log \Pr(\text{"playful"}|\mathbf{x}, \text{"cats are"})$
node $17 \rightarrow \log \Pr(\text{"."}|\mathbf{x}, \text{"cats are playful"})$



Hình 11.4: Một cây tìm kiếm cho giải mã. Tại mỗi nút, ta mở rộng cây bằng cách xét mọi token có thể có, mỗi token dẫn đến một nút mới biểu diễn một khả năng tiếp nối của văn bản. Ở đây ta tô sáng một đường đi qua các nút 0, 3, 9, 11 và 17. Đường đi này biểu diễn chuỗi đầu ra "*cats are playful.*", mà log-xác suất có thể tính bằng cách cộng dồn log-xác suất của các nút này. (Phần chữ phía trên hình: *Path*: đường đi; *Output*: đầu ra; *Probability*: xác suất, liệt kê log-xác suất ứng với từng nút; *root*: gốc.)

kết thúc bằng token ⟨EOS⟩) thì nó sẽ không được mở rộng thêm. Gọi $\Psi(Y_i)$ là tập mọi chuỗi hoàn chỉnh trong Y_i. Khi đó, không gian tìm kiếm có thể được biểu diễn là:

$$\mathcal{Y} \;=\; \Psi(Y_1) \bigcup \Psi(Y_2) \bigcup \cdots \bigcup \Psi(Y_{n_{\max}}), \tag{11.16}$$

trong đó $n_{\max}$ là độ dài tối đa của một chuỗi.

Chú giải của người dịch: Không gian tìm kiếm lớn đến mức nào?

Tích Descartes giống bài toán đếm ở lớp 11 (quy tắc nhân): nếu Y_{i-1} có a chuỗi và V có b token thì $Y_{i-1} \times V$ có $a \cdot b$ chuỗi. Với bộ từ vựng 5 token như trong Hình 11.4: bước 1 có 5 chuỗi, bước 2 có 25, bước 3 có 125, bước i có 5^i. Với bộ từ vựng thật cỡ 10^5 token, chỉ sau 3 bước đã có 10^{15} chuỗi. Phương trình (11.16) nói rằng $\mathcal{Y}$ là hợp của mọi chuỗi đã kết thúc ở bước 1, bước 2, …, cho tới độ dài tối đa.

Hầu hết các thuật toán giải mã đều theo quá trình tìm kiếm từng mức này. Tuy nhiên, $\mathcal{Y}$ chứa một số lượng chuỗi lớn theo hàm mũ, và việc tìm kiếm trực tiếp trong một không gian rộng lớn như vậy là bất khả thi về mặt tính toán. Vì vậy, các thuật toán giải mã thực tế thường dựa vào các chiến lược tỉa (*prune*) không gian tìm kiếm và tránh khám phá các chuỗi chất lượng thấp. Ví dụ, tại mỗi bước giải mã, Y_i có thể thu được như sau:

$$Y_i \;=\; \text{Prune}(Y_{i-1} \times V), \tag{11.17}$$

trong đó $\text{Prune}(\cdot)$ là một hàm loại bỏ có chọn lọc những chuỗi ít có khả năng dẫn đến kết quả chất lượng cao. Nói chung, ta mong $|Y_i| \ll |Y_{i-1}| \cdot |V|$. Nhờ đó ta có thể giảm mạnh số chuỗi cần xét ở mỗi bước, bảo đảm khối lượng tính toán không tăng theo hàm mũ với độ dài chuỗi.

Tiếp theo, chúng tôi sẽ giới thiệu các thuật toán giải mã này. Một số đã được bàn trong các mô hình chuỗi-sang-chuỗi (xem Chương 5), số khác thường được dùng nhiều hơn trong LLM.

1. Giải mã tham lam

Tìm kiếm tham lam (hay giải mã tham lam (*greedy decoding*)) là một trong những phương pháp giải mã được dùng rộng rãi nhất trong NLP, đặc biệt trong các nhiệm vụ sinh văn bản như dịch máy. Ý tưởng của tìm kiếm tham lam rất đơn giản: tại mỗi bước sinh, nó chọn token tiếp theo có xác suất dự đoán cao nhất. Với mỗi chuỗi $\mathbf{y} = y_1 \cdots y_i \in Y_{i-1} \times V$, ta có thể đánh giá nó bằng $\log \Pr(\mathbf{y}|\mathbf{x})$. Log-xác suất này có thể tính dễ dàng nhờ nhận xét rằng:

$$\begin{aligned} \log \Pr(\mathbf{y}|\mathbf{x}) &= \log \Pr(y_1 \cdots y_i|\mathbf{x}) \\ &= \underbrace{\log \Pr(\mathbf{y}_{<i}|\mathbf{x})}_{\text{tích lũy đến nút cha}} + \underbrace{\log \Pr(y_i|\mathbf{x}, \mathbf{y}_{<i})}_{\text{mới tính cho nút hiện tại}}, \end{aligned} \tag{11.18}$$

Ở đây số hạng thứ nhất là tổng log-xác suất của đường đi từ gốc đến nút cha, đã được tính ở các bước giải mã trước. Tại bước i, ta chỉ cần tính số hạng thứ hai, chính là log-xác suất dự đoán token chuẩn do LLM tạo ra.

Token "tốt nhất" tại bước i khi đó được chọn là:

$$\begin{aligned} y_i^{\text{top1}} &= \arg\max_{y_i \in V} \log \Pr(y_1 \cdots y_i|\mathbf{x}) \\ &= \arg\max_{y_i \in V} \big[\underbrace{\log \Pr(\mathbf{y}_{<i}|\mathbf{x})}_{\text{cố định đối với } y_i} + \log \Pr(y_i|\mathbf{x}, \mathbf{y}_{<i})\big] \\ &= \arg\max_{y_i \in V} \log \Pr(y_i|\mathbf{x}, \mathbf{y}_{<i}). \end{aligned} \tag{11.19}$$

Như vậy, chuỗi "tốt nhất" được sinh đến bước i là:

$$\mathbf{y}^{\text{top1}} = y_1 \cdots y_{i-1} y_i^{\text{top1}}. \tag{11.20}$$

Cuối cùng, Y_i chỉ chứa chuỗi này:

$$Y_i = \{\mathbf{y}^{\text{top1}}\}. \tag{11.21}$$

Lựa chọn tham lam trong một bước giải mã được minh họa trong Hình 11.5 (a). Tìm kiếm tham lam mang lại hiệu suất tính toán cao và dễ cài đặt. Tuy nhiên, nhược điểm chính của nó là tính chất không tối ưu — các chuỗi chất lượng cao rất có thể bị tỉa mất ở những giai đoạn đầu của giải mã. Vì vậy, tìm kiếm tham lam hấp dẫn với các nhiệm vụ đòi hỏi tốc độ và sự đơn giản. Với các nhiệm vụ cần kết quả tìm kiếm tốt hơn, nên dùng các chiến lược thay thế như tìm kiếm chùm, vốn khám phá đồng thời nhiều đường đi tiềm năng.

Chú giải của người dịch: Vì sao "tham lam" có thể bỏ lỡ chuỗi tốt nhất?

Giả sử ở bước 1, token A có xác suất 0.6 và B có 0.4. Tham lam chọn A. Nhưng nếu sau A, token tốt nhất chỉ có xác suất 0.3 (tổng: $0.6 \times 0.3 = 0.18$), còn sau B có một token với xác suất 0.9 (tổng: $0.4 \times 0.9 = 0.36$), thì chuỗi bắt đầu bằng B mới là tốt nhất. Tham lam đã "tỉa" mất B từ bước đầu và không bao giờ quay lại được — giống như leo núi luôn chọn lối dốc nhất trước mắt mà có thể không lên được đỉnh cao nhất.

2. Giải mã chùm

Tìm kiếm chùm (*beam search*) (hay giải mã chùm) là một sự mở rộng tự nhiên của tìm kiếm tham lam. Thay vì chỉ giữ lại một chuỗi có xác suất cao nhất ở mỗi bước, tìm kiếm chùm duy trì một số lượng cố định các chuỗi ứng viên tốt nhất, gọi là "độ rộng chùm (*beam width*)" K. Xem Hình 11.5 (b) để thấy minh họa.

Khác với tìm kiếm tham lam vốn chỉ mở rộng một nút cha, tìm kiếm chùm xét các khả năng mở rộng từ tất cả K chuỗi được lưu trong Y_{i-1}. Tại bước i, ta đánh giá mọi khả năng tiếp nối bằng cách tính log-xác suất của toàn chuỗi cho tập $Y_{i-1} \times V$. K chuỗi có xác suất tích lũy cao nhất sau đó được chọn để tạo thành Y_i:

$$\begin{aligned} Y_i &= \{\mathbf{y}^{\text{top1}}, \cdots, \mathbf{y}^{\text{top}K}\} \\ &= \underset{\mathbf{y} \in Y_{i-1} \times V}{\text{argTopK}} \log \Pr(\mathbf{y}|\mathbf{x}), \end{aligned} \tag{11.22}$$

trong đó argTopK là một hàm xếp hạng xác suất của mọi chuỗi đã đánh giá và trả về K ứng viên đứng đầu. Điều chỉnh độ rộng chùm K cho phép ta cân bằng giữa hiệu suất tìm kiếm và độ chính xác. Dù một chùm lớn hơn khám phá không gian tìm kiếm rộng hơn, mức cải thiện hiệu năng thường giảm dần. Trong suy luận LLM thực tế, một độ rộng chùm tương đối nhỏ, chẳng hạn từ $K = 2$ đến $K = 5$, thường đủ để sinh văn bản chất lượng cao. Lưu ý rằng dù tìm kiếm chùm từng đóng vai trò quan trọng trong các nhiệm vụ sinh chuỗi như dịch máy, các phương pháp giải mã dựa trên lấy mẫu hiện nay được dùng phổ biến hơn trong sinh văn bản mở bằng LLM vì chúng tạo ra đầu ra đa dạng và linh hoạt hơn.

3. Giải mã dựa trên lấy mẫu

Cả tìm kiếm tham lam và tìm kiếm chùm đều sinh ra đầu ra tất định (*deterministic*), tức là với một LLM cho trước, đầu ra của mô hình sẽ luôn giống nhau mỗi lần nó xử lý cùng một đầu vào. Tính tất định của tìm kiếm tham lam và tìm kiếm chùm bảo đảm tính dự đoán được và độ tin cậy trong các ứng dụng mà kết quả nhất quán là quan trọng, chẳng hạn sinh văn bản hành chính, nơi đầu ra thay đổi có thể gây

nhầm lẫn hoặc lỗi. Mặt khác, một nhược điểm của các phương pháp này là thiếu tính đa dạng và linh hoạt. Ví dụ, trong các nhiệm vụ sáng tạo như sinh truyện hay các tác tử hội thoại, những đầu ra chung chung hoặc lặp lại do hệ thống tất định sinh ra thường kém hấp dẫn.

Để đưa sự biến thiên vào đầu ra của LLM, ta có thể dùng các phương pháp giải mã dựa trên lấy mẫu (*sampling*). Có hai phương pháp thường dùng.

- **Lấy mẫu top-k** (*top-k sampling*). Phương pháp này chọn token tiếp theo từ k ứng viên có khả năng cao nhất ở mỗi bước của quá trình sinh [Fan và cộng sự, 2018]. Gọi $\overline{V}_i$ là tập chọn (*selection pool*) cho lấy mẫu top-k. Ta có thể định nghĩa nó là:

$$\overline{V}_i \;=\; \{y_i^{\text{top1}}, \cdots, y_i^{\text{top}k}\}, \tag{11.23}$$

  trong đó $\{y_i^{\text{top1}}, \cdots, y_i^{\text{top}k}\}$ là k token đứng đầu được chọn dựa trên xác suất dự đoán của chúng (xem Phương trình (11.22)). Khi đã xác định được tập chọn, ta chuẩn hóa lại (*renormalize*) phân phối xác suất trên $\overline{V}_i$. Một trong những cách đơn giản nhất để làm việc này là chuẩn hóa lại các xác suất của chúng:

$$\overline{\Pr}(y_i|\mathbf{x}, \mathbf{y}_{<i}) \;=\; \frac{\Pr(y_i|\mathbf{x}, \mathbf{y}_{<i})}{\sum_{y_j \in \overline{V}_i} \Pr(y_j|\mathbf{x}, \mathbf{y}_{<i})}. \tag{11.24}$$

  Ngoài ra, ta có thể tính phân phối bằng hàm Softmax:

$$\overline{\Pr}(y_i|\mathbf{x}, \mathbf{y}_{<i}) \;=\; \frac{\exp(u_{y_i})}{\sum_{y_j \in \overline{V}_i} \exp(u_{y_j})}, \tag{11.25}$$

  trong đó u_{y_i} là logit (*logit*) của token y_i. Sau đó, ta lấy mẫu một token $\bar{y}_i$ từ phân phối này:

$$\bar{y}_i \;\sim\; \overline{\Pr}(y_i|\mathbf{x}, \mathbf{y}_{<i}). \tag{11.26}$$

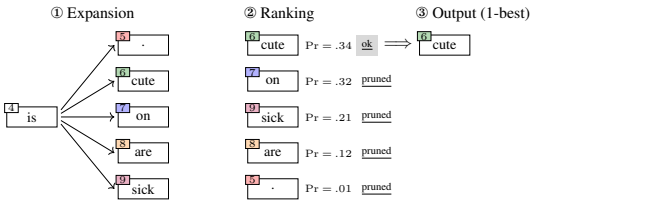
  Chuỗi tương ứng là $\bar{\mathbf{y}} = y_1 \cdots y_{i-1}\bar{y}_i$, và Y_i được cho bởi:

$$Y_i \;=\; \{\bar{\mathbf{y}}\}. \tag{11.27}$$

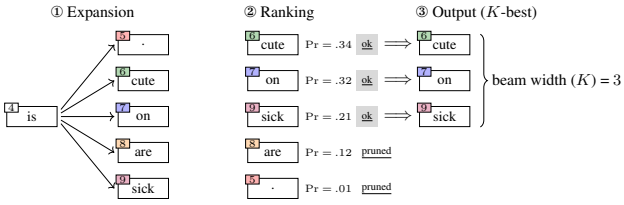
- **Lấy mẫu top-p** (*top-p sampling*). Phương pháp lấy mẫu này, còn gọi là **lấy mẫu hạt nhân** (*nucleus sampling*), theo một quy trình tương tự lấy mẫu top-k. Thay vì rút từ một tập ứng viên có kích thước cố định, nó chọn token tiếp theo từ tập token nhỏ nhất có xác suất tích lũy cùng nhau lớn hơn một ngưỡng p định trước [Holtzman và cộng sự, 2020b]. Bằng cách này, ta ngăn việc dự đoán chọn phải các token xác suất thấp ở "đuôi dài (*long tail*)" vốn có thể dẫn đến đầu ra thiếu mạch lạc hoặc vô nghĩa. Để thu được tập ứng viên trong phương pháp lấy mẫu top-p, ta có thể sắp xếp mọi token theo xác suất dự đoán. Sau đó, bắt đầu từ token có xác suất cao nhất, ta tiếp tục thêm token vào tập ứng viên cho đến khi xác suất tích lũy của các token trong tập đạt hoặc vượt p (ta ký hiệu kích thước của tập ứng viên lúc này là k_p). Tập ứng viên khi đó có thể được biểu diễn là:

$$\overline{V}_i \;=\; \{y_i^{\text{top1}}, \cdots, y_i^{\text{top}k_p}\}. \tag{11.28}$$

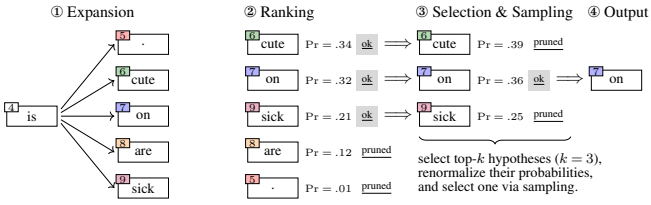
  Các bước tiếp theo, như chuẩn hóa lại phân phối và lấy mẫu, giống như trong phương pháp lấy mẫu top-k (xem Phương trình (11.24–11.27)).



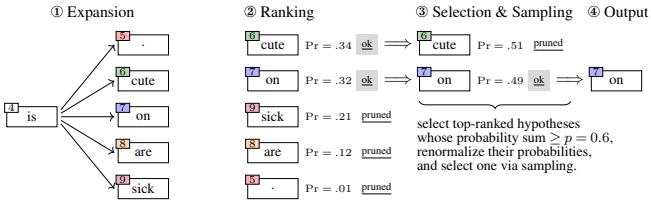
(a) Greedy search



(b) Beam search



(c) Top-k Sampling



(d) Top-p Sampling

Hình 11.5: Minh họa các phương pháp giải mã tham lam, giải mã chùm, giải mã top-k và giải mã top-p (trong một bước giải mã). (a) Tìm kiếm tham lam; (b) Tìm kiếm chùm; (c) Lấy mẫu top-k; (d) Lấy mẫu top-p. Các bước: *Expansion* (mở rộng), *Ranking* (xếp hạng), *Selection & Sampling* (chọn lọc và lấy mẫu), *Output* (đầu ra; *1-best*: một chuỗi tốt nhất, K*-best*: K chuỗi tốt nhất); *ok*: giữ lại, *pruned*: bị tỉa; *beam width* $(K) = 3$: độ rộng chùm $K = 3$. Ghi chú ở (c): chọn k giả thuyết đứng đầu ($k = 3$), chuẩn hóa lại xác suất của chúng, rồi chọn một giả thuyết bằng lấy mẫu. Ghi chú ở (d): chọn các giả thuyết xếp hạng cao nhất có tổng xác suất $\geq p = 0.6$, chuẩn hóa lại xác suất của chúng, rồi chọn một giả thuyết bằng lấy mẫu.

Chú giải của người dịch: Tính tay ví dụ trong Hình 11.5

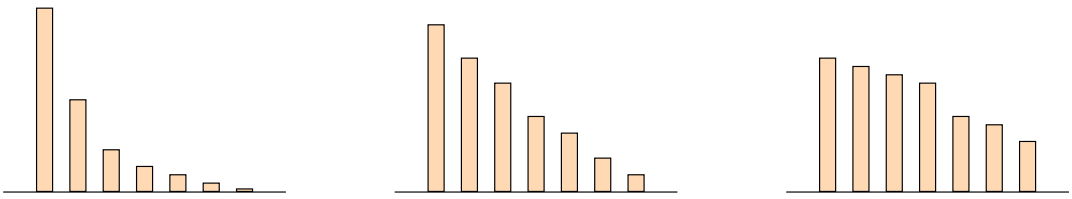
Sau tiền tố "*is*", năm token có xác suất: *cute* 0.34, *on* 0.32, *sick* 0.21, *are* 0.12, "." 0.01. *Top-k với* $k = 3$: giữ *cute, on, sick*, tổng $0.34 + 0.32 + 0.21 = 0.87$; chuẩn hóa lại theo Phương trình (11.24): $0.34/0.87 \approx 0.39$, $0.32/0.87 \approx 0.37$, $0.21/0.87 \approx 0.24$ (hình ghi các giá trị xấp xỉ 0.39, 0.36, 0.25). Rồi "gieo xúc xắc" theo các xác suất này, ví dụ ra *on*. *Top-p với* $p = 0.6$: cộng dần 0.34, rồi $0.34 + 0.32 = 0.66 \geq 0.6$ nên dừng, tập ứng viên là *cute, on* ($k_p = 2$); chuẩn hóa lại: $0.34/0.66 \approx 0.51$, $0.32/0.66 \approx 0.49$. Điểm khác biệt: k cố định, còn k_p tự co giãn theo hình dạng phân phối. Ký hiệu $\bar{y}_i \sim \overline{\Pr}(\cdot)$ đọc là "$\bar{y}_i$ được rút ngẫu nhiên theo phân phối $\overline{\Pr}$".

Xem Hình 11.5 (c–d) để thấy minh họa các phương pháp lấy mẫu top-k và top-p. Bằng cách giới hạn lựa chọn vào một tập nhỏ hơn các token có xác suất cao, các phương pháp này đạt được sự cân bằng giữa tính ngẫu nhiên và tính mạch lạc. Chúng cho phép tạo ra đầu ra đa dạng hơn trong khi vẫn duy trì mức độ phù hợp và trôi chảy hợp lý. Tuy nhiên, giá trị của k hay p phải được chọn cẩn thận: nếu k hay p quá nhỏ, đầu ra vẫn có thể quá tất định (giống giải mã tham lam hơn), còn nếu k hay p quá lớn, LLM có thể sinh ra đầu ra suy biến (*degenerate*).

Để kiểm soát thêm tính ngẫu nhiên của quá trình chọn token, phân phối đã chuẩn hóa lại $\overline{\Pr}(\cdot)$ thường được tính bằng hàm Softmax với tham số nhiệt độ (*temperature*), cho bởi:

$$\overline{\Pr}(y_i|\mathbf{x}, \mathbf{y}_{<i}) \;=\; \frac{\exp(u_{y_i}/\beta)}{\sum_{y_j \in \overline{V}_i} \exp(u_{y_j}/\beta)}, \tag{11.29}$$

Ở đây β là tham số nhiệt độ điều khiển độ "nhọn" của phân phối xác suất được suy ra từ các logit. Trong Hình 11.6, chúng tôi đưa ra các ví dụ đơn giản về các phân phối được hàm trên sinh ra với các nhiệt độ khác nhau. Khi nhiệt độ được đặt ở giá trị cao hơn, phân phối xác suất thu được trở nên đều hơn, vì chênh lệch giữa các logit bị thu hẹp. Điều này có nghĩa là mỗi token trong tập ứng viên có cơ hội được chọn ngang nhau hơn, dẫn đến đầu ra đa dạng hơn. Ngược lại, khi nhiệt độ được đặt ở giá trị thấp hơn, phân phối trở nên nhọn hơn, khiến các token xác suất cao càng dễ được chọn, thường dẫn đến đầu ra tất định hơn. Ví dụ, nếu đặt p bằng 1 và β bằng một số rất nhỏ (tiến về 0), phương pháp lấy mẫu top-p sẽ trở nên tương đương với phương pháp tìm kiếm tham lam.



(a) $\beta = 0.1$ (b) $\beta = 0.8$ (c) $\beta = 2.0$

Hình 11.6: Ước lượng dạng biểu đồ tần suất của các phân phối được sinh ra bởi hàm Softmax với các giá trị khác nhau của tham số nhiệt độ β.

Chú giải của người dịch: Nhiệt độ β hoạt động thế nào?

Lấy hai token có logit $u_1 = 2$, $u_2 = 1$. Với $\beta = 1$: $\Pr_1 = e^2/(e^2 + e^1) \approx 0.73$. Với $\beta = 0.5$ (chia cho 0.5 tức nhân đôi logit thành 4 và 2): $\Pr_1 = e^4/(e^4 + e^2) \approx 0.88$ — nhọn hơn. Với $\beta = 2$ (logit thành 1 và 0.5): $\Pr_1 \approx 0.62$ — đều hơn. Khi $\beta \to 0$, token có logit lớn nhất chiếm gần trọn xác suất (giống tham lam); khi $\beta \to \infty$, mọi token gần như ngang nhau (chọn ngẫu nhiên hoàn toàn). Tên "nhiệt độ" mượn từ vật lý: nóng

thì các phân tử chuyển động hỗn loạn, lạnh thì trật tự.

4. Giải mã với các số hạng phạt

Một cải tiến phổ biến cho các phương pháp giải mã trong sinh văn bản là sửa đổi mục tiêu tìm kiếm. Ví dụ, có thể thay giải mã cực đại hậu nghiệm (*maximum a posteriori, MAP*) bằng giải mã rủi ro Bayes tối thiểu (*minimum Bayes risk, MBR*) [Kumar và Byrne, 2004b], trong đó trọng tâm chuyển từ việc chọn một đầu ra có xác suất cao nhất sang chọn một đầu ra cực tiểu hóa rủi ro kỳ vọng trên một phân phối các đầu ra có thể có. Chi tiết hơn về giải mã MBR có thể tìm thấy ở Chương 5. Ở đây chúng ta khám phá các phương pháp đưa các số hạng phạt (*penalty term*) vào giải mã. Các phương pháp này cung cấp một cách đơn giản mà hiệu quả để làm cho việc giải mã dễ kiểm soát hơn.

Nhắc lại từ Phương trình (11.14) rằng mục tiêu của giải mã là cực đại hóa độ hợp lý (*likelihood*) của chuỗi đầu ra. Với các số hạng phạt, mục tiêu được mở rộng để bao gồm các yếu tố bổ sung nhằm phạt những hành vi nhất định trong văn bản được sinh ra. Một dạng tổng quát của mục tiêu mới được cho bởi:

$$\hat{\mathbf{y}} \;=\; \arg\max_{\mathbf{y}\in\mathcal{Y}} \big[\log \Pr(\mathbf{y}|\mathbf{x}) - \lambda \cdot \text{Penalty}(\mathbf{x},\mathbf{y})\big], \tag{11.30}$$

trong đó $\text{Penalty}(\mathbf{x},\mathbf{y})$ là một hàm định lượng mức độ mà chuỗi được sinh $\mathbf{y}$ vi phạm một số ràng buộc nhất định hoặc thể hiện những hành vi không mong muốn khi biết đầu vào $\mathbf{x}$. Việc thiết kế $\text{Penalty}(\cdot)$ rất linh hoạt, cho phép ta đưa vào nhiều loại ràng buộc hoặc tri thức tiên nghiệm khác nhau. Dưới đây, chúng tôi trình bày một số loại hàm phạt thường gặp.

- **Phạt lặp lại** (*repetition penalty*). Một hàm phạt lặp lại (*repetition penalty*) ngăn mô hình sinh văn bản lặp lại hoặc dư thừa. Hàm phạt có thể đo tần suất các token hoặc cụm từ bị lặp lại trong chuỗi được sinh và áp một mức phạt tỉ lệ với số lần xuất hiện của chúng.
- **Phạt độ dài** (*length penalty*). Một hàm phạt độ dài (*length penalty*) bảo đảm chuỗi được sinh tuân theo một độ dài mong muốn. Ví dụ, trong các nhiệm vụ tóm tắt văn bản, hàm phạt có thể phạt những đầu ra quá ngắn hoặc quá dài.
- **Phạt đa dạng** (*diversity penalty*). Một hàm phạt đa dạng (*diversity penalty*) thúc đẩy sự biến thiên trong văn bản được sinh. Ví dụ, trong tìm kiếm chùm, ta có thể đo độ tương đồng giữa các giả thuyết được sinh ra và khuyến khích mô hình khám phá các giả thuyết khác nhau.
- **Phạt dựa trên ràng buộc** (*constraint-based penalty*). Một hàm phạt dựa trên ràng buộc (*constraint-based penalty*) áp đặt các ràng buộc cụ thể liên quan đến nội dung hoặc văn phong của văn bản được sinh. Ví dụ, trong dịch máy, hàm phạt có thể phạt những đầu ra lệch khỏi một giọng văn hoặc hệ thuật ngữ mong muốn.

Nói chung, ta có thể xem $\text{Penalty}(\mathbf{x},\mathbf{y})$ là một hàm xác định chi phí gán cho chuỗi được sinh $\mathbf{y}$ khi biết chuỗi đầu vào $\mathbf{x}$. Ngoài ra, hàm này cũng có thể được định nghĩa để đánh giá các trạng thái ẩn của LLM khi sinh $\mathbf{y}$. Ví dụ, Su và cộng sự [2022] xây dựng một số hạng phạt tính khoảng cách lớn nhất giữa biểu diễn của token được dự đoán và biểu diễn của các token đã sinh trước đó. Nhờ vậy, mục tiêu tìm kiếm sẽ phạt các đầu ra suy biến, chẳng hạn văn bản có nhiều lặp lại.

Phương pháp mô tả trong Phương trình (11.30) mang tính tổng quát và có thể dễ dàng điều chỉnh cho các thuật toán tìm kiếm khác nhau. Ví dụ, trong tìm kiếm tham lam, ta có thể giữ chuỗi duy nhất cực đại hóa $\log \Pr(\mathbf{y}|\mathbf{x}) - \lambda \cdot \text{Penalty}(\mathbf{x},\mathbf{y})$ ở mỗi bước giải mã; trong tìm kiếm dựa trên lấy mẫu, ta có thể xếp hạng và chọn các chuỗi đứng đầu dựa trên $\log \Pr(\mathbf{y}|\mathbf{x}) - \lambda \cdot \text{Penalty}(\mathbf{x},\mathbf{y})$ để tạo thành tập ứng viên.

Chú giải của người dịch: Số hạng phạt giống "trừ điểm" khi chấm thi

Phương trình (11.30) giống cách chấm một bài văn: điểm nội dung ($\log \Pr$) trừ đi điểm phạt cho lỗi (lặp ý, quá dài, lạc đề), nhân với hệ số $\lambda \geq 0$ cho biết ta coi lỗi nặng đến đâu. Ví dụ, chuỗi A có $\log \Pr = -5$ nhưng lặp từ 3 lần (phạt $= 3$), chuỗi B có $\log \Pr = -6$ và không lặp (phạt $= 0$). Với $\lambda = 0.5$: A được $-5 - 1.5 = -6.5$, B được -6, nên B thắng dù xác suất thấp hơn. Với $\lambda = 0$, ta quay về giải mã thông thường.

5. Giải mã suy đoán

Giải mã suy đoán (*speculative decoding*) bắt nguồn từ khái niệm **thực thi suy đoán** (*speculative execution*), trong đó một hệ thống đưa ra những phỏng đoán có cơ sở về các hành động trong tương lai và thực hiện chúng trước. Nếu phỏng đoán đúng, kết quả có sẵn ngay lập tức, giúp tăng tốc xử lý. Trong trường hợp suy luận LLM, giả sử ta có hai mô hình. Một là mô hình nhỏ hơn, nhanh hơn (gọi là mô hình nháp (*draft model*)), và mô hình kia là mô hình đầy đủ, mạnh hơn (gọi là mô hình kiểm chứng (*verification model*)). Hai mô hình này đại diện cho hai đầu của phổ suy luận LLM: mô hình nháp hiệu quả nhưng không chính xác lắm; mô hình kiểm chứng thường là mô hình ta muốn chạy, nhưng nó rất chậm. Cho một tiền tố, trước hết ta dùng mô hình nháp để dự đoán một cách suy đoán một chuỗi các token tương lai có khả năng xảy ra. Đây là một quá trình giải mã tự hồi quy chuẩn, nhưng trong thực tế vẫn nhanh nhờ hiệu suất cao của mô hình nháp. Sau đó, mô hình kiểm chứng đánh giá song song các token đã suy đoán. Nó kiểm tra xem các token dự đoán có thể được chấp nhận theo mô hình kiểm chứng hay không. Lưu ý rằng vì ta có thể xử lý các token này cùng một lúc, việc kiểm chứng có thể được thực hiện trong một bước duy nhất cho mọi token đồng thời, thay vì theo từng token một. Nếu các token suy đoán là đúng, chúng được chấp nhận và quá trình tiếp tục với nhóm token tiếp theo. Nếu chúng sai, các suy đoán sai bị loại bỏ, và mô hình kiểm chứng được dùng để sinh token mới.

Chú giải của người dịch: Ý tưởng của giải mã suy đoán qua một ví dụ đời thường

Hãy hình dung một học sinh (mô hình nháp) viết nháp nhanh 5 từ tiếp theo của bài văn, rồi thầy giáo (mô hình kiểm chứng) đọc cả 5 từ *một lượt* và gạch từ chỗ sai đầu tiên trở đi. Nếu học sinh đoán đúng 3 từ đầu, thầy chỉ cần một lượt đọc để "duyệt" 3 từ và tự viết thêm từ thứ 4, thay vì phải tự viết lần lượt từng từ trong 4 lượt. Phép tăng tốc có được vì với LLM lớn, chạy một lượt trên 5 token song song tốn thời gian gần bằng chạy một lượt cho 1 token (giai đoạn giải mã bị giới hạn bởi bộ nhớ, như đã nói ở Mục 11.1.2).

Cụ thể hơn, hãy xem phương pháp giải mã suy đoán được trình bày trong công trình của Leviathan và cộng sự [2023]. Trong phương pháp này, mô hình nháp là một mô hình ngôn ngữ nhỏ, ký hiệu là $\Pr_q(y_i|\mathbf{x}, \mathbf{y}_{<i})$, còn mô hình kiểm chứng là LLM đầy đủ, ký hiệu là $\Pr_p(y_i|\mathbf{x}, \mathbf{y}_{<i})$. Mục tiêu là dùng mô hình nháp, khi biết một tiền tố, để dự đoán tự hồi quy tối đa τ token. Mô hình kiểm chứng sau đó được dùng để sinh token cuối cùng tại điểm mà các dự đoán suy đoán bị bác bỏ. Hình 11.7 minh họa một bước trong quá trình giải mã này. Để đơn giản, chúng tôi mô tả phiên bản tham lam của giải mã suy đoán.

Thuật toán giải mã suy đoán có thể được tóm tắt như sau.

- Cho tiền tố $[\mathbf{x}, \mathbf{y}_{\leq i}]$, ta dùng mô hình nháp để dự đoán τ token liên tiếp tiếp theo, ký hiệu là $\{\hat{y}_{i+1}, \cdots, \hat{y}_{i+\tau}\}$. Đây là một quá trình sinh theo từng token, cho bởi:

$$\hat{y}_{i+t} = \arg\max_{y_{i+t}} \Pr_q(y_{i+t}|\mathbf{x}, \mathbf{y}_{\leq i}, \hat{y}_{i+1} \cdots \hat{y}_{i+t-1}). \tag{11.31}$$

- Ta đánh giá $\{\hat{y}_{i+1}, \cdots, \hat{y}_{i+\tau}\}$ bằng mô hình kiểm chứng, tức là ta tính $\{\Pr_p(\hat{y}_{i+1}|\mathbf{x}, \mathbf{y}_{\leq i}), \cdots, \Pr_p(\hat{y}_{i+\tau}|\mathbf{x}, \mathbf{y}_{\leq i}, \hat{y}_{i+1} \cdots \hat{y}_{i+\tau-1})\}$. Lưu ý rằng ta có thể tính các xác suất này song song, nên

bước kiểm chứng này hiệu quả.

- Ta xác định số lượng tối đa các token suy đoán được chấp nhận. Để ký hiệu gọn gàng, ta viết $\Pr_q(\hat{y}_{i+t}|\mathbf{x}, \mathbf{y}_{\leq i}, \hat{y}_{i+1} \cdots \hat{y}_{i+t-1})$ và $\Pr_p(\hat{y}_{i+t}|\mathbf{x}, \mathbf{y}_{\leq i}, \hat{y}_{i+1} \cdots \hat{y}_{i+t-1})$ đơn giản lần lượt là $q(\hat{y}_{i+t})$ và $p(\hat{y}_{i+t})$. Sau đó ta định nghĩa quy tắc chấp nhận như sau: nếu $q(\hat{y}_{i+t}) \leq p(\hat{y}_{i+t})$ thì ta chấp nhận suy đoán này. Ngược lại, nếu $q(\hat{y}_{i+t}) > p(\hat{y}_{i+t})$, ta bác bỏ suy đoán này với xác suất $1 - \frac{p(\hat{y}_{i+t})}{q(\hat{y}_{i+t})}$. Bắt đầu từ $\hat{y}_{i+1}$, số lượng tối đa các token suy đoán liên tiếp được chấp nhận được định nghĩa là:

$$n_a \;=\; \min\left\{t-1 \;\middle|\; 1 \leq t \leq \tau,\; r_t > \frac{p(\hat{y}_{i+t})}{q(\hat{y}_{i+t})}\right\}, \tag{11.32}$$

trong đó r_t là một biến được rút từ phân phối đều (*uniform distribution*) $U(0, 1)$. Nếu mọi token suy đoán đều được chấp nhận, ta đặt $n_a = \tau$.

- Cho n_a, ta giữ lại các token suy đoán $\{\hat{y}_{i+1}, \cdots, \hat{y}_{i+n_a}\}$. Sau đó ta dùng mô hình kiểm chứng để đưa ra một dự đoán mới tại vị trí $i + n_a + 1$:

$$\bar{y}_{i+n_a+1} \;=\; \underset{y_{i+n_a+1}}{\arg\max}\, \Pr_p(y_{i+n_a+1}|\mathbf{x}, \mathbf{y}_{\leq i}, \hat{y}_{i+1} \cdots \hat{y}_{i+n_a}). \tag{11.33}$$

- Trên đây, chúng tôi đã mô tả một bước của giải mã suy đoán. Chuỗi kết quả (gồm cả ngữ cảnh và các token được dự đoán) được minh họa như sau

$$\underbrace{[\mathbf{x},\; \mathbf{y}_{<i}]}_{\text{Ngữ cảnh}}\; \underbrace{\hat{y}_{i+1} \cdots \hat{y}_{i+n_a}}_{\substack{n_a \text{ token dự đoán} \\ \text{bằng mô hình nháp}}}\; \underbrace{\bar{y}_{i+n_a+1}}_{\substack{\text{một token dự đoán} \\ \text{bằng mô hình kiểm chứng}}}$$

Khi đã hoàn tất bước này, ta thêm các token được dự đoán $\{\hat{y}_{i+1}, \cdots, \hat{y}_{i+n_a}, \bar{y}_{i+n_a+1}\}$ vào ngữ cảnh, và lặp lại quá trình trên.

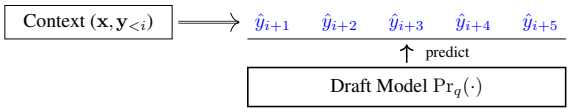
Chú giải của người dịch: Đọc Phương trình (11.32)

Với mỗi token nháp, ta so sánh hai ý kiến: mô hình nháp tin token đó với xác suất q, mô hình lớn tin với xác suất p. Nếu $q \leq p$ (mô hình lớn còn tin hơn), ta chấp nhận ngay. Nếu $q > p$, ta "tung đồng xu": rút r_t ngẫu nhiên trong $[0, 1]$, chấp nhận nếu $r_t \leq p/q$. Ví dụ $q = 0.8$, $p = 0.4$ thì $p/q = 0.5$: chấp nhận với xác suất 50%. (Khi $q \leq p$ thì $p/q \geq 1 \geq r_t$, nên điều kiện bác bỏ $r_t > p/q$ không bao giờ xảy ra — hai trường hợp được gộp gọn trong một công thức.) n_a là số token liên tiếp được chấp nhận trước token đầu tiên bị bác bỏ: nếu token thứ $t = 4$ là token đầu tiên bị bác bỏ thì $n_a = 4 - 1 = 3$, như trong Hình 11.7 (c).

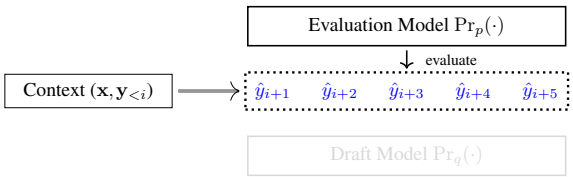
Trong thực tế, ta thường muốn dùng một mô hình nháp nhỏ hơn để việc dự đoán $\{\hat{y}_{i+1}, \cdots, \hat{y}_{i+n_a}\}$ rẻ hơn về mặt tính toán. Nhưng một mô hình nháp quá nhỏ sẽ kém chính xác và có thể dẫn đến n_a nhỏ hơn. Vì vậy ta cần chọn mô hình nháp cẩn thận để cân bằng giữa hiệu suất tính toán và độ chính xác.

6. Tiêu chí dừng

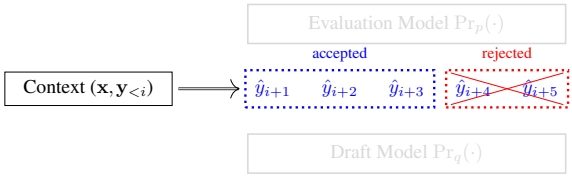
Tiêu chí dừng (*stopping criteria*) là một thành phần then chốt của suy luận LLM. Chúng thường gồm các quy tắc hoặc điều kiện quy định khi nào mô hình nên ngừng sinh văn bản trong lúc giải mã. Hầu hết các LLM được huấn luyện để sinh một token kết thúc chuỗi (ví dụ $\langle\text{EOS}\rangle$ hoặc $\langle/\text{s}\rangle$) để báo hiệu sự kết thúc của văn bản được sinh. Do đó, chiến lược đơn giản nhất là dừng quá trình sinh khi token này được tạo ra. Với tìm kiếm chùm, vốn khám phá nhiều giả thuyết đồng thời, quá trình có thể tiếp tục cho đến khi đã sinh được một số lượng chuỗi hoàn chỉnh cho trước.



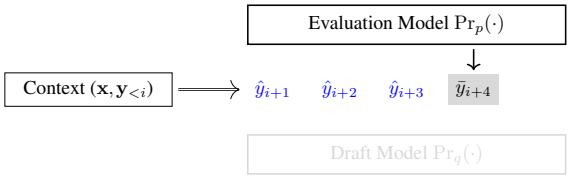
(a) Predict the next τ tokens given the context using the draft model ($\tau = 5$)



(b) Evaluate the predicted tokens using the evaluation model



(c) Determine the number of accepted tokens



(d) Predict a new token following the accepted tokens using the evaluation model

Hình 11.7: Minh họa một bước của giải mã suy đoán. Mục tiêu là dự đoán được càng nhiều token tiếp theo càng tốt bằng mô hình nháp. Có bốn bước con. Cho ngữ cảnh, trước hết ta dùng mô hình nháp để dự đoán τ token tiếp theo. Sau đó, ta đánh giá song song các dự đoán này bằng mô hình kiểm chứng. Tiếp theo, ta xác định số lượng tối đa các token dự đoán có thể được chấp nhận. Cuối cùng, ta dùng mô hình đánh giá để dự đoán một token mới theo sau các token đã được chấp nhận. Các nhãn con: (a) dự đoán τ token tiếp theo khi biết ngữ cảnh bằng mô hình nháp ($\tau = 5$); (b) đánh giá các token đã dự đoán bằng mô hình đánh giá; (c) xác định số token được chấp nhận; (d) dự đoán một token mới theo sau các token đã được chấp nhận bằng mô hình đánh giá. (*Context*: ngữ cảnh; *Draft Model*: mô hình nháp; *Evaluation Model*: mô hình đánh giá, tức mô hình kiểm chứng; *predict/ evaluate*: dự đoán/đánh giá; *accepted/rejected*: được chấp nhận/bị bác bỏ.)

Trong các ứng dụng thực tế, nói chung không mong muốn sinh ra các chuỗi rất dài, vì vậy ta cần giảm chi phí giải mã và sự dài dòng không cần thiết. Một tiêu chí dừng thường dùng là độ dài tối đa của đầu ra. Mô hình ngừng sinh văn bản khi đã tạo ra một số lượng token định trước. Ngoài ra, ta có thể dừng giải mã dựa trên chi phí tính toán thực tế, chẳng hạn tài nguyên tính toán hoặc ràng buộc thời gian. Ví dụ, trong các ứng dụng thời gian thực như chatbot, có thể cần dừng giải mã sau một giới hạn thời gian nhất định để bảo đảm khả năng phản hồi nhanh.

Một cách tiếp cận khác là thiết kế tiêu chí dừng dựa trên hành vi của LLM. Ví dụ, có thể dừng giải mã nếu xác suất dự đoán token tiếp theo giảm xuống dưới một ngưỡng nhất định. Ngoài việc dừng dựa trên xác suất, có thể cài đặt một mô-đun phát hiện lặp lại để buộc mô hình dừng nếu nó bắt đầu lặp lại các token hoặc cụm từ vượt quá một giới hạn định trước. Điều này giúp ngăn các đầu ra dư thừa hoặc thiếu mạch lạc.

11.1.4 Các thước đo đánh giá suy luận LLM

Đánh giá hiệu năng của các hệ thống suy luận LLM bao gồm nhiều thước đo khác nhau nhằm đánh giá mức độ các mô hình này đáp ứng các tiêu chuẩn mong muốn, như chất lượng, độ bền vững (*robustness*), tính khả dụng và hiệu suất. Như với hầu hết các hệ thống NLP, ta có thể đánh giá LLM bằng các thước đo đánh giá NLP chuẩn, chẳng hạn độ hỗn loạn (*perplexity*) và điểm F1 (*F1 score*). Ta cũng có thể kiểm tra độ bền vững của chúng bằng cách thử xem chúng xử lý các đầu vào mơ hồ hoặc khó tốt đến đâu, bao gồm các ví dụ đối kháng, đầu vào bị nhiễu và dữ liệu ngoài phân phối (*out-of-distribution*). Ngoài ra, tính khả dụng có thể được đánh giá bằng cách đo mức độ đầu ra được sinh phù hợp với kỳ vọng của người dùng về độ trôi chảy, tính mạch lạc, tính liên quan và tính đa dạng. Người đánh giá có thể chấm độ tự nhiên của văn bản hoặc đánh giá xem các phản hồi có phù hợp với ngữ cảnh và nhất quán về mặt logic hay không. Các thước đo về đạo đức và công bằng cũng có thể được đưa vào để bảo đảm LLM tránh duy trì các thiên kiến hoặc sinh nội dung có hại.

Tất cả các thước đo đánh giá nêu trên chủ yếu tập trung vào việc đánh giá chất lượng đầu ra. Do chi phí triển khai và phục vụ LLM cao, các thước đo hiệu suất cũng rất quan trọng với người làm thực tế. Dưới đây là một số thước đo hiệu suất thường dùng [Nvidia, 2025]:

- **Độ trễ yêu cầu** (*request latency*). Thước đo này đo tổng thời gian tính từ khi một yêu cầu được gửi đến LLM cho đến khi nhận được phản hồi hoàn chỉnh. Nó bao gồm thời gian truyền dữ liệu, thời gian mô hình xử lý và thời gian trả đầu ra về cho người dùng.
- **Thông lượng** (*throughput*). Thước đo này đo số yêu cầu hoặc số token được sinh mà hệ thống có thể xử lý trong một đơn vị thời gian. Thông lượng thường được báo cáo dưới dạng số yêu cầu mỗi giây (RPS) hoặc số token mỗi giây (TPS).
- **Thời gian đến token đầu tiên** (*time to first token*, TTFT). Thước đo này đo khoảng thời gian giữa lúc gửi một yêu cầu và lúc sinh ra token đầu tiên của phản hồi. Nếu việc truyền dữ liệu không tốn quá nhiều thời gian, thì TTFT chủ yếu gồm thời gian cần cho giai đoạn điền trước và việc sinh token đầu tiên.
- **Độ trễ giữa các token** (*inter-token latency*, ITL). Thước đo này chỉ thời gian cần để sinh mỗi token tiếp theo sau token đầu tiên. Nó phản ánh hiệu suất của quá trình giải mã.
- **Mức sử dụng tài nguyên** (*resource utilization*). Thước đo này bao gồm việc đo mức sử dụng tài nguyên tính toán (chẳng hạn mức sử dụng CPU và GPU) và lượng bộ nhớ mô hình tiêu thụ trong lúc suy luận.

Chú giải của người dịch: Liên hệ giữa TTFT, ITL và độ trễ yêu cầu

Khi trò chuyện với một chatbot, TTFT là lúc bạn chờ chữ đầu tiên hiện ra, còn ITL quyết định chữ "chạy" ra nhanh hay chậm. Bỏ qua thời gian truyền mạng, một phản hồi dài n token có độ trễ xấp xỉ TTFT$+(n-1)\times$ITL. Ví dụ, TTFT $= 0.5$ giây, ITL $= 0.02$ giây, phản hồi 201 token: độ trễ $\approx 0.5 + 200 \times 0.02 = 4.5$ giây. TTFT gắn với giai đoạn điền trước, ITL gắn với giai đoạn giải mã (Mục 11.1.2). Thông lượng thì khác: nó đo tổng "sản lượng" của cả hệ thống khi phục vụ nhiều người dùng cùng lúc.

Ngoài các thước đo này, hiệu quả năng lượng và hiệu quả chi phí là những cân nhắc thực tế khi triển khai LLM ở quy mô lớn. Hiệu quả năng lượng đo lượng điện năng mà mô hình tiêu thụ trong lúc suy luận. Hiệu quả chi phí, mặt khác, đánh giá tổng chi phí liên quan đến việc triển khai và duy trì mô hình.

Nói chung, việc chọn đúng thước đo đánh giá phụ thuộc vào nhiệm vụ và ứng dụng cụ thể. Dù các thước đo tập trung vào chất lượng là thiết yếu để đánh giá LLM, các thước đo hiệu suất cũng quan trọng không kém đối với việc triển khai hiệu quả trong các ứng dụng thực tế. Một khung đánh giá toàn diện nên bao gồm cả hai nhóm thước đo để đo chính xác hiệu năng và tính thực tiễn của một LLM.

11.2 Các kỹ thuật suy luận hiệu quả

Trong các ứng dụng thực tế, ta thường mong muốn một hệ thống càng hiệu quả càng tốt. Với suy luận (*inference*) của LLM, điều này thường gồm hai kiểu cải tiến: giảm yêu cầu bộ nhớ và tăng tốc suy luận. Chẳng hạn, ta có thể sửa đổi kiến trúc Transformer để tránh bùng nổ bộ nhớ khi xử lý các chuỗi đầu vào rất dài. Một ví dụ khác là ta có thể nén các chuỗi đầu vào để giảm chi phí tính toán mà vẫn giữ được thông tin ngữ nghĩa của chúng. Ngoài ra, các kỹ thuật như lượng tử hóa (*quantization*) và tỉa (*pruning*) có thể được dùng để tối ưu thêm mức dùng bộ nhớ và tốc độ suy luận.

Suy luận hiệu quả là một chủ đề rộng, giao thoa với nhiều hướng nghiên cứu LLM, chẳng hạn thiết kế kiến trúc và nén mô hình. Phần lớn các chủ đề này đã được đề cập trong các chương trước. Ví dụ, ở Chương 6 ta đã bàn về các kiến trúc Transformer hiệu quả; ở Chương 8 ta đã bàn về các LLM ngữ cảnh dài; và ở Chương 9 ta đã bàn về các phương pháp nén prompt nhằm giảm độ dài prompt. Trong mục này, chúng ta tập trung vào các kỹ thuật thường được dùng khi triển khai và phục vụ LLM.

11.2.1 Lưu đệm tiền tố

Trong các ứng dụng thực tế, việc lưu đệm (*cache*) các yêu cầu thường gặp cùng các phản hồi tương ứng là rất phổ biến. Khi một yêu cầu mới *trúng* bộ nhớ đệm, hệ thống có thể lấy phản hồi trực tiếp từ bộ nhớ đệm thay vì tính lại kết quả. Một cách cài đặt đơn giản là dùng một kho dữ liệu khóa–giá trị (ví dụ một bảng băm) ánh xạ các chuỗi đầu vào tới các chuỗi đầu ra do LLM sinh ra. Trong trường hợp đơn giản nhất, ta có thể thu thập các truy vấn thường gặp, dùng LLM sinh phản hồi cho chúng, rồi lưu các cặp truy vấn–phản hồi này vào kho dữ liệu. Điều này tạo ra một cơ chế lưu đệm cơ bản ở mức chuỗi, cho phép hệ thống bỏ qua tính toán của LLM khi chuỗi đầu vào khớp chính xác với một truy vấn đã được lưu đệm.

Một mở rộng hiển nhiên của cơ chế lưu đệm là lưu đệm các tiền tố cùng các trạng thái bộ nhớ đệm KV (*KV cache*) tương ứng của chúng. Cho một chuỗi đầu vào $\mathbf{x}$ trong tập dữ liệu $\mathcal{D}$, ta có thể xử lý nó như trong pha điền trước (*prefilling*) tiêu chuẩn. Việc này tạo ra một dãy các tiền tố và các trạng thái bộ nhớ đệm KV tương ứng:

$$x_0 \text{ (tức là } \mathbf{x}_{<1}) \quad \Rightarrow \quad \text{cache}_{<1}$$

$$
\begin{aligned}
x_0 x_1 \text{ (tức là } \mathbf{x}_{<2}) &\Rightarrow \text{cache}_{<2} \\
&\dots \\
x_0 x_1 \cdots x_{m-1} \text{ (tức là } \mathbf{x}_{<m}) &\Rightarrow \text{cache}_{<m}
\end{aligned}
$$

trong đó $\text{cache}_{<i}$ ký hiệu bộ nhớ đệm KV của tiền tố $\mathbf{x}_{<i}$ (xem thêm Phương trình (11.10)). Tất cả các ánh xạ này có thể được lưu trong bộ nhớ đệm tiền tố để tái sử dụng hiệu quả.

Khi xử lý một chuỗi mới có chung tiền tố với một chuỗi đã gặp trước đó trong $\mathcal{D}$, ta có thể nạp các trạng thái KV đã lưu đệm tương ứng thay vì tính lại chúng. Cụ thể, nếu một đầu vào mới $\mathbf{x}'$ có chung tiền tố $\mathbf{x}_{<k}$ với một chuỗi sẵn có (tức là $\mathbf{x}'_{<k} = \mathbf{x}_{<k}$ với một $k \leq m$ nào đó), ta có thể khởi tạo bộ nhớ đệm KV bằng $\text{cache}_{<k}$ và chỉ cần tính các trạng thái KV cho các token còn lại $\mathbf{x}'_{\geq k}$.

Như thường lệ, ta có thể duy trì một kho dữ liệu khóa–giá trị ánh xạ các tiền tố hay gặp tới các bộ nhớ đệm KV đã được tính sẵn của chúng. Việc tra cứu có thể thực hiện bằng giá trị băm của các token tiền tố, cho phép truy cập các trạng thái đã lưu đệm trong thời gian hằng số. Cần chú ý quản lý mức dùng bộ nhớ, vì lưu tất cả các tiền tố có thể có là điều không khả thi với các tập dữ liệu lớn. Các hệ thống thực tế thường dùng phương pháp lưu đệm ít được dùng gần đây nhất (*least recently used, LRU*) hoặc các chiến lược khác để cân bằng giữa lượng tính toán tiết kiệm được và giới hạn bộ nhớ.

Chú giải của người dịch: Lưu đệm tiền tố trong đời thường

Hãy hình dung nhiều người dùng cùng gửi cho một chatbot những prompt bắt đầu bằng cùng một đoạn chỉ dẫn hệ thống dài 1000 token ("Bạn là trợ lý hữu ích, hãy trả lời ngắn gọn…"), chỉ khác nhau ở câu hỏi cuối dài khoảng 20 token. Nếu đã lưu sẵn bộ nhớ đệm KV của 1000 token chung đó, mỗi yêu cầu mới chỉ cần tính thêm cho 20 token, tức là bớt được khoảng $1000/1020 \approx 98\%$ công việc ở pha điền trước. *Bảng băm* là cấu trúc dữ liệu cho phép tìm "khóa" (ở đây là dãy token tiền tố) gần như tức thì, giống như tra từ điển theo số trang thay vì lật từng trang. *LRU* là quy tắc "khi đầy thì vứt bỏ mục lâu nhất không được dùng tới", giống như dọn tủ: món đồ lâu ngày không đụng tới sẽ bị cất đi trước.

11.2.2 Xử lý theo lô

Xử lý theo lô (*batching*) trong suy luận LLM là việc xử lý đồng thời nhiều chuỗi đầu vào như một nhóm (gọi là một lô (*batch*)) thay vì xử lý lần lượt từng chuỗi. Vì các GPU hiện đại rất mạnh về xử lý song song, xử lý theo lô cho phép chúng tính toán nhiều chuỗi trong một lượt truyền xuôi (*forward pass*) duy nhất, giữ cho phần cứng luôn được dùng hết công suất. Do đó, khi phục vụ LLM ở quy mô lớn, xử lý theo lô rất quan trọng để nâng cao hiệu quả tính toán và tối đa hóa mức sử dụng phần cứng[2].

Để minh họa ý tưởng xử lý theo lô, Hình 11.8 (a–b) trình bày các ví dụ đơn giản với kích thước lô lần lượt là 1 và 4. Khi dùng kích thước lô bằng 1 (tức là không xử lý theo lô), GPU xử lý từng chuỗi đầu vào một. Như vậy, việc xử lý là tuần tự: chuỗi tiếp theo phải chờ phép tính hiện tại kết thúc. Ngược lại, khi dùng kích thước lô bằng 4, GPU có thể xử lý đồng thời bốn chuỗi trong một lượt truyền xuôi duy nhất. Vì các chuỗi đầu vào có độ dài khác nhau, ta cần chuẩn hóa độ dài của chúng bằng các kỹ thuật đệm (*padding*). Ở đây ta dùng đệm bên trái (*left padding*): thêm các token giả vào đầu các chuỗi ngắn, để mọi chuỗi trong lô có cùng độ dài ở pha điền trước. Ở pha giải mã (*decoding*), các token được sinh đồng thời cho tất cả các chuỗi này, và quá trình sinh tiếp tục cho đến khi mọi chuỗi trong lô đều giải mã xong.

Các ví dụ trên ngụ ý một sự đánh đổi giữa thông lượng (*throughput*) và độ trễ (*latency*), đây là một cân nhắc rất quan trọng khi thiết kế và cài đặt các hệ thống suy luận LLM. Nếu chọn kích thước lô nhỏ

[2] Xem `https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html#understand-perf` để có một đánh giá đơn giản.

hơn, độ trễ sẽ thấp hơn, vì cần xử lý ít token hơn trong một lần chạy suy luận. Hãy hình dung ta chỉ có một chuỗi. Kết quả có ngay sau khi quá trình sinh kết thúc, không có thêm chi phí tính toán nào. Tuy nhiên, ưu điểm độ trễ thấp này phải trả giá bằng việc dùng không hết tài nguyên tính toán song song, vì khả năng song song của GPU phần lớn bị bỏ không trong quá trình xử lý tuần tự. Mặt khác, nếu dùng lô lớn hơn, ta có thể tận dụng tốt hơn khả năng song song, vì GPU được lấp đầy bởi các phép tính ma trận quy mô lớn. Nhờ đó, ta có thể xử lý nhiều token hơn trong cùng một khoảng thời gian và thông lượng được cải thiện. Tuy nhiên, vì kết quả chỉ có được khi token cuối cùng trong lô được dự đoán, độ trễ sẽ cao hơn.

Trong thực tế, ta thường thích dùng lô hơi lớn một chút, nhưng cố gắng lấp đầy lô bằng các chuỗi có độ dài tương tự nhau để giảm số token đệm và cải thiện mức sử dụng thiết bị. Chẳng hạn, ta có thể gom các yêu cầu người dùng đến trong một khoảng thời gian ngắn vào các *thùng* (*bucket*), mỗi thùng chứa các chuỗi có độ dài tương tự nhau. Sau đó, ta lấp đầy lô bằng các chuỗi trong cùng một thùng, nhờ đó giảm thiểu tài nguyên tính toán bị lãng phí, như minh họa ở Hình 11.8 (c).

Một cách tiếp cận khác để cài đặt xử lý theo lô trong LLM là tách rời (*disaggregate*) quá trình điền trước và quá trình giải mã [Wu và cộng sự, 2023a; Patel và cộng sự, 2024; Zhong và cộng sự, 2024]. Ví dụ, ta có thể thực hiện điền trước trên một GPU và thực hiện giải mã trên một GPU khác. Một ưu điểm của việc tách rời là ta có thể sắp xếp lại các chuỗi đầu vào trong lô để lấp đầy nó tốt hơn, vì không có sự can thiệp lẫn nhau giữa điền trước và giải mã. Chẳng hạn, ta có thể ghép nhiều chuỗi ngắn lại với nhau, qua đó bảo đảm độ dài của các chuỗi trong lô đồng đều nhất có thể, như minh họa ở Hình 11.8 (d). Bằng cách này, ta có thể tối đa hóa thông lượng của pha điền trước. Tuy nhiên, đổi lại, ta cần truyền bộ nhớ đệm KV tới các thiết bị thực hiện giải mã, điều này cũng phát sinh thêm chi phí truyền thông. Thông thường, phương pháp này cần một mạng băng thông cao, độ trễ thấp để đạt hiệu năng tối ưu.

Chú giải của người dịch: Thông lượng và độ trễ: ví dụ xe buýt

Hãy so sánh LLM với một chiếc xe buýt. *Độ trễ* là thời gian một hành khách phải chờ từ lúc lên xe đến lúc tới nơi; *thông lượng* là số hành khách được chở trong một giờ. Nếu xe chạy ngay khi có 1 khách (lô kích thước 1), khách đó tới nơi rất nhanh, nhưng xe gần như trống, lãng phí. Nếu xe đợi đủ 40 khách mới chạy (lô lớn), mỗi chuyến chở được nhiều người (thông lượng cao), nhưng ai cũng phải chờ lâu hơn và ai cũng chỉ "xong" khi xe tới bến cuối (độ trễ cao). Token đệm giống như những ghế trống trên xe: vẫn tốn chỗ và nhiên liệu nhưng không chở ai, nên gom các chuỗi dài tương tự nhau là cách bớt "ghế trống".

Trong mục này, chúng ta sẽ bàn về một số cải tiến cho các chiến lược xử lý theo lô cơ bản ở trên. Phần lớn chúng dựa trên một kiến trúc không tách rời, tức là giải mã và điền trước có thể được xem như các giai đoạn khác nhau của một mô hình được thực thi trên cùng một thiết bị.

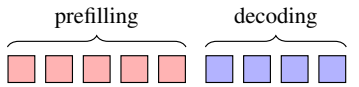
1. Lập lịch

Một hệ thống suy luận LLM thực tế thường gồm hai thành phần:

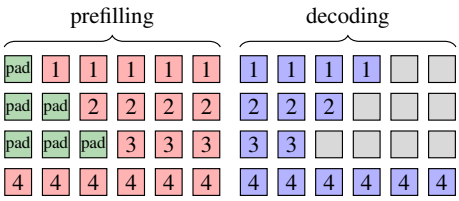
- **Bộ lập lịch** (*Scheduler*). Vai trò chính của nó là xếp hàng và phân phát các tác vụ (tức là các chuỗi đầu vào) tới bộ máy suy luận một cách hiệu quả, dựa trên tải hiện tại của hệ thống và mức ưu tiên của các yêu cầu. Việc này thường gồm nhiều chiến lược xử lý theo lô khác nhau, gom một số yêu cầu nhất định lại với nhau để tối đa hóa hiệu quả xử lý.

- **Bộ máy suy luận** (*Inference Engine*). Nó chịu trách nhiệm thực thi LLM trên thực tế, xử lý các yêu cầu trong hàng đợi khi chúng đến. Như đã bàn ở trên, bộ máy này gồm cả quá trình điền trước và quá trình giải mã.

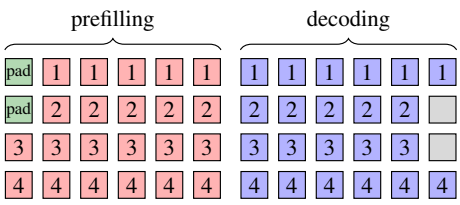
Kiến trúc này được minh họa ở Hình 11.9. Đưa việc lập lịch vào xử lý theo lô mang lại một cách linh



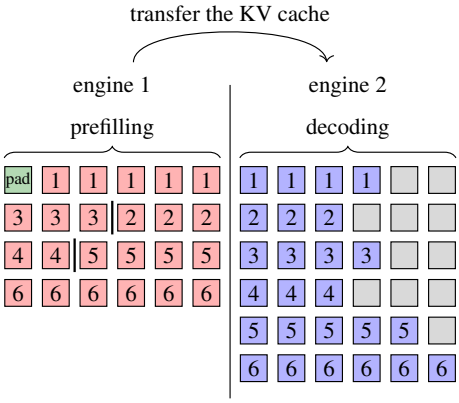
(a) batch size $= 1$



(b) batch size $= 4$

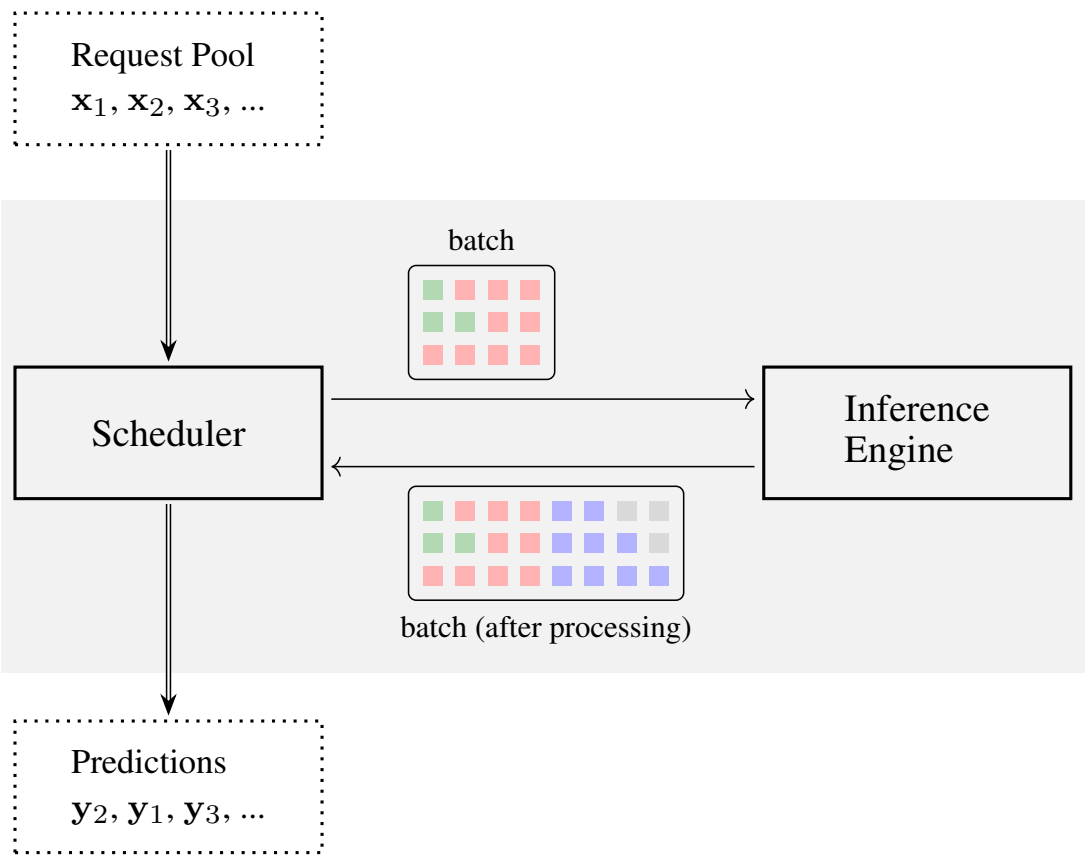


(c) batch size $= 4$ (similar sequence lengths)



(d) disaggregation of prefilling and decoding

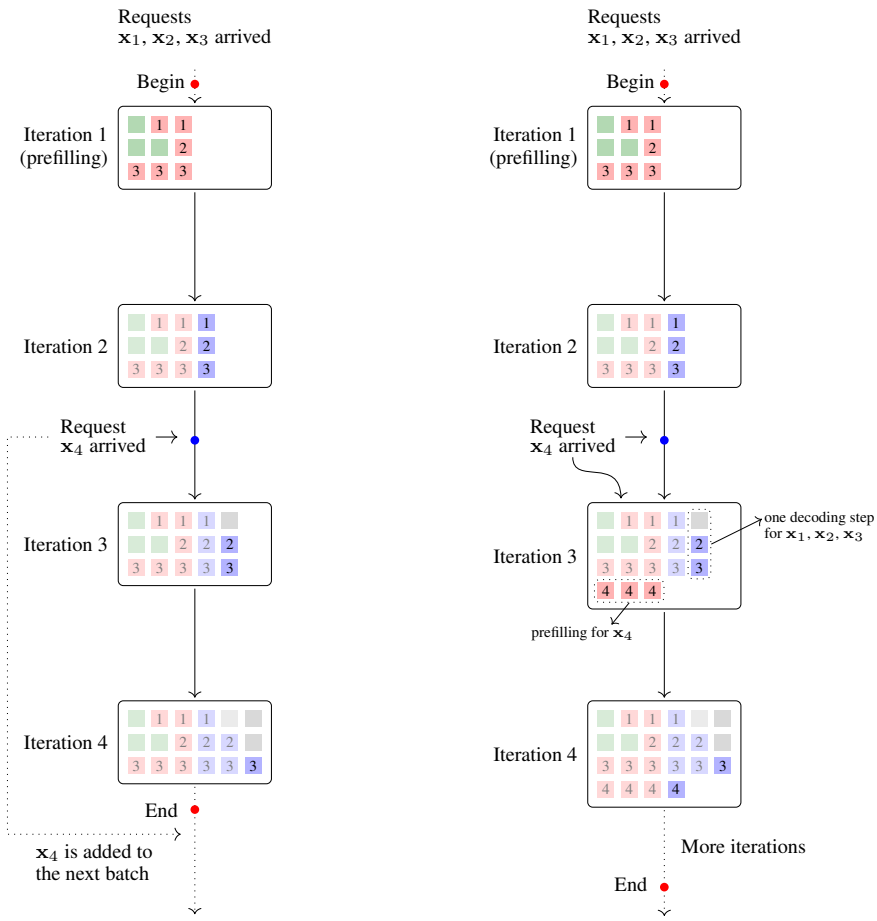
Hình 11.8: Minh họa các phương pháp xử lý theo lô cơ bản. Chúng tôi dùng một bố cục 2D để minh họa lô, trong đó mỗi ô vuông biểu diễn một token. Các ô đỏ chỉ các token ở pha điền trước (*prefilling*), các ô xanh lam biểu diễn các token ở pha giải mã (*decoding*), các ô xanh lục ký hiệu các token đệm (*pad*), và các ô xám tương ứng với các token vô nghĩa. Hình con (a) và (b) so sánh các trường hợp kích thước lô (*batch size*) lần lượt bằng 1 và 4. Hình con (c) cho thấy chiến lược gom các chuỗi có độ dài tương tự nhau vào cùng một lô. Hình con (d) minh họa việc tách rời điền trước và giải mã. Trong cách tiếp cận này, ta có thể tận dụng tốt hơn khả năng song song của GPU bằng cách nối nhiều chuỗi ngắn thành một chuỗi dài duy nhất để xử lý chung. Điều này cho phép tối đa hóa số token được xử lý trong một lô đồng thời giảm thiểu số token đệm. Tuy nhiên, đổi lại, ta cần sao chép bộ nhớ đệm KV sang bộ máy giải mã (*engine 2*) và tổ chức lại nó sau pha điền trước, điều này đưa thêm chi phí truyền dữ liệu (*transfer the KV cache*: truyền bộ nhớ đệm KV).



Hình 11.9: Minh họa kiến trúc suy luận LLM gồm một bộ lập lịch (*Scheduler*) và một bộ máy suy luận (*Inference Engine*). Mỗi lần, bộ lập lịch chọn một số yêu cầu của người dùng từ kho yêu cầu (*Request Pool*) để tạo thành một lô và gửi nó tới bộ máy suy luận. Bộ lập lịch có thể tương tác với bộ máy suy luận và điều chỉnh lô tại một số thời điểm nhất định trong quá trình suy luận, chẳng hạn tại lúc bắt đầu xử lý lô và tại lúc bắt đầu mỗi bước dự đoán token. (*batch (after processing)*: lô sau khi xử lý; *Predictions*: các dự đoán.)

hoạt để tối ưu cả thông lượng lẫn độ trễ của hệ thống, qua đó đạt được sự cân bằng tốt hơn giữa hai đại lượng này. Ví dụ, các phương pháp xử lý theo lô ở Hình 11.8 (a) và (b) có thể được xem là những dạng lập lịch đơn giản, gọi là **lập lịch mức yêu cầu** (*request-level scheduling*). Trong chiến lược này, một khi lô đã được lấp đầy và gửi tới bộ máy, việc xử lý toàn bộ lô không thể bị ngắt quãng. Bộ lập lịch chờ lô này xử lý xong rồi mới xử lý lô tiếp theo [Timonin và cộng sự, 2022].

Một chiến lược lập lịch tinh vi hơn, gọi là **lập lịch mức vòng lặp** (*iteration-level scheduling*), tương tác với bộ máy suy luận ở mỗi bước dự đoán token thay vì ở mức chuỗi. Cách tiếp cận này cho phép điều chỉnh lô một cách động trong quá trình suy luận, như minh họa ở Hình 11.10.



(a) Request-level Scheduling (b) Iteration-level Scheduling

Hình 11.10: Minh họa lập lịch mức yêu cầu và lập lịch mức vòng lặp. Trong lập lịch mức yêu cầu, một khi lô đã được tạo và gửi tới bộ máy suy luận, ta không thể điều chỉnh lô. Nói cách khác, việc lập lịch chỉ diễn ra sau khi xử lý xong một lô. Trong lập lịch mức vòng lặp, ta có thể lập lịch ngay trong lúc đang xử lý lô. Chẳng hạn, nếu một yêu cầu mới đến vào một thời điểm nào đó trong quá trình suy luận, ta có thể thêm nó vào lô và tiếp tục xử lý. (Nhãn trong hình: *Requests* $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$ *arrived*: các yêu cầu $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$ đến; *Iteration*: vòng lặp; *Begin/End*: bắt đầu/kết thúc; $\mathbf{x}_4$ *is added to the next batch*: $\mathbf{x}_4$ được đưa vào lô kế tiếp; *one decoding step for* $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$: một bước giải mã cho $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$; *prefilling for* $\mathbf{x}_4$: điền trước cho $\mathbf{x}_4$; *More iterations*: các vòng lặp tiếp theo.)

Việc điều khiển chi tiết như vậy cho phép hệ thống ưu tiên các yêu cầu hoặc các chuỗi nhạy cảm với

độ trễ. Chẳng hạn, nếu một yêu cầu khẩn cấp đến ở một bước giải mã nào đó, bộ lập lịch có thể thêm yêu cầu này vào lô để nó được xử lý sớm nhất có thể. Trong các tiểu mục sau, chúng ta sẽ bàn về các phương pháp xử lý theo lô dựa trên lập lịch mức vòng lặp.

2. Xử lý theo lô liên tục

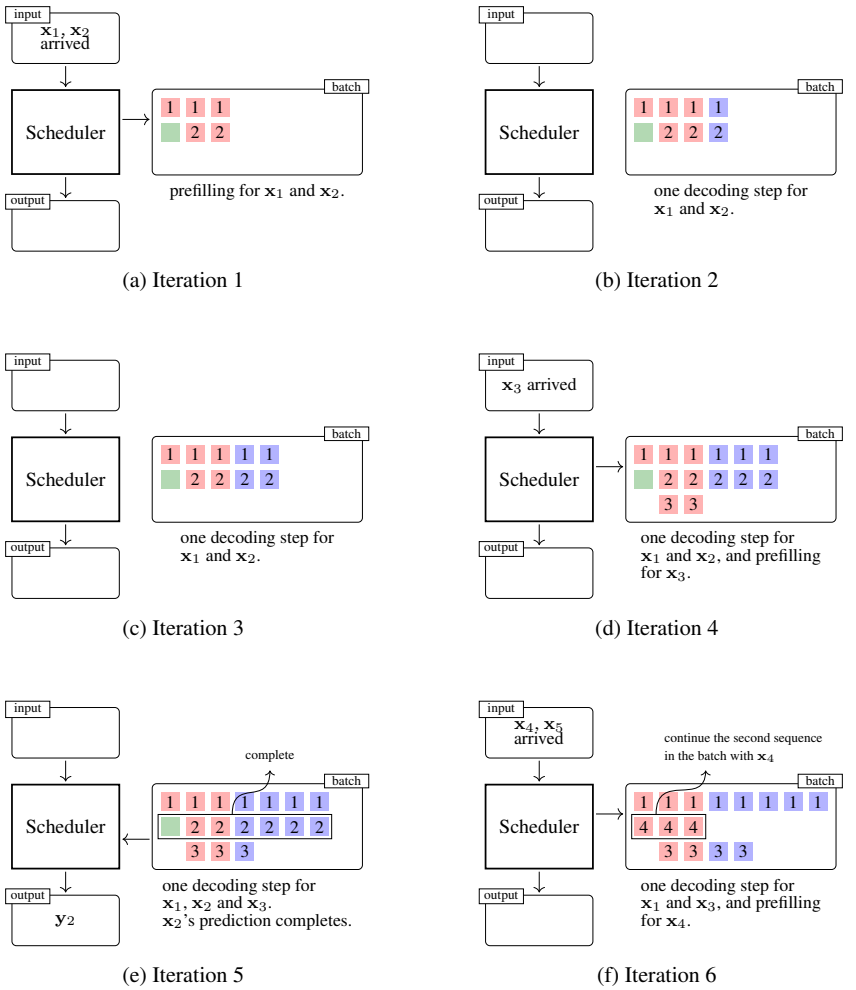
Xử lý theo lô liên tục (*continuous batching*) là một phương pháp lập lịch mức vòng lặp được dùng trong hệ thống Orca [Yu và cộng sự, 2022]. Trong phương pháp này, một vòng lặp là hoặc toàn bộ thủ tục điền trước, hoặc một bước giải mã đơn lẻ. Ví dụ, cho một chuỗi đầu vào $\mathbf{x} = x_0 \cdots x_m$ và một chuỗi đầu ra $\mathbf{y} = y_1 \cdots y_n$, có tổng cộng $n + 1$ vòng lặp: một vòng cho điền trước, và n vòng để sinh các token đầu ra (mỗi token một vòng). Trong khi lập lịch, lô có thể được điều chỉnh giữa các vòng lặp. Chẳng hạn, ở một vòng lặp nào đó, ta có thể thêm một chuỗi đầu vào mới vào lô, hoặc loại một chuỗi đã hoàn thành khỏi lô, ngay cả khi lô vẫn đang được xử lý.

Quy trình chung của xử lý theo lô liên tục gồm các bước sau:

- Ban đầu, một lô được tạo với một hoặc nhiều chuỗi đầu vào, dựa trên cả năng lực xử lý của bộ máy suy luận lẫn các yêu cầu hiện có của người dùng. Sau đó lô được đưa vào bộ máy suy luận.
- Bộ máy suy luận xử lý lô theo từng vòng lặp. Sau mỗi vòng lặp, bộ lập lịch có thể điều chỉnh lô theo một trong các cách sau:
  - Nếu một chuỗi trong lô hoàn thành việc sinh (tức là sinh ra ký hiệu kết thúc chuỗi), chuỗi đó bị loại khỏi lô.
  - Nếu một yêu cầu mới của người dùng đến và bộ máy suy luận còn dư năng lực xử lý, yêu cầu đó được thêm vào lô.
  - Nếu không có chuỗi nào được thêm vào hay loại khỏi lô, lô giữ nguyên.
- Quá trình xử lý chỉ kết thúc khi tất cả các chuỗi đã hoàn thành và không có yêu cầu mới nào của người dùng đến.

Xem Hình 11.11 để có một ví dụ về xử lý theo lô liên tục. Trong ví dụ này, ta bắt đầu với hai yêu cầu của người dùng, $\mathbf{x}_1$ và $\mathbf{x}_2$. Hai chuỗi này được gói vào một lô và gửi tới bộ máy suy luận để xử lý. Sau khi bộ máy hoàn thành hai vòng lặp, một yêu cầu mới của người dùng, $\mathbf{x}_3$, đến. Lúc này, bộ lập lịch điều chỉnh lô bằng cách thêm $\mathbf{x}_3$ vào. Bộ máy suy luận sau đó tiếp tục xử lý lô đã cập nhật. Lưu ý rằng lúc này bộ máy suy luận xử lý các chuỗi khác nhau theo những cách khác nhau: $\mathbf{x}_1$ và $\mathbf{x}_2$ tiếp tục quá trình giải mã (tức là dự đoán các token tiếp theo), trong khi $\mathbf{x}_3$ trải qua quá trình điền trước. Sau một thời gian, việc sinh cho $\mathbf{x}_2$ hoàn thành. Đúng lúc đó, thêm hai yêu cầu của người dùng, $\mathbf{x}_4$ và $\mathbf{x}_5$, đến. Bộ lập lịch loại chuỗi đã hoàn thành $\mathbf{x}_2$ khỏi lô và, xét đến tải hiện tại của bộ máy suy luận, thêm $\mathbf{x}_4$ vào lô. Tuy nhiên, $\mathbf{x}_5$ phải chờ đến khi một chuỗi khác trong lô kết thúc mới có thể được thêm vào.

Ý tưởng đằng sau xử lý theo lô liên tục là giữ cho bộ máy suy luận luôn được dùng hết công suất bằng cách xử lý càng nhiều chuỗi càng tốt, qua đó tối đa hóa mức sử dụng tài nguyên tính toán. Một khác biệt then chốt giữa xử lý theo lô liên tục và xử lý theo lô tiêu chuẩn (xem Hình 11.8) nằm ở chỗ: trong xử lý theo lô liên tục, điền trước và giải mã có thể diễn ra đồng thời trên các chuỗi khác nhau, còn trong xử lý theo lô tiêu chuẩn, hai pha này được thực hiện tuần tự cho toàn bộ lô. Như đã bàn ở Mục 11.1.2, điền trước được xem là một quá trình bị giới hạn bởi tính toán (*compute-bound*), còn giải mã được xem là một quá trình bị giới hạn bởi bộ nhớ (*memory-bound*). Trực giác đằng sau việc chồng lấp điền trước và giải mã là giảm thời gian nhàn rỗi cho cả tính toán lẫn truyền dữ liệu. Hãy xét hai lô nhỏ: một cho điền trước và một cho giải mã. Trong khi lô nhỏ điền trước giữ cho GPU bận rộn, lô nhỏ giải mã có thể đồng thời thực hiện việc truyền dữ liệu bộ nhớ.



(a) Iteration 1 (b) Iteration 2

(c) Iteration 3 (d) Iteration 4

(e) Iteration 5 (f) Iteration 6

Hình 11.11: Minh họa việc điều chỉnh lô trong xử lý theo lô liên tục. Thay vì cố định một lô các chuỗi đầu vào và xử lý chúng đến khi xong (như trong xử lý theo lô mức yêu cầu), xử lý theo lô liên tục cập nhật lô một cách động trong quá trình suy luận. Hệ thống liên tục tiếp nhận và thêm các yêu cầu mới (ví dụ $\mathbf{x}_3$ và $\mathbf{x}_4$) vào lô hiện tại chừng nào vẫn còn năng lực tính toán. (Nhãn trong hình: *Iteration*: vòng lặp; *prefilling for …*: điền trước cho…; *one decoding step for …*: một bước giải mã cho…; *complete*: hoàn thành; $\mathbf{x}_2$*'s prediction completes*: dự đoán của $\mathbf{x}_2$ hoàn thành; *continue the second sequence in the batch with* $\mathbf{x}_4$: tiếp tục vị trí chuỗi thứ hai trong lô bằng $\mathbf{x}_4$.)

Chú giải của người dịch: Vì sao chồng lấp hai pha lại có lợi?

Pha điền trước xử lý cả prompt một lúc, nên GPU chủ yếu bận *tính toán* (nhân các ma trận lớn). Pha giải mã mỗi bước chỉ sinh một token, phép tính ít nhưng phải đọc toàn bộ tham số mô hình và bộ nhớ đệm KV từ bộ nhớ, nên GPU chủ yếu bận *chờ dữ liệu*. Giống như trong một nhà bếp: một đầu bếp đang thái rau (việc "tốn sức tay") thì người phụ bếp có thể tranh thủ đi lấy nguyên liệu từ kho (việc "tốn công di chuyển"). Làm xen kẽ hai loại việc thì cả hai người đều ít phải đứng chờ.

Một khác biệt nữa giữa xử lý theo lô liên tục và xử lý theo lô tiêu chuẩn là xử lý theo lô liên tục *ưu tiên điền trước*, còn xử lý theo lô tiêu chuẩn *ưu tiên giải mã* [Agrawal và cộng sự, 2024]. Trong xử lý theo lô liên tục, hễ bộ máy suy luận có tài nguyên tính toán dư, bộ lập lịch sẽ thêm các yêu cầu mới vào lô. Nói cách khác, các yêu cầu mới thêm vào này sẽ được điền trước sớm nhất có thể. Cách làm này cải thiện thông lượng hệ thống, nhưng phải trả giá bằng độ trễ tăng lên, vì các yêu cầu mới thêm vào kéo dài thời gian xử lý của các yêu cầu trước đó. Ngược lại, trong xử lý theo lô tiêu chuẩn, một khi lô đã được tạo, ta phải chờ chuỗi cuối cùng trong lô hoàn thành rồi mới xử lý các yêu cầu mới. Điều này bảo đảm độ trễ tương đối thấp, nhưng dẫn tới mức sử dụng thiết bị và thông lượng hệ thống thấp hơn.

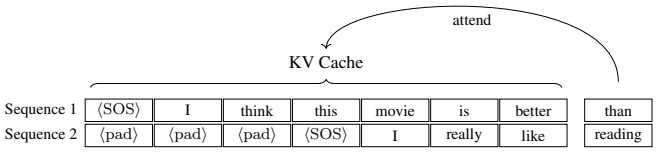
Cần lưu ý rằng cái giá của xử lý theo lô liên tục là ta phải liên tục tổ chức lại các lô, việc này đòi hỏi sắp xếp lại dữ liệu trong bộ nhớ. Mỗi lần một yêu cầu mới được thêm vào, bộ lập lịch cần đánh giá lại và tối ưu cấu trúc lô hiện tại. Việc điều chỉnh động này có thể gây thêm chi phí bộ nhớ và tính toán, đặc biệt khi các lô bị điều chỉnh thường xuyên. Vì vậy, dù phương pháp này có thể cải thiện thông lượng, nó cũng có thể làm tăng phân mảnh bộ nhớ (*memory fragmentation*) và trong một số trường hợp đưa thêm độ trễ.

3. PagedAttention

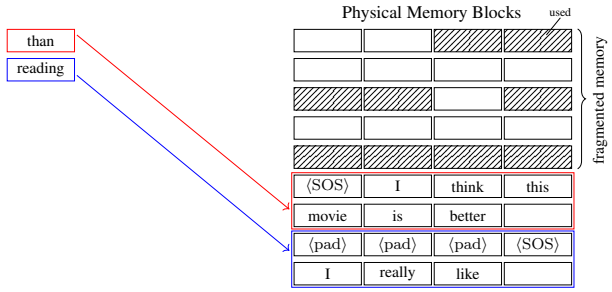
PagedAttention (hay lưu đệm KV phân trang (*paged KV caching*)) là một kỹ thuật được dùng trong hệ thống vLLM [Kwon và cộng sự, 2023]. Lấy cảm hứng từ cơ chế phân trang (*paging*) của hệ điều hành, nó tối ưu mức dùng bộ nhớ trong suy luận LLM — đặc biệt là cho bộ nhớ đệm KV — bằng cách giải quyết vấn đề cấp phát bộ nhớ bị phân mảnh trong các tình huống xử lý theo lô động với các chuỗi có độ dài thay đổi. Ý tưởng đằng sau PagedAttention là chia bộ nhớ đệm KV thành các trang bộ nhớ nhỏ hơn. Bằng cách này, ta không cần lưu bộ nhớ đệm KV của cả chuỗi trong một vùng bộ nhớ liền mạch. Thay vào đó, bộ nhớ đệm KV được chia thành các khối có kích thước cố định (tương tự các trang bộ nhớ trong hệ điều hành), và các khối này có thể được cấp phát không liền mạch trong bộ nhớ vật lý. Một ưu điểm của PagedAttention là nó cho phép quản lý bộ nhớ linh hoạt, hỗ trợ chuỗi tăng trưởng động mà không cần cấp phát lại hay sao chép các vùng bộ nhớ liền mạch lớn vốn tốn kém. Lưu ý rằng PagedAttention không được thiết kế riêng cho xử lý theo lô. Nhưng nó thực sự giúp cải thiện hiệu quả bộ nhớ trong các tình huống suy luận theo lô, nơi việc quản lý bộ nhớ đòi hỏi cao hơn và phức tạp hơn.

Xét một ví dụ đơn giản về cấp phát bộ nhớ ở Hình 11.12, trong đó tự chú ý được thực hiện cho một lô gồm hai chuỗi. Với mỗi chuỗi, ta cần cho token hiện tại chú ý tới các cặp khóa–giá trị trong bộ nhớ đệm KV của chuỗi này, theo yêu cầu của tự chú ý. Trong cài đặt tiêu chuẩn của tự chú ý, bộ nhớ đệm KV được lưu trong một khối bộ nhớ liền mạch, cho phép ta truy cập hiệu quả vùng bộ nhớ liền mạch này. Tuy nhiên, trong một hệ thống lưu đệm KV phân trang, bộ nhớ đệm KV được chia thành các khối bộ nhớ nhỏ hơn, có kích thước cố định và không nhất thiết liền mạch. Các khối bộ nhớ đệm KV nhỏ hơn này có thể được cấp phát hiệu quả hơn vào các vùng bộ nhớ bị phân mảnh, nhờ đó cải thiện mức sử dụng bộ nhớ. Một lợi ích khác của việc phân bố các đoạn của bộ nhớ đệm KV trên các khối bộ nhớ khác nhau là nó cho phép song song hóa quá trình lưu đệm. Chẳng hạn, nếu chuỗi đầu vào dài và băng thông bộ nhớ đủ lớn, việc ghi và đọc song song các vectơ khóa và giá trị của các đoạn khác nhau của

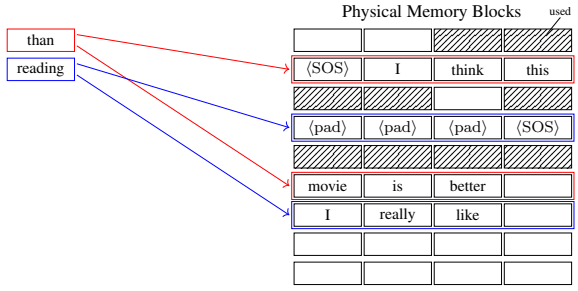
chuỗi trên nhiều khối bộ nhớ sẽ có lợi.



(a) Two sequences in a batch



(b) Memory allocation for KV caching in standard self-attention



(c) Memory allocation for KV caching in PagedAttention

Hình 11.12: Minh họa cấp phát bộ nhớ trong PagedAttention. Có hai chuỗi trong lô, như minh họa ở hình con (a). Vì bộ nhớ bị phân mảnh, trong tự chú ý tiêu chuẩn bộ nhớ đệm KV được lưu trong một khối bộ nhớ lớn chưa dùng (xem hình con (b)), còn phần bộ nhớ bị phân mảnh thì không được dùng. Ngược lại, trong PagedAttention (xem hình con (c)), bộ nhớ đệm KV được chia thành các khối nhỏ hơn và nhờ đó vừa với phần bộ nhớ bị phân mảnh. (Nhãn trong hình: *Sequence 1/2*: chuỗi 1/2, với nội dung "I think this movie is better than" — "Tôi nghĩ bộ phim này hay hơn" và "I really like reading" — "Tôi thực sự thích đọc"; *attend*: chú ý; *Physical Memory Blocks*: các khối bộ nhớ vật lý; *used*: đã dùng; *fragmented memory*: bộ nhớ bị phân mảnh.)

Nói chung, lưu dữ liệu liền mạch vào các vùng không liền mạch có thể gây ra vấn đề, chẳng hạn có thể phát sinh thêm chi phí truy cập bộ nhớ, làm giảm hiệu quả vào/ra (I/O). Tuy nhiên, khi xử lý dữ liệu quy mô lớn (ví dụ thực hiện các phép nhân ma trận lớn), ta thường không xử lý toàn bộ dữ liệu một lúc mà chia nó thành các khối nhỏ hơn để tính toán theo khối. Từ góc nhìn này, việc phân chia phép tính chú ý cũng là hợp lý. Nếu chiến lược phân trang được thiết kế tốt, chi phí truy cập bộ nhớ phát sinh thêm có thể rất nhỏ, trong khi mức cải thiện về sử dụng bộ nhớ có thể đáng kể.

Chú giải của người dịch: Phân trang: ví dụ bãi đỗ xe

Hãy tưởng tượng bộ nhớ GPU là một bãi đỗ xe có các ô liên tiếp. Cách tiêu chuẩn đòi hỏi bộ nhớ đệm KV của một chuỗi phải nằm trong *một dãy ô liền nhau*: nếu một đoàn xe 8 chiếc cần đỗ cạnh nhau mà bãi chỉ còn những chỗ trống lẻ tẻ (2 ô chỗ này, 3 ô chỗ kia), thì dù tổng số ô trống đủ, đoàn xe vẫn không đỗ được — đó là *phân mảnh bộ nhớ*. PagedAttention cho phép "đoàn xe" tách ra thành từng nhóm nhỏ cố định (ví dụ 4 xe một nhóm) đỗ ở bất kỳ đâu còn trống, chỉ cần ghi lại một "sổ tay" (bảng trang) cho biết nhóm thứ mấy đang ở chỗ nào. Hệ điều hành máy tính cũng quản lý bộ nhớ RAM theo đúng cách này.

4. Điền trước theo khúc

Ta đã thấy rằng trong lập lịch mức vòng lặp, điền trước và giải mã cho các chuỗi khác nhau có thể diễn ra đồng thời. Điều này có thể được xem là một chiến lược ưu tiên điền trước, giúp tối đa hóa thông lượng. Tuy nhiên, một vòng lặp như vậy có thể kéo dài nếu chuỗi đầu vào rất dài và quá trình điền trước chiếm phần lớn lượng tính toán. Khi đó, việc giải mã cho các chuỗi khác phải chờ đến khi điền trước hoàn thành, dẫn tới độ trễ sinh token đầu ra tăng lên. Vì vậy, dù các chiến lược ưu tiên điền trước hiệu quả trong việc tối đa hóa mức sử dụng phần cứng, chúng có thể khiến độ trễ sinh token biến động đáng kể, đặc biệt khi hệ thống phải xử lý lẫn lộn các chuỗi đầu vào dài và ngắn.

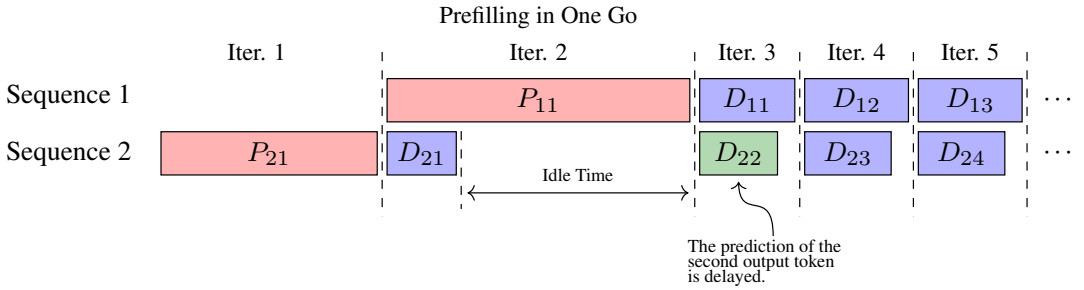
Một cách đơn giản để giảm độ trễ giải mã là làm cho lượng tính toán của các chuỗi khác nhau trong lô tương đương nhau. Một phương pháp như vậy là chia các chuỗi thành các khúc và thực hiện điền trước theo từng khúc. Cách tiếp cận này, thường được gọi là điền trước theo khúc (*chunked prefilling*), mỗi lần xử lý một phần nhỏ hơn của mỗi chuỗi, cho phép hệ thống cân bằng tốt hơn tải tính toán giữa các chuỗi [Agrawal và cộng sự, 2023]. Bằng cách chọn kích thước khúc phù hợp, ta có thể bảo đảm rằng khi điền trước và giải mã chồng lấp nhau trên hai chuỗi, việc xử lý chúng trong cùng một vòng lặp có xu hướng tốn lượng thời gian tương tự nhau. Nhờ đó, thời gian nhàn rỗi của giải mã giảm và thông lượng tổng thể được cải thiện.

Hình 11.13 minh họa điền trước theo khúc qua vài vòng lặp. Trong ví dụ này, lô chứa hai chuỗi. Toàn bộ quá trình điền trước của chuỗi thứ nhất được chia thành ba bước điền trước, tạo ra các khúc ký hiệu là P_{11}, P_{12} và P_{13}. Mỗi khúc tương ứng với một vòng lặp và do đó có thể chồng lấp với một bước giải mã. Bằng cách này, trong khi điền trước chuỗi thứ nhất, ta có thể thực hiện ba bước giải mã thay vì chỉ một bước giải mã như trong lập lịch mức vòng lặp tiêu chuẩn. Nhờ đó, thời gian nhàn rỗi của quá trình giải mã giảm và các token đầu ra có thể được sinh sớm hơn.

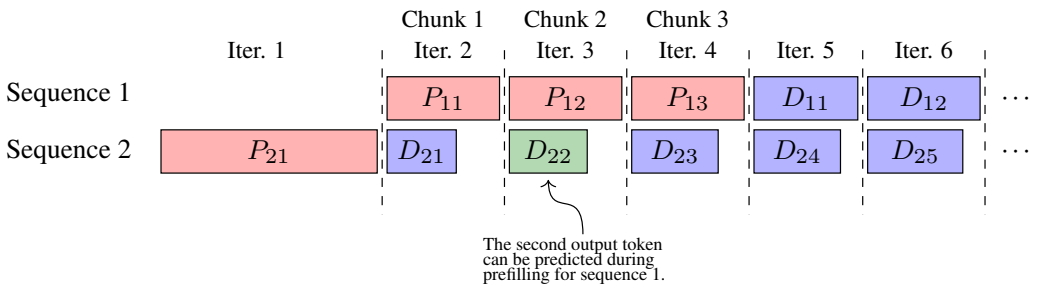
Điền trước theo khúc cải thiện hiệu quả giải mã bằng cách chồng lấp điền trước và giải mã, nhưng phải trả giá bằng chi phí bộ nhớ bổ sung và độ phức tạp lập lịch. Trong điền trước tiêu chuẩn, ta xử lý toàn bộ chuỗi đầu vào một lần, xây dựng bộ nhớ đệm KV trong một lượt. Ngược lại, trong điền trước theo khúc, mỗi khúc cần một lượt truyền xuôi riêng để tính đầu ra chú ý của nó và cập nhật bộ nhớ đệm KV. Do đó, ta cần duy trì bộ nhớ đệm KV của các khúc trước trong khi xử lý các khúc sau. Điều này cũng làm giảm khả năng song song so với việc hoàn tất điền trước cho cả chuỗi trong một lượt. Trong thực tế, thường có thể cân bằng thông lượng và độ trễ bằng cách chọn kích thước khúc phù hợp.

Chú giải của người dịch: Ví dụ số về điền trước theo khúc

Giả sử điền trước một prompt 3000 token mất 300 ms, còn một bước giải mã của chuỗi khác chỉ mất khoảng 10 ms. Nếu điền trước một lượt, chuỗi kia phải chờ khoảng 300 ms mới sinh được token tiếp theo — người dùng thấy chữ "khựng lại". Nếu chia prompt thành 3 khúc 1000 token (mỗi khúc khoảng 100 ms), sau mỗi khúc chuỗi kia lại được sinh thêm một token, nên khoảng chờ giữa hai token giảm còn khoảng 100 ms. Cái giá là phải giữ bộ nhớ đệm KV của các khúc trước và lập lịch phức tạp hơn. (Các con số chỉ mang tính minh



(a) Simple Iteration-level Scheduling



(b) Chunked Prefilling

Hình 11.13: So sánh lập lịch mức vòng lặp đơn giản và điền trước theo khúc. P_{xy} ký hiệu bước điền trước thứ y của chuỗi x, và D_{xy} ký hiệu bước giải mã thứ y của chuỗi x. Trong lập lịch mức vòng lặp đơn giản (hay lập lịch ưu tiên điền trước), vì điền trước được coi là một vòng lặp duy nhất, D_{22} phải chờ quá trình điền trước của chuỗi 1 hoàn thành. Trong điền trước theo khúc, quá trình điền trước có thể được chia thành nhiều bước. Do đó, D_{22} có thể thực thi trong khi điền trước chuỗi 1 (tức là trong P_{12}). (Nhãn trong hình: *Prefilling in One Go*: điền trước một lượt; *Iter.*: vòng lặp; *Idle Time*: thời gian nhàn rỗi; *Chunk*: khúc; *The prediction of the second output token is delayed*: việc dự đoán token đầu ra thứ hai bị trễ; *The second output token can be predicted during prefilling for sequence 1*: token đầu ra thứ hai có thể được dự đoán trong khi điền trước chuỗi 1.)

họa.)

Đáng chú ý là các phương pháp bàn trong tiểu mục này có thể được xếp chung vào nhóm các phương pháp lập lịch dựa trên mức ưu tiên (*priority-based scheduling*). Trong các phương pháp này, ta có thể ưu tiên một số yêu cầu nhất định, hoặc một số bước điền trước hay giải mã nhất định, để tài nguyên hệ thống được phân bổ phù hợp hơn với các mục tiêu hiệu năng cụ thể. Chẳng hạn, như đã trình bày ở trên, ta có thể ưu tiên giải mã hơn điền trước để giảm thiểu độ trễ sinh token, hoặc ưu tiên điền trước hơn giải mã để tối đa hóa thông lượng tổng thể trong các tình huống xử lý theo lô. Những người làm thực tế có thể thiết kế các chính sách ưu tiên tùy biến cho các nhu cầu cụ thể và các ràng buộc vận hành trong ứng dụng thực tế, chẳng hạn thời hạn của yêu cầu và mức độ quan trọng do người dùng xác định.

11.2.3 Song song hóa

Song song hóa (*parallelization*) là một cách tiếp cận được dùng rộng rãi để mở rộng quy mô suy luận LLM, đặc biệt cho các triển khai quy mô lớn. Ở Chương 7, chúng ta đã bàn về một số chiến lược phổ biến để song song hóa quá trình tiền huấn luyện (*pre-training*) LLM, chẳng hạn song song mô hình (*model parallelism*), song song tensor (*tensor parallelism*) và song song đường ống (*pipeline parallelism*). Chúng ta cũng đã bàn về các kiến trúc hiệu quả, dễ triển khai trên các hệ thống tính toán phân tán. Ví dụ, trong các mô hình MoE, ta gán các chuyên gia khác nhau cho các thiết bị khác nhau[3]. Chỉ những chuyên gia được kích hoạt cho một đầu vào cho trước mới được thực thi, điều này cải thiện đáng kể hiệu quả tính toán trong khi vẫn giữ được chất lượng mô hình. Nhiều phương pháp trong số này có thể được áp dụng trực tiếp cho suy luận LLM với rất ít sửa đổi.

Tuy nhiên, áp dụng các kỹ thuật song song hóa này cho suy luận đặt ra những thách thức mới so với tiền huấn luyện. Các vấn đề này trở nên đặc biệt rõ rệt trong các tình huống suy luận thời gian thực hoặc độ trễ thấp, nơi sự mất cân bằng tải giữa các thiết bị và chi phí truyền thông có thể ảnh hưởng đáng kể đến hiệu năng. Chẳng hạn, khác với tiền huấn luyện, nơi các lô có thể được chuẩn bị trước, suy luận phải xử lý các chuỗi có độ dài thay đổi theo thời gian thực. Điều này khiến việc duy trì mức sử dụng thiết bị tối ưu khó khăn hơn và làm phức tạp việc lập lịch trên các tài nguyên tính toán không đồng nhất. Một vấn đề liên quan là cân bằng tải (*load balancing*). Khi một số lượng lớn yêu cầu đến trong một khoảng thời gian ngắn, hệ thống phải phân phối khối lượng công việc một cách hiệu quả lên các thiết bị sẵn có. Ví dụ, các yêu cầu trong thực tế thường có nhu cầu tính toán biến động rất mạnh do khác biệt về loại nhiệm vụ và độ dài prompt. Sự biến động này khiến các cách cân bằng tải tĩnh đơn giản trở nên kém hiệu quả, vì vậy ta cần dùng những chiến lược chi tiết hơn, có thể thích ứng với điều kiện lúc chạy. Bài toán còn phức tạp hơn nữa khi ta triển khai hệ thống trên phần cứng không đồng nhất và có các ràng buộc độ trễ nghiêm ngặt.

Trong việc phát triển LLM, song song hóa gắn chặt với việc phục vụ LLM (*LLM serving*). Xây dựng một hệ thống phục vụ LLM chất lượng cao không phải là việc đơn giản — nó thường đòi hỏi kết hợp nhiều kỹ thuật, chẳng hạn thiết kế kiến trúc, phân phối khối lượng công việc, và các tối ưu phần cứng/phần mềm dành riêng cho LLM. Do đó, phục vụ LLM là một chủ đề rộng, thường đòi hỏi nhiều chuyên môn kỹ thuật. Ở đây, chúng ta sẽ không đi vào chi tiết của việc phục vụ LLM. Về các khái niệm và kỹ thuật liên quan, bạn đọc có thể tham khảo các hệ thống mã nguồn mở liên quan (như vLLM[4],

[3] Trong LLM, các chuyên gia thường là các mô-đun FFN. Vì vậy mỗi chuyên gia là một phần của thành phần FFN trong kiến trúc Transformer.

[4] `https://github.com/vllm-project/vllm`

TensorRT-LLM[5] và TGI[6]) và các bài báo [Pope và cộng sự, 2023; Li và cộng sự, 2024a].

11.2.4 Nhận xét

Trong chương này và các chương trước, chúng ta đã xem xét nhiều phương pháp nâng cao hiệu quả của LLM. Dù các cách tiếp cận này giải quyết những vấn đề khác nhau, phần lớn chúng về bản chất là khám phá các sự đánh đổi giữa nhiều yếu tố hiệu năng. Một sự đánh đổi quan trọng là giữa tốc độ suy luận và độ chính xác. Chẳng hạn, các kỹ thuật như lượng tử hóa, tỉa và chưng cất tri thức (*knowledge distillation*) có thể giảm đáng kể chi phí tính toán và độ trễ nhưng có thể làm giảm nhẹ hiệu năng mô hình. Ngược lại, dùng độ chính xác số cao hơn hoặc mô hình lớn hơn giúp tăng độ chính xác nhưng phải trả giá bằng suy luận chậm hơn và nhu cầu tài nguyên cao hơn.

Một cân nhắc quan trọng khác trong suy luận LLM là sự đánh đổi bộ nhớ–tính toán (*memory-compute trade-off*). Như trong thiết kế hệ thống máy tính, ta cần cân nhắc sự cân bằng giữa lượng bộ nhớ dùng và lượng tính toán cần thiết để sinh đầu ra. Đặc biệt, việc lưu các kết quả trung gian như bộ nhớ đệm KV trong khi suy luận có thể giảm đáng kể tính toán dư thừa, nhưng phải trả giá bằng mức dùng bộ nhớ tăng lên. Trong lưu đệm KV, việc lưu các trạng thái chú ý trước đó giúp tránh tính lại tự chú ý trên các token trước, nhờ đó giảm thời gian tính toán cho mỗi token. Tuy nhiên, khi số token tăng, dung lượng bộ nhớ của bộ nhớ đệm KV cũng tăng theo, đặc biệt khi xử lý các chuỗi rất dài hoặc nhiều chuỗi song song. Để đối phó, nhiều kỹ thuật đã được phát triển nhằm giảm tiêu thụ bộ nhớ bằng cách tính lại một phần các trạng thái trung gian. Chẳng hạn, chú ý theo khúc hoặc chú ý theo cửa sổ giới hạn phạm vi chú ý vào một tập con các token gần đây, và do đó giảm kích thước bộ nhớ đệm KV. Tuy nhiên, việc này thường đòi hỏi thêm tính toán nếu thông tin trong quá khứ phải được xử lý lại.

Lưu ý rằng việc cân nhắc đánh đổi bộ nhớ–tính toán là một nguyên lý rất tổng quát. Nó có thể được mở rộng ra ngoài cơ chế chú ý và Transformer, sang các thành phần khác trong thiết kế hệ thống. Một ví dụ là việc chọn độ chính xác dữ liệu. Dùng các định dạng có độ chính xác thấp hơn như FP16 hay INT8 có thể giảm cả mức dùng bộ nhớ lẫn yêu cầu băng thông bộ nhớ, giảm tải hiệu quả cho phân hệ bộ nhớ. Tuy nhiên, độ chính xác thấp hơn có thể dẫn tới mất ổn định số hoặc giảm nhẹ độ chính xác, đòi hỏi phải hiệu chỉnh hoặc huấn luyện lại cẩn thận. Vì vậy, sự đánh đổi này cũng có thể được xem như một tam giác bộ nhớ–tính toán–độ chính xác, trong đó cải thiện ở một chiều có thể phải trả giá bằng một chiều khác.

Chú giải của người dịch: FP16 và INT8 là gì?

Máy tính lưu mỗi số bằng một số lượng bit nhất định. FP32 dùng 32 bit (4 byte) cho một số thực, FP16 dùng 16 bit (2 byte), INT8 dùng 8 bit (1 byte) cho một số nguyên trong khoảng -128 đến 127. Một mô hình 7 tỷ tham số cần khoảng $7 \times 10^9 \times 2 = 14$ GB nếu lưu ở FP16, nhưng chỉ khoảng 7 GB nếu lưu ở INT8. Đổi lại, số được lưu "thô" hơn: giống như làm tròn điểm thi từ hai chữ số thập phân xuống còn số nguyên — gọn hơn nhưng mất bớt độ chi tiết.

Ngoài tốc độ, độ chính xác và bộ nhớ, một số chiều khác cũng ảnh hưởng tới hiệu quả suy luận LLM. Một số chiều đã được bàn trong chương này, số khác thì chưa. Ở đây chúng tôi phác thảo chúng như sau.

- **Thông lượng và độ trễ**: Trong các tình huống phục vụ LLM quy mô lớn cho nhiều người dùng, ta thường muốn tối đa hóa thông lượng hệ thống. Chẳng hạn, như đã bàn trong mục này, ta có thể gom nhiều yêu cầu thành lô để tăng số token được xử lý cùng lúc. Tuy nhiên, xử lý theo lô làm

[5] `https://github.com/NVIDIA/TensorRT-LLM`
[6] `https://github.com/huggingface/text-generation-inference`

tăng thời gian chờ và có thể dẫn tới độ trễ cao hơn cho mỗi yêu cầu, đặc biệt với các yêu cầu ngắn hoặc mang tính tương tác. Ngược lại, tối ưu cho độ trễ thấp thường đòi hỏi phục vụ từng yêu cầu riêng lẻ hoặc theo các lô nhỏ hơn, khiến tài nguyên phần cứng không được dùng hết và làm giảm thông lượng. Đạt được sự cân bằng tốt tùy thuộc vào các yêu cầu về chất lượng dịch vụ (*quality of service*) và kiểu tương tác của người dùng.

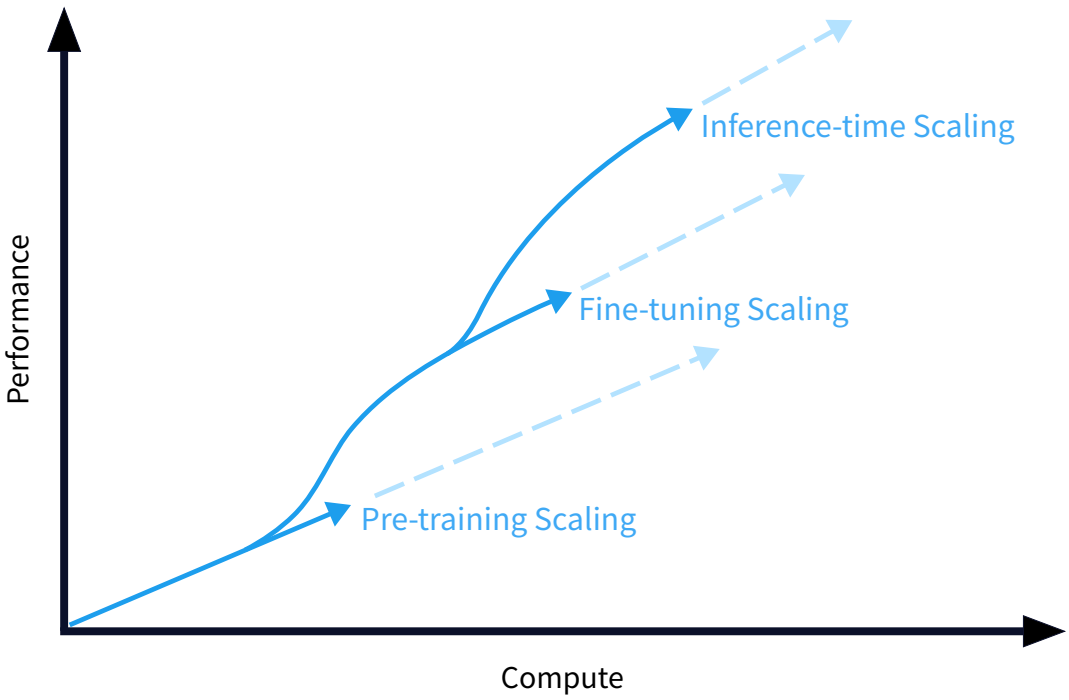
- **Tổng quát hóa và chuyên biệt hóa**: Các LLM đa năng được huấn luyện để thực hiện nhiều loại nhiệm vụ với một bộ tham số duy nhất. Dù linh hoạt, chúng có thể kém hiệu quả hoặc kém chính xác hơn cho các nhiệm vụ cụ thể. Các mô hình chuyên biệt có thể cho hiệu năng tốt hơn và chi phí suy luận thấp hơn cho các ứng dụng mục tiêu. Tuy nhiên, duy trì nhiều mô hình chuyên biệt làm tăng độ phức tạp của hệ thống và yêu cầu lưu trữ. Sự đánh đổi giữa việc duy trì một mô hình tổng quát duy nhất và nhiều mô hình chuyên biệt là một lựa chọn thiết kế quan trọng ở cấp hệ thống.
- **Hiệu quả năng lượng và hiệu năng**: Suy luận hiệu năng cao thường đòi hỏi chạy các mô hình lớn với thông lượng cao trên các bộ tăng tốc mạnh, vốn tiêu thụ rất nhiều năng lượng. Điều này có thể là vấn đề với các triển khai ở thiết bị biên hoặc các môi trường nhạy cảm về năng lượng. Các kỹ thuật như nén mô hình có thể cải thiện hiệu quả năng lượng, nhưng thường kèm theo một mức suy giảm chất lượng đầu ra hoặc tăng độ trễ. Do đó, các ràng buộc năng lượng đưa thêm một chiều quan trọng nữa vào việc tối ưu suy luận LLM.

11.3 Co giãn lúc suy luận

Định luật co giãn (*scaling law*) có thể được xem là một trong những nguyên lý nền tảng định hướng sự phát triển của LLM. Ở các chương trước, chúng ta đã nhiều lần bàn rằng việc tăng quy mô dữ liệu huấn luyện, kích thước mô hình và lượng tính toán có thể cải thiện hiệu quả hiệu năng của LLM. Thực ra, định luật co giãn cũng áp dụng cho các giai đoạn hạ nguồn như tinh chỉnh (*fine-tuning*) và suy luận (xem Hình 11.14). Ở đây ta xét **co giãn lúc suy luận** (*inference-time scaling*), một kỹ thuật đã được các LLM gần đây dùng rộng rãi để giải các bài toán khó, chẳng hạn các bài toán toán học phức tạp [Snell và cộng sự, 2025]. Khác với co giãn trong tiền huấn luyện và tinh chỉnh, vốn tập trung cải thiện LLM qua việc cập nhật tham số, co giãn lúc suy luận cải thiện hiệu năng mô hình trong lúc suy luận mà không cần huấn luyện thêm. Các phương pháp này co giãn suy luận theo nhiều cách khác nhau, chẳng hạn kết hợp nhiều đầu ra của mô hình, tăng độ dài ngữ cảnh, áp dụng các thuật toán giải mã "mạnh tay" hơn, và dùng công cụ bên ngoài để mở rộng năng lực của mô hình.

Dù co giãn lúc suy luận là một chủ đề rộng, trong mục này ta xét các phương pháp đưa thêm lượng tính toán vào suy luận (gọi là co giãn tính toán lúc suy luận (*inference-time compute scaling*)). Dưới đây là danh sách các phương pháp co giãn tính toán lúc suy luận (hay lúc kiểm tra — *test-time*), sắp xếp theo nhóm:

- **Co giãn ngữ cảnh** (*Context Scaling*). Bao gồm việc mở rộng đầu vào hoặc ngữ cảnh để cải thiện việc sinh (hoặc có thể mở rộng cả đầu ra).
- **Co giãn tìm kiếm** (*Search Scaling*). Bao gồm việc tăng công sức tính toán trong khi giải mã.
- **Kết hợp đầu ra** (*Output Ensembling*). Bao gồm việc kết hợp nhiều đầu ra của mô hình.
- **Sinh và kiểm chứng các đường suy nghĩ** (*Generating and Verifying Thinking Paths*). Bao gồm việc hướng dẫn LLM sinh và kiểm chứng các đường suy nghĩ để giải các bài toán lập luận (*reasoning*) phức tạp.



Hình 11.14: Co giãn cho các giai đoạn tiền huấn luyện, tinh chỉnh và suy luận [Briski, 2025]. (Nhãn trong hình: *Performance*: hiệu năng; *Compute*: lượng tính toán; *Pre-training Scaling*: co giãn tiền huấn luyện; *Fine-tuning Scaling*: co giãn tinh chỉnh; *Inference-time Scaling*: co giãn lúc suy luận.)

Chúng ta sẽ mô tả các phương pháp này trong các tiểu mục sau.

Chú giải của người dịch: Co giãn lúc suy luận: "nghĩ lâu hơn" thay vì "học nhiều hơn"

Ba đường cong ở Hình 11.14 cho thấy ba cách tăng hiệu năng bằng cách bỏ thêm tính toán: lúc tiền huấn luyện (mô hình to hơn, nhiều dữ liệu hơn), lúc tinh chỉnh, và lúc suy luận. Cách cuối cùng giống như một học sinh đi thi: kiến thức (tham số) đã cố định, nhưng nếu được phép nháp nhiều hơn, thử vài cách giải rồi so sánh, và kiểm tra lại bài trước khi nộp, thì vẫn có thể đạt điểm cao hơn. Cái giá là mất nhiều thời gian (tính toán) hơn cho mỗi câu hỏi.

11.3.1 Co giãn ngữ cảnh

Co giãn ngữ cảnh (*context scaling*) cải thiện hiệu năng LLM bằng cách mở rộng đầu vào cho mô hình. Một cách tiếp cận trực tiếp là đưa thêm ngữ cảnh liên quan vào lúc suy luận để mô hình có thể dựa các dự đoán của nó trên nhiều thông tin có trước hơn. Một ví dụ là prompt few-shot (*few-shot prompting*). Nó bổ sung vào ngữ cảnh nhiều ví dụ đầu vào–đầu ra, nhờ đó mô hình có thể học ngầm hành vi của nhiệm vụ từ các ví dụ này mà không cần cập nhật tham số. Bên cạnh prompt few-shot, ta có thể dùng prompt chuỗi suy nghĩ (*chain-of-thought prompting*) để khuyến khích mô hình tạo ra các bước lập luận trung gian trước khi đưa ra câu trả lời cuối cùng. Lưu ý rằng prompt chuỗi suy nghĩ là một trong những phương pháp quan trọng nhất để cải thiện hiệu năng lập luận. Bằng cách cung cấp tường minh các bước trung gian trong quá trình giải bài toán, ta có thể nhắc mô hình chia nhỏ các nhiệm vụ phức tạp thành các nhiệm vụ con đơn giản hơn, điều được cho là rất có lợi để sinh ra các đầu ra chính xác và dễ diễn giải.

Ngoài việc mở rộng prompt bằng các ví dụ hay các bước lập luận, một cách tiếp cận khác của co giãn ngữ cảnh là đưa tri thức bên ngoài vào một cách động. Điều này thường được thực hiện bằng sinh văn bản tăng cường truy xuất (*retrieval-augmented generation, RAG*). Các hệ thống RAG trước hết truy xuất các đoạn tài liệu liên quan từ một bộ sưu tập tài liệu lớn hoặc một cơ sở dữ liệu dựa trên đầu vào hiện tại. Những mẩu thông tin truy xuất được này sau đó được thêm vào ngữ cảnh cung cấp cho LLM. Về bản chất, việc này mở rộng ngữ cảnh để bao gồm tri thức bên ngoài mang tính thời sự hoặc chuyên biệt. Bằng cách đó, mô hình neo các phản hồi của nó vào tri thức cụ thể tìm thấy ở nguồn bên ngoài. Nhờ vậy, LLM có thể sinh ra các phản hồi không chỉ liên quan tới đầu vào mà còn chính xác về mặt dữ kiện và cập nhật.

Tuy nhiên, khi ngữ cảnh tăng lên, các phương pháp này thường gặp phải giới hạn của một cửa sổ ngữ cảnh (*context window*) hữu hạn. Dù các kiến trúc mô hình và kỹ thuật (như các mô hình chú ý hiệu quả) liên tục phát triển để hỗ trợ ngữ cảnh dài hơn, việc xử lý các đầu vào cực dài vẫn đặt ra thách thức. Chi phí tính toán tăng lên là một yếu tố. Quan trọng hơn, khi cửa sổ ngữ cảnh trở nên rất lớn, mô hình có thể gặp khó khăn trong việc chú ý hiệu quả tới những thông tin liên quan nhất (ví dụ hiện tượng "lạc ở giữa" — *lost in the middle*). Do đó, co giãn ngữ cảnh hiệu quả không chỉ là thêm nhiều thông tin hơn, mà còn là lựa chọn, cấu trúc và trình bày một cách có chiến lược những thông tin thích đáng nhất trong phạm vi năng lực xử lý của mô hình.

Ở đây chúng tôi bỏ qua phần bàn chi tiết về các phương pháp này, vì chúng đã được đề cập ở các chương trước. Xem Chương 8 và 9 để biết thêm chi tiết, bao gồm các phương pháp prompt, RAG và mô hình hóa chuỗi dài.

11.3.2 Co giãn tìm kiếm

Trong LLM, giải mã là một quá trình tìm kiếm nhằm tìm ra một cách hiệu quả chuỗi đầu ra tốt nhất cho chuỗi đầu vào đã cho. Co giãn tìm kiếm (*search scaling*) (hay co giãn giải mã) thường gồm hai khía cạnh: co giãn độ dài đầu ra và co giãn không gian tìm kiếm.

Co giãn độ dài đầu ra là việc tăng số token được sinh ra trong khi suy luận. Điều này đặc biệt quan trọng với các nhiệm vụ cần sinh văn bản dài, chẳng hạn viết truyện. Gần đây hơn, việc sinh đầu ra kèm các đường suy nghĩ dài đã cho hiệu năng mạnh trong giải toán và sinh mã. Chẳng hạn, người ta thấy rằng khuyến khích mô hình sinh các đường suy nghĩ dài trước khi đưa ra câu trả lời cuối cùng rất có lợi cho việc thực hiện lập luận phức tạp. Ý tưởng này đã được dùng rộng rãi trong việc phát triển các LLM lập luận gần đây, chẳng hạn o1 của OpenAI [2024] và R1 của DeepSeek [2025]. Chúng ta sẽ bàn chi tiết hơn về co giãn độ dài đầu ra ở Mục 11.3.4.

Mặt khác, co giãn không gian tìm kiếm là việc mở rộng tập các chuỗi đầu ra ứng viên được xem xét trong quá trình tìm kiếm, để có thể tìm được các đầu ra chất lượng cao hơn. Như đã bàn ở Mục 11.1.3, một ví dụ đơn giản là trong tìm kiếm chùm (*beam search*), việc tăng độ rộng chùm (*beam width*) cho phép khám phá nhiều chuỗi ứng viên hơn ở mỗi bước giải mã. Điều này làm tăng khả năng phát hiện các đầu ra tốt hơn, đặc biệt trong các nhiệm vụ mà lời giải tối ưu không thể thấy ngay từ các quyết định cục bộ.

Ngoài việc điều chỉnh thuật toán giải mã, cũng có thể khai thác các cấu trúc gọn để mã hóa một số lượng lớn đầu ra. Chẳng hạn, ta có thể xây dựng và duyệt một cây hoặc một đồ thị các bước lập luận [Yao và cộng sự, 2024]. Trong mô thức này, mỗi nút biểu diễn một lời giải bộ phận hoặc một bước trung gian, còn các cạnh biểu diễn sự chuyển tiếp giữa các trạng thái lập luận. Kiểu tìm kiếm có cấu trúc này cho phép mô hình xem xét đồng thời nhiều đường đi. Một hướng liên quan khác là giải mã lấy cảm hứng từ tìm kiếm cây Monte Carlo (*Monte Carlo tree search*), trong đó mô hình khám phá ngẫu nhiên và chấm điểm các đường đi khác nhau dựa trên các heuristic (*heuristic*) đã học hoặc các mô hình phần thưởng (*reward model*) bên ngoài.

Co giãn tìm kiếm là một ý tưởng rất tổng quát, và nó thường ngầm có mặt trong việc thiết kế các thủ tục tìm kiếm khai thác cấu trúc tìm kiếm, các heuristic và độ bất định của mô hình. Nhiều phương pháp ở trên đã được bàn trước đây, dù ban đầu chúng không được phát triển với mục tiêu chính là co giãn. Tuy nhiên, co giãn tìm kiếm vốn dĩ đi kèm chi phí tính toán. Chẳng hạn, tăng độ rộng chùm trực tiếp dẫn tới mức dùng bộ nhớ cao hơn và thời gian suy luận dài hơn. Trong thực tế, thường có một điểm *lợi ích giảm dần*, mà vượt qua đó việc mở rộng thêm không gian tìm kiếm chỉ cải thiện chút ít chất lượng đầu ra với chi phí tính toán đáng kể. Vì vậy, một chiến lược hiệu quả thường bao gồm việc tìm sự cân bằng tối ưu giữa co giãn và tính khả thi về tính toán.

Chú giải của người dịch: Heuristic và lợi ích giảm dần

Heuristic là một "mẹo kinh nghiệm" giúp đoán nhanh hướng nào có triển vọng, không bảo đảm đúng tuyệt đối — giống như khi chơi cờ, ta ưu tiên xét những nước đi chiếm trung tâm trước. *Lợi ích giảm dần*: ví dụ, tăng độ rộng chùm từ 1 lên 4 có thể làm chất lượng tăng rõ rệt, từ 4 lên 16 tăng thêm một chút, còn từ 16 lên 64 hầu như không thay đổi, trong khi chi phí tính toán vẫn tăng gấp 4 lần mỗi lần.

11.3.3 Kết hợp đầu ra

Nếu có sẵn nhiều đầu ra của mô hình, việc kết hợp chúng thường có lợi để giảm ảnh hưởng của các lỗi riêng lẻ và tạo ra một đầu ra cuối cùng đáng tin cậy hơn. Mỗi mô hình có thể nắm bắt những khía

cạnh khác nhau của phân phối dữ liệu nền, hoặc có những điểm mạnh và điểm yếu riêng. Nhờ kết hợp (*ensembling*), ta có thể lấy trung bình để triệt tiêu nhiễu hoặc các lỗi ngẫu nhiên có trong từng dự đoán riêng lẻ, dẫn tới một kết quả ổn định và đáng tin cậy hơn. Trong việc kết hợp LLM, một trong những cách đơn giản nhất là lấy trung bình các phân phối xác suất (*probability distribution*) của token tiếp theo từ mỗi mô hình, rồi chọn token tiếp theo theo phân phối trung bình này. Hoặc, nếu ta coi bài toán là một nhiệm vụ ra quyết định rời rạc, có thể dùng bỏ phiếu đa số (*majority voting*). Các phương pháp tinh vi hơn có thể bao gồm xếp hạng lại các đầu ra ứng viên do các mô hình khác nhau sinh ra dựa trên một hàm chấm điểm riêng, hoặc thậm chí dùng một bộ học meta (*meta-learner*) để kết hợp các dự đoán một cách thông minh.

Kết hợp đầu ra đòi hỏi thêm tính toán lúc suy luận, chẳng hạn chạy nhiều mô hình hoặc lấy mẫu nhiều đầu ra. Điều này không chỉ làm tăng độ trễ suy luận mà còn dẫn tới độ phức tạp hệ thống bổ sung. Nhưng chất lượng đầu ra không tiếp tục cải thiện mãi khi thêm nhiều mô hình hơn. Trong một số trường hợp, lợi ích của kết hợp đầu ra có thể giảm dần khi số mô hình thành phần trong tổ hợp vượt quá một ngưỡng nhất định. Thay vào đó, lợi ích của việc kết hợp thường lớn hơn khi các mô hình riêng lẻ đa dạng (tức là chúng mắc những lỗi khác nhau), ngay cả khi số mô hình thành phần tương đối ít. Do đó, nếu có sẵn nhiều mô hình, sự đa dạng giữa các thành viên của tổ hợp cũng là điều đáng mong muốn.

Chú giải của người dịch: Vì sao kết hợp giúp ích? Một ví dụ số

Giả sử ba mô hình cho phân phối xác suất của token tiếp theo trên hai lựa chọn "A" và "B" lần lượt là $(0.6,\ 0.4)$, $(0.3,\ 0.7)$ và $(0.7,\ 0.3)$. Trung bình là $\big((0.6{+}0.3{+}0.7)/3,\ (0.4{+}0.7{+}0.3)/3\big) \approx (0.53,\ 0.47)$, nên chọn "A". Với bỏ phiếu đa số, hai mô hình chọn "A", một chọn "B", kết quả cũng là "A". Nếu mỗi mô hình đúng với xác suất 0.7 và mắc lỗi *độc lập* với nhau, thì xác suất ít nhất 2 trên 3 mô hình đúng là $0.7^3 + 3 \cdot 0.7^2 \cdot 0.3 = 0.343 + 0.441 = 0.784 > 0.7$. Đây chính là lý do sự *đa dạng* (lỗi khác nhau) quan trọng: nếu ba mô hình luôn sai cùng một chỗ, bỏ phiếu chẳng giúp được gì.

Kết hợp đầu ra cho thấy co giãn lúc suy luận không chỉ giới hạn ở việc tăng kích thước mô hình hay độ dài đầu ra. Bằng cách phân bổ thêm tính toán để sinh và tổng hợp nhiều đầu ra ứng viên, việc kết hợp có thể cải thiện độ vững và chất lượng đầu ra. Cũng như các dạng co giãn lúc suy luận khác, những lợi ích này phải được cân đối với chi phí tính toán bổ sung.

11.3.4 Sinh và kiểm chứng đường suy nghĩ

Cho đến giờ, chúng ta đã xem co giãn lúc suy luận là một lớp phương pháp tổng quát để co giãn nhiều khía cạnh khác nhau của suy luận, chẳng hạn độ dài chuỗi, kích thước mô hình và/hoặc chiến lược tìm kiếm. Thực ra, một ứng dụng thành công là dùng co giãn lúc suy luận để nâng cao năng lực lập luận của LLM. Như ta đã thấy, prompt chuỗi suy nghĩ cải thiện hiệu năng lập luận của LLM bằng cách khuyến khích sinh các bước lập luận trung gian trước khi đi đến câu trả lời cuối cùng. Tuy nhiên, các bài toán lập luận thường phức tạp tới mức ta không thể có được lời giải chất lượng cao chỉ bằng cách đưa ra các prompt chuỗi suy nghĩ đơn giản. Chẳng hạn, khi giải một bài toán, ta thường cần lập luận qua một dãy các bước. Ở mỗi bước, ta cần tìm ra một kết quả trung gian nào đó, kiểm chứng nó, rồi quyết định làm gì tiếp theo. Đường lập luận không phải là một khuôn mẫu cố định mà là một quá trình suy nghĩ được sinh ra một cách động, thường bao gồm thử và sai, quay lui và tự sửa lỗi. Điều này đòi hỏi các chiến lược prompt hoặc các thuật toán tìm kiếm tinh vi hơn để dẫn đường trong kiểu lập luận phức tạp như vậy. Trong tiểu mục này, chúng ta tập trung vào các phương pháp co giãn lúc suy luận vượt ra ngoài chuỗi suy nghĩ đơn giản để giải quyết hiệu quả hơn các bài toán lập luận phức tạp.

Ở mức tổng quát, các phương pháp co giãn khả năng lập luận của LLM có thể được chia thành hai

lớp:

- **Các phương pháp không cần huấn luyện** (*Training-free Methods*). Các phương pháp này nhằm cải thiện năng lực lập luận mà không cập nhật các tham số của mô hình đã tiền huấn luyện. Thay vào đó, chúng tập trung vào các kỹ thuật áp dụng trong lúc suy luận, chẳng hạn các chiến lược prompt tinh vi (ví dụ chuỗi suy nghĩ) và việc điều khiển quá trình lập luận bằng thuật toán (ví dụ tìm kiếm).

- **Các phương pháp dựa trên huấn luyện** (*Training-based Methods*). Các phương pháp này bao gồm việc huấn luyện thêm hoặc tinh chỉnh các tham số mô hình để cải thiện tường minh khả năng lập luận, chẳng hạn tinh chỉnh có giám sát (*supervised fine-tuning*) trên các tập dữ liệu chứa ví dụ lập luận (ví dụ các bài toán kèm lời giải từng bước).

Sau đây, trước hết chúng ta bàn về các phương pháp không cần huấn luyện, rồi đến các phương pháp dựa trên huấn luyện.

1. Tìm kiếm mức lời giải với bộ kiểm chứng

Cho một chuỗi đầu vào (ví dụ một bài toán), có nhiều chuỗi đầu ra khả dĩ (ví dụ các lời giải của bài toán). Nếu có một mô hình để đánh giá hay kiểm chứng từng lời giải, ta có thể chọn ra lời giải tốt nhất. Đây là nguyên lý nền tảng đằng sau các phương pháp như lấy mẫu tốt nhất trong N (*best-of-N sampling*), trong đó nhiều đầu ra được sinh ra và kết quả tối ưu được chọn dựa trên một cơ chế lựa chọn nào đó. Một quá trình lựa chọn như vậy có thể được xem là một bài toán tìm kiếm, gồm hai thành phần:

- **Thuật toán tìm kiếm** (*Search Algorithm*). Thành phần này xác định chiến lược dùng để khám phá không gian các chuỗi đầu ra (lời giải) khả dĩ và sinh ra một tập ứng viên. Nó có thể đi từ lấy mẫu độc lập đơn giản tới các kỹ thuật tìm kiếm tinh vi hơn như đã bàn ở Mục 11.1.3.

- **Bộ kiểm chứng** (*Verifier*). Đây là một mô hình hoặc một hàm chịu trách nhiệm đánh giá chất lượng, tính đúng đắn hoặc mức hữu ích của từng lời giải ứng viên do thuật toán tìm kiếm sinh ra. Nó đưa ra một điểm số, một xác suất hoặc một phán quyết cho phép hệ thống chọn ra ứng viên tốt nhất. Bộ kiểm chứng có thể là một LLM khác, hoặc thậm chí là một tập luật định sẵn hay các heuristic.

Cho một bài toán đầu vào $\mathbf{x}$, ta có thể biểu diễn một lời giải đầu ra $\mathbf{y}$ dưới dạng một dãy các bước lập luận:

$$\mathbf{y} \;=\; (a_1, a_2, \cdots, a_{n_r}), \tag{11.34}$$

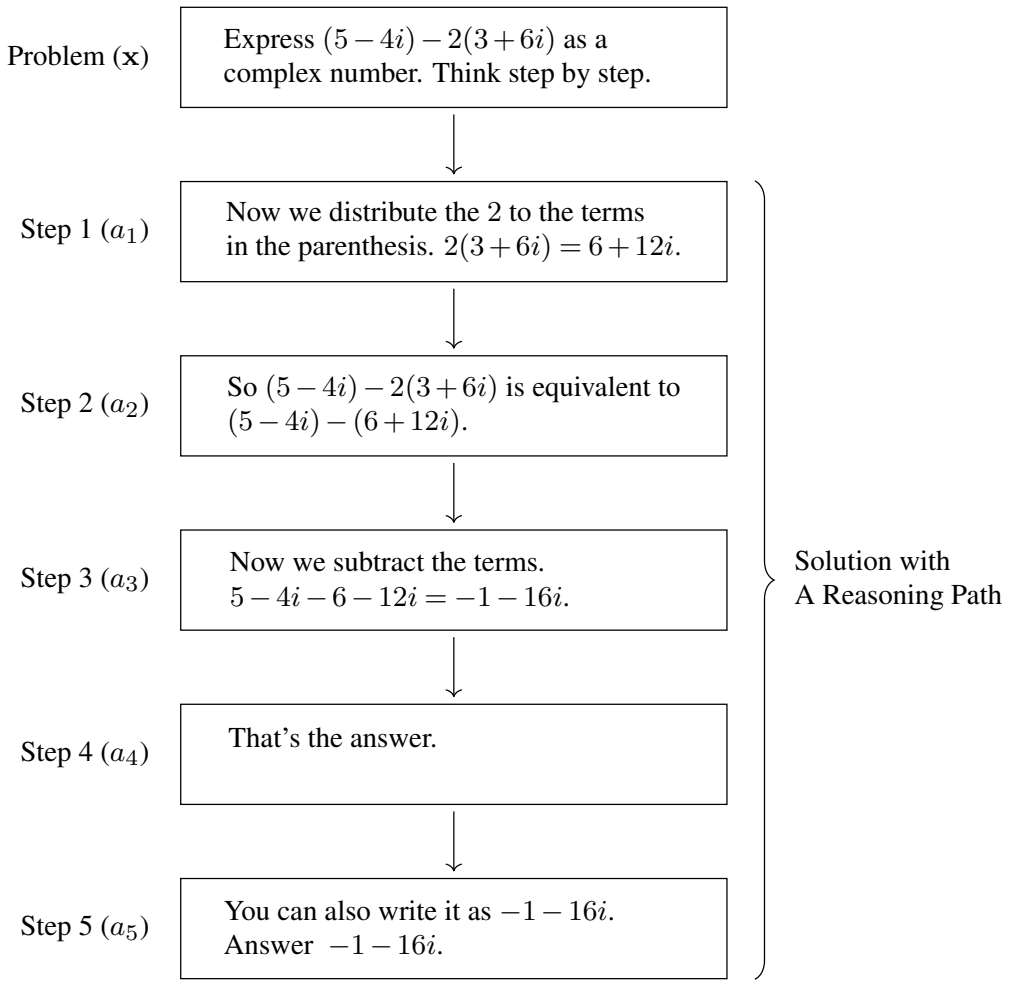
trong đó a_i là bước lập luận thứ i, và a_{n_r} là bước cuối cùng, chứa câu trả lời của bài toán. Xem Hình 11.15 để có một ví dụ về đường lập luận nhiều bước.

Thuật toán tìm kiếm có thể sinh ra một tập các lời giải ứng viên:

$$\mathcal{D}_c \;=\; \{\mathbf{y}_1, \cdots, \mathbf{y}_K\}. \tag{11.35}$$

Sau đó, ta có thể dùng một bộ kiểm chứng, đánh giá từng lời giải bằng hàm $V(\mathbf{y})$, để chấm điểm các ứng viên trong $\mathcal{D}_c$. Đầu ra cuối cùng là ứng viên tốt nhất được bộ kiểm chứng chọn:

$$\hat{\mathbf{y}} \;=\; \arg\max_{\mathbf{y}\in\mathcal{D}_c} V(\mathbf{y}). \tag{11.36}$$



Hình 11.15: Minh họa lập luận nhiều bước. Ví dụ này lấy từ tập dữ liệu PRM800K [Lightman và cộng sự, 2024]. Cho một bài toán, LLM được prompt để sinh một đường suy nghĩ (hay đường lập luận) gồm nhiều bước lập luận. Mỗi bước giải quyết một bài toán con dựa trên kết quả của các bước trước. Câu trả lời cho bài toán ban đầu nằm trong bước cuối cùng. (Nội dung trong hình: *Bài toán* (**x**): "Biểu diễn $(5-4i)-2(3+6i)$ dưới dạng một số phức. Hãy suy nghĩ từng bước." *Bước 1* (a_1): "Bây giờ ta phân phối số 2 vào các số hạng trong ngoặc. $2(3+6i)=6+12i$." *Bước 2* (a_2): "Vậy $(5-4i)-2(3+6i)$ tương đương với $(5-4i)-(6+12i)$." *Bước 3* (a_3): "Bây giờ ta trừ các số hạng. $5-4i-6-12i=-1-16i$." *Bước 4* (a_4): "Đó là câu trả lời." *Bước 5* (a_5): "Bạn cũng có thể viết nó là $-1-16i$. Đáp số $-1-16i$." *Solution with A Reasoning Path*: lời giải kèm một đường lập luận.)

Chú giải của người dịch: Đọc Phương trình (11.34–11.36)

Hãy hình dung ta cho LLM giải cùng một bài toán $K = 5$ lần, được 5 lời giải $\mathbf{y}_1, \ldots, \mathbf{y}_5$ (đó là tập $\mathcal{D}_c$). Bộ kiểm chứng giống như một giáo viên chấm bài, cho mỗi lời giải một điểm $V(\mathbf{y})$, chẳng hạn $3, 8, 5, 8.5, 2$. Ký hiệu arg max nghĩa là "lấy *lời giải* có điểm lớn nhất" (chứ không phải lấy giá trị điểm lớn nhất), nên ở đây $\hat{\mathbf{y}} = \mathbf{y}_4$. Ký hiệu n_r chỉ số bước lập luận của một lời giải, ví dụ ở Hình 11.15 thì $n_r = 5$.

Dù có thể kiểm chứng toàn bộ đường lập luận, một phương án đơn giản hơn là chỉ kiểm chứng bước lập luận cuối cùng. Bằng cách này, hàm kiểm chứng $V(\mathbf{y})$ được đơn giản hóa để chỉ phụ thuộc vào câu trả lời cuối cùng chứa trong a_{n_r}. Điều này có thể đạt được theo nhiều cách, tùy thuộc vào bản chất của bài toán và định dạng câu trả lời mong đợi.

- Với một số bài toán toán học và lập trình, ta có thể dùng các công cụ có sẵn làm bộ kiểm chứng. Ví dụ bao gồm các bộ kiểm tra chứng minh cho các định lý toán học, các trình thông dịch hoặc trình biên dịch để thực thi mã, và các hệ thống kiểm thử đơn vị để kiểm chứng tính đúng đắn của chương trình trên các ca kiểm thử định sẵn.

- Nếu có dữ liệu gán nhãn để đánh giá câu trả lời, chẳng hạn dữ liệu về sở thích của con người (*human preference*), ta có thể huấn luyện một mô hình phần thưởng trên dữ liệu đó. Mô hình phần thưởng học được sau đó được dùng làm bộ kiểm chứng, gán một điểm số vô hướng cho mỗi câu trả lời ứng viên.

- Nếu không có hệ thống sẵn có hay mô hình phần thưởng phù hợp, ta có thể dùng một LLM khác đóng vai trò bộ kiểm chứng. LLM này được prompt để đánh giá chất lượng của câu trả lời ứng viên. Nó có thể là một mô hình mạnh hơn, hoặc chính LLM đó được dùng với một prompt "người đánh giá" cụ thể.

- Ngoài ra, có thể thiết kế các bộ kiểm chứng đơn giản hơn dựa trên heuristic. Một cách thường dùng là bỏ phiếu đa số, trong đó câu trả lời xuất hiện nhiều nhất trong một tập ứng viên được chọn.

Dùng các bộ kiểm chứng này, ta có thể đánh giá một tập lời giải ứng viên được sinh ra trong quá trình tìm kiếm. Một chiến lược đơn giản, thường được gọi là **co giãn song song** (*parallel scaling*) [Brown và cộng sự, 2024; Snell và cộng sự, 2024], bao gồm việc sinh K lời giải ứng viên bằng cách chạy LLM gốc K lần một cách độc lập. Trong quá trình này, ta có thể điều chỉnh nhiệt độ (*temperature*) khi lấy mẫu để kiểm soát độ đa dạng của các đầu ra. Bộ kiểm chứng sau đó đánh giá từng lời giải trong K lời giải hoàn chỉnh này, và lời giải có điểm cao nhất được chọn làm đầu ra cuối cùng. Về mặt khái niệm, điều này rất giống với lấy mẫu tốt nhất trong N, phương pháp mà ở các chương trước chúng ta chủ yếu mô tả như một cách chọn ra đầu ra tốt nhất từ một tập đầu ra được lấy mẫu bằng một mô hình phần thưởng.

Một cách tiếp cận khác là **co giãn tuần tự** (*sequential scaling*), xây dựng một dãy lời giải theo kiểu tăng dần [Gou và cộng sự, 2024; Zhang và cộng sự, 2024]. Nó bắt đầu với một lời giải ban đầu do LLM sinh ra nhờ prompt. Sau đó, ta dùng một bộ kiểm chứng (thường chính là LLM đó) để đánh giá lời giải. Đây có thể xem là một giai đoạn *phê bình*. Đầu ra của giai đoạn này là một dạng phản hồi nào đó, chẳng hạn các nhận xét bằng văn bản chỉ ra lỗi hoặc gợi ý cải tiến, các điểm số phản ánh chất lượng lời giải, hoặc thậm chí một kế hoạch đã sửa đổi hay một bước trung gian để định hướng lần sinh tiếp theo. Phản hồi này, cùng với bài toán ban đầu và lời giải hiện tại, sau đó được dùng để prompt LLM sinh ra một lời giải có thể tốt hơn. Đây có thể xem là một giai đoạn *tinh chỉnh lại*. Chu trình phê bình–tinh chỉnh

lại này có thể được lặp lại, tạo thành một vòng lặp:

$$\mathbf{y}_{k+1} = \text{Refine}(\mathbf{x}, \mathbf{y}_k, \text{Feedback}(\mathbf{y}_k)), \tag{11.37}$$

trong đó $\text{Feedback}(\mathbf{y}_k)$ biểu diễn phản hồi từ bộ kiểm chứng. Hàm $\text{Refine}(\cdot)$ sinh ra lời giải cải tiến $\mathbf{y}_{k+1}$ bằng cách prompt LLM với bài toán ban đầu $\mathbf{x}$, lời giải trước đó $\mathbf{y}_k$ và phản hồi này. Quá trình có thể được lặp lại K vòng, hoặc cho đến khi chất lượng lời giải, theo đánh giá của bộ kiểm chứng, hội tụ về một mức thỏa đáng. Khung lặp này, trong đó một lời giải được cải thiện dần qua các chu trình sinh, đánh giá (phê bình) và sửa đổi, chính là cái tạo nên tự tinh chỉnh (*self-refinement*) [Shinn và cộng sự, 2023; Madaan và cộng sự, 2024]. Trong các tình huống như vậy, vai trò của bộ kiểm chứng không chỉ là chọn lời giải hoàn chỉnh tốt nhất từ một tập cố định, mà còn chủ động định hướng chính quá trình sinh.

Chú giải của người dịch: Song song hay tuần tự?

Co giãn song song giống như nộp cùng một đề cho 5 học sinh làm độc lập rồi chọn bài tốt nhất. Co giãn tuần tự giống như một học sinh làm bài, giáo viên phê "bước 2 sai dấu", học sinh sửa lại, giáo viên phê tiếp, và cứ thế. Trong Phương trình (11.37), $\mathbf{y}_k$ là bản làm thứ k, $\text{Feedback}(\mathbf{y}_k)$ là lời phê, còn Refine là việc "sửa bài theo lời phê" để được bản $\mathbf{y}_{k+1}$.

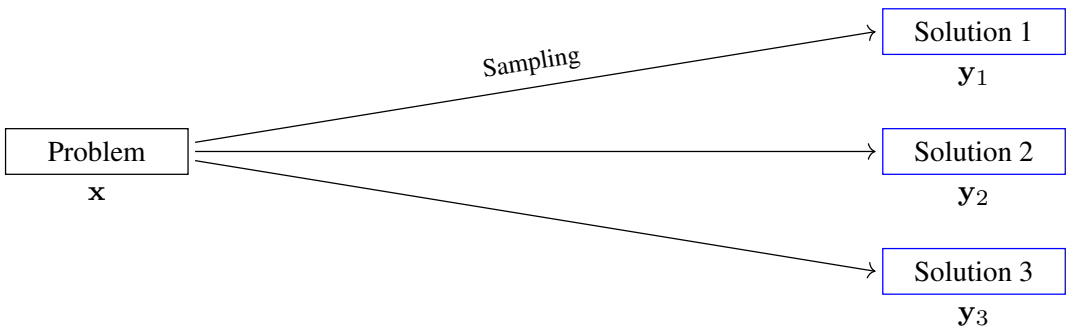
Xem Hình 11.16 để có minh họa về co giãn song song và co giãn tuần tự. Lưu ý rằng còn có những cách khác để thực hiện tìm kiếm và thu được các tập lời giải ứng viên khác nhau. Một phương pháp thay thế là tổ chức việc tìm kiếm thành một cấu trúc cây. Cách tiếp cận này, thường được gọi là tìm kiếm cây (*tree search*), mang lại một cách có cấu trúc hơn để khám phá không gian các đường lập luận khả dĩ. Trong tìm kiếm mức lời giải, mỗi nút của cây biểu diễn một lời giải hoàn chỉnh. Trong quá trình tìm kiếm, ta cần mở rộng một nút thành một tập các nút con, biểu diễn các lời giải mới có thể được xem xét để kiểm chứng. Quá trình mở rộng thường gồm việc lấy một lời giải sẵn có (nút cha) và dùng LLM để sinh ra các biến thể hoặc các lời giải thay thế.

2. Tìm kiếm mức bước với bộ kiểm chứng

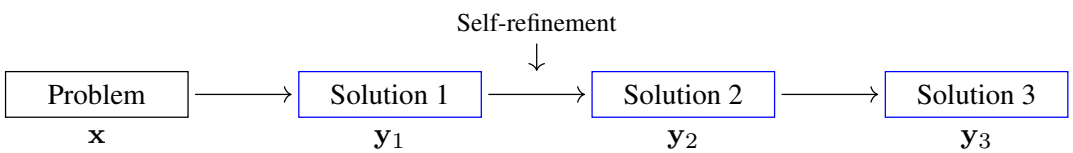
Trong khi các phương pháp bàn ở trên chủ yếu tập trung vào việc sinh các lời giải hoàn chỉnh trước khi lựa chọn cuối cùng, quá trình tìm kiếm cũng có thể được tích hợp sâu hơn vào chính việc sinh từng bước của đường lập luận. Điều này dẫn tới các cách tiếp cận thực hiện tìm kiếm mức bước (*step-level search*) với bộ kiểm chứng, trong đó việc định hướng hoặc cắt tỉa diễn ra ở các bước lập luận trung gian $\{a_1, \cdots, a_i\}$ thay vì chỉ sau khi đã hình thành một lời giải đầy đủ $\mathbf{y}$.

Việc điều khiển chi tiết như vậy đặc biệt có lợi cho các bài toán lập luận phức tạp, nơi chỉ một bước trung gian sai cũng có thể làm toàn bộ chuỗi lập luận phía sau trở nên vô giá trị. Bằng cách đánh giá hoặc định hướng việc sinh ở mỗi bước trung gian, LLM có thể khám phá không gian lập luận hiệu quả hơn, có thể sớm cắt tỉa các đường không triển vọng hoặc dành nhiều tài nguyên hơn để khám phá các đường hợp lý hơn.

Tìm kiếm mức bước với bộ kiểm chứng cũng có thể được mô hình hóa như một bài toán tìm kiếm cây. Trong mô thức này, mỗi nút (hay trạng thái) tương ứng với một đường lập luận bộ phận, $\mathbf{a}_{\leq i} = (a_1, \cdots, a_i)$, biểu diễn dãy i bước lập luận đã thực hiện cho đến lúc này (tức là một đường đi từ nút gốc tới nút hiện tại). Mục tiêu của quá trình tìm kiếm là khám phá không gian trạng thái nền, bắt đầu từ một đường rỗng ban đầu, để tìm ra một đường hoàn chỉnh tạo thành một lời giải đúng. Lưu ý rằng ở đây ta dùng $\mathbf{a}_{\leq i}$ để biểu diễn một đường lập luận bộ phận thay vì $\mathbf{y}_{\leq i}$. Dù điều này làm ký hiệu hơi thiếu nhất quán với ký hiệu dùng cho các lời giải hoàn chỉnh ($\mathbf{y}$) hay các đường đầy đủ trong tìm kiếm mức lời giải, nó giúp nhấn mạnh trọng tâm vào từng hành động hay từng bước riêng lẻ.



(a) Parallel Scaling



(b) Sequential Scaling

Hình 11.16: Minh họa co giãn song song (*Parallel Scaling*) và co giãn tuần tự (*Sequential Scaling*). Trong co giãn song song, ta thu được nhiều lời giải (*Solution*) bằng cách chạy LLM nhiều lần một cách độc lập (*Sampling*: lấy mẫu). Trong co giãn tuần tự, LLM sinh ra một lời giải ban đầu. Sau đó, ta dùng LLM để tinh chỉnh lại nó một cách lặp đi lặp lại (*Self-refinement*: tự tinh chỉnh), mỗi lần tinh chỉnh tạo ra một lời giải mới, có thể tốt hơn. (*Problem*: bài toán.)

Các thành phần cốt lõi của tìm kiếm mức bước với bộ kiểm chứng là:

- **Biểu diễn nút** (*Node Representation*). Một nút là một đường lập luận bộ phận $\mathbf{a}_{\leq i} = (a_1, \cdots, a_i)$. Nút gốc là một đường rỗng, và các nút lá (nút kết thúc) là các đường lập luận hoàn chỉnh.

- **Mở rộng nút** (*Node Expansion*). Cho một đường bộ phận hiện tại $\mathbf{a}_{\leq i}$, LLM được dùng để sinh một hoặc nhiều bước lập luận tiếp theo ứng viên $\{a_{i+1}^{(1)}, \cdots, a_{i+1}^{(M)}\}$. Mỗi bước ứng viên, khi được nối vào $\mathbf{a}_{\leq i}$, tạo thành một đường bộ phận tiềm năng mới $\mathbf{a}_{\leq i+1} = (a_1, \cdots, a_i, a_{i+1}^{(j)})$.

- **Kiểm chứng** (*Verification*). Bộ kiểm chứng $V(\cdot)$ đánh giá chất lượng của một bước mới sinh trong ngữ cảnh của đường bộ phận hiện tại $\mathbf{a}_{\leq i} = (a_1, \cdots, a_i)$ và bài toán ban đầu $\mathbf{x}$. Cũng như kiểm chứng mức lời giải, các bộ kiểm chứng mức bước có thể đưa ra một điểm số, một nhãn phân loại, hoặc phản hồi bằng văn bản.

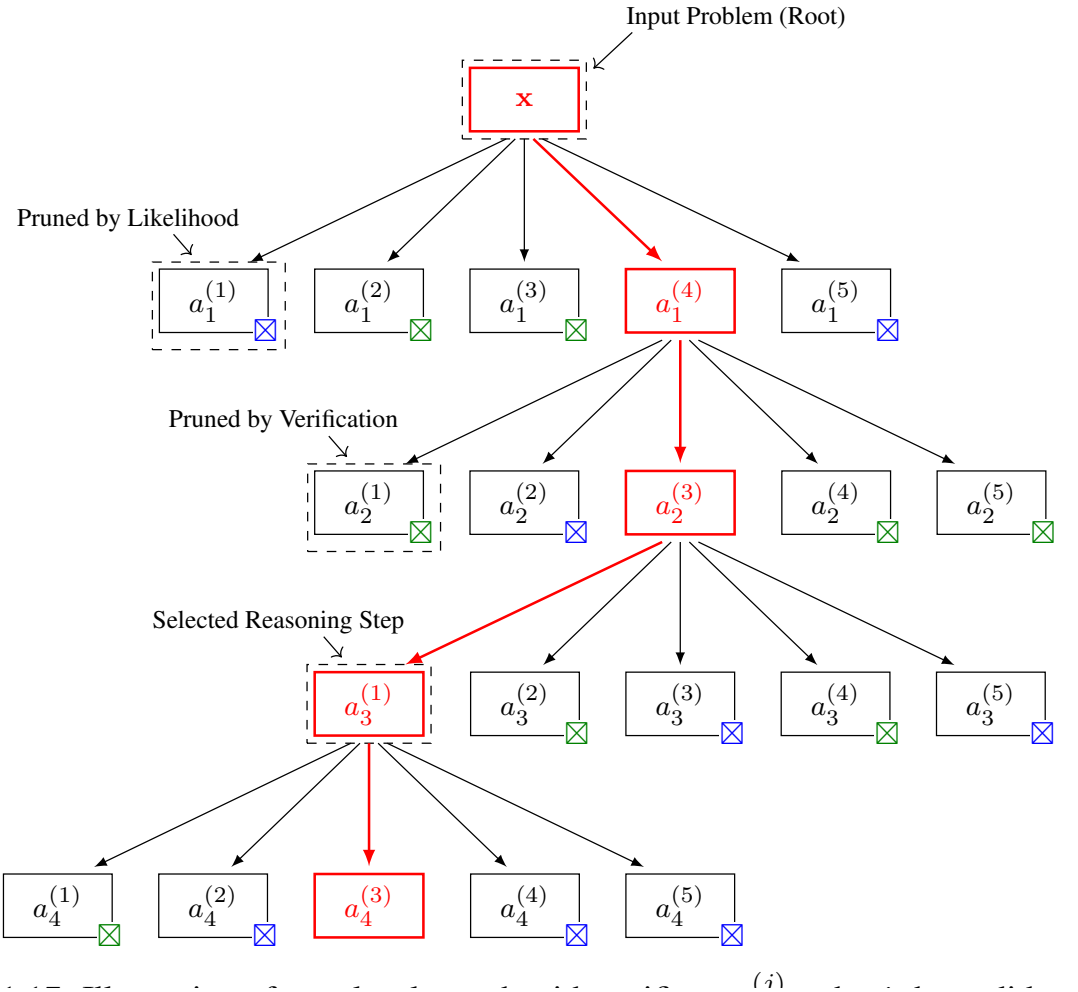
- **Tìm kiếm** (*Search*). Thành phần này chi phối cách khám phá không gian tìm kiếm. Dựa trên các đánh giá từ bộ kiểm chứng, chiến lược tìm kiếm quyết định những đường bộ phận nào được mở rộng tiếp, những đường nào bị cắt tỉa, và thứ tự khám phá.

Việc kiểm chứng từng bước này cho phép điều chỉnh động quá trình lập luận. Nếu một bước a_{i+1} bị $V(\cdot)$ đánh giá là sai hoặc không triển vọng, thuật toán tìm kiếm có thể quay lui và khám phá các bước thay thế từ $\mathbf{a}_{\leq i}$, hoặc thậm chí từ một nút sớm hơn $\mathbf{a}_{\leq i'}$ (với $i' < i$). Ngược lại, nếu một bước được đánh giá cao, tài nguyên có thể được tập trung vào việc mở rộng đường đó. Xem Hình 11.17 để có minh họa về tìm kiếm mức bước với bộ kiểm chứng.

Rõ ràng, khung tìm kiếm này rất giống khung được dùng trong các phương pháp giải mã cho LLM, như đã bàn ở Mục 11.1.3. Chẳng hạn, tìm kiếm chùm duy trì một tập gồm K chuỗi bộ phận triển vọng nhất ở mỗi bước sinh. Đây là một dạng tìm kiếm mức bước, trong đó "bộ kiểm chứng" ngầm định chính là mô hình xác suất của LLM, còn "tìm kiếm" là cơ chế cắt tỉa để duy trì kích thước chùm.

Tuy nhiên, tìm kiếm mức bước với các bộ kiểm chứng tường minh, như mô tả ở đây, khác với các phương pháp giải mã tiêu chuẩn. Một khác biệt then chốt là bộ kiểm chứng có thể là một thành phần tinh vi hơn nhiều so với chỉ các xác suất đầu ra thô của LLM sinh. Thiết kế các bộ kiểm chứng mức bước tương tự như thiết kế kiểm chứng mức lời giải. Một bộ kiểm chứng mức bước có thể là một mô hình ngôn ngữ đánh giá chất lượng của một bước lập luận riêng lẻ trong ngữ cảnh của đường đi phía trước. LLM này thậm chí có thể được tinh chỉnh để tăng cường khả năng kiểm chứng. Ngoài ra, với các lĩnh vực có luật lệ được định nghĩa rõ ràng, nó có thể là một bộ máy ký hiệu hoặc một tập các phép kiểm tra bằng chương trình. Hơn nữa, các bộ kiểm chứng có thể được thiết kế để dự đoán mức hữu ích trong tương lai hoặc khả năng thành công khi biết đường bộ phận hiện tại, lấy cảm hứng từ các hàm giá trị (*value function*) trong học tăng cường (*reinforcement learning*). Chuyên môn của con người cũng có thể được đưa vào để đưa ra phán đoán ở các bước then chốt, đặc biệt trong các tình huống có rủi ro cao.

Một ví dụ về bộ kiểm chứng mức bước như vậy, đặc biệt khi dùng phản hồi của con người để đánh giá tiến trình trung gian, là **mô hình phần thưởng quá trình** (*process reward model*) (**PRM**). Một PRM thường là một mô hình ngôn ngữ riêng biệt, được huấn luyện để đưa ra một phần thưởng vô hướng cho mỗi bước lập luận $a_{i'}$ trong một đường bộ phận $\mathbf{a}_{\leq i}$. Nó cung cấp một tín hiệu giám sát trực tiếp và chi tiết hơn so với các **mô hình phần thưởng kết quả** (*outcome reward model*) (**ORM**), vốn chỉ đánh giá lời giải cuối cùng. Tuy nhiên, việc phát triển PRM dựa vào các chú thích mức bước của con người, chẳng hạn mức ưa thích đối với các bước tiếp theo khác nhau. Thu thập tín hiệu giám sát cho từng bước trung gian tốn nhiều công sức hơn đáng kể và đòi hỏi người gán nhãn nỗ lực nhận thức nhiều hơn so với chỉ gán nhãn kết quả cuối cùng.



Hình 11.17: Minh họa tìm kiếm mức bước với bộ kiểm chứng. $a_i^{(j)}$ = ứng viên thứ j cho bước lập luận thứ i; dấu ☒ (xanh lam) = ứng viên bị cắt tỉa theo xác suất đầu ra của LLM; dấu ☒ (xanh lục) = ứng viên bị bộ kiểm chứng cắt tỉa. Với bài toán đầu vào làm nút gốc (*Input Problem (Root)*), ta mở rộng cây bằng cách sinh nhiều bước lập luận ở mỗi lần mở rộng. Mỗi ứng viên có thể bị cắt tỉa hoặc theo khả năng (*likelihood*) (như trong giải mã tiêu chuẩn; *Pruned by Likelihood*) hoặc theo kiểm chứng mức bước (*Pruned by Verification*). Các ứng viên không bị cắt tỉa sau đó được mở rộng để sinh thêm các bước lập luận (*Selected Reasoning Step*: bước lập luận được chọn). Quá trình được lặp lại cho đến khi sinh ra một chuỗi lập luận hoàn chỉnh dẫn tới câu trả lời cuối cùng, hoặc cho đến khi đạt tới một giới hạn tìm kiếm định sẵn.

Chú giải của người dịch: PRM và ORM: chấm từng bước hay chỉ chấm đáp số?

Hãy nghĩ tới cách chấm bài thi tự luận môn Toán. ORM giống như chỉ nhìn đáp số cuối cùng: đúng thì cho điểm, sai thì không, bất kể cách làm. PRM giống như chấm theo *biểu điểm từng bước*: bước 1 đúng được 0.25, bước 2 đúng được 0.5,… Nhờ vậy PRM chỉ ra được *chỗ nào* trong lời giải bắt đầu sai, rất hữu ích để cắt tỉa sớm các nhánh sai trong cây tìm kiếm ở Hình 11.17. Nhưng để huấn luyện PRM, cần người gán nhãn đọc và đánh giá từng bước, tốn công hơn nhiều.

Một cách tiếp cận thay thế để xây dựng dữ liệu huấn luyện cho kiểm chứng mức bước là dùng LLM để tự động sinh ra các chú thích như vậy. Chẳng hạn, ta có thể lấy một LLM mạnh, gọi là mô hình thầy (*teacher model*), và prompt nó để trước hết sinh một đường lập luận hoàn chỉnh cho một bài toán cho trước. Sau đó, tại mỗi bước trung gian trong đường này, ta có thể prompt chính LLM thầy đó (hoặc một LLM có năng lực khác) để sinh ra một vài bước tiếp theo ứng viên thay thế, bên cạnh bước mà nó đã chọn ban đầu. LLM thầy sau đó lại có thể được prompt để đánh giá các phương án thay thế này. Các kết quả đánh giá này (ví dụ đúng hay sai) sau đó có thể dùng làm chú thích dữ liệu. Một cách khác là tận dụng khả năng tổng quát hóa của PRM. Ta có thể huấn luyện một PRM trên các nhiệm vụ mà việc kiểm chứng mức bước dễ hơn, rồi tổng quát hóa PRM này sang các nhiệm vụ khác với rất ít hoặc không cần huấn luyện thêm.

Lưu ý rằng kiểm chứng mức bước cũng có những vấn đề riêng. Kiểm chứng thường xuyên, đặc biệt nếu dùng LLM làm bộ kiểm chứng, có thể làm tăng đáng kể chi phí tính toán và độ trễ. Bản thân việc thiết kế các bộ kiểm chứng mức bước hiệu quả cũng không hề đơn giản. Một bộ kiểm chứng không chính xác có thể loại bỏ quá sớm các đường lập luận tốt hoặc không nhận ra các đường có lỗi, từ đó dẫn dắt sai việc tìm kiếm. Điều này khiến việc phát triển các hệ thống như vậy khó khăn hơn.

3. Khuyến khích suy nghĩ dài

Cho đến giờ trong tiểu mục này, phần lớn các phương pháp đều ngầm dựa trên một ý tưởng đơn giản: sinh các đường lập luận dài hơn có thể có ích. Ngoài CoT và tìm kiếm có kiểm chứng, ta có thể xem xét các phương pháp thay thế để đạt được điều này. Chẳng hạn, ta có thể prompt LLM bằng cách yêu cầu tường minh việc cân nhắc kỹ lưỡng hơn. Ngoài prompt trực tiếp, ta cũng có thể sửa đổi chính quá trình giải mã, chẳng hạn điều chỉnh giới hạn số token hoặc áp dụng hình phạt cho các đầu ra ngắn. Một cách tiếp cận khác là dùng các sơ đồ sinh nhiều giai đoạn, trong đó mô hình xây dựng dần dần dựa trên lập luận của chính nó.

4. Co giãn dựa trên huấn luyện

Bên cạnh việc xem xét các phương pháp co giãn lúc suy luận không cần huấn luyện, chúng ta cũng muốn xem xét các phương pháp có thể cải thiện năng lực lập luận nội tại của LLM bằng cách sửa đổi tham số của chúng thông qua huấn luyện thêm. Dù các phương pháp co giãn dựa trên huấn luyện như vậy thường đòi hỏi thêm chi phí huấn luyện và tài nguyên tính toán, chúng đưa các kỹ năng lập luận mạnh hơn trực tiếp vào tham số mô hình, điều này đến lượt nó có thể dẫn tới hiệu năng lập luận được cải thiện. Ta thậm chí có thể kết hợp chúng với các phương pháp không cần huấn luyện để cải thiện hơn nữa việc co giãn lúc suy luận.

Dù phần thảo luận ở đây chỉ giới hạn trong các bài toán lập luận, các phương pháp co giãn dựa trên huấn luyện là phổ biến. Phần lớn chúng đã được bàn ở Chương 10. Ở đây, chúng tôi sẽ mô tả ngắn gọn cách các phương pháp này có thể được áp dụng để cải thiện co giãn lúc suy luận cho các bài toán lập luận.

- **Tinh chỉnh trên dữ liệu lập luận** (*Fine-tuning on Reasoning Data*). Một trong những cách trực tiếp nhất để tăng cường khả năng lập luận là tinh chỉnh các LLM đã tiền huấn luyện trên các tập dữ liệu được tuyển chọn riêng cho các nhiệm vụ lập luận. Các tập dữ liệu này có thể đi từ các cặp đầu vào–đầu ra đơn giản đến các định dạng có cấu trúc hơn bao gồm cả quá trình lập luận từng bước. Các ví dụ điển hình gồm các tập dữ liệu bài toán có lời văn, các bài tập suy diễn logic, hoặc sinh mã kèm giải thích. Nhờ huấn luyện trên những dữ liệu như vậy, mô hình học được từ các khuôn mẫu lập luận phổ biến, và do đó có thể sinh ra các đường lập luận chi tiết và mạch lạc lúc kiểm tra.

- **Học tăng cường cho lập luận** (*Reinforcement Learning for Reasoning*). Nếu coi bộ kiểm chứng là một mô hình phần thưởng, ta có thể thấy các phương pháp bàn ở tiểu mục trước là một ứng dụng trực tiếp của mô hình phần thưởng vào các bài toán lập luận, dù chúng không cần huấn luyện. Tất nhiên, ta có thể áp dụng mô hình phần thưởng này vào việc tinh chỉnh LLM. Điều này đi theo một mô thức tiêu chuẩn của học tăng cường. Cho một mô hình phần thưởng, LLM, đóng vai trò một chính sách (*policy*), được tinh chỉnh bằng các thuật toán học tăng cường. LLM sinh các bước lập luận hoặc các lời giải đầy đủ, nhận phản hồi (phần thưởng) từ mô hình phần thưởng, và cập nhật tham số để tạo ra các đầu ra tối đa hóa những phần thưởng này. Quá trình này căn chỉnh (*align*) đầu ra của LLM với hành vi lập luận chất lượng cao, qua đó khuyến khích LLM sinh ra các đường lập luận đáng tin cậy hơn. Một vấn đề then chốt khác là việc huấn luyện mô hình phần thưởng. Nói chung, mô hình phần thưởng này có thể là một mô hình phần thưởng kết quả, đánh giá tính đúng đắn hoặc chất lượng của câu trả lời cuối cùng, hoặc một mô hình phần thưởng quá trình, đánh giá chất lượng của từng bước lập luận trung gian, như đã bàn trong bối cảnh các bộ kiểm chứng mức bước. Trong một số trường hợp, ta thậm chí có thể xây dựng một mô hình phần thưởng dựa trên các luật đơn giản, chẳng hạn cộng thêm điểm thưởng cho các đầu ra dài hơn.

- **Chưng cất tri thức cho lập luận** (*Knowledge Distillation for Reasoning*). Trong cách tiếp cận này, một LLM *trò* nhỏ hơn, hiệu quả hơn được huấn luyện để bắt chước các đầu ra lập luận hoặc các biểu diễn bên trong của một LLM *thầy* lớn hơn, có năng lực hơn. Mô hình thầy có thể sinh ra các bước lập luận chi tiết cho nhiều loại bài toán. Mô hình trò sau đó học cách tái tạo những minh họa lập luận chất lượng cao này. Chiến lược này giúp năng lực lập luận mạnh hơn trở nên dễ tiếp cận hơn bằng cách triển khai chúng trong các mô hình nhỏ hơn, ít tốn kém tính toán hơn lúc suy luận.

- **Tinh chỉnh lặp** (*Iterative Refinement*). Co giãn dựa trên huấn luyện cũng có thể bao gồm việc

tinh chỉnh lặp. Chẳng hạn, một LLM có thể sinh lời giải cho một tập bài toán. Các lời giải này cùng đường lập luận của chúng sau đó được kiểm chứng, hoặc bởi con người hoặc bởi các bộ kiểm chứng tự động. Các đường lập luận đúng sau đó được thêm vào dữ liệu huấn luyện, và LLM được tinh chỉnh thêm trên tập dữ liệu đã được bổ sung này. Điều này tạo ra một chu trình trong đó LLM dần cải thiện năng lực lập luận của mình qua việc lặp lại sinh, phê bình và học.

Chú giải của người dịch: Chính sách và phần thưởng trong học tăng cường

Trong học tăng cường, *chính sách* là "cách hành động" của tác tử — ở đây chính là LLM, vì nó quyết định sinh token/bước lập luận nào tiếp theo. *Phần thưởng* là con số cho biết kết quả tốt đến đâu, ví dụ $+1$ nếu đáp số đúng và 0 nếu sai (một mô hình phần thưởng kết quả dựa trên luật). Mô hình thử nhiều lời giải, lời giải nào được thưởng cao thì tham số được điều chỉnh để lần sau dễ sinh ra kiểu lời giải đó hơn — giống như học sinh luyện đề: cách làm nào được điểm thì sẽ được dùng lại nhiều hơn.

Ưu điểm chính của các phương pháp co giãn dựa trên huấn luyện này là chúng trang bị cho LLM các kỹ năng lập luận nội tại mạnh hơn. Điều này trực tiếp góp phần cải thiện co giãn lúc suy luận theo nhiều cách: nó có thể dẫn tới suy luận hiệu quả hơn, vì LLM có thể cần tìm kiếm ít hơn hoặc ít mẫu sinh hơn để đi đến một lời giải đúng. Hơn nữa, chất lượng nền của các bước hay lời giải được sinh ra cao hơn. Do đó, một LLM được huấn luyện tốt có thể tổng quát hóa năng lực lập luận đã học sang các bài toán mới hiệu quả hơn so với một LLM chỉ dựa vào học trong ngữ cảnh (*in-context learning*) hoặc các sơ đồ suy luận không cần huấn luyện.

Mặt khác, so với các phương pháp không cần huấn luyện, các cách tiếp cận dựa trên huấn luyện cũng đặt ra những thách thức. Việc tạo ra các tập dữ liệu huấn luyện quy mô lớn, chất lượng cao cho lập luận có thể tốn kém và tốn nhiều công sức. Bản thân quá trình tinh chỉnh, đặc biệt với các LLM lớn nhất hoặc khi dùng học tăng cường, có thể đòi hỏi nhiều tính toán và công sức kỹ thuật đáng kể. Cũng có rủi ro mô hình bị quá khớp (*overfitting*) với các loại bài toán hay phong cách lập luận cụ thể có trong dữ liệu huấn luyện, có thể hạn chế hiệu năng của nó trên các nhiệm vụ ngoài phân phối (*out-of-distribution*).

11.4 Tóm tắt

Trong chương này, chúng ta đã bàn về suy luận cho LLM. Chúng ta đã trình bày khung điền trước–giải mã và các thuật toán giải mã liên quan cho suy luận LLM. Sau đó, chúng ta đã mô tả một số kỹ thuật suy luận hiệu quả. Chúng ta cũng đã bàn về co giãn lúc suy luận, một trong những cách tiếp cận quan trọng nhất để cải thiện khả năng lập luận của LLM.

Suy luận trên dữ liệu tuần tự từ lâu đã là một chủ đề nền tảng trong AI [Wozengraft và Reiffen, 1961; Viterbi, 1967; Forney, 1972]. Trong bối cảnh NLP, hướng nghiên cứu này có từ những ngày đầu tiên của nhận dạng tiếng nói (*speech recognition*) và dịch máy thống kê (*statistical machine translation*) [Koehn, 2010], khi các nhà nghiên cứu đối mặt với thách thức tìm kiếm hiệu quả trong những không gian giả thuyết khổng lồ để tìm ra chuỗi đầu ra có xác suất cao nhất. Các kỹ thuật như tìm kiếm chùm và nhiều chiến lược cắt tỉa khác nhau đã được phát triển từ thời đó để làm cho việc này khả thi về mặt tính toán. Khi ấy, các mô hình còn tương đối yếu, và phần lớn nghiên cứu tập trung vào phát triển các thuật toán tìm kiếm mạnh để giảm lỗi tìm kiếm. Những ý tưởng nền tảng này tiếp tục ảnh hưởng tới các cách tiếp cận hiện đại.

Khi bước vào kỷ nguyên do các phương pháp học sâu (*deep learning*) thống trị, các mô hình dựa trên mạng nơ-ron sâu đã trở nên cực kỳ mạnh mẽ. Ngay cả với những thuật toán tìm kiếm rất đơn giản, các mô hình này cũng có thể đạt kết quả xuất sắc. Trong bối cảnh đó, suy luận dường như không còn

"quan trọng" như trước, và sự chú ý của giới nghiên cứu dần chuyển sang kiến trúc mô hình, phương pháp huấn luyện và mở rộng quy mô mô hình.

Tuy nhiên, xu hướng này gần đây đã thay đổi. Với sự trỗi dậy của LLM, suy luận một lần nữa thu hút sự chú ý đáng kể. Sự quan tâm trở lại này chủ yếu thể hiện ở hai khía cạnh:

- Chi phí suy luận của LLM rất cao. Chẳng hạn, triển khai hiệu quả LLM trong các tình huống có nhiều yêu cầu đồng thời và đòi hỏi độ trễ thấp vẫn là một bài toán khó, khiến hiệu quả suy luận trở nên cực kỳ quan trọng. Trong bối cảnh này, các thiết kế kiến trúc hiệu quả, các thuật toán tìm kiếm được tối ưu và nhiều chiến lược tối ưu suy luận khác nhau có ý nghĩa thực tiễn đáng kể.
- Độ dài chuỗi đầu vào và đầu ra đã tăng đáng kể trong các nhiệm vụ phức tạp. Đặc biệt trong các nhiệm vụ như lập luận toán học, sự tăng trưởng độ dài chuỗi càng làm nổi bật tầm quan trọng của hiệu quả suy luận. Hơn nữa, việc co giãn quá trình suy luận gần đây đã được chứng tỏ là một cách hiệu quả để cải thiện năng lực lập luận của mô hình. Do đó, đạt được co giãn suy luận hiệu quả đang nổi lên như một hướng nghiên cứu đặc biệt hứa hẹn.

Suy luận giờ đây là một chủ đề rộng, bao gồm nhiều kỹ thuật. Nó không chỉ liên quan đến việc phát triển kiến trúc mô hình và thuật toán giải mã, mà ngày càng được định hình bởi những công việc kỹ thuật phức tạp và các tối ưu tinh vi ở cấp hệ thống cần thiết để triển khai LLM một cách hiệu quả và tiết kiệm. Nhiều kỹ thuật trong số này nằm ngoài phạm vi của NLP hay của một lĩnh vực AI cụ thể. Thay vào đó, biên giới của tối ưu suy luận LLM giờ đây đã mở rộng sâu vào những lĩnh vực vốn được coi là cốt lõi của khoa học và kỹ thuật máy tính.

Chương 12

Lập luận

Trong các chương trước, chúng ta đã xem xét LLM cho các ứng dụng đa dụng, trong đó mô hình sinh ra phản hồi dựa trên truy vấn đầu vào. Những mô hình này có thể xử lý một cách tự nhiên rất nhiều loại nhiệm vụ, chẳng hạn dịch máy và hỏi đáp. Vì vậy, nhiều bài toán NLP có thể được phát biểu thành các nhiệm vụ sinh văn bản và giải quyết bằng LLM. Vượt ra ngoài việc sinh văn bản đơn thuần, bước tiếp theo tự nhiên là phát triển những mô hình có khả năng suy nghĩ sâu hơn để giải các bài toán phức tạp hơn. Đặc biệt, ta mong muốn LLM có thể sinh ra các bước suy nghĩ trung gian và nhờ đó thể hiện khả năng lập luận (*reasoning*).

Lập luận từ lâu đã được coi là một trong những bài toán khó nhất của AI. Mặc dù cộng đồng NLP đã cố gắng giải quyết nó bằng các phương pháp như phân tích câu thành các dạng logic (*logical forms*), những nỗ lực này chủ yếu dựa vào việc xây dựng mô hình lập luận từ một chuỗi các mô-đun nối tiếp nhau theo kiểu dây chuyền (*pipeline*), vốn thường dễ bị lỗi lan truyền dây chuyền (*cascading errors*). Ngược lại, LLM cung cấp một khung tổng quát hơn cho lập luận, trong đó con đường suy nghĩ có thể được biểu đạt tường minh dưới dạng văn bản, và vì thế ta có thể cho LLM lập luận ngay trong quá trình sinh văn bản. Dù lập luận có thể được xem là một năng lực trỗi dậy (*emergent ability*) xuất hiện khi mở rộng quy mô LLM, việc khơi dậy năng lực này đòi hỏi những chiến lược cụ thể, bao gồm prompt chuỗi suy nghĩ (CoT), mở rộng quy mô lúc suy luận (*inference-time scaling*) và huấn luyện LLM để lập luận.

Trong chương này, chúng ta sẽ thảo luận các phương pháp cải thiện năng lực lập luận của LLM. Ta bắt đầu bằng việc giới thiệu các khái niệm cơ bản liên quan đến lập luận. Tiếp theo, ta trình bày các kỹ thuật để phát triển mô hình lập luận (*reasoning models*). Vì kỹ thuật prompt và các phương pháp CoT đã được bàn kỹ trong các chương trước, ở đây ta tập trung vào hai hướng nghiên cứu: 1) mở rộng lượng tính toán lúc suy luận (*inference-time compute*) và 2) học để lập luận (*learning to reason*). Cuối cùng, ta tổng kết chương bằng một số nhận xét.

Chú giải của người dịch: "Lập luận" và "suy luận" trong chương này

Trong chương này, *reasoning* được dịch là **lập luận**: quá trình đi từng bước từ dữ kiện đến kết luận, giống như khi bạn trình bày lời giải một bài toán. Còn *inference* được dịch là **suy luận**, với nghĩa kỹ thuật là *chạy mô hình đã huấn luyện để sinh ra đầu ra* (đối lập với huấn luyện). Do đó "lúc suy luận" (*inference time*) nghĩa là "lúc mô hình đang được dùng để trả lời", không phải lúc nó đang được huấn luyện.

12.1 Bối cảnh và các khái niệm

Ở đây chúng ta định nghĩa bài toán lập luận và thảo luận các khái niệm liên quan.

12.1.1 Lập luận là gì?

Trong lịch sử, nghiên cứu trí tuệ nhân tạo đã phân loại lập luận thành ba dạng logic chính: diễn dịch (*deduction*) (rút ra kết luận cụ thể từ các tiền đề đã biết, chẳng hạn các chứng minh toán học), quy nạp (*induction*) (suy ra các quy tắc hay khuôn mẫu tổng quát từ những ví dụ cụ thể) và giả định (suy ra lời giải thích tốt nhất) (*abduction*) (tìm lời giải thích hợp lý nhất cho một tập các quan sát) [Hartshorne và Weiss, 1931]. Trong một thời gian dài, dòng nghiên cứu chủ đạo của học máy và NLP chủ yếu tập trung vào các nhiệm vụ quy nạp. Chẳng hạn, các ứng dụng NLP kinh điển như phân loại văn bản và nhận dạng thực thể có tên có thể được xem là các nhiệm vụ quy nạp: mô hình được huấn luyện để khái quát hóa các hàm ánh xạ từ những tập dữ liệu lớn đã được gán nhãn.

Chú giải của người dịch: Ba kiểu lập luận qua ví dụ

- **Diễn dịch**: "Mọi số chia hết cho 6 đều chia hết cho 3; 72 chia hết cho 6; vậy 72 chia hết cho 3." Nếu tiền đề đúng thì kết luận *chắc chắn* đúng.
- **Quy nạp**: "Tôi thấy 1000 con quạ đều màu đen, vậy có lẽ mọi con quạ đều màu đen." Kết luận tổng quát rút ra từ các ví dụ, có thể sai. Học máy truyền thống chính là quy nạp: từ nhiều ví dụ có nhãn, mô hình học ra một quy luật chung.
- **Giả định** (*abduction*): "Sáng nay sân ướt; lời giải thích hợp lý nhất là đêm qua trời mưa." Ta chọn giả thuyết giải thích tốt nhất cho điều quan sát được, giống như bác sĩ chẩn đoán bệnh từ triệu chứng.

Tuy nhiên, cùng với sự phát triển của LLM, các nhà nghiên cứu đang cố gắng giải quyết những bài toán lập luận khó hơn, như toán học phức tạp, lập kế hoạch kiểu tác tử (*agentic planning*) và khám phá khoa học, vốn thường đòi hỏi lập luận diễn dịch và giả định chặt chẽ. Nghiên cứu LLM hiện đại đã cho thấy mục tiêu tiền huấn luyện tiêu chuẩn (tức là dự đoán token tiếp theo trên những kho ngữ liệu văn bản khổng lồ) đủ mạnh để giúp mô hình nội tại hóa tri thức về ngôn ngữ và thế giới, kể cả các năng lực lập luận tiềm ẩn. Tuy nhiên, thách thức chính là lập luận không được mô hình hóa một cách tường minh trong giai đoạn tiền huấn luyện. Kết quả là, khi gặp một bài toán phức tạp, LLM cố gắng đưa ra đáp án cuối cùng ngay lập tức, thay vì thực hiện một quá trình suy diễn logic từng bước.

Ví dụ, xét một bài toán đố đơn giản:

> Nếu tuổi của John gấp đôi tuổi của Mary, và 3 năm nữa Mary sẽ 10 tuổi, thì hiện nay John bao nhiêu tuổi?

Một LLM nền có thể đưa ra một phỏng đoán sai (ví dụ 20) chỉ đơn giản bằng cách liên kết các từ khóa "gấp đôi" và "10". Ngược lại, một LLM có năng lực lập luận có thể sinh ra chuỗi logic một cách tường minh trước khi trả lời:

> Hiện nay Mary $10 - 3 = 7$ tuổi. Do đó, John $7 \times 2 = 14$ tuổi.

Để giải các nhiệm vụ diễn dịch và giả định phức tạp bằng LLM, những năng lực lập luận tiềm ẩn này phải được kích hoạt một cách tường minh, dù là thông qua các kỹ thuật prompt như CoT hay thông qua các mô thức hậu huấn luyện (*post-training*) tiên tiến như học tăng cường.

Để hiểu rõ hơn bài toán lập luận, ta có thể nhìn nhận năng lực của LLM qua thuyết quá trình kép (*dual-process theory*) trong tâm lý học nhận thức: tư duy Hệ thống 1 và Hệ thống 2 [Kahneman, 2011].

- **Hệ thống 1 (tư duy trực giác).** Trong nhận thức của con người, Hệ thống 1 hoạt động tự động và nhanh chóng, hầu như không tốn công sức. Trong bối cảnh LLM, tư duy Hệ thống 1 tương ứng với mô thức trong đó mô hình đưa ra trực tiếp đáp án cuối cùng chỉ trong một lượt. Chế độ này

hiệu quả và nhìn chung đủ dùng cho các nhiệm vụ quy nạp, như hỏi đáp đơn giản. Tuy nhiên, tư duy Hệ thống 1 dễ mắc các bước nhảy logic và ảo giác khi đối mặt với những bài toán phức tạp nhiều bước.

- **Hệ thống 2 (tư duy cân nhắc).** Chế độ này chậm và có ý thức, đòi hỏi suy diễn logic từng bước. Với LLM, tư duy Hệ thống 2 được kích hoạt khi mô hình được chỉ dẫn phân bổ thêm tài nguyên tính toán lúc suy luận để sinh ra một cách tường minh các bước lập luận trung gian. Bằng cách biểu diễn quá trình suy nghĩ dưới dạng văn bản, LLM chia nhỏ các nhiệm vụ phức tạp, kiểm chứng các kết luận trung gian và duy trì một chuỗi logic mạch lạc trước khi đi đến đáp án cuối cùng.

Chú giải của người dịch: Hệ thống 1 và Hệ thống 2 trong đời thường

Khi được hỏi "$2 + 2$ bằng mấy?", bạn trả lời ngay "4" mà không cần nghĩ: đó là Hệ thống 1. Khi được hỏi "17×24 bằng mấy?", bạn phải đặt tính, nhân từng hàng, cộng lại ($17 \times 24 = 17 \times 20 + 17 \times 4 = 340 + 68 = 408$): đó là Hệ thống 2. LLM "trả lời ngay" giống như cố đoán 17×24 mà không đặt tính; LLM "lập luận" giống như viết ra từng dòng tính toán trên giấy nháp.

Đối với lập luận của LLM, mục tiêu cốt lõi là kích hoạt năng lực Hệ thống 2. Để hiểu điều này, ta phải nhìn xa hơn logic ký hiệu truyền thống. Trong AI cổ điển, lập luận thường được định nghĩa là việc áp dụng các quy tắc logic được lập trình cứng lên các cơ sở tri thức (*knowledge bases*) có cấu trúc. Tuy nhiên, với LLM, lập luận là một quá trình sinh văn bản nội tại. Nó có thể được định nghĩa là khả năng sinh ra một chuỗi mạch lạc các bước trung gian để đi đến đáp án cuối cùng. Theo một nghĩa nào đó, lập luận trong LLM là một cơ chế phân bổ động thêm lượng tính toán suy luận cho những bài toán khó hơn. Rốt cuộc, lập luận đối với một LLM là năng lực mã hóa tư duy Hệ thống 2 tiềm ẩn thành một chuỗi các bước suy nghĩ (*thinking steps*) (các bước lập luận (*reasoning steps*)).

12.1.2 Vì sao lập luận quan trọng

Lập luận quan trọng trong nghiên cứu NLP vì nó được ứng dụng trong nhiều lĩnh vực đầy thách thức:

- **Toán học.** Giải toán đòi hỏi logic chặt chẽ. Một mô hình không thể giải bài toán một cách đáng tin cậy chỉ bằng cách liên kết các con số và từ khóa trong câu hỏi với những đáp án đã thấy ở các ví dụ tương tự. Thay vào đó, nó cần áp dụng các quy tắc toán học từng bước để suy ra kết quả đúng.
- **Lập trình.** Viết phần mềm khó hơn nhiều so với việc chỉ dự đoán từ tiếp theo. Mô hình phải sinh mã dựa trên các yêu cầu cụ thể của người dùng và logic chặt chẽ của mã. Đặc biệt khi giải quyết những bài toán phức tạp, việc chuyển các yêu cầu trừu tượng thành mã chạy được đòi hỏi lập luận từng bước.
- **Lập kế hoạch kiểu tác tử.** Khi một LLM được dùng như một tác tử (*agent*) tự hành, nó phải chia các mục tiêu lớn, phức tạp thành những bước nhỏ, có thể thực hiện được. Nó cũng cần lên kế hoạch trước và điều chỉnh hành động nếu mọi việc không như ý.
- **Khám phá khoa học.** Hỗ trợ các nhà khoa học đòi hỏi phải xem xét dữ liệu, đề xuất các giả thuyết mới và kiểm chứng chúng một cách cẩn thận. Việc này dựa rất nhiều vào suy diễn và giả định logic từng bước.

Ngoài việc giải được những bài toán khó hơn, sinh ra các bước lập luận còn mang lại những lợi ích thực tiễn. Quan trọng nhất, nó làm cho mô hình trở nên diễn giải được (*interpretable*). Vì mô hình viết

ra quá trình suy nghĩ của nó dưới dạng văn bản, con người có thể đọc và kiểm tra logic. Họ thậm chí có thể tìm ra chính xác chỗ nào đã mắc lỗi. Điều này giúp hệ thống dễ kiểm chứng và dễ tin cậy hơn nhiều. Ngoài ra, lập luận từng bước giúp giảm ảo giác. Thay vì đoán bừa khi gặp câu hỏi khó, mô hình đặt đáp án cuối cùng của mình trên nền tảng các bước logic của chính nó, khiến đầu ra đáng tin cậy hơn nhiều.

12.1.3 Từ LLM đến mô hình lập luận

Trước khi LLM chiếm ưu thế, lập luận trong AI chủ yếu xoay quanh các quy tắc và logic chặt chẽ. Các nhà nghiên cứu dùng những phương pháp như AI ký hiệu (*symbolic AI*) và phân tích ngữ nghĩa (*semantic parsing*) để chuyển ngôn ngữ của con người thành các dạng mong muốn, chẳng hạn đồ thị tri thức (*knowledge graphs*) hay truy vấn cơ sở dữ liệu. Dù logic trong các hệ thống cổ điển này rất rõ ràng và dễ theo dõi, bản thân các hệ thống lại mong manh. Nếu gặp một từ chúng không biết, hoặc nếu một quy tắc nhỏ bị hỏng, toàn bộ quá trình sẽ sụp đổ.

Khi LLM xuất hiện, bài toán lập luận được phát biểu lại thành một nhiệm vụ sinh văn bản và có thể được giải quyết tốt hơn bằng các kỹ thuật hướng tới lập luận. Sự phát triển của mô hình lập luận trong kỷ nguyên LLM có thể chia thành ba giai đoạn: đoán thẳng đáp án, được prompt để suy nghĩ, và cuối cùng là học cách suy nghĩ.

Các mô hình ngôn ngữ thời kỳ đầu chủ yếu được huấn luyện để dự đoán từ tiếp theo. Khi được hỏi, chúng cố đưa ra đáp án ngay lập tức. Kiểu đoán nhanh này hoạt động tốt với những câu hỏi đơn giản hoặc các nhiệm vụ viết lách. Nhưng với các bài toán toán học hay logic phức tạp, nó không hiệu quả. Các mô hình thường mắc lỗi hoặc bị ảo giác vì chúng không có cách nào để dừng lại và giải bài toán từng bước.

Bước đột phá đầu tiên đến khi các nhà nghiên cứu nhận ra rằng LLM vốn đã sở hữu những năng lực lập luận ẩn giấu: chúng chỉ cần được đánh thức. Chỉ bằng cách dùng prompt CoT [Wei và cộng sự, 2022c] — yêu cầu mô hình "suy nghĩ từng bước" — mô hình buộc phải viết ra quá trình suy nghĩ trước khi đưa ra câu trả lời. Tuy nhiên, phương pháp này vẫn phụ thuộc nhiều vào gợi ý của con người và thực ra không dạy cho mô hình các quy tắc logic nền tảng.

Bước ngoặt lớn hơn gần đây là dùng học tăng cường để huấn luyện mô hình lập luận. Thay vì chỉ dạy mô hình sinh văn bản, các phương pháp học tăng cường huấn luyện những mô hình này thông qua thử và sai (*trial and error*). Khi mô hình tìm ra đường logic đúng và có được đáp án đúng, nó nhận được phần thưởng. Theo thời gian, các mô hình như o1 của OpenAI [OpenAI, 2024] và DeepSeek-R1 [DeepSeek, 2025] đã cho thấy những năng lực lập luận ấn tượng, chẳng hạn khám phá các ý tưởng khác nhau, tự kiểm tra lại các bước của mình và tự sửa lỗi.

Những năng lực tự sửa lỗi và khám phá này dẫn tới một hướng nghiên cứu quan trọng: mở rộng quy mô lúc suy luận [Snell và cộng sự, 2024]. Ý tưởng rất đơn giản: bằng cách phân bổ nhiều sức mạnh tính toán hơn trong quá trình sinh, mô hình có thể suy nghĩ sâu hơn để giải những bài toán khó hơn. Lý tưởng nhất, một mô hình sẽ điều chỉnh thời gian suy nghĩ dựa trên độ khó thực sự của bài toán. Tuy nhiên, nhiều mô hình lập luận hiện nay áp dụng một cách đơn giản hơn — chúng chỉ áp dụng một quá trình suy nghĩ dài cho mọi truy vấn.

Nhìn rộng hơn, việc mở rộng lượng tính toán lúc suy luận có lợi cho lập luận nói chung. Ngay cả khi một LLM chưa được huấn luyện riêng để lập luận, ta vẫn có thể tăng năng lực giải quyết vấn đề của nó chỉ bằng cách cho nó thêm tính toán để "suy nghĩ". Ví dụ, ta có thể yêu cầu mô hình sinh ra nhiều đường lập luận khác nhau rồi bỏ phiếu chọn đường tốt nhất, hoặc ta có thể dùng các thuật toán tìm kiếm để giúp nó tìm ra các bước logic đúng.

Mô hình lập luận cũng thúc đẩy sự phát triển gần đây của các tác tử dựa trên LLM [Yao và cộng

sự, 2023; Wang và cộng sự, 2024b]. Để một tác tử AI tự mình hoàn thành nhiệm vụ, nó cần lập luận để tìm ra cách đạt mục tiêu từng bước. Vì các nhiệm vụ trong thế giới thực hiếm khi diễn ra suôn sẻ hoàn hảo, tác tử còn phải nhận ra khi nào nó mắc lỗi và biết cách sửa. Tất cả những điều này đòi hỏi tác tử có năng lực lập luận tốt.

Sự phát triển nhanh chóng của các mô hình lập luận cũng mở ra những hướng nghiên cứu thú vị nhưng đầy thách thức [Li và cộng sự, 2025; Chen và cộng sự, 2026]. Một hướng tập trung vào hiệu quả. Chẳng hạn, nếu mô hình học được cách điều chỉnh động thời gian suy nghĩ, chúng sẽ không lãng phí tính toán cho những câu hỏi đơn giản mà chỉ "chậm lại" với những câu hỏi khó. Một hướng khác là mở rộng lập luận ra ngoài những lĩnh vực có đáp án đúng-sai rõ ràng như toán học hay lập trình, để xử lý các tình huống mở trong thế giới thực. Ngoài ra, việc giúp mô hình lập luận trên nhiều loại dữ liệu khác nhau, như hình ảnh, âm thanh và môi trường vật lý, là một bước then chốt hướng tới xây dựng các hệ thống AI tiên tiến hơn.

12.2 Lập luận thông qua prompt

Kỹ thuật prompt là phương pháp đơn giản nhất để khơi gợi lập luận từ LLM. Vì nó không đòi hỏi bất kỳ thay đổi nào về kiến trúc hay tham số của mô hình, ta chỉ cần dùng những văn bản đầu vào được soạn khéo léo để kích thích LLM đưa ra quá trình lập luận. Một trong những phương pháp prompt được dùng rộng rãi nhất là prompt chuỗi suy nghĩ (*chain-of-thought*) (CoT). Ý tưởng cốt lõi là khuyến khích mô hình sinh ra các bước suy diễn logic trung gian trước khi đưa ra đáp án cuối cùng. Điều này thường đạt được bằng cách cung cấp một vài ví dụ giải từng bước trong prompt, hoặc đơn giản là thêm một chỉ dẫn trực tiếp như "Hãy suy nghĩ từng bước" vào sau truy vấn. Dưới đây là một ví dụ về prompt CoT, trong đó chỉ dẫn CoT được tô sáng.

H: Tom mua 3 đôi giày, mỗi đôi \$40, và 2 chiếc áo sơ mi, mỗi chiếc \$20. Nếu anh ấy dùng phiếu giảm giá 10% cho toàn bộ đơn hàng, anh ấy phải trả tổng cộng bao nhiêu?
Đ: Hãy suy nghĩ từng bước.
Trước tiên, tính tổng tiền giày: 3 đôi nhân \$40 bằng \$120. Tiếp theo, tính tổng tiền áo: 2 chiếc nhân \$20 bằng \$40. Tổng tiền trước giảm giá là $\$120 + \$40 = \$160$. Khoản giảm giá là 10% của \$160, tức là \$16. Cuối cùng, trừ khoản giảm giá khỏi tổng tiền: $\$160 - \$16 = \$144$. Vậy đáp án là \$144.

Trong ví dụ này, LLM tạo ra một đường lập luận (*reasoning path*) (phần văn bản gạch dưới) chứa cả quá trình lập luận lẫn đáp án cuối cùng. Một cách hình thức hơn, cho một prompt CoT $\mathbf{x}$[1], LLM sinh ra đầu ra có khả năng cao nhất bằng cách cực đại hóa điểm mô hình hóa ngôn ngữ, như sau

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x}). \tag{12.1}$$

Ở đây $\Pr(\mathbf{y}|\mathbf{x})$ là xác suất sinh văn bản như trong các mô hình ngôn ngữ tiêu chuẩn. Mỗi đầu ra là một chuỗi token $\mathbf{y} = y_0 ... y_n$. Nó biểu diễn một đường lập luận gồm một chuỗi các bước lập luận:

$$\mathbf{y} = \{\bar{\mathbf{y}}_1, ..., \bar{\mathbf{y}}_{n_p}\}. \tag{12.2}$$

Ở đây n_p là số bước lập luận. Mỗi bước $\bar{\mathbf{y}}_i$ là một chuỗi con của $\mathbf{y}$, biểu diễn một bước suy diễn logic hoặc một phép tính trung gian. Lưu ý rằng có nhiều chiến lược để chia một đường lập luận thành các

[1] Ở đây một prompt CoT gồm cả đầu vào của người dùng lẫn chỉ dẫn CoT.

bước rời rạc, chẳng hạn tách theo từng phép toán. Để đơn giản, trong ví dụ này ta coi mỗi câu là một bước lập luận riêng. Do đó ta có

$\bar{\mathbf{y}}_1$: Trước tiên, tính tổng tiền giày: 3 đôi nhân \$40 bằng \$120.
$\bar{\mathbf{y}}_2$: Tiếp theo, tính tổng tiền áo: 2 chiếc nhân \$20 bằng \$40.
$\bar{\mathbf{y}}_3$: Tổng tiền trước giảm giá là $\$120 + \$40 = \$160$.
$\bar{\mathbf{y}}_4$: Khoản giảm giá là 10% của \$160, tức là \$16.
$\bar{\mathbf{y}}_5$: Cuối cùng, trừ khoản giảm giá khỏi tổng tiền: $\$160 - \$16 = \$144$.
$\bar{\mathbf{y}}_6$: Vậy đáp án là \$144.

Chú giải của người dịch: Đọc Phương trình (12.1) và (12.2)

$\Pr(\mathbf{y}|\mathbf{x})$ là xác suất mà mô hình gán cho việc "viết ra văn bản $\mathbf{y}$ khi đã đọc prompt $\mathbf{x}$". Ký hiệu $\arg\max_{\mathbf{y}}$ nghĩa là "chọn ra $\mathbf{y}$ làm cho đại lượng phía sau lớn nhất" (khác với max, vốn cho *giá trị* lớn nhất). Ví dụ, nếu chỉ có ba đầu ra khả dĩ với xác suất 0.2, 0.5, 0.3 thì max là 0.5, còn arg max là chính đầu ra thứ hai. Phương trình (12.2) chỉ nói rằng chuỗi token dài $\mathbf{y}$ có thể được cắt thành n_p khúc liền nhau, mỗi khúc là một "dòng" của lời giải; ở ví dụ trên $n_p = 6$.

Với một đường lập luận như vậy, ta có thể trích xuất đáp án từ bước cuối cùng của nó. Có nhiều cách để cải thiện cách tiếp cận lập luận đơn giản dựa trên CoT này. Chẳng hạn, các kỹ thuật như tự nhất quán (*self-consistency*) lấy mẫu nhiều đường lập luận rồi bỏ phiếu theo đa số để xác định đáp án đáng tin cậy nhất [Wang và cộng sự, 2023c]. Các chiến lược khác bao gồm phân rã bài toán (*problem decomposition*) để chia các truy vấn phức tạp thành những nhiệm vụ con dễ xử lý [Zhou và cộng sự, 2023b], các phương pháp tự phản tư (*self-reflection*) prompt mô hình tự phê bình và sửa các lỗi logic của chính nó [Shinn và cộng sự, 2023; Madaan và cộng sự, 2024], và các khung lập luận cân nhắc (*deliberate reasoning*) khám phá hiệu quả nhiều đường lập luận rẽ nhánh [Yao và cộng sự, 2024].

Vì nhiều kỹ thuật prompt dựa trên CoT đã được trình bày trước đây, chúng tôi sẽ không đi vào chi tiết ở đây. Bạn đọc được khuyến khích xem lại các kỹ thuật đã đề cập trong Chương 9.

12.3 Lập luận lúc suy luận

Lập luận lúc suy luận (*inference-time reasoning*) liên quan chặt chẽ đến việc mở rộng quy mô trong quá trình suy luận, vốn đã được giới thiệu ở Chương 11. Trong mục này, chúng ta mở rộng thảo luận trước đó và tập trung hơn vào việc cải thiện hiệu năng lập luận của LLM.

12.3.1 Mở rộng quy mô tính toán lúc suy luận

Thành công của LLM phụ thuộc rất nhiều vào việc tăng quy mô sức mạnh tính toán. Một cách phổ biến để cải thiện các mô hình này là mở rộng quy mô lúc huấn luyện (*training-time scaling*). Chỉ đơn giản bằng cách dùng nhiều dữ liệu huấn luyện hơn, kích thước mô hình lớn hơn và nhiều tính toán hơn, các nhà nghiên cứu đã xây dựng được những LLM rất mạnh [Kaplan và cộng sự, 2020; Hoffmann và cộng sự, 2022].

Tuy nhiên, chỉ mở rộng quy mô quá trình huấn luyện là chưa đủ cho các nhiệm vụ lập luận phức tạp. Không giống việc sinh văn bản đơn giản, lập luận cần logic và lập kế hoạch từng bước. Không thể cho mô hình xem mọi đường lập luận khả dĩ trong quá trình huấn luyện. Nếu chỉ dựa vào huấn luyện để giải các bài toán toán học hay logic khó, ta sẽ cần một lượng dữ liệu rất lớn. Lý tưởng nhất, ta muốn

mô hình dùng những bước logic đơn giản để tự xây dựng đường lập luận của riêng nó ngay khi đang giải. Vì vậy, việc kích hoạt các năng lực lập luận như thế trong lúc suy luận hấp dẫn hơn.

Bảng 12.1: So sánh mở rộng quy mô lúc huấn luyện và lúc suy luận

Đặc điểm	Mở rộng quy mô lúc huấn luyện	Mở rộng quy mô lúc suy luận
Cách hoạt động	Làm cho mô hình lớn hơn và nạp cho nó nhiều dữ liệu hơn trong giai đoạn học ban đầu.	Giữ nguyên kích thước mô hình, nhưng cho nó thêm tính toán để "suy nghĩ" (ví dụ: khám phá các đường khác nhau) trước khi trả lời.
Chi phí huấn luyện	Cực kỳ cao. Rất tốn kém và khó sửa nếu có gì đó sai.	Thấp đến trung bình. Có thể phải tốn chi phí huấn luyện thêm các mô hình nhỏ hơn để kiểm tra câu trả lời của nó.
Chi phí sử dụng	Dự đoán được. Chi phí sinh một token chủ yếu phụ thuộc vào kích thước mô hình và độ dài ngữ cảnh.	Linh hoạt nhưng có thể trở nên rất cao. Mô hình "suy nghĩ" càng sâu thì càng tốn kém.
Yêu cầu phần cứng (suy luận)	Cần băng thông bộ nhớ (*memory bandwidth*) cao để đọc và ghi dữ liệu nhanh.	Cần băng thông cao như vậy, cộng thêm dung lượng bộ nhớ lớn để khám phá nhiều đường lập luận dài.
Dữ liệu cần thiết	Lượng khổng lồ văn bản tổng quát.	Các ví dụ chất lượng cao, giải từng bước (như chứng minh toán học) và một cách rõ ràng để chấm xem một đáp án đúng hay sai.

Vì lý do này, một ý tưởng tự nhiên là mở rộng quy mô tính toán trong giai đoạn suy luận. Nhờ vậy, LLM có thể dành nhiều thời gian hơn để khám phá các bước lập luận khác nhau trước khi đưa ra đáp án cuối cùng. Lợi ích rõ ràng của cách tiếp cận này là nó cho phép sử dụng tài nguyên tính toán linh hoạt và hiệu quả hơn nhiều. Chẳng hạn, với những bài toán toán học hay logic phức tạp, LLM có thể phân bổ nhiều thời gian và sức mạnh hơn, và nhờ đó "suy nghĩ" lâu hơn. Trong nhiều trường hợp, một mô hình nhỏ hơn được cho thêm thời gian lập luận lúc suy luận thậm chí có thể vượt trội một mô hình lớn hơn nhiều nhưng bị buộc phải trả lời ngay lập tức. Chúng tôi tóm tắt sự khác biệt giữa mở rộng quy mô lúc huấn luyện và mở rộng quy mô lúc suy luận trong Bảng 12.1.

Ta có thể xem mở rộng quy mô lúc suy luận như một bài toán **mở rộng quy mô tìm kiếm** (*search scaling*). Khi cho mô hình thêm tính toán để "suy nghĩ", về bản chất ta đang cho phép nó khám phá một không gian các bước logic khả dĩ rộng hơn, và có cơ hội cao hơn nhiều để tìm ra lời giải tối ưu. Điều này dẫn đến ba câu hỏi then chốt:

- Làm thế nào để sinh ra các đường lập luận khác nhau?
- Làm thế nào để tìm kiếm qua các bước khả dĩ để tìm ra bước tốt nhất?
- Làm thế nào để kiểm tra xem các bước trung gian và đáp án cuối cùng có đúng không?

Trong các tiểu mục tiếp theo, chúng ta sẽ giải quyết những câu hỏi này bằng cách thảo luận ba chủ đề: lấy mẫu nhiều đường lập luận (Mục 12.3.2), các chiến lược tìm kiếm cho lập luận (Mục 12.3.3) và kiểm chứng (Mục 12.3.4).

12.3.2 Lấy mẫu nhiều đường lập luận

Cách trực tiếp nhất để mở rộng quy mô tính toán lúc suy luận là sinh ra không chỉ một mà nhiều đường lập luận cho một đầu vào duy nhất. Bằng cách sinh ra nhiều đường khác nhau, ta cho LLM nhiều cơ hội hơn để tìm ra logic đúng và đáp án tương ứng.

Để làm được điều này, thông lệ phổ biến là đưa tính ngẫu nhiên vào quá trình sinh văn bản. Một phương pháp điển hình là dùng lấy mẫu với nhiệt độ (*temperature sampling*), kết hợp với lấy mẫu top-k hoặc lấy mẫu hạt nhân (*nucleus sampling*) (top-p). Trong sinh tự hồi quy (*auto-regressive generation*) tiêu chuẩn, tại mỗi vị trí i, LLM đưa ra một phân phối xác suất $\Pr(\cdot|\mathbf{x}, \mathbf{y}_{<i})$ trên bộ từ vựng V cho token tiếp theo y_i dựa trên các logit (*logits*) u_{y_i} của nó. Lấy mẫu với nhiệt độ thay đổi phân phối này bằng cách chia các logit cho một tham số nhiệt độ $T > 0$ trước khi áp dụng hàm Softmax:

$$\Pr(y_i|\mathbf{x}, \mathbf{y}_{<i}) \;=\; \frac{\exp(u_{y_i}/T)}{\sum_{y'_i \in V} \exp(u_{y'_i}/T)}. \tag{12.3}$$

Nhiệt độ cao hơn làm phân phối "phẳng" hơn, nhờ đó khuyến khích mô hình khám phá nhiều kiểu bước lập luận đa dạng hơn. Tuy nhiên, tính ngẫu nhiên không bị ràng buộc có thể dẫn đến văn bản suy biến hoặc phi logic do thỉnh thoảng lấy mẫu từ "đuôi dài" gồm các token có xác suất thấp. Để duy trì chất lượng của các đường được sinh ra, lấy mẫu với nhiệt độ thường được kết hợp với các phương pháp cắt bớt bộ từ vựng:

- **Lấy mẫu top-k** giới hạn tập lấy mẫu chỉ còn k token tiếp theo có khả năng cao nhất, rồi phân phối lại khối lượng xác suất giữa chúng.
- **Lấy mẫu hạt nhân (top-p)** dùng chiến lược cắt bớt động. Nó chọn tập nhỏ nhất gồm các token xếp hạng cao nhất có xác suất tích lũy vượt quá một ngưỡng p.

Chú giải của người dịch: Nhiệt độ, top-k và top-p bằng số cụ thể

Giả sử chỉ có 3 token với logit $u = (2,\ 1,\ 0)$. Với $T = 1$: $e^2 \approx 7.39$, $e^1 \approx 2.72$, $e^0 = 1$, tổng ≈ 11.11, nên xác suất $\approx (0.67,\ 0.24,\ 0.09)$. Với $T = 2$ ta chia logit cho 2 được $(1,\ 0.5,\ 0)$: $e^1 \approx 2.72$, $e^{0.5} \approx 1.65$, 1, tổng ≈ 5.37, nên xác suất $\approx (0.51,\ 0.31,\ 0.19)$ — "phẳng" hơn, các token ít khả năng được chọn thường hơn. Ngược lại T nhỏ (gần 0) làm phân phối dồn gần hết vào token đứng đầu. Với phân phối $(0.67, 0.24, 0.09)$: top-k với $k = 2$ chỉ giữ hai token đầu và chia lại thành $(0.67, 0.24)/0.91 \approx (0.74, 0.26)$; top-$p$ với $p = 0.8$ cũng giữ hai token đầu vì $0.67 < 0.8$ nhưng $0.67 + 0.24 = 0.91 > 0.8$.

Bằng cách áp dụng các chiến lược giải mã ngẫu nhiên này, cho một đầu vào $\mathbf{x}$, LLM có thể lấy mẫu một tập các chuỗi đầu ra đa dạng (tức là các đường lập luận ứng viên):

$$\mathcal{Y}^{\text{top}} \;=\; \{\hat{\mathbf{y}}^1, \ldots, \hat{\mathbf{y}}^N\}. \tag{12.4}$$

Ở đây N biểu diễn ngân sách tính toán (*compute budget*) lúc suy luận. N càng lớn thì ta càng tốn nhiều tính toán để khám phá các đường lập luận khác nhau. Khi đã có tập đường đa dạng này, bài toán tiếp theo là chọn ra đường tốt nhất. Có hai phương pháp được dùng rộng rãi cho việc này: Best-of-N và tự nhất quán.

1. Best-of-N (BoN)

Trước đây chúng ta đã giới thiệu lấy mẫu Best-of-N ở Chương 10 như một phương pháp căn chỉnh lúc suy luận (*inference-time alignment*). Trong bối cảnh đó, ta dùng một mô hình phần thưởng $r(\mathbf{x}, \hat{\mathbf{y}})$ để chọn đầu ra phù hợp nhất với sở thích của con người.

Với các nhiệm vụ lập luận, khái niệm lấy mẫu BoN vẫn giữ nguyên, nhưng mục tiêu thay đổi. Thay vì một mô hình phần thưởng theo sở thích, "giám khảo" thường là một bộ kiểm chứng (*verifier*) (mà ta có thể ký hiệu là một hàm chấm điểm tổng quát v) được thiết kế để đánh giá mức độ chặt chẽ về logic của một đường lập luận. Bộ kiểm chứng gán một điểm số $v(\mathbf{x}, \hat{\mathbf{y}})$ cho mỗi đường được lấy mẫu, và ta chọn đường lập luận tốt nhất $\hat{\mathbf{y}}_{\text{best}}$ làm cực đại điểm số đó

$$\hat{\mathbf{y}}_{\text{best}} = \underset{\hat{\mathbf{y}} \in \{\hat{\mathbf{y}}^1, \ldots, \hat{\mathbf{y}}^N\}}{\arg\max} v(\mathbf{x}, \hat{\mathbf{y}}). \tag{12.5}$$

Đáp án cuối cùng sau đó được trích xuất từ đường có điểm cao nhất này. Để hiểu quá trình BoN, hãy xét một phép so sánh trực quan. Hãy tưởng tượng có một lớp học gồm N học sinh đang giải một bài toán. Một giáo viên (bộ kiểm chứng) chấm tất cả giấy nháp của các em, rồi chọn ra học sinh có điểm cao nhất.

Dù lấy mẫu BoN rất mạnh, hiệu quả của nó phụ thuộc vào chất lượng của bộ kiểm chứng. Nếu bộ kiểm chứng yếu, nó có thể dễ dàng bị đánh lừa bởi những lập luận sai mà tình cờ đi đến đáp án cuối cùng đúng (tức là khai thác lỗ hổng phần thưởng (*reward hacking*)). Việc thiết kế bộ kiểm chứng sẽ được bàn chi tiết hơn ở Mục 12.3.4.

2. Tự nhất quán

Một cách tiếp cận thay thế cho việc chọn đường tốt nhất bằng một bộ kiểm chứng được học là tự nhất quán. Tự nhất quán dựa trên trực giác rằng trong các nhiệm vụ lập luận phức tạp, thường có nhiều đường logic khác nhau cùng dẫn đến một đáp án đúng, trong khi các đường lập luận sai có xu hướng đa dạng và dẫn đến những lỗi ngẫu nhiên. Thay vì chấm điểm quá trình lập luận, tự nhất quán trực tiếp lấy biên (*marginalize out*) các đường lập luận bằng cách bỏ phiếu theo đa số trên các đáp án cuối cùng. Một cách hình thức, gọi A là tập tất cả các đáp án cuối cùng khả dĩ. Với mỗi đường được lấy mẫu $\hat{\mathbf{y}}^k$, ta trích xuất đáp án dự đoán cuối cùng của nó $\hat{a}^k = \text{Extract}(\hat{\mathbf{y}}^k)$. Chiến lược tự nhất quán chọn đáp án $\hat{a}_{\text{sc}}$ xuất hiện nhiều nhất trong N đường được lấy mẫu:

$$\hat{a}_{\text{sc}} = \underset{a \in A}{\arg\max} \sum_{k=1}^{N} \mathbf{1}(\text{Extract}(\hat{\mathbf{y}}^k) = a), \tag{12.6}$$

trong đó $\mathbf{1}(\cdot)$ là hàm chỉ thị (*indicator function*).

Chú giải của người dịch: BoN và tự nhất quán qua một ví dụ

Giả sử $N = 5$ đường lập luận cho bài toán của Tom cho các đáp án: \$144, \$160, \$144, \$144, \$136. Hàm chỉ thị $\mathbf{1}$(mệnh đề) bằng 1 nếu mệnh đề đúng và bằng 0 nếu sai; nên tổng trong Phương trình (12.6) chính là *số phiếu* của mỗi đáp án: \$144 được 3 phiếu, \$160 và \$136 mỗi đáp án 1 phiếu. Tự nhất quán chọn \$144. Còn BoN không đếm phiếu mà nhờ bộ kiểm chứng chấm điểm cả 5 lời giải (chẳng hạn $0.9, 0.4, 0.7, 0.8, 0.3$) và lấy lời giải có điểm cao nhất (lời giải thứ nhất).

3. Cân bằng giữa độ đa dạng và chất lượng khi lấy mẫu

Khi lấy mẫu nhiều đường lập luận để mở rộng quy mô tính toán lúc suy luận, ta cần tìm sự đánh đổi giữa độ đa dạng và chất lượng. Trong các nhiệm vụ lập luận, ta thường mong muốn một tập đường lấy mẫu đa dạng, điều này làm tăng khả năng tập đó chứa logic đúng. Một mẹo kinh nghiệm phổ biến là tăng nhiệt độ lấy mẫu. Bằng cách làm phân phối phẳng hơn, quá trình lấy mẫu tránh chỉ tập trung vào một vài đường có xác suất cao. Kết quả là các đường có xác suất thấp hơn có nhiều khả năng được chọn hơn, và mô hình có thể khám phá nhiều kiểu đường lập luận đa dạng hơn trong N mẫu. Tuy nhiên, khi

nhiệt độ tăng, dù độ đa dạng của các mẫu được cải thiện, chất lượng logic trung bình của từng đường có thể giảm.

Để đạt được sự cân bằng giữa độ đa dạng và chất lượng, một phương pháp đơn giản là sửa đổi mục tiêu tìm kiếm trong giai đoạn suy luận bằng cách đưa vào một số hạng phạt hoặc thưởng tường minh. Trong thiết lập này, độ đa dạng có thể được mô hình hóa toán học như một hàm được kết hợp với xác suất sinh ban đầu. Một cách hình thức hơn, mục tiêu tìm kiếm đã sửa đổi có thể được viết là

$$\text{score}(\mathbf{x}, \mathbf{y}) \;=\; \log \Pr(\mathbf{y}|\mathbf{x}) + \lambda D(\mathbf{y}, \mathcal{Y}^{\text{top}}). \tag{12.7}$$

Ở đây $\log \Pr(\mathbf{y}|\mathbf{x})$ ưu tiên các đầu ra có xác suất cao hơn, còn $D(\mathbf{y}, \mathcal{Y}^{\text{top}})$ đóng vai trò số hạng phạt hoặc thưởng nhằm khuyến khích sự đa dạng so với các đường đã lấy mẫu $\mathcal{Y}^{\text{top}}$. Siêu tham số λ điều khiển mức độ của sự đánh đổi này. Việc thiết kế $D(\cdot)$ có thể khá linh hoạt. Chẳng hạn, một hàm phạt đơn giản có thể giảm điểm của các token hoặc n-gram đã xuất hiện thường xuyên trong $\mathcal{Y}^{\text{top}}$. Ngược lại, một hàm thưởng có thể được thiết kế để khen thưởng những khác biệt rõ rệt về ngữ nghĩa. Nhờ vậy, LLM có thể tạo ra những cấu trúc lập luận mới thay vì chỉ diễn đạt lại chút ít cùng một ý.

Nhìn từ góc độ rộng hơn, thế lưỡng nan khi lấy mẫu này cũng liên quan đến sự đánh đổi giữa khám phá và khai thác (*exploration–exploitation trade-off*) được nghiên cứu trong học tăng cường [Sutton và Barto, 2018]. Trong tinh chỉnh dựa trên RL, một LLM có thể được xem là một chính sách π_θ sinh ra đường lập luận $\mathbf{y}$ cho một câu hỏi $\mathbf{x}$. Một mục tiêu có chính quy hóa tổng quát có thể được viết là

$$\max_{\pi_\theta} \; \mathbb{E}_{\mathbf{y}\sim\pi_\theta}\left[R(\mathbf{x}, \mathbf{y})\right] - \beta D_{\text{KL}}\left(\pi_\theta(\cdot|\mathbf{x}) \,\|\, \pi_{\text{ref}}(\cdot|\mathbf{x})\right) + \alpha\, \text{Entropy}\left(\pi_\theta(\cdot|\mathbf{x})\right), \tag{12.8}$$

trong đó $R(\mathbf{x}, \mathbf{y})$ đo chất lượng của đường lập luận được sinh ra. Số hạng độ phân kỳ KL (*KL-divergence*) ràng buộc chính sách được học phải luôn gần với chính sách tham chiếu (*reference policy*). Mục đích chính của nó là ngăn chính sách bị trôi dạt quá mức, và nhờ đó giảm các hành vi không mong muốn như khai thác lỗ hổng phần thưởng hay suy giảm chất lượng ngôn ngữ. Ngược lại, số hạng entropy khuyến khích tường minh chính sách duy trì một phân phối ít tập trung hơn, và vì thế thúc đẩy việc khám phá trong quá trình huấn luyện.

Chú giải của người dịch: Đọc Phương trình (12.8) theo từng số hạng

Có ba "lực" kéo chính sách π_θ (tức là LLM với tham số θ): (1) $\mathbb{E}_{\mathbf{y}\sim\pi_\theta}[R]$ là *phần thưởng trung bình* khi để mô hình tự sinh lời giải — càng lớn càng tốt; (2) D_{KL} đo mức "khác biệt" giữa phân phối của mô hình mới và mô hình tham chiếu ban đầu (bằng 0 khi hai phân phối trùng nhau); dấu trừ và hệ số β phạt việc đi quá xa mô hình gốc, giống như sợi dây níu; (3) Entropy đo độ "trải rộng" của phân phối: một phân phối dồn hết vào một lựa chọn có entropy bằng 0, còn phân phối đều có entropy lớn nhất; dấu cộng và hệ số α thưởng cho việc giữ nhiều lựa chọn, tức là khuyến khích khám phá. Các hệ số $\beta, \alpha \geq 0$ quyết định lực nào mạnh hơn.

Có một mối liên hệ về mặt khái niệm giữa mục tiêu RL và sự đánh đổi giữa đa dạng và chất lượng lúc suy luận, dù hai mục tiêu hoạt động trên những không gian khác nhau. Lúc suy luận, thủ tục tìm kiếm điều khiển mức độ rộng mà nó khám phá các đường lập luận do một mô hình cố định sinh ra. Trong tinh chỉnh dựa trên RL, chính quy hóa KL điều khiển việc chính sách được học được phép đi xa chính sách tham chiếu đến mức nào. Hệ số KL lớn hơn giữ chính sách gần mô hình tham chiếu, trong khi hệ số nhỏ hơn cho phép quá trình tối ưu hóa khám phá một vùng rộng hơn của không gian chính sách để tìm những hành vi có phần thưởng cao hơn. Những thay đổi chính sách như vậy có thể giúp mô hình sinh ra các đường lập luận ít có khả năng xuất hiện dưới chính sách tham chiếu, và vì thế có thể dẫn đến một dải đầu ra rộng hơn.

Cả tìm kiếm lúc suy luận lẫn RL có chính quy hóa KL đều đòi hỏi một sự cân bằng giữa hành vi thận trọng và khám phá rộng hơn. Điểm khác biệt là các phương pháp lúc suy luận khám phá các đầu

ra khác nhau dưới một mô hình cố định, còn RL tìm kiếm trong không gian chính sách bằng cách cập nhật tham số mô hình. Trong cả hai trường hợp, cho phép khám phá rộng hơn có thể phát hiện ra những hành vi lập luận tốt hơn và đa dạng hơn, nhưng khám phá quá mức có thể làm giảm chất lượng hoặc độ ổn định.

12.3.3 Các chiến lược tìm kiếm cho lập luận

Trong thực tế, việc tìm đường lập luận tối ưu có thể được phát biểu trực tiếp thành một bài toán tìm kiếm kinh điển: ta bắt đầu từ một trạng thái ban đầu, thực hiện một hành động để sinh ra một bước lập luận, mở rộng trạng thái hiện tại thành một tập trạng thái mới, và lặp lại quá trình này cho đến khi một điều kiện dừng được thỏa mãn. Vì chúng ta đã giới thiệu một số thuật toán tìm kiếm kinh điển (như tìm kiếm cây và tìm kiếm chùm) ở Chương 5 và Chương 11, ta sẽ không nhắc lại chi tiết các thuật toán này ở đây. Thay vào đó, ta tập trung vào cách ánh xạ việc tìm kiếm đường lập luận vào các khung tìm kiếm kinh điển này.

Để áp dụng một thuật toán tìm kiếm vào lập luận của LLM, ta cần định nghĩa các thành phần tiêu chuẩn của một bài toán tìm kiếm:

- **Trạng thái tìm kiếm.** Với lập luận của LLM, trạng thái là ngữ cảnh mà mô hình đã xây dựng được cho đến lúc đó. Cụ thể, tại bước t, trạng thái s_t gồm truy vấn gốc $\mathbf{x}$ kết hợp với đường lập luận dở dang đã sinh ra đến thời điểm này: $s_t = (\mathbf{x}, \bar{\mathbf{y}}_1, \ldots, \bar{\mathbf{y}}_t)$.
- **Hành động.** Một hành động là việc sinh ra bước lập luận tiếp theo (một đoạn văn bản) $\bar{\mathbf{y}}_{t+1}$.
- **Mô hình chuyển trạng thái.** Thực hiện một hành động đơn giản là nối đoạn văn bản mới vào trạng thái hiện có. Điều này tạo ra trạng thái tiếp theo: $s_{t+1} = (s_t, \bar{\mathbf{y}}_{t+1}) = (\mathbf{x}, \bar{\mathbf{y}}_1, \ldots, \bar{\mathbf{y}}_t, \bar{\mathbf{y}}_{t+1})$.
- **Hàm đánh giá trạng thái.** Đây là một hàm chấm điểm đánh giá mức độ đúng đắn về logic của trạng thái hiện tại trên đường đến đáp án cuối cùng. Ví dụ, để đánh giá các trạng thái tìm kiếm, ta có thể dùng các heuristic (*heuristics*) được lập trình cứng, hoặc một bộ kiểm chứng nơ-ron đã được huấn luyện.
- **Điều kiện dừng.** Tiêu chí để dừng tìm kiếm. Điều này xảy ra khi mô hình sinh ra tường minh một chuỗi đáp án cuối cùng, hoặc khi quá trình tìm kiếm chạm tới một ngân sách tính toán định trước (ví dụ: độ sâu tối đa của các nút được khám phá).

Chú giải của người dịch: Tìm kiếm đường lập luận giống như đi trong mê cung

Hãy hình dung mỗi trạng thái là "đoạn lời giải đã viết đến giờ". Từ một trạng thái, mô hình có thể viết tiếp nhiều câu khác nhau (nhiều hành động), mỗi câu dẫn đến một trạng thái mới — giống như đứng ở ngã rẽ trong mê cung. Nếu mỗi bước có 3 lựa chọn và lời giải dài 4 bước thì có $3^4 = 81$ đường khả dĩ; hàm đánh giá trạng thái giống như một người dẫn đường cho biết ngã rẽ nào "có vẻ" gần lối ra, giúp ta không phải thử hết tất cả các đường.

Khi nhiệm vụ lập luận đã được phát biểu thành một bài toán tìm kiếm, có thể áp dụng nhiều thuật toán tìm kiếm khác nhau để khám phá không gian lập luận một cách hiệu quả. Chẳng hạn, các phương pháp tìm kiếm cây đơn giản, như tìm kiếm theo chiều rộng (*breadth-first search*) và tìm kiếm theo chiều sâu (*depth-first search*), có thể được dùng để mô hình hóa quá trình lập luận của LLM. Trong các phương pháp này, mô hình khám phá nhiều ý nghĩ ứng viên ở mỗi bước và đánh giá chúng để quyết định nên đi tiếp theo một đường hay quay lui (*backtrack*) khi gặp ngõ cụt. Với các nhiệm vụ có độ bất định cao hơn hoặc phần thưởng đến muộn, tìm kiếm cây Monte Carlo (*Monte Carlo tree search*) thường

được sử dụng. Phương pháp này giúp mô hình cân bằng giữa khám phá các đường lập luận mới và khai thác các bước đã biết. Đôi khi nó còn tích hợp phản hồi từ môi trường bên ngoài và cơ chế tự phản tư để tinh chỉnh quá trình tìm kiếm [Yao và cộng sự, 2024; Hao và cộng sự, 2023; Zhou và cộng sự, 2024].

Để giải quyết sự dư thừa của cấu trúc cây, khi nhiều đường lập luận khác nhau có thể hội tụ về cùng một kết luận trung gian, một số phương pháp mô hình hóa không gian lập luận như một đồ thị (chẳng hạn đồ thị có hướng không chu trình (*directed acyclic graph*), DAG) [Zhang và cộng sự, 2025]. Bằng cách này, mô hình có thể xây dựng các cấu trúc không gian tìm kiếm hiệu quả hơn, và nắm bắt được các phụ thuộc lẫn nhau phi tuyến của suy diễn logic. Ngoài các thuật toán tìm kiếm bên ngoài, cũng có thể nội tại hóa động lực của quá trình tìm kiếm trực tiếp vào tham số của LLM. Ví dụ, bằng cách huấn luyện LLM trên các vết thực thi của những thuật toán tìm kiếm heuristic như A*, chúng có thể học cách sinh ra một cách ngầm định quá trình tìm kiếm và các kế hoạch tối ưu mà không cần dựa vào một kịch bản điều khiển bên ngoài [Lehnert và cộng sự, 2025].

Dù mở rộng quy mô tính toán lúc suy luận thông qua tìm kiếm là một chiến lược mạnh, mối quan hệ giữa công sức tìm kiếm và hiệu năng lập luận không đơn giản là tuyến tính. Theo trực giác, ta có thể kỳ vọng rằng cho mô hình nhiều thời gian khám phá hơn sẽ luôn mang lại câu trả lời tốt hơn. Tuy nhiên, trong thực tế, mức tăng hiệu năng nhanh chóng chững lại. Một lý do chính là giới hạn năng lực vốn có của LLM: nếu mô hình thiếu tri thức để giải một bài toán, dù tìm kiếm bao nhiêu cũng không thể tạo ra logic đúng từ hư không. Ngoài ra, khi công sức tìm kiếm tăng lên, độ đa dạng của các đường được sinh ra giảm xuống, nghĩa là mô hình lãng phí tính toán vào việc khám phá những ý tưởng trùng lặp, ít mang lại giá trị mới.

12.3.4 Kiểm chứng

Tiếp theo, chúng ta thảo luận bài toán kiểm chứng trong lập luận. Lưu ý rằng dù trong mục này ta tập trung vào các phương pháp lúc suy luận, nhiều bộ kiểm chứng dùng trong mô hình lập luận vẫn cần được huấn luyện. Vì vậy, chúng tôi cũng trình bày một số chi tiết về việc huấn luyện các mô hình này trong phần thảo luận dưới đây.

1. Mô hình phần thưởng theo quá trình

Có hai mô thức chính để kiểm chứng kết quả lập luận: mô hình phần thưởng theo kết quả (*outcome reward models*) (ORM) và mô hình phần thưởng theo quá trình (*process reward models*) (PRM) [Uesato và cộng sự, 2022; Lightman và cộng sự, 2024]. Một ORM chỉ đánh giá đáp án cuối cùng của một đường lập luận. Dù dễ huấn luyện hơn, ORM thường không đủ cho các nhiệm vụ lập luận vì chúng gặp vấn đề "dương tính giả (*false positives*)". Để minh họa, xét một bài toán đơn giản:

> Tìm x: $2x + 4 = 10$.

Một mô hình có thể sinh ra đường lập luận sau:

> $2x + 4 = 10$ nghĩa là x phải bằng một nửa của 10, tức là 5.
> Nhưng $5 - 2 = 3$.
> Do đó, $x = 3$.

Mặc dù đáp án cuối cùng (tức là 3) đúng, các bước logic hoàn toàn sai. Nếu chỉ dùng ORM, đường này có thể nhận điểm cao và vô tình củng cố logic sai. Để giải quyết vấn đề này, ta có thể dùng PRM để đánh giá tính đúng đắn của từng bước riêng lẻ trong đường lập luận. Cho một đường lập luận $\mathbf{y} = \{\bar{\mathbf{y}}_1, \ldots, \bar{\mathbf{y}}_{n_p}\}$, một PRM gán một điểm đúng đắn r_t cho bước hiện tại $\bar{\mathbf{y}}_t$, với điều kiện là đầu vào $\mathbf{x}$ và

tất cả các bước trước đó $\bar{\mathbf{y}}_{<t}$. Ta có thể định nghĩa hàm kiểm chứng ở mức bước là

$$r_t \;=\; v_{\text{prm}}(\bar{\mathbf{y}}_t|\mathbf{x}, \bar{\mathbf{y}}_{<t}). \tag{12.9}$$

Ở đây $v_{\text{prm}}(\cdot)$ là một mạng nơ-ron được huấn luyện để dự đoán điểm đúng đắn. Ví dụ, ta có thể gắn một đầu phân loại (*classification head*) đơn giản (chẳng hạn một tầng tuyến tính theo sau là hàm kích hoạt sigmoid) vào biểu diễn đầu ra của một LLM có sẵn.

Để xác định chất lượng tổng thể của toàn bộ đường lập luận, ta có thể tổng hợp các điểm số ở mức bước này. Một cách đơn giản là lấy điểm nhỏ nhất trên tất cả các bước. Nói cách khác, ta chỉ coi lập luận là đúng nếu "mắt xích yếu nhất" của nó chặt chẽ về logic. Khi đó điểm kiểm chứng được cho bởi

$$v(\mathbf{x}, \mathbf{y}) \;=\; \min_{1\le t\le n_p} v_{\text{prm}}(\bar{\mathbf{y}}_t|\mathbf{x}, \bar{\mathbf{y}}_{<t}). \tag{12.10}$$

Ngoài ra, ta có thể định nghĩa điểm kiểm chứng là trung bình của các điểm ở mức bước

$$v(\mathbf{x}, \mathbf{y}) \;=\; \frac{1}{n_p} \sum_{1\le t\le n_p} v_{\text{prm}}(\bar{\mathbf{y}}_t|\mathbf{x}, \bar{\mathbf{y}}_{<t}). \tag{12.11}$$

Chú giải của người dịch: Lấy min hay lấy trung bình?

Giả sử một lời giải 4 bước được PRM chấm 0.95, 0.9, 0.2, 0.95 (bước 3 rất đáng ngờ). Theo Phương trình (12.10), điểm cả đường là $\min = 0.2$: chỉ cần một bước sai là cả lời giải bị đánh giá thấp, giống như một sợi xích chỉ chắc bằng mắt xích yếu nhất. Theo Phương trình (12.11), điểm là $(0.95+0.9+0.2+0.95)/4 = 0.75$: bước sai bị "pha loãng" bởi các bước tốt. Cách lấy min khắt khe hơn, phù hợp khi một lỗi duy nhất đủ làm hỏng kết quả.

Để huấn luyện một mô hình kiểm chứng, ta cần một tập dữ liệu đã được gán nhãn. Khác với huấn luyện ORM, vốn chỉ cần biết đáp án đúng cuối cùng ở cuối đường, huấn luyện PRM đòi hỏi giám sát ở mức bước (*step-level supervision*). Có hai cách thường dùng để xây dựng tập dữ liệu huấn luyện PRM:

- **Gán nhãn bởi con người:** Các chuyên gia tự tay giải bài toán và gán nhãn "đúng" hoặc "sai" cho từng bước trung gian do LLM sinh ra. Dù phương pháp này có thể tạo ra các bộ kiểm chứng chính xác, nó tốn kém và khó mở rộng quy mô, đặc biệt với các nhiệm vụ toán học nâng cao hay lập trình đòi hỏi chuyên gia lĩnh vực.
- **Kiểm chứng tự động thông qua triển khai thử (rollout):** Ta cũng có thể dùng các phương pháp tự động để ước lượng giá trị của một bước [Wang và cộng sự, 2024c]. Chẳng hạn, nếu có một tập dữ liệu với đáp án cuối cùng đã biết (như các đề thi toán), ta có thể dùng **triển khai thử Monte Carlo** (*Monte Carlo rollouts*). Từ một bước trung gian $\bar{\mathbf{y}}_t$, ta prompt LLM tiếp tục sinh ra nhiều đường hoàn chỉnh. Nếu một tỷ lệ cao các đường này cuối cùng đạt đến đáp án đúng chuẩn a^*, ta có thể giả định rằng bước trung gian $\bar{\mathbf{y}}_t$ đang đi đúng hướng. Một cách hình thức, giá trị ước lượng $\hat{r}_t$ của một bước có thể được định nghĩa là xác suất kỳ vọng đạt được đáp án đúng:

  $$\hat{r}_t \;=\; \mathbb{E}_{\mathbf{y}_{>t}\sim\pi_\theta}\left[\mathbf{1}(\text{Extract}(\mathbf{y}_{>t}) = a^*)\right]. \tag{12.12}$$

  Ở đây $\mathbf{y}_{>t}$ là phần đường còn lại tính từ $\bar{\mathbf{y}}_t$, và $\text{Extract}(\mathbf{y}_{>t})$ biểu diễn đáp án được trích xuất từ đường $\mathbf{y}_{>t}$. Khi các giá trị này đã được ước lượng thông qua triển khai thử, có thể huấn luyện một PRM để dự đoán $\hat{r}_t$.

Chú giải của người dịch: Phương trình (12.12) chỉ là "tỷ lệ về đích đúng"

Kỳ vọng của một hàm chỉ thị chính là xác suất để điều kiện bên trong xảy ra. Trong thực tế ta ước lượng nó bằng tần suất: từ bước $\bar{\mathbf{y}}_t$, cho mô hình viết tiếp, chẳng hạn, 10 lời giải hoàn chỉnh; nếu 7 lời giải ra đúng đáp án a^* thì $\hat{r}_t \approx 7/10 = 0.7$. Bước nào "dẫn đường tốt" sẽ có tỷ lệ này cao; bước đã sai thì hầu như mọi lời giải viết tiếp đều sai, nên tỷ lệ gần 0. Đây giống cách tính xác suất bằng thống kê tần suất ở THPT.

2. Bộ kiểm chứng sinh

Tất cả các bộ kiểm chứng nói trên đều đòi hỏi sửa đổi kiến trúc và công sức huấn luyện thêm, ví dụ ta cần huấn luyện một đầu phân loại để đưa ra một điểm số. Với năng lực sinh văn bản mạnh của LLM, một cách tiếp cận tự nhiên khác là trực tiếp prompt LLM đánh giá các bước lập luận và sinh ra phản hồi kiểm chứng bằng ngôn ngữ tự nhiên. Cách tiếp cận này được gọi là **kiểm chứng sinh** (*generative verification*) [Zheng và cộng sự, 2023].

Thay vì sửa đổi kiến trúc mô hình để đưa ra một giá trị vô hướng duy nhất, ta phát biểu nhiệm vụ kiểm chứng như một bài toán sinh văn bản tiêu chuẩn. Bằng cách đưa truy vấn và các bước trung gian trở lại vào LLM, ta có thể yêu cầu nó đóng vai giám khảo. Điều này có nghĩa là mô hình hành động như một giáo viên viết lời phê bên lề bài kiểm tra, thay vì chỉ cho một điểm số cuối cùng.

Để hiểu cách hoạt động, hãy xem một ví dụ đơn giản. Giả sử mô hình đang giải một hệ phương trình:

> *Bài toán: Tìm x và y biết $2x + y = 10$ và $x - y = 2$.*

Ta giả sử mô hình đã sinh ra hai bước đúng:

> *Từ phương trình thứ hai, ta có thể biểu diễn x theo y: $x = y + 2$.*
> *Thế x vào phương trình thứ nhất: $2(y + 2) + y = 10$.*

Bây giờ nó đang thử bước thứ ba, trong đó có một lỗi tính toán:

> *Khai triển phương trình được $2y + 2 + y = 10$, rút gọn thành $3y + 2 = 10$.*

Ta có thể kết hợp truy vấn gốc, bước 1 và 2, cùng bước hiện tại 3 với một prompt kiểm chứng (ví dụ: "*Dựa trên các bước trước, hãy kiểm tra xem logic ở bước hiện tại có đúng không và giải thích kết luận của bạn.*"). Bộ kiểm chứng sinh khi đó có thể đưa ra một lời phê bình bằng văn bản như sau:

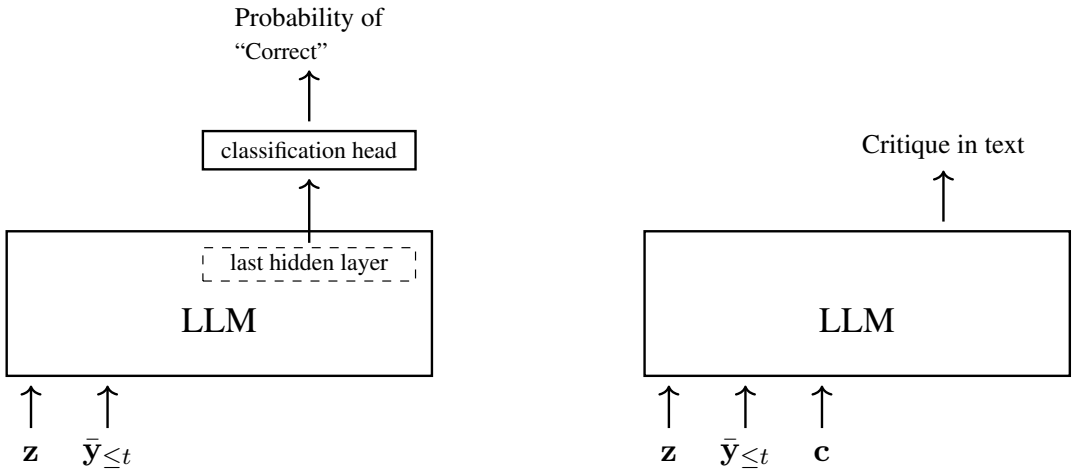
> *Nhận xét: Sai. Ở bước hiện tại, khi phân phối số 2 vào $(y + 2)$ từ Bước 2, bạn phải nhân cả hai số hạng trong ngoặc với 2. Kết quả phải là $2y + 4$, không phải $2y + 2$. Khai triển đúng là $2y + 4 + y = 10$, rút gọn thành $3y + 4 = 10$.*

Để hình thức hóa quá trình kiểm chứng sinh này, ta giả sử đầu vào gốc $\mathbf{x}$ gồm một truy vấn $\mathbf{z}$ và một chỉ dẫn $\mathbf{c}$ yêu cầu mô hình kiểm chứng bước dựa trên ngữ cảnh. Ta có thể nối truy vấn $\mathbf{z}$, lịch sử lập luận $\bar{\mathbf{y}}_{\leq t}$ và chỉ dẫn $\mathbf{c}$ thành một văn bản dài. Sau đó, ta đưa văn bản này làm đầu vào cho một LLM và thu được một lời phê bình bằng ngôn ngữ tự nhiên $\mathbf{p}_t$:

$$\mathbf{p}_t \sim \Pr(\cdot|\mathbf{z}, \bar{\mathbf{y}}_{\leq t}, \mathbf{c}). \tag{12.13}$$

Ở đây $\Pr(\cdot)$ biểu diễn phân phối xác suất do một LLM có sẵn mô hình hóa. Như vậy ta có thể trực tiếp dùng các LLM mạnh để kiểm chứng chính xác mà không cần huấn luyện. Hình 12.1 so sánh cách tiếp

cận sinh này với các bộ kiểm chứng phân biệt truyền thống.



(a) Discriminative Verifier (b) Generative Verifier

Hình 12.1: So sánh giữa các cách tiếp cận kiểm chứng phân biệt và kiểm chứng sinh. Hình con (a) cho thấy một bộ kiểm chứng phân biệt, gắn một đầu phân loại vào tầng ẩn cuối cùng của LLM để đưa ra một xác suất đúng dưới dạng số. Hình con (b) cho thấy một bộ kiểm chứng sinh, prompt LLM với truy vấn $\mathbf{z}$, lịch sử lập luận $\bar{\mathbf{y}}_{\leq t}$ và một chỉ dẫn $\mathbf{c}$ để sinh ra một lời phê bình bằng ngôn ngữ tự nhiên thay vì một giá trị vô hướng. (Nhãn trong hình: *Discriminative/Generative Verifier* = bộ kiểm chứng phân biệt/sinh; *last hidden layer* = tầng ẩn cuối; *classification head* = đầu phân loại; *Probability of "Correct"* = xác suất "Đúng"; *Critique in text* = lời phê bằng văn bản.)

Dù các lời phê bình bằng ngôn ngữ tự nhiên hữu ích cho con người, một số thuật toán tìm kiếm (như tìm kiếm cây) có thể cần điểm số dạng số để xếp hạng các đường lập luận khác nhau. Để trích xuất một điểm số liên tục từ bộ kiểm chứng sinh, một phương pháp phổ biến là đo xác suất mô hình đưa ra một token chỉ thị (như "Correct" hay "Incorrect", tức "Đúng" hay "Sai") làm từ đầu tiên của lời phê bình. Do đó, điểm kiểm chứng dạng số có thể được định nghĩa là:

$$v(\mathbf{z}, \bar{\mathbf{y}}_{\leq t}, \mathbf{c}) \;=\; \Pr(y_{\text{anchor}} = \text{"Correct"} | \mathbf{z}, \bar{\mathbf{y}}_{\leq t}, \mathbf{c}). \tag{12.14}$$

Ở đây y_{anchor} là token chỉ thị, thường được trích xuất từ văn bản đầu ra bằng các heuristic.

Chú giải của người dịch: Biến lời phê thành điểm số

Ở vị trí từ đầu tiên của lời phê, LLM tính một phân phối xác suất trên toàn bộ từ vựng. Giả sử nó gán $\Pr(\text{"Correct"}) = 0.8$ và $\Pr(\text{"Incorrect"}) = 0.15$; khi đó điểm kiểm chứng là 0.8. Một bước khác có $\Pr(\text{"Correct"}) = 0.3$ sẽ bị xếp hạng thấp hơn. Như vậy ta tận dụng "độ tự tin" của mô hình mà không cần huấn luyện thêm đầu phân loại nào.

Bộ kiểm chứng sinh có một số ưu điểm so với các mô hình phần thưởng vô hướng truyền thống. Thứ nhất, chúng mang lại khả năng diễn giải cho việc giải quyết vấn đề. Vì bộ kiểm chứng giải thích lập luận của mình bằng văn bản, các nhà phát triển có thể chẩn đoán vì sao một đường lập luận được chấp nhận hay không. Thứ hai, chúng cho phép tự sửa lỗi (*self-correction*). Vì lời phê bình bằng văn bản thường nói rõ cách sửa một lỗi (như đưa ra khai triển đúng $2y + 4$ trong ví dụ của chúng ta), hệ thống lập luận có thể trực tiếp tích hợp phản hồi này để sửa trạng thái hiện tại và tiếp tục tìm kiếm, thay

vì đơn giản loại bỏ toàn bộ đường. Thứ ba, chúng đòi hỏi sửa đổi kiến trúc tối thiểu. Miễn là các LLM hiện tại có năng lực lập luận mạnh, chính cùng một mô hình có thể được dùng vừa làm bộ giải bài toán vừa làm bộ kiểm chứng.

3. Hệ thống ký hiệu hình thức làm bộ kiểm chứng

Dù các bộ kiểm chứng nơ-ron được dùng rộng rãi, chúng có chung một điểm yếu: chúng vẫn là mạng nơ-ron. Điều này nghĩa là chúng cũng có thể bị ảo giác hoặc mắc lỗi logic. Với những nhiệm vụ đòi hỏi độ chính xác nghiêm ngặt, như toán học phức tạp hay kỹ nghệ phần mềm, ta cũng có thể dùng các hệ thống ký hiệu hình thức (*formal symbolic systems*) làm bộ kiểm chứng. Chẳng hạn, trong các nhiệm vụ tối ưu hóa mã, khi mô hình cố gắng tối ưu mã biểu diễn trung gian (*intermediate representation*) (như LLVM-IR), một bộ kiểm tra tương đương hình thức (*formal equivalency checker*) (như Alive2) có thể tự động kiểm chứng xem mã mới sinh có giữ nguyên chính xác ngữ nghĩa như mã gốc hay không.

Các hệ thống hình thức tuân thủ nghiêm ngặt những quy tắc toán học và logic được định nghĩa trước, nên phán quyết của chúng đáng tin cậy 100%. Để dùng một hệ thống hình thức làm bộ kiểm chứng, LLM phải sinh ra các bước lập luận dưới dạng mã thực thi được hoặc các mệnh đề toán học hình thức, thay vì chỉ là ngôn ngữ tự nhiên thông thường. Ví dụ, khi giải một bài toán, LLM có thể viết một đoạn mã Python nhỏ để thực hiện một phép tính trung gian cụ thể. Hệ thống sau đó thực thi đoạn mã này bằng một công cụ bên ngoài. Quá trình kiểm chứng rất đơn giản:

- Nếu mã chạy không lỗi và cho ra một đầu ra hợp lệ, bước lập luận được xác nhận là đúng.
- Nếu mã báo lỗi (chẳng hạn lỗi cú pháp hay vi phạm toán học), bước lập luận được xác nhận là sai.

Một cách hình thức, gọi một bước trung gian $\bar{\mathbf{y}}_t$ là một khối mã hoặc logic hình thức được sinh ra, và gọi E là môi trường thực thi hình thức. Điểm kiểm chứng có thể được định nghĩa là:

$$v(\mathbf{x}, \bar{\mathbf{y}}_{<t}, \bar{\mathbf{y}}_t) = \begin{cases} 1, & \text{nếu } E(\bar{\mathbf{y}}_t) \text{ trả về kết quả đúng,} \\ 0, & \text{trường hợp khác.} \end{cases} \tag{12.15}$$

Cách tiếp cận này có một ưu điểm rõ ràng: nó loại bỏ các "dương tính giả" gây ra bởi ảo giác của bộ kiểm chứng. Hơn nữa, khi thực thi thất bại, hệ thống hình thức thường sinh ra một thông báo lỗi. Thông báo lỗi này có thể được đưa trở lại LLM để giúp mô hình hiểu điều gì đã sai và sửa logic của nó ở lần thử tiếp theo.

Tuy nhiên, dùng các hệ thống ký hiệu hình thức cũng có những hạn chế. Thứ nhất, nó đòi hỏi LLM có kỹ năng lập trình hoặc hình thức hóa tốt, vì việc chuyển lập luận trừu tượng thành mã nghiêm ngặt là khó. Thứ hai, dù các hệ thống hình thức rất phù hợp cho toán học và lập trình, chúng không phù hợp với các nhiệm vụ mở trong thế giới thực (như viết một bài luận hay lên kế hoạch một chuyến đi), nơi lập luận không thể dễ dàng ánh xạ thành các quy tắc logic cứng nhắc.

Chú giải của người dịch: Một ví dụ về kiểm chứng bằng thực thi mã

Với bài toán của Tom, mô hình có thể viết bước trung gian thành mã: `total = 3*40 + 2*20; print(total*0.9)`. Máy tính chạy đoạn mã và in ra `144.0`. Kết quả do máy tính thực hiện thì không thể "ảo giác", nên nếu mô hình trước đó viết bằng lời rằng đáp số là \$146, ta biết ngay đó là sai. Đây cũng là lý do nhiều hệ thống hiện nay cho LLM "dùng máy tính bỏ túi" (công cụ thực thi mã) thay vì tự nhẩm.

12.4 Học cách lập luận

Cải thiện khả năng lập luận (*reasoning*) của các LLM là một việc khó, và vì vậy thường đòi hỏi kết hợp nhiều phương pháp. Như đã bàn ở mục trước, co giãn lúc suy luận (*inference-time scaling*) nhằm kích hoạt quá trình "suy nghĩ" của các LLM có sẵn trong lúc sinh văn bản. Một hướng khác, rất tự nhiên, là nội tại hóa các khả năng lập luận này trực tiếp vào tham số của mô hình trong quá trình huấn luyện. Trong mục này, chúng ta sẽ tập trung vào mô thức "học cách lập luận" đó.

Dữ liệu lập luận có thể được đưa vào bất kỳ giai đoạn huấn luyện nào. Ví dụ, trong giai đoạn tiền huấn luyện (*pre-training*), các đường lập luận có thể đơn giản được thêm vào như văn bản bình thường cho bài toán dự đoán token tiếp theo tiêu chuẩn. Ta cũng có thể xét việc huấn luyện dựa trên lập luận ở giai đoạn hậu huấn luyện (*post-training*). Chẳng hạn, trong tinh chỉnh có giám sát (*supervised fine-tuning*), mô hình có thể được tinh chỉnh trên những mẫu chất lượng cao về cách giải bài toán từng bước. Điều này thường được thực hiện bằng cách huấn luyện mô hình trên các tập dữ liệu chứa các quỹ đạo (*trajectory*) CoT. Hơn nữa, học tăng cường (*reinforcement learning*) có thể được áp dụng để tối ưu thêm các quá trình lập luận này. Ở đây chúng ta tập trung vào việc đưa sự giám sát dựa trên lập luận vào quá trình căn chỉnh (*alignment*) LLM. Trong các tiểu mục sau, chúng ta sẽ bàn về dữ liệu lập luận có giám sát, giám sát quá trình, các kỹ thuật học tăng cường, và các chiến lược để co giãn khả năng lập luận.

12.4.1 Dữ liệu lập luận có giám sát

Một cách hình thức, trong tinh chỉnh có giám sát, một LLM được huấn luyện trên một tập dữ liệu gồm các cặp đầu vào–đầu ra $\mathcal{D} = \{(\mathbf{x}, \mathbf{y})\}$. Hàm mục tiêu huấn luyện tiêu chuẩn là

$$\hat{\theta} \;=\; \arg\max_{\theta} \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \log \Pr_\theta(\mathbf{y}|\mathbf{x}), \tag{12.16}$$

trong đó $\Pr_\theta(\cdot)$ ký hiệu mô hình được tham số hóa bởi θ.

Nếu $\mathbf{y}$ chỉ đơn thuần là kết luận cuối cùng, mô hình sẽ học cách dự đoán câu trả lời mà không sinh ra một cách tường minh đường lập luận tương ứng. Với huấn luyện/tinh chỉnh dựa trên lập luận, ta định nghĩa lại $\mathbf{y}$ là một đường lập luận. Như trong Phương trình (12.2), ta có thể biểu diễn đầu ra như một đường lập luận gồm một chuỗi các bước lập luận $\mathbf{y} = \{\bar{\mathbf{y}}_1, ..., \bar{\mathbf{y}}_{n_p}\}$. Do đó, ta có thể viết lại hàm mục tiêu trên thành

$$\hat{\theta} \;=\; \arg\max_{\theta} \sum_{(\mathbf{x},\{\bar{\mathbf{y}}_1,...,\bar{\mathbf{y}}_{n_p}\})\in\mathcal{D}} \log \Pr_\theta(\bar{\mathbf{y}}_1, ..., \bar{\mathbf{y}}_{n_p}|\mathbf{x}). \tag{12.17}$$

Bằng cách này, mô hình được cung cấp tường minh các chi tiết lập luận và được huấn luyện để sinh ra một đường lập luận hoàn chỉnh, kể cả kết luận cuối cùng.

Chú giải của người dịch: Đọc Phương trình (12.16)–(12.17)

$\log \Pr_\theta(\mathbf{y}|\mathbf{x})$ đo mức "mô hình tin" rằng đầu ra đúng là $\mathbf{y}$ khi gặp đầu vào $\mathbf{x}$; xác suất càng gần 1 thì logarit càng gần 0 (lớn nhất). Ký hiệu $\arg\max_\theta$ nghĩa là "tìm bộ tham số θ làm cho tổng phía sau lớn nhất". Hai phương trình chỉ khác nhau ở *cái mà mô hình phải tạo ra*: ở (12.16) có thể chỉ là đáp số "12", còn ở (12.17) là toàn bộ lời giải từng bước rồi mới đến đáp số. Giống như khi chấm bài, giáo viên yêu cầu học sinh trình bày cả lời giải chứ không chỉ ghi đáp số.

Trong thực tế, để tách quá trình lập luận cốt lõi khỏi các phần khác của đầu ra, các mô hình lập luận thường dùng những token định dạng đặc biệt. Một cách làm được dùng rộng rãi là bọc các bước lập

luận cốt lõi bên trong cặp thẻ <think> và </think>. Khi một LLM sinh văn bản theo định dạng này, các bước lập luận viết bên trong cặp thẻ được gọi là một **vết CoT** (*CoT trace*) (hay đơn giản là một **vết lập luận** (*reasoning trace*)). Ta có thể coi vết CoT này như tờ *giấy nháp* của LLM. Giống như một học sinh làm bài thi toán ghi các phép tính trung gian ra giấy nháp trước khi viết đáp số cuối cùng vào tờ bài thi chính thức, LLM dùng khối <think> để "trình bày cách làm".

Ví dụ, một mẫu huấn luyện được định dạng bằng các thẻ này có thể trông như sau:

15% của 80 là bao nhiêu?
<think>
Để tính cho dễ, tôi có thể tách 15% thành 10% và 5%.
Trước hết, 10% của 80 là 8.
Tiếp theo, 5% bằng một nửa của 10%, tức là 4.
Cộng lại: $8 + 4 = 12$.
</think>
Câu trả lời là 12.

Ở đây truy vấn là "15% của 80 là bao nhiêu?". Đầu ra có năm bước, trong đó các bước 1–4 tạo thành vết CoT và bước 5 là kết luận. Cả hai thẻ <think> và </think> đều là token hợp lệ, nên mô hình sẽ học cách sinh ra chúng để đánh dấu các bước lập luận cốt lõi. Ở phía người dùng, ta có thể xóa các thẻ này để hiển thị một đầu ra gọn gàng. Lưu ý rằng trong các ứng dụng thực tế, đầu ra có thể rất phức tạp và chứa cả nội dung không liên quan đến lập luận. Vì vậy, việc đưa vào cặp thẻ <think> và </think> có thể giúp mô hình tập trung vào việc giải bài toán lập luận.

Dữ liệu lập luận cũng có thể được biểu diễn bằng nhiều định dạng khác. Một lựa chọn đơn giản là định dạng từng bước bằng ngôn ngữ tự nhiên. Thay vì dùng token đặc biệt, dữ liệu dựa vào các tiền tố văn bản như "Bước 1:", "Bước 2:", hay đơn giản là "Suy nghĩ:" rồi sau đó là "Trả lời:". Ngoài ra, dữ liệu lập luận có thể được cấu trúc bằng cách tách thông điệp theo vai trò. Trong các khuôn mẫu hội thoại (*chat template*) hiện đại, thay vì trộn lập luận và câu trả lời trong một thông điệp duy nhất, vết lập luận được gán cho một vai trò nội bộ (ví dụ vai trò "thought" hay "system_scratchpad"), còn kết luận cuối cùng được gán cho vai trò "assistant" tiêu chuẩn. Cách này tách phần suy nghĩ nội bộ của LLM khỏi phản hồi hướng tới người dùng ngay ở cấp hệ thống.

12.4.2 Giám sát quá trình

Như đã bàn ở Mục 12.3.4, các mô hình phần thưởng quá trình (*process reward model, PRM*) (PRM) là công cụ mạnh để kiểm chứng các bước lập luận trong lúc suy luận. Một mở rộng tự nhiên là dùng chúng để định hướng việc huấn luyện LLM thông qua giám sát quá trình (*process supervision*).

Trong giám sát kết quả (*outcome supervision*) tiêu chuẩn (chẳng hạn tinh chỉnh có giám sát), mô hình chỉ được đánh giá dựa trên kết quả cuối cùng của toàn bộ đường lập luận $\mathbf{y}$. Nếu mô hình sinh ra một chuỗi năm bước và mắc lỗi logic ở bước thứ ba, một cách huấn luyện dựa trên kết quả có thể chỉ đơn giản phạt toàn bộ chuỗi. Mô hình phải tự đoán xem chính xác bước nào gây ra thất bại. Đây được gọi là **bài toán phân bổ công trạng** (*credit assignment problem*) [Minsky, 1961; Sutton, 1984].

Giám sát quá trình giải quyết vấn đề này bằng cách áp dụng tín hiệu huấn luyện ở cấp từng bước. Thay vì một phần thưởng duy nhất cho cả đường lập luận, mô hình nhận phản hồi cho mỗi bước riêng lẻ $\bar{\mathbf{y}}_t$. Ví dụ, hãy tưởng tượng bạn dạy một học sinh giải một phương trình đại số phức tạp. Nếu bạn chỉ nhìn con số cuối cùng em viết ra và nói "Sai", học sinh sẽ không biết mình hiểu sai công thức, cộng nhầm, hay quên một dấu trừ. Giám sát quá trình giống như việc giáo viên chỉ đích danh dòng thứ 3 và nói "Lập luận của em hỏng ở chỗ này".

Giám sát quá trình thường được đặt dưới dạng một bài toán học tăng cường. Để áp dụng học tăng cường cho giám sát quá trình, ta thường theo một vòng lặp huấn luyện gồm ba bước:

- **Triển khai (rollout)**: LLM nhận một đầu vào $\mathbf{x}$ và sinh ra một đường lập luận đầy đủ $\mathbf{y} = \{\bar{\mathbf{y}}_1, \ldots, \bar{\mathbf{y}}_{n_p}\}$. Quá trình triển khai có thể được lặp lại nhiều lần để sinh ra một tập các đường lập luận $\mathcal{Y}$.
- **Kiểm chứng (verification)**: Một mô hình phần thưởng (tức là một PRM kiểm tra các bước trung gian) đánh giá từng đường lập luận $\mathbf{y}$ và trả về một điểm phần thưởng $R(\mathbf{x}, \mathbf{y})$.
- **Cập nhật (update)**: Một thuật toán học tăng cường dùng các phần thưởng này để cập nhật tham số θ của mô hình.

Một hàm mục tiêu huấn luyện đơn giản có thể được phát biểu là cực đại hóa phần thưởng kỳ vọng

$$J(\theta) \;=\; \mathbb{E}_{\mathbf{y}\sim \mathrm{Pr}_\theta(\cdot|\mathbf{x})}\big[R(\mathbf{x}, \mathbf{y})\big]. \tag{12.18}$$

Ta có thể sửa đổi hàm mục tiêu này để huấn luyện ổn định hơn, chẳng hạn đưa hàm ưu thế (*advantage function*) vào hàm mục tiêu như trong PPO.

Chú giải của người dịch: Phần thưởng kỳ vọng

Ký hiệu $\mathbb{E}_{\mathbf{y}\sim \mathrm{Pr}_\theta(\cdot|\mathbf{x})}[\,\cdot\,]$ là "giá trị trung bình khi $\mathbf{y}$ được lấy ngẫu nhiên theo cách mô hình sinh ra". Ví dụ, nếu với một bài toán mô hình sinh lời giải đúng (phần thưởng 1) với xác suất 0.7 và lời giải sai (phần thưởng 0) với xác suất 0.3, thì phần thưởng kỳ vọng là $0.7 \cdot 1 + 0.3 \cdot 0 = 0.7$. Huấn luyện nhằm điều chỉnh θ để con số này tăng lên, tức là mô hình ngày càng hay sinh ra lời giải được thưởng cao.

Dù điều này trông giống quy trình học tăng cường tiêu chuẩn dùng trong căn chỉnh LLM nói chung (như mô tả ở Chương 10), có một khác biệt căn bản về bản chất của phần thưởng. Trong các tác vụ căn chỉnh LLM thông thường, mô hình phần thưởng thường đánh giá toàn bộ văn bản được sinh ra và trả về một phần thưởng thưa (*sparse reward*) ở ngay cuối cùng. Khi đó thuật toán học tăng cường gặp khó khăn với bài toán phân bổ công trạng, vì nó phải đoán xem những token hay đoạn nào thực sự đóng góp vào điểm số cuối cùng đó.

Ngược lại, lập luận diễn ra từng bước một. Do đó, điểm kiểm chứng, mà để đơn giản ta ký hiệu là $v_t = v(\mathbf{x}, \bar{\mathbf{y}}_{\le t})$, vốn dĩ là một tín hiệu ở cấp bước. Đặc điểm này khiến các tác vụ lập luận rất phù hợp với học tăng cường hiện đại. Vì v_t được tạo ra theo từng bước, nó tự nhiên biến một tác vụ sinh ngôn ngữ có phần thưởng thưa thành một môi trường có phần thưởng dày (*dense reward*).

Câu hỏi kỹ thuật chính khi đó trở thành làm sao biến đổi đúng cách các điểm cấp bước tự nhiên v_t này thành những tín hiệu huấn luyện hình thức (như phần thưởng từng bước có cấu trúc r_t hay ưu thế A_t) mà các thuật toán đòi hỏi. Tùy theo định dạng đầu ra của PRM và khung học tăng cường, ta xét ba phương pháp sau.

1. Huấn luyện với điểm kiểm chứng nhị phân

Trong một số trường hợp, PRM đóng vai trò một bộ kiểm tra quy tắc nghiêm ngặt (như một máy tính toán học hay một trình biên dịch mã). Nó chỉ trả về một điểm nhị phân: $v_t \in \{0, 1\}$. Điểm 1 nghĩa là logic ở bước t là đúng khi biết các bước trước đó, còn 0 nghĩa là có lỗi logic xảy ra ở bước này.

Khi làm việc với điểm nhị phân, ý tưởng rất đơn giản: chỉ một bước sai cũng làm hỏng toàn bộ đường lập luận. Nếu muốn gán một phần thưởng cấp chuỗi $R(\mathbf{x}, \mathbf{y})$ cho cả đường lập luận, ta chỉ cần nhân tất cả điểm của các bước với nhau

$$R(\mathbf{x}, \mathbf{y}) \;=\; \prod_{t=1}^{n_p} v_t. \tag{12.19}$$

Như vậy mô hình chỉ nhận phần thưởng dương nếu điểm kiểm chứng của mọi bước đều bằng 1.

Một cách khác, nếu thuật toán học tăng cường hỗ trợ phần thưởng cấp bước (ký hiệu là r_t), ta có thể phạt mô hình đúng tại chỗ xảy ra thất bại. Ví dụ, ta có thể cho phần thưởng 1 chừng nào các bước vẫn còn đúng ($v_t = 1$). Ngay khi mô hình mắc lỗi ($v_t = 0$), ta gán một phần thưởng âm và dừng việc sinh.

2. Huấn luyện với điểm kiểm chứng giá trị thực

Trong hầu hết các PRM nơ-ron, bộ kiểm chứng xuất ra một số thực liên tục $v_t \in \mathbb{R}$. Đó có thể là một xác suất từ 0 đến 1 (thể hiện độ tin cậy rằng đường lập luận là đúng) hoặc một giá trị vô hướng chưa chuẩn hóa.

Để áp dụng điểm kiểm chứng liên tục, kỹ thuật phổ biến nhất là **định hình phần thưởng** (*reward shaping*). Thay vì dùng điểm tuyệt đối, ta định nghĩa phần thưởng tức thời của bước r_t là hiệu số biên giữa bước hiện tại và bước trước đó:

$$r_t \;=\; v_t - v_{t-1}. \tag{12.20}$$

Phương pháp này rất trực quan. Hãy tưởng tượng một PRM ước lượng xác suất đi đến đáp án cuối cùng đúng. Nếu Bước 1 có độ tin cậy 0.4, và Bước 2 chứa một suy diễn xuất sắc nâng độ tin cậy lên 0.9, thì phần thưởng cho Bước 2 là $r_2 = 0.9 - 0.4 = +0.5$. Mô hình được thưởng cho chính bước nhảy logic đó. Nếu Bước 3 đưa vào một lỗi tính toán và độ tin cậy tụt xuống 0.1, phần thưởng là $r_3 = 0.1 - 0.9 = -0.8$.

Khi cộng các phần thưởng đã định hình này trên toàn bộ chuỗi, ta được một "tổng rút gọn kiểu ống lồng (*telescoping sum*)" tạo thành phần thưởng cấp chuỗi $R(\mathbf{x}, \mathbf{y})$:

$$\begin{aligned} R(\mathbf{x}, \mathbf{y}) &= \sum_{t=1}^{n_p} r_t \\ &= (v_1 - v_0) + (v_2 - v_1) + \cdots + (v_{n_p} - v_{n_p-1}) \\ &= v_{n_p} - v_0. \end{aligned} \tag{12.21}$$

Ở đây, v_0 thường được giả định bằng 0. Trong phép biến đổi này, các điểm trung gian triệt tiêu lẫn nhau. Điều đó có nghĩa là phần thưởng tổng thể của chuỗi vẫn nhất quán với đánh giá của bước cuối cùng.

Chú giải của người dịch: Tổng "ống lồng"

Đây chính là kiểu tổng bạn có thể đã gặp khi tính $\frac{1}{1\cdot 2} + \frac{1}{2\cdot 3} + \cdots$ bằng cách tách $\frac{1}{k(k+1)} = \frac{1}{k} - \frac{1}{k+1}$: các số hạng ở giữa khử nhau, chỉ còn đầu và cuối. Với ví dụ trên ($v_0 = 0$, $v_1 = 0.4$, $v_2 = 0.9$, $v_3 = 0.1$): $r_1 = 0.4$, $r_2 = +0.5$, $r_3 = -0.8$, tổng là $0.1 = v_3 - v_0$. Như vậy định hình phần thưởng chỉ "chia nhỏ" điểm cuối cùng ra cho từng bước, chứ không làm thay đổi tổng.

Để hiểu phần thưởng dày từng bước này thực sự huấn luyện mô hình như thế nào, ta cần nối nó với hàm mục tiêu học tăng cường. Như trong Phương trình (12.18), mục tiêu của ta là cực đại hóa tổng phần thưởng kỳ vọng trên mọi đường lập luận có thể được sinh ra. Để huấn luyện hiệu quả và phân bổ công trạng hợp lý, ta thường dùng *nguyên lý nhân quả*: một hành động thực hiện ở bước t chỉ có thể ảnh hưởng tới các phần thưởng nhận được sau bước này. Vì vậy, thay vì đánh giá một bước bằng phần thưởng của cả chuỗi, ta đánh giá nó bằng lợi tức tương lai (*future return*) R_t, được định nghĩa là tổng các phần thưởng từ bước t tới cuối chuỗi:

$$\begin{aligned} R_t &= \sum_{k=t}^{n_p} r_k \\ &= v_{n_p} - v_{t-1}. \end{aligned} \tag{12.22}$$

Cuối cùng, ta có thể huấn luyện mô hình bằng hàm mất mát thay thế (*surrogate loss function*)

$$\mathcal{L}(\theta) \;=\; -\sum_{t=1}^{n_p} \log \Pr_\theta(\bar{\mathbf{y}}_t | \mathbf{x}, \bar{\mathbf{y}}_{<t}) \cdot R_t. \tag{12.23}$$

Hàm mất mát này cung cấp một cơ chế phản hồi ở cấp bước. Với bất kỳ đường lập luận nào được sinh ra, mô hình nhìn vào lợi tức tương lai R_t tại mỗi bước t. Nếu R_t dương, hàm mất mát đẩy mô hình tăng xác suất $\log \Pr_\theta(\cdot)$ sinh ra bước này. Nếu R_t âm, nó đóng vai trò một hình phạt, đẩy mô hình tránh sinh ra bước này trong tương lai.

Chú giải của người dịch: Vì sao có dấu trừ trong (12.23)?

Khi huấn luyện, ta luôn *cực tiểu hóa* hàm mất mát. Cực tiểu $-\sum \log \Pr \cdot R_t$ cũng chính là cực đại $\sum \log \Pr \cdot R_t$. Mỗi số hạng giống một "trọng số khuyến khích": bước nào có $R_t = +0.5$ thì mô hình được đẩy tăng xác suất sinh ra bước đó; bước nào có $R_t = -0.8$ thì ngược lại, xác suất bị đẩy giảm, và đẩy mạnh hơn vì $|R_t|$ lớn hơn.

3. Đưa kết quả kiểm chứng vào hàm ưu thế

Nhiều thuật toán học tăng cường (như PPO) tối ưu mô hình bằng hàm ưu thế A_t, hàm này đo một hành động tốt hơn bao nhiêu so với mức được kỳ vọng. Thay vì xem điểm kiểm chứng như phần thưởng thô, ta có thể đưa chúng vào phép tính ưu thế.

Một phương pháp là đưa kiểm chứng cấp bước vào sai số chênh lệch thời gian (*temporal difference error*). Nếu PRM được huấn luyện để ước lượng phần thưởng tương lai kỳ vọng, ta có thể xem nó như giá trị cơ sở (*value baseline*) trong học tăng cường tác nhân–phê bình (*actor-critic*) tiêu chuẩn. Ta tính ưu thế A_t bằng sai số chênh lệch thời gian

$$A_t \;=\; \tilde{r}_t + \gamma v_t - v_{t-1}, \tag{12.24}$$

trong đó $\tilde{r}_t$ là phần thưởng bên ngoài bất kỳ đến từ môi trường (thường bằng 0 ở các bước suy nghĩ trung gian) và γ là hệ số chiết khấu (*discount factor*).

Một phương pháp khác là cải tiến cách thiết kế ưu thế tương đối theo nhóm. Trong **tối ưu chính sách tương đối theo nhóm** (*group relative policy optimization*) (**GRPO**), ta bỏ hẳn các điểm kiểm chứng tuyệt đối [Shao và cộng sự, 2024]. Thay vào đó, ta tính ưu thế bằng cách so sánh một nhóm các đường lập luận. Với một đầu vào $\mathbf{x}$, LLM sinh ra một nhóm các đường lập luận khác nhau. Một bộ kiểm chứng

gán điểm $v_t^{(i)}$ cho bước thứ t của đường thứ i. Ưu thế $A_t^{(i)}$ được tính bằng cách cộng dồn các điểm đã chuẩn hóa từ bước t tới cuối đường lập luận như sau

$$A_t^{(i)} = \sum_{k=t}^{n_p^{(i)}} \tilde{v}_k^{(i)}, \tag{12.25}$$

trong đó $n_p^{(i)}$ là tổng số bước của đường thứ i, và $\tilde{v}_k^{(i)}$ là điểm đã chuẩn hóa, được định nghĩa là:

$$\tilde{v}_k^{(i)} = \frac{v_k^{(i)} - \mu}{\sigma}, \tag{12.26}$$

với μ và σ là trung bình và độ lệch chuẩn của các điểm trên tất cả các bước của các đường lập luận trong nhóm. Ví dụ, nếu một mô hình thử 5 cách khác nhau để giải một phương trình, và đường số 3 thực hiện một bước có điểm cao hơn đáng kể so với trung bình của mọi bước trong các đường này, thì đường số 3 nhận ưu thế dương. Ở đây, trung bình nhóm μ đóng vai trò giá trị cơ sở, nhờ đó không cần huấn luyện và chạy một mạng giá trị (*value network*) riêng.

Chú giải của người dịch: Chuẩn hóa điểm trong GRPO

Phép tính (12.26) chính là "điểm chuẩn" (z-score) trong thống kê mô tả: lấy điểm trừ trung bình rồi chia cho độ lệch chuẩn. Ví dụ nhóm có các điểm bước 0.2, 0.4, 0.6, 0.8: trung bình $\mu = 0.5$, độ lệch chuẩn $\sigma \approx 0.224$. Bước có điểm 0.8 được $\tilde{v} = (0.8 - 0.5)/0.224 \approx +1.34$ (tốt hơn mặt bằng chung), bước có điểm 0.2 được ≈ -1.34. Như vậy điều quan trọng không phải điểm tuyệt đối, mà là bước đó tốt hơn hay kém hơn *so với các lời giải khác trong cùng nhóm*.

Khi ưu thế A_t (hoặc $A_t^{(i)}$) đã được tính bằng một trong hai phương pháp trên, nó có thể được dùng làm trọng số co giãn trong hàm mất mát. Một phiên bản đơn giản hóa của hàm mất mát dựa trên ưu thế có thể viết là[2]:

$$\mathcal{L}(\theta) = -\sum_{t=1}^{n_p} \log \Pr_\theta(\bar{\mathbf{y}}_t | \mathbf{x}, \bar{\mathbf{y}}_{<t}) \cdot A_t. \tag{12.27}$$

Lưu ý rằng lợi ích của việc dùng ưu thế A_t thay cho lợi tức thô R_t là giảm phương sai. Bằng cách trừ đi một giá trị cơ sở, dù đó là giá trị ước lượng v_{t-1} hay trung bình nhóm μ, hàm ưu thế cung cấp một thước đo hiệu năng *tương đối*. Nó cho mô hình biết không chỉ một bước lập luận có hợp logic hay không, mà còn liệu bước đó có tốt hơn mức kỳ vọng hay không. Phản hồi tương đối này ngăn mô hình bị "choáng ngợp" bởi các truy vấn quá dễ hoặc quá khó.

12.4.3 Chưng cất on-policy cho lập luận

Dù giám sát quá trình cung cấp tín hiệu phần thưởng từng bước để định hướng lập luận, nó vẫn dựa vào một giá trị vô hướng để cho mô hình biết nó đang làm tốt đến mức nào. Để cung cấp một tín hiệu huấn luyện còn phong phú hơn, ta có thể dùng **chưng cất on-policy** (*on-policy distillation*) (**OPD**). Phương pháp này thường được dùng trong hậu huấn luyện LLM, và rất phù hợp với mục tiêu của giám sát theo bước: làm cho tín hiệu huấn luyện dày đặc nhất có thể.

[2]Lưu ý rằng các thuật toán nâng cao như PPO và GRPO thường tăng cường hàm mục tiêu này bằng cách thêm một cơ chế *cắt* (*clipping*) (giới hạn tỉ số xác suất giữa chính sách mới và chính sách cũ) để ngăn tham số mô hình thay đổi quá mạnh trong một lần cập nhật. Tuy nhiên, nguyên lý toán học cốt lõi vẫn như vậy.

Chưng cất tri thức (*knowledge distillation*) tiêu chuẩn có thể được xem là một dạng **chưng cất off-policy** (*off-policy distillation*). Một mô hình "học sinh" nhỏ hơn học bằng cách bắt chước các đường lập luận do một mô hình "giáo viên" mạnh hơn sinh ra. Học sinh được huấn luyện trên một tập dữ liệu cố định gồm các đường lập luận ghi sẵn này. Tuy nhiên, lập luận là một quá trình dài và dễ đổ vỡ. Nếu học sinh mắc một lỗi nhỏ hoặc chọn một hướng hợp lệ hơi khác trong lúc sinh thực tế, nó sẽ rơi vào một trạng thái chưa từng gặp trong dữ liệu huấn luyện. Vì học sinh không biết cách gượng dậy sau lỗi của mình, quá trình lập luận có thể sụp đổ. Hiện tượng này được gọi là thiên lệch phơi nhiễm (*exposure bias*) hay dịch chuyển phân phối (*distribution shift*).

Chú giải của người dịch: On-policy và off-policy

Trong học tăng cường, "chính sách" (*policy*) là cách mô hình hiện tại ra quyết định, ở đây là cách nó sinh từng bước. *On-policy* nghĩa là học từ dữ liệu do chính mô hình đang được huấn luyện sinh ra; *off-policy* là học từ dữ liệu do một "người khác" (giáo viên, con người, hay phiên bản cũ) sinh ra. Giống như học lái xe: xem video người khác lái (off-policy) khác với tự cầm lái và được thầy chỉnh ngay khi đi lệch (on-policy).

Ý tưởng cốt lõi của OPD là mô hình học sinh học từ chính các quỹ đạo nó sinh ra, thay vì từ các quỹ đạo được lấy mẫu từ mô hình giáo viên. Một cách hình thức, gọi mô hình giáo viên là $\Pr^t(\cdot)$ và mô hình học sinh (được tham số hóa bởi θ) là $\Pr^s_\theta(\cdot)$. Quy trình OPD điển hình gồm hai bước:

- **Học sinh triển khai.** Cho một đầu vào $\mathbf{x}$, mô hình học sinh dùng tham số hiện tại θ để sinh một đường lập luận $\mathbf{y} = \{\bar{\mathbf{y}}_1, \ldots, \bar{\mathbf{y}}_{n_p}\}$. Vì học sinh tự sinh ra đường này, nó tự nhiên khám phá các hành vi lập luận của chính mình. Điều này bao gồm việc đi chệch một chút, chọn hướng tiếp cận khác, hay thậm chí mắc những lỗi mà giáo viên không bao giờ mắc.
- **Giáo viên hướng dẫn.** Sau đó ta đưa đường lập luận do học sinh sinh ra này vào mô hình giáo viên. Tại mỗi bước t, thay vì đưa ra một phần thưởng vô hướng, mô hình giáo viên tính phân phối xác suất cho bước tiếp theo $\Pr^t(\cdot|\mathbf{x}, \bar{\mathbf{y}}_{<t})$.

Lý tưởng nhất, mô hình học sinh được cập nhật để phân phối xác suất cấp bước của nó khớp với phân phối do mô hình giáo viên sinh ra. Mục tiêu là cực tiểu hóa độ phân kỳ KL (*KL divergence*) kỳ vọng giữa hai phân phối trên các đường lập luận do mô hình học sinh sinh ra:

$$\mathcal{L}_{\text{step}}(\theta) \;=\; \mathbb{E}_{\mathbf{y}\sim \Pr^s_\theta(\cdot|\mathbf{x})}\left[\sum_{t=1}^{n_p} D_{\text{KL}}\big(\Pr^s_\theta(\cdot|\mathbf{x}, \bar{\mathbf{y}}_{<t}) \,\|\, \Pr^t(\cdot|\mathbf{x}, \bar{\mathbf{y}}_{<t})\big)\right]. \tag{12.28}$$

Tuy nhiên, trong thực tế việc tính phân phối cấp bước này là bất khả thi về mặt tính toán. Vì một bước lập luận $\bar{\mathbf{y}}_t$ thường gồm nhiều token, phân phối xác suất trên mọi bước nhiều-token có thể có nằm trong một không gian lớn theo hàm mũ.

Để giải quyết điều này, ta quay về cách tiếp cận OPD tiêu chuẩn, hoạt động ở cấp token. Bằng cách xem đường lập luận được sinh ra như một chuỗi token $\mathbf{y} = \{y_1, \ldots, y_n\}$, hàm mục tiêu được phát biểu lại để tính độ phân kỳ KL theo từng token trên bộ từ vựng:

$$\mathcal{L}_{\text{token}}(\theta) \;=\; \mathbb{E}_{\mathbf{y}\sim \Pr^s_\theta(\cdot|\mathbf{x})}\left[\sum_{i=1}^{n} D_{\text{KL}}\big(\Pr^s_\theta(\cdot|\mathbf{x}, \mathbf{y}_{<i}) \,\|\, \Pr^t(\cdot|\mathbf{x}, \mathbf{y}_{<i})\big)\right]. \tag{12.29}$$

Hàm mục tiêu này sau đó được xấp xỉ bằng lấy mẫu. Với một đầu vào $\mathbf{x}$ cho trước, ta chỉ cần thực hiện một lượt triển khai để sinh một (hoặc một lô nhỏ) đường lập luận từ mô hình học sinh, rồi tính độ phân kỳ KL cấp token trên các lượt triển khai được lấy mẫu này.

Chú giải của người dịch: Độ phân kỳ KL và vì sao cấp bước là bất khả thi

$D_{\mathrm{KL}}(P\|Q) = \sum_w P(w) \log \frac{P(w)}{Q(w)}$ đo hai phân phối P, Q "khác nhau bao xa"; nó bằng 0 khi hai phân phối trùng nhau và dương khi khác nhau. Ở cấp token, tổng chạy qua bộ từ vựng (chẳng hạn 100 000 token) — máy tính làm được. Nhưng nếu một bước gồm 20 token thì số bước có thể có là $100\,000^{20} = 10^{100}$, nhiều hơn số nguyên tử trong vũ trụ quan sát được; vì thế không thể liệt kê để tính tổng ở cấp bước.

Hình 12.2 so sánh OPD với tinh chỉnh có giám sát và giám sát quá trình. Để hiểu rõ hơn sự khác biệt giữa các phương pháp này, hãy tưởng tượng một học sinh đang cố giải một bài hình học phức tạp:

- **Tinh chỉnh có giám sát.** Giống như đưa cho học sinh một cuốn sách đầy những lời giải đúng do giáo viên viết để học thuộc. Nếu học sinh quyết định kẻ một đường phụ khác với trong sách, cuốn sách không giúp được gì về cách tiếp tục từ điểm xuất phát mới đó.
- **Giám sát quá trình** (học tăng cường với PRM). Giống như có một người chấm bài nghiêm khắc kiểm tra bài làm của học sinh từng bước và nói "+1 điểm cho bước 1", "-1 điểm cho bước 2". Nó cho học sinh biết mình sai ở đâu, nhưng không cho biết cách sửa hay bước tiếp theo đúng phải là gì.
- **Chưng cất on-policy.** Giống như có một gia sư giỏi ngồi ngay cạnh học sinh. Khi học sinh kẻ đường phụ khác thường đó (một trạng thái không có trong sách chuẩn), gia sư nhìn vào trạng thái hiện tại của học sinh và nói: "Được rồi, vì em đã kẻ đường ở đây, bước tốt nhất tiếp theo là tính góc này."

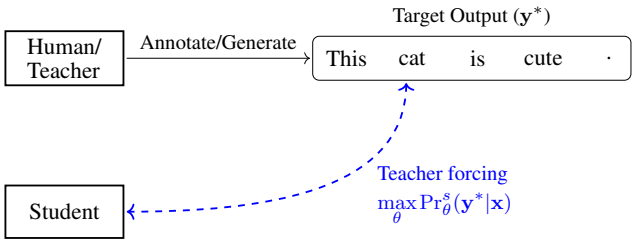
Vì sự giám sát của OPD có dạng một phân phối xác suất đầy đủ trên bộ từ vựng, học sinh nhận được một tín hiệu học rất dày và phong phú. Nó không chỉ học bước tiếp theo tốt nhất là gì, mà còn học cả xác suất của các lựa chọn hợp lệ khác. Phản hồi phong phú này rất thích hợp cho lập luận vì nó dạy mô hình học sinh cách gượng dậy sau khi đi chệch và duy trì tính nhất quán logic trên toàn bộ chuỗi đầu ra, thay vì chỉ xem một lời giải thích như một thành phần tách biệt. Tuy nhiên, áp dụng chưng cất on-policy đi kèm chi phí tính toán cao. Quá trình này đòi hỏi chạy một mô hình giáo viên lớn song song với học sinh trong lúc huấn luyện để tính các phân phối xác suất theo thời gian thực. Điều này tốn kém hơn chưng cất off-policy tiêu chuẩn, nơi đầu ra của mô hình giáo viên chỉ được sinh một lần và lưu trong một tập dữ liệu tĩnh.

Chú giải của người dịch: Đọc nhãn trong Hình 12.2

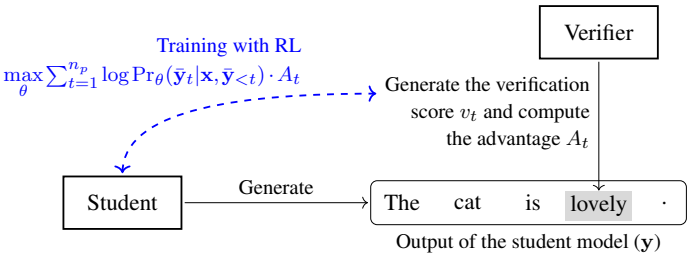
Human/Teacher: người/giáo viên; Annotate/Generate: gán nhãn/sinh ra; Target Output: đầu ra mục tiêu; Student: học sinh; Teacher forcing: ép buộc theo giáo viên (luôn đưa đáp án chuẩn làm ngữ cảnh); Training with RL: huấn luyện bằng học tăng cường; Verifier: bộ kiểm chứng; "Generate the verification score v_t and compute the advantage A_t": sinh điểm kiểm chứng v_t và tính ưu thế A_t; Output of the student model: đầu ra của mô hình học sinh; Minimizing the KL divergence: cực tiểu hóa độ phân kỳ KL; Teacher's/Student's predicted distribution: phân phối dự đoán của giáo viên/học sinh. Câu ví dụ "This cat is cute" / "The cat is lovely" nghĩa là "Con mèo này dễ thương".

12.4.4 Co giãn huấn luyện cho lập luận

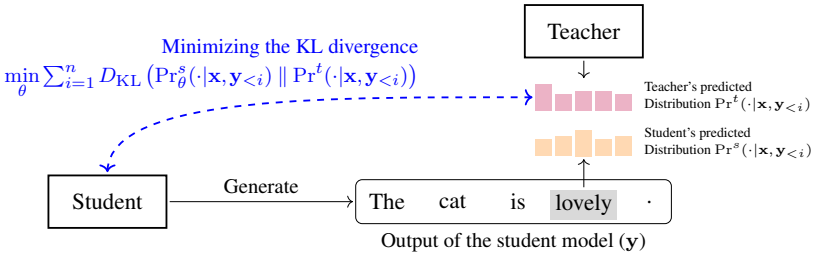
Co giãn là một chiến lược cốt lõi trong kỷ nguyên LLM. Khi xử lý các tác vụ lập luận phức tạp, co giãn quá trình huấn luyện là một ý tưởng tự nhiên. Ở đây, chúng ta bàn ngắn gọn về một số khía cạnh then chốt của việc co giãn huấn luyện cho các LLM lập luận: học tăng cường quy mô lớn, sinh dữ liệu tổng hợp, và co giãn độ dài ngữ cảnh.



(a) Supervised Fine-tuning



(b) Process Supervision



(c) On-policy Distillation

Hình 12.2: So sánh các phương pháp tinh chỉnh có giám sát, giám sát quá trình và chưng cất on-policy. Trong tinh chỉnh có giám sát (a), mô hình học sinh được huấn luyện off-policy để cực đại hóa likelihood của các đầu ra mục tiêu chuẩn do con người hoặc một mô hình giáo viên cung cấp. Trong giám sát quá trình (b), mô hình học sinh tự sinh ra đường lập luận của mình, và một bộ kiểm chứng đánh giá các bước tự sinh này để tính ưu thế. Trong chưng cất on-policy (c), học sinh cũng tự sinh ra đường lập luận của mình, nhưng thay vì phần thưởng vô hướng, một mô hình giáo viên cung cấp một phân phối xác suất dày, cấp token tại mỗi bước. Học sinh được cập nhật bằng cách cực tiểu hóa độ phân kỳ KL giữa phân phối dự đoán của chính nó và phân phối của giáo viên.

1. Học tăng cường quy mô lớn

Học tăng cường đã trở thành một cách tiếp cận chủ đạo để cải thiện khả năng lập luận "Hệ thống 2" của các LLM. Ví dụ, thành công của nhiều mô hình lập luận mạnh đến từ việc mở rộng quy mô học tăng cường chứ không chỉ dùng tinh chỉnh có giám sát. Một hiện tượng thú vị được quan sát trong quá trình huấn luyện dựa trên học tăng cường quy mô lớn là sự xuất hiện của "khoảnh khắc "à há" (*aha moment*)" [DeepSeek, 2025]. Không cần con người gán nhãn tường minh, các LLM học được cách tạm dừng, nhận ra khi mình đã mắc lỗi, hay thậm chí quay lui để thử một hướng khác. Những khả năng này khó khơi gợi được nếu chỉ dựa vào lượng dữ liệu gán nhãn hạn chế và huấn luyện có giám sát tiêu chuẩn. Khi khối lượng tính toán cho huấn luyện tăng lên, mô hình có thể dành những vết CoT dài hơn cho các bài toán khó hơn, và nhờ đó tự khám phá cách suy nghĩ sâu [Snell và cộng sự, 2024].

Chú giải của người dịch: Hệ thống 1 và Hệ thống 2

Đây là cách phân chia của nhà tâm lý học Daniel Kahneman: "Hệ thống 1" là lối nghĩ nhanh, theo trực giác (như trả lời ngay $2 + 2 = 4$); "Hệ thống 2" là lối nghĩ chậm, có chủ đích, từng bước (như giải một bài toán hình học). Các mô hình lập luận cố gắng mô phỏng Hệ thống 2 bằng cách "nghĩ ra giấy nháp" trước khi trả lời.

Các nghiên cứu thực nghiệm quy mô lớn gần đây đã cho thấy mối quan hệ giữa lượng tính toán dành cho học tăng cường và hiệu năng lập luận có thể được mô hình hóa bằng một đường cong hình chữ S [Khatri và cộng sự, 2025; Tan và cộng sự, 2026]. Một phát hiện then chốt từ định luật co giãn (*scaling law*) này là các lựa chọn huấn luyện khác nhau tác động lên đường cong theo những cách khác nhau. Chẳng hạn, tăng năng lực nền của LLM (ví dụ dùng kiến trúc lớn hơn hoặc MoE) và mở rộng độ dài cửa sổ ngữ cảnh có thể nâng giới hạn hiệu năng. Ngược lại, các cải tiến thuật toán, như thay đổi hàm mất mát hay điều chỉnh cách chuẩn hóa ưu thế, chủ yếu giúp tận dụng tốt hơn ngân sách tính toán sẵn có nhưng không cải thiện đáng kể hiệu năng cuối cùng.

Chú giải của người dịch: Đường cong chữ S

Đường cong chữ S (giống đồ thị hàm sigmoid $f(x) = \frac{1}{1+e^{-x}}$) tăng chậm lúc đầu, tăng nhanh ở giữa, rồi chững lại gần một "trần" (đường tiệm cận ngang). Câu trong sách có thể hiểu là: tăng kích thước mô hình hay độ dài ngữ cảnh làm *nâng cái trần* lên; còn các mẹo thuật toán chủ yếu làm đường cong *đi lên nhanh hơn* (đạt gần trần với ít tính toán hơn) mà không đổi trần.

Co giãn học tăng cường là việc khó. Thứ nhất, chi phí tính toán rất cao. Với mỗi mẫu huấn luyện, hệ thống thường thực hiện một lượt triển khai (chạy mô hình để suy luận) rồi gọi thêm một mô hình phần thưởng hoặc một LLM khác để đánh giá đầu ra được sinh ra. Khi số mẫu huấn luyện tăng lên, chi phí này trở nên cao hơn đáng kể so với tinh chỉnh có giám sát tiêu chuẩn. Gánh nặng tính toán này cũng khiến việc chạy thí nghiệm và thử–sai khó khăn hơn nhiều. Hơn nữa, các phương pháp học tăng cường nhìn chung phức tạp và đòi hỏi tinh chỉnh rất cẩn thận để đạt kết quả tốt. Ví dụ, chúng nhạy cảm với thiết lập siêu tham số và dữ liệu huấn luyện. Do đó, các phương pháp học tăng cường khá "mong manh". Khi huấn luyện ở quy mô khổng lồ, việc bảo đảm hệ thống chạy ổn định (chẳng hạn, bảo đảm hàm mất mát huấn luyện giảm đều đặn) không chỉ là một bài toán học máy, mà còn là một thách thức kỹ thuật.

2. Thêm dữ liệu tổng hợp cho lập luận

Vì các chú thích chất lượng cao của con người cho lập luận nhiều bước phức tạp rất khan hiếm, việc co giãn dữ liệu huấn luyện thường dựa vào sinh dữ liệu tổng hợp (*synthetic data generation*). Thay vì viết tay các chứng minh từng bước, các nhà nghiên cứu dùng những LLM mạnh để sinh ra các quỹ đạo lập luận đa dạng, sau đó lọc và chọn lọc để huấn luyện mô hình mới [Zelikman và cộng sự, 2022; Mukherjee và cộng sự, 2023].

Để sinh dữ liệu tổng hợp cho lập luận, một cách làm phổ biến là lấy mẫu nhiều đường lập luận từ các mô hình giáo viên cho một câu hỏi hạt giống cho trước [Wang và cộng sự, 2023c]. Sau đó ta có thể áp dụng các bộ kiểm chứng hoặc môi trường thực thi để đánh giá các đường này, và chỉ giữ lại những đường đi tới đáp án cuối cùng đúng qua các bước logic chặt chẽ. Ngoài ra, có thể dùng mô hình để sinh các biến thể của các bài toán có sẵn (ví dụ thay đổi giá trị số hay ràng buộc cấu trúc trong các tác vụ toán và lập trình) nhằm mở rộng phân phối của dữ liệu huấn luyện.

3. Co giãn độ dài ngữ cảnh

Khi các mô hình lập luận giải quyết những bài toán ngày càng phức tạp, độ dài của các đường lập luận đã trở thành một nút thắt cổ chai chính. Nhu cầu về cửa sổ ngữ cảnh lớn trong lập luận đến từ nhiều nguồn. Thứ nhất, để khơi gợi lối nghĩ sâu kiểu Hệ thống 2, bản thân prompt đầu vào thường cần ngữ cảnh rất dài. Điều này gồm các ràng buộc logic chi tiết, các quy tắc môi trường phức tạp, hay nhiều ví dụ CoT few-shot dài. Thứ hai, quá trình lập luận được sinh ra cho một tác vụ phức tạp có thể dễ dàng kéo dài hàng nghìn, thậm chí hàng chục nghìn token trước khi đi tới kết luận cuối cùng. Thứ ba, những tiến bộ gần đây về tác tử (*agent*) LLM cũng làm tăng nhu cầu lập luận. Lập luận kiểu tác tử thường kéo dài qua nhiều vòng tương tác, tạo ra nhu cầu còn lớn hơn về cửa sổ ngữ cảnh dài.

Nhu cầu này đã trực tiếp thúc đẩy ngành công nghiệp LLM phát triển các mô hình hỗ trợ cửa sổ ngữ cảnh khổng lồ. Chẳng hạn, nhiều LLM hiện hỗ trợ độ dài ngữ cảnh tới 128K token, và các mô hình tiên tiến hơn, như Claude 3.5 của Anthropic [Anthropic, 2024] và Gemini 1.5 của Google [Google, 2024], hỗ trợ hàng trăm nghìn token, thậm chí một triệu token. Bạn đọc quan tâm có thể tham khảo Chương 6 và Chương 8 để có thảo luận chi tiết hơn về các LLM ngữ cảnh dài.

Tuy nhiên, xử lý ngữ cảnh dài cho lập luận dẫn tới những thách thức mới vượt ra ngoài việc nén ngữ cảnh thông thường. Trong các tác vụ như tóm tắt một cuốn tiểu thuyết dài hay trích xuất ý chính từ tài liệu, LLM có thể nén thông tin và bỏ qua các chi tiết nhỏ. Ngược lại, lập luận đòi hỏi tính liên tục logic chặt chẽ, độ chính xác cao. Mỗi bước trung gian là một mắt xích quan trọng trong một chuỗi phụ thuộc lẫn nhau rất mạnh. Nếu mô hình "quên" một giả thiết cụ thể được nêu trong prompt, hoặc mất dấu một biến đã tính ở bước 2 khi đang thực hiện bước 50, toàn bộ phép suy diễn sẽ sụp đổ. Vì vậy, co giãn ngữ cảnh cho lập luận không chỉ là mở rộng bộ nhớ đệm KV (*KV cache*) để chứa nhiều token hơn. Nó đòi hỏi mô hình truy xuất thông tin một cách đáng tin cậy qua những khoảng cách xa và duy trì tính nhất quán logic chặt chẽ.

12.5 Đánh giá các LLM lập luận

Trong mục này, chúng ta bàn về cách đánh giá các LLM lập luận. Việc này tương tự đánh giá các LLM tiêu chuẩn [Chang và cộng sự, 2024b]. Tuy nhiên, các bài toán lập luận phức tạp đến mức ta thường cần đánh giá chi tiết hơn cùng những cân nhắc đặc biệt khác.

12.5.1 Đánh giá dựa trên kết quả và đánh giá dựa trên quá trình

Khi đánh giá một LLM lập luận tốt đến đâu, phương pháp dễ nhất và phổ biến nhất là **đánh giá dựa trên kết quả** (*outcome-based evaluation*). Trong cách này, ta chỉ nhìn vào câu trả lời cuối cùng mà mô hình tạo ra. Nếu câu trả lời cuối cùng khớp với đáp án chuẩn, mô hình được xem là đúng. Phương pháp này được dùng rộng rãi trong các bộ đánh giá chuẩn (*benchmark*) lập luận phổ biến, như GSM8K [Cobbe và cộng sự, 2021] và MATH [Hendrycks và cộng sự, 2021b]. Tuy nhiên, đánh giá dựa trên kết quả gặp vấn đề "đúng đáp án nhưng sai lý do", còn gọi là dương tính giả (*false positive*). Một mô hình có thể sinh ra các bước logic sai sót nhưng tình cờ đi tới con số cuối cùng đúng. Nếu chỉ kiểm tra kết quả cuối cùng, ta có thể kết luận sai rằng mô hình có năng lực lập luận mạnh trong khi nó chỉ gặp may.

Để đánh giá chi tiết hơn, ta có thể dùng **đánh giá dựa trên quá trình** (*process-based evaluation*). Thay vì chỉ kiểm tra câu trả lời cuối cùng, ta có thể xem xét toàn bộ đường lập luận từng bước [Wang và cộng sự, 2024c; Xia và cộng sự, 2025]. Một mô hình chỉ được xem là đúng nếu cả logic trung gian lẫn kết luận cuối cùng đều chặt chẽ. Đây là một phương pháp lý tưởng nhưng rất khó thực hiện trong thực tế. Thách thức chính là "dữ liệu vàng" ở cấp bước rất khan hiếm và tốn kém để tạo ra. Hơn nữa, với bất kỳ bài toán phức tạp nào, số đường lập luận hợp lệ lớn theo hàm mũ. Không thể liệt kê thủ công mọi bước trung gian đúng cho mọi đường mà một mô hình có thể đi.

Để đánh giá dựa trên quá trình thực tế hơn, ta có thể thu hẹp phạm vi để đánh giá các quyết định lập luận cục bộ. Một cách là quy bài toán về một tác vụ đánh giá bước tiếp theo đơn giản: cho một chuỗi các bước trước đó đúng, ta kiểm tra xem mô hình lập luận có chọn hoặc sinh đúng bước logic tiếp theo hay không. Để làm điều này, có thể dùng trực tiếp các PRM để đánh giá chất lượng mỗi bước do LLM sinh ra. Phương pháp này cũng liên quan đến việc đánh giá mô hình phần thưởng [Lambert và cộng sự, 2025]: thay vì xem xét đầu ra của mô hình lập luận, ta đánh giá chất lượng của các PRM. Ngoài ra, ta có thể xét các năng lực nền tảng của mô hình lập luận, chẳng hạn khả năng phát hiện lỗi trong một đường lập luận cho trước. Điều này cũng có thể được xem là một phép thử năng lực siêu lập luận (*meta-reasoning*): ta yêu cầu mô hình không chỉ lập luận, mà còn đánh giá chính quá trình lập luận [Zeng và cộng sự, 2024].

Trong thực tế, đánh giá dựa trên quá trình không dựa trên một tiêu chuẩn duy nhất. Ta có thể đánh giá chất lượng của một bước lập luận từ nhiều góc độ khác nhau, như tính nhất quán logic, tính mạch lạc và tính khách quan [Lee và Hockenmaier, 2025]. Tuy nhiên, dù đánh giá dựa trên quá trình chi tiết hơn nhiều, nó đi kèm chi phí gán nhãn cao hơn nhiều, và thường chưa đủ vững chắc. Vì vậy, phần lớn nghiên cứu thực tế vẫn dựa vào đánh giá dựa trên kết quả đơn giản hơn.

12.5.2 Các độ đo đánh giá

Dưới đây chúng tôi liệt kê một số độ đo dùng để đánh giá các LLM lập luận.

- **Khớp chính xác và độ chính xác** (*Exact Match and Accuracy*): Đây là độ đo đơn giản nhất và được dùng rộng rãi nhất cho các tác vụ có đáp án cuối cùng xác định, như các bài toán đố. Giả sử ta có một tập dữ liệu $\mathcal{D}$ gồm N bài toán. Với bài thứ i, ta trích câu trả lời cuối cùng $\hat{a}_i$ từ đường lập luận được sinh ra và so sánh với đáp án chuẩn a_i^*. Độ chính xác được định nghĩa là:

$$\text{Accuracy} \;=\; \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}(\hat{a}_i = a_i^*). \tag{12.30}$$

  Trong thực tế, phương pháp khớp chính xác (kiểm tra hai chuỗi có giống hệt nhau không) có thể quá nghiêm ngặt, chẳng hạn ta sẽ loại "0.5" khi đáp án chuẩn là "1/2". Vì vậy, các nhà nghiên cứu

thường áp dụng các đoạn mã dựa trên quy tắc để chuẩn hóa câu trả lời về một định dạng chính tắc trước khi so sánh.

- **Pass@k**: Vì LLM mang tính xác suất, một lần giải mã duy nhất có thể thất bại trong lập luận chỉ do một lỗi nhỏ. Pass@k đo xác suất mô hình tìm được ít nhất một câu trả lời đúng nếu được phép sinh k đường lập luận độc lập [Chen và cộng sự, 2021]. Độ đo này đặc biệt phổ biến trong sinh mã và các tác vụ toán phức tạp. Với bài thứ i trong $\mathcal{D}$, nếu ta lấy mẫu $N_s \geq k$ đường và có C_i đường chứa câu trả lời đúng, điểm Pass@k được định nghĩa là:

$$\text{Pass@}k \;=\; \frac{1}{N}\sum_{i=1}^{N}\left(1 - \frac{\binom{N_s - C_i}{k}}{\binom{N_s}{k}}\right). \tag{12.31}$$

  Để sinh nhiều đường lập luận khác nhau, ta có thể dùng các phương pháp lấy mẫu mô tả ở Mục 12.3.2.

Chú giải của người dịch: Hàm chỉ thị và công thức Pass@k

$\mathbf{1}(\cdot)$ là *hàm chỉ thị*: bằng 1 nếu điều kiện trong ngoặc đúng, bằng 0 nếu sai; nên (12.30) chỉ là "số bài làm đúng chia tổng số bài". Công thức (12.31) dùng tổ hợp lớp 11: $\binom{N_s}{k}$ là số cách chọn k lời giải từ N_s lời giải đã sinh; $\binom{N_s - C_i}{k}$ là số cách chọn mà *toàn bộ* k lời giải đều sai. Tỉ số là xác suất "chọn k cái đều trượt", lấy 1 trừ đi được xác suất "có ít nhất một cái đúng" (phép lấy biến cố đối). Ví dụ $N_s = 5$, $C_i = 2$, $k = 2$: xác suất đều sai là $\binom{3}{2}/\binom{5}{2} = 3/10$, vậy Pass@2 của bài này là 0.7.

- **Cons@k (Consensus@k, đồng thuận@k)**: Khác với Pass@k, vốn cần một "nhà tiên tri" (*oracle*) để kiểm chứng xem mỗi câu trả lời trong k câu được lấy mẫu có đúng không, Cons@k xây dựng một câu trả lời đồng thuận mà không cần xác định tính đúng sai của từng mẫu. Cụ thể, ta sinh k đường lập luận, trích các câu trả lời cuối cùng của chúng, rồi chọn câu trả lời xuất hiện nhiều nhất bằng bỏ phiếu đa số (*majority voting*) [Wang và cộng sự, 2023c]. Câu trả lời đồng thuận $\hat{a}_i^{\text{maj}}$ cho bài thứ i được định nghĩa là:

$$\hat{a}_i^{\text{maj}} \;=\; \arg\max_{a_i} \sum_{j=1}^{k} \mathbf{1}(\hat{a}_i^{j} = a_i), \tag{12.32}$$

  trong đó $\hat{a}_i^{j}$ là câu trả lời cuối cùng được trích từ đường lập luận thứ j. Độ chính xác cuối cùng sau đó nhận được bằng cách so sánh $\hat{a}_{\text{maj}}$ với đáp án chuẩn:

$$\text{Cons@}k \;=\; \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}(\hat{a}_i^{\text{maj}} = a_i^{*}). \tag{12.33}$$

  Cons@k cung cấp một cách đánh giá thực tế cho các hệ thống lập luận ngoài đời thực, vì nó đo xem mô hình có *nhất quán* đi tới câu trả lời đúng hay không. So với Pass@k, vốn đánh giá xem có ít nhất một phản hồi được lấy mẫu là đúng hay không, Cons@k nhấn mạnh độ tin cậy của lập luận bằng cách ưu tiên những câu trả lời được ủng hộ một cách nhất quán qua nhiều đường lập luận được lấy mẫu.

- **LLM làm giám khảo** (*LLM-as-a-Judge*): Với các tác vụ lập luận mở, như lập kế hoạch kiểu tác tử hay các câu hỏi phức tạp miền mở, việc khớp chính xác dựa trên quy tắc thường thất bại. Trong những trường hợp đó, ta có thể dùng LLM làm giám khảo tự động. Chẳng hạn, có thể dùng prompt yêu cầu một LLM giám khảo đánh giá các bước trung gian được sinh ra và chấm điểm

chúng dựa trên những tiêu chí định sẵn. Một cách khác, có thể dùng LLM để so sánh hai mô hình lập luận. Giám khảo được xem đầu ra của cả hai mô hình cho cùng một truy vấn và quyết định đường lập luận nào tốt hơn, hoặc hai bên hòa. Tỉ lệ thắng (*win rate*) khi đó phản ánh tỉ lệ phần trăm số lần một mô hình cụ thể được ưa chuộng hơn.

Chú giải của người dịch: Pass@k hay Cons@k?

Giả sử với một bài toán, mô hình sinh $k = 5$ lời giải cho các đáp số $12, 15, 12, 7, 12$, và đáp án đúng là 15. Theo Pass@5, bài này được tính là *đạt* (vì có một lời giải ra 15). Theo Cons@5, đáp án đồng thuận là 12 (xuất hiện 3 lần) nên bài này *không đạt*. Pass@k giống như "được thi k lần, chỉ cần đỗ một lần"; còn Cons@k giống như "nộp đáp án mà đa số các lần làm cho ra" — điều này sát với thực tế hơn khi ta không có sẵn đáp án để kiểm tra.

12.5.3 Các vấn đề còn bỏ ngỏ

Một vấn đề là nhiễm bẩn dữ liệu (*data contamination*). Khi các mô hình ngôn ngữ được huấn luyện trên những tập dữ liệu ngày càng khổng lồ thu thập từ internet, việc giữ dữ liệu kiểm tra nằm ngoài quá trình huấn luyện trở nên rất khó. Nếu một mô hình đã thấy các câu hỏi của một bộ đánh giá chuẩn lập luận (như MATH hay GSM8K) trong giai đoạn tiền huấn luyện, nó có thể chỉ đơn giản đọc lại một lời giải đã học thuộc thay vì thực sự lập luận tại chỗ. Điều này khiến khó xác định điểm số cao trên bộ đánh giá chuẩn phản ánh năng lực giải quyết vấn đề thực sự hay chỉ là trí nhớ tốt. Để giải quyết vấn đề này, cộng đồng được khuyến khích khám phá những bộ đánh giá chuẩn mới mà các bài toán được cập nhật liên tục, cũng như các tập dữ liệu giữ riêng không bao giờ công bố.

Một vấn đề khác là độ vững (*robustness*). Các mô hình lập luận hiện nay đôi khi rất mong manh. Một mô hình có thể giải hoàn hảo một bài toán phức tạp, nhưng thất bại hoàn toàn nếu chính câu hỏi đó được diễn đạt lại một chút hoặc có thêm các chi tiết không liên quan vào prompt. Sự nhạy cảm này gợi ý rằng mô hình đôi khi dựa vào các khuôn mẫu văn bản bề mặt thay vì thực sự hiểu logic bên dưới. Để lập luận vững hơn, ta cần các LLM dễ dàng thích ứng với những tác vụ mới, thậm chí khó hơn.

Hơn nữa, LLM có thể sinh ra các đường lập luận bị ảo giác (*hallucination*) hoặc không trung thực (*unfaithful*). Ví dụ, chúng có thể bịa ra những quy tắc toán học giả để đi tới kết luận. Trong một số trường hợp, văn bản được sinh ra trông như một phép suy diễn logic hoàn hảo, nhưng thực ra mô hình đi tới câu trả lời qua một cơ chế ẩn (như đoán may hay một lối tắt đã học thuộc). Sự không trung thực này cho thấy một khoảng cách giữa văn bản mô hình xuất ra và các biểu diễn nội tại thực sự của nó. Trong các thiết lập học tăng cường, điều này thường liên quan đến vấn đề lách phần thưởng (*reward hacking*). Nếu mô hình chỉ được thưởng cho câu trả lời cuối cùng, chúng thường học cách "gian lận" hệ thống phần thưởng bằng logic sai sót. Vì vậy, việc đưa giám sát từng bước vào học tăng cường, như giám sát quá trình, là rất có giá trị để kiểm tra xem mỗi bước trung gian có thực sự đúng hay không.

Vấn đề thứ tư là bão hòa bộ đánh giá chuẩn (*benchmark saturation*). Tiến bộ nhanh chóng của các LLM lập luận khiến các bộ đánh giá chuẩn thông thường nhanh chóng trở nên lỗi thời. Các mô hình hiện đạt điểm gần như tuyệt đối trên những bài kiểm tra mà chỉ một hai năm trước còn được xem là cực kỳ khó. Sự bão hòa này khiến việc so sánh các mô hình mới hay đo tiến bộ thực sự của lĩnh vực trở nên khó khăn. Vì vậy, các nhà nghiên cứu cần phát triển những bài kiểm tra khó hơn để đánh giá năng lực suy diễn logic thực sự trên các bài toán mới. Lưu ý rằng lập luận ở mức con người thường mang tính mở và tương tác. Một cách khác để kiểm tra các mô hình lập luận có thể là dùng chúng trong các tác tử tự hành, vốn có thể khám phá, dùng công cụ bên ngoài, và tự sửa sai.

12.6 Bàn thêm

Bây giờ chúng ta bàn về một số vấn đề thú vị liên quan đến các LLM lập luận.

12.6.1 Lập luận bên ngoài và lập luận bên trong

Trong các thảo luận trước, ta đã giả định rằng quá trình lập luận có thể được nắm bắt tốt bằng một đường lập luận dạng văn bản (ví dụ CoT). Điều này đặt ra một câu hỏi căn bản: Văn bản CoT được sinh ra có thực sự phản ánh cách mô hình lập luận hay không? Câu hỏi này dẫn ta tới việc xem xét sự khác biệt giữa lập luận được *ngoại hiện* ra văn bản và quá trình "suy nghĩ" thực sự bên trong.

Khi ta yêu cầu một LLM suy nghĩ từng bước, ta đang buộc nó ngoại hiện quá trình tính toán thành các token rời rạc. Tuy nhiên, ta không thể đơn giản xem chuỗi văn bản này là quá trình lập luận thực sự. Một mặt, một LLM có thể sinh ra một CoT chi tiết và chặt chẽ về logic, nhưng việc ra quyết định thực sự có thể đã diễn ra ngầm trong các biểu diễn ẩn của ngay vài token đầu tiên. Mặt khác, một mô hình có thể tạo ra một CoT sai sót hoặc vô nghĩa nhưng vẫn xuất ra câu trả lời cuối cùng đúng, cho thấy các biểu diễn bên trong của nó đang theo dõi tác vụ theo cách khác với những gì văn bản được sinh ra gợi ý.

Các nhà nghiên cứu đã chỉ ra rằng các lời giải thích CoT có thể gây hiểu lầm [Turpin và cộng sự, 2023; Chen và cộng sự, 2025]. Ví dụ, LLM có thể có thiên hướng sinh một CoT nghe rất logic để đi tới một câu trả lời sai, dù câu trả lời đúng đã được nhắc đến trong prompt. Để khám phá LLM thực sự suy nghĩ như thế nào, các nhà nghiên cứu chuyển sang phân tích kích hoạt nơ-ron và khả năng diễn giải cơ chế (*mechanistic interpretability*). Mục tiêu là hiểu điều gì xảy ra bên trong các tầng ẩn của một LLM trong quá trình sinh. Ví dụ, trong công trình của Fraser-Taliente và cộng sự [2026], các bộ tự mã hóa ngôn ngữ tự nhiên (*natural language auto-encoder*) được dùng để chuyển các giá trị kích hoạt thành văn bản ngôn ngữ tự nhiên. Một bộ tự mã hóa ngôn ngữ tự nhiên gồm một *bộ diễn đạt kích hoạt* (*activation verbalizer*) và một *bộ tái tạo kích hoạt* (*activation reconstructor*), được dùng để ánh xạ trực tiếp các giá trị kích hoạt trong dòng residual (*residual stream*) bên trong thành văn bản con người đọc được và ngược lại. Bằng cách này, ta có thể giải mã các giá trị kích hoạt để "đọc" suy nghĩ của LLM.

Một hướng nghiên cứu khác tập trung vào lập luận không cần văn bản, gọi là lập luận tiềm ẩn (*latent reasoning*) hay lập luận ngầm (*implicit reasoning*). Trong lập luận tiêu chuẩn, như ta đã định nghĩa, một đường lập luận gồm một chuỗi các bước bằng văn bản. Trong lập luận tiềm ẩn, quá trình này diễn ra bên trong trước khi mô hình "nói". Điều này đạt được hoặc bằng cách thay các token rời rạc bằng các vectơ liên tục, hoặc bằng cách sinh ra các từ văn bản thực sự "ở hậu trường" và giấu chúng khỏi người dùng. Dưới đây chúng tôi trình bày một số phương pháp như vậy.

- Một trong những ý tưởng đơn giản nhất là dùng "token tạm dừng (*pause token*)". Như thường lệ trong sinh tự hồi quy, mô hình ngôn ngữ có một bước tính toán nội bộ cho mỗi từ nó xuất ra. Điều thú vị là người ta phát hiện rằng nếu ta đưa vào mô hình các token giả (gọi là token tạm dừng), mô hình có thêm thời gian tính toán để xử lý thông tin trong các tầng ẩn trước khi phải xuất ra câu trả lời cuối cùng [Goyal và cộng sự, 2024]. Điều này cho thấy LLM có thể thực hiện những tính toán "thầm lặng", hay suy nghĩ tiềm ẩn, mà không cần sinh ra từ ngữ thực sự.

- Một cách tiếp cận khác dùng chưng cất tri thức để nén lập luận tường minh vào các trạng thái ẩn. Chẳng hạn, phương pháp *chuỗi suy nghĩ ngầm* (*implicit chain of thought*) huấn luyện một mô hình học sinh bắt chước trực tiếp các trạng thái ẩn bên trong của một mô hình giáo viên lớn hơn đang "nghĩ thành lời" [Deng và cộng sự, 2023]. Kết quả là mô hình học sinh học được cách bỏ

qua việc sinh văn bản. Ngoài ra, LLM cũng có thể tự học từ chính nó bằng các kỹ thuật tự chưng cất (*self-distillation*) [Shen và cộng sự, 2025]: ta có thể buộc quá trình suy nghĩ tiềm ẩn khớp với các trạng thái ẩn cuối cùng của lập luận dạng văn bản. Trong cả hai phương pháp, suy nghĩ tiềm ẩn được hiện thực hóa như một không gian toán học liên tục: thay vì sinh một chuỗi văn bản đọc được từng bước, mô hình tính các bước lập luận bằng các biểu diễn ẩn dạng vectơ.

- Một phương pháp khác để lập luận trong không gian tiềm ẩn liên tục là dùng mô thức **chuỗi suy nghĩ liên tục** (*chain of continuous thought*) (**COCONUT**) [Hao và cộng sự, 2025]. Trong sinh văn bản tiêu chuẩn, đầu ra của tầng cuối được chuyển thành một token rời rạc, rồi token đó được đưa trở lại tầng đầu tiên cho bước tiếp theo. Quá trình này buộc mô hình nén tính toán bên trong của mình vào một token duy nhất. COCONUT bỏ qua bước này trong giai đoạn lập luận. Nó lấy vectơ đầu ra của tầng cuối và đưa thẳng trở lại tầng đầu vào như một *suy nghĩ liên tục*. Bằng cách lặp lại quá trình này, mô hình xây dựng một chuỗi các vectơ liên tục trước khi bắt đầu sinh câu trả lời văn bản cuối cùng. Vì vectơ liên tục có thể chứa nhiều thông tin hơn token rời rạc, mô hình có thể khám phá nhiều đường lập luận cùng lúc mà không phải cam kết sớm vào một lựa chọn token cụ thể. Cách tiếp cận này cho thấy LLM không nhất thiết phải dựa vào ngôn ngữ của con người để giải các bài toán phức tạp. Bằng cách giữ quá trình lập luận trong một không gian liên tục, chúng có thể tính ra câu trả lời hiệu quả hơn.

- Thay vì lập luận trong không gian liên tục, ta cũng có thể dùng các token văn bản bình thường để biểu diễn suy nghĩ bên trong. Ví dụ, trong phương pháp Quiet-STaR [Zelikman và cộng sự, 2024], mô hình được huấn luyện để "nghĩ thầm" ngay giữa lúc đang đọc hay viết văn bản bình thường. Ý tưởng rất đơn giản: trước khi dự đoán token tiếp theo, mô hình sinh ra một chuỗi ngắn các token "suy nghĩ" ẩn. Bạn có thể hình dung điều này như một độc thoại nội tâm hay một tờ nháp nhanh trong đầu mà người dùng không bao giờ thấy. Sau khi sinh ra các suy nghĩ ẩn này, LLM dùng chúng để hỗ trợ đoán từ tiếp theo thực sự. Nếu các suy nghĩ này giúp LLM dự đoán đúng từ, LLM được thưởng và học được rằng cách nghĩ này hữu ích. Nếu các suy nghĩ vô ích hoặc gây rối, LLM học cách loại bỏ chúng.

Chú giải của người dịch: Token rời rạc và vectơ liên tục

Khi chọn một token, mô hình phải "chốt" một từ duy nhất trong bộ từ vựng, giống như làm tròn số $3{,}14159\ldots$ thành 3 — phần thông tin còn lại bị mất. Vectơ ở tầng cuối thì giống như con số chưa làm tròn: nó có thể đồng thời "nghiêng" về vài khả năng (chẳng hạn 60% hướng A, 40% hướng B). COCONUT giữ nguyên vectơ chưa làm tròn đó làm đầu vào cho bước sau, nên không mất thông tin, nhưng đổi lại con người không đọc được các "suy nghĩ" này.

12.6.2 Ghi nhớ và lập luận

Khi bàn về LLM, một câu hỏi thường được đặt ra: Mô hình có thực sự "suy nghĩ", hay nó chỉ có một bộ nhớ lớn và đang đọc lại một câu trả lời tương tự từng thấy trong dữ liệu huấn luyện? Điều này dẫn tới cuộc thảo luận so sánh lập luận và ghi nhớ (*memorization*) trong LLM.

Nói đơn giản, sự khác biệt giữa hai điều này có thể tóm tắt như sau:

- **Ghi nhớ là nhìn về phía sau.** Mô hình điều chỉnh tham số của nó để học các khuôn mẫu từ dữ liệu huấn luyện. Khi gặp một đầu vào tương tự, mô hình chỉ đơn giản "phát lại" các khuôn mẫu đã học.

- **Lập luận là tiến về phía trước.** Thay vì tìm một câu trả lời làm sẵn, mô hình áp dụng các quy tắc

và logic tổng quát để xây dựng một chuỗi suy nghĩ từng bước. Cốt lõi của lập luận là *khả năng tổng quát hóa*: kể cả khi đối mặt với một bài toán hoàn toàn mới chưa từng xuất hiện trong tập huấn luyện, mô hình vẫn có thể giải nó bằng các công cụ logic.

Trong những nghiên cứu học sâu thời kỳ đầu, ranh giới này không phải lúc nào cũng rõ ràng. Ví dụ, Zhang và cộng sự [2021b] chỉ ra rằng các mạng nơ-ron sâu có năng lực ghi nhớ khổng lồ, dễ dàng học thuộc cả dữ liệu nhiễu với nhãn hoàn toàn ngẫu nhiên. Với LLM, do kích thước mô hình rất lớn, chúng không chỉ có thể ghi nhớ Wikipedia mà cả những chi tiết ẩn sâu trong các kho mã nguồn.

Khả năng ghi nhớ này thường "đội lốt" lập luận. Ví dụ, nếu ta đưa cho một LLM một bài toán thuật toán kinh điển nhưng sửa nhẹ các con số, LLM có thể dựa vào "quán tính trí nhớ" và vẫn xuất ra câu trả lời gốc, do đó mắc một lỗi cơ bản. Hiện tượng này thường được gọi là "con vẹt ngẫu nhiên (*stochastic parrots*)" [Bender và cộng sự, 2021], nghĩa là LLM bắt chước một cách mù quáng và chắp vá các đoạn văn bản đã ghi nhớ mà không thực sự hiểu logic bên dưới.

Để thấy rõ hơn sự khác biệt giữa ghi nhớ và lập luận, ta có thể xem xét hai khía cạnh.

1. Hiện tượng grokking và khả năng tổng quát hóa

Khi tìm hiểu cách mô hình chuyển từ ghi nhớ sang hiểu thực sự, các nhà nghiên cứu đã có một phát hiện thú vị gọi là "grokking (*grokking*)" [Power và cộng sự, 2022]. Họ huấn luyện một mô hình Transformer nhỏ học các phép toán hai ngôi trong đại số trừu tượng (như phép cộng modulo). Họ nhận thấy một hiện tượng rất thú vị: ở đầu quá trình huấn luyện, mô hình nhanh chóng đạt độ chính xác 100% trên tập huấn luyện, nhưng độ chính xác trên tập kiểm tra gần như bằng 0. Ở giai đoạn này, rõ ràng mô hình chỉ đang học vẹt các đẳng thức. Tuy nhiên, nếu không dừng huấn luyện mà để mô hình tiếp tục lặp qua dữ liệu rất lâu sau thời điểm quá khớp (*overfitting*), một điều bất ngờ xảy ra. Đột nhiên, độ chính xác trên tập kiểm tra vọt lên 100% trong một thời gian rất ngắn. Mô hình dường như trải qua một "khoảnh khắc à há".

Các nghiên cứu sâu hơn cho thấy trước "khoảnh khắc à há" này, tham số mô hình đang ghi nhớ một cách lộn xộn các đẳng thức cụ thể (như $2 + 3 = 5$) [Nanda và cộng sự, 2023]. Nhưng khi grokking xảy ra, bên trong mạng hình thành một biểu diễn toán học tổng quát dùng các hàm lượng giác và biến đổi Fourier (*Fourier transform*). Nó đã tìm ra một thuật toán phổ quát có thể giải thích toàn bộ dữ liệu.

Chú giải của người dịch: Phép cộng modulo và lượng giác

Phép cộng modulo p là "cộng rồi lấy số dư khi chia cho p", giống như cộng giờ trên mặt đồng hồ: 9 giờ $+ 5$ giờ $= 2$ giờ (modulo 12). Nếu đặt các số $0, 1, \ldots, p-1$ thành các điểm trên đường tròn lượng giác, ứng với góc $2\pi a/p$, thì phép cộng modulo chính là *cộng góc*, và có thể tính bằng công thức lớp 11 $\cos(\alpha + \beta) = \cos\alpha\cos\beta - \sin\alpha\sin\beta$. Mạng nơ-ron sau khi grokking đã tự "khám phá" ra cách làm gần giống như vậy, thay vì học thuộc bảng cộng.

Thí nghiệm này cho ta biết lập luận không tự nhiên xuất hiện từ hư không. Nó thường nổi lên như một trạng thái tổng quát hóa ở mức cao khi việc ghi nhớ chạm tới nút thắt, hoặc sau quá trình huấn luyện kéo dài, khi mạng tổ chức lại các tham số của mình để tìm ra lời giải thích phổ quát nhất.

2. Đánh đổi về chi phí

Từ góc nhìn của lý thuyết thông tin và hiệu quả tính toán, LLM tự nhiên cân bằng giữa ghi nhớ và lập luận dựa trên chi phí. Một mặt, ghi nhớ đòi hỏi nhiều dung lượng tham số hơn. Để ghi nhớ mọi sự kiện trên thế giới, kích thước mô hình sẽ phải tăng vô hạn. Mặt khác, lập luận đòi hỏi nhiều bước tính toán hơn lúc suy luận. Chỉ cần mô hình ghi nhớ các quy tắc tổng quát của phép cộng và phép nhân, nó có thể giải nhiều bài toán bằng cách thực hiện các bước tính toán như vậy trong lúc sinh.

Vì vậy, LLM thường đi theo "con đường ít trở ngại nhất". Với các sự kiện phổ biến, truy xuất câu trả lời trực tiếp từ tham số (hay bộ nhớ) là cách nhanh nhất. Nhưng với các tác vụ hiếm gặp, nhiều bước, dùng lối nghĩ Hệ thống 2 là cách phù hợp hơn.

Điều này cũng giải thích vì sao các kỹ thuật như sinh văn bản tăng cường truy xuất (*retrieval-augmented generation*) (RAG) trở nên phổ biến. Bản chất của RAG là chuyển gánh nặng ghi nhớ sang một cơ sở dữ liệu bên ngoài, nhờ đó ta có thể giải phóng không gian tham số cho lập luận. Bằng cách tách việc truy xuất tri thức khỏi suy diễn logic, LLM không còn cần ghi nhớ những sự kiện bên ngoài liên tục thay đổi. Thay vào đó, nó biết cách tra cứu và xử lý chúng.

12.6.3 LLM có thực sự lập luận được không?

Câu hỏi liệu LLM có thực sự lập luận được hay không là một trong những chủ đề gây tranh cãi nhất trong AI hiện nay. Khi thấy một LLM giải hoàn hảo một bài toán hay lập một lịch trình phức tạp, ta rất dễ tin rằng nó hiểu thế giới giống như con người. Tuy nhiên, để trả lời câu hỏi này, ta phải xem các mô hình này thực sự hoạt động như thế nào.

Về bản chất, LLM hoạt động bằng cách dự đoán từ tiếp theo. Vì đã xử lý một lượng văn bản khổng lồ, các mô hình này nắm vững hoàn toàn các quy tắc, ngữ pháp và cấu trúc của ngôn ngữ con người. Trong khoa học nhận thức, điều này được gọi là năng lực ngôn ngữ hình thức (*formal linguistic competence*) [Chomsky, 1965; Mahowald và cộng sự, 2024]. Chúng biết chính xác các từ nên được sắp xếp theo thứ tự hợp lý như thế nào. Tuy nhiên, chúng gặp khó khăn với năng lực ngôn ngữ chức năng (*functional linguistic competence*), tức là khả năng dùng ngôn ngữ để hiểu và xoay xở trong thế giới thực [Kambhampati và cộng sự, 2024]. Khác với con người, vốn học bằng cách tương tác vật lý với môi trường, LLM chỉ biết các khuôn mẫu ẩn của văn bản. Vì không được "neo" vào thế giới thực, chúng thiếu hiểu biết thực sự về không gian vật lý và quan hệ nhân–quả chặt chẽ [Mahowald và cộng sự, 2024].

Hạn chế này trở nên rõ rệt khi ta kiểm tra LLM trên các tác vụ lập kế hoạch [Valmeekam và cộng sự, 2023]. Ví dụ, với các bài toán lập kế hoạch kinh điển, như di chuyển các khối màu trên bàn để xếp thành một chồng cụ thể, LLM thường giải được nếu bài toán dùng những từ thông dụng mà nó đã thấy trong dữ liệu huấn luyện. Nhưng nếu tên các khối và các hành động được thay bằng những từ vô nghĩa, hiệu năng giảm mạnh [Kambhampati, 2024]. Một hệ thống lập luận thực sự sẽ không quan tâm đến tên gọi, vì logic toán học bên dưới hoàn toàn giống nhau. LLM thất bại vì khuôn mẫu văn bản quen thuộc của nó bị phá vỡ.

Hơn nữa, các kỹ thuật như prompt CoT buộc mô hình xuất ra một quá trình từng bước. Điều này thể hiện phần nào hành vi suy nghĩ chậm rãi, có cân nhắc của con người. Nhưng thực ra, LLM vẫn chỉ đang tiến thẳng về phía trước một cách nghiêm ngặt và dự đoán xem một lời giải thích logic *thường trông như thế nào*. Nếu LLM mắc lỗi hoặc mang một giả định thiên lệch, nó thường tự tin sinh ra một lời giải thích sai sót chỉ để biện minh cho câu trả lời sai của mình [Chen và cộng sự, 2025].

Rốt cuộc, để trả lời LLM có "thực sự" lập luận được hay không, trước hết ta cần định nghĩa lập luận thực sự nghĩa là gì. Nếu ta kỳ vọng LLM suy nghĩ giống hệt con người, thì từ thảo luận trên rõ ràng có những khác biệt. Tuy nhiên, nếu xem lập luận thuần túy như một quá trình chức năng, câu trả lời sẽ

khác. Lập luận có thể được định nghĩa đơn giản là khả năng tiếp nhận một bài toán phức tạp, chia nhỏ nó, áp dụng các quy tắc logic, và đi tới một kết luận đúng. Khi một LLM giải đúng một phương trình toán từng bước, hay khi nó viết một đoạn mã logic để xử lý dữ liệu, nó đang thực thi một chuỗi logic hợp lệ. Ngay cả khi khả năng này nảy sinh từ việc huấn luyện các mạng nơ-ron quy mô lớn chứ không phải từ nhận thức giống con người, kết quả vẫn là giải quyết vấn đề thành công. Khi LLM ngày càng lớn hơn và học từ nhiều dữ liệu hơn, chúng vượt ra khỏi việc chỉ ghi nhớ câu trả lời và bắt đầu nắm bắt các quy tắc logic bên dưới.

12.7 Tóm tắt

Trong chương này, chúng ta đã trình bày cách áp dụng LLM vào lập luận, vốn được xem là một trong những tác vụ thách thức nhất trong NLP. Cụ thể, chúng ta đã bàn về các phương pháp cải thiện khả năng lập luận trong giai đoạn huấn luyện và giai đoạn suy luận. Chúng ta cũng đã tìm hiểu các phương pháp đánh giá LLM lập luận, cùng một số vấn đề còn bỏ ngỏ.

Phát hiện rằng LLM có thể vạch ra tường minh các bước lập luận của mình đã truyền cảm hứng cho những hướng nghiên cứu và ứng dụng thực tiễn mới. Thay vì chỉ xem lập luận là một cách để trả lời các câu hỏi riêng lẻ, nghiên cứu hiện nay xem lập luận như một công cụ thúc đẩy những hành vi phức tạp và tự chủ hơn.

Thứ nhất, lập luận gắn chặt với khái niệm "suy nghĩ dài", điều làm thay đổi căn bản cách sử dụng năng lực tính toán trong LLM. Như đã bàn ở Mục 12.3, bằng cách cho mô hình nhiều thời gian và năng lực tính toán hơn để suy nghĩ, chúng có thể giải những bài toán khó hơn nhiều. Điều này mang lại một thay đổi lớn trong cách LLM hoạt động: giờ đây ta cần tìm cách phân bổ năng lực tính toán tốt nhất trong lúc mô hình đang sinh câu trả lời, chứ không chỉ trong lúc huấn luyện ban đầu. Giải quyết vấn đề này ở cấp hệ thống nằm ngoài phạm vi cuốn sách. Tuy nhiên, xây dựng những mô hình lập luận hiệu quả cao, biết nên dành bao nhiêu thời gian để suy nghĩ, là một hướng nghiên cứu rất hứa hẹn và thiết thực.

Thứ hai, khả năng lập luận là nền tảng của lập luận kiểu tác tử (*agentic reasoning*), trong đó LLM thôi là những cỗ máy trả lời thụ động và trở thành những bên tham gia chủ động trong một môi trường. Trong sinh văn bản tiêu chuẩn, lập luận là một đường thẳng từ prompt tới câu trả lời. Ngược lại, lập luận kiểu tác tử vận hành trong một vòng lặp liên tục gồm suy nghĩ, hành động và quan sát [Yao và cộng sự, 2023]. Tuy nhiên, vòng lặp này dẫn tới những thách thức lớn. Vấn đề là LLM có thể dễ dàng bị lạc hoặc mắc kẹt trong vòng lặp. Chúng có thể lặp lại cùng một lỗi, lập kế hoạch tồi, hoặc không biết cách gượng dậy khi một hành động thất bại [Shinn và cộng sự, 2023]. Giữ cho mô hình đi đúng hướng trong thời gian dài vẫn còn rất khó.

Thứ ba, lập luận trực tiếp tạo điều kiện cho lập luận tăng cường công cụ (*tool-augmented reasoning*), giúp khắc phục những hạn chế của LLM, như không thể tính toán chính xác hay truy cập các sự kiện theo thời gian thực. Thay vì buộc LLM đoán một câu trả lời, ta có thể huấn luyện LLM nhận ra giới hạn của chính mình và viết mã [Gao và cộng sự, 2023b] hoặc gọi các API bên ngoài [Schick và cộng sự, 2024] để đảm nhận phần việc nặng. Tuy nhiên, vấn đề chính là dạy LLM khi nào cần một công cụ và chọn công cụ nào trong số rất nhiều công cụ [Qin và cộng sự, 2024]. Ngoài ra, nếu một công cụ hỏng hoặc trả về dữ liệu kỳ lạ, LLM phải đủ thông minh để phát hiện lỗi và thử lại, thay vì chỉ tin vào kết quả sai.

Bảng thuật ngữ Anh – Việt

Bảng dưới đây tập hợp các thuật ngữ dùng trong bản dịch (sắp xếp theo tiếng Anh). Khi một thuật ngữ có nhiều cách dịch, cách dịch dùng nhiều nhất được ghi trước.

Tiếng Anh	Tiếng Việt
0	
0-1 loss	hàm mất mát 0-1; mất mát 0-1
0-1 loss / surrogate loss	hàm mất mát 0-1 / hàm mất mát thay thế
A	
A search*	tìm kiếm A*
abduction	giả định (suy ra lời giải thích tốt nhất)
absolute / relative positional encoding	mã hóa vị trí tuyệt đối / tương đối
absolute positional embedding	nhúng vị trí tuyệt đối
absolute positional encoding	mã hóa vị trí tuyệt đối
action	hành động
action-value function	hàm giá trị hành động
activation function	hàm kích hoạt
active learning	học chủ động
actor	tác nhân
actor-critic	tác nhân–phê bình
adapt	thích nghi
adapter	tiếp hợp; bộ tiếp hợp (adapter)
adaptor	bộ thích nghi
adaptor / adaptor layer	bộ thích nghi / tầng thích nghi
adaptor layer	tầng thích nghi
add-α smoothing	làm trơn cộng-α
additive (add-α) smoothing	làm trơn cộng (cộng-α)
additive mask	mặt nạ cộng
additive smoothing	làm trơn cộng
addressing	định địa chỉ
advantage	lợi thế
advantage actor-critic	tác nhân–phê bình có lợi thế
advantage actor-critic (A2C) / actor / critic	tác nhân–phê bình có lợi thế (A2C) / tác nhân / bộ phê bình
advantage function	hàm ưu thế
adversarial attack	tấn công đối kháng
adversarial attacks	tấn công đối kháng
adversarial machine learning	học máy đối kháng
adversarial machine learning / adversarial samples / attacks	học máy đối kháng / mẫu đối kháng / cuộc tấn công
adversarial samples	mẫu đối kháng
affine transformation	phép biến đổi affine

Tiếng Anh	Tiếng Việt
affine transformations	phép biến đổi affine
agent	tác tử
agent / agentic reasoning	tác tử / lập luận kiểu tác tử
agent / environment / action	tác tử / môi trường / hành động
agentic planning	lập kế hoạch kiểu tác tử
agentic reasoning	lập luận kiểu tác tử
AGI	trí tuệ nhân tạo tổng quát
aha moment	khoảnh khắc “à há”; khoảnh khắc ”à há”
AI feedback	phản hồi từ AI
AI safety	an toàn AI
AIC / BIC / MML	tiêu chuẩn thông tin Akaike / tiêu chuẩn thông tin Bayes / độ dài thông điệp tối thiểu
Akaike information criterion	tiêu chuẩn thông tin Akaike
algorithmic bias	thiên kiến thuật toán
align	căn chỉnh
aligned	đã được căn chỉnh; căn chỉnh
alignment	căn chỉnh; gióng hàng; căn chỉnh (alignment)
alignment link	liên kết gióng hàng
alignment score	điểm gióng hàng
alternative hypothesis	giả thuyết đối
analytic language	ngôn ngữ đơn lập
anonymize	ẩn danh hóa
approximate randomization	ngẫu nhiên hóa xấp xỉ
approximate search	tìm kiếm xấp xỉ
arguments	tham tố
arithmetic / commonsense reasoning	suy luận số học / suy luận thường thức
arithmetic reasoning	lập luận số học
artificial intelligence	trí tuệ nhân tạo
artificial intelligence, AI	trí tuệ nhân tạo; trí tuệ nhân tạo (AI)
artificial neural networks	mạng nơ-ron nhân tạo
aspect-based sentiment analysis	phân tích cảm xúc theo khía cạnh
association rule mining	khai phá luật kết hợp
asynchronous training	huấn luyện bất đồng bộ
attacks	cuộc tấn công
attention	cơ chế chú ý
attention / self-attention / multi-head attention	cơ chế chú ý / tự chú ý / chú ý đa đầu
attention distribution	phân phối chú ý
attention field	trường chú ý
attention head	đầu chú ý
attention map	bản đồ chú ý
attention maps	bản đồ chú ý
attention matrix	ma trận chú ý
attention matrix / attention map	ma trận chú ý / bản đồ chú ý
attention mechanism	cơ chế chú ý
attention sink	bể hút chú ý
attention weight	trọng số chú ý
attention with linear biases	chú ý với độ lệch tuyến tính
attention with linear biases (ALiBi)	chú ý với độ lệch tuyến tính
augmented	tăng cường; (phiên bản) tăng cường

Tiếng Anh	Tiếng Việt
auto-encoder	bộ tự mã hóa
auto-encoders	bộ tự mã hóa
auto-encoding	tự mã hóa
automated machine learning	học máy tự động
automated machine learning (AutoML)	học máy tự động (AutoML); học máy tự động
automated prompting	prompting tự động
automatic prompt design	thiết kế prompt tự động
automatic prompt design / prompt optimization	thiết kế prompt tự động / tối ưu hóa prompt
autonomous agent	tác tử tự hành
auto-regressive	tự hồi quy
auto-regressive / auto-encoding	tự hồi quy / tự mã hóa
autoregressive decoding	giải mã tự hồi quy
auto-regressive generation	sinh tự hồi quy
autoregressive model	mô hình tự hồi quy
autoregressive process	quá trình tự hồi quy
autoregressive processes, AR processes	quá trình tự hồi quy
auxiliary memory module	mô-đun bộ nhớ phụ trợ
average pooling	gộp trung bình
average-pooling	gộp trung bình
B	
backbone	xương sống
backbone model	mô hình xương sống
background task	bài toán nền
backpropagation	lan truyền ngược
back-propagation through time	lan truyền ngược theo thời gian
back-propagation through time, BPTT	lan truyền ngược theo thời gian (BPTT); lan truyền ngược theo thời gian
backtrack	quay lui
back-translation	dịch ngược
backward pass	lượt truyền ngược
backward pass / backpropagation	lượt truyền ngược / lan truyền ngược
bagging	bagging
bag-of-words	túi từ
bag-of-words (BOW) model	mô hình túi từ (BOW)
bag-of-words model	mô hình túi từ (BOW)
base	cơ số
base learners	bộ học cơ sở
baseline	đường cơ sở; mô hình cơ sở (baseline)
baseline (in RL)	đường cơ sở
baselines	mô hình cơ sở (baseline)
basis function	hàm cơ sở
batch	lô
batch / mini-batch	lô / lô nhỏ
batch gradient descent	hạ gradient theo toàn lô
batch size	kích thước lô
batching	xử lý theo lô; chia lô
batching / batch gradient descent / mini-batch gradient descent	chia lô / hạ gradient theo toàn lô / hạ gradient theo lô nhỏ
Bayes' rule	quy tắc Bayes

Tiếng Anh	Tiếng Việt
Bayesian decision theory	lý thuyết quyết định Bayes
Bayesian inference	suy luận Bayes
Bayesian information criterion	tiêu chuẩn thông tin Bayes
Bayesian learning	học Bayes
Bayesian model averaging	lấy trung bình mô hình Bayes
Bayesian risk	rủi ro Bayes
beam search	tìm kiếm chùm
beam size	kích thước chùm
beam width	độ rộng chùm
beam width / beam size	độ rộng chùm / kích thước chùm
benchmark	bộ đánh giá chuẩn; bộ đánh giá chuẩn (benchmark)
benchmark saturation	bão hòa bộ đánh giá chuẩn
Bernoulli distribution	phân phối Bernoulli
Bernoulli naive Bayes	bayes ngây thơ Bernoulli
best-first search	tìm kiếm tốt nhất trước
best-first search / A search / value function*	tìm kiếm tốt nhất trước / tìm kiếm A* / hàm giá trị
Best-of-N (BoN) sampling	lấy mẫu tốt nhất trong N (BoN)
Best-of-N sampling	lấy mẫu tốt nhất trong N; lấy mẫu tốt nhất trong N
bias	độ lệch (bias); độ chệch
bias correction	hiệu chỉnh độ lệch
bias (statistical) / variance	độ chệch / phương sai (khác ”độ lệch (bias)” của trọng số)
bias trick	mẹo gộp độ lệch
bias-variance decomposition	phân rã độ chệch–phương sai
bias-variance tradeoff	sự đánh đổi độ chệch–phương sai; sự đánh đổi giữa độ chệch và phương sai
bias-variance tradeoff / decomposition	sự đánh đổi / phân rã độ chệch–phương sai
bidirectional	hai chiều
bi-directional models	mô hình hai chiều
bijective linear map	ánh xạ tuyến tính song ánh
bilinear transform	biến đổi song tuyến tính
bilingual data	dữ liệu song ngữ
bilingual dictionary induction	quy nạp từ điển song ngữ
binary / multi-class / multi-label / single-label / hierarchical classification	phân loại nhị phân / đa lớp / đa nhãn / đơn nhãn / phân cấp
binary classification	phân loại nhị phân
binary variable	biến nhị phân
binomial distribution	phân phối nhị thức
BIO tagging scheme	lược đồ gán nhãn BIO
black-box	hộp đen
Boltzmann distribution	phân phối Boltzmann
boosting	boosting
bootstrap	tự nâng cấp
bootstrapping	bootstrap
bottleneck	nút thắt cổ chai
Bradley-Terry model	mô hình Bradley–Terry
branch / path	nhánh / đường
branches	nhánh
breadth-first search	tìm kiếm theo chiều rộng
brevity penalty	hình phạt độ ngắn

Tiếng Anh	Tiếng Việt
bucket	thùng
buckets	"thùng"
Byte Pair Encoding, BPE	mã hóa cặp byte; mã hóa cặp byte (BPE)
C	
cache	bộ nhớ đệm; bộ đệm; lưu đệm
calculation annotation	chú thích tính toán
candidate pool	tập ứng viên
Canonical Variates analysis	phân tích biến chính tắc
canonicalization	chính tắc hóa
capacity	năng lực; dung lượng; năng lực (mô hình)
capacity interference	giao thoa năng lực
Cartesian product	tích Descartes
Cartesian product / product key	tích Descartes / khóa tích
cascaded system	hệ thống nối tầng
cascading errors	lỗi lan truyền dây chuyền
catastrophic forgetting	quên thảm khốc; hiện tượng quên thảm khốc
causal language modeling	mô hình hóa ngôn ngữ nhân quả
causal mask	mặt nạ nhân quả
centroid	tâm cụm
chain of continuous thought	chuỗi suy nghĩ liên tục
chain of continuous thought (COCONUT)	chuỗi suy nghĩ liên tục (COCONUT)
chain of thought	chuỗi suy nghĩ
chain of thought, CoT	chuỗi suy nghĩ; chuỗi suy nghĩ (CoT)
chain rule	quy tắc chuỗi
chain rule of differentiation	quy tắc chuỗi của phép lấy đạo hàm
chain-of-thought	chuỗi suy nghĩ
chain-of-thought prompting	prompt chuỗi suy nghĩ
chat template	khuôn mẫu hội thoại
chatbot	chatbot
checkpoint	điểm lưu; điểm lưu (checkpoint)
checkpoint ensembling	kết hợp các điểm lưu
checkpoint ensembling / word lattice	kết hợp các điểm lưu / lưới từ
Chinchilla scaling law	định luật co giãn Chinchilla
chunked attention	chú ý theo khối
chunked prefilling	điền trước theo khúc
chunking	phân đoạn cụm
chunk-level representation	biểu diễn mức khối
class imbalance	mất cân bằng lớp
classification	phân loại
classification head	đầu phân loại
classification model	mô hình phân loại
classifier	bộ phân loại
classifier / classification	bộ phân loại / phân loại
clipping	cắt ngưỡng
cloze task	nhiệm vụ điền vào chỗ trống
clustering	phân cụm
co-adaptation	đồng thích nghi
co-attention	đồng chú ý
code	mã

Tiếng Anh	Tiếng Việt
code-switching	chuyển mã
collective communication	truyền thông tập thể
collective operation	phép toán tập thể
collocation / idiom	cụm từ cố định / thành ngữ
column vector	vectơ cột
combinatorial optimization	tối ưu tổ hợp
commonsense reasoning	lập luận thường thức
communication overhead	chi phí truyền thông
completion	phần hoàn thành
complex conjugate	số phức liên hợp
component models	mô hình thành phần
compositional distributional semantics	ngữ nghĩa học phân bố kết hợp
compositional generalization	khái quát hóa kết hợp
compositional models	mô hình kết hợp
compositionality / compositional generalization	tính kết hợp / khái quát hóa kết hợp
compositionality / principle of compositionality	tính kết hợp / nguyên lý kết hợp
compression network	mạng nén
compressive Transformer	transformer nén
computation graph	đồ thị tính toán
computation graphs	đồ thị tính toán
compute budget	ngân sách tính toán
compute node	nút tính toán
compute-bound	bị giới hạn bởi tính toán
compute-bound / memory-bound	bị giới hạn bởi tính toán / bị giới hạn bởi bộ nhớ
computer vision	thị giác máy tính
concatenate	ghép nối
concatenated	ghép nối
concatenation	ghép nối
conditional computation	tính toán có điều kiện
conditional likelihood	hợp lý có điều kiện
conditional probability	xác suất có điều kiện
conditional random fields	trường ngẫu nhiên có điều kiện
conditional random fields (CRF) / linear-chain CRF	trường ngẫu nhiên có điều kiện (CRF) / CRF chuỗi tuyến tính
conditional random fields, CRFs	trường ngẫu nhiên có điều kiện; trường ngẫu nhiên có điều kiện (CRF)
connectionist temporal classification	phân loại thời gian kết nối
connectionist temporal classification (CTC)	phân loại thời gian kết nối (CTC); phân loại thời gian kết nối
constituency-based parse tree	cây phân tích thành phần
constituent/component system	hệ thống thành phần
constrained optimization	tối ưu có ràng buộc
constrained search	tìm kiếm có ràng buộc
constraint-based penalty	phạt dựa trên ràng buộc
context	ngữ cảnh
context / context window	ngữ cảnh / cửa sổ ngữ cảnh
context distillation	chưng cất ngữ cảnh
context distillation / knowledge distillation	chưng cất ngữ cảnh / chưng cất tri thức
context scaling	co giãn ngữ cảnh
context scaling / search scaling	co giãn ngữ cảnh / co giãn tìm kiếm

Tiếng Anh	Tiếng Việt
context vector	vectơ ngữ cảnh
context window	cửa sổ ngữ cảnh
context-free grammar	văn phạm phi ngữ cảnh
context-free grammar (CFG)	văn phạm phi ngữ cảnh
contextuality	tính phụ thuộc ngữ cảnh
contextualized representation	biểu diễn theo ngữ cảnh
contextualized representations	biểu diễn theo ngữ cảnh
contextualized word embeddings	vectơ nhúng từ theo ngữ cảnh
continual learning	học liên tục
continual training	huấn luyện liên tục
continual training / continual learning	huấn luyện liên tục / học liên tục
continuous bag-of-words model	mô hình túi từ liên tục
continuous batching	xử lý theo lô liên tục
continuous memory	bộ nhớ liên tục
continuous optimization	tối ưu liên tục
continuous skip-gram model	mô hình skip-gram liên tục
continuous-space attention	chú ý trong không gian liên tục
contraction mapping	ánh xạ co
contractive auto-encoder, CAE	bộ tự mã hóa co
contrastive learning	học tương phản
contrastive loss	hàm mất mát tương phản; mất mát tương phản
contrastive loss / contrastive learning	mất mát tương phản / học tương phản
control task	tác vụ đối chứng
control tasks	tác vụ đối chứng
conversational agent	tác tử hội thoại
conversational agents	tác tử hội thoại
convolution / convolution operation	tích chập / phép tích chập
convolution kernel / filter	nhân tích chập / bộ lọc
convolution kernels	nhân tích chập
convolution operation	phép tích chập
convolution product	tích chập
convolutional layer	tầng tích chập
convolutional layer / pooling layer	tầng tích chập / tầng gộp
convolutional neural network	mạng nơ-ron tích chập
convolutional neural network (CNN)	mạng nơ-ron tích chập (CNN)
convolutional neural networks	mạng nơ-ron tích chập
convolutional neural networks, CNNs	mạng nơ-ron tích chập
co-occur	đồng xuất hiện
co-occurrence matrix	ma trận đồng xuất hiện
copy memory task	tác vụ sao chép bộ nhớ
copy memory tasks	tác vụ sao chép bộ nhớ
coreference / anaphora	đồng tham chiếu / hồi chỉ
coreference resolution	phân giải đồng tham chiếu
coreference resolution / template filling	phân giải đồng tham chiếu / điền mẫu
corpora	kho ngữ liệu
corpus	kho ngữ liệu
corpus (pl. corpora)	kho ngữ liệu
correlation	độ tương quan
correlation coefficient	hệ số tương quan

Tiếng Anh	Tiếng Việt
correlation coefficients	hệ số tương quan
corrupted input	đầu vào bị làm hỏng
corruption	làm hỏng
corruption / corrupted input	làm hỏng / đầu vào bị làm hỏng
cosine similarity	độ tương tự cosin; độ tương đồng cosin
cost function	hàm chi phí
cost function / empirical risk	hàm chi phí / rủi ro thực nghiệm
CoT trace	vết CoT
CoT trace / reasoning trace	vết CoT / vết lập luận
covariance matrix	ma trận hiệp phương sai
coverage	độ bao phủ
coverage / coverage vector / coverage score	độ phủ / vectơ độ phủ / điểm độ phủ
coverage score	điểm độ phủ
coverage vector	vectơ độ phủ
credit assignment problem	bài toán phân bổ công trạng
critic	bộ phê bình
cross attention	chú ý chéo
cross-attention	chú ý chéo
cross-correlation	tương quan chéo
cross-entropy	entropy chéo
cross-entropy loss	hàm mất mát entropy chéo
cross-lingual	xuyên ngôn ngữ
cross-lingual language model (XLM)	mô hình ngôn ngữ xuyên ngôn ngữ
cross-lingual language models	mô hình ngôn ngữ xuyên ngôn ngữ
cross-lingual transfer learning	học chuyển giao xuyên ngôn ngữ
cross-validation	kiểm định chéo
cross-validation / k-fold	kiểm định chéo / kiểm định chéo k-phần
crowdsourcing	huy động cộng đồng
cumulative average	trung bình tích lũy
cumulative distribution function	hàm phân phối tích lũy
cumulative reward	phần thưởng tích lũy
curriculum learning	học theo giáo trình
curse of dimensionality	lời nguyền số chiều
curse of multilinguality / capacity interference	lời nguyền đa ngôn ngữ / giao thoa năng lực
curve fitting	khớp đường cong
D	
data augmentation	tăng cường dữ liệu
data augmentation / back-translation	tăng cường dữ liệu / dịch ngược
data clustering	phân cụm dữ liệu
data contamination	nhiễm bẩn dữ liệu
data modality	phương thức dữ liệu
Data Parallelism	song song dữ liệu
data sparsity	tính thưa của dữ liệu
data-generating distribution	phân phối sinh dữ liệu
data/model/tensor/pipeline parallelism	song song dữ liệu/mô hình/tensor/đường ống
datastore	kho dữ liệu
decay factor	hệ số suy giảm
decision boundary	biên quyết định
decision function	hàm quyết định

Tiếng Anh	Tiếng Việt
decision function / threshold function	hàm quyết định / hàm ngưỡng
decoder	bộ giải mã
decoder-only	chỉ có bộ giải mã
decoder-only / encoder-only	chỉ có bộ giải mã / chỉ có bộ mã hóa
decoder-only architecture	kiến trúc chỉ có bộ giải mã
decoding	giải mã
decoding / beam search / greedy search	giải mã / tìm kiếm chùm (beam search) / tìm kiếm tham lam
deduction	diễn dịch
deduction / induction / abduction	diễn dịch / quy nạp / giả định (suy ra lời giải thích tốt nhất)
deep learning	học sâu
deep neural network	mạng nơ-ron sâu
deep neural networks	mạng nơ-ron sâu
degenerate	suy biến
degenerate output	đầu ra suy biến
degeneration	suy biến
delayed reward	phần thưởng trễ
delayed reward (sparse reward)	phần thưởng trễ (phần thưởng thưa)
deliberate reasoning	lập luận cân nhắc
deliberate-then-generate	cân nhắc rồi sinh
deliberate-then-generate (DTG)	cân nhắc rồi sinh (DTG)
delta rule	quy tắc delta
demonstrations	ví dụ minh họa
denoising	khử nhiễu
denoising autoencoder	bộ tự mã hóa khử nhiễu
denoising auto-encoder (DAE)	bộ tự mã hóa khử nhiễu
denoising auto-encoders, DAEs	bộ tự mã hóa khử nhiễu
dense	dày đặc
dense reward	phần thưởng dày
densely connected network	mạng kết nối dày đặc
dependency	quan hệ phụ thuộc
dependency parse tree	cây phân tích phụ thuộc
dependency tree	cây phụ thuộc
dependent / independent variable	biến phụ thuộc / biến độc lập
dependent variable	biến phụ thuộc
depth	độ sâu
depth growth	tăng trưởng độ sâu
depth-adaptive	thích ứng độ sâu
depth-adaptive / length-adaptive	thích ứng độ sâu / thích ứng độ dài
depth-first search	tìm kiếm theo chiều sâu
depthwise CNN	CNN theo chiều sâu
derivation	dẫn xuất
deterministic	tất định
diagonal matrix	ma trận đường chéo
diagonal state-space model	mô hình không gian trạng thái đường chéo
diagonalization	chéo hóa
dialogue generation	sinh hội thoại
dialogue systems	hệ thống hội thoại
differentiable search / continuous optimization	tìm kiếm khả vi / tối ưu liên tục
dilated context window	cửa sổ ngữ cảnh giãn

Tiếng Anh	Tiếng Việt
dimension	chiều
dimensionality reduction	giảm chiều
direct preference optimization	tối ưu hóa sở thích trực tiếp
direct preference optimization (DPO)	tối ưu hóa sở thích trực tiếp (DPO)
directed	có hướng
directed acyclic graph	đồ thị có hướng không chu trình
directed (graph)	có hướng
disaggregate	tách rời
disaggregate (prefilling and decoding)	tách rời (điền trước và giải mã)
discount factor	hệ số chiết khấu
discounted return	lợi tức chiết khấu
discounted return / return / cumulative reward	lợi tức chiết khấu / lợi tức / phần thưởng tích lũy
discourse	diễn ngôn
discriminant function	hàm phân biệt
discriminative modeling	mô hình hóa phân biệt
discriminative models	mô hình phân biệt
discriminator	bộ phân biệt
distance penalty	mức phạt khoảng cách
distance-based loss	mất mát dựa trên khoảng cách
distance-based loss / curve fitting	mất mát dựa trên khoảng cách / khớp đường cong
distributed representation	biểu diễn phân tán
distributed representations	biểu diễn phân tán
distribution shift	dịch chuyển phân phối
distributional hypothesis	giả thuyết phân bố
distributional memory	bộ nhớ phân bố
distributional representation	biểu diễn phân bố
distributional semantics	ngữ nghĩa học phân bố
distributional word representation	biểu diễn từ phân bố
diverge	phân kỳ
divergence	độ phân kỳ
divergence (-based loss)	độ phân kỳ (mất mát dựa trên độ phân kỳ)
divergence-based loss	mất mát dựa trên độ phân kỳ
diversity penalty	hình phạt đa dạng; phạt đa dạng
diversity penalty / subspace method	hình phạt đa dạng / phương pháp không gian con
divide-and-conquer	chia để trị
document	tài liệu
document chunks	đoạn văn bản
dot product	tích vô hướng
dot-product	tích vô hướng
double descent	suy giảm kép
downsample	giảm mẫu
down-sampling	giảm mẫu
downsampling / progressive downsampling	giảm mẫu / giảm mẫu lũy tiến
downstream	hạ nguồn
downstream task	nhiệm vụ hạ nguồn; tác vụ hạ nguồn; bài toán hạ nguồn
downstream tasks	nhiệm vụ hạ nguồn; bài toán hạ nguồn
draft model	mô hình nháp
draft model / verification model	mô hình nháp / mô hình kiểm chứng
dropout	dropout

Tiếng Anh	Tiếng Việt
dropout rate	tỉ lệ dropout
dual-process theory	thuyết quá trình kép
dynamic network	mạng động
Dynamic Neural Networks	mạng nơ-ron động
dynamic programming	quy hoạch động
dynamic programming / lattice	quy hoạch động / lưới
dynamical systems	hệ động lực
dynamics	động lực
E	
early exit classifier	bộ phân loại thoát sớm
early exiting	thoát sớm
early stopping	dừng sớm
edge probing	thăm dò cạnh
edges	cạnh
effective degrees of freedom	số bậc tự do hiệu dụng
efficient machine learning	học máy hiệu quả
eigenvalue	giá trị riêng
eigenvector	vectơ riêng
eigenvector / eigenvalue	vectơ riêng / giá trị riêng
elastic weight consolidation	củng cố trọng số đàn hồi
elements, entries	phần tử
element-wise	theo từng phần tử
element-wise product	nhân theo từng phần tử
embedding	vectơ nhúng
embedding layer	tầng nhúng
embedding layer / word embedding table	tầng nhúng / bảng nhúng từ
embedding matrix	ma trận nhúng
embedding sharing	chia sẻ vectơ nhúng
embedding sharing / layer-wise sharing	chia sẻ vectơ nhúng / chia sẻ theo tầng
emergence	trồi sinh
emergent abilities	năng lực đột sinh
emergent ability	năng lực đột sinh; năng lực trỗi dậy
empirical risk	rủi ro thực nghiệm
encoder	bộ mã hóa
encoder / decoder	bộ mã hóa / bộ giải mã
encoder-decoder	bộ mã hóa–bộ giải mã
encoder-decoder architecture	kiến trúc bộ mã hóa–bộ giải mã
encoder-decoder attention	chú ý bộ mã hóa–bộ giải mã
encoder-only	chỉ có bộ mã hóa
encoder-only / decoder-only architecture	kiến trúc chỉ có bộ mã hóa / chỉ có bộ giải mã
encoder-only architecture	kiến trúc chỉ có bộ mã hóa
encoding blocks	khối mã hóa
encoding layer / decoding layer	tầng mã hóa / tầng giải mã
encoding layers	tầng mã hóa
end-to-end	đầu-cuối
end-to-end memory network	mạng bộ nhớ đầu-cuối
end-to-end memory networks	mạng bộ nhớ đầu-cuối
end-to-end representation learning	học biểu diễn đầu-cuối
ensemble	tổ hợp mô hình; tập hợp mô hình

Tiếng Anh	Tiếng Việt
ensemble / ensemble learning	tập hợp mô hình / học tập hợp
ensemble learning	học kết hợp; học tập hợp
ensemble learning / base learners / model averaging / soft voting	học kết hợp / bộ học cơ sở / lấy trung bình mô hình / bỏ phiếu mềm
ensembling	kết hợp; kết hợp mô hình
ensembling / component models	kết hợp (mô hình) / mô hình thành phần
ensembling / system combination / prediction averaging	kết hợp mô hình / kết hợp hệ thống / lấy trung bình dự đoán
entropy	entropy
environment	môi trường
epoch	epoch
error gradient	gradient lỗi
error surface	mặt sai số
error-based loss	mất mát dựa trên lỗi
error-driven learning	học dựa trên lỗi
estimator	bộ ước lượng
Euclidean norm	chuẩn Euclid
Euler discretization	rời rạc hóa Euler
Euler discretization / Euler method	rời rạc hóa Euler / phương pháp Euler
evaluation metric	độ đo đánh giá
event	sự kiện
event extraction	trích xuất sự kiện
evidence lower bound, ELBO	cận dưới bằng chứng
evolutionary algorithms	thuật toán tiến hóa
evolutionary computation	tính toán tiến hóa
exact search	tìm kiếm chính xác
exhaustive search	tìm kiếm vét cạn
exhaustive search / exact search / approximate search	tìm kiếm vét cạn / tìm kiếm chính xác / tìm kiếm xấp xỉ
exhaustive search / heuristic search	tìm kiếm vét cạn / tìm kiếm heuristic
expectation	kỳ vọng
Expectation-Maximization (EM)	kỳ vọng–Cực đại hóa (EM)
expected code	mã kỳ vọng
expected embedding	vectơ nhúng kỳ vọng
expected log-probability	log xác suất kỳ vọng
expected loss	mất mát kỳ vọng
expected loss / expected risk / Bayesian risk	mất mát kỳ vọng / rủi ro kỳ vọng / rủi ro Bayes
expected risk	rủi ro kỳ vọng
expected surprisal	độ bất ngờ kỳ vọng
expected value	giá trị kỳ vọng
expected value / expectation	giá trị kỳ vọng / kỳ vọng
experience replay	phát lại kinh nghiệm
explainable models	mô hình có thể giải thích
exploding / vanishing gradient	bùng nổ gradient / tiêu biến gradient
exploding and vanishing gradient problems	bài toán bùng nổ gradient và tiêu biến gradient
exploding gradient	gradient bùng nổ; bùng nổ gradient
exploitation	khai thác
exploration	khám phá
exploration / exploitation	khám phá / khai thác

Tiếng Anh	Tiếng Việt
exploration–exploitation trade-off	sự đánh đổi giữa khám phá và khai thác; đánh đổi giữa khám phá và khai thác
exposure bias	thiên lệch phơi nhiễm
exposure bias / distribution shift	thiên lệch phơi nhiễm / dịch chuyển phân phối
expressive power	năng lực biểu diễn; năng lực biểu đạt
expressiveness	khả năng biểu đạt
external memories	bộ nhớ ngoài
external memory	bộ nhớ ngoài
Extrapolation	ngoại suy
extrapolation / interpolation	ngoại suy / nội suy
extrinsic / intrinsic evaluation	đánh giá ngoại tại / nội tại
extrinsic evaluation	đánh giá ngoại tại
F	
F_1 score	điểm F_1; điểm F1
factor analysis	phân tích nhân tố
false negative	âm tính giả
false positive	dương tính giả
false positives	dương tính giả
fan-in	số đầu vào
fan-in / fan-out	số đầu vào / số đầu ra
fan-out	số đầu ra
fault-tolerant	chịu lỗi
feature	đặc trưng
feature / feature vector	đặc trưng / vectơ đặc trưng
feature engineering	kỹ nghệ đặc trưng
feature extractor	bộ trích xuất đặc trưng
feature extractors	bộ trích xuất đặc trưng
feature functions	hàm đặc trưng
feature learning	học đặc trưng
feature map	bản đồ đặc trưng; ánh xạ đặc trưng
feature mapping	ánh xạ đặc trưng
feature mapping / kernel methods / kernel function / kernel trick	ánh xạ đặc trưng / phương pháp hạt nhân / hàm hạt nhân / thủ thuật hạt nhân
feature maps	bản đồ đặc trưng
feature selection	chọn lọc đặc trưng
feature subspace	không gian con đặc trưng
feature vector	vectơ đặc trưng
feedback	phản hồi
feed-forward network	mạng truyền thẳng
feed-forward network (FFN)	mạng truyền thẳng (FFN)
feed-forward neural network	mạng nơ-ron truyền thẳng
feed-forward neural network based language model, FFNNLM	mô hình ngôn ngữ dựa trên mạng nơ-ron truyền thẳng
feed-forward neural network, FFN	mạng truyền thẳng
feed-forward neural network (FFNN)	mạng nơ-ron truyền thẳng
feed-forward neural networks, FFNNs	mạng nơ-ron truyền thẳng
fertility	độ sinh sản
fertility / coverage	độ sinh sản / độ bao phủ
fertility modeling	mô hình hóa độ sinh sản

Tiếng Anh	Tiếng Việt
few-shot / zero-shot	few-shot / zero-shot
few-shot CoT prompting	prompt CoT few-shot
few-shot learning	học few-shot
few-shot prompting	prompt few-shot
filter	bộ lọc
filters	bộ lọc
fine-tuning	tinh chỉnh
finite-state automata	ôtômat hữu hạn trạng thái
first moment	mô-men bậc nhất
first moment / second raw moment	mô-men bậc nhất / mô-men gốc bậc hai
Fisher's linear discriminant	phân biệt tuyến tính Fisher
floating-point	dấu phẩy động
forced decoding	giải mã cưỡng bức
forget gate	cổng quên
forget gate / input gate / output gate	cổng quên / cổng vào / cổng ra
formal / functional linguistic competence	năng lực ngôn ngữ hình thức / chức năng
formal equivalency checker	kiểm tra tương đương hình thức
formal linguistic competence	năng lực ngôn ngữ hình thức
formal symbolic system	hệ thống ký hiệu hình thức
formal symbolic systems	hệ thống ký hiệu hình thức
forward pass	lượt truyền xuôi
forward pass / backward pass	lượt truyền xuôi / lượt truyền ngược
forward-backward	tiến–lùi; tiến–lùi
foundation model	mô hình nền tảng
foundation models	mô hình nền tảng
Fourier transform	biến đổi Fourier
fractional count	số đếm phân số
frames	khung
free parameters	tham số tự do
freeze	đóng băng
freeze (parameters)	đóng băng
frequentist statistics	thống kê tần suất
Frobenius norm	chuẩn Frobenius
full fine-tuning	tinh chỉnh toàn bộ
full-text indexing	chỉ mục toàn văn
fully connected	kết nối đầy đủ
function approximation	xấp xỉ hàm
function approximation / basis function	xấp xỉ hàm / hàm cơ sở
function words	hư từ
functional linguistic competence	năng lực ngôn ngữ chức năng
fusion layer	tầng hợp nhất
future return	lợi tức tương lai
G	
gain	hệ số khuếch đại
gate	cổng
gate / gating vector / update gate / forget gate	cổng / vectơ cổng / cổng cập nhật / cổng quên
gated linear unit, GLU	đơn vị tuyến tính có cổng
gated recurrent unit, GRU	đơn vị hồi quy có cổng; đơn vị hồi quy có cổng (GRU)
gating model	mô hình cổng

Tiếng Anh	Tiếng Việt
gating network	mạng cổng
gating network / gating model	mạng cổng / mô hình cổng
gating vector	vectơ cổng
Gaussian distribution	phân phối Gauss
gaussian error linear unit, GeLU	đơn vị tuyến tính sai số Gauss
Gaussian mixture models	mô hình hỗn hợp Gauss
Gaussian mixture models (GMM)	mô hình hỗn hợp Gauss (GMM)
Gaussian naive Bayes	bayes ngây thơ Gauss
Gaussian noise	nhiễu Gauss
Gaussian/normal distribution	phân phối Gauss/chuẩn
GeLU	đơn vị tuyến tính sai số Gauss
gender bias	thiên lệch giới tính
generalization	tổng quát hóa
generalization error	lỗi tổng quát hóa
generalize	tổng quát hóa
generalize / generalization	tổng quát hóa
generative / discriminative model	mô hình sinh / mô hình phân biệt
generative adversarial networks	mạng đối sinh
generative adversarial networks (GAN)	mạng đối sinh (GAN)
generative models	mô hình sinh
generative verification	kiểm chứng sinh
generator	bộ sinh
generator / discriminator	bộ sinh / bộ phân biệt
geometric mean	trung bình nhân
global / random attention	chú ý toàn cục / chú ý ngẫu nhiên
global attention	chú ý toàn cục
global tokens	token toàn cục
Global vectors	vectơ toàn cục
GLU	đơn vị tuyến tính có cổng
gold-standard	đáp án chuẩn; chuẩn vàng
Goodhart's law	định luật Goodhart
gradient	gradient
gradient accumulation	tích lũy gradient
gradient clipping	cắt gradient
gradient descent	hạ gradient; hạ gradient (gradient descent)
graph	đồ thị
graphical models	mô hình đồ thị
graphical models / undirected graphical models	mô hình đồ thị / mô hình đồ thị vô hướng
greedy decoding	giải mã tham lam
greedy search	tìm kiếm tham lam
greedy strategy	chiến lược tham lam
grid search	tìm kiếm lưới
grokking	grokking
ground truth	đáp án chuẩn
grounded commonsense inference	lập luận thường thức có căn cứ; suy luận thường thức có căn cứ
ground-truth	chuẩn đúng; đúng thực tế
ground-truth / gold-standard	nhãn đúng thực tế / chuẩn vàng
group relative policy optimization	tối ưu chính sách tương đối theo nhóm

Tiếng Anh	Tiếng Việt
group relative policy optimization (GRPO)	tối ưu chính sách tương đối theo nhóm (GRPO)
grouped query attention, GQA	chú ý truy vấn theo nhóm
grouped-query attention	chú ý truy vấn theo nhóm
H	
hallucination	ảo giác; ảo giác (hallucination)
hard alignment	gióng hàng cứng
hard constraint	ràng buộc cứng
hard prompt	prompt cứng
hard prompt / soft prompt	prompt cứng / prompt mềm
hardware-aware	nhận biết phần cứng
head (attention)	đầu
heads	đầu
held-out validation set	tập kiểm định giữ lại
held-out validation set / early stopping	tập kiểm định giữ lại / dừng sớm
Hessian	ma trận Hessian
heuristic	heuristic; quy tắc kinh nghiệm
heuristic search	tìm kiếm heuristic
heuristics	quy tắc kinh nghiệm; heuristic
hidden layer	tầng ẩn
hidden layers	tầng ẩn
hidden Markov model, HMM	mô hình Markov ẩn; mô hình Markov ẩn (HMM)
hidden Markov models	mô hình Markov ẩn
hidden Markov models (HMM)	mô hình Markov ẩn (HMM)
hidden size	kích thước ẩn
hidden space	không gian ẩn
hidden state	trạng thái ẩn
hidden states	trạng thái ẩn
hierarchical classification	phân loại phân cấp
high-performance computing	tính toán hiệu năng cao
high-resource languages	ngôn ngữ nhiều tài nguyên
hindsight experience replay	phát lại kinh nghiệm nhìn lại
hinge loss	mất mát bản lề; hàm mất mát bản lề
histogram pruning	cắt tỉa theo biểu đồ tần suất
homomorphism	đồng cấu
homonymy	đồng âm
human evaluation	đánh giá bởi con người
human feedback	phản hồi của con người
human preference	sở thích của con người; sở thích (ưu tiên) của con người
human preference alignment	căn chỉnh theo sở thích của con người
human preferences	sở thích (ưu tiên) của con người
hyperparameter	siêu tham số
hyper-parameters	siêu tham số
hyperspace analogue of language	mô hình tương tự siêu không gian của ngôn ngữ
hypersphere	siêu cầu
hypothesis	giả thuyết
hypothesis / partial hypothesis	giả thuyết / giả thuyết bộ phận
hypothesis selection	chọn giả thuyết; lựa chọn giả thuyết
hypothesis space	không gian giả thuyết
I	

Tiếng Anh	Tiếng Việt
identity mapping	ánh xạ đồng nhất
identity matrix	ma trận đơn vị
identity transformation	phép biến đổi đồng nhất
i.i.d. assumption	giả định i.i.d.; giả định i.i.d. (độc lập và cùng phân phối)
image captioning	chú thích ảnh
implicit reasoning	lập luận ngầm
importance sampling	lấy mẫu theo trọng số quan trọng
impulse noise	nhiễu xung
in-context learning	học trong ngữ cảnh
independent	độc lập
independent component analysis	phân tích thành phần độc lập
independent component analysis (ICA)	phân tích thành phần độc lập
independent variable	biến độc lập
index	chỉ mục
indicator function	hàm chỉ thị
induction	quy nạp
induction / deduction	quy nạp / diễn dịch
inductive bias	thiên kiến quy nạp
inductive biases	thiên kiến quy nạp
inductive inference	suy diễn quy nạp
inductive reasoning	suy luận quy nạp
inductive reasoning / inductive inference	suy luận quy nạp / suy diễn quy nạp
inference	suy luận
inference (in Ch. 11)	suy luận (reasoning = lập luận)
inference-time alignment	căn chỉnh lúc suy luận
inference-time compute	lượng tính toán lúc suy luận
inference-time compute scaling	co giãn tính toán lúc suy luận
inference-time reasoning	lập luận lúc suy luận
inference-time scaling	co giãn lúc suy luận; co giãn tại thời điểm suy luận; mở rộng quy mô lúc suy luận
infinite context	ngữ cảnh vô hạn
inflection	biến tố
inflection points	điểm uốn
information extraction	trích xuất thông tin
information extraction / span prediction	trích xuất thông tin / dự đoán đoạn
information gain	độ lợi thông tin
information retrieval	truy xuất thông tin
information retrieval / metric learning	truy xuất thông tin / học độ đo
information retrieval, IR	truy xuất thông tin; truy xuất thông tin (IR)
inner product	tích trong
input gate	cổng vào
input inversion	đảo ngược đầu vào
instruction alignment	căn chỉnh theo chỉ dẫn
instruction fine-tuning	tinh chỉnh theo chỉ dẫn
instruction following	khả năng làm theo chỉ dẫn
interactive machine translation	dịch máy tương tác
intermediate representation	biểu diễn trung gian
internal / external memories	bộ nhớ trong / bộ nhớ ngoài
internal covariate shift	dịch chuyển hiệp biến nội tại

Tiếng Anh	Tiếng Việt
internal memories	bộ nhớ trong
Interpolation	nội suy
interpolation coefficient	hệ số nội suy
interpretable	diễn giải được
intractable	không tính được
intrinsic evaluation	đánh giá nội tại
intrinsic motivation	động lực nội tại
inverse document frequency	nghịch đảo tần suất văn bản
inverse document frequency (IDF)	nghịch đảo tần suất văn bản
IO-aware	nhận biết vào/ra
irreducible error	lỗi không thể khử
isomorphism	đẳng cấu
iteration-level scheduling	lập lịch mức vòng lặp
J	
Jacobian matrix	ma trận Jacobi
joint probability	xác suất đồng thời
K	
kernel	nhân
kernel function	hàm hạt nhân; hàm nhân
kernel fusion	hợp nhất nhân
kernel methods	phương pháp hạt nhân
kernel methods / kernel trick / kernel function	phương pháp hạt nhân / thủ thuật hạt nhân / hàm hạt nhân
kernel trick	thủ thuật hạt nhân
kernelized attention model	mô hình chú ý hạt nhân hóa
key	khóa
keys	khóa
key-value cache	bộ nhớ đệm khóa–giá trị; bộ nhớ đệm khóa-giá trị
keyword extraction	trích xuất từ khóa
k-fold cross-validation	kiểm định chéo k-phần
KL divergence	độ phân kỳ KL
KL-divergence	độ phân kỳ KL
k-means clustering	phân cụm k-means
k-nearest neighbors	k láng giềng gần nhất; k láng giềng gần nhất
k-nearest neighbors, k-NN	k láng giềng gần nhất; k láng giềng gần nhất
k-NN language modeling	mô hình hóa ngôn ngữ k-NN
k-NN language modeling, k-NN LM	mô hình hóa ngôn ngữ k-NN
knowledge bases	cơ sở tri thức
knowledge distillation	chưng cất tri thức
knowledge distillation / self-distillation	chưng cất tri thức / tự chưng cất
knowledge graphs	đồ thị tri thức
Kullback-Leibler (KL) divergence	độ phân kỳ Kullback-Leibler (KL)
KV cache	bộ nhớ đệm KV; bộ nhớ đệm KV (KV cache)
L	
l_2 regularization	chính quy hóa l_2
label mapping	ánh xạ nhãn
label smoothing	làm trơn nhãn
labeled samples	mẫu có nhãn; mẫu (có nhãn)
labels	nhãn

Tiếng Anh	Tiếng Việt
labels / tags / categories	nhãn / thẻ / hạng mục
Lagrange multiplier	nhân tử Lagrange
Lagrangian relaxation	nới lỏng Lagrange
language embedding	nhúng ngôn ngữ
language model	mô hình ngôn ngữ
language model / language modeling	mô hình ngôn ngữ / mô hình hóa ngôn ngữ
language modeling	mô hình hóa ngôn ngữ
language understanding	hiểu ngôn ngữ
language-agnostic	không phụ thuộc ngôn ngữ
large language model	mô hình ngôn ngữ lớn
large language model, LLM	mô hình ngôn ngữ lớn; mô hình ngôn ngữ lớn (LLM)
large language models	mô hình ngôn ngữ lớn
large language models, LLMs	mô hình ngôn ngữ lớn
large-margin training	huấn luyện lề lớn
Lasso / Ridge regularization	chính quy hóa Lasso / Ridge
Lasso regularization	chính quy hóa Lasso
latency	độ trễ
latent / implicit reasoning	lập luận tiềm ẩn / lập luận ngầm
latent Dirichlet allocation	phân bổ Dirichlet tiềm ẩn
latent Dirichlet allocation (LDA)	phân bổ Dirichlet tiềm ẩn
latent reasoning	lập luận tiềm ẩn
latent semantic analysis	phân tích ngữ nghĩa tiềm ẩn
latent semantic analysis (LSA)	phân tích ngữ nghĩa tiềm ẩn
latent semantic indexing	lập chỉ mục ngữ nghĩa tiềm ẩn
latent space	không gian ẩn
latent space / latent variable	không gian ẩn / biến ẩn
latent states	trạng thái ẩn
latent variable	biến ẩn
latent variables	biến ẩn
latent-variable modeling	mô hình hóa biến tiềm ẩn
lattice	lưới
layer	tầng
layer / hidden layer	tầng / tầng ẩn
layer dropout	dropout tầng
layer fusion	hợp nhất tầng
layer normalization	chuẩn hóa tầng
layers	tầng
layer-wise sharing	chia sẻ theo tầng
learning curves	đường cong học
learning rate	tốc độ học
learning rate scheduling	lịch điều chỉnh tốc độ học; lập lịch tốc độ học
learning rate scheduling / warmup	lập lịch tốc độ học / khởi động
learning rate warmup	khởi động tốc độ học
learning to reason	học để lập luận
learning-to-rank	học để xếp hạng; học xếp hạng
least recently used, LRU	ít được dùng gần đây nhất
least-to-most prompting	prompting từ dễ đến khó
left padding	đệm bên trái
left-singular vectors	vectơ suy biến trái

Tiếng Anh	Tiếng Việt
lemma	từ nguyên dạng
lemma / lemmatization	từ nguyên dạng / đưa về nguyên dạng
lemmatization	đưa về nguyên dạng
length bias	thiên lệch độ dài
length bias / length reward / length normalization	thiên lệch độ dài / thưởng độ dài / chuẩn hóa độ dài
length extrapolation	ngoại suy độ dài
length normalization	chuẩn hóa độ dài
length penalty	phạt độ dài
length reward	thưởng độ dài; phần thưởng độ dài
length-adaptive	thích ứng độ dài
lexeme	từ vị
lexical semantics	ngữ nghĩa học từ vựng
likelihood	độ hợp lý; hàm hợp lý; khả năng
likelihood / log-likelihood / conditional likelihood	hàm hợp lý / log-hợp lý / hợp lý có điều kiện
likelihood ratio	tỉ số hợp lý
linear attention	chú ý tuyến tính
linear attention model	mô hình chú ý tuyến tính
linear classifier	bộ phân loại tuyến tính
linear discriminant analysis	phân tích phân biệt tuyến tính
linear discriminant function	hàm phân biệt tuyến tính
linear interpolation	nội suy tuyến tính
linear multi-step method	phương pháp đa bước tuyến tính
linear programming	quy hoạch tuyến tính
linear regression	hồi quy tuyến tính
linear transformation	phép biến đổi tuyến tính
linear-chain CRF	CRF chuỗi tuyến tính
linearize	tuyến tính hóa
linearly separable	tách được tuyến tính
linguistic alignment	gióng hàng ngôn ngữ
linguistic alignment / word alignment	gióng hàng ngôn ngữ / gióng hàng từ
linguistic regularities	quy luật ngôn ngữ
Lipschitz constant	hằng số Lipschitz
listwise	theo danh sách
listwise ranking	xếp hạng theo danh sách
LLM serving	phục vụ LLM
load balancing	cân bằng tải
local attention	chú ý cục bộ
local minima	cực tiểu địa phương
local optima	cực trị địa phương
local truncation error	sai số chặt cụt địa phương
locally normalized	chuẩn hóa cục bộ
logical forms	dạng logic
logistic function	hàm logistic
logistic regression	hồi quy logistic
logistic regression / logistic function / sigmoid function	hồi quy logistic / hàm logistic / hàm sigmoid
logit	logit

Tiếng Anh	Tiếng Việt
logits	logit
log-likelihood	log-hợp lý
log-likelihood / negative log-likelihood	log-hợp lý / log-hợp lý âm
log-linear	log-tuyến tính
log-linear model	mô hình log-tuyến tính
log-odds	log-odds
long short-term memory, LSTM	bộ nhớ dài-ngắn hạn; bộ nhớ dài-ngắn hạn (LSTM)
long tail	đuôi dài
Long-context and long-text generation	sinh văn bản dài với ngữ cảnh dài
long-context LLM	LLM ngữ cảnh dài
long-context LLMs	LLM ngữ cảnh dài
Long-context text generation	sinh văn bản với ngữ cảnh dài
long-distance dependencies	phụ thuộc khoảng cách xa; phụ thuộc xa
long-distance dependency	phụ thuộc xa
long-term memory	trí nhớ dài hạn
Long-text generation	sinh văn bản dài
lookup table	bảng tra; bảng tra cứu
loss function	hàm mất mát
low-/high-resource languages	ngôn ngữ ít/nhiều tài nguyên
low-precision	độ chính xác thấp
low-rank approximation	xấp xỉ hạng thấp
low-rank matrix	ma trận hạng thấp
low-resource language	ngôn ngữ ít tài nguyên
low-resource languages	ngôn ngữ ít tài nguyên
l_p norm	chuẩn l_p
M	
machine learning	học máy
machine learning, ML	học máy
machine translation	dịch máy
majority voting	bỏ phiếu đa số
margin	lề
margin / margin-based loss / large-margin training	lề / mất mát dựa trên lề / huấn luyện lề lớn
marginal distribution	phân phối biên
marginal distribution / marginal likelihood	phân phối biên / hàm hợp lý biên
marginal likelihood	hàm hợp lý biên
marginal probability	xác suất biên
marginalize out	lấy biên
margin-based loss	mất mát dựa trên lề
Markov assumption	giả thiết Markov
Markov chain	xích Markov
Markov decision process	quá trình quyết định Markov
mask	mặt nạ
mask (attention)	mặt nạ
mask vector	vectơ mặt nạ
masked language model	mô hình ngôn ngữ có mặt nạ
masked language modeling	mô hình hóa ngôn ngữ có che
masked language modeling (MLM)	mô hình hóa ngôn ngữ có che
masking noise	nhiễu che

Tiếng Anh	Tiếng Việt
masking noise / salt-and-pepper noise / impulse noise	nhiễu che / nhiễu muối tiêu / nhiễu xung
masking variable	mặt nạ
masking variable / additive mask / multiplicative mask	biến mặt nạ / mặt nạ cộng / mặt nạ nhân
matrix	ma trận
matrix addition	phép cộng ma trận
matrix product	tích ma trận
matrix-matrix product	tích ma trận–ma trận
max pooling	gộp cực đại
max pooling / average pooling	gộp cực đại / gộp trung bình
max-heap	đống cực đại
max-heap / priority queue	đống cực đại / hàng đợi ưu tiên
maximum a posteriori, MAP	hậu nghiệm cực đại; cực đại hậu nghiệm
maximum a posteriori (MAP) / minimum Bayes risk (MBR)	cực đại hậu nghiệm / rủi ro Bayes tối thiểu; hậu nghiệm cực đại / rủi ro Bayes cực tiểu
maximum likelihood	hợp lý cực đại; ước lượng hợp lý cực đại
maximum likelihood estimation	ước lượng hợp lý cực đại; ước lượng hợp lý cực đại (MLE)
maximum likelihood estimation (MLE)	ước lượng hợp lý cực đại (MLE)
max-pooling	gộp cực đại
mean	giá trị trung bình
mean absolute error	sai số tuyệt đối trung bình
mean squared error	sai số bình phương trung bình
mean squared error (MSE) / mean absolute error (MAE)	sai số bình phương trung bình / sai số tuyệt đối trung bình
mechanistic interpretability	khả năng diễn giải cơ chế
memorization	ghi nhớ
memory	bộ nhớ (trí nhớ trong ngữ cảnh tâm lý học)
memory bandwidth	băng thông bộ nhớ
memory bank	kho bộ nhớ
memory cell	ô nhớ
memory footprint	lượng bộ nhớ chiếm dụng
memory fragmentation	phân mảnh bộ nhớ
memory-based methods	phương pháp dựa trên bộ nhớ
memory-bound	bị giới hạn bởi bộ nhớ
memory-compute trade-off	đánh đổi bộ nhớ–tính toán
meta-learner	bộ học meta
meta-learning	học cách học
meta-reasoning	siêu lập luận
metric	độ đo
metric learning	học độ đo
micro-batch	lô vi mô
micro-batches	lô vi mô
mini-batch	lô nhỏ
mini-batch gradient descent	hạ gradient theo lô nhỏ
minimum Bayes risk	rủi ro Bayes cực tiểu
minimum Bayes risk, MBR	rủi ro Bayes cực tiểu; rủi ro Bayes tối thiểu
minimum Bayesian risk	rủi ro Bayes cực tiểu
Minimum Description Length	độ dài mô tả tối thiểu

Tiếng Anh	Tiếng Việt
Minimum Description Length (MDL)	độ dài mô tả tối thiểu
minimum error rate training	huấn luyện cực tiểu tỉ lệ lỗi
minimum error-rate training	huấn luyện tỉ lệ lỗi tối thiểu
minimum error-rate training (MERT)	huấn luyện tỉ lệ lỗi tối thiểu (MERT)
minimum message length	độ dài thông điệp tối thiểu
minimum-spanning tree	cây khung nhỏ nhất
mixed precision	độ chính xác hỗn hợp
mixed-precision training	huấn luyện độ chính xác hỗn hợp
mixture of experts	hỗn hợp chuyên gia
mixture-of-experts	hỗn hợp chuyên gia
mixture-of-experts (MoE)	hỗn hợp chuyên gia (MoE)
mixtures of experts	hỗn hợp chuyên gia
modality	phương thức
modality / multi-modal model	phương thức / mô hình đa phương thức
model averaging	lấy trung bình mô hình
model bias	thiên kiến mô hình
model bias / algorithmic bias / sample bias	thiên kiến mô hình / thuật toán / mẫu
model capacity	năng lực của mô hình
model cascading	phân tầng mô hình
model complexity	độ phức tạp của mô hình; độ phức tạp mô hình
model complexity / model capacity	độ phức tạp của mô hình / năng lực của mô hình
model compression	nén mô hình
model compression / pruning / quantization	nén mô hình / tỉa / lượng tử hóa
model depth	độ sâu mô hình
model depth / model width	độ sâu mô hình / độ rộng mô hình
model ensembling	kết hợp mô hình
model ensembling / ensemble learning	kết hợp mô hình / học kết hợp
model error	lỗi mô hình
model errors	lỗi mô hình
model errors / search errors	lỗi mô hình / lỗi tìm kiếm
model evaluation	đánh giá mô hình
model growth	tăng trưởng mô hình
model growth / depth growth	tăng trưởng mô hình / tăng trưởng độ sâu
Model Parallelism	song song mô hình
model selection	lựa chọn mô hình
model selection / model evaluation	lựa chọn mô hình / đánh giá mô hình
model width	độ rộng mô hình
momentum	động lượng; động lượng (quán tính)
monolingual	đơn ngữ
monotonicity	tính đơn điệu
Monte Carlo rollouts	triển khai thử Monte Carlo
Monte Carlo sampling	lấy mẫu Monte Carlo
Monte Carlo tree search	tìm kiếm cây Monte Carlo
morpheme	hình vị
morphological analysis	phân tích hình thái
morphology	hình thái học
moving average	trung bình trượt
moving average / cumulative average	trung bình trượt / trung bình tích lũy
multi-branch neural network	mạng nơ-ron đa nhánh

Tiếng Anh	Tiếng Việt
multi-class classification	phân loại đa lớp
multi-head attention	chú ý đa đầu
multi-head self-attention	tự chú ý đa đầu
multi-hop question answering	hỏi đáp đa bước
multi-label classification	phân loại đa nhãn
multi-layer attention	chú ý nhiều tầng
multi-layer neural network	mạng nơ-ron nhiều tầng
multi-layer perceptron	perceptron nhiều tầng
multilingual	đa ngôn ngữ
multilingual / cross-lingual	đa ngôn ngữ / xuyên ngôn ngữ
multilingual / monolingual / bilingual	đa ngôn ngữ / đơn ngữ / song ngữ
multimodal	đa phương thức
multi-modal model	mô hình đa phương thức
multinomial distribution	phân phối đa thức
multinomial naive Bayes	bayes ngây thơ đa thức
(multinomial/Bernoulli/Gaussian) naive Bayes	bayes ngây thơ (đa thức/Bernoulli/Gauss)
multi-objective optimization	tối ưu hóa đa mục tiêu
multiplicative / additive attention	chú ý nhân / chú ý cộng
multiplicative attention	chú ý nhân
multiplicative mask	mặt nạ nhân
multi-query attention	chú ý đa truy vấn
multi-query attention / grouped-query attention	chú ý đa truy vấn / chú ý truy vấn theo nhóm
multi-query attention, MQA	chú ý đa truy vấn
multi-round prediction	dự đoán nhiều lượt
multi-scale	đa tỉ lệ
multi-store model	mô hình đa kho
multi-task learning	học đa nhiệm
multivariate regression	hồi quy đa biến
N	
naive model parallelism	song song mô hình đơn giản
named entity	thực thể có tên
named entity / named entity recognition	thực thể có tên / nhận dạng thực thể có tên
named entity recognition	nhận dạng thực thể có tên
named entity recognition (NER)	nhận dạng thực thể có tên
natural language auto-encoder	bộ tự mã hóa ngôn ngữ tự nhiên
natural language processing, NLP	xử lý ngôn ngữ tự nhiên; xử lý ngôn ngữ tự nhiên (NLP)
nearest neighbor classification	phân loại láng giềng gần nhất
needle-in-a-haystack	mò kim đáy bể
negative evidence	bằng chứng phủ định
negative log-likelihood	log-hợp lý âm
negative sample	mẫu âm
Neural Architecture Search	tìm kiếm kiến trúc mạng nơ-ron; tìm kiếm kiến trúc nơ-ron
neural architecture search (NAS)	tìm kiếm kiến trúc nơ-ron (NAS); tìm kiếm kiến trúc mạng nơ-ron
neural language model	mô hình ngôn ngữ nơ-ron
neural language models	mô hình ngôn ngữ nơ-ron
neural machine translation	dịch máy nơ-ron
neural machine translation (NMT)	dịch máy nơ-ron (NMT)
neural network	mạng nơ-ron

Tiếng Anh	Tiếng Việt
neural ODE	ODE nơ-ron
neural Turing machine	máy Turing nơ-ron
neural Turing machines	máy Turing nơ-ron
neuron	nơ-ron
neurons	nơ-ron
next sentence prediction	dự đoán câu tiếp theo
next sentence prediction (NSP)	dự đoán câu tiếp theo (NSP); dự đoán câu tiếp theo
n-gram language modeling	mô hình ngôn ngữ n-gram; mô hình ngôn ngữ n-gram
n-gram precision	độ chính xác n-gram
No Free Lunch Theorem	định lý Không có bữa trưa miễn phí
node / edge / root node	nút / cạnh / nút gốc
nodes	nút
noise injection	tiêm nhiễu
non-autoregressive decoding	giải mã phi tự hồi quy
non-autoregressive generation	sinh phi tự hồi quy
non-autoregressive model	mô hình phi tự hồi quy
non-autoregressive model/generation/decoding	mô hình/sinh/giải mã phi tự hồi quy
non-convex	không lồi
non-linear activation functions	hàm kích hoạt phi tuyến
non-parametric	phi tham số
non-parametric / parametric methods	phương pháp phi tham số / có tham số
non-parametric methods	phương pháp phi tham số
non-parametric model	mô hình phi tham số
non-parametric tests	kiểm định phi tham số
norm	chuẩn
norm / p-norm	chuẩn / chuẩn p
norm (l_p, l_1, l_2)	chuẩn
normalization	chuẩn hóa
normalization / canonicalization	chuẩn hóa / chính tắc hóa
normalization / standardization	chuẩn hóa / chuẩn tắc hóa
normalization factor	hệ số chuẩn hóa
NP-hard	NP-khó
nucleus sampling	lấy mẫu hạt nhân
null / alternative hypothesis	giả thuyết không / giả thuyết đối
null hypothesis	giả thuyết không
numerical stability	ổn định số
numerically unstable	bất ổn định số
O	
objective function	hàm mục tiêu
Occam's Razor	dao cạo Occam
ODE solver	bộ giải ODE
offline / online reinforcement learning	học tăng cường ngoại tuyến / trực tuyến
offline reinforcement learning	học tăng cường ngoại tuyến
off-policy distillation	chưng cất off-policy
one-hot	one-hot
one-hot representation	biểu diễn one-hot
one-hot representation / expected embedding / expected log-probability	biểu diễn one-hot / vectơ nhúng kỳ vọng / log xác suất kỳ vọng
one-hot representations	biểu diễn one-hot

Tiếng Anh	Tiếng Việt
one-hot vector	vectơ one-hot
one-shot / few-shot / zero-shot CoT prompting	prompt CoT one-shot / few-shot / CoT zero-shot
one-shot CoT prompting	prompt CoT one-shot
one-shot learning	học one-shot
online	trực tuyến
online learning	học trực tuyến
online reinforcement learning	học tăng cường trực tuyến
online search / simultaneous translation / wait-k	tìm kiếm trực tuyến / dịch song song / wait-k
on-policy / off-policy distillation	chưng cất on-policy / off-policy
on-policy distillation	chưng cất on-policy
open-domain QA	hỏi đáp miền mở
open-vocabulary	từ vựng mở
optimization	tối ưu hóa
optimizer	bộ tối ưu
order-invariant	bất biến theo thứ tự
ordinary differential equation (ODE)	phương trình vi phân thường (ODE)
ordinary differential equations	phương trình vi phân thường
orthonormal vectors	vectơ trực chuẩn
outcome / process reward model (ORM/PRM)	mô hình phần thưởng theo kết quả / theo quá trình
outcome reward model	mô hình phần thưởng kết quả
outcome reward model (ORM)	mô hình phần thưởng kết quả (ORM)
outcome reward models	mô hình phần thưởng theo kết quả
outcome supervision	giám sát kết quả
outcome-based / process-based approaches	phương pháp dựa trên kết quả / dựa trên quá trình
outcome-based evaluation	đánh giá dựa trên kết quả
outcome-/process-based evaluation	đánh giá dựa trên kết quả / quá trình
outer product	tích ngoài
outliers	điểm ngoại lai
out-of-distribution	ngoài phân phối
out-of-distribution generalization	tổng quát hóa ngoài phân phối
out-of-vocabulary	từ ngoài bộ từ vựng; ngoài bộ từ vựng
out-of-vocabulary (OOV) word	từ ngoài bộ từ vựng (OOV)
output ensembling / ensembling	kết hợp đầu ra / kết hợp
output equation	phương trình đầu ra
output layer	tầng đầu ra
overfit	quá khớp
overfitting	quá khớp
overfitting / underfitting	quá khớp / chưa khớp
overflow	tràn trên
overflow / underflow	tràn trên / tràn dưới
overoptimization problem	vấn đề tối ưu hóa quá mức
over-parameterized	thừa tham số
over-translation	dịch thừa
over-translation / under-translation	dịch thừa / dịch thiếu
P	
padding	đệm; phần đệm
padding / left padding	đệm / đệm bên trái
padding / zero padding	đệm / đệm không
padding / zero-padding	phần đệm / đệm số 0

Tiếng Anh	Tiếng Việt
padding words	từ đệm
paged KV caching	lưu đệm KV phân trang
paged KV caching / paging	lưu đệm KV phân trang / phân trang
paging	phân trang
pairwise / listwise / pointwise ranking	xếp hạng theo cặp / theo danh sách / theo điểm
pairwise method	phương pháp theo cặp
pairwise ranking loss	hàm mất mát xếp hạng theo cặp
paradigm shift	chuyển dịch mô thức
parallel computation	tính toán song song
parallel data	dữ liệu song song
parallel scaling	co giãn song song
parallel scaling / sequential scaling	co giãn song song / co giãn tuần tự
parallelism	tính song song
parallelization	song song hóa
parameter	tham số
parameter / hyperparameter	tham số / siêu tham số
parameter initialization	khởi tạo tham số
parameter sharing	chia sẻ tham số
parameter sharing / weight sharing	chia sẻ tham số / chia sẻ trọng số
parameter-efficient fine-tuning	tinh chỉnh hiệu quả tham số
parameters	tham số
parametric / non-parametric test	kiểm định tham số / phi tham số
parametric methods	phương pháp có tham số
parametric test	kiểm định tham số
paraphrasing	diễn đạt lại
parse tree	cây phân tích cú pháp
parsing	phân tích cú pháp
partial derivative	đạo hàm riêng
partial derivatives	đạo hàm riêng
partial hypothesis	giả thuyết bộ phận
part-of-speech	từ loại
part-of-speech, POS	từ loại
part-of-speech (POS) tagging	gán nhãn từ loại
part-of-speech tagging	gán nhãn từ loại; từ loại (gán nhãn từ loại)
part-of-speech tagging / chunking	gán nhãn từ loại / phân đoạn cụm
part-of-speech tagging, POS tagging	gán nhãn từ loại
passkey retrieval	truy xuất mật mã
patch	mảnh; mảnh (ảnh)
paths	đường
pause token	token tạm dừng
penalty term	số hạng phạt
penalty term / repetition / length / diversity / constraint-based penalty	số hạng phạt / phạt lặp lại / độ dài / đa dạng / dựa trên ràng buộc
performance function	hàm hiệu năng
performance gap recovered	tỉ lệ khoảng cách hiệu năng được thu hồi (PGR)
performance gap recovered (PGR)	tỉ lệ khoảng cách hiệu năng được thu hồi (PGR)
performance measure	thước đo hiệu năng
permutation invariance	bất biến hoán vị
permutation-invariant	bất biến với hoán vị

Tiếng Anh	Tiếng Việt
permuted language modeling	mô hình hóa ngôn ngữ hoán vị
perplexity	độ hỗn loạn
perturb	nhiễu loạn
pipeline	dây chuyền
Pipeline Parallelism	song song đường ống
placeholder	chỗ trống
Plackett-Luce model	mô hình Plackett-Luce
planning	lập kế hoạch
p-norm	chuẩn p
point estimate	ước lượng điểm
point estimates	ước lượng điểm
pointwise	theo từng điểm
pointwise methods	phương pháp theo điểm
pointwise mutual information	thông tin tương hỗ theo điểm
pointwise mutual information (PMI)	thông tin tương hỗ theo điểm
polarity	cực tính
policy	chính sách; chính sách (policy)
policy gradient	gradient chính sách
policy network	mạng chính sách
polysemous word	từ đa nghĩa
polysemy	đa nghĩa
pooling	gộp
pooling / average pooling	gộp / gộp trung bình
pooling layer	tầng gộp
pooling layer / max-pooling / average-pooling	tầng gộp / gộp cực đại / gộp trung bình
population	tổng thể
position interpolation	nội suy vị trí
positional embedding	nhúng vị trí; vectơ nhúng vị trí
positional encoding	mã hóa vị trí
positional encoding / embedding	mã hóa vị trí / nhúng vị trí
positional encodings	mã hóa vị trí
positive / negative sample	mẫu dương / mẫu âm
positive sample	mẫu dương
posterior	phân phối hậu nghiệm
posterior probability	xác suất hậu nghiệm; (xác suất) hậu nghiệm
post-norm	chuẩn hóa sau
post-norm / pre-norm	post-norm (chuẩn hóa sau) / pre-norm (chuẩn hóa trước); chuẩn hóa sau (post-norm) / chuẩn hóa trước (pre-norm); chuẩn hóa sau / chuẩn hóa trước
post-training	hậu huấn luyện
post-training quantization	lượng tử hóa sau huấn luyện
potential value	giá trị thế năng
power law	luật lũy thừa
power-law function	hàm lũy thừa
precision	độ chính xác dự đoán
precision / recall / F1	độ chính xác dự đoán / độ phủ / điểm F1
predicates	vị từ
prediction averaging	lấy trung bình dự đoán
prediction network	mạng dự đoán

Tiếng Anh	Tiếng Việt
prediction network / predictor	mạng dự đoán / bộ dự đoán
predictor	bộ dự đoán
prefill-decode	điền trước–giải mã
prefill-decode / prefilling	điền trước–giải mã / điền trước
prefilling	điền trước
prefix	tiền tố
prefix caching	lưu đệm tiền tố
prefix fine-tuning	tinh chỉnh tiền tố
prefix fine-tuning / prefix	tinh chỉnh tiền tố / tiền tố
prefix language modeling	mô hình hóa ngôn ngữ tiền tố
prefix-to-prefix	tiền tố-sang-tiền tố
pre-norm	chuẩn hóa trước
pretext task	bài toán tiền đề
pre-tokenizer	bộ tách token sơ bộ
pre-train	tiền huấn luyện
pre-trained	tiền huấn luyện
pre-training	tiền huấn luyện
pre-training / fine-tuning	tiền huấn luyện / tinh chỉnh
prime word	từ mồi
priming	mồi
priming / prime word / target word	mồi / từ mồi / từ đích
principal component	thành phần chính
principal component analysis	phân tích thành phần chính
principal component analysis, PCA	phân tích thành phần chính
principal component coefficients	hệ số thành phần chính
principal component loadings	tải thành phần chính
prior	xác suất tiên nghiệm; phân phối tiên nghiệm
prior distribution	phân phối tiên nghiệm
prior knowledge	tri thức tiên nghiệm
prior knowledge / prior	tri thức tiên nghiệm / phân phối tiên nghiệm
prior probability	xác suất tiên nghiệm
priority queue	hàng đợi ưu tiên
priority-based scheduling	lập lịch dựa trên mức ưu tiên
priors	tri thức tiên nghiệm; tri thức tiên nghiệm / tiên nghiệm
probability	xác suất
probability density	mật độ xác suất
probability density function	hàm mật độ xác suất
probability density function, PDF	hàm mật độ xác suất; mật độ xác suất (hàm mật độ xác suất)
probability distribution	phân phối xác suất
probability distributions	phân phối xác suất
probability function	hàm xác suất
probability mass function, PMF	hàm khối xác suất
probability measure	độ đo xác suất
probe / probing predictor / probing classifier	phép thăm dò / bộ dự đoán thăm dò / bộ phân loại thăm dò
probes	phép thăm dò
probing classifier	bộ phân loại thăm dò
probing predictor	bộ dự đoán thăm dò
problem decomposition	phân rã bài toán
process reward model	mô hình phần thưởng quá trình

Tiếng Anh	Tiếng Việt
process reward model, PRM	mô hình phần thưởng quá trình (PRM); mô hình phần thưởng quá trình
process reward models	mô hình phần thưởng theo quá trình
process supervision	giám sát quá trình
process supervision / outcome supervision	giám sát quá trình / giám sát kết quả
process-based evaluation	đánh giá dựa trên quá trình
product key	khóa tích
product rule	quy tắc nhân
product rule / chain rule (probability)	quy tắc nhân / quy tắc chuỗi
progressive downsampling	giảm mẫu lũy tiến
projection matrix	ma trận chiếu
prompt	prompt
prompt / prompting / prompt engineering	prompt / kỹ thuật prompt / thiết kế prompt
prompt embedding	vectơ nhúng prompt
prompt engineering	thiết kế prompt; prompt
prompt ensembling / output ensembling	kết hợp prompt / kết hợp đầu ra
prompt generator	bộ sinh prompt
prompt optimization	tối ưu hóa prompt
prompt template	khuôn mẫu prompt
prompt template / template	khuôn mẫu prompt / khuôn mẫu
prompt tuning	tinh chỉnh prompt
prompting	kỹ thuật prompt; prompting
proximal policy optimization	tối ưu chính sách cận kề
proximal policy optimization (PPO)	tối ưu chính sách cận kề (PPO)
proxy	đại diện thay thế
prune	tỉa
pruning	cắt tỉa; tỉa
pruning (histogram / threshold)	cắt tỉa (theo biểu đồ tần suất / theo ngưỡng)
pseudo embedding	vectơ nhúng giả
pseudo embedding / pseudo token	vectơ nhúng giả / token giả
pseudo token	token giả
pseudo-label	nhãn giả
p-value	giá trị p
Q	
quadratic programming	quy hoạch toàn phương
quality estimation	ước lượng chất lượng
quality of service	chất lượng dịch vụ
quantization	lượng tử hóa
quantization / post-training quantization	lượng tử hóa / lượng tử hóa sau huấn luyện
quantization / pruning	lượng tử hóa / tỉa
queries	truy vấn
query	truy vấn
query / key / value	truy vấn / khóa / giá trị
query-key-value attention	chú ý truy vấn–khóa–giá trị
question answering	hỏi đáp
Q-value function	hàm giá trị Q
R	
random attention	chú ý ngẫu nhiên
random seed	hạt giống ngẫu nhiên

Tiếng Anh	Tiếng Việt
random seeds	hạt giống ngẫu nhiên
random subsampling	lấy mẫu con ngẫu nhiên
random variable	biến ngẫu nhiên
ranking	xếp hạng
ranking loss	mất mát xếp hạng; hàm mất mát xếp hạng
ranking loss / pairwise / pointwise / listwise / learning-to-rank	mất mát xếp hạng / theo cặp / theo từng điểm / theo danh sách / học xếp hạng
ranking-based loss	mất mát dựa trên xếp hạng
ratio function	hàm tỉ số
reading comprehension	đọc hiểu
reasoning	lập luận; lập luận (reasoning) — phân biệt với inference = ” suy luận” (chạy mô hình)
reasoning (chương 11)	lập luận
reasoning (chương 12)	lập luận
reasoning model	mô hình lập luận
reasoning models	mô hình lập luận
reasoning path	đường lập luận
reasoning path / reasoning step	đường lập luận / bước lập luận
reasoning steps	bước lập luận
reasoning trace	vết lập luận
recall	độ phủ
receptive field	trường tiếp nhận
reconstruction loss	mất mát tái tạo
rectified linear unit, ReLU	đơn vị tuyến tính chỉnh lưu
recurrent cell	ô hồi quy
recurrent cell / memory cell	ô hồi quy / ô nhớ
recurrent neural network	mạng nơ-ron hồi quy
recurrent neural network (RNN)	mạng nơ-ron hồi quy (RNN)
recurrent neural networks	mạng nơ-ron hồi quy
recurrent neural networks, RNNs	mạng nơ-ron hồi quy
recurrent state	trạng thái hồi quy
recurrent unit	đơn vị hồi quy
recurrent unit / recurrent cell	đơn vị hồi quy / ô hồi quy
reduction operation	phép rút gọn
redundancy	dư thừa
redundancy / over-parameterized	dư thừa / thừa tham số
reference	tham chiếu
reference model	mô hình tham chiếu
reference policy	chính sách tham chiếu
reference policy / reference model / target model	chính sách tham chiếu / mô hình tham chiếu / mô hình đích
reference translations	bản dịch tham khảo
regression	hồi quy
regression / multivariate regression	hồi quy / hồi quy đa biến
regular expression	biểu thức chính quy
regular expressions	biểu thức chính quy
regularization	chính quy hóa
regularization term	số hạng chính quy hóa
regularizer	bộ chính quy hóa
reinforcement learning	học tăng cường

Tiếng Anh	Tiếng Việt
reinforcement learning from human feedback	học tăng cường từ phản hồi của con người; học tăng cường từ phản hồi của con người (RLHF)
reinforcement learning from human feedback (RLHF)	học tăng cường từ phản hồi của con người (RLHF)
reinforcement learning, RL	học tăng cường
rejection sampling	lấy mẫu loại bỏ
relation extraction	trích xuất quan hệ
relative entropy	entropy tương đối
relative frequency	tần suất tương đối
relative positional embedding	nhúng vị trí tương đối
relative positional embedding / representation (RPR)	nhúng vị trí tương đối / biểu diễn vị trí tương đối (RPR)
relative positional encoding	mã hóa vị trí tương đối
relative positional representation	biểu diễn vị trí tương đối
relevance	mức độ liên quan
relevance / similarity learning / metric learning	mức độ liên quan / học độ tương đồng / học độ đo
renormalize	chuẩn hóa lại
reordering	đảo trật tự
reordering problem	bài toán sắp xếp lại trật tự
reparameterization trick	mẹo tái tham số hóa
repetition penalty	phạt lặp lại
representation	biểu diễn
representation learning	học biểu diễn
representation learning / feature learning	học biểu diễn / học đặc trưng
representation subspaces	không gian con biểu diễn
representational capacity	năng lực biểu diễn
request latency / throughput / TTFT / ITL / resource utilization	độ trễ yêu cầu / thông lượng / thời gian đến token đầu tiên / độ trễ giữa các token / mức sử dụng tài nguyên
request-level scheduling	lập lịch mức yêu cầu
reranking	xếp hạng lại
resampling	lấy mẫu lại
rescore	chấm điểm lại
research paradigm	mô thức nghiên cứu
reset gate	cổng đặt lại
reset gate / update gate	cổng đặt lại / cổng cập nhật
residual block	khối phần dư
residual block / identity mapping	khối phần dư / ánh xạ đồng nhất
residual connection	kết nối tắt; kết nối tắt (residual)
residual connection / skip connection / shortcut connection	kết nối tắt (residual) / kết nối bỏ qua / kết nối đường tắt
residual connections	kết nối tắt; kết nối tắt (residual)
residual network	mạng phần dư
residual neural networks	mạng nơ-ron phần dư
residual stream	dòng residual
retrieval-augmented generation	sinh văn bản tăng cường truy xuất
retrieval-augmented generation, RAG	sinh văn bản tăng cường truy xuất; sinh văn bản tăng cường truy xuất (RAG)
return	lợi tức
reusable neural networks	mạng nơ-ron tái sử dụng

Tiếng Anh	Tiếng Việt
reversible residual network	mạng phần dư khả nghịch
reward	phần thưởng
reward / reward model	phần thưởng / mô hình phần thưởng
reward gaming	lợi dụng phần thưởng
reward hacking	lách phần thưởng; khai thác lỗ hổng phần thưởng
reward hacking / reward gaming	lách phần thưởng / lợi dụng phần thưởng
reward model	mô hình phần thưởng
reward modeling	mô hình hóa phần thưởng
reward shaping	định hình phần thưởng
reward shaping / shaping reward function	định hình phần thưởng / hàm phần thưởng định hình
Ridge regularization	chính quy hóa Ridge
right-singular vectors	vectơ suy biến phải
risk	rủi ro
risk function	hàm rủi ro
risk function / evaluation metric	hàm rủi ro / độ đo đánh giá
RMS layer normalization	chuẩn hóa tầng theo căn trung bình bình phương
robust	bền vững
robust statistics	thống kê vững
robustness	độ bền vững; độ vững; bền vững
rolled	cuộn
rollout	triển khai
root mean square	căn quân phương
root mean square (RMS)	căn quân phương
root mean square (RMS) layer normalization	chuẩn hóa tầng theo căn trung bình bình phương
root node	nút gốc
rotary position embedding	nhúng vị trí xoay
rotary position embeddings	nhúng vị trí xoay
rotary positional embedding (RoPE)	nhúng vị trí xoay
rotary positional embeddings	nhúng vị trí xoay
rotation matrix	ma trận quay
routing model	mô hình định tuyến
row vector	vectơ hàng
row vector / column vector	vectơ hàng / vectơ cột
S	
salt-and-pepper noise	nhiễu muối tiêu
sample bias	thiên kiến mẫu
sample efficiency	hiệu quả mẫu
sample space	không gian mẫu
samples	mẫu
sampling	lấy mẫu
sampling bias	sai lệch lấy mẫu
saturating / non-saturating activation	hàm kích hoạt bão hòa / không bão hòa
saturating activation functions	hàm kích hoạt bão hòa
scalability	khả năng mở rộng
scalar	số vô hướng
scalar / vector / matrix / tensor	vô hướng / vectơ / ma trận / tensor
scalar multiplication	phép nhân với số vô hướng
scaled dot-product attention	chú ý tích vô hướng có co giãn
scaling law	định luật co giãn; định luật co giãn (scaling law)

Tiếng Anh	Tiếng Việt
scaling laws	định luật co giãn
scheduler / inference engine	bộ lập lịch / bộ máy suy luận
scoring function	hàm tính điểm
search	tìm kiếm
search error	lỗi tìm kiếm
search error / model error	lỗi tìm kiếm / lỗi mô hình
search errors	lỗi tìm kiếm
search scaling	mở rộng quy mô tìm kiếm; co giãn tìm kiếm
search space	không gian tìm kiếm
second raw moment	mô-men gốc bậc hai
second-order methods	phương pháp bậc hai
seed data	dữ liệu hạt giống
seed data / pseudo-label	dữ liệu hạt giống / nhãn giả
seed prompt	prompt hạt giống
segment	đoạn
segment embedding	nhúng đoạn
segmentation	phân đoạn
selection pool	tập chọn
self-attention	tự chú ý
self-consistency	tự nhất quán
self-correction	tự sửa lỗi
self-distillation	tự chưng cất
self-instruct	tự tạo chỉ dẫn
self-paced reading	đọc tự điều chỉnh nhịp
self-refinement	tự tinh chỉnh
self-reflection	tự phản tư
self-supervised	tự giám sát
self-supervised learning	học tự giám sát
self-training	tự huấn luyện
semantic argument	tham tố ngữ nghĩa
semantic equivalence	tương đương ngữ nghĩa
semantic parsing	phân tích ngữ nghĩa
semantic relatedness	độ liên quan ngữ nghĩa
semantic relatedness / similarity	độ liên quan / độ tương tự ngữ nghĩa
semantic similarity	độ tương tự ngữ nghĩa
semi-orthogonal	nửa trực giao
semi-supervised learning	học bán giám sát
semi-unitary	nửa unita
semi-unitary / semi-orthogonal	nửa unita / nửa trực giao
sensory / short-term / long-term memory	trí nhớ giác quan / ngắn hạn / dài hạn
sensory memory	trí nhớ giác quan
sentence embedding	vectơ nhúng câu; nhúng câu
sentence length prediction	dự đoán độ dài câu
sentence representation	biểu diễn câu
sentence-level / token-level depth-adaptive model	mô hình thích nghi độ sâu mức câu / mức token
sentence-level depth-adaptive model	mô hình thích nghi độ sâu mức câu
sentiment analysis	phân tích cảm xúc
sentiment analysis / polarity	phân tích cảm xúc / cực tính

Tiếng Anh	Tiếng Việt
sentiment polarity	cực cảm xúc; cực tính cảm xúc
sentinel token	token canh gác
separating hyperplane	siêu phẳng phân tách
separating hyperplane / decision boundary	siêu phẳng phân tách / biên quyết định
separator	ký hiệu phân tách
sequence / sequence labeling	chuỗi / gán nhãn chuỗi
sequence encoding	mã hóa chuỗi
sequence generation	sinh chuỗi
sequence labeling	gán nhãn chuỗi
sequence modeling	mô hình hóa chuỗi
sequence parallelism	song song chuỗi
sequence-level ensembling	kết hợp mức chuỗi
sequence-level loss	mất mát mức chuỗi
sequence-to-sequence	chuỗi-sang-chuỗi
sequential scaling	co giãn tuần tự
shallow model	mô hình nông
shallow models	mô hình nông
shallow-to-deep training	huấn luyện từ nông đến sâu
shaping reward function	hàm phần thưởng định hình
shared activation	chia sẻ giá trị kích hoạt
shared encoder	bộ mã hóa dùng chung
shared weight	chia sẻ trọng số
shatter	tách trọn vẹn
shortcut connections	kết nối đường tắt
short-term memory	trí nhớ ngắn hạn
shrinkage estimator	bộ ước lượng co
sigmoid function	hàm sigmoid
significance level	mức ý nghĩa
significance test / significance level / p-value	kiểm định ý nghĩa thống kê / mức ý nghĩa / giá trị p
significance tests	kiểm định ý nghĩa thống kê
similarity	độ tương đồng
similarity function	hàm tương đồng
similarity learning	học độ tương đồng
similarity search	tìm kiếm tương tự
simplex	đơn hình
simplex / constrained optimization	đơn hình / tối ưu có ràng buộc
simultaneous translation	dịch song song
single-label classification	phân loại đơn nhãn
single-layer / multi-layer attention	chú ý một tầng / chú ý nhiều tầng
single-layer / multi-layer neural network	mạng nơ-ron một tầng / nhiều tầng
single-layer attention	chú ý một tầng
single-layer neural network	mạng nơ-ron một tầng
single-layer perceptron	perceptron một tầng
single-layer perceptrons	perceptron một tầng
single-round / multi-round prediction	dự đoán một lượt / nhiều lượt
single-round prediction	dự đoán một lượt
singular value / left-/right-singular vector	giá trị suy biến / vectơ suy biến trái/phải
singular value decomposition	phân tích giá trị suy biến
singular value decomposition (SVD)	phân tích giá trị suy biến

Tiếng Anh	Tiếng Việt
singular values	giá trị suy biến
sinusoidal encoding	mã hóa hình sin
sinusoidal positional encodings	mã hóa vị trí hình sin
skip connection	kết nối nhảy
skip connections	kết nối bỏ qua; kết nối nhảy
sliding window attention	chú ý cửa sổ trượt
sliding-chunk local memory	bộ nhớ cục bộ dạng khối trượt
smoothed	làm trơn
smoothing	làm trơn
soft / hard constraint	ràng buộc mềm / ràng buộc cứng
soft constraint	ràng buộc mềm
soft prompt	prompt mềm
soft prompts	prompt mềm
soft voting	bỏ phiếu mềm
soft word alignment matrix	ma trận gióng hàng từ mềm
software frameworks	khung phần mềm
source language	ngôn ngữ nguồn
source-side / target-side sequence	chuỗi phía nguồn / chuỗi phía đích
source-side sequence	chuỗi phía nguồn
space complexity	độ phức tạp không gian
span	đoạn
span / sentinel token	đoạn / token canh gác
span / span prediction	đoạn / dự đoán đoạn
span prediction	dự đoán đoạn
span-based / local attention	chú ý dựa trên khoảng / chú ý cục bộ
span-based attention	chú ý dựa trên khoảng
sparse	thưa
sparse / dense reward	phần thưởng thưa / dày
sparse / dense rewards	phần thưởng thưa / dày
sparse / sparsity penalty	thưa / hàm phạt thưa
sparse attention	chú ý thưa
sparse attention / linear attention	chú ý thưa / chú ý tuyến tính
sparse attention models	mô hình chú ý thưa
sparse auto-encoder	bộ tự mã hóa thưa
sparse auto-encoder / sparse coding	bộ tự mã hóa thưa / mã hóa thưa
sparse coding	mã hóa thưa
sparse expert model	mô hình chuyên gia thưa
sparse matrix	ma trận thưa
sparse reward	phần thưởng thưa
sparse reward / reward shaping	phần thưởng thưa / định hình phần thưởng
sparse rewards	phần thưởng thưa
sparse vector	vectơ thưa
sparsification	làm thưa
sparsification / sparse matrix / sparsity ratio	làm thưa / ma trận thưa / tỷ lệ thưa
sparsity penalty	hàm phạt thưa
sparsity ratio	tỷ lệ thưa
spectral frame	khung phổ
speculative decoding	giải mã suy đoán
speculative decoding / speculative execution	giải mã suy đoán / thực thi suy đoán

Tiếng Anh	Tiếng Việt
speculative execution	thực thi suy đoán
speech encoder	bộ mã hóa tiếng nói
speech encoder / text encoder / shared encoder	bộ mã hóa tiếng nói / văn bản / dùng chung
speech processing	xử lý tiếng nói
speech recognition	nhận dạng tiếng nói
speed-accuracy trade-off	đánh đổi tốc độ–độ chính xác
spike	đột biến
squared loss	hàm mất mát bình phương
stacking	xếp chồng
stale gradient	gradient lỗi thời
stale gradients	gradient lỗi thời
standard deviation	độ lệch chuẩn
standard normal distribution	phân phối chuẩn tắc
standardization	chuẩn tắc hóa
state	trạng thái
state / state-value function / action-value function (Q-value function)	trạng thái / hàm giá trị trạng thái / hàm giá trị hành động (hàm giá trị Q)
state equation	phương trình trạng thái
state equation / output equation	phương trình trạng thái / phương trình đầu ra
state space model	mô hình không gian trạng thái
state space models	mô hình không gian trạng thái
state variable	biến trạng thái
state-of-the-art	tiên tiến nhất
state-space model	mô hình không gian trạng thái
state-space model (SSM)	mô hình không gian trạng thái
state-value function	hàm giá trị trạng thái
statistical inference	suy diễn thống kê
statistical language modeling	mô hình hóa ngôn ngữ thống kê
statistical machine translation	dịch máy thống kê
statistical machine translation, SMT	dịch máy thống kê
statistical parsing	phân tích cú pháp thống kê
steepest descent	hạ dốc nhất
stem / stemming	gốc từ / lấy gốc từ
stemming	lấy gốc từ
step function	hàm bậc thang
step-level search	tìm kiếm mức bước
step-level supervision	giám sát ở mức bước
stochastic gradient descent	hạ gradient ngẫu nhiên
stochastic gradient descent (SGD)	hạ gradient ngẫu nhiên (SGD)
stochastic parrots	con vẹt ngẫu nhiên
stochastic process / random process	quá trình ngẫu nhiên
stopping criteria	tiêu chí dừng
stopping criteria / early stopping	tiêu chí dừng / dừng sớm
stopping criterion	tiêu chí dừng
stopping criterion / early stopping	tiêu chí dừng / dừng sớm
stream	luồng
stride	bước trượt
strided attention	chú ý theo bước trượt
strided convolution	tích chập có bước nhảy

Tiếng Anh	Tiếng Việt
structural prior	tiên nghiệm cấu trúc
structural priors	tiên nghiệm cấu trúc
structure prediction	dự đoán cấu trúc
structured pruning	cắt tỉa có cấu trúc
student model	mô hình học trò; mô hình trò
student model / teacher model	mô hình học trò / mô hình thầy
subconscious evaluation	đánh giá tiềm thức
sub-layer	tầng con
sub-layer dropout	dropout tầng con
sub-layer dropout / layer dropout	dropout tầng con / dropout tầng
sub-layers	tầng con
sub-optimal	dưới tối ưu
sub-problem generation / solving	sinh bài toán con / giải bài toán con
subspace	không gian con
subspace method	phương pháp không gian con
sub-space model	mô hình không gian con
subspaces	không gian con
subword	từ con (subword); từ con
suffix stripping	cắt hậu tố
summarization	tóm tắt; tóm tắt văn bản
summary token	token tóm tắt
superficial alignment hypothesis	giả thuyết căn chỉnh bề mặt
supervised / unsupervised / self-supervised learning	học có giám sát / không giám sát / tự giám sát
supervised fine-tuning	tinh chỉnh có giám sát; tinh chỉnh có giám sát (SFT)
supervised fine-tuning (SFT)	tinh chỉnh có giám sát (SFT)
supervised learning	học có giám sát
support vector machine (SVM)	máy vectơ hỗ trợ
support vector machines	máy vectơ hỗ trợ
surrogate losses	hàm mất mát thay thế
surrogate objective	mục tiêu thay thế
symbol bigram	bigram ký hiệu
symbolic AI	AI ký hiệu
synchronization	đồng bộ hóa
syntactic hierarchy	phân cấp cú pháp
syntactic parser	bộ phân tích cú pháp
syntactic parser / parse tree / statistical parsing	bộ phân tích cú pháp / cây phân tích cú pháp / phân tích cú pháp thống kê
syntax tree	cây cú pháp
syntax tree / constituency parse tree / dependency parse tree	cây cú pháp / cây phân tích thành phần / cây phân tích phụ thuộc
syntax-aware	có nhận biết cú pháp
syntax-aware Transformer encoders	bộ mã hóa Transformer có nhận biết cú pháp
synthetic data generation	sinh dữ liệu tổng hợp
synthetic language	ngôn ngữ tổng hợp
synthetic tasks	tác vụ tổng hợp
system combination	kết hợp hệ thống
system prompt	prompt hệ thống
systematic error	sai số hệ thống

Tiếng Anh	Tiếng Việt
T	
T5 bias	độ lệch T5
target language	ngôn ngữ đích
target word	từ đích
target-side sequence	chuỗi phía đích
Taylor series	chuỗi Taylor
t-distributed stochastic neighbor embedding	nhúng láng giềng ngẫu nhiên phân phối t
teacher forcing	teacher forcing
teacher model	mô hình thầy
teacher model / student model	mô hình thầy / mô hình trò
teacher-student training	huấn luyện thầy–trò
telescoping sum	tổng rút gọn kiểu ống lồng
temperature	nhiệt độ
temperature sampling	lấy mẫu với nhiệt độ
temperature sampling / nucleus sampling	lấy mẫu với nhiệt độ / lấy mẫu hạt nhân
temperature scaling	co giãn nhiệt độ
template	khuôn mẫu
template filling	điền mẫu
temporal difference	chênh lệch thời gian
temporal difference error	sai số chênh lệch thời gian
temporal difference (TD)	chênh lệch thời gian (TD)
temporal-difference error	sai số chênh lệch thời gian
Tensor Parallelism	song song tensor
tensors	tensor
term frequency	tần suất từ
term frequency (TF)	tần suất từ
term frequency-inverse document frequency	tần suất từ – nghịch đảo tần suất văn bản
term-document co-occurrence matrix	ma trận đồng xuất hiện thuật ngữ–văn bản
terminal state	trạng thái kết thúc
term-term co-occurrence matrix	ma trận đồng xuất hiện thuật ngữ–thuật ngữ
test dataset	tập dữ liệu kiểm tra
test error	lỗi kiểm tra
test statistic	thống kê kiểm định
test-time compute	tính toán lúc kiểm tra
text classification	phân loại văn bản
text completion	hoàn thành văn bản
text embedding	nhúng văn bản
text embedding / sentence embedding	nhúng văn bản / nhúng câu
text encoder	bộ mã hóa văn bản
text entailment	kéo theo văn bản
text generation	sinh văn bản
text simplification	đơn giản hóa văn bản
text style transfer	chuyển đổi văn phong
text summarization	tóm tắt văn bản
text transformation	biến đổi văn bản
text-to-text	văn bản-sang-văn bản
the Bayesian approach	cách tiếp cận Bayes
the curse of dimensionality	lời nguyền số chiều
the curse of multilinguality	lời nguyền đa ngôn ngữ

Tiếng Anh	Tiếng Việt
the kernel trick	thủ thuật hạt nhân
thinking steps	bước suy nghĩ
threshold function	hàm ngưỡng
threshold pruning	cắt tỉa theo ngưỡng
throughput	thông lượng
tile-based	chia ô
time complexity	độ phức tạp thời gian
time series	chuỗi thời gian
time step	bước thời gian
time-series	chuỗi thời gian
token / tokenization	token / tách token
token embedding	vectơ nhúng token; nhúng token
token pruning	cắt tỉa token
tokenization	tách token
tokenizer	bộ tách token
token-level depth-adaptive model	mô hình thích nghi độ sâu mức token
tool use	sử dụng công cụ
tool-augmented reasoning	lập luận tăng cường công cụ
top-k / top-p sampling, nucleus sampling	lấy mẫu top-k / top-p, lấy mẫu hạt nhân
topologically sorted	sắp xếp tô-pô
trace	vết
training	huấn luyện
training / inference	huấn luyện / suy luận
training dataset	tập dữ liệu huấn luyện
training epochs	epoch huấn luyện
training error	lỗi huấn luyện
training error / test error / generalization error	lỗi huấn luyện / lỗi kiểm tra / lỗi tổng quát hóa
training instability	bất ổn định khi huấn luyện
training set / validation set / test set	tập huấn luyện / tập kiểm định / tập kiểm tra
training step	bước huấn luyện
training-evaluation discrepancy	sự lệch pha giữa huấn luyện và đánh giá
training-time scaling	mở rộng quy mô lúc huấn luyện
trajectory	quỹ đạo
transcription	bản phiên âm
transcription / transcription units / labels	bản phiên âm / đơn vị phiên âm / nhãn phiên âm
transcription labels	nhãn phiên âm
transcription units	đơn vị phiên âm
transfer learning	học chuyển giao
Transformer	transformer
transition / emission probability	xác suất chuyển / xác suất phát xạ
translation invariance	tính bất biến đối với phép tịnh tiến
translation language modeling	mô hình hóa ngôn ngữ dịch
translation language modeling (TLM)	mô hình hóa ngôn ngữ dịch
transpose	chuyển vị
traveling salesman problem	bài toán người du lịch
tree	cây
tree linearization	tuyến tính hóa cây
tree search	tìm kiếm cây
tree search / step-level search	tìm kiếm cây / tìm kiếm mức bước

Tiếng Anh	Tiếng Việt
trial and error	thử và sai
truncated back-propagation	lan truyền ngược bị cắt ngắn
trust region	vùng tin cậy
t-SNE	nhúng láng giềng ngẫu nhiên phân phối t
Type I / Type II error	sai lầm loại I / loại II
Type I error	sai lầm loại I
Type II error	sai lầm loại II
U	
undercomplete auto-encoder	bộ tự mã hóa thiếu đủ
underdetermination	bất định; tính bất định
underfitting	chưa khớp
underflow	tràn dưới
under-translation	dịch thiếu
unfaithful	không trung thực
unfolded	mở rộng
uni-directional / bi-directional model	mô hình một chiều / hai chiều
uni-directional models	mô hình một chiều
uniform / binomial / Bernoulli / Gaussian distribution	phân phối đều / nhị thức / Bernoulli / Gauss
uniform distribution	phân phối đều
unlabeled data	dữ liệu không gán nhãn
unrolled	trải phẳng
unrolled / unfolded / rolled	trải phẳng / mở rộng / cuộn
unseen words	từ chưa gặp
unstructured / structured pruning	cắt tỉa phi cấu trúc / có cấu trúc
unstructured pruning	cắt tỉa phi cấu trúc
unsupervised bilingual dictionary induction	quy nạp từ điển song ngữ không giám sát
unsupervised learning	học không giám sát
update gate	cổng cập nhật
update rule	quy tắc cập nhật
update rule / delta rule	quy tắc cập nhật / quy tắc delta
update step	bước cập nhật
upsampling	tăng mẫu
upsampling / downsampling	tăng mẫu / giảm mẫu
utility function	hàm tiện ích
V	
validation	kiểm định
validation data	dữ liệu kiểm định
validation dataset	tập dữ liệu kiểm định
validation set	tập kiểm định
value	giá trị
value baseline	giá trị cơ sở
value baseline / value network	giá trị cơ sở / mạng giá trị
value function	hàm giá trị
value network	mạng giá trị
value-based	dựa trên giá trị
value-based search	tìm kiếm dựa trên giá trị
values	giá trị

Tiếng Anh	Tiếng Việt
vanishing / exploding gradient	gradient biến mất / gradient bùng nổ; tiêu biến gradient / bùng nổ gradient
vanishing gradient	gradient biến mất; tiêu biến gradient
Vapnik-Chervonenkis (VC) dimension	chiều Vapnik–Chervonenkis (VC)
variance	phương sai
variance / standard deviation	phương sai / độ lệch chuẩn
variational auto-encoder (VAE)	bộ tự mã hóa biến phân
variational auto-encoders, VAEs	bộ tự mã hóa biến phân
variational Bayesian methods	phương pháp Bayes biến phân
variational posterior	phân phối hậu nghiệm biến phân
VC dimension	chiều VC
vector	vectơ
vector database	cơ sở dữ liệu vectơ
vector space	không gian vectơ
vector space model	mô hình không gian vectơ
verification model	mô hình kiểm chứng
verifier	bộ kiểm chứng
verifier / generative verification	bộ kiểm chứng / kiểm chứng sinh
verifier / outcome-based / process-based	bộ kiểm chứng / dựa trên kết quả / dựa trên quá trình
Vision Transformer	vision Transformer
visual question answering	hỏi đáp trực quan; hỏi đáp thị giác
visual question answering (VQA)	hỏi đáp trực quan; hỏi đáp thị giác
Viterbi decoding	giải mã Viterbi
vocabulary	bộ từ vựng
W	
wait-k	wait-k
warmup	khởi động
weak-to-strong generalization	tổng quát hóa từ yếu đến mạnh
web scraping	thu thập dữ liệu web tự động
weight	trọng số
weight / bias	trọng số / độ lệch (bias)
weight decay	suy giảm trọng số
weight sharing	chia sẻ trọng số
weighted averaging	trung bình có trọng số
win rate	tỉ lệ thắng
word alignment	gióng hàng từ
word alignment weight matrix	ma trận trọng số gióng hàng từ
word analogy	loại suy từ
word clustering	phân cụm từ
word distance	khoảng cách từ
word embedding	nhúng từ; vectơ nhúng từ
word embedding / embedding	nhúng từ / vectơ nhúng (embedding)
word embedding table	bảng nhúng từ
word embeddings	vectơ nhúng từ; nhúng từ
word lattice	lưới từ
word reordering	sắp xếp lại trật tự từ
word semantic distance	khoảng cách ngữ nghĩa của từ
word sense	nghĩa từ
word sense / word sense disambiguation (WSD)	nghĩa từ / khử nhập nhằng nghĩa từ (WSD)

Tiếng Anh	Tiếng Việt
word sense disambiguation	khử nhập nhằng nghĩa của từ
word sense disambiguation, WSD	khử nhập nhằng nghĩa từ
word vector	vectơ từ
word-document co-occurrence matrix	ma trận đồng xuất hiện từ–văn bản
word-word co-occurrence matrix	ma trận đồng xuất hiện từ–từ
worker	máy thực thi
workers	máy thực thi
X	
Xavier initialization	khởi tạo Xavier
Z	
zero matrix	ma trận không
zero padding	đệm không
Zero-Order-Hold	giữ bậc không
Zero-Order-Hold (ZOH)	giữ bậc không
zero-padding	đệm số 0
zero-shot / one-shot / few-shot learning	học zero-shot / one-shot / few-shot
zero-shot CoT	coT zero-shot
zero-shot cross-lingual	xuyên ngôn ngữ zero-shot
zero-shot learning	học zero-shot
Zipf's Law	định luật Zipf

Bibliography

[Ackley et al., 1985] David H Ackley, Geoffrey E Hinton, and Terrence J Sejnowski. A learning algorithm for boltzmann machines. *Cognitive science*, 9(1):147–169, 1985.

[Acs and Kornai, 2016] Judit Acs and András Kornai. Evaluating embeddings on dictionary-based similarity. In *Proceedings of the 1st Workshop on Evaluating Vector-Space Representations for NLP*, pages 78–82, 2016.

[Adi et al., 2016] Yossi Adi, Einat Kermany, Yonatan Belinkov, Ofer Lavi, and Yoav Goldberg. Fine-grained analysis of sentence embeddings using auxiliary prediction tasks. In *Proceedings of International Conference on Learning Representations*, 2016.

[Agirre et al., 2009] Eneko Agirre, Enrique Alfonseca, Keith Hall, Jana Kravalová, Marius Pasca, and Aitor Soroa. A study on similarity and relatedness using distributional and wordnet-based approaches. In *Proceedings of Human Language Technologies: The 2009 Annual Conference of the North American Chapter of the Association for Computational Linguistics*, pages 19–27, 2009.

[Agrawal et al., 2023] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023.

[Agrawal et al., 2024] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, 2024.

[Ainslie et al., 2020] Joshua Ainslie, Santiago Ontanon, Chris Alberti, Vaclav Cvicek, Zachary Fisher, Philip Pham, Anirudh Ravula, Sumit Sanghai, Qifan Wang, and Li Yang. Etc: Encoding long and structured inputs in transformers. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 268–284, 2020.

[Ainslie et al., 2023] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.

[Akhbardeh et al., 2021] Farhad Akhbardeh, Arkady Arkhangorodsky, Magdalena Biesialska, Ondřej Bojar, Rajen Chatterjee, Vishrav Chaudhary, Marta R. Costa-jussa, Cristina España-Bonet, Angela Fan, Christian Federmann, Markus Freitag, Yvette Graham, Roman Grundkiewicz, Barry Haddow, Leonie Harter, Kenneth Heafield, Christopher Homan, Matthias Huck, Kwabena Amponsah-Kaakyire, Jungo Kasai, Daniel Khashabi, Kevin Knight, Tom Kocmi, Philipp Koehn, Nicholas Lourie, Christof Monz, Makoto Morishita, Masaaki Nagata, Ajay Nagesh, Toshiaki Nakazawa, Matteo Negri, Santanu Pal, Allahsera Auguste Tapo, Marco Turchi, Valentin Vydrin, and Marcos Zampieri. Findings of

the 2021 conference on machine translation (WMT21). In *Proceedings of the Sixth Conference on Machine Translation*, pages 1–88, 2021.

[Akyürek et al., 2023] Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. What learning algorithm is in-context learning? investigations with linear models. In *Proceedings of The Eleventh International Conference on Learning Representations*, 2023.

[Alabdulmohsin et al., 2022] Ibrahim M Alabdulmohsin, Behnam Neyshabur, and Xiaohua Zhai. Revisiting neural scaling laws in language and vision. *Advances in Neural Information Processing Systems*, 35:22300–22312, 2022.

[Alayrac et al., 2022] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob Menick, Sebastian Borgeaud, Andrew Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karen Simonyan. Flamingo: a visual language model for few-shot learning. *Advances in Neural Information Processing Systems*, 35:23716–23736, 2022.

[Allal et al., 2024] Loubna Ben Allal, Anton Lozhkov, and Daniel van Strien. cosmopedia: how to create large-scale synthetic data for pre-training. https://huggingface.co/blog/cosmopedia, 2024.

[Allauzen et al., 2014] Cyril Allauzen, Bill Byrne, Adrià de Gispert, Gonzalo Iglesias, and Michael Riley. Pushdown automata in statistical machine translation. *Computational Linguistics*, 40(3): 687–723, 2014.

[Allen and Hospedales, 2019] Carl Allen and Timothy Hospedales. Analogies explained: Towards understanding word embeddings. In *International Conference on Machine Learning*, pages 223–231. PMLR, 2019.

[Almazrouei et al., 2023] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, Daniele Mazzotta, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. The falcon series of open language models. *arXiv preprint arXiv:2311.16867*, 2023.

[Alzantot et al., 2018] Moustafa Alzantot, Yash Sharma, Ahmed Elgohary, Bo-Jhang Ho, Mani Srivastava, and Kai-Wei Chang. Generating natural language adversarial examples. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2890–2896, 2018.

[Ammar et al., 2020] Waleed Ammar, George Mulcaire, Yulia Tsvetkov, Guillaume Lample, Chris Dyer, and Noah A Smith. Massively multilingual word embeddings. In *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020.

[Anderson et al., 2017] Peter Anderson, Basura Fernando, Mark Johnson, and Stephen Gould. Guided open vocabulary image captioning with constrained beam search. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 936–945, 2017.

[Andreas et al., 2016] Jacob Andreas, Marcus Rohrbach, Trevor Darrell, and Dan Klein. Neural module networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 39–48, 2016.

[Anthropic, 2024] Anthropic. Claude 3.5 sonnet model card addendum, 2024. URL https://www-cdn.anthropic.com/fed9cc193a14b84131812372d8d5857f8f304c52/

Model_Card_Claude_3_Addendum.pdf. Technical Report.

[Antol et al., 2015] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C Lawrence Zitnick, and Devi Parikh. Vqa: Visual question answering. In *Proceedings of the IEEE international conference on computer vision*, pages 2425–2433, 2015.

[Arjovsky et al., 2016] Martin Arjovsky, Amar Shah, and Yoshua Bengio. Unitary evolution recurrent neural networks. In *International conference on machine learning*, pages 1120–1128, 2016.

[Aronoff and Fudeman, 2011] Mark Aronoff and Kirsten Fudeman. *What is morphology?*, volume 8. John Wiley & Sons, 2011.

[Arora et al., 2017] Sanjeev Arora, Yingyu Liang, and Tengyu Ma. A simple but tough-to-beat baseline for sentence embeddings. In *International conference on learning representations*, 2017.

[Artetxe et al., 2017] Mikel Artetxe, Gorka Labaka, and Eneko Agirre. Learning bilingual word embeddings with (almost) no bilingual data. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 451–462, 2017.

[Aschenbrenner, 2024] Leopold Aschenbrenner. Situational awareness: The decade ahead, 2024. URL https://situational-awareness.ai/.

[Askell et al., 2021] Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Benjamin Mann, Nova DasSarma, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Jackson Kernion, Kamal Ndousse, Catherine Olsson, Dario Amodei, Tom B. Brown, Jack Clark, Sam McCandlish, Chris Olah, and Jared Kaplan. A general language assistant as a laboratory for alignment. *arXiv preprint arXiv:2112.00861*, 2021.

[Åström and Wittenmark, 2013] Karl J Åström and Björn Wittenmark. *Computer-controlled systems: theory and design*. Courier Corporation, 2013.

[Atkinson and Shiffrin, 1968] Richard C Atkinson and Richard M Shiffrin. Human memory: A proposed system and its control processes. In *Psychology of learning and motivation*, volume 2, pages 89–195. Elsevier, 1968.

[Auguste et al., 2017] Jeremy Auguste, Arnaud Rey, and Benoit Favre. Evaluation of word embeddings against cognitive processes: primed reaction times in lexical decision and naming tasks. In *Proceedings of the 2nd workshop on evaluating vector space representations for NLP*, pages 21–26, 2017.

[Ba et al., 2016] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[Bach et al., 2022] Stephen H. Bach, Victor Sanh, Zheng Xin Yong, Albert Webson, Colin Raffel, Nihal V. Nayak, Abheesht Sharma, Taewoon Kim, M. Saiful Bari, Thibault Févry, Zaid Alyafeai, Manan Dey, Andrea Santilli, Zhiqing Sun, Srulik Ben-David, Canwen Xu, Gunjan Chhablani, Han Wang, Jason Alan Fries, Maged Saeed AlShaibani, Shanya Sharma, Urmish Thakker, Khalid Almubarak, Xiangru Tang, Dragomir R. Radev, Mike Tian-Jian Jiang, and Alexander M. Rush. Promptsource: An integrated development environment and repository for natural language prompts. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 93–104, 2022.

[Bachlechner et al., 2021] Thomas Bachlechner, Bodhisattwa Prasad Majumder, Henry Mao, Gary Cottrell, and Julian McAuley. Rezero is all you need: Fast convergence at large depth. In *Proceedings of Uncertainty in Artificial Intelligence*, pages 1352–1361. PMLR, 2021.

[Baevski et al., 2020] Alexei Baevski, Steffen Schneider, and Michael Auli. vq-wav2vec: Self-supervised learning of discrete speech representations. In *Proceedings of ICLR 2020*, 2020.

[Bahdanau et al., 2014] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

[Bahdanau et al., 2016] Dzmitry Bahdanau, Jan Chorowski, Dmitriy Serdyuk, Philemon Brakel, and Yoshua Bengio. End-to-end attention-based large vocabulary speech recognition. In *2016 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pages 4945–4949. IEEE, 2016.

[Bahl and Mercer, 1976] L. R. Bahl and R. L. Mercer. Part of speech assignment by a statistical decision algorithm. In *Proceedings of IEEE International Symposium on Information Theory*, pages 88–89, 1976.

[Bai et al., 2021] Jiangang Bai, Yujing Wang, Yiren Chen, Yaming Yang, Jing Bai, Jing Yu, and Yunhai Tong. Syntax-bert: Improving pre-trained transformers with syntax trees. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 3011–3020, 2021.

[Bakarov, 2018] Amir Bakarov. A survey of word embeddings evaluation methods. *arXiv preprint arXiv:1801.09536*, 2018.

[Banarescu et al., 2013] Laura Banarescu, Claire Bonial, Shu Cai, Madalina Georgescu, Kira Griffitt, Ulf Hermjakob, Kevin Knight, Philipp Koehn, Martha Palmer, and Nathan Schneider. Abstract Meaning Representation for sembanking. In *Proceedings of the 7th Linguistic Annotation Workshop and Interoperability with Discourse*, pages 178–186, 2013.

[Bao et al., 2021] Hangbo Bao, Li Dong, Songhao Piao, and Furu Wei. Beit: Bert pre-training of image transformers. In *Proceedings of International Conference on Learning Representations*, 2021.

[Bapna et al., 2018] Ankur Bapna, Mia Xu Chen, Orhan Firat, Yuan Cao, and Yonghui Wu. Training deeper neural machine translation models with transparent attention. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3028–3033, 2018.

[Barber, 2012] David Barber. *Bayesian Reasoning and Machine Learning*. Cambridge University Press, 2012.

[Barham et al., 2022] Paul Barham, Aakanksha Chowdhery, Jeff Dean, Sanjay Ghemawat, Steven Hand, Dan Hurt, Michael Isard, Hyeontaek Lim, Ruoming Pang, Sudip Roy, Brennan Saeta, Parker Schuh, Ryan Sepassi, Laurent El Shafey, Chandramohan A. Thekkath, and Yonghui Wu. Pathways: Asynchronous distributed dataflow for ml. In *Proceedings of Machine Learning and Systems*, volume 4, pages 430–449, 2022.

[Baroni and Lenci, 2010] Marco Baroni and Alessandro Lenci. Distributional memory: A general framework for corpus-based semantics. *Computational Linguistics*, 36(4):673–721, 2010.

[Baroni et al., 2014] Marco Baroni, Georgiana Dinu, and Germán Kruszewski. Don't count, predict! a systematic comparison of context-counting vs. context-predicting semantic vectors. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 238–247, 2014.

[Barrault et al., 2020] Loïc Barrault, Magdalena Biesialska, Ondřej Bojar, Marta R. Costa-jussà, Christian Federmann, Yvette Graham, Roman Grundkiewicz, Barry Haddow, Matthias Huck, Eric Joanis, Tom Kocmi, Philipp Koehn, Chi-kiu Lo, Nikola Ljubešić, Christof Monz, Makoto Morishita,

Masaaki Nagata, Toshiaki Nakazawa, Santanu Pal, Matt Post, and Marcos Zampieri. Findings of the 2020 conference on machine translation (WMT20). In *Proceedings of the Fifth Conference on Machine Translation*, pages 1–55, 2020.

[Baum and Petrie, 1966] Leonard E Baum and Ted Petrie. Statistical inference for probabilistic functions of finite state markov chains. *The annals of mathematical statistics*, 37(6):1554–1563, 1966.

[Baum et al., 1970] Leonard E Baum, Ted Petrie, George Soules, and Norman Weiss. A maximization technique occurring in the statistical analysis of probabilistic functions of markov chains. *The annals of mathematical statistics*, 41(1):164–171, 1970.

[Belinkov, 2022] Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022.

[Belinkov and Bisk, 2018] Yonatan Belinkov and Yonatan Bisk. Synthetic and natural noise both break neural machine translation. In *International Conference on Learning Representations*, 2018.

[Bello, 2020] Irwan Bello. Lambdanetworks: Modeling long-range interactions without attention. In *Proceedings of International Conference on Learning Representations*, 2020.

[Beltagy et al., 2020] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv:2004.05150*, 2020.

[Bender et al., 2021] Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pages 610–623, 2021.

[Bengio et al., 2015] Emmanuel Bengio, Pierre-Luc Bacon, Joelle Pineau, and Doina Precup. Conditional computation in neural networks for faster models. *arXiv preprint arXiv:1511.06297*, 2015.

[Bengio, 1991] Yoshua Bengio. *Artificial neural networks and their application to sequence recognition*. PhD thesis, McGill University, 1991.

[Bengio et al., 1994] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2):157–166, 1994.

[Bengio et al., 2000] Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. A neural probabilistic language model. *Advances in Neural Information Processing Systems*, 13, 2000.

[Bengio et al., 2003] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003a.

[Bengio et al., 2003] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3(Feb):1137–1155, 2003b.

[Bengio et al., 2006] Yoshua Bengio, Pascal Lamblin, Dan Popovici, and Hugo Larochelle. Greedy layer-wise training of deep networks. *Advances in neural information processing systems*, 19, 2006.

[Bengio et al., 2013] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[Bengio et al., 2024] Yoshua Bengio, Geoffrey Hinton, Andrew Yao, Dawn Song, Pieter Abbeel, Trevor Darrell, Yuval Noah Harari, Ya-Qin Zhang, Lan Xue, Shai Shalev-Shwartz, Gillian K. Hadfield, Jeff Clune, Tegan Maharaj, Frank Hutter, Atilim Gunes Baydin, Sheila A. McIlraith, Qiqi Gao, Ashwin

Acharya, David Krueger, Anca Dragan, Philip Torr, Stuart Russell, Daniel Kahneman, Jan Markus Brauner, and Sören Mindermann. Managing extreme ai risks amid rapid progress. *Science*, 384 (6698):842–845, 2024.

[Bentivogli and Giampiccolo, 2011] Luisa Bentivogli and Danilo Giampiccolo. Pascal recognizing textual entailment challenge (rte-7) at tac 2011. https://tac.nist.gov/2011/RTE/, 2011.

[Berg-Kirkpatrick et al., 2012] Taylor Berg-Kirkpatrick, David Burkett, and Dan Klein. An empirical investigation of statistical significance in NLP. In *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, pages 995–1005, 2012.

[Besta et al., 2024] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17682–17690, 2024.

[Bhattamishra et al., 2020] Satwik Bhattamishra, Kabir Ahuja, and Navin Goyal. On the ability and limitations of transformers to recognize formal languages. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7096–7116, 2020.

[Bhattasali et al., 2020] Shohini Bhattasali, Jonathan Brennan, Wen-Ming Luh, Berta Franzluebbers, and John Hale. The alice datasets: fMRI & EEG observations of natural language comprehension. In *Proceedings of the 12th Language Resources and Evaluation Conference*, pages 120–125, 2020.

[Bickel and Doksum, 2015] Peter J Bickel and Kjell A Doksum. *Mathematical statistics: basic ideas and selected topics, volumes I-II package*. Chapman and Hall/CRC, 2015.

[Biderman et al., 2021] Stella Biderman, Sid Black, Charles Foster, Leo Gao, Eric Hallahan, Horace He, Ben Wang, and Phil Wang. Rotary embeddings: A relative revolution. https://blog.eleuther.ai/rotary-embeddings/, 2021.

[Birch et al., 2018] Alexandra Birch, Andrew Finch, Minh-Thang Luong, Graham Neubig, and Yusuke Oda. Findings of the second workshop on neural machine translation and generation. In *Proceedings of the 2nd Workshop on Neural Machine Translation and Generation*, pages 1–10, 2018.

[Bishop, 1995] Christopher Bishop. Regularization and complexity control in feed-forward networks. In *Proceedings International Conference on Artificial Neural Networks ICANN'95*, pages 141–148, 1995a.

[Bishop, 1995] Christopher M. Bishop. Training with noise is equivalent to tikhonov regularization. *Neural computation*, 7(1):108–116, 1995b.

[Bishop, 2006] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[Blacoe and Lapata, 2012] William Blacoe and Mirella Lapata. A comparison of vector-based representations for semantic composition. In *Proceedings of the 2012 joint conference on empirical methods in natural language processing and computational natural language learning*, pages 546–556, 2012.

[Blei, 2012] David M Blei. Probabilistic topic models. *Communications of the ACM*, 55(4):77–84, 2012.

[Blei et al., 2003] David M Blei, Andrew Y Ng, and Michael I Jordan. Latent dirichlet allocation. *Journal of machine Learning research*, 3(Jan):993–1022, 2003.

[Blum and Mitchell, 1998] Avrim Blum and Tom Mitchell. Combining labeled and unlabeled data with

co-training. In *Proceedings of the eleventh annual conference on Computational learning theory*, pages 92–100, 1998.

[Bojanowski et al., 2017] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *Transactions of the association for computational linguistics*, 5:135–146, 2017.

[Bommasani et al., 2021] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, S. Buch, Dallas Card, Rodrigo Castellon, Niladri S. Chatterji, Annie S. Chen, Kathleen A. Creel, Jared Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren E. Gillespie, Karan Goel, Noah D. Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas F. Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, O. Khattab, Pang Wei Koh, Mark S. Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir P. Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Benjamin Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, J. F. Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Robert Reich, Hongyu Ren, Frieda Rong, Yusuf H. Roohani, Camilo Ruiz, Jack Ryan, Christopher R'e, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishna Parasuram Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei A. Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. On the opportunities and risks of foundation models. *ArXiv*, 2021.

[Bondarenko et al., 2021] Yelysei Bondarenko, Markus Nagel, and Tijmen Blankevoort. Understanding and overcoming the challenges of efficient transformer quantization. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7947–7969, 2021.

[Borji and Itti, 2012] Ali Borji and Laurent Itti. State-of-the-art in visual attention modeling. *IEEE transactions on pattern analysis and machine intelligence*, 35(1):185–207, 2012.

[Boulanger-Lewandowski et al., 2013] Nicolas Boulanger-Lewandowski, Yoshua Bengio, and Pascal Vincent. Audio chord recognition with recurrent neural networks. In *Proceedings of 14th International Society for Music Information Retrieval Conference*, 2013.

[Bourlard and Wellekens, 1990] Herve Bourlard and Christian J Wellekens. Links between markov models and multilayer perceptrons. *IEEE Transactions on pattern analysis and machine intelligence*, 12(12):1167–1178, 1990.

[Bourlard and Morgan, 1993] Herve A. Bourlard and Nelson Morgan. *Connectionist Speech Recognition: A Hybrid Approach*. Kluwer Academic Publishers, USA, 1993.

[Box et al., 2015] George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. *Time series analysis: forecasting and control (4th ed.)*. John Wiley & Sons, 2015.

[Bradley and Terry, 1952] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

[Brandon et al., 2024] William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan Kelly. Reducing transformer key-value cache size with cross-layer attention. *arXiv*

preprint arXiv:2405.12981, 2024.

[Breiman, 1996] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.

[Brill, 1992] Eric Brill. A simple rule-based part of speech tagger. In *Speech and Natural Language: Proceedings of a Workshop Held at Harriman, New York, February 23-26, 1992*, 1992.

[Briski, 2025] Kari Briski. How scaling laws drive smarter, more powerful ai, 2025.

[Brown et al., 2024] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

[Brown et al., 1993] Peter F. Brown, Stephen A. Della Pietra, Vincent J. Della Pietra, and Robert L. Mercer. The mathematics of statistical machine translation: Parameter estimation. *Computational Linguistics*, 19(2):263–311, 1993.

[Brown et al., 2020] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[Bubeck et al., 2023] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott M. Lundberg, Harsha Nori, Hamid Palangi, Marco Túlio Ribeiro, and Yi Zhang. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023.

[Buckman et al., 2016] Jacob Buckman, Miguel Ballesteros, and Chris Dyer. Transition-based dependency parsing with heuristic backtracking. In *Proceedings of the 2016 Conference on empirical methods in natural language processing*, pages 2313–2318, 2016.

[Bulatov et al., 2022] Aydar Bulatov, Yury Kuratov, and Mikhail Burtsev. Recurrent memory transformer. *Advances in Neural Information Processing Systems*, 35:11079–11091, 2022.

[Burchi and Vielzeuf, 2021] Maxime Burchi and Valentin Vielzeuf. Efficient conformer: Progressive downsampling and grouped attention for automatic speech recognition. In *Proceedings of 2021 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, pages 8–15. IEEE, 2021.

[Burges et al., 2005] Chris Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. Learning to rank using gradient descent. In *Proceedings of the 22nd international conference on Machine learning*, pages 89–96, 2005.

[Burnham and Anderson, 2002] Kenneth P. Burnham and David R. Anderson. *Model selection and multimodel inference: a practical information-theoretic approach*. Spinger, 2002.

[Burns et al., 2023] Collin Burns, Pavel Izmailov, Jan Hendrik Kirchner, Bowen Baker, Leo Gao, Leopold Aschenbrenner, Yining Chen, Adrien Ecoffet, Manas Joglekar, Jan Leike, Ilya Sutskever, and Jeff Wu. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. *arXiv preprint arXiv:2312.09390*, 2023a.

[Burns et al., 2023] Collin Burns, Jan Leike, Leopold Aschenbrenner, Jeffrey Wu, Pavel Izmailov, Leo Gao, Bowen Baker, and Jan Hendrik Kirchner. Weak-to-strong generalization, 2023b. URL https://https://openai.com/index/weak-to-strong-generalization.

[Buttcher et al., 2016] Stefan Buttcher, Charles LA Clarke, and Gordon V Cormack. *Information retrieval: Implementing and evaluating search engines*. MIT Press, 2016.

[Caballero et al., 2023] Ethan Caballero, Kshitij Gupta, Irina Rish, and David Krueger. Broken neural scaling laws. In *ICLR 2023 Workshop on Mathematical and Empirical Understanding of Foundation Models*, 2023.

[Campbell, 1997] Joseph P Campbell. Speaker recognition: A tutorial. *Proceedings of the IEEE*, 85(9): 1437–1462, 1997.

[Cao et al., 2007] Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. Learning to rank: from pairwise approach to listwise approach. In *Proceedings of the 24th international conference on Machine learning*, pages 129–136, 2007.

[Caron et al., 2021] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021.

[Casacuberta et al., 2009] Francisco Casacuberta, Jorge Civera, Elsa Cubel, Antonio L Lagarda, Guy Lapalme, Elliott Macklovitch, and Enrique Vidal. Human interaction for high-quality machine translation. *Communications of the ACM*, 52(10):135–138, 2009.

[Cer et al., 2018] Daniel Cer, Yinfei Yang, Sheng yi Kong, Nan Hua, Nicole Limtiaco, Rhomni St. John, Noah Constant, Mario Guajardo-Cespedes, Steve Yuan, Chris Tar, Yun-Hsuan Sung, Brian Strope, and Ray Kurzweil. Universal sentence encoder. *arXiv preprint arXiv:1803.11175*, 2018.

[Chan et al., 2016] William Chan, Navdeep Jaitly, Quoc Le, and Oriol Vinyals. Listen, attend and spell: A neural network for large vocabulary conversational speech recognition. In *2016 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pages 4960–4964. IEEE, 2016.

[Chang et al., 2024] Kaiyan Chang, Songcheng Xu, Chenglong Wang, Yingfeng Luo, Tong Xiao, and Jingbo Zhu. Efficient prompting methods for large language models: A survey. *arXiv preprint arXiv:2404.01077*, 2024a.

[Chang, 1967] Wing-Tsit Chang. *Reflections on things at hand*. Columbia University Press, 1967.

[Chang and Collins, 2011] Yin-Wen Chang and Michael Collins. Exact decoding of phrase-based translation models through lagrangian relaxation. In *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing*, pages 26–37, 2011.

[Chang et al., 2024] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, Wei Ye, Yue Zhang, Yi Chang, Philip S. Yu, Qiang Yang, and Xing Xie. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology*, 15(3), 2024b.

[Charniak, 1997] Eugene Charniak. Statistical parsing with a context-free grammar and word statistics. *AAAI/IAAI*, 2005(598-603):18, 1997.

[Chatfield, 2003] Chris Chatfield. *The analysis of time series: an introduction*. Chapman and hall/CRC, 2003.

[Chaudhari et al., 2021] Sneha Chaudhari, Varun Mithal, Gungor Polatkan, and Rohan Ramanath. An attentive survey of attention models. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 12(5):1–32, 2021.

[Chen et al., 2023] Banghao Chen, Zhaofeng Zhang, Nicolas Langrené, and Shengxin Zhu. Unleashing the potential of prompt engineering in large language models: a comprehensive review. *arXiv*

preprint arXiv:2310.14735, 2023a.

[Chen et al., 2018] Kehai Chen, Rui Wang, Masao Utiyama, Eiichiro Sumita, and Tiejun Zhao. Syntax-directed attention for neural machine translation. In *Proceedings of the AAAI conference on artificial intelligence*, 2018a.

[Chen et al., 2023] Lichang Chen, Shiyang Li, Jun Yan, Hai Wang, Kalpa Gunaratna, Vikas Yadav, Zheng Tang, Vijay Srinivasan, Tianyi Zhou, Heng Huang, and Hongxia Jin. Alpagasus: Training a better alpaca with fewer data. *arXiv preprint arXiv:2307.08701*, 2023b.

[Chen et al., 2024] Lichang Chen, Shiyang Li, Jun Yan, Hai Wang, Kalpa Gunaratna, Vikas Yadav, Zheng Tang, Vijay Srinivasan, Tianyi Zhou, Heng Huang, and Hongxia Jin. Alpagasus: Training a better alpaca with fewer data. In *The Twelfth International Conference on Learning Representations*, 2024a.

[Chen et al., 2020] Mark Chen, Alec Radford, Rewon Child, Jeffrey Wu, Heewoo Jun, David Luan, and Ilya Sutskever. Generative pretraining from pixels. In *International conference on machine learning*, pages 1691–1703. PMLR, 2020a.

[Chen et al., 2021] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[Chen et al., 2018] Mia Xu Chen, Orhan Firat, Ankur Bapna, Melvin Johnson, Wolfgang Macherey, George Foster, Llion Jones, Mike Schuster, Noam Shazeer, Niki Parmar, Ashish Vaswani, Jakob Uszkoreit, Lukasz Kaiser, Zhifeng Chen, Yonghui Wu, and Macduff Hughes. The best of both worlds: Combining recent advances in neural machine translation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 76–86, 2018b.

[Chen et al., 2026] Qiguang Chen, Libo Qin, Jinhao Liu, Dengyun Peng, Jiannan Guan, Peng Wang, Mengkang Hu, Yuhang Zhou, Te Gao, and Wanxiang Che. Towards reasoning era: A survey of long chain-of-thought for reasoning large language models. *Science China Information Sciences*, 69: 161101, 2026.

[Chen et al., 2018] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in neural information processing systems*, 31, 2018c.

[Chen et al., 2022] Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, Shujie Liu, Zhuo Chen, Jinyu Li, Naoyuki Kanda, Takuya Yoshioka, Xiong Xiao, Long Zhou, Shuo Ren, Yanmin Qian, Yao Qian, Jian Wu, Michael Zeng, and Furu Wei. Wavlm: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6):1505–1518, 2022.

[Chen et al., 2023] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint*

arXiv:2306.15595, 2023c.

[Chen and Goodman, 1999] Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech and Language*, 13:359–394, 1999.

[Chen et al., 2020] Tianlong Chen, Jonathan Frankle, Shiyu Chang, Sijia Liu, Yang Zhang, Zhangyang Wang, and Michael Carbin. The lottery ticket hypothesis for pre-trained bert networks. *Advances in neural information processing systems*, 33:15834–15846, 2020b.

[Chen et al., 2015] Tianqi Chen, Ian Goodfellow, and Jonathon Shlens. Net2net: Accelerating learning via knowledge transfer. *arXiv preprint arXiv:1511.05641*, 2015.

[Chen and He, 2021] Xinlei Chen and Kaiming He. Exploring simple siamese representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15750–15758, 2021.

[Chen et al., 2025] Yanda Chen, Joe Benton, Ansh Radhakrishnan, Jonathan Uesato, Carson Denison, John Schulman, Arushi Somani, Peter Hase, Misha Wagner, Fabien Roger, Vlad Mikulik, Samuel R. Bowman, Jan Leike, Jared Kaplan, and Ethan Perez. Reasoning models don't always say what they think. *arXiv preprint arXiv:2505.05410*, 2025.

[Chen et al., 2020] Yen-Chun Chen, Linjie Li, Licheng Yu, Ahmed El Kholy, Faisal Ahmed, Zhe Gan, Yu Cheng, and Jingjing Liu. Uniter: Universal image-text representation learning. In *Proceedings of European conference on computer vision*, pages 104–120, 2020c.

[Chen et al., 2024] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. *arXiv preprint arXiv:2401.01335*, 2024b.

[Chevalier et al., 2023] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3829–3846, 2023.

[Chi et al., 2022] Ta-Chung Chi, Ting-Han Fan, Peter J Ramadge, and Alexander Rudnicky. Kerple: Kernelized relative positional embedding for length extrapolation. *Advances in Neural Information Processing Systems*, 35:8386–8399, 2022.

[Chi et al., 2023] Ta-Chung Chi, Ting-Han Fan, Alexander Rudnicky, and Peter Ramadge. Dissecting transformer length extrapolation via the lens of receptive field analysis. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13522–13537, 2023.

[Chiang, 2005] David Chiang. A hierarchical phrase-based model for statistical machine translation. In *Proceedings of the 43rd annual meeting of the association for computational linguistics (acl'05)*, pages 263–270, 2005.

[Chiang, 2007] David Chiang. Hierarchical phrase-based translation. *computational linguistics*, 33(2): 201–228, 2007.

[Chiang and Cholak, 2022] David Chiang and Peter Cholak. Overcoming a theoretical limitation of self-attention. *arXiv preprint arXiv:2202.12172*, 2022.

[Chiang et al., 2023] David Chiang, Peter Cholak, and Anand Pillay. Tighter bounds on the expressivity of transformer encoders. *arXiv preprint arXiv:2301.10743*, 2023a.

[Chiang et al., 2023] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing.

Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023b. URL https://lmsys.org/blog/2023-03-30-vicuna/.

[Child et al., 2019] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[Chiu and Raffel, 2018] Chung-Cheng Chiu and Colin Raffel. Monotonic chunkwise attention. In *Proceedings of the 8th International Conference on Learning Representations ICLR*, 2018.

[Cho et al., 2021] Jaemin Cho, Jie Lei, Hao Tan, and Mohit Bansal. Unifying vision-and-language tasks via text generation. In *International Conference on Machine Learning*, pages 1931–1942. PMLR, 2021.

[Cho and Esipova, 2016] Kyunghyun Cho and Masha Esipova. Can neural machine translation do simultaneous translation? *arXiv preprint arXiv:1606.02012*, 2016.

[Cho et al., 2014] Kyunghyun Cho, Bart van Merriënboer, Çağlar Guùlçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.

[Choe and Charniak, 2016] Do Kook Choe and Eugene Charniak. Parsing as language modeling. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2331–2336, 2016.

[Chollet, 2021] François Chollet. *Deep Learning with Python (2nd ed.)*. Manning Publications, 2021.

[Chomsky, 1965] Noam Chomsky. *Aspects of the Theory of Syntax*, volume 11. MIT press Cambridge, MA, 1965.

[Choromanski et al., 2020] Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, David Benjamin Belanger, Lucy J. Colwell, and Adrian Weller. Rethinking attention with performers. In *Proceedings of International Conference on Learning Representations*, 2020.

[Chorowski and Jaitly, 2017] Jan Chorowski and Navdeep Jaitly. Towards better decoding and language model integration in sequence to sequence models. *Proc. Interspeech 2017*, pages 523–527, 2017.

[Chorowski et al., 2019] Jan Chorowski, Ron J Weiss, Samy Bengio, and Aäron Van Den Oord. Unsupervised speech representation learning using wavenet autoencoders. *IEEE/ACM transactions on audio, speech, and language processing*, 27(12):2041–2053, 2019.

[Chorowski et al., 2015] Jan K Chorowski, Dzmitry Bahdanau, Dmitriy Serdyuk, Kyunghyun Cho, and Yoshua Bengio. Attention-based models for speech recognition. *Advances in neural information processing systems*, 28, 2015.

[Chowdhery et al., 2022] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon

Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.

[Christiano et al., 2017] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.

[Chu et al., 2023] Zheng Chu, Jingchang Chen, Qianglong Chen, Weijiang Yu, Tao He, Haotian Wang, Weihua Peng, Ming Liu, Bing Qin, and Ting Liu. A survey of chain of thought reasoning: Advances, frontiers and future. *arXiv preprint arXiv:2309.15402*, 2023.

[Chuang et al., 2020] Yung-Sung Chuang, Chi-Liang Liu, Hung-yi Lee, and Lin-shan Lee. Speechbert: An audio-and-text jointly learned language model for end-to-end spoken question answering. In *Proceedings of Interspeech 2020*, pages 4168–4172, 2020.

[Chung et al., 2022] Hyung Won Chung, Le Hou, S. Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Wei Yu, Vincent Zhao, Yanping Huang, Andrew M. Dai, Hongkun Yu, Slav Petrov, Ed Huai hsin Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022.

[Chung et al., 2014] Junyoung Chung, Caglar Gulcehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. In *Proceedings of NIPS 2014 Workshop on Deep Learning, December 2014*, 2014.

[Church, 2011] Kenneth Church. A pendulum swung too far. *Linguistic Issues in Language Technology*, 6, 2011.

[Church and Hanks, 1990] Kenneth Ward Church and Patrick Hanks. Word association norms, mutual information, and lexicography. *Computational Linguistics*, 16(1):22–29, 1990. URL https://aclanthology.org/J90-1003.

[Clark et al., 2019] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D Manning. What does bert look at? an analysis of bert's attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 276–286, 2019a.

[Clark et al., 2019] Kevin Clark, Minh-Thang Luong, Quoc V Le, and Christopher D Manning. Electra: Pre-training text encoders as discriminators rather than generators. In *Proceedings of International Conference on Learning Representations*, 2019b.

[Clark et al., 2008] Stephen Clark, Bob Coecke, and Mehrnoosh Sadrzadeh. A compositional distributional model of meaning. In *Proceedings of the Second Quantum Interaction Symposium (QI-2008)*, pages 133–140. Oxford, 2008.

[Cline and Dhillon, 2014] Alan Kaylor Cline and Inderjit S. Dhillon. Computation of the singular value decomposition. In Leslie Hogben, editor, *Handbook of Linear Algebra (2dn ed.)*. CRC Press, 2014.

[Cobbe et al., 2021] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[Cohn et al., 2016] Trevor Cohn, Cong Duy Vu Hoang, Ekaterina Vymolova, Kaisheng Yao, Chris Dyer, and Gholamreza Haffari. Incorporating structural alignment biases into an attentional neural translation model. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 876–885, 2016.

[Collobert and Weston, 2008] Ronan Collobert and Jason Weston. A unified architecture for natural language processing: deep neural networks with multitask learning. In *Proceedings of the 25th international conference on Machine learning (ICML08)*, pages 160–167, 2008.

[Conneau et al., 2017] Alexis Conneau, Douwe Kiela, Holger Schwenk, Loïc Barrault, and Antoine Bordes. Supervised learning of universal sentence representations from natural language inference data. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 670–680, 2017a.

[Conneau et al., 2017] Alexis Conneau, Douwe Kiela, Holger Schwenk, Loïc Barrault, and Antoine Bordes. Supervised learning of universal sentence representations from natural language inference data. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 670–680, 2017b.

[Conneau et al., 2017] Alexis Conneau, Holger Schwenk, Loïc Barrault, and Yann Lecun. Very deep convolutional networks for text classification. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 1, Long Papers*, pages 1107–1116, 2017c.

[Conneau et al., 2018] Alexis Conneau, Germán Kruszewski, Guillaume Lample, Loïc Barrault, and Marco Baroni. What you can cram into a single vector: Probing sentence embeddings for linguistic properties. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2126–2136, 2018.

[Conneau et al., 2020] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Édouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451, 2020.

[Cortes and Vapnik, 1995] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Mind*, Machine Learning:273–297, 1995.

[Coste et al., 2024] Thomas Coste, Usman Anwar, Robert Kirk, and David Krueger. Reward model ensembles help mitigate overoptimization. In *The Twelfth International Conference on Learning Representations*, 2024.

[Cotterell and Schütze, 2015] Ryan Cotterell and Hinrich Schütze. Morphological word-embeddings. In *Proceedings of the 2015 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1287–1292, 2015.

[Cui et al., 2024] Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, Zhiyuan Liu, and Maosong Sun. ULTRAFEEDBACK: Boosting language models with scaled AI feedback. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 9722–9744, 2024.

[Currey and Heafield, 2018] Anna Currey and Kenneth Heafield. Multi-source syntactic neural machine translation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2961–2966, 2018.

[Cybenko, 1989] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathe-*

matics of control, signals and systems, 2(4):303–314, 1989.

[Dai et al., 2023] Damai Dai, Yutao Sun, Li Dong, Yaru Hao, Shuming Ma, Zhifang Sui, and Furu Wei. Why can gpt learn in-context? language models secretly perform gradient descent as meta-optimizers. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 4005–4019, 2023.

[Dai et al., 2019] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, 2019.

[Dao et al., 2022] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.

[Dao et al., 2023] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. Flash-decoding for long-context inference. https://pytorch.org/blog/flash-decoding/, 2023. Retrieved 2023-10-23.

[Davis and Mermelstein, 1980] Steven Davis and Paul Mermelstein. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE transactions on acoustics, speech, and signal processing*, 28(4):357–366, 1980.

[Dayan et al., 1995] Peter Dayan, Geoffrey E Hinton, Radford M Neal, and Richard S Zemel. The helmholtz machine. *Neural computation*, 7(5):889–904, 1995.

[de Gispert et al., 2010] Adrià de Gispert, Gonzalo Iglesias, Graeme Blackwood, Eduardo R. Banga, and William Byrne. Hierarchical phrase-based translation with weighted finite-state transducers and shallow-n grammars. *Computational linguistics*, 36(3):505–533, 2010.

[DeepSeek, 2025] DeepSeek. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[Deerwester et al., 1990] Scott Deerwester, Susan T Dumais, George W Furnas, Thomas K Landauer, and Richard Harshman. Indexing by latent semantic analysis. *Journal of the American society for information science*, 41(6):391–407, 1990.

[Dehghani et al., 2018] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. *arXiv preprint arXiv:1807.03819*, 2018.

[Del Corro et al., 2023] Luciano Del Corro, Allie Del Giorno, Sahaj Agarwal, Bin Yu, Ahmed Awadallah, and Subhabrata Mukherjee. Skipdecode: Autoregressive skip decoding with batching and caching for efficient llm inference. *arXiv preprint arXiv:2307.02628*, 2023.

[Deletang et al., 2024] Gregoire Deletang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, Marcus Hutter, and Joel Veness. Language modeling is compression. In *The Twelfth International Conference on Learning Representations*, 2024.

[Dempster et al., 1977] Arthur P Dempster, Nan M Laird, and Donald B Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1):1–22, 1977.

[Deng et al., 2022] Mingkai Deng, Jianyu Wang, Cheng-Ping Hsieh, Yihan Wang, Han Guo, Tianmin Shu, Meng Song, Eric Xing, and Zhiting Hu. Rlprompt: Optimizing discrete text prompts with reinforcement learning. In *Proceedings of the 2022 Conference on Empirical Methods in Natural*

Language Processing, pages 3369–3391, 2022.

[Deng et al., 2023] Yuntian Deng, Kiran Prasad, Roland Fernandez, Paul Smolensky, Vishrav Chaudhary, and Stuart Shieber. Implicit chain of thought reasoning via knowledge distillation. *arXiv preprint arXiv:2311.01460*, 2023.

[Deoras et al., 2011] Anoop Deoras, Tomáš Mikolov, and Kenneth Church. A fast re-scoring strategy to capture long-distance dependencies. In *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing*, pages 1116–1127, 2011.

[Devereux et al., 2010] Barry Devereux, Colin Kelly, and Anna Korhonen. Using fmri activation to conceptual stimuli to evaluate methods for extracting conceptual representations from corpora. In *Proceedings of the NAACL HLT 2010 First Workshop on Computational Neurolinguistics*, pages 70–78, 2010.

[Devlin et al., 2019] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019.

[Di Gangi et al., 2019] Mattia Antonino Di Gangi, Matteo Negri, Roldano Cattoni, Roberto Dessi, and Marco Turchi. Enhancing transformer for end-to-end speech-to-text translation. In *Proceedings of Machine Translation Summit XVII: Research Track*, pages 21–31, 2019.

[Ding et al., 2021] Ming Ding, Zhuoyi Yang, Wenyi Hong, Wendi Zheng, Chang Zhou, Da Yin, Junyang Lin, Xu Zou, Zhou Shao, Hongxia Yang, and Jie Tang. Cogview: Mastering text-to-image generation via transformers. *Advances in Neural Information Processing Systems*, 34:19822–19835, 2021.

[Ding et al., 2024] Yiran Ding, Li Lyna Zhang, Chengruidong Zhang, Yuanyuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. Longrope: Extending llm context window beyond 2 million tokens. *arXiv preprint arXiv:2402.13753*, 2024.

[Doersch, 2016] Carl Doersch. Tutorial on variational autoencoders. *stat*, 1050:13, 2016.

[Dolan and Brockett, 2005] Bill Dolan and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. In *Proceedings of Third International Workshop on Paraphrasing (IWP2005)*, 2005.

[Dong et al., 2019] Li Dong, Nan Yang, Wenhui Wang, Furu Wei, Xiaodong Liu, Yu Wang, Jianfeng Gao, Ming Zhou, and Hsiao-Wuen Hon. Unified language model pre-training for natural language understanding and generation. *Advances in neural information processing systems*, 32, 2019.

[Dong et al., 2022] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, and Zhifang Sui. A survey on in-context learning. *arXiv preprint arXiv:2301.00234*, 2022.

[Dong et al., 2021] Yihe Dong, Jean-Baptiste Cordonnier, and Andreas Loukas. Attention is not all you need: Pure attention loses rank doubly exponentially with depth. In *International Conference on Machine Learning*, pages 2793–2803. PMLR, 2021.

[Dosovitskiy et al., 2021] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *Proceedings of ICLR 2021*, 2021.

[Downey, 2021] Allen B. Downey. *Think Bayes: Bayesian Statistics in Python (2nd ed.)*. O'Reilly

Media, 2021.

[Dror et al., 2020] Rotem Dror, Lotem Peled-Cohen, and Segev Shlomov. *Neural Network Methods for Natural Language Processing*. Morgan & Claypool Publishers, 2020.

[Drozdov et al., 2022] Andrew Drozdov, Nathanael Schärli, Ekin Akyürek, Nathan Scales, Xinying Song, Xinyun Chen, Olivier Bousquet, and Denny Zhou. Compositional semantic parsing with large language models. In *Proceedings of The Eleventh International Conference on Learning Representations*, 2022.

[Dua et al., 2022] Dheeru Dua, Shivanshu Gupta, Sameer Singh, and Matt Gardner. Successive prompting for decomposing complex questions. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 1251–1265, 2022.

[Dubey et al., 2024] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[Dubois et al., 2024] Yann Dubois, Chen Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy S Liang, and Tatsunori B Hashimoto. Alpacafarm: A simulation framework for methods that learn from human feedback. *Advances in Neural Information Processing Systems*, 36, 2024.

[Duchi et al., 2011] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(7), 2011.

[Dufter et al., 2022] Philipp Dufter, Martin Schmitt, and Hinrich Schütze. Position information in transformers: An overview. *Computational Linguistics*, 48(3):733–763, 2022.

[Dyer et al., 2013] Chris Dyer, Victor Chahuneau, and Noah A Smith. A simple, fast, and effective reparameterization of ibm model 2. In *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 644–648, 2013.

[Ebrahimi et al., 2018] Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou. HotFlip: White-box adversarial examples for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 31–36, Melbourne, Australia, 2018.

[Edunov et al., 2018] Sergey Edunov, Myle Ott, Michael Auli, and David Grangier. Understanding back-translation at scale. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 489–500, 2018.

[Ee, 2017] Weinan Ee. A proposal on machine learning via dynamical systems. *Communications in Mathematics and Statistics*, 5:1–11, 02 2017.

[Eikema and Aziz, 2020] Bryan Eikema and Wilker Aziz. Is map decoding all you need? the inadequacy of the mode in neural machine translation. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 4506–4520, 2020.

[Eisenstein et al., 2023] Jacob Eisenstein, Chirag Nagpal, Alekh Agarwal, Ahmad Beirami, Alex D'Amour, DJ Dvijotham, Adam Fisch, Katherine Heller, Stephen Pfohl, Deepak Ramachandran, and Peter Shaw. Helping or herding? reward model ensembles mitigate but do not eliminate reward hacking. *arXiv preprint arXiv:2312.09244*, 2023.

[Elbayad et al., 2020] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive

transformer. In *Proceedings of International Conference on Learning Representations*, 2020.

[Elman, 1990] Jeffrey L Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990.

[Elsken et al., 2019] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21, 2019a.

[Elsken et al., 2019] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *The Journal of Machine Learning Research*, 20(1):1997–2017, 2019b.

[Erhan et al., 2010] Dumitru Erhan, Aaron Courville, Yoshua Bengio, and Pascal Vincent. Why does unsupervised pre-training help deep learning? In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 201–208, 2010.

[Fan et al., 2018] Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 889–898, 2018.

[Fan et al., 2019] Angela Fan, Edouard Grave, and Armand Joulin. Reducing transformer depth on demand with structured dropout. In *Proceedings of International Conference on Learning Representations*, 2019.

[Fan et al., 2021] Haoqi Fan, Bo Xiong, Karttikeya Mangalam, Yanghao Li, Zhicheng Yan, Jitendra Malik, and Christoph Feichtenhofer. Multiscale vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6824–6835, 2021.

[Fan et al., 2020] Yang Fan, Shufang Xie, Yingce Xia, Lijun Wu, Tao Qin, Xiang-Yang Li, and Tie-Yan Liu. Multi-branch attentive transformer. *arXiv preprint arXiv:2006.10270*, 2020.

[Faruqui et al., 2016] Manaal Faruqui, Yulia Tsvetkov, Pushpendre Rastogi, and Chris Dyer. Problems with evaluation of word embeddings using word similarity tasks. In *Proceedings of the 1st Workshop on Evaluating Vector-Space Representations for NLP*, pages 30–35, 2016.

[Fedus et al., 2022] William Fedus, Jeff Dean, and Barret Zoph. A review of sparse expert models in deep learning. *arXiv preprint arXiv:2209.01667*, 2022a.

[Fedus et al., 2022] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research*, 23(1):5232–5270, 2022b.

[Fellbaum, 2005] Christiane Fellbaum. Wordnet and wordnets. In Keith Brown, editor, *Encyclopedia of Language and Linguistics (2nd ed.)*. Elsevier, 2005.

[Feng et al., 2016] Shi Feng, Shujie Liu, Nan Yang, Mu Li, Ming Zhou, and Kenny Zhu. Improving attention modeling with implicit distortion and fertility for machine translation. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, pages 3082–3092, 2016.

[Feng et al., 2021] Steven Y. Feng, Varun Gangal, Jason Wei, Sarath Chandar, Soroush Vosoughi, Teruko Mitamura, and Eduard Hovy. A survey of data augmentation approaches for NLP. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 968–988, 2021.

[Fernandes et al., 2023] Patrick Fernandes, Aman Madaan, Emmy Liu, António Farinhas, Pedro Henrique Martins, Amanda Bertsch, José G. C. de Souza, Shuyan Zhou, Tongshuang Wu, Graham Neubig, and André F. T. Martins. Bridging the gap: A survey on integrating (human) feedback for natural language generation. *Transactions of the Association for Computational Linguistics*, 11: 1643–1668, 2023.

[Firth, 1957] John R Firth. A synopsis of linguistic theory, 1930-1955. *Studies in linguistic analysis*, 1957.

[Forney, 1972] GDJR Forney. Maximum-likelihood sequence estimation of digital sequences in the presence of intersymbol interference. *IEEE Transactions on Information theory*, 18(3):363–378, 1972.

[Franklin and Graesser, 1996] Stan Franklin and Art Graesser. Is it an agent, or just a program?: A taxonomy for autonomous agents. In *International workshop on agent theories, architectures, and languages*, pages 21–35. Springer, 1996.

[Fraser-Taliente et al., 2026] Kit Fraser-Taliente, Subhash Kantamneni, Euan Ong, Dan Mossing, Christina Lu, Paul C. Bogdan, Emmanuel Ameisen, James Chen, Dzmitry Kishylau, Adam Pearce, Julius Tarng, Alex Wu, Jeff Wu, Yang Zhang, Daniel M. Ziegler, Evan Hubinger, Joshua Batson, Jack Lindsey, Samuel Zimmerman, and Samuel Marks. Natural language autoencoders produce unsupervised explanations of llm activations. Transformer Circuits Thread, May 2026. URL https://transformer-circuits.pub/2026/nla/index.html.

[Freedman et al., 2007] David Freedman, Robert Pisani, and Roger Purves. *Statistics (4th ed.)*. W. W. Norton & Company, 2007.

[Freedman, 2009] David A. Freedman. *Statistical Models: Theory and Practice (2nd ed.)*. Cambridge University Press, 2009.

[Freitag and Al-Onaizan, 2017] Markus Freitag and Yaser Al-Onaizan. Beam search strategies for neural machine translation. In *Proceedings of the First Workshop on Neural Machine Translation*, pages 56–60, 2017.

[Freitag et al., 2022] Markus Freitag, David Grangier, Qijun Tan, and Bowen Liang. High quality rather than high model probability: Minimum bayes risk decoding with neural metrics. *Transactions of the Association for Computational Linguistics*, 10:811–825, 2022.

[Frensch and Funke, 2014] Peter A Frensch and Joachim Funke. *Complex problem solving: The European perspective*. Psychology Press, 2014.

[Friedl, 2006] Jeffrey Friedl. *Mastering Regular Expressions (3rd ed.)*. O'Reilly Media, 2006.

[Fuller, 2009] Wayne A Fuller. *Introduction to statistical time series*. John Wiley & Sons, 2009.

[Gage, 1994] Philip Gage. A new algorithm for data compression. *C Users Journal*, 12(2):23–38, 1994.

[Gale et al., 2019] Trevor Gale, Erich Elsen, and Sara Hooker. The state of sparsity in deep neural networks. *arXiv preprint arXiv:1902.09574*, 2019.

[Ganguli et al., 2023] Deep Ganguli, Amanda Askell, Nicholas Schiefer, Thomas I. Liao, Kamile Lukosiute, Anna Chen, Anna Goldie, Azalia Mirhoseini, Catherine Olsson, Danny Hernandez, Dawn Drain, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jackson Kernion, Jamie Kerr, Jared Mueller, Joshua Landau, Kamal Ndousse, Karina Nguyen, Liane Lovitt, Michael Sellitto, Nelson Elhage, Noemí Mercado, Nova DasSarma, Oliver Rausch, Robert Lasenby, Robin Larson, Sam Ringer, Sandipan Kundu, Saurav Kadavath, Scott Johnston, Shauna Kravec, Sheer El Showk, Tamera Lanham, Timothy Telleen-Lawton, Tom Henighan, Tristan Hume, Yuntao Bai, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, Christopher Olah, Jack Clark, Samuel R. Bowman, and Jared Kaplan. The capacity for moral self-correction in large language models. *arXiv preprint arXiv:2302.07459*, 2023.

[Gao et al., 2023] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning*, pages 10835–10866. PMLR, 2023a.

[Gao et al., 2023] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *International Conference on Machine Learning*, pages 10764–10799. PMLR, 2023b.

[Gao et al., 2023] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023c.

[Garg et al., 2019] Sarthak Garg, Stephan Peitz, Udhyakumar Nallasamy, and Matthias Paulik. Jointly learning to align and translate with transformer models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4453–4462, 2019.

[Garg et al., 2022] Shivam Garg, Dimitris Tsipras, Percy S Liang, and Gregory Valiant. What can transformers learn in-context? a case study of simple function classes. *Advances in Neural Information Processing Systems*, 35:30583–30598, 2022.

[Ge et al., 2024] Yuan Ge, Yilun Liu, Chi Hu, Weibin Meng, Shimin Tao, Xiaofeng Zhao, Hongxia Ma, Li Zhang, Boxing Chen, Hao Yang, Bei Li, Tong Xiao, and Jingbo Zhu. Clustering and ranking: Diversity-preserved instruction selection through expert-aligned quality estimation. *arXiv preprint arXiv:2402.18191*, 2024.

[Gehring et al., 2017] Jonas Gehring, Michael Auli, David Grangier, and Yann Dauphin. A convolutional encoder model for neural machine translation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 123–135, 2017a.

[Gehring et al., 2017] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N Dauphin. Convolutional sequence to sequence learning. In *International conference on machine learning*, pages 1243–1252. PMLR, 2017b.

[Gelman et al., 2020] Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. *Bayesian Data Analysis (2nd ed.)*. Chapman and Hall/CRC, 2020.

[Gemma Team, 2024] Google DeepMind Gemma Team. Gemma: Open Models Based on Gemini Research and Technology, 2024.

[Germann et al., 2004] Ulrich Germann, Michael Jahr, Kevin Knight, Daniel Marcu, and Kenji Yamada. Fast and optimal decoding for machine translation. *Artificial Intelligence*, 154(1-2):127–143, 2004.

[Géron, 2019] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems (2nd ed.)*. O'Reilly Media, 2019.

[Gers et al., 2000] Felix A Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with lstm. *Neural computation*, 12(10):2451–2471, 2000.

[Ghazvininejad et al., 2019] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. Mask-predict: Parallel decoding of conditional masked language models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 6112–6121, 2019.

[Gholami et al., 2022] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference. In

Low-Power Computer Vision, pages 291–326. Chapman and Hall/CRC, 2022.

[Gildea and Jurafsky, 2002] Daniel Gildea and Daniel Jurafsky. Automatic labeling of semantic roles. *Computational Linguistics*, 28(3):245–288, 2002.

[Gladkova et al., 2016] Anna Gladkova, Aleksandr Drozd, and Satoshi Matsuoka. Analogy-based detection of morphological and semantic relations with word embeddings: what works and what doesn't. In *Proceedings of the NAACL Student Research Workshop*, pages 8–15, 2016.

[Glorot and Bengio, 2010] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.

[Goel and Byrne, 2000] Vaibhava Goel and William J Byrne. Minimum bayes-risk automatic speech recognition. *Computer Speech & Language*, 14(2):115–135, 2000.

[Gomez et al., 2017] Aidan N Gomez, Mengye Ren, Raquel Urtasun, and Roger B Grosse. The reversible residual network: Backpropagation without storing activations. *Advances in neural information processing systems*, 30, 2017.

[Goodfellow et al., 2015] Ian Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations*, 2015.

[Goodfellow et al., 2016] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[Goodhart, 1984] Charles AE Goodhart. *Problems of monetary management: the UK experience*. Springer, 1984.

[Goodman, 1996] Joshua Goodman. Parsing algorithms and metrics. In *34th Annual Meeting of the Association for Computational Linguistics*, pages 177–183, 1996a.

[Goodman, 1996] Joshua Goodman. Parsing algorithms and metrics. In *34th Annual Meeting of the Association for Computational Linguistics*, pages 177–183, 1996b.

[Google, 2024] Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.

[Gordon et al., 2021] Mitchell A Gordon, Kevin Duh, and Jared Kaplan. Data and parameter scaling laws for neural machine translation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5915–5922, 2021.

[Gou et al., 2021] Jianping Gou, Baosheng Yu, Stephen J Maybank, and Dacheng Tao. Knowledge distillation: A survey. *International Journal of Computer Vision*, 129:1789–1819, 2021.

[Gou et al., 2024] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Yang, Nan Duan, Weizhu Chen, et al. Critic: Large language models can self-correct with tool-interactive critiquing. In *The Twelfth International Conference on Learning Representations*, 2024.

[Goyal et al., 2024] Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. In *The Twelfth International Conference on Learning Representations*, 2024.

[Graves and Jaitly, 2014] Alex Graves and Navdeep Jaitly. Towards end-to-end speech recognition with recurrent neural networks. In *Proceedings of International conference on machine learning*, pages 1764–1772, 2014.

[Graves et al., 2006] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd international conference on Machine learning*, pages 369–376, 2006.

[Graves et al., 2013] Alex Graves, Navdeep Jaitly, and Abdel-rahman Mohamed. Hybrid speech recognition with deep bidirectional lstm. In *IEEE workshop on automatic speech recognition and understanding*, pages 273–278. IEEE, 2013a.

[Graves et al., 2013] Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. Speech recognition with deep recurrent neural networks. In *2013 IEEE international conference on acoustics, speech and signal processing*, pages 6645–6649. IEEE, 2013b.

[Graves et al., 2014] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.

[Graves et al., 2016] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwinska, Sergio Gomez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adriá Puigdoménech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538 (7626):471–476, 2016.

[Gray, 1998] Robert M. Gray. Quantization. *IEEE transactions on information theory*, 44(6):2325–2383, 1998.

[Grissom II et al., 2014] Alvin Grissom II, He He, Jordan Boyd-Graber, John Morgan, and Hal Daumé III. Don't until the final verb wait: Reinforcement learning for simultaneous machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1342–1352, 2014.

[Grover and Leskovec, 2016] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 855–864, 2016.

[Gu and Dao, 2023] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[Gu et al., 2021] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *Proceedings of International Conference on Learning Representations*, 2021.

[Gu et al., 2022] Albert Gu, Karan Goel, Ankit Gupta, and Christopher Ré. On the parameterization and initialization of diagonal state space models. *Advances in Neural Information Processing Systems*, 35:35971–35983, 2022a.

[Gu et al., 2022] Albert Gu, Karan Goel, Khaled Saab, and Chris Ré. Structured state spaces: Combining continuous-time, recurrent, and convolutional models. https://hazyresearch.stanford.edu/blog/2022-01-14-s4-3, 2022b. Retrieved 2022-01-14.

[Gu et al., 2017] Jiatao Gu, Graham Neubig, Kyunghyun Cho, and Victor O.K. Li. Learning to translate in real-time with neural machine translation. In *Proceedings of the European Chapter of the Association for Computational Linguistics (EACL) Conference, 2017*, 2017.

[Gu et al., 2018] Jiatao Gu, James Bradbury, Caiming Xiong, Victor O.K. Li, and Richard Socher.

Non-autoregressive neural machine translation. In *Proceedings of International Conference on Learning Representations*, 2018.

[Gulati et al., 2020] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. Conformer: Convolution-augmented transformer for speech recognition. *Proceedings of Interspeech 2020*, pages 5036–5040, 2020.

[Gulcehre et al., 2016] Caglar Gulcehre, Marcin Moczulski, Misha Denil, and Yoshua Bengio. Noisy activation functions. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 3059–3068. PMLR, 2016.

[Gulcehre et al., 2017] Caglar Gulcehre, Orhan Firat, Kelvin Xu, Kyunghyun Cho, and Yoshua Bengio. On integrating a language model into neural machine translation. *Computer Speech & Language*, 45: 137–148, 2017.

[Gunasekar et al., 2023] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Harkirat Singh Behl, Xin Wang, Sébastien Bubeck, Ronen Eldan, Adam Tauman Kalai, Yin Tat Lee, and Yuanzhi Li. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023.

[Guo et al., 2019] Maosheng Guo, Yu Zhang, and Ting Liu. Gaussian transformer: a lightweight approach for natural language inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 6489–6496, 2019.

[Guo et al., 2024] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *The Twelfth International Conference on Learning Representations*, 2024.

[Guo et al., 2020] Qipeng Guo, Xipeng Qiu, Pengfei Liu, Xiangyang Xue, and Zheng Zhang. Multi-scale self-attention for text classification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7847–7854, 2020.

[Gupta and Berant, 2020] Ankit Gupta and Jonathan Berant. Gmat: Global memory augmentation for transformers. *arXiv preprint arXiv:2006.03274*, 2020.

[Gupta et al., 2021] Ankit Gupta, Guy Dar, Shaya Goodman, David Ciprut, and Jonathan Berant. Memory-efficient transformers via top-k attention. In *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing*, pages 39–52, 2021.

[Gupta et al., 2004] Madan Gupta, Liang Jin, and Noriyasu Homma. *Static and dynamic neural networks: from fundamentals to advanced theory*. John Wiley & Sons, 2004.

[Guu et al., 2020] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *Proceedings of International conference on machine learning*, pages 3929–3938. PMLR, 2020.

[Guyon and Elisseeff, 2003] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of machine learning research*, 3(Mar):1157–1182, 2003.

[Haber and Ruthotto, 2017] Eldad Haber and Lars Ruthotto. Stable architectures for deep neural networks. *Inverse problems*, 34(1):014004, 2017.

[Hahn, 2020] Michael Hahn. Theoretical limitations of self-attention in neural sequence models. *Transactions of the Association for Computational Linguistics*, 8:156–171, 2020.

[Hamilton, 1994] James Douglas Hamilton. *Time Series Analysis*. Princeton University Press, 1994.

[Han et al., 2022] Kai Han, Yunhe Wang, Hanting Chen, Xinghao Chen, Jianyuan Guo, Zhenhua Liu, Yehui Tang, An Xiao, Chunjing Xu, Yixing Xu, Zhaohui Yang, Yiman Zhang, and Dacheng Tao. A survey on vision transformer. *IEEE transactions on pattern analysis and machine intelligence*, 45(1): 87–110, 2022.

[Han et al., 2020] Wei Han, Zhengdong Zhang, Yu Zhang, Jiahui Yu, Chung-Cheng Chiu, James Qin, Anmol Gulati, Ruoming Pang, and Yonghui Wu. Contextnet: Improving convolutional neural networks for automatic speech recognition with global context. In *Proceedings of Interspeech 2020*, pages 3610–3614, 2020.

[Han et al., 2021] Xu Han, Zhengyan Zhang, Ning Ding, Yuxian Gu, Xiao Liu, Yuqi Huo, Jiezhong Qiu, Liang Zhang, Wentao Han, Minlie Huang, Qin Jin, Yanyan Lan, Yang Liu, Zhiyuan Liu, Zhiwu Lu, Xipeng Qiu, Ruihua Song, Jie Tang, Ji-Rong Wen, Jinhui Yuan, Wayne Xin Zhao, and Jun Zhu. Pre-trained models: Past, present and future. *AI Open*, 2:225–250, 2021a.

[Han et al., 2021] Yizeng Han, Gao Huang, Shiji Song, Le Yang, Honghui Wang, and Yulin Wang. Dynamic neural networks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(11):7436–7456, 2021b.

[Han et al., 2024] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608*, 2024.

[Hannun et al., 2014] Awni Hannun, Carl Case, Jared Casper, Bryan Catanzaro, Greg Diamos, Erich Elsen, Ryan Prenger, Sanjeev Satheesh, Shubho Sengupta, and Adam Coates. Deep speech: Scaling up end-to-end speech recognition. *arXiv preprint arXiv:1412.5567*, 2014.

[Hao et al., 2019] Jie Hao, Xing Wang, Shuming Shi, Jinfeng Zhang, and Zhaopeng Tu. Multi-granularity self-attention for neural machine translation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 887–897, 2019.

[Hao et al., 2023] Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu. Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 8154–8173, 2023.

[Hao et al., 2025] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason E Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. In *Second Conference on Language Modeling*, 2025.

[Hao et al., 2022] Yiding Hao, Dana Angluin, and Robert Frank. Formal language recognition by hard attention transformers: Perspectives from circuit complexity. *Transactions of the Association for Computational Linguistics*, 10:800–810, 2022.

[Harlap et al., 2018] Aaron Harlap, Deepak Narayanan, Amar Phanishayee, Vivek Seshadri, Nikhil Devanur, Greg Ganger, and Phil Gibbons. Pipedream: Fast and efficient pipeline parallel dnn training. *arXiv preprint arXiv:1806.03377*, 2018.

[Harris, 1954] Zellig S Harris. Distributional structure. *Word*, 10(2-3):146–162, 1954.

[Hartshorne and Weiss, 1931] Charles Hartshorne and Paul Weiss, editors. *Collected Papers of Charles*

Sanders Peirce, volume 1-6. Harvard University Press, 1931.

[Hasler et al., 2018] Eva Hasler, Adrià de Gispert, Gonzalo Iglesias, and Bill Byrne. Neural machine translation decoding with terminology constraints. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 506–512, 2018.

[Hastie et al., 2009] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Spinger, 2009.

[He et al., 2017] Di He, Hanqing Lu, Yingce Xia, Tao Qin, Liwei Wang, and Tie-Yan Liu. Decoding with value networks for neural machine translation. *Advances in Neural Information Processing Systems*, 30, 2017.

[He et al., 2015] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.

[He et al., 2016] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016a.

[He et al., 2016] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *Proceedings of ECCV 2016*, pages 630–645, 2016b.

[He et al., 2019] Kaiming He, Ross Girshick, and Piotr Dollár. Rethinking imagenet pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4918–4927, 2019.

[He et al., 2022] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022.

[He et al., 2021] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention. In *Proceedings of International Conference on Learning Representations*, 2021.

[He et al., 2016] Wei He, Zhongjun He, Hua Wu, and Haifeng Wang. Improved neural machine translation with smt features. In *Proceedings of the Thirtieth AAAI conference on artificial intelligence*, 2016c.

[He et al., 2018] Xuanli He, Gholamreza Haffari, and Mohammad Norouzi. Sequence to sequence mixture model for diverse machine translation. In *Proceedings of the 22nd Conference on Computational Natural Language Learning*, pages 583–592, 2018.

[Heafield et al., 2021] Kenneth Heafield, Qianqian Zhu, and Roman Grundkiewicz. Findings of the WMT 2021 shared task on efficient translation. In *Proceedings of the Sixth Conference on Machine Translation*, pages 639–651, 2021.

[Hendrycks et al., 2020] Dan Hendrycks, Xiaoyuan Liu, Eric Wallace, Adam Dziedzic, Rishabh Krishnan, and Dawn Song. Pretrained transformers improve out-of-distribution robustness. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2744–2751, 2020.

[Hendrycks et al., 2021] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In

Proceedings of International Conference on Learning Representations, 2021a.

[Hendrycks et al., 2021] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021b.

[Hestness et al., 2017] Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. Deep learning scaling is predictable, empirically. *arXiv preprint arXiv:1712.00409*, 2017.

[Hewitt, 2024] John Hewitt. Instruction following without instruction tuning, 2024. URL https://nlp.stanford.edu/~johnhew/instruction-following.html.

[Hewitt and Liang, 2019] John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–2743, 2019.

[Hewitt et al., 2024] John Hewitt, Nelson F Liu, Percy Liang, and Christopher D Manning. Instruction following without instruction tuning. *arXiv preprint arXiv:2409.14254*, 2024.

[Hildebrand and Vogel, 2008] Almut Silja Hildebrand and Stephan Vogel. Combination of machine translation systems via hypothesis selection from combined n-best lists. In *Proceedings of the 8th Conference of the Association for Machine Translation in the Americas: Student Research Workshop*, pages 254–261, 2008.

[Hill et al., 2016] Felix Hill, Kyunghyun Cho, and Anna Korhonen. Learning distributed representations of sentences from unlabelled data. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1367–1377, 2016.

[Hinton, 2018] Geoff Hinton. Coursera neural networks for machine learning lecture 6, 2018. URL http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf.

[Hinton et al., 2015] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[Hinton, 2007] Geoffrey E Hinton. Learning multiple layers of representation. *Trends in cognitive sciences*, 11(10):428–434, 2007.

[Hinton and Roweis, 2002] Geoffrey E Hinton and Sam Roweis. Stochastic neighbor embedding. *Advances in neural information processing systems*, 15, 2002.

[Hinton and Salakhutdinov, 2006] Geoffrey E Hinton and Ruslan R Salakhutdinov. Reducing the dimensionality of data with neural networks. *science*, 313(5786):504–507, 2006.

[Hinton et al., 2006] Geoffrey E Hinton, Simon Osindero, and Yee-Whye Teh. A fast learning algorithm for deep belief nets. *Neural computation*, 18(7):1527–1554, 2006.

[Hinton et al., 2012] Geoffrey E Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012.

[Hoang et al., 2017] Cong Duy Vu Hoang, Gholamreza Haffari, and Trevor Cohn. Towards decoding as continuous optimisation in neural machine translation. In *Proceedings of the 2017 Conference on*

Empirical Methods in Natural Language Processing, pages 146–156, 2017.

[Hochreiter and Schmidhuber, 1997] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[Hoffmann et al., 2022] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[Hokamp and Liu, 2017] Chris Hokamp and Qun Liu. Lexically constrained decoding for sequence generation using grid beam search. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1535–1546, 2017.

[Holmström and Koistinen, 1992] Lasse Holmström and Petri Koistinen. Using additive noise in back-propagation training. *IEEE Transactions on Neural Networks*, 3(1):24–38, 1992.

[Holtzman et al., 2020] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *Proceedings of the 6th International Conference on Learning Representations ICLR*, 2020a.

[Holtzman et al., 2020] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020b.

[Honovich et al., 2023] Or Honovich, Thomas Scialom, Omer Levy, and Timo Schick. Unnatural instructions: Tuning language models with (almost) no human labor. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14409–14428, 2023.

[Hopfield, 1982] John J Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the national academy of sciences*, 79(8):2554–2558, 1982.

[Hopfield, 1984] John J Hopfield. Neurons with graded response have collective computational properties like those of two-state neurons. *Proceedings of the national academy of sciences*, 81(10): 3088–3092, 1984.

[Hou et al., 2020] Lu Hou, Zhiqi Huang, Lifeng Shang, Xin Jiang, Xiao Chen, and Qun Liu. Dynabert: Dynamic bert with adaptive width and depth. *Advances in Neural Information Processing Systems*, 33:9782–9793, 2020.

[Houlsby et al., 2019] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, pages 2790–2799. PMLR, 2019.

[Howard et al., 2019] Andrew Howard, Ruoming Pang, Hartwig Adam, Quoc V. Le, Mark Sandler, Bo Chen, Weijun Wang, Liang-Chieh Chen, Mingxing Tan, Grace Chu, Vijay Vasudevan, and Yukun Zhu. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1314–1324, 2019.

[Hsu et al., 2021] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language*

Processing, 29:3451–3460, 2021.

[Hu et al., 2021] Chi Hu, Chenglong Wang, Xiangnan Ma, Xia Meng, Yinqiao Li, Tong Xiao, Jingbo Zhu, and Changliang Li. Ranknas: Efficient neural architecture search by pairwise ranking. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 2469–2480, 2021.

[Hu et al., 2022] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[Huang et al., 2018] Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Ian Simon, Curtis Hawthorne, Noam Shazeer, Andrew M Dai, Matthew D Hoffman, Monica Dinculescu, and Douglas Eck. Music transformer: Generating music with long-term structure. In *Proceedings of International Conference on Learning Representations*, 2018.

[Huang et al., 2012] Eric H Huang, Richard Socher, Christopher D Manning, and Andrew Y Ng. Improving word representations via global context and multiple word prototypes. In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 873–882, 2012.

[Huang et al., 2016] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q Weinberger. Deep networks with stochastic depth. In *Proceedings of the 14th European Conference*, pages 646–661. Springer, 2016.

[Huang et al., 2017] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017a.

[Huang, 2009] Liang Huang. Dynamic programming-based search algorithms in NLP. In *Proceedings of Human Language Technologies: The 2009 Annual Conference of the North American Chapter of the Association for Computational Linguistics, Companion Volume: Tutorial Abstracts*, 2009.

[Huang et al., 2017] Liang Huang, Kai Zhao, and Mingbo Ma. When to finish? optimal beam search for neural text generation (modulo beam size). In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 2134–2139, 2017b.

[Huang et al., 2020] Xiao Shi Huang, Felipe Perez, Jimmy Ba, and Maksims Volkovs. Improving transformer optimization through better initialization. In *Proceedings of International Conference on Machine Learning*, pages 4475–4483. PMLR, 2020a.

[Huang et al., 2019] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, and Zhifeng Chen. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.

[Huang et al., 2015] Zhiheng Huang, Wei Xu, and Kai Yu. Bidirectional lstm-crf models for sequence tagging. *arXiv preprint arXiv:1508.01991*, 2015.

[Huang et al., 2020] Zhiheng Huang, Davis Liang, Peng Xu, and Bing Xiang. Improve transformer models with better relative position embeddings. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 3327–3335, 2020b.

[Hubel and Wiesel, 1959] David H Hubel and Torsten N Wiesel. Receptive fields of single neurones in the cat's striate cortex. *The Journal of physiology*, 148(3):574, 1959.

[Hurley, 2011] Patrick Hurley. *A Concise Introduction to Logic (11th ed.)*. Wadsworth Publishing, 2011.

[Hutchins et al., 2022] DeLesley Hutchins, Imanol Schlag, Yuhuai Wu, Ethan Dyer, and Behnam Neyshabur. Block-recurrent transformers. *Advances in neural information processing systems*, 35: 33248–33261, 2022.

[Hutchison et al., 2013] Keith A Hutchison, David A Balota, James H Neely, Michael J Cortese, Emily R Cohen-Shikora, Chi-Shing Tse, Melvin J Yap, Jesse J Bengson, Dale Niemeyer, and Erin Buchanan. The semantic priming project. *Behavior research methods*, 45(4):1099–1114, 2013.

[Ioffe and Szegedy, 2015] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. PMLR, 2015.

[Ivanov et al., 2021] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems*, volume 3, pages 711–732, 2021.

[Jackendoff, 1992] Ray S Jackendoff. *Semantic structures*, volume 18. MIT press, 1992.

[Jacob et al., 2018] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2704–2713, 2018.

[Jaderberg et al., 2015] Max Jaderberg, Karen Simonyan, Andrew Zisserman, and koray kavukcuoglu. Spatial transformer networks. *Advances in neural information processing systems*, 28, 2015.

[Jaeger and Haas, 2004] Herbert Jaeger and Harald Haas. Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication. *science*, 304(5667):78–80, 2004.

[Jaegle et al., 2021] Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, Daniel Zoran, Andrew Brock, Evan Shelhamer, Olivier J. Hénaff, Matthew M. Botvinick, Andrew Zisserman, Oriol Vinyals, and João Carreira. Perceiver io: A general architecture for structured inputs & outputs. In *Proceedings of International Conference on Learning Representations*, 2021.

[Janssen, 2012] Theo M.V. Janssen. Compositionality: its historic context. In M. Werning, W. Hinzen, and E. Machery, editors, *The Oxford handbook of compositionality*. Oxford University Press, 2012.

[Jean et al., 2015] Sébastien Jean, Orhan Firat, Kyunghyun Cho, Roland Memisevic, and Yoshua Bengio. Montreal neural machine translation systems for wmt'15. In *Proceedings of the tenth workshop on statistical machine translation*, pages 134–140, 2015.

[Jelinek, 1998] Frederick Jelinek. *Statistical methods for speech recognition*. MIT Press, 1998.

[Jia and Liang, 2017] Robin Jia and Percy Liang. Adversarial examples for evaluating reading comprehension systems. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 2021–2031, 2017.

[Jiang et al., 2023] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023a.

[Jiang et al., 2023] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, 2023b.

[Jiang et al., 2020] Zhengbao Jiang, Frank F Xu, Jun Araki, and Graham Neubig. How can we know what language models know? *Transactions of the Association for Computational Linguistics*, 8: 423–438, 2020.

[Jiao et al., 2020] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. Tinybert: Distilling bert for natural language understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4163–4174, 2020.

[Jolliffe, 2002] Ian T Jolliffe. *Principal component analysis for special types of data*. Springer, 2002.

[Joshi et al., 2017] Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, 2017.

[Joshi et al., 2020] Mandar Joshi, Danqi Chen, Yinhan Liu, Daniel S Weld, Luke Zettlemoyer, and Omer Levy. Spanbert: Improving pre-training by representing and predicting spans. *Transactions of the association for computational linguistics*, 8:64–77, 2020.

[Joulin et al., 2017] Armand Joulin, Édouard Grave, Piotr Bojanowski, and Tomáš Mikolov. Bag of tricks for efficient text classification. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pages 427–431, 2017.

[Jurafsky and Martin, 2008] Dan Jurafsky and James H. Martin. *Speech and Language Processing (2nd ed.)*. Prentice Hall, 2008.

[Kahneman, 2011] Daniel Kahneman. *Thinking, fast and slow*. macmillan, 2011.

[Kalchbrenner et al., 2014] Nal Kalchbrenner, Edward Grefenstette, and Phil Blunsom. A convolutional neural network for modelling sentences. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 655–665, 2014.

[Kambhampati, 2024] Subbarao Kambhampati. Can large language models reason and plan? *Annals of the New York Academy of Sciences*, 1534(1):15–18, 2024.

[Kambhampati et al., 2024] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Paul Saldyt, and Anil B Murthy. Position: LLMs can't plan, but can help planning in LLM-modulo frameworks. In *Proceedings of the 41st International Conference on Machine Learning*, pages 22895–22907. PMLR, 2024.

[Kaplan et al., 2020] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[Katharopoulos et al., 2020] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.

[Kelly and Stone, 1975] Edward F. Kelly and Philip J. Stone. *Computer recognition of English word senses*. American Elsevier Pub, 1975.

[Kernes, 2021] Jonathan Kernes. Master positional encoding: Part i, 05 2021. URL https://towardsdatascience.com/

master-positional-encoding-part-i-63c05d90a0c3.

[Khan et al., 2020] Asifullah Khan, Anabia Sohail, Umme Zahoora, and Aqsa Saeed Qureshi. A survey of the recent architectures of deep convolutional neural networks. *Artificial intelligence review*, 53 (8):5455–5516, 2020.

[Khandelwal et al., 2019] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *Proceedings of International Conference on Learning Representations (ICLR)*, 2019.

[Khandelwal et al., 2020] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.

[Khatri et al., 2025] Devvrit Khatri, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Sai Surya Duvvuri, Manzil Zaheer, Inderjit S Dhillon, David Brandfonbrener, and Rishabh Agarwal. The art of scaling reinforcement learning compute for llms. *arXiv preprint arXiv:2510.13786*, 2025.

[Khayrallah et al., 2017] Huda Khayrallah, Gaurav Kumar, Kevin Duh, Matt Post, and Philipp Koehn. Neural lattice search for domain adaptation in machine translation. In *Proceedings of the Eighth International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pages 20–25, 2017.

[Khot et al., 2023] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks. In *Proceedings of The Eleventh International Conference on Learning Representations*, 2023.

[Kidger, 2022] Patrick Kidger. On neural differential equations. *arXiv preprint arXiv:2202.02435*, 2022.

[Kikuchi et al., 2016] Yuta Kikuchi, Graham Neubig, Ryohei Sasano, Hiroya Takamura, and Manabu Okumura. Controlling output length in neural encoder-decoders. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1328–1338, 2016.

[Kim and Cho, 2021] Gyuwan Kim and Kyunghyun Cho. Length-adaptive transformer: Train once with length drop, use anytime with search. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 6501–6511, 2021.

[Kim et al., 2019] Najoung Kim, Roma Patel, Adam Poliak, Alex Wang, Patrick Xia, R. Thomas McCoy, Ian Tenney, Alexis Ross, Tal Linzen, Benjamin Van Durme, Samuel R. Bowman, and Ellie Pavlick. Probing what different nlp tasks teach machines about function word comprehension. In *Proceedings of the Eighth Joint Conference on Lexical and Computational Semantics (* SEM 2019)*, pages 235–249, 2019.

[Kim et al., 2023] Sehoon Kim, Coleman Hooper, Thanakul Wattanawong, Minwoo Kang, Ruohan Yan, Hasan Genc, Grace Dinh, Qijing Huang, Kurt Keutzer, Michael W. Mahoney, Yakun Sophia Shao, and Amir Gholami. Full stack optimization of transformer inference: a survey. *arXiv preprint arXiv:2302.14017*, 2023.

[Kim et al., 2021] Wonjae Kim, Bokyung Son, and Ildoo Kim. Vilt: Vision-and-language transformer without convolution or region supervision. In *Proceedings of International Conference on Machine Learning*, pages 5583–5594. PMLR, 2021.

[Kim, 2014] Yoon Kim. Convolutional neural networks for sentence classification. In *Proceedings*

of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 1746–1751, October 2014.

[Kim and Rush, 2016] Yoon Kim and Alexander M Rush. Sequence-level knowledge distillation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1317–1327, 2016.

[Kim et al., 2016] Yoon Kim, Yacine Jernite, David Sontag, and Alexander M Rush. Character-aware neural language models. In *Proceedings of the Thirtieth AAAI conference on artificial intelligence*, 2016.

[Kim and Awadalla, 2020] Young Jin Kim and Hany Hassan Awadalla. Fastformers: Highly efficient transformer models for natural language understanding. In *Proceedings of SustaiNLP: Workshop on Simple and Efficient Natural Language Processing*, pages 149–158, 2020.

[Kingma and Ba, 2014] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[Kingma and Welling, 2014] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *Proceedings of 2nd International Conference on Learning Representations, ICLR 2014*, 2014.

[Kingma and Welling, 2019] Diederik P. Kingma and Max Welling. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 2019.

[Kirkpatrick et al., 2017] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114 (13):3521–3526, 2017.

[Kiros et al., 2015] Ryan Kiros, Yukun Zhu, Russ R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Skip-thought vectors. *Advances in neural information processing systems*, 28, 2015.

[Kitaev et al., 2020] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *Proceedings of International Conference on Learning Representations*, 2020.

[Klein et al., 2017] Guillaume Klein, Yoon Kim, Yuntian Deng, Jean Senellart, and Alexander M Rush. Opennmt: Open-source toolkit for neural machine translation. In *Proceedings of ACL 2017, System Demonstrations*, pages 67–72, 2017.

[Klementiev et al., 2012] Alexandre Klementiev, Ivan Titov, and Binod Bhattarai. Inducing crosslingual distributed representations of words. In *Proceedings of COLING 2012*, pages 1459–1474, 2012.

[Klerke et al., 2015] Sigrid Klerke, Héctor Martínez Alonso, and Anders Søgaard. Looking hard: Eye tracking for detecting grammaticality of automatically compressed sentences. In *Proceedings of the 20th Nordic Conference of Computational Linguistics (NODALIDA 2015)*, pages 97–105, 2015.

[Knight, 1999] Kevin Knight. Decoding complexity in word-replacement translation models. *Computational linguistics*, 25(4):607–615, 1999.

[Knight, 2009] Kevin Knight. Bayesian inference with tears, 2009.

[Knight, 2018] Linda Knight. The sparrow tweets, 2018.

[Kochenderfer and Wheeler, 2019] Mykel J. Kochenderfer and Tim A. Wheeler. *Algorithms for Optimization*. The MIT Press, 2019.

[Koehn, 2004] Philipp Koehn. Pharaoh: a beam search decoder for phrase-based statistical machine translation models. In *Conference of the Association for Machine Translation in the Americas*, pages 115–124. Springer, 2004.

[Koehn, 2010] Philipp Koehn. *Statistical Machine Translation*. Cambridge University Press, 2010.

[Koehn and Knowles, 2017] Philipp Koehn and Rebecca Knowles. Six challenges for neural machine translation. In *Proceedings of the First Workshop on Neural Machine Translation*, pages 28–39, 2017.

[Koehn et al., 2003] Philipp Koehn, Franz Josef Och, and Daniel Marcu. Statistical phrase-based translation. In *Proceedings of the 2003 Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics*, pages 127–133, 2003.

[Koehn et al., 2007] Philipp Koehn, Hieu Hoang, Alexandra Birch, Chris Callison-Burch, Marcello Federico, Nicola Bertoldi, Brooke Cowan, Wade Shen, Christine Moran, Richard Zens, Chris Dyer, Ondřej Bojar, Alexandra Constantin, and Evan Herbst. Moses: Open source toolkit for statistical machine translation. In *Proceedings of the 45th Annual Meeting of the Association for Computational Linguistics Companion Volume Proceedings of the Demo and Poster Sessions*, pages 177–180, 2007.

[Kojima et al., 2022] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022.

[Konishi and Kitagawa, 2007] Sadanori Konishi and Genshiro Kitagawa. *Information Criteria and Statistical Modeling*. Spinger, 2007.

[Korthikanti et al., 2023] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems*, 5, 2023.

[Krakovna et al., 2020] Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: the flip side of ai ingenuity. https://deepmind.google/discover/blog/specification-gaming-the-flip-side-of-ai-ingenuity, 2020.

[Krebs et al., 2018] Alicia Krebs, Alessandro Lenci, and Denis Paperno. SemEval-2018 task 10: Capturing discriminative attributes. In *Proceedings of The 12th International Workshop on Semantic Evaluation*, pages 732–740, 2018.

[Krizhevsky et al., 2017] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6):84–90, 2017.

[Kudo, 2018] Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75, 2018.

[Kudo and Richardson, 2018] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, 2018.

[Kulikov et al., 2019] Ilia Kulikov, Alexander Miller, Kyunghyun Cho, and Jason Weston. Importance of search and evaluation strategies in neural dialogue modeling. In *Proceedings of the 12th International*

Conference on Natural Language Generation, pages 76–87, 2019.

[Kulis, 2013] Brian Kulis. Metric learning: A survey. *Foundations and Trends® in Machine Learning*, 5(4):287–364, 2013.

[Kumar et al., 2016] Ankit Kumar, Ozan Irsoy, Peter Ondruska, Mohit Iyyer, James Bradbury, Ishaan Gulrajani, Victor Zhong, Romain Paulus, and Richard Socher. Ask me anything: Dynamic memory networks for natural language processing. In *International conference on machine learning*, pages 1378–1387, 2016.

[Kumar et al., 2021] Sachin Kumar, Eric Malmi, Aliaksei Severyn, and Yulia Tsvetkov. Controlled text generation as continuous optimization with multiple constraints. *Advances in Neural Information Processing Systems*, 34:14542–14554, 2021.

[Kumar and Byrne, 2004] Shankar Kumar and William Byrne. Minimum Bayes-risk decoding for statistical machine translation. In *Proceedings of the Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics: HLT-NAACL 2004*, pages 169–176, 2004a.

[Kumar and Byrne, 2004] Shankar Kumar and William Byrne. Minimum bayes-risk decoding for statistical machine translation. In *Proceedings of the Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics: HLT-NAACL 2004*, pages 169–176, 2004b.

[Kung and Peng, 2023] Po-Nien Kung and Nanyun Peng. Do models really learn to follow instructions? an empirical study of instruction tuning. *arXiv preprint arXiv:2305.11383*, 2023.

[Kupiec, 1992] Julian Kupiec. Robust part-of-speech tagging using a hidden markov model. *Computer Speech & Language*, 6:225–242, 1992.

[Kwon et al., 2023] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *arXiv preprint arXiv:2309.06180*, 2023.

[Lafferty et al., 2001] John Lafferty, Andrew McCallum, and Fernando Pereira. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of the 18th International Conference on Machine Learning 2001*, pages 282–289, 2001.

[Lagunas et al., 2021] François Lagunas, Ella Charlaix, Victor Sanh, and Alexander M Rush. Block pruning for faster transformers. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 10619–10629, 2021.

[Lai et al., 2015] Siwei Lai, Liheng Xu, Kang Liu, and Jun Zhao. Recurrent convolutional neural networks for text classification. In *Twenty-ninth AAAI conference on artificial intelligence*, 2015.

[Lake and Baroni, 2018] Brenden Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *International conference on machine learning*, pages 2873–2882. PMLR, 2018.

[Lambert et al., 2024] Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. Rewardbench: Evaluating reward models for language modeling. *arXiv preprint arXiv:2403.13787*, 2024.

[Lambert et al., 2025] Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and

Hannaneh Hajishirzi. Rewardbench: Evaluating reward models for language modeling. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1755–1797, 2025.

[Lample and Conneau, 2019] Guillaume Lample and Alexis Conneau. Cross-lingual language model pretraining. *arXiv preprint arXiv:1901.07291*, 2019.

[Lample et al., 2016] Guillaume Lample, Miguel Ballesteros, Sandeep Subramanian, Kazuya Kawakami, and Chris Dyer. Neural architectures for named entity recognition. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 260–270, 2016.

[Lample et al., 2019] Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large memory layers with product keys. *Advances in Neural Information Processing Systems*, 32, 2019.

[Lan et al., 2020] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations. In *Proceedings of International Conference on Learning Representations*, 2020.

[Landauer et al., 1998] Thomas K Landauer, Peter W Foltz, and Darrell Laham. An introduction to latent semantic analysis. *Discourse processes*, 25(2-3):259–284, 1998.

[Lang et al., 1990] Kevin J Lang, Alex H Waibel, and Geoffrey E Hinton. A time-delay neural network architecture for isolated word recognition. *Neural networks*, 3(1):23–43, 1990.

[Lapesa and Evert, 2014] Gabriella Lapesa and Stefan Evert. A large scale evaluation of distributional semantic models: Parameters, interactions and model selection. *Transactions of the Association for Computational Linguistics*, 2:531–546, 2014.

[Lawson, 2003] Mark V. Lawson. *Finite Automata (1st ed.)*. Chapman and Hall/CRC, 2003.

[Le and Mikolov, 2014] Quoc Le and Tomas Mikolov. Distributed representations of sentences and documents. In *International conference on machine learning*, pages 1188–1196. PMLR, 2014.

[Leblond et al., 2021] Rémi Leblond, Jean-Baptiste Alayrac, Laurent Sifre, Miruna Pislar, Lespiau Jean-Baptiste, Ioannis Antonoglou, Karen Simonyan, and Oriol Vinyals. Machine translation decoding beyond beam search. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 8410–8434, 2021.

[LeCun and Bengio, 1995] Yann LeCun and Yoshua Bengio. Convolutional networks for images, speech, and time series. *The handbook of brain theory and neural networks*, 3361(10):1995, 1995.

[LeCun et al., 1989] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1(4):541–551, 1989.

[LeCun et al., 2012] Yann A LeCun, Léon Bottou, Genevieve B Orr, and Klaus-Robert Müller. Efficient backprop. In *Neural networks: Tricks of the trade*, pages 9–48. Springer, 2012.

[Lee et al., 2023] Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Ren Lu, Thomas Mesnard, Johan Ferret, Colton Bishop, Ethan Hall, Victor Carbune, and Abhinav Rastogi. Rlaif: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2023.

[Lee et al., 2017] Jason Lee, Kyunghyun Cho, and Thomas Hofmann. Fully character-level neural machine translation without explicit segmentation. *Transactions of the Association for Computational Linguistics*, 5:365–378, 2017.

[Lee et al., 2020] Jason Lee, Elman Mansimov, and Kyunghyun Cho. Deterministic non-autoregressive neural sequence modeling by iterative refinement. In *2018 Conference on Empirical Methods in Natural Language Processing, EMNLP 2018*, pages 1173–1182, 2020.

[Lee and Hockenmaier, 2025] Jinu Lee and Julia Hockenmaier. Evaluating step-by-step reasoning traces: A survey. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.

[Lee et al., 2019] John Boaz Lee, Ryan A Rossi, Sungchul Kim, Nesreen K Ahmed, and Eunyee Koh. Attention models in graphs: A survey. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 13(6):1–25, 2019.

[Lehnert et al., 2025] Lucas Lehnert, Sainbayar Sukhbaatar, DiJia Su, Qinqing Zheng, Paul McVay, Michael Rabbat, and Yuandong Tian. Beyond a*: Better planning with transformers via search dynamics bootstrapping. In *First Conference on Language Modeling*, 2025.

[Lenci, 2018] Alessandro Lenci. Distributional models of word meaning. *Annual review of Linguistics*, 4:151–171, 2018.

[Lepikhin et al., 2021] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. In *Proceedings of International Conference on Learning Representations*, 2021.

[Lester et al., 2021] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059, 2021.

[Leviathan et al., 2023] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.

[Levy and Goldberg, 2014] Omer Levy and Yoav Goldberg. Dependency-based word embeddings. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 302–308, 2014a.

[Levy and Goldberg, 2014] Omer Levy and Yoav Goldberg. Linguistic regularities in sparse and explicit word representations. In *Proceedings of the eighteenth conference on computational natural language learning*, pages 171–180, 2014b.

[Levy and Goldberg, 2014] Omer Levy and Yoav Goldberg. Neural word embedding as implicit matrix factorization. *Advances in neural information processing systems*, 27, 2014c.

[Levy et al., 2015] Omer Levy, Yoav Goldberg, and Ido Dagan. Improving distributional similarity with lessons learned from word embeddings. *Transactions of the association for computational linguistics*, 3:211–225, 2015.

[Lewis et al., 2020] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, 2020a.

[Lewis et al., 2020] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020b.

[Li et al., 2024] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. Llm inference serving: Survey of recent advances and opportunities. *arXiv preprint arXiv:2407.12391*, 2024a.

[Li et al., 2020] Bei Li, Hui Liu, Ziyang Wang, Yufan Jiang, Tong Xiao, Jingbo Zhu, Tongran Liu, and Changliang Li. Does multi-encoder help? a case study on context-aware neural machine translation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 3512–3518, 2020a.

[Li et al., 2020] Bei Li, Ziyang Wang, Hui Liu, Yufan Jiang, Quan Du, Tong Xiao, Huizhen Wang, and Jingbo Zhu. Shallow-to-deep training for neural machine translation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 995–1005, 2020b.

[Li et al., 2021] Bei Li, Ziyang Wang, Hui Liu, Quan Du, Tong Xiao, Chunliang Zhang, and Jingbo Zhu. Learning light-weight translation models from deep transformer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 13217–13225, 2021a.

[Li et al., 2022] Bei Li, Quan Du, Tao Zhou, Yi Jing, Shuhan Zhou, Xin Zeng, Tong Xiao, Jingbo Zhu, Xuebo Liu, and Min Zhang. Ode transformer: An ordinary differential equation-inspired model for sequence generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8335–8351, 2022a.

[Li et al., 2022] Bei Li, Tong Zheng, Yi Jing, Chengbo Jiao, Tong Xiao, and Jingbo Zhu. Learning multiscale transformer models for sequence generation. In *International Conference on Machine Learning*, pages 13225–13241. PMLR, 2022b.

[Li et al., 2023] Bei Li, Rui Wang, Junliang Guo, Kaitao Song, Xu Tan, Hany Hassan, Arul Menezes, Tong Xiao, Jiang Bian, and JingBo Zhu. Deliberate then generate: Enhanced prompting framework for text generation. *arXiv preprint arXiv:2305.19835*, 2023a.

[Li, 2011] Hang Li. *Learning to Rank for Information Retrieval and Natural Language Processing*. Online access: Morgan & Claypool Synthesis Collection Five. Morgan & Claypool Publishers, 2011. ISBN 9781608457076.

[Li et al., 2022] Hongkang Li, Meng Wang, Sijia Liu, and Pin-Yu Chen. A theoretical understanding of shallow vision transformers: Learning, generalization, and sample complexity. In *The Eleventh International Conference on Learning Representations*, 2022c.

[Li et al., 2022] Huayang Li, Yixuan Su, Deng Cai, Yan Wang, and Lemao Liu. A survey on retrieval-augmented text generation. *arXiv preprint arXiv:2202.01110*, 2022d.

[Li et al., 2021] Jicheng Li, Pengzhi Gao, Xuanfu Wu, Yang Feng, Zhongjun He, Hua Wu, and Haifeng Wang. Mixup decoding for diverse machine translation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 312–320, 2021b.

[Li et al., 2020] Jing Li, Aixin Sun, Jianglei Han, and Chenliang Li. A survey on deep learning for named entity recognition. *IEEE Transactions on Knowledge and Data Engineering*, 34(1):50–70, 2020c.

[Li and Jurafsky, 2016] Jiwei Li and Dan Jurafsky. Mutual information and diverse decoding improve neural machine translation. *arXiv preprint arXiv:1601.00372*, 2016.

[Li et al., 2016] Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and William B Dolan. A diversity-promoting objective function for neural conversation models. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human*

Language Technologies, pages 110–119, 2016.

[Li et al., 2017] Jiwei Li, Will Monroe, and Dan Jurafsky. Learning to decode for future success. *arXiv preprint arXiv:1701.06549*, 2017a.

[Li et al., 2017] Junhui Li, Deyi Xiong, Zhaopeng Tu, Muhua Zhu, Min Zhang, and Guodong Zhou. Modeling source syntax for neural machine translation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 688–697, 2017b.

[Li et al., 2021] Junnan Li, Ramprasaath Selvaraju, Akhilesh Gotmare, Shafiq Joty, Caiming Xiong, and Steven Chu Hong Hoi. Align before fuse: Vision and language representation learning with momentum distillation. *Advances in neural information processing systems*, 34:9694–9705, 2021c.

[Li et al., 2022] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International Conference on Machine Learning*, pages 12888–12900. PMLR, 2022e.

[Li et al., 2024] Shanda Li, Chong You, Guru Guruganesh, Joshua Ainslie, Santiago Ontanon, Manzil Zaheer, Sumit Sanghai, Yiming Yang, Sanjiv Kumar, and Srinadh Bhojanapalli. Functional interpolation for relative positions improves long context transformers. In *The Twelfth International Conference on Learning Representations*, 2024b.

[Li et al., 2023] Shenggui Li, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. Sequence parallelism: Long sequence training from system perspective. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2391–2404, 2023b.

[Li and Liang, 2021] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, 2021.

[Li et al., 2019] Xintong Li, Guanlin Li, Lemao Liu, Max Meng, and Shuming Shi. On the word alignment from neural machine translation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1293–1303, 2019.

[Li et al., 2022] Yanghao Li, Chao-Yuan Wu, Haoqi Fan, Karttikeya Mangalam, Bo Xiong, Jitendra Malik, and Christoph Feichtenhofer. Mvitv2: Improved multiscale vision transformers for classification and detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4804–4814, 2022f.

[Li et al., 2018] Yanyang Li, Tong Xiao, Yinqiao Li, Qiang Wang, Changming Xu, and Jingbo Zhu. A simple and effective approach to coverage-aware neural machine translation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 292–297, 2018.

[Li, 2023] Yinheng Li. A practical survey on zero-shot prompt design for in-context learning. In *Proceedings of the 14th International Conference on Recent Advances in Natural Language Processing*, pages 641–647, 2023.

[Li et al., 2023] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353, 2023c.

[Li et al., 2021] Zewen Li, Fan Liu, Wenjie Yang, Shouheng Peng, and Jun Zhou. A survey of

convolutional neural networks: analysis, applications, and prospects. *IEEE transactions on neural networks and learning systems*, 2021d.

[Li et al., 2025] Zhong-Zhi Li, Duzhen Zhang, Ming-Liang Zhang, Jiaxin Zhang, Zengyan Liu, Yuxuan Yao, Haotian Xu, Junhao Zheng, Pei-Jie Wang, Xiuyi Chen, et al. From system 1 to system 2: A survey of reasoning large language models. *arXiv preprint arXiv:2502.17419*, 2025.

[Lialin et al., 2023] Vladislav Lialin, Vijeta Deshpande, and Anna Rumshisky. Scaling down to scale up: A guide to parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.15647*, 2023.

[Liao et al., 2021] Kaiyuan Liao, Yi Zhang, Xuancheng Ren, Qi Su, Xu Sun, and Bin He. A global past-future early exit method for accelerating inference of pre-trained language models. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2013–2023, 2021.

[Lightman et al., 2024] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*, 2024.

[Likhomanenko et al., 2021] Tatiana Likhomanenko, Qiantong Xu, Gabriel Synnaeve, Ronan Collobert, and Alex Rogozhnikov. Cape: Encoding relative positions with continuous augmented positional embeddings. *Advances in Neural Information Processing Systems*, 34:16079–16092, 2021.

[Lin et al., 2022] Tianyang Lin, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. A survey of transformers. *AI Open*, 2022a.

[Lin et al., 2022] Ye Lin, Shuhan Zhou, Yanyang Li, Anxiang Ma, Tong Xiao, and Jingbo Zhu. Multi-path transformer is better: A case study on neural machine translation. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 5646–5656, 2022b.

[Lin et al., 2017] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.

[Ling et al., 2015] Wang Ling, Chris Dyer, Alan W Black, Isabel Trancoso, Ramón Fermandez, Silvio Amir, Luis Marujo, and Tiago Luís. Finding function in form: Compositional character models for open vocabulary word representation. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1520–1530, 2015.

[Linzen, 2016] Tal Linzen. Issues in evaluating semantic spaces using word analogies. In *Proceedings of the 1st Workshop on Evaluating Vector-Space Representations for NLP*, pages 13–18, 2016.

[Lippmann, 1989] Richard P Lippmann. Review of neural networks for speech recognition. *Neural computation*, 1(1):1–38, 1989.

[Lipton et al., 2015] Zachary C Lipton, John Berkowitz, and Charles Elkan. A critical review of recurrent neural networks for sequence learning. *arXiv preprint arXiv:1506.00019*, 2015.

[Liu et al., 2024] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024a.

[Liu et al., 2020] Fenglin Liu, Xuancheng Ren, Zhiyuan Zhang, Xu Sun, and Yuexian Zou. Rethinking skip connection with layer normalization. In *Proceedings of the 28th international conference on computational linguistics*, pages 3586–3598, 2020a.

[Liu et al., 2023] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction

tuning. *arXiv preprint arXiv:2304.08485*, 2023a.

[Liu and Motoda, 2012] Huan Liu and Hiroshi Motoda. *Feature selection for knowledge discovery and data mining*, volume 454. Springer Science & Business Media, 2012.

[Liu et al., 2022] Jiachang Liu, Dinghan Shen, Yizhe Zhang, William B Dolan, Lawrence Carin, and Weizhu Chen. What makes good in-context examples for gpt-3? In *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*, pages 100–114, 2022.

[Liu et al., 2016] Lemao Liu, Masao Utiyama, Andrew Finch, and Eiichiro Sumita. Agreement on target-bidirectional neural machine translation. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 411–416, 2016a.

[Liu et al., 2016] Lemao Liu, Masao Utiyama, Andrew Finch, and Eiichiro Sumita. Neural machine translation with supervised attention. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, pages 3093–3102, 2016b.

[Liu et al., 2020] Liyuan Liu, Xiaodong Liu, Jianfeng Gao, Weizhu Chen, and Jiawei Han. Understanding the difficulty of training transformers. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5747–5763, 2020b.

[Liu et al., 2020] Liyuan Liu, Xiaodong Liu, Jianfeng Gao, Weizhu Chen, and Jiawei Han. Understanding the difficulty of training transformers. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5747–5763, November 2020c.

[Liu et al., 2023] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023b.

[Liu et al., 2018] Peter J Liu, Mohammad Saleh, Etienne Pot, Ben Goodrich, Ryan Sepassi, Lukasz Kaiser, and Noam Shazeer. Generating wikipedia by summarizing long sequences. In *Proceedings of International Conference on Learning Representations*, 2018.

[Liu et al., 2017] Shusen Liu, Peer-Timo Bremer, Jayaraman J Thiagarajan, Vivek Srikumar, Bei Wang, Yarden Livnat, and Valerio Pascucci. Visual exploration of semantic relationships in neural word embeddings. *IEEE transactions on visualization and computer graphics*, 24(1):553–562, 2017.

[Liu et al., 2024] Tianqi Liu, Yao Zhao, Rishabh Joshi, Misha Khalman, Mohammad Saleh, Peter J Liu, and Jialu Liu. Statistical rejection sampling improves preference optimization. In *The Twelfth International Conference on Learning Representations*, 2024b.

[Liu, 2009] Tie-Yan Liu. Learning to rank for information retrieval. *Foundations and Trends® in Information Retrieval*, 3(3):225–331, 2009.

[Liu et al., 2023] Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. Gpt understands, too. *AI Open*, 2023c.

[Liu et al., 2023] Xiaoxia Liu, Jingyi Wang, Jun Sun, Xiaohan Yuan, Guoliang Dong, Peng Di, Wenhai Wang, and Dongxia Wang. Prompting frameworks for large language models: A survey. *arXiv preprint arXiv:2311.12785*, 2023d.

[Liu et al., 2024] Xinyu Liu, Runsong Zhao, Pengcheng Huang, Chunyang Xiao, Bei Li, Jingang Wang, Tong Xiao, and Jingbo Zhu. Forgetting curve: A reliable method for evaluating memorization capability for long-context models. In *Proceedings of the 2024 Conference on Empirical Methods in*

Natural Language Processing, pages 4667–4682, 2024c.

[Liu et al., 2023] Yang Liu, Yao Zhang, Yixin Wang, Feng Hou, Jin Yuan, Jiang Tian, Yang Zhang, Zhongchao Shi, Jianping Fan, and Zhiqiang He. A survey of visual transformers. *IEEE Transactions on Neural Networks and Learning Systems*, 2023e.

[Liu et al., 2019] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[Liu et al., 2020] Yuchen Liu, Junnan Zhu, Jiajun Zhang, and Chengqing Zong. Bridging the modality gap for speech-to-text translation. *arXiv preprint arXiv:2010.14920*, 2020d.

[Longpre et al., 2023] Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V. Le, Barret Zoph, Jason Wei, and Adam Roberts. The flan collection: Designing data and methods for effective instruction tuning. In *International Conference on Machine Learning*, pages 22631–22648. PMLR, 2023.

[Lopez, 2008] Adam Lopez. Statistical machine translation. *ACM Computing Surveys (CSUR)*, 40(3): 1–49, 2008.

[Lu et al., 2016] Jiasen Lu, Jianwei Yang, Dhruv Batra, and Devi Parikh. Hierarchical question-image co-attention for visual question answering. *Advances in neural information processing systems*, 29, 2016.

[Lund, 1995] Kevin Lund. Semantic and associative priming in high-dimensional semantic space. In *Proc. of the 17th Annual conferences of the Cognitive Science Society, 1995*, 1995.

[Lund and Burgess, 1996] Kevin Lund and Curt Burgess. Producing high-dimensional semantic spaces from lexical co-occurrence. *Behavior research methods, instruments, & computers*, 28(2):203–208, 1996.

[Luong et al., 2015] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1412–1421, 2015.

[Ma et al., 2019] Mingbo Ma, Liang Huang, Hao Xiong, Renjie Zheng, Kaibo Liu, Baigong Zheng, Chuanqiang Zhang, Zhongjun He, Hairong Liu, Xing Li, Hua Wu, and Haifeng Wang. Stacl: Simultaneous translation with implicit anticipation and controllable latency using prefix-to-prefix framework. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3025–3036, 2019.

[Ma and Hovy, 2016] Xuezhe Ma and Eduard Hovy. End-to-end sequence labeling via bi-directional lstm-cnns-crf. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1064–1074, 2016.

[Ma et al., 2023] Xuezhe Ma, Chunting Zhou, Xiang Kong, Junxian He, Liangke Gui, Graham Neubig, Jonathan May, and Luke Zettlemoyer. Mega: Moving average equipped gated attention. In *The Eleventh International Conference on Learning Representations*, 2023.

[Ma et al., 2024] Xuezhe Ma, Xiaomeng Yang, Wenhan Xiong, Beidi Chen, Lili Yu, Hao Zhang, Jonathan May, Luke Zettlemoyer, Omer Levy, and Chunting Zhou. Megalodon: Efficient llm pretraining and inference with unlimited context length. *arXiv preprint arXiv:2404.08801*, 2024.

[Madaan et al., 2024] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bod-

hisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2024.

[Mahowald et al., 2024] Kyle Mahowald, Anna A Ivanova, Idan A Blank, Nancy Kanwisher, Joshua B Tenenbaum, and Evelina Fedorenko. Dissociating language and thought in large language models. *Trends in cognitive sciences*, 28(6):517–540, 2024.

[Malaviya et al., 2018] Chaitanya Malaviya, Pedro Ferreira, and André FT Martins. Sparse and constrained attention for neural machine translation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 370–376, 2018.

[Manning and Schütze, 1999] Chris Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. The MIT Press, 1999.

[Manning, 2022] Christopher D Manning. Human language understanding & reasoning. *Daedalus*, 151(2):127–138, 2022.

[Manning et al., 2008] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[Manning et al., 2020] Christopher D Manning, Kevin Clark, John Hewitt, Urvashi Khandelwal, and Omer Levy. Emergent linguistic structure in artificial neural networks trained by self-supervision. *Proceedings of the National Academy of Sciences*, 117(48):30046–30054, 2020.

[Marcus, 1993] Gary F Marcus. Negative evidence in language acquisition. *Cognition*, 46(1):53–85, 1993.

[Markman, 2013] Arthur B Markman. *Knowledge representation*. Psychology Press, 2013.

[Markov, 1913] AA Markov. Essai d'une recherche statistique sur le texte du roman. *Eugene Onegin" illustrant la liaison des epreuve en chain ('Example of a statistical investigation of the text of "Eugene Onegin" illustrating the dependence between samples in chain"). In: Izvistia Imperatorskoi Akademii Nauk (Bulletin de l'Académie Impériale des Sciences de St.-Pétersbourg). 6th ser*, 7:153–162, 1913.

[Martins et al., 2022] Pedro Henrique Martins, Zita Marinho, and André FT Martins. ∞-former: Infinite memory transformer-former: Infinite memory transformer. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5468–5485, 2022.

[Masoudnia and Ebrahimpour, 2014] Saeed Masoudnia and Reza Ebrahimpour. Mixture of experts: a literature survey. *The Artificial Intelligence Review*, 42(2):275, 2014.

[Matusov et al., 2006] Evgeny Matusov, Nicola Ueffing, and Hermann Ney. Computing consensus translation for multiple machine translation systems using enhanced hypothesis alignment. In *11th Conference of the European Chapter of the Association for Computational Linguistics*, pages 33–40, 2006.

[Mavi et al., 2024] Vaibhav Mavi, Anubhav Jangra, and Adam Jatowt. Multi-hop question answering. *Foundations and Trends® in Information Retrieval*, 17(5):457–586, 2024.

[McCallum et al., 2000] Andrew McCallum, Dayne Freitag, and Fernando C. N. Pereira. Maximum entropy markov models for information extraction and segmentation. In *Proceedings of the Seventeenth International Conference on Machine Learning*, pages 591–598, 2000.

[McCann et al., 2017] Bryan McCann, James Bradbury, Caiming Xiong, and Richard Socher. Learned in translation: Contextualized word vectors. *Advances in neural information processing systems*, 30,

2017.

[McCarley et al., 2019] JS McCarley, Rishav Chakravarti, and Avirup Sil. Structured pruning of a bert-based question answering model. *arXiv preprint arXiv:1910.06360*, 2019.

[McClave and Sincich, 2006] James T. McClave and Terry Sincich. *Statistics (10th ed.)*. Prentice Hall, 2006.

[McCulloch and Pitts, 1943] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, 5(4):115–133, 1943.

[McElreath, 2020] Richard McElreath. *Statistical Rethinking: A Bayesian Course with Examples in R and STAN (2nd ed.)*. Chapman and Hall/CRC, 2020.

[McNamara, 2005] Timothy P McNamara. *Semantic priming: Perspectives from memory and word recognition*. Psychology Press, 2005.

[Meister et al., 2020] Clara Meister, Tim Vieira, and Ryan Cotterell. Best-first beam search. *Transactions of the Association for Computational Linguistics*, 8:795–809, 2020.

[Merrill et al., 2022] William Merrill, Ashish Sabharwal, and Noah A Smith. Saturated transformers are constant-depth threshold circuits. *Transactions of the Association for Computational Linguistics*, 10:843–856, 2022.

[Meyer and Schvaneveldt, 1971] David E Meyer and Roger W Schvaneveldt. Facilitation in recognizing pairs of words: evidence of a dependence between retrieval operations. *Journal of experimental psychology*, 90(2):227, 1971.

[Mi et al., 2016] Haitao Mi, Baskaran Sankaran, Zhiguo Wang, and Abe Ittycheriah. Coverage embedding models for neural machine translation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 955–960, 2016a.

[Mi et al., 2016] Haitao Mi, Zhiguo Wang, and Abe Ittycheriah. Supervised attentions for neural machine translation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2283–2288, 2016b.

[Michel et al., 2019] Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? *Advances in neural information processing systems*, 32, 2019.

[Micikevicius et al., 2018] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *Proceedings of International Conference on Learning Representations*, 2018.

[Mielke et al., 2021] Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y. Lee, Benoît Sagot, and Samson Tan. Between words and characters: A brief history of open-vocabulary modeling and tokenization in nlp. *arXiv preprint arXiv:2112.10508*, 2021.

[Miettinen, 1999] Kaisa Miettinen. *Nonlinear multiobjective optimization*, volume 12. Springer Science & Business Media, 1999.

[Mikolov et al., 2010] Tomas Mikolov, Martin Karafiát, Lukas Burget, Jan Cernockỳ, and Sanjeev Khudanpur. Recurrent neural network based language model. In *Proceedings of Interspeech*, pages 1045–1048, 2010.

[Mikolov et al., 2013] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation

of word representations in vector space. In *Proceedings of the International Conference on Learning Representations (ICLR 2013)*, 2013a.

[Mikolov et al., 2013] Tomas Mikolov, Quoc V Le, and Ilya Sutskever. Exploiting similarities among languages for machine translation. *arXiv preprint arXiv:1309.4168*, 2013b.

[Mikolov et al., 2013] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Proceedings of the 26th International Conference on Neural Information Processing Systems - Volume 2*, pages 3111–3119, 2013c.

[Mikolov et al., 2013] Tomas Mikolov, Wen-tau Yih, and Geoffrey Zweig. Linguistic regularities in continuous space word representations. In *Proceedings of the 2013 conference of the north american chapter of the association for computational linguistics: Human language technologies*, pages 746–751, 2013d.

[Miller et al., 2016] Alexander Miller, Adam Fisch, Jesse Dodge, Amir-Hossein Karimi, Antoine Bordes, and Jason Weston. Key-value memory networks for directly reading documents. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1400–1409, 2016.

[Min et al., 2019] Sewon Min, Victor Zhong, Luke Zettlemoyer, and Hannaneh Hajishirzi. Multi-hop reading comprehension through question decomposition and rescoring. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 6097–6109, 2019.

[Minaee et al., 2024] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. Large language models: A survey. *arXiv preprint arXiv:2402.06196*, 2024.

[Minsky, 1961] Marvin Minsky. Steps toward artificial intelligence. *Proceedings of the IRE*, 49(1): 8–30, 1961.

[Minsky and Papert, 1969] Marvin Minsky and Seymour Papert. *Perceptrons*. MIT press, 1969.

[Mishra et al., 2022] Swaroop Mishra, Daniel Khashabi, Chitta Baral, and Hannaneh Hajishirzi. Cross-task generalization via natural language crowdsourcing instructions. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3470–3487, 2022.

[Mitchell and Lapata, 2010] Jeff Mitchell and Mirella Lapata. Composition in distributional models of semantics. *Cognitive science*, 34(8):1388–1429, 2010.

[Mitchell, 1997] Tom M. Mitchell. *Machine Learning*. McGraw-Hill Education, 1997.

[Mnih and Kavukcuoglu, 2013] Andriy Mnih and Koray Kavukcuoglu. Learning word embeddings efficiently with noise-contrastive estimation. *Advances in neural information processing systems*, 26, 2013.

[Mnih et al., 2016] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Tim Harley, Timothy P Lillicrap, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on International Conference on Machine Learning*, pages 1928–1937, 2016.

[Mohri et al., 2018] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning (2nd ed.)*. MIT Press, 2018.

[Mohtashami and Jaggi, 2024] Amirkeivan Mohtashami and Martin Jaggi. Random-access infinite

context length for transformers. *Advances in Neural Information Processing Systems*, 36, 2024.

[Montague, 1974] Richard Montague. Universal grammar. In R. Thomason, editor, *Formal Philosophy: Selected Papers of Richard Montague*. Yale University Press, 1974.

[Mu et al., 2024] Jesse Mu, Xiang Li, and Noah Goodman. Learning to compress prompts with gist tokens. *Advances in Neural Information Processing Systems*, 36, 2024.

[Mukherjee et al., 2023] Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. Orca: Progressive learning from complex explanation traces of gpt-4. *arXiv preprint arXiv:2306.02707*, 2023.

[Müller and Sennrich, 2021] Mathias Müller and Rico Sennrich. Understanding the properties of minimum bayes risk decoding in neural machine translation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 259–272, 2021.

[Munkhdalai et al., 2024] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention. *arXiv preprint arXiv:2404.07143*, 2024.

[Murphy, 2012] Kevin P. Murphy. *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.

[Murray and Chiang, 2018] Kenton Murray and David Chiang. Correcting length bias in neural machine translation. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages 212–223, 2018.

[Nagel et al., 2021] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart Van Baalen, and Tijmen Blankevoort. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295*, 2021.

[Nair and Hinton, 2009] Vinod Nair and Geoffrey E Hinton. 3d object recognition with deep belief nets. *Advances in neural information processing systems*, 22, 2009.

[Nakano et al., 2021] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.

[Nanda et al., 2023] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *The Eleventh International Conference on Learning Representations*, 2023.

[Narayanan et al., 2021] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, 2021.

[Neelakantan et al., 2015] Arvind Neelakantan, Luke Vilnis, Quoc V Le, Ilya Sutskever, Lukasz Kaiser, Karol Kurach, and James Martens. Adding gradient noise improves learning for very deep networks. *arXiv preprint arXiv:1511.06807*, 2015.

[Neisser, 2014] Ulric Neisser. *Cognitive Psychology: Classic Edition*. Psychology Press, 2014.

[Nesterov, 1983] Yurii E Nesterov. A method for solving the convex programming problem with

convergence rate o (1/kˆ 2). In *Dokl. akad. nauk Sssr*, volume 269, pages 543–547, 1983.

[Ng et al., 1999] Andrew Y Ng, Daishi Harada, and Stuart J Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 278–287, 1999.

[Nguyen et al., 2020] Xuan-Phi Nguyen, Shafiq Joty, Steven Hoi, and Richard Socher. Tree-structured attention with hierarchical accumulation. In *Proceedings of the 8th International Conference on Learning Representations ICLR*, 2020.

[Nvidia, 2025] Nvidia. Nvidia nim llms benchmarking. https://docs.nvidia.com/nim/benchmarking/llm/latest/metrics.html, 2025. Retrieved 2025-03-17.

[Och, 2003] Franz Josef Och. Minimum error rate training in statistical machine translation. In *Proceedings of the 41st annual meeting of the Association for Computational Linguistics*, pages 160–167, 2003.

[Och and Ney, 2002] Franz Josef Och and Hermann Ney. Discriminative training and maximum entropy models for statistical machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 295–302, 2002.

[Och and Ney, 2003] Franz Josef Och and Hermann Ney. A systematic comparison of various statistical alignment models. *Computational Linguistics*, 29(1):19–51, 2003.

[Olive, 2022] David Olive. Robust statistics, 2022. URL http://parker.ad.siu.edu/Olive/ol-bookp.htm.

[Olshausen and Field, 1997] Bruno A Olshausen and David J Field. Sparse coding with an overcomplete basis set: A strategy employed by v1? *Vision research*, 37(23):3311–3325, 1997.

[Oord et al., 2017] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.

[Oord et al., 2018] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

[OpenAI, 2024] OpenAI. Learning to reason with llms, September 2024. URL https://openai.com/index/learning-to-reason-with-llms/.

[Opitz and Maclin, 1999] David Opitz and Richard Maclin. Popular ensemble methods: An empirical study. *Journal of artificial intelligence research*, 11:169–198, 1999.

[Oppenheim and Schafer, 1975] Alan V Oppenheim and Ronald W Schafer. Digital signal processing(book). *Prentice-Hall*, 1975.

[Orvieto et al., 2023] Antonio Orvieto, Samuel L Smith, Albert Gu, Anushan Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. Resurrecting recurrent neural networks for long sequences. *arXiv preprint arXiv:2303.06349*, 2023.

[Osgood, 1952] Charles E Osgood. The nature and measurement of meaning. *Psychological bulletin*, 49(3):197, 1952.

[Ott et al., 2018] Myle Ott, Michael Auli, David Grangier, and Marc'Aurelio Ranzato. Analyzing uncertainty in neural machine translation. In *International Conference on Machine Learning*, pages 3956–3965. PMLR, 2018a.

[Ott et al., 2018] Myle Ott, Sergey Edunov, David Grangier, and Michael Auli. Scaling neural machine translation. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages

1–9, October 2018b.

[Ott et al., 2019] Myle Ott, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, and Michael Auli. fairseq: A fast, extensible toolkit for sequence modeling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*, pages 48–53, 2019.

[Ouyang et al., 2022] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

[Padó and Lapata, 2007] Sebastian Padó and Mirella Lapata. Dependency-based construction of semantic space models. *Computational Linguistics*, 33(2):161–199, 2007.

[Pal et al., 2023] Koyena Pal, Jiuding Sun, Andrew Yuan, Byron C Wallace, and David Bau. Future lens: Anticipating subsequent tokens from a single hidden state. In *Proceedings of the 27th Conference on Computational Natural Language Learning (CoNLL)*, pages 548–560, 2023.

[Pan et al., 2022] Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. In *International Conference on Learning Representations*, 2022.

[Pan et al., 2024] Liangming Pan, Michael Saxon, Wenda Xu, Deepak Nathani, Xinyi Wang, and William Yang Wang. Automatically correcting large language models: Surveying the landscape of diverse automated correction strategies. *Transactions of the Association for Computational Linguistics*, 12:484–506, 2024.

[Pang et al., 2002] Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. Thumbs up? sentiment classification using machine learning techniques. In *Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing (EMNLP 2002)*, pages 79–86, 2002.

[Papineni et al., 2002] Kishore Papineni, Salim Roukos, Todd Ward, and Wei jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual meeting of the Association for Computational Linguistics*, pages 311–318, 2002.

[Parisi et al., 2022] Aaron Parisi, Yao Zhao, and Noah Fiedel. Talm: Tool augmented language models. *arXiv preprint arXiv:2205.12255*, 2022.

[Parisi et al., 2019] German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural networks*, 113:54–71, 2019.

[Park et al., 2019] Wonpyo Park, Dongju Kim, Yan Lu, and Minsu Cho. Relational knowledge distillation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 3967–3976, 2019.

[Parmar et al., 2018] Niki Parmar, Ashish Vaswani, Jakob Uszkoreit, Lukasz Kaiser, Noam Shazeer, Alexander Ku, and Dustin Tran. Image transformer. In *International conference on machine learning*, pages 4055–4064. PMLR, 2018.

[Pascanu et al., 2013] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International conference on machine learning*, pages 1310–1318. PMLR, 2013.

[Patel et al., 2024] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132. IEEE, 2024.

[Pearson, 1901] Karl Pearson. On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin philosophical magazine and journal of science*, 2(11):559–572, 1901.

[Penedo et al., 2023] Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Alessandro Cappelli, Hamza Alobeidli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. The refinedweb dataset for falcon llm: outperforming curated corpora with web data, and web data only. *arXiv preprint arXiv:2306.01116*, 2023.

[Peng et al., 2019] Baoyun Peng, Xiao Jin, Jiaheng Liu, Dongsheng Li, Yichao Wu, Yu Liu, Shunfeng Zhou, and Zhaoning Zhang. Correlation congruence for knowledge distillation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 5007–5016, 2019.

[Peng et al., 2023] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Leon Derczynski, Xingjian Du, Matteo Grella, Kranthi Gv, Xuzheng He, Haowen Hou, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartlomiej Koptyra, Hayden Lau, Jiaju Lin, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Johan S. Wind, Stanislaw Wozniak, Zhenyuan Zhang, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. Rwkv: Reinventing rnns for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

[Peng et al., 2024] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *The Twelfth International Conference on Learning Representations*, 2024.

[Peng et al., 2021] H Peng, N Pappas, D Yogatama, R Schwartz, N Smith, and L Kong. Random feature attention. In *Proceedings of International Conference on Learning Representations (ICLR 2021)*, 2021.

[Pennington et al., 2014] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. In *Proceedings of Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.

[Pérez et al., 2018] Jorge Pérez, Javier Marinković, and Pablo Barceló. On the turing completeness of modern neural network architectures. In *Proceedings of International Conference on Learning Representations*, 2018.

[Perozzi et al., 2014] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 701–710, 2014.

[Peters et al., 2018] Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, 2018.

[Petroni et al., 2019] Fabio Petroni, Tim Rocktäschel, Sebastian Riedel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, and Alexander Miller. Language models as knowledge bases? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International*

Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 2463–2473, 2019.

[Pham et al., 2019] Ngoc-Quan Pham, Thai-Son Nguyen, Jan Niehues, Markus Müller, Sebastian Stüker, and Alexander Waibel. Very deep self-attention networks for end-to-end speech recognition. *arXiv preprint arXiv:1904.13377*, 2019.

[Picone, 1993] Joseph W Picone. Signal modeling techniques in speech recognition. *Proceedings of the IEEE*, 81(9):1215–1247, 1993.

[Pires et al., 2023] Telmo Pessoa Pires, António V Lopes, Yannick Assogba, and Hendra Setiawan. One wide feedforward is all you need. *arXiv preprint arXiv:2309.01826*, 2023.

[Plackett, 1975] Robin L Plackett. The analysis of permutations. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 24(2):193–202, 1975.

[Plaut et al., 1986] David C Plaut, Steven J Nowlan, and Geoffrey E Hinton. Experiments on learning by back propagation. Technical report, Carnegie-Mellon University, 1986.

[Polyak, 1964] Boris T Polyak. Some methods of speeding up the convergence of iteration methods. *Ussr computational mathematics and mathematical physics*, 4(5):1–17, 1964.

[Pope et al., 2023] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems*, 2023.

[Porter, 1980] Martin F Porter. An algorithm for suffix stripping. *Program*, 1980.

[Post and Vilar, 2018] Matt Post and David Vilar. Fast lexically constrained decoding with dynamic beam allocation for neural machine translation. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 1314–1324, 2018.

[Power et al., 2022] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.

[Prasad et al., 2023] Archiki Prasad, Peter Hase, Xiang Zhou, and Mohit Bansal. Grips: Gradient-free, edit-based instruction search for prompting large language models. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 3845–3864, 2023.

[Prechelt, 1998] Lutz Prechelt. Early stopping-but when? In *Neural Networks: Tricks of the trade*, pages 55–69. Springer, 1998.

[Press et al., 2021] Ofir Press, Noah Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *Proceedings of International Conference on Learning Representations*, 2021.

[Press et al., 2022] Ofir Press, Noah Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *Proceedings of International Conference on Learning Representations*, 2022.

[Press et al., 2023] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5687–5711, 2023.

[Provilkov et al., 2020] Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. Bpe-dropout: Simple

and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1882–1892, 2020.

[Pryzant et al., 2023] Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with "gradient descent" and beam search. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[Qin et al., 2024] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, dahai li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations*, 2024.

[Qiu et al., 2020] Jiezhong Qiu, Hao Ma, Omer Levy, Wen-tau Yih, Sinong Wang, and Jie Tang. Blockwise self-attention for long document understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2555–2565, 2020a.

[Qiu et al., 2020] Xipeng Qiu, Tianxiang Sun, Yige Xu, Yunfan Shao, Ning Dai, and Xuanjing Huang. Pre-trained models for natural language processing: A survey. *Science China Technological Sciences*, 63(10):1872–1897, 2020b.

[Rabiner and Juang, 1993] Lawrence Rabiner and Biing-Hwang Juang. *Fundamentals of speech recognition*. Prentice-Hall, Inc., 1993.

[Rabiner and Gold, 1975] Lawrence R Rabiner and Bernard Gold. Theory and application of digital signal processing. *Prentice-Hall*, 1975.

[Radford et al., 2018] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. *OpenAI*, 2018.

[Radford et al., 2019] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8), 2019.

[Radford et al., 2021] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.

[Rae et al., 2019] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, Chloe Hillier, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. In *Proceedings of International Conference on Learning Representations*, 2019a.

[Rae et al., 2019] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, Chloe Hillier, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2019b.

[Rafailov et al., 2024] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.

[Raffel et al., 2017] Colin Raffel, Minh-Thang Luong, Peter J Liu, Ron J Weiss, and Douglas Eck. Online and linear-time attention by enforcing monotonic alignments. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pages 2837–2846, 2017.

[Raffel et al., 2020] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with

a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[Ramachandran et al., 2017] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017.

[Ramshaw and Marcus, 1995] Lance Ramshaw and Mitch Marcus. Text chunking using transformation-based learning. In *Third Workshop on Very Large Corpora*, 1995.

[Reddy, 1976] D Raj Reddy. Speech recognition by machine: A review. *Proceedings of the IEEE*, 64 (4):501–531, 1976.

[Reimers and Gurevych, 2019] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

[Reisinger and Mooney, 2010] Joseph Reisinger and Raymond Mooney. Multi-prototype vector-space models of word meaning. In *Human Language Technologies: The 2010 Annual Conference of the North American Chapter of the Association for Computational Linguistics*, pages 109–117, 2010.

[Ren et al., 2017] Zhou Ren, Xiaoyu Wang, Ning Zhang, Xutao Lv, and Li-Jia Li. Deep reinforcement learning-based image captioning with embedding reward. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 290–298, 2017.

[Rifai et al., 2011] Salah Rifai, Pascal Vincent, Xavier Muller, Xavier Glorot, and Yoshua Bengio. Contractive auto-encoders: Explicit invariance during feature extraction. In *Proceedings of International Conference on Machine Learning*, 2011.

[Rogers et al., 2018] Anna Rogers, Shashwath Hosur Ananthakrishna, and Anna Rumshisky. What's in your embedding, and how it predicts task performance. In *Proceedings of the 27th International Conference on Computational Linguistics*, pages 2690–2703, 2018.

[Rolnick et al., 2019] David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. Experience replay for continual learning. *Advances in Neural Information Processing Systems*, 32, 2019.

[Romero et al., 2014] Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. Fitnets: Hints for thin deep nets. *arXiv preprint arXiv:1412.6550*, 2014.

[Rosenblatt, 1957] Frank Rosenblatt. *The perceptron, a perceiving and recognizing automaton Project Para*. Cornell Aeronautical Laboratory, 1957.

[Rosenfeld et al., 2020] Jonathan S Rosenfeld, Amir Rosenfeld, Yonatan Belinkov, and Nir Shavit. A constructive prediction of the generalization error across scales. In *Proceedings of International Conference on Learning Representations*, 2020.

[Ross, 1924] William David Ross. *Aristotle's metaphysics*. Clarendon Press, 1924.

[Rosti et al., 2007] Antti-Veikko Rosti, Spyros Matsoukas, and Richard Schwartz. Improved word-level system combination for machine translation. In *Proceedings of the 45th Annual Meeting of the Association of Computational Linguistics*, pages 312–319, 2007.

[Roy et al., 2021] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021.

[Ruan et al., 2024] Junhao Ruan, Long Meng, Weiqiao Shan, Tong Xiao, and Jingbo Zhu. A survey of llm surveys. https://github.com/NiuTrans/ABigSurveyOfLLMs, 2024.

[Rubenstein and Goodenough, 1965] Herbert Rubenstein and John B Goodenough. Contextual correlates of synonymy. *Communications of the ACM*, 8(10):627–633, 1965.

[Rubin et al., 2022] Ohad Rubin, Jonathan Herzig, and Jonathan Berant. Learning to retrieve prompts for in-context learning. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2655–2671, 2022.

[Ruder, 2017] Sebastian Ruder. Deep learning for nlp best practices. https://ruder.io/deep-learning-nlp-best-practices/index.html, 2017.

[Rumelhart et al., 1986] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.

[Rush and Collins, 2012] Alexander M Rush and MJ Collins. A tutorial on dual decomposition and lagrangian relaxation for inference in natural language processing. *Journal of Artificial Intelligence Research*, 45:305–362, 2012.

[Rush et al., 2015] Alexander M Rush, Sumit Chopra, and Jason Weston. A neural attention model for abstractive sentence summarization. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 379–389, 2015.

[Russell, 2019] Stuart Russell. *Human Compatible: Artificial Intelligence and the Problem of Controls*. Viking, 2019.

[Russell and Norvig, 2010] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach (3nd ed.)*. Prentice Hall, 2010.

[Sanh et al., 2020] Victor Sanh, Thomas Wolf, and Alexander Rush. Movement pruning: Adaptive sparsity by fine-tuning. *Advances in Neural Information Processing Systems*, 33:20378–20389, 2020.

[Sanh et al., 2022] Victor Sanh, Albert Webson, Colin Raffel, Stephen Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Arun Raja, Manan Dey, M Saiful Bari, Canwen Xu, Urmish Thakker, Shanya Sharma Sharma, Eliza Szczechla, Taewoon Kim, Gunjan Chhablani, Nihal Nayak, Debajyoti Datta, Jonathan Chang, Mike Tian-Jian Jiang, Han Wang, Matteo Manica, Sheng Shen, Zheng Xin Yong, Harshit Pandey, Rachel Bawden, Thomas Wang, Trishala Neeraj, Jos Rozen, Abheesht Sharma, Andrea Santilli, Thibault Fevry, Jason Alan Fries, Ryan Teehan, Teven Le Scao, Stella Biderman, Leo Gao, Thomas Wolf, and Alexander M Rush. Multitask prompted training enables zero-shot task generalization. In *Proceedings of International Conference on Learning Representations*, 2022.

[Sankaran et al., 2016] Baskaran Sankaran, Haitao Mi, Yaser Al-Onaizan, and Abe Ittycheriah. Temporal attention model for neural machine translation. *arXiv preprint arXiv:1608.02927*, 2016.

[Santacroce et al., 2023] Michael Santacroce, Zixin Wen, Yelong Shen, and Yuanzhi Li. What matters in the structured pruning of generative language models? *arXiv preprint arXiv:2302.03773*, 2023.

[Santos and Gatti, 2014] Cícero dos Santos and Maíra Gatti. Deep convolutional neural networks for sentiment analysis of short texts. In *Proceedings of COLING 2014, the 25th International Conference on Computational Linguistics: Technical Papers*, pages 69–78, 2014.

[Schacter and Buckner, 1998] Daniel L Schacter and Randy L Buckner. Priming and the brain. *Neuron*, 20(2):185–195, 1998.

[Schapire, 1990] Robert E. Schapire. The strength of weak learnability. *Machine Learning*, 5(2):

197–227, 1990.

[Schick et al., 2023] Timo Schick, Jane A. Yu, Zhengbao Jiang, Fabio Petroni, Patrick Lewis, Gautier Izacard, Qingfei You, Christoforos Nalmpantis, Edouard Grave, and Sebastian Riedel. PEER: A collaborative language model. In *Proceedings of The Eleventh International Conference on Learning Representations*, 2023.

[Schick et al., 2024] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2024.

[Schlag et al., 2021] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of International Conference on Machine Learning*, pages 9355–9366. PMLR, 2021.

[Schmidhuber, 2015] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural networks*, 61:85–117, 2015.

[Schnabel et al., 2015] Tobias Schnabel, Igor Labutov, David Mimno, and Thorsten Joachims. Evaluation methods for unsupervised word embeddings. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pages 298–307, 2015a.

[Schnabel et al., 2015] Tobias Schnabel, Igor Labutov, David Mimno, and Thorsten Joachims. Evaluation methods for unsupervised word embeddings. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pages 298–307, 2015b.

[Schneider et al., 2019] Steffen Schneider, Alexei Baevski, Ronan Collobert, and Michael Auli. wav2vec: Unsupervised pre-training for speech recognition. In *INTERSPEECH*, 2019.

[Schulman et al., 2015] John Schulman, Sergey Levine, Philipp Moritz, Michael Jordan, and Pieter Abbeel. Trust region policy optimization. In *Proceedings of the 32nd International Conference on International Conference on Machine Learning-Volume 37*, pages 1889–1897, 2015.

[Schulman et al., 2017] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[Schuster and Nakajima, 2012] Mike Schuster and Kaisuke Nakajima. Japanese and korean voice search. In *Proceedings of International Conference on Acoustics, Speech and Signal Processing*, pages 5149–5152, 2012.

[Schuster et al., 2022] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. *Advances in Neural Information Processing Systems*, 35:17456–17472, 2022.

[Schwartz et al., 2020] Roy Schwartz, Gabriel Stanovsky, Swabha Swayamdipta, Jesse Dodge, and Noah A Smith. The right tool for the job: Matching model and instance complexities. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 6640–6651, 2020.

[See, 2018] Abigail See. Deep learning, structure and innate priors: A discussion between yann lecun and christopher manning, 02 2018. URL http://www.abigailsee.com/2018/02/21/deep-learning-structure-and-innate-priors.html.

[See et al., 2017] Abigail See, Peter J Liu, and Christopher D Manning. Get to the point: Summarization with pointer-generator networks. In *Proceedings of the 55th Annual Meeting of the Association for*

Computational Linguistics (Volume 1: Long Papers), pages 1073–1083, 2017.

[Seni et al., 2010] Giovanni Seni, John Elder, and Robert Grossman. *Ensemble Methods in Data Mining: Improving Accuracy Through Combining Predictions*. Morgan and Claypool Publishers, 2010.

[Sennrich et al., 2016] Rico Sennrich, Barry Haddow, and Alexandra Birch. Improving neural machine translation models with monolingual data. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 86–96, 2016a.

[Sennrich et al., 2016] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, 2016b.

[Seo et al., 2017] Minjoon Seo, Aniruddha Kembhavi, Ali Farhadi, and Hannaneh Hajishirzi. Bidirectional attention flow for machine comprehension. In *Proceedings of International Conference on Learning Representations*, 2017.

[Shannon, 1948] C. E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948a.

[Shannon, 1948] Claude E. Shannon. A mathematical theory of communication. Report, Bell Labs, 1948b.

[Shannon, 1951] Claude E Shannon. Prediction and entropy of printed english. *Bell system technical journal*, 30(1):50–64, 1951.

[Shao et al., 2024] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[Shaw et al., 2018] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 464–468, 2018.

[Shazeer, 2019] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[Shazeer, 2020] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[Shazeer et al., 2017] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Proceedings of International Conference on Learning Representations*, 2017.

[Shen et al., 2020] Dinghan Shen, Mingzhi Zheng, Yelong Shen, Yanru Qu, and Weizhu Chen. A simple but tough-to-beat data augmentation approach for natural language understanding and generation. *arXiv preprint arXiv:2009.13818*, 2020a.

[Shen et al., 2020] Sheng Shen, Zhen Dong, Jiayu Ye, Linjian Ma, Zhewei Yao, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Q-bert: Hessian based ultra low precision quantization of bert. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8815–8821, 2020b.

[Shen et al., 2016] Shiqi Shen, Yong Cheng, Zhongjun He, Wei He, Hua Wu, Maosong Sun, and

Yang Liu. Minimum risk training for neural machine translation. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1683–1692, 2016.

[Shen et al., 2019] Tianxiao Shen, Myle Ott, Michael Auli, and Marc'Aurelio Ranzato. Mixture models for diverse machine translation: Tricks of the trade. In *International conference on machine learning*, pages 5719–5728, 2019.

[Shen et al., 2025] Zhenyi Shen, Hanqi Yan, Linhai Zhang, Zhanghao Hu, Yali Du, and Yulan He. Codi: Compressing chain-of-thought into continuous space via self-distillation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 677–693, 2025.

[Shi et al., 2016] Xing Shi, Inkit Padhi, and Kevin Knight. Does string-based neural mt learn source syntax? In *Proceedings of the 2016 conference on empirical methods in natural language processing*, pages 1526–1534, 2016.

[Shinn et al., 2023] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.

[Shoeybi et al., 2019] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[Shorten and Khoshgoftaar, 2019] Connor Shorten and Taghi M Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of big data*, 6(1):1–48, 2019.

[Silver et al., 2017] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.

[Singhal, 2005] Amit Singhal. Introducing the knowledge graph: things, not strings, 2005.

[Skalse et al., 2022] Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. *Advances in Neural Information Processing Systems*, 35:9460–9471, 2022.

[Skorski et al., 2021] Maciej Skorski, Alessandro Temperoni, and Martin Theobald. Revisiting weight initialization of deep neural networks. In *Asian Conference on Machine Learning*, pages 1192–1207. PMLR, 2021.

[Smith et al., 2017] Samuel L Smith, David HP Turban, Steven Hamblin, and Nils Y Hammerla. Offline bilingual word vectors, orthogonal transformations and the inverted softmax. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.

[Smith et al., 2018] Samuel L. Smith, Pieter-Jan Kindermans, Chris Ying, and Quoc V. Le. Don't decay the learning rate, increase the batch size. In *Proceedings of the 6th International Conference on Learning Representations ICLR*, 2018.

[Snell et al., 2022] Charlie Snell, Dan Klein, and Ruiqi Zhong. Learning by distilling context. *arXiv preprint arXiv:2209.15189*, 2022.

[Snell et al., 2024] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[Snell et al., 2025] Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025.

[So et al., 2019] David So, Quoc Le, and Chen Liang. The evolved transformer. In *Proceedings of International conference on machine learning*, pages 5877–5886. PMLR, 2019.

[Socher et al., 2011] Richard Socher, Cliff C Lin, Chris Manning, and Andrew Y Ng. Parsing natural scenes and natural language with recursive neural networks. In *Proceedings of the 28th international conference on machine learning (ICML-11)*, pages 129–136, 2011.

[Socher et al., 2013] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642, 2013.

[Søgaard, 2016] Anders Søgaard. Evaluating word embeddings with fmri and eye-tracking. In *Proceedings of the 1st workshop on evaluating vector-space representations for NLP*, pages 116–121, 2016.

[Solorio-Fernández et al., 2020] Saúl Solorio-Fernández, J Ariel Carrasco-Ochoa, and José Fco Martínez-Trinidad. A review of unsupervised feature selection methods. *Artificial Intelligence Review*, 53(2):907–948, 2020.

[Song et al., 2019] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. Mass: Masked sequence to sequence pre-training for language generation. In *International Conference on Machine Learning*, pages 5926–5936. PMLR, 2019.

[Sperber et al., 2018] Matthias Sperber, Jan Niehues, Graham Neubig, Sebastian Stüker, and Alex Waibel. Self-attentional acoustic models. In *Proceedings of Interspeech 2018*, pages 3723–3727, 2018.

[Srivastava et al., 2014] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.

[Srivastava et al., 2015] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. *arXiv preprint arXiv:1505.00387*, 2015.

[Stahlberg and Byrne, 2019] Felix Stahlberg and Bill Byrne. On nmt search errors and model errors: Cat got your tongue? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3356–3362, 2019.

[Stahlberg et al., 2016] Felix Stahlberg, Eva Hasler, Aurelien Waite, and Bill Byrne. Syntactically guided neural machine translation. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 299–305, 2016.

[Sternberg, 1996] Robert J Sternberg. *Cognitive psychology*. Harcourt Brace College Publishers, 1996.

[Stewart, 1993] Gilbert W Stewart. On the early history of the singular value decomposition. *SIAM review*, 35(4):551–566, 1993.

[Stiennon et al., 2020] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize with human feedback. *Advances in Neural Information Processing Systems*, 33:3008–3021, 2020.

[Stock et al., 2021] Pierre Stock, Angela Fan, Benjamin Graham, Edouard Grave, Rémi Gribonval, Herve Jegou, and Armand Joulin. Training with quantization noise for extreme model compression. In *Proceedings of International Conference on Learning Representations*, 2021.

[Strubell et al., 2018] Emma Strubell, Patrick Verga, Daniel Andor, David Weiss, and Andrew McCallum. Linguistically-informed self-attention for semantic role labeling. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 5027–5038, 2018.

[Su et al., 2021] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[Su et al., 2024] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[Su et al., 2022] Yixuan Su, Tian Lan, Yan Wang, Dani Yogatama, Lingpeng Kong, and Nigel Collier. A contrastive framework for neural text generation. *Advances in Neural Information Processing Systems*, 35:21548–21561, 2022.

[Sukhbaatar et al., 2015] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. *Advances in neural information processing systems*, 28, 2015.

[Sukhbaatar et al., 2019] Sainbayar Sukhbaatar, Édouard Grave, Piotr Bojanowski, and Armand Joulin. Adaptive attention span in transformers. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 331–335, 2019.

[Sun et al., 2023] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

[Sun et al., 2020] Zewei Sun, Shujian Huang, Hao-Ran Wei, Xin-yu Dai, and Jiajun Chen. Generating diverse translation by manipulating multi-head attention. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8976–8983, 2020a.

[Sun et al., 2020] Zhiqing Sun, Hongkun Yu, Xiaodan Song, Renjie Liu, Yiming Yang, and Denny Zhou. Mobilebert: a compact task-agnostic bert for resource-limited devices. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2158–2170, 2020b.

[Sundermeyer et al., 2012] Martin Sundermeyer, Ralf Schlüter, and Hermann Ney. Lstm neural networks for language modeling. In *Proceedings of the Thirteenth annual conference of the international speech communication association*, 2012.

[Sutskever, 2013] Ilya Sutskever. *Training recurrent neural networks*. University of Toronto Toronto, 2013.

[Sutskever et al., 2013] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *International conference on machine learning*, pages 1139–1147. PMLR, 2013.

[Sutskever et al., 2014] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. *Advances in neural information processing systems*, 27, 2014.

[Sutton and McCallum, 2012] Charles Sutton and Andrew McCallum. An introduction to conditional random fields. *Foundations and Trends® in Machine Learning*, 4(4):267–373, 2012.

[Sutton, 1984] Richard S. Sutton. Temporal credit assignment in reinforcement learning. *Phd thesis, University of Massachusetts*, 1984.

[Sutton and Barto, 2018] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction (2nd ed.)*. The MIT Press, 2018.

[Szabó, 2020] Zoltán Gendler Szabó. Compositionality. In Edward N. Zalta, editor, *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, Fall 2020 edition, 2020.

[Szegedy et al., 2014] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *Proceedings of the 2nd International Conference on Learning Representations*, 2014a.

[Szegedy et al., 2014] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *Proceedings of 2nd International Conference on Learning Representations (ICLR 2014)*, 2014b.

[Szegedy et al., 2016] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2818–2826, 2016.

[Szepesvári, 2010] Csaba Szepesvári. Algorithms for reinforcement learning. *Synthesis Lectures on Artificial Intelligence and Machine Learning*, 4(1):1–103, 2010.

[Tai et al., 2015] Kai Sheng Tai, Richard Socher, and Christopher D Manning. Improved semantic representations from tree-structured long short-term memory networks. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 1556–1566, 2015.

[Talmor and Berant, 2018] Alon Talmor and Jonathan Berant. The web as a knowledge-base for answering complex questions. *arXiv preprint arXiv:1803.06643*, 2018.

[Tan and Le, 2019] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.

[Tan et al., 2026] Zelin Tan, Hejia Geng, Xiaohang Yu, Mulei Zhang, Guancheng Wan, Yifan Zhou, Qiang He, Xiangyuan Xue, Heng Zhou, Yutao Fan, Zhongzhi Li, Zaibin Zhang, Guibin Zhang, Chen Zhang, Zhenfei Yin, Philip Torr, and Lei Bai. Scaling behaviors of llm reinforcement learning post-training: An empirical study in mathematical reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 31300–31319, 2026.

[Tang et al., 2015] Duyu Tang, Bing Qin, and Ting Liu. Document modeling with gated recurrent neural network for sentiment classification. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pages 1422–1432, 2015.

[Tank and Hopfield, 1987] David W Tank and JJ Hopfield. Neural computation by concentrating information in time. *Proceedings of the National Academy of Sciences*, 84(7):1896–1900, 1987.

[Taori et al., 2023] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

[Taskar et al., 2005] Ben Taskar, Simon Lacoste-Julien, and Dan Klein. A discriminative matching approach to word alignment. In *Proceedings of Human Language Technology Conference and Conference on Empirical Methods in Natural Language Processing*, pages 73–80, 2005.

[Tay et al., 2020] Yi Tay, Dara Bahri, Liu Yang, Donald Metzler, and Da-Cheng Juan. Sparse sinkhorn attention. In *Proceedings of International Conference on Machine Learning*, pages 9438–9447. PMLR, 2020a.

[Tay et al., 2020] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *CoRR*, abs/2009.06732, 2020b.

[Team et al., 2024] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[Teknium, 2023] Teknium. Openhermes 2.5: An open dataset of synthetic data for generalist llm assistants, 2023. URL https://huggingface.co/datasets/teknium/OpenHermes-2.5.

[Telgarsky, 2016] Matus Telgarsky. Benefits of depth in neural networks. In *Conference on learning theory*, pages 1517–1539. PMLR, 2016.

[Tenney et al., 2019] Ian Tenney, Dipanjan Das, and Ellie Pavlick. Bert rediscovers the classical nlp pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4593–4601, 2019a.

[Tenney et al., 2019] Ian Tenney, Patrick Xia, Berlin Chen, Alex Wang, Adam Poliak, R Thomas McCoy, Najoung Kim, Benjamin Van Durme, Sam Bowman, Dipanjan Das, and Ellie Pavlick. What do you learn from context? probing for sentence structure in contextualized word representations. In *Proceedings of International Conference on Learning Representations*, 2019b.

[Timonin et al., 2022] Denis Timonin, BoYang Hsueh, and Vinh Nguyen. Accelerated inference for large transformer models using nvidia triton inference server. https://developer.nvidia.com/blog/accelerated-inference-for-large-transformer-models-using-nvidia-fastertrans 2022.

[Tissier et al., 2017] Julien Tissier, Christophe Gravier, and Amaury Habrard. Dict2vec: Learning word embeddings using lexical dictionaries. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 254–263, 2017.

[Tjong Kim Sang, 2002] Erik F. Tjong Kim Sang. Introduction to the CoNLL-2002 shared task: Language-independent named entity recognition. In *Proceedings of COLING-02: The 6th Conference on Natural Language Learning 2002 (CoNLL-2002)*, 2002.

[Tjong Kim Sang and Buchholz, 2000] Erik F. Tjong Kim Sang and Sabine Buchholz. Introduction to the CoNLL-2000 shared task chunking. In *Proceedings of Fourth Conference on Computational Natural Language Learning and the Second Learning Language in Logic Workshop*, 2000.

[Touvron et al., 2023] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023a.

[Touvron et al., 2023] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa,

Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023b.

[Trentin and Gori, 2001] Edmondo Trentin and Marco Gori. A survey of hybrid ann/hmm models for automatic speech recognition. *Neurocomputing*, 37(1-4):91–126, 2001.

[Tsvetkov et al., 2015] Yulia Tsvetkov, Manaal Faruqui, Wang Ling, Guillaume Lample, and Chris Dyer. Evaluation of word vector representations by subspace alignment. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 2049–2054, 2015.

[Tu et al., 2016] Zhaopeng Tu, Zhengdong Lu, Yang Liu, Xiaohua Liu, and Hang Li. Modeling coverage for neural machine translation. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 76–85, 2016.

[Tulving and Schacter, 1990] Endel Tulving and Daniel L Schacter. Priming and human memory systems. *Science*, 247(4940):301–306, 1990.

[Turpin et al., 2023] Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. *Advances in Neural Information Processing Systems*, 36:74952–74965, 2023.

[Uesato et al., 2022] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.

[Uffink, 2017] Jos Uffink. Boltzmann's Work in Statistical Physics. In Edward N. Zalta, editor, *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, Spring 2017 edition, 2017.

[Ulyanov et al., 2016] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization. *arXiv preprint arXiv:1607.08022*, 2016.

[Valmeekam et al., 2023] Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. *Advances in Neural Information Processing Systems*, 36:38975–38987, 2023.

[Van der Maaten and Hinton, 2008] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.

[Vapnik and Chervonenkis, 1971] Vladimir Vapnik and Alexey Chervonenkis. On the uniform convergence of relative frequencies of events to their probabilities. *Theory of Probability & Its Applications*, 16(2):264–279, 1971.

[Vaswani et al., 2017] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of Advances in Neural Information Processing Systems*, volume 30, 2017.

[Veličković et al., 2018] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero,

Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.

[Vijayakumar et al., 2018] Ashwin Vijayakumar, Michael Cogswell, Ramprasaath Selvaraju, Qing Sun, Stefan Lee, David Crandall, and Dhruv Batra. Diverse beam search for improved description of complex scenes. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[Vincent et al., 2010] Pascal Vincent, Hugo Larochelle, Isabelle Lajoie, Yoshua Bengio, Pierre-Antoine Manzagol, and Léon Bottou. Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. *Journal of machine learning research*, 11(12), 2010.

[Vinyals et al., 2015] Oriol Vinyals, Łukasz Kaiser, Terry Koo, Slav Petrov, Ilya Sutskever, and Geoffrey Hinton. Grammar as a foreign language. *Advances in neural information processing systems*, 28, 2015.

[Viterbi, 1967] Andrew J Viterbi. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 1967.

[Vogel et al., 1996] Stephan Vogel, Hermann Ney, and Christoph Tillmann. Hmm-based word alignment in statistical translation. In *COLING 1996 Volume 2: The 16th International Conference on Computational Linguistics*, 1996.

[Voita et al., 2018] Elena Voita, Pavel Serdyukov, Rico Sennrich, and Ivan Titov. Context-aware neural machine translation learns anaphora resolution. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1264–1274, 2018.

[Voita et al., 2019] Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 5797–5808, 2019.

[Von Oswald et al., 2023] Johannes Von Oswald, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. Transformers learn in-context by gradient descent. In *Proceedings of International Conference on Machine Learning*, pages 35151–35174. PMLR, 2023.

[Waibel et al., 1989] Alex Waibel, Toshiyuki Hanazawa, Geoffrey Hinton, Kiyohiro Shikano, and Kevin J Lang. Phoneme recognition using time-delay neural networks. *IEEE transactions on acoustics, speech, and signal processing*, 37(3):328–339, 1989.

[Wallace et al., 2019] Eric Wallace, Yizhong Wang, Sujian Li, Sameer Singh, and Matt Gardner. Do nlp models know numbers? probing numeracy in embeddings. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 5307–5315, 2019.

[Wang et al., 2024] Chenglong Wang, Hang Zhou, Yimin Hu, Yifu Huo, Bei Li, Tongran Liu, Tong Xiao, and Jingbo Zhu. Esrl: Efficient sampling-based reinforcement learning for sequence generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 19107–19115, 2024a.

[Wang et al., 2020] Hanrui Wang, Zhanghao Wu, Zhijian Liu, Han Cai, Ligeng Zhu, Chuang Gan, and Song Han. Hat: Hardware-aware transformers for efficient natural language processing. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7675–7688, 2020a.

[Wang et al., 2022] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Dongdong Zhang, and

Furu Wei. Deepnet: Scaling transformers to 1,000 layers. *arXiv preprint arXiv:2203.00555*, 2022a.

[Wang et al., 2022] Hongyu Wang, Shuming Ma, Shaohan Huang, Li Dong, Wenhui Wang, Zhiliang Peng, Yu Wu, Payal Bajaj, Saksham Singhal, Alon Benhaim, Barun Patra, Zhun Liu, Vishrav Chaudhary, Xia Song, and Furu Wei. Foundation transformers. *arXiv preprint arXiv:2210.06423*, 2022b.

[Wang et al., 2022] Jue Wang, Ke Chen, Gang Chen, Lidan Shou, and Julian McAuley. Skipbert: Efficient inference with shallow layer skipping. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7287–7301, 2022c.

[Wang et al., 2024] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6): 186345, 2024b.

[Wang and Yoon, 2021] Lin Wang and Kuk-Jin Yoon. Knowledge distillation and student-teacher learning for visual intelligence: A review and new outlooks. *IEEE transactions on pattern analysis and machine intelligence*, 44(6):3048–3068, 2021.

[Wang et al., 2023] Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. A comprehensive survey of continual learning: Theory, method and application. *arXiv preprint arXiv:2302.00487*, 2023a.

[Wang et al., 2023] Peihao Wang, Rameswar Panda, Lucas Torroba Hennigen, Philip Greengard, Leonid Karlinsky, Rogerio Feris, David Daniel Cox, Zhangyang Wang, and Yoon Kim. Learning to grow pretrained models for efficient transformer training. In *Proceedings of The Eleventh International Conference on Learning Representations*, 2023b.

[Wang et al., 2024] Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9426–9439, 2024c.

[Wang et al., 2018] Qiang Wang, Fuxue Li, Tong Xiao, Yanyang Li, Yinqiao Li, and Jingbo Zhu. Multi-layer representation fusion for neural machine translation. In *Proceedings of the 27th International Conference on Computational Linguistics*, pages 3015–3026, 2018a.

[Wang et al., 2019] Qiang Wang, Bei Li, Tong Xiao, Jingbo Zhu, Changliang Li, Derek F Wong, and Lidia S Chao. Learning deep transformer models for machine translation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1810–1822, 2019a.

[Wang et al., 2020] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020b.

[Wang et al., 2019] Wei Wang, Vincent Wenchen Zheng, Han Yu, and Chunyan Miao. A survey of zero-shot learning. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 10:1–37, 2019b.

[Wang et al., 2018] Xin Wang, Fisher Yu, Zi-Yi Dou, Trevor Darrell, and Joseph E Gonzalez. Skipnet: Learning dynamic routing in convolutional networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 409–424, 2018b.

[Wang et al., 2022] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, and Denny Zhou. Rationale-augmented ensembles in language models. *arXiv preprint arXiv:2207.00747*, 2022d.

[Wang et al., 2023] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan

Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *Proceedings of The Eleventh International Conference on Learning Representations*, 2023c.

[Wang et al., 2020] Yaqing Wang, Quanming Yao, James T. Kwok, and Lionel M. Ni. Generalizing from a few examples: A survey on few-shot learning. *ACM Computing Surveys*, 53(3):1–34, 2020c.

[Wang et al., 2022] Yizhong Wang, Swaroop Mishra, Pegah Alipoormolabashi, Yeganeh Kordi, Amirreza Mirzaei, Atharva Naik, Arjun Ashok, Arut Selvan Dhanasekaran, Anjana Arunkumar, David Stap, Eshaan Pathak, Giannis Karamanolakis, Haizhi Gary Lai, Ishan Purohit, Ishani Mondal, Jacob Anderson, Kirby Kuznia, Krima Doshi, Kuntal Kumar Pal, Maitreya Patel, Mehrad Moradshahi, Mihir Parmar, Mirali Purohit, Neeraj Varshney, Phani Rohitha Kaza, Pulkit Verma, Ravsehaj Singh Puri, Rushang Karia, Savan Doshi, Shailaja Keyur Sampat, Siddhartha Mishra, Sujan Reddy A, Sumanta Patro, Tanay Dixit, and Xudong Shen. Super-naturalinstructions: Generalization via declarative instructions on 1600+ nlp tasks. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 5085–5109, 2022e.

[Wang et al., 2023] Yizhong Wang, Hamish Ivison, Pradeep Dasigi, Jack Hessel, Tushar Khot, Khyathi Raghavi Chandu, David Wadden, Kelsey MacMillan, Noah A. Smith, Iz Beltagy, and Hannaneh Hajishirzi. How far can camels go? exploring the state of instruction tuning on open resources. *Advances in Neural Information Processing Systems*, 36:74764–74786, 2023d.

[Wang et al., 2023] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13484–13508, 2023e.

[Wang et al., 2023] Zhenyi Wang, Enneng Yang, Li Shen, and Heng Huang. A comprehensive survey of forgetting in deep learning beyond continual learning. *arXiv preprint arXiv:2307.09218*, 2023f.

[Wang et al., 2020] Ziheng Wang, Jeremy Wohlwend, and Tao Lei. Structured pruning of large language models. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6151–6162, 2020d.

[Warstadt et al., 2019] Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. Neural network acceptability judgments. *Transactions of the Association for Computational Linguistics*, 7:625–641, 2019.

[Webster and Kit, 1992] Jonathan J Webster and Chunyu Kit. Tokenization as the initial phase in nlp. In *Proceedings of COLING 1992 volume 4: The 14th international conference on computational linguistics*, 1992.

[Wei et al., 2022] Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. In *Proceedings of International Conference on Learning Representations*, 2022a.

[Wei et al., 2022] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022b.

[Wei et al., 2022] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022c.

[Weiss et al., 2021] Gail Weiss, Yoav Goldberg, and Eran Yahav. Thinking like transformers. In *Proceedings of International Conference on Machine Learning*, pages 11080–11090. PMLR, 2021.

[Welleck et al., 2023] Sean Welleck, Ximing Lu, Peter West, Faeze Brahman, Tianxiao Shen, Daniel Khashabi, and Yejin Choi. Generating sequences by learning to self-correct. In *Proceedings of The Eleventh International Conference on Learning Representations*, 2023.

[Weng, 2021] Lilian Weng. How to train really large models on many gpus? *lilianweng.github.io*, Sep 2021. URL https://lilianweng.github.io/posts/2021-09-25-train-large/.

[Werbos, 1990] Paul J Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.

[Weston et al., 2015] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *Proceedings of the 3rd International Conference on Learning Representations, ICLR 2015*, 2015.

[Wiener, 1960] Norbert Wiener. Some moral and technical consequences of automation: As machines learn they may develop unforeseen strategies at rates that baffle their programmers. *Science*, 131 (3410):1355–1358, 1960.

[Wiggs and Martin, 1998] Cheri L Wiggs and Alex Martin. Properties and mechanisms of perceptual priming. *Current opinion in neurobiology*, 8(2):227–233, 1998.

[Wiher et al., 2022] Gian Wiher, Clara Meister, and Ryan Cotterell. On decoding strategies for neural text generators. *Transactions of the Association for Computational Linguistics*, 10:997–1012, 2022.

[Williams et al., 2018] Adina Williams, Nikita Nangia, and Samuel Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 1112–1122, 2018.

[Williams, 1992] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8:229–256, 1992.

[Williams and Peng, 1990] Ronald J Williams and Jing Peng. An efficient gradient-based algorithm for on-line training of recurrent network trajectories. *Neural computation*, 2(4):490–501, 1990.

[Williams and Zipser, 1989] Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural computation*, 1(2):270–280, 1989.

[Wingate et al., 2022] David Wingate, Mohammad Shoeybi, and Taylor Sorensen. Prompt compression and contrastive conditioning for controllability and toxicity reduction in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 5621–5634, 2022.

[Wittgenstein, 1953] Ludwig Wittgenstein. *Philosophical investigations. Philosophische Untersuchungen.* Macmillan, 1953.

[Wold et al., 1987] Svante Wold, Kim Esbensen, and Paul Geladi. Principal component analysis. *Chemometrics and intelligent laboratory systems*, 2(1-3):37–52, 1987.

[Wolpert, 1996] David H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computatoin*, 8(7):1341–1390, 1996.

[Wolpert and Macready, 1997] David H. Wolpert and William G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997.

[Wozengraft and Reiffen, 1961] John M. Wozengraft and Barney Reiffen. *Sequential Decoding*. The

MIT Press, 1961.

[Wright and Ma, 2022] John Wright and Yi Ma. *High-Dimensional Data Analysis with Low-Dimensional Models: Principles, Computation, and Applications*. Cambridge University Press, 2022.

[Wu et al., 2023] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023a.

[Wu et al., 2018] Felix Wu, Angela Fan, Alexei Baevski, Yann Dauphin, and Michael Auli. Pay less attention with lightweight and dynamic convolutions. In *Proceedings of International Conference on Learning Representations*, 2018a.

[Wu et al., 2019] Felix Wu, Angela Fan, Alexei Baevski, Yann Dauphin, and Michael Auli. Pay less attention with lightweight and dynamic convolutions. In *Proceedings of International Conference on Learning Representations*, 2019.

[Wu et al., 2024] Wilson Wu, John X Morris, and Lionel Levine. Do language models plan for future tokens? *arXiv preprint arXiv:2404.00859*, 2024.

[Wu et al., 2020] Xuanfu Wu, Yang Feng, and Chenze Shao. Generating diverse translation from model distribution with dropout. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1088–1097, 2020a.

[Wu et al., 2016] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Łukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

[Wu et al., 2021] Yuhuai Wu, Markus Norman Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *Proceedings of International Conference on Learning Representations*, 2021.

[Wu and He, 2018] Yuxin Wu and Kaiming He. Group normalization. In *Proceedings of the European conference on computer vision (ECCV)*, pages 3–19, 2018.

[Wu et al., 2023] Zeqiu Wu, Yushi Hu, Weijia Shi, Nouha Dziri, Alane Suhr, Prithviraj Ammanabrolu, Noah A. Smith, Mari Ostendorf, and Hannaneh Hajishirzi. Fine-grained human feedback gives better rewards for language model training. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023b.

[Wu et al., 2020] Zhanghao Wu, Zhijian Liu, Ji Lin, Yujun Lin, and Song Han. Lite transformer with long-short range attention. In *Proceedings of International Conference on Learning Representations (ICLR)*, 2020b.

[Wu et al., 2018] Zuxuan Wu, Tushar Nagarajan, Abhishek Kumar, Steven Rennie, Larry S Davis, Kristen Grauman, and Rogerio Feris. Blockdrop: Dynamic inference paths in residual networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8817–8826, 2018b.

[Xia et al., 2024] Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen.

Less: Selecting influential data for targeted instruction tuning. *arXiv preprint arXiv:2402.04333*, 2024.

[Xia et al., 2025] Shijie Xia, Xuefeng Li, Yixin Liu, Tongshuang Wu, and Pengfei Liu. Evaluating mathematical reasoning beyond accuracy. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 27723–27730, 2025.

[Xiao et al., 2024] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *Proceedings of The Twelfth International Conference on Learning Representations*, 2024.

[Xiao et al., 2013] Tong Xiao, Jingbo Zhu, and Tongran Liu. Bagging and boosting statistical machine translation systems. *Artificial Intelligence*, 195:496–527, 2013.

[Xiao et al., 2019] Tong Xiao, Yinqiao Li, Jingbo Zhu, Zhengtao Yu, and Tongran Liu. Sharing attention weights for fast transformer. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI-19)*, pages 5292–5298, 2019.

[Xiao et al., 2026] Tong Xiao, Junhao Ruan, Bei Li, Zhengtao Yu, Min Zhang, and Jingbo Zhu. Ordinary differential equations in vision and language. *Authorea Preprints*, 2026.

[Xie et al., 2017] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1492–1500, 2017.

[Xie et al., 2022] Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An explanation of in-context learning as implicit bayesian inference. In *Proceedings of International Conference on Learning Representations*, 2022.

[Xin et al., 2020] Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. Deebert: Dynamic early exiting for accelerating bert inference. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2246–2251, 2020.

[Xiong et al., 2020] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning*, pages 10524–10533, 2020.

[Xu et al., 2024] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. Wizardlm: Empowering large pre-trained language models to follow complex instructions. In *The Twelfth International Conference on Learning Representations*, 2024.

[Xu and Mcauley, 2023] Canwen Xu and Julian Mcauley. A survey on dynamic neural networks for natural language processing. In *Findings of the Association for Computational Linguistics: EACL 2023*, pages 2325–2336, 2023.

[Xu et al., 2021] Chen Xu, Bojie Hu, Yanyang Li, Yuhao Zhang, Shen Huang, Qi Ju, Tong Xiao, and Jingbo Zhu. Stacked acoustic-and-textual encoding: Integrating the pre-trained models into speech translation encoders. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 2619–2630, 2021a.

[Xu et al., 2023] Chen Xu, Rong Ye, Qianqian Dong, Chengqi Zhao, Tom Ko, Mingxuan Wang, Tong Xiao, and Jingbo Zhu. Recent advances in direct speech-to-text translation. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI-23): Survey Track*,

pages 6796–6804, 2023a.

[Xu et al., 2023] Chen Xu, Yuhao Zhang, Chengbo Jiao, Xiaoqian Liu, Chi Hu, Xin Zeng, Tong Xiao, Anxiang Ma, Huizhen Wang, and Jingbo Zhu. Bridging the granularity gap for acoustic modeling. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 10816–10833, 2023b.

[Xu et al., 2020] Hongfei Xu, Qiuhui Liu, Josef van Genabith, Deyi Xiong, and Jingyi Zhang. Lipschitz constrained parameter initialization for deep transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 397–402, July 2020.

[Xu et al., 2015] Kelvin Xu, Jimmy Ba, Ryan Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhudinov, Rich Zemel, and Yoshua Bengio. Show, attend and tell: Neural image caption generation with visual attention. In *International conference on machine learning*, pages 2048–2057. PMLR, 2015.

[Xu et al., 2023] Peng Xu, Xiatian Zhu, and David A Clifton. Multimodal learning with transformers: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023c.

[Xu et al., 2021] Zenan Xu, Daya Guo, Duyu Tang, Qinliang Su, Linjun Shou, Ming Gong, Wanjun Zhong, Xiaojun Quan, Daxin Jiang, and Nan Duan. Syntax-enhanced pre-trained model. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 5412–5422, 2021b.

[Yang et al., 2024] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[Yang et al., 2025] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

[Yang et al., 2018] Baosong Yang, Zhaopeng Tu, Derek F Wong, Fandong Meng, Lidia S Chao, and Tong Zhang. Modeling localness for self-attention networks. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 4449–4458, 2018a.

[Yang et al., 2018] Yilin Yang, Liang Huang, and Mingbo Ma. Breaking the beam search curse: A study of (re-) scoring methods and stopping criteria for neural machine translation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3054–3059, 2018b.

[Yang et al., 2023] Zhengyuan Yang, Linjie Li, Kevin Lin, Jianfeng Wang, Chung-Ching Lin, Zicheng Liu, and Lijuan Wang. The dawn of lmms: Preliminary explorations with gpt-4v (ision). *arXiv preprint arXiv:2309.17421*, 2023a.

[Yang et al., 2019] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R Salakhutdinov, and Quoc V Le. Xlnet: Generalized autoregressive pretraining for language understanding. *Advances in neural information processing systems*, 32, 2019.

[Yang et al., 2023] Zi Yang, Samridhi Choudhary, Siegfried Kunzmann, and Zheng Zhang. Quantization-aware and tensor-compressed training of transformers for natural language understanding. *arXiv preprint arXiv:2306.01076*, 2023b.

[Yang et al., 2016] Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In *Proceedings of the 2016 conference of the North American chapter of the association for computational linguistics: human language technologies*, pages 1480–1489, 2016.

[Yao et al., 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *11th International Conference on Learning Representations, ICLR 2023*, 2023.

[Yao et al., 2024] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2024.

[Yao et al., 2007] Yuan Yao, Lorenzo Rosasco, and Andrea Caponnetto. On early stopping in gradient descent learning. *Constructive Approximation*, 26:289–315, 2007.

[Yarowsky, 1994] David Yarowsky. Decision lists for lexical ambiguity resolution: Application to accent restoration in Spanish and French. In *Proceedings of the 32nd Annual Meeting of the Association for Computational Linguistics*, pages 88–95, 1994.

[Yarowsky, 1995] David Yarowsky. Unsupervised word sense disambiguation rivaling supervised methods. In *Proceedings of the 33rd annual meeting of the association for computational linguistics*, pages 189–196, 1995.

[Ye et al., 2021] Rong Ye, Mingxuan Wang, and Lei Li. End-to-end speech translation via cross-modal progressive training. *arXiv preprint arXiv:2104.10380*, 2021.

[Yin et al., 2023] Shukang Yin, Chaoyou Fu, Sirui Zhao, Ke Li, Xing Sun, Tong Xu, and Enhong Chen. A survey on multimodal large language models. *arXiv preprint arXiv:2306.13549*, 2023.

[You et al., 2020] Weiqiu You, Simeng Sun, and Mohit Iyyer. Hard-coded gaussian attention for neural machine translation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7689–7700, 2020.

[Yu et al., 2022] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.

[Yu et al., 2023] Yaodong Yu, Sam Buchanan, Druv Pai, Tianzhe Chu, Ziyang Wu, Shengbang Tong, Benjamin D Haeffele, and Yi Ma. White-box transformers via sparse rate reduction. *arXiv preprint arXiv:2306.01129*, 2023a.

[Yu et al., 2023] Zihan Yu, Liang He, Zhen Wu, Xinyu Dai, and Jiajun Chen. Towards better chain-of-thought prompting strategies: A survey. *arXiv preprint arXiv:2310.04959*, 2023b.

[Yuksel et al., 2012] Seniha Esen Yuksel, Joseph N Wilson, and Paul D Gader. Twenty years of mixture of experts. *IEEE transactions on neural networks and learning systems*, 23(8):1177–1193, 2012.

[Yun et al., 2019] Chulhee Yun, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank Reddi, and Sanjiv Kumar. Are transformers universal approximators of sequence-to-sequence functions? In *Proceedings of International Conference on Learning Representations*, 2019.

[Zaheer et al., 2020] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, C. Alberti, S. Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, L. Yang, and A. Ahmed. Big bird: Transformers for longer sequences. *Advances in neural information processing systems*, 33: 17283–17297, 2020.

[Zaslavskiy et al., 2009] Mikhail Zaslavskiy, Marc Dymetman, and Nicola Cancedda. Phrase-based statistical machine translation as a traveling salesman problem. In *Proceedings of the Joint Conference of the 47th Annual Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing of the AFNLP*, pages 333–341, 2009.

[Zeiler, 2012] Matthew D Zeiler. Adadelta: an adaptive learning rate method. *arXiv preprint arXiv:1212.5701*, 2012.

[Zelikman et al., 2022] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488, 2022.

[Zelikman et al., 2024] Eric Zelikman, Georges Raif Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah Goodman. Quiet-STar: Language models can teach themselves to think before speaking. In *First Conference on Language Modeling*, 2024.

[Zellers et al., 2018] Rowan Zellers, Yonatan Bisk, Roy Schwartz, and Yejin Choi. Swag: A large-scale adversarial dataset for grounded commonsense inference. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 93–104, 2018.

[Zeng et al., 2024] Zhongshen Zeng, Yinhong Liu, Yingjia Wan, Jingyao Li, Pengguang Chen, Jianbo Dai, Yuxuan Yao, Rongwu Xu, Zehan Qi, Wanru Zhao, Linling Shen, Jianqiao Lu, Haochen Tan, Yukang Chen, Hao Zhang, Zhan Shi, Bailin Wang, Zhijiang Guo, and Jiaya Jia. Mr-ben: A meta-reasoning benchmark for evaluating system-2 thinking in llms. *Advances in Neural Information Processing Systems*, 37:119466–119546, 2024.

[Zhang et al., 2021] Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola. Dive into deep learning. *arXiv preprint arXiv:2106.11342*, 2021a.

[Zhang and Sennrich, 2019] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.

[Zhang et al., 2018] Biao Zhang, Deyi Xiong, and Jinsong Su. Accelerating neural transformer via an average attention network. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1789–1798, 2018a.

[Zhang et al., 2019] Biao Zhang, Ivan Titov, and Rico Sennrich. Improving deep transformer with depth-scaled initialization and merged attention. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 898–909, 2019a.

[Zhang et al., 2021] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning (still) requires rethinking generalization. *Communications of the ACM*, 64(3):107–115, 2021b.

[Zhang et al., 2020] Jiajun Zhang, Long Zhou, Yang Zhao, and Chengqing Zong. Synchronous bidirectional inference for neural sequence generation. *Artificial Intelligence*, 281:103234, 2020a.

[Zhang et al., 2019] Juexiao Zhang, Yubei Chen, Brian Cheung, and Bruno A Olshausen. Word embedding visualization via dictionary learning. *arXiv preprint arXiv:1910.03833*, 2019b.

[Zhang et al., 2020] Wei Emma Zhang, Quan Z Sheng, Ahoud Alhazmi, and Chenliang Li. Adversarial attacks on deep-learning models in natural language processing: A survey. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 11(3):1–41, 2020b.

[Zhang et al., 2015] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. *Advances in neural information processing systems*, 28, 2015.

[Zhang et al., 2018] Xiangwen Zhang, Jinsong Su, Yue Qin, Yang Liu, Rongrong Ji, and Hongji Wang. Asynchronous bidirectional decoding for neural machine translation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018b.

[Zhang et al., 2025] Yifan Zhang, Jingqin Yang, Yang Yuan, and Andrew C Yao. Cumulative reasoning with large language models. *Transactions on Machine Learning Research*, 2025.

[Zhang and Yang, 2021] Yu Zhang and Qiang Yang. A survey on multi-task learning. *IEEE Transactions on Knowledge and Data Engineering*, pages 1–1, 2021.

[Zhang et al., 2024] Yunxiang Zhang, Muhammad Khalifa, Lajanugen Logeswaran, Jaekyeom Kim, Moontae Lee, Honglak Lee, and Lu Wang. Small language models need strong verifiers to self-correct reasoning. In *ACL (Findings)*, 2024.

[Zhang et al., 2020] Zhuosheng Zhang, Yuwei Wu, Junru Zhou, Sufeng Duan, Hai Zhao, and Rui Wang. Sg-net: Syntax-guided machine reading comprehension. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 9636–9643, 2020c.

[Zhang et al., 2023] Zhuosheng Zhang, Yao Yao, Aston Zhang, Xiangru Tang, Xinbei Ma, Zhiwei He, Yiming Wang, Mark Gerstein, Rui Wang, Gongshen Liu, and Hai Zhao. Igniting language intelligence: The hitchhiker's guide from chain-of-thought reasoning to language agents. *arXiv preprint arXiv:2311.11797*, 2023a.

[Zhang et al., 2023] Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. Automatic chain of thought prompting in large language models. In *The Eleventh International Conference on Learning Representations*, 2023b.

[Zhao et al., 2006] Hai Zhao, Chang-Ning Huang, Mu Li, and Bao-Liang Lu. Effective tag set selection in Chinese word segmentation via conditional random field modeling. In *Proceedings of the 20th Pacific Asia Conference on Language, Information and Computation*, pages 87–94, 2006.

[Zhao et al., 2024] Hao Zhao, Maksym Andriushchenko, Francesco Croce, and Nicolas Flammarion. Long is more for alignment: A simple but tough-to-beat baseline for instruction fine-tuning. *arXiv preprint arXiv:2402.04833*, 2024.

[Zhao et al., 2023] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Z. Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jianyun Nie, and Ji rong Wen. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.

[Zheng et al., 2019] Baigong Zheng, Renjie Zheng, Mingbo Ma, and Liang Huang. Simpler and faster learning of adaptive policies for simultaneous translation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1349–1354, 2019.

[Zheng et al., 2023] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623,

2023.

[Zheng et al., 2018] Zaixiang Zheng, Hao Zhou, Shujian Huang, Lili Mou, Xinyu Dai, Jiajun Chen, and Zhaopeng Tu. Modeling past and future for neural machine translation. *Transactions of the Association for Computational Linguistics*, 6:145–157, 2018.

[Zhong et al., 2024] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.

[Zhou et al., 2024] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning*, pages 62138–62160, 2024.

[Zhou et al., 2023] Chunting Zhou, Pengfei Liu, Puxin Xu, Srini Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, Susan Zhang, Gargi Ghosh, Mike Lewis, Luke Zettlemoyer, and Omer Levy. Lima: Less is more for alignment. *arXiv preprint arXiv:2305.11206*, 2023a.

[Zhou et al., 2023] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. Least-to-most prompting enables complex reasoning in large language models. In *Proceedings of The Eleventh International Conference on Learning Representations*, 2023b.

[Zhou et al., 2021] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 11106–11115, 2021.

[Zhou et al., 2017] Long Zhou, Wenpeng Hu, Jiajun Zhang, and Chengqing Zong. Neural system combination for machine translation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 378–384, 2017.

[Zhou et al., 2020] Wangchunshu Zhou, Canwen Xu, Tao Ge, Julian McAuley, Ke Xu, and Furu Wei. Bert loses patience: Fast and robust inference with early exit. *Advances in Neural Information Processing Systems*, 33:18330–18341, 2020.

[Zhou et al., 2023] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *The Eleventh International Conference on Learning Representations*, 2023c.

[Zhou, 2012] Zhi-Hua Zhou. *Ensemble Methods: Foundations and Algorithms*. Chapman and Hall/CRC, 2012a.

[Zhou, 2012] Zhi-Hua Zhou. *Ensemble methods: foundations and algorithms*. CRC press, 2012b.

[Zoph and Le, 2016] Barret Zoph and Quoc Le. Neural architecture search with reinforcement learning. In *Proceedings of International Conference on Learning Representations*, 2016.

[Zoph et al., 2020] Barret Zoph, Golnaz Ghiasi, Tsung-Yi Lin, Yin Cui, Hanxiao Liu, Ekin Dogus Cubuk, and Quoc Le. Rethinking pre-training and self-training. *Advances in neural information processing systems*, 33:3833–3845, 2020.

Index

k-NN LM, 457
k-NN language modeling, 457
k-nearest neighbor, 35, 455
l_2 regularization, 96
n-gram language modeling, 76
p-value, 51
k-NN, 455
0-1 Loss, 37
0-1 Masking, 288
A2C, 577
AIC, 46
ALiBi, 467
AR process, 180
AdaDelta, 89
AdaGrad, 88
Adam, 89
Adaptive Gradient Descent, 88
Adaptive Moment Estimation, 89
Additive Attention, 224
Agent, 427
Artificial neural network, 67
Associativity, 14
AutoML, 337, 530
BART, 390
BERT, 371
BGD, 91
BIC, 46
BLEU, 50
BPE, 128
BPTT, 80, 177
Bayesian risk, 40
Beam search, 248
Bernoulli naive Bayes, 26
Best-of-N sampling, 601
BoN sampling, 601
Boltzmann distribution, 100
Bradley-Terry model, 580
Byte Pair Encoding, 128
CAE, 109
CNN, 81
COCONUT, 694
CRF, 128, 203
CTC, 201
Classification, 19
Cloze, 296
CoT prompting, 432
CoT trace, 681
CoT, 504
Commutativity, 14
Compositionality, 138
Computation graph, 71
Conditional Probability, 17
Constant Initialization, 92
Contextuality, 138
Continuous batching, 638
Contrastive Loss, 38
Convolutional neural network, 81
Cosine Attention, 224
DAE, 110
DPO, 592
DTG, 519
Decoding, 216, 619
Denoising auto-encoder, 110
Distance-based Loss, 36
Distributivity, 14
Divergence-based Loss, 36
Document Rotation, 391
Dot-product Attention, 223

Dynamic Neural Network, 337
Emission-like Feature, 205
Error-based Loss, 39
Euler method, 307
Extrapolation, 463
Extrinsic Evaluation, 160
FFNNLM, 76
FFNN, 70
Feature mapping, 33
Feature selection, 150
Fisher's linear discriminant, 149
Fixed Learning Rate, 94
Frobenius norm, 109
GLU, 439
GMM, 208
GPT, 371
GQA, 461
GRPO, 685
GRU, 185
Gaussian mixture model, 208
Gaussian naive Bayes, 26
Gaussian noise, 102
GeLU, 438
GloVe, 154
Global vector, 154
Gradient Descent with Momentum, 87
Grouped query attention, 461
HMM, 128, 203
ICA, 150
ICL, 377, 432
IDF, 144
Inference Engine, 637
Initialization with Predefined Distribution, 92
Interpolation, 463
Intrinsic Evaluation, 160
Jacobian matrix, 109
Joint Probability, 16
KV cache, 448, 617
Knowledge distillation, 341
Kullback-Leibler (KL) divergence, 19
LDA, 148
LSA, 145
LSI, 145
LSTM, 183
Lasso regularization, 96
Layer-sensitive Initialization, 92
LeCun initialization, 93
Learning Rate Decay, 94
Learning from Human Feedback, 425
Linear Multi-step Method, 307
Lipschitz constant, 304
Long short-term memory, 183
MAE, 37
MAP, 264
MBR, 264
MDL, 46
MERT, 40
MLE, 25, 36
MML, 46
MQA, 461
MSE, 37
Margin-based Loss, 37
Marginal Probability, 17
Markov assumption, 128
Minimum Description Length, 46
MoE, 311
Model evaluation, 43
Model selection, 43
Monte Carlo rollout, 677
Multi-class classification, 21
Multi-head attention, 230
Multi-label classification, 21
NAS, 337, 530
NER, 54
NMT, 219
NSP, 384
NTK-aware scaled RoPE, 473
Neural Architecture Search, 337
No Free Lunch Theorem, 44
Non-linear activation function, 34
Non-parametric method, 34
ODE, 307
OOV, 28
OPD, 686

ORM, 656
Occam's Razor, 35
Outcome-based Approach, 598
PCA, 104, 147
PDF, 17
PGR, 568
PMF, 17
PMI, 142
POS tagging, 53
PPO, 429, 584
PRM, 656
Performance Estimation, 530
Plackett-Luce model, 586
Pre-training, 31
Prefilling, 619
Probability, 16
Process-based Approach, 598
Prompt Search Space, 530
Q-value function, 574
QKV attention, 227
RAG, 323, 458
RLHF, 426, 552
RMSProp, 89
RNN, 79
RPR, 298
Ranking-based Loss, 38
Ranking, 42
ReLU, 438
Recurrent neural network, 79
Residual neural network, 85
Reward Model, 427
Ridge regularization, 96
RoBERTa, 398
Runge-Kutta Method, 308
SFT, 425, 552
SGD, 91
SMT, 240
SSM, 333
SVD, 145
Scheduler, 637
Semantic relatedness, 162
Sentence Reordering, 391
Sequence Encoding Model, 374
Sequence Generation Model, 374
Sequence labeling, 53
Soft Masking, 289
Span Masking, 390
Speculative decoding, 629
Statistical language modeling, 54
Stopping criterion, 90
Strong Ceiling Performance, 568
Structure prediction, 42
Sub-problem Generation, 510
Sub-problem Solving, 510
Supervised Fine-tuning, 425
Supervised dimensionality reduction, 149
Supervised learning, 30
Surface Form of Words and Sentences, 296
Syntactic and Semantic Label, 295
T5, 386
TD, 578
TF-IDF, 143
TF, 143
The Expectation Step, 133
The Maximization Step, 133
The continuous bag-of-words model, 152
The continuous skip-gram model, 153
The learning problem, 21
The modeling problem, 21
The prediction problem, 21
Token Deletion, 390
Token Masking, 390
Tokenization, 121
Topic model, 148
Transformer-XL, 319
Transformer, 271
Transition-like Feature, 205
Tree, 294
Type I error, 51
Type II error, 51
VAE, 111
VQA, 238, 356
Vapnik-Chervonenkis (VC) dimension, 45
Vector, 12

ViT, 354
Vision Transformer, 354
Viterbi decoding, 202
WSD, 139
Warmup and Decay, 94
Weak Performance, 568
Weak-to-strong Performance, 568
Weighted Dot-product Attention, 224
Word Representation Learning, 121
Word alignment, 235
Word2Vec, 152
XLM, 400
Xavier initialization, 93
ZOH, 334
Zero-Order-Hold, 334
Zipf's Law, 28
Absolute positional encoding, 197, 275
Action-value function, 574
Activation function, 23
Add-α smoothing, 29
Additive smoothing, 29
Addressing, 238
Advantage actor-critic, 577
Advantage, 577
Adversarial machine learning, 104
Adversarial sample, 104
Affine transformation, 68
Alignment link, 59
Alignment score, 223
Alignment, 425
Alternative hypothesis, 51
Attack, 104
Attention field, 314
Attention head, 231
Attention weight, 222
Attention with linear bias, 467
Auto-encoder, 105
Auto-encoding, 39, 55
Auto-regressive, 54
Automated machine learning, 337, 530
Automatic prompt design, 530
Autoregressive process, 180
Average pooling, 83
Back-propagation through time, 80, 177
Back-translation, 103, 263
Background task, 150
Backpropagation, 74
Backward pass, 74
Batch gradient descent, 91
Batching, 91
Batch, 91
Beam size, 248
Beam width, 248
Bi-directional model, 180
Bias trick, 23
Bilinear transform, 334
Binary classification, 21
Binary variable, 16
Branch, 289
Calculation annotation, 504
Canonicalization, 123
Capacity, 182
Catastrophic forgetting, 406
Category, 19
Causal language modeling, 379
Chain of continuous thought, 694
Chain of thought, 504
Chain rule, 17
Chain-of-thought prompting, 432
Checkpoint ensembling, 262
Chunking, 54
Classification model, 20
Classification system, 19
Classifier, 19
Co-adaptation, 97
Co-attention, 238
Code, 105
Coding, 182
Column vector, 13
Combinatorial optimization problem, 254
Completion, 376
Component model, 42, 48
Component system, 261
Compositional generalization, 514

Compositional model, 209
Conditional computation, 337
Conditional random field, 128, 203
Connectionist temporal classification, 201
Constituent system, 261
Constrained optimization, 258
Constrained search, 266
Context window, 142
Continuous memory, 322
Continuous optimization, 257
Continuous thought, 695
Contraction mapping, 109
Contractive auto-encoder, 109
Contrastive learning, 38
Convolution kernel, 81
Convolution operation, 188
Convolution product, 189
Convolutional layer, 81
Convolution, 189
Correlation coefficient, 142
Corrupted input, 110
Corruption, 110
Cosine, 196
Cost function, 36
Coverage vector, 241
Coverage, 241
Cross-correlation, 190
Cross-entropy, 19
Cross-lingual language model, 400
Cross-validation, 49
Cumulative reward, 574
Curve fitting, 37
Data augmentation, 103, 263, 342
Decay factor, 89
Decision boundary, 24
Decoder-only architecture, 275
Decoder, 105
Decoding block, 274
Decoding layer, 274
Decoding system, 218
Decoding, 202
Deduction, 32
Deep learning, 67
Deep neural network, 34, 67
Delayed reward, 31
Deliberate-then-generate, 519
Delta rule, 87
Demonstration, 377
Denoising, 110
Dependent variable, 42
Depth growth, 304
Depth-first search, 256
Diagonal state-space model, 335
Diagonalization, 335
Dilated context window, 316
Direct preference optimization, 592
Discriminative model, 25, 26
Distributed representation, 76, 137
Distributional hypothesis, 139
Distributional representation, 139
Distributional semantics, 140, 166
Distributional word representation, 139
Distribution, 17
Dot product, 14
Dropout rate, 97
Dropout, 97
Duration, 182
Early exit classifier, 340
Early exiting, 338
Early stopping, 42, 90, 250
Edge probing, 295
Effective degree of freedom, 46
Embedding matrix, 78
Emergent ability, 293, 443
Emission probability, 204
Empirical risk, 36
Encoder-decoder architecture, 215
Encoder-decoder attention, 228
Encoder-decoder, 56
Encoder-only architecture, 275
Encoder, 105
Encoding block, 271
Encoding layer, 271
Encoding system, 217

End-to-end memory network, 183
Ensemble learning, 48
Ensembling, 261
Error gradient, 177
Error-driven learning, 27
Event extraction, 60
Evidence lower bound (ELBO), 114
Expectation, 18
Expected embedding, 258
Expected log-probability, 258
Expected loss, 39
Expected representation, 224
Expected risk, 40
Expected surprisal, 18
Expected value, 18
Exploding and vanishing gradient problem, 177
External memory, 183, 455
Factor analysis, 150
Feature learning, 104
Feature map, 189
Feature subspace, 230
Feature vector, 68
Feature, 22
Feed-forward neural network based language model, 76
Feed-forward neural network, 70
Few-shot CoT prompting, 433
Few-shot learning, 50
Filter, 81
Forget gate, 184
Formal linguistic competence, 697
Forward pass, 72
Fractional count, 133
Frame, 200
Fully connected, 69
Gated linear unit, 439
Gated recurrent unit, 185
Gate, 84
Gaussian error linear unit, 438
Generalization error, 43
Generative model, 25
Generative reward model, 608
Gradient descent, 87
Graphical model, 203
Greedy strategy, 246
Group relative policy optimization, 685
Hard prompt, 533
Head, 231
Held-out validation set, 42
Hidden Markov model, 128, 203
Hidden layer, 78
Hinge loss, 38
Histogram pruning, 248
Homonymy, 139
Human preference alignment, 551
Hypothesis selection, 261
Identity matrix, 12
Image captioning, 216
Image-to-text generation, 216
Implicit reasoning, 694
Importance sampling, 582
Impulse noise, 110
In-context learning, 377, 432, 483
Independent component analysis, 150
Independent variable, 42
Indicator function, 27
Induction, 32
Inductive bias, 32
Inductive inference, 31
Inductive reasoning, 31
Inference-time scaling, 647
Information extraction, 60
Information gain, 132
Input gate, 184
Input inversion, 564
Instruction alignment, 551
Instruction fine-tuning, 421, 553
Interactive machine translation, 253
Internal memory, 455
Inverse document frequency, 144
Irreducible error, 444
Iteration-level scheduling, 637
Kernel fusion, 347

Kernel method, 33, 325
Key-value cache, 448, 617
Keyword extraction, 60
Label mapping, 495
Label smoothing, 101
Labeled sample, 20
Label, 19
Language modeling, 54
Large-margin training, 37
Latent Dirichlet allocation, 148
Latent reasoning, 694
Latent semantic analysis, 145
Latent semantic indexing, 145
Layer dropout, 306
Layer, 68
Learning rate scheduling, 93
Learning rate, 87
Least-to-most prompting, 510
Left-singular vector, 145
Lemmatization, 123
Lemma, 123
Length normalization, 241
Length reward, 240
Lexical semantics, 138
Linear attention model, 327
Linear classifier, 23
Linear discriminant analysis, 149
Linear discriminant function, 23
Linear transformation, 68
Linear-chain CRF, 204
Linearized tree, 209
Linearly separable, 24, 33
Linguistic regularity, 162
Local attention, 237
Local truncation error, 308
Log-linear, 26
Logistic regression, 26
Long-context LLM, 446
Long-term memory, 182
Loss function, 35, 36
Low-rank approximation, 146
Margin, 37
Masked language modeling, 372, 379
Masked language model, 296
Masking noise, 110
Matrix addition, 13
Matrix product, 15
Matrix-matrix product, 15
Matrix, 12
Max pooling, 83
Maximum a posteriori, 264
Maximum likelihood estimation, 25, 36
Mean absolute error, 37
Mean squared error, 37
Mean, 18
Memory cell, 183
Memory-based method, 455
Memory, 181
Metric learning, 58
Mini-batch gradient descent, 91
Minibatch, 284
Minimum Bayes risk, 264
Minimum error-rate training, 40
Minimum-spanning tree, 294
Mixture-of-experts, 311
Model averaging, 48
Model capacity, 45
Model complexity, 45
Model depth, 70
Model error, 43
Model growth, 304
Model width, 70
Mode, 264
Momentum, 87
Monotonicity, 255
Morphological analysis, 122
Moving average, 333
Multi-branch neural network, 233
Multi-head attention, 277
Multi-layer attention, 233
Multi-layer neural network, 69
Multi-query attention, 329, 461
Multi-store model, 182
Multilingual BERT, 400

Multinomial naive Bayes, 26
Multiple linear regression model, 180
Multiplicative attention, 224
Multivariate regression, 37
Named entity recognition, 54
Neural Turing machine, 183
Neural architecture search, 530
Neural language model, 76
Neural machine translation, 219
Neural network, 67
Neural net, 67
Neuron, 67
Next sentence prediction, 384
Non-autoregressive decoding, 267
Non-autoregressive generation, 267
Non-parametric test, 51
Normalization, 85, 123
Norm, 15
Nucleus sampling, 627
Null hypothesis, 51
Objective function, 35
Off-policy distillation, 686
Offline reinforcement learning, 596
On-policy distillation, 686
One-hot representation, 76
One-hot, 136
One-shot CoT prompting, 433
Online sequence-to-sequence system, 252
Open-vocabulary, 28
Optimization, 35
Ordinary differential equation, 307
Orthonormal vector, 145
Out-of-vocabulary, 28
Outcome reward model, 656, 676
Output equation, 333
Over-translation, 241
Overfitting, 32
Overoptimization problem, 591
Padding, 82, 190
Pairwise method, 38
Parallel scaling, 652
Parameter sharing, 81
Parametric method, 34
Parse tree, 56
Part-of-speech tagging, 53
Patch, 354
Path, 289
Perceptron, 67
Performance function, 574
Performance gap recovered, 568
Permuted language modeling, 381
Pointwise mutual information, 132, 142
Polysemy, 162
Pooling layer, 81
Positional encoding, 194
Post-norm, 272
Post-training quantization, 347
Pre-norm, 279
Prefix fine-tuning, 537
Prefix language modeling, 387
Prime word, 164
Priming, 164
Principal component analysis, 104, 147
Principal component coefficient, 147
Principal component loading, 147
Principal component, 148
Probability density function, 17
Probability density, 17
Probability distribution, 17
Probability function, 16
Probability mass function, 17
Probability measure, 16
Probe, 293
Probing classifier, 293
Probing predictor, 293
Problem decomposition, 507
Process reward model, 656, 676
Product rule, 17
Progressive downsampling, 344
Prompt embedding, 542
Prompt engineering, 430, 483
Prompt optimization, 530
Proximal policy optimization, 429, 584
Query-key-value attention, 227

Random process, 173
Random variable, 16, 39
Ratio function, 582
Reasoning trace, 681
Receiver, 225
Receptive field, 81, 188
Reconstruction loss, 106
Rectified linear unit, 438
Recurrent cell, 79, 183
Recurrent unit, 79
Regression, 37
Regular expression, 123
Reinforcement learning from human feedback, 426, 552
Reinforcement learning, 31
Rejection sampling, 602
Relation extraction, 60, 498
Relative entropy, 19
Relative positional encoding, 197
Relative positional representation, 298
Reordering problem, 237
Reparameterization trick, 114
Request-level scheduling, 637
Reset gate, 185
Residual connection, 86, 179
Retrieval-augmented generation, 323, 458
Return, 574
Reversible residual network, 329
Reward gaming, 591
Reward hacking, 591
Right-singular vector, 145
Robust statistics, 102
Routing model, 311
Row vector, 13
Salt-and-pepper noise, 110
Sample efficiency, 50
Sample efficient, 565
Sample, 20
Saturating activation function, 178
Scalar multiplication, 13
Scalar, 12
Scaled dot-product attention, 224
Scaling law, 345, 443
Search error, 43, 249
Search problem, 43
Self-attention, 228
Self-consistency, 522
Self-instruct, 561
Self-paced reading, 164
Self-supervised learning, 31, 61, 373
Self-training, 373
Semantic similarity, 162
Semi-orthogonal, 145
Semi-supervised learning, 30
Semi-unitary, 145
Sender, 225
Sensory memory, 182
Sentence embedding, 209
Sentence length prediction, 296
Sentence-level depth-adaptive model, 340
Separating hyperplane, 24
Seq2seq, 215
Sequence-to-sequence, 55, 215
Sequential scaling, 652
Shallow-to-deep training, 304
Shared encoder, 352
Short-term memory, 182
Shortcut connection, 86
Shrinkage estimator, 100
Significance level, 51
Significance test, 51
Similarity function, 58
Similarity learning, 58
Simplex, 258
Simultaneous translation, 253
Sine, 195
Single-label classification, 21
Single-layer attention, 233
Single-layer neural network, 69
Single-layer perceptron, 67
Single-round prediction, 556
Singular value decomposition, 145
Singular value, 145
Skip connection, 86, 179

Soft prompt, 533
Soft word alignment matrix, 59
Source sequence, 216
Source-side sequence, 216
Span prediction, 60
Sparse attention model, 298
Sparse auto-encoder, 107
Sparse coding, 108
Sparse expert model, 338
Sparsity penalty, 107
Sparsity ratio, 314
Speculative execution, 629
Speech encoder, 352
Speed-accuracy trade-off, 251
Standard deviation, 18
Standardization, 85
State equation, 333
State variable, 333
State-space model, 333
State-value function, 573
Statistical machine translation, 240
Statistical parsing, 56
Steepest descent, 87
Stemming, 123
Stem, 123
Step function, 68
Stochastic gradient descent, 91
Stochastic process, 173
Stride, 190
Structure prediction, 21
Structured pruning, 343
Sub-layer dropout, 306
Subspace method, 260
Subword, 29, 128
Suffix stripping, 123
Superficial alignment hypothesis, 565
Supervised fine-tuning, 552
Supervised learning, 373
Support vector machine, 34
Surrogate objective, 582
Syntactic hierarchy, 291
Syntactic parser, 56
Syntax tree, 56
Syntax-aware Transformer encoder, 286
System combination method, 261
Systematic error, 47
Tag, 19
Target sequence, 216
Target word, 164
Target-side sequence, 216
Teacher forcing, 240
Teacher-student training, 342
Temporal difference, 578
Term frequency-inverse document frequency, 143
Term frequency, 143
Term-document co-occurrence matrix, 142
Term-term co-occurrence matrix, 140
Test error, 43
Text completion, 499
Text embedding, 209
Text encoder, 352
Text generation, 216
Text transformation, 499
Text-to-image generation, 216
The Akaike information criterion, 46
The Bayesian approach, 40
The Bayesian information criterion, 46
The CBOW model, 152
The Expectation-Maximization (EM) algorithm, 133
The bag-of-words (BOW) model, 22
The curse of dimensionality, 32
The curse of multilinguality, 402
The kernel trick, 34
The minimum message length, 46
The principle of compositionality, 138
The skip-gram model, 153
Threshold function, 23
Threshold pruning, 249
Time series, 173
Token pruning, 344
Token-level depth-adaptive model, 340
Token, 121

Training epoch, 90
Training error, 43
Training step, 87
Transcription label, 200
Transcription unit, 200
Transcription, 200
Transfer learning, 341
Transition probability, 204
Translation language modeling, 401
Transpose, 13
Tree linearization, 286
Trust region, 583
Under-translation, 241
Undercomplete auto-encoder, 106
Underfitting, 41
Undirected graphical model, 205
Unfolded, 79
Uni-directional model, 180
Unrolled, 79
Unseen word, 28
Unstructured pruning, 343
Unsupervised bilingual dictionary induction, 30
Unsupervised learning, 30, 373
Update gate, 185
Update rule, 87
Update step, 87
Validation data, 41
Value function, 263
Value-based search, 263
Variable, 16
Variance, 18
Variational auto-encoder, 111
Vector database, 322
Visual question answering, 238, 356
Weak-to-strong generalization, 567
Weight decay, 97
Weight sharing, 81
Word alignment weight matrix, 59
Word alignment, 59
Word clustering, 30
Word distance, 161
Word embedding, 78, 137, 150
Word segmentation, 122
Word semantic distance, 161
Word sense disambiguation, 139
Word sense, 135
Word-document co-occurrence matrix, 142
Word-word co-occurrence matrix, 140
Zero matrix, 12
Zero padding, 284
Zero-shot CoT, 433
Zero-shot learning, 423

fastText, 159

mBERT, 400

t-distributed stochastic neighbor embedding, 165
t-SNE, 165

vLBL, 159